

STAR Robot Control System

Complete Technical Documentation

System Architecture, Protocols, Style Guides & API Reference

Version 1.0
February 2026

STAR Development Team
Locked Inc.

Contents

I	System Architecture & Protocols	2
1	Nanopb (Protobuf) Protocol with HARQ and CRC for SPI Communication	2
1.1	Executive Summary	2
1.1.1	Control Loop Frequency Rationale	2
1.2	Key Design Decisions	3
1.2.1	Protocol Stack	3
1.2.2	Byte Order and Standards Compliance	3
1.2.3	CRC - 32 Implementation	4
1.2.4	Memory Budget	5
1.3	Implementation Requirements	6
1.3.1	Protocol Stack Architecture	6
1.3.2	RX72N(SPI Peripheral) Implementation	6
1.3.3	RPi5(SPI Controller) Implementation	8
1.3.4	HARQ Symmetry and State Management	8
1.3.5	Example Transaction : RPi5 Sends Velocity Command	9
1.4	HARQ Communication Flow	10
1.4.1	Normal Operation(No Retransmission)	10
1.4.2	Error Recovery Scenarios	11
1.5	Error Handling Strategies	12
2	Protocol Buffer Schema Architecture	13
2.1	Architecture Overview	13
2.2	Service Specifications	13
2.2.1	MotorControlService	13
2.2.2	TelemetryService	14
2.2.3	BatteryManagementService	15
2.2.4	ConfigurationService	15
2.3	common.proto- Shared Types	16
2.4	Code Generation and Integration	16
3	Hardware Pin Connections	17
3.1	RX72N Microcontroller Pinout- - 144 - Pin LFQFP(R5F572NNHxFB)	17
3.2	Motor Control Architecture	22
3.2.1	GPTW vs MTU Selection for PWM Generation	22
3.2.2	Pin Allocation Strategy	23
3.2.3	Communication Bandwidth Analysis	24
3.2.4	DS18B20+ Temperature Sensor	25
3.3	IMU	26
3.4	Motor Driver Control Architecture	26
3.5	RX72N Configuration Tables	27
3.6	Raspberry Pi 5 Pinout	28
3.7	Additional Peripheral Connections	28
4	End - to - End Communication Stack	29
4.1	Overview	29

4.2	Transport Priority	29
4.3	Protocol Stack(5 Layers)	30
4.4	Protocol Primer	30
4.4.1	CRC-32 (Cyclic Redundancy Check)	30
4.4.2	ARQ(Automatic Repeat Request)	31
4.4.3	FEC (Forward Error Correction)	32
4.4.4	Viterbi Decoder (Soft-Decision)	32
4.4.5	HARQ(Hybrid Automatic Repeat Request)	33
4.4.6	Chase Combining(Type I HARQ)	33
4.4.7	USB CDC(Communications Device Class)	34
4.4.8	SPI(Serial Peripheral Interface)	35
4.4.9	Protocol Composition Hierarchy	35
4.4.10	Module Map	36
4.5	Command Flow(Downward : UI to Motor)	36
4.6	Telemetry Flow(Upward : Sensor to UI)	37
4.7	Frame Protocol Summary	37
4.7.1	Frame Flags	38
4.8	Reliability: SPI HARQ Send/Receive Paths	38
4.8.1	SPI Send Path (HARQ)	38
4.8.2	SPI Receive Path (Chase Combining + Viterbi)	38
4.8.3	USB CDC Path: No HARQ	39
4.9	Safety Features	39
4.10	Latency Budget	40
4.11	Cross-References	40
5	Transport Failover and Hot Switching	41
5.1	Architecture Overview	41
5.2	Priority System	41
5.3	Dual - Detection Heartbeat	42
5.3.1	Implicit Heartbeat(200 ms Timeout)	42
5.3.2	Explicit Heartbeat (1s PING/PONG)	42
5.4	Health Monitoring	42
5.4.1	Health Thresholds	42
5.4.2	Probing Inactive Transports	43
5.5	Failover State Machine	43
5.5.1	State Transitions	44
5.6	Smart Drain	44
5.7	Shared SessionState(Preventing the Handoff Problem)	45
5.7.1	Gap Tolerance	45
5.8	USB Hot - Plug Detection	46
5.9	Failback Damping(30 - Second Cooldown)	46
5.10	Reset Handshake(Gateway Restart Recovery)	47
5.11	Firmware Side : Passive Listener	47
5.12	Timing Summary	48
5.13	Cross - References	48
6	Dual - Channel Arbitration on the RX72N	49
6.1	The Question	49

6.2	Gateway Guarantee: Single Active Transport	49
6.3	When Overlap Can Occur	49
6.4	Sequential Polling : USB First, Then SPI	49
6.5	Frame Processing Pipeline	50
6.6	Last Writer Wins	50
6.7	Why This Is Correct During Failover	51
6.8	Sequence Validation	51
6.9	Channel Parameter(Currently Unused)	51
6.10	Communication Watchdog Interaction	52
6.11	Summary: Dual - Channel Behavior Matrix	52
6.12	Cross - References	53
7	USB CDC Protocol	53
7.1	Overview	53
7.1.1	Port Assignment	53
7.1.2	Hardware Configuration	53
7.2	USB Descriptor Hierarchy	54
7.2.1	Device Descriptor	54
7.2.2	Configuration Descriptor	54
7.2.3	Interface Association Descriptor(IAD)	54
7.2.4	Endpoint Configuration	55
7.3	USB State Machine	55
7.4	CDC - ACM Class Requests	55
7.5	Bulk Transfer Protocol	56
7.5.1	Data Flow	56
7.5.2	FIFO Access Requirements(RX72N - Specific)	56
7.5.3	FIFO Clear Sequence	57
7.6	Frame Protocol	57
7.6.1	Frame Types	58
7.6.2	Control Frame Details	58
7.6.3	Cross-Compatibility Test Vectors	58
7.7	USB CDC Lightweight Protocol	58
7.7.1	Send Path	59
7.7.2	Receive Path	59
7.7.3	Key Design Decisions	59
7.8	Transport Comparison(USB CDC vs SPI)	60
7.9	Heartbeat Mechanism	60
7.9.1	Implicit Timeout (Primary) – 200ms	60
7.9.2	Explicit PING (Secondary) – 1s Idle-Link Probe	60
7.9.3	Timing Budget	61
7.9.4	Control Frame Dispatching	61
7.10	Debug Logging Over USB CDC Port 2	61
7.10.1	Architecture	61
7.10.2	Boot Log Sequence	61
7.10.3	Implementation	61
7.10.4	Thread Safety	63
7.10.5	Statistics Tracking	63
7.11	Performance Characteristics	64

7.11.1	Throughput	64
7.11.2	Memory Usage	64
7.12	Error Handling and Recovery	64
7.12.1	USB Not Configured	64
7.12.2	TX Buffer Full	65
7.12.3	Cable Disconnect	65
7.13	NASA Power of 10 Compliance	65
7.14	References	65
II	Style Guides	66
8	Protocol Buffer Style Guide	67
8.1	Terminology Standard	67
8.2	Protocol Buffer Naming Conventions(Boston Dynamics Style)	67
8.3	Unit Suffixes(MKS System)	67
8.4	General Documentation Standards	68
9	C Style Guide	69
9.1	General Guidelines for ASM and Inline ASM	69
9.2	When to Use ASM and Inline ASM	70
9.3	Guidelines	70
9.4	Platform-Specific Error Checking Macros	70
9.5	General Rules	71
9.6	Control Structures	71
9.6.1	For Loops	71
9.6.2	While Loops	72
9.7	Functions	72
9.8	Structs and Enums	73
9.9	Initializer Lists	74
9.10	Closing Brace Placement	74
9.11	Summary Table	75
9.12	General Rules	75
9.13	Basic Switch Structure	75
9.14	Case Block Braces	76
9.15	Break Statement Requirements	77
9.16	Default Case Handling	78
9.17	Switch on Enum Types	79
9.18	Indentation	80
9.19	Summary	80
9.20	General Commenting Guidelines	81
9.21	File Header Comments	81
9.22	Source File Header	82
9.23	Doxygen Documentation	82
9.23.1	Function Documentation	82
9.23.2	Parameter Direction Markers	83
9.23.3	Code Examples with @code / @endcode	84
9.23.4	Struct Member Documentation	84

9.24	Special Comment Keywords	85
9.24.1	Keyword Definitions	85
9.25	Section Comments	85
9.26	Aligned Comments	86
9.27	Implementation Comments	86
9.28	Private Function Documentation	87
9.29	Summary	88
9.30	RTOS Types, Mutexes, Semaphores, and Atomic Operations	88
9.31	General Rules	88
9.32	Basic Enum Structure	89
9.33	Explicit Value Assignment	89
9.34	Documentation	89
9.35	Enum to String Functions	90
9.36	Using the Count Sentinel	90
9.37	Switch Statement Pattern	91
9.38	Forward Declaration	91
9.39	Naming Convention Summary	92
9.40	General Principles	92
9.41	Platform Error Codes	92
9.42	Validation Macros	93
9.42.1	RX_RETURN_ON_FALSE	93
9.42.2	ESP_GOTO_ON_ERROR	94
9.42.3	ESP_ERROR_CHECK	95
9.43	NULL Safety Patterns	95
9.44	Mutex Error Handling	96
9.45	Error Interface Dependency Injection	97
9.46	Cleanup Patterns	97
9.47	Error Logging	98
9.48	Summary	98
9.49	Return Value Conventions	99
9.50	Private and Exposed Private Functions	99
9.50.1	Private Functions (static)	99
9.50.2	Exposed Private Functions (priv_)	100
9.51	Parameter Validation	100
9.52	Function Declaration Formatting	101
9.53	Doxygen Documentation	102
9.54	Private Function Documentation	102
9.55	Function Implementation Patterns	102
9.55.1	Single Exit Point with Cleanup	102
9.55.2	Factory Function Pattern	103
9.56	Callback Functions	104
9.57	Summary	105
9.58	General Principles	105
9.59	Static File-Scoped Variables (s_ prefix)	105
9.60	Why static const Over #define	106
9.61	When to Use #define	106
9.62	Explicit Type Usage - Always Use Fixed - Width Integer Types	107
9.62.1	Why Explicit Types Matter	107

9.62.2	Correct Type Usage	107
9.62.3	Special Case: Characters and Strings	108
9.62.4	Loop Counters and Array Indices	108
9.62.5	Function Parameters and Return Types	108
9.62.6	Type Casting for Enum Comparisons	109
9.62.7	Summary: Type Usage Rules	109
9.63	True Global Variables (g_ prefix)	109
9.64	Dependency Injection Alternative	110
9.65	Static Variables for Module State	110
9.66	Thread Safety for Shared State	111
9.67	Summary	112
9.68	General Principles	112
9.69	Include Guard Naming Convention	112
9.70	C++ Compatibility	113
9.71	Header File Structure	113
9.72	Include Order	114
9.73	Forward Declarations	115
9.74	Separating Types from Functions	115
9.75	Doxygen File Documentation	116
9.76	Complete Header Template	117
9.77	Summary	118
9.78	General Rules	118
9.79	Space After Control Keywords	118
9.80	No Space After Function Names	119
9.81	Binary Operators	119
9.82	Unary Operators	120
9.83	Pointer Declarations	120
9.84	Comma and Semicolon Spacing	120
9.85	Parentheses Spacing	121
9.86	Parameter Alignment	121
9.87	Variable and Comment Alignment	122
9.88	Struct Initializer Alignment	122
9.89	Summary	123
9.90	General Rules	123
9.91	Include Order	123
9.92	Complete Example from Codebase	124
9.93	FreeRTOS Include Order	124
9.94	System Includes	125
9.95	Platform HAL Includes	125
9.96	Project Includes	125
9.97	Header File Includes	126
9.98	Conditional Includes	126
9.99	Include Comments	126
9.100	Avoiding Circular Dependencies	127
9.101	Summary	127
9.102	General Rules	128
9.103	Functions and Variables	128
9.104	Module Prefixes	128

9.105	Constants and Macros	129
9.106	Type Names	129
9.107	Enum Members	130
9.108	Static Variables (File-Scoped)	131
9.109	Global Variables	131
9.110	Private Functions	131
9.110.1	File-Scoped Static Functions (internal_ prefix)	131
9.110.2	Exposed Private Functions (priv_ prefix)	132
9.111	Summary Table	133
9.112	Typedef Struct Pattern	133
9.112.1	Standard Pattern	133
9.112.2	Error Handler Example	134
9.113	Union Usage for Protocol Configurations	134
9.113.1	Protocol - Specific Structures	135
9.114	Callback Function Types	135
9.115	Interface Patterns (Dependency Injection)	136
9.115.1	Interface Usage Pattern	137
9.116	Opaque Handle Patterns	138
9.117	Operations Structure Pattern	138
9.118	Manager Structure Pattern	139
9.119	Best Practices	139
9.120	Library Directory Structure	140
9.120.1	Example: star_bus Library	140
9.121	CMakeLists.txt Structure	140
9.122	Header File Organization	141
9.122.1	Header File Template	141
9.122.2	Header Section Order	142
9.123	Include Guard Naming Convention	143
9.124	Source File Organization	143
9.124.1	Source File Template	143
9.124.2	Source Section Order	145
9.125	Types Header Separation	145
9.125.1	Controller Header Pattern	146
9.126	Project Root Structure	146
9.127	Best Practices	147
9.128	Overview	147
9.129	SPI Bus Terminology	147
9.129.1	COPI / CIPO(Not MOSI / MISO)	147
9.130	I2C and General Bus Terminology	148
9.130.1	Controller / Peripheral(Not Master / Slave)	148
9.131	Mapping Platform Legacy APIs	149
9.132	Documentation Guidelines	150
9.133	Code Review Checklist	150
9.134	Common Patterns	151
9.135	Summary	151
9.136	Dependency Injection (DIP)	151
9.136.1	Interface Definition	151
9.136.2	Interface Implementation	152

9.136.3	Dependency Injection Pattern	152
9.136.4	NULL Interface Handling	153
9.137	Bus Abstraction Pattern	153
9.137.1	Unified Configuration	153
9.137.2	Factory Functions	154
9.137.3	Manager Pattern	154
9.138	Operations Structure Pattern	155
9.139	Thread-Safe Callback Pattern	156
9.140	Thread Safety Patterns	156
9.140.1	Mutex Timeout Convention	157
9.140.2	Mutex with Cleanup(goto pattern)	157
9.141	Error Handler Ownership Pattern	158
9.142	Linked List Management	159
9.143	Best Practices Summary	159
9.144	Prefer Alternatives to Macros	160
9.145	When Macros Are Necessary	160
9.146	Macro Safety Rules	160
9.146.1	Wrap Statement - Like Macros in <code>do -while (0)</code>	160
9.146.2	Parenthesize All Macro Arguments	161
9.146.3	Beware of Side Effects and Multiple Evaluation	161
9.147	Multi - line Macro Formatting	161
9.148	Naming Conventions	162
9.149	Include Guards	162
9.150	Conditional Compilation	162
9.151	Overview	163
9.152	Benefits	163
9.153	Standard Unit Suffixes	164
9.154	Usage Examples	164
9.154.1	Distance and Velocity	164
9.154.2	Angular Measurements	164
9.154.3	Temperature	165
9.154.4	Time	165
9.154.5	Electrical Measurements	165
9.154.6	Percentage	166
9.155	Protocol Buffer Integration	166
9.156	Conversion Functions	166
9.157	Summary	167
9.158	No Dynamic Allocation	167
9.158.1	Rationale	167
9.158.2	Alternatives	168
9.158.3	Platform Heap Functions	168
9.159	Avoid Large Stack Allocations	169
9.159.1	Stack Size Constraints	169
9.159.2	Examples	169
9.159.3	Stack Usage Guidelines	170
9.160	Use <code>const</code> for Flash Storage	170
9.160.1	Rationale	170
9.160.2	Examples	170

9.160.3	Pointer Declarations	171
9.161	Validate All Buffers	171
9.161.1	Rationale	171
9.161.2	Validation Patterns	171
9.161.3	DMA Buffer Validation	172
9.161.4	Ring Buffer Validation	173
9.162	Summary	173
9.163	Interrupt Service Routines(ISRs)	174
9.163.1	Minimize ISR Work	174
9.163.2	ISR Constraints	174
9.164	No Blocking Delays	175
9.165	Real - Time and Safety	176
9.165.1	Timing Constraints	176
9.165.2	Watchdog Timers	176
9.165.3	Sensor Timeouts	177
9.165.4	Fail - Safe Design	177
9.165.5	CRC Validation	178
9.166	Hardware Abstraction	179
9.166.1	Use Abstract Bus Interfaces	179
9.166.2	Debounce Mechanical Inputs	180
9.166.3	Enable Brown - Out Detection	180
9.166.4	Flash Wear - Leveling	180
9.166.5	Always Use Pull - Up / Pull - Down	181
9.167	Performance and Real - Time	181
9.167.1	Benchmark, Don't Guess	181
9.167.2	Avoid Blocking Operations	182
9.168	Power Management	182
9.168.1	Use Sleep Modes	182
9.168.2	Use DMA to Reduce CPU Load	183
9.169	Testing and Debugging	183
9.169.1	Use Mock Drivers	183
9.169.2	Meaningful Logging	183
9.170	Reliability and Fault Tolerance	183
9.170.1	Reset Peripherals if Stuck	183
9.171	Summary	184
9.172	nanopb Overview	184
9.173	Why Static Allocation ?	184
9.174	Field Size Configuration	184
9.175	Standard Field Size Constraints	185
9.176	Examples	185
9.176.1	String Fields	185
9.176.2	Repeated Fields	185
9.176.3	Bytes Fields	185
9.177	Sizing Guidelines	186
9.177.1	String Sizes	186
9.177.2	Repeated Field Counts	186
9.177.3	Bytes Field Sizes	186
9.178	Memory Impact	187

9.178.1	Buffer Size Calculation	187
9.178.2	RAM Usage Example	187
9.179	Best Practices	187
9.179.1	Right - Size Your Buffers	187
9.179.2	Use Fixed - Size Types	187
9.179.3	Validate Message Sizes	188
9.180	Integration with Code Generation	188
9.180.1	buf.gen.yaml Configuration	188
9.180.2	Options File Naming	188
9.181	Common Pitfalls	188
9.181.1	Missing Options File	188
9.181.2	Mismatched Field Names	189
9.181.3	Forgetting Array Count	189
9.182	Summary	189
9.183	NASA Power of 10: Rules for Safety-Critical Code	189
9.183.1	Compliance Status Legend	190
9.183.2	Rule 1 : Simplify Control Flow	190
9.183.3	Rule 2 : Fixed Loop Upper - Bounds	190
9.183.4	Rule 3 : No Dynamic Memory After Initialization	191
9.183.5	Rule 4 : Keep Functions Short	191
9.183.6	Rule 5 : Use Assertions Generously	192
9.183.7	Rule 6 : Declare Data at Smallest Scope	192
9.183.8	Rule 7 : Check All Return Values and Parameters	193
9.183.9	Rule 8 : Limit Preprocessor Use	193
9.183.10	Rule 9 : Restrict Pointer Use	194
9.183.11	Rule 10 : Compile with Maximum Warnings	194
9.183.12	Summary: STAR Compliance Matrix	195
9.183.13	Defensive Coding Measures	195
9.183.14	References	196
10	ROS2 C++ Style Guide	196
10.1	Overview	196
10.1.1	When to Use This Guide	196
10.1.2	Key Differences	197
10.2	Naming Conventions	197
10.2.1	Classes and Types	197
10.2.2	Methods and Functions	197
10.2.3	Variables	198
10.2.4	Namespaces	198
10.3	File Naming and Organization	198
10.3.1	Header Files	198
10.3.2	Source Files	199
10.4	Header Organization	199
10.5	Code Formatting	199
10.5.1	Line Length	199
10.5.2	Indentation	200
10.5.3	Braces	200
10.6	ROS2-Specific Patterns	201

10.6.1	Node Inheritance	201
10.6.2	Publishers and Subscribers	201
10.6.3	Timers	202
10.7	Error Handling	202
10.8	Logging	202
10.9	Documentation	203
10.10	Constants	203
10.11	Formatting Enforcement	204
10.11.1	Manual Formatting	204
10.11.2	CI Enforcement	204
10.12	References	205
10.13	Integration with Existing Tools	205
10.14	Summary	205
III	Gateway & ROS2	205
11	Gateway Architecture	206
11.1	Overview	206
11.2	Protocol Stack Implementation	206
11.3	Directory Structure	206
11.4	Frame Package	207
11.4.1	Frame Constants	207
11.4.2	Frame Types	207
11.5	Transport Package	207
11.5.1	Available Transports	207
11.5.2	SPI Configuration	208
11.5.3	Transport Interface	208
11.6	Link Layer (HARQ)	208
11.6.1	SPILink	208
11.6.2	CDCLink	208
11.6.3	Shared Session State	208
11.6.4	HARQ Parameters	209
11.7	Transport Manager	209
11.7.1	Transport Priorities	209
11.7.2	Failover Behavior	209
11.8	gRPC Services	209
11.9	Build and Test	210
11.10	Dependencies	210
12	ROS2 Integration and Sensor Fusion Architecture	211
12.1	Overview	211
12.2	Visual-LiDAR Sensor Fusion with RTAB-Map	211
12.2.1	Fusion Architecture	211
12.2.2	Mapping Strategy	211
12.3	Coordinate Frames (REP-105)	211
12.4	Custom ROS2 Nodes and Interfaces	212
12.4.1	star_spi_bridge	212

12.4.2	<code>star_gateway_bridge</code>	212
12.4.3	<code>star_safety_monitor</code>	212
12.5	Multi-Machine Communication	212
12.6	Hardware Persistence (udev rules)	212
IV	API Reference – RX72N Firmware (C)	213
	RX72N Firmware API Reference	214
V	API Reference – Go Gateway	2895
	Go Gateway API Reference	2896
VI	API Reference – ROS2 (C++)	2963
	ROS2 API Reference	2964

Part I

System Architecture & Protocols

```

== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == == == == == == == == == == == == == == == == ==

```

1 Nanopb (Protobuf) Protocol with HARQ and CRC for SPI Communication

1.1 Executive Summary

Overview : Protocol Buffers(nanopb) encapsulated in frames with CRC - 32 and Hybrid Automatic Repeat Request(HARQ) with Forward Error Correction (FEC) for reliable SPI/USB communication between RPi5 and RX72N.



System Constraints:

- Renesas RX72N : 4MB flash, 1MB SRAM
- 100Hz SPI communication(10ms period)
- 250Hz motor control loop(4ms period)
- Static allocation only(no malloc)

1.1.1 Control Loop Frequency Rationale

Why different loop rates ?

The nested loop architecture follows classic control systems design .The outer loop(SPI at 100Hz) handles command distribution and telemetry collection, while the inner loop(PID at 250Hz) responds quickly to motor dynamics .Running PID at 2.5 times the communication rate ensures smooth control between commands without wasting CPU on excessive computation.

Why 250Hz for motor control?

This is a Nyquist sampling consideration.With DC gearmotors at 210 RPM(3.5 Hz mechanical speed) and a first - order time constant $\tau = 75$ ms(cutoff frequency $f_c = 1/(2\pi\tau) \approx 2.1$ Hz), the 250Hz control rate provides approximately 60 times oversampling of the mechanical dynamics - well above the 2 times Nyquist minimum.

Running at 250Hz balances :

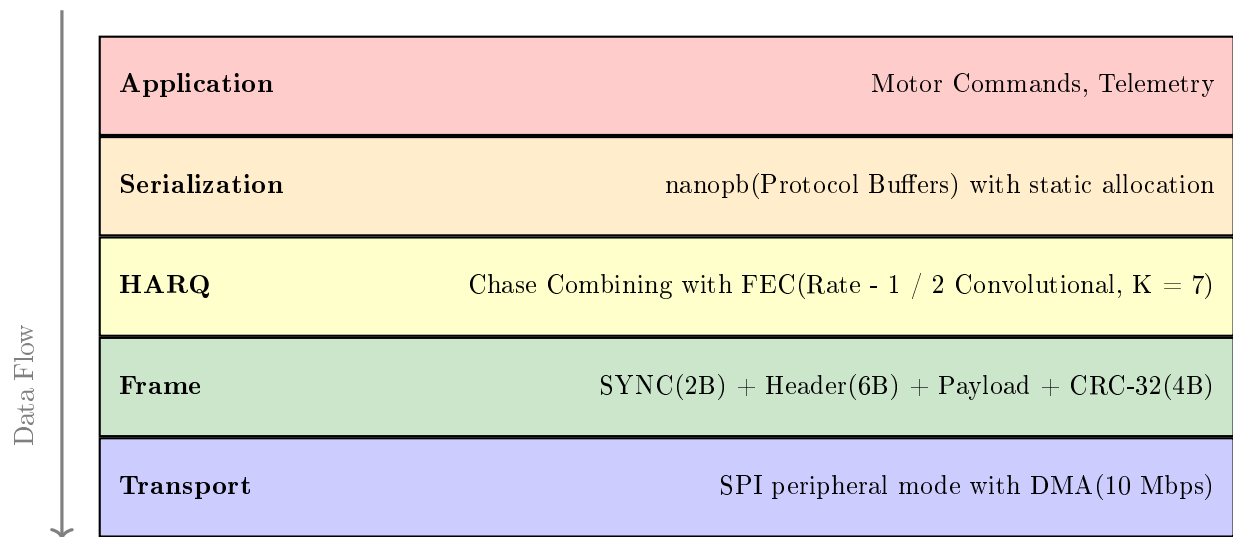
- Adequate sampling of motor dynamics(60 times oversampling)
- Efficient CPU utilization(4 motors $\times$ PID + encoder reads)
- Low latency between velocity commands and motor response
- Power efficiency(fewer ADC conversions, GPIO reads, and computations)

For context, control loop frequencies should match system dynamics :

- DC gearmotors : 50 - 500 Hz typical
- High - speed servos : 1 - 10 kHz
- Motor current control : 10 - 50 kHz
- Temperature control : 0.1 - 10 Hz

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Byte Order and Standards Compliance

The protocol uses little-endian byte ordering for all multi-byte fields, matching the native byte order of both the RX72N (Renesas RXv3 core, 32-bit CISC, little-endian) and Raspberry Pi 5 (ARM little-endian). This eliminates byte-swapping overhead on both platforms.

Table 1: Byte Order Standards by Protocol Field

Field	Byte Order	Standard	Rationale
SYNC	Little - endian	Project convention	Matches native byte order on both platforms
SEQ	Little - endian	Project convention	No byte - swap needed on RX72N or RPi5
LEN	Little - endian	Project convention	No byte - swap needed on RX72N or RPi5
TYPE	N / A(1 byte)	–	Single byte, no endianness
FLAGS	N / A(1 byte)	–	Single byte, no endianness
CRC - 32	Little - endian	IEEE 802.3	Standard CRC - 32 polynomial

SYNC Magic Word (0x55AA): The frame synchronization marker uses the distinctive bit pattern 0x55AA (binary: 0101010101010101). This alternating pattern is easily recognizable and unlikely to appear in random data, making it ideal for frame alignment. The value 0x55AA is transmitted in little-endian format as bytes [0xAA, 0x55] on the wire.

Little-Endian Convention: All multi-byte header fields (SYNC, SEQ, LEN) use little-endian format, matching the native byte order of both communicating platforms (ARM RPi5 and Renesas RX72N). This eliminates the need for byte-swapping, reducing latency and code complexity in the real-time motor control path.

IEEE 802.3 (CRC-32): The frame checksum uses IEEE 802.3 polynomial (0x04C11DB7) in little-endian format, compatible with standard Ethernet CRC libraries. This allows zero-copy CRC validation using hardware accelerators where available.

Key Insight: Uniform little-endian byte order across all fields eliminates byte-swapping overhead on both RX72N and RPi5, which are natively little-endian. CRC-32 follows IEEE 802.3 for hardware acceleration and library compatibility.

1.2.3 CRC - 32 Implementation

The CRC - 32 module supports both hardware and software implementations with compile - time selection.

Table 2: CRC - 32 Hardware / Software Selection

Build Target	Default Implementation	Override
RX72N(<code>__RX__</code> defined)	Hardware CRC Calculator	<code>-DRX_CRC32_USE_SOFTWARE</code>
Host(testing)	Software lookup table	N/A(automatic)

Hardware CRC(RX72N) :

- Peripheral : CRC Calculator at 0x00088280
- Module stop control : MSTPCRB bit 23
- Polynomial : IEEE 802.3(0x04C11DB7), LSB - first(reflected)
- Configuration : CRCCR = 0x43(CRC - 32 + LSB - first)

Software CRC(Host / Fallback) :

- Algorithm : 256 - entry lookup table(1 KB)
- Polynomial : Reflected form 0xEDB88320
- Always available for incremental CRC operations

API (identical for both implementations):

```
1 uint32_t rx_crc32_ieee(const uint8_t *data, size_t len);
2 uint32_t rx_crc32_update(uint32_t crc, const uint8_t *data, size_t len);
```

Go Compatibility : Both implementations produce bit - exact results matching Go 's `crc32.ChecksumIEEE()`, verified by unit tests including the canonical test vector "123456789" = 0xCBF43926.

References:

- RX72N Group User's Manual: Hardware (CRC Calculator section)
- https://renesas.github.io/fsp/group___c_r_c.html

1.2.4 Memory Budget

Table 3: SRAM Usage(512 KB total)

Component	Size(KB)	Usage(%)
DMA double - buffer	8.0	1.56
RX / TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
HARQ state	4.0	0.78
HARQ soft buffers(3x)	48.0	9.38
Viterbi decoder	3.0	0.59
Total	66.5	12.99

Table 4: Flash Usage(16 MB total)

Component	Size(KB)	Usage(%)
Generated protobuf	80	0.49
Nanopb runtime	12	0.07
Frame + HARQ layers	12	0.07
FEC codec(Conv + Viterbi)	8	0.05
Total	112	0.68

Memory Budget Calculations:

SRAM(512 KB total) :

- **DMA double - buffer** : $2 \times 4092\text{bytes} = 8184\text{bytes} = 8.0\text{KB}$
Calculation : Two ping - pong DMA buffers at STAR_SPI_DMA_MAX_SIZE(4092 bytes each)
Usage : $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX / TX buffers** : $1024 + 1024 = 2048\text{bytes} = 2.0\text{KB}$
Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs** : $1536\text{bytes} = 1.5\text{KB}$
Calculation : Largest message structures(SetVelocityResponse ≈ 600 bytes, TelemetryData ≈ 400 bytes)
Usage : $\frac{1.5}{512} \times 100 = 0.29\%$
- **HARQ state** : $4 + 4 + 8 + 4 + 4092 = 4112\text{ bytes} \approx 4.0\text{KB}$
Calculation : Sequence number(4B) + retry counter(4B) + timestamp(8B) + state(4B) + retry buffer(4092B)
Usage : $\frac{4.0}{512} \times 100 = 0.78\%$
- **HARQ soft buffers(3x)** : $3 \times 1024 \times 16 = 49152\text{ bytes} \approx 48.0\text{KB}$
Calculation: 3 combining buffers $\times 1024\text{B}$ max payload $\times (2\text{ output bits per input bit} \times 8\text{ bits} / \text{byte} = 16\text{ encoded bits} / \text{byte})$
Usage : $\frac{48.0}{512} \times 100 = 9.38\%$
- **Viterbi decoder** : $\approx 3.0\text{KB}$
Core state: 64-state path metrics (128B) + traceback survivors (280B) = 408B
Working buffers: New path metrics (128B) + decoded bits ($\sim 1\text{KB}$ for max payload) + temp buffers $\approx 2.5\text{ KB}$
Usage: $\frac{3.0}{512} \times 100 = 0.59\%$
- **Total SRAM** : $8.0 + 2.0 + 1.5 + 4.0 + 48.0 + 3.0 = 66.5\text{KB}$
Total usage : $\frac{66.5}{512} \times 100 = 12.99\%$

Flash(16 MB = 16384 KB total) :

- **Generated protobuf** : 80KB
Calculation: 6 proto files(common, motor_control, telemetry, battery_management, configuration, firmware_update) $\times 13\text{ KB}$ avg compiled size
Usage : $\frac{80}{16384} \times 100 = 0.49\%$

- **Nanopb runtime : 12KB**

Calculation : Compiled size of pb_encode.c, pb_decode.c, pb_common.c

Usage : $\frac{12}{16384} \times 100 = 0.07\%$

- **Frame + HARQ layers : 12KB**

Calculation : Framing logic + CRC - 32 + HARQ state machine + Chase combining logic

Usage : $\frac{12}{16384} \times 100 = 0.07\%$

- **FEC codec(Conv + Viterbi) : 8KB**

Calculation : Convolutional encoder + Viterbi decoder(K = 7, rate 1 / 2)

Usage : $\frac{8}{16384} \times 100 = 0.05\%$

- **Total Flash : 80 + 12 + 12 + 8 = 112KB**

Total usage : $\frac{112}{16384} \times 100 = 0.68\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5(SPI controller) and RX72N(SPI peripheral) .Each layer has specific responsibilities :

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode / decode(nanopb)	Both sides
3. HARQ	Chase Combining, Convolutional FEC, soft decode	Both sides(symmetrical)
2. Framing	Header / footer, CRC - 32 verification	Both sides(identical)
1. SPI + DMA	Physical transport with double - buffering	Both sides(different roles)

Table 5: Protocol stack layers and implementation requirements

Key Insight : Layers 2 - 4 are nearly identical on both platforms.Only the application layer(Layer 5) and SPI role(Layer 1 : controller vs peripheral) differ.

1.3.2 RX72N(SPI Peripheral) Implementation

Layer 1 : SPI Peripheral with DMA

- Configure RSPi in peripheral mode
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2 : Frame Layer(Custom Code)

RX Path:

- Parse frame header(magic bytes, length, sequence number)
- Verify CRC - 32 over header + payload
- If CRC fails → discard packet, send NACK
- If CRC valid → extract payload, pass to ARQ layer

TX Path:

- Build frame : [Header | Payload | CRC32 | Footer]
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3 : HARQ Layer with FEC(Custom Code)

RX Path(with Chase Combining) :

- Extract soft bits from received signal
- If first attempt : Viterbi decode, verify CRC
- If CRC fails : Store soft bits in combining buffer, send NACK
- On retransmission : Combine soft bits with buffer, Viterbi decode
- If decode success : send ACK, pass payload to application

TX Path(with FEC Encoding) :

- Encode payload with rate - 1 / 2 convolutional code(K = 7)
- Assign `tx_sequence_number`, store in retry buffer
- Start timeout timer(10ms), retransmit same frame on NACK
- Chase Combining : receiver accumulates soft bits across retries
- On ACK : clear retry buffer, increment `tx_seq`

Layer 4 : Protobuf(nanopb library)

- Decode incoming bytes → `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs → `SetVelocityResponse`, `TelemetryData`
- Uses `star.v1` package following Boston Dynamics style guide
- Static allocation(no malloc) with `.options` file constraints

Layer 5 : Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop(250 Hz per motor)
- Send telemetry responses(encoder feedback, current, temperature)

1.3.3 RPi5(SPI Controller) Implementation

Layer 1 : SPI Controller with DMA

- Use Linux / dev / spidev interface
- Configure SPI3 : 10 Mbps, mode 0, 8 - bit words
- Use ioctl(SPI_IOC_MESSAGE) for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2 : Frame Layer (*Custom Code - SAME AS RX72N*)

RX Path : Parse frame header, verify CRC - 32, extract payload or discard on CRC failure

TX Path : Build frame [Header | Payload | CRC32 | Footer], calculate CRC - 32, send via SPI

Layer 3 : HARQ Layer with FEC (*Custom Code - SAME AS RX72N*)

RX Path : Extract soft bits, Viterbi decode, Chase Combining on retry, verify CRC, send ACK / NACK

TX Path : FEC encode(rate - 1 / 2, K = 7), assign sequence, store in retry buffer, retransmit identical frame on NACK

Layer 4 : Protobuf(Go with protoc - gen - go)

- Encode : SetVelocityRequest, EmergencyStopRequest
- Decode : SetVelocityResponse, TelemetryData, EncoderData
- Uses star.v1 package following Boston Dynamics style guide

Layer 5 : Application(star - gateway)

- Send motor velocity commands from Nav2 (autonomous) or UI (manual)
- All control flows through Nav2 stack for consistent behavior
- Receive encoder feedback for odometry calculations
- Bridge ROS2 topics to/from RX72N via SPI

1.3.4 HARQ Symmetry and State Management

The HARQ layer uses **Chase Combining** with Stop-and-Wait protocol. Each side sends one FEC-encoded frame at a time, waits for acknowledgment (ACK) with a timeout, and retries up to 3 times. On the receiver side, soft bits from failed transmissions are stored and combined with subsequent retries, improving decode probability with each attempt. This approach provides:

- Simple implementation suitable for embedded systems
- Predictable worst-case latency for real-time control
- 4-5 dB coding gain from FEC reduces retry frequency
- Soft combining improves success probability with each retry
- Sufficient throughput for 10 Mbps SPI with <1ms frame time

The HARQ protocol is **symmetric** – both sides implement identical Chase Combining logic with FEC encode/decode but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores FEC-encoded outgoing packets for retransmission
- **Soft combining buffer**: Stores soft bits from failed RX attempts (3 slots)
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	RX72N(Peripheral)	RPi5(Controller)
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own <code>tx_seq</code>	Own <code>tx_seq</code>
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for RX72N
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's <code>seq</code>	Checks RX72N's <code>seq</code>

Table 6: HARQ role distribution between controller and peripheral

Important : The SPI controller / peripheral roles(Layer 1) are **hardware - level only**. The HARQ layer(Layer 3) treats both sides as equals – each can send, FEC encode / decode, combine soft bits, and retry independently.

1.3.5 Example Transaction : RPi5 Sends Velocity Command

1. **RPi5 Application** : Create `SetVelocityRequest`(left=1.0 m / s, right=1.0 m / s)
2. **RPi5 Protobuf** : Encode to binary(nanopb)
3. **RPi5 HARQ** : FEC encode(rate - 1 / 2, K = 7), assign `seq` = 42, save to retry buffer, start 10ms timeout
4. **RPi5 Frame** : Build packet with header, CRC - 32, footer
5. **RPi5 SPI** : DMA transfer to RX72N(controller initiates)
6. **RX72N SPI** : DMA receives packet in double - buffer
7. **RX72N Frame** : Extract soft bits from received signal
8. **RX72N HARQ** : Viterbi decode, verify CRC - 32 $\rightarrow$ valid, check `seq` = 42 = `rx_last_ack` + 1 $\rightarrow$ accept
9. **RX72N Protobuf** : Decode to `SetVelocityRequest` struct
10. **RX72N Application** : Update motor setpoints, execute PID control
11. **RX72N HARQ** : Build ACK packet with `ack_seq` = 42
12. **RX72N Frame** : Wrap ACK in frame with CRC - 32

13. **RX72N SPI** : DMA transmits ACK(peripheral responds)
14. **RPi5 SPI** : DMA receives ACK packet
15. **RPi5 Frame** : Verify CRC - 32 $\rightarrow$ valid
16. **RPi5 HARQ** : Got ACK(42) $\rightarrow$ clear retry buffer, cancel timeout

Outcome : Command successfully delivered with guaranteed delivery via HARQ.If CRC failed, RX72N would store soft bits and NACK; on retry, it combines soft bits from both attempts before Viterbi decoding, improving decode probability.

1.4 HARQ Communication Flow

1.4.1 Normal Operation(No Retransmission)

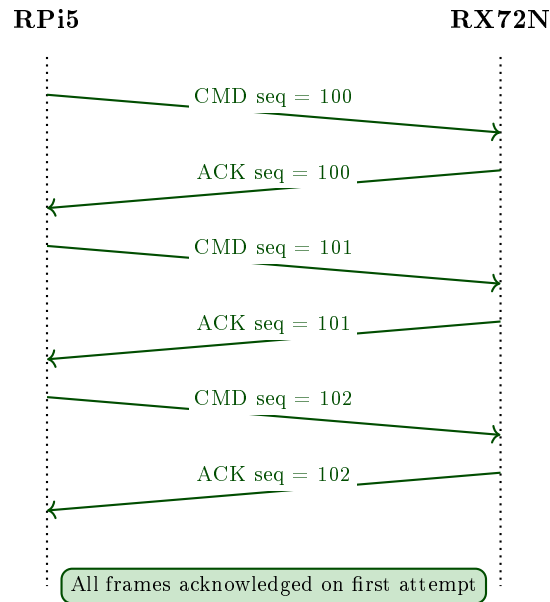


Figure 1: Normal HARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

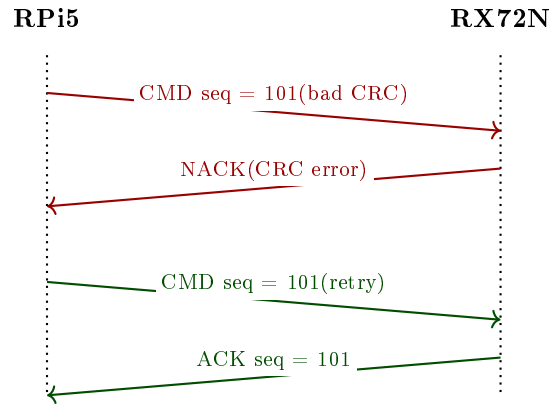


Figure 2: CRC Error - Retry with Same Sequence Number

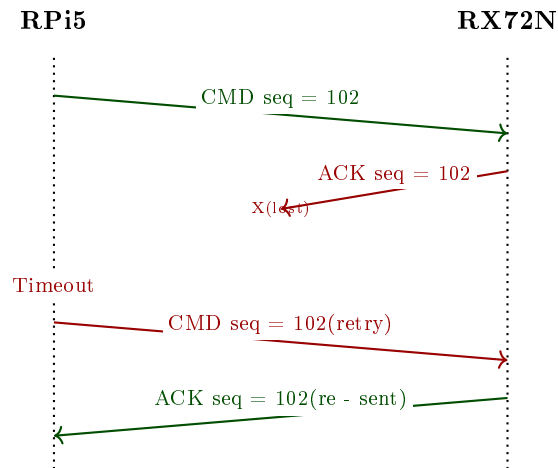


Figure 3: Duplicate Packet Scenario(Lost ACK)

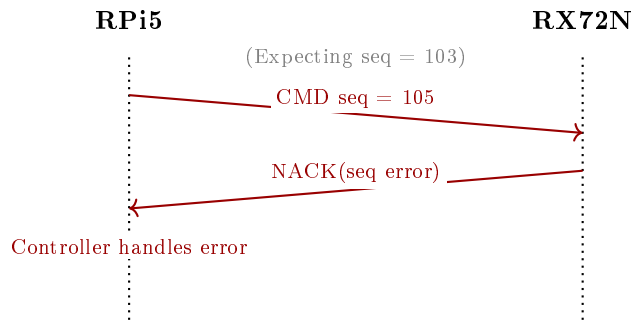


Figure 4: Out - of - Order Packet Scenario

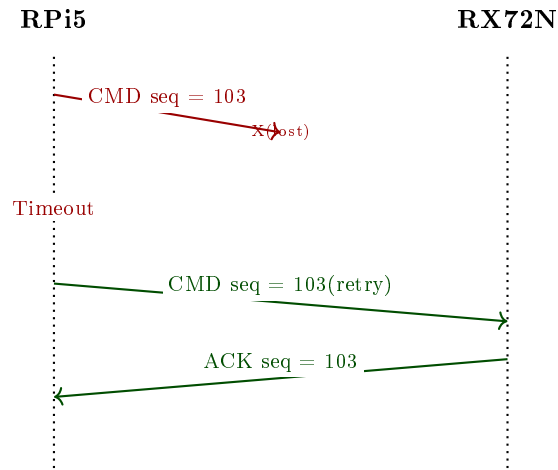


Figure 5: Lost Packet Scenario(Timeout)

1.5 Error Handling Strategies

CRC Mismatch

Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment `crc_errors`

Sequence Error

Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout

RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

Invalid Protobuf

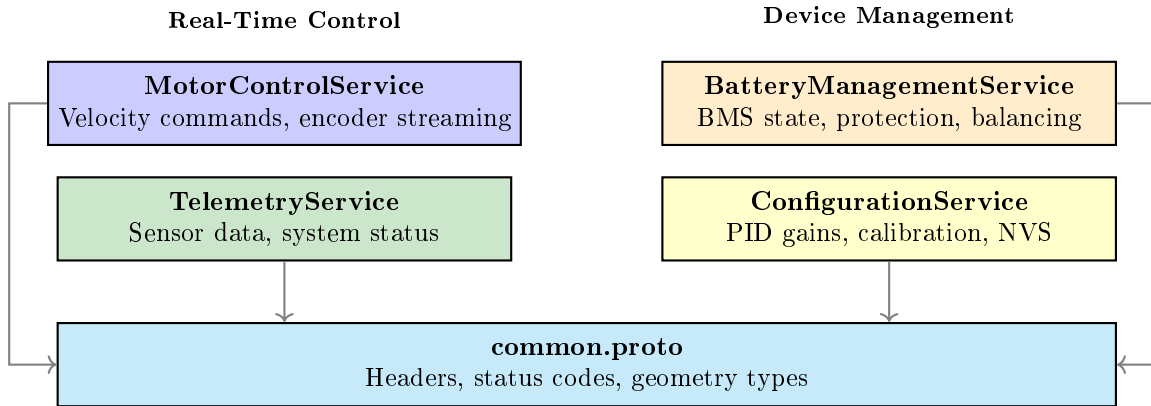
Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment `proto_errors`

2 Protocol Buffer Schema Architecture

This section documents the Protocol Buffer schema architecture for RPi5↔RX72N communication. All schemas follow the **Boston Dynamics Protobuf Style Guide** for consistency, maintainability, and industry best practices.

2.1 Architecture Overview

Overview: Five gRPC services define the complete API surface for robot control, organized by functional domain. Services communicate via the nanopb-serialized frames described in Section 1.



Key Design Principles

- **Versioned Package :** All services use `star.v1` namespace for API evolution
- **Request/Response Headers:** Every RPC includes tracing, timestamps, and status
- **Streaming Support:** Continuous data uses server streaming (up to 100Hz)
- **Static Allocation:** All messages sized for nanopb constraints (no dynamic memory)

Note: Naming conventions, unit suffixes, and coding standards are consolidated in Section 8 (Style Guide and Conventions).

2.2 Service Specifications

2.2.1 MotorControlService

Purpose: Low-latency differential drive control with real-time encoder feedback.

Target Latency: <10ms round-trip for velocity commands.

Table 7: MotorControlService RPCs

RPC	Type	Description
SetVelocity	Unary	Set left / right wheel velocities(m / s)
EmergencyStop	Unary	Immediate stop, priority over all commands
SetMotorPower	Unary	Direct PWM control(bypass PID, testing only)
StreamEncoders	Server stream	Encoder ticks and velocity at 1 - 100Hz
ControlStream	Bidirectional	Velocity in, encoder feedback out

Key Messages:

- **VelocityCommand** –Left / right velocity(m / s), sequence number, timestamp
- **EncoderData** –Motor ID, tick count(341.2 PPR), velocity, timestamp
- **MotorStatus** –Duty cycle, velocity, temperature, current, fault flags
- **PidConfig** –Kp, Ki, Kd gains with output saturation limits

2.2.2 TelemetryService

Purpose : Real - time sensor data aggregation and system health monitoring.

Table 8: TelemetryService RPCs

RPC	Type	Description
GetTelemetry	Unary	Snapshot of all sensor data
StreamTelemetry	Server stream	Continuous sensor updates at 1 - 100Hz
GetSystemStatus	Unary	Firmware version, uptime, connection states

Key Messages:

- **TelemetryData** –IMU, GPS, battery %, WiFi signal, CPU usage, temperature
- **ImuData** –Pitch / roll / yaw(rad), acceleration(m / s²), angular velocity(rad / s)
- **GpsData** –Lat / lon(degrees), altitude(m), accuracy, satellite count, fix type
- **SystemStatus** –Connection states, operating mode, free heap, uptime

Enumerations:

- **RobotMode** –IDLE, MANUAL, AUTONOMOUS, MAPPING, EMERGENCY_STOP
- **ConnectionStatus** –CONNECTED, DISCONNECTED, CONNECTING, ERROR
- **GpsFix** –NONE, 2D, 3D, DGPS, RTK_FLOAT, RTK_FIXED

2.2.3 BatteryManagementService

Purpose: BQ7850 BMS monitoring for lithium-ion battery pack safety and management.

Hardware: TI BQ7850 16-cell BMS with I2C interface.

Table 9: BatteryManagementService RPCs

RPC	Type	Description
GetBatteryState	Unary	Complete battery snapshot(cells, temps, SOC)
StreamBatteryState	Server stream	Continuous battery monitoring
Get / SetProtectionThresholds	Unary	OV / UV / OC / OT protection limits
Enable / DisableCellBalancing	Unary	Per - cell passive balancing control
ControlFets	Unary	Charge / discharge FET enable / disable
GetDeviceInfo	Unary	Manufacturer, serial, chemistry, versions

Key Messages:

- **BatteryState** –Composite of all battery data
- **CellData** –Individual cell voltages(mV), pack voltage, valid cell count
- **BatteryTemperatureData** –Thermistor readings(0.1°C), average
- **StateOfChargeData** –Relative / absolute SOC(%), capacity(mAh), cycle count
- **ProtectionThresholds** –OV / UV voltage limits, OC current limits, OT temperatures

2.2.4 ConfigurationService

Purpose : Runtime parameter management with NVS persistence across power cycles.

Table 10: ConfigurationService RPCs

RPC	Type	Description
GetConfiguration	Unary	Read current system configuration
SetConfiguration	Unary	Apply configuration(optional NVS persist)
ResetToDefaults	Unary	Restore factory defaults
ValidateConfiguration	Unary	Validate without applying
SaveConfiguration	Unary	Persist in - memory config to NVS
Get / SetMotorPidConfig	Unary	Per - motor PID tuning

Key Messages:

- **SystemConfiguration** –Complete config with version and CRC32
- **MotorPidConfig** –Per - motor PID gains(4 motors supported)
- **CurrentSensorCalibration** – Offset and scale for current sensing

- **SafetyThresholds** –Overcurrent, thermal shutdown, cell imbalance limits
- **TimingConfig** –Motor period, telemetry period, communication timeout

2.3 common.proto– Shared Types

All services import `common.proto` for standardized request / response handling.

Table 11: Standard Headers

Message	Fields
RequestHeader	request_id(UUID), client_timestamp, client_version, timeout
ResponseHeader	request_id(echo), server_timestamp, status, error_message, latency_us

Status Codes : UNKNOWN, OK, INVALID_REQUEST, INTERNAL_ERROR, NOT_FOUND, TIMEOUT, INVALID_STATE, ESTOP_ACTIVE

Geometric Types:

- **Vector3** –Generic 3D vector(x, y, z)
- **Quaternion** –Orientation(w, x, y, z)
- **SE2Pose** –2D pose(x_m, y_m, angle_rad)
- **SE2Velocity** –2D velocity(vx_mps, vy_mps, omega_rad_per_s)

2.4 Code Generation and Integration

Table 12: Code Generation Targets

Target	Generator	Output	Usage
Go	protoc-gen-go + grpc	gen/go/	star-gateway(RPi5)
nanopb	nanopb_generator	gen/nanopb/	RX72N firmware

nanopb Constraints: RX72N requires static allocation. Field sizes are configured in `.options` files:

- String fields: `max_size:64` (request_id), `max_size:128` (error_message)
- Repeated fields: `max_count:16` (cell voltages), `max_count:4` (motors)
- Bytes fields: `max_size:1024` (firmware chunks)

Build Integration:

- **Linting:** `buf lint` enforces style guide compliance
- **Breaking Changes:** `buf breaking` detects API incompatibilities
- **CI Pipeline :** Automatic generation and testing on PR

```

=====
=====

```


3 Hardware Pin Connections

3.1 RX72N Microcontroller Pinout– - 144 - Pin LFQFP(R5F572NNHxFB)

Color Legend:

Function Category	Color
PWM Motor Control (GPTW) / Motor Driver GPIO (DRV0FF, nSLEEP)	Blue
Encoder Inputs (MTU / TPU)	Green
Host SPI / IRQ Communication (RSPI2)	Orange
I2C Communication (RIIC0 / RIIC1 / IMU SCL,SDA)	Violet
ADC Current Sensing	Red
Debug / JTAG / Boot / USB	Yellow
Power / Ground	Gray
GTETRGM PWM Ext Trig / Temp / IMU INT,RST	Lime
Ultrasonic Sonar (HC-SR04)	Cyan
Status LEDs	Teal
Unused / Available	White

Pin	Pin Name	Connected To	Note / Comment
1	AVSS0	Ground	Analog ground
2	P05 / IRQ13	Unused	Available GPIO (IRQ13)
3	AVCC1	Boards 3v3	Analog power supply(3.3V, connect to VCC via branch with 0.1μF bypass to AVSS1)
4	P03/IRQ11	Sonar 0 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
5	AVSS1	Ground	Analog ground
6	P02/IRQ10	Sonar 1 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
7	P01/IRQ9	Sonar 2 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
8	P00/IRQ8	Sonar 3 ECHO	HC-SR04 ultrasonic sensor (IRQ for microsecond timing)
9	PF5	Sonar 0 TRIG	HC - SR04 ultrasonic sensor trigger(10μs GPIO pulse)
10	EMLE	Emulator Configuration jumper	See Table ??
11	PJ5	Sonar 1 TRIG	HC - SR04 ultrasonic sensor trigger(10μs GPIO pulse)
12	VSS	Ground	Power supply
13	PJ3	Sonar 2 TRIG	HC - SR04 ultrasonic sensor trigger(10μs GPIO pulse)
14	VCL	220nF cap to ground	Voltage regulator
15	VBATT	Boards 3v3	Battery backup
16	MD / FINED	E1E2 - Emulator - Interface, DIP switch	See Table ??Default : OFF, OFF
17	XCIN	10k resistor to ground	RTC crystal input(not used)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
18	XCOUT	Floating	RTC crystal output(not used)
19	RES#	E1E2 - Emulator - Interface, Reset button	MCU reset
20	P37 / XTAL	24MHz crystal(to pin 22)	System clock crystal
21	VSS	Ground	Power supply
22	P36 / EXTAL	24MHz crystal(to pin 20)	System clock crystal
23	VCC	Boards 3v3	Power supply
24	P35/NMI/UPSEL	UPSEL configuration jumper	Can be used for NMI (not used). See Table ??
25	P34 / TRST#	E1E2 - Emulator - Interface	JTAG(TRST#)
26	P33	Sonar 3 TRIG	HC - SR04 ultrasonic sensor trigger(10µs GPIO pulse)
27	P32	IMU INT	IMU interrupt line (active-low, IRQ input)
28	P31 / TMS	E1E2 - Emulator - Interface	JTAG(TMS)
29	P30 / TDI	E1E2 - Emulator - Interface	JTAG(TDI)
30	P27 / TCK	E1E2 - Emulator - Interface	JTAG(TCK)
31	P26 / TDO	E1E2 - Emulator - Interface	JTAG(TDO)
32	P25 / MTCLKB	Encoder 0 Phase B	MTU1 encoder(quadrature counting)
33	P24 / MTCLKA	Encoder 0 Phase A	MTU1 encoder(quadrature counting)
34	P23/GTI0C0A	Motor 0 IN2	GPTW PWM channel 0A (32-bit)
35	P22/GTI0C1A	Motor 1 IN2	GPTW PWM channel 1A (32-bit)
36	P21/SCL1	IMU SCL	IMU I2C clock (RIIC1)
37	P20/SDA1	IMU SDA	IMU I2C data (RIIC1)
38	P17/GTI0C0B	Motor 0 IN1	GPTW PWM channel 0B (32-bit)
39	P87	Unused	Available GPIO
40	P16 / USB0_VBUS	USB0 VBUS detection	USB CDC debug interface
41	P86/GTI0C2B	Motor 2 IN1	GPTW PWM channel 2B (32-bit)
42	P15/GTETRGA	PWM Ext Trig 0	GPTW external trigger A (GTETRGA)
43	P14/GTETRGD	PWM Ext Trig 3	GPTW external trigger D (GTETRGD)
44	P13/SDA0	Host I2C Data	RIIC0 for RPi5 communication (FM+ capable)
45	P12/SCL0	Host I2C Clock	RIIC0 for RPi5 communication (FM+ capable)
46	VCC_USB	Boards 3v3	USB power supply
47	USB0_DM	USB0 DN(27R resistor)	USB data -
48	USB0_DP	USB0 DP(27R resistor)	USB data +
49	VSS_USB	Ground	USB ground

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
50	P56	Unused	Available GPIO
51	P55	Unused	Available GPIO
52	P54	Unused	Available GPIO
53	P53	Unused	Not available as GPIO when external bus enabled(BCLK)
54	P52	Unused	Available GPIO
55	P51	DS18B20 DQ	1 - Wire temperature sensor data pin
56	P50	Unused	Available GPIO
57	VSS	Ground	Power supply
58	P83	IMU RST	IMU reset (active-low, GPIO output)
59	VCC	Boards 3v3	Power supply
60	PC7 / UB	E1E2 - Emulator - Interface, DIP switch	Operation mode configuration. See Table ??
61	PC6/GTIOC3B	Motor 3 IN1	GPTW PWM channel 3B (32-bit)
62	PC5 / MTCLKD	Encoder 1 Phase B	MTU2 encoder(quadrature counting)
63	P82	Unused	Available GPIO(GTIOC2A, SCI10, Ethernet)
64	P81	Unused	Available GPIO(GTIOC0B, SCI10, Ethernet, QSPI)
65	P80	Unused	Available GPIO(SCK10, Ethernet, QSPI)
66	PC4/GTETRGC	PWM Ext Trig 2	GPTW external trigger C (GTETRGC)
67	PC3/GTIOC1B	Motor 1 IN1	GPTW PWM channel 1B (32-bit)
68	P77	Unused	Available GPIO(SCI11, Ethernet, QSPI, SDHI)
69	P76	Unused	Available GPIO(SCI11, Ethernet, QSPI, SDHI)
70	PC2 / TCLKA	Encoder 2 Phase A	TPU1 encoder(quadrature counting, 16 - bit)
71	P75	Unused	Available GPIO(SCI11, Ethernet, SDHI)
72	P74/CS4#	Unused	Available GPIO
73	PC1	Unused	Available GPIO
74	VCC	Boards 3v3	Power supply
75	PC0 / TCLKC	Encoder 3 Phase A	TPU2 encoder(quadrature counting, 16 - bit)
76	VSS	Ground	Power supply
77	P73	Unused	Available GPIO(Ethernet)
78	PB7 / TXD9	Debug SCI USB - C RX	UART crossed. See Table ??

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
79	PB6 / RXD9	Debug SCI USB - C TX	UART crossed. See Table ??
80	PB5	Unused	Available GPIO
81	PB4	Unused	Available GPIO
82	PB3 / TCLKD	Encoder 3 Phase B	TPU2 encoder (quadrature counting, 16 - bit)
83	PB2	LED5	Status LED (GPIO output)
84	PB1	LED4	Status LED (GPIO output)
85	P72	LED3	Status LED (GPIO output)
86	P71	LED2	Status LED (GPIO output)
87	PB0	LED1	Status LED (GPIO output)
88	PA7	LED0	Status LED (GPIO output)
89	PA6/GTETRGB	PWM Ext Trig 1	GPTW external trigger B (GTETRGB)
90	PA5	Unused	Available GPIO (GTIOC0A, RSPI_A SCLK, Ethernet)
91	VCC	Boards 3v3	Power supply
92	PA4	Unused	Available GPIO (RSPI_A CS, Ethernet, IRQ5)
93	VSS	Ground	Power supply
94	PA3 / TCLKB	Encoder 2 Phase B	TPU1 encoder (quadrature counting, 16 - bit)
95	PA2	Unused	Available GPIO (GTIOC1A, SCI5 RX)
96	PA1 / MTCLKC	Encoder 1 Phase A	MTU2 encoder (quadrature counting)
97	PA0	Unused	Available GPIO (GTIOC0B, Ethernet)
98	P67 / IRQ15	HOST_IRQ	Host interrupt output to RPi5 (active - low, data ready)
99	P66	Unused	Available GPIO (GTIOC2B, CAN2 TX)
100	P65	Unused	Available GPIO (external bus CKE)
101	PE7/GTIOC3A	Motor 3 IN2	GPTW PWM channel 3A (32-bit)
102	PE6/GTIOC3B	Unused	Available GPIO (GTIOC3B)
103	VCC	Boards 3v3	Power supply
104	P70	Unused	Available GPIO (SDCLK)
105	VSS	Ground	Power supply
106	PE5/GTIOC0A	Unused	Available GPIO (GTIOC0A)
107	PE4/GTIOC1A	Unused	Available GPIO (GTIOC1A)
108	PE3/GTIOC2A	Motor 2 IN2	GPTW PWM channel 2A (32-bit)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
109	PE2/GTIOC0B	Motor 3 DRV0FF	DRV8263H motor 3 drive output disable (GPIO output)
110	PE1/GTIOC1B	Motor 3 nSLEEP	DRV8263H motor 3 sleep control (GPIO output, active-low)
111	PE0/GTIOC2B	Motor 2 DRV0FF	DRV8263H motor 2 drive output disable (GPIO output)
112	P64	Motor 2 nSLEEP	DRV8263H motor 2 sleep control (GPIO output, active-low)
113	P63	Motor 1 DRV0FF	DRV8263H motor 1 drive output disable (GPIO output)
114	P62	Motor 1 nSLEEP	DRV8263H motor 1 sleep control (GPIO output, active-low)
115	P61	Motor 0 DRV0FF	DRV8263H motor 0 drive output disable (GPIO output, active-high; held HIGH at boot to keep bridge disabled until firmware ready)
116	VSS	Ground	Power supply
117	P60	Motor 0 nSLEEP	DRV8263H motor 0 sleep control (GPIO output, active-low)
118	VCC	Boards 3v3	Power supply
119	PD7	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ7, AN107)
120	PD6	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ6, AN106)
121	PD5	Unused	Available GPIO(QSPI, SDHI, Ethernet, IRQ5, AN113)
122	PD4 / SSLC0	RPi5 RSPI2 CS	RSPI2 - A chip select
123	PD3 / RSPCKC	RPi5 RSPI2 SCLK	RSPI2 - A clock
124	PD2 / MISOC	RPi5 RSPI2 CIPO	RSPI2 - A Controller In Peripheral Out
125	PD1 / MOSIC	RPi5 RSPI2 COPI	RSPI2 - A Controller Out Peripheral In
126	PD0	Unused	Available GPIO(GTIOC1B, IRQ0, AN108)
127	P93	Unused	Available GPIO(SCI7, Ethernet, AN117)
128	P92/SMISO7	Unused	Available GPIO (SCI7)
129	P91/SCK7	Unused	Available GPIO (SCI7)
130	VSS	Ground	Power supply
131	P90/SMOSI7	Unused	Available GPIO (SCI7)

Continued on next page

Pin	Pin Name	Connected To	Note / Comment
132	VCC	Boards 3v3	Power supply
133	P47 / AN007	Motor 0 Current Sense	ADC Unit 0, 12 - bit, ~0.661A max at 3.3V ADC reference (IPROPI from DRV8263H, 4990 Ω sense, 1000:1 ratio)
134	P46 / AN006	Motor 1 Current Sense	ADC Unit 0, 12 - bit, ~0.661A max at 3.3V ADC reference (IPROPI from DRV8263H, 4990 Ω sense, 1000:1 ratio)
135	P45 / AN005	Motor 2 Current Sense	ADC Unit 0, 12 - bit, ~0.661A max at 3.3V ADC reference (IPROPI from DRV8263H, 4990 Ω sense, 1000:1 ratio)
136	P44 / AN004	Motor 3 Current Sense	ADC Unit 0, 12 - bit, ~0.661A max at 3.3V ADC reference (IPROPI from DRV8263H, 4990 Ω sense, 1000:1 ratio)
137	P43	Unused	Available GPIO (IRQ11, AN003)
138	P42	Unused	Available GPIO (IRQ10, AN002)
139	P41	Unused	Available GPIO (IRQ9, AN001)
140	VREFL0	Ground	ADC reference low
141	P40	Unused	Available GPIO (IRQ8, AN000)
142	VREFH0	3.3V	ADC reference high
143	AVCC0	Boards 3v3	ADC power supply
144	P07	Unused	Floating

3.2 Motor Control Architecture

3.2.1 GPTW vs MTU Selection for PWM Generation

The RX72N provides two timer peripherals suitable for motor control PWM generation: the General PWM Timer (GPTW) and the Multi-Function Timer Unit (MTU). After detailed analysis, **GPTW** was selected for the following technical reasons :

Resolution and Performance:

- **GPTW:** 32 - bit timer resolution enabling precise duty cycle control
- **MTU:** 16 - bit timer resolution(65, 536 maximum count)
- At 20 kHz PWM frequency with 120 MHz peripheral clock :
 - GPTW provides 6, 000 discrete duty cycle steps(120 MHz / 20 kHz)
 - MTU limited to maximum 3, 276 steps before counter overflow

Channel Architecture:

- **GPTW:** 4 independent channels, each with 2 complementary outputs(GTIOC0A / B through GTIOC3A / B)
- **Perfect match** for 4 motors in IN2 / IN1 (sign-magnitude) control mode
- Each motor driver(DRV8263H) requires exactly 2 PWM signals

Peripheral Separation:

- Using GPTW for PWM provides dedicated high-resolution motor control
- Encoders use MTU (front wheels) and TPU (rear wheels) for phase counting
- No resource conflicts between PWM generation and encoder reading

Encoder Peripheral Selection:

- **Front encoders (0, 1):** MTU1/MTU2 - 32-bit counters with phase counting support
- **Rear encoders (2, 3):** TPU1/5, TPU2/4 - 16-bit counters with phase counting support
- **IMPORTANT:** MTU3 and MTU4 do NOT support phase counting mode per RX72N Manual Ch24 §24.1.2
- Only MTU1 and MTU2 have phase counting capability, hence TPU required for rear encoders
- TPU provides adequate performance for 210 RPM max speed (13.7s overflow period with 16-bit counter)
- Software overflow detection at 100 Hz control loop provides 137× safety margin

Pin Flexibility:

- GPTW signals available on multiple pin locations via MPC(Multi - Function Pin Controller)
- Example : GTIOC0A available on pins 34, 90, 106, 123
- Enabled conflict - free allocation by selecting alternate pin groups

3.2.2 Pin Allocation Strategy

Design Philosophy:

1. **Peripheral Grouping :** Place related functions on contiguous port pins where possible
 - GPTW PWM (IN1/IN2) : Motors spread across PORT E, PORT C, and PORT P (pins 34, 35, 38, 41, 61, 67, 101, 108)
 - Motor Driver GPIO (DRV0FF/nSLEEP) : PORT E and PORT P6x (pins 109-115, 117)
 - Host SPI to RPi5 : PD1 - PD4 (RSPI2_A, pins 122 - 125)
 - MTU Encoders : P24 / P25 (MTU1), PA1 / PC5 (MTU2)
 - TPU Encoders : PC2 / PA3 (TPU1), PC0 / PB3 (TPU2)
 - I2C : P12 - P13 (RIIC0 Host RPi5), P20 - P21 (RIIC1 IMU)
2. **Conflict Resolution via Alternate Functions:**

- 144-pin package provides more pins, reducing conflicts vs. 100-pin
- Table 1.9 analysis revealed all peripherals have 2-4 alternate pin locations
- GTETRGA inputs used as PWM external triggers (GTETRGA/B/C/D on pins 42, 89, 66, 43)
- RIIC1 (P20/P21) repurposed for IMU after BMS removal
- Motor driver GPIO (DRV0FF/nSLEEP) placed on PORT E and PORT P6x to keep routing compact

3. PCB Routing Optimization:

- Host SPI signals on PD1-PD4 (pins 122-125) for compact layout
- Motor PWM (IN1/IN2) and driver GPIO (DRV0FF/nSLEEP) on PORT E and PORT P6x
- Sonar sensors grouped on pins 4-13 (top of package)
- ADC current sense on P44-P47 (pins 133-136) for short analog traces

4. Future Expandability:

- 43 GPIO pins available after critical allocations (of 144 total)
- Unused Port D pins (PD5-PD7) available for additional QSPI/SDHI
- Port E pins split between motor PWM (PE3, PE7), motor GPIO (PE0-PE2), and available GPIO (PE4-PE6)
- Ethernet pins (Port B/P7x) available for future network connectivity
- Additional ADC channels available (AN000-AN003, AN100-AN117)

3.2.3 Communication Bandwidth Analysis

SPI Configuration: 10 Mbps is Optimal

Peak bandwidth calculation for RPi5 ↔ RX72N communication:

- **Velocity Commands(100 Hz) :**
 - Message size : 34 bytes(Nanopb encoded MotorVelocityCommand)
 - Bandwidth : $34 \text{ bytes} \times 100 \text{ Hz} \times 8 = 27.2\text{Kbps}$
- **Encoder Feedback(100 Hz) :**
 - Message size : 18 bytes(4 encoders $\times$ 4 bytes + header)
 - Bandwidth : $18 \text{ bytes} \times 100 \text{ Hz} \times 8 = 14.4\text{Kbps}$
- **Telemetry(50 Hz) :**
 - Message size : 64 bytes(battery, motor current, temperature, status)
 - Bandwidth : $64 \text{ bytes} \times 50 \text{ Hz} \times 8 = 25.6\text{Kbps}$
- **ARQ Protocol Overhead(estimated 40%) :**
 - Headers : 8 bytes per frame(sequence, CRC - 32, flags)
 - Retransmissions : Average 5% packet loss $\times$ 3 retry attempts

– Total overhead : $(27.2 + 14.4 + 25.6) \times 1.4 = 94.1\text{Kbps}$

Total Peak Bandwidth :

$$\text{Peak} = 27.2 + 14.4 + 25.6 + 94.1 = \mathbf{161.3 \text{ Kbps}}$$

SPI Utilization :

$$\frac{161.3\text{Kbps}}{10\text{Mbps}} = \mathbf{1.6\% \text{ utilization}}$$

Margin Analysis:

- Available headroom: $10,000/161.3 = 62\times$ current requirements
- No need for Ethernet (would add PHY chip, 9+ pins, PCB complexity)
- No need for QSPI (RPI5 doesn't support quad-lane SPI natively)
- 10 Mbps provides excellent margin for future features (camera data, additional sensors)

3.2.4 DS18B20+ Temperature Sensor

The DS18B20+ is a digital 1-Wire temperature sensor connected to P51 (pin 55).

Specifications:

- **Temperature Range:** -55°C to $+125^{\circ}\text{C}$
- **Accuracy:** $\pm 0.5^{\circ}\text{C}$ from -10°C to $+85^{\circ}\text{C}$
- **Resolution:** 9-bit to 12-bit configurable (0.5°C to 0.0625°C)
- **Interface:** 1-Wire protocol on DQ pin (P51)
- **Power:** Parasite power or separate VDD (3.0V to 5.5V)

Pin Connections:

- **DQ (Data):** P51 (pin 55) - 1-Wire data line
- **VDD:** 3.3V power supply
- **GND:** Ground

Required External Components:

- $4.7\text{k}\Omega$ pull-up resistor on DQ line (P51 to 3.3V)
- $0.1\mu\text{F}$ decoupling capacitor near VDD pin
- Optional: 100nF ceramic capacitor if using parasite power mode

Firmware Implementation:

- Use 1-Wire protocol library (timing-critical, requires precise delays)
- Typical read cycle: 750ms for 12-bit conversion
- Multiple sensors can share same 1-Wire bus (each has unique 64-bit ROM code)

Use Cases:

- Ambient temperature monitoring
- Motor driver thermal management (supplement DRV8263H internal temp)
- System enclosure temperature

3.3 IMU

The IMU is connected to the RX72N via I2C with a dedicated interrupt and reset line.

Pin Connections:

- **SCL (Clock):** P21 (pin 36) — I2C clock (RIIC1)
- **SDA (Data):** P20 (pin 37) — I2C data (RIIC1)
- **INT (Interrupt):** P32 (pin 27) — active-low interrupt output from IMU to RX72N IRQ input
- **RST (Reset):** P83 (pin 58) — active-low reset, driven as GPIO output by RX72N
- **VDD:** 3.3V power supply
- **GND:** Ground

Required External Components:

- 4.7k Ω pull-up resistors on SCL and SDA lines (to 3.3V)
- 100nF decoupling capacitor near VDD pin

Firmware Implementation:

- Configure INT pin as IRQ input (falling-edge triggered)
- Assert RST low for ≥ 10 ms at startup to perform a hardware reset before initialisation
- Use I2C to read sensor data registers on each INT assertion

3.4 Motor Driver Control Architecture

The DRV8263H motor drivers are controlled via three interfaces:

Real-Time Control (PWM):

- **Signals:** IN1 and IN2 distributed across GPTW channels 0-3
- **Purpose:** Motor speed/direction control at 20 kHz PWM frequency
- **Update rate:** 100 Hz velocity commands from RPi5
- **Pin mapping:** IN2 = GTIOC_A, IN1 = GTIOC_B (sign-magnitude PWM)

Driver Enable/Sleep Control (GPIO):

- **DRV0FF signals:** PE2 (Motor 3), PE0 (Motor 2), P63 (Motor 1), P61 (Motor 0)

- **nSLEEP signals:** PE1 (Motor 3), P64 (Motor 2), P62 (Motor 1), P60 (Motor 0)
- **DRV0FF purpose:** Disable H-bridge outputs without entering sleep mode (GPIO output)
- **nSLEEP purpose:** Put driver into low-power sleep mode when pulled low (GPIO output, active-low)
- **Update rate:** On demand for enable/disable/sleep transitions

External PWM Triggers (**GTETR**G):

- **Signals:** GTETRGA (pin 42), GTETRGB (pin 89), GTETRGC (pin 66), GTETRGD (pin 43)
- **Purpose:** GPTW external trigger inputs for synchronised PWM control

3.5 RX72N Configuration Tables

Table 14: Emulator Configuration Jumper Settings

Jumper Position	Configuration
Shorted Pin 1-2 (normal Operation)	E1/E2 Lite normal debugging (use this mode for normal operation). Microcontroller single operation (without emulator)
Shorted Pin 2-3 (debug with E1/E2)	E1/E2 Lite debugging with Hot plug-in
All open	DO NOT SET

Table 15: Operation Configuration Mode DIP Switch Settings

Note: There are 2 USB-C ports. To flash we can use the SCI boot mode if we want to flash with the SCI USB-C. To flash we can use the USB0 boot mode if we want to flash with the USB0 USB-C.

Pin1	Pin2	UPSEL_CFG	OP Mode
OFF	X	X	Single Chip Mode
ON	OFF	X	SCI Boot Mode (SCI1)
ON	ON	1/2	USB (Bus Powered) (USB0)
ON	ON	2/3	USB (Self Powered) (USB0)

Table 16: UPSEL Power Configuration for USB Boot Mode

Jumper Position	Power Configuration for USB Boot Mode
Shorted Pin 1-2 (No battery pack)	Bus Powered
Shorted Pin 2-3 (Battery Pack)	Self Powered

Table 17: Required Pin States Upon Reset Release

To enter a mode capable of flashing firmware, the following pin configurations must be held when the **RES#pin** goes from Low to High:

Desired Flashing Mode	MD Pin Level	UB Pin Level	Interface Used
Boot Mode (SCI)	Low	Low	Serial (SCI)
Boot Mode (USB)	Low	High	USB
Boot Mode (FINE)	Low $\rightarrow$ High*	Low	FINE Interface

*For FINE interface, the MD pin must be Low at reset release and then switched to High within 20 to 100 ms.

Table 18: SCI9 Firmware Configuration(MPC Settings)

Note: Pins are multiplexed. Firmware must explicitly configure the Multi-Function Pin Controller (MPC) as follows:

Step	Configuration
1. UNLOCK MPC	Unlock registers using the Write-Protect Register (PWPR)
2. PIN FUNCTION SELECT	PIN 78 (PB7) → TXD9 : Set PB7PFS.PSEL[5:0] = 001010b (0x0A) PIN 79 (PB6) → RXD9 : Set PB6PFS.PSEL[5:0] = 001010b (0x0A)
3. ENABLE PERIPHERAL	Set Port Mode Register (PMR) bit to '1' for both PB7 and PB6

USB-UART IC Configuration: Use Cypress configuration utility to enable BUSDETECT.

3.6 Raspberry Pi 5 Pinout

Status: Pin assignments are pending hardware verification and will be documented once validated.

3.7 Additional Peripheral Connections

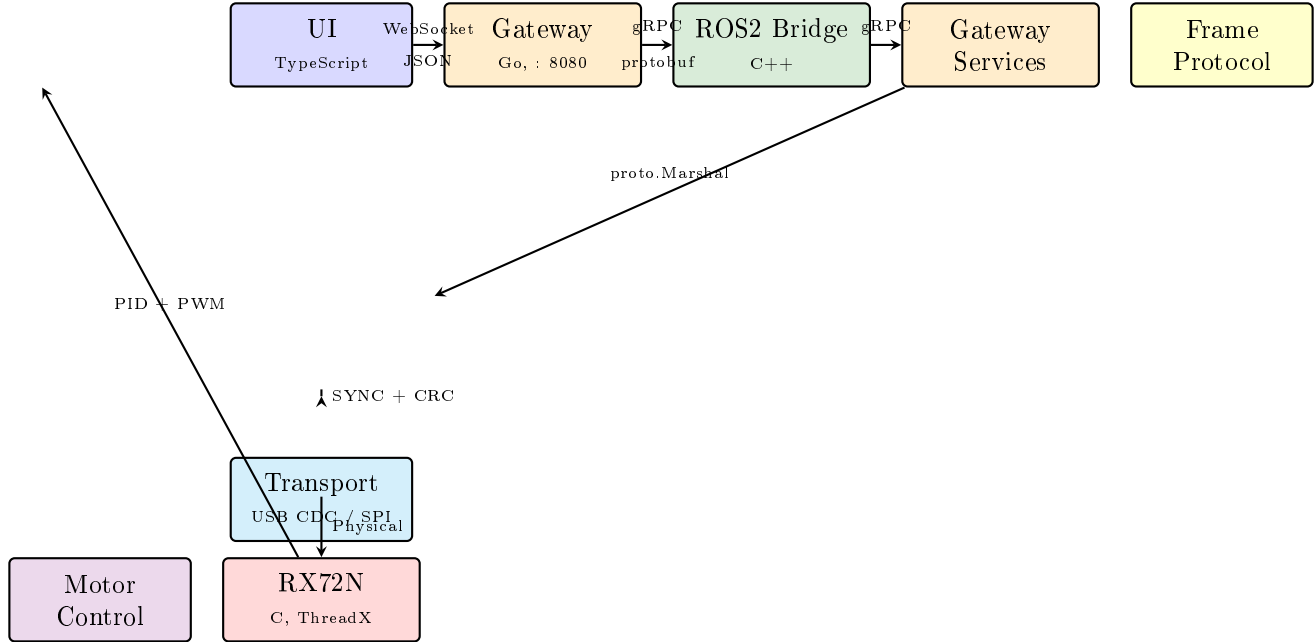
Status: Peripheral connections are pending hardware verification and will be documented once validated.

=====

4 End - to - End Communication Stack

4.1 Overview

The STAR communication stack connects the user interface through a layered architecture down to the RX72N motor controller. Each layer serves a distinct purpose : the UI provides human interaction, the gateway bridges high - level commands to the robot control framework, ROS2 handles autonomous navigation and sensor fusion, and the frame protocol delivers serialized messages over either USB CDC or SPI to the real - time firmware.



Key Insight : All control paths (autonomous Nav2 and manual UI) converge at the gateway, ensuring a single serialization and transport layer regardless of command source. The RX72N firmware is transport - agnostic - it processes identical frames whether they arrive over USB CDC or SPI.

4.2 Transport Priority

USB CDC is the **primary** transport and SPI is the **backup**. Priority values are defined in `star-gateway/internal/manager/types.go`:

```

1 PriorityUSB = 10 // higher = preferred
2 PrioritySPI = 5

```

Listing 1: Transport Priority Constants

The `TransportManager` selects the highest-priority healthy transport at runtime. USB is preferred for three reasons:

- **Lower latency:** 0.5 – 1 ms(USB) vs 1 – 2 ms(SPI)
- **Hardware CRC and bulk retries :** Built into the USB protocol , making application - level HARQ redundant on this path
- **Simplified software :** No ACK / NACK handshake or FEC codec overhead required

SPI enables full HARQ(per ADR - 001) because it lacks USB's built-in reliability mechanisms. A shared `SessionState` (TX/RX sequence counters) spans both transports, preventing sequence desynchronization during failover. Both USB and SPI increment the same sequence counter.

4.3 Protocol Stack(5 Layers)

Table 19: Communication Protocol Stack

Layer	Name	Go Package	C Library	Des
5	Application	<code>internal / service /</code>	<code>src / tasks /</code>	gRF
4	Serialization	<code>google.golang.org / protobuf</code>	<code>libs / rx_nanopb /</code>	man
3	HARQ + FEC	<code>internal / link /</code>	<code>libs / rx_harq /, libs / rx_spi_link /</code>	Pro
				Go,
				HAR
				(RS
				/ C
				/ F
				retr
2	Framing	<code>internal / frame /</code>	<code>libs / rx_frame /</code>	SYN
				load
1	Transport	<code>internal / transport /</code>	RSPi0 / USB peripheral	SPI
				Full

```

== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == == == == == == == == == == == == == == == == ==

```

4.4 Protocol Primer

This subsection explains each protocol building block from first principles. Understanding these concepts is essential for working on any part of the communication stack.

4.4.1 CRC-32 (Cyclic Redundancy Check)

A CRC-32 is a 4-byte “fingerprint” computed over a block of data. The sender computes the CRC over the frame contents and appends it; the receiver recomputes the CRC over the same data and compares.If even a single bit flipped during transmission, the CRCs will not match and the receiver knows the frame is corrupt.

How it works:

1. Treat the data bytes as a long binary polynomial .
2. Divide this polynomial by a fixed *generator polynomial*(IEEE 802.3 : 0x04C11DB7) .
3. The 32 - bit remainder is the CRC.Append it to the frame .
4. The receiver performs the same division.If the remainder is zero, the frame is intact.

Properties:

- **Detection only, not correction :** CRC tells you *something* is wrong, but not *what* or *where*. If a CRC fails, the only option is to discard the frame and request retransmission .
- **Detects all single - bit, double - bit, and odd - bit errors.** Detects all burst errors up to 32 bits.
- **Hardware - accelerated on the RX72N :** The CRC Calculator peripheral computes CRC - 32 in hardware, avoiding CPU overhead.
- **Used on every frame :** Both USB CDC and SPI frames include CRC-32, regardless of whether HARQ is enabled.

Analogy : CRC is like a check digit on a credit card number. It catches typos instantly but cannot fix them— you have to re - enter the number.

4.4.2 ARQ(Automatic Repeat Request)

ARQ is the simplest reliability mechanism : if a frame arrives corrupted(CRC fails) or does not arrive at all(timeout), the receiver asks the sender to retransmit. This is **error detection + re-transmission**, not error correction.

Protocol flow:

1. Sender transmits frame with **REQUIRES_ACK** flag set .
2. Sender starts a timer(10 ms timeout in STAR) .
3. Receiver validates CRC :
 - CRC valid → send **ACK**(positive acknowledgment) .
 - CRC invalid → send **NACK**(negative acknowledgment) .
4. Sender receives response :
 - ACK → success, move to next frame .
 - NACK → retransmit the same frame(set **RETRANSMIT** flag) .
 - Timeout → retransmit(assume frame was lost) .
5. After 3 failed retries, declare the link failed.

Table 20: ARQ Protocol Parameters(STAR Configuration)

Parameter	Value	Description
Max retries	3	Attempts before failure
ACK timeout	10 ms	Wait time per attempt
Worst - case delay	40 ms	Initial send + 3 retransmissions
Sequence range	0 -65535	16 - bit wraparound counter

Limitation: Pure ARQ wastes the corrupted frame entirely. If 99% of the bits arrived correctly and 1% were wrong, ARQ discards all of it and retransmits from scratch. This is where FEC comes in.

4.4.3 FEC (Forward Error Correction)

FEC adds *redundant* data to the transmission so the receiver can **correct** errors without retransmission. The idea is simple: send more bits than strictly necessary, so that even if some are corrupted, the original data can be recovered mathematically.

Everyday analogy: If you spell out a phone number as “five-five-five, one-two-three-four”, someone hearing “five-five-STATIC-one-two-three-four” can guess the missing digit because they know phone numbers are 7 digits and the pattern is obvious. FEC formalizes this intuition.

STAR uses convolutional coding, a specific FEC technique:

- **Constraint length $K = 7$:** The encoder has a 6-bit shift register (memory), so each output bit depends on the current input bit and the 6 previous input bits. This creates complex relationships between bits that the decoder can exploit.
- **Rate 1/2:** For every 1 input bit, the encoder produces 2 output bits. This means the encoded data is exactly $2\times$ the size of the original, doubling bandwidth usage in exchange for error correction capability.
- **Generator polynomials:** $G_1 = 171_8$ (0x79) and $G_2 = 133_8$ (0x5B). These are NASA standard polynomials used in deep-space communication, chosen for their optimal distance properties.

Encoding process:

1. Each input bit shifts into a 6 - stage register.
2. Two output bits are produced by XOR - ing specific register taps(determined by G_1 and G_2).
3. A 100 - byte payload becomes a 200 - byte encoded payload plus tail bits.

4.4.4 Viterbi Decoder (Soft-Decision)

The Viterbi algorithm is the standard method for decoding convolutional codes. It finds the *most likely* original data sequence given a noisy received signal.

Hard bits vs. soft bits:

- **Hard bits :** Binary 0 or 1. No confidence information. “The bit is 1.”
- **Soft bits :** A signed value from -127 to $+127$ (stored as `int8_t`) .The sign indicates the likely bit value(negative = likely 0, positive = likely 1), and the *magnitude* indicates confidence.For example :

- $+127$ = “very confident this is a 1”
- $+10$ = “probably a 1, but not sure”
- -127 = “very confident this is a 0”
- 0 = “complete erasure, no idea”

Why soft bits matter : Soft - decision decoding provides approximately 2 dB gain over hard - decision decoding.This means the decoder can correct more errors because it uses *confidence* information, not just guesses.

Viterbi algorithm overview:

1. The encoder can be modeled as a state machine with $2^{K-1} = 64$ states (for $K = 7$).
2. At each time step, the decoder computes the “distance” (error metric) between the received soft bits and what each possible state transition would have produced.

3. It keeps only the best path (lowest cumulative error) into each state — this is the “survivor path.”
4. After processing all received bits, the decoder traces back through the survivor paths to find the most likely original bit sequence.

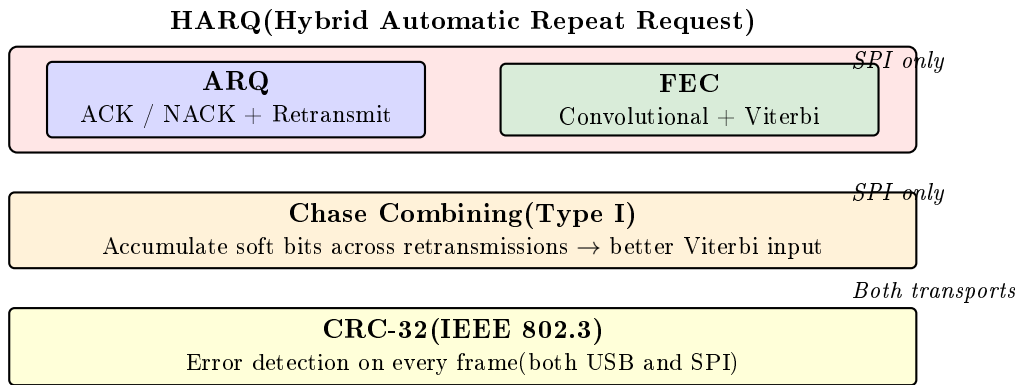
Table 21: Viterbi Decoder Parameters

Parameter	Value	Description
States	64	2^{K-1} for $K = 7$
Traceback depth	35	$\sim 5 \times K$
Decision type	Soft (int8)	-127 to +127
Path metric type	uint32	Cumulative error metric
Memory (RX72N)	~ 40 KB	State arrays + traceback

4.4.5 HARQ(Hybrid Automatic Repeat Request)

HARQ is **not a separate protocol** — it is the **combination** of ARQ and FEC working together. The key insight is that each mechanism compensates for the other’s weakness:

- **ARQ alone:** Discards corrupted frames entirely, wastes bandwidth retransmitting data that was mostly correct.
- **FEC alone:** Can correct limited errors but cannot request retransmission if corruption is too severe.
- **HARQ (both):** FEC corrects minor errors in-place. If corruption exceeds FEC capability, ARQ requests retransmission. On retransmission, Chase Combining accumulates evidence from both attempts for even better decoding.



The formula is : **HARQ = ARQ + FEC + Chase Combining**.CRC - 32 is the foundation used by all paths .

4.4.6 Chase Combining(Type I HARQ)

Chase Combining is the mechanism that makes retransmissions *more valuable* than just sending the same data again.Instead of discarding the failed frame and using only the retransmitted copy, Chase Combining **accumulates soft - bit observations** from every transmission attempt.

How it works:

1. **First transmission:** Receiver gets soft bits (e.g., $[+80, -30, +5, -120, \dots]$). CRC fails.
2. **Receiver stores soft bits** in an `int16` accumulator array (not discarded!).
3. **Retransmission:** Receiver gets new soft bits for the same data (e.g., $[+60, -90, +40, -100, \dots]$).
4. **Combine:** Add element-wise: $[+80+60, -30+(-90), +5+40, -120+(-100), \dots] = [+127, -120, +45, -127, \dots]$
5. **Clamp** combined values to $[-127, +127]$ (int8 range) and feed to Viterbi decoder.
6. Bits where both transmissions agreed become *more confident* (higher magnitude). Bits where they disagreed partially cancel, correctly reflecting uncertainty.

Gain : Each additional transmission provides approximately 3 dB of combining gain. Two transmissions combined can correct errors that neither could individually. After 3 transmissions with combining, the system achieves roughly 4 -5 dB total coding gain over a single attempt.

Table 22: Chase Combining Example(4 soft bits, 2 transmissions)

	Bit 0	Bit 1	Bit 2	Bit 3
TX 1(soft bits)	+80	-30	+5	-120
TX 2(soft bits)	+60	-90	+40	-100
Combined(sum)	+140	-120	+45	-220
Clamped to int8	+127	-120	+45	-127
Interpretation	Strong 1	Likely 0	Weak 1	Strong 0

Key Insight : Chase Combining is why HARQ is strictly better than pure ARQ. A pure ARQ system discards the failed frame and starts fresh. HARQ with Chase Combining treats every transmission as additional evidence, making each retry exponentially more likely to succeed.

4.4.7 USB CDC(Communications Device Class)

USB CDC is a standard USB device class that creates a **virtual serial port** over USB. When the RX72N is plugged into the Raspberry Pi 5 via USB, the Linux kernel automatically creates a `/dev / ttyACM0` device that applications can read / write like a regular serial port.

Why USB CDC needs no HARQ:

- **Hardware CRC-16:** Every USB packet (max 64 bytes for Full-Speed) includes a 16-bit CRC computed and verified by the USB hardware controller. Corrupted packets are automatically discarded at the hardware level.
- **Automatic bulk retries:** USB bulk transfers automatically retry failed packets up to 3 times at the hardware level, transparent to software. The application never sees the retransmission.
- **Differential signaling:** USB uses a twisted differential pair (D+/D-), which provides inherent common-mode noise rejection. This makes bit errors extremely rare compared to single-ended SPI.
- **Handshake protocol:** Every bulk transfer includes a three-phase handshake (TOKEN → DATA → HANDSHAKE), ensuring reliable delivery.

What STAR's CDCLink does provide:

- Application-level CRC-32 (end-to-end integrity, covering the full frame including header).
- Sequence number tracking via shared **SessionState**.
- Gap tolerance (accepts sequence gaps < 10 frames for rare USB packet loss).
- Send serialization via **sendMutex** (prevents out-of-order transmission).
- Send timeout protection (50 ms timeout prevents blocking on USB disconnect).

4.4.8 SPI(Serial Peripheral Interface)

SPI is a synchronous 4 - wire bus connecting the Raspberry Pi 5(Controller)to the RX72N(Peripheral). Unlike USB, SPI is a raw electrical interface with **zero built - in error detection or correction**.

SPI wires:

- **SCLK** : Clock signal(10 MHz), driven by the RPi5 Controller.
- **COPI** : Controller Out, Peripheral In(data from RPi5 to RX72N).
- **CIPO** : Controller In, Peripheral Out(data from RX72N to RPi5) .
- **CS** : Chip Select(active low, selects the RX72N) .

Why SPI needs full HARQ:

- **No hardware CRC** : SPI shifts raw bits.A single bit flip from EMI or crosstalk passes through silently unless the application checks.
- **No retransmission** : SPI is fire - and-forget.If a byte is corrupted, there is no hardware mechanism to request it again.
- **Single - ended signaling** : SPI lines are not differential, making them more susceptible to electromagnetic interference(EMI) from nearby motor drivers, switched power supplies, and PWM signals.
- **Motor environment**: The STAR robot runs 4 brushed DC motors with $4 \times$ DRV8263H-Q1 H-bridge drivers. PWM switching at 20 kHz generates significant EMI that can couple into SPI traces.

STAR's SPI protection stack:

1. **CRC - 32** : Detects corruption(every frame) .
2. **FEC encoding** : Adds redundancy (rate 1/2 convolutional code).
3. **ACK / NACK** : Requests retransmission if FEC cannot correct.
4. **Chase Combining** : Accumulates soft bits across retransmissions.
5. **Viterbi decoding** : Finds most likely original data from combined soft bits.

4.4.9 Protocol Composition Hierarchy

The following table summarizes which protocol mechanisms are active on each transport :

Table 23: Protocol Mechanisms by Transport

Mechanism	USB CDC	SPI	Rationale
CRC - 32(application)	✓	✓	End - to - end integrity on both paths
Sequence tracking	✓	✓	Duplicate / ordering detection
ARQ(ACK / NACK)		✓	SPI has no built - in retry
FEC(convolutional)		✓	SPI has no built - in error correction
Viterbi decoding		✓	Decodes FEC - encoded data
Chase Combining		✓	Improves retransmission success
Hardware CRC - 16	✓		USB hardware provides this
Hardware bulk retry	✓		USB hardware provides this
sendMutex	✓	✓	Atomic sequence + send
Send timeout	✓		Prevents blocking on USB disconnect

4.4.10 Module Map

The following table maps each protocol concept to its implementation in both the Go gateway and the C firmware :

Table 24: Protocol Implementation Module Map

Concept	Go(Gateway)	C(RX72N)
CRC - 32	<code>internal / frame /</code>	<code>libs / rx_crc /(HW - accelerated)</code>
Frame encode / decode	<code>internal / frame /</code>	<code>libs / rx_frame /</code>
FEC encoder	<code>internal / fec / convolutional.go</code>	<code>libs / rx_fec /</code>
Viterbi decoder	<code>internal / fec / viterbi.go</code>	<code>libs / rx_fec /</code>
Chase Combiner	<code>internal / fec / combiner.go</code>	<code>libs / rx_harq /</code>
HARQ state machine	<code>internal / link / spi.go</code>	<code>libs / rx_spi_link /</code>
CDC link(lightweight)	<code>internal / link / cdc.go</code>	<code>libs / rx_usb_comm /</code>
SPI raw transport	<code>internal / transport / spi.go</code>	<code>libs / rx_spi_comm /</code>
USB raw transport	<code>internal / transport / cdc.go</code>	<code>libs / rx_usb_comm /</code>
Session state	<code>internal / manager / session.go</code>	<code>libs / rx_session /</code>
Transport manager	<code>internal / manager / manager.go</code>	<code>libs / rx_comm_manager /</code>

4.5 Command Flow(Downward : UI to Motor)

The following steps trace a velocity command from the user interface down to the motor driver :

1. UI sends JSON over WebSocket to Gateway(: 8080) .
2. Gateway `GatewayService` caches the `VelocityCommand( mutex - protected)` .
3. ROS2 bridge polls `GetTeleopCommand()` at 50 Hz via gRPC .
4. Gateway `MotorControlService.SetVelocity()` serializes with `proto.Marshal()` .
5. `SPILink.Send()`(or `CDCLink.Send()`) wraps in frame : SYNC + header + CRC - 32.
6. Transmit over active transport(USB CDC or SPI).
7. RX72N : `rx_comm_manager` validates CRC, dispatches to callback.

8. `comm_task`(ThreadX priority 5, 100 Hz)cascade - decodes : tries `rx_nanopb_decode_velocity_request()`, then `rx_nanopb_decode_estop_request()`.
9. On success : builds `motor_command_t`, writes to `shared_data`(mutex + event flag).
10. `motor_control_task`(priority 8, 250 Hz)wakes on event, runs PID, writes PWM to DRV8263H-Q1.

Key Insight : The inner motor control loop(250 Hz) runs at $2.5 \times$ the communication rate(100 Hz), ensuring smooth PID control between incoming commands.ThreadX event flags provide zero - copy, zero - latency signaling between the communication and motor tasks.

4.6 Telemetry Flow(Upward : Sensor to UI)

1. RX72N telemetry task(10 Hz) gathers encoder, IMU, and battery data.
2. `rx_nanopb_encode_telemetry()` serializes the data; frame + CRC-32 applied.
3. Transmit over active transport (best-effort, no `REQUIRES_ACK` for telemetry).
4. Gateway dispatcher (100 Hz loop) decodes frame, validates CRC, routes to `TelemetryService`.
5. `GatewayService` caches data; ROS2 bridge publishes to `/telemetry/*` topics.
6. WebSocket streams to UI clients.

4.7 Frame Protocol Summary

The wire format is shared across both transports. Full specification is in Section ??.

Table 25: Frame Wire Format

SYNC	SEQ	LEN	TYPE	FLAGS	PAYLOAD	CRC-32
2 B	2 B	2 B	1 B	1 B	0–1024 B	4 B

Table 26: Frame Type Definitions

Value	Name	Description
0x00	PING	Heartbeat request
0x01	PONG	Heartbeat response
0x10	COMMAND	Command frame carrying protobuf messages
0x11	RESPONSE	Response frame carrying protobuf messages
0x12	ACK	Acknowledgment (SPI HARQ protocol)
0x13	NACK	Negative acknowledgment (SPI HARQ protocol)
0xFE	RESET_ACK	Acknowledgment of session reset
0xFF	RESET	Request to reset session (synchronize sequences)

4.7.1 Frame Flags

Frame flags are bitfield values in the FLAGS byte of the header. Multiple flags can be combined:

Table 27: Frame Flag Definitions

Bit	Name	Description
0	REQUIRES_ACK	Sender expects ACK/NACK response (SPI HARQ only)
1	FEC_ENABLED	Payload is FEC-encoded (convolutional rate 1/2)
2	RETRANSMIT	This frame is a retransmission (not first attempt)
3	SOFT_NACK	NACK was triggered by soft-decision failure, not hard CRC

See Section ?? for the full frame specification including byte ordering, CRC polynomial, and cross-compatibility test vectors.

4.8 Reliability: SPI HARQ Send/Receive Paths

4.8.1 SPI Send Path (HARQ)

When the gateway sends a command over SPI, the following pipeline executes:

1. `SPILink.Send()` acquires the send mutex (atomic sequence assignment).
2. Payload is FEC-encoded: convolutional encoder produces $2\times$ the input size.
3. Frame is built: SYNC + header (with `REQUIRES_ACK` and `FEC_ENABLED` flags) + encoded payload + CRC-32.
4. Frame is transmitted over SPI via `transport.Send()`.
5. `SPILink` starts a 10 ms ACK timer and waits.
6. If ACK received: success, return to caller.
7. If NACK received or timeout:
 - (a) Increment retry counter.
 - (b) Set `RETRANSMIT` flag in header.
 - (c) Retransmit the same encoded frame.
 - (d) Wait for ACK again (up to 3 total attempts).
8. After 3 failures: return error, trigger health monitor.

4.8.2 SPI Receive Path (Chase Combining + Viterbi)

When the RX72N receives a frame on SPI:

1. `rx_spi_link_receive()` reads raw bytes from `rx_spi_comm`.

2. If `FEC_ENABLED` flag is set:

- (a) Convert raw bytes to soft bits: each byte produces 8 soft bits using MSB-first extraction, mapping hard 0 $\rightarrow$ -127 and hard 1 $\rightarrow$ $+127$.
- (b) Feed soft bits to `rx_harq_decode()`.
- (c) If `RETRANSMIT` flag is set, the HARQ module performs Chase Combining: adds new soft bits to the accumulator from the previous attempt, then clamps to $[-127, +127]$.
- (d) Run Viterbi decoder on the combined soft bits.
- (e) Check decoded CRC:
 - CRC valid $\rightarrow$ send ACK, deliver decoded payload to application.
 - CRC invalid $\rightarrow$ send NACK, keep accumulator for next retry.

3. If FEC disabled: standard CRC-32 validation only.

4.8.3 USB CDC Path: No HARQ

USB CDC does not require application-level HARQ. The USB hardware provides CRC-16 on every packet and automatic bulk retry on error, making ARQ and FEC redundant. The `CDCLink` implementation (`internal/link/cdc.go` and `libs/rx_usb_comm/`) contains **zero references** to HARQ, FEC, Viterbi, or Chase Combining code.

4.9 Safety Features

- **Communication watchdog (500 ms):** The RX72N triggers an emergency stop if no valid frame is received within 500 ms, ensuring the robot halts on link failure.
- **Duplicate detection:** Shared `SessionState` sequence numbers detect and discard duplicate frames across both transports.
- **CRC-32 (IEEE 802.3):** Every frame is protected by CRC-32, hardware-accelerated on the RX72N CRC Calculator peripheral.
- **Failover continuity:** Shared `SessionState` prevents sequence desynchronization during USB $\leftrightarrow$ SPI failover.
- **Atomic sequence assignment:** A `sendMutex` ensures that sequence number assignment and frame transmission are atomic, preventing out-of-order frames from concurrent goroutines.

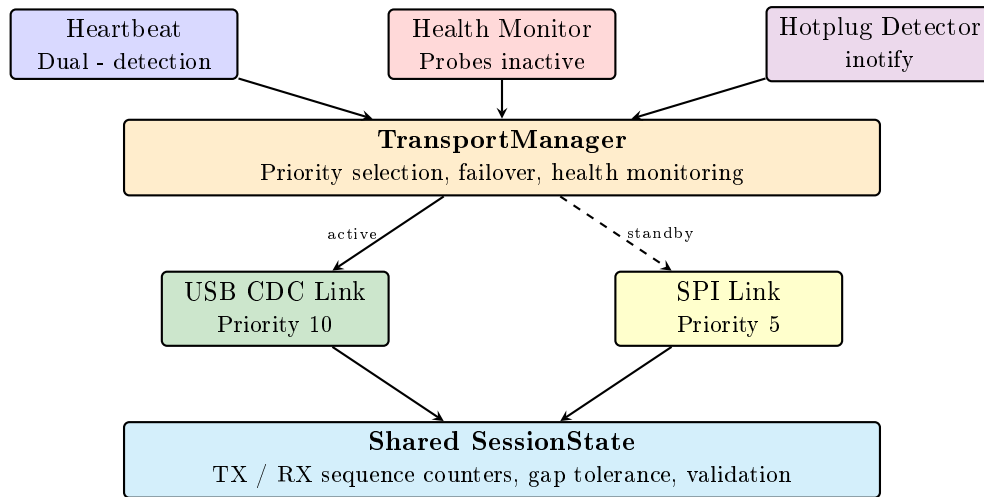
Important: The 500 ms communication watchdog is the last line of defense. If all transports fail and no valid frame arrives within this window, the RX72N immediately disables all motor outputs and enters the emergency stop state. Recovery requires an explicit reset command after communication is restored.

5 Transport Failover and Hot Switching

This section documents how the STAR gateway automatically detects transport failures, switches between USB CDC and SPI, recovers when a failed transport comes back online, and maintains communication continuity throughout.

5.1 Architecture Overview

The gateway's `TransportManager` (`internal/manager/manager.go`) orchestrates all failover logic. It maintains exactly **one active transport** at any time and proxies all `Send()`/`Receive()` operations through it. The RX72N firmware is a **passive listener** — it does not participate in transport selection decisions.



Key Insight : The gateway sends on exactly one transport at a time .It never sends the same command on both USB and SPI simultaneously .The RX72N firmware listens on both channels but the gateway's single-active-transport design means only one will have data at any given moment (except during brief failover transitions).

5.2 Priority System

Each transport is registered with a numeric priority.Higher values indicate preferred transports :

Table 29: Transport Priority Configuration

Transport	Priority	Role	Rationale
USB CDC	10	Primary	Lower latency, hardware CRC, no HARQ overhead
SPI	5	Backup	Always available, full HARQ protection

Priority rules:

1. At startup, the transport with the highest priority that is healthy becomes active.
2. If the active transport fails, switch to the highest - priority healthy alternative.
3. If a higher - priority transport recovers, switch back to it(after damping period).
4. Priority values are constants defined at compile time, not configurable at runtime.

5.3 Dual - Detection Heartbeat

The heartbeat system uses two independent mechanisms to detect transport failures. This provides both fast failure detection (implicit) and explicit liveness checking (PING / PONG) .

5.3.1 Implicit Heartbeat(200 ms Timeout)

Any valid frame received on a transport resets that transport's implicit heartbeat timer. If no frame arrives within 200 ms, the transport is considered unhealthy.

- **Timer reset** : Every valid frame (COMMAND, RESPONSE, TELEMETRY, PONG, ACK) resets the timer.
- **Timeout**: 200 ms with no frames → mark transport as **degraded** .
- **No extra traffic** : Piggybacked on existing data flow. In normal operation at 100 Hz (10 ms between frames), the implicit timer never fires.
- **Fast detection** : If the USB cable is unplugged mid - stream , detection occurs within 200 ms.

5.3.2 Explicit Heartbeat (1 s PING/PONG)

When the link is idle (no data frames for 1 second), the gateway sends a PING frame and expects a PONG response within 200 ms.

- **PING interval**: Sent after 1 second of idle (no other frames sent or received).
- **PONG timeout**: 200 ms. If no PONG arrives, the transport is marked unhealthy .
- **Purpose** : Detects silent failures where the physical connection exists but the remote device is unresponsive (e.g., firmware crash, watchdog reset) .
- **Frame types** : PING = 0x00, PONG = 0x01 (see frame type table in Section refsec : communication - stack) .

Why two mechanisms ?

- **Implicit** catches failures during active communication (fast : 200 ms) .
- **Explicit** catches failures during idle periods (slower : 1 s + 200 ms) .
- Neither generates unnecessary traffic alone. Combined, they cover all scenarios.

5.4 Health Monitoring

The `HealthMonitor( internal / manager / health_monitor.go)` runs a background goroutine that periodically probes **inactive** transports to detect when they recover . It does not interfere with the active transport.

5.4.1 Health Thresholds

A transport is considered unhealthy if **any** of the following conditions are met :

Table 30: Health Monitoring Thresholds

Metric	Threshold	Detection Method
Consecutive failures	≥ 3	Send or receive returns error 3 times in a row
Average latency	> 200 ms	Exponential moving average of round - trip times
Packet loss rate	$> 10\%$	Sliding window over recent operations

5.4.2 Probing Inactive Transports

The health monitor checks inactive transports every 5 seconds (configurable via `HealthCheckInterval`):

1. Select all transports that are **not** the currently active transport.
2. For each inactive transport, send a PING and wait for PONG.
3. If PONG received within timeout: mark transport as healthy, record recovery timestamp.
4. If probe fails: keep transport marked as unhealthy.
5. If a recovered transport has **higher priority** than the current active transport, trigger failover (subject to damping window).

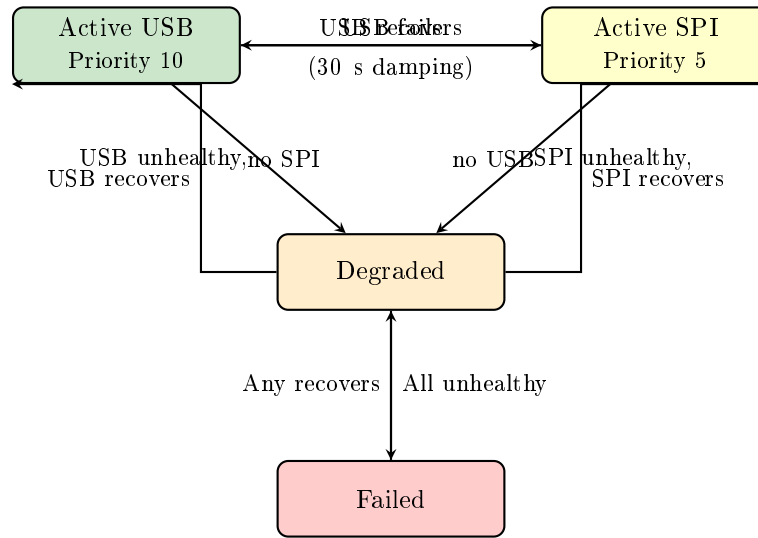
5.5 Failover State Machine

The `TransportManager` operates as a state machine with the following states :

Table 31: Transport Manager States

State	Constant	Description
Active USB	<code>StateActiveUSB</code>	USB CDC is the active transport, SPI is standby
Active SPI	<code>StateActiveSPI</code>	SPI is the active transport, USB is unavailable or unhealthy
Degraded	<code>StateDegraded</code>	Active transport is unhealthy but no better alternative exists
Failed	<code>StateFailed</code>	No healthy transports available

5.5.1 State Transitions



5.6 Smart Drain

When switching away from a transport, the gateway uses “smart drain” to handle in-flight frames gracefully:

- **Hard failure (IO error, disconnect):** Skip drain entirely. Switch immediately. In-flight frames are lost but speed is critical — the motor control loop cannot afford a 500 ms pause.
- **Graceful degradation (high latency, packet loss):** Wait up to 500 ms for in-flight frames to complete before switching. This avoids losing frames that are nearly delivered.

Failover timing with smart drain:

Table 32: Failover Timing by Failure Type

Failure Type	Detection	Total Failover Time
USB cable unplugged	~200 ms(implicit timeout)	< 300 ms(no drain)
USB device hang	~1 .2 s(PING timeout)	< 1.5 s(no drain)
High latency	~200 ms(threshold)	< 700 ms(500 ms drain)
RX72N firmware crash	~1 .2 s(PING timeout)	< 1.5 s(no drain)

Important : The RX72N’s 500 ms communication watchdog monitors *total* communication silence across all transports, not individual transport health. During failover, the gateway completes the transport switch (pause → drain → activate standby) and resumes sending on the new transport. Worst-case timing:

- **Hard failure**(cable pull, IO error) : < 300 ms(no drain, immediate switch)
- **Graceful / timeout** : < 700 ms(500 ms drain + 200 ms switch overhead)
- **Silent failure**(PING timeout) : Up to 1.5 s detection + switch time

For silent failures exceeding 500 ms, the RX72N watchdog **will trigger**, initiating safe motor shutdown. The gateway detects this via health monitoring and resumes communication on the new transport. This is the intended fail - safe behavior : motor safety takes precedence over seamless failover.

5.7 Shared SessionState(Preventing the Handoff Problem)

The “Handoff Problem” occurs when two transports maintain separate sequence counters :

1. Gateway sends frames 1 –100 over USB(USB sequence = 100, SPI sequence = 0) .
2. USB fails. Gateway switches to SPI.
3. If SPI has its own counter starting at 0, the RX72N sees sequence “jump” from 100 to 0.
4. The RX72N rejects frame 0 as a duplicate(already seen sequence 0 from USB earlier) .
5. **Result** : Total communication failure after failover.

Solution : Both transports share a single `SessionState` object that owns the TX and RX sequence counters :

```

1      SessionState struct {
2          mu sync.Mutex txSequence uint16 // Shared TX counter (both USB and SPI)
3          rxSequence uint16              // Shared RX counter (both USB and SPI)
4          gapMax uint16 // Maximum acceptable sequence gap (default: 10)
5      }
6
7      func(s *SessionState) NextTxSequence() uint16 {
8          s.mu.Lock() defer s.mu.Unlock() seq := s.txSequence s.txSequence++
9          return seq
10     }
```

Listing 2: Shared SessionState(manager / session.go)

After failover : Gateway sends frames 1 –100 over USB, then frame 101 over SPI. The RX72N sees a continuous sequence regardless of which physical transport delivered it .

5.7.1 Gap Tolerance

The shared `SessionState` accepts sequence gaps of up to 10 frames(`gapMax` = 10) . This handles :

- **Rare USB packet loss** : A frame lost during transmission creates a gap of 1. The next frame(sequence $N + 1$) is accepted despite the missing N .
- **Failover transitions** : During the switch, 1 –3 frames may be in - flight on the old transport and never delivered . The new transport picks up at the next sequence number .

- **Duplicate rejection** : Frames with sequences $\leq$ the last accepted sequence are rejected as duplicates(prevents replay) .

5.8 USB Hot - Plug Detection

The gateway uses Linux `inotify` to detect USB device attach / detach events in real time, without polling :

- **Watch path**: `/ sys / bus / usb / devices`(Linux sysfs)
- **Events**: `IN_CREATE`(device attached), `IN_DELETE`(device removed)
- **Latency** : Near - instant(< 50 ms from physical plug / unplug to software notification)
- **Filter** : Only reacts to changes matching the RX72N's USB VID/PID (045B:0261, Renesas RX)

Hot - plug flow:

1. USB cable plugged in $\rightarrow$ `inotify` fires `IN_CREATE`.
2. Hotplug detector opens `/ dev / ttyACM0`, creates USB transport and CDC link .
3. Registers new transport with `TransportManager`(priority 10) .
4. If USB has higher priority than current transport $\rightarrow$ switch to USB .
5. USB cable unplugged $\rightarrow$ `inotify` fires `IN_DELETE` .
6. Hotplug detector notifies `TransportManager` $\rightarrow$ immediate failover to SPI.

5.9 Failback Damping(30 - Second Cooldown)

When a previously failed transport recovers, the gateway does **not** switch back immediately. A 30-second damping window prevents rapid oscillation (“flapping”) between transports:

- **Scenario**: USB fails, gateway switches to SPI. USB recovers 2 seconds later.
- **Without damping**: Gateway immediately switches back to USB. If USB is intermittent, it fails again 1 second later. Gateway switches to SPI again. This “flapping” disrupts communication.
- **With damping**: Gateway records the USB recovery timestamp. Waits 30 seconds. Only if USB has been continuously healthy for 30 seconds does the gateway switch back.

Damping parameters:

Table 33: Failback Damping Parameters

Parameter	Value	Description
Damping window	30 s	Minimum healthy duration before failback
Health check rate	5 s	How often inactive transports are probed
Recovery condition	6 consecutive healthy probes	$6 \times 5 \text{ s} = 30 \text{ s}$ of health

5.10 Reset Handshake(Gateway Restart Recovery)

When the gateway restarts(process crash, reboot, update), the RX72N may still be running with sequence counters at, say, 5000. The restarted gateway's counters start at 0. Without synchronization, the RX72N would reject all frames as out-of-sequence.

Three - way reset handshake:

1. **Gateway → RX72N :** Send RESET frame(type 0xFF) .
2. **RX72N → Gateway :** Respond with RESET_ACK frame(type 0xFE) .RX72N resets its TX / RX counters to 0.
3. **Gateway :** Receives RESET_ACK, resets its own SessionState counters to 0.
4. Both sides are now synchronized at sequence 0. Normal communication resumes.

Timeout : If no RESET_ACK is received within 500 ms, the gateway retries the RESET frame(up to 3 times) .If all retries fail, the gateway assumes the RX72N is unreachable and enters the StateFailed state.

Important : The reset handshake prevents a “restart deadlock” where the gateway cannot communicate because sequences are mismatched, and the RX72N cannot be told to reset because it rejects the out - of - sequence reset frame.The RESET frame type is handled specially by the RX72N- - it is always accepted regardless of sequence number.

5.11 Firmware Side : Passive Listener

The RX72N firmware does not participate in transport selection.It is a **passive listener** that :

1. Listens on both USB CDC and SPI simultaneously .
2. Accepts valid frames from either channel(validated by CRC - 32 and sequence number) .
3. Does not know or care which transport the gateway considers “active .”
4. Routes all received commands through the same callback(`internal_frame_callback`) .
5. Sends telemetry on whichever channel last received a command(implicit response routing) .

The `rx_comm_manager_poll()` function(`texttrx_comm_manager.c : 1317`) polls both channels sequentially every 10 ms(100 Hz) :

```

1      rx_comm_manager_poll(rx_comm_manager_t * mgr) {
2      /* Poll USB first */
3      rx_err_t usb_err = internal_poll_usb(mgr);
4      /* Poll SPI second */
5      rx_err_t spi_err = internal_poll_spi(mgr);
6      /* Process event queue */
7      internal_process_event_queue(mgr);
8      /* Check heartbeat timers */
9      internal_check_heartbeat(mgr);
10     return received ? k_rx_ok : k_rx_err_timeout;
11 }
```

Listing 3: RX72N Dual - Channel Polling

5.12 Timing Summary

Table 34: Transport Failover Timing Summary

Event	Timing
Implicit heartbeat timeout	200 ms
Explicit PING interval	1 s idle
PING / PONG timeout	200 ms
Health check interval	5 s
Smart drain(graceful)	up to 500 ms
Smart drain(hard failure)	0 ms(immediate)
Failback damping window	30 s
Reset handshake timeout	500 ms per attempt, 3 retries
RX72N comm watchdog	500 ms
USB hot - plug detection	< 50 ms
Worst - case failover(cable pull)	< 300 ms
Worst - case failover(silent)	< 1.5 s

5.13 Cross - References

- **Section ??** : Protocol stack, HARQ details, frame format
- **Section ??**: How the RX72N handles simultaneous data on both channels
- **Section 11**: Gateway service architecture and directory structure
- **Section ??**: USB CDC transport details
- `star-gateway/internal/manager/manager.go`: TransportManager implementation
- `star-gateway/internal/manager/heartbeat.go`: Dual-detection heartbeat
- `star-gateway/internal/manager/health_monitor.go`: Health probing for inactive transports
- `star-gateway/internal/manager/hotplug_detector.go`: USB inotify-based hot-plug detection
- `star-gateway/internal/manager/session.go`: Shared SessionState

```

=====
=====

```


6 Dual - Channel Arbitration on the RX72N

This section documents how the RX72N firmware handles the case where data arrives on both USB CDC and SPI simultaneously, what ordering guarantees exist, and why the architecture prevents conflicts .

6.1 The Question

The RX72N listens on both USB CDC and SPI at all times .What happens if frames arrive on both channels at the same time, potentially carrying different data ?

6.2 Gateway Guarantee: Single Active Transport

Under normal operation, this scenario **cannot occur** because the gateway's TransportManager maintains exactly one active transport:

```

1          Send(ctx context.Context, data[] byte, ...)
2          error {
3      tm.mu.RLock() active := tm.activeTransport // Only ONE transport
4          activeName
5          := tm.activeTransportName tm.mu.RUnlock()
6
7          if active ==
8          nil{return errors.New("no active transport available")}
9
10         err := active.Send(ctx, data, p...) // Send on THAT one only
11         // ...

```

Listing 4: Gateway sends on ONE transport only(manager.go)

The gateway **never** sends the same command on both USB and SPI simultaneously.It selects one(USB preferred, priority 10) and sends exclusively on that.

6.3 When Overlap Can Occur

Despite the single - active - transport guarantee, brief overlap is possible during failover transitions :

1. **In - flight frame during failover :** Frame N was transmitted on USB and is still being processed by the USB hardware when the gateway detects a failure and switches to SPI.Frame $N + 1$ is sent on SPI.The RX72N receives both frames in the same 10 ms poll cycle.
2. **Telemetry crossing :** The RX72N sends telemetry on USB(because the last command came from USB), while the gateway has already switched to SPI and sends a new command on SPI .The firmware receives the SPI command while USB telemetry is outgoing.

6.4 Sequential Polling : USB First, Then SPI

The `rx_comm_manager_poll()` function processes channels **sequentially**, not in parallel.There is no race condition because the entire function runs in a single thread(`comm_task`, ThreadX Priority 5, 100 Hz) :

```

12     rx_comm_manager_poll(rx_comm_manager_t * mgr) {
13     bool received = false;
14
15     /* Step 1: Poll USB channel */
16     rx_err_t usb_err = internal_poll_usb(mgr);
17     if (usb_err == k_rx_ok) {
18         received = true; // USB frame processed
19     }
20
21     /* Step 2: Poll SPI channel */
22     rx_err_t spi_err = internal_poll_spi(mgr);
23     if (spi_err == k_rx_ok) {
24         received = true; // SPI frame processed
25     }
26
27     /* Step 3: Process event queue */
28     internal_process_event_queue(mgr);
29
30     /* Step 4: Check heartbeat timers */
31     internal_check_heartbeat(mgr);
32
33     return received ? k_rx_ok : k_rx_err_timeout;
34 }

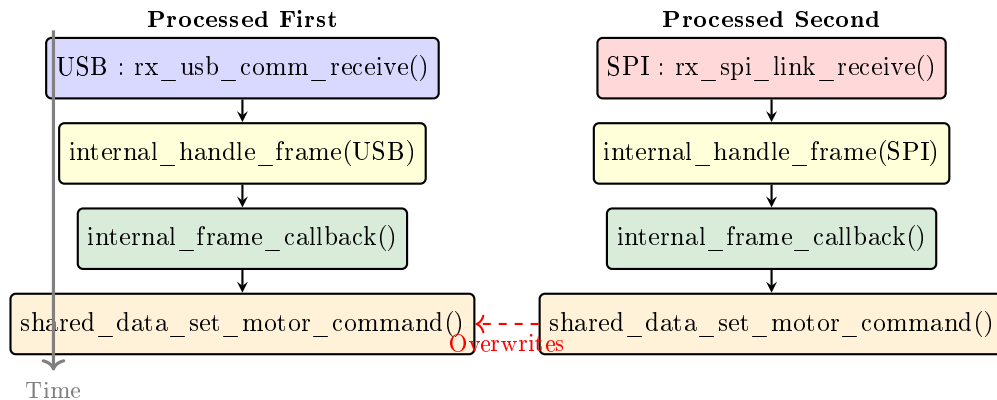
```

Listing 5: Sequential channel polling(rx_comm_manager_poll)

Ordering guarantee : Within any single poll cycle, USB is always processed before SPI. This is deterministic and repeatable.

6.5 Frame Processing Pipeline

Both `internal_poll_usb()` and `internal_poll_spi()` follow the same pipeline :



6.6 Last Writer Wins

Both frames call `shared_data_set_motor_command(& cmd)`, which is mutex - protected. Since the two calls execute sequentially (not concurrently), the second call **overwrites** the first :

1. USB frame arrives → `shared_data_set_motor_command()` writes motor velocities from USB frame.
2. SPI frame arrives → `shared_data_set_motor_command()` **overwrites** with motor velocities from SPI frame.
3. Motor task wakes up → reads the **SPI frame's data** (the last one written).

The **SPI command wins because it is polled second**. The USB command is effectively superseded.

6.7 Why This Is Correct During Failover

During a failover, the overlap works **correctly** because the gateway uses shared `SessionState` sequence numbers:

1. The gateway increments the sequence number for each `Send()` call (shared counter).
2. Frame N goes out on USB (the old transport).
3. Frame $N + 1$ goes out on SPI (the new transport).
4. The RX72N processes USB (N) first, then SPI ($N + 1$) second.
5. The motor task gets frame $N + 1$, which is the **newer** command.
6. This is the correct behavior — the motor should execute the most recent command.

Key Insight : The “last writer wins” semantics combined with sequential polling(USB before SPI) and monotonically increasing sequence numbers guarantee that the motor task always receives the most recent command, even during transport failover.

6.8 Sequence Validation

The shared `rx_session` module on the RX72N validates every received sequence number :

- **Duplicate detection :** If the same sequence number is received twice (e.g., frame retransmitted on both channels), the second copy is rejected by `rx_session_validate_rx()`.
- **Gap tolerance :** Sequence gaps of up to 10 frames are accepted (see Section 5.7). This handles frames lost during failover.
- **Wraparound:** 16 - bit sequence numbers wrap from 65535 to 0. The validation logic handles this correctly.

6.9 Channel Parameter(Currently Unused)

The `internal_handle_command_frame()` function receives a `channel` parameter indicating whether the frame arrived on USB or SPI :

```

1  internal_handle_command_frame(rx_comm_channel_t channel,
2                                const rx_frame_t *frame) {
3  (void)channel; // Currently unused
4                // ... decode protobuf, update shared_data ...
5  }
```

Listing 6: Channel parameter in command handler(`internal_handle_velocity_request`)

The channel parameter is currently cast to `void` (unused). A future enhancement could use it to:

- Route response/telemetry back on the same channel that sent the command.
- Log which channel is actively receiving commands for diagnostics.
- Implement channel-specific command filtering (e.g., only accept emergency stop from USB).

6.10 Communication Watchdog Interaction

The 500 ms communication watchdog(`shared_data_update_last_comm_tick()`) is updated on every valid frame from either channel :

```

6         internal_frame_callback(rx_comm_channel_t channel,
7                                 const rx_frame_t *frame, void *ctx) {
8     (void)ctx;
9     if (frame == nullptr) {
10         return;
11     }
12
13     /* Update watchdog on ANY valid frame from ANY channel */
14     shared_data_update_last_comm_tick();
15
16     switch (frame->header.type) {
17     case k_frame_type_command:
18         internal_handle_command_frame(channel, frame);
19         break;
20         // ...
21     }
22 }
```

Listing 7: Watchdog update on any frame(`internal_update_watchdog`)

This means :

- During failover, if USB stops sending but SPI starts, the watchdog is still satisfied.
- The 500 ms timer only triggers emergency stop if **both** channels are completely silent .
- A single valid frame on either channel resets the watchdog and prevents emergency stop.

6.11 Summary: Dual - Channel Behavior Matrix

Table 35: RX72N Behavior for Dual-Channel Scenarios

Scenario	What Happens	Result
Normal operation	Gateway sends on ONE channel only	Single frame per cycle
Failover overlap	Both channels have data	USB processed first, SPI overwrites (newer command = correct)
Duplicate frame	Same sequence on both channels	Sequence validation rejects second
Different senders	Cannot happen (single gateway)	Architecture prevents this
All channels silent	No frames for 500 ms	Emergency stop triggered
One channel fails	Other channel still delivers frames	Watchdog still satisfied

6.12 Cross - References

- **Section ??** : Protocol stack, frame format, CRC validation
- **Section 5** : Transport failover, health monitoring, shared SessionState
- `e2 - studio - star - rx72n - firmware / libs / rx_comm_manager / src / rx_comm_manager.c` : Dual - channel polling implementation
- `e2 - studio - star - rx72n - firmware / src / tasks / comm_task.c` : Frame callback, command dispatch, watchdog update
- `star - gateway / internal / manager / manager.go` : Single - active - transport guarantee

7 USB CDC Protocol

7.1 Overview

The RX72N firmware implements a USB CDC - ACM(Communications Device Class - Abstract Control Model) composite device with three independent virtual serial ports. This enables simultaneous binary protocol communication , ASCII debugging, and log streaming over a single USB connection.

7.1.1 Port Assignment

Port	Purpose	Linux Device	RX(B)	TX(B)	Usage
0	Protocol	/dev/ttyACM0	1024	1024	nanopb frames, motor commands
1	Decoded	/dev/ttyACM1	256	512	ASCII frame dumps, debugging
2	Log	/dev/ttyACM2	256	1024	System logging, diagnostics

Table 36: USB CDC Port Configuration

7.1.2 Hardware Configuration

- **USB Speed** : Full - Speed(12 Mbps)
- **Max Packet Size** : 64 bytes(bulk endpoints)
- **Pipes Used** : 9 pipes total(1 control + 3 CDC x (1 interrupt + 2 bulk))
- **Pins** : USB_DP(PA4), USB_DM(PA5), USB_VBUS(PA6)
- **Clock** : 48 MHz USB clock from PLL

7.2 USB Descriptor Hierarchy

7.2.1 Device Descriptor

```

1 {
2   .bLength = 18, .bDescriptorType = USB_DEVICE_DESCRIPTOR,
3   .bcdUSB = 0x0200,           // USB 2.0
4   .bDeviceClass = 0xEF,       // Miscellaneous (IAD)
5   .bDeviceSubClass = 0x02,    // Common Class
6   .bDeviceProtocol = 0x01,    // Interface Association
7   .bMaxPacketSize0 = 64,
8   .idVendor = 0x045B,         // Renesas
9   .idProduct = 0x0235,        // RX72N CDC
10  .bcdDevice = 0x0100,         // Device v1.0
11  .iManufacturer = 1,         // "STAR Project"
12  .iProduct = 2,               // "RX72N CDC Composite"
13  .iSerialNumber = 3,          // "RX72N-00001"
14  .bNumConfigurations = 1
15 }

```

Listing 8: USB Device Descriptor

7.2.2 Configuration Descriptor

The configuration descriptor contains :

- 3 x Interface Association Descriptors (IAD) - Groups each CDC function
- 3 x CDC Control Interface (Class 0x02, Subclass 0x02)
- 3 x CDC Data Interface (Class 0x0A, Subclass 0x00)
- 3 x Interrupt IN endpoints (control)
- 6 x Bulk endpoints (3 IN + 3 OUT for data)

7.2.3 Interface Association Descriptor(IAD)

Each CDC port uses an IAD to group its control and data interfaces :

```

1 {
2   .bLength = 8,
3   .bDescriptorType = 0x0B,     // IAD
4   .bFirstInterface = 0,        // Control interface
5   .bInterfaceCount = 2,        // Control + Data
6   .bFunctionClass = 0x02,      // CDC
7   .bFunctionSubClass = 0x02,   // ACM
8   .bFunctionProtocol = 0x01,   // AT Commands
9   .iFunction = 4                // "Protocol Port"
10 }

```

Listing 9: IAD for Port 0(Protocol Port)

Port	Type	EP	Dir	Size	Interval
0	Interrupt	EP1	IN	16	10ms
0	Bulk OUT	EP2	OUT	64	-
0	Bulk IN	EP3	IN	64	-
1	Interrupt	EP4	IN	16	10ms
1	Bulk OUT	EP5	OUT	64	-
1	Bulk IN	EP6	IN	64	-
2	Interrupt	EP7	IN	16	10ms
2	Bulk OUT	EP8	OUT	64	-
2	Bulk IN	EP9	IN	64	-

Table 37: USB Endpoint Assignments

7.2.4 Endpoint Configuration

7.3 USB State Machine

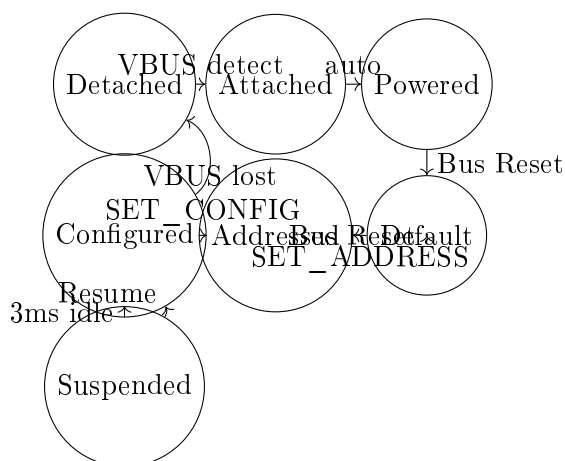


Figure 6: USB Device State Machine

Critical State : Only in **Configured** state can `rx_usb_write` / `read` transfer data.

7.4 CDC - ACM Class Requests

The USB CDC driver handles these class - specific requests :

Request	Code	Description
SET_LINE_CODING	0x20	Host configures baud rate, parity, stop bits
GET_LINE_CODING	0x21	Host queries current line coding
SET_CONTROL_LINE_STATE	0x22	DTR / RTS control signals(ignored)

Table 38: CDC - ACM Class Requests

Note : Line coding settings(baud rate, parity, etc.) are informational only.USB CDC operates at USB Full - Speed(12 Mbps), not the configured baud rate.

7.5 Bulk Transfer Protocol

7.5.1 Data Flow

Bulk OUT(Host -> Device)

1. Host writes data to / dev / ttyACM *
2. USB driver sends bulk OUT packet(max 64 bytes)
3. RX72N receives packet -> BRDY interrupt fires
4. ISR reads FIFO -> copies to RX ring buffer
5. Application calls `rx_usb_read()` -> retrieves data

Bulk IN(Device -> Host)

1. Application calls `rx_usb_write()` -> copies to TX ring buffer
2. ISR writes FIFO -> sends bulk IN packet(max 64 bytes)
3. BEMP interrupt fires when packet sent
4. Host reads data from / dev / ttyACM *

7.5.2 FIFO Access Requirements(RX72N - Specific)

CRITICAL : RX72N USB FIFO registers **must** be accessed as 16 - bit words, NOT byte - by - byte.

```

1      // CORRECT: Read CFIFO as 16-bit words
2      for (uint32_t i = 0; i < len; i += 2) {
3      const uint16_t word = usb0()->cfifo; // 16-bit read
4      data[i] = (uint8_t)(word & 0xFF);    // Low byte
5      if ((i + 1) < len) {
6          data[i + 1] = (uint8_t)((word >> 8) & 0xFF); // High byte
7      }
8  }
```

Listing 10: Correct FIFO Read(16 - bit access)

```

1      // WRONG: Byte-by-byte access causes corruption
2      for (uint32_t i = 0; i < len; i++) {
3      data[i] = (uint8_t)(usb0()->cfifo & 0xFF); // BAD!
4  }
```

Listing 11: Incorrect FIFO Read(causes data corruption)

Impact : Byte - by - byte access was the root cause of all bulk transfer failures before Phase 2 fixes.

7.5.3 FIFO Clear Sequence

Before writing to FIFO, the BCLR(Buffer Clear) sequence must be executed :

```

1      // Set BCLR bit (hardware clears FIFO)
2      usb0() -> cfifoctr
3      |= k_usb_fifoctr_bclr;
4
5      // Wait for BCLR to complete (hardware clears bit)
6      uint32_t timeout = 10000;
7      while ((usb0()->cfifoctr & k_usb_fifoctr_bclr) && timeout--) {
8          __asm__ volatile("nop");
9      }
10
11     // Now safe to write data
12     for (uint32_t i = 0; i < len; i += 2) {
13         // ... 16-bit write ...
14     }

```

Listing 12: FIFO Clear Before Write

Impact : Missing BCLR sequence caused first packet corruption with stale data.

7.6 Frame Protocol

All frames use the same wire format regardless of transport(USB CDC or SPI) :

SYNC	SEQ	LEN	TYPE	FLAGS	PAYLOAD	CRC - 32
(LE)	(LE)	(LE)				(LE)
2B	2B	2B	1B	1B	0 - 1024B	4B

Table 39: Frame Wire Format

Header Fields(little - endian) :

- **SYNC** : Magic number 0x55AA for frame synchronization
- **SEQ** : Sequence number(0 - 65535, wraps around)
- **LEN** : Payload length in bytes
- **TYPE** : Frame type(see Table ??)
- **FLAGS** : Control flags(RequiresAck, HasFEC, etc.)
- **PAYLOAD** : Variable - length data(Protocol Buffer message)

Checksum(little - endian, IEEE 802.3 LSB - first) : CRC - 32 computed over SYNC + header + payload, stored in little - endian byte order.

Value	Name	Description
0x00	PING	Heartbeat request – sent when link idle for >1s
0x01	PONG	Heartbeat response – echoes PING counter
0x10	COMMAND	Command frame carrying protobuf messages
0x11	RESPONSE	Response frame carrying protobuf messages
0x12	ACK	Acknowledgment (SPI only, HARQ protocol)
0x13	NACK	Negative acknowledgment (SPI only, HARQ protocol)
0xFE	RESET_ACK	Acknowledgment of session reset
0xFF	RESET	Request to reset session (synchronize sequences)

Table 40: Frame Type Definitions

7.6.1 Frame Types

7.6.2 Control Frame Details

PING Frame: TYPE 0x00, payload is a 4-byte counter (little-endian uint32). Sent as an explicit idle-link probe when no frames have been exchanged for >1s. Both firmware and gateway auto-respond with PONG on PING receipt.

PONG Frame: TYPE 0x01, payload echoes the PING counter. Confirms link health and validates round-trip latency.

RESET Frame: TYPE 0xFF, no payload. Requests session reset to synchronize sequence counters.

RESET_ACK Frame: TYPE 0xFE, no payload. Acknowledges session reset and confirms synchronization.

7.6.3 Cross-Compatibility Test Vectors

Both Go (gateway) and C (firmware) encode/decode frames identically. This is verified by 8 byte-exact test vectors with hardcoded expected wire bytes including CRC-32:

Test locations:

- Go : `star - gateway / internal / frame / frame_test.go`
- C : `e2 - studio - star - rx72n - firmware / tests / test_rx_frame.c`

7.7 USB CDC Lightweight Protocol

The USB CDC protocol is designed for minimal overhead, relying on USB hardware for reliability. Unlike the SPI path which uses HARQ and retransmission, the CDC path omits these as redundant with USB's built-in CRC and retry mechanisms.

Vector	TYPE	SEQ	Payload	CRC - 32
PING	0x00	0	(empty)	0x9B3FAEEB
PONG	0x01	0	counter = 42	0x287737DF
COMMAND	0x10	1	“TEST” + ACK flag	0xDEF35E60
RESPONSE	0x11	1	“OK”	0x6FC08EF4
ACK	0x12	1	(empty)	0xDEABF788
NACK	0x13	1	(empty)	0xC7B0C6C9
RESET	0xFF	0	(empty)	0x081B5399
RESET_ACK	0xFE	0	(empty)	0x110062D8

Table 41: Cross - Compatibility Test Vectors

7.7.1 Send Path

1. Acquire send mutex (serialize concurrent sends)
2. Get next TX sequence from shared SessionState
3. Create frame with sequence, CRC32
4. Encode to wire format
5. Write to `/dev/ttyACM0` with 50ms deadline
6. Release mutex (no wait for ACK)

7.7.2 Receive Path

1. Read from `/dev/ttyACM0`
2. Decode frame, validate CRC32
3. Check sequence via SessionState :
 - Accept exact match(expected sequence)
 - Accept small gaps(< 10 frames, log warning)
 - Reject duplicates or large gaps
4. Return payload to caller

7.7.3 Key Design Decisions

- ✓ Sequence tracking via shared SessionState
- ✓ CRC32 validation
- × No application - level retries(USB hardware handles retries)
- × No FEC encoding(redundant with USB reliability)
- × No ACK / NACK frames(USB provides implicit ACK)

Feature	USB CDC	SPI	Rationale
Physical Layer	Half - duplex, async	Full - duplex, sync	Hardware characteristic
Error Detection	CRC32 only	CRC32 + FEC	USB has built - in CRC
Retransmission	Hardware - level only	Application - level(HARQ)	USB bulk handles retries
FEC	Disabled	Viterbi + Chase Combining	Redundant with USB
ACK / NACK	Not sent	Required	USB provides implicit ACK
Sequence Tracking	Shared SessionState	Shared SessionState	Continuity across transports
Priority	10(preferred)	5(backup)	Prefer simpler USB
Typical Latency	0.5 - 1ms	1 - 2ms	USB has lower overhead
Reliability	Higher(hardware CRC)	High(with retry mechanisms)	USB generally more reliable due to hardware CRC

Table 42: USB CDC vs SPI Transport Comparison

7.8 Transport Comparison(USB CDC vs SPI)

7.9 Heartbeat Mechanism

The heartbeat system uses two complementary detection mechanisms to monitor link health:

7.9.1 Implicit Timeout (Primary) – 200ms

Update **LastSeen** timestamp when ANY valid frame arrives (COMMAND, RESPONSE, PING, etc.). If no frames for 200ms, declare link dead and trigger failover. Zero overhead when link is active — data frames implicitly serve as heartbeats.

7.9.2 Explicit PING (Secondary) – 1s Idle-Link Probe

Send PING with 4-byte counter if link idle for >1s. Wait for PONG echo; validate counter matches. 1 consecutive miss triggers failover. Rarely fires under normal traffic; only needed when link is genuinely idle.

Interaction between Implicit Timeout and Explicit PING : The 200ms implicit timeout is the primary dead - link detection mechanism. On an active link, data frames arrive faster than 200ms and continuously reset **LastSeen**, so the implicit timeout never fires. The 1s PING probe is a secondary mechanism : on a link that has occasional traffic(keeping the implicit timer alive) but where the application wants to verify bidirectional health, the PING / PONG exchange confirms round - trip connectivity. Note that on a truly idle link with zero traffic from either side, the 200ms implicit timeout fires before the 1s PING interval elapses— - this is by design, as an idle link with no frames in 200ms is considered dead and triggers failover.

7.9.3 Timing Budget

Parameter	Value	Description
Implicit timeout	200ms	Primary detection, any frame resets
Check interval	50ms	failureTimeout / 4, worst - case detection ~ 250ms
PING interval	1000ms	Secondary, idle - link probe
WDT timeout	1000ms	RX72N hardware watchdog
Safety margin	4x	250ms worst - case $\ll$ 1000ms WDT

Table 43: Heartbeat Timing Budget

7.9.4 Control Frame Dispatching

Both firmware(C) and gateway(Go) auto - respond PONG to PING and RESET_ACK to RESET. Control frames(PING, PONG, ACK, NACK, RESET_ACK) are consumed internally. Only data frames(COMMAND, RESPONSE) are returned to callers.

7.10 Debug Logging Over USB CDC Port 2

7.10.1 Architecture

The logging system supports dual output to both UART and USB CDC Port 2 :

- **UART(SCI12)** : Always works, independent of USB state
- **USB CDC Port 2** : Enabled when `USB_LOG_MIRROR = 1`
- **Boot buffering**: 512B ring buffer for logs during USB enumeration (0-200ms)
- **Thread safety**: ThreadX mutex for multi-task logging
- **Non-blocking**: Drops logs on TX full, tracks statistics

7.10.2 Boot Log Sequence

7.10.3 Implementation

Boot Buffer Structure

```

1 typedef struct {
2     char data[512]; // Ring buffer storage
3     uint16_t head; // Write index
4     uint16_t count; // Buffered bytes
5     bool overflow; // Overflow flag
6 } usb_log_boot_buffer_t;
7
8 static usb_log_boot_buffer_t s_boot_buffer = {0};
9 static volatile bool s_usb_ready = false;

```

Listing 13: Boot Log Ring Buffer

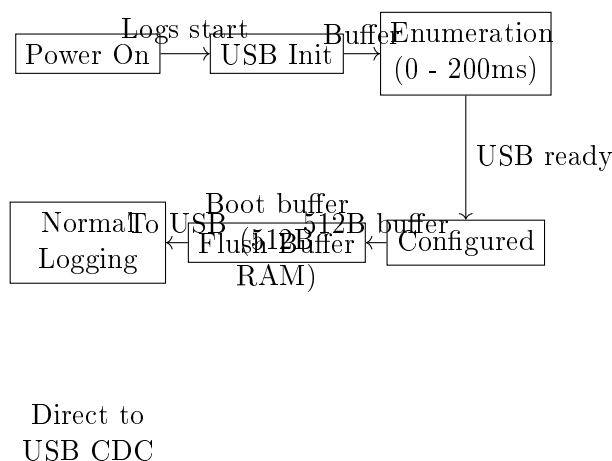


Figure 7: Boot Log Buffering Sequence

Log Write Flow

1. Application calls `rx_log_info()`, `rx_log_warn()`, etc.
2. Macro expands to `uart_debug_puts()` + `rx_log_usb_puts()`
3. `rx_log_usb_puts()` acquires ThreadX mutex
4. Check USB state :
 - **Not configured** : Buffer to 512B ring buffer
 - **Configured** : Write directly to USB CDC Port 2 via `rx_usb_write()`
5. If USB TX buffer full(`k_rx_err_busy`) : Drop log, increment `dropped_bytes`
6. Release mutex

Automatic Flush

```

1 static void internal_check_usb_ready(void) {
2     if (s_usb_ready)
3         return; // Already flushed
4
5     if (!rx_usb_is_configured(k_usb_port_log)) {
6         return; // Not ready yet, keep buffering
7     }
8
9     // USB ready! Flush boot buffer in 64-byte chunks
10    while (s_boot_buffer.count > 0) {
11        uint16_t chunk_len = min(s_boot_buffer.count, 64);
12        rx_err_t err =
13            rx_usb_write(k_usb_port_log, &s_boot_buffer.data[tail],
14                        chunk_len);
15        if (err == k_rx_err_busy)
16            return; // Retry later
17
18        // Update indices
19        tail = (tail + chunk_len) % 512;

```

```

19     s_boot_buffer.count -= chunk_len;
20 }
21
22     s_usb_ready = true; // Mark as flushed
23
24     // Log overflow warning if buffer overflowed
25     if (s_boot_buffer.overflow) {
26         rx_usb_puts(k_usb_port_log,
27                     "[WARN] Boot buffer overflow, some logs dropped\r\n");
28     }
29 }

```

Listing 14: Automatic Boot Buffer Flush

7.10.4 Thread Safety

ThreadX mutex protects all logging operations :

```

1 void rx_log_usb_puts(const char *str) {
2     // Lazy mutex initialization
3     if (!s_mutex_initialized) {
4         tx_mutex_create(&s_log_mutex, "usb_log_mutex", TX_NO_INHERIT);
5         s_mutex_initialized = true;
6     }
7
8     // Acquire mutex (blocks until available)
9     tx_mutex_get(&s_log_mutex, TX_WAIT_FOREVER);
10
11     // Critical section: write to USB
12     internal_write_usb(str, strlen(str));
13
14     // Release mutex
15     tx_mutex_put(&s_log_mutex);
16 }

```

Listing 15: Thread - Safe Logging

7.10.5 Statistics Tracking

```

1 typedef struct {
2     uint32_t total_bytes; // Total bytes sent/buffered
3     uint32_t dropped_bytes; // Bytes dropped (TX full/overflow)
4     uint32_t boot_buffered; // Bytes buffered during boot
5     uint32_t usb_errors; // USB error count
6 } usb_log_stats_t;
7
8 // Query statistics
9 usb_log_stats_t stats;
10 rx_log_usb_get_stats(&stats);
11 rx_log_info("USB_LOG", "Dropped: %u bytes (%.1f%%)", stats.dropped_bytes,
12            100.0f * stats.dropped_bytes / stats.total_bytes);

```

Listing 16: USB CDC Logging Statistics

7.11 Performance Characteristics

7.11.1 Throughput

Metric	Value	Notes
USB Speed	12 Mbps	Full - Speed(USB 2.0)
Practical Max	10 Mbps	Protocol overhead reduces from 12 Mbps theoretical
Typical Throughput	5 - 8 Mbps	Under normal application load
Packet Size	64 bytes	Full - Speed bulk endpoint limit
Latency(send 1KB)	0.5 - 1 ms	rx_usb_write() to host receive
Latency(receive 1KB)	0.5 - 1 ms	Host write to rx_usb_read()

Table 44: USB CDC Performance Metrics

7.11.2 Memory Usage

Component	Size(bytes)	Description
Port 0 RX Buffer	1024	Protocol port receive ring buffer
Port 0 TX Buffer	1024	Protocol port transmit ring buffer
Port 1 RX Buffer	256	Decoded port receive ring buffer
Port 1 TX Buffer	512	Decoded port transmit ring buffer
Port 2 RX Buffer	256	Log port receive ring buffer
Port 2 TX Buffer	1024	Log port transmit ring buffer
Boot Log Buffer	512	Boot log buffering(USB CDC logging)
USB Descriptors	512	Device, config, interface, endpoint
State Structures	128	3 ports x 42 bytes per port
Total	5.3 KB	Static allocation(no malloc / free)

Table 45: USB CDC Memory Usage

7.12 Error Handling and Recovery

7.12.1 USB Not Configured

Symptom : rx_usb_write / read returns k_rx_err_invalid_state

Recovery :

- Wait for USB enumeration: `while (!rx_usb_is_configured(port))`
- For logging : Buffer to boot buffer, flush after USB ready
- For protocol / data : Retry after delay, or use alternate channel(UART)

7.12.2 TX Buffer Full

Symptom : `rx_usb_write` returns `k_rx_err_busy`

Recovery :

- **Logging :** Drop message, increment statistics(non - blocking policy)
- **Critical data :** Retry with timeout, or signal error to application
- Poll `rx_usb_tx_available()` before write to avoid full condition

7.12.3 Cable Disconnect

Event : `k_usb_event_detached` delivered to callback

Recovery :

- Abort pending transfers
- Clear TX / RX ring buffers
- Reset state to `Detached`
- Wait for cable reconnect (`k_usb_event_attached`)

7.13 NASA Power of 10 Compliance

Rule	Status	Implementation
1. Simple control flow	✓	No goto/setjmp/recursion
2. Fixed loop bounds	✓	All loops bounded by constants
3. No dynamic memory	✓	Zero malloc/free, all buffers static
4. Short functions	✓	All functions <60 lines
5. Assertions	✓	Min 2 checks per function
6. Small scope	✓	Variables near use, static for module data
7. Check returns	✓	All <code>rx_usb_*</code> return values checked
8. Limited preprocessor	✓	C23 typed enums, minimal macros
9. Restrict pointers	Deviation	Function pointers for callbacks (DIP)
10. Compiler warnings	✓	-Wall -Wextra -Werror, zero warnings

Table 46: NASA Power of 10 Compliance Matrix

Intentional Deviation: Rule 9 (function pointers) violated for Dependency Inversion Principle - enables testability via mock implementations.

7.14 References

- **RX72N User's Manual:** Chapter 40 - USB 2.0 Full-Speed Module
- **USB 2.0 Specification:** Chapter 9 - Device Framework
- **USB CDC-ACM Specification:** Class Definitions for Communications Devices v1.2
- **Transport Architecture:** `star - gateway / TRANSPORT_ARCHITECTURE.md`

- **Implementation:** `libs / rx_usb / inc / rx_usb.h, libs / rx_core / src / rx_log_usb.c`
- **Cross-Compatibility Tests:** `star-gateway/internal/frame/frame_test.go, e2 - studio - star - rx72n - firmware / tests / test_rx_frame.c`
- **Testing Guide:** `e2 - studio - star - rx72n - firmware / USB_CDC_PHASE2_TESTING_GUIDE.md`

Part II

Style Guides

```
== == == == == == == == == == == == == == == == == == == == ==
== == == == == == == == == == == == == == == == == == == == ==
```

8 Protocol Buffer Style Guide

This section defines naming standards and conventions for all Protocol Buffer definitions in the STAR project, following Boston Dynamics style guidelines.

8.1 Terminology Standard

The project uses inclusive terminology following OSHWA (Open Source Hardware Association) standards.

Table 47: Communication Bus Terminology

Use	Avoid	Context
Controller / Peripheral	Master / Slave	I2C, SPI, 1 - Wire bus roles
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
Primary / Main	Master	Configuration structures

8.2 Protocol Buffer Naming Conventions(Boston Dynamics Style)

The STAR project follows Boston Dynamics' Protocol Buffer style guide for all .proto definitions.

Table 48: Protocol Buffer Naming Conventions

Element	Convention	Example
Package	Versioned namespace	<code>star.v1</code>
Messages	PascalCase	<code>VelocityCommand</code> , <code>BatteryState</code>
Fields	snake_case + unit suffix	<code>velocity_mps</code> , <code>temperature_celsius</code>
Enums	SCREAMING_SNAKE_CASE	<code>MOTOR_STATE_RUNNING</code>
Enum Zero Value	_UNKNOWN suffix	<code>STATUS_UNKNOWN = 0</code>
Enum Values	Type name prefix	<code>ROBOT_MODE_MANUAL</code>
RPC Pattern	{Verb} Request/Response	<code>SetVelocityRequest</code>

8.3 Unit Suffixes(MKS System)

All numeric fields use SI units with explicit suffixes .This eliminates unit ambiguity and enables automatic documentation generation.

9 C Style Guide

This section defines comprehensive style guidelines for C firmware development in the STAR project. All firmware code for the RX72N and future platforms must follow these conventions to ensure consistency, maintainability, and safety in embedded systems development.

In this project, **Assembly code** and **Inline Assembly** should be used with extreme caution. While it can offer performance improvements or enable access to low-level hardware features, its use can make code harder to understand, maintain, and port. Always prefer writing in C unless absolutely necessary.

9.1 General Guidelines for ASM and Inline ASM

- **Avoid ASM When Possible** : Always try to write in C before resorting to assembly language. C compilers are highly optimized and can often produce efficient machine code that rivals hand - written assembly.
- **Document ASM Code Thoroughly** : Assembly code is often harder to understand than C, so it should be accompanied by thorough comments that explain **why** ASM was used, as well as what the code does.

```
1  /* Set register to zero using ASM */
2  mov r0,
3  #0 /* Initialize r0 to 0 */
```

- **Use Inline ASM Sparingly** : Inline assembly allows you to embed assembly instructions directly in C code. It should only be used when absolutely necessary, such as when interacting with hardware registers or performing CPU - specific optimizations.
- **Constrain the Scope of Inline ASM** : When using inline assembly, constrain its scope to as few lines as possible and avoid intermixing it with complex C logic . This helps isolate potential issues and reduces the risk of bugs.

```
1 void set_flag(void) {
2     asm volatile("mov r0, #1" : : : "r0"); /* Set flag in register r0 */
3 }
```

- **Always Use volatile for Inline ASM** : Inline assembly should be marked as `volatile` to prevent the compiler from optimizing it out.
- **Do Not Use Inline ASM for Optimizations** : Modern C compilers perform aggressive optimizations. Inline ASM should not be used to try and optimize code performance unless absolutely necessary, as it is often more error-prone than compiler optimizations.
- **Use Constraints Correctly**: When using inline ASM in C, use the appropriate constraints to inform the compiler how the assembly instructions interact with C variables.

```

1 int add(int a, int b) {
2     int result;
3     asm volatile("add %[r], %[a], %[b]"
4                 : [r] "=r"(result)      /* Output */
5                 : [a] "r"(a), [b] "r"(b) /* Inputs */
6     );
7     return result;
8 }

```

9.2 When to Use ASM and Inline ASM

- **Hardware Access:** Use ASM when direct hardware access is required and C alone cannot provide the necessary control (e.g., setting or clearing specific registers).
- **Performance Critical Code:** ASM may be justified in performance-critical sections where C compiler optimizations fall short, but this should be documented and used as a last resort.
- **CPU-Specific Instructions:** Use ASM for executing instructions that are unique to the target CPU (e.g., specific ARM or x86 instructions that are not exposed in C).

In this project, **assertions** are used as a tool to catch programming errors and invalid states during development. They help ensure that assumptions made in the code are correct, and they provide valuable debugging information when things go wrong. However, assertions should never be relied on to handle runtime errors or expected conditions in production code.

9.3 Guidelines

- **Use Assertions for Debugging :** Assertions are intended to catch conditions that should never occur during normal program execution. Use them to assert assumptions about the state of the program, such as parameter validation or invariants within algorithms.

```

1 #include <assert.h>
2
3 void
4 process_data(int *data) {
5     assert(data != NULL); /* Assert that data is not NULL */
6     /* Continue processing */
7 }

```

9.4 Platform-Specific Error Checking Macros

Different platforms provide error checking macros for development and debugging:

- **ESP32 (ESP-IDF):** ESP_ERROR_CHECK aborts on error, ESP_ERROR_CHECK_WITHOUT_ABORT logs but continues
- **RX72N:** Custom error checking macros following similar patterns
- **General Pattern:** Check return values and handle errors appropriately for your platform

Note : While platform - specific macros are useful during development, production code should use explicit error handling with proper cleanup and recovery mechanisms .

Proper brace placement makes code cleaner and easier to read. This section establishes consistent brace conventions used throughout the STAR firmware codebase.

9.5 General Rules

- **Always Use Braces for Control Statements :** Every `if` , `for` , `while`, or similar control structure must have braces, even for single statements.
- **Same Line for Control Structures and Blocks :** For control structures like `if` , `for` , `while` , and blocks like structs and enums, always place the opening brace `{` on the same line as the statement.
- **Functions Have Braces on a New Line:** The only exception to the same-line rule is for functions, where the opening brace goes on its own line.

9.6 Control Structures

Opening brace on the **same line** as the control statement.

```

1      /* CORRECT - if/else with braces on same line */
2      if (ret != RX_OK) {
3          RX_LOG_ERROR(s_TAG, "Operation failed: %s", rx_err_to_name(ret));
4          return ret;
5      }
6
7      if (manager == NULL) {
8          return RX_ERR_INVALID_ARG;
9      } else if (manager->mutex == NULL) {
10         return RX_ERR_INVALID_STATE;
11      } else {
12         /* Proceed with operation */
13      }
14
15      /* CORRECT - Single statement still requires braces */
16      if (pin < 0 || pin >= GPIO_NUM_MAX) {
17          return RX_OK; /* Not an error, pin is not used */
18      }
19
20      /* WRONG - Missing braces */
21      if (ret != RX_OK)
22          return ret;
23
24      /* WRONG - Brace on new line */
25      if (ret != RX_OK) {
26          return ret;
27      }

```

9.6.1 For Loops

```

1      /* CORRECT - for loop with brace on same line */
2      for (star_bus_config_t *curr = manager->buses; curr;
3           curr = curr->next) {
4          if (curr->name && strcmp(curr->name, name) == 0) {
5              found_bus = curr;
6              break;
7          }
8      }
9
10     for (uint32_t i = 0; i < SPI_HOST_MAX; ++i) {
11         manager->spi_host_initialized[i] = false;
12         manager->spi_device_count[i] = 0;
13     }
14
15     /* WRONG */
16     for (uint32_t i = 0; i < count; ++i)
17         process_item(i); /* Missing braces */

```

9.6.2 While Loops

```

1      /* CORRECT - while loop with brace on same line */
2      while (curr) {
3          star_bus_config_t *next = curr->next;
4          esp_err_t destroy_result = star_bus_config_destroy(curr);
5          if (destroy_result != RX_OK && first_error == RX_OK) {
6              first_error = destroy_result;
7          }
8          curr = next;
9      }
10
11     while (error_handler_can_retry(&handler)) {
12         ret = attempt_operation();
13         if (ret == RX_OK) {
14             break;
15         }
16     }
17
18     /* WRONG */
19     while (curr) {
20         process_node(curr); /* Brace on new line */
21         curr = curr->next;
22     }

```

9.7 Functions

Opening brace on a **new line** for function definitions.

```

1      /* CORRECT - Function brace on new line */
2      rx_err_t
3      star_bus_manager_init(star_bus_manager_t * manager, const
4                           char *tag,
5                           star_error_interface_t *error_iface,

```



```

5         star_pin_interface_t *pin_iface) {
6     RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
7         "Manager pointer is NULL");
8
9     manager->buses = NULL;
10    manager->error_iface = error_iface;
11    manager->pin_iface = pin_iface;
12
13    /* ... */
14
15    return RX_OK;
16 }
17
18 static rx_err_t internal_register_bus_pin(
19     star_pin_interface_t * pin_iface, gpio_num_t pin,
20     const char *bus_name, const char *usage, bool can_be_shared) {
21     if (pin < 0 || pin >= GPIO_NUM_MAX) {
22         return RX_OK;
23     }
24     /* ... */
25 }
26
27 /* WRONG - Function brace on same line */
28 rx_err_t star_bus_manager_init(star_bus_manager_t * manager, ...) {
29     /* ... */
30 }

```

9.8 Structs and Enums

Opening brace on the **same line** as the typedef / struct / enum keyword.

```

1     /* CORRECT - Struct brace on same line */
2     typedef struct star_bus_manager {
3         star_bus_config_t *buses;
4         SemaphoreHandle_t mutex;
5         const char *tag;
6         star_error_interface_t *error_iface;
7         star_pin_interface_t *pin_iface;
8         bool spi_host_initialized[SPI_HOST_MAX];
9         int spi_device_count[SPI_HOST_MAX];
10    } star_bus_manager_t;
11
12    /* CORRECT - Enum brace on same line, C23 typed enum */
13    typedef enum : uint8_t {
14        k_star_bus_type_none,
15        k_star_bus_type_i2c,
16        k_star_bus_type_spi,
17        k_star_bus_type_gpio,
18        k_star_bus_type_adc,
19        k_star_bus_type_count,
20    } star_bus_type_t;
21
22    /* CORRECT - Union brace on same line */
23    union {

```

```

24     star_i2c_bus_config_t i2c;
25     star_spi_bus_config_t spi;
26     star_gpio_bus_config_t gpio;
27     star_adc_bus_config_t adc;
28 } proto;
29
30 /* WRONG - Brace on new line */
31 typedef struct star_bus_manager {
32     /* ... */
33 } star_bus_manager_t;

```

9.9 Initializer Lists

Opening brace on the **same line** as the assignment.

```

1     /* CORRECT - Initializer brace on same line */
2     star_bus_event_t event = {
3         .bus_type = k_star_bus_type_i2c,
4         .bus_name = config->name,
5         .i2c = {
6             .port = port, .address = address, .is_write = true, .len =
7                 len}};
8
9     spi_device_interface_config_t spi_dev_cfg = {
10         .clock_speed_hz = 10 * 1000 * 1000,
11         .mode = 0,
12         .spics_io_num = GPIO_NUM_5,
13         .queue_size = 7,
14         .pre_cb = NULL,
15         .post_cb = NULL,
16     };
17
18     /* CORRECT - Short initializers can be on one line */
19     i2c_config_t i2c_conf = {.mode = I2C_MODE_MASTER};
20
21     /* WRONG - Brace on new line */
22     star_bus_event_t event = {
23         .bus_type = k_star_bus_type_i2c,
24         /* ... */
25     };

```

9.10 Closing Brace Placement

Closing braces are always on their own line, except for `else`, `else if`, and `while` in `do-while`.

```

1     /* CORRECT - else on same line as closing brace */
2     if (ret != RX_OK) {
3         handle_error(ret);
4     }
5     else {
6         continue_operation();
7     }

```

```

8
9  /* CORRECT - do-while */
10 do {
11     ret = try_operation();
12 } while (ret == RX_ERR_TIMEOUT && retries-- > 0);
13
14 /* CORRECT - else if chain */
15 if (config->type == k_star_bus_type_i2c) {
16     /* Handle I2C */
17 } else if (config->type == k_star_bus_type_spi) {
18     /* Handle SPI */
19 } else {
20     /* Handle other */
21 }

```

9.11 Summary Table

Construct	Opening Brace Position
Functions	New line
if/else/else if	Same line
for	Same line
while	Same line
do-while	Same line
switch	Same line
struct	Same line
enum	Same line
union	Same line
Initializer lists	Same line

Switch statements follow specific formatting rules to ensure consistency and readability throughout the STAR firmware codebase.

9.12 General Rules

- **Same Line for Switch and Opening Brace** : The opening brace `{` for a `switch` statement should be on the same line as the `switch` keyword.
- **Braces for Case Blocks** : Each `case` should have its own block of code enclosed in braces `{}`.
- **Break Statements**: Every case must end with `break`, `return`, or a fall-through comment.
- **Default Case**: Always include a `default` case, even if it only logs a warning.

9.13 Basic Switch Structure

```

1      /* CORRECT - from star_bus_manager.c */
2      switch (config->type) {
3          case k_star_bus_type_i2c: {
4              ret =

```

```

5      internal_register_bus_pin(pin_iface,
6                                config->proto.i2c.config.sda_io_num,
7                                config->name, "I2C SDA", true);
8      if (ret != RX_OK && first_error == RX_OK) {
9          first_error = ret;
10     }
11
12     ret =
13         internal_register_bus_pin(pin_iface,
14                                   config->proto.i2c.config.scl_io_num,
15                                   config->name, "I2C SCL", true);
16     if (ret != RX_OK && first_error == RX_OK) {
17         first_error = ret;
18     }
19     break;
20 }
21
22 case k_star_bus_type_spi: {
23     ret = internal_register_bus_pin(pin_iface,
24                                     config->proto.spi.copi_io_num,
25                                     config->name, "SPI COPI", true);
26     if (ret != RX_OK && first_error == RX_OK) {
27         first_error = ret;
28     }
29     /* ... more pin registrations ... */
30     break;
31 }
32
33 default: {
34     RX_LOG_DEBUG(s_TAG, "No pin registration for bus type: %d",
35                 config->type);
36     break;
37 }
38 }

```

9.14 Case Block Braces

Each case must have its own braces. This prevents variable scope issues and improves readability.

```

1      /* CORRECT - Each case has braces */
2      switch (bus_type) {
3          case k_star_bus_type_i2c: {
4              i2c_port_t port = config->proto.i2c.port; /* Local variable */
5              uint8_t address = config->proto.i2c.address;
6              RX_LOG_INFO(s_TAG, "I2C bus on port %d, address 0x%02X", port,
7                          address);
8              break;
9          }
10         case k_star_bus_type_spi: {
11             spi_host_device_t host = config->proto.spi.host; /* Different scope */
12             /*
13              * RX_LOG_INFO(s_TAG, "SPI bus on host %d", host);
14              */
15             break;
16         }
17     }

```

```

14     case k_star_bus_type_gpio: {
15         uint8_t pin_count = config->proto.gpio.pin_count;
16         RX_LOG_INFO(s_TAG, "GPIO bus with %d pins", pin_count);
17         break;
18     }
19     default: {
20         RX_LOG_WARN(s_TAG, "Unknown bus type: %d", bus_type);
21         break;
22     }
23 }
24
25 /* WRONG - Missing braces */
26 switch (bus_type) {
27 case k_star_bus_type_i2c:
28     handle_i2c();
29     break;
30 case k_star_bus_type_spi:
31     handle_spi();
32     break;
33 }

```

9.15 Break Statement Requirements

Every case must have a clear exit: **break**, **return**, or explicit fall - through.

```

1         /* CORRECT - Using break */
2         switch (config->type) {
3             case k_star_bus_type_i2c: {
4                 process_i2c(config);
5                 break;
6             }
7             case k_star_bus_type_spi: {
8                 process_spi(config);
9                 break;
10            }
11            default: {
12                break;
13            }
14        }
15
16        /* CORRECT - Using return */
17        const char *star_bus_type_to_string(star_bus_type_t type) {
18            switch (type) {
19                case k_star_bus_type_none: {
20                    return "NONE";
21                }
22                case k_star_bus_type_i2c: {
23                    return "I2C";
24                }
25                case k_star_bus_type_spi: {
26                    return "SPI";
27                }
28                case k_star_bus_type_gpio: {
29                    return "GPIO";

```

```

30     }
31     case k_star_bus_type_adc: {
32         return "ADC";
33     }
34     case k_star_bus_type_count: {
35         return "COUNT";
36     }
37     default: {
38         return "UNKNOWN";
39     }
40 }
41 }
42
43 /* CORRECT - Explicit fall-through (rare, must be commented) */
44 switch (priority) {
45 case PRIORITY_CRITICAL: {
46     log_critical(message);
47     /* FALLTHROUGH */
48 }
49 case PRIORITY_HIGH: {
50     notify_admin(message);
51     /* FALLTHROUGH */
52 }
53 case PRIORITY_NORMAL: {
54     log_message(message);
55     break;
56 }
57 default: {
58     break;
59 }
60 }

```

9.16 Default Case Handling

Always include a default case. Use it for error handling or logging unexpected values.

```

1 /* CORRECT - Default case logs warning */
2 switch (curr->type) {
3     case k_star_bus_type_i2c: {
4         reg_result =
5             register_pin_if_valid(curr->proto.i2c.config.sda_io_num, bus_name,
6                                 "I2C SDA", reg_func, user_ctx, true);
7
8         /* ... */
9         break;
10    }
11    case k_star_bus_type_spi: {
12        reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
13                                          bus_name,
14                                          "SPI COPI", reg_func, user_ctx,
15                                          true);
16
17        /* ... */
18        break;
19    }
20    default: {

```

```

17     RX_LOG_WARN(manager->tag,
18                 "Skipping pin registration for "
19                 "unsupported bus type: %d",
20                 curr->type);
21     break;
22 }
23 }
24
25 /* CORRECT - Default case returns error for invalid input */
26 rx_err_t process_bus_type(star_bus_type_t type) {
27     switch (type) {
28         case k_star_bus_type_i2c: {
29             return process_i2c();
30         }
31         case k_star_bus_type_spi: {
32             return process_spi();
33         }
34         default: {
35             RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
36             return RX_ERR_INVALID_ARG;
37         }
38     }
39 }

```

9.17 Switch on Enum Types

When switching on an enum, consider handling all values explicitly.

```

1     /* CORRECT - All enum values handled */
2     switch (config->type) {
3 case k_star_bus_type_none: {
4     RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
5     break;
6 }
7 case k_star_bus_type_i2c: {
8     /* Initialize I2C */
9     break;
10 }
11 case k_star_bus_type_spi: {
12     /* Initialize SPI */
13     break;
14 }
15 case k_star_bus_type_gpio: {
16     /* Initialize GPIO */
17     break;
18 }
19 case k_star_bus_type_adc: {
20     /* Initialize ADC */
21     break;
22 }
23 case k_star_bus_type_count: {
24     /* Sentinel value - should never be used */
25     RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
26     return RX_ERR_INVALID_ARG;

```

```

27 }
28  /* No default needed when all enum values are handled */
29  /* Compiler warns if new enum values are added */
30 }

```

9.18 Indentation

Case labels are at the same indentation level as the switch. Case body is indented one level.

```

1  /* CORRECT indentation */
2  switch (state) {
3  case STATE_IDLE: {
4  /* Case body indented */
5  if (condition) {
6  /* Nested code further indented */
7  }
8  break;
9  }
10 case STATE_RUNNING: {
11     process_running();
12     break;
13 }
14 default: {
15     break;
16 }
17 }
18
19 /* WRONG - Case labels indented */
20 switch (state) {
21 case STATE_IDLE: {
22     break;
23 }
24 }

```

9.19 Summary

- Opening brace on same line as **switch**
- Each **case** has its own braces { ... }
- Every case ends with **break**, **return**, or fall - through comment
- Always include **default** case
- Case labels at switch indentation level
- Case body indented one level inside braces

The maximum line length is **80 characters**. Lines longer than this should be split.

- **Break at Natural Points:** Break lines at natural points, such as after operators.
- **Indentation for Continuation Lines :** When breaking lines, ensure the continued line is indented properly.

All functions must include Doxygen comments. This section establishes comprehensive commenting standards for the STAR firmware codebase.

9.20 General Commenting Guidelines

- **Use Block Comments** (*/* */*): Always prefer block comments over line comments (*//*).
- **Explain Why, Not What** : Comments should explain the reasoning behind code, not what the code does(which should be clear from the code itself) .
- **Keep Comments Updated** : Outdated comments are worse than no comments .
- **English Only** : All comments must be in English.

```

1      /* CORRECT - Block comment style */
2      /* Initialize the mutex for thread-safe access */
3      manager->mutex = xSemaphoreCreateMutex();
4
5      /* WRONG - Line comment style */
6      // Initialize the mutex for thread-safe access
7      manager->mutex = xSemaphoreCreateMutex();

```

9.21 File Header Comments

Every source and header file should begin with a file path comment, followed by include guards (for headers) and a `@file` Doxygen block for headers with significant content.

```

1      /* lib/star_bus/include/star_bus_manager.h */
2
3      #ifndef STAR_COMPONENT_BUS_MANAGER_H
4      #define STAR_COMPONENT_BUS_MANAGER_H
5
6      #ifdef __cplusplus
7      extern "C" {
8      #endif
9
10     #include "esp_err.h"
11     #include "star_bus_manager_types.h"
12
13     /**
14      * @file star_bus_manager.h
15      * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
16      * protocols
17      *
18      * The bus manager provides a central registry for managing multiple bus
19      * configurations. It supports dependency injection of error handlers
20      * and
21      * pin validators for loose coupling.
22      *
23      * Key Features:
24      * - Thread-safe bus management with mutex protection
25      * - Support for multiple bus types (I2C, SPI, GPIO, DHT22, etc.)
26      * - Automatic pin registration and conflict detection

```

```

25  * - Safe bus access via callback pattern (prevents use-after-free)
26  *
27  * Example Usage:
28  * @code
29  * // === Basic Bus Manager Setup ===
30  *
31  * #include "star_bus_manager.h"
32  * #include "star_bus_config.h"
33  * #include "star_error_handler.h"
34  *
35  * // 1. Create error handler and pin validator interfaces
36  * error_handler_t error_handler;
37  * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
38  *
39  * star_error_interface_t error_iface;
40  * star_pin_interface_t pin_iface;
41  * error_handler_get_interface(&error_iface, &error_handler);
42  * pin_validator_get_interface(&pin_iface);
43  *
44  * // 2. Initialize bus manager with dependency injection
45  * star_bus_manager_t bus_manager;
46  * star_bus_manager_init(&bus_manager, "main", &error_iface,
47  *                       &pin_iface);
48  * @endcode
49  */

```

9.22 Source File Header

Source files begin with a file path comment.

```

1      /* lib/star_bus/src/star_bus_manager.c */
2
3  #include "star_bus_manager.h"
4
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/semphr.h"
7      /* ... */

```

9.23 Doxygen Documentation

9.23.1 Function Documentation

All public functions must have Doxygen documentation in the header file.

```

1  /**
2   * @brief Initialize a bus manager instance.
3   *
4   * Allocates resources (like a mutex) for the manager. Must be
5   * called
6   * before any other manager functions.
7   *
8   * @param[out] manager      Pointer to bus manager structure to
9   *                           initialize.

```

```

8      *                               Must not be NULL.
9      * @param[in] tag                Optional tag string for logging
      *                               associated with
10     *                               this manager instance. If NULL or empty,
      *                               a
11     *                               default tag is used.
12     * @param[in] error_iface         Optional error handler interface (can be
      *                               NULL).
13     * @param[in] pin_iface           Optional pin validator interface (can be
      *                               NULL).
14     * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
15     *                               is NULL, ESP_ERR_NO_MEM if mutex creation
      *                               fails.
16     */
17     rx_err_t
18     star_bus_manager_init(star_bus_manager_t * manager, const char *tag,
19                          star_error_interface_t *error_iface,
20                          star_pin_interface_t *pin_iface);

```

9.23.2 Parameter Direction Markers

Use [in], [out], or [in,out] to indicate parameter direction.

```

1  /**
2  * @brief Write data to an I2C device register.
3  *
4  * @param[in] config                Pointer to I2C bus configuration.
5  * @param[in] data                  Pointer to data buffer to write.
6  * @param[in] len                   Number of bytes to write.
7  * @param[in] reg_addr              Target register address.
8  * @param[out] bytes_written        Pointer to store actual bytes written.
9  *                                 Can be NULL if not needed.
10 * @return rx_err_t RX_OK on success.
11 */
12 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
      config,
13 const uint8_t*      data,
14 size_t              len,
15 uint8_t             reg_addr,
16 size_t*             bytes_written);
17
18 /**
19 * @brief Record an error and manage retry/reset logic
20 *
21 * @param[in,out] handler            Pointer to error handler structure
22 * @param[in] error                  Error code that occurred
23 * @param[in] description            Human-readable description of the error
24 * @param[in] file                   Source file where error occurred
25 * @param[in] line                   Line number where error occurred
26 * @param[in] func                   Function name where error occurred
27 * @return RX_OK if error was handled, or the original error code.
28 */
29 rx_err_t error_handler_record_error(error_handler_t * handler, rx_err_t
      error,

```

```

30         const char *description, const char
31             *file,
32             int32_t line, const char *func);

```

9.23.3 Code Examples with @code / @endcode

Include usage examples in documentation blocks.

```

1  /**
2   * @file star_error_interface.h
3   * @brief Error handler interface for dependency inversion
4   *
5   * Example Usage:
6   * @code
7   * // == Creating and Using Error Interface ==
8   *
9   * #include "star_error_interface.h"
10  * #include "star_error_handler.h"
11  *
12  * // 1. Create concrete error handler
13  * error_handler_t error_handler;
14  * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
15  *
16  * // 2. Get interface from implementation
17  * star_error_interface_t error_iface;
18  * error_handler_get_interface(&error_iface, &error_handler);
19  *
20  * // 3. Inject into components that need error handling
21  * star_bus_manager_init(&bus_manager, "main", &error_iface,
22  *                       &pin_iface);
23  *
24  * // 4. Use the interface
25  * if (error_iface.can_retry(error_iface.ctx)) {
26  *     // Retry operation...
27  * }
28  * @endcode
29  */

```

9.23.4 Struct Member Documentation

Document struct members inline with Doxygen.

```

1  /**
2   * @brief Bus configuration structure (common part).
3   */
4  typedef struct star_bus_config {
5  const char *name;           /**< Unique name for this bus/device instance */
6  star_bus_type_t type;      /**< Type of bus (I2C, SPI, etc.) */
7  bool initialized;         /**< Whether this bus has been initialized */
8  void *handle;              /**< Driver/device handle */
9  void *user_ctx;           /**< User context pointer for callbacks */
10
11  union {

```

```

12     star_i2c_bus_config_t i2c;    /**< I2C-specific configuration */
13     star_spi_bus_config_t spi;    /**< SPI-specific configuration */
14     star_gpio_bus_config_t gpio;  /**< GPIO-specific configuration */
15     star_adc_bus_config_t adc;    /**< ADC-specific configuration */
16 } proto;
17
18     struct star_bus_config *next; /**< Next bus config in linked list */
19 } star_bus_config_t;

```

9.24 Special Comment Keywords

Use these keywords to mark special comments for tracking.

```

1  /* TODO: Implement retry logic for I2C bus reset */
2
3  /* FIXME: Memory leak when bus removal fails mid-operation */
4
5  /* BUG: Race condition possible if mutex timeout is too short */
6
7  /* XXX: This code assumes little-endian byte order */
8
9  /* NOTE: ESP-IDF uses mosi_io_num internally - we map to COPI here */
10
11 /* WARNING: Must hold mutex before calling this function */

```

9.24.1 Keyword Definitions

- **TODO** : Work that needs to be done but is not urgent.
- **FIXME** : Known bugs or issues that need to be fixed.
- **BUG** : Confirmed bugs in the code.
- **XXX** : Dangerous or hacky code that works but should be reviewed.
- **NOTE** : Important information about the code.
- **WARNING** : Critical information that must be understood before modifying.

9.25 Section Comments

Use section comments to organize code within files.

```

1     /* === Includes === */
2 #include "freertos/FreeRTOS.h"
3 #include "star_bus_manager.h"
4
5     /* === Constants === */
6     static const char *s_TAG = "STAR_BUS_MANAGER";
7
8     /* === Private Function Prototypes === */
9 static esp_err_t internal_register_bus_pin(...);
10

```

```

11  /* === Public Functions === */
12  esp_err_t star_bus_manager_init(...) { /* ... */
13
14  /* === Private Functions === */
15  static rx_err_t internal_register_bus_pin(...) { /* ... */
16
17  /* --- Pin Registration Helpers --- */
18  /* Subsection within a section */

```

9.26 Aligned Comments

Align comments after declarations for improved readability.

```

1  typedef struct error_handler {
2      uint32_t error_count;           /**< Total errors recorded */
3      uint32_t max_retries;          /**< Max retry attempts */
4      uint32_t current_retry;        /**< Current retry counter */
5      uint32_t base_retry_delay;     /**< Base delay (ms) */
6      uint32_t max_retry_delay;      /**< Max delay (ms) */
7      uint32_t current_retry_delay;  /**< Current delay with backoff */
8      rx_err_t last_error;           /**< Last recorded error code */
9      bool in_error_state;           /**< Whether in error state */
10     void *reset_context;            /**< Context for reset function */
11     SemaphoreHandle_t mutex;        /**< Mutex for thread-safety */
12
13     rx_err_t (*reset_fn)(void *context); /**< Optional reset function */
14 } error_handler_t;

```

9.27 Implementation Comments

Comments within function bodies explain non - obvious logic.

```

1  rx_err_t star_bus_manager_add_bus(
2      star_bus_manager_t * manager, star_bus_config_t * config) {
3      /* Validate inputs using ESP-IDF macro */
4      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
5                          "Manager or config pointer is NULL");
6
7      rx_err_t ret = RX_OK;
8
9      /* Acquire mutex with timeout to prevent deadlock */
10     if (xSemaphoreTake(manager->mutex,
11                        pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
12         !=
13         pdTRUE) {
14         RX_LOG_ERROR(manager->tag, "Timeout acquiring mutex");
15         return RX_ERR_TIMEOUT;
16     }
17
18     /* Check for duplicate name - bus names must be unique */
19     for (star_bus_config_t *curr = manager->buses; curr; curr = curr->next)
20     {
21         if (curr->name && strcmp(curr->name, config->name) == 0) {

```

```

20     ret = RX_ERR_INVALID_STATE;
21     goto add_give_mutex; /* Single exit point pattern */
22 }
23 }
24
25 /* Initialize hardware before adding to list
26 * This ensures manager's SPI host tracking is updated correctly */
27 if (!config->initialized) {
28     ret = star_bus_config_init(config, manager);
29     if (ret != RX_OK) {
30         goto add_give_mutex;
31     }
32 }
33
34 /* Add to head of list (O(1) insertion) */
35 config->next = manager->buses;
36 manager->buses = config;
37
38 add_give_mutex:
39     xSemaphoreGive(manager->mutex);
40     return ret;
41 }

```

9.28 Private Function Documentation

Static functions should have brief documentation in the source file.

```

1  /**
2  * @brief Helper to register a single pin via the interface
3  */
4  static rx_err_t
5  internal_register_bus_pin(star_pin_interface_t * pin_iface,
6                          gpio_num_t pin,
7                          const char *bus_name, const char *usage,
8                          bool can_be_shared) {
9
10     if (pin < 0 || pin >= GPIO_NUM_MAX) {
11         return RX_OK; /* Not an error, pin is not used */
12     }
13
14     if (!pin_iface || !pin_iface->register_pin) {
15         return RX_OK; /* No interface, skip registration */
16     }
17
18     /* Create description: "BusName: Usage" (e.g., "I2C0: SDA") */
19     char desc[64];
20     snprintf(desc, sizeof(desc), "%s: %s", bus_name, usage);
21
22     return pin_iface->register_pin(pin_iface->ctx, pin, desc,
23                                   can_be_shared);

```

9.29 Summary

- Use block comments (`/* */`), never line comments (`/**`)
- Document all public functions with Doxygen in headers
- Use `[in]`, `[out]`, `[ in, out ]` for parameter direction
- Include `@code/@endcode` examples in file and complex function docs
- Use `TODO`, `FIXME`, `BUG`, `XXX`, `NOTE`, `WARNING` keywords for tracking
- Align comments after struct members for readability
- Use section comments to organize source files

This section outlines the best practices for ensuring thread-safe and efficient concurrent programming.

9.30 RTOS Types, Mutexes, Semaphores, and Atomic Operations

- **RTOS Types:** Use RTOS types when developing for embedded systems like ESP32.
- **Mutexes:** Use a mutex when you need **exclusive access** to a shared resource.
- **Semaphores:** Use semaphores when you need to manage **shared resources** that multiple threads can access simultaneously.
- **Atomic Operations:** Use atomic operations when working with **simple data types** such as counters or flags.

This section establishes the standards for defining and using enumerations in the STAR firmware codebase.

9.31 General Rules

- **Enum Names Should End in `_t`** : All enum types should end with `_t` .
- **Use Snake Case for Enum Values** : All enum values should follow `snake_case` naming conventions.
- **Enum Values Should Start with `k_`**: To distinguish enum members from other constants, values should start with `k_`.
- **Use `typedef`**: Always use `typedef` to define the enum type.
- **Include Count Sentinel**: Include a `_count` sentinel value for iteration.

9.32 Basic Enum Structure

```

1      /* CORRECT - C23 typed enum with explicit underlying type */
2      typedef enum : uint8_t {
3          k_star_bus_type_none = 0, /**< No bus type specified */
4          k_star_bus_type_i2c = 1, /**< I2C bus */
5          k_star_bus_type_spi = 2, /**< SPI bus */
6          k_star_bus_type_gpio = 3, /**< GPIO bus */
7          k_star_bus_type_adc = 4, /**< ADC bus */
8          k_star_bus_type_count = 5, /**< Number of bus types (sentinel)
9              */
10     } star_bus_type_t;
11
12     /* WRONG - Various style violations */
13     typedef enum { /* Missing ': uint8_t' */
14         STAR_BUS_TYPE_I2C, /* Missing k_ prefix */
15         StarBusTypeSpi, /* PascalCase not allowed */
16         star_bus_type_gpio, /* Missing k_ prefix */
17     } bad_enum_style; /* Missing _t suffix */

```

9.33 Explicit Value Assignment

Assign explicit values when needed for stability or compatibility.

```

1      /* CORRECT - C23 typed enum with explicit values */
2      typedef enum : uint8_t {
3          k_error_none = 0, /**< No error */
4          k_error_minor = 1, /**< Minor error, operation can continue */
5          k_error_major = 2, /**< Major error, operation should stop */
6          k_error_fatal = 3, /**< Fatal error, system needs reset */
7          k_error_count = 4, /**< Number of error levels */
8      } error_level_t;
9
10     /* CORRECT - Bit flags with explicit underlying type */
11     typedef enum : uint8_t {
12         k_bus_flag_none = 0x00, /**< No flags */
13         k_bus_flag_initialized = 0x01, /**< Bus is initialized */
14         k_bus_flag_busy = 0x02, /**< Bus is busy */
15         k_bus_flag_error = 0x04, /**< Bus has error */
16         k_bus_flag_dma = 0x08, /**< DMA enabled */
17     } bus_flags_t;

```

9.34 Documentation

Document each enum value with Doxygen comments.

```

1      /**
2       * @brief Types of supported bus interfaces.
3       */
4      typedef enum : uint8_t {
5          k_star_bus_type_none, /**< No bus type specified */
6          k_star_bus_type_i2c, /**< I2C bus */
7          k_star_bus_type_spi, /**< SPI bus */

```

```

8      k_star_bus_type_gpio,    /**< GPIO bus */
9      k_star_bus_type_adc,    /**< ADC bus */
10     k_star_bus_type_count, /**< Number of bus types */
11 } star_bus_type_t;

```

9.35 Enum to String Functions

Provide a string conversion function for each enum type.

```

1  /* In header file */
2  const char* star_bus_type_to_string(star_bus_type_t type);
3
4  /* In source file */
5  const char *star_bus_type_to_string(star_bus_type_t type) {
6      switch (type) {
7          case k_star_bus_type_none: {
8              return "NONE";
9          }
10         case k_star_bus_type_i2c: {
11             return "I2C";
12         }
13         case k_star_bus_type_spi: {
14             return "SPI";
15         }
16         case k_star_bus_type_gpio: {
17             return "GPIO";
18         }
19         case k_star_bus_type_adc: {
20             return "ADC";
21         }
22         case k_star_bus_type_count: {
23             return "COUNT";
24         }
25         default: {
26             return "UNKNOWN";
27         }
28     }
29 }
30
31 /* Usage */
32 RX_LOG_INFO(s_TAG, "Added bus config: %s (Type: %s)", config->name,
33             star_bus_type_to_string(config->type));

```

9.36 Using the Count Sentinel

The `_count` value enables iteration and array sizing.

```

1  /* Array indexed by enum */
2  static const char *s_bus_type_names[k_star_bus_type_count] = {
3      [k_star_bus_type_none] = "NONE", [k_star_bus_type_i2c] = "I2C",
4      [k_star_bus_type_spi] = "SPI",   [k_star_bus_type_gpio] = "GPIO",
5      [k_star_bus_type_adc] = "ADC",
6  };

```

```

7
8  /* Iteration using count */
9  for (star_bus_type_t type = k_star_bus_type_none; type <
10      k_star_bus_type_count;
11      ++type) {
12      RX_LOG_INFO(s_TAG, "Bus type %d: %s", type, s_bus_type_names[type]);
13  }
14
15  /* Validation using count */
16  if (type >= k_star_bus_type_count) {
17      RX_LOG_ERROR(s_TAG, "Invalid bus type: %d", type);
18      return RX_ERR_INVALID_ARG;
19  }

```

9.37 Switch Statement Pattern

Always handle all enum values or include a default case.

```

1      /* All values handled explicitly */
2      switch (config->type) {
3          case k_star_bus_type_none: {
4              RX_LOG_WARN(s_TAG, "Bus type is NONE - skipping");
5              break;
6          }
7          case k_star_bus_type_i2c: {
8              ret = init_i2c_bus(config);
9              break;
10         }
11         case k_star_bus_type_spi: {
12             ret = init_spi_bus(config);
13             break;
14         }
15         case k_star_bus_type_gpio: {
16             ret = init_gpio_bus(config);
17             break;
18         }
19         case k_star_bus_type_adc: {
20             ret = init_adc_bus(config);
21             break;
22         }
23         case k_star_bus_type_count: {
24             /* Sentinel value should never be used */
25             RX_LOG_ERROR(s_TAG, "Invalid bus type: count sentinel");
26             return RX_ERR_INVALID_ARG;
27         }
28         /* No default - compiler warns if new enum values added */
29     }

```

9.38 Forward Declaration

Enums can be forward-declared for header dependency reduction.

```

1  /* Define enum with explicit underlying type (C23) before typedef */

```

```

2 enum star_bus_type : int32_t {
3     k_star_bus_type_none,
4     k_star_bus_type_i2c,
5     /* ... */
6 };
7 typedef enum star_bus_type star_bus_type_t;
8
9 /* Or forward-declare in header, define in source (C23 with underlying
   type) */
10 /* In header: */
11 enum star_bus_type : int32_t; /* Forward declaration with explicit type
   */
12 typedef enum star_bus_type star_bus_type_t;
13
14 /* In source: */
15 enum star_bus_type : int32_t {
16     k_star_bus_type_none,
17     k_star_bus_type_i2c,
18     /* ... */
19 };

```

9.39 Naming Convention Summary

Element	Convention	Example
Type name	snake_case_t	star_bus_type_t
Member prefix	k_	k_star_bus_type_i2c
Member naming	snake_case	k_star_bus_type_i2c
Count sentinel	_count	k_star_bus_type_count
String function	_to_string	star_bus_type_to_string

This section establishes error handling patterns for the STAR firmware codebase (RX72N and future targets).

9.40 General Principles

- **Return Error Codes:** Functions should return error codes via `esp_err_t`.
- **Check All Return Values:** Never ignore return values from functions that can fail.
- **Use goto for Clean - up:** Use `goto` to jump to a clean-up section when multiple resources need release.
- **Fail Fast:** Return early on invalid arguments or precondition failures.

9.41 Platform Error Codes

Use platform-specific error codes consistently. STAR defines generic error types that map to platform implementations.

```

1 /* Generic STAR error codes (map to platform-specific implementations) */
2 RX_OK /* Success */
3 RX_ERR_FAIL /* Generic failure */

```

```

4  RX_ERR_NO_MEM           /* Out of memory */
5  RX_ERR_INVALID_ARG      /* Invalid argument */
6  RX_ERR_INVALID_STATE    /* Invalid state for operation */
7  RX_ERR_INVALID_SIZE     /* Invalid size */
8  RX_ERR_NOT_FOUND        /* Requested resource not found */
9  RX_ERR_NOT_SUPPORTED    /* Operation not supported */
10 RX_ERR_TIMEOUT          /* Operation timed out */
11
12 /* Usage example */
13 rx_err_t star_bus_manager_init(star_bus_manager_t* manager, ...)
14 {
15     if (manager == NULL) {
16         return RX_ERR_INVALID_ARG;
17     }
18
19     manager->mutex = xSemaphoreCreateMutex();
20     if (manager->mutex == NULL) {
21         return RX_ERR_NO_MEM;
22     }
23
24     return RX_OK;
25 }

```

9.42 Validation Macros

Use platform-provided validation macros for parameter checking and early returns. These provide consistent error handling patterns across the codebase.

9.42.1 RX_RETURN_ON_FALSE

Returns an error if condition is false, with logging. ESP32 platforms use ESP_RETURN_ON_FALSE, RX72N uses RX_RETURN_ON_FALSE.

```

1  /* Platform-specific: RX72N uses rx_check.h */
2  #include "rx_check.h"
3
4  rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
5  const char* tag,
6  star_error_interface_t* error_iface,
7  star_pin_interface_t* pin_iface)
8  {
9      /* Validate required parameter with automatic logging */
10     RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
11                         "Manager pointer is NULL");
12
13     /* Multiple validations */
14     RX_RETURN_ON_FALSE(manager->mutex, RX_ERR_INVALID_STATE,
15                         manager->tag,
16                         "Manager mutex not initialized");
17
18     RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
19                         config->type < k_star_bus_type_count,
20                         RX_ERR_INVALID_ARG, manager->tag,

```

```

20         "Invalid bus type in config");
21
22     return RX_OK;
23 }

```

9.42.2 ESP_GOTO_ON_ERROR

Jumps to a label on error, with logging. Use for cleanup patterns.

```

1 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
      config,
2 const uint8_t*      data,
3 size_t              len,
4 uint8_t              reg_addr,
5 size_t*              bytes_written)
6 {
7     rx_err_t ret = RX_OK;
8     i2c_cmd_handle_t cmd = NULL;
9
10    cmd = i2c_cmd_link_create();
11    RX_RETURN_ON_FALSE(cmd, RX_ERR_NO_MEM, s_TAG,
12        "Failed to create I2C command link");
13
14    ret = i2c_master_start(cmd);
15    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': START
16        failed: %s",
17        config->name, rx_err_to_name(ret));
18
19    ret = i2c_master_write_byte(cmd, (address << 1) | I2C_MASTER_WRITE,
20        true);
21    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
22        "Write '%s': Address (W) failed: %s",
23        config->name,
24        rx_err_to_name(ret));
25
26    ret = i2c_master_write(cmd, data, len, true);
27    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG,
28        "Write '%s': Data write failed: %s", config->name,
29        rx_err_to_name(ret));
30
31    ret = i2c_master_stop(cmd);
32    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': STOP failed:
33        %s",
34        config->name, rx_err_to_name(ret));
35
36    /* Execute command */
37    ret = i2c_master_cmd_begin(port, cmd, pdMS_TO_TICKS(1000));
38    ESP_GOTO_ON_ERROR(ret, write_fail, s_TAG, "Write '%s': CMD failed",
39        config->name);
40
41    /* Success path */
42    i2c_cmd_link_delete(cmd);
43    if (bytes_written) {
44        *bytes_written = len;

```

```

41     }
42     return RX_OK;
43
44     write_fail:
45     if (cmd) {
46         i2c_cmd_link_delete(cmd);
47     }
48     return ret;
49 }

```

9.42.3 ESP_ERROR_CHECK

Use during development to catch fatal errors.

```

1  /* Development only - aborts on error */
2  /* Platform-specific error checking example (ESP32 shown) */
3  ESP_ERROR_CHECK(i2c_driver_install(port, I2C_MODE_MASTER, 0, 0, 0));
4
5  /* Generic pattern: check return value and handle errors */
6  rx_err_t ret = optional_init();
7  if (ret != RX_OK) {
8      RX_LOG_WARN(TAG, "Optional init failed: %d", ret);
9  }

```

9.43 NULL Safety Patterns

Always validate pointer parameters.

```

1  /* At function start - validate all pointers */
2  rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
3  const char*                name,
4  const uint8_t*             data,
5  size_t                    len,
6  uint8_t                   reg_addr,
7  size_t*                   bytes_written)
8  {
9      RX_RETURN_ON_FALSE(manager && name && data, RX_ERR_INVALID_ARG,
10         s_TAG,
11         "Manager, name, or data is NULL");
12      RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
13         "Write length must be > 0");
14
15      /* Clear output parameter initially */
16      if (bytes_written) {
17          *bytes_written = 0;
18      }
19
20      /* ... rest of implementation ... */
21
22      /* Check optional interface before use */
23      if (manager->error_iface && manager->error_iface->record_error) {
24          manager->error_iface->record_error(manager->error_iface->ctx, ret,

```

```

25         "Bus hardware initialization
26         failed",
27         __FILE__, __LINE__, __func__);
28     }
29     /* Use convenience macro for safe interface calls */
30     #define STAR_IFACE_RECORD_ERROR(iface, err, msg)
31         \
32         ((iface) && (iface)->record_error
33             \
34             ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
35             \
36             __LINE__, __func__)
37             \
38             : (err))
39
40 /* Usage */
41 STAR_IFACE_RECORD_ERROR(manager->error_iface, ret, "Operation failed");

```

9.44 Mutex Error Handling

Handle mutex acquisition failures properly.

```

1 rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2 star_bus_config_t* config)
3 {
4     /* Validate arguments first */
5     RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
6         "Invalid arguments");
7     RX_RETURN_ON_FALSE(manager->mutex, ESP_ERR_INVALID_STATE,
8         manager->tag,
9         "Mutex not initialized");
10
11     rx_err_t ret = RX_OK;
12
13     /* Acquire mutex with timeout */
14     if (xSemaphoreTake(
15         manager->mutex,
16         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
17         pdTRUE) {
18         RX_LOG_ERROR(manager->tag,
19             "Timeout acquiring mutex for adding bus '%s'",
20             config->name);
21         return RX_ERR_TIMEOUT;
22     }
23
24     /* Critical section */
25     /* ... operations ... */
26
27     /* Single exit point ensures mutex release */
28     add_give_mutex:
29     xSemaphoreGive(manager->mutex);
30     return ret;
31 }

```


9.45 Error Interface Dependency Injection

Components receive error handlers through dependency injection.

```

1  /* Interface definition */
2  typedef struct star_error_interface {
3      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
4          *message,
5          const char *file, int line, const char
6              *func);
7      bool (*can_retry)(void *ctx);
8      rx_err_t (*reset_state)(void *ctx);
9      void *ctx;
10 } star_error_interface_t;
11
12 /* Injecting error handler into component */
13 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
14     const char* tag,
15     star_error_interface_t* error_iface,
16     star_pin_interface_t* pin_iface)
17 {
18     manager->error_iface = error_iface; /* Can be NULL */
19     /* ... */
20 }
21
22 /* Using the error interface */
23 if (ret != RX_OK) {
24     if (manager->error_iface && manager->error_iface->record_error) {
25         manager->error_iface->record_error(manager->error_iface->ctx, ret,
26             "Bus initialization failed",
27             __FILE__, __LINE__, __func__);
28     }
29 }

```

9.46 Cleanup Patterns

Use goto for consistent cleanup when multiple resources are acquired.

```

1  rx_err_t complex_init(complex_t* obj)
2  {
3      rx_err_t ret = RX_OK;
4
5      /* Allocate resources */
6      obj->buffer = malloc(BUFFER_SIZE);
7      if (!obj->buffer) {
8          ret = RX_ERR_NO_MEM;
9          goto cleanup;
10     }
11
12     obj->mutex = xSemaphoreCreateMutex();
13     if (!obj->mutex) {

```

```

14     ret = RX_ERR_NO_MEM;
15     goto cleanup;
16 }
17
18     ret = hardware_init(obj);
19     if (ret != RX_OK) {
20         goto cleanup;
21     }
22
23     return RX_OK;
24
25 cleanup:
26     /* Release resources in reverse order */
27     if (obj->mutex) {
28         vSemaphoreDelete(obj->mutex);
29         obj->mutex = NULL;
30     }
31     if (obj->buffer) {
32         free(obj->buffer);
33         obj->buffer = NULL;
34     }
35     return ret;
36 }

```

9.47 Error Logging

Use appropriate log levels for different error severities.

```

1  /* RX_LOG_ERROR - Errors that prevent operation */
2  RX_LOG_ERROR(s_TAG, "Failed to initialize bus '%s': %s",
3  config->name, rx_err_to_name(ret));
4
5  /* RX_LOG_WARN - Warnings that don't prevent operation */
6  RX_LOG_WARN(manager->tag, "Failed to register pins for bus '%s': %s "
7  "(bus still added)", config->name, rx_err_to_name(ret));
8
9  /* RX_LOG_INFO - Informational messages */
10 RX_LOG_INFO(manager->tag, "Added bus config: %s (Type: %s)",
11 config->name, star_bus_type_to_string(config->type));
12
13 /* RX_LOG_DEBUG - Debug messages (compiled out in release) */
14 RX_LOG_DEBUG(s_TAG, "Write Success: %zu bytes to '%s' (Addr 0x%02X)",
15 len, config->name, address);

```

9.48 Summary

- Return `rx_err_t` from all functions that can fail
- Use `ESP_RETURN_ON_FALSE` for parameter validation
- Use `ESP_GOTO_ON_ERROR` for cleanup patterns
- Always validate pointers before dereferencing

- Handle mutex timeouts as `ESP_ERR_TIMEOUT`
- Use dependency injection for error handlers
- Release resources in reverse order of acquisition
- Log errors with appropriate severity levels

This section establishes standards for function design, implementation, and documentation in the STAR firmware codebase.

9.49 Return Value Conventions

- If the name of a function is an action (or an imperative command), it should return an **error-code integer** (`esp_err_t`).
- If the name is a predicate (a function that returns true/false), it should return a **succeeded boolean**.

```

1  /* Action functions return rx_err_t */
2  rx_err_t star_bus_manager_init(...);
3  rx_err_t star_bus_manager_add_bus(...);
4  rx_err_t star_bus_config_destroy(...);
5
6  /* Predicate functions return bool */
7  bool error_handler_can_retry(error_handler_t* handler);
8  bool is_bus_initialized(const star_bus_config_t* config);
9
10 /* Getter functions return the value or pointer */
11 star_bus_config_t* star_bus_manager_find_bus(...);
12 const char* star_bus_type_to_string(star_bus_type_t type);

```

9.50 Private and Exposed Private Functions

9.50.1 Private Functions (static)

Functions internal to a single source file should be declared `static` with the `internal_` prefix.

```

1  /* In .c file - private static functions */
2  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
3  gpio_num_t          pin,
4  const char*         bus_name,
5  const char*         usage,
6  bool                can_be_shared)
7  {
8      if (pin < 0 || pin >= GPIO_NUM_MAX) {
9          return RX_OK; /* Not an error, pin is not used */
10     }
11     /* ... implementation ... */
12 }
13
14 static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
15     config,

```

```

15 i2c_port_t          port,
16 i2c_cmd_handle_t   cmd,
17 const char*         operation_name)
18 {
19     /* ... implementation ... */
20 }
21
22 /* Declare prototypes at top of file */
23 static rx_err_t internal_register_bus_pin(...);
24 static rx_err_t internal_unregister_bus_pin(...);
25 static rx_err_t internal_register_config_pins(...);

```

9.50.2 Exposed Private Functions (priv_)

If a function cannot be `static` and must be exposed across files within a module (but not publicly), prefix its name with `priv_`.

```

1 /* In internal header (e.g., star_bus_internal.h) */
2 rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
3 rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
4 void      priv_star_bus_release_mutex(star_bus_manager_t* manager);

```

9.51 Parameter Validation

Validate all parameters at the start of public functions.

```

1 rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2 star_bus_config_t* config)
3 {
4     /* Validate required parameters */
5     ESP_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
6                          "Manager or config pointer is NULL");
7
8     /* Validate config contents */
9     ESP_RETURN_ON_FALSE(config->name && strlen(config->name) > 0,
10                          RX_ERR_INVALID_ARG, manager->tag,
11                          "Config name is NULL or empty");
12
13     /* Validate enum range */
14     RX_RETURN_ON_FALSE(config->type > k_star_bus_type_none &&
15                          config->type < k_star_bus_type_count,
16                          RX_ERR_INVALID_ARG, manager->tag,
17                          "Invalid bus type in config");
18
19     /* Validate state */
20     RX_RETURN_ON_FALSE(manager->mutex, RX_ERR_INVALID_STATE,
21                          manager->tag,
22                          "Manager mutex not initialized");
23
24     /* ... rest of implementation ... */
25 }
26 rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,

```

```

27  const char*          name,
28  const uint8_t*       data,
29  size_t              len,
30  uint8_t             reg_addr,
31  size_t*             bytes_written)
32  {
33      /* Validate all required pointers */
34      RX_RETURN_ON_FALSE(manager && name && data, RX_ERR_INVALID_ARG,
35                          s_TAG,
36                          "Manager, name, or data is NULL");
37
38      /* Validate numeric constraints */
39      RX_RETURN_ON_FALSE(len > 0, RX_ERR_INVALID_ARG, s_TAG,
40                          "Write length must be > 0");
41
42      /* Clear output parameters initially */
43      if (bytes_written) {
44          *bytes_written = 0;
45      }
46
47      /* ... rest of implementation ... */
48  }

```

9.52 Function Declaration Formatting

Align parameters vertically when they span multiple lines.

```

1  /* CORRECT - Aligned parameters */
2  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3  const char*          tag,
4  star_error_interface_t* error_iface,
5  star_pin_interface_t* pin_iface);
6
7  star_bus_config_t* star_bus_config_create_spi_device(
8  const char*          name,
9  spi_host_device_t    host,
10 gpio_num_t           copi_pin,
11 gpio_num_t           cipo_pin,
12 gpio_num_t           sclk_pin,
13 int32_t              dma_chan,
14 const spi_device_interface_config_t* dev_cfg);
15
16 /* Function pointers in structs */
17 typedef struct star_error_interface {
18     rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
19                             *message,
20                             const char *file, int line, const char
21                             *func);
22     bool (*can_retry)(void *ctx);
23     rx_err_t (*reset_state)(void *ctx);
24     void *ctx;
25 } star_error_interface_t;

```

9.53 Doxygen Documentation

All public functions must have complete Doxygen documentation in header files.

```

1  /**
2  * @brief Add a new bus/device configuration to the manager.
3  *
4  * Adds a previously created configuration (via star_bus_config_create_*)
5  * to the manager's internal list. The manager takes ownership
6  * conceptually,
7  * but the configuration still needs to be destroyed eventually (typically
8  * via
9  * star_bus_manager_remove_bus or star_bus_manager_deinit).
10 *
11 * The configuration must have a unique name. This function is thread-safe.
12 *
13 * @param[in] manager Pointer to the initialized bus manager.
14 *                  Must not be NULL.
15 * @param[in] config Pointer to the bus configuration structure to add.
16 *                  Must not be NULL and must have a valid name.
17 * @return rx_err_t RX_OK on success,
18 *         ESP_ERR_INVALID_ARG if manager or config is invalid,
19 *         ESP_ERR_INVALID_STATE if a bus with the same name
20 *         already exists or mutex error,
21 *         ESP_ERR_TIMEOUT if mutex times out.
22 */
23 rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
24 star_bus_config_t* config);

```

9.54 Private Function Documentation

Static functions should have brief documentation in the source file.

```

1  /**
2  * @brief Helper: Execute I2C command with retry logic
3  *
4  * Wraps i2c_master_cmd_begin() with automatic retry for transient errors.
5  *
6  * @param[in] config Pointer to I2C bus configuration
7  * @param[in] port I2C port number
8  * @param[in] cmd I2C command handle
9  * @param[in] operation_name Name of operation for logging
10 * @return rx_err_t Final result after retries
11 */
12 static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
13 config,
14 i2c_port_t port,
15 i2c_cmd_handle_t cmd,
16 const char* operation_name);

```

9.55 Function Implementation Patterns

9.55.1 Single Exit Point with Cleanup

```

1 rx_err_t star_bus_manager_remove_bus(star_bus_manager_t* manager,
2   const char*      name)
3 {
4     ESP_RETURN_ON_FALSE(manager && name && strlen(name) > 0,
5                          RX_ERR_INVALID_ARG, s_TAG, "Invalid arguments");
6
7     star_bus_config_t *to_remove = NULL;
8     star_bus_config_t *prev = NULL;
9     rx_err_t ret = RX_ERR_NOT_FOUND;
10
11     /* Acquire mutex */
12     if (xSemaphoreTake(
13         manager->mutex,
14         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
15         pdTRUE) {
16         return RX_ERR_TIMEOUT;
17     }
18
19     /* Find and unlink */
20     for (star_bus_config_t *curr = manager->buses; curr;
21          prev = curr, curr = curr->next) {
22         if (curr->name && strcmp(curr->name, name) == 0) {
23             to_remove = curr;
24             if (prev == NULL) {
25                 manager->buses = curr->next;
26             } else {
27                 prev->next = curr->next;
28             }
29             to_remove->next = NULL;
30             ret = RX_OK;
31             break;
32         }
33     }
34
35     if (ret == ESP_ERR_NOT_FOUND) {
36         goto remove_give_mutex;
37     }
38
39     /* Cleanup operations */
40     rx_err_t destroy_result = star_bus_config_destroy(to_remove);
41     if (destroy_result != RX_OK) {
42         ret = destroy_result;
43     }
44
45     remove_give_mutex:
46     xSemaphoreGive(manager->mutex);
47     return ret;
48 }

```

9.55.2 Factory Function Pattern

```

1 star_bus_config_t* star_bus_config_create_i2c(const char* name,

```

```

2  i2c_port_t    port,
3  uint8_t      address,
4  gpio_num_t   sda_pin,
5  gpio_num_t   scl_pin,
6  uint32_t     clk_speed)
7  {
8      if (name == NULL || strlen(name) == 0) {
9          RX_LOG_ERROR(s_TAG, "Invalid name for I2C bus config");
10         return NULL;
11     }
12
13     star_bus_config_t *config = calloc(1, sizeof(star_bus_config_t));
14     if (config == NULL) {
15         RX_LOG_ERROR(s_TAG, "Failed to allocate memory for I2C config");
16         return NULL;
17     }
18
19     /* Initialize common fields */
20     config->name = strdup(name);
21     config->type = k_star_bus_type_i2c;
22     config->initialized = false;
23     config->handle = NULL;
24     config->next = NULL;
25
26     /* Initialize I2C-specific fields */
27     config->proto.i2c.port = port;
28     config->proto.i2c.address = address;
29     config->proto.i2c.config.sda_io_num = sda_pin;
30     config->proto.i2c.config.scl_io_num = scl_pin;
31     config->proto.i2c.config.master.clk_speed = clk_speed;
32
33     /* Set default operations */
34     star_bus_i2c_init_default_ops(&config->proto.i2c.ops);
35
36     return config;
37 }

```

9.56 Callback Functions

```

1  /* Callback type definition */
2  typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
3  void* user_ctx);
4
5  /* Function using callback */
6  rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
7  const char* name,
8  star_bus_callback_t callback,
9  void* user_ctx)
10 {
11     ESP_RETURN_ON_FALSE(manager && name && callback, RX_ERR_INVALID_ARG,
12     s_TAG, "Invalid arguments");
13
14     /* ... find bus and invoke callback while holding mutex ... */

```



```

15     rx_err_t callback_result = callback(found_bus, user_ctx);
16
17     xSemaphoreGive(manager->mutex);
18     return callback_result;
19 }
20
21 /* Using the callback pattern */
22 rx_err_t my_bus_callback(star_bus_config_t* bus, void* ctx) {
23     ESP_LOGI(TAG, "Bus type: %s", star_bus_type_to_string(bus->type));
24     return RX_OK;
25 }
26
27 star_bus_manager_with_bus(&manager, "sensor_i2c", my_bus_callback, NULL);

```

9.57 Summary

- Action functions return `esp_err_t`; predicates return `bool`
- Use `static` with `internal_` prefix for file-local functions
- Use `priv_` prefix for exposed-but-not-public functions
- Validate all parameters at function start
- Align parameters vertically in multi-line declarations
- Document all public functions with complete Doxygen
- Use single exit point with cleanup for complex functions
- Use factory functions for object creation

Global variables should be avoided when possible. This section establishes patterns for when they are necessary.

9.58 General Principles

- **Avoid Globals:** Global variables are strongly discouraged. Use dependency injection instead.
- **File-Scoped When Necessary:** If a global is needed, limit its scope to a single file using `static`.
- **Use Prefix Conventions:** `s_` for static file-scoped, `g_` for true globals.
- **Prefer `static const` Over `#define`:** For typed constants, use `static const`.

9.59 Static File-Scoped Variables (`s_` prefix)

Use `static` with `s_` prefix for variables that should be accessible only within a single source file.

```

1 /* CORRECT - Static file-scoped variables */
2
3 /* Logging tag - most common pattern */
4 static const char* s_TAG = "STAR_BUS_MANAGER";

```

```

5  /* Configuration constants - prefer static const over #define */
6  static const uint32_t s_i2c_timeout_ms = 1000;
7  static const uint32_t s_default_retry_count = 3;
8
9
10 /* State variables scoped to this file */
11 static bool s_module_initialized = false;
12
13 /* From star_bus_i2c.c */
14 static const char* s_TAG = "STAR_BUS_I2C";
15 static const uint32_t s_i2c_timeout_ms = 1000;
16
17 /* From star_bus_config.c */
18 static const char* s_TAG = "STAR_BUS_CONFIG";

```

9.60 Why static const Over #define

static const provides type safety and scope control that #define lacks.

```

1  /* PREFERRED - Type-safe, scoped, debugger-visible */
2  static const uint32_t s_timeout_ms = 1000;
3  static const size_t s_buffer_size = 256;
4
5  /* Use the constant with type checking */
6  uint32_t timeout = s_timeout_ms; /* Compiler checks type */
7
8  /* AVOID - No type safety, global namespace pollution */
9  #define TIMEOUT_MS 1000
10 #define BUFFER_SIZE 256
11
12 /* Issues with #define: */
13 /* 1. No type checking */
14 /* 2. Can collide with other defines */
15 /* 3. Not visible in debugger */
16 /* 4. Can have unexpected macro expansion */

```

9.61 When to Use #define

Use #define for:

- Compile-time constants for array sizes
- Conditional compilation
- Macros that cannot be functions

```

1  /* Array sizes must be compile-time constants */
2  #define STAR_BUS_NAME_MAX_LEN (32)
3
4  char bus_name[STAR_BUS_NAME_MAX_LEN]; /* OK - compile-time constant */
5
6  /* Conditional compilation */
7  #ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING

```

```

8  RX_LOG_DEBUG(s_TAG, "Debug info: %d", value);
9  #endif
10
11  /* Macros with special features */
12  #define RECORD_ERROR(handler, code, desc)
13      \
14      do {
15          \
16          error_handler_record_error((handler), (code), (desc), __FILE__,
17                                  \
18                                  __LINE__, __func__);
19      } while (0)

```

9.62 Explicit Type Usage - Always Use Fixed - Width Integer Types

CRITICAL RULE: Never use implicit types like `int`, `char`, `short`, or `long`. Always use explicit fixed-width types from `<stdint.h>`.

9.62.1 Why Explicit Types Matter

- **Platform Independence:** `int` can be 16-bit or 32-bit depending on platform
- **Clear Intent:** `uint32_t` explicitly shows the variable is unsigned and 32-bit
- **Prevents Bugs:** Implicit integer promotion and sign extension bugs are avoided
- **Embedded Safety:** Embedded systems require precise control over data sizes
- **Code Portability:** Code works identically on ESP32, RX72N, and future platforms

9.62.2 Correct Type Usage

```

1  #include <stdbool.h>
2  #include <stdint.h>
3
4  /* CORRECT - Always use explicit fixed-width types */
5
6  /* Unsigned integers */
7  uint8_t   byte_value    = 0xFF;      /* 8-bit unsigned (0 to 255) */
8  uint16_t  word_value    = 0xFFFF;    /* 16-bit unsigned (0 to 65535) */
9  uint32_t  counter       = 0;         /* 32-bit unsigned */
10 uint64_t  timestamp_us  = 0;         /* 64-bit unsigned */
11
12 /* Signed integers */
13 int8_t     temperature   = -40;       /* 8-bit signed (-128 to 127) */
14 int16_t    offset        = -1000;    /* 16-bit signed */
15 int32_t    position      = 0;        /* 32-bit signed */
16 int64_t    delta_time_us = 0;       /* 64-bit signed */
17
18 /* Boolean (not int!) */
19 bool       is_enabled    = false;    /* Use stdbool.h */

```

```

20
21 /* Size and pointer types */
22 size_t    buffer_size    = 256;           /* For sizes and counts */
23 uintptr_t address        = 0x20000000; /* For storing addresses */
24
25 /* WRONG - Never use these implicit types */
26 int       bad_counter;    /* Platform-dependent size! */
27 unsigned  bad_value;      /* Ambiguous width! */
28 char      bad_byte;       /* Is it signed or unsigned? */
29 short     bad_small;      /* 16-bit? Not guaranteed! */
30 long      bad_large;      /* 32-bit or 64-bit? Depends on platform! */

```

9.62.3 Special Case: Characters and Strings

Only use `char` for actual character strings, never for byte data.

```

1 /* CORRECT - char only for strings */
2 const char* message = "Hello, STAR!";
3 char        buffer[64];
4 char        letter = 'A';
5
6 /* WRONG - Don't use char for byte data */
7 char data[256]; /* Should be uint8_t data[256] */
8 char byte = 0xFF; /* Should be uint8_t byte = 0xFF */
9
10 /* CORRECT - uint8_t for byte data */
11 uint8_t data_buffer[256];
12 uint8_t byte_value = 0xFF;

```

9.62.4 Loop Counters and Array Indices

Use `uint32_t` or `size_t` for loop counters and array indices.

```

1 /* CORRECT - Explicit types for loops */
2 for (uint32_t i = 0; i < array_length; i++) {
3     process_element(array[i]);
4 }
5
6 /* For sizes from sizeof() or string length */
7 for (size_t i = 0; i < buffer_size; i++) {
8     buffer[i] = 0;
9 }
10
11 /* WRONG - Never use int for loops */
12 for (int i = 0; i < count; i++) { /* What if count > INT_MAX? */
13     /* ... */
14 }

```

9.62.5 Function Parameters and Return Types

Always use explicit types in function signatures.

```

1  /* CORRECT - Explicit types in function signatures */
2  uint32_t calculate_checksum(const uint8_t* data, size_t length);
3  int32_t  read_temperature_celsius(uint8_t sensor_id);
4  bool     is_motor_running(uint8_t motor_id);
5
6  /* WRONG - Implicit types */
7  int calculate_value(char* data, int len);  /* Ambiguous! */

```

9.62.6 Type Casting for Enum Comparisons

When comparing enums of different types, cast to the appropriate fixed-width type.

```

1  typedef enum : uint8_t {
2      k_channel_0 = 0,
3      k_channel_1 = 1,
4      k_channel_max = 2,
5  } channel_t;
6
7  /* CORRECT - Cast to explicit type for comparison */
8  if ((int32_t)channel >= k_channel_max) {
9      return ERROR_INVALID_CHANNEL;
10 }
11
12 /* WRONG - Direct enum comparison can trigger warnings */
13 if (channel >= k_channel_max) { /* -Werror=enum-compare */
14     return ERROR_INVALID_CHANNEL;
15 }

```

9.62.7 Summary: Type Usage Rules

- **ALWAYS:** Use `uint8_t`, `int32_t`, etc. from `<stdint.h>`
- **NEVER:** Use `int`, `unsigned`, `short`, `long`
- **ONLY:** Use `char` for actual character strings
- **USE:** `bool` from `<stdbool.h>` for boolean values
- **USE:** `size_t` for sizes and counts from `sizeof()`
- **CAST:** Enums to `int32_t` when comparing different enum types

9.63 True Global Variables (`g_` prefix)

If a truly global variable is unavoidable, use the `g_` prefix. This should be rare.

```

1  /* DISCOURAGED but sometimes necessary */
2  /* g_ prefix for true globals (visible across files) */
3  g_system_state_t g_system_state;
4
5  /* Prefer dependency injection instead */
6  /* BETTER approach: */
7  typedef struct app_context {

```

```

8     star_bus_manager_t bus_manager;
9     error_handler_t error_handler;
10    /* ... */
11 } app_context_t;
12
13 /* Pass context to functions that need it */
14 void app_process_sensors(app_context_t* ctx);

```

9.64 Dependency Injection Alternative

Replace globals with dependency injection.

```

1  /* WRONG - Global error handler */
2  error_handler_t g_error_handler; /* Global - bad */
3
4  void some_function(void) {
5      if (error) {
6          RECORD_ERROR(&g_error_handler, err, "Failed"); /* Uses global */
7      }
8  }
9
10 /* CORRECT - Dependency injection */
11 typedef struct star_bus_manager {
12     star_error_interface_t *error_iface; /* Injected dependency */
13     /* ... */
14 } star_bus_manager_t;
15
16 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
17 const char* tag,
18 star_error_interface_t* error_iface, /* Injected */
19 star_pin_interface_t* pin_iface) /* Injected */
20 {
21     manager->error_iface = error_iface;
22     /* ... */
23 }
24
25 /* Usage */
26 error_handler_t error_handler;
27 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
28
29 star_error_interface_t error_iface;
30 error_handler_get_interface(&error_iface, &error_handler);
31
32 star_bus_manager_t bus_manager;
33 star_bus_manager_init(&bus_manager, "main", &error_iface, NULL);

```

9.65 Static Variables for Module State

When a module needs to maintain state, prefer explicit initialization and deinitialization.

```

1  /* Module-private state */
2  static struct {
3      bool initialized;

```

```

4     uint32_t transaction_count;
5     /* ... */
6 } s_module_state = {
7     .initialized = false,
8     .transaction_count = 0,
9 };
10
11 rx_err_t module_init(void)
12 {
13     if (s_module_state.initialized) {
14         return RX_ERR_INVALID_STATE;
15     }
16
17     /* Initialize module */
18     s_module_state.initialized = true;
19     return RX_OK;
20 }
21
22 rx_err_t module_deinit(void)
23 {
24     if (!s_module_state.initialized) {
25         return RX_ERR_INVALID_STATE;
26     }
27
28     /* Clean up */
29     s_module_state.initialized = false;
30     s_module_state.transaction_count = 0;
31     return RX_OK;
32 }

```

9.66 Thread Safety for Shared State

If static variables are accessed from multiple threads, protect with mutex.

```

1 /* Thread-safe module state */
2 static struct {
3     bool initialized;
4     SemaphoreHandle_t mutex;
5     uint32_t counter;
6 } s_shared_state;
7
8 rx_err_t safe_increment_counter(void)
9 {
10     if (xSemaphoreTake(s_shared_state.mutex, pdMS_TO_TICKS(1000)) !=
11         pdTRUE) {
12         return RX_ERR_TIMEOUT;
13     }
14
15     s_shared_state.counter++;
16
17     xSemaphoreGive(s_shared_state.mutex);
18     return RX_OK;
19 }

```

9.67 Summary

Type	Prefix	Usage
Static file-scoped	s_	Prefer this for module-internal state
True global	g_	Avoid; use dependency injection
static const	s_	Type-safe constants
#define	None	Array sizes, macros, conditional compilation

- Use `static const` for type-safe constants
- Use `s_TAG` for module logging tags
- Prefer dependency injection over global variables
- Protect shared state with mutexes
- Limit variable scope to the smallest necessary

This section establishes standards for header file organization and content in the STAR firmware codebase.

9.68 General Principles

- **Header Files Should Only Contain Declarations:** No implementations except for inline functions.
- **Minimal Includes in Headers:** Include only what is necessary for the declarations.
- **Self-Contained Headers:** Each header should compile independently.
- **One Purpose Per Header:** Split large headers into focused sub-headers.

9.69 Include Guard Naming Convention

All header files must use include guards following the pattern: `STAR_COMPONENT_FILENAME_H`

```

1  /* CORRECT - from star_bus_manager.h */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4
5      /* ... declarations ... */
6
7  #endif /* STAR_COMPONENT_BUS_MANAGER_H */
8
9  /* CORRECT - from star_bus_config.h */
10 #ifndef STAR_COMPONENT_BUS_CONFIG_H
11 #define STAR_COMPONENT_BUS_CONFIG_H
12     /* ... */
13 #endif /* STAR_COMPONENT_BUS_CONFIG_H */
14
15 /* CORRECT - from star_error_interface.h */
16 #ifndef STAR_ERROR_INTERFACE_H
17 #define STAR_ERROR_INTERFACE_H
18     /* ... */

```



```

19 #endif /* STAR_ERROR_INTERFACE_H */
20
21     /* CORRECT - from star_pin_interface.h */
22 #ifndef STAR_PIN_INTERFACE_H
23 #define STAR_PIN_INTERFACE_H
24     /* ... */
25 #endif /* STAR_PIN_INTERFACE_H */

```

9.70 C++ Compatibility

All headers must include C++ extern “C” blocks for compatibility with C++ compilers.

```

1 #ifndef STAR_COMPONENT_BUS_MANAGER_H
2 #define STAR_COMPONENT_BUS_MANAGER_H
3
4 #ifdef __cplusplus
5 extern "C" {
6 #endif
7
8     /* All declarations go here */
9
10 #ifdef __cplusplus
11 }
12 #endif
13
14 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

9.71 Header File Structure

Headers should follow this organization:

```

1     /* lib/star_bus/include/star_bus_manager.h */
2
3 #ifndef STAR_COMPONENT_BUS_MANAGER_H
4 #define STAR_COMPONENT_BUS_MANAGER_H
5
6 #ifdef __cplusplus
7 extern "C" {
8 #endif
9
10     /* === 1. Includes === */
11 #include "esp_err.h"
12 #include "star_bus_manager_types.h"
13
14     /* === 2. File Documentation === */
15 /**
16  * @file star_bus_manager.h
17  * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
18  *        protocols
19  *
20  * The bus manager provides a central registry for managing multiple
21  * bus configurations. It supports dependency injection of error
22  * handlers

```

```

21  * and pin validators for loose coupling.
22  *
23  * Example Usage:
24  * @code
25  * // Initialize bus manager with dependency injection
26  * star_bus_manager_t bus_manager;
27  * star_bus_manager_init(&bus_manager, "main", &error_iface,
28  *   &pin_iface);
29  * @endcode
30  */
31  /* === 3. Forward Declarations (if needed) === */
32  /* Forward declarations reduce header dependencies */
33
34  /* === 4. Macros and Constants === */
35 #define STAR_BUS_NAME_MAX_LEN (32)
36  /* === 5. Type Definitions === */
37  /* (Usually in separate _types.h header) */
38
39  /* === 6. Function Declarations === */
40
41  /**
42   * @brief Initialize a bus manager instance.
43   *
44   * @param[out] manager    Pointer to bus manager to initialize.
45   * @param[in]  tag        Optional tag string for logging.
46   * @param[in]  error_iface Optional error handler interface.
47   * @param[in]  pin_iface  Optional pin validator interface.
48   * @return rx_err_t RX_OK on success.
49   */
50 rx_err_t star_bus_manager_init(
51     star_bus_manager_t * manager, const char *tag,
52     star_error_interface_t *error_iface, star_pin_interface_t
53         *pin_iface);
54
55     /* ... more function declarations ... */
56
57 #ifdef __cplusplus
58 }
59 #endif
60 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

9.72 Include Order

Includes should be ordered in groups, separated by blank lines:

```

1  /* 1. Primary header (in .c file, include corresponding .h first) */
2  #include "star_bus_manager.h"
3
4  /* 2. FreeRTOS includes */
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/semphr.h"
7  #include "freertos/task.h"

```

```

8
9      /* 3. Standard library includes */
10 #include <inttypes.h>
11 #include <stdbool.h>
12 #include <stddef.h>
13 #include <stdint.h>
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17
18      /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
19 #include "driver/gpio.h"
20 #include "driver/i2c.h"
21      /* Platform-specific: RX72N uses rx_check.h */
22 #include "esp_err.h"
23 #include "esp_log.h"
24 #include "rx_check.h"
25 #include "sdkconfig.h"
26
27      /* 5. Project includes */
28 #include "star_bus_config.h"
29 #include "star_bus_i2c.h"
30 #include "star_bus_types.h"
31 #include "star_error_interface.h"

```

9.73 Forward Declarations

Use forward declarations to minimize header dependencies.

```

1  /* In star_bus_config.h - forward declare instead of including */
2  /* Forward declaration */
3  struct star_bus_manager; /* Don't need full definition */
4
5  /* Function using forward-declared type */
6  rx_err_t star_bus_config_init(star_bus_config_t*      config,
7  struct star_bus_manager* manager);
8
9  /* In star_bus_common_types.h */
10 /* Forward declarations for pointer-only usage */
11 typedef struct star_bus_config  star_bus_config_t;
12 typedef struct star_bus_manager star_bus_manager_t;

```

9.74 Separating Types from Functions

For complex modules, split type definitions into dedicated headers.

```

1  /* File organization for star_bus module */
2  star_bus/include/
3  star_bus_manager.h      /* Main API (functions) */
4  star_bus_manager_types.h /* Types for manager */
5  star_bus_common_types.h /* Shared types */
6  star_bus_protocol_types.h /* Protocol-specific types */
7  star_bus_function_types.h /* Function pointer types */

```

```

8 star_bus_event_types.h      /* Event structures */
9 star_bus_types.h           /* Controller - includes all */
10
11     /* Controller header pattern */
12     /* star_bus_types.h */
13 #ifndef STAR_COMPONENT_BUS_TYPES_H
14 #define STAR_COMPONENT_BUS_TYPES_H
15
16 #ifdef __cplusplus
17 extern "C" {
18 #endif
19
20     /* Include all type headers */
21 #include "star_bus_common_types.h"
22 #include "star_bus_event_types.h"
23 #include "star_bus_function_types.h"
24 #include "star_bus_manager_types.h"
25 #include "star_bus_protocol_types.h"
26
27 #ifdef __cplusplus
28 }
29 #endif
30
31 #endif /* STAR_COMPONENT_BUS_TYPES_H */

```

9.75 Doxygen File Documentation

Include @file documentation for headers with significant content.

```

1 /**
2  * @file star_error_handler.h
3  * @brief Error handling with retry logic and exponential backoff
4  *
5  * This module provides a concrete implementation of the error interface
6  * with automatic retry management, exponential backoff delays, and
7  * optional reset function callbacks.
8  *
9  * Key Features:
10 * - Configurable max retries and backoff delays
11 * - Exponential backoff between retries
12 * - Optional custom reset function for recovery
13 * - Thread-safe with mutex protection
14 *
15 * Example Usage:
16 * @code
17 * error_handler_t error_handler;
18 * error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
19 *
20 * if (operation_failed()) {
21 *     RECORD_ERROR(&error_handler, ESP_ERR_TIMEOUT, "Timeout");
22 * }
23 * @endcode
24 */

```

9.76 Complete Header Template

```

1      /* lib/star_module/include/star_module.h */
2
3      #ifndef STAR_COMPONENT_MODULE_H
4      #define STAR_COMPONENT_MODULE_H
5
6      #ifdef __cplusplus
7      extern "C" {
8      #endif
9
10     /* === Includes === */
11     #include <stdbool.h>
12     #include <stdint.h>
13
14     #include "esp_err.h"
15     /* === File Documentation === */
16     /**
17      * @file star_module.h
18      * @brief Brief description of this module
19      *
20      * Detailed description of what this module provides and how to use
21      * it.
22      */
23
24     /* === Forward Declarations === */
25     typedef struct star_module star_module_t;
26
27     /* === Macros === */
28     #define STAR_MODULE_DEFAULT_TIMEOUT_MS (1000)
29
30     /* === Type Definitions === */
31     typedef struct star_module {
32         bool initialized;
33         uint32_t timeout_ms;
34         void *user_ctx;
35     } star_module_t;
36
37     /* === Function Declarations === */
38
39     /**
40      * @brief Initialize the module.
41      *
42      * @param[out] module Pointer to module structure to initialize.
43      * @param[in] timeout Timeout in milliseconds.
44      * @return rx_err_t RX_OK on success.
45      */
46     esp_err_t star_module_init(star_module_t * module, uint32_t
47                               timeout);
48
49     /**
50      * @brief Deinitialize the module.
51      *

```

```

50      * @param[in,out] module Pointer to module structure to
      *   deinitialize.
51      * @return rx_err_t RX_OK on success.
52      */
53      esp_err_t star_module_deinit(star_module_t * module);
54
55  #ifdef __cplusplus
56  }
57  #endif
58
59  #endif /* STAR_COMPONENT_MODULE_H */

```

9.77 Summary

- Use include guards: `STAR_COMPONENT_FILENAME_H`
- Include C++ extern “C” blocks
- Order includes: RTOS, standard library, platform HAL, project
- Use forward declarations to minimize dependencies
- Split types into separate `_types.h` headers for complex modules
- Document files with `@file` Doxygen blocks
- Keep headers focused on declarations, not implementations

This section establishes standards for horizontal spacing in the STAR firmware codebase.

9.78 General Rules

- **Space After Keywords:** Add space after `if`, `for`, `while`, `switch`.
- **Binary Operators:** Add a single space around binary operators.
- **No Space After Function Names:** No space between function name and opening parenthesis.
- **Alignment:** Align related declarations and comments.

9.79 Space After Control Keywords

Add a space between keywords and opening parenthesis.

```

1  /* CORRECT - Space after keywords */
2  if (ret != RX_OK) {
3      return ret;
4  }
5
6  for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
7      process(curr);
8  }
9

```

```

10 while (retry_count > 0) {
11     attempt_operation();
12     retry_count--;
13 }
14
15 switch (config->type) {
16     case k_star_bus_type_i2c: {
17         /* ... */
18         break;
19     }
20 }
21
22 /* WRONG - No space after keywords */
23 if(ret != RX_OK) { /* Missing space */
24     return ret;
25 }
26
27 for(int i = 0; i < 10; i++) { /* Missing space */
28     /* ... */
29 }

```

9.80 No Space After Function Names

Do not add space between function name and opening parenthesis.

```

1 /* CORRECT - No space after function name */
2 rx_err_t ret = star_bus_manager_init(&manager, "main", NULL, NULL);
3 RX_LOG_INFO(s_TAG, "Initialized bus manager");
4 free(buffer);
5
6 /* WRONG - Space after function name */
7 esp_err_t ret = star_bus_manager_init (&manager, "main", NULL, NULL);
8 RX_LOG_INFO (s_TAG, "Initialized bus manager");
9 free (buffer);

```

9.81 Binary Operators

Add a single space around binary operators.

```

1 /* CORRECT - Spaces around binary operators */
2 int result = a + b;
3 bool is_valid = (count > 0) && (size <= MAX_SIZE);
4 uint32_t mask = flags | FLAG_ENABLED;
5 int shifted = value << 2;
6 size_t remaining = total - processed;
7
8 /* Assignment operators */
9 count += 1;
10 flags |= FLAG_INITIALIZED;
11 value >>= 4;
12
13 /* Comparison operators */
14 if (ret != RX_OK) { /* ... */

```

```

15 if (count >= threshold) { /* ... */}
16 if (ptr == NULL) { /* ... */}
17
18 /* WRONG - Missing spaces */
19 int result = a+b;
20 bool is_valid = (count>0)&&(size<=MAX_SIZE);
21 if (ret!=ESP_OK) { /* ... */}

```

9.82 Unary Operators

No space between unary operator and operand.

```

1  /* CORRECT - No space with unary operators */
2  count++;
3  --remaining;
4  bool is_null = !ptr;
5  int negative = -value;
6  uint32_t complement = ~mask;
7  size_t size = sizeof(star_bus_config_t);
8  int* ptr = &value;
9  int deref = *ptr;
10
11 /* WRONG - Space with unary operators */
12 count ++;
13 -- remaining;
14 bool is_null = ! ptr;

```

9.83 Pointer Declarations

Place asterisk with the type, not the variable name.

```

1  /* CORRECT - Asterisk with type */
2  star_bus_config_t* config;
3  const char* name;
4  void* user_ctx;
5
6  /* Function parameters */
7  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
8  const char*          tag,
9  star_error_interface_t* error_iface,
10 star_pin_interface_t*  pin_iface);
11
12 /* Multiple pointers on same line (avoid when possible) */
13 int *a, *b; /* Both are pointers */
14
15 /* WRONG - Various incorrect styles */
16 star_bus_config_t *config; /* Asterisk with variable */
17 star_bus_config_t * config; /* Spaces around asterisk */

```

9.84 Comma and Semicolon Spacing

Space after comma and semicolon, not before.


```

1  /* CORRECT */
2  star_bus_manager_init(&manager, "main", &error_iface, &pin_iface);
3
4  for (int i = 0; i < count; i++) {
5      /* ... */
6  }
7
8  int array[] = {1, 2, 3, 4, 5};
9
10 /* WRONG */
11 star_bus_manager_init(&manager, "main", &error_iface, &pin_iface);
12 for (int i = 0 ; i < count ; i++) { /* ... */}
13 int array[] = {1 ,2 ,3 ,4 ,5};

```

9.85 Parentheses Spacing

No space after opening parenthesis or before closing parenthesis.

```

1  /* CORRECT */
2  if (ret != RX_OK) {
3      return ret;
4  }
5
6  result = calculate(a, b, c);
7
8  /* WRONG - Spaces inside parentheses */
9  if ( ret != RX_OK ) {
10     return ret;
11 }
12
13 result = calculate( a, b, c );

```

9.86 Parameter Alignment

Align parameters vertically in multi-line function declarations and calls.

```

1  /* CORRECT - Aligned parameters */
2  rx_err_t star_bus_manager_init(star_bus_manager_t*    manager,
3  const char*      tag,
4  star_error_interface_t* error_iface,
5  star_pin_interface_t*  pin_iface);
6
7  star_bus_config_t* star_bus_config_create_spi_device(
8  const char*      name,
9  spi_host_device_t host,
10 gpio_num_t      copi_pin,
11 gpio_num_t      cipo_pin,
12 gpio_num_t      sclk_pin,
13 int32_t          dma_chan,
14 const spi_device_interface_config_t* dev_cfg);
15
16 /* Long function calls */

```

```

17 ESP_RETURN_ON_FALSE(manager && config,
18 ESP_ERR_INVALID_ARG,
19 s_TAG,
20 "Manager or config pointer is NULL");

```

9.87 Variable and Comment Alignment

Align related variables and their comments.

```

1  /* CORRECT - Aligned declarations and comments */
2  typedef struct error_handler {
3      uint32_t error_count;           /**< Total errors recorded */
4      uint32_t max_retries;          /**< Max retry attempts */
5      uint32_t current_retry;        /**< Current retry counter */
6      uint32_t base_retry_delay;     /**< Base delay (ms) */
7      uint32_t max_retry_delay;      /**< Max delay (ms) */
8      uint32_t current_retry_delay;  /**< Current delay with backoff */
9      rx_err_t last_error;           /**< Last recorded error code */
10     bool in_error_state;            /**< Whether in error state */
11     void *reset_context;            /**< Context for reset function */
12     SemaphoreHandle_t mutex;        /**< Mutex for thread-safety */
13 } error_handler_t;
14
15 /* Aligned local variables */
16 i2c_port_t      port      = config->proto.i2c.port;
17 uint8_t         address   = config->proto.i2c.address;
18 rx_err_t        ret       = RX_OK;
19 i2c_cmd_handle_t cmd      = NULL;

```

9.88 Struct Initializer Alignment

Align designated initializers.

```

1  /* CORRECT - Aligned initializers */
2  star_bus_event_t event = {
3      .bus_type = k_star_bus_type_i2c,
4      .bus_name = config->name,
5      .i2c      = {
6          .port      = port,
7          .address   = address,
8          .is_write  = true,
9          .len       = len
10     }
11 };
12
13 spi_device_interface_config_t spi_cfg = {
14     .clock_speed_hz = 10 * 1000 * 1000,
15     .mode           = 0,
16     .spics_io_num   = GPIO_NUM_5,
17     .queue_size     = 7,
18     .pre_cb         = NULL,
19     .post_cb        = NULL,
20 };

```

9.89 Summary

Context	Rule
After <code>if</code> , <code>for</code> , <code>while</code> , <code>switch</code>	Space
After function name	No space
Around binary operators	Space
With unary operators	No space
After comma/semicolon	Space
Inside parentheses	No space
Pointer asterisk	With type
Multi-line parameters	Align vertically

This section establishes standards for include statement organization and ordering in the STAR firmware codebase.

9.90 General Rules

- **Use Angle Brackets for System/Library Headers:** `<stdio.h>`.
- **Use Quotation Marks for Project Headers:** `"my_project.h"`.
- **Minimal Includes in Header Files:** Only include what is necessary.
- **Group and Order Includes:** Follow consistent ordering.

9.91 Include Order

Includes should be ordered in these groups, separated by blank lines:

1. Primary header (in `.c` files)
2. FreeRTOS includes
3. Standard library includes
4. Platform HAL includes
5. Project includes

```

1      /* lib/star_bus/src/star_bus_i2c.c */
2
3      /* 1. Primary header - always first in .c file */
4      #include "star_bus_i2c.h"
5
6      /* 2. FreeRTOS includes */
7      #include "freertos/FreeRTOS.h"
8
9      /* 3. Standard library includes */
10     #include <inttypes.h>
11     #include <string.h>
12
13     /* 4. Platform HAL includes (ESP-IDF, Renesas, etc.) */
14     /* Platform-specific: RX72N uses rx_check.h */

```

```

15 #include "esp_log.h"
16 #include "rx_check.h"
17
18 /* 5. Project includes */
19 #include "star_bus_manager.h"
20 #include "star_bus_types.h"

```

9.92 Complete Example from Codebase

```

1  /* lib/star_bus/src/star_bus_manager.c */
2
3  /* Primary header */
4  #include "star_bus_manager.h"
5
6  /* FreeRTOS */
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/semphr.h"
9
10 /* Standard library */
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14
15 /* Platform HAL (ESP-IDF, Renesas, etc.) */
16 /* Platform-specific: RX72N uses rx_check.h */
17 #include "esp_log.h"
18 #include "rx_check.h"
19 #include "sdkconfig.h"
20
21 /* Project includes */
22 #include "star_bus_adc.h"
23 #include "star_bus_config.h"
24 #include "star_bus_gpio.h"
25 #include "star_bus_i2c.h"
26 #include "star_bus_spi.h"

```

9.93 FreeRTOS Include Order

FreeRTOS requires specific include ordering. Always include `FreeRTOS.h` first.

```

1  /* CORRECT - FreeRTOS.h must come first */
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/event_groups.h"
4  #include "freertos/queue.h"
5  #include "freertos/semphr.h"
6  #include "freertos/task.h"
7
8  /* WRONG - Other FreeRTOS headers before FreeRTOS.h */
9  #include "freertos/FreeRTOS.h"
10 #include "freertos/semphr.h" /* Error: FreeRTOS.h not included first */

```

9.94 System Includes

Use angle brackets for system and standard library headers.

```
1      /* Standard C library */
2  #include <inttypes.h>
3  #include <stdbool.h>
4  #include <stddef.h>
5  #include <stdint.h>
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <string.h>
```

9.95 Platform HAL Includes

Platform HAL headers (ESP-IDF, Renesas, etc.) use quotation marks and are grouped separately.

```
1      /* Platform-specific driver includes (example: ESP32) */
2  #include "driver/gpio.h"
3  #include "driver/i2c.h"
4  #include "driver/spi_master.h"
5  #include "driver/uart.h"
6
7      /* Platform system includes */
8      /* Platform-specific: RX72N uses rx_check.h */
9  #include "platform_system.h"
10 #include "rx_check.h"
11 #include "rx_err.h" /* Platform error types */
12 #include "rx_log.h" /* Platform logging */
13
14      /* Platform peripheral includes (if needed) */
15 #include "esp_adc/adc_oneshot.h"
16 #include "platform_adc.h"
17
18      /* Configuration */
19 #include "sdkconfig.h"
```

9.96 Project Includes

Use quotation marks for project headers. Alphabetize within each group.

```
1      /* Project includes - alphabetized */
2  #include "star_bus_adc.h"
3  #include "star_bus_config.h"
4  #include "star_bus_gpio.h"
5  #include "star_bus_i2c.h"
6  #include "star_bus_manager.h"
7  #include "star_bus_spi.h"
8  #include "star_bus_types.h"
9  #include "star_error_interface.h"
10 #include "star_pin_interface.h"
```

9.97 Header File Includes

Header files should minimize includes and use forward declarations when possible.

```

1      /* star_bus_manager.h - Header file includes */
2  #ifndef STAR_COMPONENT_BUS_MANAGER_H
3  #define STAR_COMPONENT_BUS_MANAGER_H
4
5  #ifdef __cplusplus
6  extern "C" {
7  #endif
8
9      /* Only include what's needed for declarations */
10 #include "esp_err.h"
11 #include "star_bus_manager_types.h"
12      /* Use forward declarations for pointer-only usage */
13      struct star_bus_config; /* Don't include star_bus_config.h */
14
15      /* Function using forward-declared type */
16      rx_err_t star_bus_manager_add_bus(star_bus_manager_t * manager,
17                                       struct star_bus_config * config);
18
19 #ifdef __cplusplus
20 }
21 #endif
22
23 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

9.98 Conditional Includes

Use `#ifdef` for platform-specific or optional includes.

```

1      /* Platform-specific includes */
2  #ifdef CONFIG_IDF_TARGET_ESP32S3
3  #include "esp32s3/rom/gpio.h"
4  #else
5  #include "esp32/rom/gpio.h"
6  #endif
7
8      /* Optional feature includes */
9  #ifdef CONFIG_STAR_ENABLE_DEBUG_LOGGING
10 #include "esp_debug_helpers.h"
11 #endif

```

9.99 Include Comments

Add comments for non-obvious includes explaining why they are needed.

```

1  #include "star_bus_manager.h"
2
3  #include "freertos/FreeRTOS.h"
4
5  #include <inttypes.h>
6  #include <string.h>

```

```

7      /* Platform-specific: RX72N uses rx_check.h */
8
9  #include "esp_log.h"
10 #include "rx_check.h"
11 #include "star_bus_manager.h" /* Needed for star_bus_manager_find_bus */
12 #include "star_bus_types.h"  /* Needed for star_bus_config_t and union */

```

9.100 Avoiding Circular Dependencies

Structure your includes to avoid circular dependencies.

```

1  /* Problem: Circular dependency */
2  /* star_bus_manager.h includes star_bus_config.h */
3  /* star_bus_config.h includes star_bus_manager.h */
4
5  /* Solution: Use forward declarations */
6  /* star_bus_config.h */
7  struct star_bus_manager; /* Forward declare, don't include */
8
9  rx_err_t star_bus_config_init(star_bus_config_t* config,
10 struct star_bus_manager* manager);
11
12 /* star_bus_config.c - Include full header here */
13 #include "star_bus_config.h"
14 #include "star_bus_manager.h" /* Full definition needed in .c */

```

9.101 Summary

Order	Category	Style
1	Primary header (.c files)	"..."
2	FreeRTOS	"freertos/..."
3	Standard library	<...>
4	Platform HAL	"..."
5	Project	"..."

- Always include **FreeRTOS.h** before other FreeRTOS headers
- Separate groups with blank lines
- Alphabetize within each group
- Minimize includes in headers; use forward declarations
- Add comments for non-obvious include dependencies

This project follows the rule of **2 spaces** for indentation, without using tabs.

Unix-style line endings ('LF', represented as '\n') are mandatory.

Logging is handled using platform-specific logging macros (e.g., **RX_LOG_*** for STAR, **ESP_LOG*** for ESP-IDF compatibility layer).

- **RX_LOG_ERROR** / **ESP_LOGE** - Error

- **RX_LOG_WARN / ESP_LOGW** - Warning
- **RX_LOG_INFO / ESP_LOGI** - Info
- **RX_LOG_DEBUG / ESP_LOGD** - Debug
- **RX_LOG_VERBOSE / ESP_LOGV** - Verbose

This section establishes comprehensive naming standards for all identifiers in the STAR firmware codebase. Consistent naming improves readability, maintainability, and reduces cognitive load when navigating the codebase.

9.102 General Rules

- **Language:** All identifiers must be in English.
- **Clarity:** Names should be descriptive and self-documenting.
- **Abbreviations:** Avoid abbreviations unless they are widely understood (e.g., GPIO, I2C, SPI).

9.103 Functions and Variables

Functions and variables use `snake_case`—all lowercase with words separated by underscores.

```

1  /* CORRECT - snake_case for functions and variables */
2  esp_err_t star_bus_manager_init(star_bus_manager_t* manager, const char*
   tag);
3  uint32_t  error_count;
4  size_t    bytes_written;
5  bool      is_initialized;
6
7  /* WRONG - other naming styles */
8  rx_err_t  StarBusManagerInit();    /* PascalCase */
9  rx_err_t  starBusManagerInit();    /* camelCase */
10 uint32_t  ErrorCount;               /* PascalCase */

```

9.104 Module Prefixes

All public functions must use the module prefix pattern: `star_[module]_[function]`.

```

1  /* From star_bus_manager.h */
2  rx_err_t star_bus_manager_init(...);
3  rx_err_t star_bus_manager_add_bus(...);
4  rx_err_t star_bus_manager_remove_bus(...);
5  rx_err_t star_bus_manager_deinit(...);
6
7  /* From star_bus_config.h */
8  star_bus_config_t* star_bus_config_create_i2c(...);
9  star_bus_config_t* star_bus_config_create_spi_device(...);
10 rx_err_t          star_bus_config_destroy(...);
11
12 /* From star_bus_i2c.h */
13 rx_err_t star_bus_i2c_write(...);

```



```

14 rx_err_t star_bus_i2c_read(...);
15 void      star_bus_i2c_init_default_ops(...);
16
17 /* From star_error_handler.h */
18 rx_err_t error_handler_init(...);
19 rx_err_t error_handler_record_error(...);
20 bool     error_handler_can_retry(...);

```

9.105 Constants and Macros

Constants and macros use `SCREAMING_SNAKE_CASE`—all uppercase with words separated by underscores.

```

1 /* CORRECT - SCREAMING_SNAKE_CASE for macros and constants */
2 #define STAR_BUS_NAME_MAX_LEN (32)
3 #define CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS (1000)
4 #define I2C_MASTER_WRITE 0
5 #define I2C_MASTER_READ 1
6 #define GPIO_NUM_MAX 48
7
8 /* Convenience macro with proper naming */
9 #define RECORD_ERROR(handler, code, desc)
10     \
11     do {
12         \
13         error_handler_record_error((handler), (code), (desc), __FILE__,
14             \
15             __LINE__, __func__);
16     } while (0)
17
18 #define STAR_IFACE_RECORD_ERROR(iface, err, msg)
19     \
20     ((iface) && (iface)->record_error
21         \
22         ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
23             \
24             __LINE__, __func__)
25         : (err))
26
27 /* WRONG */
28 #define star_bus_name_max_len 32 /* lowercase */
29 #define StarBusNameMaxLen 32 /* PascalCase */

```

9.106 Type Names

All custom types must use `snake_case` with the `_t` suffix.

```

1 /* CORRECT - snake_case_t for type names */
2 typedef struct star_bus_config star_bus_config_t;
3 typedef struct star_bus_manager star_bus_manager_t;

```

```

4 typedef struct error_handler    error_handler_t;
5 typedef enum star_bus_type      star_bus_type_t;
6
7 /* Callback function types also use _t suffix */
8 typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus, void*
   user_ctx);
9 typedef rx_err_t (*pin_registration_function_t)(gpio_num_t pin_num,
10 const char* bus_name,
11 const char* pin_usage,
12 void* user_ctx);
13
14 /* WRONG */
15 typedef struct StarBusConfig StarBusConfig; /* PascalCase */
16 typedef struct star_bus_config star_bus_config; /* missing _t suffix */

```

9.107 Enum Members

Enum members use the `k_` prefix followed by `snake_case`.

```

1 /* CORRECT - k_prefix with snake_case, C23 typed enum */
2 typedef enum : uint8_t {
3     k_star_bus_type_none,    /**< No bus type specified */
4     k_star_bus_type_i2c,    /**< I2C bus */
5     k_star_bus_type_spi,    /**< SPI bus */
6     k_star_bus_type_gpio,   /**< GPIO bus */
7     k_star_bus_type_adc,    /**< ADC bus */
8     k_star_bus_type_count,  /**< Number of bus types (sentinel) */
9 } star_bus_type_t;
10
11 /* Usage in switch statements */
12 switch (config->type) {
13     case k_star_bus_type_i2c: {
14         /* Handle I2C */
15         break;
16     }
17     case k_star_bus_type_spi: {
18         /* Handle SPI */
19         break;
20     }
21     default: {
22         RX_LOG_WARN(s_TAG, "Unknown bus type: %d", config->type);
23         break;
24     }
25 }
26
27 /* WRONG */
28 typedef enum {
29     STAR_BUS_TYPE_I2C,    /* SCREAMING_SNAKE_CASE without k_ */
30     StarBusTypeI2c,      /* PascalCase */
31     starBusTypeI2c,      /* camelCase */
32 } bad_enum_t;

```

9.108 Static Variables (File-Scoped)

Static file-scoped variables use the `s_` prefix followed by `snake_case`.

```

1  /* CORRECT - s_ prefix for static file-scoped variables */
2  static const char* s_TAG = "STAR_BUS_MANAGER";
3  static const uint32_t s_i2c_timeout_ms = 1000;
4
5  /* From star_bus_i2c.c */
6  static const char* s_TAG = "STAR_BUS_I2C";
7  static const uint32_t s_i2c_timeout_ms = 1000;
8
9  /* From star_bus_config.c */
10 static const char* s_TAG = "STAR_BUS_CONFIG";
11
12 /* WRONG - missing s_ prefix */
13 static const char* TAG = "MODULE";
14 static uint32_t timeout_ms = 1000;

```

9.109 Global Variables

Global variables use the `g_` prefix followed by `snake_case`. However, global variables are **strongly discouraged**. Prefer dependency injection or static file-scoped variables.

```

1  /* CORRECT (but discouraged) - g_ prefix for globals */
2  g_system_state_t g_system_state;
3  uint32_t g_error_count;
4
5  /* PREFERRED - use dependency injection instead */
6  typedef struct {
7      star_error_interface_t *error_iface; /* Injected */
8      star_pin_interface_t *pin_iface;     /* Injected */
9  } star_bus_manager_t;
10
11 /* Initialize with injected dependencies */
12 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
13 const char* tag,
14 star_error_interface_t* error_iface,
15 star_pin_interface_t* pin_iface);

```

9.110 Private Functions

9.110.1 File-Scoped Static Functions (internal_ prefix)

Static functions that are internal to a single source file use the `internal_` prefix.

```

1  /* From star_bus_manager.c - internal helper functions */
2  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
3  gpio_num_t pin,
4  const char* bus_name,
5  const char* usage,
6  bool can_be_shared)
7  {
8      if (pin < 0 || pin >= GPIO_NUM_MAX) {

```

```

9         return RX_OK; /* Not an error, pin is not used */
10    }
11    /* ... implementation ... */
12 }
13
14 static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
15     pin_iface,
16     gpio_num_t      pin,
17     const char*      bus_name,
18     const char*      usage)
19 {
20     /* ... implementation ... */
21 }
22
23 static rx_err_t internal_register_config_pins(star_pin_interface_t*
24     pin_iface,
25     const star_bus_config_t* config)
26 {
27     /* ... implementation ... */
28 }
29
30 /* From star_bus_i2c.c */
31 static rx_err_t internal_star_bus_i2c_write(const star_bus_config_t*
32     config,
33     const uint8_t*      data,
34     size_t              len,
35     uint8_t             reg_addr,
36     size_t*             bytes_written);
37
38 static rx_err_t internal_i2c_execute_with_retry(const star_bus_config_t*
39     config,
40     i2c_port_t          port,
41     i2c_cmd_handle_t     cmd,
42     const char*          operation_name);

```

9.110.2 Exposed Private Functions (priv_ prefix)

Functions that must be exposed across files within a module (but are not part of the public API) use the `priv_` prefix.

```

1  /* In private header (e.g., star_bus_internal.h) */
2  /* These functions are used by multiple .c files in the same module
3  but should not be called by external code */
4
5  rx_err_t priv_star_bus_validate_config(const star_bus_config_t* config);
6  rx_err_t priv_star_bus_acquire_mutex(star_bus_manager_t* manager);
7  void      priv_star_bus_release_mutex(star_bus_manager_t* manager);
8
9  /* Used in implementation files within the module */
10 rx_err_t star_bus_config_init(star_bus_config_t* config,
11 struct star_bus_manager* manager)
12 {
13     /* Validate using private helper */

```

```

14     esp_err_t ret = priv_star_bus_validate_config(config);
15     if (ret != RX_OK) {
16         return ret;
17     }
18     /* ... rest of implementation ... */
19 }

```

9.111 Summary Table

Identifier Type	Convention	Example
Functions	snake_case	star_bus_manager_init
Variables	snake_case	bytes_written
Constants/Macros	SCREAMING_SNAKE_CASE	STAR_BUS_NAME_MAX_LEN
Type names	snake_case_t	star_bus_config_t
Enum members	k_snake_case	k_star_bus_type_i2c
Static variables	s_snake_case	s_TAG
Global variables	g_snake_case	g_system_state
Internal functions	internal_snake_case	internal_register_bus_pin
Private functions	priv_snake_case	priv_star_bus_validate_config
Module prefix	star_[module]_	star_bus_, star_error_

This section describes the patterns and conventions for defining custom types in the STAR firmware codebase. Consistent type definition patterns improve code readability and enable better tooling support.

9.112 Typedef Struct Pattern

All structures should be defined using the typedef pattern with the `_t` suffix.

9.112.1 Standard Pattern

```

1  /* Forward declaration (in header for dependencies) */
2  typedef struct star_bus_config star_bus_config_t;
3  typedef struct star_bus_manager star_bus_manager_t;
4
5  /* Full definition (in header or implementation) */
6  typedef struct star_bus_config {
7      const char *name;      /**< Unique name for this bus/device instance
8          */
9      star_bus_type_t type; /**< Type of bus (I2C, SPI, etc.) */
10     bool initialized;      /**< Whether this bus has been initialized */
11     void *handle;          /**< Driver/device handle */
12     void *user_ctx;        /**< User context pointer for callbacks */
13
14     /* Protocol-specific configuration (see union section) */
15     union {
16         star_i2c_bus_config_t i2c;
17         star_spi_bus_config_t spi;
18         star_gpio_bus_config_t gpio;
19         star_adc_bus_config_t adc;

```

```

19     } proto;
20
21     struct star_bus_config *next; /**< Linked list pointer */
22 } star_bus_config_t;

```

9.112.2 Error Handler Example

```

1 /* From star_error_handler.h */
2 typedef struct error_handler {
3     uint32_t error_count; /**< Total number of errors recorded
4     */
5     uint32_t max_retries; /**< Maximum retry attempts allowed */
6     uint32_t current_retry; /**< Current retry attempt counter */
7     uint32_t base_retry_delay; /**< Base delay between retries (ms)
8     */
9     uint32_t max_retry_delay; /**< Maximum retry delay (ms) */
10    uint32_t current_retry_delay; /**< Current delay with backoff */
11    rx_err_t last_error; /**< Last recorded error code */
12    bool in_error_state; /**< Whether in error state */
13    void *reset_context; /**< Context for reset function */
14    SemaphoreHandle_t mutex; /**< Mutex for thread-safety */
15
16    rx_err_t (*reset_fn)(void *context); /**< Optional reset function */
17 } error_handler_t;

```

9.113 Union Usage for Protocol Configurations

Unions are used to store protocol-specific configurations efficiently. This pattern allows a single configuration structure to support multiple bus types.

```

1 /* From star_bus_manager_types.h */
2 typedef struct star_bus_config {
3     const char *name;
4     star_bus_type_t type; /* Discriminator for the union */
5     bool initialized;
6     void *handle;
7     void *user_ctx;
8
9     /* Union holds protocol-specific configuration */
10    union {
11        star_i2c_bus_config_t i2c; /**< I2C-specific configuration */
12        star_spi_bus_config_t spi; /**< SPI-specific configuration */
13        star_gpio_bus_config_t gpio; /**< GPIO-specific configuration */
14        star_adc_bus_config_t adc; /**< ADC-specific configuration */
15    } proto;
16
17    struct star_bus_config *next;
18 } star_bus_config_t;
19
20 /* Accessing union members based on type discriminator */
21 switch (config->type) {
22     case k_star_bus_type_i2c: {

```

```

23     i2c_port_t port = config->proto.i2c.port;
24     uint8_t address = config->proto.i2c.address;
25     /* ... */
26     break;
27 }
28 case k_star_bus_type_spi: {
29     spi_host_device_t host = config->proto.spi.host;
30     gpio_num_t copi_pin = config->proto.spi.copi_io_num;
31     /* ... */
32     break;
33 }
34 /* ... other cases ... */
35 }

```

9.113.1 Protocol - Specific Structures

```

1  /* I2C bus configuration (from star_bus_protocol_types.h) */
2  typedef struct star_i2c_bus_config {
3      i2c_port_t port;                /**< I2C port number */
4      i2c_config_t config;           /**< Platform I2C configuration */
5      uint8_t address;               /**< 7-bit I2C device address */
6      star_i2c_callbacks_t callbacks; /**< User callbacks */
7      star_i2c_ops_t ops;            /**< Operation function pointers */
8  } star_i2c_bus_config_t;
9
10 /* SPI bus configuration */
11 typedef struct star_spi_bus_config {
12     spi_host_device_t host;          /**< SPI host number */
13     bool is_peripheral;              /**< True if SPI peripheral
14                                     mode */
15     spi_bus_config_t bus_cfg;        /**< Platform SPI bus config
16                                     */
17     spi_device_interface_config_t dev_cfg; /**< Device config */
18     star_spi_callbacks_t callbacks;
19     star_spi_ops_t ops;
20
21     /* Inclusive terminology pin names (see Terminology section) */
22     gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
23     gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
24     gpio_num_t sclk_io_num; /**< Serial Clock */
25     gpio_num_t cs_io_num; /**< Chip Select */
26     /* ... */
27 } star_spi_bus_config_t;

```

9.114 Callback Function Types

Callback function pointer types follow a consistent pattern with the `_t` suffix.

```

1  /* From star_bus_common_types.h - Pin registration callback */
2  typedef rx_err_t (*pin_registration_function_t)(gpio_num_t pin_num,
3  const char* bus_name,
4  const char* pin_usage,

```

```

5 void*          user_ctx);
6
7 /* From star_bus_manager.h - Bus access callback */
8 typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
9 void*          user_ctx);
10
11 /* From star_bus_function_types.h - I2C operation callbacks */
12 typedef rx_err_t (*star_i2c_write_fn_t)(const star_bus_config_t* config,
13 const uint8_t*      data,
14 size_t              len,
15 uint8_t             reg_addr,
16 size_t*             bytes_written);
17
18 typedef rx_err_t (*star_i2c_read_fn_t)(const star_bus_config_t* config,
19 uint8_t*          data,
20 size_t            len,
21 uint8_t           reg_addr,
22 size_t*           bytes_read);
23
24 /* Event callbacks */
25 typedef void (*star_i2c_transfer_complete_cb_t)(const star_bus_event_t*
26 event,
27 void*          user_ctx);
28
29 typedef void (*star_i2c_error_cb_t)(const star_bus_event_t* event,
30 rx_err_t          error,
31 void*          user_ctx);
32
33 /* Grouping related callbacks in a struct */
34 typedef struct star_i2c_callbacks {
35     star_i2c_transfer_complete_cb_t on_transfer_complete;
36     star_i2c_error_cb_t on_error;
37 } star_i2c_callbacks_t;

```

9.115 Interface Patterns (Dependency Injection)

Interfaces use structs with function pointers and an opaque context pointer. This pattern enables dependency injection and testability.

```

1 /* From star_error_interface.h */
2 typedef struct star_error_interface {
3     /**
4      * @brief Record an error
5      * @param ctx Implementation context
6      * @param error Error code
7      * @param message Error message
8      * @param file Source file
9      * @param line Line number
10     * @param func Function name
11     * @return ESP_OK on success
12     */
13    rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
14    *message,

```



```

14         const char *file, int line, const char
15             *func);
16
17     /**
18     * @brief Check if retry is possible
19     * @param ctx Implementation context
20     * @return true if retry is allowed
21     */
22     bool (*can_retry)(void *ctx);
23
24     /**
25     * @brief Reset error state
26     * @param ctx Implementation context
27     * @return ESP_OK on success
28     */
29     rx_err_t (*reset_state)(void *ctx);
30
31     /**
32     * @brief Implementation context (opaque pointer)
33     */
34     void *ctx;
35 } star_error_interface_t;
36
37 /* From star_pin_interface.h */
38 typedef struct star_pin_interface {
39     rx_err_t (*register_pin)(void *ctx, int pin, const char
40         *description,
41         bool shared);
42     rx_err_t (*unregister_pin)(void *ctx, int pin, const char
43         *description);
44     rx_err_t (*validate)(void *ctx);
45     void *ctx;
46 } star_pin_interface_t;

```

9.115.1 Interface Usage Pattern

```

1  /* Creating an interface from concrete implementation */
2  error_handler_t error_handler;
3  error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
4
5  star_error_interface_t error_iface;
6  error_handler_get_interface(&error_iface, &error_handler);
7
8  /* Injecting interfaces into components */
9  star_bus_manager_t bus_manager;
10 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
11
12 /* Using interface through function pointers */
13 if (error_iface.can_retry(error_iface.ctx)) {
14     /* Retry operation */
15 }
16
17 /* Or using the convenience macro */

```

```

18 STAR_IFACE_RECORD_ERROR(&error_iface, ESP_ERR_TIMEOUT, "Operation timed
    out");

```

9.116 Opaque Handle Patterns

For complex objects that should not be directly manipulated, use opaque handles.

```

1  /* Forward declaration creates an opaque type */
2  typedef struct star_bus_manager star_bus_manager_t;
3
4  /* Public API only uses pointers to the opaque type */
5  esp_err_t star_bus_manager_init(star_bus_manager_t* manager, ...);
6  rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager, ...);
7
8  /* Full definition is only in the .c file or private header */
9  /* Users cannot access internal fields directly */
10
11 /* Example usage - users allocate storage but don't access internals */
12 star_bus_manager_t bus_manager; /* Stack allocation */
13 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
14
15 /* Or heap allocation */
16 star_bus_manager_t* manager_ptr = malloc(sizeof(star_bus_manager_t));
17 star_bus_manager_init(manager_ptr, "main", &error_iface, &pin_iface);

```

9.117 Operations Structure Pattern

Group related function pointers in an operations structure for pluggable implementations.

```

1  /* From star_bus_protocol_types.h */
2  typedef struct star_i2c_ops {
3      star_i2c_write_fn_t write; /**< Write with register
4      */
5      star_i2c_read_fn_t read; /**< Read with register
6      */
7      star_i2c_write_command_fn_t write_command; /**< Write command byte
8      */
9      star_i2c_read_raw_fn_t read_raw; /**< Read raw bytes */
10 } star_i2c_ops_t;
11
12 typedef struct star_spi_ops {
13     star_spi_transmit_fn_t transmit; /**< Transmit only */
14     star_spi_receive_fn_t receive; /**< Receive only */
15     star_spi_transfer_fn_t transfer; /**< Full-duplex transfer */
16 } star_spi_ops_t;
17
18 typedef struct star_gpio_ops {
19     rx_err_t (*write_digital)(const struct star_bus_config *config,
20                             uint8_t pin_index, uint8_t value);
21     rx_err_t (*read_digital)(const struct star_bus_config *config,
22                             uint8_t pin_index, uint8_t *value);
23 } star_gpio_ops_t;
24
25

```

```

22  /* Initializing default operations */
23  void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
24  {
25      if (ops == NULL) {
26          RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
27          return;
28      }
29      ops->write = internal_star_bus_i2c_write;
30      ops->read = internal_star_bus_i2c_read;
31      ops->write_command = internal_star_bus_i2c_write_command;
32      ops->read_raw = internal_star_bus_i2c_read_raw;
33  }

```

9.118 Manager Structure Pattern

Manager structures coordinate multiple resources with thread-safe access.

```

1  /* From star_bus_manager_types.h */
2  typedef struct star_bus_manager {
3      star_bus_config_t *buses;           /**< Linked list of bus
        configs */
4      SemaphoreHandle_t mutex;           /**< Mutex for thread-safety */
5      const char *tag;                   /**< Logging tag */
6      star_error_interface_t *error_iface; /**< Injected error handler */
7      star_pin_interface_t *pin_iface;    /**< Injected pin validator */
8
9      /* Resource tracking */
10     bool spi_host_initialized[SPI_HOST_MAX];
11     int spi_device_count[SPI_HOST_MAX];
12 } star_bus_manager_t;

```

9.119 Best Practices

- **Always use `_t` suffix:** Makes types easily distinguishable from variables.
- **Use forward declarations:** Minimize header dependencies and compilation time.
- **Document all fields:** Use Doxygen comments for struct members.
- **Keep unions discriminated:** Always pair unions with a type field.
- **Prefer static const over `#define`:** For typed constants within structures.
- **Use opaque types for encapsulation:** Hide implementation details from users.
- **Group related function pointers:** Use ops structs for pluggable implementations.

This section describes the standard file and directory structure for the STAR firmware codebase. Consistent organization improves navigability and maintainability.

9.120 Library Directory Structure

Each library follows a standard component structure (PlatformIO/ESP-IDF on ESP32, CMake on RX72N):

```
lib/star_component/  
  CMakeLists.txt      # CMake build configuration  
  library.json        # PlatformIO library metadata  
  include/            # Public headers (API)  
    star_component.h  
    star_component_types.h  
  src/                # Implementation files  
    star_component.c
```

9.120.1 Example: star_bus Library

```
lib/star_bus/  
  CMakeLists.txt  
  library.json  
  include/  
    star_bus_adc.h  
    star_bus_common_types.h  
    star_bus_config.h  
    star_bus_event_types.h  
    star_bus_function_types.h  
    star_bus_gpio.h  
    star_bus_i2c.h  
    star_bus_manager.h  
    star_bus_manager_types.h  
    star_bus_protocol_types.h  
    star_bus_spi.h  
    star_bus_types.h  
  src/  
    star_bus_adc.c  
    star_bus_config.c  
    star_bus_gpio.c  
    star_bus_i2c.c  
    star_bus_manager.c  
    star_bus_spi.c  
    star_bus_types.c
```

9.121 CMakeLists.txt Structure

```
1 #lib / star_bus / CMakeLists.txt  
2  
3 idf_component_register(  
4   SRCS  
5     "src/star_bus_adc.c"  
6     "src/star_bus_config.c"
```

```

7 "src/star_bus_gpio.c"
8 "src/star_bus_i2c.c"
9 "src/star_bus_manager.c"
10 "src/star_bus_spi.c"
11 "src/star_bus_types.c"
12 INCLUDE_DIRS
13 "include"
14 REQUIRES
15 driver
16 esp_adc
17 freertos
18 star_core
19 )

```

9.122 Header File Organization

Header files must follow a consistent internal structure.

9.122.1 Header File Template

```

1      /* lib/star_bus/include/star_bus_manager.h */
2
3 #ifndef STAR_COMPONENT_BUS_MANAGER_H
4 #define STAR_COMPONENT_BUS_MANAGER_H
5
6 #ifdef __cplusplus
7 extern "C" {
8 #endif
9
10     /* === Includes === */
11 #include "esp_err.h"
12 #include "star_bus_manager_types.h"
13 /**
14  * @file star_bus_manager.h
15  * @brief Unified bus manager for I2C, SPI, GPIO, and other bus
16  *        protocols
17  *
18  * The bus manager provides a central registry for managing multiple
19  * bus configurations. It supports dependency injection of error
20  * handlers
21  * and pin validators for loose coupling.
22  *
23  * Key Features:
24  * - Thread-safe bus management with mutex protection
25  * - Support for multiple bus types (I2C, SPI, GPIO, etc.)
26  * - Automatic pin registration and conflict detection
27  * - Safe bus access via callback pattern
28  *
29  * Example Usage:
30  * @code
31  * // Initialize bus manager with dependency injection
32  * star_bus_manager_t bus_manager;

```

```

31     * star_bus_manager_init(&bus_manager, "main", &error_iface,
32         &pin_iface);
33     *
34     * // Create and add an I2C bus
35     * star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
36         "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22,
37         400000);
38     * star_bus_manager_add_bus(&bus_manager, i2c_bus);
39     * @endcode
40     */
41     /* === Forward Declarations === */
42     /* (if needed beyond what's in included headers) */
43
44     /* === Public Functions === */
45
46     /**
47     * @brief Initialize a bus manager instance.
48     *
49     * @param[out] manager      Pointer to bus manager to initialize.
50     *                          Must not
51     *                          be NULL.
52     * @param[in]  tag          Optional tag string for logging. NULL
53     *                          uses
54     *                          default.
55     * @param[in]  error_iface  Optional error handler interface (can be
56     *                          NULL).
57     * @param[in]  pin_iface    Optional pin validator interface (can be
58     *                          NULL).
59     * @return rx_err_t RX_OK on success, RX_ERR_INVALID_ARG if manager
60     *         is
61     *         NULL, ESP_ERR_NO_MEM if mutex creation fails.
62     */
63     rx_err_t star_bus_manager_init(
64         star_bus_manager_t * manager, const char *tag,
65         star_error_interface_t *error_iface, star_pin_interface_t
66         *pin_iface);
67
68     /* ... more function declarations ... */
69
70 #ifdef __cplusplus
71 }
72 #endif
73
74 #endif /* STAR_COMPONENT_BUS_MANAGER_H */

```

9.122.2 Header Section Order

1. **File path comment:** Identifies the file location
2. **Include guard:** `#ifndef STAR_COMPONENT_FILENAME_H`
3. **C++ extern “C” opening:** For C++ compatibility

4. **Includes:** System, then platform HAL, then project headers
5. **File documentation:** @file Doxygen block with overview
6. **Forward declarations:** If needed
7. **Constants and macros:** Public defines
8. **Type definitions:** Enums, structs, typedefs
9. **Function declarations:** Grouped by category
10. **C++ extern “C” closing:** Matches opening
11. **Include guard closing:** #endif /* ... */

9.123 Include Guard Naming Convention

Include guards follow the pattern: `STAR_COMPONENT_FILENAME_H`

```

1      /* star_bus_manager.h */
2 #ifndef STAR_COMPONENT_BUS_MANAGER_H
3 #define STAR_COMPONENT_BUS_MANAGER_H
4      /* ... */
5 #endif /* STAR_COMPONENT_BUS_MANAGER_H */
6
7      /* star_bus_config.h */
8 #ifndef STAR_COMPONENT_BUS_CONFIG_H
9 #define STAR_COMPONENT_BUS_CONFIG_H
10     /* ... */
11 #endif /* STAR_COMPONENT_BUS_CONFIG_H */
12
13     /* star_error_interface.h */
14 #ifndef STAR_ERROR_INTERFACE_H
15 #define STAR_ERROR_INTERFACE_H
16     /* ... */
17 #endif /* STAR_ERROR_INTERFACE_H */
18
19     /* star_pin_interface.h */
20 #ifndef STAR_PIN_INTERFACE_H
21 #define STAR_PIN_INTERFACE_H
22     /* ... */
23 #endif /* STAR_PIN_INTERFACE_H */

```

9.124 Source File Organization

Source files follow a consistent internal structure.

9.124.1 Source File Template

```

1      /* lib/star_bus/src/star_bus_manager.c */
2
3      /* === Primary Header === */
4 #include "star_bus_manager.h"

```

```

5
6      /* === System Includes === */
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/semphr.h"
9
10 #include <stdio.h>
11 #include <stdlib.h>
12 #include <string.h>
13
14      /* === Platform HAL Includes === */
15      /* Platform-specific: RX72N uses rx_check.h */
16  #include "esp_log.h"
17  #include "rx_check.h"
18  #include "sdkconfig.h"
19
20      /* === Project Includes === */
21  #include "star_bus_adc.h"
22  #include "star_bus_config.h"
23  #include "star_bus_gpio.h"
24
25  /* === Constants === */
26
27  static const char* s_TAG = "STAR_BUS_MANAGER";
28
29  /* === Private Function Prototypes === */
30
31  static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
32  gpio_num_t                pin,
33  const char*                bus_name,
34  const char*                usage,
35  bool                       can_be_shared);
36
37  static rx_err_t internal_unregister_bus_pin(star_pin_interface_t*
38  pin_iface,
39  gpio_num_t                pin,
40  const char*                bus_name,
41  const char*                usage);
42
43  /* === Public Functions === */
44
45  rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
46  const char*                tag,
47  star_error_interface_t* error_iface,
48  star_pin_interface_t* pin_iface)
49  {
50      RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
51                          "Manager pointer is NULL");
52
53      manager->buses = NULL;
54      manager->error_iface = error_iface;
55      manager->pin_iface = pin_iface;
56
57      /* ... rest of implementation ... */

```



```

58     return RX_OK;
59 }
60
61 /* ... more public functions ... */
62
63 /* === Private Functions === */
64
65 static rx_err_t internal_register_bus_pin(star_pin_interface_t* pin_iface,
66 gpio_num_t          pin,
67 const char*         bus_name,
68 const char*         usage,
69 bool                can_be_shared)
70 {
71     if (pin < 0 || pin >= GPIO_NUM_MAX) {
72         return RX_OK; /* Not an error, pin is not used */
73     }
74     /* ... implementation ... */
75 }

```

9.124.2 Source Section Order

1. **File path comment:** Identifies the file location
2. **Primary header:** The corresponding .h file for this .c file
3. **System includes:** FreeRTOS, standard library (<...>)
4. **Platform HAL includes:** Platform-specific components
5. **Project includes:** Other project headers ("...")
6. **Constants:** static const variables, including s_TAG
7. **Private function prototypes:** Forward declarations of static functions
8. **Public functions:** Implementation of API functions
9. **Private functions:** Implementation of static helper functions

9.125 Types Header Separation

For complex modules, separate type definitions into dedicated headers:

```

star_bus/include/
    star_bus_manager.h          # Main API (functions)
    star_bus_manager_types.h    # Types for manager
    star_bus_common_types.h     # Shared types (enums, forward decls)
    star_bus_protocol_types.h   # Protocol-specific types
    star_bus_function_types.h   # Function pointer types
    star_bus_event_types.h      # Event structures
    star_bus_types.h            # Controller header (includes all)

```

9.125.1 Controller Header Pattern

```

1      /* star_bus_types.h - includes all type headers */
2  #ifndef STAR_COMPONENT_BUS_TYPES_H
3  #define STAR_COMPONENT_BUS_TYPES_H
4
5  #ifdef __cplusplus
6  extern "C" {
7  #endif
8
9      /* Controller include file for all public types */
10 #include "star_bus_common_types.h"
11 #include "star_bus_event_types.h"
12 #include "star_bus_function_types.h"
13 #include "star_bus_manager_types.h"
14 #include "star_bus_protocol_types.h"
15
16 #ifdef __cplusplus
17 }
18 #endif
19
20 #endif /* STAR_COMPONENT_BUS_TYPES_H */

```

9.126 Project Root Structure

```

star-esp32-firmware/
  CLAUDE.md          # Claude Code instructions
  platformio.ini      # PlatformIO configuration
  sdkconfig.defaults  # ESP-IDF default config
  lib/               # Libraries/components
    star_core/        # Core interfaces
    star_bus/         # Bus abstraction
    star_error_handler/
    star_pin_validator/
    star_sensor_*/    # Sensor drivers
    star_motor/
    star_pid/
  src/               # Main application
    main.c
  partitions/        # Partition tables
    partitions_4mb.csv
    partitions_16mb.csv
  docs/             # Documentation
    style_guide.tex
    style_guide_sections/
  scripts/          # Build/utility scripts
  test/             # Test files

```

9.127 Best Practices

- **One header per component:** Each component has its primary public header.
- **Minimize header dependencies:** Use forward declarations to reduce includes.
- **Self-contained headers:** Each header should compile independently.
- **Consistent naming:** Files match the component they contain.
- **Group related files:** Keep related headers and sources together.
- **Separate types from functions:** For complex modules, split into `_types.h`.
- **Document file purpose:** Use `@file` Doxygen blocks.

This section establishes the OSHWA (Open Source Hardware Association) inclusive terminology standards used throughout the STAR firmware codebase. These standards replace legacy terminology with more inclusive alternatives.

9.128 Overview

The STAR project follows inclusive terminology guidelines to ensure our codebase is welcoming to all contributors and avoids language with harmful historical connotations. This is particularly relevant in embedded systems where certain legacy terms have been deeply entrenched.

9.129 SPI Bus Terminology

9.129.1 COPI / CIPO(Not MOSI / MISO)

STAR Term	Legacy Term	Description
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
SCLK	SCLK	Serial Clock (unchanged)
CS	SS	Chip Select (unchanged)

```

1  /* CORRECT - OSHWA terminology */
2  typedef struct star_spi_bus_config {
3      gpio_num_t copi_io_num; /**< Controller Out, Peripheral In */
4      gpio_num_t cipo_io_num; /**< Controller In, Peripheral Out */
5      gpio_num_t sclk_io_num; /**< Serial Clock */
6      gpio_num_t cs_io_num;   /**< Chip Select */
7      /* ... */
8  } star_spi_bus_config_t;
9
10 /* Factory function uses inclusive naming */
11 star_bus_config_t* star_bus_config_create_spi_device(
12     const char*      name,
13     spi_host_device_t host,
14     gpio_num_t       copi_pin,   /* NOT mosi_pin */
15     gpio_num_t       cipo_pin,   /* NOT miso_pin */
16     gpio_num_t       sclk_pin,
17     int32_t           dma_chan,

```

```

18  const spi_device_interface_config_t* dev_cfg);
19
20  /* In documentation and comments */
21  /* CORRECT */
22  reg_result = register_pin_if_valid(curr->proto.spi.copi_io_num,
23  bus_name,
24  "SPI COPI", /* Descriptive usage */
25  reg_func,
26  user_ctx,
27  true);
28
29  /* WRONG */
30  reg_result = register_pin_if_valid(curr->proto.spi.mosi_io_num,
31  bus_name,
32  "SPI MOSI", /* Legacy terminology */
33  reg_func,
34  user_ctx,
35  true);

```

9.130 I2C and General Bus Terminology

9.130.1 Controller / Peripheral(Not Master / Slave)

STAR Term	Legacy Term	Description
Controller	Master	Device that initiates communication
Peripheral	Slave	Device that responds to controller
Primary	Master	For configuration structures
Main	Master	For general reference

```

1  /* CORRECT - Controller/Peripheral terminology */
2  typedef struct star_spi_bus_config {
3      spi_host_device_t host;
4      bool is_peripheral; /**< True if operating as SPI peripheral */
5      /* ... */
6  } star_spi_bus_config_t;
7
8  /* Factory function for peripheral mode */
9  star_bus_config_t* star_bus_config_create_spi_peripheral(
10  const char*      name,
11  spi_host_device_t host,
12  gpio_num_t      copi_pin,
13  gpio_num_t      cipo_pin,
14  gpio_num_t      sclk_pin,
15  gpio_num_t      cs_pin,
16  int32_t          queue_size,
17  uint8_t          mode);
18
19  /* In comments and documentation */
20  /**
21   * @brief I2C Controller operations
22   *
23   * The controller device initiates all transactions on the bus.

```

```

24  * Peripheral devices respond when addressed by the controller.
25  */
26
27  /* WRONG */
28  bool is_slave;           /* Use is_peripheral */
29  star_bus_config_create_spi_slave(); /* Use _peripheral */

```

9.131 Mapping Platform Legacy APIs

Some platform HALs (like ESP-IDF) still use legacy terminology internally. When interfacing with these APIs, document the mapping clearly.

```

1  /* From star_bus_protocol_types.h */
2  typedef struct star_spi_bus_config {
3      /* ... */
4
5      /* SPI pin naming follows OSHWA standard:
6      * - COPI (Controller Out, Peripheral In) - replaces MOSI
7      * - CIPO (Controller In, Peripheral Out) - replaces MISO
8      * This modern terminology removes master/slave language.
9      * Some platforms (like ESP-IDF) use legacy mosi/miso terminology,
10     * which we map here.
11     */
12     gpio_num_t
13         copi_io_num; /**< GPIO for COPI (maps to platform mosi_io_num)
14         */
15     gpio_num_t
16         cipo_io_num; /**< GPIO for CIPO (maps to platform miso_io_num)
17         */
18     /* ... */
19 } star_spi_bus_config_t;
20
21 /* When setting up platform HAL structures (example: ESP32) */
22 static rx_err_t internal_setup_spi_bus(star_bus_config_t* config)
23 {
24     spi_bus_config_t bus_cfg = {
25         /* Map our inclusive terminology to platform legacy names */
26         .mosi_io_num = config->proto.spi.copi_io_num,
27         .miso_io_num = config->proto.spi.cipo_io_num,
28         .sclk_io_num = config->proto.spi.sclk_io_num,
29         .quadwp_io_num = -1,
30         .quadhd_io_num = -1,
31         .max_transfer_sz = config->proto.spi.max_transfer_sz,
32     };
33     /* ... */
34 }
35
36 /* Similarly for I2C */
37 /* Some platforms use I2C_MODE_SLAVE/MASTER, we document as
38 peripheral/controller */
39 i2c_config_t i2c_conf = {
40     .mode = I2C_MODE_MASTER, /* Controller mode in STAR terminology */
41     /* or */

```

```

39     .mode = I2C_MODE_SLAVE,    /* Peripheral mode in STAR terminology */
40     /* ... */
41 };

```

9.132 Documentation Guidelines

When writing comments, documentation, or user-facing text:

```

1  /* CORRECT - Use inclusive terminology in all documentation */
2
3  /**
4   * @brief Configure ESP32 as SPI peripheral device
5   *
6   * The peripheral will respond to transactions initiated by an
7   * external SPI controller. Data flows:
8   * - COPI: Data received from controller
9   * - CIPO: Data sent to controller
10  *
11  * @param[in] name Unique name for this SPI peripheral instance
12  * @param[in] host SPI host device
13  * @param[in] copi_pin GPIO for Controller Out, Peripheral In
14  * @param[in] cipo_pin GPIO for Controller In, Peripheral Out
15  * ...
16  */
17 star_bus_config_t* star_bus_config_create_spi_peripheral(...);
18
19 /* WRONG - Legacy terminology */
20 /**
21  * @brief Configure ESP32 as SPI slave device
22  *
23  * The slave will respond to transactions initiated by the
24  * SPI master. Data flows:
25  * - MOSI: Data from master to slave
26  * - MISO: Data from slave to master
27  */

```

9.133 Code Review Checklist

When reviewing code, check for:

- **Variable names:** No “master”, “slave”, “mosi”, “miso”
- **Function names:** Use “controller”, “peripheral”, “copi”, “cipo”
- **Comments:** Update any legacy terminology
- **Documentation:** Ensure Doxygen uses inclusive terms
- **String literals:** Check log messages and error strings
- **Platform HAL mapping:** Document when interfacing with legacy APIs

9.134 Common Patterns

```

1  /* Logging with inclusive terminology */
2  RX_LOG_INFO(s_TAG, "SPI peripheral initialized on host %d", host);
3  RX_LOG_INFO(s_TAG, "I2C controller configured on port %d", port);
4
5  /* Error messages */
6  ESP_LOGE(s_TAG, "Failed to initialize SPI peripheral: %s",
7  rx_err_to_name(ret));
8  ESP_LOGE(s_TAG, "I2C controller transaction failed");
9
10 /* Pin registration descriptions */
11 internal_register_bus_pin(pin_iface, config->proto.spi.copi_io_num,
12 config->name, "SPI COPI", true);
13 internal_register_bus_pin(pin_iface, config->proto.spi.cipo_io_num,
14 config->name, "SPI CIPO", true);

```

9.135 Summary

Use This	Not This
Controller	Master
Peripheral	Slave
COPI	MOSI
CIPO	MISO
Primary	Master (for configs)
Main	Master (general)
Allowlist	Whitelist
Blocklist	Blacklist

Following these guidelines ensures our codebase is inclusive and welcoming while maintaining technical clarity. The terminology accurately describes the functional roles of devices without relying on harmful historical language.

This section describes the key architectural patterns used throughout the STAR firmware codebase. These patterns enable loose coupling, testability, and maintainability in embedded systems.

9.136 Dependency Injection (DIP)

The Dependency Inversion Principle (DIP) is the foundational architectural pattern of the STAR framework. High-level modules depend on abstract interfaces rather than concrete implementations.

9.136.1 Interface Definition

Interfaces are defined as structs containing function pointers and an opaque context pointer.

```

1  /* From star_error_interface.h */
2  typedef struct star_error_interface {
3      rx_err_t (*record_error)(void *ctx, rx_err_t error, const char
4          *message,
5          const char *file, int line, const char
6          *func);
7      bool (*can_retry)(void *ctx);

```

```

6     rx_err_t (*reset_state)(void *ctx);
7     void *ctx; /* Opaque context pointer */
8 } star_error_interface_t;
9
10 /* From star_pin_interface.h */
11 typedef struct star_pin_interface {
12     rx_err_t (*register_pin)(void *ctx, int pin, const char
13         *description,
14         bool shared);
15     rx_err_t (*unregister_pin)(void *ctx, int pin, const char
16         *description);
17     rx_err_t (*validate)(void *ctx);
18     void *ctx;
19 } star_pin_interface_t;

```

9.136.2 Interface Implementation

Concrete implementations provide the actual functionality.

```

1 /* Create error handler and get interface */
2 error_handler_t error_handler;
3 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
4
5 star_error_interface_t error_iface;
6 error_handler_get_interface(&error_iface, &error_handler);
7
8 /* Implementation of get_interface */
9 void error_handler_get_interface(star_error_interface_t* iface,
10 error_handler_t* handler)
11 {
12     iface->record_error = adapter_record_error; /* Adapter function */
13     iface->can_retry = adapter_can_retry;
14     iface->reset_state = adapter_reset_state;
15     iface->ctx = handler; /* Concrete handler as context */
16 }

```

9.136.3 Dependency Injection Pattern

Components receive their dependencies through initialization functions.

```

1 /* Component declares interface dependencies */
2 typedef struct star_bus_manager {
3     star_bus_config_t *buses;
4     SemaphoreHandle_t mutex;
5     const char *tag;
6     star_error_interface_t *error_iface; /* Injected */
7     star_pin_interface_t *pin_iface; /* Injected */
8     /* ... */
9 } star_bus_manager_t;
10
11 /* Initialization receives dependencies */
12 rx_err_t star_bus_manager_init(star_bus_manager_t* manager,
13 const char* tag,

```



```

14 star_error_interface_t* error_iface,
15 star_pin_interface_t*   pin_iface)
16 {
17     RX_RETURN_ON_FALSE(manager, RX_ERR_INVALID_ARG, s_TAG,
18                         "Manager pointer is NULL");
19
20     manager->buses = NULL;
21     manager->error_iface = error_iface; /* Can be NULL */
22     manager->pin_iface = pin_iface;     /* Can be NULL */
23     /* ... */
24 }
25
26 /* Usage with injected dependencies */
27 error_handler_t error_handler;
28 error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
29
30 star_error_interface_t error_iface;
31 star_pin_interface_t pin_iface;
32 error_handler_get_interface(&error_iface, &error_handler);
33 pin_validator_get_interface(&pin_iface);
34
35 star_bus_manager_t bus_manager;
36 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

```

9.136.4 NULL Interface Handling

Components gracefully handle NULL interfaces.

```

1  /* Check interface before use */
2  if (manager->error_iface && manager->error_iface->record_error) {
3      manager->error_iface->record_error(manager->error_iface->ctx, ret,
4                                          "Operation failed", __FILE__,
5                                          __LINE__,
6                                          __func__);
7  }
8
9  /* Or use convenience macro */
10 #define STAR_IFACE_RECORD_ERROR(iface, err, msg)
11     \
12     ((iface) && (iface)->record_error
13         \
14         ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__,
15                                 \
16                                 __LINE__, __func__)
17         \
18         : (err))

```

9.137 Bus Abstraction Pattern

The bus abstraction provides a unified interface for different communication protocols.

9.137.1 Unified Configuration

```

1  /* Single config type with protocol union */
2  typedef struct star_bus_config {
3      const char *name;
4      star_bus_type_t type; /* Discriminator */
5      bool initialized;
6      void *handle;
7      void *user_ctx;
8
9      union {
10         star_i2c_bus_config_t i2c;
11         star_spi_bus_config_t spi;
12         star_gpio_bus_config_t gpio;
13         star_adc_bus_config_t adc;
14     } proto;
15
16     struct star_bus_config *next;
17 } star_bus_config_t;

```

9.137.2 Factory Functions

Factory functions create configurations for specific protocols.

```

1  /* I2C bus factory */
2  star_bus_config_t* star_bus_config_create_i2c(
3  const char* name,
4  i2c_port_t port,
5  uint8_t address,
6  gpio_num_t sda_pin,
7  gpio_num_t scl_pin,
8  uint32_t clk_speed);
9
10 /* SPI device factory */
11 star_bus_config_t* star_bus_config_create_spi_device(
12 const char* name,
13 spi_host_device_t host,
14 gpio_num_t cop_i_pin,
15 gpio_num_t cipo_pin,
16 gpio_num_t sclk_pin,
17 int32_t dma_chan,
18 const spi_device_interface_config_t* dev_cfg);
19
20 /* GPIO bus factory */
21 star_bus_config_t* star_bus_config_create_gpio(
22 const char* name,
23 gpio_num_t* pins,
24 uint8_t pin_count);

```

9.137.3 Manager Pattern

The bus manager coordinates multiple bus configurations.

```

1  /* Initialize manager */

```

```

2 star_bus_manager_t bus_manager;
3 star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
4
5 /* Add buses */
6 star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
7 "sensor_i2c", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
8 star_bus_manager_add_bus(&bus_manager, i2c_bus);
9
10 /* Find and use buses */
11 star_bus_config_t* bus = star_bus_manager_find_bus(&bus_manager,
12 "sensor_i2c");
13
14 /* Safe access with callback (prevents use-after-free) */
15 rx_err_t my_callback(star_bus_config_t* bus, void* ctx) {
16     /* Access bus safely while mutex is held */
17     return RX_OK;
18 }
19 star_bus_manager_with_bus(&bus_manager, "sensor_i2c", my_callback, NULL);

```

9.138 Operations Structure Pattern

Group related function pointers for pluggable implementations.

```

1 /* Define operations structure */
2 typedef struct star_i2c_ops {
3     star_i2c_write_fn_t write;
4     star_i2c_read_fn_t read;
5     star_i2c_write_command_fn_t write_command;
6     star_i2c_read_raw_fn_t read_raw;
7 } star_i2c_ops_t;
8
9 /* Initialize with default implementations */
10 void star_bus_i2c_init_default_ops(star_i2c_ops_t* ops)
11 {
12     if (ops == NULL) {
13         RX_LOG_ERROR(s_TAG, "Cannot initialize NULL ops pointer");
14         return;
15     }
16     ops->write = internal_star_bus_i2c_write;
17     ops->read = internal_star_bus_i2c_read;
18     ops->write_command = internal_star_bus_i2c_write_command;
19     ops->read_raw = internal_star_bus_i2c_read_raw;
20 }
21
22 /* Use through function pointers */
23 rx_err_t star_bus_i2c_write(const star_bus_manager_t* manager,
24 const char* name, ...)
25 {
26     star_bus_config_t *bus_config = star_bus_manager_find_bus(manager,
27 name);
28     /* ... validation ... */
29     return bus_config->proto.i2c.ops.write(bus_config, data, len,
30 reg_addr,
31 bytes_written);

```

30 }

9.139 Thread-Safe Callback Pattern

Safely access shared resources through callbacks.

```

1  /* Callback type */
2  typedef rx_err_t (*star_bus_callback_t)(star_bus_config_t* bus,
3  void* user_ctx);
4
5  /* Thread-safe access function */
6  rx_err_t star_bus_manager_with_bus(star_bus_manager_t* manager,
7  const char* name,
8  star_bus_callback_t callback,
9  void* user_ctx)
10 {
11     ESP_RETURN_ON_FALSE(manager && name && callback, RX_ERR_INVALID_ARG,
12     s_TAG, "Invalid arguments");
13
14     /* Acquire mutex */
15     if (xSemaphoreTake(
16         manager->mutex,
17         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
18         pdTRUE) {
19         return RX_ERR_TIMEOUT;
20     }
21
22     /* Find bus while holding mutex */
23     star_bus_config_t *found_bus = NULL;
24     for (star_bus_config_t *curr = manager->buses; curr; curr =
25         curr->next) {
26         if (curr->name && strcmp(curr->name, name) == 0) {
27             found_bus = curr;
28             break;
29         }
30     }
31
32     if (found_bus == NULL) {
33         xSemaphoreGive(manager->mutex);
34         return ESP_ERR_NOT_FOUND;
35     }
36
37     /* Invoke callback while holding mutex - prevents use-after-free */
38     rx_err_t callback_result = callback(found_bus, user_ctx);
39
40     xSemaphoreGive(manager->mutex);
41     return callback_result;

```

9.140 Thread Safety Patterns

9.140.1 Mutex Timeout Convention

Standard mutex timeout is 1000ms (configurable via Kconfig).

```

1  /* Standard mutex acquisition pattern */
2  if (xSemaphoreTake(manager->mutex,
3  pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS))
4  != pdTRUE) {
5      ESP_LOGE(manager->tag, "Timeout acquiring mutex");
6      return ESP_ERR_TIMEOUT;
7  }
8
9  /* Critical section */
10 /* ... protected operations ... */
11
12 xSemaphoreGive(manager->mutex);

```

9.140.2 Mutex with Cleanup(goto pattern)

```

1  rx_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
2  star_bus_config_t* config)
3  {
4      RX_RETURN_ON_FALSE(manager && config, RX_ERR_INVALID_ARG, s_TAG,
5                          "Invalid arguments");
6
7      rx_err_t ret = RX_OK;
8
9      if (xSemaphoreTake(
10         manager->mutex,
11         pdMS_TO_TICKS(CONFIG_STAR_KCONFIG_BUS_MUTEX_TIMEOUT_MS)) !=
12         pdTRUE) {
13         return RX_ERR_TIMEOUT;
14     }
15
16     /* Check for duplicate name */
17     for (star_bus_config_t *curr = manager->buses; curr; curr =
18         curr->next) {
19         if (curr->name && strcmp(curr->name, config->name) == 0) {
20             ret = RX_ERR_INVALID_STATE;
21             goto add_give_mutex; /* Single exit point */
22         }
23     }
24
25     /* Initialize bus hardware */
26     ret = star_bus_config_init(config, manager);
27     if (ret != RX_OK) {
28         goto add_give_mutex;
29     }
30
31     /* Add to list */
32     config->next = manager->buses;
33     manager->buses = config;

```

```

34     add_give_mutex:
35         xSemaphoreGive(manager->mutex);
36         return ret;
37     }

```

9.141 Error Handler Ownership Pattern

Components track whether they own their error handler for proper cleanup.

```

1  /* Sensor handle with ownership tracking */
2  typedef struct mpu6050_handle {
3      star_bus_manager_t *bus_manager;
4      const char *bus_name;
5      star_error_interface_t *error_iface;
6      bool owns_error_handler; /* Ownership flag */
7      SemaphoreHandle_t mutex;
8      bool initialized;
9      /* ... */
10 } mpu6050_handle_t;
11
12 /* Initialization with optional interface */
13 rx_err_t star_sensor_mpu6050_init(mpu6050_handle_t* handle,
14 star_bus_manager_t* bus_manager,
15 const char* bus_name,
16 star_error_interface_t* error_iface,
17 const mpu6050_config_t* config)
18 {
19     if (error_iface != NULL) {
20         /* Use provided interface - we don't own it */
21         handle->error_iface = error_iface;
22         handle->owns_error_handler = false;
23     } else {
24         /* Create default - we own it */
25         handle->error_iface = star_error_interface_create_default();
26         handle->owns_error_handler = true;
27     }
28     /* ... */
29 }
30
31 /* Cleanup respects ownership */
32 rx_err_t star_sensor_mpu6050_deinit(mpu6050_handle_t* handle)
33 {
34     /* Clean up error interface if we own it */
35     star_error_interface_cleanup(handle->error_iface,
36                                handle->owns_error_handler);
37     handle->error_iface = NULL;
38     /* ... */
39 }
40
41 /* Helper for cleanup */
42 void star_error_interface_cleanup(star_error_interface_t* error_iface,
43 bool owns_handler)
44 {
45     if (!owns_handler || !error_iface) {

```

```

46     return; /* Not our responsibility */
47 }
48 error_handler_t *handler = (error_handler_t *)error_iface->ctx;
49 error_handler_destroy_default(handler);
50 free(error_iface);
51 }

```

9.142 Linked List Management

Bus configurations are managed as a singly-linked list.

```

1  /* Add to head of list (O(1)) */
2  config->next = manager->buses;
3  manager->buses = config;
4
5  /* Remove from list (O(n)) */
6  star_bus_config_t* prev = NULL;
7  for (star_bus_config_t* curr = manager->buses; curr; prev = curr, curr =
8      curr->next) {
9      if (strcmp(curr->name, name) == 0) {
10         if (prev == NULL) {
11             manager->buses = curr->next; /* Remove head */
12         } else {
13             prev->next = curr->next; /* Remove middle/tail */
14         }
15         curr->next = NULL;
16         /* ... cleanup curr ... */
17         break;
18     }
19 }
20
21 /* Iterate through list */
22 for (star_bus_config_t* curr = manager->buses; curr; curr = curr->next) {
23     /* Process each configuration */
24 }

```

9.143 Best Practices Summary

- **Use dependency injection:** Components receive interfaces, not concrete types.
- **Handle NULL interfaces:** Always check before dereferencing.
- **Track ownership:** Know who is responsible for cleanup.
- **Use callbacks for safe access:** Prevents use-after-free with shared resources.
- **Consistent mutex timeout:** 1000ms standard timeout.
- **Single exit point with goto:** Ensures mutex release on all paths.
- **Factory functions:** Create configurations with proper initialization.
- **Operations structures:** Enable pluggable implementations.

9.144 Prefer Alternatives to Macros

Rule: Avoid macros unless absolutely necessary. Use modern C alternatives:

- `static const` for constants
- `inline` functions for simple operations
- `enum` for related constants

Rationale: Macros lack type safety, scope control, and debugger visibility.

Good:

```
static const uint32_t k_max_buffer_size = 256;
static const float k_pi = 3.14159F;

static inline int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Bad:

```
#define MAX_BUFFER_SIZE 256
#define PI 3.14159
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

9.145 When Macros Are Necessary

Use macros only for:

1. Array sizes (compile-time constants)
2. Conditional compilation (`#ifdef`, `#if`)
3. Token pasting and stringification
4. Platform-specific code paths

9.146 Macro Safety Rules

9.146.1 Wrap Statement - Like Macros in `do -while (0)`

Rule: Always wrap multi-statement macros in `do { ... } while (0)`.

Why: Prevents bugs when used in if-statements without braces.

Correct:

```
#define LOG_ERROR(msg)                                \
do {                                                    \
    log_timestamp();                                    \
    log_message(msg);                                  \
} while (0)
```

Incorrect(causes bugs) :


```

#define LOG_ERROR(msg)                                \
    log_timestamp();                                  \
    log_message(msg)

/* Bug example: */
if (error)
    LOG_ERROR("fail"); /* Only log_timestamp() is conditional! */
else
    handle_success();

```

9.146.2 Parenthesize All Macro Arguments

Rule: Wrap all argument uses and the entire expression in parentheses.

Correct:

```

#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

```

Incorrect(causes bugs) :

```

#define SQUARE(x) x * x

int result = SQUARE(2 + 3); /* Expands to: 2 + 3 * 2 + 3 = 11, not 25! */

```

9.146.3 Beware of Side Effects and Multiple Evaluation

Rule: Document if macro arguments are evaluated multiple times. Never pass arguments with side effects.

Dangerous:

```

#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int i = 5;
int result = MAX(i++, 10); /* i is incremented TWICE! */

```

Solution: Use inline functions or add warning comment:

```

/* WARNING: Arguments evaluated multiple times - no side effects! */
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

```

9.147 Multi - line Macro Formatting

Rules:

- Use backslash continuation aligned consistently
- Indent continuation lines for readability
- Place opening brace on same line as `do`

Example:

```

#define RECORD_ERROR(handler, code, desc)          \
    do {                                           \
        if ((handler)->error_iface != NULL) {     \
            (handler)->error_iface->record_error((code), (desc), __FILE__, \
                                                    __LINE__); \
        }                                         \
    } while (0)

```

9.148 Naming Conventions

Rule: Use SCREAMING_SNAKE_CASE for all macros.

Examples:

```

/* Simple constants */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)

/* Function-like macros (always use parentheses) */
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

/* Multi-statement macros (always use do-while(0)) */
#define LOG_ERROR(msg)
do {
    log_timestamp();
    log_message(msg);
} while (0)

#define RECORD_ERROR(handler, code, desc)
do {
    error_handler_record_error((handler), (code), (desc), __FILE__,
                               __LINE__, __func__);
} while (0)

```

9.149 Include Guards

Use traditional include guards for maximum portability:

```

#ifndef STAR_COMPONENT_FILENAME_H
#define STAR_COMPONENT_FILENAME_H

/* File contents */

#endif /* STAR_COMPONENT_FILENAME_H */

```

Do not use `#pragma once` (see Section 12 for details).

9.150 Conditional Compilation

Best Practices:

- Document why conditionals exist
- Keep conditional blocks small
- Use configuration headers instead of scattered `#ifdef`

Good:

```
/* Platform-specific GPIO initialization */
#if defined(CONFIG_STAR_BOARD_ESP32_S3)
    configure_esp32s3_gpio();
#elif defined(CONFIG_STAR_BOARD_ESP32_WROOM)
    configure_esp32_wroom_gpio();
#else
#error "Unsupported board configuration"
#endif
```

This section establishes the unit suffix naming standard used throughout the STAR firmware codebase. All numeric fields use SI (International System of Units) units with explicit suffixes to eliminate unit ambiguity and enable automatic documentation generation.

9.151 Overview

The firmware uses the MKS (Meter-Kilogram-Second) system as the foundation for all physical measurements. Every numeric variable that represents a physical quantity must include a unit suffix that clearly identifies its meaning.

9.152 Benefits

- **Eliminates Unit Ambiguity:** No confusion about whether a value is in meters or millimeters, degrees or radians
- **Self-Documenting Code:** Variable names explicitly state their units
- **Protocol Buffer Compatibility:** Matches naming conventions used in `star-proto/` messages
- **Prevents Conversion Errors:** Explicit units reduce the risk of unit mismatch bugs

9.153 Standard Unit Suffixes

Table 50: Standard Unit Suffixes for STAR Firmware

Suffix	Unit	Proto Example	C Example
_m	meters	x_m	distance_m
_mps	meters/second	velocity_mps	speed_mps
_mps2	m/s ²	accel_x_mps2	acceleration_mps2
_rad	radians	angle_rad	heading_rad
_rad_per_s	rad/second	omega_rad_per_s	angular_vel_rad_per_s
_celsius	degrees C	temperature_celsius	motor_temp_celsius
_deci_celsius	0.1°C	temp_deci_celsius	(BMS native format)
_us	microseconds	timestamp_us	elapsed_us
_ms	milliseconds	timeout_ms	period_ms
_ma	milliamps	current_ma	motor_current_ma
_mv	millivolts	cell_mv	pack_voltage_mv
_mw	milliwatts	power_mw	consumption_mw
_percent	0–100%	soc_percent	duty_cycle_percent

9.154 Usage Examples

9.154.1 Distance and Velocity

```

1  /* CORRECT - Distance and velocity with units */
2  float distance_m = 1.5F;           /* 1.5 meters */
3  float velocity_mps = 2.0F;         /* 2.0 meters per second */
4  float acceleration_mps2 = 9.81F;   /* 9.81 m/s^2 (gravity) */
5
6  /* Motor velocity tracking */
7  float target_velocity_mps = 0.5F;
8  float current_velocity_mps;
9  star_encoder_get_velocity_mps(&encoder, 10.0F, 500,
10                               &current_velocity_mps);
11
12 /* WRONG - No unit suffix */
13 float distance = 1.5F;             /* What unit? Meters? Millimeters? Feet? */
14 float velocity = 2.0F;             /* Meters per second? RPM? */

```

9.154.2 Angular Measurements

```

1  /* CORRECT - Angular measurements in radians */
2  float heading_rad = 1.57F;         /* 90 degrees in radians */
3  float angular_velocity_rad_per_s = 3.14F; /* pi radians per second */
4
5  /* Robot pose */
6  typedef struct {
7      float x_m;
8      float y_m;
9      float theta_rad; /* Heading angle */

```

```

10 } robot_pose_t;
11
12 /* WRONG - Ambiguous angle units */
13 float heading = 90.0F; /* Degrees or radians? */
14 float omega = 3.14F; /* What unit? */

```

9.154.3 Temperature

```

1 /* CORRECT - Temperature with units */
2 float motor_temp_celsius;
3 star_sensor_ds18b20_read_temp(&temp_sensor, &motor_temp_celsius);
4
5 /* BMS uses deci-celsius (0.1 degree resolution) */
6 int16_t cell_temp_deci_celsius = 235; /* 23.5 degrees C */
7
8 /* Thermal shutdown threshold */
9 static const float s_motor_thermal_shutdown_celsius = 85.0F;
10
11 if (motor_temp_celsius > s_motor_thermal_shutdown_celsius) {
12     rx_motor_stop(&motor, true); /* Emergency brake */
13 }
14
15 /* WRONG */
16 float motor_temp = 75.0F; /* Celsius? Fahrenheit? Kelvin? */

```

9.154.4 Time

```

1 /* CORRECT - Time units */
2 uint32_t timeout_ms = 1000; /* 1 second timeout */
3 uint64_t timestamp_us; /* Microsecond timestamp */
4 star_encoder_get_timestamp_us(&encoder, &timestamp_us);
5
6 /* Timing for control loops */
7 const uint32_t control_period_ms = 1; /* 1ms = 1000Hz */
8 const uint32_t sensor_poll_us = 10000; /* 10ms in microseconds */
9
10 /* WRONG */
11 uint32_t timeout = 1000; /* Milliseconds? Seconds? Ticks? */
12 uint64_t timestamp; /* What resolution? */

```

9.154.5 Electrical Measurements

```

1 /* CORRECT - Electrical units */
2 uint32_t motor_current_ma;
3 rx_motor_get_current_ma(&motor, &motor_current_ma);
4
5 uint32_t cell_voltage_mv = 3700; /* 3.7V in millivolts */
6 uint32_t pack_voltage_mv = 11100; /* 11.1V (3S LiPo) */
7 uint32_t power_mw = 7400; /* 7.4W in milliwatts */
8

```

```

9      /* Current limiting */
10     typedef enum : uint32_t {
11         k_max_motor_current_ma = 2000, /**< 2A motor current limit */
12     } motor_current_limits_t;
13
14     if (motor_current_ma > k_max_motor_current_ma) {
15         ESP_LOGW(s_TAG, "Motor current limit exceeded: %lu mA",
16                 motor_current_ma);
17         rx_motor_set_duty(&motor, 0.0F);
18     }
19
20     /* WRONG */
21     uint32_t motor_current = 1500; /* Milliamps? Amps? */
22     uint32_t voltage = 3700; /* Millivolts? Volts? */

```

9.154.6 Percentage

```

1     /* CORRECT - Percentage (0-100 range) */
2     float duty_cycle_percent = 75.0F; /* 75% duty cycle */
3     uint8_t battery_soc_percent = 85; /* 85% state of charge */
4     float throttle_percent = 50.0F; /* 50% throttle */
5
6     rx_motor_set_duty(&motor, duty_cycle_percent);
7
8     /* WRONG */
9     float duty_cycle = 0.75F; /* Normalized (0-1)? Percentage (0-100)? */
10    uint8_t battery_soc = 85; /* Percentage? Raw ADC value? */

```

9.155 Protocol Buffer Integration

The unit suffix convention matches the naming used in Protocol Buffer messages:

```

1     // From star-proto/proto/robot_command.proto
2     message VelocityCommand {
3         float velocity_mps = 1; /* Meters per second
4         float angular_velocity_rad_per_s = 2; /* Radians per second
5     }
6
7     message BatteryState {
8         repeated uint32 cell_mv = 1; /* Cell voltages in millivolts
9         uint32 current_ma = 2; /* Pack current in milliamps
10        uint8 soc_percent = 3; /* State of charge (0-100%)
11        int16 temperature_celsius = 4; /* Pack temperature
12    }

```

9.156 Conversion Functions

When converting between units, use descriptive function names:

```

1     /* CORRECT - Clear unit conversion */
2     static const float s_seconds_per_minute = 60.0F;
3

```

```

4 float rpm_to_mps(float rpm, float wheel_circumference_m)
5 {
6     float revolutions_per_second = rpm / s_seconds_per_minute;
7     return revolutions_per_second * wheel_circumference_m;
8 }
9
10 static const float s_deci_celsius_per_celsius = 10.0F;
11
12 float celsius_to_deci_celsius(float temp_celsius)
13 {
14     return temp_celsius * s_deci_celsius_per_celsius;
15 }
16
17 static const float s_mv_per_volt = 1000.0F;
18
19 float mv_to_volts(uint32_t voltage_mv)
20 {
21     return voltage_mv / s_mv_per_volt;
22 }
23
24 /* WRONG - Ambiguous conversions */
25 float convert_rpm(float rpm); /* Convert to what? */
26 float scale_temp(float temp); /* What units? */

```

9.157 Summary

- Always use unit suffixes for physical quantities
- Match Protocol Buffer conventions for consistency
- Use SI base units when possible (meters, radians, seconds)
- Document conversions clearly in function names
- Prefer millivolts/milliamps over floating-point volts/amps for integer precision

This convention eliminates an entire class of bugs caused by unit ambiguity and makes the codebase more maintainable.

This section establishes memory management rules for STAR firmware across all platforms. Embedded platforms have limited RAM (e.g., RX72N: 1MB SRAM), making proper memory management critical for system stability and performance.

9.158 No Dynamic Allocation

Rule: Never use `malloc()`, `calloc()`, `realloc()`, or `free()` in production firmware.

9.158.1 Rationale

- **Heap Fragmentation:** Repeated allocation/deallocation causes memory fragmentation over time
- **Memory Leaks:** Forgotten `free()` calls lead to gradual RAM exhaustion

- **Unpredictable Timing:** Allocation time is non-deterministic (breaks real-time constraints)
- **Out-of-Memory Failures:** No recovery mechanism when heap is exhausted
- **Debugging Difficulty:** Memory corruption bugs are extremely hard to track down

9.158.2 Alternatives

Use static allocation instead:

```

1  /* WRONG - Dynamic allocation */
2  void process_sensor_data(void)
3  {
4      float *samples = (float *)malloc(1000 * sizeof(float));
5      if (samples == NULL) {
6          RX_LOG_ERROR(s_TAG, "Out of memory!");
7          return;
8      }
9
10     /* Process samples... */
11
12     free(samples); /* Easy to forget! */
13 }
14
15 /* CORRECT - Static allocation */
16 static float s_samples[1000]; /* Allocated once at compile time */
17
18 void process_sensor_data(void)
19 {
20     /* Use s_samples directly - no allocation, no leaks */
21     for (int i = 0; i < 1000; i++) {
22         s_samples[i] = read_sensor();
23     }
24 }
```

9.158.3 Platform Heap Functions

Some platforms provide specialized heap functions (e.g., ESP-IDF's `heap_caps_malloc`) that are **only acceptable** for:

- **One-time initialization:** Allocating resources during startup that persist for the lifetime of the program
- **FreeRTOS task stacks:** Created once during task initialization
- **Third-party libraries:** When required by external dependencies (document clearly)

```

1  /* ACCEPTABLE - One-time allocation during init */
2  rx_err_t sensor_driver_init(sensor_handle_t* handle)
3  {
4      /* Allocate persistent buffer once */
5      handle->rx_buffer = heap_caps_malloc(DMA_BUFFER_SIZE,
        MALLOC_CAP_DMA);
```



```

6         if (handle->rx_buffer == NULL) {
7             return ESP_ERR_NO_MEM;
8         }
9         handle->owns_buffer = true; /* Track ownership */
10        return RX_OK;
11    }
12
13    /* Must have corresponding cleanup */
14    rx_err_t sensor_driver_deinit(sensor_handle_t* handle)
15    {
16        if (handle->owns_buffer && handle->rx_buffer) {
17            free(handle->rx_buffer);
18            handle->rx_buffer = NULL;
19        }
20        return RX_OK;
21    }

```

9.159 Avoid Large Stack Allocations

Rule: Never allocate large arrays on the stack. Use static or global storage instead.

9.159.1 Stack Size Constraints

- Typical embedded task stack: **2KB–8KB** (platform and RTOS dependent)
- ISR stack: Even smaller (~1KB)
- Stack overflow causes silent corruption or hard faults

9.159.2 Examples

```

1  /* WRONG - Large stack allocation */
2  void process_image(void)
3  {
4      uint8_t frame_buffer[640 * 480]; /* 307KB on stack! */
5      /* Stack overflow guaranteed! */
6  }
7
8  /* WRONG - Nested large allocations */
9  void motor_control_loop(void)
10 {
11     float pid_state[100]; /* 400 bytes */
12     uint32_t encoder_samples[500]; /* 2000 bytes */
13     /* Total: 2400 bytes - may overflow 3KB stack */
14 }
15
16 /* CORRECT - Use static storage */
17 static uint8_t s_frame_buffer[640 * 480]; /* In .bss, not stack */
18
19 void process_image(void)
20 {
21     /* Use s_frame_buffer safely */
22     memset(s_frame_buffer, 0, sizeof(s_frame_buffer));

```

```

23 }
24
25 /* CORRECT - Small stack allocations are fine */
26 void calculate_checksum(const uint8_t* data, size_t len)
27 {
28     uint32_t crc = 0; /* 4 bytes - perfectly fine on stack */
29     /* ... */
30 }

```

9.159.3 Stack Usage Guidelines

Table 51: Stack Allocation Guidelines

Size	Allowed	Storage Location
< 256 bytes	Yes	Stack (automatic)
256 bytes – 1 KB	Caution	Prefer static if persistent
> 1 KB	No	Static or global only
> 10 KB	Never	Static with explicit comment

9.160 Use const for Flash Storage

Rule: Place all read-only data in flash/ROM using the `const` keyword.

9.160.1 Rationale

- Embedded platforms typically have large flash but limited RAM (e.g., RX72N: 4MB flash / 1MB RAM)
- `const` data is stored in flash and accessed directly (no RAM copy)
- Saves precious SRAM for runtime variables

9.160.2 Examples

```

1 /* WRONG - Lookup table in RAM (wastes 256 bytes) */
2 static uint8_t sine_lookup[256] = {
3     0, 3, 6, 9, 12, /* ... */
4 };
5
6 /* CORRECT - Lookup table in flash (zero RAM usage) */
7 static const uint8_t sine_lookup[256] = {
8     0, 3, 6, 9, 12, /* ... */
9 };
10
11 /* WRONG - String literals without const */
12 static char* error_messages[] = {
13     "Sensor timeout",
14     "Invalid checksum",
15     "Bus error"
16 };
17

```

```

18 /* CORRECT - Strings in flash */
19 static const char* const error_messages[] = {
20     "Sensor timeout",
21     "Invalid checksum",
22     "Bus error"
23 };
24
25 /* CORRECT - Default configuration in flash */
26 static const star_pid_config_t default_pid_config = {
27     .kp = 1.0F,
28     .ki = 0.5F,
29     .kd = 0.1F,
30     .output_min = -100.0F,
31     .output_max = 100.0F,
32 };
33
34 /* Usage: Copy to RAM only when needed */
35 star_pid_config_t runtime_config;
36 memcpy(&runtime_config, &default_pid_config, sizeof(runtime_config));

```

9.160.3 Pointer Declarations

Be careful with pointer const placement:

```

1 /* Pointer to const data (data in flash, pointer in RAM) */
2 const char* error_msg = "Timeout";
3
4 /* Const pointer to data (pointer in flash, data in RAM) */
5 char* const buffer_ptr = buffer;
6
7 /* Const pointer to const data (both in flash) */
8 const char* const error_table[] = {"Error 1", "Error 2"};

```

9.161 Validate All Buffers

Rule: Always validate buffer sizes before `memcpy()`, DMA operations, or ring buffer manipulations.

9.161.1 Rationale

- Buffer overruns cause memory corruption
- Corruption may manifest much later, making debugging extremely difficult
- Real-time systems have no memory protection (unlike Linux)

9.161.2 Validation Patterns

```

1 /* WRONG - Unchecked memcpy */
2 void receive_packet(uint8_t* data, size_t len)
3 {
4     memcpy(rx_buffer, data, len); /* What if len > buffer size? */
5 }

```

```

6
7      /* CORRECT - Validate before copy */
8      #define RX_BUFFER_SIZE (512)
9      static uint8_t rx_buffer[RX_BUFFER_SIZE];
10
11 void receive_packet(const uint8_t* data, size_t len)
12 {
13     if (data == NULL || len > RX_BUFFER_SIZE) {
14         RX_LOG_ERROR(s_TAG, "Invalid packet: len=%zu (max=%d)", len,
15                     RX_BUFFER_SIZE);
16         return;
17     }
18     memcpy(rx_buffer, data, len);
19 }
20
21 /* CORRECT - Clamp to buffer size */
22 void receive_packet_safe(const uint8_t* data, size_t len,
23 size_t* bytes_copied)
24 {
25     size_t copy_len = (len < RX_BUFFER_SIZE) ? len : RX_BUFFER_SIZE;
26     memcpy(rx_buffer, data, copy_len);
27     *bytes_copied = copy_len;
28
29     if (len > RX_BUFFER_SIZE) {
30         RX_LOG_WARN(s_TAG, "Packet truncated: %zu bytes lost",
31                   len - RX_BUFFER_SIZE);
32     }
33 }

```

9.161.3 DMA Buffer Validation

```

1  /* DMA buffers must be validated AND aligned */
2  rx_err_t start_dma_transfer(const uint8_t* src, size_t len)
3  {
4      /* Validate buffer pointer */
5      if (src == NULL) {
6          return RX_ERR_INVALID_ARG;
7      }
8
9      /* Validate length */
10     if (len == 0 || len > MAX_DMA_TRANSFER_SIZE) {
11         RX_LOG_ERROR(s_TAG, "Invalid DMA length: %zu", len);
12         return ESP_ERR_INVALID_SIZE;
13     }
14
15     /* Check alignment (DMA requires 4-byte alignment) */
16     if (((uintptr_t)src & 0x3) != 0) {
17         RX_LOG_ERROR(s_TAG, "DMA buffer not 4-byte aligned");
18         return RX_ERR_INVALID_ARG;
19     }
20
21     /* Proceed with DMA */
22     return spi_device_transmit(spi_handle, &transaction);

```

23 }

9.161.4 Ring Buffer Validation

```

1  /* CORRECT - Ring buffer with overflow protection */
2  typedef struct {
3      uint8_t buffer[256];
4      size_t head;
5      size_t tail;
6      size_t count;
7  } ring_buffer_t;
8
9  rx_err_t ring_buffer_write(ring_buffer_t* rb, const uint8_t* data,
10 size_t len)
11 {
12     if (rb == NULL || data == NULL) {
13         return RX_ERR_INVALID_ARG;
14     }
15
16     /* Check available space */
17     size_t available = sizeof(rb->buffer) - rb->count;
18     if (len > available) {
19         RX_LOG_WARN(s_TAG, "Ring buffer overflow: %zu bytes lost",
20                     len - available);
21         return ESP_ERR_NO_MEM;
22     }
23
24     /* Safe to write */
25     for (size_t i = 0; i < len; i++) {
26         rb->buffer[rb->head] = data[i];
27         rb->head = (rb->head + 1) % sizeof(rb->buffer);
28         rb->count++;
29     }
30
31     return RX_OK;
32 }

```

9.162 Summary

Table 52: Memory Management Rules Summary

Rule	Implementation
No dynamic allocation	Use static/global buffers instead of malloc/free
Avoid large stack arrays	Use static storage for arrays > 256 bytes
Use <code>const</code>	Place lookup tables, strings, defaults in flash
Validate all buffers	Check lengths before memcpy, DMA, ring buffers

Following these rules ensures stable, predictable memory usage and prevents an entire class of memory-related bugs that are extremely difficult to debug in embedded systems.

This section establishes best practices for real-time embedded systems development across all STAR platforms. These guidelines are critical for motor control (1000Hz loops), sensor fusion, and safety-critical robotics applications.

9.163 Interrupt Service Routines(ISRs)

9.163.1 Minimize ISR Work

Rule: Keep ISRs as short as possible. Set flags or queue work to background tasks instead of performing complex operations.

```

1  /* WRONG - Too much work in ISR */
2  static void IRAM_ATTR gpio_isr_handler(void* arg)
3  {
4      /* Reading I2C sensor blocks for milliseconds! */
5      float temperature = read_i2c_sensor();
6      update_pid_controller(temperature);
7      adjust_motor_speed();
8      log_telemetry_to_sd_card(); /* File I/O in ISR! */
9  }
10
11 /* CORRECT - Minimal ISR, queue work to task */
12 static volatile bool s_encoder_event = false;
13 static QueueHandle_t s_encoder_queue;
14
15 static void IRAM_ATTR encoder_isr_handler(void* arg)
16 {
17     /* Only set flag and queue event */
18     s_encoder_event = true;
19
20     encoder_event_t event = {.timestamp_us = esp_timer_get_time()};
21     xQueueSendFromISR(s_encoder_queue, &event, NULL);
22
23     /* ISR done in microseconds, not milliseconds */
24 }
25
26 /* Background task processes events */
27 void encoder_task(void* arg)
28 {
29     encoder_event_t event;
30     while (1) {
31         if (xQueueReceive(s_encoder_queue, &event, portMAX_DELAY)) {
32             /* Heavy processing here, not in ISR */
33             process_encoder_event(&event);
34         }
35     }
36 }

```

9.163.2 ISR Constraints

- **No blocking operations:** No mutexes, delays, or I/O
- **Use IRAM_ATTR:** Places ISR in IRAM for faster execution

- **FromISR variants:** Use `xQueueSendFromISR()`, not `xQueueSend()`
- **Keep stack usage low:** ISR stack is typically 1KB or less
- **No floating-point:** Context save/restore is expensive

9.164 No Blocking Delays

Rule: Never use `vTaskDelay()` or busy-wait loops in time-critical code. Use event-driven design.

```

1  /* WRONG - Blocking delay in motor control */
2  void motor_control_loop(void)
3  {
4      while (1) {
5          read_encoder();
6          compute_pid();
7          update_motor();
8
9          vTaskDelay(pdMS_TO_TICKS(1)); /* Blocks, jitter from scheduler */
10     }
11 }
12
13 /* CORRECT - Timer-driven execution */
14 static void motor_control_timer_callback(void* arg)
15 {
16     /* Called exactly every 1ms by hardware timer */
17     read_encoder();
18     compute_pid();
19     update_motor();
20 }
21
22 void setup_motor_control(void)
23 {
24     esp_timer_create_args_t timer_args = {
25         .callback = motor_control_timer_callback, .name = "motor_ctrl"};
26
27     esp_timer_handle_t timer;
28     esp_timer_create(&timer_args, &timer);
29     esp_timer_start_periodic(timer, 1000); /* 1ms = 1000us */
30 }
31
32 /* CORRECT - Event-driven sensor reading */
33 void sensor_task(void* arg)
34 {
35     while (1) {
36         /* Wait for event, no polling */
37         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
38
39         /* Event occurred, process it */
40         process_sensor_data();
41     }
42 }

```

9.165 Real - Time and Safety

Table 53: Real - Time Best Practices

Practice	Implementation
Know timing constraints	Profile worst-case execution times (WCET)
Use watchdog timers	Automatic reset on firmware lockup
Handle sensor timeouts	Never assume sensors always respond
Fail safe	On error, disable actuators and enter safe state
CRC critical data	Validate packets and stored configuration

9.165.1 Timing Constraints

```

1  /* Profile execution time using GPIO toggle */
2  void motor_control_loop(void)
3  {
4      gpio_set_level(DEBUG_PIN, 1); /* Pin HIGH */
5
6      /* Critical timing section */
7      read_encoder();
8      compute_pid();
9      update_motor();
10
11     gpio_set_level(DEBUG_PIN, 0); /* Pin LOW */
12
13     /* Measure with oscilloscope:
14       * Pulse width = worst-case execution time (WCET) */
15 }
16
17 /* Or use platform-specific cycle counter (ESP32, RX72N, etc.) */
18 uint32_t start_cycles = esp_cpu_get_cycle_count();
19
20 critical_function();
21
22 uint32_t end_cycles = esp_cpu_get_cycle_count();
23 uint32_t elapsed_us = (end_cycles - start_cycles) / (CPU_FREQ_MHZ);
24 RX_LOG_INFO(s_TAG, "Execution time: %lu us", elapsed_us);

```

9.165.2 Watchdog Timers

```

1  /* Initialize task watchdog */
2  #include "esp_task_wdt.h"
3
4  void motor_control_task(void* arg)
5  {
6      /* Subscribe to watchdog (5 second timeout) */
7      esp_task_wdt_add(NULL);
8
9      while (1) {
10         /* Kick watchdog to prove we're alive */

```



```

11     esp_task_wdt_reset();
12
13     /* Control loop */
14     motor_control_iteration();
15
16     vTaskDelay(pdMS_TO_TICKS(1));
17 }
18 }
19
20 /* If task hangs, watchdog resets system after timeout
   (platform-specific) */

```

9.165.3 Sensor Timeouts

```

1 /* WRONG - Assumes sensor always responds */
2 float read_temperature(void)
3 {
4     send_i2c_command(TEMP_SENSOR_ADDR, READ_TEMP_CMD);
5     return receive_i2c_data(TEMP_SENSOR_ADDR); /* Hangs if sensor fails
   */
6 }
7
8 /* CORRECT - Timeout and error handling */
9 rx_err_t read_temperature_safe(float* temp_celsius)
10 {
11     rx_err_t ret;
12     uint8_t data[2];
13
14     /* I2C read with 100ms timeout */
15     ret = star_bus_i2c_read(bus_manager, "temp_i2c", data, 2,
16                             TEMP_READ_REG,
17                             NULL);
18     if (ret != RX_OK) {
19         RX_LOG_WARN(s_TAG, "Temperature sensor timeout");
20         return ret;
21     }
22
23     *temp_celsius = (data[0] << 8 | data[1]) / 10.0F;
24     return RX_OK;
25 }
26
27 /* Use sensor timeout for safety */
28 rx_err_t ret = read_temperature_safe(&motor_temp);
29 if (ret != RX_OK) {
30     /* Sensor failed - enter safe state */
31     rx_motor_stop(&motor, true);
32     system_enter_safe_mode();
33 }

```

9.165.4 Fail - Safe Design

```

1  /* Safe state: all motors stopped, systems halted */
2  typedef enum : uint8_t {
3      k_num_motors = 4, /**< Number of motors on STAR platform */
4  } motor_count_t;
5
6  void system_enter_safe_mode(void)
7  {
8      RX_LOG_ERROR(s_TAG, "ENTERING SAFE MODE");
9
10     /* Stop all motors */
11     for (uint8_t i = 0; i < k_num_motors; i++) {
12         rx_motor_stop(&motors[i], true); /* Brake */
13     }
14
15     /* Disable all actuators */
16     gpio_set_level(RELAY_ENABLE_PIN, 0);
17
18     /* Set error LED */
19     gpio_set_level(ERROR_LED_PIN, 1);
20
21     /* Log error to flash */
22     log_critical_error_to_nvs("Safe mode triggered");
23
24     /* Infinite loop - require manual reset */
25     while (1) {
26         vTaskDelay(pdMS_TO_TICKS(100));
27     }
28 }
29
30 /* Trigger safe mode on critical errors */
31 typedef enum : uint16_t {
32     k_battery_critical_mv = 3000, /**< Critical per-cell voltage threshold
33     (3.0V) */
34 } battery_limits_t;
35
36 static const float s_motor_thermal_shutdown_celsius = 85.0F;
37
38 if (cell_voltage_mv < k_battery_critical_mv) {
39     system_enter_safe_mode();
40 }
41
42 if (motor_temp_celsius > s_motor_thermal_shutdown_celsius) {
43     system_enter_safe_mode();
44 }

```

9.165.5 CRC Validation

```

1  /* Validate critical configuration with CRC */
2  typedef struct {
3      star_pid_config_t pid;
4      uint32_t crc32; /* CRC of pid structure */
5  } stored_config_t;

```

```

6
7 rx_err_t save_config_to_nvs(const star_pid_config_t* config)
8 {
9     stored_config_t stored;
10    memcpy(&stored.pid, config, sizeof(star_pid_config_t));
11
12    /* Calculate CRC-32 */
13    stored.crc32 =
14        esp_crc32_le(0, (uint8_t *)&stored.pid,
15                     sizeof(star_pid_config_t));
16
17    /* Save to NVS */
18    return nvs_set_blob(nvs_handle, "pid_config", &stored,
19                        sizeof(stored_config_t));
20 }
21
22 rx_err_t load_config_from_nvs(star_pid_config_t* config)
23 {
24     stored_config_t stored;
25     size_t len = sizeof(stored_config_t);
26
27     esp_err_t ret = nvs_get_blob(nvs_handle, "pid_config", &stored,
28                                 &len);
29     if (ret != RX_OK) {
30         return ret;
31     }
32
33     /* Verify CRC */
34     uint32_t calculated_crc =
35         esp_crc32_le(0, (uint8_t *)&stored.pid,
36                     sizeof(star_pid_config_t));
37     if (calculated_crc != stored.crc32) {
38         RX_LOG_ERROR(s_TAG, "Config CRC mismatch! Flash corrupted?");
39         return ESP_ERR_INVALID_CRC;
40     }
41
42     memcpy(config, &stored.pid, sizeof(star_pid_config_t));
43     return RX_OK;
44 }

```

9.166 Hardware Abstraction

9.166.1 Use Abstract Bus Interfaces

```

1  /* CORRECT - Driver depends on bus abstraction, not platform HAL */
2  rx_err_t mpu6050_init(mpu6050_handle_t* handle,
3                        star_bus_manager_t* bus_manager,
4                        const char* bus_name)
5  {
6      handle->bus_manager = bus_manager;
7      handle->bus_name = bus_name;
8
9      /* Access I2C through bus manager */

```

```

10     uint8_t whoami;
11     esp_err_t ret = star_bus_i2c_read(bus_manager, bus_name, &whoami, 1,
12                                     MPU6050_WHO_AM_I, NULL);
13
14     /* Easy to mock for testing */
15     return ret;
16 }

```

9.166.2 Debounce Mechanical Inputs

```

1  /* Software debounce for button */
2  typedef struct {
3      gpio_num_t pin;
4      bool state;
5      uint32_t last_change_ms;
6      uint32_t debounce_ms;
7  } debounced_button_t;
8
9  bool button_read(debounced_button_t* btn)
10 {
11     bool raw_state = gpio_get_level(btn->pin);
12     uint32_t now_ms = xTaskGetTickCount() * portTICK_PERIOD_MS;
13
14     if (raw_state != btn->state) {
15         /* State change detected */
16         if (now_ms - btn->last_change_ms > btn->debounce_ms) {
17             /* Stable for debounce period */
18             btn->state = raw_state;
19             btn->last_change_ms = now_ms;
20         }
21     }
22
23     return btn->state;
24 }

```

9.166.3 Enable Brown - Out Detection

```

1  /* In sdkconfig or menuconfig:
2  * CONFIG_ESP32_BROWNOUT_DET=y
3  * CONFIG_ESP32_BROWNOUT_DET_LVL_SEL_7=y (3.08V threshold)
4  */
5
6  /* ESP32 automatically resets on voltage dip below threshold */
7  /* Prevents flash corruption and erratic behavior */

```

9.166.4 Flash Wear - Leveling

```

1  /* Use NVS (Non-Volatile Storage) for wear-leveling */
2  #include "nvs_flash.h"
3

```

```

4 rx_err_t init_nvs(void)
5 {
6     esp_err_t ret = nvs_flash_init();
7     if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
8         ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
9         /* Erase and retry */
10        /* Platform-specific: handle flash erase error appropriately */
11        nvs_flash_erase();
12        ret = nvs_flash_init();
13    }
14    return ret;
15 }
16
17 /* NVS automatically rotates writes across sectors */
18 nvs_handle_t nvs_handle;
19 nvs_open("storage", NVS_READWRITE, &nvs_handle);
20 nvs_set_u32(nvs_handle, "boot_count", boot_count);
21 nvs_commit(nvs_handle);

```

9.166.5 Always Use Pull - Up / Pull - Down

```

1 /* WRONG - Floating input */
2 gpio_config_t io_conf = {
3     .pin_bit_mask = (1ULL << GPIO_NUM_35),
4     .mode = GPIO_MODE_INPUT,
5     /* No pull-up or pull-down! Reads random values */
6 };
7
8 /* CORRECT - Enable pull-up */
9 gpio_config_t io_conf = {
10    .pin_bit_mask = (1ULL << GPIO_NUM_35),
11    .mode = GPIO_MODE_INPUT,
12    .pull_up_en = GPIO_PULLUP_ENABLE,
13    .pull_down_en = GPIO_PULLDOWN_DISABLE,
14 };
15 gpio_config(&io_conf);
16
17 /* For active-low signals (like nFAULT), use pull-up */
18 /* For active-high signals, use pull-down */

```

9.167 Performance and Real - Time

9.167.1 Benchmark, Don't Guess

```

1 /* Use GPIO toggle to measure timing on oscilloscope */
2 #define BENCHMARK_START() gpio_set_level(BENCHMARK_PIN, 1)
3 #define BENCHMARK_END() gpio_set_level(BENCHMARK_PIN, 0)
4
5 void
6 pid_control_loop(void) {
7     BENCHMARK_START();
8

```

```

9      float velocity_rpm;
10     star_encoder_get_velocity_rpm(&encoder, 1.0F, 500, &velocity_rpm);
11
12     float output;
13     star_pid_compute(&pid, setpoint, velocity_rpm, 0.001F, &output);
14
15     star_motor_set_duty(&motor, output);
16
17     BENCHMARK_END();
18     /* Oscilloscope shows pulse width = execution time */
19 }

```

9.167.2 Avoid Blocking Operations

```

1      /* WRONG - Blocking I2C read in control loop */
2      void
3      control_loop(void) {
4      while (1) {
5          uint8_t data[6];
6          i2c_master_read_from_device(I2C_NUM_0, SENSOR_ADDR, data, 6,
7                                     100 / portTICK_PERIOD_MS);
8          /* Blocks for up to 100ms! */
9
10         process_data(data);
11     }
12 }
13
14 /* CORRECT - Non-blocking with DMA */
15 void setup_nonblocking_i2c(void) {
16     /* Configure I2C interrupt + DMA */
17     /* When data ready, ISR triggers */
18     /* Background task processes results */
19 }

```

9.168 Power Management

9.168.1 Use Sleep Modes

```

1      /* Light sleep between sensor reads */
2      void
3      sensor_polling_task(void *arg) {
4      while (1) {
5          read_sensors();
6          process_data();
7
8          /* Sleep for 100ms instead of busy-wait */
9          esp_sleep_enable_timer_wakeup(100000); /* 100ms in us */
10         esp_light_sleep_start();
11     }
12 }

```

9.168.2 Use DMA to Reduce CPU Load

```

1      /* DMA-enabled SPI transfer (CPU-free) */
2      spi_device_transmit(spi_handle, &transaction);
3      /* DMA handles transfer, CPU does other work */

```

9.169 Testing and Debugging

9.169.1 Use Mock Drivers

```

1      /* Create mock bus for unit testing */
2      star_bus_manager_t mock_bus_manager;
3      star_bus_manager_init(&mock_bus_manager, "mock", NULL, NULL);
4
5      /* Test driver without hardware */
6      mpu6050_handle_t sensor;
7      mpu6050_init(&sensor, &mock_bus_manager, "mock_i2c");

```

9.169.2 Meaningful Logging

```

1      /* Include timestamp, module, and error codes */
2      ESP_LOGE(s_TAG,
3              "[%lu] Motor fault detected: current=%lu mA (limit=%d
4              mA)",
5              esp_timer_get_time() / 1000, /* Timestamp in ms */
6              current_ma, MAX_MOTOR_CURRENT_MA);

```

9.170 Reliability and Fault Tolerance

9.170.1 Reset Peripherals if Stuck

```

1      /* I2C bus recovery */
2      rx_err_t
3      recover_i2c_bus(i2c_port_t port) {
4      ESP_LOGW(s_TAG, "Recovering I2C bus %d", port);
5
6      /* Delete driver */
7      i2c_driver_delete(port);
8
9      /* Wait */
10     vTaskDelay(pdMS_TO_TICKS(10));
11
12     /* Re-initialize */
13     i2c_config_t conf = { /* ... */ };
14     i2c_param_config(port, &conf);
15     return i2c_driver_install(port, conf.mode, 0, 0, 0);
16 }

```

9.171 Summary

Following these real - time and embedded best practices ensures :

- **Predictable timing** for motor control and sensor fusion
- **Robust error handling** for fault tolerance
- **Safe operation** in robotics applications
- **Maintainable code** through proper abstractions
- **Testability** with mock drivers and measurement tools

These practices are the foundation of reliable embedded systems for robotics.

This section establishes configuration requirements for Protocol Buffer messages used in STAR embedded firmware. The STAR project uses **nanopb** (Nano Protocol Buffers) for embedded systems, which requires static memory allocation.

9.172 nanopb Overview

nanopb is a small, portable Protocol Buffers implementation designed for embedded systems:

- *Static Allocation* : All buffers allocated at compile time(no malloc)
- *Small Code Size* : Minimal flash / RAM footprint
- *C Language* : Compatible with C99(optional C11 features; not C++)
- *Field Size Constraints* : Maximum sizes configured in `.options` files

9.173 Why Static Allocation ?

STAR firmware follows strict "no dynamic allocation" rules(see Section 25) .nanopb field sizes must be explicitly configured to enable compile - time buffer allocation .

9.174 Field Size Configuration

Field sizes are configured in `.options` files located alongside `.proto` files :

```
1 #File : star - proto / proto / star.options
2     star.v1.RequestHeader.request_id max_size :
3         64 star.v1.ResponseHeader.error_message
4     max_size : 128 star.v1.FirmwareChunk.data max_size : 1024
```


9.175 Standard Field Size Constraints

Table 54: nanopb Field Size Constraints for Embedded Platforms

Field Type	Constraint	Example
String(UUID)	max_size : 64	request_id
String(error)	max_size : 128	error_message
String(version)	max_size : 32	firmware_version
Repeated(cells)	max_count : 16	cell_mv[]
Repeated(motors)	max_count : 4	motor_status[]
Bytes(firmware)	max_size : 1024	chunk_data

9.176 Examples

9.176.1 String Fields

```

1 // File: robot_command.proto
2 message RequestHeader {
3   string request_id = 1; // UUID (36 chars + null terminator)
4 }
5
6 // File: robot_command.options
7 RequestHeader.request_id max_size : 64

```

Generated C code :

```

1 typedef struct _RequestHeader {
2   char request_id[64]; /* Static allocation */
3 } RequestHeader;

```

9.176.2 Repeated Fields

```

1 // File: battery_state.proto
2 message BatteryState {
3   repeated uint32 cell_mv = 1; // Cell voltages in millivolts
4 }
5
6 // File: battery_state.options
7 BatteryState.cell_mv max_count : 16

```

Generated C code :

```

1 typedef struct _BatteryState {
2   uint32_t cell_mv[16]; /* Static array */
3   pb_size_t cell_mv_count; /* Actual element count */
4 } BatteryState;

```

9.176.3 Bytes Fields

```

1  // File: firmware_update.proto
2  message FirmwareChunk {
3    bytes data = 1; // Binary firmware data
4  }
5
6  // File: firmware_update.options
7  FirmwareChunk.data max_size : 1024

```

Generated C code :

```

1  typedef struct _FirmwareChunk {
2    pb_bytes_array_t data; /* Contains uint8_t bytes[1024] */
3  } FirmwareChunk;

```

9.177 Sizing Guidelines

9.177.1 String Sizes

Table 55: Recommended String Field Sizes

Purpose	Size	Example
UUID(RFC 4122)	64 bytes	request_id, session_id
Error messages	128 bytes	error_message
Version strings	32 bytes	firmware_version
Short names	32 bytes	component_name
Log messages	256 bytes	log_text

9.177.2 Repeated Field Counts

Table 56: Recommended Repeated Field Counts

Purpose	Count	Example
Battery cells(3S LiPo)	3	cell_mv[]
Motors(quadraped)	4	motor_status[]
Motors(hexapod)	8	motor_status[]
Sensor array	8 –16	temperature_sensors[]
Telemetry samples	32 –64	velocity_samples[]

9.177.3 Bytes Field Sizes

Table 57: Recommended Bytes Field Sizes

Purpose	Size	Example
Firmware chunk	1024 bytes	chunk_data
Image thumbnail	4096 bytes	thumbnail
Small binary data	256 bytes	calibration_blob
Cryptographic key	64 bytes	aes_key

9.178 Memory Impact

9.178.1 Buffer Size Calculation

Each message allocates memory for the largest possible size:

```

1  /* Example: BatteryState with 3 cells (3S LiPo) */
2  typedef struct {
3      uint32_t cell_mv[3];      /* 3 * 4 = 12 bytes */
4      pb_size_t cell_mv_count; /* 4 bytes */
5      uint32_t current_ma;      /* 4 bytes */
6      uint8_t soc_percent;      /* 1 byte */
7      /* Total: ~21 bytes + padding */
8  } BatteryState;
9
10 /* Even if only 1 cell used, full array is allocated */

```

9.178.2 RAM Usage Example

```

1      /* Static allocation of message buffers */
2      static RequestHeader s_request;      /* 64 bytes */
3  static ResponseHeader s_response;      /* 128 bytes */
4  static BatteryState s_battery_state;    /* 24 bytes */
5  static FirmwareChunk s_firmware_chunk; /* 1024 bytes */
6
7  /* Total: ~1240 bytes of static RAM */

```

9.179 Best Practices

9.179.1 Right - Size Your Buffers

- **Don't over-allocate:** max_size:1024 for a 20-byte string wastes 1004 bytes
- **Don't under-allocate:** Truncated data causes silent bugs
- **Measure actual usage :** Profile message sizes in production

9.179.2 Use Fixed - Size Types

```

1      /* CORRECT - Fixed-size integer types */
2      message MotorCommand {
3          int32 speed_percent = 1; // Always 4 bytes
4          uint64 timestamp_us = 2; // Always 8 bytes
5      }
6
7      /* AVOID - Variable-length encoding */
8      message MotorCommand {
9          sint32 speed_percent = 1; // 1-5 bytes (varint encoding)
10     }

```

9.179.3 Validate Message Sizes

```

1      /* Check encoded message size before transmission */
2      uint8_t buffer[256];
3      pb_ostream_t stream = pb_ostream_from_buffer(buffer, sizeof(buffer));
4
5      bool status = pb_encode(&stream, RequestHeader_fields, &request);
6      if (!status) {
7          RX_LOG_ERROR(s_TAG, "Encoding failed: %s", PB_GET_ERROR(&stream));
8          return RX_ERR_FAIL;
9      }
10
11     size_t message_length = stream.bytes_written;
12     RX_LOG_INFO(s_TAG, "Encoded message: %zu bytes", message_length);
13
14     /* Verify it fits in SPI transaction buffer */
15     if (message_length > SPI_MAX_TRANSFER_SIZE) {
16         RX_LOG_ERROR(s_TAG, "Message too large for SPI: %zu > %d",
17             message_length,
18             SPI_MAX_TRANSFER_SIZE);
19         return ESP_ERR_INVALID_SIZE;
20     }

```

9.180 Integration with Code Generation

9.180.1 buf.gen.yaml Configuration

```

1 #File : star - proto / buf.gen.yaml
2     version : v2 plugins : -local : nanopb_generator out : gen /
3                                     nanopb_opt
4     : - --extension =.pb

```

9.180.2 Options File Naming

```

1 #Options file must match.proto file name
2     proto /
3     robot_command.proto->robot_command.options battery_state.proto
4     ->battery_state.options firmware_update.proto->firmware_update
5     .options

```

9.181 Common Pitfalls

9.181.1 Missing Options File

```

1      /* ERROR: No .options file for string field */
2      // robot_command.proto
3      message Command {
4          string name = 1; // No max_size specified
5      }
6
7      /* Generated code uses default (usually too small or error) */

```

Fix: Always create `.options` file for messages with strings, bytes, or repeated fields.

9.181.2 Mismatched Field Names

```

1  #WRONG - Field name mismatch
2  #robot_command.options
3      Command.command_name max_size : 32 #Field is 'name',
4      not 'command_name'
5
6  #CORRECT
7      Command.name max_size : 32

```

9.181.3 Forgetting Array Count

```

1      /* Common mistake: Forgetting to set count field */
2      BatteryState battery = {0};
3      battery.cell_mv[0] = 3700;
4      battery.cell_mv[1] = 3720;
5      battery.cell_mv[2] = 3710;
6      /* BUG: battery.cell_mv_count not set! Encoder sends 0 cells */
7
8      /* CORRECT */
9      battery.cell_mv_count = 3; /* Set actual element count (3S LiPo) */

```

9.182 Summary

- Always create `.options` files for strings, bytes, repeated fields
- Right-size buffers based on actual usage (measure, don't guess)
- Use fixed-size types to avoid varint encoding overhead
- Validate encoded sizes before transmission
- Set array count fields for repeated fields

Properly configured nanopb constraints enable reliable, predictable Protocol Buffer communication on resource-constrained embedded hardware.

```

=====
=====

```

9.183 NASA Power of 10: Rules for Safety-Critical Code

This section documents the NASA/JPL Power of 10 coding rules developed by Gerard J. Holzmann in 2006 for safety-critical embedded systems. These rules prioritize verification and predictability over coding flexibility.

STAR Compliance Philosophy: The STAR project follows the Power of 10 rules, with one **intentional architectural deviation** for Dependency Inversion Principle (DIP) – a conscious trade-off for testability and modularity.

9.183.1 Compliance Status Legend

Symbol	Meaning
✓ COMPLIANT	STAR follows this rule
△ !INTENTIONAL DEVIATION	Architectural choice(DIP pattern)

9.183.2 Rule 1 : Simplify Control Flow

Rule: Restrict all code to very simple control flow constructs – no goto, setjmp/longjmp, or recursion (direct or indirect).

Rationale: Creates predictable program structure for human understanding and static analysis. Recursion causes unbounded stack usage – dangerous in embedded systems.

STAR Compliance: ✓ **COMPLIANT**

```

1  /* COMPLIANT - No goto, no recursion */
2  esp_err_t
3  star_motor_init(star_motor_handle_t *handle,
4                  const star_motor_config_t *config) {
5  if (handle == NULL || config == NULL) {
6      return ESP_ERR_INVALID_ARG;
7  }
8
9  /* All control flow uses if/while/for */
10 esp_err_t ret = configure_timer(handle, config);
11 if (ret != ESP_OK) {
12     return ret;
13 }
14
15 return configure_gpio(handle, config);
16 }
```

9.183.3 Rule 2 : Fixed Loop Upper - Bounds

Rule : All loops must have a fixed upper bound that static analysis tools can trivially prove.

Rationale : Prevents runaway code, guarantees termination, enables real - time deadline verification.

STAR Compliance: ✓ **COMPLIANT**

```

1  /* COMPLIANT - Fixed iteration count */
2  #define MAX_RETRIES (3)
3
4  for (uint8_t i = 0; i < MAX_RETRIES; i++) {
5      if (sensor_read() == ESP_OK) {
6          break;
7      }
8      vTaskDelay(pdMS_TO_TICKS(100));
9  }
10
11 /* COMPLIANT - Array with known size */
12 #define BQ7850_MAX_CELLS (16)
13
14 for (uint8_t cell = 0; cell < BQ7850_MAX_CELLS; cell++) {
```

```

15     read_cell_voltage(cell, &voltages[cell]);
16 }

```

9.183.4 Rule 3 : No Dynamic Memory After Initialization

Rule : Prohibit `malloc` /`free` after initialization phase.

Rationale : Dynamic allocation causes unpredictable performance, memory leaks, fragmentation, and corruption.

STAR Compliance: ✓ **COMPLIANT**

Best Practice : Use pre - allocated static pools instead of runtime memory requests.

```

1  /* COMPLIANT - Static allocation with fixed limits */
2  #define MAX_ITEMS (8)
3  #define MAX_DESC_LEN (64)
4
5      typedef struct {
6          char items[MAX_ITEMS][MAX_DESC_LEN]; /* Static 2D array */
7          uint8_t item_count;
8      } static_pool_t;
9
10 /* Usage */
11 if (pool->item_count >= MAX_ITEMS) {
12     return ESP_ERR_NO_MEM;
13 }
14 strncpy(pool->items[pool->item_count], desc, MAX_DESC_LEN - 1);
15 pool->items[pool->item_count][MAX_DESC_LEN - 1] = '\0';
16 pool->item_count++;

```

9.183.5 Rule 4 : Keep Functions Short

Rule : Functions should not exceed ~60 lines(one printed page, one statement per line) .

Rationale : Represents single verifiable logical units; improves comprehension, review, and testability.

STAR Compliance: ✓ **MOSTLY COMPLIANT**

```

1      /* COMPLIANT - Single responsibility, short function */
2      esp_err_t
3      rx_motor_get_current_ma(rx_motor_handle_t *handle,
4                              float *current_ma) {
5          if (handle == NULL || current_ma == NULL || !handle->initialized) {
6              return ESP_ERR_INVALID_ARG;
7          }
8
9          int raw_value = 0;
10         esp_err_t ret = read_adc_channel(handle, &raw_value);
11         if (ret != ESP_OK) {
12             return ret;
13         }
14
15         *current_ma = (float)raw_value * handle->ki_propi / 1000.0F;
16         return ESP_OK;
17     }

```

9.183.6 Rule 5 : Use Assertions Generously

Rule : Minimum two assertions per function; assertions must be side - effect - free.

Rationale : Acts as executable documentation verifying pre - conditions, post - conditions, and invariants.

STAR Compliance: ✓ **COMPLIANT**

Note: STAR uses explicit parameter validation instead of assertions for production code (assertions disabled in release builds).

```

1  /* COMPLIANT - Explicit validation (better than assert for production) */
2  esp_err_t star_pid_compute(star_pid_handle_t* handle, float setpoint,
3                             float measurement, float dt, float* output)
4  {
5      /* Pre-condition checks (effectively runtime assertions) */
6      if (handle == NULL) {
7          return ESP_ERR_INVALID_ARG;
8      }
9      if (!handle->initialized) {
10         return ESP_ERR_INVALID_STATE;
11     }
12     if (output == NULL) {
13         return ESP_ERR_INVALID_ARG;
14     }
15     if (dt <= 0.0F || dt > 1.0F) { /* Sanity check time delta */
16         return ESP_ERR_INVALID_ARG;
17     }
18
19     /* Function logic */
20     float error = setpoint - measurement;
21     handle->integral += error * dt;
22
23     /* Post-condition: integral within bounds */
24     if (handle->integral > handle->integral_max) {
25         handle->integral = handle->integral_max;
26     } else if (handle->integral < handle->integral_min) {
27         handle->integral = handle->integral_min;
28     }
29
30     *output = handle->kp * error + handle->ki * handle->integral;
31     return ESP_OK;
32 }

```

9.183.7 Rule 6 : Declare Data at Smallest Scope

Rule : Minimize variable scope to reduce accidental corruption risks.

Rationale : Limits chances of accidental reference or corruption from other code.

STAR Compliance: ✓ **COMPLIANT**

```

1  /* COMPLIANT - Variables declared at minimal scope */
2  void
3  process_sensors(void) {
4      /* Loop variable declared in for statement */
5      for (uint8_t i = 0; i < SENSOR_COUNT; i++) {

```



```

6      /* Temporary variable for single sensor */
7      float temp_c = 0.0F;
8      if (read_sensor(i, &temp_c) == ESP_OK) {
9          /* Process immediately, temp_c out of scope after block */
10         update_telemetry(i, temp_c);
11     }
12 }
13 }

```

9.183.8 Rule 7 : Check All Return Values and Parameters

Rule : Validate all function returns and input parameters; treat warnings as errors.

Rationale : Forces explicit failure - condition handling rather than ignoring error codes.

STAR Compliance: ✓ **COMPLIANT**

```

1      /* COMPLIANT - All returns checked, all parameters validated */
2      esp_err_t
3      star_bus_manager_add_bus(star_bus_manager_t *manager,
4                              star_bus_config_t *config) {
5          /* Parameter validation */
6          if (manager == NULL || config == NULL) {
7              return ESP_ERR_INVALID_ARG;
8          }
9
10         /* Check return value and propagate errors */
11         esp_err_t ret = star_bus_config_init(config);
12         if (ret != ESP_OK) {
13             ESP_LOGE(TAG, "Bus initialization failed: %s", esp_err_to_name(ret));
14             return ret;
15         }
16
17         /* Mutex operation checked */
18         if (xSemaphoreTake(manager->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
19             ESP_LOGE(TAG, "Failed to acquire mutex");
20             return ESP_ERR_TIMEOUT;
21         }
22
23         /* Critical section */
24         add_to_list(manager, config);
25         xSemaphoreGive(manager->mutex);
26
27         return ESP_OK;
28     }

```

9.183.9 Rule 8 : Limit Preprocessor Use

Rule : Restrict to header inclusion and simple macros; forbid token pasting, recursive macros, and incomplete syntactic units.

Rationale : Preprocessor complexity obscures operations, fooling developers and static analysis tools.

STAR Compliance: ✓ **COMPLIANT**

```

1  /* COMPLIANT - Simple constant macros */
2  #define MAX_RETRY_COUNT (3)
3  #define MOTOR_PWM_FREQ_HZ (20000)
4  #define TIMEOUT_MS (1000)
5
6      /* COMPLIANT - Simple inline functions (better than complex macros) */
7      static inline float
8      voltage_mv_to_v(uint16_t mv) {
9      return (float)mv / 1000.0F;
10 }
11
12 /* AVOID - Complex token-pasting macros */
13 // #define CREATE_HANDLER(name) handler_##name##_init() /* Too complex */

```

9.183.10 Rule 9 : Restrict Pointer Use

Rule : Maximum one dereferencing level(*ptr OK, **ptr forbidden); no function pointers.

Rationale : Multiple indirection and function pointers make static data - flow analysis impossible.

STAR Compliance: △ !INTENTIONAL DEVIATION

Intentional Architectural Deviation: STAR uses function pointers for **Dependency Inversion Principle(DIP)** – an industry-standard pattern for testability and modularity. This is a conscious trade-off for better architecture.

DIP Pattern(Intentional Function Pointer Use) :

```

1      /* INTENTIONAL DEVIATION - DIP interface with function pointers */
2      typedef struct star_error_interface {
3      esp_err_t (*record_error)(void *ctx, esp_err_t error, const char
4          *message,
5          const char *file, int line, const char *func);
6      bool (*can_retry)(void *ctx);
7      esp_err_t (*reset_state)(void *ctx);
8      void *ctx; /* Implementation context */
9      } star_error_interface_t;
10
11 /* Why we keep this:
12  * 1. Enables mock implementations for unit testing
13  * 2. Allows swapping error handlers without recompilation
14  * 3. Follows SOLID principles (Dependency Inversion)
15  * 4. Industry-standard pattern (used in Linux kernel, RTOS, etc.)
16  */

```

9.183.11 Rule 10 : Compile with Maximum Warnings

Rule : Enable all compiler warnings at highest pedantry; achieve zero - warning builds with static analyzers .

Rationale : Modern compilers catch bugs before runtime.Zero warnings, no excuses.

STAR Compliance: ✓ COMPLIANT

```

1  #platformio.ini - Compiler flags
2      build_flags = -Wall #Enable all warnings - Wextra #Extra warnings -

```

```

3      Werror #Treat warnings as errors - Wno - unused -
4      parameter #Allow unused params in interface
5      implementations -
      Wno - missing - field - initializers

```

CI / CD Enforcement : Build fails on any compiler warning.

9.183.12 Summary: STAR Compliance Matrix

Table 58: NASA Power of 10 Compliance Status

Rule	Description	STAR Status
1	No <code>goto</code> / recursion	✓ COMPLIANT
2	Fixed loop bounds	✓ COMPLIANT
3	No dynamic memory	✓ COMPLIANT
4	Short functions(<60 lines)	✓ COMPLIANT
5	Use assertions	✓ COMPLIANT
6	Minimal variable scope	✓ COMPLIANT
7	Check all returns	✓ COMPLIANT
8	Limit preprocessor	✓ COMPLIANT
9	Restrict pointers	△ !INTENTIONAL(DIP)
10	Maximum warnings	✓ COMPLIANT

9.183.13 Defensive Coding Measures

Beyond the Power of 10 rules, STAR implements additional defensive coding measures for electrical noise resilience in the RX72N motor controller firmware.

Independent Watchdog Timer(IWDT) :

- 120 kHz dedicated oscillator (independent of main clock)
- 1 second timeout, fed every 4ms from 250Hz control loop
- Detects infinite loops, deadlocks, and system hangs
- Logs reset cause on startup for diagnostics

Register Guard(ESD / EMI Protection) :

- Captures “golden” register values after initialization
- Periodically refreshes critical registers (PORT PDR, MSTPCR)
- Recovers from transient bit-flips caused by electrical noise
- Tracks correction count for field diagnostics

IRQ Digital Filtering:

- Hardware noise rejection on interrupt pins
- Configurable filter clock divisor(PCLK / 1 to PCLK / 64)

- 3 - sample debounce prevents spurious interrupts

```

1      /* Defensive coding in motor control loop */
2      while (true) {
3      /* ... control logic ... */
4
5      rx_iwdt_feed(); /* Feed watchdog every 4ms */
6
7      if (++guard_counter >= 250) { /* Every 1 second */
8          rx_register_guard_refresh();
9          guard_counter = 0;
10     }
11 }

```

9.183.14 References

- Holzmann, Gerard J. “*The Power of 10: Rules for Developing Safety-Critical Code.*” IEEE Computer, 39(6), pp. 95-97, June 2006.
- JPL Institutional Coding Standard for the C Programming Language (JPL DOCID D-60411)
- MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems

10 ROS2 C++ Style Guide

10.1 Overview

The STAR project uses different coding conventions for ROS2 C++ code compared to RX72N firmware. This section documents the ROS2 C++ style guide, which supplements the C firmware style guide (Section ??).

10.1.1 When to Use This Guide

- **ROS2 packages** (`star - ros2 / src` /*/): Use ROS2 C++ style
- **RX72N firmware** (`star-rx72n-firmware/`): Use C firmware style
- **Gateway** (`star-gateway/`): Follow Go conventions

10.1.2 Key Differences

Feature	C Firmware (RX72N)	ROS2 C++
Classes	N/A	CamelCase
Methods	snake_case	snake_case (same)
Variables	snake_case	snake_case (same)
Member vars	s_ prefix	trailing _
Headers	.h	.hpp
Line limit	100 characters	120 characters
Namespaces	Not used	star::package_name::
Error handling	Return codes	Exceptions
Logging	uart_puts()	RCLCPP_INFO/WARN/ERROR
Include guards	STAR_RX72N_FILE_H	PACKAGE_FILE_HPP_
Doxygen	/** and /**<	/** and /**< (same)

Table 59: C vs C++ Style Comparison

10.2 Naming Conventions

10.2.1 Classes and Types

Use CamelCase for classes following ROS2 conventions:

```

1 // CamelCase for classes (ROS2 convention)
2 class StarGatewayBridgeNode : public rclcpp::Node {
3 // ...
4 };
5
6 // CamelCase for structs used as types
7 struct TelemetryData {
8 double battery_voltage_;
9 int32_t current_ma_;
10 };
11
12 // Type aliases use CamelCase
13 using TelemetryPtr = std::shared_ptr<TelemetryData>;

```

Listing 17: Class naming examples

10.2.2 Methods and Functions

Use snake_case for method names (same as C firmware):

```

1 // snake_case for methods (same as C firmware)
2 void publish_telemetry(const TelemetryData & data);
3 bool is_connected() const;
4 void on_battery_state_received(const
    sensor_msgs::msg::BatteryState::SharedPtr
5 msg);
6
7 // Use verb-based names that clarify actions
8 void check_for_errors(); // Good: Clear intent

```

```
9 void error_check();           // Bad: Noun-first is confusing
```

Listing 18: Method naming examples

10.2.3 Variables

Use `under_scored` for variables with trailing underscores for member variables:

```
1 // under_scored for variables
2 int loop_counter = 0;
3 std::string node_name = "gateway_bridge";
4
5 // Member variables with trailing underscore
6 class MyNode : public rclcpp::Node {
7 private:
8   rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
9   std::shared_ptr<grpc::Channel> grpc_channel_;
10  bool is_connected_;
11 };
12
13 // Constants: ALL_CAPITALS
14 const int MAX_RETRIES = 3;
15 const double DEFAULT_TIMEOUT_S = 5.0;
```

Listing 19: Variable naming examples

10.2.4 Namespaces

Package-based namespaces use `under_scored` naming:

```
1 namespace star { namespace spi_bridge {
2
3   class SpiDriverNode : public rclcpp::Node {
4     // ...
5   };
6
7 } // namespace spi_bridge
8 } // namespace star
```

Listing 20: Namespace examples

10.3 File Naming and Organization

10.3.1 Header Files

C++ headers use the `.hpp` extension (not `.h`):

- `star_gateway_bridge_node.hpp`
- `message_converter.hpp`
- `grpc_client.hpp`

Include guards follow the pattern: `PACKAGE_FILE_NAME_HPP_`

```
1 #ifndef STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
2 #define STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
3
4 // ... code ...
5
6 #endif // STAR_GATEWAY_BRIDGE_STAR_GATEWAY_BRIDGE_NODE_HPP_
```

Listing 21: Include guard example

10.3.2 Source Files

Source files use the `.cpp` extension with `under_scored` naming:

- `star_gateway_bridge_node.cpp`
- `message_converter.cpp`
- `grpc_client.cpp`

10.4 Header Organization

Headers must be organized in a standard order (enforced by `.clang-format`):

1. License and copyright
2. Include guard
3. ROS2 core includes (`<rclcpp/...>`)
4. ROS2 message includes (`<geometry_msgs/...>`, etc.)
5. Project includes (`"star/..."`)
6. System C++ includes (`<memory>`, `<string>`, etc.)
7. System C includes (`<stdint>`, etc.)
8. Namespace declaration
9. Class/function declarations

See `CLAUDE.md` for complete example.

10.5 Code Formatting

10.5.1 Line Length

- **ROS2 C++:** 120 characters (`star-ros2/.clang-format`)
- **C firmware:** 100 characters (`star-rx72n-firmware/.clang-format`)

10.5.2 Indentation

All code uses 2-space indentation (same as C firmware):

```

1 class MyNode : public rclcpp::Node { public: MyNode() :
2 Node("my_node") { timer_ = this->create_wall_timer(
3 std::chrono::milliseconds(100),
4 std::bind(&MyNode::timerCallback, this));
5 }
6
7 private:
8 void timerCallback() {
9 // ...
10 }
11
12 rclcpp::TimerBase::SharedPtr timer_;
13 };

```

Listing 22: Indentation example

10.5.3 Braces

- **Functions:** Braces on new line (same as C firmware)
- **Control statements:** Cuddled braces (if (x) {})
- **Namespaces:** No indentation inside namespace

```

1 // Functions: Braces on new line
2 void myFunction()
3 {
4 // ...
5 }
6
7 // Control statements: Cuddled braces
8 if (condition) {
9 // ...
10 } else {
11 // ...
12 }
13
14 // Namespaces: No indentation inside
15 namespace star {
16 namespace spi_bridge {
17
18 // Content at zero indent
19 class MyClass {
20 };
21
22 } // namespace spi_bridge
23 } // namespace star

```

Listing 23: Brace style examples

10.6 ROS2-Specific Patterns

10.6.1 Node Inheritance

Inherit from `rclcpp::Node` for basic nodes or `rclcpp_lifecycle::LifecycleNode` for safety-critical nodes:

```

1 // Basic node
2 class StarGatewayBridgeNode : public rclcpp::Node {
3 public:
4   StarGatewayBridgeNode();
5
6 private:
7   void telemetryCallback();
8
9   rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
10  rclcpp::TimerBase::SharedPtr telemetry_timer_;
11 };
12
13 // Safety-critical lifecycle node
14 class StarSpiDriverNode : public rclcpp_lifecycle::LifecycleNode {
15 public:
16   using CallbackReturn =
17     rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;
18
19   StarSpiDriverNode();
20
21 // Lifecycle transitions
22   CallbackReturn on_configure(const rclcpp_lifecycle::State &) override;
23   CallbackReturn on_activate(const rclcpp_lifecycle::State &) override;
24   CallbackReturn on_deactivate(const rclcpp_lifecycle::State &) override;
25   CallbackReturn on_cleanup(const rclcpp_lifecycle::State &) override;
26 };

```

Listing 24: Node inheritance

10.6.2 Publishers and Subscribers

```

1 class MyNode : public rclcpp::Node { public: MyNode() :
2   Node("my_node") {
3     // Publisher: use SharedPtr
4     telemetry_pub_ = this->create_publisher<std_msgs::msg::String>(
5       "/telemetry",
6       10 // QoS depth
7     );
8
9     // Subscriber: use std::bind
10    cmd_sub_ = this->create_subscription<geometry_msgs::msg::Twist>(
11      "/cmd_vel",
12      10,
13      std::bind(&MyNode::cmdVelCallback, this, std::placeholders::_1)
14    );
15  }
16 }

```

```

17 private:
18 void cmdVelCallback(const geometry_msgs::msg::Twist::SharedPtr msg) {
19     // Handle message
20 }
21
22 rclcpp::Publisher<std_msgs::msg::String>::SharedPtr telemetry_pub_;
23 rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_sub_;
24 };

```

Listing 25: Publishers and subscribers

10.6.3 Timers

Use `create_wall_timer` for periodic operations:

```

1 timer_ = this->create_wall_timer(
2     std::chrono::milliseconds(100), // 10 Hz
3     std::bind(&MyNode::timerCallback, this)
4 );

```

Listing 26: Timer usage

10.7 Error Handling

ROS2 uses exceptions (different from C firmware return codes):

```

1 // Throw exceptions for errors
2 void publishTelemetry()
3 {
4     if (!grpc_channel_) {
5         throw std::runtime_error("gRPC channel not initialized");
6     }
7     // ...
8 }
9
10 // Catch exceptions in callbacks (avoid crashing node)
11 void timerCallback()
12 {
13     try {
14         publishTelemetry();
15     } catch (const std::exception & e) {
16         RCLCPP_ERROR(this->get_logger(), "Failed to publish: %s", e.what());
17     }
18 }

```

Listing 27: Exception handling

10.8 Logging

Use `RCLCPP_*` macros for logging, not `printf` or `cout`:

```

1 // ROS2 logging macros (preferred)
2 RCLCPP_INFO(this->get_logger(), "Node started");
3 RCLCPP_WARN(this->get_logger(), "Connection lost, retrying...");

```

```

4 RCLCPP_ERROR(this->get_logger(), "Failed to initialize: %s",
   error_msg.c_str());
5 RCLCPP_DEBUG(this->get_logger(), "Processing message %d", count);
6
7 // Throttled logging (max once per 5 seconds)
8 RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
9 "Command stale (%ldms > %dms)", cmd_age_ms, timeout_ms_);

```

Listing 28: ROS2 logging

Important: Never use `printf()` or `std::cout` in ROS2 nodes.

10.9 Documentation

All public APIs must have Doxygen comments using `/**` and `/**<` (same as C firmware):

```

1 /**
2  * @brief Brief description of class
3  *
4  * Detailed description can span multiple lines. Explain purpose,
5  * usage, and any important details.
6  */
7     class StarGatewayBridgeNode : public rclcpp::Node {
8 public:
9     /**
10    * @brief Constructor for gateway bridge node
11    *
12    * @param options Node options for configuration
13    */
14    explicit StarGatewayBridgeNode(
15        const rclcpp::NodeOptions &options = rclcpp::NodeOptions());
16
17 private:
18     /**
19    * @brief Callback for telemetry publishing timer
20    *
21    * Forwards latest telemetry to Gateway via gRPC. Called at 10 Hz.
22    */
23    void telemetry_timer_callback();
24
25    rclcpp::TimerBase::SharedPtr telemetry_timer_; /**< Timer for
26        telemetry */
27 };

```

Listing 29: Doxygen documentation

10.10 Constants

Prefer enums and `const` (same philosophy as C firmware) :

```

1 // Enum for related constants
2 enum class MotorState {
3     kIdle = 0,
4     kRunning = 1,
5     kError = 2

```

```

6      };
7
8      // const for single values
9      const int DEFAULT_QOS_DEPTH = 10;
10     const double MAX_LINEAR_VELOCITY_MPS = 1.0;
11
12     // static const for class-specific constants
13     class MyNode : public rclcpp::Node {
14     private:
15         static constexpr int kMaxRetries = 3;
16         static constexpr double kTimeoutS = 5.0;
17     };

```

Listing 30: Constants

10.11 Formatting Enforcement

All ROS2 C++ code uses `clang - format` with configuration in `star - ros2 /.clang - format` :

- 2-space indentation (same as C firmware)
- 120-character line limit (ROS2 standard)
- Cuddled braces for control statements
- Function braces on new line
- No indentation inside namespaces
- ROS2-specific include order (core, messages, system, project)

10.11.1 Manual Formatting

Format code before committing :

```

cd star -
  ros2 find src - name '*.cpp' - o - name '*.hpp' |
  xargs clang - format - i

```

10.11.2 CI Enforcement

The `.github / workflows / ros2.yml` workflow runs `clang - format` checks on all PRs :

```

find src -
  name '*.cpp' - o - name '*.hpp' |
  xargs clang - format-- dry -
  run-- Werror

```

If formatting check fails, the build will fail with instructions to fix.

10.12 References

- **ROS2 C++ Style Guide** : <https://docs.ros.org/en/rolling/The-ROS2-Project/Contributing/Code-Style-Language-Versions.html>
- **ROS C++ Best Practices** : <https://wiki.ros.org/CppStyleGuide>
- **Google C++ Style Guide** : <https://google.github.io/styleguide/cppguide.html>
- **STAR C Firmware Style** : CLAUDE.md Section "Code Style"
- **STAR Project Documentation** : docs / star_documentation.pdf

10.13 Integration with Existing Tools

The ROS2 C++ style guide integrates seamlessly with existing STAR project tools :

Tool	Purpose	Configuration
clang - format	Code formatting	star - ros2 /.clang - format
ament_cppcheck	Static analysis	ROS2 workflow
ament_cpplint	Style checking	ROS2 workflow
colcon build	Build system	CMakeLists.txt
colcon test	Unit testing	gtest integration
GitHub Actions	CI / CD enforcement	.github / workflows / ros2.yml

Table 60: ROS2 Development Tools

10.14 Summary

The STAR project maintains separate but complementary style guides for C firmware and C++ ROS2 code:

- **C firmware** follows embedded best practices with strict NASA Power of 10 compliance
- **ROS2 C++** follows ROS community conventions while maintaining STAR project philosophy
- Both styles share common principles : enums over macros, error checking, documentation
- Automated tooling enforces consistency across all code

For complete examples and additional details, refer to CLAUDE.md.

Part III

Gateway & ROS2

```
== == == == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == == == == == == == == == == == == == == == == == == ==
```

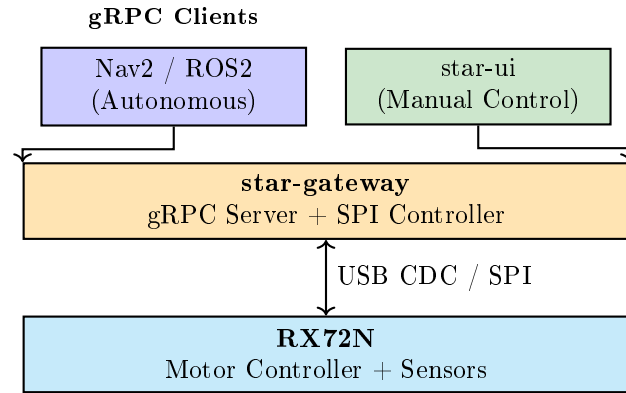
11 Gateway Architecture

This section documents the `star-gateway` Go service that bridges the Raspberry Pi 5 control stack with the RX72N motor controller.

11.1 Overview

The gateway service runs on the Raspberry Pi 5 and serves as the communication bridge between:

- **High-level side:** gRPC clients (Nav2/ROS2 for autonomous navigation, UI for manual control)
- **Low-level side:** RX72N motor controller via USB CDC (primary) or SPI (backup)



11.2 Protocol Stack Implementation

The gateway implements all layers of the communication protocol(see Section ??) .

Table 61: Gateway Protocol Stack Implementation

Layer	Name	Go Package	Status
5	Application	<code>internal/service/</code>	Implemented
4	Serialization	<code>star-proto/gen/go/</code>	Implemented
3	HARQ + FEC	<code>internal/link/</code> , <code>internal/harq/</code> , <code>internal/fec/</code>	Implemented
2	Framing	<code>internal/frame/</code>	Implemented
1	Transport	<code>internal/transport/</code>	Implemented

11.3 Directory Structure

```

star - gateway / +--cmd / | +--star - gateway / | |
  +--main.go #Entry point | +--virtual_rx72n / |
  +--main.go #RX72N simulator + --internal / |
  +--transport / #Layer 1 : Physical transport | |
  +--spi.go #SPI transport(periph.io) | | +--cdc.go #USB CDC transport | |
  +--socket.go #Socket transport(simulation) |
  +--frame / #Layer 2 : Frame protocol | | +--frame.go #Constants and types |
  | +--encoder.go #Frame encoder | | +--decoder.go #Stream decoder |
  +--link / #Layer 3 : Reliability(HARQ) | | +--spi.go #SPILink(HARQ) |
  +--harq / #HARQ interface and types | | +--harq.go | +--fec / #FEC codec | |
  
```

```

+--convolutional.go #Rate 1 / 2,
K = 7 encoder | | +--viterbi.go #Viterbi decoder | |
  +--combiner.go #Chase Combiner | +--manager / #Transport management | |
  +--manager.go #TransportManager + SessionState |
  +--service / #Layer 5 : gRPC services | | +--motor_control.go | |
  +--telemetry.go | | +--battery.go | | +--configuration.go | |
  +--gateway.go #UI bridge service | +--app / #Application wiring | |
  +--gateway.go | +--server / #gRPC server |
  +--server.go + --go.mod + --CLAUDE.md

```

11.4 Frame Package

The `internal / frame` package defines the wire protocol constants and types.

11.4.1 Frame Constants

Table 62: Frame Protocol Constants

Constant	Value	Description
SyncWord	0x55AA	Frame sync marker(alternating bits : [0x55, 0xAA])
SyncSize	2 bytes	Sync word size
HeaderSize	6 bytes	SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1)
CRCSize	4 bytes	CRC - 32 checksum
MaxPayloadSize	1024 bytes	Maximum protobuf payload
MaxFrameSize	1036 bytes	Total frame size
MinFrameSize	12 bytes	Empty payload frame

11.4.2 Frame Types

- `FrameTypeCommand` –Command from controller to peripheral
- `FrameTypeResponse` –Response from peripheral to controller
- `FrameTypeAck` –Acknowledgment of successful receipt
- `FrameTypeNack` –Negative acknowledgment(CRC error, etc.)

11.5 Transport Package

The `internal/transport` package provides physical transport implementations for communicating with the RX72N.

11.5.1 Available Transports

Table 63: Transport Implementations

Transport	Device	Speed	Usage
SPI	/ dev / spidev0 .0	10 MHz	Backup(priority 5)
USB CDC	/ dev / ttyACMO	12 Mbps(Full - Speed)	Primary(priority 10)
Socket	Unix socket	N / A	Simulation mode

11.5.2 SPI Configuration

Table 64: SPI Configuration Constants

Parameter	Value	Description
Device	/ dev / spidev0 .0	SPI device path
Speed	10 MHz	SPI clock frequency
Mode	0	CPOL = 0, CPHA = 0
Bits per word	8	Word size
Timeout	100 ms	Default operation timeout

11.5.3 Transport Interface

```

type Transport interface {
    Send(data[] byte)(int, error) Receive(maxLen int)([] byte, error)
    Transfer(txData[] byte)([] byte, error) Close() error
}

```

11.6 Link Layer (HARQ)

The `internal/link` package implements the reliability layer with two link types optimized for their respective transports.

11.6.1 SPILink

`SPILink` provides full Hybrid ARQ(HARQ) with forward error correction for the SPI transport, where the physical channel is more error-prone.

- **HARQ with ACK / NACK** –Retransmission on negative acknowledgment or timeout
- **FEC encoding / decoding** –Convolutional code($K = 7$, Rate $1 / 2$) with Viterbi decoding
- **Chase Combining** – Soft-combining of retransmitted frames for improved error recovery

11.6.2 CDCLink

`CDCLink` provides a lightweight link layer for USB CDC, since USB handles reliability at the hardware level.

- **No HARQ** –USB protocol provides its own error correction and retransmission
- **Sequence tracking** – Maintains sequence numbers for ordering

11.6.3 Shared Session State

Both link types use the shared `SessionState` from `internal / manager /` to maintain sequence number continuity across transport failovers.

11.6.4 HARQ Parameters

Table 65: HARQ Protocol Parameters

Parameter	Value	Description
Max Retries	3	Maximum transmission attempts
ACK Timeout	10 ms	ACK wait timeout
Sequence Max	0xFFFF	16 - bit wraparound
FEC Codec	Convolutional K = 7, Rate 1 / 2	Forward error correction

11.7 Transport Manager

The `internal / manager` package provides priority - based transport selection and automatic failover.

11.7.1 Transport Priorities

Table 66: Transport Manager Priority Configuration

Transport	Priority	Role
USB CDC	10	Primary transport
SPI	5	Backup transport

11.7.2 Failover Behavior

- **Shared SessionState** –Sequence continuity maintained across transports
- **Automatic failover** –Health monitoring detects transport failures
- **Primary timeout** –200 ms implicit timeout before failover
- **Secondary probe** –1 s PING probe to detect recovery of higher - priority transport

11.8 gRPC Services

The gateway exposes five gRPC services (see Section ?? for detailed specifications):

Table 67: Gateway gRPC Services

Service	Description
MotorControlService	Motor velocity commands, emergency stop, and control loop configuration
TelemetryService	Real-time sensor data streaming (encoders, IMU, battery)
BatteryManagementService	Battery monitoring, state of charge, and safety limits
ConfigurationService	Runtime parameter updates and system configuration
FirmwareUpdateService	Over-the-air firmware update for RX72N controller

11.9 Build and Test

```
#Build the gateway binary
    cd star -
    gateway go build./ cmd / star -
    gateway

#Run all tests
    go test./
    ...

#Run with verbose output
    go test -
    v./
    ...

#Format code
    go fmt./
    ...

#Static analysis
    go vet./
    ...
```

11.10 Dependencies

The gateway depends on :

- [github.com / Locked - Inc / star - proto](https://github.com/Locked-Inc/star-proto) / `gen / go` -Generated protobuf and gRPC code
- [google.golang.org / grpc](https://google.golang.org/grpc) -gRPC framework
- [google.golang.org / protobuf](https://google.golang.org/protobuf) - Protocol Buffers runtime

The `go.work` file at the repository root enables workspace mode for seamless cross-module development:

```
go 1.21

use (
    ./star-proto/gen/go
    ./star-proto/tests/go
    ./star-gateway
)
```

For the complete end - to - end communication path from UI to motor controller, see Section ??.

```
== == == == == == == == == == == == == == == == == == == == == == ==
== == == == == == == == == == == == == == == == == == == == == == ==
```

12 ROS2 Integration and Sensor Fusion Architecture

12.1 Overview

The STAR platform utilizes ROS2 Jazzy Jalisco as the primary high-level orchestration layer. The integration strategy focuses on providing a modular, reliable, and deterministic environment for SLAM, navigation, and mission control. This section details the architecture for Visual-LiDAR sensor fusion, coordinate frame definitions, and custom node interfaces.

12.2 Visual-LiDAR Sensor Fusion with RTAB-Map

The platform implements a heterogeneous sensor fusion strategy combining geometric data from LiDAR and appearance-based data from an RGB-D camera using **RTAB-Map** (Real-Time Appearance-Based Mapping).

12.2.1 Fusion Architecture

The fusion process is divided into two main layers:

1. **Local State Estimation(EKF)** : The `robot_localization` package runs an Extended Kalman Filter(EKF) fusing :
 - Wheel Odometry(from RX72N via `star_spi_bridge`)
 - IMU Linear Acceleration and Angular Velocity(from OAK - D or dedicated IMU)
2. **Global SLAM and Loop Closure(RTAB - Map)** : The `rtabmap_ros` node consumes :
 - Laser Scan data from RPLiDAR C1.
 - RGB - D images and PointClouds from the OAK - D camera.
 - Filtered Odometry from the EKF.

12.2.2 Mapping Strategy

RTAB-Map performs loop closure detection using a bag-of-words approach. When a loop is detected, it optimizes the pose graph to minimize drift. The resulting map is published as a 2D occupancy grid for Nav2 and a 3D octomap for obstacle avoidance.

12.3 Coordinate Frames (REP-105)

The STAR platform adheres to the **REP-105** standard for coordinate frames. The transform tree (`tf2`) is structured as follows:

- **map**: The global coordinate frame. Fixed to the world.
- **odom**: The local coordinate frame. Drift-prone, but continuous. The `map` $\rightarrow$ `odom` transform is published by RTAB-Map (Loop Closure).
- **base_link**: The center of the robot's drive axis. The `odom` $\rightarrow$ `base_link` transform is published by the EKF.
- **laser_frame**: The reference frame for the RPLiDAR C1.
- **camera_link**: The reference frame for the OAK-D Depth Camera.

12.4 Custom ROS2 Nodes and Interfaces

Three primary custom nodes facilitate the interaction between the STAR hardware and the ROS2 ecosystem.

12.4.1 `star_spi_bridge`

Bridges the low - level SPI protocol(RX72N) to ROS2 topics.

- **Subscribed Topics:** / `cmd_vel`(`geometry_msgs / Twist`)
- **Published Topics:** / `odom` / `unfiltered`(`nav_msgs / Odometry`), / `joint_states`(`sensor_msgs / JointState`)
- **Parameters:** `spi_device`, `spi_speed_hz`, `gear_ratio`

12.4.2 `star_gateway_bridge`

Provides a bridge between the gRPC / WebSocket gateway and the ROS2 computational graph.

- **Subscribed Topics:** / `robot_status`, / `battery_state`
- **Published Topics:** /`teleop/cmd_vel` (overrides for manual control)
- **Services:** / `set_pid_gains`, / `trigger_calibration`

12.4.3 `star_safety_monitor`

Ensures platform integrity and manages emergency states.

- **Logic :** Monitors battery voltage, motor current, and heartbeat from the RPi5 and RX72N.
- **Actions :** Publishes / `emergency_stop` or resets the motor controller if a timeout occurs.

12.5 Multi-Machine Communication

To ensure isolation and prevent interference in multi-robot environments, the `ROS_DOMAIN_ID` environment variable must be set uniquely for each STAR unit (e.g., `export ROS_DOMAIN_ID=42`).

12.6 Hardware Persistence (udev rules)

Consistent device naming is critical for reliable startup. The following `udev` rules are required in `/etc/udev/rules.d/99-star.rules`:

```
#RPLiDAR C1
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", SYMLINK+="ttyLidar"
#RX72N Motor Controller(via USB - Serial if applicable, or SPI identification)
SUBSYSTEM=="spidev", KERNEL=="spidev0.0", GROUP=="spi", MODE=="0660"
```

Note : The `spi` group must exist on the system before applying the `udev` rules .The STAR Docker container automatically creates this group via `groupadd - f spi` during build.On bare - metal installations, create the group manually :

```
sudo groupadd spi sudo usermod - aG spi $USER
```

Part IV

API Reference – RX72N Firmware (C)

RX72N Firmware API Reference

This section contains the complete auto-generated API reference for the STAR RX72N motor controller firmware. Generated from source code comments using Doxygen.

Source: `e2-studio-star-rx72n-firmware/src/`

Language: C (GNURX compiler, C23 features)

RTOS: ThreadX

Generator: Doxygen 1.16.1

STAR RX72N Firmware

API Reference

Renesas RX72N Motor Controller Firmware

ThreadX RTOS – GNURX Compiler

Auto-generated from source code by Doxygen + star-docs

February 2026

STAR Development Team – Locked Inc.

Contents

RX72N Firmware API Reference	55
rx_bus_adc.h	55
rx_bus_adc_init	55
rx_bus_adc_read	59
rx_bus_adc_read_voltage_mv	63
rx_bus_command.h	68
rx_bus_config_t	68
rx_bus_command_execute_fn	68
rx_bus_command_init	69
rx_bus_config.h	72
rx_bus_config_init_gpio	72
rx_bus_config_init_adc	77
rx_bus_config_init_i2c	82
rx_bus_config_init_onewire	85
rx_bus_config_init_uart	88
rx_bus_gpio.h	92
rx_bus_gpio_init	92
rx_bus_gpio_write	96
rx_bus_gpio_read	99
rx_bus_gpio_toggle	103
rx_bus_i2c.h	107
rx_bus_i2c_init	107
rx_bus_i2c_write	109
rx_bus_i2c_read	112
rx_bus_i2c_write_read	115
rx_bus_manager.h	118
rx_bus_callback_t	118
rx_bus_manager_init	119
rx_bus_manager_deinit	120
rx_bus_manager_add_bus	123
rx_bus_manager_remove_bus	125
rx_bus_manager_find_bus	127
rx_bus_manager_with_bus	130
rx_bus_manager_execute_command	132
rx_bus_onewire.h	132
onewire_timing_us_t	133
onewire_rom_command_t	134
onewire_rom_size_t	135
rx_bus_onewire_init	135
rx_bus_onewire_deinit	137
rx_bus_onewire_reset	138
rx_bus_onewire_write_bit	139
rx_bus_onewire_read_bit	140
rx_bus_onewire_write_byte	141
rx_bus_onewire_read_byte	142
rx_bus_onewire_write	143
rx_bus_onewire_read	144

rx_bus_owewire_skip_rom	145
rx_bus_owewire_match_rom	146
rx_bus_owewire_read_rom	146
rx_bus_owewire_search	147
rx_bus_types.h	148
rx_bus_type_t	149
rx_riic_limits_t	149
rx_rspi_limits_t	149
rx_adc_limits_t	149
rx_sci_limits_t	149
rx_adc_channel_limits_t	150
rx_adc_resolution_t	150
rx_i2c_constants_t	150
bit_masks_t	150
bus_manager_limits_t	150
bus_manager_timeout_t	151
rx_bus_adc.c	151
adc_voltage_limits_t	151
s_tag	151
internal_adc_init_callback	151
internal_adc_read_callback	155
internal_adc_voltage_callback	158
rx_bus_adc_init	161
rx_bus_adc_read	163
rx_bus_adc_read_voltage_mv	166
rx_bus_config.c	168
rx_port_t	168
rx_pin_t	169
s_tag	169
internal_validate_port_pin	169
rx_bus_config_init_gpio	170
rx_bus_config_init_adc	171
rx_bus_config_init_i2c	174
rx_bus_config_init_uart	176
rx_bus_config_init_owewire	178
rx_bus_gpio.c	179
s_tag	180
internal_gpio_init_callback	180
internal_gpio_write_callback	183
internal_gpio_read_callback	185
internal_gpio_toggle_callback	188
rx_bus_gpio_init	191
rx_bus_gpio_write	193
rx_bus_gpio_read	195
rx_bus_gpio_toggle	198
rx_bus_i2c.c	200
i2c_validation_constants_t	200
s_tag	200
internal_i2c_init_callback	200

internal_i2c_write_callback	203
internal_i2c_read_callback	206
internal_i2c_write_read_callback	209
rx_bus_i2c_init	212
rx_bus_i2c_write	213
rx_bus_i2c_read	214
rx_bus_i2c_write_read	215
rx_bus_manager.c	216
s_tag	216
internal_execute_command_callback	217
rx_bus_manager_init	217
rx_bus_manager_deinit	218
rx_bus_manager_add_bus	221
rx_bus_manager_remove_bus	223
rx_bus_manager_find_bus	225
rx_bus_manager_with_bus	228
rx_bus_manager_execute_command	230
rx_bus_owewire.c	230
CHECK_ONEWIRE_BUS	231
owewire_driver_constants_t	231
owewire_delay_hw_constants_t	232
owewire_delay_tick_constants_t	233
s_tag	233
s_owewire_max_search_devices	233
s_state_pool	233
s_delay_timer_initialized	233
internal_delay_timer_init	233
internal_delay_us	236
internal_acquire_state	237
internal_get_state	239
internal_set_drive_mode	240
internal_drive_low	240
internal_release_line	241
internal_read_line	241
internal_reset_search_state	242
internal_reset_pulse	242
internal_write_bit	244
internal_read_bit	245
internal_write_byte	247
internal_read_byte	248
internal_search_step	248
internal_search_bits	249
internal_search_iteration	250
internal_owewire_init_callback	252
internal_release_state	253
internal_owewire_deinit_callback	254
internal_owewire_reset_callback	255
internal_owewire_write_bit_callback	256
internal_owewire_read_bit_callback	257

internal_owewire_write_byte_callback	258
internal_owewire_read_byte_callback	259
internal_owewire_write_buffer_callback	260
internal_owewire_read_buffer_callback	261
internal_owewire_skip_rom_callback	263
internal_owewire_match_rom_callback	264
internal_owewire_read_rom_callback	265
internal_owewire_search_callback	268
rx_bus_owewire_init	269
rx_bus_owewire_deinit	271
rx_bus_owewire_reset	272
rx_bus_owewire_write_bit	273
rx_bus_owewire_read_bit	274
rx_bus_owewire_write_byte	275
rx_bus_owewire_read_byte	276
rx_bus_owewire_write	277
rx_bus_owewire_read	278
rx_bus_owewire_skip_rom	279
rx_bus_owewire_match_rom	280
rx_bus_owewire_read_rom	281
rx_bus_owewire_search	282
rx_comm_manager.h	283
rx_comm_manager_constants_t	284
rx_comm_heartbeat_constants_t	284
rx_comm_event_retry_constants_t	284
rx_comm_channel_t	285
rx_comm_link_status_t	285
rx_comm_frame_callback_t	285
rx_comm_link_status_callback_t	286
rx_comm_manager_init	286
rx_comm_manager_deinit	290
rx_comm_manager_poll	292
rx_comm_manager_send	294
rx_comm_manager_respond	298
rx_comm_manager_stream_send	300
rx_comm_manager_event_send	301
rx_comm_manager_link_status	303
rx_comm_manager_channel_ready	304
rx_comm_manager_channel_name	307
rx_comm_manager_process_retransmits	308
rx_comm_manager_set_auto_retransmit	309
rx_comm_manager.c	310
rx_comm_sim_time_t	310
internal_constants_t	310
s_tag	311
s_zero_mgr	311
internal_get_time_ms	311
internal_output_decoded	312
internal_handle_frame	313

internal_poll_usb	316
internal_poll_spi	317
internal_process_event_queue	321
internal_check_heartbeat	323
rx_comm_manager_init	324
rx_comm_manager_deinit	328
rx_comm_manager_poll	330
rx_comm_manager_send	332
rx_comm_manager_respond	336
rx_comm_manager_stream_send	338
rx_comm_manager_event_send	338
rx_comm_manager_link_status	341
rx_comm_manager_channel_ready	342
rx_comm_manager_channel_name	345
rx_comm_manager_process_retransmits	346
rx_comm_manager_set_auto_retransmit	347
rx_bit_constants.h	348
rx_bit_sizes_t	348
rx_bit_shifts_t	349
rx_check.h	349
RX_ASSERT	349
RX_ERROR_CHECK	350
RX_ERROR_CHECK_WITHOUT_ABORT	350
RX_RETURN_ON_ERROR	350
RX_RETURN_VOID_ON_ERROR	351
RX_RETURN_NULL_ON_ERROR	351
RX_CHECK_NULL_PTR	351
RX_CHECK_RANGE	352
RX_CHECK_RANGE_TAG	352
RX_VALIDATE_PTR	352
RX_VALIDATE_INIT	352
internal_rx_fatal_error	353
rx_err.h	353
rx_err_codes_t	353
rx_err_t	355
rx_err_from_threadx	355
rx_err_is_ok	355
rx_err_is_error	356
rx_error_handler.h	356
error_handler_limits_t	356
error_handler_init	357
error_handler_get_interface	361
error_handler_deinit	365
rx_error_interface.h	368
rx_error_report_fn	368
rx_error_get_count_fn	369
rx_error_get_component_count_fn	369
rx_error_clear_fn	369
rx_error_is_retry_limit_reached_fn	370

rx_error_reset_retry_fn	370
rx_error_get_backoff_delay_fn	370
rx_error_interface_validate	371
rx_exception.h	373
rx_exception_type_t	373
rx_exception_vector_offset_t	374
rx_exception_init	374
rx_exception_get_stats	375
rx_exception_get_name	376
rx_exc_undefined_instruction_handler	377
rx_exc_privileged_instruction_handler	377
rx_exc_access_handler	377
rx_exc_address_handler	377
rx_exc_floating_point_handler	378
rx_exc_nmi_handler	378
rx_gpio_constants.h	378
rx_gpio_constants_t	378
rx_error_handler_defaults_t	379
rx_prcr_constants_t	379
rx_infrastructure.h	379
RX_REPORT_ERROR	380
RX_RESERVE_PIN	380
RX_RELEASE_PIN	380
rx_infrastructure_init	381
rx_infrastructure_deinit	383
rx_infrastructure_get_error_interface	384
rx_infrastructure_get_pin_interface	384
rx_iwdt.h	384
iwdt_constants_t	385
iwdt_timeout_period_t	385
system_state_t	386
iwdt_reset_reason_t	386
rx_iwdt_init	387
rx_iwdt_feed	389
rx_iwdt_register_task	391
rx_iwdt_task_heartbeat	394
rx_iwdt_set_timeout_for_state	396
rx_iwdt_set_state	398
rx_iwdt_get_status	400
rx_iwdt_was_reset	402
rx_iwdt_check_tasks	404
rx_log.h	406
USB_LOG_MIRROR	406
LOG_PUTC	406
LOG_PUTS	406
LOG_PUTINT	406
LOG_PUTHEX	407
LOG_LEVEL	407
RX_LOG_VAL_IMPL	407

rx_log_error	407
rx_log_error_val	407
rx_log_error_hex	407
rx_log_error_str	407
rx_log_warn	408
rx_log_warn_val	408
rx_log_warn_hex	408
rx_log_warn_str	408
rx_log_info	408
rx_log_info_val	408
rx_log_info_hex	408
rx_log_info_str	408
rx_log_debug	408
rx_log_debug_val	409
rx_log_debug_hex	409
rx_log_debug_str	409
rx_log_verbose	409
rx_log_verbose_val	409
rx_log_verbose_hex	409
rx_log_verbose_str	409
log_level_t	409
log_format_constants_t	410
uart_debug_putc	410
uart_debug_puts	411
uart_debug_putint	412
uart_debug_puthex	414
rx_log_usb_putc	416
rx_log_usb_puts	416
rx_log_usb_putint	417
rx_log_usb_puthex	418
rx_log_usb_get_stats	418
rx_log_usb_notify_ready	419
internal_log_header	420
internal_rx_log_error	422
internal_rx_log_error_u8	422
internal_rx_log_error_u16	423
internal_rx_log_error_u32	423
internal_rx_log_error_i32	424
internal_rx_log_error_err	424
internal_rx_log_error_hex	425
internal_rx_log_error_str	425
internal_rx_log_warn	426
internal_rx_log_warn_u8	426
internal_rx_log_warn_u16	427
internal_rx_log_warn_u32	427
internal_rx_log_warn_i32	428
internal_rx_log_warn_err	428
internal_rx_log_warn_hex	429
internal_rx_log_warn_str	430

internal_rx_log_info	430
internal_rx_log_info_u8	431
internal_rx_log_info_u16	431
internal_rx_log_info_u32	432
internal_rx_log_info_i32	432
internal_rx_log_info_err	433
internal_rx_log_info_hex	433
internal_rx_log_info_str	434
internal_rx_log_debug	434
internal_rx_log_debug_u8	435
internal_rx_log_debug_u16	435
internal_rx_log_debug_u32	436
internal_rx_log_debug_i32	436
internal_rx_log_debug_err	437
internal_rx_log_debug_hex	437
internal_rx_log_debug_str	438
internal_rx_log_verbose	438
internal_rx_log_verbose_u8	439
internal_rx_log_verbose_u16	439
internal_rx_log_verbose_u32	440
internal_rx_log_verbose_i32	440
internal_rx_log_verbose_err	441
internal_rx_log_verbose_hex	441
internal_rx_log_verbose_str	442
rx_pin_interface.h	442
pin_interface_limits_t	442
rx_pin_validate_fn	443
rx_pin_reserve_fn	443
rx_pin_release_fn	443
rx_pin_is_reserved_fn	444
rx_pin_get_function_fn	444
rx_pin_clear_all_fn	444
rx_pin_interface_validate	445
rx_pin_validator.h	445
pin_validator_limits_t	445
pin_validator_init	445
pin_validator_get_interface	446
pin_validator_deinit	447
rx_port_constants.h	448
rx_port_number_t	448
rx_pin_number_t	449
port_encoding_t	449
rx_port_pin_t	450
rx_port_from_pin	454
rx_pin_from_pin	456
rx_register_guard.h	458
rx_register_guard_config_t	458
rx_register_guard_init	458
rx_register_guard_refresh	462

rx_register_guard_get_correction_count	466
rx_register_guard_reset_count	469
rx_register_guard_is_initialized	471
rx_register_protection.h	473
rx_prcr_values_t	474
rx_simulator_config.h	474
RX_IS_SIMULATOR	475
simulator_clock_timing_t	475
rx_threadx_config.h	475
s_rx_threadx_tick_rate_hz	476
rx_time_constants.h	476
rx_time_conversion_t	476
rx_time_interface.h	476
rx_time_sleep_ms_fn	476
rx_time_get_ms_fn	477
rx_time_is_elapsed_fn	479
rx_time_threadx_get_interface	480
rx_time_interface_validate	483
rx_wdt.h	484
wdt_timeout_cycles_t	484
wdt_clock_division_t	484
rx_wdt_init	485
rx_wdt_start	490
rx_wdt_stop	493
rx_wdt_feed	495
rx_error_handler.c	500
error_handler_backoff_constants_t	500
internal_find_component	500
internal_find_or_create_component	502
impl_report_error	503
impl_get_error_count	505
impl_get_component_error_count	506
impl_clear_errors	507
impl_is_retry_limit_reached	509
impl_reset_retry_counter	511
impl_get_backoff_delay	512
error_handler_init	514
error_handler_get_interface	516
error_handler_deinit	518
rx_exception.c	520
s_log_tag	520
s_exception_names	520
s_exception_stats	520
s_initialized	521
rx_exc_undefined_instruction_c_handler	521
rx_exc_privileged_instruction_c_handler	521
rx_exc_access_c_handler	522
rx_exc_address_c_handler	523
rx_exc_floating_point_c_handler	523

rx_exc_nmi_c_handler	524
internal_process_exception	525
internal_halt_cpu	526
rx_exception_init	527
rx_exception_get_stats	528
rx_exception_get_name	529
rx_infrastructure.c	530
s_global_error_handler	530
s_global_pin_validator	530
s_global_error_interface	530
s_global_pin_interface	531
s_infrastructure_initialized	531
rx_infrastructure_init	531
rx_infrastructure_deinit	534
rx_infrastructure_get_error_interface	535
rx_infrastructure_get_pin_interface	536
rx_iwdt.c	537
s_iwdt_state	538
internal_get_tick_count	538
internal_find_task	539
internal_find_free_slot	540
internal_init_default_config	541
rx_iwdt_init	542
rx_iwdt_feed	544
rx_iwdt_register_task	546
rx_iwdt_task_heartbeat	548
rx_iwdt_set_timeout_for_state	550
rx_iwdt_set_state	552
rx_iwdt_get_status	554
rx_iwdt_was_reset	556
rx_iwdt_check_tasks	557
rx_iwdt.c	559
iwdt_timeout_limits_t	559
iwdt_init_state_t	560
iwdt_buffer_size_constants_t	560
s_timeout_table	561
s_iwdt_timeout_table_size	561
s_tag	561
s_iwdt_initialized	561
internal_find_timeout_config	561
internal_configure_iwdt_control_register	563
internal_configure_iwdt_registers	564
rx_hal_iwdt_init	566
rx_hal_iwdt_feed	569
rx_hal_iwdt_was_reset	570
rx_hal_iwdt_get_reset_cause	571
rx_hal_iwdt_clear_status	573
rx_log_usb.c	574
usb_log_buffer_config_t	575

s_boot_buffer	575
s_usb_ready	575
s_stats	575
s_log_mutex	575
s_mutex_initialized	575
internal_init_mutex_once	575
internal_buffer_boot_log	576
internal_check_usb_ready	578
internal_write_usb	579
rx_log_usb_putc	579
rx_log_usb_puts	580
rx_log_usb_putint	581
rx_log_usb_puthex	581
rx_log_usb_get_stats	581
rx_log_usb_notify_ready	582
rx_pin_validator.c	583
s_zero_validator	583
internal_port_to_index	583
internal_validate_port	584
internal_validate_pin	585
impl_validate_pin	585
impl_reserve_pin	586
impl_release_pin	587
impl_is_pin_reserved	588
impl_get_pin_function	588
impl_clear_all_reservations	589
pin_validator_init	590
pin_validator_get_interface	591
pin_validator_deinit	592
rx_register_guard.c	593
register_guard_defaults_t	593
s_state	594
internal_capture_pdr	594
internal_capture_mstpcr	597
internal_refresh_pdr	598
internal_refresh_mstpcr	600
rx_register_guard_init	602
rx_register_guard_refresh	604
rx_register_guard_get_correction_count	606
rx_register_guard_reset_count	607
rx_register_guard_is_initialized	609
rx_time_threadx.c	610
rx_time_threadx_zero_t	610
rx_time_ceil_offset_t	610
impl_sleep_ms	611
floor(ms / 10) + (1 if ms mod 10 > 0, else 0)	612
impl_get_ms	612
impl_is_elapsed	613
rx_time_threadx_get_interface	615

rx_wdt.c	617
s_wdt_state	617
rx_wdt_init	617
rx_wdt_start	620
rx_wdt_stop	621
rx_wdt_feed	623
rx_crc.h	626
rx_crc_init	626
rx_crc_deinit	629
rx_crc32_ieee	631
rx_crc32_update	634
rx_crc8_maxim	637
rx_crc_internal.h	640
RX_CRC32_USE_HARDWARE	640
rx_crc_shared_constants_t	640
rx_crc_init	640
rx_crc_deinit	642
rx_crc32_ieee_impl	643
rx_crc32_update_impl	646
rx_crc32_update_sw	647
rx_crc32.c	649
rx_crc32_ieee	649
rx_crc32_update	650
rx_crc32_hw.c	650
rx_crc_hw_constants_t	651
rx_crc_hw_loop_constants_t	651
s_crc_initialized	651
rx_crc_init	651
rx_crc_deinit	653
rx_crc32_ieee_impl	653
rx_crc32_update_impl	656
rx_crc32_sw.c	657
rx_crc32_sw_constants_t	657
s_crc32_table	657
rx_crc32_update_sw	657
rx_crc8.c	659
rx_crc8_constants_t	659
rx_crc8_maxim	659
rx_ds18b20.h	660
ds18b20_command_t	660
ds18b20_scratchpad_index_t	661
ds18b20_config_bits_t	662
ds18b20_resolution_t	662
ds18b20_conversion_time_ms_t	662
ds18b20_temp_mask_t	662
ds18b20_conversion_constants_t	663
ds18b20_write_idx_t	663
ds18b20_init_values_t	663
ds18b20_power_mode_t	663

rx_ds18b20_init	664
rx_ds18b20_deinit	666
rx_ds18b20_trigger_conversion	667
rx_ds18b20_read_temperature	669
rx_ds18b20_read_temperature_raw	671
rx_ds18b20_set_resolution	674
rx_ds18b20_get_resolution	677
rx_ds18b20_save_config	678
rx_ds18b20_recall_config	681
rx_ds18b20_read_power_mode	684
rx_ds18b20_read_scratchpad	687
rx_ds18b20_get_conversion_time_ms	688
rx_ds18b20.c	690
CHECK_DS18B20_HANDLE	690
internal_ds18b20_validate_config	690
internal_ds18b20_verify_device_presence	692
internal_ds18b20_select_device	694
internal_ds18b20_read_scratchpad_raw	696
internal_ds18b20_write_scratchpad	699
internal_ds18b20_resolution_to_config	703
internal_ds18b20_get_temp_mask	703
internal_ds18b20_raw_to_celsius	704
rx_ds18b20_init	705
rx_ds18b20_deinit	707
rx_ds18b20_trigger_conversion	708
rx_ds18b20_read_temperature	710
rx_ds18b20_read_temperature_raw	712
rx_ds18b20_set_resolution	714
rx_ds18b20_get_resolution	718
rx_ds18b20_save_config	719
rx_ds18b20_recall_config	722
rx_ds18b20_read_power_mode	724
rx_ds18b20_read_scratchpad	727
rx_ds18b20_get_conversion_time_ms	728
rx_encoder_tpu.h	729
rx_tpu_encoder_init	730
rx_tpu_encoder_read_raw	732
rx_tpu_encoder_read_count	733
rx_tpu_encoder_read_velocity	735
rx_tpu_encoder_reset	737
rx_tpu_encoder_set_count	740
rx_tpu_encoder_deinit	742
rx_mtu_encoder.h	744
rx_encoder_init	744
rx_encoder_read_raw	746
rx_encoder_read_count	748
rx_encoder_read_velocity	749
rx_encoder_reset	754
rx_encoder_set_count	757

rx_encoder_deinit	759
rx_encoder_tpu.c	762
tpu_enc_channel_limits_t	762
tpu_enc_counter_t	762
tpu_enc_params_t	762
tpu_enc_velocity_t	763
s_tag	763
k_tpu_enc_initial_position_deg	763
k_tpu_enc_min_delta_time_s	763
k_tpu_enc_max_delta_time_s	763
s_initialized	763
s_state	763
s_counts_per_rev	763
s_invert_direction	763
s_last_count	763
internal_is_valid_channel	764
internal_update_state	764
rx_tpu_encoder_init	767
rx_tpu_encoder_read_raw	769
rx_tpu_encoder_read_count	770
rx_tpu_encoder_read_velocity	772
rx_tpu_encoder_reset	774
rx_tpu_encoder_set_count	777
rx_tpu_encoder_deinit	779
rx_mtu_encoder.c	781
encoder_constants_t	781
encoder_calc_constants_t	781
encoder_init_values_t	781
encoder_velocity_limits_t	782
s_tag	782
k_encoder_initial_position_deg	782
k_min_delta_time_s	782
k_max_delta_time_s	782
s_encoder_initialized	782
s_encoder_state	782
s_counts_per_rev	782
s_invert_direction	782
s_last_count	783
internal_enable_mtu_module	783
internal_configure_encoder_timer	785
internal_verify_timer_counting	786
internal_initialize_encoder_state	786
internal_update_state_from_count	788
internal_is_valid_channel	789
internal_get_mtu_base	789
rx_encoder_init	791
rx_encoder_read_raw	795
rx_encoder_read_count	797
rx_encoder_read_velocity	798

rx_encoder_reset	803
rx_encoder_set_count	806
rx_encoder_deinit	808
rx_fec.h	811
rx_fec_params_t	811
rx_soft_bit_limits_t	811
rx_fec_buffer_limits_t	812
rx_soft_bit_t	812
rx_fec_encoder_init	812
rx_fec_encoder_deinit	813
rx_fec_encode	815
rx_fec_encoded_len	817
rx_fec_decoder_init	817
rx_fec_decoder_deinit	819
rx_fec_decode_soft	819
rx_fec_decode_hard	822
rx_fec_hard_to_soft	826
rx_fec_soft_to_hard	826
rx_fec.c	826
rx_fec_impl_constants_t	827
rx_fec_output_index_t	827
internal_parity	827
internal_set_output_bit	828
internal_get_bit	829
internal_encode_bit	829
internal_init_branch_table	830
internal_branch_metric	831
internal_viterbi_process_symbol	832
internal_viterbi_forward_pass	833
internal_viterbi_traceback	834
rx_fec_encoder_init	835
rx_fec_encoder_deinit	836
rx_fec_encoded_len	836
rx_fec_encode	837
internal_validate_decode_params	840
rx_fec_decoder_init	841
rx_fec_decoder_deinit	843
rx_fec_decode_soft	843
rx_fec_decode_hard	846
rx_frame.h	850
rx_frame_constants_t	850
rx_frame_type_t	850
rx_frame_flags_t	851
rx_le16_byte_idx_t	851
rx_byte_order_constants_t	851
rx_le32_byte_idx_t	852
rx_le32_shift_t	852
rx_frame_encoder_init	852
rx_frame_encoder_deinit	853

rx_frame_encode	854
rx_frame_encoded_size	856
rx_frame_read_le16	856
rx_frame_write_le16	857
rx_frame_read_le32	858
rx_frame_decoder_init	858
rx_frame_decoder_deinit	859
rx_frame_decode	860
rx_frame_decode_with_resync	862
rx_frame_create_ack	866
rx_frame_create_nack	868
rx_frame_create_ping	869
rx_frame_create_pong	872
rx_frame_create_reset	874
rx_frame_create_reset_ack	876
rx_frame_type_valid	877
rx_frame.c	878
byte_shift_t	879
init_state_t	879
frame_offset_t	879
frame_resync_discarded_t	880
internal_write_le32	880
internal_read_le32	881
internal_decode_header	883
internal_verify_crc	886
internal_find_sync_offset	887
rx_frame_encoder_init	889
rx_frame_encoder_deinit	890
rx_frame_encode	891
rx_frame_decoder_init	893
rx_frame_decoder_deinit	893
rx_frame_decode	894
rx_frame_decode_with_resync	896
rx_frame_create_ack	898
rx_frame_create_nack	899
rx_frame_create_ping	901
rx_frame_create_pong	903
rx_frame_create_reset	904
rx_frame_create_reset_ack	907
rx_frame_ascii.h	908
rx_frame_ascii_constants_t	909
rx_frame_ascii_format	909
rx_frame_ascii_type_name	912
rx_frame_ascii_flags_str	915
rx_frame_ascii.c	917
format_constants_t	917
buffer_size_constants_t	918
decimal_buf_constants_t	919
s_hex_chars	919

rx_frame_ascii_type_name	919
rx_frame_ascii_flags_str	920
rx_frame_ascii_format	921
hardware.h	923
adc_unit_t	923
adc_channel_t	923
rspi_mode_t	924
rspi_channel_t	924
rspi_freq_constants_t	924
uart_defaults_t	925
uart_channel_t	925
adc_resolution_t	925
gpio_set_output	925
gpio_set_input	926
gpio_write_high	927
gpio_write_low	928
gpio_toggle	929
gpio_read	930
timer_init	931
timer_stop	933
timer_get_count	934
adc_init	938
adc_read	941
adc_read_voltage_mv	943
riic_init	944
riic_write	947
riic_read	949
riic_write_read	951
rspi_init_peripheral	953
rspi_peripheral_transfer	955
rspi_peripheral_read_available	958
rspi_peripheral_write_ready	959
rspi_deinit	961
rspi_init_controller	964
rspi_controller_set_cs	966
rspi_controller_transfer_16bit	968
rspi_controller_deinit	971
uart_init_channel	975
uart_deinit_channel	978
uart_putc_channel	980
uart_puts_channel	983
uart_write_channel	985
uart_getc_channel	987
uart_read_channel	991
uart_rx_available	994
uart_debug_init	997
uart_debug_putc	998
uart_debug_puts	999
uart_debug_putint	1001

uart_debug_puthex	1002
rx72n_adc_regs.h	1003
rx72n_clock.h	1004
rx_clock_frequencies_t	1004
rx72n_cmt_regs.h	1004
rx72n_crc_regs.h	1005
rx_crc_addresses_t	1005
rx_crc_reserved_sizes_t	1005
rx_crc_crccr_shifts_t	1005
rx_crc_crccr_masks_t	1006
rx_crc_crccr_bits_t	1006
crc_regs	1007
rx72n_dmac_regs.h	1007
dmac_addresses_t	1008
dmac_offsets_t	1008
dmac_channel_t	1009
dmtmd_bits_t	1009
dmint_bits_t	1010
dmamd_bits_t	1010
dmcnt_bits_t	1011
dmreq_bits_t	1011
dmsts_bits_t	1012
dmcs1_bits_t	1012
dmast_bits_t	1012
dmist_bits_t	1012
dmac_mstpcr_t	1013
dmac_channel	1013
dmac0	1014
dmac1	1014
dmac2	1015
dmac3	1015
dmac4	1016
dmac5	1016
dmac6	1016
dmac7	1017
dmac_dmast_reg	1017
dmac_dmist_reg	1018
rx72n_dtc_regs.h	1018
dtc_addresses_t	1018
dtc_offsets_t	1019
dtccr_bits_t	1019
dtcadmod_bits_t	1020
dtcst_bits_t	1020
dtcsts_bits_t	1020
dtcor_bits_t	1020
dtcsqe_bits_t	1021
dtc_mra_bits_t	1021
dtc_mrb_bits_t	1022
dtc_mstpcr_t	1022

dtc_dtccr_reg	1022
dtc_dtcvbr_reg	1023
dtc_dtcadmod_reg	1023
dtc_dtcst_reg	1024
dtc_dtcsts_reg	1024
dtc_dtcibr_reg	1025
dtc_dtcdr_reg	1025
dtc_dtcscr_reg	1025
dtc_dtcdisp_reg	1026
rx72n_elc_regs.h	1026
elc_addresses_t	1027
elcr_bits_t	1028
elsr_event_t	1029
elop_timer_op_t	1031
elopa_bits_t	1031
elopb_bits_t	1031
elopc_bits_t	1032
elopd_bits_t	1032
pgc_bits_t	1032
pel_bits_t	1033
elsegr_bits_t	1033
elc_mstpcr_t	1033
elc_elcr_reg	1034
elc_elsr_reg	1034
elc_elsr0_reg	1034
elc_elsr15_reg	1035
elc_elsr45_reg	1035
elc_elsr48_reg	1036
elc_elopa_reg	1036
elc_elopb_reg	1036
elc_pgr1_reg	1037
elc_pgr2_reg	1037
elc_pgc1_reg	1037
elc_pgc2_reg	1038
elc_pdbf1_reg	1038
elc_pdbf2_reg	1039
elc_pel_reg	1039
elc_elsegr_reg	1039
rx72n_flash_regs.h	1040
rx_rom_cache_addresses_t	1040
rx_romce_bits_t	1040
rx_romciv_bits_t	1041
rx_fweprror_addresses_t	1041
rx_fweprror_bits_t	1041
rx_faci_addresses_t	1042
rx_eepfclk_addresses_t	1042
rx_fastat_bits_t	1042
rx_faeint_bits_t	1043
rx_frdyie_bits_t	1043

rx_fstatr_bits_t	1043
rx_fentryr_bits_t	1045
rx_fsuinitr_bits_t	1045
rx_fcldr_bits_t	1046
rx_faci_commands_t	1046
rx_fpckar_bits_t	1046
rx_fbcnt_bits_t	1047
rx_fbcstat_bits_t	1047
rx_uidr_addresses_t	1047
rx_flash_memory_addresses_t	1048
romce_reg	1048
romciv_reg	1048
fwepror_reg	1049
fastat_reg	1049
faeint_reg	1050
frdyie_reg	1050
fstatr_reg	1050
fentryr_reg	1051
fsuinitr_reg	1051
fsaddr_reg	1052
feaddr_reg	1052
fcldr_reg	1052
faci_cmd_area	1053
fpckar_reg	1053
eeptclk_reg	1054
fbcnt_reg	1054
fbstat_reg	1054
fpsaddr_reg	1055
fawmon_reg	1055
fcpsr_reg	1056
fsuacr_reg	1056
uidr0_reg	1056
uidr1_reg	1057
uidr2_reg	1057
uidr3_reg	1058
rx72n_gptw_regs.h	1058
rx72n_icu_regs.h	1058
rx72n_iwdt_regs.h	1059
rx_iwdt_addresses_t	1059
iwdt_reserved_sizes_t	1059
iwdt_refresh_sequence_t	1059
iwdt_iwdtcr_shifts_t	1060
iwdt_iwdtcr_bits_t	1060
iwdt_iwdtsr_bits_t	1061
iwdt_iwdtrcr_bits_t	1062
iwdt_iwdtcstpr_shifts_t	1062
iwdt_iwdtcstpr_bits_t	1062
iwdt	1062
rx72n_lpc_regs.h	1064

rx_lpc_addresses_t	1064
rx_lpc_sizes_t	1065
rx_opccr_bits_t	1065
rx_rstckcr_bits_t	1066
rx_dpsbycr_bits_t	1066
rx_dpsier0_bits_t	1066
rx_dpsier1_bits_t	1067
rx_dpsier2_bits_t	1067
rx_dpsier3_bits_t	1068
rx_dpsifr0_bits_t	1068
rx_dpsifr1_bits_t	1068
rx_dpsifr2_bits_t	1069
rx_dpsifr3_bits_t	1069
rx_dpsiegr0_bits_t	1069
rx_dpsiegr1_bits_t	1070
rx_dpsiegr2_bits_t	1070
rx_dpsiegr3_bits_t	1071
opccr_reg	1071
rstckcr_reg	1071
dps_regs	1072
dpsbkr	1072
rx72n_lvda_regs.h	1073
rx_lvda_addresses_t	1073
rx_lvd_cr1_bits_t	1074
rx_lvd_sr_bits_t	1074
rx_lvcmpcr_bits_t	1074
rx_lvdlvr_bits_t	1075
rx_lvd_cr0_bits_t	1075
rx_lvd_interrupts_t	1076
lvd1cr1_reg	1076
lvd1sr_reg	1077
lvd2cr1_reg	1077
lvd2sr_reg	1078
lvcmpcr_reg	1078
lvdlvr_reg	1079
lvd1cr0_reg	1079
lvd2cr0_reg	1080
rx72n_mpc_regs.h	1080
rx72n_mpu_regs.h	1081
mpu_addresses_t	1081
mpu_region_t	1082
mpu_page_t	1082
rspage_bits_t	1083
repage_bits_t	1083
mpen_bits_t	1084
mpbac_bits_t	1085
mpeclr_bits_t	1085
mpests_bits_t	1085
mpops_bits_t	1086

mpopi_bits_t	1086
mhiti_bits_t	1086
mhيتد_bits_t	1087
mpu_region	1088
mpu_rspage_reg	1089
mpu_repage_reg	1089
mpu_mpen_reg	1090
mpu_mpbac_reg	1090
mpu_mpeclr_reg	1091
mpu_mpests_reg	1091
mpu_mpdea_reg	1091
mpu_mpsa_reg	1092
mpu_mpops_reg	1092
mpu_mpopi_reg	1093
mpu_mhiti_reg	1093
mpu_mhitd_reg	1093
mpu_addr_to_page	1094
mpu_page_to_reg	1094
rx72n_mtu_regs.h	1095
rx72n_poe3_regs.h	1095
poe3_addresses_t	1096
poe3_offsets_t	1097
icsr_poemode_t	1097
icsr_bits_t	1098
icsr6_bits_t	1098
ocsr_bits_t	1098
spoer_bits_t	1099
poecr1_bits_t	1099
poecr2_bits_t	1099
poecr4_bits_t	1100
poecr5_bits_t	1100
alr1_bits_t	1100
poe3_mstpcr_t	1101
poe3_icsr1_reg	1101
poe3_icsr2_reg	1101
poe3_icsr3_reg	1102
poe3_icsr4_reg	1102
poe3_icsr5_reg	1102
poe3_icsr6_reg	1103
poe3_ocsr1_reg	1103
poe3_ocsr2_reg	1104
poe3_spoer_reg	1104
poe3_poecr1_reg	1104
poe3_poecr2_reg	1105
poe3_poecr4_reg	1105
poe3_poecr5_reg	1106
poe3_alr1_reg	1106
poe3_m0selr1_reg	1106
poe3_m0selr2_reg	1107

poe3_m3selr_reg	1107
poe3_m4selr1_reg	1107
poe3_m4selr2_reg	1108
poe3_m6selr_reg	1108
rx72n_poeg_regs.h	1108
rx_poeg_addresses_t	1109
rx_poegg_bit_shifts_t	1109
rx_poegg_bits_t	1110
rx_poeg_interrupts_t	1111
poegga	1112
poeggb	1113
poeggc	1113
poeggd	1114
rx72n_port_regs.h	1114
rx72n_ram_regs.h	1115
rx_ram_region_addr_t	1115
rx_ram_reg_addr_t	1116
rx_ram_reg_offset_t	1116
rx_exram_reg_offset_t	1116
rx_eccram_reg_offset_t	1116
rx_rammode_bits_t	1117
rx_ramsts_bits_t	1117
rx_ramprcr_bits_t	1117
rx_exrammode_bits_t	1118
rx_exramsts_bits_t	1118
rx_exramprcr_bits_t	1118
rx_eccrammode_bits_t	1119
rx_eccram2sts_bits_t	1119
rx_eccram1sts_bits_t	1119
rx_eccramprcr_bits_t	1120
rx_eccramprcr2_bits_t	1120
rx_eccrametst_bits_t	1120
rx_ram_mstpcrc_bits_t	1121
ram_regs	1121
exram_regs	1121
eccram_regs	1122
rx72n_regs.h	1122
rx72n_riic_regs.h	1123
rx72n_rsipi_regs.h	1123
rx_rsipi_addresses_t	1124
rsipi_spcr_bits_t	1124
rsipi_spsr_bits_t	1124
rsipi_sppcr_bits_t	1125
rsipi_spdcr_bits_t	1125
rsipi0	1126
rsipi1	1126
rsipi2	1127
rx72n_rtc_regs.h	1128

rtc_addresses_t	1128
rtc_offsets_t	1128
rcr3_bits_t	1129
rtc_rcr3_reg	1130
rtc_rcr1_reg	1130
rtc_rcr2_reg	1131
rtc_rcr4_reg	1131
rx72n_sci_regs.h	1132
rx72n_system_regs.h	1132
rx_system_addresses_t	1133
system_reserved_sizes_t	1133
mdmonr_bits_t	1133
syscr0_bits_t	1134
syscr1_bits_t	1134
ckocr_bits_t	1135
sckcr_divider_t	1136
sckcr_bits_t	1136
sckcr3_cksel_t	1137
pllcr_bits_t	1138
pllcr2_bits_t	1139
rx_module_stop_bits_a_t	1140
rx_module_stop_bits_b_t	1140
rx_rstsr_addresses_t	1140
rstsr0_bits_t	1141
rstsr1_bits_t	1141
rstsr2_bits_t	1141
rx_swrr_addresses_t	1141
rx_swrr_values_t	1141
rx_memwait_addresses_t	1142
rx_memwait_bits_t	1142
rx_ppll_addresses_t	1142
rx_ppll_config_t	1142
rx_ppll_control_t	1142
rx_ppll_flags_t	1143
system_regs	1143
prcr_reg	1145
rstsr01	1147
rstsr2	1147
swrr_reg	1148
memwait_reg	1148
pllcr_reg	1149
pllcr2_reg	1149
rx72n_temps_regs.h	1149
temps_addresses_t	1149
tscr_bits_t	1150
temps_timing_t	1150
temps_calibration_t	1150
temps_mstpocr_t	1151
temps_tscr_reg	1151

temps_tscdr_reg	1152
rx72n_tpu_regs.h	1153
rx_tpu_addresses_t	1153
rx_tpu_channel_count_t	1154
rx_tpu_tstr_bits_t	1154
rx_tpu_tsyrr_bits_t	1154
rx_tpu_tcr_shift_t	1155
rx_tpu_tcr_cclr_t	1155
rx_tpu_tcr_masks_t	1156
rx_tpu_tmdr_md_t	1156
rx_tpu_tmdr_shift_t	1156
rx_tpu_tmdr_masks_t	1157
rx_tpu_tier_bits_t	1157
rx_tpu_tsr_bits_t	1158
rx_tpu_tsr_reserved_t	1158
rx_tpu_nfcr_bits_t	1158
rx_tpu_nfcr_clock_t	1159
rx_tpu_nfcr_masks_t	1159
rx_tpu_mstpcra_t	1159
rx_tpu_intb_source_t	1160
tpu_control	1161
tpu0	1162
tpu1	1163
tpu2	1164
tpu3	1164
tpu4	1165
tpu5	1166
rx72n_usb_regs.h	1166
usb_register_offsets_t	1166
usb_reserved_sizes_t	1168
rx_usb_addresses_t	1168
usb_syscfg_bits_t	1168
usb_syssts0_bits_t	1169
usb_dvstctr0_bits_t	1169
usb_intenb0_bits_t	1169
usb_intsts0_bits_t	1170
usb_dcpcfg_bits_t	1170
usb_dcpcctr_bits_t	1171
usb_pipe_bits_t	1171
usb_pipecfg_bits_t	1172
usb_pipectr_bits_t	1172
usb_fifoel_bits_t	1173
usb_fifoctr_bits_t	1173
rx_usb_interrupt_vector_t	1173
usb_cdc_endpoints_t	1173
usb0	1174
rx72n_wdt_regs.h	1176
rx_wdt_addresses_t	1176
wdt_reserved_sizes_t	1176

wdt_refresh_sequence_t	1176
wdt_wdtrcr_pos_t	1177
wdt_wdtrcr_bits_t	1177
wdt_wdtsr_pos_t	1178
wdt_wdtsr_bits_t	1178
wdt_wdtrcr_bits_t	1179
wdt	1179
rx_cmt.h	1180
rx_cmt_channel_t	1180
rx_cmt_callback_t	1181
rx_cmt_init	1181
rx_cmt_start	1183
rx_cmt_stop	1186
rx_cmt_get_count	1189
rx_cmt_deinit	1192
rx_gptw.h	1195
rx_gptw_channel_t	1195
rx_gptw_output_t	1195
rx_gptw_wave_mode_t	1196
rx_gptw_duty_calc_constants_t	1196
rx_gptw_channel_id	1197
rx_gptw_output_id	1198
rx_gptw_init_all_staggered	1199
rx_gptw_init_pwm	1201
rx_gptw_set_duty	1202
rx_gptw_set_duty_raw	1203
rx_gptw_get_duty	1206
rx_gptw_get_period	1210
rx_gptw_enable_output	1212
rx_gptw_start	1214
rx_gptw_stop	1216
rx_gptw_deinit	1218
rx_hal_iwdt.h	1221
rx_iwdt_timeout_t	1221
rx_iwdt_reset_cause_t	1221
rx_iwdt_limits_t	1222
rx_hal_iwdt_init	1222
rx_hal_iwdt_feed	1225
rx_hal_iwdt_was_reset	1226
rx_hal_iwdt_get_reset_cause	1228
rx_hal_iwdt_clear_status	1230
rx_host_irq.h	1231
rx_host_irq_init	1232
rx_host_irq_deinit	1234
rx_host_irq_assert	1236
rx_host_irq_deassert	1237
rx_host_irq_is_asserted	1238
rx_irq_filter.h	1240
rx_irq_filter_limits_t	1240

rx_irq_filter_clk_t	1240
s_irq_max	1241
rx_irq_filter_enable	1241
rx_irq_filter_disable	1243
rx_irq_filter_is_enabled	1245
rx_irq_filter_get_clock	1246
rx_mpc.h	1248
rx_pin_function_t	1249
rx_pin_psel_t	1249
rx_mpc_set_gpio	1250
rx_mpc_set_peripheral	1253
rx_mpc_set_mtu_pwm	1257
rx_mpc_set_mtu_encoder	1259
rx_mpc_set_tpu_encoder	1261
rx_mpc_set_adc	1263
rx_mpc_set_sci	1266
rx_mpc_set_riic	1267
rx_mpc_set_rspi	1269
rx_mpc_set_gptw	1271
rx_mpc_set_usb_vbus	1274
rx_mpc_set_irq	1276
rx_mtu.h	1279
rx_mtu_channel_t	1279
rx_mtu_output_t	1280
rx_mtu_init_pwm	1280
rx_mtu_set_duty	1283
rx_mtu_set_duty_raw	1286
rx_mtu_get_duty	1288
rx_mtu_get_period	1290
rx_mtu_enable_output	1292
rx_mtu_start	1295
rx_mtu_stop	1297
rx_mtu_deinit	1299
rx_poeg.h	1302
rx_poeg_motor_count_t	1302
rx_poeg_isr_priority_t	1302
rx_poeg_init	1302
rx_poeg_get_fault_status	1305
rx_poeg_clear_fault	1306
rx_poeg_software_stop	1309
rx_poeg_clear_software_stop	1310
rx_port_utils.h	1311
rx_port_get_base	1311
rx_tpu.h	1316
rx_tpu_channel_t	1316
rx_tpu_phase_mode_t	1316
rx_tpu_init_phase_count	1317
rx_tpu_start	1320
rx_tpu_stop	1321

rx_tpu_read_count	1324
rx_tpu_read_direction	1326
rx_tpu_reset_count	1329
rx_tpu_deinit	1330
adc.c	1334
adc_constants_t	1334
adc_bit_constants_t	1334
adc_module_stop_bits_t	1334
adc_adcer_bits_t	1335
adc_adcsr_bits_t	1335
adc_channel_boundaries_t	1335
adc_misc_constants_t	1336
adc_raw_value_t	1336
s_tag	1336
s_adc_unit_initialized	1336
s_adc_unit_resolution	1336
internal_get_adc_base	1337
internal_configure_adc_unit	1338
internal_validate_unit_channel	1340
internal_read_channel_value	1340
adc_init	1342
adc_read	1344
adc_read_voltage_mv	1346
gpio.c	1347
gpio_bit_constants_t	1348
internal_validate_port_pin	1348
gpio_set_output	1349
gpio_set_input	1350
gpio_write_high	1350
gpio_write_low	1351
gpio_toggle	1352
gpio_read	1353
riic.c	1354
riic_constants_t	1354
riic_channel_num_t	1355
riic_module_stop_bits_t	1355
riic_frequency_t	1356
riic_bit_rate_t	1356
riic_icmr1_values_t	1356
riic_icmr2_values_t	1357
riic_icmr3_bits_t	1357
riic_address_bits_t	1357
s_tag	1358
s_riic_channel_initialized	1358
internal_get_riic_base	1358
internal_calculate_bit_rate	1359
internal_wait_bus_ready	1360
internal_send_start	1361
internal_send_stop	1362

internal_write_byte	1363
internal_read_byte	1364
internal_riic_write_phase	1365
internal_riic_read_phase	1366
riic_init	1368
riic_write	1370
riic_read	1372
riic_write_read	1374
rsPIC	1376
rsPIC_constants_t	1376
rsPIC_cs_defaults_t	1376
rsPIC_module_stop_bits_t	1377
rsPIC_controller_constants_t	1377
rsPIC_mode_limits_t	1377
rsPIC_spcmd_bits_t	1377
rsPIC_spdcr_local_t	1378
rsPIC_register_defaults_t	1378
rsPIC_bit_constants_t	1378
rsPIC_transfer_constants_t	1379
s_tag	1379
s_rsPIC_channel_initialized	1379
s_rsPIC_controller_initialized	1380
s_rsPIC_cs_config	1380
internal_get_rsPIC_base	1380
internal_set_mstpcrb_for_channel	1381
rsPIC_init_peripheral	1383
internal_wait_tx_ready	1385
internal_wait_rx_ready	1386
rsPIC_peripheral_transfer	1387
rsPIC_peripheral_read_available	1390
rsPIC_peripheral_write_ready	1391
rsPIC_deinit	1392
internal_timing_delay	1395
internal_configure_spcmd	1396
internal_calculate_spbr	1397
internal_configure_cs_gpio	1398
internal_validate_controller_args	1400
internal_prepare_controller	1402
internal_configure_registers	1404
rsPIC_init_controller	1407
rsPIC_controller_set_cs	1408
internal_controller_assert_cs_with_setup	1410
internal_controller_deassert_cs_with_hold	1412
internal_controller_do_16bit_transfer	1413
rsPIC_controller_transfer_16bit	1414
rsPIC_controller_deinit	1416
rx_cmt.c	1419
cmt_constants_t	1419
cmt_divider_values_t	1419

cmt_module_stop_bits_t	1419
cmt_cmcr_bits_t	1420
cmt_period_constants_t	1420
cmt_cmstr_bits_t	1420
cmt_bit_constants_t	1420
s_tag	1420
s_cmt_initialized	1420
s_cmt_callback	1421
s_cmt_user_data	1421
cmt1_isr	1421
cmt2_isr	1422
cmt3_isr	1423
internal_get_cmt_base	1424
internal_calculate_cmt_params	1426
internal_enable_cmt_module_clock	1430
internal_configure_cmt_timer_registers	1431
internal_configure_cmt_interrupt	1433
internal_validate_cmt_init_params	1435
internal_save_cmt_callback	1437
rx_cmt_init	1439
rx_cmt_start	1443
rx_cmt_stop	1446
rx_cmt_get_count	1449
rx_cmt_deinit	1451
rx_gptw.c	1454
gptw_constants_t	1454
gptw_period_constants_t	1454
gptw_duty_constants_t	1454
mpc_pwpr_constants_t	1455
mpc_pfs_offset_constants_t	1455
gptw_bit_constants_t	1455
gptw_channel_mask_t	1455
gptw_direction_constants_t	1455
porte_gptw_pin_t	1455
gptw_phase_divisor_t	1456
s_tag	1456
s_gptw_period_max	1456
s_gptw_ns_per_second	1457
s_gptw_initialized	1457
s_gptw_period	1457
internal_get_gptw_base	1457
internal_is_triangle_mode	1458
internal_get_gtcr_mode	1458
internal_calculate_period	1459
internal_configure_mpc	1460
internal_configure_port_pins	1463
internal_enable_gptw_module_clock	1464
internal_configure_gptw_hardware	1465
internal_prepare_gptw_pwm_init	1467

rx_gptw_init_pwm	1468
rx_gptw_set_duty	1469
rx_gptw_set_duty_raw	1471
rx_gptw_get_duty	1473
rx_gptw_get_period	1476
rx_gptw_enable_output	1477
rx_gptw_start	1478
rx_gptw_stop	1479
internal_calculate_phase_offset	1480
internal_configure_channel_staggered	1481
rx_gptw_init_all_staggered	1482
rx_gptw_deinit	1485
rx_host_irq.c	1486
host_irq_gpio_constants_t	1487
s_tag	1487
s_initialized	1487
rx_host_irq_init	1487
rx_host_irq_deinit	1490
rx_host_irq_assert	1492
rx_host_irq_deassert	1493
rx_host_irq_is_asserted	1494
rx_irq_filter.c	1496
irq_filter_constants_t	1496
rx_irq_filter_enable	1496
rx_irq_filter_disable	1497
rx_irq_filter_is_enabled	1498
rx_irq_filter_get_clock	1499
rx_mpc.c	1501
mpc_pin_constants_t	1501
mpc_pfs_offset_t	1501
mpc_port_number_t	1501
mpc_port_offset_t	1502
mpc_psel_constants_t	1503
mpc_pfs_gpio_t	1503
mpc_pwpr_t	1504
s_tag	1504
internal_get_pfs_register	1504
internal_mpc_unlock	1507
internal_mpc_lock	1508
internal_write_pfs	1509
rx_mpc_set_gpio	1511
rx_mpc_set_peripheral	1512
rx_mpc_set_mtu_pwm	1514
rx_mpc_set_mtu_encoder	1515
rx_mpc_set_tpu_encoder	1517
rx_mpc_set_adc	1518
rx_mpc_set_sci	1519
rx_mpc_set_riic	1520
rx_mpc_set_rspi	1522

rx_mpc_set_gptw	1523
rx_mpc_set_usb_vbus	1524
rx_mpc_set_irq	1525
rx_mtu.c	1527
mtu_constants_t	1528
mtu_module_stop_bits_t	1528
mtu_bit_constants_t	1528
mtu_period_constants_t	1528
mtu_tior_shift_t	1528
mtu_tior_mask_t	1528
mtu_tior_disabled_t	1529
mtu_duty_constants_t	1529
s_tag	1529
s_mtu_initialized	1529
s_mtu_period	1529
internal_get_mtu_base	1530
internal_is_valid_channel	1532
internal_clear_tstr_bit	1532
internal_calculate_period	1534
internal_get_tgr_register	1535
internal_set_duty_raw_mtu	1536
rx_mtu_init_pwm	1537
rx_mtu_set_duty	1540
rx_mtu_set_duty_raw	1543
rx_mtu_get_duty	1544
rx_mtu_get_period	1547
rx_mtu_enable_output	1549
rx_mtu_start	1551
rx_mtu_stop	1553
rx_mtu_deinit	1555
rx_poeg.c	1558
poeg_icu_constants_t	1558
poeg_init_config_t	1558
poeg_gtintad_values_t	1559
s_tag	1559
s_poeg_groups	1559
s_gptw_channels	1559
s_gtintad_values	1559
s_poeg_vectors	1560
internal_enable_poeg_irq	1560
internal_configure_gtintad	1561
internal_poeg_isr_handler	1561
poeg_group_a_isr	1563
poeg_group_b_isr	1563
poeg_group_c_isr	1564
poeg_group_d_isr	1564
rx_poeg_init	1565
rx_poeg_get_fault_status	1568
rx_poeg_clear_fault	1569

rx_poeg_software_stop	1572
rx_poeg_clear_software_stop	1573
rx_tpu.c	1574
tpu_phase_channel_count_t	1574
tpu_channel_index_t	1575
tpu_tmdr_reset_t	1575
tpu_tcr_phase_count_t	1575
tpu_tier_disable_t	1575
tpu_tcmt_zero_t	1576
s_tag	1576
s_tpu_initialized	1576
internal_channel_to_index	1576
internal_get_regs	1578
internal_get_cst_bit	1580
internal_get_tmdr_mode	1582
internal_enable_tpu_module_clock	1583
rx_tpu_init_phase_count	1584
rx_tpu_start	1586
rx_tpu_stop	1587
rx_tpu_read_count	1589
rx_tpu_read_direction	1591
rx_tpu_reset_count	1594
rx_tpu_deinit	1595
timer.c	1598
cmt0_config_t	1598
cmt0_cmcr_values_t	1598
cmt0_cmstr_bits_t	1599
cmt0_ier_constants_t	1599
cmt0_counter_values_t	1599
icu_ir_values_t	1599
cmt0_isr	1600
_tx_timer_interrupt	1602
timer_init	1603
timer_stop	1607
timer_get_count	1609
uart.c	1613
uart_config_constants_t	1613
sci_register_values_t	1613
uart_int_constants_t	1613
uart_hex_constants_t	1614
brr_constants_t	1614
uart_mstpcrb_constants_t	1614
uart_internal_constants_t	1614
uart_validation_limits_t	1615
uart_timeout_t	1615
uart_flag_constants_t	1615
uart_buffer_constants_t	1616
sci_mstpb_bits_t	1616
uart_debug_pins_t	1616

uart_gpio_constants_t	1616
s_channel_initialized	1617
internal_calculate_brr	1617
internal_clear_errors	1619
internal_get_mstpb_bit	1621
internal_enable_sci_clock	1622
internal_configure_uart_pins	1624
uart_init_channel	1627
uart_deinit_channel	1631
uart_putc_channel	1632
uart_puts_channel	1636
uart_write_channel	1638
uart_getc_channel	1640
uart_read_channel	1644
uart_rx_available	1647
uart_debug_init	1650
uart_debug_putc	1651
uart_debug_puts	1652
uart_debug_putint	1654
uart_debug_puthex	1655
rx_harq.h	1657
rx_harq_params_t	1657
rx_harq_state_t	1658
rx_chase_combiner_init	1658
rx_chase_combiner_deinit	1659
rx_chase_combiner_add	1660
rx_chase_combiner_combined	1661
rx_chase_combiner_reset	1662
rx_chase_combiner_can_add	1663
rx_chase_combiner_count	1664
rx_harq_init	1664
rx_harq_deinit	1667
rx_harq_get_state	1668
rx_harq_reset	1669
rx_harq_encode	1670
rx_harq_decode	1671
rx_harq_get_retry_count	1674
rx_harq_can_retry	1674
rx_harq.c	1675
harq_fec_constants_t	1675
harq_cfg_sentinel_t	1676
harq_bool_t	1676
bit_constants_t	1676
rx_chase_combiner_init	1677
rx_chase_combiner_deinit	1678
rx_chase_combiner_add	1678
rx_chase_combiner_combined	1679
rx_chase_combiner_reset	1680
rx_chase_combiner_can_add	1681

rx_chase_combiner_count	1682
rx_harq_init	1682
rx_harq_deinit	1684
rx_harq_get_state	1685
rx_harq_reset	1686
rx_harq_encode	1687
internal_soft_to_hard	1688
internal_handle_fec_result	1689
rx_harq_decode	1690
rx_harq_get_retry_count	1693
rx_harq_can_retry	1693
rx_hcsr04.h	1694
rx_hcsr04_timing_t	1694
rx_hcsr04_range_t	1695
rx_hcsr04_echo_mode_t	1696
rx_hcsr04_irq_t	1697
rx_hcsr04_irq_priority_t	1697
rx_hcsr04_sensor_index_t	1698
rx_hcsr04_callback_t	1699
rx_hcsr04_init	1699
rx_hcsr04_deinit	1703
rx_hcsr04_worker_init	1705
rx_hcsr04_worker_deinit	1707
rx_hcsr04_measure_blocking	1709
rx_hcsr04_measure	1710
rx_hcsr04_measure_async	1713
rx_hcsr04_is_busy	1715
rx_hcsr04_cancel	1715
rx_hcsr04_set_temperature	1716
rx_hcsr04_disable_temp_compensation	1717
rx_hcsr04_is_temp_compensation_enabled	1718
rx_hcsr04_get_temperature	1719
rx_hcsr04_cm_to_inches	1720
rx_hcsr04_echo_to_cm	1720
rx_hcsr04_echo_to_cm_with_temp	1721
rx_hcsr04_get_speed_of_sound	1722
rx_hcsr04_get_stats	1723
rx_hcsr04_reset_stats	1724
rx_hcsr04.c	1724
rx_hcsr04_worker_priority_t	1725
rx_hcsr04_worker_stack_t	1725
rx_hcsr04_event_flags_t	1725
rx_hcsr04_conversion_t	1726
rx_hcsr04_scale_t	1726
rx_hcsr04_shutdown_t	1726
rx_hcsr04_poll_limits_t	1726
rx_hcsr04_irq_range_t	1726
rx_hcsr04_irq_port_t	1727
rx_hcsr04_irq_yield_t	1727

rx_hcsr04_irq_poll_limits_t	1728
s_speed_of_sound_base_mps	1728
s_speed_of_sound_coeff	1728
s_default_temperature_celsius	1728
s_min_temp_celsius	1729
s_max_temp_celsius	1729
s_mps_to_cm_per_us	1729
s_roundtrip_divisor	1729
s_hcsr04_worker_thread	1729
s_worker_stack	1729
s_measurement_request	1729
s_worker_shutdown_sem	1730
s_pending_mutex	1730
s_pending	1730
s_worker_initialized	1731
internal_send_trigger_pulse	1731
internal_wait_for_echo	1733
internal_measure_echo_pulse	1735
internal_measure_echo_pulse_irq	1737
hcsr04_worker_entry	1739
rx_hcsr04_worker_init	1741
rx_hcsr04_worker_deinit	1743
internal_init_irq_mode	1745
rx_hcsr04_init	1748
rx_hcsr04_deinit	1750
internal_trigger_and_measure	1752
rx_hcsr04_measure_blocking	1753
rx_hcsr04_measure	1754
rx_hcsr04_measure_async	1757
rx_hcsr04_is_busy	1759
rx_hcsr04_cancel	1759
rx_hcsr04_set_temperature	1760
rx_hcsr04_disable_temp_compensation	1761
rx_hcsr04_is_temp_compensation_enabled	1762
rx_hcsr04_get_temperature	1762
rx_hcsr04_cm_to_inches	1763
rx_hcsr04_echo_to_cm	1764
rx_hcsr04_get_speed_of_sound	1764
rx_hcsr04_echo_to_cm_with_temp	1765
rx_hcsr04_get_stats	1766
rx_hcsr04_reset_stats	1767
rx_hcsr04_hal.h	1767
hcsr04_hal_gpio_set_output	1768
hcsr04_hal_gpio_set_input	1769
hcsr04_hal_gpio_write_high	1771
hcsr04_hal_gpio_write_low	1773
hcsr04_hal_gpio_read	1775
hcsr04_hal_gpio_deinit	1777
hcsr04_hal_delay_us	1778

hcsr04_hal_get_time_us	1781
hcsr04_hal_get_time_us_isr	1785
rx_hcsr04_hal_hw.c	1787
cmt2_timing_constants_t	1787
s_cmt2_initialized	1789
s_time_mutex	1789
s_time_mutex_initialized	1790
internal_validate_port_pin	1790
hcsr04_hal_gpio_set_output	1792
hcsr04_hal_gpio_set_input	1793
hcsr04_hal_gpio_write_high	1794
hcsr04_hal_gpio_write_low	1795
hcsr04_hal_gpio_read	1797
hcsr04_hal_gpio_deinit	1797
internal_time_mutex_init	1798
internal_cmt2_init	1799
hcsr04_hal_delay_us	1802
hcsr04_hal_get_time_us	1805
hcsr04_hal_get_time_us_isr	1807
rx_hcsr04_icu.c	1808
icu_constants_t	1808
s_tag	1809
rx_hcsr04_icu_configure	1809
rx_hcsr04_icu_disable	1813
rx_hcsr04_icu.h	1816
rx_hcsr04_icu_configure	1816
rx_hcsr04_icu_disable	1821
rx_hcsr04_isr.c	1825
isr_constants_t	1825
isr_timer_constants_t	1826
isr_irq_numbers_t	1827
s_irq_state	1827
s_sensor_map	1827
internal_irq_handler	1828
rx_hcsr04_isr_register	1830
rx_hcsr04_isr_unregister	1831
rx_hcsr04_isr_start	1833
rx_hcsr04_isr_disarm	1834
rx_hcsr04_isr_get_duration	1836
INT_IRQ8	1838
INT_IRQ9	1839
INT_IRQ10	1840
INT_IRQ11	1841
rx_hcsr04_isr.h	1842
s_hcsr04_us_per_cm	1842
rx_hcsr04_isr_register	1843
rx_hcsr04_isr_unregister	1845
rx_hcsr04_isr_get_duration	1847
rx_hcsr04_isr_start	1850

rx_hcsr04_isr_disarm	1852
INT_IRQ8	1854
INT_IRQ9	1856
INT_IRQ10	1857
INT_IRQ11	1858
rx_motor.h	1860
rx_motor_init	1860
rx_motor_deinit	1865
rx_motor_set_duty	1868
rx_motor_stop	1873
rx_motor_get_duty	1876
rx_motor_emergency_stop	1878
rx_motor.c	1883
motor_duty_limits_t	1883
motor_in2_signal_t	1883
motor_validation_limits_t	1884
s_tag	1885
internal_clamp_duty	1885
internal_init_gptw_outputs	1886
rx_motor_init	1890
rx_motor_deinit	1895
rx_motor_set_duty	1898
rx_motor_stop	1903
rx_motor_get_duty	1906
rx_motor_emergency_stop	1908
rx_obstacle_detect.h	1912
rx_obstacle_detect_constants_t	1913
rx_obstacle_detect_state_t	1913
rx_obstacle_detect_callback_t	1914
rx_obstacle_detect_init	1914
rx_obstacle_detect_deinit	1919
rx_obstacle_detect_start	1921
rx_obstacle_detect_stop	1923
rx_obstacle_detect_clear_obstacle	1925
rx_obstacle_detect_get_state	1927
rx_obstacle_detect_is_obstacle_detected	1929
rx_obstacle_detect_get_stats	1930
rx_obstacle_detect_reset_stats	1932
rx_obstacle_detect.c	1934
event_flags_t	1935
validation_constants_t	1935
task_constants_t	1936
s_zero_handle	1936
internal_detection_task_entry	1936
internal_validate_config	1937
internal_stop_all_motors	1938
internal_poll_sensors	1939
internal_invoke_callback	1941
rx_obstacle_detect_init	1941

rx_obstacle_detect_deinit	1945
rx_obstacle_detect_start	1947
rx_obstacle_detect_stop	1948
rx_obstacle_detect_clear_obstacle	1950
rx_obstacle_detect_get_state	1952
rx_obstacle_detect_is_obstacle_detected	1954
rx_obstacle_detect_get_stats	1955
rx_obstacle_detect_reset_stats	1957
rx_pid.h	1959
rx_pid_init	1959
false	1961
rx_pid_deinit	1964
rx_pid_compute	1966
rx_pid_reset	1968
rx_pid_set_gains	1972
rx_pid_set_output_limits	1976
rx_pid_set_integral_limits	1979
rx_pid.c	1983
s_tag	1983
internal_clamp	1984
rx_pid_init	1985
false	1988
rx_pid_deinit	1990
rx_pid_compute	1992
rx_pid_reset	1994
rx_pid_set_gains	1998
rx_pid_set_output_limits	2002
rx_pid_set_integral_limits	2005
rx_session.h	2009
rx_session_constants_t	2010
rx_session_validate_result_t	2010
rx_session_init	2010
rx_session_deinit	2014
rx_session_next_tx	2016
rx_session_validate_rx	2018
rx_session_reset	2020
rx_session_get_tx	2022
rx_session_get_rx	2023
rx_session.c	2027
s_tag	2027
s_session_mutex	2027
internal_lock	2027
internal_unlock	2028
rx_session_init	2029
rx_session_deinit	2032
rx_session_next_tx	2033
rx_session_validate_rx	2035
rx_session_reset	2037
rx_session_get_tx	2038

rx_session_get_rx	2040
rx_spi_comm.h	2043
rx_spi_comm_constants_t	2043
rx_retransmit_defaults_retries_t	2043
rx_retransmit_defaults_timing_t	2044
rx_spi_comm_init	2044
rx_spi_comm_deinit	2049
rx_spi_comm_send	2051
rx_spi_comm_send_ack	2058
rx_spi_comm_send_nack	2060
rx_spi_comm_receive	2062
rx_spi_comm_data_available	2069
rx_spi_comm_set_control_callbacks	2070
rx_spi_comm_send_pong	2072
rx_spi_comm_send_reset_ack	2074
rx_spi_comm_process_retransmits	2076
rx_spi_comm_set_auto_retransmit	2078
rx_spi_comm_set_retransmit_callbacks	2080
rx_spi_comm.c	2082
backoff_limit_t	2082
frame_header_offset_t	2082
poll_timing_t	2082
ack_wait_t	2083
receive_bounds_t	2083
polling_limits_t	2083
ctrl_dispatch_result_t	2083
s_tag	2083
s_zero_handle	2083
internal_retransmit_frame	2084
internal_decode_header	2085
internal_verify_crc	2089
internal_wait_for_ack	2091
internal_spi_transfer	2093
rx_spi_comm_init	2096
rx_spi_comm_deinit	2100
internal_build_frame	2101
rx_spi_comm_send	2105
rx_spi_comm_send_ack	2109
rx_spi_comm_send_nack	2110
internal_wait_for_data	2111
internal_read_frame_header	2112
internal_decode_frame	2113
internal_dispatch_control_frame	2115
rx_spi_comm_receive	2117
rx_spi_comm_data_available	2121
rx_spi_comm_set_control_callbacks	2121
rx_spi_comm_send_pong	2123
rx_spi_comm_send_reset_ack	2125
rx_spi_comm_process_retransmits	2126

rx_spi_comm_set_auto_retransmit	2128
rx_spi_comm_set_retransmit_callbacks	2129
rx_spi_link.h	2130
rx_spi_link_constants_t	2131
rx_spi_link_timing_t	2131
rx_spi_link_buffer_t	2132
rx_spi_link_state_t	2132
rx_spi_link_init	2133
rx_spi_link_deinit	2135
rx_spi_link_send	2137
rx_spi_link_receive	2139
rx_spi_link_reset	2141
rx_spi_link_get_state	2143
rx_spi_link_fec_enabled	2144
rx_spi_link.c	2145
internal_soft_bit_values_t	2145
internal_bit_constants_t	2146
s_tag	2146
s_zero_link	2146
internal_bytes_to_soft_bits	2146
internal_wait_for_ack	2147
rx_spi_link_init	2150
rx_spi_link_deinit	2152
internal_send_attempt	2154
internal_prepare_tx_payload	2155
rx_spi_link_send	2157
internal_receive_fec_decode	2159
internal_receive_passthrough	2162
rx_spi_link_receive	2163
rx_spi_link_reset	2165
rx_spi_link_get_state	2167
rx_spi_link_fec_enabled	2167
rx_usb.h	2167
rx_usb_port_id_t	2168
rx_usb_event_t	2169
rx_usb_endpoints.h	2169
usb_endpoint_addresses_t	2170
rx_usb_private.h	2170
rx_usb.c	2170
flush_loop_bounds_t	2170
ring_buffer_constants_t	2171
port_config_constants_t	2171
port_pipe_assignments_t	2171
int_conversion_constants_t	2172
s_tag	2172
s_flush_poll_interval_ms	2172
s_port_configs	2172
s_port0_rx_buf	2172
s_port0_tx_buf	2172

s_port1_rx_buf	2172
s_port1_tx_buf	2172
s_port2_rx_buf	2173
s_port2_tx_buf	2173
s_usb	2173
internal_port_is_valid	2173
internal_state_is_valid	2175
priv_ring_buffer_init	2175
priv_ring_buffer_available	2176
priv_ring_buffer_free	2177
priv_ring_buffer_write	2177
priv_ring_buffer_read	2178
internal_trigger_tx_if_idle	2179
internal_init_port_buffers	2181
internal_init_line_coding	2182
rx_usb_init	2182
rx_usb_deinit	2186
rx_usb_is_configured	2188
rx_usb_get_state	2188
rx_usb_write	2189
rx_usb_read	2192
rx_usb_rx_available	2196
rx_usb_tx_available	2197
rx_usb_flush	2198
rx_usb_set_callback	2200
rx_usb_get_line_coding	2200
rx_usb_get_stats	2201
rx_usb_reset_stats	2202
rx_usb_putc	2203
rx_usb_puts	2204
rx_usb_putint	2205
rx_usb_puthex	2206
rx_usb_set_state	2209
rx_usb_set_line_coding	2209
rx_usb_rx_push	2210
rx_usb_tx_pop	2210
rx_usb_count_bus_reset	2211
rx_usb_count_suspend	2211
rx_usb_get_port_config	2212
rx_usb_find_port_by_pipe	2212
rx_usb_find_port_by_interface	2213
rx_usb_invoke_callback	2213
rx_usb_cdc.c	2214
usb_descriptor_defaults_t	2214
usb_descriptor_type_t	2215
usb_class_code_t	2215
cdc_request_code_t	2215
usb_request_code_t	2215
usb_device_id_t	2216

usb_version_t	2216
cdc_descriptor_subtype_t	2216
usb_control_endpoint_t	2217
usb_endpoint_type_t	2217
usb_config_attributes_t	2217
usb_max_power_t	2217
cdc_acm_capabilities_t	2217
usb_string_index_t	2217
usb_packet_size_t	2218
usb_polling_interval_t	2218
usb_string_size_t	2218
bit_shift_t	2218
byte_mask_t	2219
usb_bit_constants_t	2219
usb_control_line_constants_t	2219
usb_request_type_mask_t	2219
cdc_line_coding_index_t	2219
usb_pipe_number_t	2220
usb_config_value_t	2220
iad_constants_t	2220
composite_config_size_t	2221
s_tag	2221
s_device_desc	2221
s_config_desc	2221
length	2221
descriptor_type	2221
langid	2221
s_string_langid	2221
string	2221
s_string_manufacturer	2222
s_string_product	2222
s_string_serial	2222
s_cdc_initialized	2222
s_line_coding	2222
s_control_line_state	2222
internal_send_descriptor	2222
internal_handle_get_descriptor	2223
internal_handle_set_address	2225
internal_disable_pipe	2225
internal_rollback_pipes	2226
internal_handle_set_configuration	2227
internal_handle_set_line_coding	2229
internal_handle_get_line_coding	2231
internal_handle_set_control_line_state	2233
rx_usb_cdc_init	2233
internal_handle_standard_request	2236
internal_handle_class_request	2237
rx_usb_cdc_handle_setup	2238
rx_usb_cdc_handle_bulk_out	2242

rx_usb_cdc_handle_bulk_in	2246
rx_usb_hw.c	2249
usb_hw_timing_t	2250
usb_syscfg_value_t	2250
usb_fifo_constants_t	2250
usb_address_mask_t	2250
usb_icu_config_t	2250
usb_validation_limits_t	2251
s_tag	2251
s_hw_initialized	2251
rx_usb_hw_init	2251
rx_usb_hw_deinit	2253
rx_usb_hw_attach	2255
rx_usb_hw_detach	2255
rx_usb_hw_fifo_read	2256
rx_usb_hw_fifo_write	2257
rx_usb_hw_get_bus_state	2259
rx_usb_hw_set_address	2259
rx_usb_hw_configure_pipe	2260
rx_usb_internal.h	2262
usb_interface_number_t	2262
rx_usb_get_port_config	2262
rx_usb_hw_fifo_read	2263
rx_usb_hw_fifo_write	2264
rx_usb_hw_set_address	2266
rx_usb_hw_configure_pipe	2267
rx_usb_hw_init	2269
rx_usb_hw_deinit	2271
rx_usb_hw_attach	2273
rx_usb_hw_detach	2273
rx_usb_hw_get_bus_state	2274
rx_usb_set_state	2275
rx_usb_set_line_coding	2276
rx_usb_rx_push	2277
rx_usb_tx_pop	2277
rx_usb_count_bus_reset	2278
rx_usb_count_suspend	2278
rx_usb_find_port_by_pipe	2279
rx_usb_find_port_by_interface	2279
rx_usb_invoke_callback	2280
rx_usb_cdc_init	2280
rx_usb_cdc_handle_setup	2283
rx_usb_cdc_handle_bulk_in	2287
rx_usb_cdc_handle_bulk_out	2291
rx_usb_isr.c	2294
usb_pipe_numbers_t	2294
usb_pipe_assignment_t	2295
s_tag	2295
usb0_usbi_isr	2295

usb0_d0fifo_isr	2296
usb0_d1fifo_isr	2297
usb0_usbr_isr	2298
internal_handle_vbus_interrupt	2298
internal_handle_dvst_interrupt	2299
internal_handle_ctr_t_interrupt	2301
internal_handle_brdy_interrupt	2303
internal_handle_bemp_interrupt	2303
internal_handle_resume_interrupt	2304
rx_usb_isr_handler	2305
rx_usb_comm.h	2307
rx_usb_comm_constants_t	2307
rx_usb_comm_mode_t	2307
rx_usb_comm_init	2308
rx_usb_comm_deinit	2312
rx_usb_comm_send	2314
rx_usb_comm_receive	2317
rx_usb_comm_data_available	2320
rx_usb_comm_is_ready	2321
rx_usb_comm_flush_rx	2322
rx_usb_comm_set_mode	2323
rx_usb_comm_get_mode	2325
rx_usb_comm_set_control_callbacks	2326
rx_usb_comm_send_pong	2328
rx_usb_comm_send_reset_ack	2330
rx_usb_comm.c	2332
rx_usb_comm_header_size_t	2333
rx_usb_comm_sleep_t	2333
rx_usb_comm_sequence_constants_t	2333
rx_usb_comm_sync_constants_t	2333
rx_usb_comm_loop_limits_t	2333
frame_header_offset_t	2334
rx_usb_comm_sync_result_t	2334
rx_receive_result_t	2334
s_tag	2334
s_zero_handle	2334
internal_decode_header	2334
internal_verify_crc	2337
internal_read_usb_data	2338
internal_compact_rx_buffer	2340
internal_find_sync	2341
internal_sleep_and_advance	2343
internal_is_timed_out	2344
internal_handle_no_sync	2344
internal_wait_for_data	2345
internal_decode_frame	2346
internal_align_to_sync	2347
internal_parse_header	2348
internal_receive_iteration	2349

rx_usb_comm_init	2352
rx_usb_comm_deinit	2355
internal_build_frame	2357
rx_usb_comm_send	2358
rx_usb_comm_receive	2362
rx_usb_comm_data_available	2365
rx_usb_comm_is_ready	2366
rx_usb_comm_flush_rx	2366
rx_usb_comm_set_control_callbacks	2367
rx_usb_comm_send_pong	2369
rx_usb_comm_send_reset_ack	2371
rx_usb_comm_set_mode	2373
rx_usb_comm_get_mode	2375
default_interrupt_handlers.c	2375
boot_common.h	2375
INTERNAL_NOT_USED	2375
platform.h	2376
r_bsp.h	2376
r_rx_intrinsic_functions.c	2376
bsp_get_bpsw	2377
bsp_get_bpc	2378
bsp_move_from_acc_hi_long	2378
bsp_move_from_acc_mi_long	2378
R_BSP_ChangeToUserMode	2379
R_BSP_SetBPSW	2382
R_BSP_PRAGMA_STATIC_INLINE_ASM	2385
R_BSP_GetBPSW	2386
R_BSP_SetBPC	2386
R_BSP_PRAGMA_STATIC_INLINE_ASM	2387
R_BSP_GetBPC	2387
R_BSP_MoveToAccHiLong	2387
R_BSP_PRAGMA_INLINE_ASM	2388
R_BSP_PRAGMA_STATIC_INLINE_ASM	2388
R_BSP_MoveFromAccHiLong	2388
R_BSP_MoveFromAccMiLong	2388
R_BSP_BitSet	2388
R_BSP_PRAGMA_INLINE_ASM	2389
R_BSP_PRAGMA_INLINE_ASM	2389
hardware_init.c	2389
rx_mpc_pin_t	2390
gpio_pin_counts_t	2391
gptw_freq_t	2392
gptw_deadtime_t	2392
i2c_channel_t	2392
i2c_freq_t	2392
adc_count_t	2392
gpio_reg_constants_t	2393
s_sckcr3_reset_state	2393
internal_gpio_init_i2c	2393

internal_gpio_init_host_spi	2394
internal_gpio_init_mtu_encoders	2396
internal_gpio_init_tpu_encoders	2397
internal_gpio_init_gptw_pwm	2399
internal_gpio_set_output	2402
internal_gpio_set_input	2403
internal_gpio_init_imu	2405
internal_gpio_init_motor_driver_ctrl	2407
internal_gpio_init_adc	2409
internal_gpio_init_usb	2410
internal_gpio_init_sonar_triggers	2411
internal_gpio_init_sonar_echoes	2413
gpio_init	2415
gptw_pwm_init	2417
spi_init	2418
i2c_init	2420
adc_init_channels	2421
validate_peripherals	2424
hardware_init	2425
hardware_config.h	2427
hardware_init.h	2427
hardware_init	2427
rx_clock_power_init.h	2430
rx_clock_power_init	2430
rx_stack_monitor.h	2431
rx_stack_monitor_init	2431
rx_stack_monitor_get_free_bytes	2432
main.c	2433
main_ret_t	2434
riic_channel_num_t	2434
i2c_frequency_t	2434
iwdt_hardware_timeout_ms_t	2435
iwdt_state_timeout_ms_t	2435
iwdt_task_timeout_ms_t	2437
s_onewire0_config	2439
s_gpio_config	2439
s_adc0_config	2439
s_iwdt_config	2440
internal_check_porf	2440
internal_check_rstsr2_flag_clear	2441
internal_check_iwdtrf	2441
internal_check_wdtrf	2442
internal_check_swrf	2442
internal_check_lvd0rf	2443
internal_check_cwsf	2443
internal_report_startup_flags	2444
internal_check_startup_flags	2445
internal_init_bus_manager	2445
internal_register_system_buses	2446

internal_init_shared_data_and_watchdog	2448
internal_register_iwdt_tasks	2449
internal_create_system_tasks	2451
internal_init_stack_monitor	2453
tx_application_define	2454
main	2455
rx_clock_power_init.c	2457
oscillator_timing_t	2457
oscillator_control_t	2457
pll_config_t	2457
system_clock_config_t	2458
mstpcra_bits_t	2458
mstpcrb_bits_t	2458
mstpcrc_bits_t	2458
module_init_config_t	2458
s_tag	2458
internal_clock_init	2459
internal_module_stop_init	2461
internal_verify_system_state	2463
rx_clock_power_init	2465
rx_stack_monitor.c	2465
stack_fill_constants_t	2465
s_tag	2466
internal_stack_overflow_handler	2466
rx_stack_monitor_init	2467
rx_stack_monitor_get_free_bytes	2468
shared_data.c	2470
shared_data_internal_constants_t	2470
s_default_pid_kp	2471
s_default_pid_ki	2471
s_default_pid_kd	2471
s_default_pid_output_min	2471
s_default_pid_output_max	2472
s_default_pid_integral_min	2472
s_default_pid_integral_max	2472
s_tag	2473
s_estop_pending_from_isr	2473
s_pending_estop_reason	2473
g_bus_manager	2473
g_shared_data	2474
shared_data_init	2474
shared_data_set_motor_command	2475
shared_data_get_motor_command	2476
shared_data_update_motor_state	2477
shared_data_get_motor_state	2477
shared_data_set_pid_gains	2478
shared_data_get_pid_gains	2479
shared_data_pid_update_pending	2479
shared_data_clear_pid_update_flag	2480

shared_data_trigger_estop	2480
shared_data_trigger_estop_isr_safe	2480
shared_data_commit_isr_estop	2481
shared_data_clear_estop	2482
shared_data_is_estop_active	2483
shared_data_get_estop_reason	2483
shared_data_update_temp	2484
shared_data_get_temp	2484
shared_data_update_obstacle	2485
shared_data_get_obstacle	2486
shared_data_is_comm_timeout	2487
shared_data_update_last_comm_tick	2488
shared_data_set_event	2488
shared_data_wait_event	2488
shared_data.h	2489
estop_reason_t	2489
motor_mode_t	2490
shared_event_flags_t	2490
shared_data_constants_t	2491
g_shared_data	2491
shared_data_init	2491
shared_data_set_motor_command	2492
shared_data_get_motor_command	2493
shared_data_update_motor_state	2494
shared_data_get_motor_state	2495
shared_data_set_pid_gains	2496
shared_data_get_pid_gains	2497
shared_data_pid_update_pending	2498
shared_data_clear_pid_update_flag	2498
shared_data_trigger_estop	2499
shared_data_trigger_estop_isr_safe	2499
shared_data_commit_isr_estop	2501
shared_data_clear_estop	2503
shared_data_is_estop_active	2504
shared_data_get_estop_reason	2504
shared_data_update_temp	2505
shared_data_get_temp	2506
shared_data_update_obstacle	2506
shared_data_get_obstacle	2507
shared_data_is_comm_timeout	2508
shared_data_update_last_comm_tick	2509
shared_data_set_event	2509
shared_data_wait_event	2510
comm_task.c	2511
comm_task_constants_t	2512
comm_motor_constants_t	2512
retransmit_retries_limits_t	2512
retransmit_timing_limits_t	2512
s_comm_thread	2513

s_comm_stack	2513
s_comm_created	2513
g_comm_manager	2513
s_tag	2513
s_session_state	2514
s_usb_comm_handle	2514
s_spi_comm_handle	2514
internal_comm_task_entry	2514
internal_init_transports	2516
internal_frame_callback	2518
internal_handle_command_frame	2519
internal_handle_velocity_command	2521
internal_handle_estop_command	2523
internal_handle_pid_gains_command	2525
internal_handle_retransmit_config_command	2527
comm_task_create	2529
comm_task.h	2531
comm_task_create	2532
led_status_task.h	2534
led_status_task_create	2535
motor_control_task.h	2537
motor_count_t	2538
motor_control_task_create	2538
motor_control_task_get_motors	2540
obstacle_detect_task.h	2543
obstacle_detect_task_create	2544
telemetry_task.h	2546
telemetry_task_create	2546
temp_sensor_task.h	2548
temp_sensor_task_create	2549
watchdog_monitor_task.h	2551
watchdog_monitor_task_create	2552
led_status_task.c	2554
led_task_constants_t	2555
led_timing_constants_t	2555
led_timing_divisor_t	2555
led_index_t	2556
s_led_thread	2556
s_led_stack	2556
s_led_created	2556
s_tag	2556
s_led_ports	2556
s_led_pins	2556
s_heartbeat_counter	2557
s_error_counter	2557
s_comm_pulse_remaining	2557
s_last_comm_sequence	2557
internal_led_task_entry	2557
internal_led_set	2560

internal_led_init_gpio	2561
led_status_task_create	2562
motor_control_task.c	2564
motor_task_constants_t	2565
motor_control_constants_t	2565
motor_index_t	2566
encoder_limits_t	2567
motor_hw_constants_t	2568
g_bus_manager	2568
s_pid_kp_default	2569
s_pid_ki_default	2569
s_pid_kd_default	2569
s_pid_output_min_percent	2569
s_pid_output_max_percent	2569
s_pid_integral_min_percent	2569
s_pid_integral_max_percent	2569
s_velocity_near_stopped_mps	2570
s_motor_thread	2570
s_motor_stack	2570
s_motor_created	2570
s_motor_stack_initialized	2570
s_motors	2570
s_motor_ptrs	2570
s_encoder_state	2571
s_pids	2571
s_dt_sec	2571
s_timeout_estop_triggered	2571
s_active_brake_in_progress	2571
s_tag	2571
s_task_name	2571
internal_motor_task_entry	2572
internal_init_motor_stack	2574
internal_init_pid_controllers	2576
internal_init_encoders	2578
internal_control_loop_iteration	2579
internal_active_brake_sequence	2580
internal_apply_pid_updates	2581
internal_check_comm_timeout	2584
internal_read_encoder_velocity	2584
internal_read_encoder_count	2585
internal_update_motor_state	2586
internal_get_target_velocity	2588
motor_control_task_create	2588
motor_control_task_get_motors	2589
obstacle_detect_task.c	2591
obstacle_task_constants_t	2591
obstacle_sensor_constants_t	2592
obstacle_sensor_idx_t	2592
obstacle_motor_count_t	2593

s_obstacle_thread	2594
s_obstacle_stack	2594
s_obstacle_created	2594
s_obstacle_handle	2594
s_tag	2594
s_sensors	2595
s_sensor_ptrs	2595
internal_obstacle_task_entry	2595
internal_obstacle_callback	2598
obstacle_detect_task_create	2599
telemetry_task.c	2601
telemetry_task_constants_t	2601
telemetry_time_constants_t	2601
telem_motor_idx_t	2602
telem_fault_shift_t	2602
telem_sensor_idx_t	2602
telemetry_transport_t	2602
s_telem_thread	2603
s_telem_stack	2603
s_telem_created	2604
s_telem_buffer	2604
g_comm_manager	2604
s_tag	2605
s_sequence	2605
s_cdegc_per_degree	2605
internal_telem_task_entry	2606
internal_build_and_send_telemetry	2606
internal_select_transport	2608
internal_populate_motor_telemetry	2610
internal_collect_state	2612
internal_encode_telemetry	2614
internal_send_via_channel	2615
telemetry_task_create	2617
temp_sensor_task.c	2618
temp_task_constants_t	2619
temp_sensor_constants_t	2619
s_temp_thread	2619
s_temp_stack	2619
s_temp_created	2619
s_ds18b20	2619
s_tag	2620
g_bus_manager	2620
s_onewire_bus_name	2620
s_cdegc_per_degree	2620
internal_send_iwdt_heartbeat	2620
internal_temp_task_entry	2621
temp_sensor_task_create	2623
watchdog_monitor_task.c	2624
watchdog_task_constants_t	2625

s_watchdog_thread	2625
s_watchdog_stack	2626
s_watchdog_created	2626
s_tag	2626
s_task_name	2626
internal_watchdog_monitor_task_entry	2627
watchdog_monitor_task_create	2628
Index	2630
_	2630
A	2630
B	2631
C	2631
D	2632
E	2633
F	2634
G	2635
H	2636
I	2636
K	2645
L	2645
M	2645
N	2647
O	2647
P	2647
R	2649
S	2671
T	2677
U	2677
V	2680
W	2680

RX72N Firmware API Reference

rx_bus_adc.h

ADC (Analog-to-Digital Converter) Bus Abstraction for RX72N.

Provides bus manager integration for Renesas S12ADFa (Successive Approximation ADC Fast) peripheral operations. Wraps the low-level ADC HAL with the bus abstraction pattern for consistent API across all bus types (I2C, SPI, UART, GPIO, 1-Wire, ADC) with mutex-protected access.

Where:

- V_in = input voltage (0 to VREF)
- V_REF = reference voltage (typically 3.3V or 5V)
- bits = resolution (8, 10, or 12)

Voltage conversion (reverse):

Example for 12-bit @ 3.3V VREF:

Where:

- ADCS = sampling state count (0-255)
- PCLKD = peripheral clock D (60 MHz on RX72N)

Example: ADCS = 74 @ 60 MHz:

Example: V_IPROPI = 1.65V (mid-scale):

1.0.0

2026-01-29

Copyright (c) 2026 STAR Project

Includes:

- <stdint.h>
- "rx_bus_manager.h"
- "rx_err.h"

rx_bus_adc_init

```
rx_err_t rx_bus_adc_init(rx_bus_manager_t *manager, const char *bus_name)
```

Initialize ADC channel through bus manager.

Configures the S12ADFa ADC peripheral for analog voltage measurement. This function must be called before any read operations. The initialization process:

1. Acquires bus manager mutex (thread-safe)
2. Looks up bus configuration by name
3. Validates bus type is ADC
4. Configures MPC for analog input pin function
5. Initializes S12AD unit with resolution and sampling time
6. Enables ADC channel
7. Performs initial conversion (discard for settling)
8. Releases mutex

Public API function to initialize an S12ADFa ADC channel for analog voltage measurement. Delegates to bus manager which calls `internal_adc_init_callback()` with mutex protection and bus lookup.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context for operation result
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context
5. Return result to caller

This function demonstrates the callback pattern for bus operations:

- Prepares context on stack (zero allocation)
- Delegates to bus manager for mutex protection
- Callback executes with mutex held
- Results extracted after mutex released

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via <code>rx_bus_manager_init()</code> Must have ADC bus registered via <code>rx_bus_register()</code>
in	bus_name	ADC bus name Null-terminated string Must match name used in <code>rx_bus_register()</code> Maximum 31 characters
in	manager	Bus manager instance Must be initialized via <code>rx_bus_manager_init()</code> Must have ADC bus registered
in	bus_name	Name of the ADC bus Must match registered bus name Null-terminated string

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, ADC channel initialized and ready
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found in manager
<code>k_rx_err_invalid_arg</code>	Bus is not ADC type
<code>k_rx_err_timeout</code>	Mutex acquisition timeout (default 1000ms)
<code>k_rx_err_hw_init_failed</code>	ADC hardware initialization failed
<code>k_rx_ok</code>	Success, ADC initialized
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_arg</code>	Bus is not ADC type
<code>k_rx_err_timeout</code>	Mutex timeout
<code>k_rx_err_hw_init_failed</code>	ADC hardware initialization failed

Pre: manager must be initialized via `rx_bus_manager_init()`

Pre: Bus must be registered via `rx_bus_register()` with type `k_rx_bus_type_adc`

Pre: Analog input pin must not be driven externally above `VREFH`

Pre: manager must be initialized

Pre: Bus must be registered with type `k_rx_bus_type_adc`

Pre: Analog input voltage must be within 0 to `VREFH`

Post: S12AD unit configured and enabled

Post: Analog pin configured for ADC input

Post: Bus state set to initialized

Post: Initial dummy conversion performed (settling time)

Post: S12AD unit configured and enabled

Post: Analog pin configured for ADC input

Post: Bus marked as initialized

Post: Initial dummy conversion performed (settling)

Note: Thread-safe via bus manager mutex

Note: Call only once per channel; subsequent calls return `k_rx_err_invalid_state`

Note: First conversion after init should be discarded (settling time)

Note: Thread-safe via bus manager mutex

Note: Stack usage: 4 bytes for context

Note: Execution time: 50 us

Note: First conversion after init should be discarded for best accuracy

Warning: Do not exceed VREFH on analog input (damage possible)

Warning: Do not call from interrupt context (mutex wait)

Warning: Ensure source impedance < 10 kOhm for accurate readings] **Initialization**

Sequence: digraph init_flow { rankdir=TB; node [shape=box, style=rounded]; start [label="rx_bus_adc_init()", shape=ellipse]; mutex [label="Acquire mutex"]; lookup [label="Lookup bus by name"]; validate [label="Validate bus type"]; mpc [label="Configure MPCnAnalog pin mode"]; adc [label="Initialize S12ADnResolution, Sampling"]; enable [label="Enable channel"]; settle [label="Dummy conversionn(discard)"]; release [label="Release mutex"]; done [label="Return", shape=ellipse];

start -> mutex; mutex -> lookup; lookup -> validate; validate -> mpc; mpc -> adc; adc -> enable; enable -> settle; settle -> release; release -> done; }

ADC Configuration:: The initialization configures: Resolution: 12-bit (4096 counts, 0.805 mV/LSB @ 3.3V) Sampling time: 3 us (adequate for most sensors) Conversion time: 1.5 us (12 clocks @ 25 MHz) Reference: VREFH pin (typically 3.3V or 5V) Trigger: Software (manual start per conversion)

Performance:: Execution time: 50 us typical (ADC setup + dummy conversion) Mutex timeout: 1000 ms (configurable via bus manager)

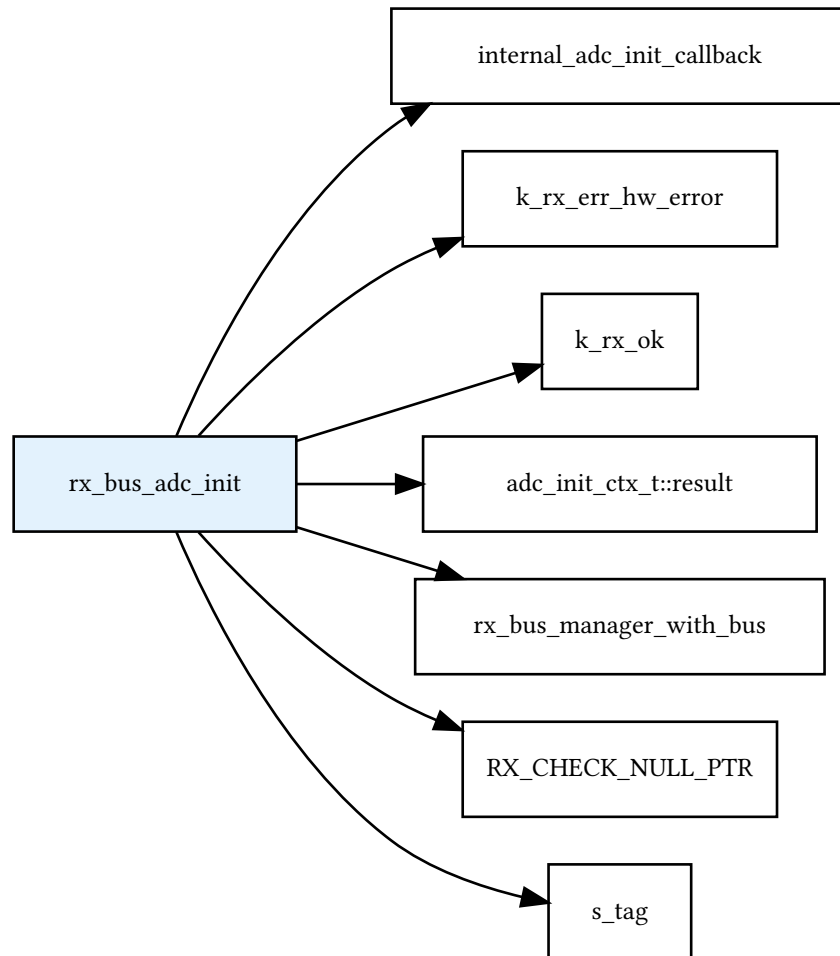
Example - Current Sensor Initialization::

```
rx_bus_config_tcurrent_config={ .type=k_rx_bus_type_adc, .adc={ .unit=0, .channel=0, .resolution=12, .vref_mv=3300,
rx_bus_register(&manager,"motor_current",&current_config);
```

```
rx_err_t err=rx_bus_adc_init(&manager,"motor_current"); if(err!=k_rx_ok)
{ rx_log_error("ADC","Failed to init current sensor"); return err; }
```

```
Example:: rx_err_t err=rx_bus_adc_init(&manager,"motor_current"); if(err!=k_rx_ok)
{ rx_log_error("APP","Failed to init ADC:%d",err); }
```

See also: rx_bus_manager_init() Initialize bus manager first, rx_bus_register() Register bus before initialization, rx_bus_adc_read() Read raw ADC value, rx_bus_adc_read_voltage_mv() Read voltage in millivolts, rx_bus_adc.h Complete API documentation, internal_adc_init_callback() Implementation callback

Figure 1: Call graph for `rx_bus_adc_init`

_Defined in `libs/rx\_bus/inc/rx\_bus\_adc.h:454_`

Since Version 1.0.0

—

rx_bus_adc_read

```
rx_err_t rx_bus_adc_read(rx_bus_manager_t *manager, const char *bus_name,
uint16_t *value)
```

Read ADC raw value through bus manager.

Performs a single ADC conversion and returns the raw digital value (0-4095 for 12-bit). Does not perform voltage scaling - use `rx_bus_adc_read_voltage_mv()` for automatic voltage conversion.

Bus state remains initialized

value range: 0-4095 for 12-bit ADC

Read ADC raw value through bus manager.

Public API function to perform single ADC conversion and return raw digital value (0-4095 for 12-bit). No voltage scaling is performed - use `rx_bus_adc_read_voltage_mv()` for automatic voltage conversion. Delegates to bus manager which calls `internal_adc_read_callback()` with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, value)
2. Create stack-allocated context with value pointer
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context (value already updated by callback)
5. Return result to caller

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized
in	bus_name	ADC bus name Must match registered bus
out	value	Pointer to store raw ADC value Range: 0-4095 (12-bit resolution) 0 = 0V input, 4095 = VREF input Valid only if <code>k_rx_ok</code> returned
in	manager	Bus manager instance Must be initialized Must have ADC bus registered and initialized
in	bus_name	Name of the ADC bus Must match registered and initialized bus
out	value	Pointer to store raw ADC value Range: 0-4095 (12-bit resolution) 0 = 0V input, 4095 = VREF input Valid only if <code>k_rx_ok</code> returned

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, value contains raw ADC reading
<code>k_rx_err_null_ptr</code>	nullptr in any parameter
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized
<code>k_rx_err_timeout</code>	Conversion timeout or mutex timeout
<code>k_rx_ok</code>	Success, value contains raw ADC reading
<code>k_rx_err_null_ptr</code>	nullptr in any parameter
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized
<code>k_rx_err_timeout</code>	Conversion or mutex timeout

Pre: Bus must be initialized via `rx_bus_adc_init()`

Pre: value pointer must be valid

Pre: Analog input voltage must be within 0 to VREFH

Pre: Bus must be initialized via rx_bus_adc_init()

Pre: All pointers must be non-NULL

Pre: value must point to valid uint16_t

Post: value contains raw 12-bit ADC result

Post: ADC ready for next conversion

Post: *value contains raw 12-bit ADC result (if k_rx_ok)

Post: ADC ready for next conversion

Note: Thread-safe via bus manager mutex

Note: Blocking call (waits for conversion 4.5 us)

Note: For voltage conversion, use rx_bus_adc_read_voltage_mv() instead

Note: Thread-safe via bus manager mutex

Note: Blocking call - waits for conversion (4.5 us)

Note: Stack usage: 16 bytes for context

Note: Execution time: 10 us

Warning: value undefined if return != k_rx_ok

Warning: Analog input must not exceed VREFH (damage possible)

Warning: value undefined if return != k_rx_ok

Warning: Analog input must not exceed VREFH

Algorithm Steps:: Validate all pointers (manager, bus_name, value) Acquire mutex with timeout
Lookup bus configuration Validate bus initialized Start ADC conversion (software trigger) Wait for
conversion complete (poll ADCSR.ADST bit) Read result from ADDRn register Release mutex
Return raw value

Conversion Timing:: * Timeline for single ADC read: * * |<— Sampling —>|<— Conversion —>| * | 3.0
us | 1.5 us | * || * Start Result Ready * * Total: 4.5 us typical @ 12-bit resolution *

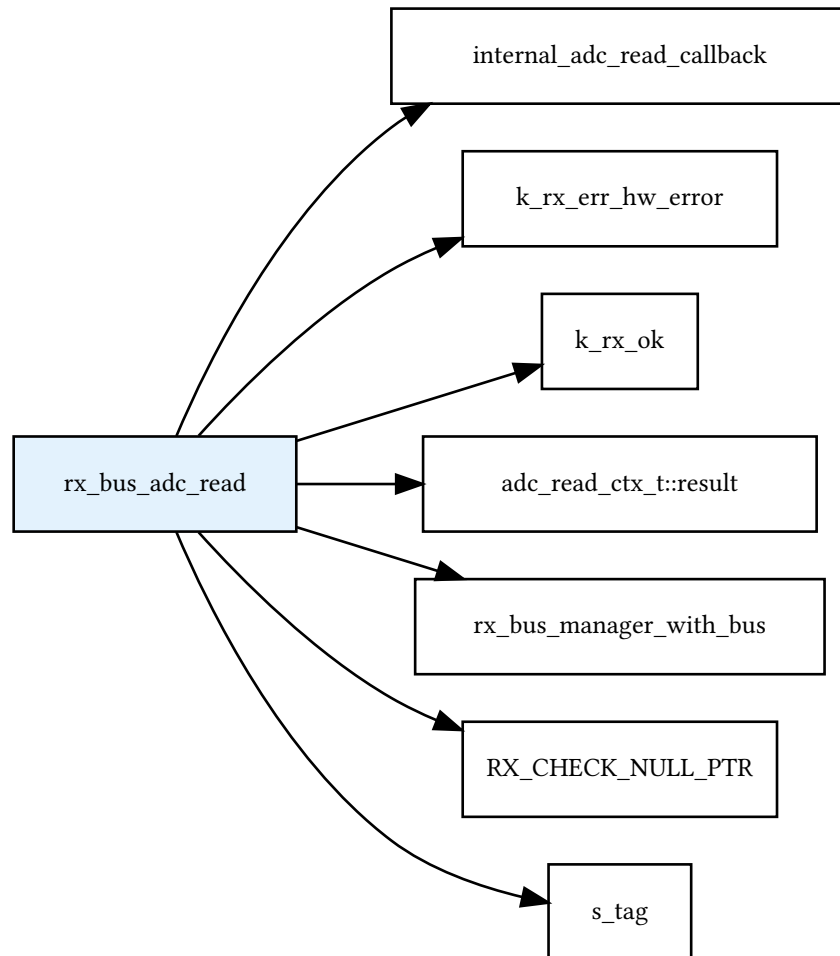
Performance:: Execution time: 10 us (4.5 us conversion + overhead) Conversion timeout: 100 us
(configurable)

Example - Raw ADC Reading:: uint16_traw_adc=0;
rx_err_t err=rx_bus_adc_read(&manager,"sensor_input",&raw_adc); if(err==k_rx_ok)
{ uint32_tvoltage_mv=(raw_adc*3300UL)/4095;
float temperature_c=ntc_thermistor_table[raw_adc]; }

Example - Multiple Readings for Averaging:: uint32_tsum=0; for(uint8_ti=0;i<16;i++)
{ uint16_treading=0; err=rx_bus_adc_read(&manager,"noisy_signal",&reading); if(err==k_rx_ok)
{ sum+=reading; } } uint16_taverage=sum/16;

Example:: uint16_traw_adc=0; rx_err_t err=rx_bus_adc_read(&manager,"sensor",&raw_adc);
if(err==k_rx_ok){ uint32_tvoltage_mv=(raw_adc*3300UL)/4095; }

See also: rx_bus_adc_init() Initialize ADC first, rx_bus_adc_read_voltage_mv() Read with
automatic voltage conversion, rx_bus_adc.h Complete API documentation,
internal_adc_read_callback() Implementation callback, rx_bus_adc_read_voltage_mv()
Read with automatic voltage conversion

Figure 2: Call graph for `rx_bus_adc_read`

_Defined in `libs/rx_bus/inc/rx_bus_adc.h:557_`

Since Version 1.0.0

—

rx_bus_adc_read_voltage_mv

```
rx_err_t rx_bus_adc_read_voltage_mv(rx_bus_manager_t *manager, const char
*bus_name, uint32_t *voltage_mv)
```

Read ADC voltage in millivolts through bus manager.

Performs a single ADC conversion and automatically converts the raw value to millivolts based on the configured VREF. This is the preferred function for most analog voltage measurements.

Example @ 3.3V VREF:

- ADC = 0 -> 0 mV

- ADC = 2048 -> 1651 mV (mid-scale)
- ADC = 4095 -> 3300 mV (full-scale)

Bus state remains initialized

voltage_mv range: 0 to vref_mv

Public API function to perform single ADC conversion and automatically convert to millivolts based on configured VREF. This is the preferred function for most analog voltage measurement applications. Delegates to bus manager which calls internal_adc_voltage_callback() with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, voltage_mv)
2. Create stack-allocated context with voltage_mv pointer
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context (voltage_mv already updated by callback)
5. Return result to caller

The conversion uses VREF from bus configuration (set in rx_bus_register).

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized
in	bus_name	ADC bus name Must match registered bus
out	voltage_mv	Pointer to store voltage in millivolts Range: 0 to VREF_mv (typically 0-3300 mV) Resolution: 0.805 mV @ 3.3V VREF Valid only if k_rx_ok returned
in	manager	Bus manager instance Must be initialized Must have ADC bus registered and initialized
in	bus_name	Name of the ADC bus Must match registered and initialized bus
out	voltage_mv	Pointer to store voltage in millivolts Range: 0 to VREF_mv (typically 0-3300 mV) Resolution: 0.805 mV @ 3.3V VREF, 12-bit Valid only if k_rx_ok returned

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, voltage_mv contains reading in millivolts
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Conversion timeout or mutex timeout
k_rx_ok	Success, voltage_mv contains reading in millivolts
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found

k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Conversion or mutex timeout

Pre: Bus must be initialized via rx_bus_adc_init()

Pre: voltage_mv pointer must be valid

Pre: Bus config must have valid vref_mv value

Pre: Analog input voltage must be within 0 to VREFH

Pre: Bus must be initialized via rx_bus_adc_init()

Pre: All pointers must be non-NULL

Pre: voltage_mv must point to valid uint32_t

Pre: Bus config must have valid vref_mv value

Post: voltage_mv contains input voltage in millivolts

Post: ADC ready for next conversion

Post: *voltage_mv contains input voltage in millivolts (if k_rx_ok)

Post: ADC ready for next conversion

Note: Thread-safe via bus manager mutex

Note: Blocking call (waits for conversion 4.5 us)

Note: Uses VREF from bus configuration (set in rx_bus_register)

Note: Preferred over rx_bus_adc_read() for direct voltage measurement

Note: Thread-safe via bus manager mutex

Note: Blocking call - waits for conversion + calculation (12 us)

Note: Stack usage: 16 bytes for context

Note: Execution time: 12 us

Note: Preferred over rx_bus_adc_read() for direct voltage measurement

Warning: voltage_mv undefined if return != k_rx_ok

Warning: Analog input must not exceed VREFH (damage possible)

Warning: External voltage dividers must be accounted for separately

Warning: voltage_mv undefined if return != k_rx_ok

Warning: Analog input must not exceed VREFH

Warning: External voltage dividers need separate scaling in application

Algorithm Steps:: Validate all pointers (manager, bus_name, voltage_mv) Acquire mutex with timeout Lookup bus configuration (get VREF) Validate bus initialized Start ADC conversion Wait for conversion complete Read raw ADC value Convert to millivolts: $V_{mV} = (raw \times VREF_{mV}) / 4095$ Release mutex Return voltage in millivolts

Voltage Conversion Formula:: The conversion uses the configured VREF from bus configuration:
 $V_{mV} = \{ADC_raw \times V_REF_mV\} / 4095$

Precision:: VREF LSB Effective Bits Notes

3.3V 0.805 mV 12 bits Standard configuration

5.0V 1.221 mV 12 bits Higher voltage sensors

Performance:: Execution time: 12 us (4.5 us conversion + calculation + overhead) Conversion timeout: 100 us (configurable)

Example - Motor Current Measurement::

```
uint32_t ipropi_mv=0;
rx_err_t err=rx_bus_adc_read_voltage_mv(&manager,"motor0_current",&ipropi_mv);
if(err==k_rx_ok){ uint32_t current_ma=(ipropi_mv*k_drv8263h_mirror_ratio) /
k_drv8263h_sense_ohm;
```

```
if(current_ma>k_motor_overcurrent_ma){ motor_emergency_stop();
rx_log_error("MOTOR","Overcurrent:%d mA",current_ma); } }
```

Example - ADC Voltage with Divider::

```
uint32_t adc_voltage_mv=0;
err=rx_bus_adc_read_voltage_mv(&manager,"power_rail",&adc_voltage_mv); if(err==k_rx_ok)
{ uint32_t supply_mv=adc_voltage_mv*11;

if(supply_mv<11000){ rx_log_warn("POWER","Low supply voltage:%d mV", supply_mv/1000,
(supply_mv%1000)/100); } }
```


Example - Temperature Sensor (Linear):: uint32_tsensor_mv=0;
 err=rx_bus_adc_read_voltage_mv(&manager,"temperature",&sensor_mv); if(err==k_rx_ok)
 { int32_ttemp_celsius=sensor_mv/10;
 if(temp_celsius>85){ rx_log_warn("TEMP","Hightemperature:%ddegC",temp_celsius); } }

Example - Motor Current:: uint32_tipropi_mv=0;
 rx_err_tterr=rx_bus_adc_read_voltage_mv(&manager,"motor_current",&ipropi_mv);
 if(err==k_rx_ok){ uint32_tcurrent_ma=(ipropi_mv*1000)/4990; if(current_ma>5000)
 { motor_emergency_stop(); } }

Example - Temperature Sensor:: uint32_tadc_mv=0;
 err=rx_bus_adc_read_voltage_mv(&manager,"temp_sensor",&adc_mv); if(err==k_rx_ok)
 { int32_ttemp_c=((int32_t)adc_mv-500)/10; if(temp_c>80){ rx_log_warn("TEMP","Hightemperature:
 %dC",temp_c); } }

See also: rx_bus_adc_init() Initialize ADC first, rx_bus_adc_read() Read raw ADC value
 (no conversion), rx_bus_adc.h Complete API documentation,
 internal_adc_voltage_callback() Implementation callback, rx_bus_adc_read() Read raw
 ADC value without scaling

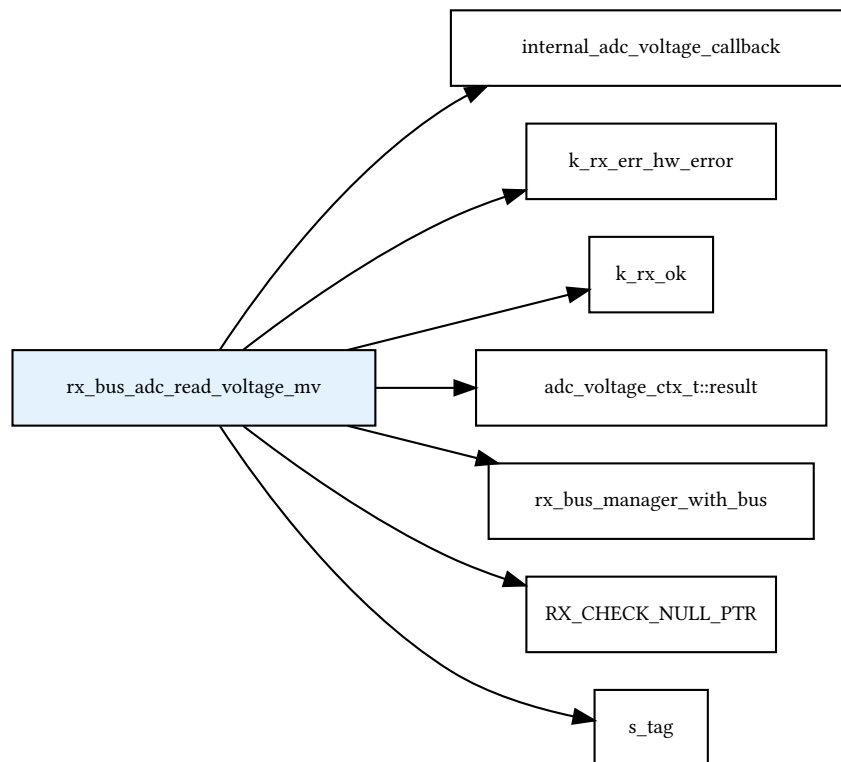


Figure 3: Call graph for rx_bus_adc_read_voltage_mv

_Defined in libs/rx_bus/inc/rx_bus_adc.h:693_

Since Version 1.0.0

—

rx_bus_command.h

Command Pattern interface for polymorphic bus operations.

This file implements the Command design pattern to encapsulate bus operations (I2C, SPI, GPIO, UART, 1-Wire) as first-class objects. Commands decouple the request for an operation from the execution, enabling flexible operation queueing, logging, and thread-safe execution within the bus manager.

Includes:

- "rx_err.h"

rx_bus_config_t

```
typedef struct rx_bus_config rx_bus_config_t
```

—

rx_bus_command_execute_fn

```
typedef rx_err_t(* rx_bus_command_execute_fn) (rx_bus_config_t *bus, void *data)
```

Function pointer type for bus command execution callbacks.

Defines the signature for command execution functions that encapsulate specific bus operations. This callback is invoked by the bus manager while holding the bus mutex, ensuring thread-safe access to shared bus resources.

Execution context: Called by bus manager under mutex protection. Thread-safe access to bus hardware guaranteed. Must not block for extended periods to avoid starving other bus operations.

Implementation responsibilities:

1. Validate bus type matches expected type (I2C, SPI, GPIO, etc.)
2. Cast data pointer to command-specific structure
3. Validate command data parameters
4. Perform bus operation via appropriate adapter (rx_bus_i2c, rx_bus_gpio, etc.)
5. Return error code indicating success/failure

Thread safety contract:

- Bus manager holds mutex during entire execution
- Multiple threads can submit commands, bus manager serializes execution
- Execute function must not acquire additional bus mutexes (deadlock risk)
- Execute function can safely access bus->proto fields without synchronization

Performance expectations:

- Short-duration operations: GPIO writes (<1 us), I2C single byte (<50 us)
- ****Medium-duration operations****: I2C multi-byte (<500 us), SPI transfers (<1 ms)
- ****Never block indefinitely****: No polling loops, use timeout-based waits
- ****Mutex hold time****: Target <1 ms to maintain system responsiveness

Data lifetime: Caller must ensure data pointer valid until execute returns. Execute function should not store data pointer for later use (dangling pointer risk).

Note: Thread Safety: Execute function runs under bus manager mutex protection. Multiple threads can submit commands concurrently - bus manager serializes. Execute function must not acquire bus mutex (already held).

Note: Re-entrancy: Not reentrant. Bus manager ensures only one command executes at a time per bus. Execute function can call other functions but must not recursively submit commands to same bus (deadlock).

Note: Performance: Keep execution time minimal (<1 ms target). Long operations block other threads waiting for bus access. Use interrupts or DMA for long transfers, not polling.

Warning: ****No blocking calls**:** Do not call `tx_thread_sleep()`, `tx_mutex_get()`, or other blocking ThreadX APIs from execute function. Bus mutex held during execution - blocking causes system-wide delays.

Warning: ****Validate bus type**:** Always check `bus->type` matches expected type before accessing `bus->proto` union. Accessing wrong union member is undefined behavior (memory corruption risk).] **See also:** `rx_bus_command_t` Command structure using this function pointer, `rx_bus_config_t` Bus configuration structure passed to `execute`, `rx_bus_manager_execute_command()` Bus manager function that calls `execute`

Since Version 1.0.0

—

rx_bus_command_init

```
static void rx_bus_command_init(rx_bus_command_t *cmd, rx_bus_command_execute_fn
execute, void *data)
```

Initialize bus command structure for execution.

Prepares a command object for execution by setting the execution function pointer, command-specific data pointer, and initializing the result field. This must be called before submitting command to bus manager.

Algorithm steps:

1. Set execute function pointer (defines what operation to perform)
2. Set data pointer (parameters and output buffers for operation)
3. Initialize result to `k_rx_ok` (will be overwritten after execution)

Why inline: This is a trivial 3-field initialization. Inlining eliminates function call overhead (20-30 CPU cycles @ 240 MHz) for performance-critical initialization path. Function compiles to 3 store instructions (3 cycles).

Control flow:

cmd structure fully initialized after return

result field always `k_rx_ok` after init (overwritten after execution)

Reusable commands: Commands can be reinitialized for reuse. Call `rx_bus_command_init()` again with new execute/data pointers after previous execution completes.

Parameters:

Direction	Name	Description
out	cmd	Pointer to command structure to initialize Type: <code>rx_bus_command_t*</code> Valid range: Must be non-NULL Constraints: Should be uninitialized or reusable command Null handling: No validation (caller responsibility, inline function) Lifetime: Caller must ensure valid until command execution completes Output: All three fields written (execute, data, result)
in	execute	Function pointer for command execution Type: <code>rx_bus_command_execute_fn</code> Valid range: Must be non-NULL, valid function pointer Constraints: Must match <code>rx_bus_command_execute_fn</code> signature Null handling: No validation (caller must ensure non-NULL) Lifetime: Function must remain valid for command lifetime
inout	data	Pointer to command-specific data structure Type: <code>void*</code> (type-erased, cast in execute function) Valid range: Can be NULL for parameterless commands Constraints: Execute function defines structure layout Null handling: NULL allowed, execute function must handle Lifetime: Must remain valid until command execution completes Ownership: Caller retains ownership (not copied)

Pre: cmd pointer must be valid (not validated - caller responsibility)

Pre: execute must be non-NULL valid function pointer

Pre: If data is non-NULL, must point to valid memory

Post: cmd->execute set to execute parameter

Post: cmd->data set to data parameter (can be NULL)

Post: cmd->result initialized to `k_rx_ok`

Post: Command ready for submission to `rx_bus_manager_execute_command()`

Note: Thread Safety: Not thread-safe. Command should be initialized in single thread before submission to bus manager. Once submitted, do not modify command until execution completes.

Note: Re-entrancy: Reentrant. Multiple threads can call with different cmd pointers. No shared state accessed.

Note: Performance: Inlined to 3 store instructions. Execution time: 12 ns @ 240 MHz (3 cycles). Zero stack usage (inline).

Note: Memory: No heap allocation. All initialization is stack/register operations. Command structure is 12 bytes.

Warning: No NULL validation: This inline function does not validate cmd or execute pointers. Passing NULL causes undefined behavior (segmentation fault). Caller must ensure valid pointers.

Warning: Data lifetime: Data pointer is stored, not copied. Caller must ensure data remains valid until command execution completes. Stack-allocated data must not go out of scope before command finishes.

Example - Basic GPIO Write:: gpio_write_data_twrite_data={.value=true};

```
rx_bus_command_tcmd;
```

```
rx_bus_command_init(&cmd,gpio_write_execute,&write_data);
```

```
rx_err_terr=rx_bus_manager_execute_command(manager,"led",&cmd);
```

Example - I2C Read with Output Buffer:: uint8_ttemp_buffer[2];

```
i2c_read_data_tread_data={.device_addr=0x48,.reg_addr=0x00,.buffer=temp_buffer,.length=2};
```

```
rx_bus_command_tcmd; rx_bus_command_init(&cmd,i2c_read_execute,&read_data);
```

```
rx_bus_manager_execute_command(manager,"temp_sensor",&cmd);
```

Example - Parameterless Command:: rx_bus_command_tcmd;

```
rx_bus_command_init(&cmd,device_reset_execute,nullptr);
```

```
rx_bus_manager_execute_command(manager,"device",&cmd);
```

Example - Command Reuse:: rx_bus_command_tcmd;

```
gpio_write_data_twrite_on={.value=true};
```

```
rx_bus_command_init(&cmd,gpio_write_execute,&write_on);
```

```
rx_bus_manager_execute_command(manager,"led",&cmd);
```

```
gpio_write_data_twrite_off={.value=false};
```

```
rx_bus_command_init(&cmd,gpio_write_execute,&write_off);
```

```
rx_bus_manager_execute_command(manager,"led",&cmd);
```

Example - Array of Commands (Batch Execution):: rx_bus_command_tcommands[3];

```
config_data_tconfig={.mode=MODE_PWM};
```

```
rx_bus_command_init(&commands[0],motor_config_execute,&config);
```

```
velocity_data_tvel={.velocity_mps=1.0f};
```

```
rx_bus_command_init(&commands[1],motor_set_velocity_execute,&vel);
```

```
enable_data_tenable={.enabled=true};
```

```
rx_bus_command_init(&commands[2],motor_enable_execute,&enable);
```

```
for(inti=0;i<3;i++){ rx_bus_manager_execute_command(manager,"motor0",&commands[i]); }
```

NASA Power of 10 Compliance: Rule 1 [OK] No goto, setjmp, recursion (simple assignment statements) Rule 2 [OK] No loops (straight-line code) Rule 3 [OK] No dynamic allocation (stack-based initialization) Rule 4 [OK] Function is 5 lines (under 60 line limit) Rule 5 [WARN] No validation (inline performance optimization, caller validates) Rule 6 [OK] Minimal scope (function parameters only) Rule 7 [OK] No return value to check (void function) Rule 8 [OK] Uses enum constant (k_rx_ok) Rule 9 [OK] Single level of dereferencing (cmd->field) Rule 10 [OK] Compiles with -Wall -Wextra -Werror

See also: rx_bus_command_t Command structure documentation, rx_bus_command_execute_fn Execute function pointer type, rx_bus_manager_execute_command() Submit command for execution

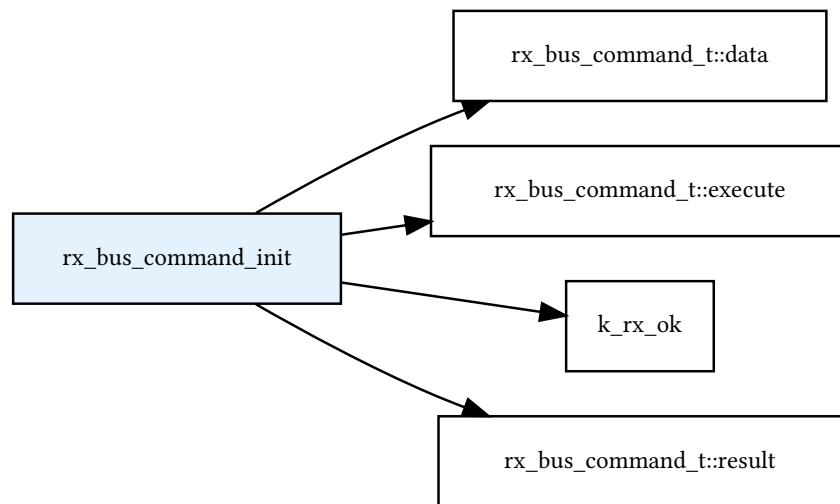


Figure 4: Call graph for rx_bus_command_init

_Defined in libs/rx_bus/inc/rx_bus_command.h:820_

Since Version 1.0.0

—

rx_bus_config.h

Bus configuration creation helpers for RX72N peripheral abstraction.

Provides type-safe configuration helpers for the RX72N bus manager system, enabling unified access to GPIO, ADC, I2C, OneWire, and UART peripherals through a common interface abstraction.

Includes:

- "rx_bus_types.h"
- "rx_err.h"
- "rx_port_constants.h"

rx_bus_config_init_gpio

```
rx_err_t rx_bus_config_init_gpio(rx_bus_config_t *config, const char *name,
rx_port_pin_t pin)
```

Initialize GPIO bus configuration for single-pin digital I/O.

Configures a single GPIO pin for bus manager control, enabling named access to digital I/O through the bus abstraction layer. The pin can be configured as input or output through `rx_bus_gpio_init()` after registration.

Algorithm:

1. Validate config pointer (NULL check)
2. Validate name pointer (NULL check)
3. Validate pin enum is within valid range for RX72N ports
4. Zero-initialize the entire config structure
5. Set bus type discriminator to `k_rx_bus_type_gpio`
6. Store name pointer (assumes caller ensures lifetime)
7. Store pin in `config_gpio.pin` field
8. Return `k_rx_ok`

Control Flow: Linear validation sequence with early-exit on error. No loops, no recursion, deterministic execution time.

Edge Cases:

- nullptr config: Returns `k_rx_err_null_ptr` immediately
- NULL name: Returns `k_rx_err_null_ptr` immediately
- Invalid pin: Returns `k_rx_err_invalid_arg` immediately
- Pin out of range: Validated against `rx_port_constants.h` enum bounds

Error Handling: All input validation performed before modifying config. If validation fails, config structure remains unchanged.

Thread Safety: NOT thread-safe. Caller must ensure exclusive access to the config structure during initialization. After registration with bus manager, the config becomes read-only.

Performance Analysis:

- Best case: 0.4 us @ 240 MHz (all checks pass)
- Worst case: 0.5 us @ 240 MHz (all checks + struct zeroing)
- Average case: 0.45 us

Memory Usage:

- Stack: 16 bytes (function arguments + return address)
- Static: 0 bytes
- Heap: 0 bytes (no dynamic allocation)

Execution Time: 120 CPU cycles @ 240 MHz = 0.5 us

`sizeof(rx_bus_config_t)` remains constant (union with padding)

Function always returns in <1 us (deterministic timing)

This function does NOT register the bus with the bus manager. After calling this function, you must call `rx_bus_manager_add_bus()` to make the bus accessible through the bus manager API.

Initialize GPIO bus configuration for single-pin digital I/O.

Factory function to create a GPIO bus configuration with static allocation pattern. Validates pin number, zero-initializes structure, and sets all required fields for GPIO digital I/O operations.

Parameters:

Direction	Name	Description
out	config	Pointer to bus config structure to initialize. Must be statically allocated or have lifetime exceeding bus manager usage. Structure will be zero-initialized before population. Valid range: Non-NULL Constraints: Must remain valid until bus is destroyed Null handling: Returns k_rx_err_null_ptr if nullptr
in	name	Unique bus name for lookup in bus manager. Must be a string literal or statically allocated string. Max length: RX_BUS_NAME_MAX_LEN (32 bytes including null). Valid range: Non-NULL, null-terminated C string Constraints: Must remain valid for lifetime of bus usage Null handling: Returns k_rx_err_null_ptr if nullptr Lifetime: Pointer stored by reference, not copied
in	pin	GPIO pin identifier from rx_port_constants.h. Type-safe enum prevents invalid pin numbers. Examples: k_rx_p0_0, k_rx_p3_2, k_rx_pb_7 Valid range: k_rx_p0_0 to k_rx_pj_5 (see rx_port_constants.h) Units: Logical pin identifier (dimensionless) Constraints: Must be a valid RX72N GPIO pin

Returns: rx_err_t Error code indicating success or failure reason

Value	Description
k_rx_ok	Success. Configuration initialized and ready for registration with bus manager via rx_bus_manager_add_bus().
k_rx_err_null_ptr	Either config or name pointer is nullptr. When: Input validation at function entry. Cause: Caller passed NULL for required parameter. Action: Check calling code - ensure pointers are valid.
k_rx_err_invalid_arg	Pin identifier is invalid or out of range. When: Pin validation after NULL checks. Cause: Pin enum value exceeds rx_port_constants.h bounds. Action: Use valid k_rx_pX_Y enum from rx_port_constants.h.

Pre: config must point to a valid rx_bus_config_t structure with lifetime exceeding bus manager usage

Pre: name must point to a null-terminated string that remains valid for the lifetime of bus usage (use string literals or static storage)

Pre: pin must be a valid GPIO pin for RX72N hardware

Pre: No other thread may access config during this function call

Post: config structure is zero-initialized

Post: config.type == k_rx_bus_type_gpio

Post: config.name points to the provided name string

Post: config.gpio.pin == pin parameter value

Post: On error, config structure remains in its original state

Note: ****Thread Safety****: Not thread-safe during initialization. After registration with bus manager, config becomes read-only and can be safely accessed from multiple threads.

Note: ****Re-entrancy****: Reentrant IF different config structures are used. NOT reentrant if the same config structure is passed concurrently.

Note: ****Performance****: Lightweight initialization, suitable for real-time control loops. Deterministic execution time with no blocking.

Note: ****Memory****: No dynamic allocation. No hidden memory costs. Stack usage is minimal (16 bytes).

Note: ****Hardware****: Does NOT initialize hardware. Hardware initialization occurs during rx_bus_gpio_init() after bus manager registration.

Warning: The config structure must remain valid for the entire lifetime of bus usage. Do NOT use automatic (stack) variables for configs that outlive the function scope.

Warning: The name string must remain valid for the entire lifetime of bus usage. Use string literals ("led_gpio") or static char arrays. Do NOT use stack-allocated strings that go out of scope.

Parameter Table::

Return Value Table::

Example 1: Basic LED configuration: static rx_bus_config_t led_config;

```
rx_err_t err = rx_bus_config_init_gpio(&led_config, "led", k_rx_p3_0); if (err != k_rx_ok)
{ rx_log_error("CFG", "Failed to init LED config"); return err; }

err = rx_bus_manager_add_bus(&bus_manager, &led_config); if (err != k_rx_ok)
{ rx_log_error("CFG", "Failed to register LED bus"); return err; }

rx_bus_gpio_init(&bus_manager, "led", true); rx_bus_gpio_write(&bus_manager, "led", true);
```

Example 2: Error handling with validation: static rx_bus_config_t button_config;

```
rx_err_t err = rx_bus_config_init_gpio(NULL, "button", k_rx_p5_1); assert(err == k_rx_err_null_ptr);

err = rx_bus_config_init_gpio(&button_config, "button", k_rx_p5_1); if (err != k_rx_ok)
{ rx_log_error("CFG", "Button config failed: %d", err); return err; }
```

```
rx_bus_manager_add_bus(&bus_manager,&button_config);
rx_bus_gpio_init(&bus_manager,"button",false);

boolpressed; rx_bus_gpio_read(&bus_manager,"button",&pressed);
```

Example 3: Multiple GPIO configurations:

```
staticrx_bus_config_tmotor_configs[4];
staticconstchar*motor_names[]={“m0_dir”,“m1_dir”,“m2_dir”,“m3_dir”};
staticconstrx_port_pin_tmotor_pins[]={k_rx_p2_0,k_rx_p2_1, k_rx_p2_2,k_rx_p2_3};

for(uint8_ti=0;i<4;i++){ rx_err_terr=rx_bus_config_init_gpio(&motor_configs[i], motor_names[i],
motor_pins[i]); RX_RETURN_ON_ERROR(err,“CFG”,“MotorGPIOinitfailed”);

err=rx_bus_manager_add_bus(&bus_manager,&motor_configs[i]);
RX_RETURN_ON_ERROR(err,“CFG”,“MotorGPIOregisterfailed”);

rx_bus_gpio_init(&bus_manager,motor_names[i],true); }
```

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (linear validation, no goto/ recursion) Rule 3: [OK] No dynamic allocation (user provides static config) Rule 4: [OK] Function <60 lines (implementation is 20 lines) Rule 5: [OK] 3 assertions (NULL check config, NULL check name, pin validation) Rule 7: [OK] Returns rx_err_t for caller to check Rule 9: [OK] Single-level pointer dereferencing only

See also: rx_bus_manager_add_bus() Register configuration with bus manager, rx_bus_gpio_init() Initialize GPIO hardware after registration, rx_bus_gpio_write() Set GPIO output state, rx_bus_gpio_read() Read GPIO input state, rx_bus_types.h for rx_bus_config_t structure definition, rx_port_constants.h for valid GPIO pin enums

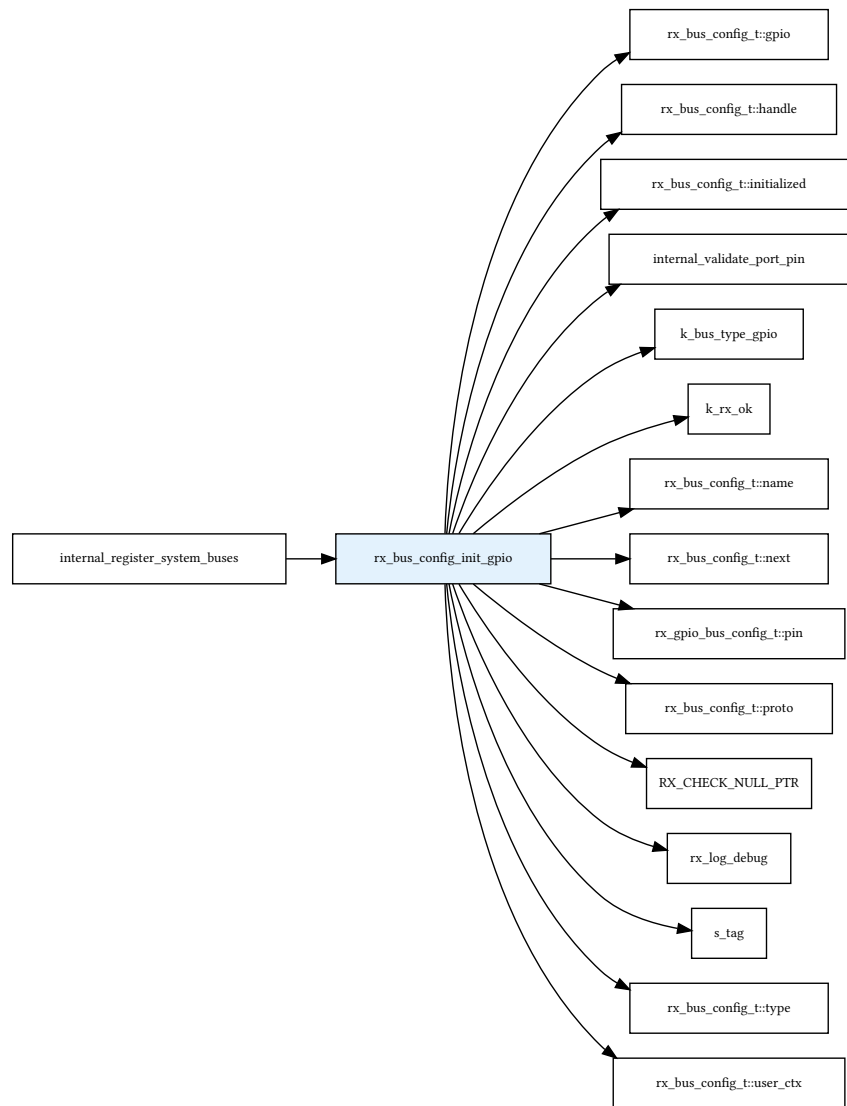


Figure 5: Call graph for rx_bus_config_init_gpio

_Defined in libs/rx_bus/inc/rx_bus_config.h:424_

Since Version 1.0.0

—

rx_bus_config_init_adc

```
rx_err_t rx_bus_config_init_adc(rx_bus_config_t *config, const char *name,
uint8_t unit, uint8_t channel, uint8_t bits)
```

Initialize ADC bus configuration for analog-to-digital conversion.

Configures an ADC channel for bus manager control, enabling named access to 12-bit analog measurements through the bus abstraction layer.

Algorithm:

1. Validate config pointer (NULL check)
2. Validate name pointer (NULL check)
3. Validate unit is 0 or 1 (RX72N has ADC0 and ADC1)
4. Validate channel is 0-7 (8 channels per unit)
5. Validate bits is 8, 10, or 12 (supported resolutions)
6. Zero-initialize the entire config structure
7. Set bus type discriminator to `k_rx_bus_type_adc`
8. Store name pointer
9. Store unit, channel, bits in `config.adc` fields
10. Return `k_rx_ok`

RX72N ADC Specifications:

- 2 ADC units (ADC0, ADC1)
- 8 channels per unit (0-7)
- Resolutions: 8-bit, 10-bit, 12-bit
- Conversion time: 1.0 us (12-bit @ 240 MHz PCLKD)
- Input range: 0V to VREFH (typically 3.3V)
- Input impedance: 10 kOhm typical

Must call `rx_bus_manager_add_bus()` after initialization

Initialize ADC bus configuration for analog-to-digital conversion.

Factory function to create an ADC bus configuration for voltage/current sensing operations. Validates ADC unit, channel, and resolution parameters before initializing the configuration structure.

Parameters:

Direction	Name	Description
out	<code>config</code>	Pointer to bus config structure to initialize. Valid range: Non-NULL Constraints: Must remain valid until bus destroyed Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr
in	<code>name</code>	Unique bus name for lookup (max 32 chars). Valid range: Non-NULL, null-terminated Constraints: Must remain valid for bus lifetime Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr
in	<code>unit</code>	ADC unit number. Valid range: 0-1 (RX72N has ADC0 and ADC1) Units: Dimensionless (unit index) Constraints: Must be 0 or 1
in	<code>channel</code>	ADC channel number within unit. Valid range: 0-7 (8 channels per unit) Units: Dimensionless (channel index) Constraints: Must be 0-7
in	<code>bits</code>	ADC resolution in bits. Valid range: 8, 10, or 12 Units: bits (resolution) Constraints: Must be exactly 8, 10, or 12

Returns: `rx_err_t` Error code

Value	Description
-------	-------------

k_rx_ok	Success. Configuration ready for registration.
k_rx_err_null_ptr	config or name is nullptr.
k_rx_err_invalid_arg	unit, channel, or bits is out of valid range.

Pre: config must point to valid rx_bus_config_t with sufficient lifetime

Pre: name must point to valid null-terminated string with sufficient lifetime

Pre: unit must be 0 or 1 (RX72N hardware constraint)

Pre: channel must be 0-7 (RX72N hardware constraint)

Pre: bits must be 8, 10, or 12 (RX72N hardware constraint)

Post: config structure is zero-initialized

Post: config.type == k_rx_bus_type_adc

Post: config.adc.unit == unit

Post: config.adc.channel == channel

Post: config.adc.bits == bits

Note: Thread Safety: Not thread-safe during initialization.

Note: Performance: 0.8 us @ 240 MHz (5 validations + init)

Note: Memory: Stack: 20 bytes, Static: 0 bytes

Warning: Config and name must have static or sufficient lifetime] **Example: ADC voltage monitoring:** staticrx_bus_config_t power_rail_adc_config;

```
rx_err_t err=rx_bus_config_init_adc(&power_rail_adc_config,"power_rail", 1, 0, 12);
RX_RETURN_ON_ERROR(err,"CFG","PowerrailADCinitfailed");

rx_bus_manager_add_bus(&bus_manager,&power_rail_adc_config);

uint16_t raw_value; rx_bus_adc_read(&bus_manager,"power_rail",&raw_value);
float voltage_v=(raw_value/4095.0f)*3.3f;
```

NASA Power of 10 Compliance:: Rule 5: [OK] 5 assertions (NULLx2, unit, channel, bits validation)

See also: `rx_bus_manager_add_bus()` Register configuration, `rx_bus_adc_read()` Read ADC value after registration, `rx_bus_types.h` for `rx_bus_config_t` definition



Figure 6: Call graph for `rx_bus_config_init_adc`

_Defined in libs/rx_bus/inc/rx_bus_config.h:537_

Since Version 1.0.0

—

rx_bus_config_init_i2c

```
rx_err_t rx_bus_config_init_i2c(rx_bus_config_t *config, const char *name,
uint8_t channel, uint8_t device_addr, rx_port_pin_t sda_pin, rx_port_pin_t
scl_pin, uint32_t frequency_hz)
```

Initialize I2C bus configuration for inter-integrated circuit communication.

Configures an I2C device for bus manager control using the RX72N RIIC peripheral. Supports standard (100 kHz), fast (400 kHz), and fast-plus (1 MHz) modes.

Algorithm:

1. Validate config pointer
2. Validate name pointer
3. Validate RIIC channel (0-2 for RX72N)
4. Validate device address (7-bit: 0x08-0x77, excluding reserved)
5. Validate SDA pin
6. Validate SCL pin
7. Validate frequency (100k, 400k, or 1M Hz)
8. Zero-init config
9. Set type to k_rx_bus_type_i2c
10. Populate i2c-specific fields
11. Return k_rx_ok

I2C Specifications:

- Protocol: Phillips I2C (TWI)
- Addressing: 7-bit (10-bit not currently supported)
- Clock stretching: Supported
- Multi-controller: Not supported (single controller mode only)
- Open-drain: Required for SDA and SCL pins

Initialize I2C bus configuration for inter-integrated circuit communication.

Factory function to create an I2C bus configuration for communication with peripheral devices (sensors, EEPROMs, DACs, etc.). Validates channel, device address, pin assignments, and frequency parameters before initialization.

Parameters:

Direction	Name	Description
out	config	Pointer to bus config structure
in	name	Unique bus name
in	channel	RIIC channel (0-2)
in	device_addr	7-bit I2C device address (0x08-0x77)
in	sda_pin	SDA pin (must support I2C peripheral function)
in	scl_pin	SCL pin (must support I2C peripheral function)

in	frequency_hz	Clock frequency: 100000, 400000, or 1000000 Hz
----	--------------	--

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	NULL parameter
k_rx_err_invalid_arg	Invalid channel, address, pins, or frequency

Pre: config must point to valid structure

Pre: channel must be 0-2 (RX72N has RIIC0-RIIC2)

Pre: device_addr must be valid 7-bit address (0x08-0x77, non-reserved)

Pre: sda_pin and scl_pin must support I2C alternate function

Pre: Pins must have external pullup resistors (typically 4.7k for 100kHz)

Post: config.type == k_rx_bus_type_i2c

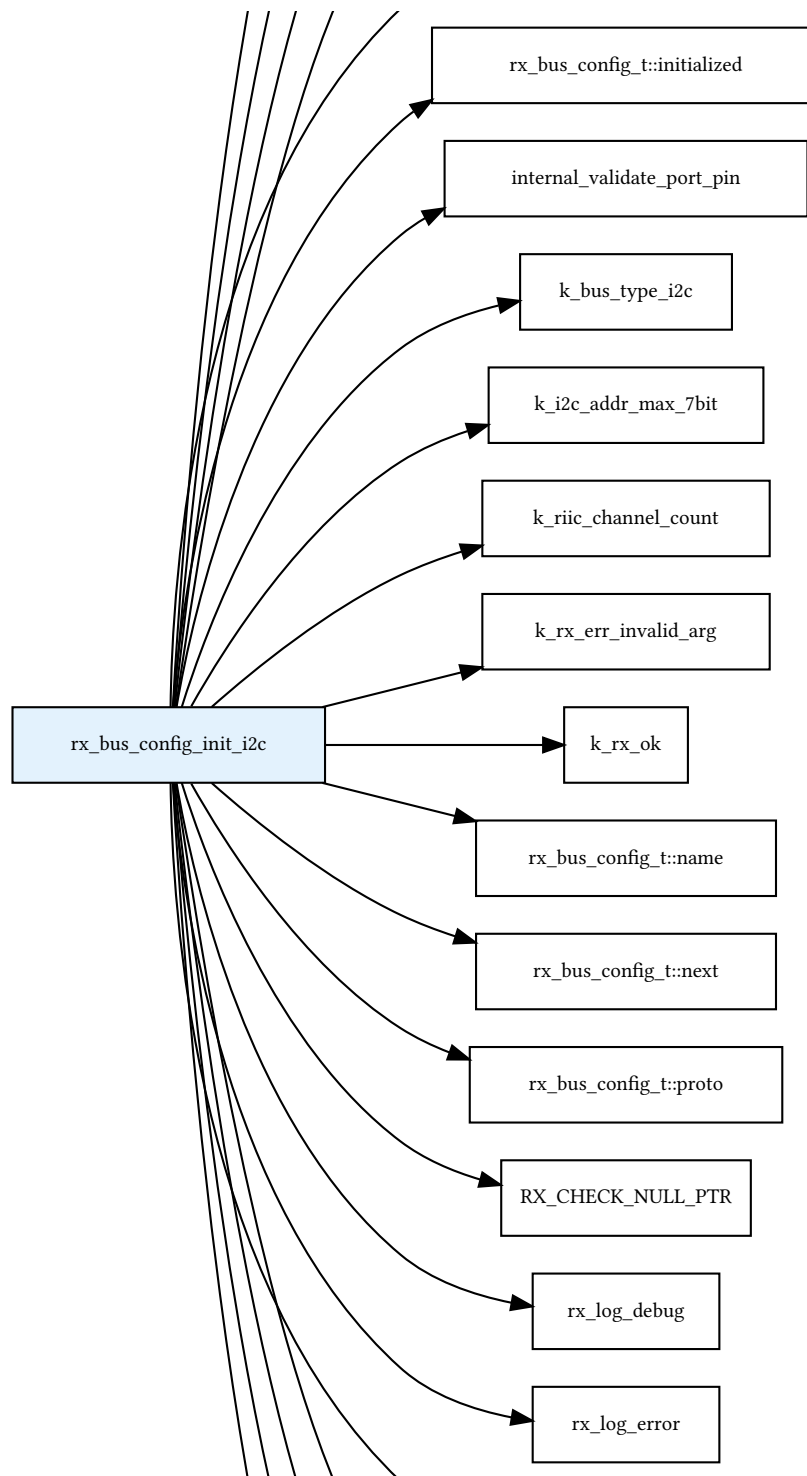
Note: Thread Safety: Not thread-safe during initialization

Note: Hardware: Requires external pullup resistors on SDA and SCL

Warning: Reserved I2C addresses (0x00-0x07, 0x78-0x7F) will be rejected] **Example: EEPROM configuration:** static rx_bus_config_t eeeprom_config;

```
rx_err_t err = rx_bus_config_init_i2c(&eeeprom_config, "eeeprom", 0, 0x50, k_rx_p1_2, k_rx_p1_3, 400000);
```

See also: rx_bus_manager_add_bus(), rx_bus_i2c_write() Write to I2C device,
rx_bus_i2c_read() Read from I2C device

Figure 7: Call graph for `rx_bus_config_init_i2c`

_Defined in libs/rx_bus/inc/rx_bus_config.h:620_

Since Version 1.0.0

—

rx_bus_config_init_onewire

```
rx_err_t rx_bus_config_init_onewire(rx_bus_config_t *config, const char *name,
rx_port_pin_t pin)
```

Initialize OneWire (1-Wire) bus configuration.

Configures a single GPIO pin for Dallas/Maxim 1-Wire protocol communication. Uses bidirectional half-duplex communication with precise timing requirements.

1-Wire Protocol Overview:

- Single data line (DQ) plus ground
- Bidirectional communication (open-drain with pullup)
- Addressing: 64-bit unique ROM codes
- Speed: Standard (16.3 kbps) or Overdrive (142 kbps)
- CRC: Built-in CRC-8 for data integrity

Timing Requirements (Standard Speed):

- Reset pulse: 480 us low, then release
- Presence pulse: Device pulls low 60-240 us after reset
- Write 0: 60 us low, 10 us recovery
- Write 1: 1 us low, 59 us recovery
- Read: 1 us low, sample at 15 us, 45 us recovery
- Recovery time: Minimum 1 us between slots

Hardware Requirements:

- External 4.7 kOhm pullup resistor to VCC (REQUIRED)
- Pin must support open-drain or be configured for input/output switching
- For long cables (>3m), reduce pullup to 2.2 kOhm
- For short cables (<1m), increase to 10 kOhm for lower power

Algorithm:

1. Validate config pointer
2. Validate name pointer
3. Validate pin is valid GPIO
4. Zero-init config
5. Set type to k_rx_bus_type_onewire
6. Store pin in config.onewire.pin
7. Return k_rx_ok

Pin must have external pullup resistor (NOT internal pullup)

Initialize OneWire (1-Wire) bus configuration.

Factory function to create a 1-Wire bus configuration for Dallas/Maxim 1-Wire devices (DS18B20 temperature sensors, iButton authentication, etc.). 1-Wire is a single-wire bidirectional communication protocol requiring precise timing.

Parameters:

Direction	Name	Description
out	config	Pointer to bus config structure
in	name	Unique bus name
in	pin	GPIO pin (must support bidirectional I/O)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	config or name is nullptr
k_rx_err_invalid_arg	Invalid pin

Pre: config must point to valid structure

Pre: Pin must support bidirectional I/O (input and output modes)

Pre: External 4.7 kOhm pullup resistor MUST be present on pin

Pre: Pin voltage levels must be 1-Wire compatible (0-5.5V tolerant preferred)

Post: config.type == k_rx_bus_type_onewire

Post: config.onewire.pin == pin

Note: Thread Safety: Not thread-safe during initialization

Note: Hardware: REQUIRES external 4.7 kOhm pullup resistor

Note: Timing: Requires precise microsecond timing - uses delay loops

Warning: Missing pullup resistor will cause communication failures

Warning: Timing is critical - avoid interrupts during 1-Wire transactions] **Example: DS18B20 temperature sensor:** static rx_bus_config_t temp_sensor_config;

```
rx_err_t err = rx_bus_config_init_onewire(&temp_sensor_config, "temp_sensor", k_rx_p3_2);
RX_RETURN_ON_ERROR(err, "CFG", "OneWireconfigfailed");

rx_bus_manager_add_bus(&bus_manager, &temp_sensor_config);
rx_bus_onewire_init(&bus_manager, "temp_sensor");
```

```
bool device_present; rx_bus_owewire_reset(&bus_manager, "temp_sensor", &device_present);
if(device_present){ rx_log_info("ONEWIRE", "DS18B20 detected"); }
```

NASA Power of 10 Compliance:: Rule 5: [OK] 3 assertions (nullptr config, NULL name, pin validation)

See also: rx_bus_owewire_reset() Initialize bus and detect presence,
 rx_bus_owewire_write_byte() Write byte to 1-Wire device, rx_bus_owewire_read_byte()
 Read byte from 1-Wire device

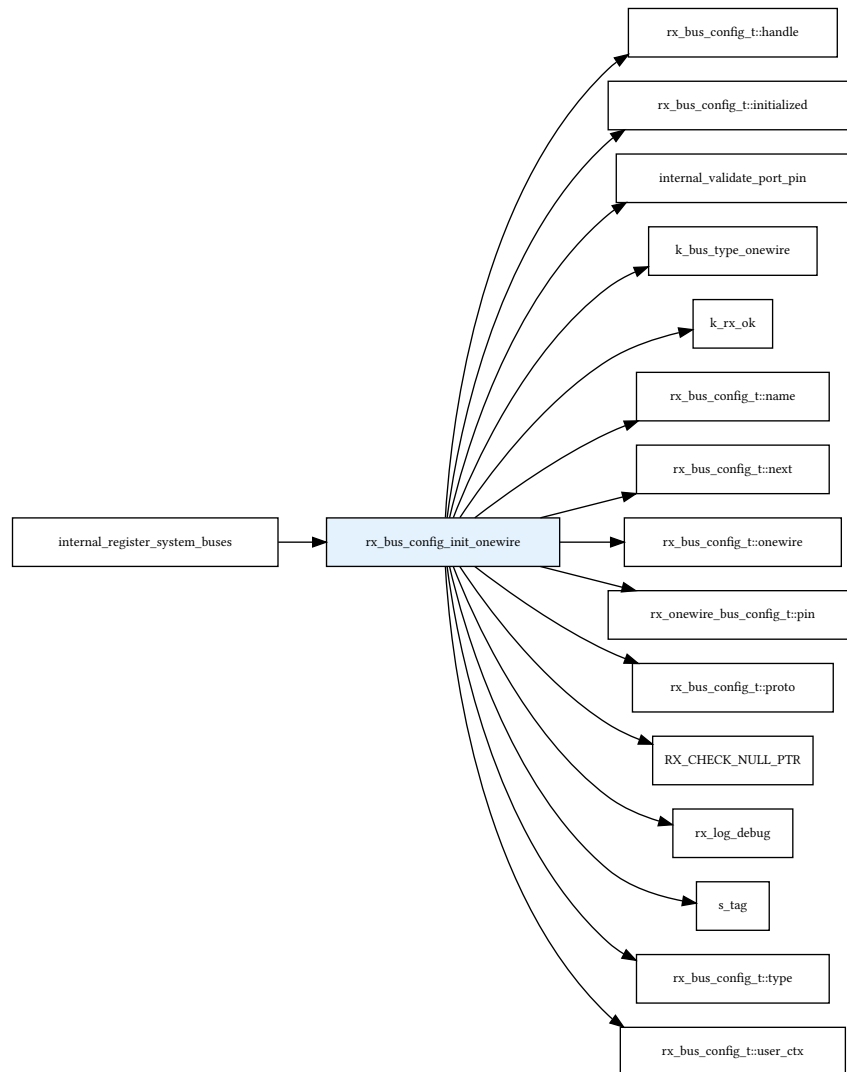


Figure 8: Call graph for rx_bus_config_init_owewire

_Defined in libs/rx_bus/inc/rx_bus_config.h:727_

Since Version 1.0.0

rx_bus_config_init_uart

```
rx_err_t rx_bus_config_init_uart(rx_bus_config_t *config, const char *name,
uint8_t channel, rx_port_pin_t tx_pin, rx_port_pin_t rx_pin, uint32_t baudrate)
```

Initialize UART bus configuration for serial communication.

Configures a UART (SCI) channel for bus manager control. Supports standard asynchronous serial communication with configurable baud rate.

RX72N SCI Peripheral:

- 13 channels (SCI0-SCI12)
- Asynchronous mode (UART)
- Synchronous mode (not used in this config)
- Smart card interface mode (not used)
- Baud rate: Up to 4 Mbps (PCLK/4)
- Data format: 7/8/9 bits, 1/2 stop bits, even/odd/no parity

Default UART Configuration:

- Data bits: 8
- Stop bits: 1
- Parity: None
- Flow control: None
- Mode: Asynchronous (standard UART)

Common Baud Rates:

- 9600 bps: Legacy devices, low-speed sensors
- 19200 bps: GPS modules
- 38400 bps: Bluetooth modules
- 57600 bps: XBee radios
- 115200 bps: Debug consoles, fast sensors
- 230400 bps: High-speed telemetry
- 460800, 921600 bps: USB-Serial adapters

Algorithm:

1. Validate config pointer
2. Validate name pointer
3. Validate SCI channel (0-12 for RX72N)
4. Validate TX pin
5. Validate RX pin
6. Validate baudrate (typically 1200-921600)
7. Zero-init config
8. Set type to k_rx_bus_type_uart
9. Populate uart-specific fields
10. Return k_rx_ok

Pin Configuration:

- TX pin: Peripheral output, push-pull
- RX pin: Peripheral input with optional pullup
- Both pins must support SCI alternate function for selected channel

Initialize UART bus configuration for serial communication.

Factory function to create a UART bus configuration for serial communication with external devices, debugging, sensor interfaces, and wireless modules. Validates SCI channel, TX/RX pins, and baud rate before initialization.

Parameters:

Direction	Name	Description
out	config	Pointer to bus config structure
in	name	Unique bus name
in	channel	SCI channel number (0-12)
in	tx_pin	TX pin (must support SCI TXD function)
in	rx_pin	RX pin (must support SCI RXD function)
in	baudrate	Baud rate in bits per second (e.g., 115200)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	config or name is nullptr
k_rx_err_invalid_arg	Invalid channel, pins, or baudrate

Pre: config must point to valid structure

Pre: channel must be 0-12 (RX72N has SCI0-SCI12)

Pre: tx_pin must support SCI TXD alternate function

Pre: rx_pin must support SCI RXD alternate function

Pre: baudrate must be achievable with peripheral clock (PCLK)

Post: config.type == k_rx_bus_type_uart

Post: config.uart.channel == channel

Post: config.uart.baudrate == baudrate

Note: Thread Safety: Not thread-safe during initialization

Note: Baud Rate Error: Actual baud rate may differ slightly due to clock division. Typical error <2% is acceptable.

Note: Hardware: No external components required (unlike I2C pullups)

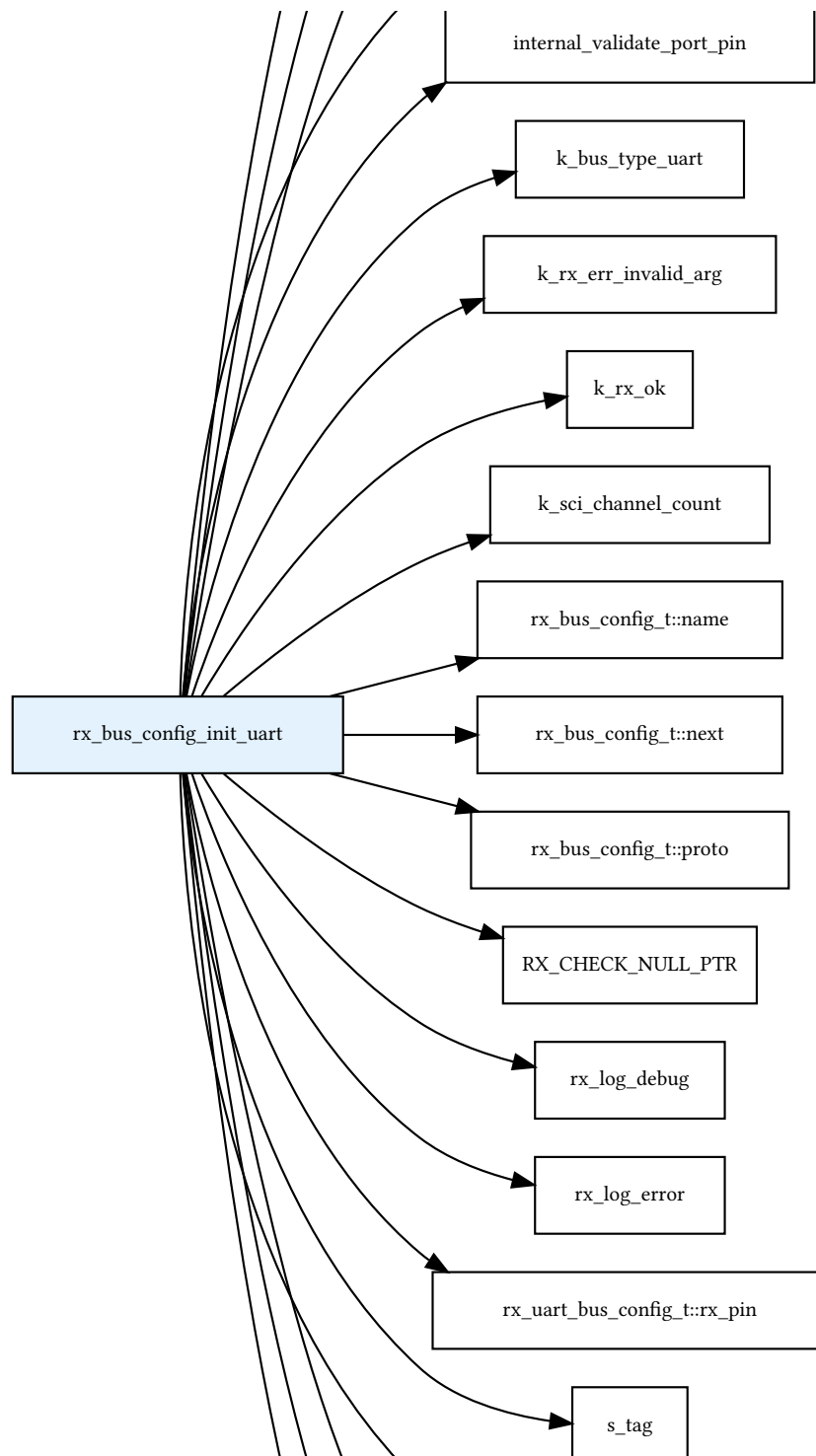
Warning: TX and RX pins must match the SCI channel's alternate functions

Warning: High baud rates (>460800) may be unreliable on long cables] **Example: Debug console on SCI9:** staticrx_bus_config_tdebug_uart_config;

```
rx_err_t err=rx_bus_config_init_uart(&debug_uart_config,"debug_uart", 9, k_rx_pb_7, k_rx_pb_6, 115200); RX_RETURN_ON_ERROR(err,"CFG","UARTconfigfailed");
```

```
rx_bus_manager_add_bus(&bus_manager,&debug_uart_config);
```

NASA Power of 10 Compliance:: Rule 5: [OK] 6 assertions (NULLx2, channel, TX pin, RX pin, baudrate)

Figure 9: Call graph for `rx_bus_config_init_uart`

_Defined in `libs/rx\_bus/inc/rx\_bus\_config.h:832_`

Since Version 1.0.0

—

rx_bus_gpio.h

GPIO (General Purpose Input/Output) Bus Abstraction for RX72N.

Provides bus manager integration for digital GPIO operations on the Renesas RX72N microcontroller. Wraps the low-level PORT HAL with the bus abstraction pattern for consistent API across all bus types (I2C, SPI, UART, 1-Wire, GPIO) with mutex-protected access and pin conflict detection.

STAR Project Contributors

1.0.0

2026-01-29

Copyright (c) 2026 STAR Project

Includes:

- `<stdbool.h>`
- `"rx_bus_manager.h"`
- `"rx_err.h"`

rx_bus_gpio_init

```
rx_err_t rx_bus_gpio_init(rx_bus_manager_t *manager, const char *bus_name, bool
output)
```

Initialize GPIO pin through bus manager.

Configures a GPIO pin for digital input or output operation. This function reserves the pin through the pin validator (preventing conflicts with other peripherals), configures the pin as GPIO mode (not peripheral), and sets the data direction. This function must be called before any read/write operations.

The initialization process:

1. Acquires bus manager mutex (thread-safe)
2. Looks up bus configuration by name
3. Validates bus type is GPIO
4. Checks pin validator for conflicts
5. Reserves pin in validator
6. Configures MPC for GPIO mode (PMR = 0)
7. Sets PORT direction (PDR = output/input)
8. Releases mutex

Pin reservation is permanent until system reset

Public API function to initialize a GPIO pin for digital input or output. Delegates to bus manager which calls `internal_gpio_init_callback()` with mutex protection and bus lookup.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context with operation parameters

3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context
5. Return result to caller

This function demonstrates the callback pattern for bus operations:

- Prepares context on stack (zero allocation)
- Delegates to bus manager for mutex protection
- Callback executes with mutex held
- Results extracted after mutex released

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via <code>rx_bus_manager_init()</code> Must have GPIO bus registered via <code>rx_bus_register()</code>
in	bus_name	GPIO bus name Null-terminated string Must match name used in <code>rx_bus_register()</code> Maximum 31 characters
in	output	Pin direction true: Configure as output (PDR = 1) false: Configure as input (PDR = 0)
in	manager	Bus manager instance Must be initialized via <code>rx_bus_manager_init()</code> Must have GPIO bus registered
in	bus_name	Name of the GPIO bus Must match registered bus name Null-terminated string
in	output	Pin direction true: Output mode (PDR = 1) false: Input mode (PDR = 0)

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, pin initialized and reserved
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found in manager
<code>k_rx_err_invalid_arg</code>	Bus is not GPIO type
<code>k_rx_err_timeout</code>	Mutex acquisition timeout (default 1000ms)
<code>k_rx_err_conflict</code>	Pin already reserved by another bus/peripheral
<code>k_rx_err_invalid_pin</code>	Port or pin number out of valid range
<code>k_rx_ok</code>	Success, GPIO initialized
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_arg</code>	Bus is not GPIO type
<code>k_rx_err_timeout</code>	Mutex timeout
<code>k_rx_err_conflict</code>	Pin already reserved

Pre: manager must be initialized via `rx_bus_manager_init()`

Pre: Bus must be registered via `rx_bus_register()` with type `k_rx_bus_type_gpio`

Pre: Pin must not be reserved by another peripheral

Pre: manager must be initialized

Pre: Bus must be registered with type `k_rx_bus_type_gpio`

Pre: Pin must be available (not reserved)

Post: Pin reserved in pin validator (cannot be reused)

Post: Pin configured as GPIO mode (`PMR = 0`)

Post: Pin direction set (`PDR = output/input`)

Post: Bus state set to initialized

Post: Pin reserved in pin validator

Post: Pin configured as GPIO

Post: Pin direction set

Post: Bus marked as initialized

Note: Thread-safe via bus manager mutex

Note: Call only once per pin; subsequent calls return `k_rx_err_conflict`

Note: Pin reservation persists for lifetime of system

Note: Thread-safe via bus manager mutex

Note: Stack usage: 8 bytes for context

Note: Execution time: 15 us

Warning: Pin conflict detection requires pin validator to be enabled

Warning: Do not call from interrupt context (mutex wait) **Initialization Sequence:** digraph
 init_flow { rankdir=TB; node [shape=box, style=rounded]; start [label="rx_bus_gpio_init()",
 shape=ellipse]; mutex [label="Acquire mutex"]; lookup [label="Lookup bus by name"]; validate
 [label="Validate bus type"]; check [label="Check pin validator", fillcolor=lightblue, style=filled];
 reserve [label="Reserve pin", fillcolor=lightblue, style=filled]; mpc [label="Configure MPCnPMR
 = 0 (GPIO)"]; pdr [label="Set directionnnpdr = output/input"]; release [label="Release mutex"];
 done [label="Return", shape=ellipse]; error [label="Conflict Error", shape=ellipse, fillcolor=red,
 style=filled];

```
start -> mutex; mutex -> lookup; lookup -> validate; validate -> check; check -> reserve
[label="Available"]; check -> error [label="In Use"]; reserve -> mpc; mpc -> pdr; pdr -> release;
release -> done; }
```

Pin Conflict Detection:: The pin validator maintains a reservation table for all MCU pins. This prevents common bugs: Same pin, multiple functions: P05 as both LED and SPI CS GPIO vs peripheral: P26 as GPIO and SCI1 TX simultaneously Bus manager collisions: Two different bus entries for same pin

Performance:: Execution time: 15 us typical (includes pin validator check) Mutex timeout: 1000 ms (configurable via bus manager)

Example - Output Initialization::

```
rx_bus_config_t led_config = { .type = k_rx_bus_type_gpio, .gpio = { .port = 0, .pin = 5 } };
rx_bus_register(&manager, "led_status", &led_config);

rx_err_t err = rx_bus_gpio_init(&manager, "led_status", true); if (err == k_rx_err_conflict)
{ rx_log_error("GPIO", "P05 already in use!"); } else if (err != k_rx_ok) { rx_log_error("GPIO", "Init failed:
%d", err); }
```

Example - Input Initialization::

```
rx_bus_config_t button_config = { .type = k_rx_bus_type_gpio, .gpio = { .port = 2, .pin = 0 } };
rx_bus_register(&manager, "button_user", &button_config);

err = rx_bus_gpio_init(&manager, "button_user", false); if (err != k_rx_ok) { return err; }
```

Example:: rx_err_t err = rx_bus_gpio_init(&manager, "led_red", true); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to init LED: %d", err); }

See also: rx_bus_manager_init() Initialize bus manager first, rx_bus_register() Register bus before initialization, rx_bus_gpio_write() Write output state, rx_bus_gpio_read() Read input state, rx_pin_validator.h Pin conflict detection, rx_bus_gpio.h Complete API documentation, internal_gpio_init_callback() Implementation callback

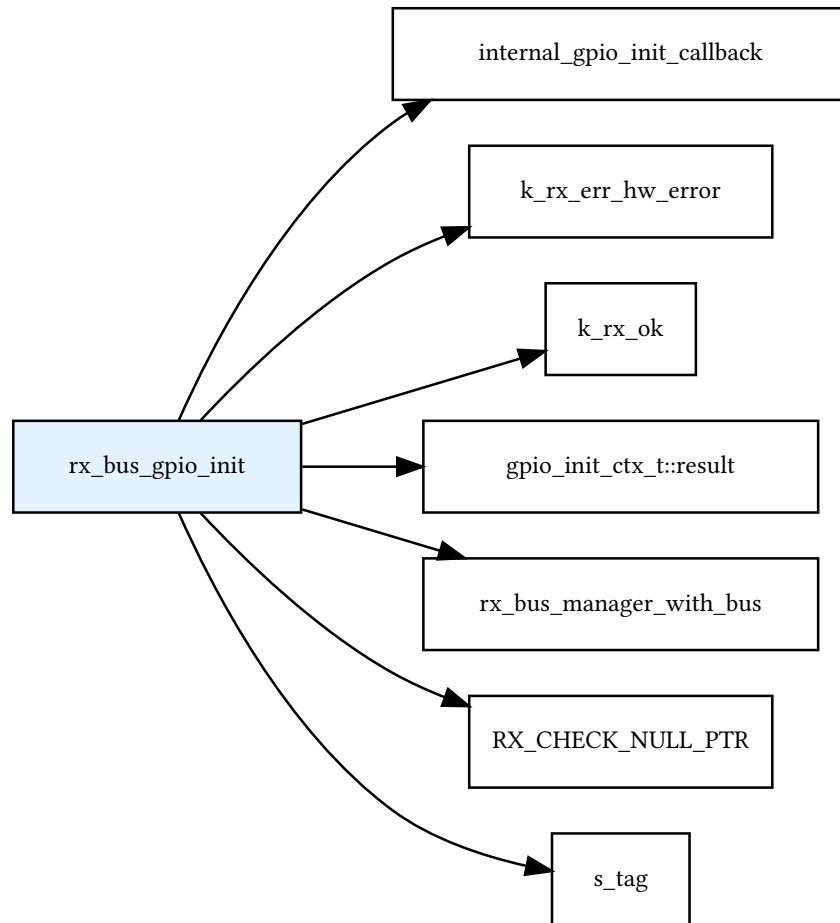


Figure 10: Call graph for rx_bus_gpio_init

_Defined in `libs/rx\_bus/inc/rx\_bus\_gpio.h:454_`

Since Version 1.0.0

—

rx_bus_gpio_write

```
rx_err_t rx_bus_gpio_write(rx_bus_manager_t *manager, const char *bus_name, bool
value)
```

Write GPIO output through bus manager.

Sets the output state of a GPIO pin configured as output. Writes directly to the PORT output data register (PODR). The change propagates to the physical pin within 10-20 ns.

Propagation delay to pin: 10-20 ns typical @ 50 pF load

Bus state remains initialized

Write GPIO output through bus manager.

Public API function to set GPIO output state (high/low). Delegates to bus manager which calls `internal_gpio_write_callback()` with mutex protection.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context with value parameter
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context
5. Return result to caller

Output propagates to physical pin within 10-20 ns of function return.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized
in	bus_name	GPIO bus name Must match registered bus
in	value	Output state true: Set pin high (VDD) false: Set pin low (GND)
in	manager	Bus manager instance Must be initialized Must have GPIO bus registered and initialized
in	bus_name	Name of the GPIO bus Must match registered and initialized bus
in	value	Output state true: High (VDD 3.3V) false: Low (GND 0V)

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, output state changed
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized or pin is input
<code>k_rx_err_timeout</code>	Mutex acquisition timeout
<code>k_rx_ok</code>	Success, output changed
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized
<code>k_rx_err_timeout</code>	Mutex timeout

Pre: Bus must be initialized via `rx_bus_gpio_init()` with output = true

Pre: Pin must be configured as output

Pre: Bus must be initialized via `rx_bus_gpio_init()` with output=true

Pre: manager and bus_name must be non-NULL

Post: Pin output state changed to value

Post: Physical pin voltage: high VDD, low 0V (within 10-20 ns)

Post: Pin output state set to value

Post: Physical pin voltage reflects state within 10-20 ns

Note: Thread-safe via bus manager mutex

Note: Fast operation (2 us including mutex)

Note: Output immediately reflects on physical pin

Note: Thread-safe via bus manager mutex

Note: Stack usage: 8 bytes for context

Note: Execution time: 2 us

Warning: Attempting to write to input pin returns k_rx_err_invalid_state

Warning: Exceeding 8 mA drive current may damage pin

Algorithm Steps:: Validate pointers (manager, bus_name) Acquire mutex with timeout Lookup bus configuration Validate bus initialized Validate pin is output mode Write value to PODR register Release mutex Return success

Register Access:: For port P and pin n: Output high: PORTP.PODR.BIT.Bn = 1 Output low: PORTP.PODR.BIT.Bn = 0

Performance:: Execution time: 2 us (with mutex) Propagation delay: 10-20 ns to pin edge

Example - LED Control:: rx_err_t err=rx_bus_gpio_write(&manager,"led_red",false); if(err!=k_rx_ok){ rx_log_error("GPIO","LEDwritefailed"); }

rx_bus_gpio_write(&manager,"led_red",true);

Example - Motor Driver Enable:: err=rx_bus_gpio_write(&manager,"motor_enable",true); tx_thread_sleep(k_motor_wake_ticks);

rx_bus_gpio_write(&manager,"motor_enable",false);

Example:: rx_bus_gpio_write(&manager,"led_red",false);

See also: `rx_bus_gpio_init()` Initialize GPIO as output first, `rx_bus_gpio_read()` Read current output state (via PIDR), `rx_bus_gpio_toggle()` Toggle output state, `rx_bus_gpio.h` Complete API documentation, `internal_gpio_write_callback()` Implementation callback

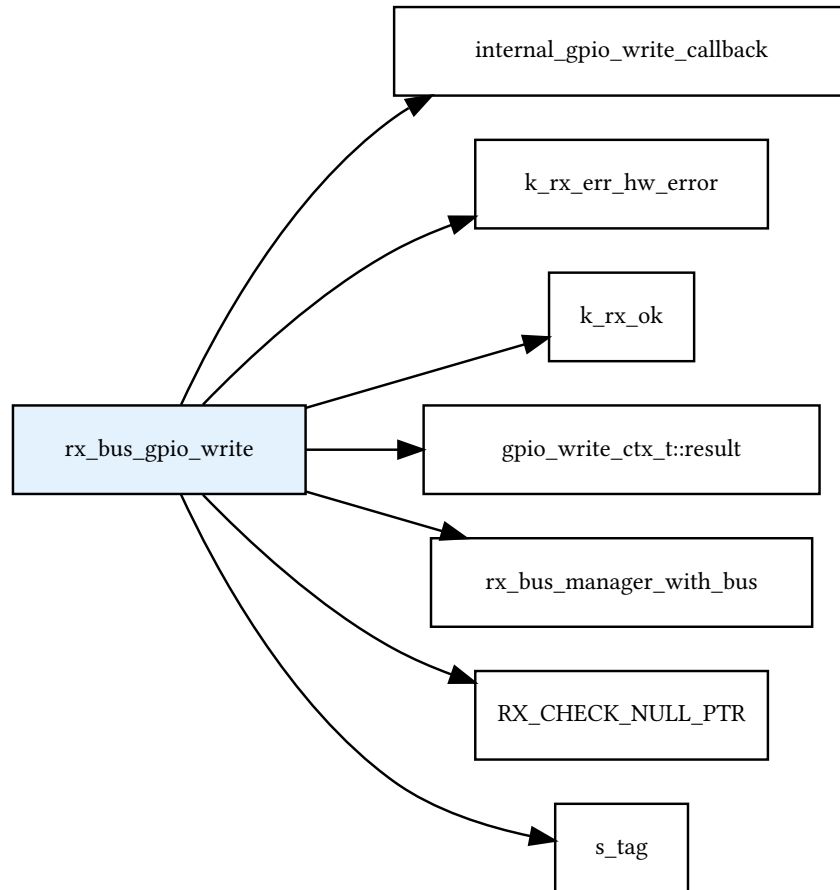


Figure 11: Call graph for `rx_bus_gpio_write`

Defined in `libs/rx\_bus/inc/rx\_bus\_gpio.h:544`

Since Version 1.0.0

—

rx_bus_gpio_read

```
rx_err_t rx_bus_gpio_read(rx_bus_manager_t *manager, const char *bus_name, bool
*value)
```

Read GPIO input through bus manager.

Reads the current state of a GPIO pin. Can be used for both input and output pins (reading output pins returns the actual pin state via PIDR, useful for verifying output or detecting external conflicts).

PIDR always reflects actual pin state:

- Input mode: Reads external signal
- Output mode: Reads back written value (or external override)

Bus state remains initialized

Read GPIO input through bus manager.

Public API function to read GPIO pin state. Works for both input and output pins (reads actual pin state via PIDR register). Delegates to bus manager which calls `internal_gpio_read_callback()` with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, value)
2. Create stack-allocated context with value pointer
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context (value already updated by callback)
5. Return result to caller

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized
in	bus_name	GPIO bus name Must match registered bus
out	value	Pin state true: Pin is high ($> 0.7 \cdot VDD$) false: Pin is low ($< 0.3 \cdot VDD$) Valid only if <code>k_rx_ok</code> returned
in	manager	Bus manager instance Must be initialized Must have GPIO bus registered and initialized
in	bus_name	Name of the GPIO bus Must match registered and initialized bus
out	value	Pointer to store pin state true: Pin is high ($> 0.7 \cdot VDD$) false: Pin is low ($< 0.3 \cdot VDD$) Valid only if <code>k_rx_ok</code> returned

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, value contains pin state
<code>k_rx_err_null_ptr</code>	nullptr in any parameter
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized
<code>k_rx_err_timeout</code>	Mutex acquisition timeout
<code>k_rx_ok</code>	Success, value contains pin state
<code>k_rx_err_null_ptr</code>	nullptr in any parameter
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized
<code>k_rx_err_timeout</code>	Mutex timeout

Pre: Bus must be initialized via `rx_bus_gpio_init()`

Pre: value pointer must be valid

Pre: Bus must be initialized via `rx_bus_gpio_init()`

Pre: All pointers must be non-NULL

Pre: value must point to valid bool

Post: value contains current pin state

Post: Pin state unchanged (non-destructive read)

Post: *value contains pin state (if `k_rx_ok`)

Post: Pin state unchanged (non-destructive read)

Note: Thread-safe via bus manager mutex

Note: Works for both input and output pins

Note: Fast operation (2 us including mutex)

Note: For inputs, enable internal pull-up if needed (via PORT PCR register)

Note: Thread-safe via bus manager mutex

Note: Works for both input and output pins

Note: Stack usage: 16 bytes for context

Note: Execution time: 2 us

Warning: value undefined if return `!= k_rx_ok`

Warning: Floating inputs may read unpredictably (use pull-up/down)

Warning: value undefined if return != k_rx_ok

Warning: Floating inputs read unpredictably

Algorithm Steps:: Validate all pointers (manager, bus_name, value) Acquire mutex with timeout
Lookup bus configuration Validate bus initialized Read PIDR register (port input data) Set *value =
pin state Release mutex Return success

Register Access:: For port P and pin n: Read: value = PORTP.PIDR.BIT.Bn

Input Voltage Thresholds:: Voltage Interpretation

< 0.3*VDD Logic low (false)

> 0.7*VDD Logic high (true)

0.3-0.7*VDD Undefined (avoid)

Performance:: Execution time: 2 us (with mutex) Sampling delay: < 1 us from pin change to
software read

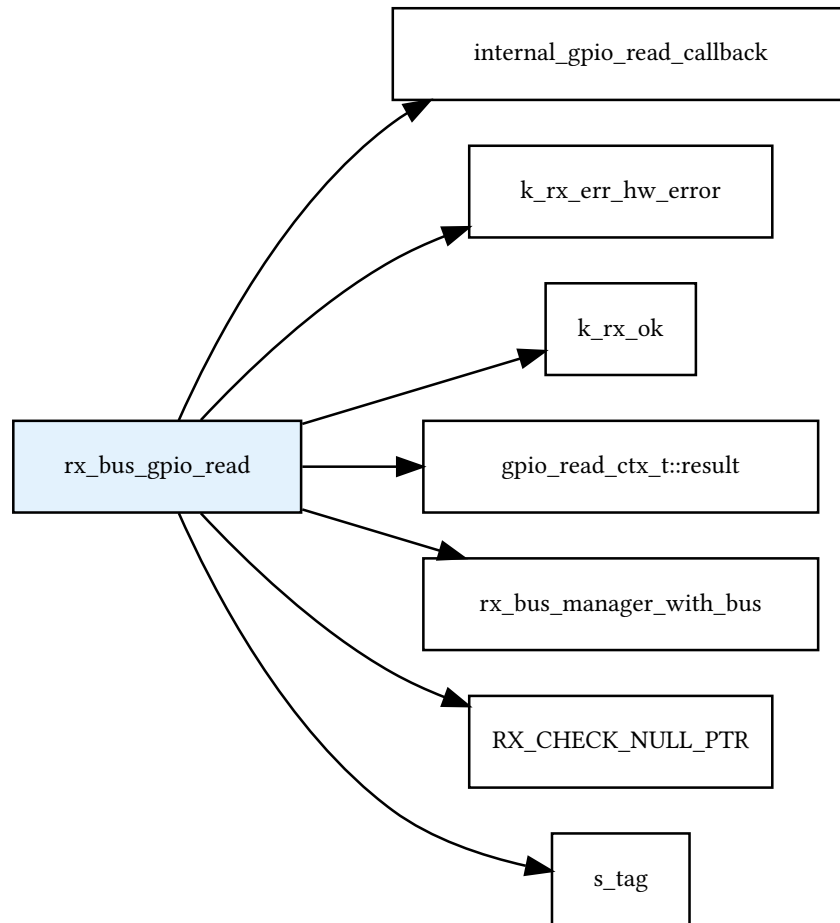
Example - Button Reading:: boolbutton_state=false;
rx_err_t err=rx_bus_gpio_read(&manager,"button_user",&button_state); if(err==k_rx_ok){ if(!
button_state){ handle_button_press(); } }

Example - Sensor Detect:: boolsensor_present=false;
err=rx_bus_gpio_read(&manager,"sensor_detect",&sensor_present);
if(err==k_rx_ok&&sensor_present){ init_sensor(); }

Example - Output Verification:: rx_bus_gpio_write(&manager,"motor_enable",true);
boolactual_state=false; rx_bus_gpio_read(&manager,"motor_enable",&actual_state); if(!actual_state)
{ rx_log_error("GPIO","Motorenablesshorted!"); }

Example:: boolbutton_state; rx_err_t err=rx_bus_gpio_read(&manager,"button",&button_state);
if(err==k_rx_ok&&!button_state){ }

See also: rx_bus_gpio_init() Initialize GPIO first, rx_bus_gpio_write() Write output
state, rx_bus_gpio.h Complete API documentation, internal_gpio_read_callback()
Implementation callback

Figure 12: Call graph for `rx_bus_gpio_read`

_Defined in `libs/rx\_bus/inc/rx\_bus\_gpio.h:656_`

Since Version 1.0.0

—

rx_bus_gpio_toggle

```
rx_err_t rx_bus_gpio_toggle(rx_bus_manager_t *manager, const char *bus_name)
```

Toggle GPIO output through bus manager.

Inverts the current state of a GPIO output pin (high -> low, low -> high). Performs an atomic read-modify-write operation protected by mutex. Useful for LED blinking and signal generation.

Bus state remains initialized

Toggle GPIO output through bus manager.

Public API function to invert GPIO output state. Performs atomic read-modify-write operation. Delegates to bus manager which calls `internal_gpio_toggle_callback()` with mutex protection.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context (no parameters needed)
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context
5. Return result to caller

Atomic operation: mutex ensures no other thread can modify pin between read and write operations.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized
in	bus_name	GPIO bus name Must match registered bus
in	manager	Bus manager instance Must be initialized Must have GPIO bus registered and initialized
in	bus_name	Name of the GPIO bus Must match registered and initialized bus Must be configured as output

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, pin state toggled
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized or pin is input
<code>k_rx_err_timeout</code>	Mutex acquisition timeout
<code>k_rx_ok</code>	Success, pin state inverted
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_state</code>	Bus not initialized or pin is input
<code>k_rx_err_timeout</code>	Mutex timeout

Pre: Bus must be initialized via `rx_bus_gpio_init()` with output = true

Pre: Pin must be configured as output

Pre: Bus must be initialized via `rx_bus_gpio_init()` with output=true

Pre: manager and bus_name must be non-NULL

Pre: Pin must be configured as output

Post: Pin output state inverted (high <-> low)

Post: Physical pin reflects new state within 10-20 ns

Post: Pin output state inverted (high -> low or low -> high)

Post: Physical pin reflects new state within 10-20 ns

Note: Thread-safe - atomic read-modify-write via mutex

Note: Slightly slower than write (3 us vs 2 us) due to read

Note: Useful for periodic signals (LED blink, clock generation)

Note: Thread-safe - atomic read-modify-write via mutex

Note: Stack usage: 4 bytes for context

Note: Execution time: 3 us (read + write)

Warning: Attempting to toggle input pin returns `k_rx_err_invalid_state`

Warning: High-frequency toggling (>100 kHz) should use direct register access

Warning: Attempting to toggle input pin returns `k_rx_err_invalid_state`

Algorithm Steps:: Validate pointers (manager, bus_name) Acquire mutex with timeout Lookup bus configuration Validate bus initialized and pin is output Read current state from PIDR Write inverted state to PODR Release mutex Return success

Atomic Read-Modify-Write:: The toggle operation is atomic (no race conditions):
`current=PORTP.PIDR.BIT.Bn; PORTP.PODR.BIT.Bn=!current;` Mutex ensures no other thread can modify pin between read and write.

Performance:: Execution time: 3 us (read + write with mutex) Maximum toggle rate: 330 kHz (not recommended - use PWM instead) Practical toggle rate: < 10 kHz for bus manager API

Example - LED Blinking:: `while(1){ rx_bus_gpio_toggle(&manager,"led_status");
tx_thread_sleep(50); }`

Example - Heartbeat LED:: void heartbeat_task_entry(ULONG arg)
 { rx_bus_manager_t* mgr=(rx_bus_manager_t*)arg; while(1)
 { rx_bus_gpio_toggle(mgr,"led_heartbeat"); tx_thread_sleep(100); } }

Example - Test Signal Generation:: for(uint32_t i=0; i<1000; i++)
 { rx_bus_gpio_toggle(&manager,"test_signal"); tx_thread_sleep(1); }

Performance:: Execution time: 3 us (slower than write due to read) Maximum toggle rate: 330 kHz
 (theoretical) Practical toggle rate: < 10 kHz (recommended)

Example - LED Blinking:: while(1){ rx_bus_gpio_toggle(&manager,"led_status");
 tx_thread_sleep(50); }

See also: rx_bus_gpio_init() Initialize GPIO as output first, rx_bus_gpio_write() Set
 explicit state (faster if known), rx_bus_gpio_read() Read current state before
 deciding, rx_bus_gpio.h Complete API documentation, internal_gpio_toggle_callback()
 Implementation callback, rx_bus_gpio_write() Use this if new state is known (faster)

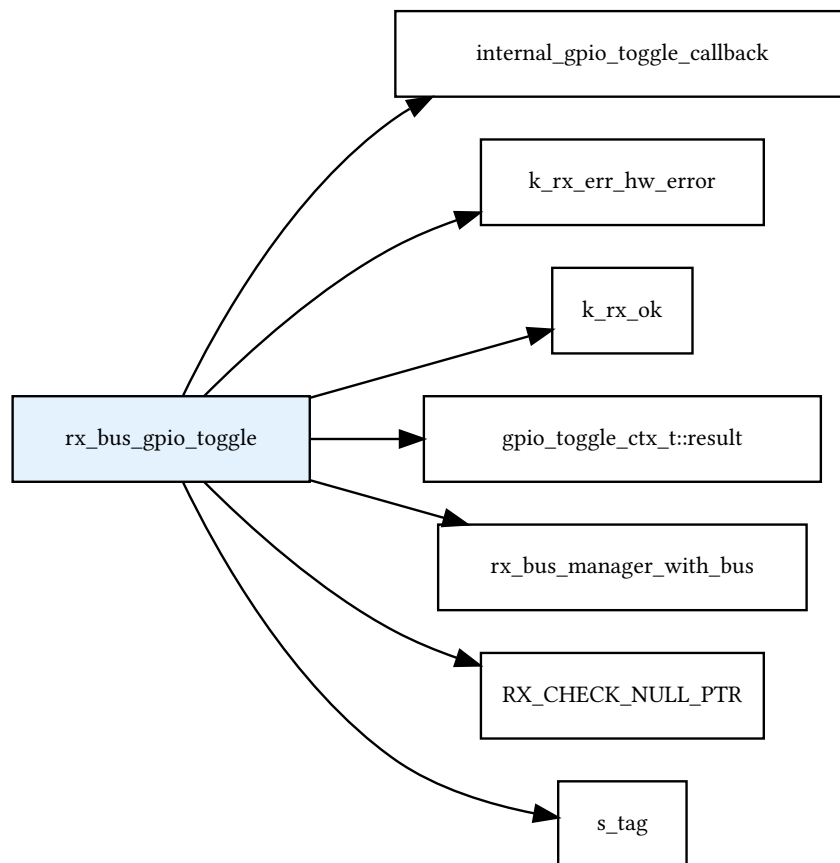


Figure 13: Call graph for rx_bus_gpio_toggle

_Defined in libs/rx_bus/inc/rx_bus_gpio.h:753_

Since Version 1.0.0

rx_bus_i2c.h

I2C (Inter-Integrated Circuit) Bus Abstraction for RX72N.

Provides bus manager integration for Renesas RIIC (I2C) peripheral operations. Wraps the low-level RIIC HAL with the bus abstraction pattern for consistent API across all bus types (I2C, SPI, 1-Wire, UART) with mutex-protected access.

1.0.0

2026-01-01

Copyright (c) 2026 STAR Project

Includes:

- <stdint.h>
- "rx_bus_manager.h"
- "rx_err.h"

rx_bus_i2c_init

```
rx_err_t rx_bus_i2c_init(rx_bus_manager_t *manager, const char *bus_name)
```

Initialize I2C bus through bus manager.

Configures the RIIC peripheral channel and associated GPIO pins for I2C communication. This function must be called before any read/write operations. The initialization process:

1. Acquires bus manager mutex (thread-safe)
2. Looks up bus configuration by name
3. Validates bus type is I2C
4. Configures MPC for SCL/SDA pin functions
5. Initializes RIIC peripheral with configured speed
6. Releases mutex

Initialize I2C bus through bus manager.

Initializes the RX72N RIIC peripheral for the specified I2C bus. Configures clock frequency, enables peripheral, and marks bus as ready for transactions. Must be called once before any read/write operations.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via rx_bus_manager_init() Must have I2C bus registered via rx_bus_register()
in	bus_name	I2C bus name Null-terminated string Must match name used in rx_bus_register() Maximum 31 characters

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, bus initialized and ready

k_rx_err_null_ptr	nullptr in manager or bus_name
k_rx_err_not_found	Bus name not found in manager
k_rx_err_invalid_arg	Bus is not I2C type
k_rx_err_timeout	Mutex acquisition timeout (default 1000ms)
k_rx_err_invalid_state	Bus already initialized

Pre: manager must be initialized via rx_bus_manager_init()

Pre: Bus must be registered via rx_bus_register() with type k_rx_bus_type_i2c

Post: RIIC channel configured and ready for transactions

Post: SCL/SDA pins configured as open-drain with internal pullups disabled

Post: Bus state set to initialized

Note: Thread-safe via bus manager mutex

Note: Call only once per bus; subsequent calls return k_rx_err_invalid_state

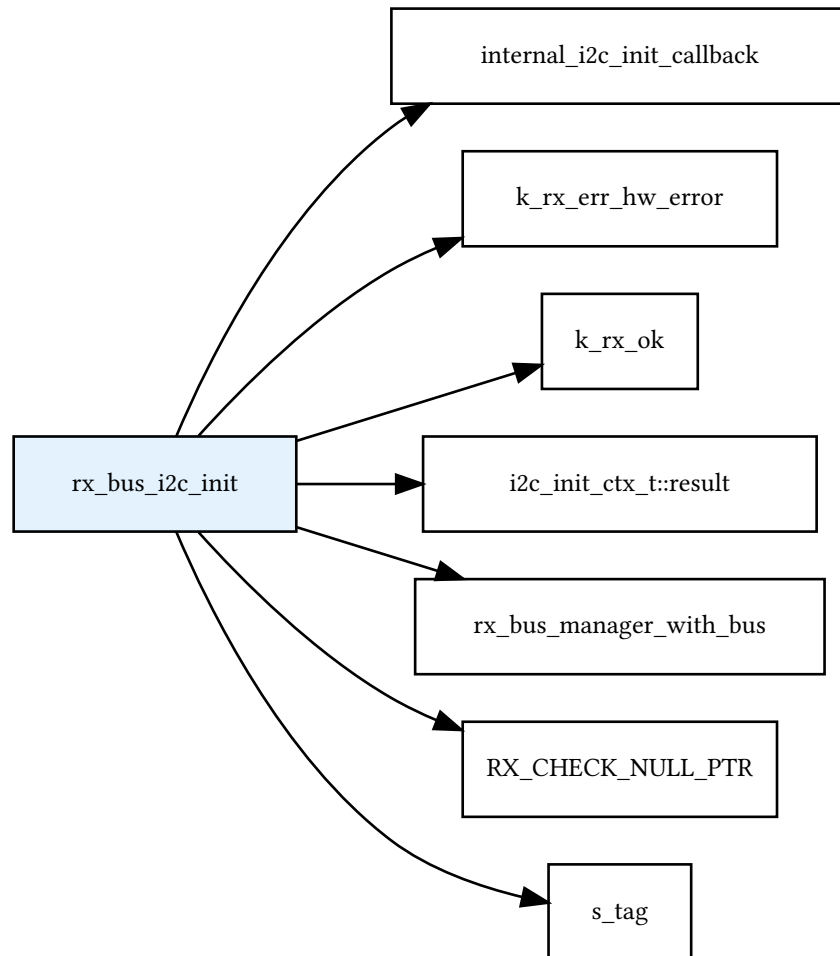
Warning: External pullup resistors (2.2-10 kOhm) required on SCL/SDA

Warning: Do not call from interrupt context (mutex wait)] **Initialization Sequence:** digraph
init_flow { rankdir=TB; node [shape=box, style=rounded]; start [label="rx_bus_i2c_init()",
shape=ellipse]; mutex [label="Acquire mutex"]; lookup [label="Lookup bus by name"]; validate
[label="Validate bus type"]; mpc [label="Configure MPC pins"]; riic [label="Initialize RIIC"];
release [label="Release mutex"]; done [label="Return", shape=ellipse];

```
start -> mutex; mutex -> lookup; lookup -> validate; validate -> mpc; mpc -> riic; riic -> release;
release -> done; }
```

Performance:: Execution time: 100 us typical (RIIC initialization) Mutex timeout: 1000 ms
(configurable via bus manager)

See also: rx_bus_manager_init() Initialize bus manager first, rx_bus_register() Register
bus before initialization, rx_bus_i2c_write() Write data after initialization

Figure 14: Call graph for `rx_bus_i2c_init`

_Defined in `libs/rx_bus/inc/rx_bus_i2c.h:342_`

Since Version 1.0.0

—

rx_bus_i2c_write

```
rx_err_t rx_bus_i2c_write(rx_bus_manager_t *manager, const char *bus_name, const
uint8_t *data, uint16_t length)
```

Write data to I2C device through bus manager.

Transmits data bytes to the I2C peripheral device configured for this bus. The device address is taken from the bus configuration. This function handles the complete I2C write transaction:

1. Acquire bus manager mutex
2. Generate START condition

3. Send device address with W bit (0)
4. Wait for ACK from peripheral
5. Send each data byte, waiting for ACK
6. Generate STOP condition
7. Release mutex

Write data to I2C device through bus manager.

Transmits N bytes to the I2C device specified in bus configuration. Typically used to send commands, write configuration registers, or transfer data to peripherals. Generates START, device address (write), N data bytes, STOP.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via rx_bus_manager_init() Must not be nullptr
in	bus_name	I2C bus name Null-terminated string matching registered bus Must not be nullptr
in	data	Pointer to data buffer to write Must not be nullptr Must remain valid for duration of call
in	length	Number of bytes to write Range: 1-65535 bytes 0 is valid (address-only probe)

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, all bytes transmitted and ACKed
k_rx_err_null_ptr	nullptr in manager, bus_name, or data
k_rx_err_not_found	Bus name not found in manager
k_rx_err_invalid_state	Bus not initialized via rx_bus_i2c_init()
k_rx_err_timeout	I2C transaction timeout or mutex timeout
k_rx_err_nack	Device NACK received (address or data)
k_rx_err_busy	Bus arbitration lost

Pre: Bus must be initialized via rx_bus_i2c_init()

Pre: data buffer must be valid for duration of call

Post: All bytes transmitted to device on success

Post: Bus returned to idle state (STOP sent)

Note: Thread-safe via bus manager mutex

Note: Blocks until transaction complete or timeout

Warning: Do not call from interrupt context

Warning: NACK on address byte indicates device not responding] **I2C Write Transaction:** *

_____ * SDA _____/ || || || || _____/ * | Address + W |

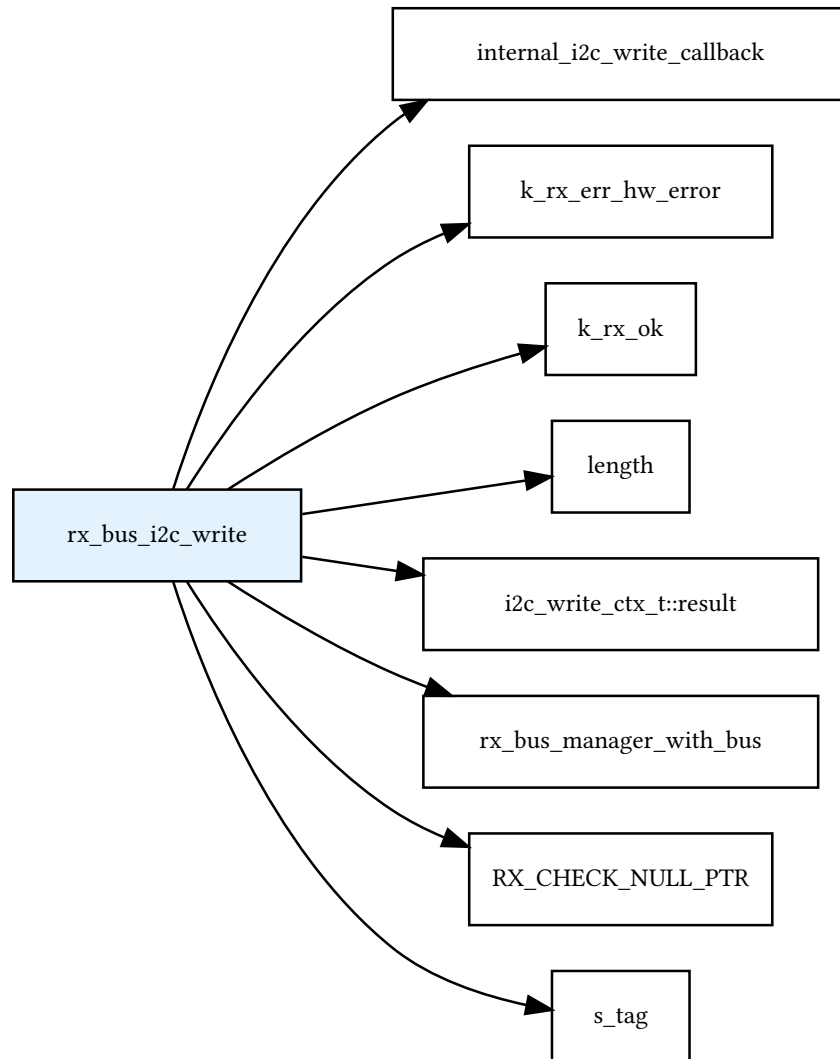
ACK | Data | ACK | * START | <-- 8 bits --> | <-- 8 bits --> | STOP * _____ *

SCL _/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_*

Performance (100 kHz standard mode):: 1 byte: 100 us 8 bytes: 800 us Overhead: 20 us (START/STOP/address)

```
Example:: uint8_t write_buf[3]={0x00,0x12,0x34};
rx_err_t err=rx_bus_i2c_write(&manager,"eeprom",write_buf,3); if(err==k_rx_err_ack){}
```

See also: `rx_bus_i2c_init()` Initialize bus first, `rx_bus_i2c_read()` Read data from device, `rx_bus_i2c write_read()` Combined write-read for register access

Figure 15: Call graph for `rx_bus_i2c_write`

_Defined in `libs/rx_bus/inc/rx_bus_i2c.h:425_`

Since Version 1.0.0

—

rx_bus_i2c_read

```
rx_err_t rx_bus_i2c_read(rx_bus_manager_t *manager, const char *bus_name, uint8_t
*data, uint16_t length)
```

Read data from I2C device through bus manager.

Receives data bytes from the I2C peripheral device configured for this bus. The device address is taken from the bus configuration. This function handles the complete I2C read transaction:

1. Acquire bus manager mutex
2. Generate START condition
3. Send device address with R bit (1)
4. Wait for ACK from peripheral
5. Receive each data byte, sending ACK (except last byte)
6. Send NACK on last byte to signal end
7. Generate STOP condition
8. Release mutex

Read data from I2C device through bus manager.

Receives N bytes from the I2C device specified in bus configuration. Generates START, device address (read), receives N data bytes with ACKs, sends final NACK, STOP. Used for reading sensor data, status registers, or bulk data from device memory.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via rx_bus_manager_init() Must not be nullptr
in	bus_name	I2C bus name Null-terminated string matching registered bus Must not be nullptr
out	data	Pointer to buffer for received data Must not be nullptr Must have capacity for 'length' bytes
in	length	Number of bytes to read Range: 1-65535 bytes 0 returns immediately with k_rx_ok

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, all bytes received
k_rx_err_null_ptr	nullptr in manager, bus_name, or data
k_rx_err_not_found	Bus name not found in manager
k_rx_err_invalid_state	Bus not initialized via rx_bus_i2c_init()
k_rx_err_timeout	I2C transaction timeout or mutex timeout
k_rx_err_nack	Device NACK on address (device not responding)
k_rx_err_busy	Bus arbitration lost

Pre: Bus must be initialized via rx_bus_i2c_init()

Pre: data buffer must have capacity for 'length' bytes

Post: data buffer contains received bytes on success

Post: Bus returned to idle state (STOP sent)

Note: Thread-safe via bus manager mutex

Note: Blocks until transaction complete or timeout

Note: For register reads, use rx_bus_i2c_write_read() instead

Warning: Do not call from interrupt context

Warning: Raw read without prior register address write may return unexpected data] **I2C Read**

Transaction: * _____ * SDA _____ / | | | | | | | / _____ *
 | Address + R | ACK | Data | NACK | * START | <--- 8 bits --> | <--- 8 bits --> | STOP * | Controller | Per
 | Peripheral | Ctrl | * _____ * SCL _ / _ / _ / _ / _ / _ / _ / _ / _ / *

Performance (100 kHz standard mode):: 1 byte: 100 us 8 bytes: 800 us Overhead: 20 us (START/STOP/address)

See also: rx_bus_i2c_init() Initialize bus first, rx_bus_i2c_write() Write data to device, rx_bus_i2c_write_read() Preferred for register reads

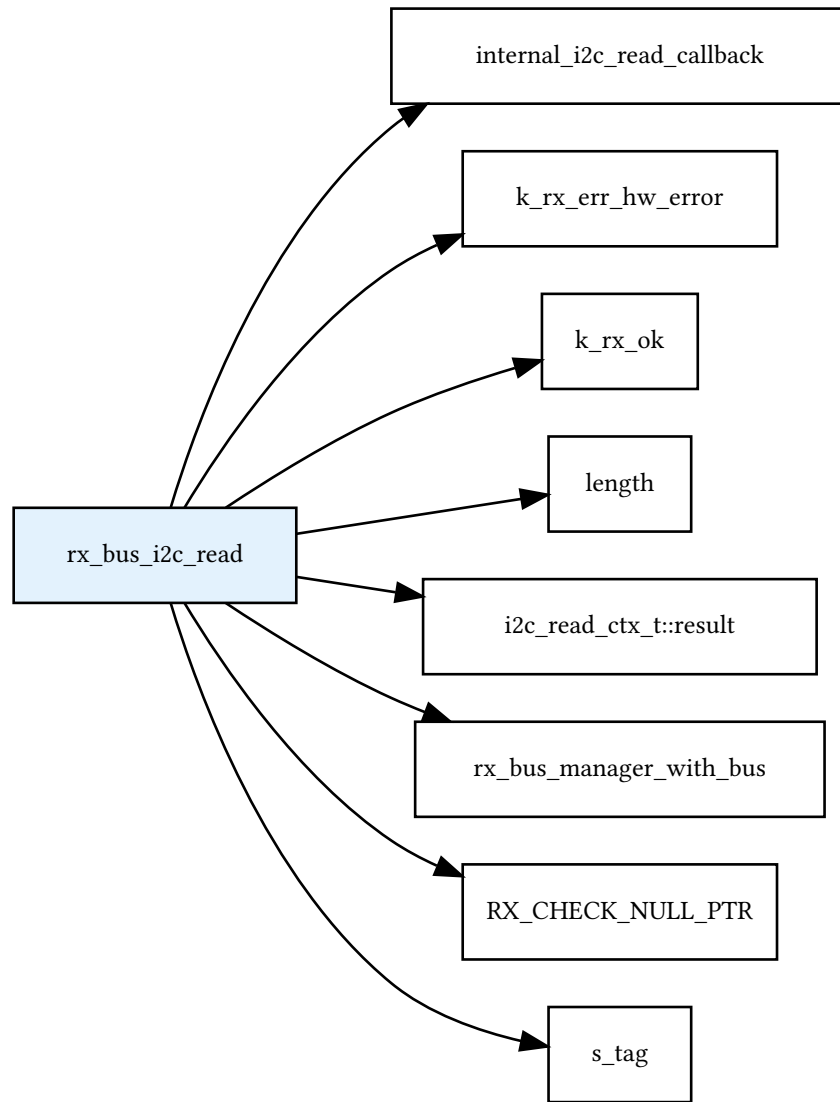


Figure 16: Call graph for rx_bus_i2c_read

_Defined in `libs/rx\_bus/inc/rx\_bus\_i2c.h:505_`

Since Version 1.0.0

—

rx_bus_i2c_write_read

```

rx_err_t rx_bus_i2c_write_read(rx_bus_manager_t *manager, const char *bus_name,
const uint8_t *write_data, uint16_t write_length, uint8_t *read_data, uint16_t
read_length)
  
```

Write then read from I2C device through bus manager.

Performs a combined write-read transaction, which is the standard pattern for reading device registers. The write phase sends the register address, then a repeated START initiates the read phase without releasing the bus.

Transaction sequence:

1. Acquire bus manager mutex
2. Generate START condition
3. Send device address with W bit (0)
4. Send write_data bytes (typically register address)
5. Generate REPEATED START (no STOP)
6. Send device address with R bit (1)
7. Receive read_data bytes
8. Generate STOP condition
9. Release mutex

Bus remains locked for entire write+read sequence

Write then read from I2C device through bus manager.

Combined write-read operation for register access. Writes N bytes (typically register address), sends repeated START, then reads M bytes (register contents). This is the MOST COMMON I2C operation for sensors and peripherals.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via rx_bus_manager_init() Must not be nullptr
in	bus_name	I2C bus name Null-terminated string matching registered bus Must not be nullptr
in	write_data	Pointer to data to write (register address) Must not be nullptr Typically 1-2 bytes for register address
in	write_length	Number of bytes to write Range: 1-65535 bytes (typically 1-2)
out	read_data	Pointer to buffer for received data Must not be nullptr Must have capacity for 'read_length' bytes
in	read_length	Number of bytes to read Range: 1-65535 bytes

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, transaction completed
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found in manager
k_rx_err_invalid_state	Bus not initialized via rx_bus_i2c_init()
k_rx_err_timeout	I2C transaction timeout or mutex timeout
k_rx_err_nack	Device NACK on address or write data

k_rx_err_busy	Bus arbitration lost
---------------	----------------------

Pre: Bus must be initialized via rx_bus_i2c_init()

Pre: write_data buffer must be valid for duration of call

Pre: read_data buffer must have capacity for 'read_length' bytes

Post: read_data buffer contains received register data on success

Post: Bus returned to idle state (STOP sent)

Note: Thread-safe via bus manager mutex

Note: Uses REPEATED START to maintain bus ownership

Note: Preferred method for register reads (atomic operation)

Warning: Do not call from interrupt context

Warning: Some devices require delays between write and read phases] **Combined Write-Read Transaction:** * _____ * SDA __/ Addr+W | Data |__/
Addr+R | Data __/ * S |Wr Reg| Sr |Rd Reg| P * ||| * _____ *
SCL _/ - - - - - _/ - - - - - _/ * S = START condition * Sr = REPEATED START (no STOP
between) * P = STOP condition *

Common Use Cases: Use Case Write Data Read Length Example

Read 8-bit register 1 byte (reg addr) 1 byte Temperature

Read 16-bit register 1 byte (reg addr) 2 bytes Voltage

Read block 1 byte (reg addr) N bytes EEPROM page

Performance (100 kHz standard mode):: 1 write + 1 read: 200 us 1 write + 8 read: 900 us

Overhead: 40 us (START/RESTART/STOP/addresses)

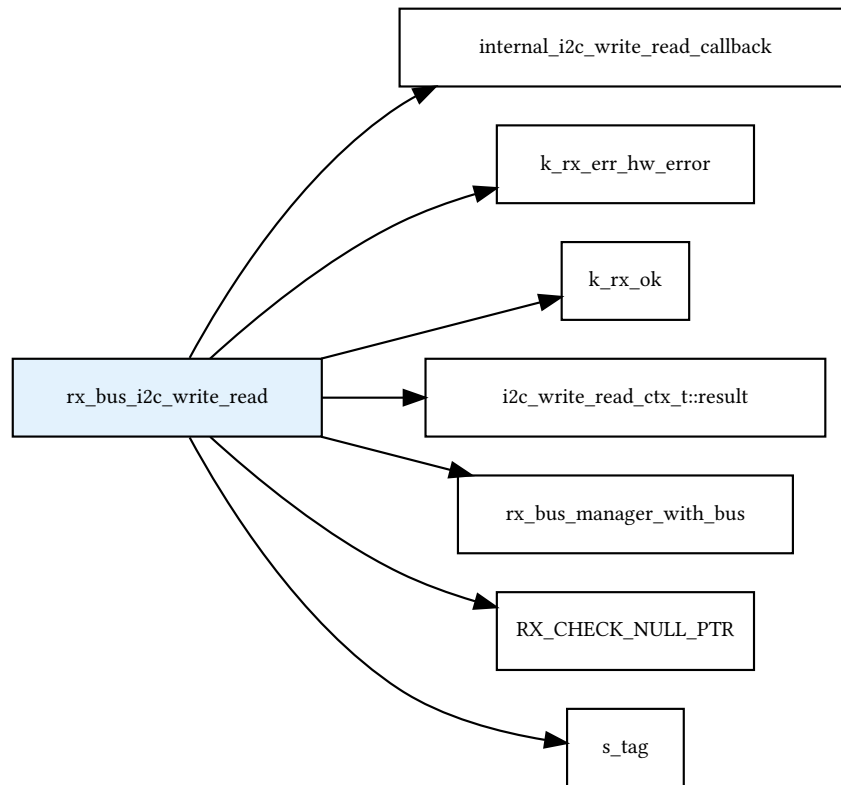
Example - Reading IMU Register:: uint8_t reg_addr=0x3B; uint8_t accel_raw[2];

```
rx_err_t err=rx_bus_i2c_write_read(&manager,"imu_i2c", &reg_addr,1, accel_raw,2); if(err==k_rx_ok)
{ int16_t accel_x=(int16_t)((accel_raw[0]<<8)|accel_raw[1]); }
```

Example - Reading EEPROM Block:: uint8_t eeprom_addr[2]={0x01,0x00}; uint8_t data[16];

```
rx_err_t err=rx_bus_i2c_write_read(&manager,"eeprom", eeprom_addr,2, data,16);
```

See also: rx_bus_i2c_init() Initialize bus first, rx_bus_i2c_write() Write-only operations, rx_bus_i2c_read() Read-only operations

Figure 17: Call graph for `rx_bus_i2c_write_read`

_Defined in `libs/rx_bus/inc/rx_bus_i2c.h:625_`

Since Version 1.0.0

—

rx_bus_manager.h

Unified Bus Manager API for RX72N Multi-Protocol Communication.

Includes:

- `"rx_bus_command.h"`
- `"rx_bus_types.h"`
- `"rx_err.h"`

rx_bus_callback_t

`typedef rx_err_t(* rx_bus_callback_t) (rx_bus_config_t *bus_config, void *user_ctx)`

Callback function type for `rx_bus_manager_with_bus()`.

User-provided callback executed while bus is locked (mutex held). Allows safe access to bus configuration and hardware operations.

—

rx_bus_manager_init

```
rx_err_t rx_bus_manager_init(rx_bus_manager_t *manager, const char *tag,
rx_error_interface_t *error_iface, rx_pin_interface_t *pin_iface)
```

Initialize bus manager with injected dependencies.

Creates and initializes a bus manager instance with ThreadX mutex for thread-safe operations. Follows Dependency Inversion Principle by accepting optional error handler and pin validator interfaces.

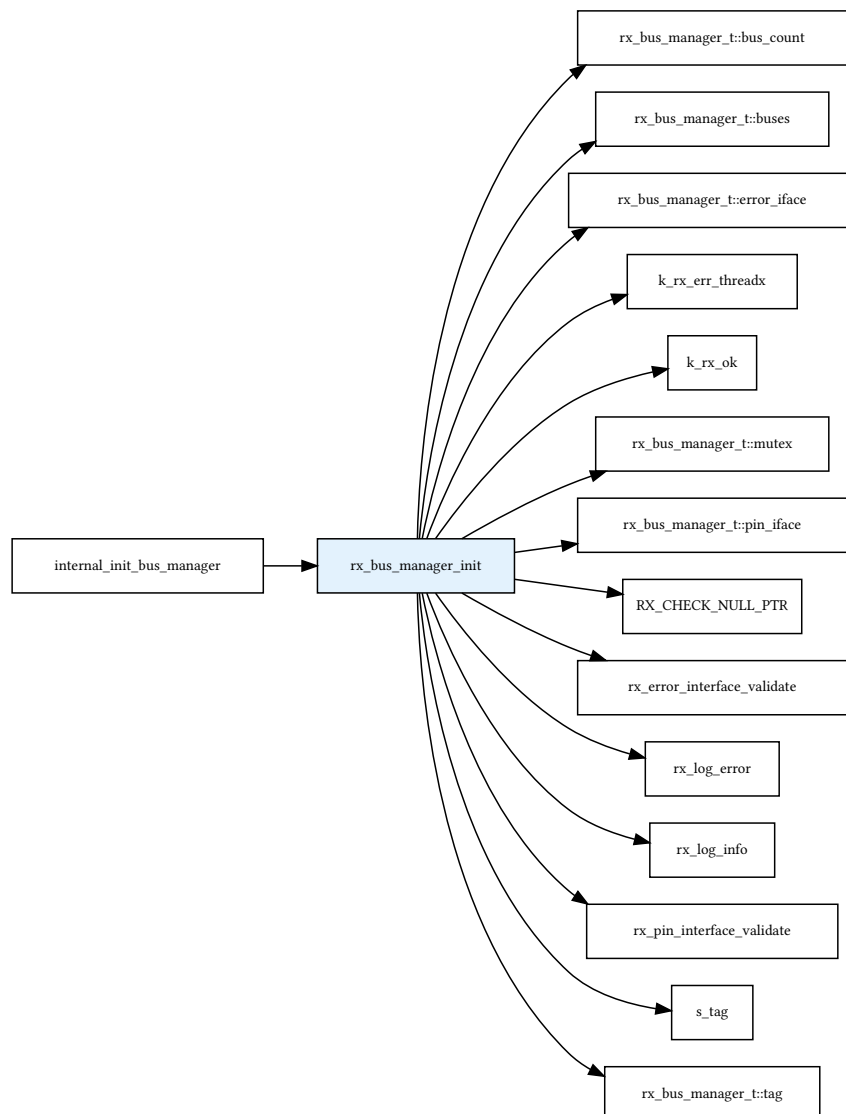


Figure 18: Call graph for rx_bus_manager_init

_Defined in libs/rx_bus/inc/rx_bus_manager.h:285_

—

rx_bus_manager_deinit

```
rx_err_t rx_bus_manager_deinit(rx_bus_manager_t *manager)
```

Deinitialize bus manager and release all resources.

Performs complete cleanup of bus manager:

1. Removes and deinitializes all registered buses
2. Releases references to all bus configurations
3. Deletes ThreadX mutex
4. Resets all tracking state to initial values

Performs complete cleanup of bus manager by removing all registered buses, deleting the ThreadX mutex, and resetting all state. Safe to call on partially initialized managers (cleans up what exists).

Parameters:

Direction	Name	Description
inout	manager	Bus manager instance to deinitialize

Returns: rx_err_t Deinitialization status

Value	Description
k_rx_ok	Manager deinitialized successfully
k_rx_err_null_ptr	manager parameter is nullptr

Pre: manager was initialized via rx_bus_manager_init()

Pre: No other threads are using this manager

Post: All buses removed and hardware deinitialized

Post: Mutex deleted

Post: Manager not safe to use until reinitialized

Note: Safe to call on partially initialized manager (cleanup what exists)

Note: Does not free the manager structure itself (caller's responsibility)

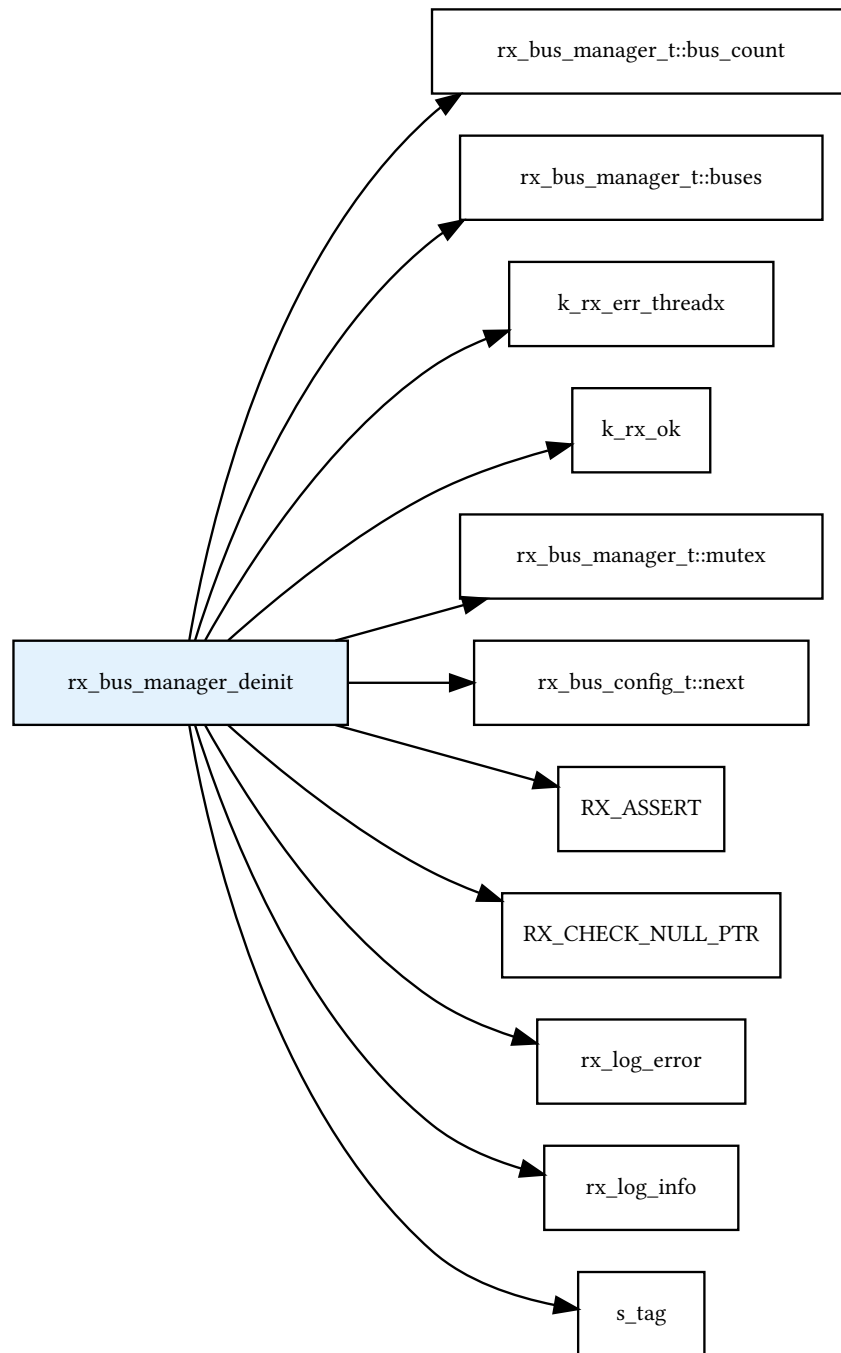
Warning: Call only when no other threads are using manager

Warning: Manager not usable after this call without reinit

Thread Safety:: Not thread-safe. Ensure no other threads access manager during deinit.

Example:: `rx_err_t err = rx_bus_manager_deinit(&manager); if(err != k_rx_ok) { rx_log_warn("MAIN", "Busmanager deinit issue: %d", err); }`

See also: `rx_bus_manager_init()` Initialize manager

Figure 19: Call graph for `rx_bus_manager_deinit`

Defined in `libs/rx_bus/inc/rx_bus_manager.h:336`

Since Version 1.0.0

—

rx_bus_manager_add_bus

```
rx_err_t rx_bus_manager_add_bus(rx_bus_manager_t *manager, rx_bus_config_t
*bus_config)
```

Register a bus configuration with the manager.

Adds a bus to the manager's registry. The manager stores a reference to the bus_config pointer; caller retains ownership and must ensure the configuration remains valid until removal. The bus hardware is NOT initialized until first access (lazy initialization).

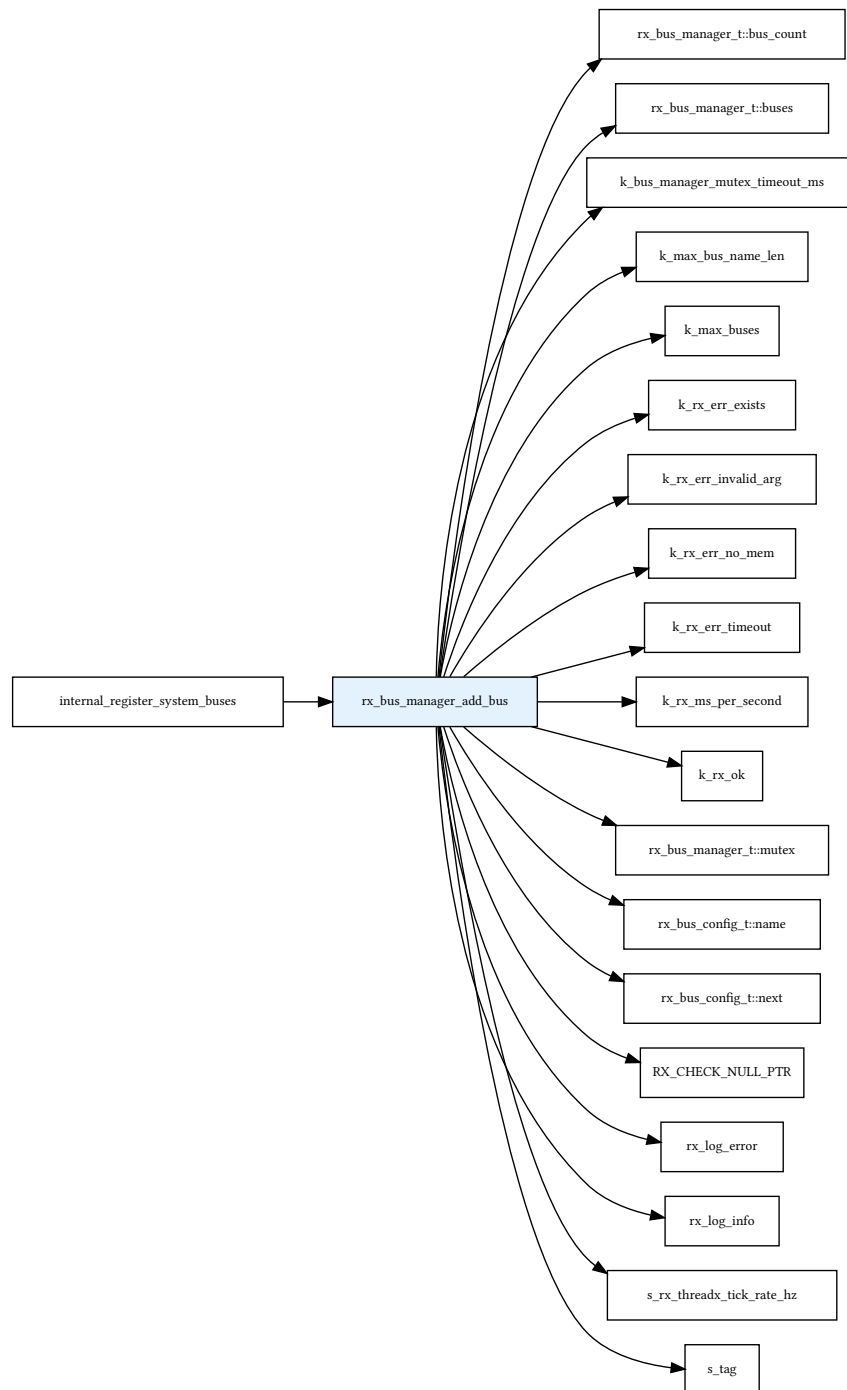


Figure 20: Call graph for rx_bus_manager_add_bus

_Defined in libs/rx_bus/inc/rx_bus_manager.h:418_
—

rx_bus_manager_remove_bus

```
rx_err_t rx_bus_manager_remove_bus(rx_bus_manager_t *manager, const char *name)
```

Unregister and cleanup a bus by name.

Removes a bus from the manager's registry and deinitializes the hardware (if initialized). The configuration reference is released.

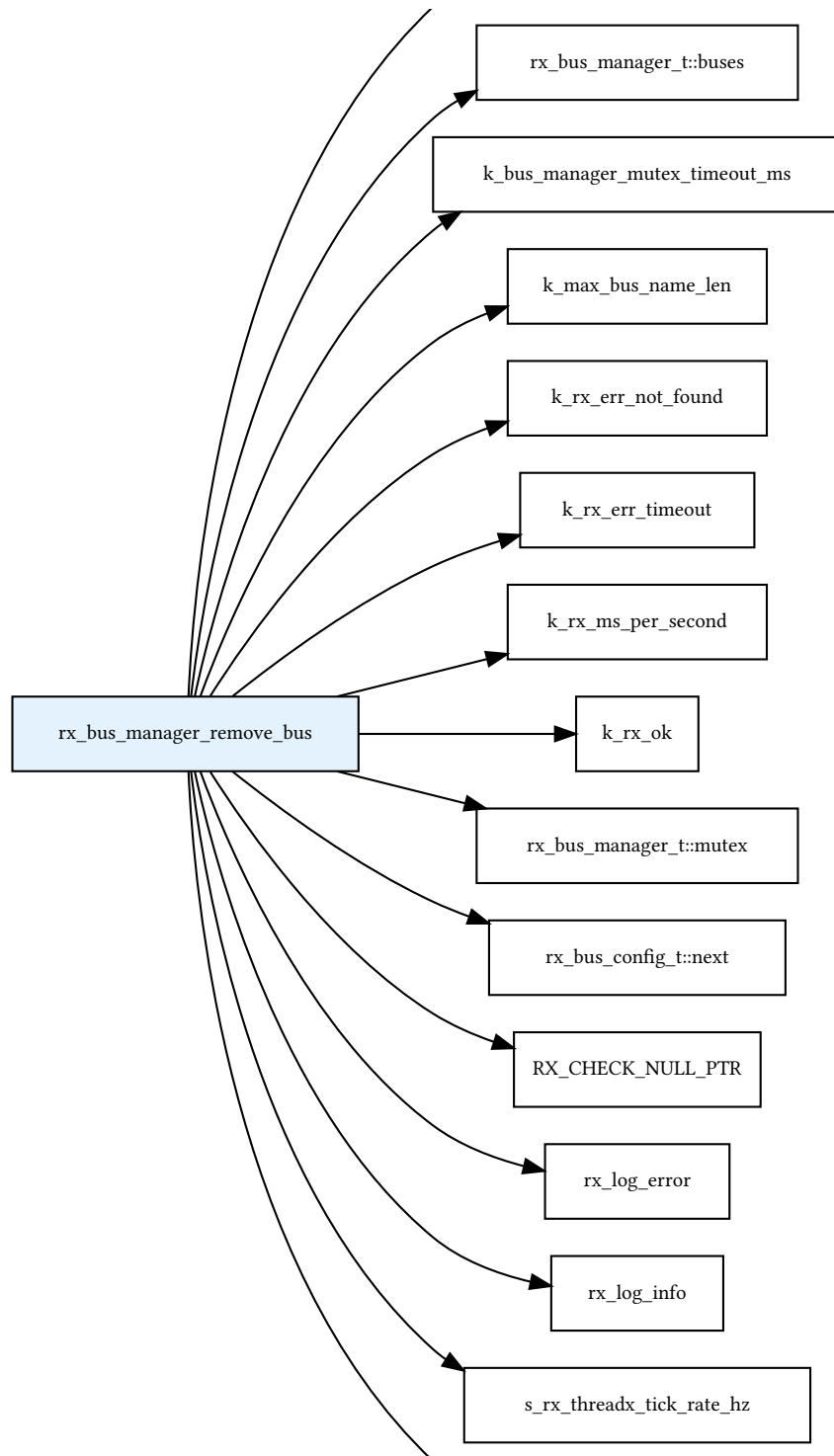


Figure 21: Call graph for `rx_bus_manager_remove_bus`

_Defined in libs/rx_bus/inc/rx_bus_manager.h:478_

—

rx_bus_manager_find_bus

```
rx_err_t rx_bus_manager_find_bus(rx_bus_manager_t *manager, const char *name,
rx_bus_config_t **bus_config)
```

Find a bus by name (thread-safe lookup).

Searches the bus registry for a bus with the given name and returns a pointer to its configuration. The lookup is thread-safe, but the returned pointer is NOT protected after the function returns.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	name	Bus name to search for (case-sensitive)
out	bus_config	Pointer to receive bus configuration address

Returns: rx_err_t Lookup status

Value	Description
k_rx_ok	Bus found, bus_config points to configuration
k_rx_err_null_ptr	manager, name, or bus_config is nullptr
k_rx_err_not_found	No bus with given name exists
k_rx_err_timeout	Mutex timeout acquiring lock

Pre: manager initialized via rx_bus_manager_init()

Post: *bus_config points to found configuration (if k_rx_ok)

Post: *bus_config unchanged (if error)

Note: Lookup is O(n) where n = bus_count

Warning: Race Condition Risk: Prefer rx_bus_manager_with_bus() instead. The returned bus_config pointer could become invalid if another thread calls rx_bus_manager_remove_bus() after this function returns.

Warning: Returned pointer not protected from concurrent removal

Thread Safety:: Lookup is thread-safe, but returned pointer is NOT protected. Use rx_bus_manager_with_bus() for protected access.

Example (Simple, but has race condition risk):: rx_bus_config_t*bus;
rx_err_t err=rx_bus_manager_find_bus(&manager,"imu",&bus); if(err==k_rx_ok)
{ rx_log_info("MAIN","Foundbus:%s,type:%d",bus->name,bus->type); }

See also: rx_bus_manager_with_bus() Recommended thread-safe access pattern

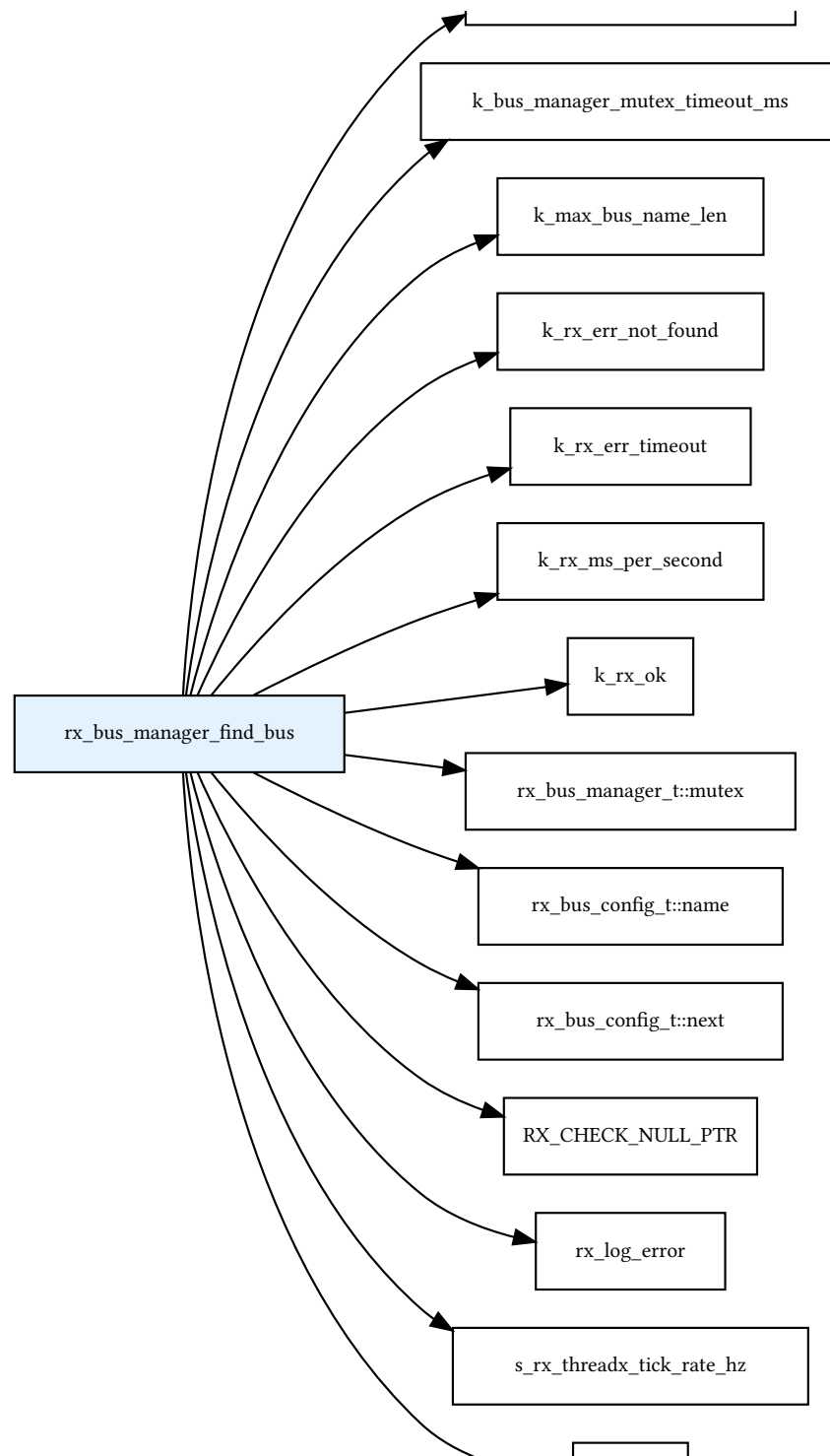


Figure 22: Call graph for `rx_bus_manager_find_bus`

_Defined in `libs/rx\_bus/inc/rx\_bus\_manager.h:535_`

Since Version 1.0.0

—

rx_bus_manager_with_bus

```
rx_err_t rx_bus_manager_with_bus(rx_bus_manager_t *manager, const char *name,  
rx_bus_callback_t callback, void *user_ctx)
```

Execute callback with bus locked (RECOMMENDED access pattern).

The recommended pattern for thread-safe bus access. Holds the manager mutex while the callback executes, preventing bus removal race conditions.

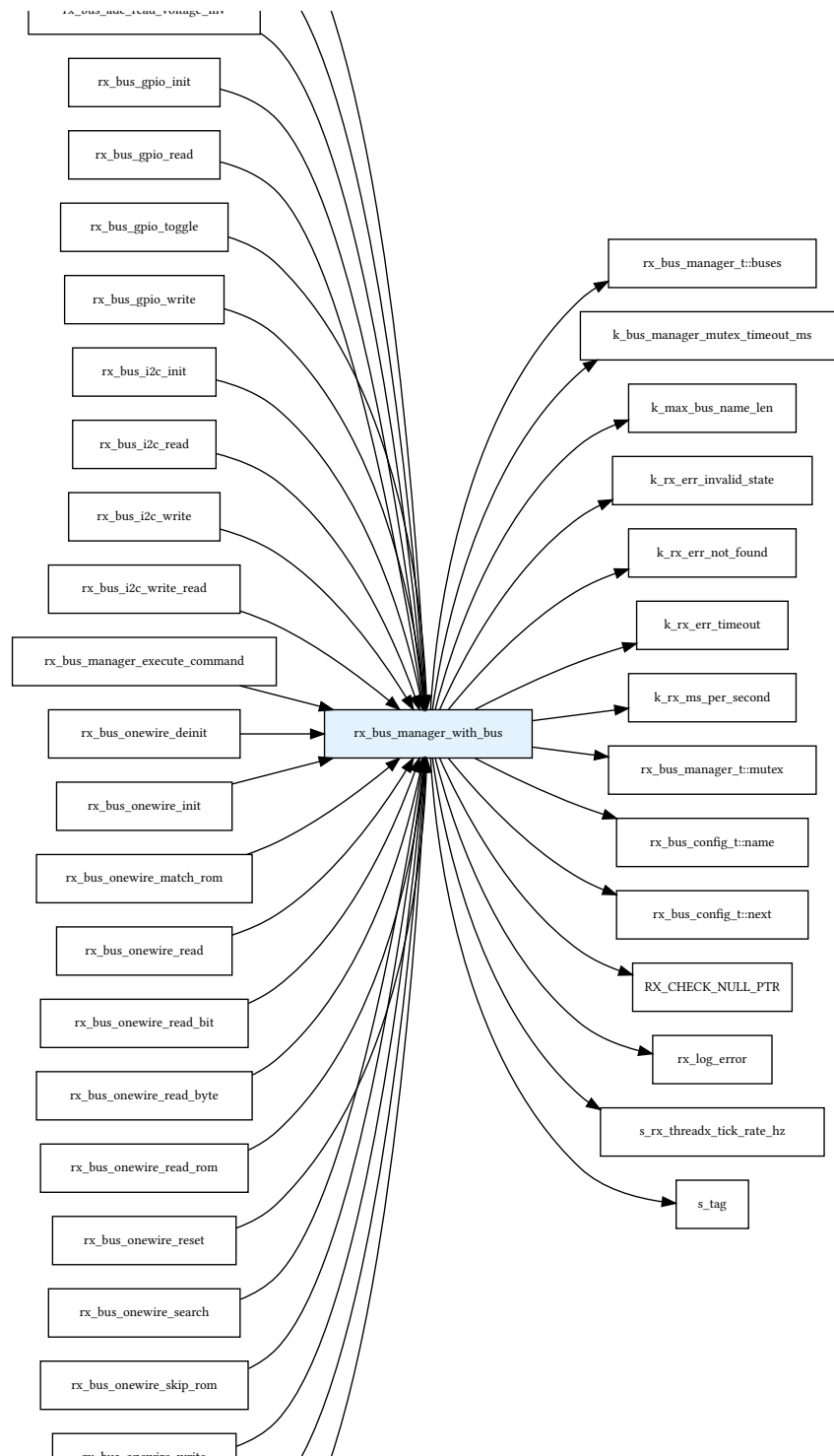


Figure 23: Call graph for rx_bus_manager_with_bus

_Defined in libs/rx_bus/inc/rx_bus_manager.h:681_

—

rx_bus_manager_execute_command

```
rx_err_t rx_bus_manager_execute_command(rx_bus_manager_t *manager, const char
*name, rx_bus_command_t *command)
```

Execute a command on a bus using Command Pattern (RECOMMENDED for extensibility).

The recommended interface for implementing new bus operations following the Open/Closed Principle. New operations can be added as commands without modifying the bus manager internals.

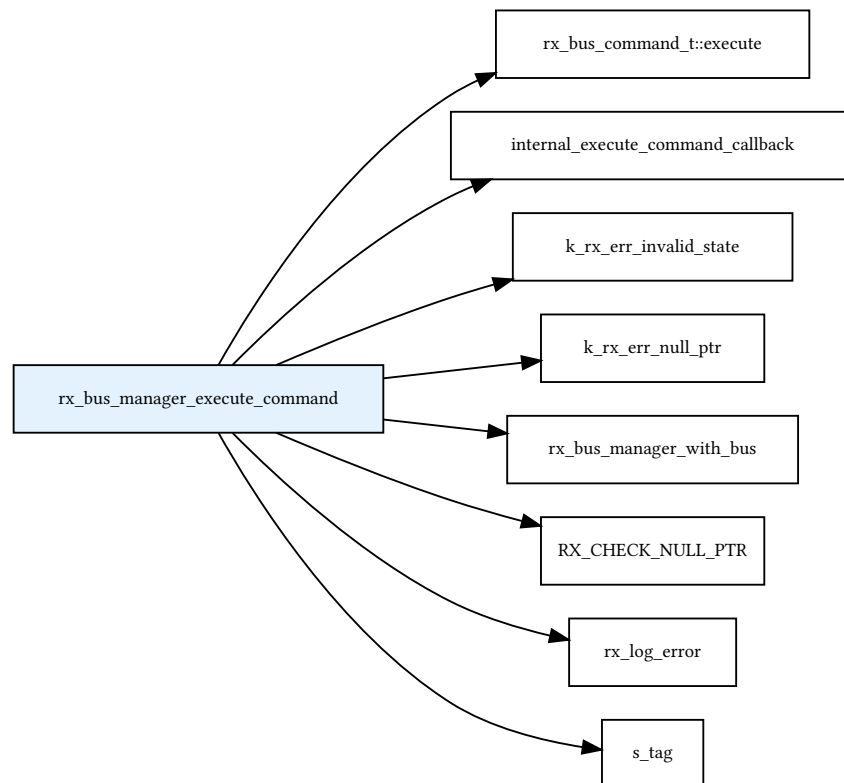


Figure 24: Call graph for `rx_bus_manager_execute_command`

_Defined in libs/rx_bus/inc/rx_bus_manager.h:848_

—

rx_bus_onewire.h

OneWire (1-Wire) Bus Abstraction for RX72N.

Provides bus manager integration for Dallas/Maxim OneWire (1-Wire) protocol operations. Implements bit-banging protocol using GPIO with precise microsecond-level timing control for reliable device communication.

STAR Team

2026-01-02

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_bus_manager.h"
- "rx_err.h"

onewire_timing_us_t

OneWire protocol timing parameters in microseconds.

Precise timing values based on Dallas/Maxim OneWire specification. All values in microseconds (us). These timings are critical for reliable communication - timing violations cause bus errors.

Slot timing must be ≥ 60 us for reliable communication

Sample timing must be within 15 us window

Value	Name	Description
= 480	k_onewire_reset_pulse_us	Reset pulse duration. Controller drives bus LOW for this duration. Value: 480 us (minimum per spec) Longer pulses (up to 960 us) are allowed
= 70	k_onewire_presence_wait_us	Wait time before sampling presence. After releasing reset, wait before sampling. Value: 70 us (within 60-75 us window) Device pulls low 15-60 us after reset release
= 410	k_onewire_presence_tail_us	Remaining presence window after sampling. Wait after sampling to complete the full 960 us reset slot. Value: 410 us (= 480 us recovery - 70 us pre-sample wait) Total reset slot: $480 + 70 + 410 = 960$ us (per spec) Device presence pulse is 60-240 us; this wait covers the remaining recovery window regardless of pulse width
= 960	k_onewire_reset_slot_total_us	Total reset slot duration in microseconds. Sum of reset pulse, presence wait, and presence tail: $k_onewire_reset_pulse_us + k_onewire_presence_wait_us + k_onewire_presence_tail_us = 480 + 70 + 410 = 960$ us. Value: 960 us (per Dallas/Maxim 1-Wire spec)
= 10	k_onewire_write_1_low_us	Write-1 LOW pulse duration. Short pulse for logic 1. Value: 10 us (within 6-15 us window)
= 55	k_onewire_write_1_high_us	Write-1 recovery time. Bus HIGH after LOW pulse. Value: 55 us (slot = 65 us total)

= 60	k_owewire_write_0_low_us	Write-0 LOW pulse duration. Long pulse for logic 0. Value: 60 us (minimum per spec)
= 10	k_owewire_write_0_high_us	Write-0 recovery time. Bus HIGH after LOW pulse. Value: 10 us (short recovery)
= 6	k_owewire_read_init_us	Read initiation LOW pulse. Controller pulls LOW to start read slot. Value: 6 us (within 1-15 us window)
= 9	k_owewire_read_sample_us	Read sample time from slot start. Sample bus state at this time. Value: 9 us (within 15 us window) Must sample before device releases bus
= 55	k_owewire_read_recovery_us	Read slot recovery time. Wait for slot completion. Value: 55 us
= 65	k_owewire_slot_duration_us	Total time slot duration. Minimum slot time + recovery. Value: 65 us (60 us min + 5 us margin)
= 1	k_owewire_recovery_us	Inter-slot recovery time. Minimum time between slots. Value: 1 us (minimum per spec)

See also: Dallas/Maxim Application Note AN126 for timing details

Since Version 1.0.0

—

onewire_rom_command_t

OneWire ROM command codes per Dallas/Maxim specification.

Standard ROM commands used for device addressing on the OneWire bus. All transactions start with reset, followed by a ROM command to select which device(s) will respond.

Value	Name	Description
= 0xF0	k_owewire_cmd_search_rom	Search ROM command. Enumerate all devices using binary search. Value: 0xF0 rx_bus_owewire_search()
= 0x33	k_owewire_cmd_read_rom	Read ROM command. Read 64-bit ROM from single device. Value: 0x33 Only use with single device on bus rx_bus_owewire_read_rom()
= 0x55	k_owewire_cmd_match_rom	Match ROM command. Address specific device by ROM code. Value: 0x55 Followed by 8-byte ROM address rx_bus_owewire_match_rom()
= 0xCC	k_owewire_cmd_skip_rom	Skip ROM command. Address all devices (broadcast). Value: 0xCC Use with single device or broadcast rx_bus_owewire_skip_rom()

= 0xEC	k_owewire_cmd_alarm_search	Alarm search command. Find devices with alarm condition. Value: 0xEC Device-specific alarm criteria
--------	----------------------------	--

See also: rx_bus_owewire_skip_rom() Skip ROM implementation, rx_bus_owewire_match_rom() Match ROM implementation, rx_bus_owewire_search() Search ROM implementation

Since Version 1.0.0

—

owewire_rom_size_t

OneWire ROM code size constant.

ROM code structure (8 bytes / 64 bits):

- Byte 0: Family code (device type)
- Bytes 1-6: Serial number (48 bits, unique)
- Byte 7: CRC-8 of bytes 0-6

Value	Name	Description
= 8	k_owewire_rom_bytes	ROM code size in bytes. 64-bit address = 8 bytes. Value: 8

Since Version 1.0.0

—

rx_bus_owewire_init

```
rx_err_t rx_bus_owewire_init(rx_bus_manager_t *manager, const char *bus_name)
```

Initialize OneWire bus through bus manager.

Configures GPIO pin for OneWire communication (open-drain mode). The pin should have an external 4.7k pullup resistor.

Initialize OneWire bus through bus manager.

Initializes a 1-Wire bus by configuring the GPIO pin as input (released) and allocating runtime state from the static pool. Verifies pullup resistor is present by checking line reads high after release.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	manager	Bus manager handle
in	bus_name	Bus configuration name (must match registered bus)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, bus ready for use

k_rx_err_null_ptr	nullptr manager or bus_name
k_rx_err_not_found	Bus name not registered
k_rx_err_invalid_arg	Bus is not 1-Wire type
k_rx_err_no_mem	Static state pool exhausted
k_rx_err_hw_error	GPIO configuration failed

Pre: manager != nullptr and initialized

Pre: bus_name registered via rx_bus_manager_register()

Pre: 4.7kOhm pullup resistor connected to data line

Post: Bus ready for reset, read, write operations

Note: Logs warning if pullup not detected (may still work)

Algorithm:: Validate manager and bus_name pointers Acquire runtime state from pool Configure GPIO as input (high-Z, pullup drives high) Verify line reads high (warn if not) Reset search state Mark bus as initialized

Example:: rx_err_t err = rx_bus_owewire_init(&bus_manager, "temp_sensor"); if (err != k_rx_ok) { rx_log_error("1W", "Init failed"); }

See also: rx_bus_owewire_reset() First operation after init

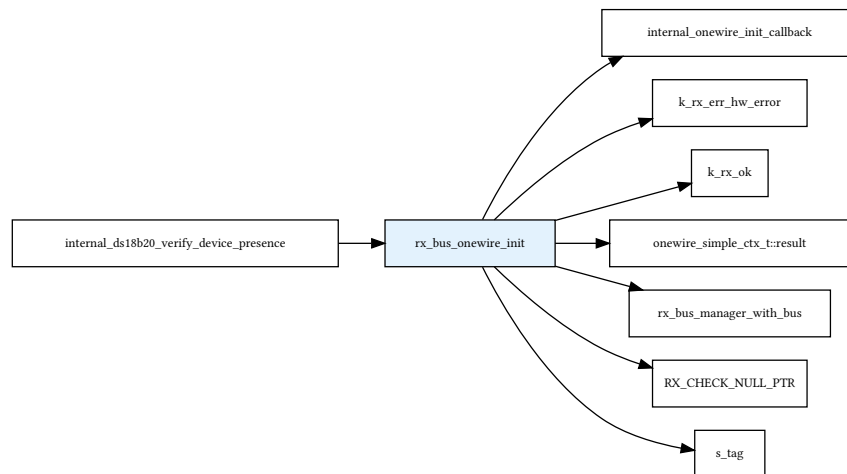


Figure 25: Call graph for rx_bus_owewire_init

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:413_

rx_bus_owewire_deinit

```
rx_err_t rx_bus_owewire_deinit(rx_bus_manager_t *manager, const char *bus_name)
```

Deinitialize OneWire bus and release state pool entry.

Releases the runtime state allocated by rx_bus_owewire_init() back to the static pool. Must be called before destroying the bus manager to prevent state pool exhaustion.

Releases the runtime state allocated by rx_bus_owewire_init() back to the static pool. Prevents state pool exhaustion in long-running test suites.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	manager	Bus manager handle
in	bus_name	Bus configuration name

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	manager or bus_name is nullptr
k_rx_err_not_found	Bus not registered
k_rx_err_invalid_arg	Bus is not OneWire type

Pre: Bus was initialized via rx_bus_owewire_init()

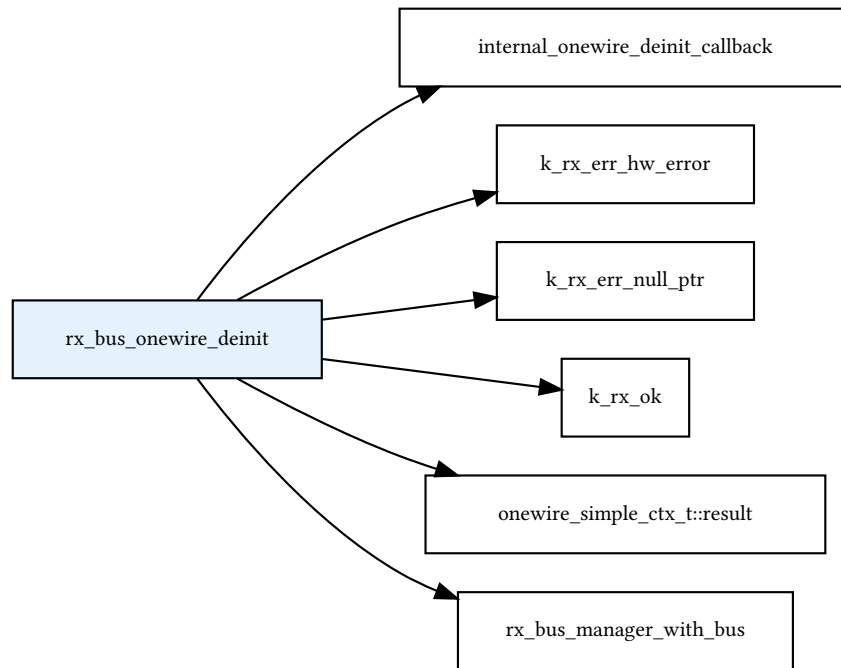
Pre: Bus was initialized via rx_bus_owewire_init()

Post: State pool entry released for reuse

Post: bus_config->initialized set to false

Post: State pool entry released for reuse

Post: bus_config->initialized == false] **See also:** rx_bus_owewire_init() Initialization counterpart, rx_bus_owewire_init() Initialization counterpart

Figure 26: Call graph for `rx_bus_owewire_deinit`

_Defined in `libs/rx\_bus/inc/rx\_bus\_owewire.h:439_`

Since Version 1.0.0

—

`rx_bus_owewire_reset`

```
rx_err_t rx_bus_owewire_reset(rx_bus_manager_t *manager, const char *bus_name,
bool *presence)
```

Perform OneWire reset and check for presence pulse.

Sends reset pulse (480us low) and samples for presence pulse from device. This is the first step in any OneWire transaction.

Perform OneWire reset and check for presence pulse.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
out	presence	Presence detected (true if device present)
in	manager	Bus manager handle
in	bus_name	Bus configuration name

out	presence	True if device responded, false otherwise
-----	----------	---

Returns: k_rx_ok on success, error code on failure

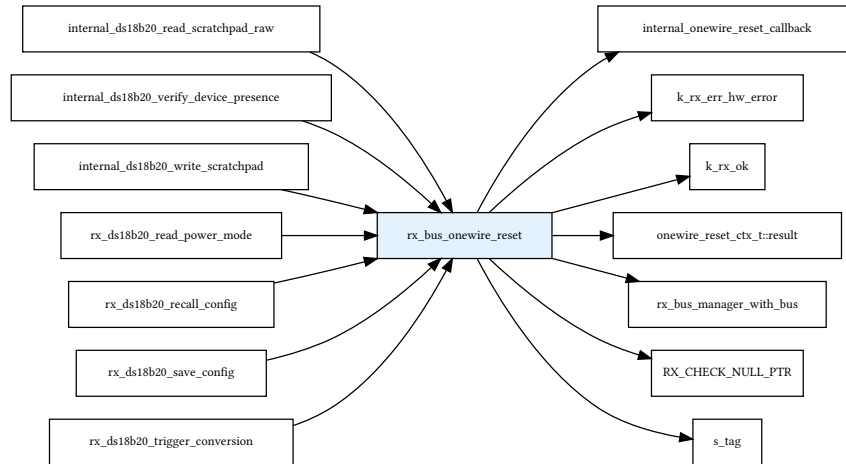


Figure 27: Call graph for rx_bus_owewire_reset

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:458_

rx_bus_owewire_write_bit

```
rx_err_t rx_bus_owewire_write_bit(rx_bus_manager_t *manager, const char
*bus_name, bool bit)
```

Write a single bit to OneWire bus.

Timing:

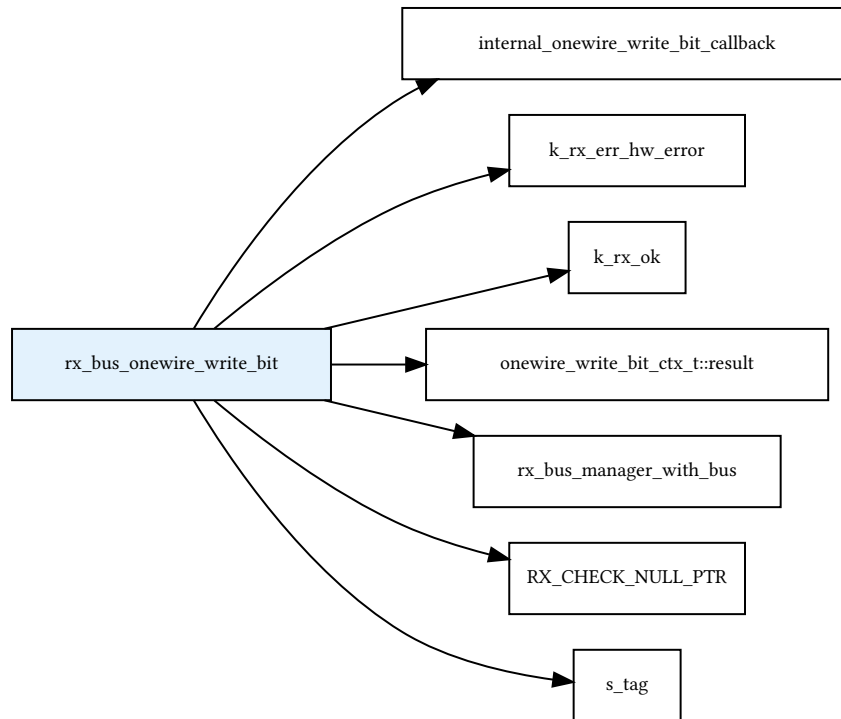
- Write 1: 10us low, 55us high
- Write 0: 60us low, 10us high

Write a single bit to OneWire bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	bit	Bit value to write (true = 1, false = 0)
in	manager	Bus manager handle
in	bus_name	Bus configuration name
in	bit	Bit value to write (true=1, false=0)

Returns: k_rx_ok on success, error code on failure

Figure 28: Call graph for `rx_bus_owewire_write_bit`

_Defined in `libs/rx\_bus/inc/rx\_bus\_owewire.h:478_`

`rx_bus_owewire_read_bit`

```
rx_err_t rx_bus_owewire_read_bit(rx_bus_manager_t *manager, const char *bus_name,
bool *bit)
```

Read a single bit from OneWire bus.

Timing:

- 6us low pulse to initiate read
- Sample at 9us from start
- 55us recovery time

Read a single bit from OneWire bus.

Parameters:

Direction	Name	Description
in	<code>manager</code>	Bus manager instance
in	<code>bus_name</code>	OneWire bus name
out	<code>bit</code>	Bit value read (true = 1, false = 0)

in	manager	Bus manager handle
in	bus_name	Bus configuration name
out	bit	Bit value read (true=1, false=0)

Returns: k_rx_ok on success, error code on failure

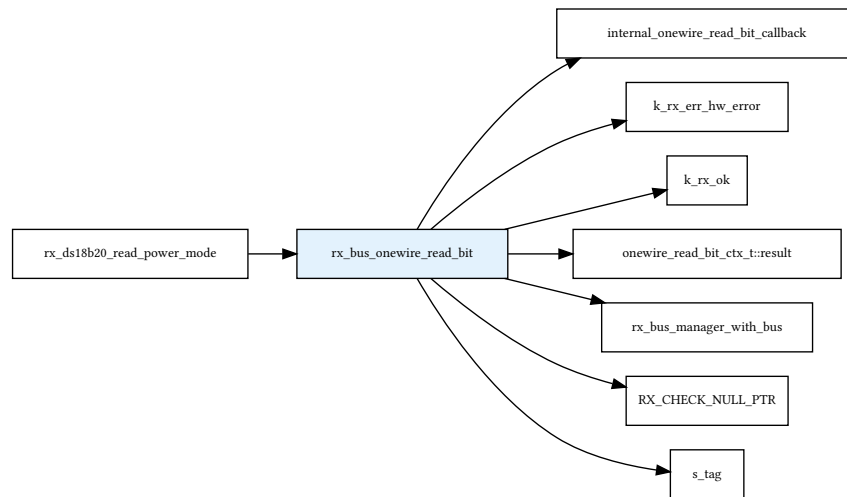


Figure 29: Call graph for rx_bus_owewire_read_bit

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:499_

rx_bus_owewire_write_byte

```
rx_err_t rx_bus_owewire_write_byte(rx_bus_manager_t *manager, const char
*bus_name, uint8_t byte)
```

Write a byte to OneWire bus.

Writes 8 bits LSB first.

Write a byte to OneWire bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	byte	Byte value to write
in	manager	Bus manager handle
in	bus_name	Bus configuration name
in	byte	Byte value to write

Returns: k_rx_ok on success, error code on failure

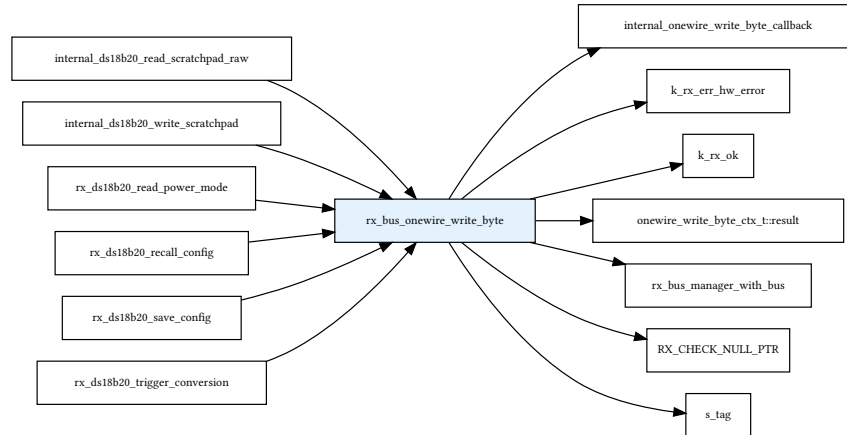


Figure 30: Call graph for `rx_bus_owewire_write_byte`

_Defined in `libs/rx\_bus/inc/rx\_bus\_owewire.h:517_`

`rx_bus_owewire_read_byte`

```
rx_err_t rx_bus_owewire_read_byte(rx_bus_manager_t *manager, const char
*bus_name, uint8_t *byte)
```

Read a byte from OneWire bus.

Reads 8 bits LSB first.

Read a byte from OneWire bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
out	byte	Byte value read
in	manager	Bus manager handle
in	bus_name	Bus configuration name
out	byte	Byte value read

Returns: k_rx_ok on success, error code on failure

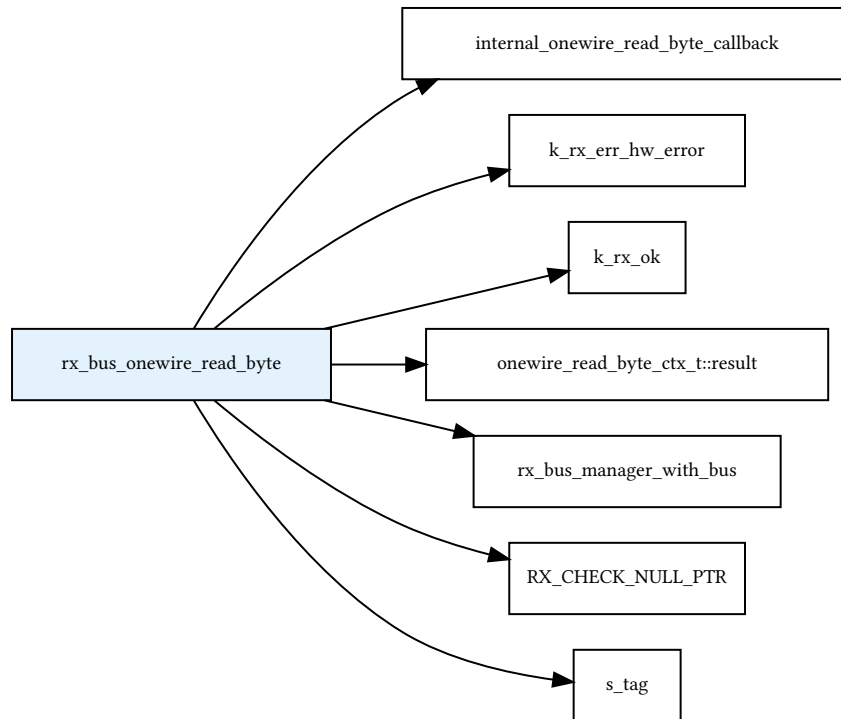


Figure 31: Call graph for rx_bus_owewire_read_byte

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:535_

rx_bus_owewire_write

```
rx_err_t rx_bus_owewire_write(rx_bus_manager_t *manager, const char *bus_name,
const uint8_t *data, uint32_t length)
```

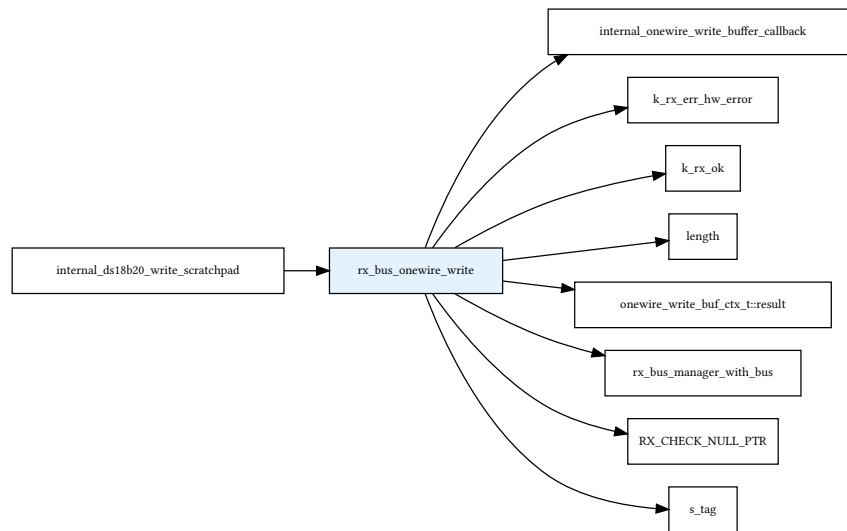
Write multiple bytes to OneWire bus.

Convenience function for writing buffers.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	data	Pointer to data buffer
in	length	Number of bytes to write

Returns: k_rx_err_timeout if mutex timeout

Figure 32: Call graph for `rx_bus_owewire_write`

_Defined in `libs/rx\_bus/inc/rx\_bus\_owewire.h:553_`

rx_bus_owewire_read

```
rx_err_t rx_bus_owewire_read(rx_bus_manager_t *manager, const char *bus_name,
uint8_t *data, uint32_t length)
```

Read multiple bytes from OneWire bus.

Convenience function for reading buffers.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
out	data	Pointer to receive buffer
in	length	Number of bytes to read

Returns: `k_rx_err_timeout` if mutex timeout

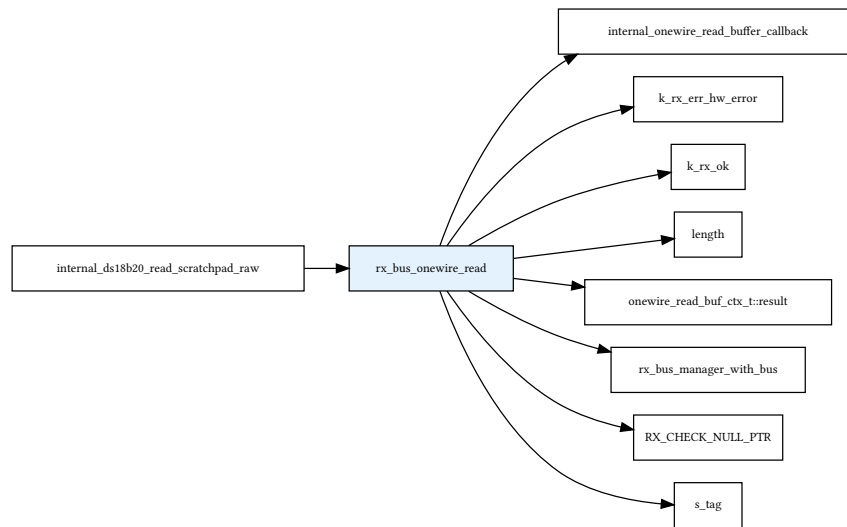


Figure 33: Call graph for rx_bus_owewire_read

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:574_

—

rx_bus_owewire_skip_rom

```
rx_err_t rx_bus_owewire_skip_rom(rx_bus_manager_t *manager, const char *bus_name)
```

Send Skip ROM command.

Addresses all devices on the bus (broadcast). Use when only one device is present or for global commands.

Sequence:

1. Reset
2. Write SKIP_ROM command (0xCC)

Send Skip ROM command.

Sends Skip ROM command (0xCC) to address all devices on the bus. Use when only one device is connected or when broadcasting to all devices.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	manager	Bus manager handle
in	bus_name	Bus configuration name

Returns: k_rx_err_not_found if no device presence detected

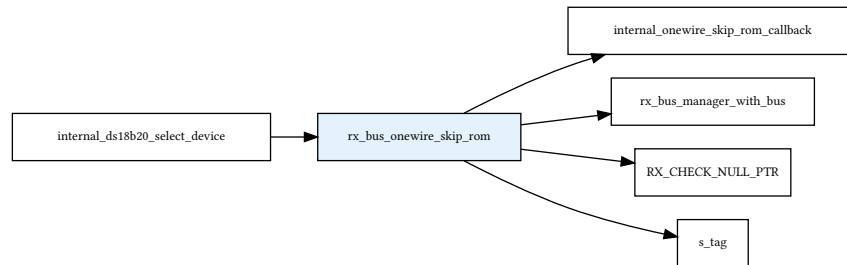


Figure 34: Call graph for rx_bus_owewire_skip_rom

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:598_

rx_bus_owewire_match_rom

```
rx_err_t rx_bus_owewire_match_rom(rx_bus_manager_t *manager, const char
*bus_name, const uint8_t rom[k_owewire_rom_bytes])
```

Send Match ROM command with specific device address.

Addresses a specific device by its 64-bit ROM code.

Sequence:

1. Reset
2. Write MATCH_ROM command (0x55)
3. Write 8-byte ROM code

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	rom	8-byte ROM code of target device

Returns: k_rx_err_timeout if mutex timeout

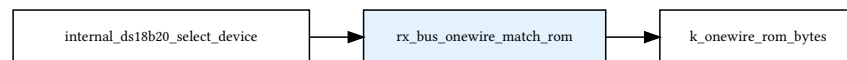


Figure 35: Call graph for rx_bus_owewire_match_rom

_Defined in libs/rx_bus/inc/rx_bus_owewire.h:620_

rx_bus_owewire_read_rom

```
rx_err_t rx_bus_owewire_read_rom(rx_bus_manager_t *manager, const char *bus_name,
uint8_t rom[k_owewire_rom_bytes])
```

Read ROM code from single device.

Reads the 64-bit ROM code from the only device on the bus. Only works when exactly one device is connected.

Sequence:

1. Reset
2. Write READ_ROM command (0x33)
3. Read 8 bytes (ROM code)

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
out	rom	8-byte buffer for ROM code

Returns: k_rx_err_hw_error if multiple devices present (ROM collision)

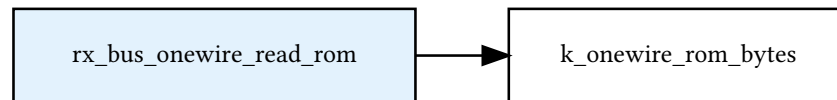


Figure 36: Call graph for `rx_bus_owewire_read_rom`

_Defined in `libs/rx_bus/inc/rx_bus_owewire.h:646_`

rx_bus_owewire_search

```
rx_err_t rx_bus_owewire_search(rx_bus_manager_t *manager, const char *bus_name,
uint8_t *roms, uint32_t max_devices, uint32_t *num_devices)
```

Search for all devices on the OneWire bus.

Implements the OneWire search algorithm to enumerate all devices. The search algorithm uses binary tree traversal to discover all unique 64-bit ROM codes on the bus.

Sequence:

1. Reset
2. Write SEARCH_ROM command (0xF0)
3. For each bit position (64 bits):

Read bit Read complement Write search direction

4. Repeat until all devices found

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
out	roms	Array of ROM codes (8 bytes each)
in	max_devices	Maximum number of devices to search for

out	num_devices	Number of devices found
-----	-------------	-------------------------

Returns: k_rx_err_crc if ROM CRC check fails

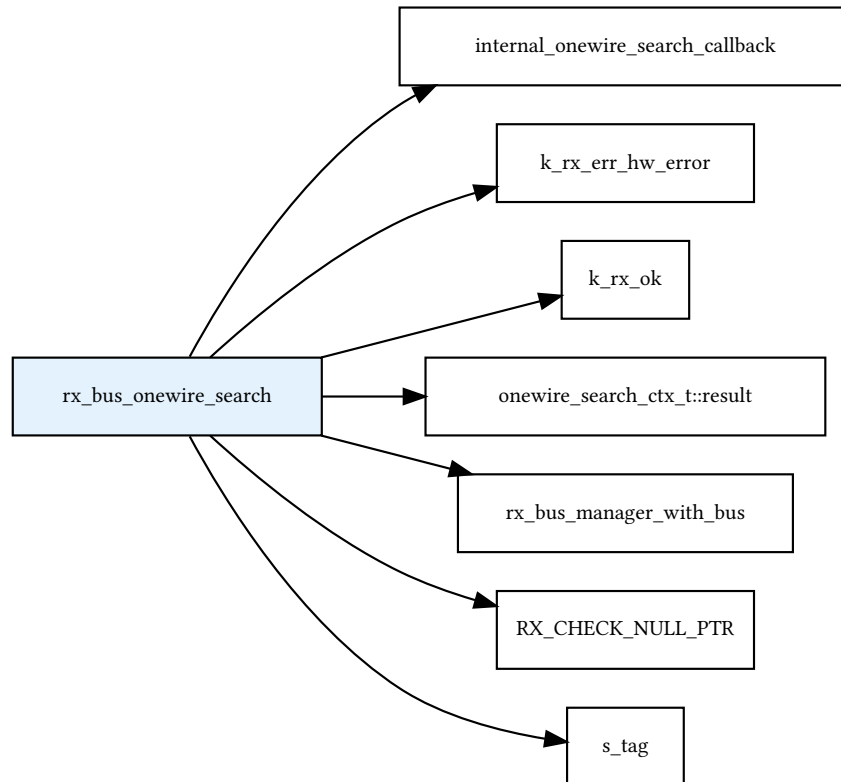


Figure 37: Call graph for `rx_bus_owewire_search`

_Defined in `libs/rx\_bus/inc/rx\_bus\_owewire.h:680_`

rx_bus_types.h

Bus Manager Type Definitions for RX72N Multi-Protocol Communication.

Includes:

- `<stdbool.h>`
- `<stddef.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_error_interface.h"`
- `"rx_gpio_constants.h"`
- `"rx_pin_interface.h"`
- `"rx_port_constants.h"`
- `"tx_api.h"`

rx_bus_type_t

Supported bus/protocol types for RX72N bus manager.

Enumerates all communication protocols supported by the bus manager. Each type corresponds to a specific RX72N peripheral or software bit-bang implementation.

Value	Name	Description
= 0	k_bus_type_gpio	General Purpose Input/Output (digital I/O).
	k_bus_type_adc	Analog-to-Digital Converter (analog input).
	k_bus_type_i2c	Inter-Integrated Circuit (I2C/IIC/TWI).
	k_bus_type_spi	Serial Peripheral Interface (4-wire synchronous).
	k_bus_type_uart	Universal Asynchronous Receiver/Transmitter.
	k_bus_type_onewire	Dallas/Maxim 1-Wire protocol (single-wire bidirectional).
	k_bus_type_max	Sentinel value for bounds checking.

—

rx_riic_limits_t

RX72N RIIC channel count.

Value	Name	Description
= 3	k_riic_channel_count	RIIC channels 0-2

—

rx_rspi_limits_t

RX72N RSPI channel count.

Value	Name	Description
= 3	k_rspi_channel_count	RSPI channels 0-2

—

rx_adc_limits_t

RX72N ADC unit count.

Value	Name	Description
= 2	k_adc_unit_count	ADC units 0-1

—

rx_sci_limits_t

RX72N SCI channel count.

Value	Name	Description
= 13	k_sci_channel_count	SCI channels 0-12

—

rx_adc_channel_limits_t

RX72N ADC channel limits.

Value	Name	Description
= 7	k_adc_channel_max	ADC channels 0-7

—

rx_adc_resolution_t

RX72N ADC resolution options.

Value	Name	Description
= 8	k_adc_resolution_8bit	8-bit resolution
= 10	k_adc_resolution_10bit	10-bit resolution
= 12	k_adc_resolution_12bit	12-bit resolution

—

rx_i2c_constants_t

I2C protocol constants.

Value	Name	Description
= 0	k_i2c_write_bit	I2C write bit (R/W = 0)
= 1	k_i2c_read_bit	I2C read bit (R/W = 1)
= 1	k_i2c_addr_shift	Bit shift for 7-bit address
= 127	k_i2c_addr_max_7bit	Maximum 7-bit I2C address (127)
= 8	k_bits_per_byte	Bits per byte

—

bit_masks_t

Bit manipulation masks.

Value	Name	Description
= 255	k_byte_mask	Full byte mask (all 8 bits)
= 128	k_byte_msb_mask	Most significant bit of a byte (bit 7)

—

bus_manager_limits_t

Bus manager capacity and name limits.

Defines compile-time limits for the bus manager to enable static memory allocation and bounds checking.

Value	Name	Description
= 32	k_max_buses	Maximum number of buses that can be registered.

= 31	k_max_bus_name_len	Maximum bus name length (excluding null terminator).
------	--------------------	--

Since Version 1.0.0

—

bus_manager_timeout_t

Bus manager configuration constants.

Default mutex timeout for bus operations

Value	Name	Description
= 1000U	k_bus_manager_mutex_timeout_ms	Default mutex timeout in milliseconds

—

rx_bus_adc.c

ADC (Analog-to-Digital Converter) Bus Abstraction Implementation for RX72N.

Production-quality ADC bus abstraction for Renesas S12ADFa (Successive Approximation ADC Fast) peripheral. Provides thread-safe analog voltage measurement with automatic scaling, mutex protection, and comprehensive error handling through the bus manager abstraction pattern.

Includes:

- "rx_bus_adc.h"
- "hardware.h"
- "rx_bus_types.h"
- "rx_check.h"
- "rx_log.h"

adc_voltage_limits_t

ADC voltage range safety limits in millivolts.

Value	Name	Description
= 5500U	k_adc_max_safety_voltage_mv	5.5 V upper bound for sanity check on raw ADC output

—

s_tag

```
const char s_tag[]
```

—

internal_adc_init_callback

```
static rx_err_t internal_adc_init_callback(rx_bus_config_t *bus_config, void  
*user_ctx)
```

Internal callback for ADC channel initialization.

Initializes an S12ADFa ADC channel for analog voltage measurement. Called by bus manager via `rx_bus_manager_with_bus()` callback mechanism. Configures ADC unit, channel, resolution, and performs initial dummy conversion for hardware settling.

Algorithm steps:

1. Cast `user_ctx` to `adc_init_ctx_t*`
2. Validate bus type is `k_bus_type_adc`
3. Call `adc_init()` with unit, channel, resolution
4. Perform test read for hardware settling verification
5. Mark bus as initialized
6. Set `ctx->result` and return

Bus type remains `k_bus_type_adc`

Failure leaves ADC unit in partially initialized state

Exceeding `VREFH` on analog input may damage MCU

Parameters:

Direction	Name	Description
inout	<code>bus_config</code>	Bus configuration structure Must have type = <code>k_bus_type_adc</code> proto.adc.unit: 0 or 1 (S12AD0 or S12AD1) proto.adc.channel: 0-15 (unit-specific channels) proto.adc.bits: 8, 10, or 12 (resolution) initialized flag set to true on success
inout	<code>user_ctx</code>	User context (<code>adc_init_ctx_t*</code>) output: <code>ctx->result</code> contains operation result

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, ADC initialized
<code>k_rx_err_invalid_arg</code>	Bus type is not ADC
<code>k_rx_err_hw_init_failed</code>	ADC HAL initialization failed
<code>k_rx_err_timeout</code>	Test read timeout (hardware not responding)

Pre: `bus_config` pointer is valid (validated by bus manager)

Pre: `user_ctx` pointer is valid and points to `adc_init_ctx_t`

Pre: ADC unit/channel combination is valid

Pre: Analog input voltage within 0 to `VREFH`

Post: `bus_config->initialized` = true on success

Post: ctx->result contains operation result

Post: S12AD unit enabled and configured

Post: Analog pin configured for ADC input

Post: Initial dummy conversion performed (hardware settling)

Note: Called from bus manager with mutex held (thread-safe)

Note: Test read may fail on first attempt (settling time - acceptable)

Note: Execution time: 50 us (includes dummy conversion)

Note: First real conversion after init should be discarded for best accuracy

Warning: Do not call directly - use rx_bus_adc_init() instead

ADC Initialization Sequence:: Hardware initialization performed by adc_init(): Enable S12AD module clock (MSTPCRA.MSTPA17 or MSTPA16) Configure pin as analog input (MPC, PMR, PDR) Set resolution (ADCER.ADPRC = 12-bit) Set sampling time (ADSSTR0-7 = 75 states 3 us @ 25 MHz) Set trigger mode (ADCSR.EXTRG = 0, software trigger) Enable ADC unit (ADCSR.ADST cleared initially)

See also: rx_bus_adc_init() Public API that calls this callback, adc_init() HAL function for S12ADFa initialization, adc_read() HAL function for test read verification

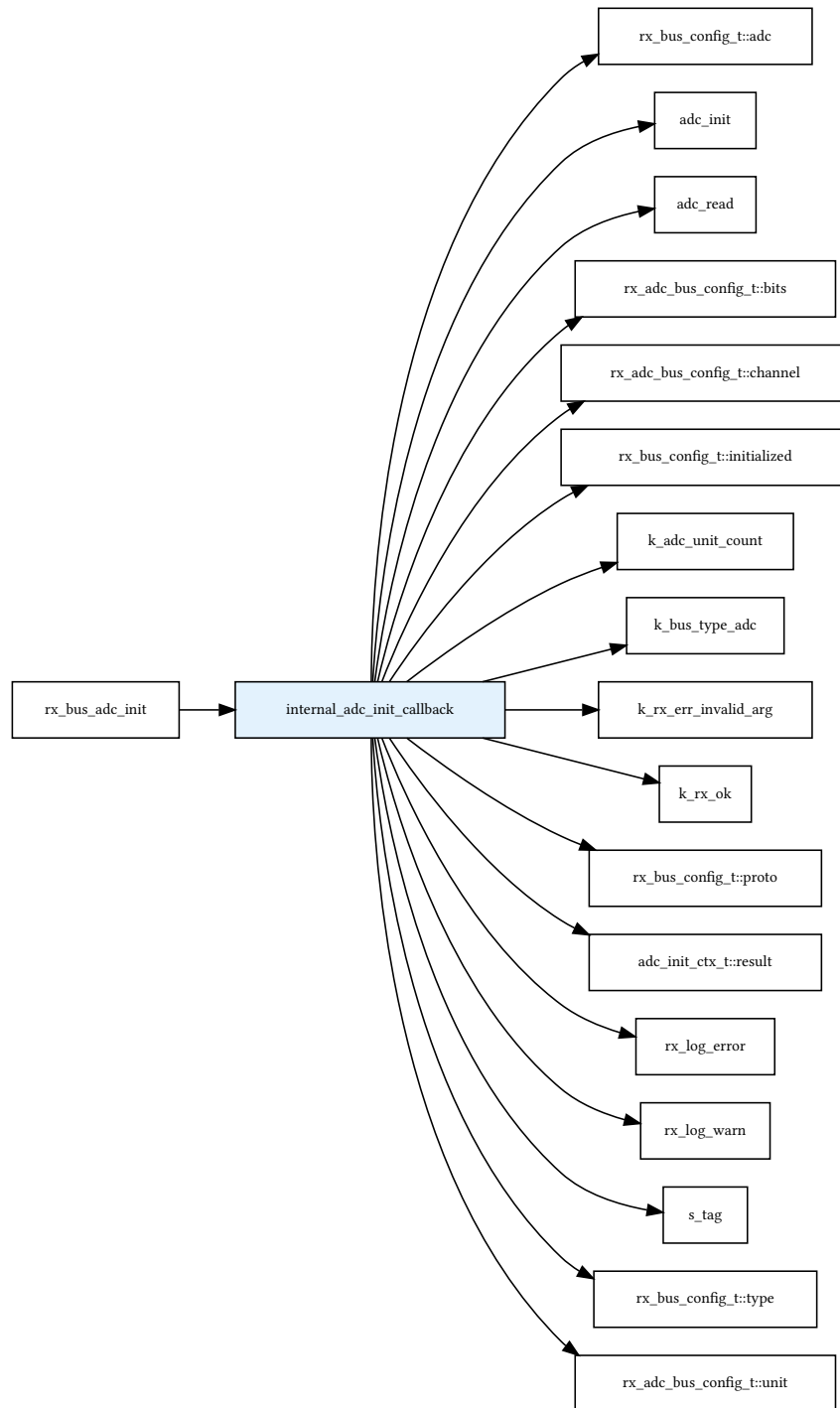


Figure 38: Call graph for internal_adc_init_callback

_Defined in libs/rx_bus/src/rx_bus_adc.c:397_

Since Version 1.0.0

—

internal_adc_read_callback

```
static rx_err_t internal_adc_read_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for ADC raw value read operation.

Performs a single ADC conversion and returns the raw digital value (0-4095 for 12-bit resolution). Called by bus manager via rx_bus_manager_with_bus(). Triggers conversion, waits for completion, and reads result register.

Algorithm steps:

1. Cast user_ctx to adc_read_ctx_t*
2. Validate bus initialized
3. Call adc_read() to trigger conversion and wait
4. Store raw value in *ctx->value
5. Verify value within valid range for configured resolution
6. Set ctx->result and return

Bus remains initialized

value range: 0 to (2^{bits} - 1)

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration structure Must be initialized (initialized flag = true) proto.adc.unit: 0 or 1 (S12AD0/1) proto.adc.channel: ADC channel number proto.adc.bits: Resolution for range validation
inout	user_ctx	User context (adc_read_ctx_t*) input: ctx->value points to output uint16_t output: *ctx->value set to raw ADC result output: ctx->result contains operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, value contains raw ADC reading
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Conversion timeout (hardware fault)
k_rx_err_hw_error	ADC read failed

Pre: bus_config->initialized must be true

Pre: user_ctx points to valid adc_read_ctx_t

Pre: ctx->value points to valid uint16_t

Pre: Analog input voltage within 0 to VREFH

Post: *ctx->value contains raw ADC result (0 to 2^{bits} - 1)

Post: ctx->result contains operation result

Post: ADC ready for next conversion

Note: Called from bus manager with mutex held (thread-safe)

Note: Blocking call - waits for conversion completion (4.5 us)

Note: Value exceeding max_value logged as warning but operation succeeds

Note: For voltage conversion, use internal_adc_voltage_callback() instead

Warning: Analog input exceeding VREFH may damage MCU

Warning: Value undefined if return != k_rx_ok

ADC Conversion Process:: Hardware operations performed by adc_read(): Set ADCSR.ADST = 1 (start conversion) Wait for ADCSR.ADST = 0 (conversion complete, 4.5 us) Read ADDRn register (n = channel number) Clear ADCSR.ADIE interrupt flag

Timing:: Sampling: 3.0 us (75 ADCLK cycles @ 25 MHz) Conversion: 1.5 us (12 ADCLK cycles for 12-bit) Total: 4.5 us typical

See also: rx_bus_adc_read() Public API that calls this callback, adc_read() HAL function to perform conversion, internal_adc_voltage_callback() For automatic voltage scaling

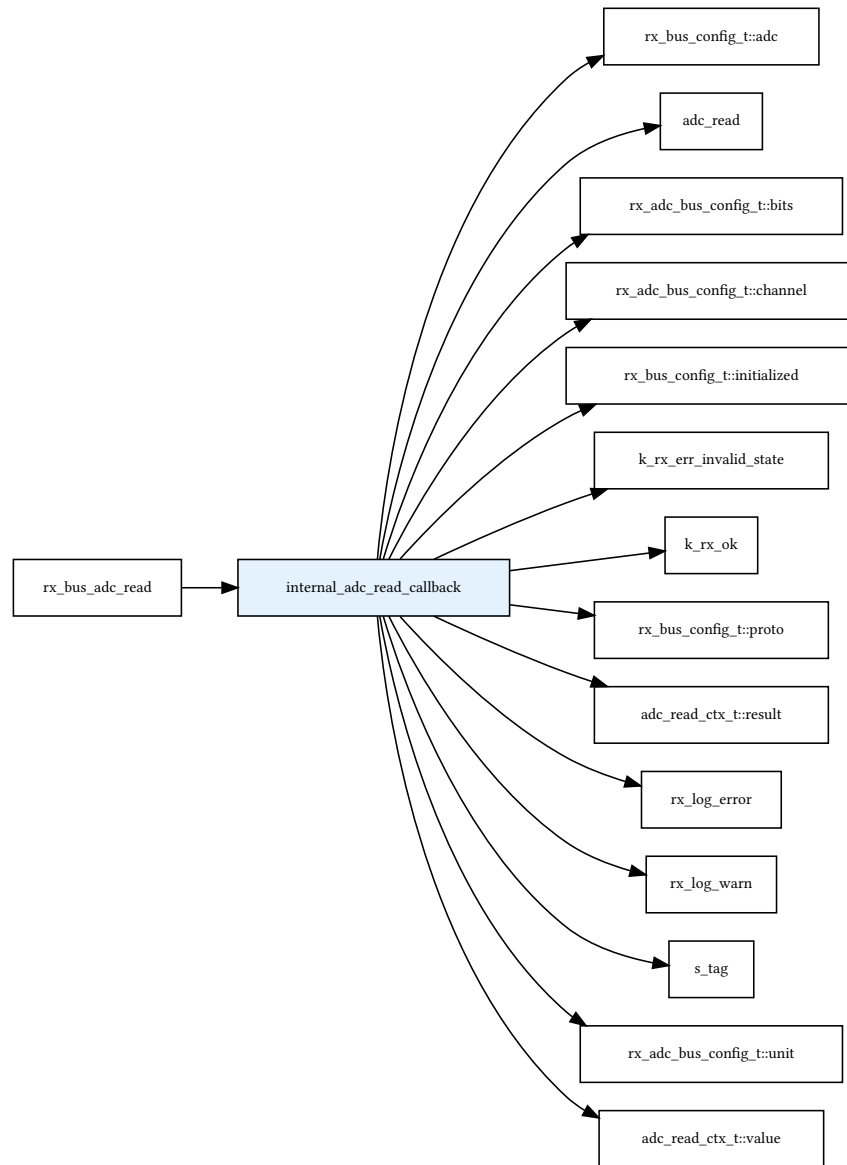


Figure 39: Call graph for `internal_adc_read_callback`

Defined in `libs/rx_bus/src/rx_bus_adc.c:509`

Since Version 1.0.0

—

internal_adc_voltage_callback

```
static rx_err_t internal_adc_voltage_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for ADC voltage read operation with automatic scaling.

Performs a single ADC conversion and automatically converts the raw value to millivolts based on configured VREF. Called by bus manager via rx_bus_manager_with_bus(). This is the preferred function for direct voltage measurement applications.

Algorithm steps:

1. Cast user_ctx to adc_voltage_ctx_t*
2. Validate bus initialized
3. Call adc_read_voltage_mv() with unit, channel, resolution, VREF
4. Store voltage in millivolts in *ctx->voltage_mv
5. Verify voltage within reasonable range (0-5.5V safety check)
6. Set ctx->result and return

Example @ 12-bit, 3.3V VREF:

- ADC = 0 -> V = 0 mV
- ADC = 2048 -> V = 1651 mV (mid-scale)
- ADC = 4095 -> V = 3300 mV (full-scale)

Precision: 0.805 mV/LSB @ 3.3V VREF, 12-bit

Bus remains initialized

voltage_mv range: 0 to vref_mv

For temperature sensing (analog sensor):

- Read voltage from ADC
- Apply sensor-specific conversion (see sensor datasheet)

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration structure Must be initialized (initialized flag = true) proto.adc.unit: S12AD unit number (0 or 1) proto.adc.channel: ADC channel number proto.adc.bits: Resolution (8, 10, or 12) proto.adc.vref_mv: Reference voltage in millivolts (typically 3300 or 5000)
inout	user_ctx	User context (adc_voltage_ctx_t*) input: ctx->voltage_mv points to output uint32_t input: ctx->bits contains resolution (from bus_config) output: *ctx->voltage_mv set to voltage in mV output: ctx->result contains operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, voltage_mv contains reading in millivolts
k_rx_err_invalid_state	Bus not initialized

k_rx_err_timeout	Conversion timeout (hardware fault)
k_rx_err_hw_error	ADC voltage read failed

Pre: bus_config->initialized must be true

Pre: user_ctx points to valid adc_voltage_ctx_t

Pre: ctx->voltage_mv points to valid uint32_t

Pre: bus_config->proto.adc.vref_mv is valid (typically 3300 or 5000)

Pre: Analog input voltage within 0 to VREFH

Post: *ctx->voltage_mv contains input voltage in millivolts

Post: ctx->result contains operation result

Post: ADC ready for next conversion

Note: Called from bus manager with mutex held (thread-safe)

Note: Blocking call - waits for conversion + calculation (12 us)

Note: Voltage > 5.5V logged as warning but operation succeeds

Note: External voltage dividers must be accounted for in application layer

Note: Uses VREF from bus configuration (set during rx_bus_register)

Warning: Analog input exceeding VREFH may damage MCU

Warning: voltage_mv undefined if return != k_rx_ok

Warning: Voltage dividers for scaling must be handled in the application layer

Voltage Conversion Formula:: The HAL function adc_read_voltage_mv() implements: $V_{mV} = \{ADC_raw \times V_REF_mV\} / 2^{bits} - 1$

Example Usage Context:: For motor current sensing (DRV8263H IPROP): Read voltage from ADC
 Convert using: $I_{\text{motor}} = V_{\text{sense}} * k_{\text{ipropi_ratio}} / k_{\text{ipropi_sense_ohm}}$ Example: $I = (V_{\text{sense}} / 4990) * 1000$ (see rx72n_adc_regs.h for constants)

See also: rx_bus_adc_read_voltage_mv() Public API that calls this callback,
 adc_read_voltage_mv() HAL function to perform conversion and scaling,
 internal_adc_read_callback() For raw ADC value without scaling

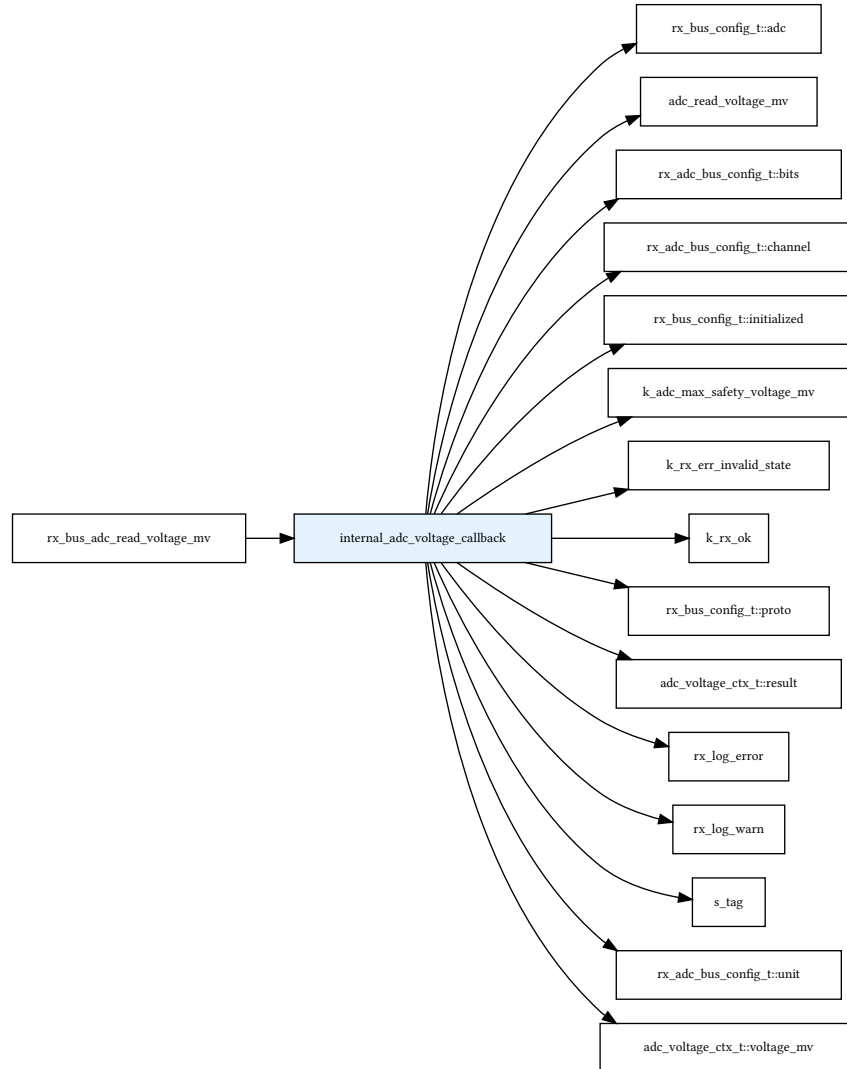


Figure 40: Call graph for internal_adc_voltage_callback

_Defined in libs/rx_bus/src/rx_bus_adc.c:628_

Since Version 1.0.0

rx_bus_adc_init

```
rx_err_t rx_bus_adc_init(rx_bus_manager_t *manager, const char *bus_name)
```

Initialize ADC channel through bus manager.

Public API function to initialize an S12ADFa ADC channel for analog voltage measurement. Delegates to bus manager which calls `internal_adc_init_callback()` with mutex protection and bus lookup.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context for operation result
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context
5. Return result to caller

This function demonstrates the callback pattern for bus operations:

- Prepares context on stack (zero allocation)
- Delegates to bus manager for mutex protection
- Callback executes with mutex held
- Results extracted after mutex released

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via <code>rx_bus_manager_init()</code> Must have ADC bus registered
in	bus_name	Name of the ADC bus Must match registered bus name Null-terminated string

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, ADC initialized
<code>k_rx_err_null_ptr</code>	nullptr in manager or bus_name
<code>k_rx_err_not_found</code>	Bus name not found
<code>k_rx_err_invalid_arg</code>	Bus is not ADC type
<code>k_rx_err_timeout</code>	Mutex timeout
<code>k_rx_err_hw_init_failed</code>	ADC hardware initialization failed

Pre: manager must be initialized

Pre: Bus must be registered with type `k_rx_bus_type_adc`

Pre: Analog input voltage must be within 0 to VREFH

Post: S12AD unit configured and enabled

Post: Analog pin configured for ADC input

Post: Bus marked as initialized

Post: Initial dummy conversion performed (settling)

Note: Thread-safe via bus manager mutex

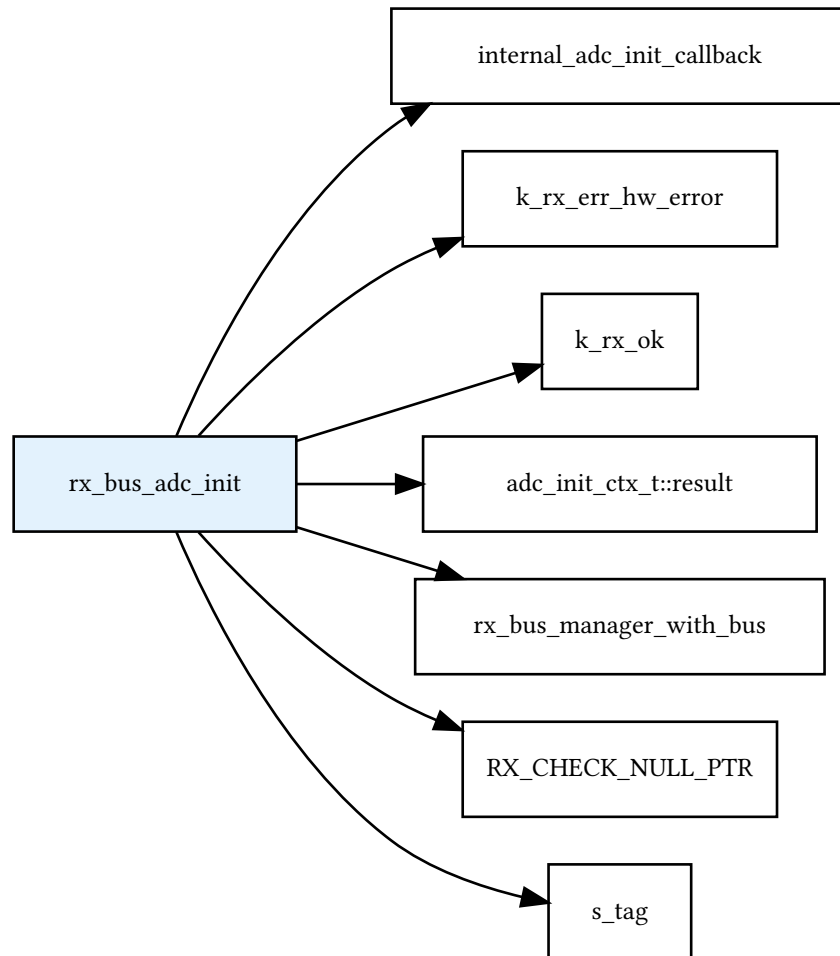
Note: Stack usage: 4 bytes for context

Note: Execution time: 50 us

Note: First conversion after init should be discarded for best accuracy

Example:: `rx_err_t err=rx_bus_adc_init(&manager,"motor_current"); if(err!=k_rx_ok)
{ rx_log_error("APP","Failed to init ADC:%d",err); }`

See also: `rx_bus_adc.h` Complete API documentation, `internal_adc_init_callback()`
Implementation callback

Figure 41: Call graph for `rx_bus_adc_init`

_Defined in `libs/rx_bus/src/rx_bus_adc.c:729`

Since Version 1.0.0

—

rx_bus_adc_read

```
rx_err_t rx_bus_adc_read(rx_bus_manager_t *manager, const char *bus_name,
uint16_t *value)
```

Read raw ADC value through bus manager.

Read ADC raw value through bus manager.

Public API function to perform single ADC conversion and return raw digital value (0-4095 for 12-bit). No voltage scaling is performed - use `rx_bus_adc_read_voltage_mv()` for automatic voltage

conversion. Delegates to bus manager which calls `internal_adc_read_callback()` with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, value)
2. Create stack-allocated context with value pointer
3. Call `rx_bus_manager_with_bus()` to execute callback
4. Extract result from context (value already updated by callback)
5. Return result to caller

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized Must have ADC bus registered and initialized
in	bus_name	Name of the ADC bus Must match registered and initialized bus
out	value	Pointer to store raw ADC value Range: 0-4095 (12-bit resolution) 0 = 0V input, 4095 = VREF input Valid only if k_rx_ok returned

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, value contains raw ADC reading
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Conversion or mutex timeout

Pre: Bus must be initialized via `rx_bus_adc_init()`

Pre: All pointers must be non-NULL

Pre: value must point to valid `uint16_t`

Post: *value contains raw 12-bit ADC result (if k_rx_ok)

Post: ADC ready for next conversion

Note: Thread-safe via bus manager mutex

Note: Blocking call - waits for conversion (4.5 us)

Note: Stack usage: 16 bytes for context

Note: Execution time: 10 us

Warning: value undefined if return != k_rx_ok

Warning: Analog input must not exceed VREFH

Example:: uint16_t raw_adc=0; rx_err_t err=rx_bus_adc_read(&manager,"sensor",&raw_adc);
if(err!=k_rx_ok){ uint32_t voltage_mv=(raw_adc*3300UL)/4095; }

See also: rx_bus_adc.h Complete API documentation, internal_adc_read_callback()
Implementation callback, rx_bus_adc_read_voltage_mv() Read with automatic voltage conversion

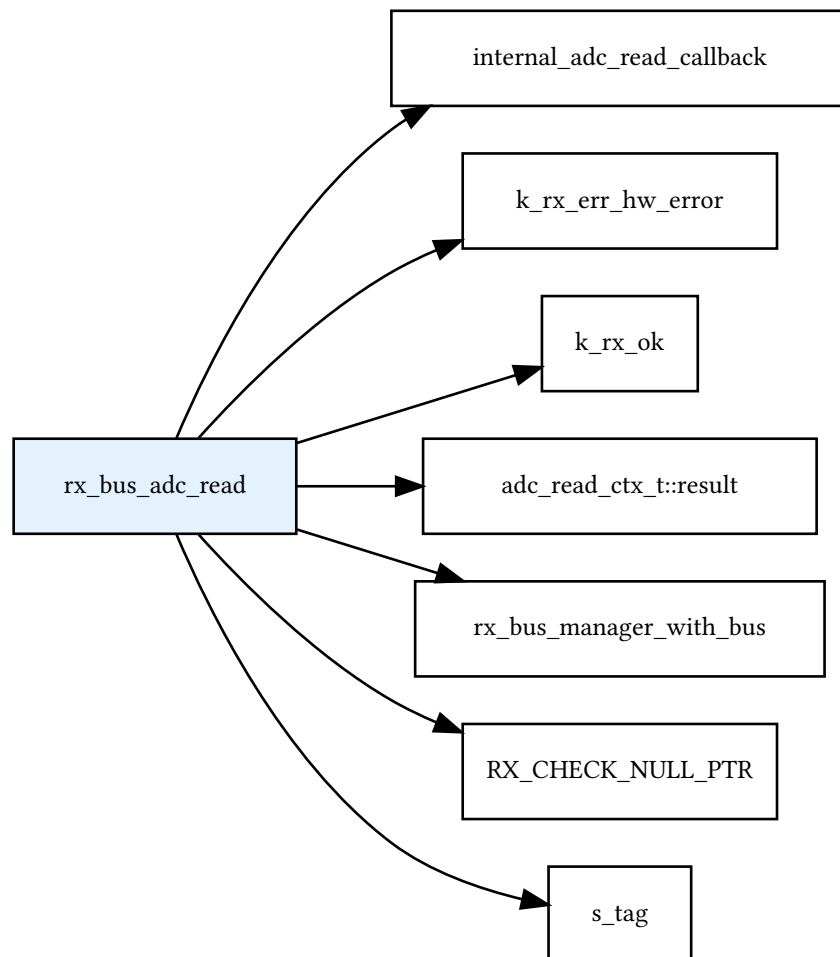


Figure 42: Call graph for rx_bus_adc_read

_Defined in libs/rx_bus/src/rx_bus_adc.c:809_

Since Version 1.0.0

—

rx_bus_adc_read_voltage_mv

```
rx_err_t rx_bus_adc_read_voltage_mv(rx_bus_manager_t *manager, const char
*bus_name, uint32_t *voltage_mv)
```

Read ADC voltage in millivolts through bus manager.

Public API function to perform single ADC conversion and automatically convert to millivolts based on configured VREF. This is the preferred function for most analog voltage measurement applications. Delegates to bus manager which calls internal_adc_voltage_callback() with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, voltage_mv)
2. Create stack-allocated context with voltage_mv pointer
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context (voltage_mv already updated by callback)
5. Return result to caller

The conversion uses VREF from bus configuration (set in rx_bus_register).

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized Must have ADC bus registered and initialized
in	bus_name	Name of the ADC bus Must match registered and initialized bus
out	voltage_mv	Pointer to store voltage in millivolts Range: 0 to VREF_mV (typically 0-3300 mV) Resolution: 0.805 mV @ 3.3V VREF, 12-bit Valid only if k_rx_ok returned

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, voltage_mv contains reading in millivolts
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Conversion or mutex timeout

Pre: Bus must be initialized via rx_bus_adc_init()

Pre: All pointers must be non-NULL

Pre: voltage_mv must point to valid uint32_t

Pre: Bus config must have valid vref_mv value

Post: *voltage_mv contains input voltage in millivolts (if k_rx_ok)

Post: ADC ready for next conversion

Note: Thread-safe via bus manager mutex

Note: Blocking call - waits for conversion + calculation (12 us)

Note: Stack usage: 16 bytes for context

Note: Execution time: 12 us

Note: Preferred over rx_bus_adc_read() for direct voltage measurement

Warning: voltage_mv undefined if return != k_rx_ok

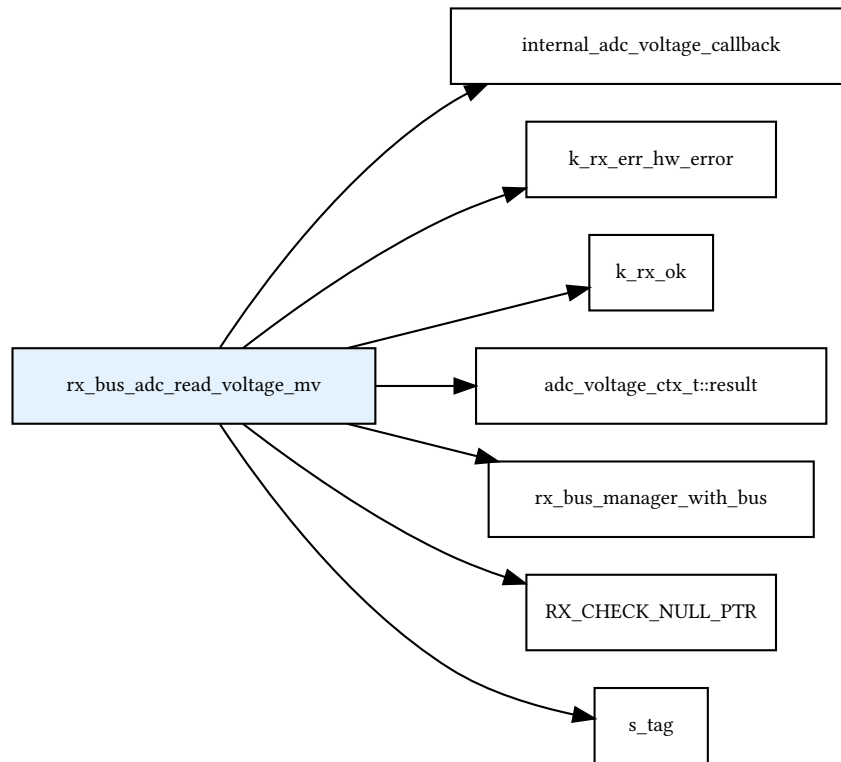
Warning: Analog input must not exceed VREFH

Warning: External voltage dividers need separate scaling in application

Example - Motor Current:: uint32_tipropi_mv=0;
 rx_err_t err=rx_bus_adc_read_voltage_mv(&manager,"motor_current",&ipropi_mv);
 if(err==k_rx_ok){ uint32_t current_ma=(ipropi_mv*1000)/4990; if(current_ma>5000)
 { motor_emergency_stop(); } }

Example - Temperature Sensor:: uint32_tadc_mv=0;
 err=rx_bus_adc_read_voltage_mv(&manager,"temp_sensor",&adc_mv); if(err==k_rx_ok)
 { int32_t temp_c=((int32_t)adc_mv-500)/10; if(temp_c>80){ rx_log_warn("TEMP","Hightemperature:
 %dC",temp_c); } }

See also: rx_bus_adc.h Complete API documentation, internal_adc_voltage_callback()
 Implementation callback, rx_bus_adc_read() Read raw ADC value without scaling

Figure 43: Call graph for `rx_bus_adc_read_voltage_mv`

_Defined in `libs/rx\_bus/src/rx\_bus\_adc.c:911_`

Since Version 1.0.0

—

rx_bus_config.c

Bus Configuration Creation Helpers - Static Initialization Pattern.

Includes:

- `"rx_bus_config.h"`
- `<string.h>`
- `"rx_check.h"`
- `"rx_log.h"`
- `"rx_port_constants.h"`

rx_port_t

```
typedef uint8_t rx_port_t
```

Port number type for RX72N GPIO port identification.

Aliases `uint8_t` to provide semantic clarity when a value represents an RX72N port number (0 through Port J = 10). Values are extracted from `rx_port_pin_t` using `rx_port_from_pin()`.

Since Version 1.0.0

—

rx_pin_t

```
typedef uint8_t rx_pin_t
```

Pin number type for RX72N GPIO pin identification within a port.

Aliases `uint8_t` to provide semantic clarity when a value represents a pin number within a port (0 through `k_rx_pin_max = 7`). Values are extracted from `rx_port_pin_t` using `rx_pin_from_pin()`.

Since Version 1.0.0

—

s_tag

```
const char s_tag[]
```

Logging tag for all `rx_bus_config` module log messages.

Note: Read-only; must never be modified at runtime

Warning: Direct modification would corrupt all log output from this module

Since Version 1.0.0

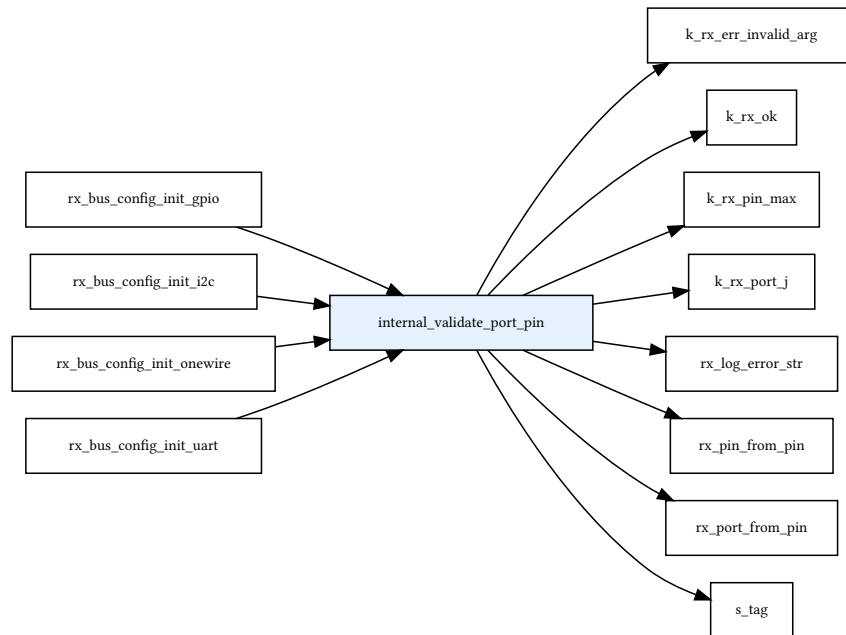
—

internal_validate_port_pin

```
static rx_err_t internal_validate_port_pin(const rx_port_pin_t pin, const char
*context_tag)
```

Validate port and pin numbers for GPIO configuration.

Internal helper that validates a combined port/pin value against RX72N hardware limits. Extracts port and pin numbers using type-safe accessor functions and checks against maximum values.

Figure 44: Call graph for `internal_validate_port_pin`

_Defined in `libs/rx\_bus/src/rx\_bus\_config.c:287_`

rx_bus_config_init_gpio

```
rx_err_t rx_bus_config_init_gpio(rx_bus_config_t *config, const char *name,
rx_port_pin_t pin)
```

Initialize GPIO bus configuration structure.

Initialize GPIO bus configuration for single-pin digital I/O.

Factory function to create a GPIO bus configuration with static allocation pattern. Validates pin number, zero-initializes structure, and sets all required fields for GPIO digital I/O operations.

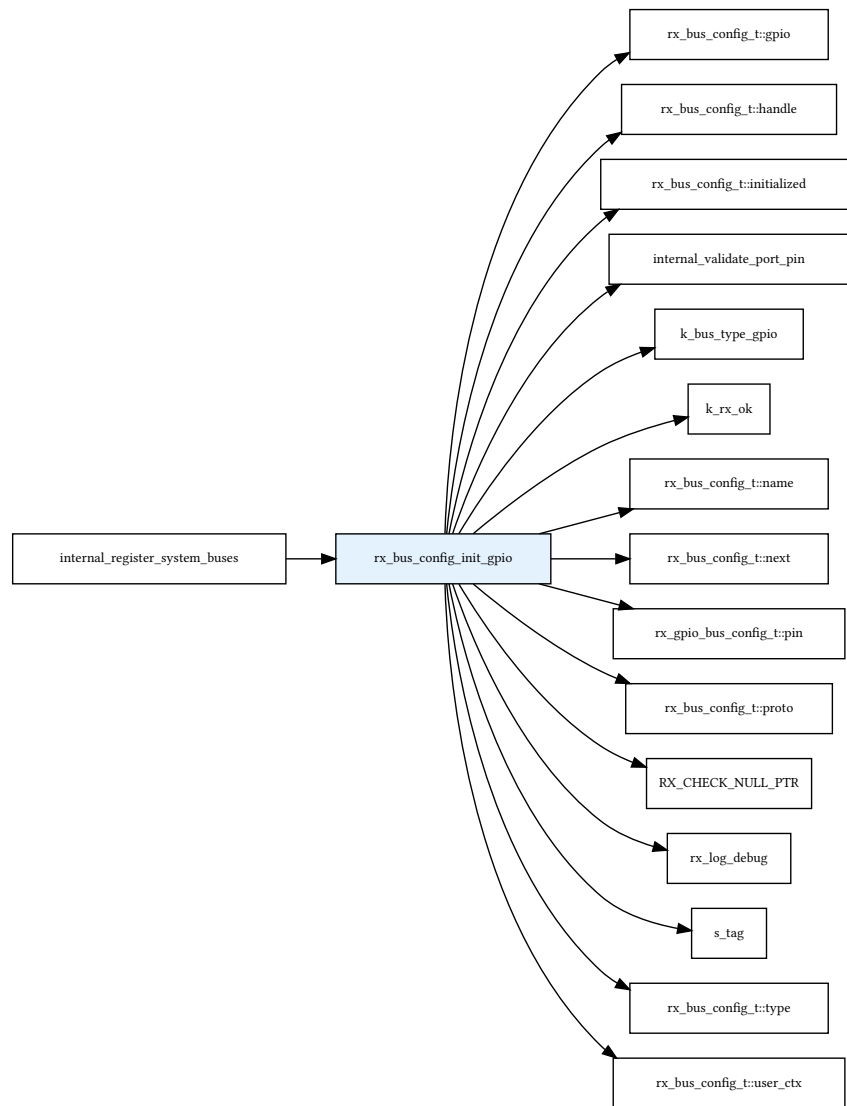


Figure 45: Call graph for rx_bus_config_init_gpio

_Defined in libs/rx_bus/src/rx_bus_config.c:435_

rx_bus_config_init_adc

```
rx_err_t rx_bus_config_init_adc(rx_bus_config_t *config, const char *name, const
uint8_t unit, const uint8_t channel, const uint8_t bits)
```

Initialize ADC (Analog-to-Digital Converter) bus configuration structure.

Initialize ADC bus configuration for analog-to-digital conversion.

Factory function to create an ADC bus configuration for voltage/current sensing operations. Validates ADC unit, channel, and resolution parameters before initializing the configuration structure.



Figure 46: Call graph for `rx_bus_config_init_adc`

_Defined in libs/rx_bus/src/rx_bus_config.c:651_

—

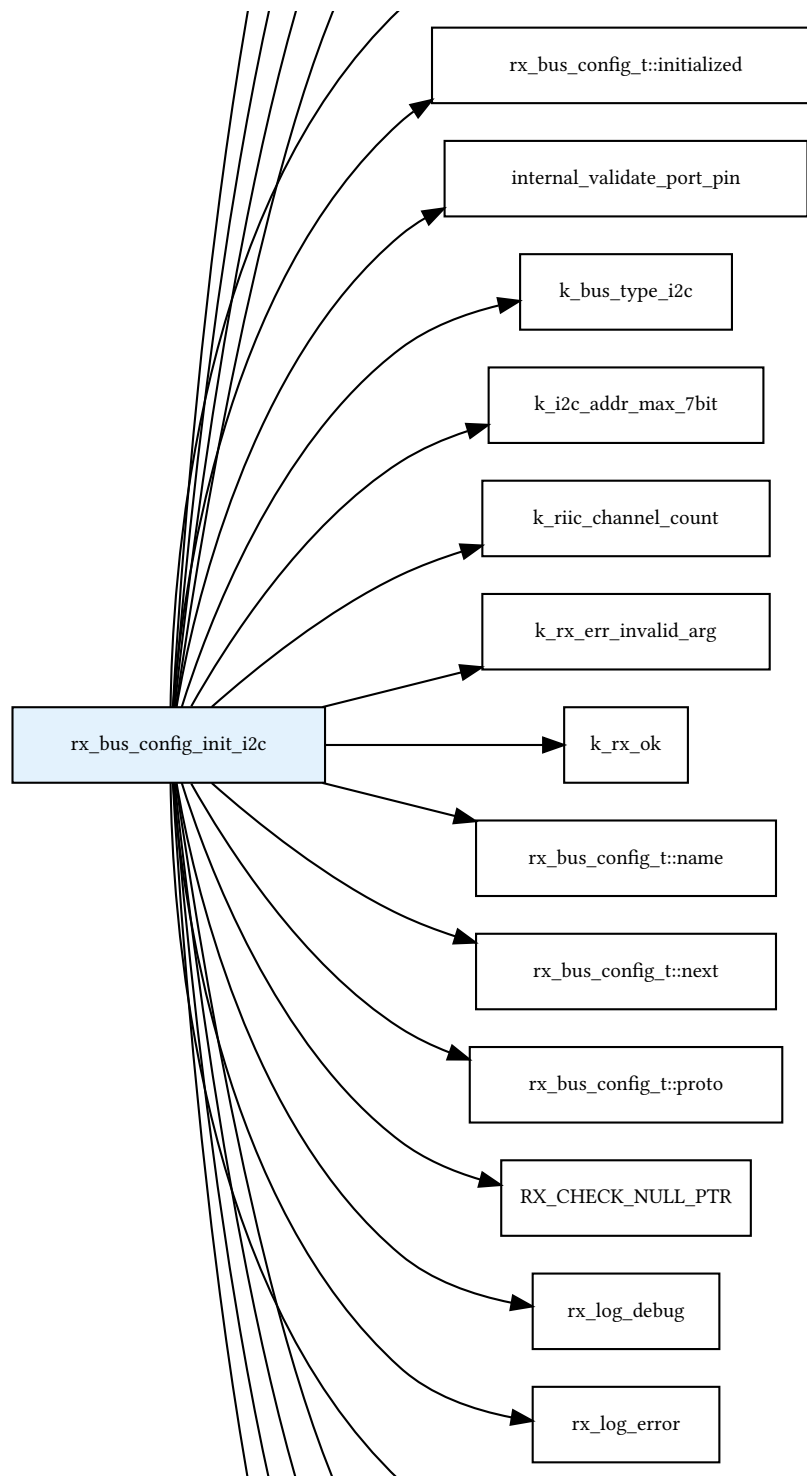
rx_bus_config_init_i2c

```
rx_err_t rx_bus_config_init_i2c(rx_bus_config_t *config, const char *name, const
uint8_t channel, const uint8_t device_addr, const rx_port_pin_t sda_pin, const
rx_port_pin_t scl_pin, const uint32_t frequency_hz)
```

Initialize I2C (Inter-Integrated Circuit) bus configuration structure.

Initialize I2C bus configuration for inter-integrated circuit communication.

Factory function to create an I2C bus configuration for communication with peripheral devices (sensors, EEPROMs, DACs, etc.). Validates channel, device address, pin assignments, and frequency parameters before initialization.

Figure 47: Call graph for `rx_bus_config_init_i2c`

_Defined in libs/rx_bus/src/rx_bus_config.c:907_

—

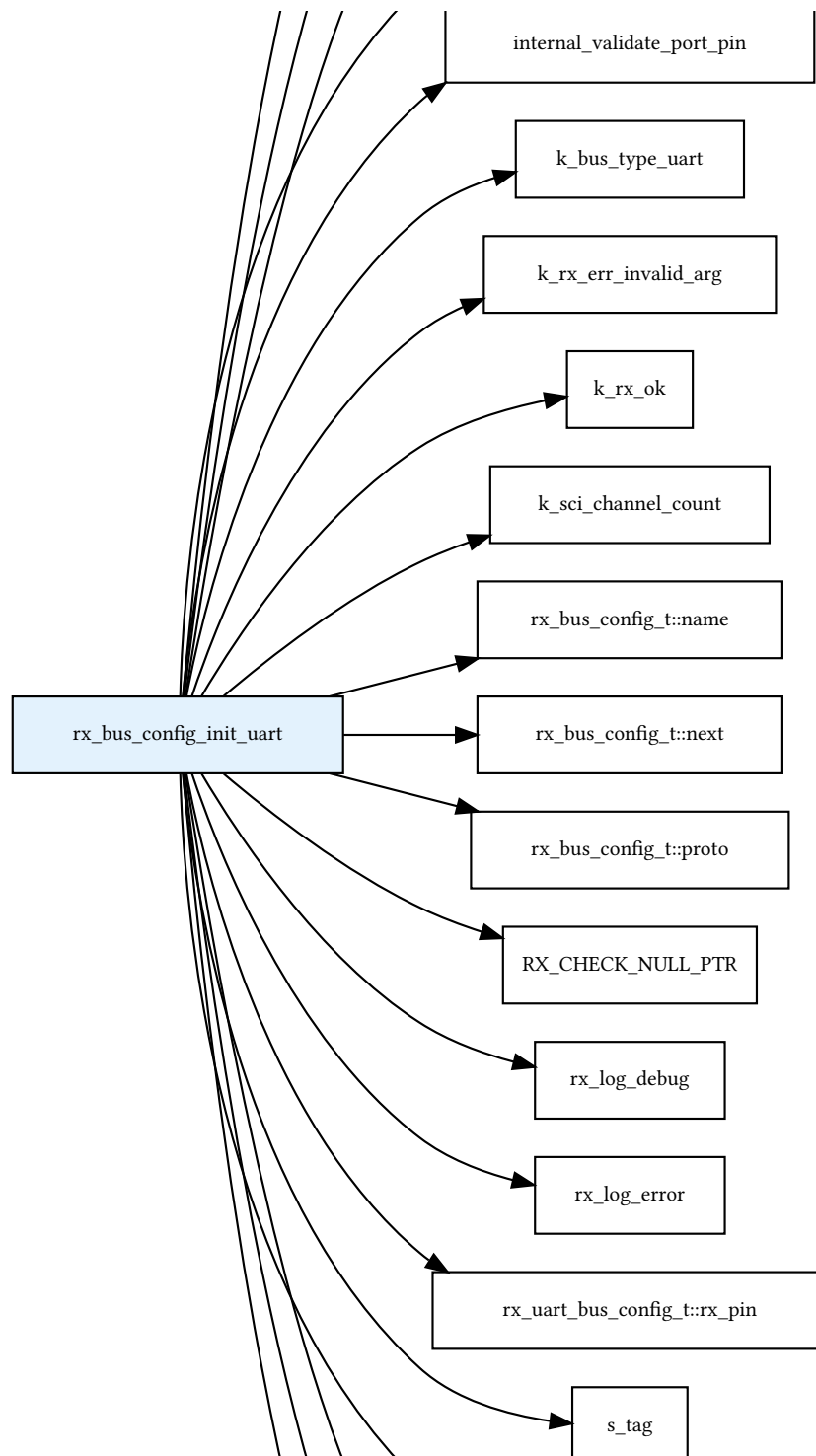
rx_bus_config_init_uart

```
rx_err_t rx_bus_config_init_uart(rx_bus_config_t *config, const char *name, const
uint8_t channel, const rx_port_pin_t tx_pin, const rx_port_pin_t rx_pin, const
uint32_t baudrate)
```

Initialize UART (Universal Asynchronous Receiver/Transmitter) bus configuration.

Initialize UART bus configuration for serial communication.

Factory function to create a UART bus configuration for serial communication with external devices, debugging, sensor interfaces, and wireless modules. Validates SCI channel, TX/RX pins, and baud rate before initialization.

Figure 48: Call graph for `rx_bus_config_init_uart`

_Defined in libs/rx_bus/src/rx_bus_config.c:1190_
—

rx_bus_config_init_onewire

```
rx_err_t rx_bus_config_init_onewire(rx_bus_config_t *config, const char *name,  
rx_port_pin_t pin)
```

Initialize 1-Wire bus configuration structure.

Initialize OneWire (1-Wire) bus configuration.

Factory function to create a 1-Wire bus configuration for Dallas/Maxim 1-Wire devices (DS18B20 temperature sensors, iButton authentication, etc.). 1-Wire is a single-wire bidirectional communication protocol requiring precise timing.

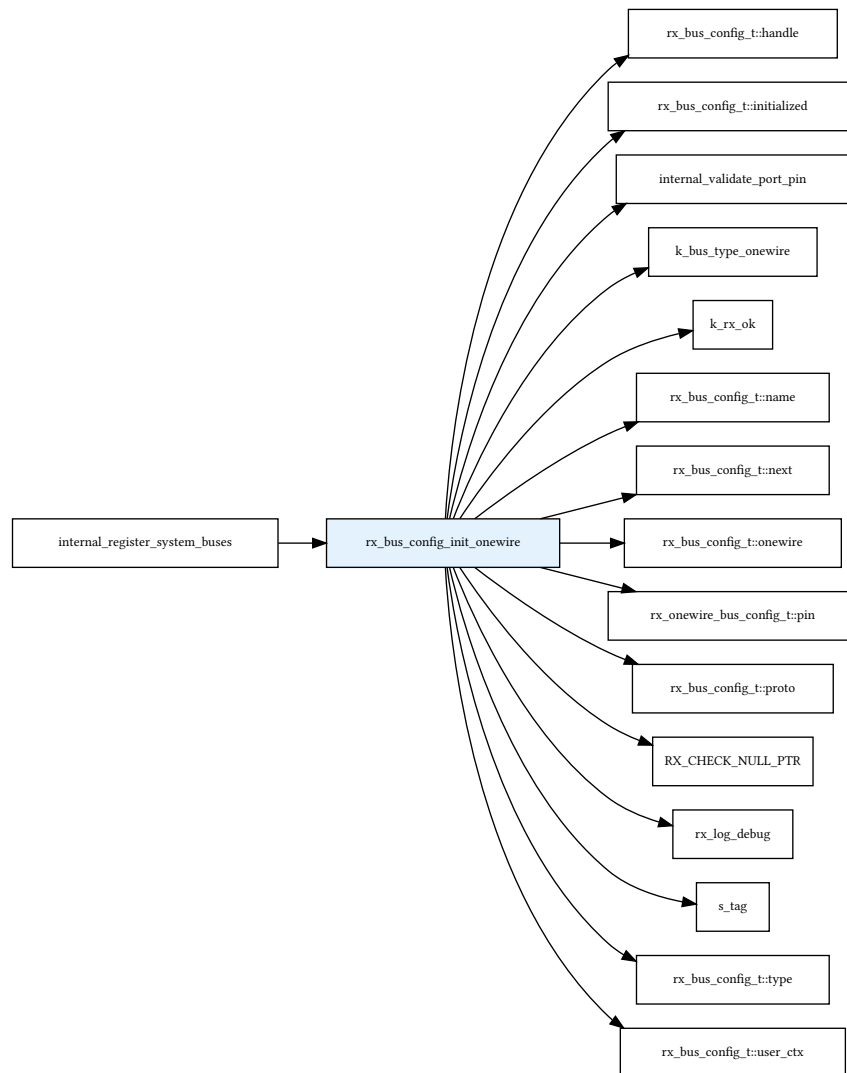


Figure 49: Call graph for rx_bus_config_init_owewire

_Defined in libs/rx_bus/src/rx_bus_config.c:1483_

rx_bus_gpio.c

GPIO Bus Abstraction Implementation for Renesas RX72N.

Production-quality GPIO bus abstraction that wraps low-level PORT hardware with high-level bus manager API providing thread-safe operations, pin conflict detection, and consistent error handling across all bus types.

Includes:

- "rx_bus_gpio.h"

- "hardware.h"
- "rx_bus_command.h"
- "rx_check.h"
- "rx_log.h"

s_tag

```
const char s_tag[]
```

Logging tag for all rx_bus_gpio module log messages.

Note: Read-only; must never be modified at runtime

Warning: Direct modification would corrupt all log output from this module

Since Version 1.0.0

—

internal_gpio_init_callback

```
static rx_err_t internal_gpio_init_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for GPIO pin initialization.

Initializes a GPIO pin for input or output operation. Called by bus manager via rx_bus_manager_with_bus() callback mechanism. Configures pin direction (PDR register) and performs post-initialization verification read.

Algorithm steps:

1. Cast user_ctx to gpio_init_ctx_t*
2. Validate bus type is k_bus_type_gpio
3. Call gpio_set_output() or gpio_set_input() based on ctx->output
4. Verify GPIO responsive via test read (PIDR)
5. Mark bus as initialized
6. Set ctx->result and return

Bus type remains k_bus_type_gpio

Failure leaves bus in uninitialized state

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration structure Must have type = k_bus_type_gpio proto.gpio.pin must be valid PORT/pin combination initialized flag set to true on success
inout	user_ctx	User context (gpio_init_ctx_t*) input: ctx->output specifies direction output: ctx->result contains operation result

Returns: rx_err_t Error code indicating result

Value	Description
-------	-------------

k_rx_ok	Success, GPIO initialized
k_rx_err_invalid_arg	Bus type is not GPIO
k_rx_err_hw_error	GPIO HAL initialization failed

Pre: bus_config pointer is valid (validated by bus manager)

Pre: user_ctx pointer is valid and points to gpio_init_ctx_t

Pre: Pin must be available (validated by pin validator)

Post: bus_config->initialized = true on success

Post: ctx->result contains operation result

Post: Pin configured as GPIO mode (PMR = 0)

Post: Pin direction set (PDR = output/input)

Note: Called from bus manager with mutex held (thread-safe)

Note: Post-init verification read may fail on output-only pins (acceptable)

Note: Execution time: 15 us (includes pin validator check)

Warning: Do not call directly - use rx_bus_gpio_init() instead] **See also:** rx_bus_gpio_init()
Public API that calls this callback, gpio_set_output() HAL function for output configuration, gpio_set_input() HAL function for input configuration

Figure 50: Call graph for `internal_gpio_init_callback`

Defined in `libs/rx\_bus/src/rx\_bus\_gpio.c:358`

Since Version 1.0.0

internal_gpio_write_callback

```
static rx_err_t internal_gpio_write_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for GPIO output write operation.

Writes a digital value (high/low) to a GPIO output pin. Called by bus manager via rx_bus_manager_with_bus(). Writes to PORT output data register (PODR) and performs optional post-write verification readback.

Algorithm steps:

1. Cast user_ctx to gpio_write_ctx_t*
2. Validate bus initialized
3. Call gpio_write_high() or gpio_write_low() based on ctx->value
4. Verify written value via readback (PIDR)
5. Set ctx->result and return

Bus remains initialized

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration structure Must be initialized (initialized flag = true) Must be configured as output (PDR = 1) proto.gpio.pin specifies PORT/pin to write
inout	user_ctx	User context (gpio_write_ctx_t*) input: ctx->value specifies output state output: ctx->result contains operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, output state changed
k_rx_err_invalid_state	Bus not initialized
k_rx_err_hw_error	GPIO write failed (hardware fault)

Pre: bus_config->initialized must be true

Pre: Pin must be configured as output (not validated - caller responsibility)

Pre: user_ctx points to valid gpio_write_ctx_t

Post: ctx->result contains operation result

Post: Pin output state set to ctx->value

Post: Physical pin voltage reflects state within 10-20 ns

Note: Called from bus manager with mutex held (thread-safe)

Note: Post-write readback may mismatch on some hardware (acceptable warning)

Note: Execution time: 2 us (including readback verification)

Warning: Readback verification may fail on output-only pins

Warning: Readback mismatch logged as warning but operation succeeds] **See also:**
`rx_bus_gpio_write()` Public API that calls this callback, `gpio_write_high()` HAL
function to set pin high, `gpio_write_low()` HAL function to set pin low

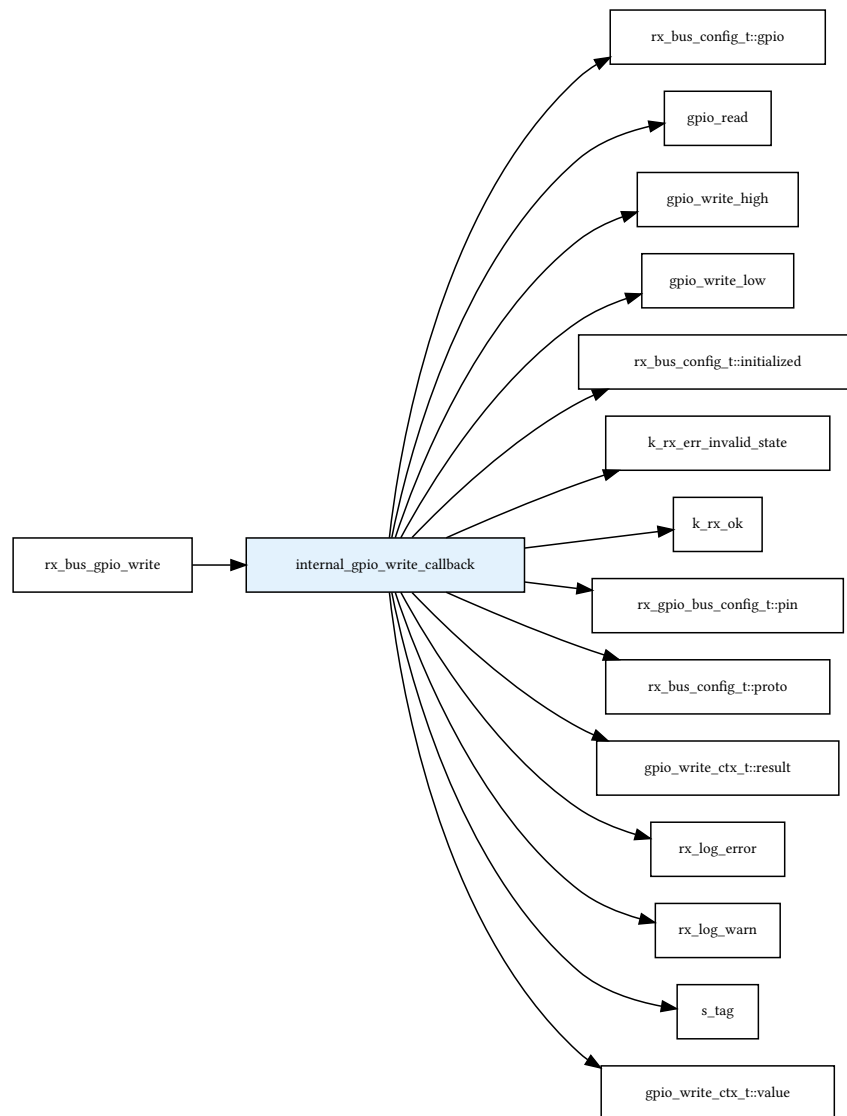


Figure 51: Call graph for internal_gpio_write_callback

_Defined in `libs/rx\bus/src/rx\bus\_gpio.c:445_`

Since Version 1.0.0

—

internal_gpio_read_callback

```
static rx_err_t internal_gpio_read_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for GPIO input read operation.

Reads the current state of a GPIO pin (works for both input and output pins). Called by bus manager via `rx_bus_manager_with_bus()`. Reads from PORT input data register (PIDR), which reflects actual pin state regardless of direction.

Algorithm steps:

1. Validate all pointers (`bus_config`, `user_ctx`, `ctx->value`)
2. Cast `user_ctx` to `gpio_read_ctx_t*`
3. Validate bus initialized
4. Call `gpio_read()` to read PIDR register
5. Set `*ctx->value` = pin state
6. Set `ctx->result` and return

Bus remains initialized

Pin state unchanged

`RX_CHECK_NULL_PTR` validates pointers early (NASA Rule 5)

Parameters:

Direction	Name	Description
in	<code>bus_config</code>	Bus configuration structure Must be initialized (initialized flag = true) Can be input or output (read works for both) <code>proto.gpio.pin</code> specifies PORT/pin to read
inout	<code>user_ctx</code>	User context (<code>gpio_read_ctx_t*</code>) input: <code>ctx->value</code> points to output bool output: <code>*ctx->value</code> set to pin state output: <code>ctx->result</code> contains operation result

Returns: `rx_err_t` Error code indicating result

Value	Description
<code>k_rx_ok</code>	Success, value read and stored
<code>k_rx_err_null_ptr</code>	nullptr in <code>bus_config</code> , <code>user_ctx</code> , or <code>ctx->value</code>
<code>k_rx_err_invalid_state</code>	Bus not initialized
<code>k_rx_err_hw_error</code>	GPIO read failed (hardware fault)

Pre: `bus_config` must be non-NULL

Pre: `bus_config->initialized` must be true

Pre: `user_ctx` must be non-NULL

Pre: `ctx->value` must be non-NULL and point to valid bool

Post: `*ctx->value` contains pin state (true=high, false=low)

Post: ctx->result contains operation result

Post: Pin state unchanged (non-destructive read)

Note: Called from bus manager with mutex held (thread-safe)

Note: Works for both input and output pins (reads PIDR)

Note: Execution time: 2 us

Note: Floating inputs may read unpredictably (enable pull-up if needed)

Warning: ctx->value undefined if return != k_rx_ok] **See also:** rx_bus_gpio_read() Public API that calls this callback, gpio_read() HAL function to read pin state

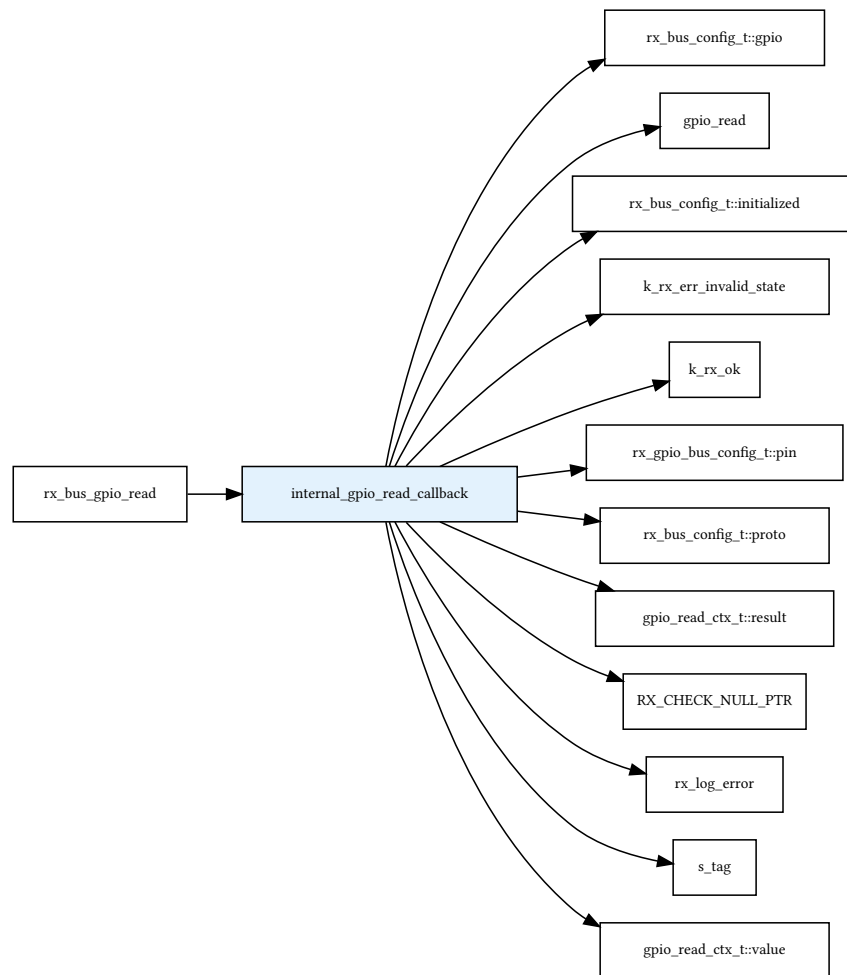


Figure 52: Call graph for internal_gpio_read_callback

Defined in `libs/rx\_bus/src/rx\_bus\_gpio.c:537`

Since Version 1.0.0

—

internal_gpio_toggle_callback

```
static rx_err_t internal_gpio_toggle_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for GPIO output toggle operation.

Inverts the current state of a GPIO output pin (high -> low, low -> high). Called by bus manager via `rx_bus_manager_with_bus()`. Performs atomic read-modify-write operation protected by mutex.

Algorithm steps:

1. Cast user_ctx to gpio_toggle_ctx_t*
2. Validate bus initialized
3. Read current state from PIDR (pre-toggle verification)
4. Call gpio_toggle() to invert output
5. Read new state from PIDR (post-toggle verification)
6. Verify state actually changed
7. Set ctx->result and return

Bus remains initialized

Read-modify-write is atomic only because mutex is held

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration structure Must be initialized (initialized flag = true) Must be configured as output (PDR = 1) proto.gpio.pin specifies PORT/pin to toggle
inout	user_ctx	User context (gpio_toggle_ctx_t*) output: ctx->result contains operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, pin state inverted
k_rx_err_invalid_state	Bus not initialized or pin is input
k_rx_err_hw_error	GPIO toggle failed (hardware fault)

Pre: bus_config->initialized must be true

Pre: Pin must be configured as output (not validated - caller responsibility)

Pre: user_ctx points to valid gpio_toggle_ctx_t

Post: ctx->result contains operation result

Post: Pin output state inverted (high <-> low)

Post: Physical pin reflects new state within 10-20 ns

Note: Called from bus manager with mutex held (atomic read-modify-write)

Note: Pre-toggle read may fail on output-only pins (logged as warning)

Note: Post-toggle verification may fail on some hardware (logged as warning)

Note: Execution time: 3 us (read + toggle + read)

Warning: Pre/post-toggle verification may fail on output-only pins

Warning: Verification failures logged but operation proceeds

Atomic Operation:: The toggle is atomic with respect to other bus operations (mutex held):
`current=read_pin(); write_pin(!current); verify=read_pin(); assert(verify!=current);`

See also: `rx_bus_gpio_toggle()` Public API that calls this callback, `gpio_toggle()` HAL function to invert output state, `gpio_read()` HAL function for verification reads

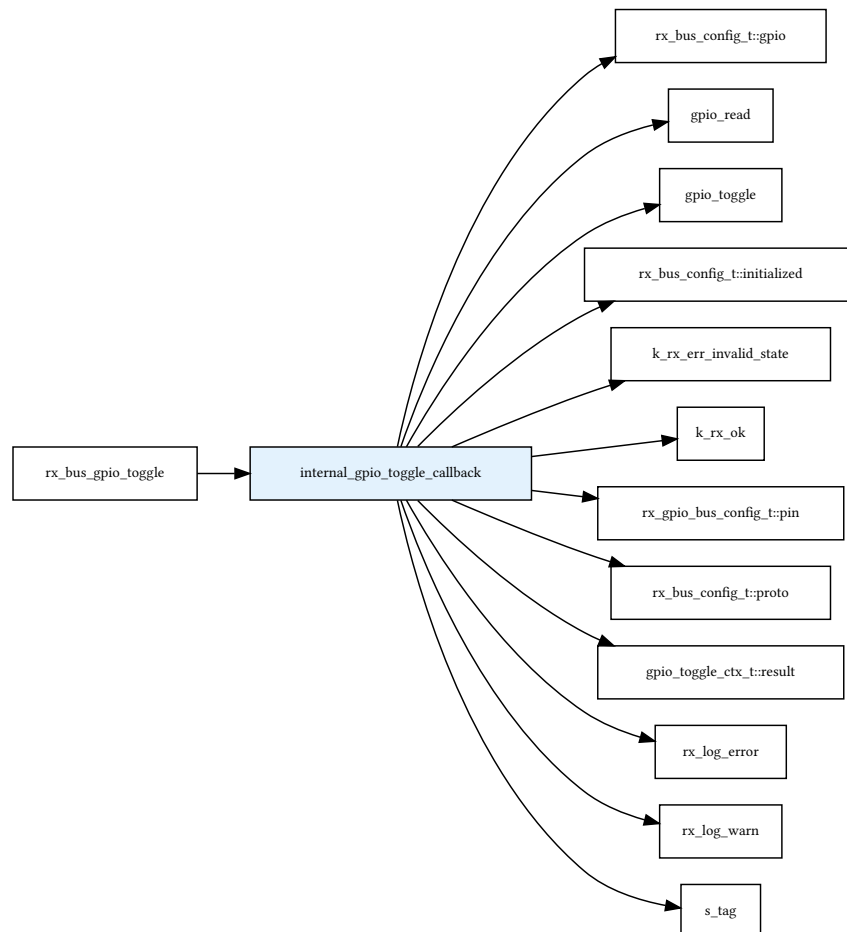


Figure 53: Call graph for `internal_gpio_toggle_callback`

_Defined in `libs/rx\_bus/src/rx\_bus\_gpio.c:628_`

Since Version 1.0.0

—

rx_bus_gpio_init

```
rx_err_t rx_bus_gpio_init(rx_bus_manager_t *manager, const char *bus_name, bool
output)
```

Initialize GPIO pin through bus manager.

Public API function to initialize a GPIO pin for digital input or output. Delegates to bus manager which calls internal_gpio_init_callback() with mutex protection and bus lookup.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context with operation parameters
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context
5. Return result to caller

This function demonstrates the callback pattern for bus operations:

- Prepares context on stack (zero allocation)
- Delegates to bus manager for mutex protection
- Callback executes with mutex held
- Results extracted after mutex released

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized via rx_bus_manager_init() Must have GPIO bus registered
in	bus_name	Name of the GPIO bus Must match registered bus name Null-terminated string
in	output	Pin direction true: Output mode (PDR = 1) false: Input mode (PDR = 0)

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, GPIO initialized
k_rx_err_null_ptr	nullptr in manager or bus_name
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_arg	Bus is not GPIO type
k_rx_err_timeout	Mutex timeout
k_rx_err_conflict	Pin already reserved

Pre: manager must be initialized

Pre: Bus must be registered with type `k_rx_bus_type_gpio`

Pre: Pin must be available (not reserved)

Post: Pin reserved in pin validator

Post: Pin configured as GPIO

Post: Pin direction set

Post: Bus marked as initialized

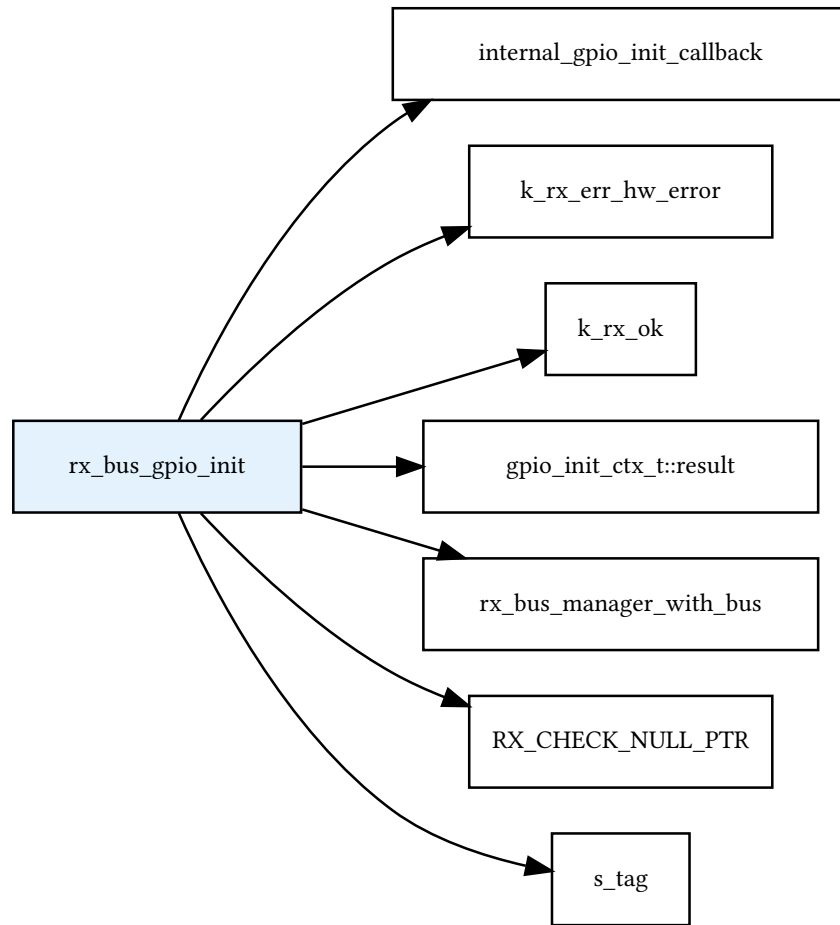
Note: Thread-safe via bus manager mutex

Note: Stack usage: 8 bytes for context

Note: Execution time: 15 us

Example:: `rx_err_t err = rx_bus_gpio_init(&manager, "led_red", true); if (err != k_rx_ok) { rx_log_error("APP", "Failed to init LED: %d", err); }`

See also: `rx_bus_gpio.h` Complete API documentation, `internal_gpio_init_callback()` Implementation callback

Figure 54: Call graph for `rx_bus_gpio_init`

_Defined in `libs/rx_bus/src/rx_bus_gpio.c:742_`

Since Version 1.0.0

—

rx_bus_gpio_write

```
rx_err_t rx_bus_gpio_write(rx_bus_manager_t *manager, const char *bus_name, bool
value)
```

Write digital value to GPIO output pin.

Write GPIO output through bus manager.

Public API function to set GPIO output state (high/low). Delegates to bus manager which calls `internal_gpio_write_callback()` with mutex protection.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context with value parameter
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context
5. Return result to caller

Output propagates to physical pin within 10-20 ns of function return.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized Must have GPIO bus registered and initialized
in	bus_name	Name of the GPIO bus Must match registered and initialized bus
in	value	Output state true: High (VDD - 3.3V) false: Low (GND - 0V)

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, output changed
k_rx_err_null_ptr	nullptr in manager or bus_name
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Mutex timeout

Pre: Bus must be initialized via rx_bus_gpio_init() with output=true

Pre: manager and bus_name must be non-NULL

Post: Pin output state set to value

Post: Physical pin voltage reflects state within 10-20 ns

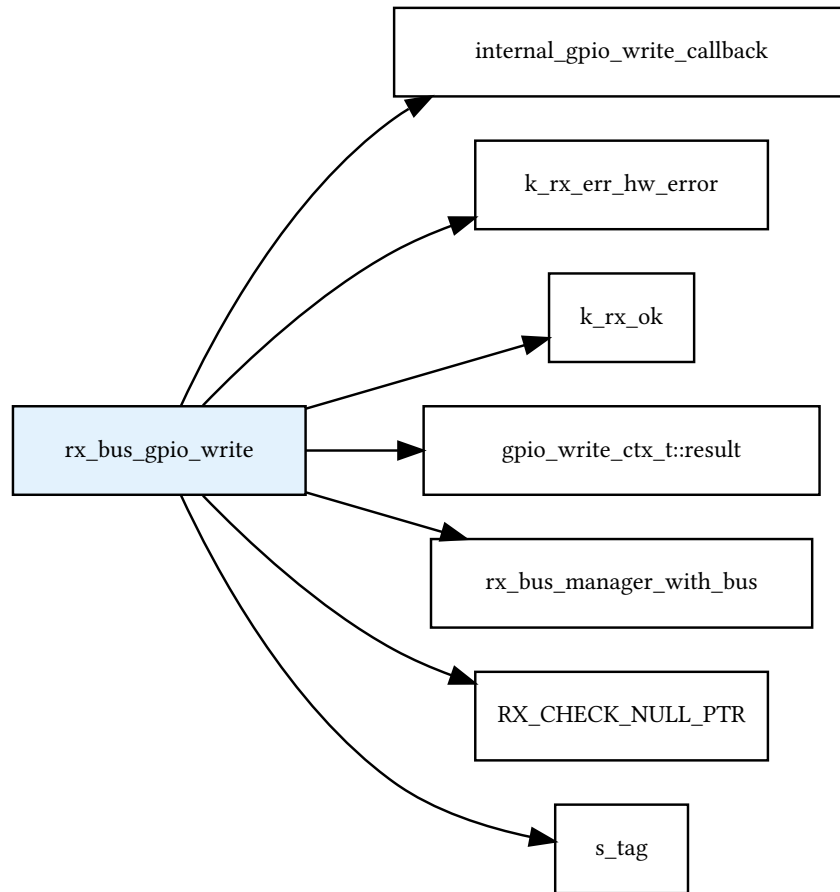
Note: Thread-safe via bus manager mutex

Note: Stack usage: 8 bytes for context

Note: Execution time: 2 us

Example:: rx_bus_gpio_write(&manager,"led_red",false);

See also: rx_bus_gpio.h Complete API documentation, internal_gpio_write_callback() Implementation callback

Figure 55: Call graph for `rx_bus_gpio_write`

_Defined in `libs/rx\_bus/src/rx\_bus\_gpio.c:812_`

Since Version 1.0.0

—

rx_bus_gpio_read

```
rx_err_t rx_bus_gpio_read(rx_bus_manager_t *manager, const char *bus_name, bool
*value)
```

Read digital value from GPIO pin.

Read GPIO input through bus manager.

Public API function to read GPIO pin state. Works for both input and output pins (reads actual pin state via PIDR register). Delegates to bus manager which calls `internal_gpio_read_callback()` with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, value)
2. Create stack-allocated context with value pointer
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context (value already updated by callback)
5. Return result to caller

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized Must have GPIO bus registered and initialized
in	bus_name	Name of the GPIO bus Must match registered and initialized bus
out	value	Pointer to store pin state true: Pin is high ($> 0.7 \cdot VDD$) false: Pin is low ($< 0.3 \cdot VDD$) Valid only if k_rx_ok returned

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, value contains pin state
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Mutex timeout

Pre: Bus must be initialized via rx_bus_gpio_init()

Pre: All pointers must be non-NULL

Pre: value must point to valid bool

Post: *value contains pin state (if k_rx_ok)

Post: Pin state unchanged (non-destructive read)

Note: Thread-safe via bus manager mutex

Note: Works for both input and output pins

Note: Stack usage: 16 bytes for context

Note: Execution time: 2 us

Warning: value undefined if return != k_rx_ok

Warning: Floating inputs read unpredictably

Example:: bool button_state; rx_err_t err = rx_bus_gpio_read(&manager, "button", &button_state);
if(err != k_rx_ok && !button_state){ }

See also: rx_bus_gpio.h Complete API documentation, internal_gpio_read_callback()
Implementation callback

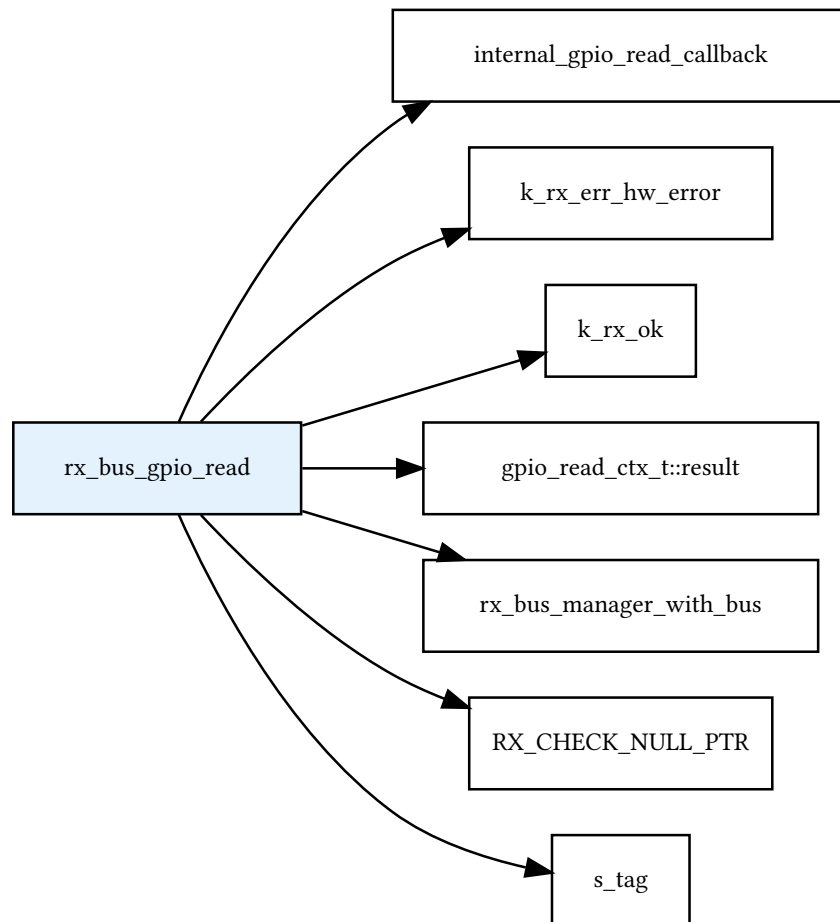


Figure 56: Call graph for rx_bus_gpio_read

_Defined in libs/rx_bus/src/rx_bus_gpio.c:890_

Since Version 1.0.0

—

rx_bus_gpio_toggle

```
rx_err_t rx_bus_gpio_toggle(rx_bus_manager_t *manager, const char *bus_name)
```

Toggle GPIO output pin state (high <-> low).

Toggle GPIO output through bus manager.

Public API function to invert GPIO output state. Performs atomic read-modify-write operation. Delegates to bus manager which calls internal_gpio_toggle_callback() with mutex protection.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context (no parameters needed)
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context
5. Return result to caller

Atomic operation: mutex ensures no other thread can modify pin between read and write operations.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance Must be initialized Must have GPIO bus registered and initialized
in	bus_name	Name of the GPIO bus Must match registered and initialized bus Must be configured as output

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, pin state inverted
k_rx_err_null_ptr	nullptr in manager or bus_name
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized or pin is input
k_rx_err_timeout	Mutex timeout

Pre: Bus must be initialized via rx_bus_gpio_init() with output=true

Pre: manager and bus_name must be non-NULL

Pre: Pin must be configured as output

Post: Pin output state inverted (high -> low or low -> high)

Post: Physical pin reflects new state within 10-20 ns

Note: Thread-safe - atomic read-modify-write via mutex

Note: Stack usage: 4 bytes for context

Note: Execution time: 3 us (read + write)

Warning: Attempting to toggle input pin returns k_rx_err_invalid_state

Performance:: Execution time: 3 us (slower than write due to read) Maximum toggle rate: 330 kHz (theoretical) Practical toggle rate: < 10 kHz (recommended)

Example - LED Blinking:: while(1){ rx_bus_gpio_toggle(&manager,"led_status");
tx_thread_sleep(50); }

See also: rx_bus_gpio.h Complete API documentation, internal_gpio_toggle_callback()
Implementation callback, rx_bus_gpio_write() Use this if new state is known (faster)

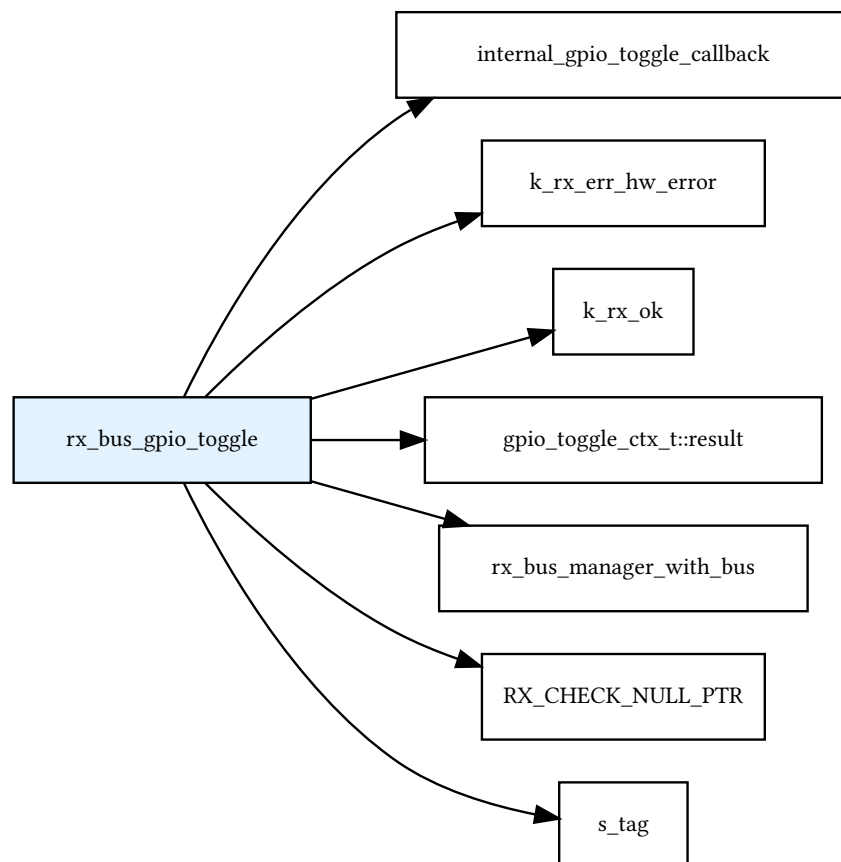


Figure 57: Call graph for rx_bus_gpio_toggle

_Defined in `libs/rx\_bus/src/rx\_bus\_gpio.c:973_`

Since Version 1.0.0

—

rx_bus_i2c.c

I2C Bus Abstraction Implementation - Callback-Based Thread-Safe Operations.

Includes:

- `"rx_bus_i2c.h"`
- `"hardware.h"`
- `"rx_check.h"`
- `"rx_log.h"`

i2c_validation_constants_t

I2C validation constants.

Value	Name	Description
= 0	<code>k_i2c_length_zero</code>	Zero length value for comparisons

—

s_tag

```
const char s_tag[]
```

Logging tag for all `rx_bus_i2c` module log messages.

Note: Read-only; must never be modified at runtime

Warning: Direct modification would corrupt all log output from this module

Since Version 1.0.0

—

internal_i2c_init_callback

```
static rx_err_t internal_i2c_init_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for I2C peripheral initialization.

Initializes the RX72N RIIC peripheral for the I2C bus described by `bus_config`. Called by the bus manager via `rx_bus_manager_with_bus()` with the bus mutex held. Validates bus type, configures the RIIC channel at the requested frequency, and marks the bus as initialized on success.

Algorithm steps:

1. Cast `user_ctx` to `i2c_init_ctx_t*`
2. Validate bus type is `k_bus_type_i2c`
3. Extract RIIC channel from `bus_config->proto.i2c.channel`
4. Call `riic_init()` with channel and frequency_hz
5. Warn if `device_addr` exceeds 7-bit maximum (non-fatal)

6. Set bus_config->initialized = true
7. Store k_rx_ok in ctx->result and return

Bus type remains k_bus_type_i2c throughout the callback

Repeated calls to riic_init() on an already-initialized channel may cause glitches on the I2C bus

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration structure type must be k_bus_type_i2c proto.i2c.channel specifies the RIIC channel to initialize proto.i2c.frequency_hz specifies the clock frequency initialized flag set to true on success
inout	user_ctx	User context (i2c_init_ctx_t*) output: ctx->result contains the operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	RIIC peripheral initialized and bus marked ready
k_rx_err_invalid_arg	Bus type is not k_bus_type_i2c
k_rx_err_hw_error	RIIC HAL initialization failed (check pins/clocks)

Pre: bus_config must be non-NULL (validated by bus manager before dispatch)

Pre: user_ctx must be non-NULL and point to a valid i2c_init_ctx_t

Post: bus_config->initialized == true on k_rx_ok

Post: ctx->result contains the operation outcome

Note: Not thread-safe; called from bus manager with bus lock held

Warning: Do not call directly - use rx_bus_i2c_init() instead

Performance:: Execution time: 50 us @ 240 MHz (includes RIIC peripheral setup)

Example:: i2c_init_ctx_tctx={.result=k_rx_err_invalid_state};
rx_err_t err=rx_bus_manager_with_bus(manager,"imu_i2c", internal_i2c_init_callback,&ctx);
if(err==k_rx_ok&&ctx.result==k_rx_ok){ }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions, 2 postconditions Rule 4: Function is 20 lines (well under 60 limit)

See also: rx_bus_i2c_init() Public API that invokes this callback, i2c_init_ctx_t Context structure passed as user_ctx, riic_init() Underlying RIIC HAL initialization function

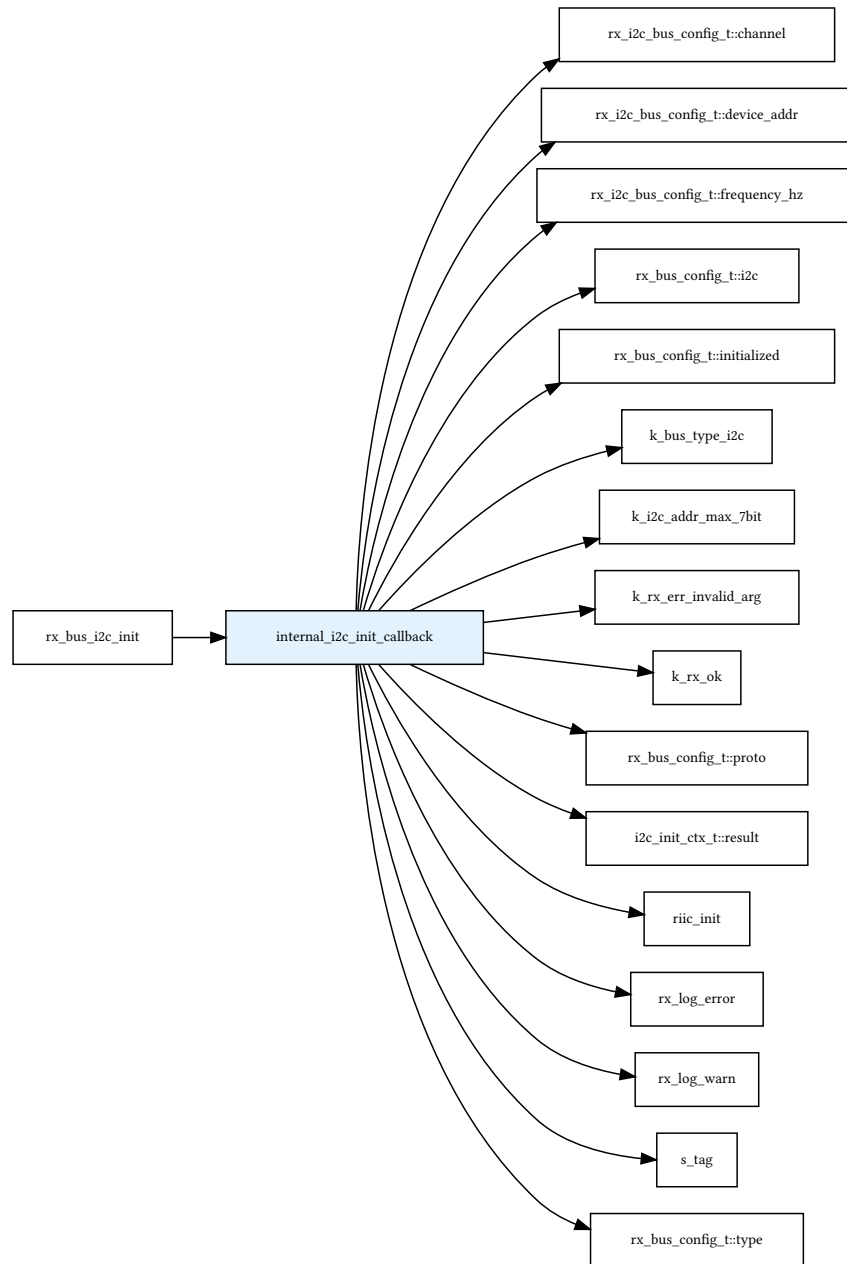


Figure 58: Call graph for internal_i2c_init_callback

Defined in `libs/rx_bus/src/rx_bus_i2c.c:474`

Since Version 1.0.0

—

internal_i2c_write_callback

```
static rx_err_t internal_i2c_write_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for I2C write transfer.

Internal callback implementing a raw I2C write transfer. Validates the bus is initialized and the data pointer is non-NULL when length > 0, then performs the I2C write via the RIIC HAL. Stores the operation result in the context structure for retrieval by the caller.

Algorithm steps:

1. Cast user_ctx to i2c_write_ctx_t*
2. Validate bus is initialized (check bus_config->initialized)
3. Validate ctx->data non-NULL when ctx->length > 0
4. Extract riic_channel and device_addr from bus_config->proto.i2c
5. Call riic_write() with channel, device address, data, and length
6. Set ctx->result to operation outcome and return

bus_config->initialized remains unchanged by this function

NAK from the device is reported as k_rx_err_hw_error; verify that the device address in bus_config matches the physical device

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration structure initialized must be true proto.i2c.channel specifies the RIIC channel proto.i2c.device_addr specifies the 7-bit target address
inout	user_ctx	User context (i2c_write_ctx_t*) input: ctx->data points to bytes to transmit (may be NULL when length=0) input: ctx->length specifies number of bytes to write output: ctx->result contains the operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, all data bytes transmitted and ACKed by device
k_rx_err_invalid_state	Bus not initialized (call rx_bus_i2c_init() first)
k_rx_err_invalid_arg	ctx->data is nullptr and ctx->length > 0
k_rx_err_hw_error	RIIC write failed (NAK received, bus error, or timeout)

Pre: bus_config->initialized must be true

Pre: ctx->data must be non-NULL when ctx->length > 0

Post: ctx->result contains the operation outcome

Post: ctx->length bytes transmitted on the I2C bus on k_rx_ok

Note: Not thread-safe; called from bus manager with bus lock held

Warning: Zero-length writes (length=0) are permitted and generate an I2C address probe (START + ADDR+W + ACK check + STOP)

Performance:: Execution time: 2 us overhead + 22.5 us per byte @ 400 kHz Fast mode

Example::

```
constuint8_t reg_data[]={0x01,0x23};
i2c_write_ctx_t ctx={ .data=reg_data, .length=sizeof(reg_data), .result=k_rx_err_invalid_state, };
rx_err_t err=rx_bus_manager_with_bus(manager,bus_name, internal_i2c_write_callback,&ctx);
if(err!=k_rx_ok&&ctx.result!=k_rx_ok){ }
```

NASA Power of 10 Compliance:: Rule 5: 2 preconditions, 2 postconditions Rule 4: Function is 20 lines (well under 60 limit) Rule 7: All riic_write() return values checked

See also: rx_bus_i2c_write() Public API that invokes this callback, i2c_write_ctx_t Context structure passed as user_ctx, riic_write() Underlying RIIC HAL write function



Figure 59: Call graph for internal_i2c_write_callback

_Defined in `libs/rx\_bus/src/rx\_bus\_i2c.c:583_`

Since Version 1.0.0

—

internal_i2c_read_callback

```
static rx_err_t internal_i2c_read_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Internal callback for I2C read transfer.

Internal callback implementing a raw I2C read transfer. Validates the bus is initialized and the data pointer is non-NULL when length > 0, then performs the I2C read via the RIIC HAL. The received bytes are stored directly in the caller-supplied buffer.

Algorithm steps:

1. Cast user_ctx to i2c_read_ctx_t*
2. Validate bus is initialized (check bus_config->initialized)
3. Validate ctx->data non-NULL when ctx->length > 0
4. Extract riic_channel and device_addr from bus_config->proto.i2c
5. Call riic_read() with channel, device address, data buffer, and length
6. Set ctx->result to operation outcome and return

bus_config->initialized remains unchanged by this function

ctx->data contents are undefined if return value is not k_rx_ok; always check the return code before reading the buffer

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration structure initialized must be true proto.i2c.channel specifies the RIIC channel proto.i2c.device_addr specifies the 7-bit target address
inout	user_ctx	User context (i2c_read_ctx_t*) input: ctx->data points to receive buffer (may be NULL when length=0) input: ctx->length specifies number of bytes to read output: ctx->data buffer filled with received bytes on k_rx_ok output: ctx->result contains the operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, ctx->length bytes received into ctx->data buffer
k_rx_err_invalid_state	Bus not initialized (call rx_bus_i2c_init() first)
k_rx_err_invalid_arg	ctx->data is nullptr and ctx->length > 0
k_rx_err_hw_error	RIIC read failed (NAK received, bus error, or timeout)

Pre: bus_config->initialized must be true

Pre: ctx->data must be non-NULL and point to a buffer of at least ctx->length bytes

Post: ctx->result contains the operation outcome

Post: ctx->data buffer contains ctx->length received bytes on k_rx_ok

Note: Not thread-safe; called from bus manager with bus lock held

Warning: The receive buffer must remain valid for the entire duration of the callback; do not use temporary stack-allocated buffers

Performance:: Execution time: 2 us overhead + 22.5 us per byte @ 400 kHz Fast mode

Example::

```
uint8_t recv_buf[4];
i2c_read_ctx_t ctx={ .data=recv_buf, .length=sizeof(recv_buf), .result=k_rx_err_invalid_state, };
rx_err_t err=rx_bus_manager_with_bus(manager,bus_name, internal_i2c_read_callback,&ctx);
if(err==k_rx_ok&&ctx.result==k_rx_ok){ }
```

NASA Power of 10 Compliance:: Rule 5: 2 preconditions, 2 postconditions Rule 4: Function is 20 lines (well under 60 limit) Rule 7: All riic_read() return values checked

See also: rx_bus_i2c_read() Public API that invokes this callback, i2c_read_ctx_t Context structure passed as user_ctx, riic_read() Underlying RIIC HAL read function

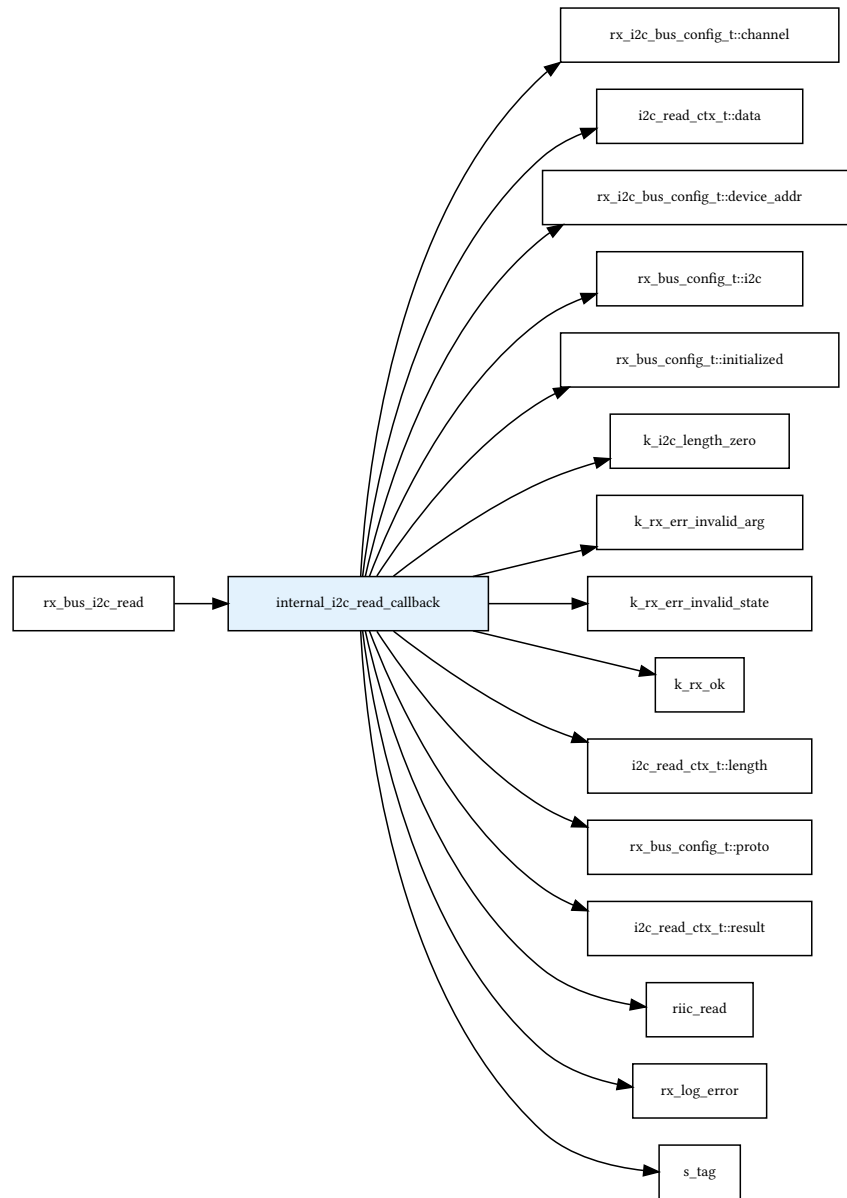


Figure 60: Call graph for internal_i2c_read_callback

Defined in `libs/rx_bus/src/rx_bus_i2c.c:692`

Since Version 1.0.0

—

internal_i2c_write_read_callback

```
static rx_err_t internal_i2c_write_read_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback for I2C combined write-read (register read) operation.

Internal callback implementing a combined I2C write-then-read transfer with a repeated START between phases. This is the standard protocol for reading device registers: write the register address byte(s), issue a repeated START, then read the response without releasing the bus between phases.

Algorithm steps:

1. Validate bus is initialized
2. Validate write_data non-NULL when write_length > 0
3. Validate read_data non-NULL when read_length > 0
4. Call riic_write_read() with both write and read parameters
5. Set ctx->result and return

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration structure initialized must be true proto.i2c.channel specifies the RIIC channel proto.i2c.device_addr specifies the 7-bit target address
inout	user_ctx	User context (i2c_write_read_ctx_t*) input: ctx->write_data points to bytes to write (register address) input: ctx->write_length specifies number of bytes to write input: ctx->read_data points to receive buffer for response bytes input: ctx->read_length specifies number of bytes to read output: ctx->read_data buffer filled with received bytes on success output: ctx->result contains the operation result

Returns: rx_err_t Error code indicating result

Value	Description
k_rx_ok	Success, combined write-read completed
k_rx_err_invalid_state	Bus not initialized
k_rx_err_invalid_arg	write_data nullptr with write_length > 0, or read_data nullptr with read_length > 0
k_rx_err_hw_error	RIIC write-read failed (NAK, bus error, or timeout)

Pre: bus_config->initialized must be true

Pre: ctx->write_data non-NULL when ctx->write_length > 0

Post: ctx->result contains the operation result

Post: ctx->read_data buffer contains device response bytes on k_rx_ok

Note: Not thread-safe; called from bus manager with bus lock held

Warning: Both write and read buffers must remain valid throughout the callback] **Example:**

```
const uint8_t reg_addr[] = {0x09}; // voltage register
uint8_t resp[2] = {0, 0};
i2c_write_read_ctx_t ctx = {
    .write_data = reg_addr,
    .write_length = sizeof(reg_addr),
    .read_data = resp,
    .read_length = sizeof(resp),
    .result = k_rx_err_invalid_state,
};
rx_err_t err = rx_bus_manager_with_bus(manager, bus_name,
    internal_i2c_write_read_callback, &ctx);
```

See also: `rx_bus_i2c_write_read()` Public API that invokes this callback,
`i2c_write_read_ctx_t` Context structure passed as `user_ctx`, `riic_write_read()`
Underlying RIIC HAL combined transfer function

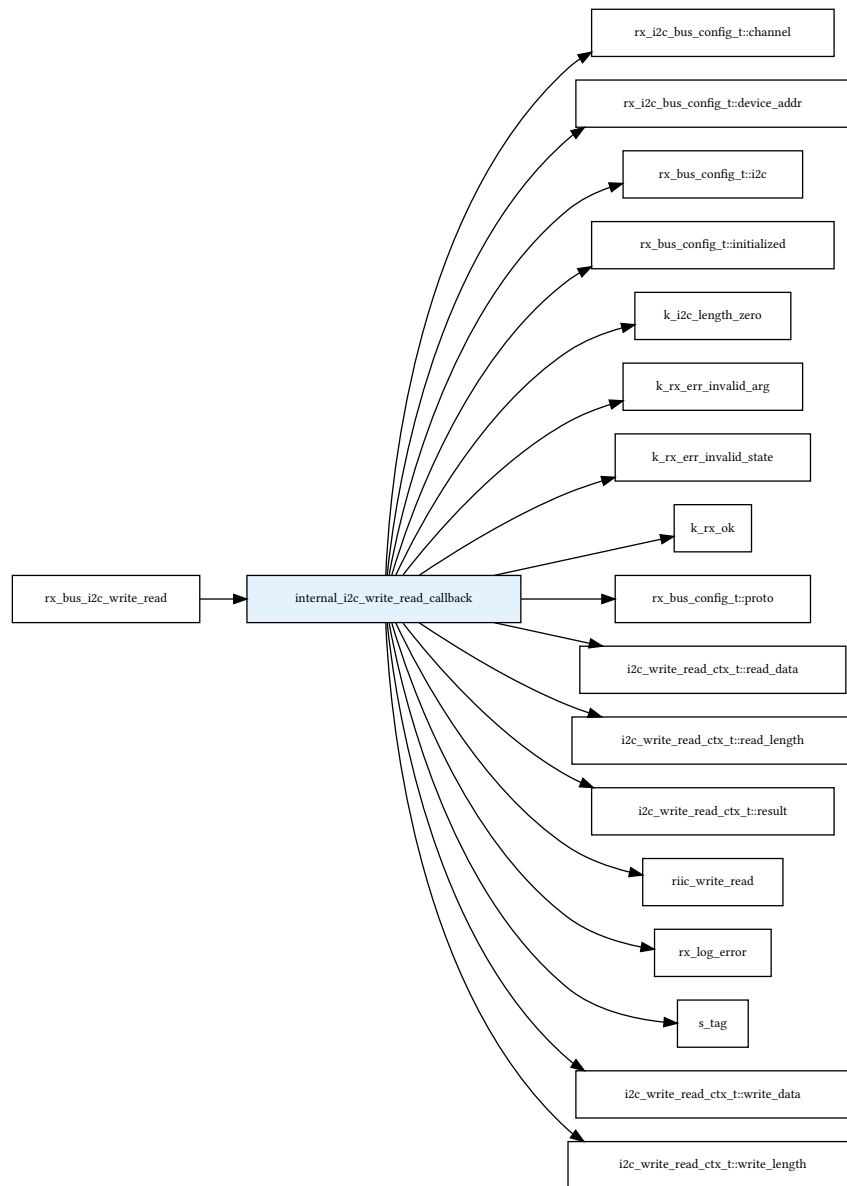


Figure 61: Call graph for internal_i2c_write_read_callback

_Defined in libs/rx_bus/src/rx_bus_i2c.c:789_

Since Version 1.0.0

—

rx_bus_i2c_init

```
rx_err_t rx_bus_i2c_init(rx_bus_manager_t *manager, const char *bus_name)
```

Initialize I2C bus peripheral hardware.

Initialize I2C bus through bus manager.

Initializes the RX72N RIIC peripheral for the specified I2C bus. Configures clock frequency, enables peripheral, and marks bus as ready for transactions. Must be called once before any read/write operations.

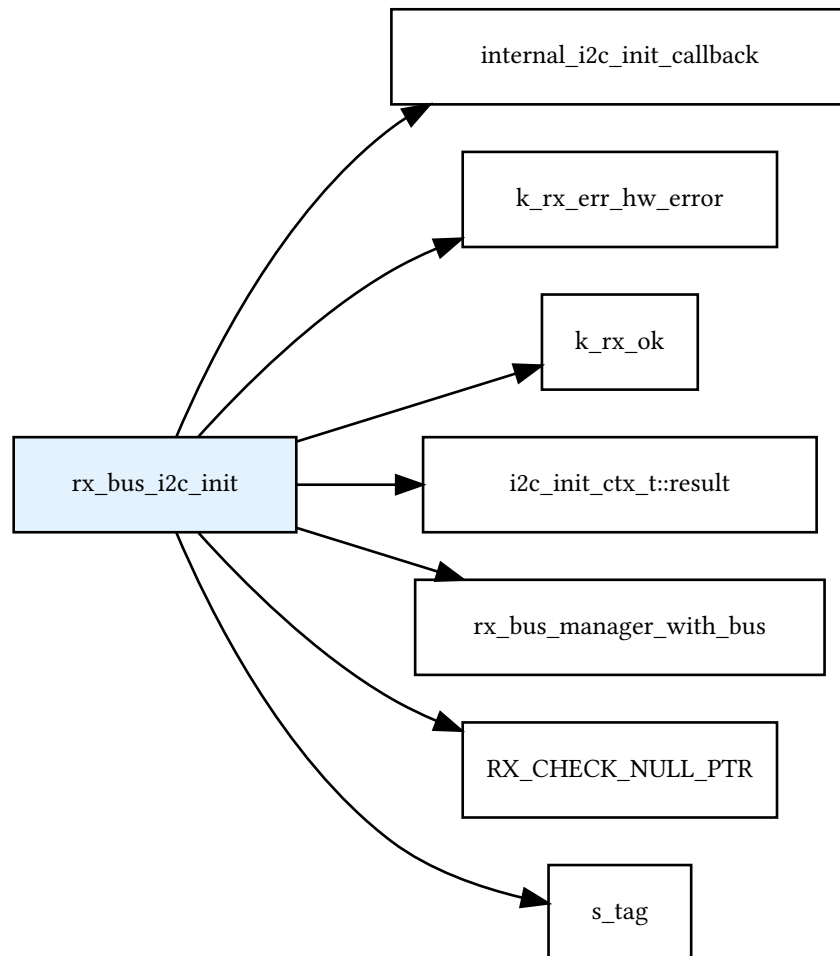


Figure 62: Call graph for `rx_bus_i2c_init`

_Defined in `libs/rx_bus/src/rx_bus_i2c.c:985_`

—

rx_bus_i2c_write

```
rx_err_t rx_bus_i2c_write(rx_bus_manager_t *manager, const char *bus_name, const
uint8_t *data, const uint16_t length)
```

Write data to I2C device.

Write data to I2C device through bus manager.

Transmits N bytes to the I2C device specified in bus configuration. Typically used to send commands, write configuration registers, or transfer data to peripherals. Generates START, device address (write), N data bytes, STOP.

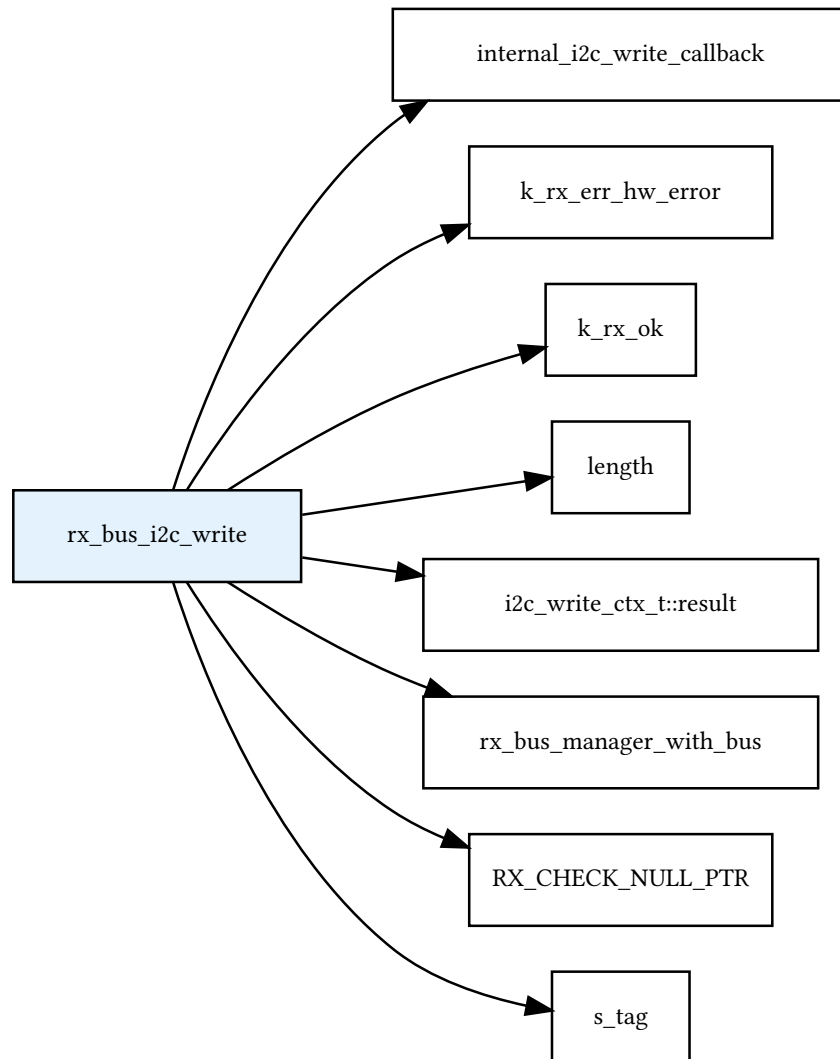


Figure 63: Call graph for `rx_bus_i2c_write`

_Defined in libs/rx_bus/src/rx_bus_i2c.c:1147_
—

rx_bus_i2c_read

```
rx_err_t rx_bus_i2c_read(rx_bus_manager_t *manager, const char *bus_name, uint8_t  
*data, uint16_t length)
```

Read data from I2C device.

Read data from I2C device through bus manager.

Receives N bytes from the I2C device specified in bus configuration. Generates START, device address (read), receives N data bytes with ACKs, sends final NACK, STOP. Used for reading sensor data, status registers, or bulk data from device memory.

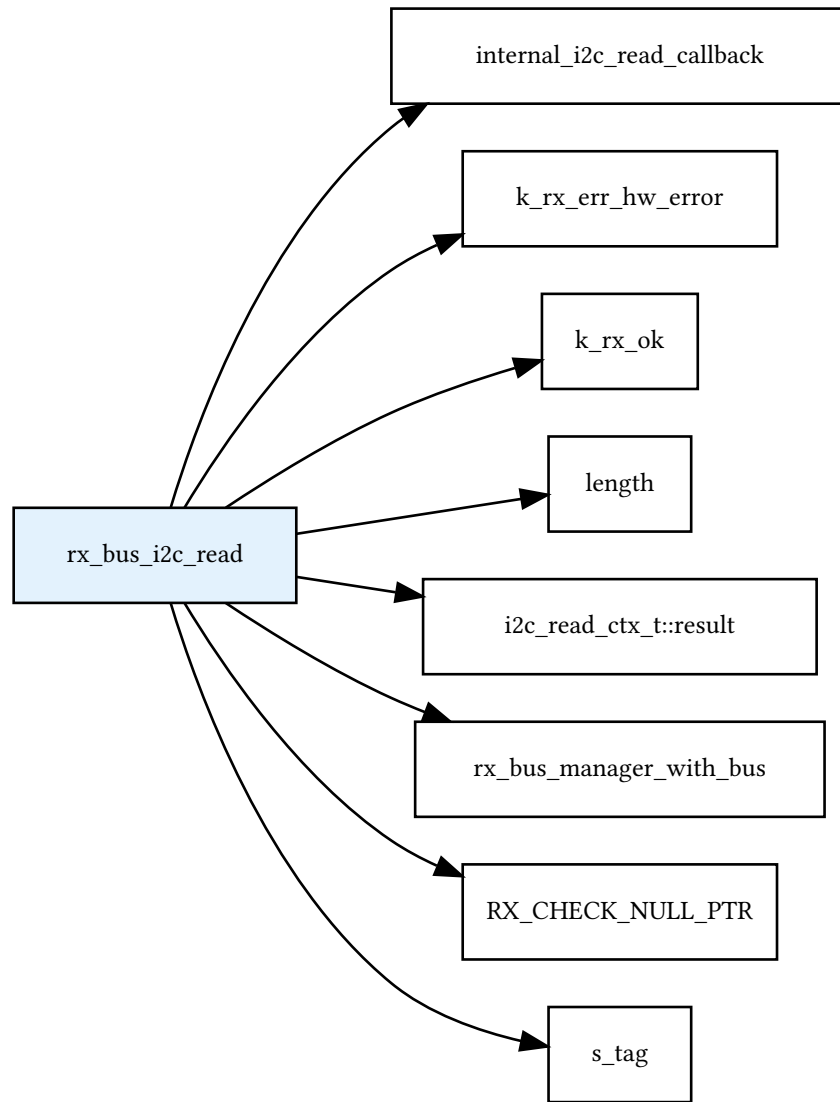


Figure 64: Call graph for `rx_bus_i2c_read`

_Defined in `libs/rx_bus/src/rx_bus_i2c.c:1219_`

—

rx_bus_i2c_write_read

```

rx_err_t rx_bus_i2c_write_read(rx_bus_manager_t *manager, const char *bus_name,
const uint8_t *write_data, const uint16_t write_length, uint8_t *read_data, const
uint16_t read_length)

```

Write then read from I2C device (register access pattern).

Write then read from I2C device through bus manager.

Combined write-read operation for register access. Writes N bytes (typically register address), sends repeated START, then reads M bytes (register contents). This is the MOST COMMON I2C operation for sensors and peripherals.

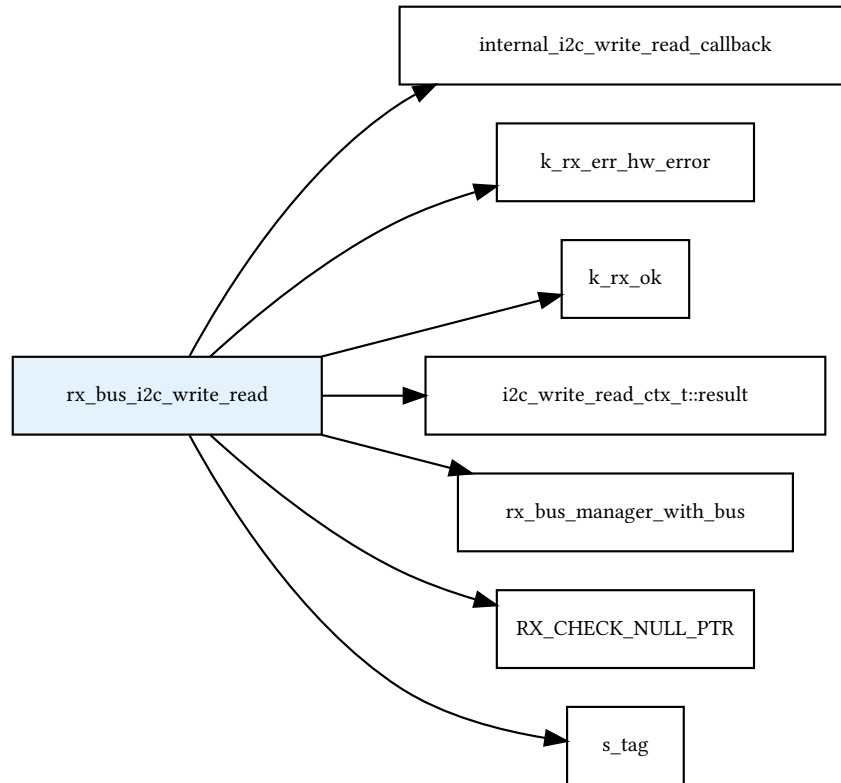


Figure 65: Call graph for `rx_bus_i2c_write_read`

_Defined in `libs/rx_bus/src/rx_bus_i2c.c:1414_`

rx_bus_manager.c

Bus Manager Implementation - Thread-Safe Multi-Protocol Registry.

Includes:

- `"rx_bus_manager.h"`
- `<string.h>`
- `"rx_check.h"`
- `"rx_log.h"`
- `"rx_threadx_config.h"`
- `"rx_time_constants.h"`

s_tag

`const char* s_tag`

internal_execute_command_callback

```
static rx_err_t internal_execute_command_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Adapter callback to execute command via rx_bus_manager_with_bus.

Internal adapter function that bridges the command pattern interface (rx_bus_manager_execute_command) to the existing callback infrastructure (rx_bus_manager_with_bus). Avoids code duplication by reusing mutex locking and bus lookup logic.

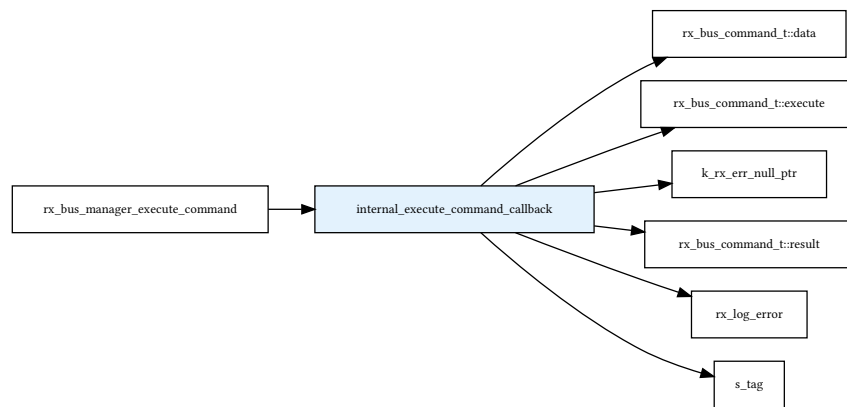


Figure 66: Call graph for internal_execute_command_callback

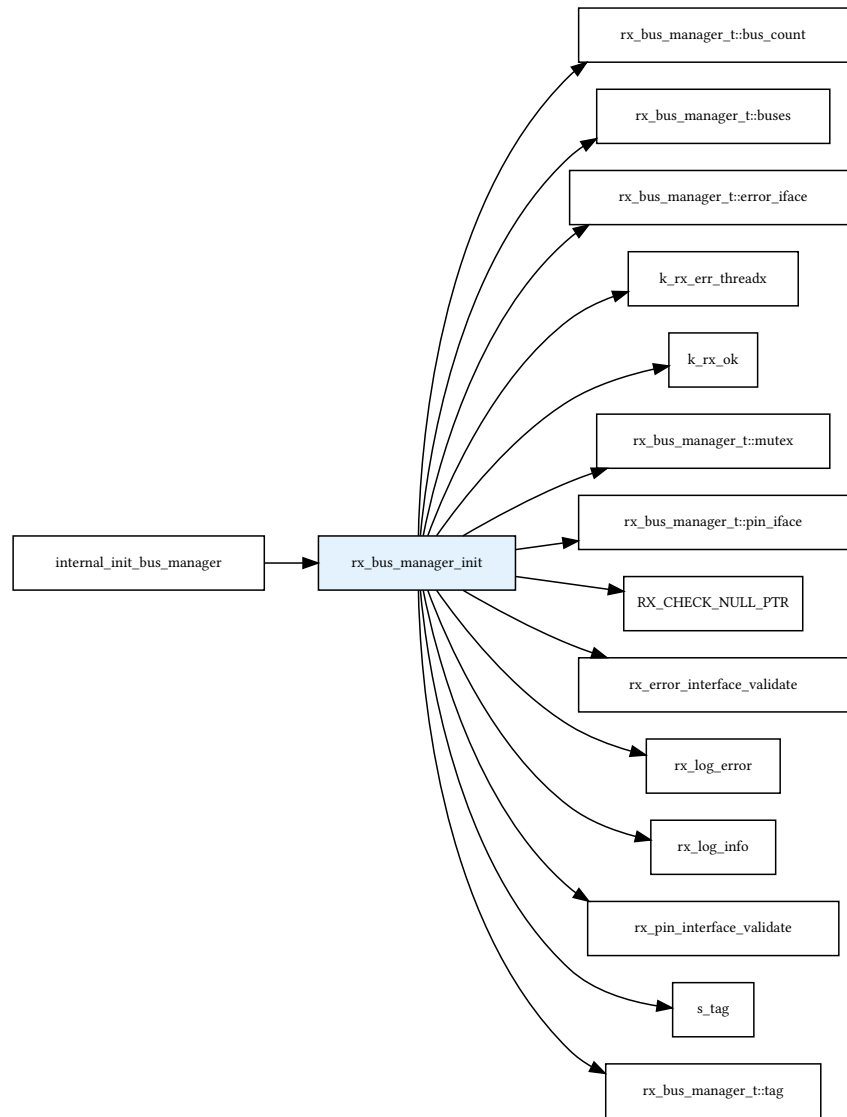
_Defined in libs/rx_bus/src/rx_bus_manager.c:257_

rx_bus_manager_init

```
rx_err_t rx_bus_manager_init(rx_bus_manager_t *manager, const char *tag,
rx_error_interface_t *error_iface, rx_pin_interface_t *pin_iface)
```

Initialize bus manager with injected dependencies.

Creates and initializes a bus manager instance with ThreadX mutex for thread-safe operations. Follows Dependency Inversion Principle (DIP) by accepting injected error handler and pin validator interfaces.

Figure 67: Call graph for `rx_bus_manager_init`

_Defined in `libs/rx\_bus/src/rx\_bus\_manager.c:465_`

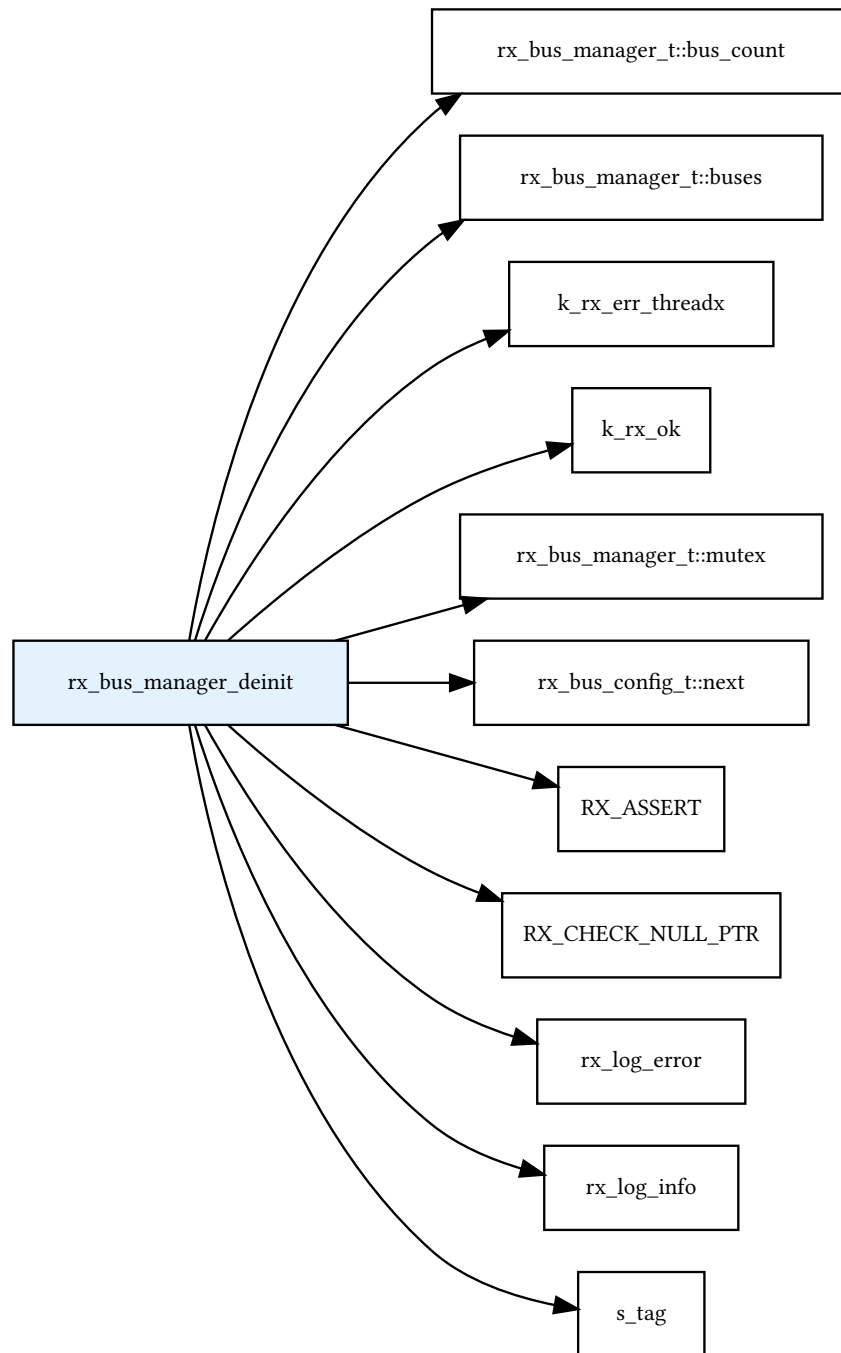
—

rx_bus_manager_deinit

```
rx_err_t rx_bus_manager_deinit(rx_bus_manager_t *manager)
```

Deinitialize bus manager and release all resources.

Performs complete cleanup of bus manager by removing all registered buses, deleting the ThreadX mutex, and resetting all state. Safe to call on partially initialized managers (cleans up what exists).

Figure 68: Call graph for `rx_bus_manager_deinit`

Defined in `libs/rx_bus/src/rx_bus_manager.c:660`

—

rx_bus_manager_add_bus

```
rx_err_t rx_bus_manager_add_bus(rx_bus_manager_t *manager, rx_bus_config_t
*bus_config)
```

Register a bus configuration with the manager.

Adds a bus to the manager's registry using intrusive linked list. The manager stores the bus_config pointer but does NOT take ownership of the memory (caller must free when done). Bus hardware is NOT initialized until first access (lazy initialization pattern).

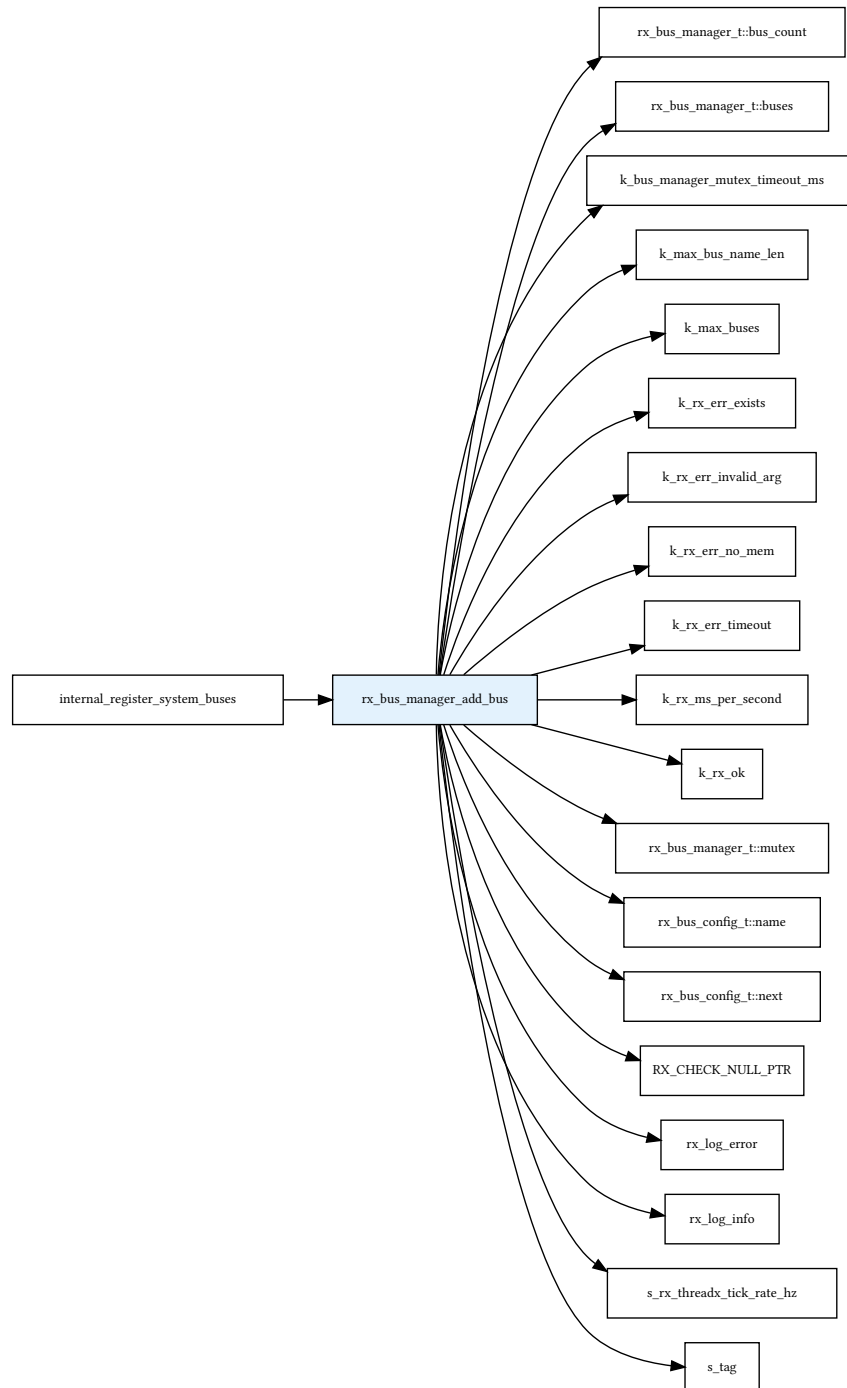


Figure 69: Call graph for rx_bus_manager_add_bus

_Defined in libs/rx_bus/src/rx_bus_manager.c:878_

—

rx_bus_manager_remove_bus

```
rx_err_t rx_bus_manager_remove_bus(rx_bus_manager_t *manager, const char *name)
```

Unregister and remove a bus by name.

Unregister and cleanup a bus by name.

Removes a bus from the manager's registry using the indirect pointers technique for elegant linked list node removal. Does NOT free bus_config memory (caller owns it). Does NOT deinitialize hardware (assumes bus idle).

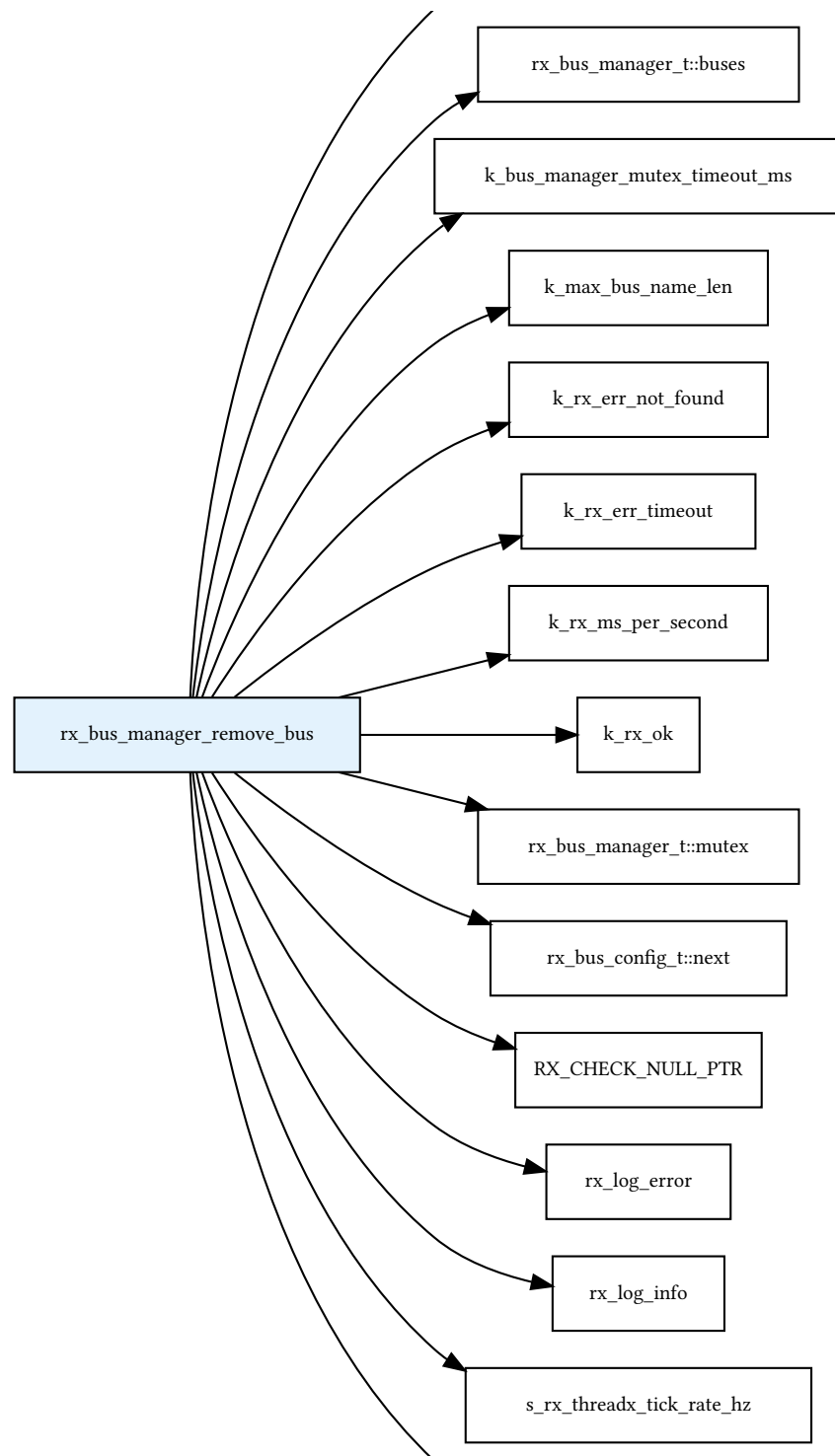


Figure 70: Call graph for `rx_bus_manager_remove_bus`

_Defined in libs/rx_bus/src/rx_bus_manager.c:1131_

—

rx_bus_manager_find_bus

```
rx_err_t rx_bus_manager_find_bus(rx_bus_manager_t *manager, const char *name,
rx_bus_config_t **bus_config)
```

Find a bus by name (thread-safe lookup).

Searches the bus registry for a bus with the given name and returns a pointer to its configuration. The lookup is thread-safe, but the returned pointer is NOT protected after the function returns.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	name	Bus name to search for (case-sensitive)
out	bus_config	Pointer to receive bus configuration address

Returns: rx_err_t Lookup status

Value	Description
k_rx_ok	Bus found, bus_config points to configuration
k_rx_err_null_ptr	manager, name, or bus_config is nullptr
k_rx_err_not_found	No bus with given name exists
k_rx_err_timeout	Mutex timeout acquiring lock

Pre: manager initialized via rx_bus_manager_init()

Post: *bus_config points to found configuration (if k_rx_ok)

Post: *bus_config unchanged (if error)

Note: Lookup is O(n) where n = bus_count

Warning: Race Condition Risk: Prefer rx_bus_manager_with_bus() instead. The returned bus_config pointer could become invalid if another thread calls rx_bus_manager_remove_bus() after this function returns.

Warning: Returned pointer not protected from concurrent removal

Thread Safety:: Lookup is thread-safe, but returned pointer is NOT protected. Use rx_bus_manager_with_bus() for protected access.

Example (Simple, but has race condition risk):: rx_bus_config_t*bus;
rx_err_t err=rx_bus_manager_find_bus(&manager,"imu",&bus); if(err==k_rx_ok)
{ rx_log_info("MAIN","Foundbus:%s,type:%d",bus->name,bus->type); }

See also: rx_bus_manager_with_bus() Recommended thread-safe access pattern

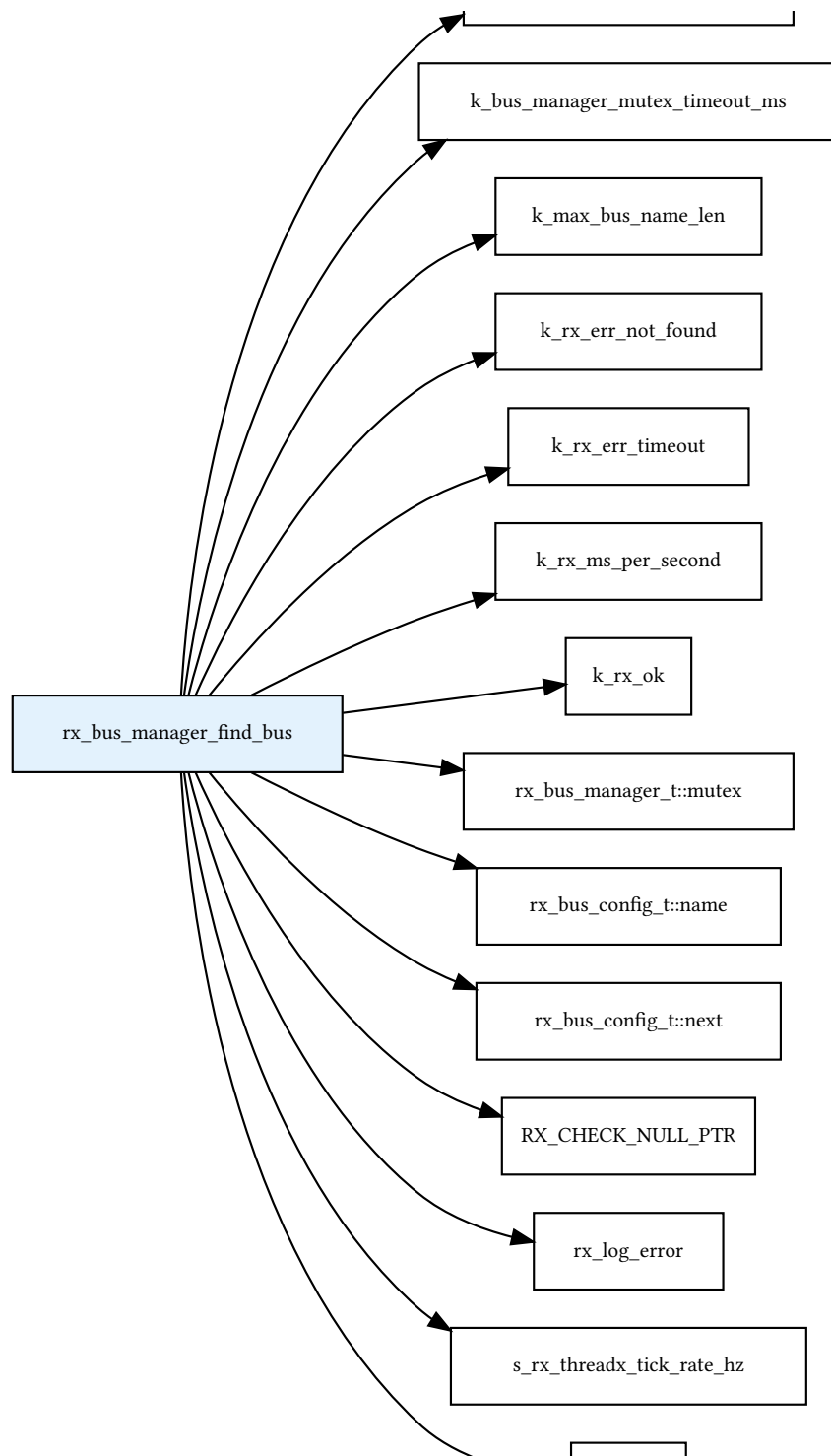


Figure 71: Call graph for `rx_bus_manager_find_bus`

_Defined in `libs/rx\_bus/src/rx\_bus\_manager.c:1181_`

Since Version 1.0.0

—

rx_bus_manager_with_bus

```
rx_err_t rx_bus_manager_with_bus(rx_bus_manager_t *manager, const char *name,  
const rx_bus_callback_t callback, void *user_ctx)
```

Execute callback with bus locked (RECOMMENDED access pattern).

The recommended pattern for thread-safe bus access. Holds the manager mutex while the callback executes, preventing bus removal race conditions.

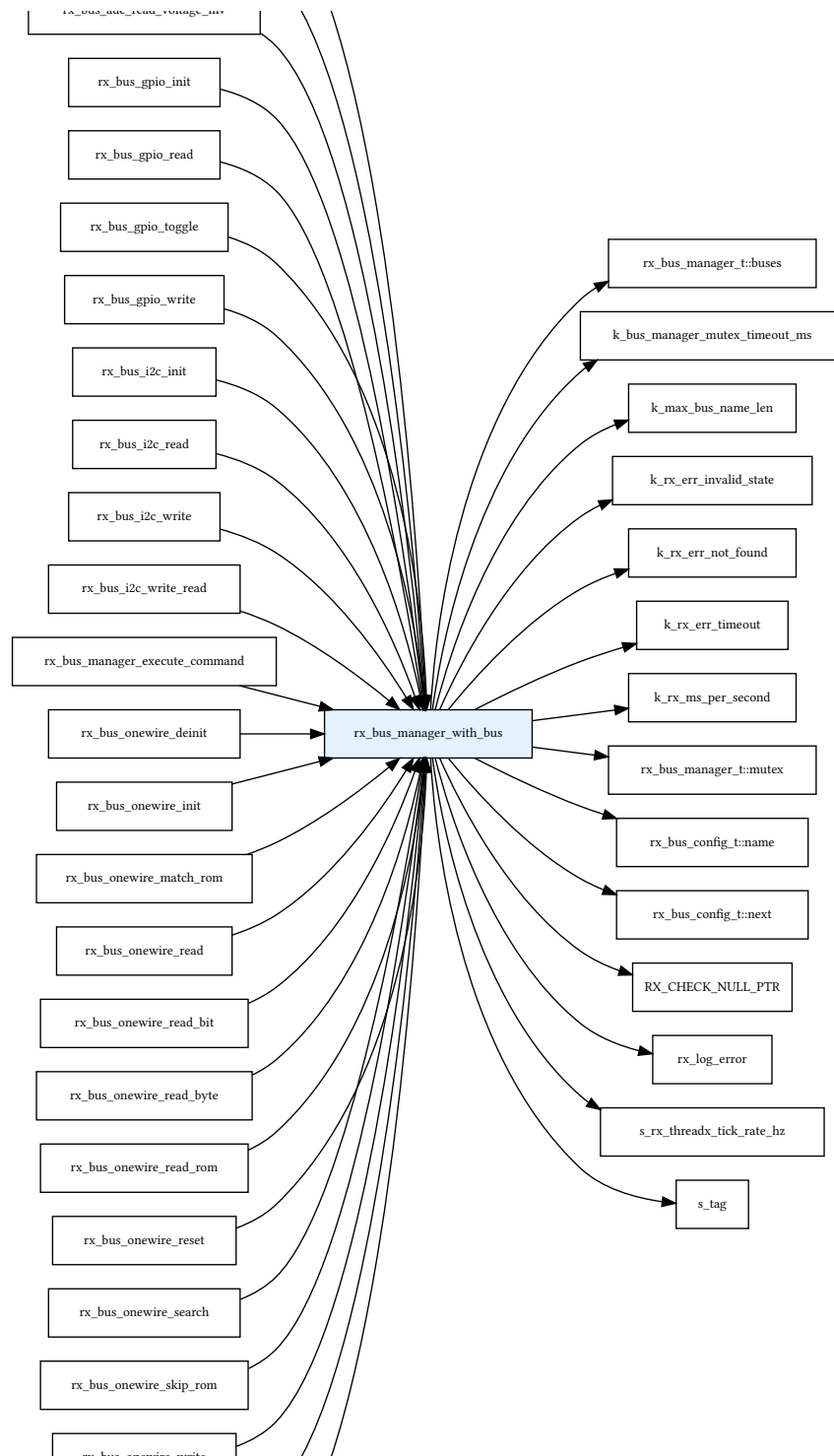


Figure 72: Call graph for rx_bus_manager_with_bus

_Defined in libs/rx_bus/src/rx_bus_manager.c:1219_
 —

rx_bus_manager_execute_command

```
rx_err_t rx_bus_manager_execute_command(rx_bus_manager_t *manager, const char
*name, rx_bus_command_t *command)
```

Execute a command on a bus using Command Pattern (RECOMMENDED for extensibility).

The recommended interface for implementing new bus operations following the Open/Closed Principle. New operations can be added as commands without modifying the bus manager internals.

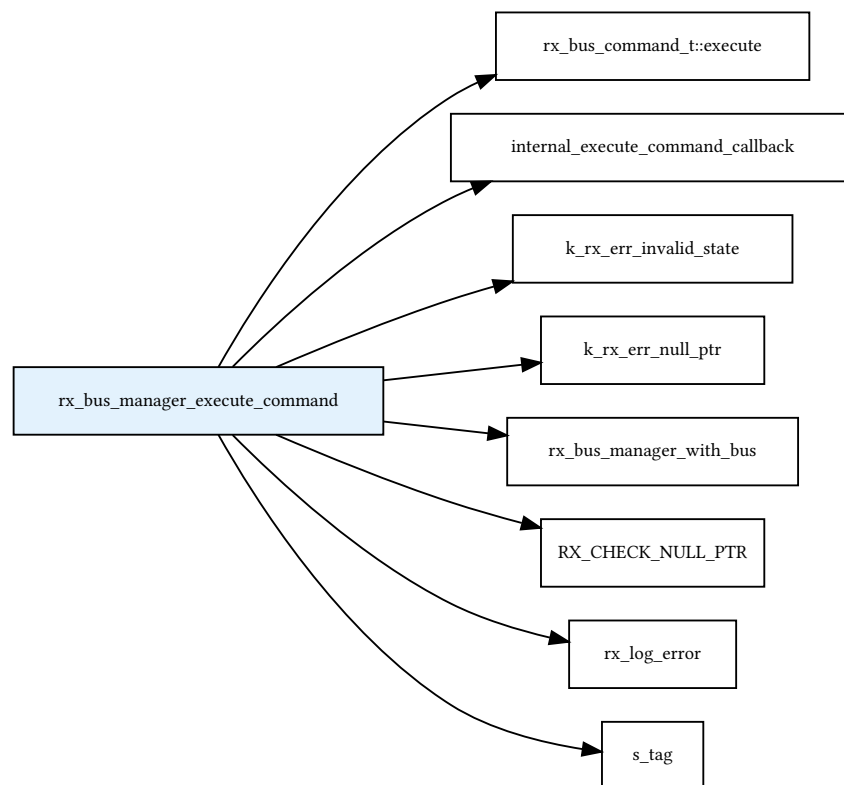


Figure 73: Call graph for `rx_bus_manager_execute_command`

_Defined in libs/rx_bus/src/rx_bus_manager.c:1270_
 —

rx_bus_onewire.c

OneWire (1-Wire) Bus Implementation Using GPIO Bit-Banging.

Implements the Dallas/Maxim 1-Wire protocol using software bit-banging on GPIO pins. Provides all standard 1-Wire operations: reset/presence detect, bit/byte read/write, and ROM commands (Skip, Match, Read, Search).

STAR Team

2026-01-03

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_bus_owewire.h"
- <string.h>
- "hardware.h"
- "rx72n_clock.h"
- "rx72n_cmt_regs.h"
- "rx72n_regs.h"
- "rx72n_system_regs.h"
- "rx_bus_types.h"
- "rx_check.h"
- "rx_crc.h"
- "rx_log.h"
- "rx_register_protection.h"

CHECK_ONEWIRE_BUS

```
#define CHECK_ONEWIRE_BUS do
{
    if ((bus_config)->type != k_bus_type_owewire)
    {
        rx_log_error(s_tag, "Bus is not OneWire type");
        (ctx)->result = k_rx_err_invalid_arg;
        return (ctx)->result;
    }
    if ((check_initialized) && !(bus_config)->initialized)
    {
        rx_log_error(s_tag, "Bus not initialized");
        (ctx)->result = k_rx_err_invalid_state;
        return (ctx)->result;
    }
} while (0)
```

Helper macro to validate OneWire bus type and initialization state.

—

onewire_driver_constants_t

OneWire driver constants.

Value	Name	Description
= k_max_buses	k_owewire_max_instances	

= 0x01U	k_owewire_single_bit_mask	
= 0U	k_owewire_search_last_zero_init	
= 1U	k_owewire_first_bit_number	
= 1U	k_owewire_rom_bit_mask_reset	
= 0U	k_owewire_no_discrepancy	
= 255U	k_owewire_max_buf_bytes	
= 1U	k_owewire_bit_mask_lsb	
= 1U	k_owewire_crc_offset	
= 0U	k_owewire_rom_family_code_idx	
= 0x00U	k_owewire_invalid_family_code	
= 0U	k_owewire_instance_idx_start	Starting index for instance pool iteration
= 7U	k_owewire_rom_crc_idx	CRC byte index in ROM (k_owewire_rom_crc_idx)

onewire_delay_hw_constants_t

Hardware timer configuration for microsecond delays.

Uses CMT channel 3 in free-run mode (no interrupts) as a 16-bit counter clocked from PCLKB/8 (7.5 MHz). Delays are implemented by measuring elapsed ticks to ensure consistent timing independent of compiler optimizations.

Value	Name	Description
= 15	k_owewire_mstpb_cmt_bit	CMT module stop bit in MSTPCRB
= 1	k_owewire_cmt3_start_bit	CMSTR1 bit controlling CMT3
= 0	k_owewire_cmt_divider_setting	CKS = 0 -> PCLKB/8
= 8	k_owewire_cmt_divider_value	Actual divider value
= 0	k_owewire_cmt_clk_shift	CMCR clock select shift
= 0xFFFF	k_owewire_timer_counter_max	
= 1000000UL	k_owewire_us_per_second	
= k_owewire_us_per_second - 1UL	k_owewire_timer_rounding	
= 1U	k_owewire_bit_set	
= 0U	k_owewire_counter_reset	
= 0U	k_owewire_zero_microseconds	

onewire_delay_tick_constants_t

Value	Name	Description
= 0ULL	k_onewire_ticks_zero	
= 1ULL	k_onewire_min_ticks	

—

s_tag`const char* s_tag`

—

s_onewire_max_search_devices`const uint32_t s_onewire_max_search_devices`

—

s_state_pool`onewire_state_entry_t s_state_pool[k_onewire_max_instances]`

—

s_delay_timer_initialized`bool s_delay_timer_initialized`

—

internal_delay_timer_init

```
static void internal_delay_timer_init(void)
```

Initialize CMT3 timer for microsecond-precision delays.

Configures CMT channel 3 as a free-running 16-bit counter for timing delays. Uses PCLKB/8 clock source (7.5 MHz at 60 MHz PCLKB) for 133ns resolution. Timer runs continuously without interrupts.

Pre: System clocks initialized (PCLKB running)

Post: CMT3 running as free-running counter

Post: s_delay_timer_initialized == true

Note: Called automatically by internal_delay_us() on first use

Note: Timer resolution: $1 / (\text{PCLKB}/8) = 133\text{ns}$ @ 60MHz PCLKB

Algorithm:: Check if already initialized (early return) Assert register pointers are valid Unlock MSTPCRB via PRCR Enable CMT module clock (clear MSTPB15) Lock PRCR Stop CMT3 (clear CMSTR1 bit 1) Configure CMCR for PCLKB/8, no interrupts Set CMCOR to 0xFFFF (full 16-bit range) Reset counter to 0 Start timer (set CMSTR1 bit 1)

See also: `internal_delay_us()` Uses this timer for delays

Figure 74: Call graph for `internal_delay_timer_init`

_Defined in libs/rx_bus/src/rx_bus_onewire.c:202_

—

internal_delay_us

```
static void internal_delay_us(uint32_t microseconds)
```

Delay for specified microseconds using CMT3 timer.

Implements busy-wait delay using CMT3 counter. Handles delays longer than the 16-bit counter maximum by looping. Automatically initializes timer on first call.

Parameters:

Direction	Name	Description
in	microseconds	Delay duration in microseconds (0 = no delay)

Pre: System clocks initialized

Post: Delay of approximately microseconds us elapsed

Note: Busy-wait implementation, blocks CPU

Note: Accuracy: +/-1 timer tick (133ns @ 7.5MHz)

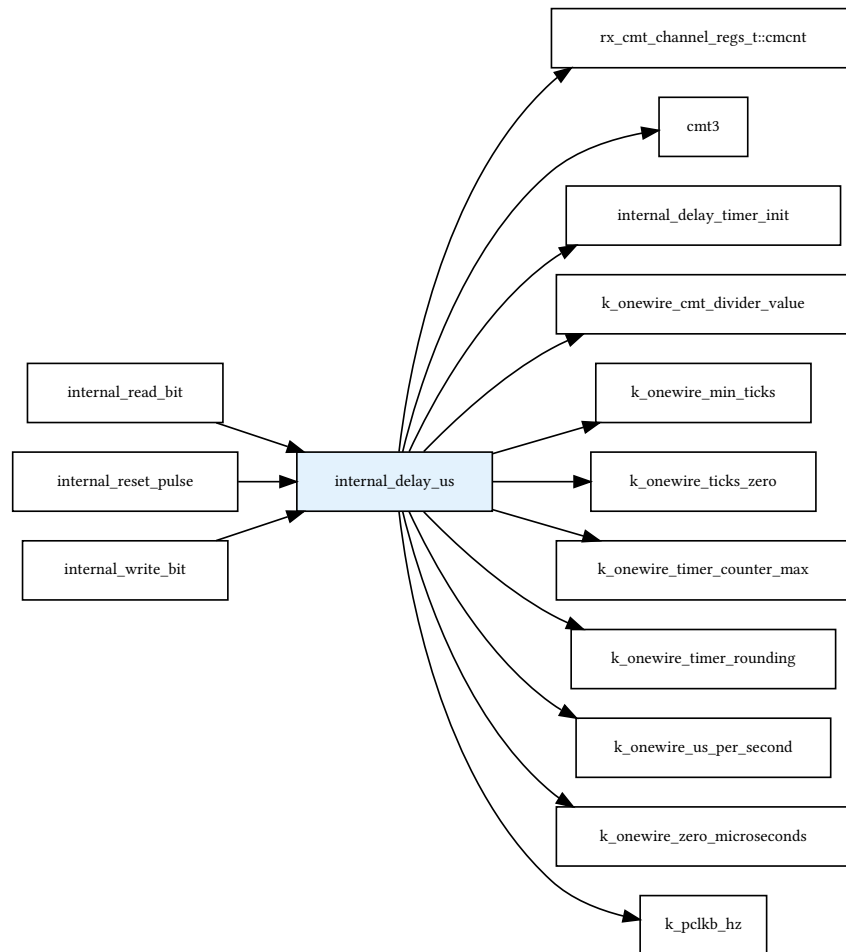
Note: Maximum single delay: limited by uint32_t microseconds

Warning: Disables preemption during delay (busy-wait)

Algorithm:: Return immediately if microseconds == 0 Initialize timer if not already done Calculate required ticks: (us * timer_hz + round) / 1000000 Ensure minimum 1 tick for very short delays Loop while ticks > 0: a. Calculate wait_ticks (capped at 0xFFFF) b. Read start counter value c. Busy-wait until (current - start) >= wait_ticks d. Subtract wait_ticks from remaining

Performance:: Overhead: 10 cycles for function call Resolution: 133ns (1 tick @ 7.5MHz) Accuracy: +/-0.5% typical

See also: internal_delay_timer_init() Timer initialization

Figure 75: Call graph for `internal_delay_us`

_Defined in `libs/rx\bus/src/rx\bus\onewire.c:268_`

—

internal_acquire_state

```
static rx_err_t internal_acquire_state(rx_bus_config_t *bus_config,
onewire_runtime_state_t **state)
```

Acquire or allocate runtime state for a 1-Wire bus.

Ensures a runtime state structure exists for the bus. If the bus already has state (`handle != nullptr`), returns existing state. Otherwise allocates from static pool. Uses zero-allocation pattern for NASA Rule 3 compliance.

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration node (handle field updated)
out	state	Pointer to receive runtime state

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, state available
k_rx_err_no_mem	Static pool exhausted (k_owewire_max_instances)

Pre: bus_config != nullptr

Pre: state != nullptr

Post: *state points to valid runtime state

Post: bus_config->handle points to same state

Note: Uses static pool, no dynamic allocation

Note: Pool size: k_owewire_max_instances (= k_max_buses)

Algorithm:: If bus_config->handle != nullptr, return existing state Search static pool for unused entry Mark entry as in_use, zero-initialize state Store pointer in bus_config->handle Return state pointer via output parameter

See also: internal_get_state() Retrieve existing state

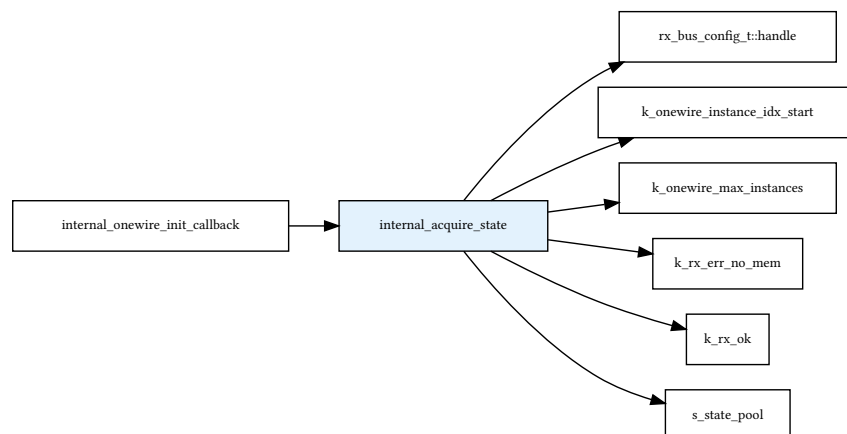


Figure 76: Call graph for `internal_acquire_state`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:331_`

internal_get_state

```
static onewire_runtime_state_t * internal_get_state(const rx_bus_config_t
*bus_config)
```

Retrieve runtime state for already-initialized bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node

Returns: Pointer to state or nullptr if missing

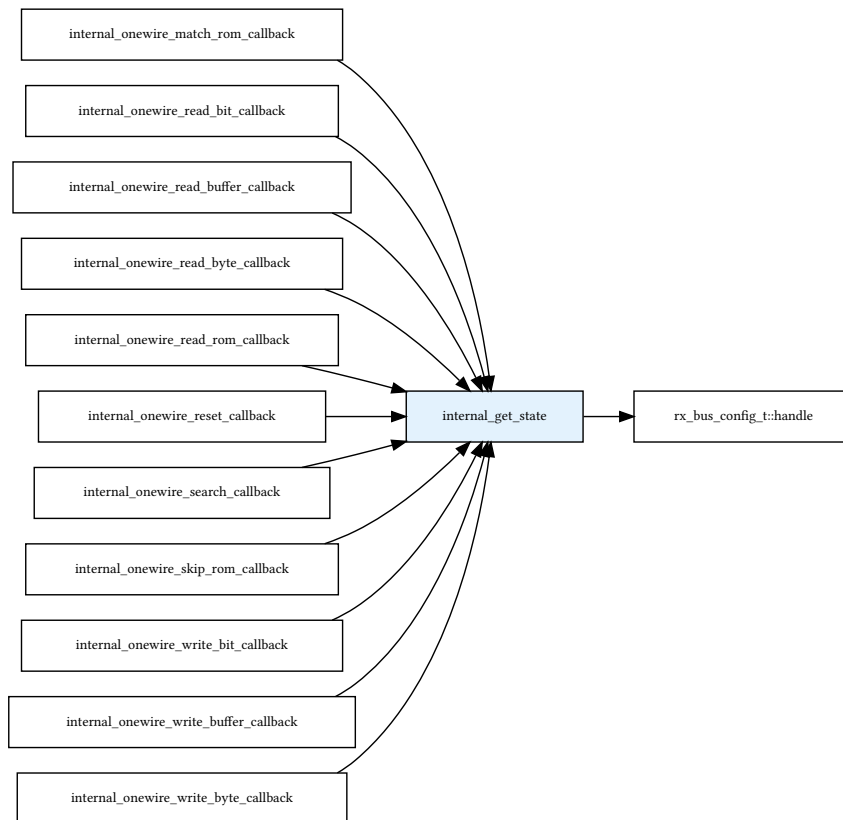


Figure 77: Call graph for internal_get_state

_Defined in libs/rx_bus/src/rx_bus_onewire.c:358_

internal_set_drive_mode

```
static rx_err_t internal_set_drive_mode(const rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, const bool output)
```

Configure GPIO drive mode (output low vs input/high-Z).

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	state	Runtime state tracking current mode
in	output	True to drive (output), false to release (input)

Returns: k_rx_ok on success

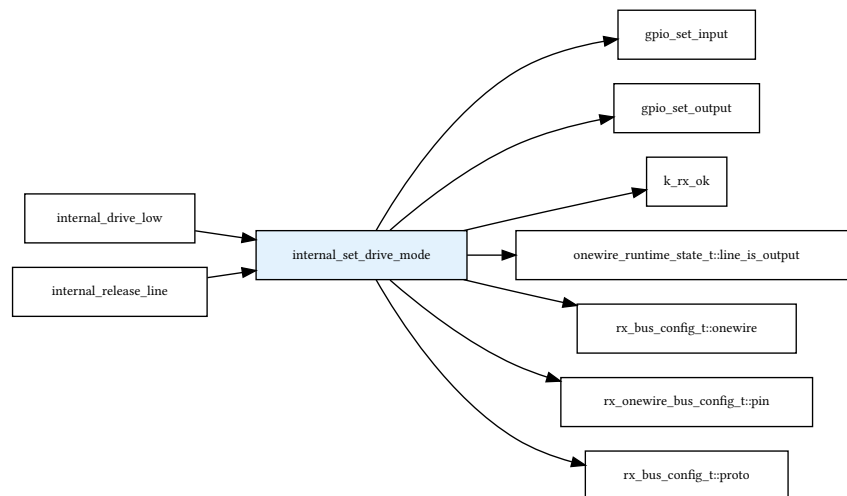


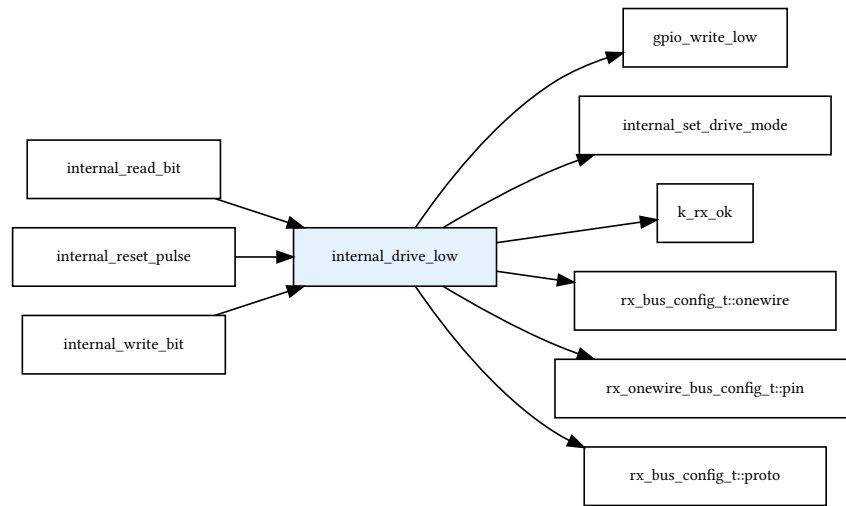
Figure 78: Call graph for `internal_set_drive_mode`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:372_`

internal_drive_low

```
static rx_err_t internal_drive_low(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state)
```

Drive the OneWire line low (open-drain style).

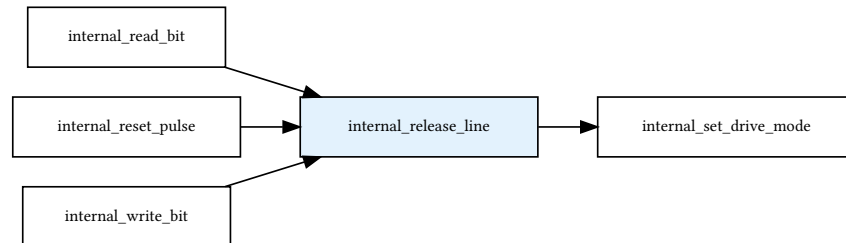
Figure 79: Call graph for `internal_drive_low`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:397_`

internal_release_line

```
static rx_err_t internal_release_line(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state)
```

Release the OneWire line (input/high-Z, external pull-up drives high).

Figure 80: Call graph for `internal_release_line`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:409_`

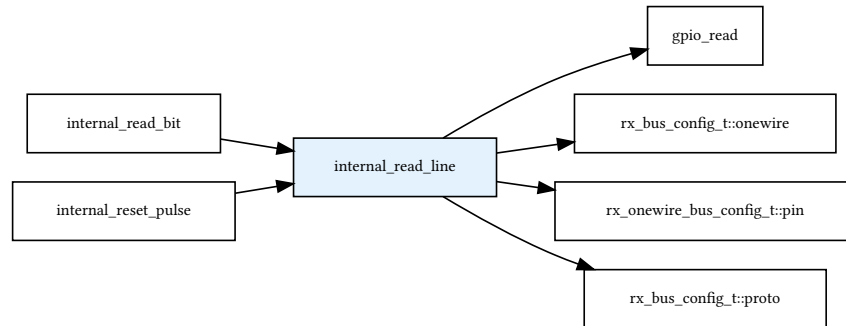
internal_read_line

```
static rx_err_t internal_read_line(const rx_bus_config_t *bus_config, bool *high)
```

Read current level on the OneWire line.

Parameters:

Direction	Name	Description
out	high	True if line is high

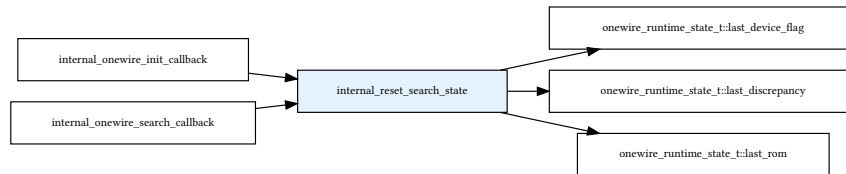
Figure 81: Call graph for `internal_read_line`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:419_`

`internal_reset_search_state`

```
static void internal_reset_search_state(onewire_runtime_state_t *state)
```

Reset runtime search tracking.

Figure 82: Call graph for `internal_reset_search_state`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:427_`

`internal_reset_pulse`

```
static rx_err_t internal_reset_pulse(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, bool *presence)
```

Issue 1-Wire reset pulse and detect device presence.

Executes the 1-Wire reset/presence detect sequence. Controller drives line low for 480us (reset pulse), then releases and samples during presence window. Devices respond by pulling line low within 60us of release.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	bus_config	Bus configuration with GPIO pin
inout	state	Runtime state for GPIO mode tracking
out	presence	Set to true if device responded (line went low)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, presence result valid
k_rx_err_*	GPIO operation failed

Pre: bus_config != nullptr

Pre: state != nullptr

Pre: presence != nullptr

Post: *presence indicates device response

Post: Line released (high via pullup)

Note: Total reset slot: 960us

Note: Multiple devices can respond simultaneously

Timing Diagram:: |<-480us->|<-70us->|<-410us->| _____||_____||_____||_____||
 _____||| ^^^^ |||| DriveReleaseSampleComplete LowPresence

Algorithm:: Drive line low via internal_drive_low() Delay 480us (reset pulse duration) Release line via internal_release_line() Delay 70us (presence wait) Sample line level presence = !line_high (low = device present) Delay 410us (complete reset slot)

See also: k_onewire_reset_pulse_us Reset pulse duration (480us),
 k_onewire_presence_wait_us Presence sample delay (70us), k_onewire_presence_tail_us
 Remaining window after sample (410us)

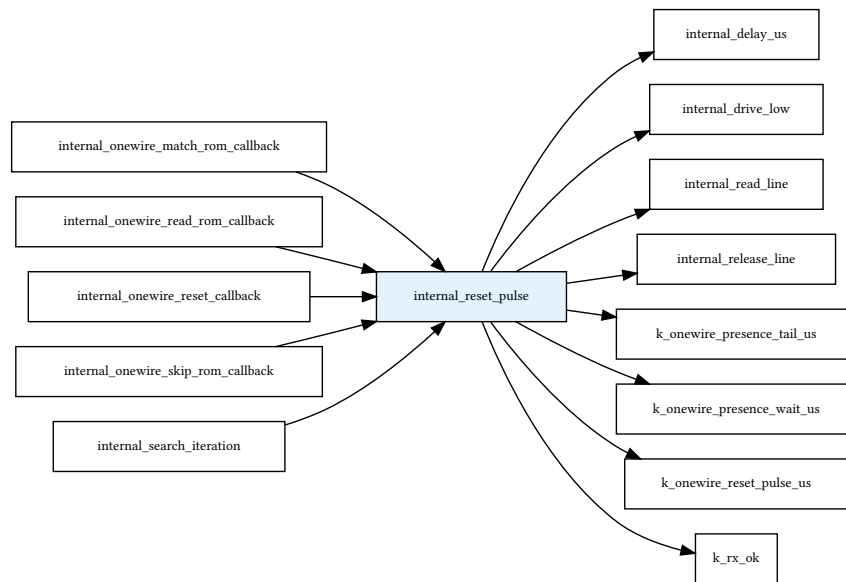


Figure 83: Call graph for internal_reset_pulse

_Defined in libs/rx_bus/src/rx_bus_owire.c:489_

internal_write_bit

```
static rx_err_t internal_write_bit(rx_bus_config_t *bus_config,
    onewire_runtime_state_t *state, const bool bit)
```

Write a single bit on the 1-Wire bus.

Writes one bit using the 1-Wire write time slot protocol. Write-1 and Write-0 differ in how long the line is held low.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration with GPIO pin
inout	state	Runtime state for GPIO mode tracking
in	bit	Value to write (true=1, false=0)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, bit written
k_rx_err_*	GPIO operation failed

Pre: bus_config != nullptr

Pre: state != nullptr

Post: Bit transmitted, line released high

Note: Total write slot: 70us

Write-1 Timing:: |<-6us->|<---64us--->| |_____|| |_____||

Write-0 Timing:: |<---60us--->|<10us>| |_____|| |_____||

See also: internal_write_byte() Writes 8 bits LSB-first

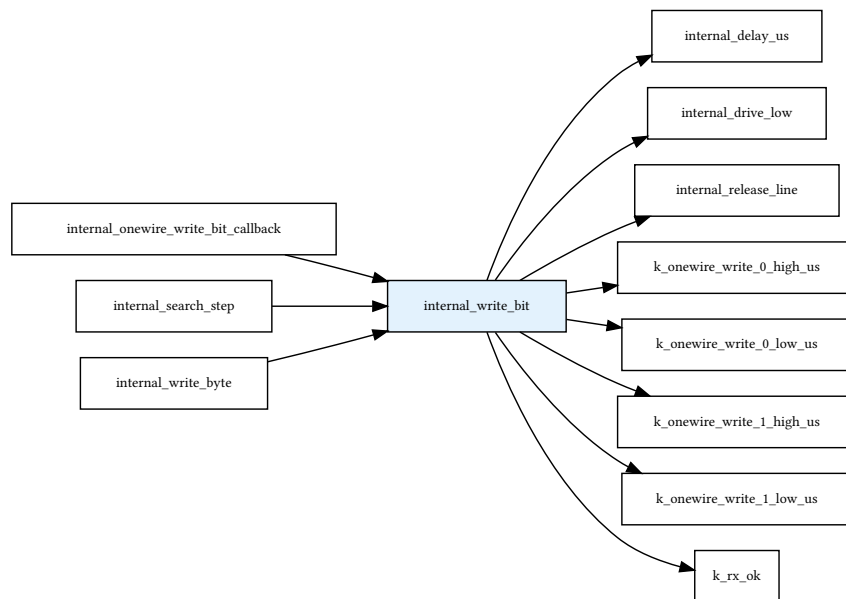


Figure 84: Call graph for internal_write_bit

_Defined in libs/rx_bus/src/rx_bus_onewire.c:556_

internal_read_bit

```
static rx_err_t internal_read_bit(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, bool *bit)
```

Read a single bit from the 1-Wire bus.

Reads one bit using the 1-Wire read time slot protocol. Controller initiates by driving low briefly, then releases and samples the line. Device drives low for 0, leaves high for 1.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	bus_config	Bus configuration with GPIO pin
inout	state	Runtime state for GPIO mode tracking
out	bit	Read value (true=1/high, false=0/low)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, bit read into output
k_rx_err_*	GPIO operation failed

Pre: bus_config != nullptr

Pre: state != nullptr

Pre: bit != nullptr

Post: *bit contains read value

Post: Line released high

Note: Total read slot: 70us

Note: Sample point: 15us after slot start

Read Timing:: |<6us>|<-9us->|<--55us-->| |_____| |_____| |_____| |(reading0) |
 _____ |(reading1) ^^ || ReleaseSample

See also: internal_read_byte() Reads 8 bits LSB-first

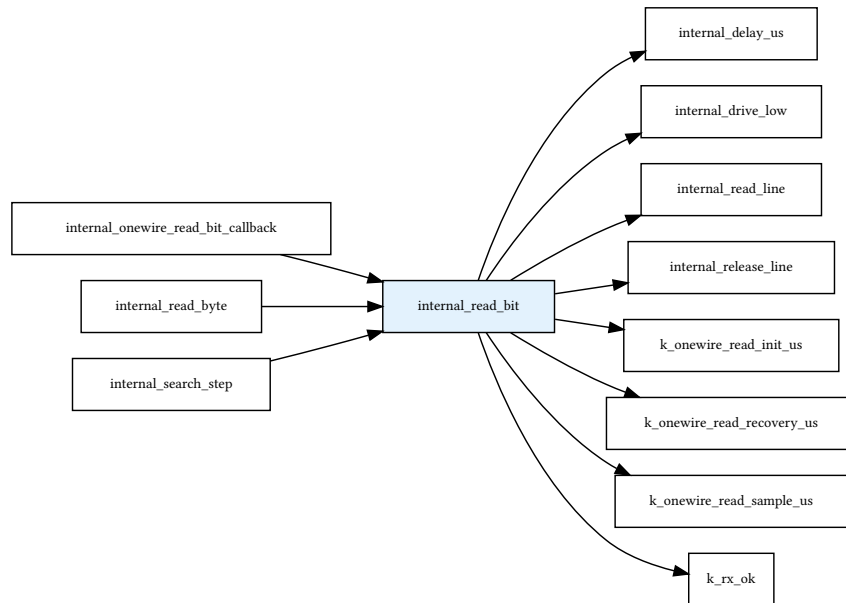


Figure 85: Call graph for `internal_read_bit`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:621_`

`internal_write_byte`

```
static rx_err_t internal_write_byte(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, const uint8_t byte)
```

Write a byte (LSB first).

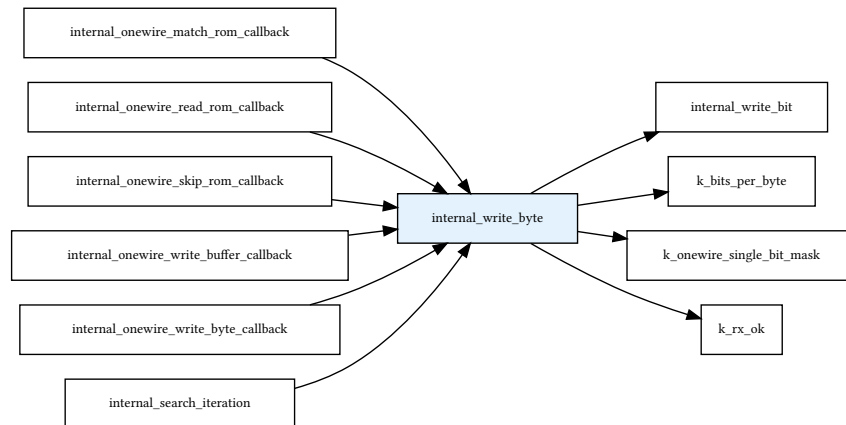


Figure 86: Call graph for `internal_write_byte`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:657_`

internal_read_byte

```
static rx_err_t internal_read_byte(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, uint8_t *byte)
```

Read a byte (LSB first).

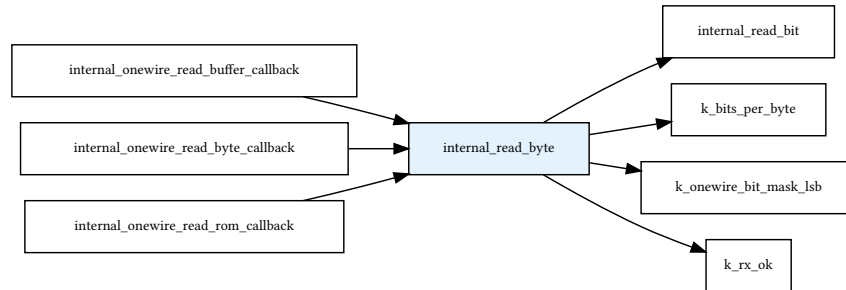


Figure 87: Call graph for `internal_read_byte`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:674_`

internal_search_step

```
static rx_err_t internal_search_step(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, uint8_t rom[k_owewire_rom_bytes], const uint8_t
bit_number, uint8_t *rom_byte_index, uint8_t *rom_bit_mask, uint8_t *last_zero)
```

Process one ROM search bit position.

Parameters:

Direction	Name	Description
in	<code>bus_config</code>	Bus configuration node
in	<code>state</code>	Runtime OneWire state
inout	<code>rom</code>	ROM buffer under construction
in	<code>bit_number</code>	Current bit index (1..64)
inout	<code>rom_byte_index</code>	Current ROM byte index
inout	<code>rom_bit_mask</code>	Current ROM bit mask
inout	<code>last_zero</code>	Last discrepancy bit index

Returns: `k_rx_ok` on success, error code on failure

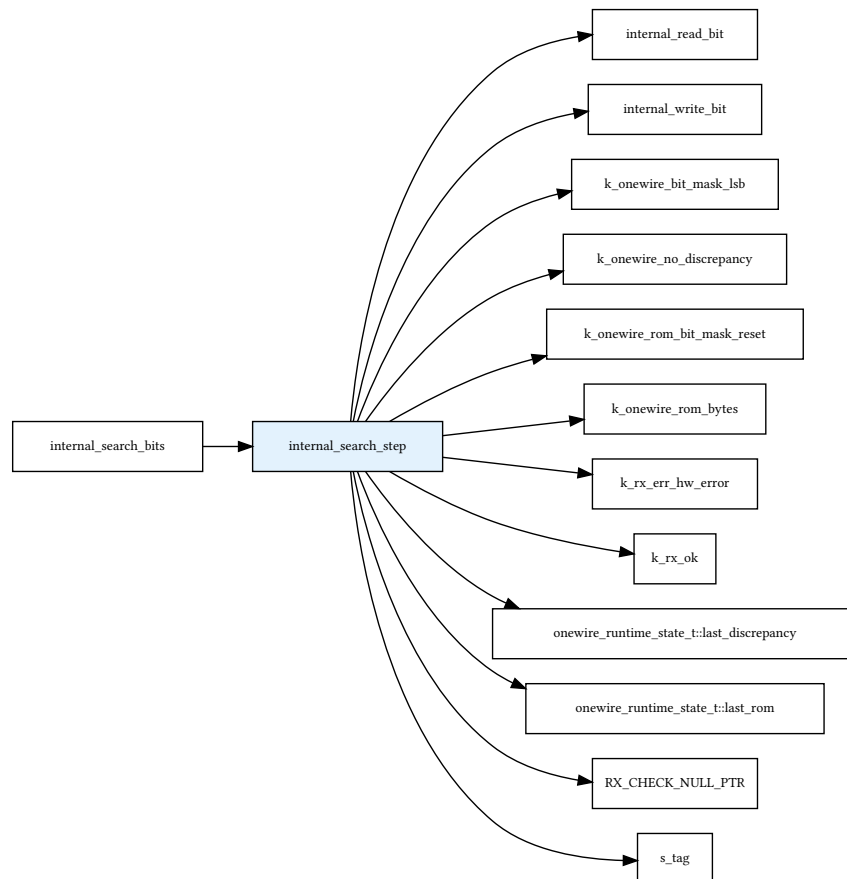


Figure 88: Call graph for internal_search_step

_Defined in libs/rx_bus/src/rx_bus_onewire.c:705_

internal_search_bits

```
static rx_err_t internal_search_bits(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, uint8_t rom[k_owewire_rom_bytes], uint8_t
*last_zero)
```

Execute the ROM search bit loop.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
in	state	Runtime OneWire state
out	rom	ROM buffer under construction

out	last_zero	Last discrepancy bit index
-----	-----------	----------------------------

Returns: k_rx_ok on success, error code on failure

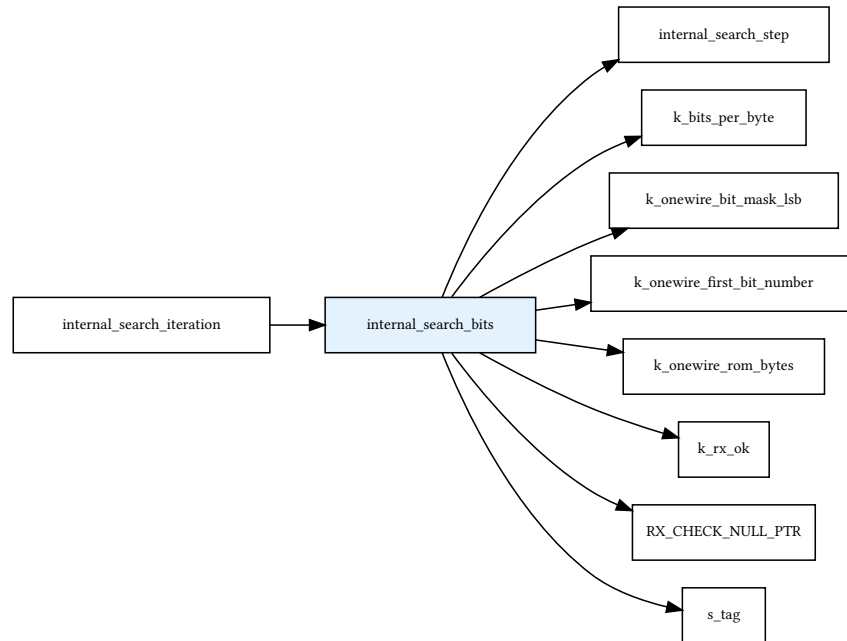


Figure 89: Call graph for `internal_search_bits`

_Defined in `libs/rx\bus/src/rx\bus\onewire.c:775_`

internal_search_iteration

```
static rx_err_t internal_search_iteration(rx_bus_config_t *bus_config,
onewire_runtime_state_t *state, uint8_t rom[k_owewire_rom_bytes], bool
*device_found)
```

Execute one iteration of the 1-Wire ROM search algorithm.

Performs a single pass of the binary tree ROM search algorithm. Each call discovers one device ROM by traversing the binary tree of device addresses. Tracks search state (discrepancy points) to enumerate all devices.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	state	Runtime state with search tracking
out	rom	8-byte buffer for discovered ROM
out	device_found	Set to true if valid ROM discovered

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success (check device_found for result)
k_rx_err_crc_mismatch	ROM CRC validation failed
k_rx_err_hw_error	Bus collision detected
k_rx_err_*	Other bus/GPIO errors

Pre: bus_config != nullptr

Pre: state != nullptr

Pre: rom != nullptr

Pre: device_found != nullptr

Post: On success with device_found: rom contains valid 64-bit address

Post: state->last_rom updated for next iteration

Post: state->last_device_flag set when no more devices

Note: Call repeatedly until device_found == false

Note: Uses CRC-8 (Maxim/Dallas polynomial) for ROM validation

Search Algorithm:: The 64-bit ROM forms a binary tree. At each bit position: Read bit and complement from all devices If discrepancy (0,0): choose direction based on last_discrepancy If no discrepancy: follow the bit value Write chosen direction to select devices Track discrepancy points for subsequent iterations

Algorithm Steps:: Check last_device_flag (return if all found) Issue reset, check presence Send SEARCH_ROM command (0xF0) For each of 64 bits: internal_search_step() Update last_discrepancy for next iteration Verify ROM CRC-8 Copy ROM to last_rom for next iteration

See also: rx_bus_onewire_search() Public API using this

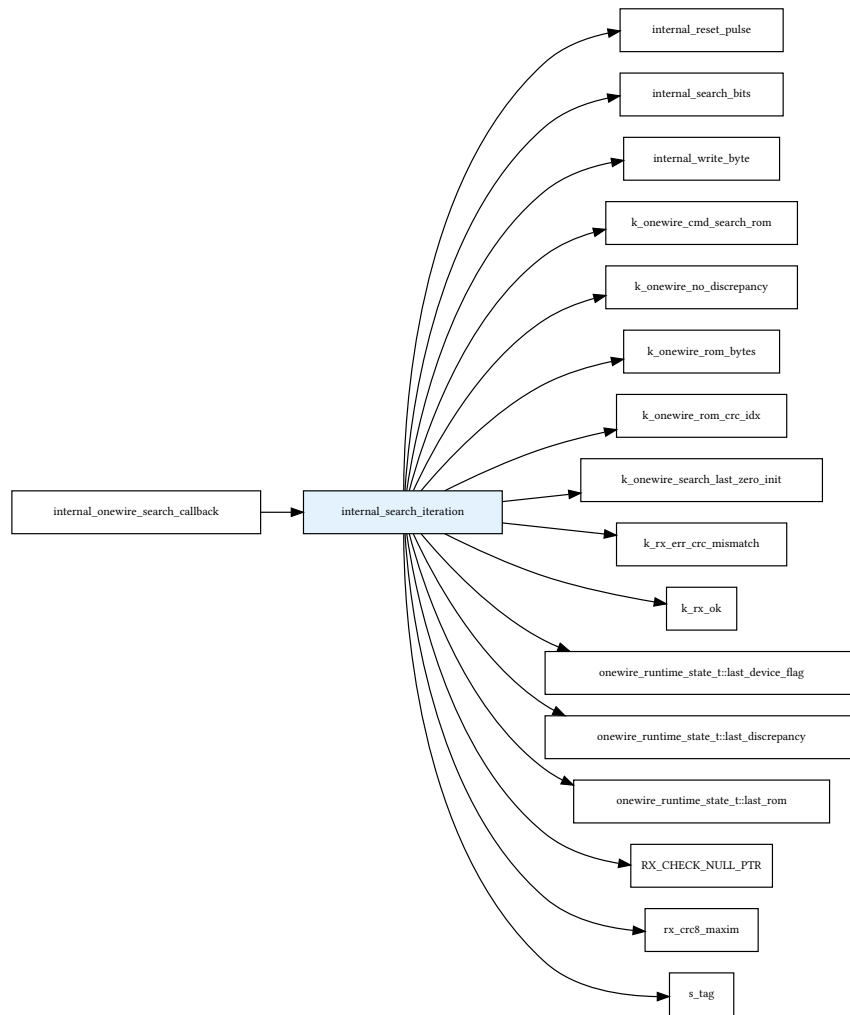
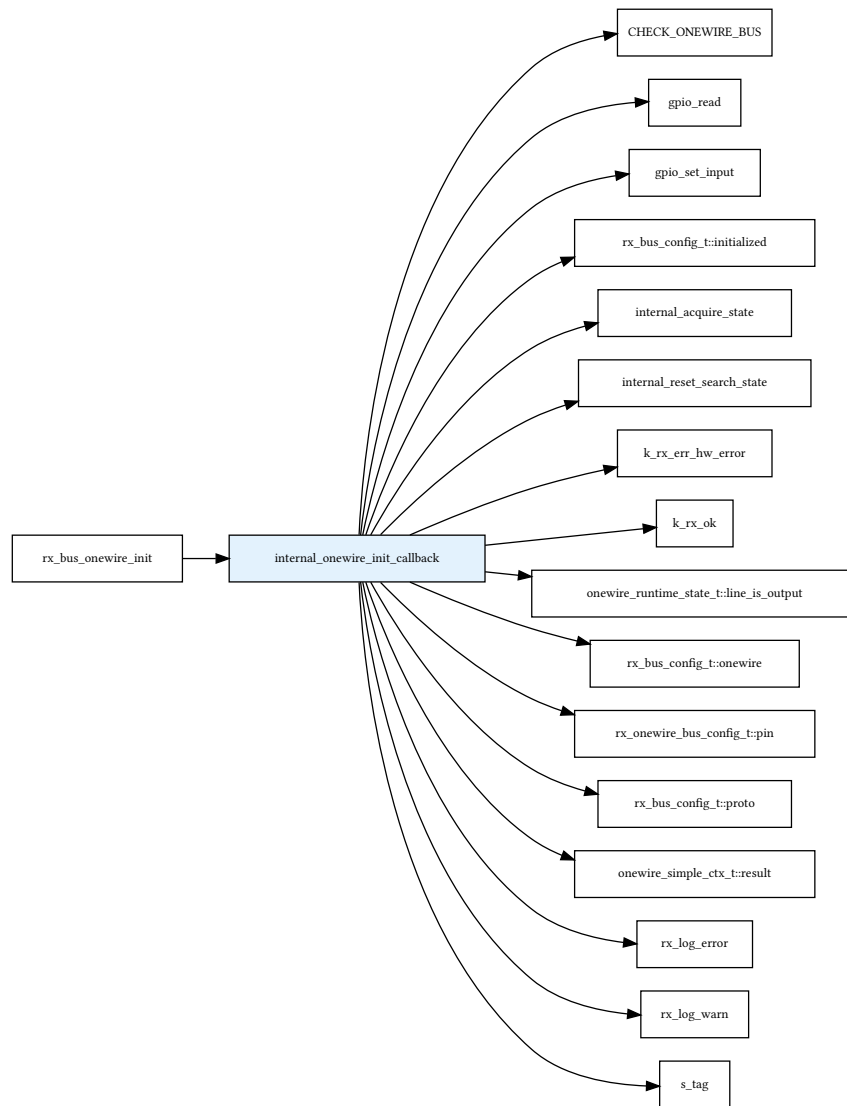


Figure 90: Call graph for internal_search_iteration

_Defined in libs/rx_bus/src/rx_bus_onewire.c:854_

internal_owire_init_callback

```
static rx_err_t internal_owire_init_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Figure 91: Call graph for `internal_owire_init_callback`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1003_`

internal_release_state

```
static void internal_release_state(rx_bus_config_t *bus_config)
```

Release a state pool entry for a given bus config handle.

Scans the static state pool for the entry matching the bus config's handle pointer. Clears the `in_use` flag and nulls the handle so the slot can be reused by a future `rx_bus_owire_init()` call.

Parameters:

Direction	Name	Description
inout	bus_config	Bus configuration whose state to release

Pre: bus_config != nullptr

Post: Matching pool entry marked !in_use

Post: bus_config->handle == nullptr

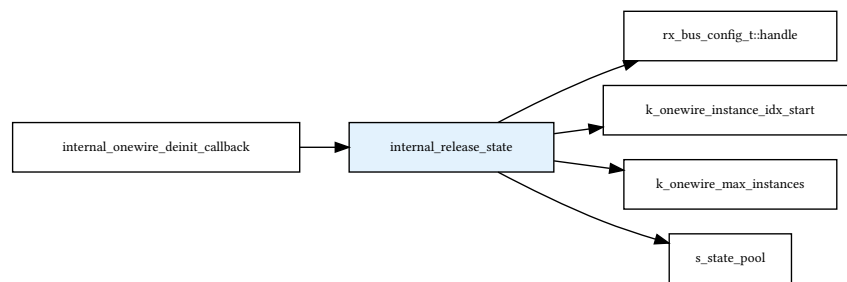


Figure 92: Call graph for internal_release_state

_Defined in libs/rx_bus/src/rx_bus_onewire.c:1058_

internal_owewire_deinit_callback

```
static rx_err_t internal_owewire_deinit_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback for OneWire bus deinitialization.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Simple context with result field

Returns: k_rx_ok on success, error code on failure

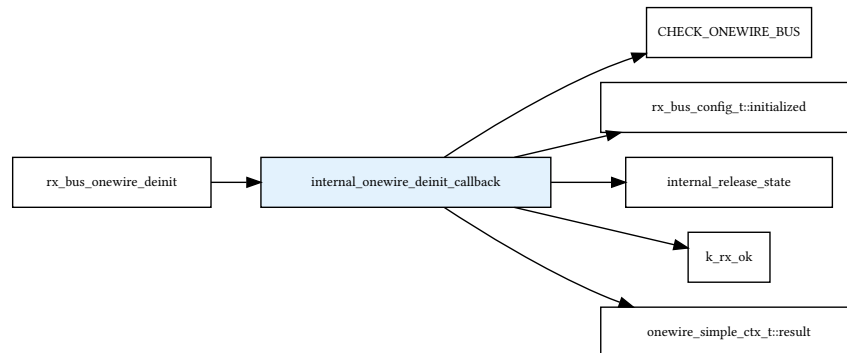


Figure 93: Call graph for internal_owewire_deinit_callback

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1084_`

internal_owewire_reset_callback

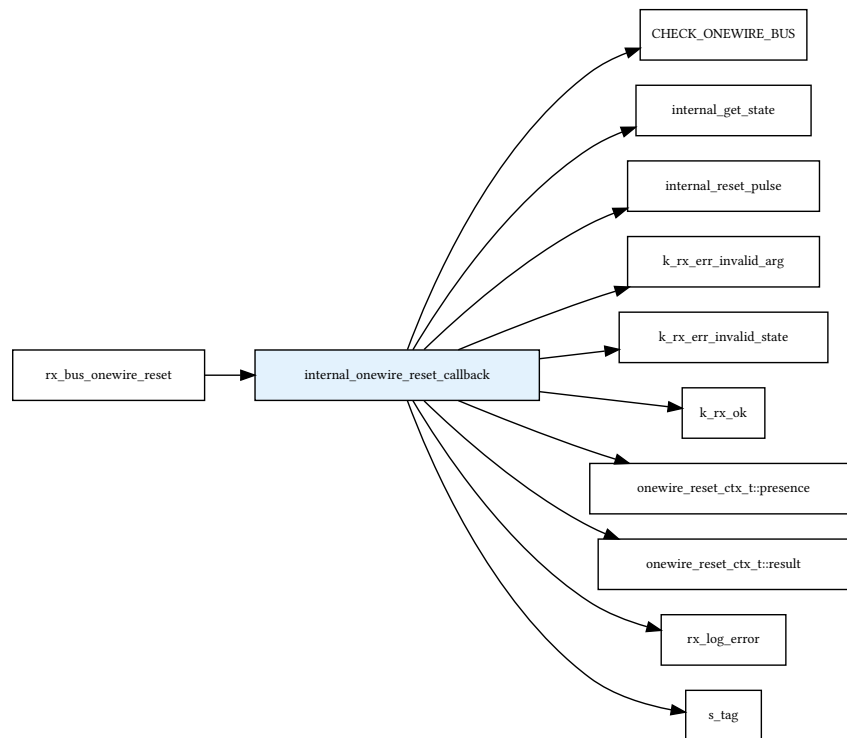
```
static rx_err_t internal_owewire_reset_callback(rx_bus_config_t *bus_config, void
*user_ctx)
```

Callback to perform OneWire reset pulse and detect presence.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing presence output pointer

Returns: `k_rx_ok` on success, error code on failure

Figure 94: Call graph for `internal_owewire_reset_callback`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:1105_`

`internal_owewire_write_bit_callback`

```
static rx_err_t internal_owewire_write_bit_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback to write a single bit on the OneWire bus.

Parameters:

Direction	Name	Description
in	<code>bus_config</code>	Bus configuration node
inout	<code>user_ctx</code>	Context containing bit value to write

Returns: `k_rx_ok` on success, error code on failure

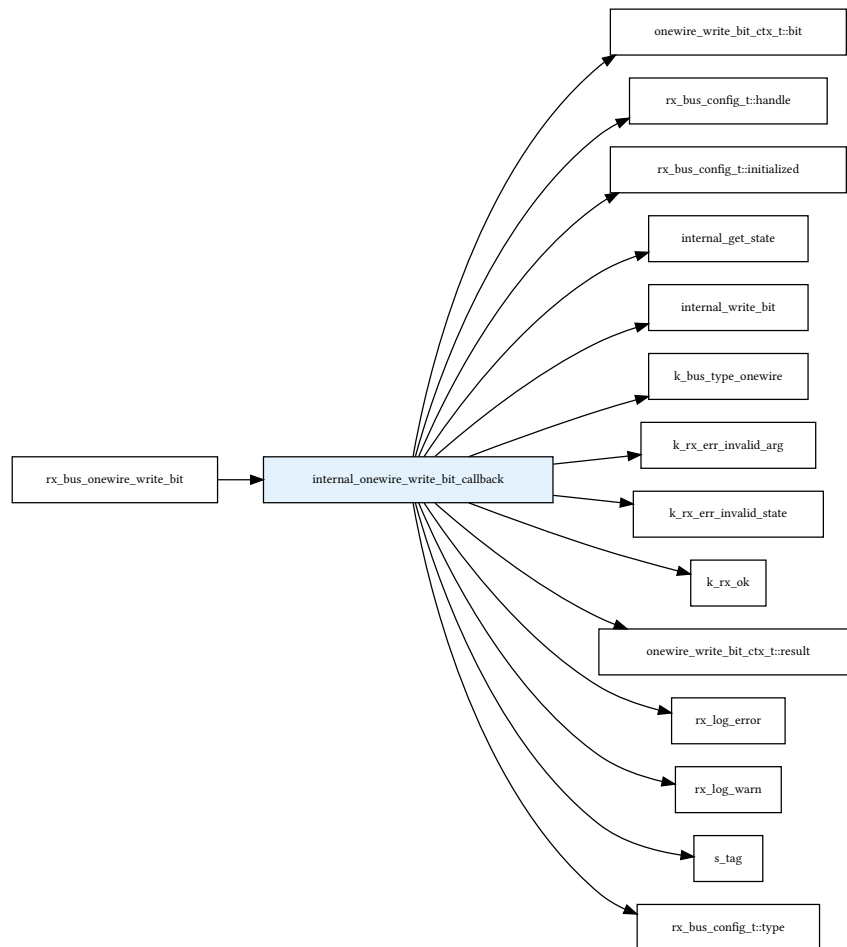


Figure 95: Call graph for internal_owewire_write_bit_callback

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1146_

internal_owewire_read_bit_callback

```
static rx_err_t internal_owewire_read_bit_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback to read a single bit from the OneWire bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing bit output pointer

Returns: k_rx_ok on success, error code on failure

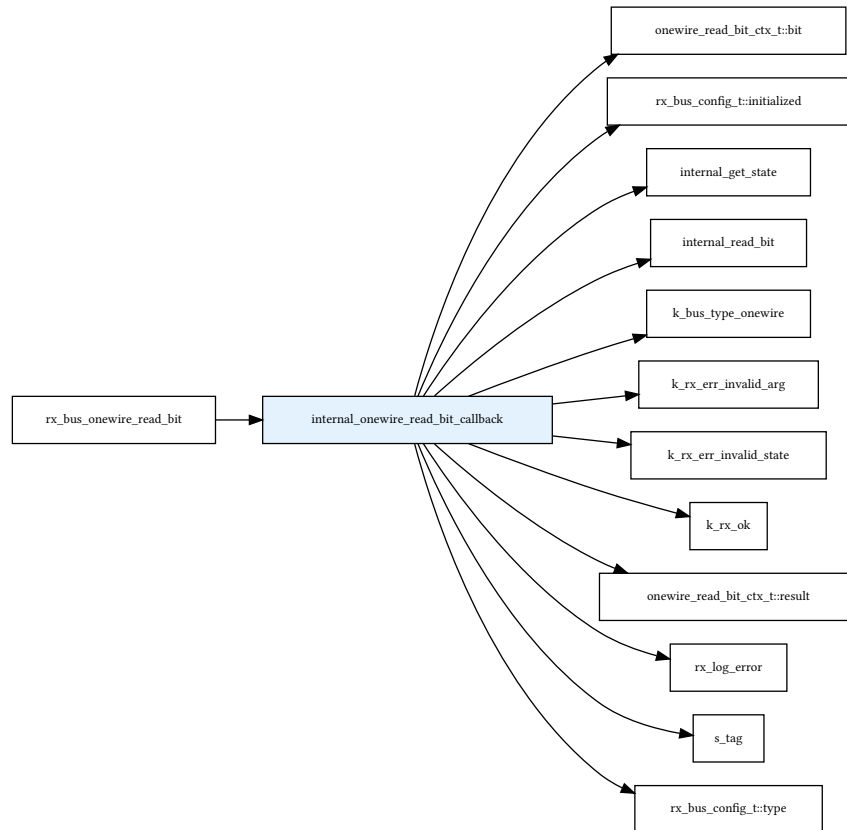


Figure 96: Call graph for internal_owire_read_bit_callback

_Defined in libs/rx_bus/src/rx_bus_onewire.c:1194_

internal_owire_write_byte_callback

```
static rx_err_t internal_owire_write_byte_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback to write a single byte on the OneWire bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing byte value to write

Returns: k_rx_ok on success, error code on failure

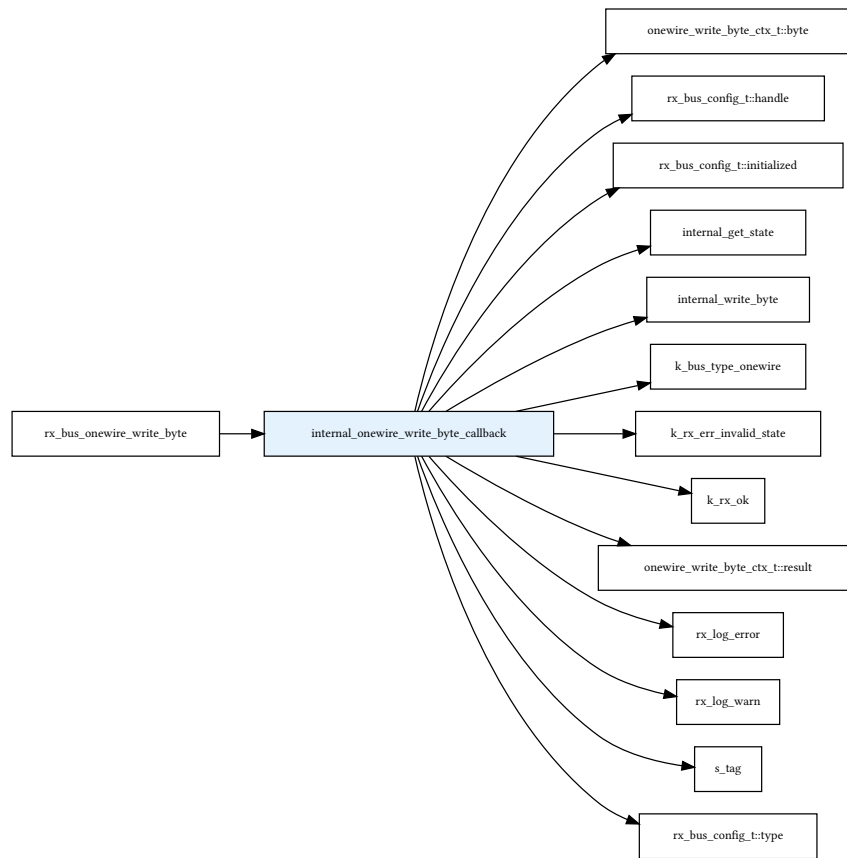


Figure 97: Call graph for internal_owewire_write_byte_callback

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1241_

internal_owewire_read_byte_callback

```
static rx_err_t internal_owewire_read_byte_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback to read a single byte from the OneWire bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing byte output pointer

Returns: k_rx_ok on success, error code on failure

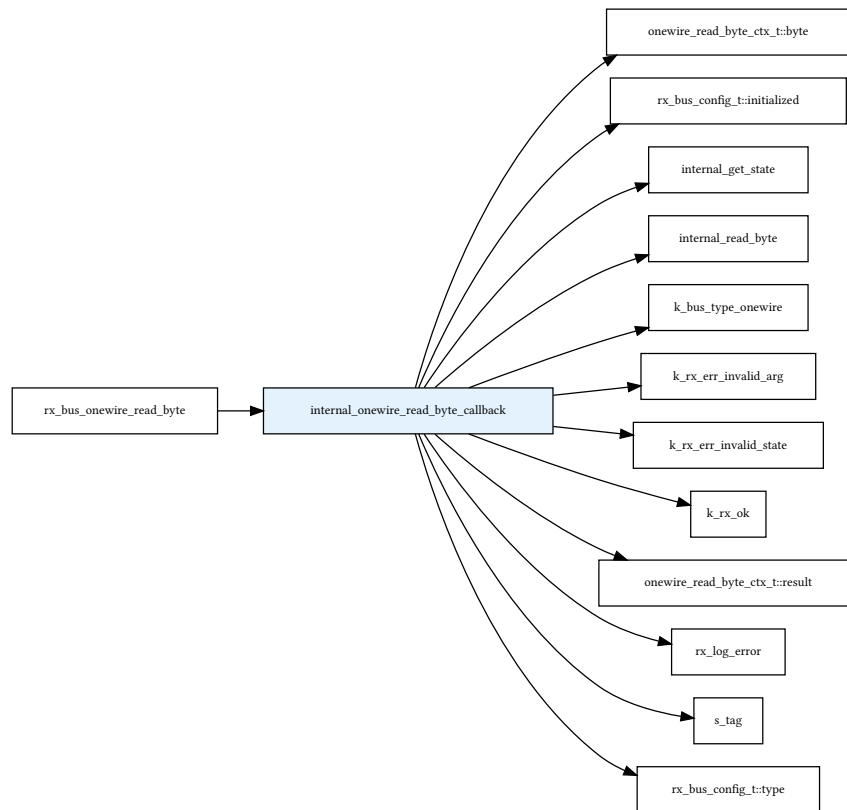


Figure 98: Call graph for internal_owewire_read_byte_callback

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1282_

internal_owewire_write_buffer_callback

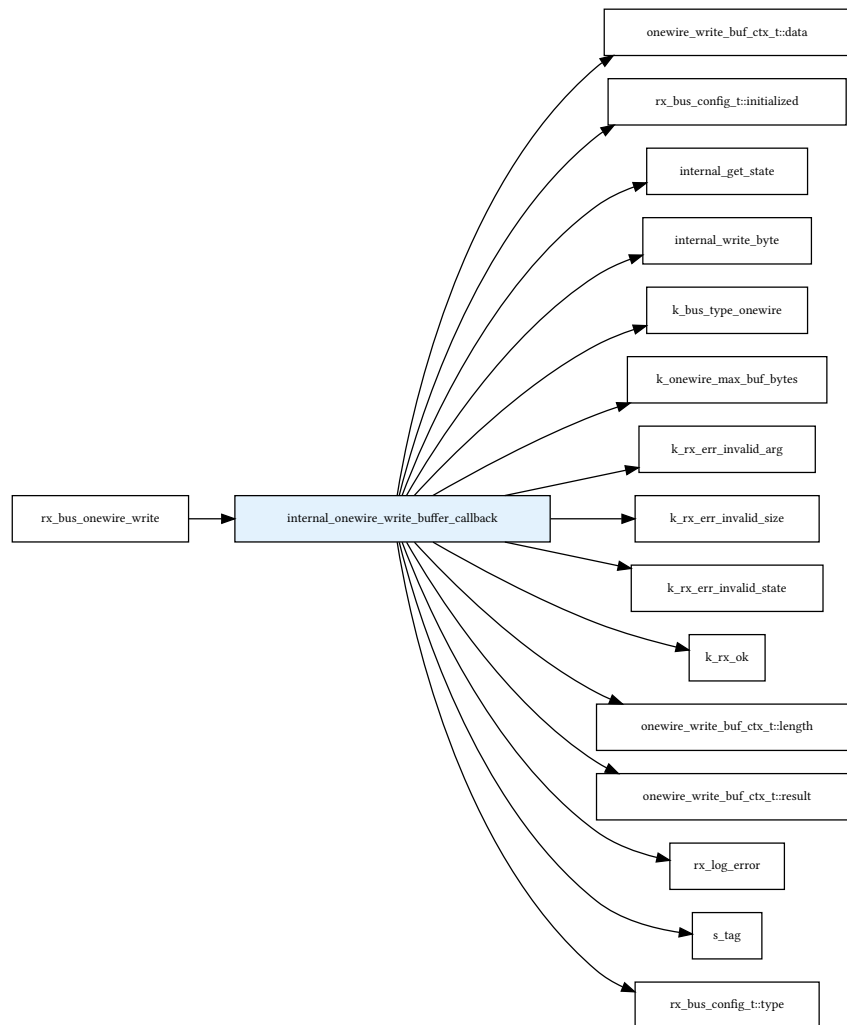
```
static rx_err_t internal_owewire_write_buffer_callback(rx_bus_config_t
*bus_config, void *user_ctx)
```

Callback to write a buffer of bytes on the OneWire bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing data pointer and length

Returns: k_rx_ok on success, error code on failure

Figure 99: Call graph for `internal_owewire_write_buffer_callback`

_Defined in `libs/rx/_bus/src/rx/_bus/_owewire.c:1324_`

`internal_owewire_read_buffer_callback`

```
static rx_err_t internal_owewire_read_buffer_callback(rx_bus_config_t
*bus_config, void *user_ctx)
```

Callback to read a buffer of bytes from the OneWire bus.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	bus_config	Bus configuration node
inout	user_ctx	Context containing data pointer and length

Returns: k_rx_ok on success, error code on failure

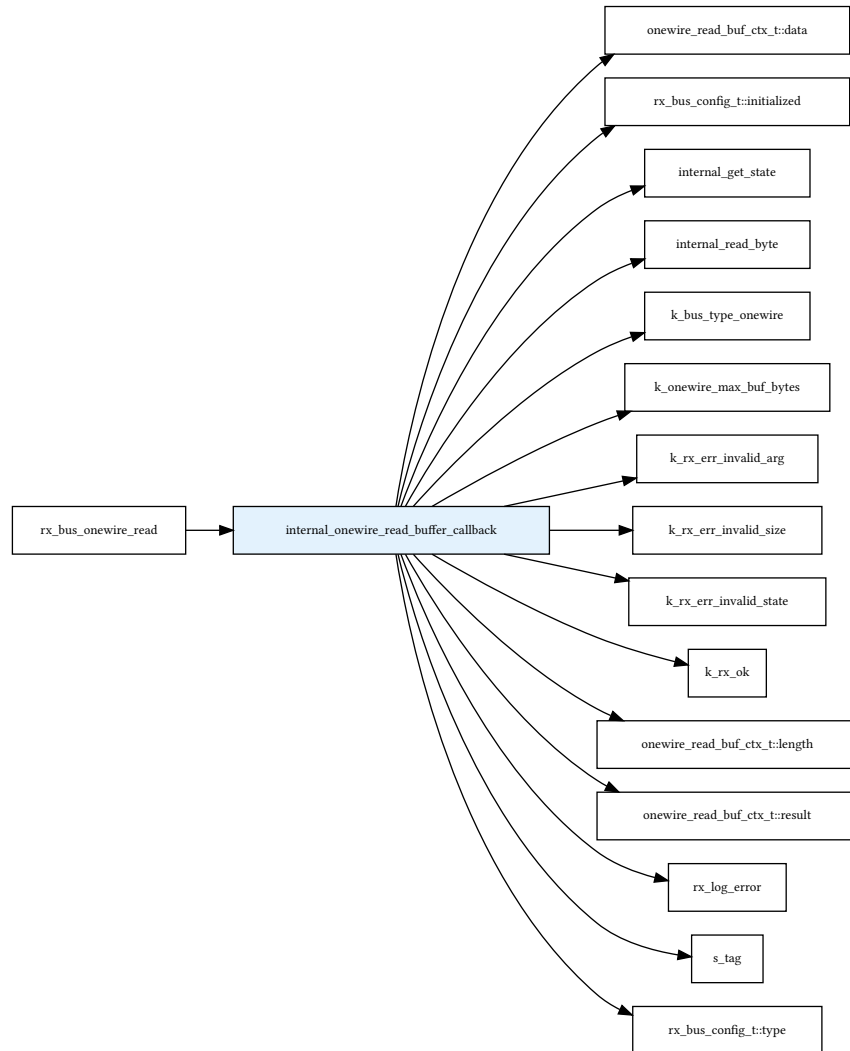


Figure 100: Call graph for `internal_owire_read_buffer_callback`

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:1379_`

internal_owewire_skip_rom_callback

```
static rx_err_t internal_owewire_skip_rom_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback to issue OneWire Skip ROM command.

Sends the Skip ROM command (0xCC) to address all devices on the bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
in	user_ctx	User context (unused)

Returns: k_rx_err_not_found if no device presence detected

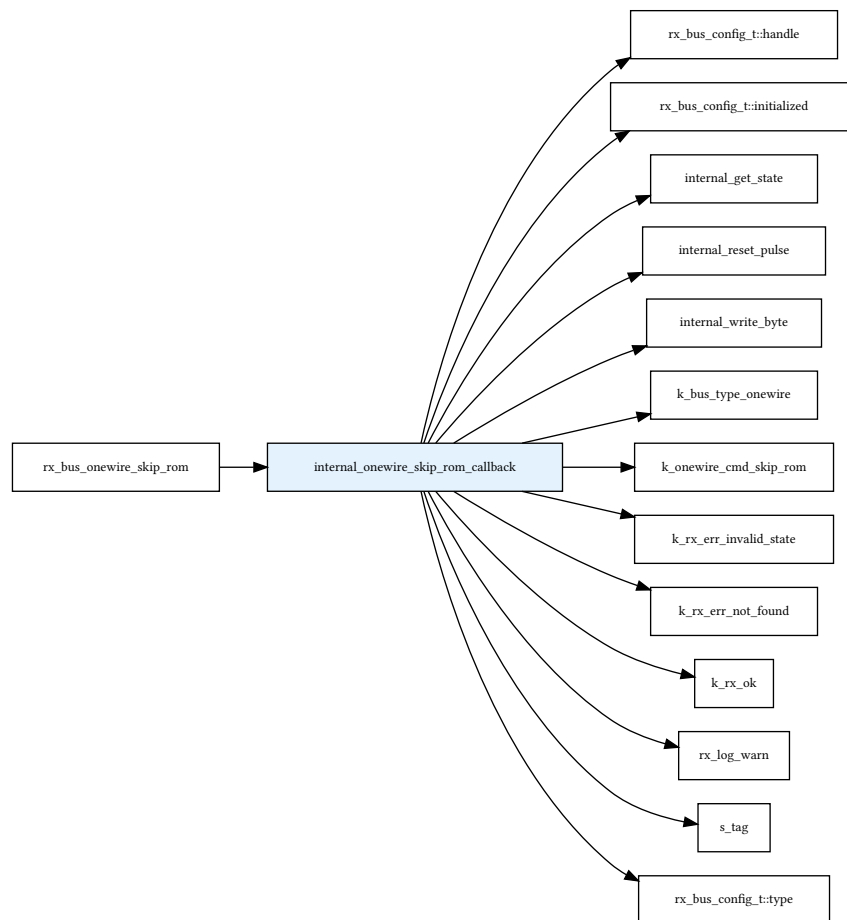


Figure 101: Call graph for `internal_owewire_skip_rom_callback`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1449_`

—

internal_onewire_match_rom_callback

```
static rx_err_t internal_onewire_match_rom_callback(rx_bus_config_t *bus_config,  
void *user_ctx)
```

Callback to issue OneWire Match ROM command.

Sends the Match ROM command (0x55) followed by 64-bit ROM address to select a specific device on a multi-device bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing ROM address

Returns: k_rx_err_not_found if no device presence detected

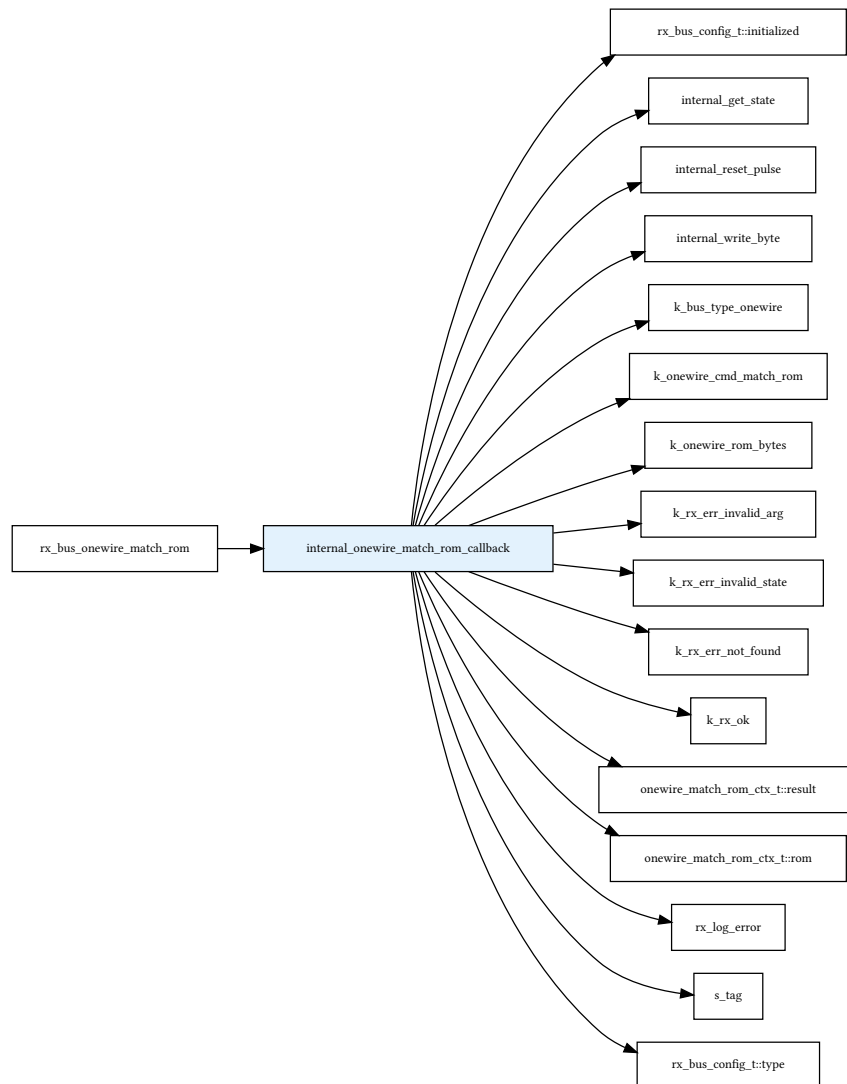


Figure 102: Call graph for `internal_owewire_match_rom_callback`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1499_`

internal_owewire_read_rom_callback

```
static rx_err_t internal_owewire_read_rom_callback(rx_bus_config_t *bus_config,
void *user_ctx)
```

Callback to read ROM address from a single OneWire device.

Sends the Read ROM command (0x33) and reads 64-bit ROM address with CRC. Only works when exactly one device is on the bus.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing ROM output buffer

Returns: k_rx_err_crc_mismatch if ROM CRC validation fails

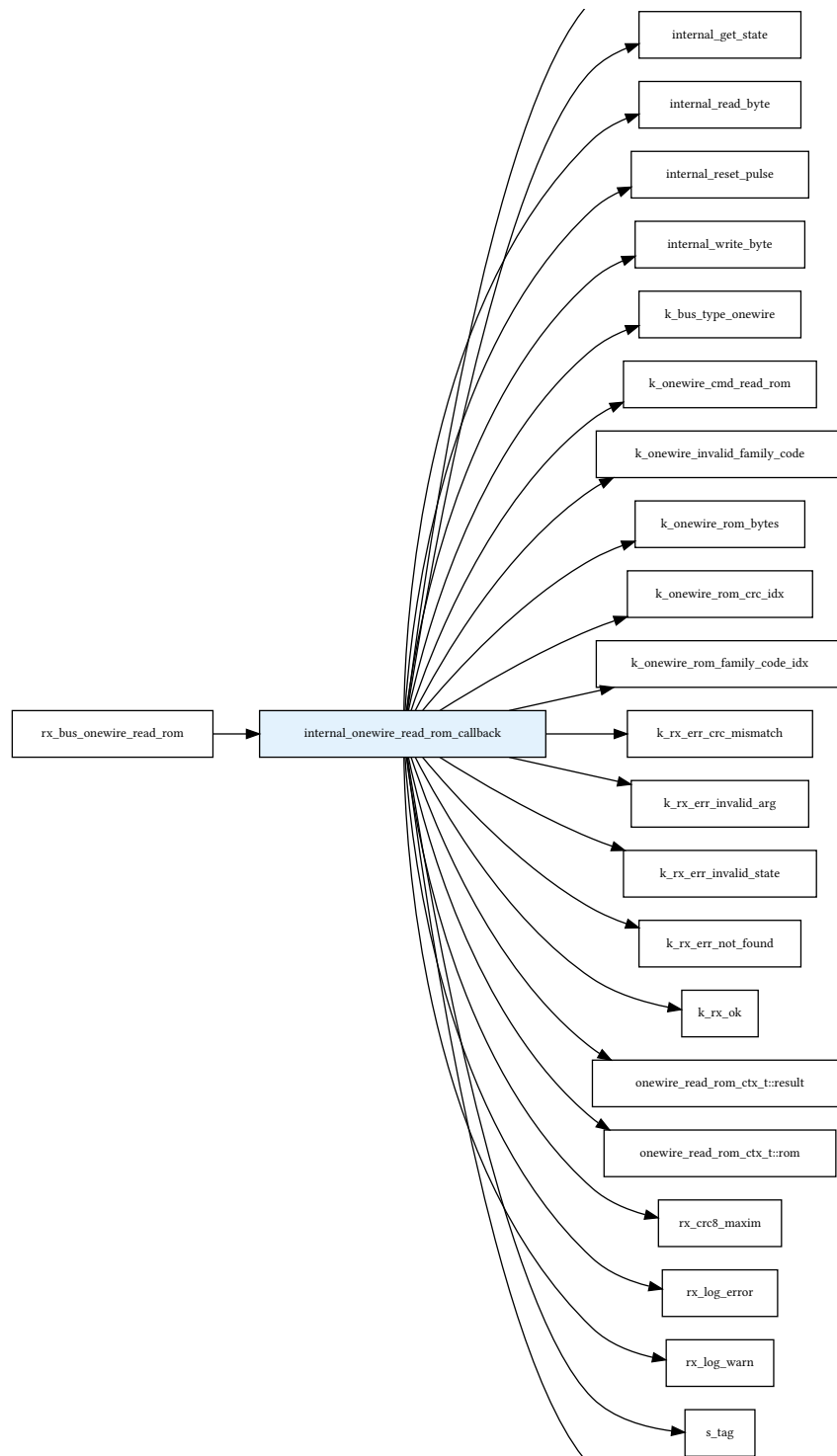


Figure 103: Call graph for internal_owire_read_rom_callback

_Defined in libs/rx_bus/src/rx_bus_onewire.c:1564_
—

internal_onewire_search_callback

```
static rx_err_t internal_onewire_search_callback(rx_bus_config_t *bus_config,  
void *user_ctx)
```

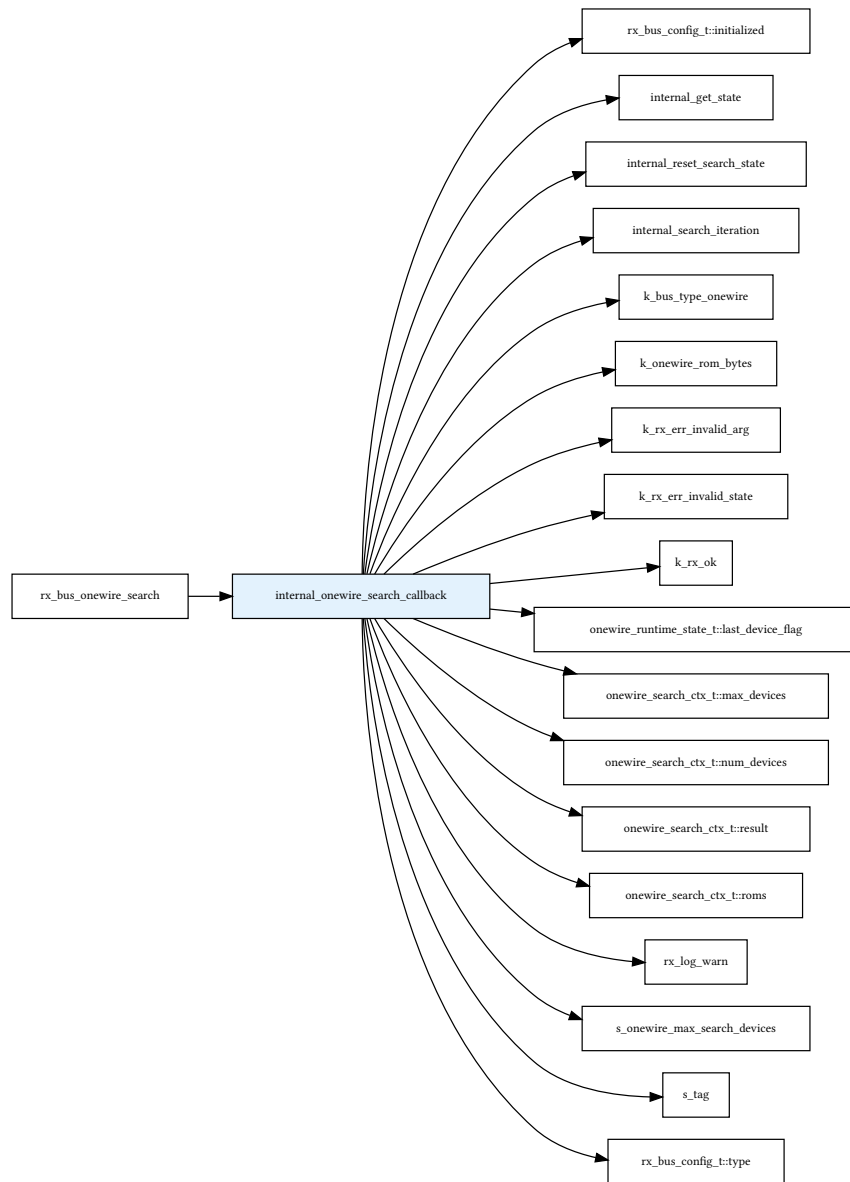
Callback to search for all OneWire devices on the bus.

Implements the OneWire ROM search algorithm to discover all devices on a multi-device bus.
Returns array of 64-bit ROM addresses with CRC.

Parameters:

Direction	Name	Description
in	bus_config	Bus configuration node
inout	user_ctx	Context containing ROM buffer, max count, and output count

Returns: k_rx_err_invalid_arg if parameters invalid or max_devices exceeded

Figure 104: Call graph for `internal_owewire_search_callback`

Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1643`

`rx_bus_owewire_init`

```
rx_err_t rx_bus_owewire_init(rx_bus_manager_t *manager, const char *bus_name)
```

Initialize a 1-Wire bus instance.

Initialize OneWire bus through bus manager.

Initializes a 1-Wire bus by configuring the GPIO pin as input (released) and allocating runtime state from the static pool. Verifies pullup resistor is present by checking line reads high after release.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name (must match registered bus)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, bus ready for use
k_rx_err_null_ptr	nullptr manager or bus_name
k_rx_err_not_found	Bus name not registered
k_rx_err_invalid_arg	Bus is not 1-Wire type
k_rx_err_no_mem	Static state pool exhausted
k_rx_err_hw_error	GPIO configuration failed

Pre: manager != nullptr and initialized

Pre: bus_name registered via rx_bus_manager_register()

Pre: 4.7kOhm pullup resistor connected to data line

Post: Bus ready for reset, read, write operations

Note: Logs warning if pullup not detected (may still work)

Algorithm:: Validate manager and bus_name pointers
Acquire runtime state from pool
Configure GPIO as input (high-Z, pullup drives high)
Verify line reads high (warn if not)
Reset search state
Mark bus as initialized

Example:: rx_err_t err=rx_bus_owewire_init(&bus_manager,"temp_sensor"); if(err!=k_rx_ok)
{ rx_log_error("1W","Initfailed"); }

See also: rx_bus_owewire_reset() First operation after init

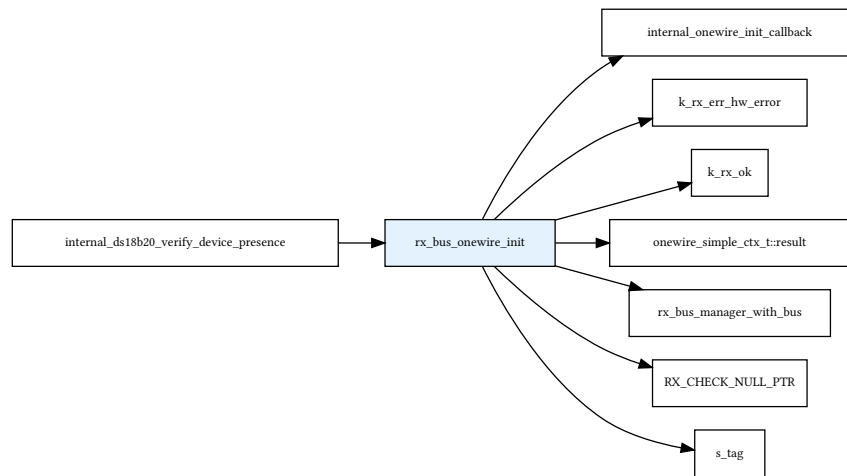


Figure 105: Call graph for rx_bus_owewire_init

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1764_

rx_bus_owewire_deinit

```
rx_err_t rx_bus_owewire_deinit(rx_bus_manager_t *manager, const char *bus_name)
```

Deinitialize OneWire bus and release state pool entry.

Releases the runtime state allocated by rx_bus_owewire_init() back to the static pool. Prevents state pool exhaustion in long-running test suites.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	manager or bus_name is nullptr
k_rx_err_not_found	Bus not registered
k_rx_err_invalid_arg	Bus is not OneWire type

Pre: Bus was initialized via rx_bus_owewire_init()

Post: State pool entry released for reuse

Post: bus_config->initialized == false] **See also:** rx_bus_owewire_init() Initialization counterpart

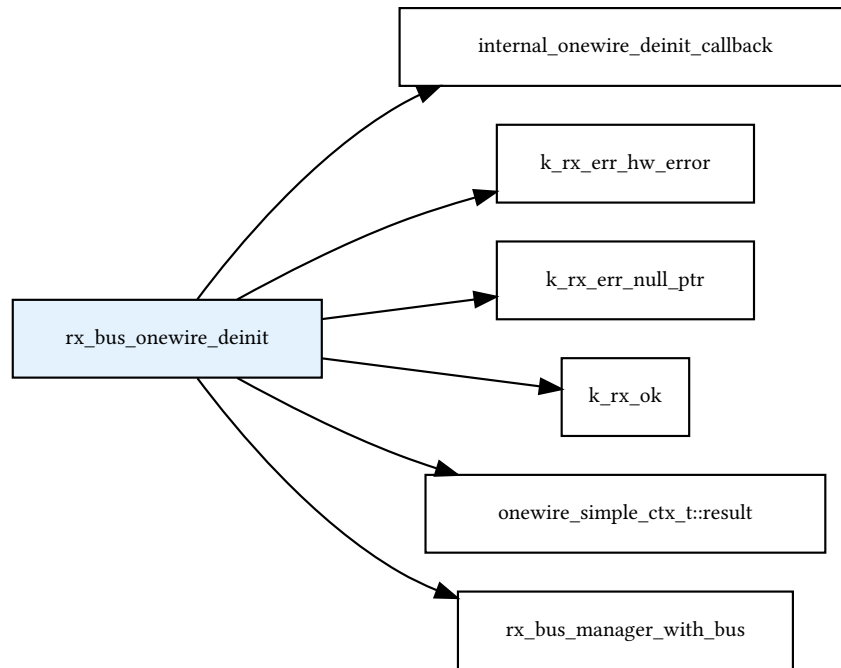


Figure 106: Call graph for rx_bus_owewire_deinit

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1800_

—

rx_bus_owewire_reset

```
rx_err_t rx_bus_owewire_reset(rx_bus_manager_t *manager, const char *bus_name,
bool *presence)
```

Perform OneWire reset pulse and detect device presence.

Perform OneWire reset and check for presence pulse.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name
out	presence	True if device responded, false otherwise

Returns: k_rx_ok on success, error code on failure

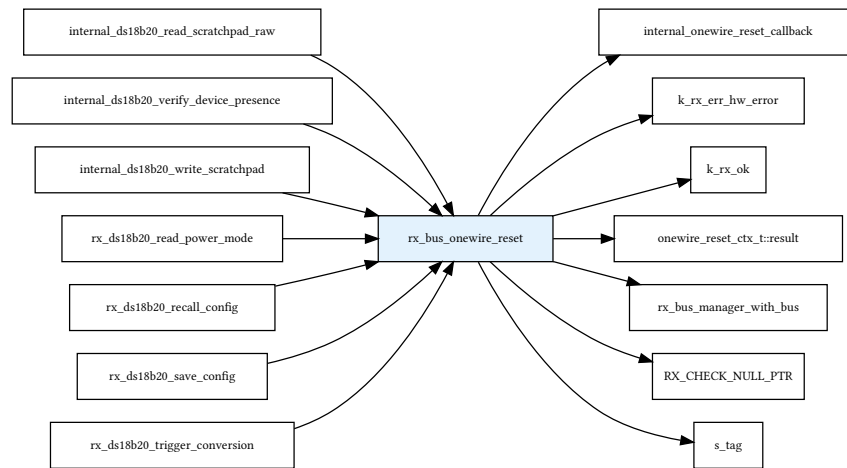


Figure 107: Call graph for rx_bus_owewire_reset

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1827_

rx_bus_owewire_write_bit

```
rx_err_t rx_bus_owewire_write_bit(rx_bus_manager_t *manager, const char
*bus_name, bool bit)
```

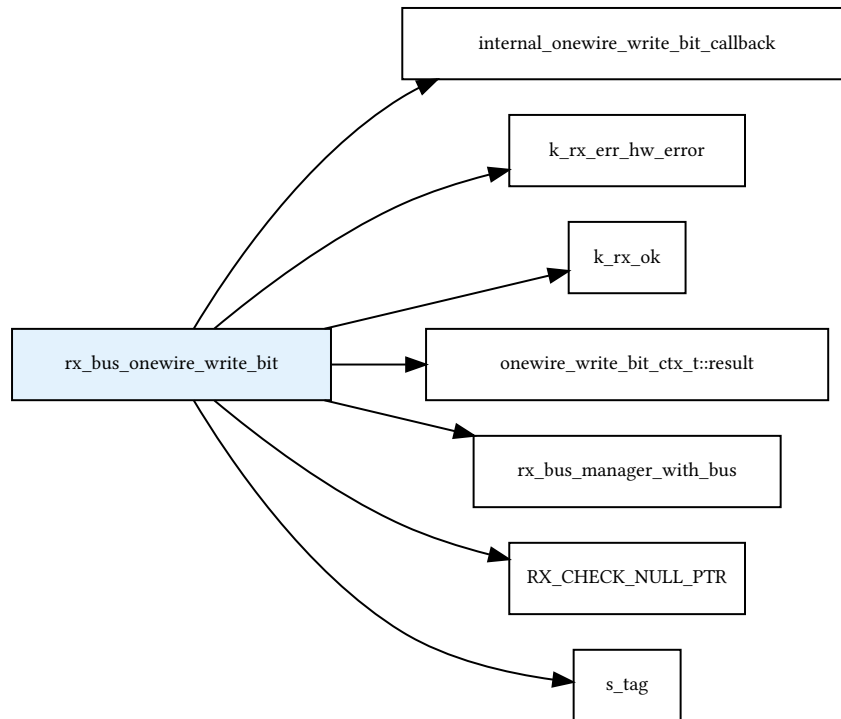
Write a single bit on the OneWire bus.

Write a single bit to OneWire bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name
in	bit	Bit value to write (true=1, false=0)

Returns: k_rx_ok on success, error code on failure

Figure 108: Call graph for `rx_bus_owewire_write_bit`

_Defined in `libs/rx_bus/src/rx_bus_owewire.c:1851_`

rx_bus_owewire_read_bit

```
rx_err_t rx_bus_owewire_read_bit(rx_bus_manager_t *manager, const char *bus_name,
bool *bit)
```

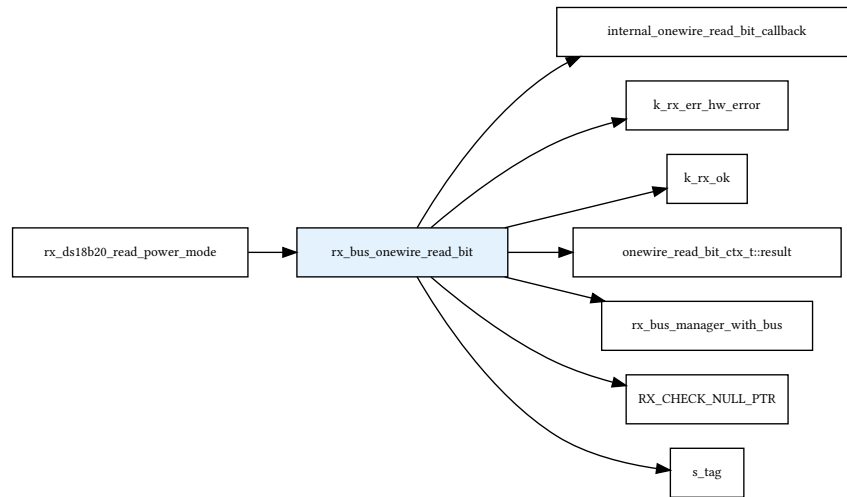
Read a single bit from the OneWire bus.

Read a single bit from OneWire bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name
out	bit	Bit value read (true=1, false=0)

Returns: `k_rx_ok` on success, error code on failure

Figure 109: Call graph for `rx_bus_owewire_read_bit`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1874_`

rx_bus_owewire_write_byte

```
rx_err_t rx_bus_owewire_write_byte(rx_bus_manager_t *manager, const char
*bus_name, uint8_t byte)
```

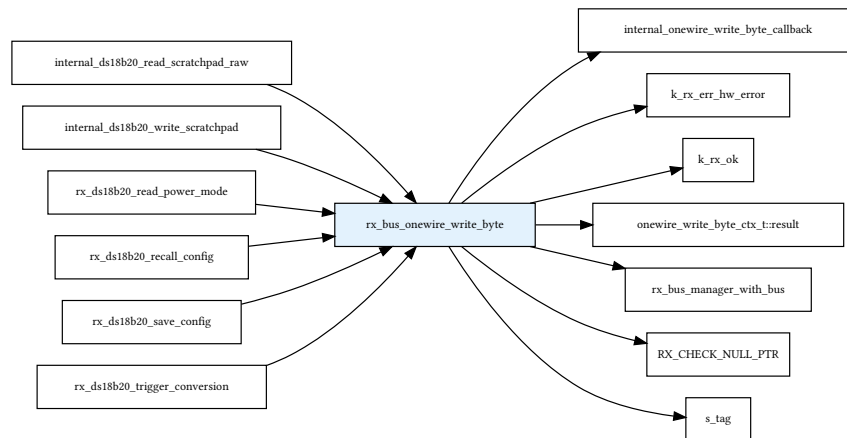
Write a single byte on the OneWire bus (LSB first).

Write a byte to OneWire bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name
in	byte	Byte value to write

Returns: `k_rx_ok` on success, error code on failure

Figure 110: Call graph for `rx_bus_owewire_write_byte`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1898_`

`rx_bus_owewire_read_byte`

```
rx_err_t rx_bus_owewire_read_byte(rx_bus_manager_t *manager, const char
*bus_name, uint8_t *byte)
```

Read a single byte from the OneWire bus (LSB first).

Read a byte from OneWire bus.

Parameters:

Direction	Name	Description
in	<code>manager</code>	Bus manager handle
in	<code>bus_name</code>	Bus configuration name
out	<code>byte</code>	Byte value read

Returns: `k_rx_ok` on success, error code on failure

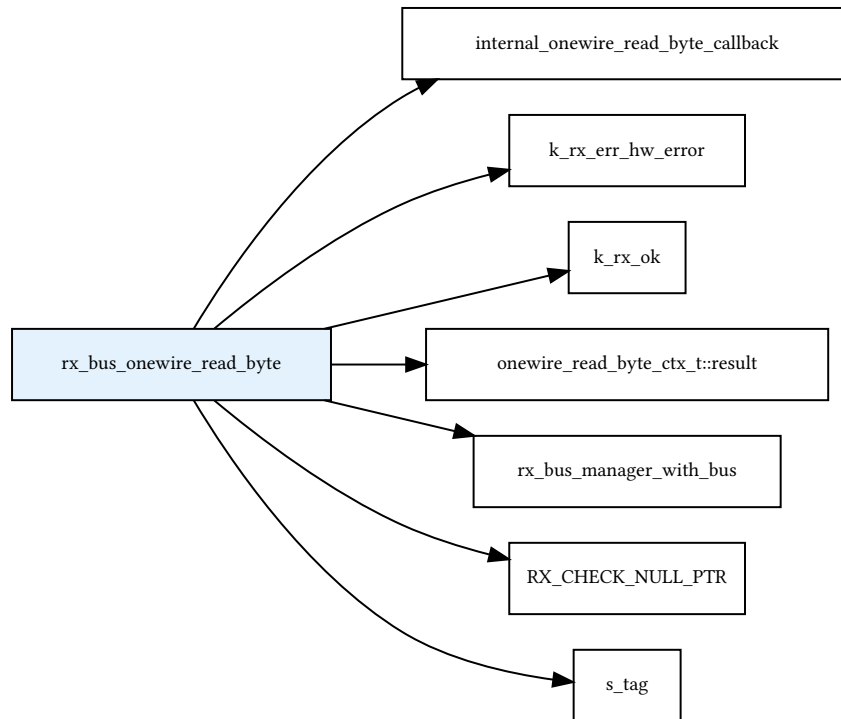


Figure 111: Call graph for rx_bus_owewire_read_byte

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1921_

rx_bus_owewire_write

```
rx_err_t rx_bus_owewire_write(rx_bus_manager_t *manager, const char *bus_name,
const uint8_t *data, const uint32_t length)
```

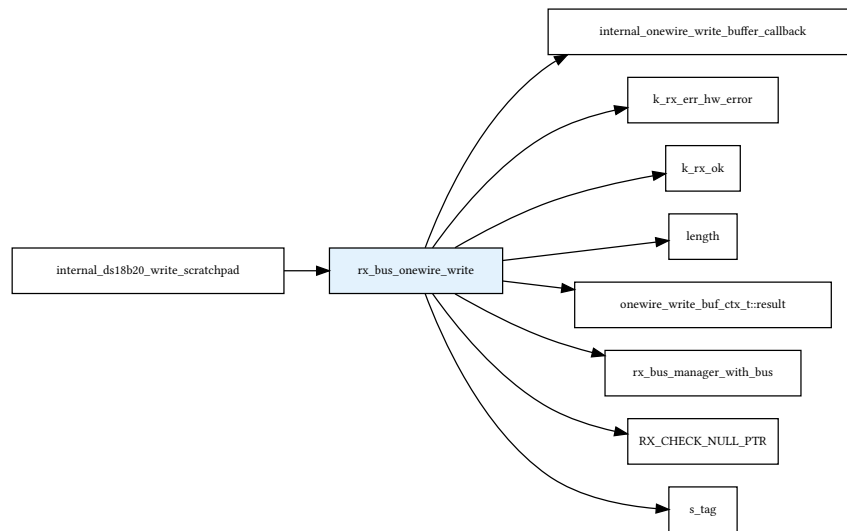
Write multiple bytes to OneWire bus.

Convenience function for writing buffers.

Parameters:

Direction	Name	Description
in	manager	Bus manager instance
in	bus_name	OneWire bus name
in	data	Pointer to data buffer
in	length	Number of bytes to write

Returns: k_rx_err_timeout if mutex timeout

Figure 112: Call graph for `rx_bus_owewire_write`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:1936_`

`rx_bus_owewire_read`

```
rx_err_t rx_bus_owewire_read(rx_bus_manager_t *manager, const char *bus_name,
uint8_t *data, uint32_t length)
```

Read multiple bytes from OneWire bus.

Convenience function for reading buffers.

Parameters:

Direction	Name	Description
in	<code>manager</code>	Bus manager instance
in	<code>bus_name</code>	OneWire bus name
out	<code>data</code>	Pointer to receive buffer
in	<code>length</code>	Number of bytes to read

Returns: `k_rx_err_timeout` if mutex timeout

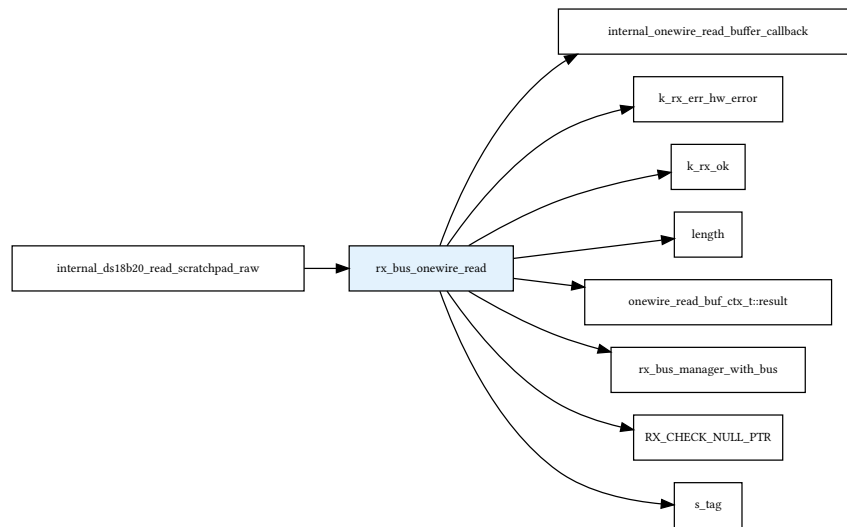


Figure 113: Call graph for rx_bus_owewire_read

_Defined in `libs/rx\_bus/src/rx\_bus\_onewire.c:1957_`

—

rx_bus_owewire_skip_rom

```
rx_err_t rx_bus_owewire_skip_rom(rx_bus_manager_t *manager, const char *bus_name)
```

Issue OneWire Skip ROM command.

Send Skip ROM command.

Sends Skip ROM command (0xCC) to address all devices on the bus. Use when only one device is connected or when broadcasting to all devices.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name

Returns: `k_rx_err_not_found` if no device presence detected

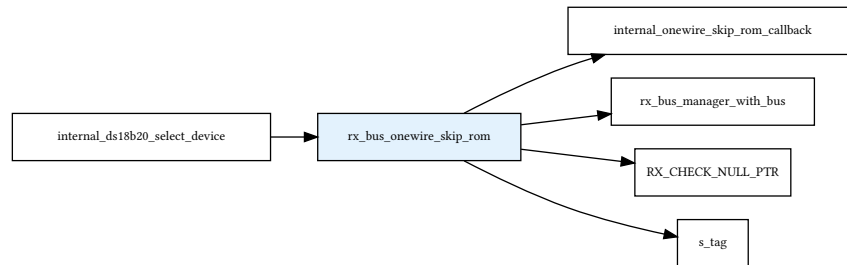


Figure 114: Call graph for rx_bus_owewire_skip_rom

_Defined in libs/rx_bus/src/rx_bus_owewire.c:1986_

rx_bus_owewire_match_rom

```
rx_err_t rx_bus_owewire_match_rom(rx_bus_manager_t *manager, const char
*bus_name, const uint8_t rom[8])
```

Issue OneWire Match ROM command.

Sends Match ROM command (0x55) followed by 64-bit ROM address to select a specific device on a multi-device bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name
in	rom	8-byte ROM address (family code + serial + CRC)

Returns: k_rx_err_not_found if no device presence detected

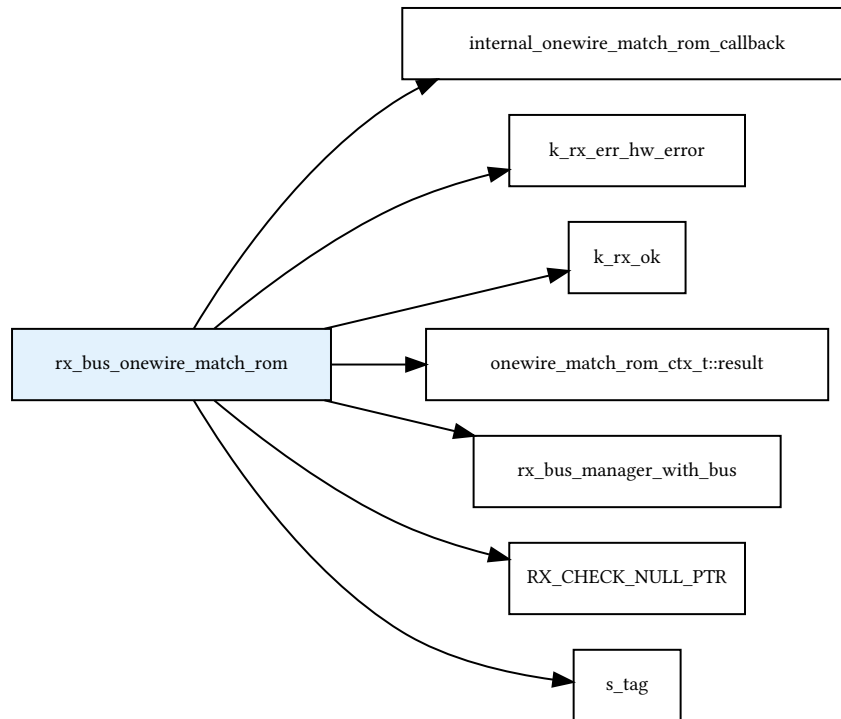


Figure 115: Call graph for rx_bus_owewire_match_rom

_Defined in libs/rx_bus/src/rx_bus_owewire.c:2008_

rx_bus_owewire_read_rom

```
rx_err_t rx_bus_owewire_read_rom(rx_bus_manager_t *manager, const char *bus_name,
uint8_t rom[8])
```

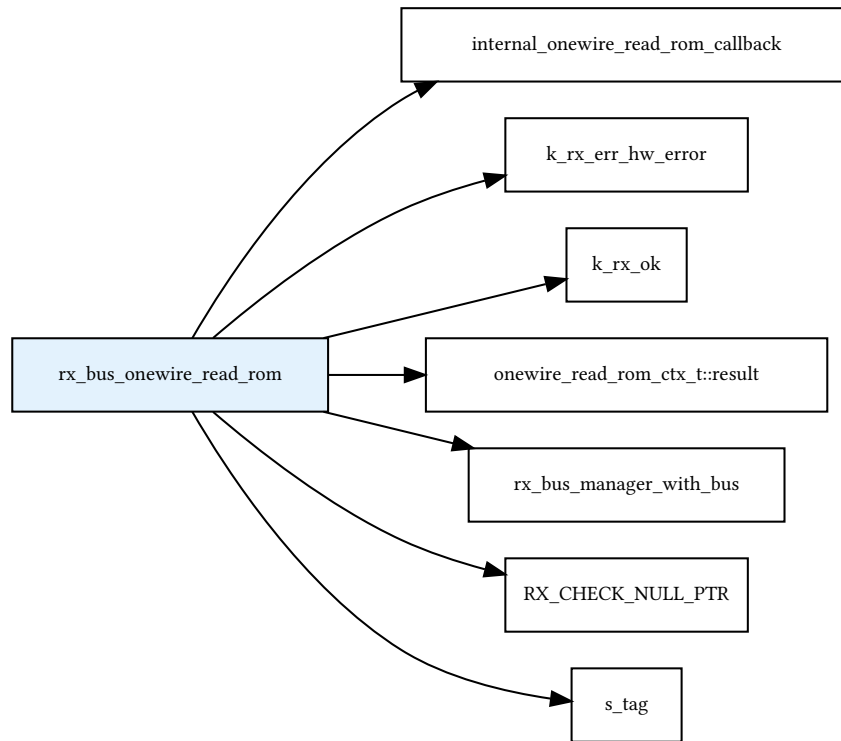
Read ROM address from single OneWire device.

Sends Read ROM command (0x33) and reads 64-bit ROM address with CRC. Only works when exactly one device is present on the bus.

Parameters:

Direction	Name	Description
in	manager	Bus manager handle
in	bus_name	Bus configuration name
out	rom	8-byte buffer for ROM address (family code + serial + CRC)

Returns: k_rx_err_crc_mismatch if ROM CRC validation fails

Figure 116: Call graph for `rx_bus_owewire_read_rom`

_Defined in `libs/rx\_bus/src/rx\_bus\_owewire.c:2037_`

rx_bus_owewire_search

```
rx_err_t rx_bus_owewire_search(rx_bus_manager_t *manager, const char *bus_name,
uint8_t *roms, const uint32_t max_devices, uint32_t *num_devices)
```

Search for all devices on the OneWire bus.

Implements the OneWire search algorithm to enumerate all devices. The search algorithm uses binary tree traversal to discover all unique 64-bit ROM codes on the bus.

Sequence:

1. Reset
2. Write SEARCH_ROM command (0xF0)
3. For each bit position (64 bits):

Read bit Read complement Write search direction

4. Repeat until all devices found

Parameters:

Direction	Name	Description
-----------	------	-------------

in	manager	Bus manager instance
in	bus_name	OneWire bus name
out	roms	Array of ROM codes (8 bytes each)
in	max_devices	Maximum number of devices to search for
out	num_devices	Number of devices found

Returns: k_rx_err_crc if ROM CRC check fails

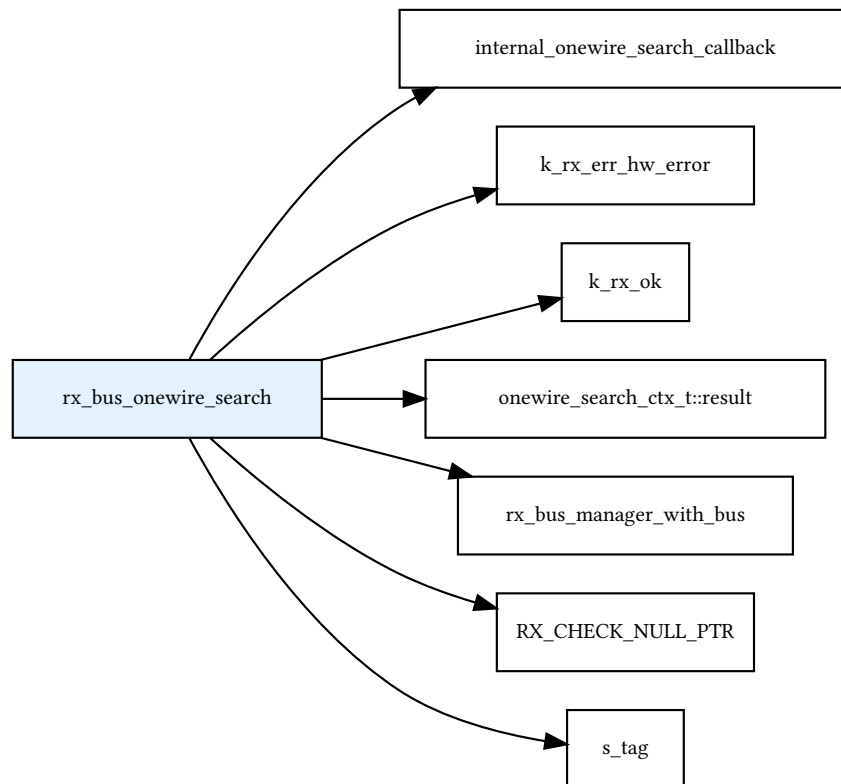


Figure 117: Call graph for `rx_bus_owewire_search`

_Defined in `libs/rx_bus/src/rx_bus_owewire.c:2052_`

rx_comm_manager.h

Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_frame.h"`

- "rx_spi_comm.h"
- "rx_spi_link.h"
- "rx_usb_comm.h"

rx_comm_manager_constants_t

Communication manager compile-time constants.

Defines buffer sizes and limits for the communication manager. These constants are chosen based on typical frame sizes and ASCII formatting requirements.

Design Rationale:

- 2048-byte ASCII buffer: Accommodates maximum frame size (1024 bytes payload) plus formatting overhead (1000 bytes for hex dump + headers)
- Buffer is stack-allocated in handle structure (not dynamically allocated)
- Sufficient for debugging without excessive memory usage

Value	Name	Description
= 2048	k_comm_manager_ascii_buf_size	ASCII formatting buffer size in bytes.
= 8	k_comm_event_queue_depth	Maximum number of entries in the event FIFO queue.

See also: rx_comm_manager_t.ascii_buffer

Since Version 1.0.0

—

rx_comm_heartbeat_constants_t

Heartbeat timing constants.

Dual-detection heartbeat model:

- Implicit timeout (200ms): Any valid frame resets the timer. If no frames arrive within 200ms, the link is declared dead.
- Explicit PING (1000ms): Sent when link is idle for >1s as a probe. Firmware primarily receives PINGs from the gateway.
- Check interval (50ms): How often to evaluate heartbeat status.

Value	Name	Description
= 200	k_heartbeat_implicit_timeout_ms	Declare link dead if no frame for 200ms
= 1000	k_heartbeat_ping_interval_ms	Send PING probe when idle for 1s
= 50	k_heartbeat_check_interval_ms	Heartbeat evaluation interval (200ms / 4)

Since Version 1.1.0

—

rx_comm_event_retry_constants_t

Event queue retry constants for SPI HARQ.

Events (commands) sent via SPI use retry with exponential backoff. USB sends are fire-and-forget (no retries).

Value	Name	Description
= 3	k_event_max_retries	Maximum SPI retry attempts
= 10	k_event_initial_backoff_ms	Initial backoff delay (ms)
= 100	k_event_max_backoff_ms	Maximum backoff delay (ms)

Since Version 1.1.0

—

rx_comm_channel_t

Communication channel identifiers.

Enumerates the available communication channels. The manager can handle multiple channels simultaneously, with each channel operating independently.

Channel Characteristics:

Value	Name	Description
= 0	k_comm_channel_usb	USB CDC channel (Port 0 - protocol).
= 1	k_comm_channel_spi	SPI peripheral channel.
= 2	k_comm_channel_count	Total number of supported channels.

See also: rx_comm_manager_channel_name() Get human-readable channel name,
rx_comm_frame_callback_t Callback receives channel identifier

Since Version 1.0.0

—

rx_comm_link_status_t

Link health status from heartbeat monitoring.

Reported per-transport link health. Determined by the dual-detection heartbeat model (implicit 200ms timeout + explicit 1s PING).

Value	Name	Description
= 0	k_link_status_unknown	No frames received yet
= 1	k_link_status_healthy	Frames received within implicit timeout
= 2	k_link_status_dead	No frames within implicit timeout (200ms)

Since Version 1.1.0

—

rx_comm_frame_callback_t

```
typedef void(* rx_comm_frame_callback_t) (rx_comm_channel_t channel, const rx_frame_t
*frame, void *ctx)
```

Frame received callback function type.

User-defined function invoked by communication manager when a complete, valid frame is received on any enabled channel. The callback receives:

- Channel identifier (which channel the frame arrived on)
- Parsed frame structure (type, flags, payload)
- User context pointer (passed during initialization)

—

rx_comm_link_status_callback_t

```
typedef void(* rx_comm_link_status_callback_t) (rx_comm_channel_t channel,
rx_comm_link_status_t new_status, void *ctx)
```

Callback invoked when a link status changes.

Since Version 1.1.0

—

rx_comm_manager_init

```
rx_err_t rx_comm_manager_init(rx_comm_manager_t *mgr, const
rx_comm_manager_config_t *cfg)
```

Initialize communication manager.

Initializes the communication manager with the provided configuration. Validates configuration parameters and sets up internal state for USB and/or SPI channel operation.

Initialize communication manager.

Performs complete initialization of communication manager handle, configuring enabled channels, callback registration, and ASCII debugging output. This function must be called before any other communication manager operations.

Algorithm:

1. Validate parameters: Check mgr pointer for nullptr (NASA Rule 5)
2. Clear handle: Zero-fill entire structure via memset (328 bytes)
3. Apply configuration: Copy config fields if cfg is non-NULL

usb_handle: Pointer to initialized USB comm handle (may be NULL) spi_handle: Pointer to initialized SPI comm handle (may be NULL) callback: User frame handler function (may be NULL) callback_ctx: User context pointer (may be NULL) enable_decoded_output: ASCII debugging flag

4. Mark initialized: Set mgr->initialized = true
5. Post-condition validation: Verify ascii_buffer properly cleared

Configuration Options:

- cfg = NULL: All channels disabled, no callback, ASCII output off
- usb_handle = NULL: USB channel disabled (poll skips USB)
- spi_handle = NULL: SPI channel disabled (poll skips SPI)
- callback = NULL: No callback invoked (silent frame consumption)
- enable_decoded_output = true: ASCII debugging enabled (Port 1)

Parameters:

Direction	Name	Description
out	mgr	Pointer to manager handle
in	cfg	Configuration (NULL for defaults, both channels disabled)

out	mgr	Pointer to communication manager handle to initialize Valid range: Non-nullptr to allocated rx_comm_manager_t (328 bytes) Constraints: Must be allocated by caller (typically static or global) Side effects: Entire structure zero-filled, then populated from cfg Lifetime: Must remain valid until rx_comm_manager_deinit() called
in	cfg	Optional configuration structure Valid range: NULL or pointer to valid rx_comm_manager_config_t NULL behavior: All channels disabled, no callback, ASCII off Lifetime: Not stored - values copied into mgr Constraints: If non-NULL, all fields must be valid or nullptr as appropriate

Returns: rx_err_t Error code indicating initialization success or failure

Value	Description
k_rx_ok	Success - manager fully initialized and ready for operation
k_rx_err_invalid_arg	mgr is nullptr
k_rx_err_invalid_state	Post-condition validation failed (buffer not cleared)

Pre: mgr must not be nullptr

Pre: If cfg->spi_link is non-NULL, spi_link must be initialized via rx_spi_link_init()

Pre: If cfg->spi_link is non-NULL, spi_link->spi_handle must equal cfg->spi_handle

Pre: cfg may be nullptr (results in default configuration with both channels disabled)

Pre: mgr must point to allocated memory (uninitialized content OK)

Pre: USB/SPI handles (if provided) must already be initialized

Pre: Callback function (if provided) must be valid and reentrant

Post: mgr->usb_handle == cfg->usb_handle (or nullptr if cfg is nullptr)

Post: mgr->spi_handle == cfg->spi_handle (or nullptr if cfg is nullptr)

Post: mgr->spi_link == cfg->spi_link (or nullptr if cfg is nullptr or cfg->spi_link is nullptr)

Post: All internal state initialized (event queue empty, heartbeat timers reset)

Post: mgr->initialized = true on success

Post: All mgr fields set to config values or zero

Post: mgr->ascii_buffer is zero-filled (256 bytes)

Note: Thread-safe: May be called from any thread, but concurrent calls with same mgr are prohibited

Note: This function does NOT initialize USB or SPI transports - caller must initialize rx_usb_comm and rx_spi_comm separately before passing handles to this function

Note: On success, at least one channel should be enabled (usb_handle or spi_handle non-NULL)

Note: Thread-safe ONLY if each thread uses a different mgr handle

Warning: Must be called before any other manager operations on this handle

Performance:: Execution time: 5 us @ 240 MHz Dominated by memset (328 bytes)

Configuration Example - Both Channels Enabled:: staticrx_comm_manager_tg_comm_mgr;
staticrx_usb_comm_handle_tg_usb_handle; staticrx_spi_comm_handle_tg_spi_handle;

rx_usb_comm_config_tusb_cfg={.port=k_usb_port_proto};

rx_err_terr=rx_usb_comm_init(&g_usb_handle,&tusb_cfg); if(err!=k_rx_ok)returnerr;

rx_spi_comm_config_tspi_cfg={.spi_channel=0}; err=rx_spi_comm_init(&g_spi_handle,&tspi_cfg);
if(err!=k_rx_ok)returnerr;

rx_comm_manager_config_tcfg={.usb_handle=&g_usb_handle, .spi_handle=&g_spi_handle, .callback=on_frame_rec
err=rx_comm_manager_init(&g_comm_mgr,&cfg);

Configuration Example - USB Only, No Callback::

rx_comm_manager_config_tcfg={.usb_handle=&g_usb_handle, .spi_handle=nullptr, .callback=nullptr, .callback_ctx=1
rx_err_terr=rx_comm_manager_init(&g_comm_mgr,&cfg);

See also: rx_spi_link_init() Must call this before passing spi_link to manager,
rx_comm_manager_deinit() Cleanup function, rx_comm_manager_deinit() Cleanup and
resource release, rx_comm_manager_poll() Poll for incoming frames, rx_usb_comm_init()
Initialize USB CDC transport, rx_spi_comm_init() Initialize SPI transport



Figure 118: Call graph for `rx_comm_manager_init`

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1146`

Since Version 1.0.0

—

rx_comm_manager_deinit

```
rx_err_t rx_comm_manager_deinit(rx_comm_manager_t *mgr)
```

Deinitialize communication manager.

Does not deinitialize the underlying USB/SPI handles.

Deinitialize communication manager.

Marks communication manager as uninitialized, preventing further operations. This function does NOT deinitialize USB or SPI transport handles - caller must separately call `rx_usb_comm_deinit()` and `rx_spi_comm_deinit()` if needed.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated). Function primarily serves as safety guard to prevent use-after-deinit.

Parameters:

Direction	Name	Description
inout	mgr	Pointer to manager handle
inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must have been initialized via <code>rx_comm_manager_init()</code> Side effects: Clears <code>mgr->initialized</code> flag

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success - manager deinitialized
<code>k_rx_err_invalid_arg</code>	mgr is nullptr
<code>k_rx_err_invalid_state</code>	mgr was not initialized

Pre: mgr must be non-NULL

Pre: mgr must be initialized (`mgr->initialized == true`)

Post: `mgr->initialized = false`

Post: Subsequent calls to `rx_comm_manager_poll/send` will return `k_rx_err_invalid_state`

Note: Does NOT free memory (handle is statically allocated)

Note: Does NOT deinitialize transport handles (USB/SPI)

Note: Thread-safe if no concurrent operations on same mgr handle

Example:: rx_err_t err=rx_comm_manager_deinit(&g_comm_mgr); if(err==k_rx_ok){
rx_usb_comm_deinit(&g_usb_handle); rx_spi_comm_deinit(&g_spi_handle); }

See also: rx_comm_manager_init() Initialization, rx_usb_comm_deinit() USB transport cleanup, rx_spi_comm_deinit() SPI transport cleanup

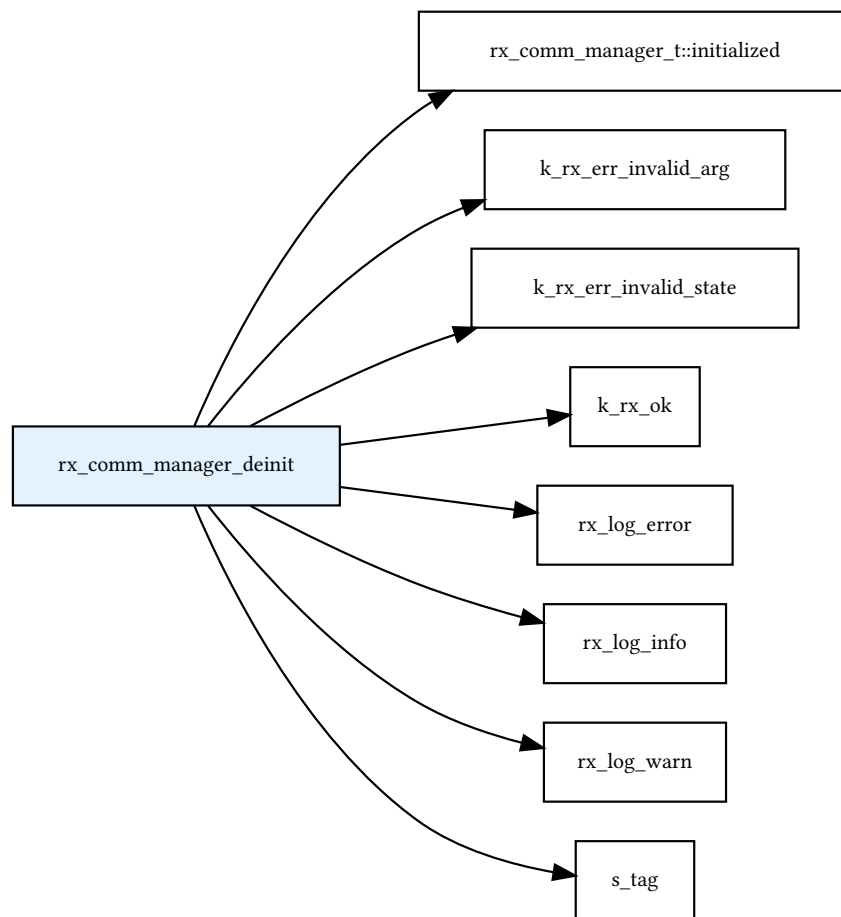


Figure 119: Call graph for rx_comm_manager_deinit

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1157`

Since Version 1.0.0

rx_comm_manager_poll

```
rx_err_t rx_comm_manager_poll(rx_comm_manager_t *mgr)
```

Poll all enabled channels for incoming frames.

Non-blocking check of USB and SPI channels. For each complete frame received:

1. If decoded output enabled, format and send ASCII to Port 1
2. Invoke user callback with frame data and channel

Call this from main loop or dedicated communication task.

Poll all enabled channels for incoming frames.

Performs non-blocking poll of all enabled channels (USB and SPI) sequentially, dispatching any received frames to the registered user callback. Returns aggregate result indicating whether any frames were received across all channels.

Algorithm:

1. Validate parameters: Check mgr pointer and initialized flag (NASA Rule 5)
2. Poll USB channel via internal_poll_usb():

If k_rx_ok: Mark received=true, continue to SPI If k_rx_err_timeout: Continue to SPI (no data is normal) If other error: Return error immediately (critical failure)

3. Poll SPI channel via internal_poll_spi():

If k_rx_ok: Mark received=true If k_rx_err_timeout: Continue (no data is normal) If other error: Return error immediately (critical failure)

4. Return result:

k_rx_ok if ANY channel received frame k_rx_err_timeout if NO channels received (normal condition) Other error codes propagated from transport layers

Performance: 15-200 us typical, depends on:

- Number of enabled channels (USB/SPI/both)
- Whether frames are available (data path vs fast timeout path)
- ASCII debugging enabled (adds 50-150 us per frame)
- User callback execution time (not included in measurement)

Design Rationale:

- Non-blocking: Never blocks waiting for data (0 ms timeout)
- Fair scheduling: Both channels polled each iteration
- Timeout tolerance: Timeout is expected, not error condition
- Error prioritization: Critical errors returned immediately
- Aggregation: k_rx_ok if ANY channel received data

Parameters:

Direction	Name	Description
inout	mgr	Pointer to initialized manager
inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_comm_manager_init() Side effects: Invokes callback, modifies ascii_buffer if frames received

Returns: rx_err_t Error code indicating aggregate poll result

Value	Description
k_rx_ok	At least one frame received on any channel (success)
k_rx_err_timeout	No frames received on any channel (normal, not an error)
k_rx_err_invalid_arg	mgr is nullptr
k_rx_err_invalid_state	mgr not initialized
k_rx_err_crc	CRC verification failed on received frame (data corruption)
(other)	Transport layer critical errors propagated

Pre: mgr must be non-NULL

Pre: mgr must be initialized

Pre: At least one channel should be enabled (usb_handle or spi_handle non-NULL)

Post: User callback invoked for each successfully received frame

Post: ASCII output sent to Port 1 if enabled

Note: Non-blocking - always returns immediately

Note: k_rx_err_timeout is NORMAL - means no data available (not an error)

Note: User callback execution time is NOT included in performance measurements

Warning: Call this function regularly (e.g., 100 Hz) to avoid buffer overruns

Warning: User callback must not block for extended periods

Performance:: No frames available: 15 us (both channels timeout, fast path) USB frame received: 100-250 us (USB bulk transfer + callback) SPI frame received: 50-150 us (SPI hardware transfer + callback) Both frames: 150-400 us (both channels + callbacks) With ASCII debugging: Add 50-150 us per frame

Typical Usage - ThreadX Communication Task::

```
static void comm_task_entry(ULONG thread_input) { (void) thread_input;

while(1) { rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);

if(err != k_rx_ok && err != k_rx_err_timeout) { log_error("Comm poll failed: %d", err);
tx_thread_sleep(100); continue; }

tx_thread_sleep(10); }
```

Error Handling - Distinguishing Timeout from Errors::

```
rx_err_t rx_comm_manager_poll(&g_comm_mgr); if(err==k_rx_ok){ }
elseif(err==k_rx_err_timeout){ }else{ handle_comm_error(err); }
```

See also: `internal_poll_usb()` USB channel polling implementation, `internal_poll_spi()` SPI channel polling implementation, `internal_handle_frame()` Frame dispatch, `rx_comm_manager_init()` Initialization with channel configuration

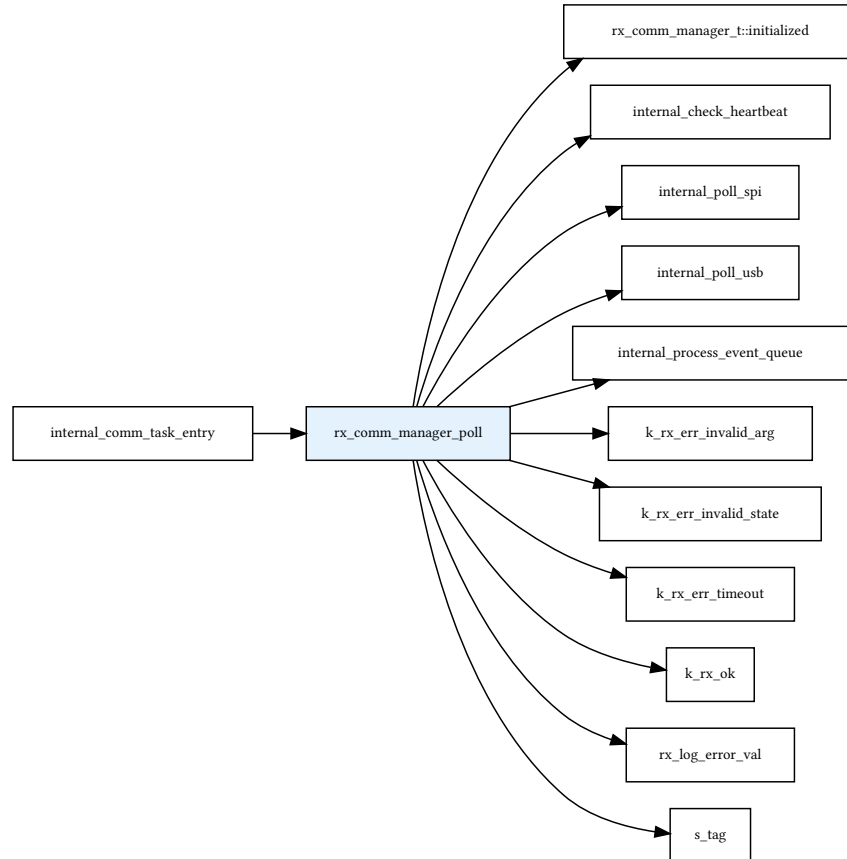


Figure 120: Call graph for `rx_comm_manager_poll`

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1180`

Since Version 1.0.0

—

rx_comm_manager_send

```
rx_err_t rx_comm_manager_send(rx_comm_manager_t *mgr, const rx_comm_send_params_t
*params)
```

Send frame on specific channel.

Encodes and sends a frame on the specified channel. If decoded output is enabled, also sends ASCII representation to Port 1.

Send frame on specific channel.

Routes frame transmission to the appropriate transport layer (USB or SPI) based on specified channel. Supports configurable frame type, flags, and variable-length payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.

Algorithm:

1. Validate parameters (NASA Rule 5):

mgr and params pointers non-NULL Manager initialized flag Payload pointer valid if payload_len > 0 payload_len <= k_frame_max_payload (255 bytes)

2. Route to channel:

USB: Verify usb_handle non-NULL, call rx_usb_comm_send() SPI: Verify spi_handle non-NULL, call rx_spi_comm_send() Invalid channel: Return k_rx_err_invalid_arg

3. On success, if ASCII debugging enabled:

Build temporary rx_frame_t from params Format as ASCII via internal_output_decoded() Send to USB Port 1 (TX direction)

Performance:

- USB: 30-150 us (depends on payload size, USB bulk transfer)
- SPI: 20-100 us (depends on payload size, SPI hardware transfer)
- ASCII debugging: Add 50-150 us if enabled

Parameters:

Direction	Name	Description
inout	mgr	Pointer to initialized manager
in	params	Send parameters (channel, type, flags, payload, payload_len)
in	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_comm_manager_init() Side effects: May modify ascii_buffer if decoded output enabled
in	params	Send parameters structure Valid range: Non-nullptr to valid rx_comm_send_params_t Constraints: All fields must be valid: channel: k_comm_channel_usb or k_comm_channel_spi type: Frame type (command, request, response, etc.) flags: Frame flags (k_frame_flag_none or specific flags) payload: Non-NULL if payload_len > 0, may be NULL if payload_len == 0 payload_len: [0, k_frame_max_payload] (0-255 bytes) Lifetime: Not stored - values used immediately

Returns: rx_err_t Error code indicating send success or failure

Value	Description
k_rx_ok	Frame sent successfully
k_rx_err_invalid_arg	mgr or params is nullptr, or payload invalid
k_rx_err_invalid_arg	payload_len > k_frame_max_payload (255 bytes)

k_rx_err_invalid_arg	Invalid channel specified
k_rx_err_invalid_arg	Payload is nullptr but payload_len > 0
k_rx_err_invalid_state	Manager not initialized
k_rx_err_invalid_state	Channel not configured (handle is nullptr)
k_rx_err_hw	USB/SPI hardware error (transport layer)
(other)	Transport layer errors propagated

Pre: mgr must be non-NULL and initialized

Pre: params must be non-NULL with all valid fields

Pre: Channel handle (USB/SPI) must be initialized

Pre: If payload_len > 0, payload must point to valid data

Post: Frame transmitted on specified channel if successful

Post: ASCII output sent to Port 1 if enabled and successful

Note: Payload sequence number and CRC-32 managed by transport layer

Note: ASCII debugging adds overhead - disable in production

Warning: Ensure channel is ready before sending (use rx_comm_manager_channel_ready)

Send Parameters Structure:: typedef struct { rx_comm_channel_t channel; uint8_t type; uint8_t flags; const uint8_t* payload; uint32_t payload_len; } rx_comm_send_params_t;

Example - Send Command on USB::

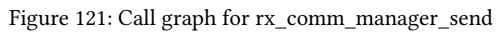
```
uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

rx_comm_send_params_t params = { .channel = k_comm_channel_usb, .type = k_frame_type_command, .flags = k_frame_flag_none,
                                 .payload = cmd_payload, .payload_len = sizeof(cmd_payload) };
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params); if (err == k_rx_ok) { }
```

Example - Send Empty Frame (No Payload)::

```
rx_comm_send_params_t params = { .channel = k_comm_channel_spi, .type = k_frame_type_response, .flags = k_frame_flag_none,
                                 .payload = NULL, .payload_len = 0 };
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
```

See also: rx_comm_manager_respond() Convenience wrapper for response frames,
rx_comm_send_params_t Send parameters structure, rx_usb_comm_send() USB CDC send implementation, rx_spi_comm_send() SPI send implementation



Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1200`

Since Version 1.0.0

—

rx_comm_manager_respond

```
rx_err_t rx_comm_manager_respond(rx_comm_manager_t *mgr, rx_comm_channel_t
channel, const uint8_t *payload, uint32_t payload_len)
```

Send response on same channel as received frame.

Convenience function for responding to commands. Sends a RESPONSE type frame on the specified channel.

Send response on same channel as received frame.

Convenience function for sending response frames. Automatically sets frame type to `k_frame_type_response` and flags to `k_frame_flag_none`. Simplifies common use case of responding to commands/requests.

Algorithm:

1. Build `rx_comm_send_params_t` with:

channel: User-specified type: `k_frame_type_response` (fixed) flags: `k_frame_flag_none` (fixed)

payload: User-specified payload_len: User-specified

2. Call `rx_comm_manager_send()` with constructed params

3. Return result from `rx_comm_manager_send()`

When to Use:

- Responding to commands with telemetry data
- Sending acknowledgments
- Replying to requests

When NOT to Use:

- Need custom frame type (use `rx_comm_manager_send` instead)
- Need custom flags (use `rx_comm_manager_send` instead)

Parameters:

Direction	Name	Description
inout	<code>mgr</code>	Pointer to initialized manager
in	<code>channel</code>	Channel to respond on (from callback)
in	<code>payload</code>	Response payload data
in	<code>payload_len</code>	Response payload length
in	<code>mgr</code>	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via <code>rx_comm_manager_init()</code>
in	<code>channel</code>	Channel to send response on Valid range: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> Constraints: Channel handle must be configured

in	payload	Response payload data Valid range: Non-NULL if payload_len > 0, may be NULL if payload_len == 0 Constraints: Must contain valid data for payload_len bytes Lifetime: Copied immediately, does not need to persist
in	payload_len	Response payload length in bytes Valid range: [0, k_frame_max_payload] (0-255 bytes) Units: Bytes

Returns: rx_err_t Error code (same as rx_comm_manager_send)

Value	Description
k_rx_ok	Response sent successfully
k_rx_err_invalid_arg	mgr is nullptr or payload invalid
k_rx_err_invalid_state	Manager not initialized or channel not configured
(other)	Errors propagated from rx_comm_manager_send()

Pre: Same preconditions as rx_comm_manager_send()

Post: Response frame transmitted with type=response, flags=none

Note: Equivalent to calling rx_comm_manager_send() with type=k_frame_type_response

Note: This is a thin wrapper - no additional validation performed

Example - Send Telemetry Response::

```
static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
{
    uint8_t telemetry[16];

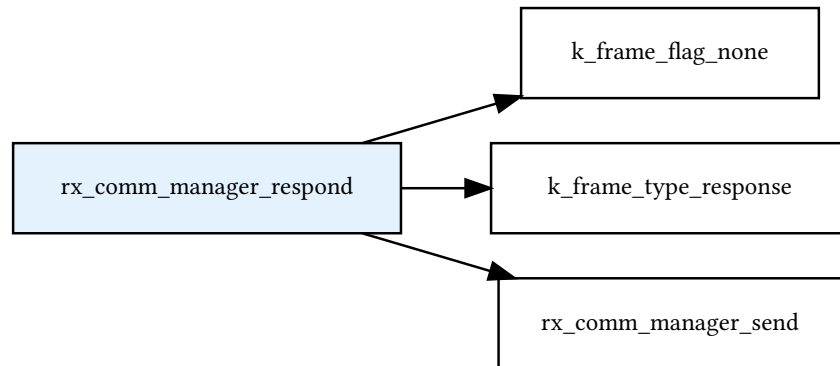
    telemetry[0] = get_temperature_celsius();
    telemetry[1] = get_current_ma();

    rx_err_t err = rx_comm_manager_respond(&g_comm_mgr, channel, telemetry, sizeof(telemetry));
    if (err != k_rx_ok) {}
}
```

Example - Send Empty Acknowledgment::

```
rx_err_t err = rx_comm_manager_respond(&g_comm_mgr, k_comm_channel_usb, NULL, 0);
```

See also: rx_comm_manager_send() Full send function with all parameters,
rx_comm_send_params_t Send parameters structure

Figure 122: Call graph for `rx_comm_manager_respond`

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1216`

Since Version 1.0.0

—

rx_comm_manager_stream_send

```
rx_err_t rx_comm_manager_stream_send(rx_comm_manager_t *mgr, const
rx_comm_send_params_t *params)
```

Send data on the stream path (fire-and-forget).

Stream path is for time-sensitive data like telemetry where stale data has no value. Sends immediately with NO retries on any transport. If the send fails, the data is simply dropped.

- USB CDC: Direct send, no retries (USB HW provides reliability)
- SPI: Direct send, no retries (stale telemetry is worthless)

Parameters:

Direction	Name	Description
inout	<code>mgr</code>	Pointer to initialized manager
in	<code>params</code>	Send parameters (channel, type, flags, payload, payload_len)

Returns: `k_rx_err_invalid_state` if not initialized or channel not enabled

Pre: mgr must be initialized

Pre: params must be valid

Post: Data sent or silently dropped on failure

Note: This is a thin wrapper over `rx_comm_manager_send()` for semantic clarity] **See also:** `rx_comm_manager_event_send()` For reliable command delivery



Figure 123: Call graph for rx_comm_manager_stream_send

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1253`

Since Version 1.1.0

—

rx_comm_manager_event_send

```
rx_err_t rx_comm_manager_event_send(rx_comm_manager_t *mgr, const
rx_comm_send_params_t *params)
```

Queue data on the event path (reliable, SPI retry).

Event path is for commands and other data requiring reliable delivery. The payload is copied into a static FIFO queue and processed by the poll loop. Retry behavior depends on transport:

- SPI: Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
- USB: Fire-and-forget (USB HW reliability is sufficient)

Queue is processed in FIFO order by rx_comm_manager_poll().

Parameters:

Direction	Name	Description
inout	mgr	Pointer to initialized manager
in	params	Send parameters (channel, type, flags, payload, payload_len)

Returns: k_rx_err_no_mem if event queue is full

Pre: mgr must be initialized

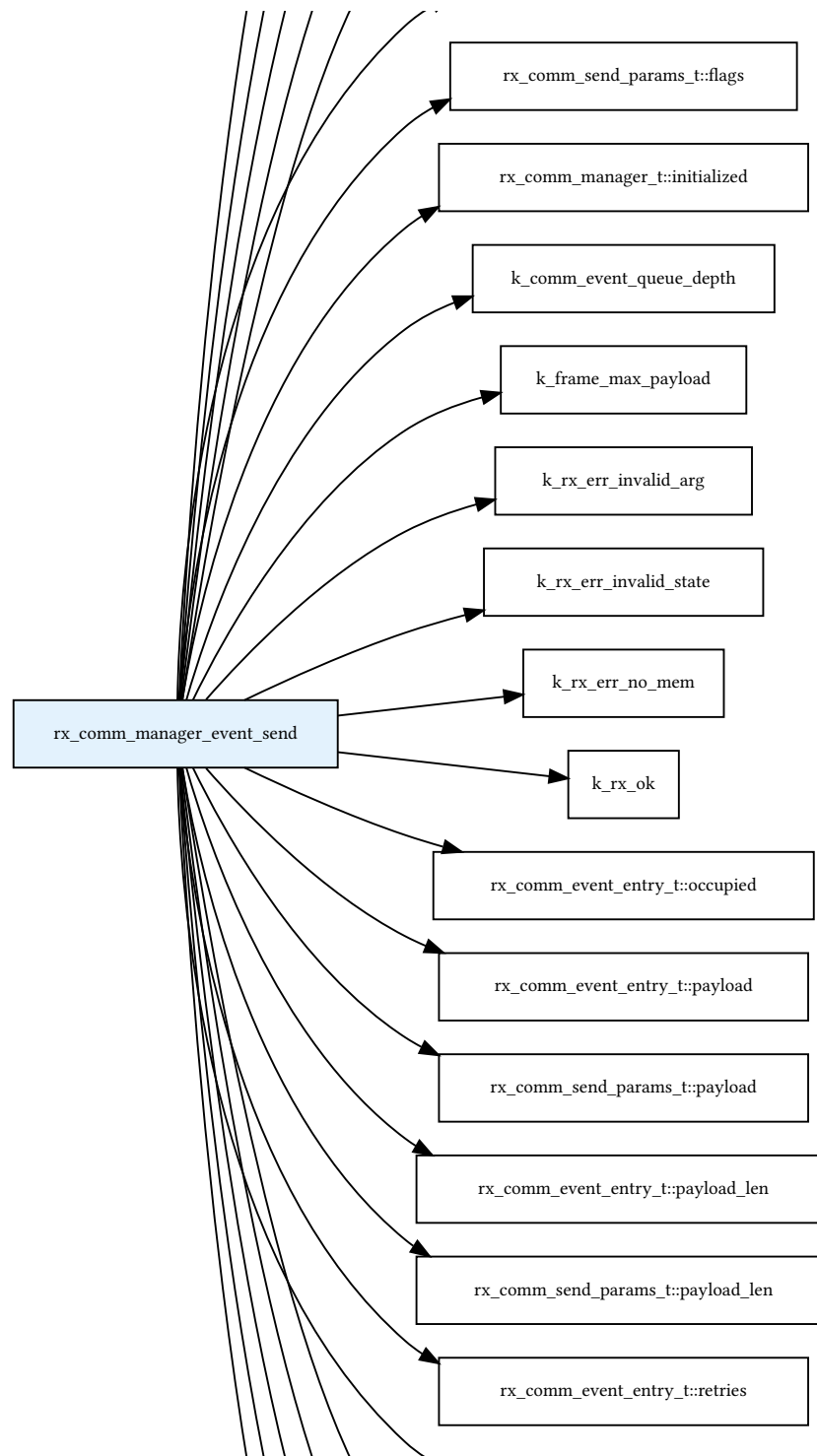
Pre: params must be valid

Pre: params->payload_len <= k_frame_max_payload

Post: Payload copied to queue, will be sent by next poll()

Note: Payload is COPIED into queue - caller can reuse buffer immediately

Warning: Queue depth is k_comm_event_queue_depth (8). If full, returns error.] **See also:** rx_comm_manager_stream_send() For fire-and-forget telemetry

Figure 124: Call graph for `rx_comm_manager_event_send`

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1288`

Since Version 1.1.0

—

rx_comm_manager_link_status

```
rx_err_t rx_comm_manager_link_status(const rx_comm_manager_t *mgr,  
rx_comm_channel_t channel, rx_comm_link_status_t *status)
```

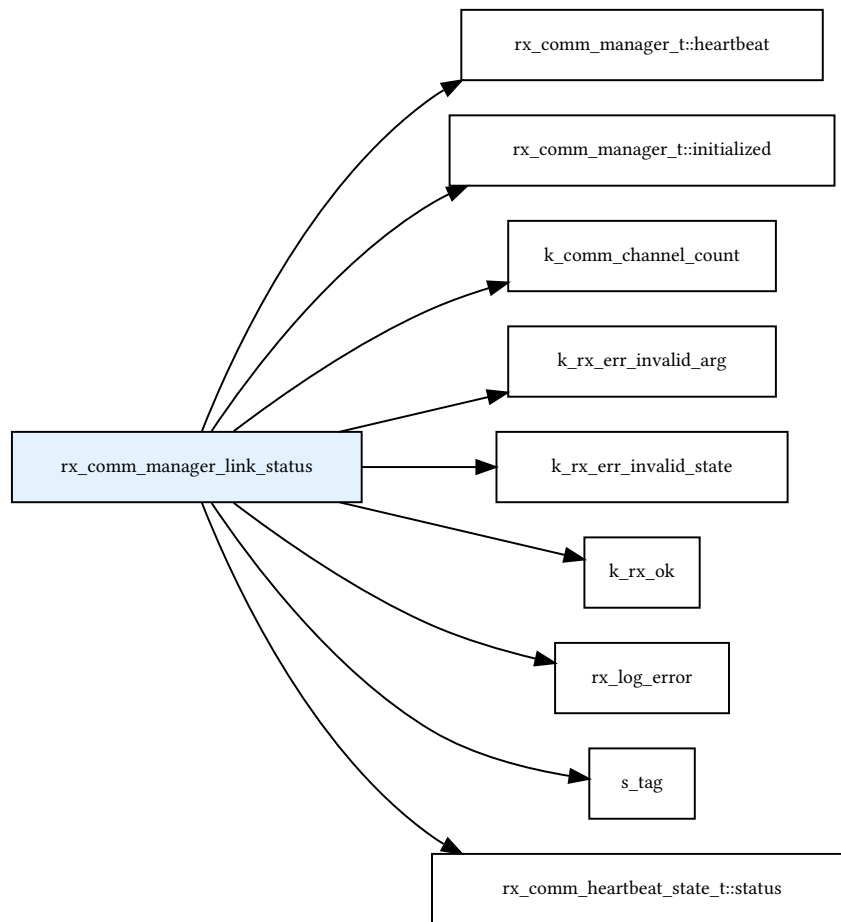
Query link health status for a transport channel.

Returns the current heartbeat-derived link status for the given channel. Status is updated by `rx_comm_manager_poll()` based on the dual-detection heartbeat model (200ms implicit timeout).

Parameters:

Direction	Name	Description
in	<code>mgr</code>	Pointer to initialized manager
in	<code>channel</code>	Channel to query
out	<code>status</code>	Receives current link status

Returns: `k_rx_err_invalid_arg` if `mgr` or `status` is `nullptr`

Figure 125: Call graph for `rx_comm_manager_link_status`

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1313`

Since Version 1.1.0

—

rx_comm_manager_channel_ready

```
rx_err_t rx_comm_manager_channel_ready(rx_comm_manager_t *mgr, rx_comm_channel_t
channel, bool *ready)
```

Check if channel is ready for communication.

Check if channel is ready for communication.

Checks if specified channel is configured and ready for communication. USB channel readiness depends on USB enumeration and configuration by host. SPI channel is ready immediately if handle is configured (no enumeration required).

Readiness Criteria:

- USB: `usb_handle` non-NULL AND USB device enumerated and configured by host
- SPI: `spi_handle` non-NULL (always ready if configured)

Use Cases:

- Check readiness before sending to avoid errors
- Implement fallback logic (try USB, fall back to SPI)
- Log channel availability status

Parameters:

Direction	Name	Description
in	<code>mgr</code>	Pointer to initialized manager
in	<code>channel</code>	Channel to check
out	<code>ready</code>	Set to true if channel is ready
in	<code>mgr</code>	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via <code>rx_comm_manager_init()</code>
in	<code>channel</code>	Channel to query Valid range: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> Constraints: Must be valid channel ID
out	<code>ready</code>	Pointer to receive ready status Valid range: Non-nullptr to bool Output values: true (ready) or false (not ready) On error: Set to false

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Query successful, ready flag updated
<code>k_rx_err_invalid_arg</code>	<code>mgr</code> or <code>ready</code> is nullptr, or channel invalid
<code>k_rx_err_invalid_state</code>	Manager not initialized

Pre: `mgr` must be non-NULL and initialized

Pre: `ready` must be non-NULL

Pre: `channel` must be valid (USB or SPI)

Post: `*ready` set to true or false based on channel state

Post: On error, `*ready` set to false (safe default)

Note: USB readiness can change at runtime (host connects/disconnects)

Note: SPI readiness is static (always ready if handle configured)

Note: This function does NOT initiate communication - only queries state

Example - Send with Fallback:: uint8_tpayload[16]={...};

```
boolusb_ready=false; rx_err_terr=rx_comm_manager_channel_ready(&g_comm_mgr,
k_comm_channel_usb, &usb_ready); if(err==k_rx_ok&&usb_ready)
{ err=rx_comm_manager_respond(&g_comm_mgr, k_comm_channel_usb, payload,
sizeof(payload)); }else{ boolspi_ready=false; err=rx_comm_manager_channel_ready(&g_comm_mgr,
k_comm_channel_spi, &spi_ready); if(err==k_rx_ok&&spi_ready)
{ err=rx_comm_manager_respond(&g_comm_mgr, k_comm_channel_spi, payload,
sizeof(payload)); } }
```

Example - Log Channel Status:: voidlog_channel_status(void)

```
{ boolusb_ready=false,spi_ready=false;
```

```
rx_comm_manager_channel_ready(&g_comm_mgr,k_comm_channel_usb,&usb_ready);
```

```
rx_comm_manager_channel_ready(&g_comm_mgr,k_comm_channel_spi,&spi_ready);
```

```
printf("USB:%s,SPI:%sn", usb_ready?"READY":"NOTREADY", spi_ready?"READY":"NOTREADY"); }
```

See also: rx_usb_is_configured() USB enumeration status, rx_comm_manager_send() Send frame on channel

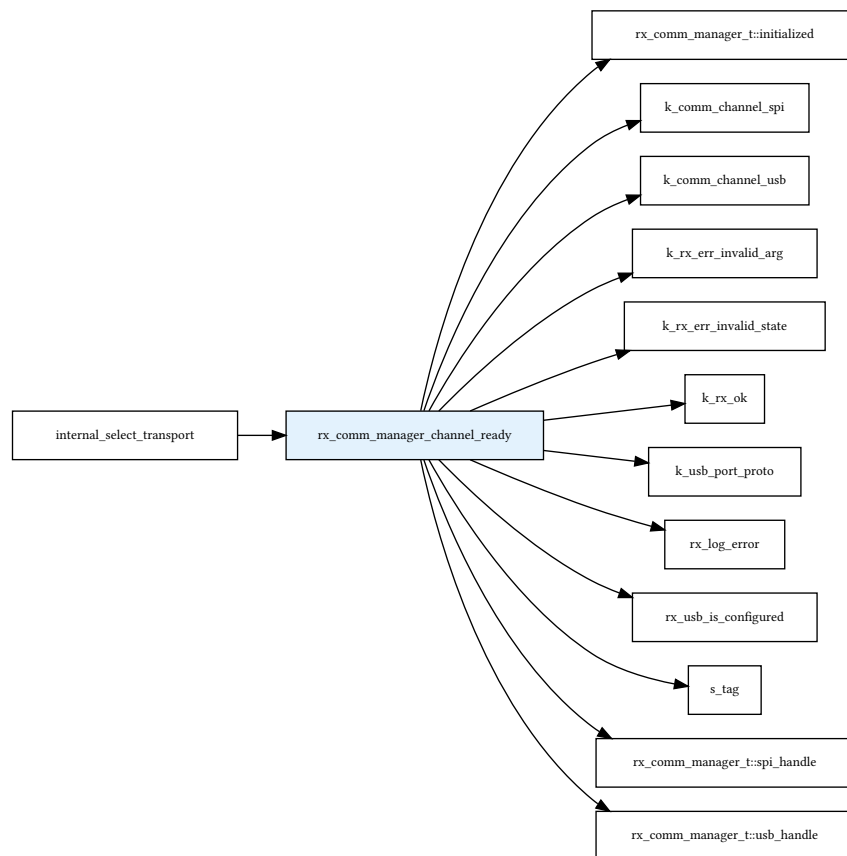


Figure 126: Call graph for rx_comm_manager_channel_ready

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1333`

Since Version 1.0.0

—

rx_comm_manager_channel_name

```
const char * rx_comm_manager_channel_name(rx_comm_channel_t channel)
```

Get channel name string.

Get channel name string.

Returns static string name for given channel enum value. Used for logging, debugging, and user interface display. Always returns a valid string (never NULL).

Returned Strings:

- k_comm_channel_usb -> "USB"
- k_comm_channel_spi -> "SPI"
- Invalid/unknown -> "UNKNOWN"

Use Cases:

- Logging channel activity
- Debugging frame traffic
- User interface display
- Error messages

Parameters:

Direction	Name	Description
in	channel	Channel identifier
in	channel	Communication channel enum value Valid range: Any rx_comm_channel_t value Constraints: None (invalid values return "UNKNOWN")

Returns: const char* Human-readable channel name (never NULL)

Value	Description
USB	If channel == k_comm_channel_usb
SPI	If channel == k_comm_channel_spi
UNKNOWN	If channel is invalid or out of range

Pre: None (accepts any input)

Post: Returns pointer to static string (valid for program lifetime)

Note: Returned string is static - do not free

Note: Thread-safe (returns read-only static strings)

Note: Never returns NULL - always safe to use in printf

Example - Logging:: static void on_frame_received(rx_comm_channel_t channel, const rx_frame_t *frame, void *ctx) { printf("Frame received on %s: type=%d len=%d\n", rx_comm_manager_channel_name(channel), frame->header.type, frame->header.length); }

Example - Error Messages:: rx_err_t err = rx_comm_manager_send(&g_comm_mgr, ¶ms); if (err != k_rx_ok) { fprintf(stderr, "Failed to send on %s: error %d\n", rx_comm_manager_channel_name(params.channel), err); }

See also: rx_comm_channel_t Channel enumeration

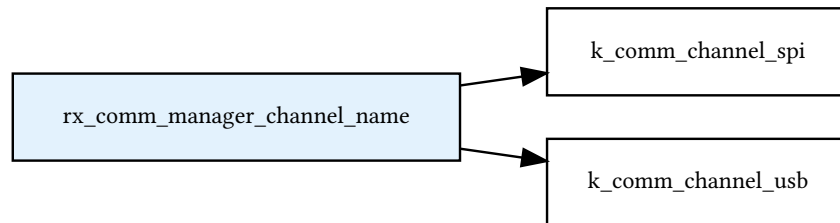


Figure 127: Call graph for rx_comm_manager_channel_name

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1341`

Since Version 1.0.0

—

rx_comm_manager_process_retransmits

```
rx_err_t rx_comm_manager_process_retransmits(rx_comm_manager_t *mgr, uint32_t
current_time_ms)
```

Process pending retransmissions on SPI channel.

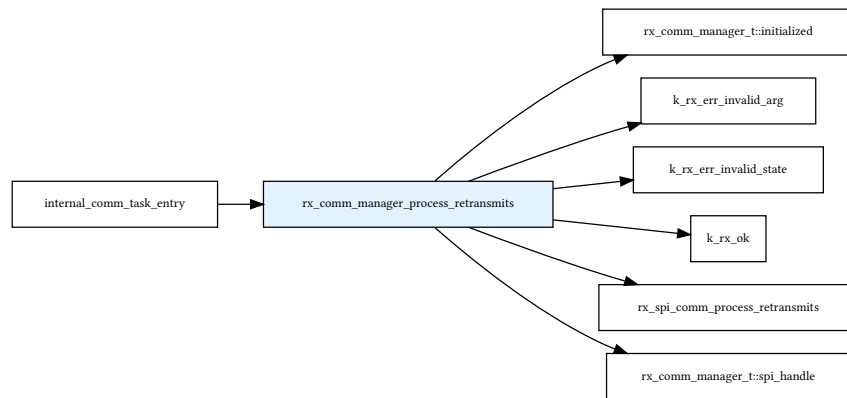
Thin pass-through to `rx_spi_comm_process_retransmits()` on the SPI handle. Call this from the communication task polling loop.

Parameters:

Direction	Name	Description
inout	mgr	Communication manager
in	current_time_ms	Current system time in milliseconds

Returns: rx_err_t Error code from `rx_spi_comm_process_retransmits()`

Value	Description
k_rx_ok	No action needed or retransmit sent
k_rx_err_invalid_arg	mgr is nullptr
k_rx_err_invalid_state	mgr not initialized
k_rx_err_retry_limit	Max retries exceeded

Figure 128: Call graph for `rx_comm_manager_process_retransmits`

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1366`

Since Version 1.1.0

—

rx_comm_manager_set_auto_retransmit

```
rx_err_t rx_comm_manager_set_auto_retransmit(rx_comm_manager_t *mgr, bool
enabled, const rx_spi_comm_retransmit_config_t *config)
```

Enable or disable automatic retransmission on SPI channel.

Thin pass-through to `rx_spi_comm_set_auto_retransmit()` on the SPI handle.

Parameters:

Direction	Name	Description
inout	<code>mgr</code>	Communication manager
in	<code>enabled</code>	true to enable, false to disable
in	<code>config</code>	Retransmit config (NULL to keep current/defaults)

Returns: `rx_err_t` Error code from `rx_spi_comm_set_auto_retransmit()`

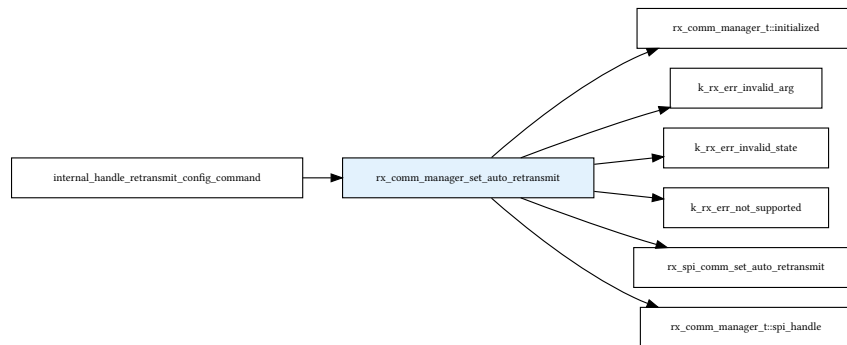


Figure 129: Call graph for rx_comm_manager_set_auto_retransmit

Defined in `libs/rx_comm_manager/inc/rx_comm_manager.h:1384`

Since Version 1.1.0

—

rx_comm_manager.c

Unified Multi-Channel Communication Manager Implementation.

Includes:

- "rx_comm_manager.h"
- <string.h>
- "rx_frame_ascii.h"
- "rx_log.h"
- "rx_simulator_config.h"
- "rx_time_constants.h"
- "rx_usb.h"

rx_comm_sim_time_t

Get current time in milliseconds.

Returns the current system time in milliseconds. On RX72N hardware, this uses ThreadX `tx_time_get()` multiplied by tick period. In simulator builds, returns 0 (timing not available).

Simulator time stub value (time is unavailable in simulator)

Value	Name	Description
= 0	k_sim_time_ms_zero	Simulator returns zero (no real-time clock)

Since Version 1.1.0

—

internal_constants_t

Internal constants for communication manager implementation.

Defines compile-time constants used internally by the communication manager for timeout values, placeholder values, and buffer indexing.

Design Notes:

- Non-blocking timeout (0 ms) ensures poll never blocks
- Placeholder values filled by transport layer (sequence, CRC)
- Explicit array index constant for bounds checking

Value	Name	Description
= 0	k_receive_timeout_ms	Non-blocking receive timeout for polling.
= 0	k_frame_seq_placeholder	Placeholder for frame sequence number (filled by transport layer).
= 0	k_frame_crc_placeholder	Placeholder for frame CRC-32 (filled by transport layer).
= 0	k_ascii_buffer_first_idx	First index of ascii_buffer for post-condition validation.
= 0	k_zero_u32	Typed zero constant for uint32_t comparisons.

Since Version 1.0.0

—

s_tag

```
const char s_tag[]
```

—

s_zero_mgr

```
const rx_comm_manager_t s_zero_mgr
```

Zero-initialized comm manager template for stack-safe initialization.

Note: Read-only; never modified after static initialization

Warning: Do not remove - prevents stack overflow in init function] **See also:**

rx_comm_manager_init() Uses this template to zero-initialize the manager handle

Since Version 1.0.0

—

internal_get_time_ms

```
static uint32_t internal_get_time_ms(void)
```

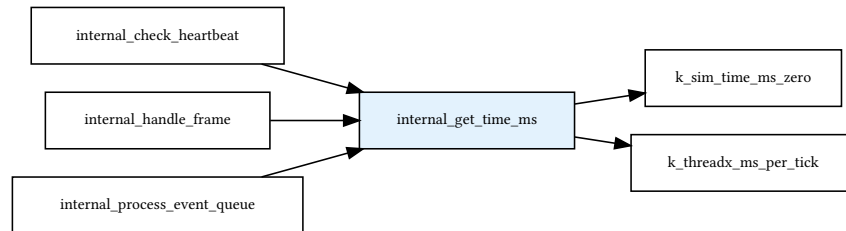


Figure 130: Call graph for internal_get_time_ms

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:435`

internal_output_decoded

```
static void internal_output_decoded(rx_comm_manager_t *mgr, const rx_frame_t
*frame, bool is_tx)
```

Output decoded ASCII frame to USB Port 1 for debugging.

Formats the given frame as human-readable ASCII text and outputs to USB Port 1 (decoded debug port) if decoded output is enabled. Used for development/debugging to monitor frame traffic without binary protocol parsing.

Algorithm:

1. Validate parameters (NULL checks)
2. Check if decoded output is enabled (cfg->enable_decoded_output)
3. Format frame as ASCII via rx_frame_ascii_format()
4. Send ASCII string to USB Port 1 via rx_usb_puts()

Performance: 50-150 us depending on frame size (ASCII formatting overhead). Disable in production for optimal performance.

Parameters:

Direction	Name	Description
inout	mgr	Communication manager handle (uses ascii_buffer) Valid range: Non-nullptr to initialized handle Constraints: Must have valid ascii_buffer (256 bytes) Side effects: Modifies mgr->ascii_buffer content
in	frame	Frame to format and output Valid range: Non-nullptr to valid rx_frame_t Constraints: Header and payload must be valid Units: Binary frame structure
in	is_tx	Direction flag (true = TX, false = RX) Valid range: true (outgoing frame) or false (incoming frame) Purpose: Prefix ASCII output with "TX:" or "RX:" for clarity

Returns: void (no return value - errors silently ignored)

Pre: mgr must be initialized via rx_comm_manager_init()

Pre: frame must be a valid frame structure

Post: mgr->ascii_buffer content is modified (temporary)

Post: ASCII output sent to USB Port 1 if enabled and successful

Note: Not thread-safe - caller must ensure exclusive access to mgr->ascii_buffer

Note: Errors are silently ignored (formatting/USB send failures)

Note: USB Port 1 must be configured for ASCII output to appear

Example Output:: RX:

[USB]Type=0x01Flags=0x00Len=8Seq=42Payload=0102030405060708CRC=ABCD1234 TX:

[SPI]Type=0x02Flags=0x80Len=4Seq=43Payload=01020304CRC=5678CDEF

See also: `rx_frame_ascii_format()` Frame-to-ASCII conversion, `rx_usb_puts()` USB debug output

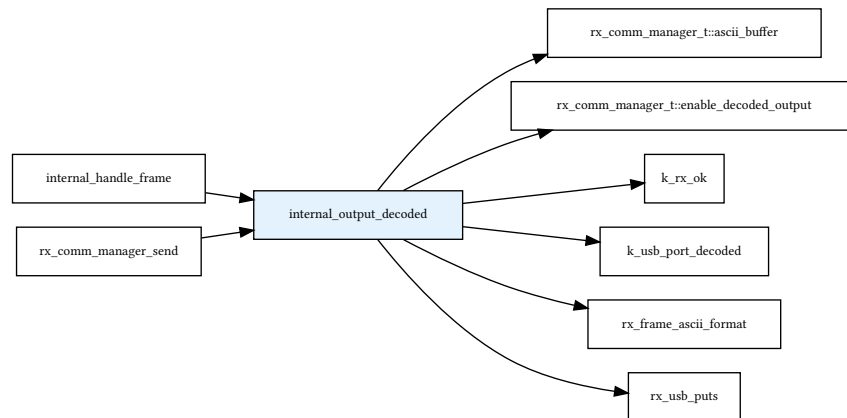


Figure 131: Call graph for `internal_output_decoded`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:569`

Since Version 1.0.0

`internal_handle_frame`

```
static void internal_handle_frame(rx_comm_manager_t *mgr, rx_comm_channel_t
channel, const rx_frame_t *frame)
```

Handle received frame by invoking user callback and optional ASCII output.

Central frame dispatch point for all channels. Performs optional ASCII debugging output, then invokes user-registered callback with frame data. This function isolates callback invocation logic from channel-specific polling.

Algorithm:

1. Validate parameters (NULL checks, channel range)
2. Output decoded ASCII to Port 1 if enabled (RX direction)
3. Invoke user callback if registered

Design Rationale: Centralizing frame handling logic ensures consistent behavior across USB and SPI channels (single point of dispatch).

Parameters:

Direction	Name	Description
-----------	------	-------------

inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_comm_manager_init() Side effects: May modify ascii_buffer if decoded output enabled
in	channel	Channel the frame was received on Valid range: k_comm_channel_usb or k_comm_channel_spi Constraints: Must be < k_comm_channel_count Purpose: Passed to user callback for channel identification
in	frame	Received frame data Valid range: Non-nullptr to valid, CRC-verified frame Constraints: Already validated by transport layer Lifetime: Valid only for duration of this function call

Returns: void (no return value - errors silently handled)

Pre: mgr must be initialized

Pre: frame must be valid and CRC-verified by transport layer

Pre: channel must be in valid range 0, k_comm_channel_count)

Post: User callback invoked if registered (callback may be NULL)

Post: ASCII output sent to Port 1 if enabled

Note: Thread safety depends on user callback implementation

Note: Callback errors are not returned (callback should handle internally)

Warning: User callback must not block for extended periods (affects polling rate)

Performance:: Without decoded output: 2 us (callback overhead only) With decoded output: 50-150 us (ASCII formatting + USB puts)

See also: internal_output_decoded() ASCII debugging output, rx_comm_frame_callback_t User callback type

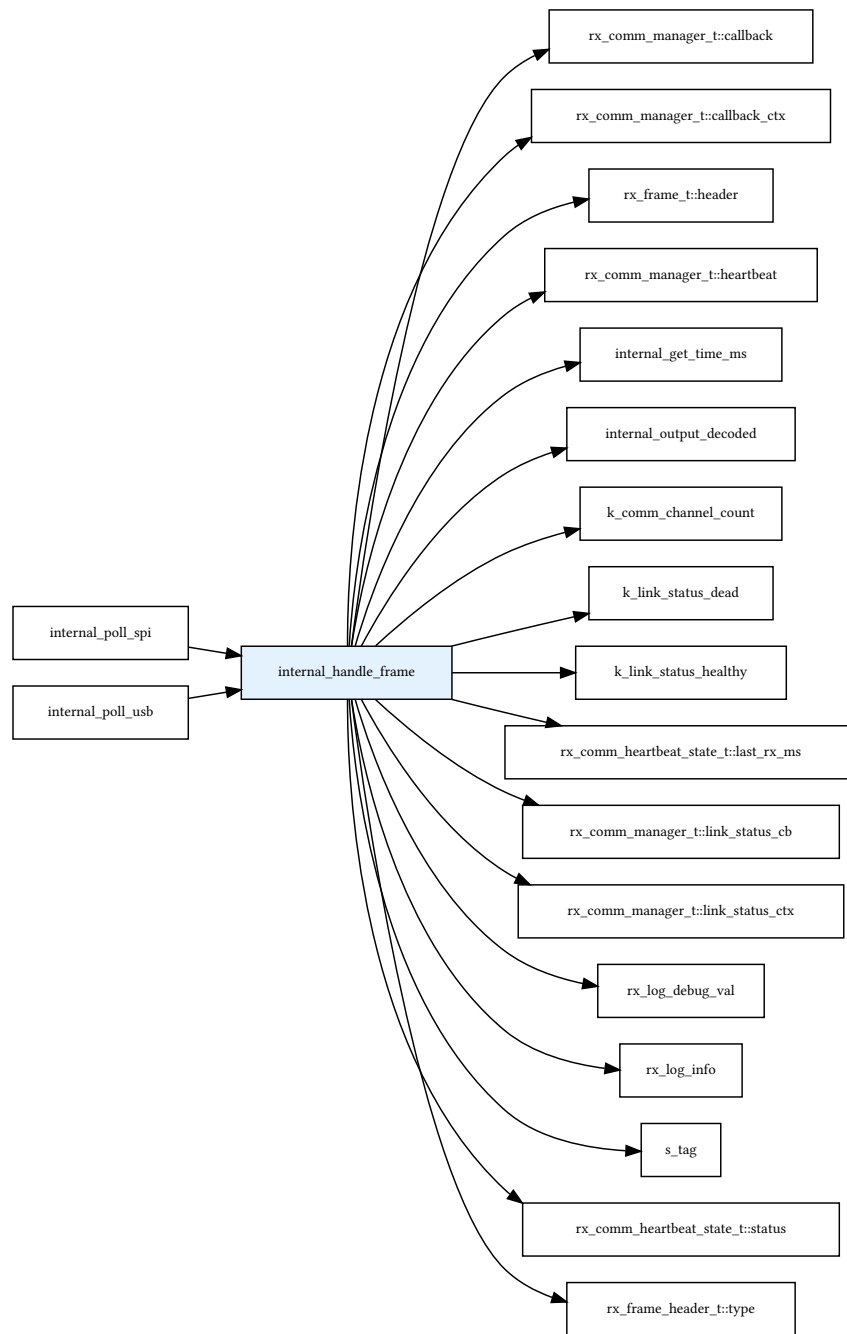


Figure 132: Call graph for `internal_handle_frame`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:641`

Since Version 1.0.0

internal_poll_usb

```
static rx_err_t internal_poll_usb(rx_comm_manager_t *mgr)
```

Poll USB channel for incoming frames (non-blocking).

Attempts to receive one frame from USB CDC channel with zero timeout (non-blocking). If frame is successfully received, it is dispatched via `internal_handle_frame()`. Timeout (no data available) is expected and normal - not an error condition.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Skip if USB handle is nullptr (channel not configured)
3. Call `rx_usb_comm_receive()` with 0 ms timeout
4. On success: dispatch frame via `internal_handle_frame()`
5. On timeout: return `k_rx_err_timeout` (expected, not error)
6. On other errors: propagate error code

Design Note: This function treats timeout as non-error. The calling poll function (`rx_comm_manager_poll`) decides whether overall operation succeeded based on aggregate results from all channels.

Parameters:

Direction	Name	Description
inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must have valid <code>usb_handle</code> if USB channel enabled Side effects: May invoke callback, modify <code>ascii_buffer</code>

Returns: `rx_err_t` Error code indicating poll result

Value	Description
<code>k_rx_ok</code>	Frame received and handled successfully
<code>k_rx_err_timeout</code>	No data available (normal, expected condition)
<code>k_rx_err_invalid_arg</code>	mgr is nullptr
<code>k_rx_err_crc</code>	CRC verification failed (corrupted frame)
<code>k_rx_err_invalid_state</code>	USB peripheral not ready
(other)	Transport layer errors propagated

Pre: mgr must be non-NULL

Pre: If USB channel enabled, `usb_handle` must be initialized

Post: On `k_rx_ok`: callback invoked, ASCII output sent (if enabled)

Post: On timeout: no side effects (no frame received)

Note: Non-blocking - always returns immediately

Note: Timeout is NOT an error - it means no data is available

Warning: Do not call if usb_handle is uninitialized (will return timeout)

Performance:: No data available: 10 us (fast path) Frame received: 100-250 us (USB bulk transfer + callback)

See also: internal_handle_frame() Frame dispatch, rx_usb_comm_receive() USB CDC receive implementation

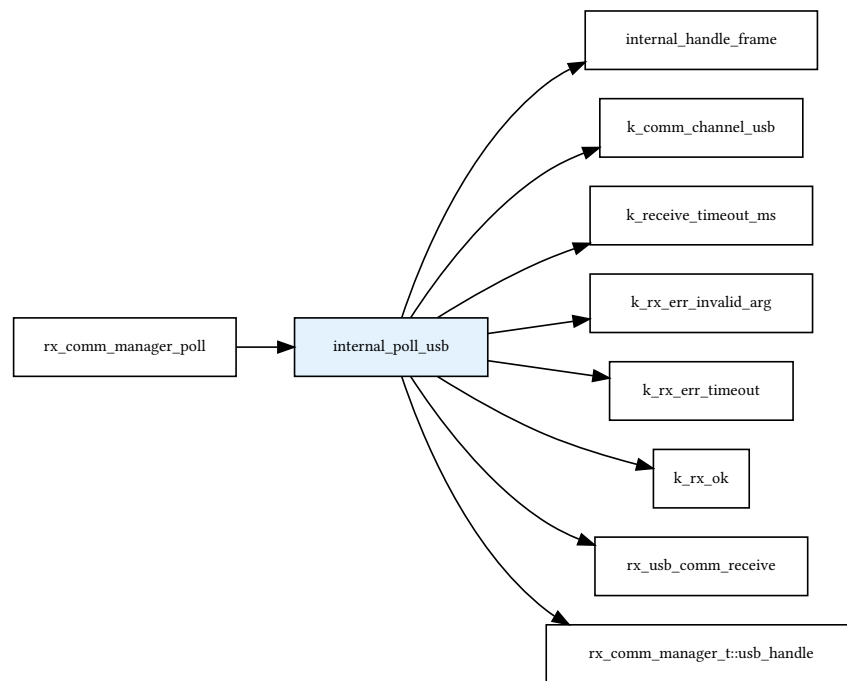


Figure 133: Call graph for internal_poll_usb

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:733`

Since Version 1.0.0

—

internal_poll_spi

```
static rx_err_t internal_poll_spi(rx_comm_manager_t *mgr)
```

Poll SPI channel for incoming frames (non-blocking).

Attempts to receive one frame from SPI channel with zero timeout (non-blocking). If frame is successfully received, it is dispatched via `internal_handle_frame()`. Timeout (no data available) is expected and normal - not an error condition.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Skip if SPI handle is nullptr (channel not configured)
3. Call `rx_spi_comm_receive()` with 0 ms timeout
4. On success: dispatch frame via `internal_handle_frame()`
5. On timeout: return `k_rx_err_timeout` (expected, not error)
6. On other errors: propagate error code

Design Note: Identical logic to `internal_poll_usb()` but for SPI channel. Code duplication is intentional (NASA Rule 4: keep functions short and simple).

Parameters:

Direction	Name	Description
inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must have valid spi_handle if SPI channel enabled Side effects: May invoke callback, modify ascii_buffer

Returns: `rx_err_t` Error code indicating poll result

Value	Description
<code>k_rx_ok</code>	Frame received and handled successfully
<code>k_rx_err_timeout</code>	No data available (normal, expected condition)
<code>k_rx_err_invalid_arg</code>	mgr is nullptr
<code>k_rx_err_crc</code>	CRC verification failed (corrupted frame)
<code>k_rx_err_invalid_state</code>	SPI peripheral not ready
(other)	Transport layer errors propagated

Pre: mgr must be non-NULL

Pre: If SPI channel enabled, spi_handle must be initialized

Post: On `k_rx_ok`: callback invoked, ASCII output sent (if enabled)

Post: On timeout: no side effects (no frame received)

Note: Non-blocking - always returns immediately

Note: Timeout is NOT an error - it means no data is available

Warning: Do not call if spi_handle is uninitialized (will return timeout)

Performance:: No data available: 5 us (fast path) Frame received: 50-150 us (SPI hardware transfer + callback)

See also: internal_handle_frame() Frame dispatch, rx_spi_comm_receive() SPI receive implementation

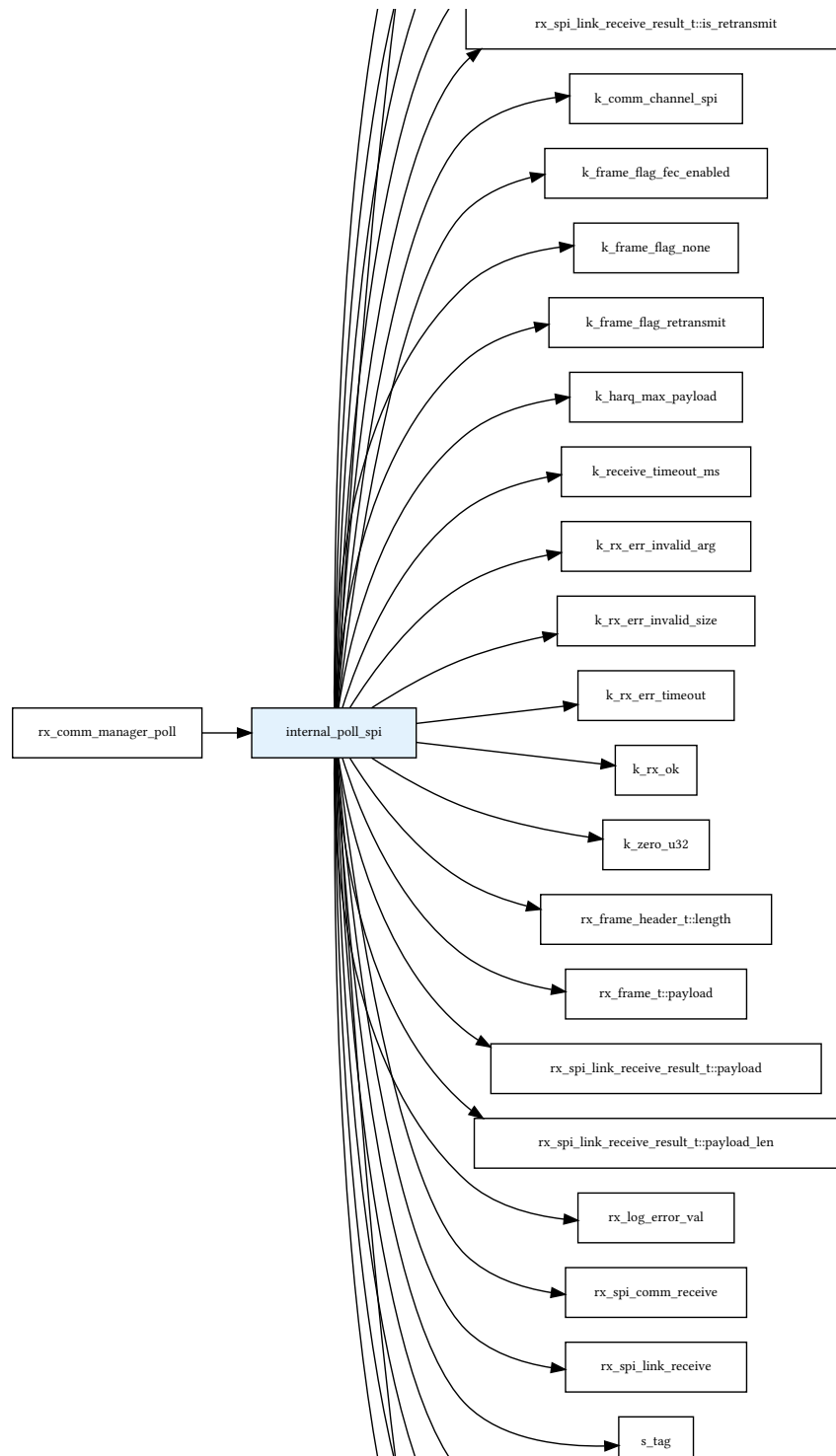


Figure 134: Call graph for internal_poll_spi

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:805`

Since Version 1.0.0

—

internal_process_event_queue

```
static void internal_process_event_queue(rx_comm_manager_t *mgr)
```

Process one event from the FIFO queue.

Dequeues the oldest event and attempts to send it. For SPI events, if the send fails, the event is re-enqueued with incremented retry count (up to `k_event_max_retries`). For USB events, failure is simply dropped (fire-and-forget).

Parameters:

Direction	Name	Description
inout	mgr	Communication manager handle

Pre: mgr must be non-NULL and initialized

Post: One event processed (sent or dropped on max retries)

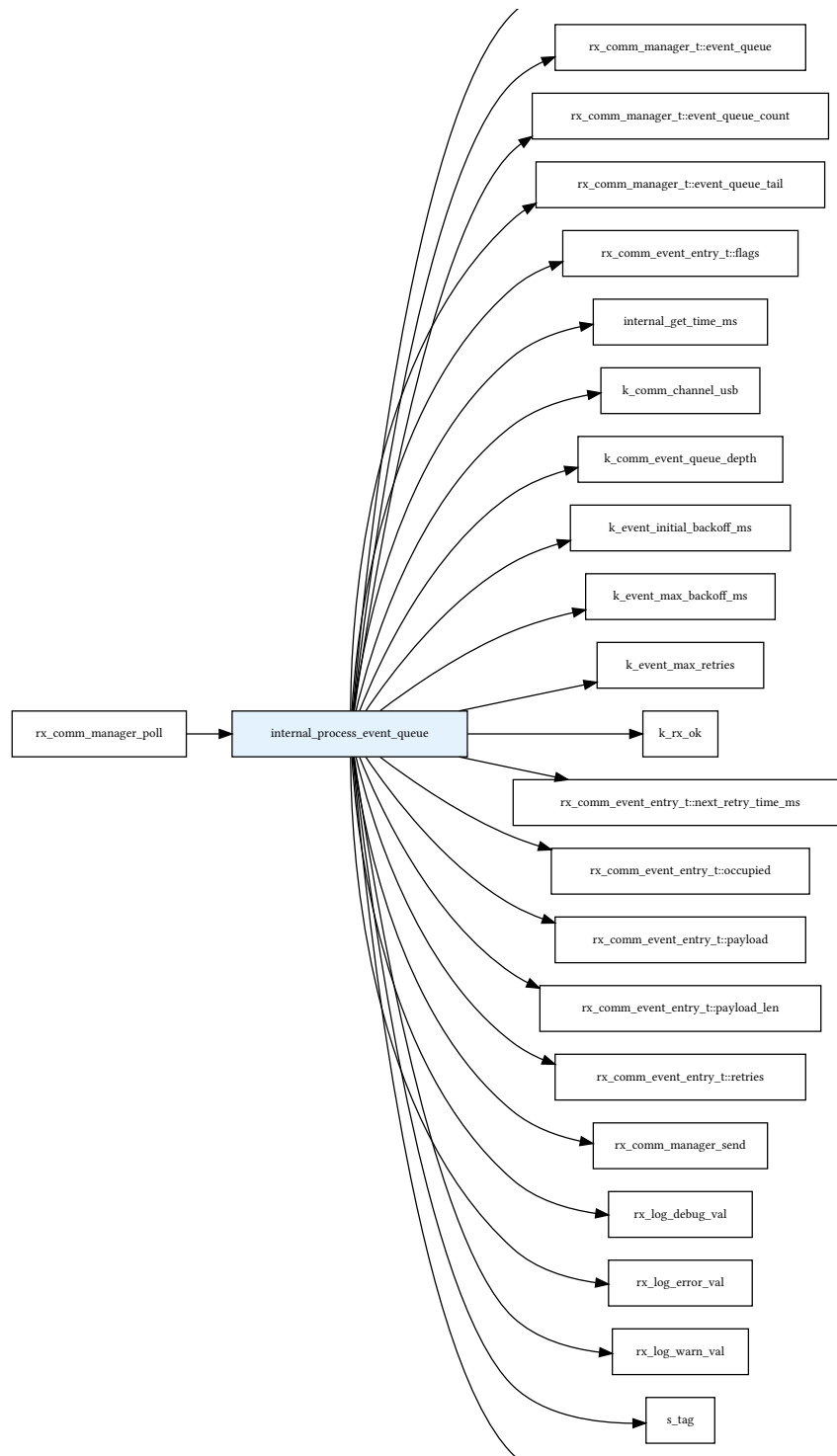


Figure 135: Call graph for internal_process_event_queue

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:889`

Since Version 1.1.0

—

internal_check_heartbeat

```
static void internal_check_heartbeat(rx_comm_manager_t *mgr)
```

Check heartbeat implicit timeout on all transports.

Evaluates the dual-detection heartbeat model. If no valid frame has been received on a transport within 200ms, the link is declared dead and the status callback is invoked.

Parameters:

Direction	Name	Description
inout	<code>mgr</code>	Communication manager handle

Pre: mgr must be non-NULL and initialized

Post: Link status updated for each transport

Figure 136: Call graph for `internal_check_heartbeat`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:976`

Since Version 1.1.0

—

rx_comm_manager_init

```
rx_err_t rx_comm_manager_init(rx_comm_manager_t *mgr, const
rx_comm_manager_config_t *cfg)
```

Initialize communication manager with channel configuration.

Initialize communication manager.

Performs complete initialization of communication manager handle, configuring enabled channels, callback registration, and ASCII debugging output. This function must be called before any other communication manager operations.

Algorithm:

1. Validate parameters: Check mgr pointer for nullptr (NASA Rule 5)
2. Clear handle: Zero-fill entire structure via memset (328 bytes)
3. Apply configuration: Copy config fields if cfg is non-NULL

usb_handle: Pointer to initialized USB comm handle (may be NULL) spi_handle: Pointer to initialized SPI comm handle (may be NULL) callback: User frame handler function (may be NULL) callback_ctx: User context pointer (may be NULL) enable_decoded_output: ASCII debugging flag

4. Mark initialized: Set mgr->initialized = true
5. Post-condition validation: Verify ascii_buffer properly cleared

Configuration Options:

- cfg = NULL: All channels disabled, no callback, ASCII output off
- usb_handle = NULL: USB channel disabled (poll skips USB)
- spi_handle = NULL: SPI channel disabled (poll skips SPI)
- callback = NULL: No callback invoked (silent frame consumption)
- enable_decoded_output = true: ASCII debugging enabled (Port 1)

Parameters:

Direction	Name	Description
out	mgr	Pointer to communication manager handle to initialize Valid range: Non-nullptr to allocated rx_comm_manager_t (328 bytes) Constraints: Must be allocated by caller (typically static or global) Side effects: Entire structure zero-filled, then populated from cfg Lifetime: Must remain valid until rx_comm_manager_deinit() called
in	cfg	Optional configuration structure Valid range: NULL or pointer to valid rx_comm_manager_config_t NULL behavior: All channels disabled, no callback, ASCII off Lifetime: Not stored - values copied into mgr Constraints: If non-NULL, all fields must be valid or nullptr as appropriate

Returns: rx_err_t Error code indicating initialization success or failure

Value	Description
k_rx_ok	Success - manager fully initialized and ready for operation
k_rx_err_invalid_arg	mgr is nullptr
k_rx_err_invalid_state	Post-condition validation failed (buffer not cleared)

Pre: mgr must point to allocated memory (uninitialized content OK)

Pre: USB/SPI handles (if provided) must already be initialized

Pre: Callback function (if provided) must be valid and reentrant

Post: mgr->initialized = true on success

Post: All mgr fields set to config values or zero

Post: mgr->ascii_buffer is zero-filled (256 bytes)

Note: This function does NOT initialize USB or SPI transports - caller must initialize rx_usb_comm and rx_spi_comm separately before passing handles to this function

Note: On success, at least one channel should be enabled (usb_handle or spi_handle non-NULL)

Note: Thread-safe ONLY if each thread uses a different mgr handle

Performance:: Execution time: 5 us @ 240 MHz Dominated by memset (328 bytes)

Configuration Example - Both Channels Enabled:: static rx_comm_manager_tg_comm_mgr;
static rx_usb_comm_handle_tg_usb_handle; static rx_spi_comm_handle_tg_spi_handle;

rx_usb_comm_config_tusb_cfg={.port=k_usb_port_proto};

rx_err_t err=rx_usb_comm_init(&g_usb_handle,&tusb_cfg); if(err!=k_rx_ok)return err;

rx_spi_comm_config_tspi_cfg={.spi_channel=0}; err=rx_spi_comm_init(&g_spi_handle,&tspi_cfg);

if(err!=k_rx_ok)return err;

rx_comm_manager_config_tcfg={.usb_handle=&g_usb_handle,.spi_handle=&g_spi_handle,.callback=on_frame_rec
err=rx_comm_manager_init(&g_comm_mgr,&cfg);

Configuration Example - USB Only, No Callback::

rx_comm_manager_config_tcfg={.usb_handle=&g_usb_handle,.spi_handle=NULLPTR,.callback=NULLPTR,.callback_ctx=1

rx_err_t err=rx_comm_manager_init(&g_comm_mgr,&cfg);

See also: rx_comm_manager_deinit() Cleanup and resource release, rx_comm_manager_poll()

Poll for incoming frames, rx_usb_comm_init() Initialize USB CDC transport,

rx_spi_comm_init() Initialize SPI transport



Figure 137: Call graph for `rx_comm_manager_init`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1129`

Since Version 1.0.0

—

rx_comm_manager_deinit

```
rx_err_t rx_comm_manager_deinit(rx_comm_manager_t *mgr)
```

Deinitialize communication manager and release resources.

Deinitialize communication manager.

Marks communication manager as uninitialized, preventing further operations. This function does NOT deinitialize USB or SPI transport handles - caller must separately call `rx_usb_comm_deinit()` and `rx_spi_comm_deinit()` if needed.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated). Function primarily serves as safety guard to prevent use-after-deinit.

Parameters:

Direction	Name	Description
inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must have been initialized via <code>rx_comm_manager_init()</code> Side effects: Clears mgr->initialized flag

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success - manager deinitialized
<code>k_rx_err_invalid_arg</code>	mgr is nullptr
<code>k_rx_err_invalid_state</code>	mgr was not initialized

Pre: mgr must be non-NULL

Pre: mgr must be initialized (mgr->initialized == true)

Post: mgr->initialized = false

Post: Subsequent calls to `rx_comm_manager_poll/send` will return `k_rx_err_invalid_state`

Note: Does NOT free memory (handle is statically allocated)

Note: Does NOT deinitialize transport handles (USB/SPI)

Note: Thread-safe if no concurrent operations on same mgr handle

Example:: `rx_err_t err=rx_comm_manager_deinit(&g_comm_mgr); if(err==k_rx_ok){
rx_usb_comm_deinit(&g_usb_handle); rx_spi_comm_deinit(&g_spi_handle); }`

See also: `rx_comm_manager_init()` Initialization, `rx_usb_comm_deinit()` USB transport cleanup, `rx_spi_comm_deinit()` SPI transport cleanup

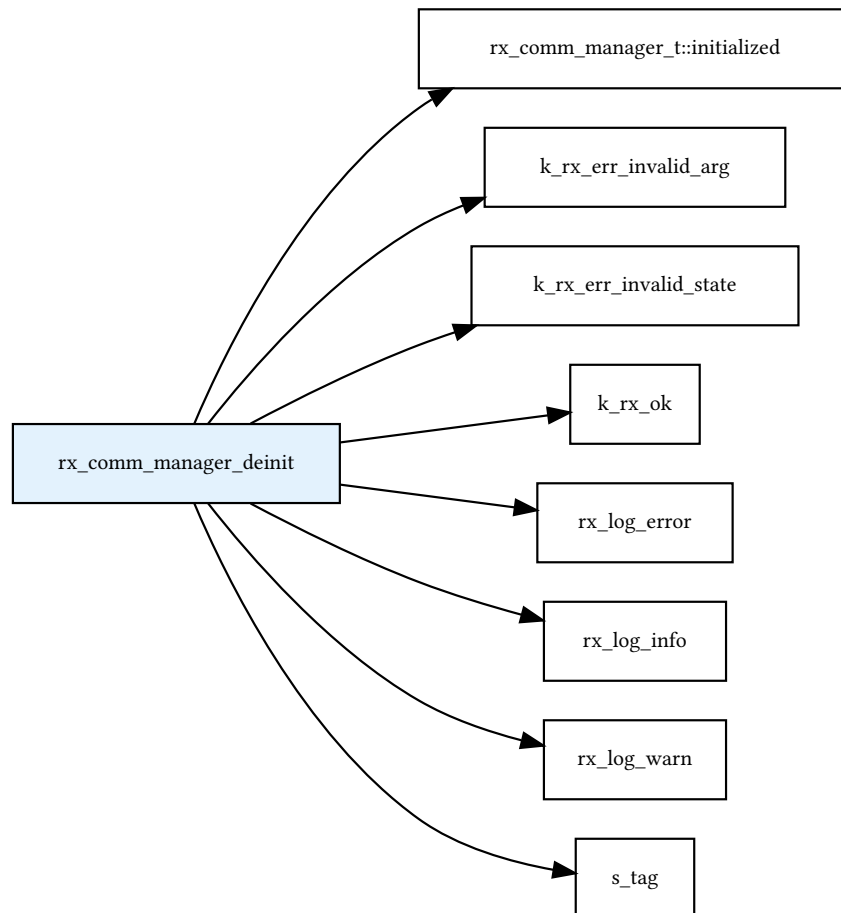


Figure 138: Call graph for `rx_comm_manager_deinit`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1249`

Since Version 1.0.0

—

rx_comm_manager_poll

```
rx_err_t rx_comm_manager_poll(rx_comm_manager_t *mgr)
```

Poll all enabled communication channels for incoming frames (non-blocking).

Poll all enabled channels for incoming frames.

Performs non-blocking poll of all enabled channels (USB and SPI) sequentially, dispatching any received frames to the registered user callback. Returns aggregate result indicating whether any frames were received across all channels.

Algorithm:

1. Validate parameters: Check mgr pointer and initialized flag (NASA Rule 5)
2. Poll USB channel via internal_poll_usb():

If k_rx_ok: Mark received=true, continue to SPI If k_rx_err_timeout: Continue to SPI (no data is normal) If other error: Return error immediately (critical failure)

3. Poll SPI channel via internal_poll_spi():

If k_rx_ok: Mark received=true If k_rx_err_timeout: Continue (no data is normal) If other error: Return error immediately (critical failure)

4. Return result:

k_rx_ok if ANY channel received frame k_rx_err_timeout if NO channels received (normal condition) Other error codes propagated from transport layers

Performance: 15-200 us typical, depends on:

- Number of enabled channels (USB/SPI/both)
- Whether frames are available (data path vs fast timeout path)
- ASCII debugging enabled (adds 50-150 us per frame)
- User callback execution time (not included in measurement)

Design Rationale:

- Non-blocking: Never blocks waiting for data (0 ms timeout)
- Fair scheduling: Both channels polled each iteration
- Timeout tolerance: Timeout is expected, not error condition
- Error prioritization: Critical errors returned immediately
- Aggregation: k_rx_ok if ANY channel received data

Parameters:

Direction	Name	Description
inout	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_comm_manager_init() Side effects: Invokes callback, modifies ascii_buffer if frames received

Returns: rx_err_t Error code indicating aggregate poll result

Value	Description
k_rx_ok	At least one frame received on any channel (success)
k_rx_err_timeout	No frames received on any channel (normal, not an error)
k_rx_err_invalid_arg	mgr is nullptr

k_rx_err_invalid_state	mgr not initialized
k_rx_err_crc	CRC verification failed on received frame (data corruption)
(other)	Transport layer critical errors propagated

Pre: mgr must be non-NULL

Pre: mgr must be initialized

Pre: At least one channel should be enabled (usb_handle or spi_handle non-NULL)

Post: User callback invoked for each successfully received frame

Post: ASCII output sent to Port 1 if enabled

Note: Non-blocking - always returns immediately

Note: k_rx_err_timeout is NORMAL - means no data available (not an error)

Note: User callback execution time is NOT included in performance measurements

Warning: Call this function regularly (e.g., 100 Hz) to avoid buffer overruns

Warning: User callback must not block for extended periods

Performance:: No frames available: 15 us (both channels timeout, fast path) USB frame received: 100-250 us (USB bulk transfer + callback) SPI frame received: 50-150 us (SPI hardware transfer + callback) Both frames: 150-400 us (both channels + callbacks) With ASCII debugging: Add 50-150 us per frame

Typical Usage - ThreadX Communication Task::

```
static void comm_task_entry(ULONG thread_input) { (void) thread_input;

while(1) { rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);

if (err != k_rx_ok && err != k_rx_err_timeout) { log_error("Comm poll failed: %d", err);
tx_thread_sleep(100); continue; }

tx_thread_sleep(10); }
```

Error Handling - Distinguishing Timeout from Errors::

```
rx_err_t err = rx_comm_manager_poll(&g_comm_mgr); if (err == k_rx_ok) { }
else if (err == k_rx_err_timeout) { } else { handle_comm_error(err); }
```

See also: internal_poll_usb() USB channel polling implementation, internal_poll_spi() SPI channel polling implementation, internal_handle_frame() Frame dispatch, rx_comm_manager_init() Initialization with channel configuration

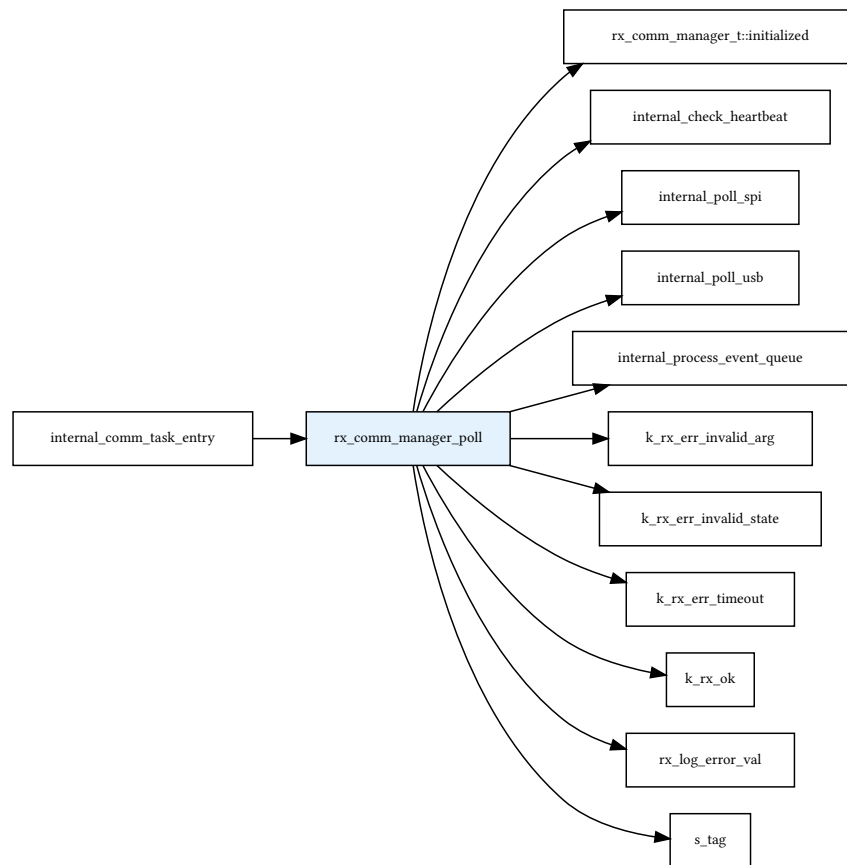


Figure 139: Call graph for rx_comm_manager_poll

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1385`

Since Version 1.0.0

—

rx_comm_manager_send

```
rx_err_t rx_comm_manager_send(rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)
```

Send frame on specified communication channel.

Send frame on specific channel.

Routes frame transmission to the appropriate transport layer (USB or SPI) based on specified channel. Supports configurable frame type, flags, and variable-length payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.

Algorithm:

1. Validate parameters (NASA Rule 5):

mgr and params pointers non-NULL Manager initialized flag Payload pointer valid if payload_len > 0 payload_len <= k_frame_max_payload (255 bytes)

2. Route to channel:

USB: Verify usb_handle non-NULL, call rx_usb_comm_send() SPI: Verify spi_handle non-NULL, call rx_spi_comm_send() Invalid channel: Return k_rx_err_invalid_arg

3. On success, if ASCII debugging enabled:

Build temporary rx_frame_t from params Format as ASCII via internal_output_decoded() Send to USB Port 1 (TX direction)

Performance:

- USB: 30-150 us (depends on payload size, USB bulk transfer)
- SPI: 20-100 us (depends on payload size, SPI hardware transfer)
- ASCII debugging: Add 50-150 us if enabled

Parameters:

Direction	Name	Description
in	mgr	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_comm_manager_init() Side effects: May modify ascii_buffer if decoded output enabled
in	params	Send parameters structure Valid range: Non-nullptr to valid rx_comm_send_params_t Constraints: All fields must be valid: channel: k_comm_channel_usb or k_comm_channel_spi type: Frame type (command, request, response, etc.) flags: Frame flags (k_frame_flag_none or specific flags) payload: Non-NULL if payload_len > 0, may be NULL if payload_len == 0 payload_len: [0, k_frame_max_payload] (0-255 bytes) Lifetime: Not stored - values used immediately

Returns: rx_err_t Error code indicating send success or failure

Value	Description
k_rx_ok	Frame sent successfully
k_rx_err_invalid_arg	mgr or params is nullptr, or payload invalid
k_rx_err_invalid_arg	payload_len > k_frame_max_payload (255 bytes)
k_rx_err_invalid_arg	Invalid channel specified
k_rx_err_invalid_arg	Payload is nullptr but payload_len > 0
k_rx_err_invalid_state	Manager not initialized
k_rx_err_invalid_state	Channel not configured (handle is nullptr)
k_rx_err_hw	USB/SPI hardware error (transport layer)
(other)	Transport layer errors propagated

Pre: mgr must be non-NULL and initialized

Pre: params must be non-NULL with all valid fields

Pre: Channel handle (USB/SPI) must be initialized

Pre: If payload_len > 0, payload must point to valid data

Post: Frame transmitted on specified channel if successful

Post: ASCII output sent to Port 1 if enabled and successful

Note: Payload sequence number and CRC-32 managed by transport layer

Note: ASCII debugging adds overhead - disable in production

Warning: Ensure channel is ready before sending (use rx_comm_manager_channel_ready)

Send Parameters Structure:: typedef struct { rx_comm_channel_t channel; uint8_t type; uint8_t flags; const uint8_t* payload; uint32_t payload_len; } rx_comm_send_params_t;

Example - Send Command on USB::

```
uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

rx_comm_send_params_t params = { .channel = k_comm_channel_usb, .type = k_frame_type_command, .flags = k_frame_flag_none,
                                 .payload = cmd_payload, .payload_len = 8 };

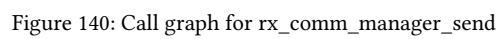
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params); if (err == k_rx_ok) { }
```

Example - Send Empty Frame (No Payload)::

```
rx_comm_send_params_t params = { .channel = k_comm_channel_spi, .type = k_frame_type_response, .flags = k_frame_flag_none,
                                 .payload = NULL, .payload_len = 0 };

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
```

See also: rx_comm_manager_respond() Convenience wrapper for response frames,
rx_comm_send_params_t Send parameters structure, rx_usb_comm_send() USB CDC send implementation, rx_spi_comm_send() SPI send implementation



Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1540`

Since Version 1.0.0

—

rx_comm_manager_respond

```
rx_err_t rx_comm_manager_respond(rx_comm_manager_t *mgr, rx_comm_channel_t
channel, const uint8_t *payload, uint32_t payload_len)
```

Send response frame on specified channel (convenience wrapper).

Send response on same channel as received frame.

Convenience function for sending response frames. Automatically sets frame type to `k_frame_type_response` and flags to `k_frame_flag_none`. Simplifies common use case of responding to commands/requests.

Algorithm:

1. Build `rx_comm_send_params_t` with:

channel: User-specified type: `k_frame_type_response` (fixed) flags: `k_frame_flag_none` (fixed)

payload: User-specified payload_len: User-specified

2. Call `rx_comm_manager_send()` with constructed params

3. Return result from `rx_comm_manager_send()`

When to Use:

- Responding to commands with telemetry data
- Sending acknowledgments
- Replying to requests

When NOT to Use:

- Need custom frame type (use `rx_comm_manager_send` instead)
- Need custom flags (use `rx_comm_manager_send` instead)

Parameters:

Direction	Name	Description
in	<code>mgr</code>	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via <code>rx_comm_manager_init()</code>
in	<code>channel</code>	Channel to send response on Valid range: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> Constraints: Channel handle must be configured
in	<code>payload</code>	Response payload data Valid range: Non-NULL if <code>payload_len > 0</code> , may be NULL if <code>payload_len == 0</code> Constraints: Must contain valid data for <code>payload_len</code> bytes Lifetime: Copied immediately, does not need to persist
in	<code>payload_len</code>	Response payload length in bytes Valid range: [0, <code>k_frame_max_payload</code>] (0-255 bytes) Units: Bytes

Returns: `rx_err_t` Error code (same as `rx_comm_manager_send`)

Value	Description
k_rx_ok	Response sent successfully
k_rx_err_invalid_arg	mgr is nullptr or payload invalid
k_rx_err_invalid_state	Manager not initialized or channel not configured
(other)	Errors propagated from rx_comm_manager_send()

Pre: Same preconditions as rx_comm_manager_send()

Post: Response frame transmitted with type=response, flags=none

Note: Equivalent to calling rx_comm_manager_send() with type=k_frame_type_response

Note: This is a thin wrapper - no additional validation performed

Example - Send Telemetry Response::

```
static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
{
    uint8_t telemetry[16];

    telemetry[0] = get_temperature_celsius();
    telemetry[1] = get_current_ma();

    rx_err_t err = rx_comm_manager_respond(&g_comm_mgr, channel, telemetry, sizeof(telemetry));
    if (err != k_rx_ok) {}
}
```

Example - Send Empty Acknowledgment::

```
rx_err_t err = rx_comm_manager_respond(&g_comm_mgr, k_comm_channel_usb, NULL, 0);
```

See also: rx_comm_manager_send() Full send function with all parameters,
rx_comm_send_params_t Send parameters structure

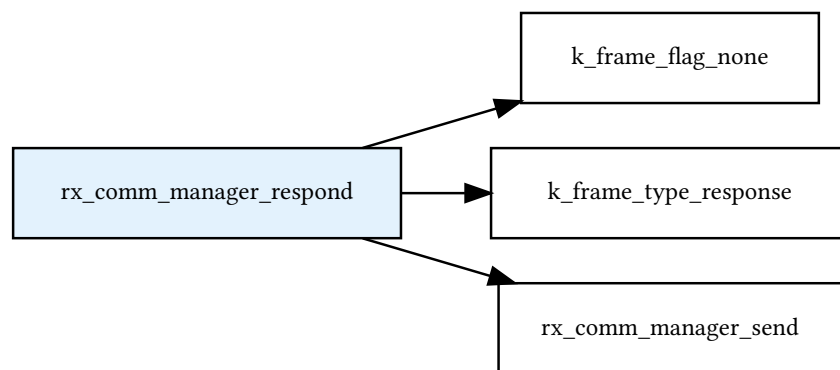


Figure 141: Call graph for rx_comm_manager_respond

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1723`

Since Version 1.0.0

rx_comm_manager_stream_send

```
rx_err_t rx_comm_manager_stream_send(rx_comm_manager_t *mgr, const
rx_comm_send_params_t *params)
```

Send data on the stream path (fire-and-forget).

Stream path is for time-sensitive data like telemetry where stale data has no value. Sends immediately with NO retries on any transport. If the send fails, the data is simply dropped.

- USB CDC: Direct send, no retries (USB HW provides reliability)
- SPI: Direct send, no retries (stale telemetry is worthless)

Parameters:

Direction	Name	Description
inout	mgr	Pointer to initialized manager
in	params	Send parameters (channel, type, flags, payload, payload_len)

Returns: k_rx_err_invalid_state if not initialized or channel not enabled

Pre: mgr must be initialized

Pre: params must be valid

Post: Data sent or silently dropped on failure

Note: This is a thin wrapper over rx_comm_manager_send() for semantic clarity] **See also:** rx_comm_manager_event_send() For reliable command delivery



Figure 142: Call graph for rx_comm_manager_stream_send

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1743`

Since Version 1.1.0

—

rx_comm_manager_event_send

```
rx_err_t rx_comm_manager_event_send(rx_comm_manager_t *mgr, const
rx_comm_send_params_t *params)
```

Queue data on the event path (reliable, SPI retry).

Event path is for commands and other data requiring reliable delivery. The payload is copied into a static FIFO queue and processed by the poll loop. Retry behavior depends on transport:

- SPI: Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)

- USB: Fire-and-forget (USB HW reliability is sufficient)

Queue is processed in FIFO order by `rx_comm_manager_poll()`.

Parameters:

Direction	Name	Description
inout	<code>mgr</code>	Pointer to initialized manager
in	<code>params</code>	Send parameters (channel, type, flags, payload, payload_len)

Returns: `k_rx_err_no_mem` if event queue is full

Pre: mgr must be initialized

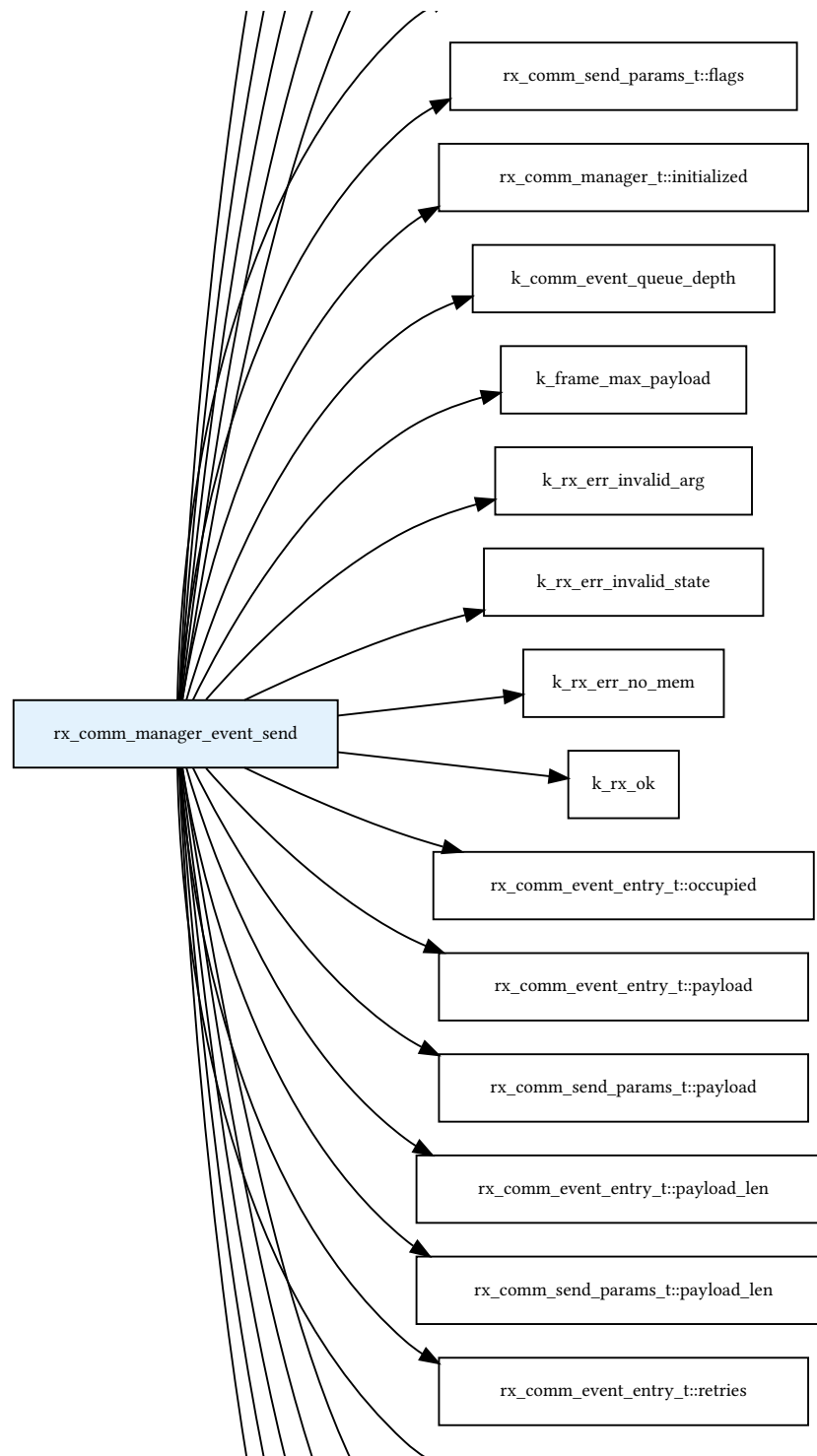
Pre: params must be valid

Pre: `params->payload_len <= k_frame_max_payload`

Post: Payload copied to queue, will be sent by next poll()

Note: Payload is COPIED into queue - caller can reuse buffer immediately

Warning: Queue depth is `k_comm_event_queue_depth` (8). If full, returns error.] **See also:** `rx_comm_manager_stream_send()` For fire-and-forget telemetry

Figure 143: Call graph for `rx_comm_manager_event_send`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1751`

Since Version 1.1.0

—

rx_comm_manager_link_status

```
rx_err_t rx_comm_manager_link_status(const rx_comm_manager_t *mgr,  
rx_comm_channel_t channel, rx_comm_link_status_t *status)
```

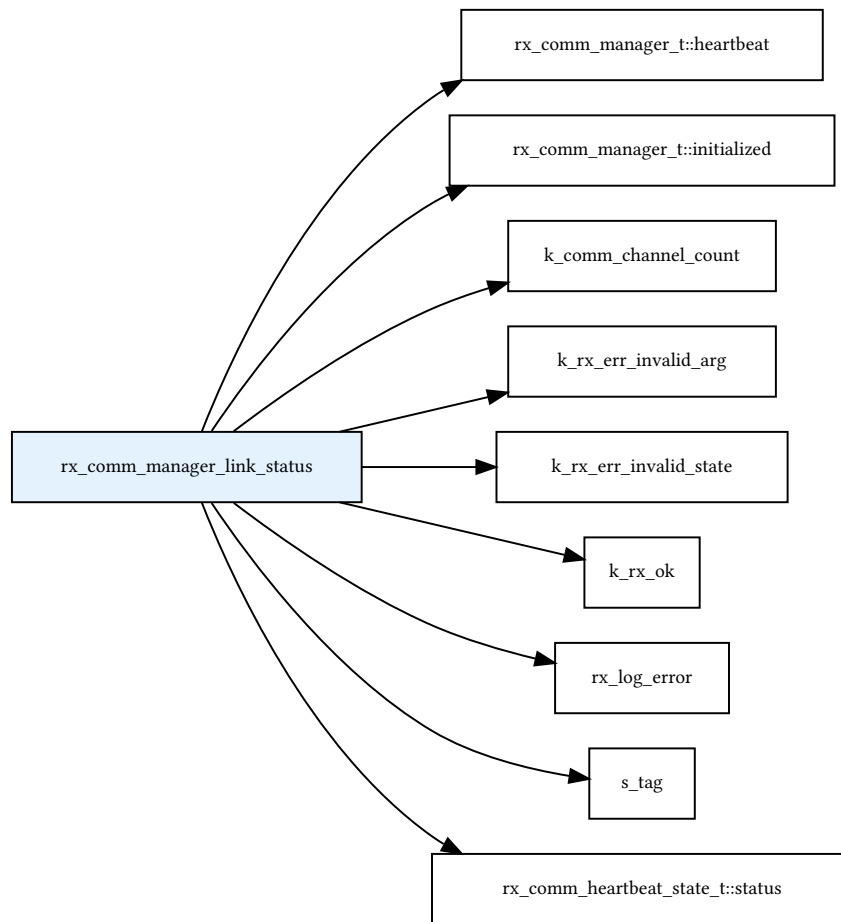
Query link health status for a transport channel.

Returns the current heartbeat-derived link status for the given channel. Status is updated by `rx_comm_manager_poll()` based on the dual-detection heartbeat model (200ms implicit timeout).

Parameters:

Direction	Name	Description
in	<code>mgr</code>	Pointer to initialized manager
in	<code>channel</code>	Channel to query
out	<code>status</code>	Receives current link status

Returns: `k_rx_err_invalid_arg` if `mgr` or `status` is `nullptr`

Figure 144: Call graph for `rx_comm_manager_link_status`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1804`

Since Version 1.1.0

—

rx_comm_manager_channel_ready

```
rx_err_t rx_comm_manager_channel_ready(rx_comm_manager_t *mgr, rx_comm_channel_t
channel, bool *ready)
```

Query if communication channel is ready for send/receive operations.

Check if channel is ready for communication.

Checks if specified channel is configured and ready for communication. USB channel readiness depends on USB enumeration and configuration by host. SPI channel is ready immediately if handle is configured (no enumeration required).

Readiness Criteria:

- USB: `usb_handle` non-NULL AND USB device enumerated and configured by host
- SPI: `spi_handle` non-NULL (always ready if configured)

Use Cases:

- Check readiness before sending to avoid errors
- Implement fallback logic (try USB, fall back to SPI)
- Log channel availability status

Parameters:

Direction	Name	Description
in	<code>mgr</code>	Communication manager handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via <code>rx_comm_manager_init()</code>
in	<code>channel</code>	Channel to query Valid range: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> Constraints: Must be valid channel ID
out	<code>ready</code>	Pointer to receive ready status Valid range: Non-nullptr to bool Output values: true (ready) or false (not ready) On error: Set to false

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Query successful, ready flag updated
<code>k_rx_err_invalid_arg</code>	<code>mgr</code> or <code>ready</code> is nullptr, or channel invalid
<code>k_rx_err_invalid_state</code>	Manager not initialized

Pre: `mgr` must be non-NULL and initialized

Pre: `ready` must be non-NULL

Pre: channel must be valid (USB or SPI)

Post: `*ready` set to true or false based on channel state

Post: On error, `*ready` set to false (safe default)

Note: USB readiness can change at runtime (host connects/disconnects)

Note: SPI readiness is static (always ready if handle configured)

Note: This function does NOT initiate communication - only queries state

Example - Send with Fallback:: `uint8_tpayload[16]={...};`

```
boolusb_ready=false; rx_err_terr=rx_comm_manager_channel_ready(&g_comm_mgr,
k_comm_channel_usb, &usb_ready); if(err==k_rx_ok&&usb_ready)
```

```
{ err=rx_comm_manager_respond(&g_comm_mgr, k_comm_channel_usb, payload,
sizeof(payload)); }else{ boolspi_ready=false; err=rx_comm_manager_channel_ready(&g_comm_mgr,
k_comm_channel_spi, &spi_ready); if(err==k_rx_ok&&spi_ready)
{ err=rx_comm_manager_respond(&g_comm_mgr, k_comm_channel_spi, payload,
sizeof(payload)); } }
```

Example - Log Channel Status:: voidlog_channel_status(void)

```
{ boolusb_ready=false,spi_ready=false;
```

```
rx_comm_manager_channel_ready(&g_comm_mgr,k_comm_channel_usb,&usb_ready);
```

```
rx_comm_manager_channel_ready(&g_comm_mgr,k_comm_channel_spi,&spi_ready);
```

```
printf("USB:%s,SPI:%sn", usb_ready?"READY":"NOTREADY", spi_ready?"READY":"NOTREADY"); }
```

See also: rx_usb_is_configured() USB enumeration status, rx_comm_manager_send() Send frame on channel

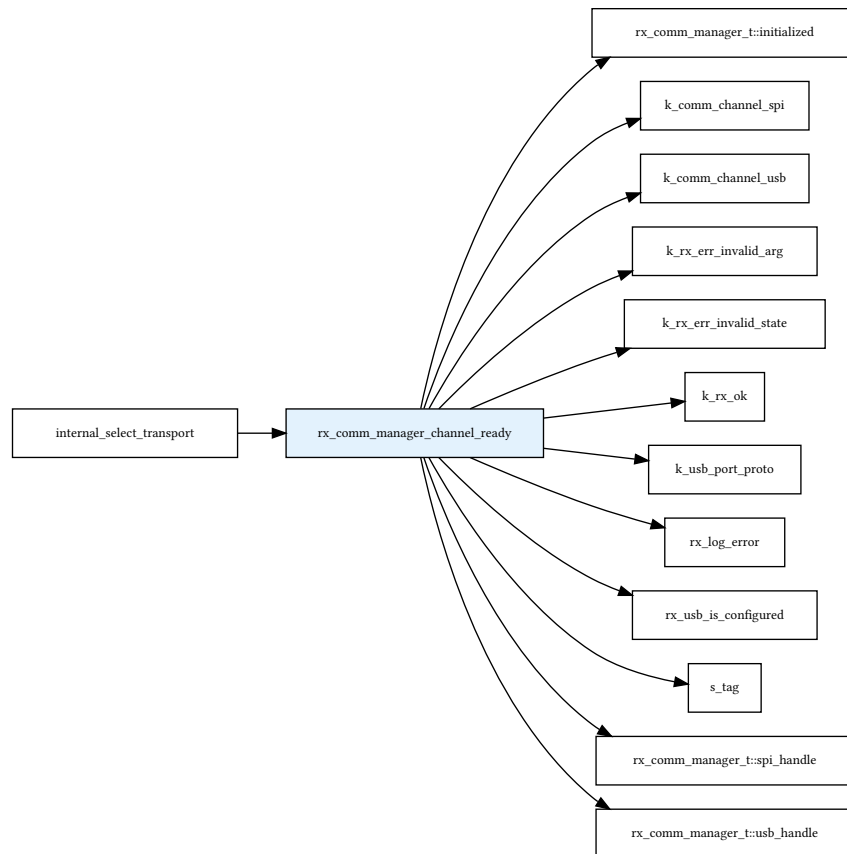


Figure 145: Call graph for rx_comm_manager_channel_ready

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:1922`

Since Version 1.0.0

rx_comm_manager_channel_name

```
const char * rx_comm_manager_channel_name(rx_comm_channel_t channel)
```

Get human-readable name for communication channel.

Get channel name string.

Returns static string name for given channel enum value. Used for logging, debugging, and user interface display. Always returns a valid string (never NULL).

Returned Strings:

- k_comm_channel_usb -> "USB"
- k_comm_channel_spi -> "SPI"
- Invalid/unknown -> "UNKNOWN"

Use Cases:

- Logging channel activity
- Debugging frame traffic
- User interface display
- Error messages

Parameters:

Direction	Name	Description
in	channel	Communication channel enum value Valid range: Any rx_comm_channel_t value Constraints: None (invalid values return "UNKNOWN")

Returns: const char* Human-readable channel name (never NULL)

Value	Description
USB	If channel == k_comm_channel_usb
SPI	If channel == k_comm_channel_spi
UNKNOWN	If channel is invalid or out of range

Pre: None (accepts any input)

Post: Returns pointer to static string (valid for program lifetime)

Note: Returned string is static - do not free

Note: Thread-safe (returns read-only static strings)

Note: Never returns NULL - always safe to use in printf

Example - Logging:: static void on_frame_received(rx_comm_channel_t channel, const rx_frame_t *frame, void *ctx) { printf("Frame received on %s: type=%d len=%d\n", rx_comm_manager_channel_name(channel), frame->header.type, frame->header.length); }

Example - Error Messages:: rx_err_t err = rx_comm_manager_send(&g_comm_mgr, ¶ms); if (err != k_rx_ok) { fprintf(stderr, "Failed to send on %s: error %d\n", rx_comm_manager_channel_name(params.channel), err); }

See also: rx_comm_channel_t Channel enumeration

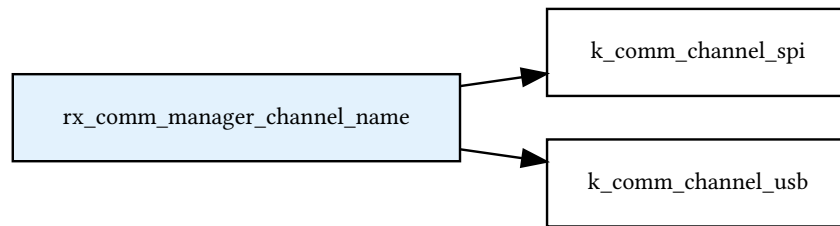


Figure 146: Call graph for rx_comm_manager_channel_name

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:2015`

Since Version 1.0.0

—

rx_comm_manager_process_retransmits

```
rx_err_t rx_comm_manager_process_retransmits(rx_comm_manager_t *mgr, const
uint32_t current_time_ms)
```

Process pending retransmissions on SPI channel.

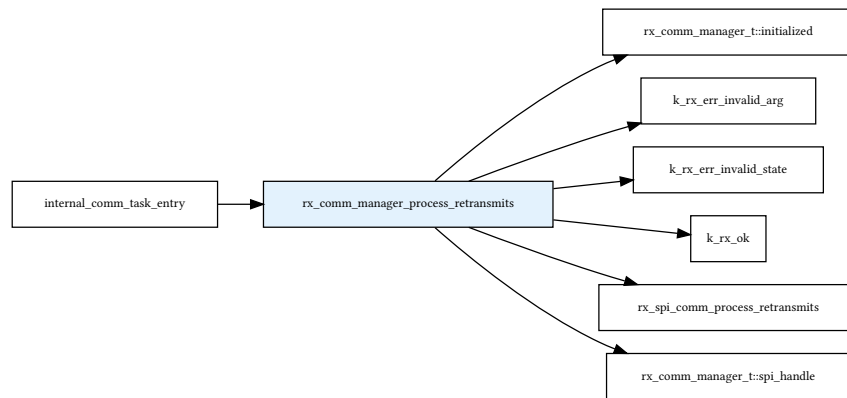
Thin pass-through to `rx_spi_comm_process_retransmits()` on the SPI handle. Call this from the communication task polling loop.

Parameters:

Direction	Name	Description
inout	mgr	Communication manager
in	current_time_ms	Current system time in milliseconds

Returns: rx_err_t Error code from `rx_spi_comm_process_retransmits()`

Value	Description
k_rx_ok	No action needed or retransmit sent
k_rx_err_invalid_arg	mgr is nullptr
k_rx_err_invalid_state	mgr not initialized
k_rx_err_retry_limit	Max retries exceeded

Figure 147: Call graph for `rx_comm_manager_process_retransmits`

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:2032`

Since Version 1.1.0

—

`rx_comm_manager_set_auto_retransmit`

```
rx_err_t rx_comm_manager_set_auto_retransmit(rx_comm_manager_t *mgr, const bool
enabled, const rx_spi_comm_retransmit_config_t *config)
```

Enable or disable automatic retransmission on SPI channel.

Thin pass-through to `rx_spi_comm_set_auto_retransmit()` on the SPI handle.

Parameters:

Direction	Name	Description
inout	<code>mgr</code>	Communication manager
in	<code>enabled</code>	true to enable, false to disable
in	<code>config</code>	Retransmit config (NULL to keep current/defaults)

Returns: `rx_err_t` Error code from `rx_spi_comm_set_auto_retransmit()`

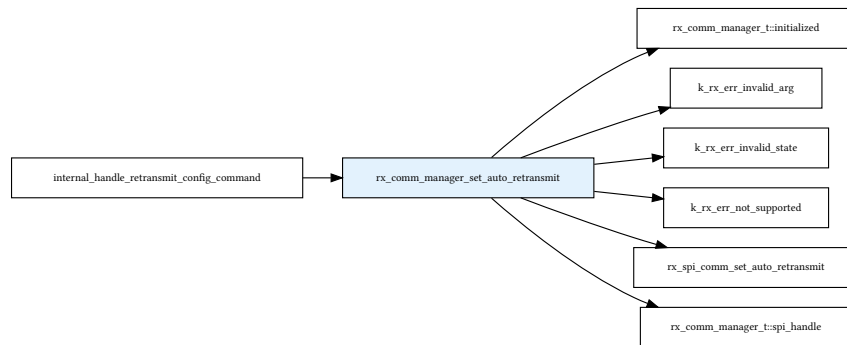


Figure 148: Call graph for rx_comm_manager_set_auto_retransmit

Defined in `libs/rx_comm_manager/src/rx_comm_manager.c:2049`

Since Version 1.1.0

rx_bit_constants.h

Fundamental Bit and Byte Size Constants for Protocol and Data Manipulation.

Includes:

- `<stdint.h>`

rx_bit_sizes_t

Fundamental bit and byte size constants for data type sizes.

Defines universal constants for bit counts in standard data types (byte, word16, word32, word64). These constants eliminate magic numbers in protocol encoding, buffer calculations, and bit manipulation operations.

Underlying Type: `uint8_t` (8-bit unsigned) - sufficient for all values (max = 64) Guaranteed Sizes: All values are compile-time constants (no runtime overhead)

Value	Name	Description
= 8	<code>k_rx_bits_per_byte</code>	Bits per byte. Universal constant (CHAR_BIT = 8 per C99). Used for byte extraction, buffer sizing, bit-to-byte conversion. Range: Always 8 on modern systems.
= 16	<code>k_rx_bits_per_word16</code>	Bits per 16-bit word (<code>uint16_t</code>). Guaranteed by C99 exact-width types. Used for protocol fields, endianness conversion, CRC-16 operations. Range: Always 16.
= 32	<code>k_rx_bits_per_word32</code>	Bits per 32-bit word (<code>uint32_t</code>). Guaranteed by C99 exact-width types. Used for CRC-32, hash values, register access, Protocol Buffer fixed32. Range: Always 32.
= 64	<code>k_rx_bits_per_word64</code>	Bits per 64-bit word (<code>uint64_t</code>). Guaranteed by C99 exact-width types. Used for timestamps, large counters, Protocol Buffer fixed64. Range: Always 64.

rx_bit_shifts_t

Bit shift constants for byte and word extraction from multi-byte values.

Provides named constants for common bit-shift operations used in protocol encoding, endianness conversion, and byte-wise data access. These constants make code self- documenting and eliminate magic shift values.

Underlying Type: uint8_t (8-bit unsigned) - sufficient for all values (max = 24) Relationship to rx_bit_sizes_t:

- k_rx_shift_byte = k_rx_bits_per_byte (both = 8)
- k_rx_shift_word16 = k_rx_bits_per_word16 (both = 16)
- k_rx_shift_word24 = 3 x k_rx_bits_per_byte (3 bytes = 24 bits)

Value	Name	Description
= 8	k_rx_shift_byte	Shift amount to access next byte (8 bits = 1 byte). Use for extracting byte N from multi-byte word. Example: byte1 = (word >> k_rx_shift_byte) & 0xFF. Equivalent to k_rx_bits_per_byte but semantically clearer for shift operations.
= 16	k_rx_shift_word16	Shift amount to access upper 16 bits of 32-bit word (16 bits = 2 bytes). Use for extracting high word from dword. Example: high_word = (dword >> k_rx_shift_word16) & 0xFFFF. Also used for big-endian 16-bit packing/unpacking.
= 24	k_rx_shift_word24	Shift amount to access byte 3 (MSB) of 32-bit word (24 bits = 3 bytes). Use for extracting most significant byte. Example: msb = (word >> k_rx_shift_word24) & 0xFF. Critical for big-endian 32-bit encoding.

—

rx_check.h

Validation and Error Checking Macros for RX72N Firmware.

Comprehensive error checking and validation infrastructure for safety-critical embedded systems. Provides type-safe macros for parameter validation, error propagation, and assertion checking aligned with NASA Power of 10 Rule 5 (minimum 2 assertions per function).

Includes:

- "rx_err.h"
- "rx_log.h"

RX_ASSERT

```
#define RX_ASSERT do
{
    if (!(condition))
    {
        internal_rx_fatal_error("ASSERT", message, k_rx_fail);
    }
} while (0)
```

Runtime assertion with fatal error on failure (NASA Rule 5 compliance).

```
—  
  
RX_ERROR_CHECK  
#define RX_ERROR_CHECK do  
{  
    rx_err_t err_rc_ = (err);  
    \br/>    if (rx_err_is_error(err_rc_))  
    {  
        \br/>        internal_rx_fatal_error("ERROR_CHECK", "Fatal error detected", err_rc_);  
    }  
    \br/>} while (0)
```

Fatal error checking - halt system if error code indicates failure.

```
—  
  
RX_ERROR_CHECK_WITHOUT_ABORT  
#define RX_ERROR_CHECK_WITHOUT_ABORT do  
{  
    rx_err_t err_rc_ = (err);  
    \br/>    if (rx_err_is_error(err_rc_))  
    {  
        \br/>        rx_log_error_val("ERROR_CHECK", "Error", err_rc_);  
    }  
    \br/>} while (0)
```

Check error code and log on failure (non-fatal).

```
—  
  
RX_RETURN_ON_ERROR  
#define RX_RETURN_ON_ERROR do  
{  
    rx_err_t err_rc_ = (err);  
    \br/>    if (rx_err_is_error(err_rc_))  
    {  
        \br/>        rx_log_error(tag, message);  
        \br/>        rx_log_error_val(tag, "Error", err_rc_);  
        \br/>        return err_rc_;  
    }  
    \br/>} while (0)
```

Return early on error.

—

RX_RETURN_VOID_ON_ERROR

```
#define RX_RETURN_VOID_ON_ERROR do
{
    rx_err_t err_rc_ = (err);
    \
    if (rx_err_is_error(err_rc_))
    {
        rx_log_error(tag, message);
        \
        rx_log_error_val(tag, "Error", err_rc_);
        \
        return;
    }
    \
} while (0)
```

Return void on error (for void functions).

—

RX_RETURN_NULL_ON_ERROR

```
#define RX_RETURN_NULL_ON_ERROR do
{
    rx_err_t err_rc_ = (err);
    \
    if (rx_err_is_error(err_rc_))
    {
        rx_log_error(tag, message);
        \
        rx_log_error_val(tag, "Error", err_rc_);
        \
        return nullptr;
    }
    \
} while (0)
```

Return NULL on error (for pointer-returning functions).

—

RX_CHECK_NULL_PTR

```
#define RX_CHECK_NULL_PTR do
{
    if ((ptr) == nullptr)
    {
        rx_log_error(tag, message);
        \
        return k_rx_err_null_ptr;
    }
    \
} while (0)
```

Check for null pointer and return error.

—

RX_CHECK_RANGE

```
#define RX_CHECK_RANGE do
{
    if (((value) < (min)) || ((value) > (max)))
    {
        rx_log_error("CHECK", "Range check failed");
    }
} while (0)
```

Check that a value is within an inclusive range.

RX_CHECK_RANGE_TAG

```
#define RX_CHECK_RANGE_TAG do
{
    (void)(min); /* Document minimum bound; explicit check needed if min > 0 */
    if ((value) > (max))
    {
        rx_log_error(tag, "Range check failed");
    }
} while (0)
```

Check that a value is within an inclusive range with tag.

Note: Only checks upper bound to avoid -Wtype-limits warnings when min == 0 and value is unsigned (comparison is always false). For non-zero min bounds, add an explicit lower-bound check before this macro. The min parameter is documented but not checked.

RX_VALIDATE_PTR

```
#define RX_VALIDATE_PTR RX_CHECK_NULL_PTR(ptr, tag, message)
```

Validate pointer pre-condition and return error.

RX_VALIDATE_INIT

```
#define RX_VALIDATE_INIT do
{
    if (!(initialized))
    {
        rx_log_error(tag, message);
    }
} while (0)
```

```
\
} while (0)
```

Validate initialization state pre-condition.

internal_rx_fatal_error

```
static void internal_rx_fatal_error(const char *tag, const char *message,
rx_err_t err)
```

Halt execution with diagnostic error message (fatal error handler).

Terminates firmware execution after logging detailed error diagnostics to debug UART. This is a non-returning function used for unrecoverable errors where continuing execution would be unsafe.

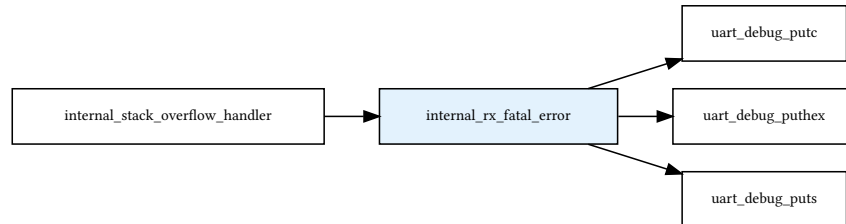


Figure 149: Call graph for `internal_rx_fatal_error`

Defined in `libs/rx_core/inc/rx_check.h:390`

rx_err.h

Error Code Definitions for RX72N Firmware.

Comprehensive error code system for the STAR RX72N motor controller firmware. Provides type-safe error handling with categorized error codes following C23 typed enum standards for embedded safety-critical systems.

Includes:

- `<stdbool.h>`
- `<stdint.h>`

rx_err_codes_t

Comprehensive error code enumeration for RX72N firmware.

Type-safe error codes using C23 typed enum syntax with explicit `uint16_t` underlying type. All error codes are compile-time constants organized into logical categories for systematic error handling and debugging.

Value	Name	Description
<code>= 0</code>	<code>k_rx_ok</code>	Success - operation completed as requested.
<code>= 0x101</code>	<code>k_rx_fail</code>	Generic unspecified failure.
<code>= 0x102</code>	<code>k_rx_err_no_mem</code>	Out of memory - allocation failed.

= 0x103	k_rx_err_invalid_arg	Invalid function argument.
= 0x104	k_rx_err_invalid_state	Invalid state for requested operation.
= 0x105	k_rx_err_invalid_size	Invalid size parameter.
= 0x106	k_rx_err_not_found	Requested item not found.
= 0x107	k_rx_err_not_supported	Feature or operation not supported.
= 0x108	k_rx_err_timeout	Operation timed out.
= 0x109	k_rx_err_busy	Resource busy - operation cannot proceed.
= 0x10A	k_rx_err_no_data	No application data available (control frame received).
= 0x10B	k_rx_err_would_block	Non-blocking operation would block.
= 0x10C	k_rx_err_exists	Item already exists - cannot create.
= 0x10D	k_rx_err_empty	Container empty - no data available.
= 0x10E	k_rx_err_cancelled	Operation cancelled by request.
= 0x10F	k_rx_err_not_initialized	Module not initialized.
= 0x110	k_rx_err_estop	Emergency stop active - operation forbidden.
= 0x201	k_rx_err_hw_init_failed	Hardware initialization failed.
= 0x202	k_rx_err_hw_not_ready	Hardware not ready for operation.
= 0x203	k_rx_err_hw_timeout	Hardware operation timed out.
= 0x204	k_rx_err_hw_error	Generic hardware error.
= 0x205	k_rx_err_gpio_conflict	GPIO pin conflict detected.
= 0x206	k_rx_err_gpio_invalid_port	Invalid GPIO port number.
= 0x207	k_rx_err_gpio_invalid_pin	Invalid GPIO pin number.
= 0x208	k_rx_err_out_of_range	Sensor measurement out of valid range.
= 0x209	k_rx_err_hw_unmapped	Hardware register block not mapped in memory.
= 0x301	k_rx_err_rtos_error	Generic RTOS error.
= 0x301	k_rx_err_threadx	ThreadX error (alias for k_rx_err_rtos_error).
= 0x302	k_rx_err_rtos_thread_create	ThreadX thread creation failed.
= 0x303	k_rx_err_rtos_semaphore	Semaphore operation failed.
= 0x304	k_rx_err_rtos_mutex	Mutex operation failed.
= 0x305	k_rx_err_rtos_queue	Queue operation failed.
= 0x306	k_rx_err_rtos_timer	Timer operation failed.
= 0x401	k_rx_err_comm_error	Generic communication error.
= 0x402	k_rx_err_spi_error	SPI communication error.
= 0x403	k_rx_err_uart_error	UART communication error.
= 0x404	k_rx_err_i2c_error	I2C communication error.
= 0x405	k_rx_err_crc_mismatch	CRC validation failed.

= 0x406	k_rx_err_protocol_error	Protocol error detected.
= 0x407	k_rx_err_nack	NACK (Not Acknowledged) received.
= 0x408	k_rx_err_conflict	Resource conflict in communication.
= 0x409	k_rx_err_retry_limit	Retransmission limit exceeded.
= 0x501	k_rx_err_validation_failed	Generic validation failed.
= 0x502	k_rx_err_checksum_mismatch	Checksum mismatch detected.
= 0x503	k_rx_err_range_check_failed	Range check failed.
= 0x504	k_rx_err_null_ptr	Null pointer detected.

—

rx_err_t

```
typedef int32_t rx_err_t
```

Error code type for RX72N firmware.

Signed 32-bit integer type used for all function return values that indicate success or failure. This type provides a consistent error handling interface across the entire firmware codebase.

—

rx_err_from_threadx

```
static rx_err_t rx_err_from_threadx(uint32_t tx_status)
```

Convert ThreadX status code to rx_err_t.

Converts ThreadX API return values (UINT status codes) to rx_err_t error codes. ThreadX uses 0 for success (TX_SUCCESS) and non-zero for various error conditions. This function performs a simple binary conversion: success -> k_rx_ok, any error -> k_rx_err_rtos_error.

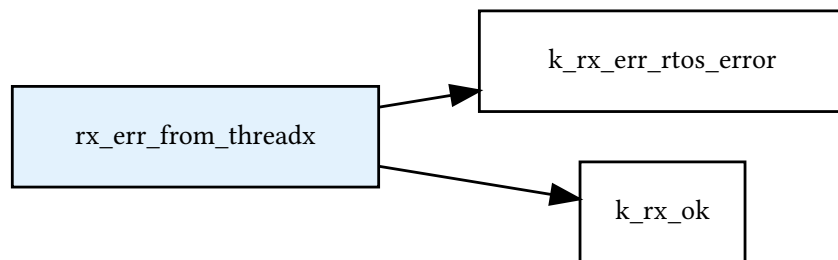


Figure 150: Call graph for rx_err_from_threadx

Defined in `libs/rx_core/inc/rx_err.h:1155`

—

rx_err_is_ok

```
static bool rx_err_is_ok(rx_err_t err)
```

Check if error code represents success.

Tests whether an `rx_err_t` value indicates successful operation (`k_rx_ok`). This is the preferred way to check for success in STAR firmware, providing explicit intent and type safety.

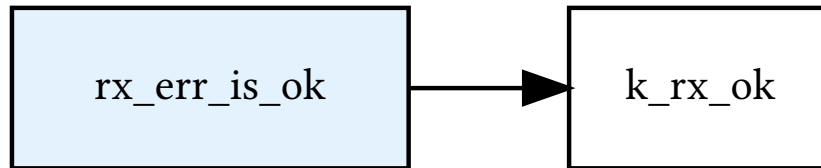


Figure 151: Call graph for `rx_err_is_ok`

Defined in `libs/rx_core/inc/rx_err.h:1256`

rx_err_is_error

```
static bool rx_err_is_error(rx_err_t err)
```

Check if error code represents failure.

Tests whether an `rx_err_t` value indicates a failure condition (any non-zero value). This is the logical inverse of `rx_err_is_ok()` and is used when error-checking logic is more naturally expressed as “if error occurred” rather than “if success”.

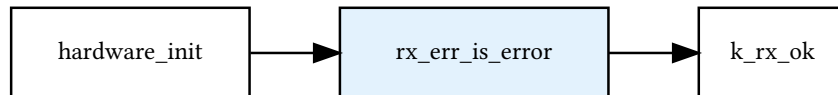


Figure 152: Call graph for `rx_err_is_error`

Defined in `libs/rx_core/inc/rx_err.h:1396`

rx_error_handler.h

Error Handler Concrete Implementation with Retry Logic and Exponential Backoff.

Production-grade error management system providing comprehensive error tracking, intelligent retry logic, and exponential backoff for graceful degradation. Implements the Dependency Inversion Principle through `rx_error_interface_t`, allowing modules to depend on abstract error handling without coupling to this concrete implementation.

Includes:

- `<stddef.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_error_interface.h"`
- `"rx_gpio_constants.h"`
- `"tx_api.h"`

error_handler_limits_t

Error Handler Configuration Constants and Array Size Limits.

Defines compile-time constants for the error handler including maximum number of trackable components and component name length. These constants determine static array sizes and must be chosen to balance functionality with memory usage.

Value	Name	Description
= 16	k_error_handler_max_components	Maximum number of components that can be tracked simultaneously. Limits component array size to 700 bytes. Sized for typical STAR system (motors + sensors + comm). Increase if more components needed (recompile required)
= 32	k_error_handler_component_name_max	Maximum component name length in characters including null terminator. Allows 31-character names like "UltrasonicSensor_Channel3". Longer names will be truncated

error_handler_init

```
rx_err_t error_handler_init(error_handler_t *handler, const
error_handler_config_t *config)
```

Initialize error handler with specified configuration.

Initializes the error handler structure, creates ThreadX mutex, zeroes all counters, and configures retry behavior. This function must be called before using the error handler or obtaining its interface. Initialization is idempotent-safe (returns error if already initialized).

Algorithm steps:

1. Validate handler and config pointers (NULL check)
2. Check if already initialized (prevent double-init)
3. Create ThreadX mutex for thread-safe access
4. Zero all error counters (total_error_count = 0)
5. Mark all component slots as unused (in_use = false)
6. Copy retry configuration from config parameter
7. Set initialized flag to true
8. Return success

Memory Operations:

- Calls tx_mutex_create() (ThreadX heap allocation for mutex control block)
- Zeroes components[] array (704 bytes)
- No user heap allocations (all static within error_handler_t)

After success, handler can be safely used by multiple threads

Configuration values copied exactly from config parameter

After init, use error_handler_get_interface() to obtain abstract interface

Initialize error handler with specified configuration.

Initializes an error handler instance with the specified configuration. Creates a ThreadX mutex for thread-safe operation and clears all internal state. Must be called before any other error handler operations.

Parameters:

Direction	Name	Description
inout	handler	Pointer to error_handler_t structure to initialize. Must be statically allocated or have lifetime exceeding all users. After success, structure is ready for use and interface extraction.
in	config	Pointer to configuration parameters. Values are copied into handler structure. Original can be freed after this call. NULL not allowed.
out	handler	Pointer to error handler instance to initialize Must be valid non-nullptr Will be completely overwritten (memset) Caller maintains ownership and lifetime
in	config	Configuration parameters max_retries: Maximum retries before limit (0 = unlimited) initial_backoff_ms: Starting backoff delay max_backoff_ms: Maximum backoff delay cap

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, handler initialized and ready for use
k_rx_err_null_ptr	handler or config parameter is nullptr
k_rx_err_invalid_state	handler already initialized (call deinit first)
k_rx_err_rtos_mutex	ThreadX mutex creation failed (out of resources)
k_rx_ok	Handler initialized successfully
k_rx_err_null_ptr	handler or config is nullptr
k_rx_err_invalid_arg	max_backoff_ms < initial_backoff_ms, or initial_backoff_ms > 0 with max_backoff_ms == 0
k_rx_err_rtos_mutex	ThreadX mutex creation failed

Pre: handler must point to valid error_handler_t structure

Pre: config must point to valid error_handler_config_t structure

Pre: handler must not be already initialized

Pre: ThreadX must be initialized (tx_kernel_enter() called)

Pre: handler points to allocated error_handler_t (not previously initialized)

Pre: config points to valid configuration with consistent values

Post: If success, handler->initialized == true

Post: If success, handler->mutex is valid and can be locked

Post: If success, handler->total_error_count == 0

Post: If success, all components.in_use == false

Post: handler->initialized == true

Post: handler->mutex created and ready

Post: All component slots cleared (in_use == false)

Post: total_error_count == 0

Note: Thread-safe - uses no shared state (operates on passed handler only)

Note: Must call error_handler_deinit() before destroying handler structure

Note: Configuration cannot be changed after init (requires deinit/re-init)

Note: Not thread-safe: do not call concurrently for same handler

Note: Call error_handler_deinit() before reinitializing

Warning: Do not call twice on same handler without deinit (returns error)

Warning: Handler must remain valid for lifetime of all users

Warning: Mutex creation can fail if ThreadX out of resources

Warning: Do not initialize same handler twice without deinit

Performance:: Execution time: 50 us @ 240 MHz (mutex creation dominates) Memory: No dynamic allocation (mutex uses ThreadX pool) Stack usage: < 64 bytes

Example (Basic Initialization):: #include "rx_error_handler.h"

```
staticerror_handler_ts_error_handler;

voidsystem_init(void)
{ consterror_handler_config_tconfig={ .max_retries=3, .initial_backoff_ms=10, .max_backoff_ms=5000 };

rx_err_tterr=error_handler_init(&s_error_handler,&config); if(err!=k_rx_ok)
{ uart_debug_puts("[FATAL]Errorhandlerinitfailedrn"); while(1); }

uart_debug_puts("[INFO]Errorhandlerinitializedrn"); }
```

Example (Error Handling):: rx_err_tterr=error_handler_init(&handler,&config);

```
switch(err){ casek_rx_ok: uart_debug_puts("[INFO]Handlerreadyrn"); break; casek_rx_err_null_ptr:
uart_debug_puts("[ERROR]nullptrrn"); break; casek_rx_err_invalid_state:
uart_debug_puts("[WARN]Alreadyinitializedrn"); break; casek_rx_err_rtos_mutex:
uart_debug_puts("[ERROR]Mutexcreationfailedrn"); break; default:
uart_debug_puts("[ERROR]Unknownerrornrn"); break; }
```

NASA Power of 10 Compliance:: Rule 3: [OK] No dynamic allocation (uses ThreadX pool for mutex) Rule 5: [OK] 4 preconditions, 4 postconditions Rule 7: [OK] Returns rx_err_t, caller must check

Algorithm: Validate handler and config pointers Validate backoff configuration consistency Clear all handler state (memset to 0) Copy configuration values Create ThreadX mutex Set initialized flag Log initialization success

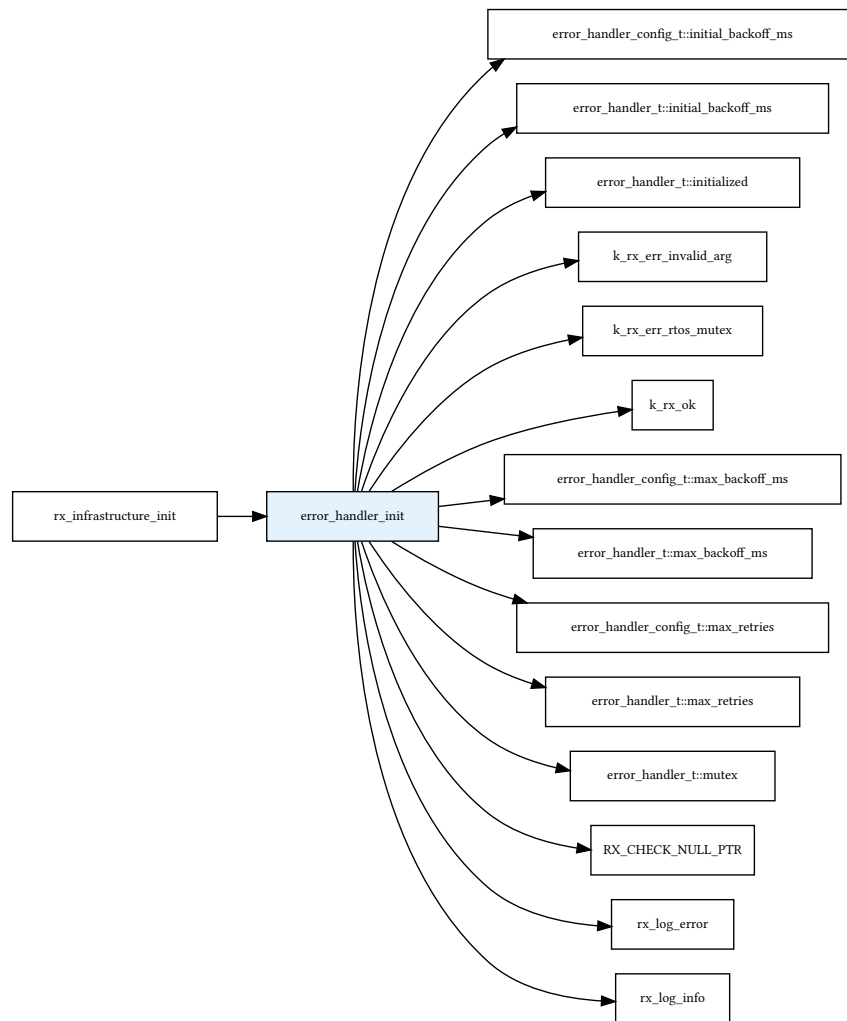
Example: staticerror_handler_tg_error_handler;

```
voidsystem_init(void)
{ error_handler_config_tconfig={ .max_retries=5, .initial_backoff_ms=100, .max_backoff_ms=5000 };

rx_err_tterr=error_handler_init(&g_error_handler,&config); if(err!=k_rx_ok){ while(1){ } }
```

Performance: 50-100 us (memset + mutex creation)

See also: error_handler_config_t Configuration structure definition,
error_handler_get_interface() Get abstract interface after init,
error_handler_deinit() Cleanup before destroying handler, rx_error_interface_t
Abstract interface for Dependency Inversion, error_handler_deinit() Cleanup handler,
error_handler_get_interface() Get interface for dependency injection

Figure 153: Call graph for `error_handler_init`

Defined in `libs/rx\core/inc/rx_error_handler.h:878`

Since Version 1.0.0

—

error_handler_get_interface

```
rx_err_t error_handler_get_interface(rx_error_interface_t *iface, error_handler_t
*handler)
```

Get abstract interface from concrete error handler (Dependency Inversion).

Extracts an `rx_error_interface_t` from the concrete `error_handler_t` implementation. This function is the key to Dependency Inversion: high-level modules depend on the abstract interface, not the

concrete implementation. The returned interface contains function pointers and context for polymorphic error handling.

Algorithm steps:

1. Validate iface and handler pointers (NULL check)
2. Check handler is initialized (initialized flag)
3. Fill iface->ctx with handler pointer (context for callbacks)
4. Assign function pointers to concrete implementations:

iface->report_error -> internal report function iface->is_retry_limit_reached -> internal retry check

iface->get_backoff_delay -> internal backoff calculator iface->reset_component_state -> internal reset function

5. Return success

Dependency Inversion Pattern:

Benefits:

- High-level code doesn't know about error_handler_t
- Can swap implementations without changing clients
- Enables unit testing with mock error handlers
- Clear separation of interface and implementation

iface lifetime must not exceed handler lifetime

iface remains valid as long as handler is initialized

Interface must not outlive handler structure

Get abstract interface from concrete error handler (Dependency Inversion).

Populates an rx_error_interface_t structure with function pointers to the handler's implementation functions. This enables dependency injection following the Dependency Inversion Principle (DIP).

Parameters:

Direction	Name	Description
out	iface	Pointer to rx_error_interface_t to fill with function pointers and context. After success, contains fully initialized interface ready for use.
inout	handler	Pointer to initialized error_handler_t concrete implementation. Must be initialized via error_handler_init(). Serves as context (iface->ctx).
out	iface	Pointer to interface structure to populate All fields will be overwritten Caller maintains ownership of the structure
in	handler	Pointer to initialized error handler instance Must be initialized via error_handler_init() Must remain valid for lifetime of interface usage

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, iface filled with valid function pointers and context
k_rx_err_null_ptr	iface or handler parameter is nullptr
k_rx_err_invalid_state	handler not initialized (call error_handler_init first)
k_rx_ok	Interface populated successfully

k_rx_err_null_ptr	iface or handler is nullptr
k_rx_err_invalid_state	handler not initialized

Pre: iface must point to valid rx_error_interface_t structure

Pre: handler must point to valid error_handler_t structure

Pre: handler must be initialized (handler->initialized == true)

Pre: iface points to allocated rx_error_interface_t

Pre: handler initialized via error_handler_init()

Post: If success, iface->ctx == handler (context pointer)

Post: If success, all iface function pointers are non-NULL and valid

Post: If success, iface can be used for all error operations

Post: iface->ctx == handler

Post: All function pointers set to implementation functions

Note: Thread-safe - uses no shared state

Note: Interface is lightweight (just function pointers and one context pointer)

Note: Can call multiple times to get multiple interface copies (all point to same handler)

Note: Handler must remain valid while interface is in use

Note: Interface structure is a shallow copy (pointers only)

Warning: iface becomes invalid if handler is deinitialized

Warning: Do not use iface after error_handler_deinit(handler) called

Performance:: Execution time: 2 us @ 240 MHz (simple pointer assignments) Memory: No allocation, just fills existing structure Stack usage: < 16 bytes

Example (Dependency Inversion in Action):: static error_handler_ts_handler;
 error_handler_init(&s_handler,&config);

```
rx_error_interface_t error_iface; rx_err_t err=error_handler_get_interface(&error_iface,&s_handler);
if(err!=k_rx_ok){ uart_debug_puts("[ERROR]Failed to get error interface\n"); return err; }

motor_controller_init(&error_iface); spi_driver_init(&error_iface);
sensor_manager_init(&error_iface);
```

Example (Using Interface):: void spi_transfer(rx_error_interface_t* error_if, uint8_t* data, size_t len)
 { rx_err_t err=spi_hw_transfer(data,len);

```
if(err!=k_rx_ok){ error_if->report_error(error_if->ctx,err,"SPI","Transfer failed");

if(!error_if->is_retry_limit_reached(error_if->ctx,"SPI")){ uint32_t delay_ms=error_if->
  get_backoff_delay(error_if->ctx,"SPI"); tx_thread_sleep(delay_ms/10); } }
```

SOLID Principles:: Dependency Inversion (D): This is the DIP implementation function. High-level modules depend on rx_error_interface_t (abstraction). Low-level error_handler_t implements the abstraction. This function bridges concrete to abstract.

Algorithm: Validate iface and handler pointers. Verify handler is initialized. Populate interface structure: ctx = handler (opaque context). Function pointers to impl_* functions.

Interface Binding Diagram: digraph interface_binding { rankdir=LR; node [shape=record]; iface [label="rx_error_interface_t|ctx|report_error|get_error_count|get_component_error_count|clear_errors|is_retry_limit_reached|reset_retry_counter|get_backoff_delay"]; handler [label="error_handler_t|mutex|components[]|total_error_count|config..."]; impl [label="Implementation|impl_report_error()|impl_get_error_count()|impl_get_component_error_count()|impl_clear_errors()|impl_is_retry_limit_reached()|impl_reset_retry_counter()|impl_get_backoff_delay()"]; iface:ctx -> handler [label="points to"]; iface:report_error -> impl:impl_report_error; iface:get_error_count -> impl:impl_get_error_count; }

Example: static error_handler_t g_error_handler; static rx_error_interface_t g_error_iface;

```
void system_init(void){ error_handler_config_t config={...};
error_handler_init(&g_error_handler,&config);

error_handler_get_interface(&g_error_iface,&g_error_handler);

motor_init(&motor,&g_error_iface); comm_init(&comm,&g_error_iface); }

void motor_report_fault(motor_t* motor){ motor->error_iface->report_error( motor->error_iface->
  ctx, k_rx_err_hw_fault, "MOTOR", "Overcurrent detected" ); }
```

Performance: 1 us (pointer assignments only)

Example:

```
High-LevelModule(MotorController)
vdepends on v
rx_error_interface_t (Abstract) <- THIS FUNCTION PRODUCES THIS
^implements^
error_handler_t (Concrete)
```

See also: error_handler_init() Must call first to initialize handler,
 rx_error_interface_t Abstract interface structure definition, error_handler_t Concrete implementation structure, error_handler_init() Initialize handler first,
 rx_error_interface_t Abstract interface definition

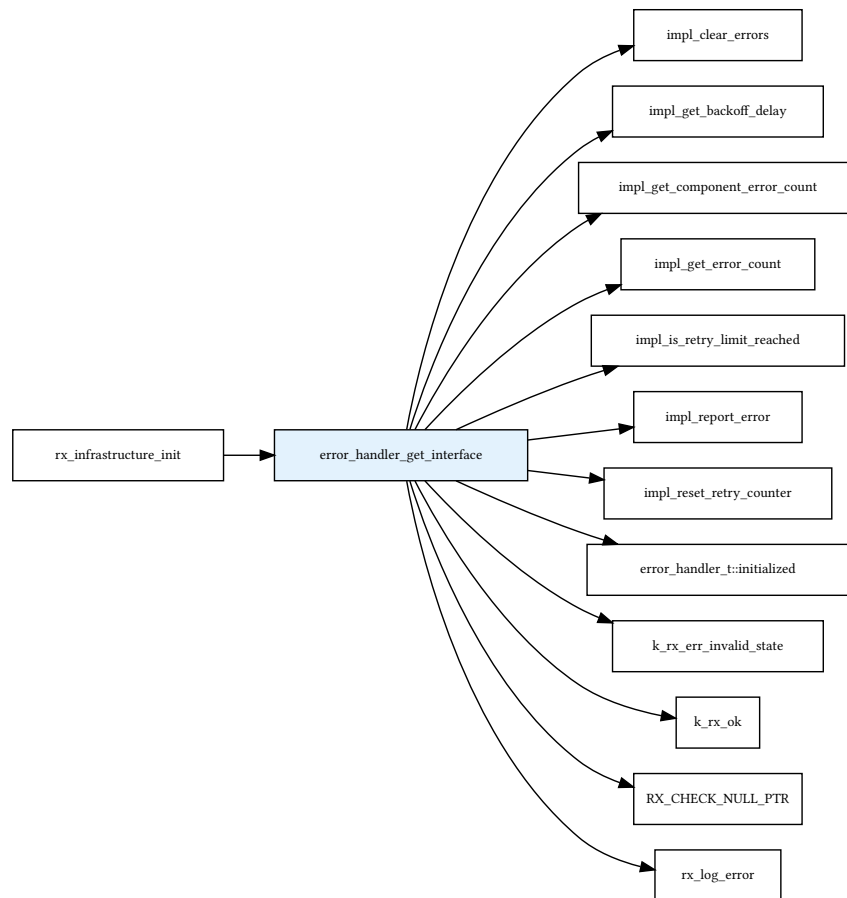


Figure 154: Call graph for error_handler_get_interface

_Defined in libs/rx_core/inc/rx_error_handler.h:1005_

Since Version 1.0.0

—

error_handler_deinit

```
rx_err_t error_handler_deinit(error_handler_t *handler)
```

Deinitialize error handler and cleanup resources.

Cleans up error handler resources, deletes ThreadX mutex, and marks handler as uninitialized. Call this function before destroying the handler structure or on system shutdown. After calling, the handler cannot be used until re-initialized.

Algorithm steps:

1. Validate handler pointer (NULL check)
2. Check if initialized (skip if not initialized)
3. Delete ThreadX mutex (tx_mutex_delete)

4. Zero all counters and arrays (security/cleanup)
5. Set initialized flag to false
6. Return success

Cleanup Operations:

- Deletes TX_MUTEX (frees ThreadX resources)
- Zeros components[] array (704 bytes)
- Zeros total_error_count
- No heap deallocations needed (static structure)

Idempotent - safe to call on already-deinitialized handler

After deinit, handler can be re-initialized via error_handler_init()

Always call before destroying handler or on system shutdown

Deinitialize error handler and cleanup resources.

Releases resources held by the error handler, including the ThreadX mutex. Safe to call on already-deinitialized handler (idempotent).

Parameters:

Direction	Name	Description
inout	handler	Pointer to error_handler_t to deinitialize. After success, handler->initialized == false and structure cannot be used until re-initialized.
inout	handler	Pointer to error handler instance May be initialized or already deinitialized Mutex will be deleted

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, handler deinitialized and resources freed
k_rx_err_null_ptr	handler parameter is nullptr
k_rx_ok	Handler deinitialized (or was already deinitialized)
k_rx_err_null_ptr	handler is nullptr

Pre: handler must point to valid error_handler_t structure

Pre: Caller must ensure no other threads are using handler

Pre: handler points to valid error_handler_t (initialized or not)

Post: If success, handler->initialized == false

Post: If success, handler->mutex is deleted (invalid)

Post: If success, all counters and arrays zeroed

Post: Handler structure still exists (not freed) but unusable

Post: handler->initialized == false

Post: handler->mutex deleted (if was initialized)

Note: Thread-safe for the actual deinit, but caller must ensure exclusivity

Note: Calling on uninitialized handler is safe (returns success)

Note: Structure memory is NOT freed (caller responsibility if dynamically allocated)

Note: Idempotent: safe to call multiple times

Note: Not thread-safe: ensure no other threads using handler

Warning: All outstanding rx_error_interface_t pointers become invalid

Warning: Caller must ensure no threads are using handler before calling

Warning: Mutex deletion can fail if still locked by another thread

Warning: Invalidates any rx_error_interface_t pointing to this handler

Warning: Do not use interface after deinit

Performance:: Execution time: 30 us @ 240 MHz (mutex deletion dominates) Memory: Frees ThreadX mutex resources Stack usage: < 32 bytes

Example (System Shutdown):: void system_shutdown(void)
 { uart_debug_puts("[INFO]System shutdown initiated\r\n");
 rx_err_t err = error_handler_deinit(&s_error_handler); if(err != k_rx_ok)
 { uart_debug_puts("[WARN]Error handler deinit failed\r\n"); }
 }

Example (Re-initialization):: error_handler_deinit(&handler);
 const error_handler_config_t new_config = { .max_retries = 10, .initial_backoff_ms = 50, .max_backoff_ms = 10000 };
 error_handler_init(&handler, &new_config);

NASA Power of 10 Compliance:: Rule 3: [OK] No dynamic allocation, only resource cleanup Rule 7: [OK] Returns rx_err_t, caller should check

Algorithm: Validate handler pointer If not initialized, return k_rx_ok (idempotent) Delete ThreadX mutex Set initialized = false Log deinitialization

Example: void system_shutdown(void){ motor_deinit(&motor); comm_deinit(&comm);
error_handler_deinit(&g_error_handler); }

Performance: 10-50 us (mutex deletion)

See also: error_handler_init() Initialize handler (can call after deinit),
error_handler_get_interface() Interfaces become invalid after deinit,
error_handler_init() Reinitialize after deinit if needed

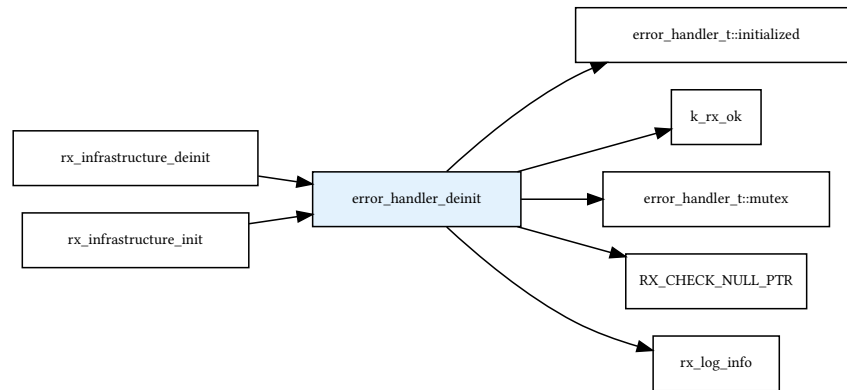


Figure 155: Call graph for error_handler_deinit

_Defined in libs/rx_core/inc/rx_error_handler.h:1100_

Since Version 1.0.0

—

rx_error_interface.h

Abstract Error Handler Interface (Dependency Inversion Principle - SOLID).

Pure abstract interface defining the contract for error handling in the STAR RX72N firmware. This is the “D” in SOLID - Dependency Inversion Principle in action. High-level modules depend on this abstraction, not concrete implementations.

Includes:

- <stdbool.h>
- <stddef.h>
- <stdint.h>
- "rx_err.h"

rx_error_report_fn

```
typedef rx_err_t(* rx_error_report_fn) (void *ctx, rx_err_t err, const char
*component, const char *message)
```

Function Pointer Type for Error Reporting (Core Interface Function).

Defines the signature for error reporting functions in concrete implementations. This is the PRIMARY function in the error interface - all implementations MUST provide this function. Called when a component detects an error that needs to be tracked, logged, or handled.

—

rx_error_get_count_fn

```
typedef uint32_t(* rx_error_get_count_fn) (void *ctx)
```

Function Pointer for Getting Total Error Count (REQUIRED Interface Function).

Returns the total number of errors reported across ALL components since initialization or last clear. Useful for system health monitoring and diagnostics.

REQUIRED: All implementations must provide this function.

Note: Thread-safe implementations recommended

Note: Returns 0 if ctx is nullptr (defensive programming)] **See also:** rx_error_get_component_count_fn For per-component error counts

—

rx_error_get_component_count_fn

```
typedef uint32_t(* rx_error_get_component_count_fn) (void *ctx, const char *component)
```

Function Pointer for Getting Component-Specific Error Count (OPTIONAL).

Returns the number of errors reported for a specific component. Enables per-component diagnostics and targeted debugging.

OPTIONAL: Can be NULL if implementation doesn't track per-component stats.

Note: Returns 0 if component not registered

Note: Case-sensitive component name matching (typically)] **See also:** rx_error_get_count_fn For total error count across all components

—

rx_error_clear_fn

```
typedef rx_err_t(* rx_error_clear_fn) (void *ctx)
```

Function Pointer for Clearing All Error Counters (REQUIRED Interface Function).

Resets all error counters to zero and clears retry state. Used for:

- Starting fresh after diagnostics
- Resetting after error recovery
- Clearing transient errors after system stabilization

REQUIRED: All implementations must provide this function.

Note: Thread-safe implementations recommended

Warning: Clears ALL error history - use with caution] **See also:** `rx_error_reset_retry_fn`
For resetting retry counters without clearing error counts

—

rx_error_is_retry_limit_reached_fn

`typedef bool(* rx_error_is_retry_limit_reached_fn) (void *ctx, const char *component)`

Function Pointer for Checking Retry Limit Status (OPTIONAL).

Determines if a component has exceeded its maximum retry attempts. Used in retry logic to decide whether to:

- Continue retrying (limit not reached)
- Give up and propagate error (limit reached)

OPTIONAL: Can be NULL if implementation doesn't support retry tracking. Clients should check if function pointer is nullptr before calling.

Note: Returns false if component not found (allows first retry)

Note: Returns false if ctx is nullptr (defensive programming)] **See also:**

`rx_error_get_backoff_delay_fn` For calculating retry delay, `rx_error_reset_retry_fn`
For resetting retry counter

—

rx_error_reset_retry_fn

`typedef rx_err_t(* rx_error_reset_retry_fn) (void *ctx, const char *component)`

Function Pointer for Resetting Component Retry Counter (OPTIONAL).

Resets the retry counter for a specific component back to zero. Called after:

- Successful operation (clear retry state)
- Manual intervention (give component another chance)
- Configuration change (fresh start)

OPTIONAL: Can be NULL if implementation doesn't support retry tracking.

Note: Does NOT clear error count, only retry counter

Note: Thread-safe implementations recommended] **See also:**

`rx_error_is_retry_limit_reached_fn` For checking retry limit, `rx_error_clear_fn` For
clearing all counters including errors

—

rx_error_get_backoff_delay_fn

`typedef uint32_t(* rx_error_get_backoff_delay_fn) (void *ctx, const char *component)`

Function Pointer for Calculating Exponential Backoff Delay (OPTIONAL).

Calculates the recommended delay before retrying an operation, typically using exponential backoff algorithm:

OPTIONAL: Can be NULL if implementation doesn't support backoff calculation.

Example Backoff Progression (initial=10ms, max=5s):

- Retry 0: 10ms
- Retry 1: 20ms
- Retry 2: 40ms
- Retry 3: 80ms
- Retry 4: 160ms
- ...
- Retry 9: 5120ms -> capped at 5000ms

Note: Returns 0 if ctx is nullptr or component not found

Note: Delay increases exponentially with retry count

Note: Implementation may cap delay at configured maximum] **See also:**
 rx_error_is_retry_limit_reached_fn For checking if should retry,
 rx_error_reset_retry_fn For resetting retry counter on success

Example:

```
delay=initial_backoff*(2^retry_count)
delay=min(delay,max_backoff)//Capatmaximum
```

rx_error_interface_validate

```
static rx_err_t rx_error_interface_validate(const rx_error_interface_t *iface)
```

Validate that error interface is properly initialized with required functions.

Performs validation checks on an error interface before use. Ensures that all REQUIRED function pointers are non-NULL. Optional function pointers may be NULL. Call this function after obtaining an interface from a concrete implementation to verify correctness before injecting into dependent modules.

Validation Checks:

1. Interface pointer is non-NULL
2. report_error function pointer is non-NULL (REQUIRED)
3. get_error_count function pointer is non-NULL (REQUIRED)
4. clear_errors function pointer is non-NULL (REQUIRED)

Not Validated (optional functions can be NULL):

- get_component_error_count
- is_retry_limit_reached
- reset_retry_counter
- get_backoff_delay
- ctx (can be NULL for stateless implementations)

Performance: Very fast (10 CPU cycles), just NULL checks.

Always validate after getting interface from implementation

Parameters:

Direction	Name	Description
in	iface	Pointer to interface structure to validate. Can be NULL (returns error).

Returns: rx_err_t Validation result

Value	Description
k_rx_ok	Interface is valid and ready to use
k_rx_err_null_ptr	iface parameter is nullptr
k_rx_err_invalid_state	One or more REQUIRED function pointers are nullptr

Note: This is a static inline function (no function call overhead)

Note: Only checks REQUIRED functions, optional functions may be nullptr

Note: Does not validate function pointer targets (assumes correct binding)

Warning: Do not use interface if validation fails

Usage Example (Recommended Pattern):: rx_error_interface_t error_iface;
error_handler_get_interface(&error_iface, &handler);

```
rx_err_t err = rx_error_interface_validate(&error_iface); if (err != k_rx_ok)
{ uart_debug_puts("[ERROR] Invalid error interface\n"); return err; }

spi_driver_init(&spi, &error_iface);
```

Usage Example (Error Handling):: rx_err_t err = rx_error_interface_validate(&iface);

```
switch (err) { case k_rx_ok: uart_debug_puts("[INFO] Interface valid\n"); break; case k_rx_err_null_ptr:
uart_debug_puts("[ERROR] NULL interface pointer\n"); break; case k_rx_err_invalid_state:
uart_debug_puts("[ERROR] Missing required function pointers\n"); break; }
```

Example (Defensive Programming)::

```
void module_init(module_t* mod, rx_error_interface_t* error_iface)
{ rx_err_t err = rx_error_interface_validate(error_iface); if (err != k_rx_ok) { return err; }

mod->error_iface = error_iface; return k_rx_ok; }
```

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (sequential checks) Rule 4:
[OK] Function < 10 lines (very small) Rule 5: [OK] Precondition checks (NULL validation)

See also: rx_error_interface Structure being validated, error_handler_get_interface()
Example interface provider

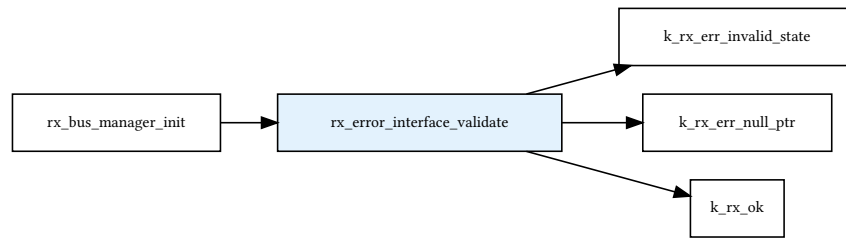


Figure 156: Call graph for rx_error_interface_validate

_Defined in libs/rx\core\inc\rx_error_interface.h:904_

Since Version 1.0.0

—

rx_exception.h

RX72N CPU Exception Handling.

Provides exception handling for RX72N CPU exceptions as defined in the hardware manual Chapter 14. This module captures debug information when exceptions occur and provides handlers for:

- Undefined instruction exception (EXTB + 0x5C)
- Privileged instruction exception (EXTB + 0x50)
- Access exception (EXTB + 0x54)
- Address exception (EXTB + 0x60)
- Single-precision floating-point exception (EXTB + 0x64)
- Non-maskable interrupt (EXTB + 0x78)

The RXv3 CPU saves PC and PSW to the stack (ISP) on exception entry, shifts to supervisor mode, and disables interrupts (I=0, U=0, PM=0).

Copyright (c) 2026 STAR Project

Includes:

- <stdint.h>

rx_exception_type_t

CPU exception types supported by RXv3 core.

Defines all exception types from Chapter 14 of the RX72N hardware manual. Exception priority from highest to lowest:

1. Reset (highest)
2. Non-maskable interrupt
3. Interrupt
4. Instruction access exception
5. Undefined/Privileged instruction exception
6. Unconditional trap
7. Address exception
8. Operand access exception
9. Single-precision floating-point exception (lowest)

Value	Name	Description
-------	------	-------------

= 0	k_rx_exception_undefined_instruction	Undefined instruction exception.
= 1	k_rx_exception_privileged_instruction	Privileged instruction exception.
= 2	k_rx_exception_access	Access exception (instruction or operand).
= 3	k_rx_exception_address	Address exception.
= 4	k_rx_exception_floating_point	Single-precision floating-point exception.
= 5	k_rx_exception_nmi	Non-maskable interrupt (NMI).
= 6	k_rx_exception_type_count	Total count of exception types.

Since Version 1.0.0

—

rx_exception_vector_offset_t

Exception vector table offsets from EXTB base.

The RX72N uses EXTB register to specify the exception vector table base. Default EXTB value after reset is 0xFFFF_FF80. Each offset specifies where the exception handler address is stored.

Value	Name	Description
= 0x50	k_rx_exc_offset_privileged	Privileged instruction exception vector offset.
= 0x54	k_rx_exc_offset_access	Access exception vector offset.
= 0x5C	k_rx_exc_offset_undefined	Undefined instruction exception vector offset.
= 0x60	k_rx_exc_offset_address	Address exception vector offset.
= 0x64	k_rx_exc_offset_fpu	Floating-point exception vector offset.
= 0x78	k_rx_exc_offset_nmi	Non-maskable interrupt vector offset.
= 0x7C	k_rx_exc_offset_reset	Reset vector offset.

Since Version 1.0.0

—

rx_exception_init

```
void rx_exception_init(void)
```

Initialize exception handling module.

Initializes exception statistics and optionally installs custom handlers. Should be called early in system startup, after basic hardware init.

Pre: None

Post: Exception statistics zeroed

Post: Default handlers installed (if not using assembly stubs)

Note: Thread-safe: No, call only once during startup

Example:: void main(void){ rx_exception_init(); }

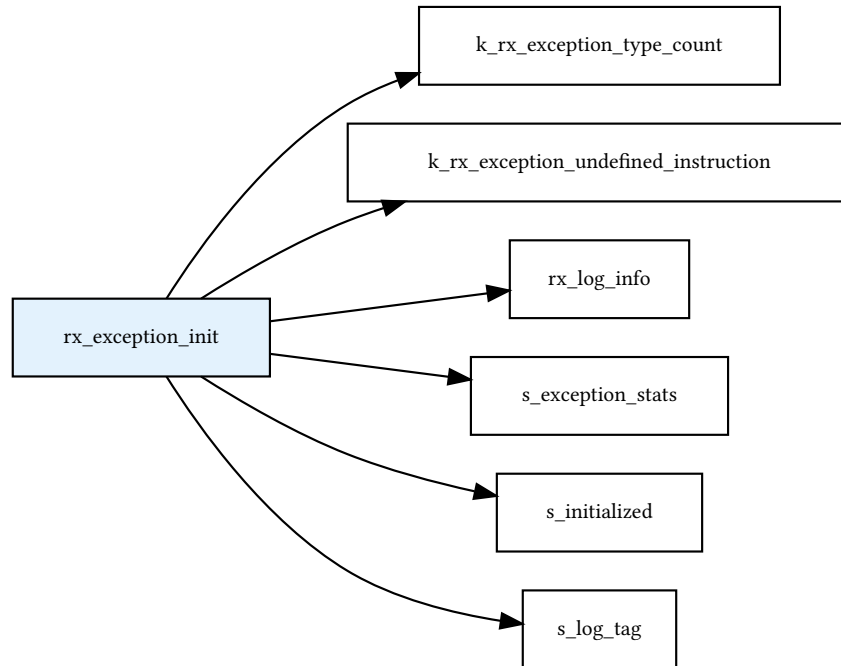


Figure 157: Call graph for `rx_exception_init`

Defined in `libs/rx_core/inc/rx_exception.h:332`

Since Version 1.0.0

—

rx_exception_get_stats

```
const rx_exception_stats_t * rx_exception_get_stats(void)
```

Get exception statistics.

Returns pointer to read-only exception statistics structure. Statistics are updated by exception handlers.

Returns: Pointer to exception statistics (never NULL)

Pre: `rx_exception_init()` must have been called

Post: No side effects

Note: Thread-safe: Read access only

Example:: `constrx_exception_stats_t*stats=rx_exception_get_stats(); if(stats->count[k_rx_exception_nmi]>0){ }`

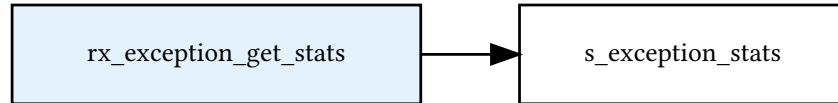


Figure 158: Call graph for rx_exception_get_stats

Defined in `libs/rx_core/inc/rx_exception.h:358`

Since Version 1.0.0

—

rx_exception_get_name

```
const char * rx_exception_get_name(rx_exception_type_t type)
```

Get human-readable name for exception type.

Returns a string describing the exception type for logging.

Parameters:

Direction	Name	Description
in	type	Exception type enumeration value

Returns: Pointer to static string (never NULL)

Value	Description
Unknown	If type is out of range

Pre: None

Post: No side effects

Note: Thread-safe: Yes (returns pointer to const data)

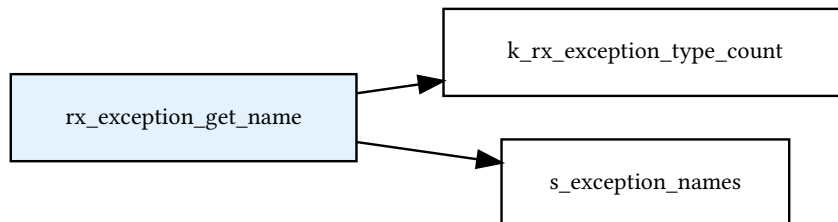


Figure 159: Call graph for rx_exception_get_name

Defined in `libs/rx_core/inc/rx_exception.h:378`

Since Version 1.0.0

—

rx_exc_undefined_instruction_handler

```
void rx_exc_undefined_instruction_handler(void)
```

Undefined instruction exception handler (ASM entry point).

Called from vector at EXTB + 0x5C. Saves context and calls C handler.

Warning: Do not call directly from C code] *Defined in libs/rx_core/inc/rx_exception.h:389*

—

rx_exc_privileged_instruction_handler

```
void rx_exc_privileged_instruction_handler(void)
```

Privileged instruction exception handler (ASM entry point).

Called from vector at EXTB + 0x50. Saves context and calls C handler.

Warning: Do not call directly from C code] *Defined in libs/rx_core/inc/rx_exception.h:396*

—

rx_exc_access_handler

```
void rx_exc_access_handler(void)
```

Access exception handler (ASM entry point).

Called from vector at EXTB + 0x54. Saves context and calls C handler.

Warning: Do not call directly from C code] *Defined in libs/rx_core/inc/rx_exception.h:403*

—

rx_exc_address_handler

```
void rx_exc_address_handler(void)
```

Address exception handler (ASM entry point).

Called from vector at EXTB + 0x60. Saves context and calls C handler.

Warning: Do not call directly from C code] *Defined in libs/rx_core/inc/rx_exception.h:410*

—

rx_exc_floating_point_handler

```
void rx_exc_floating_point_handler(void)
```

Floating-point exception handler (ASM entry point).

Called from vector at EXTB + 0x64. Saves context and calls C handler.

Warning: Do not call directly from C code] *Defined in libs/rx_core/inc/rx_exception.h:417*

—

rx_exc_nmi_handler

```
void rx_exc_nmi_handler(void)
```

Non-maskable interrupt handler (ASM entry point).

Called from vector at EXTB + 0x78. NEVER returns.

Warning: Do not call directly from C code

Warning: MUST NOT return - system must reset or halt] *Defined in libs/rx_core/inc/rx_exception.h:425*

—

rx_gpio_constants.h

GPIO and System Constants for RX72N Microcontroller.

This header provides typed enumeration constants for GPIO configuration, error handling defaults, and system register protection (PRCR) values used throughout the RX72N firmware. All constants are implemented as C23 typed enums to ensure type safety, predictable memory layout, and eliminate magic numbers from the codebase.

Includes:

- <stdint.h>

rx_gpio_constants_t

GPIO port and pin constants for RX72N microcontroller.

The Renesas RX72N uses a unique dual-numbering scheme for GPIO ports:

- Decimal ports: PORT0 through PORT9 (addresses 0x00-0x09)
- Hexadecimal ports: PORTA through PORTG (addresses 0x0A-0x10)

Each port contains up to 8 pins numbered 0-7. Not all ports implement all 8 pins; refer to the RX72N hardware manual for specific port capabilities.

These constants provide:

- Pin and port validation bounds
- Invalid sentinel values for error detection
- Addressing offsets for port array indexing

Value	Name	Description
= 8	k_pins_per_port	Number of pins per GPIO port.
= 9	k_max_decimal_port	Maximum decimal port number.
= 0xA	k_hex_port_start	First hexadecimal port address.
= 0x10	k_hex_port_end	Last hexadecimal port address.
= 10	k_hex_port_offset	Offset for hex port array mapping.
= 0xFFFF	k_invalid_pin_index	Sentinel value indicating invalid pin array index.
= 0xFF	k_invalid_port	Sentinel value indicating invalid port number.

rx_error_handler_defaults_t

Error handler default configuration values.

Default parameters for exponential backoff retry mechanism used throughout the RX72N firmware. These values provide a reasonable starting point for transient error recovery (I2C bus errors, SPI timeouts, sensor read failures).

Value	Name	Description
= 3	k_error_handler_default_max_retries	Default maximum retry count.
= 100	k_error_handler_default_initial_backoff_ms	Default initial backoff delay in milliseconds.
= 5000	k_error_handler_default_max_backoff_ms	Default maximum backoff delay in milliseconds.

rx_prcr_constants_t

PRCR (Protect Register) constants for RX72N system register protection.

The PRCR (Protect Register) is a critical safety feature of the RX72N that prevents accidental modification of system-critical registers. This protection mechanism requires a two-step unlock sequence:

1. Write unlock key (0xA5) to upper byte of PRCR
2. Set protection control bit (PRC0, PRC1, PRC3) in lower byte

Value	Name	Description
= 0xA5	k_prcr_key	PRCR unlock key value.
= 8	k_prcr_key_shift	Bit shift to position PRCR key into upper byte.
= 0x00	k_prcr_lock_all	Lock all protected registers (default safe state).
= 0x02	k_prcr_lock_prc1	Unlock PRC1 bit (module stop control registers).

rx_infrastructure.h

Global Infrastructure Initialization and Service Locator.

Includes:

- "rx_err.h"
- "rx_error_interface.h"
- "rx_pin_interface.h"

RX_REPORT_ERROR

```
#define RX_REPORT_ERROR do
{
    rx_error_interface_t* iface_ = rx_infrastructure_get_error_interface();
    \
    if (iface_ != nullptr)
    {
        iface_->report_error(iface_>ctx, err, component, message);
        \
    }
    \
} while (0)
```

Report error through global error handler (convenience macro).

—

RX_RESERVE_PIN

```
#define RX_RESERVE_PIN
({
    rx_err_t          err_ = k_rx_err_invalid_state;
    \
    rx_pin_interface_t* iface_ = rx_infrastructure_get_pin_interface();
    \
    if (iface_ != nullptr)
    {
        err_ = iface_->reserve_pin(iface_>ctx, port, pin, function);
        \
    }
    \
    err_;
    \
})
```

Reserve GPIO pin through global pin validator (convenience macro with return value).

—

RX_RELEASE_PIN

```
#define RX_RELEASE_PIN
({
    rx_err_t          err_ = k_rx_err_invalid_state;
    \
    rx_pin_interface_t* iface_ = rx_infrastructure_get_pin_interface();
    \
    if (iface_ != nullptr)
    {
        err_ = iface_->release_pin(iface_>ctx, port, pin);
        \
    }
    \
    err_;
    \
})
```

```
\  
})
```

Release reserved GPIO pin through global pin validator (convenience macro with return value).

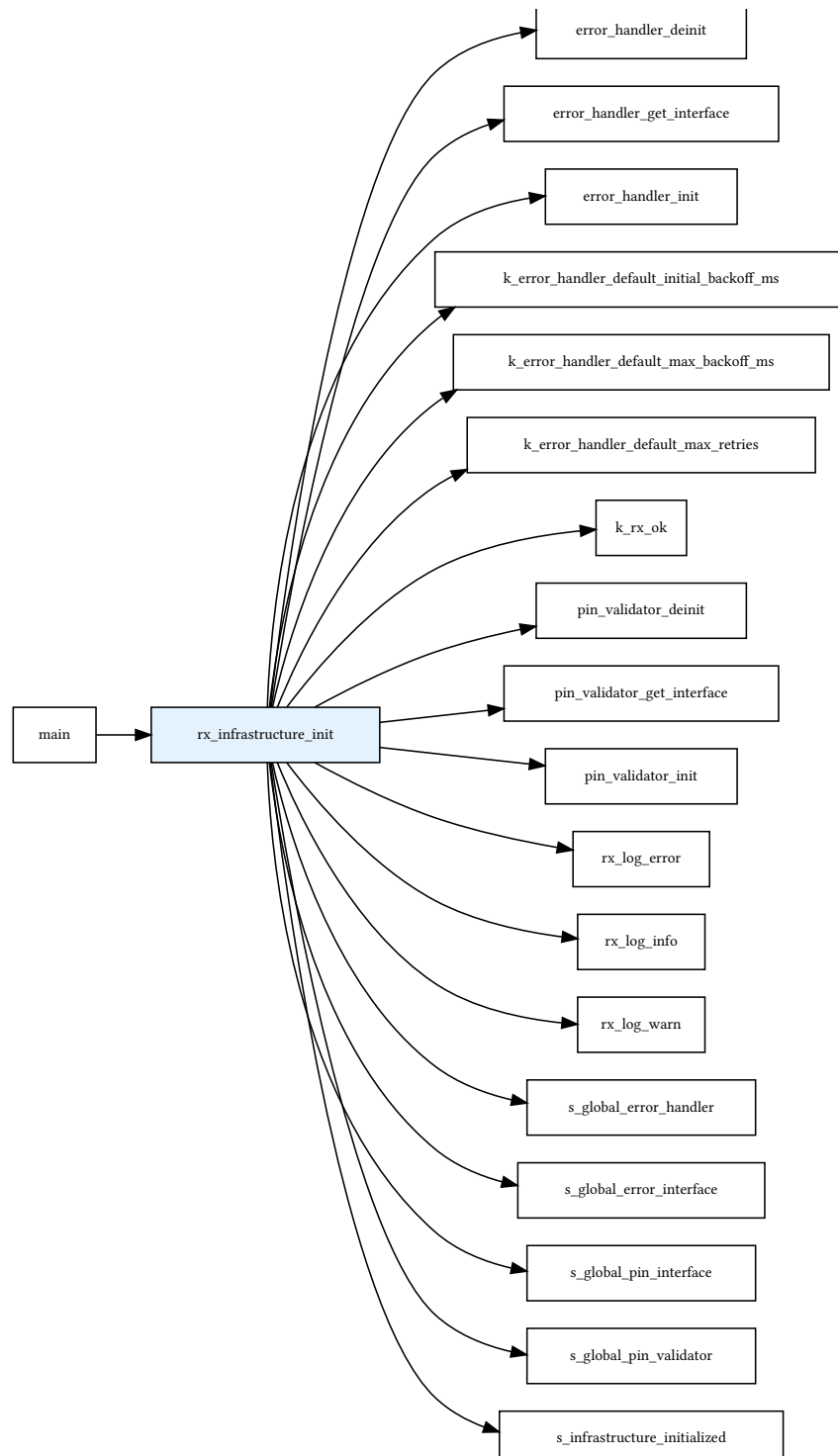
—

rx_infrastructure_init

```
rx_err_t rx_infrastructure_init(void)
```

Initialize global infrastructure (MUST BE FIRST CALL IN main()).

Performs one-time initialization of all global infrastructure services. This function MUST be the first thing called in `main()`, before any other module initialization and before starting ThreadX. Failure to initialize infrastructure will cause nullptr pointer dereferences when other modules attempt to use `RX_REPORT_ERROR()` or `RX_RESERVE_PIN()`.

Figure 160: Call graph for `rx_infrastructure_init`

Defined in `libs/rx_core/inc/rx_infrastructure.h:500`

—

rx_infrastructure_deinit

```
rx_err_t rx_infrastructure_deinit(void)
```

Deinitialize global infrastructure (graceful shutdown only).

Performs graceful shutdown of all infrastructure services. In embedded systems, this function is RARELY CALLED because the system typically runs indefinitely until power loss or reset. However, it's provided for:

- Unit testing (setup/teardown)
- Firmware update procedures (safe state before flash)
- Power management (deep sleep entry)
- Development/debugging (clean restarts)

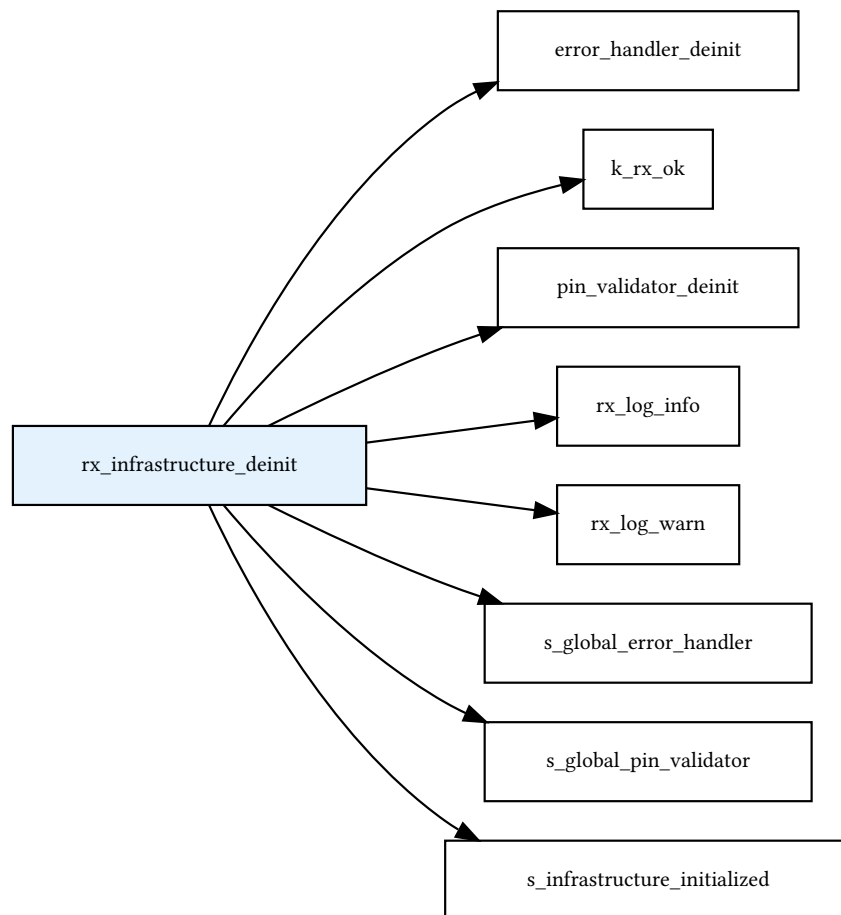


Figure 161: Call graph for `rx_infrastructure_deinit`

Defined in *libs/rx_core/inc/rx_infrastructure.h*:759

—

rx_infrastructure_get_error_interface

```
rx_error_interface_t * rx_infrastructure_get_error_interface(void)
```

Get global error handler interface (thread-safe accessor).

Returns a pointer to the global `rx_error_interface_t` that provides access to the error handling subsystem. This is the primary way for modules to report errors, check error counts, and implement retry logic with exponential backoff.

The returned pointer is valid for the entire program lifetime after successful initialization via `rx_infrastructure_init()`. Modules can cache this pointer for performance-critical code paths.

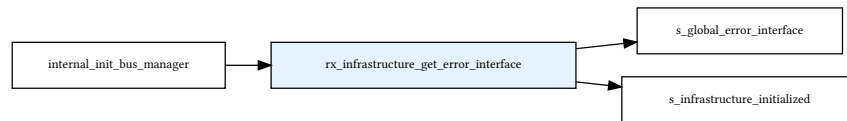


Figure 162: Call graph for `rx_infrastructure_get_error_interface`

Defined in *libs/rx_core/inc/rx_infrastructure.h*:977

—

rx_infrastructure_get_pin_interface

```
rx_pin_interface_t * rx_infrastructure_get_pin_interface(void)
```

Get global pin validator interface (thread-safe accessor).

Returns a pointer to the global `rx_pin_interface_t` that provides access to the pin validation and reservation subsystem. This interface prevents accidental pin conflicts by tracking which GPIO pins are reserved by which modules.

The returned pointer is valid for the entire program lifetime after successful initialization via `rx_infrastructure_init()`. Modules can cache this pointer for performance-critical code paths.

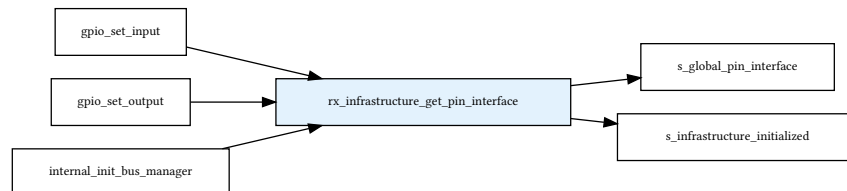


Figure 163: Call graph for `rx_infrastructure_get_pin_interface`

Defined in *libs/rx_core/inc/rx_infrastructure.h*:1266

—

rx_iwdt.h

Independent Watchdog Timer (IWDT) Driver for RX72N Microcontroller.

Production-grade watchdog driver providing safety-critical system monitoring with advanced task tracking, state-dependent timeouts, and comprehensive diagnostics. Designed for real-time embedded systems requiring fault detection and automatic recovery from software hangs, deadlocks, and system failures.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

iwdt_constants_t

IWDT Configuration Constants and Limits.

Defines compile-time constants for the IWDT driver including array sizes, timeout limits, and time conversion factors. These constants are chosen based on hardware limitations and system requirements.

Value	Name	Description
= 16	k_iwdt_max_tasks	Maximum number of tasks that can be registered for monitoring. Limits task array size to 1 KB. Increase if more tasks needed (recompile required)
= 32	k_iwdt_task_name_len	Maximum task name length in characters including null terminator. Matches ThreadX TX_THREAD_NAME_SIZE. Allows 31-character names
= 2000	k_iwdt_default_timeout_ms	Default watchdog timeout in milliseconds (2 seconds). Provides good balance between fast failure detection and false positive avoidance
= 128	k_iwdt_min_timeout_ms	Minimum valid timeout in milliseconds (128ms). Hardware limit with CKS=0. Use only for very fast control loops (> 10 Hz)
= 16384	k_iwdt_max_timeout_ms	Maximum valid timeout in milliseconds (16 seconds). Conservative limit for reasonable hang detection. Hardware supports up to 8s with CKS=3
= 1000	k_ms_per_second	Milliseconds per second (1000 ms/s). Time conversion constant for timeout calculations and timestamp arithmetic

iwdt_timeout_period_t

IWDT timeout periods (hardware divider settings).

Value	Name	Description
= 0	k_iwdt_timeout_128ms	128ms (CKS=00)
= 1	k_iwdt_timeout_512ms	512ms (CKS=01)
= 2	k_iwdt_timeout_2048ms	2s (CKS=10)

= 3	k_iwdt_timeout_8192ms	8s (CKS=11)
-----	-----------------------	-------------

—

system_state_t

System Operational States for State-Dependent Watchdog Timeouts.

Defines all possible system states, each with its own watchdog timeout period. The watchdog timeout is automatically adjusted when transitioning between states, allowing shorter timeouts during active operations and longer timeouts during idle periods.

Value	Name	Description
= 0	k_system_state_init	System initialization phase. Hardware setup, peripheral configuration. Use long timeout (5s) to allow slow startup. Transitions to: Idle
= 1	k_system_state_idle	Idle state, no active operations. Waiting for commands. Use long timeout (5s) since no critical operations. Transitions to: Running
= 2	k_system_state_running	Normal operation, moderate activity. Background tasks executing. Use default timeout (2s). Transitions to: MotorActive, CommActive, Idle, Error
= 3	k_system_state_motor_active	Motor control active, high-frequency updates. Critical real-time loop. Use short timeout (500ms) for fast failure detection. Transitions to: Running, Error
= 4	k_system_state_comm_active	Active communication (SPI/USB). Network I/O in progress. Use medium timeout (1s) to tolerate network delays. Transitions to: Running, Error
= 5	k_system_state_error	Error recovery mode. Diagnostics and cleanup. Use very long timeout (10s) to allow recovery actions and logging. Transitions to: Idle, Init
= 6	k_system_state_count	Number of system states (not a valid state). Used for array sizing (state_timeouts_ms[k_system_state_count]) and validation checks

—

iwdt_reset_reason_t

Watchdog reset reasons.

Value	Name	Description
= 0	k_iwdt_reset_unknown	Unknown reset
= 1	k_iwdt_reset_power_on	Power-on reset
= 2	k_iwdt_reset_watchdog	Watchdog timeout
= 3	k_iwdt_reset_external	External reset
= 4	k_iwdt_reset_low_voltage	Low voltage detect

rx_iwdt_init

```
rx_err_t rx_iwdt_init(const rx_iwdt_config_t *config)
```

Initialize the Independent Watchdog Timer.

Initializes the IWDT with specified configuration. Once started, the IWDT cannot be stopped (hardware safety feature).

Initialize the Independent Watchdog Timer.

Initializes the IWDT driver, validates configuration, checks for previous watchdog reset, and performs initial hardware feed. If config is nullptr, uses safe default values (1000ms timeout, task monitoring enabled).

Algorithm:

1. Check if already initialized (return error if so)
2. If config is nullptr, create default configuration
3. Validate timeout within [k_iwdt_min_timeout_ms, k_iwdt_max_timeout_ms]
4. Clear and initialize all state
5. Check RSTSR2 for previous watchdog reset
6. Perform initial feed to IWDTRR (0x00 then 0xFF)
7. Set initialized flag

Parameters:

Direction	Name	Description
in	config	Configuration structure (NULL for defaults)
in	config	Configuration pointer, or nullptr for defaults

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid configuration
k_rx_err_invalid_state	Already initialized
k_rx_err_hw_init_failed	Hardware initialization failed
k_rx_ok	Success
k_rx_err_invalid_state	Already initialized
k_rx_err_invalid_arg	Timeout out of valid range

Pre: IWDT not already initialized

Post: IWDT driver ready for use

Post: Initial feed performed

Post: Reset status captured

Note: This function must be called before any other IWDT functions

Note: Hardware timeout set by OFS, not this driver

Note: nullptr config uses safe defaults (1000ms, monitoring enabled)

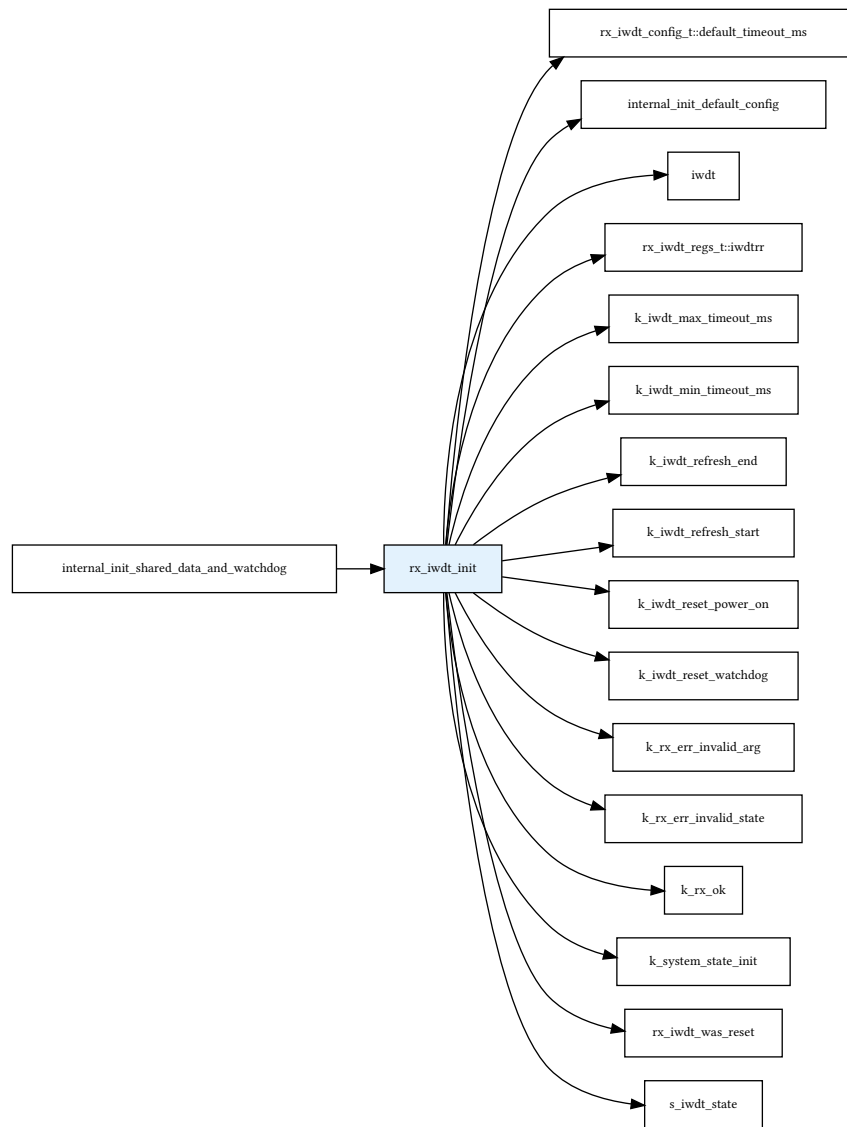
Warning: IWDT cannot be stopped once started - requires hard reset

Example::

```
rx_iwdt_config_tconfig={.default_timeout_ms=2000,.enable_task_monitoring=true,.reset_on_timeout=true};
config.state_timeouts_ms[k_system_state_running]=1000;
config.state_timeouts_ms[k_system_state_motor_active]=500;

rx_err_t err=rx_iwdt_init(&config); if(err!=k_rx_ok){ }
```

OFS Configuration Note:: The IWDT hardware timeout is configured at flash-time via OFS (Option Function Select) registers and cannot be changed at runtime. The timeout values in this driver are used for software task monitoring only. The hardware IWDT will timeout based on OFS settings regardless of software configuration.

Figure 164: Call graph for `rx_iwdt_init`

Defined in `libs/rx_core/inc/rx_iwdt.h:570`

Since Version 1.0.0

—

rx_iwdt_feed

```
rx_err_t rx_iwdt_feed(void)
```

Feed the watchdog timer.

Resets the watchdog counter to prevent timeout. Must be called periodically at a rate faster than the configured timeout.

Feed the watchdog timer.

Writes the two-byte refresh sequence (0x00 then 0xFF) to the IWDTRR Refresh Register (IWDTRR) to reset the hardware watchdog counter. This must be called periodically within the configured hardware timeout window or the IWDTRR will reset the microcontroller.

Per the RX72N hardware manual the refresh sequence is strictly ordered: writing 0x00 to IWDTRR starts the window; writing 0xFF completes the refresh. The entire sequence must occur within the permitted refresh window.

Also increments the software diagnostic counter `s_iwdt_state.status.watchdog_feeds` for telemetry and debugging purposes.

Algorithm steps:

1. Check `s_iwdt_state.initialized`; return `k_rx_err_not_initialized` if false
2. Write `k_iwdt_refresh_start` (0x00) to `regs->iwdtrr`
3. Write `k_iwdt_refresh_end` (0xFF) to `regs->iwdtrr`
4. Increment `status.watchdog_feeds` counter

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success
<code>k_rx_err_not_initialized</code>	IWDTRR not initialized
<code>k_rx_ok</code>	Watchdog counter successfully refreshed
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called

Pre: `rx_iwdt_init()` must have been called successfully

Pre: Must be called within the hardware IWDTRR refresh window (no gap > configured timeout)

Post: Hardware IWDTRR counter is reset; timeout window restarts

Post: `s_iwdt_state.status.watchdog_feeds` is incremented by one

Note: Thread-safe - can be called from multiple threads/ISRs

Note: Not thread-safe; concurrent feeds from multiple tasks are benign for the hardware but may produce incorrect feed counts

Warning: Missing feeds will cause system reset

Warning: Failing to call within the hardware timeout triggers a system reset

Example:: while(1){ rx_iwdt_feed(); tx_thread_sleep(50); }

Performance:: Execution time: <1 us @ 240 MHz; safe to call from any task or ISR context

Example:: while(1){ rx_err_t err=rx_iwdt_feed(); if(err!=k_rx_ok)
{ rx_log_error("wdt","IWDNotinitialized"); } tx_thread_sleep(k_watchdog_feed_interval_ticks); }

NASA Power of 10 Compliance:: Rule 5: 1 precondition (initialized check), 2 postconditions (HW reset, counter++)

See also: rx_iwdt_init() Initialize IWD before feeding, rx_iwdt_check_tasks() Verify all tasks are alive before feeding

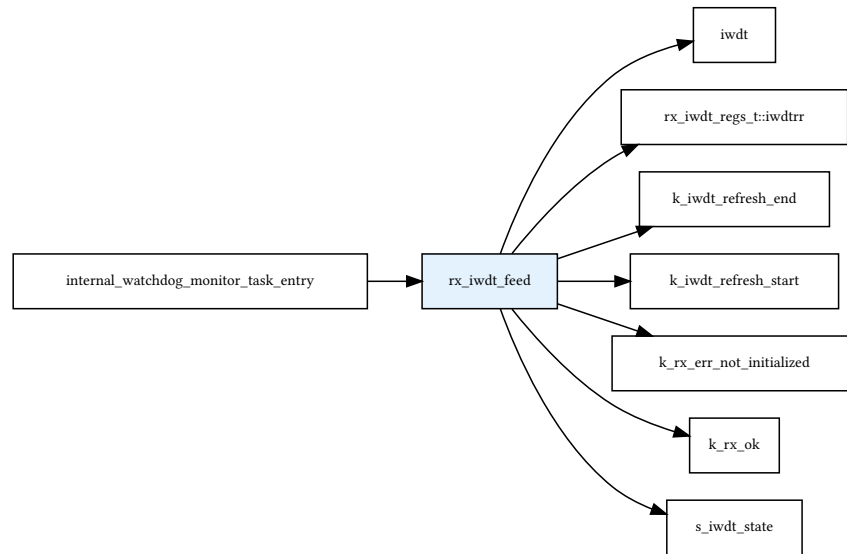


Figure 165: Call graph for rx_iwdt_feed

Defined in `libs/rx_core/inc/rx_iwdt.h:595`

Since Version 1.0.0

—

rx_iwdt_register_task

```
rx_err_t rx_iwdt_register_task(const char *task_name, uint32_t timeout_ms)
```

Register a task for heartbeat monitoring.

Registers a task with the watchdog for periodic heartbeat checks. Task must call `rx_iwdt_task_heartbeat()` within timeout period.

Register a task for heartbeat monitoring.

Adds a task entry to the IWD software monitoring table, associating the task name with a per-task timeout. Once registered, the task must call `rx_iwdt_task_heartbeat()` at least once per `timeout_ms` milliseconds or `rx_iwdt_check_tasks()` will report `k_rx_err_timeout`.

The registration also records the current tick count as the initial `last_heartbeat_tick`, giving the task a full `timeout_ms` window before the first heartbeat is required.

Algorithm steps:

1. Validate `task_name` is non-null
2. Verify `s_iwdt_state.initialized`
3. Validate `timeout_ms` is in `[k_iwdt_min_timeout_ms, k_iwdt_max_timeout_ms]`
4. Check task name length is in `(0, k_iwdt_task_name_len)`
5. Check task is not already registered via `internal_find_task()`
6. Find a free slot via `internal_find_free_slot()`
7. Initialize slot with task name, timeout, current tick, `active=true`, `timed_out=false`
8. Increment `status.active_tasks`

Parameters:

Direction	Name	Description
in	<code>task_name</code>	Unique task name (max 31 chars)
in	<code>timeout_ms</code>	Task-specific timeout in milliseconds
in	<code>task_name</code>	Unique task identifier string (non-empty, max <code>k_iwdt_task_name_len-1</code> chars, null-terminated); must remain valid for the lifetime of registration
in	<code>timeout_ms</code>	Per-task heartbeat timeout in milliseconds; valid range <code>[k_iwdt_min_timeout_ms(100), k_iwdt_max_timeout_ms(60000)]</code>

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success
<code>k_rx_err_null_ptr</code>	<code>task_name</code> is nullptr
<code>k_rx_err_invalid_arg</code>	Invalid timeout or duplicate task
<code>k_rx_err_not_initialized</code>	IWDT not initialized
<code>k_rx_err_no_mem</code>	Max tasks reached
<code>k_rx_ok</code>	Task successfully registered
<code>k_rx_err_null_ptr</code>	<code>task_name</code> is nullptr
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called
<code>k_rx_err_invalid_arg</code>	<code>timeout_ms</code> out of range, name empty, name too long, or task already registered
<code>k_rx_err_no_mem</code>	No free task slots (maximum <code>k_iwdt_max_tasks</code> tasks)

Pre: `rx_iwdt_init()` must have been called successfully

Pre: `task_name` must be unique among registered tasks

Post: Task entry is active in `s_iwdt_state.tasks`

Post: status.active_tasks is incremented by one

Note: Not thread-safe; serialize registrations from multiple contexts

Example:: rx_err_t err = rx_iwdt_register_task("MotorCtrl", 500); if (err != k_rx_ok) { }

Example:: rx_err_t err = rx_iwdt_register_task("motor_ctrl", 500); if (err != k_rx_ok) { rx_log_error("wdt", "Failed to register motor_ctrl task"); }

NASA Power of 10 Compliance:: Rule 5: 5 preconditions (null, initialized, timeout, name, duplicate), 2 postconditions

See also: rx_iwdt_task_heartbeat() Update heartbeat for a registered task,
rx_iwdt_check_tasks() Check all registered tasks for timeout

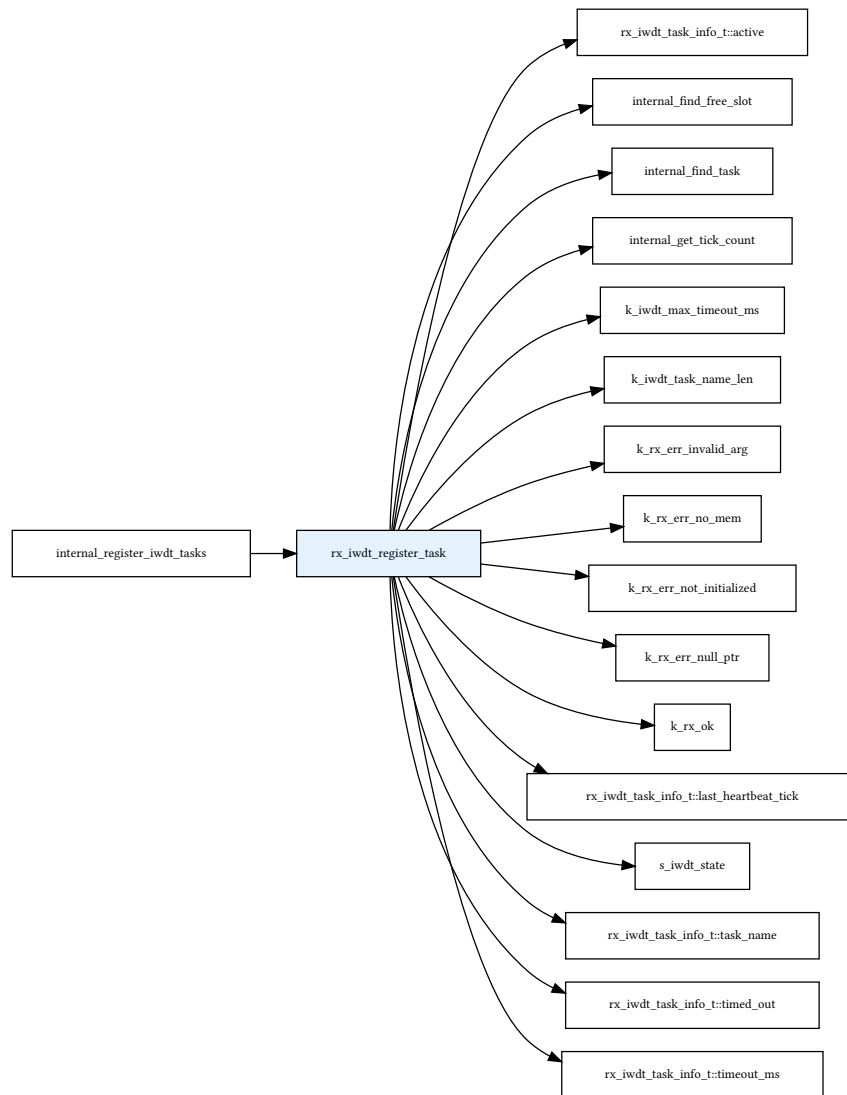


Figure 166: Call graph for rx_iwdt_register_task

Defined in `libs/rx_core/inc/rx_iwdt.h:621`

Since Version 1.0.0

—

rx_iwdt_task_heartbeat

```
rx_err_t rx_iwdt_task_heartbeat(const char *task_name)
```

Report task heartbeat.

Updates the heartbeat timestamp for a registered task. Must be called periodically faster than the task's timeout.

Report task heartbeat.

Updates the `last_heartbeat_tick` for the named task to the current ThreadX tick count, resetting the timeout window. Also clears the `timed_out` flag if the task had previously been marked as timed out.

Tasks must call this function at least once per their registered `timeout_ms` window (configured in `rx_iwdt_register_task()`) to remain in good standing. If the interval between heartbeats exceeds `timeout_ms`, the next call to `rx_iwdt_check_tasks()` will return `k_rx_err_timeout`.

Algorithm steps:

1. Validate `task_name` is non-null
2. Verify `s_iwdt_state.initialized`
3. Find the task entry via `internal_find_task()`
4. Set `task->last_heartbeat_tick = internal_get_tick_count()`
5. Clear `task->timed_out`

Parameters:

Direction	Name	Description
in	<code>task_name</code>	Name of the task
in	<code>task_name</code>	Null-terminated name of the registered task sending heartbeat; must exactly match the name used in <code>rx_iwdt_register_task()</code>

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success
<code>k_rx_err_null_ptr</code>	<code>task_name</code> is nullptr
<code>k_rx_err_not_found</code>	Task not registered
<code>k_rx_err_not_initialized</code>	IWDT not initialized
<code>k_rx_ok</code>	Heartbeat timestamp updated and <code>timed_out</code> flag cleared
<code>k_rx_err_null_ptr</code>	<code>task_name</code> is nullptr
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called
<code>k_rx_err_not_found</code>	No task registered with the given name

Pre: `rx_iwdt_init()` must have been called successfully

Pre: Task must have been registered via `rx_iwdt_register_task()`

Post: `task->last_heartbeat_tick` is refreshed to the current tick count

Post: `task->timed_out` is false

Note: Not thread-safe; concurrent heartbeats from different tasks are safe as they write to distinct slots, but concurrent writes to the same slot are not

Example:: static void motor_control_task(ULONG input){ while(1)
{ rx_iwdt_task_heartbeat("MotorCtrl"); tx_thread_sleep(10); } }

Example:: while(1){ rx_iwdt_task_heartbeat("motor_ctrl"); motor_control_update();
tx_thread_sleep(k_motor_ctrl_period_ticks); }

NASA Power of 10 Compliance:: Rule 5: 3 preconditions (null, initialized, registered), 2 postconditions

See also: rx_iwdt_register_task() Register a task before sending heartbeats,
rx_iwdt_check_tasks() Verify all tasks are within their timeout windows

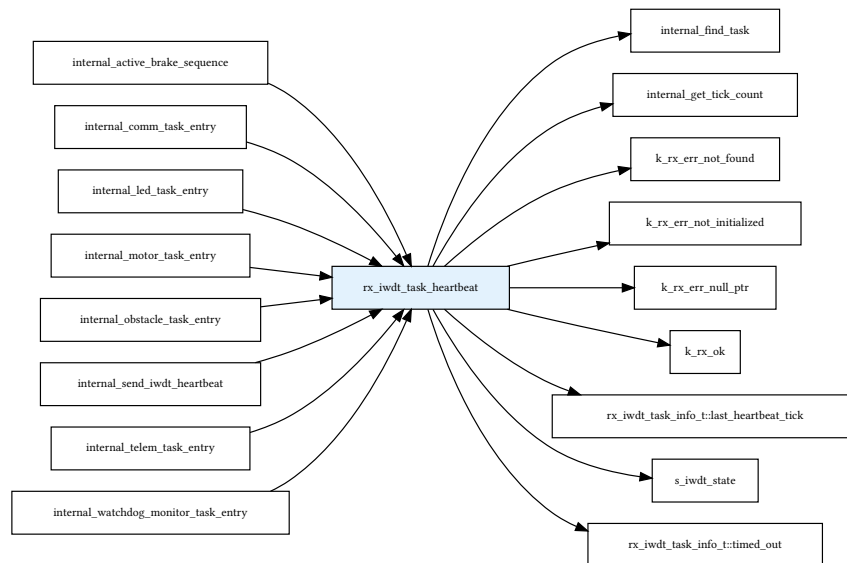


Figure 167: Call graph for rx_iwdt_task_heartbeat

Defined in `libs/rx_core/inc/rx_iwdt.h:647`

Since Version 1.0.0

—

rx_iwdt_set_timeout_for_state

```
rx_err_t rx_iwdt_set_timeout_for_state(system_state_t state, uint32_t timeout_ms)
```

Set timeout for system state.

Configures timeout period for a specific system state. Allows different timeout values based on system activity.

Set timeout for system state.

Configures the state-dependent software timeout used by `rx_iwdt_check_tasks()` when the system is in the specified state. This allows different task heartbeat windows depending on the operational mode (e.g. longer timeouts during initialization, tighter timeouts during normal operation).

Note: This does NOT change the hardware IWDG timeout. The hardware watchdog timeout is fixed at compile/flash time via OFS registers and cannot be altered at runtime. Only software task monitoring timeouts are affected.

If state matches the currently active state (`s_iwdt_state.current_state`), the active timeout (`status.current_timeout_ms`) is also updated immediately.

Algorithm steps:

1. Validate state is less than `k_system_state_count`
2. Verify `s_iwdt_state.initialized`
3. Validate `timeout_ms` is in `[k_iwdt_min_timeout_ms, k_iwdt_max_timeout_ms]`
4. Set `config.state_timeouts_ms[state] = timeout_ms`
5. If `state == current_state`, update `status.current_timeout_ms`

Parameters:

Direction	Name	Description
in	<code>state</code>	System state
in	<code>timeout_ms</code>	Timeout in milliseconds
in	<code>state</code>	System state identifier to configure (must be $< k_system_state_count$)
in	<code>timeout_ms</code>	Software task monitoring timeout in milliseconds; valid range <code>[k_iwdt_min_timeout_ms(100), k_iwdt_max_timeout_ms(60000)]</code>

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success
<code>k_rx_err_invalid_arg</code>	Invalid state or timeout
<code>k_rx_err_not_initialized</code>	IWDG not initialized
<code>k_rx_ok</code>	Timeout successfully updated
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called
<code>k_rx_err_invalid_arg</code>	<code>state</code> $\geq k_system_state_count$ or <code>timeout_ms</code> out of range

Pre: `rx_iwdt_init()` must have been called successfully

Pre: state must be a valid `system_state_t` value less than `k_system_state_count`

Post: `config.state_timeouts_ms[state]` equals `timeout_ms`

Post: If state is current, `status.current_timeout_ms` is updated immediately

Note: Not thread-safe; serialize state configuration changes

Warning: Hardware IWDT timeout is unaffected; only software task monitoring changes

Example:: `rx_iwdt_set_timeout_for_state(k_system_state_motor_active,500);`
`rx_iwdt_set_timeout_for_state(k_system_state_idle,5000);`

Example:: `rx_iwdt_set_timeout_for_state(k_system_state_initializing,5000);`
`rx_iwdt_set_timeout_for_state(k_system_state_running,500);`

NASA Power of 10 Compliance:: Rule 5: 3 preconditions (initialized, valid state, valid timeout), 2 postconditions

See also: `rx_iwdt_set_state()` Change the active system state, `rx_iwdt_check_tasks()` Uses current state timeout to evaluate task heartbeats

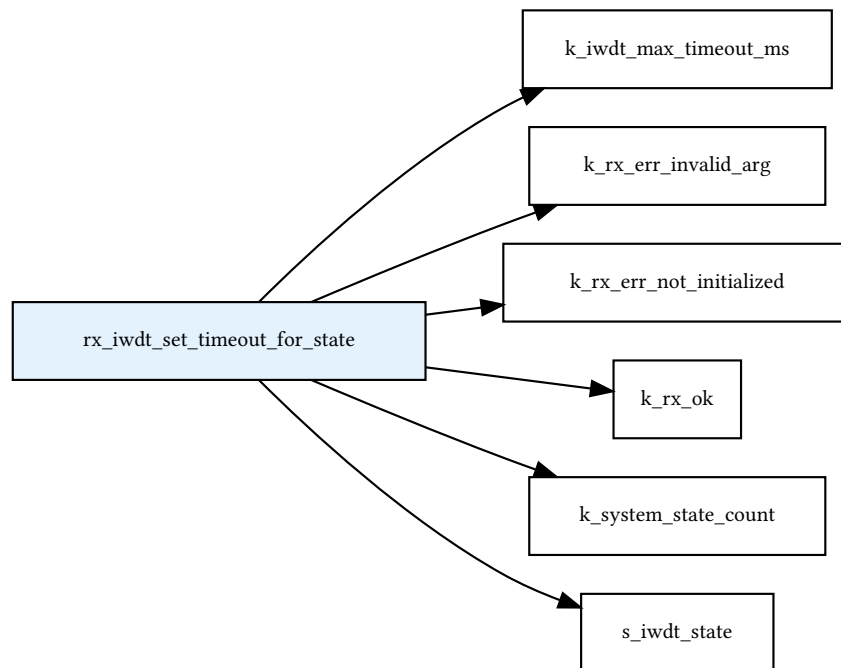


Figure 168: Call graph for `rx_iwdt_set_timeout_for_state`

Defined in `libs/rx_core/inc/rx_iwdt.h:670`

Since Version 1.0.0

—

rx_iwdt_set_state

```
rx_err_t rx_iwdt_set_state(system_state_t state)
```

Set current system state.

Updates the current system state, which determines the active watchdog timeout period.

Set current system state.

Updates the active system state used by `rx_iwdt_check_tasks()` to select per-state task heartbeat timeout windows. Transitions between states (e.g. initializing -> running) change which timeout threshold is applied when evaluating task heartbeat intervals.

This function updates three tracking fields atomically:

- `s_iwdt_state.current_state`
- `s_iwdt_state.status.current_state`
- `s_iwdt_state.status.current_timeout_ms` (set from `config.state_timeouts_ms[state]`)

Important: This does NOT modify the hardware IWDT peripheral. The hardware watchdog timeout is a fixed constant set in OFS registers at compile time. Only the software task monitoring timeout threshold changes.

Algorithm steps:

1. Validate state is less than `k_system_state_count`
2. Verify `s_iwdt_state.initialized`
3. Update `current_state` and `status.current_state = state`
4. Update `status.current_timeout_ms = config.state_timeouts_ms[state]`

Parameters:

Direction	Name	Description
in	<code>state</code>	New system state
in	<code>state</code>	New system state (must be < <code>k_system_state_count</code>)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success
<code>k_rx_err_invalid_arg</code>	Invalid state
<code>k_rx_err_not_initialized</code>	IWDT not initialized
<code>k_rx_ok</code>	State successfully updated
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called
<code>k_rx_err_invalid_arg</code>	<code>state >= k_system_state_count</code>

Pre: `rx_iwdt_init()` must have been called successfully

Pre: state must be a valid `system_state_t` value less than `k_system_state_count`

Post: `s_iwdt_state.current_state` reflects the new state

Post: `status.current_timeout_ms` reflects the configured timeout for the new state

Note: Not thread-safe; serialize state transitions from multiple contexts

Warning: Hardware IWDT is unaffected; only software task monitoring changes

Example:: rx_iwdt_set_state(k_system_state_motor_active);
rx_iwdt_set_state(k_system_state_running);

Example:: rx_err_t rx_iwdt_set_state(k_system_state_running); if(err!=k_rx_ok)
{ rx_log_error("wdt","Failed to set IWDT system state"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (initialized, valid state), 2 postconditions

See also: rx_iwdt_set_timeout_for_state() Configure per-state timeouts,
rx_iwdt_get_status() Query current state and timeout

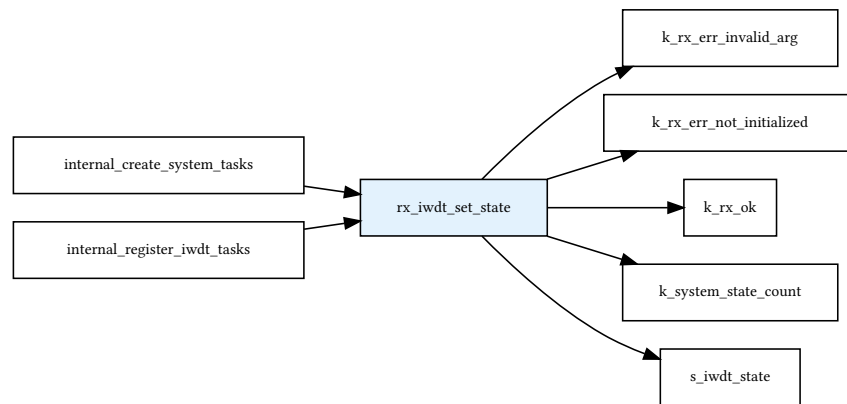


Figure 169: Call graph for rx_iwdt_set_state

Defined in `libs/rx_core/inc/rx_iwdt.h:693`

Since Version 1.0.0

—

rx_iwdt_get_status

```
rx_err_t rx_iwdt_get_status(rx_iwdt_status_t *status)
```

Get watchdog status.

Retrieves current watchdog status including reset history, failed tasks, and monitoring statistics.

Get watchdog status.

Copies the internal `s_iwdt_state.status` structure into the caller-provided buffer via `memcpy`. The status snapshot includes: active task count, total feed count, current system state, current timeout threshold, and the name of the last task that timed out (if any).

This is a diagnostic/telemetry function intended for logging and health monitoring. The returned data is a point-in-time snapshot and may become stale if the detection task or other threads update IWDT state concurrently.

Algorithm steps:

1. Validate status pointer is non-null
2. Verify `s_iwdt_state.initialized`
3. `memcpy(&s_iwdt_state.status, status, sizeof(rx_iwdt_status_t))`

Parameters:

Direction	Name	Description
out	<code>status</code>	Pointer to status structure
out	<code>status</code>	Pointer to caller-allocated <code>rx_iwdt_status_t</code> structure to receive the status snapshot; must be non-null; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success
<code>k_rx_err_null_ptr</code>	<code>status</code> is nullptr
<code>k_rx_err_not_initialized</code>	IWDT not initialized
<code>k_rx_ok</code>	Status snapshot successfully copied
<code>k_rx_err_null_ptr</code>	<code>status</code> is nullptr
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called

Pre: `rx_iwdt_init()` must have been called successfully

Pre: `status` must be a valid non-null pointer to an `rx_iwdt_status_t`

Post: `*status` contains a copy of `s_iwdt_state.status` at the time of the call

Post: Internal IWDT state is unchanged

Note: Not thread-safe; status fields may be updated concurrently by the task monitor

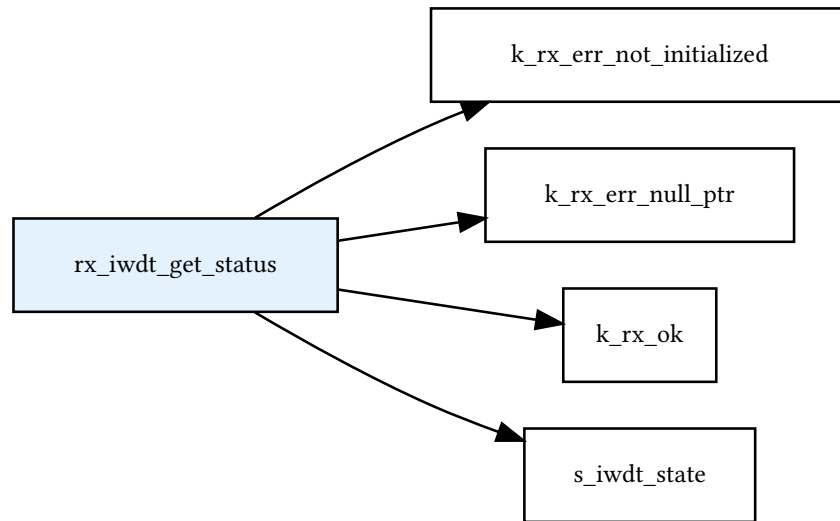
Note: Snapshot accuracy depends on timing; use as diagnostic data, not safety-critical logic

Example:: `rx_iwdt_status_t status; rx_err_t err = rx_iwdt_get_status(&status); if(err == k_rx_ok) { printf("Total resets: %lu\n", status.total_resets); printf("Last failed task: %sn", status.last_failed_task); }`

Example:: `rx_iwdt_status_t status; rx_err_t err = rx_iwdt_get_status(&status); if(err == k_rx_ok) { rx_log_info("wdt", "Active tasks: %u, Feeds: %u", (unsigned)status.active_tasks, (unsigned)status.watchdog_feeds); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: `rx_iwdt_was_reset()` Check if last reset was hardware IWDT timeout, `rx_iwdt_check_tasks()` Check all tasks for software timeout

Figure 170: Call graph for `rx_iwdt_get_status`

Defined in `libs/rx_core/inc/rx_iwdt.h:717`

Since Version 1.0.0

—

rx_iwdt_was_reset

```
bool rx_iwdt_was_reset(void)
```

Check if last reset was caused by watchdog.

Determines if the previous system reset was triggered by watchdog timeout.

Check if last reset was caused by watchdog.

Reads the IWDT Reset Flag (IWDTRF) from the Reset Status Register 2 (RSTSR2) of the RX72N. This register is located at a separate address from RSTSR0/1 and retains the reset cause flags across a reset event (values are cleared on power-on reset only).

The IWDTRF bit (bit position `k_rstsr2_iwdtrf`) is set by hardware when the IWDT counter underflows (i.e., the watchdog was not refreshed in time). It remains set until explicitly cleared or a power-on reset occurs.

This function is used during startup to detect and log prior watchdog resets, which may indicate a firmware hang or task scheduling failure.

Algorithm steps:

1. Read the RSTSR2 register byte via `rstsr2()` inline accessor
2. Mask with `k_rstsr2_iwdtrf` to extract the IWDT reset flag bit
3. Return true if the bit is set, false otherwise

Returns: bool IWDT reset detection result

Value	Description
-------	-------------

true	RSTSR2.IWDTRF is set; last reset was caused by IWDT timeout
false	RSTSR2.IWDTRF is clear; last reset was not caused by IWDT

Pre: None safe to call before rx_iwdt_init() (reads hardware register directly)

Pre: Must be called before the application clears the reset status register

Post: Hardware RSTSR2 register is read-only; this function does not modify it

Post: Return value reflects hardware register state at the instant of the call

Note: Can be called before rx_iwdt_init()

Note: Thread-safe reads a single hardware register without shared state

Note: Typically called once at startup to log the reset cause

Warning: RSTSR2.IWDTRF remains set across warm resets; clear it in startup if needed

Example:: if(rx_iwdt_was_reset())
{ uart_debug_puts("[WARN]Systemrecoveredfromwatchdogresetrn"); }

Example:: if(rx_iwdt_was_reset()){ rx_log_error("boot","PriorresetcausedbyIWDTimeout-checktaskhealth"); } rx_iwdt_init(&iwdt_config);

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (safe before init, called before register clear), 2 postconditions

See also: rx_iwdt_feed() Feed the hardware watchdog to prevent reset,
rx_iwdt_check_tasks() Monitor software task heartbeats

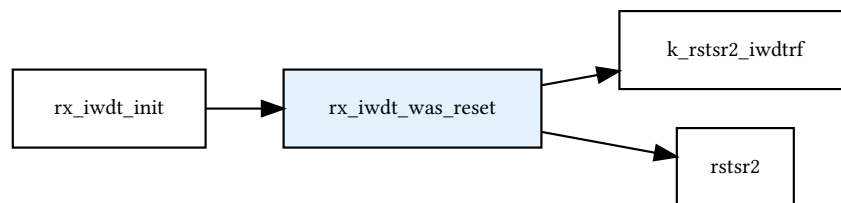


Figure 171: Call graph for rx_iwdt_was_reset

Defined in `libs/rx_core/inc/rx_iwdt.h:737`

Since Version 1.0.0

—

rx_iwdt_check_tasks

```
rx_err_t rx_iwdt_check_tasks(void)
```

Check for task timeouts.

Checks all registered tasks for timeout. Should be called periodically from a monitoring task.

Check for task timeouts.

Iterates over all active task slots and compares the elapsed tick count since each task's last heartbeat against that task's configured timeout (converted from milliseconds to ThreadX ticks). If any task has exceeded its timeout, the task's `timed_out` flag is set, the task name is recorded in `status.last_failed_task`, and the function returns `k_rx_err_timeout`.

If task monitoring is disabled (`config.enable_task_monitoring == false`), returns `k_rx_ok` immediately without checking any tasks.

The caller (typically the watchdog task) should conditionally feed the hardware IWDT only if this function returns `k_rx_ok` i.e., only refresh the hardware watchdog when all software tasks are alive.

Algorithm steps:

1. Verify `s_iwdt_state.initialized`
2. If `!config.enable_task_monitoring`, return `k_rx_ok` early
3. Capture `current_tick = internal_get_tick_count()`
4. For each active task slot ($i < k_iwdt_max_tasks$):
 - a. Skip inactive slots
 - b. Compute `elapsed_ticks = current_tick - task->last_heartbeat_tick`
 - c. Compute `timeout_in_ticks` from `task->timeout_ms` and `TX_TIMER_TICKS_PER_SECOND`
 - d. If `elapsed_ticks > timeout_in_ticks`: set `timed_out=true`, record task name, set `any_timeout=true`
5. Return `k_rx_err_timeout` if `any_timeout`, else `k_rx_ok`

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	No timeouts detected
<code>k_rx_err_timeout</code>	One or more tasks timed out
<code>k_rx_err_not_initialized</code>	IWDT not initialized
<code>k_rx_ok</code>	All active tasks are within their heartbeat timeout windows
<code>k_rx_err_timeout</code>	One or more registered tasks have exceeded their timeout; name of first failing task is recorded in <code>status.last_failed_task</code>
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called

Pre: `rx_iwdt_init()` must have been called successfully

Pre: Registered tasks must be calling `rx_iwdt_task_heartbeat()` on schedule

Post: Timed-out tasks have their `timed_out` flag set

Post: status.last_failed_task contains the name of the most recently detected failure

Note: This function does NOT reset the hardware watchdog

Note: Not thread-safe; task heartbeat fields may be updated concurrently

Warning: Task timeouts trigger watchdog reset if not cleared

Warning: Do NOT feed the hardware IWD (rx_iwdt_feed()) if this returns k_rx_err_timeout; allow the hardware watchdog to fire and reset the system

Example:: static void watchdog_monitor_task(ULONG input){ while(1)
{ rx_err_t err=rx_iwdt_check_tasks(); if(err==k_rx_err_timeout)
{ uart_debug_puts(“[ERROR]Tasktimeoutdetected\rn”); } rx_iwdt_feed(); tx_thread_sleep(10); } }

Example:: while(1){ if(rx_iwdt_check_tasks()!=k_rx_ok){ rx_iwdt_feed(); }
else{ rx_log_error(“wdt”,“Tasktimeoutdetected-allowingIWDTrset”); }
tx_thread_sleep(k_watchdog_check_interval_ticks); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (initialized, tasks registered), 2 postconditions

See also: rx_iwdt_feed() Feed hardware watchdog (call only when this returns k_rx_ok),
rx_iwdt_task_heartbeat() Update per-task heartbeat from each monitored task,
rx_iwdt_register_task() Register a task for monitoring

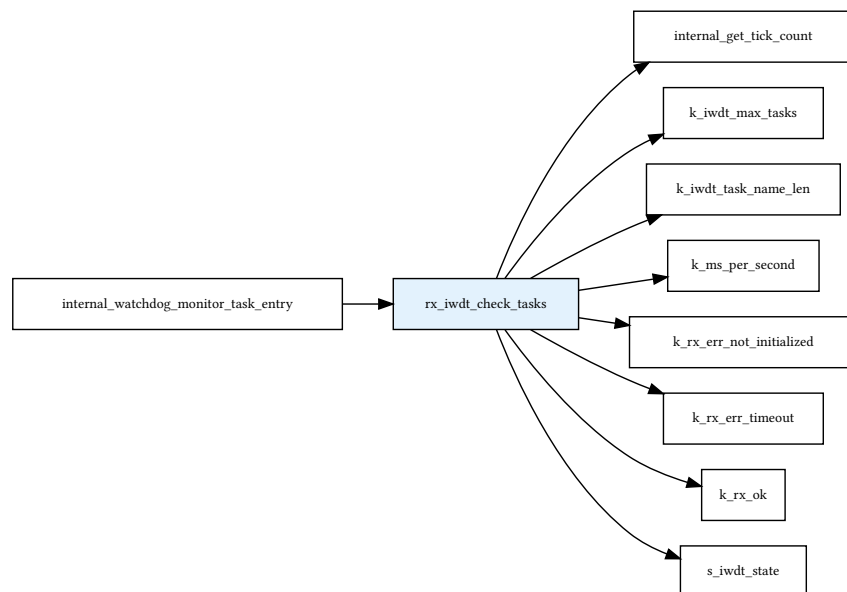


Figure 172: Call graph for rx_iwdt_check_tasks

Defined in libs/rx_core/inc/rx_iwdt.h:768

Since Version 1.0.0

—

rx_log.h

Type-Safe Logging System for RX72N Firmware with Zero-Overhead Compile-Time Filtering.

Includes:

- <stdint.h>
- <string.h>
- "rx_err.h"
- "rx_simulator_config.h"

USB_LOG_MIRROR

```
#define USB_LOG_MIRROR 1
```

—

LOG_PUTC

```
#define LOG_PUTC do
{
    char _log_c = (c);
    \
    uart_debug_putc(_log_c);
    \
    rx_log_usb_putc(_log_c);
    \
} while (0)
```

—

LOG_PUTS

```
#define LOG_PUTS do
{
    const char* _log_s = (s);
    \
    uart_debug_puts(_log_s);
    \
    rx_log_usb_puts(_log_s);
    \
} while (0)
```

—

LOG_PUTINT

```
#define LOG_PUTINT do
{
    int32_t _log_v = (v);
    \
    uart_debug_putint(_log_v);
    \
    rx_log_usb_putint(_log_v);
    \
} while (0)
```

—

LOG_PUTHEX

```
#define LOG_PUTHEX do
{
    uint32_t _log_vhex = (v);
    \
    uint8_t _log_d = (d);
    \
    uart_debug_puthex(_log_vhex, _log_d);
    \
    rx_log_usb_puthex(_log_vhex, _log_d);
    \
} while (0)
```

—

LOG_LEVEL

```
#define LOG_LEVEL (k_log_info)
```

Default log level configuration.

—

RX_LOG_VAL_IMPL

```
#define RX_LOG_VAL_IMPL _Generic((val),
\
    uint8_t: internal_rx_log_##level##_u8,
\
    uint16_t: internal_rx_log_##level##_u16,
\
    uint32_t: internal_rx_log_##level##_u32,
\
    int32_t: internal_rx_log_##level##_err)(tag, msg, val)
```

Internal macro implementing C23 _Generic automatic type dispatch for logging.

—

rx_log_error

```
#define rx_log_error internal_rx_log_error((tag), (msg))
```

Log critical ERROR-level message (compile-time filtered).

—

rx_log_error_val

```
#define rx_log_error_val RX_LOG_VAL_IMPL(error, (tag), (msg), (val))
```

Log ERROR-level message with automatic type-dispatched value.

—

rx_log_error_hex

```
#define rx_log_error_hex internal_rx_log_error_hex((tag), (msg), (val), (digits))
```

Log ERROR-level message with hexadecimal value.

—

rx_log_error_str

```
#define rx_log_error_str internal_rx_log_error_str((tag), (msg), (str), (len))
```

Log ERROR-level message with bounded string value (NASA-compliant).

—

rx_log_warn

```
#define rx_log_warn internal_rx_log_warn((tag), (msg))
```

Log warning message (no value).

—

rx_log_warn_val

```
#define rx_log_warn_val RX_LOG_VAL_IMPL(warn, (tag), (msg), (val))
```

Log warning with typed value (automatic type dispatch).

—

rx_log_warn_hex

```
#define rx_log_warn_hex internal_rx_log_warn_hex((tag), (msg), (val), (digits))
```

Log warning with hex value.

—

rx_log_warn_str

```
#define rx_log_warn_str internal_rx_log_warn_str((tag), (msg), (str), (len))
```

Log warning with string value (bounded length).

—

rx_log_info

```
#define rx_log_info internal_rx_log_info((tag), (msg))
```

Log info message (no value).

—

rx_log_info_val

```
#define rx_log_info_val RX_LOG_VAL_IMPL(info, (tag), (msg), (val))
```

Log info with typed value (automatic type dispatch).

—

rx_log_info_hex

```
#define rx_log_info_hex internal_rx_log_info_hex((tag), (msg), (val), (digits))
```

Log info with hex value.

—

rx_log_info_str

```
#define rx_log_info_str internal_rx_log_info_str((tag), (msg), (str), (len))
```

Log info with string value (bounded length).

—

rx_log_debug

```
#define rx_log_debug internal_rx_log_debug((tag), (msg))
```

Log debug message (no value).

—

rx_log_debug_val

```
#define rx_log_debug_val RX_LOG_VAL_IMPL(debug, (tag), (msg), (val))
```

Log debug with typed value (automatic type dispatch).

—

rx_log_debug_hex

```
#define rx_log_debug_hex internal_rx_log_debug_hex((tag), (msg), (val), (digits))
```

Log debug with hex value.

—

rx_log_debug_str

```
#define rx_log_debug_str internal_rx_log_debug_str((tag), (msg), (str), (len))
```

Log debug with string value (bounded length).

—

rx_log_verbose

```
#define rx_log_verbose internal_rx_log_verbose((tag), (msg))
```

Log verbose message (no value).

—

rx_log_verbose_val

```
#define rx_log_verbose_val RX_LOG_VAL_IMPL(verbose, (tag), (msg), (val))
```

Log verbose with typed value (automatic type dispatch).

—

rx_log_verbose_hex

```
#define rx_log_verbose_hex internal_rx_log_verbose_hex((tag), (msg), (val), (digits))
```

Log verbose with hex value.

—

rx_log_verbose_str

```
#define rx_log_verbose_str internal_rx_log_verbose_str((tag), (msg), (str), (len))
```

Log verbose with string value (bounded length).

—

log_level_t

Log level enumeration defining severity hierarchy for compile-time filtering.

Defines the log level hierarchy used for compile-time filtering. Each level includes all messages from higher-priority (lower-numbered) levels. The LOG_LEVEL macro determines which levels are compiled into the binary.

Value	Name	Description
-------	------	-------------

= 0	k_log_none	No logging (production silent mode). All logs eliminated at compile time.
= 1	k_log_error	Critical errors only. Operation-preventing failures, requires immediate attention.
= 2	k_log_warn	Errors and warnings. Recoverable issues, degraded functionality.
= 3	k_log_info	Errors, warnings, and informational messages. Default level for development.
= 4	k_log_debug	Errors, warnings, info, and debug messages. Detailed troubleshooting information.
= 5	k_log_verbose	All messages including verbose trace. Maximum detail for complex debugging.

—

log_format_constants_t

Logging formatting and safety constants.

Defines compile-time constants for log output formatting and memory safety bounds. These constants ensure predictable behavior and prevent buffer overruns.

Value	Name	Description
= 8	k_log_hex_width_err	Hex digit width for rx_err_t error codes (32-bit = 8 digits). Example: 0x00000108
= 256	k_log_str_max_len	Maximum string length for bounded logging (NASA Rule 2 compliance). Prevents unbounded loops.

—

uart_debug_putc

```
void uart_debug_putc(char data)
```

Transmit single character on debug UART (SCI9).

Convenience wrapper around `uart_putc_channel()` for the fixed debug channel (SCI9). Errors are silently ignored to allow use in early initialization contexts where error propagation is not yet possible.

Algorithm steps:

1. Call `uart_putc_channel(k_uart_debug_channel, data)`
2. Cast return value to (void) - errors discarded

Parameters:

Direction	Name	Description
in	data	Character to transmit (0x00-0xFF) Special handling: ‘ ‘ is NOT converted; use <code>uart_debug_puts()</code> for text

Pre: `uart_debug_init()` must have been called successfully

Pre: SCI9 must be initialized and TX enabled

Post: Character written to SCI9 TDR and transmission started

Post: Any errors are silently discarded

Note: Error return from `uart_putc_channel()` is intentionally discarded

Note: For error-checked output use `uart_putc_channel()` directly

Warning: Only available when `RX_IS_SIMULATOR` is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us @ 115200 baud Stack usage: 16 bytes

Example:: `uart_debug_putc('A'); uart_debug_putc('r'); uart_debug_putc('n');`

See also: `uart_debug_puts()` Transmit string with newline conversion, `uart_putc_channel()` Error-checked single character transmit

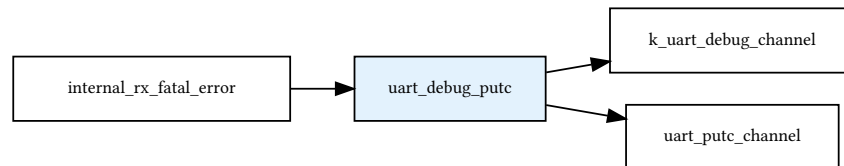


Figure 173: Call graph for `uart_debug_putc`

Defined in `libs/rx_core/inc/rx_log.h:399`

Since Version 1.0.0

—

uart_debug_puts

```
void uart_debug_puts(const char *str)
```

Transmit null-terminated string on debug UART with newline conversion.

Transmits a null-terminated string to SCI9 with automatic LF-to-CR+LF conversion for terminal compatibility. Enforces a maximum length of `k_uart_max_str_len` (256) characters per NASA Power of 10 Rule 2. Silently returns on NULL pointer to allow safe use in early init.

Algorithm steps:

1. Return immediately if `str` is nullptr (defensive, no error return)
2. For each character up to `k_uart_max_str_len`: a. Stop at null terminator b. If ‘

‘, send ‘r’ first c. Send character via `uart_debug_putc()`

Parameters:

Direction	Name	Description
in	str	Pointer to null-terminated ASCII string Maximum length: 256 characters (k_uart_max_str_len) Null handling: Returns silently if nullptr Encoding: ASCII; ‘ ‘ converted to ‘r ‘

Pre: `uart_debug_init()` must have been called successfully

Pre: str should point to a null-terminated string in valid memory

Post: All characters up to null terminator transmitted to SCI9

Post: Each ‘] ‘ replaced by ‘r ‘ in the transmitted output

Note: No return value - errors from `uart_debug_putc()` are silently discarded

Note: String truncated at `k_uart_max_str_len` (256) characters

Warning: Only available when `RX_IS_SIMULATOR` is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes

Example:: `uart_debug_puts("Hello,World!\n");` `uart_debug_puts("[INFO]Bootcompleten");`

See also: `uart_debug_putc()` Transmit single character, `uart_debug_putint()` Transmit decimal integer, `uart_puts_channel()` Error-checked string transmit

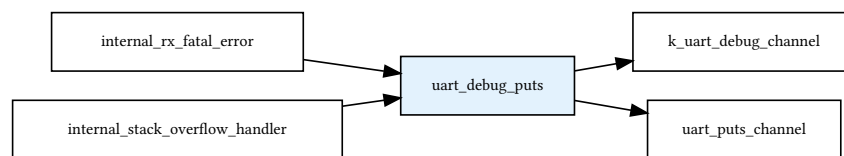


Figure 174: Call graph for `uart_debug_puts`

Defined in `libs/rx_core/inc/rx_log.h:400`

Since Version 1.0.0

—

uart_debug_putint

```
void uart_debug_putint(int32_t value)
```

Transmit signed 32-bit integer as decimal string on debug UART.

Converts a signed 32-bit integer to a decimal ASCII string and transmits it on SCI9. Handles INT32_MIN correctly by using int64_t for the negation. Uses a fixed-size stack buffer (k_uart_int_buffer_size = 12) built in reverse then passed to uart_debug_puts().

Algorithm steps:

1. Determine sign; compute abs_value as uint32_t
2. Null-terminate the buffer end
3. Build digits right-to-left (statically bounded by k_uart_int_buffer_size)
4. Prepend '-' if negative
5. Call uart_debug_puts() with the resulting substring pointer

Parameters:

Direction	Name	Description
in	value	Signed 32-bit integer to print Valid range: INT32_MIN (-2147483648) to INT32_MAX (2147483647) INT32_MIN handling: Correctly handled via int64_t cast

Pre: uart_debug_init() must have been called successfully

Pre: uart_debug_puts() must be functional

Post: Decimal representation of value transmitted on SCI9

Post: Leading zeros suppressed; negative values prefixed with '-'

Note: No return value - output errors are silently discarded

Note: Buffer is stack-allocated; safe for re-entrant calls on different tasks

Warning: Only available when RX_IS_SIMULATOR is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per digit @ 115200 baud + digit-loop overhead Stack usage: 32 bytes (buffer + locals)

Example:: uart_debug_puts("Value:"); uart_debug_putint(42); uart_debug_putint(-2147483648);
uart_debug_putc('n');

See also: uart_debug_puthex() Print value in hexadecimal, uart_debug_puts() Underlying string transmit

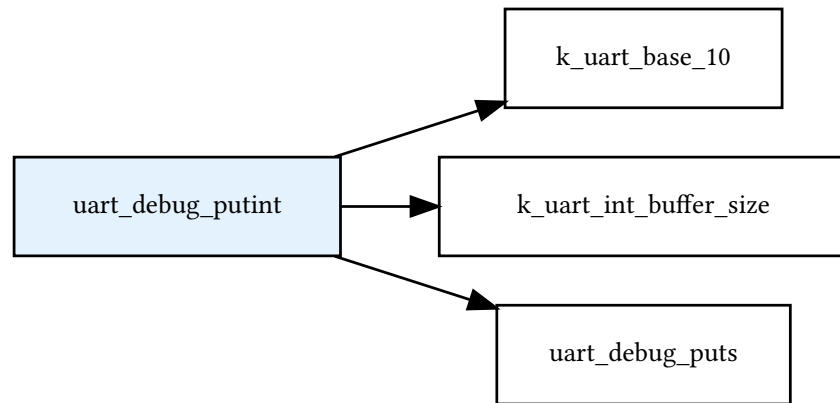


Figure 175: Call graph for uart_debug_putint

Defined in `libs/rx_core/inc/rx_log.h:401`

Since Version 1.0.0

—

uart_debug_puthex

```
void uart_debug_puthex(uint32_t value, uint8_t digits)
```

Transmit 32-bit unsigned integer as hexadecimal string on debug UART.

Transmits “0x” prefix followed by the specified number of uppercase hex digits of `value` on SCI9. Digits are always printed most-significant first. The `digits` parameter is clamped to `[k_uart_hex_min_digits, k_uart_hex_max_digits]` (1-8) before use.

Algorithm steps:

1. Send “0x” prefix via `uart_debug_puts()`
2. Clamp digits to `[1, 8]`
3. Iterate nibbles from MSN to LSN (statically bounded by `k_uart_hex_max_digits`)
4. For each nibble index < digits, extract nibble and print uppercase hex char

Parameters:

Direction	Name	Description
in	value	Unsigned 32-bit value to display in hexadecimal Valid range: 0x00000000 to 0xFFFFFFFF
in	digits	Number of hex digits to print (1-8) Valid range: 1 to 8 (clamped; 0 -> 1, >8 -> 8) Common values: 2 for byte, 4 for word, 8 for full 32-bit

Pre: `uart_debug_init()` must have been called successfully

Pre: `uart_debug_puts()` and `uart_debug_putc()` must be functional

Post: “0x” followed by digits uppercase hex characters transmitted on SCI9

Post: digits clamped to 1, 8 if out of range

Note: Uses static lookup table s_hex for digit-to-character conversion

Note: Always prefixes output with “0x”

Warning: Only available when RX_IS_SIMULATOR is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes

Example:: `uart_debug_puts(“Addr:”); uart_debug_puthex(0xDEADBEEF,8);`
`uart_debug_puthex(0x42,2); uart_debug_putc(‘n’);`

See also: `uart_debug_putint()` Print signed decimal value, `uart_debug_puts()` Underlying string transmit

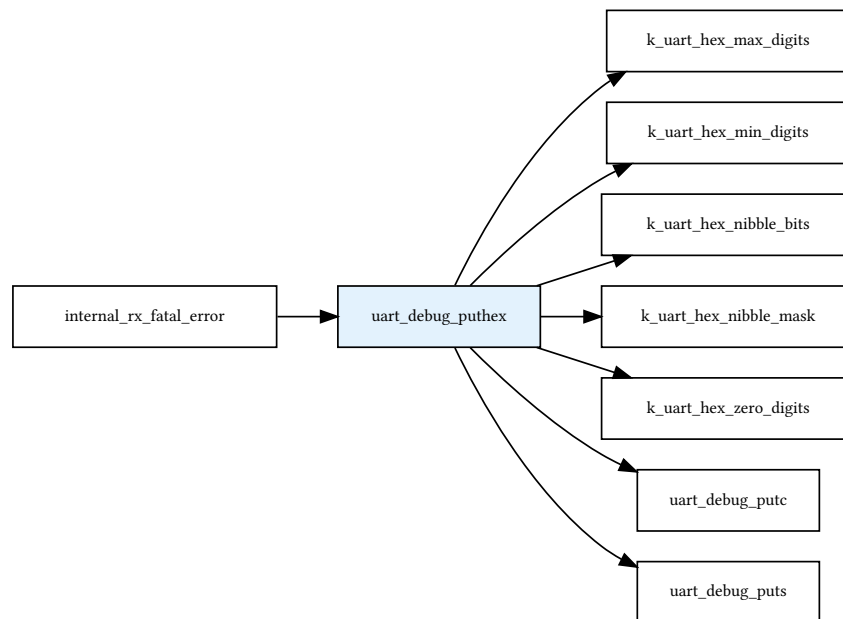


Figure 176: Call graph for `uart_debug_puthex`

Defined in `libs/rx_core/inc/rx_log.h:402`

Since Version 1.0.0

rx_log_usb_putc

```
void rx_log_usb_putc(char c)
```

Write a single character to USB CDC log port.

Thread-safe wrapper around `internal_write_usb()`. Buffers character during boot, sends to USB after enumeration.

Parameters:

Direction	Name	Description
in	c	Character to write

Note: Thread-safe, non-blocking

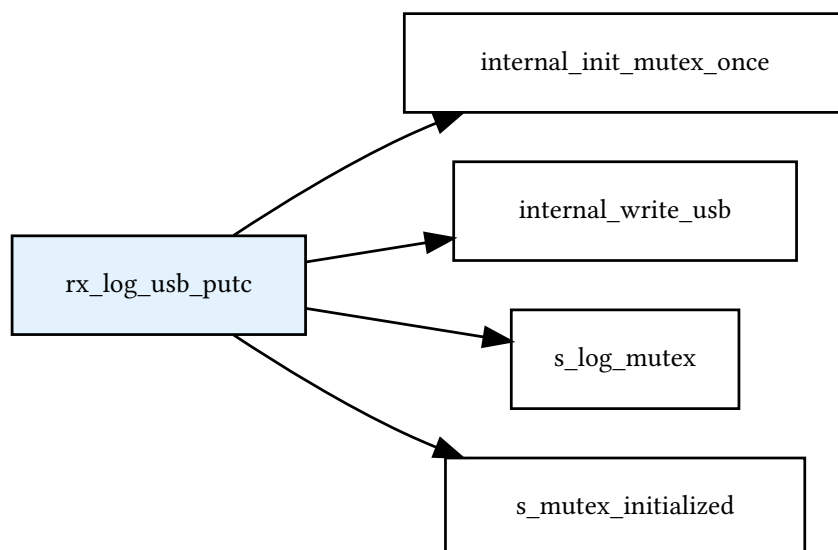


Figure 177: Call graph for `rx_log_usb_putc`

Defined in `libs/rx_core/inc/rx_log.h:447`

—

rx_log_usb_puts

```
void rx_log_usb_puts(const char *str)
```

Write a null-terminated string to USB CDC log port.

Thread-safe wrapper around `internal_write_usb()`. Buffers string during boot, sends to USB after enumeration.

Parameters:

Direction	Name	Description
in	str	String to write (must be null-terminated)

Note: Thread-safe, non-blocking

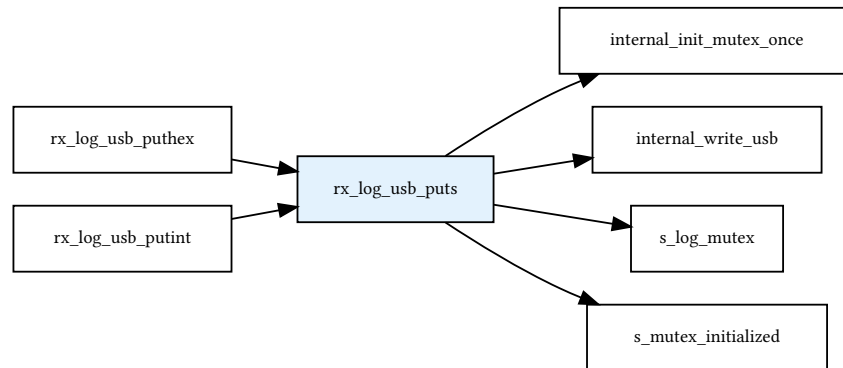


Figure 178: Call graph for `rx_log_usb_puts`

Defined in `libs/rx_core/inc/rx_log.h:454`

—

`rx_log_usb_putint`

```
void rx_log_usb_putint(int32_t value)
```

Write a signed integer as decimal text to USB CDC log port.

Converts integer to ASCII decimal string and writes to USB. Handles negative numbers with leading '-' sign.

Parameters:

Direction	Name	Description
in	value	Integer value to write

Note: Thread-safe, non-blocking

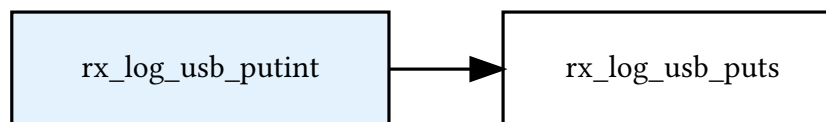


Figure 179: Call graph for `rx_log_usb_putint`

Defined in `libs/rx_core/inc/rx_log.h:461`

—

rx_log_usb_puthex

```
void rx_log_usb_puthex(uint32_t value, uint8_t digits)
```

Write an unsigned integer as hexadecimal text to USB CDC log port.

Converts integer to ASCII hex string (uppercase A-F) with zero-padding. Useful for register values, memory addresses, bit masks, and error codes.

Parameters:

Direction	Name	Description
in	value	Value to write as hex
in	digits	Number of hex digits (1-8, zero-padded)

Note: Thread-safe, non-blocking

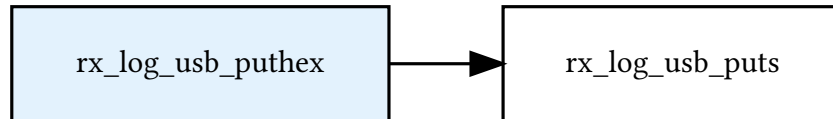


Figure 180: Call graph for rx_log_usb_puthex

Defined in `libs/rx_core/inc/rx_log.h:469`

—

rx_log_usb_get_stats

```
void rx_log_usb_get_stats(usb_log_stats_t *stats)
```

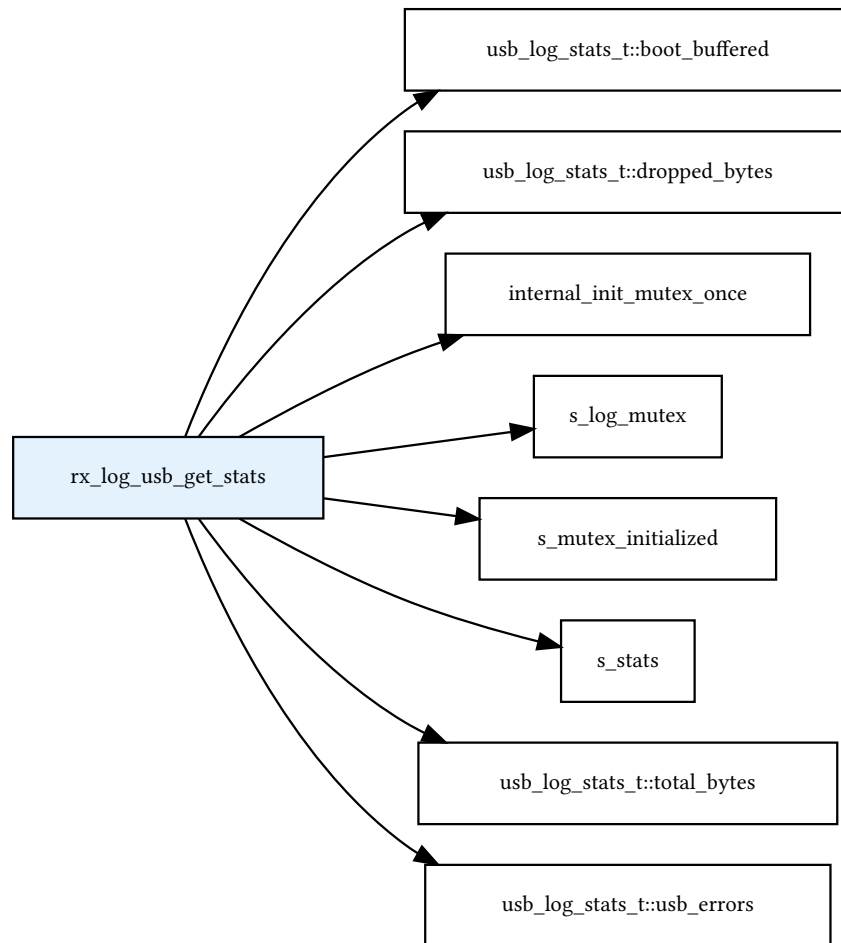
Get USB CDC logging statistics.

Returns current logging statistics (total, dropped, buffered, errors). Useful for diagnostics and performance monitoring.

Parameters:

Direction	Name	Description
out	stats	Statistics structure to fill

Note: Thread-safe

Figure 181: Call graph for `rx_log_usb_get_stats`

Defined in `libs/rx_core/inc/rx_log.h:486`

—

rx_log_usb_notify_ready

```
void rx_log_usb_notify_ready(void)
```

Notify logging system that USB is ready (optional callback).

Notify logging system that USB is ready (optional callback).

Called by USB stack after successful enumeration (`k_usb_state_configured`). Triggers immediate flush of boot buffer. Optional - flush also happens automatically on next log write.

Note: Thread-safe, optional (automatic flush happens on next log write)

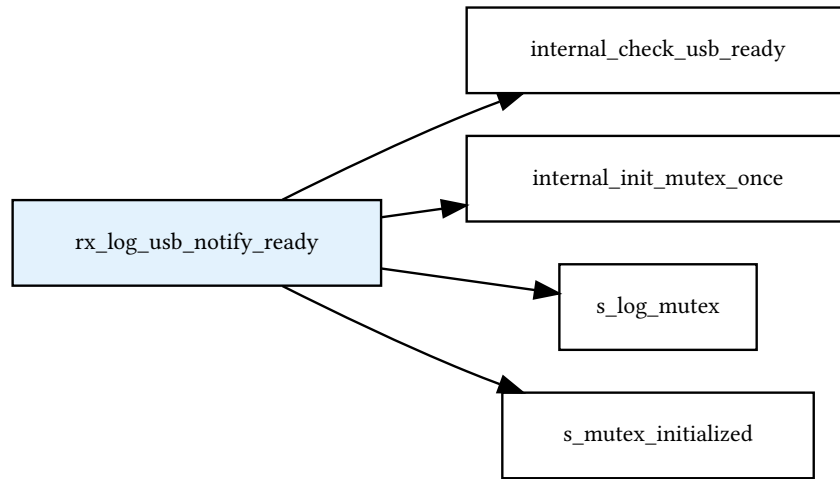


Figure 182: Call graph for `rx_log_usb_notify_ready`

Defined in `libs/rx_core/inc/rx_log.h:492`

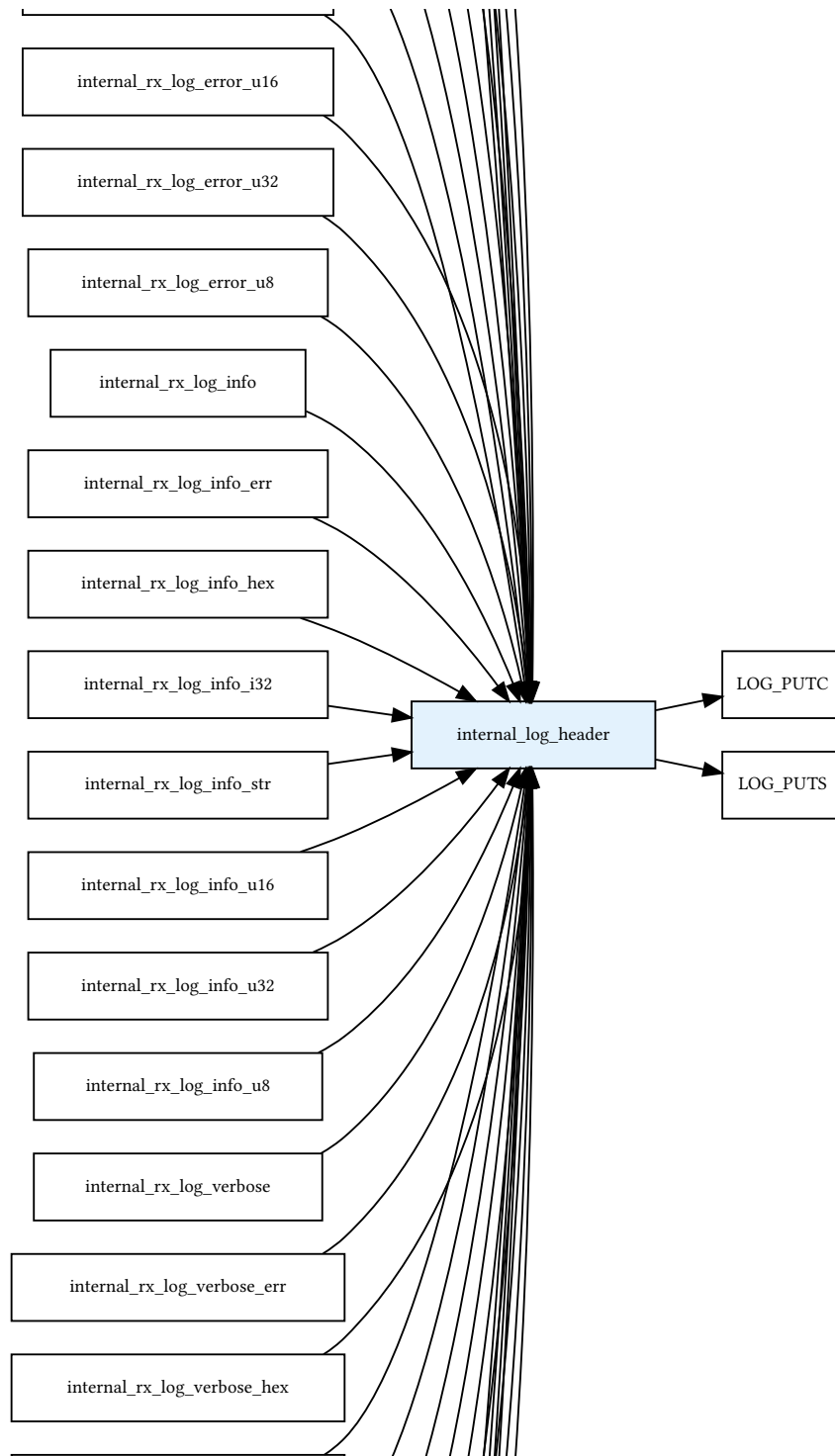
—

internal_log_header

```
static void internal_log_header(const char *level_str, const char *tag)
```

Internal helper to format and output log message header.

Outputs standardized log header in format: [LEVEL] [TAG] This prefix precedes all log messages for consistent parsing by log analysis tools.

Figure 183: Call graph for `internal_log_header`

Defined in `libs/rx_core/inc/rx_log.h:745`

—

internal_rx_log_error

```
static void internal_rx_log_error(const char *tag, const char *message)
```

Log ERROR-level message without additional value.

Outputs critical error message with standardized header format. Used for operation-preventing failures that require immediate attention.

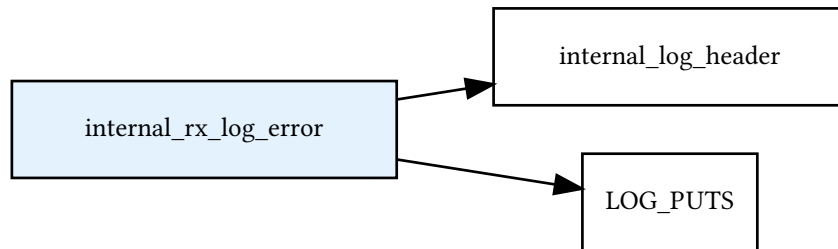


Figure 184: Call graph for `internal_rx_log_error`

Defined in `libs/rx_core/inc/rx_log.h:828`

—

internal_rx_log_error_u8

```
static void internal_rx_log_error_u8(const char *tag, const char *message,  
uint8_t value)
```

Log error with `uint8_t` value.

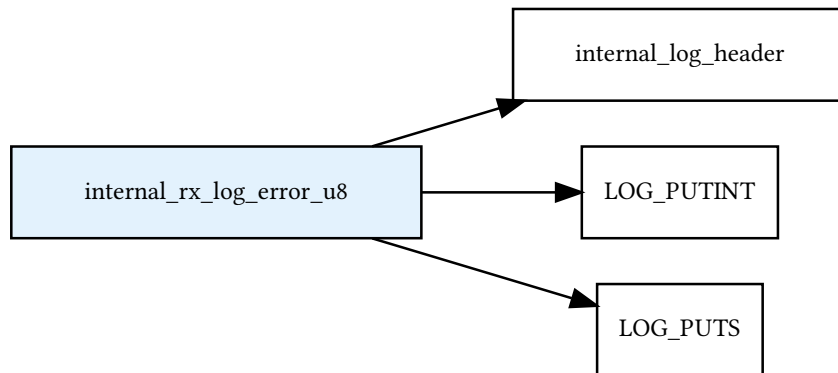


Figure 185: Call graph for `internal_rx_log_error_u8`

Defined in `libs/rx_core/inc/rx_log.h:842`

—

internal_rx_log_error_u16

```
static void internal_rx_log_error_u16(const char *tag, const char *message,  
uint16_t value)
```

Log error with uint16_t value.

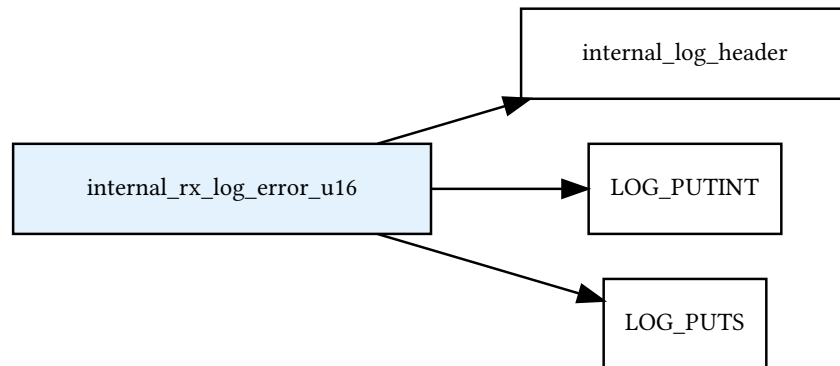


Figure 186: Call graph for `internal_rx_log_error_u16`

Defined in `libs/rx_core/inc/rx_log.h:854`

—

internal_rx_log_error_u32

```
static void internal_rx_log_error_u32(const char *tag, const char *message,  
uint32_t value)
```

Log error with uint32_t value.

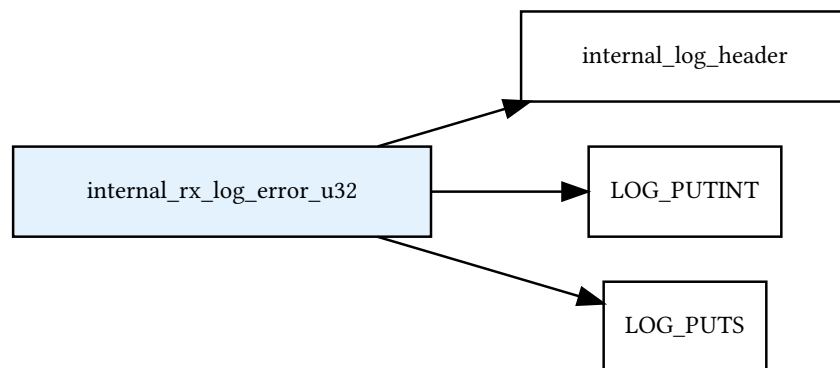


Figure 187: Call graph for `internal_rx_log_error_u32`

Defined in `libs/rx_core/inc/rx_log.h:866`

—

internal_rx_log_error_i32

```
static void internal_rx_log_error_i32(const char *tag, const char *message,  
int32_t value)
```

Log error with int32_t value.

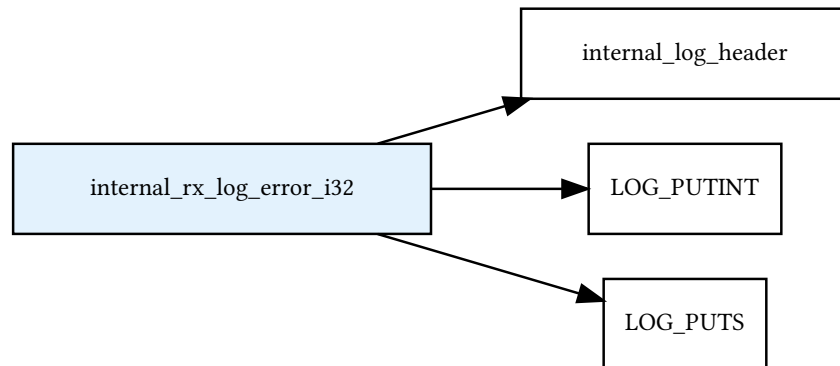


Figure 188: Call graph for `internal_rx_log_error_i32`

Defined in `libs/rx_core/inc/rx_log.h:878`

—

internal_rx_log_error_err

```
static void internal_rx_log_error_err(const char *tag, const char *message,  
rx_err_t err)
```

Log error with error code (`rx_err_t`).

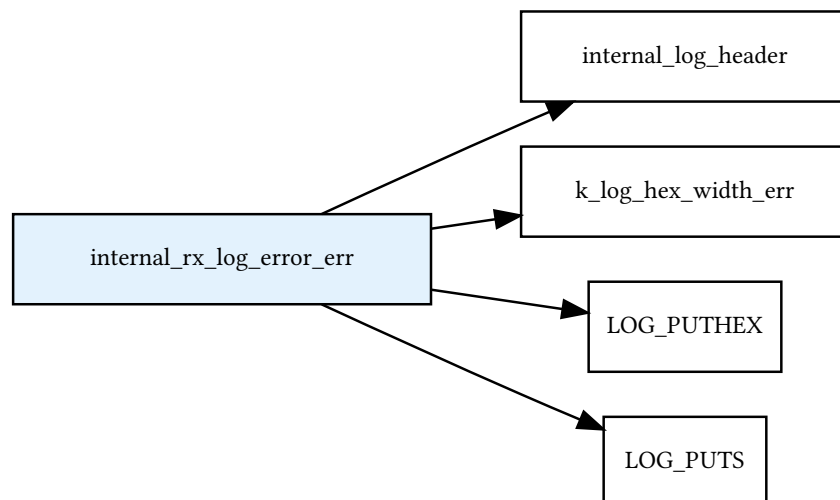


Figure 189: Call graph for `internal_rx_log_error_err`

Defined in *libs/rx_core/inc/rx_log.h:890*

—

internal_rx_log_error_hex

```
static void internal_rx_log_error_hex(const char *tag, const char *message,  
uint32_t value, uint8_t digits)
```

Log error with hex value.

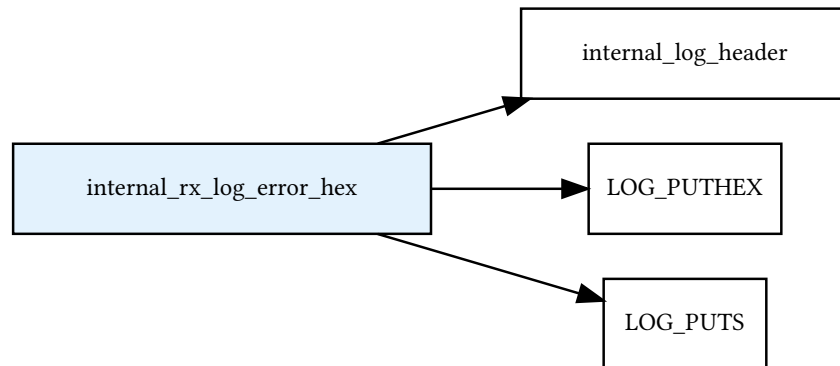


Figure 190: Call graph for `internal_rx_log_error_hex`

Defined in *libs/rx_core/inc/rx_log.h:903*

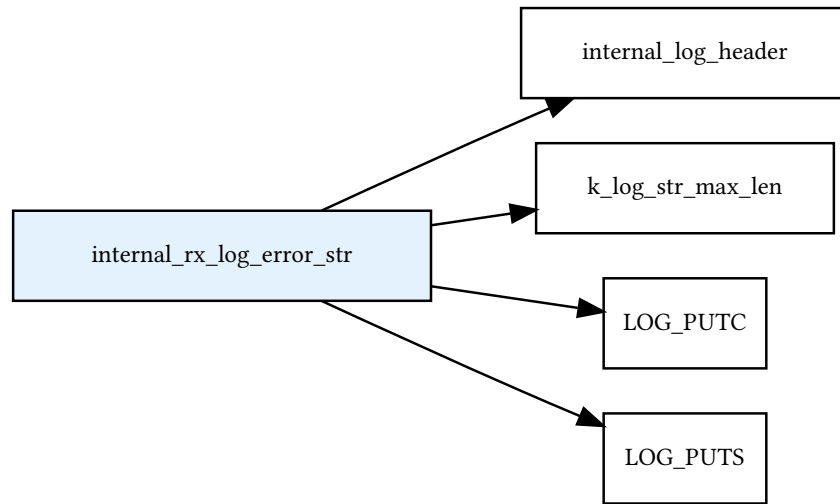
—

internal_rx_log_error_str

```
static void internal_rx_log_error_str(const char *tag, const char *message, const  
char *str_value, uint32_t len)
```

Log ERROR-level message with bounded string value (NASA-compliant).

Outputs critical error message with associated string value using bounded iteration for memory safety. String output is limited to prevent unbounded loops and buffer overruns (NASA Power of 10 Rule 2 compliance).

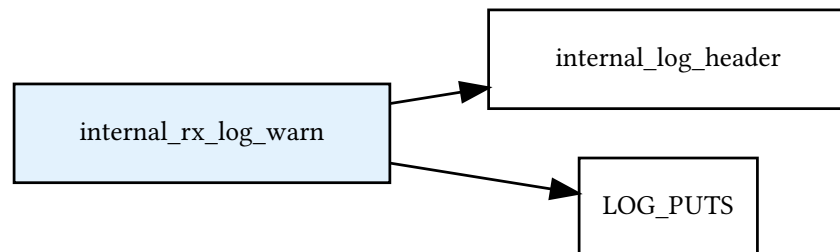
Figure 191: Call graph for `internal_rx_log_error_str`

Defined in `libs/rx_core/inc/rx_log.h:1010`

internal_rx_log_warn

```
static void internal_rx_log_warn(const char *tag, const char *message)
```

Log warning message (no value).

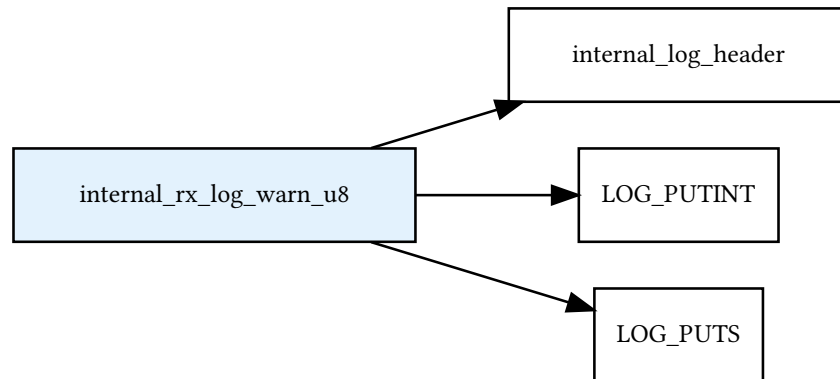
Figure 192: Call graph for `internal_rx_log_warn`

Defined in `libs/rx_core/inc/rx_log.h:1043`

internal_rx_log_warn_u8

```
static void internal_rx_log_warn_u8(const char *tag, const char *message, uint8_t value)
```

Log warning with `uint8_t` value.

Figure 193: Call graph for `internal_rx_log_warn_u8`

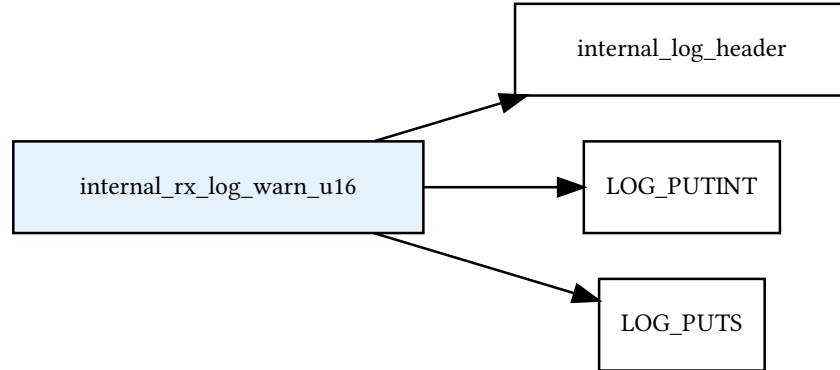
Defined in `libs/rx_core/inc/rx_log.h:1057`

—

`internal_rx_log_warn_u16`

```
static void internal_rx_log_warn_u16(const char *tag, const char *message,  
uint16_t value)
```

Log warning with `uint16_t` value.

Figure 194: Call graph for `internal_rx_log_warn_u16`

Defined in `libs/rx_core/inc/rx_log.h:1069`

—

`internal_rx_log_warn_u32`

```
static void internal_rx_log_warn_u32(const char *tag, const char *message,  
uint32_t value)
```

Log warning with `uint32_t` value.

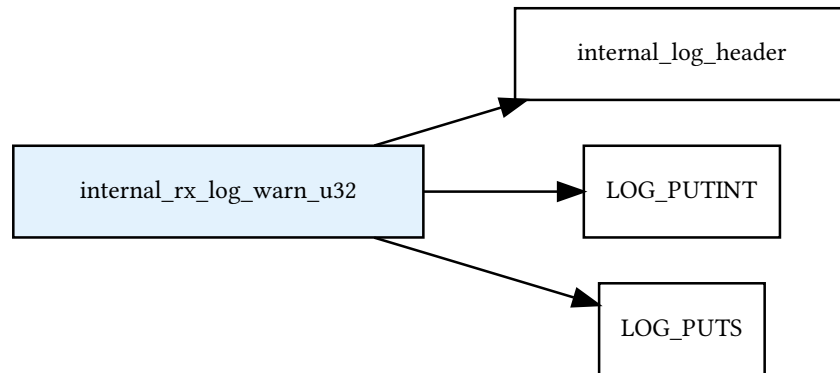


Figure 195: Call graph for `internal_rx_log_warn_u32`

Defined in `libs/rx_core/inc/rx_log.h:1081`

—

internal_rx_log_warn_i32

```
static void internal_rx_log_warn_i32(const char *tag, const char *message,
int32_t value)
```

Log warning with `int32_t` value.

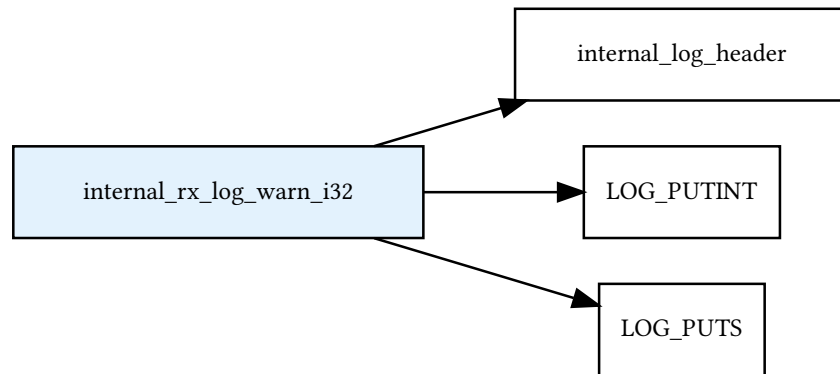


Figure 196: Call graph for `internal_rx_log_warn_i32`

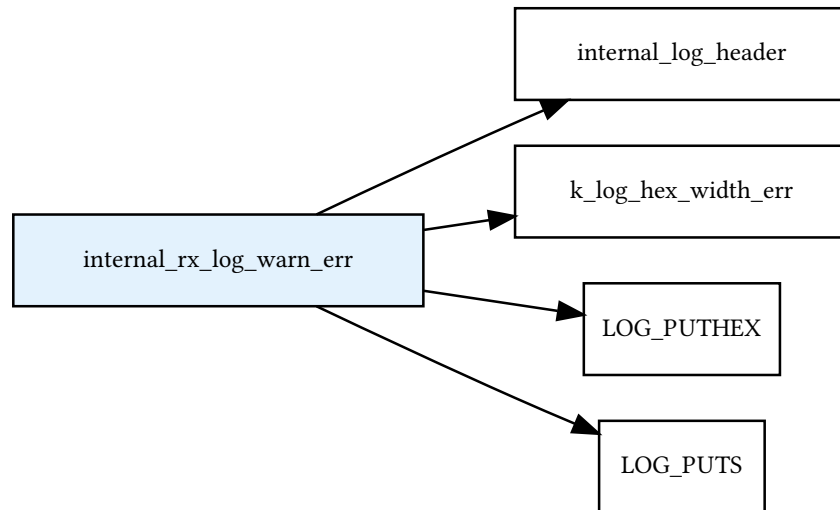
Defined in `libs/rx_core/inc/rx_log.h:1093`

—

internal_rx_log_warn_err

```
static void internal_rx_log_warn_err(const char *tag, const char *message,
rx_err_t err)
```

Log warning with error code (`rx_err_t`).

Figure 197: Call graph for `internal_rx_log_warn_err`

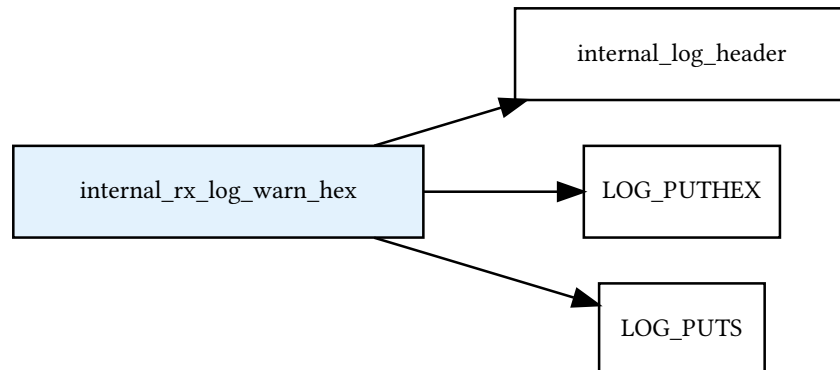
Defined in `libs/rx_core/inc/rx_log.h:1105`

—

`internal_rx_log_warn_hex`

```
static void internal_rx_log_warn_hex(const char *tag, const char *message,  
uint32_t value, uint8_t digits)
```

Log warning with hex value.

Figure 198: Call graph for `internal_rx_log_warn_hex`

Defined in `libs/rx_core/inc/rx_log.h:1118`

—

internal_rx_log_warn_str

```
static void internal_rx_log_warn_str(const char *tag, const char *message, const  
char *str_value, uint32_t len)
```

Log warning with string value (bounded length).

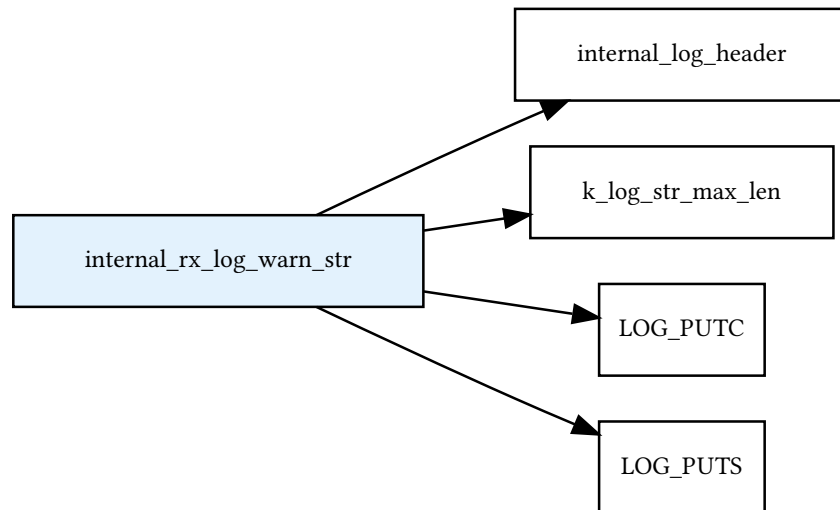


Figure 199: Call graph for internal_rx_log_warn_str

Defined in `libs/rx_core/inc/rx_log.h:1131`

—

internal_rx_log_info

```
static void internal_rx_log_info(const char *tag, const char *message)
```

Log info message (no value).

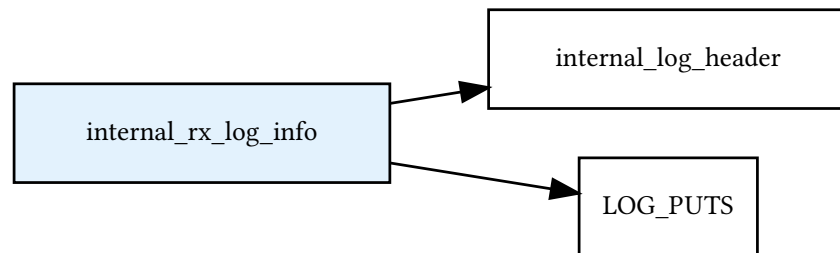


Figure 200: Call graph for internal_rx_log_info

Defined in `libs/rx_core/inc/rx_log.h:1164`

—

internal_rx_log_info_u8

```
static void internal_rx_log_info_u8(const char *tag, const char *message, uint8_t value)
```

Log info with uint8_t value.

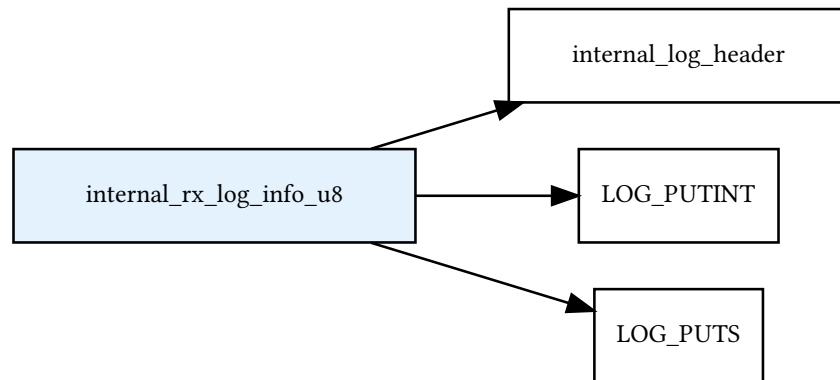


Figure 201: Call graph for `internal_rx_log_info_u8`

Defined in `libs/rx_core/inc/rx_log.h:1178`

—

internal_rx_log_info_u16

```
static void internal_rx_log_info_u16(const char *tag, const char *message, uint16_t value)
```

Log info with uint16_t value.

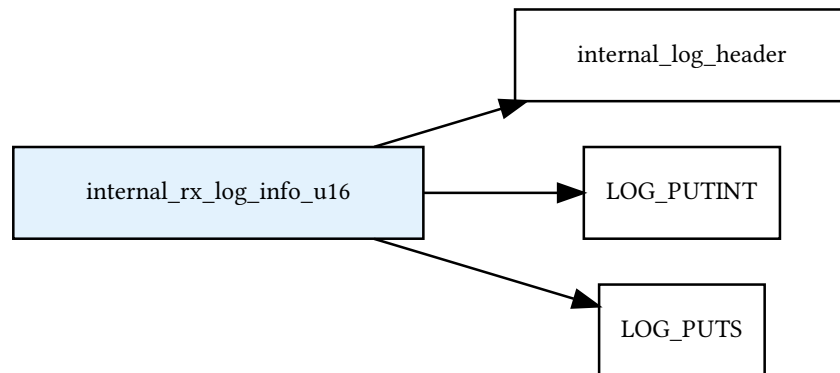


Figure 202: Call graph for `internal_rx_log_info_u16`

Defined in `libs/rx_core/inc/rx_log.h:1190`

—

internal_rx_log_info_u32

```
static void internal_rx_log_info_u32(const char *tag, const char *message,  
uint32_t value)
```

Log info with uint32_t value.

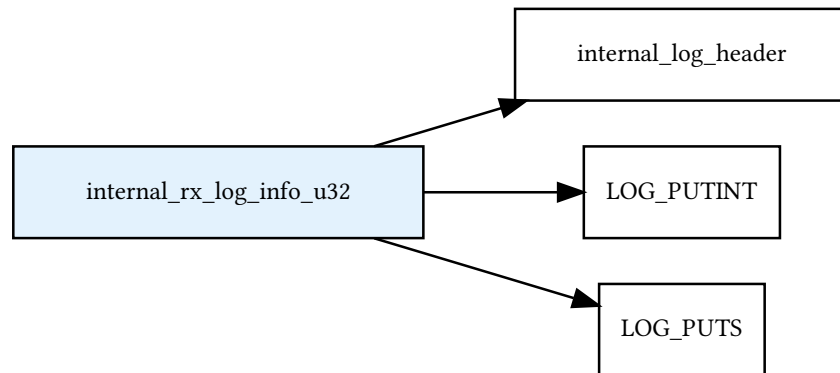


Figure 203: Call graph for `internal_rx_log_info_u32`

Defined in `libs/rx_core/inc/rx_log.h:1202`

—

internal_rx_log_info_i32

```
static void internal_rx_log_info_i32(const char *tag, const char *message,  
int32_t value)
```

Log info with int32_t value.

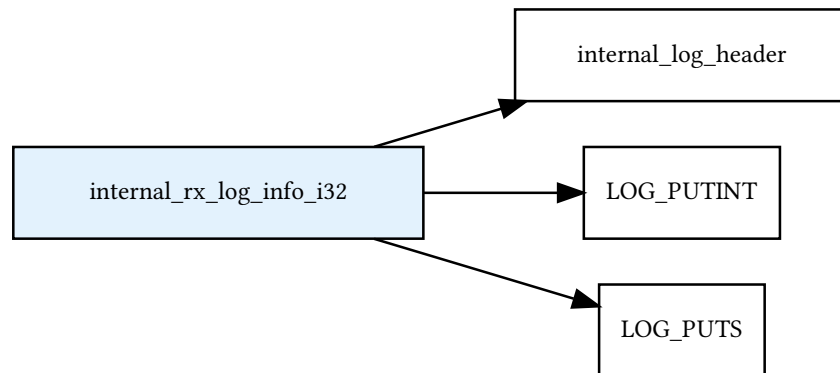


Figure 204: Call graph for `internal_rx_log_info_i32`

Defined in `libs/rx_core/inc/rx_log.h:1214`

—

internal_rx_log_info_err

```
static void internal_rx_log_info_err(const char *tag, const char *message,  
rx_err_t err)
```

Log info with error code (rx_err_t).

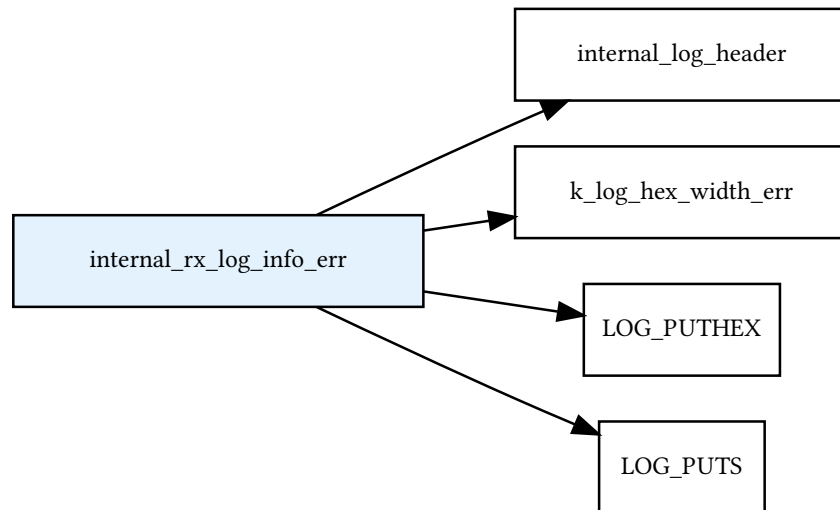


Figure 205: Call graph for internal_rx_log_info_err

Defined in `libs/rx_core/inc/rx_log.h:1226`

internal_rx_log_info_hex

```
static void internal_rx_log_info_hex(const char *tag, const char *message,  
uint32_t value, uint8_t digits)
```

Log info with hex value.

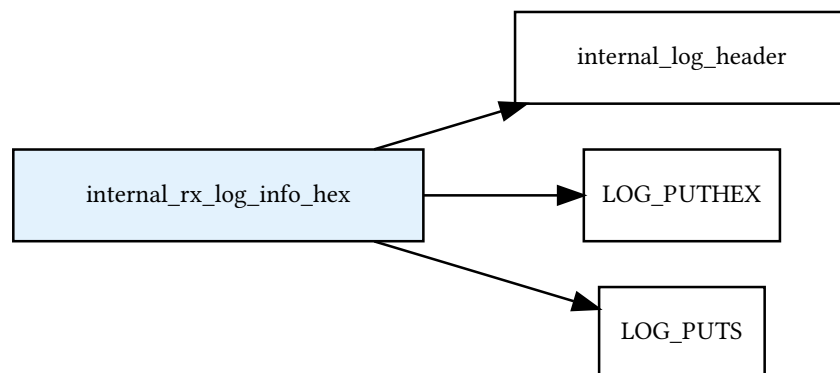


Figure 206: Call graph for internal_rx_log_info_hex

Defined in `libs/rx_core/inc/rx_log.h:1239`

—

internal_rx_log_info_str

```
static void internal_rx_log_info_str(const char *tag, const char *message, const
char *str_value, uint32_t len)
```

Log info with string value (bounded length).

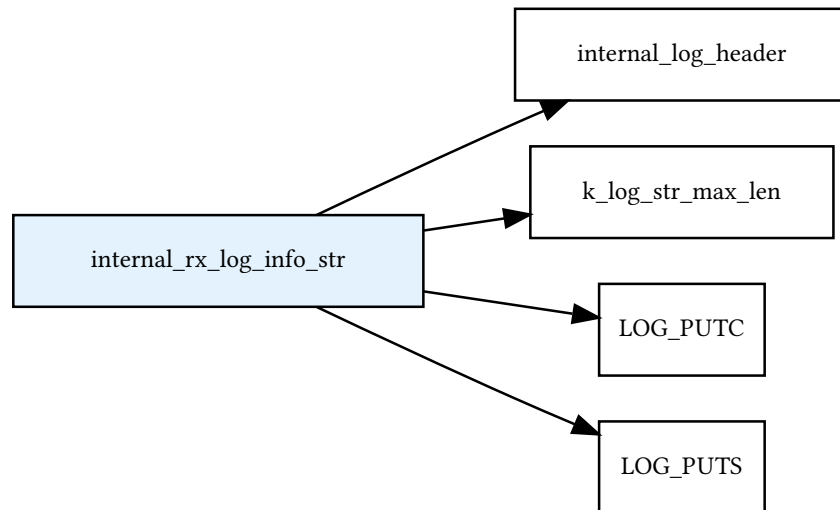


Figure 207: Call graph for `internal_rx_log_info_str`

Defined in `libs/rx_core/inc/rx_log.h:1252`

—

internal_rx_log_debug

```
static void internal_rx_log_debug(const char *tag, const char *message)
```

Log debug message (no value).

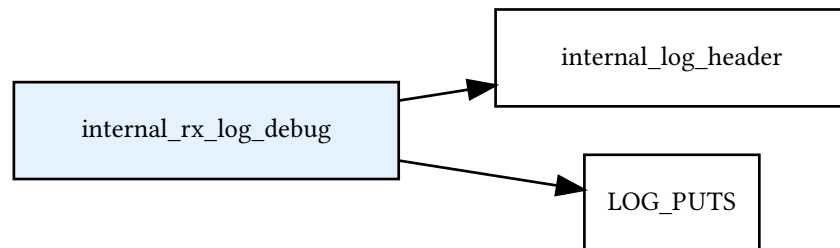


Figure 208: Call graph for `internal_rx_log_debug`

Defined in `libs/rx_core/inc/rx_log.h:1285`

internal_rx_log_debug_u8

```
static void internal_rx_log_debug_u8(const char *tag, const char *message,  
uint8_t value)
```

Log debug with uint8_t value.

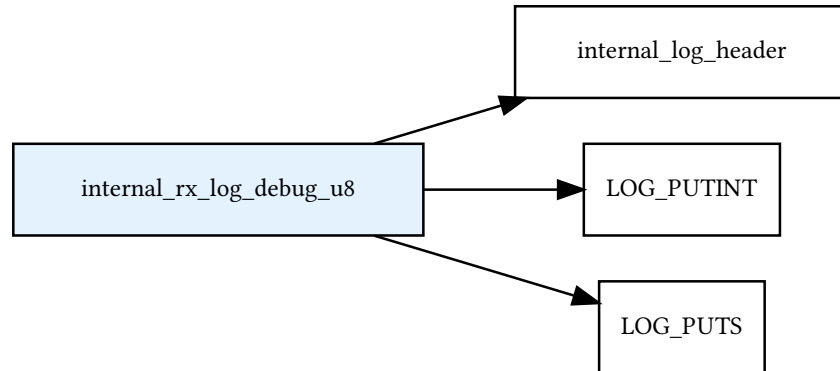


Figure 209: Call graph for `internal_rx_log_debug_u8`

Defined in `libs/rx_core/inc/rx_log.h:1299`

internal_rx_log_debug_u16

```
static void internal_rx_log_debug_u16(const char *tag, const char *message,  
uint16_t value)
```

Log debug with uint16_t value.

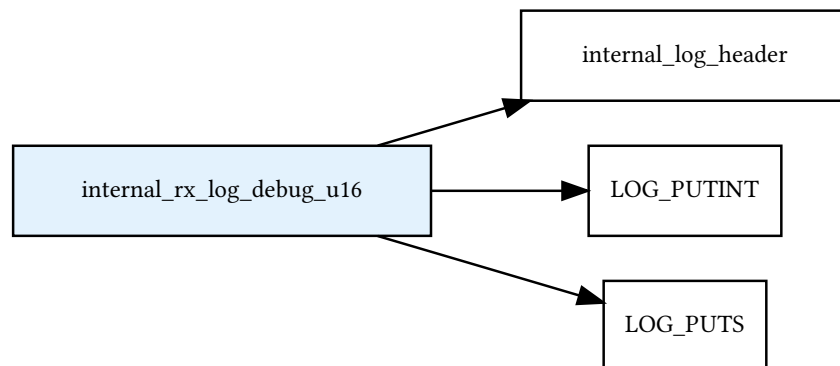


Figure 210: Call graph for `internal_rx_log_debug_u16`

Defined in `libs/rx_core/inc/rx_log.h:1311`

internal_rx_log_debug_u32

```
static void internal_rx_log_debug_u32(const char *tag, const char *message,  
uint32_t value)
```

Log debug with uint32_t value.

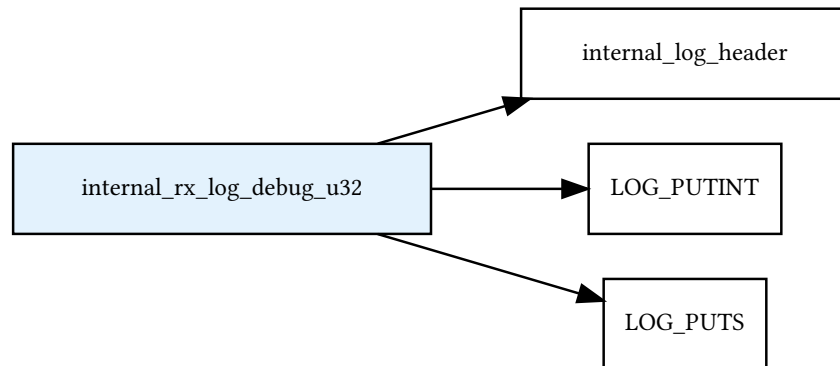


Figure 211: Call graph for `internal_rx_log_debug_u32`

Defined in `libs/rx_core/inc/rx_log.h:1323`

—

internal_rx_log_debug_i32

```
static void internal_rx_log_debug_i32(const char *tag, const char *message,  
int32_t value)
```

Log debug with int32_t value.

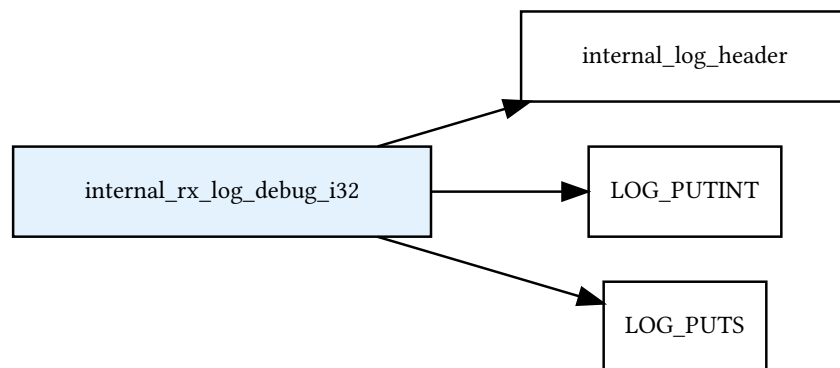


Figure 212: Call graph for `internal_rx_log_debug_i32`

Defined in `libs/rx_core/inc/rx_log.h:1335`

—

internal_rx_log_debug_err

```
static void internal_rx_log_debug_err(const char *tag, const char *message,  
rx_err_t err)
```

Log debug with error code (rx_err_t).

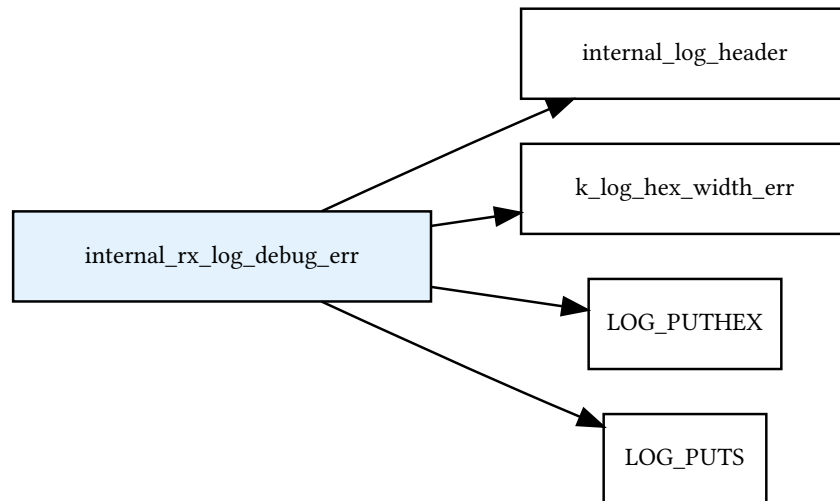


Figure 213: Call graph for `internal_rx_log_debug_err`

Defined in `libs/rx_core/inc/rx_log.h:1347`

—

internal_rx_log_debug_hex

```
static void internal_rx_log_debug_hex(const char *tag, const char *message,  
uint32_t value, uint8_t digits)
```

Log debug with hex value.

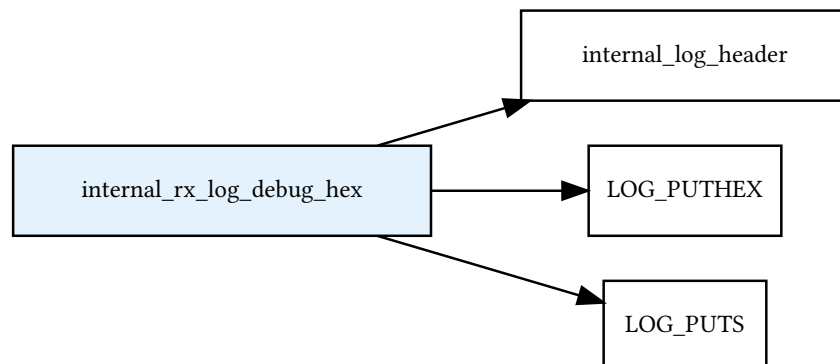


Figure 214: Call graph for `internal_rx_log_debug_hex`

Defined in `libs/rx_core/inc/rx_log.h:1360`

—

internal_rx_log_debug_str

```
static void internal_rx_log_debug_str(const char *tag, const char *message, const  
char *str_value, uint32_t len)
```

Log debug with string value (bounded length).

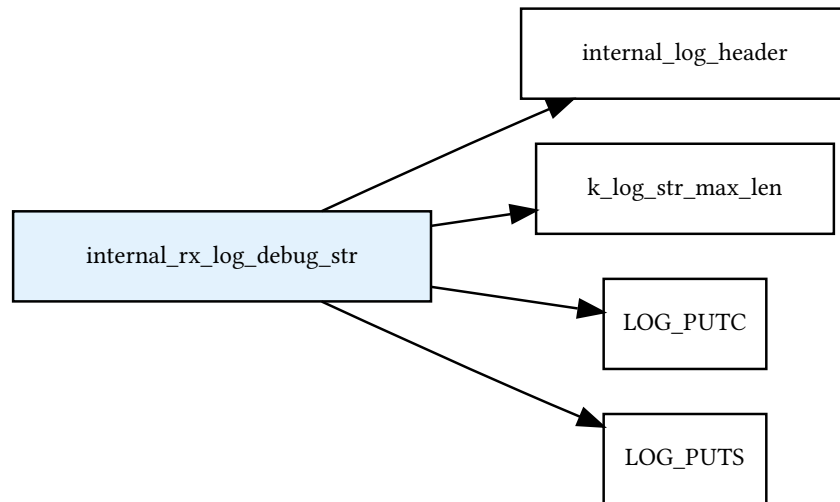


Figure 215: Call graph for `internal_rx_log_debug_str`

Defined in `libs/rx_core/inc/rx_log.h:1373`

—

internal_rx_log_verbose

```
static void internal_rx_log_verbose(const char *tag, const char *message)
```

Log verbose message (no value).

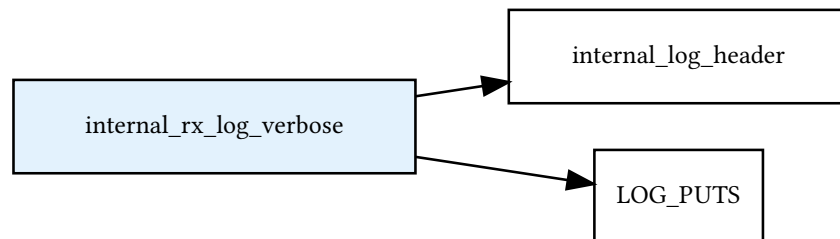


Figure 216: Call graph for `internal_rx_log_verbose`

Defined in `libs/rx_core/inc/rx_log.h:1406`

internal_rx_log_verbose_u8

```
static void internal_rx_log_verbose_u8(const char *tag, const char *message,  
uint8_t value)
```

Log verbose with uint8_t value.

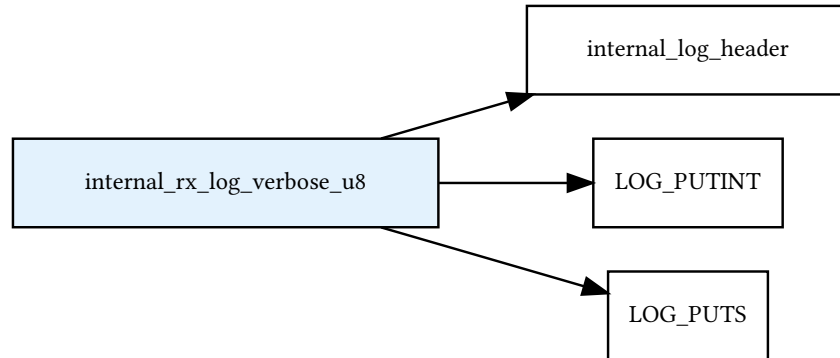


Figure 217: Call graph for `internal_rx_log_verbose_u8`

Defined in `libs/rx_core/inc/rx_log.h:1420`

internal_rx_log_verbose_u16

```
static void internal_rx_log_verbose_u16(const char *tag, const char *message,  
uint16_t value)
```

Log verbose with uint16_t value.

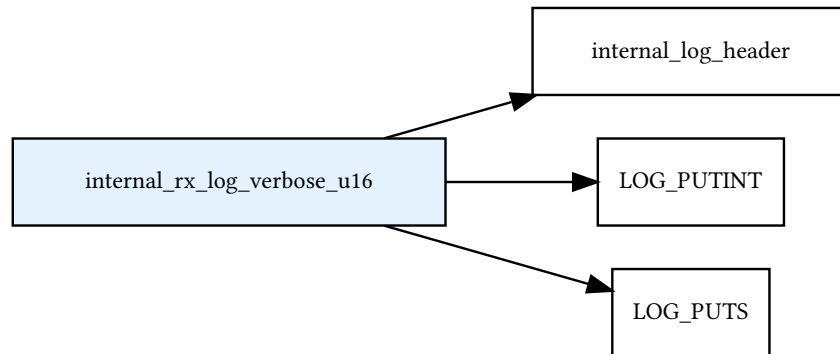


Figure 218: Call graph for `internal_rx_log_verbose_u16`

Defined in `libs/rx_core/inc/rx_log.h:1432`

internal_rx_log_verbose_u32

```
static void internal_rx_log_verbose_u32(const char *tag, const char *message,  
uint32_t value)
```

Log verbose with uint32_t value.

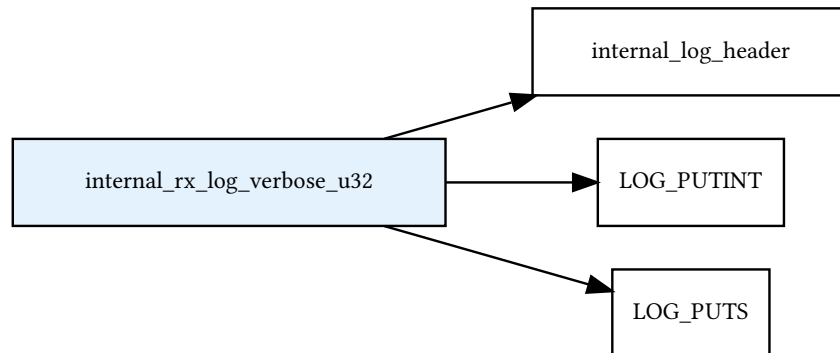


Figure 219: Call graph for `internal_rx_log_verbose_u32`

Defined in `libs/rx_core/inc/rx_log.h:1444`

—

internal_rx_log_verbose_i32

```
static void internal_rx_log_verbose_i32(const char *tag, const char *message,  
int32_t value)
```

Log verbose with int32_t value.

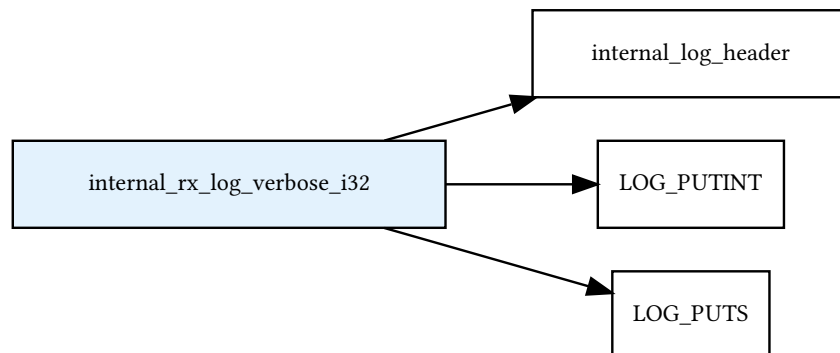


Figure 220: Call graph for `internal_rx_log_verbose_i32`

Defined in `libs/rx_core/inc/rx_log.h:1456`

—

internal_rx_log_verbose_err

```
static void internal_rx_log_verbose_err(const char *tag, const char *message,  
rx_err_t err)
```

Log verbose with error code (rx_err_t).

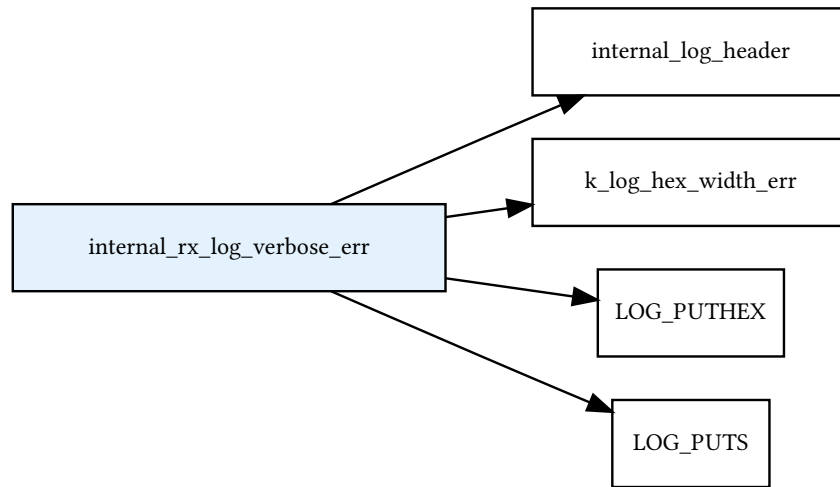


Figure 221: Call graph for `internal_rx_log_verbose_err`

Defined in `libs/rx_core/inc/rx_log.h:1468`

—

internal_rx_log_verbose_hex

```
static void internal_rx_log_verbose_hex(const char *tag, const char *message,  
uint32_t value, uint8_t digits)
```

Log verbose with hex value.

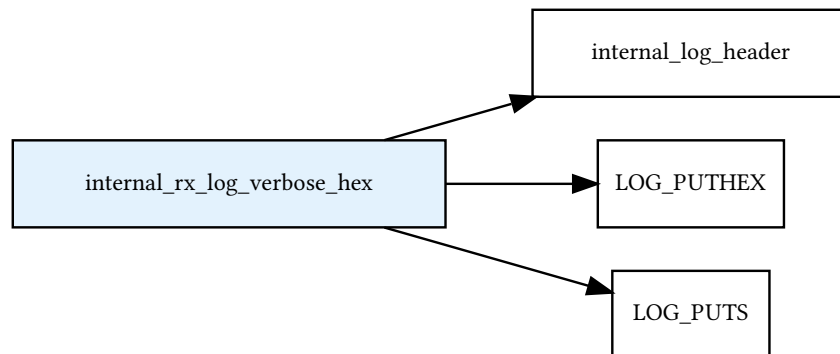


Figure 222: Call graph for `internal_rx_log_verbose_hex`

Defined in `libs/rx_core/inc/rx_log.h:1481`

internal_rx_log_verbose_str

```
static void internal_rx_log_verbose_str(const char *tag, const char *message,
const char *str_value, uint32_t len)
```

Log verbose with string value (bounded length).

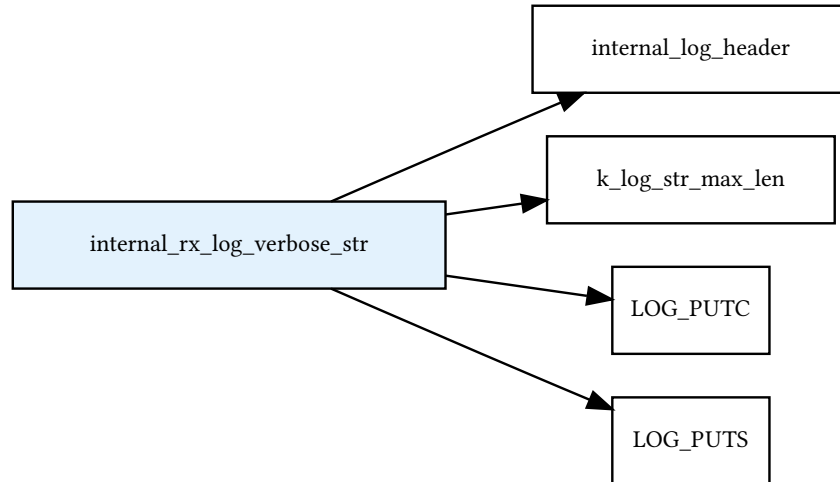


Figure 223: Call graph for `internal_rx_log_verbose_str`

Defined in `libs/rx_core/inc/rx_log.h:1493`

rx_pin_interface.h

Abstract Pin Validator Interface - THE “I” and “D” in SOLID.

Includes:

- `<stdbool.h>`
- `<stddef.h>`
- `<stdint.h>`
- `"rx_err.h"`

pin_interface_limits_t

Pin Interface Configuration Constants.

Defines compile-time constants for the pin interface system. These limits ensure predictable memory usage and prevent buffer overflows in pin function name handling.

Value	Name	Description
= 32	<code>k_pin_function_name_max_len</code>	Maximum length of pin function name including null terminator.

rx_pin_validate_fn

```
typedef rx_err_t(* rx_pin_validate_fn) (void *ctx, const uint8_t port, const uint8_t pin)
```

Validate if a GPIO pin exists and is accessible on the RX72N.

Function pointer type for validating pin existence before reservation or use. Implementations must check both port and pin validity against the RX72N hardware.

Implementation Requirements:

- Check port number is in valid range (0x0-0x9, 0xA-0xG for RX72N)
- Check pin number is in valid range (0-7)
- Verify that specific port/pin combination exists (not all ports have all pins)
- Return appropriate error codes for invalid port vs invalid pin
- Must be thread-safe (multiple threads may validate concurrently)

—

rx_pin_reserve_fn

```
typedef rx_err_t(* rx_pin_reserve_fn) (void *ctx, const uint8_t port, const uint8_t pin, const char *function)
```

Reserve a GPIO pin for exclusive use by a specific function/module.

Function pointer type for reserving pins to prevent conflicts. Once reserved, a pin cannot be reserved again until released. This is the primary conflict prevention mechanism in the STAR firmware.

Implementation Requirements:

- Validate port and pin numbers first (call validate_pin internally)
- Check if pin is already reserved
- Store function name (up to k_pin_function_name_max_len bytes)
- Mark pin as reserved atomically
- Must be thread-safe with mutex protection
- Function name string is not copied - must remain valid

—

rx_pin_release_fn

```
typedef rx_err_t(* rx_pin_release_fn) (void *ctx, const uint8_t port, const uint8_t pin)
```

Release a previously reserved GPIO pin, making it available for reuse.

Function pointer type for releasing pin reservations. After release, the pin becomes available for reservation by other modules. Typically called during module deinitialization or reconfiguration.

Implementation Requirements:

- Validate port and pin numbers first
- Check if pin is currently reserved (error if not)
- Clear reservation flag atomically
- Clear function name storage
- Must be thread-safe with mutex protection
- Idempotent safe: Releasing unreserved pin returns error (doesn't crash)

—

rx_pin_is_reserved_fn

```
typedef bool(* rx_pin_is_reserved_fn) (void *ctx, const uint8_t port, const uint8_t pin)
```

Check if a GPIO pin is currently reserved.

Function pointer type for querying pin reservation status. Returns boolean result without error codes. Useful for defensive checks before attempting reservation or for debugging pin conflicts.

Implementation Requirements:

- Return true if pin is reserved, false if available
- Invalid port/pin should return false (not reserved)
- Must be thread-safe (read-only operation)
- Fast lookup (no blocking operations)

—

rx_pin_get_function_fn

```
typedef rx_err_t(* rx_pin_get_function_fn) (void *ctx, const uint8_t port, const uint8_t pin, char *function_out, const uint32_t function_len)
```

Get the function name associated with a reserved GPIO pin.

Function pointer type for querying which function/module reserved a pin. Useful for debugging pin conflicts and generating pin usage reports.

Implementation Requirements:

- Validate port and pin numbers
- Check if pin is reserved (error if not)
- Copy function name to output buffer (with null terminator)
- Truncate if buffer too small
- Must be thread-safe with mutex protection

—

rx_pin_clear_all_fn

```
typedef rx_err_t(* rx_pin_clear_all_fn) (void *ctx)
```

Clear all pin reservations (reset to initial state).

Function pointer type for clearing all pin reservations at once. Typically used during:

- System reset/reinitialization
- Unit test teardown
- Error recovery (start fresh)
- Development/debugging

Implementation Requirements:

- Clear all reservation flags atomically
- Clear all function name strings
- Reset to clean state (as if just initialized)
- Must be thread-safe with exclusive mutex lock
- WARNING: Dangerous operation - no confirmation

—

rx_pin_interface_validate

```
static rx_err_t rx_pin_interface_validate(const rx_pin_interface_t *iface)
```

Validate that a pin interface is properly initialized and safe to use.

Performs comprehensive validation of an `rx_pin_interface_t` to ensure all function pointers are non-NULL and the interface is ready for use. This function should be called before using any interface received from external sources.

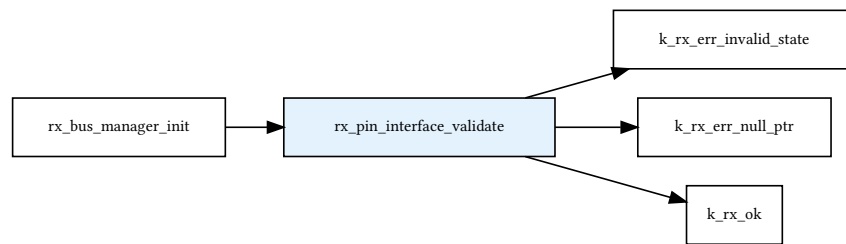


Figure 224: Call graph for `rx_pin_interface_validate`

_Defined in `libs/rx\core/inc/rx\_pin\_interface.h:1221_`

rx_pin_validator.h

Concrete Pin Validator Implementation - Dependency Inversion Pattern.

Includes:

- `<stdbool.h>`
- `<stddef.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_pin_interface.h"`
- `"tx_api.h"`

pin_validator_limits_t

Pin Validator Configuration Constants.

Defines compile-time array dimensions for the pin reservation table. These limits are based on the RX72N hardware capabilities and determine memory footprint.

Value	Name	Description
= 17	<code>k_pin_validator_max_ports</code>	Maximum number of ports on RX72N (0-9, A-G).
= 8	<code>k_pin_validator_max_pins</code>	Maximum pins per port (0-7).

pin_validator_init

```
rx_err_t pin_validator_init(pin_validator_t *validator)
```

Initialize pin validator (MUST be called before use).

Performs one-time initialization of the pin validator structure. Creates ThreadX mutex, clears reservation table, and marks validator as initialized. Must be called exactly once before extracting interface via `pin_validator_get_interface()`.

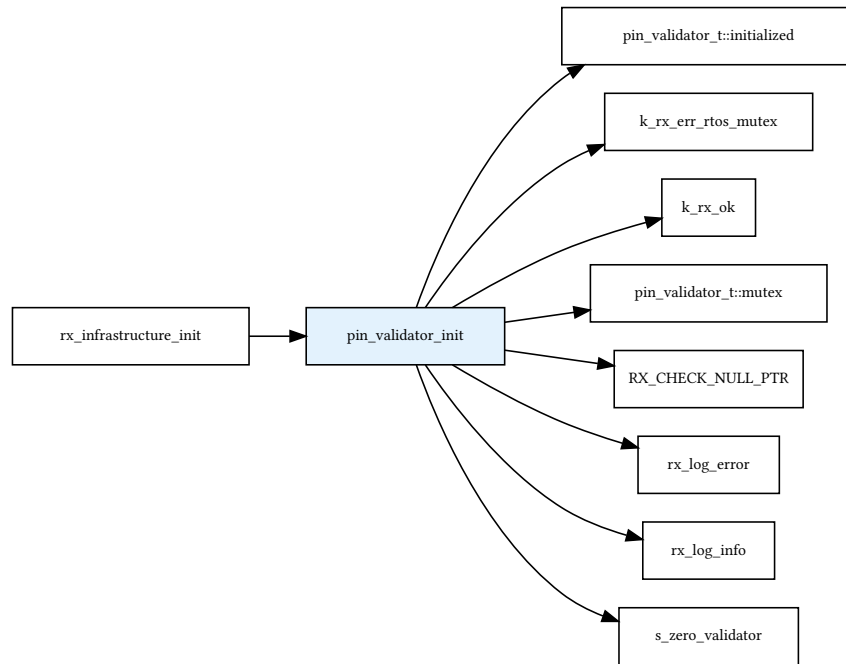


Figure 225: Call graph for `pin_validator_init`

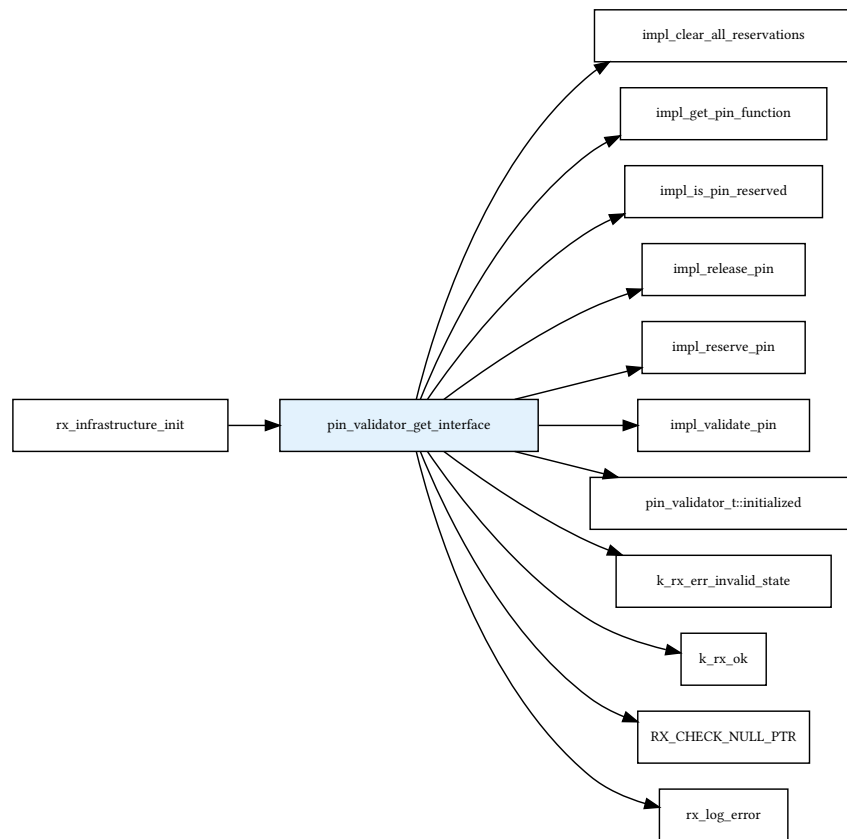
Defined in `libs/rx_core/inc/rx_pin_validator.h:681`

pin_validator_get_interface

```
rx_err_t pin_validator_get_interface(rx_pin_interface_t *iface, pin_validator_t
*validator)
```

Extract abstract interface from concrete pin validator (Dependency Inversion).

Extracts an `rx_pin_interface_t` from the concrete `pin_validator_t` implementation. This function is the key to Dependency Inversion: high-level modules depend on the abstract interface, not the concrete validator type.

Figure 226: Call graph for `pin_validator_get_interface`

_Defined in `libs/rx\core/inc/rx\pin_validator.h:775_`

pin_validator_deinit

```
rx_err_t pin_validator_deinit(pin_validator_t *validator)
```

Deinitialize pin validator (graceful shutdown only).

Performs cleanup of pin validator resources. Destroys ThreadX mutex, clears reservation table, and marks validator as uninitialized. In embedded systems, this is RARELY CALLED because the system typically runs indefinitely.

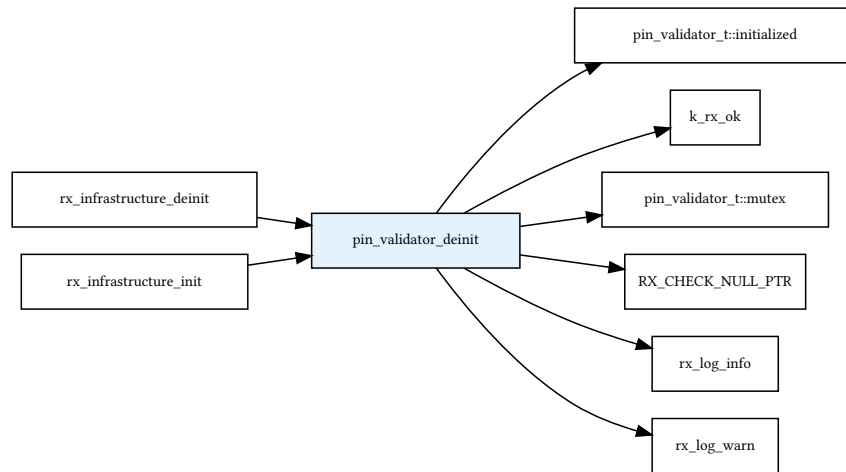


Figure 227: Call graph for pin_validator_deinit

_Defined in libs/rx\core/inc/rx_pin_validator.h:861_

rx_port_constants.h

Centralized RX72N Port Number Definitions.

LIBRARY-LEVEL SINGLE SOURCE OF TRUTH for all port and pin numbers used throughout the STAR RX72N firmware. This file is the ONLY location in the library layer that contains hexadecimal values for port/pin identifiers.

Includes:

- <stdint.h>
- "rx_check.h"

rx_port_number_t

RX72N Port Number Constants (Single-Byte Identifiers).

Defines all GPIO port numbers available on the Renesas RX72N microcontroller. Used by HAL layer for hardware register access and by application layer for constructing full port/pin identifiers.

Value	Name	Description
= 0x00	k_rx_port_0	Port 0 - 6 pins on 144-pin (P00-P03, P05, P07)
= 0x01	k_rx_port_1	Port 1 - 6 pins on 144-pin (P12-P17)
= 0x02	k_rx_port_2	Port 2 - Full 8-pin port on 144-pin (P20-P27)
= 0x03	k_rx_port_3	Port 3 - Full 8-pin port on 144-pin (P30-P37)
= 0x04	k_rx_port_4	Port 4 - Full 8-pin port on 144-pin (P40-P47)
= 0x05	k_rx_port_5	Port 5 - 7 pins on 144-pin (P50-P56)
= 0x06	k_rx_port_6	Port 6 - Full 8-pin port on 144-pin (P60-P67)
= 0x07	k_rx_port_7	Port 7 - Full 8-pin port on 144-pin (P70-P77)

= 0x08	k_rx_port_8	Port 8 - 6 pins on 144-pin (P80-P83, P86-P87)
= 0x09	k_rx_port_9	Port 9 - 4 pins on 144-pin (P90-P93)
= 0x0A	k_rx_port_a	Port A - Full 8-pin port available on 144-pin
= 0x0B	k_rx_port_b	Port B - Full 8-pin port available on 144-pin
= 0x0C	k_rx_port_c	Port C - Full 8-pin port available on 144-pin
= 0x0D	k_rx_port_d	Port D - Full 8-pin port available on 144-pin
= 0x0E	k_rx_port_e	Port E - Full 8-pin port available on 144-pin
= 0x0F	k_rx_port_f	Port F - Partially available on 144-pin (PF5)
= 0x10	k_rx_port_g	Port G - Not available on 144-pin package
= 0x13	k_rx_port_j	Port J - Partial availability on 144-pin (PJ3, PJ5) - NON-CONTIGUOUS (0x11, 0x12 unused)

—

rx_pin_number_t

Pin Number Constants Within Each Port (0-7).

Defines pin numbers within each GPIO port on the RX72N. All RX72N ports are 8-bit wide, meaning each port has pins numbered 0 through 7. These constants provide self-documenting alternatives to raw numeric literals.

Value	Name	Description
= 0	k_rx_pin_0	Pin 0 - LSB of port data register, bit position 0
= 1	k_rx_pin_1	Pin 1 - Bit position 1 in port data register
= 2	k_rx_pin_2	Pin 2 - Bit position 2 in port data register
= 3	k_rx_pin_3	Pin 3 - Bit position 3 in port data register
= 4	k_rx_pin_4	Pin 4 - Bit position 4 in port data register
= 5	k_rx_pin_5	Pin 5 - Bit position 5 in port data register
= 6	k_rx_pin_6	Pin 6 - Bit position 6 in port data register
= 7	k_rx_pin_7	Pin 7 - MSB of port data register, bit position 7
= 0	k_rx_pin_min	Minimum valid pin number - used for range validation in assertions
= 7	k_rx_pin_max	Maximum valid pin number - used for range validation in assertions

—

port_encoding_t

Bit Shift and Mask Constants for Port/Pin Encoding.

Defines the constants used to encode and decode 16-bit gpio_pin_t values. These constants implement the port/pin encoding scheme used throughout the firmware for type-safe GPIO pin identification.

Value	Name	Description
-------	------	-------------

= 8	k_port_shift	Left shift amount to move port number into upper byte (bits 15-8). Used for encoding: <code>gpio_pin_t = (port << k_port_shift) pin</code>
= 0xFF	k_port_mask	Bitmask (0xFF) to extract pin number from lower byte (bits 7-0). Used for decoding: <code>pin = gpio_pin_t & k_port_mask</code>

rx_port_pin_t

Pre-Computed Port/Pin Combinations for All RX72N GPIO Pins.

Provides ready-to-use 16-bit `gpio_pin_t` constants for all 144 theoretical port/pin combinations on the RX72N microcontroller (18 ports x 8 pins = 144). These pre-computed constants eliminate the need for runtime bit-shift operations and provide cleaner, more readable code.

Value	Name	Description
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_0</code>	<code>k_rx_p0_0</code>	P00 (pin 8)
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_1</code>	<code>k_rx_p0_1</code>	P01 (pin 7)
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_2</code>	<code>k_rx_p0_2</code>	P02 (pin 6)
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_3</code>	<code>k_rx_p0_3</code>	P03 (pin 4)
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_4</code>	<code>k_rx_p0_4</code>	P04 - N/A on 144-pin
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_5</code>	<code>k_rx_p0_5</code>	P05 (pin 2)
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_6</code>	<code>k_rx_p0_6</code>	P06 - N/A on 144-pin
<code>= (k_rx_port_0 << k_port_shift) k_rx_pin_7</code>	<code>k_rx_p0_7</code>	P07 (pin 144)
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_0</code>	<code>k_rx_p1_0</code>	P10 - N/A on 144-pin
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_1</code>	<code>k_rx_p1_1</code>	P11 - N/A on 144-pin
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_2</code>	<code>k_rx_p1_2</code>	P12 (pin 45)
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_3</code>	<code>k_rx_p1_3</code>	P13 (pin 44)
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_4</code>	<code>k_rx_p1_4</code>	P14 (pin 43)
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_5</code>	<code>k_rx_p1_5</code>	P15 (pin 42)
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_6</code>	<code>k_rx_p1_6</code>	P16 (pin 40)
<code>= (k_rx_port_1 << k_port_shift) k_rx_pin_7</code>	<code>k_rx_p1_7</code>	P17 (pin 38)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_0</code>	<code>k_rx_p2_0</code>	P20 (pin 37)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_1</code>	<code>k_rx_p2_1</code>	P21 (pin 36)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_2</code>	<code>k_rx_p2_2</code>	P22 (pin 35)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_3</code>	<code>k_rx_p2_3</code>	P23 (pin 34)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_4</code>	<code>k_rx_p2_4</code>	P24 (pin 33)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_5</code>	<code>k_rx_p2_5</code>	P25 (pin 32)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_6</code>	<code>k_rx_p2_6</code>	P26 (pin 31)
<code>= (k_rx_port_2 << k_port_shift) k_rx_pin_7</code>	<code>k_rx_p2_7</code>	P27 (pin 30)
<code>= (k_rx_port_3 << k_port_shift) k_rx_pin_0</code>	<code>k_rx_p3_0</code>	P30 (pin 29)

= (k_rx_port_3 << k_port_shift) k_rx_pin_1	k_rx_p3_1	P31 (pin 28)
= (k_rx_port_3 << k_port_shift) k_rx_pin_2	k_rx_p3_2	P32 (pin 27)
= (k_rx_port_3 << k_port_shift) k_rx_pin_3	k_rx_p3_3	P33 (pin 26)
= (k_rx_port_3 << k_port_shift) k_rx_pin_4	k_rx_p3_4	P34 (pin 25)
= (k_rx_port_3 << k_port_shift) k_rx_pin_5	k_rx_p3_5	P35 (pin 24)
= (k_rx_port_3 << k_port_shift) k_rx_pin_6	k_rx_p3_6	P36 (pin 22)
= (k_rx_port_3 << k_port_shift) k_rx_pin_7	k_rx_p3_7	P37 (pin 20)
= (k_rx_port_4 << k_port_shift) k_rx_pin_0	k_rx_p4_0	P40 (pin 141)
= (k_rx_port_4 << k_port_shift) k_rx_pin_1	k_rx_p4_1	P41 (pin 139)
= (k_rx_port_4 << k_port_shift) k_rx_pin_2	k_rx_p4_2	P42 (pin 138)
= (k_rx_port_4 << k_port_shift) k_rx_pin_3	k_rx_p4_3	P43 (pin 137)
= (k_rx_port_4 << k_port_shift) k_rx_pin_4	k_rx_p4_4	P44 (pin 136)
= (k_rx_port_4 << k_port_shift) k_rx_pin_5	k_rx_p4_5	P45 (pin 135)
= (k_rx_port_4 << k_port_shift) k_rx_pin_6	k_rx_p4_6	P46 (pin 134)
= (k_rx_port_4 << k_port_shift) k_rx_pin_7	k_rx_p4_7	P47 (pin 133)
= (k_rx_port_5 << k_port_shift) k_rx_pin_0	k_rx_p5_0	P50 (pin 56)
= (k_rx_port_5 << k_port_shift) k_rx_pin_1	k_rx_p5_1	P51 (pin 55)
= (k_rx_port_5 << k_port_shift) k_rx_pin_2	k_rx_p5_2	P52 (pin 54)
= (k_rx_port_5 << k_port_shift) k_rx_pin_3	k_rx_p5_3	P53 (pin 53)
= (k_rx_port_5 << k_port_shift) k_rx_pin_4	k_rx_p5_4	P54 (pin 52)
= (k_rx_port_5 << k_port_shift) k_rx_pin_5	k_rx_p5_5	P55 (pin 51)
= (k_rx_port_5 << k_port_shift) k_rx_pin_6	k_rx_p5_6	P56 (pin 50)
= (k_rx_port_5 << k_port_shift) k_rx_pin_7	k_rx_p5_7	P57 - N/A on 144-pin
= (k_rx_port_6 << k_port_shift) k_rx_pin_0	k_rx_p6_0	P60 (pin 117)
= (k_rx_port_6 << k_port_shift) k_rx_pin_1	k_rx_p6_1	P61 (pin 115)
= (k_rx_port_6 << k_port_shift) k_rx_pin_2	k_rx_p6_2	P62 (pin 114)
= (k_rx_port_6 << k_port_shift) k_rx_pin_3	k_rx_p6_3	P63 (pin 113)
= (k_rx_port_6 << k_port_shift) k_rx_pin_4	k_rx_p6_4	P64 (pin 112)
= (k_rx_port_6 << k_port_shift) k_rx_pin_5	k_rx_p6_5	P65 (pin 100)
= (k_rx_port_6 << k_port_shift) k_rx_pin_6	k_rx_p6_6	P66 (pin 99)
= (k_rx_port_6 << k_port_shift) k_rx_pin_7	k_rx_p6_7	P67 (pin 98)
= (k_rx_port_7 << k_port_shift) k_rx_pin_0	k_rx_p7_0	P70 (pin 104)
= (k_rx_port_7 << k_port_shift) k_rx_pin_1	k_rx_p7_1	P71 (pin 86)
= (k_rx_port_7 << k_port_shift) k_rx_pin_2	k_rx_p7_2	P72 (pin 85)
= (k_rx_port_7 << k_port_shift) k_rx_pin_3	k_rx_p7_3	P73 (pin 77)
= (k_rx_port_7 << k_port_shift) k_rx_pin_4	k_rx_p7_4	P74 (pin 72)

= (k_rx_port_7 << k_port_shift) k_rx_pin_5	k_rx_p7_5	P75 (pin 71)
= (k_rx_port_7 << k_port_shift) k_rx_pin_6	k_rx_p7_6	P76 (pin 69)
= (k_rx_port_7 << k_port_shift) k_rx_pin_7	k_rx_p7_7	P77 (pin 68)
= (k_rx_port_8 << k_port_shift) k_rx_pin_0	k_rx_p8_0	P80 (pin 65)
= (k_rx_port_8 << k_port_shift) k_rx_pin_1	k_rx_p8_1	P81 (pin 64)
= (k_rx_port_8 << k_port_shift) k_rx_pin_2	k_rx_p8_2	P82 (pin 63)
= (k_rx_port_8 << k_port_shift) k_rx_pin_3	k_rx_p8_3	P83 (pin 58)
= (k_rx_port_8 << k_port_shift) k_rx_pin_4	k_rx_p8_4	P84 - N/A on 144-pin
= (k_rx_port_8 << k_port_shift) k_rx_pin_5	k_rx_p8_5	P85 - N/A on 144-pin
= (k_rx_port_8 << k_port_shift) k_rx_pin_6	k_rx_p8_6	P86 (pin 41)
= (k_rx_port_8 << k_port_shift) k_rx_pin_7	k_rx_p8_7	P87 (pin 39)
= (k_rx_port_9 << k_port_shift) k_rx_pin_0	k_rx_p9_0	P90 (pin 131)
= (k_rx_port_9 << k_port_shift) k_rx_pin_1	k_rx_p9_1	P91 (pin 129)
= (k_rx_port_9 << k_port_shift) k_rx_pin_2	k_rx_p9_2	P92 (pin 128)
= (k_rx_port_9 << k_port_shift) k_rx_pin_3	k_rx_p9_3	P93 (pin 127)
= (k_rx_port_9 << k_port_shift) k_rx_pin_4	k_rx_p9_4	P94 - N/A on 144-pin
= (k_rx_port_9 << k_port_shift) k_rx_pin_5	k_rx_p9_5	P95 - N/A on 144-pin
= (k_rx_port_9 << k_port_shift) k_rx_pin_6	k_rx_p9_6	P96 - N/A on 144-pin
= (k_rx_port_9 << k_port_shift) k_rx_pin_7	k_rx_p9_7	P97 - N/A on 144-pin
= (k_rx_port_a << k_port_shift) k_rx_pin_0	k_rx_pa_0	PA0 (pin 97)
= (k_rx_port_a << k_port_shift) k_rx_pin_1	k_rx_pa_1	PA1 (pin 96)
= (k_rx_port_a << k_port_shift) k_rx_pin_2	k_rx_pa_2	PA2 (pin 95)
= (k_rx_port_a << k_port_shift) k_rx_pin_3	k_rx_pa_3	PA3 (pin 94)
= (k_rx_port_a << k_port_shift) k_rx_pin_4	k_rx_pa_4	PA4 (pin 92)
= (k_rx_port_a << k_port_shift) k_rx_pin_5	k_rx_pa_5	PA5 (pin 90)
= (k_rx_port_a << k_port_shift) k_rx_pin_6	k_rx_pa_6	PA6 (pin 89)
= (k_rx_port_a << k_port_shift) k_rx_pin_7	k_rx_pa_7	PA7 (pin 88)
= (k_rx_port_b << k_port_shift) k_rx_pin_0	k_rx_pb_0	PB0 (pin 87)
= (k_rx_port_b << k_port_shift) k_rx_pin_1	k_rx_pb_1	PB1 (pin 84)
= (k_rx_port_b << k_port_shift) k_rx_pin_2	k_rx_pb_2	PB2 (pin 83)
= (k_rx_port_b << k_port_shift) k_rx_pin_3	k_rx_pb_3	PB3 (pin 82)
= (k_rx_port_b << k_port_shift) k_rx_pin_4	k_rx_pb_4	PB4 (pin 81)
= (k_rx_port_b << k_port_shift) k_rx_pin_5	k_rx_pb_5	PB5 (pin 80)
= (k_rx_port_b << k_port_shift) k_rx_pin_6	k_rx_pb_6	PB6 (pin 79)
= (k_rx_port_b << k_port_shift) k_rx_pin_7	k_rx_pb_7	PB7 (pin 78)
= (k_rx_port_c << k_port_shift) k_rx_pin_0	k_rx_pc_0	PC0 (pin 75)

= (k_rx_port_c << k_port_shift) k_rx_pin_1	k_rx_pc_1	PC1 (pin 73)
= (k_rx_port_c << k_port_shift) k_rx_pin_2	k_rx_pc_2	PC2 (pin 70)
= (k_rx_port_c << k_port_shift) k_rx_pin_3	k_rx_pc_3	PC3 (pin 67)
= (k_rx_port_c << k_port_shift) k_rx_pin_4	k_rx_pc_4	PC4 (pin 66)
= (k_rx_port_c << k_port_shift) k_rx_pin_5	k_rx_pc_5	PC5 (pin 62)
= (k_rx_port_c << k_port_shift) k_rx_pin_6	k_rx_pc_6	PC6 (pin 61)
= (k_rx_port_c << k_port_shift) k_rx_pin_7	k_rx_pc_7	PC7 (pin 60)
= (k_rx_port_d << k_port_shift) k_rx_pin_0	k_rx_pd_0	PD0 (pin 126)
= (k_rx_port_d << k_port_shift) k_rx_pin_1	k_rx_pd_1	PD1 (pin 125)
= (k_rx_port_d << k_port_shift) k_rx_pin_2	k_rx_pd_2	PD2 (pin 124)
= (k_rx_port_d << k_port_shift) k_rx_pin_3	k_rx_pd_3	PD3 (pin 123)
= (k_rx_port_d << k_port_shift) k_rx_pin_4	k_rx_pd_4	PD4 (pin 122)
= (k_rx_port_d << k_port_shift) k_rx_pin_5	k_rx_pd_5	PD5 (pin 121)
= (k_rx_port_d << k_port_shift) k_rx_pin_6	k_rx_pd_6	PD6 (pin 120)
= (k_rx_port_d << k_port_shift) k_rx_pin_7	k_rx_pd_7	PD7 (pin 119)
= (k_rx_port_e << k_port_shift) k_rx_pin_0	k_rx_pe_0	PE0 (pin 111)
= (k_rx_port_e << k_port_shift) k_rx_pin_1	k_rx_pe_1	PE1 (pin 110)
= (k_rx_port_e << k_port_shift) k_rx_pin_2	k_rx_pe_2	PE2 (pin 109)
= (k_rx_port_e << k_port_shift) k_rx_pin_3	k_rx_pe_3	PE3 (pin 108)
= (k_rx_port_e << k_port_shift) k_rx_pin_4	k_rx_pe_4	PE4 (pin 107)
= (k_rx_port_e << k_port_shift) k_rx_pin_5	k_rx_pe_5	PE5 (pin 106)
= (k_rx_port_e << k_port_shift) k_rx_pin_6	k_rx_pe_6	PE6 (pin 102)
= (k_rx_port_e << k_port_shift) k_rx_pin_7	k_rx_pe_7	PE7 (pin 101)
= (k_rx_port_f << k_port_shift) k_rx_pin_0	k_rx_pf_0	PF0 - N/A on 144-pin
= (k_rx_port_f << k_port_shift) k_rx_pin_1	k_rx_pf_1	PF1 - N/A on 144-pin
= (k_rx_port_f << k_port_shift) k_rx_pin_2	k_rx_pf_2	PF2 - N/A on 144-pin
= (k_rx_port_f << k_port_shift) k_rx_pin_3	k_rx_pf_3	PF3 - N/A on 144-pin
= (k_rx_port_f << k_port_shift) k_rx_pin_4	k_rx_pf_4	PF4 - N/A on 144-pin
= (k_rx_port_f << k_port_shift) k_rx_pin_5	k_rx_pf_5	PF5 (pin 9)
= (k_rx_port_f << k_port_shift) k_rx_pin_6	k_rx_pf_6	PF6 - N/A on 144-pin
= (k_rx_port_f << k_port_shift) k_rx_pin_7	k_rx_pf_7	PF7 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_0	k_rx_pg_0	PG0 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_1	k_rx_pg_1	PG1 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_2	k_rx_pg_2	PG2 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_3	k_rx_pg_3	PG3 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_4	k_rx_pg_4	PG4 - N/A on 144-pin

= (k_rx_port_g << k_port_shift) k_rx_pin_5	k_rx_pg_5	PG5 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_6	k_rx_pg_6	PG6 - N/A on 144-pin
= (k_rx_port_g << k_port_shift) k_rx_pin_7	k_rx_pg_7	PG7 - N/A on 144-pin
= (k_rx_port_j << k_port_shift) k_rx_pin_0	k_rx_pj_0	PJ0 - N/A on 144-pin
= (k_rx_port_j << k_port_shift) k_rx_pin_1	k_rx_pj_1	PJ1 - N/A on 144-pin
= (k_rx_port_j << k_port_shift) k_rx_pin_2	k_rx_pj_2	PJ2 - N/A on 144-pin
= (k_rx_port_j << k_port_shift) k_rx_pin_3	k_rx_pj_3	PJ3 (pin 13)
= (k_rx_port_j << k_port_shift) k_rx_pin_4	k_rx_pj_4	PJ4 - N/A on 144-pin
= (k_rx_port_j << k_port_shift) k_rx_pin_5	k_rx_pj_5	PJ5 (pin 11)
= (k_rx_port_j << k_port_shift) k_rx_pin_6	k_rx_pj_6	PJ6 - N/A on 144-pin
= (k_rx_port_j << k_port_shift) k_rx_pin_7	k_rx_pj_7	PJ7 - N/A on 144-pin

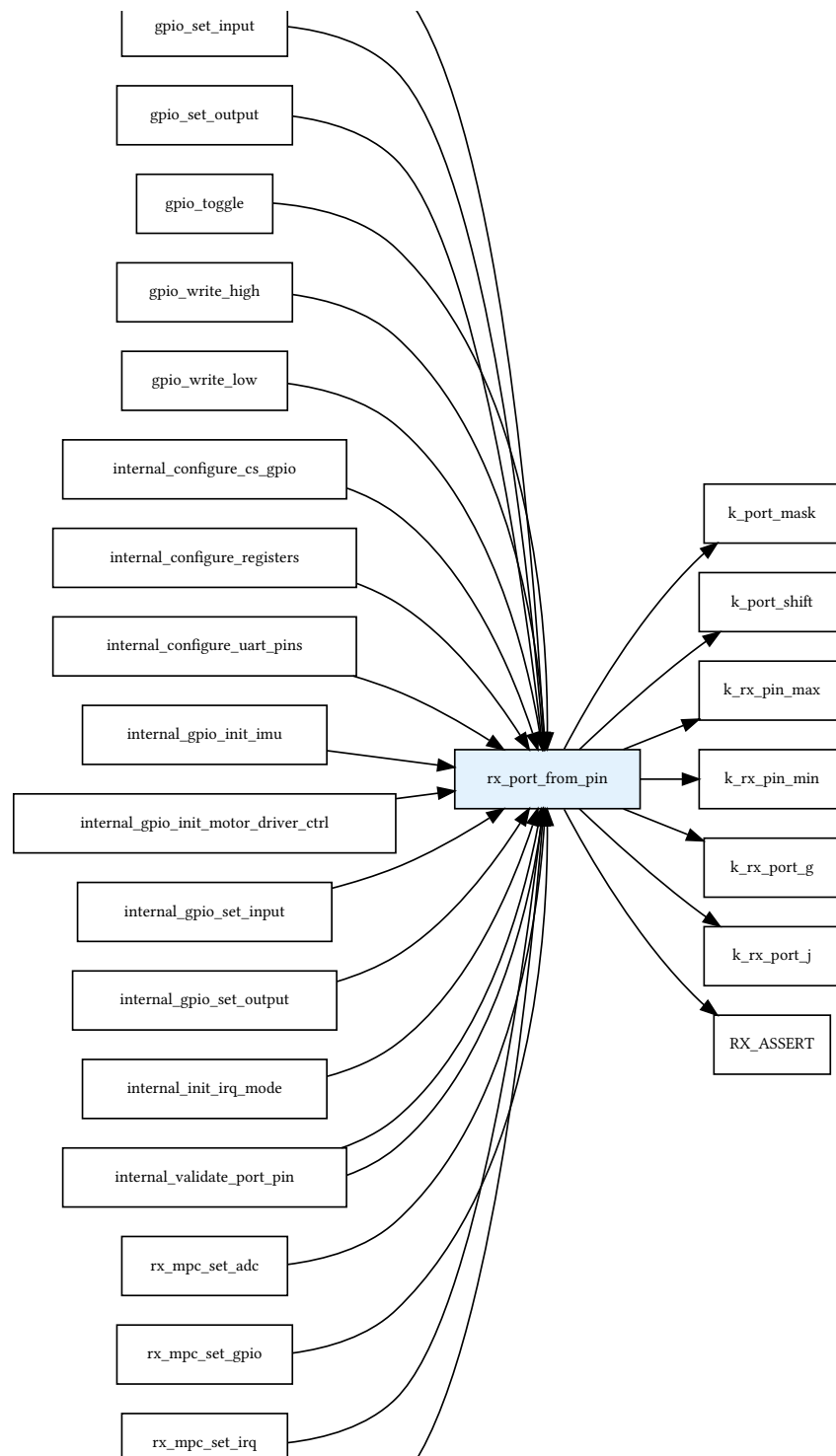
—

rx_port_from_pin

```
static uint8_t rx_port_from_pin(rx_port_pin_t pin)
```

Extract port number from combined port/pin identifier.

Decodes the port number (upper byte) from a 16-bit `gpio_pin_t` value using right-shift by `k_port_shift` (8 bits). This extracts bits 15-8 which contain the `rx_port_number_t` value.

Figure 228: Call graph for `rx_port_from_pin`

_Defined in libs/rx_core/inc/rx_port_constants.h:1106_

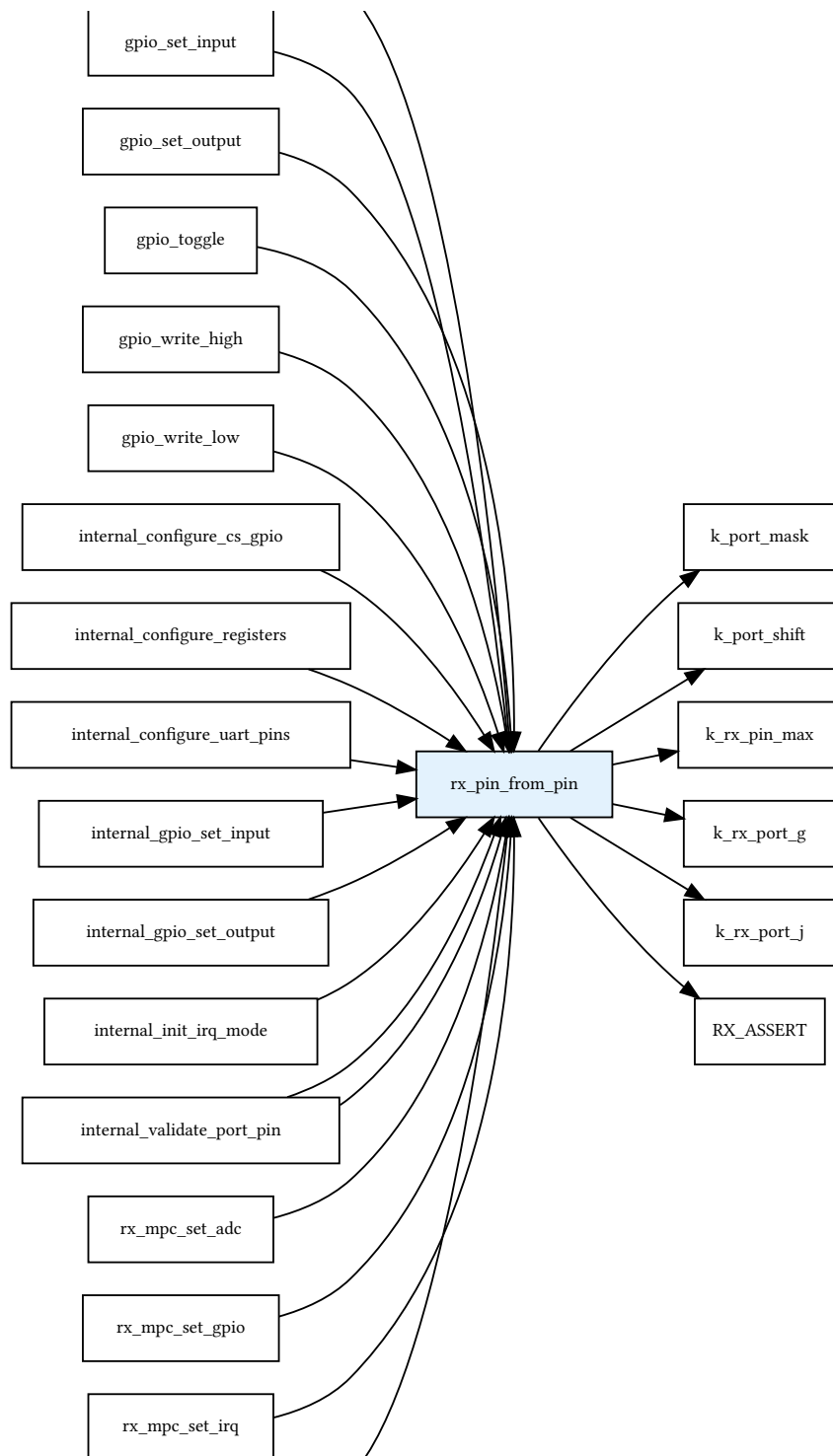
—

rx_pin_from_pin

```
static uint8_t rx_pin_from_pin(rx_port_pin_t pin)
```

Extract pin number from combined port/pin identifier.

Decodes the pin number (lower byte) from a 16-bit gpio_pin_t value using bitwise AND with k_port_mask (0xFF). This extracts bits 7-0 which contain the rx_pin_number_t value.

Figure 229: Call graph for `rx_pin_from_pin`

_Defined in libs/rx_core/inc/rx_port_constants.h:1268_

—

rx_register_guard.h

Register Guard - Periodic Refresh for ESD/EMI Protection.

Provides periodic refresh of critical hardware registers to recover from ESD (Electrostatic Discharge) or EMI (Electromagnetic Interference) induced bit flips that could corrupt system configuration in harsh electrical environments.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

rx_register_guard_config_t

Register guard configuration constants.

Defines compile-time limits for register guard arrays. These values balance memory usage against protection coverage. Sizing is based on STAR project peripheral usage (4 motors = 8 GPIOs, plus sensors and communication).

Memory allocation:

- PORT guards: 10 x 1 byte = 10 bytes
- IPR guards: 16 x 1 byte = 16 bytes
- MSTPCR guards: 4 x 4 bytes = 16 bytes (hardcoded in .c file)
- Metadata: 10 bytes (counters, flags)
- Total: 52 bytes static RAM

Value	Name	Description
= 10	k_register_guard_max_ports	Maximum number of PORT registers to guard (STAR project: 10).
= 16	k_register_guard_max_ipr	Maximum number of IPR registers to guard (STAR project: 16).

See also: rx_register_guard_init() Uses these limits for array allocation

Since Version 1.0.0

—

rx_register_guard_init

```
rx_err_t rx_register_guard_init(void)
```

Initialize register guard module and capture golden reference values.

Reads current values of critical hardware registers and stores them as the “golden reference” that will be maintained by periodic refresh operations. This function must be called AFTER all peripheral initialization is complete, so the captured values represent the correct operating configuration.

Algorithm steps:

1. Check if already initialized (allow re-initialization)

2. Read all PORT.PDR registers (GPIO direction)
3. Read all ICU.IPR registers (interrupt priorities)
4. Read SYSTEM.MSTPCR registers (module stop control)
5. Store values in static arrays for later comparison
6. Set initialized flag
7. Reset correction counter to zero

This function captures CURRENT register values. If peripherals are misconfigured at init time, those incorrect values will be preserved!

1.0.0 Initial implementation

Initialize register guard module and capture golden reference values.

Initializes the register guard module by capturing the current values of all critical hardware registers as the “golden reference”. These values will be maintained by subsequent refresh operations. This function must be called AFTER all peripheral initialization is complete.

Algorithm:

1. Capture all PORT PDR registers via `internal_capture_pdr()`
2. Capture all MSTPCR registers via `internal_capture_mstpcr()`
3. Reset correction counter to 0
4. Set initialized flag to 1
5. Return `k_rx_ok` (always succeeds)

Captures CURRENT register values - ensure peripherals are correctly configured before calling or incorrect values will be preserved!

Returns: `rx_err_t` Error code indicating initialization status

Value	Description
<code>k_rx_ok</code>	Success, golden values captured
<code>k_rx_ok</code>	Success, golden values captured and module initialized

Pre: No preconditions - safe to call anytime

Pre: Peripherals should be fully initialized before calling

Pre: Peripherals should be fully initialized before calling

Pre: Clock and power configuration should be stable

Post: Module marked as initialized (`rx_register_guard_is_initialized()` returns true)

Post: Golden reference values stored in static arrays

Post: Correction counter reset to zero

Post: rx_register_guard_is_initialized() returns true

Post: s_state.pdr contains current PORT PDR values

Post: s_state.mstpcr contains current MSTPCR values

Post: s_state.corrections == 0

Note: Thread Safety: Not thread-safe. Call only from main thread during startup.

Note: Re-initialization: Safe to call multiple times. Use to update golden values after intentional configuration changes (e.g., mode switch).

Note: Execution Time: 2.5 us @ 240 MHz (read-only, no writes)

Note: Memory: Allocates 52 bytes static RAM on first call

Note: Thread Safety: Not thread-safe. Call from main thread during startup.

Note: Re-initialization: Safe to call multiple times to re-capture golden values after intentional configuration changes.

Note: Execution Time: 2.2 us @ 240 MHz

Note: Conditional Compilation: On non-RX targets (unit tests), capture functions are no-ops but module state is still updated.

Captured Registers::

Example - Normal Initialization:: intmain(void){ hardware_init(); motor_init(); sensor_init(); communication_init();

```
rx_err_t err=rx_register_guard_init(); if(err!=k_rx_ok)
{ rx_log_error("MAIN","Registerguardinitfailed"); }
main_control_loop(); }
```

Example - Re-initialization After Mode Change:: voidenter_low_power_mode(void)
{ motor_disable_all();

```
gpio_set_low_power_config();
rx_register_guard_init();
rx_log_info("POWER","Enteredlowpowermode"); }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 2: [OK] Loop bounds: k_register_guard_max_ports, k_register_guard_max_ipr Rule 5: [OK] 2 preconditions, 3 postconditions

Initialization Sequence:: App, Guard, PDR, MSTPCR;

```
App => Guard [label="rx_register_guard_init()"]; Guard => PDR [label="Read PORT0-PORTJ"]; PDR
>> Guard [label="PDR values"]; Guard => MSTPCR [label="Read MSTPCRA-D"]; MSTPCR >>
Guard [label="MSTPCR values"]; Guard box Guard [label="Store golden valuesnReset counternSet
initialized"]; Guard >> App [label="k_rx_ok"];
```

Example:: hardware_init(); motor_init(); comm_init();

```
rx_err_t err=rx_register_guard_init(); assert(err==k_rx_ok);
assert(rx_register_guard_is_initialized());
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 2 preconditions, 4 postconditions Rule 7: [OK] Return value always k_rx_ok (documented)

See also: rx_register_guard_refresh() Periodic refresh using golden values,
 rx_register_guard_is_initialized() Check initialization status,
 rx_register_guard_get_correction_count() Monitor ESD/EMI events,
 rx_register_guard_refresh() Uses captured golden values,
 rx_register_guard_is_initialized() Check if init completed, internal_capture_pdr() PDR
 capture helper, internal_capture_mstpcr() MSTPCR capture helper

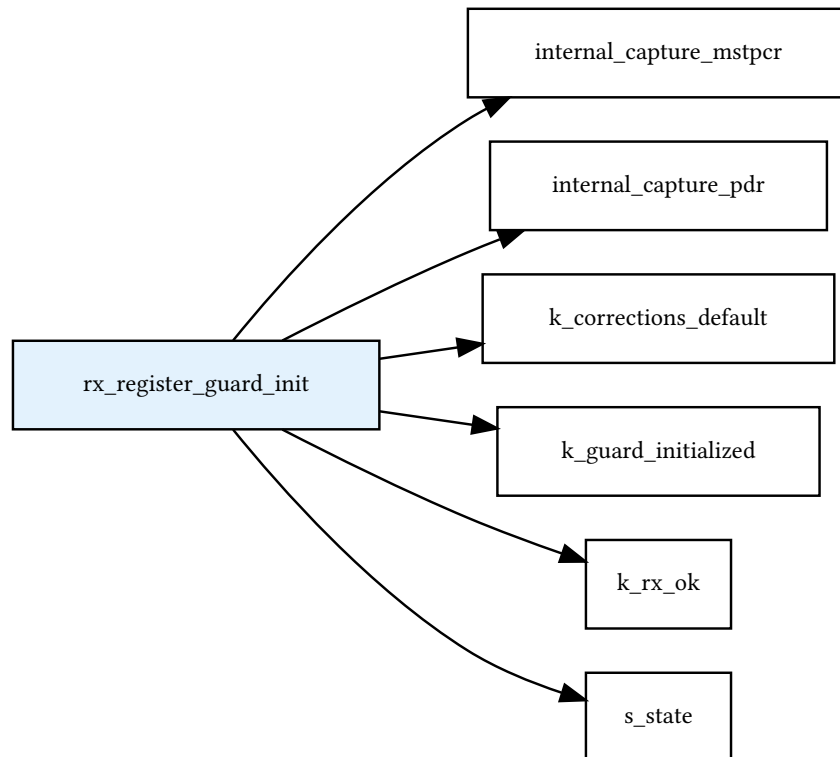


Figure 230: Call graph for rx_register_guard_init

_Defined in libs/rx_core/inc/rx_register_guard.h:403_

Since Version 1.0.0

—

rx_register_guard_refresh

```
void rx_register_guard_refresh(void)
```

Refresh all critical registers by comparing against golden reference.

Compares current hardware register values against the golden reference captured during initialization. Any registers that differ (indicating corruption from ESD/EMI) are restored to their golden values. Correction events are counted for diagnostics and telemetry.

This function is the core of the register guard protection mechanism. It executes quickly (<5 us) and is designed to be called periodically from the main control loop at 1-10 Hz.

Algorithm steps:

1. Check if module is initialized (early return if not)
2. For each PORT.PDR register: a. Read current value b. Compare against golden value c. If mismatch: write golden value, increment counter
3. For each ICU.IPR register: a. Read current value b. Compare against golden value c. If mismatch: write golden value, increment counter
4. For SYSTEM.MSTPCR registers (A/B/C/D): a. Unlock PRCR protection (brief interrupt disable 200ns) b. Read current value c. Compare against golden value d. If mismatch: write golden value, increment counter e. Re-lock PRCR protection (re-enable interrupts)

****Best Case**** (no corrections needed):

- Execution time: 4.2 us @ 240 MHz
- Memory reads: 30 (10 PORTs + 16 IPRs + 4 MSTCPRs)
- Memory writes: 0
- Interrupt latency: 200 ns (PRCR unlock/lock)

****Worst Case**** (all registers corrupted):

- Execution time: 5.8 us @ 240 MHz
- Memory reads: 30
- Memory writes: 30
- Interrupt latency: 200 ns (same)

****Average Case**** (1-2 corrections per refresh):

- Execution time: 4.5 us @ 240 MHz
- CPU overhead at 10 Hz: 0.0045% (negligible)

Golden reference values unchanged (read-only after init)

Correction counter monotonically increasing (never decreases)

Frequent corrections (>10/second) indicate severe EMI problems. Consider hardware fixes: filtering, shielding, grounding.

1.0.0 Initial implementation

Refresh all critical registers by comparing against golden reference.

Core protection function that compares all monitored hardware registers against golden reference values and restores any that have been corrupted. Designed to be called periodically from the main control loop at 1-10 Hz.

Algorithm:

1. Check initialized flag - return immediately if not initialized
2. Call `internal_refresh_pdr()` - check/restore all PORT PDR registers
3. Call `internal_refresh_mstpcr()` - check/restore all MSTPCR registers (includes brief interrupt disable for PRCR unlock)

Golden values unchanged (read-only after init)

Correction counter monotonically increasing

Returns: void (no return value)

Pre: Module must be initialized via `rx_register_guard_init()`

Pre: Should be called from main task context (not ISR)

Pre: Module should be initialized via `rx_register_guard_init()`

Pre: Should be called from main task context (not ISR)

Post: All monitored registers match golden reference values

Post: Correction counter incremented for each mismatch detected

Post: System configuration is consistent with initialization

Post: All monitored registers match golden values

Post: `s_state.corrections` incremented for each mismatch

Post: Interrupts restored to original state

Note: ****Thread Safety**:** Not thread-safe. Call only from main control loop.

Note: ****Interrupt Latency**:** Brief 200 ns interrupt disable during PRCR unlock to maintain atomic MSTPCR register updates.

Note: ****Recommended Call Rate**:** 1-10 Hz. Higher rates waste CPU, lower rates increase window where corruption goes undetected.

Note: ****Silent Operation**:** Does not log corrections (use `get_correction_count()` for monitoring). Avoids log spam in noisy environments.

Note: Thread Safety: Not thread-safe. Call from main control loop only.

Note: ISR Safety: NOT safe to call from ISRs (brief interrupt disable).

Note: Silent No-op: Returns silently if not initialized (defensive).

Note: Conditional Compilation: On non-RX targets, refresh functions are no-ops but initialization check still occurs.

Warning: Do NOT call from interrupt handlers - brief interrupt disable would cause reentrant interrupt disable leading to deadlock.

Warning: If called before `rx_register_guard_init()`, function returns immediately with no effect (silent failure for safety).

Warning: Do NOT call from interrupt handlers

Warning: High correction rates (>10/sec) indicate hardware EMI problems

Performance Analysis::

Execution Timing::

Example - Main Loop Integration:: `void main_control_loop(void)`
`{ typedef enum: uint32_t { k_refresh_interval_100hz=100, } refresh_config_t;`
`uint32_t loop_counter=0;`
`while(1){ motor_update(); sensor_update(); communication_update();`
`if(++loop_counter>=k_refresh_interval_100hz){ rx_register_guard_refresh(); loop_counter=0; }`
`tx_thread_sleep(1); }`

Example - With Diagnostics:: `void main_control_loop_with_diagnostics(void)`
`{ uint32_t loop_counter=0; uint32_t last_correction_count=0;`
`while(1){`
`if(++loop_counter>=100){ rx_register_guard_refresh();`
`uint32_t current_count=rx_register_guard_get_correction_count(); if(current_count!`
`=last_correction_count){ uint32_t new_corrections=current_count-last_correction_count;`
`rx_log_warn("GUARD","EMI detected: %lu corrections", new_corrections);`
`if(new_corrections>10){ rx_log_error("GUARD","SEVERE EMI-Check hardware!"); }`
`last_correction_count=current_count; }`

```
loop_counter=0; }
tx_thread_sleep(1); }
```

Example - Variable Refresh Rate Based on Environment:: voidadaptive_refresh(void)
{ staticuint32_tcounter=0; uint32_trefresh_interval=motors_active()?10:100;
if(++counter>=refresh_interval){ rx_register_guard_refresh(); counter=0; } }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 2: [OK] Loop
bounds: k_register_guard_max_ports, k_register_guard_max_ipr Rule 5: [OK] 2 preconditions, 3
postconditions, 2 invariants Rule 9: [WARN] Brief interrupt disable unavoidable for PRCR atomicity

Execution Flow:: digraph refresh_flow { rankdir=TB; node [shape=box, style=rounded];
start [label="rx_register_guard_refresh()"]; check_init [label="initialized?", shape=diamond];
early_return [label="Return (no-op)"]; refresh_pdr [label="internal_refresh_pdr()n 1.5 us"];
refresh_mstpcr [label="internal_refresh_mstpcr()n 0.5-1.5 us"]; done [label="Done"];
start -> check_init; check_init -> early_return [label="no"]; check_init -> refresh_pdr [label="yes"];
refresh_pdr -> refresh_mstpcr; refresh_mstpcr -> done; early_return -> done; }

Performance:: Scenario Time @ 240 MHz Notes

Not initialized 10 ns Early return

No corrections 4.2 us Typical case

All corrected 5.8 us Worst case

CPU overhead @ 10 Hz 0.006% Negligible

Example - Main Loop Integration:: voidcontrol_loop(void){ uint32_tcounter=0; while(1)
{ motor_update(); sensor_update();
if(++counter>=100){ rx_register_guard_refresh(); counter=0; } tx_thread_sleep(1); } }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 2
preconditions, 3 postconditions, 2 invariants Rule 9: [WARN] Brief interrupt disable in
internal_refresh_mstpcr() (documented)

See also: rx_register_guard_init() Must call first to establish golden values,
rx_register_guard_get_correction_count() Monitor correction frequency,
rx_register_guard_reset_count() Clear counter for periodic logging,
rx_register_protection.h PRCR unlock/lock mechanism, rx_register_guard_init() Must
call first, rx_register_guard_get_correction_count() Monitor EMI events,
internal_refresh_pdr() PORT PDR refresh helper, internal_refresh_mstpcr() MSTPCR
refresh helper

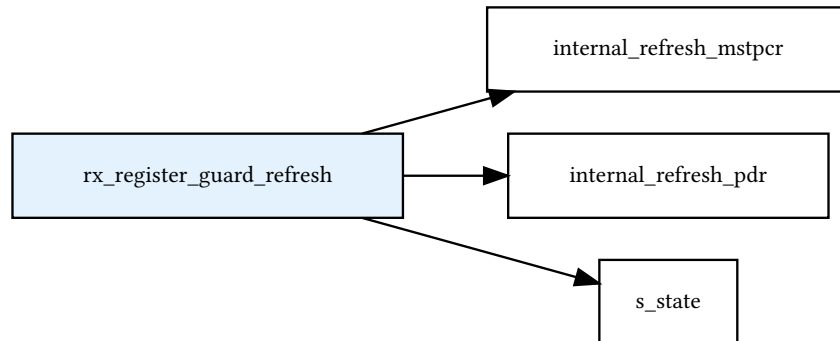


Figure 231: Call graph for rx_register_guard_refresh

_Defined in libs/rx_core/inc/rx_register_guard.h:596_

Since Version 1.0.0

—

rx_register_guard_get_correction_count

```
uint32_t rx_register_guard_get_correction_count(void)
```

Get count of register corrections since last reset.

Returns the total number of register corrections performed since module initialization or since the last call to rx_register_guard_reset_count(). Each individual register mismatch detected during refresh increments this counter by 1.

Non-zero correction counts indicate ESD/EMI events occurred that corrupted hardware registers. High correction rates (>10/second) suggest severe electromagnetic interference requiring hardware investigation.

This function is used for:

- Diagnostics: Detect EMI problems during development
- Telemetry: Report corruption events to ground station
- Logging: Periodic logging of environmental robustness
- Alerting: Trigger warnings when correction rate exceeds threshold

Algorithm:

1. Read static correction counter variable
2. Return value directly (no side effects)

Counter monotonically increases (never decreases except wrap or reset)

1.0.0 Initial implementation

Get count of register corrections since last reset.

Returns the cumulative count of register corrections since module initialization or the last call to rx_register_guard_reset_count(). Each individual register mismatch detected during refresh increments this counter by 1.

Non-zero counts indicate ESD/EMI events that corrupted hardware registers. This value is used for:

- Diagnostics during development

- Telemetry to ground station
- EMI alert threshold monitoring

Algorithm:

1. Check if module is initialized
2. If not initialized, return 0 (defensive)
3. Return current value of s_state.corrections

Returns: uint32_t Number of register corrections since init/reset

Value	Description
0	No register corruption detected (ideal case)
1-10	Occasional corruption (acceptable in noisy environments)
\>100	Frequent corruption (investigate EMI sources)
UINT32_MAX	Counter wrapped (extremely rare, 4 billion corrections)
0	No corrections (ideal) OR module not initialized
\>0	Number of ESD/EMI events detected and corrected

Pre: No preconditions - safe to call anytime

Pre: Module does not need to be initialized (returns 0 if not initialized)

Pre: None - safe to call anytime

Post: Counter value unchanged (read-only operation)

Post: Counter value unchanged (read-only operation)

Note: Thread Safety: Thread-safe for reading (atomic load on 32-bit MCU). However, refresh() may increment while reading, causing race condition. For critical monitoring, disable interrupts around read if needed.

Note: Counter Wrap: At UINT32_MAX, counter wraps to 0. With 10 Hz refresh and 1 correction/refresh, wrap occurs after 13.6 years of continuous operation. Not a practical concern.

Note: Execution Time: 20 ns @ 240 MHz (single memory read)

Note: Reset Behavior: Counter is cleared on rx_register_guard_init() and rx_register_guard_reset_count().

Note: Thread Safety: Mostly thread-safe (atomic read on 32-bit MCU). However, refresh() may increment while reading (race condition).

Note: Execution Time: 18 ns @ 240 MHz

Note: Returns 0 if not initialized (distinguishable only via is_initialized())

Example - Periodic Diagnostics:: void periodic_diagnostics(void){ static uint32_t last_log_time=0; uint32_t current_time=get_system_time_ms(); if(current_time-last_log_time>=10000) { uint32_t corrections=rx_register_guard_get_correction_count(); rx_log_info("DIAG","Register corrections:%lu",corrections); rx_register_guard_reset_count(); last_log_time=current_time; } }

Example - EMI Alert Threshold:: void check_emi_threshold(void){ static uint32_t last_count=0; static uint32_t last_check_time=0; uint32_t current_time=get_system_time_ms(); uint32_t current_count=rx_register_guard_get_correction_count(); uint32_t time_delta_ms=current_time-last_check_time; uint32_t count_delta=current_count-last_count; if(time_delta_ms>=1000){ float corrections_per_sec=(float)count_delta/(time_delta_ms/1000.0f); if(corrections_per_sec>10.0f){ rx_log_error("EMI","Severe interference:%.1fcorr/sec",corrections_per_sec); } else if(corrections_per_sec>1.0f) { rx_log_warn("EMI","Moderate interference:%.1fcorr/sec",corrections_per_sec); } last_count=current_count; last_check_time=current_time; } }

Example - Telemetry Reporting:: void send_telemetry(void) { telemetry_packet_t packet={ .timestamp=get_system_time_ms(), .temperature_celsius=read_temperature_celsius(), .n transmit_telemetry(&packet); }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 2: [OK] No loops Rule 5: [OK] 2 preconditions, 1 postcondition, 1 invariant

Example - Periodic Monitoring:: void monitor_emi(void){ static uint32_t last_count=0; uint32_t current=rx_register_guard_get_correction_count(); if(current>last_count){ uint32_t new_events=current-last_count; rx_log_warn("EMI","Detected %lu corrections",new_events); last_count=current; } }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 1 precondition, 1 postcondition

See also: rx_register_guard_refresh() Increments this counter on corrections, rx_register_guard_reset_count() Clear counter to zero, rx_register_guard_init() Also resets counter to zero, rx_register_guard_refresh() Increments this counter, rx_register_guard_reset_count() Clear counter to zero

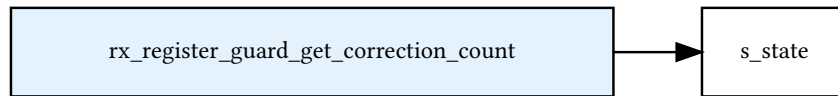


Figure 232: Call graph for rx_register_guard_get_correction_count

_Defined in libs/rx_core/inc/rx_register_guard.h:726_

Since Version 1.0.0

—

rx_register_guard_reset_count

```
void rx_register_guard_reset_count(void)
```

Reset the correction counter to zero.

Clears the register correction counter back to zero. This function is typically used for periodic diagnostics where you want to measure the correction rate over specific time intervals (e.g., corrections per minute).

Does not affect any other module state - golden reference values and initialization status remain unchanged.

Algorithm:

1. Set static correction counter to 0
2. Return (no other side effects)

1.0.0 Initial implementation

Reset the correction counter to zero.

Clears the register correction counter to zero. Used for periodic diagnostics where you want to measure correction rate over specific time intervals (e.g., corrections per minute).

Does not affect golden reference values or initialization status.

Algorithm:

1. Check if module is initialized
2. If not initialized, return immediately (defensive no-op)
3. Set s_state.corrections to k_corrections_default (0)
4. Assert postcondition (debug builds only)

Returns: void (no return value)

Pre: No preconditions - safe to call anytime

Pre: Module does not need to be initialized (no-op if not initialized)

Pre: None - safe to call anytime

Post: Correction counter set to 0

Post: Next call to `rx_register_guard_get_correction_count()` returns 0

Post: `s_state.corrections == k_corrections_default (0)`

Post: Next `get_correction_count()` returns 0

Note: Thread Safety: Not thread-safe with concurrent `refresh()` calls. If `refresh()` runs during reset, the counter may be non-zero immediately after reset. For critical measurements, disable interrupts around reset.

Note: Execution Time: 30 ns @ 240 MHz (single memory write)

Note: Use Case: Periodic logging with reset allows measuring correction rate over fixed intervals without calculating deltas.

Note: Thread Safety: Not thread-safe with concurrent `refresh()` calls.

Note: Execution Time: 30 ns @ 240 MHz

Note: Silent no-op if not initialized (defensive)

Example - Periodic Logging with Reset:: `void periodic_logging(void)`

```
{ static uint32_t last_log_time=0; uint32_t current_time=get_system_time_ms();
  if(current_time-last_log_time>=60000)
  { uint32_t corrections=rx_register_guard_get_correction_count();
    rx_log_info("GUARD","Corrections last minute:%lu",corrections);
    rx_register_guard_reset_count();
    last_log_time=current_time; } }
```

Example - Manual Diagnostics:: `void cmd_show_emi_stats(void)`

```
{ uint32_t corrections=rx_register_guard_get_correction_count(); printf("Total corrections since boot:
%lu\n",corrections);
  rx_register_guard_reset_count(); printf("Counter reset. Monitoring for new events...\n"); }
```

Example - Test Harness:: `void test_correction_counting(void){ rx_register_guard_reset_count();`

```
  PORT0.PDR.BYTE=0xFF;
  rx_register_guard_refresh();
  uint32_t corrections=rx_register_guard_get_correction_count(); assert(corrections==1);
  rx_register_guard_reset_count(); }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 2: [OK] No loops

Rule 5: [OK] 2 preconditions, 2 postconditions

Example - Periodic Logging with Reset: voidlog_emi_stats(void)
 { uint32_tcorrections=rx_register_guard_get_correction_count();
 rx_log_info("EMI","Correctionsthisperiod:%lu",corrections); rx_register_guard_reset_count(); }

NASA Power of 10 Compliance: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 1 precondition, 2 postconditions (with assert validation)

See also: rx_register_guard_get_correction_count() Read current count,
 rx_register_guard_refresh() Increments counter on corrections,
 rx_register_guard_init() Also resets counter to zero,
 rx_register_guard_get_correction_count() Read counter before reset,
 rx_register_guard_init() Also resets counter

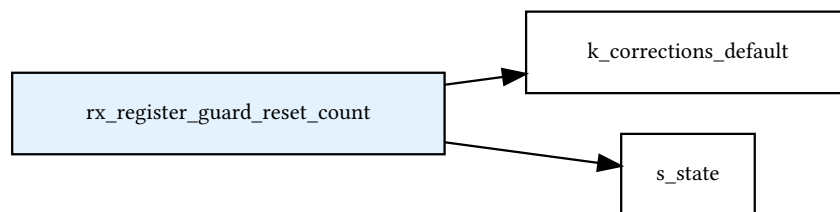


Figure 233: Call graph for rx_register_guard_reset_count

_Defined in libs/rx\core\inc\rx_register_guard.h:832_

Since Version 1.0.0

—

rx_register_guard_is_initialized

```
bool rx_register_guard_is_initialized(void)
```

Check if register guard module is initialized.

Returns the initialization status of the register guard module. This function allows application code to verify that the guard is active before relying on its protection, or to conditionally skip operations that depend on the guard.

The module is considered initialized after a successful call to rx_register_guard_init(). There is no “uninitialized” function - once initialized, the module remains active until MCU reset.

Algorithm:

1. Read static initialized flag variable
2. Return boolean value directly

1.0.0 Initial implementation

Check if register guard module is initialized.

Returns the initialization status of the register guard module. Allows application code to verify that protection is active before relying on it, or to conditionally skip operations that depend on the guard.

The module is initialized after rx_register_guard_init() completes. There is no de-initialization function - once initialized, the module remains active until MCU reset.

Algorithm:

1. Read `s_state.initialized` flag
2. Return true if non-zero, false if zero

Returns: bool Initialization status

Value	Description
true	Module is initialized, golden values captured, refresh() is active
false	Module not initialized, refresh() will return immediately without effect
true	Module initialized, refresh() is active
false	Module not initialized, refresh() is no-op

Pre: No preconditions - safe to call anytime

Pre: None - safe to call anytime

Post: Module state unchanged (read-only query)

Post: Module state unchanged (read-only query)

Note: Thread Safety: Thread-safe (atomic read on embedded systems)

Note: Execution Time: 15 ns @ 240 MHz (single memory read)

Note: Use Case: Primarily for defensive programming and diagnostics. Normal application flow should ensure `init()` is called at startup.

Note: Thread Safety: Thread-safe (atomic read on 32-bit MCU)

Note: ISR Safety: Safe to call from ISRs

Note: Execution Time: 15 ns @ 240 MHz

Example - Defensive Programming:: `void main_control_loop(void){ if(! rx_register_guard_is_initialized()){ rx_log_error("MAIN","Registerguardnotinitialized!"); rx_err_t err=rx_register_guard_init(); if(err!=k_rx_ok) { rx_log_fatal("MAIN","Cannot initialize registerguard"); return; } } while(1){ rx_register_guard_refresh(); } }`

Example - Conditional Refresh:: `void periodic_maintenance(void){ sensor_calibration_check(); motor_temperature_check(); if(rx_register_guard_is_initialized()){ rx_register_guard_refresh(); } }`

Example - Diagnostics Report:: void print_system_status(void){ printf("SystemDiagnostics:n");
printf("MCU:RX72N@240MHzn"); printf("Uptime:%lusecondsn",get_uptime_seconds());
printf("RegisterGuard:%sn", rx_register_guard_is_initialized()? "ACTIVE": "INACTIVE");

if(rx_register_guard_is_initialized()){ uint32_t corrections=rx_register_guard_get_correction_count();
printf("-Totalcorrections:%lun",corrections); printf("-Correctionrate:%.2f/hourn", corrections/
(get_uptime_seconds()/3600.0f)); } }

Example - Unit Test Setup:: void test_refresh_requires_init(void){ assert(!
rx_register_guard_is_initialized());

rx_register_guard_refresh(); assert(rx_register_guard_get_correction_count()==0);

rx_err_t err=rx_register_guard_init(); assert(err==k_rx_ok);
assert(rx_register_guard_is_initialized());

rx_register_guard_refresh(); }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 2: [OK] No loops
Rule 5: [OK] 1 precondition, 1 postcondition

Example - Defensive Programming:: void start_control_loop(void){ if(!
rx_register_guard_is_initialized()){ rx_log_error("MAIN","Registerguardnotactive!");
rx_register_guard_init(); } }

Example - Diagnostics:: void print_status(void){ printf("RegisterGuard:%sn",
rx_register_guard_is_initialized()? "ACTIVE": "INACTIVE"); }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 1
precondition, 1 postcondition

See also: rx_register_guard_init() Initializes module, sets flag to true,
rx_register_guard_refresh() Checks this flag before operating,
rx_register_guard_init() Sets initialized flag to true

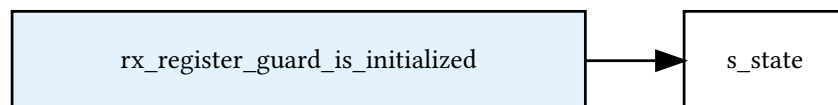


Figure 234: Call graph for rx_register_guard_is_initialized

_Defined in libs/rx_core/inc/rx_register_guard.h:957_

Since Version 1.0.0

—

rx_register_protection.h

RX72N Register Protection Control (PRCR) - Write Protection Constants.

Provides centralized definitions for the Protection Register (PRCR) unlock/lock sequences used throughout the RX72N firmware to safely modify write-protected system registers. The PRCR mechanism prevents accidental corruption of critical configuration registers through a key-code based protection scheme.

Includes:

- <stdint.h>

rx_prcr_values_t

PRCR register unlock/lock values for write-protected system registers.

Defines all valid PRCR (Protect Register) values for unlocking and locking write-protected system registers on the RX72N. The PRCR uses a key code mechanism (0xA5 in upper byte) combined with protection enable bits (lower byte) to selectively unlock register groups.

Register Format (16-bit SYSTEM.PRCR at 0x000803FE):

Key Code Security:

- Writing without 0xA5 key -> ignored (no effect)
- Prevents accidental unlock from pointer errors
- Prevents unlock from EMI-induced bit flips

Protection Groups:

- PRC0 (bit 0): Protects SCKCR, PLLCR, PLLCR2, SCKCR3, MOSCWTCR, etc.
- PRC1 (bit 1): Protects MSTPCRA, MSTPCRB, MSTPCRC, MSTPCRD
- PRC3 (bit 3): Protects LVCMPCR, LVDXCR, LVDYCR

Value	Name	Description
= 0xA502	k_rx_prcr_unlock_prc1	Unlock PRC1 only (Module Stop Control registers).
= 0xA503	k_rx_prcr_unlock_prc0_prc1	Unlock PRC0+PRC1 (Clock Generation + Module Stop Control).
= 0xA50B	k_rx_prcr_unlock_prc1_prc3	Unlock PRC1+PRC3 (Module Stop + Low-Voltage Detection).
= 0xA50F	k_rx_prcr_unlock_all	Unlock all protection groups (PRC0+PRC1+PRC3).
= 0xA500	k_rx_prcr_lock	Lock all protection groups (re-enable write protection).
= 0x000F	k_rx_prcr_readback_mask	Mask for PRC protection bits (for readback verification).

Example:

```

Bits15-8: [0xA5]Keycode(requiredforanywritetotakeeffect)
Bits7-4: [Reserved]Alwayswrite0
Bit3: [PRC3]Low-VoltageDetectionprotection(1=unlocked,0=locked)
Bit2: [Reserved]Alwayswrite0
Bit1: [PRC1]ModuleStopControlprotection(1=unlocked,0=locked)
Bit0: [PRC0]ClockGenerationCircuitprotection(1=unlocked,0=locked)

```

—

rx_simulator_config.h

Configuration for e^2 studio simulator support.

This header provides simulator-specific configuration for testing firmware logic without real hardware. The Renesas e^2 studio simulator cannot fully model certain hardware features:

Hardware limitations in simulator:

- External oscillators (24 MHz crystal) - Not modeled
- PLL/PPLL lock behavior - Status flags never set
- UART/SCI peripheral hardware - TX/RX don't work
- USB controller - Limited or no enumeration

- SPI transfers - No real external devices
- Timing accuracy - Not cycle-accurate

WARNING: Simulator builds are FOR LOGIC TESTING ONLY. Do not use for timing validation, peripheral verification, or hardware-specific testing. Always validate critical paths on real hardware before deployment.

Includes:

- <stdint.h>

RX_IS_SIMULATOR

```
#define RX_IS_SIMULATOR 0
```

Compile-time flag indicating simulator build.

Note: Prefer RX_IS_SIMULATOR over direct #ifdef RX_SIMULATOR_MODE for consistency and readability.] **Example:**

```
#ifRX_IS_SIMULATOR
//Simulator-specificcode
#else
//Hardware-specificcode
#endif
```

—

simulator_clock_timing_t

Simulator-specific clock stabilization timeouts.

Hardware requires multi-millisecond delays for oscillator and PLL stabilization (polling OSCOVFSR register bits until they set). Simulator cannot model these delays as the hardware circuits don't exist, so we provide instant-ready constants.

Hardware behavior:

- Main oscillator: 10 ms stabilization (2.4M instruction cycles)
- PLL: 200 us stabilization (poll OSCOVFSR bit 2)
- PPLL: 200 us stabilization (poll OSCOVFSR bit 3)

Simulator behavior:

- All clocks: Assume instant ready (skip polling loops)

Value	Name	Description
= 1	k_sim_clock_instant	Skip delay entirely (assume immediate ready)

Since Version 1.0.0

—

rx_threadx_config.h

ThreadX RTOS Tick Rate Configuration.

This header defines the ThreadX system tick frequency used throughout the RX72N firmware for RTOS time conversions. The tick rate is configured to 100 Hz (10ms period) via hardware timer CMT0 (Compare Match Timer 0).

Includes:

- <stdint.h>

s_rx_threadx_tick_rate_hz

```
const uint16_t s_rx_threadx_tick_rate_hz
```

ThreadX system tick frequency (100 Hz).

—

rx_time_constants.h

Time Unit Conversion Constants for RX72N Firmware.

This header provides typed enumeration constants for time unit conversions used throughout the RX72N firmware. All time conversions follow SI (International System of Units) standards with milliseconds, microseconds, and nanoseconds.

Includes:

- <stdint.h>

rx_time_conversion_t

Time unit conversion factors following SI standards.

Standard International System of Units (SI) time unit conversions used for timeout calculations, RTOS tick conversions, and hardware timer configuration throughout the RX72N firmware.

Value	Name	Description
= 1000	k_rx_ms_per_second	Milliseconds per second (SI standard).
= 1000	k_rx_us_per_ms	Microseconds per millisecond (SI standard).
= 1000	k_rx_ns_per_us	Nanoseconds per microsecond (SI standard).
= 1000000	k_rx_us_per_second	Microseconds per second (derived SI constant).
= 1000000	k_rx_ns_per_ms	Nanoseconds per millisecond (derived SI constant).
= 10	k_threadx_ms_per_tick	Milliseconds per ThreadX tick (RTOS configuration).

—

rx_time_interface.h

Time Interface - Dependency Inversion for Timing Operations.

Defines the abstract interface for time operations in RX72N firmware, implementing the Dependency Inversion Principle (DIP) from SOLID design. This header contains ONLY the interface definition with NO implementation details, enabling complete decoupling between high-level modules and low-level timing mechanisms.

Includes:

- <stdbool.h>
- <stddef.h>
- <stdint.h>
- "rx_err.h"

rx_time_sleep_ms_fn

```
typedef void(* rx_time_sleep_ms_fn) (void *ctx, uint32_t ms)
```

Function pointer: Sleep/delay for specified milliseconds.

Blocks (or simulates blocking) for the specified number of milliseconds. Behavior depends on concrete implementation:

ThreadX Implementation:

- Calls `tx_thread_sleep(ticks)` where `ticks = ms * (TX_TIMER_TICKS_PER_SECOND / 1000)`
- Yields CPU to other threads (cooperative multitasking)
- Actual sleep time may be slightly longer due to tick granularity
- Example: 10ms sleep with 10ms tick = exactly 1 tick
- Example: 15ms sleep with 10ms tick = 2 ticks = 20ms actual

Mock Implementation (manual control):

- Returns immediately without advancing simulated time
- Test code manually advances time via `mock_time_advance_ms()`
- Use for fine-grained control of timing in unit tests

Mock Implementation (auto-advance):

- Returns immediately AND advances simulated time by ms
- Useful for testing timeout logic without real delays
- 1000ms timeout completes in 1us real time

Null Implementation (bare-metal):

- Function pointer set to `nullptr`
- Caller must check for `nullptr` before calling
- Use for polling-only code that never sleeps

Never call from interrupt context (ThreadX implementation will fault).

Excessive sleeps (>1000ms) may indicate architectural problems.

Note: Thread Safety: ThreadX implementation is thread-safe. Mock depends on test harness (single-threaded tests are safe).

Note: Accuracy: ThreadX sleep accuracy limited by tick rate (typically +/-10ms). Use hardware timers for precise delays <1ms.

Note: Execution Time: ThreadX context switch 50-200us. Mock returns in <1us.

Note: Blocking: ThreadX yields CPU (other threads run). Mock returns immediately.

Warning: ThreadX tick granularity causes rounding: 15ms sleep with 10ms tick = 20ms actual.]
See also: `rx_time_get_ms_fn` Get current time for timeout calculations,
`rx_time_is_elapsed_fn` Check if timeout occurred, `tx_thread_sleep()` ThreadX implementation calls this

Since Version 1.0.0

—

rx_time_get_ms_fn

```
typedef uint32_t(* rx_time_get_ms_fn) (void *ctx)
```

Function pointer: Get current monotonic time in milliseconds.

Returns a continuously increasing time value in milliseconds, suitable for measuring elapsed time and implementing timeouts. This is NOT wall-clock time (not UTC, not human-readable) - it's a monotonic counter that starts at system boot and increases until overflow (49.7 days).

Monotonic Guarantee: Time NEVER decreases, even if system clock is adjusted. This makes it reliable for timeouts and performance measurements.

Wrap-Around Behavior: 32-bit counter wraps to 0 after 2^{32} milliseconds (4,294,967,296 ms = 49.71 days). Properly written timeout code handles this correctly using unsigned arithmetic.

Implementation Details:

ThreadX Implementation:

- Returns `tx_time_get() * (1000 / TX_TIMER_TICKS_PER_SECOND)`
- Resolution matches ThreadX tick rate (typically 10ms on RX72N)
- Examples:

100 Hz tick: Resolution = 10ms (0, 10, 20, 30, ...) 1000 Hz tick: Resolution = 1ms (0, 1, 2, 3, ...)

- Overhead: 50 cycles (200 ns @ 240 MHz)

Mock Implementation:

- Returns static counter maintained by mock
- Manual control: Counter only changes via `mock_time_advance_ms()`
- Auto-advance: Counter increases during `sleep_ms()` calls
- Resolution: 1ms (arbitrary, no hardware limits in simulation)
- Overhead: 10 cycles (40 ns @ 240 MHz, no RTOS)

Bare-Metal Implementation (user-provided):

- Could use CMT hardware timer counter
- Could use SysTick (if using ARM Cortex-M)
- Resolution depends on hardware timer configuration

`get_ms(t2) >= get_ms(t1)` for $t2 > t1$ (unless wrap)

Resolution matches implementation tick rate (e.g., 10ms for ThreadX)

NOT wall-clock time! Do not use for timestamps or logging absolute time.

Wraps every 49.7 days. Long-running systems must handle wrap correctly.

Note: Thread Safety: ThreadX `tx_time_get()` is thread-safe. Mock reads volatile counter (safe for single-threaded tests).

Note: Execution Time: ThreadX 200ns, Mock 40ns @ 240 MHz (very fast)

Note: Wrap-Around: Use unsigned arithmetic for timeouts to handle wrap correctly: `(uint32_t) (current - start) >= timeout` works across wrap boundary.

Note: Resolution: Limited by ThreadX tick rate. For sub-millisecond timing, use hardware timers (CMT, GPT) directly.] **See also:** `rx_time_sleep_ms_fn` Sleep for specified duration, `rx_time_is_elapsed_fn` Convenience function for timeout checks, `tx_time_get()` ThreadX implementation uses this

Since Version 1.0.0

—

rx_time_is_elapsed_fn

```
typedef bool(* rx_time_is_elapsed_fn) (void *ctx, uint32_t start_ms, uint32_t
timeout_ms)
```

Function pointer: Check if timeout has elapsed (optional convenience).

Convenience function that combines `get_ms()` and timeout arithmetic into a single call. Equivalent to: `(get_ms(ctx) - start_ms) \>= timeout_ms` with correct handling of 32-bit wrap-around.

This function is OPTIONAL - implementations may set this pointer to NULL. Callers should either:

1. Check for nullptr before calling, OR
2. Implement timeout check manually using `get_ms()`

Why Optional?: Some implementations (bare-metal, minimal mocks) may want to provide only the core `get_ms()` function and let callers handle timeout logic themselves. This keeps the interface minimal.

Wrap-Around Safety: Uses unsigned 32-bit arithmetic which correctly handles wraparound at `UINT32_MAX`:

- Example: `start=4294967290, timeout=100, current=50`
- `Elapsed = (50 - 4294967290) = 110` via unsigned underflow
- `110 >= 100 -> true` (timeout occurred) [OK]

Algorithm:

Function pointer may be NULL - check before dereferencing!

`timeout_ms > 25` days may behave unexpectedly near wrap boundary

Note: Thread Safety: Thread-safe if `get_ms()` is thread-safe

Note: Execution Time: 250ns @ 240 MHz (one `get_ms()` call + arithmetic)

Note: Optional Function: May be NULL in minimal implementations. Always check!

Note: Wrap-Around: Correctly handles wrap using unsigned arithmetic

Note: Resolution: Limited by `get_ms()` resolution (typically 10ms for ThreadX)

Warning: `start_ms` MUST be from `get_ms()`, not arbitrary value or hardware timer] **See also:** `rx_time_get_ms_fn` Used internally by this function, `rx_time_sleep_ms_fn` Often used together in timeout loops

Example:

```
1. current_ms=get_ms(ctx)
2. elapsed=current_ms-start_ms//Unsignedsubtraction(wrap-safe)
3. return(elapsed>=timeout_ms)
```

Since Version 1.0.0

—

rx_time_threadx_get_interface

```
rx_err_t rx_time_threadx_get_interface(rx_time_interface_t *iface)
```

Validate that a time interface is properly initialized.

Performs defensive validation of a time interface to ensure it meets the minimum requirements for safe usage. Checks that the interface pointer is non-NULL and that required function pointers (sleep_ms, get_ms) are populated.

This function implements “fail-fast” validation - it’s better to detect a misconfigured interface immediately at initialization than to crash later when dereferencing NULL function pointers.

Validation Rules:

1. Interface pointer must be non-NULL
2. sleep_ms function pointer must be non-NULL (REQUIRED)
3. get_ms function pointer must be non-NULL (REQUIRED)
4. ctx pointer may be NULL (implementation-dependent)
5. is_elapsed pointer may be NULL (OPTIONAL)

Algorithm steps:

1. Check if iface pointer is nullptr -> return k_rx_err_null_ptr
2. Check if sleep_ms is nullptr -> return k_rx_err_invalid_state
3. Check if get_ms is nullptr -> return k_rx_err_invalid_state
4. All checks passed -> return k_rx_ok

Validation does NOT guarantee thread-safety of implementations. ThreadX is thread-safe, but custom implementations may not be.

1.0.0 Initial implementation

Validate that a time interface is properly initialized.

Populates an rx_time_interface_t structure with function pointers to the ThreadX implementation. This enables Dependency Inversion: callers depend on the abstract interface, not the concrete ThreadX API.

Interface functions remain valid for program lifetime

Parameters:

Direction	Name	Description
in	iface	Pointer to time interface to validate May be NULL (will return k_rx_err_null_ptr) Typically points to stack or static rx_time_interface_t
out	iface	Pointer to interface structure to populate Must not be nullptr Caller owns memory Structure is fully initialized on success

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Interface is valid and safe to use

k_rx_err_null_ptr	iface pointer is nullptr (cannot validate)
k_rx_err_invalid_state	Required function pointer(s) are NULL (sleep_ms or get_ms)
k_rx_ok	Success, interface populated
k_rx_err_null_ptr	iface is nullptr

Pre: No preconditions - safe to call with any input (even NULL)

Pre: iface != nullptr

Post: Interface not modified (read-only validation)

Post: Return value indicates whether interface is usable

Post: iface->ctx == nullptr

Post: iface->sleep_ms, get_ms, is_elapsed are valid function pointers

Post: Interface is ready for use

Note: Thread Safety: Thread-safe (read-only operation, no shared state)

Note: Execution Time: 50 ns @ 240 MHz (3 null checks)

Note: Use Case: Call at initialization before using interface

Note: Optional Fields: Does NOT check ctx or is_elapsed (both may be NULL)

Note: Thread Safety: Safe - only writes to caller-owned structure

Note: Re-entrancy: Safe - no shared state accessed

Warning: Does NOT validate that function pointers point to valid functions! Only checks for nullptr. Invalid pointers will cause crash on call.

Example - Defensive Initialization:

```
voidsystem_init(void){ rx_time_interface_ttime_iface;
rx_time_threadx_get_interface(&time_iface);

rx_err_tterr=rx_time_interface_validate(&time_iface); if(err==k_rx_err_null_ptr)
{ rx_log_fatal("INIT","Timeinterfaceisnullptr"); return; }elseif(err==k_rx_err_invalid_state)
{ rx_log_fatal("INIT","Timeinterfacemissingrequiredfunctions"); return; }
```

```
usb_init(&time_iface); spi_init(&time_iface); }
```

Example - Validation in Module Init:: rx_err_tusb_comm_init(constrx_time_interface_t*time)

```
{ rx_err_tterr=rx_time_interface_validate(time); if(err!=k_rx_ok)
{ rx_log_error("USB","Invalidtimeinterface:%d",err); returnerr; }

s_usb_time=time;

returnk_rx_ok; }
```

Example - Unit Test Verification:: voidtest_validate_catches_null_interface(void)

```
{ rx_err_tterr=rx_time_interface_validate(nullptr); assert(err==k_rx_err_null_ptr); }
```

voidtest_validate_catches_missing_sleep(void){ rx_time_interface_ttime={0};

```
time.get_ms=mock_get_ms;

rx_err_tterr=rx_time_interface_validate(&time); assert(err==k_rx_err_invalid_state); }
```

voidtest_validate_allows_null_ctx(void){ rx_time_interface_ttime={0}; time.ctx=nullptr;

```
time.sleep_ms=mock_sleep_ms; time.get_ms=mock_get_ms;

rx_err_tterr=rx_time_interface_validate(&time); assert(err==k_rx_ok); }
```

Example - Production Assertion:: voidcritical_init(constrx_time_interface_t*time)

```
{ rx_err_tterr=rx_time_interface_validate(time);

RX_CHECK(err==k_rx_ok,"Timeinterfacevalidationfailed");

}
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 2: [OK] No loops
Rule 5: [OK] 1 precondition, 2 postconditions Rule 4: [OK] Function is 10 lines (well under 60)

Algorithm Steps:: Validate iface is not nullptr Set ctx to NULL (ThreadX uses global state) Set
function pointers to implementation functions Return success

Interface Population:: Field Value Description

ctx NULL ThreadX uses global state

sleep_ms impl_sleep_ms tx_thread_sleep() wrapper

get_ms impl_get_ms tx_time_get() x 10 wrapper

is_elapsed impl_is_elapsed Wraparound-safe timeout check

Example - Basic Usage:: rx_time_interface_ttime_if;

```
rx_err_tterr=rx_time_threadx_get_interface(&time_if); if(err!=k_rx_ok)
{ rx_log_error("TIME","Failedtogetinterface"); returnerr; }
```

```
uint32_tstart=time_if.get_ms(time_if.ctx); do_some_work();
uint32_tduration=time_if.get_ms(time_if.ctx)-start;
```

Example - Timeout Loop:: rx_time_interface_ttime_if; rx_time_threadx_get_interface(&time_if);

```
uint32_tstart=time_if.get_ms(time_if.ctx); while(!sensor_data_ready())
{ if(time_if.is_elapsed(time_if.ctx,start,500)){ returnk_rx_err_timeout; }
time_if.sleep_ms(time_if.ctx,10); }
```

Example - Dependency Injection:: rx_time_interface_ttime_if;

```
rx_time_threadx_get_interface(&time_if); motor_init(&motor,&config,&time_if);
```



```
rx_time_interface_ttime_if; mock_time_get_interface(&time_if);
motor_init(&motor,&config,&time_if);
```

NASA Power of 10 Compliance: Rule 5: [OK] 1 precondition (NULL check), 3 postconditions Rule 9: [OK] Function pointers for DIP (intentional deviation)

See also: rx_time_interface Structure being validated, rx_time_threadx_get_interface() Produces valid interface, mock_time_get_interface() Produces valid test interface, rx_time_interface_t Interface structure definition, mock_time_get_interface() Unit test mock version, impl_sleep_ms() Sleep implementation details, impl_get_ms() Time retrieval details, impl_is_elapsed() Timeout check details

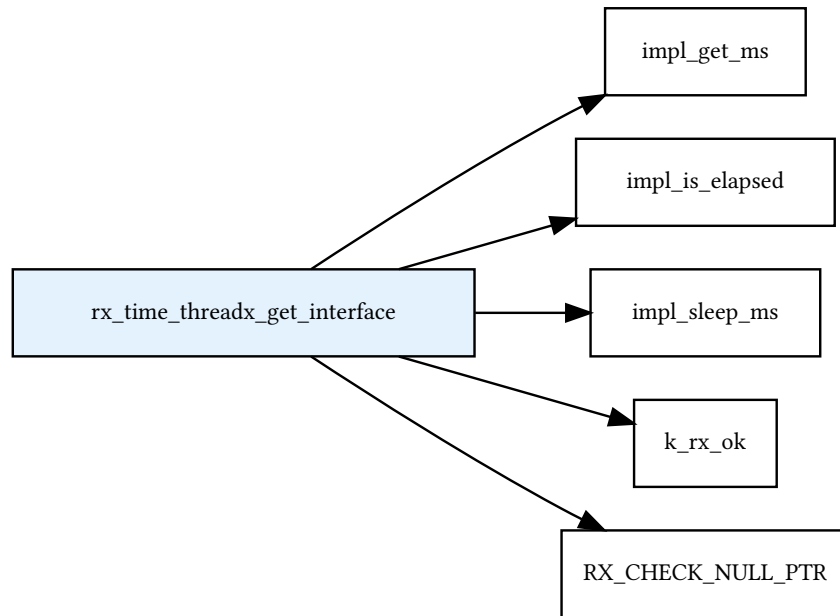


Figure 235: Call graph for rx_time_threadx_get_interface

_Defined in libs/rx_core/inc/rx_time_interface.h:1028_

Since Version 1.0.0

—

rx_time_interface_validate

```
static rx_err_t rx_time_interface_validate(const rx_time_interface_t *iface)
```

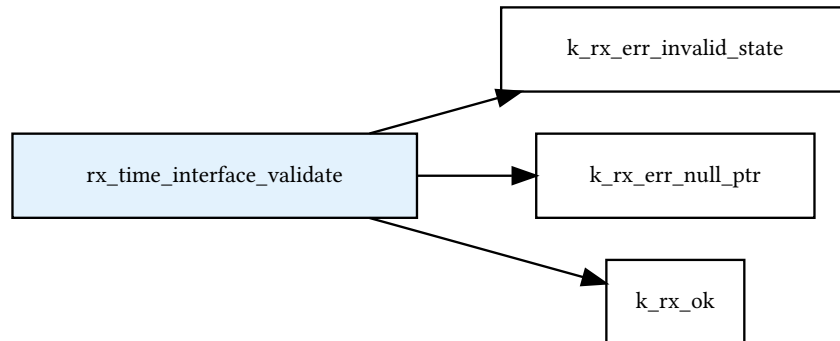


Figure 236: Call graph for rx_time_interface_validate

_Defined in libs/rx\core/inc/rx_time_interface.h:1030_

rx_wdt.h

Watchdog Timer (WDT) Driver for RX72N Microcontroller.

Software-controllable watchdog driver providing flexible system monitoring complementary to the Independent Watchdog (IWDG). Designed for development, debugging, and scenarios requiring runtime watchdog control. Works alongside IWDG to provide defense-in-depth fault detection.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

wdt_timeout_cycles_t

WDT Timeout Period Selection (TOPS bits in WDTCR register).

Defines the four available watchdog timeout periods, corresponding to hardware TOPS[1:0] bits in WDTCR register. Each setting determines the number of divided clock cycles before watchdog timeout.

Value	Name	Description
= 0	k_wdt_timeout_1024_cycles	1024 cycles (TOPS=00). Counter loads 0x03FF.
= 1	k_wdt_timeout_4096_cycles	4096 cycles (TOPS=01). Counter loads 0x0FFF.
= 2	k_wdt_timeout_8192_cycles	8192 cycles (TOPS=10). Counter loads 0x1FFF.
= 3	k_wdt_timeout_16384_cycles	16384 cycles (TOPS=11). Counter loads 0x3FFF. Max timeout.

wdt_clock_division_t

WDT Clock Division Ratio Selection (CKS bits in WDTCR register).

Defines the six available clock division ratios for the WDT. Combined with TOPS (timeout period), this determines the actual watchdog timeout.

Value	Name	Description
= 0x01	k_wdt_clock_div_4	PCLKB / 4 (CKS=0001). Fastest, shortest timeout.
= 0x04	k_wdt_clock_div_64	PCLKB / 64 (CKS=0100).
= 0x0F	k_wdt_clock_div_128	PCLKB / 128 (CKS=1111).
= 0x06	k_wdt_clock_div_512	PCLKB / 512 (CKS=0110).
= 0x07	k_wdt_clock_div_2048	PCLKB / 2048 (CKS=0111).
= 0x08	k_wdt_clock_div_8192	PCLKB / 8192 (CKS=1000). Slowest, longest timeout.

rx_wdt_init

```
rx_err_t rx_wdt_init(const rx_wdt_config_t *config)
```

Initialize the Watchdog Timer (WDT) with specified configuration.

Configures and optionally starts the RX72N Watchdog Timer peripheral. Sets up timeout period (clock divider), reset behavior, and auto-start mode. Once initialized, the WDT can be started/stopped via rx_wdt_start() and rx_wdt_stop().

Algorithm steps:

1. Validate configuration parameters (NULL check, timeout_cycles range)
2. Check if WDT already initialized (prevent double initialization)
3. Configure WDTCR register:

Set CKS[1:0] bits based on timeout_cycles Set RPES bit for reset vs interrupt mode

4. If enable_on_init=true, start WDT counter immediately
5. Store configuration for runtime reference
6. Mark driver as initialized

Hardware Configuration:

- Writes to WDTCR (Watchdog Timer Control Register) at 0x00088020
- Sets CKS bits: timeout_cycles value (0-3)
- Sets RPES bit: reset_on_timeout (1=reset, 0=interrupt)
- If enable_on_init=true, sets WDT counter start bit

Control Flow:

WDT hardware state matches configuration

Only one successful init call allowed (until stop/reset)

Configuration immutable after init (requires re-init to change)

Very short timeouts (us range) may be difficult to meet reliably

Initialize the Watchdog Timer (WDT) with specified configuration.

Configures WDT driver state and optionally starts the hardware counter. If config is nullptr, uses safe defaults (longest timeout, manual start, reset mode).

Algorithm:

1. Check if already initialized (return error if so)
2. If config is nullptr, create default configuration:

timeout_period = k_wdt_timeout_4096_cycles (68us at 60MHz) enable_on_init = false (manual start) reset_on_timeout = true (reset on timeout)

3. Copy configuration to static state
4. Set initialized flag to true
5. Set running flag to false
6. If enable_on_init, call rx_wdt_start()
7. Return k_rx_ok (or propagate start error)

Parameters:

Direction	Name	Description
in	config	Configuration structure pointer. Must point to valid rx_wdt_config_t. timeout_cycles: Must be valid wdt_timeout_cycles_t (0-3) enable_on_init: true=start immediately, false=manual start reset_on_timeout: true=reset on timeout, false=interrupt NULL not allowed (returns k_rx_err_null_ptr)
in	config	Configuration structure pointer, or nullptr for defaults

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, WDT initialized and optionally started
k_rx_err_null_ptr	config parameter is nullptr
k_rx_err_invalid_arg	timeout_cycles out of valid range (0-3)
k_rx_err_invalid_state	WDT already initialized (call rx_wdt_stop first)
k_rx_ok	Success, WDT initialized
k_rx_err_invalid_state	Already initialized

Pre: config must point to valid rx_wdt_config_t structure

Pre: config->timeout_cycles must be in range 0, 3

Pre: WDT must not be already initialized

Pre: System must be in a state that allows WDTCR register writes

Pre: WDT must not already be initialized

Post: If success, WDT is configured with specified settings

Post: If enable_on_init=true, WDT counter is running

Post: If enable_on_init=false, WDT is configured but stopped

Post: Driver marked as initialized, subsequent init calls will fail

Post: `s_wdt_state.initialized == true`

Post: `s_wdt_state.config` contains configuration

Post: If `enable_on_init`, WDT counter is running

Note: Thread-safe - uses mutex protection

Note: This function must be called before any other WDT functions

Note: If `enable_on_init=true`, ensure feed loop is ready before calling

Note: Configuration is copied internally, original can be freed after call

Note: `nullptr` config uses safe defaults (recommended for most cases)

Note: Re-initialization requires system reset (no deinit function)

Warning: WDT cannot be reconfigured while initialized (must stop first)

Warning: If `enable_on_init=true`, `rx_wdt_feed()` must be called within timeout period

Warning: `timeout_cycles` determines actual timeout based on PCLKB frequency

Thread Safety:: Thread-safe. Uses internal mutex to prevent concurrent initialization. Multiple threads can safely call this function (first succeeds, rest fail with `k_rx_err_invalid_state`).

Performance:: Execution time: 20 us @ 240 MHz (register configuration) Memory: No dynamic allocation, configuration copied to static storage Stack usage: < 32 bytes

Re-entrancy:: Not reentrant (uses static state). Concurrent calls are serialized by mutex.

Example (Basic Initialization):: `#include "rx_wdt.h" #include "uart_debug.h"`

```
void system_init(void)
{ rx_wdt_config_t config = { .timeout_cycles = k_wdt_timeout_16384_cycles, .enable_on_init = false, .reset_on_timeout = true };
  rx_err_t err = rx_wdt_init(&config); if (err != k_rx_ok) { uart_debug_puts("[ERROR] WDT init failed\n"); return; }
  uart_debug_puts("[INFO] WDT initialized\n");
  rx_wdt_start(); }
```

Example (Auto-Start for Production)::

```
rx_wdt_config_t prod_config={.timeout_cycles=k_wdt_timeout_16384_cycles,.enable_on_init=true,.reset_on_timeou
rx_err_t err=rx_wdt_init(&prod_config); if(err!=k_rx_ok){ system_fault_handler(); }
```

Example (Error Handling):: rx_wdt_config_t config={0}; rx_err_t err=rx_wdt_init(&config);

```
switch(err){ case k_rx_ok: uart_debug_puts("[INFO]WDTreadyrn"); break; case k_rx_err_null_ptr:
uart_debug_puts("[ERROR]nullptrconfigpointerrn"); break; case k_rx_err_invalid_arg:
uart_debug_puts("[ERROR]Invalidtimeoutperiodrn"); break; case k_rx_err_invalid_state:
uart_debug_puts("[WARN]WDTalreadyinitializedrn"); break; default:
uart_debug_puts("[ERROR]UnknownWDTerrorrn"); break; }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto/setjmp/recursion, sequential control flow

Rule 3: [OK] No dynamic allocation, static configuration storage Rule 5: [OK] 4 preconditions, 4

postconditions Rule 7: [OK] Returns rx_err_t, caller must check

Flow Diagram:: digraph init_flow { rankdir=TB; node [shape=box, style=rounded];

```
start [label="rx_wdt_init(config)"]; check_init [label="Already initialized?", shape=diamond];
err_state [label="Return k_rx_err_invalid_state", fillcolor=lightcoral, style="rounded, filled"];
check_null [label="config == nullptr?", shape=diamond]; use_default [label="Create default config"];
copy_config [label="memcpy config to state"]; set_flags [label="initialized=true running=false"];
check_auto [label="enable_on_init?", shape=diamond]; call_start [label="rx_wdt_start()"]; success
[label="Return k_rx_ok", fillcolor=lightgreen, style="rounded, filled"];
```

```
start -> check_init; check_init -> err_state [label="yes"]; check_init -> check_null [label="no"];
check_null -> use_default [label="yes"]; check_null -> copy_config [label="no"]; use_default ->
copy_config; copy_config -> set_flags; set_flags -> check_auto; check_auto -> call_start
[label="yes"]; check_auto -> success [label="no"]; call_start -> success; }
```

See also: rx_wdt_config_t Configuration structure definition, rx_wdt_start() Start watchdog if enable_on_init=false, rx_wdt_stop() Stop watchdog for reconfiguration, rx_wdt_feed() Feed watchdog to prevent timeout, wdt_timeout_cycles_t Timeout period enum values, rx_wdt_start() Called automatically if enable_on_init=true

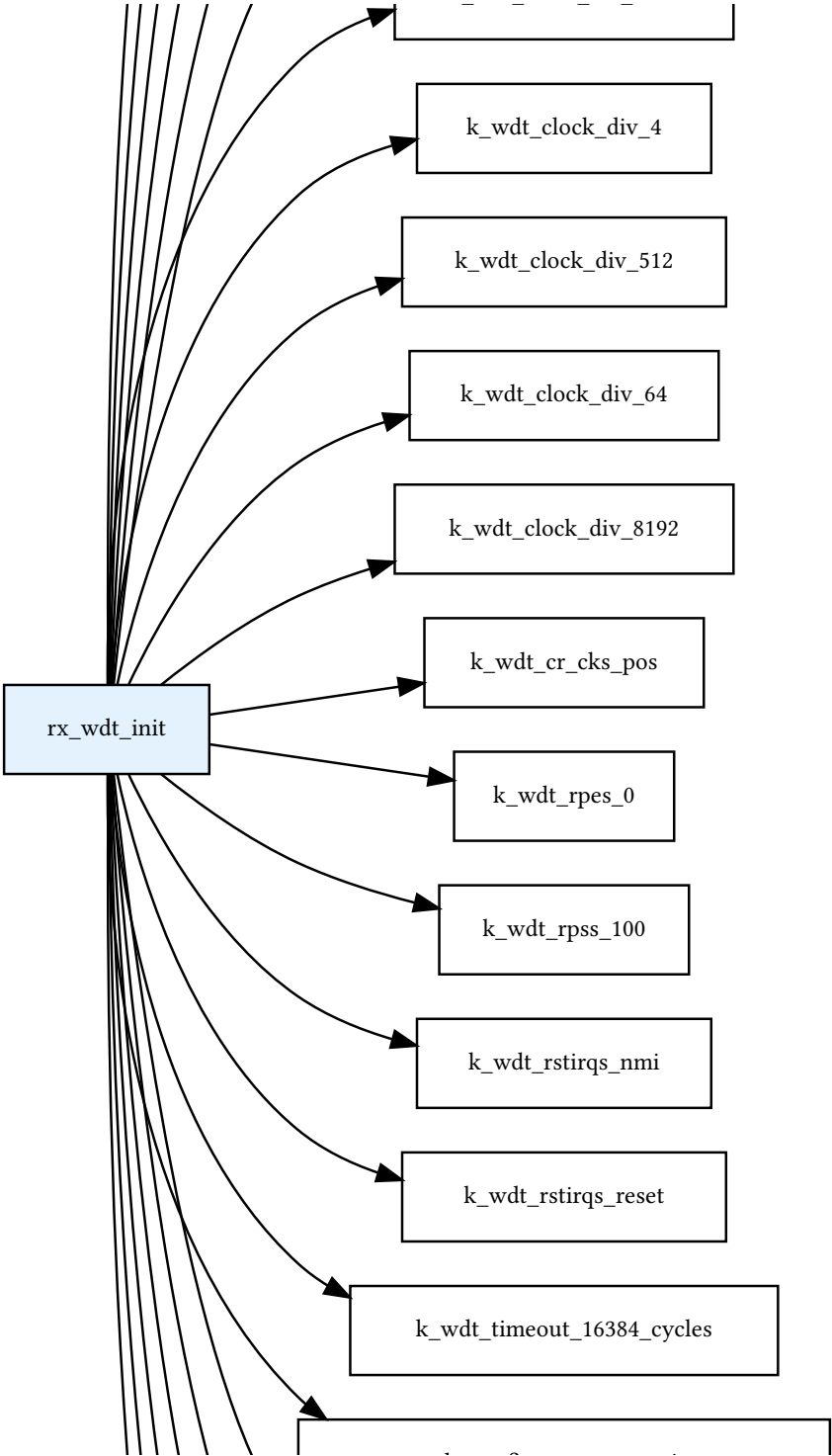


Figure 237: Call graph for rx_wdt_init

Defined in `libs/rx_core/inc/rx_wdt.h:717`

Since Version 1.0.0

—

rx_wdt_start

```
rx_err_t rx_wdt_start(void)
```

Start the WDT counter (enable watchdog monitoring).

Starts the Watchdog Timer counter, beginning countdown to timeout. Once started, `rx_wdt_feed()` must be called periodically to prevent timeout. This function is used when WDT was initialized with `enable_on_init=false`.

Algorithm steps:

1. Check if WDT is initialized (fail if not)
2. Set WDT counter start bit in WDTCR register
3. Begin countdown from configured timeout period
4. Return success

Hardware Operation:

- Writes to WDTCR register (0x00088020)
- Sets TME (Timer Enable) bit to 1
- Counter begins decrementing from timeout period value

WDT configuration unchanged (timeout period, reset mode)

Starting already-started WDT is safe (idempotent)

After starting, missing even ONE feed will cause timeout/reset

Start the WDT counter (enable watchdog monitoring).

Starts the WDT counter by writing the refresh sequence to WDTRR. Once started, `rx_wdt_feed()` must be called periodically to prevent timeout.

Algorithm:

1. Check if WDT is initialized (return error if not)
2. Get pointer to WDT registers via `wdt()` accessor
3. Write start sequence to WDTRR (0x00 then 0xFF)
4. Set `s_wdt_state.running` to true
5. Return `k_rx_ok`

Hardware Operation: The WDTRR refresh/start sequence:

- First write: 0x00 (`k_wdt_refresh_start`)
- Second write: 0xFF (`k_wdt_refresh_end`)
- Sequence resets counter to timeout period value
- Counter begins decrementing immediately

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, WDT counter started
<code>k_rx_err_not_initialized</code>	WDT not initialized via <code>rx_wdt_init()</code>

k_rx_ok	Success, WDT counter started
k_rx_err_not_initialized	WDT not initialized

Pre: WDT must be initialized via rx_wdt_init()

Pre: Feed loop must be ready to call rx_wdt_feed() within timeout period

Pre: WDT must be initialized via rx_wdt_init()

Post: WDT counter is running

Post: rx_wdt_feed() must be called within timeout period to prevent reset

Post: If timeout expires, system will reset (or interrupt if configured)

Post: WDT counter is running and decrementing

Post: s_wdt_state.running == true

Post: rx_wdt_feed() must be called within timeout period

Note: Thread-safe - uses mutex protection

Note: Can be called multiple times (idempotent operation)

Note: Complementary to rx_wdt_stop() for runtime control

Note: In auto-start mode, this function acts as a feed operation

Warning: Ensure rx_wdt_feed() will be called within timeout period before starting

Warning: At 60 MHz PCLKB with max divider, timeout is only 273 us (very short!)

Warning: Missing feeds after start will cause timeout/reset

Thread Safety:: Thread-safe. Uses internal mutex to prevent concurrent start/stop operations.

Performance:: Execution time: 2 us @ 240 MHz (single register write) Memory: No allocation
Stack usage: < 16 bytes

Re-entrancy:: Not reentrant (uses static state). Concurrent calls serialized by mutex.

Example (Staged Startup)::

```
rx_wdt_config_tconfig={.timeout_cycles=k_wdt_timeout_16384_cycles,.enable_on_init=false,.reset_on_timeout=true};
rx_wdt_init(&config);
```

```
rx_err_t err=rx_wdt_start(); if(err!=k_rx_ok){ uart_debug_puts("[ERROR]WDTstartfailedrn"); }
```

Example (Start/Stop Control):: rx_wdt_start(); while(running){ main_loop_work();
rx_wdt_feed(); }

```
rx_wdt_stop(); flash_erase_sector(); rx_wdt_start();
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto/setjmp/recursion Rule 3: [OK] No dynamic allocation Rule 5: [OK] 2 preconditions, 3 postconditions

OFS Register Configuration:: The WDT can be configured at flash-time via OFS (Option Function Select) registers. This function only works if WDT is in “register-start mode”: Register-start mode: WDT starts when WDTRR sequence written Auto-start mode: WDT starts at reset, this function refreshes only

See also: rx_wdt_init() Initialize WDT before starting, rx_wdt_stop() Stop WDT counter, rx_wdt_feed() Feed watchdog to prevent timeout, rx_wdt_init() Must be called first, rx_wdt_feed() Call periodically after start, k_wdt_refresh_start, k_wdt_refresh_end Refresh sequence constants

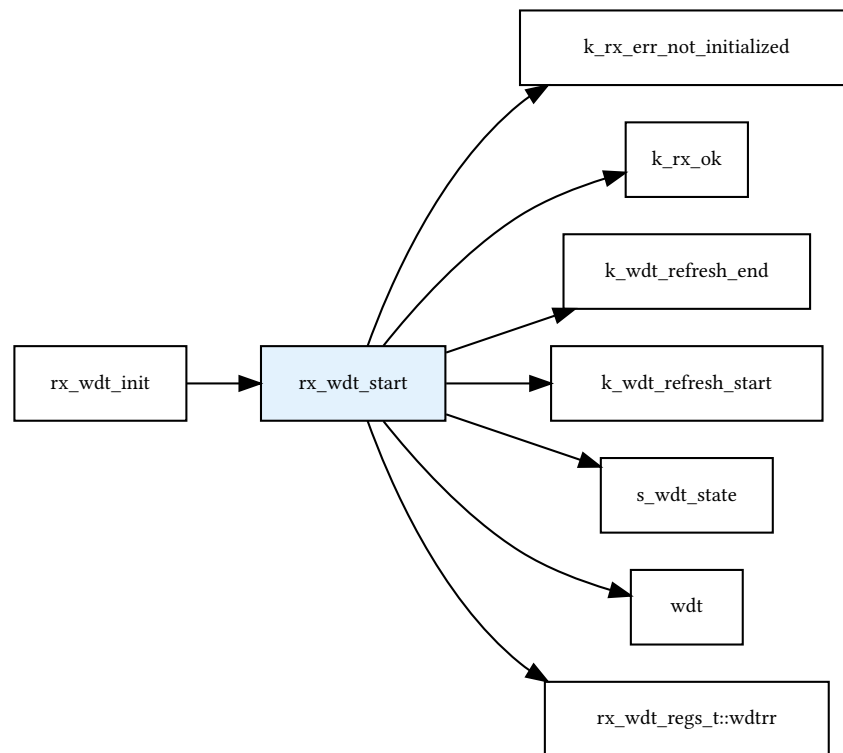


Figure 238: Call graph for rx_wdt_start

Defined in `libs/rx_core/inc/rx_wdt.h:819`

Since Version 1.0.0

—

rx_wdt_stop

```
rx_err_t rx_wdt_stop(void)
```

Stop the WDT counter (disable watchdog monitoring).

Stops the Watchdog Timer counter, preventing timeout. Use this function to suspend watchdog monitoring during long operations (flash updates, debugging) or for safe shutdown. Unlike IWDT, WDT can be stopped and restarted.

Algorithm steps:

1. Check if WDT is initialized (fail if not)
2. Clear WDT counter start bit in WDTCR register
3. Counter stops decrementing (frozen at current value)
4. Return success

Hardware Operation:

- Writes to WDTCR register (0x00088020)
- Clears TME (Timer Enable) bit to 0
- Counter stops, timeout cannot occur

Use Cases:

- Flash updates: Long erase/write operations (milliseconds to seconds)
- Debugging: Freeze counter during breakpoint sessions
- Safe mode: Disable monitoring during error recovery
- Shutdown: Stop watchdog before power-down

WDT configuration unchanged (timeout period, reset mode)

Stopping already-stopped WDT is safe (idempotent)

If using both WDT and IWDT, system still protected by IWDT

Stop the WDT counter (disable watchdog monitoring).

Updates software state to indicate WDT is stopped. Note that this does NOT actually stop the hardware counter if WDT is configured in auto-start mode via OFS registers.

Algorithm:

1. Check if WDT is initialized (return error if not)
2. Set s_wdt_state.running to false
3. Return k_rx_ok

Hardware Limitation: The RX72N WDT hardware behavior depends on OFS (Option Function Select) register configuration at flash time:

- Register-start mode: Hardware can be stopped
- Auto-start mode: Hardware cannot be stopped once running!

This function ONLY updates software state. If using auto-start mode, you must continue calling rx_wdt_feed() even after rx_wdt_stop().

Check OFS configuration to know if hardware can actually stop

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, WDT counter stopped
k_rx_err_not_initialized	WDT not initialized via rx_wdt_init()
k_rx_ok	Success, software state updated
k_rx_err_not_initialized	WDT not initialized

Pre: WDT must be initialized via rx_wdt_init()

Pre: WDT must be initialized via rx_wdt_init()

Post: WDT counter is stopped (not decrementing)

Post: No timeout will occur until rx_wdt_start() called

Post: System is unprotected by WDT until restarted

Post: s_wdt_state.running == false

Post: In auto-start mode, hardware still requires feeding!

Note: Thread-safe - uses mutex protection

Note: Can be called multiple times (idempotent operation)

Note: Complementary to rx_wdt_start() for runtime control

Warning: System is unprotected while WDT stopped (use with caution)

Warning: Remember to call rx_wdt_start() to resume monitoring

Warning: In auto-start mode, this does NOT stop hardware counter

Warning: System may still reset if feeds are stopped

Thread Safety:: Thread-safe. Uses internal mutex to prevent concurrent start/stop operations.

Performance:: Execution time: 2 us @ 240 MHz (single register write) Memory: No allocation
Stack usage: < 16 bytes

Re-entrancy:: Not reentrant (uses static state). Concurrent calls serialized by mutex.

Example (Flash Update):: rx_wdt_stop();

```
rx_err_t err=flash_erase_sector(secto_addr); if(err==k_rx_ok)
{ err=flash_write_page(secto_addr,data,size); }
```

rx_wdt_start();

Example (Debugging Session):: rx_wdt_stop();

```
uart_debug_puts(“[DEBUG]Breakpointlocationrn”);
```

rx_wdt_start();

Example (Safe Shutdown):: voidsystem_shutdown(void)

```
{ uart_debug_puts(“[INFO]Systemshutdowninitiatedrn”);
```

rx_wdt_stop();

```
motor_disable_all(); comm_close_all(); save_state_to_flash();
```

```
enter_low_power_mode(); }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto/setjmp/recursion Rule 3: [OK] No dynamic allocation Rule 5: [OK] 1 precondition, 3 postconditions

See also: rx_wdt_init() Initialize WDT before stopping, rx_wdt_start() Restart WDT after stopping, rx_iwdt.h Independent watchdog that cannot be stopped, rx_wdt_start() Restart WDT after stopping

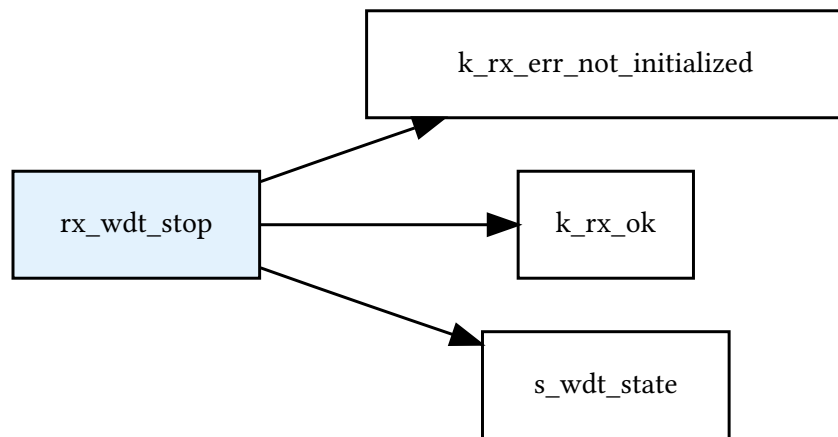


Figure 239: Call graph for rx_wdt_stop

Defined in `libs/rx_core/inc/rx_wdt.h:936`

Since Version 1.0.0

rx_wdt_feed

```
rx_err_t rx_wdt_feed(void)
```

Feed the watchdog timer (reset counter to prevent timeout).

Resets the WDT counter to its initial value, preventing timeout. This function must be called periodically at a rate faster than the configured timeout period. Typically called from main control loop or dedicated watchdog monitoring task.

Algorithm steps:

1. Check if WDT is initialized (fail if not)
2. Write refresh sequence to WDTRR register (0x00 then 0xFF)
3. Counter resets to initial timeout period value
4. Countdown resumes from reset value
5. Return success

Hardware Operation:

- Writes two-byte sequence to WDTRR (Watchdog Timer Refresh Register):

First write: 0x00 Second write: 0xFF

- Sequence must be written in correct order (hardware validation)
- Counter resets to timeout_cycles value

Feed Requirements:

- Frequency: Must call faster than timeout period
- Example: With k_wdt_timeout_16384_cycles at 60 MHz PCLKB:

Timeout = 273 us Feed frequency > 3.6 kHz (every 273 us) Recommended: Feed at 2x margin = 130 us interval

Counter value after feed equals timeout_cycles configuration value

Configuration unchanged (timeout period, reset mode)

Do NOT call from multiple locations - defeats failure detection

Feed the watchdog timer (reset counter to prevent timeout).

Resets the WDT counter to its initial value by writing the refresh sequence to WDTRR. This must be called periodically at a rate faster than the configured timeout period.

Algorithm:

1. Check if WDT is initialized (return error if not)
2. Check if WDT is running (return error if not)
3. Get pointer to WDT registers via wdt() accessor
4. Write refresh sequence: 0x00 then 0xFF to WDTRR
5. Return k_rx_ok

Hardware Operation: The WDTRR refresh sequence must be written in exact order:

- First write: 0x00 (k_wdt_refresh_start)
- Second write: 0xFF (k_wdt_refresh_end)

Invalid sequences (wrong order, wrong values) are ignored by hardware. Valid sequence resets counter to timeout period value.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, WDT counter reset to initial value
k_rx_err_not_initialized	WDT not initialized via rx_wdt_init()
k_rx_ok	Success, WDT counter reset

k_rx_err_not_initialized	WDT not initialized
k_rx_err_invalid_state	WDT not running (stopped or never started)

Pre: WDT must be initialized via rx_wdt_init()

Pre: WDT must be started via rx_wdt_start() or enable_on_init=true

Pre: Called from location that executes faster than timeout period

Pre: WDT must be initialized via rx_wdt_init()

Pre: WDT must be running (rx_wdt_start() called or enable_on_init=true)

Post: WDT counter reset to initial timeout period value

Post: Countdown timer begins again from reset value

Post: Timeout prevented (for this cycle)

Post: WDT counter reset to timeout period value

Post: Countdown restarts from reset value

Note: Thread-safe - can be called from multiple threads (mutex protected)

Note: ISR-safe - can be called from interrupt context (optimized fast path)

Note: Calling when WDT stopped has no effect (safe but wasteful)

Note: This is the ONLY function that prevents watchdog timeout

Note: This is the ONLY function that prevents watchdog timeout

Note: Execution time: 1 us (two register writes)

Warning: Missing even ONE feed within timeout period will cause reset

Warning: Feed frequency must account for worst-case execution time variations

Warning: Very short timeouts (273 us max) require very frequent feeds

Warning: Missing a single feed within timeout period causes reset

Thread Safety:: Thread-safe. Uses atomic register writes (no mutex needed for performance). Multiple threads can safely call this function.

Performance:: Execution time: 1 us @ 240 MHz (two register writes) Memory: No allocation Stack usage: < 8 bytes Fastest WDT operation

Re-entrancy:: Fully reentrant. Uses only register writes, no static state modification.

Example (Main Loop):: void main_loop(void){ while(1){ motor_update(); comm_process(); sensor_read(); rx_err_t err=rx_wdt_feed(); if(err!=k_rx_ok){ uart_debug_puts("[ERROR]WDTfeedfailedrn"); } delay_us(100); } }

Example (Dedicated Watchdog Task):: void watchdog_task(ULONG input){ (void)input; while(1){ rx_wdt_feed(); tx_thread_sleep(1); } }

Example (Timed Feed with Margin):: const uint32_t timeout_us=273; const uint32_t safety_margin=2; const uint32_t feed_interval_us=timeout_us/safety_margin; void main_loop(void){ uint32_t last_feed_time=get_time_us(); while(1){ main_loop_work(); uint32_t now=get_time_us(); if((now-last_feed_time)>=feed_interval_us){ rx_wdt_feed(); last_feed_time=now; } } }

Example (Error Detection):: rx_err_t err=rx_wdt_feed(); if(err==k_rx_err_not_initialized){ uart_debug_puts("[ERROR]WDTnotinitialized-cannotfeedrn"); system_fault_handler(); }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto/setjmp/recursion Rule 3: [OK] No dynamic allocation Rule 4: [OK] Function < 10 lines (simple register write) Rule 5: [OK] 3 preconditions, 3 postconditions

Timing Requirements:: Feed must occur faster than timeout period: Timeout Setting Cycles At 60MHz Required Feed Rate

k_wdt_timeout_256_cycles 256 4.3 us >233 kHz

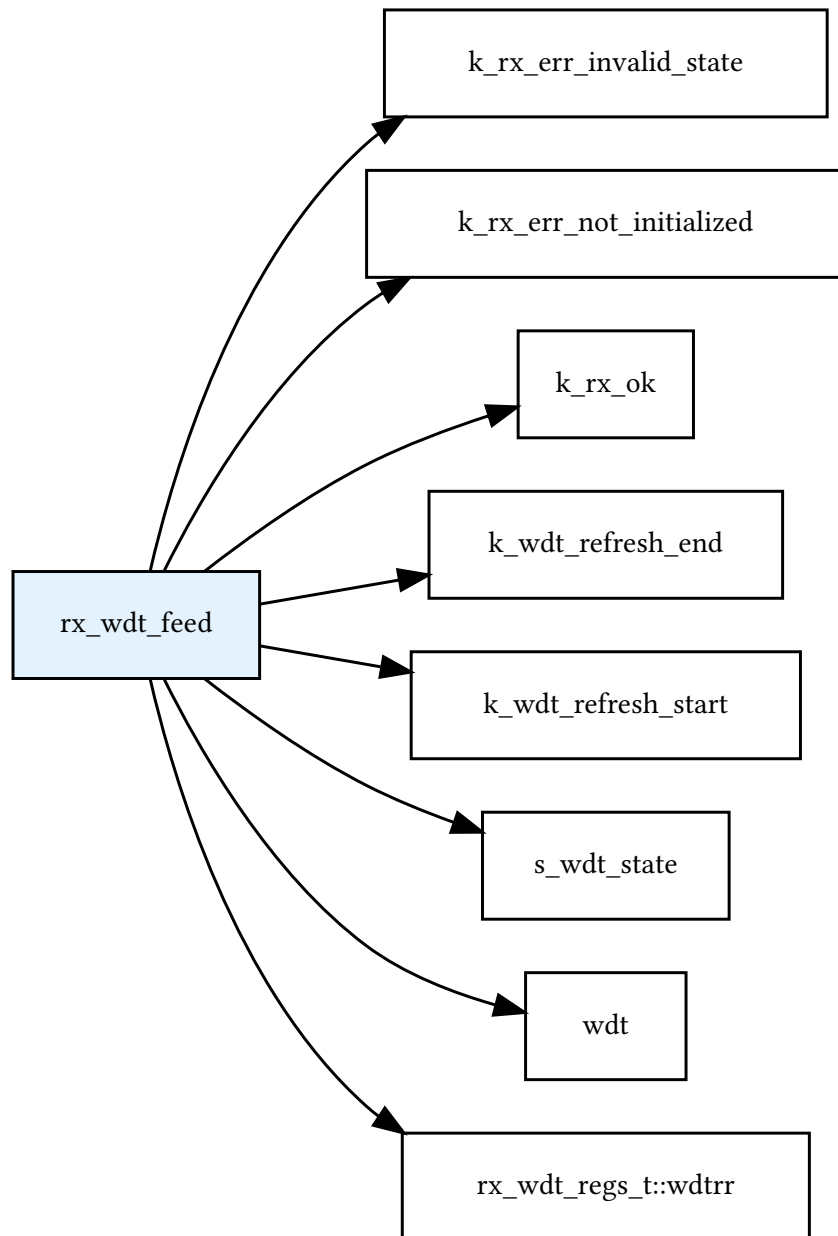
k_wdt_timeout_1024_cycles 1024 17 us >59 kHz

k_wdt_timeout_4096_cycles 4096 68 us >14.7 kHz

k_wdt_timeout_16384_cycles 16384 273 us >3.6 kHz

Example:: while(1){ do_work(); rx_err_t err=rx_wdt_feed(); if(err!=k_rx_ok){ handle_wdt_error(err); } delay_us(100); }

See also: rx_wdt_init() Initialize WDT before feeding, rx_wdt_start() Start WDT to enable timeout, rx_wdt_stop() Stop WDT to disable timeout, wdt_timeout_cycles_t Timeout periods that determine feed frequency, rx_wdt_start() Must be called before feed, wdt_timeout_period_t Timeout period settings

Figure 240: Call graph for `rx_wdt_feed`

Defined in `libs/rx_core/inc/rx_wdt.h:1090`

Since Version 1.0.0

—

rx_error_handler.c

Error Handler Concrete Implementation.

Implements the `rx_error_interface_t` abstract interface with concrete error tracking, retry logic, and exponential backoff capabilities. This module provides a centralized error handling mechanism for the STAR firmware, enabling consistent error reporting, component-level error tracking, and intelligent retry strategies.

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_error_handler.h"
- <string.h>
- "rx_check.h"
- "rx_log.h"

error_handler_backoff_constants_t

Internal constants for exponential backoff algorithm.

These constants control the exponential backoff behavior used by `impl_get_backoff_delay()`. The algorithm doubles the delay on each retry up to the configured maximum.

`k_exponential_backoff_multiplier` must be 2 for binary exponential

`k_error_handler_max_retries` bounds all retry loops (NASA Rule 2)

Value	Name	Description
= 2	<code>k_exponential_backoff_multiplier</code>	Exponential backoff multiplier (base 2).
= 0	<code>k_error_handler_no_retry_limit</code>	Sentinel value indicating unlimited retries.
= 1	<code>k_error_handler_first_retry</code>	Starting index for retry loop iteration.
= 32	<code>k_error_handler_max_retries</code>	Maximum retry iterations for backoff calculation (NASA Rule 2).

Since Version 1.0.0

—

internal_find_component

```
static error_component_state_t * internal_find_component(error_handler_t
*handler, const char *component)
```

Find component tracking slot by name.

Searches the handler's component array for an existing entry matching the given component name. Uses bounded string comparison with `k_error_handler_component_name_max` limit.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	handler	Pointer to error handler instance (must not be NULL)
in	component	Component name to search for (must not be NULL)

Returns: Pointer to matching component state slot

Value	Description
non-NULL	Pointer to found component entry
NULL	Component not found in tracking array

Pre: handler != nullptr (caller responsibility, not validated)

Pre: component != nullptr (caller responsibility, not validated)

Pre: Caller holds handler->mutex (thread safety)

Post: Return value is either NULL or points to valid components entry

Post: Handler state unchanged (read-only operation)

Note: Not thread-safe; caller must hold mutex

Note: Time complexity: $O(n)$ where $n = k_error_handler_max_components$ **Algorithm:** Iterate through all $k_error_handler_max_components$ slots For each slot marked in_use, compare name strings Return pointer to first matching slot, or nullptr if not found

Performance: Best case: $O(1)$ if first slot matches Worst case: $O(n)$ full scan, $n=16$ typical 2-10 us depending on array occupancy

See also: internal_find_or_create_component() Creates entry if not found, $k_error_handler_max_components$ Maximum tracked components

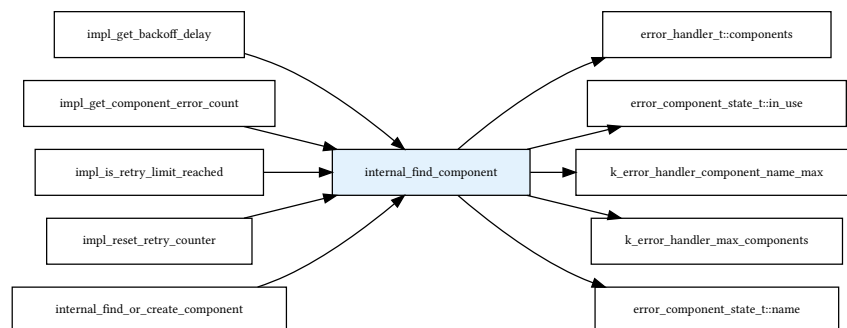


Figure 241: Call graph for `internal_find_component`

_Defined in `libs/rx_core/src/rx_error_handler.c:275_`

Since Version 1.0.0

—

internal_find_or_create_component

```
static error_component_state_t *  
internal_find_or_create_component(error_handler_t *handler, const char  
*component)
```

Find existing component entry or create new one.

First attempts to find an existing component entry via `internal_find_component()`. If not found, searches for an empty slot (`in_use == false`) and initializes it with the component name and zeroed counters.

Total `in_use` entries $\leq$ `k_error_handler_max_components`

Parameters:

Direction	Name	Description
in	handler	Pointer to error handler instance (must not be NULL)
in	component	Component name to find or create (must not be NULL)

Returns: Pointer to component state slot (existing or newly created)

Value	Description
non-NULL	Pointer to component entry (may be new or existing)
NULL	All component slots are in use (array full)

Pre: handler != nullptr (caller responsibility, not validated)

Pre: component != nullptr (caller responsibility, not validated)

Pre: Caller holds handler->mutex (thread safety)

Post: If return != nullptr, component entry exists and is marked `in_use`

Post: If new entry created, `error_count` and `retry_count` are 0

Post: Component name is null-terminated in entry

Note: Not thread-safe; caller must hold mutex

Note: Name truncated to `k_error_handler_component_name_max - 1` chars

Warning: Returns NULL when component array is full - caller must handle] **Algorithm:** Call `internal_find_component()` to search for existing entry. If found, return pointer to existing entry. Otherwise, scan for first empty slot (`in_use == false`). Initialize empty slot: copy name, zero counters, set `in_use = true`. Return pointer to newly initialized slot, or `nullptr` if array full.

Performance: Best case: $O(1)$ if component found in first slot. Worst case: $O(2n)$ two full scans (find + create). 5-20 us typical.

See also: `internal_find_component()` Used internally for search, `k_error_handler_max_components` Maximum trackable components.

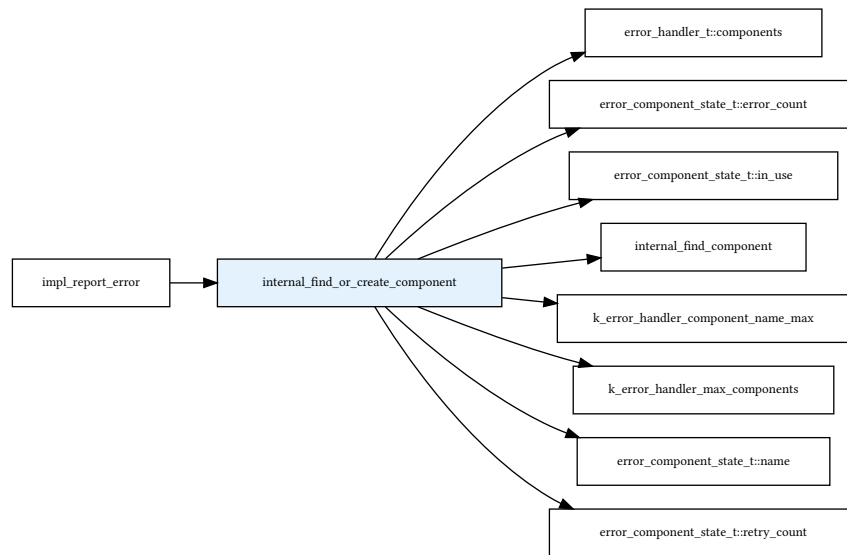


Figure 242: Call graph for `internal_find_or_create_component`

_Defined in `libs/rx\core/src/rx\_error\_handler.c:358_`

Since Version 1.0.0

—

impl_report_error

```
static rx_err_t impl_report_error(void *ctx, rx_err_t err, const char *component,
const char *message)
```

Report error implementation (`rx_error_interface_t.report_error`).

Records an error occurrence for the specified component. Increments both the total error count and the component-specific error and retry counters. The error is also logged via `rx_log_error()`.

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to <code>error_handler_t*</code>)

in	err	Error code being reported
in	component	Component name (e.g., "MOTOR", "USB", "SPI") Max length: k_error_handler_component_name_max - 1 characters Truncated if longer
in	message	Human-readable error message

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Error recorded successfully
k_rx_err_null_ptr	ctx, component, or message is nullptr
k_rx_err_invalid_state	Handler not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: ctx points to initialized error_handler_t

Pre: component is a valid null-terminated string

Pre: message is a valid null-terminated string

Post: total_error_count incremented by 1

Post: Component's error_count and retry_count incremented by 1

Note: Thread-safe: protected by internal mutex

Note: Component created automatically if not exists (up to max slots)

Note: If component array full, error still logged but not tracked per-component] **Algorithm:** Validate input pointers (NULL check) Verify handler is initialized Acquire mutex for thread-safe access Increment total_error_count Find or create component tracking entry If component entry exists, increment error_count and retry_count Release mutex Log error message and error code

Performance: 5-50 us depending on component lookup and logging overhead

See also: impl_clear_errors() Reset all error counters, impl_reset_retry_counter() Reset only retry counter

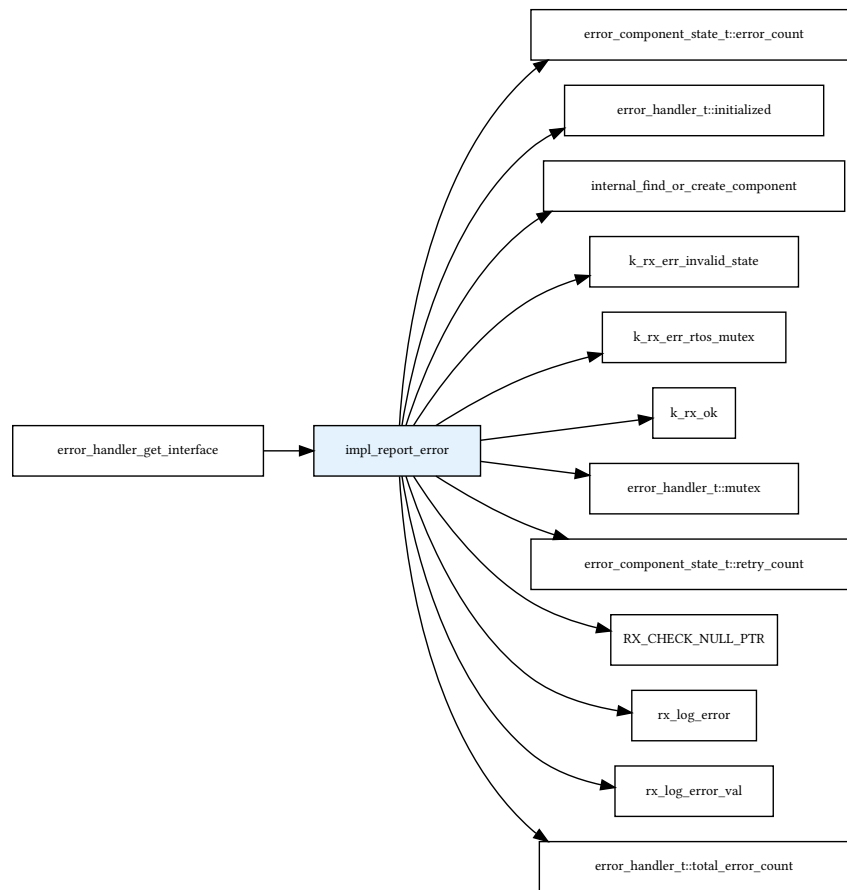


Figure 243: Call graph for impl_report_error

_Defined in libs/rx_core/src/rx_error_handler.c:470_

Since Version 1.0.0

—

impl_get_error_count

```
static uint32_t impl_get_error_count(void *ctx)
```

Get total error count implementation (rx_error_interface_t.get_error_count).

Returns the total number of errors reported across all components since initialization or last clear_errors() call.

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to error_handler_t*)

Returns: Total error count across all components

Value	Description
0	If ctx is nullptr, not initialized, or mutex fails
\>0	Actual total error count

Pre: ctx points to initialized error_handler_t (for meaningful result)

Post: Handler state unchanged (read-only operation)

Note: Thread-safe: protected by internal mutex

Note: Returns 0 on any error condition (fail-safe behavior)] **Algorithm:** Validate context pointer and initialization state Acquire mutex Read total_error_count Release mutex Return count (or 0 on any failure)

Performance: 2-10 us (mutex overhead only)

See also: impl_get_component_error_count() Per-component error count, impl_clear_errors() Reset total count

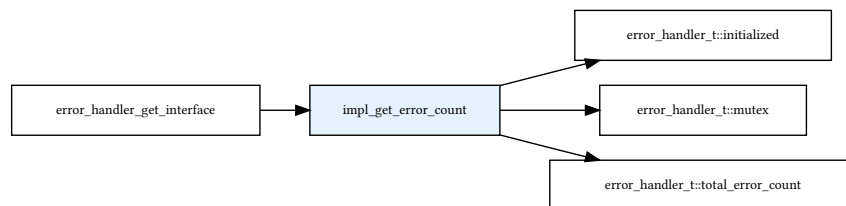


Figure 244: Call graph for impl_get_error_count

_Defined in libs/rx\core/src/rx\error_handler.c:543_

Since Version 1.0.0

—

impl_get_component_error_count

```
static uint32_t impl_get_component_error_count(void *ctx, const char *component)
```

Get component-specific error count (rx_error_interface_t.get_component_error_count).

Returns the number of errors reported for a specific component. Returns 0 if the component has never reported an error (not tracked).

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to error_handler_t*)

in	component	Component name to query
----	-----------	-------------------------

Returns: Error count for the specified component

Value	Description
0	If ctx/component NULL, not initialized, component not found, or mutex fails
\>0	Actual error count for the component

Pre: ctx points to initialized error_handler_t (for meaningful result)

Pre: component is a valid null-terminated string

Post: Handler state unchanged (read-only operation)

Note: Thread-safe: protected by internal mutex

Note: Component name comparison is bounded by k_error_handler_component_name_max]

Algorithm: Validate context and component pointers Verify handler is initialized Acquire mutex Search for component via internal_find_component() If found, read error_count; otherwise use 0 Release mutex Return count

Performance: 3-15 us (mutex + component search)

See also: impl_get_error_count() Total error count, impl_clear_errors() Reset component counts

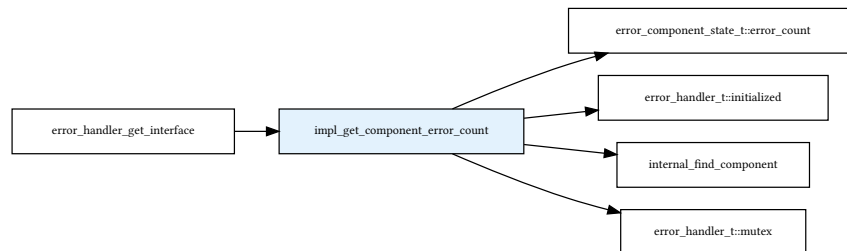


Figure 245: Call graph for impl_get_component_error_count

_Defined in libs/rx\core/src/rx_error_handler.c:607_

Since Version 1.0.0

impl_clear_errors

```
static rx_err_t impl_clear_errors(void *ctx)
```

Clear all error counters (rx_error_interface_t.clear_errors).

Resets the total error count and all component-specific error and retry counters to zero. Component tracking entries remain valid (`in_use` stays true) but their counts are zeroed.

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to <code>error_handler_t*</code>)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	All counters cleared successfully
<code>k_rx_err_null_ptr</code>	ctx is nullptr
<code>k_rx_err_invalid_state</code>	Handler not initialized
<code>k_rx_err_rtos_mutex</code>	Mutex acquisition failed

Pre: ctx points to initialized `error_handler_t`

Post: `total_error_count == 0`

Post: All component `error_count` and `retry_count == 0`

Post: Component `in_use` flags unchanged (entries still tracked)

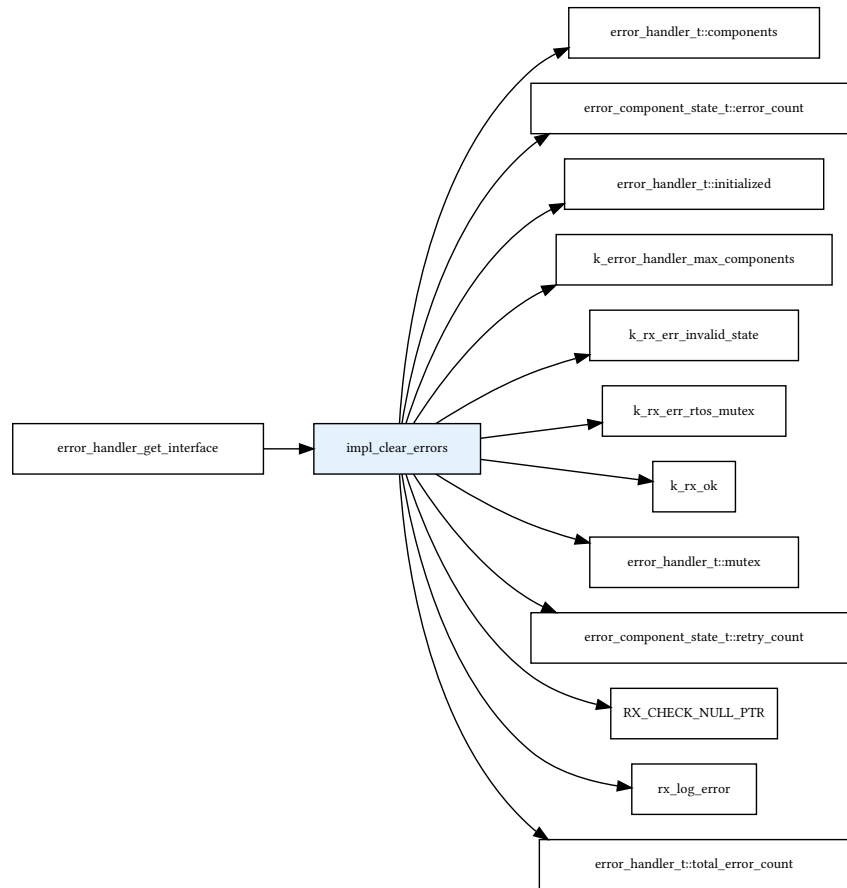
Note: Thread-safe: protected by internal mutex

Note: Does NOT remove component entries, only clears their counts

Warning: Clears `retry_count` too, affecting backoff calculations] **Algorithm:** Validate context pointer Verify handler is initialized Acquire mutex Set `total_error_count = 0` For each component slot: set `error_count = 0`, `retry_count = 0` Release mutex

Performance: 5-30 us (mutex + loop over all slots)

See also: `impl_reset_retry_counter()` Reset only one component's retry counter

Figure 246: Call graph for `impl_clear_errors`

Defined in `libs/rx_core/src/rx_error_handler.c:678`

Since Version 1.0.0

—

`impl_is_retry_limit_reached`

```
static bool impl_is_retry_limit_reached(void *ctx, const char *component)
```

Check if component has reached retry limit (`rx_error_interface_t.is_retry_limit_reached`).

Determines whether the specified component has exceeded its configured maximum retry count.

Used to implement fail-fast behavior after repeated errors.

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to <code>error_handler_t*</code>)

in	component	Component name to check
----	-----------	-------------------------

Returns: bool Retry limit status

Value	Description
true	Retry limit reached, or error condition (fail-safe)
false	Retries still available, or unlimited retries configured

Pre: ctx points to initialized error_handler_t

Pre: component is a valid null-terminated string

Post: Handler state unchanged (read-only operation)

Note: Thread-safe: protected by internal mutex

Note: Fail-safe: returns true (limit reached) on any error condition

Note: If max_retries == 0, always returns false (unlimited)

Note: If component not found, returns false (no retries yet)] **Algorithm:** Validate context and component pointers (return true on failure - fail-safe) If max_retries == 0, return false (unlimited retries) Acquire mutex Find component via internal_find_component() Compare retry_count against max_retries Release mutex Return comparison result

Performance: 3-15 us (mutex + component search)

See also: impl_reset_retry_counter() Reset retry count after recovery,
impl_get_backoff_delay() Get delay before next retry

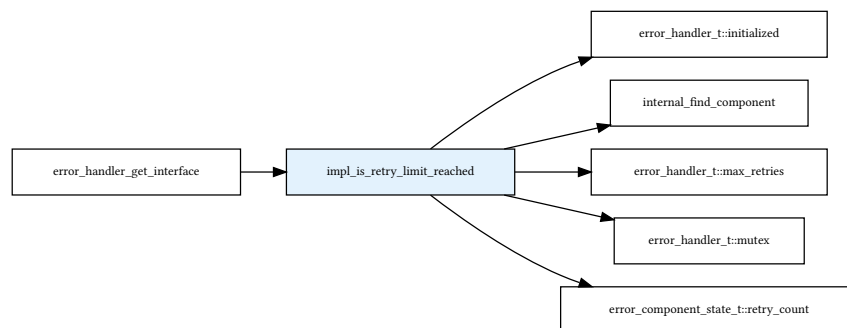


Figure 247: Call graph for `impl_is_retry_limit_reached`

_Defined in `libs/rx\core/src/rx\_error\_handler.c:751_`

Since Version 1.0.0

impl_reset_retry_counter

```
static rx_err_t impl_reset_retry_counter(void *ctx, const char *component)
```

Reset component retry counter (rx_error_interface_t.reset_retry_counter).

Resets only the retry counter for a specific component, leaving the error count unchanged. Call this after successful recovery to enable fresh retries.

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to error_handler_t*)
in	component	Component name to reset

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Retry counter reset (or component not found - no-op)
k_rx_err_null_ptr	ctx or component is nullptr
k_rx_err_invalid_state	Handler not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: ctx points to initialized error_handler_t

Pre: component is a valid null-terminated string

Post: If component found, retry_count == 0

Post: error_count unchanged (only retry reset)

Note: Thread-safe: protected by internal mutex

Note: No-op if component not found (returns k_rx_ok)

Note: Use after successful operation to reset backoff] **Algorithm:** Validate context and component pointers Verify handler is initialized Acquire mutex Find component via internal_find_component() If found, set retry_count = 0 Release mutex

Example: if(usb_operation_succeeded){ iface->reset_retry_counter(iface->ctx,"USB"); }

Performance: 3-15 us (mutex + component search)

See also: impl_clear_errors() Reset all counters for all components,
impl_get_backoff_delay() Retry count affects backoff

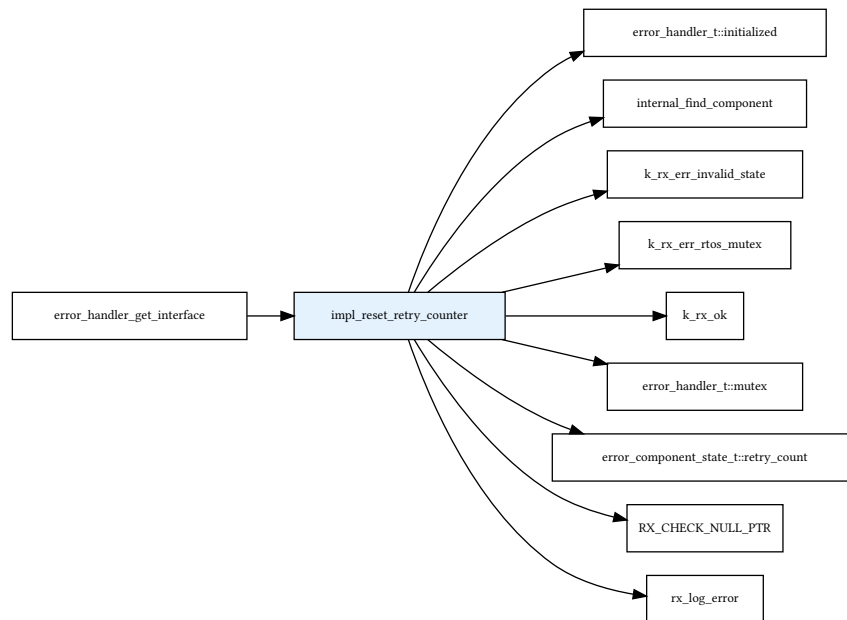


Figure 248: Call graph for impl_reset_retry_counter

_Defined in libs/rx_core/src/rx_error_handler.c:835_

Since Version 1.0.0

—

impl_get_backoff_delay

```
static uint32_t impl_get_backoff_delay(void *ctx, const char *component)
```

Calculate exponential backoff delay (rx_error_interface_t.get_backoff_delay).

Computes the delay (in milliseconds) to wait before the next retry attempt using binary exponential backoff. Delay doubles with each retry, capped at the configured maximum.

Return value <= max_backoff_ms (when successful)

Parameters:

Direction	Name	Description
in	ctx	Opaque context pointer (cast to error_handler_t*)
in	component	Component name to calculate backoff for

Returns: Backoff delay in milliseconds

Value	Description
0	If ctx/component NULL, not initialized, component not found, retry_count == 0, or mutex fails

\>0	Computed backoff delay (up to max_backoff_ms)
-----	---

Pre: ctx points to initialized error_handler_t

Pre: component is a valid null-terminated string

Post: Handler state unchanged (read-only operation)

Note: Thread-safe: protected by internal mutex

Note: Returns 0 for first attempt (retry_count == 0)

Note: Loop bounded by k_error_handler_max_retries (NASA Rule 2)] **Algorithm:** Validate context and component pointers (return 0 on failure) Acquire mutex Find component via internal_find_component() If retry_count == 0, delay = 0 (no backoff on first attempt) Otherwise, compute: delay = initial x 2^(retry_count - 1) Cap delay at max_backoff_ms Release mutex Return computed delay

Exponential Backoff Formula: delay = min(initial_backoff_ms x 2^(n-1), max_backoff_ms) where n = retry_count (capped at 32 to prevent overflow)

Example: uint32_t delay = iface->get_backoff_delay(iface->ctx, "SPI"); if (delay > 0) { tx_thread_sleep(delay * TX_TIMER_TICKS_PER_MS); }

Performance: 3-20 us (mutex + search + backoff loop)

See also: impl_reset_retry_counter() Reset to clear backoff, impl_report_error() Increments retry_count

Figure 249: Call graph for `impl_get_backoff_delay`

Defined in `libs/rx\core/src/rx\_error\_handler.c:960`

Since Version 1.0.0

—

error_handler_init

```
rx_err_t error_handler_init(error_handler_t *handler, const
error_handler_config_t *config)
```

Initialize error handler with retry logic and exponential backoff.

Initialize error handler with specified configuration.

Initializes an error handler instance with the specified configuration. Creates a ThreadX mutex for thread-safe operation and clears all internal state. Must be called before any other error handler operations.

Parameters:

Direction	Name	Description
-----------	------	-------------

out	handler	Pointer to error handler instance to initialize Must be valid non-nullptr Will be completely overwritten (memset) Caller maintains ownership and lifetime
in	config	Configuration parameters max_retries: Maximum retries before limit (0 = unlimited) initial_backoff_ms: Starting backoff delay max_backoff_ms: Maximum backoff delay cap

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Handler initialized successfully
k_rx_err_null_ptr	handler or config is nullptr
k_rx_err_invalid_arg	max_backoff_ms < initial_backoff_ms, or initial_backoff_ms > 0 with max_backoff_ms == 0
k_rx_err_rtos_mutex	ThreadX mutex creation failed

Pre: handler points to allocated error_handler_t (not previously initialized)

Pre: config points to valid configuration with consistent values

Post: handler->initialized == true

Post: handler->mutex created and ready

Post: All component slots cleared (in_use == false)

Post: total_error_count == 0

Note: Not thread-safe: do not call concurrently for same handler

Note: Call error_handler_deinit() before reinitializing

Warning: Do not initialize same handler twice without deinit] **Algorithm:** Validate handler and config pointers Validate backoff configuration consistency Clear all handler state (memset to 0) Copy configuration values Create ThreadX mutex Set initialized flag Log initialization success

Example: staticerror_handler_tg_error_handler;

```
voidsystem_init(void)
{ error_handler_config_tconfig={ .max_retries=5, .initial_backoff_ms=100, .max_backoff_ms=5000 };
rx_err_terr=error_handler_init(&g_error_handler,&config); if(err!=k_rx_ok){ while(1){} } }
```

Performance: 50-100 us (memset + mutex creation)

See also: `error_handler_deinit()` Cleanup handler, `error_handler_get_interface()` Get interface for dependency injection

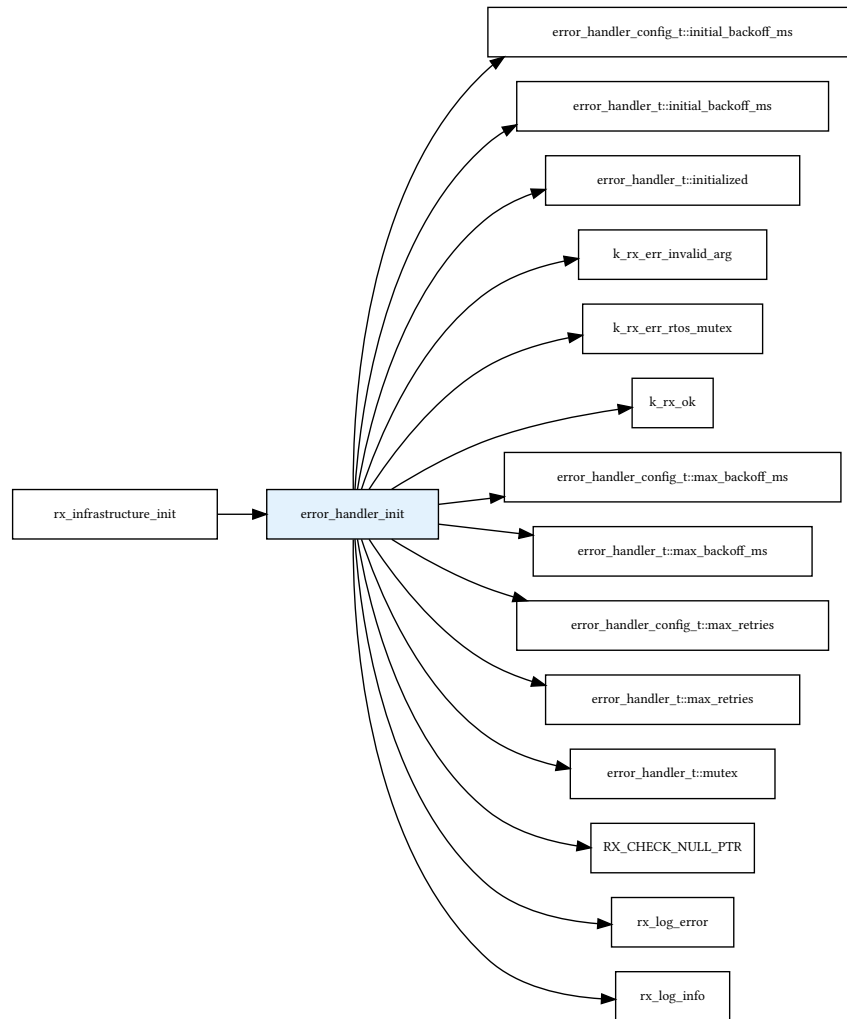


Figure 250: Call graph for `error_handler_init`

_Defined in `libs/rx\_core/src/rx\_error\_handler.c:1120_`

Since Version 1.0.0

error_handler_get_interface

```
rx_err_t error_handler_get_interface(rx_error_interface_t *iface, error_handler_t
*handler)
```

Get error handler interface for dependency injection.

Get abstract interface from concrete error handler (Dependency Inversion).

Populates an `rx_error_interface_t` structure with function pointers to the handler's implementation functions. This enables dependency injection following the Dependency Inversion Principle (DIP).

Parameters:

Direction	Name	Description
out	<code>iface</code>	Pointer to interface structure to populate All fields will be overwritten Caller maintains ownership of the structure
in	<code>handler</code>	Pointer to initialized error handler instance Must be initialized via <code>error_handler_init()</code> Must remain valid for lifetime of interface usage

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Interface populated successfully
<code>k_rx_err_null_ptr</code>	<code>iface</code> or <code>handler</code> is nullptr
<code>k_rx_err_invalid_state</code>	handler not initialized

Pre: `iface` points to allocated `rx_error_interface_t`

Pre: handler initialized via `error_handler_init()`

Post: `iface->ctx == handler`

Post: All function pointers set to implementation functions

Note: Handler must remain valid while interface is in use

Note: Interface structure is a shallow copy (pointers only)] **Algorithm:** Validate `iface` and handler pointers Verify handler is initialized Populate interface structure: `ctx = handler` (opaque context) Function pointers to `impl_*` functions

Interface Binding Diagram: digraph interface_binding { rankdir=LR; node [shape=record]; iface [label="rx_error_interface_t|ctx|report_error|get_error_count|get_component_error_count|clear_errors|is_retry_limit_reached|reset_retry_counter|get_backoff_delay"]; handler [label="error_handler_t|mutex|components[]|total_error_count|config..."]; impl [label="Implementation|impl_report_error()|impl_get_error_count()|impl_get_component_error_count()|impl_clear_errors()|impl_is_retry_limit_reached()|impl_reset_retry_counter()|impl_get_backoff_delay()"]; iface:ctx -> handler [label="points to"]; iface:report_error -> impl:impl_report_error; iface:get_error_count -> impl:impl_get_error_count; }

Example: `staticerror_handler_tg_error_handler; staticrx_error_interface_tg_error_iface;`

```
voidsystem_init(void){ error_handler_config_tconfig={...};
error_handler_init(&g_error_handler,&config);
```

```

error_handler_get_interface(&g_error_iface,&g_error_handler);
motor_init(&motor,&g_error_iface); comm_init(&comm,&g_error_iface); }

void motor_report_fault(motor_t* motor){ motor->error_iface->report_error( motor->error_iface-
>ctx, k_rx_err_hw_fault, "MOTOR", "Overcurrentdetected" ); }

```

Performance: 1 us (pointer assignments only)

See also: error_handler_init() Initialize handler first, rx_error_interface_t Abstract interface definition

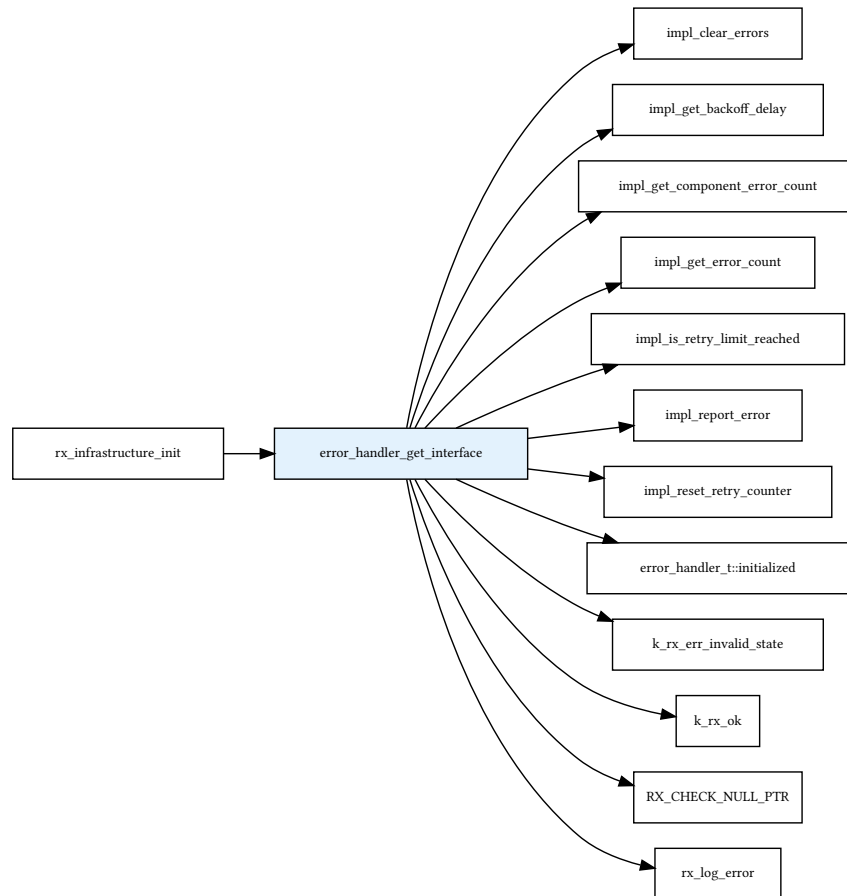


Figure 251: Call graph for error_handler_get_interface

_Defined in libs/rx_core/src/rx_error_handler.c:1241_

Since Version 1.0.0

—

error_handler_deinit

```
rx_err_t error_handler_deinit(error_handler_t *handler)
```

Deinitialize error handler and release resources.

Deinitialize error handler and cleanup resources.

Releases resources held by the error handler, including the ThreadX mutex. Safe to call on already-deinitialized handler (idempotent).

Parameters:

Direction	Name	Description
inout	handler	Pointer to error handler instance May be initialized or already deinitialized Mutex will be deleted

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Handler deinitialized (or was already deinitialized)
k_rx_err_null_ptr	handler is nullptr

Pre: handler points to valid error_handler_t (initialized or not)

Post: handler->initialized == false

Post: handler->mutex deleted (if was initialized)

Note: Idempotent: safe to call multiple times

Note: Not thread-safe: ensure no other threads using handler

Warning: Invalidates any rx_error_interface_t pointing to this handler

Warning: Do not use interface after deinit] **Algorithm:** Validate handler pointer If not initialized, return k_rx_ok (idempotent) Delete ThreadX mutex Set initialized = false Log deinitialization

Example: voidsystem_shutdown(void){ motor_deinit(&motor); comm_deinit(&comm);
error_handler_deinit(&g_error_handler); }

Performance: 10-50 us (mutex deletion)

See also: error_handler_init() Reinitialize after deinit if needed

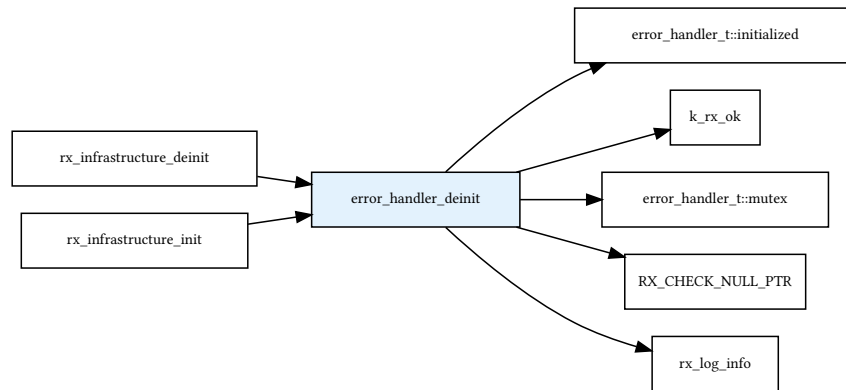


Figure 252: Call graph for error_handler_deinit

_Defined in `libs/rx\core/src/rx\_error\_handler.c:1316_`

Since Version 1.0.0

—

rx_exception.c

RX72N CPU Exception Handling Implementation.

Implements exception handlers for RX72N CPU exceptions. Each handler:

1. Captures the exception context (PC, PSW from stack)
2. Updates exception statistics
3. Logs the exception via UART/USB (if available)
4. Either halts (NMI) or allows return (other exceptions)

Copyright (c) 2026 STAR Project

Includes:

- `"rx_exception.h"`
- `<stddef.h>`
- `"rx_log.h"`

s_log_tag

`const char* const s_log_tag`

Log tag for exception module.

—

s_exception_names

`const char* const s_exception_names[k_rx_exception_type_count]`

Exception type names for logging.

—

s_exception_stats

`rx_exception_stats_t s_exception_stats`

Exception statistics storage.

Warning: Direct modification discouraged - use handler functions

s_initialized

`uint8_t s_initialized`

Initialization flag.

rx_exc_undefined_instruction_c_handler

```
void rx_exc_undefined_instruction_c_handler(uint32_t pc, uint32_t psw)
```

C-level undefined instruction exception handler.

Called from assembly stub after context is saved. The faulting instruction PC is on the stack.

Parameters:

Direction	Name	Description
in	pc	Program counter of undefined instruction
in	psw	Processor status word at exception

Note: Called with interrupts disabled (I=0)

Note: Can return if instruction can be emulated/skipped

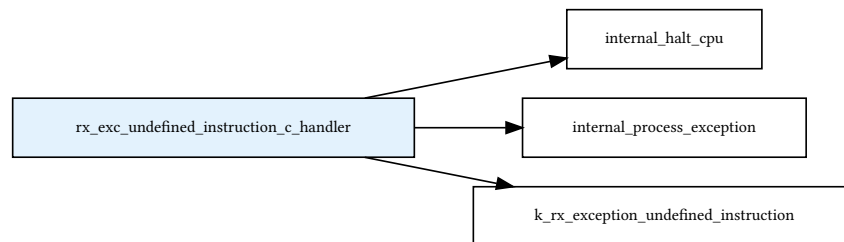


Figure 253: Call graph for `rx_exc_undefined_instruction_c_handler`

Defined in `libs/rx_core/src/rx_exception.c:196`

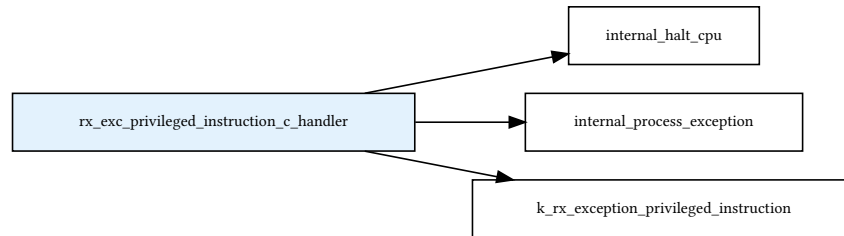
rx_exc_privileged_instruction_c_handler

```
void rx_exc_privileged_instruction_c_handler(uint32_t pc, uint32_t psw)
```

C-level privileged instruction exception handler.

Parameters:

Direction	Name	Description
in	pc	Program counter of privileged instruction
in	psw	Processor status word at exception

Figure 254: Call graph for `rx_exc_privileged_instruction_c_handler`

Defined in `libs/rx_core/src/rx_exception.c:215`

—

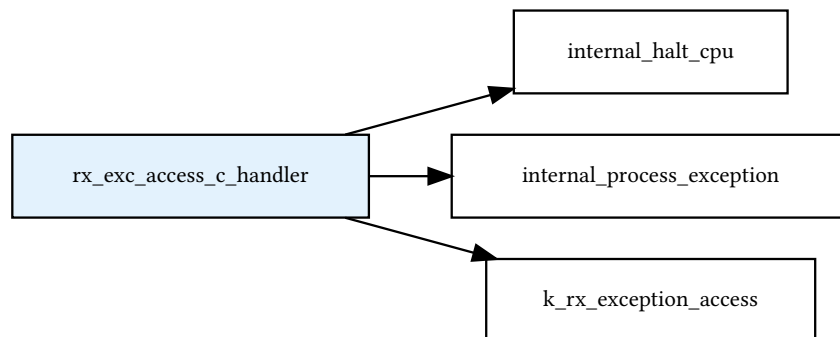
rx_exc_access_c_handler

```
void rx_exc_access_c_handler(uint32_t pc, uint32_t psw)
```

C-level access exception handler.

Parameters:

Direction	Name	Description
in	pc	Program counter of faulting access
in	psw	Processor status word at exception

Figure 255: Call graph for `rx_exc_access_c_handler`

Defined in `libs/rx_core/src/rx_exception.c:234`

—

rx_exc_address_c_handler

```
void rx_exc_address_c_handler(uint32_t pc, uint32_t psw)
```

C-level address exception handler.

Parameters:

Direction	Name	Description
in	pc	Program counter of misaligned access
in	psw	Processor status word at exception

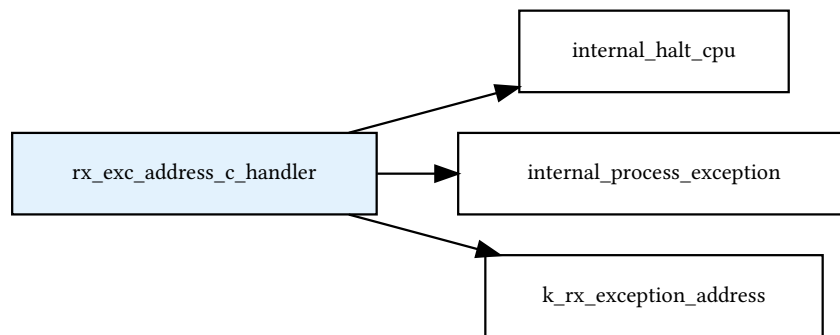


Figure 256: Call graph for `rx_exc_address_c_handler`

Defined in `libs/rx_core/src/rx_exception.c:253`

—

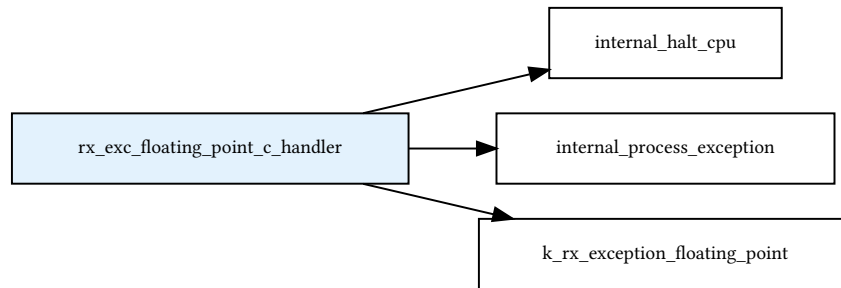
rx_exc_floating_point_c_handler

```
void rx_exc_floating_point_c_handler(uint32_t pc, uint32_t psw)
```

C-level floating-point exception handler.

Parameters:

Direction	Name	Description
in	pc	Program counter of FPU instruction
in	psw	Processor status word at exception

Figure 257: Call graph for `rx_exc_floating_point_c_handler`

Defined in `libs/rx_core/src/rx_exception.c:272`

—

rx_exc_nmi_c_handler

```
void rx_exc_nmi_c_handler(uint32_t pc, uint32_t psw)
```

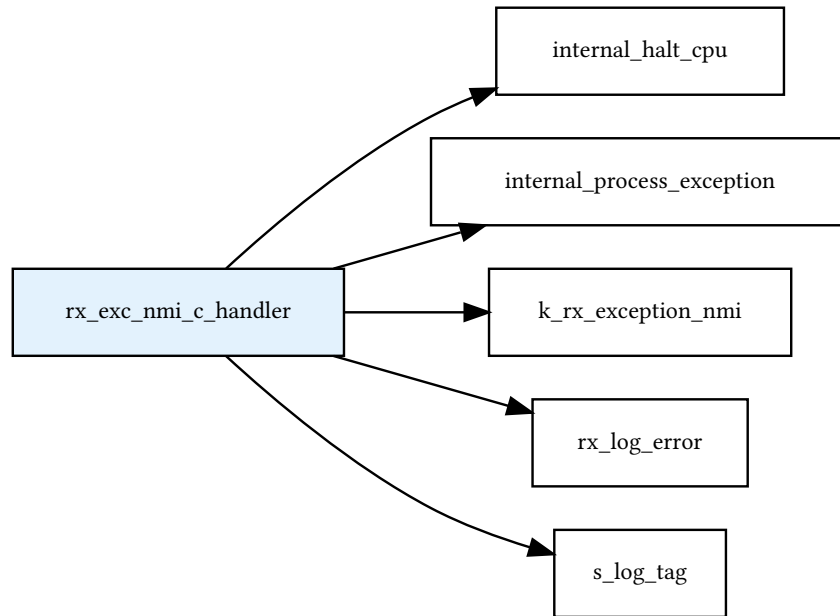
C-level non-maskable interrupt handler.

NMI indicates a fatal system fault. Per the manual (Section 14.1.7): “Never use the non-maskable interrupt with an attempt to return to the program that was being executed at the time of interrupt generation.”

Parameters:

Direction	Name	Description
in	pc	Program counter when NMI occurred
in	psw	Processor status word at NMI

Warning: This function NEVER returns

Figure 258: Call graph for `rx_exc_nmi_c_handler`

Defined in `libs/rx_core/src/rx_exception.c:298`

internal_process_exception

```
static void internal_process_exception(const rx_exception_frame_t *frame)
```

Common exception processing logic.

Called by all exception handlers to:

1. Update statistics
2. Save frame for post-mortem analysis
3. Log the exception

Parameters:

Direction	Name	Description
in	frame	Captured exception context

Pre: frame is not nullptr

Pre: frame->type is valid enum value

Post: Statistics updated

Post: Last frame saved

Note: Not thread-safe - called from exception context with interrupts disabled

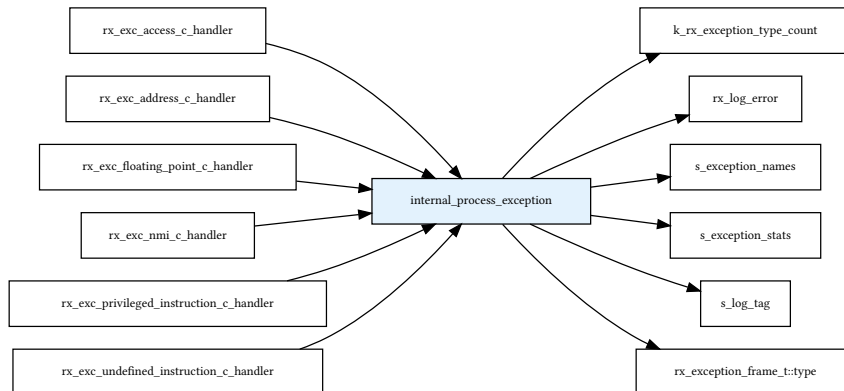


Figure 259: Call graph for `internal_process_exception`

Defined in `libs/rx_core/src/rx_exception.c:98`

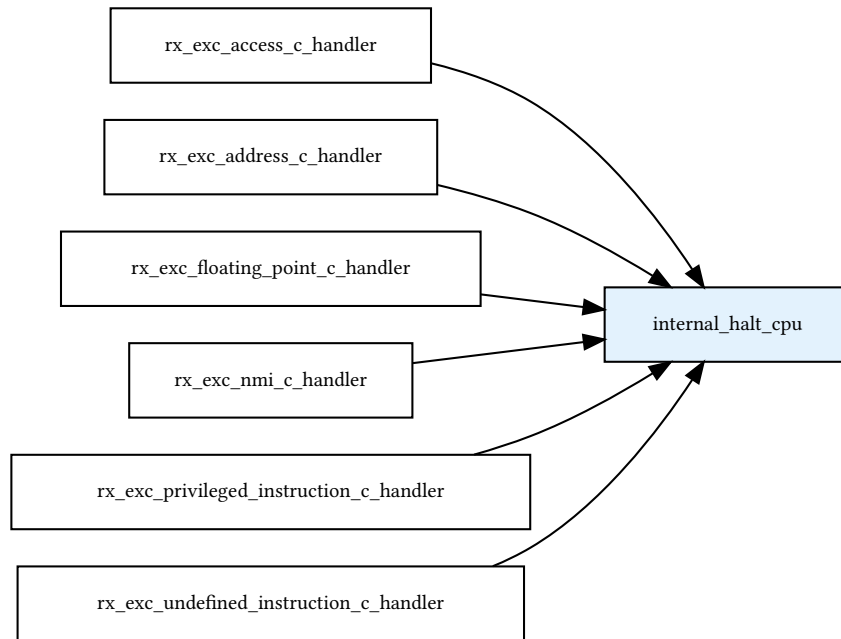
internal_halt_cpu

```
static void internal_halt_cpu(void)
```

Halt CPU in infinite loop.

Used after fatal exceptions (NMI) where recovery is not possible. Enters low-power wait state to reduce power consumption while halted.

Note: This function never returns

Figure 260: Call graph for `internal_halt_cpu`

Defined in `libs/rx_core/src/rx_exception.c:124`

—

rx_exception_init

```
void rx_exception_init(void)
```

Initialize exception handling module.

Initializes exception statistics and optionally installs custom handlers. Should be called early in system startup, after basic hardware init.

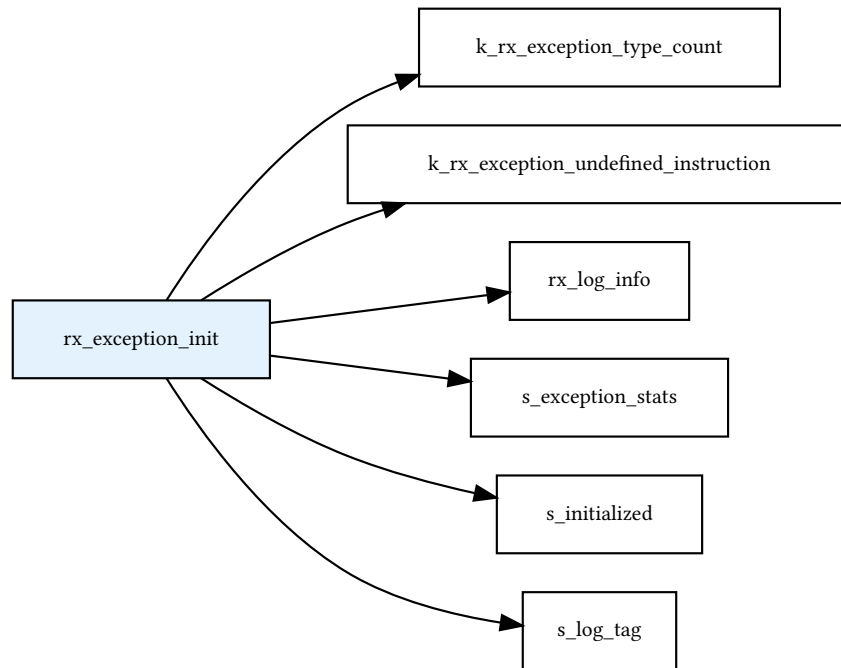
Pre: None

Post: Exception statistics zeroed

Post: Default handlers installed (if not using assembly stubs)

Note: Thread-safe: No, call only once during startup

Example:: `voidmain(void){ rx_exception_init(); }`

Figure 261: Call graph for `rx_exception_init`

Defined in `libs/rx_core/src/rx_exception.c:139`

Since Version 1.0.0

—

`rx_exception_get_stats`

```
const rx_exception_stats_t * rx_exception_get_stats(void)
```

Get exception statistics.

Returns pointer to read-only exception statistics structure. Statistics are updated by exception handlers.

Returns: Pointer to exception statistics (never NULL)

Pre: `rx_exception_init()` must have been called

Post: No side effects

Note: Thread-safe: Read access only

Example:: `constrx_exception_stats_t*stats=rx_exception_get_stats(); if(stats->count[k_rx_exception_nmi]>0){ }`

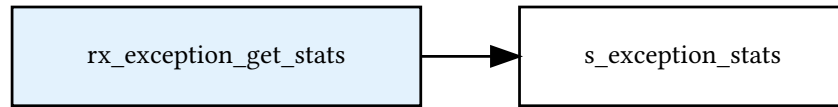


Figure 262: Call graph for rx_exception_get_stats

Defined in `libs/rx_core/src/rx_exception.c:163`

Since Version 1.0.0

—

rx_exception_get_name

```
const char * rx_exception_get_name(rx_exception_type_t type)
```

Get human-readable name for exception type.

Returns a string describing the exception type for logging.

Parameters:

Direction	Name	Description
in	type	Exception type enumeration value

Returns: Pointer to static string (never NULL)

Value	Description
Unknown	If type is out of range

Pre: None

Post: No side effects

Note: Thread-safe: Yes (returns pointer to const data)

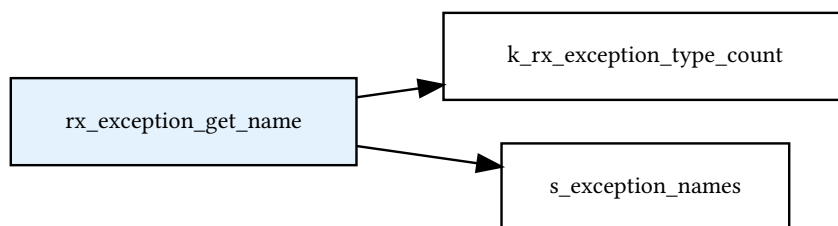


Figure 263: Call graph for rx_exception_get_name

Defined in `libs/rx_core/src/rx_exception.c:171`

Since Version 1.0.0

—

rx_infrastructure.c

Global Infrastructure Initialization Implementation.

Implements the global infrastructure layer that initializes and provides access to core system services. This module serves as the central hub for shared singleton instances used throughout the firmware.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_infrastructure.h"
- "rx_check.h"
- "rx_error_handler.h"
- "rx_gpio_constants.h"
- "rx_log.h"
- "rx_pin_validator.h"

s_global_error_handler

error_handler_t [s_global_error_handler](#)

Global error handler instance (singleton).

Note: Thread Safety: Structure is thread-safe after initialization

Warning: Do not access directly - use interface accessor

Since Version 1.0.0

—

s_global_pin_validator

pin_validator_t [s_global_pin_validator](#)

Global pin validator instance (singleton).

Note: Thread Safety: Structure requires external synchronization

Warning: Do not access directly - use interface accessor

Since Version 1.0.0

—

s_global_error_interface

rx_error_interface_t [s_global_error_interface](#)

Global error handler interface (function pointers).

See also: rx_error_interface_t Interface structure definition

Since Version 1.0.0

—

s_global_pin_interface

rx_pin_interface_t s_global_pin_interface

Global pin validator interface (function pointers).

See also: rx_pin_interface_t Interface structure definition

Since Version 1.0.0

—

s_infrastructure_initialized

bool s_infrastructure_initialized

Infrastructure initialization flag.

Since Version 1.0.0

—

rx_infrastructure_init

```
rx_err_t rx_infrastructure_init(void)
```

Initialize global infrastructure services.

Initialize global infrastructure (MUST BE FIRST CALL IN main()).

Initializes all global infrastructure services in the correct order, with proper cleanup on failure. Uses default configuration values for error handler retry/backoff parameters.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, infrastructure ready
k_rx_err_invalid_arg	Configuration validation failed
k_rx_err_fail	Initialization error

Pre: None - can be called at any time

Post: s_infrastructure_initialized == true on success

Post: All s_global_* variables contain valid data on success

Post: Interface accessors return valid pointers on success

Note: Thread Safety: NOT thread-safe, call once from main()

Note: Idempotent: Safe to call multiple times, subsequent calls are no-op

Warning: Must be called before any module uses infrastructure services

Warning: Not thread-safe - call before starting RTOS threads

Algorithm Steps:: Check if already initialized (idempotent, return success) Initialize error handler with default config Get error handler interface (function pointers) Initialize pin validator Get pin validator interface (function pointers) Set initialized flag

Initialization Flow Diagram:: digraph init_flow { rankdir=TB; node [shape=box];

```
start [label="Start" shape=ellipse]; check_init [label="Already initialized?"]; return_ok
[label="Return k_rx_ok" shape=ellipse]; init_error [label="error_handler_init()"]; get_error_if
[label="error_handler_get_interface()"]; init_pin [label="pin_validator_init()"]; get_pin_if
[label="pin_validator_get_interface()"]; set_flag [label="s_initialized = true"]; cleanup1
[label="deinit error_handler"]; cleanup2 [label="deinit pin_validatorndeinit error_handler"];
return_err [label="Return error" shape=ellipse];
```

```
start -> check_init; check_init -> return_ok [label="yes"]; check_init -> init_error [label="no"];
init_error -> get_error_if [label="ok"]; init_error -> return_err [label="fail"]; get_error_if -> init_pin
[label="ok"]; get_error_if -> cleanup1 [label="fail"]; init_pin -> get_pin_if [label="ok"]; init_pin ->
cleanup1 [label="fail"]; get_pin_if -> set_flag [label="ok"]; get_pin_if -> cleanup2 [label="fail"];
cleanup1 -> return_err; cleanup2 -> return_err; set_flag -> return_ok; }
```

Default Configuration:: Parameter Value

max_retries k_error_handler_default_max_retries (3)

initial_backoff_ms k_error_handler_default_initial_backoff_ms

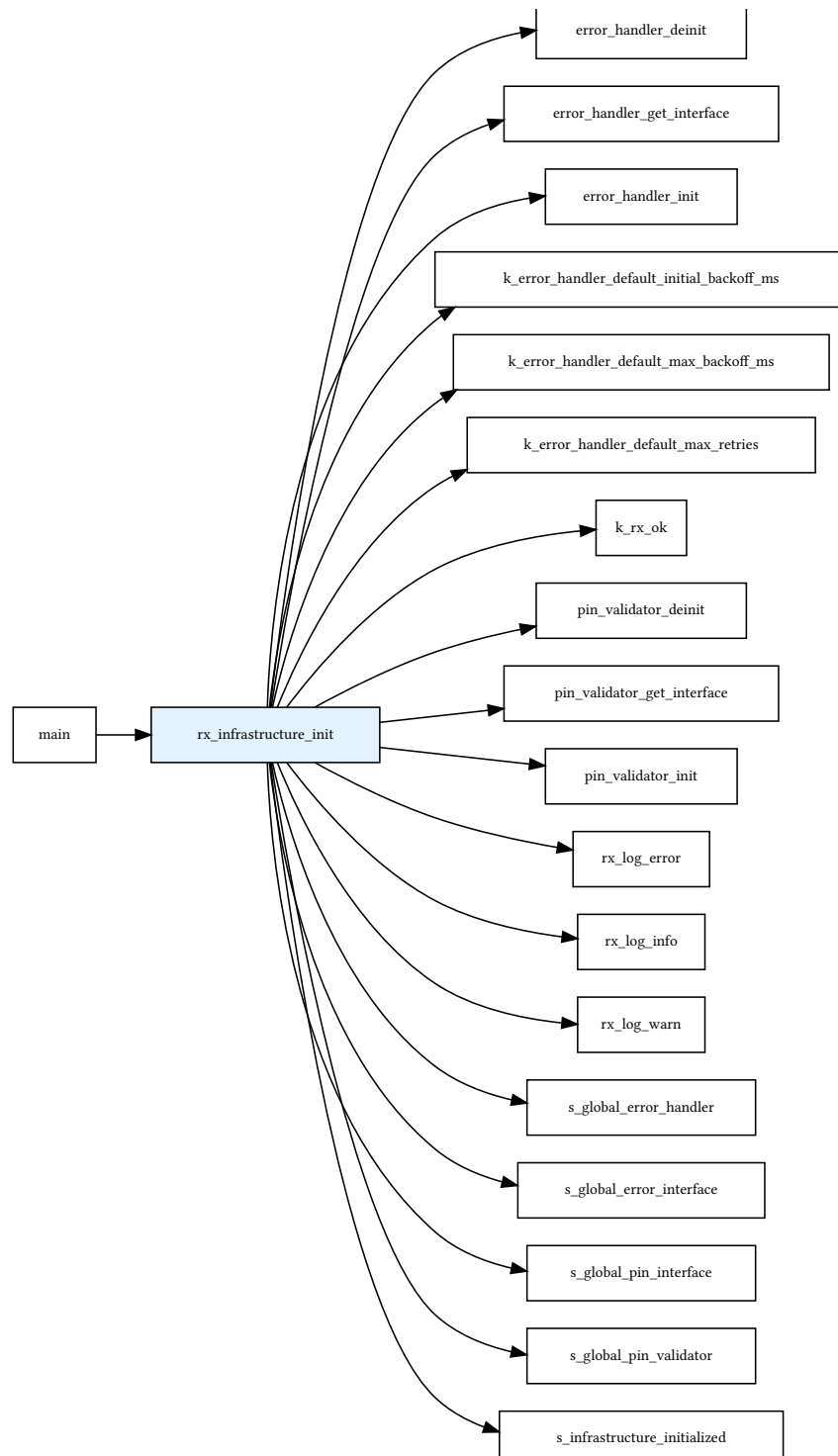
max_backoff_ms k_error_handler_default_max_backoff_ms

Example:: rx_err_t err=rx_infrastructure_init(); if(err!=k_rx_ok){ while(1){ }

rx_error_interface_t* error_if=rx_infrastructure_get_error_interface();

NASA Power of 10 Compliance:: Rule 5: [OK] Cleanup on failure (4 cleanup paths) Rule 7: [OK] All return values checked

See also: rx_infrastructure_deinit() Cleanup function,
rx_infrastructure_get_error_interface() Access error handler,
rx_infrastructure_get_pin_interface() Access pin validator

Figure 264: Call graph for `rx_infrastructure_init`

Defined in `libs/rx_core/src/rx_infrastructure.c:358`

Since Version 1.0.0

—

rx_infrastructure_deinit

```
rx_err_t rx_infrastructure_deinit(void)
```

Deinitialize global infrastructure and release resources.

Deinitialize global infrastructure (graceful shutdown only).

Releases all global infrastructure resources in reverse initialization order. Clears the initialized flag so interface accessors return NULL.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, infrastructure deinitialized

Pre: None - safe to call anytime

Post: s_infrastructure_initialized == false

Post: Interface accessors return nullptr

Post: All s_global_* structures are in undefined state

Note: Thread Safety: NOT thread-safe

Note: Idempotent: Safe to call multiple times

Warning: Call only after all threads have stopped using infrastructure

Warning: After deinit, interface accessors return nullptr

Algorithm Steps:: Check if initialized (return early if not) Deinitialize pin validator Deinitialize error handler Clear initialized flag

Cleanup Order:: Resources are released in reverse initialization order to respect dependencies: Pin validator (uses error handler for logging) Error handler (foundation service)

Example:: motor_deinit(&motor); sensor_deinit(&sensor);

rx_infrastructure_deinit();

See also: rx_infrastructure_init() Initialization function

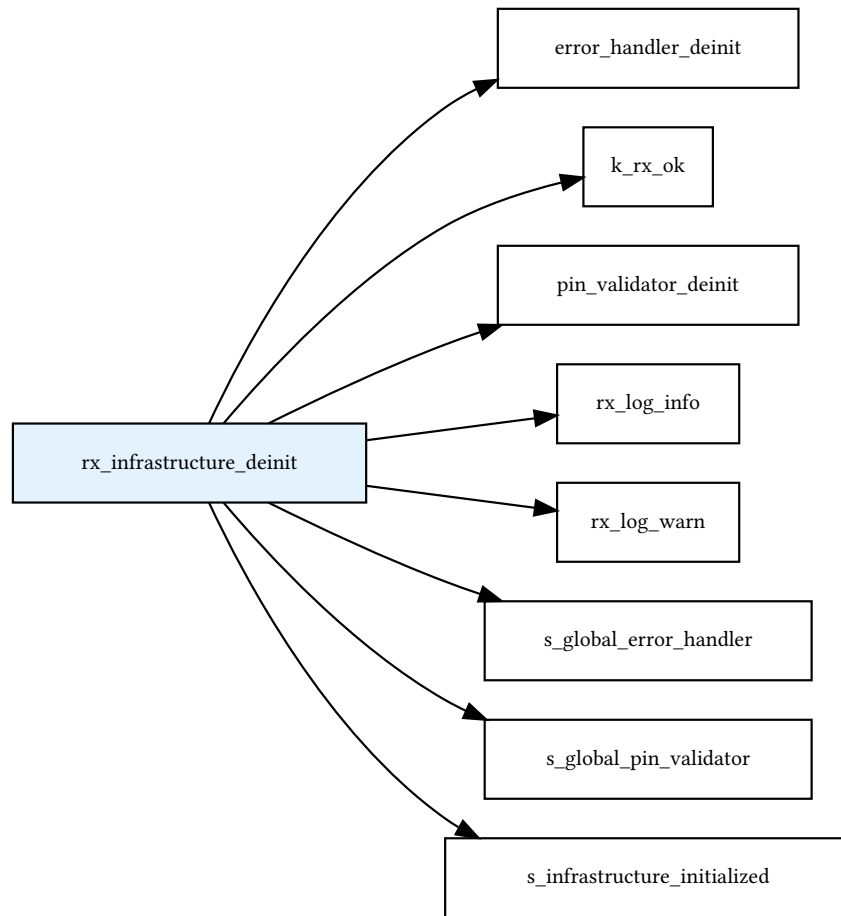


Figure 265: Call graph for rx_infrastructure_deinit

Defined in `libs/rx_core/src/rx_infrastructure.c:459`

Since Version 1.0.0

—

rx_infrastructure_get_error_interface

```
rx_error_interface_t * rx_infrastructure_get_error_interface(void)
```

Get global error handler interface.

Get global error handler interface (thread-safe accessor).

Returns a pointer to the global error handler interface structure. The interface provides function pointers for error recovery operations.

Returns: Pointer to error interface, or nullptr if not initialized

Value	Description
&s_global_error_interface	Infrastructure is initialized
NULL	Infrastructure not initialized

Pre: rx_infrastructure_init() should have been called

Post: Returned pointer valid for program lifetime (if not nullptr)

Note: Thread Safety: Safe - returns pointer to static data

Note: The returned pointer should not be freed

Interface Functions:: Function Description

try_recover Attempt recovery with retry and backoff

reset Reset error handler state

Example:: rx_error_interface_t*err_if=rx_infrastructure_get_error_interface(); if(err_if==nullptr)
{ rx_log_error("APP","Infrastructurenotinitialized"); returnk_rx_err_not_initialized; }

rx_err_tresult=err_if->try_recover(err_if->ctx,my_operation,&context);

See also: rx_error_interface_t Interface structure, rx_infrastructure_init() Must be called first

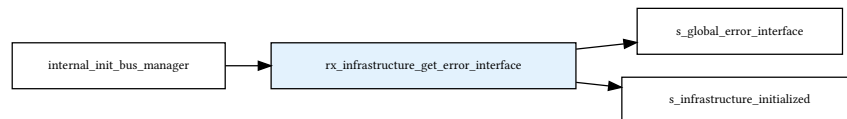


Figure 266: Call graph for rx_infrastructure_get_error_interface

Defined in `libs/rx_core/src/rx_infrastructure.c:529`

Since Version 1.0.0

—

rx_infrastructure_get_pin_interface

```
rx_pin_interface_t * rx_infrastructure_get_pin_interface(void)
```

Get global pin validator interface.

Get global pin validator interface (thread-safe accessor).

Returns a pointer to the global pin validator interface structure. The interface provides function pointers for GPIO pin management.

Returns: Pointer to pin interface, or nullptr if not initialized

Value	Description
-------	-------------

<code>&s_global_pin_interface</code>	Infrastructure is initialized
<code>NULL</code>	Infrastructure not initialized

Pre: `rx_infrastructure_init()` should have been called

Post: Returned pointer valid for program lifetime (if not nullptr)

Note: Thread Safety: Safe for read - returns pointer to static data

Note: The returned pointer should not be freed

Warning: Pin claim/release operations may not be thread-safe

Interface Functions:: Function Description

`claim_pin` Claim a GPIO pin for a specific module

`release` Release a previously claimed GPIO pin

`is_claimed` Check if a pin is already claimed

Example:: `rx_pin_interface_t*pin_if=rx_infrastructure_get_pin_interface(); if(pin_if==nullptr) { rx_log_error("MOTOR","Infrastructurenottinitialized"); returnk_rx_err_not_initialized; }`

`rx_err_terr=pin_if->claim_pin(pin_if->ctx, k_port_1,k_pin_5, "MOTOR_PWM"); if(err!=k_rx_ok) { rx_log_error("MOTOR","PinP15alreadyinuse"); returnerr; }`

See also: `rx_pin_interface_t` Interface structure, `rx_infrastructure_init()` Must be called first

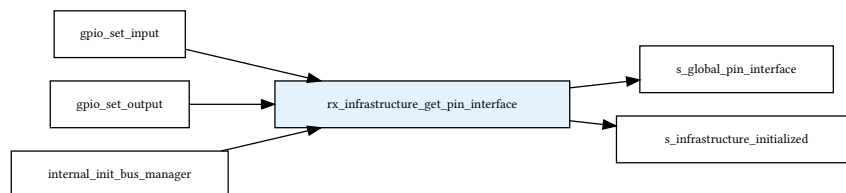


Figure 267: Call graph for `rx_infrastructure_get_pin_interface`

Defined in `libs/rx_core/src/rx_infrastructure.c:588`

Since Version 1.0.0

—

rx_iwdt.c

Independent Watchdog Timer (IWDT) Implementation for RX72N.

Implements the hardware-independent watchdog timer driver with enhanced task monitoring capabilities. The IWDT uses a dedicated 125 kHz clock source, providing reliable watchdog functionality even during main clock failures - a critical safety feature for robotics applications.

Includes:

- "rx_iwdt.h"
- <string.h>
- "rx72n_iwdt_regs.h"
- "rx72n_system_regs.h"
- "rx_check.h"
- "tx_api.h"

s_iwdt_state

rx_iwdt_state_t s_iwdt_state

Global IWDT driver state.

Note: Not thread-safe for concurrent writes. Use external synchronization if calling from multiple threads.] **See also:** rx_iwdt_state_t Structure definition

Since Version 1.0.0

—

internal_get_tick_count

```
static uint32_t internal_get_tick_count(void)
```

Get current system tick count from ThreadX.

Wrapper around ThreadX tx_time_get() to get the current tick count. Used for task heartbeat timing calculations.

Algorithm:

1. Call tx_time_get() to get ThreadX tick count
2. Cast ULONG to uint32_t for consistent type
3. Return tick count

Returns: uint32_t Current tick count from ThreadX timer

Note: ThreadX tick rate defined by TX_TIMER_TICKS_PER_SECOND

Note: Tick count wraps at UINT32_MAX (49.7 days at 1 kHz tick rate)

Note: Thread-safe - tx_time_get() is reentrant] **See also:** tx_time_get() ThreadX API, TX_TIMER_TICKS_PER_SECOND Tick rate configuration

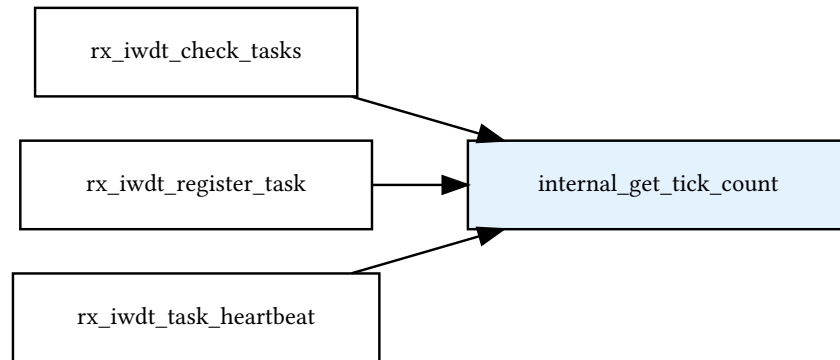


Figure 268: Call graph for internal_get_tick_count

Defined in `libs/rx_core/src/rx_iwdt.c:320`

Since Version 1.0.0

—

internal_find_task

```
static rx_iwdt_task_info_t * internal_find_task(const char *task_name)
```

Find registered task by name.

Searches the task array for a task with matching name that is currently active. Uses strcmp() for exact string comparison.

Algorithm:

1. Iterate through tasks[0..k_iwdt_max_tasks-1]
2. For each entry: check if active AND name matches
3. Return pointer to matching entry, or nullptr if not found

Parameters:

Direction	Name	Description
in	task_name	Task name to search for (null-terminated string)

Returns: rx_iwdt_task_info_t* Pointer to task info if found

Value	Description
NULL	Task not found or not active

Pre: task_name must be non-NULL (caller validates)

Note: Linear search O(n) where n = k_iwdt_max_tasks (8)

Note: String comparison is case-sensitive] **See also:** rx_iwdt_register_task() Adds tasks to array, rx_iwdt_task_heartbeat() Uses this to find task

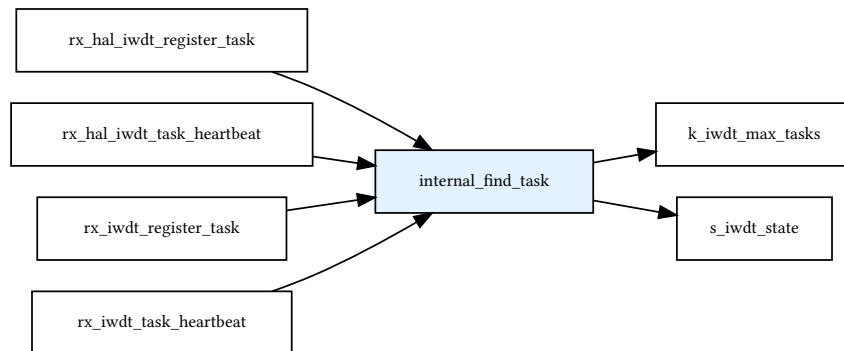


Figure 269: Call graph for internal_find_task

Defined in `libs/rx_core/src/rx_iwdt.c:352`

Since Version 1.0.0

—

internal_find_free_slot

```
static rx_iwdt_task_info_t * internal_find_free_slot(void)
```

Find free slot in task array.

Searches for the first inactive slot in the task array where a new task can be registered.

Algorithm:

1. Iterate through tasks[0..k_iwdt_max_tasks-1]
2. For each entry: check if NOT active
3. Return pointer to first inactive slot, or nullptr if all full

Returns: rx_iwdt_task_info_t* Pointer to free slot if available

Value	Description
NULL	No free slots (all 8 tasks registered)

Note: Linear search $O(n)$ where $n = k_iwdt_max_tasks$ (8)

Note: Returns first available slot (lowest index)] **See also:** rx_iwdt_register_task() Uses this to allocate slot, k_iwdt_max_tasks Maximum number of tasks (8)

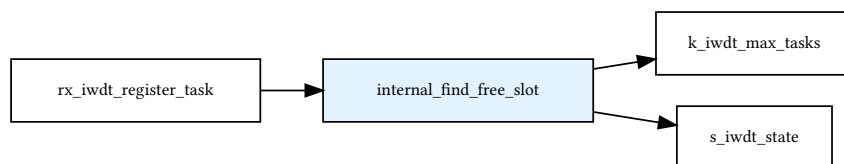


Figure 270: Call graph for internal_find_free_slot

Defined in `libs/rx_core/src/rx_iwdt.c:386`

Since Version 1.0.0

—

internal_init_default_config

```
static void internal_init_default_config(rx_iwdt_config_t *config)
```

Initialize default configuration structure.

Populates a configuration structure with safe default values suitable for most applications. Called when rx_iwdt_init() receives nullptr config.

Default Values:

- default_timeout_ms = k_iwdt_default_timeout_ms (1000 ms)
- enable_task_monitoring = true
- reset_on_timeout = true
- All state_timeouts_ms[] = k_iwdt_default_timeout_ms

Algorithm:

1. Set default_timeout_ms to 1000 ms
2. Enable task monitoring
3. Enable reset on timeout
4. Initialize all state timeouts to default value

Parameters:

Direction	Name	Description
out	config	Configuration structure to initialize

Pre: config must be non-NULL (caller validates)

Post: config contains default values ready for use

Note: Loop bounded by k_system_state_count (NASA Rule 2)] **See also:** rx_iwdt_init() Calls this when config is nullptr, k_iwdt_default_timeout_ms Default timeout constant, k_system_state_count Number of system states

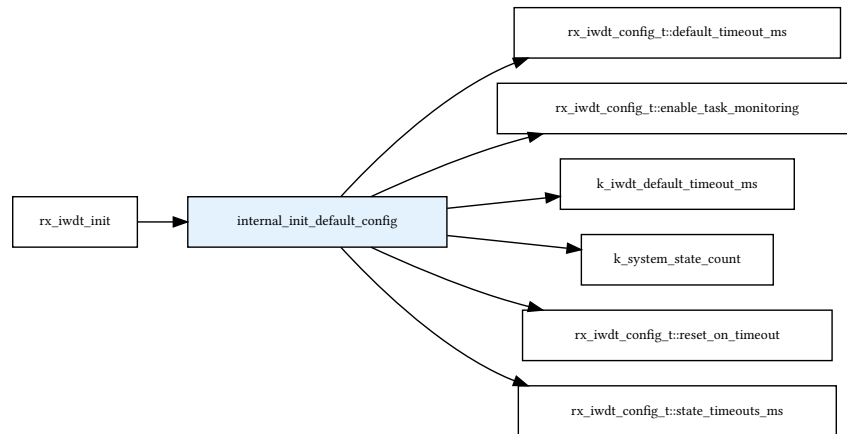


Figure 271: Call graph for internal_init_default_config

Defined in `libs/rx_core/src/rx_iwdt.c:430`

Since Version 1.0.0

—

rx_iwdt_init

```
rx_err_t rx_iwdt_init(const rx_iwdt_config_t *config)
```

Initialize IWDT driver with configuration and task monitoring.

Initialize the Independent Watchdog Timer.

Initializes the IWDT driver, validates configuration, checks for previous watchdog reset, and performs initial hardware feed. If config is nullptr, uses safe default values (1000ms timeout, task monitoring enabled).

Algorithm:

1. Check if already initialized (return error if so)
2. If config is nullptr, create default configuration
3. Validate timeout within [k_iwdt_min_timeout_ms, k_iwdt_max_timeout_ms]
4. Clear and initialize all state
5. Check RSTSR2 for previous watchdog reset
6. Perform initial feed to IWDTRR (0x00 then 0xFF)
7. Set initialized flag

Parameters:

Direction	Name	Description
in	config	Configuration pointer, or nullptr for defaults

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success

k_rx_err_invalid_state	Already initialized
k_rx_err_invalid_arg	Timeout out of valid range

Pre: IWDT not already initialized

Post: IWDT driver ready for use

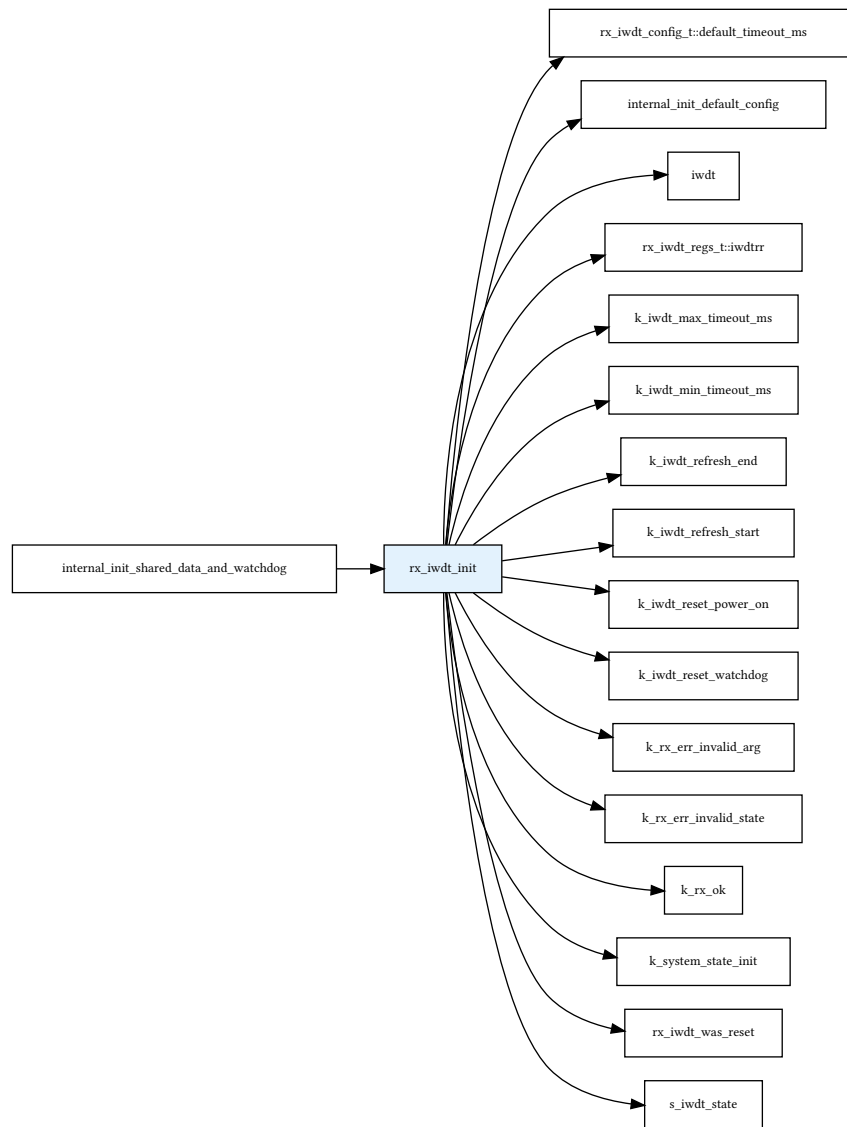
Post: Initial feed performed

Post: Reset status captured

Note: Hardware timeout set by OFS, not this driver

Note: nullptr config uses safe defaults (1000ms, monitoring enabled)

OFS Configuration Note:: The IWDT hardware timeout is configured at flash-time via OFS (Option Function Select) registers and cannot be changed at runtime. The timeout values in this driver are used for software task monitoring only. The hardware IWDT will timeout based on OFS settings regardless of software configuration.

Figure 272: Call graph for `rx_iwdt_init`

Defined in `libs/rx_core/src/rx_iwdt.c:489`

Since Version 1.0.0

—

rx_iwdt_feed

```
rx_err_t rx_iwdt_feed(void)
```

Feed the independent watchdog timer to prevent system reset.

Feed the watchdog timer.

Writes the two-byte refresh sequence (0x00 then 0xFF) to the IWDT Refresh Register (IWDTRR) to reset the hardware watchdog counter. This must be called periodically within the configured hardware timeout window or the IWDT will reset the microcontroller.

Per the RX72N hardware manual the refresh sequence is strictly ordered: writing 0x00 to IWDTRR starts the window; writing 0xFF completes the refresh. The entire sequence must occur within the permitted refresh window.

Also increments the software diagnostic counter `s_iwdt_state.status.watchdog_feeds` for telemetry and debugging purposes.

Algorithm steps:

1. Check `s_iwdt_state.initialized`; return `k_rx_err_not_initialized` if false
2. Write `k_iwdt_refresh_start` (0x00) to `regs->iwdtrr`
3. Write `k_iwdt_refresh_end` (0xFF) to `regs->iwdtrr`
4. Increment `status.watchdog_feeds` counter

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Watchdog counter successfully refreshed
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called

Pre: `rx_iwdt_init()` must have been called successfully

Pre: Must be called within the hardware IWDT refresh window (no gap > configured timeout)

Post: Hardware IWDT counter is reset; timeout window restarts

Post: `s_iwdt_state.status.watchdog_feeds` is incremented by one

Note: Not thread-safe; concurrent feeds from multiple tasks are benign for the hardware but may produce incorrect feed counts

Warning: Failing to call within the hardware timeout triggers a system reset

Performance:: Execution time: <1 us @ 240 MHz; safe to call from any task or ISR context

Example::

```
while(1){ rx_err_t err=rx_iwdt_feed(); if(err!=k_rx_ok)
{ rx_log_error("wdt","IWDTnotinitialized"); } tx_thread_sleep(k_watchdog_feed_interval_ticks); }
```

NASA Power of 10 Compliance:: Rule 5: 1 precondition (initialized check), 2 postconditions (HW reset, counter++)

See also: `rx_iwdt_init()` Initialize IWDT before feeding, `rx_iwdt_check_tasks()` Verify all tasks are alive before feeding

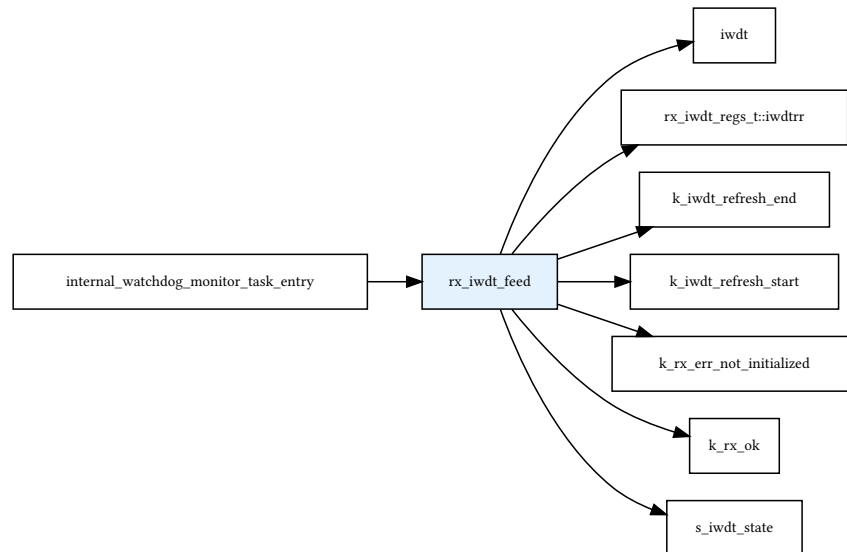


Figure 273: Call graph for rx_iwdt_feed

Defined in `libs/rx_core/src/rx_iwdt.c:602`

Since Version 1.0.0

—

rx_iwdt_register_task

```
rx_err_t rx_iwdt_register_task(const char *task_name, uint32_t timeout_ms)
```

Register a ThreadX task for software watchdog monitoring.

Register a task for heartbeat monitoring.

Adds a task entry to the IWDT software monitoring table, associating the task name with a per-task timeout. Once registered, the task must call `rx_iwdt_task_heartbeat()` at least once per `timeout_ms` milliseconds or `rx_iwdt_check_tasks()` will report `k_rx_err_timeout`.

The registration also records the current tick count as the initial `last_heartbeat_tick`, giving the task a full `timeout_ms` window before the first heartbeat is required.

Algorithm steps:

1. Validate `task_name` is non-null
2. Verify `s_iwdt_state.initialized`
3. Validate `timeout_ms` is in `[k_iwdt_min_timeout_ms, k_iwdt_max_timeout_ms]`
4. Check task name length is in `(0, k_iwdt_task_name_len)`
5. Check task is not already registered via `internal_find_task()`
6. Find a free slot via `internal_find_free_slot()`
7. Initialize slot with task name, timeout, current tick, `active=true`, `timed_out=false`
8. Increment `status.active_tasks`

Parameters:

Direction	Name	Description
in	task_name	Unique task identifier string (non-empty, max k_iwdt_task_name_len-1 chars, null-terminated); must remain valid for the lifetime of registration
in	timeout_ms	Per-task heartbeat timeout in milliseconds; valid range [k_iwdt_min_timeout_ms(100), k_iwdt_max_timeout_ms(60000)]

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task successfully registered
k_rx_err_null_ptr	task_name is nullptr
k_rx_err_not_initialized	rx_iwdt_init() has not been called
k_rx_err_invalid_arg	timeout_ms out of range, name empty, name too long, or task already registered
k_rx_err_no_mem	No free task slots (maximum k_iwdt_max_tasks tasks)

Pre: rx_iwdt_init() must have been called successfully

Pre: task_name must be unique among registered tasks

Post: Task entry is active in s_iwdt_state.tasks

Post: status.active_tasks is incremented by one

Note: Not thread-safe; serialize registrations from multiple contexts

Example:: rx_err_t err = rx_iwdt_register_task("motor_ctrl", 500); if (err != k_rx_ok) { rx_log_error("wdt", "Failed to register motor_ctrl task"); }

NASA Power of 10 Compliance:: Rule 5: 5 preconditions (null, initialized, timeout, name, duplicate), 2 postconditions

See also: rx_iwdt_task_heartbeat() Update heartbeat for a registered task, rx_iwdt_check_tasks() Check all registered tasks for timeout

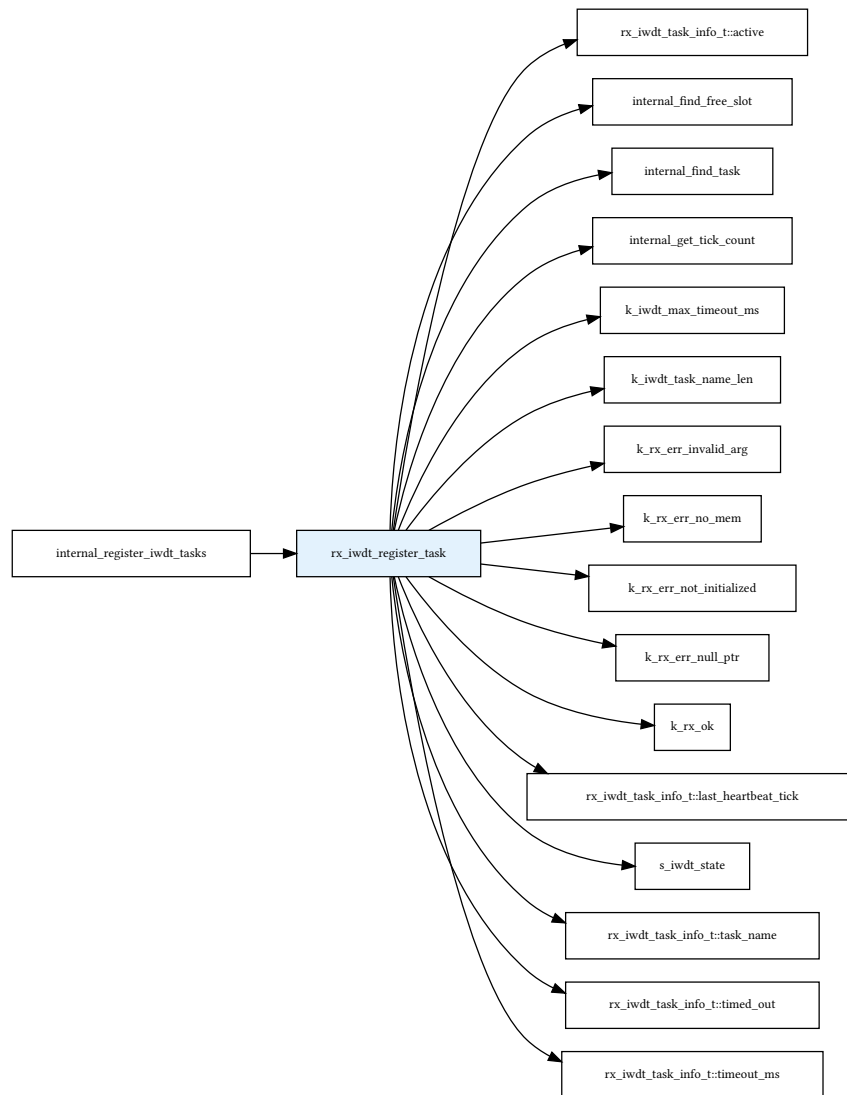


Figure 274: Call graph for rx_iwdt_register_task

Defined in `libs/rx_core/src/rx_iwdt.c:680`

Since Version 1.0.0

rx_iwdt_task_heartbeat

```
rx_err_t rx_iwdt_task_heartbeat(const char *task_name)
```

Send a heartbeat signal for a registered task to reset its timeout.

Report task heartbeat.

Updates the `last_heartbeat_tick` for the named task to the current ThreadX tick count, resetting the timeout window. Also clears the `timed_out` flag if the task had previously been marked as timed out.

Tasks must call this function at least once per their registered `timeout_ms` window (configured in `rx_iwdt_register_task()`) to remain in good standing. If the interval between heartbeats exceeds `timeout_ms`, the next call to `rx_iwdt_check_tasks()` will return `k_rx_err_timeout`.

Algorithm steps:

1. Validate `task_name` is non-null
2. Verify `s_iwdt_state.initialized`
3. Find the task entry via `internal_find_task()`
4. Set `task->last_heartbeat_tick = internal_get_tick_count()`
5. Clear `task->timed_out`

Parameters:

Direction	Name	Description
in	<code>task_name</code>	Null-terminated name of the registered task sending heartbeat; must exactly match the name used in <code>rx_iwdt_register_task()</code>

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Heartbeat timestamp updated and <code>timed_out</code> flag cleared
<code>k_rx_err_null_ptr</code>	<code>task_name</code> is nullptr
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called
<code>k_rx_err_not_found</code>	No task registered with the given name

Pre: `rx_iwdt_init()` must have been called successfully

Pre: Task must have been registered via `rx_iwdt_register_task()`

Post: `task->last_heartbeat_tick` is refreshed to the current tick count

Post: `task->timed_out` is false

Note: Not thread-safe; concurrent heartbeats from different tasks are safe as they write to distinct slots, but concurrent writes to the same slot are not

Example::

```
while(1){ rx_iwdt_task_heartbeat("motor_ctrl"); motor_control_update();
tx_thread_sleep(k_motor_ctrl_period_ticks); }
```

NASA Power of 10 Compliance:: Rule 5: 3 preconditions (null, initialized, registered), 2 postconditions

See also: `rx_iwdt_register_task()` Register a task before sending heartbeats, `rx_iwdt_check_tasks()` Verify all tasks are within their timeout windows

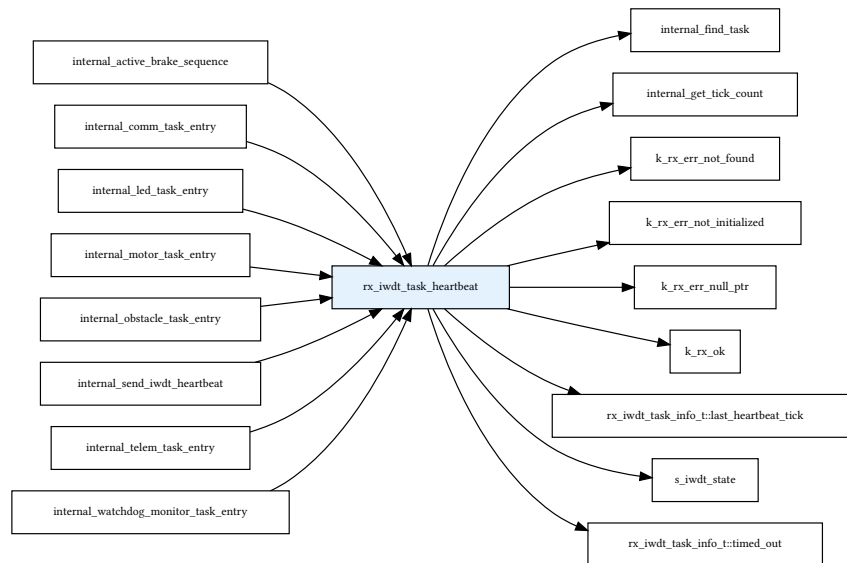


Figure 275: Call graph for rx_iwdt_task_heartbeat

Defined in `libs/rx_core/src/rx_iwdt.c:781`

Since Version 1.0.0

—

rx_iwdt_set_timeout_for_state

```
rx_err_t rx_iwdt_set_timeout_for_state(const system_state_t state, const uint32_t timeout_ms)
```

Set the software task monitoring timeout for a specific system state.

Set timeout for system state.

Configures the state-dependent software timeout used by `rx_iwdt_check_tasks()` when the system is in the specified state. This allows different task heartbeat windows depending on the operational mode (e.g. longer timeouts during initialization, tighter timeouts during normal operation).

Note: This does NOT change the hardware IWDT timeout. The hardware watchdog timeout is fixed at compile/flash time via OFS registers and cannot be altered at runtime. Only software task monitoring timeouts are affected.

If state matches the currently active state (`s_iwdt_state.current_state`), the active timeout (`status.current_timeout_ms`) is also updated immediately.

Algorithm steps:

1. Validate state is less than `k_system_state_count`
2. Verify `s_iwdt_state.initialized`
3. Validate `timeout_ms` is in `[k_iwdt_min_timeout_ms, k_iwdt_max_timeout_ms]`
4. Set `config.state_timeouts_ms[state] = timeout_ms`
5. If `state == current_state`, update `status.current_timeout_ms`

Parameters:

Direction	Name	Description
in	state	System state identifier to configure (must be < k_system_state_count)
in	timeout_ms	Software task monitoring timeout in milliseconds; valid range [k_iwdt_min_timeout_ms(100), k_iwdt_max_timeout_ms(60000)]

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Timeout successfully updated
k_rx_err_not_initialized	rx_iwdt_init() has not been called
k_rx_err_invalid_arg	state >= k_system_state_count or timeout_ms out of range

Pre: rx_iwdt_init() must have been called successfully

Pre: state must be a valid system_state_t value less than k_system_state_count

Post: config.state_timeouts_msstate equals timeout_ms

Post: If state is current, status.current_timeout_ms is updated immediately

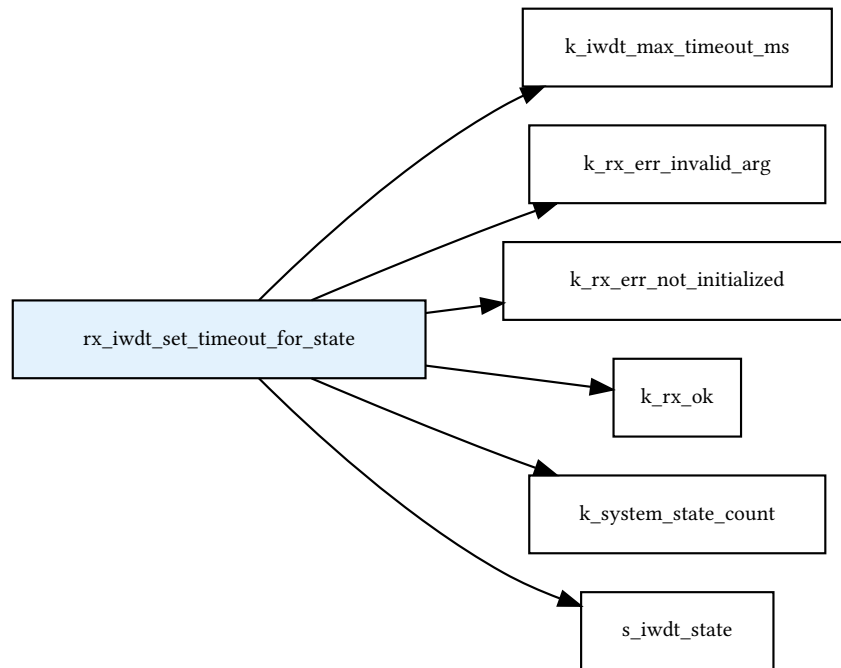
Note: Not thread-safe; serialize state configuration changes

Warning: Hardware IWDT timeout is unaffected; only software task monitoring changes

Example:: rx_iwdt_set_timeout_for_state(k_system_state_initializing,5000);
rx_iwdt_set_timeout_for_state(k_system_state_running,500);

NASA Power of 10 Compliance:: Rule 5: 3 preconditions (initialized, valid state, valid timeout), 2 postconditions

See also: rx_iwdt_set_state() Change the active system state, rx_iwdt_check_tasks() Uses current state timeout to evaluate task heartbeats

Figure 276: Call graph for `rx_iwdt_set_timeout_for_state`

Defined in `libs/rx_core/src/rx_iwdt.c:860`

Since Version 1.0.0

—

rx_iwdt_set_state

```
rx_err_t rx_iwdt_set_state(const system_state_t state)
```

Set the current system state for software task monitoring.

Set current system state.

Updates the active system state used by `rx_iwdt_check_tasks()` to select per-state task heartbeat timeout windows. Transitions between states (e.g. initializing -> running) change which timeout threshold is applied when evaluating task heartbeat intervals.

This function updates three tracking fields atomically:

- `s_iwdt_state.current_state`
- `s_iwdt_state.status.current_state`
- `s_iwdt_state.status.current_timeout_ms` (set from `config.state_timeouts_ms[state]`)

Important: This does NOT modify the hardware IWDt peripheral. The hardware watchdog timeout is a fixed constant set in OFS registers at compile time. Only the software task monitoring timeout threshold changes.

Algorithm steps:

1. Validate state is less than `k_system_state_count`

2. Verify `s_iwdt_state.initialized`
3. Update `current_state` and `status.current_state = state`
4. Update `status.current_timeout_ms = config.state_timeouts_ms[state]`

Parameters:

Direction	Name	Description
in	<code>state</code>	New system state (must be $< k_system_state_count$)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	State successfully updated
<code>k_rx_err_not_initialized</code>	<code>rx_iwdt_init()</code> has not been called
<code>k_rx_err_invalid_arg</code>	$state \geq k_system_state_count$

Pre: `rx_iwdt_init()` must have been called successfully

Pre: `state` must be a valid `system_state_t` value less than `k_system_state_count`

Post: `s_iwdt_state.current_state` reflects the new state

Post: `status.current_timeout_ms` reflects the configured timeout for the new state

Note: Not thread-safe; serialize state transitions from multiple contexts

Warning: Hardware IWDT is unaffected; only software task monitoring changes

Example:: `rx_err_t err = rx_iwdt_set_state(k_system_state_running); if (err != k_rx_ok) { rx_log_error("wdt", "Failed to set IWDT system state"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (initialized, valid state), 2 postconditions

See also: `rx_iwdt_set_timeout_for_state()` Configure per-state timeouts,
`rx_iwdt_get_status()` Query current state and timeout

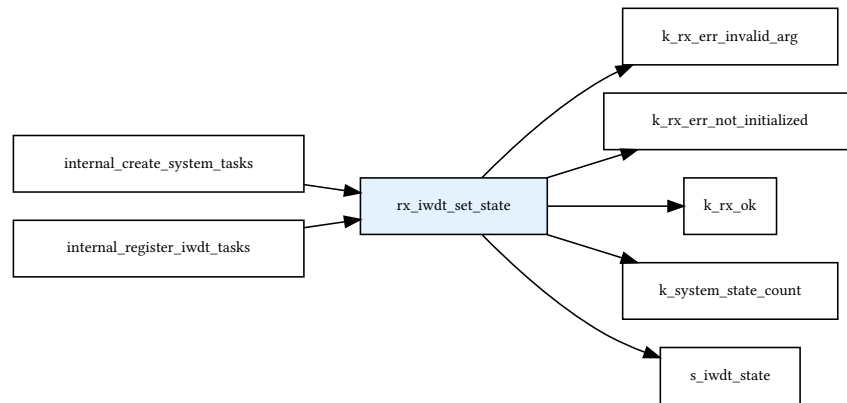


Figure 277: Call graph for rx_iwdt_set_state

Defined in `libs/rx_core/src/rx_iwdt.c:942`

Since Version 1.0.0

—

rx_iwdt_get_status

```
rx_err_t rx_iwdt_get_status(rx_iwdt_status_t *status)
```

Get a snapshot of the current IWDT module status.

Get watchdog status.

Copies the internal `s_iwdt_state.status` structure into the caller-provided buffer via `memcpy`. The status snapshot includes: active task count, total feed count, current system state, current timeout threshold, and the name of the last task that timed out (if any).

This is a diagnostic/telemetry function intended for logging and health monitoring. The returned data is a point-in-time snapshot and may become stale if the detection task or other threads update IWDT state concurrently.

Algorithm steps:

1. Validate status pointer is non-null
2. Verify `s_iwdt_state.initialized`
3. `memcpy(&s_iwdt_state.status, status, sizeof(rx_iwdt_status_t))`

Parameters:

Direction	Name	Description
out	status	Pointer to caller-allocated <code>rx_iwdt_status_t</code> structure to receive the status snapshot; must be non-null; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Status snapshot successfully copied

k_rx_err_null_ptr	status is nullptr
k_rx_err_not_initialized	rx_iwdt_init() has not been called

Pre: rx_iwdt_init() must have been called successfully

Pre: status must be a valid non-null pointer to an rx_iwdt_status_t

Post: *status contains a copy of s_iwdt_state.status at the time of the call

Post: Internal IWDT state is unchanged

Note: Not thread-safe; status fields may be updated concurrently by the task monitor

Note: Snapshot accuracy depends on timing; use as diagnostic data, not safety-critical logic

Example:: rx_iwdt_status_t status; rx_err_t err = rx_iwdt_get_status(&status); if(err == k_rx_ok) { rx_log_info("wdt", "Activetasks:%u, Feeds:%u", (unsigned)status.active_tasks, (unsigned)status.watchdog_feeds); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_iwdt_was_reset() Check if last reset was hardware IWDT timeout, rx_iwdt_check_tasks() Check all tasks for software timeout

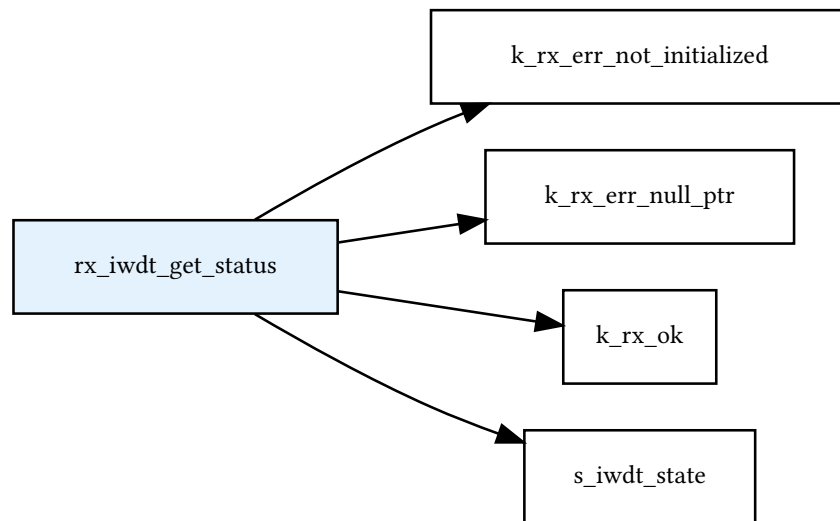


Figure 278: Call graph for rx_iwdt_get_status

Defined in `libs/rx_core/src/rx_iwdt.c:1018`

Since Version 1.0.0

rx_iwdt_was_reset

```
bool rx_iwdt_was_reset(void)
```

Check whether the last microcontroller reset was caused by an IWDTC timeout.

Check if last reset was caused by watchdog.

Reads the IWDTC Reset Flag (IWDTCF) from the Reset Status Register 2 (RSTSR2) of the RX72N. This register is located at a separate address from RSTSR0/1 and retains the reset cause flags across a reset event (values are cleared on power-on reset only).

The IWDTCF bit (bit position k_rstsr2_iwdtcf) is set by hardware when the IWDTC counter underflows (i.e., the watchdog was not refreshed in time). It remains set until explicitly cleared or a power-on reset occurs.

This function is used during startup to detect and log prior watchdog resets, which may indicate a firmware hang or task scheduling failure.

Algorithm steps:

1. Read the RSTSR2 register byte via rstsr2() inline accessor
2. Mask with k_rstsr2_iwdtcf to extract the IWDTC reset flag bit
3. Return true if the bit is set, false otherwise

Returns: bool IWDTC reset detection result

Value	Description
true	RSTSR2.IWDTCF is set; last reset was caused by IWDTC timeout
false	RSTSR2.IWDTCF is clear; last reset was not caused by IWDTC

Pre: None safe to call before rx_iwdt_init() (reads hardware register directly)

Pre: Must be called before the application clears the reset status register

Post: Hardware RSTSR2 register is read-only; this function does not modify it

Post: Return value reflects hardware register state at the instant of the call

Note: Thread-safe reads a single hardware register without shared state

Note: Typically called once at startup to log the reset cause

Warning: RSTSR2.IWDTCF remains set across warm resets; clear it in startup if needed

Example:: if(rx_iwdt_was_reset()){ rx_log_error("boot","PriorresetcausedbyIWDTCtimeout-checktaskhealth"); } rx_iwdt_init(&iwdt_config);

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (safe before init, called before register clear), 2 postconditions

See also: `rx_iwdt_feed()` Feed the hardware watchdog to prevent reset,
`rx_iwdt_check_tasks()` Monitor software task heartbeats

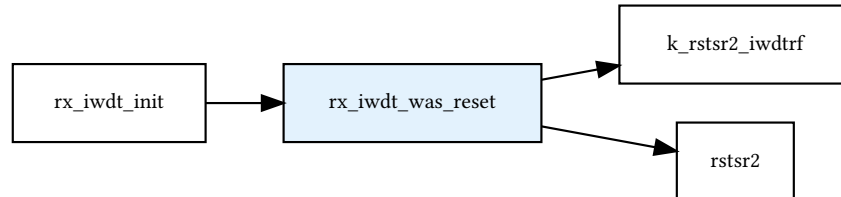


Figure 279: Call graph for rx_iwdt_was_reset

Defined in `libs/rx_core/src/rx_iwdt.c:1085`

Since Version 1.0.0

—

rx_iwdt_check_tasks

```
rx_err_t rx_iwdt_check_tasks(void)
```

Check all registered tasks for software heartbeat timeout.

Check for task timeouts.

Iterates over all active task slots and compares the elapsed tick count since each task's last heartbeat against that task's configured timeout (converted from milliseconds to ThreadX ticks). If any task has exceeded its timeout, the task's `timed_out` flag is set, the task name is recorded in `status.last_failed_task`, and the function returns `k_rx_err_timeout`.

If task monitoring is disabled (`config.enable_task_monitoring == false`), returns `k_rx_ok` immediately without checking any tasks.

The caller (typically the watchdog task) should conditionally feed the hardware IWDT only if this function returns `k_rx_ok` i.e., only refresh the hardware watchdog when all software tasks are alive.

Algorithm steps:

1. Verify `s_iwdt_state.initialized`
2. If `!config.enable_task_monitoring`, return `k_rx_ok` early
3. Capture `current_tick = internal_get_tick_count()`
4. For each active task slot (`i < k_iwdt_max_tasks`):
 - a. Skip inactive slots
 - b. Compute `elapsed_ticks = current_tick - task->last_heartbeat_tick`
 - c. Compute `timeout_in_ticks` from `task->timeout_ms` and `TX_TIMER_TICKS_PER_SECOND`
 - d. If `elapsed_ticks > timeout_in_ticks`: set `timed_out=true`, record task name, set `any_timeout=true`
5. Return `k_rx_err_timeout` if `any_timeout`, else `k_rx_ok`

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	All active tasks are within their heartbeat timeout windows

k_rx_err_timeout	One or more registered tasks have exceeded their timeout; name of first failing task is recorded in status.last_failed_task
k_rx_err_not_initialized	rx_iwdt_init() has not been called

Pre: rx_iwdt_init() must have been called successfully

Pre: Registered tasks must be calling rx_iwdt_task_heartbeat() on schedule

Post: Timed-out tasks have their timed_out flag set

Post: status.last_failed_task contains the name of the most recently detected failure

Note: Not thread-safe; task heartbeat fields may be updated concurrently

Warning: Do NOT feed the hardware IWDG (rx_iwdt_feed()) if this returns k_rx_err_timeout; allow the hardware watchdog to fire and reset the system

```
Example:: while(1){ if(rx_iwdt_check_tasks()==k_rx_ok){ rx_iwdt_feed(); }  
else{ rx_log_error("wdt","Tasktimeoutdetected-allowingIWDGreset"); }  
tx_thread_sleep(k_watchdog_check_interval_ticks); }
```

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (initialized, tasks registered), 2 postconditions

See also: rx_iwdt_feed() Feed hardware watchdog (call only when this returns k_rx_ok),
rx_iwdt_task_heartbeat() Update per-task heartbeat from each monitored task,
rx_iwdt_register_task() Register a task for monitoring

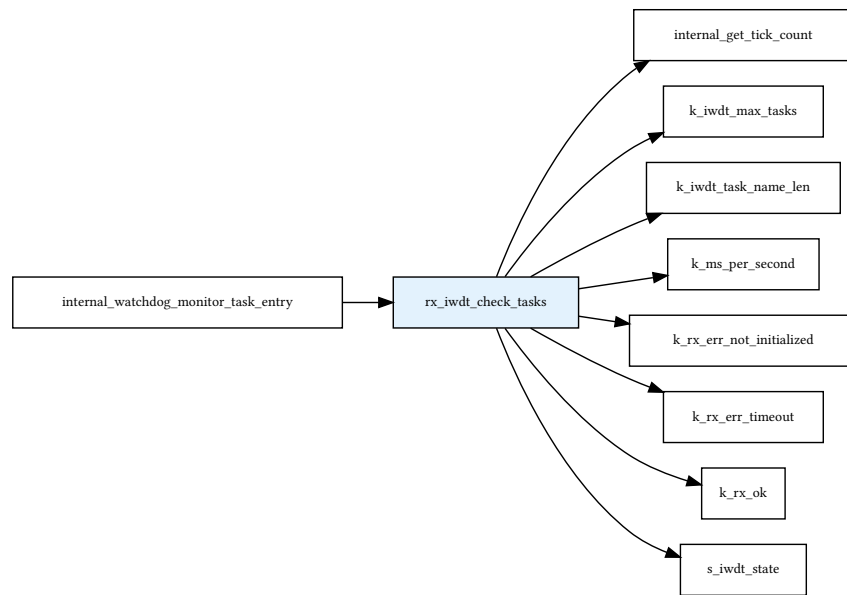


Figure 280: Call graph for rx_iwdt_check_tasks

Defined in `libs/rx_core/src/rx_iwdt.c:1162`

Since Version 1.0.0

—

rx_iwdt.c

Independent Watchdog Timer (IWDT) Driver Implementation for RX72N.

Implements hardware and software watchdog functionality for safety-critical system monitoring. The IWDT provides protection against software hangs, deadlocks, and infinite loops through automatic system reset.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `"rx_hal_iwdt.h"`
- `"rx72n_regs.h"`
- `<inttypes.h>`
- `<stdio.h>`
- `<string.h>`
- `"rx_log.h"`
- `"tx_api.h"`

iwdt_timeout_limits_t

IWDT timeout validation limits.

Defines the valid range of timeout values for the IWDT. These limits are derived from hardware capabilities and practical system requirements.

`k_iwdt_timeout_min_ms < k_iwdt_timeout_max_ms`

Values are compile-time constants

Value	Name	Description
= 1	<code>k_iwdt_timeout_min_ms</code>	Minimum valid timeout in milliseconds.
= 16384	<code>k_iwdt_timeout_max_ms</code>	Maximum valid timeout in milliseconds.

Since Version 1.0.0

—

`iwdt_init_state_t`

IWDT initialization state tracking.

Tracks whether the IWDT has been initialized. Used to prevent:

- Double initialization (hardware watchdog cannot be reconfigured)
- API calls before initialization
- Feed operations on uninitialized watchdog

Value	Name	Description
= 0	<code>k_iwdt_not_initialized</code>	IWDT not initialized - API calls will fail
= 1	<code>k_iwdt_initialized</code>	IWDT initialized - feed required periodically

Warning: Once initialized, IWDT cannot be disabled (hardware feature)

Since Version 1.0.0

—

`iwdt_buffer_size_constants_t`

Task monitoring string buffer size constants.

Defines buffer sizes for task names and log messages. Sized for embedded system constraints while allowing descriptive task names.

`k_task_name_cmp_len == k_task_name_max_len - 1`

`k_log_msg_buffer_size >= 48` (minimum for deadlock message)

Value	Name	Description
= 16	<code>k_task_name_max_len</code>	Maximum task name length including null terminator
= 15	<code>k_task_name_cmp_len</code>	Length for strncmp comparison (excludes null)
= 64	<code>k_log_msg_buffer_size</code>	Log message buffer size for snprintf formatting

Since Version 1.0.0

—

s_timeout_table

```
const iwdt_timeout_entry_t s_timeout_table[]
```

IWDT timeout configuration lookup table.

Note: Values are slightly longer than requested to provide safety margin

Note: Table is const and placed in .rodata section (48 bytes total)

Warning: Do not modify table at runtime (const data)] **See also:**
internal_find_timeout_config() Table lookup function, iwdt_timeout_entry_t Entry
structure definition

Since Version 1.0.0

—

s_iwdt_timeout_table_size

```
const uint32_t s_iwdt_timeout_table_size
```

Number of entries in the timeout lookup table.

Since Version 1.0.0

—

s_tag

```
const char* const s_tag
```

Log tag for IWDT module.

Note: Pointer to string literal (stored in .rodata section)

Warning: Do not modify (const pointer to const data)

Since Version 1.0.0

—

s_iwdt_initialized

```
iwdt_init_state_t s_iwdt_initialized
```

Flag indicating IWDT has been initialized.

Warning: Do not modify directly - use rx_hal_iwdt_init() only

Since Version 1.0.0

—

internal_find_timeout_config

```
static rx_err_t internal_find_timeout_config(const uint32_t timeout_ms, const  
iwdt_timeout_entry_t **entry)
```

Find best timeout configuration for requested timeout.

Performs linear search through `s_timeout_table` to find the first entry whose `timeout_ms` is $\geq$ the requested value. This provides the smallest timeout that meets or exceeds the requirement.

Parameters:

Direction	Name	Description
in	<code>timeout_ms</code>	Requested timeout in milliseconds (1-16384)
out	<code>entry</code>	Pointer to receive configuration entry pointer (points into <code>s_timeout_table</code> , do not free)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Valid configuration found and stored in <code>*entry</code>
<code>k_rx_err_invalid_arg</code>	entry is nullptr, table empty, or timeout out of range

Pre: `timeout_ms` $\geq$ `k_iwdt_timeout_min_ms`

Pre: `entry` $\neq$ nullptr

Pre: `s_timeout_table` populated (compile-time guarantee)

Post: `*entry` points to valid table entry with `timeout_ms` $\geq$ requested

Post: `*entry` is const - returned entry must not be modified

Note: Returns pointer to static data - valid for program lifetime

Note: Selected timeout may be longer than requested (next available)] **Algorithm:** * 1. Validate table is not empty * 2. Validate output pointer is not nullptr * 3. Linear scan through sorted table: * - If `entry.timeout_ms` $\geq$ requested: return entry pointer * 4. If no entry found: requested timeout exceeds maximum *

Complexity: Time: $O(n)$ where $n = s_iwdt_timeout_table_size$ (typically 6) Space: $O(1)$ - no additional memory allocated

Example:: `const iwdt_timeout_entry_t* config = nullptr;`
`rx_err_t err = internal_find_timeout_config(1000, &config); if (err == k_rx_ok) { }`

See also: `s_timeout_table` Source data for lookup, `iwdt_timeout_entry_t` Structure returned

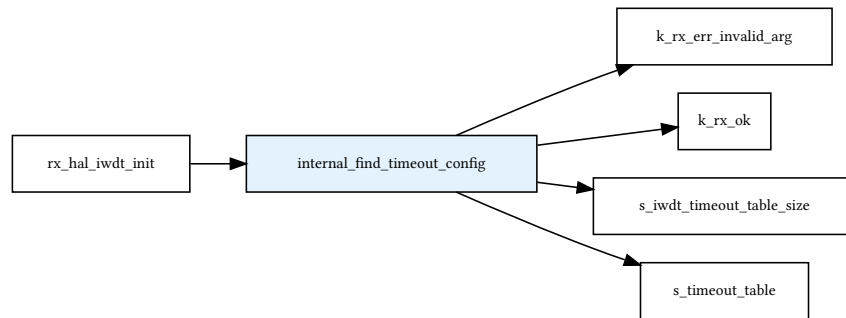


Figure 281: Call graph for internal_find_timeout_config

Defined in `libs/rx_hal/src/rx_iwdt.c:480`

Since Version 1.0.0

internal_configure_iwdt_control_register

```
static rx_err_t internal_configure_iwdt_control_register(const
iwdt_timeout_entry_t *config)
```

Configure IWDT control register (IWDTCR) from timeout table entry.

Builds and writes the IWDTCR register value from a timeout configuration entry. Combines TOPS, CKS, RPES, and RPSS fields into a single 16-bit value.

Parameters:

Direction	Name	Description
in	config	Pointer to timeout configuration entry from lookup table

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Register written successfully
k_rx_err_invalid_arg	config is nullptr
k_rx_err_hw_unmapped	IWDT registers not accessible

Pre: config must point to valid iwdt_timeout_entry_t

Pre: IWDT not yet started (first refresh not performed)

Pre: iwdt() returns valid hardware register pointer

Post: IWDTCR configured with specified timeout and window settings

Note: Not thread-safe: must be called from single-threaded initialization context

Note: Window mode disabled (RPES=0%, RPSS=100%)

Note: Register can only be configured before first refresh

Warning: IWDTCR becomes read-only after first refresh sequence] **Register Construction:** *
 IWDTCR = TOPS | CKS | RPES | RPSS * * Where: * - TOPS[1:0]: Timeout Period Select (from
 config) * - CKS[7:4]: Clock Division Ratio (from config) * - RPES[9:8]: Window End = 0%
 (disabled) * - RPSS[13:12]: Window Start = 100% (full window) *

Register Write Sequence: * 1. Validate config pointer (NULL check) * 2. Validate hardware pointer
 (iwdt()) check) * 3. Build IWDTCR value from fields * 4. Write to IWDTCR register *

See also: internal_find_timeout_config() Source of config parameter

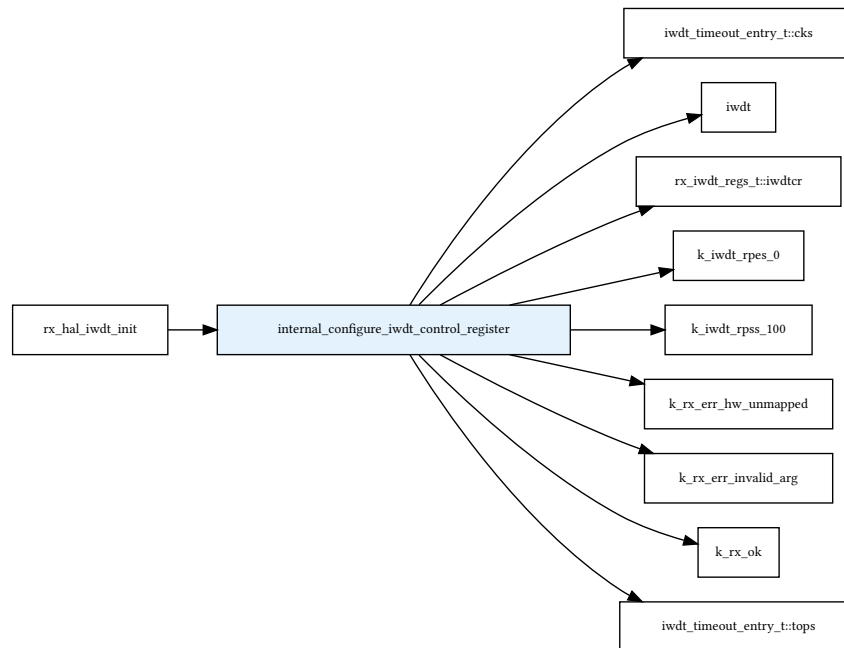


Figure 282: Call graph for internal_configure_iwdt_control_register

Defined in `libs/rx_hal/src/rx_iwdt.c:558`

Since Version 1.0.0

internal_configure_iwdt_registers

```
static rx_err_t internal_configure_iwdt_registers(void)
```

Configure IWDTCR reset and sleep-count behavior registers.

Configures IWDTRCR (Reset Control) and IWDTCSTPR (Count Stop Control) registers for safety-critical operation:

- Full chip reset on timeout (not NMI)
- Continue counting during sleep modes

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Registers configured successfully
k_rx_err_hw_unmapped	IWDTCR registers not accessible
k_rx_err_invalid_state	IWDTCR already initialized

Pre: IWDTCR not yet initialized (s_iwdtc_initialized == k_iwdtc_not_initialized)

Pre: iwdtc() returns valid hardware register pointer

Pre: Called after internal_configure_iwdtc_control_register()

Post: IWDTRCR configured for reset action

Post: IWDTCSTPR configured to continue counting in sleep

Note: Not thread-safe: must be called from single-threaded initialization context

Note: These registers can only be written before first refresh

Warning: Changing RSTIRQS to NMI may compromise system safety] **Register**

Configuration: * IWDTRCR (Reset Control Register): * +-----+ * |
 Bit 7 (RSTIRQS) = 1: Reset on underflow/error | * | 0: NMI on underflow/error (not used) | * +
 -----+ * Using reset (1) for safety - NMI could be masked/ignored *
 * IWDTCSTPR (Count Stop Control Register): * +-----+ * | Bit 7
 (SLCSTP) = 0: Continue counting in sleep mode | * | 1: Stop counting in sleep (not used) | * +
 -----+ * Continue counting for safety - sleeping should not bypass
 watchdog *

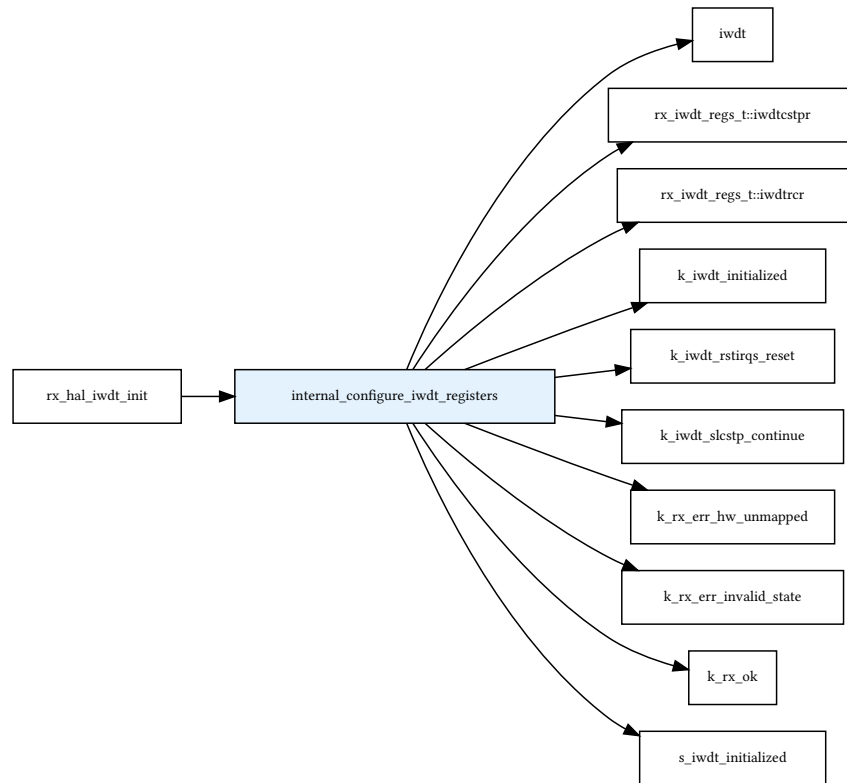


Figure 283: Call graph for internal_configure_iwdt_registers

Defined in `libs/rx_hal/src/rx_iwdt.c:622`

Since Version 1.0.0

—

rx_hal_iwdt_init

```
rx_err_t rx_hal_iwdt_init(const uint32_t timeout_ms)
```

Initialize the Independent Watchdog Timer.

Configures and starts the hardware IWDT with the specified timeout period. Once started, the watchdog cannot be stopped - periodic calls to `rx_hal_iwdt_feed()` are required to prevent system reset.

Parameters:

Direction	Name	Description
in	timeout_ms	Requested timeout period in milliseconds (1-16384) Actual timeout may be slightly longer (next available)

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	IWDT initialized and running
k_rx_err_invalid_state	Already initialized
k_rx_err_invalid_arg	timeout_ms out of valid range
k_rx_err_hw_unmapped	Hardware registers not accessible

Pre: IWDT must not be already initialized

Pre: timeout_ms in range k_iwdt_timeout_min_ms, k_iwdt_timeout_max_ms

Pre: System startup, single-threaded context

Post: IWDT running with selected timeout period

Post: s_iwdt_initialized set to k_iwdt_initialized

Post: Periodic rx_hal_iwdt_feed() calls now required

Note: On host builds (non-RX), skips hardware access but sets initialized flag

Note: Actual timeout $\geq$ requested (selected from lookup table)

Warning: Once initialized, IWDT cannot be disabled (requires hard reset)

Warning: Must call rx_hal_iwdt_feed() faster than timeout period] **Initialization Sequence:** *

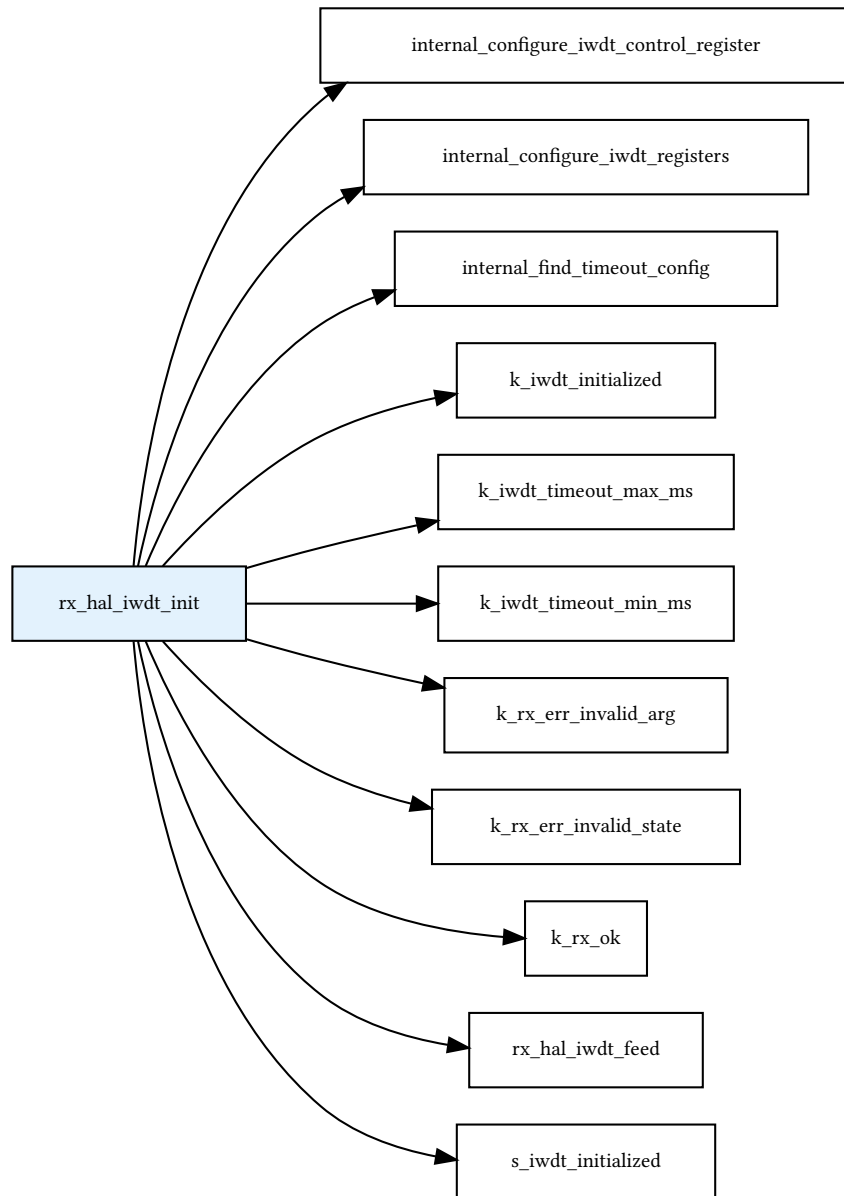
1. Validate not already initialized *
2. Validate timeout_ms within range *
3. Look up timeout configuration in table *
4. Configure IWDTCR (timeout period, clock divisor, window) *
5. Configure IWDTRCR (reset action - not NMI) *
6. Configure IWDTCSTPR (continue counting in sleep) *
7. Perform first refresh (starts the watchdog) *
8. Set initialized flag *

Point of No Return: * After step 7 (first refresh), the IWDT is running and CANNOT be stopped. *
Any failure after this point leaves system in watchdog-protected state. *

Example:: rx_err_t err=rx_hal_iwdt_init(2000); if(err!=k_rx_ok){ while(1){ }

while(1){ rx_hal_iwdt_feed(); tx_thread_sleep(100); }

See also: rx_hal_iwdt_feed() Reset watchdog counter, rx_hal_iwdt.h Public API and configuration details

Figure 284: Call graph for `rx_hal_iwdt_init`

Defined in `libs/rx_hal/src/rx_iwdt.c:709`

Since Version 1.0.0

—

rx_hal_iwdt_feed

```
void rx_hal_iwdt_feed(void)
```

Feed the watchdog timer to prevent reset.

Feed (refresh) the watchdog timer.

Performs the IWDTR refresh sequence by writing the magic values 0x00 then 0xFF to the IWDTRR register. This resets the down-counter to its initial value, preventing timeout and system reset.

Pre: IWDTR must be initialized via rx_hal_iwdt_init()

Pre: Called at rate faster than configured timeout

Post: IWDTR counter reset to initial value

Post: Interrupt state restored to previous value

Note: Safe to call from ISRs (uses atomic disable/restore)

Note: On host builds, function is a no-op

Warning: Incomplete refresh sequence causes refresh error -> system reset

Warning: Missing calls to this function cause timeout -> system reset] **Refresh Sequence (Atomic Operation):** * +-----+ * | 1. Disable interrupts (save PSW) | * | 2. Write 0x00 to IWDTRR | * | 3. Write 0xFF to IWDTRR | * | 4. Restore interrupt state | * +-----+ * **CRITICAL:** Steps 2 and 3 must not be interrupted! *

Timing Requirements: * - Execution time: 2 us including interrupt disable/enable * - Must be called faster than configured timeout period * - Typical call rate: Every 100-500 ms depending on timeout *

Example:: while(1){ rx_err_t err=rx_hal_iwdt_check_tasks(); if(err==k_rx_ok){ rx_hal_iwdt_feed(); } tx_thread_sleep(100); }

See also: rx_hal_iwdt_init() Start the watchdog, rx_hal_iwdt_check_tasks() Verify task health before feeding

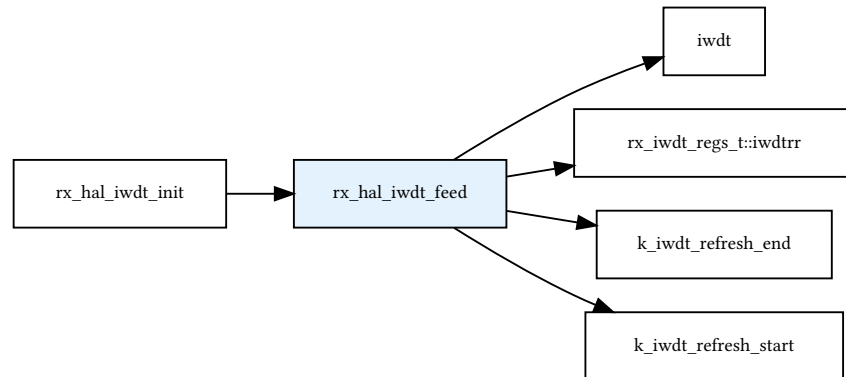


Figure 285: Call graph for rx_hal_iwdt_feed

Defined in `libs/rx_hal/src/rx_iwdt.c:844`

Since Version 1.0.0

—

rx_hal_iwdt_was_reset

```
bool rx_hal_iwdt_was_reset(void)
```

Check if system was reset by watchdog.

Check if last reset was caused by watchdog timeout.

Reads the IWDT Status Register (IWDTSR) to determine if the previous system reset was caused by watchdog timeout (underflow) or refresh error. This function should be called early in startup for diagnostics.

Returns: bool Reset cause indication

Value	Description
true	Last reset was caused by watchdog (underflow or refresh error)
false	Last reset was NOT caused by watchdog (power-on, external, etc.)

Pre: Can be called before rx_hal_iwdt_init()

Pre: Status flags preserved across reset (hardware feature)

Post: Status flags NOT cleared (call rx_hal_iwdt_clear_status() to clear)

Note: On host builds, always returns false

Note: Call rx_hal_iwdt_get_reset_cause() for specific cause] **Status Register Flags:** * IWDTSR Bit 14 (UNDF) - Underflow Flag: * - 1: Counter reached 0 (timeout occurred) * - 0: No underflow

**** IWDTSR Bit 15 (REFEF) - Refresh Error Flag: * - 1: Invalid refresh sequence detected * - 0: No refresh error ***

Example:: if(rx_hal_iwdt_was_reset()){ uart_debug_puts("[WARN]Recoveredfromwatchdogreset!rn"); constchar*failed=rx_hal_iwdt_get_failed_task(); if(failed!=nullptr) { uart_debug_printf("Failedtask:%srn",failed); } }

See also: rx_hal_iwdt_get_reset_cause() Get specific reset cause,
rx_hal_iwdt_clear_status() Clear status flags after reading,
rx_hal_iwdt_get_failed_task() Get name of task that caused timeout

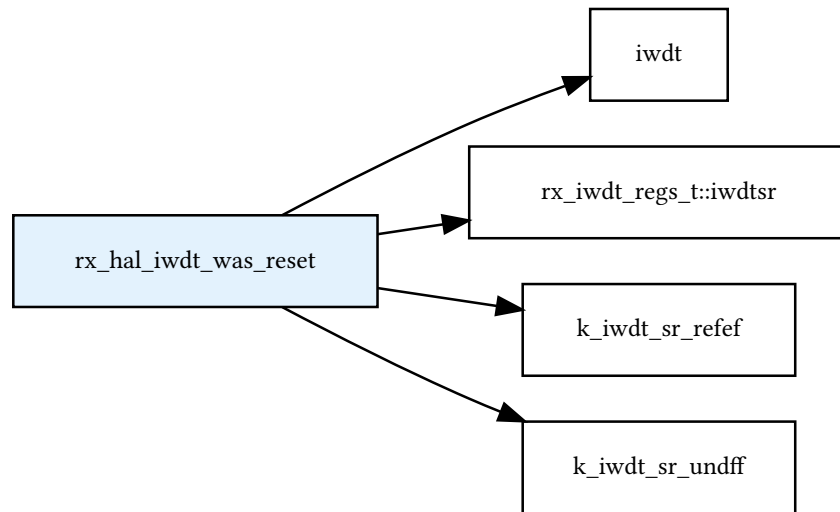


Figure 286: Call graph for rx_hal_iwdt_was_reset

Defined in `libs/rx_hal/src/rx_iwdt.c:927`

Since Version 1.0.0

—

rx_hal_iwdt_get_reset_cause

```
rx_iwdt_reset_cause_t rx_hal_iwdt_get_reset_cause(void)
```

Get detailed watchdog reset cause.

Get detailed reset cause from IWDTSR.

Reads the IWDTSR Status Register to determine the specific cause of the last watchdog reset. Refresh errors take priority over underflow since they indicate a software bug in the refresh sequence.

Returns: rx_iwdt_reset_cause_t Specific reset cause

Value	Description
k_iwdt_reset_refresh_error	Invalid refresh sequence detected
k_iwdt_reset_underflow	Counter timeout (feed not called in time)

k_iwdt_reset_none	No watchdog reset (power-on, external, etc.)
-------------------	--

Pre: Can be called before rx_hal_iwdt_init()

Pre: Status flags preserved across reset

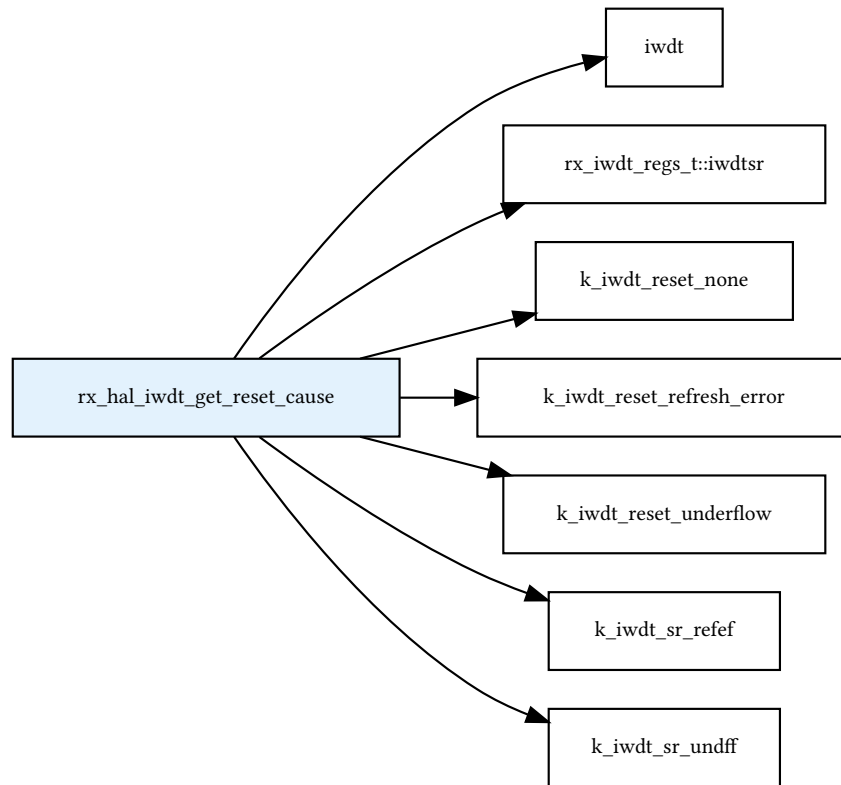
Post: Status flags NOT cleared (call rx_hal_iwdt_clear_status() to clear)

Note: On host builds, always returns k_iwdt_reset_none

Note: Refresh error has priority over underflow in return value] **Reset Cause Priority:** * 1. Refresh Error (REFEF=1): Invalid refresh sequence * - Incomplete 0x00/0xFF write * - Write occurred outside window (if window mode enabled) * * 2. Underflow (UNDF=1): Counter reached zero * - rx_hal_iwdt_feed() not called within timeout period * - Software hang, deadlock, or infinite loop * * 3. None: No watchdog reset occurred * - Power-on reset, external reset, or other cause *

Example:: rx_iwdt_reset_cause_tcause=rx_hal_iwdt_get_reset_cause(); switch(cause)
 { casek_iwdt_reset_refresh_error: uart_debug_puts("ERROR:Watchdogrefreshsequenceerror!\n");
 break; casek_iwdt_reset_underflow: uart_debug_puts("WARN:Watchdogtimeout-
 softwarehangdetected\n"); break; casek_iwdt_reset_none: default:
 uart_debug_puts("Normalstartup\n"); break; } rx_hal_iwdt_clear_status();

See also: rx_hal_iwdt_was_reset() Quick check for any watchdog reset,
 rx_hal_iwdt_clear_status() Clear status flags

Figure 287: Call graph for `rx_hal_iwdt_get_reset_cause`

Defined in `libs/rx_hal/src/rx_iwdt.c:1006`

Since Version 1.0.0

—

rx_hal_iwdt_clear_status

```
void rx_hal_iwdt_clear_status(void)
```

Clear watchdog status flags.

Clears the UNDF (underflow) and REFEF (refresh error) flags in the IWDT Status Register. These flags are “write-0-to-clear” type - writing 0 to a flag clears it, writing 1 has no effect.

Pre: Status has been read via `rx_hal_iwdt_was_reset()` or `rx_hal_iwdt_get_reset_cause()`

Post: UNDF flag cleared

Post: REFEF flag cleared

Note: On host builds, function is a no-op

Note: Call after reading status at startup to prepare for next reset] **Clear Sequence:** * 1. Read IWDTSR (get current status) * 2. AND with (UNDFE | REFEF) to clear both flags * 3. Write back to IWDTSR * * Note: Read-modify-write is safe because flags are write-0-to-clear *

Example:: if(rx_hal_iwdt_was_reset()){ rx_iwdt_reset_cause_tcause=rx_hal_iwdt_get_reset_cause(); log_watchdog_reset(cause); } rx_hal_iwdt_clear_status();

See also: rx_hal_iwdt_was_reset() Check reset status, rx_hal_iwdt_get_reset_cause() Get specific cause

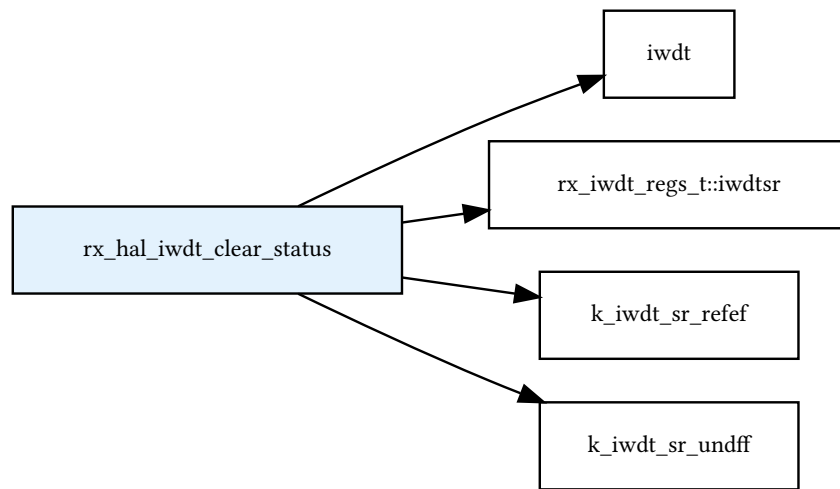


Figure 288: Call graph for rx_hal_iwdt_clear_status

Defined in `libs/rx_hal/src/rx_iwdt.c:1067`

Since Version 1.0.0

—

rx_log_usb.c

USB CDC logging backend implementation with boot buffering and thread safety.

Provides enhanced USB CDC logging support for the rx_log system with:

- Boot log buffering: 512B ring buffer for logs during USB enumeration
- Thread safety: ThreadX mutex for multi-task logging
- Error handling: Graceful handling of USB TX buffer full conditions
- Statistics: Tracking of total, dropped, and buffered logs
- Non-blocking: Never blocks on USB TX full (drops logs with statistics)

Includes:

- "rx_log.h"
- <stdbool.h>
- <stdint.h>
- <string.h>

- "rx_usb.h"
- "tx_api.h"

usb_log_buffer_config_t

Boot log buffer configuration.

Value	Name	Description
= 512	k_boot_buffer_size	Boot log buffer size in bytes.
= 64	k_flush_chunk_size	Maximum chunk size for buffer flush.

—

s_boot_buffer

usb_log_boot_buffer_t [s_boot_buffer](#)

Boot log buffer (used until USB configured).

—

s_usb_ready

volatile bool [s_usb_ready](#)

USB ready flag (set when k_usb_state_configured reached).

—

s_stats

usb_log_stats_t [s_stats](#)

Logging statistics.

—

s_log_mutex

TX_MUTEX [s_log_mutex](#)

ThreadX mutex for thread safety.

—

s_mutex_initialized

bool [s_mutex_initialized](#)

Mutex initialization flag (lazy init on first use).

—

internal_init_mutex_once

```
static void internal_init_mutex_once(void)
```

Initialize mutex on first use (lazy initialization).

Creates ThreadX mutex with no priority inheritance. Called automatically on first log write. Subsequent calls are no-ops.

Returns: void (no error indication - failures silently ignored)

Pre: ThreadX kernel must be running (called after tx_kernel_enter())

Post: s_mutex_initialized = true on success, s_log_mutex created

Post: Logging proceeds without mutex if creation fails (unsafe but functional)

Note: Thread Safety: Not thread-safe itself, but called before mutex is needed

Note: Failure Mode: Silent failure if tx_mutex_create() fails (logs continue without protection)

Algorithm:: Check if already initialized (s_mutex_initialized flag) If not, create ThreadX mutex with TX_NO_INHERIT policy Set s_mutex_initialized = true on success On failure, leave flag false (logging proceeds unprotected)

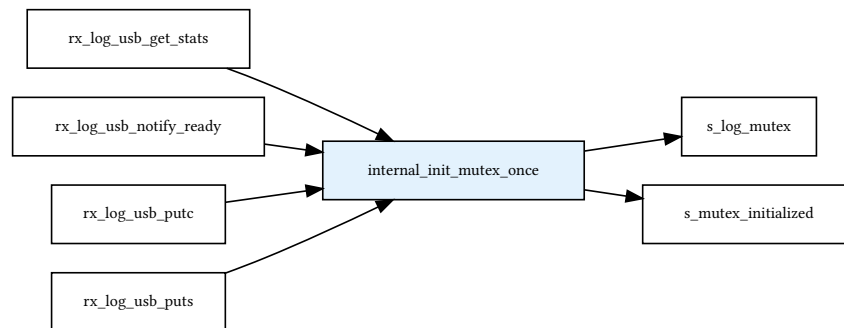


Figure 289: Call graph for internal_init_mutex_once

_Defined in libs/rx_core/src/rx_log_usb.c:162_

Since Version 1.0.0

—

internal_buffer_boot_log

```
static rx_err_t internal_buffer_boot_log(const char *data, uint16_t len)
```

Buffer log data to boot buffer (before USB ready).

Appends data to ring buffer if space available. If buffer full, drops data and sets overflow flag. Overflow warning logged after USB ready.

Boot log buffering enables logs emitted during system startup (before USB enumeration completes) to be preserved and flushed to USB once the CDC interface becomes ready. The ring buffer wraps writes using a head index and a running byte count.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	data	Pointer to data bytes to buffer (must not be NULL)
in	len	Number of bytes to buffer (must be > 0)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Data buffered successfully
k_rx_err_no_mem	Buffer full, data dropped and overflow flag set

Pre: Mutex locked by caller (internal_init_mutex_once() invoked before call)

Pre: s_boot_buffer.head is within 0, k_boot_buffer_size)

Post: s_boot_buffer.count incremented by len on success

Post: s_stats.boot_buffered incremented by len on success

Post: s_boot_buffer.overflow set and s_stats.dropped_bytes incremented on overflow

Note: Not thread-safe on its own; caller must hold s_log_mutex before invoking

Example:: rx_err_t result=internal_buffer_boot_log("bootmessagen",14);
if(result==k_rx_err_no_mem){ }

See also: internal_check_usb_ready() Flushes this buffer once USB is configured,
rx_log_usb_puts() Public API that triggers this path during boot

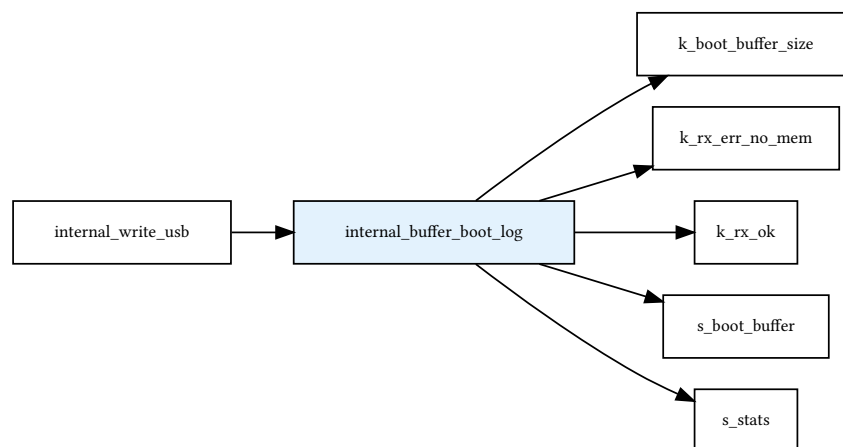


Figure 290: Call graph for internal_buffer_boot_log

_Defined in libs/rx\core/src/rx_log_usb.c:214_

Since Version 1.0.0

—

internal_check_usb_ready

```
static void internal_check_usb_ready(void)
```

Check USB ready status and flush boot buffer if ready.

Polls USB CDC Port 2 configuration status. If configured and not yet flushed, sends all buffered boot logs to USB in 64-byte chunks.

Flush Strategy:

- Sends in k_flush_chunk_size (64 byte) chunks for efficiency
- If USB TX full during flush, aborts and retries on next call
- Logs overflow warning if boot buffer overflowed

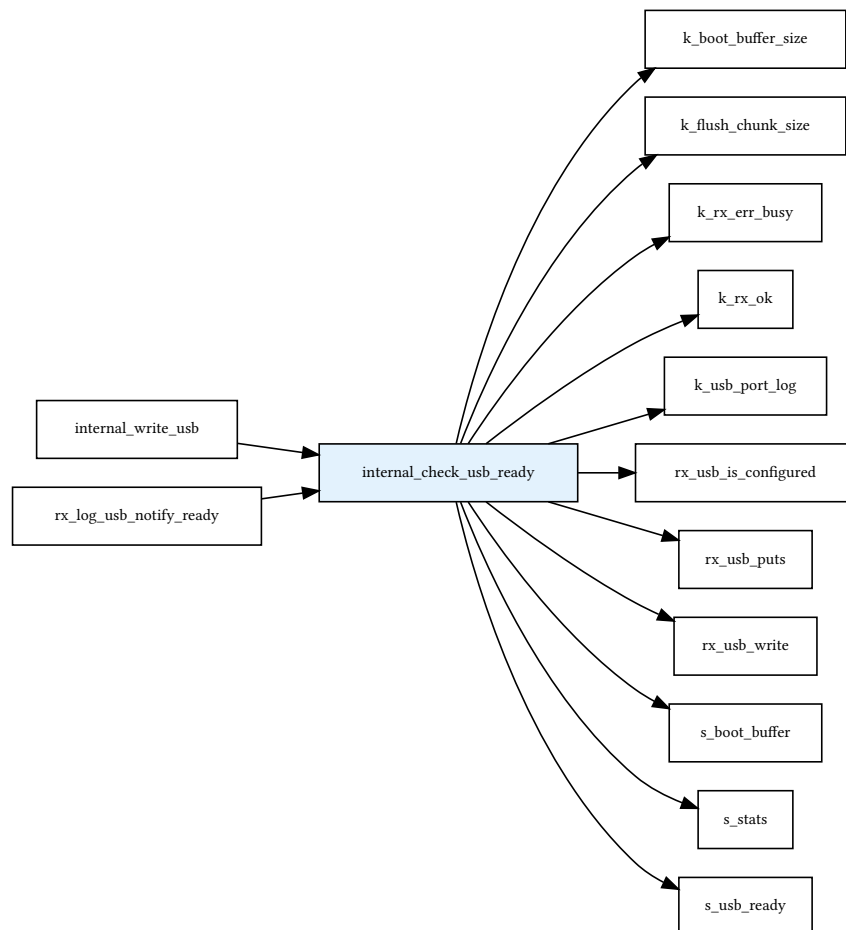


Figure 291: Call graph for internal_check_usb_ready

_Defined in libs/rx_core/src/rx_log_usb.c:278_

internal_write_usb

```
static void internal_write_usb(const char *data, uint16_t len)
```

Write data to USB CDC with error handling.

Sends data to USB CDC Port 2 if configured, otherwise buffers to boot buffer. Handles errors gracefully:

- k_rx_err_busy: Drop data, increment dropped count (non-blocking)
- Other errors: Log error, increment error count

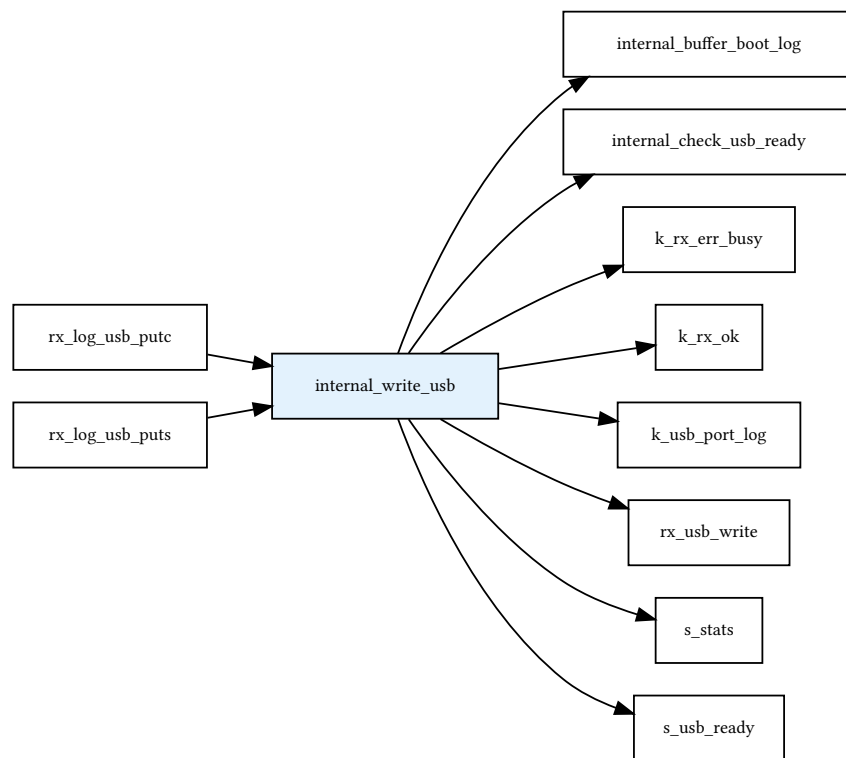


Figure 292: Call graph for internal_write_usb

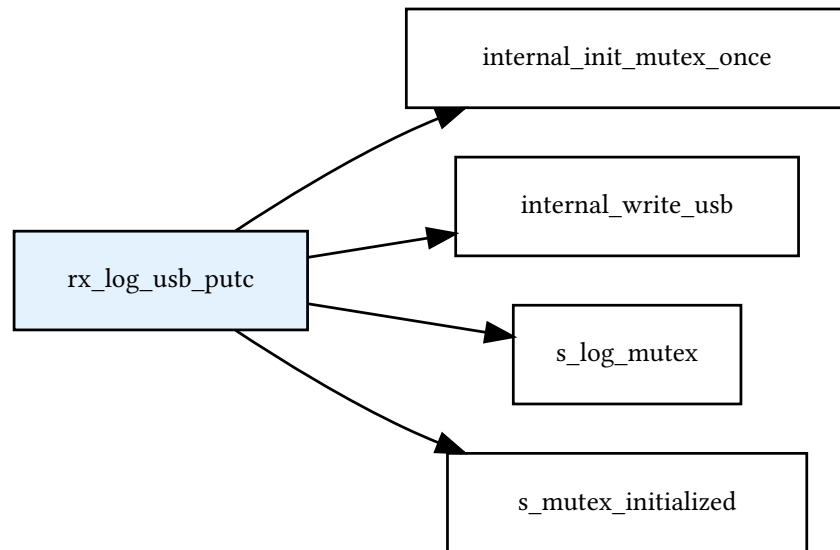
_Defined in libs/rx_core/src/rx_log_usb.c:383_

rx_log_usb_putc

```
void rx_log_usb_putc(char c)
```

Write a single character to USB CDC log port.

Thread-safe wrapper around `internal_write_usb()`. Buffers character during boot, sends to USB after enumeration.

Figure 293: Call graph for `rx_log_usb_putc`

Defined in `libs/rx\_core/src/rx\_log\_usb.c:456`

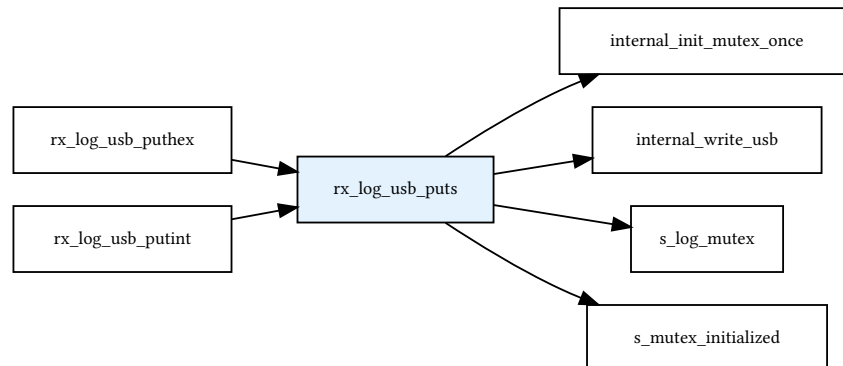
—

rx_log_usb_puts

```
void rx_log_usb_puts(const char *str)
```

Write a null-terminated string to USB CDC log port.

Thread-safe wrapper around `internal_write_usb()`. Buffers string during boot, sends to USB after enumeration.

Figure 294: Call graph for `rx_log_usb_puts`

Defined in `libs/rx\_core/src/rx\_log\_usb.c:524`

—

rx_log_usb_putint

```
void rx_log_usb_putint(int32_t value)
```

Write a signed integer as decimal text to USB CDC log port.

Converts integer to ASCII decimal string and writes to USB. Handles negative numbers with leading '-' sign.

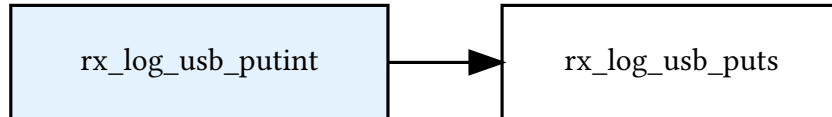


Figure 295: Call graph for rx_log_usb_putint

_Defined in libs/rx_core/src/rx_log_usb.c:608_

rx_log_usb_puthex

```
void rx_log_usb_puthex(uint32_t value, uint8_t digits)
```

Write an unsigned integer as hexadecimal text to USB CDC log port.

Converts integer to ASCII hex string (uppercase A-F) with zero-padding. Useful for register values, memory addresses, bit masks, and error codes.

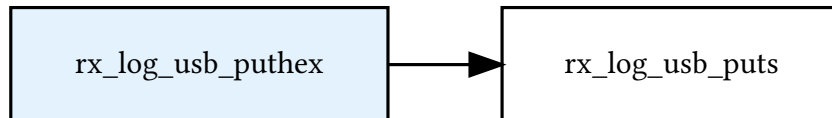


Figure 296: Call graph for rx_log_usb_puthex

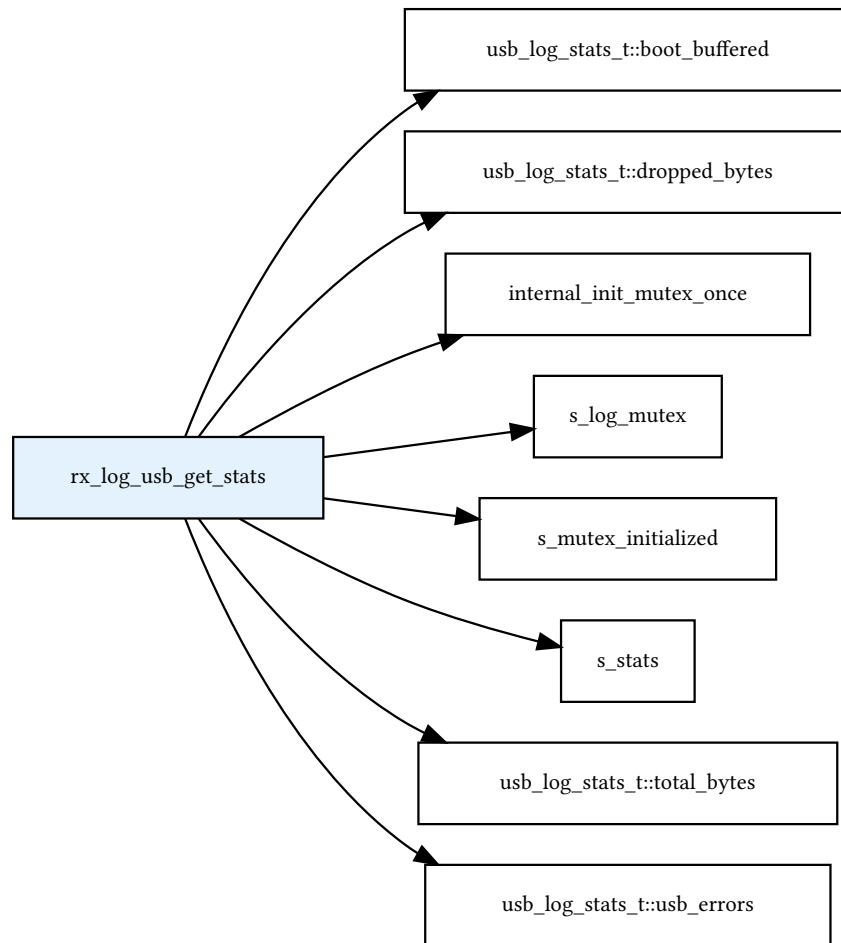
_Defined in libs/rx_core/src/rx_log_usb.c:716_

rx_log_usb_get_stats

```
void rx_log_usb_get_stats(usb_log_stats_t *stats)
```

Get USB CDC logging statistics.

Returns current logging statistics (total, dropped, buffered, errors). Useful for diagnostics and performance monitoring.

Figure 297: Call graph for `rx_log_usb_get_stats`

_Defined in `libs/rx_core/src/rx_log_usb.c:832_`

—

rx_log_usb_notify_ready

```
void rx_log_usb_notify_ready(void)
```

Notify logging system that USB is ready (external callback).

Notify logging system that USB is ready (optional callback).

Called by USB stack after successful enumeration (`k_usb_state_configured`). Triggers immediate flush of boot buffer. Optional - flush also happens automatically on next log write.

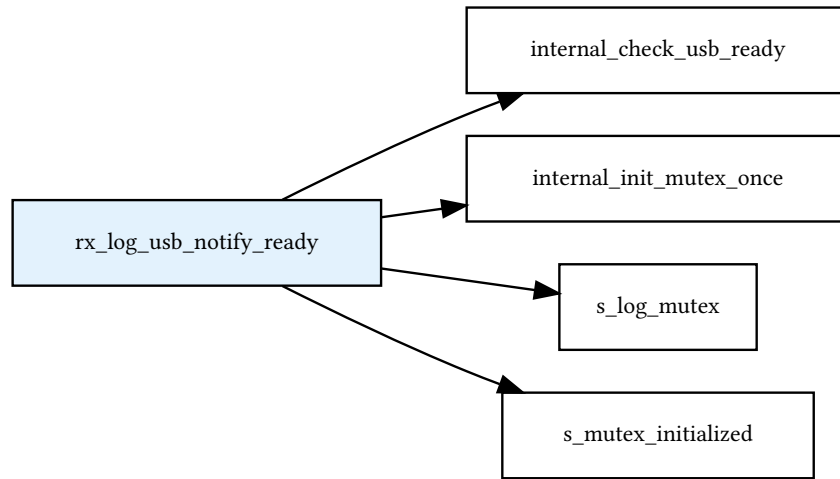


Figure 298: Call graph for rx_log_usb_notify_ready

_Defined in libs/rx_core/src/rx_log_usb.c:942_

rx_pin_validator.c

Pin Validator Concrete Implementation - GPIO Pin Tracking and Conflict Prevention.

Includes:

- "rx_pin_validator.h"
- <string.h>
- "rx_check.h"
- "rx_gpio_constants.h"
- "rx_log.h"

s_zero_validator

`const pin_validator_t s_zero_validator`

Zero-initialized pin validator template for stack-safe initialization.

Note: Read-only; never modified after static initialization

Warning: Do not remove - prevents stack overflow in init function] **See also:**
`pin_validator_init()` Uses this template to zero-initialize the validator

Since Version 1.0.0

internal_port_to_index

```
static uint8_t internal_port_to_index(const uint8_t port)
```

Convert port number to internal array index.

Maps the RX72N's non-contiguous port numbering scheme to a contiguous array index. The RX72N uses decimal port numbers 0-9 and hexadecimal port letters A-G (0xA-0x10). This function normalizes both schemes to a single 0-16 index range.

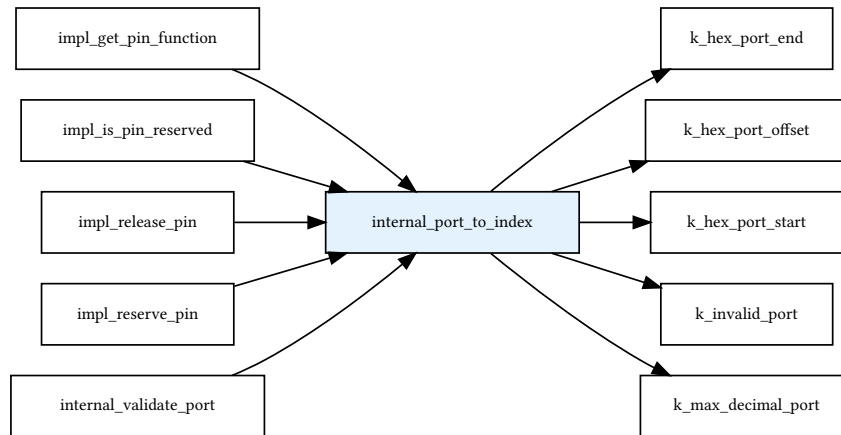


Figure 299: Call graph for `internal_port_to_index`

_Defined in `libs/rx\core/src/rx\_pin\_validator.c:214_`

`internal_validate_port`

```
static rx_err_t internal_validate_port(const uint8_t port)
```

Validate port number against RX72N hardware limits.

Checks if the given port number corresponds to a valid GPIO port on the RX72N microcontroller. Uses `internal_port_to_index()` to perform the actual validation.

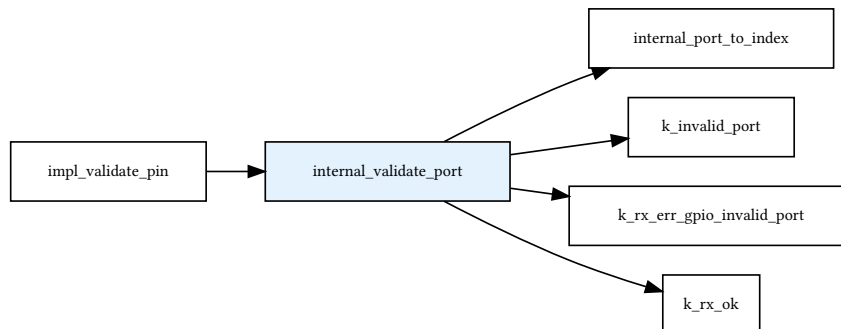


Figure 300: Call graph for `internal_validate_port`

_Defined in `libs/rx\core/src/rx\_pin\_validator.c:258_`

internal_validate_pin

```
static rx_err_t internal_validate_pin(const uint8_t pin)
```

Validate pin number against RX72N per-port limits.

Checks if the given pin number is within the valid range for any RX72N GPIO port. Each port supports up to 8 pins (0-7), though not all ports have all 8 pins physically available on the package.

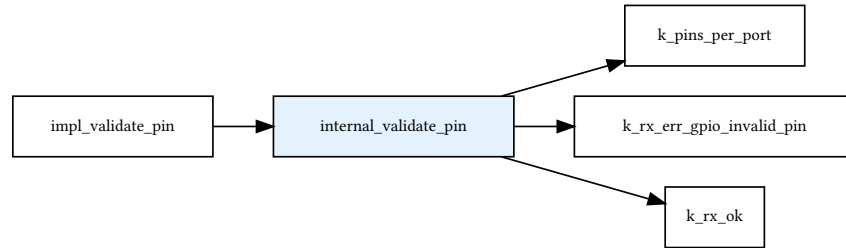


Figure 301: Call graph for `internal_validate_pin`

Defined in `libs/rx\_core/src/rx\_pin\_validator.c:302`

impl_validate_pin

```
static rx_err_t impl_validate_pin(void *ctx, const uint8_t port, const uint8_t pin)
```

Validate pin - Interface implementation for `rx_pin_interface_t::validate_pin`.

Checks if a port/pin combination is valid for the RX72N hardware. This function does NOT check reservation status - it only validates that the port and pin numbers are within valid hardware ranges.

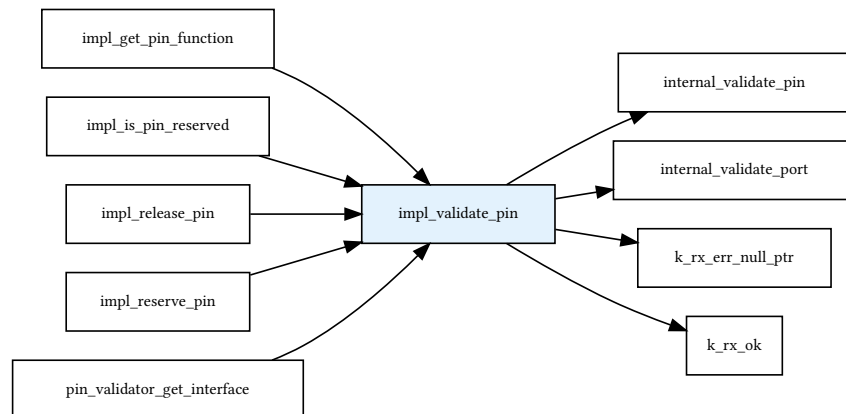


Figure 302: Call graph for `impl_validate_pin`

Defined in `libs/rx\_core/src/rx\_pin\_validator.c:357`

impl_reserve_pin

```
static rx_err_t impl_reserve_pin(void *ctx, const uint8_t port, const uint8_t
pin, const char *function)
```

Reserve pin - Interface implementation for rx_pin_interface_t::reserve_pin.

Attempts to reserve a GPIO pin for exclusive use by the specified function. This is the core conflict-prevention mechanism: if a pin is already reserved, the function returns an error instead of allowing double-allocation.

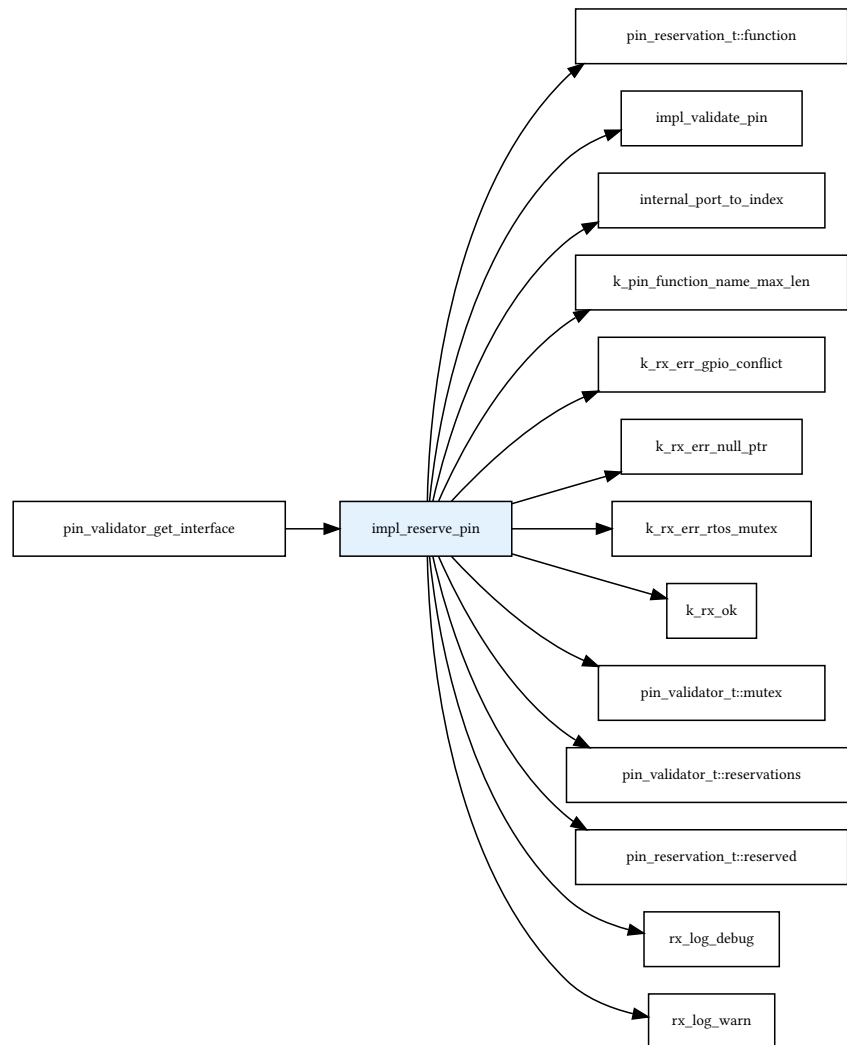


Figure 303: Call graph for `impl_reserve_pin`

_Defined in `libs/rx\core/src/rx\_pin\_validator.c:477_`

impl_release_pin

```
static rx_err_t impl_release_pin(void *ctx, const uint8_t port, const uint8_t pin)
```

Release pin - Interface implementation for rx_pin_interface_t::release_pin.

Releases a previously reserved GPIO pin, making it available for reservation by other functions. Must be called when a module no longer needs exclusive access to a pin.



Figure 304: Call graph for `impl_release_pin`

_Defined in `libs/rx\core/src/rx\_pin\_validator.c:590_`

impl_is_pin_reserved

```
static bool impl_is_pin_reserved(void *ctx, const uint8_t port, const uint8_t
pin)
```

Check if pin is reserved - Interface implementation for rx_pin_interface_t::is_pin_reserved.

Queries the reservation status of a GPIO pin without attempting to reserve it. Useful for checking pin availability before making allocation decisions.

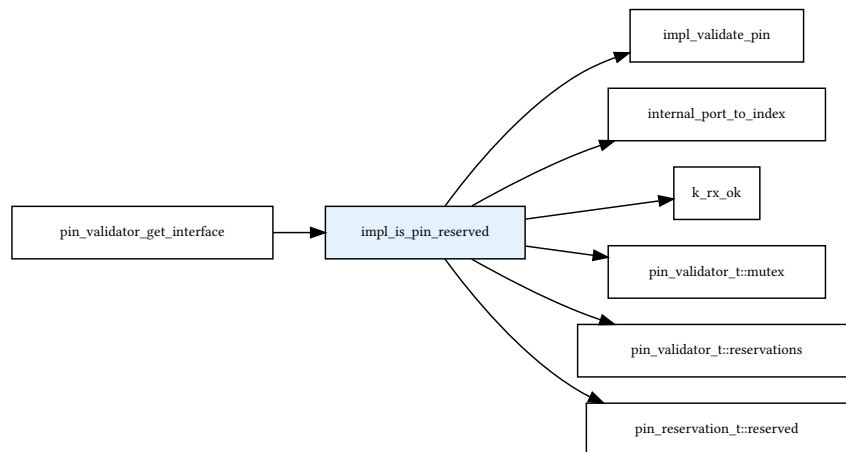


Figure 305: Call graph for `impl_is_pin_reserved`

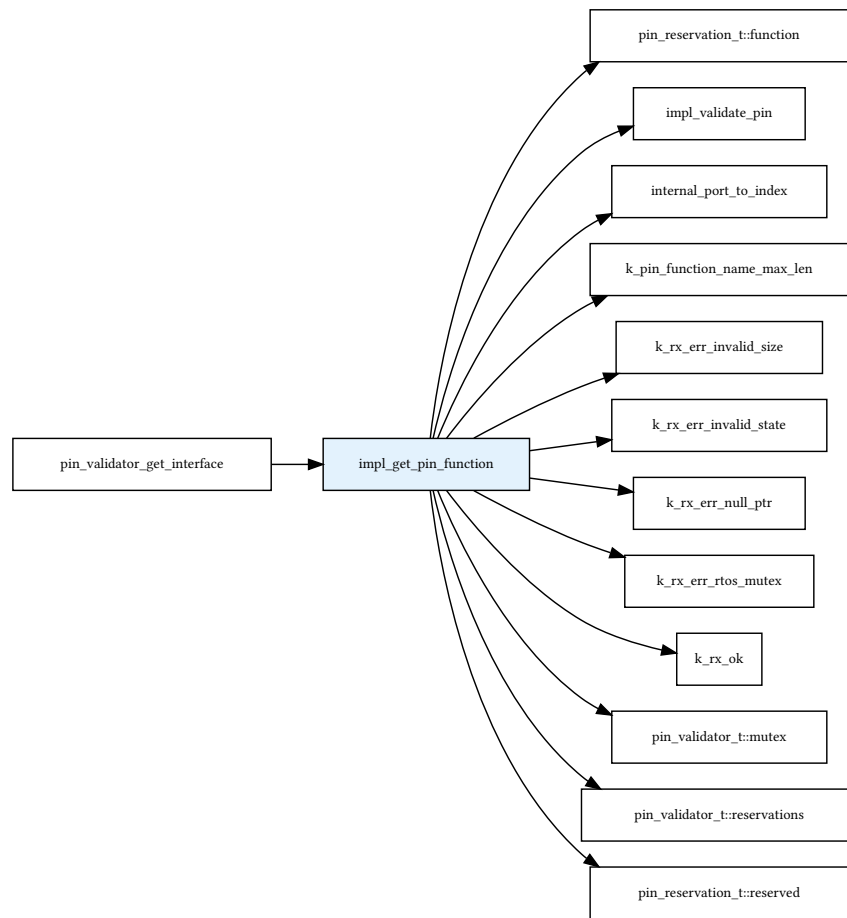
_Defined in `libs/rx_core/src/rx_pin_validator.c:687_`

impl_get_pin_function

```
static rx_err_t impl_get_pin_function(void *ctx, const uint8_t port, const
uint8_t pin, char *function_out, const uint32_t function_len)
```

Get pin function - Interface implementation for rx_pin_interface_t::get_pin_function.

Retrieves the function name that reserved a specific GPIO pin. Useful for debugging pin conflicts by identifying which module is using a contested pin.

Figure 306: Call graph for `impl_get_pin_function`

_Defined in `libs/rx_core/src/rx_pin_validator.c:787_`

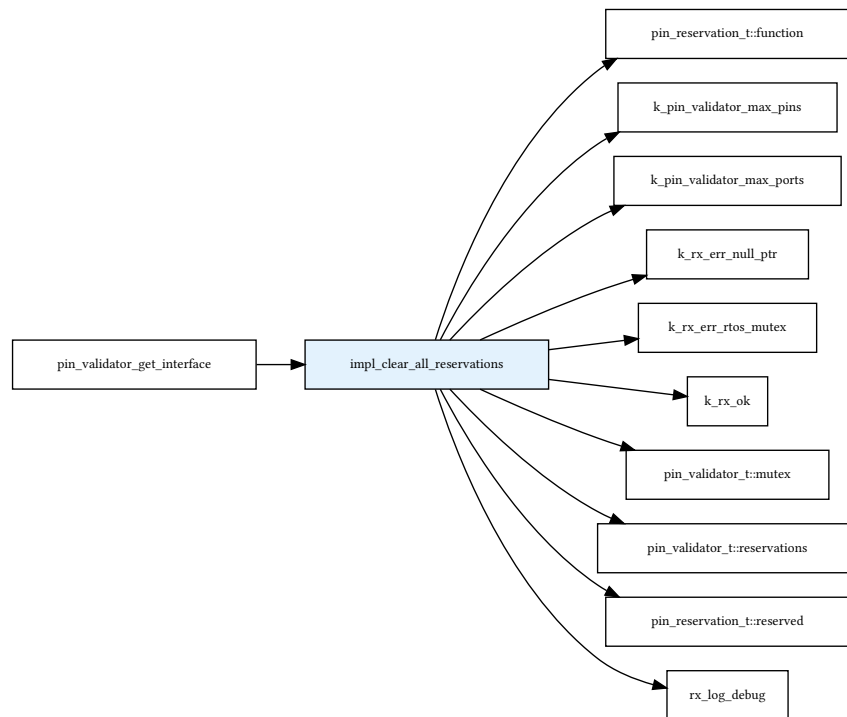
impl_clear_all_reservations

```
static rx_err_t impl_clear_all_reservations(void *ctx)
```

Clear all reservations - Interface implementation for `rx_pin_interface_t::clear_all_reservations`.

Releases ALL GPIO pin reservations at once, resetting the validator to its initial state. Primarily used for:

- Unit test teardown (clean state between tests)
- System reset/reboot preparation
- Emergency fault recovery

Figure 307: Call graph for `impl_clear_all_reservations`

_Defined in `libs/rx_core/src/rx_pin_validator.c:905_`

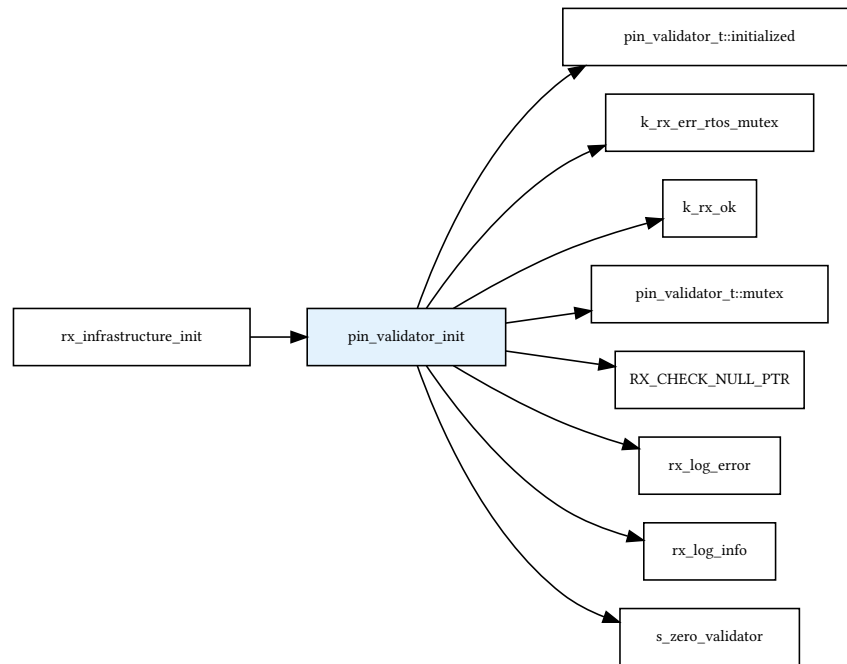
pin_validator_init

```
rx_err_t pin_validator_init(pin_validator_t *validator)
```

Initialize pin validator for tracking GPIO pin reservations.

Initialize pin validator (MUST be called before use).

Performs one-time initialization of a pin validator instance. This function creates the ThreadX mutex, clears the reservation table, and marks the validator as initialized. Must be called exactly once before any other `pin_validator_*` functions.

Figure 308: Call graph for `pin_validator_init`

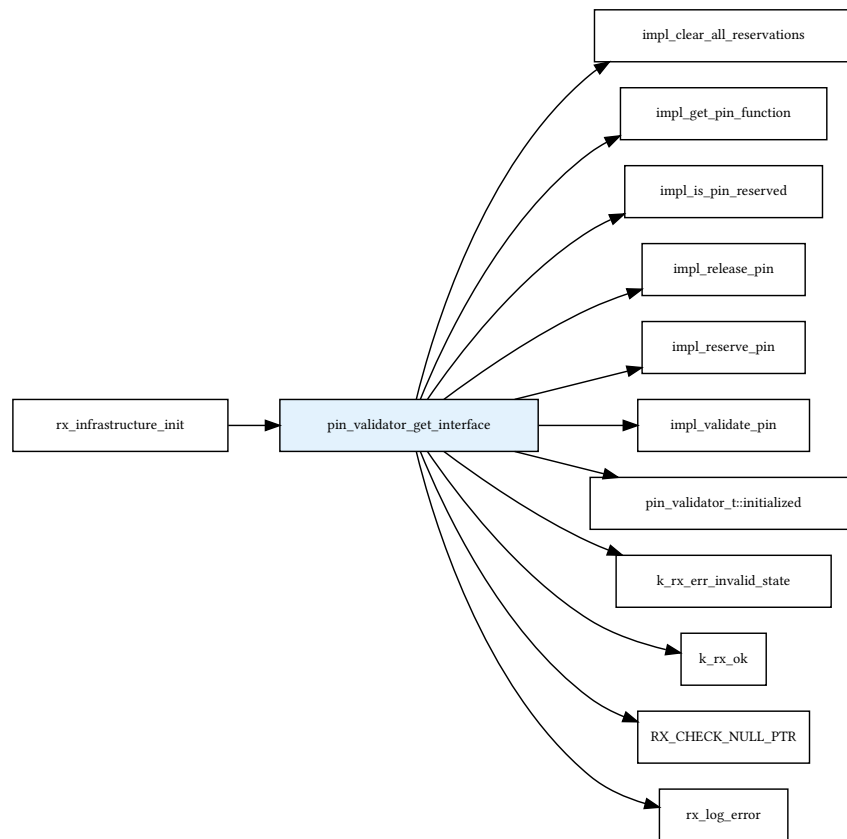
_Defined in `libs/rx\core/src/rx\_pin\_validator.c:1042_`

pin_validator_get_interface

```
rx_err_t pin_validator_get_interface(rx_pin_interface_t *iface, pin_validator_t
*validator)
```

Extract abstract interface from concrete pin validator (Dependency Inversion).

Populates an `rx_pin_interface_t` structure with function pointers to this validator's implementation functions. This is the bridge between the concrete implementation and the abstract interface that high-level modules use.

Figure 309: Call graph for `pin_validator_get_interface`

_Defined in `libs/rx\core/src/rx\_pin\_validator.c:1162_`

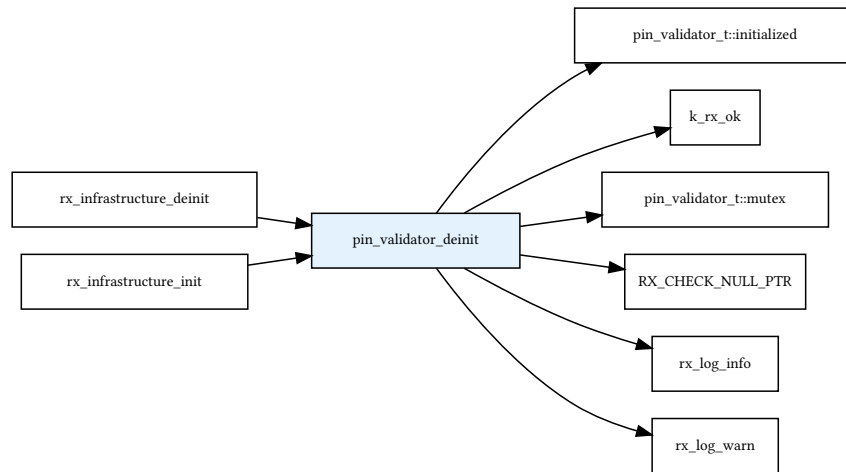
pin_validator_deinit

```
rx_err_t pin_validator_deinit(pin_validator_t *validator)
```

Deinitialize pin validator and release resources.

Deinitialize pin validator (graceful shutdown only).

Performs cleanup of a pin validator instance, destroying the ThreadX mutex and marking the validator as uninitialized. This function is rarely needed in production embedded systems (which run indefinitely) but is essential for unit test teardown.

Figure 310: Call graph for `pin_validator_deinit`

_Defined in `libs/rx\core/src/rx\pin\_validator.c:1268_`

rx_register_guard.c

Register Guard Implementation - ESD/EMI Protection for Critical Registers.

Implements the register guard module that provides ESD/EMI protection by periodically comparing critical hardware registers against known-good “golden” values and restoring them if corruption is detected. This is essential for mobile robotics operating in electromagnetically noisy environments with high-current motor drivers and PWM switching.

Includes:

- `"rx_register_guard.h"`
- `<assert.h>`
- `"rx_gpio_constants.h"`
- `"rx72n_regs.h"`

register_guard_defaults_t

Default values for register guard counters and sentinel flags.

Provides named sentinel values used during register guard initialization and state reset operations. Using typed enums instead of const variables ensures predictable size (`uint32_t`), ABI stability, and debugger visibility per project C23 style requirements.

`k_corrections_default` must remain zero to correctly reset counters

`k_guard_initialized` must remain 1 to match the initialized flag type

Value	Name	Description
= 0	<code>k_corrections_default</code>	Default correction counter value (zero, no corrections applied)
= 1	<code>k_guard_initialized</code>	Sentinel: register guard has been initialized

See also: `rx_register_guard_init()`, `rx_register_guard_reset_count()`

Example:

```
s_state.corrections=k_corrections_default;
s_state.initialized=k_guard_initialized;
```

Since Version 1.0.0

—

s_state

`register_guard_state_t` `s_state`

Static module state containing all golden values and counters.

Note: Not thread-safe. All writes should occur from the main task.

Warning: Do not access directly from ISRs. Use public API functions.] **See also:** `register_guard_state_t` Structure definition

Since Version 1.0.0

—

internal_capture_pdr

```
static void internal_capture_pdr(void)
```

Capture current PORT PDR values as golden reference.

Reads all PORT Direction Registers (PDR) for ports available on the 144-pin RX72N package and stores them in the golden reference structure. This function is called during initialization to establish the baseline configuration that will be maintained by subsequent refresh operations.

Algorithm:

1. Read PORT0.PDR through memory-mapped accessor `port0()->pdr`
2. Store value in `s_state.pdr.port0_pdr`
3. Repeat for all 12 ports currently monitored on 144-pin package

Ports captured (144-pin package):

- PORT0-PORT5 (6 ports, contiguous)
- PORTA-PORTE (5 ports, contiguous)
- PORTJ (1 port, separate)

Ports available on 144-pin but NOT yet captured:

- PORT6-PORT9 (available on 144-pin, partial pin availability)
- PORTF (available on 144-pin, partial pin availability)

Ports NOT available on 144-pin package:

- PORTG, PORTH (176-pin only)

Pre: None - safe to call anytime

Post: `s_state.pdr` contains current PDR values

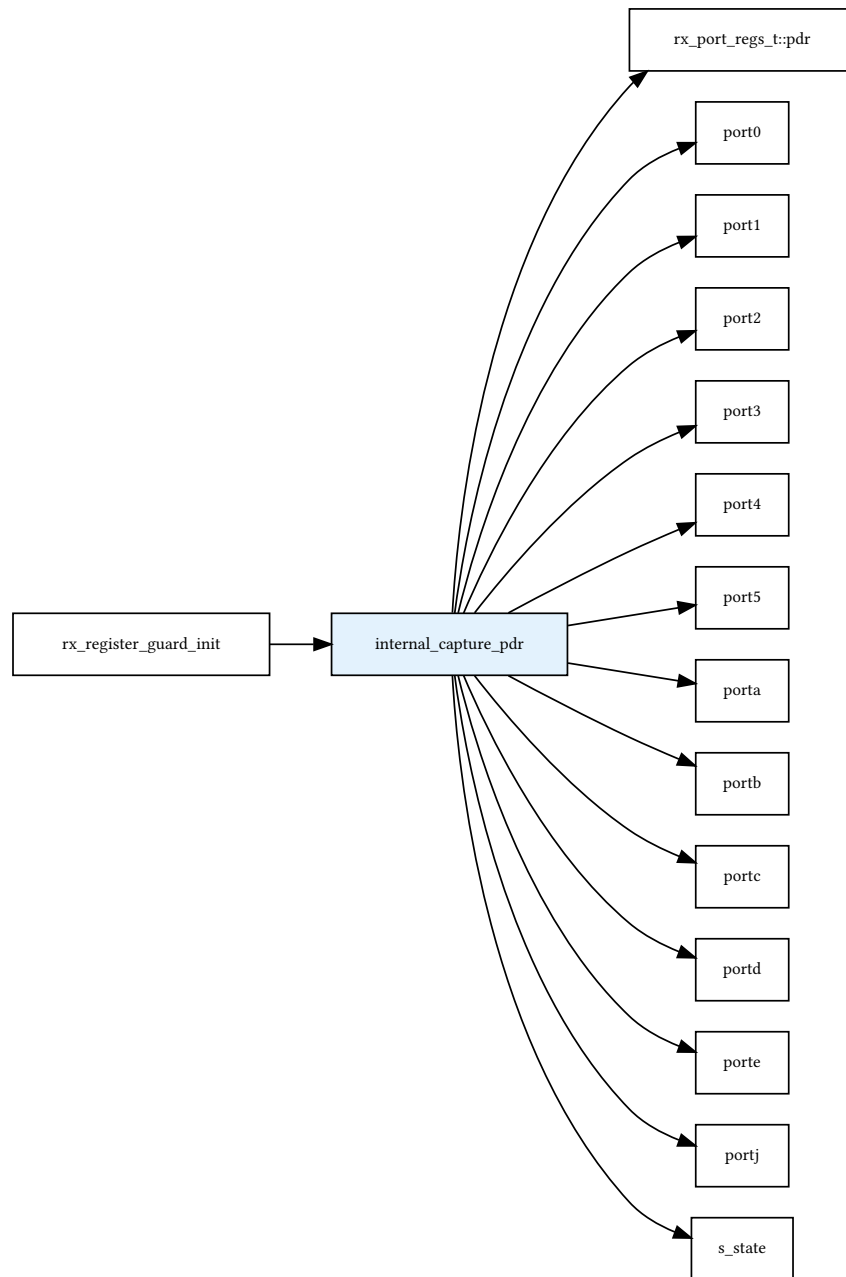
Post: No hardware state modified (read-only)

Note: Not thread-safe - call from main thread only

Note: 144-pin package: Accessing non-existent ports (PORTG, PORTH) would fault

Execution Timing:: 12 register reads x 15 cycles = 180 cycles At 240 MHz: 0.75 us

See also: `port0()` through `portj()` Register accessor functions, `pdr_golden_t` Structure storing captured values, `internal_refresh_pdr()` Uses these golden values

Figure 311: Call graph for `internal_capture_pdr`

_Defined in `libs/rx\_core/src/rx\_register\_guard.c:461_`

Since Version 1.0.0

—

internal_capture_mstpcr

```
static void internal_capture_mstpcr(void)
```

Capture current MSTPCR values as golden reference.

Reads all Module Stop Control Registers (MSTPCRA through MSTPCRD) and stores them in the golden reference structure. These registers control power to peripheral modules - corruption could disable critical systems.

Algorithm:

1. Read MSTPCRA through memory-mapped accessor system_regs()->mstpcra
2. Store value in s_state.mstpcr.mstpcra
3. Repeat for MSTPCRB, MSTPCRC, MSTPCRD

Modules controlled by each register:

Pre: None - safe to call anytime

Post: s_state.mstpcr contains current MSTPCR values

Post: No hardware state modified (read-only)

Note: MSTPCR read does NOT require PRCR unlock (write does)

Note: Not thread-safe - call from main thread only

Execution Timing:: 4 register reads x 20 cycles = 80 cycles At 240 MHz: 0.33 us

See also: system_regs() System register accessor function, mstpcr_golden_t Structure storing captured values, internal_refresh_mstpcr() Uses these golden values

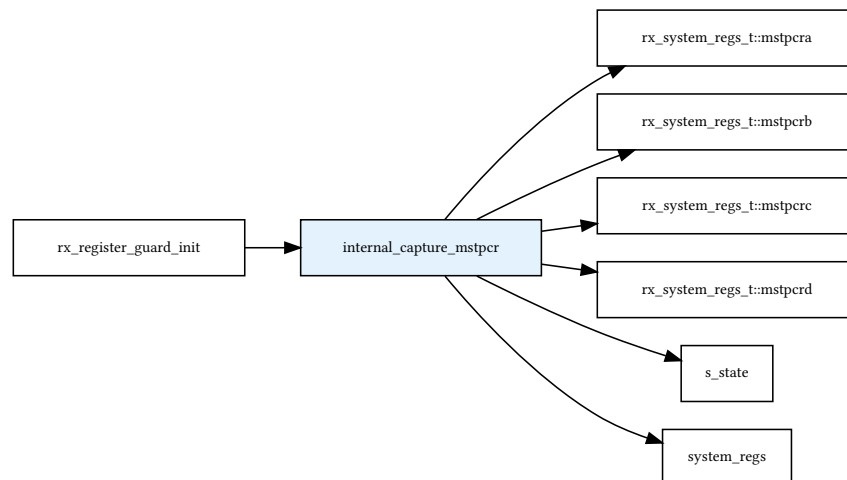


Figure 312: Call graph for internal_capture_mstpcr

_Defined in libs/rx_core/src/rx_register_guard.c:515_

Since Version 1.0.0

—

internal_refresh_pdr

```
static void internal_refresh_pdr(void)
```

Refresh PORT PDR values, counting corrections.

Compares current PORT Direction Register values against golden reference and restores any that differ. Each mismatch increments the correction counter for diagnostics and telemetry.

Algorithm for each port:

1. Read current PORTx.PDR value via memory-mapped accessor
2. Compare against golden value in s_state.pdr.portx_pdr
3. If mismatch detected: a. Write golden value back to PORTx.PDR b. Increment s_state.corrections counter
4. Proceed to next port

Pre: s_state.pdr must contain valid golden values (init() called)

Post: All PORT PDR registers match golden values

Post: s_state.corrections incremented for each mismatch found

Note: 144-pin package: Currently ports 0-5, A-E, J are checked

Note: PORT PDR does NOT require write protection unlock

Note: Not thread-safe - call from main thread only

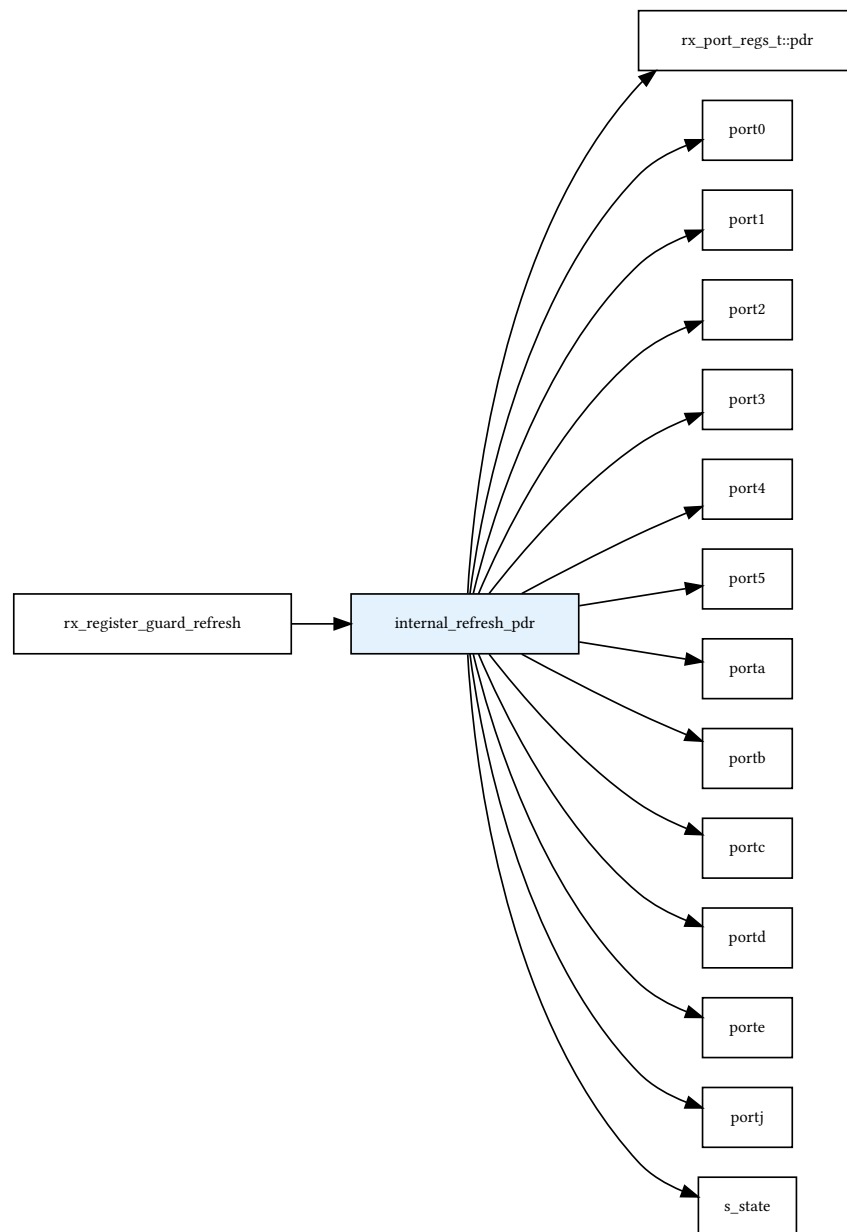
Flow Diagram:: digraph refresh_pdr_flow { rankdir=TB; node [shape=box, style=rounded];

start [label="Start"]; read [label="Read PORTx.PDR"]; compare [label="Compare with golden", shape=diamond]; restore [label="Write golden to PORTx.PDR"]; increment [label="Increment corrections"]; next [label="Next port", shape=diamond]; done [label="Done"];

start -> read; read -> compare; compare -> restore [label="mismatch"]; compare -> next [label="match"]; restore -> increment; increment -> next; next -> read [label="more ports"]; next -> done [label="all checked"]; }

Performance:: Best case (no mismatches): 12 reads, 0 writes = 1.5 us Worst case (all mismatched): 12 reads, 12 writes = 2.5 us Typical case: 12 reads, 0-1 writes = 1.6 us

See also: internal_capture_pdr() Captures golden values, rx_register_guard_refresh() Calls this function

Figure 313: Call graph for `internal_refresh_pdr`

_Defined in `libs/rx_core/src/rx_register_guard.c:582_`

Since Version 1.0.0

—

internal_refresh_mstpcr

```
static void internal_refresh_mstpcr(void)
```

Refresh MSTPCR values, counting corrections.

Compares current Module Stop Control Register values against golden reference and restores any that differ. This function requires PRCR write protection unlock and brief interrupt disable for atomic register access.

Algorithm:

1. Check if any MSTPCR register differs from golden (fast path check)
2. If no differences, return immediately (typical case)
3. If differences found: a. Save current PSW (interrupt state) b. Disable interrupts (clrpsw i) c. Unlock PRCR with PRC1 bit (enables MSTPCR writes) d. For each MSTPCR: compare and restore if different, increment counter e. Lock PRCR (disable all protected writes) f. Restore PSW (re-enable interrupts)

Pre: s_state.mstpcr must contain valid golden values (init() called)

Post: All MSTPCR registers match golden values

Post: s_state.corrections incremented for each mismatch found

Post: Interrupts restored to original state

Note: Interrupt disable is brief (200 ns) but unavoidable for atomicity

Warning: Do NOT call from ISR context - would cause nested interrupt disable

Warning: Do NOT call before all peripherals are initialized

PRCR Register Format:: Bits Name Description

15:8 PRKEY Key code (0xA5 required for write)

7:4 Reserved Read as 0

3 PRC3 Protects LVD registers

2 PRC2 Reserved

1 PRC1 Protects MSTPCR registers

0 PRC0 Protects clock registers

Unlock Sequence:: PRCR=0xA502;

MSTPCRA=golden_value;

PRCR=0xA500;

Flow Diagram:: digraph refresh_mstpcr_flow { rankdir=TB; node [shape=box, style=rounded];

```
start [label="Start"]; check [label="Any MSTPCRndiffers?", shape=diamond]; early_return
[label="Returnn(fast path)"]; save_psw [label="Save PSWn(interrupt state)"]; disable_int
[label="Disable interruptsncrlpsw i"]; unlock [label="Unlock PRCRnPRCR = 0xA502"]; restore_regs
[label="Restore changednMSTPCR registers"]; lock [label="Lock PRCRnPRCR = 0xA500"];
restore_psw [label="Restore PSW"]; done [label="Done"];
```

```
start -> check; check -> early_return [label="no"]; check -> save_psw [label="yes"]; save_psw ->
disable_int; disable_int -> unlock; unlock -> restore_regs; restore_regs -> lock; lock -> restore_psw;
restore_psw -> done; early_return -> done; }
```

Performance:: Best case (no mismatches): 4 reads, fast return = 0.5 us Worst case (all mismatched): 8 reads, 4 writes, PRCR unlock = 1.5 us Interrupt latency: 200 ns (time interrupts are disabled)

Inline Assembly (NASA Rule 9 Deviation):: Uses inline assembly for PSW manipulation because there is no C library function to access the RX Program Status Word register: mvfc psw, %0 - Move From Control register (read PSW) clrpsw i - Clear PSW I-bit (disable interrupts) mvtc %0, psw - Move To Control register (restore PSW)

NASA Power of 10 Rule 9:: Inline ASM required - no C API for PSW access. Usage is: Minimal (3 instructions) Documented Unavoidable for MSTPCR protection

See also: internal_capture_mstpcr() Captures golden values, rx_register_guard_refresh() Calls this function, k_prcr_key PRCR key code constant, k_prcr_lock_prc1 PRCR PRC1 bit mask

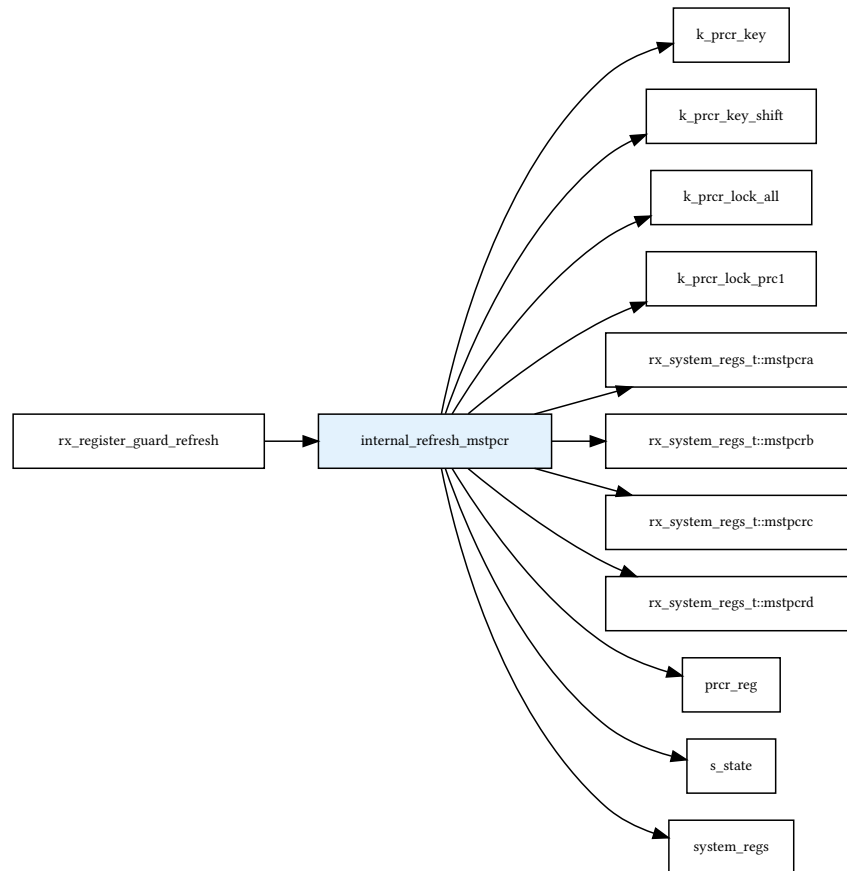


Figure 314: Call graph for internal_refresh_mstpcr

_Defined in libs/rx_core/src/rx_register_guard.c:740_

Since Version 1.0.0

—

rx_register_guard_init

```
rx_err_t rx_register_guard_init(void)
```

Initialize register guard and capture golden register values.

Initialize register guard module and capture golden reference values.

Initializes the register guard module by capturing the current values of all critical hardware registers as the “golden reference”. These values will be maintained by subsequent refresh operations. This function must be called AFTER all peripheral initialization is complete.

Algorithm:

1. Capture all PORT PDR registers via internal_capture_pdr()
2. Capture all MSTPCR registers via internal_capture_mstpcr()

3. Reset correction counter to 0
4. Set initialized flag to 1
5. Return k_rx_ok (always succeeds)

Captures CURRENT register values - ensure peripherals are correctly configured before calling or incorrect values will be preserved!

Returns: rx_err_t Error code indicating initialization status

Value	Description
k_rx_ok	Success, golden values captured and module initialized

Pre: Peripherals should be fully initialized before calling

Pre: Clock and power configuration should be stable

Post: rx_register_guard_is_initialized() returns true

Post: s_state.pdr contains current PORT PDR values

Post: s_state.mstpcr contains current MSTPCR values

Post: s_state.corrections == 0

Note: Thread Safety: Not thread-safe. Call from main thread during startup.

Note: Re-initialization: Safe to call multiple times to re-capture golden values after intentional configuration changes.

Note: Execution Time: 2.2 us @ 240 MHz

Note: Conditional Compilation: On non-RX targets (unit tests), capture functions are no-ops but module state is still updated.

Initialization Sequence:: App, Guard, PDR, MSTPCR;

```
App => Guard [label="rx_register_guard_init()"]; Guard => PDR [label="Read PORT0-PORTJ"]; PDR
>> Guard [label="PDR values"]; Guard => MSTPCR [label="Read MSTPCRA-D"]; MSTPCR >>
Guard [label="MSTPCR values"]; Guard box Guard [label="Store golden valuesnReset counternSet
initialized"]; Guard >> App [label="k_rx_ok"];
```

Example:: hardware_init(); motor_init(); comm_init();

```
rx_err_t err=rx_register_guard_init(); assert(err==k_rx_ok);
assert(rx_register_guard_is_initialized());
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 2 preconditions, 4 postconditions Rule 7: [OK] Return value always k_rx_ok (documented)

See also: rx_register_guard_refresh() Uses captured golden values, rx_register_guard_is_initialized() Check if init completed, internal_capture_pdr() PDR capture helper, internal_capture_mstpcr() MSTPCR capture helper

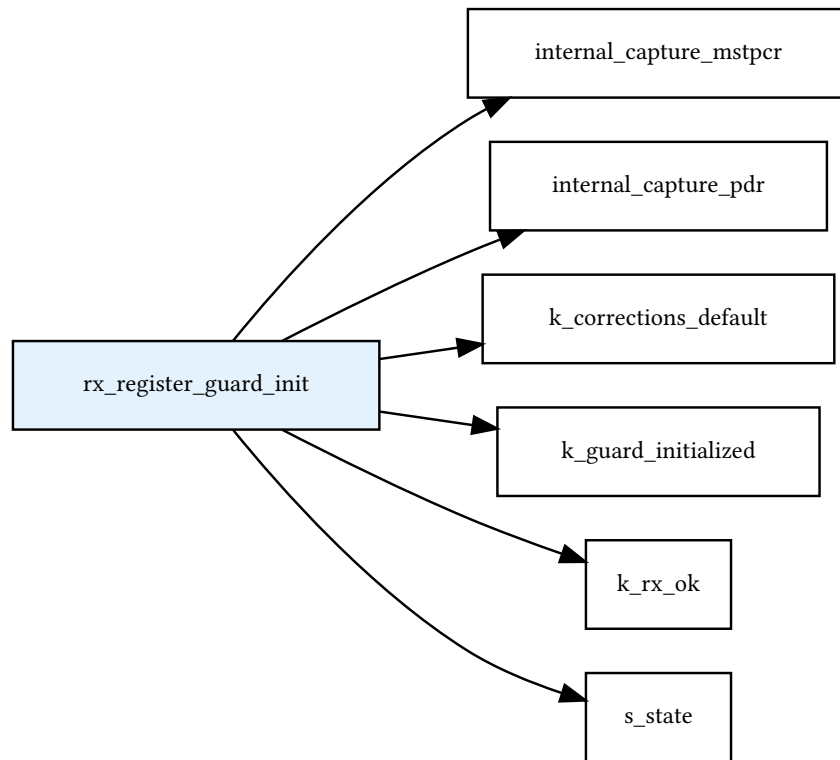


Figure 315: Call graph for rx_register_guard_init

_Defined in libs/rx_core/src/rx_register_guard.c:876_

Since Version 1.0.0

—

rx_register_guard_refresh

```
void rx_register_guard_refresh(void)
```

Refresh critical registers from golden values (ESD/EMI recovery).

Refresh all critical registers by comparing against golden reference.

Core protection function that compares all monitored hardware registers against golden reference values and restores any that have been corrupted. Designed to be called periodically from the main control loop at 1-10 Hz.

Algorithm:

1. Check initialized flag - return immediately if not initialized
2. Call `internal_refresh_pdr()` - check/restore all PORT PDR registers
3. Call `internal_refresh_mstpcr()` - check/restore all MSTPCR registers (includes brief interrupt disable for PRCR unlock)

Golden values unchanged (read-only after init)

Correction counter monotonically increasing

Returns: void (no return value)

Pre: Module should be initialized via `rx_register_guard_init()`

Pre: Should be called from main task context (not ISR)

Post: All monitored registers match golden values

Post: `s_state.corrections` incremented for each mismatch

Post: Interrupts restored to original state

Note: Thread Safety: Not thread-safe. Call from main control loop only.

Note: ISR Safety: NOT safe to call from ISRs (brief interrupt disable).

Note: Silent No-op: Returns silently if not initialized (defensive).

Note: Conditional Compilation: On non-RX targets, refresh functions are no-ops but initialization check still occurs.

Warning: Do NOT call from interrupt handlers

Warning: High correction rates (>10/sec) indicate hardware EMI problems

Execution Flow:: digraph refresh_flow { rankdir=TB; node [shape=box, style=rounded];
 start [label="rx_register_guard_refresh()"]; check_init [label="initialized?", shape=diamond];
 early_return [label="Return (no-op)"]; refresh_pdr [label="internal_refresh_pdr(n 1.5 us)"];
 refresh_mstpcr [label="internal_refresh_mstpcr(n 0.5-1.5 us)"]; done [label="Done"];
 start -> check_init; check_init -> early_return [label="no"]; check_init -> refresh_pdr [label="yes"];
 refresh_pdr -> refresh_mstpcr; refresh_mstpcr -> done; early_return -> done; }

Performance:: Scenario Time @ 240 MHz Notes

Not initialized 10 ns Early return

No corrections 4.2 us Typical case

All corrected 5.8 us Worst case

CPU overhead @ 10 Hz 0.006% Negligible

Example - Main Loop Integration:: voidcontrol_loop(void){ uint32_tcounter=0; while(1)
{ motor_update(); sensor_update();
if(++counter>=100){ rx_register_guard_refresh(); counter=0; } tx_thread_sleep(1); } }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 2
preconditions, 3 postconditions, 2 invariants Rule 9: [WARN] Brief interrupt disable in
internal_refresh_mstpcr() (documented)

See also: rx_register_guard_init() Must call first,
rx_register_guard_get_correction_count() Monitor EMI events, internal_refresh_pdr()
PORT PDR refresh helper, internal_refresh_mstpcr() MSTPCR refresh helper

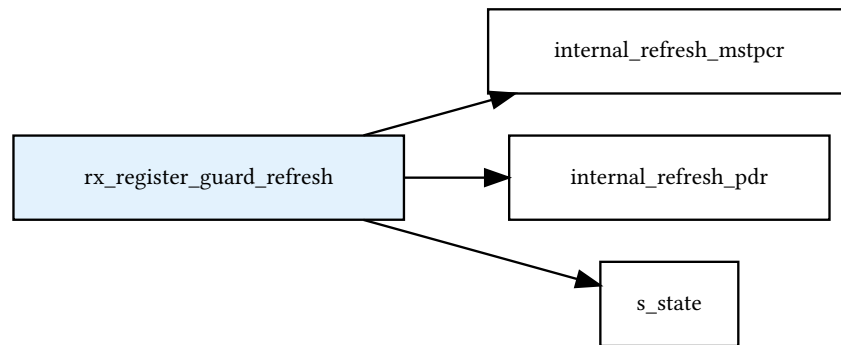


Figure 316: Call graph for rx_register_guard_refresh

_Defined in libs/rx_core/src/rx_register_guard.c:985_

Since Version 1.0.0

rx_register_guard_get_correction_count

```
uint32_t rx_register_guard_get_correction_count(void)
```

Get total number of register corrections performed.

Get count of register corrections since last reset.

Returns the cumulative count of register corrections since module initialization or the last call to rx_register_guard_reset_count(). Each individual register mismatch detected during refresh increments this counter by 1.

Non-zero counts indicate ESD/EMI events that corrupted hardware registers. This value is used for:

- Diagnostics during development
- Telemetry to ground station
- EMI alert threshold monitoring

Algorithm:

1. Check if module is initialized

2. If not initialized, return 0 (defensive)
3. Return current value of s_state.corrections

Returns: uint32_t Number of register corrections since init/reset

Value	Description
0	No corrections (ideal) OR module not initialized
\>0	Number of ESD/EMI events detected and corrected

Pre: None - safe to call anytime

Post: Counter value unchanged (read-only operation)

Note: Thread Safety: Mostly thread-safe (atomic read on 32-bit MCU). However, refresh() may increment while reading (race condition).

Note: Execution Time: 18 ns @ 240 MHz

Note: Returns 0 if not initialized (distinguishable only via is_initialized())

Example - Periodic Monitoring:: void monitor_emi(void){ static uint32_t last_count=0; uint32_t current=rx_register_guard_get_correction_count(); if(current>last_count){ uint32_t new_events=current-last_count; rx_log_warn("EMI","Detected%lu corrections",new_events); last_count=current; } }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 1 precondition, 1 postcondition

See also: rx_register_guard_refresh() Increments this counter, rx_register_guard_reset_count() Clear counter to zero

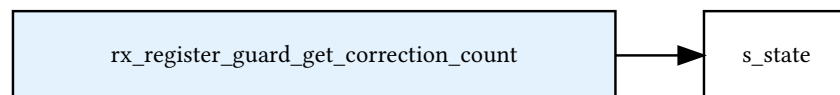


Figure 317: Call graph for rx_register_guard_get_correction_count

_Defined in libs/rx\core/src/rx_register_guard.c:1053_

Since Version 1.0.0

—

rx_register_guard_reset_count

```
void rx_register_guard_reset_count(void)
```

Reset correction counter to zero.

Reset the correction counter to zero.

Clears the register correction counter to zero. Used for periodic diagnostics where you want to measure correction rate over specific time intervals (e.g., corrections per minute).

Does not affect golden reference values or initialization status.

Algorithm:

1. Check if module is initialized
2. If not initialized, return immediately (defensive no-op)
3. Set s_state.corrections to k_corrections_default (0)
4. Assert postcondition (debug builds only)

Returns: void (no return value)

Pre: None - safe to call anytime

Post: s_state.corrections == k_corrections_default (0)

Post: Next get_correction_count() returns 0

Note: Thread Safety: Not thread-safe with concurrent refresh() calls.

Note: Execution Time: 30 ns @ 240 MHz

Note: Silent no-op if not initialized (defensive)

Example - Periodic Logging with Reset:: voidlog_emi_stats(void)
 { uint32_tcorrections=rx_register_guard_get_correction_count();
 rx_log_info("EMI","Correctionthisperiod:%lu",corrections); rx_register_guard_reset_count(); }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 1 precondition, 2 postconditions (with assert validation)

See also: rx_register_guard_get_correction_count() Read counter before reset,
 rx_register_guard_init() Also resets counter

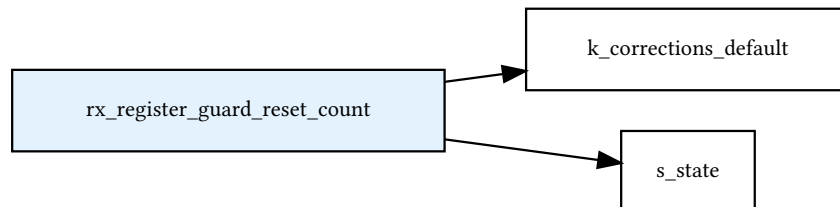


Figure 318: Call graph for rx_register_guard_reset_count

_Defined in libs/rx\core/src/rx_register_guard.c:1107_

Since Version 1.0.0

—

rx_register_guard_is_initialized

```
bool rx_register_guard_is_initialized(void)
```

Check if register guard is initialized.

Check if register guard module is initialized.

Returns the initialization status of the register guard module. Allows application code to verify that protection is active before relying on it, or to conditionally skip operations that depend on the guard.

The module is initialized after rx_register_guard_init() completes. There is no de-initialization function - once initialized, the module remains active until MCU reset.

Algorithm:

1. Read s_state.initialized flag
2. Return true if non-zero, false if zero

Returns: bool Initialization status

Value	Description
true	Module initialized, refresh() is active
false	Module not initialized, refresh() is no-op

Pre: None - safe to call anytime

Post: Module state unchanged (read-only query)

Note: Thread Safety: Thread-safe (atomic read on 32-bit MCU)

Note: ISR Safety: Safe to call from ISRs

Note: Execution Time: 15 ns @ 240 MHz

Example - Defensive Programming:: void start_control_loop(void){ if(! rx_register_guard_is_initialized()){ rx_log_error("MAIN","Registerguardnotactive!"); rx_register_guard_init(); } }

Example - Diagnostics:: void print_status(void){ printf("RegisterGuard:%sn", rx_register_guard_is_initialized()? "ACTIVE": "INACTIVE"); }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto, setjmp, recursion Rule 5: [OK] 1 precondition, 1 postcondition

See also: rx_register_guard_init() Sets initialized flag to true

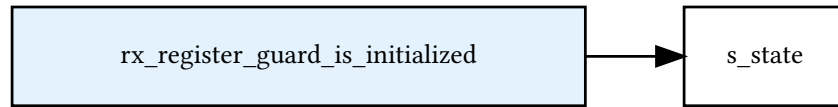


Figure 319: Call graph for rx_register_guard_is_initialized

_Defined in libs/rx_core/src/rx_register_guard.c:1174_

Since Version 1.0.0

—

rx_time_threadx.c

ThreadX Time Implementation for RX72N.

Real implementation of rx_time_interface_t using ThreadX RTOS primitives. This file provides hardware-independent time services by leveraging the ThreadX kernel tick infrastructure (CMT0 timer).

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_check.h"
- "rx_time_constants.h"
- "rx_time_interface.h"
- "tx_api.h"

rx_time_threadx_zero_t

Zero sentinel for ThreadX time computations.

Used in ceiling-division and guard checks to avoid magic number 0.

k_rx_zero equals zero; used as a guard against zero-division in tick ceiling calculation

Value	Name	Description
= 0u	k_rx_zero	Zero sentinel used in overflow-safe ceiling division

See also: rx_time_threadx_ms_to_ticks()

Example:

```
//Guard:onlysleepifticks>0
if(ticks>k_rx_zero){
(void)tx_thread_sleep(ticks);
}
```

Since Version 1.0.0

—

rx_time_ceil_offset_t

Ceiling-division rounding offset.

Added to the remainder check when computing tick ceiling from milliseconds.

k_ceil_div_offset equals 1; adding this before integer division implements ceiling division

Value	Name	Description
= 1u	k_ceil_div_offset	Standard ceiling-division rounding offset

See also: rx_time_threadx_ms_to_ticks()

Example:

```
//Ceilingdivision:ticks=ceil(ms/k_threadx_ms_per_tick)
constuint32_tquotient=ms/k_threadx_ms_per_tick;
constuint32_tremainder=ms%k_threadx_ms_per_tick;
constuint32_tticks=quotient+(remainder>k_rx_zero?k_ceil_div_offset:k_rx_zero);
```

Since Version 1.0.0

—

impl_sleep_ms

```
static void impl_sleep_ms(void *ctx, uint32_t ms)
```

Sleep for specified milliseconds using ThreadX scheduler.

Yields CPU to other threads via tx_thread_sleep(), allowing the ThreadX scheduler to run higher-priority threads while this thread waits.

Parameters:

Direction	Name	Description
in	ctx	Unused context pointer (ThreadX uses global state) Must be NULL for this implementation
in	ms	Sleep duration in milliseconds Valid range: [0, UINT32_MAX] 0: No-op (returns immediately) 1-10: Sleeps 1 tick (10ms actual) 11-20: Sleeps 2 ticks (20ms actual)

Pre: Calling thread must be a valid ThreadX thread

Pre: ThreadX kernel must be running (tx_kernel_enter() called)

Post: Thread has slept for at least ms milliseconds (rounded up to tick)

Post: Other threads have had opportunity to run

Note: Implemented via overflow-safe quotient/remainder to avoid wrap at high ms values.

Note: Thread Safety: Safe - tx_thread_sleep() is thread-safe

Note: Cannot be called from ISR context

Warning: Actual sleep time is rounded up to nearest 10ms tick boundary

Warning: Sleep may be longer than requested due to higher-priority threads

Algorithm Steps:: Convert milliseconds to ThreadX ticks with ceiling division If ticks > 0, call tx_thread_sleep() to yield CPU Thread is rescheduled after tick count expires

Tick Conversion Formula:: ticks = ceil(ms / 10)

floor(ms / 10) + (1 if ms mod 10 > 0, else 0)

Example:: impl_sleep_ms(NULL,50);

impl_sleep_ms(NULL,1);

See also: tx_thread_sleep() ThreadX sleep function, k_threadx_ms_per_tick Tick period constant (10ms)

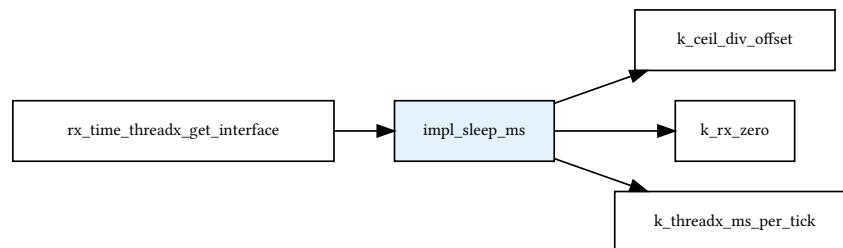


Figure 320: Call graph for impl_sleep_ms

_Defined in libs/rx\core/src/rx_time_threadx.c:226_

Since Version 1.0.0

—

impl_get_ms

```
static uint32_t impl_get_ms(void *ctx)
```

Get current time in milliseconds from ThreadX tick counter.

Reads the ThreadX system tick counter and converts to milliseconds. The tick counter starts at 0 when tx_kernel_enter() is called and increments at 100 Hz (every 10ms) via CMT0 timer interrupt.

Parameters:

Direction	Name	Description
in	ctx	Unused context pointer (ThreadX uses global state) Must be NULL for this implementation

Returns: Current time in milliseconds since kernel start

Value	Description
0	Immediately after kernel start

UINT32_MAX	* 10 Wraps at 49.7 days
------------	-------------------------

Pre: ThreadX kernel must be running

Post: Return value represents time in 10ms increments

Note: Thread Safety: Safe - tx_time_get() is atomic

Note: Can be called from ISR context

Note: Resolution is 10ms (cannot distinguish times < 10ms apart)

Algorithm Steps:: Read ThreadX tick counter via tx_time_get() Multiply by k_threadx_ms_per_tick (10ms) to get milliseconds Return result (may wrap at 49.7 days)

Conversion Formula:: ms = ticks x 10

Wraparound Handling:: Time wraps after 49.7 days: 2^{32} ticks x 10ms = 49,710,269,491 ms Use impl_is_elapsed() for correct timeout handling across wraparound.

Example:: uint32_tstart=impl_get_ms(NULL); do_operation();
uint32_telapsed=impl_get_ms(NULL)-start; rx_log_debug("TIMING","Operationtook%lums",elapsed);

See also: tx_time_get() ThreadX tick counter, impl_is_elapsed() Wraparound-safe timeout check

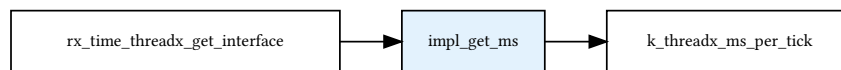


Figure 321: Call graph for impl_get_ms

_Defined in libs/rx\core/src/rx_time_threadx.c:290_

Since Version 1.0.0

impl_is_elapsed

```
static bool impl_is_elapsed(void *ctx, uint32_t start_ms, uint32_t timeout_ms)
```

Check if timeout has elapsed since start time.

Determines if the specified timeout has elapsed since start_ms. Uses unsigned subtraction which handles 32-bit wraparound correctly due to modular arithmetic properties.

Parameters:

Direction	Name	Description
in	ctx	Unused context pointer (ThreadX uses global state)

in	start_ms	Start time from get_ms(), in milliseconds Must be from a previous get_ms() call
in	timeout_ms	Timeout duration in milliseconds Valid range: [0, UINT32_MAX] 0: Always returns true

Returns: true if timeout has elapsed, false otherwise

Value	Description
true	(now - start_ms) >= timeout_ms
false	(now - start_ms) < timeout_ms

Pre: start_ms must be from a previous get_ms() call

Pre: Elapsed time since start_ms < 24.8 days (half wraparound period)

Post: No side effects

Note: Thread Safety: Safe - only reads atomic tick counter

Note: Can be called from ISR context

Warning: For correct wraparound handling, timeout_ms should be < 24.8 days

Algorithm Steps:: Get current time via tx_time_get() x 10 Calculate elapsed = now - start_ms (unsigned, handles wrap) Return true if elapsed >= timeout_ms

Wraparound Correctness:: Using unsigned 32-bit arithmetic, the subtraction now - start_ms yields the correct elapsed time even when now has wrapped past zero, as long as the actual elapsed time is less than $2^{32} / 100 / 2$ 24.8 days.

Example - Polling with Timeout:: rx_time_interface_time_if;
 rx_time_threadx_get_interface(&time_if);
 uint32_tstart=time_if.get_ms(time_if.ctx); constuint32_ttimeout_ms=1000;
 while(!device_ready()){ if(time_if.is_elapsed(time_if.ctx,start,timeout_ms))
 { rx_log_error("DEV","Timeoutwaitingfordevice"); returnk_rx_err_timeout; }
 time_if.sleep_ms(time_if.ctx,10); }

Example - Wraparound Scenario::

See also: impl_get_ms() Get start time, impl_sleep_ms() Delay between polls

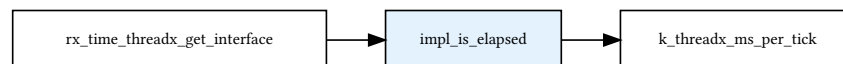


Figure 322: Call graph for impl_is_elapsed

Defined in libs/rx\core/src/rx\time\threadx.c:370

Since Version 1.0.0

—

rx_time_threadx_get_interface

```
rx_err_t rx_time_threadx_get_interface(rx_time_interface_t *iface)
```

Get time interface structure for ThreadX RTOS.

Validate that a time interface is properly initialized.

Populates an rx_time_interface_t structure with function pointers to the ThreadX implementation. This enables Dependency Inversion: callers depend on the abstract interface, not the concrete ThreadX API.

Interface functions remain valid for program lifetime

Parameters:

Direction	Name	Description
out	iface	Pointer to interface structure to populate Must not be nullptr Caller owns memory Structure is fully initialized on success

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, interface populated
k_rx_err_null_ptr	iface is nullptr

Pre: iface != nullptr

Post: iface->ctx == nullptr

Post: iface->sleep_ms, get_ms, is_elapsed are valid function pointers

Post: Interface is ready for use

Note: Thread Safety: Safe - only writes to caller-owned structure

Note: Re-entrancy: Safe - no shared state accessed

Algorithm Steps:: Validate iface is not nullptr Set ctx to NULL (ThreadX uses global state) Set function pointers to implementation functions Return success

Interface Population:: Field Value Description

ctx NULL ThreadX uses global state

sleep_ms impl_sleep_ms tx_thread_sleep() wrapper

get_ms impl_get_ms tx_time_get() x 10 wrapper

is_elapsed impl_is_elapsed Wraparound-safe timeout check

Example - Basic Usage:: rx_time_interface_ttime_if;
rx_err_t err=rx_time_threadx_get_interface(&time_if); if(err!=k_rx_ok)
{ rx_log_error("TIME","Failedtogetinterface"); returnerr; }

uint32_tstart=time_if.get_ms(time_if.ctx); do_some_work();
uint32_tduration=time_if.get_ms(time_if.ctx)-start;

Example - Timeout Loop:: rx_time_interface_ttime_if; rx_time_threadx_get_interface(&time_if);
uint32_tstart=time_if.get_ms(time_if.ctx); while(!sensor_data_ready())
{ if(time_if.is_elapsed(time_if.ctx,start,500)){ returnk_rx_err_timeout; }
time_if.sleep_ms(time_if.ctx,10); }

Example - Dependency Injection:: rx_time_interface_ttime_if;
rx_time_threadx_get_interface(&time_if); motor_init(&motor,&config,&time_if);
rx_time_interface_ttime_if; mock_time_get_interface(&time_if);
motor_init(&motor,&config,&time_if);

NASA Power of 10 Compliance:: Rule 5: [OK] 1 precondition (NULL check), 3 postconditions Rule
9: [OK] Function pointers for DIP (intentional deviation)

See also: rx_time_interface_t Interface structure definition, mock_time_get_interface()
Unit test mock version, impl_sleep_ms() Sleep implementation details, impl_get_ms()
Time retrieval details, impl_is_elapsed() Timeout check details

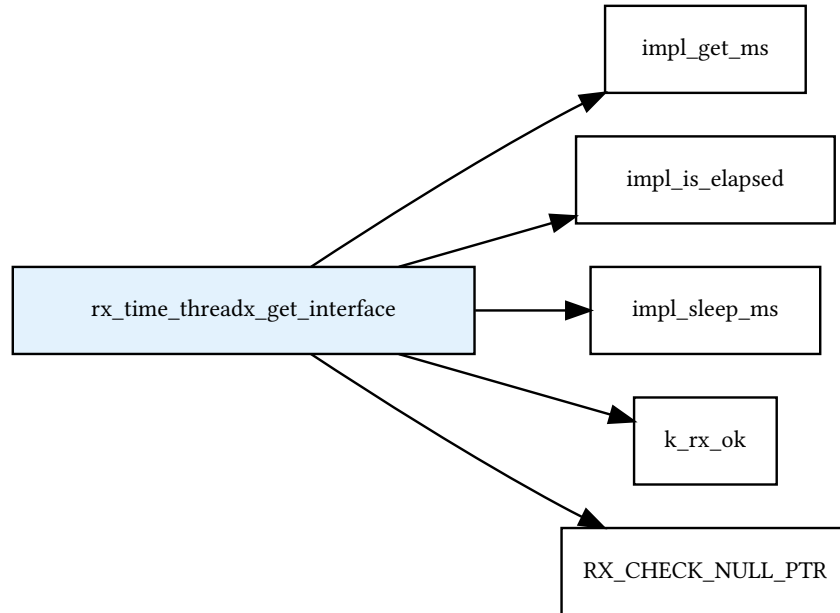


Figure 323: Call graph for rx_time_threadx_get_interface

_Defined in libs/rx\core/src/rx_time_threadx.c:484_

Since Version 1.0.0

—

rx_wdt.c

Watchdog Timer (WDT) Implementation for RX72N Microcontroller.

Implements the software-controllable Watchdog Timer (WDT) driver for the RX72N microcontroller. Provides runtime start/stop control for development flexibility, complementing the hardware-independent IWDT for defense-in-depth.

Includes:

- "rx_wdt.h"
- <string.h>
- "rx72n_system_regs.h"
- "rx72n_wdt_regs.h"

s_wdt_state

rx_wdt_state_t s_wdt_state

Global WDT driver state.

Note: Not thread-safe for concurrent writes. Use external synchronization if calling start/stop from multiple threads.] **See also:** rx_wdt_state_t Structure definition

Since Version 1.0.0

—

rx_wdt_init

```
rx_err_t rx_wdt_init(const rx_wdt_config_t *config)
```

Initialize the Watchdog Timer with specified or default configuration.

Initialize the Watchdog Timer (WDT) with specified configuration.

Configures WDT driver state and optionally starts the hardware counter. If config is nullptr, uses safe defaults (longest timeout, manual start, reset mode).

Algorithm:

1. Check if already initialized (return error if so)
2. If config is nullptr, create default configuration:

timeout_period = k_wdt_timeout_4096_cycles (68us at 60MHz) enable_on_init = false (manual start) reset_on_timeout = true (reset on timeout)

3. Copy configuration to static state
4. Set initialized flag to true
5. Set running flag to false
6. If enable_on_init, call rx_wdt_start()
7. Return k_rx_ok (or propagate start error)

Parameters:

Direction	Name	Description
-----------	------	-------------

in	config	Configuration structure pointer, or nullptr for defaults
----	--------	--

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, WDT initialized
k_rx_err_invalid_state	Already initialized

Pre: WDT must not already be initialized

Post: s_wdt_state.initialized == true

Post: s_wdt_state.config contains configuration

Post: If enable_on_init, WDT counter is running

Note: nullptr config uses safe defaults (recommended for most cases)

Note: Re-initialization requires system reset (no deinit function)

Flow Diagram:: digraph init_flow { rankdir=TB; node [shape=box, style=rounded];
 start [label="rx_wdt_init(config)"]; check_init [label="Already initialized?", shape=diamond];
 err_state [label="Return k_rx_err_invalid_state", fillcolor=lightcoral, style="rounded,filled"];
 check_null [label="config == nullptr?", shape=diamond]; use_default [label="Create default config"];
 copy_config [label="memcpy config to state"]; set_flags [label="initialized=true,running=false"];
 check_auto [label="enable_on_init?", shape=diamond]; call_start [label="rx_wdt_start()"]; success
 [label="Return k_rx_ok", fillcolor=lightgreen, style="rounded,filled"];
 start -> check_init; check_init -> err_state [label="yes"]; check_init -> check_null [label="no"];
 check_null -> use_default [label="yes"]; check_null -> copy_config [label="no"]; use_default ->
 copy_config; copy_config -> set_flags; set_flags -> check_auto; check_auto -> call_start
 [label="yes"]; check_auto -> success [label="no"]; call_start -> success; }

See also: rx_wdt_start() Called automatically if enable_on_init=true

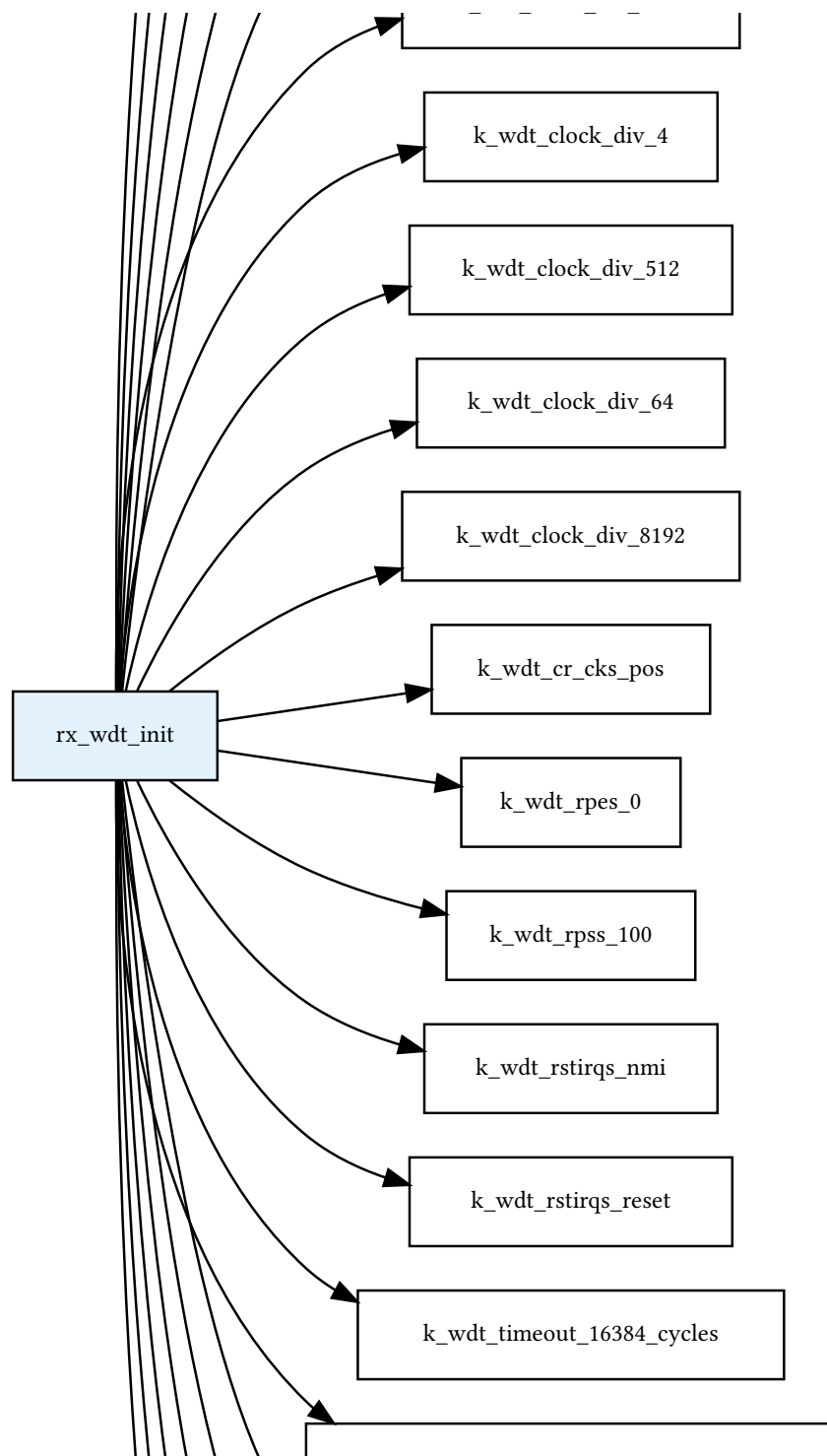


Figure 324: Call graph for rx_wdt_init

Defined in `libs/rx_core/src/rx_wdt.c:324`

Since Version 1.0.0

—

rx_wdt_start

```
rx_err_t rx_wdt_start(void)
```

Start the watchdog timer (register-start mode only).

Start the WDT counter (enable watchdog monitoring).

Starts the WDT counter by writing the refresh sequence to WDTRR. Once started, `rx_wdt_feed()` must be called periodically to prevent timeout.

Algorithm:

1. Check if WDT is initialized (return error if not)
2. Get pointer to WDT registers via `wdt()` accessor
3. Write start sequence to WDTRR (0x00 then 0xFF)
4. Set `s_wdt_state.running` to true
5. Return `k_rx_ok`

Hardware Operation: The WDTRR refresh/start sequence:

- First write: 0x00 (`k_wdt_refresh_start`)
- Second write: 0xFF (`k_wdt_refresh_end`)
- Sequence resets counter to timeout period value
- Counter begins decrementing immediately

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, WDT counter started
<code>k_rx_err_not_initialized</code>	WDT not initialized

Pre: WDT must be initialized via `rx_wdt_init()`

Post: WDT counter is running and decrementing

Post: `s_wdt_state.running == true`

Post: `rx_wdt_feed()` must be called within timeout period

Note: In auto-start mode, this function acts as a feed operation

Warning: Missing feeds after start will cause timeout/reset

OFS Register Configuration:: The WDT can be configured at flash-time via OFS (Option Function Select) registers. This function only works if WDT is in “register-start mode”: Register-start mode:

WDT starts when WDTRR sequence written Auto-start mode: WDT starts at reset, this function refreshes only

See also: `rx_wdt_init()` Must be called first, `rx_wdt_feed()` Call periodically after start, `k_wdt_refresh_start`, `k_wdt_refresh_end` Refresh sequence constants

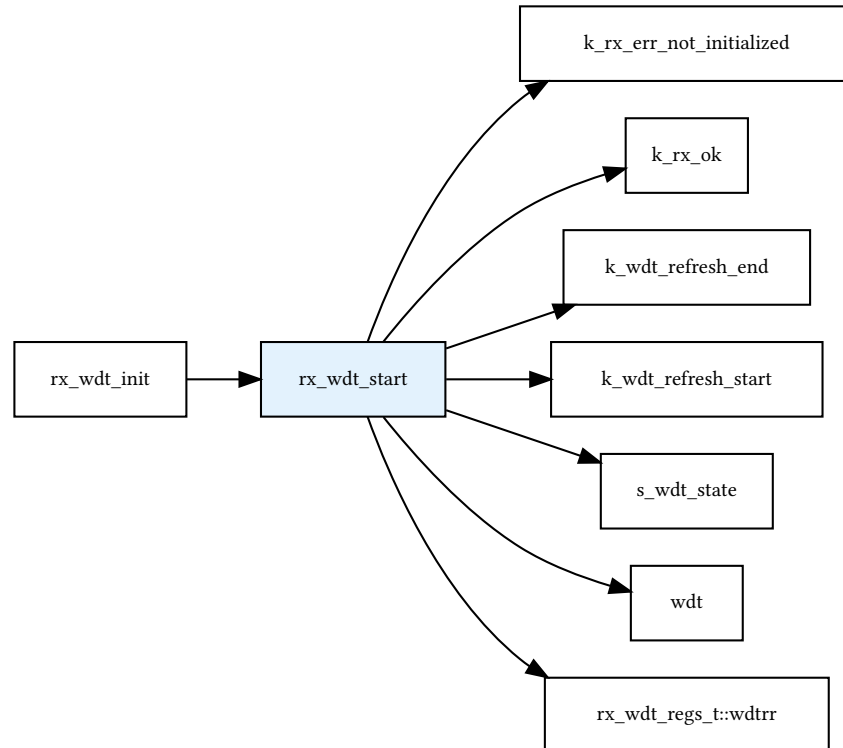


Figure 325: Call graph for `rx_wdt_start`

Defined in `libs/rx_core/src/rx_wdt.c:437`

Since Version 1.0.0

—

rx_wdt_stop

```
rx_err_t rx_wdt_stop(void)
```

Stop the watchdog timer (software state only).

Stop the WDT counter (disable watchdog monitoring).

Updates software state to indicate WDT is stopped. Note that this does NOT actually stop the hardware counter if WDT is configured in auto-start mode via OFS registers.

Algorithm:

1. Check if WDT is initialized (return error if not)
2. Set `s_wdt_state.running` to false

3. Return `k_rx_ok`

Hardware Limitation: The RX72N WDT hardware behavior depends on OFS (Option Function Select) register configuration at flash time:

- Register-start mode: Hardware can be stopped
- Auto-start mode: Hardware cannot be stopped once running!

This function **ONLY** updates software state. If using auto-start mode, you must continue calling `rx_wdt_feed()` even after `rx_wdt_stop()`.

Check OFS configuration to know if hardware can actually stop

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, software state updated
<code>k_rx_err_not_initialized</code>	WDT not initialized

Pre: WDT must be initialized via `rx_wdt_init()`

Post: `s_wdt_state.running == false`

Post: In auto-start mode, hardware still requires feeding!

Warning: In auto-start mode, this does NOT stop hardware counter

Warning: System may still reset if feeds are stopped] **See also:** `rx_wdt_start()` Restart WDT after stopping

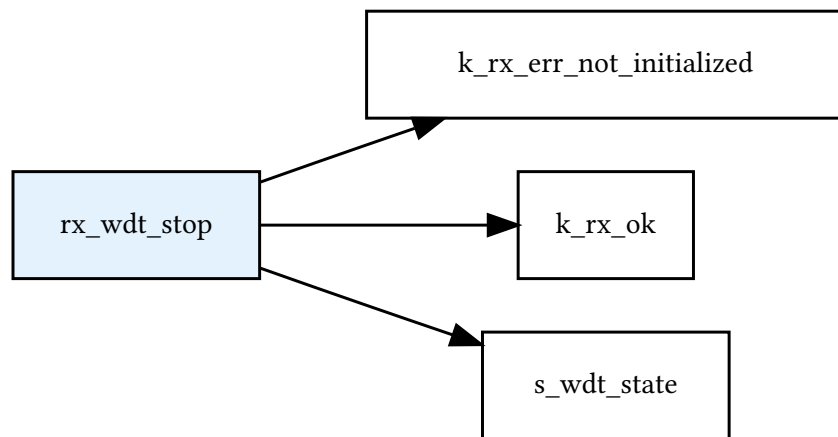


Figure 326: Call graph for `rx_wdt_stop`

Defined in `libs/rx_core/src/rx_wdt.c:493`

Since Version 1.0.0

rx_wdt_feed

```
rx_err_t rx_wdt_feed(void)
```

Feed the watchdog timer to prevent timeout.

Feed the watchdog timer (reset counter to prevent timeout).

Resets the WDT counter to its initial value by writing the refresh sequence to WDTRR. This must be called periodically at a rate faster than the configured timeout period.

Algorithm:

1. Check if WDT is initialized (return error if not)
2. Check if WDT is running (return error if not)
3. Get pointer to WDT registers via wdt() accessor
4. Write refresh sequence: 0x00 then 0xFF to WDTRR
5. Return k_rx_ok

Hardware Operation: The WDTRR refresh sequence must be written in exact order:

- First write: 0x00 (k_wdt_refresh_start)
- Second write: 0xFF (k_wdt_refresh_end)

Invalid sequences (wrong order, wrong values) are ignored by hardware. Valid sequence resets counter to timeout period value.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, WDT counter reset
k_rx_err_not_initialized	WDT not initialized
k_rx_err_invalid_state	WDT not running (stopped or never started)

Pre: WDT must be initialized via rx_wdt_init()

Pre: WDT must be running (rx_wdt_start() called or enable_on_init=true)

Post: WDT counter reset to timeout period value

Post: Countdown restarts from reset value

Note: This is the ONLY function that prevents watchdog timeout

Note: Execution time: 1 us (two register writes)

Warning: Missing a single feed within timeout period causes reset

Timing Requirements:: Feed must occur faster than timeout period: Timeout Setting Cycles At 60MHz Required Feed Rate

k_wdt_timeout_256_cycles 256 4.3 us >233 kHz

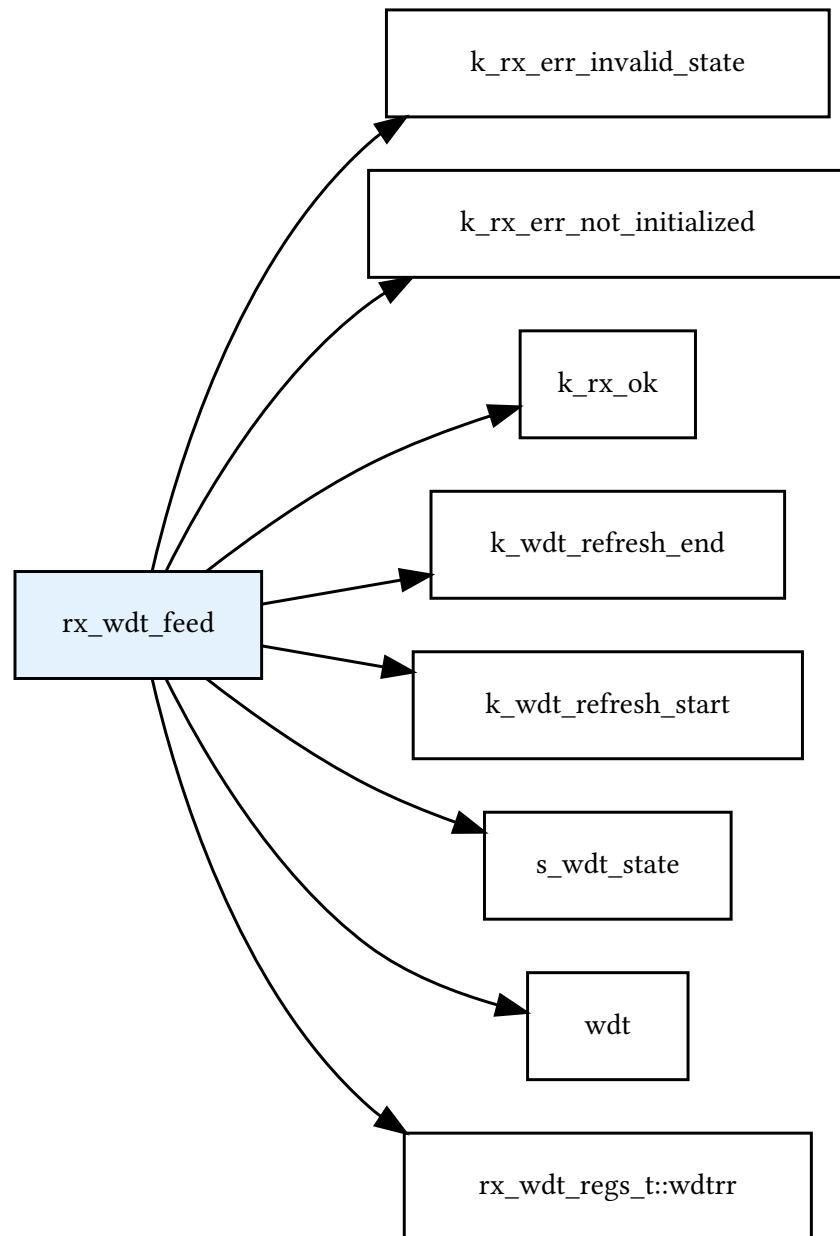
k_wdt_timeout_1024_cycles 1024 17 us >59 kHz

k_wdt_timeout_4096_cycles 4096 68 us >14.7 kHz

k_wdt_timeout_16384_cycles 16384 273 us >3.6 kHz

Example:: while(1){ do_work(); rx_err_t err=rx_wdt_feed(); if(err!=k_rx_ok)
{ handle_wdt_error(err); } delay_us(100); }

See also: rx_wdt_start() Must be called before feed,wdt_timeout_period_t Timeout period settings

Figure 327: Call graph for `rx_wdt_feed`

Defined in `libs/rx_core/src/rx_wdt.c:573`

Since Version 1.0.0

—

rx_crc.h

Cyclic Redundancy Check (CRC) API for Data Integrity Validation.

Includes:

- <stdint.h>
- "rx_err.h"

rx_crc_init

```
rx_err_t rx_crc_init(void)
```

Initialize CRC backend (hardware peripheral if available).

Initializes the CRC subsystem, enabling hardware CRC peripheral on RX72N targets for accelerated CRC-32 computation. On host builds (x86/ARM), this is a no-op since only software CRC is available.

Platform Behavior:

- RX72N (RX defined, RX_CRC32_USE_HARDWARE defined):

Enables CRCA (CRC Calculator) peripheral via module stop control Configures CRC polynomial to 0x04C11DB7 (IEEE 802.3) Sets LSB-first bit order (reflected) Initializes registers for 32-bit CRC computation Returns k_rx_ok if hardware available and configured

- Host builds (Linux, macOS, Windows for testing):

No-op, returns k_rx_ok immediately Software CRC tables already initialized statically

Algorithm (RX72N Hardware Init):

1. Validate not already initialized (prevent double-init)
2. Enable CRCA module clock via MSTPCRB register (bit 23 = 0)
3. Reset CRC peripheral by writing control register
4. Configure polynomial to 0x04C11DB7 (IEEE 802.3 standard)
5. Set bit order to LSB-first (reflected, compatible with software CRC)
6. Set initial value to 0xFFFFFFFF (IEEE 802.3 standard)
7. Mark initialized (set internal flag)

Initialize CRC backend (hardware peripheral if available).

Enables the hardware CRC module by clearing the module stop bit (MSTPCRB[23]) and configures the peripheral for IEEE 802.3 CRC-32 polynomial with LSB-first bit reflection (compatible with `crc32.ChecksumIEEE()` in Go).

This function is idempotent - calling it multiple times is safe and returns success immediately if already initialized.

Returns: rx_err_t Initialization result

Value	Description
k_rx_ok	Initialization successful (hardware or no-op)
k_rx_err_invalid_state	Already initialized (RX72N only)
k_rx_err_hardware	Hardware CRC peripheral fault (RX72N only, rare)

Pre: System clock (PCLKA) must be configured and running

Pre: Module stop registers must be accessible

Post: On RX72N, CRCA peripheral ready for rx_crc32_ieee() hardware acceleration

Post: On host, software CRC tables ready for use

Post: rx_crc32_ieee() will use fastest available implementation

Note: Call this ONCE during system initialization before any CRC operations

Note: Thread-safe ONLY if called single-threaded during init

Warning: Do not call from interrupt context

Warning: On RX72N, failure to call this results in slower software CRC fallback

Performance Impact:: With init (HW CRC): 32 us per 100 bytes (4x faster) Without init (SW CRC): 130 us per 100 bytes

Example - System Initialization:: voidsystem_init(void) { rx_clock_init();

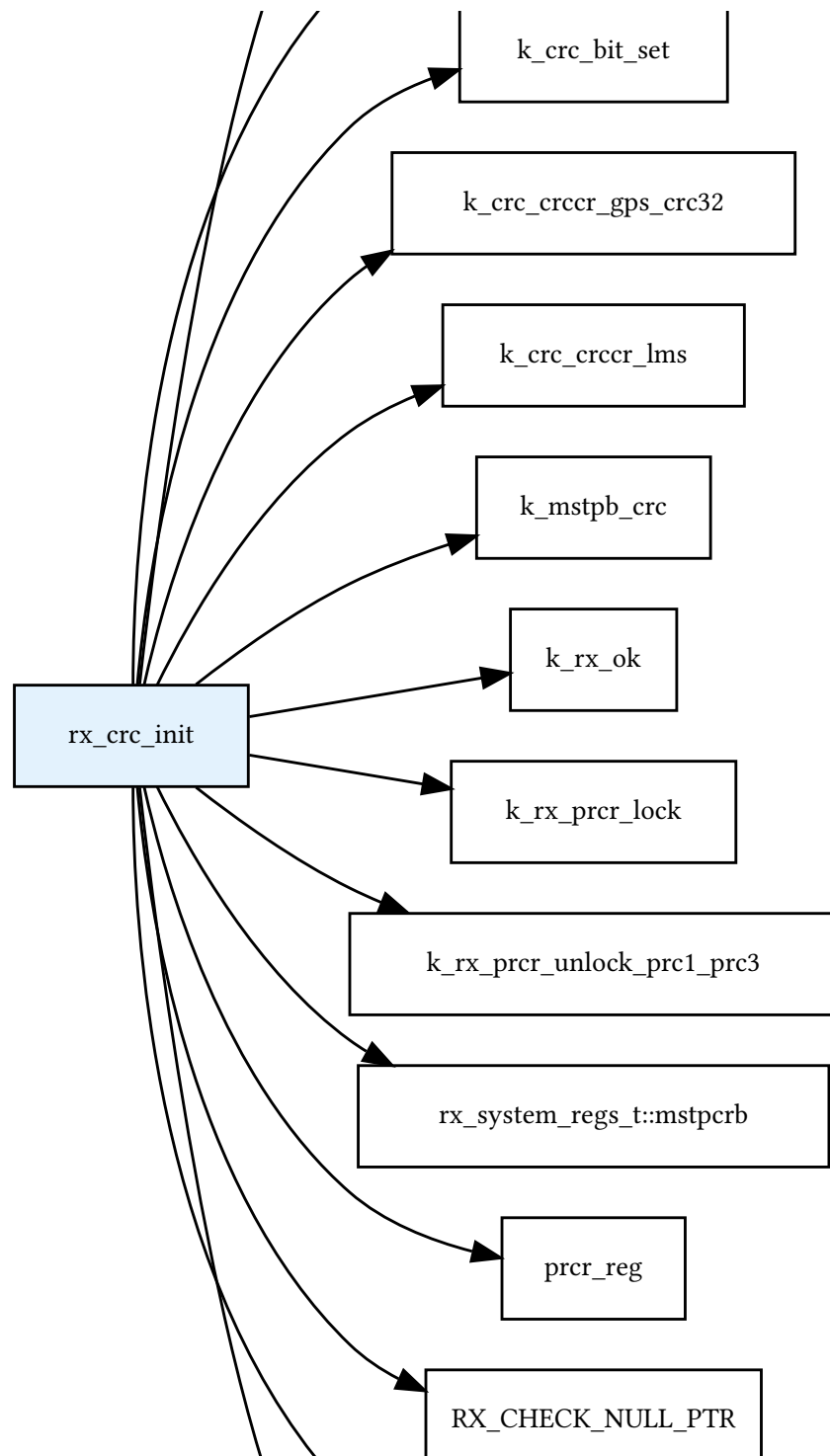
```
rx_err_t err=rx_crc_init(); if(err==k_rx_ok){ rx_log_info("CRC","CRCinitialized(HWaccelerated)"); }
else{ rx_log_warn("CRC","CRCinitfailed:%d(SWfallback)",err); }
```

```
uint32_t crc=rx_crc32_ieee(data,len); }
```

Example - Conditional Hardware CRC:: #ifdefRX if(rx_crc_init()==k_rx_ok)

```
{ rx_log_info("CRC","UsinghardwareCRC(4xfaster)"); }
else{ rx_log_warn("CRC","UsingsoftwareCRC"); } #else rx_crc_init();
rx_log_info("CRC","UsingsoftwareCRC(portable)"); #endif
```

See also: rx_crc_deinit() Disable CRC hardware when done, rx_crc32_ieee() CRC-32 computation (uses HW if initialized), rx_crc32_hw.c Hardware CRC implementation

Figure 328: Call graph for `rx_crc_init`

Defined in `libs/rx_crc/inc/rx_crc.h:449`

Since Version 1.0.0

—

rx_crc_deinit

```
rx_err_t rx_crc_deinit(void)
```

Deinitialize CRC backend (hardware peripheral shutdown).

Disables the CRC hardware peripheral on RX72N targets to save power when CRC operations are no longer needed. On host builds (x86/ARM), this is a no-op since software CRC requires no deinitialization.

Platform Behavior:

- RX72N (RX defined, RX_CRC32_USE_HARDWARE defined):

Disables CRCA (CRC Calculator) peripheral via module stop control Sets MSTPCRB bit 23 = 1 to stop peripheral clock Clears internal initialization flag Frees hardware resource for other modules
Note: Current implementation intentionally leaves CRC enabled for continuous availability

- Host builds (Linux, macOS, Windows for testing):

No-op, returns `k_rx_ok` immediately Software CRC tables remain in memory (statically allocated)

Algorithm (RX72N Hardware Deinit):

1. Validate initialized (prevent deinit of uninitialized module)
2. Disable CRCA module clock via MSTPCRB register (bit 23 = 1)
3. Clear registers (reset peripheral state)
4. Mark uninitialized (clear internal flag)

Power Savings:

- With deinit: CRCA module stopped, saves 0.2 mA @ 3.3V (negligible)
- Without deinit: CRCA remains clocked, minimal power overhead
- Recommendation: Leave CRC enabled unless power is critical (low-power mode)

If power optimization is critical (low-power sleep modes), modify implementation to actually disable peripheral.

Deinitialize CRC backend (hardware peripheral shutdown).

Note: This implementation intentionally does NOT disable the CRC module. The module remains enabled for continuous availability and minimal overhead.

Returns: `k_rx_ok` always (no-op deinit)

Value	Description
<code>k_rx_ok</code>	Deinitialization successful (hardware or no-op)
<code>k_rx_err_invalid_state</code>	Not initialized (RX72N only)

Pre: CRC module must have been initialized via `rx_crc_init()`

Pre: CRC module must have been initialized

Post: On RX72N, CRCA peripheral disabled and clock stopped (if implementation changed)

Post: rx_crc32_ieee() will fall back to software CRC

Post: On host, no observable change (software CRC unaffected)

Post: CRC module state unchanged

Note: Call this ONCE during system shutdown if power optimization is critical

Note: Thread-safe ONLY if called single-threaded during shutdown

Note: After deinit, rx_crc32_ieee() still works (software fallback)

Warning: Do not call from interrupt context

Warning: Current implementation is a no-op to keep CRC available

Design Rationale (No-Op Deinit):: The current implementation intentionally does NOT disable the CRC module after initialization. Reasons: CRC calculations may be needed at any time for frame validation Module stop/start overhead is significant (50 us per init) Power savings from disabling are minimal (<0.2 mA) No functional benefit in typical operation

Performance Impact:: After deinit (SW CRC): 130 us per 100 bytes (4x slower) After init (HW CRC): 32 us per 100 bytes

Example - System Shutdown::

```
void system_shutdown(void) { rx_err_t err = rx_crc_deinit();
if(err != k_rx_ok) { rx_log_info("CRC", "CRC deinitialized (powersaving)"); }
else { rx_log_warn("CRC", "CRC deinit failed: %d", err); }

rx_gpio_deinit(); rx_clock_deinit(); }
```

Example - Conditional Deinit (Low-Power Mode)::

```
void enter_low_power_mode(void)
{ rx_crc_deinit();

uint32_t crc = rx_crc32_ieee(data, len);

rx_power_enter_sleep(); }

void exit_low_power_mode(void) { rx_crc_init();

uint32_t crc = rx_crc32_ieee(data, len); }
```

See also: rx_crc_init() Initialize CRC hardware, rx_crc32_ieee() CRC-32 computation (unaffected by deinit), rx_crc32_hw.c Hardware CRC implementation

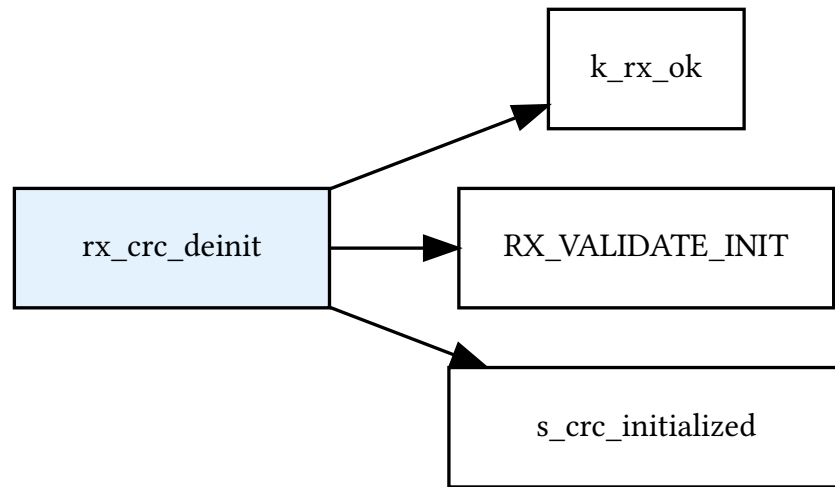


Figure 329: Call graph for rx_crc_deinit

Defined in `libs/rx_crc/inc/rx_crc.h:559`

Since Version 1.0.0

—

rx_crc32_ieee

```
uint32_t rx_crc32_ieee(const uint8_t *data, uint32_t len)
```

Compute CRC-32/IEEE 802.3 checksum over buffer.

Computes 32-bit cyclic redundancy check using IEEE 802.3 polynomial (0x04C11DB7 in reflected/LSB-first form 0xEDB88320). Automatically selects hardware-accelerated or software table-based implementation at compile time.

Algorithm Selection:

- RX72N (if rx_crc_init() called): Hardware CRC peripheral (4x faster)
- RX72N (no init) or Host: Software table lookup (portable, slower)

CRC-32/IEEE 802.3 Specification:

- Polynomial: $x^{32} + x^{26} + x^{23} + \dots + x + 1$ (0x04C11DB7)
- Initial value: 0xFFFFFFFF
- Final XOR: 0xFFFFFFFF
- Bit order: LSB-first (reflected)
- Byte order: Process bytes sequentially, LSB of each byte first

Compatibility:

- Matches Go `crc32.ChecksumIEEE()`
- Matches Python `zlib.crc32()`
- Matches Ethernet FCS (Frame Check Sequence)
- Matches ZIP, PNG, Gzip CRC-32

Algorithm (Software):

1. Initialize CRC: `crc = 0xFFFFFFFF`

2. For each byte:

```
table_index = (crc ^ byte) & 0xFF; crc = (crc >> 8) ^ lookup_table[table_index]
```

3. Finalize: $\text{crc} \wedge = 0xFFFFFFFF$

4. Return final CRC value

Algorithm (Hardware, RX72N):

1. Write data to CRCA data register (DMA or CPU)

2. Read CRC from CRCA result register

3. Apply final XOR ($0xFFFFFFFF$)

4. Return final CRC value

Compute CRC-32/IEEE 802.3 checksum over buffer.

Computes the 32-bit Cyclic Redundancy Check using IEEE 802.3 polynomial (0x04C11DB7). This is the primary entry point for one-shot CRC calculation over contiguous data buffers.

This function is a facade that delegates to the appropriate backend implementation selected at compile time:

- Hardware: rx_crc32_hw.c (RX72N CRC Calculator peripheral)
- Software: rx_crc32_sw.c (256-entry lookup table)

The backend selection is transparent to the caller - the API is identical and results are bit-exact regardless of implementation.

Parameters:

Direction	Name	Description
in	data	Input buffer to checksum Valid range: NULL or pointer to buffer of size len NULL handling: If NULL, len must be 0 (returns $0xFFFFFFFF \text{ XOR } 0xFFFFFFFF = 0$) Alignment: No alignment requirement (byte-addressable)
in	len	Number of bytes to include in CRC Valid range: 0 - 4294967295 ($0 - 2^{32}-1$) Typical: 10-275 bytes (frame sizes) Zero handling: len=0 returns init value XOR final XOR = 0

Returns: uint32_t Computed CRC-32 checksum Range: 0x00000000 - 0xFFFFFFFF (all 32-bit values possible) Deterministic: Same input always produces same output Zero for zero: Empty buffer (len=0) returns 0

Pre: If data != nullptr, data must point to valid buffer of size len

Pre: len must accurately reflect data buffer size (no bounds checking)

Post: Return value is valid IEEE 802.3 CRC-32

Post: No side effects, pure function (thread-safe)

Note: This function is thread-safe (stateless, read-only tables)

Note: For incremental CRC over multiple buffers, use `rx_crc32_update()`

Warning: Passing invalid data pointer or incorrect len causes undefined behavior

Warning: For large buffers (>10 KB), consider chunking with `rx_crc32_update()`

Performance:: Hardware (RX72N): 32 us / 100 bytes @ 240 MHz Software (RX72N): 130 us / 100 bytes @ 240 MHz Software (x86): 50 us / 100 bytes @ 3 GHz

Example - Basic Usage:: `uint8_tdata[]="HelloWorld";
uint32_tcrc=rx_crc32_ieee(data,sizeof(data)-1); printf("CRC-32:0x%08Xn",crc);`

Example - Frame Integrity Check:: `uint8_ttx_frame[279];
uint32_tcrc=rx_crc32_ieee(tx_frame,275);

tx_frame[275]=(crc>>0)&0xFF; tx_frame[276]=(crc>>8)&0xFF; tx_frame[277]=(crc>>16)&0xFF;
tx_frame[278]=(crc>>24)&0xFF;

uint8_trx_frame[279];

uint32_treceived_crc=(rx_frame[275]<<0)|(rx_frame[276]<<8)|(rx_frame[277]<<16)|
(rx_frame[278]<<24);

uint32_tcalculated_crc=rx_crc32_ieee(rx_frame,275);

if(received_crc==calculated_crc){ }else{ rx_log_error("CRC","Mismatch:rx=0x%08Xcalc=0x%08X",
received_crc,calculated_crc); }`

Example - Zero-Length Buffer:: `uint32_tcrc=rx_crc32_ieee(nullptr,0);`

Example - Performance Comparison:: `#include"rx_time.h"

uint8_tdata[1024]; memset(data,0x55,sizeof(data));

uint32_tstart=rx_time_get_us(); uint32_tcrc_sw=rx_crc32_ieee(data,sizeof(data));
uint32_telapsed_sw=rx_time_get_us()-start; rx_log_info("CRC","Software:%uus",elapsed_sw);

rx_crc_init(); start=rx_time_get_us(); uint32_tcrc_hw=rx_crc32_ieee(data,sizeof(data));
uint32_telapsed_hw=rx_time_get_us()-start; rx_log_info("CRC","Hardware:%uus(%1.fxfaster)",
elapsed_hw,(float)elapsed_sw/elapsed_hw);

assert(crc_sw==crc_hw);`

See also: `rx_crc32_update()` Incremental CRC for multiple buffers, `rx_crc_init()` Initialize hardware CRC (optional, for performance), `rx_crc32_hw.c` Hardware CRC implementation (RX72N), `rx_crc32_sw.c` Software CRC implementation (portable)

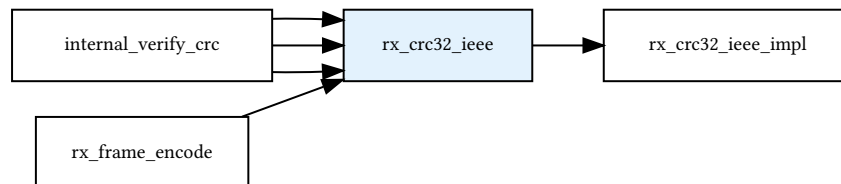


Figure 330: Call graph for `rx_crc32_ieee`

Defined in `libs/rx_crc/inc/rx_crc.h:711`

Since Version 1.0.0

—

rx_crc32_update

```
uint32_t rx_crc32_update(uint32_t crc, const uint8_t *data, uint32_t len)
```

Update CRC-32 with additional data (incremental computation).

Continues a CRC-32/IEEE 802.3 calculation from a previous partial result, allowing incremental CRC computation over non-contiguous buffers or streaming data without concatenation. Uses software table-based implementation (hardware CRC peripheral does not support loading arbitrary initial state).

Use Cases:

1. Streaming data: Compute CRC as data arrives in chunks
2. Non-contiguous buffers: CRC over header + payload without copying
3. Large data sets: Process data in manageable chunks
4. Memory-constrained: Avoid allocating large concatenation buffer

Implementation:

- Always uses software CRC (even on RX72N with hardware support)
- Rationale: Hardware CRC peripheral cannot load arbitrary initial CRC state
- Performance: Same as software `rx_crc32_ieee()` (130 us per 100 bytes @ 240 MHz)

Algorithm (Software Table Lookup):

1. Un-finalize input CRC: $\text{crc} \wedge = 0xFFFFFFFF$ (reverse final XOR)
2. For each byte:

$\text{table_index} = (\text{crc} \wedge \text{byte}) \& 0xFF$ $\text{crc} = (\text{crc} \gg 8) \wedge \text{lookup_table}[\text{table_index}]$

3. Re-finalize CRC: $\text{crc} \wedge = 0xFFFFFFFF$ (apply final XOR)
4. Return updated CRC value

Mathematical Equivalence: Computing CRC over concatenated data produces identical result:

Update CRC-32 with additional data (incremental computation).

Continues CRC-32 calculation from a previous CRC value, allowing computation over non-contiguous buffers or streaming data without buffer concatenation.

This function is a facade that delegates to the backend implementation (hardware or software) selected at compile time, just like `rx_crc32_ieee()`.

Parameters:

Direction	Name	Description
in	crc	Previous CRC-32 value Valid range: 0x00000000 - 0xFFFFFFFF (any 32-bit value) Initial value: Use result from <code>rx_crc32_ieee()</code> or previous <code>rx_crc32_update()</code> Special case: 0xFFFFFFFF is valid previous CRC (not same as init value)

in	data	Additional data buffer to include in CRC Valid range: NULL or pointer to buffer of size len NULL handling: If NULL, len must be 0 (returns input crc unchanged) Alignment: No alignment requirement (byte-addressable)
in	len	Number of bytes in data buffer Valid range: 0 - 4294967295 (0 - 2 ³² -1) Typical: 10-275 bytes (frame chunks) Zero handling: len=0 returns input crc unchanged

Returns: uint32_t Updated CRC-32 checksum Range: 0x00000000 - 0xFFFFFFFF (all 32-bit values possible) Deterministic: Same inputs always produce same output Associative: Order of updates matters, but partial results can be cached

Pre: If data != nullptr, data must point to valid buffer of size len

Pre: len must accurately reflect data buffer size (no bounds checking)

Post: Return value is valid IEEE 802.3 CRC-32

Post: Input crc is unchanged (pure function)

Post: No side effects (thread-safe)

Note: This function is thread-safe (stateless, read-only tables)

Note: For single-shot CRC over contiguous buffer, use rx_crc32_ieee() (simpler)

Note: Software CRC only - hardware peripheral does not support incremental mode

Warning: Passing invalid data pointer or incorrect len causes undefined behavior

Warning: Do not mix rx_crc32_ieee() and rx_crc32_update() incorrectly (see examples)

Performance:: Software (RX72N): 130 us / 100 bytes @ 240 MHz (same as rx_crc32_ieee) Software (x86): 50 us / 100 bytes @ 3 GHz

Example - Basic Incremental CRC:: uint8_t header[10]={0xAA,0x55,...}; uint8_t payload[64]={...};
uint32_t crc=rx_crc32_ieee(header,sizeof(header));
crc=rx_crc32_update(crc,payload,sizeof(payload));
printf("CRC-32:0x%08Xn",crc);

Example - Streaming Data:: uint32_t crc=rx_crc32_ieee(nullptr,0); crc^=0xFFFFFFFF;
uint8_t chunk1[32]; spi_receive(chunk1,sizeof(chunk1));
crc=rx_crc32_update(crc,chunk1,sizeof(chunk1));

```
uint8_tchunk2[64]; spi_receive(chunk2,sizeof(chunk2));
crc=rx_crc32_update(crc,chunk2,sizeof(chunk2));
```

```
uint8_tchunk3[16]; spi_receive(chunk3,sizeof(chunk3));
crc=rx_crc32_update(crc,chunk3,sizeof(chunk3));
```

```
printf("StreamingCRC-32:0x%08Xn",crc);
```

Example - Frame Header + Payload:: typedef struct{ uint16_tsync; uint16_tseq; uint8_tlen;
uint8_ttype; uint8_tflags; uint8_treserved; }spi_header_t;

```
spi_header_theader; uint8_tpayload[255];
```

```
uint32_tcrc=rx_crc32_ieee((uint8_t*)&header,sizeof(header));
crc=rx_crc32_update(crc,payload,header.len);
```

```
uint8_tcrc_bytes[4]={ (crc>>0)&0xFF, (crc>>8)&0xFF, (crc>>16)&0xFF, (crc>>24)&0xFF, };
spi_transmit(crc_bytes,sizeof(crc_bytes));
```

Example - Multi-Buffer CRC (Memory Efficient):: uint8_tbuf1[64]; uint8_tbuf2[128];
uint8_tbuf3[32];

```
uint32_tcrc=rx_crc32_ieee(buf1,sizeof(buf1)); crc=rx_crc32_update(crc,buf2,sizeof(buf2));
crc=rx_crc32_update(crc,buf3,sizeof(buf3));
```

Example - WRONG Usage (Common Mistake):: uint32_tcrc=rx_crc32_ieee(nullptr,0);
crc=rx_crc32_update(crc,buf1,len1);

```
uint32_tcrc=rx_crc32_ieee(buf1,len1); crc=rx_crc32_update(crc,buf2,len2);
```

```
uint32_tcrc=0xFFFFFFFF; crc=rx_crc32_update(crc,buf1,len1); crc=rx_crc32_update(crc,buf2,len2);
crc^=0xFFFFFFFF;
```

Example:

```
//Method1:Incremental
uint32_tcrc=rx_crc32_ieee(buf1,len1);
crc=rx_crc32_update(crc,buf2,len2);

//Method2:Concatenated(equivalent)
uint8_tcombined[len1+len2];
memcpy(&combined[0],buf1,len1);
memcpy(&combined[len1],buf2,len2);
uint32_tcrc=rx_crc32_ieee(combined,len1+len2);
```

```
//BothmethodsproduceidenticalCRC
```

See also: rx_crc32_ieee() Single-shot CRC-32 computation, rx_crc32_sw.c Software CRC implementation (used by this function)

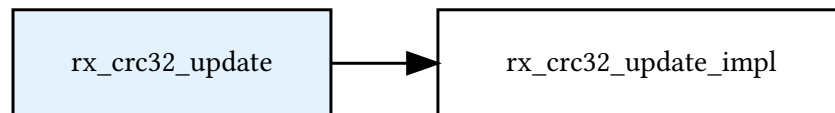


Figure 331: Call graph for rx_crc32_update

Defined in `libs/rx_crc/inc/rx_crc.h:909`

Since Version 1.0.0

rx_crc8_maxim

```
uint8_t rx_crc8_maxim(const uint8_t *data, uint32_t len)
```

Compute CRC-8/Maxim checksum for Dallas/Maxim OneWire devices.

Computes 8-bit cyclic redundancy check using Dallas/Maxim polynomial (0x31) in reflected/LSB-first form (0x8C). This CRC-8 variant is used by all Dallas Semiconductor and Maxim Integrated OneWire devices for ROM code and scratchpad data validation.

CRC-8/Maxim Specification:

- Polynomial: $x^8 + x^5 + x^4 + 1$ (0x31 normal, 0x8C reflected)
- Initial value: 0x00
- Final XOR: 0x00 (no finalization)
- Bit order: LSB-first (reflected)
- Byte order: Process bytes sequentially, LSB of each byte first

Compatibility:

- DS18B20 temperature sensor (ROM code validation)
- DS2482 OneWire bridge (command/response validation)
- DS2431 EEPROM (memory page validation)
- DS28EA00 IO device (ROM code validation)
- All Dallas/Maxim OneWire devices

Algorithm (Software Table Lookup):

1. Initialize CRC: $crc = 0x00$
2. For each byte:

$table_index = crc \oplus byte$ $crc = lookup_table[table_index]$

3. Return final CRC value (no finalization XOR)

Mathematical Definition:

Polynomial:

Binary representation: 0x31 (normal), 0x8C (reflected)

Algorithm (LSB-first):

Properties:

- Detects all single-bit errors
- Detects all 2-bit errors within 8 bits
- Detects all burst errors ≤ 8 bits
- Detects 99.6% of all other errors (256 possible values)

Compute CRC-8/Maxim checksum for Dallas/Maxim OneWire devices.

Computes the Dallas/Maxim CRC-8 checksum used by all OneWire devices for ROM code validation and data integrity checking. Uses a bitwise algorithm (no lookup table) optimized for the small payloads typical in OneWire communications.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	data	Input buffer to checksum Valid range: NULL or pointer to buffer of size len NULL handling: If NULL, len must be 0 (returns 0x00) Alignment: No alignment requirement (byte-addressable) Typical size: 7-8 bytes (OneWire ROM code)
in	len	Number of bytes to include in CRC Valid range: 0 - 4294967295 (0 - $2^{32}-1$) Typical: 7 bytes (ROM code without CRC) or 8 bytes (ROM with CRC) Zero handling: len=0 returns 0x00

Returns: uint8_t Computed CRC-8 checksum Range: 0x00 - 0xFF (all 8-bit values possible)
Deterministic: Same input always produces same output Zero for zero: Empty buffer (len=0) returns 0x00

Pre: If data != nullptr, data must point to valid buffer of size len

Pre: len must accurately reflect data buffer size (no bounds checking)

Post: Return value is valid CRC-8/Maxim checksum

Post: No side effects, pure function (thread-safe)

Note: This function is thread-safe (stateless, read-only tables)

Note: Software implementation only (no hardware acceleration on RX72N)

Note: For OneWire ROM codes, compute CRC over first 7 bytes, compare to byte 8

Warning: Passing invalid data pointer or incorrect len causes undefined behavior

Warning: Different from other CRC-8 variants (CRC-8/CCITT, CRC-8/WCDMA, etc.)

Performance:: Software (RX72N): 15 us / 8 bytes @ 240 MHz Software (x86): 5 us / 8 bytes @ 3 GHz

Example - OneWire ROM Code Validation::

```
uint8_t rom[8]={0x28,0xFF,0x64,0x1F,0x21,0x16,0x04,0xB3};
uint8_t calculated_crc=rx_crc8_maxim(rom,7); uint8_t received_crc=rom[7];
if(calculated_crc==received_crc){ printf("ROMvalid:Family=0x%02XCRC=0x%02Xn",rom[0],rom[7]); }
else{ printf("ROMinvalid:expected=0x%02Xactual=0x%02Xn", calculated_crc,received_crc); }
```

Example - DS18B20 Scratchpad Validation:: uint8_t scratchpad[9];

```
rx_onewire_read_scratchpad(scratchpad,sizeof(scratchpad));
```

```
uint8_t calculated_crc=rx_crc8_maxim(scratchpad,8); uint8_t received_crc=scratchpad[8];
```

```

if(calculated_crc==received_crc){ int16_t raw_temp=(scratchpad[1]<<8)|scratchpad[0];
float temp_c=raw_temp/16.0f; printf("Temperature: %.2fdegC\n",temp_c); }
else{ rx_log_error("OneWire","ScratchpadCRCMismatch"); }

```

Example - Computing CRC for Transmission::

```

void send_owewire_command(uint8_t cmd,uint8_t param) { uint8_t buffer[3]; buffer[0]=cmd;
buffer[1]=param;

buffer[2]=rx_crc8_maxim(buffer,2);

rx_owewire_write(buffer,sizeof(buffer)); }

```

Example - Batch ROM Validation:: typedef struct{ uint8_t rom[8]; bool valid; } onewire_device_t;

```

onewire_device_t devices[4];

for(uint8_t i=0;i<4;i++){ if(rx_owewire_search(&devices[i].rom[0]))
{ uint8_t crc=rx_crc8_maxim(devices[i].rom,7); devices[i].valid=(calc_crc==devices[i].rom[7]);

if(devices[i].valid)
{ rx_log_info("OneWire","Device%u: Family=0x%02X Serial=%02X%02X%02X%02X%02X",
i,devices[i].rom[0], devices[i].rom[6],devices[i].rom[5],devices[i].rom[4],
devices[i].rom[3],devices[i].rom[2],devices[i].rom[1]); }
else{ rx_log_error("OneWire","Device%u: Invalid ROMCRC",i); } } }

```

Example - Zero-Length Buffer:: uint8_t crc=rx_crc8_maxim(nullptr,0);

Example - Manual CRC Verification (Algorithm Understanding)::

```

uint8_t data[]={0x28,0xFF,0x64,0x1F,0x21,0x16,0x04};

uint8_t crc_lib=rx_crc8_maxim(data,sizeof(data)); printf("LibraryCRC:0x%02X\n",crc_lib);

extern const uint8_t rx_crc8_maxim_table[256]; uint8_t crc_manual=0x00; for(uint8_t i=0;i<sizeof(data);i++)
{ crc_manual=crc8_maxim_table[crc_manual^data[i]]; }

printf("ManualCRC:0x%02X\n",crc_manual);

assert(crc_lib==crc_manual);

```

See also: rx_crc32_ieee() CRC-32 for frame protocols and Ethernet, rx_crc8.c CRC-8/Maxim implementation (software table), rx_hcsr04.h OneWire interface for DS18B20 and similar devices

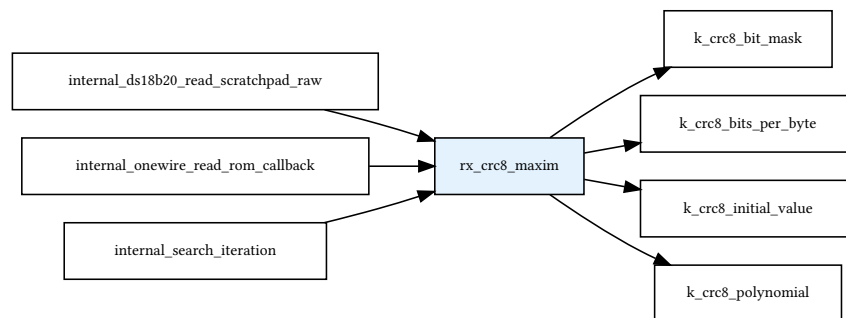


Figure 332: Call graph for rx_crc8_maxim

Defined in `libs/rx_crc/inc/rx_crc.h:1116`

Since Version 1.0.0

rx_crc_internal.h

Internal CRC-32 Abstraction Layer - Hardware/Software Implementation Selection.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

RX_CRC32_USE_HARDWARE

```
#define RX_CRC32_USE_HARDWARE
```

rx_crc_shared_constants_t

Shared constants for CRC validation and NASA-compliant bounded loops.

This enumeration defines shared constants used by both hardware and software CRC implementations. These constants ensure:

1. Consistent validation across implementations
2. NASA Power of 10 Rule 2 compliance - All loops have fixed upper bounds
3. Predictable memory access patterns - No unbounded iterations

Value	Name	Description
= 1	k_crc_len_min	Minimum CRC data length (1 byte).
= 65535	k_crc_len_max	Maximum CRC data length for bounded loops (64KB - 1).
= 0	k_crc_idx_start	Loop start index (zero-based).

rx_crc_init

```
rx_err_t rx_crc_init(void)
```

Initialize CRC module (internal implementation).

Initializes the CRC subsystem based on compile-time configuration:

Hardware Implementation (RX72N, RX_CRC32_USE_HARDWARE defined):

1. Validate not already initialized (prevent double-init)
2. Clear module stop bit (MSTPCRB.MSTPB23 = 0) to enable CRCA clock
3. Wait for peripheral ready (typically 1 us)
4. Configure polynomial register to 0x04C11DB7 (IEEE 802.3)
5. Set bit order to LSB-first (reflected mode)
6. Set initial value to 0xFFFFFFFF
7. Mark module as initialized

Software Implementation (Host or RX_CRC32_USE_SOFTWARE defined):

1. No-op (lookup table is statically allocated)
2. Return k_rx_ok immediately

Initialize CRC module (internal implementation).

Initialize CRC backend (hardware peripheral if available).

Enables the hardware CRC module by clearing the module stop bit (MSTPCRB[23]) and configures the peripheral for IEEE 802.3 CRC-32 polynomial with LSB-first bit reflection (compatible with `crc32.ChecksumIEEE()` in Go).

This function is idempotent - calling it multiple times is safe and returns success immediately if already initialized.

Returns: `rx_err_t` Initialization result

Value	Description
<code>k_rx_ok</code>	Initialization successful
<code>k_rx_err_invalid_state</code>	Already initialized (hardware only)
<code>k_rx_err_hardware</code>	Peripheral configuration failed (hardware only)

Pre: System clocks must be configured and running

Pre: For hardware: Module stop registers must be accessible

Post: Hardware: CRCA peripheral ready for CRC computation

Post: Software: No state change (lookup table already in ROM)

Note: Thread safety: NOT thread-safe, call during single-threaded init

Note: Called by public `rx_crc_init()` in `rx_crc.h`

Warning: Do not call from interrupt context

Initialization Flow:: digraph init_flow { rankdir=TB; node [shape=box, style=rounded]; start [label="rx_crc_init()"]; check [label="Already initialized?", shape=diamond]; hw_check [label="Hardware CRC?", shape=diamond]; enable_clock [label="Enable CRCA clockn(MSTPCRB.MSTPB23=0)"]; config [label="Configure polynomial and bit order"]; sw_noop [label="No-opn(table is static)"]; set_flag [label="Set initialized flag"]; ret_ok [label="Return k_rx_ok"]; ret_err [label="Return k_rx_err_invalid_state"];

start -> check; check -> ret_err [label="Yes"]; check -> hw_check [label="No"]; hw_check -> enable_clock [label="Yes"]; hw_check -> sw_noop [label="No"]; enable_clock -> config; config -> set_flag; sw_noop -> ret_ok; set_flag -> ret_ok; }

Performance:: Hardware init: 50 us (includes clock stabilization) Software init: < 1 us (no-op)

Example (Internal Use Only):: `rx_err_t err=rx_crc_init(); if(err!=k_rx_ok){ s_use_hardware=false; }`

See also: `rx_crc.h` Public API (use this in application code), `rx_crc_deinit()` Shutdown CRC module

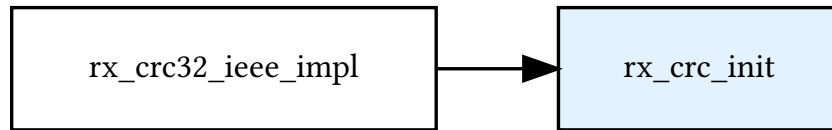


Figure 333: Call graph for rx_crc_init

_Defined in libs/rx_crc/inc/rx_crc_internal.h:404_

Since Version 1.0.0

—

rx_crc_deinit

```
rx_err_t rx_crc_deinit(void)
```

Deinitialize CRC module (internal implementation).

Shuts down the CRC subsystem based on compile-time configuration:

Hardware Implementation (RX72N, RX_CRC32_USE_HARDWARE defined):

1. Validate module is initialized
2. Set module stop bit (MSTPCRB.MSTPB23 = 1) to disable CRCA clock
3. Clear internal initialization flag

Software Implementation (Host or RX_CRC32_USE_SOFTWARE defined):

1. No-op (lookup table remains in memory)
2. Return k_rx_ok immediately

Design Note: Current implementation intentionally leaves CRC enabled after deinit because:

- CRC may be needed at any time for frame validation
- Module stop/start overhead is significant (50 us)
- Power savings are minimal (< 0.2 mA)

Deinitialize CRC module (internal implementation).

Deinitialize CRC backend (hardware peripheral shutdown).

Note: This implementation intentionally does NOT disable the CRC module. The module remains enabled for continuous availability and minimal overhead.

Returns: k_rx_ok always (no-op deinit)

Value	Description
k_rx_ok	Deinitialization successful
k_rx_err_invalid_state	Not initialized (hardware only)

Pre: Module must be initialized via rx_crc_init()

Pre: CRC module must have been initialized

Post: Hardware: CRCA peripheral stopped (power saving)

Post: Software: No state change

Post: CRC module state unchanged

Note: Thread safety: NOT thread-safe, call during single-threaded shutdown

Note: Called by public `rx_crc_deinit()` in `rx_crc.h`

Warning: Do not call from interrupt context

Example (Internal Use Only):: `rx_err_t err=rx_crc_deinit();`

See also: `rx_crc.h` Public API (use this in application code), `rx_crc_init()` Initialize CRC module

_Defined in `libs/rx\crc/inc/rx\crc\_internal.h:452_`

Since Version 1.0.0

—

rx_crc32_ieee_impl

```
uint32_t rx_crc32_ieee_impl(const uint8_t *data, uint32_t len)
```

Calculate IEEE 802.3 CRC-32 (internal implementation).

Internal implementation function that computes CRC-32 using the compile-time selected algorithm (hardware or software).

Algorithm (IEEE 802.3 CRC-32):

Hardware Path (RX72N):

1. Reset CRC peripheral data register
2. Write data bytes to CRCDIR register
3. Read result from CRCDOR register
4. Apply final XOR (0xFFFFFFFF)

Software Path:

1. Initialize CRC to 0xFFFFFFFF
2. For each byte: table lookup and XOR
3. Apply final XOR (0xFFFFFFFF)

Calculate IEEE 802.3 CRC-32 (internal implementation).

Computes 32-bit Cyclic Redundancy Check using the RX72N CRC Calculator peripheral, providing 5x faster throughput compared to software implementation.

This is the internal implementation called by `rx_crc32_ieee()` in `rx_crc32.c` when hardware backend is selected at compile time.

Result is bit-exact compatible with:

- Go: `crc32.ChecksumIEEE(data)`
- Python: `zlib.crc32(data) & 0xFFFFFFFF`

- C++: boost::crc_32_type
- Linux: cksum -o3 (after byte-swap)

Parameters:

Direction	Name	Description
in	data	Input data buffer Valid range: NULL or pointer to valid buffer NULL handling: Returns 0 if nullptr Alignment: No requirement (byte-addressable)
in	len	Data length in bytes Valid range: 0 to k_crc_len_max (65535) Zero handling: Returns 0 (init XOR final)

Returns: uint32_t CRC-32 checksum Range: 0x00000000 to 0xFFFFFFFF Returns 0 if data is nullptr or len is 0

Pre: If data != nullptr, buffer must be valid for len bytes

Pre: len must not exceed k_crc_len_max

Post: No side effects (pure function in software mode)

Post: Hardware mode: CRC peripheral state modified

Note: Called by public rx_crc32_ieee() in rx_crc.h

Note: Thread safety: SW=Yes, HW=No (shared peripheral)

Warning: Do not call directly - use rx_crc32_ieee() from rx_crc.h

Performance:: Hardware: 32 us / 100 bytes @ 240 MHz Software: 130 us / 100 bytes @ 240 MHz

See also: rx_crc32_ieee() Public API wrapper, rx_crc32_update_impl() Incremental CRC computation

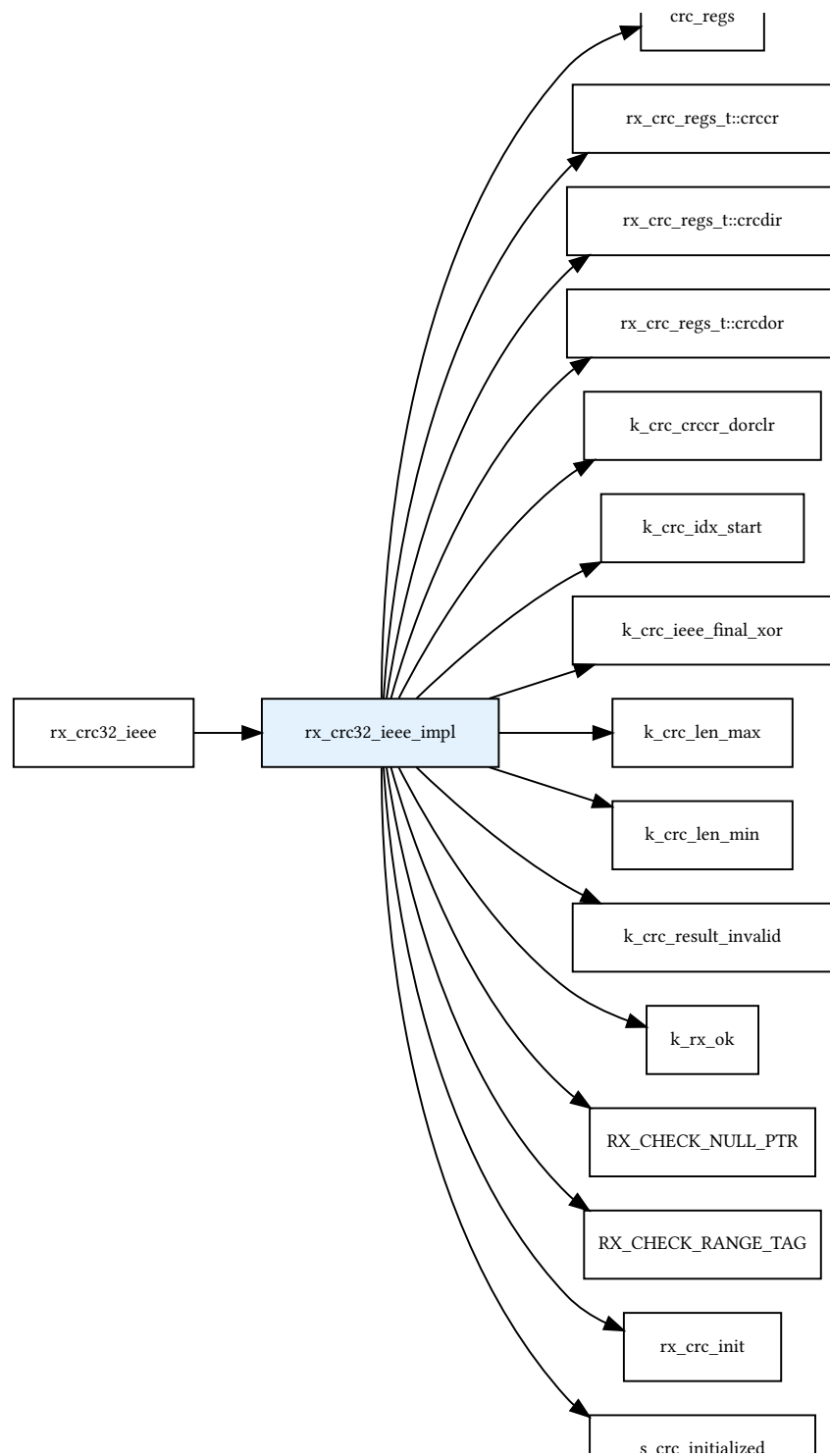


Figure 334: Call graph for rx_crc32_ieee_impl

_Defined in libs/rx_crc/inc/rx_crc_internal.h:515_

Since Version 1.0.0

—

rx_crc32_update_impl

```
uint32_t rx_crc32_update_impl(uint32_t crc, const uint8_t *data, uint32_t len)
```

Update CRC-32 with additional data (internal implementation).

Internal implementation for incremental CRC-32 computation. This function continues a CRC calculation from a previous partial result, enabling CRC computation over non-contiguous buffers.

Algorithm:

1. Un-finalize input CRC: $\text{crc} \wedge= 0xFFFFFFFF$
2. For each byte: table lookup and XOR update
3. Re-finalize CRC: $\text{crc} \wedge= 0xFFFFFFFF$

Implementation Note: This function ALWAYS uses software CRC because the hardware CRC peripheral cannot load an arbitrary initial CRC state.

Update CRC-32 with additional data (internal implementation).

Computes incremental CRC-32 by updating a previous CRC value with new data. Uses software implementation because RX72N hardware doesn't support loading arbitrary initial CRC state.

Parameters:

Direction	Name	Description
in	crc	Previous CRC-32 value (finalized) Valid range: 0x00000000 to 0xFFFFFFFF Should be result from previous rx_crc32_ieee() or rx_crc32_update()
in	data	Additional data buffer Valid range: NULL or pointer to valid buffer NULL handling: Returns input crc unchanged Alignment: No requirement
in	len	Data length in bytes Valid range: 0 to k_crc_len_max (65535) Zero handling: Returns input crc unchanged
in	crc	Previous CRC value to continue from
in	data	Pointer to new data buffer
in	len	Length of new data in bytes

Returns: Updated CRC-32 checksum

Pre: If data != nullptr, buffer must be valid for len bytes

Pre: crc should be from previous CRC computation

Pre: data must be non-NULL

Pre: len must be in valid range

Post: Return value can be passed to next rx_crc32_update() call

Post: No side effects (pure function)

Note: Always uses software implementation (hardware limitation)

Note: Thread safety: Yes (stateless, read-only tables)

Warning: Do not call directly - use rx_crc32_update() from rx_crc.h

Mathematical Equivalence:: uint32_t crc = rx_crc32_ieee(buf1, len1);

crc = rx_crc32_update(crc, buf2, len2);

uint8_t combined[len1 + len2]; memcpy(&combined[0], buf1, len1);

memcpy(&combined[len1], buf2, len2); uint32_t t_crc = rx_crc32_ieee(combined, len1 + len2);

See also: rx_crc32_update() Public API wrapper, rx_crc32_update_sw() Actual software implementation

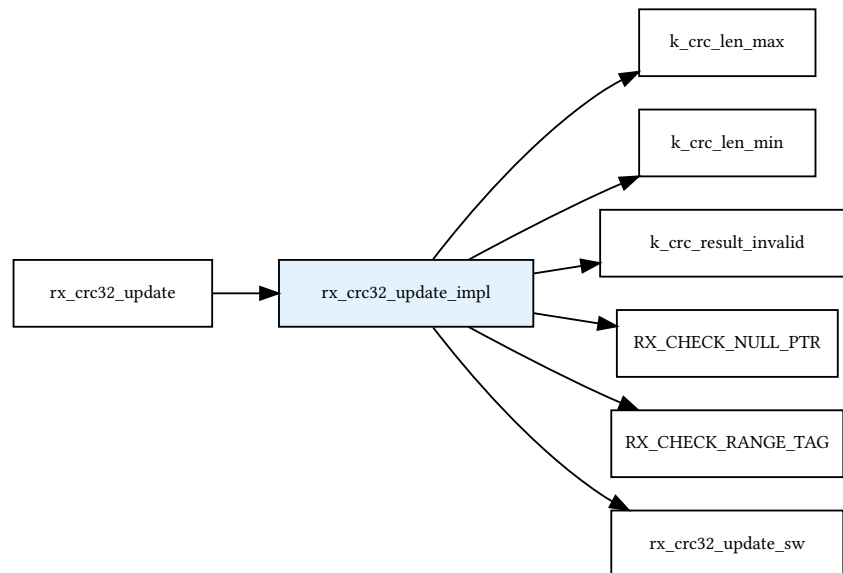


Figure 335: Call graph for rx_crc32_update_impl

_Defined in libs/rx_crc/inc/rx_crc_internal.h:574_

Since Version 1.0.0

rx_crc32_update_sw

```
uint32_t rx_crc32_update_sw(uint32_t crc, const uint8_t *data, uint32_t len)
```

Software CRC-32 update (always available fallback).

Pure software table-based CRC-32 update function. This function is ALWAYS compiled and available, even when hardware CRC is the default, because:

1. Host testing: No hardware available on x86/ARM hosts
2. Incremental CRC: Hardware cannot load arbitrary initial state
3. A/B testing: Compare hardware vs software results for validation

Algorithm (Table Lookup, LSB-first):

Performance Characteristics:

- Time complexity: $O(n)$ where $n = \text{len}$
- Space complexity: $O(1)$ stack + $O(1024)$ lookup table (shared)
- Cache behavior: Good locality (sequential table access)

Lookup table is read-only and never modified

Function has no side effects

Parameters:

Direction	Name	Description
in	<code>crc</code>	Previous CRC-32 value (finalized form) Valid range: 0x00000000 to 0xFFFFFFFF Initial call: Use result from <code>rx_crc32_ieee()</code> on first buffer Subsequent calls: Use result from previous <code>rx_crc32_update_sw()</code>
in	<code>data</code>	Additional data buffer to include in CRC Valid range: NULL or pointer to valid buffer of size <code>len</code> NULL handling: Returns input <code>crc</code> unchanged (safe no-op) Alignment: No requirement (byte-addressable)
in	<code>len</code>	Number of bytes in data buffer Valid range: 0 to <code>k_crc_len_max</code> (65535) Zero handling: Returns input <code>crc</code> unchanged

Returns: `uint32_t` Updated CRC-32 checksum Finalized form (ready for comparison or next update)
Range: 0x00000000 to 0xFFFFFFFF

Pre: If `data != nullptr`, data must point to valid buffer of size `len`

Pre: `len` must not exceed `k_crc_len_max` (NASA Rule 2)

Post: Return value is valid IEEE 802.3 CRC-32

Post: Input `crc` is unchanged (pure function)

Note: Thread safety: Yes (stateless, read-only global table)

Note: This function is called by `rx_crc32_update_impl()`

Note: Safe to call from interrupt context (no blocking)

Warning: Do not exceed `k_crc_len_max` to maintain NASA compliance

Memory Usage:: Stack: 16 bytes (loop counter, temps) Lookup table: 1024 bytes (shared static, in .rodata)

Performance:: 130 us / 100 bytes @ 240 MHz RX72N 50 us / 100 bytes @ 3 GHz x86 Throughput: 0.78 MB/s (RX72N)

Example:: `uint8_t header[10]={0xAA,0x55,...}; uint8_t payload[64]={...};`

`uint32_t crc=rx_crc32_ieee(header,sizeof(header));`

`crc=rx_crc32_update_sw(crc,payload,sizeof(payload));`

See also: `rx_crc32_update()` Public API that calls this function, `rx_crc32_ieee()` Single-shot CRC computation, `rx_crc32_sw.c` Implementation file

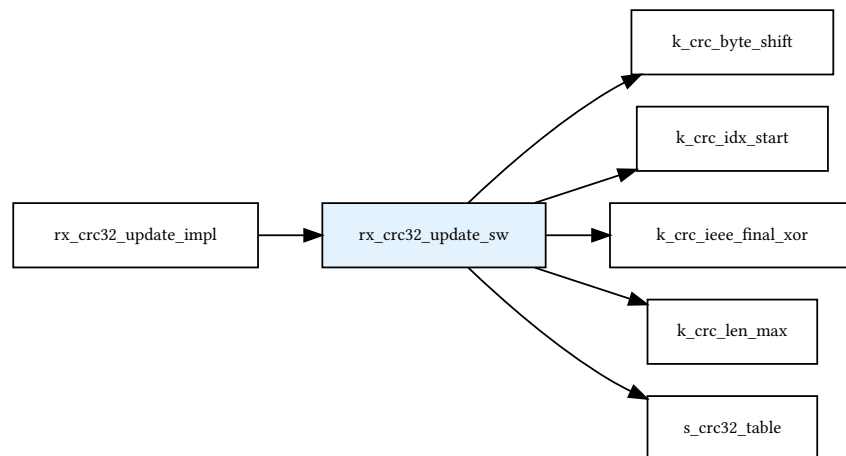


Figure 336: Call graph for `rx_crc32_update_sw`

_Defined in `libs/rx\crc/inc/rx\crc\_internal.h:666_`

Since Version 1.0.0

—

rx_crc32.c

IEEE 802.3 CRC-32 Public API with Hardware/Software Dispatcher.

Includes:

- `"rx_crc.h"`
- `"rx_crc_internal.h"`

rx_crc32_ieee

```
uint32_t rx_crc32_ieee(const uint8_t *data, uint32_t len)
```

Calculate IEEE 802.3 CRC-32 checksum over entire buffer.

Compute CRC-32/IEEE 802.3 checksum over buffer.

Computes the 32-bit Cyclic Redundancy Check using IEEE 802.3 polynomial (0x04C11DB7). This is the primary entry point for one-shot CRC calculation over contiguous data buffers.

This function is a facade that delegates to the appropriate backend implementation selected at compile time:

- Hardware: rx_crc32_hw.c (RX72N CRC Calculator peripheral)
- Software: rx_crc32_sw.c (256-entry lookup table)

The backend selection is transparent to the caller - the API is identical and results are bit-exact regardless of implementation.

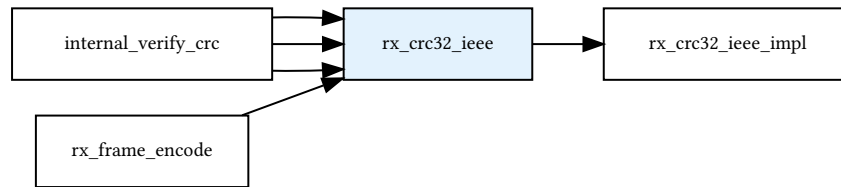


Figure 337: Call graph for rx_crc32_ieee

Defined in `libs/rx_crc/src/rx_crc32.c:610`

rx_crc32_update

```
uint32_t rx_crc32_update(const uint32_t crc, const uint8_t *data, uint32_t len)
```

Update CRC-32 with additional data (incremental/streaming calculation).

Update CRC-32 with additional data (incremental computation).

Continues CRC-32 calculation from a previous CRC value, allowing computation over non-contiguous buffers or streaming data without buffer concatenation.

This function is a facade that delegates to the backend implementation (hardware or software) selected at compile time, just like `rx_crc32_ieee()`.

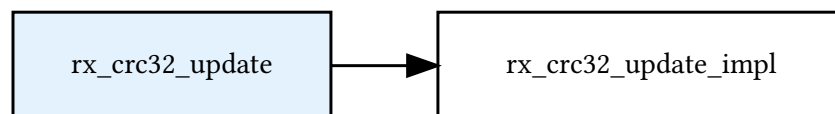


Figure 338: Call graph for rx_crc32_update

Defined in `libs/rx_crc/src/rx_crc32.c:896`

rx_crc32_hw.c

RX72N Hardware CRC Calculator - IEEE 802.3 CRC-32 Acceleration.

Includes:

- `"rx_crc_internal.h"`
- `<stddef.h>`
- `"rx72n_regs.h"`

- "rx_check.h"
- "rx_gpio_constants.h"
- "rx_register_protection.h"

rx_crc_hw_constants_t

CRC hardware constants.

Note: Protection register constants are from rx_register_protection.h

Value	Name	Description
= -1	k_crc_ieee_final_xor	IEEE 802.3 CRC-32 final XOR (0xFFFFFFFF as signed int32_t)

—

rx_crc_hw_loop_constants_t

CRC hardware-specific constants.

Shared loop constants (k_crc_len_min, k_crc_len_max, k_crc_idx_start) are defined in rx_crc_internal.h to ensure consistency across implementations.

Value	Name	Description
= 0	k_crc_result_invalid	Invalid CRC result (error case)
= 1	k_crc_bit_set	Bit set value for register manipulation

—

s_crc_initialized

`bool s_crc_initialized`

—

rx_crc_init

```
rx_err_t rx_crc_init(void)
```

Initialize RX72N CRC Calculator peripheral for IEEE 802.3 CRC-32.

Initialize CRC module (internal implementation).

Initialize CRC backend (hardware peripheral if available).

Enables the hardware CRC module by clearing the module stop bit (MSTPCRB[23]) and configures the peripheral for IEEE 802.3 CRC-32 polynomial with LSB-first bit reflection (compatible with `crc32.ChecksumIEEE()` in Go).

This function is idempotent - calling it multiple times is safe and returns success immediately if already initialized.

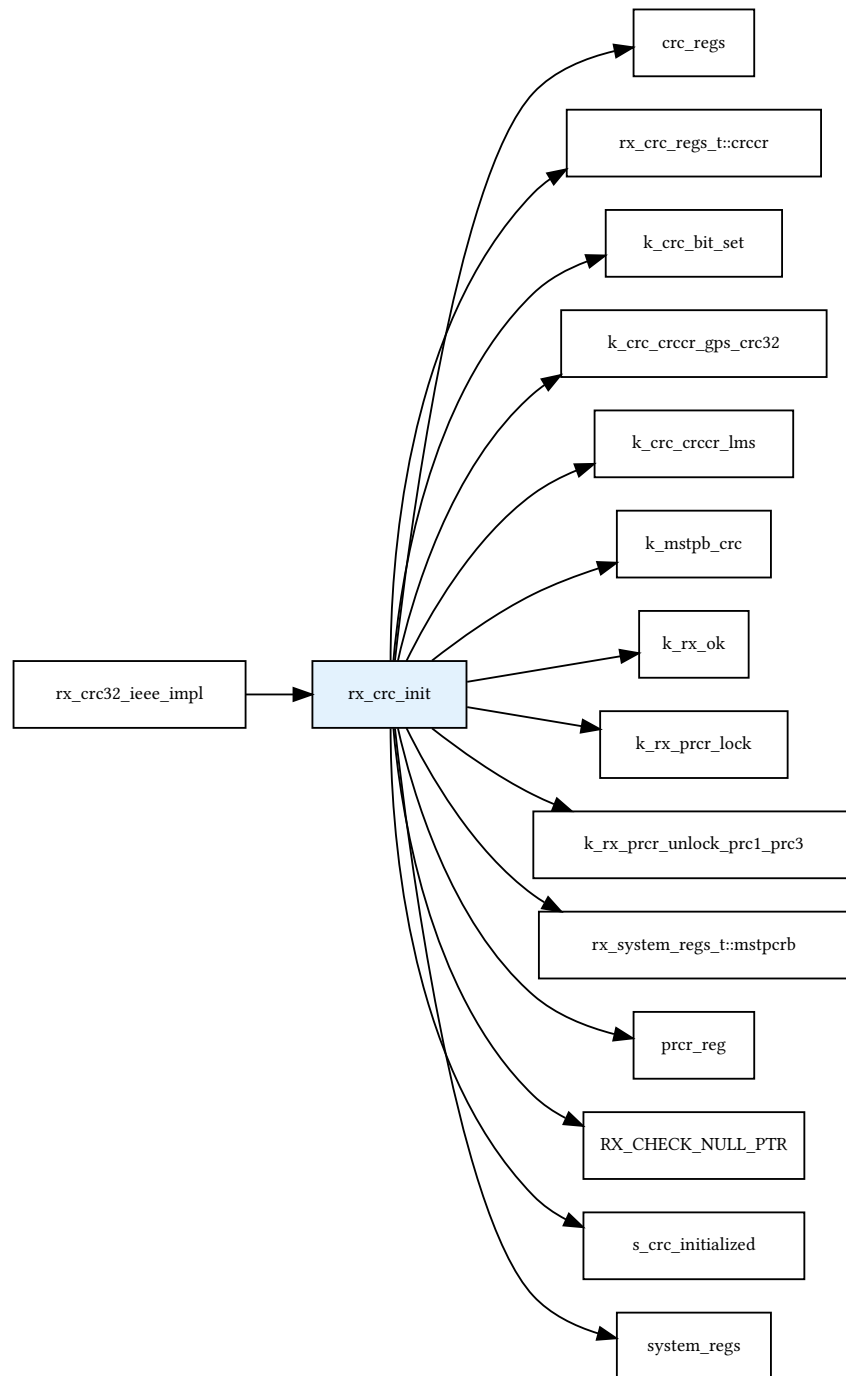


Figure 339: Call graph for rx_crc_init

_Defined in libs/rx_crc/src/rx_crc32_hw.c:512_

rx_crc_deinit

```
rx_err_t rx_crc_deinit(void)
```

Deinitialize hardware CRC module.

Deinitialize CRC module (internal implementation).

Deinitialize CRC backend (hardware peripheral shutdown).

Note: This implementation intentionally does NOT disable the CRC module. The module remains enabled for continuous availability and minimal overhead.

Returns: k_rx_ok always (no-op deinit)

Pre: CRC module must have been initialized

Post: CRC module state unchanged

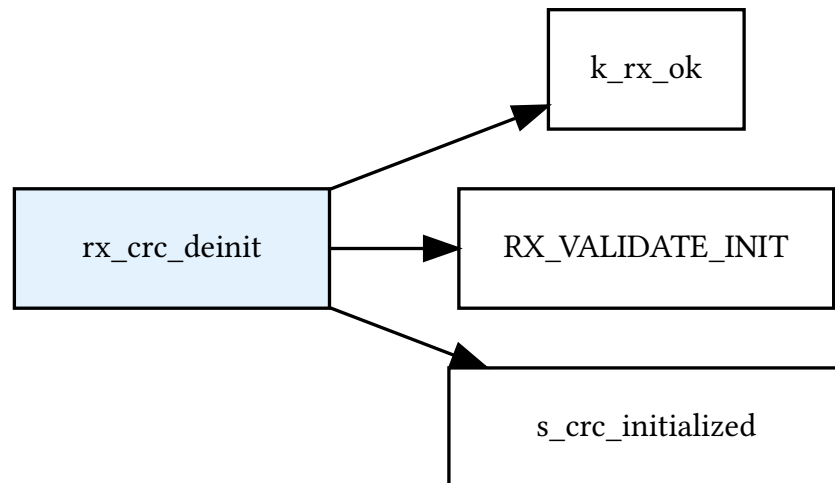


Figure 340: Call graph for rx_crc_deinit

_Defined in libs/rx_crc/src/rx_crc32_hw.c:553_

rx_crc32_ieee_impl

```
uint32_t rx_crc32_ieee_impl(const uint8_t *data, uint32_t len)
```

Calculate IEEE 802.3 CRC-32 using RX72N hardware peripheral (implementation function).

Calculate IEEE 802.3 CRC-32 (internal implementation).

Computes 32-bit Cyclic Redundancy Check using the RX72N CRC Calculator peripheral, providing 5x faster throughput compared to software implementation.

This is the internal implementation called by `rx_crc32_ieee()` in `rx_crc32.c` when hardware backend is selected at compile time.

Result is bit-exact compatible with:

- Go: `crc32.ChecksumIEEE(data)`
- Python: `zlib.crc32(data) & 0xFFFFFFFF`
- C++: `boost::crc_32_type`
- Linux: `cksum -o3` (after byte-swap)

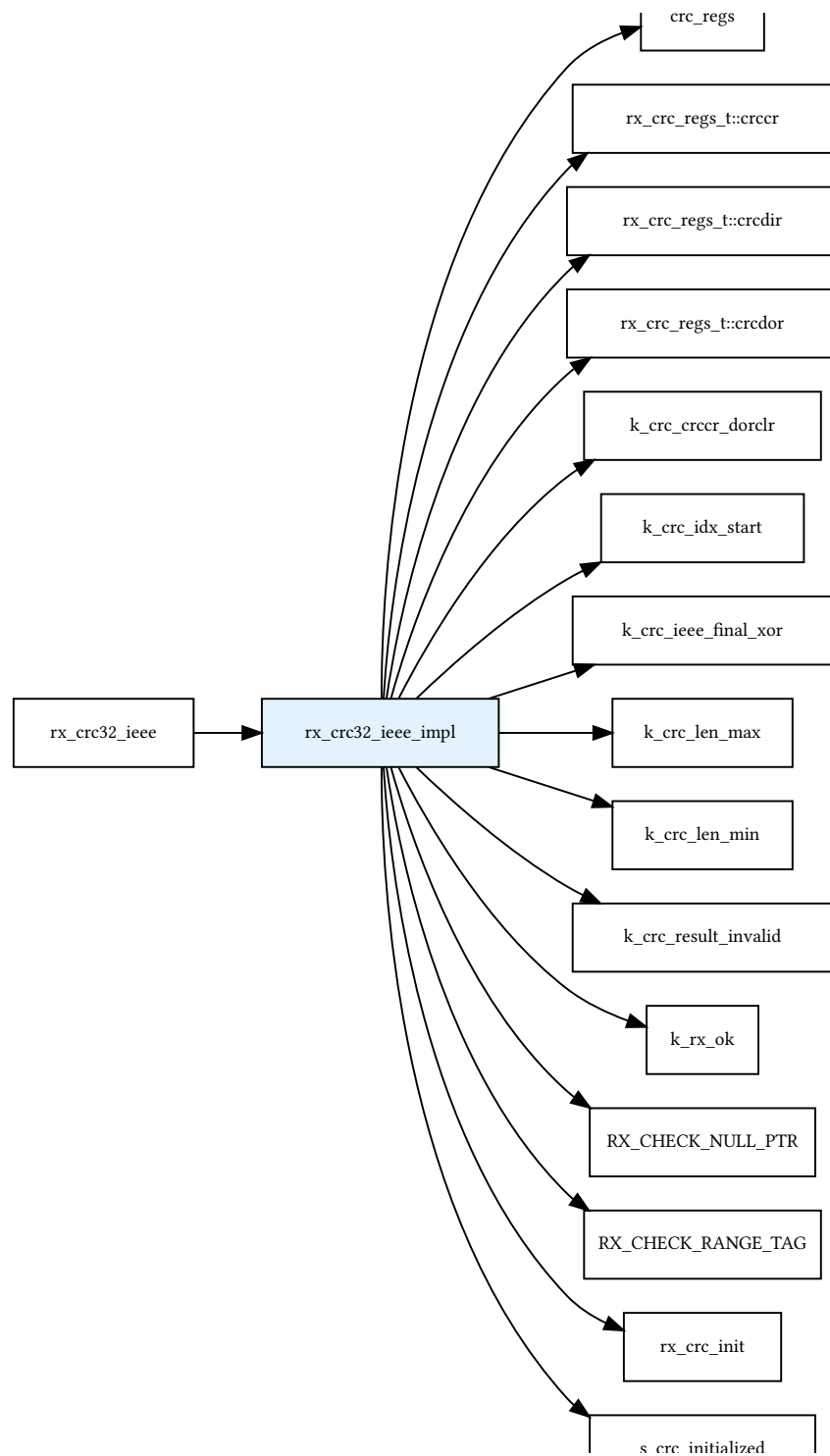


Figure 341: Call graph for rx_crc32_ieee_impl

_Defined in libs/rx_crc/src/rx_crc32_hw.c:829_
 —

rx_crc32_update_impl

```
uint32_t rx_crc32_update_impl(uint32_t crc, const uint8_t *data, uint32_t len)
```

Update CRC-32 with additional data (software fallback).

Update CRC-32 with additional data (internal implementation).

Computes incremental CRC-32 by updating a previous CRC value with new data. Uses software implementation because RX72N hardware doesn't support loading arbitrary initial CRC state.

Parameters:

Direction	Name	Description
in	crc	Previous CRC value to continue from
in	data	Pointer to new data buffer
in	len	Length of new data in bytes

Returns: Updated CRC-32 checksum

Pre: data must be non-NULL

Pre: len must be in valid range

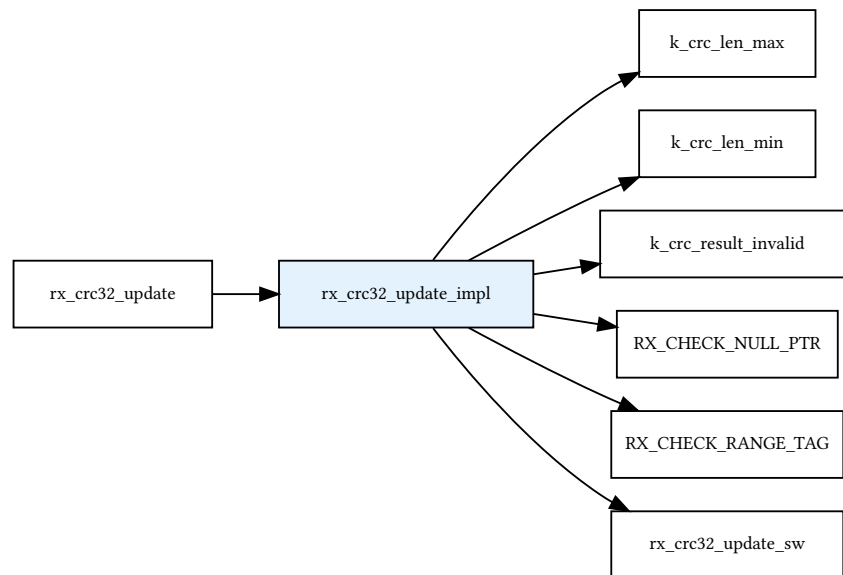


Figure 342: Call graph for rx_crc32_update_impl

_Defined in libs/rx_crc/src/rx_crc32_hw.c:902_
 —

—

rx_crc32_sw.c

Software CRC-32 Implementation - 256-Entry Lookup Table.

Includes:

- <stdint.h>
- "rx_crc_internal.h"

rx_crc32_sw_constants_t

CRC-32 software implementation constants.

Value	Name	Description
= 256	k_crc32_table_size	Number of entries in lookup table
= 8	k_crc_byte_shift	Bit shift for table lookup
= -1	k_crc_ieee_final_xor	IEEE 802.3 CRC-32 final XOR (0xFFFFFFFF as signed int32_t)

—

s_crc32_table`const uint32_t s_crc32_table[k_crc32_table_size]`

Precomputed CRC-32 lookup table (reflected polynomial).

—

rx_crc32_update_sw

```
uint32_t rx_crc32_update_sw(uint32_t crc, const uint8_t *data, uint32_t len)
```

Software CRC-32 update (always available fallback).

Pure software table-based CRC-32 update function. This function is ALWAYS compiled and available, even when hardware CRC is the default, because:

1. Host testing: No hardware available on x86/ARM hosts
2. Incremental CRC: Hardware cannot load arbitrary initial state
3. A/B testing: Compare hardware vs software results for validation

Algorithm (Table Lookup, LSB-first):

Performance Characteristics:

- Time complexity: O(n) where n = len
- Space complexity: O(1) stack + O(1024) lookup table (shared)
- Cache behavior: Good locality (sequential table access)

Lookup table is read-only and never modified

Function has no side effects

Parameters:

Direction	Name	Description
-----------	------	-------------

in	crc	Previous CRC-32 value (finalized form) Valid range: 0x00000000 to 0xFFFFFFFF Initial call: Use result from rx_crc32_ieee() on first buffer Subsequent calls: Use result from previous rx_crc32_update_sw()
in	data	Additional data buffer to include in CRC Valid range: NULL or pointer to valid buffer of size len NULL handling: Returns input crc unchanged (safe no-op) Alignment: No requirement (byte-addressable)
in	len	Number of bytes in data buffer Valid range: 0 to k_crc_len_max (65535) Zero handling: Returns input crc unchanged

Returns: uint32_t Updated CRC-32 checksum Finalized form (ready for comparison or next update)
Range: 0x00000000 to 0xFFFFFFFF

Pre: If data != nullptr, data must point to valid buffer of size len

Pre: len must not exceed k_crc_len_max (NASA Rule 2)

Post: Return value is valid IEEE 802.3 CRC-32

Post: Input crc is unchanged (pure function)

Note: Thread safety: Yes (stateless, read-only global table)

Note: This function is called by rx_crc32_update_impl()

Note: Safe to call from interrupt context (no blocking)

Warning: Do not exceed k_crc_len_max to maintain NASA compliance

Memory Usage:: Stack: 16 bytes (loop counter, temps) Lookup table: 1024 bytes (shared static, in .rodata)

Performance:: 130 us / 100 bytes @ 240 MHz RX72N 50 us / 100 bytes @ 3 GHz x86 Throughput: 0.78 MB/s (RX72N)

Example:: uint8_theader[10]={0xAA,0x55,...}; uint8_tpayload[64]={...};

```
uint32_tcrc=rx_crc32_ieee(header,sizeof(header));
```

```
crc=rx_crc32_update_sw(crc,payload,sizeof(payload));
```

See also: rx_crc32_update() Public API that calls this function, rx_crc32_ieee() Single-shot CRC computation, rx_crc32_sw.c Implementation file

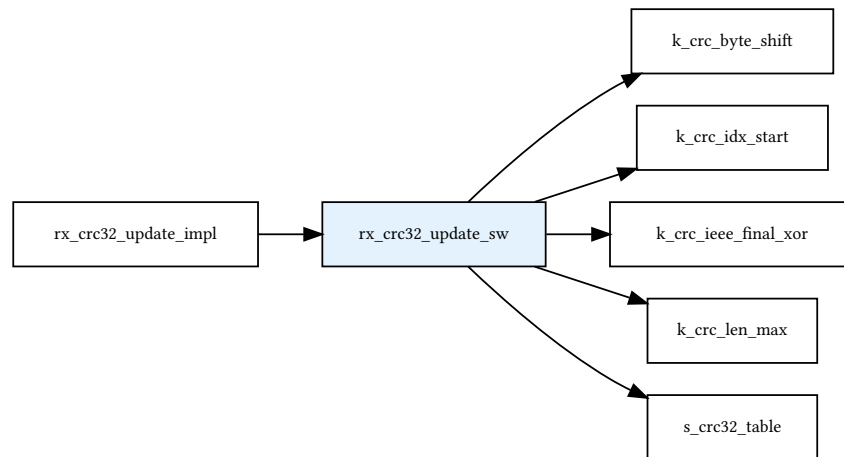


Figure 343: Call graph for rx_crc32_update_sw

_Defined in libs/rx_crc/src/rx_crc32_sw.c:415_

Since Version 1.0.0

—

rx_crc8.c

Dallas/Maxim CRC-8 Implementation for OneWire Device Validation.

Includes:

- <stdint.h>
- "rx_crc.h"

rx_crc8_constants_t

Constants for the Dallas/Maxim CRC-8 algorithm (LSB-first/reflected).

These typed enum constants define all magic numbers used in the CRC-8/Maxim algorithm, ensuring NASA Power of 10 Rule 8 compliance (no magic numbers).

Value	Name	Description
= 0x00	k_crc8_initial_value	CRC-8/Maxim initial value (0x00).
= 0x01	k_crc8_bit_mask	LSB mask for bit extraction (0x01).
= 0x8C	k_crc8_polynomial	Reflected polynomial (0x8C).
= 8	k_crc8_bits_per_byte	Bits per byte for inner loop bound (8).

—

rx_crc8_maxim

```
uint8_t rx_crc8_maxim(const uint8_t *data, uint32_t len)
```

Compute CRC-8/Maxim checksum for Dallas OneWire devices.

Compute CRC-8/Maxim checksum for Dallas/Maxim OneWire devices.

Computes the Dallas/Maxim CRC-8 checksum used by all OneWire devices for ROM code validation and data integrity checking. Uses a bitwise algorithm (no lookup table) optimized for the small payloads typical in OneWire communications.

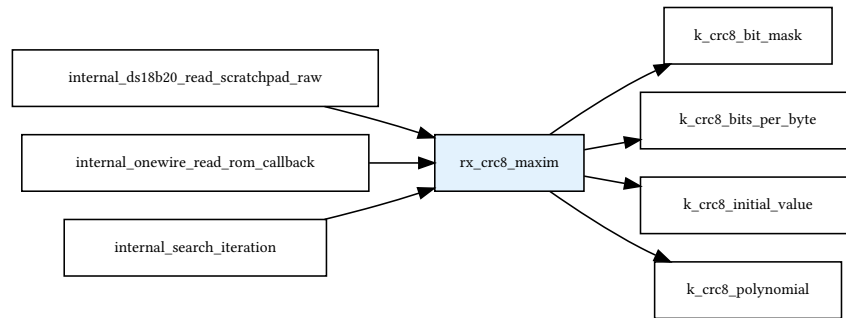


Figure 344: Call graph for rx_crc8_maxim

Defined in `libs/rx_crc/src/rx_crc8.c:434`

rx_ds18b20.h

DS18B20 1-Wire Digital Temperature Sensor Driver API.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_bus_manager.h"`
- `"rx_bus_1wire.h"`
- `"rx_err.h"`

ds18b20_command_t

DS18B20 function commands per datasheet specification.

Command codes sent after ROM selection (Match ROM or Skip ROM) to initiate DS18B20 operations. Each command triggers a specific sensor function.

Protocol Sequence:

1. Reset pulse + presence detection
2. ROM command (Match ROM 0x55 or Skip ROM 0xCC)
3. Function command (one of these values)
4. Additional data bytes (if required by command)

Command Details:

Value	Name	Description
= 0x44	k_ds18b20_cmd_convert_t	Trigger temperature conversion (ADC operation)
= 0x4E	k_ds18b20_cmd_write_scratch	Write to scratchpad memory (TH, TL, Config bytes)
= 0xBE	k_ds18b20_cmd_read_scratch	Read from scratchpad memory (9 bytes: data + CRC)
= 0x48	k_ds18b20_cmd_copy_scratch	Copy scratchpad to EEPROM (non-volatile storage)
= 0xB8	k_ds18b20_cmd_recall_eeprom	Recall EEPROM to scratchpad (restore saved values)

= 0xB4	k_ds18b20_cmd_read_power	Read power supply mode (0=parasitic, 1=external)
--------	--------------------------	--

See also: DS18B20 datasheet section "Function Commands" for timing specifications, rx_bus_onewire.h for ROM command functions

Since Version 1.0.0

—

ds18b20_scratchpad_index_t

DS18B20 scratchpad memory layout indices.

The DS18B20's 9-byte scratchpad RAM contains temperature data, alarm thresholds, configuration, and a CRC-8 checksum. Use these indices to access specific bytes when reading via Read Scratchpad command (0xBE).

Memory Layout:

Temperature Format (Bytes 0-1):

16-bit signed integer in 1/16degC units (two's complement):

- S: Sign bits (all 0 for positive, all 1 for negative)
- T10-T0: Temperature bits (11 bits used, T0-T2 undefined for lower resolutions)
- Value range: -880 to +2000 (represents -55.0degC to +125.0degC)
- Conversion: temp_celsius = raw_value / 16.0

Configuration Register (Byte 4):

Format: 0 R1 R0 1 1 1 1 where R1:R0 = resolution bits

CRC-8 Calculation (Byte 8):

- Polynomial: $x^8 + x^5 + x^4 + 1$ (0x8C)
- Initial value: 0x00
- Data: Bytes 0-7 (inclusive)
- Purpose: Detect transmission errors on 1-Wire bus

Byte 5 (reserved1) is always 0xFF

CRC (byte 8) is valid for bytes 0-7

Temperature bytes (0-1) represent value in range [-880, +2000]

Value	Name	Description
= 0	k_ds18b20_scratch_temp_lsb	Temperature LSB (bits 0-7 of 16-bit value)
= 1	k_ds18b20_scratch_temp_msb	Temperature MSB (sign bits + bits 8-11)
= 2	k_ds18b20_scratch_th_reg	TH (high alarm) register (signed byte, -55 to +125)
= 3	k_ds18b20_scratch_tl_reg	TL (low alarm) register (signed byte, -55 to +125)
= 4	k_ds18b20_scratch_config	Configuration register (resolution bits at [6:5])
= 5	k_ds18b20_scratch_reserved1	Reserved byte (always 0xFF, do not write)
= 6	k_ds18b20_scratch_reserved2	Reserved byte (factory-programmed, do not write)
= 7	k_ds18b20_scratch_reserved3	Reserved byte (factory-programmed, do not write)
= 8	k_ds18b20_scratch_crc	CRC-8 of bytes 0-7 (polynomial 0x8C)

= 9	k_ds18b20_scratchpad_bytes	Total scratchpad size in bytes
-----	----------------------------	--------------------------------

See also: rx_ds18b20_read_scratchpad() Read entire scratchpad with CRC validation, rx_crc8_maxim() CRC-8 calculation function, DS18B20 datasheet section "Memory" for detailed scratchpad specification

Example:

MSB(byte1):SSSSST10T9T8

LSB(byte0):T7T6T5T4T3T2T1T0

+-----+--Temperaturebits(12-bitfor12-bitresolution)

Since Version 1.0.0

—

ds18b20_config_bits_t

DS18B20 configuration register bit positions.

Value	Name	Description
= 5	k_ds18b20_config_r0_bit	Resolution bit 0
= 6	k_ds18b20_config_r1_bit	Resolution bit 1

—

ds18b20_resolution_t

DS18B20 temperature resolution modes.

Value	Name	Description
= 0	k_ds18b20_resolution_9bit	9-bit: 0.5degC, 93.75ms
= 1	k_ds18b20_resolution_10bit	10-bit: 0.25degC, 187.5ms
= 2	k_ds18b20_resolution_11bit	11-bit: 0.125degC, 375ms
= 3	k_ds18b20_resolution_12bit	12-bit: 0.0625degC, 750ms

—

ds18b20_conversion_time_ms_t

DS18B20 conversion times (milliseconds).

Maximum conversion time for each resolution. Add margin for safety (spec values + 10%).

Value	Name	Description
= 100	k_ds18b20_conv_time_9bit_ms	9-bit conversion: 100ms
= 200	k_ds18b20_conv_time_10bit_ms	10-bit conversion: 200ms
= 400	k_ds18b20_conv_time_11bit_ms	11-bit conversion: 400ms
= 800	k_ds18b20_conv_time_12bit_ms	12-bit conversion: 800ms

—

ds18b20_temp_mask_t

DS18B20 temperature masks for resolution.

Lower bits are undefined for resolutions < 12-bit and must be masked. This prevents garbage bits from corrupting the temperature reading.

Value	Name	Description
= 0xFF8	k_ds18b20_temp_mask_9bit	Mask for 9-bit (discard bits 0-2)
= 0xFFC	k_ds18b20_temp_mask_10bit	Mask for 10-bit (discard bits 0-1)
= 0xFFE	k_ds18b20_temp_mask_11bit	Mask for 11-bit (discard bit 0)
= 0xFFF	k_ds18b20_temp_mask_12bit	Mask for 12-bit (all bits valid)

—

ds18b20_conversion_constants_t

DS18B20 temperature conversion constants.

Value	Name	Description
= 4	k_ds18b20_temp_shift	Shift to get integer temperature
= 0x8000	k_ds18b20_sign_bit	Sign bit in 16-bit temperature
= 8	k_ds18b20_crc_bytes	Number of bytes for CRC calculation
= 8	k_ds18b20_shift_byte	Shift for byte positioning
= 3	k_ds18b20_scratchpad_write_bytes	Scratchpad write size (TH, TL, Config)
= -880	k_ds18b20_temp_min_raw	Minimum temperature (-55degC raw value)
= 2000	k_ds18b20_temp_max_raw	Maximum temperature (+125degC raw value)

—

ds18b20_write_idx_t

DS18B20 scratchpad write buffer indices.

Value	Name	Description
= 0	k_ds18b20_write_idx_th	TH (high alarm) register index
= 1	k_ds18b20_write_idx_tl	TL (low alarm) register index
= 2	k_ds18b20_write_idx_config	Configuration register index

—

ds18b20_init_values_t

DS18B20 initialization values.

Value	Name	Description
= 0	k_ds18b20_config_register_cleared	Cleared config register value

—

ds18b20_power_mode_t

DS18B20 power supply modes.

Value	Name	Description
= 0	k_ds18b20_power_parasitic	Parasitic power mode
= 1	k_ds18b20_power_external	External power mode

—

rx_ds18b20_init

```
rx_err_t rx_ds18b20_init(rx_ds18b20_handle_t *handle, const rx_ds18b20_config_t *config)
```

Initialize DS18B20 sensor.

Initializes the DS18B20 sensor and configures resolution. If ROM matching is disabled, uses Skip ROM (single device mode).

Parameters:

Direction	Name	Description
out	handle	Pointer to DS18B20 handle. Must not be nullptr.
in	config	Pointer to configuration. Must not be nullptr.

Returns: k_rx_err_crc if scratchpad CRC check fails



Figure 345: Call graph for `rx_ds18b20_init`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:730`

—

rx_ds18b20_deinit

```
rx_err_t rx_ds18b20_deinit(rx_ds18b20_handle_t *handle)
```

Deinitialize DS18B20 sensor.

Deinitialize DS18B20 sensor.

Tears down a previously initialized DS18B20 handle by zeroing all internal state via `memset`. After this call the handle is safe to reinitialize with `rx_ds18b20_init()`.

Algorithm steps:

1. Validate handle is not nullptr
2. Verify handle is currently initialized
3. Clear the entire handle structure with `memset` (zeros all fields)
4. Log deinitialization success

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
inout	handle	Sensor handle to deinitialize (must be initialized, non-null)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Handle successfully cleared
<code>k_rx_err_null_ptr</code>	handle pointer is nullptr
<code>k_rx_err_invalid_state</code>	handle was not previously initialized

Pre: handle must be a valid non-null pointer

Pre: handle must have been successfully initialized via `rx_ds18b20_init()`

Post: All handle fields are zeroed (initialized flag is false)

Post: handle may be safely reused with `rx_ds18b20_init()`

Note: Not thread-safe; caller must ensure no concurrent access to handle

Example:: `rx_ds18b20_handle_t sensor; rx_ds18b20_init(&sensor,&config);
rx_err_t err=rx_ds18b20_deinit(&sensor); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to deinit DS18B20"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: `rx_ds18b20_init()` Initialize the handle before use

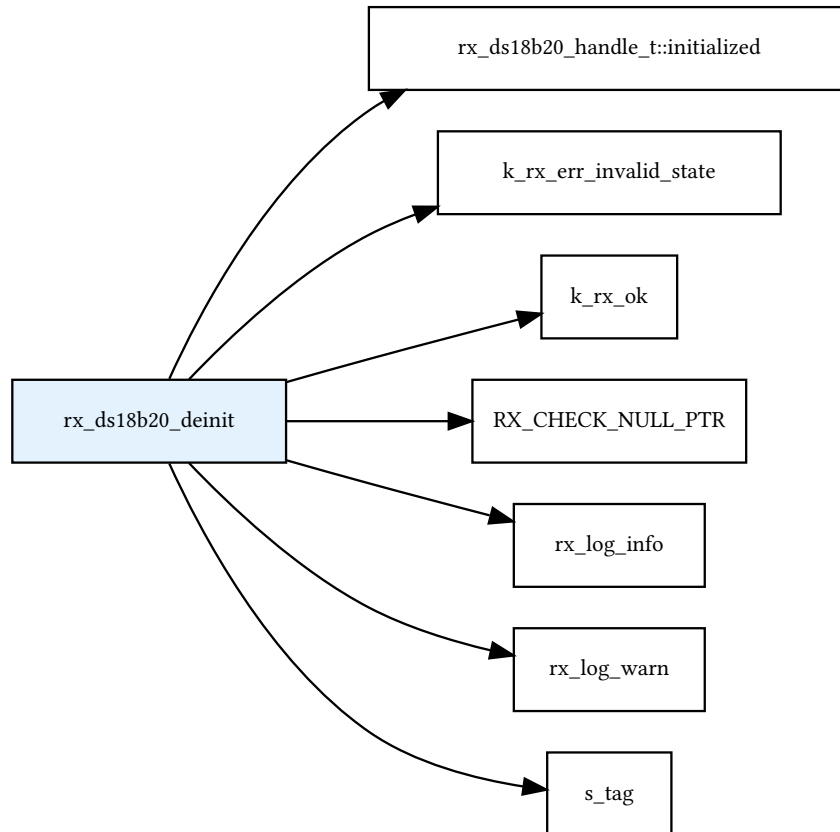


Figure 346: Call graph for `rx_ds18b20_deinit`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:742`

Since Version 1.0.0

—

rx_ds18b20_trigger_conversion

```
rx_err_t rx_ds18b20_trigger_conversion(const rx_ds18b20_handle_t *handle)
```

Trigger temperature conversion.

Sends Convert T command to start temperature measurement. Caller must wait for conversion time before reading temperature.

Conversion times:

- 9-bit: 100ms
- 10-bit: 200ms

- 11-bit: 400ms
- 12-bit: 800ms

Trigger temperature conversion.

Issues the Convert T (0x44) command over the 1-Wire bus to start an autonomous temperature measurement. The DS18B20 will perform the ADC conversion internally; the caller must wait the appropriate conversion time before reading results with `rx_ds18b20_read_temperature()`.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Convert T command byte (0x44) to the bus

Conversion time depends on resolution:

- 9-bit: 94 ms
- 10-bit: 188 ms
- 11-bit: 375 ms
- 12-bit: 750 ms

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
in	handle	Sensor handle (must be initialized, non-null)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Conversion command issued successfully
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized or device not present on bus
<code>k_rx_err_bus_error</code>	1-Wire bus reset or write operation failed

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: DS18B20 device must be physically connected and powered on the bus

Post: DS18B20 ADC conversion is in progress

Post: Caller must wait `rx_ds18b20_get_conversion_time_ms()` before reading

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Do not call `rx_ds18b20_read_temperature()` before the conversion time elapses

Example:: rx_err_t rx_ds18b20_trigger_conversion(&sensor); if(err!=k_rx_ok)
 { tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor)); floattemp_celsius=0.0f;
 err=rx_ds18b20_read_temperature(&sensor,&temp_celsius); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions

See also: rx_ds18b20_read_temperature() Read converted temperature in Celsius,
 rx_ds18b20_get_conversion_time_ms() Get required wait time after conversion,
 rx_ds18b20_set_resolution() Configure ADC resolution

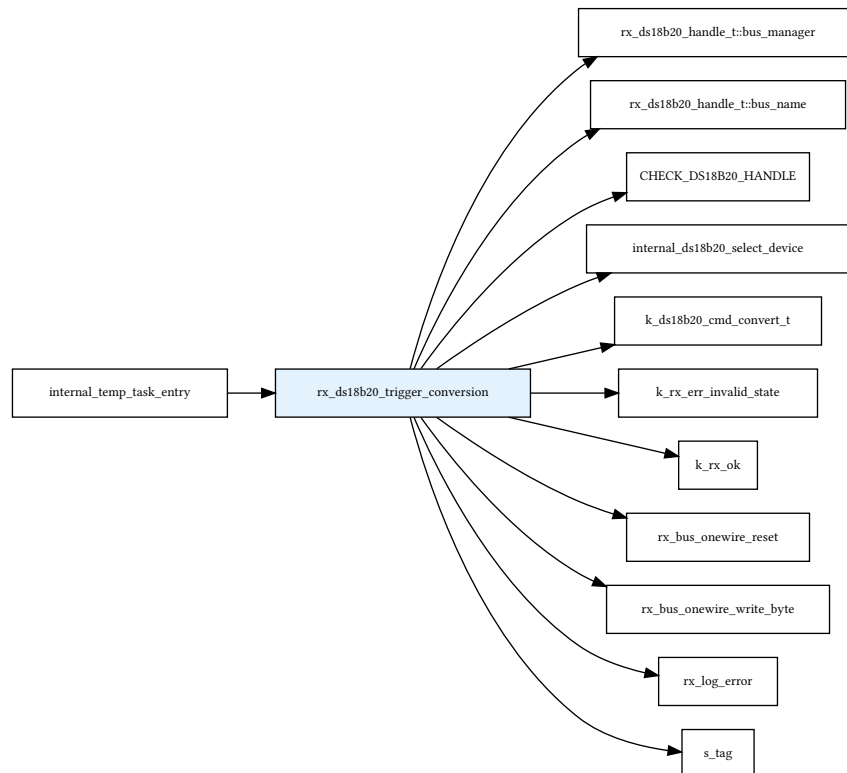


Figure 347: Call graph for rx_ds18b20_trigger_conversion

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:762`

Since Version 1.0.0

rx_ds18b20_read_temperature

```
rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t *handle, float
*temperature_celsius)
```

Read temperature in Celsius.

Reads the scratchpad and converts raw temperature to Celsius. Temperature conversion must complete before calling (see conversion times).

Read temperature in Celsius.

Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw temperature value, applies the resolution mask to clear undefined low-order bits, validates the result is within the sensor's physical range, and converts to a floating-point Celsius value using the DS18B20 fixed-point encoding (1 LSB = 1/16 degC).

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Call `rx_ds18b20_read_temperature_raw()` to obtain masked raw value
3. Convert raw signed 16-bit value to float: $\text{celsius} = \text{raw} / 16.0f$
4. Write result to `*temperature_celsius`

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
out	temperature_celsius	Pointer to store temperature in degC. Must not be nullptr.
inout	handle	Sensor handle (must be initialized)
out	temperature_celsius	Converted temperature in degrees Celsius, valid range [-55.0, +125.0]; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Temperature read and converted successfully
<code>k_rx_err_null_ptr</code>	handle or temperature_celsius is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
<code>k_rx_err_out_of_range</code>	raw temperature outside [-55degC, +125degC]
<code>k_rx_err_crc_failure</code>	Scratchpad CRC mismatch
<code>k_rx_err_bus_error</code>	1-Wire bus communication failure

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: `rx_ds18b20_trigger_conversion()` must have been called and conversion time elapsed

Post: `*temperature_celsius` contains valid temperature in -55.0, +125.0 on `k_rx_ok`

Post: `handle->stats.read_count` incremented (via `internal_ds18b20_read_scratchpad_raw`)

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Returns stale data if conversion time has not elapsed after trigger

Example:: floattemp_celsius=0.0f;
 rx_err_t err=rx_ds18b20_read_temperature(&sensor,&temp_celsius); if(err==k_rx_ok)
 { rx_log_info("app","Temperature:%.4fC",(double)temp_celsius); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_ds18b20_trigger_conversion() Must be called before reading,
 rx_ds18b20_read_temperature_raw() Read the raw 16-bit value,
 rx_ds18b20_get_conversion_time_ms() Determine required wait time

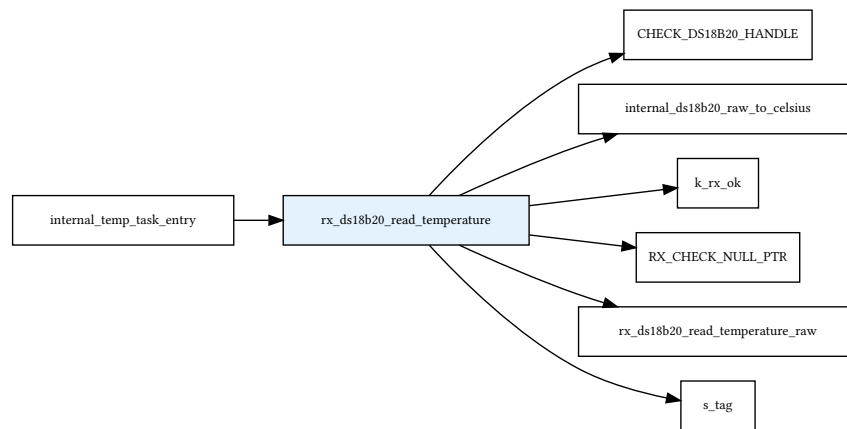


Figure 348: Call graph for rx_ds18b20_read_temperature

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:778`

Since Version 1.0.0

—

rx_ds18b20_read_temperature_raw

```

rx_err_t rx_ds18b20_read_temperature_raw(rx_ds18b20_handle_t *handle, int16_t
*raw_temp)

```

Read raw temperature value.

Reads the raw 16-bit temperature from scratchpad without conversion. Useful for debugging or custom temperature processing.

Read raw temperature value.

Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit unsigned value, applies the resolution-dependent mask to clear undefined low-order bits, casts to signed `int16_t`, and range-validates the result.

The DS18B20 temperature encoding uses a signed fixed-point format where 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:

- Minimum: -55 degC = 0xFC90 (int16: -880)

- Maximum: +125 degC = 0x07D0 (int16: +2000)

Resolution masks (clears undefined bits in lower-resolution modes):

- 9-bit: 0xFFF8 (3 undefined bits)
- 10-bit: 0xFFFC (2 undefined bits)
- 11-bit: 0xFFFE (1 undefined bit)
- 12-bit: 0xFFFF (no undefined bits)

Algorithm steps:

1. Validate handle and output pointer are non-null and handle is initialized
2. Read raw scratchpad bytes via `internal_ds18b20_read_scratchpad_raw()`
3. Combine temperature LSB and MSB bytes into `uint16_t`
4. Apply resolution mask from `internal_ds18b20_get_temp_mask()`
5. Cast to `int16_t` and validate range [`k_ds18b20_temp_min_raw`, `k_ds18b20_temp_max_raw`]
6. Write validated result to `*raw_temp`

Parameters:

Direction	Name	Description
in	<code>handle</code>	Pointer to initialized handle. Must not be nullptr.
out	<code>raw_temp</code>	Pointer to store raw temperature. Must not be nullptr.
inout	<code>handle</code>	Sensor handle (must be initialized)
out	<code>raw_temp</code>	Signed 16-bit raw temperature in 1/16 degC units, valid range [<code>k_ds18b20_temp_min_raw</code> , <code>k_ds18b20_temp_max_raw</code>]; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Raw temperature read, masked, and validated successfully
<code>k_rx_err_null_ptr</code>	<code>handle</code> or <code>raw_temp</code> is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
<code>k_rx_err_out_of_range</code>	raw value outside physical sensor range
<code>k_rx_err_crc_failure</code>	Scratchpad CRC check failed
<code>k_rx_err_bus_error</code>	1-Wire bus communication failure

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: `rx_ds18b20_trigger_conversion()` must have been called and conversion time elapsed

Post: `*raw_temp` is in `k_ds18b20_temp_min_raw`, `k_ds18b20_temp_max_raw` on `k_rx_ok`

Post: Resolution mask has been applied; undefined bits are cleared

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Returns stale data if conversion time has not elapsed after trigger

Example:: `int16_t raw=0; rx_err_t err=rx_ds18b20_read_temperature_raw(&sensor,&raw);
if(err!=k_rx_ok){ float celsius=(float)raw/16.0f; }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)

See also: `rx_ds18b20_read_temperature()` Convenience wrapper returning float Celsius,
`rx_ds18b20_trigger_conversion()` Must be called before reading,
`rx_ds18b20_set_resolution()` Configure ADC resolution and mask

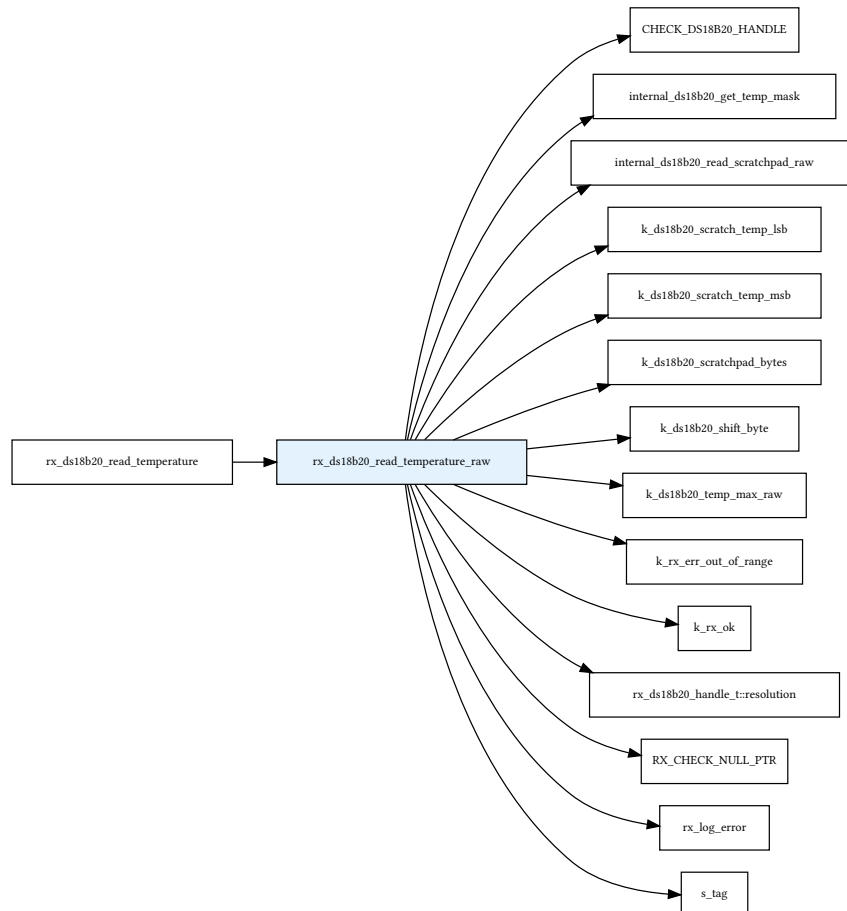


Figure 349: Call graph for `rx_ds18b20_read_temperature_raw`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:795`

Since Version 1.0.0

rx_ds18b20_set_resolution

```
rx_err_t rx_ds18b20_set_resolution(rx_ds18b20_handle_t *handle,
ds18b20_resolution_t resolution)
```

Set temperature resolution.

Changes the sensor resolution (9, 10, 11, or 12 bits). Higher resolution provides more precision but takes longer to convert.

Set temperature resolution.

Updates the configuration register in the DS18B20 scratchpad to change the ADC resolution. The existing TH and TL alarm threshold bytes are preserved by reading the current scratchpad first, then writing back only the updated configuration byte.

Higher resolution provides finer temperature steps but increases conversion time:

- 9-bit: 0.5 degC steps, 94 ms conversion
- 10-bit: 0.25 degC steps, 188 ms conversion
- 11-bit: 0.125 degC steps, 375 ms conversion
- 12-bit: 0.0625 degC steps, 750 ms conversion

Algorithm steps:

1. Validate handle is initialized and resolution is a legal enum value
2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
3. Convert resolution enum to configuration register byte via `internal_ds18b20_resolution_to_config()`
4. Write scratchpad with new config byte (TH/TL unchanged)
5. Update handle->resolution to reflect new setting

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
in	resolution	Resolution mode (9-12 bits)
inout	handle	Sensor handle (must be initialized); resolution field updated on success
in	resolution	New resolution setting (k_ds18b20_resolution_9bit through k_ds18b20_resolution_12bit); all other values are invalid

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Resolution updated in scratchpad and handle
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_invalid_arg	resolution is not a valid ds18b20_resolution_t value
k_rx_err_crc_failure	Scratchpad read CRC check failed
k_rx_err_bus_error	1-Wire bus communication failure during read or write

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: resolution must be one of `k_ds18b20_resolution_9bit/10bit/11bit/12bit`

Post: handle->resolution reflects the newly configured value on `k_rx_ok`

Post: DS18B20 scratchpad configuration register updated with new resolution bits

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Change is not persistent across power cycles unless `rx_ds18b20_save_config()` is called

Example:: `rx_err_t err=rx_ds18b20_set_resolution(&sensor,k_ds18b20_resolution_12bit);`
`if(err!=k_rx_ok){ rx_ds18b20_save_config(&sensor); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions

See also: `rx_ds18b20_get_resolution()` Read back the currently configured resolution,
`rx_ds18b20_save_config()` Persist resolution change to EEPROM,
`rx_ds18b20_get_conversion_time_ms()` Get required conversion time for new resolution

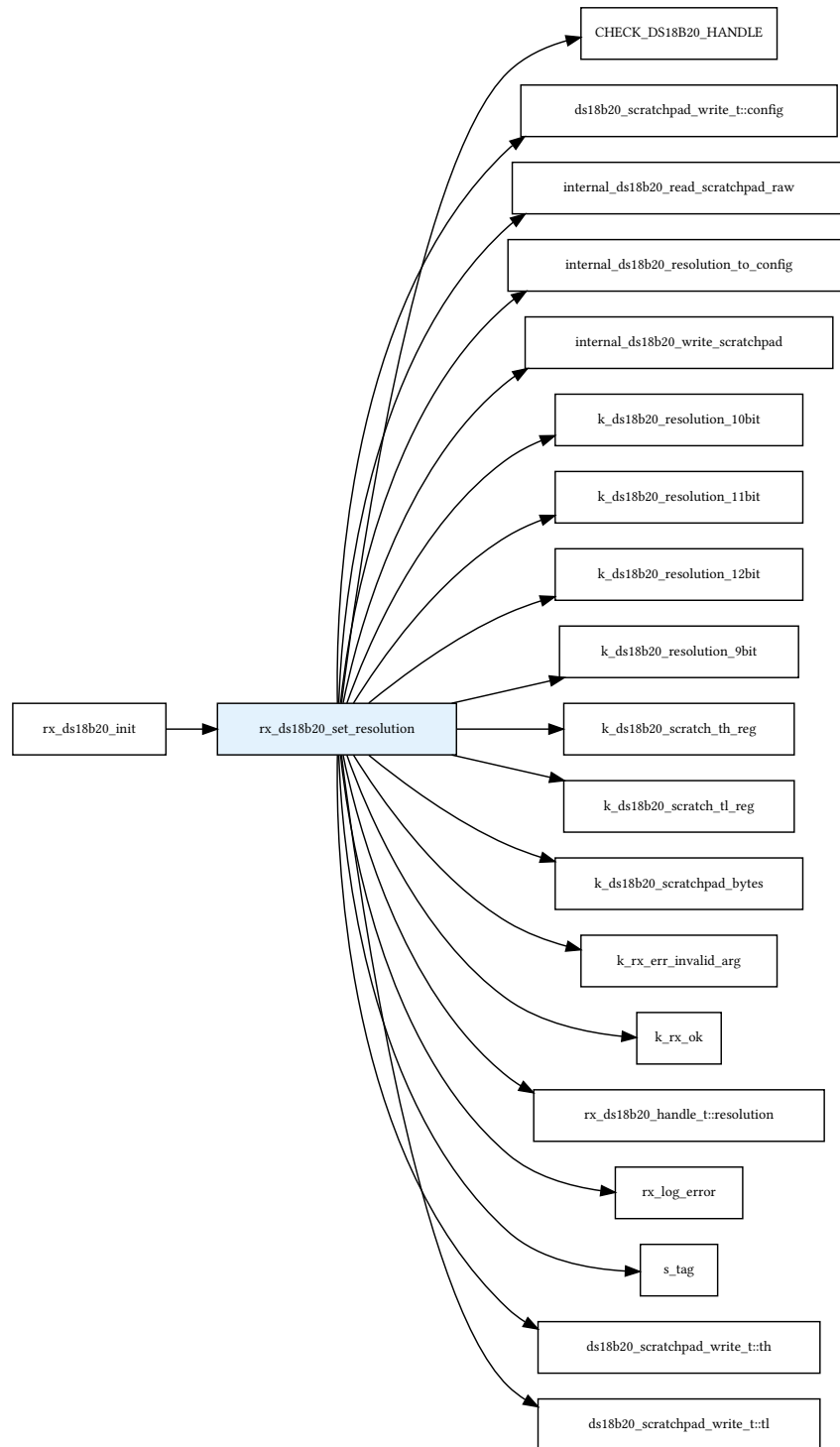


Figure 350: Call graph for `rx_ds18b20_set_resolution`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:812`

Since Version 1.0.0

—

rx_ds18b20_get_resolution

```
rx_err_t rx_ds18b20_get_resolution(const rx_ds18b20_handle_t *handle,  
ds18b20_resolution_t *resolution)
```

Get current temperature resolution.

Get current temperature resolution.

Returns the resolution value stored in the handle, which reflects the last successfully applied resolution (set during initialization or via `rx_ds18b20_set_resolution()`). This is a cached read from the handle state; it does not issue a 1-Wire bus transaction.

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Verify handle is initialized
3. Copy handle->resolution to *resolution

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
out	resolution	Pointer to store resolution. Must not be nullptr.
in	handle	Sensor handle (must be initialized, non-null)
out	resolution	Receives the currently configured <code>ds18b20_resolution_t</code> value; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Resolution successfully returned
<code>k_rx_err_null_ptr</code>	handle or resolution pointer is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized (<code>CHECK_DS18B20_HANDLE</code> fails)

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: resolution must be a valid non-null pointer

Post: *resolution contains the current resolution value on `k_rx_ok`

Post: handle state is unchanged

Note: Not thread-safe; caller must ensure handle is not concurrently modified

Note: Returns cached value from handle does not issue 1-Wire bus transaction

Example:: ds18b20_resolution_tres; rx_err_t err=rx_ds18b20_get_resolution(&sensor,&res);
 if(err!=k_rx_ok){ uint32_t conv_ms=rx_ds18b20_get_conversion_time_ms(&sensor);
 rx_log_info("app","Resolution%d,convtime%ums",(int)res,(unsigned)conv_ms); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_ds18b20_set_resolution() Change the ADC resolution,
 rx_ds18b20_get_conversion_time_ms() Get conversion time for current resolution

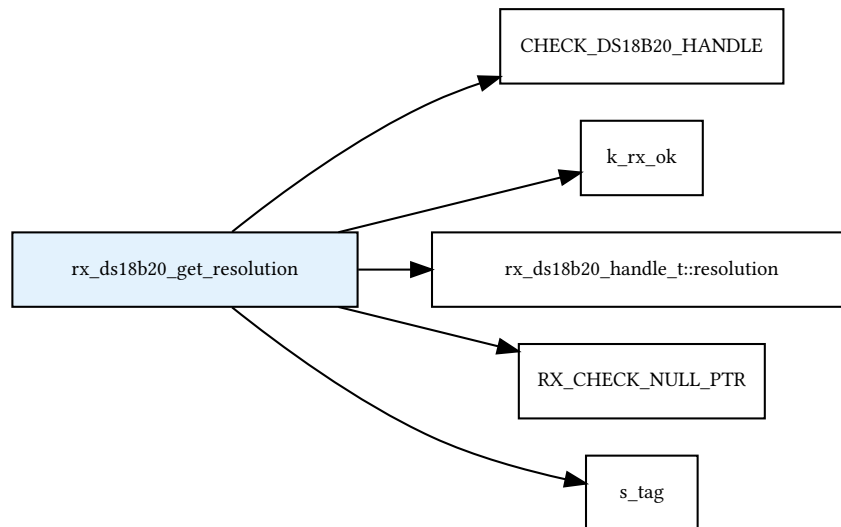


Figure 351: Call graph for rx_ds18b20_get_resolution

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:825`

Since Version 1.0.0

—

rx_ds18b20_save_config

```
rx_err_t rx_ds18b20_save_config(const rx_ds18b20_handle_t *handle)
```

Save configuration to EEPROM.

Copies scratchpad bytes 2-4 (TH, TL, Config) to non-volatile EEPROM. Values persist across power cycles.

Save configuration to EEPROM.

Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from volatile scratchpad RAM to non-

volatile EEPROM. The saved values persist across power cycles and are automatically restored on the next power-up via the Recall E2 sequence.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Copy Scratchpad command byte (0x48)

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
in	handle	Sensor handle (must be initialized, non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Copy Scratchpad command successfully issued
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset or write operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected and externally powered (Copy Scratchpad is not guaranteed in parasitic power mode)

Post: DS18B20 EEPROM contains current TH, TL, and configuration register values

Post: Configuration will survive next power cycle

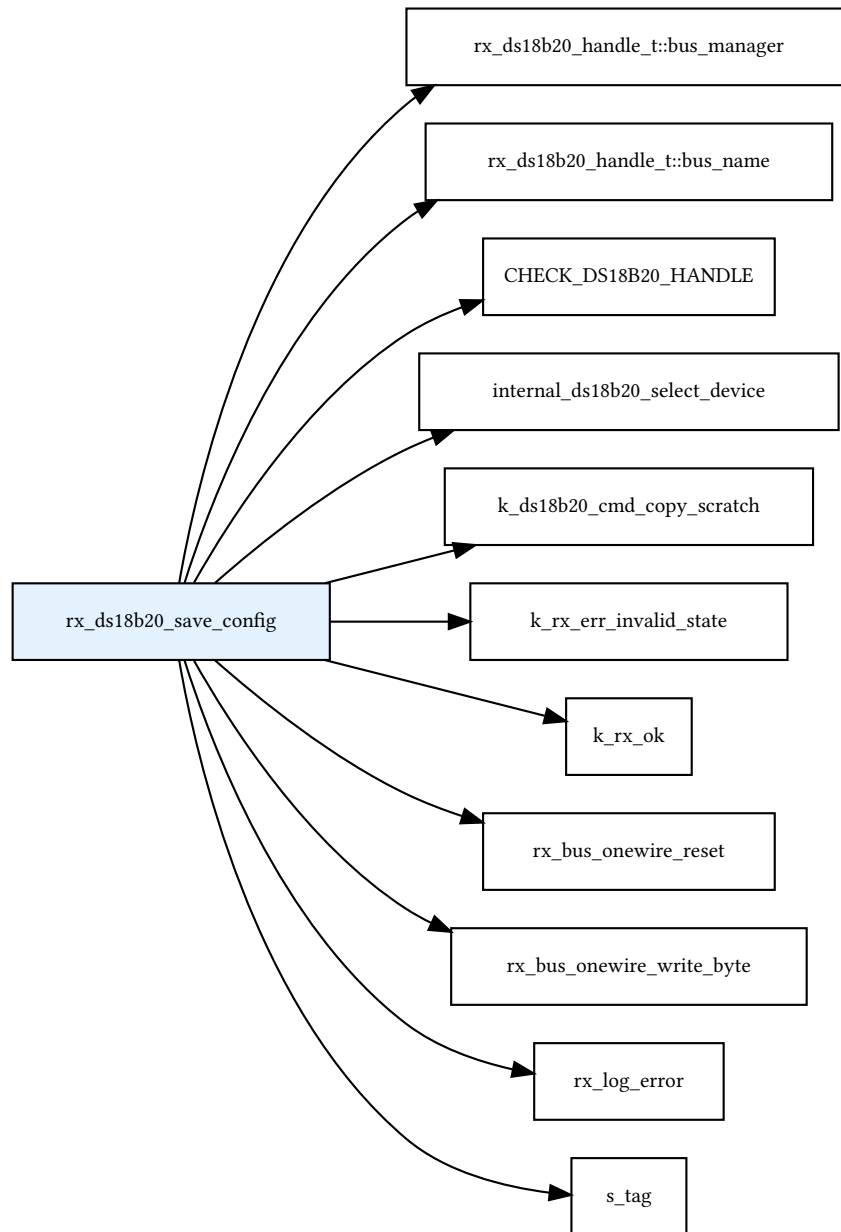
Note: Not thread-safe; caller must serialize access to the same handle

Warning: Not reliable in parasitic power mode; use external power for EEPROM writes

```
Example:: rx_ds18b20_set_resolution(&sensor,k_ds18b20_resolution_12bit);
rx_err_t err=rx_ds18b20_save_config(&sensor); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to save DS18B20 config to EEPROM"); }
```

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

See also: rx_ds18b20_recall_config() Restore EEPROM contents to scratchpad,
rx_ds18b20_set_resolution() Configure resolution before saving

Figure 352: Call graph for `rx_ds18b20_save_config`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:840`

Since Version 1.0.0

—

rx_ds18b20_recall_config

```
rx_err_t rx_ds18b20_recall_config(const rx_ds18b20_handle_t *handle)
```

Recall configuration from EEPROM.

Restores scratchpad bytes 2-4 (TH, TL, Config) from EEPROM. Useful after power-up to restore saved settings.

Recall configuration from EEPROM.

Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from non-volatile EEPROM back into volatile scratchpad RAM. This is the same operation the device performs automatically on power-up.

Use this function to discard unsaved scratchpad changes and revert to the last saved EEPROM state without cycling power.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Recall E2 command byte (0xB8)

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
in	handle	Sensor handle (must be initialized, non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Recall E2 command successfully issued
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset or write operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected to the bus

Post: DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save

Post: Any unsaved scratchpad changes since last rx_ds18b20_save_config() are discarded

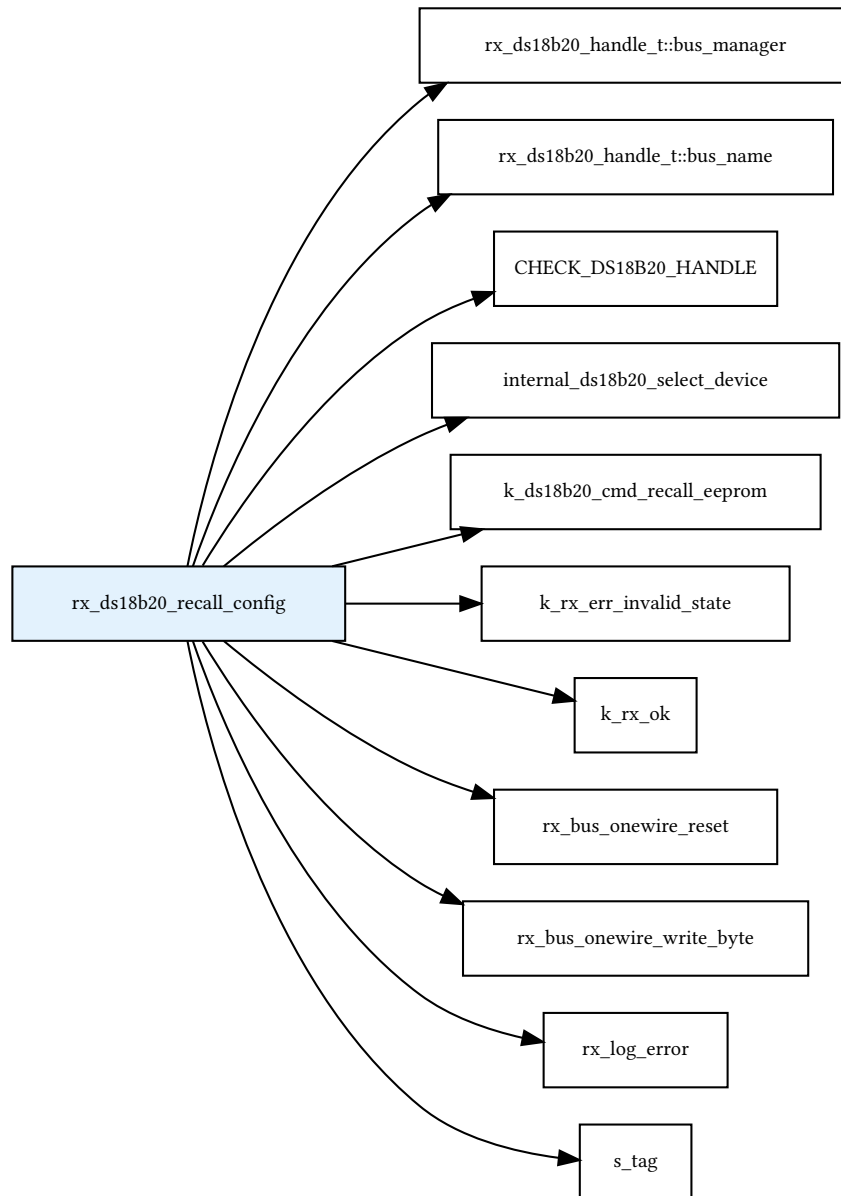
Note: Not thread-safe; caller must serialize access to the same handle

Note: The Recall E2 operation is performed automatically by the device on power-up

Example:: `rx_err_t err = rx_ds18b20_recall_config(&sensor); if(err != k_rx_ok)
{ rx_log_error("app", "Failed to recall DS18B20 config from EEPROM"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

See also: `rx_ds18b20_save_config()` Persist scratchpad configuration to EEPROM,
`rx_ds18b20_set_resolution()` Modify the scratchpad configuration

Figure 353: Call graph for `rx_ds18b20_recall_config`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:854`

Since Version 1.0.0

—

rx_ds18b20_read_power_mode

```
rx_err_t rx_ds18b20_read_power_mode(const rx_ds18b20_handle_t *handle, bool
*external_power)
```

Read power supply mode.

Checks if sensor is using parasitic power or external power.

Read power supply mode.

Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads back a single bit to determine whether the DS18B20 is operating on an external voltage supply or in parasitic (bus-powered) mode.

Per the DS18B20 datasheet: the sensor pulls the bus low for one read time slot to indicate parasitic power mode (0), or releases the bus (1) to indicate external power mode.

Algorithm steps:

1. Validate handle and output pointer are non-null; handle is initialized
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Read Power Supply command byte (0xB4)
5. Read one bit from the bus (0 = parasitic, 1 = external)
6. Write result to *external_power

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
out	external_power	True if external power, false if parasitic. Must not be nullptr.
in	handle	Sensor handle (must be initialized, non-null)
out	external_power	Set to true if device uses external power supply, false if device uses parasitic (bus-powered) mode; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Power mode successfully read
k_rx_err_null_ptr	handle or external_power pointer is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset, write, or read operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected to the bus

Post: *external_power reflects the device power supply mode on k_rx_ok

Post: handle state is unchanged

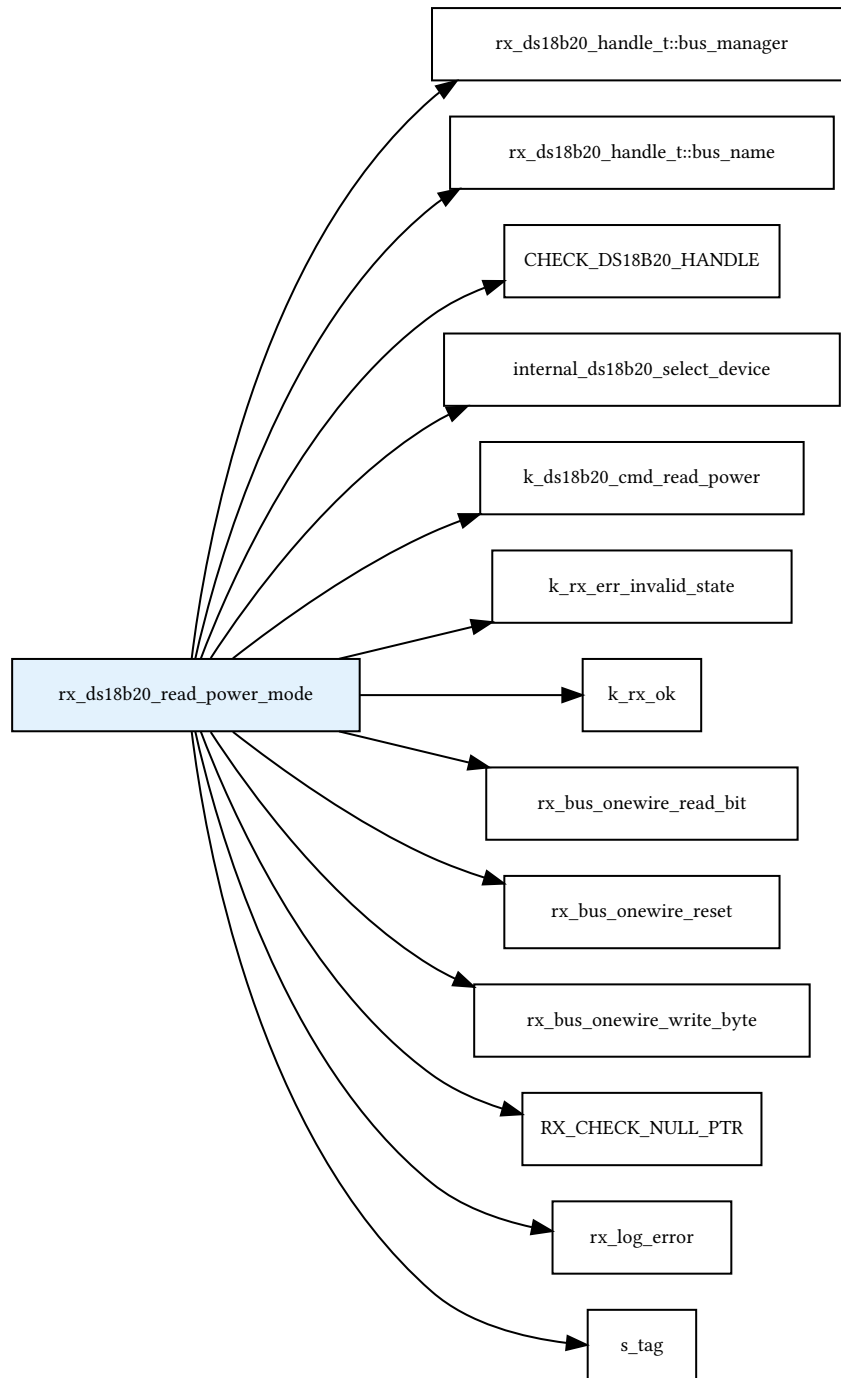
Note: Not thread-safe; caller must serialize access to the same handle

Warning: Some operations (Copy Scratchpad) are unreliable in parasitic power mode

Example:: `boolext_power=false; rx_err_t err=rx_ds18b20_read_power_mode(&sensor,&ext_power);
if(err!=k_rx_ok&&!ext_power)
{ rx_log_warn("app","DS18B20inparasiticmode:EEPROMwritesmayfail"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions

See also: `rx_ds18b20_save_config()` EEPROM write (requires external power),
`rx_ds18b20_init()` Initialize sensor handle

Figure 354: Call graph for `rx_ds18b20_read_power_mode`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:868`

Since Version 1.0.0

—

rx_ds18b20_read_scratchpad

```
rx_err_t rx_ds18b20_read_scratchpad(rx_ds18b20_handle_t *handle, uint8_t
scratchpad[k_ds18b20_scratchpad_bytes])
```

Read scratchpad memory.

Reads the entire 9-byte scratchpad (8 data bytes + 1 CRC byte). Performs CRC validation.

Read scratchpad memory.

Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw buffer to the caller. The scratchpad layout is:

CRC is verified internally by `internal_ds18b20_read_scratchpad_raw()` before the buffer is returned. This function is a thin public wrapper around the internal helper.

Algorithm steps:

1. Validate handle and scratchpad buffer pointer are non-null
2. Verify handle is initialized
3. Delegate to `internal_ds18b20_read_scratchpad_raw()` which issues the 1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command, reads 9 bytes, and verifies CRC-8

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
out	scratchpad	Pointer to 9-byte buffer. Must not be nullptr.
inout	handle	Sensor handle (must be initialized, non-null)
out	scratchpad	Buffer of exactly <code>k_ds18b20_scratchpad_bytes</code> (9) bytes to receive the raw scratchpad contents; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Scratchpad successfully read and CRC verified
<code>k_rx_err_null_ptr</code>	handle or scratchpad buffer is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
<code>k_rx_err_crc_failure</code>	Scratchpad CRC-8 check failed (data corrupted)
<code>k_rx_err_bus_error</code>	1-Wire bus communication failure

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: scratchpad must point to a buffer of at least `k_ds18b20_scratchpad_bytes` bytes

Post: scratchpad0..8 contains valid data on k_rx_ok

Post: CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok

Note: Not thread-safe; caller must serialize access to the same handle

Note: Use rx_ds18b20_read_temperature() for converted Celsius value

Example:: uint8_t raw[k_ds18b20_scratchpad_bytes];
 rx_err_t err=rx_ds18b20_read_scratchpad(&sensor,raw); if(err==k_rx_ok)
 { uint8_t config_reg=raw[4]; }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_ds18b20_read_temperature() High-level temperature read returning float Celsius, rx_ds18b20_read_temperature_raw() Read raw 16-bit temperature with masking

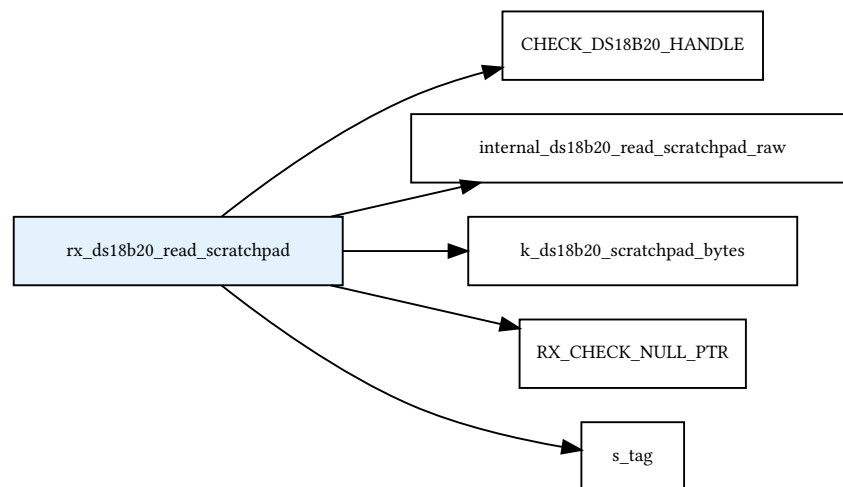


Figure 355: Call graph for rx_ds18b20_read_scratchpad

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:885`

Since Version 1.0.0

rx_ds18b20_get_conversion_time_ms

```
uint32_t rx_ds18b20_get_conversion_time_ms(const rx_ds18b20_handle_t *handle)
```

Get conversion time for current resolution.

Returns the maximum conversion time in milliseconds for the configured resolution. Caller should wait this long after triggering conversion before reading temperature.

Get conversion time for current resolution.

Returns the required wait time after triggering conversion based on the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized handle. Must not be nullptr.
in	handle	Sensor handle

Returns: Conversion time in milliseconds, or invalid value if handle is nullptr/uninitialized

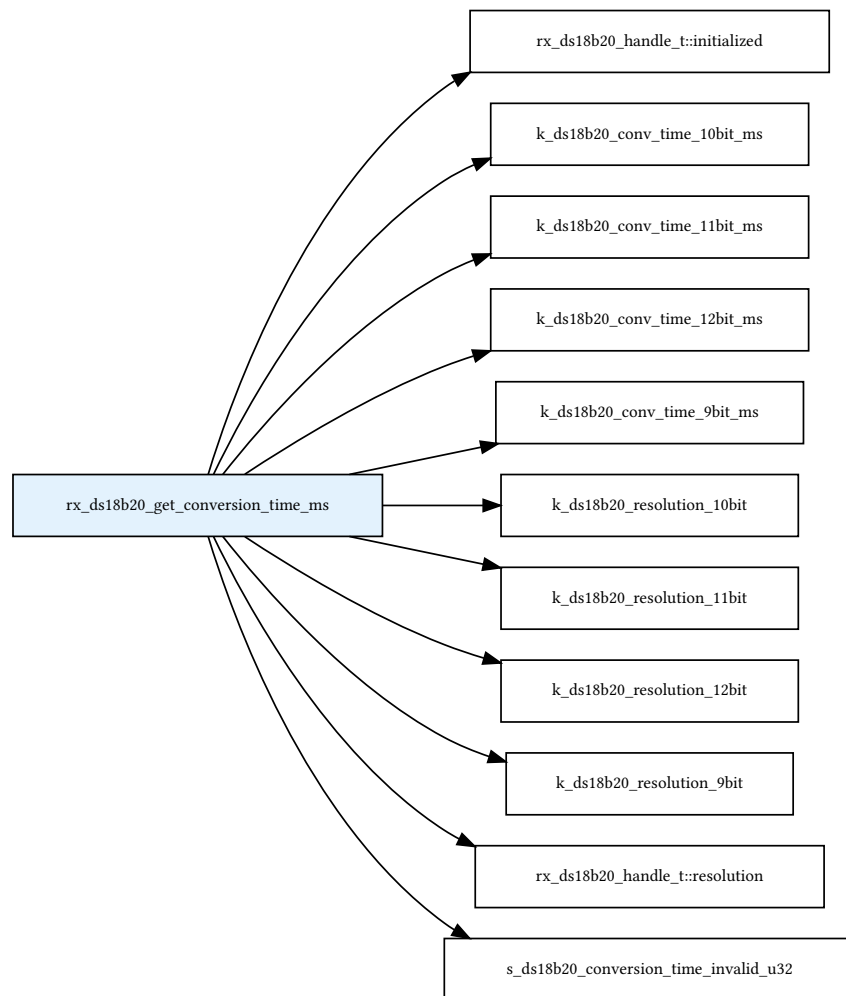


Figure 356: Call graph for `rx_ds18b20_get_conversion_time_ms`

Defined in `libs/rx_ds18b20/inc/rx_ds18b20.h:898`

rx_ds18b20.c

DS18B20 1-Wire Digital Temperature Sensor Driver Implementation.

Complete DS18B20 driver implementation with hardware abstraction via bus manager. Supports all DS18B20 features: temperature measurement, resolution configuration, ROM matching for multi-drop buses, and non-volatile EEPROM storage.

Includes:

- "rx_ds18b20.h"
- <math.h>
- <string.h>
- "rx_bus_onewire.h"
- "rx_check.h"
- "rx_crc.h"
- "rx_log.h"

CHECK_DS18B20_HANDLE

```
#define CHECK_DS18B20_HANDLE do
{
    RX_CHECK_NULL_PTR(handle, tag, "handle is nullptr");
    \
    if (!(handle)->initialized)
    {
        rx_log_error(tag, "DS18B20 not initialized");
        \
        return k_rx_err_invalid_state;
    }
    \
} while (0)
```

Validate DS18B20 handle pointer and initialization state.

Note: This is a statement macro (uses do-while(0) pattern) - requires semicolon.

Warning: Do not use in expressions - this macro contains early returns.] **See also:**
RX_CHECK_NULL_PTR

internal_ds18b20_validate_config

```
static rx_err_t internal_ds18b20_validate_config(const rx_ds18b20_config_t
*config, const rx_ds18b20_handle_t *handle)
```

Validate DS18B20 configuration parameters before initialization.

Comprehensive validation of configuration structure and handle state. Called by rx_ds18b20_init() to enforce preconditions and prevent misconfiguration.

Algorithm:

1. Check config pointer is not nullptr

2. Check bus_manager pointer is not nullptr
3. Check bus_name string is not nullptr
4. Verify handle is not already initialized (prevent re-initialization)
5. Validate resolution is in valid range (0-3)
6. If ROM matching enabled, verify family code is 0x28 (DS18B20)

Validation Rules:

Parameters:

Direction	Name	Description
in	config	Configuration structure to validate
in	handle	Handle to check for existing initialization

Returns: k_rx_ok Configuration is valid

Value	Description
k_rx_err_null_ptr	config, bus_manager, or bus_name is nullptr
k_rx_err_invalid_state	handle already initialized
k_rx_err_invalid_arg	Invalid resolution or wrong family code

Pre: config != nullptr

Pre: handle != nullptr (checked by caller)

Post: If k_rx_ok returned, all fields are valid for initialization

Note: This is an internal helper - not exposed in public API

Warning: Do not call after initialization - will return error] **Example:**

```
+-----+
|Field|Check|ErrorCode|
+-----+
|config|!=nullptr|k_rx_err_null_ptr|
|bus_manager|!=nullptr|k_rx_err_null_ptr|
|bus_name|!=nullptr|k_rx_err_null_ptr|
|initialized|==false|k_rx_err_invalid_state|
|resolution|<=3|k_rx_err_invalid_arg|
|rom[0](ifROM)|==0x28|k_rx_err_invalid_arg|
+-----+
```

See also: rx_ds18b20_init(), ds18b20_resolution_t

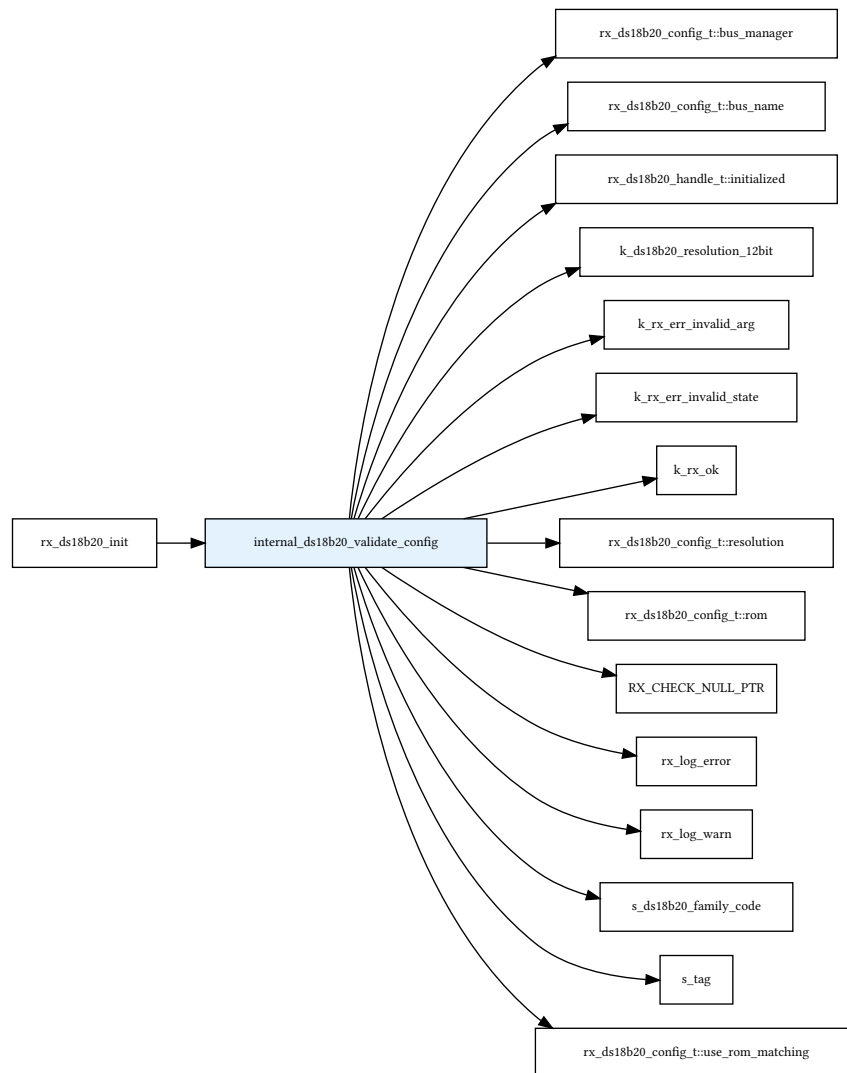


Figure 357: Call graph for internal_ds18b20_validate_config

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1277`

internal_ds18b20_verify_device_presence

```
static rx_err_t internal_ds18b20_verify_device_presence(rx_ds18b20_handle_t
*handle)
```

Verify DS18B20 device presence and establish communication.

Three-step verification process to ensure sensor is physically connected and responding correctly before completing initialization.

Algorithm:

Verification Steps:

1. Bus Initialization - Configure GPIO for 1-Wire timing
2. Presence Detection - Send reset, check for presence pulse
3. Communication Test - Read scratchpad with CRC validation

Timing:

- Reset pulse: 480-960 us LOW
- Presence detect: 60-240 us after reset
- Scratchpad read: 5ms (9 bytes + CRC)

Parameters:

Direction	Name	Description
inout	handle	DS18B20 handle with bus configuration

Returns: k_rx_ok Device present and communicating

Value	Description
k_rx_err_invalid_state	No presence pulse detected
k_rx_err_*	Bus initialization or scratchpad read error

Pre: handle->bus_manager is valid

Pre: handle->bus_name is valid

Post: If k_rx_ok, device is ready for configuration

Note: This function is called during rx_ds18b20_init()

Warning: Blocks for 100ms total (bus init + scratchpad read)

Performance: Execution time: 100ms (bus init 50ms + reset 1ms + scratchpad 5ms + margin)

Example:

```
@startuml
start
:InitializeOneWirebus;
if(Businitsuccessful?)then(yes)
:Sendresetpulse;
if(Presencepulsedetected?)then(yes)
:Readscratchpad(withCRC);
if(Scratchpadvalid?)then(yes)
:Returnk_rx_ok;
stop
else(CRCfail)
:Returnerror;
endif
```

```

else(nopresence)
:Log"devicenotpresent";
:Returnk_rx_err_invalid_state;
stop
endif
else(businitfail)
:Returnerror;
endif
@enduml

```

See also: rx_bus_owewire_init(), rx_bus_owewire_reset(),
internal_ds18b20_read_scratchpad_raw()

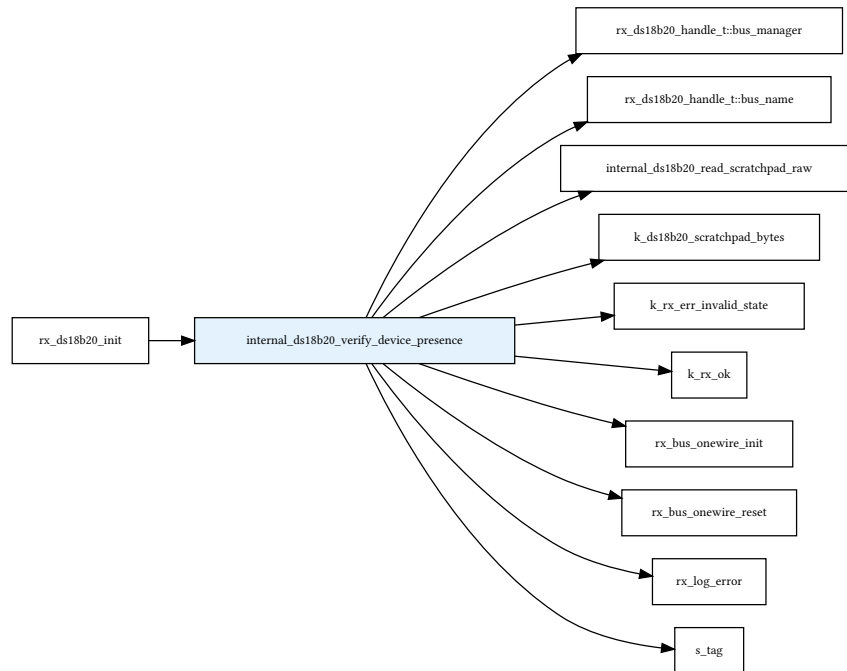


Figure 358: Call graph for internal_ds18b20_verify_device_presence

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1365`

internal_ds18b20_select_device

```
static rx_err_t internal_ds18b20_select_device(const rx_ds18b20_handle_t *handle)
```

Select DS18B20 device using ROM matching or Skip ROM command.

Sends appropriate ROM command based on bus configuration:

- Match ROM (0x55) - Address specific device on multi-drop bus
- Skip ROM (0xCC) - Broadcast to single device or all devices

Decision Flow:

ROM Matching (Multi-Drop Bus):

- Used when multiple DS18B20 sensors on one bus
- Requires 64-bit ROM code (obtained via search or label)
- Ensures commands go to intended sensor only

Skip ROM (Single Device):

- Faster - no need to send 64-bit ROM
- Safe when only one device on bus
- Caution: broadcasts if multiple devices present

Timing:

- Match ROM: 3ms (1 byte command + 8 bytes ROM)
- Skip ROM: 1ms (1 byte command)

Parameters:

Direction	Name	Description
in	handle	DS18B20 handle with ROM configuration

Returns: k_rx_ok Device selected successfully

Value	Description
k_rx_err_*	Bus communication error

Pre: handle->use_rom_matching is configured

Pre: If ROM matching: handle->rom contains valid 64-bit ID

Post: Selected device ready for function commands

Note: Called before every DS18B20 function command

Warning: Skip ROM on multi-drop bus affects ALL devices] **Example: ROM Code Format:**
 Byte0:FamilyCode(0x28forDS18B20) Byte1-6:SerialNumber(48-bituniqueID)
 Byte7:CRC-8checksum

Example:

```
@startuml
start
if(use_rom_matching?)then(yes)
:SendMatchROM(0x55);
:Send64-bitROMcode;
noteright
Addressesspecificdevice
byuniqueROMID
endnote
else(no)
:SendSkipROM(0xCC);
noteright
Singledevice:addressesonlydevice
```

```

Multipledevices:broadcaststoall
endnote
endif
:Deviceselected;
:Readyforfunctioncommand;
stop
@enduml

```

See also: rx_bus_onewire_match_rom(), rx_bus_onewire_skip_rom()

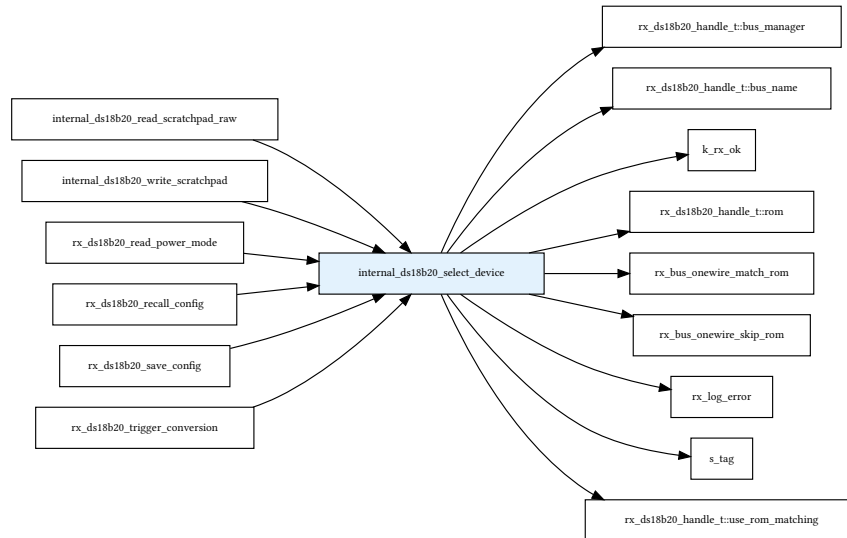


Figure 359: Call graph for internal_ds18b20_select_device

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1466`

internal_ds18b20_read_scratchpad_raw

```

static rx_err_t internal_ds18b20_read_scratchpad_raw(const rx_ds18b20_handle_t
*handle, uint8_t scratchpad[k_ds18b20_scratchpad_bytes])

```

Read DS18B20 scratchpad memory with CRC-8 validation.

Reads all 9 bytes of scratchpad memory and validates data integrity using CRC-8/MAXIM checksum. This is the primary method for reading temperature and configuration data from the sensor.

Algorithm:

Scratchpad Memory Map:

CRC-8/MAXIM Calculation:

- Polynomial: $0x31 (x^8 + x^5 + x^4 + 1)$
- Initial value: `0x00`
- Input: First 8 bytes of scratchpad
- Result: Must match byte 8

Post-Conditions:

- Scratchpad buffer contains valid sensor data
- CRC verified (data integrity guaranteed)
- Reserved byte checked (warning if unexpected)

Parameters:

Direction	Name	Description
in	handle	DS18B20 sensor handle (must be initialized)
out	scratchpad	9-byte buffer to receive scratchpad data Array must be at least k_ds18b20_scratchpad_bytes (9) bytes

Returns: k_rx_ok Scratchpad read successful, CRC valid

Value	Description
k_rx_err_null_ptr	handle or scratchpad is nullptr
k_rx_err_invalid_state	Device not present on bus
k_rx_err_crc_mismatch	CRC validation failed (corrupted data)
k_rx_err_*	Bus communication error

Pre: handle != nullptr

Pre: handle->initialized == true

Pre: scratchpad != nullptr

Pre: scratchpad size >= 9 bytes

Post: If k_rx_ok: scratchpad contains validated sensor data

Post: If error: scratchpad contents undefined

Note: Reserved byte validation is non-fatal (log warning only)

Warning: Do not use scratchpad data if CRC check fails

Performance:: Execution time: 5ms (reset 1ms + select 1ms + read 3ms)

Example: Manual Scratchpad Access: uint8_tscratchpad[9];
 rx_err_t err=internal_ds18b20_read_scratchpad_raw(&sensor,scratchpad); if(err==k_rx_ok)
 { int16_t temp_raw=scratchpad[0]|(scratchpad[1]<<8); uint8_t config=scratchpad[4];
 uint8_t resolution=(config>5)&0x03; }

Example:

```
@startuml
start
```

```

:Sendresetpulse;
if(Presencedetected?)then(yes)
:Selectdevice(Match/SkipROM);
:SendReadScratchpad(0xBE);
:Read9bytes(8data+1CRC);
:CalculateCRC-8offirst8bytes;
if(CRCmatchesbyte8?)then(yes)
if(Reservedbyte==0xFF?)then(yes)
:Returnk_rx_ok;
stop
else(reservedinvalid)
:Logwarning;
:Returnk_rx_okanyway;
noteright:Non-fatal,continue
endif
else(CRCmismatch)
:Logerror;
:Returnk_rx_err_crc_mismatch;
stop
endif
else(nopresence)
:Returnk_rx_err_invalid_state;
endif
@enduml

```

Example:

Byte	Field	Description
0	TempLSB	Temperaturelowbyte
1	TempMSB	Temperaturehighbyte
2	THRegister	Highalarmthreshold
3	TLRegister	Lowalarmthreshold
4	Config	Resolutionbits(6:5)
5	Reserved	Always0xFF
6	Reserved	Factory-programmed
7	Reserved	Countremain(legacy)
8	CRC	CRC-8/MAXIMchecksum

See also: rx_crc8_maxim(), internal_ds18b20_select_device()

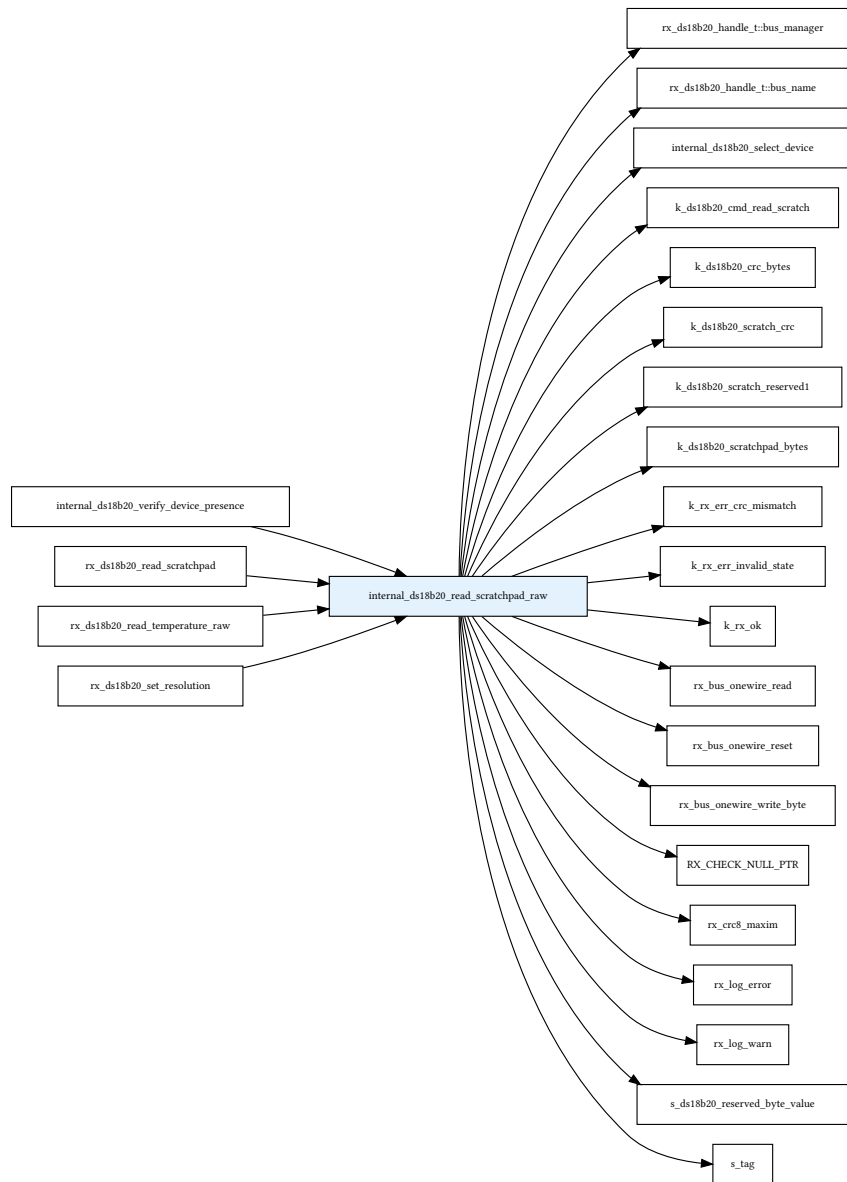


Figure 360: Call graph for internal_ds18b20_read_scratchpad_raw

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1590`

internal_ds18b20_write_scratchpad

```
static rx_err_t internal_ds18b20_write_scratchpad(const rx_ds18b20_handle_t
*handle, const ds18b20_scratchpad_write_t *scratchpad)
```

Write DS18B20 scratchpad registers (TH, TL, Config).

Writes the three user-modifiable scratchpad registers: high alarm (TH), low alarm (TL), and configuration (resolution). This is the only way to change sensor resolution and alarm thresholds.

Algorithm:

Write Sequence:

1. Reset bus and check presence
2. Select device (Match ROM or Skip ROM)
3. Send Write Scratchpad command (0x4E)
4. Write 3 bytes in order: TH, TL, Config

Configuration Register Encoding:

Persistence:

- Scratchpad writes are volatile (RAM only)
- Lost on power cycle or reset
- Use Copy Scratchpad (0x48) to save to EEPROM

Parameters:

Direction	Name	Description
in	handle	DS18B20 sensor handle (must be initialized)
in	scratchpad	Write parameters (TH, TL, config)

Returns: k_rx_ok Scratchpad written successfully

Value	Description
k_rx_err_null_ptr	handle or scratchpad is nullptr
k_rx_err_invalid_state	Device not present on bus
k_rx_err_*	Bus communication error

Pre: handle != nullptr

Pre: handle->initialized == true

Pre: scratchpad != nullptr

Post: Sensor resolution/alarms updated (volatile, RAM only)

Note: Changes are immediate but volatile (lost on power cycle)

Warning: No CRC verification - ensure bus integrity

Performance:: Execution time: 3ms (reset 1ms + select 1ms + write 1ms)

Example: Change Resolution to 10-bit: uint8_tscratchpad_data[9];
internal_ds18b20_read_scratchpad_raw(&sensor,scratchpad_data);

```

ds18b20_scratchpad_write_twwrite_cfg={ .th=scratchpad_data[2], .tl=scratchpad_data[3], .config=(0x01<<5)|
0x1F }; internal_ds18b20_write_scratchpad(&sensor,&write_cfg);

rx_ds18b20_save_config(&sensor);

```

Example:

```

@startuml
start
:Sendresetpulse;
if(Presencedetected?)then(yes)
:Selectdevice(Match/SkipROM);
:SendWriteScratchpad(0x4E);
:WriteTHbyte;
:WriteTLbyte;
:WriteConfigbyte;
:Returnk_rx_ok;
noteright
Changestateeffectimmediately
butarevolatile(lostonpowercycle)
endnote
stop
else(nopresence)
:Returnk_rx_err_invalid_state;
endif
@enduml

```

Example:

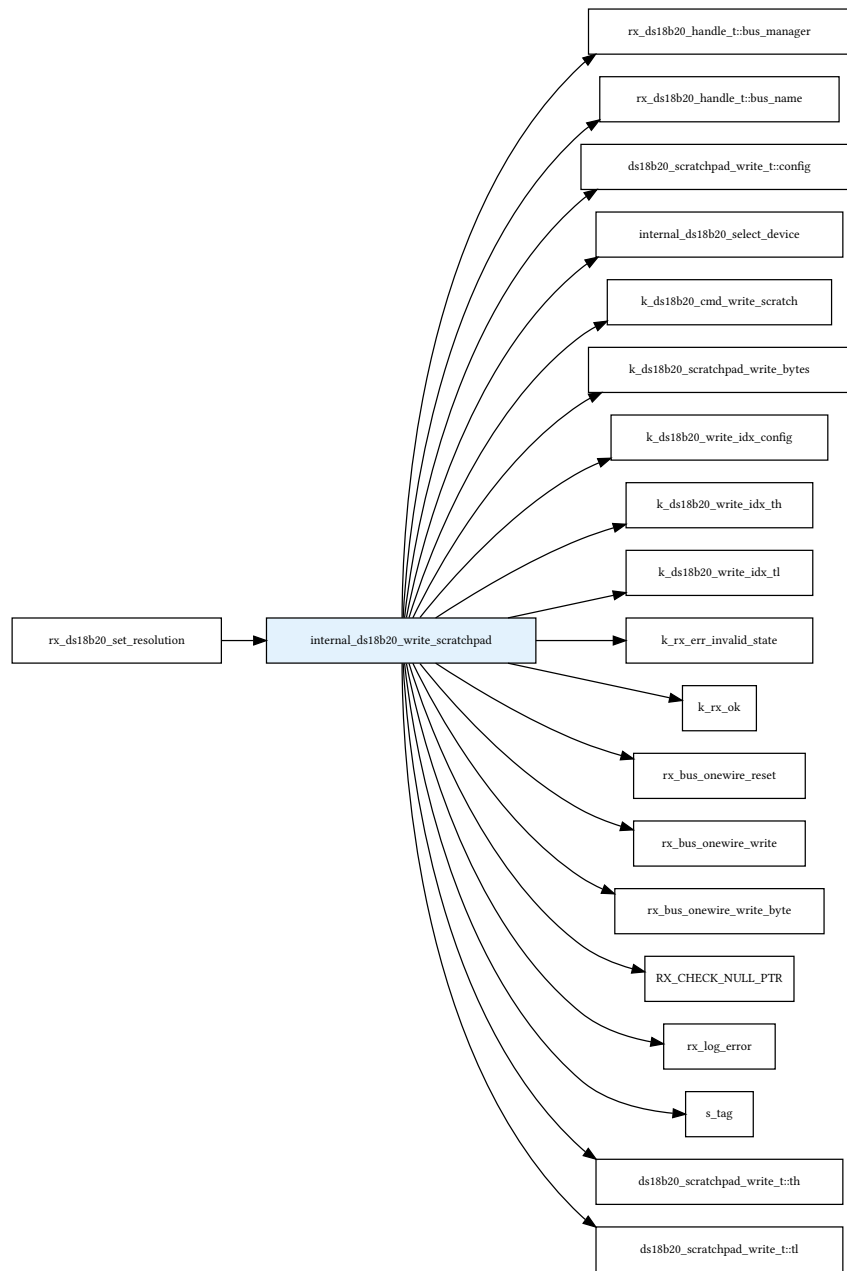
```

Bit:76543210
+---+---+---+---+---+---+---+
|0|R1|R0|1|1|1|1|1|
+---+---+---+---+---+---+---+
+---+---+
Resolution

R1R0|Resolution|ConversionTime
-----+-----
00|9-bit|94ms
01|10-bit|188ms
10|11-bit|375ms
11|12-bit|750ms

```

See also: rx_ds18b20_save_config() Make changes permanent,
internal_ds18b20_select_device(), ds18b20_scratchpad_write_t

Figure 361: Call graph for `internal_ds18b20_write_scratchpad`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1747`

internal_ds18b20_resolution_to_config

```
static uint8_t internal_ds18b20_resolution_to_config(const ds18b20_resolution_t resolution)
```

Convert resolution enum to config register value.

Parameters:

Direction	Name	Description
in	resolution	Resolution mode

Returns: Configuration register value

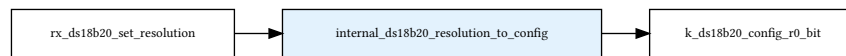


Figure 362: Call graph for `internal_ds18b20_resolution_to_config`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1800`

—

internal_ds18b20_get_temp_mask

```
static uint16_t internal_ds18b20_get_temp_mask(const ds18b20_resolution_t resolution)
```

Get temperature mask for resolution.

Lower bits are undefined for resolutions < 12-bit and must be masked.

Parameters:

Direction	Name	Description
in	resolution	Resolution mode

Returns: Temperature mask

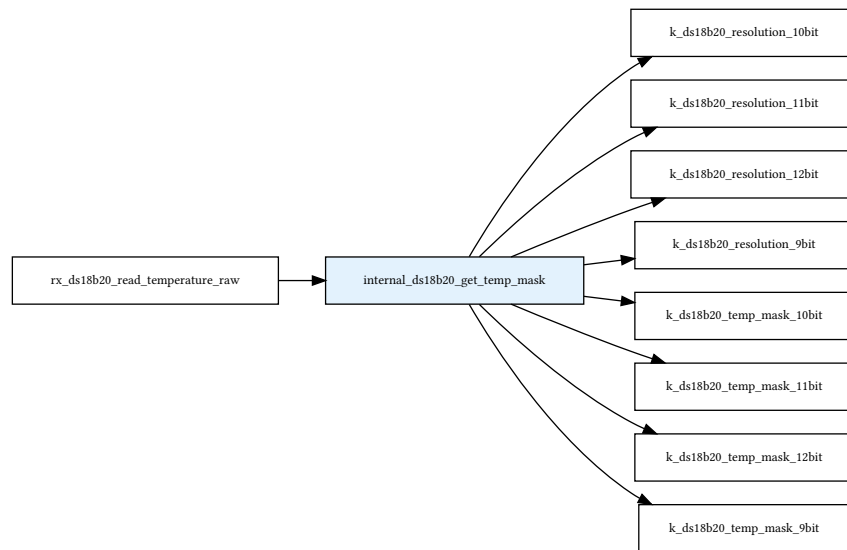


Figure 363: Call graph for internal_ds18b20_get_temp_mask

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1817`

internal_ds18b20_raw_to_celsius

```
static float internal_ds18b20_raw_to_celsius(const int16_t raw_temp)
```

Convert raw temperature to Celsius.

DS18B20 stores temperature as 16-bit signed value in 1/16degC units. Divide by 16 to get Celsius.

Parameters:

Direction	Name	Description
in	raw_temp	Raw 16-bit temperature value

Returns: Temperature in degrees Celsius

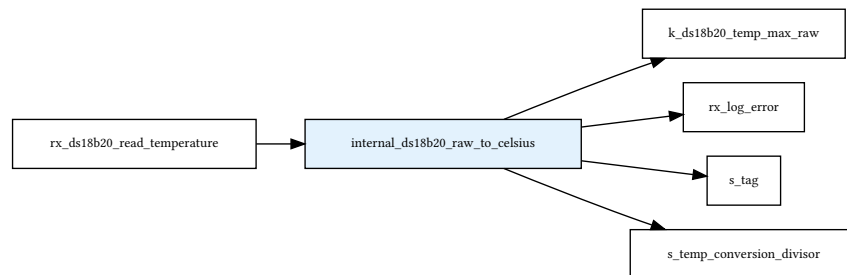


Figure 364: Call graph for internal_ds18b20_raw_to_celsius

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1843`

—

rx_ds18b20_init

```
rx_err_t rx_ds18b20_init(rx_ds18b20_handle_t *handle, const rx_ds18b20_config_t
*config)
```

Initialize DS18B20 sensor.

Initializes the DS18B20 sensor and configures resolution. If ROM matching is disabled, uses Skip ROM (single device mode).

Parameters:

Direction	Name	Description
out	handle	Pointer to DS18B20 handle. Must not be nullptr.
in	config	Pointer to configuration. Must not be nullptr.

Returns: k_rx_err_crc if scratchpad CRC check fails



Figure 365: Call graph for `rx_ds18b20_init`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:356`

—

rx_ds18b20_deinit

```
rx_err_t rx_ds18b20_deinit(rx_ds18b20_handle_t *handle)
```

Deinitialize DS18B20 sensor handle and release all resources.

Deinitialize DS18B20 sensor.

Tears down a previously initialized DS18B20 handle by zeroing all internal state via `memset`. After this call the handle is safe to reinitialize with `rx_ds18b20_init()`.

Algorithm steps:

1. Validate handle is not nullptr
2. Verify handle is currently initialized
3. Clear the entire handle structure with `memset` (zeros all fields)
4. Log deinitialization success

Parameters:

Direction	Name	Description
inout	handle	Sensor handle to deinitialize (must be initialized, non-null)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Handle successfully cleared
<code>k_rx_err_null_ptr</code>	handle pointer is nullptr
<code>k_rx_err_invalid_state</code>	handle was not previously initialized

Pre: handle must be a valid non-null pointer

Pre: handle must have been successfully initialized via `rx_ds18b20_init()`

Post: All handle fields are zeroed (initialized flag is false)

Post: handle may be safely reused with `rx_ds18b20_init()`

Note: Not thread-safe; caller must ensure no concurrent access to handle

Example:: `rx_ds18b20_handle_t sensor; rx_ds18b20_init(&sensor, &config);
rx_err_t err = rx_ds18b20_deinit(&sensor); if (err != k_rx_ok)
{ rx_log_error("app", "Failed to deinit DS18B20"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: `rx_ds18b20_init()` Initialize the handle before use

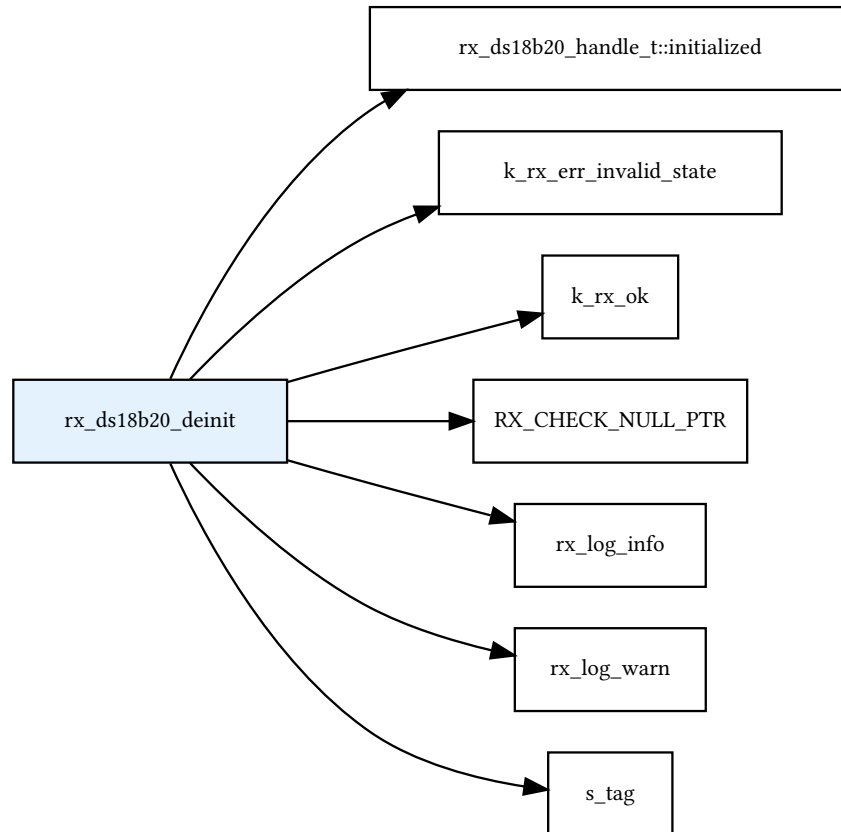


Figure 366: Call graph for `rx_ds18b20_deinit`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:441`

Since Version 1.0.0

—

rx_ds18b20_trigger_conversion

```
rx_err_t rx_ds18b20_trigger_conversion(const rx_ds18b20_handle_t *handle)
```

Trigger temperature conversion on DS18B20 sensor.

Trigger temperature conversion.

Issues the Convert T (0x44) command over the 1-Wire bus to start an autonomous temperature measurement. The DS18B20 will perform the ADC conversion internally; the caller must wait the appropriate conversion time before reading results with `rx_ds18b20_read_temperature()`.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse

3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Convert T command byte (0x44) to the bus

Conversion time depends on resolution:

- 9-bit: 94 ms
- 10-bit: 188 ms
- 11-bit: 375 ms
- 12-bit: 750 ms

Parameters:

Direction	Name	Description
in	handle	Sensor handle (must be initialized, non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Conversion command issued successfully
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset or write operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected and powered on the bus

Post: DS18B20 ADC conversion is in progress

Post: Caller must wait rx_ds18b20_get_conversion_time_ms() before reading

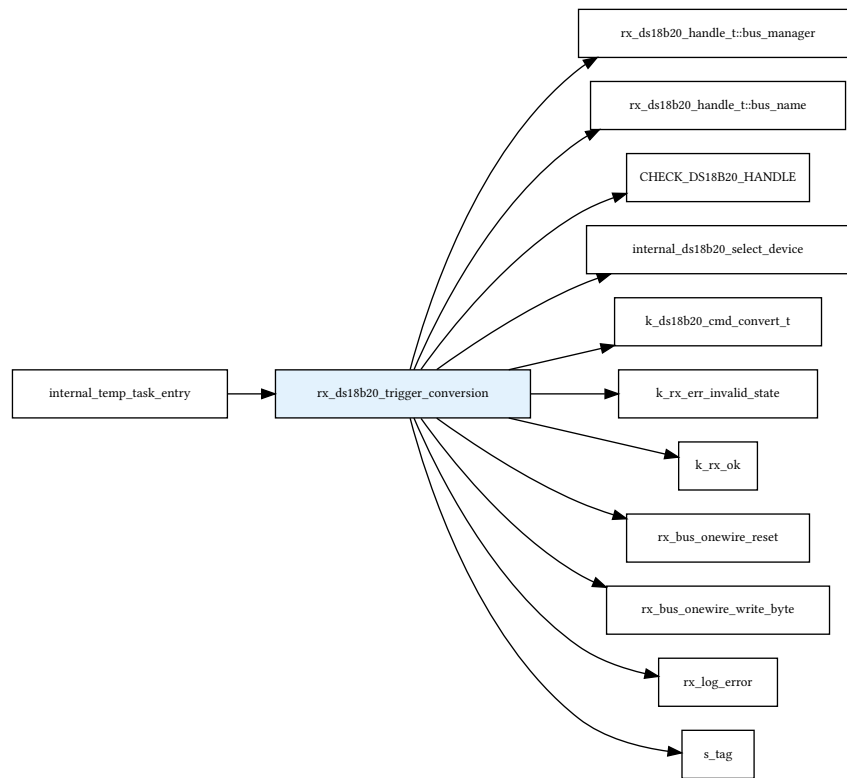
Note: Not thread-safe; caller must serialize access to the same handle

Warning: Do not call rx_ds18b20_read_temperature() before the conversion time elapses

Example:: rx_err_t err=rx_ds18b20_trigger_conversion(&sensor); if(err!=k_rx_ok)
{ tx_thread_sleep(rx_ds18b20_get_conversion_time_ms(&sensor)); floattemp_celsius=0.0f;
err=rx_ds18b20_read_temperature(&sensor,&temp_celsius); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (initialized handle, device present), 2 postconditions

See also: rx_ds18b20_read_temperature() Read converted temperature in Celsius,
rx_ds18b20_get_conversion_time_ms() Get required wait time after conversion,
rx_ds18b20_set_resolution() Configure ADC resolution

Figure 367: Call graph for `rx_ds18b20_trigger_conversion`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:513`

Since Version 1.0.0

—

rx_ds18b20_read_temperature

```
rx_err_t rx_ds18b20_read_temperature(rx_ds18b20_handle_t *handle, float
*temperature_celsius)
```

Read temperature from DS18B20 sensor in degrees Celsius.

Read temperature in Celsius.

Reads the full 9-byte scratchpad from the DS18B20, extracts the 16-bit raw temperature value, applies the resolution mask to clear undefined low-order bits, validates the result is within the sensor's physical range, and converts to a floating-point Celsius value using the DS18B20 fixed-point encoding (1 LSB = 1/16 degC).

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Call `rx_ds18b20_read_temperature_raw()` to obtain masked raw value
3. Convert raw signed 16-bit value to float: $\text{celsius} = \text{raw} / 16.0f$
4. Write result to `*temperature_celsius`

Parameters:

Direction	Name	Description
inout	handle	Sensor handle (must be initialized)
out	temperature_celsius	Converted temperature in degrees Celsius, valid range [-55.0, +125.0]; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Temperature read and converted successfully
k_rx_err_null_ptr	handle or temperature_celsius is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_out_of_range	raw temperature outside [-55degC, +125degC]
k_rx_err_crc_failure	Scratchpad CRC mismatch
k_rx_err_bus_error	1-Wire bus communication failure

Pre: handle must be initialized via rx_ds18b20_init()

Pre: rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed

Post: *temperature_celsius contains valid temperature in -55.0, +125.0 on k_rx_ok

Post: handle->stats.read_count incremented (via internal_ds18b20_read_scratchpad_raw)

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Returns stale data if conversion time has not elapsed after trigger

Example:: float temp_celsius=0.0f;
 rx_err_t err=rx_ds18b20_read_temperature(&sensor,&temp_celsius); if(err==k_rx_ok)
 { rx_log_info("app","Temperature:%.4fC",(double)temp_celsius); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_ds18b20_trigger_conversion() Must be called before reading,
 rx_ds18b20_read_temperature_raw() Read the raw 16-bit value,
 rx_ds18b20_get_conversion_time_ms() Determine required wait time

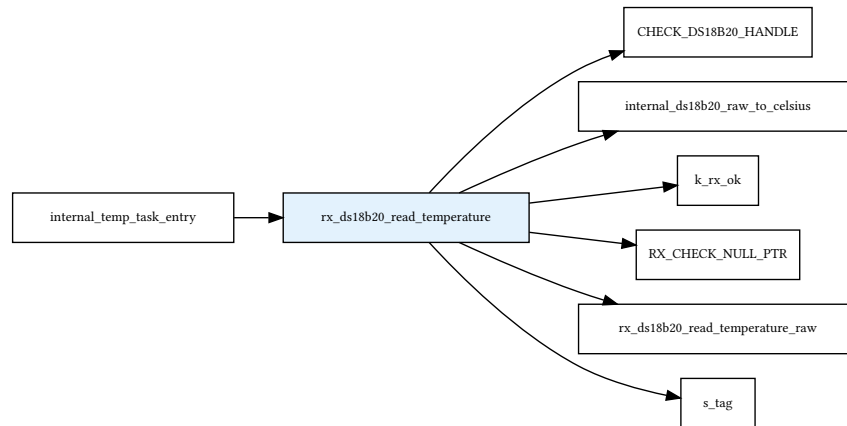


Figure 368: Call graph for rx_ds18b20_read_temperature

Defined in *libs/rx_ds18b20/src/rx_ds18b20.c:595*

Since Version 1.0.0

—

rx_ds18b20_read_temperature_raw

```
rx_err_t rx_ds18b20_read_temperature_raw(rx_ds18b20_handle_t *handle, int16_t
*raw_temp)
```

Read raw temperature value from DS18B20 sensor in 1/16 degree Celsius units.

Read raw temperature value.

Reads the 9-byte DS18B20 scratchpad register, extracts the two temperature bytes (LSB at offset 0, MSB at offset 1), combines them into a 16-bit unsigned value, applies the resolution-dependent mask to clear undefined low-order bits, casts to signed int16_t, and range-validates the result.

The DS18B20 temperature encoding uses a signed fixed-point format where 1 LSB = 1/16 degC (12-bit mode). The valid raw range is:

- Minimum: -55 degC = 0xFC90 (int16: -880)
- Maximum: +125 degC = 0x07D0 (int16: +2000)

Resolution masks (clears undefined bits in lower-resolution modes):

- 9-bit: 0xFFF8 (3 undefined bits)
- 10-bit: 0xFFFC (2 undefined bits)
- 11-bit: 0xFFFE (1 undefined bit)
- 12-bit: 0xFFFF (no undefined bits)

Algorithm steps:

1. Validate handle and output pointer are non-null and handle is initialized
2. Read raw scratchpad bytes via internal_ds18b20_read_scratchpad_raw()
3. Combine temperature LSB and MSB bytes into uint16_t
4. Apply resolution mask from internal_ds18b20_get_temp_mask()
5. Cast to int16_t and validate range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]
6. Write validated result to *raw_temp

Parameters:

Direction	Name	Description
inout	handle	Sensor handle (must be initialized)
out	raw_temp	Signed 16-bit raw temperature in 1/16 degC units, valid range [k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw]; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Raw temperature read, masked, and validated successfully
k_rx_err_null_ptr	handle or raw_temp is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_out_of_range	raw value outside physical sensor range
k_rx_err_crc_failure	Scratchpad CRC check failed
k_rx_err_bus_error	1-Wire bus communication failure

Pre: handle must be initialized via rx_ds18b20_init()**Pre:** rx_ds18b20_trigger_conversion() must have been called and conversion time elapsed**Post:** *raw_temp is in k_ds18b20_temp_min_raw, k_ds18b20_temp_max_raw on k_rx_ok**Post:** Resolution mask has been applied; undefined bits are cleared**Note:** Not thread-safe; caller must serialize access to the same handle**Warning:** Returns stale data if conversion time has not elapsed after trigger

Example:: int16_t raw=0; rx_err_t err=rx_ds18b20_read_temperature_raw(&sensor,&raw);
 if(err!=k_rx_ok){ float celsius=(float)raw/16.0f; }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions (range, mask)

See also: rx_ds18b20_read_temperature() Convenience wrapper returning float Celsius,
 rx_ds18b20_trigger_conversion() Must be called before reading,
 rx_ds18b20_set_resolution() Configure ADC resolution and mask

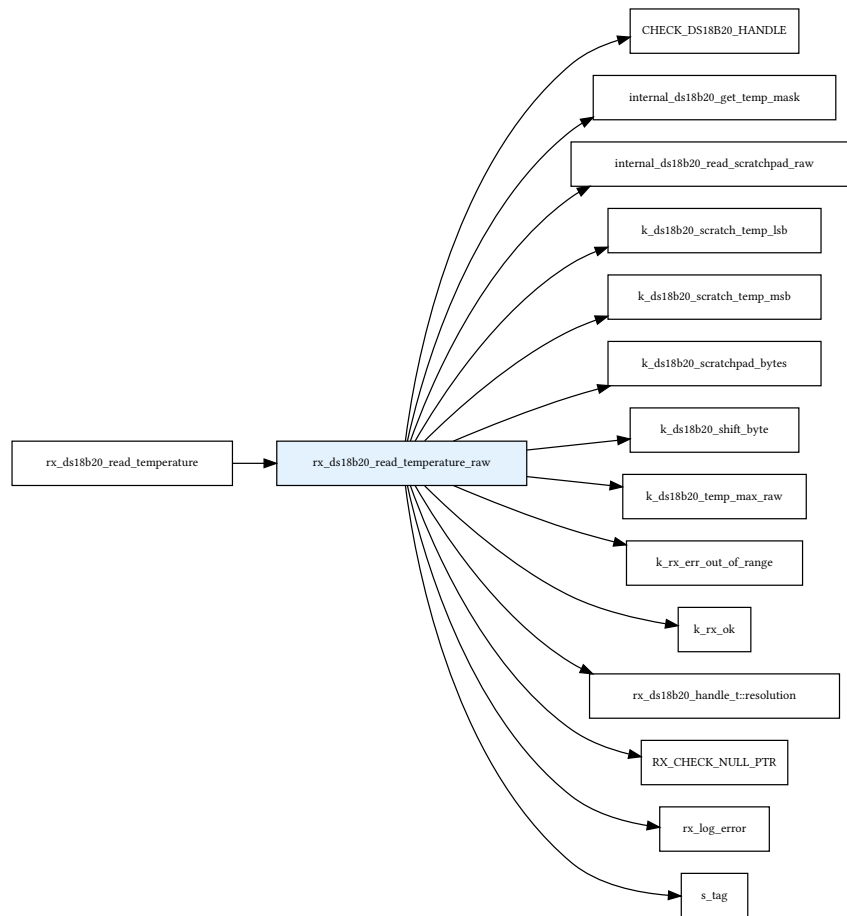


Figure 369: Call graph for rx_ds18b20_read_temperature_raw

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:680`

Since Version 1.0.0

—

rx_ds18b20_set_resolution

```
rx_err_t rx_ds18b20_set_resolution(rx_ds18b20_handle_t *handle, const
ds18b20_resolution_t resolution)
```

Set ADC measurement resolution on DS18B20 sensor.

Set temperature resolution.

Updates the configuration register in the DS18B20 scratchpad to change the ADC resolution. The existing TH and TL alarm threshold bytes are preserved by reading the current scratchpad first, then writing back only the updated configuration byte.

Higher resolution provides finer temperature steps but increases conversion time:

- 9-bit: 0.5 degC steps, 94 ms conversion
- 10-bit: 0.25 degC steps, 188 ms conversion
- 11-bit: 0.125 degC steps, 375 ms conversion
- 12-bit: 0.0625 degC steps, 750 ms conversion

Algorithm steps:

1. Validate handle is initialized and resolution is a legal enum value
2. Read current 9-byte scratchpad to preserve TH/TL alarm bytes
3. Convert resolution enum to configuration register byte via `internal_ds18b20_resolution_to_config()`
4. Write scratchpad with new config byte (TH/TL unchanged)
5. Update handle->resolution to reflect new setting

Parameters:

Direction	Name	Description
inout	handle	Sensor handle (must be initialized); resolution field updated on success
in	resolution	New resolution setting (<code>k_ds18b20_resolution_9bit</code> through <code>k_ds18b20_resolution_12bit</code>); all other values are invalid

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Resolution updated in scratchpad and handle
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
<code>k_rx_err_invalid_arg</code>	resolution is not a valid <code>ds18b20_resolution_t</code> value
<code>k_rx_err_crc_failure</code>	Scratchpad read CRC check failed
<code>k_rx_err_bus_error</code>	1-Wire bus communication failure during read or write

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: resolution must be one of `k_ds18b20_resolution_9bit`/`10bit`/`11bit`/`12bit`

Post: handle->resolution reflects the newly configured value on `k_rx_ok`

Post: DS18B20 scratchpad configuration register updated with new resolution bits

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Change is not persistent across power cycles unless `rx_ds18b20_save_config()` is called

Example:: `rx_err_t err = rx_ds18b20_set_resolution(&sensor, k_ds18b20_resolution_12bit);`
`if (err != k_rx_ok) { rx_ds18b20_save_config(&sensor); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null/initialized, valid resolution), 2 postconditions

See also: `rx_ds18b20_get_resolution()` Read back the currently configured resolution,
`rx_ds18b20_save_config()` Persist resolution change to EEPROM,
`rx_ds18b20_get_conversion_time_ms()` Get required conversion time for new resolution

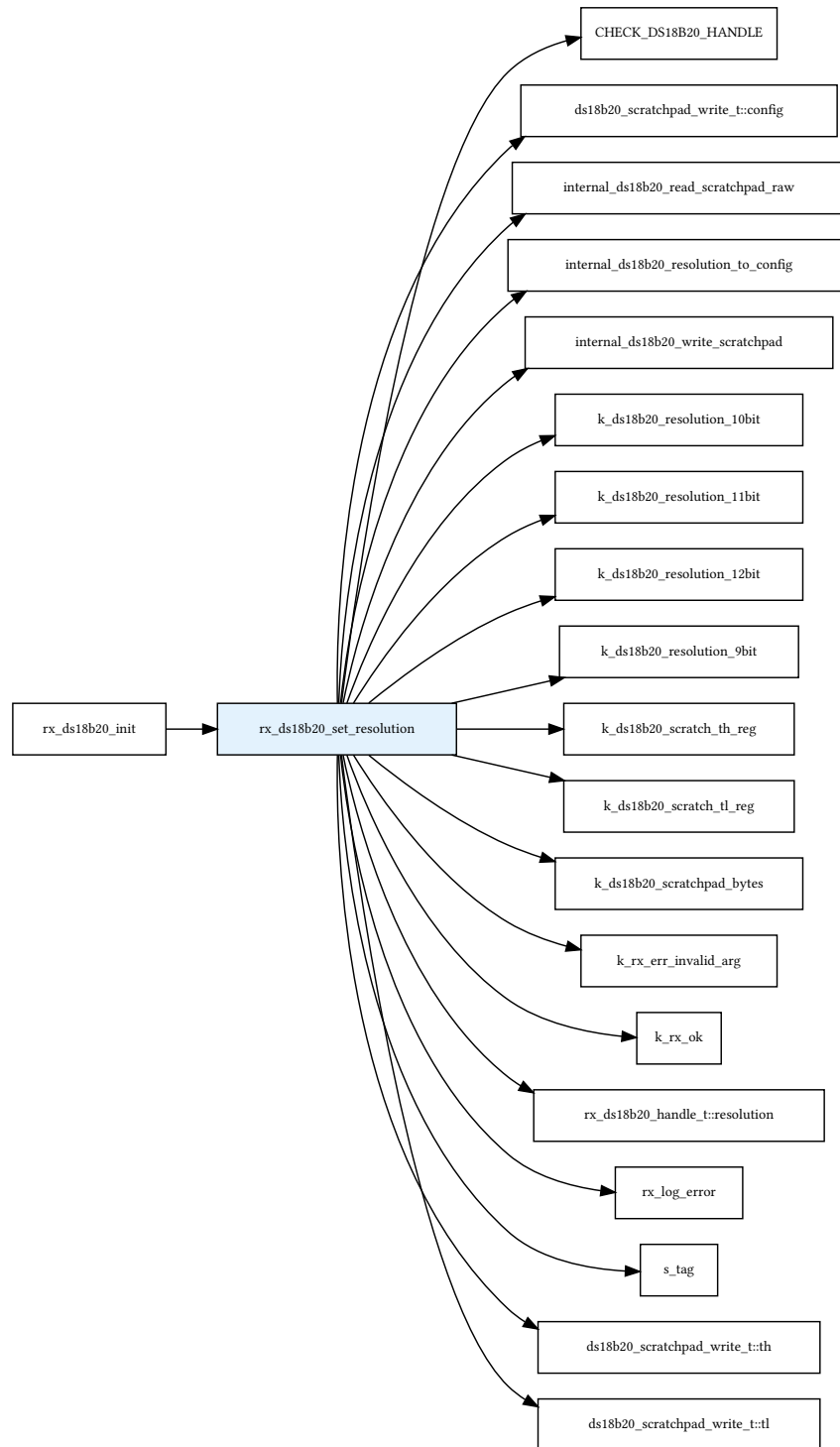


Figure 370: Call graph for `rx_ds18b20_set_resolution`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:772`

Since Version 1.0.0

—

rx_ds18b20_get_resolution

```
rx_err_t rx_ds18b20_get_resolution(const rx_ds18b20_handle_t *handle,
                                   ds18b20_resolution_t *resolution)
```

Get the currently configured ADC resolution from DS18B20 handle.

Get current temperature resolution.

Returns the resolution value stored in the handle, which reflects the last successfully applied resolution (set during initialization or via `rx_ds18b20_set_resolution()`). This is a cached read from the handle state; it does not issue a 1-Wire bus transaction.

Algorithm steps:

1. Validate handle and output pointer are non-null
2. Verify handle is initialized
3. Copy handle->resolution to *resolution

Parameters:

Direction	Name	Description
in	handle	Sensor handle (must be initialized, non-null)
out	resolution	Receives the currently configured ds18b20_resolution_t value; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Resolution successfully returned
k_rx_err_null_ptr	handle or resolution pointer is nullptr
k_rx_err_invalid_state	handle not initialized (CHECK_DS18B20_HANDLE fails)

Pre: handle must be initialized via `rx_ds18b20_init()`

Pre: resolution must be a valid non-null pointer

Post: *resolution contains the current resolution value on k_rx_ok

Post: handle state is unchanged

Note: Not thread-safe; caller must ensure handle is not concurrently modified

Note: Returns cached value from handle does not issue 1-Wire bus transaction

Example:: ds18b20_resolution_tres; rx_err_t err=rx_ds18b20_get_resolution(&sensor,&res);
 if(err==k_rx_ok){ uint32_t conv_ms=rx_ds18b20_get_conversion_time_ms(&sensor);
 rx_log_info("app","Resolution%d,convtime%ums",(int)res,(unsigned)conv_ms); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_ds18b20_set_resolution() Change the ADC resolution,
 rx_ds18b20_get_conversion_time_ms() Get conversion time for current resolution

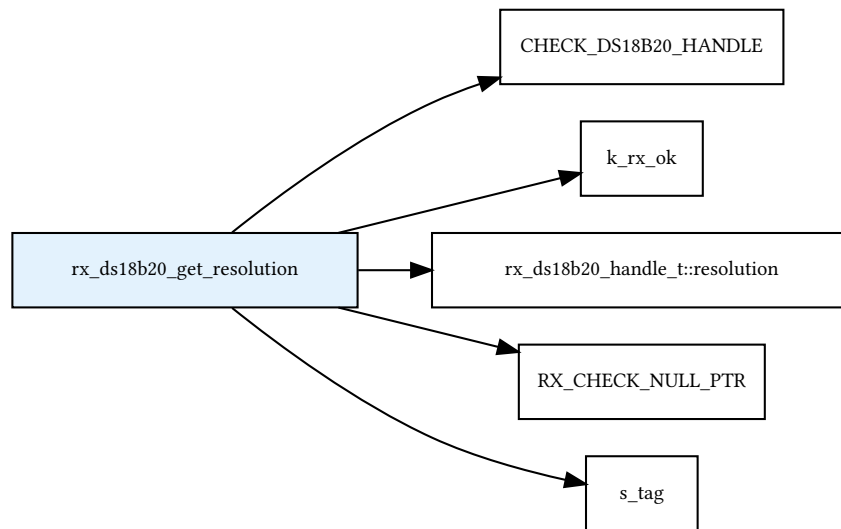


Figure 371: Call graph for rx_ds18b20_get_resolution

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:858`

Since Version 1.0.0

—

rx_ds18b20_save_config

```
rx_err_t rx_ds18b20_save_config(const rx_ds18b20_handle_t *handle)
```

Save current scratchpad configuration to DS18B20 EEPROM.

Save configuration to EEPROM.

Sends the Copy Scratchpad (0x48) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from volatile scratchpad RAM to non-volatile EEPROM. The saved values persist across power cycles and are automatically restored on the next power-up via the Recall E2 sequence.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse

3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Copy Scratchpad command byte (0x48)

Parameters:

Direction	Name	Description
in	handle	Sensor handle (must be initialized, non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Copy Scratchpad command successfully issued
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset or write operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected and externally powered (Copy Scratchpad is not guaranteed in parasitic power mode)

Post: DS18B20 EEPROM contains current TH, TL, and configuration register values

Post: Configuration will survive next power cycle

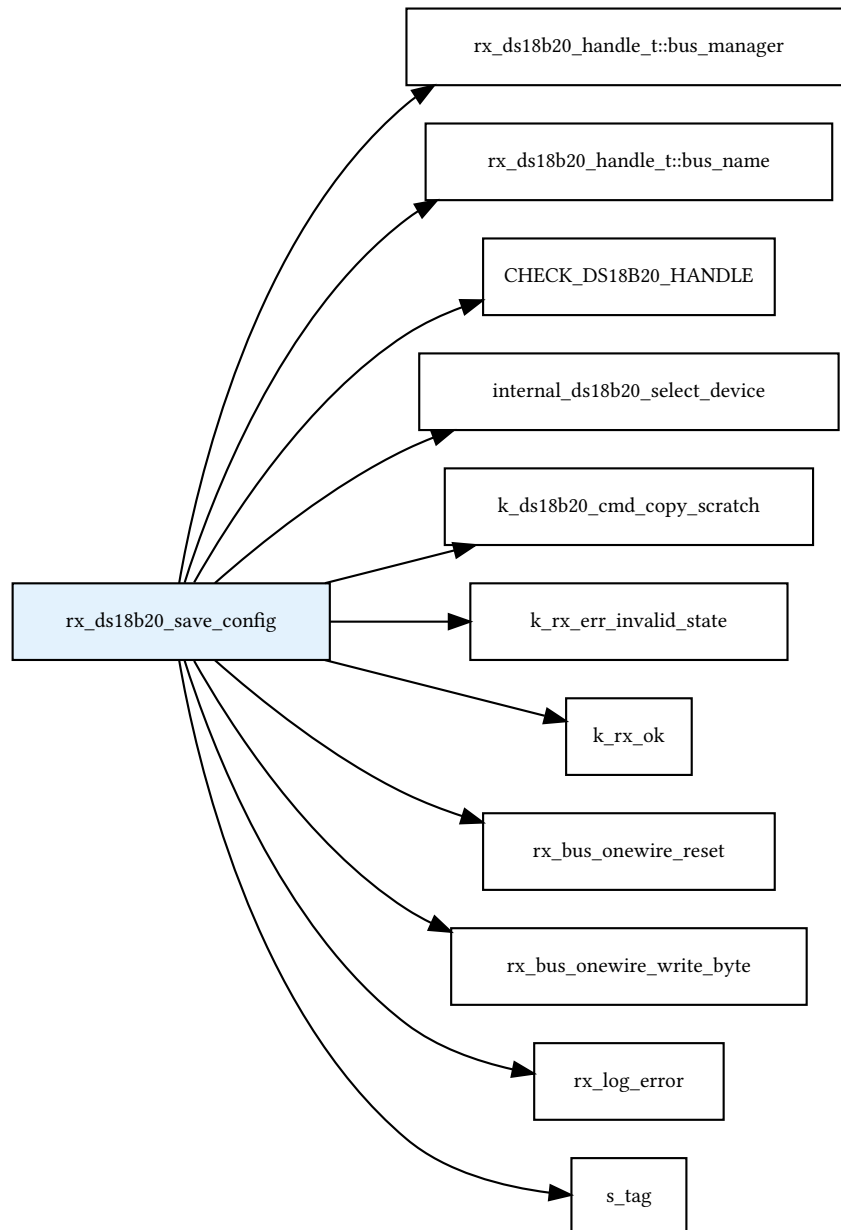
Note: Not thread-safe; caller must serialize access to the same handle

Warning: Not reliable in parasitic power mode; use external power for EEPROM writes

Example:: rx_ds18b20_set_resolution(&sensor,k_ds18b20_resolution_12bit);
rx_err_t err=rx_ds18b20_save_config(&sensor); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to save DS18B20 config to EEPROM"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

See also: rx_ds18b20_recall_config() Restore EEPROM contents to scratchpad,
rx_ds18b20_set_resolution() Configure resolution before saving

Figure 372: Call graph for `rx_ds18b20_save_config`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:919`

Since Version 1.0.0

—

rx_ds18b20_recall_config

```
rx_err_t rx_ds18b20_recall_config(const rx_ds18b20_handle_t *handle)
```

Recall configuration from DS18B20 EEPROM into scratchpad RAM.

Recall configuration from EEPROM.

Sends the Recall E2 (0xB8) command over the 1-Wire bus, which instructs the DS18B20 to copy the TH alarm, TL alarm, and configuration register bytes from non-volatile EEPROM back into volatile scratchpad RAM. This is the same operation the device performs automatically on power-up.

Use this function to discard unsaved scratchpad changes and revert to the last saved EEPROM state without cycling power.

Algorithm steps:

1. Validate handle is initialized and non-null
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Recall E2 command byte (0xB8)

Parameters:

Direction	Name	Description
in	handle	Sensor handle (must be initialized, non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Recall E2 command successfully issued
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset or write operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected to the bus

Post: DS18B20 scratchpad TH, TL, and configuration bytes reflect last EEPROM save

Post: Any unsaved scratchpad changes since last rx_ds18b20_save_config() are discarded

Note: Not thread-safe; caller must serialize access to the same handle

Note: The Recall E2 operation is performed automatically by the device on power-up

Example:: rx_err_t err=rx_ds18b20_recall_config(&sensor); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to recall DS18B20 config from EEPROM"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null/initialized, device present), 2 postconditions

See also: `rx_ds18b20_save_config()` Persist scratchpad configuration to EEPROM,
`rx_ds18b20_set_resolution()` Modify the scratchpad configuration

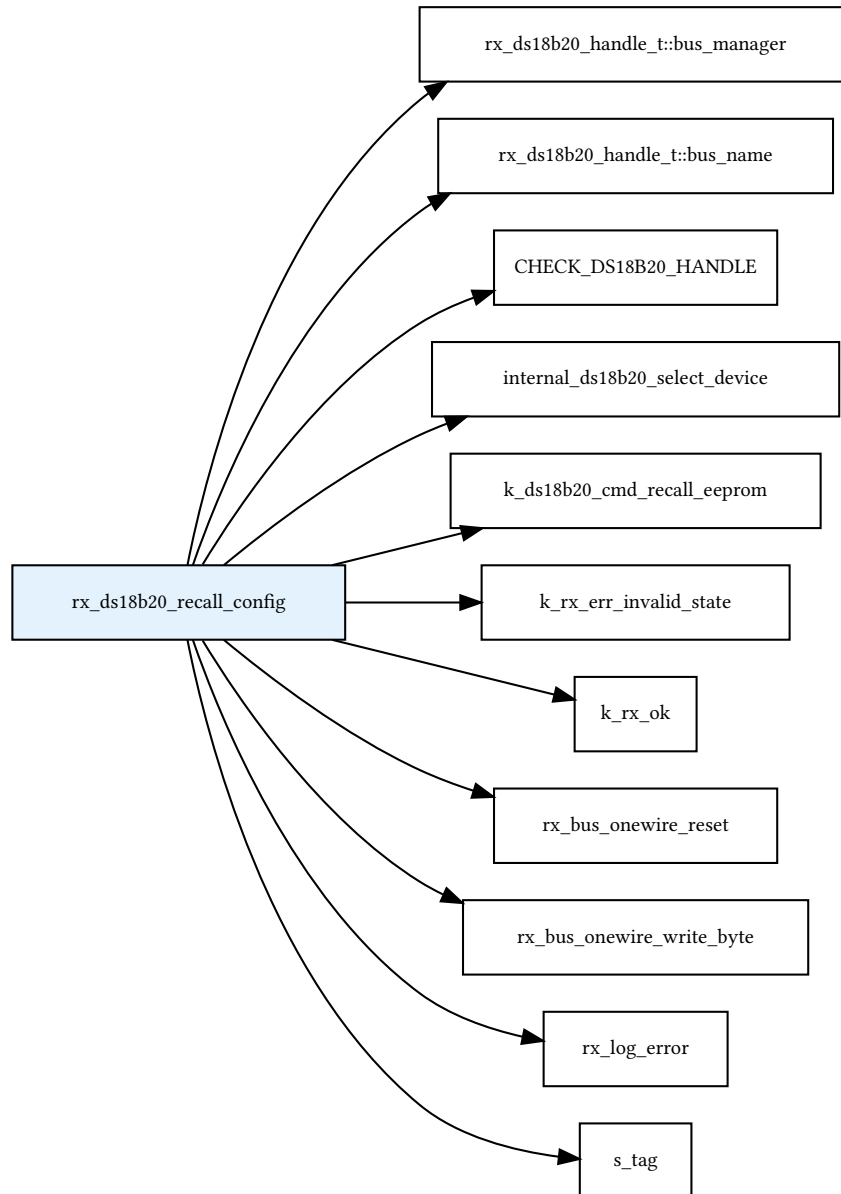


Figure 373: Call graph for `rx_ds18b20_recall_config`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:999`

Since Version 1.0.0

rx_ds18b20_read_power_mode

```
rx_err_t rx_ds18b20_read_power_mode(const rx_ds18b20_handle_t *handle, bool
*external_power)
```

Read DS18B20 power supply mode (external vs. parasitic).

Read power supply mode.

Issues the Read Power Supply (0xB4) command over the 1-Wire bus and reads back a single bit to determine whether the DS18B20 is operating on an external voltage supply or in parasitic (bus-powered) mode.

Per the DS18B20 datasheet: the sensor pulls the bus low for one read time slot to indicate parasitic power mode (0), or releases the bus (1) to indicate external power mode.

Algorithm steps:

1. Validate handle and output pointer are non-null; handle is initialized
2. Issue 1-Wire bus reset and verify device presence pulse
3. Select the target device (Match ROM or Skip ROM depending on config)
4. Write the Read Power Supply command byte (0xB4)
5. Read one bit from the bus (0 = parasitic, 1 = external)
6. Write result to *external_power

Parameters:

Direction	Name	Description
in	handle	Sensor handle (must be initialized, non-null)
out	external_power	Set to true if device uses external power supply, false if device uses parasitic (bus-powered) mode; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Power mode successfully read
k_rx_err_null_ptr	handle or external_power pointer is nullptr
k_rx_err_invalid_state	handle not initialized or device not present on bus
k_rx_err_bus_error	1-Wire bus reset, write, or read operation failed

Pre: handle must be initialized via rx_ds18b20_init()

Pre: DS18B20 device must be physically connected to the bus

Post: *external_power reflects the device power supply mode on k_rx_ok

Post: handle state is unchanged

Note: Not thread-safe; caller must serialize access to the same handle

Warning: Some operations (Copy Scratchpad) are unreliable in parasitic power mode

Example:: `boolext_power=false; rx_err_t err=rx_ds18b20_read_power_mode(&sensor,&ext_power);
if(err!=k_rx_ok&&!ext_power)
{ rx_log_warn("app","DS18B20inparasiticmode:EEPROMwritesmayfail"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks/initialized, device present), 2 postconditions

See also: `rx_ds18b20_save_config()` EEPROM write (requires external power),
`rx_ds18b20_init()` Initialize sensor handle

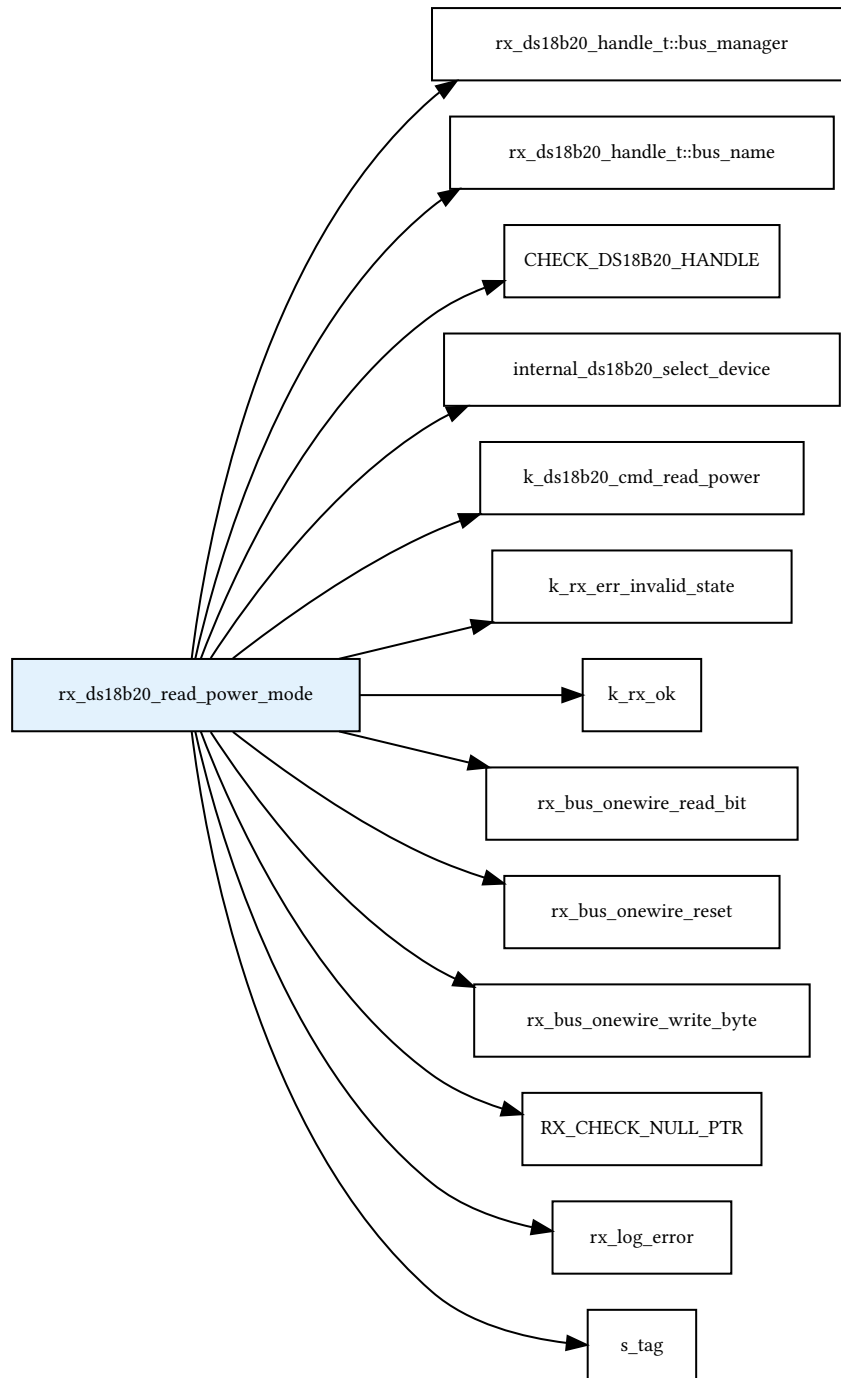


Figure 374: Call graph for `rx_ds18b20_read_power_mode`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1083`

Since Version 1.0.0

—

rx_ds18b20_read_scratchpad

```
rx_err_t rx_ds18b20_read_scratchpad(rx_ds18b20_handle_t *handle, uint8_t  
scratchpad[k_ds18b20_scratchpad_bytes])
```

Read the full 9-byte scratchpad register from DS18B20 sensor.

Read scratchpad memory.

Reads all nine bytes of the DS18B20 scratchpad memory and returns the raw buffer to the caller. The scratchpad layout is:

CRC is verified internally by internal_ds18b20_read_scratchpad_raw() before the buffer is returned. This function is a thin public wrapper around the internal helper.

Algorithm steps:

1. Validate handle and scratchpad buffer pointer are non-null
2. Verify handle is initialized
3. Delegate to internal_ds18b20_read_scratchpad_raw() which issues the 1-Wire reset, Match/Skip ROM selection, Read Scratchpad (0xBE) command, reads 9 bytes, and verifies CRC-8

Parameters:

Direction	Name	Description
inout	handle	Sensor handle (must be initialized, non-null)
out	scratchpad	Buffer of exactly k_ds18b20_scratchpad_bytes (9) bytes to receive the raw scratchpad contents; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Scratchpad successfully read and CRC verified
k_rx_err_null_ptr	handle or scratchpad buffer is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_crc_failure	Scratchpad CRC-8 check failed (data corrupted)
k_rx_err_bus_error	1-Wire bus communication failure

Pre: handle must be initialized via rx_ds18b20_init()

Pre: scratchpad must point to a buffer of at least k_ds18b20_scratchpad_bytes bytes

Post: scratchpad0..8 contains valid data on k_rx_ok

Post: CRC of scratchpad bytes 0-7 matches byte 8 on k_rx_ok

Note: Not thread-safe; caller must serialize access to the same handle

Note: Use `rx_ds18b20_read_temperature()` for converted Celsius value

Example:: `uint8_t raw[k_ds18b20_scratchpad_bytes];`
`rx_err_t err=rx_ds18b20_read_scratchpad(&sensor,raw); if(err!=k_rx_ok)`
`{ uint8_t config_reg=raw[4]; }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: `rx_ds18b20_read_temperature()` High-level temperature read returning float Celsius, `rx_ds18b20_read_temperature_raw()` Read raw 16-bit temperature with masking

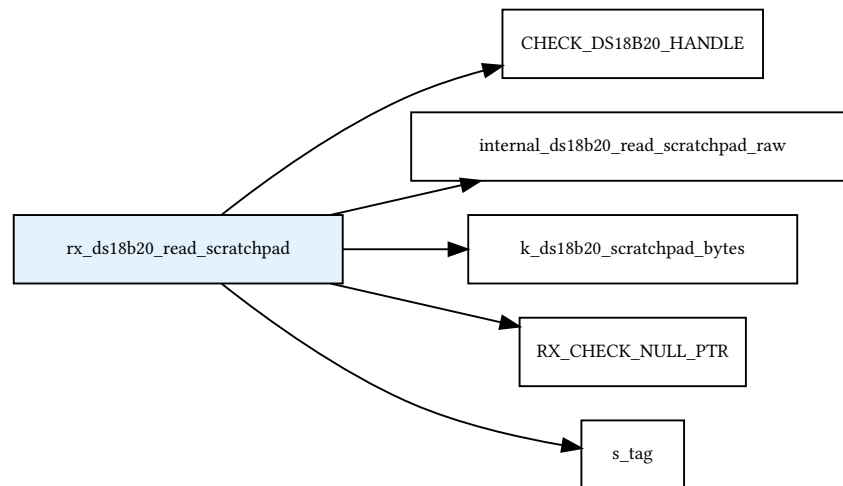


Figure 375: Call graph for `rx_ds18b20_read_scratchpad`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1185`

Since Version 1.0.0

—

`rx_ds18b20_get_conversion_time_ms`

```
uint32_t rx_ds18b20_get_conversion_time_ms(const rx_ds18b20_handle_t *handle)
```

Get temperature conversion time for current resolution.

Get conversion time for current resolution.

Returns the required wait time after triggering conversion based on the configured resolution (9-bit: 94ms, 10-bit: 188ms, 11-bit: 375ms, 12-bit: 750ms).

Parameters:

Direction	Name	Description
-----------	------	-------------

in	handle	Sensor handle
----	--------	---------------

Returns: Conversion time in milliseconds, or invalid value if handle is nullptr/uninitialized

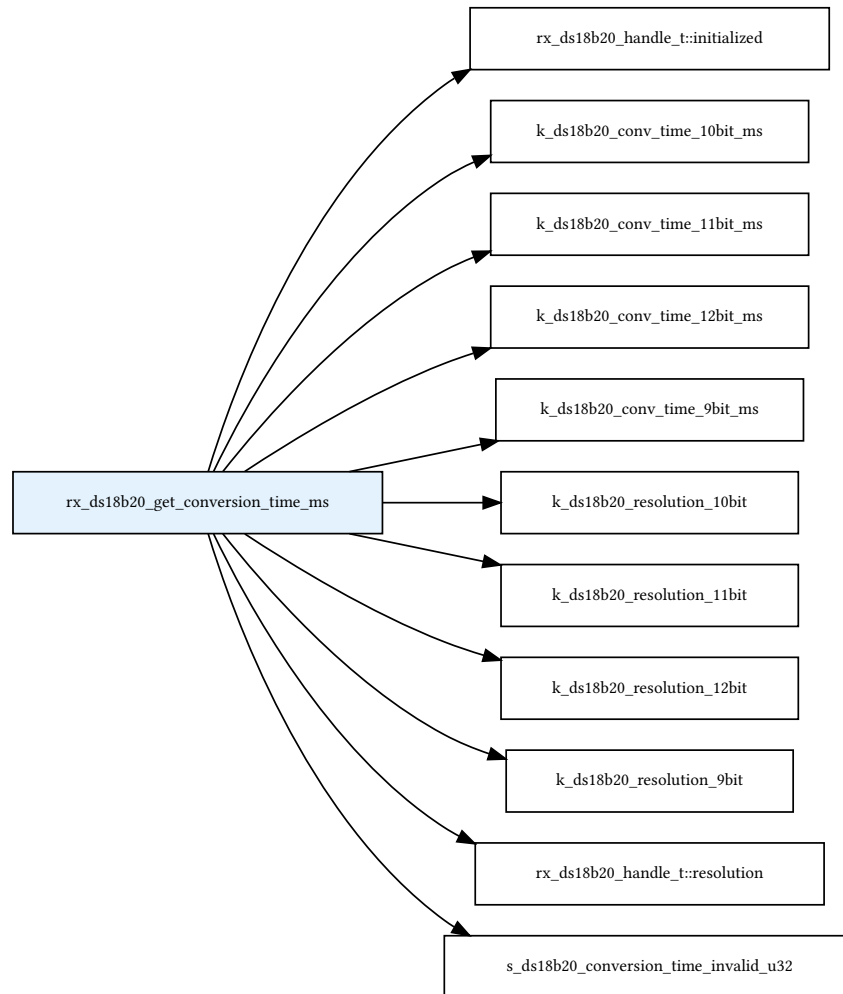


Figure 376: Call graph for `rx_ds18b20_get_conversion_time_ms`

Defined in `libs/rx_ds18b20/src/rx_ds18b20.c:1204`

rx_encoder_tpu.h

TPU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

Hardware-accelerated quadrature encoder interface using the RX72N Timer Pulse Unit (TPU) peripheral in phase counting mode. Provides zero-CPU-overhead encoder counting with automatic edge detection and direction sensing for the STAR robot's rear wheel encoders.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"
- "rx_mtu_encoder.h"
- "rx_tpu.h"

rx_tpu_encoder_init

```
rx_err_t rx_tpu_encoder_init(const rx_tpu_encoder_config_t *config)
```

Initialize TPU channel for hardware quadrature encoder counting.

Configures a TPU channel in phase counting mode (4x decoding) and initializes software overflow tracking state. After initialization, the encoder is counting and ready for position/velocity reads.

Initialization steps:

1. Validate configuration parameters
2. Initialize TPU HAL in phase counting mode 4
3. Start the TPU counter
4. Initialize software overflow detection state

Parameters:

Direction	Name	Description
in	config	Pointer to encoder configuration

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, encoder initialized and counting
k_rx_err_null_ptr	config is nullptr
k_rx_err_invalid_arg	Invalid channel or counts_per_rev == 0
k_rx_err_invalid_state	Channel already initialized

Pre: GPIO pins for TCLKA-D configured as peripheral I/O

Pre: Encoder physically connected

Post: TPU channel in phase counting mode, counter running

Post: Software state zeroed (count=0, position=0)

Note: Thread Safety: Not thread-safe, call during init only] **See also:** rx_tpu_encoder_deinit()
Release resources, rx_tpu_encoder_read_count() Read position after init

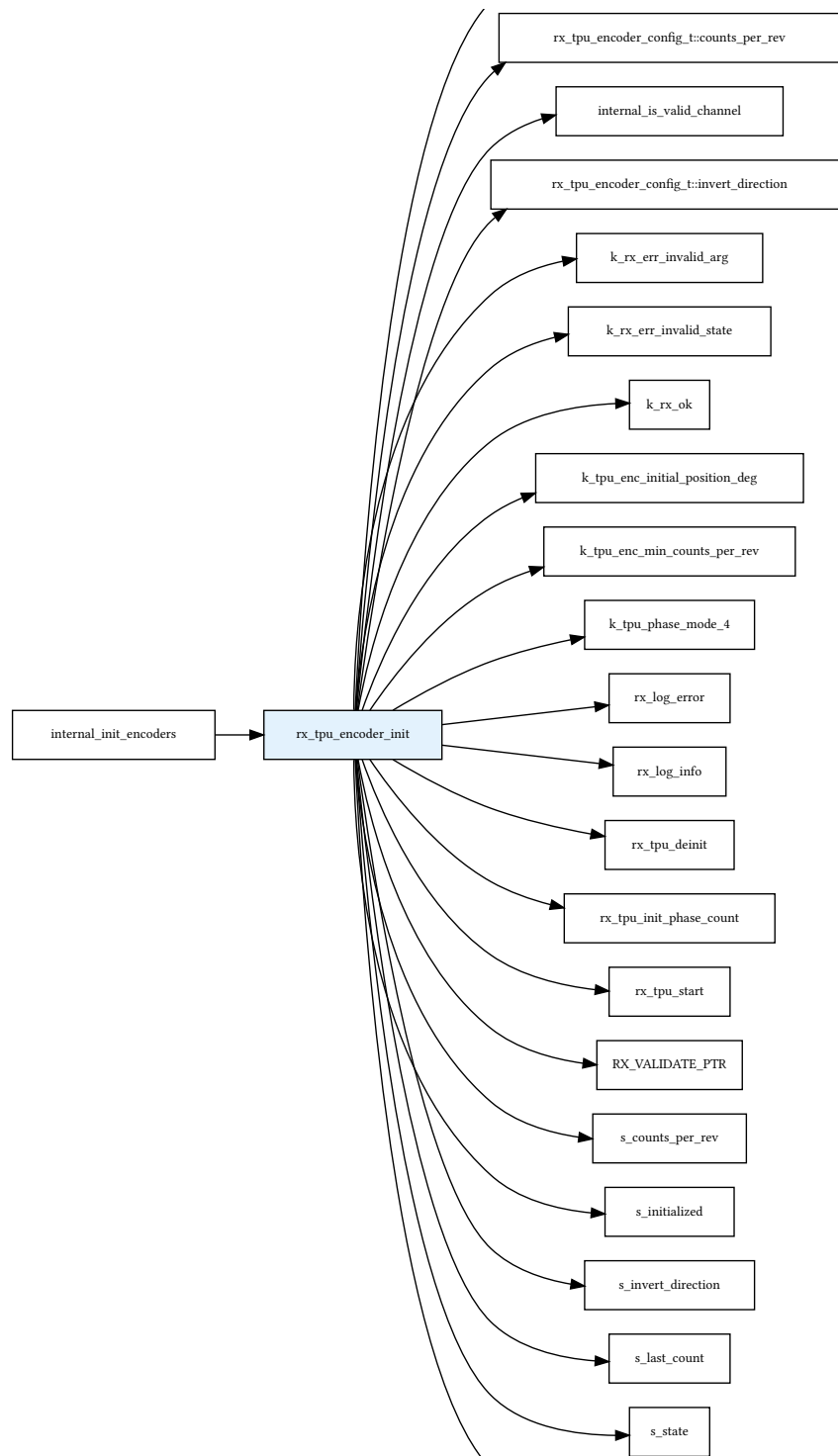


Figure 377: Call graph for rx_tpu_encoder_init

_Defined in libs/rx_encoder/inc/rx_encoder_tpu.h:185_

Since Version 1.0.0

—

rx_tpu_encoder_read_raw

```
rx_err_t rx_tpu_encoder_read_raw(rx_tpu_channel_t channel, uint16_t *count)
```

Read raw 16-bit counter value from TPU channel.

Reads the current TCNT value without overflow handling. For position tracking with overflow correction, use rx_tpu_encoder_read_count().

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
out	count	Pointer to store raw 16-bit count (0-65535)

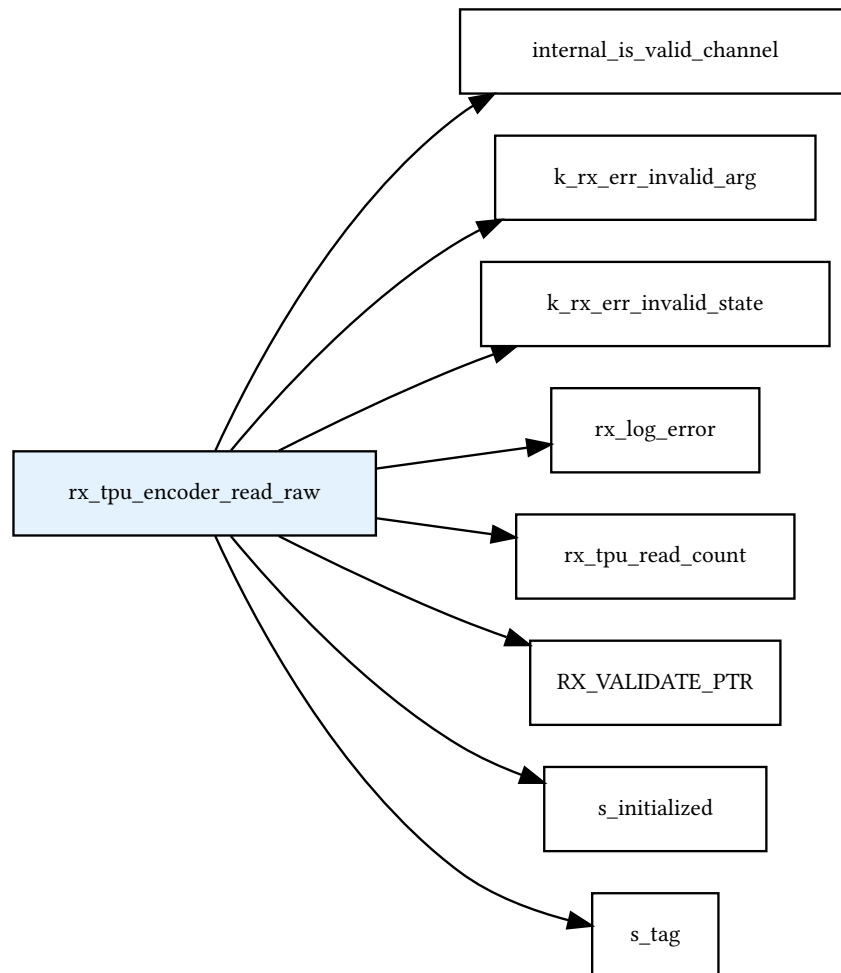
Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	count is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized via rx_tpu_encoder_init()

Post: count contains current TCNT register value

Note: Thread Safety: Safe (single volatile register read)] **See also:**
rx_tpu_encoder_read_count() Read with overflow handling

Figure 378: Call graph for `rx_tpu_encoder_read_raw`

_Defined in `libs/rx\_encoder/inc/rx\_encoder\_tpu.h:211_`

Since Version 1.0.0

—

rx_tpu_encoder_read_count

```
rx_err_t rx_tpu_encoder_read_count(rx_tpu_channel_t channel, rx_encoder_state_t
*state)
```

Read encoder count with automatic 16-bit overflow handling.

Reads the current 16-bit hardware counter and updates the 32-bit accumulated count with automatic overflow/underflow correction. Also computes derived position (degrees) and revolution count.

Must be called frequently enough to catch overflows: at 210 RPM max motor speed with 1364 counts/rev, the safe interval is 6.8 seconds. The 250 Hz control loop provides a 1666x safety margin.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
out	state	Pointer to encoder state structure to update

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, state updated
k_rx_err_null_ptr	state is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized via rx_tpu_encoder_init()

Pre: Called frequently enough to catch overflows

Post: state->total_count updated with overflow correction

Post: state->last_raw_count updated

Post: state->revolutions and position_deg computed

Note: Thread Safety: Not thread-safe for same channel] **See also:**
rx_tpu_encoder_read_velocity() Calculate velocity

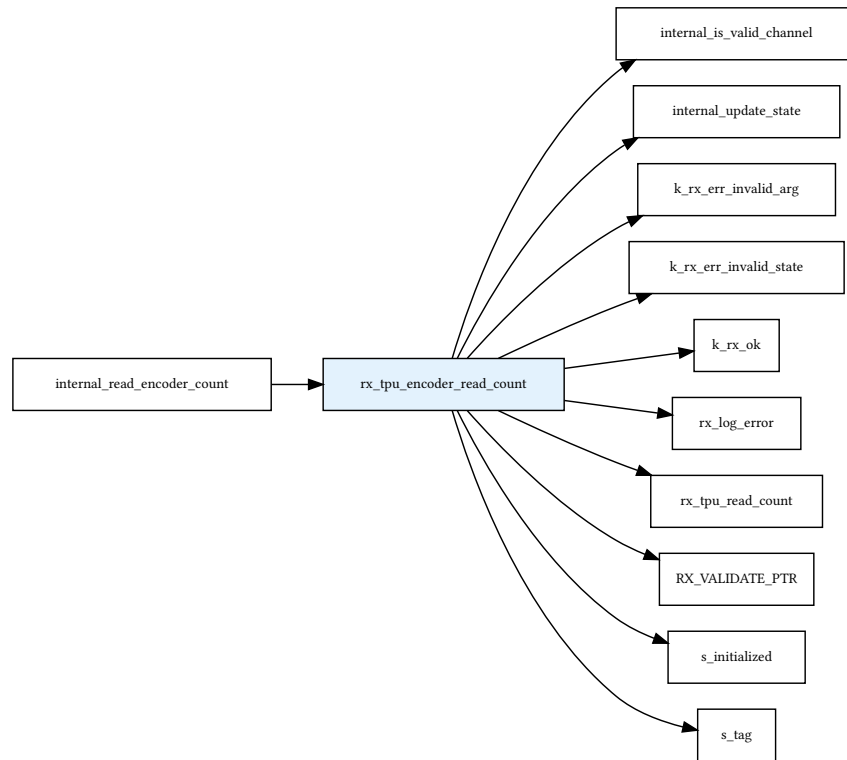


Figure 379: Call graph for rx_tpu_encoder_read_count

_Defined in libs/rx_encoder/inc/rx_encoder_tpu.h:246_

Since Version 1.0.0

—

rx_tpu_encoder_read_velocity

```
rx_err_t rx_tpu_encoder_read_velocity(float *velocity_rps, float delta_time_s,
rx_tpu_channel_t channel)
```

Calculate encoder velocity in revolutions per second.

Derives angular velocity from the change in encoder position over a known time interval. Tracks previous count internally for delta calculation.

Parameters:

Direction	Name	Description
out	velocity_rps	Pointer to store velocity (rev/sec)
in	delta_time_s	Time since last call in seconds (must be > 0)
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	velocity_rps is nullptr
k_rx_err_invalid_arg	Invalid delta_time_s or channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized via rx_tpu_encoder_init()

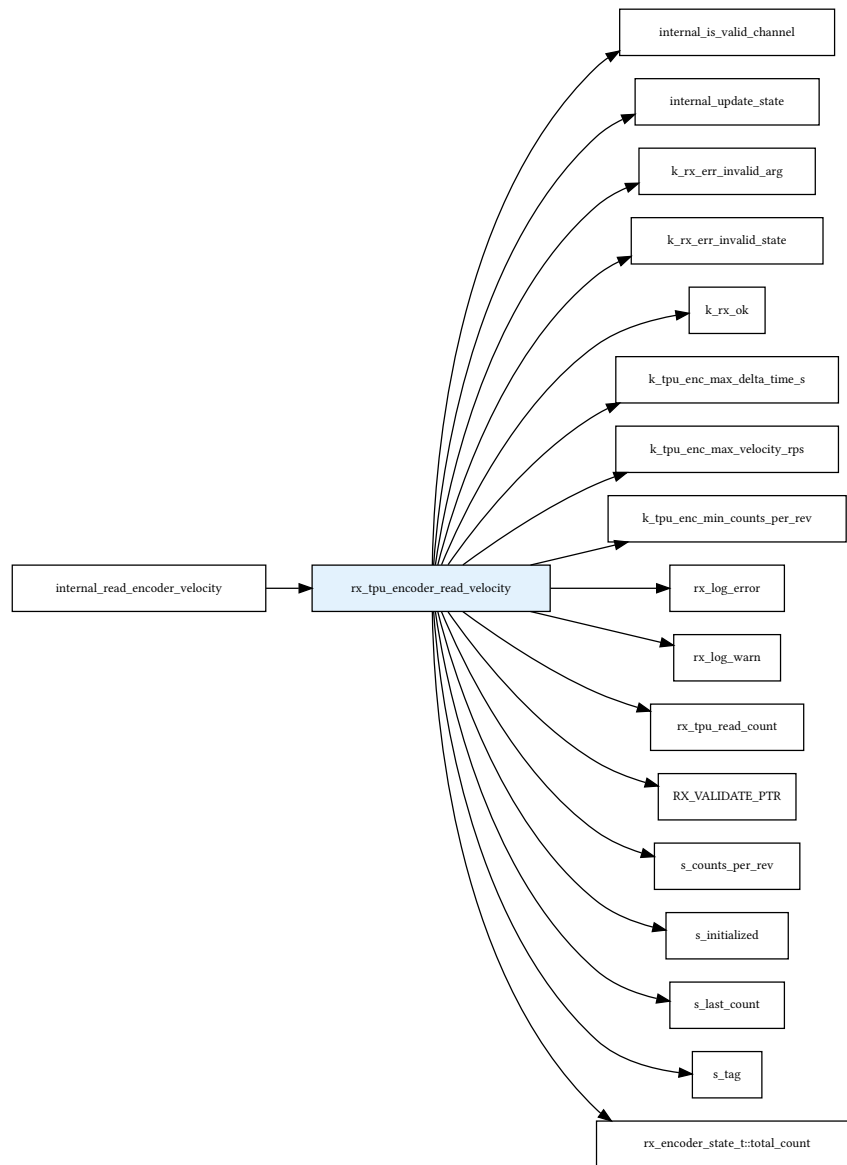
Pre: delta_time_s matches actual elapsed time

Post: velocity_rps contains calculated velocity

Post: Internal last_count updated for next delta

Note: Parameter order: output, time, channel (prevents swaps)

Note: Thread Safety: Not thread-safe for same channel] **See also:**
rx_tpu_encoder_read_count() Read position

Figure 380: Call graph for `rx_tpu_encoder_read_velocity`

_Defined in `libs/rx\_encoder/inc/rx\_encoder\_tpu.h:279_`

Since Version 1.0.0

—

rx_tpu_encoder_reset

```
rx_err_t rx_tpu_encoder_reset(rx_tpu_channel_t channel)
```

Reset encoder count to zero.

Resets both the TPU hardware counter and software accumulated state.

Parameters:

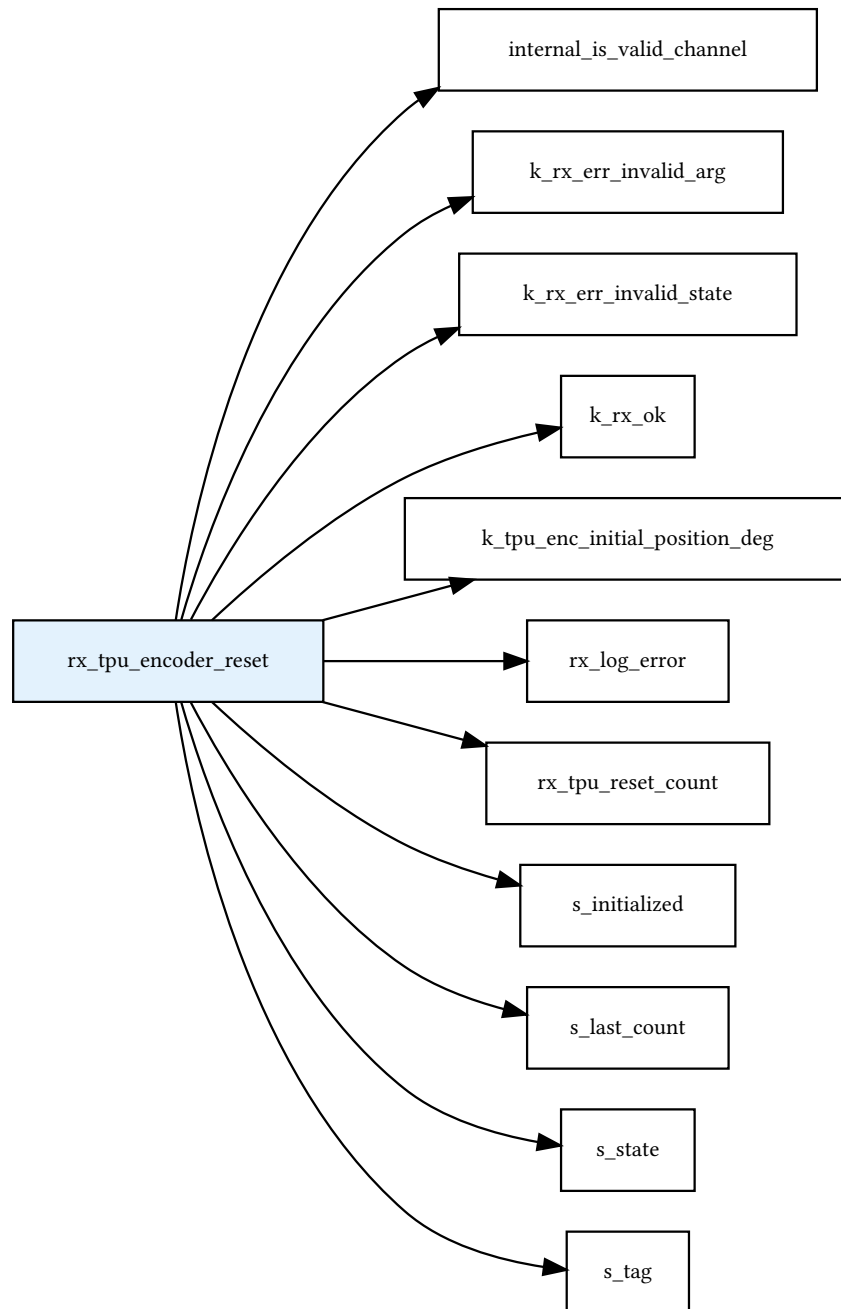
Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized

Post: TCNT=0, total_count=0, position=0] **See also:** rx_tpu_encoder_set_count() Set to specific value

Figure 381: Call graph for `rx_tpu_encoder_reset`

_Defined in `libs/rx\_encoder/inc/rx\_encoder\_tpu.h:300_`

Since Version 1.0.0

—

rx_tpu_encoder_set_count

```
rx_err_t rx_tpu_encoder_set_count(int32_t count, rx_tpu_channel_t channel)
```

Set encoder count to specific value.

Sets both hardware counter (low 16 bits) and software accumulated count.

Parameters:

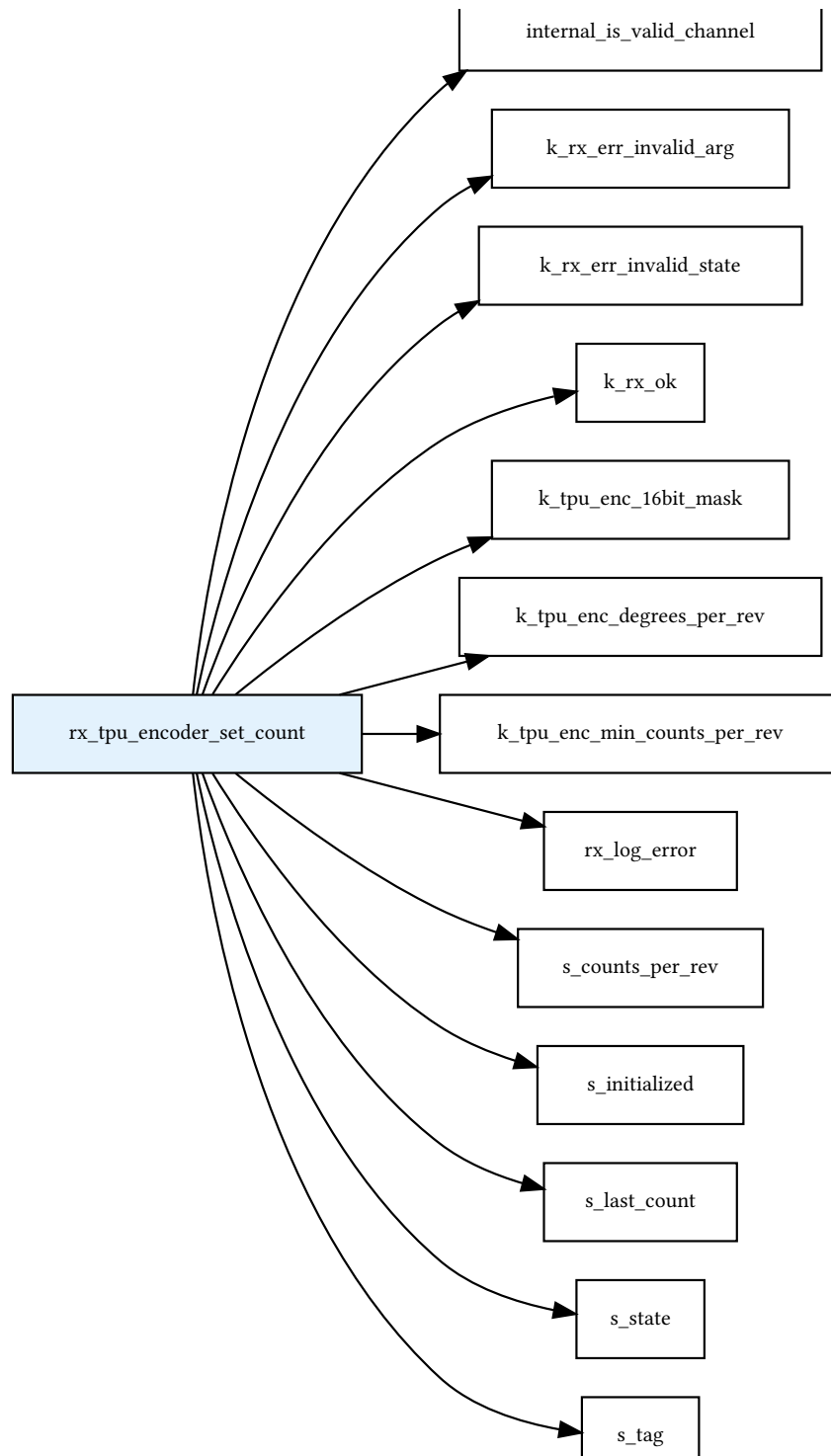
Direction	Name	Description
in	count	Count value to set
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized

Post: Software and hardware counts updated

Figure 382: Call graph for `rx_tpu_encoder_set_count`

_Defined in libs/rx_encoder/inc/rx_encoder_tpu.h:321_

Since Version 1.0.0

—

rx_tpu_encoder_deinit

```
rx_err_t rx_tpu_encoder_deinit(rx_tpu_channel_t channel)
```

Deinitialize TPU encoder.

Stops counter and releases resources. Can be reinitialized after.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel previously initialized

Post: Counter stopped, channel marked uninitialized] **See also:** rx_tpu_encoder_init()
Reinitialize after deinit

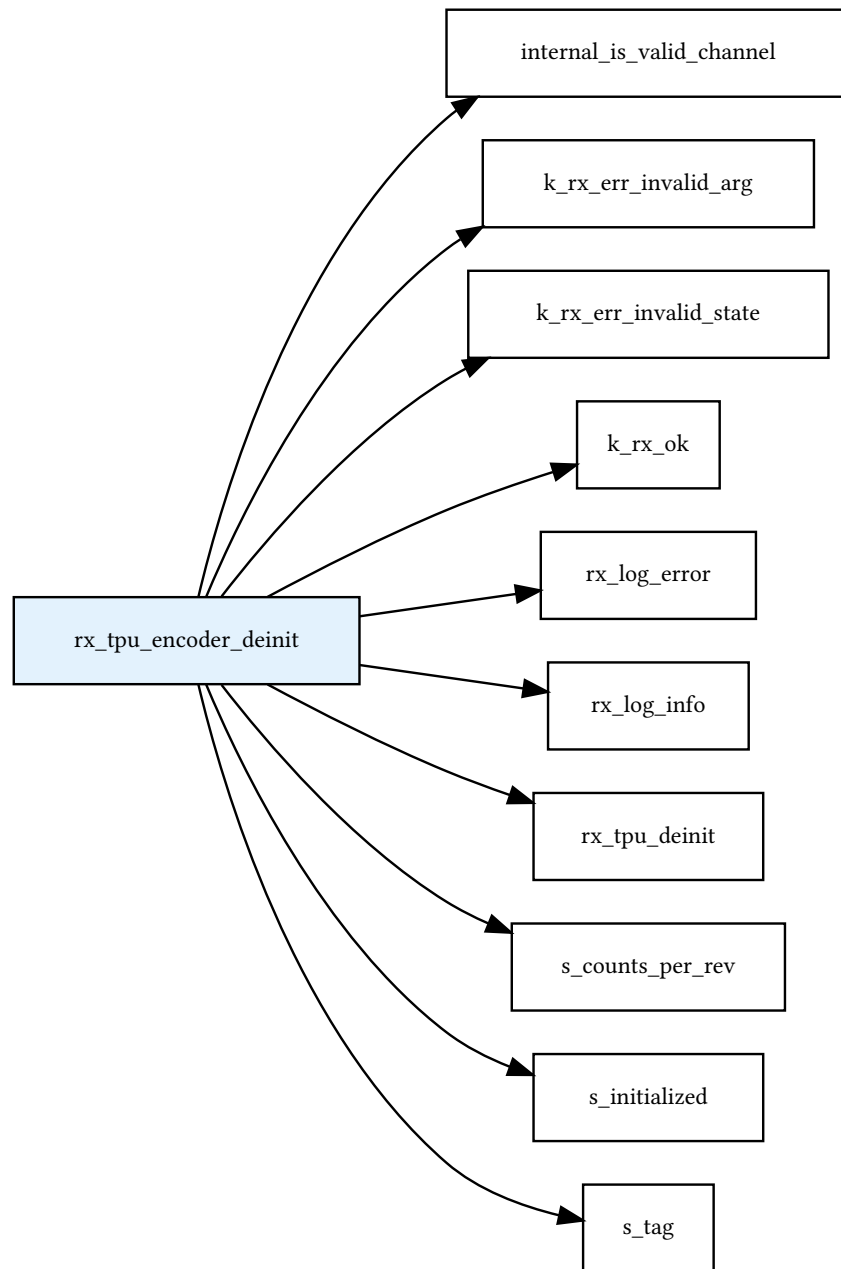


Figure 383: Call graph for rx_tpu_encoder_deinit

_Defined in `libs/rx\_encoder/inc/rx\_encoder\_tpu.h:342_`

Since Version 1.0.0

—

rx_mtu_encoder.h

MTU Quadrature Encoder Driver for RX72N (Hardware Phase Counting Mode).

Hardware-accelerated quadrature encoder interface using the RX72N Multi-Function Timer (MTU) peripheral in phase counting mode. Provides zero-CPU-overhead encoder counting with automatic edge detection and direction sensing.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"
- "rx_mtu.h"

rx_encoder_init

```
rx_err_t rx_encoder_init(const rx_encoder_config_t *config)
```

Initialize MTU channel for hardware quadrature encoder counting.

Configures a Multi-Function Timer (MTU) channel in phase counting mode for hardware-accelerated quadrature encoder decoding. This function:

1. Validates configuration parameters
2. Configures MTU pins (MTCLKA, MTCLKB) as inputs
3. Sets MTU mode to phase counting with 4x decoding
4. Initializes software state tracking (zero position)
5. Starts hardware counter

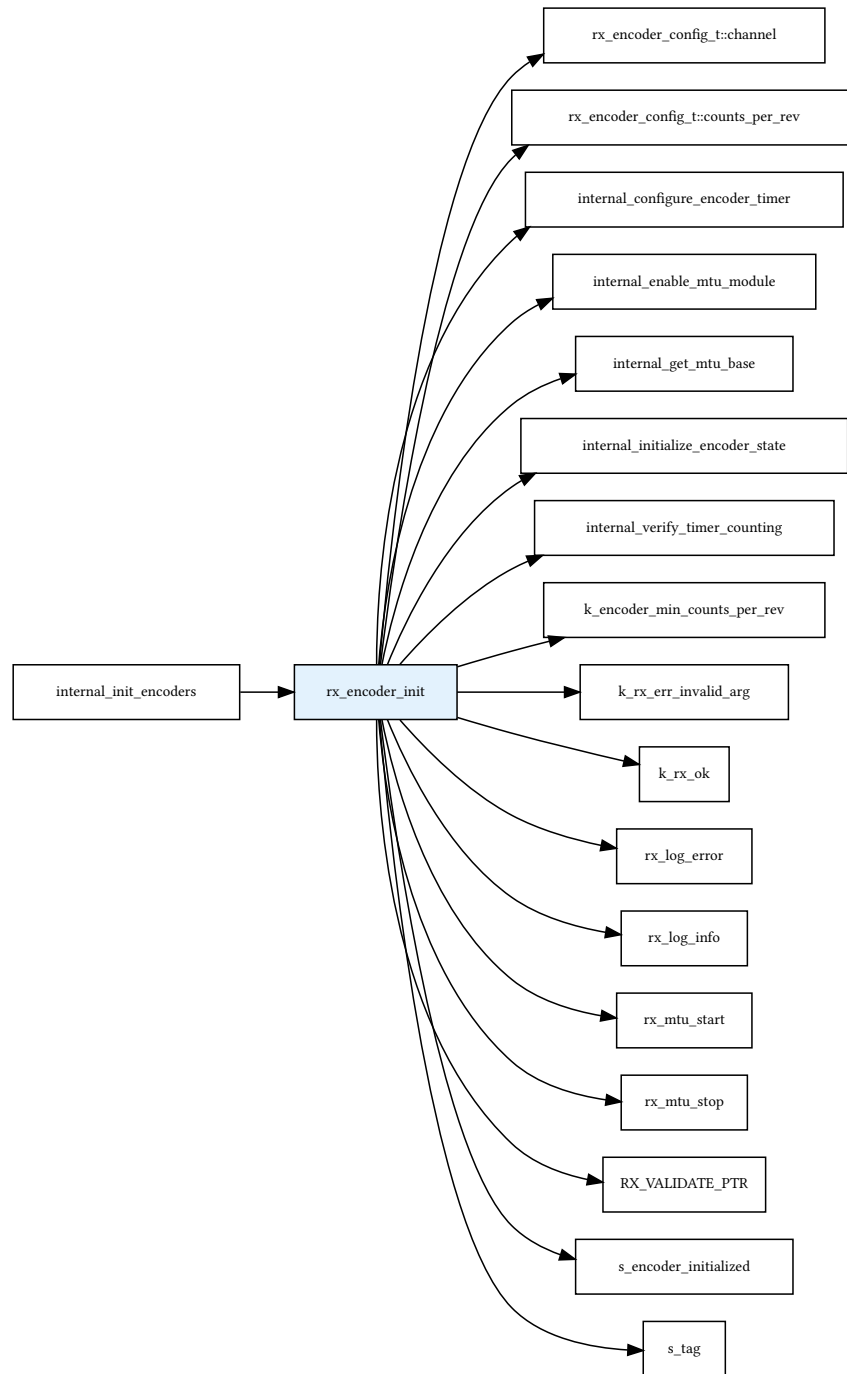


Figure 384: Call graph for `rx_encoder_init`

_Defined in `libs/rx\_encoder/inc/rx\_mtu\_encoder.h:791_`

rx_encoder_read_raw

```
rx_err_t rx_encoder_read_raw(rx_mtu_channel_t channel, uint16_t *count)
```

Read raw encoder count.

Returns the current 16-bit counter value. Does not handle overflow - use rx_encoder_read_count() for overflow handling.

Read raw encoder count.

Reads the current value of the MTU TCNT (Timer Counter) register without any processing. This provides the raw 16-bit hardware count which wraps at 65536. For position tracking with wraparound handling, use rx_encoder_read_count() instead.

Use Cases:

- Debugging: Verify hardware is counting encoder edges
- Diagnostics: Check for stuck counter (hardware fault)
- Testing: Validate encoder wiring and signal quality
- Low-level access: When software wraparound tracking not needed

Hardware Behavior:

- Counter increments on forward encoder motion (Phase A leads Phase B)
- Counter decrements on reverse encoder motion (Phase B leads Phase A)
- Counter wraps: 65535 -> 0 (forward) or 0 -> 65535 (reverse)
- No software processing applied to value

Parameters:

Direction	Name	Description
in	channel	MTU channel
out	count	Pointer to store raw count (0-65535)
in	channel	MTU channel to read Must be valid channel (0-4, 6-7, excluding 5) Must be initialized via rx_encoder_init()
out	count	Pointer to store raw 16-bit counter value Must not be NULL (validated) Value range: [0, 65535] Value unchanged if function returns error

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, count contains current TCNT value
k_rx_err_null_ptr	count pointer is nullptr
k_rx_err_invalid_arg	channel invalid (not 0-4, 6-7)
k_rx_err_invalid_state	Encoder not initialized on this channel

Pre: channel must be initialized via rx_encoder_init()

Pre: count must point to valid uint16_t variable

Post: On success: *count contains current TCNT register value

Post: Hardware counter unchanged (read-only operation)

Note: Thread-safe for read-only access (atomic 16-bit register read)

Note: Very fast operation (200 ns) - single register read

Note: Does NOT handle wraparound - use rx_encoder_read_count() for that

Performance:: Execution time: 0.5 us @ 240 MHz Safe to call at very high frequency (> 10 kHz tested)

Example - Diagnostic Check:: uint16_t raw_count;
rx_err_t err = rx_encoder_read_raw(k_mtu_channel_1, &raw_count); if (err == k_rx_ok)
{ rx_log_info("DIAG", "Rawcounter: %u", raw_count);

tx_thread_sleep(10); uint16_t new_count; rx_encoder_read_raw(k_mtu_channel_1, &new_count);
if (new_count == raw_count) { rx_log_warn("DIAG", "Encoder not moving - check wiring"); } }

See also: rx_encoder_read_count() Read position with wraparound handling,
rx_encoder_init() Must call before reading

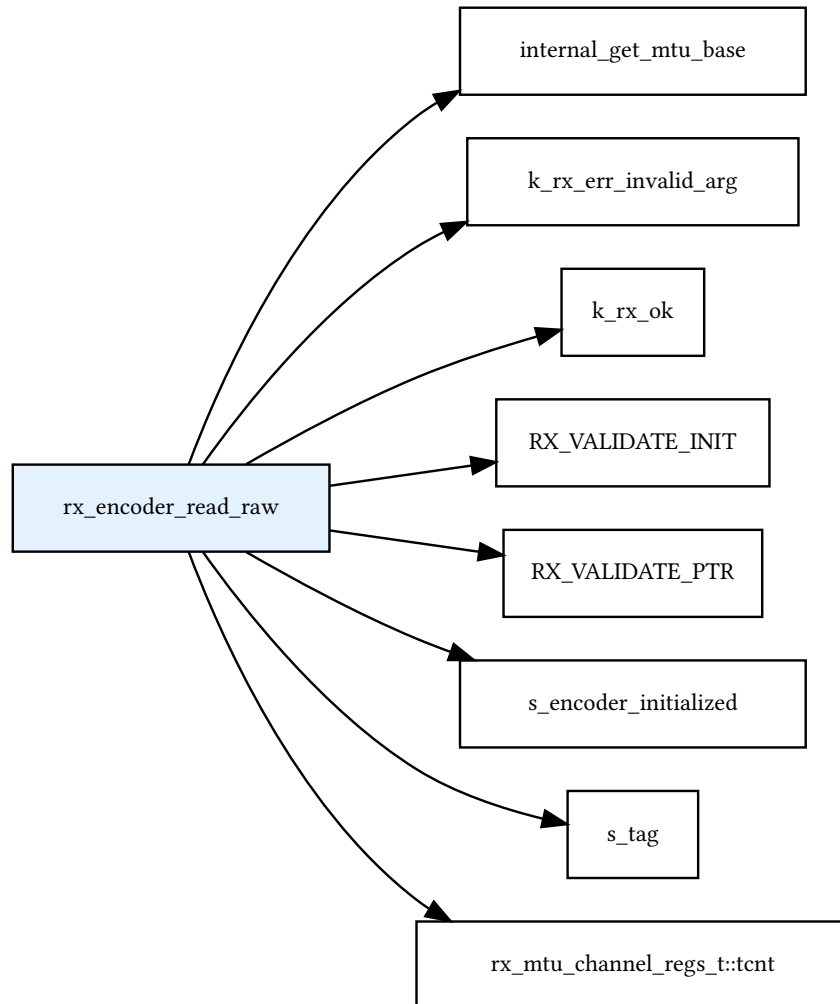


Figure 385: Call graph for rx_encoder_read_raw

_Defined in libs/rx_encoder/inc/rx_mtu_encoder.h:807_

Since Version 1.0.0

—

rx_encoder_read_count

```
rx_err_t rx_encoder_read_count(rx_mtu_channel_t channel, rx_encoder_state_t
*state)
```

Read encoder count with automatic 16-bit overflow handling.

Reads the current 16-bit hardware counter value and updates the accumulated 32-bit signed count, automatically detecting and correcting for counter overflow/underflow. This is the primary function for encoder position tracking.

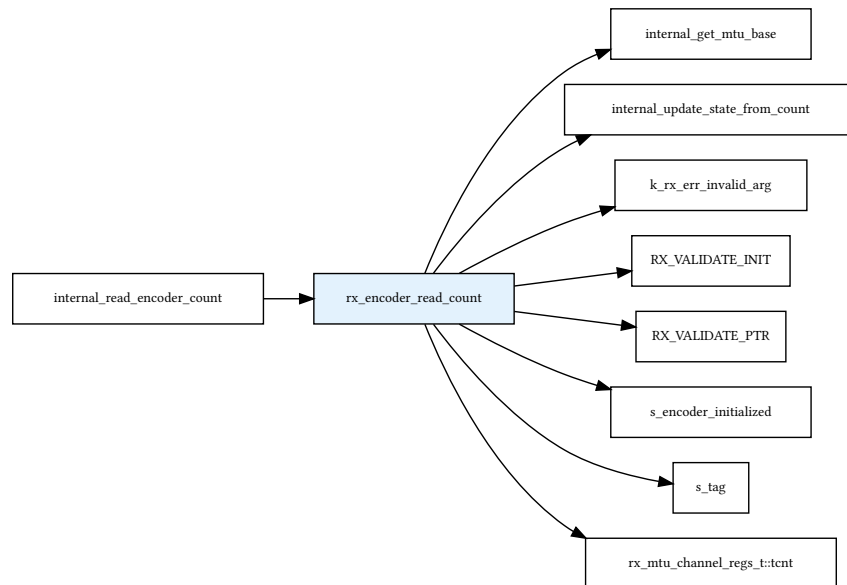


Figure 386: Call graph for rx_encoder_read_count

_Defined in libs/rx_encoder/inc/rx_mtu_encoder.h:1032_

rx_encoder_read_velocity

```
rx_err_t rx_encoder_read_velocity(float *velocity_rps, float delta_time_s,
rx_mtu_channel_t channel)
```

Read encoder velocity.

Calculates velocity based on count change over time period. Assumes periodic calls at known rate (e.g., 250Hz control loop).

NOTE: Parameter order changed to prevent accidental swapping of channel/delta_time_s.

Read encoder velocity.

Derives angular velocity by measuring the change in encoder position over a known time interval. This function tracks the previous count and calculates delta, then converts to revolutions per second based on the configured counts_per_revolution parameter.

Velocity Calculation Algorithm:

Where:

- $\Delta_count = total_count[n] - total_count[n-1]$ (count change since last call)

-counts_per_rev= encoder resolution (e.g., 1364 for STAR motors) -Delta t = time between calls (delta_time_s parameter)

STAR Project Example (100 Hz control loop):

Resolution and Accuracy:

- Velocity resolution @ 100 Hz: $1 \text{ count} / (1364 \times 0.01\text{s}) = 0.073 \text{ RPS (4.4 RPM)}$
- Velocity resolution @ 1 kHz: $1 \text{ count} / (1364 \times 0.001\text{s}) = 0.733 \text{ RPS (44 RPM)}$
- Higher sample rates -> lower resolution but faster response
- Lower sample rates -> higher resolution but slower response

State Tracking: Function maintains internal state (`s_last_count[channel]`) to calculate delta between calls. First call after init returns velocity based on change since initialization (typically zero).

Velocity resolution inversely proportional to `delta_time_s`

Parameters:

Direction	Name	Description
out	<code>velocity_rps</code>	Pointer to store velocity in revolutions per second
in	<code>delta_time_s</code>	Time since last call in seconds
in	<code>channel</code>	MTU channel
out	<code>velocity_rps</code>	Pointer to store calculated velocity in revolutions per second Must not be NULL (validated) Units: Revolutions per second (RPS) Positive: Forward rotation, negative: Reverse rotation Range: Typically $[-10, +10]$ RPS for STAR motors (± 600 RPM) Unrealistic values (> 100 RPS = 6000 RPM) trigger warning but succeed
in	<code>delta_time_s</code>	Time interval since last velocity measurement Must be > 0 and ≤ 10 seconds (validated to catch parameter swaps) Units: Seconds (s) Typical: 0.01s (100 Hz control loop) or 0.001s (1 kHz) Smaller values -> lower resolution, faster response Larger values -> higher resolution, slower response
in	<code>channel</code>	MTU channel to read Must be valid channel (0-4, 6-7, excluding 5) Must be initialized via <code>rx_encoder_init()</code>

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, velocity calculated and written to <code>*velocity_rps</code>
<code>k_rx_err_null_ptr</code>	<code>velocity_rps</code> pointer is nullptr
<code>k_rx_err_invalid_arg</code>	<code>delta_time_s</code> ≤ 0 or > 10 (possible parameter swap with channel)
<code>k_rx_err_invalid_arg</code>	channel invalid (not 0-4, 6-7)
<code>k_rx_err_invalid_state</code>	Encoder not initialized on this channel
<code>k_rx_err_invalid_state</code>	<code>counts_per_rev</code> < 1 (state corrupted)

Pre: channel must be initialized via `rx_encoder_init()`

Pre: `delta_time_s` must match actual time since last call for accurate results

Pre: velocity_rps must point to valid float variable

Post: On success: *velocity_rps contains calculated angular velocity

Post: Internal last_count updated for next delta calculation

Post: Warning logged if velocity exceeds 100 RPS (possible encoder fault)

Note: Not thread-safe, must call from single task or with external mutex

Note: First call after init returns zero velocity (no previous count to compare)

Note: Parameter order designed to prevent common bugs (output first, time second, channel last)

Note: Unrealistic velocity (> 100 RPS) logs warning but does not return error

Warning: delta_time_s must be accurate - timing errors propagate to velocity error

Warning: Very long delta_time (> 1s) may miss wraparounds if motor speed high

Performance:: Execution time: 3 us @ 240 MHz Suitable for 100 Hz - 10 kHz control loops STAR platform: Called at 100 Hz from PID velocity controller

Example - 100 Hz Control Loop:: void motor_control_task(void*arg)
 { rx_mtu_channel_t encoder_ch=k_mtu_channel_1; const float dt_s=0.01f;
 while(1){ float velocity_rps; rx_err_t err=rx_encoder_read_velocity(&velocity_rps,dt_s,encoder_ch);
 if(err!=k_rx_ok){ float pid_output=pid_compute(&pid,target_rps,velocity_rps,dt_s);
 motor_set_duty(&motor,pid_output); }
 tx_thread_sleep(10); } }

Example - Velocity Monitoring:: const float dt_s=0.1f; float velocity_rps;
 rx_err_t err=rx_encoder_read_velocity(&velocity_rps,dt_s,k_mtu_channel_1); if(err!=k_rx_ok)
 { float velocity_rpm=velocity_rps*60.0f; rx_log_info("MOTOR","Velocity:
 %.1fRPM(%.2fRPS)",velocity_rpm,velocity_rps);
 if(fabsf(velocity_rps)>5.0f){ rx_log_warn("MOTOR","Motor overspeed detected"); } }

Example - Error Handling:: float velocity;
 rx_err_t err=rx_encoder_read_velocity(&velocity,0.01f,k_mtu_channel_1);
 switch(err){ case k_rx_ok: break; case k_rx_err_invalid_arg:
 rx_log_error("APP","Invalid delta time or channel"); break; case k_rx_err_invalid_state:
 rx_log_error("APP","Encoder not initialized or state corrupted"); break; default:
 rx_log_error("APP","Unexpected error: %d",err); break; }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 3 preconditions (NULL check, time range check, initialized check) Rule 5: [OK] 2 postconditions (velocity calculated, last_count updated) Rule 7: [OK] Division guard (counts_per_rev validated before division)

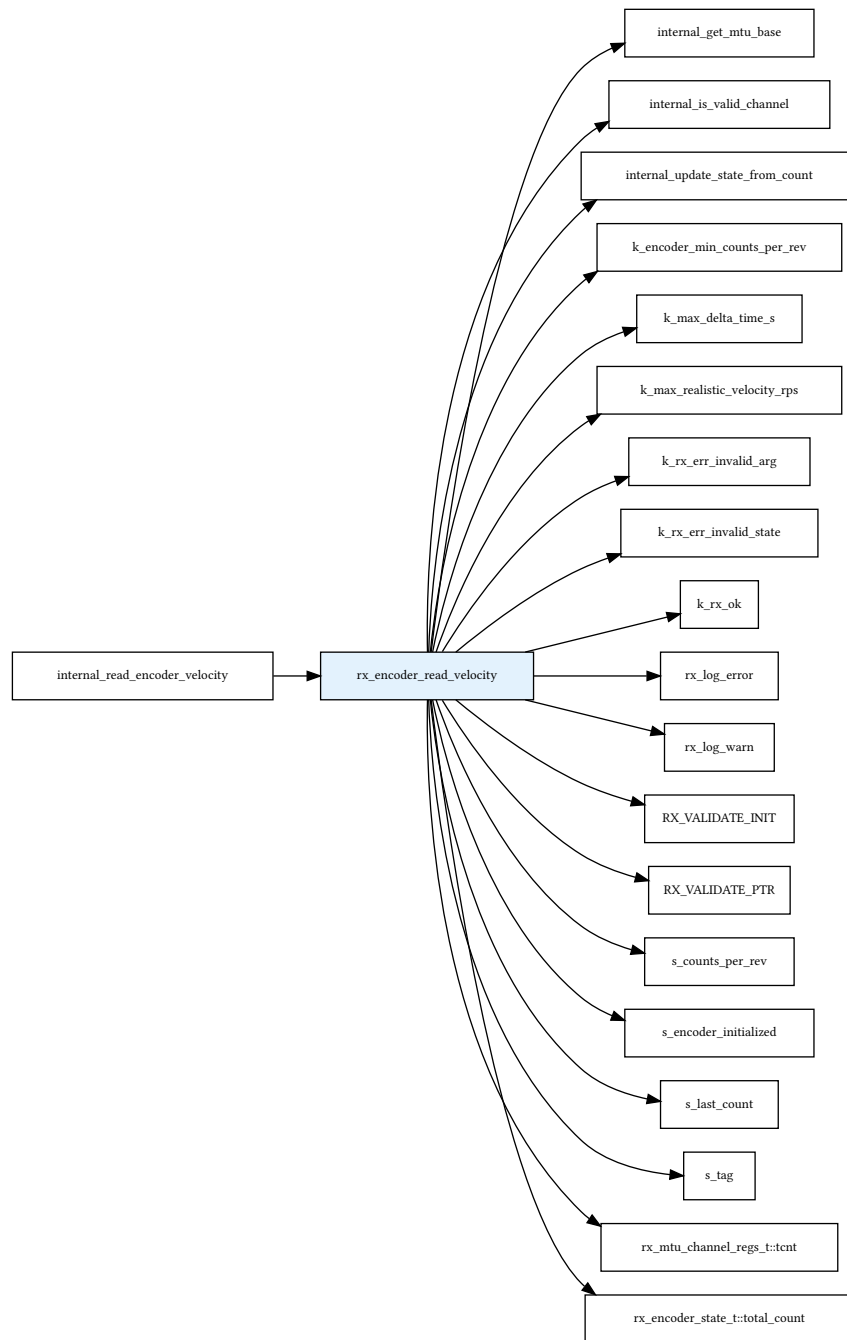
Example:

Motor: 210RPMnominal=3.5RPS
Encoder: 1364counts/rev(341PPRx4xdecoding)
Loopperiod: 10ms=0.01seconds

Expectedcountchangepercall:
 $\text{delta_count} = 3.5\text{RPS} \times 1364\text{counts/rev} \times 0.01\text{s} = 47.74\text{counts}$

Ifmeasureddelta_count=48:
 $\text{velocity} = 48 / (1364 \times 0.01) = 3.52\text{RPS} = 211.2\text{RPM}$

See also: rx_encoder_init() Must call before using this function,
rx_encoder_read_count() Alternative for position-based velocity, rx_pid_compute() PID controller uses this for feedback

Figure 387: Call graph for `rx_encoder_read_velocity`

_Defined in `libs/rx\_encoder/inc/rx\_mtu\_encoder.h:1052_`

Since Version 1.0.0

rx_encoder_reset

```
rx_err_t rx_encoder_reset(rx_mtu_channel_t channel)
```

Reset encoder count to zero.

Resets hardware counter and accumulated count.

Reset encoder count to zero.

Resets both hardware counter (TCNT register) and software state to zero, establishing a new zero reference position. This is typically used during system initialization, homing sequences, or when the robot reaches a known calibration point.

Reset Actions:

1. Hardware: TCNT register set to 0
2. Software: total_count = 0
3. Software: revolutions = 0
4. Software: position_deg = 0.0deg
5. Software: last_raw_count = 0
6. Velocity: s_last_count = 0 (next velocity call measures from this point)

Use Cases:

- Homing: Reset when limit switch or home sensor triggered
- Calibration: Reset at known mechanical position
- Error recovery: Clear accumulated position after fault
- Testing: Reset between test runs

Hardware counter continues incrementing after reset if motor moving

Parameters:

Direction	Name	Description
in	channel	MTU channel
in	channel	MTU channel to reset Must be valid channel (0-4, 6-7, excluding 5) Must be initialized via rx_encoder_init()

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, encoder reset to zero
k_rx_err_invalid_arg	channel invalid (not 0-4, 6-7)
k_rx_err_invalid_state	Encoder not initialized on this channel

Pre: channel must be initialized via rx_encoder_init()

Pre: No other tasks reading encoder during reset (race condition possible)

Post: Hardware TCNT = 0

Post: Software state zeroed (count=0, position=0deg, rev=0)

Post: Next velocity measurement starts from zero reference

Note: Not thread-safe, ensure exclusive access during reset

Note: Motor may be moving during reset - count starts accumulating immediately

Note: Does NOT physically stop motor (use rx_motor_stop for that)

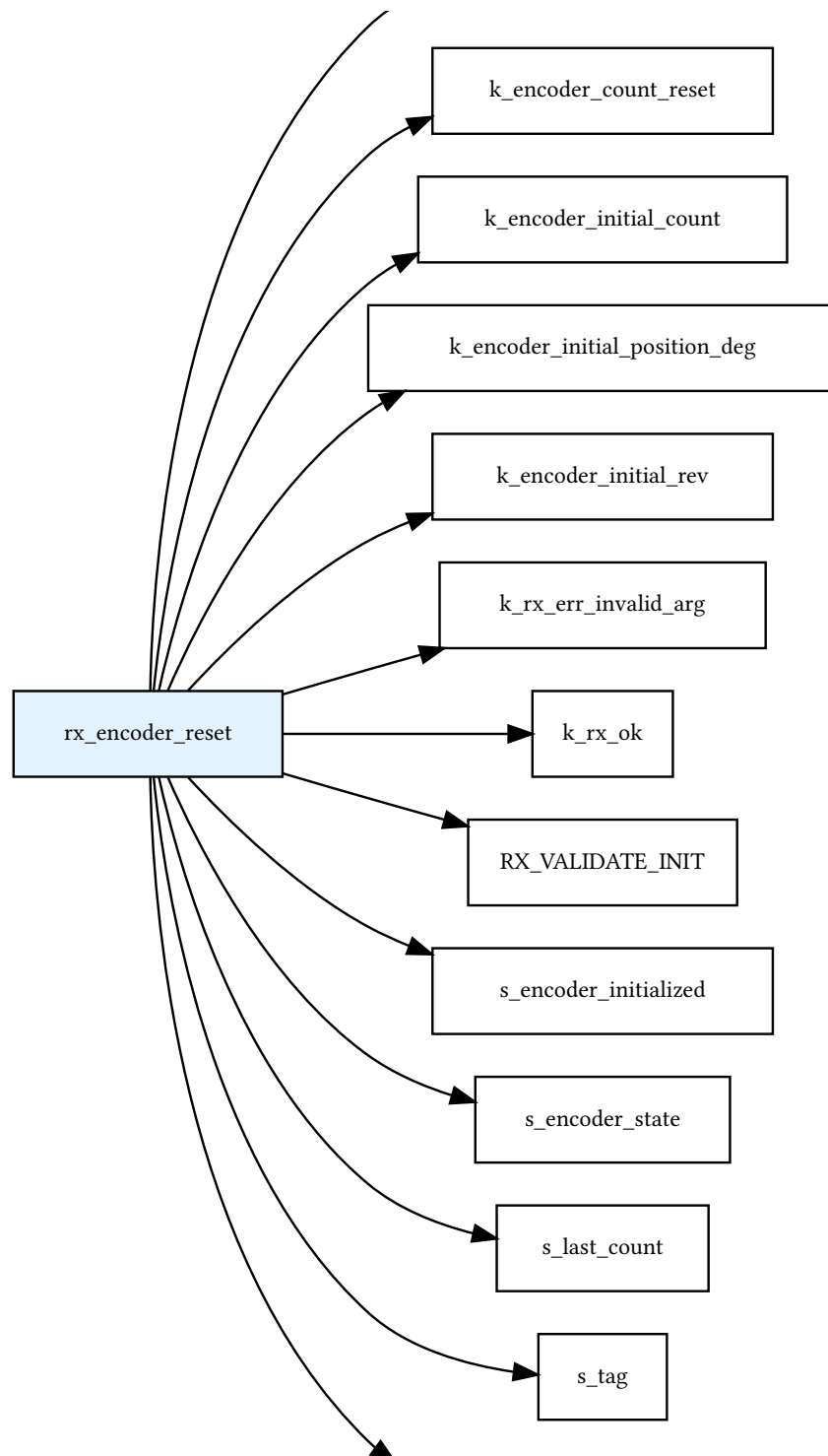
Warning: Do not call while other tasks are reading encoder (race condition)

Performance:: Execution time: 1 us @ 240 MHz (single register write + state clear)

Example - Homing Sequence:: motor_set_duty(&motor,-20.0f);
 while(gpio_read(LIMIT_SWITCH_PIN)==GPIO_HIGH){ tx_thread_sleep(1); }
 motor_stop(&motor,false); rx_encoder_reset(k_mtu_channel_1);
 rx_log_info("APP","Homingcomplete-encoderresettozero");
Example - Periodic Calibration:: voidcalibrate_encoder(rx_mtu_channel_tchannel)
 { rx_encoder_state_tstate; rx_encoder_read_count(channel,&state);
 rx_log_info("CAL","Pre-calibration:%dcounts,%drevs", state.total_count,state.revolutions);
 rx_err_terr=rx_encoder_reset(channel); if(err==k_rx_ok)
 { rx_log_info("CAL","Encodercalibratedatdockposition"); } }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (channel valid, initialized) Rule 5: [OK] 2 postconditions (TCNT=0, state zeroed)

See also: rx_encoder_set_count() Set encoder to specific non-zero value,
 rx_encoder_init() Initial setup, rx_motor_stop() Stop motor before reset if stationary
 reset needed

Figure 388: Call graph for `rx_encoder_reset`

_Defined in libs/rx_encoder/inc/rx_mtu_encoder.h:1065_

Since Version 1.0.0

—

rx_encoder_set_count

```
rx_err_t rx_encoder_set_count(int32_t count, rx_mtu_channel_t channel)
```

Set encoder count value.

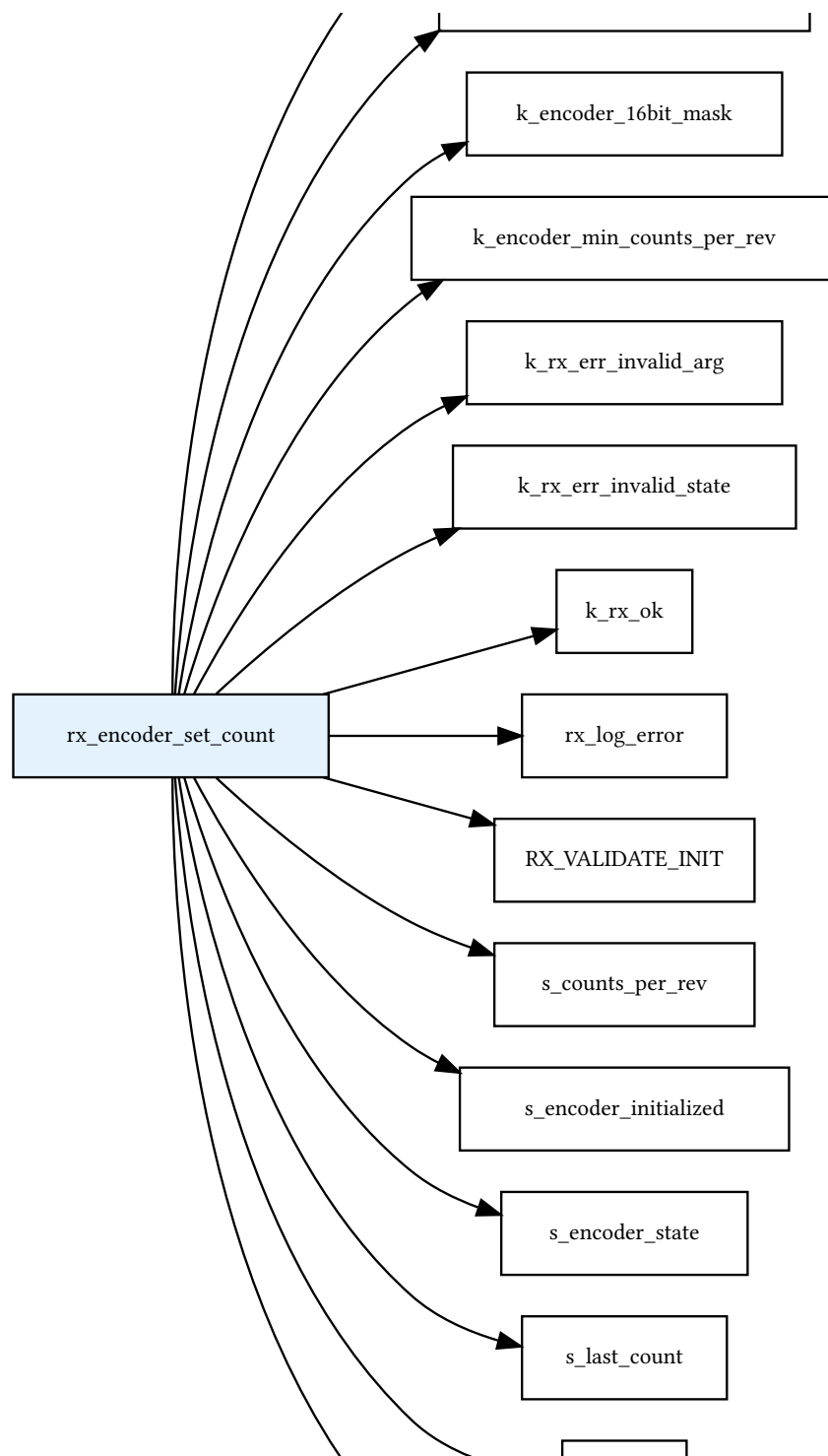
Sets both hardware counter and accumulated count. Useful for homing or calibration.

Set encoder count value.

Parameters:

Direction	Name	Description
in	count	Count value to set
in	channel	MTU channel
in	count	Desired count value
in	channel	MTU channel

Returns: k_rx_ok on success, error code otherwise

Figure 389: Call graph for `rx_encoder_set_count`

_Defined in libs/rx_encoder/inc/rx_mtu_encoder.h:1080_

—

rx_encoder_deinit

```
rx_err_t rx_encoder_deinit(rx_mtu_channel_t channel)
```

Deinitialize encoder.

Stops counter and releases resources.

Deinitialize encoder.

Performs orderly shutdown of encoder by stopping MTU timer and clearing initialization flag. After deinitialization, encoder can be reinitialized with different configuration parameters via rx_encoder_init().

Deinitialization Sequence:

1. Validate channel is valid and initialized
2. Stop MTU timer (halt edge counting)
3. Clear initialized flag (mark channel as uninitialized)
4. Clear counts_per_rev (prevent accidental division by stale value)

Use Cases:

- System shutdown: Release resources before power-down
- Reconfiguration: Change encoder parameters (requires deinit -> init)
- Error recovery: Clean up after hardware fault
- Resource sharing: Free MTU channel for other use

Encoder state preserved but inaccessible until reinitialized

Parameters:

Direction	Name	Description
in	channel	MTU channel
in	channel	MTU channel to deinitialize Must be valid channel (0-4, 6-7, excluding 5) Must be initialized (double-deinit returns error)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, encoder deinitialized and timer stopped
k_rx_err_invalid_arg	channel invalid (not 0-4, 6-7)
k_rx_err_invalid_state	Encoder not initialized (already deinitialized)

Pre: channel must be initialized via rx_encoder_init()

Pre: No other tasks are currently accessing this encoder

Post: s_encoder_initializedchannel = false

Post: MTU timer stopped (no longer counting)

Post: s_counts_per_revchannel = 0

Post: Other state (total_count, position) preserved but inaccessible

Note: If rx_mtu_stop() fails, warning logged but deinit continues

Note: Timer stop affects entire channel (shared resource)

Note: Thread-safe only with external synchronization

Warning: Ensure no other tasks are using encoder before calling

Performance:: Execution time: 2 us @ 240 MHz (timer stop + flag clear) Called once per encoder during shutdown (not performance-critical)

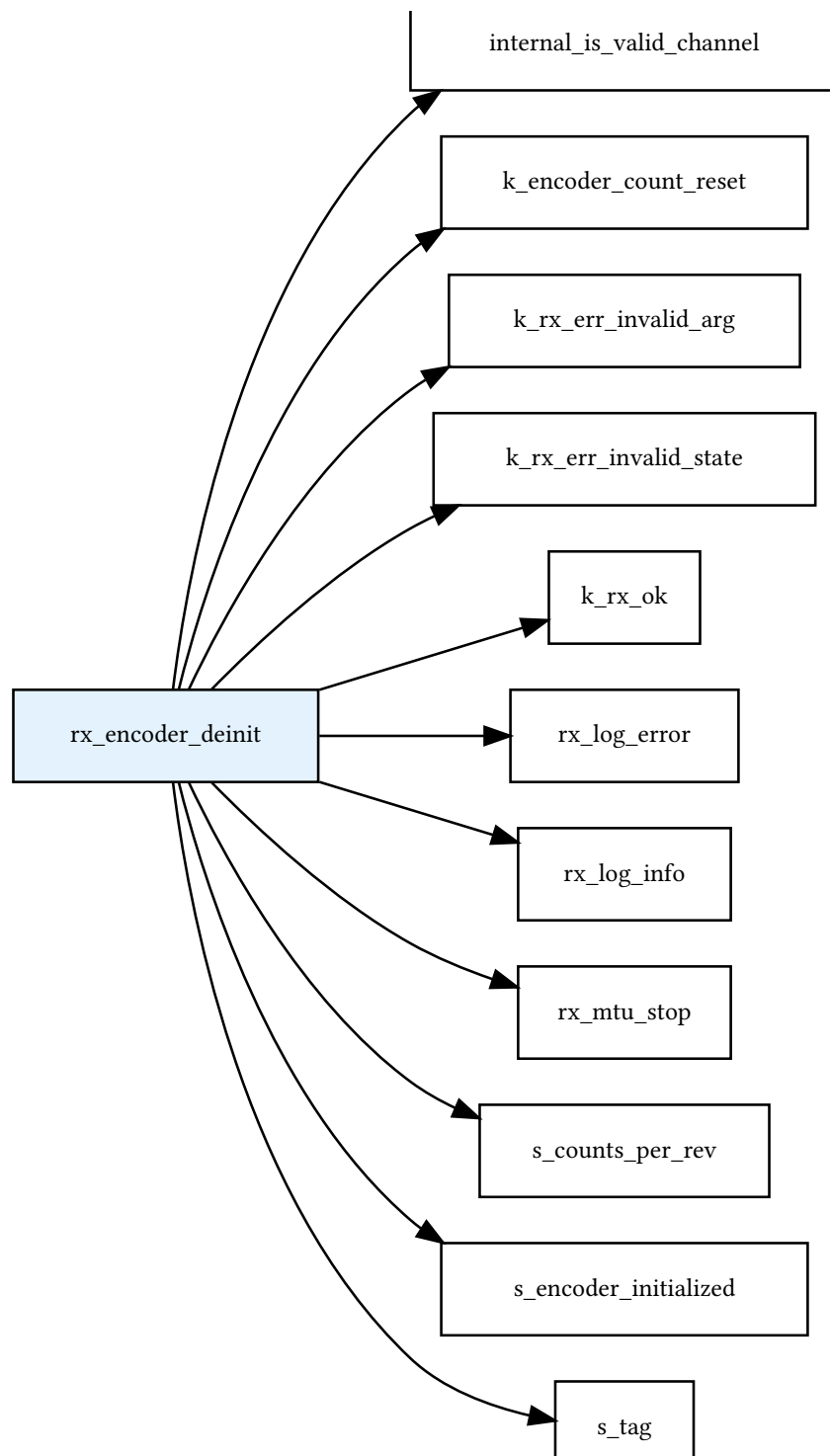
Example - Clean Shutdown:: rx_encoder_state_t final_state;
rx_encoder_read_count(k_mtu_channel_1, &final_state); rx_log_info("SHUTDOWN", "Final position: %d counts", final_state.total_count);

```
rx_err_t err = rx_encoder_deinit(k_mtu_channel_1); if (err != k_rx_ok)
{ rx_log_error("SHUTDOWN", "Failed to deinit encoder: %d", err); }
```

Example - Reconfiguration:: rx_encoder_deinit(k_mtu_channel_1);
rx_encoder_config_t new_config = { .channel = k_mtu_channel_1, .counts_per_rev = 2048, .invert_direction = false, };
rx_encoder_init(&new_config);

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (channel valid, initialized) Rule 5: [OK] 1 postcondition (initialized flag cleared)

See also: rx_encoder_init() Reinitialize after deinit, rx_mtu_stop() Underlying timer stop function

Figure 390: Call graph for `rx_encoder_deinit`

_Defined in libs/rx_encoder/inc/rx_mtu_encoder.h:1092_

Since Version 1.0.0

—

rx_encoder_tpu.c

TPU Quadrature Encoder Driver Implementation (Phase Counting Mode).

Hardware-accelerated quadrature encoder position tracking using the RX72N TPU peripheral in phase counting mode for rear wheel encoders (motors 2-3). The algorithm mirrors rx_mtu_encoder.c: 4x decoding, 16-bit overflow detection, 32-bit accumulated count, and velocity calculation.

Includes:

- "rx_encoder_tpu.h"
- <math.h>
- "rx_check.h"
- "rx_log.h"
- "rx_tpu.h"

tpu_enc_channel_limits_t

TPU encoder channel mapping constants.

Value	Name	Description
= 6	k_tpu_enc_max_channels	Max TPU channel index + 1

—

tpu_enc_counter_t

16-bit counter overflow detection constants

Value	Name	Description
= 65536	k_tpu_enc_counter_max	16-bit counter range
= 32768	k_tpu_enc_counter_half	Half-range for wrap detection
= 0xFFFF	k_tpu_enc_16bit_mask	16-bit mask

—

tpu_enc_params_t

Encoder parameter limits.

Value	Name	Description
= 1	k_tpu_enc_min_counts_per_rev	Minimum valid counts per rev
= 0	k_tpu_enc_count_reset	Counter reset value
= 360	k_tpu_enc_degrees_per_rev	Degrees in one revolution
= 2	k_tpu_enc_turns_multiplier	+/-720 deg range check

—

tpu_enc_velocity_t

Velocity calculation limits.

Value	Name	Description
= 100	k_tpu_enc_max_velocity_rps	Max realistic velocity (6000 RPM)

—

s_tag

```
const char* s_tag
```

—

k_tpu_enc_initial_position_deg

```
const float k_tpu_enc_initial_position_deg
```

—

k_tpu_enc_min_delta_time_s

```
const float k_tpu_enc_min_delta_time_s
```

—

k_tpu_enc_max_delta_time_s

```
const float k_tpu_enc_max_delta_time_s
```

—

s_initialized

```
bool s_initialized[k_tpu_enc_max_channels]
```

Per-channel initialization flag.

—

s_state

```
rx_encoder_state_t s_state[k_tpu_enc_max_channels]
```

Per-channel encoder state (accumulated count, position).

—

s_counts_per_rev

```
uint16_t s_counts_per_rev[k_tpu_enc_max_channels]
```

Per-channel counts per revolution.

—

s_invert_direction

```
bool s_invert_direction[k_tpu_enc_max_channels]
```

Per-channel direction inversion flag.

—

s_last_count

```
int32_t s_last_count[k_tpu_enc_max_channels]
```

Per-channel last total count for velocity delta.

—

internal_is_valid_channel

```
static bool internal_is_valid_channel(const rx_tpu_channel_t channel)
```

Check if a TPU channel index is valid for encoder use.

Parameters:

Direction	Name	Description
in	channel	TPU channel

Returns: true if channel is 1, 2, 4, or 5

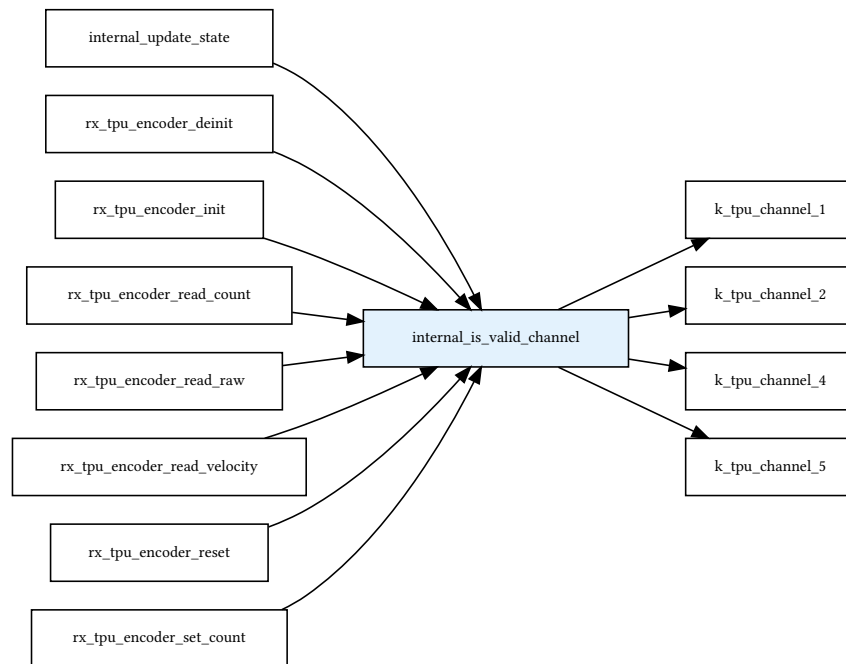


Figure 391: Call graph for `internal_is_valid_channel`

_Defined in `libs/rx\_encoder/src/rx\_encoder\_tpu.c:129_`

Since Version 1.0.0

—

internal_update_state

```
static rx_err_t internal_update_state(rx_encoder_state_t *state, const
rx_tpu_channel_t channel, const uint16_t current_count)
```

Update encoder state from a raw 16-bit counter reading.

Implements the 16-bit overflow detection algorithm. Calculates the signed delta from the previous reading, detects wraps using the half-range threshold, applies direction inversion, and accumulates into the 32-bit total count. Also derives revolutions and degrees.

Parameters:

Direction	Name	Description
out	state	Output state structure
in	channel	TPU channel
in	current_count	Current 16-bit TCNT reading

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel or null state
k_rx_err_invalid_state	Encoder not initialized or corrupted
k_rx_err_out_of_range	Position exceeds +/-720 degrees

Pre: s_initializedchannel == true

Post: state filled with updated values

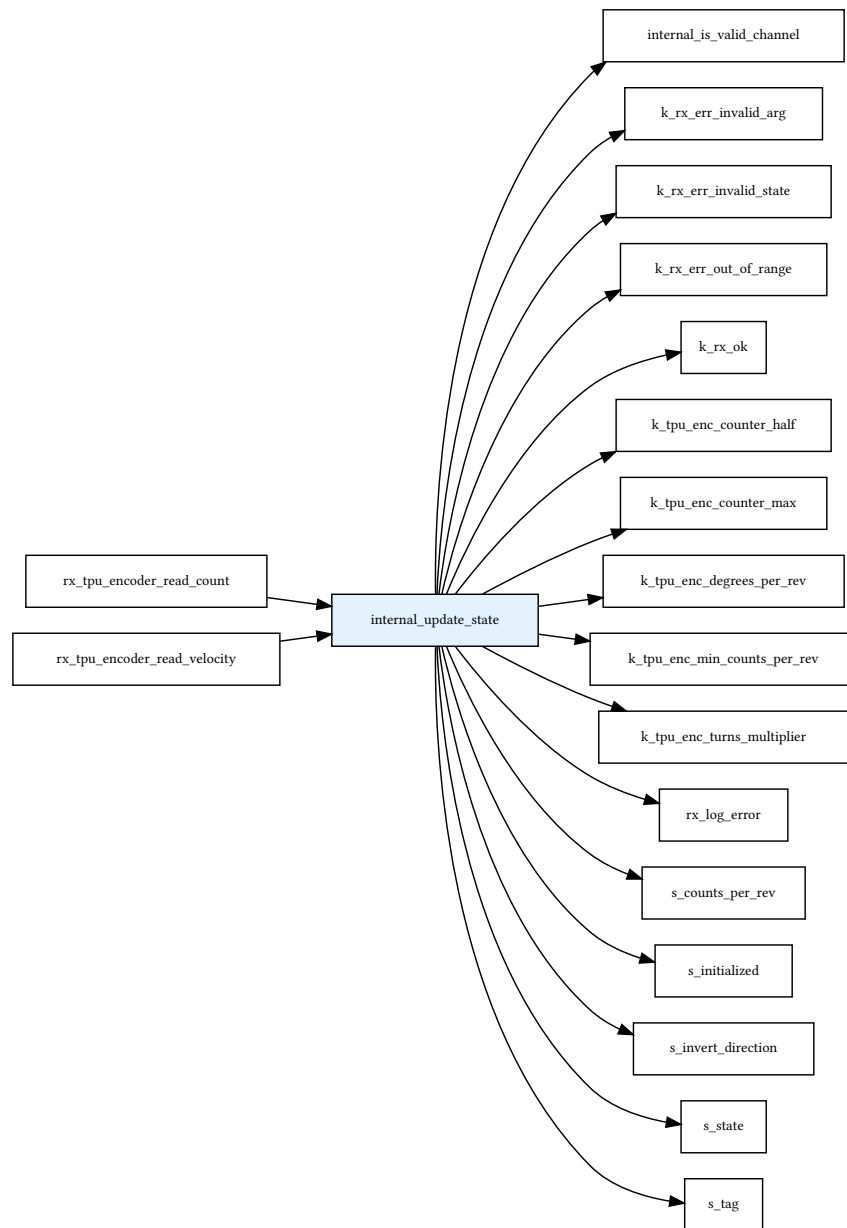


Figure 392: Call graph for `internal_update_state`

_Defined in `libs/rx\_encoder/src/rx\_encoder\_tpu.c:159_`

Since Version 1.0.0

—

rx_tpu_encoder_init

```
rx_err_t rx_tpu_encoder_init(const rx_tpu_encoder_config_t *config)
```

Initialize TPU channel for hardware quadrature encoder counting.

Configures a TPU channel in phase counting mode (4x decoding) and initializes software overflow tracking state. After initialization, the encoder is counting and ready for position/velocity reads.

Initialization steps:

1. Validate configuration parameters
2. Initialize TPU HAL in phase counting mode 4
3. Start the TPU counter
4. Initialize software overflow detection state

Parameters:

Direction	Name	Description
in	config	Pointer to encoder configuration

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, encoder initialized and counting
k_rx_err_null_ptr	config is nullptr
k_rx_err_invalid_arg	Invalid channel or counts_per_rev == 0
k_rx_err_invalid_state	Channel already initialized

Pre: GPIO pins for TCLKA-D configured as peripheral I/O

Pre: Encoder physically connected

Post: TPU channel in phase counting mode, counter running

Post: Software state zeroed (count=0, position=0)

Note: Thread Safety: Not thread-safe, call during init only] **See also:** rx_tpu_encoder_deinit()
Release resources, rx_tpu_encoder_read_count() Read position after init

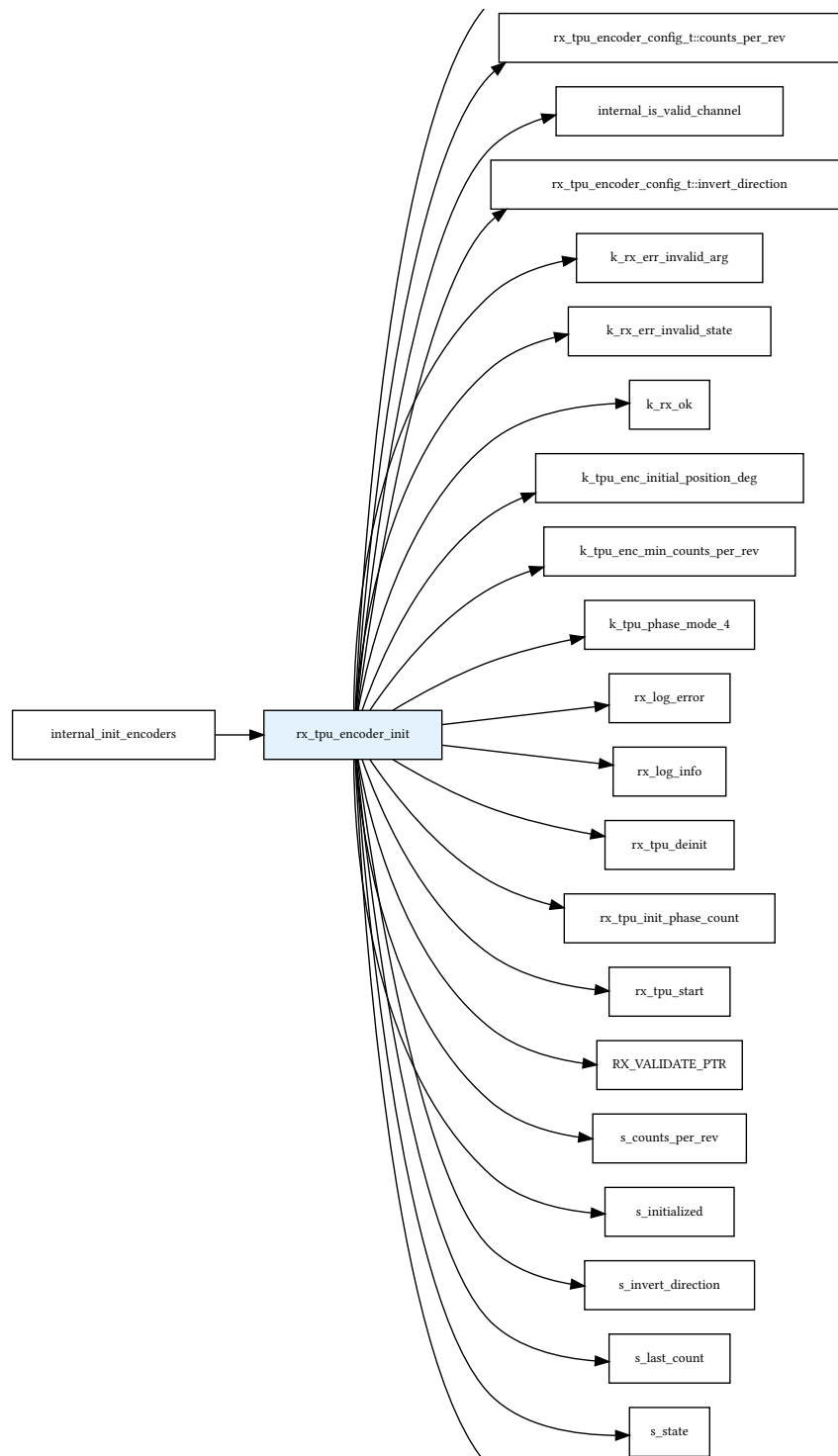


Figure 393: Call graph for rx_tpu_encoder_init

_Defined in libs/rx_encoder/src/rx_encoder_tpu.c:224_

Since Version 1.0.0

—

rx_tpu_encoder_read_raw

```
rx_err_t rx_tpu_encoder_read_raw(const rx_tpu_channel_t channel, uint16_t *count)
```

Read raw 16-bit counter value from TPU channel.

Reads the current TCNT value without overflow handling. For position tracking with overflow correction, use rx_tpu_encoder_read_count().

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
out	count	Pointer to store raw 16-bit count (0-65535)

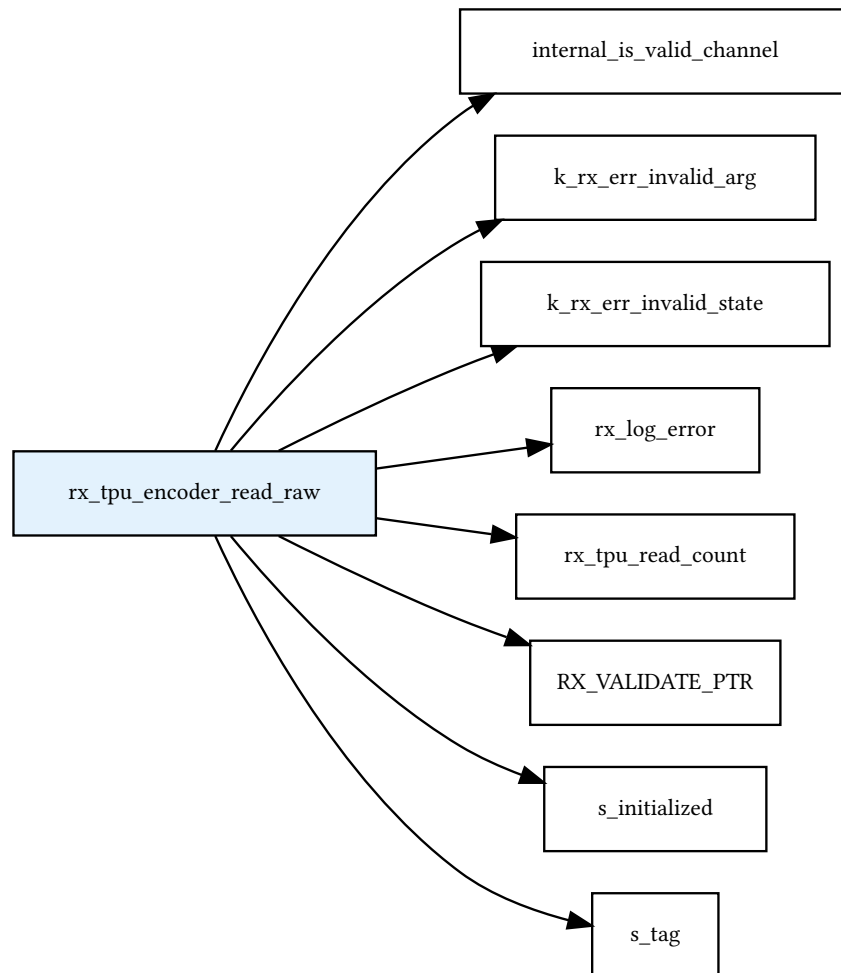
Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	count is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized via rx_tpu_encoder_init()

Post: count contains current TCNT register value

Note: Thread Safety: Safe (single volatile register read)] **See also:**
rx_tpu_encoder_read_count() Read with overflow handling

Figure 394: Call graph for `rx_tpu_encoder_read_raw`

_Defined in `libs/rx\_encoder/src/rx\_encoder\_tpu.c:281_`

Since Version 1.0.0

—

rx_tpu_encoder_read_count

```
rx_err_t rx_tpu_encoder_read_count(const rx_tpu_channel_t channel,
rx_encoder_state_t *state)
```

Read encoder count with automatic 16-bit overflow handling.

Reads the current 16-bit hardware counter and updates the 32-bit accumulated count with automatic overflow/underflow correction. Also computes derived position (degrees) and revolution count.

Must be called frequently enough to catch overflows: at 210 RPM max motor speed with 1364 counts/rev, the safe interval is 6.8 seconds. The 250 Hz control loop provides a 1666x safety margin.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
out	state	Pointer to encoder state structure to update

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, state updated
k_rx_err_null_ptr	state is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized via rx_tpu_encoder_init()

Pre: Called frequently enough to catch overflows

Post: state->total_count updated with overflow correction

Post: state->last_raw_count updated

Post: state->revolutions and position_deg computed

Note: Thread Safety: Not thread-safe for same channel] **See also:**
rx_tpu_encoder_read_velocity() Calculate velocity

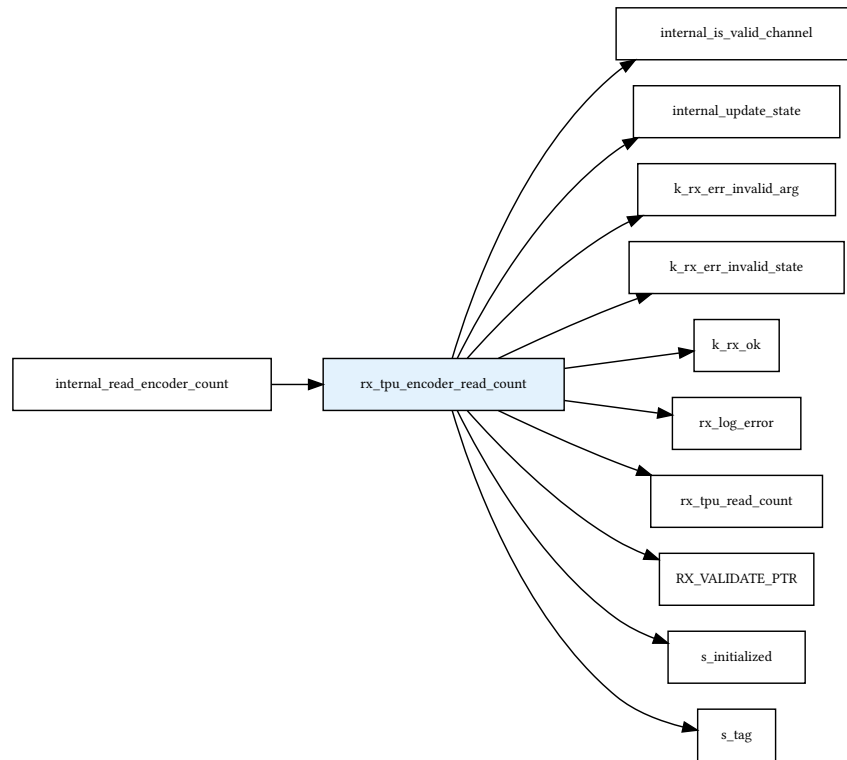


Figure 395: Call graph for rx_tpu_encoder_read_count

_Defined in libs/rx_encoder/src/rx_encoder_tpu.c:297_

Since Version 1.0.0

—

rx_tpu_encoder_read_velocity

```
rx_err_t rx_tpu_encoder_read_velocity(float *velocity_rps, const float
delta_time_s, const rx_tpu_channel_t channel)
```

Calculate encoder velocity in revolutions per second.

Derives angular velocity from the change in encoder position over a known time interval. Tracks previous count internally for delta calculation.

Parameters:

Direction	Name	Description
out	velocity_rps	Pointer to store velocity (rev/sec)
in	delta_time_s	Time since last call in seconds (must be > 0)
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	velocity_rps is nullptr
k_rx_err_invalid_arg	Invalid delta_time_s or channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized via rx_tpu_encoder_init()

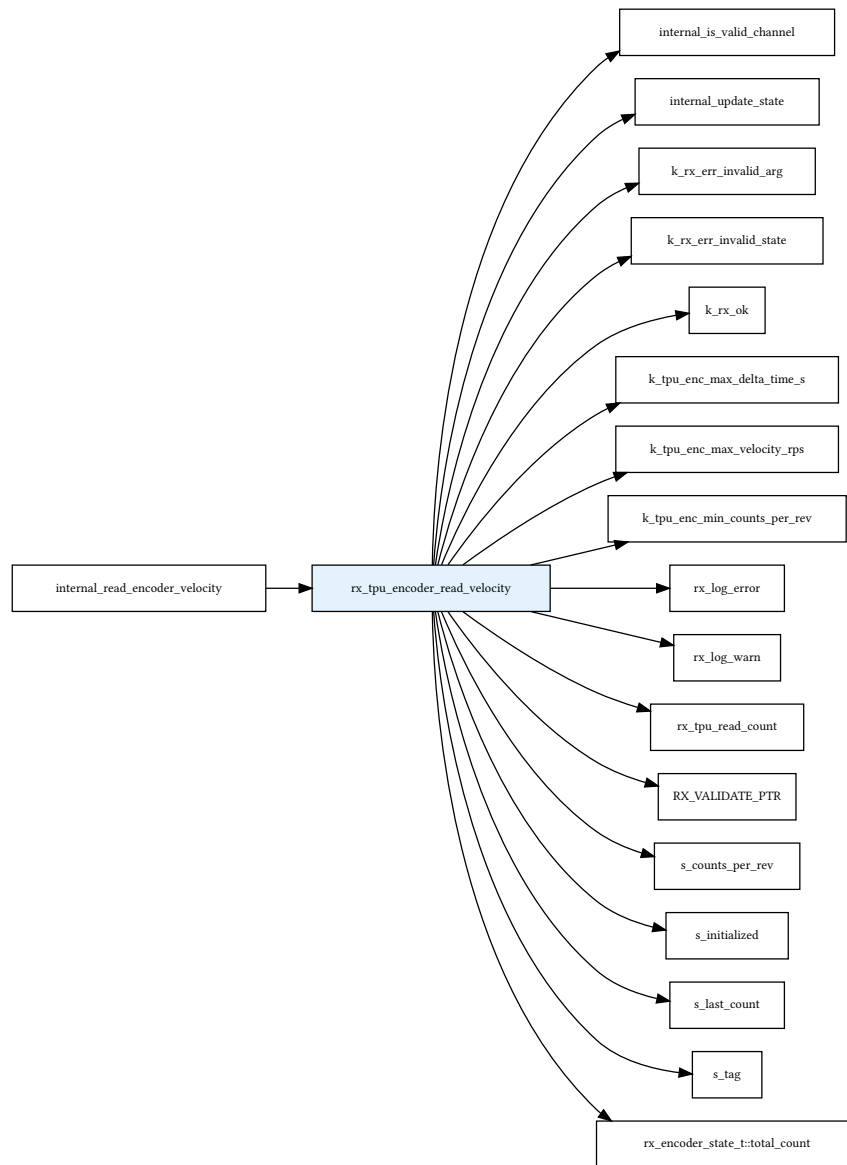
Pre: delta_time_s matches actual elapsed time

Post: velocity_rps contains calculated velocity

Post: Internal last_count updated for next delta

Note: Parameter order: output, time, channel (prevents swaps)

Note: Thread Safety: Not thread-safe for same channel] **See also:**
rx_tpu_encoder_read_count() Read position

Figure 396: Call graph for `rx_tpu_encoder_read_velocity`_Defined in `libs/rx\_encoder/src/rx\_encoder\_tpu.c:320_`*Since Version 1.0.0*

—

rx_tpu_encoder_reset

```
rx_err_t rx_tpu_encoder_reset(const rx_tpu_channel_t channel)
```

Reset encoder count to zero.

Resets both the TPU hardware counter and software accumulated state.

Parameters:

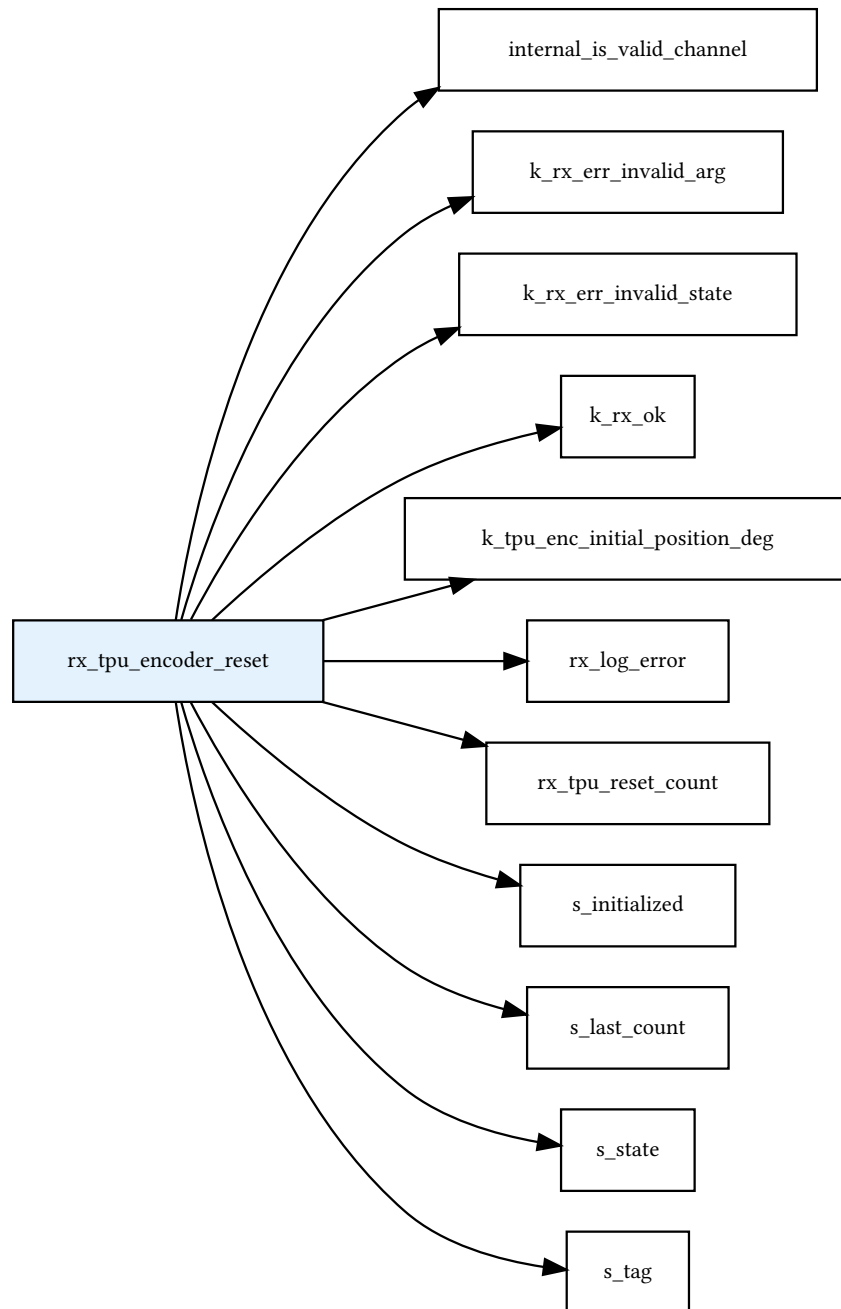
Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized

Post: TCNT=0, total_count=0, position=0] **See also:** rx_tpu_encoder_set_count() Set to specific value

Figure 397: Call graph for `rx_tpu_encoder_reset`

_Defined in `libs/rx\_encoder/src/rx\_encoder\_tpu.c:375_`

Since Version 1.0.0

—

rx_tpu_encoder_set_count

```
rx_err_t rx_tpu_encoder_set_count(const int32_t count, const rx_tpu_channel_t channel)
```

Set encoder count to specific value.

Sets both hardware counter (low 16 bits) and software accumulated count.

Parameters:

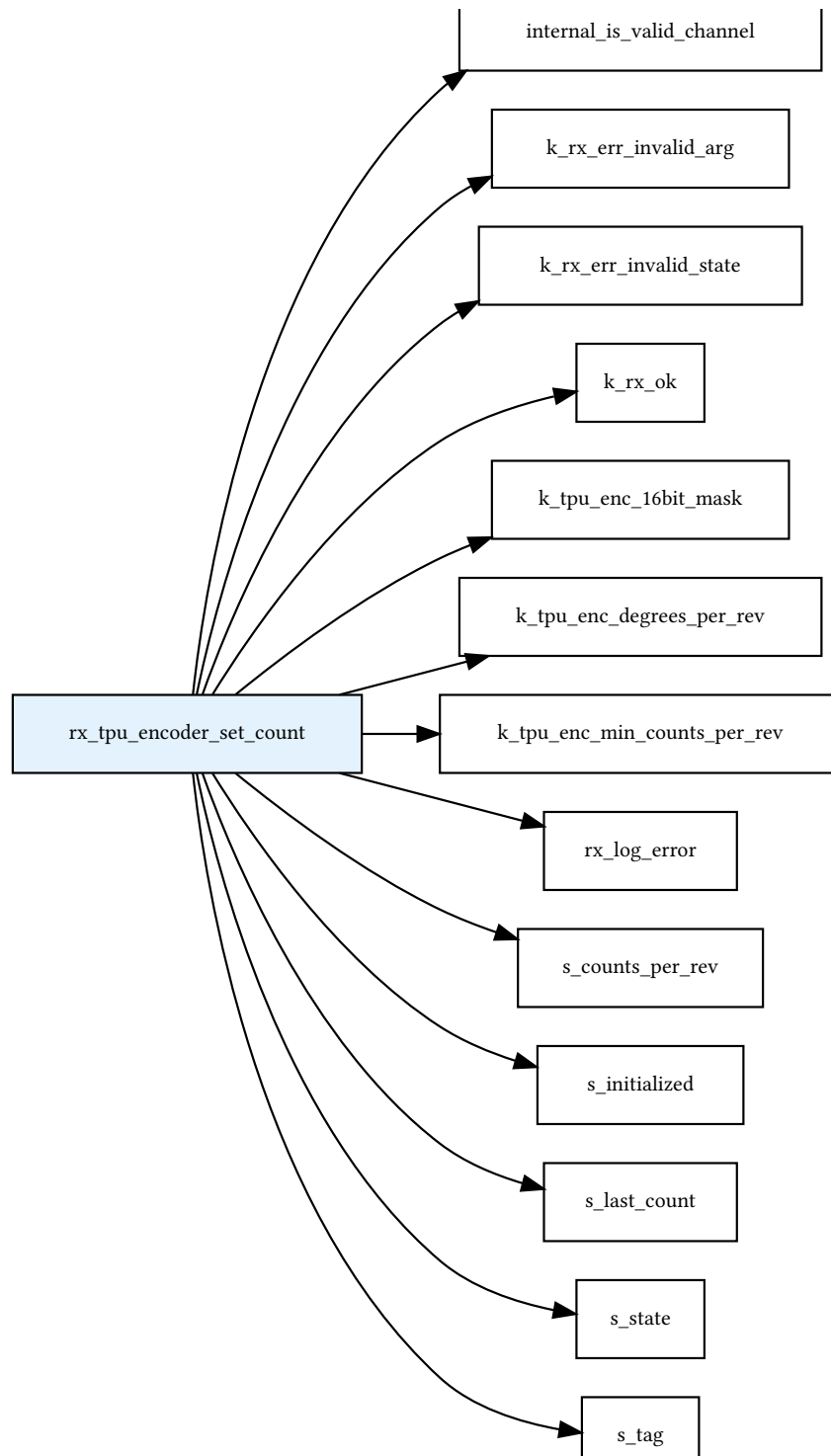
Direction	Name	Description
in	count	Count value to set
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel initialized

Post: Software and hardware counts updated

Figure 398: Call graph for `rx_tpu_encoder_set_count`

_Defined in libs/rx_encoder/src/rx_encoder_tpu.c:402_

Since Version 1.0.0

—

rx_tpu_encoder_deinit

```
rx_err_t rx_tpu_encoder_deinit(const rx_tpu_channel_t channel)
```

Deinitialize TPU encoder.

Stops counter and releases resources. Can be reinitialized after.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Encoder not initialized

Pre: Channel previously initialized

Post: Counter stopped, channel marked uninitialized] **See also:** rx_tpu_encoder_init()
Reinitialize after deinit

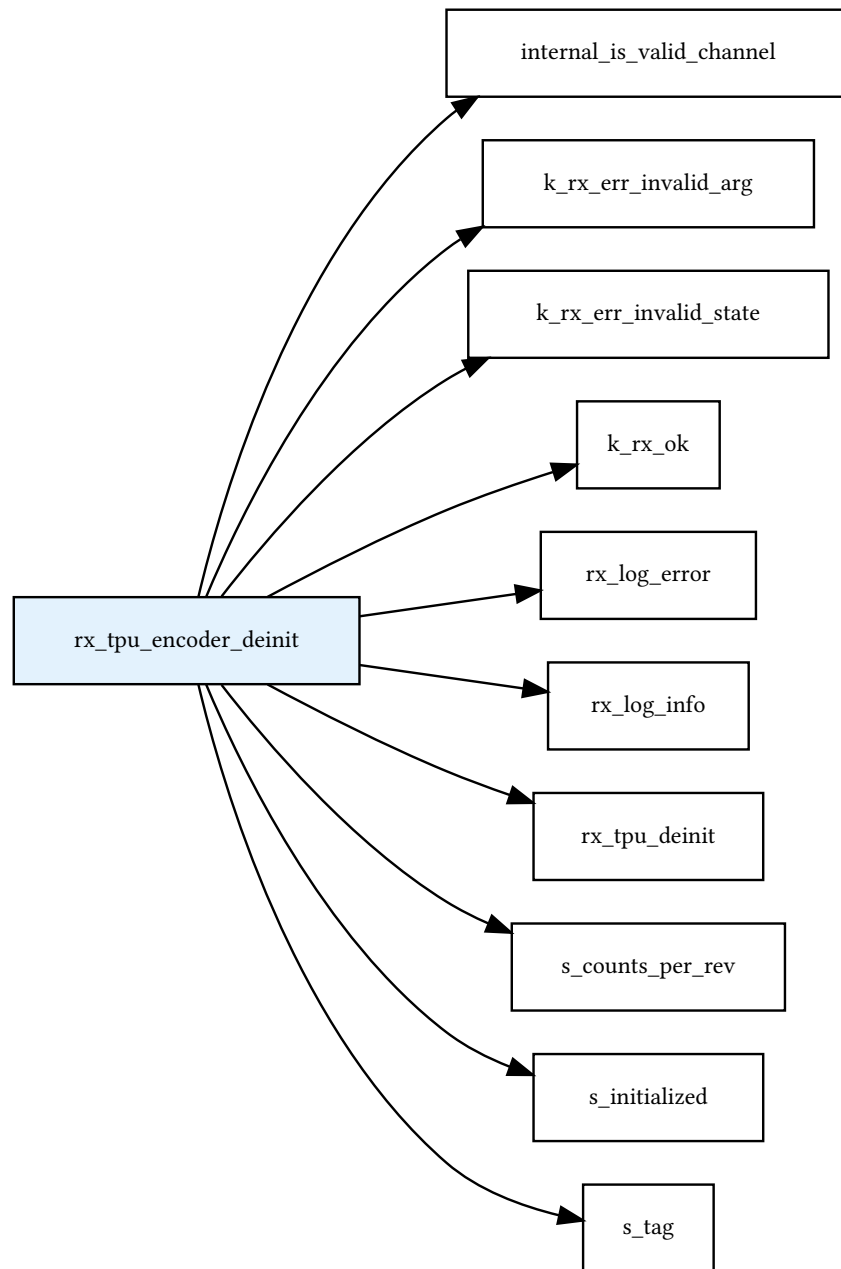


Figure 399: Call graph for rx_tpu_encoder_deinit

_Defined in `libs/rx\_encoder/src/rx\_encoder\_tpu.c:433_`

Since Version 1.0.0

—

rx_mtu_encoder.c

RX72N MTU Quadrature Encoder Driver Implementation (Phase Counting Mode).

Hardware-accelerated quadrature encoder position tracking using the Renesas RX72N Multi-Function Timer Pulse Unit (MTU) in phase counting mode. Provides 4x decoding (count on all edges), automatic direction detection, 16-bit hardware counter with software wraparound handling, and derived velocity calculation.

Includes:

- "rx_mtu_encoder.h"
- <math.h>
- "rx72n_regs.h"
- "rx_check.h"
- "rx_gpio_constants.h"
- "rx_log.h"
- "rx_register_protection.h"

encoder_constants_t

Encoder configuration and hardware constants.

Value	Name	Description
= 8	k_encoder_max_channels	Maximum MTU channels for encoders (max index 7)
= 0x04	k_tmdr_phase_counting_mode_1	Phase counting mode 1 (4x decoding)
= 65536	k_encoder_counter_max	16-bit counter maximum value
= 32768	k_encoder_counter_half	Half of counter for wraparound detection
= 0xFFFF	k_encoder_16bit_mask	Bitmask for 16-bit values
= 9	k_mtu_mstpcra_mtu0_4_bit	MSTPCRA bit for MTU0-MTU4
= 8	k_mtu_mstpcra_mtu6_7_bit	MSTPCRA bit for MTU6-MTU7
= 0x00	k_tcr_external_clock_no_prescaler	External clock, no prescaler
= 0x00	k_tior_disabled	I/O control disabled

—

encoder_calc_constants_t

Encoder calculation constants.

Value	Name	Description
= 2	k_turns_multiplier	Multiplier for turns range validation (+/-720deg)
= 360	k_degrees_per_revolution	Degrees in one full revolution

—

encoder_init_values_t

Encoder initialization values.

Value	Name	Description
-------	------	-------------

= 0	k_encoder_count_reset	Counter reset value
= 0	k_encoder_initial_count	Initial total count
= 0	k_encoder_initial_rev	Initial revolution count
= 1	k_encoder_min_counts_per_rev	Minimum valid counts per revolution
= 65535	k_encoder_max_counts_per_rev	Maximum valid counts per revolution

—

encoder_velocity_limits_t

Encoder velocity limits.

Value	Name	Description
= 100	k_max_realistic_velocity_rps	Maximum realistic velocity (6000 RPM)

—

s_tag

```
const char s_tag[]
```

—

k_encoder_initial_position_deg

```
const float k_encoder_initial_position_deg
```

—

k_min_delta_time_s

```
const float k_min_delta_time_s
```

—

k_max_delta_time_s

```
const float k_max_delta_time_s
```

—

s_encoder_initialized

```
bool s_encoder_initialized[k_encoder_max_channels]
```

—

s_encoder_state

```
rx_encoder_state_t s_encoder_state[k_encoder_max_channels]
```

—

s_counts_per_rev

```
uint16_t s_counts_per_rev[k_encoder_max_channels]
```

—

s_invert_direction

```
bool s_invert_direction[k_encoder_max_channels]
```

—

s_last_count

`int32_t s_last_count[k_encoder_max_channels]`

—

internal_enable_mtu_module

```
static rx_err_t internal_enable_mtu_module(const rx_mtu_channel_t channel)
```

Enable MTU module by clearing module stop bit.

Parameters:

Direction	Name	Description
in	channel	MTU channel

Returns: k_rx_ok on success

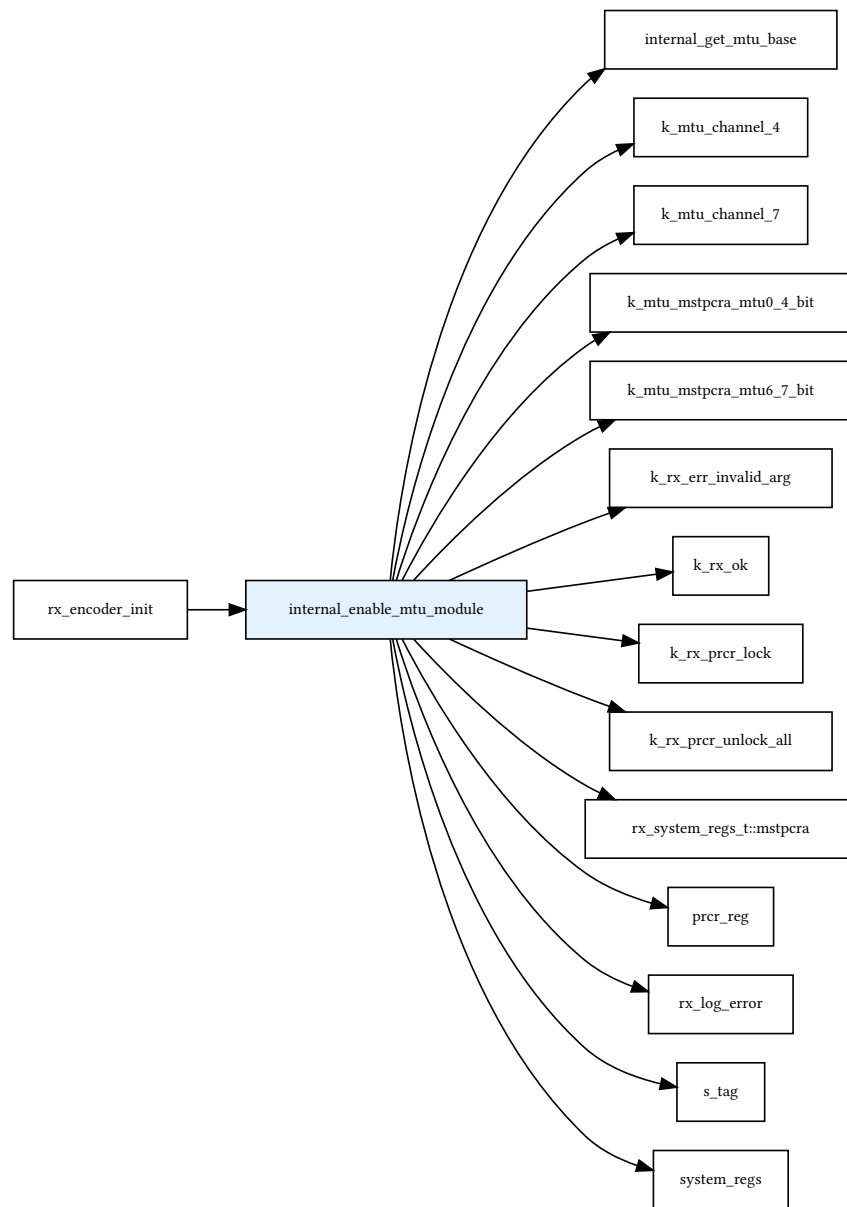


Figure 400: Call graph for `internal_enable_mtu_module`

_Defined in `libs/rx_encoder/src/rx_mtu_encoder.c:1292_`

internal_configure_encoder_timer

```
static rx_err_t internal_configure_encoder_timer(volatile rx_mtu_channel_regs_t
*mtu)
```

Configure MTU timer for encoder phase counting mode.

Parameters:

Direction	Name	Description
in	mtu	Pointer to MTU channel registers

Returns: k_rx_ok on success

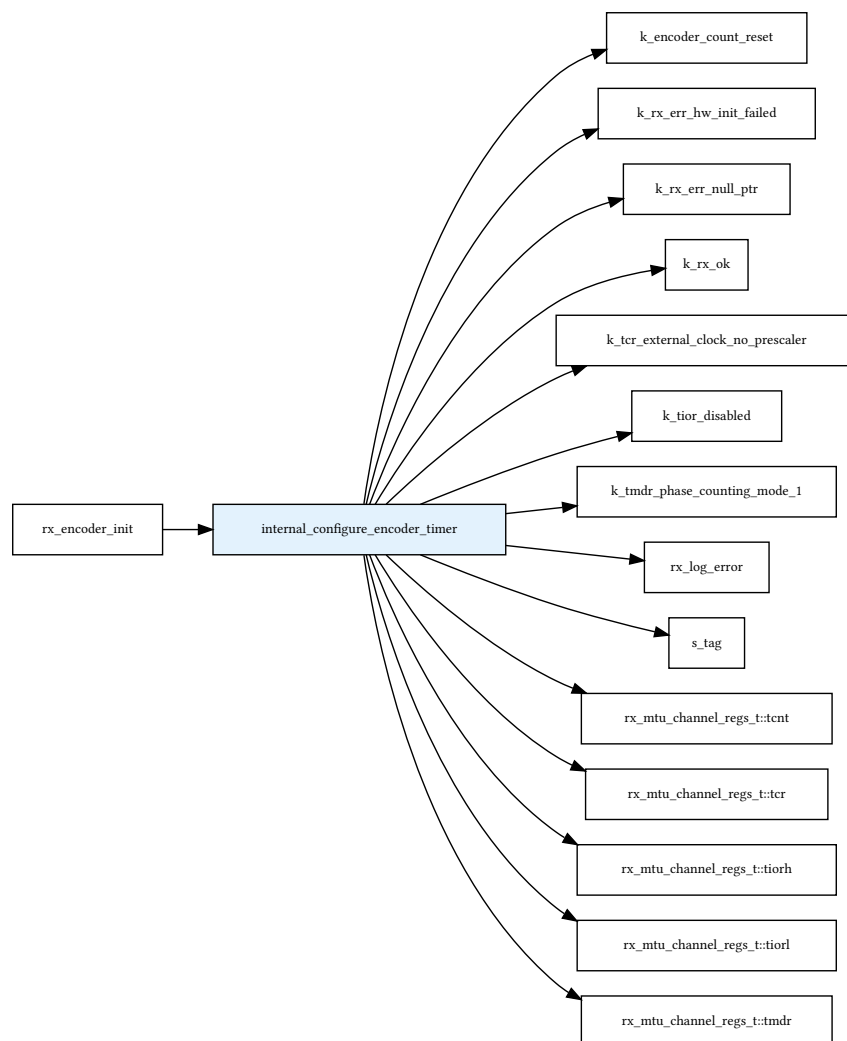


Figure 401: Call graph for `internal_configure_encoder_timer`

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:1330_

—

internal_verify_timer_counting

```
static rx_err_t internal_verify_timer_counting(const volatile
rx_mtu_channel_regs_t *mtu)
```

Verify timer configuration after initialization (NASA Rule 5: min 2 checks).

Since encoder counting verification requires encoder motion which may not be available at initialization time, this function validates the timer configuration instead of actual counting.

Parameters:

Direction	Name	Description
in	mtu	Pointer to MTU channel registers

Returns: k_rx_err_hw_init_failed if timer mode is incorrectly configured

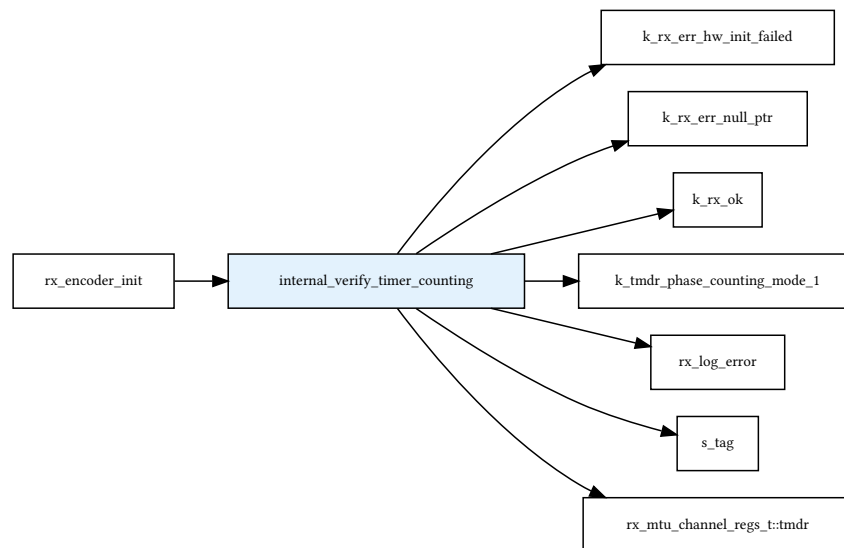


Figure 402: Call graph for internal_verify_timer_counting

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:1382_

—

internal_initialize_encoder_state

```
static rx_err_t internal_initialize_encoder_state(const rx_mtu_channel_t channel,
const rx_encoder_config_t *config)
```

Initialize encoder state variables.

Parameters:

Direction	Name	Description
in	channel	MTU channel
in	config	Encoder configuration

Returns: k_rx_ok on success

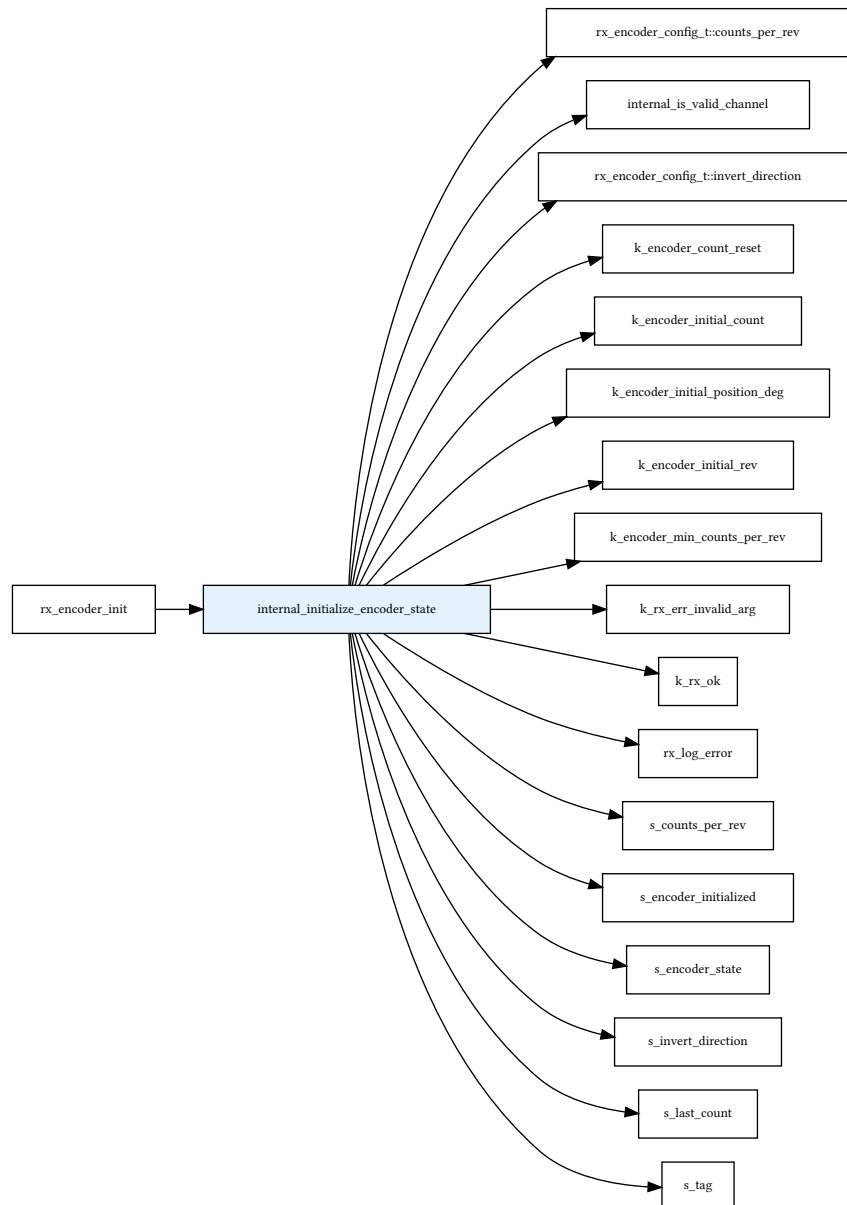


Figure 403: Call graph for `internal_initialize_encoder_state`

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:1408_
—

internal_update_state_from_count

```
static rx_err_t internal_update_state_from_count(rx_encoder_state_t *state,  
rx_mtu_channel_t channel, uint16_t current_count)
```

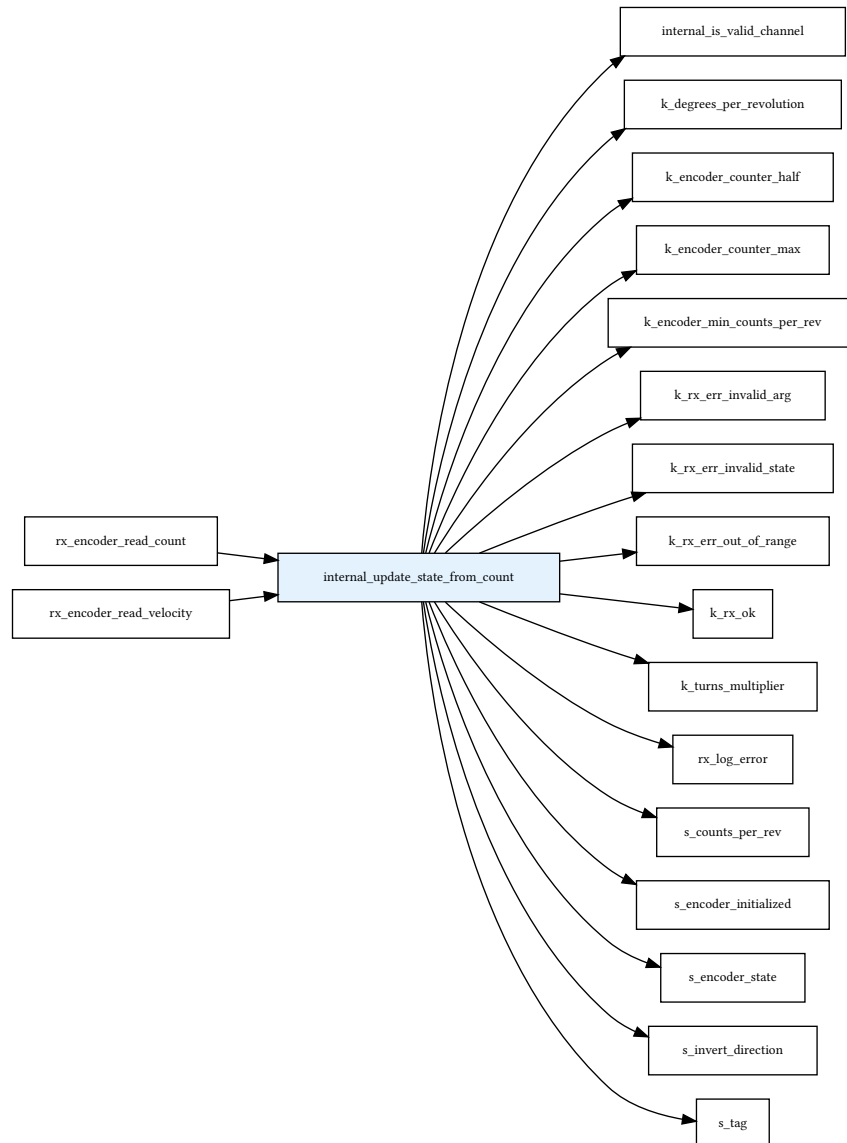


Figure 404: Call graph for internal_update_state_from_count

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:1436_
—

internal_is_valid_channel

```
static bool internal_is_valid_channel(rx_mtu_channel_t channel)
```

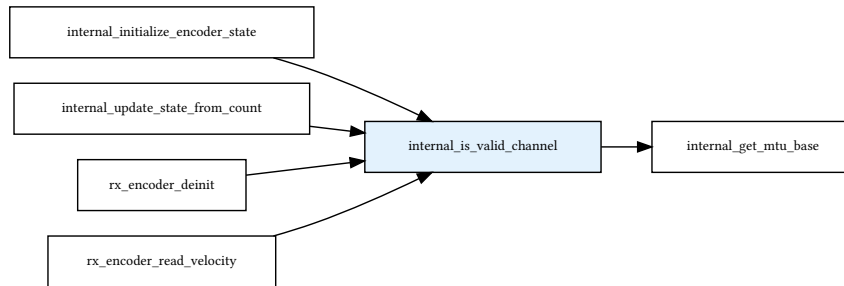


Figure 405: Call graph for internal_is_valid_channel

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:489_
_

internal_get_mtu_base

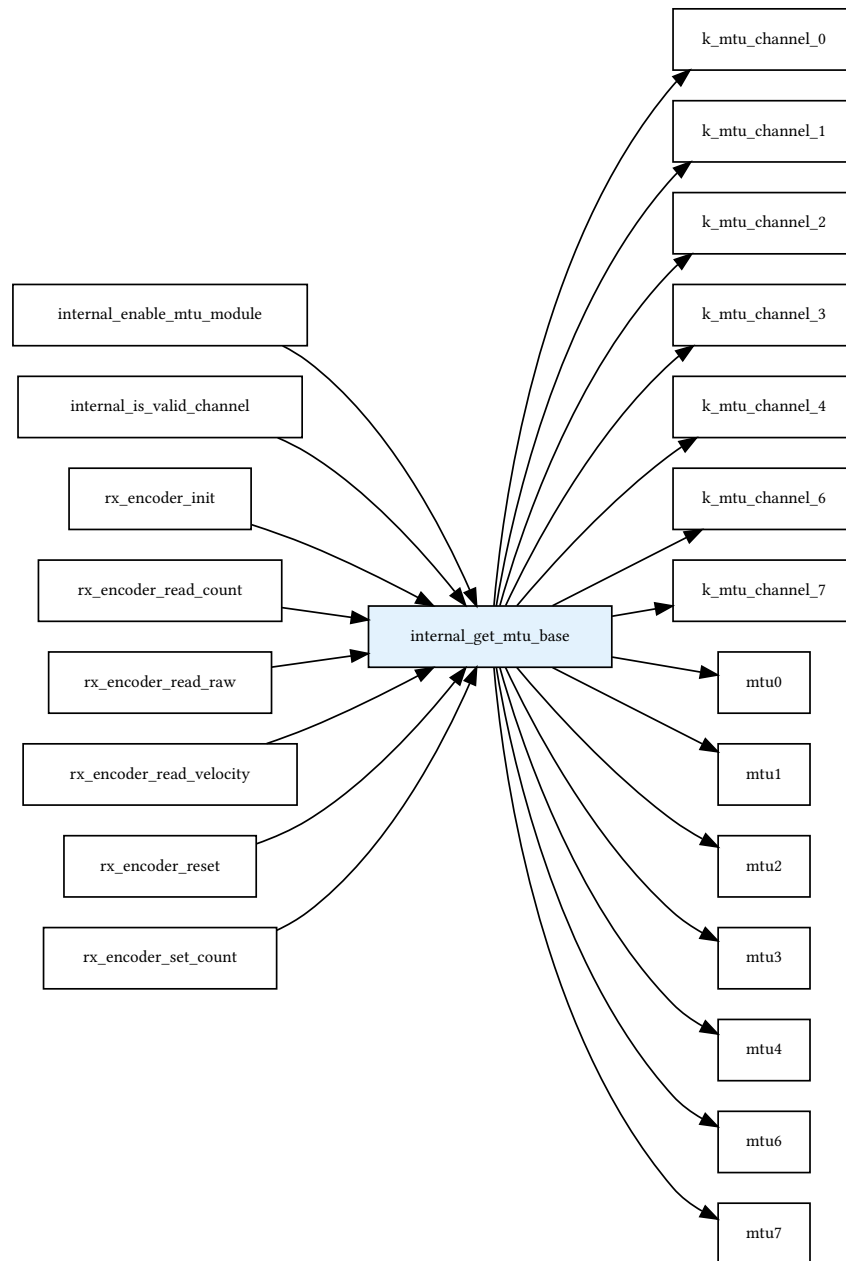
```
static volatile rx_mtu_channel_regs_t * internal_get_mtu_base(const  
rx_mtu_channel_t channel)
```

Get MTU channel base address.

Parameters:

Direction	Name	Description
in	channel	MTU channel

Returns: Pointer to MTU register base, or nullptr if invalid

Figure 406: Call graph for `internal_get_mtu_base`

Defined in `libs/rx\_encoder/src/rx\_mtu\_encoder.c:467`

rx_encoder_init

```
rx_err_t rx_encoder_init(const rx_encoder_config_t *config)
```

Initialize MTU channel for quadrature encoder position tracking.

Initialize MTU channel for hardware quadrature encoder counting.

Configures specified MTU channel in phase counting mode (4x decoding) for hardware-accelerated quadrature encoder counting. This function performs complete initialization including module enable, timer configuration, state initialization, and verification.

Initialization Sequence:

1. Validate configuration parameters (NULL check, counts_per_rev range)
2. Enable MTU peripheral module (clear MSTPCRA module stop bit)
3. Stop timer before configuration (prevents glitches during setup)
4. Configure MTU registers (TMDR=0x04 for phase counting, TCR for external clock)
5. Initialize software state (total_count, position, velocity tracking)
6. Start timer (begin counting encoder edges)
7. Verify timer is counting (post-condition check per NASA Rule 5)

Hardware Configuration Applied:

- TMDR (Timer Mode Register) = 0x04 -> Phase counting mode 1 (4x decoding)
- TCR (Timer Control Register) = 0x00 -> External clock, no prescaler
- TIOR (I/O Control) = 0x00 -> Disabled (not used in phase counting mode)
- TCNT (Counter Register) = 0 -> Reset to zero

Encoder Resolution Calculation: For STAR project motors (341 PPR encoder):

- Manufacturer PPR: 341 pulses/revolution
- 4x decoding: $341 \times 4 = 1364$ counts/revolution
- Angular resolution: $360\text{deg} / 1364 = 0.264\text{deg}$ per count

Timer mode verification reads back hardware registers (post-condition check)

Parameters:

Direction	Name	Description
in	config	Pointer to encoder configuration structure Must not be NULL (validated) config->channel: MTU channel to use (k_mtu_channel_0 to k_mtu_channel_7, excluding 5) config->counts_per_rev: Encoder resolution after 4x decoding [1, 65535] STAR motors: $341 \text{ PPR} \times 4 = 1364$ counts/rev Must be ≥ 1 to prevent division by zero config->invert_direction: true to reverse count direction, false for normal

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, encoder initialized and counting
k_rx_err_null_ptr	config pointer is nullptr
k_rx_err_invalid_arg	counts_per_rev < 1 (would cause division by zero)
k_rx_err_invalid_arg	channel invalid or not supported (e.g., channel 5)

k_rx_err_*	Propagated from rx_mtu_stop() (failed to stop timer before config)
k_rx_err_hw_init_failed	Timer configuration not latched (hardware fault)
k_rx_err_*	Propagated from rx_mtu_start() (failed to start timer)

Pre: config must point to valid rx_encoder_config_t structure

Pre: MTU channel must not already be in use (no double initialization check)

Pre: GPIO pins (MTCLKA/MTCLKB) must be configured for peripheral function

Pre: Encoder must be physically connected to MTCLKA/MTCLKB pins

Post: On success: s_encoder_initializedchannel = true

Post: On success: MTU timer counting encoder edges

Post: On success: Software state initialized (count=0, position=0deg, rev=0)

Post: On failure: s_encoder_initializedchannel = false, timer stopped

Note: Thread-safe only if different channels initialized from different tasks

Note: GPIO pin configuration must be done externally (not handled by this function)

Note: Counter starts at zero regardless of actual encoder position

Note: Can only initialize once per channel (no re-initialization without deinit)

Warning: Must configure GPIO pins before calling this function

Warning: Encoder motion during initialization may cause initial count offset

Performance:: Execution time: 25 us @ 240 MHz (includes module enable and register verification)
One-time initialization cost, called once per encoder at system startup

Example - Single Encoder::

```
rx_encoder_config_t encoder_config = { .channel = k_mtu_channel_1, .counts_per_rev = 1364, .invert_direction = false, };
rx_err_t err = rx_encoder_init(&encoder_config); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to initialize encoder: %d", err); return err; }
```

Example - Four Encoders (STAR Platform)::

```
constrx_encoder_config_t encoder_configs[4] = { { .channel=k_mtu_channel_1, counts_per_rev=1364, invert_direction=
{.channel=k_mtu_channel_2, counts_per_rev=1364, invert_direction=true},
{.channel=k_mtu_channel_3, counts_per_rev=1364, invert_direction=false},
{.channel=k_mtu_channel_4, counts_per_rev=1364, invert_direction=true}, };

for(uint8_ti=0; i<4; i++){ rx_err_t err=rx_encoder_init(&encoder_configs[i]); if(err!=k_rx_ok)
{ for(uint8_tj=0; j<i; j++){ (void)rx_encoder_deinit(encoder_configs[j].channel); } return err; } }
rx_log_info("APP", "All 4 encoders initialized");
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential initialization steps
Rule 3: [OK] No dynamic memory allocation (static state arrays) Rule 5: [OK] 3 preconditions (NULL
check, range check, channel valid) Rule 5: [OK] 2 postconditions (initialized flag set, timer counting)
Rule 7: [OK] All return values checked (enable, stop, configure, start, verify)

See also: rx_encoder_deinit() Cleanup function, rx_encoder_read_count() Read position
after initialization, rx_encoder_read_velocity() Read velocity after initialization,
rx_encoder_config_t Configuration structure definition

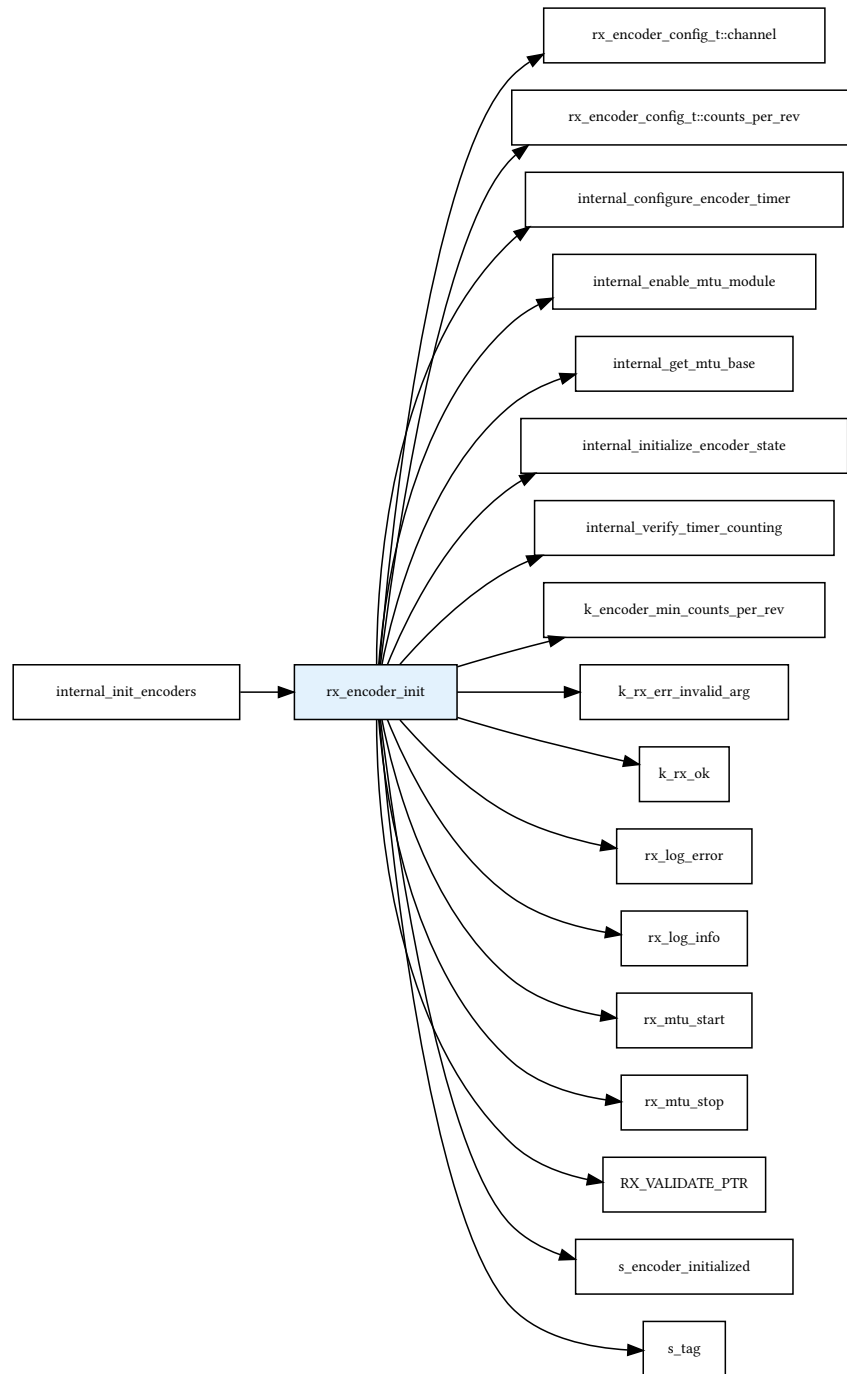


Figure 407: Call graph for `rx_encoder_init`

_Defined in `libs/rx\_encoder/src/rx\_mtu\_encoder.c:623_`

Since Version 1.0.0

—

rx_encoder_read_raw

```
rx_err_t rx_encoder_read_raw(const rx_mtu_channel_t channel, uint16_t *count)
```

Read raw 16-bit hardware counter value directly from TCNT register.

Read raw encoder count.

Reads the current value of the MTU TCNT (Timer Counter) register without any processing. This provides the raw 16-bit hardware count which wraps at 65536. For position tracking with wraparound handling, use rx_encoder_read_count() instead.

Use Cases:

- Debugging: Verify hardware is counting encoder edges
- Diagnostics: Check for stuck counter (hardware fault)
- Testing: Validate encoder wiring and signal quality
- Low-level access: When software wraparound tracking not needed

Hardware Behavior:

- Counter increments on forward encoder motion (Phase A leads Phase B)
- Counter decrements on reverse encoder motion (Phase B leads Phase A)
- Counter wraps: 65535 -> 0 (forward) or 0 -> 65535 (reverse)
- No software processing applied to value

Parameters:

Direction	Name	Description
in	channel	MTU channel to read Must be valid channel (0-4, 6-7, excluding 5) Must be initialized via rx_encoder_init()
out	count	Pointer to store raw 16-bit counter value Must not be NULL (validated) Value range: [0, 65535] Value unchanged if function returns error

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, count contains current TCNT value
k_rx_err_null_ptr	count pointer is nullptr
k_rx_err_invalid_arg	channel invalid (not 0-4, 6-7)
k_rx_err_invalid_state	Encoder not initialized on this channel

Pre: channel must be initialized via rx_encoder_init()

Pre: count must point to valid uint16_t variable

Post: On success: *count contains current TCNT register value

Post: Hardware counter unchanged (read-only operation)

Note: Thread-safe for read-only access (atomic 16-bit register read)

Note: Very fast operation (200 ns) - single register read

Note: Does NOT handle wraparound - use `rx_encoder_read_count()` for that

Performance:: Execution time: 0.5 us @ 240 MHz Safe to call at very high frequency (> 10 kHz tested)

Example - Diagnostic Check::

```
uint16_t raw_count;
rx_err_t err = rx_encoder_read_raw(k_mtu_channel_1, &raw_count);
if (err != k_rx_ok)
{
    rx_log_info("DIAG", "Rawcounter: %u", raw_count);

    tx_thread_sleep(10);
    uint16_t new_count;
    rx_encoder_read_raw(k_mtu_channel_1, &new_count);
    if (new_count == raw_count)
    {
        rx_log_warn("DIAG", "Encoder not moving - check wiring");
    }
}
```

See also: `rx_encoder_read_count()` Read position with wraparound handling,
`rx_encoder_init()` Must call before reading

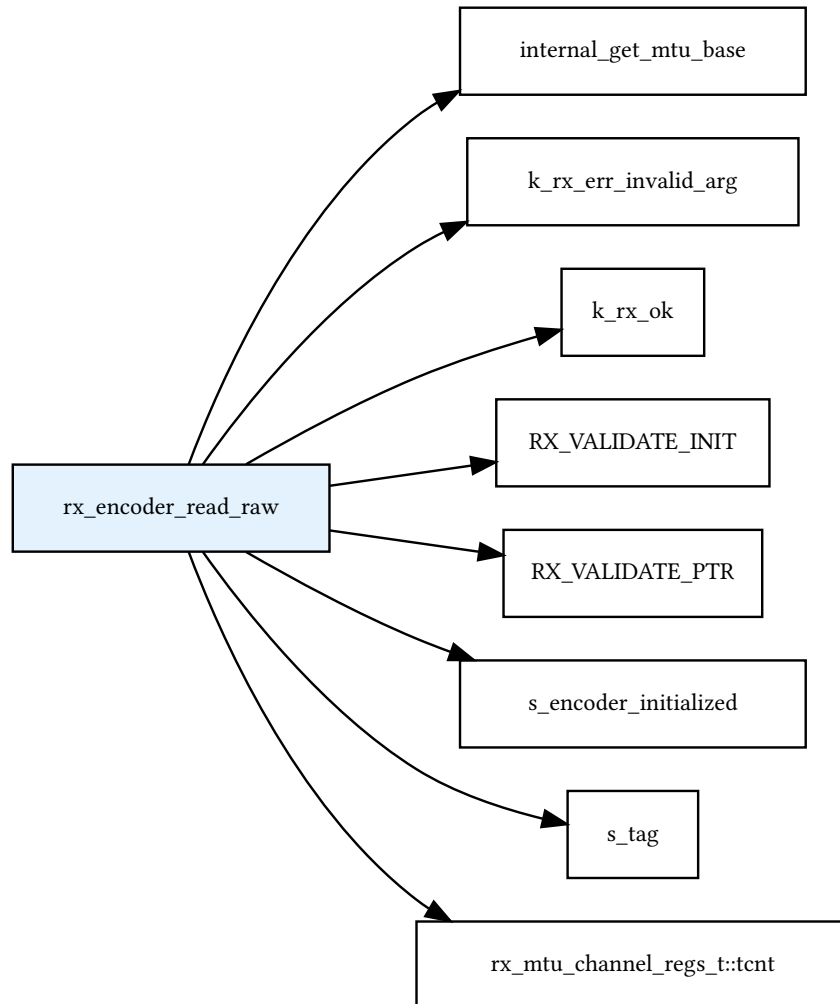


Figure 408: Call graph for rx_encoder_read_raw

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:757_

Since Version 1.0.0

—

rx_encoder_read_count

```
rx_err_t rx_encoder_read_count(const rx_mtu_channel_t channel, rx_encoder_state_t
*state)
```

Read encoder count and position.

Read encoder count with automatic 16-bit overflow handling.

Parameters:

Direction	Name	Description
in	channel	MTU channel
out	state	Pointer to store encoder state (count, position, revolutions)

Returns: k_rx_ok on success, error code otherwise

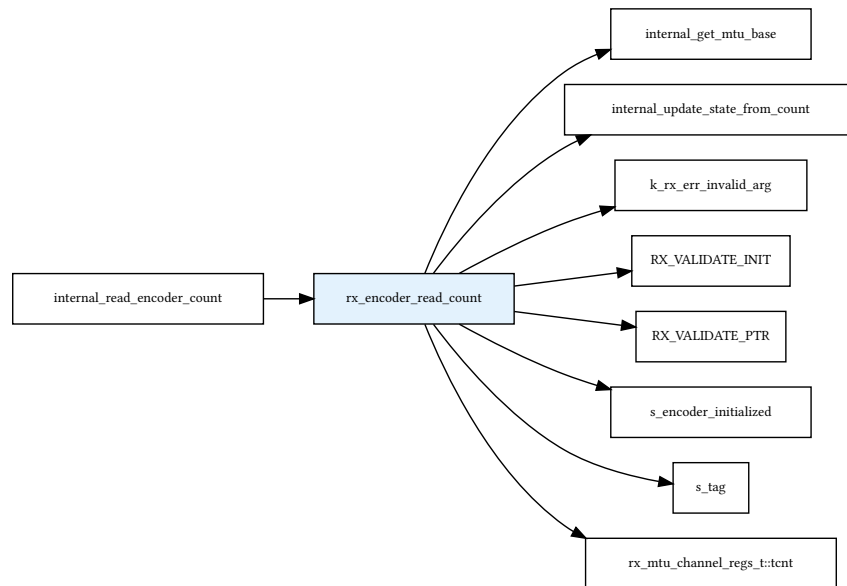


Figure 409: Call graph for `rx_encoder_read_count`

_Defined in `libs/rx\_encoder/src/rx\_mtu\_encoder.c:778_`

—

rx_encoder_read_velocity

```
rx_err_t rx_encoder_read_velocity(float *velocity_rps, const float delta_time_s,
const rx_mtu_channel_t channel)
```

Calculate encoder velocity in revolutions per second from count change.

Read encoder velocity.

Derives angular velocity by measuring the change in encoder position over a known time interval. This function tracks the previous count and calculates delta, then converts to revolutions per second based on the configured `counts_per_revolution` parameter.

Velocity Calculation Algorithm:

Where:

- $\Delta_{count} = total_count[n] - total_count[n-1]$ (count change since last call)

-`counts_per_rev` = encoder resolution (e.g., 1364 for STAR motors) - Δt = time between calls (`delta_time_s` parameter)

STAR Project Example (100 Hz control loop):

Resolution and Accuracy:

- Velocity resolution @ 100 Hz: $1 \text{ count} / (1364 \times 0.01\text{s}) = 0.073 \text{ RPS}$ (4.4 RPM)
- Velocity resolution @ 1 kHz: $1 \text{ count} / (1364 \times 0.001\text{s}) = 0.733 \text{ RPS}$ (44 RPM)
- Higher sample rates -> lower resolution but faster response
- Lower sample rates -> higher resolution but slower response

State Tracking: Function maintains internal state (`s_last_count[channel]`) to calculate delta between calls. First call after init returns velocity based on change since initialization (typically zero).

Velocity resolution inversely proportional to `delta_time_s`

Parameters:

Direction	Name	Description
out	<code>velocity_rps</code>	Pointer to store calculated velocity in revolutions per second Must not be NULL (validated) Units: Revolutions per second (RPS) Positive: Forward rotation, negative: Reverse rotation Range: Typically $[-10, +10]$ RPS for STAR motors (± 600 RPM) Unrealistic values ($> 100 \text{ RPS} = 6000 \text{ RPM}$) trigger warning but succeed
in	<code>delta_time_s</code>	Time interval since last velocity measurement Must be > 0 and ≤ 10 seconds (validated to catch parameter swaps) Units: Seconds (s) Typical: 0.01s (100 Hz control loop) or 0.001s (1 kHz) Smaller values -> lower resolution, faster response Larger values -> higher resolution, slower response
in	<code>channel</code>	MTU channel to read Must be valid channel (0-4, 6-7, excluding 5) Must be initialized via <code>rx_encoder_init()</code>

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, velocity calculated and written to <code>*velocity_rps</code>
<code>k_rx_err_null_ptr</code>	<code>velocity_rps</code> pointer is nullptr
<code>k_rx_err_invalid_arg</code>	<code>delta_time_s</code> ≤ 0 or > 10 (possible parameter swap with channel)
<code>k_rx_err_invalid_arg</code>	channel invalid (not 0-4, 6-7)
<code>k_rx_err_invalid_state</code>	Encoder not initialized on this channel
<code>k_rx_err_invalid_state</code>	<code>counts_per_rev</code> < 1 (state corrupted)

Pre: channel must be initialized via `rx_encoder_init()`

Pre: `delta_time_s` must match actual time since last call for accurate results

Pre: `velocity_rps` must point to valid float variable

Post: On success: `*velocity_rps` contains calculated angular velocity

Post: Internal last_count updated for next delta calculation

Post: Warning logged if velocity exceeds 100 RPS (possible encoder fault)

Note: Not thread-safe, must call from single task or with external mutex

Note: First call after init returns zero velocity (no previous count to compare)

Note: Parameter order designed to prevent common bugs (output first, time second, channel last)

Note: Unrealistic velocity (> 100 RPS) logs warning but does not return error

Warning: delta_time_s must be accurate - timing errors propagate to velocity error

Warning: Very long delta_time (> 1s) may miss wraparounds if motor speed high

Performance:: Execution time: 3 us @ 240 MHz Suitable for 100 Hz - 10 kHz control loops STAR platform: Called at 100 Hz from PID velocity controller

Example - 100 Hz Control Loop:: void motor_control_task(void*arg)
 { rx_mtu_channel_t encoder_ch=k_mtu_channel_1; const float dt_s=0.01f;
 while(1){ float velocity_rps; rx_err_t err=rx_encoder_read_velocity(&velocity_rps,dt_s,encoder_ch);
 if(err!=k_rx_ok){ float pid_output=pid_compute(&pid,target_rps,velocity_rps,dt_s);
 motor_set_duty(&motor,pid_output); }
 tx_thread_sleep(10); } }

Example - Velocity Monitoring:: const float dt_s=0.1f; float velocity_rps;
 rx_err_t err=rx_encoder_read_velocity(&velocity_rps,dt_s,k_mtu_channel_1); if(err!=k_rx_ok)
 { float velocity_rpm=velocity_rps*60.0f; rx_log_info("MOTOR","Velocity:
 %.1fRPM(%.2fRPS)",velocity_rpm,velocity_rps);
 if(fabsf(velocity_rps)>5.0f){ rx_log_warn("MOTOR","Motor overspeed detected"); } }

Example - Error Handling:: float velocity;
 rx_err_t err=rx_encoder_read_velocity(&velocity,0.01f,k_mtu_channel_1);
 switch(err){ case k_rx_ok: break; case k_rx_err_invalid_arg:
 rx_log_error("APP","Invalid delta time or channel"); break; case k_rx_err_invalid_state:
 rx_log_error("APP","Encoder not initialized or state corrupted"); break; default:
 rx_log_error("APP","Unexpected error: %d",err); break; }

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 3 preconditions (NULL check, time range check, initialized check) Rule 5: [OK] 2 postconditions (velocity calculated, last_count updated) Rule 7: [OK] Division guard (counts_per_rev validated before division)

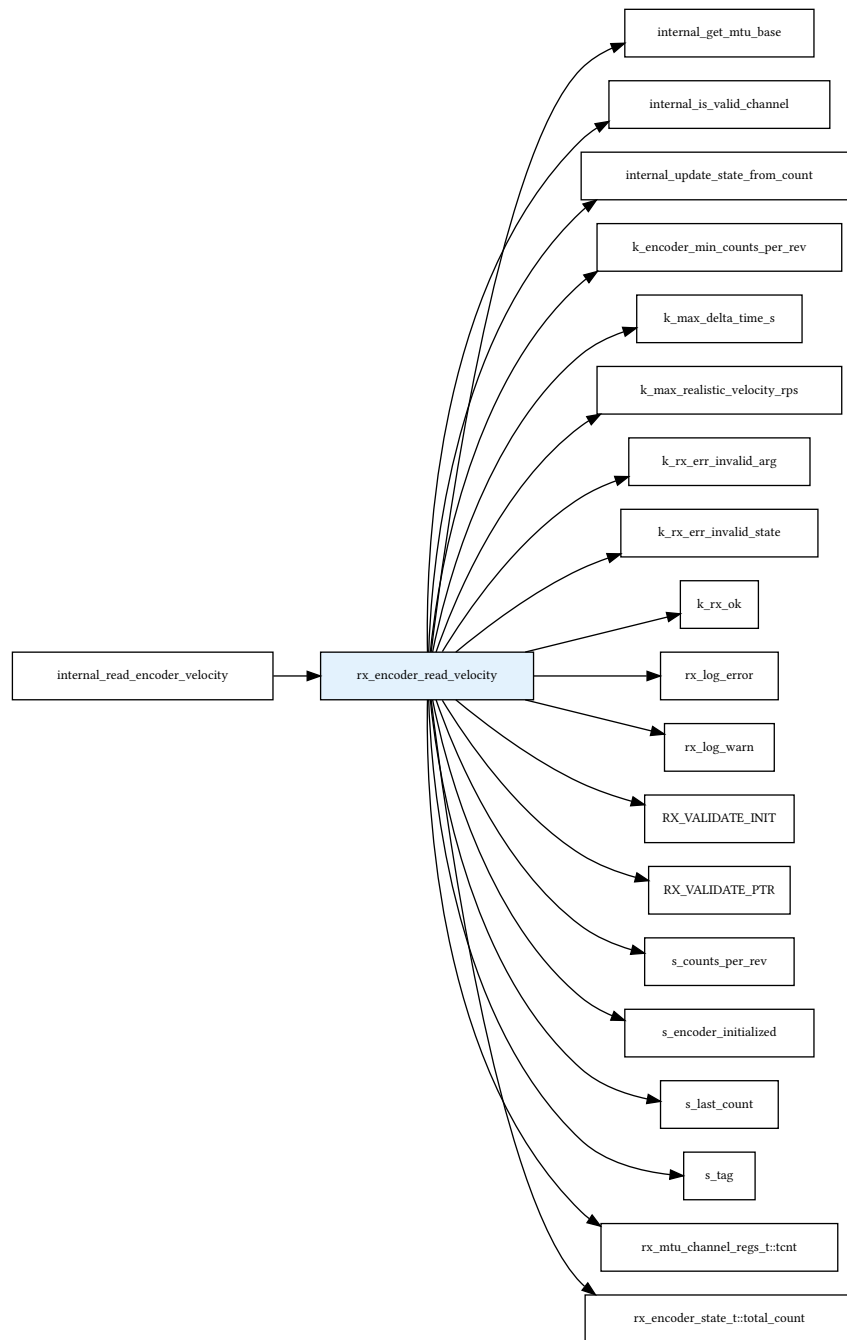
Example:

Motor: 210RPMnominal=3.5RPS
Encoder: 1364counts/rev(341PPRx4xdecoding)
Loopperiod: 10ms=0.01seconds

Expectedcountchangepercall:
 $\text{delta_count} = 3.5\text{RPS} \times 1364\text{counts/rev} \times 0.01\text{s} = 47.74\text{counts}$

Ifmeasureddelta_count=48:
 $\text{velocity} = 48 / (1364 \times 0.01) = 3.52\text{RPS} = 211.2\text{RPM}$

See also: rx_encoder_init() Must call before using this function,
rx_encoder_read_count() Alternative for position-based velocity, rx_pid_compute() PID
controller uses this for feedback

Figure 410: Call graph for `rx_encoder_read_velocity`

_Defined in `libs/rx\_encoder/src/rx\_mtu\_encoder.c:955_`

Since Version 1.0.0

rx_encoder_reset

```
rx_err_t rx_encoder_reset(const rx_mtu_channel_t channel)
```

Reset encoder position to zero (home/calibration operation).

Reset encoder count to zero.

Resets both hardware counter (TCNT register) and software state to zero, establishing a new zero reference position. This is typically used during system initialization, homing sequences, or when the robot reaches a known calibration point.

Reset Actions:

1. Hardware: TCNT register set to 0
2. Software: total_count = 0
3. Software: revolutions = 0
4. Software: position_deg = 0.0deg
5. Software: last_raw_count = 0
6. Velocity: s_last_count = 0 (next velocity call measures from this point)

Use Cases:

- Homing: Reset when limit switch or home sensor triggered
- Calibration: Reset at known mechanical position
- Error recovery: Clear accumulated position after fault
- Testing: Reset between test runs

Hardware counter continues incrementing after reset if motor moving

Parameters:

Direction	Name	Description
in	channel	MTU channel to reset Must be valid channel (0-4, 6-7, excluding 5) Must be initialized via rx_encoder_init()

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, encoder reset to zero
k_rx_err_invalid_arg	channel invalid (not 0-4, 6-7)
k_rx_err_invalid_state	Encoder not initialized on this channel

Pre: channel must be initialized via rx_encoder_init()

Pre: No other tasks reading encoder during reset (race condition possible)

Post: Hardware TCNT = 0

Post: Software state zeroed (count=0, position=0deg, rev=0)

Post: Next velocity measurement starts from zero reference

Note: Not thread-safe, ensure exclusive access during reset

Note: Motor may be moving during reset - count starts accumulating immediately

Note: Does NOT physically stop motor (use rx_motor_stop for that)

Warning: Do not call while other tasks are reading encoder (race condition)

Performance:: Execution time: 1 us @ 240 MHz (single register write + state clear)

Example - Homing Sequence:: motor_set_duty(&motor,-20.0f);

```
while(gpio_read(LIMIT_SWITCH_PIN)==GPIO_HIGH){ tx_thread_sleep(1); }
```

```
motor_stop(&motor,false); rx_encoder_reset(k_mtu_channel_1);
```

```
rx_log_info("APP","Homingcomplete-encoderresettozero");
```

Example - Periodic Calibration:: voidcalibrate_encoder(rx_mtu_channel_tchannel)

```
{ rx_encoder_state_tstate; rx_encoder_read_count(channel,&state);
```

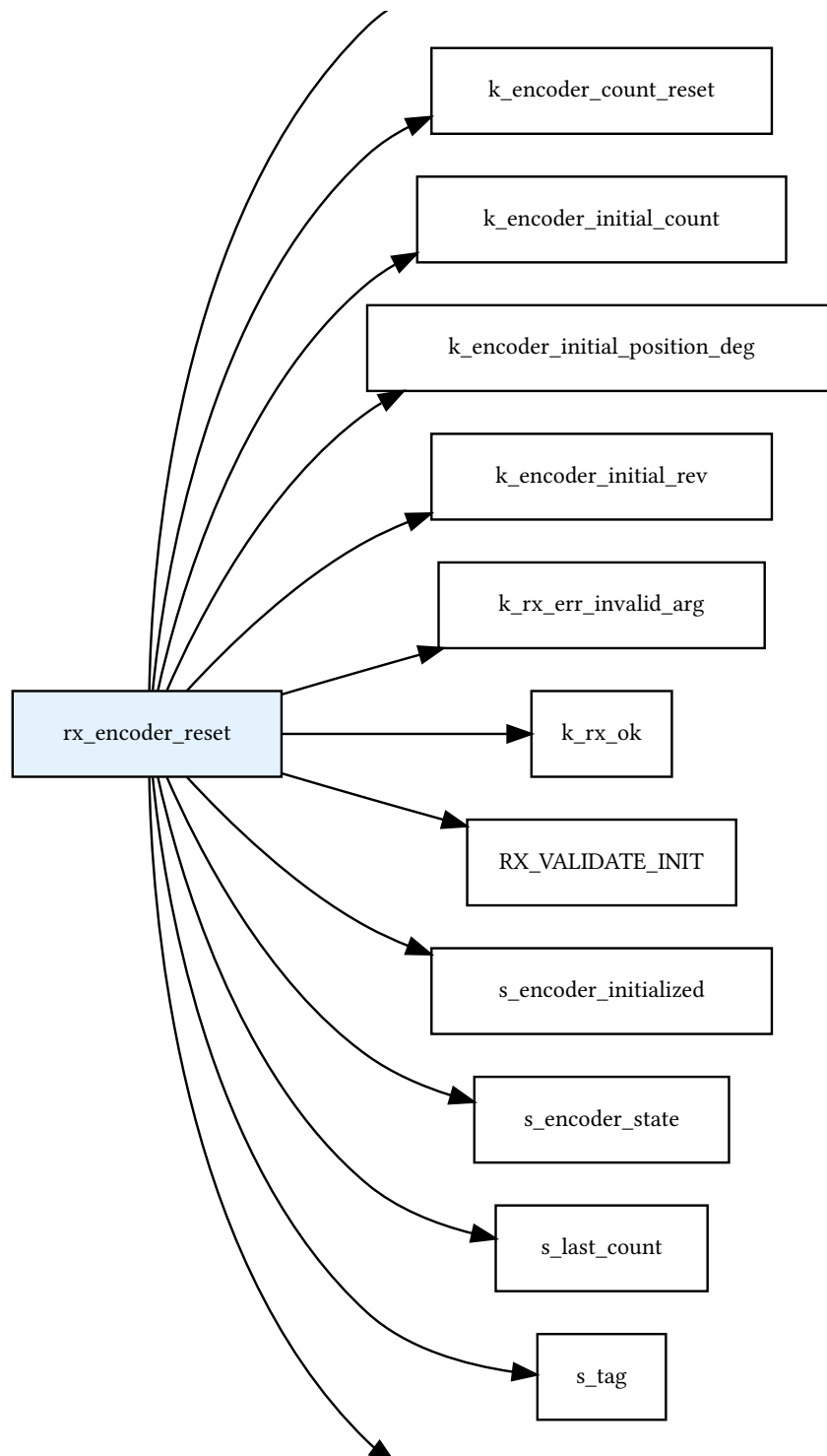
```
rx_log_info("CAL","Pre-calibration:%dcounts,%drevs", state.total_count,state.revolutions);
```

```
rx_err_terr=rx_encoder_reset(channel); if(err==k_rx_ok)
```

```
{ rx_log_info("CAL","Encodercalibratedatdockposition"); } }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (channel valid, initialized) Rule 5: [OK] 2 postconditions (TCNT=0, state zeroed)

See also: rx_encoder_set_count() Set encoder to specific non-zero value,
rx_encoder_init() Initial setup, rx_motor_stop() Stop motor before reset if stationary
reset needed

Figure 411: Call graph for `rx_encoder_reset`

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:1107_

Since Version 1.0.0

—

rx_encoder_set_count

```
rx_err_t rx_encoder_set_count(const int32_t count, const rx_mtu_channel_t  
channel)
```

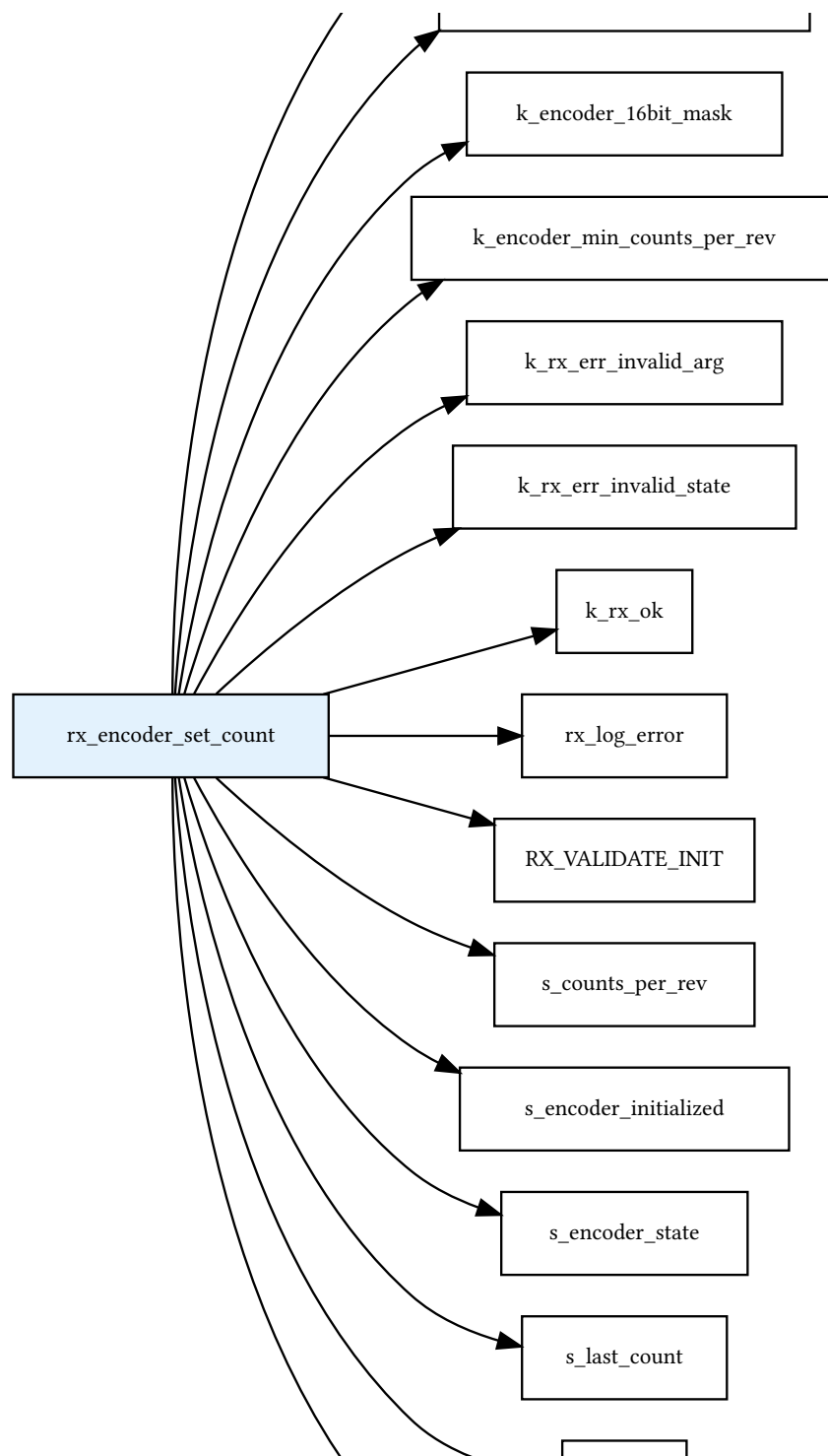
Set encoder count to specific value.

Set encoder count value.

Parameters:

Direction	Name	Description
in	count	Desired count value
in	channel	MTU channel

Returns: k_rx_ok on success, error code otherwise

Figure 412: Call graph for `rx_encoder_set_count`

_Defined in libs/rx_encoder/src/rx_mtu_encoder.c:1137_

—

rx_encoder_deinit

```
rx_err_t rx_encoder_deinit(const rx_mtu_channel_t channel)
```

Deinitialize encoder and release MTU peripheral resources.

Deinitialize encoder.

Performs orderly shutdown of encoder by stopping MTU timer and clearing initialization flag. After deinitialization, encoder can be reinitialized with different configuration parameters via rx_encoder_init().

Deinitialization Sequence:

1. Validate channel is valid and initialized
2. Stop MTU timer (halt edge counting)
3. Clear initialized flag (mark channel as uninitialized)
4. Clear counts_per_rev (prevent accidental division by stale value)

Use Cases:

- System shutdown: Release resources before power-down
- Reconfiguration: Change encoder parameters (requires deinit -> init)
- Error recovery: Clean up after hardware fault
- Resource sharing: Free MTU channel for other use

Encoder state preserved but inaccessible until reinitialized

Parameters:

Direction	Name	Description
in	channel	MTU channel to deinitialize Must be valid channel (0-4, 6-7, excluding 5) Must be initialized (double-deinit returns error)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, encoder deinitialized and timer stopped
k_rx_err_invalid_arg	channel invalid (not 0-4, 6-7)
k_rx_err_invalid_state	Encoder not initialized (already deinitialized)

Pre: channel must be initialized via rx_encoder_init()

Pre: No other tasks are currently accessing this encoder

Post: s_encoder_initializedchannel = false

Post: MTU timer stopped (no longer counting)

Post: s_counts_per_revchannel = 0

Post: Other state (total_count, position) preserved but inaccessible

Note: If rx_mtu_stop() fails, warning logged but deinit continues

Note: Timer stop affects entire channel (shared resource)

Note: Thread-safe only with external synchronization

Warning: Ensure no other tasks are using encoder before calling

Performance:: Execution time: 2 us @ 240 MHz (timer stop + flag clear) Called once per encoder during shutdown (not performance-critical)

Example - Clean Shutdown:: rx_encoder_state_tfinal_state;
rx_encoder_read_count(k_mtu_channel_1,&final_state); rx_log_info("SHUTDOWN","Finalposition: %dcounts",final_state.total_count);

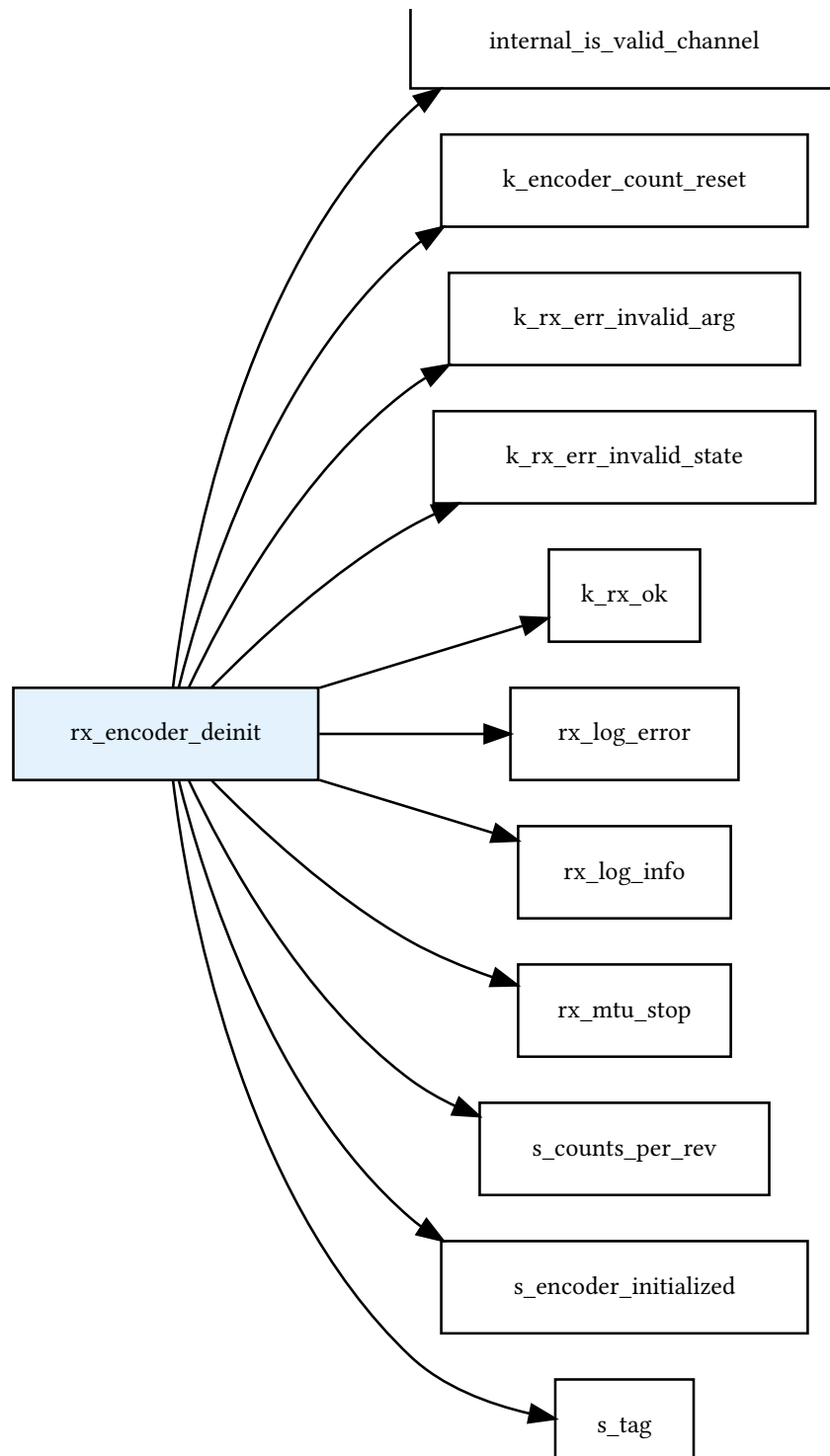
```
rx_err_t err=rx_encoder_deinit(k_mtu_channel_1); if(err!=k_rx_ok)
{ rx_log_error("SHUTDOWN","Failed to deinit encoder: %d",err); }
```

Example - Reconfiguration:: rx_encoder_deinit(k_mtu_channel_1);

```
rx_encoder_config_t new_config={ .channel=k_mtu_channel_1, .counts_per_rev=2048, .invert_direction=false, };
rx_encoder_init(&new_config);
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (channel valid, initialized) Rule 5: [OK] 1 postcondition (initialized flag cleared)

See also: rx_encoder_init() Reinitialize after deinit, rx_mtu_stop() Underlying timer stop function

Figure 413: Call graph for `rx_encoder_deinit`

_Defined in `libs/rx\_encoder/src/rx\_mtu\_encoder.c:1257_`

Since Version 1.0.0

—

rx_fec.h

Forward Error Correction (FEC) Codec for RX72N.

Implements NASA-standard K=7 constraint length convolutional encoder and soft-decision Viterbi decoder for reliable communication over noisy channels. This is a C port of the Go FEC implementation in star-gateway, ensuring bit-exact compatibility between RX72N firmware and RPi5 gateway.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_bit_constants.h"`
- `"rx_err.h"`

rx_fec_params_t

FEC convolutional code parameters.

Defines the NASA-standard K=7, rate 1/2 convolutional code used throughout the STAR project. These parameters were selected for optimal balance between coding gain (5 dB @ BER 10^{-5}) and computational complexity.

Value	Name	Description
= 7	<code>k_fec_constraint_length</code>	Constraint length K=7 (shift register holds K-1 = 6 bits).
= 64	<code>k_fec_num_states</code>	Tail bits for termination: K-1 = 6 zeros.
= 6	<code>k_fec_tail_bits</code>	Generator polynomial G1 in octal: 171 (binary 1111001).
= 0171	<code>k_fec_g1_octal</code>	Generator polynomial G2 in octal: 133 (binary 1011011).
= 0133	<code>k_fec_g2_octal</code>	Traceback depth: $5 \times K = 35$ symbols.
= 35	<code>k_fec_traceback_depth</code>	Number of possible input values: 2 (binary: 0 or 1).
= 2	<code>k_fec_num_input_values</code>	Number of output bits per input bit: 2 (rate 1/2).
= 2	<code>k_fec_num_outputs</code>	

—

rx_soft_bit_limits_t

Soft bit value range limits.

Defines the valid range for soft bit values. These limits ensure that soft bits fit in an `int8_t` and provide symmetric positive/negative ranges.

Value	Name	Description
= 127	<code>k_soft_bit_max</code>	Maximum soft bit value: +127 (confident bit=1).
= -127	<code>k_soft_bit_min</code>	Zero confidence: 0 (erasure).
= 0	<code>k_soft_bit_zero</code>	

—

rx_fec_buffer_limits_t

Maximum buffer sizes for static FEC decoder allocation.

Defines compile-time buffer limits to enable zero dynamic allocation (NASA Rule 3). Values calculated from maximum payload size (1024 bytes) and FEC parameters.

Value	Name	Description
= 8200	k_fec_max_symbols	Maximum symbols for Viterbi trellis: 8200.
= (uint16_t)((k_fec_max_symbols - k_fec_tail_bits) / k_rx_bits_per_byte)	k_fec_max_input_bytes	

—

rx_soft_bit_t

```
typedef int8_t rx_soft_bit_t
```

Soft bit type for soft-decision decoding.

Soft bits carry both demodulated bit value and reliability information. This enables the Viterbi decoder to make better decisions than hard-decision decoding, providing approximately 2 dB coding gain improvement.

—

rx_fec_encoder_init

```
rx_err_t rx_fec_encoder_init(rx_fec_encoder_t *enc)
```

Initialize FEC encoder.

Prepares encoder for use by setting initialization flag. The encoder itself is stateless (no shift register persists between calls), so initialization only validates the handle.

Algorithm:

1. Validate enc pointer (NULL check)
2. Set enc->initialized = true
3. Return success

Parameters:

Direction	Name	Description
out	enc	Pointer to encoder handle to initialize Must not be nullptr Contents will be overwritten Lifetime: Caller-managed, must persist until deinit
in	enc	Pointer to encoder structure

Returns: k_rx_ok on success, k_rx_err_invalid_arg if enc is nullptr

Value	Description
k_rx_ok	Successfully initialized, encoder ready for use
k_rx_err_invalid_arg	enc is nullptr

Pre: enc must point to valid rx_fec_encoder_t storage

Pre: Sufficient stack/heap space for encoder structure (4 bytes)

Post: enc->initialized == true

Post: Encoder is ready for rx_fec_encode() calls

Note: Thread Safety: Safe to call concurrently on different encoder handles. NOT safe to call concurrently on the same handle.

Note: Re-initialization: Safe to call on already-initialized encoder (idempotent operation).]

Example: rx_fec_encoder_t encoder; rx_err_t err=rx_fec_encoder_init(&encoder); if(err!=k_rx_ok){ return err; }

```
uint8_t input[100]={...}; uint8_t output[250]; uint32_t output_len;
rx_fec_encode(&encoder,input,100,output,&output_len);
rx_fec_encoder_deinit(&encoder);
```

See also: rx_fec_encoder_deinit() Deinitialize encoder, rx_fec_encode() Encode data

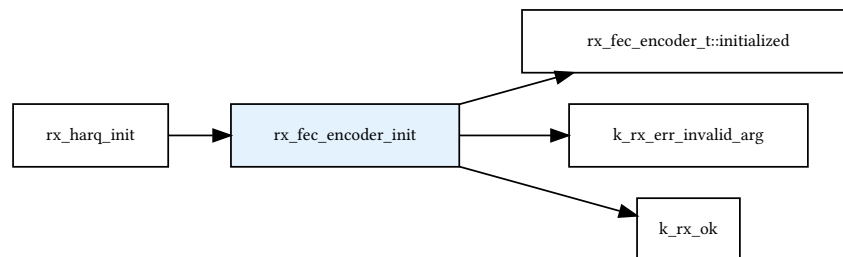


Figure 414: Call graph for rx_fec_encoder_init

Defined in `libs/rx_fec/inc/rx_fec.h:585`

Since Version 1.0.0

—

rx_fec_encoder_deinit

```
rx_err_t rx_fec_encoder_deinit(rx_fec_encoder_t *enc)
```

Deinitialize FEC encoder.

Marks encoder as uninitialized. After this call, encoder must be re-initialized before use. This function provides symmetric init/deinit API for resource management patterns.

Algorithm:

1. Validate enc pointer (NULL check)
2. Set enc->initialized = false
3. Return success

Parameters:

Direction	Name	Description
inout	enc	Pointer to encoder handle to deinitialize Must not be nullptr May be uninitialized (idempotent) Contents will be cleared
in	enc	Pointer to encoder structure

Returns: k_rx_ok on success, k_rx_err_invalid_arg if enc is nullptr

Value	Description
k_rx_ok	Successfully deinitialized
k_rx_err_invalid_arg	enc is nullptr

Pre: enc must point to valid memory (may be uninitialized)

Post: enc->initialized == false

Post: Encoder cannot be used until rx_fec_encoder_init() called again

Note: Thread Safety: Safe to call concurrently on different encoder handles. NOT safe to call concurrently on the same handle.

Note: Idempotent: Safe to call multiple times on same encoder.

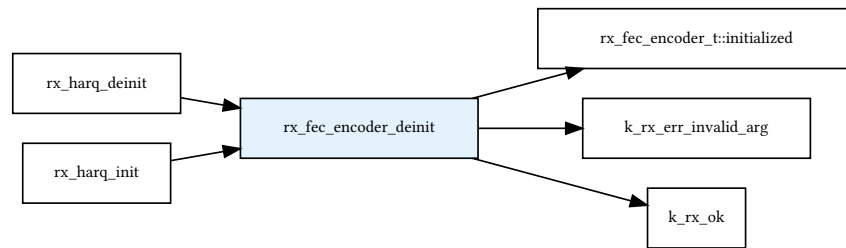
Warning: Do NOT call rx_fec_encode() after deinit - will return k_rx_err_invalid_state]

Example: rx_fec_encoder_tencoder; rx_fec_encoder_init(&encoder);

```
rx_err_t err=rx_fec_encoder_deinit(&encoder); assert(err==k_rx_ok);
```

```
rx_fec_encoder_init(&encoder);
```

See also: rx_fec_encoder_init() Initialize encoder

Figure 415: Call graph for `rx_fec_encoder_deinit`

Defined in `libs/rx_fec/inc/rx_fec.h:640`

Since Version 1.0.0

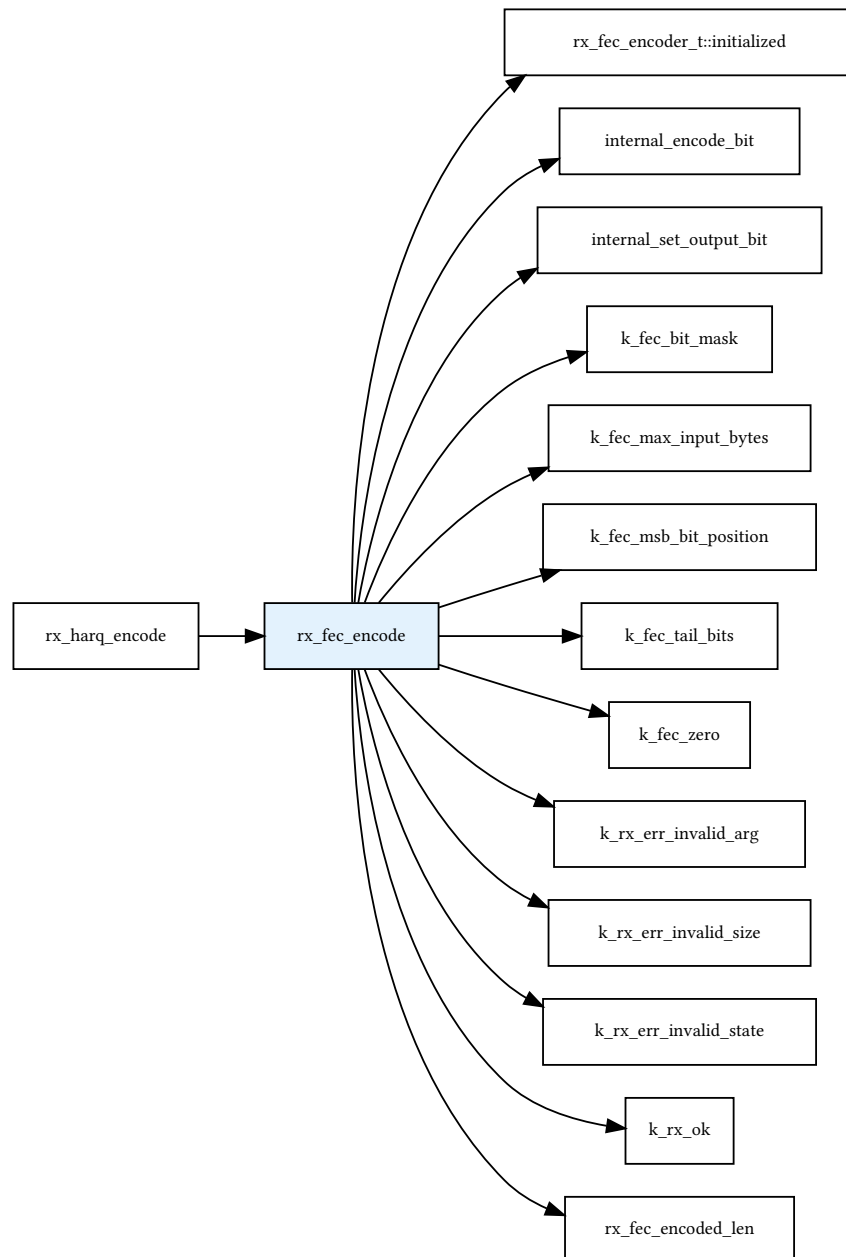
—

rx_fec_encode

```
rx_err_t rx_fec_encode(const rx_fec_encoder_t *enc, const uint8_t *input,  
uint32_t input_len, uint8_t *output, uint32_t *output_len)
```

Encode data using K=7, rate 1/2 convolutional code.

Encodes input bytes into FEC-protected output using NASA-standard convolutional code. Each input bit produces 2 output bits (G1 and G2), followed by 6 tail bits to flush encoder to zero state.

Figure 416: Call graph for `rx_fec_encode`

Defined in `libs/rx_fec/inc/rx_fec.h:838`

—

rx_fec_encoded_len

```
uint32_t rx_fec_encoded_len(uint32_t input_len)
```

Calculate FEC encoded output length.

Computes the number of bytes required to store FEC-encoded output for a given input length. This function should be called before rx_fec_encode() to allocate appropriately-sized output buffers.

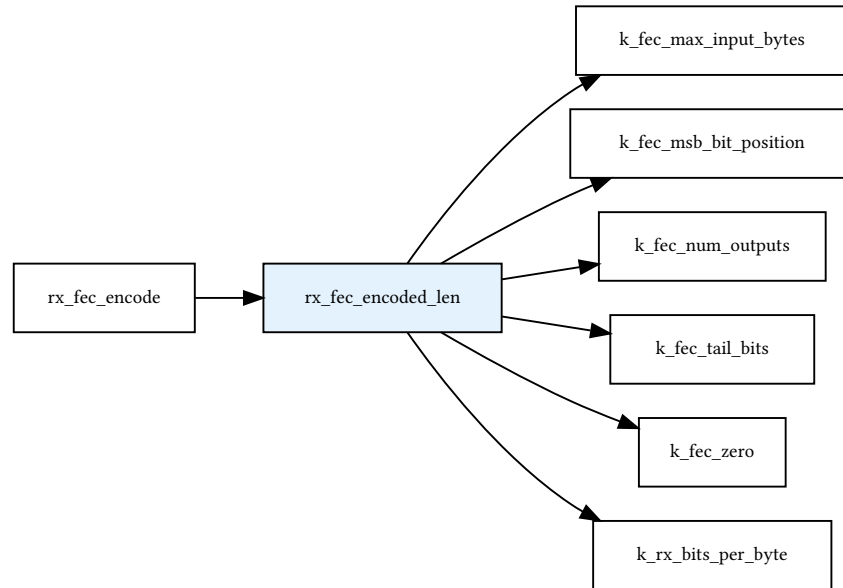


Figure 417: Call graph for rx_fec_encoded_len

Defined in `libs/rx_fec/inc/rx_fec.h:943`

—

rx_fec_decoder_init

```
rx_err_t rx_fec_decoder_init(rx_fec_decoder_t *dec, uint64_t *survivors_buf,
uint32_t survivors_len)
```

Initialize FEC decoder.

The caller must provide a survivors buffer for traceback. Size should be at least $(\text{max_symbols} / 64 + 1) * \text{k_fec_num_states}$ `uint64_t` elements.

Initializes decoder state and associates it with provided survivors buffer. The buffer is used for Viterbi algorithm path storage.

Parameters:

Direction	Name	Description
out	dec	Pointer to decoder handle

in	survivors_buf	Caller-provided buffer for survivor bits
in	survivors_len	Number of uint64_t elements in survivors_buf
inout	dec	Pointer to decoder structure
in	survivors_buf	Pointer to survivors buffer for Viterbi algorithm
in	survivors_len	Length of survivors buffer in uint64_t entries

Returns: k_rx_err_invalid_size if survivors_len is 0

Pre: dec and survivors_buf must be non-nullptr

Pre: survivors_len must be > 0 (minimum 1 entry per symbol)

Post: dec->initialized set to true

Post: Branch table initialized

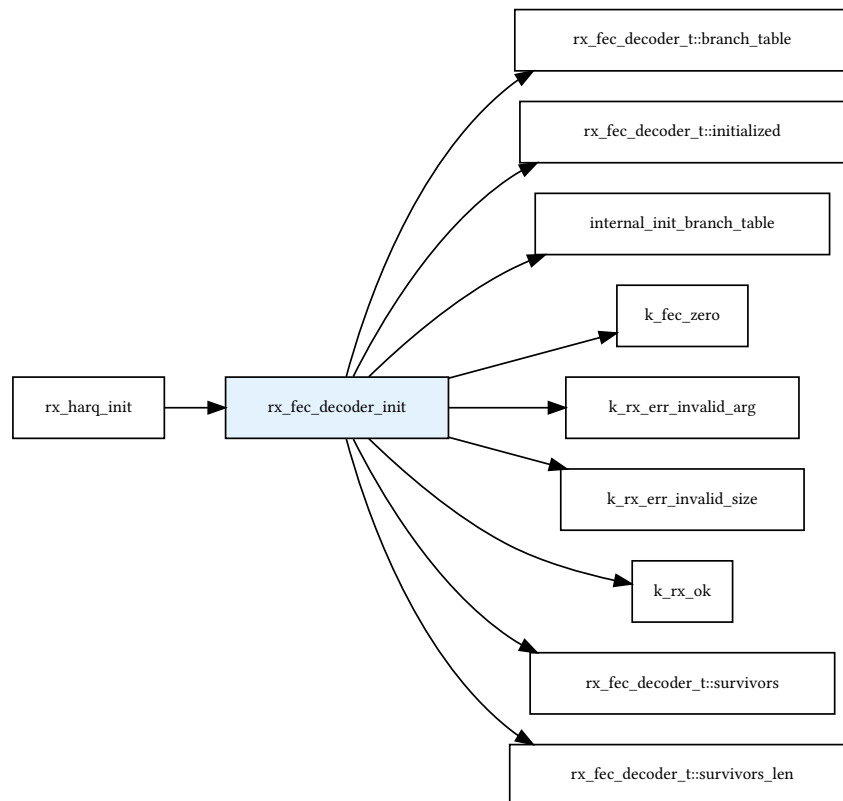


Figure 418: Call graph for `rx_fec_decoder_init`

Defined in `libs/rx_fec/inc/rx_fec.h:1028`

rx_fec_decoder_deinit

```
rx_err_t rx_fec_decoder_deinit(rx_fec_decoder_t *dec)
```

Deinitialize FEC decoder.

Parameters:

Direction	Name	Description
inout	dec	Pointer to decoder handle
in	dec	Pointer to decoder structure

Returns: k_rx_ok on success, k_rx_err_invalid_arg if dec is nullptr

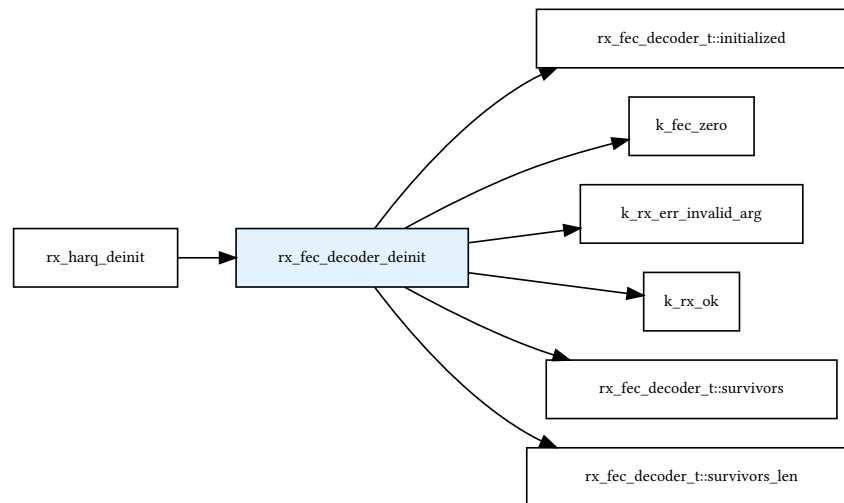


Figure 419: Call graph for rx_fec_decoder_deinit

Defined in `libs/rx_fec/inc/rx_fec.h:1036`

rx_fec_decode_soft

```
rx_err_t rx_fec_decode_soft(rx_fec_decoder_t *dec, const
rx_fec_decode_soft_params_t *params)
```

Decode soft bits using Viterbi algorithm.

Performs soft-decision Viterbi decoding on received soft bits. Soft bits should be in pairs (G1, G2) for each encoded symbol.

Decode soft bits using Viterbi algorithm.

Implements Rate-1/2 Viterbi soft-decision decoding with constraint length 7. Accepts soft-decision values (signed integers in range $[-128, 127]$) to achieve approximately 3 dB coding gain over hard-decision decoding.

Algorithm steps:

1. Validate parameters via `internal_validate_decode_params()`
2. Forward pass: compute branch and path metrics for all symbols
3. Calculate data bit count (`num_symbols - tail bits`)
4. Traceback: extract maximum-likelihood decoded bit sequence
5. Pack decoded bits into output byte array

Parameters:

Direction	Name	Description
in	<code>dec</code>	Initialized decoder handle
inout	<code>params</code>	Decode parameters (input soft bits, output buffer)
in	<code>dec</code>	Pointer to initialized FEC decoder (must be initialized via <code>rx_fec_decoder_init()</code> ; survivors buffer must be sized for input)
in	<code>params</code>	Decode parameters: <code>soft_bits</code> : signed soft values, G1/G2 interleaved pairs <code>soft_len</code> : must be non-zero and divisible by 2 <code>expected_output_len</code> : expected decoded byte count (> 0) <code>output</code> : output buffer (must be $\geq$ <code>expected_output_len</code> bytes) <code>output_len</code> : written with actual decoded byte count on success

Returns: `rx_err_t` Status code

Value	Description
<code>k_rx_ok</code>	Decoding succeeded; <code>output_len</code> updated with byte count
<code>k_rx_err_invalid_arg</code>	<code>dec</code> or <code>params</code> (or required fields) are nullptr; or <code>soft_len</code> is zero or not divisible by 2
<code>k_rx_err_invalid_state</code>	<code>dec</code> is not initialized
<code>k_rx_err_invalid_size</code>	<code>expected_output_len</code> out of range; or derived <code>num_symbols</code> falls outside <code>[k_fec_tail_bits, k_fec_max_symbols]</code> ; or <code>data_bits == 0</code> after subtracting tail bits

Pre: `dec` must be initialized via `rx_fec_decoder_init()`

Pre: `params->soft_len` must be divisible by 2 (G1/G2 symbol pairs)

Pre: `params->expected_output_len` must be > 0 and $\leq k_fec_max_input_bytes$

Post: `*params->output_len` written with actual decoded byte count on `k_rx_ok`

Post: `params->output` contains decoded data bytes on `k_rx_ok`

Post: dec path metric arrays updated (internal state consumed by traceback)

Note: Not thread-safe; caller must ensure exclusive access to dec and params

Note: Soft values: positive = bit-1 likely, negative = bit-0 likely (signed).] **Example:**

```
//Example:soft-decisiondecodeofapreviouslyFEC-encodedbuffer
rx_fec_decoder_tdec;
uint64_tsurvivors[k_fec_max_symbols];
rx_soft_bit_tsoft_buf[k_fec_max_symbols*k_fec_num_outputs];
uint8_tout_buf[k_fec_max_input_bytes];
uint32_tout_len=k_fec_zero;

(void)rx_fec_decoder_init(&dec,survivors,k_fec_max_symbols);

//Populatesoft_buffromchannel(positive=bit-1likely)
rx_fec_decode_soft_params_tp={
    .soft_bits=soft_buf,
    .soft_len=encoded_len_bits,
    .expected_output_len=original_data_len,
    .output=out_buf,
    .output_len=&out_len,
};
rx_err_terr=rx_fec_decode_soft(&dec,&p);
```

See also: rx_fec_decoder_init() Initialize decoder before calling this function,
rx_fec_decode_hard() Hard-decision wrapper (lower coding gain, simpler input),
rx_fec_encode() Encoder that produces the data this function decodes

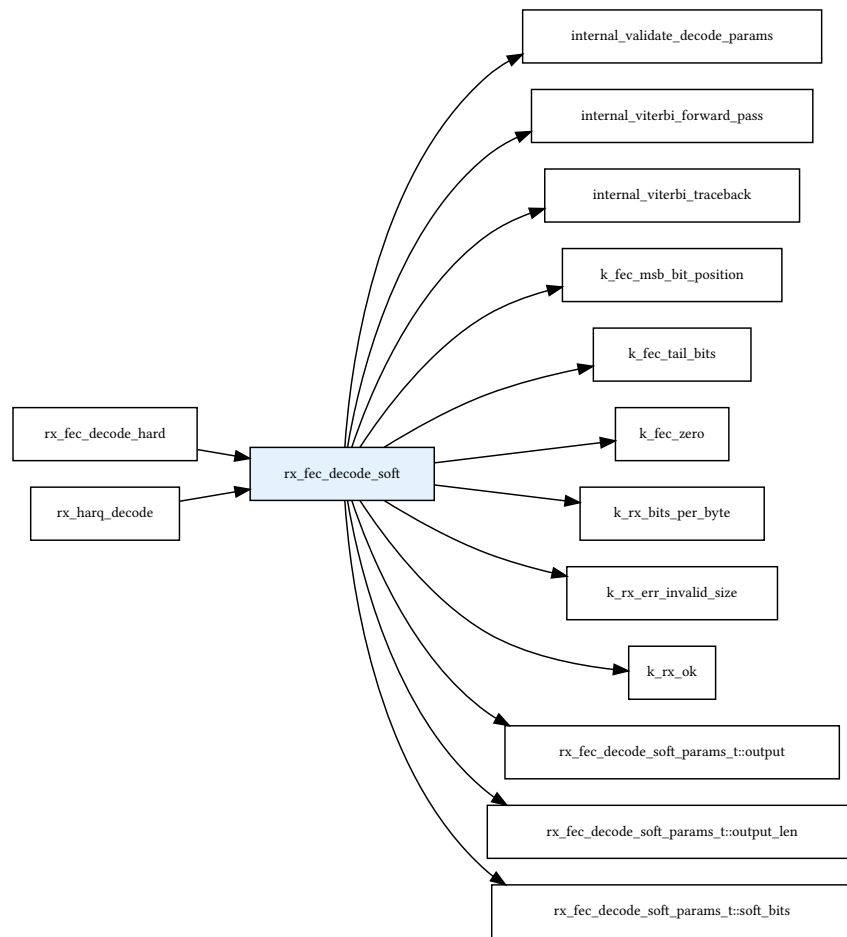


Figure 420: Call graph for rx_fec_decode_soft

Defined in `libs/rx_fec/inc/rx_fec.h:1065`

Since Version 1.0.0

—

rx_fec_decode_hard

```
rx_err_t rx_fec_decode_hard(rx_fec_decoder_t *dec, const
rx_fec_decode_hard_params_t *params)
```

Decode hard bits using Viterbi algorithm.

Converts hard bits to soft bits internally, then performs Viterbi decoding. Less accurate than soft-decision decoding but useful for testing.

The caller must provide a working buffer for soft bit conversion to ensure thread safety. Size should be at least `k_fec_max_symbols * k_fec_num_outputs`.

Decode hard bits using Viterbi algorithm.

Convenience wrapper that converts hard-decision bits (0/1) to soft-decision values and delegates to `rx_fec_decode_soft()`. Accepts raw bit-packed data (MSB-first) as produced by standard digital logic or RSPI transfers.

Algorithm steps:

1. Validate all input pointers and size constraints
2. Convert each hard bit to a soft value via `rx_fec_hard_to_soft()` (maps 0 -> negative confidence, 1 -> positive confidence)
3. Build `rx_fec_decode_soft_params_t` from the converted soft bits
4. Delegate to `rx_fec_decode_soft()` for full Viterbi decode

Parameters:

Direction	Name	Description
in	<code>dec</code>	Initialized decoder handle
inout	<code>params</code>	Decode parameters (input data, buffers)
in	<code>dec</code>	Pointer to initialized FEC decoder (must be initialized via <code>rx_fec_decoder_init()</code> ; survivors buffer sized for decoded output)
in	<code>params</code>	Decode parameters: <code>data</code> : bit-packed hard-decision input (MSB-first per byte) <code>data_len</code> : byte count of hard data; must be > 0 and $\leq k_fec_max_input_bytes$ <code>soft_bits_buffer</code> : caller-supplied scratch buffer for soft conversion; must be $\geq data_len * 8$ entries <code>soft_buffer_len</code> : element count of <code>soft_bits_buffer</code> <code>expected_output_len</code> : expected decoded byte count <code>output</code> : output buffer (must be $\geq expected_output_len$ bytes) <code>output_len</code> : written with actual decoded byte count on success

Returns: `rx_err_t` Status code

Value	Description
<code>k_rx_ok</code>	Decoding succeeded; <code>output_len</code> updated
<code>k_rx_err_invalid_arg</code>	<code>dec</code> or required <code>params</code> fields are nullptr; or <code>data_len == 0</code>
<code>k_rx_err_invalid_state</code>	<code>dec</code> is not initialized
<code>k_rx_err_invalid_size</code>	<code>data_len > k_fec_max_input_bytes</code> ; or <code>soft_bits_buffer</code> too small for converted bits; or errors propagated from <code>rx_fec_decode_soft()</code>

Pre: `dec` must be initialized via `rx_fec_decoder_init()`

Pre: `params->soft_bits_buffer` must be large enough: `soft_buffer_len  $\geq$  data_len * 8`

Pre: `params->data_len` must be > 0 and $\leq k_fec_max_input_bytes$

Post: `*params->output_len` written with actual decoded byte count on `k_rx_ok`

Post: params->output contains decoded data bytes on k_rx_ok

Post: params->soft_bits_buffer populated with converted soft values (side effect)

Note: Not thread-safe; caller must ensure exclusive access to dec and params

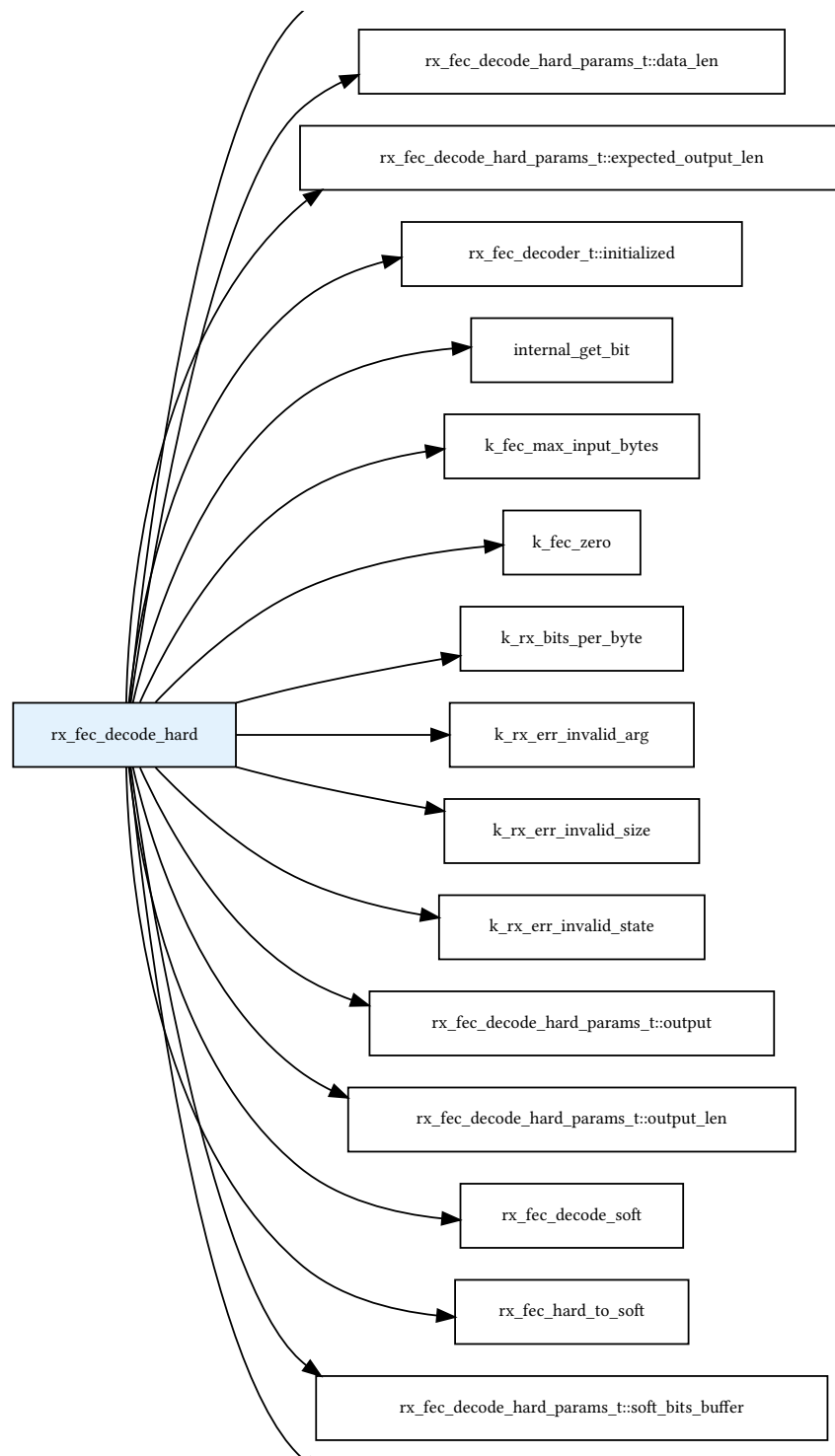
Note: Hard-decision decoding has 3 dB lower coding gain than soft-decision] **Example:**

```
//Example:hard-decisiondecodeofpackedFEC-encodedbytes
rx_fec_decoder_tdec;
uint64_tsurvivors[k_fec_max_symbols];
rx_soft_bit_tsoft_scratch[k_fec_max_symbols*k_fec_num_outputs];
uint8_tout_buf[k_fec_max_input_bytes];
uint32_tout_len=k_fec_zero;

(void)rx_fec_decoder_init(&dec,survivors,k_fec_max_symbols);

rx_fec_decode_hard_params_tp={
.data=received_bytes,
.data_len=received_len,
.soft_bits_buffer=soft_scratch,
.soft_buffer_len=k_fec_max_symbols*k_fec_num_outputs,
.expected_output_len=original_data_len,
.output=out_buf,
.output_len=&out_len,
};
rx_err_terr=rx_fec_decode_hard(&dec,&p);
```

See also: rx_fec_decode_soft() Preferred: use when soft values are available,
rx_fec_decoder_init() Initialize decoder before calling this function,
rx_fec_hard_to_soft() Conversion function used internally, rx_fec_encode() Encoder
that produces data this function decodes

Figure 421: Call graph for `rx_fec_decode_hard`

Defined in `libs/rx_fec/inc/rx_fec.h:1100`

Since Version 1.0.0

—

rx_fec_hard_to_soft

```
static rx_soft_bit_t rx_fec_hard_to_soft(uint8_t bit)
```

Convert hard bit to soft bit.

Parameters:

Direction	Name	Description
in	bit	Hard bit value (0 or 1)

Returns: Soft bit: k_soft_bit_min for 0, k_soft_bit_max for 1

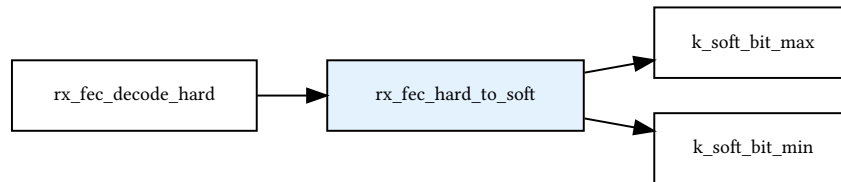


Figure 422: Call graph for `rx_fec_hard_to_soft`

Defined in `libs/rx_fec/inc/rx_fec.h:1125`

—

rx_fec_soft_to_hard

```
static uint8_t rx_fec_soft_to_hard(rx_soft_bit_t soft)
```

Convert soft bit to hard bit.

Parameters:

Direction	Name	Description
in	soft	Soft bit value

Returns: Hard bit: 1 if soft ≥ 0 , 0 otherwise

Defined in `libs/rx_fec/inc/rx_fec.h:1136`

—

rx_fec.c

Forward Error Correction (FEC) Codec Implementation.

Implements NASA-standard K=7, rate 1/2 convolutional encoder and soft-decision Viterbi decoder.

This is a C port of star-gateway/internal/fec/ ensuring bit-exact compatibility between RX72N firmware and RPi5 gateway.

Includes:

- "rx_fec.h"
- <string.h>
- "rx_bit_constants.h"
- "rx_check.h"

rx_fec_impl_constants_t

FEC implementation constants.

Value	Name	Description
= 0	k_fec_zero	Zero constant
= 7	k_fec_msb_bit_position	MSB position in byte (0-indexed)
= 6	k_fec_shift_register_bits	K-1 shift register size
= 32768	k_fec_correlation_offset	Correlation metric offset (2^{15})
= 5	k_fec_state_shift_amount	k_fec_constraint_length - 2
= 1	k_fec_bit_mask	Mask for extracting single bit

—

rx_fec_output_index_t

FEC output indices for G1 and G2 generator polynomials.

Value	Name	Description
= 0	k_fec_output_g1	G1 generator output (first encoded bit)
= 1	k_fec_output_g2	G2 generator output (second encoded bit)

—

internal_parity

```
static uint8_t internal_parity(uint8_t x)
```

Calculate parity (XOR of all bits) of a byte.

Uses parallel XOR reduction to compute parity in $O(\log n)$ operations. This is more efficient than looping through each bit.

Algorithm:

- Step 1 ($x \wedge= x \gg 4$): XOR upper nibble with lower nibble Original: [b7 b6 b5 b4][b3 b2 b1 b0]
Result: [x x x x][b7^b3 b6^b2 b5^b1 b4^b0]
- Step 2 ($x \wedge= x \gg 2$): XOR pairs of bits Result bits [1:0] now contain XOR of all 8 original bits
- Step 3 ($x \wedge= x \gg 1$): Final XOR to get single parity bit Result bit [0] = XOR of all original bits

Parameters:

Direction	Name	Description
in	x	Input byte

Returns: 0 if even parity (even number of 1s), 1 if odd parity

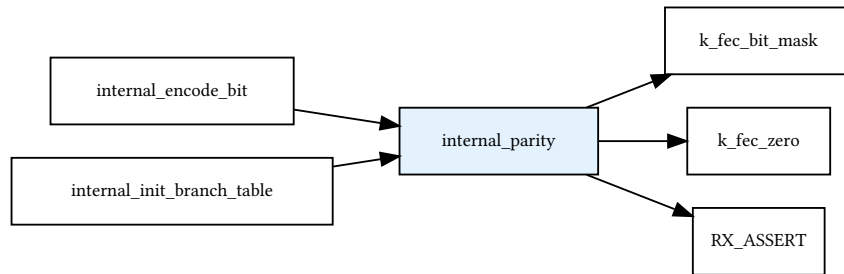


Figure 423: Call graph for internal_parity

Defined in `libs/rx_fec/src/rx_fec.c:151`

internal_set_output_bit

```
static void internal_set_output_bit(uint8_t *output, const uint32_t bit_idx,
uint8_t value)
```

Set a bit at specified index in output buffer (MSB first).

Bit ordering (MSB first, network byte order): bit_idx=0 -> byte 0, bit 7 (MSB) bit_idx=7 -> byte 0, bit 0 (LSB) bit_idx=8 -> byte 1, bit 7 (MSB)

Parameters:

Direction	Name	Description
out	output	Output buffer
in	bit_idx	Bit index (0 = MSB of first byte)
in	value	Bit value (0 or 1)

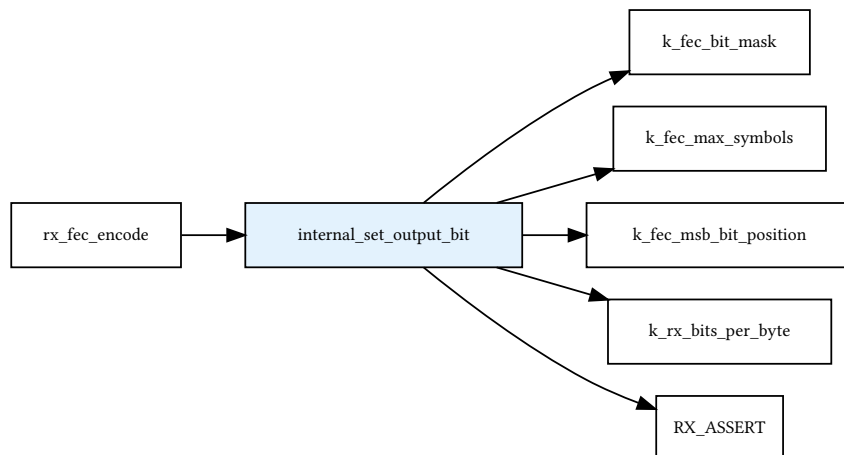


Figure 424: Call graph for internal_set_output_bit

Defined in `libs/rx_fec/src/rx_fec.c:179`

internal_get_bit

```
static uint8_t internal_get_bit(const uint8_t *data, uint32_t bit_idx)
```

Get a bit at specified index from buffer (MSB first).

Bit ordering (MSB first, network byte order): bit_idx=0 -> byte 0, bit 7 (MSB) bit_idx=7 -> byte 0, bit 0 (LSB) bit_idx=8 -> byte 1, bit 7 (MSB)

Parameters:

Direction	Name	Description
in	data	Input buffer
in	bit_idx	Bit index (0 = MSB of first byte)

Returns: Bit value (0 or 1)

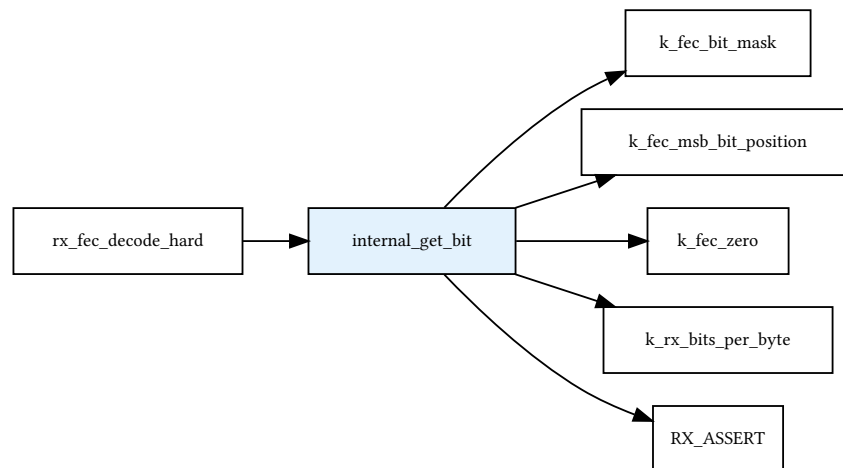


Figure 425: Call graph for internal_get_bit

Defined in `libs/rx_fec/src/rx_fec.c:208`

internal_encode_bit

```
static void internal_encode_bit(uint8_t *state, const uint8_t input_bit, uint8_t *out0, uint8_t *out1)
```

Encode a single bit and update encoder state.

Parameters:

Direction	Name	Description
-----------	------	-------------

inout	state	Current encoder state (6-bit shift register). Modified: shifts right and incorporates input_bit.
in	input_bit	Input bit (0 or 1)
out	out0	First output bit (G1)
out	out1	Second output bit (G2)

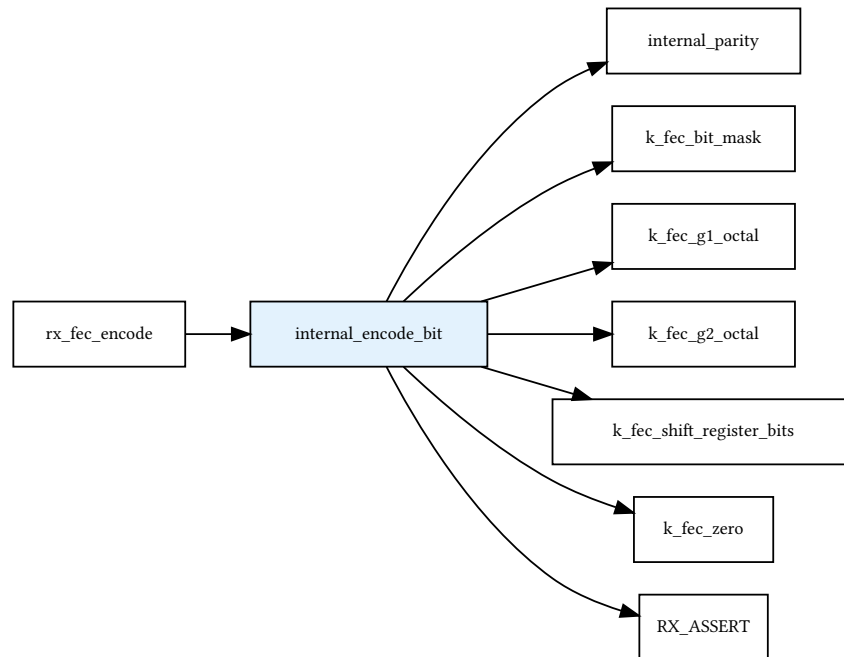


Figure 426: Call graph for internal_encode_bit

Defined in `libs/rx_fec/src/rx_fec.c:234`

internal_init_branch_table

```
static void internal_init_branch_table(uint8_t branch_table[k_fec_num_states]
[k_fec_num_input_values][k_fec_num_outputs])
```

Initialize the branch table for Viterbi decoder.

Precomputes expected output bits for each state and input combination.

Parameters:

Direction	Name	Description
out	branch_table	Table to populate [state][input][output]

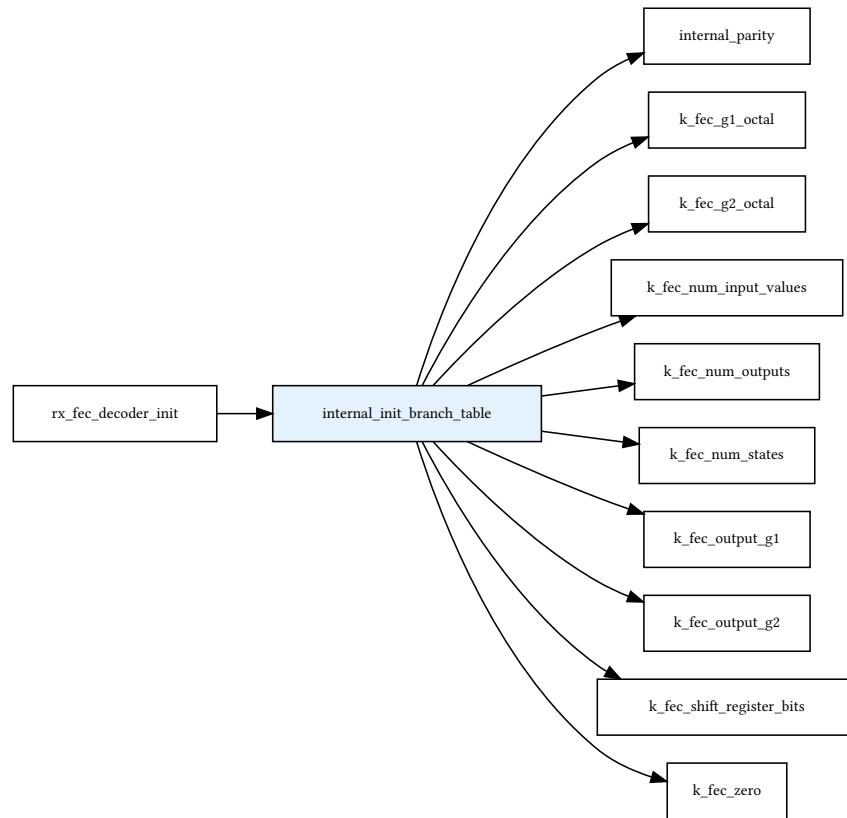


Figure 427: Call graph for internal_init_branch_table

Defined in `libs/rx_fec/src/rx_fec.c:262`

—

internal_branch_metric

```
static int32_t internal_branch_metric(const rx_soft_bit_t soft0, const
rx_soft_bit_t soft1, const uint8_t exp0, const uint8_t exp1)
```

Compute branch metric for soft decoding.

Uses correlation metric: higher correlation = lower metric (better). Formula: $32768 - (\text{soft0} * \text{exp0_soft} + \text{soft1} * \text{exp1_soft})$

Parameters:

Direction	Name	Description
in	soft0	First received soft bit
in	soft1	Second received soft bit
in	exp0	Expected first bit (0 or 1)

in	exp1	Expected second bit (0 or 1)
----	------	------------------------------

Returns: Branch metric (lower is better)

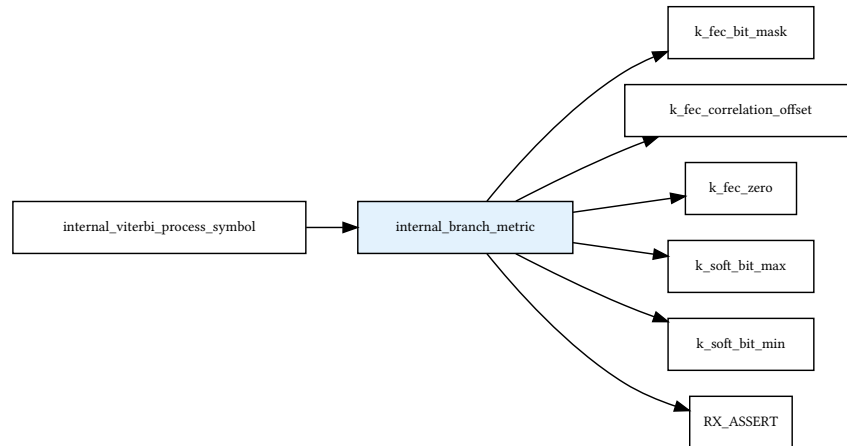


Figure 428: Call graph for `internal_branch_metric`

Defined in `libs/rx_fec/src/rx_fec.c:290`

`internal_viterbi_process_symbol`

```
static void internal_viterbi_process_symbol(rx_fec_decoder_t *dec, const
rx_soft_bit_t soft0, const rx_soft_bit_t soft1, const uint32_t symbol_idx)
```

Process one symbol pair through the Viterbi trellis.

Updates path metrics and survivors for one time step.

Parameters:

Direction	Name	Description
inout	dec	Decoder handle. Modified: path_metrics and survivors updated.
in	soft0	First soft bit of symbol pair
in	soft1	Second soft bit of symbol pair
in	symbol_idx	Time step index

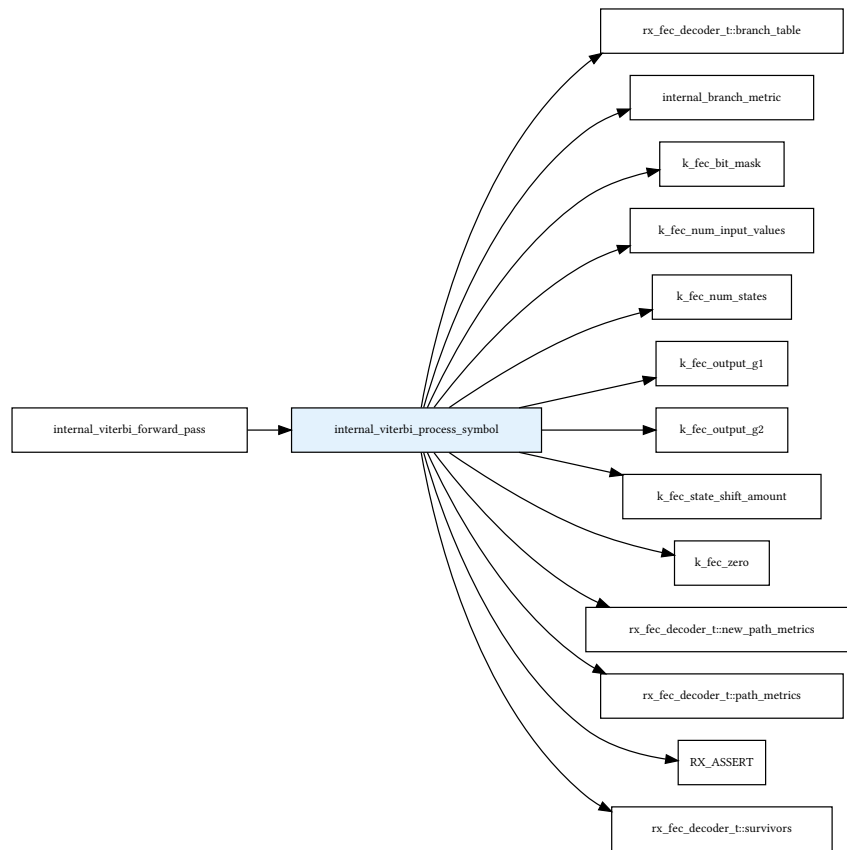


Figure 429: Call graph for internal_viterbi_process_symbol

Defined in `libs/rx_fec/src/rx_fec.c:320`

internal_viterbi_forward_pass

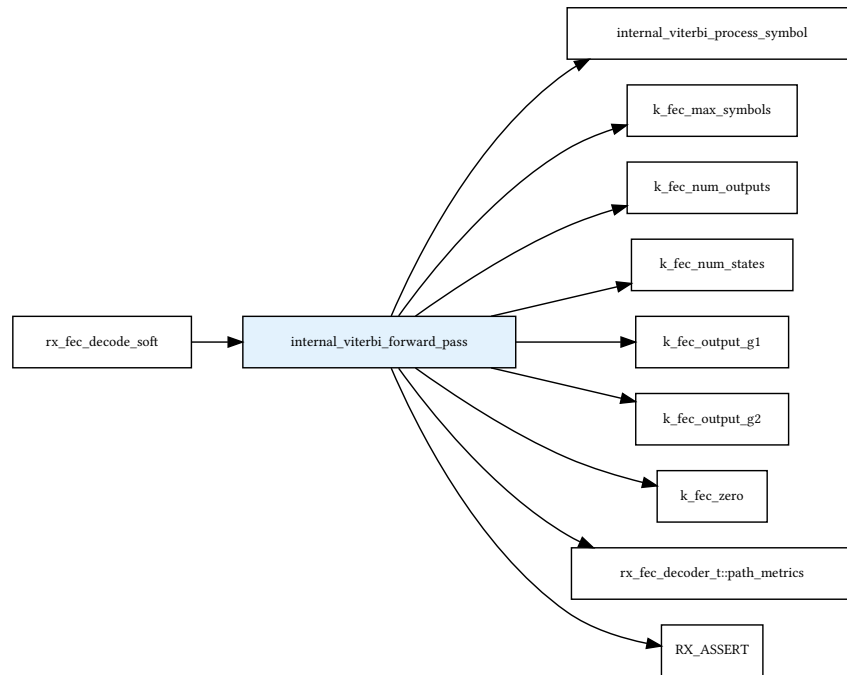
```
static void internal_viterbi_forward_pass(rx_fec_decoder_t *dec, const
rx_soft_bit_t *soft_bits, const uint32_t num_symbols)
```

Run Viterbi forward pass through trellis.

Initializes path metrics and processes all symbols through the trellis.

Parameters:

Direction	Name	Description
inout	dec	Decoder handle. Modified: path_metrics updated.
in	soft_bits	Received soft bits (pairs)
in	num_symbols	Number of symbols to process

Figure 430: Call graph for `internal_viterbi_forward_pass`

Defined in `libs/rx_fec/src/rx_fec.c:388`

`internal_viterbi_traceback`

```
static void internal_viterbi_traceback(const rx_fec_decoder_t *dec, const
uint32_t num_symbols, const uint32_t data_bits, uint8_t *output, const uint32_t
output_bytes)
```

Perform Viterbi traceback to extract decoded bits.

Works backwards through the trellis from the terminal state to recover the maximum likelihood input sequence.

Parameters:

Direction	Name	Description
in	<code>dec</code>	Decoder handle
in	<code>num_symbols</code>	Total number of symbols processed
in	<code>data_bits</code>	Number of data bits (excluding tail)
out	<code>output</code>	Decoded output buffer
out	<code>output_bytes</code>	Number of output bytes

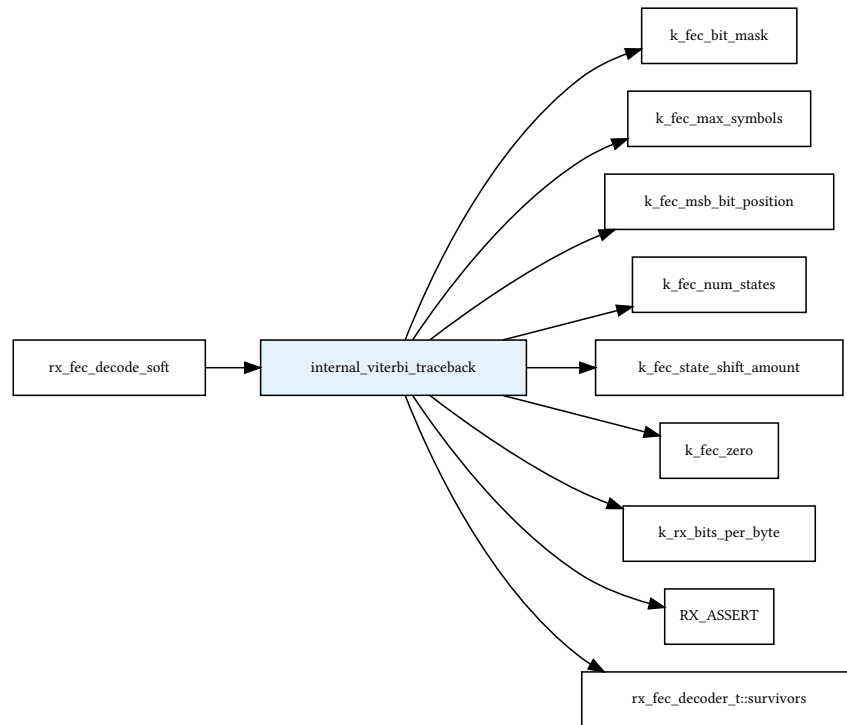


Figure 431: Call graph for internal_viterbi_traceback

Defined in `libs/rx_fec/src/rx_fec.c:423`

—

rx_fec_encoder_init

```
rx_err_t rx_fec_encoder_init(rx_fec_encoder_t *enc)
```

Initialize FEC encoder.

Parameters:

Direction	Name	Description
in	enc	Pointer to encoder structure

Returns: `k_rx_ok` on success, `k_rx_err_invalid_arg` if `enc` is `nullptr`

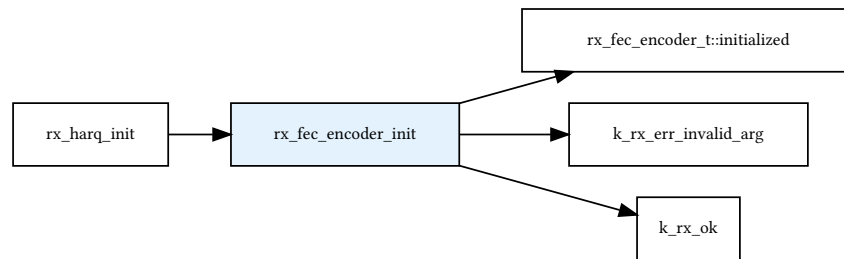


Figure 432: Call graph for rx_fec_encoder_init

Defined in `libs/rx_fec/src/rx_fec.c:481`

rx_fec_encoder_deinit

```
rx_err_t rx_fec_encoder_deinit(rx_fec_encoder_t *enc)
```

Deinitialize FEC encoder.

Parameters:

Direction	Name	Description
in	enc	Pointer to encoder structure

Returns: k_rx_ok on success, k_rx_err_invalid_arg if enc is nullptr

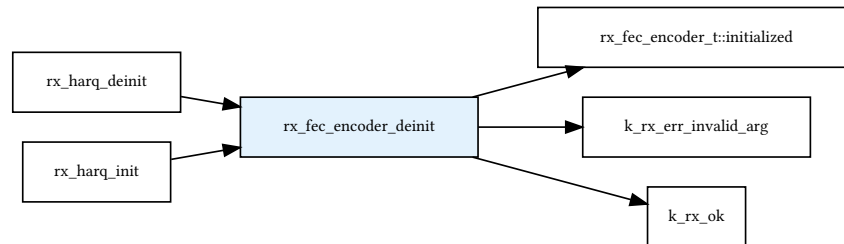


Figure 433: Call graph for rx_fec_encoder_deinit

Defined in `libs/rx_fec/src/rx_fec.c:498`

rx_fec_encoded_len

```
uint32_t rx_fec_encoded_len(const uint32_t input_len)
```

Calculate encoded output length for given input.

Calculate FEC encoded output length.

Computes the number of bytes required for FEC-encoded output based on input data length. Accounts for tail bits and rate-1/2 encoding.

Parameters:

Direction	Name	Description
in	input_len	Length of input data in bytes

Returns: Encoded output length in bytes, or 0 if input length invalid

Pre: input_len must be in range 1, k_fec_max_input_bytes

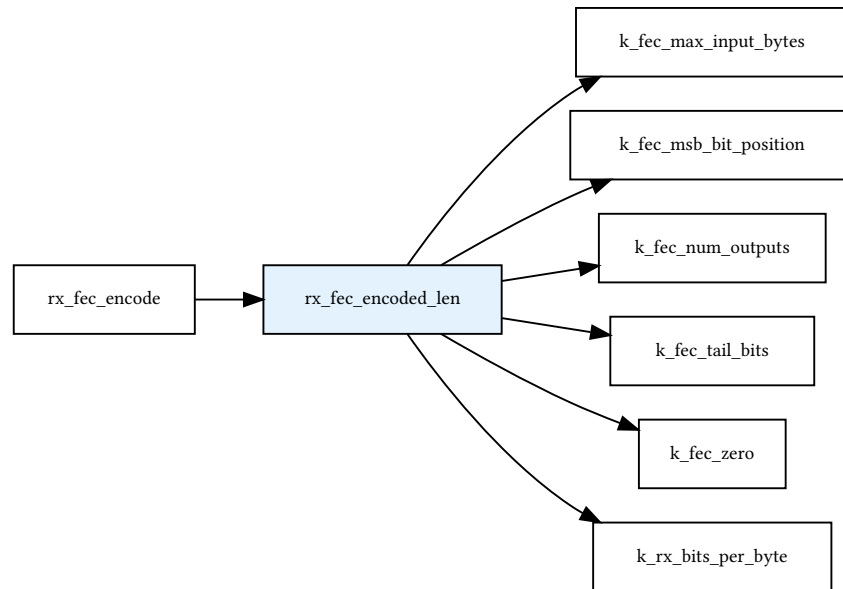


Figure 434: Call graph for rx_fec_encoded_len

Defined in `libs/rx_fec/src/rx_fec.c:520`

rx_fec_encode

```
rx_err_t rx_fec_encode(const rx_fec_encoder_t *enc, const uint8_t *input, const
uint32_t input_len, uint8_t *output, uint32_t *output_len)
```

Encode data using convolutional FEC.

Encode data using K=7, rate 1/2 convolutional code.

Applies rate-1/2, constraint-length-7 convolutional encoding to input data. Output length is approximately 2x input length.

Parameters:

Direction	Name	Description
in	enc	Pointer to initialized encoder
in	input	Pointer to input data buffer

in	input_len	Length of input data (1 to k_fec_max_input_bytes)
out	output	Pointer to output buffer (must be $\geq$ rx_fec_encoded_len(input_len))
out	output_len	Pointer to store actual output length

Returns: k_rx_err_invalid_size if input_len exceeds maximum

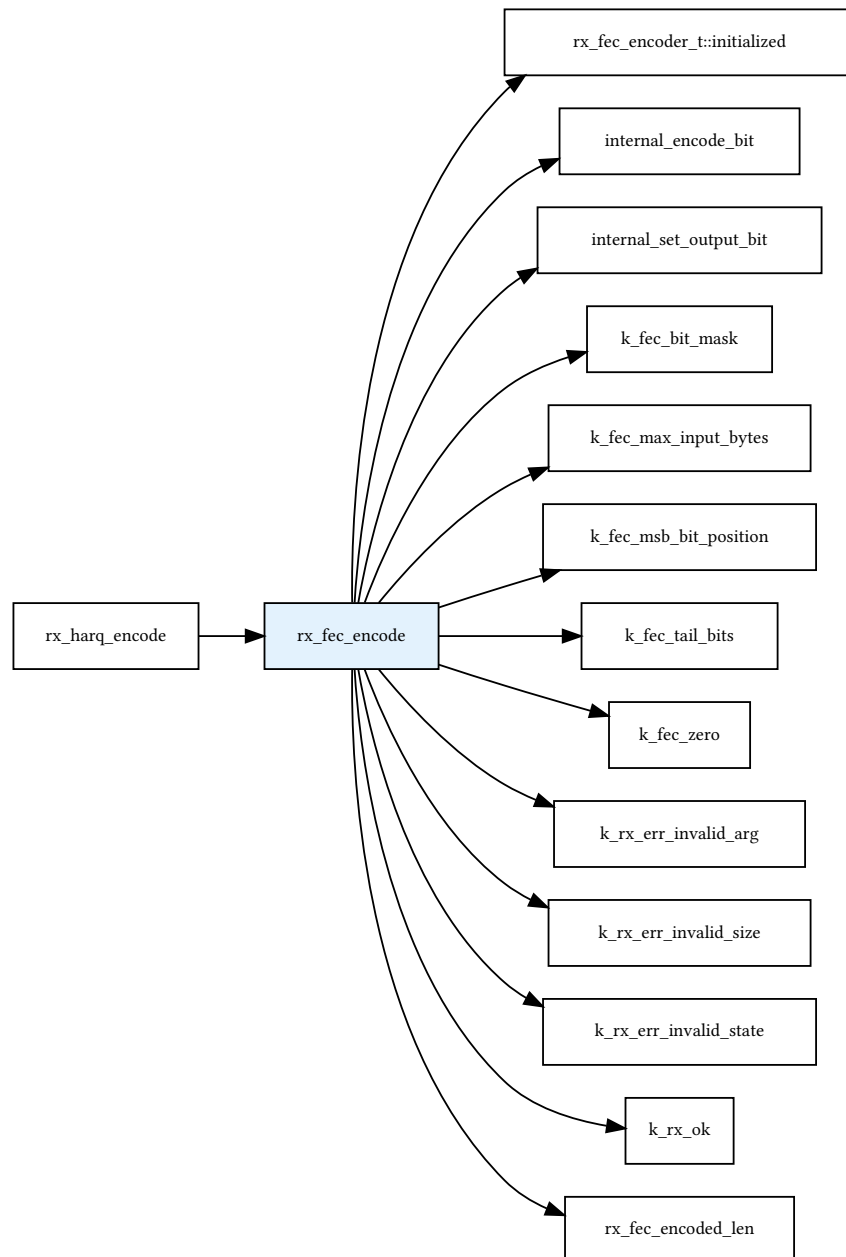
Pre: enc must be initialized via rx_fec_encoder_init()

Pre: All pointers must be non-nullptr

Pre: input_len in range 1, k_fec_max_input_bytes

Post: output buffer contains FEC-encoded data

Post: *output_len set to actual encoded length

Figure 435: Call graph for `rx_fec_encode`

Defined in `libs/rx_fec/src/rx_fec.c:563`

—

internal_validate_decode_params

```
static rx_err_t internal_validate_decode_params(rx_fec_decoder_t *dec, const
rx_fec_decode_soft_params_t *params, uint32_t *num_symbols_out)
```

Validate decode parameters before Viterbi decoding.

Pre-conditions:

- dec, params, num_symbols_out must be non-nullptr
- dec must be initialized
- params->soft_bits, output, output_len must be non-nullptr
- params->soft_len must be non-zero and divisible by 2 (G1, G2 pairs)

Validation checks:

- Calculated num_symbols from expected_output_len must be valid and within bounds
- Sufficient soft bits must be available for calculated num_symbols (NOT silently reduced - caller error if params are inconsistent)
- Survivors buffer must be large enough for num_symbols

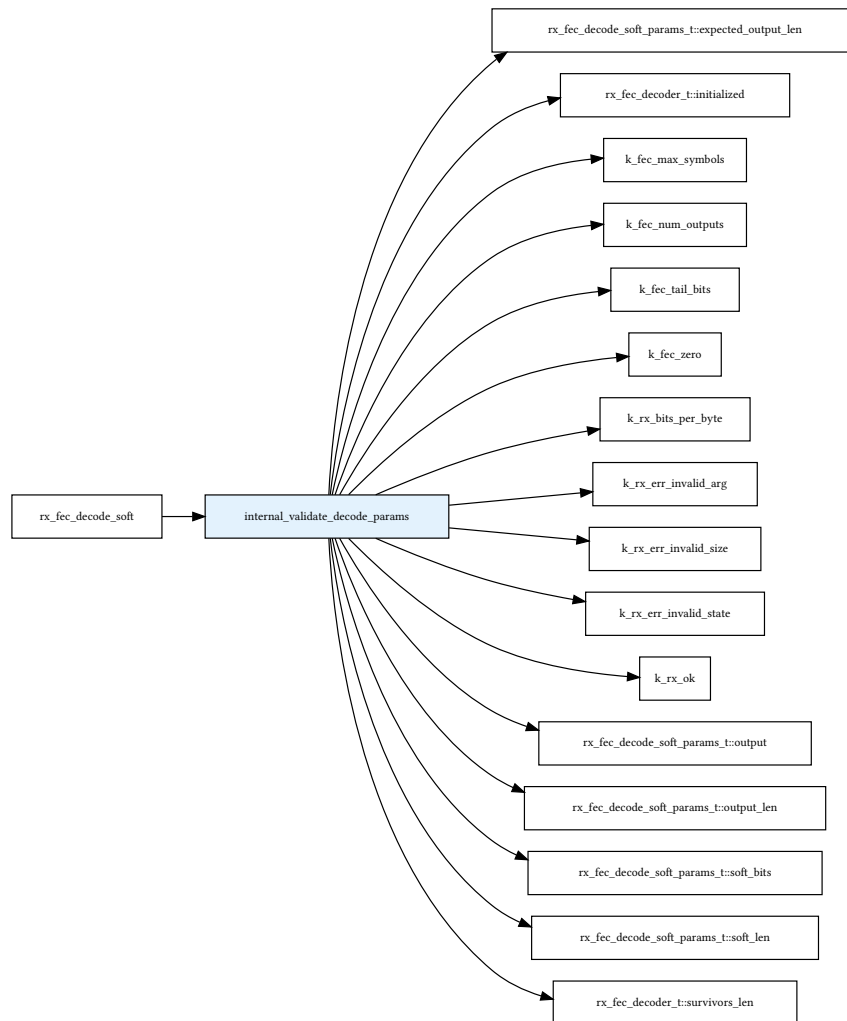
Post-condition:

- If successful, num_symbols_out contains the number of symbols to decode
- If parameters are invalid or inconsistent, returns appropriate error code

Parameters:

Direction	Name	Description
in	dec	Pointer to FEC decoder (must be initialized)
in	params	Soft-bit decode parameters
out	num_symbols_out	Calculated number of symbols to decode

Returns: k_rx_err_invalid_size if calculated symbols out of range or insufficient soft bits

Figure 436: Call graph for `internal_validate_decode_params`

Defined in `libs/rx_fec/src/rx_fec.c:661`

`rx_fec_decoder_init`

```
rx_err_t rx_fec_decoder_init(rx_fec_decoder_t *dec, uint64_t *survivors_buf,
    const uint32_t survivors_len)
```

Initialize FEC decoder.

Initializes decoder state and associates it with provided survivors buffer. The buffer is used for Viterbi algorithm path storage.

Parameters:

Direction	Name	Description
inout	dec	Pointer to decoder structure
in	survivors_buf	Pointer to survivors buffer for Viterbi algorithm
in	survivors_len	Length of survivors buffer in uint64_t entries

Returns: k_rx_err_invalid_size if survivors_len is 0

Pre: dec and survivors_buf must be non-nullptr

Pre: survivors_len must be > 0 (minimum 1 entry per symbol)

Post: dec->initialized set to true

Post: Branch table initialized

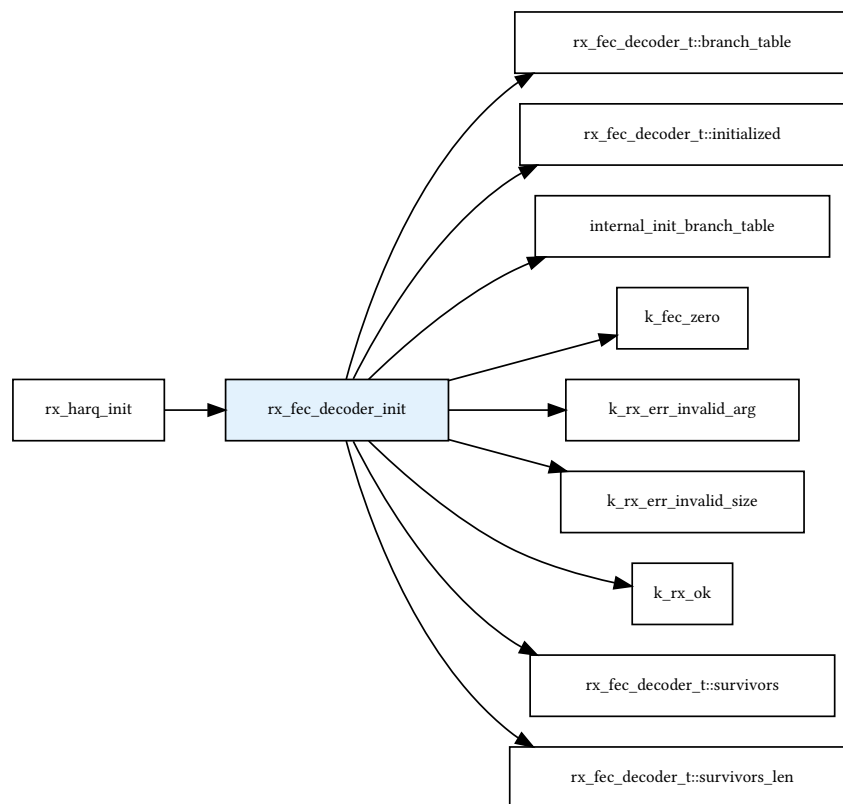


Figure 437: Call graph for rx_fec_decoder_init

Defined in `libs/rx_fec/src/rx_fec.c:734`

rx_fec_decoder_deinit

```
rx_err_t rx_fec_decoder_deinit(rx_fec_decoder_t *dec)
```

Deinitialize FEC decoder.

Parameters:

Direction	Name	Description
in	dec	Pointer to decoder structure

Returns: k_rx_ok on success, k_rx_err_invalid_arg if dec is nullptr

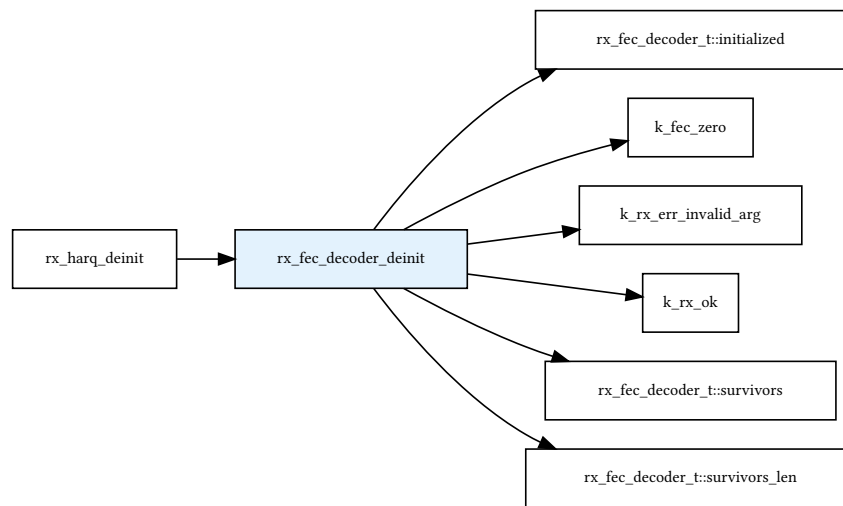


Figure 438: Call graph for rx_fec_decoder_deinit

Defined in `libs/rx_fec/src/rx_fec.c:762`

—

rx_fec_decode_soft

```
rx_err_t rx_fec_decode_soft(rx_fec_decoder_t *dec, const
rx_fec_decode_soft_params_t *params)
```

Decode FEC-encoded data using soft-decision Viterbi algorithm.

Decode soft bits using Viterbi algorithm.

Implements Rate-1/2 Viterbi soft-decision decoding with constraint length 7. Accepts soft-decision values (signed integers in range $[-128, 127]$) to achieve approximately 3 dB coding gain over hard-decision decoding.

Algorithm steps:

1. Validate parameters via `internal_validate_decode_params()`
2. Forward pass: compute branch and path metrics for all symbols
3. Calculate data bit count (`num_symbols - tail bits`)

4. Traceback: extract maximum-likelihood decoded bit sequence
5. Pack decoded bits into output byte array

Parameters:

Direction	Name	Description
in	dec	Pointer to initialized FEC decoder (must be initialized via rx_fec_decoder_init()); survivors buffer must be sized for input)
in	params	Decode parameters: soft_bits: signed soft values, G1/G2 interleaved pairs soft_len: must be non-zero and divisible by 2 expected_output_len: expected decoded byte count (> 0) output: output buffer (must be >= expected_output_len bytes) output_len: written with actual decoded byte count on success

Returns: rx_err_t Status code

Value	Description
k_rx_ok	Decoding succeeded; output_len updated with byte count
k_rx_err_invalid_arg	dec or params (or required fields) are nullptr; or soft_len is zero or not divisible by 2
k_rx_err_invalid_state	dec is not initialized
k_rx_err_invalid_size	expected_output_len out of range; or derived num_symbols falls outside [k_fec_tail_bits, k_fec_max_symbols]; or data_bits == 0 after subtracting tail bits

Pre: dec must be initialized via rx_fec_decoder_init()

Pre: params->soft_len must be divisible by 2 (G1/G2 symbol pairs)

Pre: params->expected_output_len must be > 0 and <= k_fec_max_input_bytes

Post: *params->output_len written with actual decoded byte count on k_rx_ok

Post: params->output contains decoded data bytes on k_rx_ok

Post: dec path metric arrays updated (internal state consumed by traceback)

Note: Not thread-safe; caller must ensure exclusive access to dec and params

Note: Soft values: positive = bit-1 likely, negative = bit-0 likely (signed).] **Example:**

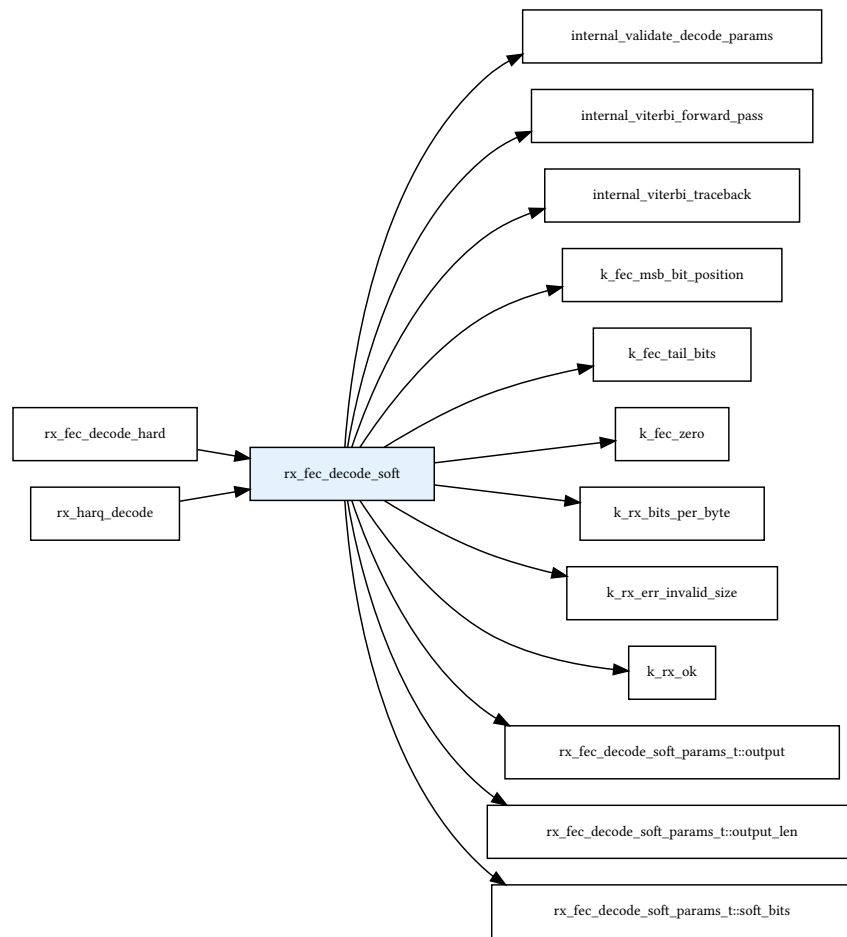
```
//Example: soft-decision decode of a previously FEC-encoded buffer
rx_fec_decoder_t dec;
uint64_t survivors[k_fec_max_symbols];
```

```
rx_soft_bit_tsoft_buf[k_fec_max_symbols*k_fec_num_outputs];
uint8_tout_buf[k_fec_max_input_bytes];
uint32_tout_len=k_fec_zero;

(void)rx_fec_decoder_init(&dec,survivors,k_fec_max_symbols);

//Populatessoft_buffromchannel(positive=bit-1likely)
rx_fec_decode_soft_params_tp={
    .soft_bits=soft_buf,
    .soft_len=encoded_len_bits,
    .expected_output_len=original_data_len,
    .output=out_buf,
    .output_len=&out_len,
};
rx_err_terr=rx_fec_decode_soft(&dec,&p);
```

See also: rx_fec_decoder_init() Initialize decoder before calling this function,
rx_fec_decode_hard() Hard-decision wrapper (lower coding gain, simpler input),
rx_fec_encode() Encoder that produces the data this function decodes

Figure 439: Call graph for `rx_fec_decode_soft`

Defined in `libs/rx_fec/src/rx_fec.c:846`

Since Version 1.0.0

—

rx_fec_decode_hard

```
rx_err_t rx_fec_decode_hard(rx_fec_decoder_t *dec, const
rx_fec_decode_hard_params_t *params)
```

Decode FEC-encoded data using hard-decision bits (converts to soft internally).

Decode hard bits using Viterbi algorithm.

Convenience wrapper that converts hard-decision bits (0/1) to soft-decision values and delegates to `rx_fec_decode_soft()`. Accepts raw bit-packed data (MSB-first) as produced by standard digital logic or RSPI transfers.

Algorithm steps:

1. Validate all input pointers and size constraints
2. Convert each hard bit to a soft value via `rx_fec_hard_to_soft()` (maps 0 -> negative confidence, 1 -> positive confidence)
3. Build `rx_fec_decode_soft_params_t` from the converted soft bits
4. Delegate to `rx_fec_decode_soft()` for full Viterbi decode

Parameters:

Direction	Name	Description
in	dec	Pointer to initialized FEC decoder (must be initialized via <code>rx_fec_decoder_init()</code> ; survivors buffer sized for decoded output)
in	params	Decode parameters: data: bit-packed hard-decision input (MSB-first per byte) data_len: byte count of hard data; must be > 0 and <= <code>k_fec_max_input_bytes</code> soft_bits_buffer: caller-supplied scratch buffer for soft conversion; must be >= data_len*8 entries soft_buffer_len: element count of soft_bits_buffer expected_output_len: expected decoded byte count output: output buffer (must be >= expected_output_len bytes) output_len: written with actual decoded byte count on success

Returns: `rx_err_t` Status code

Value	Description
<code>k_rx_ok</code>	Decoding succeeded; output_len updated
<code>k_rx_err_invalid_arg</code>	dec or required params fields are nullptr; or data_len == 0
<code>k_rx_err_invalid_state</code>	dec is not initialized
<code>k_rx_err_invalid_size</code>	data_len > <code>k_fec_max_input_bytes</code> ; or soft_bits_buffer too small for converted bits; or errors propagated from <code>rx_fec_decode_soft()</code>

Pre: dec must be initialized via `rx_fec_decoder_init()`

Pre: params->soft_bits_buffer must be large enough: `soft_buffer_len >= data_len * 8`

Pre: params->data_len must be > 0 and <= `k_fec_max_input_bytes`

Post: *params->output_len written with actual decoded byte count on `k_rx_ok`

Post: params->output contains decoded data bytes on `k_rx_ok`

Post: params->soft_bits_buffer populated with converted soft values (side effect)

Note: Not thread-safe; caller must ensure exclusive access to dec and params

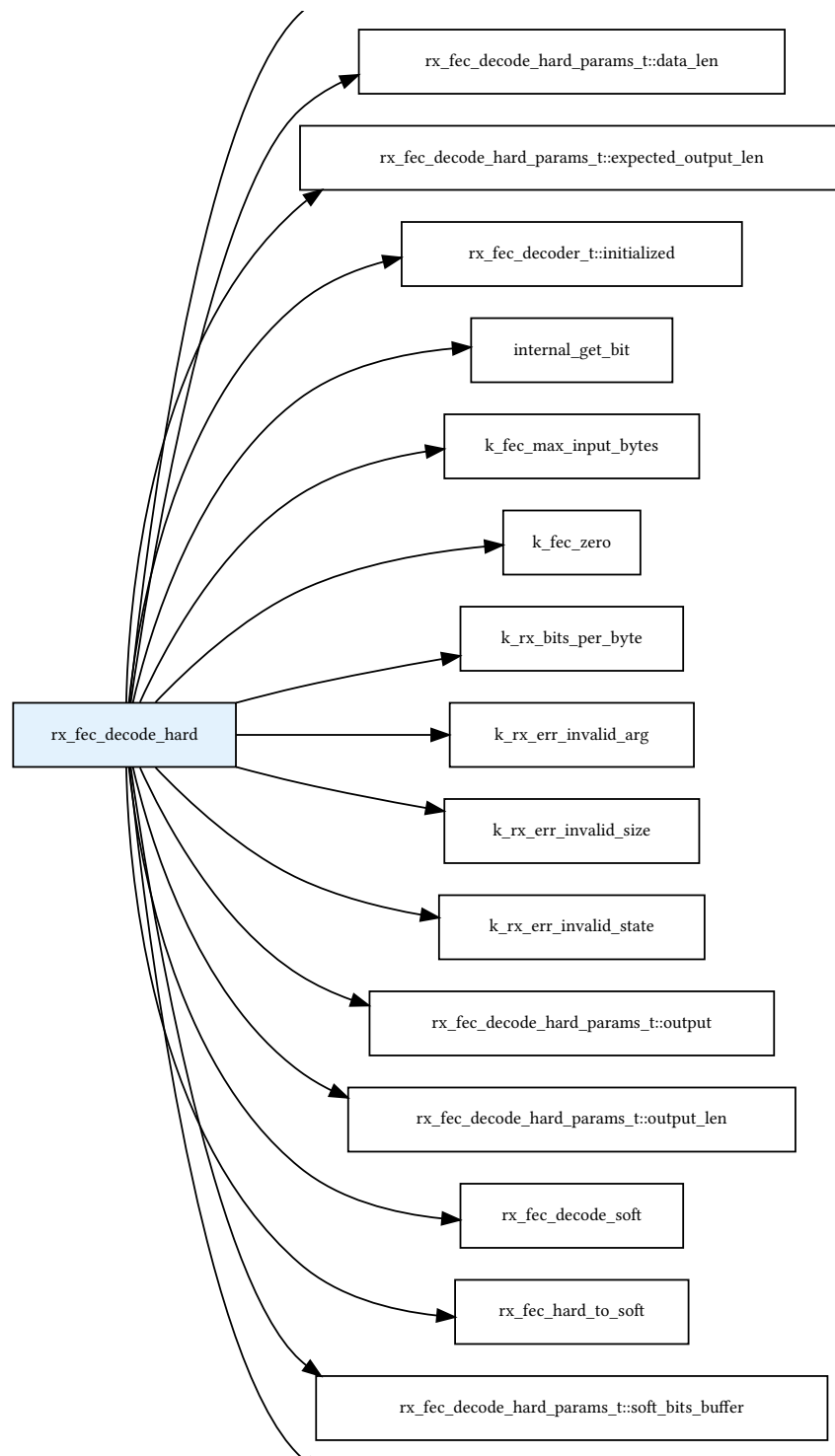
Note: Hard-decision decoding has 3 dB lower coding gain than soft-decision] **Example:**

```
//Example:hard-decisiondecodeofpackedFEC-encodedbytes
rx_fec_decoder_tdec;
uint64_tsurvivors[k_fec_max_symbols];
rx_soft_bit_tsoft_scratch[k_fec_max_symbols*k_fec_num_outputs];
uint8_tout_buf[k_fec_max_input_bytes];
uint32_tout_len=k_fec_zero;
```

```
(void)rx_fec_decoder_init(&dec,survivors,k_fec_max_symbols);
```

```
rx_fec_decode_hard_params_tp={
.data=received_bytes,
.data_len=received_len,
.soft_bits_buffer=soft_scratch,
.soft_buffer_len=k_fec_max_symbols*k_fec_num_outputs,
.expected_output_len=original_data_len,
.output=out_buf,
.output_len=&out_len,
};
rx_err_terr=rx_fec_decode_hard(&dec,&p);
```

See also: rx_fec_decode_soft() Preferred: use when soft values are available,
rx_fec_decoder_init() Initialize decoder before calling this function,
rx_fec_hard_to_soft() Conversion function used internally, rx_fec_encode() Encoder
that produces data this function decodes

Figure 440: Call graph for `rx_fec_decode_hard`

Defined in `libs/rx_fec/src/rx_fec.c:949`

Since Version 1.0.0

—

rx_frame.h

Frame Layer Protocol for SPI Communication.

Includes:

- `<stdbool.h>`
- `<stddef.h>`
- `<stdint.h>`
- `"rx_err.h"`

rx_frame_constants_t

Frame structure constants for SPI protocol.

These constants define the frame format parameters that must exactly match the Go implementation in `star-gateway/internal/frame/`. Any mismatch will cause protocol interoperability failures.

Value	Name	Description
= 0x55AA	<code>k_frame_sync_word</code>	Frame synchronization marker (0x55AA).
= 2	<code>k_frame_sync_size</code>	SYNC field size in bytes (2).
= 2	<code>k_frame_seq_size</code>	SEQ field size in bytes (2).
= 2	<code>k_frame_len_size</code>	LEN field size in bytes (2).
= 1	<code>k_frame_type_size</code>	TYPE field size in bytes (1).
= 1	<code>k_frame_flags_size</code>	FLAGS field size in bytes (1).
= 4	<code>k_frame_crc_size</code>	CRC-32 field size in bytes (4).
= 6	<code>k_frame_header_size</code>	Header size excluding SYNC (6 bytes).
= 0	<code>k_frame_min_payload</code>	Minimum payload length (0 bytes).
= 1024	<code>k_frame_max_payload</code>	Maximum payload length (1024 bytes).
= 12	<code>k_frame_min_size</code>	Minimum frame size (12 bytes).
= 1036	<code>k_frame_max_size</code>	Maximum frame size (1036 bytes).
= <code>k_frame_max_size</code>	<code>k_frame_max_scan_bytes</code>	Maximum bytes to scan forward during sync recovery (1036).

—

rx_frame_type_t

Frame types for SPI protocol (matches Go `FrameType`).

Defines the type of message contained in a frame. The frame type determines the expected payload content and required response handling.

Value	Name	Description
= 0x00	<code>k_frame_type_ping</code>	Ping request (0x00).

= 0x01	k_frame_type_pong	Pong response (0x01).
= 0x10	k_frame_type_command	Command from controller to peripheral (0x10).
= 0x11	k_frame_type_response	Response from peripheral to controller (0x11).
= 0x12	k_frame_type_ack	Positive acknowledgment (0x12, SPI only).
= 0x13	k_frame_type_nack	Negative acknowledgment (0x13, SPI only).
= 0xFE	k_frame_type_reset_ack	Reset acknowledgment (0xFE).
= 0xFF	k_frame_type_reset	Reset request (0xFF).

rx_frame_flags_t

Frame control flags (matches Go FrameFlags).

Bit flags that modify frame handling behavior. Multiple flags can be combined using bitwise OR.

Value	Name	Description
= 0x00	k_frame_flag_none	No flags set (0x00).
= 0x01	k_frame_flag_requires_ack	Frame requires acknowledgment (0x01, bit 0).
= 0x02	k_frame_flag_retransmit	Retransmission of previous frame (0x02, bit 1).
= 0x04	k_frame_flag_priority	High-priority frame (0x04, bit 2).
= 0x08	k_frame_flag_fec_enabled	Forward error correction enabled (0x08, bit 3).
= 0x10	k_frame_flag_soft_nack	NACK with soft error information (0x10, bit 4).

rx_le16_byte_idx_t

Byte indices for 16-bit little-endian serialization.

In little-endian format, the low byte (LSB) is stored at index 0 and the high byte (MSB) is stored at index 1.

Value	Name	Description
= 0	k_le16_byte_low	Low byte (LSB) at index 0
= 1	k_le16_byte_high	High byte (MSB) at index 1

rx_byte_order_constants_t

Byte manipulation constants for endianness conversions.

Value	Name	Description
= (8)	k_rx_le16_high_shift	Bit shift for high byte in 16-bit value
= (0xFFU)	k_rx_byte_mask	Mask to extract one byte

rx_le32_byte_idx_t

Byte indices for 32-bit little-endian serialization.

Value	Name	Description
= 0	k_le32_byte_0	Byte 0 (LSB)
= 1	k_le32_byte_1	Byte 1
= 2	k_le32_byte_2	Byte 2
= 3	k_le32_byte_3	Byte 3 (MSB)

—

rx_le32_shift_t

Bit shift amounts for extracting bytes from 32-bit values.

Value	Name	Description
= 0	k_rx_le32_shift_0	Shift 0 bits for byte 0
= 8	k_rx_le32_shift_1	Shift 8 bits for byte 1
= 16	k_rx_le32_shift_2	Shift 16 bits for byte 2
= 24	k_rx_le32_shift_3	Shift 24 bits for byte 3

—

rx_frame_encoder_init

```
rx_err_t rx_frame_encoder_init(rx_frame_encoder_t *enc)
```

Initialize frame encoder.

Initialize frame encoder.

Prepares the encoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
out	enc	Pointer to encoder handle
out	enc	Encoder instance to initialize

Returns: k_rx_ok on successful initialization

Value	Description
k_rx_err_invalid_arg	if enc is nullptr
k_rx_err_validation_failed	if initialization verification fails

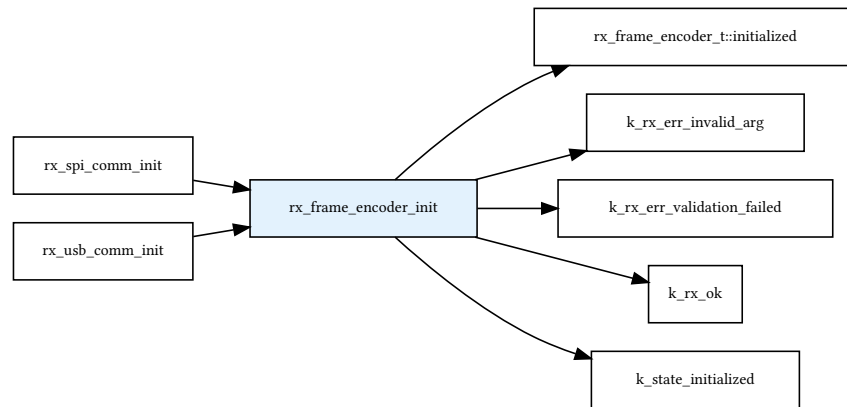


Figure 441: Call graph for rx_frame_encoder_init

Defined in `libs/rx_frame/inc/rx_frame.h:867`

rx_frame_encoder_deinit

```
rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t *enc)
```

Deinitialize frame encoder.

Deinitialize frame encoder.

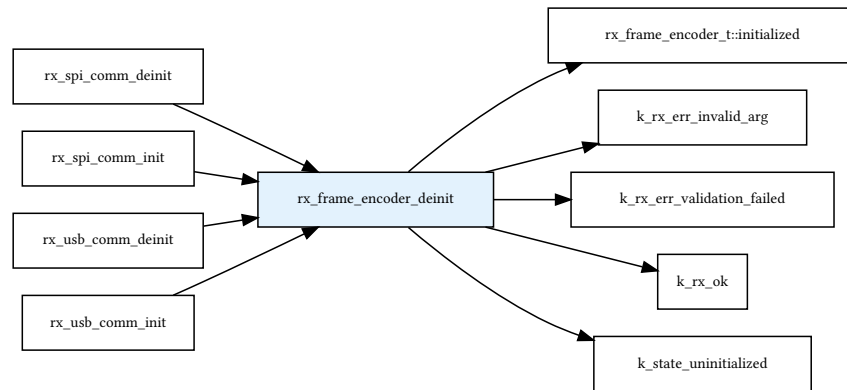
Marks the encoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
inout	enc	Pointer to encoder handle
out	enc	Encoder instance to deinitialize

Returns: k_rx_ok on successful deinitialization

Value	Description
k_rx_err_invalid_arg	if enc is nullptr
k_rx_err_validation_failed	if deinitialization verification fails

Figure 442: Call graph for `rx_frame_encoder_deinit`

Defined in `libs/rx_frame/inc/rx_frame.h:875`

`rx_frame_encode`

```
rx_err_t rx_frame_encode(const rx_frame_encoder_t *enc, const rx_frame_t *frame,
uint8_t *output, uint32_t *output_len)
```

Encode frame to wire format.

Serializes frame to wire format with SYNC, header, payload, and CRC-32. All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).

Parameters:

Direction	Name	Description
in	enc	Encoder handle
in	frame	Frame to encode
out	output	Output buffer (min <code>k_frame_min_size</code> + payload bytes)
out	output_len	Actual number of bytes written

Returns: `k_rx_err_invalid_size` if payload exceeds max

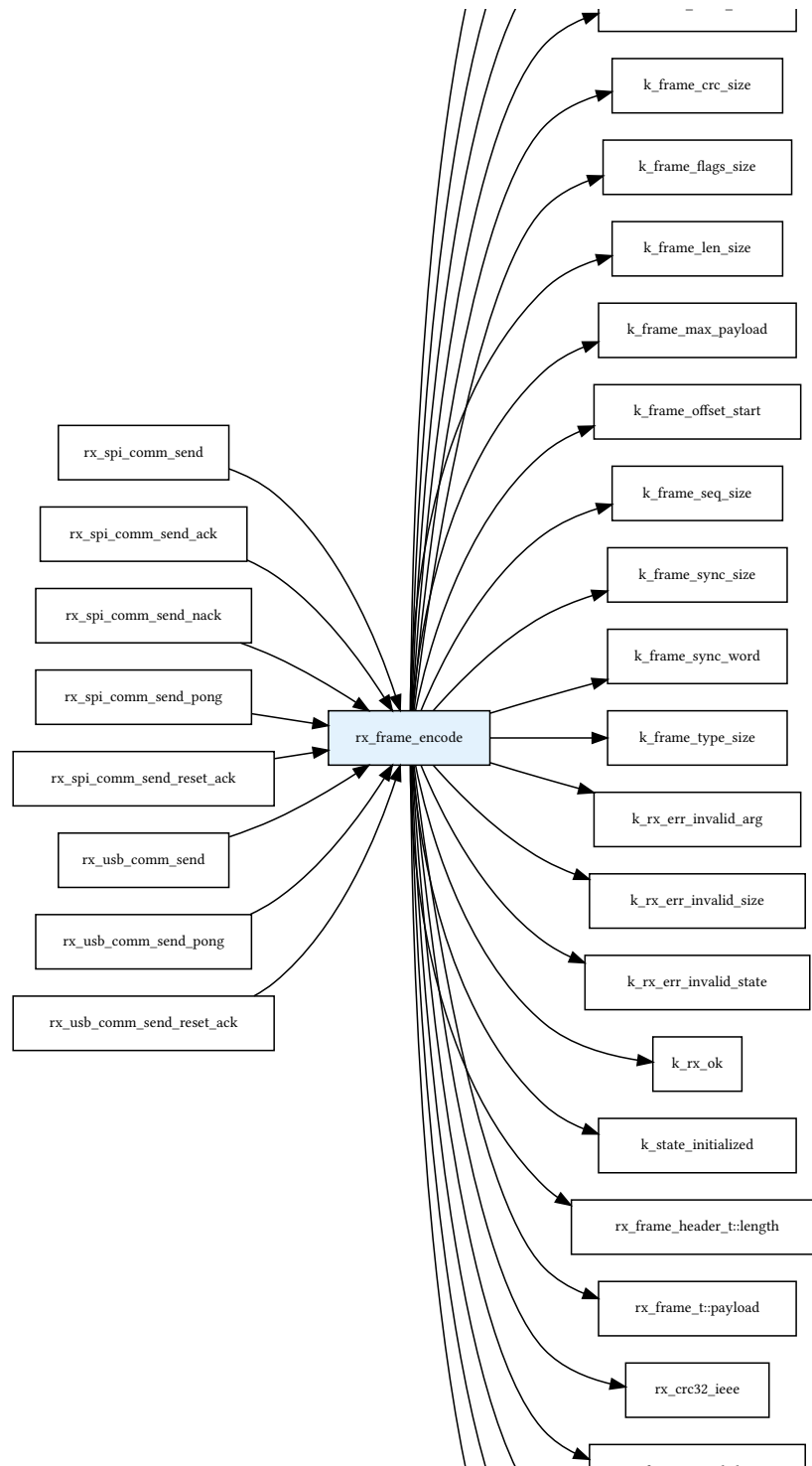


Figure 443: Call graph for `rx_frame_encode`

Defined in `libs/rx_frame/inc/rx_frame.h:893`

—

rx_frame_encoded_size

```
static uint32_t rx_frame_encoded_size(uint32_t payload_len)
```

Calculate encoded frame size.

Parameters:

Direction	Name	Description
in	payload_len	Payload length

Returns: Total frame size in bytes

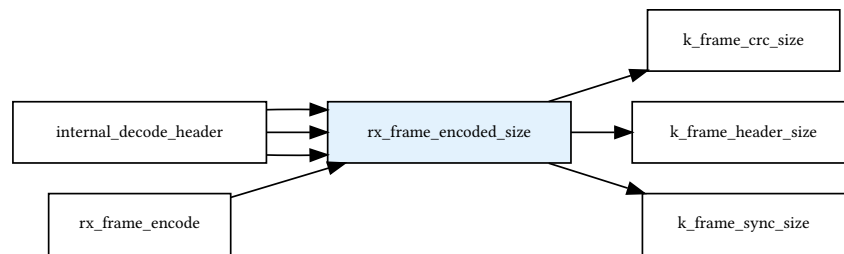


Figure 444: Call graph for `rx_frame_encoded_size`

Defined in `libs/rx_frame/inc/rx_frame.h:918`

—

rx_frame_read_le16

```
static uint16_t rx_frame_read_le16(const uint8_t *buf)
```

Read uint16 from little-endian buffer.

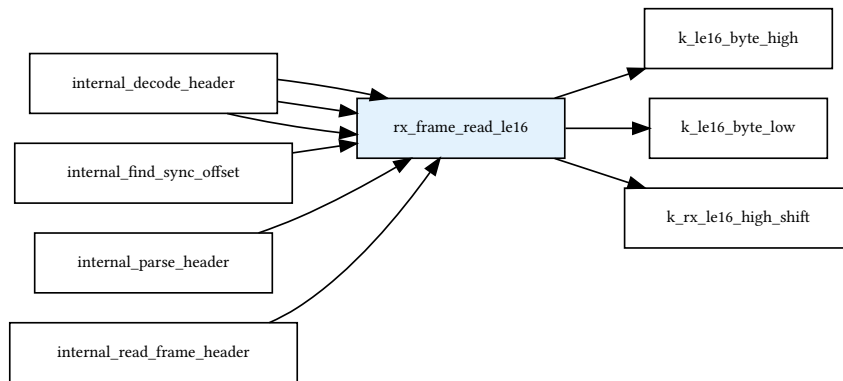
Reads a 16-bit unsigned integer from a buffer in little-endian format. The low byte (LSB) is at index 0, the high byte (MSB) is at index 1.

Parameters:

Direction	Name	Description
in	buf	Input buffer (at least 2 bytes)

Returns: Decoded 16-bit value in host byte order

Note: This is the preferred function for parsing multi-byte protocol fields.

Figure 445: Call graph for `rx_frame_read_le16`

Defined in `libs/rx_frame/inc/rx_frame.h:990`

`rx_frame_write_le16`

```
static void rx_frame_write_le16(uint8_t *buf, uint16_t val)
```

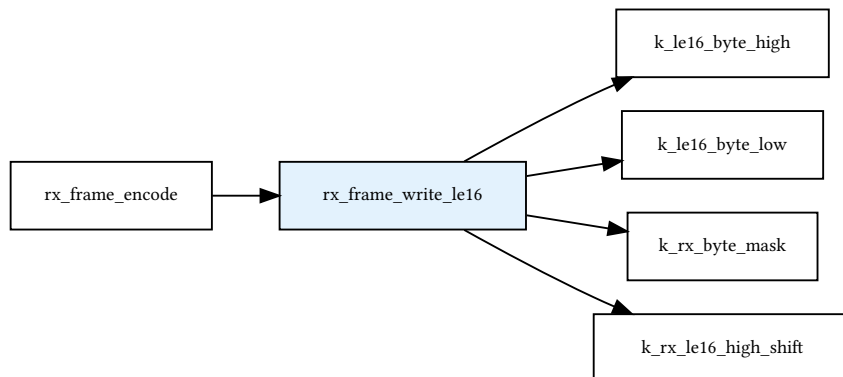
Write uint16 to little-endian buffer.

Writes a 16-bit unsigned integer to a buffer in little-endian format. The low byte (LSB) is written at index 0, the high byte (MSB) is written at index 1.

Parameters:

Direction	Name	Description
out	<code>buf</code>	Output buffer (at least 2 bytes)
in	<code>val</code>	Value to write in host byte order

Note: This is the preferred function for serializing multi-byte protocol fields.

Figure 446: Call graph for `rx_frame_write_le16`

Defined in `libs/rx_frame/inc/rx_frame.h:1008`

—

rx_frame_read_le32

```
static uint32_t rx_frame_read_le32(const uint8_t *buf)
```

Read uint32 from little-endian buffer.

Reads a 32-bit unsigned integer from a buffer in little-endian format.

Parameters:

Direction	Name	Description
in	buf	Input buffer (at least 4 bytes)

Returns: Decoded 32-bit value in host byte order

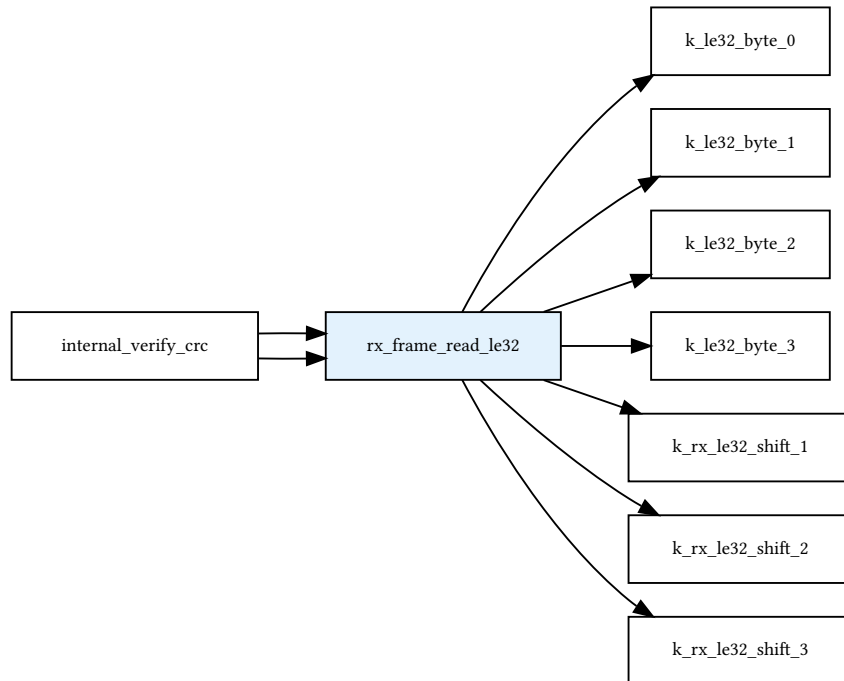


Figure 447: Call graph for `rx_frame_read_le32`

Defined in `libs/rx_frame/inc/rx_frame.h:1022`

—

rx_frame_decoder_init

```
rx_err_t rx_frame_decoder_init(rx_frame_decoder_t *dec)
```

Initialize frame decoder.

Initialize frame decoder.

Prepares the decoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
out	dec	Pointer to decoder handle
out	dec	Decoder instance to initialize

Returns: k_rx_ok on successful initialization

Value	Description
k_rx_err_invalid_arg	if dec is nullptr
k_rx_err_validation_failed	if initialization verification fails

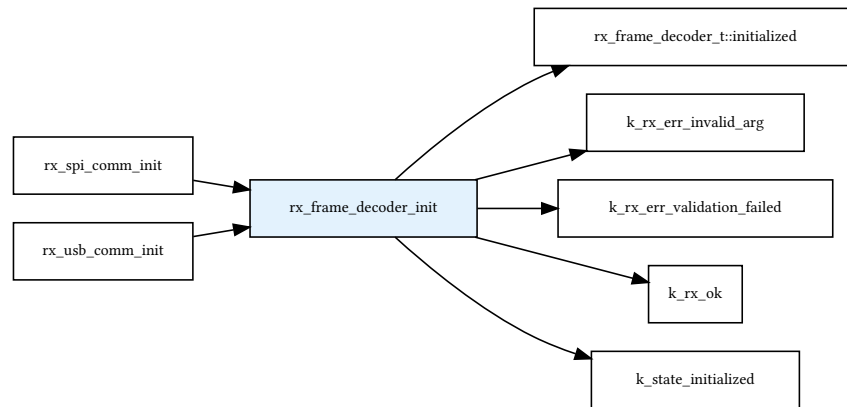


Figure 448: Call graph for `rx_frame_decoder_init`

Defined in `libs/rx_frame/inc/rx_frame.h:1040`

—

rx_frame_decoder_deinit

```
rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t *dec)
```

Deinitialize frame decoder.

Deinitialize frame decoder.

Marks the decoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
inout	dec	Pointer to decoder handle

out	dec	Decoder instance to deinitialize
-----	-----	----------------------------------

Returns: k_rx_ok on successful deinitialization

Value	Description
k_rx_err_invalid_arg	if dec is nullptr
k_rx_err_validation_failed	if deinitialization verification fails

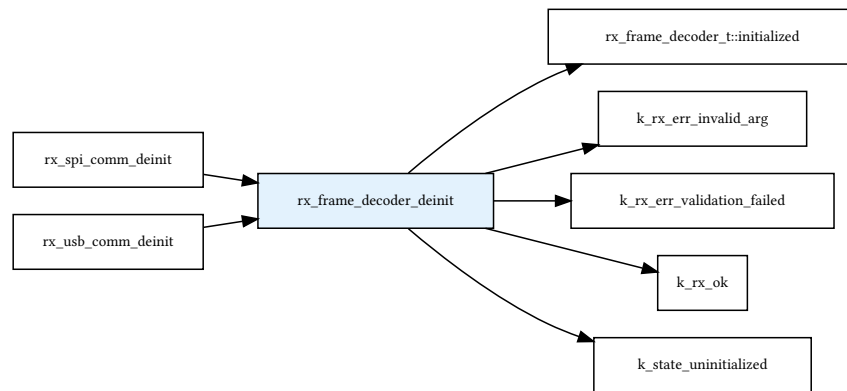


Figure 449: Call graph for rx_frame_decoder_deinit

Defined in `libs/rx_frame/inc/rx_frame.h:1048`

—

rx_frame_decode

```
rx_err_t rx_frame_decode(const rx_frame_decoder_t *dec, const uint8_t *data,
                        const uint32_t data_len, rx_frame_t *frame)
```

Decode wire format to frame.

Parses wire format and validates SYNC word and CRC-32.

Decode wire format to frame.

Validates the decoder state, extracts the frame header, copies the payload, and verifies the CRC-32 checksum.

Parameters:

Direction	Name	Description
in	dec	Decoder handle
in	data	Wire format data
in	data_len	Data length
out	frame	Decoded frame
in	dec	Initialized frame decoder instance

in	data	Raw frame data buffer
in	data_len	Length of data buffer in bytes
out	frame	Decoded frame (header, payload, CRC)

Returns: k_rx_err_crc_mismatch if CRC validation fails

Value	Description
k_rx_ok	Success - frame decoded and CRC verified
k_rx_err_invalid_arg	Any pointer parameter is nullptr or other invalid arguments
k_rx_err_invalid_state	Decoder not initialized
k_rx_err_crc	CRC-32 verification failed
k_rx_err_invalid_size	Payload exceeds maximum frame size

Note: This function performs validation at multiple points:] Null pointer checks on all parameters Decoder initialization check Header parsing via internal_decode_header() CRC verification via internal_verify_crc()

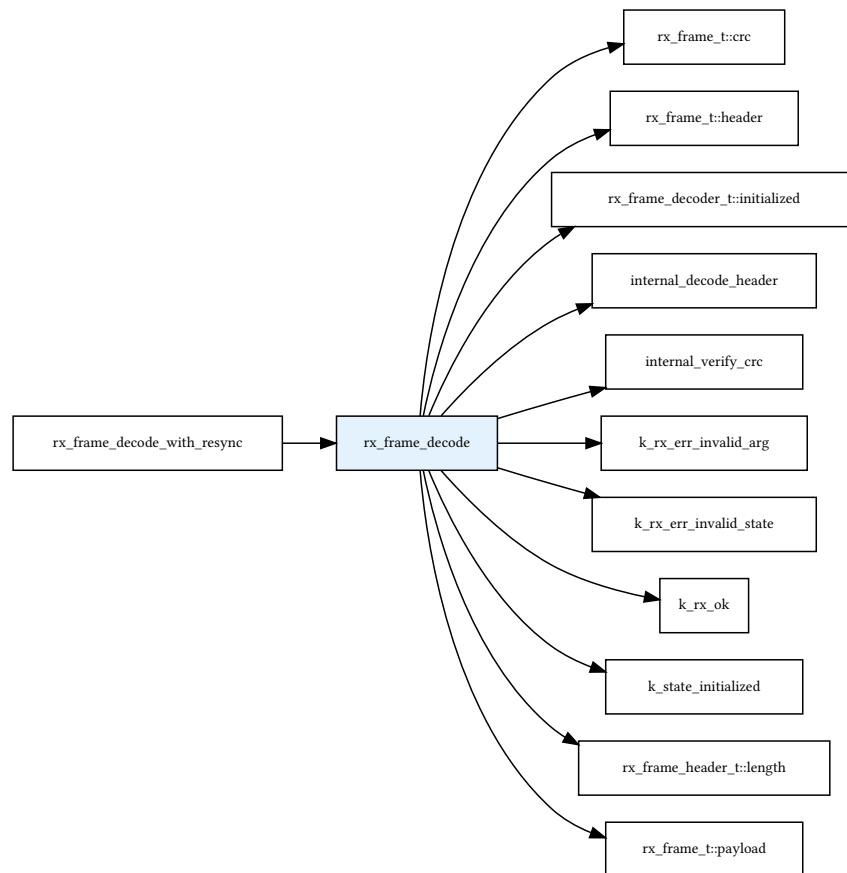


Figure 450: Call graph for rx_frame_decode

Defined in `libs/rx_frame/inc/rx_frame.h:1067`

rx_frame_decode_with_resync

```
rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t *dec, const uint8_t
*data, const uint32_t data_len, rx_frame_t *frame, uint32_t *bytes_discarded_out)
```

Decode wire format to frame with bounded sync-word resynchronization.

Extends rx_frame_decode() with automatic stream recovery. When the sync word is not found at offset 0 (i.e., the stream is byte-misaligned due to a dropped or inserted byte), this function scans forward up to k_frame_max_scan_bytes positions looking for the next sync word occurrence. Bytes discarded during the scan are reported via bytes_discarded_out for diagnostic logging by the caller.

If the sync word is located within the scan window, decoding proceeds from that offset as if data had been provided aligned. If no sync word is found within the bounded window, k_rx_err_protocol_error is returned.

Algorithm:

1. Attempt normal decode at offset 0.
2. On `k_rx_err_protocol_error` (sync mismatch), invoke `internal_find_sync_offset()` to locate the next sync word.
3. If found, set `*bytes_discarded_out = sync_offset` caller must advance the stream read pointer by this value even if the subsequent decode fails (e.g., `k_rx_err_crc_mismatch`), to avoid infinite re-scan.
4. Decode from the discovered offset (trimmed view of data).
5. All other errors (CRC, size) are propagated unchanged.

Decode wire format to frame with bounded sync-word resynchronization.

Provides stream recovery when the byte stream has become misaligned due to a dropped or inserted byte. On sync mismatch, scans forward up to `k_frame_max_scan_bytes` positions for the next occurrence of the sync word, discards the leading bytes, and reattempts decoding from that position.

This function is safe for repeated calls on a stream: the caller must advance its stream read pointer by `*bytes_discarded_out` on `k_rx_ok` and also on `k_rx_err_crc_mismatch` (any case where `bytes_discarded_out` is set) to avoid re-processing discarded bytes in subsequent calls.

Recovery algorithm:

1. Attempt `rx_frame_decode()` on `data[0..data_len-1]`.
2. If result is not `k_rx_err_protocol_error`, return immediately.
3. Call `internal_find_sync_offset()` to locate next sync word in `data[1..]`.
4. If not found, return `k_rx_err_protocol_error` with `*bytes_discarded_out = 0`.
5. Decode from `data[sync_offset..data_len-1]` using the trimmed sub-buffer.
6. Write `sync_offset` to `*bytes_discarded_out` and return result.

Parameters:

Direction	Name	Description
in	<code>dec</code>	Initialized frame decoder handle
in	<code>data</code>	Raw byte buffer (may be misaligned)
in	<code>data_len</code>	Length of data buffer in bytes
out	<code>frame</code>	Decoded frame (header, payload, CRC)
out	<code>bytes_discarded_out</code>	Number of bytes skipped to reach sync word (0 when sync was found at offset 0)
in	<code>dec</code>	Initialized frame decoder handle
in	<code>data</code>	Raw byte buffer (may be stream-misaligned)
in	<code>data_len</code>	Length of data buffer in bytes
out	<code>frame</code>	Decoded frame (header, payload, CRC)
out	<code>bytes_discarded_out</code>	Bytes skipped before sync word (0 = aligned)

Returns: `rx_err_t` `k_rx_ok` on success, or `k_rx_err_invalid_arg`, `k_rx_err_invalid_state`, `k_rx_err_invalid_size`, `k_rx_err_protocol_error`, or `k_rx_err_crc_mismatch` on failure

Value	Description
<code>k_rx_ok</code>	Frame decoded and CRC verified
<code>k_rx_err_invalid_arg</code>	Any pointer parameter is nullptr

k_rx_err_invalid_state	Decoder not initialized
k_rx_err_invalid_size	Data buffer too short to contain a valid frame
k_rx_err_protocol_error	No sync word found within k_frame_max_scan_bytes
k_rx_err_crc_mismatch	Sync found but CRC validation failed
k_rx_ok	Frame decoded and CRC verified
k_rx_err_invalid_arg	Any pointer parameter is nullptr
k_rx_err_invalid_state	Decoder not initialized
k_rx_err_invalid_size	Buffer too short to contain a valid frame
k_rx_err_protocol_error	No sync word found within scan window
k_rx_err_crc_mismatch	Sync found but CRC verification failed

Pre: dec must be initialized via rx_frame_decoder_init()

Pre: data must point to a buffer of at least data_len bytes

Pre: frame must be a non-null pointer to a writable rx_frame_t

Pre: bytes_discarded_out must be a non-null pointer to a writable uint32_t

Pre: dec must be initialized via rx_frame_decoder_init()

Pre: data must point to a buffer of at least data_len bytes

Pre: frame must point to a valid rx_frame_t storage location (not NULL)

Pre: bytes_discarded_out must point to a valid uint32_t storage location (not NULL)

Post: On k_rx_ok, frame contains the decoded frame with verified CRC

Post: bytes_discarded_out contains the number of bytes skipped (≥ 0)

Post: On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer

Post: On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word

Post: On k_rx_ok, frame contains a fully verified decoded frame

Post: On `k_rx_err_crc_mismatch`, `*bytes_discarded_out == bytes skipped`; caller must still advance stream pointer

Note: Thread-safe after init; the decoder handle is read-only and the function uses no shared mutable state (consistent with the file-level “Stateless after init”).

Note: The scan window is bounded at `k_frame_max_scan_bytes` (NASA Rule 2).

Note: Caller should log `bytes_discarded_out > 0` as a diagnostic warning.

Note: Parameter order follows the canonical convention: decoder input -> data input -> frame output -> diagnostic output (matching `rx_frame_decode()`).

Note: Not thread-safe. Caller must synchronize access.

Note: The scan is bounded at `k_frame_max_scan_bytes` (NASA Rule 2).

Note: Log `*bytes_discarded_out > 0` as a stream-health diagnostic warning.

Warning: Caller must advance its read pointer by `*bytes_discarded_out` bytes after any call that returns `k_rx_ok` or `k_rx_err_crc_mismatch` to avoid infinite re-scan; `*bytes_discarded_out` is unmodified only when `k_rx_err_invalid_arg` is returned.

Warning: Caller must advance stream read pointer by `*bytes_discarded_out` after any call that returns `k_rx_ok` or `k_rx_err_crc_mismatch` to avoid infinite re-scan; `*bytes_discarded_out` is unmodified only when `k_rx_err_invalid_arg` is returned.

Performance:: Worst case: scans `k_frame_max_scan_bytes` (1036) before failing. Best case: identical to `rx_frame_decode()` (0 bytes discarded).

Example:: `rx_frame_decoder_tdec; rx_frame_decoder_init(&dec);`

```
rx_frame_tframe; uint32_tdiscarded=0;
rx_err_tterr=rx_frame_decode_with_resync(&dec,wire_buf,wire_len, &frame,&discarded);
if(discarded>0){ rx_log_warn("FRAME","Streamresync:discarded%ubytes",discarded); }
if(err==k_rx_ok){ process_frame(&frame); }
```

```
Example:: uint32_tdiscarded=k_bytes_discarded_none;
rx_err_tterr=rx_frame_decode_with_resync(dec,data,data_len,&frame,&discarded); if(err==k_rx_ok)
{ stream_read_ptr+=discarded; }elseif(err==k_rx_err_crc_mismatch){ stream_read_ptr+=discarded;
rx_log_error(TAG,"syncfoundbutCRCfailed"); }
else{ rx_log_error(TAG,"nosyncwordwithinscanwindow"); }
```

See also: `rx_frame_decode()` Simple decode without resynchronization,
`k_frame_max_scan_bytes` Maximum scan window (1036 bytes), `rx_frame_decode()` Simple

decode without stream recovery, internal_find_sync_offset() Bounded sync word scanner,
k_frame_max_scan_bytes Scan window upper bound

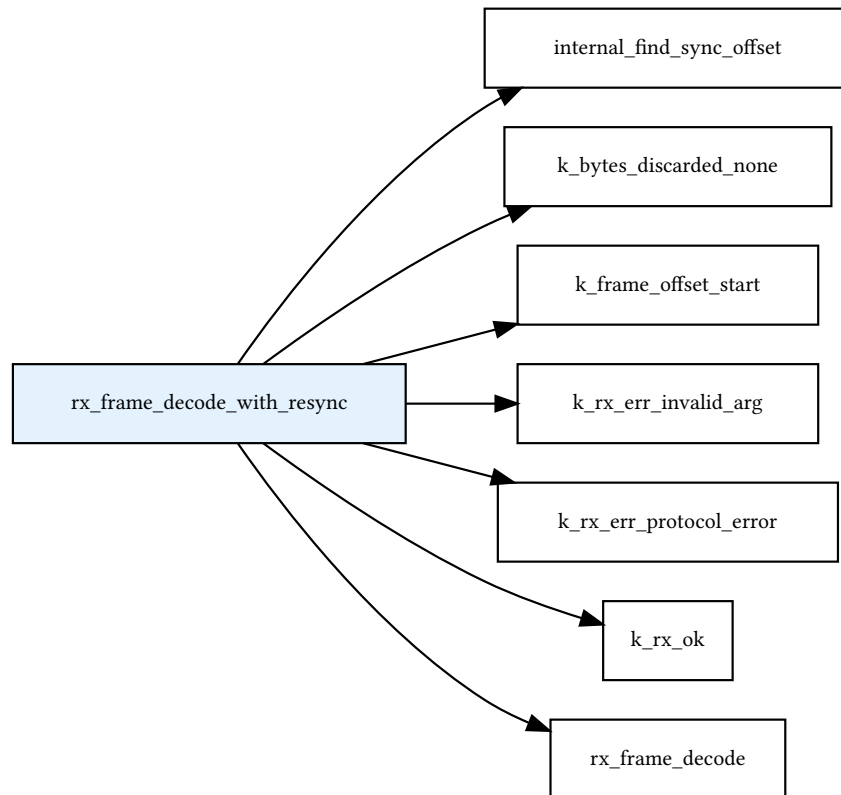


Figure 451: Call graph for `rx_frame_decode_with_resync`

Defined in `libs/rx_frame/inc/rx_frame.h:1154`

Since Version 1.1.0

—

rx_frame_create_ack

```
rx_err_t rx_frame_create_ack(rx_frame_t *frame, uint16_t sequence)
```

Create ACK frame for given sequence.

Create ACK frame for given sequence.

Constructs a frame with type=ACK, empty payload, and the specified sequence number. The ACK response frames are used by the receiver to confirm successful reception of a command or data frame from the sender.

Parameters:

Direction	Name	Description
-----------	------	-------------

out	frame	Frame to initialize
in	sequence	Sequence number to acknowledge
out	frame	Frame structure to populate with ACK
in	sequence	Sequence number of the frame being acknowledged

Returns: k_rx_ok on successful frame creation

Value	Description
k_rx_err_invalid_arg	if frame is nullptr
k_rx_err_validation_failed	if ACK type verification fails

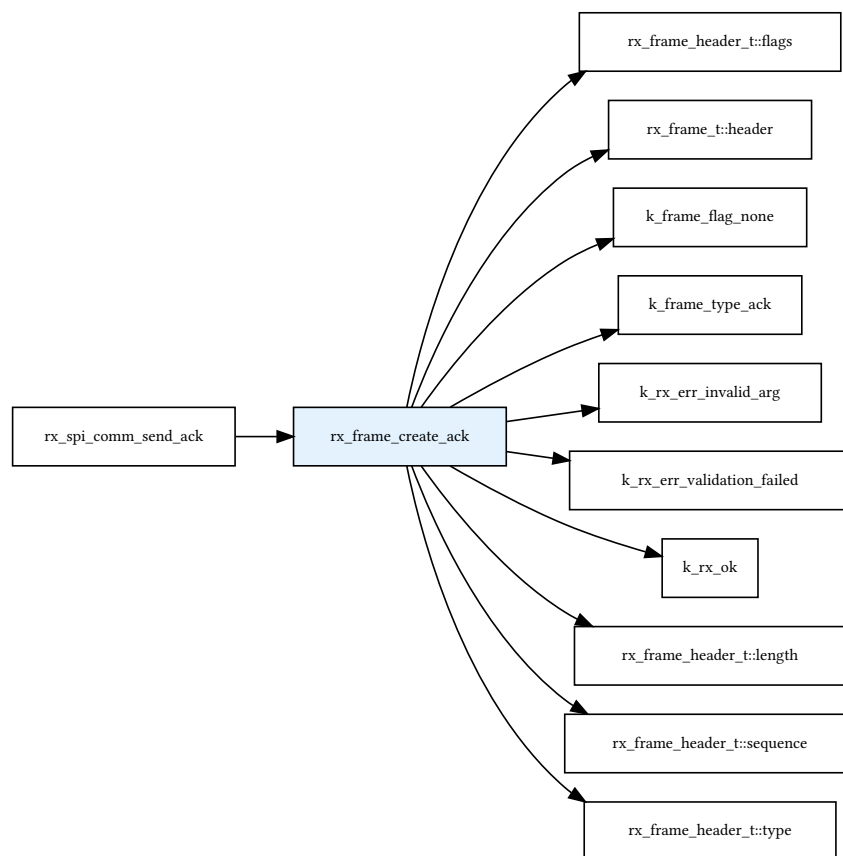


Figure 452: Call graph for `rx_frame_create_ack`

Defined in `libs/rx_frame/inc/rx_frame.h:1172`

rx_frame_create_nack

```
rx_err_t rx_frame_create_nack(rx_frame_t *frame, uint16_t sequence, uint8_t flags)
```

Create NACK frame for given sequence.

Create NACK frame for given sequence.

Constructs a frame with type=NACK, empty payload, the specified sequence number, and error/status flags. NACK frames are sent by the receiver to indicate that a frame was not processed successfully due to an error condition (specified in flags).

Parameters:

Direction	Name	Description
out	frame	Frame to initialize
in	sequence	Sequence number
in	flags	Additional flags (e.g., k_frame_flag_soft_nack)
out	frame	Frame structure to populate with NACK
in	sequence	Sequence number of the frame being negative-acknowledged
in	flags	Status/error flags indicating reason for NACK

Returns: k_rx_ok on successful frame creation

Value	Description
k_rx_err_invalid_arg	if frame is nullptr
k_rx_err_validation_failed	if NACK type verification fails

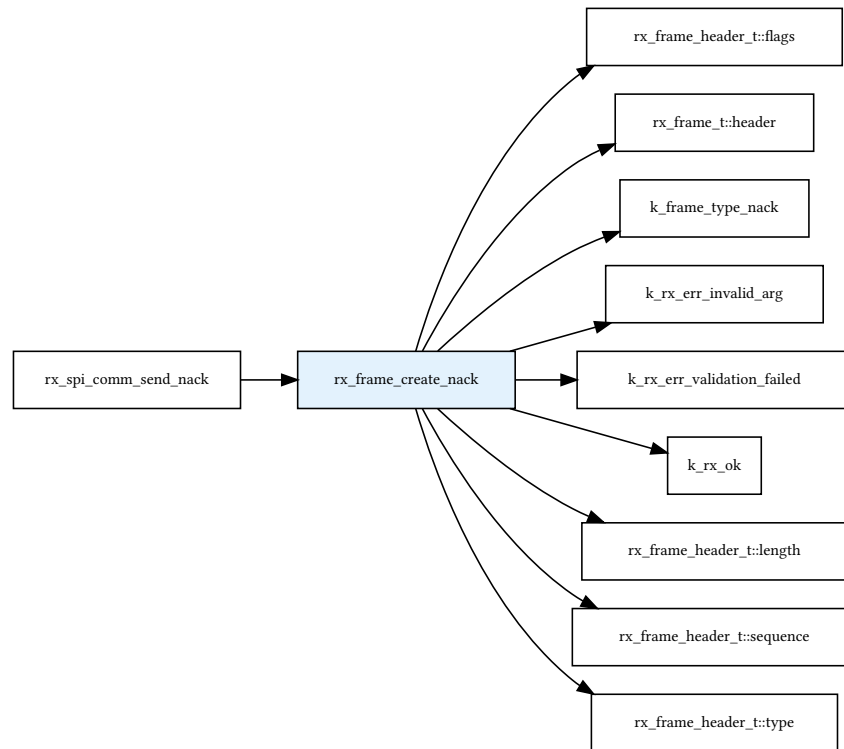


Figure 453: Call graph for rx_frame_create_nack

Defined in `libs/rx_frame/inc/rx_frame.h:1182`

—

rx_frame_create_ping

```
rx_err_t rx_frame_create_ping(rx_frame_t *frame, const uint16_t sequence, const
uint8_t *payload, const uint32_t payload_len)
```

Create PING frame for keepalive/heartbeat.

Create PING frame for keepalive/heartbeat.

Initializes frame as a PING request for connection liveness checks. Receiver should respond with a PONG frame echoing the payload. Payload is optional; a 4-byte LE counter is typical.

Parameters:

Direction	Name	Description
out	frame	Frame to initialize
in	sequence	Sequence number
in	payload	Optional payload to include (may be NULL if payload_len is 0)
in	payload_len	Payload length in bytes

out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number for this frame
in	payload	Optional payload (NULL when payload_len is 0)
in	payload_len	Payload length in bytes [0, k_frame_max_payload]

Returns: k_rx_ok on success

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is NULL, or payload NULL with len > 0
k_rx_err_invalid_size	payload_len exceeds k_frame_max_payload
k_rx_err_validation_failed	Post-condition type check failed

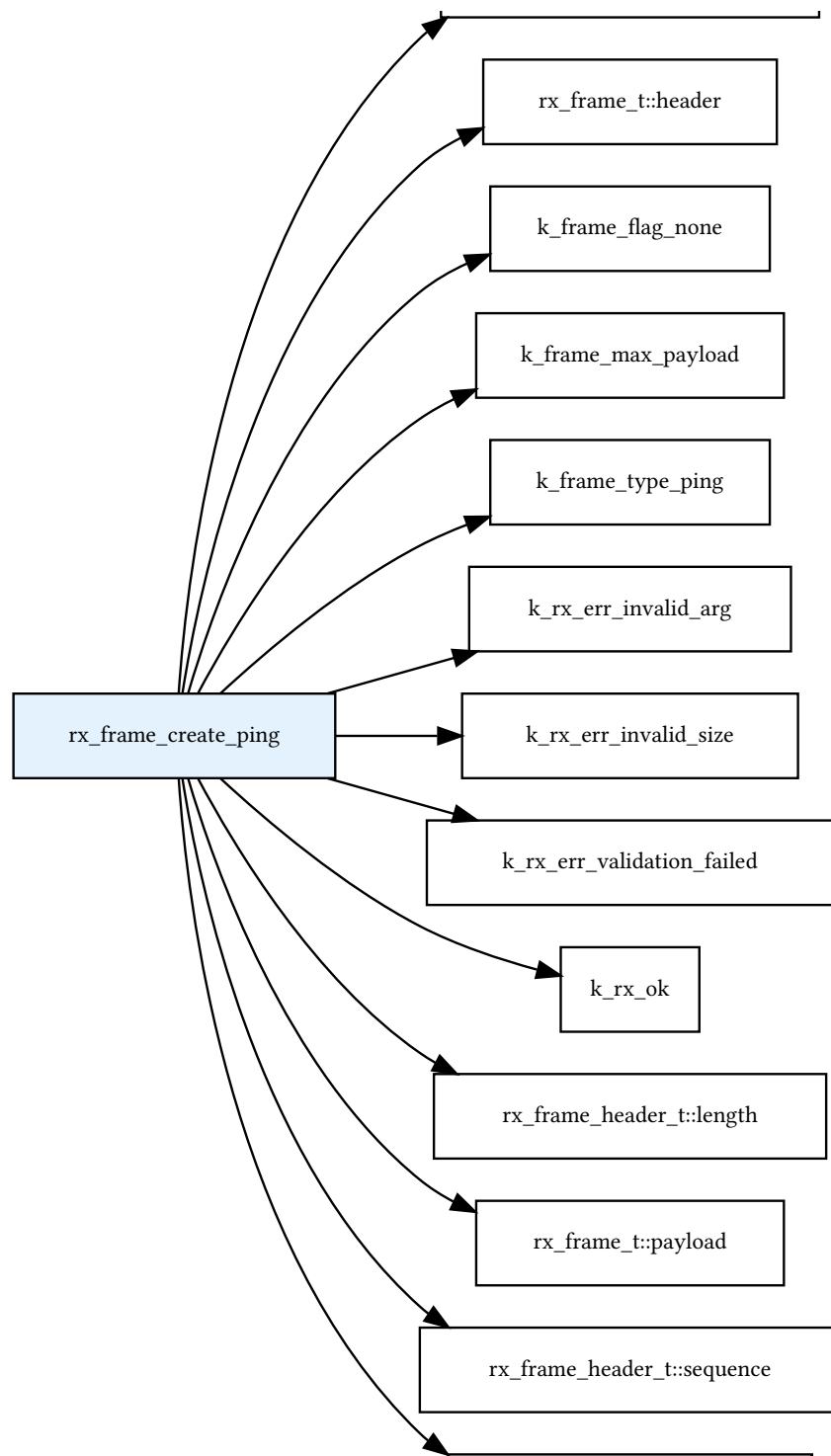
Pre: frame != nullptr

Pre: payload != NULL when payload_len > 0

Post: frame->header.type == k_frame_type_ping

Post: frame->header.length == payload_len

Note: Stateless; safe to call from any context.] **See also:** rx_frame_create_pong()
Corresponding response creator

Figure 454: Call graph for `rx_frame_create_ping`

Defined in `libs/rx_frame/inc/rx_frame.h:1193`

Since Version 1.0.0

—

rx_frame_create_pong

```
rx_err_t rx_frame_create_pong(rx_frame_t *frame, const uint16_t sequence, const
uint8_t *payload, const uint32_t payload_len)
```

Create PONG frame (response to PING).

Create PONG frame (response to PING).

Initializes frame as a PONG response to a received PING. Echoes the PING payload so the sender can validate round-trip data.

Parameters:

Direction	Name	Description
out	frame	Frame to initialize
in	sequence	Sequence number (matches ping)
in	payload	Optional payload to include (may be NULL if payload_len is 0)
in	payload_len	Payload length in bytes
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number for this frame
in	payload	Payload to echo (NULL when payload_len is 0)
in	payload_len	Payload length in bytes [0, k_frame_max_payload]

Returns: k_rx_ok on success

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is NULL, or payload NULL with len > 0
k_rx_err_invalid_size	payload_len exceeds k_frame_max_payload
k_rx_err_validation_failed	Post-condition type check failed

Pre: frame != nullptr

Pre: payload != NULL when payload_len > 0

Post: frame->header.type == k_frame_type_pong

Post: frame->header.length == payload_len

Note: Stateless; safe to call from any context.] **See also:** `rx_frame_create_ping()`
Corresponding request creator

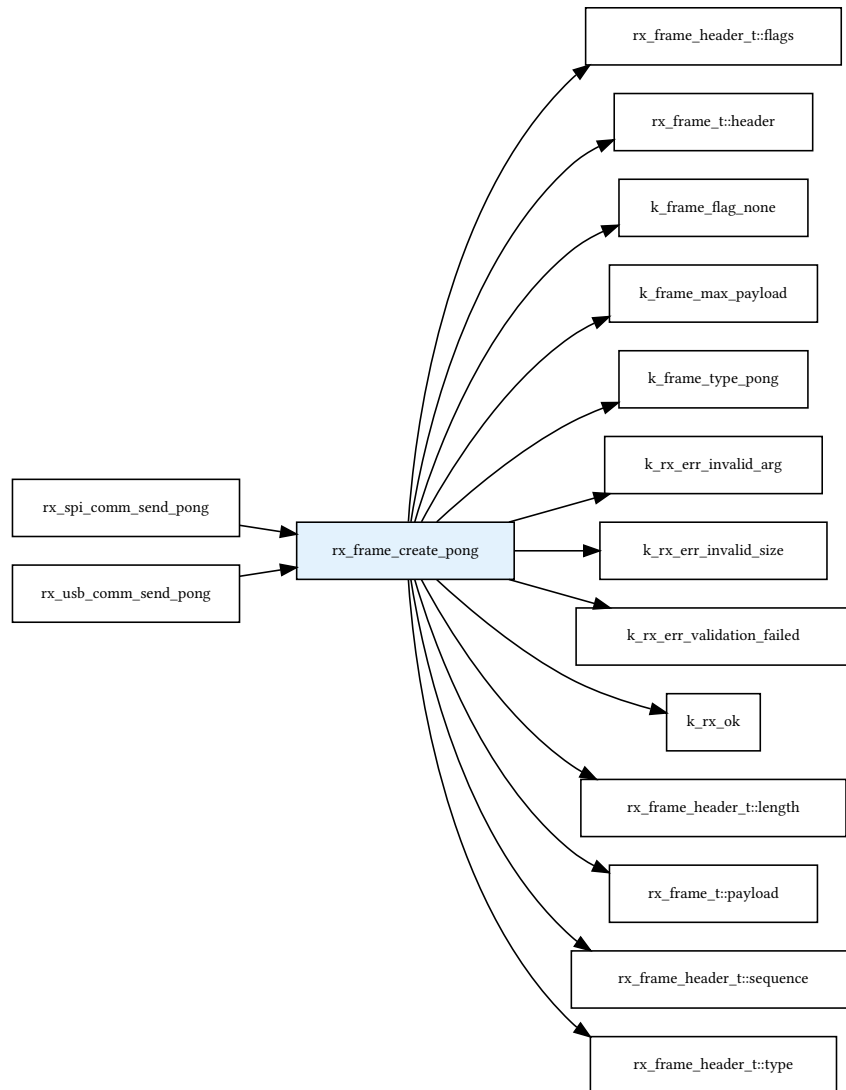


Figure 455: Call graph for `rx_frame_create_pong`

Defined in `libs/rx_frame/inc/rx_frame.h:1207`

Since Version 1.0.0

—

rx_frame_create_reset

```
rx_err_t rx_frame_create_reset(rx_frame_t *frame, const uint16_t sequence)
```

Create RESET frame to reset communication state.

Create RESET frame to reset communication state.

Initializes frame as a RESET request that asks the peer to reset communication state (sequence numbers, buffers). The receiver should respond with a RESET_ACK frame. Payload is always empty.

Parameters:

Direction	Name	Description
out	frame	Frame to initialize
in	sequence	Sequence number
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number for this frame

Returns: k_rx_ok on success

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is nullptr
k_rx_err_validation_failed	Post-condition type check failed

Pre: frame != nullptr

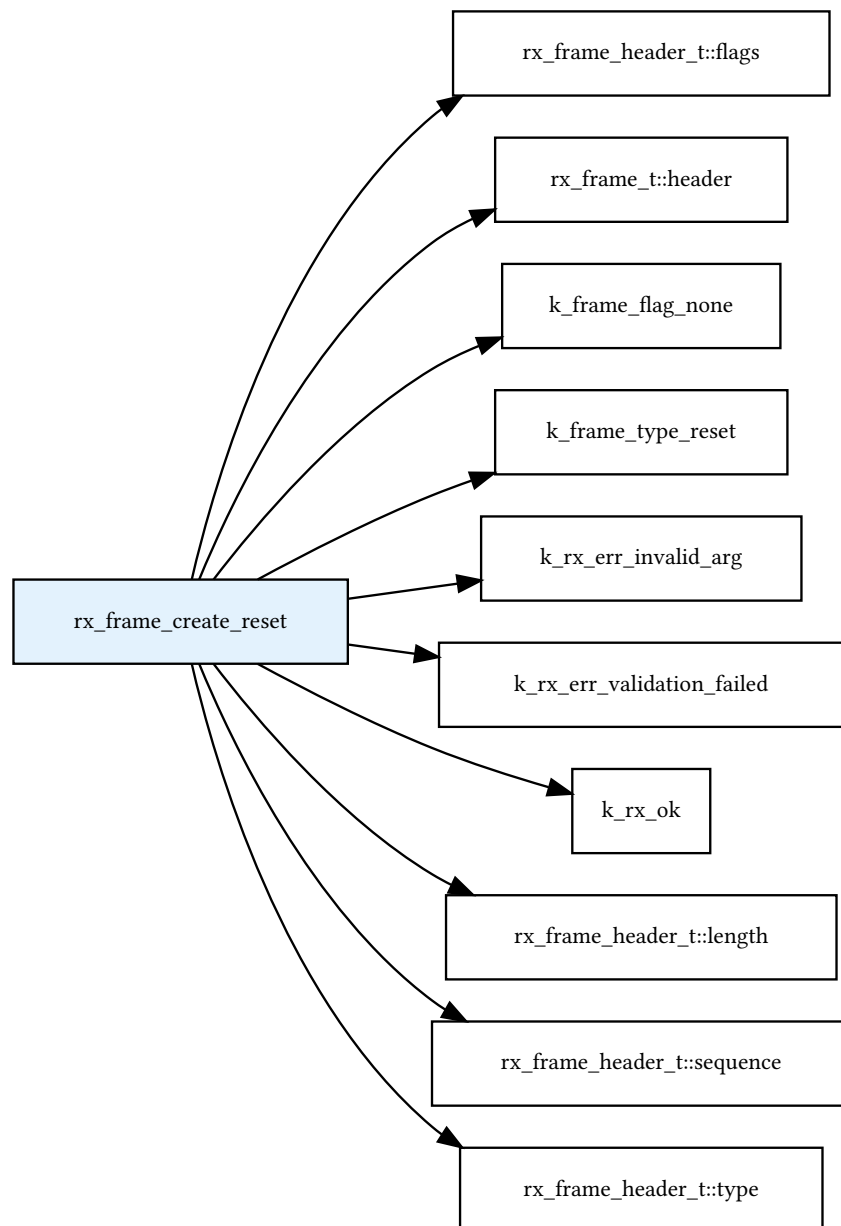
Pre: Caller has determined that a reset is necessary

Post: frame->header.type == k_frame_type_reset

Post: frame->header.length == 0

Note: Stateless; safe to call from any context.

Warning: Reset clears all sequence number state on both sides.] **See also:**
rx_frame_create_reset_ack() Corresponding acknowledgment creator

Figure 456: Call graph for `rx_frame_create_reset`

Defined in `libs/rx_frame/inc/rx_frame.h:1219`

Since Version 1.0.0

—

rx_frame_create_reset_ack

```
rx_err_t rx_frame_create_reset_ack(rx_frame_t *frame, const uint16_t sequence)
```

Create RESET_ACK frame (response to RESET).

Create RESET_ACK frame (response to RESET).

Initializes frame as a RESET_ACK response to a received RESET request. Confirms that the receiver has completed its reset procedure. Payload is always empty; sequence matches the RESET request.

Parameters:

Direction	Name	Description
out	frame	Frame to initialize
in	sequence	Sequence number (matches reset)
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number (should match received RESET)

Returns: k_rx_ok on success

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is nullptr
k_rx_err_validation_failed	Post-condition type check failed

Pre: frame != nullptr

Pre: A RESET frame was received and processed

Post: frame->header.type == k_frame_type_reset_ack

Post: frame->header.length == 0

Note: Stateless; safe to call from any context.] **See also:** rx_frame_create_reset()
Corresponding request creator

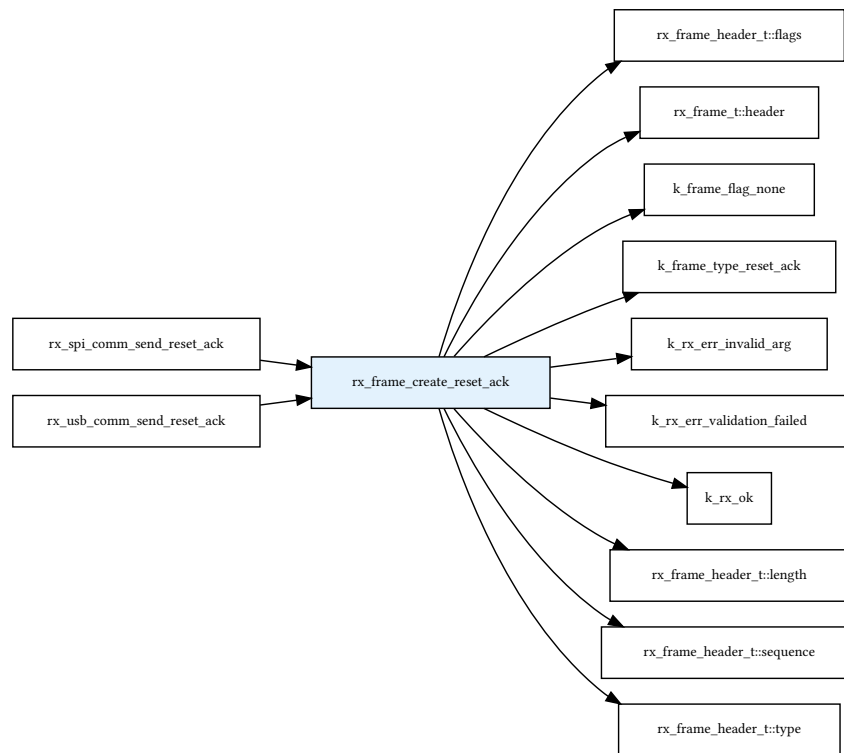


Figure 457: Call graph for rx_frame_create_reset_ack

Defined in `libs/rx_frame/inc/rx_frame.h:1228`

Since Version 1.0.0

—

rx_frame_type_valid

```
static bool rx_frame_type_valid(uint8_t type)
```

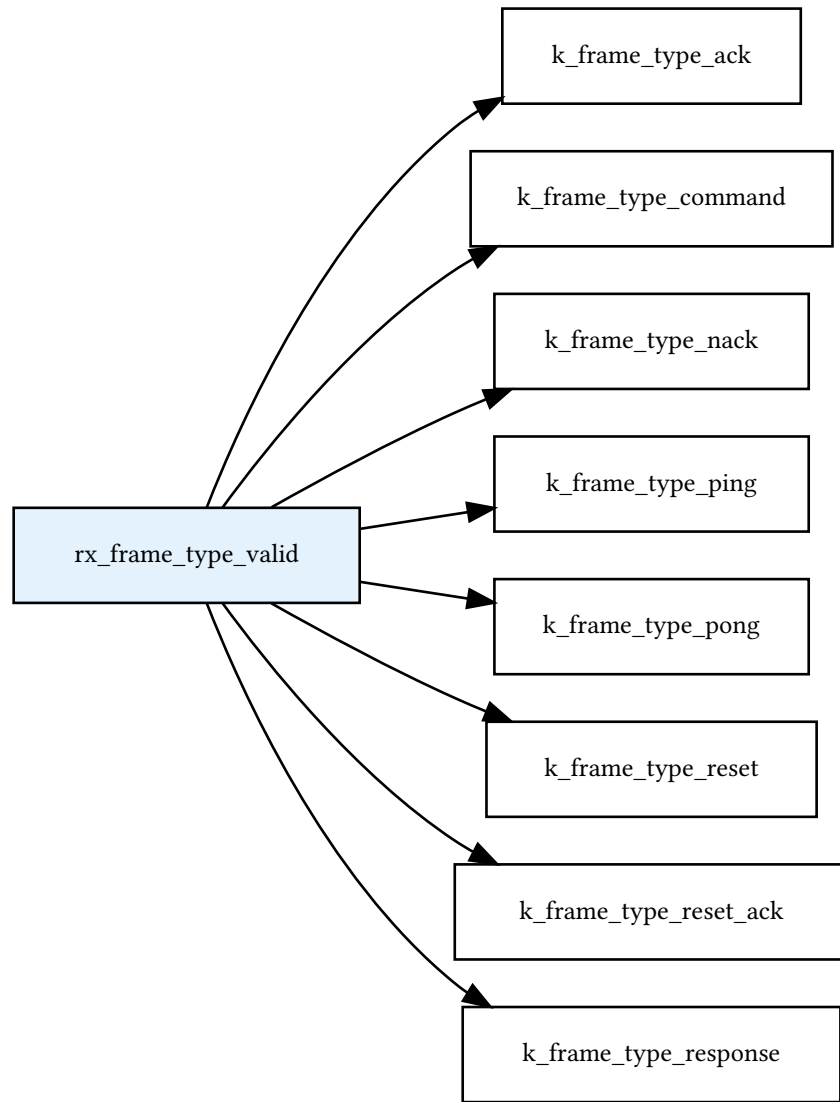
Check if frame type is valid.

Static inline for same reasons as `rx_frame_encoded_size()` above.

Parameters:

Direction	Name	Description
in	type	Frame type to check

Returns: true if valid, false otherwise

Figure 458: Call graph for `rx_frame_type_valid`

Defined in `libs/rx_frame/inc/rx_frame.h:1238`

—

rx_frame.c

Frame Layer Protocol Implementation.

Includes:

- `"rx_frame.h"`
- `<string.h>`
- `"rx_check.h"`

- "rx_crc.h"

byte_shift_t

Bit shift amounts for extracting bytes from multi-byte values.

Each byte position requires shifting by (position * 8) bits.

Value	Name	Description
= 0	k_shift_byte_0	No shift needed for byte 0 (LSB)
= 8	k_shift_byte_1	Shift 8 bits for byte 1
= 16	k_shift_byte_2	Shift 16 bits for byte 2
= 24	k_shift_byte_3	Shift 24 bits for byte 3 (MSB)

—

init_state_t

Initialization state values.

Value	Name	Description
= 0	k_state_uninitialized	Object not initialized
= 1	k_state_initialized	Object initialized and ready

—

frame_offset_t

Frame byte offsets used during encoding, decoding, and sync scanning.

Defines named byte-offset constants used by the frame encoder, decoder, and the bounded sync-word scanner. `k_frame_offset_start` marks the beginning of a frame buffer for normal aligned decoding. `k_frame_scan_start_offset` is the index at which `internal_find_sync_offset()` begins its forward scan offset 0 is omitted because `rx_frame_decode()` has already tested it before invoking the scanner, so the scan starts at the next candidate position.

`k_frame_scan_start_offset > k_frame_offset_start`, ensuring the scanner never re-tests the byte that `rx_frame_decode()` already checked.

Value	Name	Description
= 0	k_frame_offset_start	Start offset for frame parsing
= 1	k_frame_scan_start_offset	First offset checked in <code>internal_find_sync_offset()</code> ; offset 0 is presumed already tested by <code>rx_frame_decode()</code>

See also: `rx_frame_decode()` Uses `k_frame_offset_start` for normal aligned decoding, `internal_find_sync_offset()` Begins scan at `k_frame_scan_start_offset`

Example:

```
//Normalaligneddecodestartsatk_frame_offset_start(0)
rx_err_t err=rx_frame_decode(dec,data,data_len,frame);

//Onsyncmismatch,internal_find_sync_offset()scansfromoffset1:
uint8_t sync_off=k_frame_scan_start_offset;
//...scanneradvancesync_offuntilsyncwordfoundorscanlimitreached
```

Since Version 1.1.0

—

frame_resync_discarded_t

Byte-discard counter initial value for rx_frame_decode_with_resync().

k_bytes_discarded_none is the zero sentinel written to *bytes_discarded_out at the start of rx_frame_decode_with_resync() before any sync search. Using a named constant makes the intent auditable and prevents a raw 0 literal from appearing in the implementation (NASA Rule 8 / STAR no-magic-numbers policy).

k_bytes_discarded_none == 0 (matches the initial no-discard state)

Value	Name	Description
= 0U	k_bytes_discarded_none	No bytes have been discarded; stream is aligned

See also: rx_frame_decode_with_resync() Only caller of this constant

Example:

```
uint32_t bytes_discarded=k_bytes_discarded_none;
rx_err_t err=rx_frame_decode_with_resync(&dec,data,data_len,&frame,&bytes_discarded);
```

Since Version 1.1.0

—

internal_write_le32

```
static rx_err_t internal_write_le32(uint8_t *buf, const uint32_t buf_len, const
uint32_t val)
```

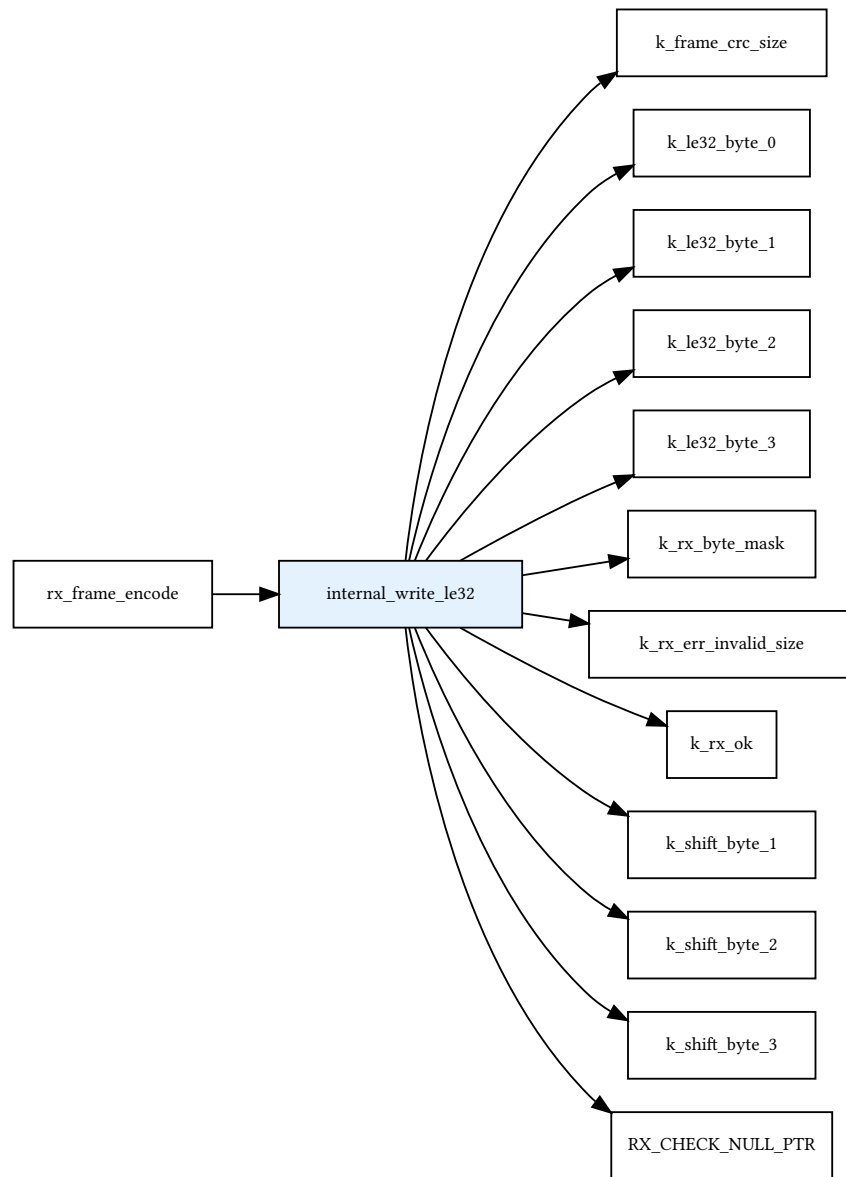
Write uint32 in little-endian format (for CRC-32).

Parameters:

Direction	Name	Description
out	buf	Output buffer (at least 4 bytes)
in	buf_len	Size of output buffer in bytes (must be >= 4)
in	val	Value to write

Returns: k_rx_ok on success

Value	Description
k_rx_err_invalid_arg	if buf is nullptr
k_rx_err_invalid_size	if buf_len < 4

Figure 459: Call graph for `internal_write_le32`

Defined in `libs/rx_frame/src/rx_frame.c:287`

`internal_read_le32`

```
static rx_err_t internal_read_le32(const uint8_t *buf, const uint32_t buf_len,  
uint32_t *out_val)
```

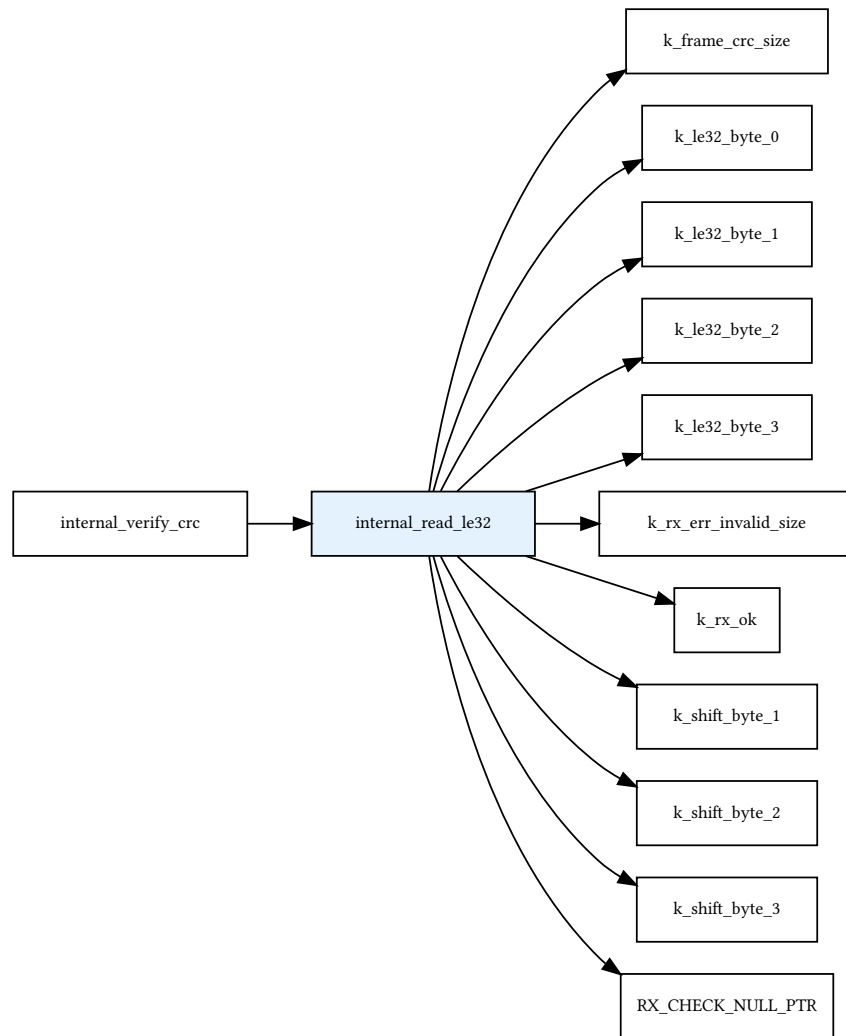
Read uint32 in little-endian format (for CRC-32).

Parameters:

Direction	Name	Description
in	buf	Input buffer (at least 4 bytes)
in	buf_len	Size of input buffer in bytes (must be ≥ 4)
out	out_val	Pointer to store decoded value

Returns: k_rx_ok on success

Value	Description
k_rx_err_invalid_arg	if buf or out_val is nullptr
k_rx_err_invalid_size	if buf_len < 4

Figure 460: Call graph for `internal_read_le32`

Defined in `libs/rx_frame/src/rx_frame.c:312`

internal_decode_header

```
static rx_err_t internal_decode_header(const uint8_t *data, const uint32_t
data_len, rx_frame_t *frame, uint32_t *offset_out)
```

Decode frame header from raw data buffer.

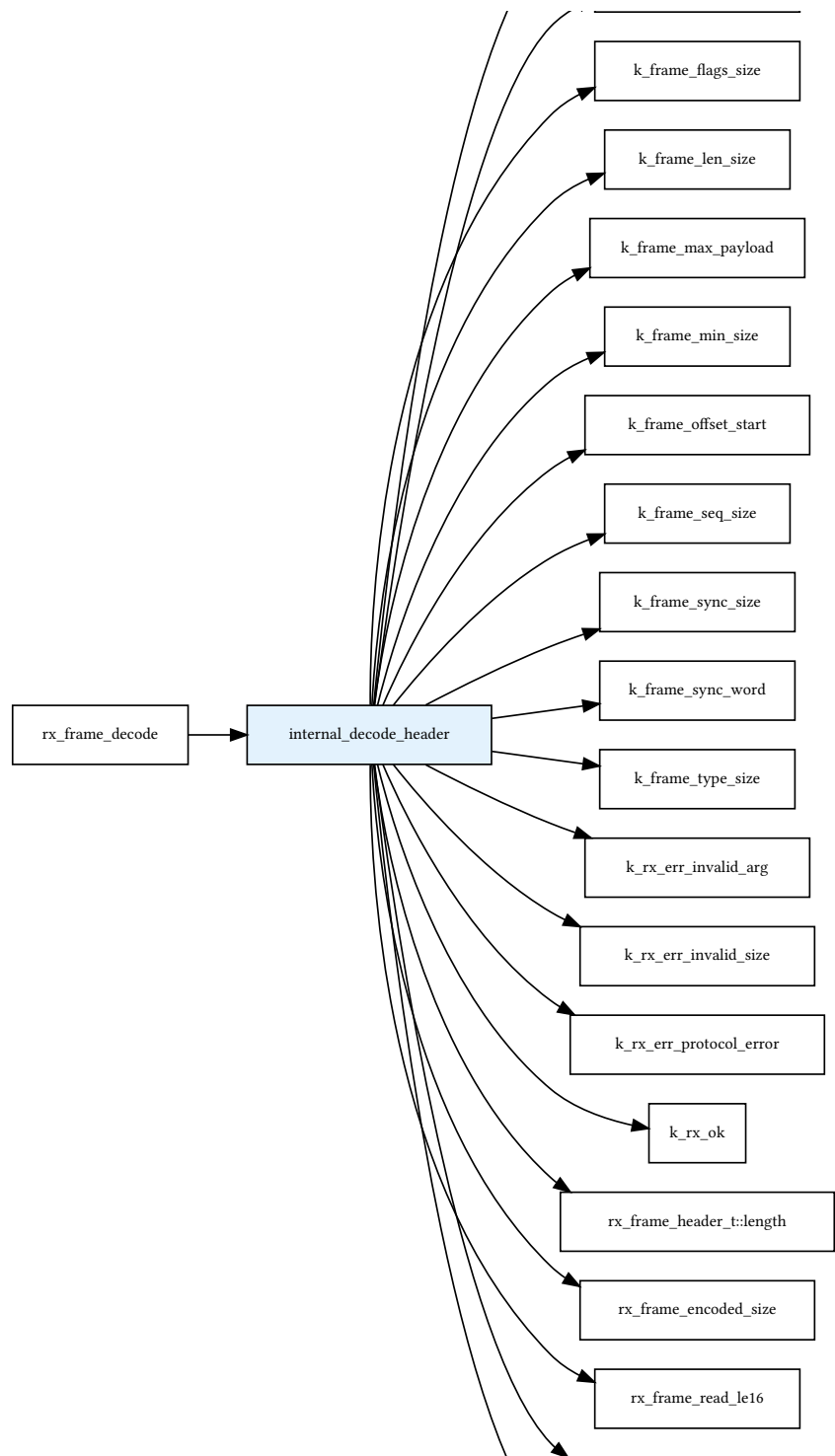
Extracts and validates frame header fields (sync word, sequence, length, type, flags) from the beginning of a raw data buffer. Verifies the sync word matches expected value and validates payload length is within bounds. Outputs the offset where payload data begins for subsequent processing.

Parameters:

Direction	Name	Description
in	data	Input data buffer containing encoded frame
in	data_len	Length of input buffer in bytes
out	frame	Frame structure to populate with decoded header fields
out	offset_out	Offset in buffer where payload begins (after header and sync)

Returns: k_rx_ok on successful header decode

Value	Description
k_rx_err_invalid_arg	if data, frame, or offset_out is nullptr
k_rx_err_invalid_size	if data_len is too small or decoded payload length is out of bounds
k_rx_err_protocol_error	if sync word does not match expected value (k_frame_sync_word)

Figure 461: Call graph for `internal_decode_header`

Defined in `libs/rx_frame/src/rx_frame.c:344`

—

internal_verify_crc

```
static rx_err_t internal_verify_crc(const uint8_t *data, uint32_t data_len,  
uint32_t offset, uint32_t *crc_out)
```

Verify CRC-32 checksum at specified offset in data buffer.

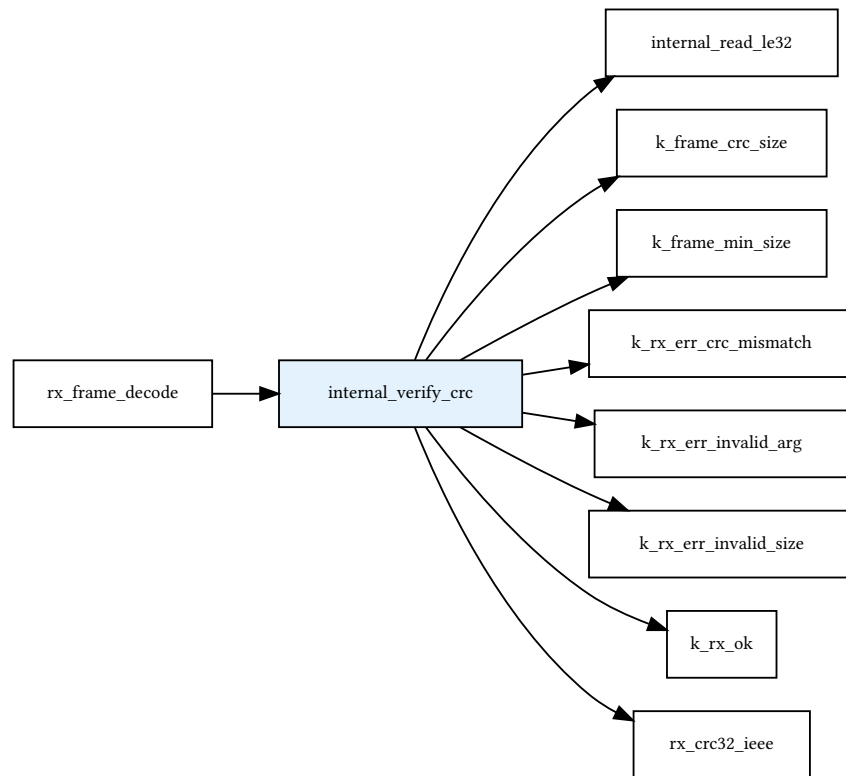
Reads the CRC-32 value stored at the specified offset in the buffer, calculates the expected CRC over the data preceding it, and compares for match. Uses IEEE 802.3 CRC-32 polynomial (0x04C11DB7), compatible with Go's `crc32.ChecksumIEEE()`. On success, outputs the received CRC value for caller inspection.

Parameters:

Direction	Name	Description
in	data	Input data buffer containing payload followed by CRC
in	data_len	Total length of input buffer in bytes
in	offset	Byte offset in buffer where CRC value is stored
out	crc_out	Pointer to store received CRC value from buffer

Returns: `k_rx_ok` on successful CRC verification (received matches calculated)

Value	Description
<code>k_rx_err_invalid_arg</code>	if data or crc_out is nullptr
<code>k_rx_err_invalid_size</code>	if offset + CRC size exceeds buffer length or data_len too small
<code>k_rx_err_crc_mismatch</code>	if calculated CRC does not match received CRC value

Figure 462: Call graph for `internal_verify_crc`

Defined in `libs/rx_frame/src/rx_frame.c:411`

—

`internal_find_sync_offset`

```
static rx_err_t internal_find_sync_offset(const uint8_t *data, const uint32_t
data_len, uint32_t *offset_out)
```

Scan a byte buffer for the frame sync word with a fixed upper bound.

Performs a linear scan of the buffer starting at offset 1 (offset 0 is presumed to have already failed sync check) looking for the 2-byte sync pattern `k_frame_sync_word` in little-endian wire encoding (first byte 0xAA, second byte 0x55). The scan is bounded at `k_frame_max_scan_bytes` to satisfy NASA Rule 2 (fixed loop upper-bounds) and prevent infinite consumption of a permanently corrupt stream.

The function stops and reports success at the first occurrence of the sync pattern. If no pattern is found within the window, `k_rx_err_protocol_error` is returned.

Algorithm:

1. Validate preconditions (non-null pointers, minimum buffer size).
2. Compute scan limit = `min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)`.

3. Iterate `i` from 1 to `scan_limit` inclusive: a. Read 16-bit LE value at `data[i]`. b. If it matches `k_frame_sync_word`, write `i` to `offset_out` and return `k_rx_ok`.
4. Return `k_rx_err_protocol_error` if no match found.

Parameters:

Direction	Name	Description
in	<code>data</code>	Buffer to scan (at least 2 bytes)
in	<code>data_len</code>	Length of buffer in bytes (must be ≥ 2)
out	<code>offset_out</code>	Byte offset of the first sync word found (≥ 1)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Sync word found; offset written to <code>offset_out</code>
<code>k_rx_err_invalid_arg</code>	<code>data</code> or <code>offset_out</code> is <code>nullptr</code>
<code>k_rx_err_invalid_size</code>	<code>data_len</code> < 2 (cannot contain a sync word)
<code>k_rx_err_protocol_error</code>	No sync word found within <code>k_frame_max_scan_bytes</code>

Pre: `data` must point to a readable buffer of at least `data_len` bytes

Pre: `data_len` must be $\geq k_frame_sync_size$ (2)

Post: On `k_rx_ok`, `offset_out` is in range 1, $\min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)$

Post: On error, `offset_out` is unchanged

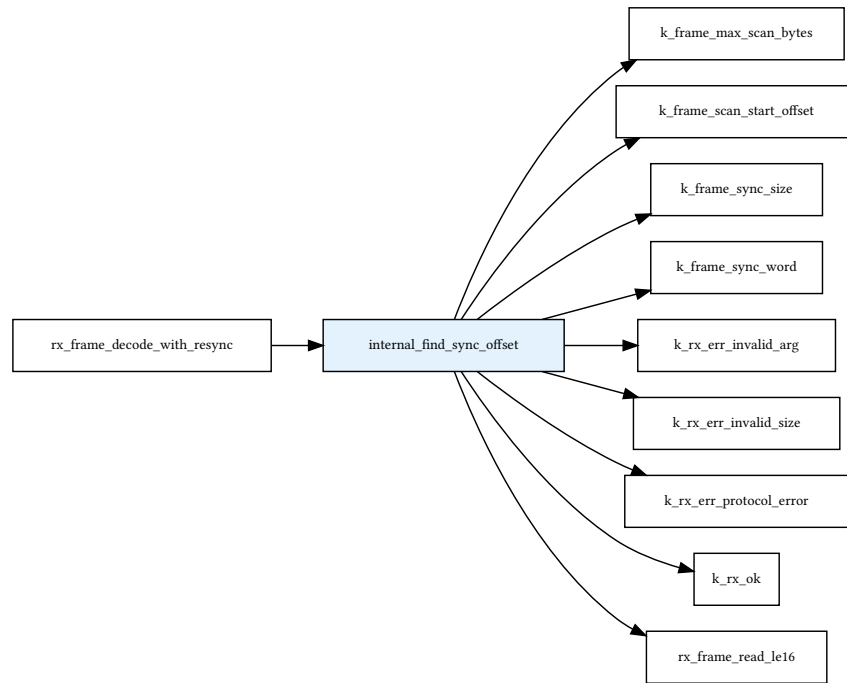
Note: Not thread-safe. Caller must ensure exclusive access to `data` buffer.

Note: The loop bound is $\min(data_len - k_frame_sync_size, k_frame_max_scan_bytes)$, which is statically bounded by `k_frame_max_scan_bytes` = 1036 (NASA Rule 2).

Performance: $O(N)$ where $N \leq k_frame_max_scan_bytes$. Each iteration performs one 16-bit LE read and one comparison. Worst case: 1036 iterations (no sync found).

Example:: `uint8_t data[]={0x00,0xAA,0x55,0x01,0x00}; uint32_t offset=k_frame_offset_start; rx_err_t err=internal_find_sync_offset(data,sizeof(data),&offset); if(err==k_rx_ok){ }else{ }`

See also: `rx_frame_decode_with_resync()` Public API that calls this function, `k_frame_sync_word` Sync word value (0x55AA), `k_frame_max_scan_bytes` Maximum scan window (1036 bytes)

Figure 463: Call graph for `internal_find_sync_offset`

Defined in `libs/rx_frame/src/rx_frame.c:502`

Since Version 1.1.0

—

rx_frame_encoder_init

```
rx_err_t rx_frame_encoder_init(rx_frame_encoder_t *enc)
```

Initialize a frame encoder instance.

Initialize frame encoder.

Prepares the encoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
out	enc	Encoder instance to initialize

Returns: `k_rx_ok` on successful initialization

Value	Description
<code>k_rx_err_invalid_arg</code>	if enc is nullptr
<code>k_rx_err_validation_failed</code>	if initialization verification fails

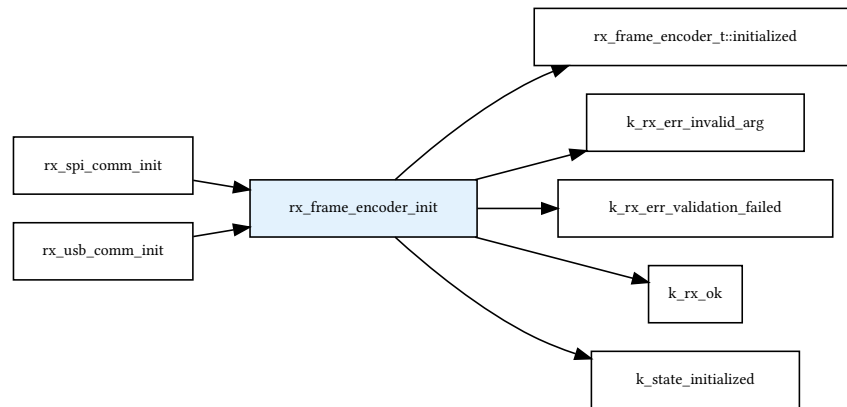


Figure 464: Call graph for rx_frame_encoder_init

Defined in `libs/rx_frame/src/rx_frame.c:549`

rx_frame_encoder_deinit

```
rx_err_t rx_frame_encoder_deinit(rx_frame_encoder_t *enc)
```

Deinitialize a frame encoder instance.

Deinitialize frame encoder.

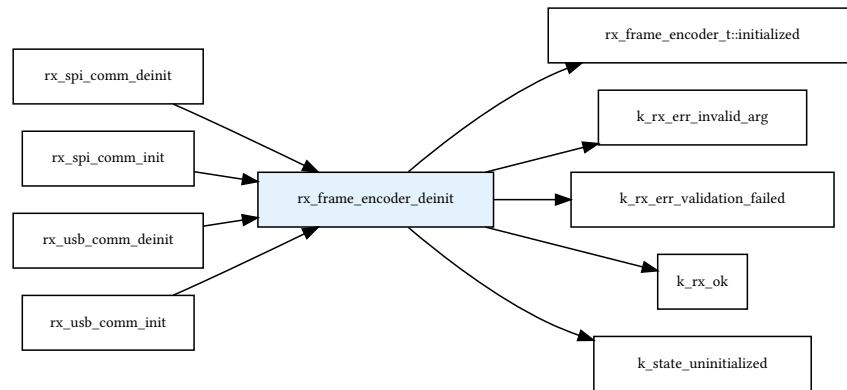
Marks the encoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
out	enc	Encoder instance to deinitialize

Returns: k_rx_ok on successful deinitialization

Value	Description
k_rx_err_invalid_arg	if enc is nullptr
k_rx_err_validation_failed	if deinitialization verification fails

Figure 465: Call graph for `rx_frame_encoder_deinit`

Defined in `libs/rx_frame/src/rx_frame.c:574`

`rx_frame_encode`

```
rx_err_t rx_frame_encode(const rx_frame_encoder_t *enc, const rx_frame_t *frame,
uint8_t *output, uint32_t *output_len)
```

Encode frame to wire format.

Serializes frame to wire format with SYNC, header, payload, and CRC-32. All multi-byte fields are little-endian (SYNC, SEQ, LEN, CRC-32).

Parameters:

Direction	Name	Description
in	enc	Encoder handle
in	frame	Frame to encode
out	output	Output buffer (min <code>k_frame_min_size</code> + payload bytes)
out	output_len	Actual number of bytes written

Returns: `k_rx_err_invalid_size` if payload exceeds max

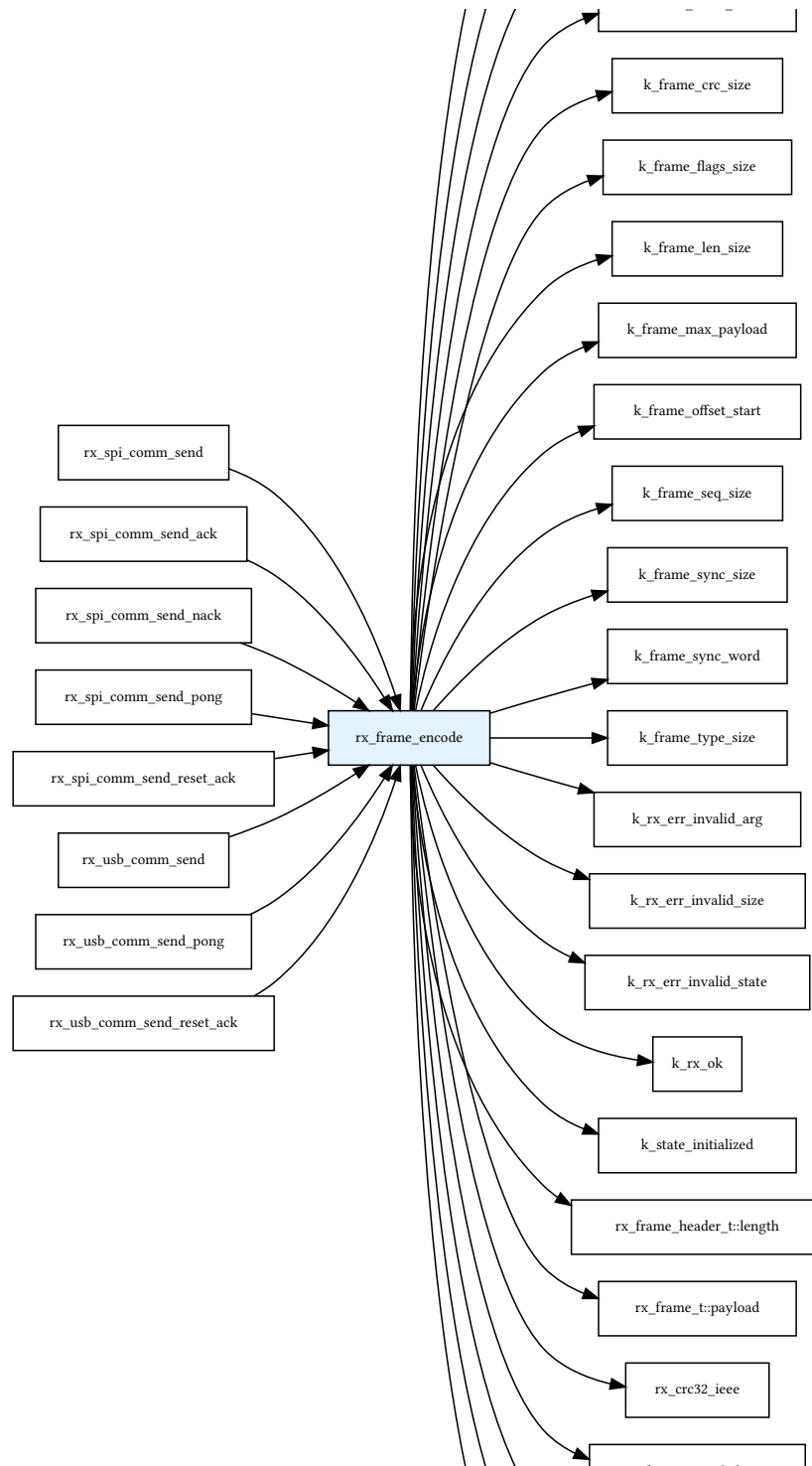


Figure 466: Call graph for `rx_frame_encode`

Defined in `libs/rx_frame/src/rx_frame.c:587`

—

rx_frame_decoder_init

```
rx_err_t rx_frame_decoder_init(rx_frame_decoder_t *dec)
```

Initialize a frame decoder instance.

Initialize frame decoder.

Prepares the decoder for use by setting the initialized flag and verifying the initialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
out	dec	Decoder instance to initialize

Returns: `k_rx_ok` on successful initialization

Value	Description
<code>k_rx_err_invalid_arg</code>	if dec is nullptr
<code>k_rx_err_validation_failed</code>	if initialization verification fails

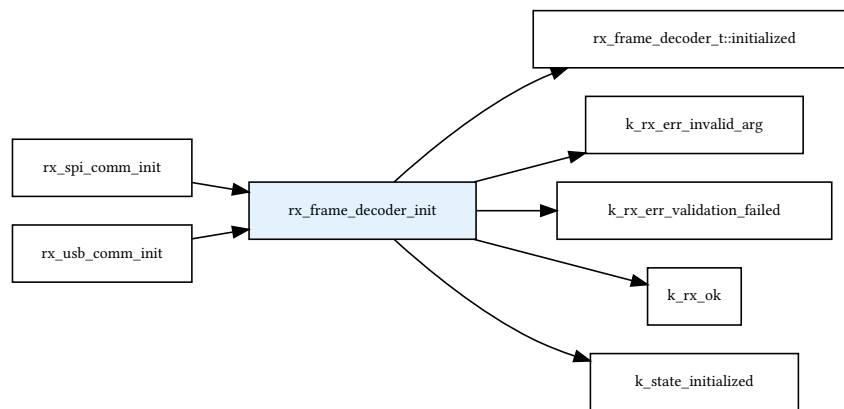


Figure 467: Call graph for `rx_frame_decoder_init`

Defined in `libs/rx_frame/src/rx_frame.c:666`

—

rx_frame_decoder_deinit

```
rx_err_t rx_frame_decoder_deinit(rx_frame_decoder_t *dec)
```

Deinitialize a frame decoder instance.

Deinitialize frame decoder.

Marks the decoder as uninitialized, preventing further use. Verifies deinitialization completed successfully (redundant validation per Rule 5).

Parameters:

Direction	Name	Description
out	dec	Decoder instance to deinitialize

Returns: k_rx_ok on successful deinitialization

Value	Description
k_rx_err_invalid_arg	if dec is nullptr
k_rx_err_validation_failed	if deinitialization verification fails

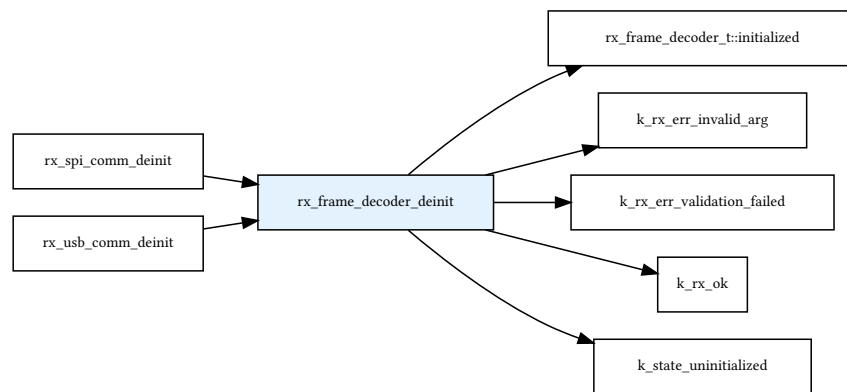


Figure 468: Call graph for `rx_frame_decoder_deinit`

Defined in `libs/rx_frame/src/rx_frame.c:691`

rx_frame_decode

```
rx_err_t rx_frame_decode(const rx_frame_decoder_t *dec, const uint8_t *data,
                        const uint32_t data_len, rx_frame_t *frame)
```

Decode a frame from raw data.

Decode wire format to frame.

Validates the decoder state, extracts the frame header, copies the payload, and verifies the CRC-32 checksum.

Parameters:

Direction	Name	Description
in	dec	Initialized frame decoder instance
in	data	Raw frame data buffer
in	data_len	Length of data buffer in bytes

out	frame	Decoded frame (header, payload, CRC)
-----	-------	--------------------------------------

Value	Description
k_rx_ok	Success - frame decoded and CRC verified
k_rx_err_invalid_arg	Any pointer parameter is nullptr or other invalid arguments
k_rx_err_invalid_state	Decoder not initialized
k_rx_err_crc	CRC-32 verification failed
k_rx_err_invalid_size	Payload exceeds maximum frame size

Note: This function performs validation at multiple points:] Null pointer checks on all parameters Decoder initialization check Header parsing via internal_decode_header() CRC verification via internal_verify_crc()

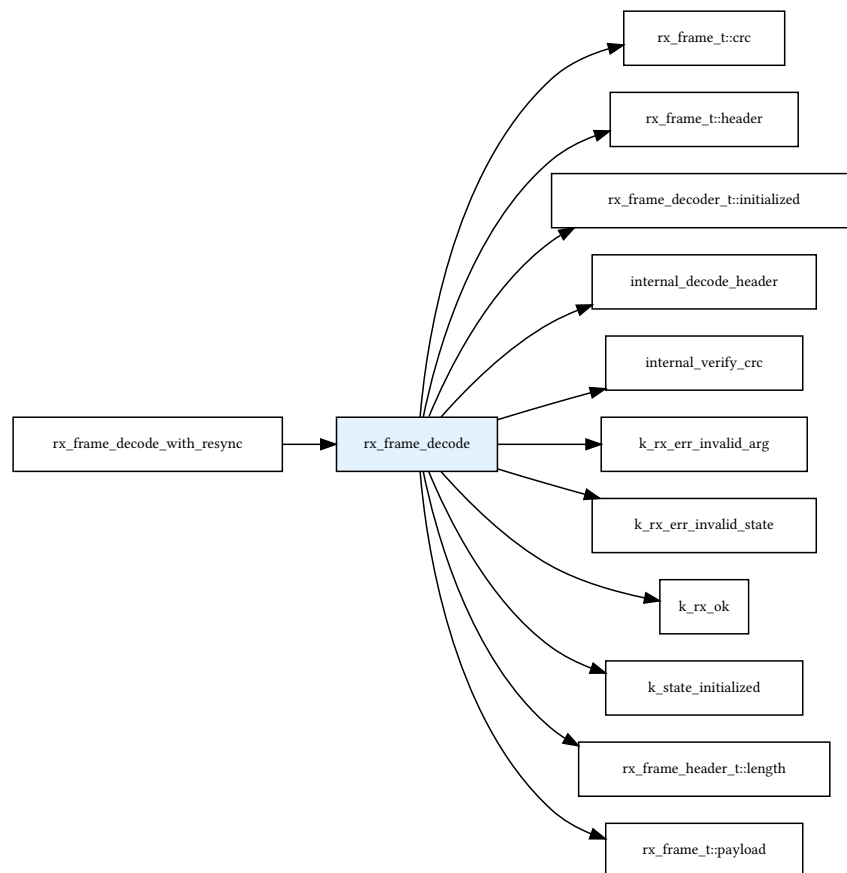


Figure 469: Call graph for `rx_frame_decode`

Defined in `libs/rx_frame/src/rx_frame.c:727`

rx_frame_decode_with_resync

```
rx_err_t rx_frame_decode_with_resync(const rx_frame_decoder_t *dec, const uint8_t
*data, const uint32_t data_len, rx_frame_t *frame, uint32_t *bytes_discarded_out)
```

Decode a frame from raw data with bounded sync-word resynchronization.

Decode wire format to frame with bounded sync-word resynchronization.

Provides stream recovery when the byte stream has become misaligned due to a dropped or inserted byte. On sync mismatch, scans forward up to `k_frame_max_scan_bytes` positions for the next occurrence of the sync word, discards the leading bytes, and reattempts decoding from that position.

This function is safe for repeated calls on a stream: the caller must advance its stream read pointer by `*bytes_discarded_out` on `k_rx_ok` and also on `k_rx_err_crc_mismatch` (any case where `bytes_discarded_out` is set) to avoid re-processing discarded bytes in subsequent calls.

Recovery algorithm:

1. Attempt `rx_frame_decode()` on `data[0..data_len-1]`.
2. If result is not `k_rx_err_protocol_error`, return immediately.
3. Call `internal_find_sync_offset()` to locate next sync word in `data[1..]`.
4. If not found, return `k_rx_err_protocol_error` with `*bytes_discarded_out = 0`.
5. Decode from `data[sync_offset..data_len-1]` using the trimmed sub-buffer.
6. Write `sync_offset` to `*bytes_discarded_out` and return result.

Parameters:

Direction	Name	Description
in	dec	Initialized frame decoder handle
in	data	Raw byte buffer (may be stream-misaligned)
in	data_len	Length of data buffer in bytes
out	frame	Decoded frame (header, payload, CRC)
out	bytes_discarded_out	Bytes skipped before sync word (0 = aligned)

Returns: `rx_err_t` `k_rx_ok` on success, or `k_rx_err_invalid_arg`, `k_rx_err_invalid_state`, `k_rx_err_invalid_size`, `k_rx_err_protocol_error`, or `k_rx_err_crc_mismatch` on failure

Value	Description
<code>k_rx_ok</code>	Frame decoded and CRC verified
<code>k_rx_err_invalid_arg</code>	Any pointer parameter is nullptr
<code>k_rx_err_invalid_state</code>	Decoder not initialized
<code>k_rx_err_invalid_size</code>	Buffer too short to contain a valid frame
<code>k_rx_err_protocol_error</code>	No sync word found within scan window
<code>k_rx_err_crc_mismatch</code>	Sync found but CRC verification failed

Pre: dec must be initialized via `rx_frame_decoder_init()`

Pre: data must point to a buffer of at least data_len bytes

Pre: frame must point to a valid rx_frame_t storage location (not NULL)

Pre: bytes_discarded_out must point to a valid uint32_t storage location (not NULL)

Post: On k_rx_ok, *bytes_discarded_out == bytes skipped to reach sync word

Post: On k_rx_ok, frame contains a fully verified decoded frame

Post: On k_rx_err_crc_mismatch, *bytes_discarded_out == bytes skipped; caller must still advance stream pointer

Note: Parameter order follows the canonical convention: decoder input -> data input -> frame output -> diagnostic output (matching rx_frame_decode()).

Note: Not thread-safe. Caller must synchronize access.

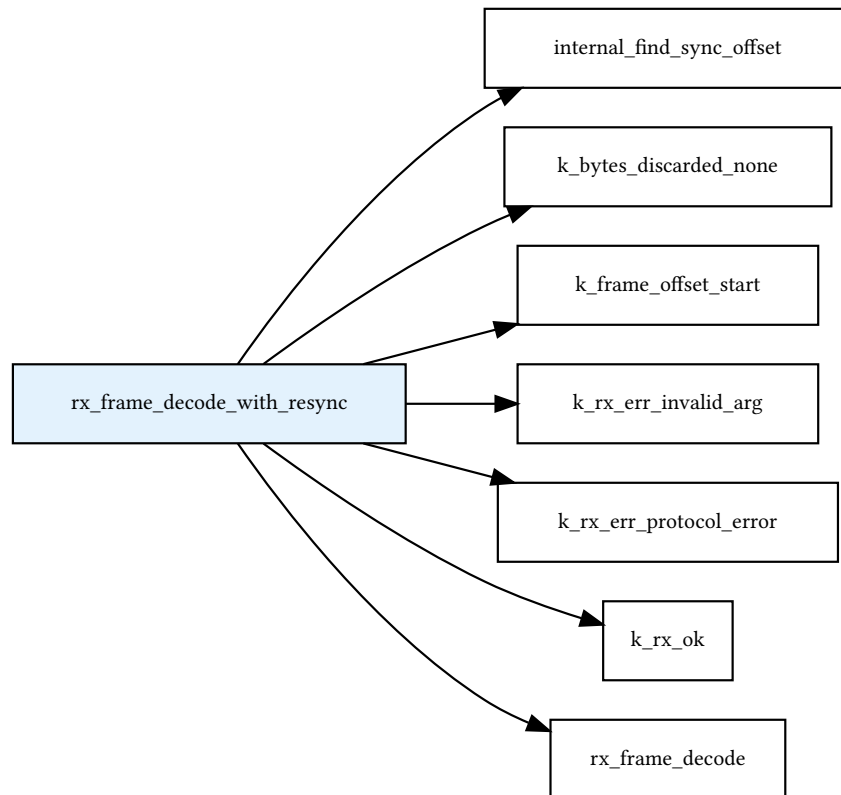
Note: The scan is bounded at k_frame_max_scan_bytes (NASA Rule 2).

Note: Log *bytes_discarded_out > 0 as a stream-health diagnostic warning.

Warning: Caller must advance stream read pointer by *bytes_discarded_out after any call that returns k_rx_ok or k_rx_err_crc_mismatch to avoid infinite re-scan; *bytes_discarded_out is unmodified only when k_rx_err_invalid_arg is returned.

Example:: uint32_tdiscarded=k_bytes_discarded_none;
 rx_err_t err=rx_frame_decode_with_resync(dec,data,data_len,&frame,&discarded); if(err==k_rx_ok)
 { stream_read_ptr+=discarded; }elseif(err==k_rx_err_crc_mismatch){ stream_read_ptr+=discarded;
 rx_log_error(TAG,"syncfoundbutCRCfailed"); }
 else{ rx_log_error(TAG,"nosyncwordwithinscanwindow"); }

See also: rx_frame_decode() Simple decode without stream recovery,
 internal_find_sync_offset() Bounded sync word scanner, k_frame_max_scan_bytes Scan
 window upper bound

Figure 470: Call graph for `rx_frame_decode_with_resync`

Defined in `libs/rx_frame/src/rx_frame.c:837`

Since Version 1.1.0

—

rx_frame_create_ack

```
rx_err_t rx_frame_create_ack(rx_frame_t *frame, const uint16_t sequence)
```

Create an ACK (acknowledgment) frame.

Create ACK frame for given sequence.

Constructs a frame with type=ACK, empty payload, and the specified sequence number. The ACK response frames are used by the receiver to confirm successful reception of a command or data frame from the sender.

Parameters:

Direction	Name	Description
out	frame	Frame structure to populate with ACK
in	sequence	Sequence number of the frame being acknowledged

Returns: k_rx_ok on successful frame creation

Value	Description
k_rx_err_invalid_arg	if frame is nullptr
k_rx_err_validation_failed	if ACK type verification fails

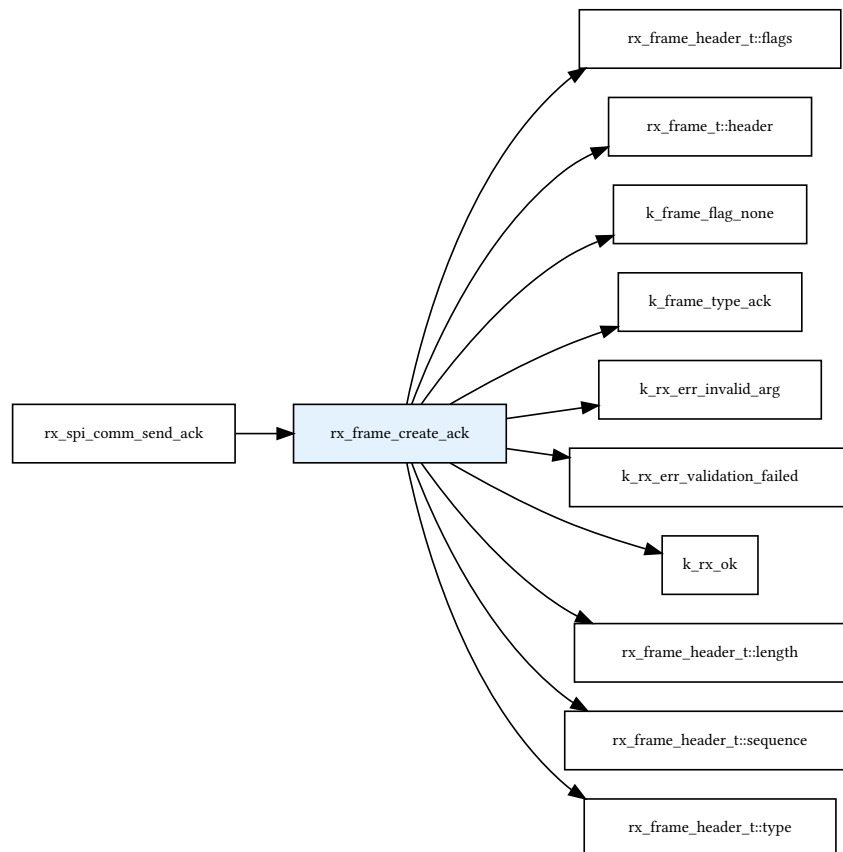


Figure 471: Call graph for `rx_frame_create_ack`

Defined in `libs/rx_frame/src/rx_frame.c:893`

rx_frame_create_nack

```
rx_err_t rx_frame_create_nack(rx_frame_t *frame, const uint16_t sequence, uint8_t flags)
```

Create a NACK (negative acknowledgment) frame.

Create NACK frame for given sequence.

Constructs a frame with type=NACK, empty payload, the specified sequence number, and error/status flags. NACK frames are sent by the receiver to indicate that a frame was not processed successfully due to an error condition (specified in flags).

Parameters:

Direction	Name	Description
out	frame	Frame structure to populate with NACK
in	sequence	Sequence number of the frame being negative-acknowledged
in	flags	Status/error flags indicating reason for NACK

Returns: k_rx_ok on successful frame creation

Value	Description
k_rx_err_invalid_arg	if frame is nullptr
k_rx_err_validation_failed	if NACK type verification fails

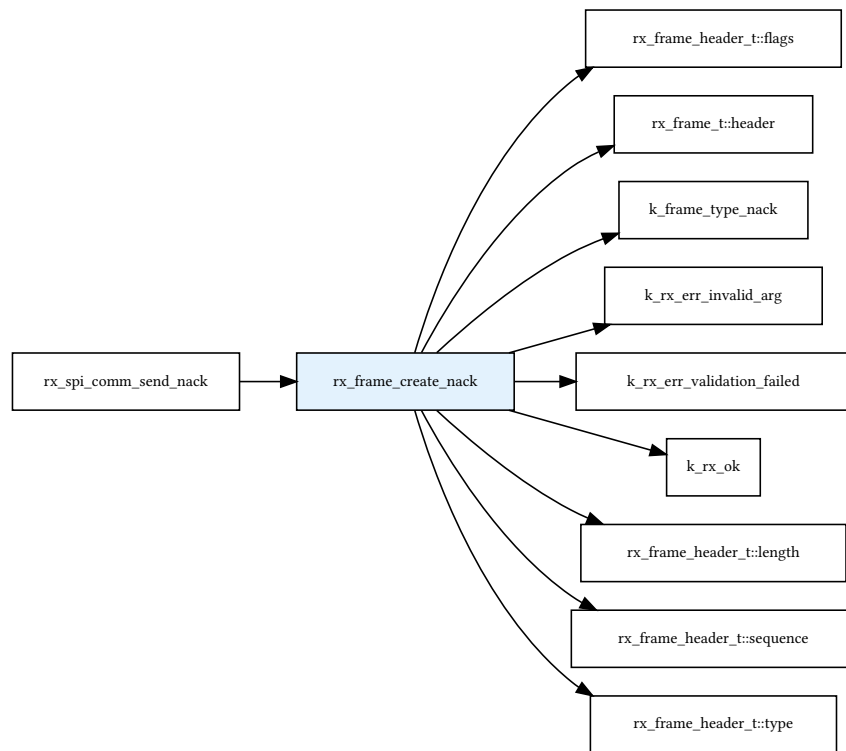


Figure 472: Call graph for rx_frame_create_nack

Defined in `libs/rx_frame/src/rx_frame.c:927`

rx_frame_create_ping

```
rx_err_t rx_frame_create_ping(rx_frame_t *frame, const uint16_t sequence, const
uint8_t *payload, const uint32_t payload_len)
```

Create a PING keepalive frame.

Create PING frame for keepalive/heartbeat.

Initializes frame as a PING request for connection liveness checks. Receiver should respond with a PONG frame echoing the payload. Payload is optional; a 4-byte LE counter is typical.

Parameters:

Direction	Name	Description
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number for this frame
in	payload	Optional payload (NULL when payload_len is 0)
in	payload_len	Payload length in bytes [0, k_frame_max_payload]

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is NULL, or payload NULL with len > 0
k_rx_err_invalid_size	payload_len exceeds k_frame_max_payload
k_rx_err_validation_failed	Post-condition type check failed

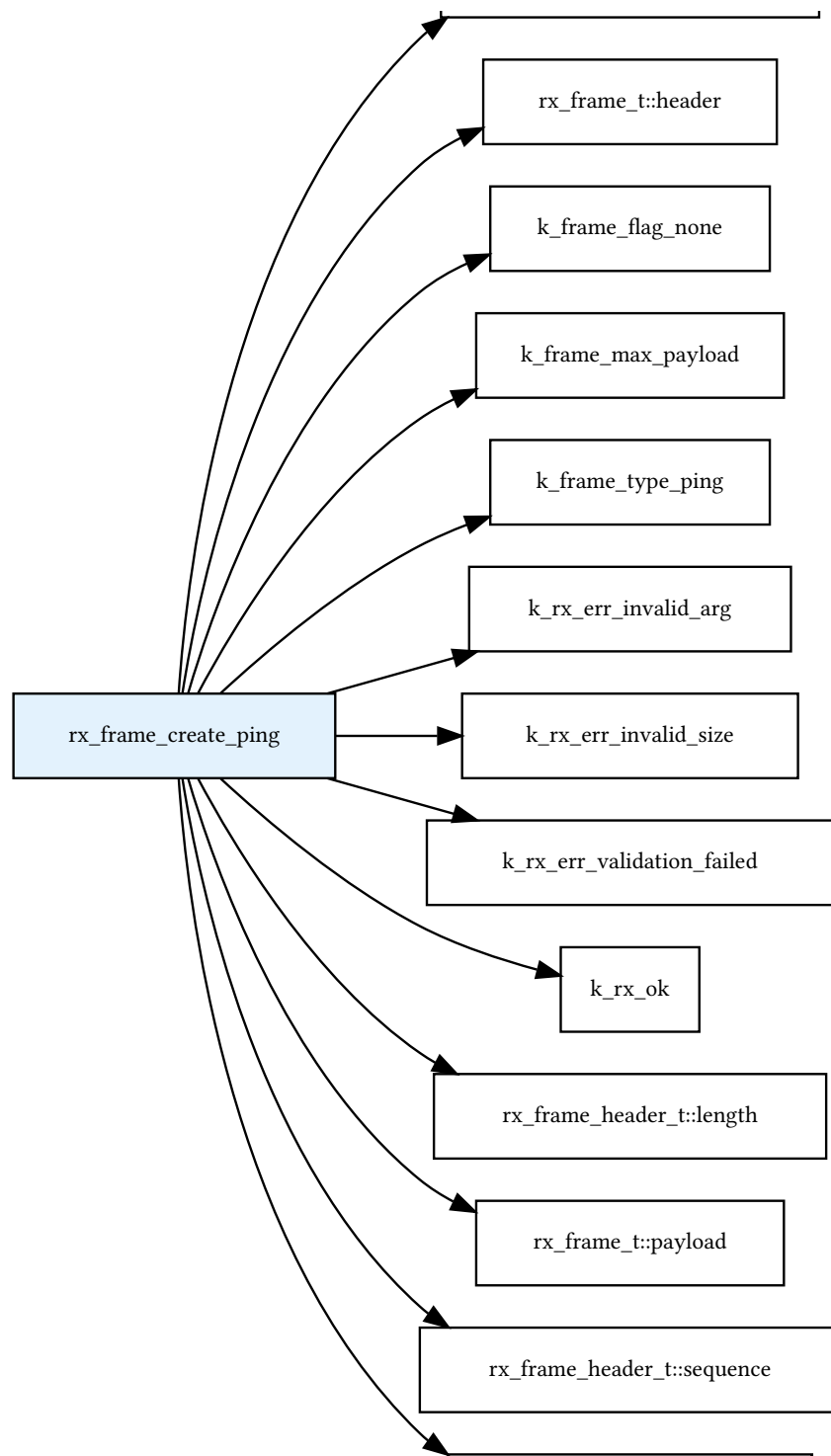
Pre: frame != nullptr

Pre: payload != NULL when payload_len > 0

Post: frame->header.type == k_frame_type_ping

Post: frame->header.length == payload_len

Note: Stateless; safe to call from any context.] **See also:** rx_frame_create_pong()
Corresponding response creator

Figure 473: Call graph for `rx_frame_create_ping`

Defined in `libs/rx_frame/src/rx_frame.c:973`

Since Version 1.0.0

—

rx_frame_create_pong

```
rx_err_t rx_frame_create_pong(rx_frame_t *frame, const uint16_t sequence, const
uint8_t *payload, const uint32_t payload_len)
```

Create a PONG response frame.

Create PONG frame (response to PING).

Initializes frame as a PONG response to a received PING. Echoes the PING payload so the sender can validate round-trip data.

Parameters:

Direction	Name	Description
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number for this frame
in	payload	Payload to echo (NULL when payload_len is 0)
in	payload_len	Payload length in bytes [0, k_frame_max_payload]

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is NULL, or payload NULL with len > 0
k_rx_err_invalid_size	payload_len exceeds k_frame_max_payload
k_rx_err_validation_failed	Post-condition type check failed

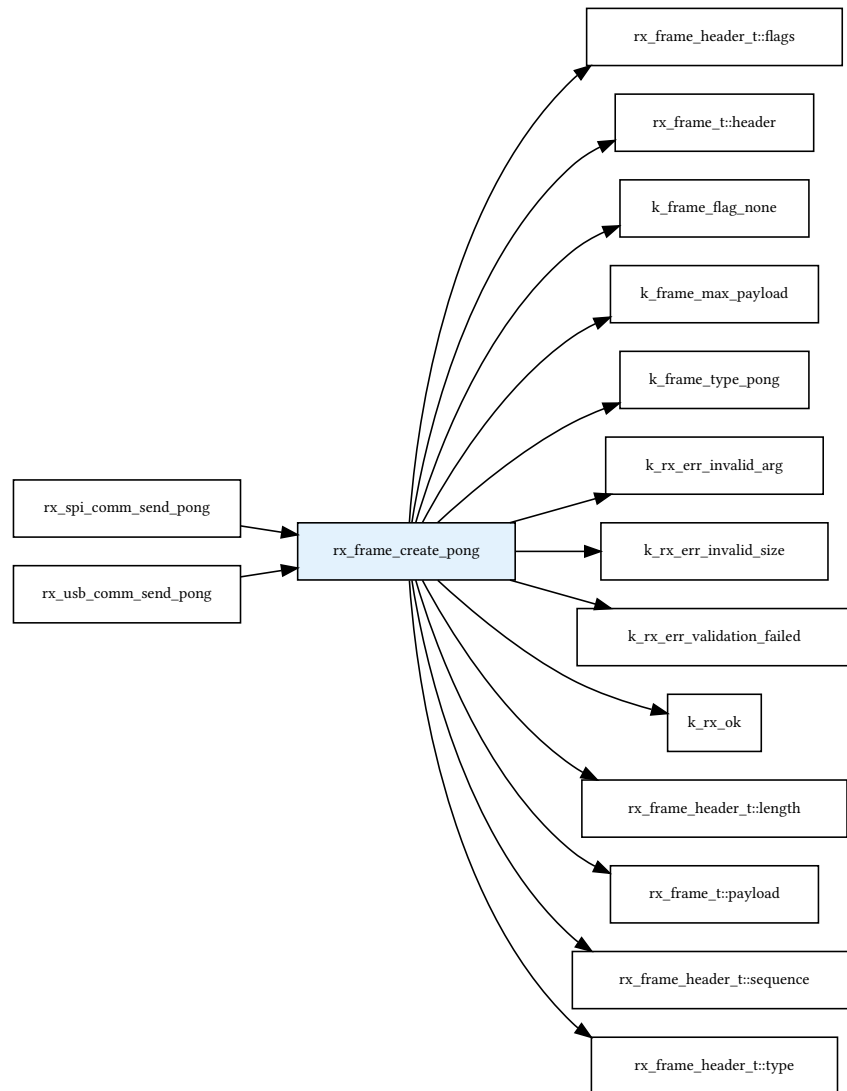
Pre: frame != nullptr

Pre: payload != NULL when payload_len > 0

Post: frame->header.type == k_frame_type_pong

Post: frame->header.length == payload_len

Note: Stateless; safe to call from any context.] **See also:** rx_frame_create_ping()
Corresponding request creator

Figure 474: Call graph for `rx_frame_create_pong`

Defined in `libs/rx_frame/src/rx_frame.c:1033`

Since Version 1.0.0

—

rx_frame_create_reset

```
rx_err_t rx_frame_create_reset(rx_frame_t *frame, const uint16_t sequence)
```

Create a RESET request frame.

Create RESET frame to reset communication state.

Initializes frame as a RESET request that asks the peer to reset communication state (sequence numbers, buffers). The receiver should respond with a RESET_ACK frame. Payload is always empty.

Parameters:

Direction	Name	Description
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number for this frame

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is nullptr
k_rx_err_validation_failed	Post-condition type check failed

Pre: frame != nullptr

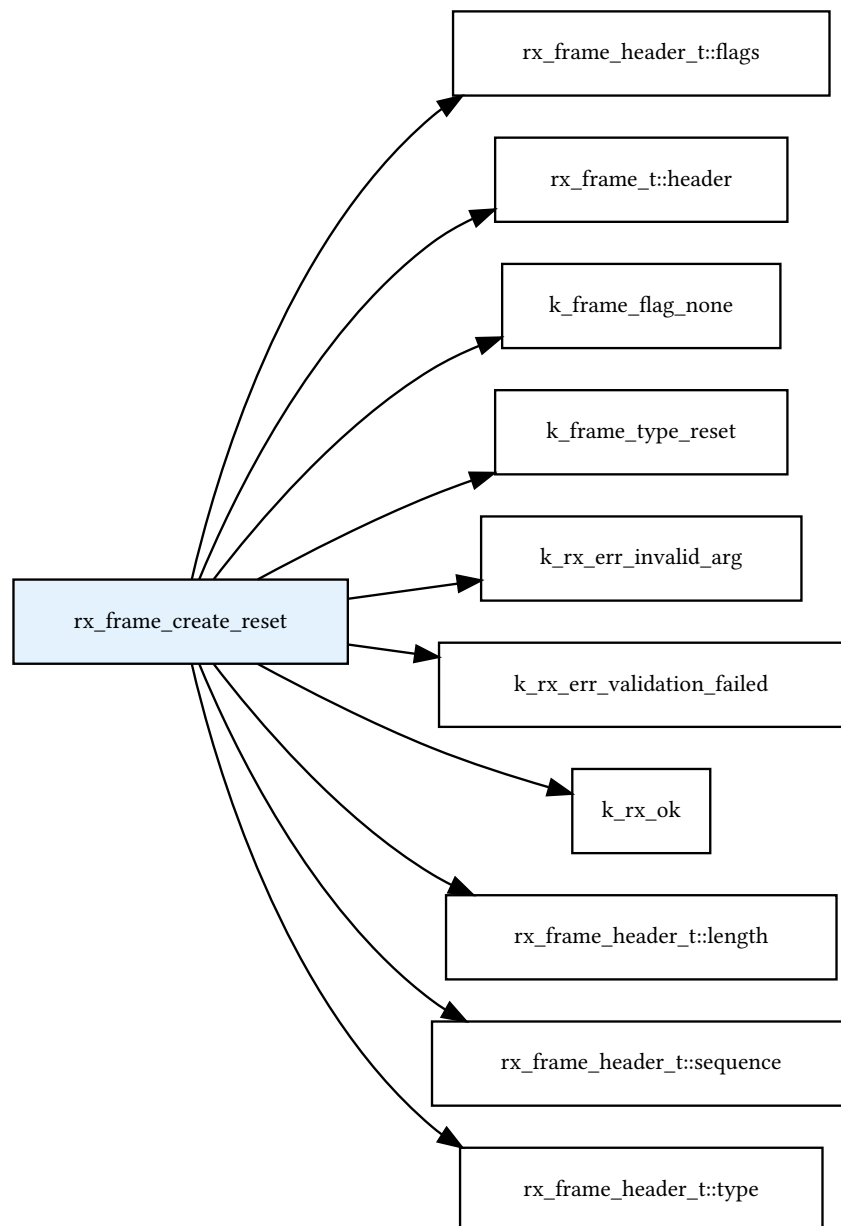
Pre: Caller has determined that a reset is necessary

Post: frame->header.type == k_frame_type_reset

Post: frame->header.length == 0

Note: Stateless; safe to call from any context.

Warning: Reset clears all sequence number state on both sides.] **See also:**
rx_frame_create_reset_ack() Corresponding acknowledgment creator

Figure 475: Call graph for `rx_frame_create_reset`

Defined in `libs/rx_frame/src/rx_frame.c:1092`

Since Version 1.0.0

—

rx_frame_create_reset_ack

```
rx_err_t rx_frame_create_reset_ack(rx_frame_t *frame, const uint16_t sequence)
```

Create a RESET_ACK confirmation frame.

Create RESET_ACK frame (response to RESET).

Initializes frame as a RESET_ACK response to a received RESET request. Confirms that the receiver has completed its reset procedure. Payload is always empty; sequence matches the RESET request.

Parameters:

Direction	Name	Description
out	frame	Frame to populate (zeroed then filled)
in	sequence	Sequence number (should match received RESET)

Value	Description
k_rx_ok	Frame created successfully
k_rx_err_invalid_arg	frame is nullptr
k_rx_err_validation_failed	Post-condition type check failed

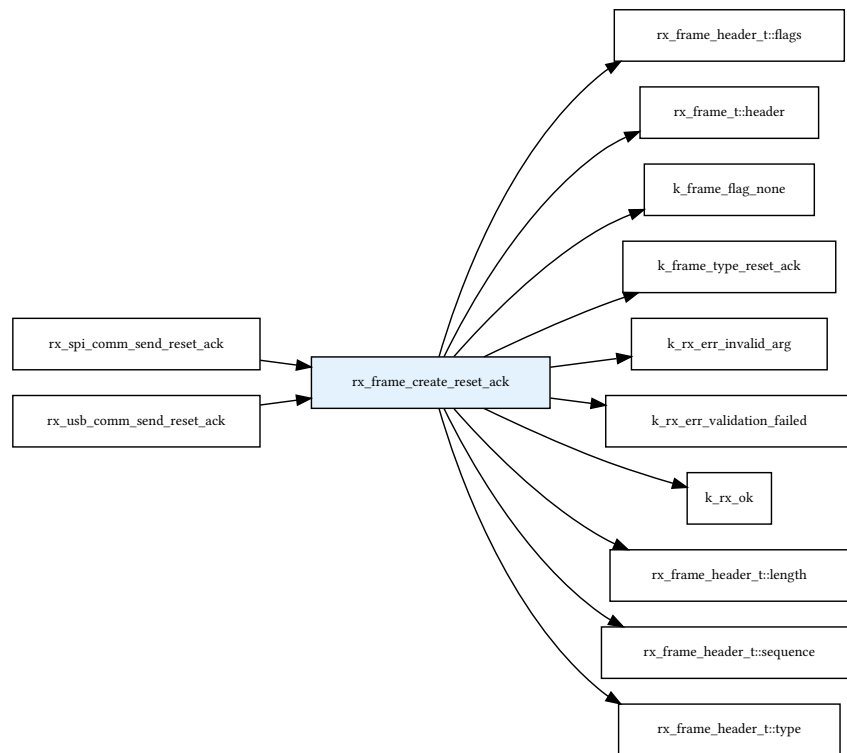
Pre: frame != nullptr

Pre: A RESET frame was received and processed

Post: frame->header.type == k_frame_type_reset_ack

Post: frame->header.length == 0

Note: Stateless; safe to call from any context.] **See also:** rx_frame_create_reset()
Corresponding request creator

Figure 476: Call graph for `rx_frame_create_reset_ack`

Defined in `libs/rx_frame/src/rx_frame.c:1135`

Since Version 1.0.0

—

rx_frame_ascii.h

Frame ASCII Formatter for Human-Readable Frame Dumps.

Converts binary protocol frames into human-readable ASCII text for debugging and diagnostic output. This module provides a debug visualization layer that transforms the binary frame protocol (sync, headers, payload, CRC) into a structured text format suitable for terminal viewing or log analysis.

STAR Team

2026-01-26

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_frame.h"`

rx_frame_ascii_constants_t

ASCII formatter configuration constants.

Compile-time constants for buffer sizing, line formatting, and output structure. These values are tuned for typical debugging scenarios with protobuf message payloads up to 2KB.

$k_frame_ascii_max_output \geq 4 * k_frame_max_payload + \text{headers}$

Value	Name	Description
= 2048	k_frame_ascii_max_output	Maximum recommended output buffer size in bytes.
= 16	k_frame_ascii_hex_per_line	Bytes per line in hex payload dump.
= 64	k_frame_ascii_header_len	Maximum header line length in characters.
= 32	k_frame_ascii_crc_len	Maximum CRC line length in characters.

See also: rx_frame_ascii_format() Uses these constants for buffer validation

Since Version 1.0.0

—

rx_frame_ascii_format

```
rx_err_t rx_frame_ascii_format(const rx_frame_t *frame, bool is_tx, char *output,
uint32_t output_max_len, uint32_t *output_len)
```

Format binary frame as human-readable ASCII.

Converts a complete frame structure into a formatted ASCII string suitable for debugging output, logging, or terminal display. The output is structured in three sections: header metadata, hex payload dump with ASCII sidebar, and CRC footer.

Output never exceeds output_max_len bytes

Format binary frame as human-readable ASCII.

Implementation of rx_frame_ascii_format(). Orchestrates header, payload, and CRC formatting using internal helper functions.

Parameters:

Direction	Name	Description
in	frame	Frame to format Must be valid non-nullptr header.length must be $\leq k_frame_max_payload$ payload data must be valid for header.length bytes
in	is_tx	Direction indicator true: Prefixes output with "[TX] " (transmit direction) false: Prefixes output with "[RX] " (receive direction)
out	output	Output buffer for ASCII text Must be valid non-nullptr Caller maintains ownership Will be null-terminated on success
in	output_max_len	Maximum bytes to write to output buffer Must be ≥ 64 (minimum for header + CRC with empty payload) Recommended: k_frame_ascii_max_output (2048) Larger payloads require proportionally larger buffers

out	output_len	Actual bytes written (excluding null terminator) Must be valid non-nullptr Set only on success
in	frame	Frame to format (must not be NULL)
in	is_tx	Direction (true=TX, false=RX)
out	output	Output buffer for formatted string
in	output_max_len	Maximum output buffer size
out	output_len	Pointer to receive actual output length

Returns: k_rx_err_no_mem if output buffer is too small

Value	Description
k_rx_ok	Frame formatted successfully
k_rx_err_invalid_arg	frame, output, or output_len is nullptr, or frame->header.length > k_frame_max_payload
k_rx_err_no_mem	Output buffer too small (< 64 bytes or truncation occurred)

Pre: frame != nullptr

Pre: output != nullptr

Pre: output_len != nullptr

Pre: output_max_len >= 64

Pre: frame->header.length <= k_frame_max_payload

Post: output is null-terminated

Post: *output_len contains actual output length (on success)

Post: output buffer contains formatted ASCII text (on success)

Note: Thread-safe: no static mutable state

Note: Reentrant: all state passed via parameters

Warning: Truncation returns k_rx_err_no_mem; output may be partial

Warning: Empty payload (length=0) produces PAYLOAD (empty) line] **Algorithm Steps:**
Validate all input pointers and buffer size Format header line: direction, sequence, length, type, flags Format payload as hex dump with 16 bytes per line + ASCII sidebar Format CRC footer with 8-digit hex value Null-terminate output and return length

Output Structure: [TX]seq=1234len=10type=COMMANDflags=ACK|PRI
[0000]48656C6C6F2055534221000000000000HelloUSB!..... [CRC]12345678

Basic Usage Example: rx_frame_tframe; charoutput[2048]; uint32_toutput_len;

```
rx_usb_comm_receive(&handle,&frame,100);
```

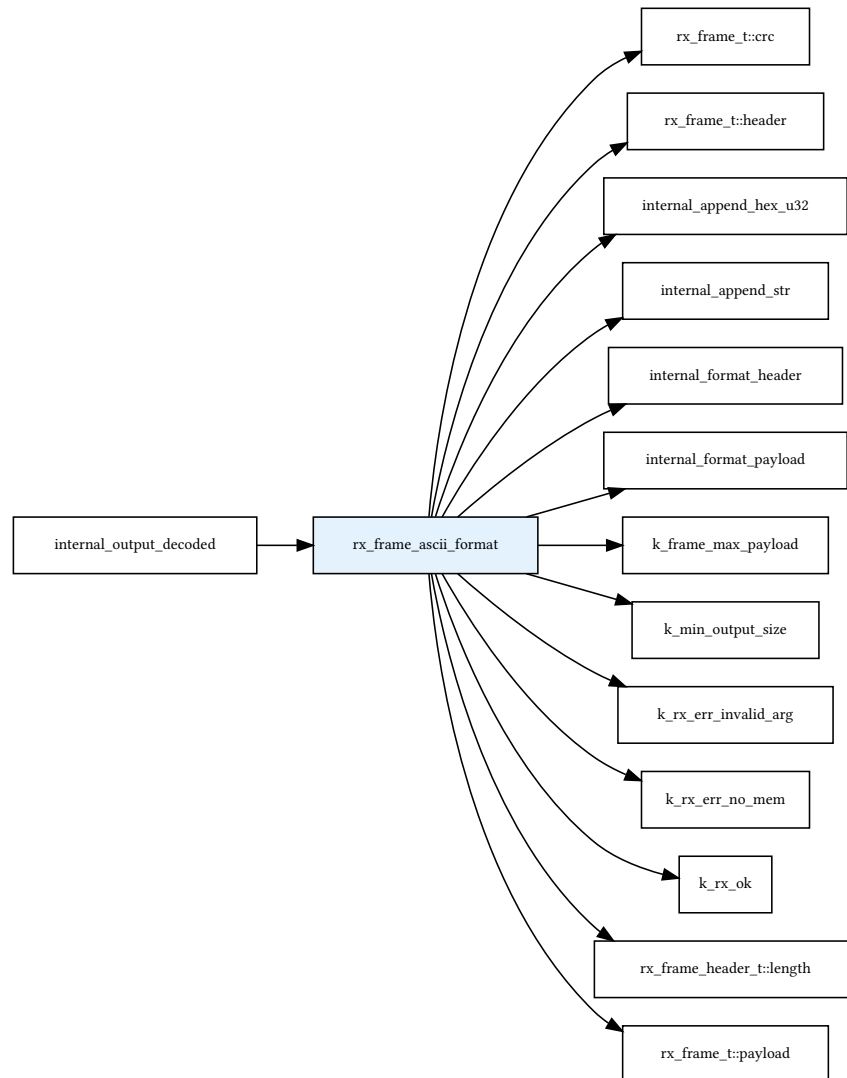
```
rx_err_terr=rx_frame_ascii_format(&frame,false,output,sizeof(output),&output_len);  
if(err!=k_rx_ok){ rx_usb_write(k_usb_port_debug,(constuint8_t*)output,output_len); }
```

Error Handling Example: rx_err_terr=rx_frame_ascii_format(&frame,true,buf,sizeof(buf),&len);
switch(err){ casek_rx_ok: break; casek_rx_err_invalid_arg:
rx_log_error("ASCII","Invalidframeorbufferpointer"); break; casek_rx_err_no_mem:
rx_log_warn("ASCII","Buffertoosmall,outputtruncated"); break; default:
rx_log_error("ASCII","Unexpectederror"); break; }

Performance: Execution time: 50 us for 64-byte payload @ 240 MHz Stack usage: 100 bytes (fixed)
No heap allocation

Implementation Notes: Validates all inputs before any output Calls internal_format_header() for metadata line Calls internal_format_payload() for hex dump Appends CRC footer directly Detects truncation and returns k_rx_err_no_mem

See also: rx_frame_ascii_type_name() Convert type enum to string,
rx_frame_ascii_flags_str() Convert flags to string, rx_frame_t Frame structure
definition, rx_frame_ascii.h for full documentation

Figure 477: Call graph for `rx_frame_ascii_format`

Defined in `libs/rx_frame_ascii/inc/rx_frame_ascii.h:359`

Since Version 1.0.0

—

rx_frame_ascii_type_name

```
const char * rx_frame_ascii_type_name(rx_frame_type_t type)
```

Get frame type as human-readable string.

Converts a frame type enumeration value to its corresponding string name for display or logging.
Returns a static string literal that does not require freeing.

Get frame type as human-readable string.

Implementation of `rx_frame_ascii_type_name()`. Uses switch statement for O(1) lookup with compile-time string literals.

Parameters:

Direction	Name	Description
in	type	Frame type enumeration value Any <code>rx_frame_type_t</code> value Unknown values return "UNKNOWN"
in	type	Frame type enum value

Returns: String representation ("PING", "PONG", "COMMAND", "RESPONSE", "ACK", "NACK", "RESET_ACK", "RESET", or "UNKNOWN")

Value	Description
PING	For <code>k_frame_type_ping</code>
PONG	For <code>k_frame_type_pong</code>
COMMAND	For <code>k_frame_type_command</code>
RESPONSE	For <code>k_frame_type_response</code>
ACK	For <code>k_frame_type_ack</code>
NACK	For <code>k_frame_type_nack</code>
RESET_ACK	For <code>k_frame_type_reset_ack</code>
RESET	For <code>k_frame_type_reset</code>
UNKNOWN	For unrecognized values

Pre: None (pure function, always valid)

Post: Return value is non-nullptr to null-terminated string

Post: Return value points to static read-only memory

Note: Thread-safe: returns pointer to static const data

Note: Reentrant: no mutable state

Note: Do NOT free the returned pointer] **Type Mapping:** Type Value String Output

`k_frame_type_ping` (0x00) "PING"

`k_frame_type_pong` (0x01) "PONG"

k_frame_type_command (0x10) "COMMAND"

k_frame_type_response (0x11) "RESPONSE"

k_frame_type_ack (0x12) "ACK"

k_frame_type_nack (0x13) "NACK"

k_frame_type_reset_ack (0xFE) "RESET_ACK"

k_frame_type_reset (0xFF) "RESET"

(any other value) "UNKNOWN"

Example: rx_frame_type_ttype=k_frame_type_command;

constchar*name=rx_frame_ascii_type_name(type);

constchar*unknown=rx_frame_ascii_type_name(rx_frame_type_t)0xFF);

Performance: Execution time: O(1) - switch statement Stack usage: 0 bytes additional

See also: rx_frame_type_t Frame type enumeration, rx_frame_ascii_format() Uses this function internally, rx_frame_ascii.h for full documentation

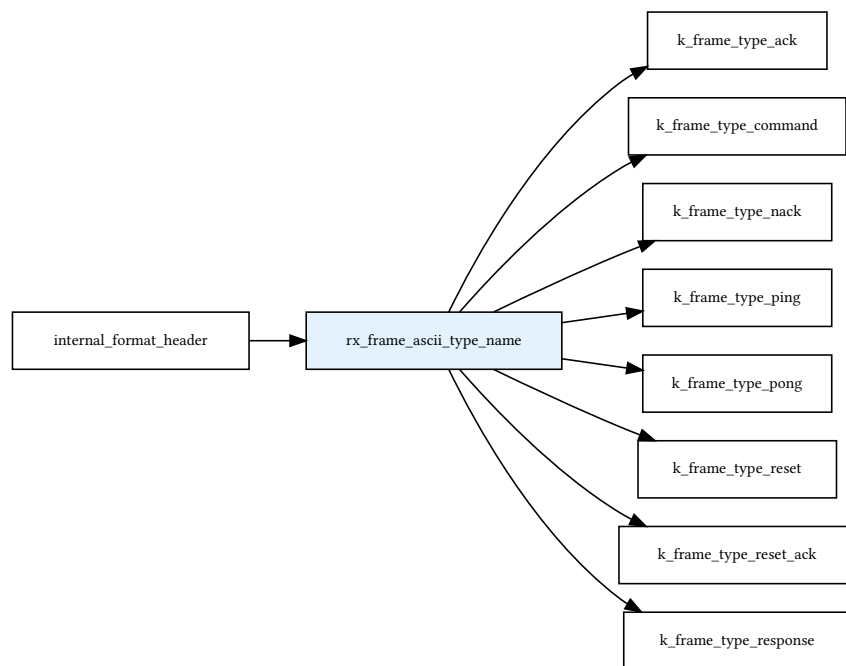


Figure 478: Call graph for rx_frame_ascii_type_name

Defined in `libs/rx_frame_ascii/inc/rx_frame_ascii.h:430`

Since Version 1.0.0

—

rx_frame_ascii_flags_str

```
const char * rx_frame_ascii_flags_str(uint8_t flags, char *buffer, uint32_t
buffer_len)
```

Get frame flags as human-readable string.

Formats a flags bitmask into a compact pipe-separated string showing all active flags. If no flags are set (value == 0), returns "NONE".

Get frame flags as human-readable string.

Implementation of rx_frame_ascii_flags_str(). Iterates through known flag bits, appending names with pipe separators.

Parameters:

Direction	Name	Description
in	flags	Frame flags bitmask Value from rx_frame_flags_t or combination thereof 0 produces "NONE"
out	buffer	Output buffer for formatted string Must be valid non-nullptr Caller maintains ownership Will be null-terminated
in	buffer_len	Size of output buffer in bytes Minimum recommended: 32 bytes Must be > 0
in	flags	Frame flags bitmask
out	buffer	Output buffer for formatted string
in	buffer_len	Size of output buffer in bytes

Returns: Pointer to buffer (or "" if buffer is nullptr)

Value	Description
buffer	On success (contains formatted string)
""	(empty string) If buffer is nullptr or buffer_len is 0

Pre: buffer != nullptr (or returns empty string)

Pre: buffer_len > 0 (or returns empty string)

Post: buffer is null-terminated (if valid)

Post: Return value == buffer (if valid)

Note: Thread-safe: output written to caller-provided buffer

Note: Reentrant: no static mutable state

Note: Flags formatted as pipe-separated: “ACK|RETX|PRI”

Warning: Buffer must be large enough for all flags + separators

Warning: Returns empty string literal “” if buffer is nullptr] **Flag Mapping:** Flag Bit String Description

k_frame_flag_requires_ack (0x01) “ACK” Acknowledgment required

k_frame_flag_retransmit (0x02) “RETX” Retransmission

k_frame_flag_priority (0x04) “PRI” High priority

k_frame_flag_fec_enabled (0x08) “FEC” FEC encoding

k_frame_flag_soft_nack (0x10) “SOFT” Soft NACK

k_frame_flag_none (0x00) “NONE” No flags set

Output Examples: Input Flags Output String

0x00 “NONE”

0x01 “ACK”

| 0x03 | “ACK|RETX” || 0x07 | “ACK|RETX|PRI” || 0x1F | “ACK|RETX|PRI|FEC|SOFT” |

Basic Usage Example: charflags_buf[32]; uint8_tflags=k_frame_flag_requires_ack|
k_frame_flag_priority;

constchar*str=rx_frame_ascii_flags_str(flags,flags_buf,sizeof(flags_buf));

Error Handling Example: constchar*empty=rx_frame_ascii_flags_str(0x01,nullptr,0);

chartiny[1]; constchar*also_empty=rx_frame_ascii_flags_str(0x01,tiny,0);

Performance: Execution time: O(n) where n = number of flag bits checked Stack usage: 10 bytes

See also: rx_frame_flags_t Flag definitions, rx_frame_ascii_format() Uses this function internally, rx_frame_ascii.h for full documentation



Figure 479: Call graph for rx_frame_ascii_flags_str

Defined in `libs/rx_frame_ascii/inc/rx_frame_ascii.h:516`

Since Version 1.0.0

—

rx_frame_ascii.c

Frame ASCII Formatter Implementation.

Implements the ASCII frame formatting functions declared in `rx_frame_ascii.h`. This module converts binary protocol frames into human-readable text for debugging and diagnostic output.

STAR Team

2026-01-26

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `"rx_frame_ascii.h"`
- `<string.h>`

format_constants_t

String formatting constants for hex and decimal conversion.

Internal constants used by the formatting helper functions. These define bit manipulation parameters, ASCII character ranges, and output formatting dimensions.

Value	Name	Description
= 4	k_hex_digit_shift	Bits to shift for upper hex nibble extraction. Value: 4 (half a byte = 4 bits)
= 0xF	k_hex_digit_mask	Mask for extracting single hex digit (4 bits). Value: 0x0F (binary: 00001111)
= 0x20	k_printable_min	Minimum printable ASCII character (space). Value: 0x20 (32 decimal) internal_is_printable()
= 0x7E	k_printable_max	Maximum printable ASCII character (tilde). Value: 0x7E (126 decimal) internal_is_printable()
= 16	k_bytes_per_line	Bytes per line in hex dump output. Matches standard xxd/hexdump format. Value: 16 bytes
= 64	k_min_output_size	Minimum output buffer size accepted. Fits header + CRC with empty payload. Value: 64 bytes
= 2	k_hex_space_gap	Spaces between hex and ASCII sections. Value: 2 spaces
= 0xFF	k_mask_u8	Mask for uint8_t extraction from larger type. Value: 0xFF (8 bits)
= 0xFFFF	k_mask_u16	Mask for uint16_t extraction from uint32_t. Value: 0xFFFF (16 bits)
= 10	k_decimal_base	Decimal number base for digit extraction. Value: 10
= 5	k_u16_decimal_max_digits	Maximum decimal digits for uint16_t (65535). Value: 5
= 32	k_flags_buf_size	Buffer size for flags string formatting. Fits "ACK RETX PRI FEC SOFT" + null. Value: 32 bytes

Since Version 1.0.0

—

buffer_size_constants_t

Buffer size and offset constants for string building.

Constants for buffer boundary calculations and hex character sizing. All values fit in uint8_t for efficient comparison operations.

Value	Name	Description
= 0	k_empty_buffer	Sentinel for empty/invalid buffer size. Value: 0
= 1	k_null_terminator	Space required for null terminator byte. Value: 1 byte
= 2	k_hex_chars_per_byte	Hex characters per byte (e.g., "FF"). Value: 2
= 4	k_hex_chars_per_u16	Hex characters for uint16_t (e.g., "FFFF"). Value: 4
= 8	k_hex_chars_per_u32	Hex characters for uint32_t (e.g., "FFFFFFFF"). Value: 8
= 16	k_u32_high_u16_shift	Bit shift to extract high uint16 from uint32. Value: 16 bits

Since Version 1.0.0

—

decimal_buf_constants_t

Decimal conversion buffer size constants.

Defines buffer sizes for temporary decimal string conversion.

Value	Name	Description
= k_u16_decimal_max_digits + 1	k_u16_decimal_buf_size	Buffer size for uint16_t decimal with null terminator (6 bytes). Calculation: 5 digits + 1 null

Since Version 1.0.0

—

s_hex_chars

```
const char s_hex_chars[]
```

Lookup table for hexadecimal digit characters.

Note: Thread-safe: read-only constant data

Since Version 1.0.0

—

rx_frame_ascii_type_name

```
const char * rx_frame_ascii_type_name(rx_frame_type_t type)
```

Convert frame type enum to human-readable string.

Get frame type as human-readable string.

Implementation of rx_frame_ascii_type_name(). Uses switch statement for O(1) lookup with compile-time string literals.

Parameters:

Direction	Name	Description
in	type	Frame type enum value

Returns: String representation (“PING”, “PONG”, “COMMAND”, “RESPONSE”, “ACK”, “NACK”, “RESET_ACK”, “RESET”, or “UNKNOWN”)

See also: rx_frame_ascii.h for full documentation

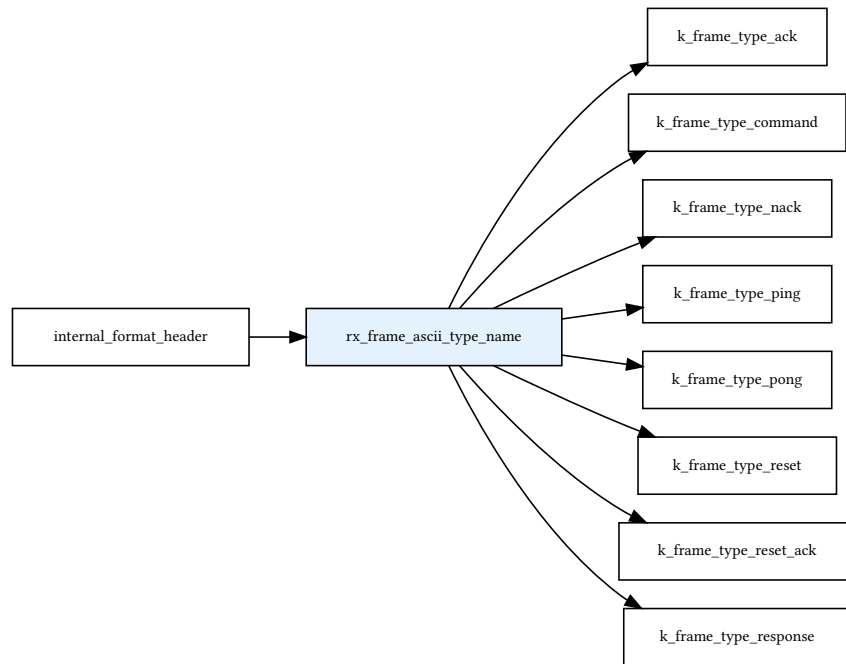


Figure 480: Call graph for rx_frame_ascii_type_name

Defined in `libs/rx_frame_ascii/src/rx_frame_ascii.c:776`

—

rx_frame_ascii_flags_str

```
const char * rx_frame_ascii_flags_str(uint8_t flags, char *buffer, uint32_t
buffer_len)
```

Convert frame flags bitmask to human-readable string.

Get frame flags as human-readable string.

Implementation of rx_frame_ascii_flags_str(). Iterates through known flag bits, appending names with pipe separators.

Parameters:

Direction	Name	Description
in	flags	Frame flags bitmask
out	buffer	Output buffer for formatted string
in	buffer_len	Size of output buffer in bytes

Returns: Pointer to buffer (or “” if buffer is nullptr)

Note: Flags formatted as pipe-separated: "ACK|RETX|PRI"] **See also:** `rx_frame_ascii.h` for full documentation

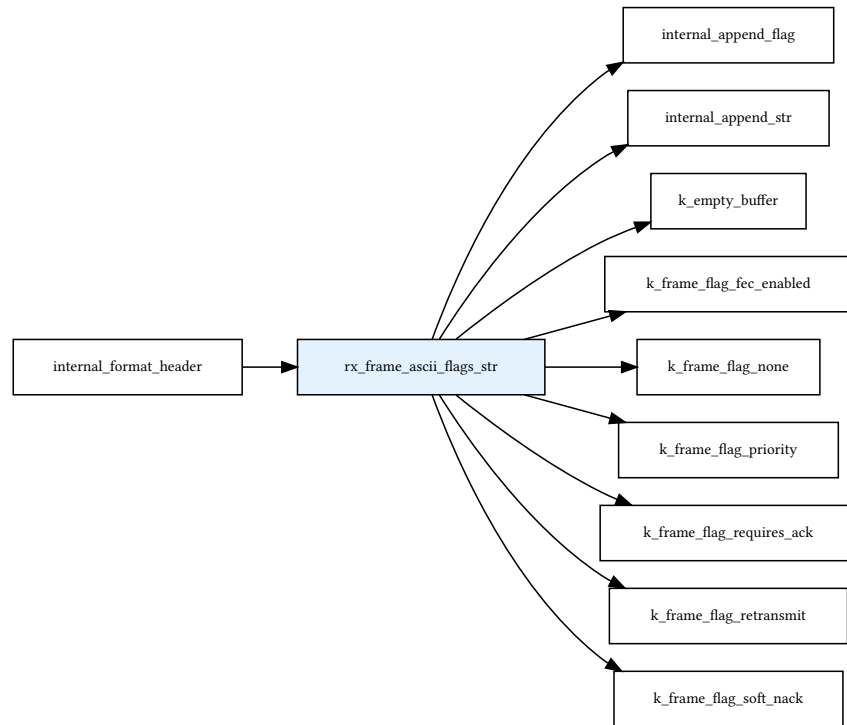


Figure 481: Call graph for `rx_frame_ascii_flags_str`

Defined in `libs/rx_frame_ascii/src/rx_frame_ascii.c:874`

`rx_frame_ascii_format`

```
rx_err_t rx_frame_ascii_format(const rx_frame_t *frame, bool is_tx, char *output,
uint32_t output_max_len, uint32_t *output_len)
```

Format frame as human-readable ASCII string.

Format binary frame as human-readable ASCII.

Implementation of `rx_frame_ascii_format()`. Orchestrates header, payload, and CRC formatting using internal helper functions.

Parameters:

Direction	Name	Description
in	<code>frame</code>	Frame to format (must not be NULL)
in	<code>is_tx</code>	Direction (true=TX, false=RX)

out	output	Output buffer for formatted string
in	output_max_len	Maximum output buffer size
out	output_len	Pointer to receive actual output length

Returns: k_rx_err_no_mem if output buffer is too small

Implementation Notes: Validates all inputs before any output Calls internal_format_header() for metadata line Calls internal_format_payload() for hex dump Appends CRC footer directly Detects truncation and returns k_rx_err_no_mem

See also: rx_frame_ascii.h for full documentation

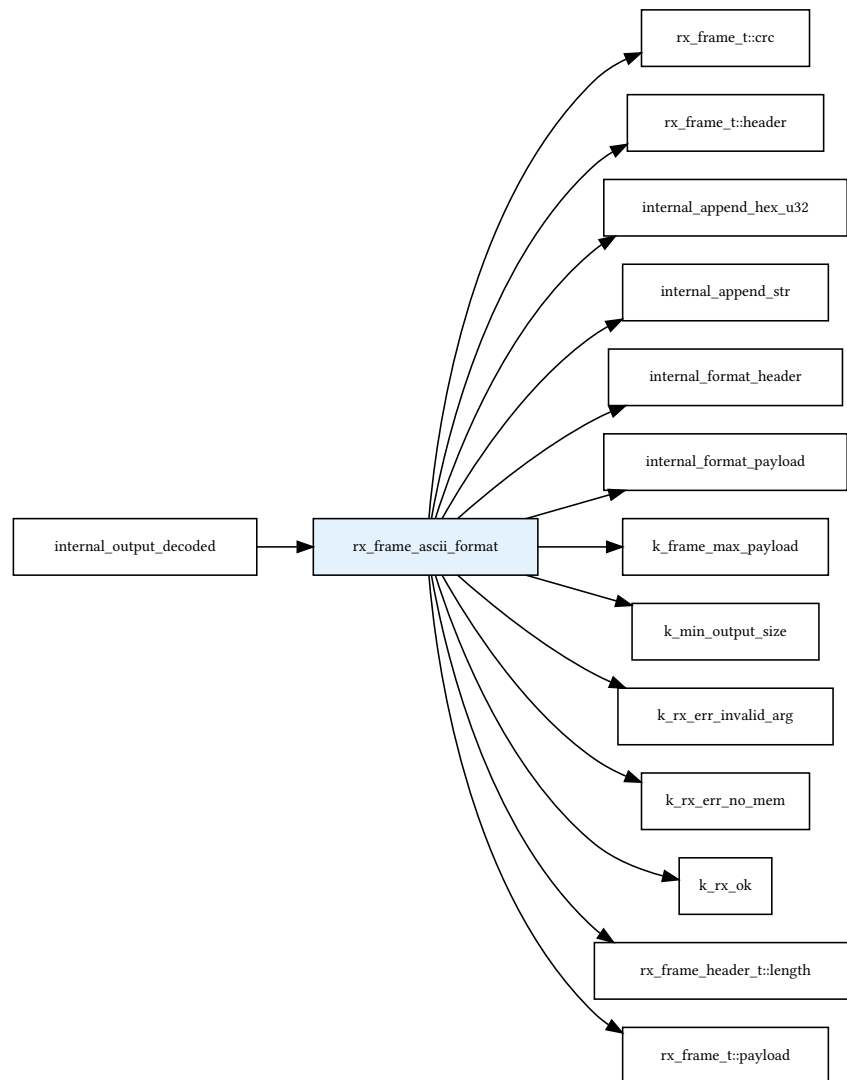


Figure 482: Call graph for rx_frame_ascii_format

Defined in `libs/rx_frame_ascii/src/rx_frame_ascii.c:931`

—

hardware.h

Hardware Abstraction Layer (HAL) - Unified Interface for RX72N Peripherals.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_check.h"`
- `"rx_err.h"`
- `"rx_infrastructure.h"`
- `"rx_log.h"`
- `"rx_port_constants.h"`
- `"rx_bus_types.h"`

adc_unit_t

ADC unit enumeration with type safety for 12-bit SAR ADC modules.

Identifies which of the two independent 12-bit successive approximation ADC units (S12AD) on the RX72N to use. Each unit has separate channels, registers, and conversion control. Using a typed enum prevents accidental swapping of unit and channel parameters.

Value	Name	Description
= 0	<code>k_adc_unit_0</code>	S12AD0 - ADC unit 0 (channels AN000-AN007). Used for motor current sensing.
= 1	<code>k_adc_unit_1</code>	S12AD1 - ADC unit 1 (channels AN100-AN120). Reserved for expansion sensors.

—

adc_channel_t

ADC channel enumeration with type safety for analog input selection.

Identifies which analog input channel (0-7) to use within an ADC unit. Each unit (S12AD0, S12AD1) has 8 independent channels with dedicated analog input pins. Using a typed enum prevents accidental parameter swaps with unit or resolution.

Value	Name	Description
= 0	<code>k_adc_channel_0</code>	Channel 0 - AN000 (unit 0) or AN100 (unit 1). Available.
= 1	<code>k_adc_channel_1</code>	Channel 1 - AN001 (unit 0) or AN101 (unit 1). Available.
= 2	<code>k_adc_channel_2</code>	Channel 2 - AN002 (unit 0) or AN102 (unit 1). Available.
= 3	<code>k_adc_channel_3</code>	Channel 3 - AN003 (unit 0) or AN103 (unit 1). Available.
= 4	<code>k_adc_channel_4</code>	Channel 4 - AN004/P44 (unit 0, Motor 3 current) or AN104 (unit 1)
= 5	<code>k_adc_channel_5</code>	Channel 5 - AN005/P45 (unit 0, Motor 2 current) or AN105 (unit 1)
= 6	<code>k_adc_channel_6</code>	Channel 6 - AN006/P46 (unit 0, Motor 1 current) or AN106 (unit 1)
= 7	<code>k_adc_channel_7</code>	Channel 7 - AN007/P47 (unit 0, Motor 0 current) or AN107 (unit 1)

rspi_mode_t

SPI mode enumeration defining clock polarity (CPOL) and phase (CPHA) combinations.

Defines the four standard SPI modes based on clock polarity and phase settings. These modes determine when data is sampled relative to the clock edge and what the idle state of the clock line is.

Value	Name	Description
= 0	k_rspi_mode_0	CPOL=0, CPHA=0 - Data sampled on rising edge, idle low. Most common mode.
= 1	k_rspi_mode_1	CPOL=0, CPHA=1 - Data sampled on falling edge, idle low. Used by SD cards.
= 2	k_rspi_mode_2	CPOL=1, CPHA=0 - Data sampled on falling edge, idle high. Rarely used.
= 3	k_rspi_mode_3	CPOL=1, CPHA=1 - Data sampled on rising edge, idle high. Alternative for some devices.

rspi_channel_t

RSPI channel enumeration with type safety for SPI peripheral selection.

Identifies which of the three RSPI channels (RSPI0, RSPI1, RSPI2) on the RX72N to use. Using a typed enum instead of plain uint8_t prevents accidental argument swaps and provides compile-time validation of channel numbers.

Value	Name	Description
= 0	k_rspi_channel_0	RSPI0 - RPi5 communication (peripheral mode in STAR project)
= 1	k_rspi_channel_1	RSPI1 - External sensor SPI (controller mode, reserved)
= 2	k_rspi_channel_2	RSPI2 - Unused (available for expansion)

rspi_freq_constants_t

Common SPI clock frequency constants for controller mode.

Named constants for commonly used SPI clock frequencies. Use these instead of magic literals when configuring controller-mode SPI clock rate. The RSPI bit rate register (SPBR) is calculated from these values and the peripheral clock (PCLKB).

All values must be positive and expressible by the SPBR register

Value	Name	Description
= 1000000	k_rspi_freq_1mhz	1 MHz SPI clock frequency
= 100000000	k_rspi_freq_10mhz	10 MHz SPI clock frequency (RPi5 link)

See also: `rspi_init_controller()` Uses `freq_hz` to calculate SPBR

Since Version 1.0.0

—

uart_defaults_t

Default debug UART channel (SCI9 - CY7C65213 USB bridge).

Value	Name	Description
= 9	k_uart_debug_channel	Debug UART on SCI9 (PB7/TXD9, PB6/RXD9)

—

uart_channel_t

UART channel enumeration with type safety.

Prevents accidental argument swaps and provides compile-time type checking. RX72N has SCI channels 0-12 (13 channels total).

Value	Name	Description
= 0	k_uart_channel_0	
= 1	k_uart_channel_1	
= 2	k_uart_channel_2	
= 3	k_uart_channel_3	
= 4	k_uart_channel_4	
= 5	k_uart_channel_5	
= 6	k_uart_channel_6	
= 7	k_uart_channel_7	
= 8	k_uart_channel_8	
= 9	k_uart_channel_9	
= 10	k_uart_channel_10	
= 11	k_uart_channel_11	
= 12	k_uart_channel_12	

—

adc_resolution_t

```
typedef rx_adc_resolution_t adc_resolution_t
```

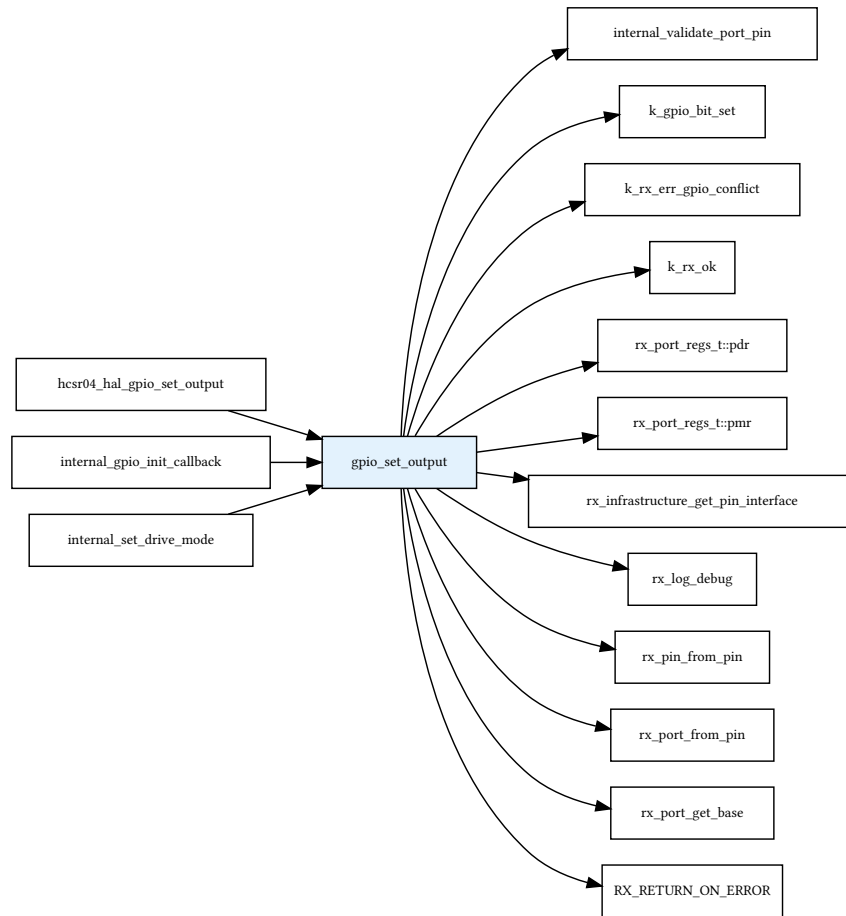
—

gpio_set_output

```
rx_err_t gpio_set_output(rx_port_pin_t pin)
```

Configure GPIO pin as digital output.

Configures the specified GPIO pin as a digital output with push-pull drive. Performs full hardware initialization including port direction register (PDR), port mode register (PMR), and open-drain control (ODR). Validates pin assignment and checks for peripheral conflicts.

Figure 483: Call graph for `gpio_set_output`

Defined in `libs/rx\_hal/inc/hardware.h:389`

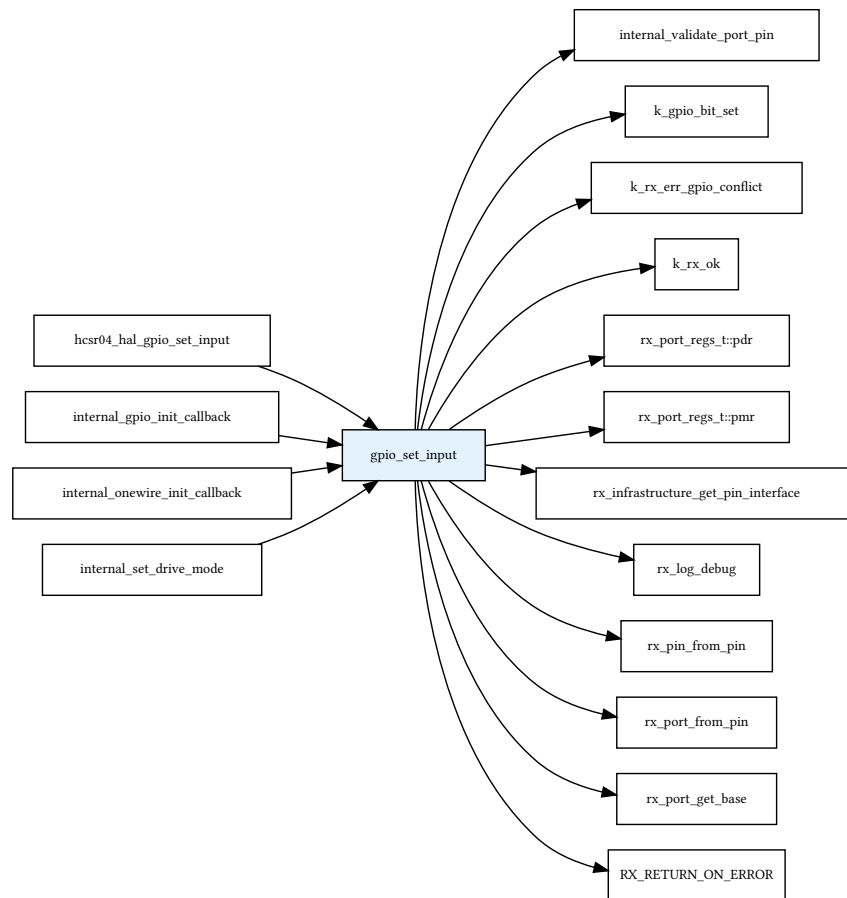
—

gpio_set_input

```
rx_err_t gpio_set_input(rx_port_pin_t pin)
```

Configure GPIO pin as digital input.

Configures the specified GPIO pin as a digital input with high-impedance state. Performs full hardware initialization including port direction register (PDR), port mode register (PMR), and input buffer control. Validates pin assignment and checks for peripheral conflicts.

Figure 484: Call graph for `gpio_set_input`

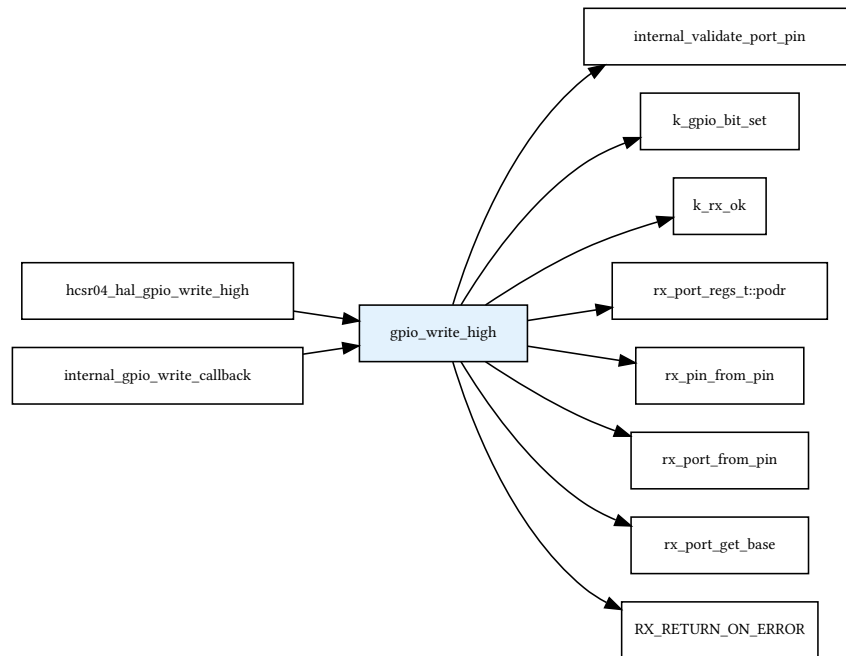
_Defined in `libs/rx\_hal/inc/hardware.h:448_`

gpio_write_high

```
rx_err_t gpio_write_high(rx_port_pin_t pin)
```

Set GPIO output pin to logic high (VDD).

Drives the specified GPIO pin to logic high by setting the corresponding bit in the Port Output Data Register (PODR). The pin must have been previously configured as output via `gpio_set_output()`. Writing to an input pin has no effect on the physical pin state.

Figure 485: Call graph for `gpio_write_high`

_Defined in `libs/rx\_hal/inc/hardware.h:497_`

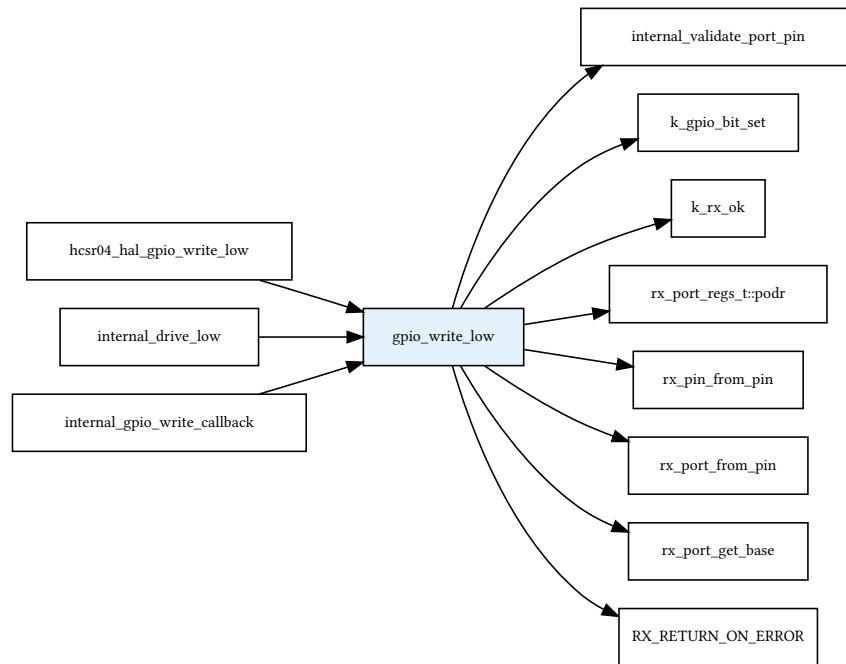
—

gpio_write_low

```
rx_err_t gpio_write_low(rx_port_pin_t pin)
```

Set GPIO output pin to logic low (GND).

Drives the specified GPIO pin to logic low by clearing the corresponding bit in the Port Output Data Register (PODR). The pin must have been previously configured as output via `gpio_set_output()`. Writing to an input pin has no effect on the physical pin state.

Figure 486: Call graph for `gpio_write_low`

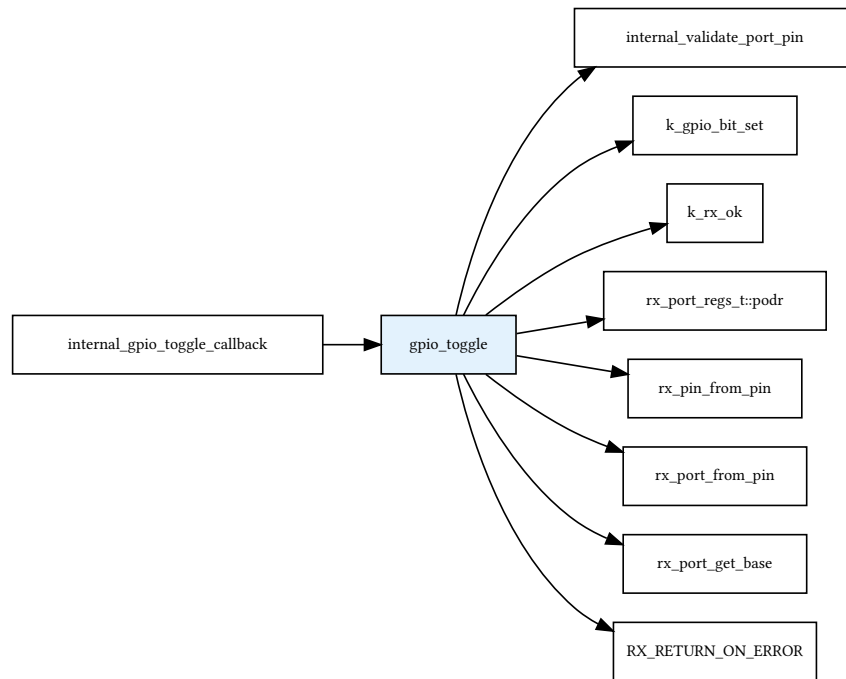
_Defined in `libs/rx\_hal/inc/hardware.h:546_`

gpio_toggle

```
rx_err_t gpio_toggle(rx_port_pin_t pin)
```

Toggle GPIO output pin state (high to low, or low to high).

Inverts the current output level of the specified GPIO pin by XOR-ing the corresponding bit in the Port Output Data Register (PODR). If the pin is currently high it becomes low, and vice versa. The pin must have been previously configured as output via `gpio_set_output()`.

Figure 487: Call graph for `gpio_toggle`

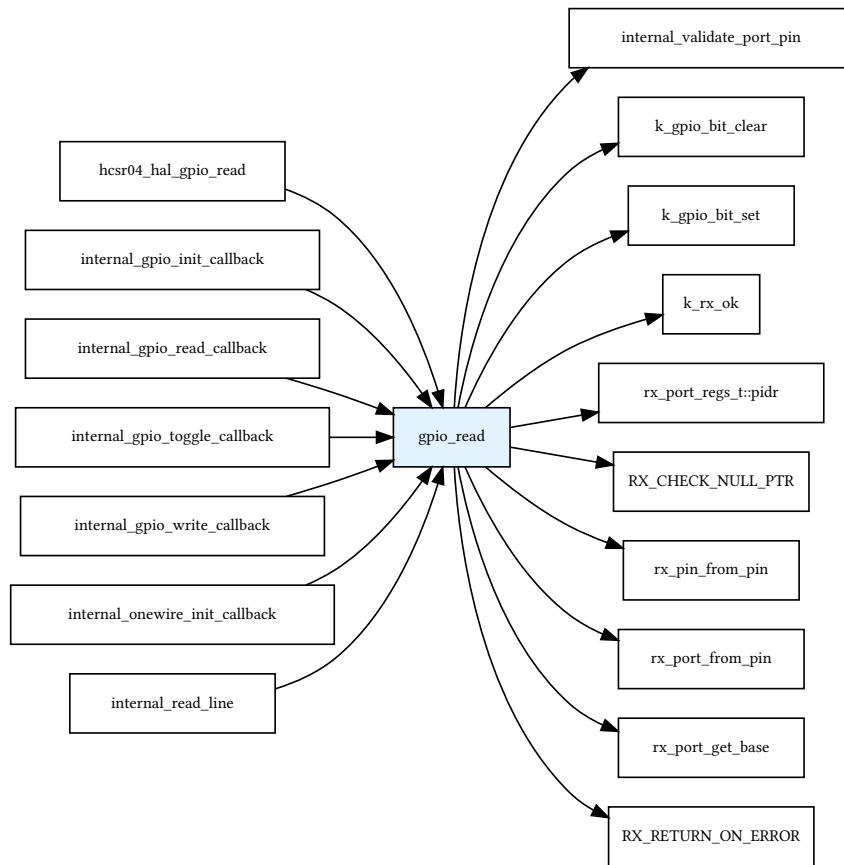
_Defined in `libs/rx\_hal/inc/hardware.h:598_`

gpio_read

```
rx_err_t gpio_read(rx_port_pin_t pin, bool *value)
```

Read current logic level of GPIO pin.

Reads the current logic level from the specified GPIO pin's Port Input Data Register (PIDR). Works for both input and output pins. For output pins, reads back the actual pin state (not the output register), which may differ if external circuitry pulls the pin.

Figure 488: Call graph for `gpio_read`

_Defined in `libs/rx\_hal/inc/hardware.h:708_`

timer_init

```
rx_err_t timer_init(void)
```

Initialize CMT0 for ThreadX system tick (100 Hz).

Configures Compare Match Timer channel 0 (CMT0) to generate periodic interrupts at 100 Hz (10 ms interval) for the ThreadX RTOS system tick. Performs full hardware initialization including module stop release, clock source selection (PCLKB/8), compare match constant calculation, and interrupt priority setup.

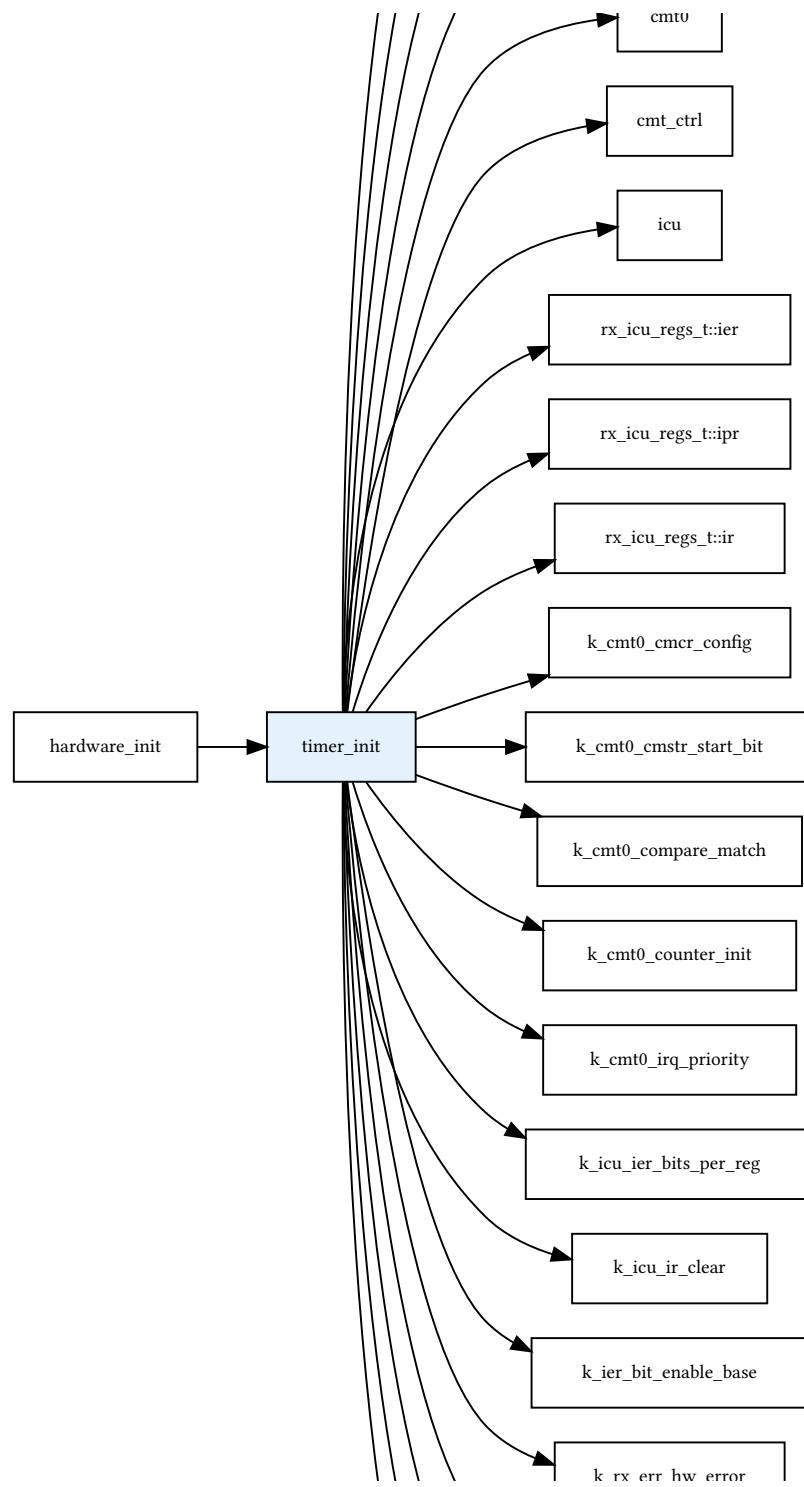


Figure 489: Call graph for timer_init

_Defined in libs/rx_hal/inc/hardware.h:763_

—

timer_stop

```
rx_err_t timer_stop(void)
```

Stop CMT0 timer and disable compare match interrupts.

Halts the CMT0 counter by clearing the start bit (CMSTR0.STR0 = 0) and disables the compare match interrupt. The counter value is preserved and can be read via timer_get_count(). After stopping, timer_init() may be called to restart the timer.

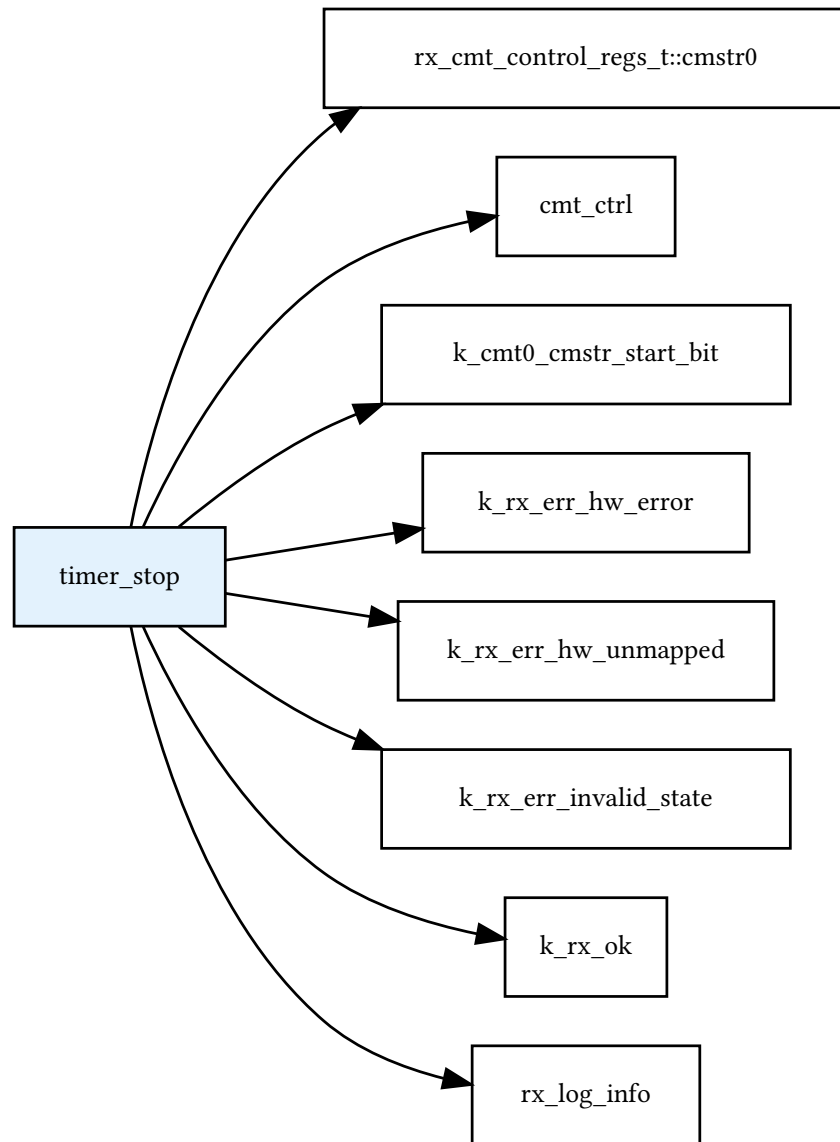


Figure 490: Call graph for timer_stop

_Defined in libs/rx_hal/inc/hardware.h:810_

—

timer_get_count

```
rx_err_t timer_get_count(uint16_t *count)
```

Get current CMT0 counter value.

Reads the 16-bit free-running counter register (CMCNT) from CMT0. The counter increments at PCLKB/8 rate and resets to 0 on compare match with CMCOR. This can be used for sub-tick timing measurements or to check elapsed time within a 10 ms system tick period. Works whether the timer is running or stopped.

Get current CMT0 counter value.

Reads the current value of CMT0.CMCNT register, which increments from 0 to CMCOR (4687) at 468.75 kHz. This provides sub-millisecond timing resolution for diagnostics, performance measurement, and jitter analysis.

Algorithm steps:

1. Validate count pointer is not nullptr
2. Validate CMT0 hardware accessibility
3. Read CMCNT register (atomic 16-bit read)
4. Validate count is within expected range (0-4687)
5. Store count value to output parameter

Counter behavior:

- Increments every $1/(468.75 \text{ kHz}) = 2.133 \text{ us}$
- Resets to 0 when reaching CMCOR (4687)
- Wraps every 10 ms (one system tick)
- Can be read at any time without affecting timer operation

Timing resolution:

Maximum readable value:

Return value `k_rx_ok` guarantees `*count` in `[0, 4687]`

Parameters:

Direction	Name	Description
out	count	Pointer to store 16-bit counter value Must point to valid <code>uint16_t</code> variable Value range: 0 to CMCOR (compare match constant) Unchanged if function returns error
out	count	Pointer to <code>uint16_t</code> to store current counter value Type: <code>uint16_t*</code> Valid range: Must be non-NULL Output range: 0 to <code>k_cmt0_compare_match</code> (4687) Units: Timer counts (1 count = 2.133 us) Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr Lifetime: Caller must ensure pointer valid until return

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Counter value successfully read and stored in <code>*count</code>
<code>k_rx_err_null_ptr</code>	count pointer is nullptr
<code>k_rx_ok</code>	Success - counter value written to <code>*count</code>
<code>k_rx_err_null_ptr</code>	count parameter is nullptr
<code>k_rx_err_hw_unmapped</code>	CMT0 register inaccessible
<code>k_rx_err_hw_error</code>	Counter value exceeds CMCOR (hardware fault)

Pre: count pointer must be valid (not nullptr)

Pre: CMT0 must have been initialized via timer_init() for meaningful values

Pre: timer_init() must have been called successfully

Pre: count pointer must be valid and non-NULL

Post: *count contains current CMCNT register value if function succeeds

Post: *count unchanged if function returns error

Post: If k_rx_ok: *count contains current CMCNT value (0-4687)

Post: If error: *count unchanged (nullptr) or contains invalid value (hw_error)

Post: Timer operation not affected (read-only access)

Note: Thread Safety: NOT thread-safe. The 16-bit register read is atomic on RX72N but the pointer write is not protected.

Note: Thread Safety: Thread-safe for reading. Multiple threads can call simultaneously. CMCNT is atomic 16-bit register on RX72N. No need for mutual exclusion.

Note: Re-entrancy: Fully reentrant. Safe to call from ISRs and tasks concurrently. No shared state modified.

Note: Performance: Executes in 500 ns @ 240 MHz (single register read). Suitable for high-frequency polling.

Note: Memory: No heap allocation. Stack usage: 4 bytes (local variable).

Warning: Counter wraps: Counter resets to 0 every 10 ms. For measuring intervals longer than 10 ms, use ThreadX system time (tx_time_get) instead.

Warning: Race condition: Counter value is snapshot at time of read. By the time caller processes value, hardware counter has advanced. Don't assume static value.] **Example: Sub-Tick Timing:**
uint16_t start, end; timer_get_count(&start); timer_get_count(&end);
uint16_t elapsed_ticks = (end >= start) ? (end - start) : (0xFFFF - start + end);

Return Value Details::

Example - Basic Usage:: uint16_tcount; rx_err_t err=timer_get_count(&count); if(err==k_rx_ok) { float us=count*2.133f; rx_log_info("TIMER","Current time interval: %.1fus",us); }

Example - Performance Measurement:: uint16_t start,end; timer_get_count(&start);
motor_update();
timer_get_count(&end);
uint16_t elapsed; if(end>=start){ elapsed=end-start; }else{ elapsed=(4687-start)+end; }
float us=elapsed*2.133f; rx_log_info("PERF","motor_update took %.1fus",us);

Example - Jitter Analysis:: static uint16_t last_count=0;
void analyze_jitter(void) { uint16_t current; timer_get_count(¤t);
int16_t jitter=current-last_count; if(abs(jitter)>10){ rx_log_warn("TIMER","High jitter detected: %d counts",jitter); }
last_count=current; }

Example - Error Handling:: uint16_t count; rx_err_t err=timer_get_count(&count); switch(err) { case k_rx_ok: break; case k_rx_err_null_ptr: rx_log_error("TIMER","Invalid pointer"); return err; case k_rx_err_hw_unmapped: rx_log_error("TIMER","CMT0 not initialized"); return err; case k_rx_err_hw_error: rx_log_error("TIMER","Counter overflow: %u",count); return err; }

NASA Power of 10 Compliance: Rule 1 [OK] No goto, setjmp, recursion Rule 2 [OK] No loops Rule 3 [OK] No dynamic allocation Rule 4 [OK] Function is 13 lines (under 60 line limit) Rule 5 [OK] 2 preconditions, 3 postconditions Rule 6 [OK] Minimal scope (single output parameter) Rule 7 [OK] All return values validated (RX_CHECK_NULL_PTR) Rule 8 [OK] Uses C23 typed enums (k_cmt0_compare_match) Rule 9 [OK] Single level of pointer dereferencing Rule 10 [OK] Compiles with -Wall -Wextra -Werror

See also: timer_init() Initialize CMT0 timer, timer_stop() Stop CMT0 timer, timer_init() Initialize timer before reading, tx_time_get() For millisecond-resolution system time, RX72N User's Manual Section 23.2.5 - CMCNT Register

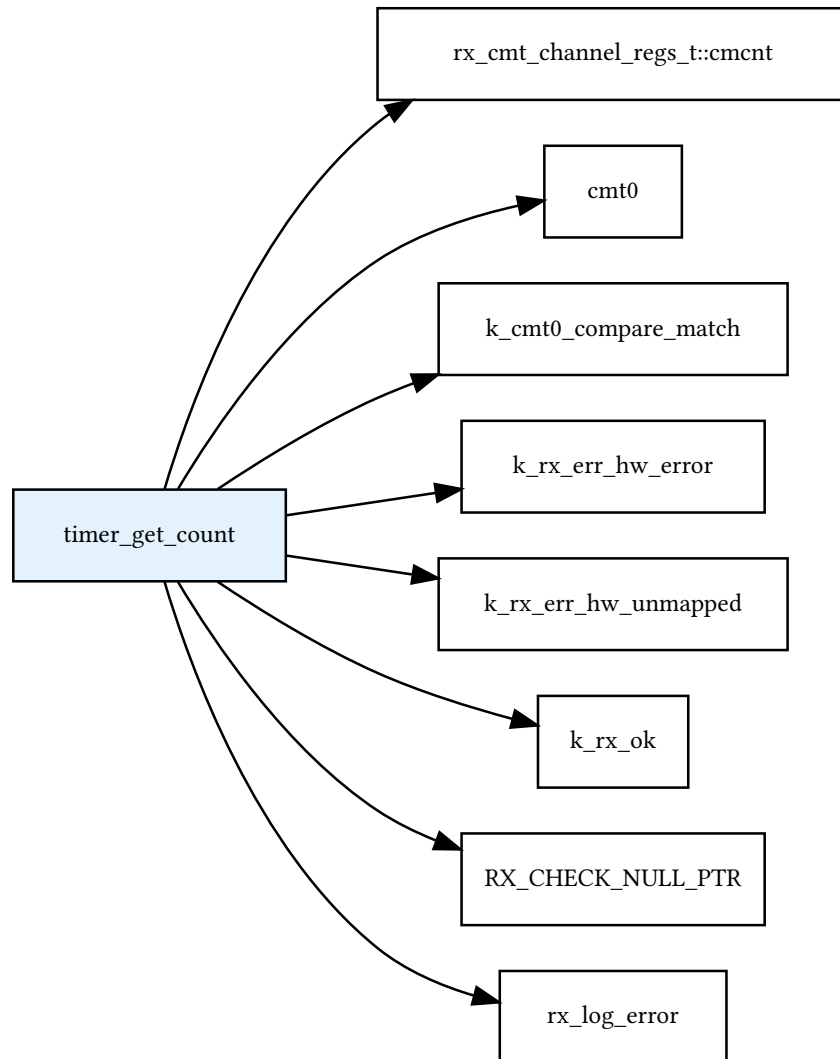


Figure 491: Call graph for timer_get_count

_Defined in `libs/rx_hal/inc/hardware.h:852_`

Since Version 1.0.0

—

adc_init

```
rx_err_t adc_init(adc_unit_t unit, adc_channel_t channel, adc_resolution_t bits)
```

Initialize ADC unit and channel.

Initialize ADC unit and channel.

Configures the S12ADFa peripheral for A/D conversion. On first call for each unit, enables the module clock, configures resolution, and initializes the hardware. Subsequent calls to the same unit enable additional channels.

Parameters:

Direction	Name	Description
in	<code>unit</code>	ADC unit (0 or 1)
in	<code>channel</code>	ADC channel (0-7)
in	<code>bits</code>	Resolution (8, 10, or 12 bits)

Returns: `k_rx_ok` on success, `k_rx_err_invalid_arg` if unit, channel, or bits is invalid, `k_rx_err_gpio_conflict` if pin already reserved

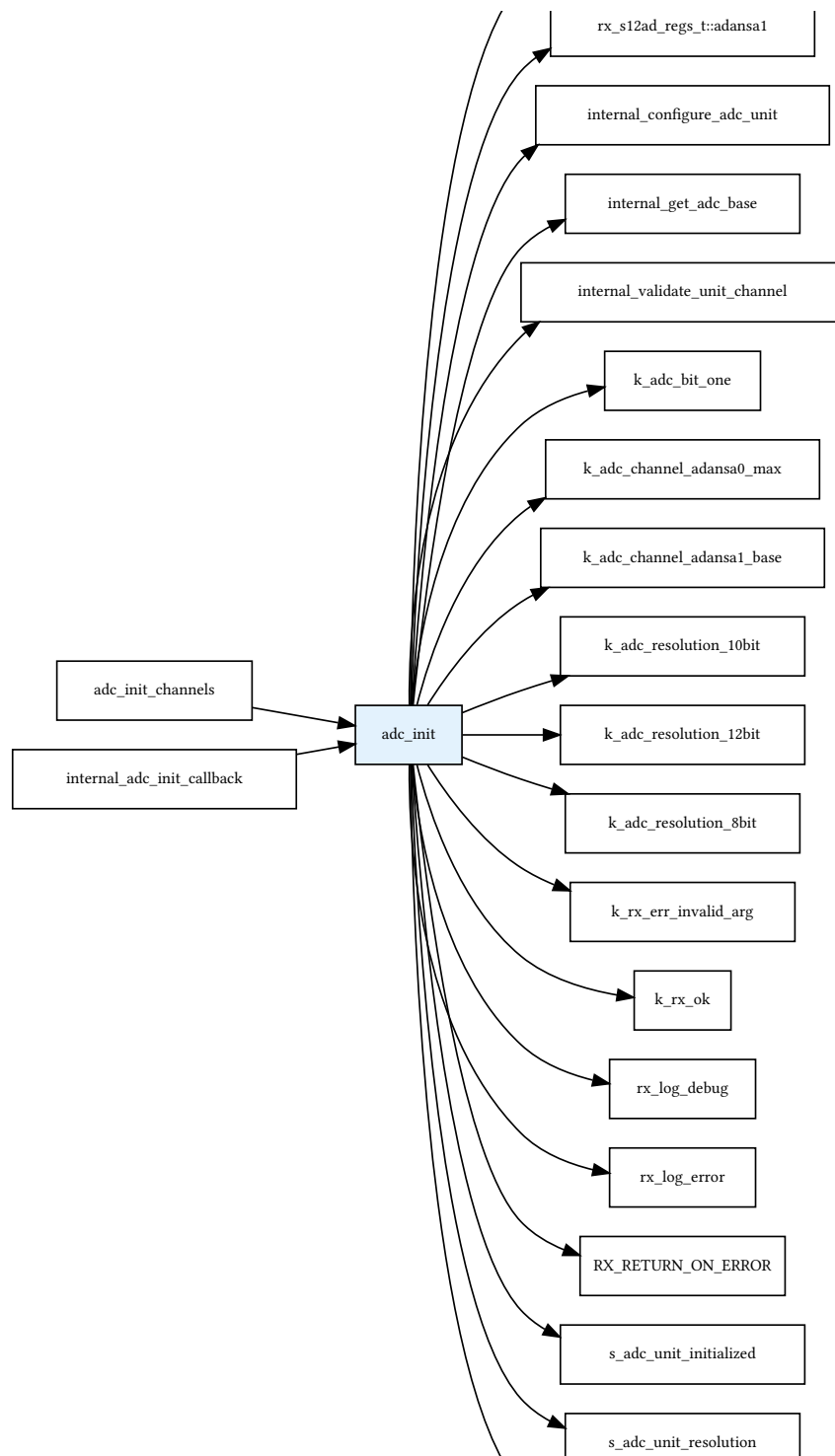


Figure 492: Call graph for `adc_init`

_Defined in libs/rx_hal/inc/hardware.h:980_

—

adc_read

```
rx_err_t adc_read(adc_unit_t unit, adc_channel_t channel, uint16_t *value)
```

Read ADC value.

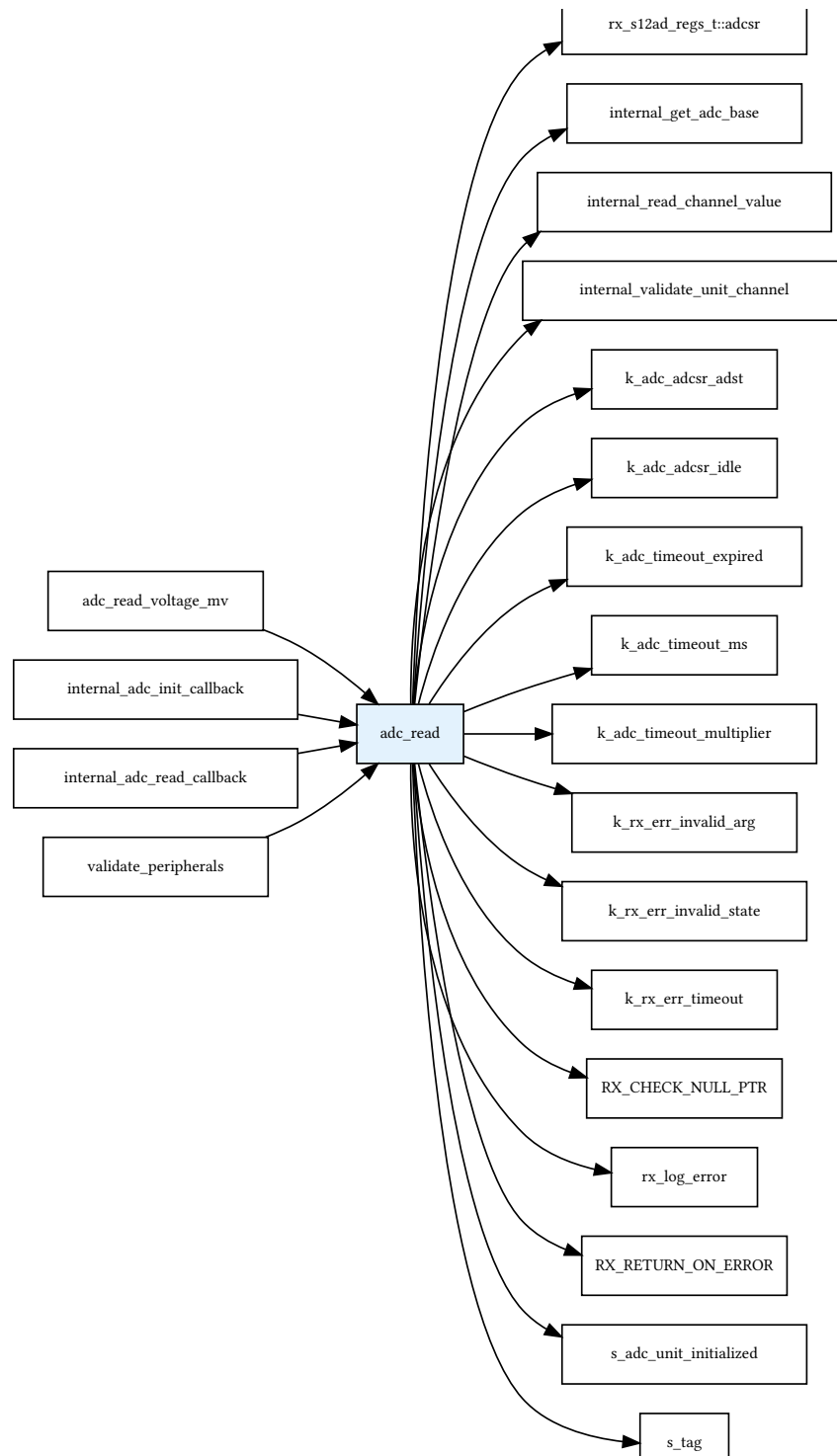
Read ADC value.

Performs a single-scan A/D conversion and returns the raw digital value. This is a blocking operation that polls the ADST bit until conversion completes or timeout occurs.

Parameters:

Direction	Name	Description
in	unit	ADC unit (0 or 1)
in	channel	ADC channel (0-7)
out	value	Pointer to store ADC value

Returns: k_rx_ok on success, k_rx_err_null_ptr if value is nullptr, k_rx_err_invalid_arg if unit or channel is invalid, k_rx_err_invalid_state if ADC unit not initialized, k_rx_err_timeout if conversion times out

Figure 493: Call graph for `adc_read`

_Defined in libs/rx_hal/inc/hardware.h:995_

—

adc_read_voltage_mv

```
rx_err_t adc_read_voltage_mv(adc_unit_t unit, adc_channel_t channel,  
adc_resolution_t bits, uint32_t *voltage_mv)
```

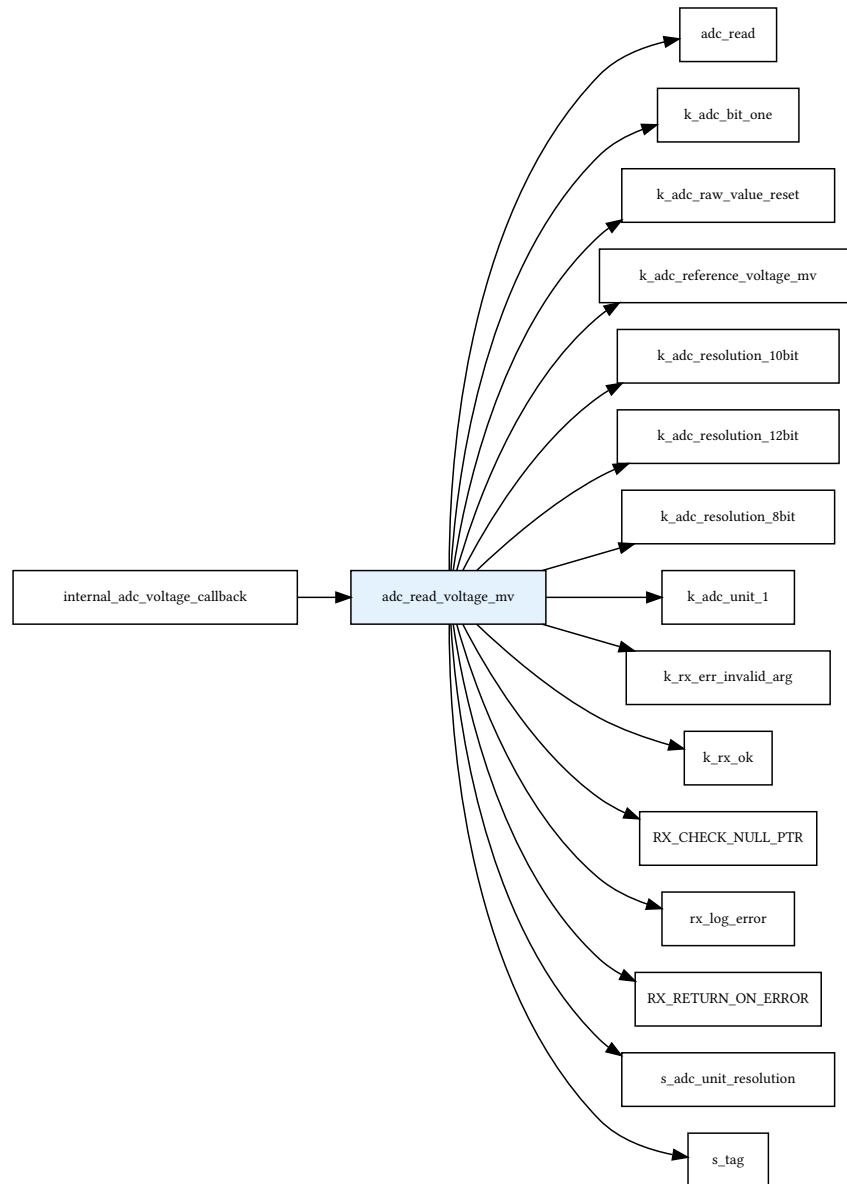
Read ADC value and convert to millivolts.

Performs an A/D conversion and scales the result to millivolts. This is a convenience function that calls `adc_read()` and applies the voltage calculation formula.

Parameters:

Direction	Name	Description
in	unit	ADC unit (0 or 1)
in	channel	ADC channel (0-7)
in	bits	Resolution used during init (8, 10, or 12)
out	voltage_mv	Pointer to store voltage in millivolts

Returns: `k_rx_ok` on success, `k_rx_err_null_ptr` if `voltage_mv` is nullptr, `k_rx_err_invalid_arg` if unit or channel is invalid, `k_rx_err_invalid_state` if ADC unit not initialized, `k_rx_err_timeout` if conversion times out

Figure 494: Call graph for `adc_read_voltage_mv`_Defined in `libs/rx\_hal/inc/hardware.h:1011_`

—

riic_init

```
rx_err_t riic_init(riic_channel_t channel, uint32_t frequency_hz)
```

Initialize RIIC channel for I2C controller communication.

Configures the specified RIIC peripheral channel for I2C controller mode at the requested clock frequency. Performs full hardware initialization including module stop release, pin function selection (SCL/SDA via MPC), internal reset sequence, bit rate register (BRH/BRL) calculation from PCLKB, and interrupt configuration.

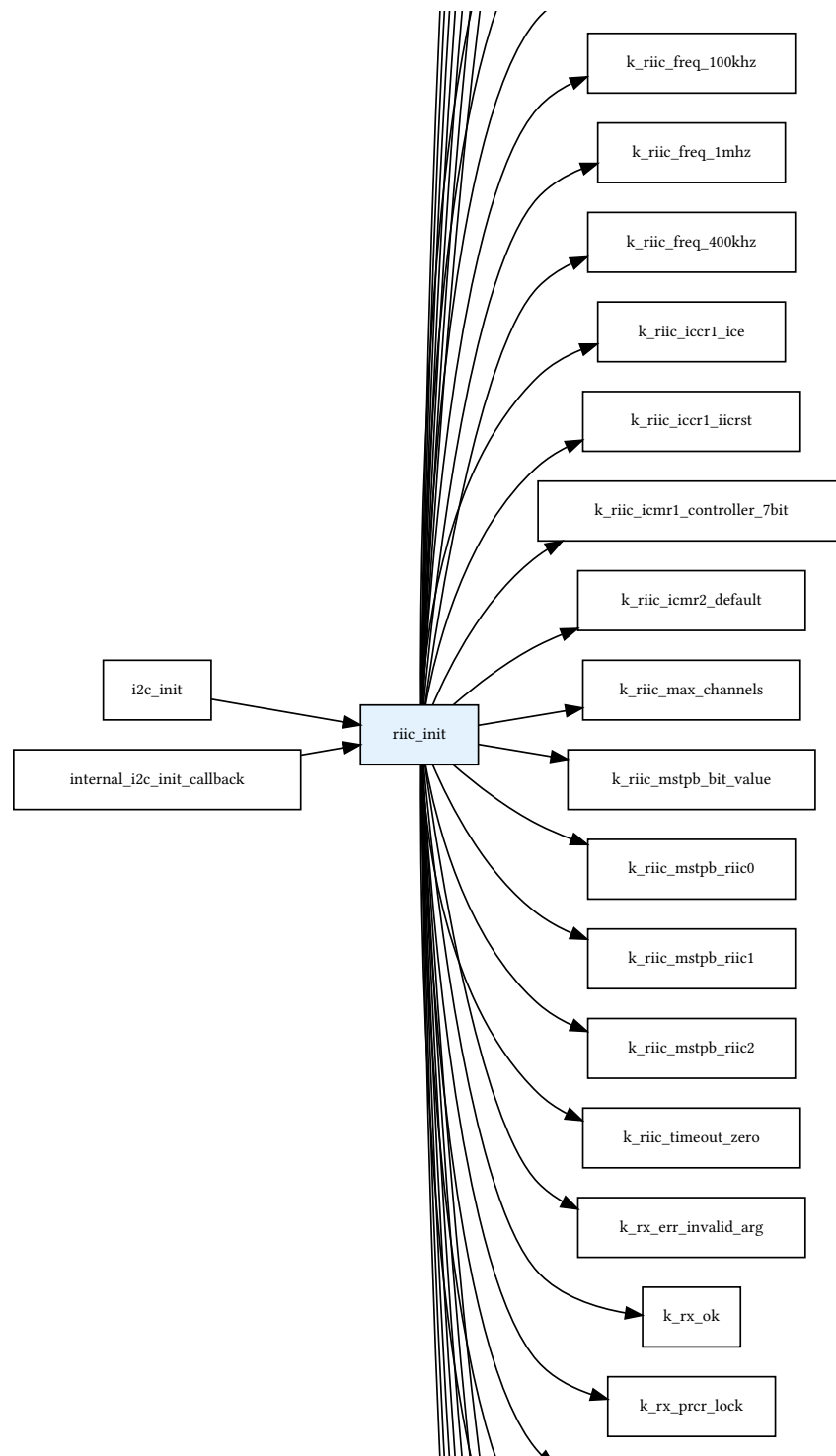


Figure 495: Call graph for `riic_init`

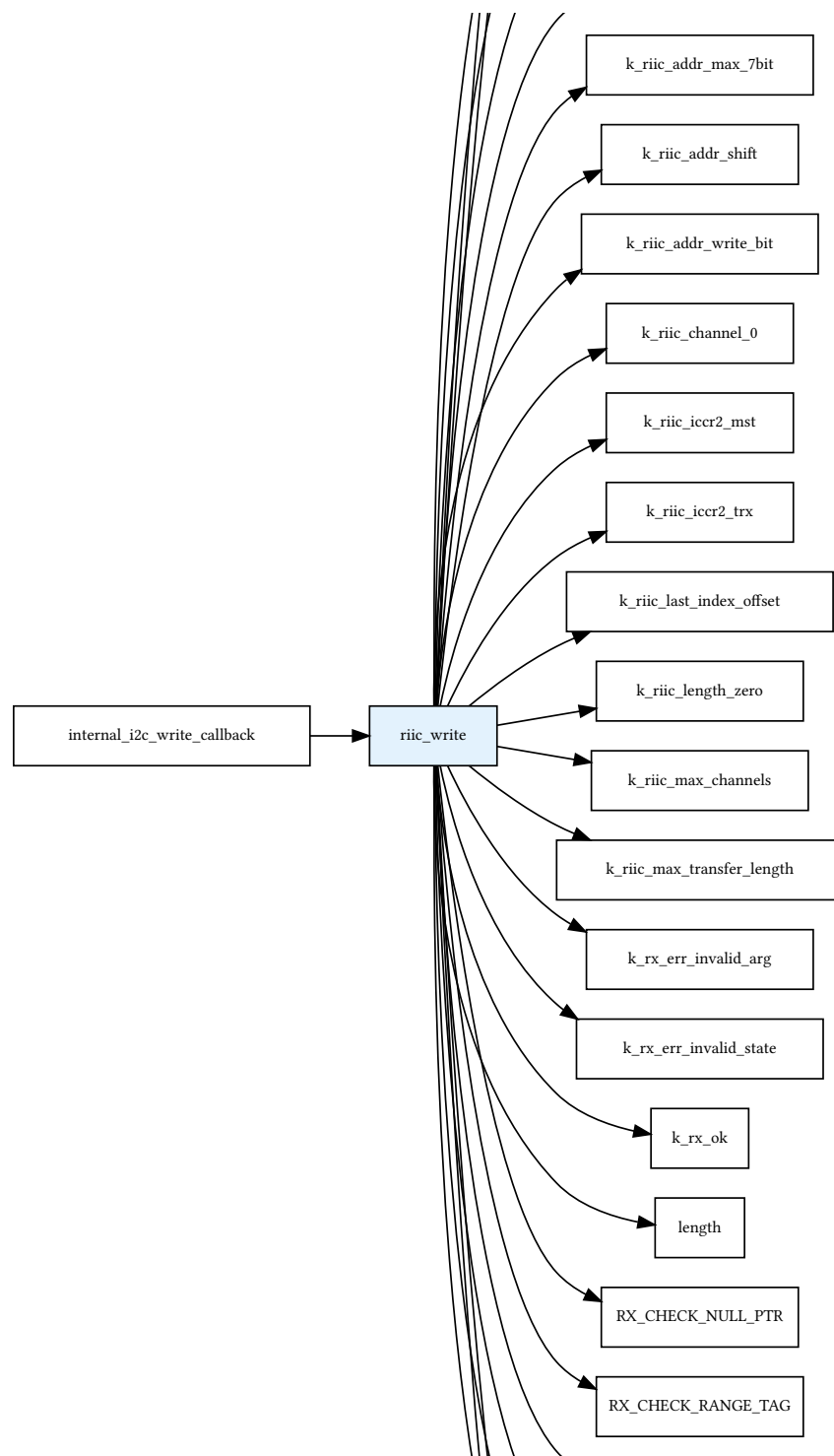
_Defined in libs/rx_hal/inc/hardware.h:1283_
—

riic_write

```
rx_err_t riic_write(riic_channel_t channel, i2c_device_addr_t device_addr, const
uint8_t *data, const uint16_t length)
```

Write data to I2C device on specified RIIC channel.

Performs a complete I2C write transaction: generates START condition, transmits device address with write bit (R/W=0), sends data bytes, and generates STOP condition. Each byte is acknowledged by the peripheral device. The function blocks until all bytes are transmitted or an error occurs.

Figure 496: Call graph for `riic_write`

_Defined in libs/rx_hal/inc/hardware.h:1346_

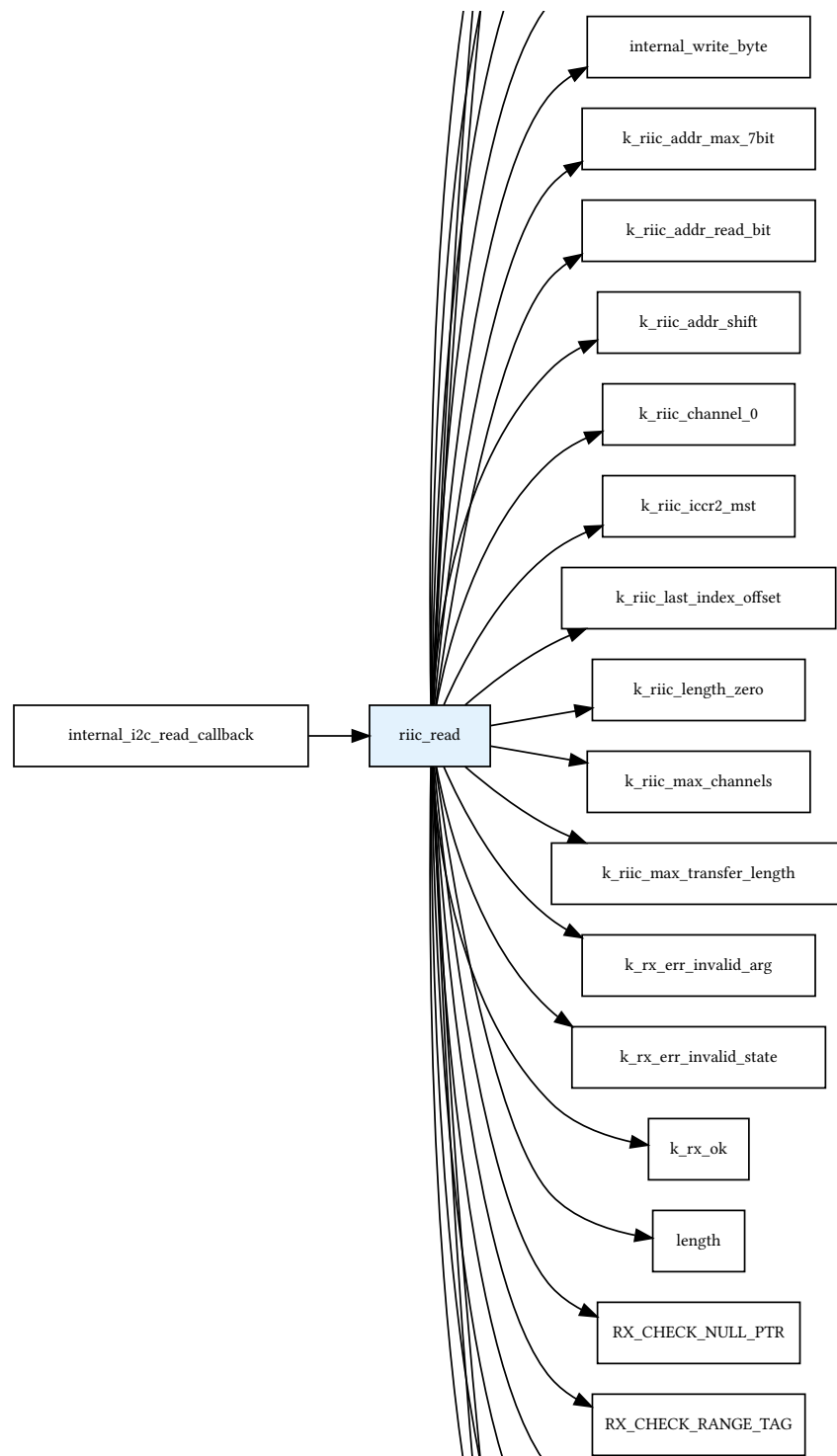
—

riic_read

```
rx_err_t riic_read(riic_channel_t channel, i2c_device_addr_t device_addr, uint8_t
*data, const uint16_t length)
```

Read data from I2C device on specified RIIC channel.

Performs a complete I2C read transaction: generates START condition, transmits device address with read bit (R/W=1), receives data bytes with ACK (NACK on last byte), and generates STOP condition. The function blocks until all bytes are received or an error occurs.

Figure 497: Call graph for `riic_read`

_Defined in libs/rx_hal/inc/hardware.h:1412_

—

riic_write_read

```
rx_err_t riic_write_read(riic_channel_t channel, i2c_device_addr_t device_addr,  
const uint8_t *write_data, uint16_t write_length, uint8_t *read_data, uint16_t  
read_length)
```

Write then read from I2C device using repeated START (combined transaction).

Performs an I2C combined write-then-read transaction using a repeated START condition. This is the standard pattern for reading device registers: write the register address, then issue a repeated START and read back the register data without releasing the bus between operations.



Figure 498: Call graph for `riic_write_read`

_Defined in libs/rx_hal/inc/hardware.h:1488_
—

rspi_init_peripheral

```
rx_err_t rspi_init_peripheral(rspi_channel_t channel, const rspi_config_t  
*config)
```

Initialize RSPI channel in peripheral mode for RPi5 communication.

Configures the specified RSPI channel as an SPI peripheral device. In peripheral mode the RX72N receives the clock signal from an external SPI controller (Raspberry Pi 5). Performs full hardware initialization including module stop release, pin function selection (COPI/CIPO/CLK/CS via MPC), SPI mode register configuration, and frame size setup.

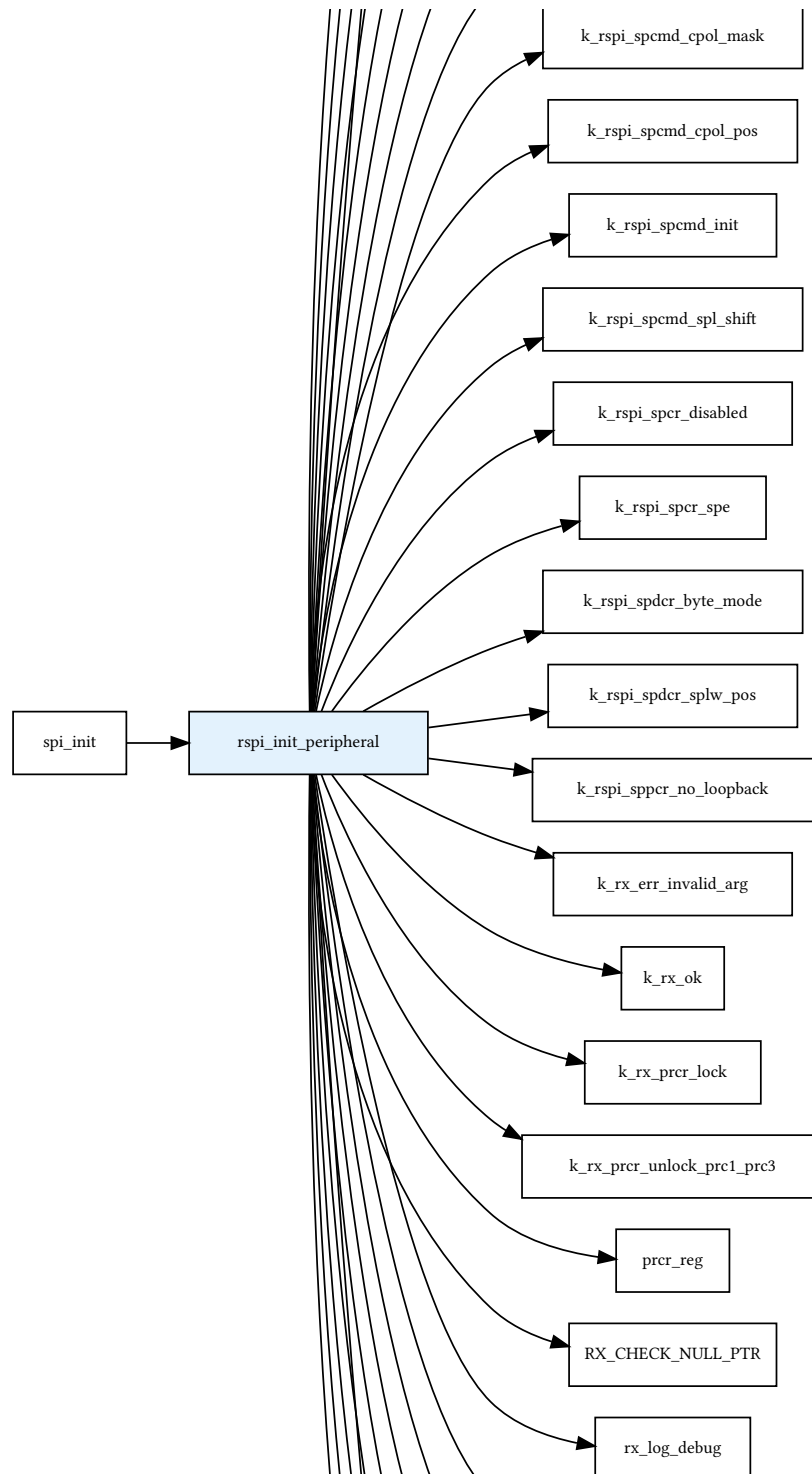


Figure 499: Call graph for `rspi_init_peripheral`

_Defined in libs/rx_hal/inc/hardware.h:1725_

—

rspi_peripheral_transfer

```
rx_err_t rspi_peripheral_transfer(rspi_channel_t channel, const uint8_t *tx_data,
uint8_t *rx_data, uint16_t length)
```

Full-duplex SPI transfer in peripheral mode.

Executes a byte-by-byte full-duplex SPI transfer on the specified RSPI channel operating in peripheral mode. For each byte, the function waits for the transmit buffer to empty, writes the TX byte, waits for the receive buffer to fill, then reads the RX byte. Both TX and RX wait operations are bounded by a 10 ms timeout to prevent infinite loops.

The transfer loop is bounded by k_rspi_transfer_len_max (65535) per NASA Rule 2 to ensure statically provable loop termination.

Full-duplex SPI transfer in peripheral mode.

Executes a full-duplex (simultaneous transmit and receive) SPI transfer on the specified RSPI channel operating in peripheral mode. Transfers data byte-by-byte with timeout protection on both transmit buffer ready and receive buffer full operations.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
in	tx_data	Pointer to transmit data buffer (must not be nullptr)
out	rx_data	Pointer to receive data buffer (must not be nullptr, must have capacity for at least length bytes)
in	length	Number of bytes to transfer (valid: 1 to 65535)
in	channel	RSPI channel number (0-2)
in	tx_data	Pointer to transmit data buffer (not nullptr)
out	rx_data	Pointer to receive data buffer (not nullptr)
in	length	Number of bytes to transfer (1 to 65535)

Returns: k_rx_err_timeout if: Transmit buffer does not become ready within timeout Receive buffer does not fill within timeout

Value	Description
k_rx_ok	Transfer completed successfully, rx_data filled
k_rx_err_null_ptr	tx_data or rx_data pointer is nullptr
k_rx_err_invalid_arg	length is zero, or RSPI base address lookup failed for the channel
k_rx_err_invalid_state	channel >= 3 or channel not initialized via rspi_init_peripheral()
k_rx_err_timeout	Transmit buffer did not empty or receive buffer did not fill within the 10 ms timeout

Pre: channel must be initialized via `rspi_init_peripheral()`

Pre: `tx_data` and `rx_data` must be valid non-null pointers

Pre: length must be in range 1, 65535

Pre: channel must be initialized via `rspi_init_peripheral()` before calling

Pre: `tx_data` pointer must be non-NULL

Pre: `rx_data` pointer must be non-NULL

Pre: length must be in range 1, 65535

Post: `rx_data` buffer contains received bytes corresponding to each transmitted byte offset on success

Post: SPSR status flags (SPRF, OVRF) cleared after each byte transfer

Post: If successful, `rx_data` buffer is filled with received bytes corresponding to transmitted data at each byte offset

Note: Not thread-safe. Caller must provide external synchronization when accessing the same channel from multiple threads.] **See also:** `rspi_init_peripheral()` Must be called before this function, `rspi_peripheral_read_available()` Non-blocking receive check, `rspi_peripheral_write_ready()` Non-blocking transmit check, `rspi_deinit()` Deinitialize RSPI channel

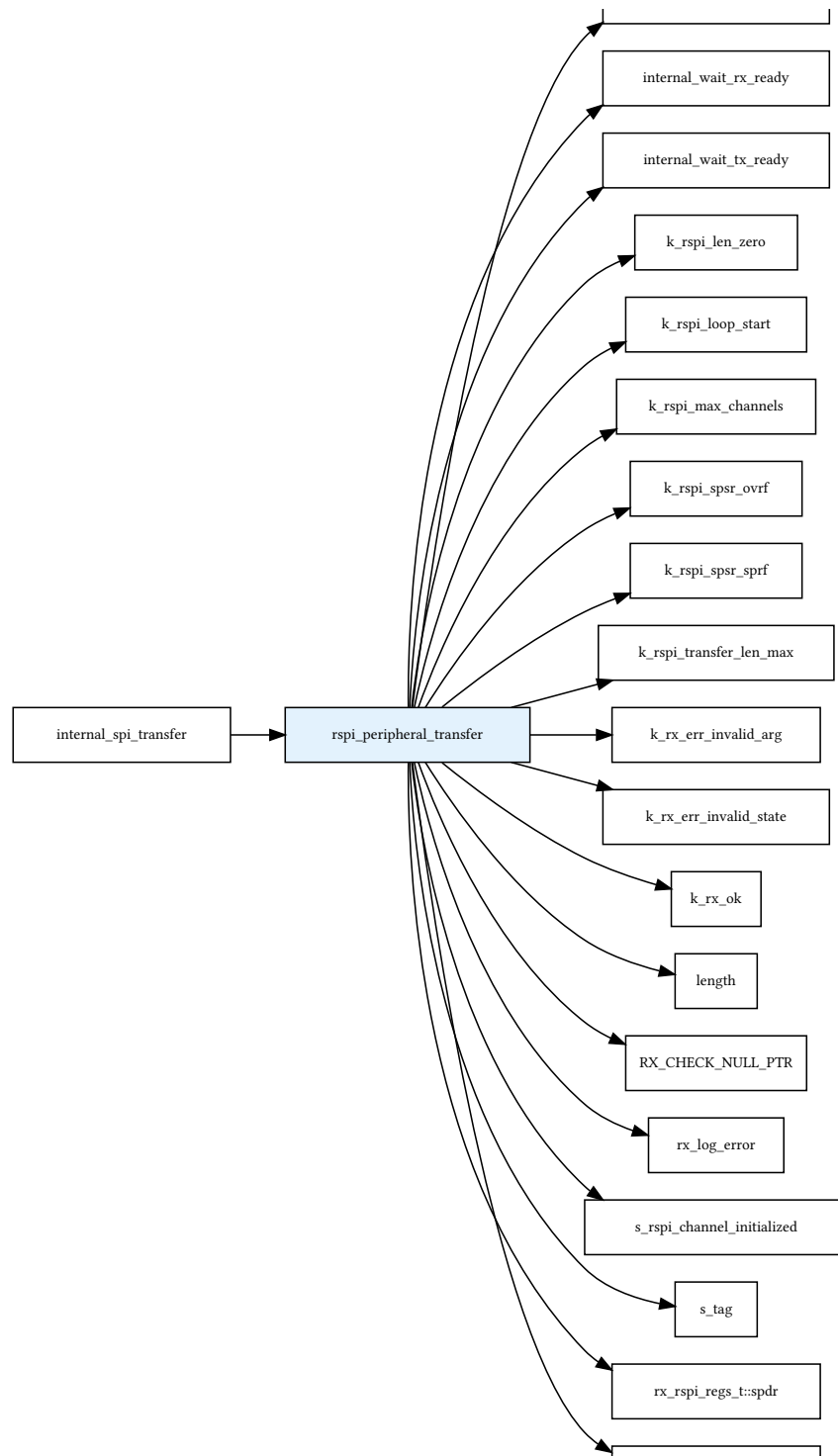


Figure 500: Call graph for `rspi_peripheral_transfer`

_Defined in libs/rx_hal/inc/hardware.h:1774_

Since Version 1.0.0

—

rspi_peripheral_read_available

```
rx_err_t rspi_peripheral_read_available(rspi_channel_t channel, bool *available)
```

Check if receive data is available in the RSPI receive buffer.

Polls the RSPI Status Register (SPSR) SPRF flag to determine whether the receive buffer contains data ready for reading. This is a non-blocking status check that returns immediately without waiting for data. Use this to implement polled receive patterns without incurring the overhead of a full transfer call.

Check if receive data is available in the RSPI receive buffer.

Polls the RSPI status register to determine if the receive buffer contains data ready for reading. Returns the status without blocking.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
out	available	Pointer to bool set to true if SPRF flag indicates data is available, false otherwise (must not be nullptr)
in	channel	RSPI channel number (0, 1, or 2)
out	available	Pointer to bool flag set to true if data is available, false otherwise

Returns: k_rx_err_invalid_arg if RSPI base address retrieval fails

Value	Description
k_rx_ok	Status successfully read into *available
k_rx_err_null_ptr	available pointer is nullptr
k_rx_err_invalid_state	channel >= 3 or channel not initialized via rspi_init_peripheral()
k_rx_err_invalid_arg	RSPI base address lookup failed for the channel

Pre: channel must be initialized via rspi_init_peripheral()

Pre: available must be a valid non-null pointer

Pre: Channel must be initialized via rspi_init_peripheral() before calling this function

Pre: available must be a valid non-nullptr

Post: *available reflects current SPRF flag state on success

Post: *available set to false on RSPI base address lookup failure

Note: Not thread-safe. Caller must provide external synchronization when accessing the same channel from multiple threads.] **See also:** `rspi_peripheral_transfer()` Full-duplex transfer in peripheral mode, `rspi_peripheral_write_ready()` Non-blocking transmit buffer check, `rspi_init_peripheral()` Must be called before this function

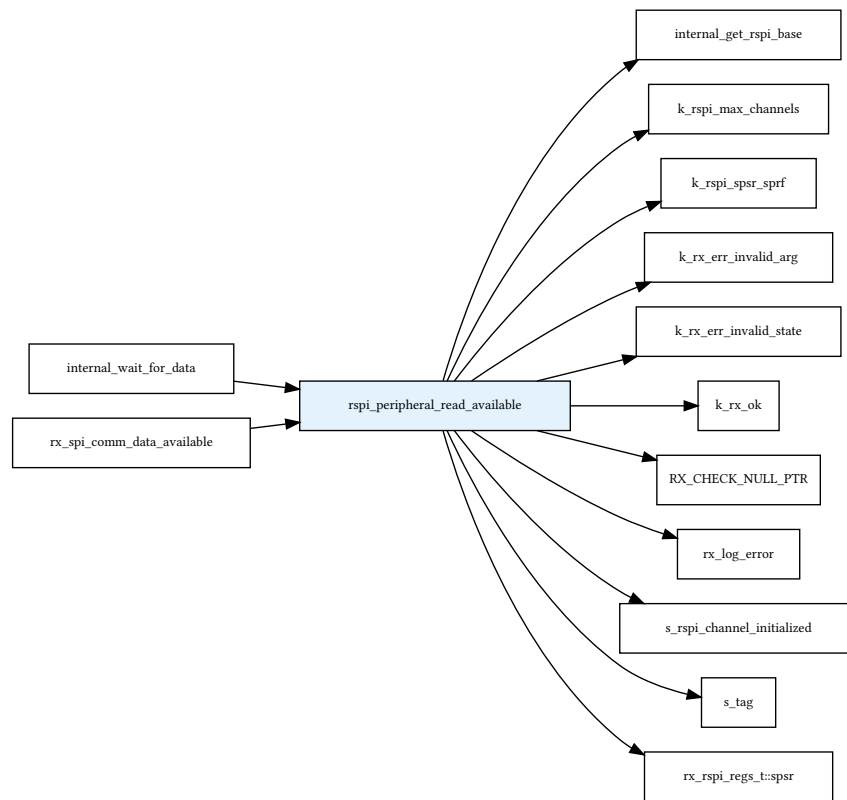


Figure 501: Call graph for `rspi_peripheral_read_available`

_Defined in `libs/rx_hal/inc/hardware.h:1815_`

Since Version 1.0.0

`rspi_peripheral_write_ready`

```
rx_err_t rspi_peripheral_write_ready(rspi_channel_t channel, bool *ready)
```

Check if the RSPI transmit buffer is ready for new data.

Polls the RSPi Status Register (SPSR) SPTEF flag to determine whether the transmit buffer is empty and ready to accept new data. This is a non-blocking status check that returns immediately without waiting for the buffer to drain. Use this to implement polled transmit patterns or to verify readiness before writing to the SPI data register.

Check if the RSPi transmit buffer is ready for new data.

Polls the RSPi status register to determine if the transmit buffer is empty and ready to accept new data. Returns the status without blocking.

Parameters:

Direction	Name	Description
in	channel	RSPi channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
out	ready	Pointer to bool set to true if SPTEF flag indicates the transmit buffer is empty, false otherwise (must not be nullptr)
in	channel	RSPi channel number (0-2)
out	ready	Set to true if ready to transmit, false otherwise

Returns: k_rx_err_invalid_arg if RSPi base address retrieval fails

Value	Description
k_rx_ok	Status successfully read into *ready
k_rx_err_null_ptr	ready pointer is nullptr
k_rx_err_invalid_state	channel >= 3 or channel not initialized via rspi_init_peripheral()
k_rx_err_invalid_arg	RSPi base address lookup failed for the channel

Pre: channel must be initialized via rspi_init_peripheral()

Pre: ready must be a valid non-null pointer

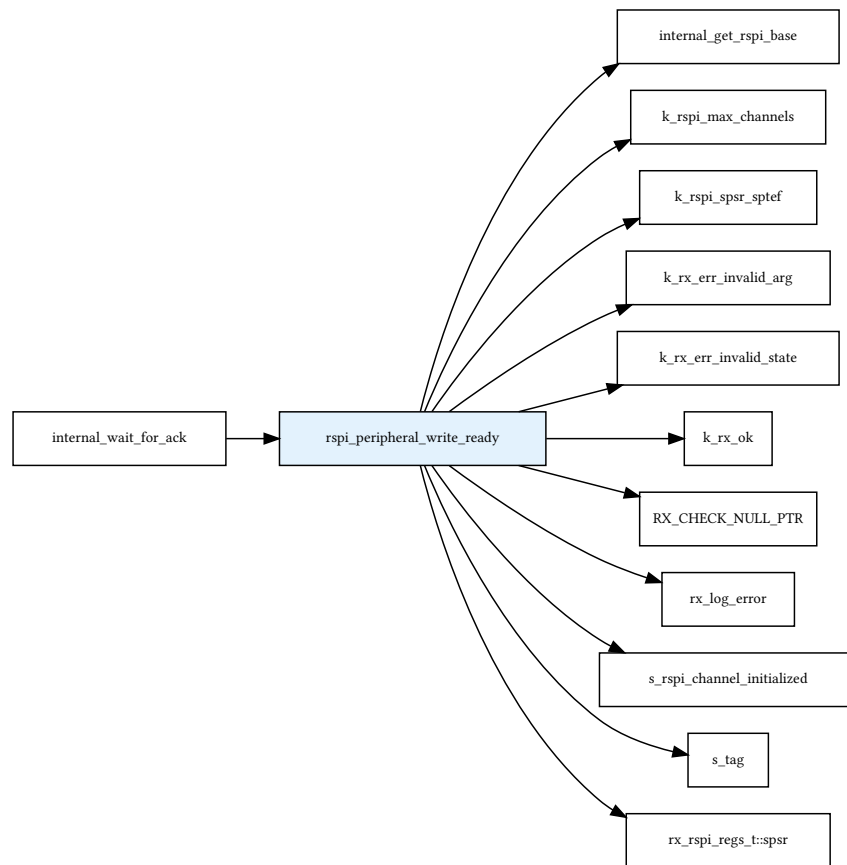
Pre: Channel must be initialized via rspi_init_peripheral()

Pre: ready must be a valid non-nullptr

Post: *ready reflects current SPTEF flag state on success

Post: *ready set to false on RSPi base address lookup failure

Note: Not thread-safe. Caller must provide external synchronization when accessing the same channel from multiple threads.] **See also:** rspi_peripheral_transfer() Full-duplex transfer in peripheral mode, rspi_peripheral_read_available() Non-blocking receive buffer check, rspi_init_peripheral() Must be called before this function

Figure 502: Call graph for `rspi_peripheral_write_ready`

_Defined in `libs/rx\hal/inc/hardware.h:1853_`

Since Version 1.0.0

—

rspi_deinit

```
rx_err_t rspi_deinit(rspi_channel_t channel)
```

Deinitialize and disable an RSPI peripheral-mode channel.

Performs a safe shutdown of the specified RSPI channel previously initialized in peripheral mode. The deinitialization sequence disables SPI bus activity (`SPCR.SPE = 0`), gates the module clock via `MSTPCRB` to reduce power consumption, and clears the internal initialization flag. Register protection (`PRCR`) is temporarily unlocked for module stop control.

This function is safe to call on channels that were never initialized; it will still disable the hardware and clear the init flag. To reinitialize a channel after deinit, call `rspi_init_peripheral()` again.

Deinitialize and disable an RSPI peripheral-mode channel.

Disables the specified RSPI channel, clears the initialization flag, and optionally disables the module (via module stop bit) to reduce power consumption.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
in	channel	RSPI channel number (0, 1, or 2)

Returns: k_rx_err_invalid_arg if: channel is out of range (≥ 3) RSPI base address lookup fails for the channel

Value	Description
k_rx_ok	Channel successfully deinitialized and module stopped
k_rx_err_invalid_arg	channel ≥ 3 or RSPI base address lookup failed for the channel

Pre: channel must be in range 0, 2 (k_rspi_channel_0 through k_rspi_channel_2)

Pre: No active SPI transfer should be in progress on this channel

Pre: Channel should have been initialized via rspi_init_peripheral() previously

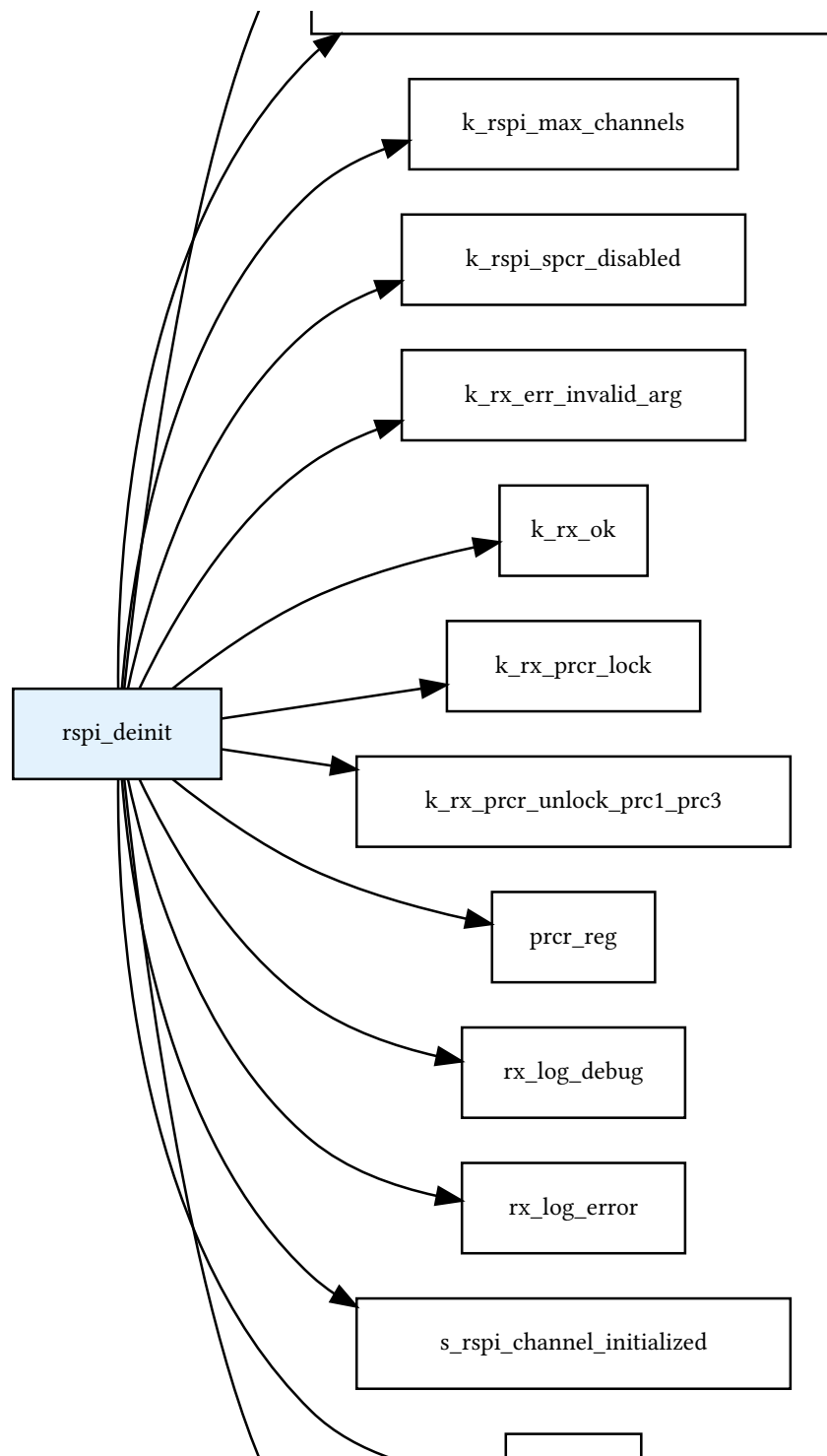
Post: RSPI peripheral disabled (SPCR.SPE = 0)

Post: Module clock gated (MSTPCRB module stop bit set)

Post: s_rspi_channel_initialized[channel] set to false

Post: If successful:] RSPI peripheral is disabled (SPCR.SPE = 0)
s_rspi_channel_initialized[channel] is set to false Channel is no longer available for SPI operations

Note: Not thread-safe. Caller must ensure no concurrent transfers or accesses to the same channel during deinitialization.] **See also:** rspi_init_peripheral() Reinitialize channel after deinit, rspi_controller_deinit() Deinit for controller-mode channels

Figure 503: Call graph for `rspi_deinit`

_Defined in libs/rx_hal/inc/hardware.h:1891_

Since Version 1.0.0

—

rspi_init_controller

```
rx_err_t rspi_init_controller(rspi_channel_t channel, const
rspi_controller_config_t *config)
```

Initialize RSPI channel in controller mode for external device communication.

Configures the specified RSPI channel as an SPI controller (bus master) for communicating with external SPI peripheral devices such as sensors. Performs argument validation, SPBR bit rate calculation, CS GPIO pin configuration, and full hardware register setup following the RX72N Hardware Manual Section 38.3.6 initialization sequence. Always configures 16-bit data frames with word access mode.

The initialization is split into three internal phases: argument validation, hardware resource preparation (SPBR calculation, base address lookup, GPIO setup), and register configuration (module clock enable, SPI mode, bit rate, SPI enable). On any failure, no partial hardware state is left behind.

Initialize RSPI channel in controller mode for external device communication.

Configures the specified RSPI channel as a SPI controller to communicate with SPI peripheral devices. Enables the module, configures clock frequency, SPI mode, and chip select GPIO pin.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
in	config	Pointer to rspi_controller_config_t containing: spi_mode: SPI mode (0-3) for CPOL/CPHA configuration freq_hz: SPI clock frequency in Hz (100 kHz to 10 MHz) cs: GPIO pin for chip select (rx_port_pin_t encoding)
in	channel	RSPI channel number (0-2)
in	config	Pointer to controller configuration structure containing: freq_hz: SPI clock frequency (100 kHz to 10 MHz) spi_mode: SPI mode (0-3) for CPOL/CPHA configuration cs: GPIO pin for chip select (type-safe rx_port_pin_t)

Returns: k_rx_err_invalid_state if channel is already initialized

Value	Description
k_rx_ok	Channel successfully initialized in controller mode
k_rx_err_null_ptr	config pointer is nullptr
k_rx_err_invalid_arg	channel >= 3, spi_mode > 3, freq_hz outside 100 kHz - 10 MHz range, RSPI base address lookup failed, or CS GPIO port/pin is invalid
k_rx_err_invalid_state	channel already initialized in controller mode (call rspi_controller_deinit() first)

Pre: config must be a valid non-null pointer

Pre: channel must not already be initialized in controller mode

Pre: System clocks must be configured (PCLKB running at 60 MHz)

Pre: config must be non-NULL

Pre: channel < k_rspi_max_channels

Pre: channel must not be already initialized

Pre: CS GPIO port and pin must be valid and available

Post: RSPI module clock enabled, registers configured, SPI enabled in controller mode (SPCR.MSTR = 1, SPCR.SPE = 1)

Post: CS GPIO configured as output, driven high (inactive / deasserted)

Post: s_rspi_controller_initializedchannel set to true

Post: If successful:] RSPI module is enabled via internal_set_mstpcrb_for_channel() *prcr_reg() is unlocked/locked for register protection RSPI registers configured: spbr, spcmd0, spcr, spdcr, sppcr CS GPIO configured as output, driven high (inactive) s_rspi_cs_config[channel] stores CS port/pin s_rspi_controller_initialized[channel] = true

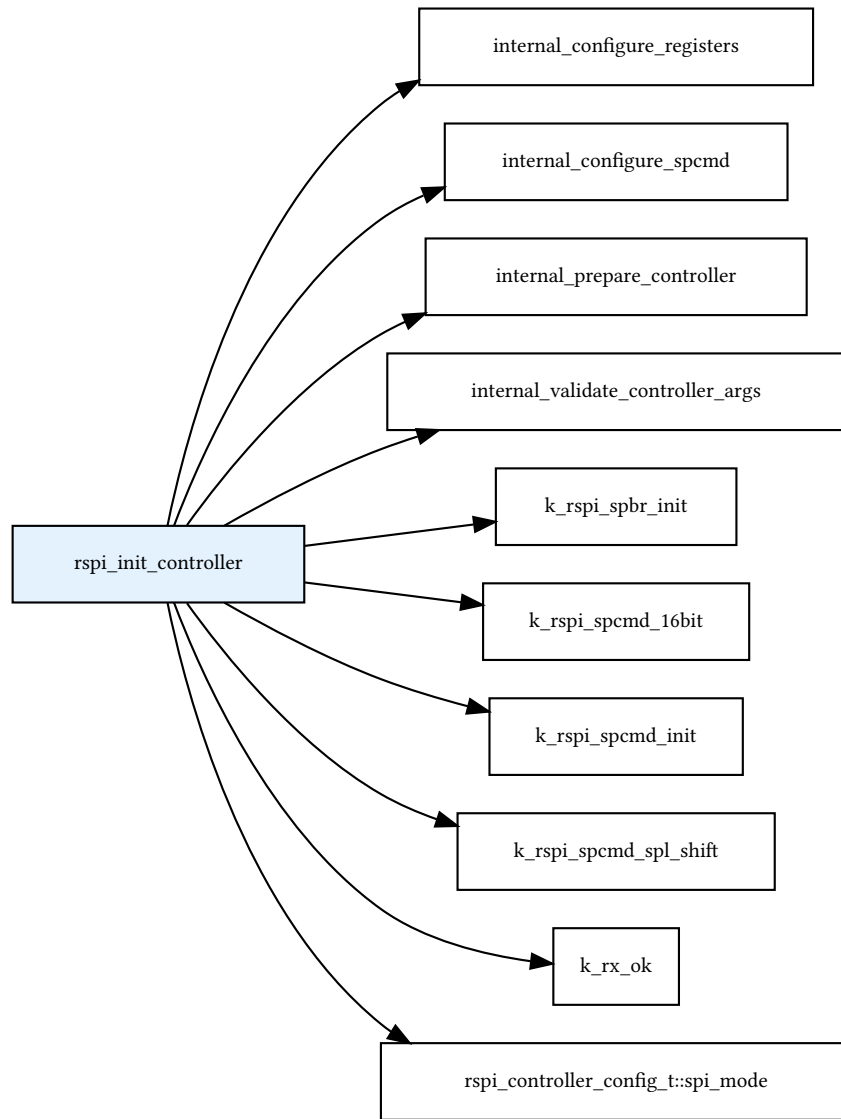
Post: On error:] Hardware state is unchanged Channel remains uninitialized

Note: Not thread-safe. Initialize each channel from a single thread during system startup.

Note: Bit rate calculation: $\text{freq} = \text{PCLKB} / (2 * (\text{SPBR} + 1))$

Note: Always configures 16-bit data frames with word access mode

Note: CS is active-low (asserted=0, deasserted=1)] **See also:**
rspi_controller_transfer_16bit() Perform 16-bit transfers after init,
rspi_controller_set_cs() Manual chip select control, rspi_controller_deinit()
Deinitialize controller-mode channel, rspi_controller_config_t Configuration structure

Figure 504: Call graph for `rspi_init_controller`

_Defined in `libs/rx\_hal/inc/hardware.h:1979_`

Since Version 1.0.0

—

rspi_controller_set_cs

```
rx_err_t rspi_controller_set_cs(rspi_channel_t channel, bool active)
```

Set chip select line state for an RSPI controller-mode channel.

Controls the GPIO chip select pin associated with the specified RSPI channel operating in controller mode. The CS signal is active-low: when active is true, the GPIO pin is driven low (CS asserted, peripheral selected); when active is false, the pin is driven high (CS deasserted, peripheral released). The port and pin are read from the internal `s_rspi_cs_config[]` array populated during `rspi_init_controller()`.

This function provides direct CS control for protocols that require multi-transfer CS framing. For single 16-bit transfers with automatic CS handling, use `rspi_controller_transfer_16bit()` instead.

Set chip select line state for an RSPI controller-mode channel.

Controls the GPIO chip select pin for the specified RSPI channel operating in controller mode. CS is active-low (asserted low, deasserted high).

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: <code>k_rspi_channel_0</code> , <code>k_rspi_channel_1</code> , <code>k_rspi_channel_2</code> ; range 0-2)
in	active	true to assert CS (drive GPIO low), false to deassert CS (drive GPIO high)
in	channel	RSPI channel number (0-2)
in	active	True to assert CS (drive low), false to deassert CS (drive high)

Returns: `k_rx_err_invalid_arg` if GPIO port base lookup fails

Value	Description
<code>k_rx_ok</code>	CS line successfully driven to requested state
<code>k_rx_err_invalid_state</code>	channel ≥ 3 or channel not initialized in controller mode via <code>rspi_init_controller()</code>
<code>k_rx_err_invalid_arg</code>	GPIO port base address lookup failed for the stored CS port configuration

Pre: channel must be initialized via `rspi_init_controller()`

Pre: `s_rspi_cs_config` channel must contain valid port/pin from init

Pre: Channel must be initialized via `rspi_init_controller()`

Pre: channel < `k_rspi_max_channels`

Pre: `s_rspi_controller_initialized` channel == true

Post: GPIO PODR bit for the CS pin reflects the requested state

Post: No other GPIO pins on the same port are modified (read-modify-write)

Post: GPIO port_regs->podr is modified to assert/deassert CS pin

Note: Not thread-safe. Caller must provide external synchronization when accessing the same channel or GPIO port from multiple threads.

Note: Reads s_rspi_cs_configchannel for port and pin configuration] **See also:**
 rspi_init_controller() Configures CS pin during initialization,
 rspi_controller_transfer_16bit() Automatic CS handling for transfers,
 rspi_controller_deinit() Deasserts CS during cleanup

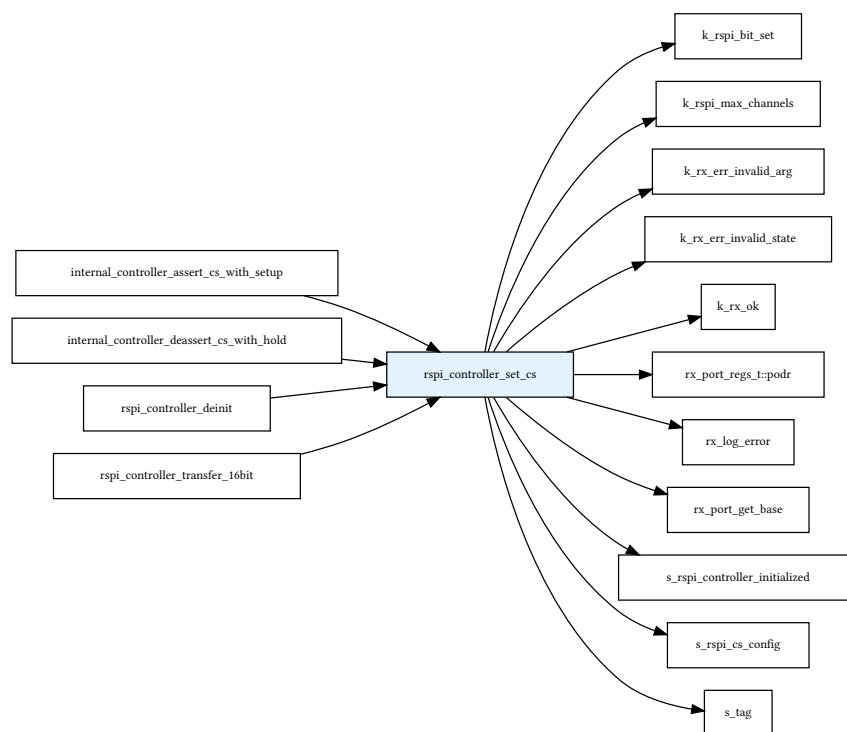


Figure 505: Call graph for rspi_controller_set_cs

_Defined in libs/rx_hal/inc/hardware.h:2023_

Since Version 1.0.0

rspi_controller_transfer_16bit

```

rx_err_t rspi_controller_transfer_16bit(rspi_channel_t channel, uint16_t tx_data,
uint16_t *rx_data)

```

Perform a 16-bit full-duplex SPI transfer in controller mode with automatic CS.

Executes a single 16-bit SPI transfer with full chip select lifecycle management. The transfer sequence is:

1. Assert CS (drive low) with 300 ns setup delay
2. Wait for TX buffer empty, write 16-bit tx_data to SPDR
3. Wait for RX buffer full, read 16-bit response from SPDR
4. Clear SPSR status flags (SPRF, OVRF)
5. Apply 300 ns hold delay, then deassert CS (drive high)

On transfer error (timeout), CS is forcibly deasserted before returning to avoid leaving the peripheral in a selected state. The CS setup and hold delays use cycle-accurate NOP loops calibrated for 240 MHz core clock.

Perform a 16-bit full-duplex SPI transfer in controller mode with automatic CS.

Executes a full-duplex 16-bit SPI transfer with automatic chip select assertion/deassertion and timing delays.

Sequence:

1. Assert CS (active low) with setup delay (300ns)
2. Wait for TX buffer ready, then transmit 16-bit data
3. Wait for RX buffer full, then read 16-bit response
4. Hold CS for minimum hold time (300ns)
5. Deassert CS (inactive high)
6. Clear status flags

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
in	tx_data	16-bit data word to transmit
out	rx_data	Pointer to store the received 16-bit response (must not be nullptr)
in	channel	RSPI channel number (0-2)
in	tx_data	16-bit data to transmit
out	rx_data	Pointer to receive 16-bit response (must be non-NULL)

Returns: Other errors from rspi_controller_set_cs() on CS control failure

Value	Description
k_rx_ok	Transfer completed, *rx_data contains response, CS deasserted
k_rx_err_null_ptr	rx_data pointer is nullptr
k_rx_err_invalid_state	channel >= 3 or channel not initialized via rspi_init_controller()
k_rx_err_invalid_arg	RSPI base address lookup failed, or GPIO port lookup failed during CS assertion/deassertion
k_rx_err_timeout	TX buffer did not empty or RX buffer did not fill within the 10 ms timeout (CS is deasserted on this error)

Pre: channel must be initialized via `rspi_init_controller()`

Pre: `rx_data` must be a valid non-null pointer

Pre: Channel must be initialized via `rspi_init_controller()`

Pre: `rx_data` must be a valid non-nullptr

Post: `*rx_data` contains received 16-bit value on success

Post: CS is deasserted (GPIO driven high) on both success and error

Post: SPSR status flags SPRF and OVRF are cleared on success

Post: If successful:] `rx_data` is filled with received 16-bit value Status flags (SPRF, OVRF) are cleared CS is deasserted (high)

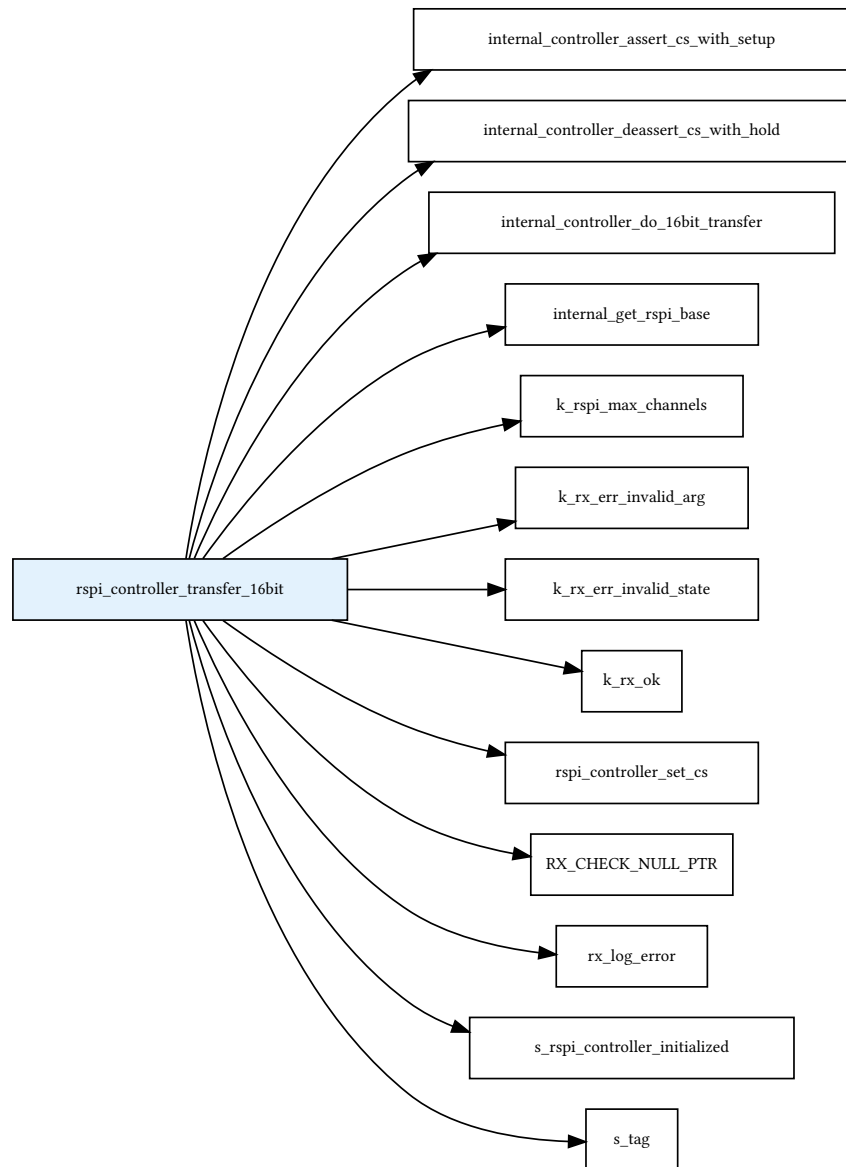
Post: On error:] CS is deasserted if possible `rx_data` contents are undefined

Note: Not thread-safe. Caller must provide external synchronization when accessing the same channel from multiple threads.

Note: CS setup delay: `k_rspi_cs_setup_delay` NOP loops (300ns)

Note: CS hold delay: `k_rspi_cs_hold_delay` NOP loops (300ns)

Note: Uses inline assembly NOP for precise timing control] **See also:** `rspi_init_controller()` Must be called before this function, `rspi_controller_set_cs()` Manual CS control for multi-transfer framing, `rspi_controller_deinit()` Deinitialize controller-mode channel

Figure 506: Call graph for `rspi_controller_transfer_16bit`_Defined in `libs/rx_hal/inc/hardware.h:2073_`*Since Version 1.0.0*

—

`rspi_controller_deinit`

```
rx_err_t rspi_controller_deinit(rspi_channel_t channel)
```

Deinitialize and disable an RSPI controller-mode channel.

Performs a safe shutdown of the specified RSPI channel previously initialized in controller mode. The deinitialization sequence is ordered to prevent bus glitches: first deasserts CS to release any connected peripheral, then disables SPI (SPCR.SPE = 0) to stop clock generation, then gates the module clock via MSTPCRB for power savings, and finally clears internal state (CS pin config and initialization flag). Register protection (PRCR) is temporarily unlocked for module stop control.

This function is safe to call on channels that were never initialized in controller mode; it will still disable the hardware and clear the state.

Deinitialize and disable an RSPI controller-mode channel.

Disables the RSPI controller, deasserts chip select, clears configuration, and stops the module to reduce power consumption.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2; range 0-2)
in	channel	RSPI channel number (0-2)

Returns: k_rx_err_invalid_arg if channel is out of range or base address lookup fails

Value	Description
k_rx_ok	Channel successfully deinitialized and module stopped
k_rx_err_invalid_arg	channel >= 3 or RSPI base address lookup failed for the channel

Pre: channel must be in range 0, 2 (k_rspi_channel_0 through k_rspi_channel_2)

Pre: No active SPI transfer should be in progress on this channel

Pre: Should be called after rspi_init_controller() to properly clean up

Post: CS GPIO deasserted (driven high) if channel was initialized

Post: RSPI peripheral disabled (SPCR.SPE = 0)

Post: Module clock gated (MSTPCRB module stop bit set)

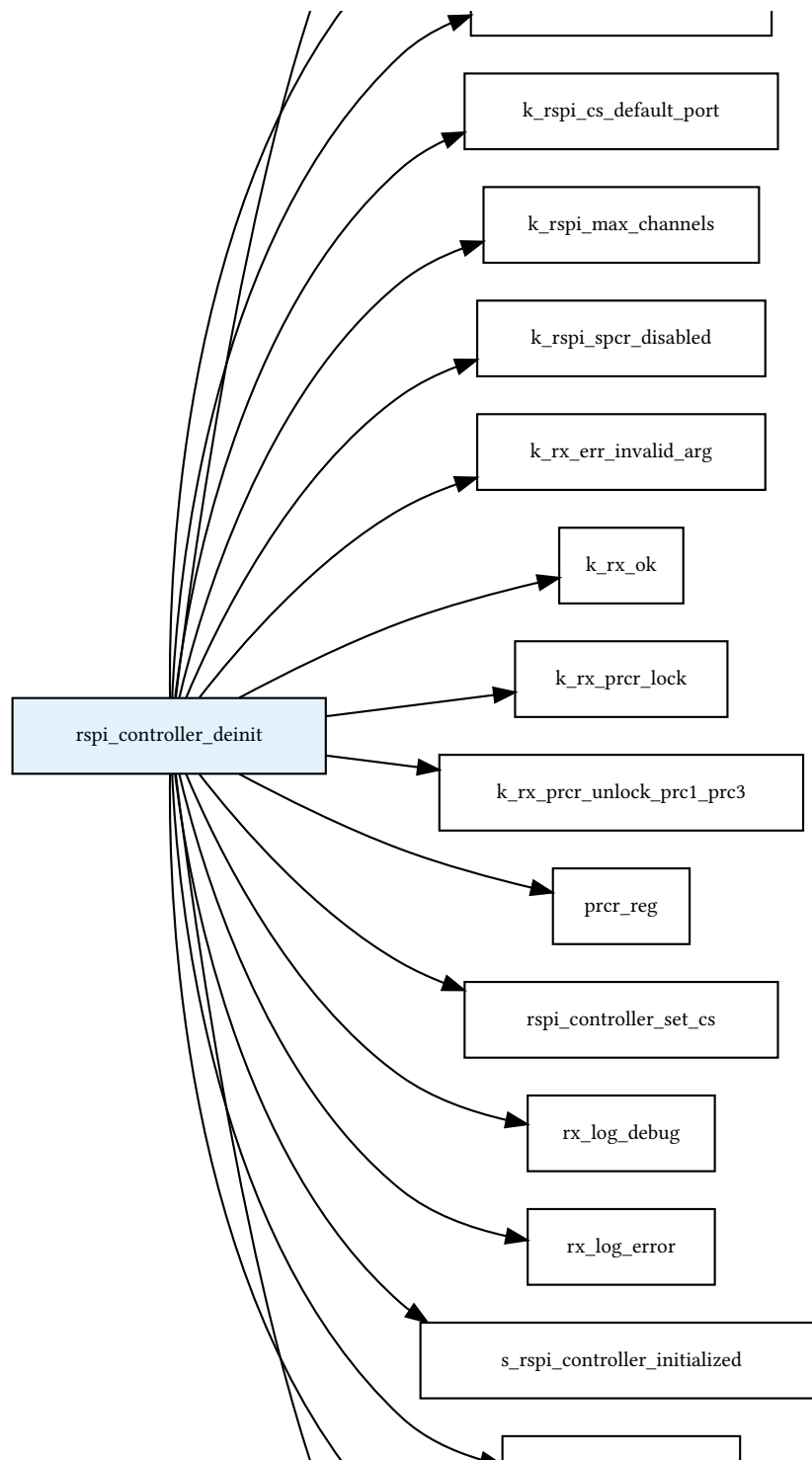
Post: s_rspi_cs_configchannel cleared to defaults (port=0, pin=0)

Post: s_rspi_controller_initializedchannel set to false

Post: If successful:] CS is deasserted via `rspi_controller_set_cs(channel, false)` RSPi is disabled (`rspi->spcr = k_rspi_spcr_disabled`) Module clock is stopped via `internal_set_mstpcrb_for_channel()` `s_rspi_cs_config[channel]` is cleared (`port=0, pin=0`) `s_rspi_controller_initialized[channel] = false` Register protection is unlocked/locked via `*prcr_reg()`

Note: Not thread-safe. Caller must ensure no concurrent transfers or accesses to the same channel during deinitialization.

Note: Thread-safe only if caller ensures no concurrent access to the same channel] **See also:** `rspi_init_controller()` Reinitialize channel after deinit, `rspi_deinit()` Deinit for peripheral-mode channels

Figure 507: Call graph for `rspi_controller_deinit`

_Defined in libs/rx_hal/inc/hardware.h:2114_

Since Version 1.0.0

—

uart_init_channel

```
rx_err_t uart_init_channel(const uart_channel_config_t *config)
```

Initialize SCI channel for UART communication.

Performs full hardware initialization including:

- Module stop control (MSTPCRB)
- Pin function configuration (MPC)
- GPIO direction setup (PDR/PMR)
- SCI register configuration

Initialize SCI channel for UART communication.

Configures and enables an SCI peripheral for asynchronous UART operation. Handles module clock enable, pin mux configuration, and SCI register setup.

Algorithm steps:

1. Validate configuration parameters (NULL check, channel range, baud rate)
2. Get SCI register base address for channel
3. Check if channel is already initialized (prevent double-init)
4. Enable SCI module clock (clear MSTPB bit)
5. Configure TX/RX pins via MPC and GPIO registers
6. Disable TX/RX during configuration (SCR = 0)
7. Set serial mode: async, 8N1, PCLK/1 (SMR)
8. Calculate and set baud rate divisor (BRR)
9. Wait for 1 bit time (synchronization)
10. Enable TX and RX (SCR = 0x30)
11. Mark channel as initialized

Parameters:

Direction	Name	Description
in	config	UART channel configuration (must not be NULL)
in	config	Pointer to channel configuration structure. channel: SCI channel (0-12) baudrate: Target baud rate (1 to PCLKB/32) tx_gpio: TX pin (rx_port_pin_t) rx_gpio: RX pin (rx_port_pin_t) Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, channel initialized and ready
k_rx_err_null_ptr	config parameter is nullptr
k_rx_err_invalid_arg	Invalid channel (>=13) or invalid baud rate

k_rx_err_invalid_state	Channel already initialized
------------------------	-----------------------------

Pre: config must point to valid uart_channel_config_t structure

Pre: config->channel must be in range 0, 12

Pre: config->baudrate must be in valid range for PCLKB

Pre: Channel must not be already initialized

Post: Channel configured for 8N1 async operation

Post: TX and RX enabled and ready for use

Post: s_channel_initializedchannel set to true

Post: TX/RX pins configured for peripheral function

Note: Not thread-safe - call once during initialization

Note: Includes 1ms delay for baud rate synchronization

Warning: Does not validate pin assignments against hardware

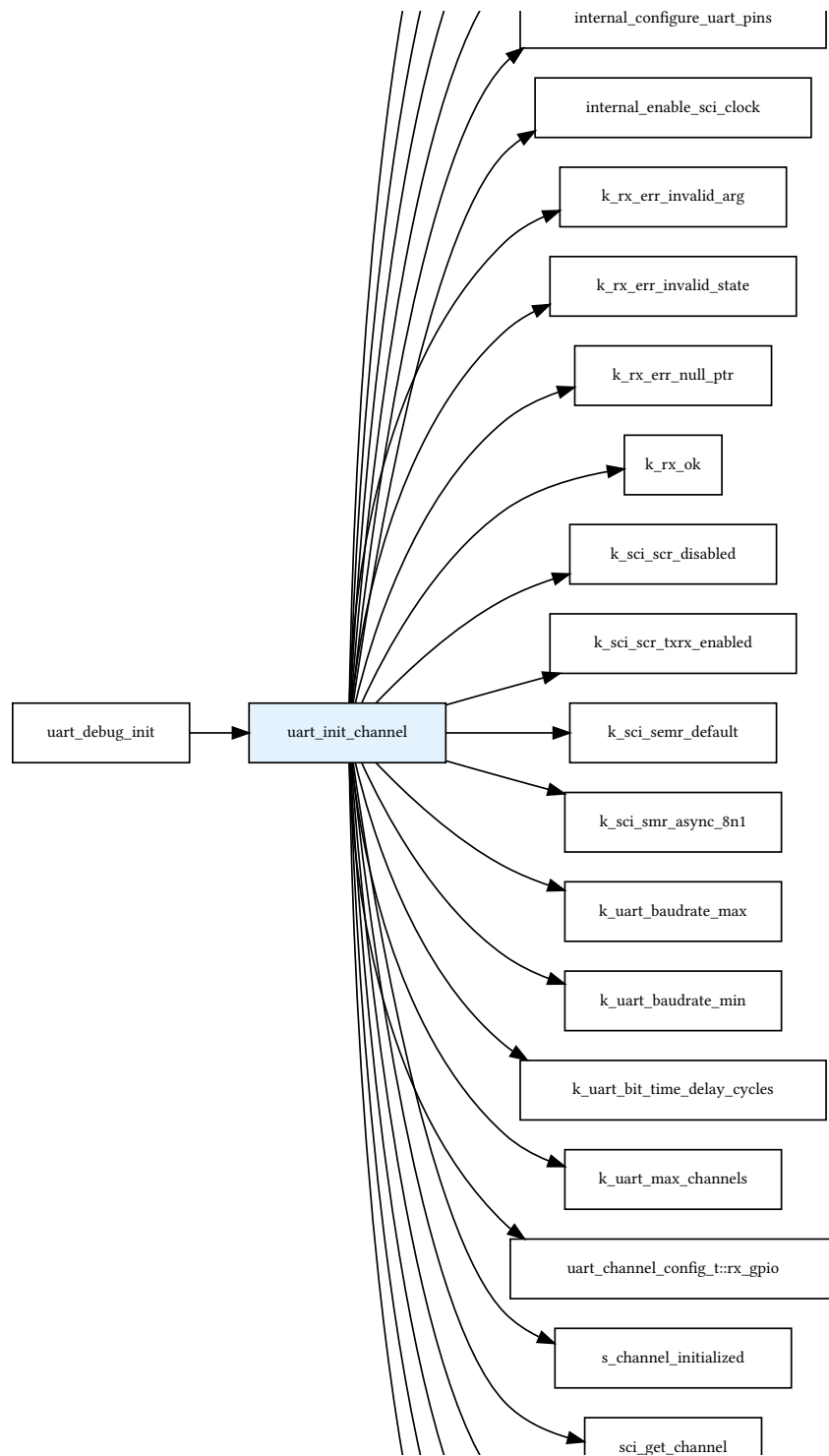
Thread Safety:: Not thread-safe. Call once per channel during system initialization.

Performance:: Execution time: 50 us (includes MPC and delay) Stack usage: 48 bytes

Example::

```
const uart_channel_config_t config = { .channel = k_uart_channel_9, .baudrate = 115200, .tx_gpio = k_rx_pb_7, .rx_gpio = k_
rx_err_tterr = uart_init_channel(&config); if(err != k_rx_ok){ }
```

See also: `uart_deinit_channel()` Deinitialize channel, `uart_putc_channel()` Transmit single character, `uart_channel_config_t` Configuration structure

Figure 508: Call graph for `uart_init_channel`

_Defined in libs/rx_hal/inc/hardware.h:2184_

Since Version 1.0.0

—

uart_deinit_channel

```
rx_err_t uart_deinit_channel(uart_channel_t channel)
```

Deinitialize SCI channel.

Deinitialize SCI channel.

Disables transmit and receive on the specified SCI channel and marks it as uninitialized. The module clock is left running (safe to re-initialize without re-enabling). After this call the channel may be re-initialized with `uart_init_channel()`.

Algorithm steps:

1. Validate channel number (0-12)
2. Get SCI register base address
3. Write SCR = 0 to disable TX and RX
4. Clear `s_channel_initialized[channel]`

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
in	channel	UART channel to deinitialize (0-12) Valid range: 0 to 12 (SCI0 through SCI12) Recommended: Use <code>k_uart_channel_X</code> constants

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, channel TX/RX disabled
<code>k_rx_err_invalid_arg</code>	Invalid channel number (≥ 13) or invalid register pointer

Pre: channel must be in range 0, 12

Pre: Channel should be initialized before deinitializing (safe to call on uninit channel)

Post: TX and RX disabled (SCR = 0)

Post: `s_channel_initializedchannel` set to false

Note: Not thread-safe - call during shutdown, not during normal operation

Note: Module stop clock is NOT re-asserted (peripheral clock left enabled)

Thread Safety:: Not thread-safe. Do not call while another thread is using the channel.

Performance:: Execution time: 1 us Stack usage: 16 bytes

Example:: `rx_err_t err=uart_deinit_channel(k_uart_channel_9); if(err!=k_rx_ok){ }`

See also: `uart_init_channel()` Initialize channel, `uart_debug_init()` Initialize debug channel (SCI9)

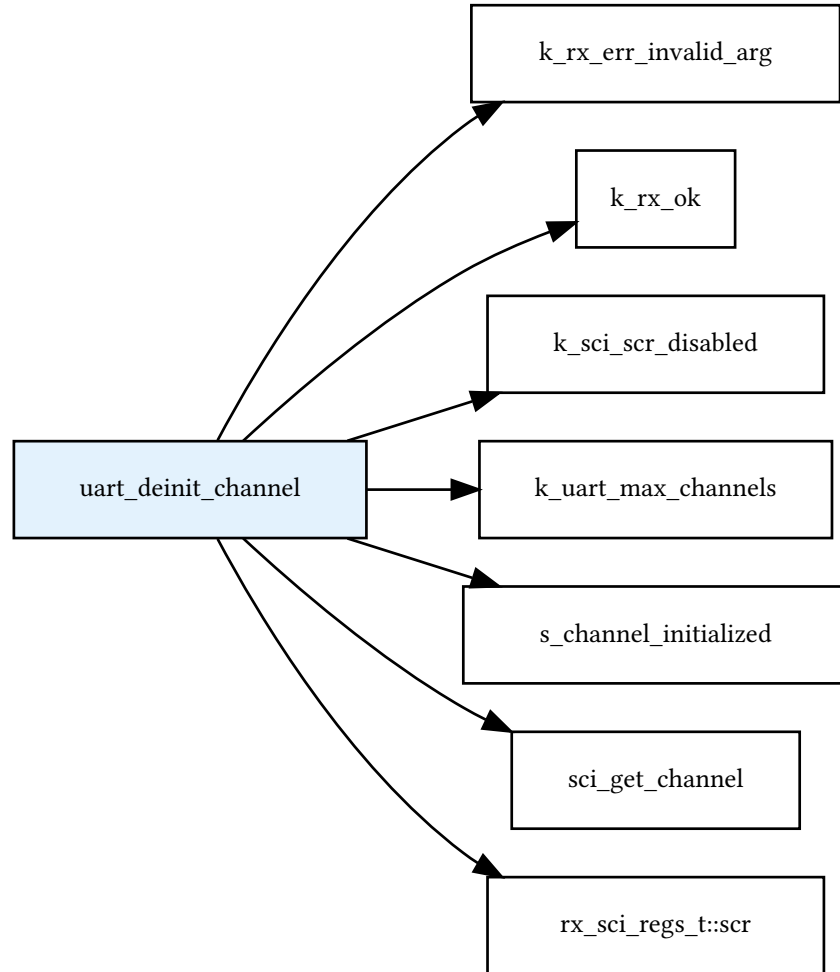


Figure 509: Call graph for `uart_deinit_channel`

_Defined in `libs/rx\_hal/inc/hardware.h:2194_`

Since Version 1.0.0

—

uart_putc_channel

```
rx_err_t uart_putc_channel(uart_channel_t channel, char data)
```

Transmit a single character on specified channel.

Transmit a single character on specified channel.

Transmits a single byte on the specified SCI channel. Waits for the transmit data register to become empty (TDRE flag) with timeout protection, then writes the data to TDR. Uses polling mode (no interrupts).

Algorithm steps:

1. Validate channel number (0-12)
2. Check channel is initialized
3. Get SCI register base address
4. Wait for TDRE flag with timeout (k_uart_tx_timeout iterations)
5. Write data byte to TDR register
6. Clear TDRE flag (read SSR, write back with TDRE=0)

Character transmission timing at 115200 baud:

- Start bit: 8.68 us
- 8 data bits: 69.44 us
- Stop bit: 8.68 us
- Total: 86.8 us per character

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
in	data	Character to transmit
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12) Recommended: Use k_uart_channel_X constants
in	data	Character to transmit Valid range: 0x00 to 0xFF (any byte value) Special handling: ' ' not converted (use uart_puts_channel for conversion)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, character transmitted
k_rx_err_invalid_arg	Invalid channel number (>=13)
k_rx_err_invalid_state	Channel not initialized
k_rx_err_timeout	Transmit buffer did not become empty within timeout

Pre: Channel must be initialized via uart_init_channel()

Pre: UART TX must be enabled (happens during init)

Post: Character written to TDR and transmission started

Post: TDRE flag cleared (TDR occupied)

Note: Blocking call - waits for TDRE flag

Note: Timeout prevents infinite wait if hardware fails

Warning: May block for up to 100ms if TX line is held low

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 90 us @ 115200 baud (one character time) Stack usage: 16 bytes

Example:: `rx_err_t err=uart_putc_channel(k_uart_channel_9,'A'); if(err!=k_rx_ok){ }
uart_putc_channel(k_uart_channel_9,'r'); uart_putc_channel(k_uart_channel_9,'n');`

See also: `uart_puts_channel()` Transmit string with newline conversion,
`uart_write_channel()` Transmit binary data

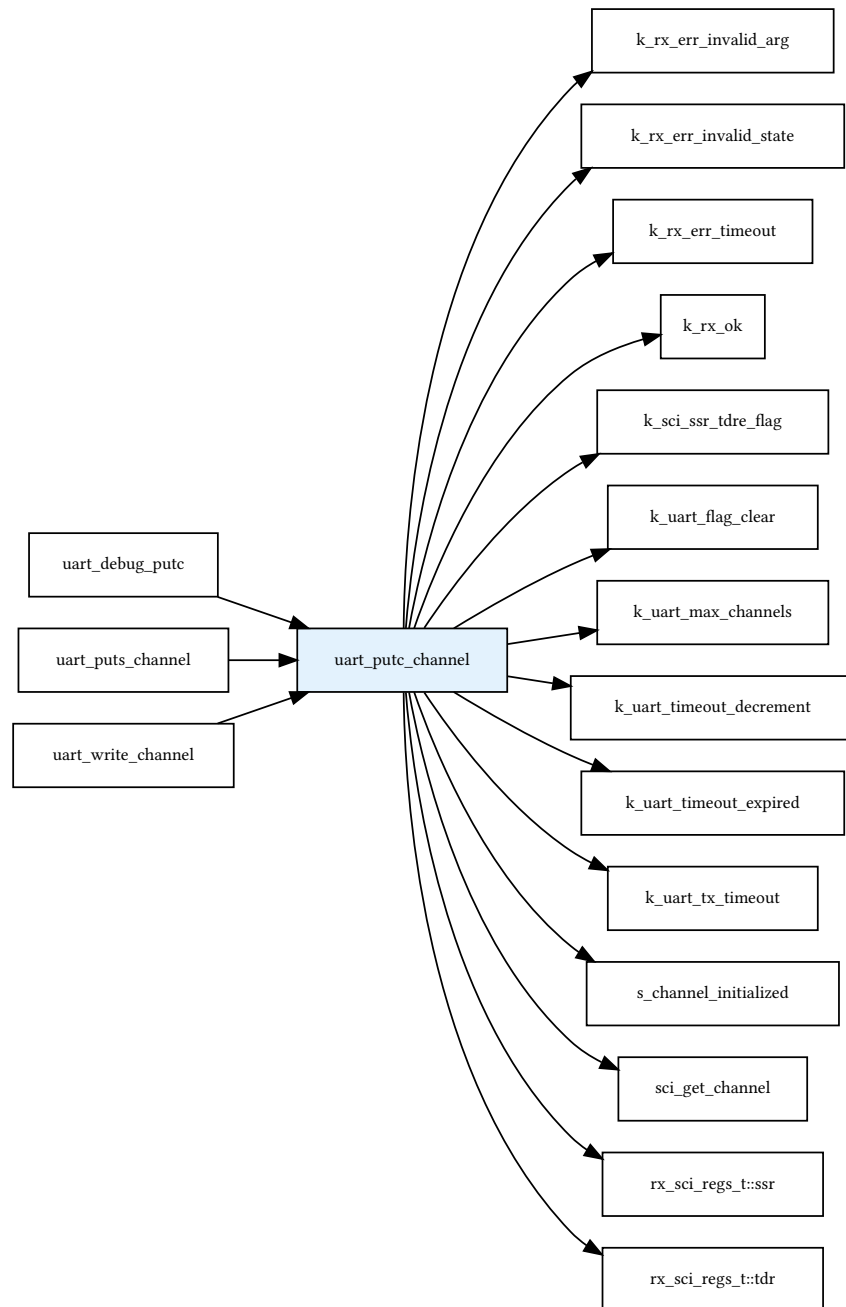


Figure 510: Call graph for `uart_putc_channel`

_Defined in `libs/rx_hal/inc/hardware.h:2206_`

Since Version 1.0.0

uart_puts_channel

```
rx_err_t uart_puts_channel(uart_channel_t channel, const char *str)
```

Transmit a null-terminated string on specified channel.

Transmit a null-terminated string on specified channel.

Transmits a null-terminated string with automatic LF to CR+LF conversion for terminal compatibility. Enforces maximum string length (k_uart_max_str_len = 256) to comply with NASA Power of 10 Rule 2 (bounded loops).

Algorithm steps:

1. Validate string pointer (NULL check)
2. Validate channel number and initialization state
3. For each character up to k_uart_max_str_len: a. Check for null terminator (success exit) b. If ‘
’, send ‘r’ first (CR+LF conversion) c. Transmit character via uart_putc_channel() d. Propagate any errors immediately
4. If limit reached without terminator, return k_rx_err_invalid_size

Newline conversion:

- Input ‘
(LF) -> Output ‘r’ (CR+LF)
- Required for Windows terminals (e.g., PuTTY, TeraTerm)
- Ensures cursor returns to column 0 before line feed

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
in	str	Pointer to string
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
in	str	Pointer to null-terminated string Valid range: Non-nullptr to valid memory Maximum length: 256 characters (k_uart_max_str_len) Null handling: Returns k_rx_err_null_ptr if nullptr Encoding: ASCII (extended characters passed through)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, entire string transmitted
k_rx_err_null_ptr	str parameter is nullptr
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized
k_rx_err_invalid_size	String exceeds 256 characters
k_rx_err_timeout	Hardware timeout during transmission

Pre: Channel must be initialized via `uart_init_channel()`

Pre: `str` must point to null-terminated string in valid memory

Post: All characters transmitted including converted newlines

Post: On error, partial string may have been transmitted

Note: Blocking call - waits for each character to transmit

Note: String length limited to 256 characters for safety

Warning: Long strings may take significant time (256 chars 22ms @ 115200)

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes Example: 100-char string 9 ms

Example:: `uart_puts_channel(k_uart_channel_9, "Hello, World!\n");`
`uart_puts_channel(k_uart_channel_9, "Line1\nLine2\nLine3\n");`
`rx_err_t err = uart_puts_channel(k_uart_channel_9, msg); if (err != k_rx_ok) {}`

See also: `uart_putc_channel()` Transmit single character, `uart_write_channel()` Transmit binary data (no conversion), `uart_debug_puts()` Simplified debug wrapper

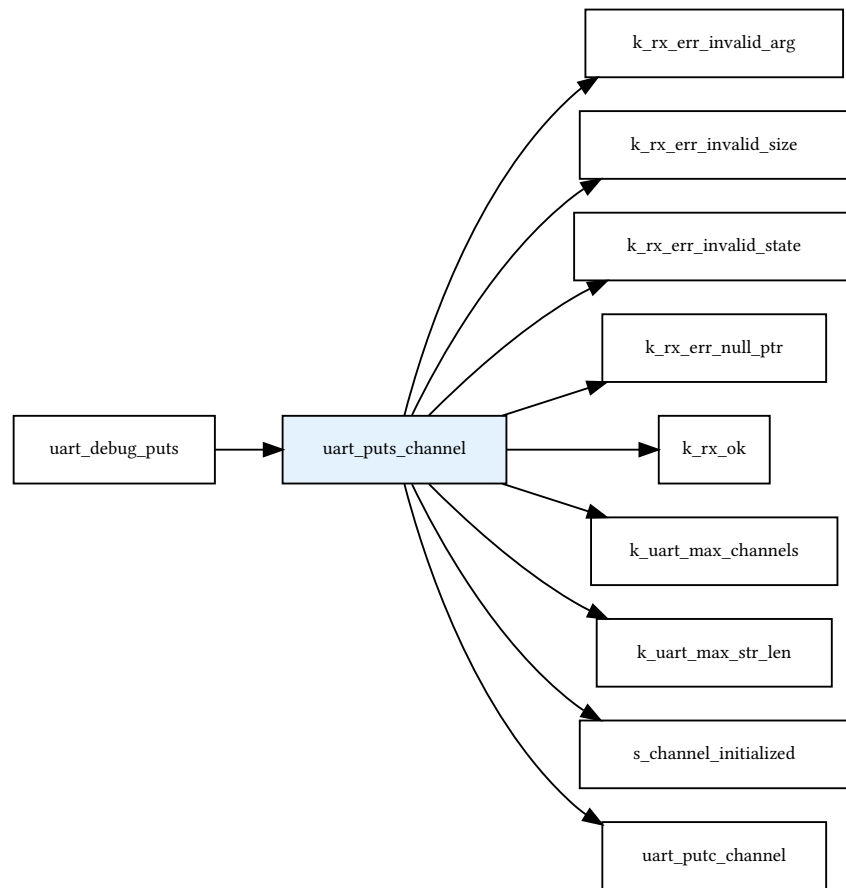


Figure 511: Call graph for uart_puts_channel

Defined in libs/rx\hal/inc/hardware.h:2219

Since Version 1.0.0

—

uart_write_channel

```
rx_err_t uart_write_channel(uart_channel_t channel, const uint8_t *data, uint16_t length)
```

Write buffer to specified channel.

Write buffer to specified channel.

Transmits a block of raw bytes on the specified SCI channel using uart_putc_channel() for each byte. No LF-to-CR+LF conversion is performed, making this function suitable for binary protocol data.

Algorithm steps:

1. Validate data pointer (NULL check)
2. Validate channel number and initialization state

3. For each byte in [0, length): call `uart_putc_channel()` and propagate errors

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
in	data	Pointer to data buffer
in	length	Number of bytes to write
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
in	data	Pointer to buffer containing bytes to transmit Valid range: Non-nullptr pointing to at least length bytes Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr
in	length	Number of bytes to write Valid range: 0 to <code>UINT16_MAX</code> ; 0 returns <code>k_rx_ok</code> immediately

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, all bytes transmitted
<code>k_rx_err_null_ptr</code>	data parameter is nullptr
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_invalid_state</code>	Channel not initialized
<code>k_rx_err_timeout</code>	Hardware timeout during transmission

Pre: Channel must be initialized via `uart_init_channel()`

Pre: data must point to at least length bytes of valid memory

Post: All length bytes transmitted in order

Post: On error, partial data may have been transmitted

Note: Blocking call - waits for each byte to be accepted by TDR

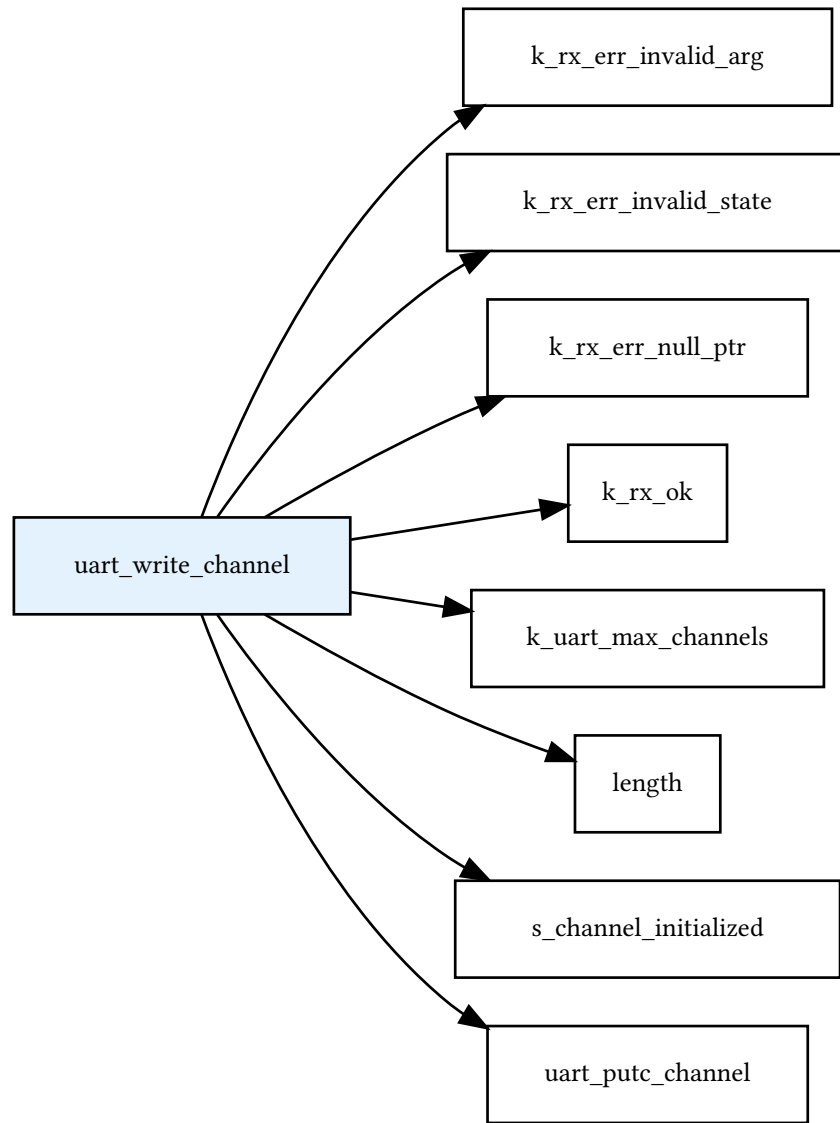
Note: No newline conversion - use `uart_puts_channel()` for text output

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 90 us per byte @ 115200 baud Stack usage: 24 bytes

Example:: `constuint8_tframe[]={0xAA,0x55,0x01,0x02,0x03};
rx_err_terr=uart_write_channel(k_uart_channel_9,frame,sizeof(frame)); if(err!=k_rx_ok){ }`

See also: `uart_puts_channel()` Transmit text string with newline conversion,
`uart_read_channel()` Read binary data

Figure 512: Call graph for `uart_write_channel`

_Defined in `libs/rx_hal/inc/hardware.h:2234_`

Since Version 1.0.0

—

uart_getc_channel

```
rx_err_t uart_getc_channel(uart_channel_t channel, char *data)
```

Receive a single character from specified channel (non-blocking).

Receive a single character from specified channel (non-blocking).

Attempts to read a single byte from the specified SCI channel's receive data register (RDR). This is a non-blocking operation - returns immediately with `k_rx_err_empty` if no data is available.

Algorithm steps:

1. Validate data pointer and channel number
2. Check channel initialization state
3. Get SCI register base address
4. Clear any error flags (ORER, FER, PER)
5. Check RDRF flag (Receive Data Register Full)

If not set, return `k_rx_err_empty` immediately

6. Read data byte from RDR register
7. Clear RDRF flag (read SSR, write back with RDRF=0)

Error flag handling:

- ORER (Overrun Error): Previous data overwritten before read
- FER (Framing Error): Stop bit not detected
- PER (Parity Error): Parity mismatch (not used in 8N1 mode)

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
out	data	Pointer to store received character
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
out	data	Pointer to store received character Valid range: Non-nullptr to char On success: Contains received byte (0x00-0xFF) On error: Content undefined Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, received character stored in *data
<code>k_rx_err_null_ptr</code>	data parameter is nullptr
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_invalid_state</code>	Channel not initialized
<code>k_rx_err_empty</code>	No data available (RDRF not set)

Pre: Channel must be initialized via `uart_init_channel()`

Pre: data must point to valid memory for single char

Post: On success, *data contains received byte

Post: RDRF flag cleared (ready for next byte)

Post: Error flags cleared if any were set

Note: Non-blocking - returns immediately if no data

Note: Clears error flags automatically before read

Note: Single byte buffer - call frequently to avoid overrun

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 5 us (no wait) Stack usage: 16 bytes

Example (Polling Loop):: charc; rx_err_terr;

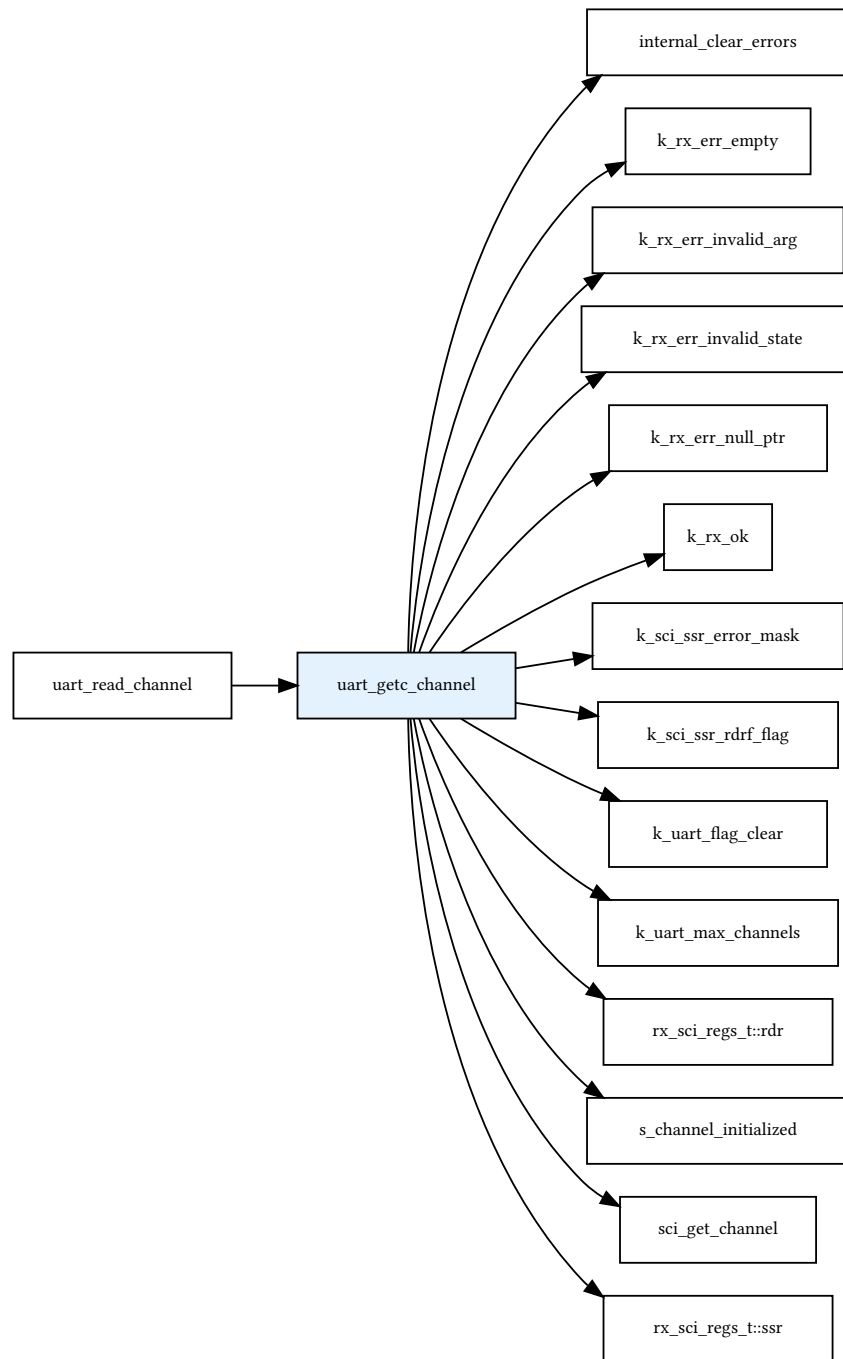
```
do{ err=uart_getc_channel(k_uart_channel_9,&c); if(err==k_rx_ok){ process_character(c); } }
while(err==k_rx_ok);
```

Example (Timeout Loop):: charc; uint32_ttimeout=100000;

```
while(timeout>0){ rx_err_terr=uart_getc_channel(k_uart_channel_9,&c); if(err==k_rx_ok){ break; }
timeout--; }
```

```
if(timeout==0){ }
```

See also: `uart_read_channel()` Read multiple bytes, `uart_rx_available()` Check if data is available

Figure 513: Call graph for `uart_getc_channel`

_Defined in `libs/rx_hal/inc/hardware.h:2248_`

Since Version 1.0.0

—

uart_read_channel

```
rx_err_t uart_read_channel(uart_channel_t channel, uint8_t *data, uint16_t
length, uint16_t *bytes_read)
```

Read available data from specified channel (non-blocking).

Reads up to length bytes that are currently available.

Read available data from specified channel (non-blocking).

Reads up to length bytes from the specified SCI channel using `uart_getc_channel()`. Stops immediately when no more data is available (RDRF flag not set) rather than waiting. Actual bytes received is reported via `bytes_read`.

Algorithm steps:

1. Validate data and bytes_read pointers (NULL check)
2. Validate channel number and initialization state
3. Set *bytes_read = 0
4. For each slot in [0, length): a. Call `uart_getc_channel()`; if `k_rx_err_empty`, break (done) b. On other error, propagate immediately c. Store byte and increment *bytes_read
5. Return `k_rx_ok` (even if zero bytes were read)

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
out	data	Pointer to buffer for received data
in	length	Maximum number of bytes to read
out	bytes_read	Pointer to store actual bytes read
in	channel	UART channel to read from (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
out	data	Pointer to buffer to store received bytes Valid range: Non-nullptr pointing to at least length bytes Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr
in	length	Maximum number of bytes to read Valid range: 0 to <code>UINT16_MAX</code>
out	bytes_read	Pointer to store actual number of bytes read Valid range: Non-nullptr to <code>uint16_t</code> On success: Set to number of bytes actually received (0..length) Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success; *bytes_read contains actual count (may be 0)
<code>k_rx_err_null_ptr</code>	data or bytes_read is nullptr

k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized

Pre: Channel must be initialized via `uart_init_channel()`

Pre: data must point to at least length bytes of writable memory

Post: `*bytes_read` set to actual number of bytes received

Post: `data[0..bytes_read-1]` contain the received bytes

Note: Non-blocking - returns immediately with whatever data is available

Note: `k_rx_ok` with `*bytes_read == 0` means no data was available

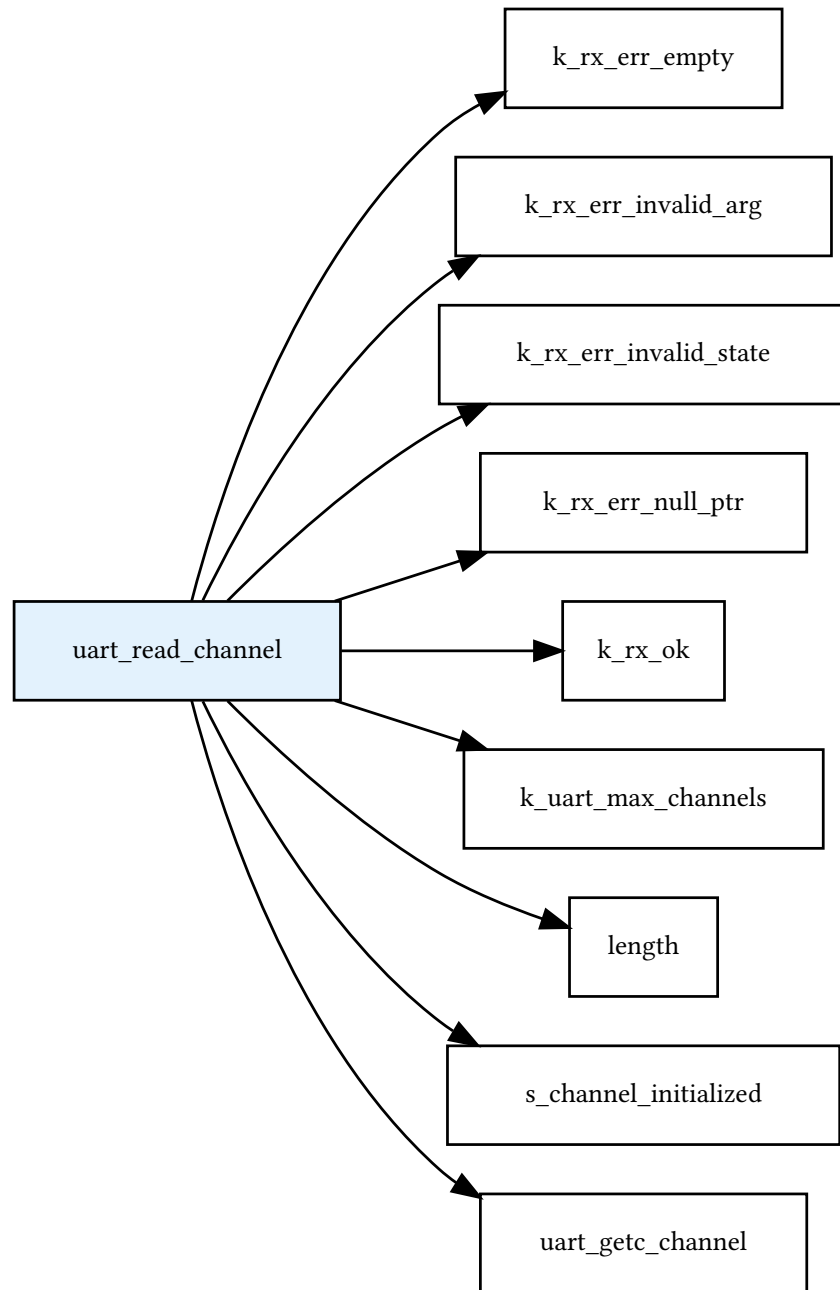
Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 5 us per byte received + 5 us when no data Stack usage: 24 bytes

Example::

```
uint8_t buf[64]; uint16_t count=0;
rx_err_t err=uart_read_channel(k_uart_channel_9,buf,sizeof(buf),&count);
if(err==k_rx_ok&&count>0){ process_bytes(buf,count); }
```

See also: `uart_getc_channel()` Read single character, `uart_rx_available()` Check if data is available, `uart_write_channel()` Write binary data

Figure 514: Call graph for `uart_read_channel`

_Defined in `libs/rx_hal/inc/hardware.h:2266_`

Since Version 1.0.0

—

uart_rx_available

```
rx_err_t uart_rx_available(uart_channel_t channel, bool *available)
```

Check if receive data is available on channel.

Check if receive data is available on channel.

Checks the RDRF (Receive Data Register Full) flag in the SSR register of the specified SCI channel. Provides a non-destructive peek at receive availability without consuming any data from the buffer.

Algorithm steps:

1. Validate available pointer (NULL check)
2. Validate channel number and initialization state
3. Get SCI register base address
4. Read SSR and test RDRF bit; store boolean result in *available

Parameters:

Direction	Name	Description
in	channel	SCI channel (0-12)
out	available	Pointer to store availability status
in	channel	UART channel to check (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
out	available	Pointer to store result On success: true if RDRF flag set (data ready), false otherwise On error: value undefined Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success; *available set to data availability status
k_rx_err_null_ptr	available is nullptr
k_rx_err_invalid_arg	Invalid channel number or invalid register pointer
k_rx_err_invalid_state	Channel not initialized

Pre: Channel must be initialized via uart_init_channel()

Pre: available must point to valid bool storage

Post: *available reflects current RDRF state

Post: No data is consumed from the receive buffer

Note: Non-destructive - does not read RDR or clear any flags

Note: RDRF can be set again immediately after reading if UART is receiving

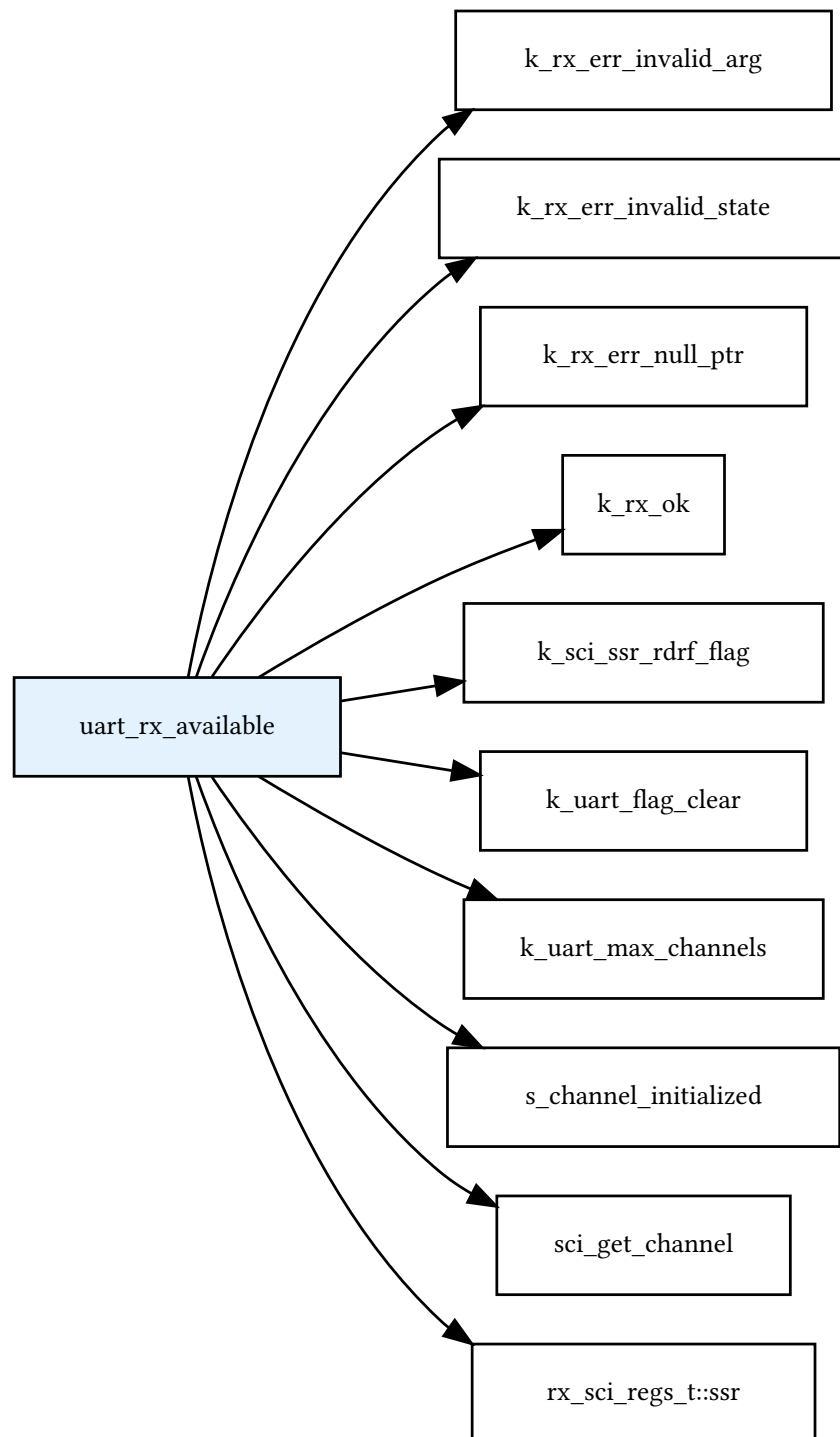
Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 2 us Stack usage: 16 bytes

Example::

```
boolready=false; if(uart_rx_available(k_uart_channel_9,&ready)==k_rx_ok&&ready)
{ charc; uart_getc_channel(k_uart_channel_9,&c); }
```

See also: `uart_getc_channel()` Read single character, `uart_read_channel()` Read multiple bytes

Figure 515: Call graph for `uart_rx_available`

_Defined in `libs/rx\_hal/inc/hardware.h:2279_`

Since Version 1.0.0

—

uart_debug_init

```
rx_err_t uart_debug_init(void)
```

Initialize debug UART (SCI9, 115200 bps, 8N1).

Convenience wrapper for `uart_init_channel(k_uart_debug_channel, 115200)`. Used by `rx_log` for early boot logging before ThreadX is initialized.

Initialize debug UART (SCI9, 115200 bps, 8N1).

Convenience function to initialize SCI9 with default debug settings:

- Channel: SCI9
- Baud rate: 115200 bps
- TX pin: PB7 (TXD9)
- RX pin: PB6 (RXD9)
- Format: 8N1 (8 data bits, no parity, 1 stop bit)

SCI9 is connected to the CY7C65213 USB-UART bridge on the STAR PCB, providing a virtual COM port when connected via USB.

Algorithm steps:

1. Create `uart_channel_config_t` with default values
2. Call `uart_init_channel()` with the configuration
3. Return result

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, debug UART ready
<code>k_rx_err_invalid_state</code>	SCI9 already initialized

Pre: System clocks must be initialized (PCLKB = 60 MHz)

Pre: SCI9 must not be already initialized

Post: SCI9 configured and ready for TX/RX at 115200 baud

Post: `uart_debug_puts()`, `uart_debug_putc()` etc. are functional

Note: Call once during early system initialization

Note: Call before any other debug output functions

Warning: Must be called before ThreadX starts if debug output is needed during init

Thread Safety:: Not thread-safe. Call once during startup before ThreadX.

Performance:: Execution time: 50 us Stack usage: 64 bytes

Example:: void system_init(void) { rx_clock_init();

rx_err_t err = uart_debug_init(); if (err != k_rx_ok) { return; }

uart_debug_puts("[INFO]System starting...n"); }

See also: uart_init_channel() Generic channel initialization, uart_debug_puts() Debug string output, uart_debug_putc() Debug character output

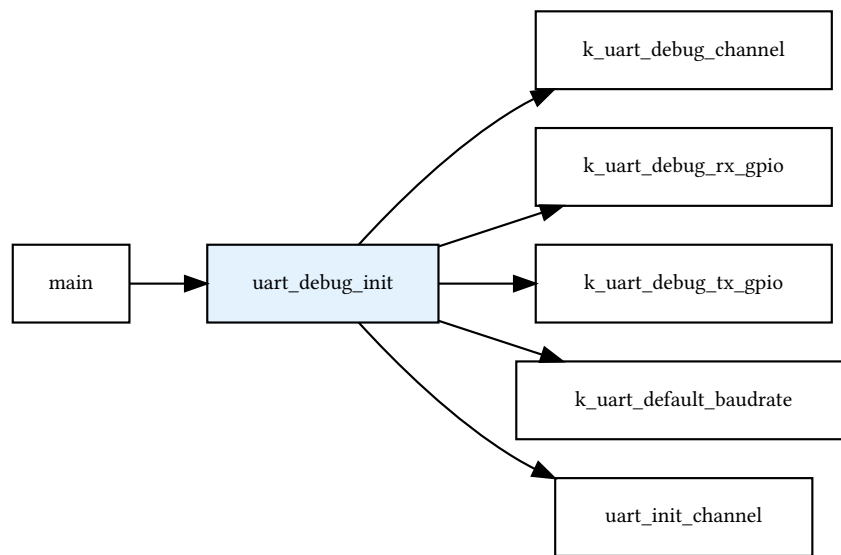


Figure 516: Call graph for `uart_debug_init`

_Defined in `libs/rx\hal/inc/hardware.h:2294_`

Since Version 1.0.0

—

uart_debug_putc

```
void uart_debug_putc(char data)
```

Transmit a single character on debug UART (SCI9).

Transmit a single character on debug UART (SCI9).

Convenience wrapper around `uart_putc_channel()` for the fixed debug channel (SCI9). Errors are silently ignored to allow use in early initialization contexts where error propagation is not yet possible.

Algorithm steps:

1. Call `uart_putc_channel(k_uart_debug_channel, data)`

2. Cast return value to (void) - errors discarded

Parameters:

Direction	Name	Description
in	data	Character to transmit
in	data	Character to transmit (0x00-0xFF) Special handling: ‘ ‘ is NOT converted; use uart_debug_puts() for text

Pre: uart_debug_init() must have been called successfully

Pre: SCI9 must be initialized and TX enabled

Post: Character written to SCI9 TDR and transmission started

Post: Any errors are silently discarded

Note: Error return from uart_putc_channel() is intentionally discarded

Note: For error-checked output use uart_putc_channel() directly

Warning: Only available when RX_IS_SIMULATOR is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us @ 115200 baud Stack usage: 16 bytes

Example:: uart_debug_putc('A'); uart_debug_putc('r'); uart_debug_putc('n');

See also: uart_debug_puts() Transmit string with newline conversion, uart_putc_channel() Error-checked single character transmit

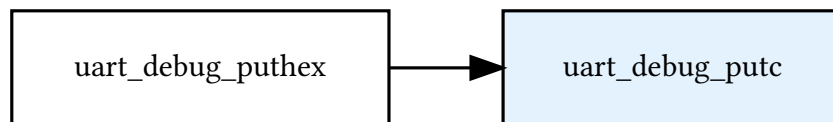


Figure 517: Call graph for uart_debug_putc

_Defined in libs/rx_hal/inc/hardware.h:2301_

Since Version 1.0.0

—

uart_debug_puts

```
void uart_debug_puts(const char *str)
```

Transmit a null-terminated string on debug UART (SCI9).

Transmit a null-terminated string on debug UART (SCI9).

Transmits a null-terminated string to SCI9 with automatic LF-to-CR+LF conversion for terminal compatibility. Enforces a maximum length of `k_uart_max_str_len` (256) characters per NASA Power of 10 Rule 2. Silently returns on NULL pointer to allow safe use in early init.

Algorithm steps:

1. Return immediately if `str` is `nullptr` (defensive, no error return)
2. For each character up to `k_uart_max_str_len`: a. Stop at null terminator b. If ‘
‘, send ‘r’ first c. Send character via `uart_debug_putc()`

Parameters:

Direction	Name	Description
in	<code>str</code>	Pointer to string
in	<code>str</code>	Pointer to null-terminated ASCII string Maximum length: 256 characters (<code>k_uart_max_str_len</code>) Null handling: Returns silently if <code>nullptr</code> Encoding: ASCII; ‘ ‘ converted to ‘r ‘

Pre: `uart_debug_init()` must have been called successfully

Pre: `str` should point to a null-terminated string in valid memory

Post: All characters up to null terminator transmitted to SCI9

Post: Each ‘] ‘ replaced by ‘r ‘ in the transmitted output

Note: No return value - errors from `uart_debug_putc()` are silently discarded

Note: String truncated at `k_uart_max_str_len` (256) characters

Warning: Only available when `RX_IS_SIMULATOR` is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes

Example:: `uart_debug_puts("Hello,World!\n");` `uart_debug_puts("[INFO]Bootcompleten");`

See also: `uart_debug_putc()` Transmit single character, `uart_debug_putint()` Transmit decimal integer, `uart_puts_channel()` Error-checked string transmit

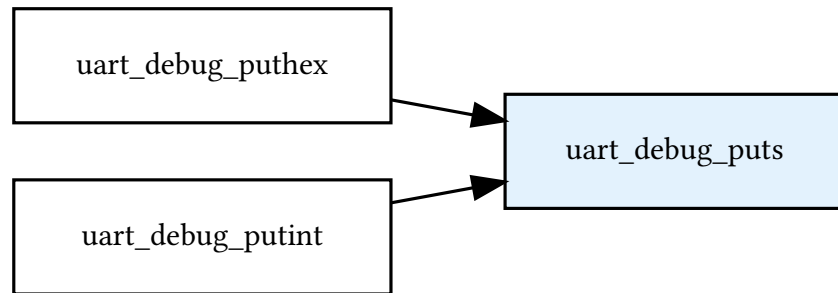


Figure 518: Call graph for uart_debug_puts

_Defined in libs/rx_hal/inc/hardware.h:2308_

Since Version 1.0.0

—

uart_debug_putint

```
void uart_debug_putint(int32_t value)
```

Print a signed integer on debug UART (SCI9).

Print a signed integer on debug UART (SCI9).

Converts a signed 32-bit integer to a decimal ASCII string and transmits it on SCI9. Handles INT32_MIN correctly by using int64_t for the negation. Uses a fixed-size stack buffer (k_uart_int_buffer_size = 12) built in reverse then passed to uart_debug_puts().

Algorithm steps:

1. Determine sign; compute abs_value as uint32_t
2. Null-terminate the buffer end
3. Build digits right-to-left (statically bounded by k_uart_int_buffer_size)
4. Prepend '-' if negative
5. Call uart_debug_puts() with the resulting substring pointer

Parameters:

Direction	Name	Description
in	value	Integer value to print
in	value	Signed 32-bit integer to print Valid range: INT32_MIN (-2147483648) to INT32_MAX (2147483647) INT32_MIN handling: Correctly handled via int64_t cast

Pre: uart_debug_init() must have been called successfully

Pre: uart_debug_puts() must be functional

Post: Decimal representation of value transmitted on SCI9

Post: Leading zeros suppressed; negative values prefixed with ‘-’

Note: No return value - output errors are silently discarded

Note: Buffer is stack-allocated; safe for re-entrant calls on different tasks

Warning: Only available when RX_IS_SIMULATOR is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per digit @ 115200 baud + digit-loop overhead Stack usage: 32 bytes (buffer + locals)

Example:: `uart_debug_puts("Value:"); uart_debug_putint(42); uart_debug_putint(-2147483648);
uart_debug_putc('n');`

See also: `uart_debug_puthex()` Print value in hexadecimal, `uart_debug_puts()` Underlying string transmit

Defined in `libs/rx\hal/inc/hardware.h:2315_`

Since Version 1.0.0

—

uart_debug_puthex

```
void uart_debug_puthex(uint32_t value, uint8_t digits)
```

Print a hexadecimal value on debug UART (SCI9).

Print a hexadecimal value on debug UART (SCI9).

Transmits “0x” prefix followed by the specified number of uppercase hex digits of `value` on SCI9. Digits are always printed most-significant first. The `digits` parameter is clamped to `[k_uart_hex_min_digits, k_uart_hex_max_digits]` (1-8) before use.

Algorithm steps:

1. Send “0x” prefix via `uart_debug_puts()`
2. Clamp digits to `[1, 8]`
3. Iterate nibbles from MSN to LSN (statically bounded by `k_uart_hex_max_digits`)
4. For each nibble index < digits, extract nibble and print uppercase hex char

Parameters:

Direction	Name	Description
in	<code>value</code>	Value to print
in	<code>digits</code>	Number of hex digits (1-8)
in	<code>value</code>	Unsigned 32-bit value to display in hexadecimal Valid range: 0x00000000 to 0xFFFFFFFF

in	digits	Number of hex digits to print (1-8) Valid range: 1 to 8 (clamped; 0 -> 1, >8 -> 8) Common values: 2 for byte, 4 for word, 8 for full 32-bit
----	--------	---

Pre: `uart_debug_init()` must have been called successfully

Pre: `uart_debug_puts()` and `uart_debug_putc()` must be functional

Post: “0x” followed by digits uppercase hex characters transmitted on SCI9

Post: digits clamped to 1, 8 if out of range

Note: Uses static lookup table `s_hex` for digit-to-character conversion

Note: Always prefixes output with “0x”

Warning: Only available when `RX_IS_SIMULATOR` is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes

Example:: `uart_debug_puts("Addr:"); uart_debug_puthex(0xDEADBEEF,8);`
`uart_debug_puthex(0x42,2); uart_debug_putc('n');`

See also: `uart_debug_putint()` Print signed decimal value, `uart_debug_puts()` Underlying string transmit

_Defined in `libs/rx\_hal/inc/hardware.h:2323_`

Since Version 1.0.0

—

rx72n_adc_regs.h

RX72N S12AD 12-bit A/D Converter Register Definitions.

This header provides comprehensive register definitions for the RX72N’s 12-bit Successive Approximation A/D Converter (S12ADFa). The S12AD is essential for motor current sensing in the STAR project’s DRV8263H H-bridge motor control system.

STAR Team

2026-01-28

1.0.0

MIT License

Includes:

- `<stddef.h>`
- `<stdint.h>`

rx72n_clock.h

RX72N Clock Frequency Definitions and Configuration.

Defines all system clock frequencies for the RX72N microcontroller configured with external 12 MHz crystal and PLL for maximum 240 MHz CPU operation. These constants are used throughout the firmware for timing calculations, baud rate generation, PWM frequency configuration, and peripheral initialization.

Includes:

- `<stdint.h>`

rx_clock_frequencies_t

System clock frequencies for RX72N @ 240 MHz operation.

Defines all clock domain frequencies for the STAR project's RX72N configuration. These values are determined by the PLL and clock divider settings in `rx_clock_power_init()` and are guaranteed to be accurate for timing calculations.

Clock Source: 12 MHz external crystal -> PLL (x40) -> 480 MHz -> divided per domain

Frequency Derivation:

- PLL output: 12 MHz x 40 = 480 MHz
- ICLK: 480 MHz / 2 = 240 MHz (CPU)
- PCLKA: 240 MHz / 2 = 120 MHz (high-speed peripherals)
- PCLKB/C/D: 240 MHz / 4 = 60 MHz (standard peripherals)
- FCLK: 240 MHz / 4 = 60 MHz (flash - max allowed)
- BCLK: 240 MHz / 2 = 120 MHz (external bus - not used in STAR)

Value	Name	Description
= 240000000UL	k_iclk_hz	CPU core clock frequency (ICLK) - 240 MHz.
= 120000000UL	k_pclka_hz	Peripheral Clock A (PCLKA) - 120 MHz.
= 60000000UL	k_pclkb_hz	Peripheral Clock B (PCLKB) - 60 MHz.
= 60000000UL	k_pclkc_hz	Peripheral Clock C (PCLKC) - 60 MHz (unused in STAR project).
= 60000000UL	k_pclkd_hz	Peripheral Clock D (PCLKD) - 60 MHz.
= 120000000UL	k_bclk_hz	External Bus Clock (BCLK) - 120 MHz (unused in STAR project).
= 60000000UL	k_fclk_hz	Flash Memory Clock (FCLK) - 60 MHz (hardware limit).

—

rx72n_cmt_regs.h

RX72N Compare Match Timer (CMT) Register Definitions.

Register definitions for the Compare Match Timer (CMT) peripheral on the RX72N microcontroller. CMT provides simple periodic interrupt generation used for RTOS system tick and general timing purposes.

Where:

- Tperiod = Timer period in seconds
- CMCOR = Compare match register value (0-65535)
- DIV = Clock divider (8, 32, 128, or 512)
- fPCLKB = Peripheral clock B frequency (60 MHz)

2026-01-28

Copyright (c) 2026 STAR Project

Includes:

- <stddef.h>
- <stdint.h>

rx72n_crc_regs.h

RX72N CRC Calculator Register Definitions.

Register definitions for the hardware CRC Calculator used for CRC-32 acceleration. The CRC module provides hardware-accelerated cyclic redundancy check computation, significantly faster than software implementations.

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx_crc_addresses_t

CRC module base address.

Base address for the CRC Calculator register block. The CRC module provides hardware-accelerated CRC computation with configurable polynomials.

Value	Name	Description
= 0x00088280	k_crc_base_addr	CRC register base address (0x00088280).

See also: `crc_regs()` Accessor function

Since Version 1.0.0

—

rx_crc_reserved_sizes_t

CRC register reserved field sizes.

Defines reserved byte counts for structure padding to match hardware layout.

Value	Name	Description
= 3	k_crc_reserved_after_crccr_bytes	Reserved bytes @ 0x01-0x03

Since Version 1.0.0

—

rx_crc_crccr_shifts_t

CRC Control Register (CRCCR) Bit Field Positions.

Bit positions for CRCCR register fields. Used to construct field values.

Value	Name	Description
= 1	k_crc_bit_mask	Single-bit mask
= 6	k_crc_crccr_lms_shift	LMS field position (bit 6)
= 7	k_crc_crccr_dorclr_shift	DORCLR field position (bit 7)

Since Version 1.0.0

—

rx_crc_crccr_masks_t

CRC Control Register (CRCCR) Bit Masks.

Bit masks for isolating specific fields in the CRCCR register.

Value	Name	Description
= (k_crc_bit_mask << k_crc_crccr_lms_shift)	k_crc_crccr_lms_mask	LSB/ MSB First mask (bit 6)
= (k_crc_bit_mask << k_crc_crccr_dorclr_shift)	k_crc_crccr_dorclr_mask	DORCLR mask (bit 7)

Since Version 1.0.0

—

rx_crc_crccr_bits_t

CRC Control Register (CRCCR) Bit Definitions.

Configuration values for CRC polynomial selection and bit order. Only CRC-32 IEEE 802.3 is used in the STAR project.

For IEEE 802.3 compatibility (Go crc32.ChecksumIEEE), use LSB-first.

Value	Name	Description
= 0x03	k_crc_crccr_gps_crc32	CRC-32 IEEE 802.3 polynomial (GPS=0x03).
= k_crc_crccr_lms_mask	k_crc_crccr_lms	LSB-first (reflected) bit order (LMS=1).
= k_crc_crccr_dorclr_mask	k_crc_crccr_dorclr	Clear Data Output Register (DORCLR=1).

See also: rx_crc_regs_t::crccr Control register

Since Version 1.0.0

—

crc_regs

```
static volatile rx_crc_regs_t * crc_regs(void)
```

Get pointer to CRC registers.

Returns a volatile pointer to the CRC Calculator register structure at address 0x00088280. The CRC module provides hardware-accelerated CRC computation for protocol integrity checking.

Returns: Volatile pointer to CRC register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: CRC module must be enabled (MSTPCRB.MSTPB23 = 0)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Warning: Ensure module stop bit is cleared before accessing registers

Example:: `volatile rx_crc_regs_t *crc=crc_regs(); crc->cccr=k_crc_cccr_gps_crc32|k_crc_cccr_lms|k_crc_cccr_doreclr; crc->crkdir=data_word; uint32_tresult=crc->crclor^0xFFFFFFFF;`

See also: `k_crc_base_addr` Base address constant, `rx_crc32_init()` Higher-level initialization

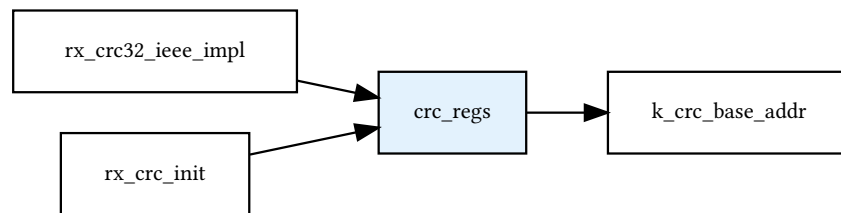


Figure 519: Call graph for `crc_regs`

_Defined in `libs/rx_hal/inc/rx72n_crc_regs.h:309_`

Since Version 1.0.0

—

rx72n_dmac_regs.h

RX72N DMA Controller (DMAC) Register Definitions.

Complete register definitions for the RX72N 8-channel DMA Controller. The DMAC transfers data without CPU intervention, supporting normal, repeat, and block transfer modes.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

dmac_addresses_t

DMAC module addresses.

Base addresses for DMAC channels and shared registers. All addresses verified against RX72N Hardware Manual Ch18.

Value	Name	Description
= 0x00082000U	k_dmac_base_addr	DMAC module base address (channel 0).
= 0x40U	k_dmac_channel_size	Channel register block size (0x40 = 64 bytes per channel).
= 0x00082000U	k_dmac0_base_addr	DMAC channel 0 base address.
= 0x00082040U	k_dmac1_base_addr	DMAC channel 1 base address.
= 0x00082080U	k_dmac2_base_addr	DMAC channel 2 base address.
= 0x000820C0U	k_dmac3_base_addr	DMAC channel 3 base address.
= 0x00082100U	k_dmac4_base_addr	DMAC channel 4 base address.
= 0x00082140U	k_dmac5_base_addr	DMAC channel 5 base address.
= 0x00082180U	k_dmac6_base_addr	DMAC channel 6 base address.
= 0x000821C0U	k_dmac7_base_addr	DMAC channel 7 base address.
= 0x00082200U	k_dmac_dmast_addr	DMAC Module Start Register (DMAST) address.
= 0x00082204U	k_dmac_dmist_addr	DMAC74 Interrupt Status Monitor Register (DMIST) address.

Note: Addresses verified 2026-01-29 against manual section 18.2

Since Version 1.0.0

—

dmac_offsets_t

Per-channel register offsets.

Offsets from channel base address for each register. All offsets verified against RX72N Hardware Manual Ch18 section 18.2.

Value	Name	Description
= 0x00U	k_dmac_offset_dmsar	Source Address Register (32-bit)
= 0x04U	k_dmac_offset_dmdar	Destination Address Register (32-bit)

= 0x08U	k_dmac_offset_dmcrA	Transfer Count Register A (32-bit)
= 0x0CU	k_dmac_offset_dmcrB	Transfer Count Register B (16-bit)
= 0x10U	k_dmac_offset_dmtmd	Transfer Mode Register (16-bit)
= 0x13U	k_dmac_offset_dmint	Interrupt Setting Register (8-bit)
= 0x14U	k_dmac_offset_dmamd	Address Mode Register (16-bit)
= 0x18U	k_dmac_offset_dmoFr	Offset Register (32-bit, ch0 only!)
= 0x1CU	k_dmac_offset_dmcnt	DMA Transfer Enable Register (8-bit)
= 0x1DU	k_dmac_offset_dmreq	DMA Software Start Register (8-bit)
= 0x1EU	k_dmac_offset_dmsts	DMA Status Register (8-bit)
= 0x1FU	k_dmac_offset_dmcsl	DMA Channel Select Register (8-bit)

Note: Offsets verified 2026-01-29 against manual

Since Version 1.0.0

—

dmac_channel_t

DMAC channel numbers.

Channel indices for use with accessor functions.

Value	Name	Description
= 0U	k_dmac_channel_0	DMAC channel 0 (highest priority)
= 1U	k_dmac_channel_1	DMAC channel 1
= 2U	k_dmac_channel_2	DMAC channel 2
= 3U	k_dmac_channel_3	DMAC channel 3
= 4U	k_dmac_channel_4	DMAC channel 4
= 5U	k_dmac_channel_5	DMAC channel 5
= 6U	k_dmac_channel_6	DMAC channel 6
= 7U	k_dmac_channel_7	DMAC channel 7 (lowest priority)
= 8U	k_dmac_channel_count	Total number of DMAC channels

Since Version 1.0.0

—

dmtmd_bits_t

DMA Transfer Mode Register (DMTMD) bit definitions.

Controls transfer mode, data size, repeat area selection, and trigger source.

Value	Name	Description
= 0x0000U	k_dmtmd_dctg_software	Software trigger

= 0x0001U	k_dmtmd_dctg_interrupt	Peripheral/external interrupt
= 0x0003U	k_dmtmd_dctg_mask	DCTG field mask
= 0x0000U	k_dmtmd_sz_8bit	8-bit transfer
= 0x0100U	k_dmtmd_sz_16bit	16-bit transfer
= 0x0200U	k_dmtmd_sz_32bit	32-bit transfer
= 0x0300U	k_dmtmd_sz_mask	SZ field mask
= 0x0000U	k_dmtmd_dts_dest	Destination is repeat/block area
= 0x1000U	k_dmtmd_dts_source	Source is repeat/block area
= 0x2000U	k_dmtmd_dts_none	No repeat/block area
= 0x3000U	k_dmtmd_dts_mask	DTS field mask
= 0x0000U	k_dmtmd_md_normal	Normal transfer mode
= 0x4000U	k_dmtmd_md_repeat	Repeat transfer mode
= 0x8000U	k_dmtmd_md_block	Block transfer mode
= 0xC000U	k_dmtmd_md_mask	MD field mask

Note: Manual ref: Ch18 section 18.2.5

Since Version 1.0.0

—

dmint_bits_t

DMA Interrupt Setting Register (DMINT) bit definitions.

Controls interrupt enable for various DMA events.

Value	Name	Description
= 0x01U	k_dmint_darie	Dest addr extended repeat overflow int enable
= 0x02U	k_dmint_sarie	Source addr extended repeat overflow int enable
= 0x04U	k_dmint_rptie	Repeat size end interrupt enable
= 0x08U	k_dmint_esie	Transfer escape end interrupt enable
= 0x10U	k_dmint_dtie	Transfer end interrupt enable

Note: Manual ref: Ch18 section 18.2.6

Since Version 1.0.0

—

dmamd_bits_t

DMA Address Mode Register (DMAMD) bit definitions.

Controls source/destination address update mode and extended repeat area.

Value	Name	Description
= 0x0003U	k_dmamd_dara_mask	DARA field mask
= 0x0000U	k_dmamd_dm_fixed	Destination address fixed
= 0x0040U	k_dmamd_dm_offset	Add offset after transfer (ch0 only)
= 0x0080U	k_dmamd_dm_increment	Increment after transfer
= 0x00C0U	k_dmamd_dm_decrement	Decrement after transfer
= 0x00C0U	k_dmamd_dm_mask	DM field mask
= 0x0300U	k_dmamd_sara_mask	SARA field mask
= 0x0000U	k_dmamd_sm_fixed	Source address fixed
= 0x4000U	k_dmamd_sm_offset	Add offset after transfer (ch0 only)
= 0x8000U	k_dmamd_sm_increment	Increment after transfer
= 0xC000U	k_dmamd_sm_decrement	Decrement after transfer
= 0xC000U	k_dmamd_sm_mask	SM field mask

Note: Manual ref: Ch18 section 18.2.7

Since Version 1.0.0

—

dmcnt_bits_t

DMA Transfer Enable Register (DMCNT) bit definitions.

Value	Name	Description
= 0x01U	k_dmcnt_dte	DMA Transfer Enable (0=disabled, 1=enabled)

Note: Manual ref: Ch18 section 18.2.9

Since Version 1.0.0

—

dmreq_bits_t

DMA Software Start Register (DMREQ) bit definitions.

Value	Name	Description
= 0x01U	k_dmreq_swreq	Software start (write 1 to trigger)
= 0x10U	k_dmreq_clr	Auto-clear SWREQ after transfer start

Note: Manual ref: Ch18 section 18.2.10

Since Version 1.0.0

—

dmsts_bits_t

DMA Status Register (DMSTS) bit definitions.

Value	Name	Description
= 0x01U	k_dmsts_esif	Transfer escape end flag
= 0x10U	k_dmsts_dtif	Transfer end flag
= 0x80U	k_dmsts_act	DMA active flag (1=transfer in progress)

Note: Manual ref: Ch18 section 18.2.11

Since Version 1.0.0

—

dmcs1_bits_t

DMA Channel Select Register (DMCSL) bit definitions.

Value	Name	Description
= 0x01U	k_dmcs1_disel	Interrupt select (0=transfer end, 1=DTIF)

Note: Manual ref: Ch18 section 18.2.12

Since Version 1.0.0

—

dmast_bits_t

DMAC Module Start Register (DMAST) bit definitions.

Value	Name	Description
= 0x01U	k_dmast_dmst	DMAC module start (0=suspended, 1=operating)

Note: Manual ref: Ch18 section 18.2.13

Since Version 1.0.0

—

dmist_bits_t

DMAC74 Interrupt Status Monitor Register (DMIST) bit definitions.

Value	Name	Description
= 0x10U	k_dmist_dmis4	Channel 4 interrupt status (bit 4)
= 0x20U	k_dmist_dmis5	Channel 5 interrupt status (bit 5)
= 0x40U	k_dmist_dmis6	Channel 6 interrupt status (bit 6)
= 0x80U	k_dmist_dmis7	Channel 7 interrupt status (bit 7)

Note: Manual ref: Ch18 section 18.2.14

Since Version 1.0.0

—

dmac_mstpcr_t

DMAC module stop control bits.

The DMAC is controlled by MSTPCRA.MSTPA28. Must be cleared (0) before accessing DMAC registers.

Value	Name	Description
= (1U \<\< 28U)	k_dmac_mstpcra_bit	MSTPCRA bit for DMAC (bit 28).

Note: Manual ref: Ch11 (Low Power Consumption) module stop tables

Since Version 1.0.0

—

dmac_channel

```
static volatile rx_dmac_channel_regs_t * dmac_channel(uint8_t channel)
```

Get pointer to DMAC channel registers.

Parameters:

Direction	Name	Description
in	channel	Channel number (0-7)

Returns: Volatile pointer to channel register structure

Pre: channel < k_dmac_channel_count (8)

Pre: DMAC module stop bit (MSTPCRA.MSTPA28) must be cleared

Note: Thread Safety: Safe - returns constant hardware address

Example:: volatilerx_dmac_channel_regs_t*ch0=dmac_channel(k_dmac_channel_0); ch0->dmsar=(uint32_t)src_buffer; ch0->dmdar=(uint32_t)dst_buffer; ch0->dmcra=transfer_count; ch0->dmtmd=k_dmtmd_md_normal|k_dmtmd_sz_32bit|k_dmtmd_dctg_software; ch0->dmamd=k_dmamd_sm_increment|k_dmamd_dm_increment; ch0->dmcnt=k_dmcnt_dte;

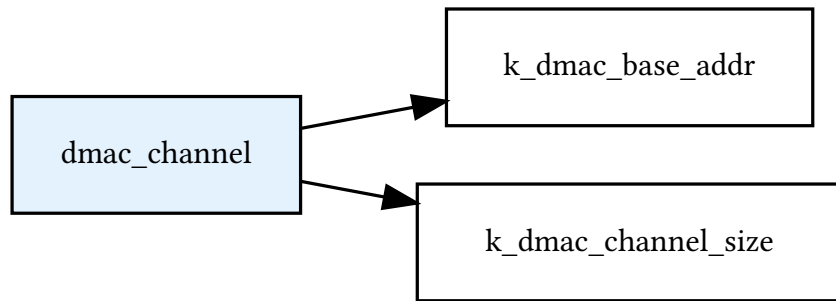


Figure 520: Call graph for dmac_channel

_Defined in libs/rx_hal/inc/rx72n_dmac_regs.h:637_

Since Version 1.0.0

—

dmac0

```
static volatile rx_dmac_channel_regs_t * dmac0(void)
```

Get pointer to DMAC channel 0 registers.

Returns: Volatile pointer to DMAC0 register structure

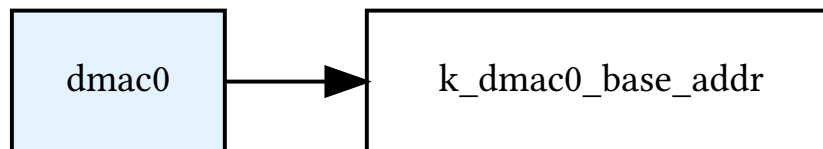


Figure 521: Call graph for dmac0

_Defined in libs/rx_hal/inc/rx72n_dmac_regs.h:647_

Since Version 1.0.0

—

dmac1

```
static volatile rx_dmac_channel_regs_t * dmac1(void)
```

Get pointer to DMAC channel 1 registers.

Returns: Volatile pointer to DMAC1 register structure

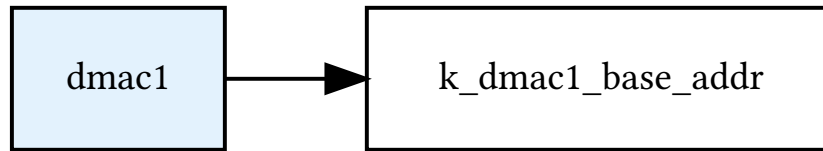


Figure 522: Call graph for dmac1

_Defined in `libs/rx\_hal/inc/rx72n\_dmac\_regs.h:657_`

Since Version 1.0.0

—

dmac2

```
static volatile rx_dmac_channel_regs_t * dmac2(void)
```

Get pointer to DMAC channel 2 registers.

Returns: Volatile pointer to DMAC2 register structure

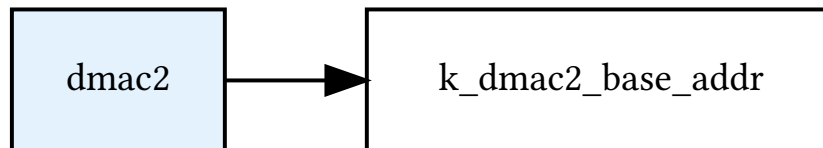


Figure 523: Call graph for dmac2

_Defined in `libs/rx\_hal/inc/rx72n\_dmac\_regs.h:667_`

Since Version 1.0.0

—

dmac3

```
static volatile rx_dmac_channel_regs_t * dmac3(void)
```

Get pointer to DMAC channel 3 registers.

Returns: Volatile pointer to DMAC3 register structure

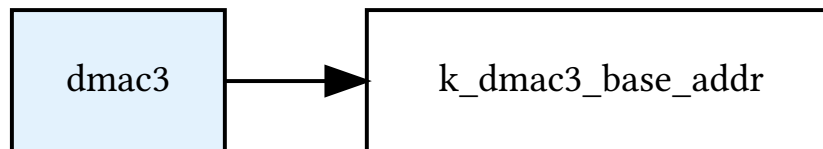


Figure 524: Call graph for dmac3

_Defined in `libs/rx\_hal/inc/rx72n\_dmac\_regs.h:677_`

Since Version 1.0.0

—

dmac4

```
static volatile rx_dmac_channel_regs_t * dmac4(void)
```

Get pointer to DMAC channel 4 registers.

Returns: Volatile pointer to DMAC4 register structure

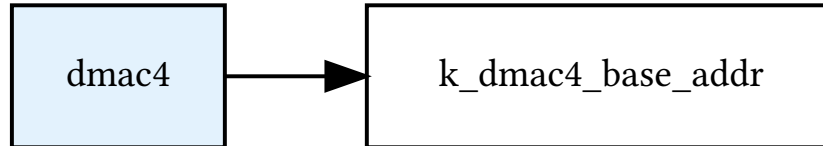


Figure 525: Call graph for dmac4

Defined in `libs/rx\_hal/inc/rx72n\_dmac\_regs.h`:687

Since Version 1.0.0

—

dmac5

```
static volatile rx_dmac_channel_regs_t * dmac5(void)
```

Get pointer to DMAC channel 5 registers.

Returns: Volatile pointer to DMAC5 register structure

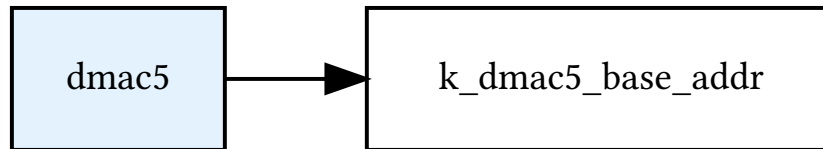


Figure 526: Call graph for dmac5

Defined in `libs/rx\_hal/inc/rx72n\_dmac\_regs.h`:697

Since Version 1.0.0

—

dmac6

```
static volatile rx_dmac_channel_regs_t * dmac6(void)
```

Get pointer to DMAC channel 6 registers.

Returns: Volatile pointer to DMAC6 register structure

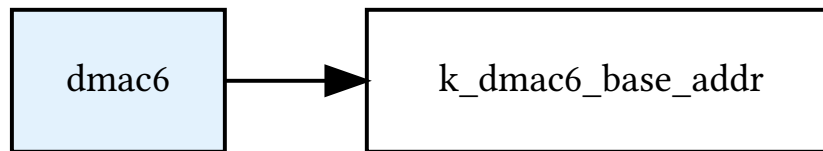


Figure 527: Call graph for dmac6

_Defined in libs\rx_hal\inc\rx72n_dmac_regs.h:707_

Since Version 1.0.0

—

dmac7

```
static volatile rx_dmac_channel_regs_t * dmac7(void)
```

Get pointer to DMAC channel 7 registers.

Returns: Volatile pointer to DMAC7 register structure

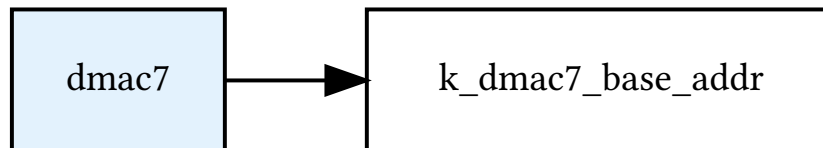


Figure 528: Call graph for dmac7

_Defined in libs\rx_hal\inc\rx72n_dmac_regs.h:717_

Since Version 1.0.0

—

dmac_dmast_reg

```
static volatile uint8_t * dmac_dmast_reg(void)
```

Get pointer to DMAC Module Start Register (DMAST).

Returns: Volatile pointer to DMAST register (8-bit)

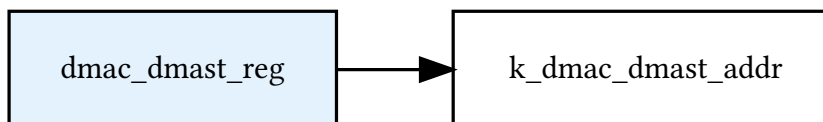


Figure 529: Call graph for dmac_dmast_reg

_Defined in libs\rx_hal\inc\rx72n_dmac_regs.h:727_

Since Version 1.0.0

—

dmac_dmist_reg

```
static volatile uint8_t * dmac_dmist_reg(void)
```

Get pointer to DMAC74 Interrupt Status Monitor Register (DMIST).

Returns: Volatile pointer to DMIST register (8-bit)

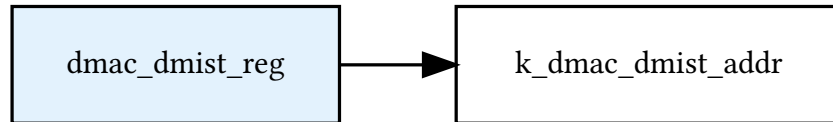


Figure 530: Call graph for `dmac_dmist_reg`

_Defined in `libs/rx_hal/inc/rx72n_dmac_regs.h:737_`

Since Version 1.0.0

—

rx72n_dtc_regs.h

RX72N Data Transfer Controller (DTCb) Register Definitions.

Complete register definitions for the RX72N Data Transfer Controller. The DTC performs interrupt-driven data transfers without CPU intervention, supporting normal, repeat, block, and sequence transfer modes.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stddef.h>`
- `<stdint.h>`

dtc_addresses_t

DTC module addresses.

Base address and individual register addresses for DTC control registers. All addresses verified against RX72N Hardware Manual Ch20.

Value	Name	Description
= 0x00082400U	<code>k_dtc_base_addr</code>	DTC base address.
= 0x00082400U	<code>k_dtc_dtccr_addr</code>	DTC Control Register address.
= 0x00082404U	<code>k_dtc_dtcvbr_addr</code>	DTC Vector Base Register address.
= 0x00082408U	<code>k_dtc_dtcadmod_addr</code>	DTC Address Mode Register address.
= 0x0008240CU	<code>k_dtc_dtcst_addr</code>	DTC Module Start Register address.
= 0x0008240EU	<code>k_dtc_dtcsts_addr</code>	DTC Status Register address.
= 0x00082410U	<code>k_dtc_dtcibr_addr</code>	DTC Index Table Base Register address.

= 0x00082414U	k_dtc_dtcdr_addr	DTC Operation Register address.
= 0x00082416U	k_dtc_dtcsqe_addr	DTC Sequence Transfer Enable Register address.
= 0x00082418U	k_dtc_dtcdisp_addr	DTC Address Displacement Register address.

Note: Addresses verified 2026-01-29 against manual section 20.2

Since Version 1.0.0

—

dtc_offsets_t

Register offsets from DTC base address.

Offsets for each register relative to k_dtc_base_addr.

Value	Name	Description
= 0x00U	k_dtc_offset_dtccr	DTCCR offset (8-bit)
= 0x04U	k_dtc_offset_dtcvbr	DTCVBR offset (32-bit)
= 0x08U	k_dtc_offset_dtcadmod	DTCADMOD offset (8-bit)
= 0x0CU	k_dtc_offset_dtcst	DTCST offset (8-bit)
= 0x0EU	k_dtc_offset_dtcsts	DTCSTS offset (16-bit)
= 0x10U	k_dtc_offset_dtcibr	DTCIBR offset (32-bit)
= 0x14U	k_dtc_offset_dtcdr	DTCOR offset (8-bit)
= 0x16U	k_dtc_offset_dtcsqe	DTCSQE offset (16-bit)
= 0x18U	k_dtc_offset_dtcdisp	DTCDISP offset (32-bit)

Note: Offsets verified 2026-01-29 against manual

Since Version 1.0.0

—

dtccr_bits_t

DTC Control Register (DTCCR) bit definitions.

Controls DTC read skip functionality.

Value	Name	Description
= 0x08U	k_dtccr_reserved_bit3	Reserved bit 3 (must write 1).
= 0x10U	k_dtccr_rrs	Transfer Information Read Skip Enable (bit 4).

Note: Manual ref: Ch20 section 20.2.8

Since Version 1.0.0

—

dtcadmod_bits_t

DTC Address Mode Register (DTCADMOD) bit definitions.

Selects short-address or full-address mode for DTC transfers.

Value	Name	Description
= 0x01U	k_dtcadmod_short	Short-Address Mode Set (bit 0).

Note: Manual ref: Ch20 section 20.2.10

Since Version 1.0.0

—

dtcst_bits_t

DTC Module Start Register (DTCST) bit definitions.

Enables or disables the DTC module.

Value	Name	Description
= 0x01U	k_dtcst_dtcst	DTC Module Start (bit 0).

Note: Manual ref: Ch20 section 20.2.11

Since Version 1.0.0

—

dtcsts_bits_t

DTC Status Register (DTCSTS) bit definitions.

Read-only register providing DTC transfer status.

Value	Name	Description
= 0x00FFU	k_dtcsts_vecn_mask	Vector number field mask (bits 7:0).
= 0x8000U	k_dtcsts_act	DTC Active Flag (bit 15).

Note: Manual ref: Ch20 section 20.2.12

Since Version 1.0.0

—

dtcor_bits_t

DTC Operation Register (DTCOR) bit definitions.

Controls DTC operation, specifically sequence transfer termination.

Value	Name	Description
= 0x01U	k_dtcor_sqtfrl	Sequence Transfer Terminate (bit 0, write-only).

Note: Manual ref: Ch20 section 20.2.14

Since Version 1.0.0

—

dtcsqe_bits_t

DTC Sequence Transfer Enable Register (DTCSQE) bit definitions.

Enables sequence transfer and specifies the trigger vector number.

Value	Name	Description
= 0x00FFU	k_dtcsqe_vecn_mask	Vector number field mask (bits 7:0).
= 0x8000U	k_dtcsqe_espsel	Sequence Transfer Enable (bit 15).

Note: Manual ref: Ch20 section 20.2.15

Since Version 1.0.0

—

dte_mra_bits_t

DTC Mode Register A (MRA) bit definitions for transfer info.

MRA is part of DTC transfer information stored in RAM, not directly accessible as a memory-mapped register.

Value	Name	Description
= 0x01U	k_dtc_mra_wbdis	Don't write back transfer info
= 0x00U	k_dtc_mra_sm_fixed	Source address fixed
= 0x08U	k_dtc_mra_sm_increment	Source address increment
= 0x0CU	k_dtc_mra_sm_decrement	Source address decrement
= 0x0CU	k_dtc_mra_sm_mask	SM field mask
= 0x00U	k_dtc_mra_sz_byte	8-bit transfer
= 0x10U	k_dtc_mra_sz_word	16-bit transfer
= 0x20U	k_dtc_mra_sz_long	32-bit transfer
= 0x30U	k_dtc_mra_sz_mask	SZ field mask
= 0x00U	k_dtc_mra_md_normal	Normal transfer mode
= 0x40U	k_dtc_mra_md_repeat	Repeat transfer mode
= 0x80U	k_dtc_mra_md_block	Block transfer mode
= 0xC0U	k_dtc_mra_md_mask	MD field mask

Note: Manual ref: Ch20 section 20.2.1

Since Version 1.0.0

dtc_mrb_bits_t

DTC Mode Register B (MRB) bit definitions for transfer info.

MRB is part of DTC transfer information stored in RAM, not directly accessible as a memory-mapped register.

Value	Name	Description
= 0x01U	k_dtc_mrb_sqend	End sequence transfer
= 0x02U	k_dtc_mrb_indx	Refer to index table
= 0x00U	k_dtc_mrb_dm_fixed	Destination address fixed
= 0x08U	k_dtc_mrb_dm_increment	Destination address increment
= 0x0CU	k_dtc_mrb_dm_decrement	Destination address decrement
= 0x0CU	k_dtc_mrb_dm_mask	DM field mask
= 0x10U	k_dtc_mrb_dts	Source is repeat/block area (1=src, 0=dest)
= 0x20U	k_dtc_mrb_disel	Interrupt after each transfer (not just end)
= 0x40U	k_dtc_mrb_chns	Chain only when counter becomes 0
= 0x80U	k_dtc_mrb_chne	Chain transfer enable

Note: Manual ref: Ch20 section 20.2.2

Since Version 1.0.0

dtc_mstpcr_t

DTC module stop control bits.

The DTC is controlled by MSTPCRA.MSTPA28. Must be cleared (0) before accessing DTC registers.

Value	Name	Description
= (1U <= 28U)	k_dtc_mstpcra_bit	MSTPCRA bit for DTC (bit 28).

Note: Manual ref: Ch11 (Low Power Consumption) module stop tables

Note: DTC and DMAC share the same module stop bit (MSTPA28)

Since Version 1.0.0

dtc_dtccr_reg

```
static volatile uint8_t * dtc_dtccr_reg(void)
```

Get pointer to DTC Control Register (DTCCR).

Returns: Volatile pointer to DTCCR register (8-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

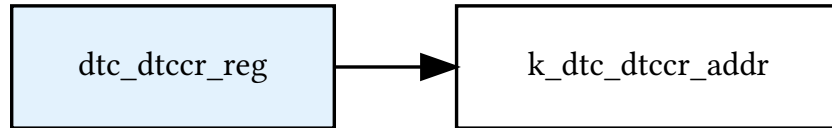


Figure 531: Call graph for `dtc_dtccr_reg`

_Defined in `libs/rx_hal/inc/rx72n_dtc_regs.h:532_`

Since Version 1.0.0

—

dtc_dtcvbr_reg

```
static volatile uint32_t * dtc_dtcvbr_reg(void)
```

Get pointer to DTC Vector Base Register (DTCVBR).

Returns: Volatile pointer to DTCVBR register (32-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

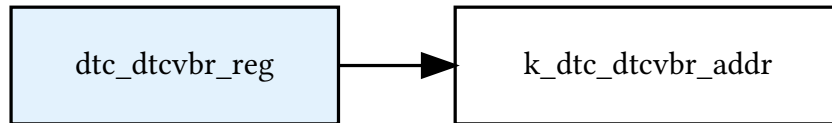


Figure 532: Call graph for `dtc_dtcvbr_reg`

_Defined in `libs/rx_hal/inc/rx72n_dtc_regs.h:543_`

Since Version 1.0.0

—

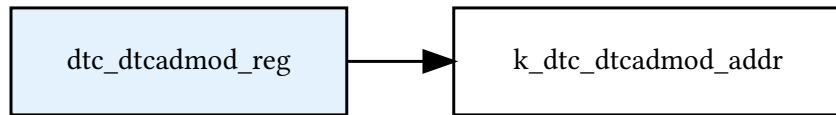
dtc_dtcadmod_reg

```
static volatile uint8_t * dtc_dtcadmod_reg(void)
```

Get pointer to DTC Address Mode Register (DTCADMOD).

Returns: Volatile pointer to DTCADMOD register (8-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

Figure 533: Call graph for `dtc_dtcadmod_reg`

_Defined in `libs/rx_hal/inc/rx72n_dtc_regs.h:554_`

Since Version 1.0.0

—

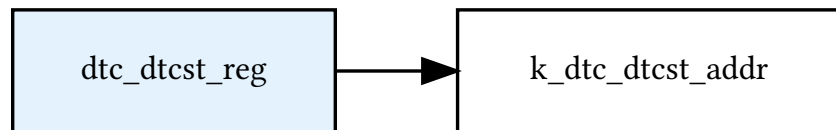
dtc_dtcst_reg

```
static volatile uint8_t * dtc_dtcst_reg(void)
```

Get pointer to DTC Module Start Register (DTCST).

Returns: Volatile pointer to DTCST register (8-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

Figure 534: Call graph for `dtc_dtcst_reg`

_Defined in `libs/rx_hal/inc/rx72n_dtc_regs.h:565_`

Since Version 1.0.0

—

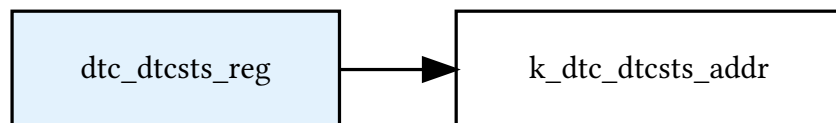
dtc_dtcsts_reg

```
static volatile uint16_t * dtc_dtcsts_reg(void)
```

Get pointer to DTC Status Register (DTCSTS).

Returns: Volatile pointer to DTCSTS register (16-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

Figure 535: Call graph for `dtc_dtcsts_reg`

_Defined in `libs/rx_hal/inc/rx72n_dtc_regs.h:576_`

Since Version 1.0.0

—

dtc_dtcibr_reg

```
static volatile uint32_t * dtc_dtcibr_reg(void)
```

Get pointer to DTC Index Table Base Register (DTCIBR).

Returns: Volatile pointer to DTCIBR register (32-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

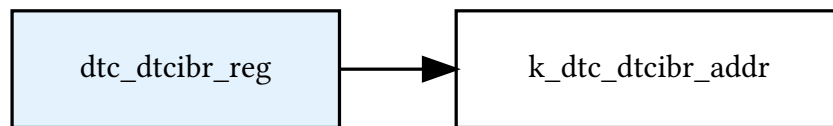


Figure 536: Call graph for `dtc_dtcibr_reg`

_Defined in `libs/rx\_hal/inc/rx72n\_dtc\_regs.h:587_`

Since Version 1.0.0

—

dtc_dtcdr_reg

```
static volatile uint8_t * dtc_dtcdr_reg(void)
```

Get pointer to DTC Operation Register (DTCOR).

Returns: Volatile pointer to DTCOR register (8-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

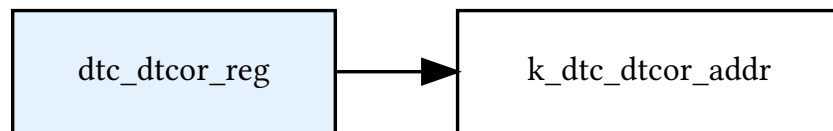


Figure 537: Call graph for `dtc_dtcdr_reg`

_Defined in `libs/rx\_hal/inc/rx72n\_dtc\_regs.h:598_`

Since Version 1.0.0

—

dtc_dtcsqe_reg

```
static volatile uint16_t * dtc_dtcsqe_reg(void)
```

Get pointer to DTC Sequence Transfer Enable Register (DTCSQE).

Returns: Volatile pointer to DTCSQE register (16-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

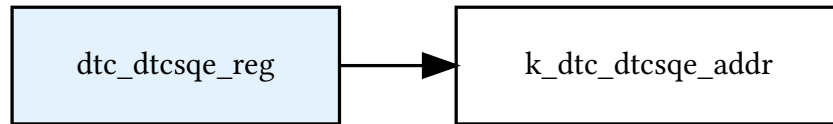


Figure 538: Call graph for dtc_dtcsqe_reg

_Defined in libs/rx_hal/inc/rx72n_dtc_regs.h:609_

Since Version 1.0.0

—

dtc_dtcdisp_reg

```
static volatile uint32_t * dtc_dtcdisp_reg(void)
```

Get pointer to DTC Address Displacement Register (DTCDISP).

Returns: Volatile pointer to DTCDISP register (32-bit)

Pre: DTC module stop bit (MSTPCRA.MSTPA28) must be cleared

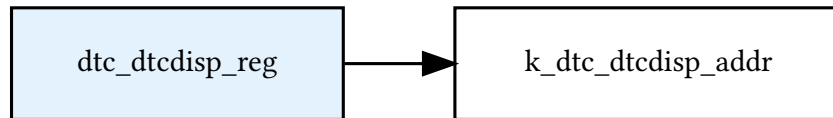


Figure 539: Call graph for dtc_dtcdisp_reg

_Defined in libs/rx_hal/inc/rx72n_dtc_regs.h:620_

Since Version 1.0.0

—

rx72n_elc_regs.h

RX72N Event Link Controller (ELC) Register Definitions.

Complete register definitions for the RX72N Event Link Controller. The ELC uses interrupt requests from peripheral modules as event signals to interconnect peripherals without CPU intervention.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

elc_addresses_t

ELC register addresses.

All addresses verified against RX72N Hardware Manual Ch21.

Value	Name	Description
= 0x0008B100U	k_elc_base_addr	ELC module base address.
= 0x0008B100U	k_elc_elcr_addr	ELCR - Event Link Control Register.
= 0x0008B101U	k_elc_elsr0_addr	MTU0
= 0x0008B104U	k_elc_elsr3_addr	MTU3
= 0x0008B105U	k_elc_elsr4_addr	MTU4
= 0x0008B108U	k_elc_elsr7_addr	CMT1
= 0x0008B10BU	k_elc_elsr10_addr	TMR0
= 0x0008B10CU	k_elc_elsr11_addr	TMR1
= 0x0008B10DU	k_elc_elsr12_addr	TMR2
= 0x0008B10EU	k_elc_elsr13_addr	TMR3
= 0x0008B110U	k_elc_elsr15_addr	S12AD (ELCTRG00N)
= 0x0008B111U	k_elc_elsr16_addr	DA0
= 0x0008B113U	k_elc_elsr18_addr	ICU Interrupt 1
= 0x0008B114U	k_elc_elsr19_addr	ICU Interrupt 2
= 0x0008B115U	k_elc_elsr20_addr	Output port group 1
= 0x0008B116U	k_elc_elsr21_addr	Output port group 2
= 0x0008B117U	k_elc_elsr22_addr	Input port group 1
= 0x0008B118U	k_elc_elsr23_addr	Input port group 2
= 0x0008B119U	k_elc_elsr24_addr	Single port 0
= 0x0008B11AU	k_elc_elsr25_addr	Single port 1
= 0x0008B11BU	k_elc_elsr26_addr	Single port 2
= 0x0008B11CU	k_elc_elsr27_addr	Single port 3
= 0x0008B11DU	k_elc_elsr28_addr	Clock source switch to LOCO
= 0x0008B131U	k_elc_elsr33_addr	CMTW0
= 0x0008B133U	k_elc_elsr35_addr	TPU0
= 0x0008B134U	k_elc_elsr36_addr	TPU1
= 0x0008B135U	k_elc_elsr37_addr	TPU2
= 0x0008B136U	k_elc_elsr38_addr	TPU3
= 0x0008B13DU	k_elc_elsr45_addr	S12AD1 (ELCTRG10N)
= 0x0008B146U	k_elc_elsr48_addr	GPTW event source A
= 0x0008B147U	k_elc_elsr49_addr	GPTW event source B
= 0x0008B148U	k_elc_elsr50_addr	GPTW event source C

= 0x0008B149U	k_elc_elsr51_addr	GPTW event source D
= 0x0008B14AU	k_elc_elsr52_addr	GPTW event source E
= 0x0008B14BU	k_elc_elsr53_addr	GPTW event source F
= 0x0008B14CU	k_elc_elsr54_addr	GPTW event source G
= 0x0008B14DU	k_elc_elsr55_addr	GPTW event source H
= 0x0008B14EU	k_elc_elsr56_addr	S12AD (ELCTRG01N)
= 0x0008B14FU	k_elc_elsr57_addr	S12AD1 (ELCTRG11N)
= 0x0008B11FU	k_elc_elopa_addr	MTU0/MTU3 option setting
= 0x0008B120U	k_elc_elopb_addr	MTU4 option setting
= 0x0008B121U	k_elc_elopc_addr	CMT1 option setting
= 0x0008B122U	k_elc_elopd_addr	TMR0-3 option setting
= 0x0008B13FU	k_elc_elopf_addr	TPU0-3 option setting
= 0x0008B141U	k_elc_eloph_addr	CMTW0 option setting
= 0x0008B123U	k_elc_pgr1_addr	Port Group Register 1
= 0x0008B124U	k_elc_pgr2_addr	Port Group Register 2
= 0x0008B125U	k_elc_pgc1_addr	Port Group Control 1
= 0x0008B126U	k_elc_pgc2_addr	Port Group Control 2
= 0x0008B127U	k_elc_pdbf1_addr	Port Data Buffer 1
= 0x0008B128U	k_elc_pdbf2_addr	Port Data Buffer 2
= 0x0008B129U	k_elc_pel0_addr	Port Event Link 0
= 0x0008B12AU	k_elc_pel1_addr	Port Event Link 1
= 0x0008B12BU	k_elc_pel2_addr	Port Event Link 2
= 0x0008B12CU	k_elc_pel3_addr	Port Event Link 3
= 0x0008B12DU	k_elc_elsegr_addr	Software Event Generation

Note: Addresses verified 2026-01-29 against manual section 21.2

Since Version 1.0.0

—

elcr_bits_t

Event Link Control Register (ELCR) bit definitions.

Value	Name	Description
= 0x7FU	k_elcr_reserved	Reserved bits mask (write 1).
= 0x80U	k_elcr_elcon	All Event Link Enable (0=disabled, 1=enabled).

Note: Manual ref: Ch21 section 21.2.1

Since Version 1.0.0

—

elsr_event_t

Event signal values for ELSRn registers.

Values to set in ELSRn.ELS[7:0] to select the event signal source. Only a subset of commonly-used events are defined here.

Value	Name	Description
= 0x00U	k_elsr_event_disabled	Event output disabled
= 0x01U	k_elsr_mtu0_cmpa	MTU0 compare match A
= 0x02U	k_elsr_mtu0_cmpb	MTU0 compare match B
= 0x03U	k_elsr_mtu0_cmpc	MTU0 compare match C
= 0x04U	k_elsr_mtu0_cmpd	MTU0 compare match D
= 0x05U	k_elsr_mtu0_cmpe	MTU0 compare match E
= 0x06U	k_elsr_mtu0_cmpf	MTU0 compare match F
= 0x07U	k_elsr_mtu0_ovf	MTU0 overflow
= 0x10U	k_elsr_mtu3_cmpa	MTU3 compare match A
= 0x11U	k_elsr_mtu3_cmpb	MTU3 compare match B
= 0x12U	k_elsr_mtu3_cmpc	MTU3 compare match C
= 0x13U	k_elsr_mtu3_cmpd	MTU3 compare match D
= 0x14U	k_elsr_mtu3_ovf	MTU3 overflow
= 0x15U	k_elsr_mtu4_cmpa	MTU4 compare match A
= 0x16U	k_elsr_mtu4_cmpb	MTU4 compare match B
= 0x17U	k_elsr_mtu4_cmpc	MTU4 compare match C
= 0x18U	k_elsr_mtu4_cmpd	MTU4 compare match D
= 0x19U	k_elsr_mtu4_ovf	MTU4 overflow
= 0x1AU	k_elsr_mtu4_udf	MTU4 underflow
= 0x1FU	k_elsr_cmt1_cmp	CMT1 compare match
= 0x22U	k_elsr_tmr0_cmpa	TMR0 compare match A
= 0x23U	k_elsr_tmr0_cmpb	TMR0 compare match B
= 0x24U	k_elsr_tmr0_ovf	TMR0 overflow
= 0x58U	k_elsr_s12ad0_end	S12AD A/D conversion end
= 0x6CU	k_elsr_s12ad1_end	S12AD1 A/D conversion end
= 0x5DU	k_elsr_dmac0_end	DMAC0 transfer end
= 0x5EU	k_elsr_dmac1_end	DMAC1 transfer end
= 0x5FU	k_elsr_dmac2_end	DMAC2 transfer end
= 0x60U	k_elsr_dmac3_end	DMAC3 transfer end

= 0x61U	k_elsr_dtc_end	DTC transfer end
= 0x63U	k_elsr_port_grp1	Input port group 1 edge
= 0x64U	k_elsr_port_grp2	Input port group 2 edge
= 0x65U	k_elsr_port_single0	Single port 0 edge
= 0x66U	k_elsr_port_single1	Single port 1 edge
= 0x67U	k_elsr_port_single2	Single port 2 edge
= 0x68U	k_elsr_port_single3	Single port 3 edge
= 0x69U	k_elsr_software	Software event
= 0x80U	k_elsr_gptw0_cmpa	GPTW0 compare match A
= 0x81U	k_elsr_gptw0_cmpb	GPTW0 compare match B
= 0x82U	k_elsr_gptw0_cmpc	GPTW0 compare match C
= 0x83U	k_elsr_gptw0_cmpd	GPTW0 compare match D
= 0x84U	k_elsr_gptw0_cmpe	GPTW0 compare match E
= 0x85U	k_elsr_gptw0_cmpf	GPTW0 compare match F
= 0x86U	k_elsr_gptw0_ovf	GPTW0 overflow
= 0x87U	k_elsr_gptw0_udf	GPTW0 underflow
= 0x88U	k_elsr_gptw1_cmpa	GPTW1 compare match A
= 0x89U	k_elsr_gptw1_cmpb	GPTW1 compare match B
= 0x8AU	k_elsr_gptw1_cmpc	GPTW1 compare match C
= 0x8BU	k_elsr_gptw1_cmpd	GPTW1 compare match D
= 0x8CU	k_elsr_gptw1_cmpe	GPTW1 compare match E
= 0x8DU	k_elsr_gptw1_cmpf	GPTW1 compare match F
= 0x8EU	k_elsr_gptw1_ovf	GPTW1 overflow
= 0x8FU	k_elsr_gptw1_udf	GPTW1 underflow
= 0x90U	k_elsr_gptw2_cmpa	GPTW2 compare match A
= 0x91U	k_elsr_gptw2_cmpb	GPTW2 compare match B
= 0x92U	k_elsr_gptw2_cmpc	GPTW2 compare match C
= 0x93U	k_elsr_gptw2_cmpd	GPTW2 compare match D
= 0x94U	k_elsr_gptw2_cmpe	GPTW2 compare match E
= 0x95U	k_elsr_gptw2_cmpf	GPTW2 compare match F
= 0x96U	k_elsr_gptw2_ovf	GPTW2 overflow
= 0x97U	k_elsr_gptw2_udf	GPTW2 underflow
= 0x98U	k_elsr_gptw3_cmpa	GPTW3 compare match A
= 0x99U	k_elsr_gptw3_cmpb	GPTW3 compare match B
= 0x9AU	k_elsr_gptw3_cmpc	GPTW3 compare match C
= 0x9BU	k_elsr_gptw3_cmpd	GPTW3 compare match D

= 0x9CU	k_elsr_gptw3_cmpe	GPTW3 compare match E
= 0x9DU	k_elsr_gptw3_cmpf	GPTW3 compare match F
= 0x9EU	k_elsr_gptw3_ovf	GPTW3 overflow
= 0x9FU	k_elsr_gptw3_udf	GPTW3 underflow

Note: Manual ref: Ch21 Table 21.3

Since Version 1.0.0

—

elop_timer_op_t

Timer operation selection bits for ELOPA/B/C/D/F/H registers.

Each timer can be configured to perform different operations when an event signal is received. These 2-bit fields are used in the ELOPx registers.

Value	Name	Description
= 0x00U	k_elop_timer_compare_match	Compare match
= 0x01U	k_elop_timer_input_capture	Input capture
= 0x02U	k_elop_timer_reserved	Reserved - do not use
= 0x03U	k_elop_timer_disabled	Event output disabled

Note: Manual ref: Ch21 sections 21.2.3-21.2.8

Since Version 1.0.0

—

elopa_bits_t

ELOPA register bit positions (MTU0/MTU3).

Value	Name	Description
= 0U	k_elopa_mtu0op_shift	MTU0 operation field shift
= 0x03U	k_elopa_mtu0op_mask	MTU0 operation field mask
= 4U	k_elopa_mtu3op_shift	MTU3 operation field shift
= 0x30U	k_elopa_mtu3op_mask	MTU3 operation field mask

Since Version 1.0.0

—

elopb_bits_t

ELOPB register bit positions (MTU4).

Value	Name	Description
= 0U	k_elopb_mtu4op_shift	MTU4 operation field shift

= 0x03U	k_elopb_mtu4op_mask	MTU4 operation field mask
---------	---------------------	---------------------------

Since Version 1.0.0

—

elopc_bits_t

ELOPC register bit positions (CMT1).

Value	Name	Description
= 0U	k_elopc_cmt1op_shift	CMT1 operation field shift
= 0x03U	k_elopc_cmt1op_mask	CMT1 operation field mask

Since Version 1.0.0

—

elopd_bits_t

ELOPD register bit positions (TMR0-3).

Value	Name	Description
= 0U	k_elopd_tmr0op_shift	TMR0 operation field shift
= 0x03U	k_elopd_tmr0op_mask	TMR0 operation field mask
= 2U	k_elopd_tmr1op_shift	TMR1 operation field shift
= 0x0CU	k_elopd_tmr1op_mask	TMR1 operation field mask
= 4U	k_elopd_tmr2op_shift	TMR2 operation field shift
= 0x30U	k_elopd_tmr2op_mask	TMR2 operation field mask
= 6U	k_elopd_tmr3op_shift	TMR3 operation field shift
= 0xC0U	k_elopd_tmr3op_mask	TMR3 operation field mask

Since Version 1.0.0

—

pgc_bits_t

Port Group Control (PGC1/PGC2) register bit definitions.

Value	Name	Description
= 0x01U	k_pgc_pgcove	Port group output value transfer enable
= 0x02U	k_pgc_pgcosel	Output value selection (0=ELSR, 1=PDBF)
= 0x08U	k_pgc_pgco	Output event output control
= 4U	k_pgc_pgci_shift	Input port group mode shift
= 0x30U	k_pgc_pgci_mask	Input port group mode mask
= 0x00U	k_pgc_pgci_and	AND operation
= 0x10U	k_pgc_pgci_or	OR operation
= 0x20U	k_pgc_pgci_xor	XOR operation

Since Version 1.0.0

—

pel_bits_t

Port Event Link (PEL0-3) register bit definitions.

Value	Name	Description
= 0x03U	k_pel_psb_mask	Port B pin select mask (bits 1:0)
= 0x04U	k_pel_psp_porte	Select Port E (else Port B)
= 0x08U	k_pel_psm_group	Group mode (else single mode)
= 5U	k_pel_eld_shift	Edge detection shift
= 0x60U	k_pel_eld_mask	Edge detection mask
= 0x00U	k_pel_eld_none	No edge detection
= 0x20U	k_pel_eld_rising	Rising edge detection
= 0x40U	k_pel_eld_falling	Falling edge detection
= 0x60U	k_pel_eld_both	Both edges detection

Since Version 1.0.0

—

elsegr_bits_t

Software Event Generation Register (ELSEGR) bit definitions.

Value	Name	Description
= 0x01U	k_elsegr_seg	Software event generation
= 0x40U	k_elsegr_we	Write enable
= 0x80U	k_elsegr_wi	Write disable (set 1 to disable)

Note: To write: Set WE=1, WI=0, then set SEG

Since Version 1.0.0

—

elc_mstpcr_t

ELC module stop control bits.

The ELC is controlled by MSTPCRB.MSTPB9. Must be cleared (0) before accessing ELC registers.

Value	Name	Description
= (1U << 9U)	k_elc_mstpcrb_bit	MSTPCRB bit for ELC (bit 9).

Note: Manual ref: Ch11 (Low Power Consumption) module stop tables

Since Version 1.0.0

elc_elcr_reg

```
static volatile uint8_t * elc_elcr_reg(void)
```

Get pointer to ELCR register (Event Link Control).

Returns: Volatile pointer to ELCR register (8-bit)

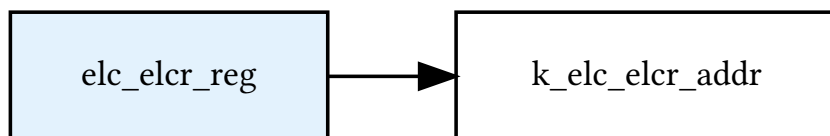


Figure 540: Call graph for elc_elcr_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:517_

Since Version 1.0.0

elc_elsr_reg

```
static volatile uint8_t * elc_elsr_reg(uint8_t n)
```

Get pointer to ELSRn register.

Parameters:

Direction	Name	Description
in	n	Register index (0, 3, 4, 7, 10-13, 15, 16, 18-28, 33, 35-38, 45, 48-57)

Returns: Volatile pointer to ELSRn register (8-bit)

Warning: Not all indices are valid - check manual for valid values

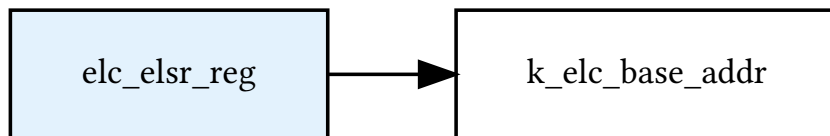


Figure 541: Call graph for elc_elsr_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:529_

Since Version 1.0.0

elc_elsr0_reg

```
static volatile uint8_t * elc_elsr0_reg(void)
```


Get pointer to ELSR0 register (MTU0).

Returns: Volatile pointer to ELSR0 register (8-bit)

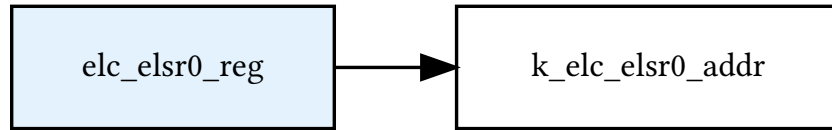


Figure 542: Call graph for `elc_elsr0_reg`

_Defined in `libs/rx_hal/inc/rx72n_elc_regs.h:539_`

Since Version 1.0.0

—

elc_elsr15_reg

```
static volatile uint8_t * elc_elsr15_reg(void)
```

Get pointer to ELSR15 register (S12AD ELCTRG00N).

Returns: Volatile pointer to ELSR15 register (8-bit)

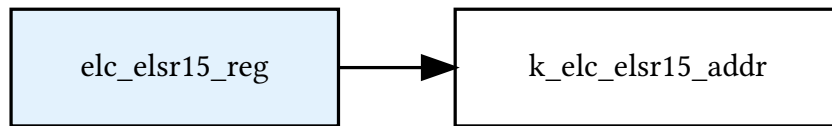


Figure 543: Call graph for `elc_elsr15_reg`

_Defined in `libs/rx_hal/inc/rx72n_elc_regs.h:549_`

Since Version 1.0.0

—

elc_elsr45_reg

```
static volatile uint8_t * elc_elsr45_reg(void)
```

Get pointer to ELSR45 register (S12AD1 ELCTRG10N).

Returns: Volatile pointer to ELSR45 register (8-bit)

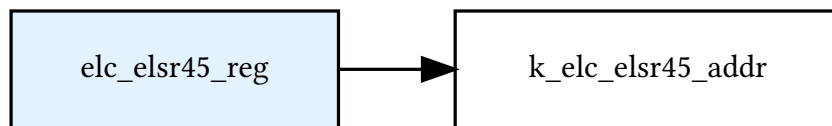


Figure 544: Call graph for `elc_elsr45_reg`

_Defined in `libs/rx_hal/inc/rx72n_elc_regs.h:559_`

Since Version 1.0.0

—

elc_elsr48_reg

```
static volatile uint8_t * elc_elsr48_reg(void)
```

Get pointer to ELSR48 register (GPTW event source A).

Returns: Volatile pointer to ELSR48 register (8-bit)

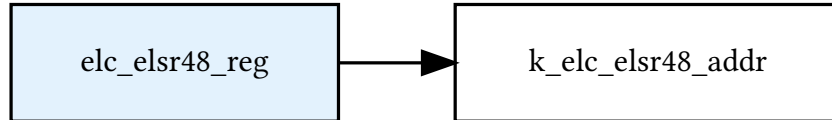


Figure 545: Call graph for elc_elsr48_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:569_

Since Version 1.0.0

—

elc_elopa_reg

```
static volatile uint8_t * elc_elopa_reg(void)
```

Get pointer to ELOPA register (MTU0/MTU3 option).

Returns: Volatile pointer to ELOPA register (8-bit)

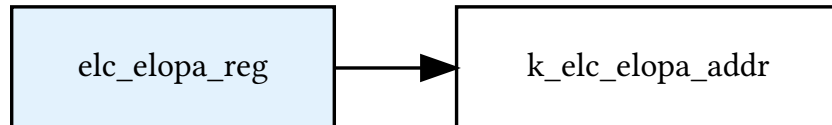


Figure 546: Call graph for elc_elopa_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:579_

Since Version 1.0.0

—

elc_elopb_reg

```
static volatile uint8_t * elc_elopb_reg(void)
```

Get pointer to ELOPB register (MTU4 option).

Returns: Volatile pointer to ELOPB register (8-bit)

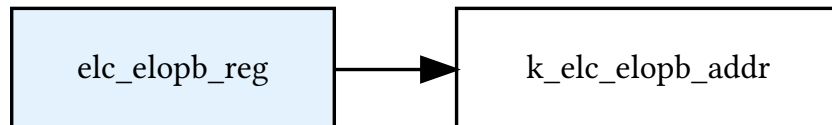


Figure 547: Call graph for elc_elopb_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:589_

Since Version 1.0.0

—

elc_pgr1_reg

```
static volatile uint8_t * elc_pgr1_reg(void)
```

Get pointer to PGR1 register (Port Group 1).

Returns: Volatile pointer to PGR1 register (8-bit)

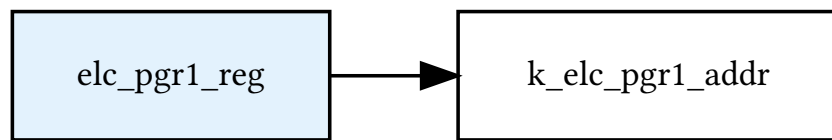


Figure 548: Call graph for `elc_pgr1_reg`

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:599_

Since Version 1.0.0

—

elc_pgr2_reg

```
static volatile uint8_t * elc_pgr2_reg(void)
```

Get pointer to PGR2 register (Port Group 2).

Returns: Volatile pointer to PGR2 register (8-bit)

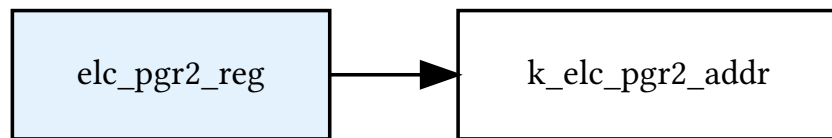


Figure 549: Call graph for `elc_pgr2_reg`

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:609_

Since Version 1.0.0

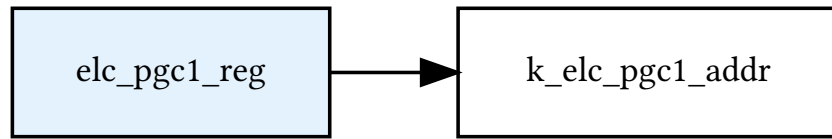
—

elc_pgc1_reg

```
static volatile uint8_t * elc_pgc1_reg(void)
```

Get pointer to PGC1 register (Port Group Control 1).

Returns: Volatile pointer to PGC1 register (8-bit)

Figure 550: Call graph for `elc_pgc1_reg`

_Defined in `libs/rx\_hal/inc/rx72n\_elc\_regs.h:619_`

Since Version 1.0.0

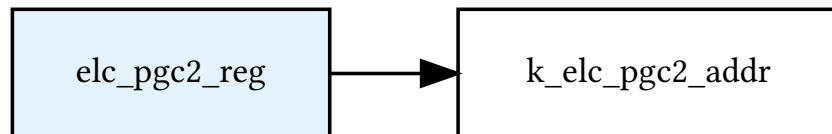
—

`elc_pgc2_reg`

```
static volatile uint8_t * elc_pgc2_reg(void)
```

Get pointer to PGC2 register (Port Group Control 2).

Returns: Volatile pointer to PGC2 register (8-bit)

Figure 551: Call graph for `elc_pgc2_reg`

_Defined in `libs/rx\_hal/inc/rx72n\_elc\_regs.h:629_`

Since Version 1.0.0

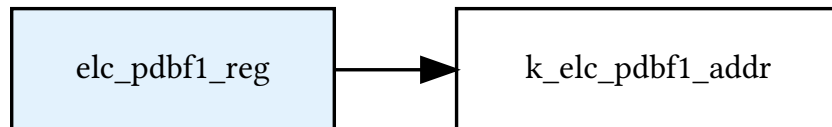
—

`elc_pdbf1_reg`

```
static volatile uint8_t * elc_pdbf1_reg(void)
```

Get pointer to PDBF1 register (Port Data Buffer 1).

Returns: Volatile pointer to PDBF1 register (8-bit)

Figure 552: Call graph for `elc_pdbf1_reg`

_Defined in `libs/rx\_hal/inc/rx72n\_elc\_regs.h:639_`

Since Version 1.0.0

—

elc_pdbf2_reg

```
static volatile uint8_t * elc_pdbf2_reg(void)
```

Get pointer to PDBF2 register (Port Data Buffer 2).

Returns: Volatile pointer to PDBF2 register (8-bit)

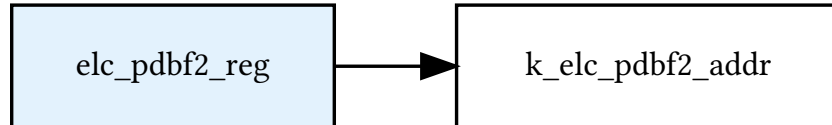


Figure 553: Call graph for elc_pdbf2_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:649_

Since Version 1.0.0

—

elc_pel_reg

```
static volatile uint8_t * elc_pel_reg(uint8_t n)
```

Get pointer to PELn register (Port Event Link).

Parameters:

Direction	Name	Description
in	n	Register index (0-3)

Returns: Volatile pointer to PELn register (8-bit)

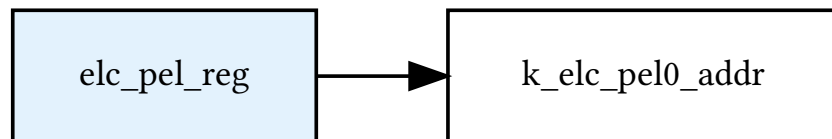


Figure 554: Call graph for elc_pel_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:660_

Since Version 1.0.0

—

elc_elsegr_reg

```
static volatile uint8_t * elc_elsegr_reg(void)
```

Get pointer to ELSEGR register (Software Event Generation).

Returns: Volatile pointer to ELSEGR register (8-bit)

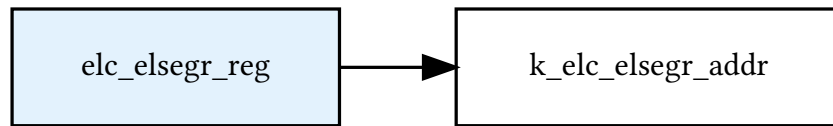


Figure 555: Call graph for elc_elsegr_reg

_Defined in libs/rx_hal/inc/rx72n_elc_regs.h:670_

Since Version 1.0.0

—

rx72n_flash_regs.h

RX72N Flash Memory Register Definitions.

Register definitions for the Flash Memory module including ROM cache control, flash programming/erase operations (FACI), and access protection. The RX72N has 4MB code flash and 32KB data flash with hardware-accelerated operations.

Enable FWEPROR before any programming operations

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx_rom_cache_addresses_t

ROM cache register addresses.

Base addresses for ROM cache control registers. The ROM cache improves code fetch performance with 8KB direct-mapped cache.

Value	Name	Description
= 0x00081000	k_rom_cache_base_addr	ROM cache register base
= 0x00081000	k_romce_addr	ROM Cache Enable Register
= 0x00081004	k_romciv_addr	ROM Cache Invalidate Register
= 0x00081040	k_ncrg0_addr	Non-Cacheable Area 0 Address
= 0x00081044	k_ncrc0_addr	Non-Cacheable Area 0 Control
= 0x00081048	k_ncrg1_addr	Non-Cacheable Area 1 Address
= 0x0008104C	k_ncrc1_addr	Non-Cacheable Area 1 Control

Since Version 1.0.0

—

rx_romce_bits_t

ROM Cache Enable Register (ROMCE) bit definitions.

ROMCE @ 0x00081000 controls ROM cache enable/disable.

Value	Name	Description
= (1U \<\< 0)	k_romce_romcen	ROM Cache Enable (1=enabled, 0=disabled)

Since Version 1.0.0

—

rx_romciv_bits_t

ROM Cache Invalidate Register (ROMCIV) bit definitions.

ROMCIV @ 0x00081004 invalidates all cache lines. Read: 0=complete/not started, 1=in progress

Write: 1=start invalidation, 0=no effect

Value	Name	Description
= (1U \<\< 0)	k_romciv_romciv	ROM Cache Invalidate

Since Version 1.0.0

—

rx_fweprror_addresses_t

Flash Write/Erase Protect Register addresses.

Value	Name	Description
= 0x0008C296	k_fweprror_addr	Flash P/E Protect Register

Since Version 1.0.0

—

rx_fweprror_bits_t

Flash P/E Protect Register (FWEPROR) bit definitions.

FWEPROR @ 0x0008C296 enables/disables flash program and erase operations.

- FLWE = 00b: Program, block erase, blank check disabled
- FLWE = 01b: Program, block erase, blank check enabled
- FLWE = 10b: Disabled
- FLWE = 11b: Disabled

Value	Name	Description
= 0x03	k_fweprror_flwe_mask	FLWE[1:0] mask
= 0x00	k_fweprror_flwe_disable	Program/erase disabled
= 0x01	k_fweprror_flwe_enable	Program/erase enabled
= 0x02	k_fweprror_reset_value	Value after reset

Warning: Register is reset on deep software standby mode entry

Since Version 1.0.0

—

rx_faci_addresses_t

FACI (Flash Application Command Interface) register addresses.

FACI registers control flash programming and erase operations. All registers are in the 0x007FE0xx range.

Value	Name	Description
= 0x007FE000	k_faci_base_addr	FACI register base (conceptual)
= 0x007FE010	k_fastat_addr	Flash Access Status Register
= 0x007FE014	k_faeint_addr	Flash Access Error Interrupt Enable
= 0x007FE018	k_frdyie_addr	Flash Ready Interrupt Enable
= 0x007FE030	k_fsaddr_addr	FACI Start Address Register
= 0x007FE034	k_feaddr_addr	FACI End Address Register
= 0x007FE080	k_fstatr_addr	Flash Status Register
= 0x007FE084	k_fentryr_addr	Flash P/E Mode Entry Register
= 0x007FE08C	k_fsuinitr_addr	Flash Sequencer Setup Init Register
= 0x007FE0A0	k_fcldr_addr	FACI Command Register
= 0x007FE0D0	k_fbcnt_addr	Blank Check Control Register
= 0x007FE0D4	k_fbcstat_addr	Blank Check Status Register
= 0x007FE0D8	k_fpsaddr_addr	Flash Processing Switch Address
= 0x007FE0DC	k_fawmon_addr	Flash Access Window Monitor
= 0x007FE0E0	k_fcpsr_addr	Flash Sequencer Processing Switch
= 0x007FE0E4	k_fpckar_addr	Flash Sequencer Clock Frequency
= 0x007FE0E8	k_fsuacr_addr	Flash Startup Area Select
= 0x007E0000	k_faci_cmd_area	FACI Command Issuing Area

Since Version 1.0.0

—

rx_eepfclk_addresses_t

Data Flash Memory Access Frequency register address.

Value	Name	Description
= 0x007FC040	k_eepfclk_addr	Data Flash Access Frequency Setting

Since Version 1.0.0

—

rx_fastat_bits_t

Flash Access Status Register (FASTAT) bit definitions.

FASTAT @ 0x007FE010 indicates flash access violations.

Value	Name	Description
-------	------	-------------

= 3	k_fastat_dfae_shift	DFAE bit position
= 4	k_fastat_cmdlk_shift	CMDLK bit position
= 7	k_fastat_cfae_shift	CFAE bit position
= (1U \<\< 3)	k_fastat_dfae	Data Flash Access Violation
= (1U \<\< 4)	k_fastat_cmdlk	Command Lock Flag
= (1U \<\< 7)	k_fastat_cfae	Code Flash Access Violation

Note: Write 0 to clear CFAE/DFAE after reading 1

Since Version 1.0.0

—

rx_faeint_bits_t

Flash Access Error Interrupt Enable Register (FAEINT) bit definitions.

FAEINT @ 0x007FE014 enables FIFERR interrupt generation. Reset value: 0x98 (all interrupts enabled)

Value	Name	Description
= 3	k_faeint_dfaeie_shift	DFAEIE bit position
= 4	k_faeint_cmdlkie_shift	CMDLKIE bit position
= 7	k_faeint_cfaeie_shift	CFAEIE bit position
= (1U \<\< 3)	k_faeint_dfaeie	Data Flash Access Violation IE
= (1U \<\< 4)	k_faeint_cmdlkie	Command Lock Interrupt Enable
= (1U \<\< 7)	k_faeint_cfaeie	Code Flash Access Violation IE
= 0x98	k_faeint_reset_value	Reset value (all enabled)

Since Version 1.0.0

—

rx_frdyie_bits_t

Flash Ready Interrupt Enable Register (FRDYIE) bit definitions.

Value	Name	Description
= (1U \<\< 0)	k_frdyie_frdyie	Flash Ready Interrupt Enable

Since Version 1.0.0

—

rx_fstatr_bits_t

Flash Status Register (FSTATR) bit definitions.

FSTATR @ 0x007FE080 indicates flash sequencer state. This is the primary register for monitoring flash operations.

Value	Name	Description
= 6	k_fstatr_flweerr_shift	
= 8	k_fstatr_prghspd_shift	
= 9	k_fstatr_ersspd_shift	
= 10	k_fstatr_dbfull_shift	
= 11	k_fstatr_susrdy_shift	
= 12	k_fstatr_prgerr_shift	
= 13	k_fstatr_erserr_shift	
= 14	k_fstatr_ilglerr_shift	
= 15	k_fstatr_frdy_shift	
= 20	k_fstatr_oterr_shift	
= 21	k_fstatr_secerr_shift	
= 22	k_fstatr_feseterr_shift	
= 23	k_fstatr_ilgcomerr_shift	
= (1UL << 6)	k_fstatr_flweerr	Flash P/ E Protect Error
= (1UL << 8)	k_fstatr_prghspd	Program Suspend Status
= (1UL << 9)	k_fstatr_ersspd	Erase Suspend Status
= (1UL << 10)	k_fstatr_dbfull	Data Buffer Full
= (1UL << 11)	k_fstatr_susrdy	Suspend Ready
= (1UL << 12)	k_fstatr_prgerr	Program Error
= (1UL << 13)	k_fstatr_erserr	Erase Error
= (1UL << 14)	k_fstatr_ilglerr	Illegal Error
= (1UL << 15)	k_fstatr_frdy	Flash Ready
= (1UL << 20)	k_fstatr_oterr	Other Error

= (1UL << 21)	k_fstatr_secerr	Security Error
= (1UL << 22)	k_fstatr_feseterr	FENTRY Setting Error
= (1UL << 23)	k_fstatr_ilgcomerr	Illegal Command Error
= (k_fstatr_flweerr k_fstatr_prgerr k_fstatr_erserr k_fstatr_ilglerr k_fstatr_oterr k_fstatr_secerr k_fstatr_feseterr k_fstatr_ilgcomerr)	k_fstatr_error_mask	Reset value (FRDY=1)
= 0x00008000	k_fstatr_reset_value	

Since Version 1.0.0

—

rx_fentryr_bits_t

Flash P/E Mode Entry Register (FENTRYR) bit definitions.

FENTRYR @ 0x007FE084 controls entry into P/E (Program/Erase) mode. Write with KEY=0xAA to modify FENTRYC/FENTRYD bits.

Value	Name	Description
= (1U << 0)	k_fentryr_fentryc	Code Flash P/E Mode Entry
= (1U << 7)	k_fentryr_fentryd	Data Flash P/E Mode Entry
= 8	k_fentryr_key_shift	Key code position
= 0xAA00	k_fentryr_key	Key code (write only)
= 0xAA01	k_fentryr_code_pe	Enter code flash P/E mode
= 0xAA80	k_fentryr_data_pe	Enter data flash P/E mode
= 0xAA00	k_fentryr_read_mode	Exit P/E mode

Warning: Writing 0xAA81 causes ILGLERR error

Since Version 1.0.0

—

rx_fsunitr_bits_t

Flash Sequencer Setup Initialization Register (FSUNITR) bit defs.

FSUNITR @ 0x007FE08C initializes flash sequencer setup registers. Write with KEY=0x2D to enable SUNIT write.

Value	Name	Description
-------	------	-------------

= (1U \<\< 0)	k_fsuinitr_suinit	Setup Initialization
= 0x2D00	k_fsuinitr_key	Key code for write enable
= 0x2D01	k_fsuinitr_init_cmd	Initialize flash sequencer setup

Since Version 1.0.0

—

rx_fcmdr_bits_t

FACI Command Register (FCMDR) bit definitions.

FCMDR @ 0x007FE0A0 contains the two most recent commands received. Read-only register for debugging.

Value	Name	Description
= 0xFF00	k_fcmdr_pcmdr_mask	Previous command mask
= 8	k_fcmdr_pcmdr_shift	Previous command position
= 0x00FF	k_fcmdr_cmdr_mask	Current command mask

Since Version 1.0.0

—

rx_faci_commands_t

FACI command codes.

Commands are written to the FACI command issuing area @ 0x007E0000. Command sequences vary; see manual Ch62 Section 62.8.1 for details.

Value	Name	Description
= 0xE8	k_faci_cmd_program	Program command
= 0x20	k_faci_cmd_block_erase	Block erase command
= 0xB0	k_faci_cmd_pe_suspend	P/E suspend command
= 0xD0	k_faci_cmd_pe_resume	P/E resume command
= 0x50	k_faci_cmd_status_clear	Status clear command
= 0xB3	k_faci_cmd_forced_stop	Forced stop command
= 0x71	k_faci_cmd_blank_check	Blank check command
= 0x40	k_faci_cmd_config_set	Configuration set command
= 0x71	k_faci_cmd_lockbit_read	Lock bit read command
= 0xD0	k_faci_cmd_d0	Command terminator

Since Version 1.0.0

—

rx_fpckar_bits_t

Flash Sequencer Clock Notification Register (FPCKAR) bit definitions.

FPCKAR @ 0x007FE0E4 notifies the flash sequencer of FCLK frequency. Must be set before issuing FACI commands for correct timing. Write with KEY=0x1E to modify PCKA field.

Value	Name	Description
= 0x00FF	k_fpckar_pcka_mask	PCKA frequency field mask
= 8	k_fpckar_key_shift	Key code position
= 0x1E00	k_fpckar_key	Key code for write enable

Since Version 1.0.0

—

rx_fbcnt_bits_t

Blank Check Control Register (FBCCNT) bit definitions.

Value	Name	Description
= (1U << 0)	k_fbcnt_bcdir	Blank Check Direction (0=forward, 1=reverse)

Since Version 1.0.0

—

rx_fbcstat_bits_t

Blank Check Status Register (FBCSTAT) bit definitions.

Value	Name	Description
= (1U << 0)	k_fbcstat_bcst	Blank Check Status (0=blank, 1=not blank)

Since Version 1.0.0

—

rx_uidr_addresses_t

Unique ID Register addresses (Ch62 Section 62.4.23).

UIDR0-3 contain a 128-bit unique identifier for each chip. These are read-only registers in the FLASHCONST area (on-chip ROM).

Value	Name	Description
= 0xFE7F7D90	k_uidr0_addr	Unique ID Register 0 (bits 31-0)
= 0xFE7F7D94	k_uidr1_addr	Unique ID Register 1 (bits 63-32)
= 0xFE7F7D98	k_uidr2_addr	Unique ID Register 2 (bits 95-64)
= 0xFE7F7D9C	k_uidr3_addr	Unique ID Register 3 (bits 127-96)

Note: Values are factory-programmed and unique to each chip

Note: These addresses are in FLASHCONST area (FE7F xxxx), not the typical FACI register space (007F xxxx) - this is expected per manual

Since Version 1.0.0

—

rx_flash_memory_addresses_t

Flash memory region addresses.

Value	Name	Description
= 0xFFC00000	k_code_flash_start	Code flash start (4 MB device)
= 0xFFFFFFFF	k_code_flash_end	Code flash end
= 0x00400000	k_code_flash_size	Code flash size (4 MB)
= 0x00100000	k_data_flash_start	Data flash start
= 0x00107FFF	k_data_flash_end	Data flash end
= 0x00008000	k_data_flash_size	Data flash size (32 KB)
= 0x00002000	k_code_flash_block_8k	8 KB block (blocks 0-7)
= 0x00008000	k_code_flash_block_32k	32 KB block (blocks 8+)
= 0x00000040	k_data_flash_block_64	64 byte block (data flash)
= 128	k_code_flash_program_unit	Code flash program unit (bytes)
= 4	k_data_flash_program_unit	Data flash program unit (bytes)

Since Version 1.0.0

—

romce_reg

```
static volatile uint16_t * romce_reg(void)
```

Get pointer to ROM Cache Enable Register (ROMCE).

Returns: Volatile pointer to ROMCE register

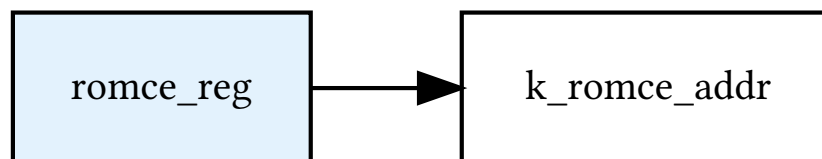


Figure 556: Call graph for romce_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:164_

Since Version 1.0.0

—

romciv_reg

```
static volatile uint16_t * romciv_reg(void)
```

Get pointer to ROM Cache Invalidate Register (ROMCIV).

Returns: Volatile pointer to ROMCIV register

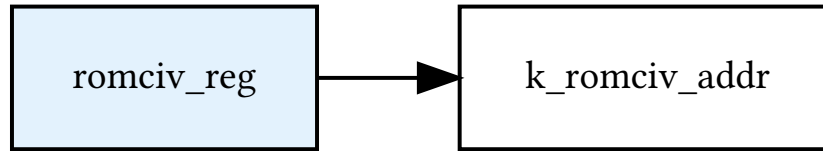


Figure 557: Call graph for romciv_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:174_

Since Version 1.0.0

—

fwepror_reg

```
static volatile uint8_t * fwepror_reg(void)
```

Get pointer to Flash P/E Protect Register (FWEPROR).

Returns: Volatile pointer to FWEPROR register

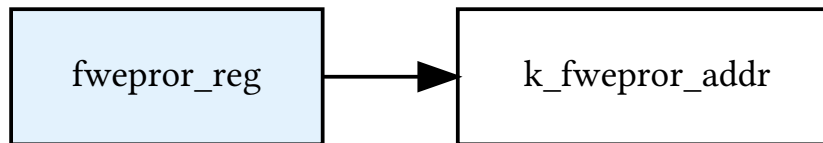


Figure 558: Call graph for fwepror_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:221_

Since Version 1.0.0

—

fastat_reg

```
static volatile uint8_t * fastat_reg(void)
```

Get pointer to Flash Access Status Register (FASTAT).

Returns: Volatile pointer to FASTAT register

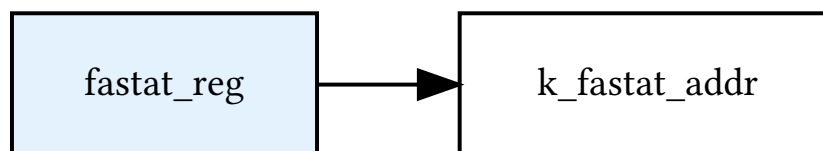


Figure 559: Call graph for fastat_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:332_

Since Version 1.0.0

—

faeint_reg

```
static volatile uint8_t * faeint_reg(void)
```

Get pointer to Flash Access Error Interrupt Enable (FAEINT).

Returns: Volatile pointer to FAEINT register

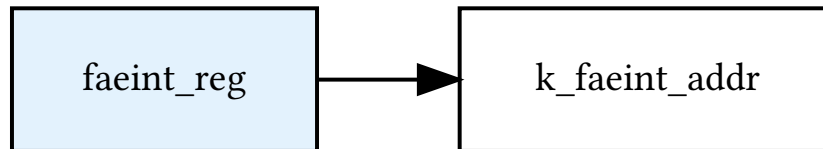


Figure 560: Call graph for faeint_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:369_`

Since Version 1.0.0

—

frdyie_reg

```
static volatile uint8_t * frdyie_reg(void)
```

Get pointer to Flash Ready Interrupt Enable (FRDYIE).

Returns: Volatile pointer to FRDYIE register

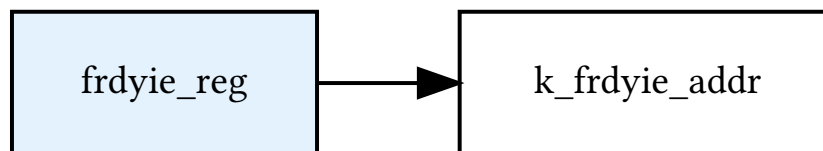


Figure 561: Call graph for frdyie_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:394_`

Since Version 1.0.0

—

fstatr_reg

```
static volatile uint32_t * fstatr_reg(void)
```

Get pointer to Flash Status Register (FSTATR).

Returns: Volatile pointer to FSTATR register

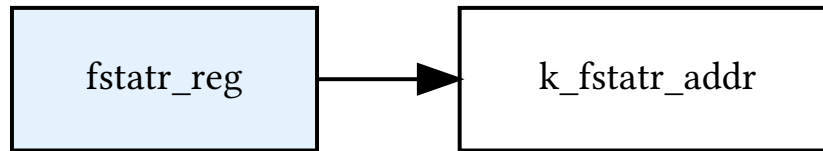


Figure 562: Call graph for fstatr_reg

_Defined in `libs/rx\_hal/inc/rx72n\_flash\_regs.h:474_`

Since Version 1.0.0

—

fentryr_reg

```
static volatile uint16_t * fentryr_reg(void)
```

Get pointer to Flash P/E Mode Entry Register (FENTRYR).

Returns: Volatile pointer to FENTRYR register

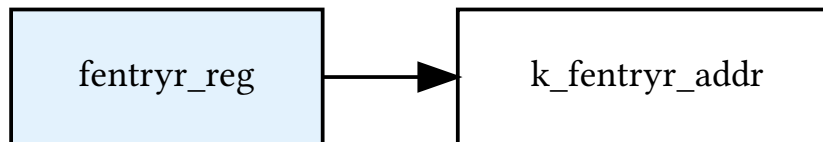


Figure 563: Call graph for fentryr_reg

_Defined in `libs/rx\_hal/inc/rx72n\_flash\_regs.h:517_`

Since Version 1.0.0

—

fsuinitr_reg

```
static volatile uint16_t * fsuinitr_reg(void)
```

Get pointer to Flash Sequencer Setup Init Register (FSUINITR).

Returns: Volatile pointer to FSUINITR register

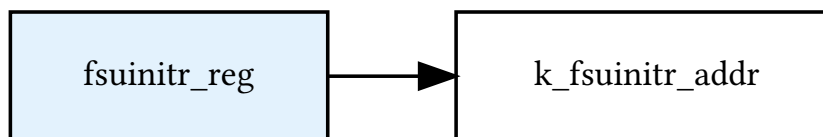


Figure 564: Call graph for fsuinitr_reg

_Defined in `libs/rx\_hal/inc/rx72n\_flash\_regs.h:549_`

Since Version 1.0.0

—

fsaddr_reg

```
static volatile uint32_t * fsaddr_reg(void)
```

Get pointer to FSCI Start Address Register (FSADDR).

Returns: Volatile pointer to FSADDR register

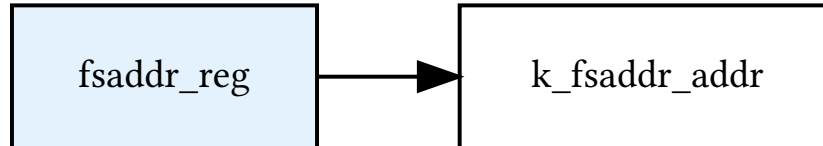


Figure 565: Call graph for `fsaddr_reg`

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:564_`

Since Version 1.0.0

—

feaddr_reg

```
static volatile uint32_t * feaddr_reg(void)
```

Get pointer to FSCI End Address Register (FEADDR).

Returns: Volatile pointer to FEADDR register

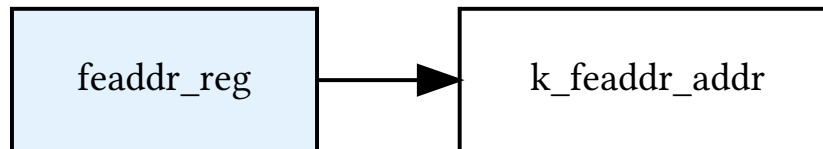


Figure 566: Call graph for `feaddr_reg`

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:574_`

Since Version 1.0.0

—

fcmdr_reg

```
static volatile uint16_t * fcmdr_reg(void)
```

Get pointer to FSCI Command Register (FCMDR).

Returns: Volatile pointer to FCMDR register

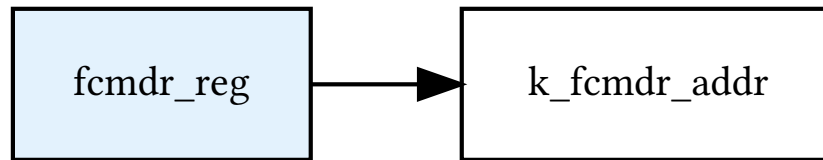


Figure 567: Call graph for fcmdr_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:605_

Since Version 1.0.0

—

faci_cmd_area

```
static volatile uint8_t * faci_cmd_area(void)
```

Get pointer to FACI Command Issuing Area.

Commands are issued by writing to this 4-byte area. The write sequence depends on the command type.

Returns: Volatile pointer to FACI command area

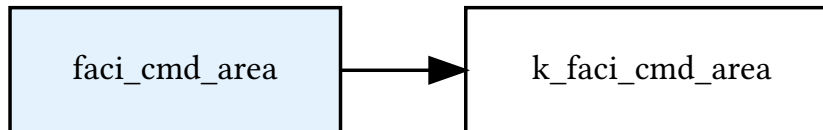


Figure 568: Call graph for faci_cmd_area

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:648_

Since Version 1.0.0

—

fpckar_reg

```
static volatile uint16_t * fpckar_reg(void)
```

Get pointer to Flash Sequencer Clock Register (FPCKAR).

Returns: Volatile pointer to FPCKAR register

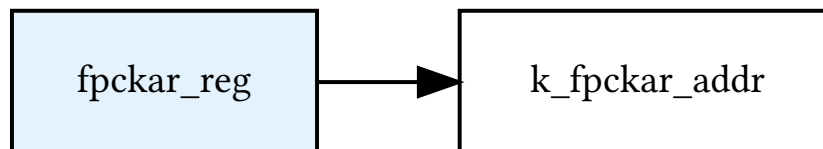


Figure 569: Call graph for fpckar_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:684_

Since Version 1.0.0

—

eepfclk_reg

```
static volatile uint8_t * eepfclk_reg(void)
```

Get pointer to Data Flash Access Frequency Register (EPPFCLK).

Returns: Volatile pointer to EPPFCLK register

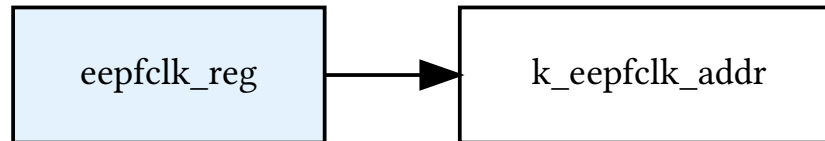


Figure 570: Call graph for eepfclk_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:699_`

Since Version 1.0.0

—

fbccnt_reg

```
static volatile uint8_t * fbccnt_reg(void)
```

Get pointer to Blank Check Control Register (FBCCNT).

Returns: Volatile pointer to FBCCNT register

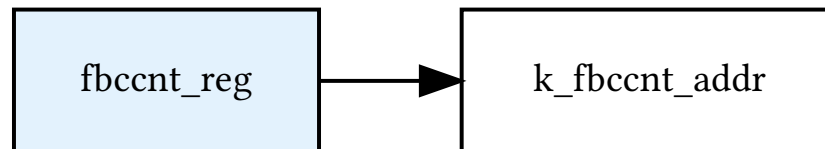


Figure 571: Call graph for fbccnt_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:724_`

Since Version 1.0.0

—

fbccstat_reg

```
static volatile uint8_t * fbccstat_reg(void)
```

Get pointer to Blank Check Status Register (FBCCSTAT).

Returns: Volatile pointer to FBCCSTAT register

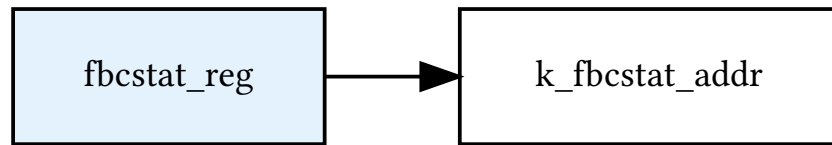


Figure 572: Call graph for fbcstat_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:749_`

Since Version 1.0.0

—

fpsaddr_reg

```
static volatile uint32_t * fpsaddr_reg(void)
```

Get pointer to Flash Processing Switch Address (FPSADDR).

Returns: Volatile pointer to FPSADDR register

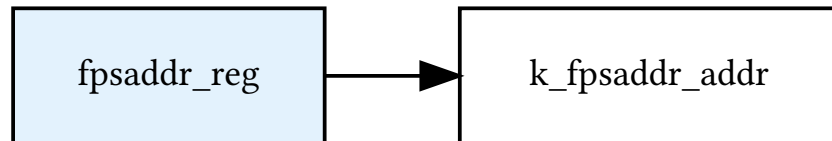


Figure 573: Call graph for fpsaddr_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:764_`

Since Version 1.0.0

—

fawmon_reg

```
static volatile uint32_t * fawmon_reg(void)
```

Get pointer to Flash Access Window Monitor (FAWMON).

Returns: Volatile pointer to FAWMON register

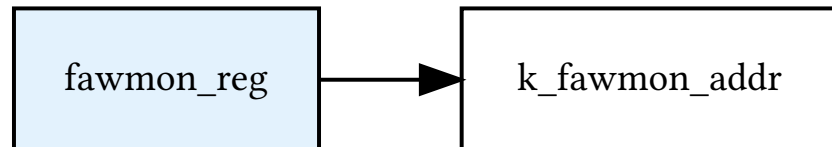


Figure 574: Call graph for fawmon_reg

_Defined in `libs/rx_hal/inc/rx72n_flash_regs.h:774_`

Since Version 1.0.0

—

fcpsr_reg

```
static volatile uint16_t * fcpsr_reg(void)
```

Get pointer to Flash Sequencer Processing Switch (FCPSR).

Returns: Volatile pointer to FCPSR register

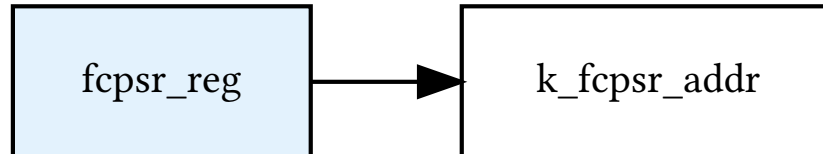


Figure 575: Call graph for fcpsr_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:784_

Since Version 1.0.0

—

fsuacr_reg

```
static volatile uint16_t * fsuacr_reg(void)
```

Get pointer to Flash Startup Area Select (FSUACR).

Returns: Volatile pointer to FSUACR register

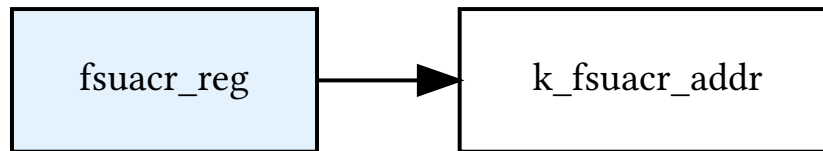


Figure 576: Call graph for fsuacr_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:794_

Since Version 1.0.0

—

uidr0_reg

```
static volatile const uint32_t * uidr0_reg(void)
```

Get pointer to Unique ID Register 0 (UIDR0).

Returns: Volatile pointer to UIDR0 register (read-only)

Note: Access may be restricted depending on security settings

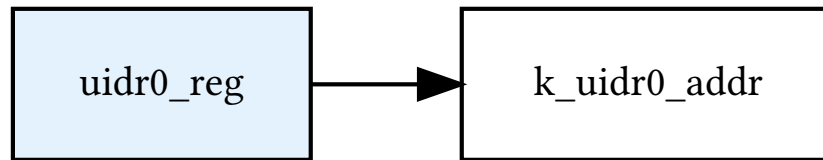


Figure 577: Call graph for uidr0_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:846_

Since Version 1.0.0

—

uidr1_reg

```
static volatile const uint32_t * uidr1_reg(void)
```

Get pointer to Unique ID Register 1 (UIDR1).

Returns: Volatile pointer to UIDR1 register (read-only)

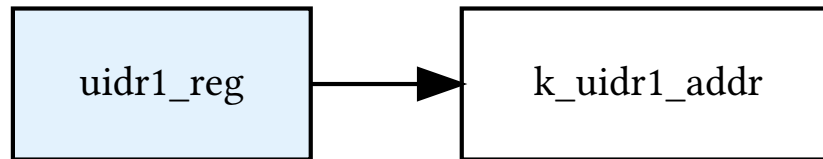


Figure 578: Call graph for uidr1_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:856_

Since Version 1.0.0

—

uidr2_reg

```
static volatile const uint32_t * uidr2_reg(void)
```

Get pointer to Unique ID Register 2 (UIDR2).

Returns: Volatile pointer to UIDR2 register (read-only)

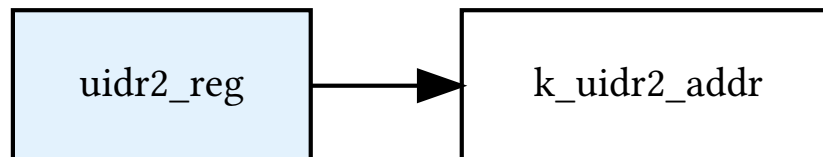


Figure 579: Call graph for uidr2_reg

_Defined in libs/rx_hal/inc/rx72n_flash_regs.h:866_

Since Version 1.0.0

uidr3_reg

```
static volatile const uint32_t * uidr3_reg(void)
```

Get pointer to Unique ID Register 3 (UIDR3).

Returns: Volatile pointer to UIDR3 register (read-only)

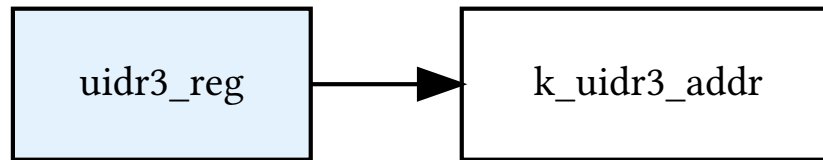


Figure 580: Call graph for uidr3_reg

_Defined in `libs/rx\hal\inc/rx72n\flash\_regs.h:876_`

Since Version 1.0.0

rx72n_gptw_regs.h

RX72N GPTW (General PWM Timer) Register Definitions.

This header provides comprehensive register definitions for the RX72N's General PWM Timer (GPTW). The GPTW is a 32-bit timer optimized for motor PWM generation with dead-time control, complementary outputs, and hardware-synchronized ADC triggering.

STAR Team

2026-01-28

1.1.0

MIT License

Includes:

- `<stddef.h>`
- `<stdint.h>`

rx72n_icu_regs.h

RX72N ICU Interrupt Controller Unit Register Definitions.

This header provides comprehensive register definitions for the RX72N's Interrupt Controller Unit (ICU). The ICU manages all 256 interrupt sources with configurable priorities, enabling/disabling, and status monitoring.

STAR Team

2026-01-28

1.2.0

MIT License

Includes:

- <stddef.h>
- <stdint.h>

rx72n_iwdt_regs.h

RX72N IWDT Independent Watchdog Timer Register Definitions.

Register definitions for the Independent Watchdog Timer (IWDT) module on the RX72N microcontroller. The IWDT provides critical system monitoring with a dedicated clock source independent of the main system clock.

Example with IWDTCLK = 120 kHz, TOPS = 16384, CKS = 128:

*Note: 35s exceeds practical limits; use shorter timeouts for safety.

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx_iwdt_addresses_t

IWDT base address constant.

Base address for the Independent Watchdog Timer register block at 0x00088030. The IWDT has a 9-byte register space with 5 registers and 2 reserved bytes.

Value	Name	Description
= 0x00088030	k_iwdt_base_addr	IWDT register base address (0x00088030).

See also: iwdt() Accessor function

Since Version 1.0.0

—

iwdt_reserved_sizes_t

IWDT register reserved field sizes.

Defines reserved byte counts for structure padding to match hardware layout.

Value	Name	Description
= 1	k_iwdt_reserved_after_iwdtrr_bytes	Reserved byte after IWDTRR @ 0x01
= 1	k_iwdt_reserved_after_iwdtrcr_bytes	Reserved byte after IWDTRCR @ 0x07

Since Version 1.0.0

—

iwdt_refresh_sequence_t

IWDT Refresh Register (IWDTRR) Write Sequence Values.

The IWDT counter is refreshed by writing a specific two-byte sequence:

1. Write 0x00 (k_iwdt_refresh_start)

2. Write 0xFF (k_iwdt_refresh_end)

The first refresh also starts the IWDTCR, which cannot be stopped until reset.

Value	Name	Description
= 0x00	k_iwdt_refresh_start	First write value for refresh sequence (0x00).
= 0xFF	k_iwdt_refresh_end	Second write value for refresh sequence (0xFF).

Warning: First refresh starts IWDTCR permanently

Warning: Both writes must complete in sequence - partial refresh is invalid] **See also:**
rx_iwdt_regs_t::iwdtrr Refresh register

Since Version 1.0.0

—

iwdt_iwdtcr_shifts_t

IWDTCR Control Register (IWDTCR) Bit Field Positions.

Bit positions for IWDTCR register fields. Used to construct field values.

Value	Name	Description
= 4	k_iwdt_iwdtcr_cks_shift	CKS field position (bits 7:4)
= 8	k_iwdt_iwdtcr_rpes_shift	RPES field position (bits 9:8)
= 12	k_iwdt_iwdtcr_rpss_shift	RPSS field position (bits 13:12)

Since Version 1.0.0

—

iwdt_iwdtcr_bits_t

IWDTCR Control Register (IWDTCR) Bit Field Values.

Configuration values for IWDTCR timeout period, clock division, and window protection. These values can be OR'd together to configure IWDTCR.

Value	Name	Description
= 0x0000	k_iwdt_tops_1024	1024 cycles timeout (8.5ms at 120kHz, no division)
= 0x0001	k_iwdt_tops_4096	4096 cycles timeout (34ms at 120kHz, no division)
= 0x0002	k_iwdt_tops_8192	8192 cycles timeout (68ms at 120kHz, no division)
= 0x0003	k_iwdt_tops_16384	16384 cycles timeout (137ms at 120kHz, no division)

= (0x00 << k_iwdt_iwdtcr_cks_shift)	k_iwdt_cks_div_1	IWDTCLK divided by 1 (fastest, shortest timeout).
= (0x02 << k_iwdt_iwdtcr_cks_shift)	k_iwdt_cks_div_16	IWDTCLK divided by 16.
= (0x03 << k_iwdt_iwdtcr_cks_shift)	k_iwdt_cks_div_32	IWDTCLK divided by 32.
= (0x04 << k_iwdt_iwdtcr_cks_shift)	k_iwdt_cks_div_64	IWDTCLK divided by 64.
= (0x0F << k_iwdt_iwdtcr_cks_shift)	k_iwdt_cks_div_128	IWDTCLK divided by 128 (recommended for 1s timeout).
= (0x05 << k_iwdt_iwdtcr_cks_shift)	k_iwdt_cks_div_256	IWDTCLK divided by 256 (slowest, longest timeout).
= (0x00 << k_iwdt_iwdtcr_rpes_shift)	k_iwdt_rpes_75	Window closes at 75% counter remaining.
= (0x01 << k_iwdt_iwdtcr_rpes_shift)	k_iwdt_rpes_50	Window closes at 50% counter remaining.
= (0x02 << k_iwdt_iwdtcr_rpes_shift)	k_iwdt_rpes_25	Window closes at 25% counter remaining.
= (0x03 << k_iwdt_iwdtcr_rpes_shift)	k_iwdt_rpes_0	No window end (refresh allowed until timeout).
= (0x00 << k_iwdt_iwdtcr_rpss_shift)	k_iwdt_rpss_25	Window opens at 25% counter remaining.
= (0x01 << k_iwdt_iwdtcr_rpss_shift)	k_iwdt_rpss_50	Window opens at 50% counter remaining.
= (0x02 << k_iwdt_iwdtcr_rpss_shift)	k_iwdt_rpss_75	Window opens at 75% counter remaining.
= (0x03 << k_iwdt_iwdtcr_rpss_shift)	k_iwdt_rpss_100	No window start (refresh allowed immediately).

See also: rx_iwdt_regs_t::iwdtcr Control register

Since Version 1.0.0

—

iwdt_iwdtsr_bits_t

IWDT Status Register (IWDTSR) Bit Field Values.

Contains the current down-counter value and error flags.

Value	Name	Description
= 0x3FFF	k_iwdt_sr_cntval_mask	Counter value mask (bits 13:0).
= 14	k_iwdt_sr_undff_shift	Underflow flag bit position.
= 15	k_iwdt_sr_refef_shift	Refresh error flag bit position.
= (1U << 14)	k_iwdt_sr_undff	Underflow flag - Bit 14.
= (1U << 15)	k_iwdt_sr_refef	Refresh error flag - Bit 15.

See also: `rx_iwdt_regs_t::iwdtcsr` Status register

Since Version 1.0.0

—

iwdt_iwdtrcr_bits_t

IWDT Reset Control Register (IWDTRCR) Bit Field Values.

Controls the action taken on IWDT timeout.

Value	Name	Description
= 7	<code>k_iwdt_rstirqs_pos</code>	RSTIRQS bit position.
= (1U << 7)	<code>k_iwdt_rcr_rstirqs_mask</code>	RSTIRQS bit mask.
= (1U << 7)	<code>k_iwdt_rstirqs_reset</code>	Generate internal reset on timeout (RSTIRQS=1).
= 0x00	<code>k_iwdt_rstirqs_nmi</code>	Generate NMI on timeout (RSTIRQS=0).

See also: `rx_iwdt_regs_t::iwdtrcr` Reset control register

Since Version 1.0.0

—

iwdt_iwdtcstpr_shifts_t

IWDTCSTPR bit field positions.

Value	Name	Description
= 7	<code>k_iwdt_cstpr_slcstp_pos</code>	Sleep Mode Count Stop bit position

Since Version 1.0.0

—

iwdt_iwdtcstpr_bits_t

IWDT Count Stop Control Register (IWDTCSTPR) Bit Field Values.

Controls whether the IWDT counter continues during sleep modes. This register is unique to IWDT (WDT does not have this capability).

Value	Name	Description
= (1U << 7)	<code>k_iwdt_cstpr_slcstp_mask</code>	SLCSTP bit mask.
= (1U << 7)	<code>k_iwdt_slcstp_stop</code>	Stop counting during sleep (SLCSTP=1).
= 0x00	<code>k_iwdt_slcstp_continue</code>	Continue counting during sleep (SLCSTP=0).

See also: `rx_iwdt_regs_t::iwdtcstpr` Count stop control register

Since Version 1.0.0

—

iwdt

```
static volatile rx_iwdt_regs_t * iwdt(void)
```

Get pointer to IWDT registers.

Returns a volatile pointer to the IWDT register structure at address 0x00088030. The IWDT is recommended for production systems due to its clock-independent operation.

Returns: Volatile pointer to IWDT register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: None (hardware register always accessible)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Example:: `iwdt()->iwdtrr=k_iwdt_refresh_start; iwdt()->iwdtrr=k_iwdt_refresh_end;`

`uint16_tcounter=iwdt()->iwdtsr&k_iwdt_sr_cntval_mask;`

See also: `k_iwdt_base_addr` Base address constant, `rx_iwdt_init()` Higher-level initialization

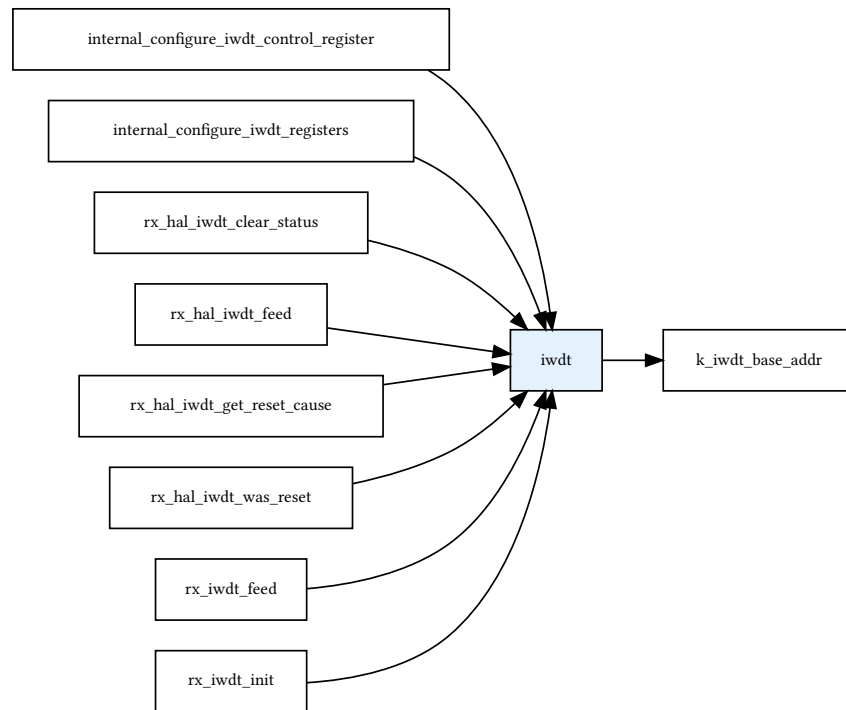


Figure 581: Call graph for iwdt

_Defined in `libs/rx\_hal/inc/rx72n\_iwdt\_regs.h:340_`

Since Version 1.0.0

—

rx72n_lpc_regs.h

RX72N Low Power Consumption Register Definitions.

Low Power Consumption (LPC) register definitions for operating power control, deep software standby mode, and wake-up interrupt configuration on the RX72N microcontroller. This header complements `rx72n_system_regs.h` which contains the main standby (SBYCR) and module stop (MSTPCRA-D) registers.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stddef.h>`
- `<stdint.h>`

rx_lpc_addresses_t

Low Power Consumption register addresses.

Addresses for operating power control and deep standby registers. These registers control low power mode behavior and wake-up sources.

Value	Name	Description
= 0x000800A0U	k_opccr_addr	Operating Power Control Register
= 0x000800A1U	k_rstckcr_addr	Sleep Mode Return Clock Switch Register
= 0x0008C280U	k_dpsbycr_base_addr	Deep Standby Control base
= 0x0008C280U	k_dpsbycr_addr	Deep Standby Control Register
= 0x0008C282U	k_dpsier0_addr	Deep Standby Interrupt Enable 0
= 0x0008C283U	k_dpsier1_addr	Deep Standby Interrupt Enable 1
= 0x0008C284U	k_dpsier2_addr	Deep Standby Interrupt Enable 2
= 0x0008C285U	k_dpsier3_addr	Deep Standby Interrupt Enable 3
= 0x0008C286U	k_dpsifr0_addr	Deep Standby Interrupt Flag 0
= 0x0008C287U	k_dpsifr1_addr	Deep Standby Interrupt Flag 1
= 0x0008C288U	k_dpsifr2_addr	Deep Standby Interrupt Flag 2
= 0x0008C289U	k_dpsifr3_addr	Deep Standby Interrupt Flag 3
= 0x0008C28AU	k_dpsiegr0_addr	Deep Standby Interrupt Edge 0
= 0x0008C28BU	k_dpsiegr1_addr	Deep Standby Interrupt Edge 1
= 0x0008C28CU	k_dpsiegr2_addr	Deep Standby Interrupt Edge 2
= 0x0008C28DU	k_dpsiegr3_addr	Deep Standby Interrupt Edge 3
= 0x0008C2A0U	k_dpsbkr_base_addr	Deep Standby Backup RAM base

Since Version 1.0.0

—

rx_lpc_sizes_t

Low Power Consumption register and buffer sizes.

Value	Name	Description
= 32U	k_dpsbkr_size	Deep standby backup RAM size (bytes)
= 4U	k_dps_regs_count	Number of DPSIERx/DPSIFRx/DPSIEGRx registers

Since Version 1.0.0

—

rx_opccr_bits_t

OPCCR bit definitions.

Operating Power Control Register controls operating power mode for normal operation, sleep mode, and all-module clock stop mode.

Value	Name	Description
= 0x00U	k_opccr_opcm_highspeed	High-speed operating mode (240 MHz max)

= 0x06U	k_opccr_opcm_lowspeed1	Low-speed operating mode 1 (1 MHz max)
= 0x07U	k_opccr_opcm_lowspeed2	Low-speed operating mode 2 (264 kHz max)
= 0x07U	k_opccr_opcm_mask	OPCM[2:0] bit mask
= 0x10U	k_opccr_opcmtsf	1 = Mode transition in progress

Since Version 1.0.0

—

rx_rstckcr_bits_t

RSTCKCR bit definitions.

Controls clock source switching at the time of release from sleep mode. When enabled, the selected clock source is automatically started upon wake-up from sleep mode.

Value	Name	Description
= 0x01U	k_rstckcr_rstcksel_hoco	HOCO selected for wake-up
= 0x02U	k_rstckcr_rstcksel_main	Main oscillator selected for wake-up
= 0x07U	k_rstckcr_rstcksel_mask	RSTCKSEL[2:0] bit mask
= 0x80U	k_rstckcr_rstcken	1 = Enable clock source switching

Since Version 1.0.0

—

rx_dpsbycr_bits_t

DPSBYCR bit definitions.

Controls deep software standby mode entry and power supply configuration.

Value	Name	Description
= 0x00U	k_dpsbycr_deepcut_ram_usb_on	Standby RAM & USB powered
= 0x01U	k_dpsbycr_deepcut_ram_usb_off	Standby RAM & USB not powered
= 0x03U	k_dpsbycr_deepcut_lvd_off	LVD stopped, lowest power
= 0x03U	k_dpsbycr_deepcut_mask	DEEPCUT[1:0] bit mask
= 0x40U	k_dpsbycr_iokeep	1 = Retain I/O state after deep standby exit
= 0x80U	k_dpsbycr_dpsby	1 = Enter deep standby (with SBYCR.SSBY=1)

Note: 10b is a prohibited setting for DEEPCUT1:0.

Since Version 1.0.0

—

rx_dpsier0_bits_t

DPSIER0 bit definitions (IRQ0-7 wake-up enable).

Enables external IRQ0-DS to IRQ7-DS pins as wake-up sources from deep software standby mode.

Value	Name	Description
= 0x01U	k_dpsier0_dirq0e	IRQ0-DS pin enable
= 0x02U	k_dpsier0_dirq1e	IRQ1-DS pin enable
= 0x04U	k_dpsier0_dirq2e	IRQ2-DS pin enable
= 0x08U	k_dpsier0_dirq3e	IRQ3-DS pin enable
= 0x10U	k_dpsier0_dirq4e	IRQ4-DS pin enable
= 0x20U	k_dpsier0_dirq5e	IRQ5-DS pin enable
= 0x40U	k_dpsier0_dirq6e	IRQ6-DS pin enable
= 0x80U	k_dpsier0_dirq7e	IRQ7-DS pin enable

Since Version 1.0.0

—

rx_dpsier1_bits_t

DPSIER1 bit definitions (IRQ8-15 wake-up enable).

Enables external IRQ8-DS to IRQ15-DS pins as wake-up sources from deep software standby mode.

Value	Name	Description
= 0x01U	k_dpsier1_dirq8e	IRQ8-DS pin enable
= 0x02U	k_dpsier1_dirq9e	IRQ9-DS pin enable
= 0x04U	k_dpsier1_dirq10e	IRQ10-DS pin enable
= 0x08U	k_dpsier1_dirq11e	IRQ11-DS pin enable
= 0x10U	k_dpsier1_dirq12e	IRQ12-DS pin enable
= 0x20U	k_dpsier1_dirq13e	IRQ13-DS pin enable
= 0x40U	k_dpsier1_dirq14e	IRQ14-DS pin enable
= 0x80U	k_dpsier1_dirq15e	IRQ15-DS pin enable

Since Version 1.0.0

—

rx_dpsier2_bits_t

DPSIER2 bit definitions (LVD, RTC, NMI, I2C, USB wake-up enable).

Enables various peripheral wake-up sources from deep software standby mode.

Value	Name	Description
= 0x01U	k_dpsier2_dlvdl1e	LVD1 (Voltage Monitor 1) enable
= 0x02U	k_dpsier2_dlvdl2e	LVD2 (Voltage Monitor 2) enable
= 0x04U	k_dpsier2_drtciie	RTC periodic interrupt enable
= 0x08U	k_dpsier2_drtcaie	RTC alarm interrupt enable
= 0x10U	k_dpsier2_dnmie	NMI pin enable (write-once)

= 0x20U	k_dpsier2_driicdie	SDA2-DS (I2C data) enable
= 0x40U	k_dpsier2_driiccie	SCL2-DS (I2C clock) enable
= 0x80U	k_dpsier2_dusbie	USB suspend/resume enable

Since Version 1.0.0

—

rx_dpsier3_bits_t

DPSIER3 bit definitions (CAN wake-up enable).

Enables CAN receive pin as wake-up source from deep software standby mode.

Value	Name	Description
= 0x01U	k_dpsier3_dcanie	CRX1-DS (CAN1 receive) enable

Since Version 1.0.0

—

rx_dpsifr0_bits_t

DPSIFR0 bit definitions (IRQ0-7 wake-up flags).

Indicates which IRQ0-DS to IRQ7-DS pin triggered wake-up from deep software standby mode.

Write 0 after reading 1 to clear.

Value	Name	Description
= 0x01U	k_dpsifr0_dirq0f	IRQ0-DS wake-up flag
= 0x02U	k_dpsifr0_dirq1f	IRQ1-DS wake-up flag
= 0x04U	k_dpsifr0_dirq2f	IRQ2-DS wake-up flag
= 0x08U	k_dpsifr0_dirq3f	IRQ3-DS wake-up flag
= 0x10U	k_dpsifr0_dirq4f	IRQ4-DS wake-up flag
= 0x20U	k_dpsifr0_dirq5f	IRQ5-DS wake-up flag
= 0x40U	k_dpsifr0_dirq6f	IRQ6-DS wake-up flag
= 0x80U	k_dpsifr0_dirq7f	IRQ7-DS wake-up flag

Since Version 1.0.0

—

rx_dpsifr1_bits_t

DPSIFR1 bit definitions (IRQ8-15 wake-up flags).

Indicates which IRQ8-DS to IRQ15-DS pin triggered wake-up from deep software standby mode.

Write 0 after reading 1 to clear.

Value	Name	Description
= 0x01U	k_dpsifr1_dirq8f	IRQ8-DS wake-up flag
= 0x02U	k_dpsifr1_dirq9f	IRQ9-DS wake-up flag

= 0x04U	k_dpsifr1_dirq10f	IRQ10-DS wake-up flag
= 0x08U	k_dpsifr1_dirq11f	IRQ11-DS wake-up flag
= 0x10U	k_dpsifr1_dirq12f	IRQ12-DS wake-up flag
= 0x20U	k_dpsifr1_dirq13f	IRQ13-DS wake-up flag
= 0x40U	k_dpsifr1_dirq14f	IRQ14-DS wake-up flag
= 0x80U	k_dpsifr1_dirq15f	IRQ15-DS wake-up flag

Since Version 1.0.0

—

rx_dpsifr2_bits_t

DPSIFR2 bit definitions (LVD, RTC, NMI, I2C, USB wake-up flags).

Indicates which peripheral triggered wake-up from deep software standby mode. Write 0 after reading 1 to clear.

Value	Name	Description
= 0x01U	k_dpsifr2_dlvdlif	LVD1 wake-up flag
= 0x02U	k_dpsifr2_dlvdl2if	LVD2 wake-up flag
= 0x04U	k_dpsifr2_drtciif	RTC periodic interrupt flag
= 0x08U	k_dpsifr2_drtcaif	RTC alarm interrupt flag
= 0x10U	k_dpsifr2_dnmif	NMI wake-up flag
= 0x20U	k_dpsifr2_driicdif	SDA2-DS wake-up flag
= 0x40U	k_dpsifr2_driiccif	SCL2-DS wake-up flag
= 0x80U	k_dpsifr2_dusbif	USB suspend/resume flag

Since Version 1.0.0

—

rx_dpsifr3_bits_t

DPSIFR3 bit definitions (CAN wake-up flag).

Indicates CAN receive triggered wake-up from deep software standby mode. Write 0 after reading 1 to clear.

Value	Name	Description
= 0x01U	k_dpsifr3_dcanif	CRX1-DS wake-up flag

Since Version 1.0.0

—

rx_dpsiegr0_bits_t

DPSIEGR0 bit definitions (IRQ0-7 edge select).

Selects edge polarity for IRQ0-DS to IRQ7-DS wake-up detection. 0 = Falling edge, 1 = Rising edge.

Value	Name	Description
= 0x01U	k_dpsiegr0_dirq0eg	IRQ0-DS edge select
= 0x02U	k_dpsiegr0_dirq1eg	IRQ1-DS edge select
= 0x04U	k_dpsiegr0_dirq2eg	IRQ2-DS edge select
= 0x08U	k_dpsiegr0_dirq3eg	IRQ3-DS edge select
= 0x10U	k_dpsiegr0_dirq4eg	IRQ4-DS edge select
= 0x20U	k_dpsiegr0_dirq5eg	IRQ5-DS edge select
= 0x40U	k_dpsiegr0_dirq6eg	IRQ6-DS edge select
= 0x80U	k_dpsiegr0_dirq7eg	IRQ7-DS edge select

Since Version 1.0.0

—

rx_dpsiegr1_bits_t

DPSIEGR1 bit definitions (IRQ8-15 edge select).

Selects edge polarity for IRQ8-DS to IRQ15-DS wake-up detection. 0 = Falling edge, 1 = Rising edge.

Value	Name	Description
= 0x01U	k_dpsiegr1_dirq8eg	IRQ8-DS edge select
= 0x02U	k_dpsiegr1_dirq9eg	IRQ9-DS edge select
= 0x04U	k_dpsiegr1_dirq10eg	IRQ10-DS edge select
= 0x08U	k_dpsiegr1_dirq11eg	IRQ11-DS edge select
= 0x10U	k_dpsiegr1_dirq12eg	IRQ12-DS edge select
= 0x20U	k_dpsiegr1_dirq13eg	IRQ13-DS edge select
= 0x40U	k_dpsiegr1_dirq14eg	IRQ14-DS edge select
= 0x80U	k_dpsiegr1_dirq15eg	IRQ15-DS edge select

Since Version 1.0.0

—

rx_dpsiegr2_bits_t

DPSIEGR2 bit definitions (LVD, NMI, I2C edge select).

Selects edge polarity for various peripheral wake-up detection. For LVD: 0 = VCC fall ($VCC < V_{det}$), 1 = VCC rise ($VCC \geq V_{det}$). For pins: 0 = Falling edge, 1 = Rising edge.

Value	Name	Description
= 0x01U	k_dpsiegr2_dlvd1eg	LVD1 edge: 0=fall, 1=rise
= 0x02U	k_dpsiegr2_dlvd2eg	LVD2 edge: 0=fall, 1=rise
= 0x10U	k_dpsiegr2_dnmieg	NMI edge select
= 0x20U	k_dpsiegr2_driicdeg	SDA2-DS edge select

= 0x40U	k_dpsiegr2_driicceg	SCL2-DS edge select
---------	---------------------	---------------------

Since Version 1.0.0

—

rx_dpsiegr3_bits_t

DPSIEGR3 bit definitions (CAN edge select).

Selects edge polarity for CAN receive wake-up detection. 0 = Falling edge, 1 = Rising edge.

Value	Name	Description
= 0x01U	k_dpsiegr3_dcanieg	CRX1-DS edge select

Since Version 1.0.0

—

opccr_reg

```
static volatile uint8_t * opccr_reg(void)
```

Get pointer to OPCCR register.

OPCCR controls operating power mode for power optimization.

Returns: Volatile pointer to Operating Power Control Register

Example - Enter Low-Speed Mode 1: *opccr_reg()=k_opccr_opcm_lowspeed1;
while(*opccr_reg()&k_opccr_opcmstsf){}

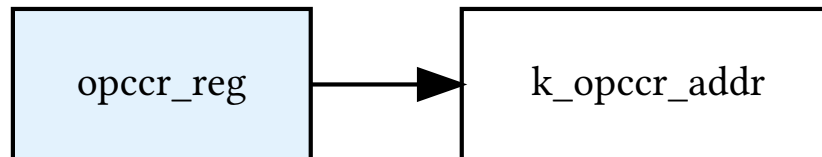


Figure 582: Call graph for opccr_reg

_Defined in libs/rx_hal/inc/rx72n_lpc_regs.h:577_

Since Version 1.0.0

—

rstckcr_reg

```
static volatile uint8_t * rstckcr_reg(void)
```

Get pointer to RSTCKCR register.

RSTCKCR controls automatic clock source switching on sleep mode exit.

Returns: Volatile pointer to Sleep Mode Return Clock Source Register

Example - Auto-switch to HOCO on wake-up: *rstckcr_reg()=k_rstckcr_rstcken|
k_rstckcr_rstcksel_hoco;

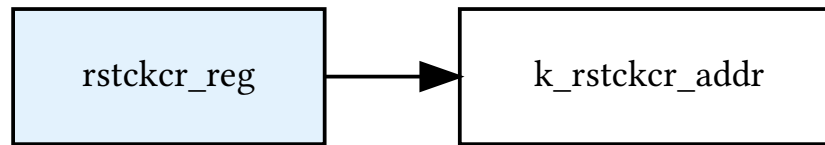


Figure 583: Call graph for rstckcr_reg

_Defined in libs/rx_hal/inc/rx72n_lpc_regs.h:597_

Since Version 1.0.0

—

dps_regs

```
static volatile rx_dps_regs_t * dps_regs(void)
```

Get pointer to Deep Standby registers.

Provides access to DPSBYCR, DPSIERx, DPSIFRx, and DPSIEGRx registers.

Returns: Volatile pointer to Deep Standby register structure

Example - Configure IRQ0 as wake-up source:: volatilerx_dps_regs_t*dps=dps_regs();

dps->dpsier0=k_dpsier0_dirq0e; dps->dpsiegr0=k_dpsiegr0_dirq0eg; dps->dpsifr0=0x00;

dps->dpsbycr=k_dpsbycr_dpsby;

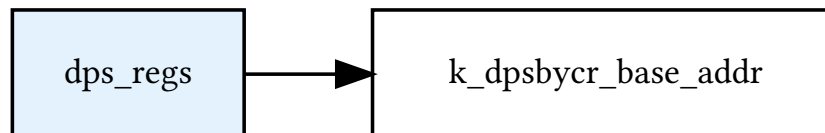


Figure 584: Call graph for dps_regs

_Defined in libs/rx_hal/inc/rx72n_lpc_regs.h:623_

Since Version 1.0.0

—

dpsbkr

```
static volatile uint8_t * dpsbkr(void)
```

Get pointer to Deep Standby Backup RAM.

DPSBKRY (y=0-31) is 32 bytes of RAM retained during deep software standby mode. Use this to preserve critical state across deep sleep.

Returns: Volatile pointer to 32-byte backup RAM array

Warning: Backup RAM is undefined after power-on; initialize before use.

Example - Save/restore state across deep standby:: volatileuint8_t*bkram=dpsbkr();
bkram[0]=magic_byte_1; bkram[1]=magic_byte_2; bkram[2]=(uint8_t)(counter>>8);
bkram[3]=(uint8_t)(counter&0xFF);
if(bkram[0]==magic_byte_1&&bkram[1]==magic_byte_2){ counter=((uint16_t)bkram[2]<<8)|
bkram[3]; }

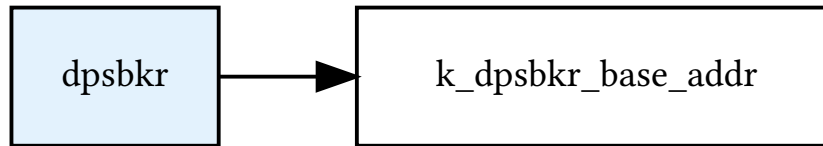


Figure 585: Call graph for dpsbkr

_Defined in libs/rx\hal/inc/rx72n\lpc_regs.h:656_

Since Version 1.0.0

—

rx72n_lvda_regs.h

RX72N Voltage Detection Circuit (LVDA) Register Definitions.

Register definitions for the LVDA module which provides brownout protection and voltage monitoring for safety-critical motor control. The LVDA can trigger resets or interrupts when VCC drops below configurable thresholds.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdint.h>

rx_lvda_addresses_t

LVDA register addresses.

The LVDA registers are located in two non-contiguous memory regions:

- Control/Status registers at 0x000800E0-0x000800E3
- Configuration registers at 0x0008C297-0x0008C29B

Value	Name	Description
= 0x000800E0	k_lvd1cr1_addr	Voltage Monitor 1 Control Register 1
= 0x000800E1	k_lvd1sr_addr	Voltage Monitor 1 Status Register
= 0x000800E2	k_lvd2cr1_addr	Voltage Monitor 2 Control Register 1
= 0x000800E3	k_lvd2sr_addr	Voltage Monitor 2 Status Register
= 0x0008C297	k_lvcmpcr_addr	Voltage Monitor Circuit Control Register
= 0x0008C298	k_lvd1vlr_addr	Voltage Detection Level Select Register
= 0x0008C29A	k_lvd1cr0_addr	Voltage Monitor 1 Control Register 0

= 0x0008C29B	k_lvd2cr0_addr	Voltage Monitor 2 Control Register 0
--------------	----------------	--------------------------------------

Note: Manual: Ch8.2 - Register Descriptions

Since Version 1.0.0

—

rx_lvd_cr1_bits_t

LVD1CR1/LVD2CR1 register bit definitions.

Control bits for interrupt generation condition and type selection.

Value	Name	Description
= 0x00	k_lvd_idtsel_rise	Interrupt on VCC >= Vdet (rise)
= 0x01	k_lvd_idtsel_drop	Interrupt on VCC < Vdet (drop)
= 0x02	k_lvd_idtsel_both	Interrupt on both rise and drop
= (0 < < 2)	k_lvd_irqsel_nmi	Non-maskable interrupt
= (1 < < 2)	k_lvd_irqsel_maskable	Maskable interrupt
= 0x03	k_lvd_idtsel_mask	IDTSEL field mask (bits 1:0)
= 0x04	k_lvd_irqsel_mask	IRQSEL field mask (bit 2)

Since Version 1.0.0

—

rx_lvd_sr_bits_t

LVD1SR/LVD2SR register bit definitions.

Status bits for voltage detection and monitoring.

Value	Name	Description
= 0x00	k_lvd_det_clear	Clear detection flag (write)
= 0x01	k_lvd_det_detected	Vdet passage detected (read)
= 0x00	k_lvd_mon_below	VCC < Vdet (read)
= 0x02	k_lvd_mon_above	VCC >= Vdet or monitor disabled (read)
= 0x01	k_lvd_det_mask	DET flag mask (bit 0)
= 0x02	k_lvd_mon_mask	MON flag mask (bit 1)

Since Version 1.0.0

—

rx_lvcmpr_bits_t

LVCMPR register bit definitions.

Enable bits for voltage detection circuits 1 and 2.

Value	Name	Description
= (1 < 5)	k_lvd1e_enable	Enable voltage detection 1 circuit
= (0 < 5)	k_lvd1e_disable	Disable voltage detection 1 circuit
= (1 < 6)	k_lvd2e_enable	Enable voltage detection 2 circuit
= (0 < 6)	k_lvd2e_disable	Disable voltage detection 2 circuit
= 0x20	k_lvd1e_mask	LVD1E field mask (bit 5)
= 0x40	k_lvd2e_mask	LVD2E field mask (bit 6)

Note: VCC must be at least 80mV above detection level when enabling

Since Version 1.0.0

—

rx_lvd1vlr_bits_t

LVDLVLr register bit definitions.

Voltage detection level selection for monitors 1 and 2.

Value	Name	Description
= 0x09	k_lvd1lvl_2v99	Vdet1 = 2.99V (highest)
= 0x0A	k_lvd1lvl_2v92	Vdet1 = 2.92V
= 0x0B	k_lvd1lvl_2v85	Vdet1 = 2.85V (lowest)
= 0x90	k_lvd2lvl_2v99	Vdet2 = 2.99V (highest)
= 0xA0	k_lvd2lvl_2v92	Vdet2 = 2.92V
= 0xB0	k_lvd2lvl_2v85	Vdet2 = 2.85V (lowest)
= 0x0F	k_lvd1lvl_mask	LVD1LVL field mask (bits 3:0)
= 0xF0	k_lvd2lvl_mask	LVD2LVL field mask (bits 7:4)
= 0xBB	k_lvd1vlr_reset_value	Both at 2.85V after reset

Note: Can only change when both LVD1E and LVD2E are 0

Warning: Other values are prohibited

Since Version 1.0.0

—

rx_lvd_cr0_bits_t

LVD1CR0/LVD2CR0 register bit definitions.

Control bits for interrupt/reset enable, digital filter, and mode selection.

Value	Name	Description
= (0 < 0)	k_lvd_rie_disable	Disable interrupt/reset

= (1 \<\< 0)	k_lvd_rie_enable	Enable interrupt/reset
= (0 \<\< 1)	k_lvd_dfdis_filter_on	Digital filter enabled
= (1 \<\< 1)	k_lvd_dfdis_filter_off	Digital filter disabled
= (0 \<\< 2)	k_lvd_cmpe_disable	Output disabled
= (1 \<\< 2)	k_lvd_cmpe_enable	Output enabled
= (0 \<\< 4)	k_lvd_fsamp_loco_div2	LOCO/2 (120 kHz)
= (1 \<\< 4)	k_lvd_fsamp_loco_div4	LOCO/4 (60 kHz)
= (2 \<\< 4)	k_lvd_fsamp_loco_div8	LOCO/8 (30 kHz)
= (3 \<\< 4)	k_lvd_fsamp_loco_div16	LOCO/16 (15 kHz)
= (0 \<\< 6)	k_lvd_ri_interrupt	Generate interrupt on Vdet passage
= (1 \<\< 6)	k_lvd_ri_reset	Generate reset when voltage falls below Vdet
= (0 \<\< 7)	k_lvd_rn_vcc_above	Negate after VCC > Vdet detected
= (1 \<\< 7)	k_lvd_rn_after_assert	Negate after stabilization time from assert
= 0x01	k_lvd_rie_mask	RIE field mask (bit 0)
= 0x02	k_lvd_dfdis_mask	DFDIS field mask (bit 1)
= 0x04	k_lvd_cmpe_mask	CMPE field mask (bit 2)
= 0x30	k_lvd_fsamp_mask	FSAMP field mask (bits 5:4)
= 0x40	k_lvd_ri_mask	RI field mask (bit 6)
= 0x80	k_lvd_rn_mask	RN field mask (bit 7)

Since Version 1.0.0

—

rx_lvd_interrupts_t

LVD interrupt vector numbers.

ICU interrupt vector numbers for voltage monitoring. These can be configured as NMI or maskable interrupts.

Value	Name	Description
= 88	k_lvd1_irq	LVD1 - Voltage monitor 1 interrupt
= 89	k_lvd2_irq	LVD2 - Voltage monitor 2 interrupt

Since Version 1.0.0

—

lvd1cr1_reg

```
static volatile uint8_t * lvd1cr1_reg(void)
```

Get pointer to LVD1CR1 register.

Voltage Monitoring 1 Circuit Control Register 1. Controls interrupt generation condition and interrupt type selection.

Returns: Volatile pointer to LVD1CR1 register

Register Bits:: b1:0 LVD1IDTSEL[1:0]: Interrupt generation condition select b2 LVD1IRQSEL: Interrupt type (0=NMI, 1=Maskable)

See also: rx_lvd1cr1_bits_t Bit definitions

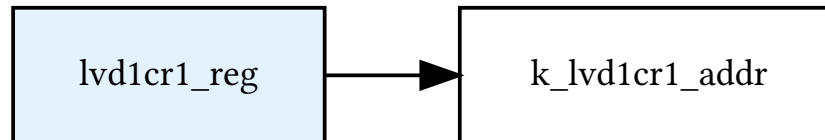


Figure 586: Call graph for lvd1cr1_reg

_Defined in libs/rx_hal/inc/rx72n_lvda_regs.h:181_

Since Version 1.0.0

—

lvd1sr_reg

```
static volatile uint8_t * lvd1sr_reg(void)
```

Get pointer to LVD1SR register.

Voltage Monitoring 1 Status Register. Contains detection flag and voltage monitor flag.

Returns: Volatile pointer to LVD1SR register

Register Bits:: b0 LVD1DET: Voltage change detection flag (write 0 to clear) b1 LVD1MON: Voltage monitor flag (0=VCC<Vdet1, 1=VCC>=Vdet1)

See also: rx_lvd1sr_bits_t Bit definitions

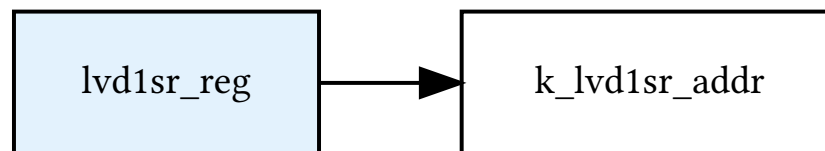


Figure 587: Call graph for lvd1sr_reg

_Defined in libs/rx_hal/inc/rx72n_lvda_regs.h:202_

Since Version 1.0.0

—

lvd2cr1_reg

```
static volatile uint8_t * lvd2cr1_reg(void)
```

Get pointer to LVD2CR1 register.

Voltage Monitoring 2 Circuit Control Register 1. Controls interrupt generation condition and interrupt type selection.

Returns: Volatile pointer to LVD2CR1 register

See also: `rx_lvd2cr1_bits_t` Bit definitions

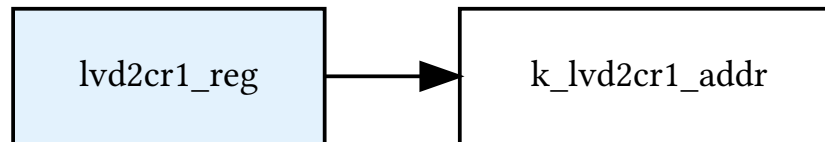


Figure 588: Call graph for `lvd2cr1_reg`

_Defined in `libs/rx_hal/inc/rx72n_lvda_regs.h:219_`

Since Version 1.0.0

—

`lvd2sr_reg`

```
static volatile uint8_t * lvd2sr_reg(void)
```

Get pointer to LVD2SR register.

Voltage Monitoring 2 Status Register. Contains detection flag and voltage monitor flag.

Returns: Volatile pointer to LVD2SR register

See also: `rx_lvd2sr_bits_t` Bit definitions

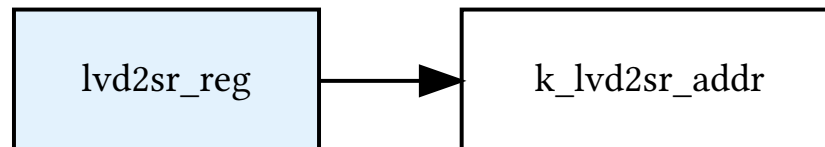


Figure 589: Call graph for `lvd2sr_reg`

_Defined in `libs/rx_hal/inc/rx72n_lvda_regs.h:236_`

Since Version 1.0.0

—

`lvcmpr_reg`

```
static volatile uint8_t * lvcmpr_reg(void)
```

Get pointer to LVCMPCR register.

Voltage Monitoring Circuit Control Register. Enables/disables voltage detection circuits 1 and 2.

Returns: Volatile pointer to LVCMPCR register

Register Bits:: b5 LVD1E: Voltage detection 1 circuit enable b6 LVD2E: Voltage detection 2 circuit enable

See also: `rx_lvcmpr_bits_t` Bit definitions

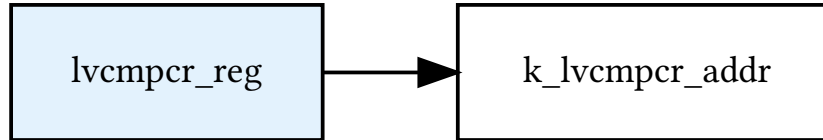


Figure 590: Call graph for `lvcmpr_reg`

_Defined in `libs/rx_hal/inc/rx72n_lvda_regs.h:257_`

Since Version 1.0.0

—

lvdvlr_reg

```
static volatile uint8_t * lvdvlr_reg(void)
```

Get pointer to LVDLVL register.

Voltage Detection Level Select Register. Sets the detection voltage thresholds for both voltage monitors.

Returns: Volatile pointer to LVDLVL register

Register Bits:: b3:0 LVD1LVL[3:0]: Voltage detection 1 level (2.85V/2.92V/2.99V) b7:4 LVD2LVL[3:0]: Voltage detection 2 level (2.85V/2.92V/2.99V)

See also: `rx_lvdvlr_bits_t` Bit definitions

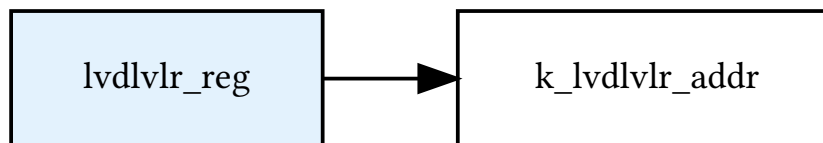


Figure 591: Call graph for `lvdvlr_reg`

_Defined in `libs/rx_hal/inc/rx72n_lvda_regs.h:278_`

Since Version 1.0.0

—

lvd1cr0_reg

```
static volatile uint8_t * lvd1cr0_reg(void)
```

Get pointer to LVD1CR0 register.

Voltage Monitoring 1 Circuit Control Register 0. Controls interrupt/reset enable, digital filter, and comparison output enable.

Returns: Volatile pointer to LVD1CR0 register

Register Bits:: b0 LVD1RIE: Interrupt/reset enable b1 LVD1DFDIS: Digital filter disable (0=enabled, 1=disabled) b2 LVD1CMPE: Comparison result output enable b5:4 LVD1FSAMP[1:0]: Digital filter sampling clock select b6 LVD1RI: Mode select (0=interrupt, 1=reset) b7 LVD1RN: Reset negate select

See also: rx_lvd1cr0_bits_t Bit definitions

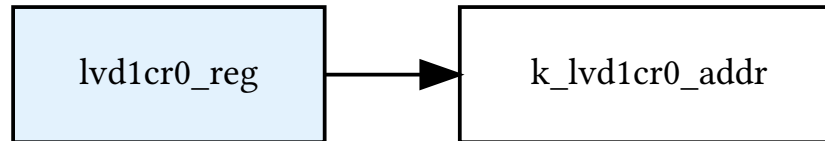


Figure 592: Call graph for lvd1cr0_reg

_Defined in libs/rx_hal/inc/rx72n_lvda_regs.h:303_

Since Version 1.0.0

—

lvd2cr0_reg

```
static volatile uint8_t * lvd2cr0_reg(void)
```

Get pointer to LVD2CR0 register.

Voltage Monitoring 2 Circuit Control Register 0. Controls interrupt/reset enable, digital filter, and comparison output enable.

Returns: Volatile pointer to LVD2CR0 register

See also: rx_lvd2cr0_bits_t Bit definitions

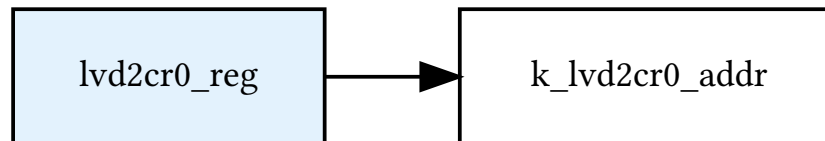


Figure 593: Call graph for lvd2cr0_reg

_Defined in libs/rx_hal/inc/rx72n_lvda_regs.h:320_

Since Version 1.0.0

—

rx72n_mpc_regs.h

RX72N Multi-Function Pin Controller (MPC) Register Definitions.

Register definitions for the Multi-Function Pin Controller (MPC) peripheral on the RX72N microcontroller. MPC configures which peripheral function is connected to each GPIO pin (pin multiplexing).

2026-01-28

Copyright (c) 2026 STAR Project

Includes:

- <stddef.h>
- <stdint.h>

rx72n_mpu_regs.h

RX72N Memory Protection Unit (MPU) Register Definitions.

Complete register definitions for the RX72N Memory Protection Unit. The MPU provides address-based access control for user-mode programs, protecting code and data regions from unauthorized access.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

mpu_addresses_t

MPU module addresses.

Base addresses for MPU registers. All addresses verified against RX72N Hardware Manual Ch17.

Value	Name	Description
= 0x00086400U	k_mpu_region_base_addr	MPU region registers base address.
= 0x00086500U	k_mpu_control_base_addr	MPU control registers base address.
= 0x08U	k_mpu_region_size	Region register block size (8 bytes per region: RSPAGE + REPAGE).
= 0x00086400U	k_mpu_rspage0_addr	RSPAGE0 address
= 0x00086408U	k_mpu_rspage1_addr	RSPAGE1 address
= 0x00086410U	k_mpu_rspage2_addr	RSPAGE2 address
= 0x00086418U	k_mpu_rspage3_addr	RSPAGE3 address
= 0x00086420U	k_mpu_rspage4_addr	RSPAGE4 address
= 0x00086428U	k_mpu_rspage5_addr	RSPAGE5 address
= 0x00086430U	k_mpu_rspage6_addr	RSPAGE6 address
= 0x00086438U	k_mpu_rspage7_addr	RSPAGE7 address
= 0x00086404U	k_mpu_repage0_addr	REPAGE0 address
= 0x0008640CU	k_mpu_repage1_addr	REPAGE1 address
= 0x00086414U	k_mpu_repage2_addr	REPAGE2 address
= 0x0008641CU	k_mpu_repage3_addr	REPAGE3 address
= 0x00086424U	k_mpu_repage4_addr	REPAGE4 address
= 0x0008642CU	k_mpu_repage5_addr	REPAGE5 address
= 0x00086434U	k_mpu_repage6_addr	REPAGE6 address
= 0x0008643CU	k_mpu_repage7_addr	REPAGE7 address

= 0x00086500U	k_mpu_mpen_addr	Memory Protection Enable
= 0x00086504U	k_mpu_mpbac_addr	Background Access Control
= 0x00086508U	k_mpu_mpeclr_addr	Error Status Clearing
= 0x0008650CU	k_mpu_mpests_addr	Error Status
= 0x00086514U	k_mpu_mpdea_addr	Data Error Address
= 0x00086520U	k_mpu_mpsa_addr	Region Search Address
= 0x00086524U	k_mpu_mpops_addr	Region Search Operation
= 0x00086526U	k_mpu_mpopi_addr	Region Invalidation Operation
= 0x00086528U	k_mpu_mhiti_addr	Instruction Hit Region
= 0x0008652CU	k_mpu_mhitd_addr	Data Hit Region

Note: Addresses verified 2026-01-29 against manual section 17.2

Since Version 1.0.0

—

mpu_region_t

MPU region numbers.

Region indices for use with accessor functions.

Value	Name	Description
= 0U	k_mpu_region_0	MPU region 0
= 1U	k_mpu_region_1	MPU region 1
= 2U	k_mpu_region_2	MPU region 2
= 3U	k_mpu_region_3	MPU region 3
= 4U	k_mpu_region_4	MPU region 4
= 5U	k_mpu_region_5	MPU region 5
= 6U	k_mpu_region_6	MPU region 6
= 7U	k_mpu_region_7	MPU region 7
= 8U	k_mpu_region_count	Total number of MPU regions

Since Version 1.0.0

—

mpu_page_t

MPU page configuration constants.

The MPU divides the address space into 16-byte pages. Page number = address[31:4] (upper 28 bits).

Value	Name	Description
= 16U	k_mpu_page_size	Page size in bytes (16 bytes).

= 4U	k_mpu_page_shift	Shift amount to convert address to page number.
= 0xFFFFFFFF0U	k_mpu_page_mask	Mask for page number bits [31:4].
= 0xFFFFFFFFFU	k_mpu_page_max	Maximum page number ($2^{28} - 1$).

Since Version 1.0.0

—

rspage_bits_t

Region Start Page Number Register (RSPAGEn) bit definitions.

Specifies the starting page number for a protection region. The page number corresponds to address bits [31:4].

Value	Name	Description
= 0x0000000FU	k_rspage_reserved_mask	Reserved bits mask (bits 3:0).
= 0xFFFFFFFF0U	k_rspage_rspn_mask	Start page number field mask (bits 31:4).
= 4U	k_rspage_rspn_shift	Start page number shift amount.

Note: Manual ref: Ch17 section 17.2.1

Since Version 1.0.0

—

repage_bits_t

Region End Page Number Register (REPAGEn) bit definitions.

Specifies the ending page number and access control for a protection region. The end page is INCLUDED in the protected region.

Value	Name	Description
= 0x00000001U	k_repage_v	Valid bit (bit 0) - Region enabled when set.
= 0x00000002U	k_repage_uac_x	Execute permission (bit 1) - Allow instruction fetch.
= 0x00000004U	k_repage_uac_w	Write permission (bit 2) - Allow data write.

= 0x00000008U	k_repage_uac_r	Read permission (bit 3) - Allow data read.
= 0x0000000EU	k_repage_uac_mask	Access control bits mask (UAC[2:0] = bits 3:1).
= 1U	k_repage_uac_shift	Access control shift amount.
= 0xFFFFFFFFU	k_repage_repn_mask	End page number field mask (bits 31:4).
= 4U	k_repage_repn_shift	End page number shift amount.
= k_repage_uac_r	k_repage_uac_ro	Read-only region (R).
= k_repage_uac_r k_repage_uac_w	k_repage_uac_rw	Read-write region (RW).
= k_repage_uac_x	k_repage_uac_xo	Execute-only region (X).
= k_repage_uac_r k_repage_uac_x	k_repage_uac_rx	Read-execute region (RX).
= k_repage_uac_r k_repage_uac_w k_repage_uac_x	k_repage_uac_rwx	Full access region (RWX).

Note: Manual ref: Ch17 section 17.2.2

Since Version 1.0.0

—

mpen_bits_t

Memory Protection Enable Register (MPEN) bit definitions.

Enables or disables memory protection globally. Protection takes effect when CPU transitions to user mode.

Value	Name	Description
= 0x00000001U	k_mpen_mpen	Memory Protection Enable bit (bit 0).

Note: Manual ref: Ch17 section 17.2.3

Since Version 1.0.0

—

mpbac_bits_t

Background Access Control Register (MPBAC) bit definitions.

Sets default access permissions for addresses not covered by any region. The background region covers the entire address space. Default is all access prohibited (0).

Value	Name	Description
= 0x00000002U	k_mpbac_ubac_x	Execute permission (bit 1) - Allow instruction fetch.
= 0x00000004U	k_mpbac_ubac_w	Write permission (bit 2) - Allow data write.
= 0x00000008U	k_mpbac_ubac_r	Read permission (bit 3) - Allow data read.
= 0x0000000EU	k_mpbac_ubac_mask	Access control bits mask (UBAC[2:0] = bits 3:1).
= 1U	k_mpbac_ubac_shift	Access control shift amount.

Note: Manual ref: Ch17 section 17.2.4

Since Version 1.0.0

—

mpeclr_bits_t

Error Status Clearing Register (MPECLR) bit definitions.

Writing 1 to CLR clears the IMPER, DMPER, and DRW bits in MPESTS. Always reads as 0.

Value	Name	Description
= 0x00000001U	k_mpeclr_clr	Error Status Clear bit (bit 0) - Write 1 to clear MPESTS.

Note: Manual ref: Ch17 section 17.2.5

Since Version 1.0.0

—

mpests_bits_t

Memory Protection Error Status Register (MPESTS) bit definitions.

Indicates the type of memory protection error that occurred. Bits are cleared by writing 1 to MPECLR.CLR.

Value	Name	Description
= 0x00000001U	k_mpests_imper	Instruction Memory Protection Error (bit 0).
= 0x00000002U	k_mpests_dmper	Data Memory Protection Error (bit 1).
= 0x00000004U	k_mpests_drw	Data Read/Write indicator (bit 2) - 0=read, 1=write.

Note: Manual ref: Ch17 section 17.2.6

Since Version 1.0.0

—

mpops_bits_t

Region Search Operation Register (MPOPS) bit definitions.

Writing 1 to S initiates a region search using the address in MPSA. Results are stored in MHITD. Always reads as 0.

Value	Name	Description
= 0x0001U	k_mpops_s	Region Search Start bit (bit 0).

Note: Manual ref: Ch17 section 17.2.9

Since Version 1.0.0

—

mpopi_bits_t

Region Invalidation Operation Register (MPOPI) bit definitions.

Writing 1 to INV clears the V (valid) bit in all REPAGEn registers, effectively disabling all region protections (only background remains). Always reads as 0.

Value	Name	Description
= 0x0001U	k_mpopi_inv	Region Invalidate Start bit (bit 0).

Note: Manual ref: Ch17 section 17.2.10

Since Version 1.0.0

—

mhiti_bits_t

Instruction Hit Region Register (MHITI) bit definitions.

After an instruction memory protection error, this register indicates which region caused the error and what permissions were set.

Value	Name	Description
= 0x00000002U	k_mhiti_uhaci_x	Execute permission in hit region (bit 1).

= 0x00000004U	k_mhiti_uhaci_w	Write permission in hit region (bit 2).
= 0x00000008U	k_mhiti_uhaci_r	Read permission in hit region (bit 3).
= 0x0000000EU	k_mhiti_uhaci_mask	Access control bits mask (bits 3:1).
= 0x00010000U	k_mhiti_hiti0	Region 0 hit flag (bit 16).
= 0x00020000U	k_mhiti_hiti1	Region 1 hit flag (bit 17).
= 0x00040000U	k_mhiti_hiti2	Region 2 hit flag (bit 18).
= 0x00080000U	k_mhiti_hiti3	Region 3 hit flag (bit 19).
= 0x00100000U	k_mhiti_hiti4	Region 4 hit flag (bit 20).
= 0x00200000U	k_mhiti_hiti5	Region 5 hit flag (bit 21).
= 0x00400000U	k_mhiti_hiti6	Region 6 hit flag (bit 22).
= 0x00800000U	k_mhiti_hiti7	Region 7 hit flag (bit 23).
= 0x00FF0000U	k_mhiti_hiti_mask	Hit region flags mask (bits 23:16).
= 16U	k_mhiti_hiti_shift	Hit region flags shift amount.

Note: HITI7:0 = 0 means error in background region

Note: Manual ref: Ch17 section 17.2.11

Since Version 1.0.0

—

mhited_bits_t

Data Hit Region Register (MHITD) bit definitions.

After a data memory protection error or region search, this register indicates which region was hit and what permissions were set.

Value	Name	Description
= 0x00000002U	k_mhited_uhacd_x	Execute permission in hit region (bit 1).
= 0x00000004U	k_mhited_uhacd_w	Write permission in hit region (bit 2).
= 0x00000008U	k_mhited_uhacd_r	Read permission in hit region (bit 3).
= 0x0000000EU	k_mhited_uhacd_mask	Access control bits mask (bits 3:1).
= 0x00010000U	k_mhited_hited0	Region 0 hit flag (bit 16).
= 0x00020000U	k_mhited_hited1	Region 1 hit flag (bit 17).
= 0x00040000U	k_mhited_hited2	Region 2 hit flag (bit 18).
= 0x00080000U	k_mhited_hited3	Region 3 hit flag (bit 19).
= 0x00100000U	k_mhited_hited4	Region 4 hit flag (bit 20).
= 0x00200000U	k_mhited_hited5	Region 5 hit flag (bit 21).
= 0x00400000U	k_mhited_hited6	Region 6 hit flag (bit 22).
= 0x00800000U	k_mhited_hited7	Region 7 hit flag (bit 23).

= 0x00FF0000U	k_mhited_hitd_mask	Hit region flags mask (bits 23:16).
= 16U	k_mhited_hitd_shift	Hit region flags shift amount.

Note: HITD7:0 = 0 means error in background region

Note: Manual ref: Ch17 section 17.2.12

Since Version 1.0.0

—

mpu_region

```
static volatile rx_mpu_region_regs_t * mpu_region(uint8_t region)
```

Get pointer to MPU region register pair.

Parameters:

Direction	Name	Description
in	region	Region number (0-7)

Returns: Volatile pointer to region register structure

Pre: region < k_mpu_region_count (8)

Note: Thread Safety: Safe - returns constant hardware address

Example:: volatile rx_mpu_region_regs_t *r0=mpu_region(k_mpu_region_0); r0->rspage=0x00000000; r0->repage=(0x0007FFF0)|k_repage_uac_rw|k_repage_v;

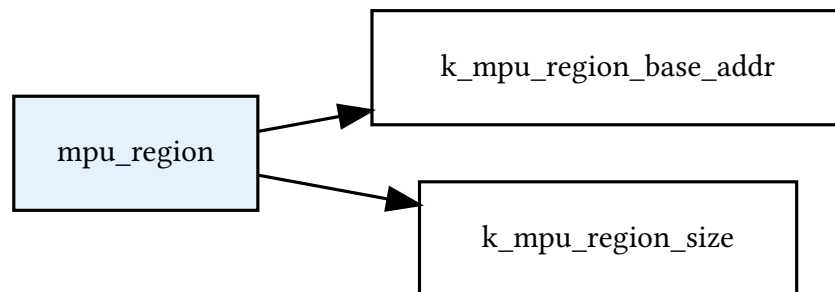


Figure 594: Call graph for mpu_region

_Defined in libs\rx_hal\inc\rx72n_mpu_regs.h:669_

Since Version 1.0.0

—

mpu_rspage_reg

```
static volatile uint32_t * mpu_rspage_reg(uint8_t region)
```

Get pointer to RSPAGE register for a region.

Parameters:

Direction	Name	Description
in	region	Region number (0-7)

Returns: Volatile pointer to RSPAGE register (32-bit)

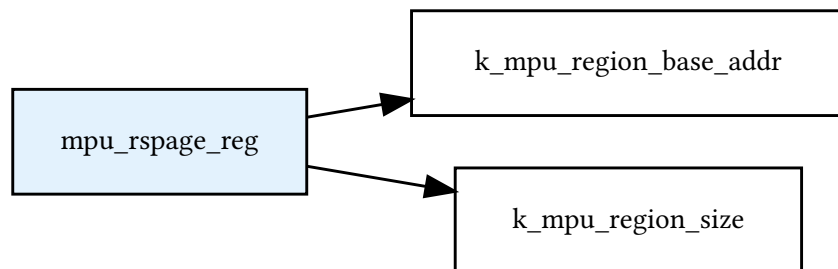


Figure 595: Call graph for `mpu_rspage_reg`

_Defined in `libs/rx\hal/inc/rx72n\mpu\_regs.h:681_`

Since Version 1.0.0

—

mpu_repage_reg

```
static volatile uint32_t * mpu_repage_reg(uint8_t region)
```

Get pointer to REPAGE register for a region.

Parameters:

Direction	Name	Description
in	region	Region number (0-7)

Returns: Volatile pointer to REPAGE register (32-bit)

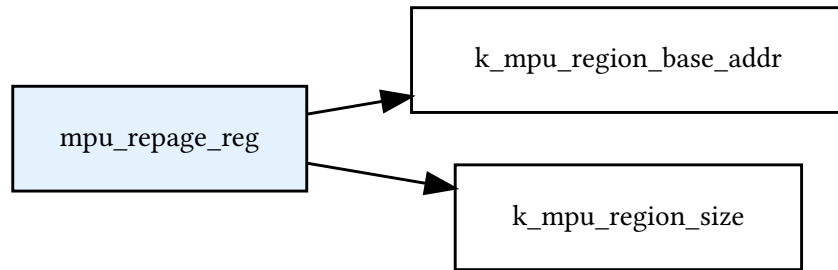


Figure 596: Call graph for mpu_repage_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:693_

Since Version 1.0.0

—

mpu_mpen_reg

```
static volatile uint32_t * mpu_mpen_reg(void)
```

Get pointer to MPEN (Memory Protection Enable) register.

Returns: Volatile pointer to MPEN register (32-bit)

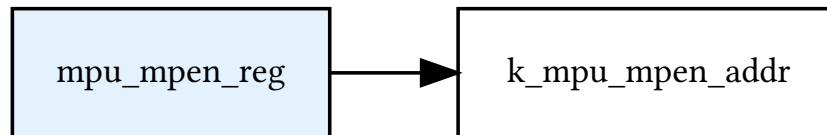


Figure 597: Call graph for mpu_mpen_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:708_

Since Version 1.0.0

—

mpu_mpbac_reg

```
static volatile uint32_t * mpu_mpbac_reg(void)
```

Get pointer to MPBAC (Background Access Control) register.

Returns: Volatile pointer to MPBAC register (32-bit)

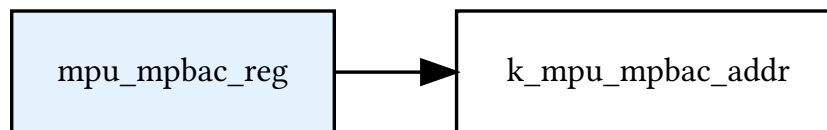


Figure 598: Call graph for mpu_mpbac_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:718_

Since Version 1.0.0

—

mpu_mpeclr_reg

```
static volatile uint32_t * mpu_mpeclr_reg(void)
```

Get pointer to MPECLR (Error Status Clearing) register.

Returns: Volatile pointer to MPECLR register (32-bit)

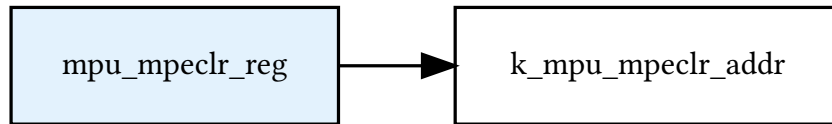


Figure 599: Call graph for mpu_mpeclr_reg

_Defined in `libs/rx_hal/inc/rx72n_mpu_regs.h:728_`

Since Version 1.0.0

—

mpu_mpests_reg

```
static volatile uint32_t * mpu_mpests_reg(void)
```

Get pointer to MPESTS (Error Status) register.

Returns: Volatile pointer to MPESTS register (32-bit)

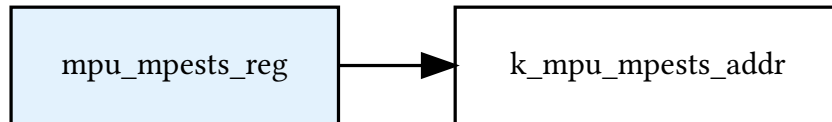


Figure 600: Call graph for mpu_mpests_reg

_Defined in `libs/rx_hal/inc/rx72n_mpu_regs.h:738_`

Since Version 1.0.0

—

mpu_mpdea_reg

```
static volatile uint32_t * mpu_mpdea_reg(void)
```

Get pointer to MPDEA (Data Error Address) register.

Returns: Volatile pointer to MPDEA register (32-bit)

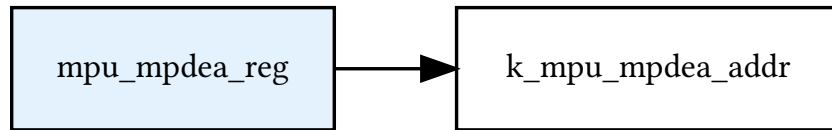


Figure 601: Call graph for mpu_mpdea_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:748_

Since Version 1.0.0

—

mpu_mpsa_reg

```
static volatile uint32_t * mpu_mpsa_reg(void)
```

Get pointer to MPSA (Region Search Address) register.

Returns: Volatile pointer to MPSA register (32-bit)

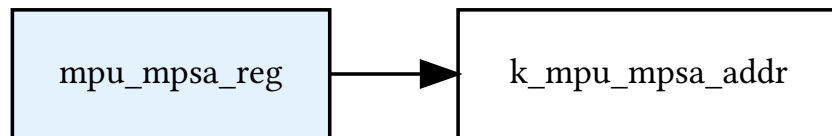


Figure 602: Call graph for mpu_mpsa_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:758_

Since Version 1.0.0

—

mpu_mpops_reg

```
static volatile uint16_t * mpu_mpops_reg(void)
```

Get pointer to MPOPS (Region Search Operation) register.

Returns: Volatile pointer to MPOPS register (16-bit)

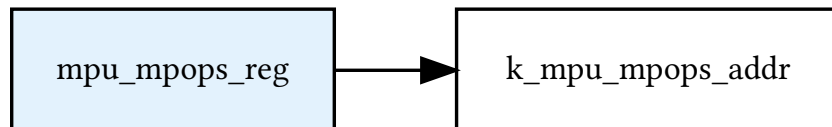


Figure 603: Call graph for mpu_mpops_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:768_

Since Version 1.0.0

—

mpu_mpopi_reg

```
static volatile uint16_t * mpu_mpopi_reg(void)
```

Get pointer to MPOPI (Region Invalidation) register.

Returns: Volatile pointer to MPOPI register (16-bit)

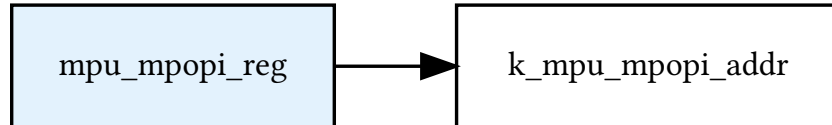


Figure 604: Call graph for mpu_mpopi_reg

Defined in `libs/rx\_hal/inc/rx72n\_mpu\_regs.h`:778

Since Version 1.0.0

—

mpu_mhiti_reg

```
static volatile uint32_t * mpu_mhiti_reg(void)
```

Get pointer to MHITI (Instruction Hit Region) register.

Returns: Volatile pointer to MHITI register (32-bit)

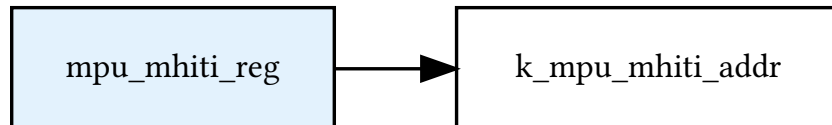


Figure 605: Call graph for mpu_mhiti_reg

Defined in `libs/rx\_hal/inc/rx72n\_mpu\_regs.h`:788

Since Version 1.0.0

—

mpu_mhitd_reg

```
static volatile uint32_t * mpu_mhitd_reg(void)
```

Get pointer to MHITD (Data Hit Region) register.

Returns: Volatile pointer to MHITD register (32-bit)

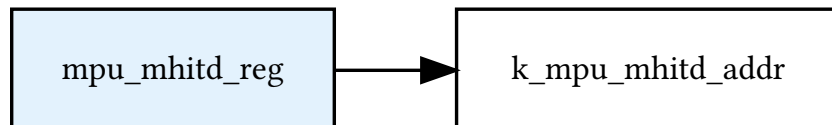


Figure 606: Call graph for mpu_mhitd_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:798_

Since Version 1.0.0

—

mpu_addr_to_page

```
static uint32_t mpu_addr_to_page(uint32_t addr)
```

Convert address to page number.

Parameters:

Direction	Name	Description
in	addr	Memory address

Returns: Page number (address[31:4])

Example:: uint32_tpage=mpu_addr_to_page(0x00012340);

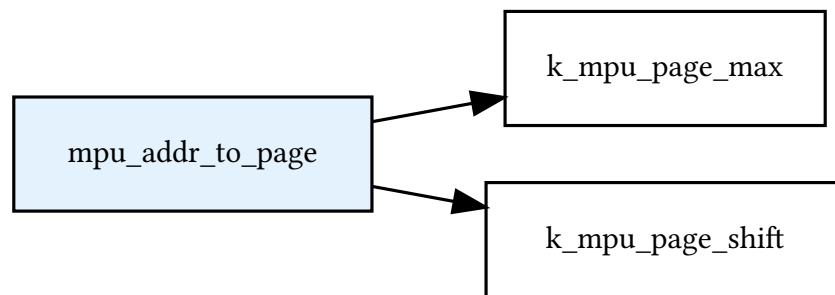


Figure 607: Call graph for mpu_addr_to_page

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:822_

Since Version 1.0.0

—

mpu_page_to_reg

```
static uint32_t mpu_page_to_reg(uint32_t page)
```

Convert page number to RSPAGE/REPAGE register value.

Parameters:

Direction	Name	Description
in	page	Page number (28 bits)

Returns: Register value with page number in bits [31:4]

Note: For REPAGE, caller must OR in access control and valid bits.

Example:: uint32_trspage_val=mpu_page_to_reg(0x00001234);
uint32_trepage_val=mpu_page_to_reg(0x00001234)|k_repage_uac_rw|k_repage_v;

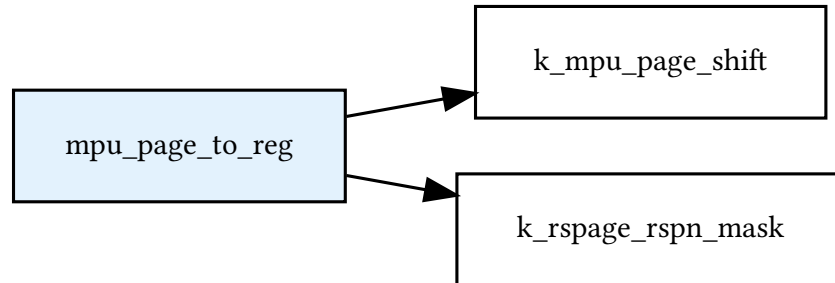


Figure 608: Call graph for mpu_page_to_reg

_Defined in libs/rx_hal/inc/rx72n_mpu_regs.h:843_

Since Version 1.0.0

—

rx72n_mtu_regs.h

RX72N Multi-Function Timer Unit (MTU3a) register definitions.

This file provides complete hardware register definitions for the RX72N's Multi-Function Timer Unit version 3a (MTU3a). The MTU3a is a highly flexible timer peripheral supporting PWM generation, pulse counting, phase counting (quadrature decoding), and general-purpose timing functions.

With 32-bit counter at 341 PPR x 4 edges = 1364 counts/rev:

- Range: +/-3,146,879 revolutions before overflow
- At 210 RPM max: 10.4 days continuous operation without wraparound

Available prescaler values: 1, 4, 16, 64, 256, 1024

Example: 20 kHz PWM at 120 MHz PCLKA, prescaler=1:

STAR Project Contributors

2026-01-01

1.0.0

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx72n_poe3_regs.h

RX72N Port Output Enable 3 (POE3) Register Definitions.

Complete register definitions for the RX72N Port Output Enable 3 module. POE3 provides emergency output disable for MTU (Multi-Function Timer Pulse Unit) pins, enabling hardware-level motor protection.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdint.h>
- <stdint.h>

poe3_addresses_t

POE3 module addresses.

Base address and register addresses for POE3. All addresses verified against RX72N Hardware Manual Ch25.

Value	Name	Description
= 0x0008C4C0U	k_poe3_base_addr	POE3 module base address.
= 0x0008C4C0U	k_poe3_icsr1_addr	ICSR1 address - Input Level Control/Status 1 (POE0#).
= 0x0008C4C4U	k_poe3_icsr2_addr	ICSR2 address - Input Level Control/Status 2 (POE4#).
= 0x0008C4C8U	k_poe3_icsr3_addr	ICSR3 address - Input Level Control/Status 3 (POE8#).
= 0x0008C4D6U	k_poe3_icsr4_addr	ICSR4 address - Input Level Control/Status 4 (POE10#).
= 0x0008C4D8U	k_poe3_icsr5_addr	ICSR5 address - Input Level Control/Status 5 (POE11#).
= 0x0008C4DCU	k_poe3_icsr6_addr	ICSR6 address - Input Level Control/Status 6 (osc stop).
= 0x0008C4C2U	k_poe3_ocsr1_addr	OCSR1 address - Output Level Control/Status 1 (MTU3/4).
= 0x0008C4C6U	k_poe3_ocsr2_addr	OCSR2 address - Output Level Control/Status 2 (MTU6/7).
= 0x0008C4CAU	k_poe3_spoer_addr	SPOER address - Software Port Output Enable Register.
= 0x0008C4CBU	k_poe3_poecr1_addr	POECR1 address - Port Output Enable Control 1 (MTU0).
= 0x0008C4CCU	k_poe3_poecr2_addr	POECR2 address - Port Output Enable Control 2 (MTU3/4).
= 0x0008C4D0U	k_poe3_poecr4_addr	POECR4 address - Port Output Enable Control 4 (MTU6/7).
= 0x0008C4D2U	k_poe3_poecr5_addr	POECR5 address - Port Output Enable Control 5 (additional).
= 0x0008C4DAU	k_poe3_alr1_addr	ALR1 address - Active Level Register 1.
= 0x0008C4E4U	k_poe3_m0selr1_addr	M0SELR1 address - MTU0 Pin Select 1.
= 0x0008C4E5U	k_poe3_m0selr2_addr	M0SELR2 address - MTU0 Pin Select 2.
= 0x0008C4E6U	k_poe3_m3selr_addr	M3SELR address - MTU3 Pin Select.
= 0x0008C4E7U	k_poe3_m4selr1_addr	M4SELR1 address - MTU4 Pin Select 1.
= 0x0008C4E8U	k_poe3_m4selr2_addr	M4SELR2 address - MTU4 Pin Select 2.
= 0x0008C4EAU	k_poe3_m6selr_addr	M6SELR address - MTU6 Pin Select.

Note: Addresses verified 2026-01-29 against manual section 25.2

Since Version 1.0.0

—

poe3_offsets_t

POE3 register offsets from base address.

Offsets from POE3 base address (0x0008C4C0) for each register.

Value	Name	Description
= 0x00U	k_poe3_offset_icsr1	ICSR1 offset (16-bit)
= 0x02U	k_poe3_offset_ocrs1	OCSR1 offset (16-bit)
= 0x04U	k_poe3_offset_icsr2	ICSR2 offset (16-bit)
= 0x06U	k_poe3_offset_ocrs2	OCSR2 offset (16-bit)
= 0x08U	k_poe3_offset_icsr3	ICSR3 offset (16-bit)
= 0x0AU	k_poe3_offset_spoer	SPOER offset (8-bit)
= 0x0BU	k_poe3_offset_poecr1	POECR1 offset (8-bit)
= 0x0CU	k_poe3_offset_poecr2	POECR2 offset (8-bit)
= 0x10U	k_poe3_offset_poecr4	POECR4 offset (16-bit)
= 0x12U	k_poe3_offset_poecr5	POECR5 offset (16-bit)
= 0x16U	k_poe3_offset_icsr4	ICSR4 offset (16-bit)
= 0x18U	k_poe3_offset_icsr5	ICSR5 offset (16-bit)
= 0x1AU	k_poe3_offset_alr1	ALR1 offset (16-bit)
= 0x1CU	k_poe3_offset_icsr6	ICSR6 offset (16-bit)
= 0x24U	k_poe3_offset_m0selr1	M0SELR1 offset (8-bit)
= 0x25U	k_poe3_offset_m0selr2	M0SELR2 offset (8-bit)
= 0x26U	k_poe3_offset_m3selr	M3SELR offset (8-bit)
= 0x27U	k_poe3_offset_m4selr1	M4SELR1 offset (8-bit)
= 0x28U	k_poe3_offset_m4selr2	M4SELR2 offset (8-bit)
= 0x2AU	k_poe3_offset_m6selr	M6SELR offset (8-bit)

Since Version 1.0.0

—

icsr_poemode_t

POE# Mode Select bits (POE0M/POE4M/POE8M/POE10M/POE11M).

Selects input detection mode for POE# pins. These bits can only be modified once after reset.

Value	Name	Description
= 0x0000U	k_icsr_poemode_falling_edge	Accept on falling edge of POE# input.

= 0x0001U	k_icsr_poemode_lowlevel_pclk8	Low-level sampling: 16 samples at PCLK/8.
= 0x0002U	k_icsr_poemode_lowlevel_pclk16	Low-level sampling: 16 samples at PCLK/16.
= 0x0003U	k_icsr_poemode_lowlevel_pclk128	Low-level sampling: 16 samples at PCLK/128.
= 0x0003U	k_icsr_poemode_mask	POEm mode select mask (bits 1:0).

Note: Manual ref: Ch25 sections 25.2.1-25.2.5

Since Version 1.0.0

—

icsr_bits_t

ICSR common bit definitions.

Value	Name	Description
= 0x0100U	k_icsr_pie	Port Interrupt Enable (bit 8).
= 0x1000U	k_icsr_poef	POE# Flag (bit 12) - high-impedance request detected.
= 0x0200U	k_icsr3_poe8e	POE8 High-Impedance Enable (bit 9, ICSR3 only).

Since Version 1.0.0

—

icsr6_bits_t

ICSR6 bit definitions (Oscillation Stop Detection).

Value	Name	Description
= 0x0100U	k_icsr6_ostste	Oscillation Stop High-Impedance Enable (bit 8).
= 0x1000U	k_icsr6_oststf	Oscillation Stop Flag (bit 12).

Since Version 1.0.0

—

ocsr_bits_t

OCSR bit definitions (simultaneous conduction detection).

Used to detect and respond to simultaneous active-level output on complementary PWM pin pairs (shoot-through detection).

Value	Name	Description
= 0x0100U	k_ocsr_oie	Simultaneous Conduction Interrupt Enable (bit 8).
= 0x0200U	k_ocsr_oce	Simultaneous Conduction High-Impedance Enable (bit 9).
= 0x8000U	k_ocsr_osf	Simultaneous Conduction Flag (bit 15).

Note: Manual ref: Ch25 sections 25.2.7, 25.2.8

Since Version 1.0.0

spoer_bits_t

SPOER bit definitions (software high-impedance control).

Software-triggered emergency stop for MTU pins. Writing 1 puts outputs in high-impedance state immediately.

Value	Name	Description
= 0x01U	k_spoer_mtuch34hiz	MTU3/4 Pins High-Impedance Enable (bit 0).
= 0x02U	k_spoer_mtuch67hiz	MTU6/7 Pins High-Impedance Enable (bit 1).
= 0x04U	k_spoer_mtuch0hiz	MTU0 Pins High-Impedance Enable (bit 2).

Note: Manual ref: Ch25 section 25.2.10

Since Version 1.0.0

poecr1_bits_t

POECR1 bit definitions (MTU0 pin high-impedance enable).

Value	Name	Description
= 0x01U	k_poecr1_mtu0aze	MTIOC0A high-impedance enable
= 0x02U	k_poecr1_mtu0bze	MTIOC0B high-impedance enable
= 0x04U	k_poecr1_mtu0cze	MTIOC0C high-impedance enable
= 0x08U	k_poecr1_mtu0dze	MTIOC0D high-impedance enable

Note: These bits can only be modified once after reset.

Since Version 1.0.0

poecr2_bits_t

POECR2 bit definitions (MTU3/4 complementary PWM control).

Value	Name	Description
= 0x01U	k_poecr2_mtu3bdze	MTIOC3B/3D high-impedance enable
= 0x02U	k_poecr2_mtu4acze	MTIOC4A/4C high-impedance enable
= 0x04U	k_poecr2_mtu4bdze	MTIOC4B/4D high-impedance enable

Note: These bits can only be modified once after reset.

Since Version 1.0.0

poecr4_bits_t

POECR4 bit definitions (MTU6/7 complementary PWM control).

Value	Name	Description
= 0x0001U	k_poecr4_mtu6bdze	MTIOC6B/6D high-impedance enable
= 0x0002U	k_poecr4_mtu7acze	MTIOC7A/7C high-impedance enable
= 0x0004U	k_poecr4_mtu7bdze	MTIOC7B/7D high-impedance enable

Note: These bits can only be modified once after reset.

Since Version 1.0.0

—

poecr5_bits_t

POECR5 bit definitions (additional POE# to pin mappings).

Value	Name	Description
= 0x0010U	k_poecr5_poe10e_mtu0	POE10# affects MTU0
= 0x0020U	k_poecr5_poe10e_mtu34	POE10# affects MTU3/4
= 0x0040U	k_poecr5_poe10e_mtu67	POE10# affects MTU6/7
= 0x0100U	k_poecr5_poe11e_mtu0	POE11# affects MTU0
= 0x0200U	k_poecr5_poe11e_mtu34	POE11# affects MTU3/4
= 0x0400U	k_poecr5_poe11e_mtu67	POE11# affects MTU6/7

Since Version 1.0.0

—

alr1_bits_t

ALR1 bit definitions (active level for shoot-through detection).

When OLSEN=0, active level determined by MTU.TOCR registers. When OLSEN=1, active level determined by OLSGnA/B bits (0=low, 1=high).

Value	Name	Description
= 0x0001U	k_alr1_olsg0a	Active level for group 0 positive
= 0x0002U	k_alr1_olsg0b	Active level for group 0 negative
= 0x0004U	k_alr1_olsg1a	Active level for group 1 positive
= 0x0008U	k_alr1_olsg1b	Active level for group 1 negative
= 0x0010U	k_alr1_olsg2a	Active level for group 2 positive
= 0x0020U	k_alr1_olsg2b	Active level for group 2 negative
= 0x0080U	k_alr1_olsen	Active level select enable

Since Version 1.0.0

—

poe3_mstpcr_t

POE3 module stop control bits.

POE3 is controlled by MSTPCRA.MSTPA9. Must be cleared (0) before accessing POE3 registers.

Value	Name	Description
= (1U << 9U)	k_poe3_mstpcra_bit	MSTPCRA bit for POE3 (bit 9).

Note: Manual ref: Ch11 (Low Power Consumption) module stop tables

Since Version 1.0.0

—

poe3_icsr1_reg

```
static volatile uint16_t * poe3_icsr1_reg(void)
```

Get pointer to ICSR1 register (POE0# control).

Returns: Volatile pointer to ICSR1 register (16-bit)

Pre: POE3 module stop bit (MSTPCRA.MSTPA9) must be cleared

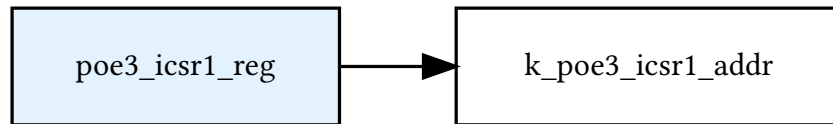


Figure 609: Call graph for poe3_icsr1_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:541_

Since Version 1.0.0

—

poe3_icsr2_reg

```
static volatile uint16_t * poe3_icsr2_reg(void)
```

Get pointer to ICSR2 register (POE4# control).

Returns: Volatile pointer to ICSR2 register (16-bit)

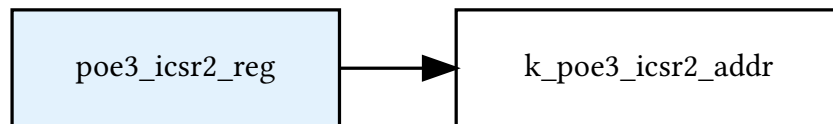


Figure 610: Call graph for poe3_icsr2_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:551_

Since Version 1.0.0

poe3_icsr3_reg

```
static volatile uint16_t * poe3_icsr3_reg(void)
```

Get pointer to ICSR3 register (POE8# control).

Returns: Volatile pointer to ICSR3 register (16-bit)

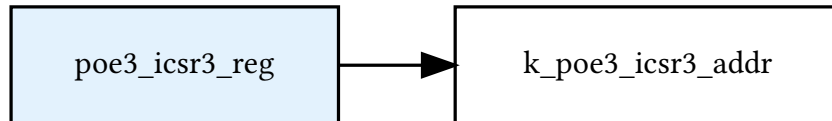


Figure 611: Call graph for `poe3_icsr3_reg`

_Defined in `libs/rx_hal/inc/rx72n_poe3_regs.h:561_`

Since Version 1.0.0

poe3_icsr4_reg

```
static volatile uint16_t * poe3_icsr4_reg(void)
```

Get pointer to ICSR4 register (POE10# control).

Returns: Volatile pointer to ICSR4 register (16-bit)

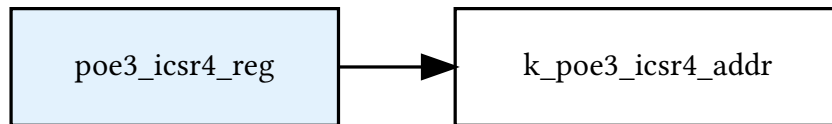


Figure 612: Call graph for `poe3_icsr4_reg`

_Defined in `libs/rx_hal/inc/rx72n_poe3_regs.h:571_`

Since Version 1.0.0

poe3_icsr5_reg

```
static volatile uint16_t * poe3_icsr5_reg(void)
```

Get pointer to ICSR5 register (POE11# control).

Returns: Volatile pointer to ICSR5 register (16-bit)

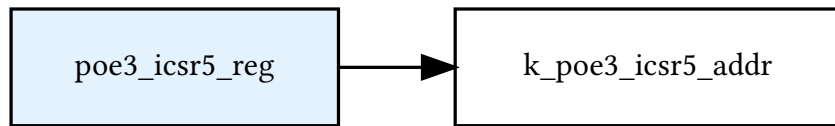


Figure 613: Call graph for poe3_icsr5_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:581_

Since Version 1.0.0

—

poe3_icsr6_reg

```
static volatile uint16_t * poe3_icsr6_reg(void)
```

Get pointer to ICSR6 register (oscillation stop).

Returns: Volatile pointer to ICSR6 register (16-bit)

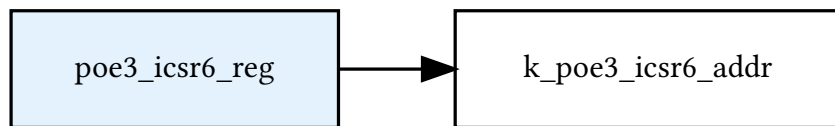


Figure 614: Call graph for poe3_icsr6_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:591_

Since Version 1.0.0

—

poe3_ocr1_reg

```
static volatile uint16_t * poe3_ocr1_reg(void)
```

Get pointer to OCSR1 register (MTU3/4 shoot-through).

Returns: Volatile pointer to OCSR1 register (16-bit)

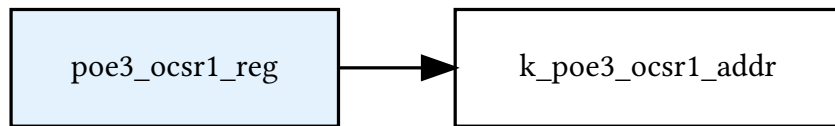


Figure 615: Call graph for poe3_ocr1_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:601_

Since Version 1.0.0

—

poe3_ocr2_reg

```
static volatile uint16_t * poe3_ocr2_reg(void)
```

Get pointer to OCSR2 register (MTU6/7 shoot-through).

Returns: Volatile pointer to OCSR2 register (16-bit)

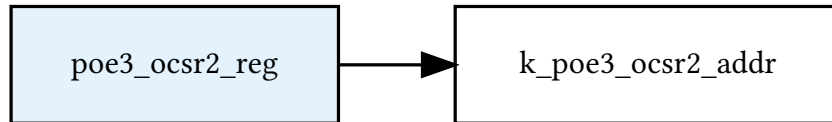


Figure 616: Call graph for `poe3_ocr2_reg`

_Defined in `libs/rx_hal/inc/rx72n\poe3\_regs.h:611_`

Since Version 1.0.0

—

poe3_spoer_reg

```
static volatile uint8_t * poe3_spoer_reg(void)
```

Get pointer to SPOER register (software emergency stop).

Returns: Volatile pointer to SPOER register (8-bit)

Example - Emergency stop all MTU outputs:: `*poe3_spoer_reg()=k_spoer_mtuch0hiz|k_spoer_mtuch34hiz|k_spoer_mtuch67hiz;`

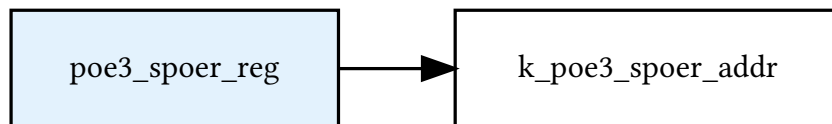


Figure 617: Call graph for `poe3_spoer_reg`

_Defined in `libs/rx_hal/inc/rx72n\poe3\_regs.h:628_`

Since Version 1.0.0

—

poe3_poecr1_reg

```
static volatile uint8_t * poe3_poecr1_reg(void)
```

Get pointer to POECR1 register (MTU0 pin control).

Returns: Volatile pointer to POECR1 register (8-bit)

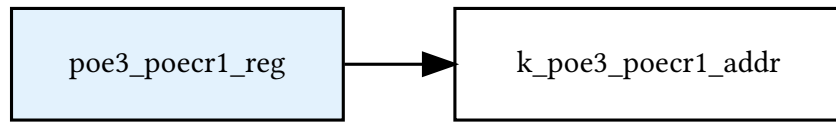


Figure 618: Call graph for poe3_poecr1_reg

Defined in `libs/rx_hal/inc/rx72n_poe3_regs.h`:638

Since Version 1.0.0

—

poe3_poecr2_reg

```
static volatile uint8_t * poe3_poecr2_reg(void)
```

Get pointer to POECR2 register (MTU3/4 pin control).

Returns: Volatile pointer to POECR2 register (8-bit)

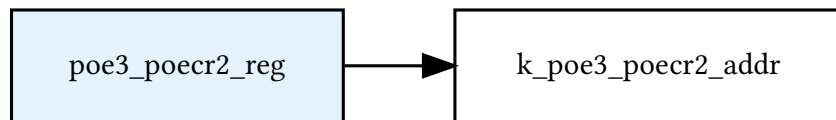


Figure 619: Call graph for poe3_poecr2_reg

Defined in `libs/rx_hal/inc/rx72n_poe3_regs.h`:648

Since Version 1.0.0

—

poe3_poecr4_reg

```
static volatile uint16_t * poe3_poecr4_reg(void)
```

Get pointer to POECR4 register (MTU6/7 pin control).

Returns: Volatile pointer to POECR4 register (16-bit)

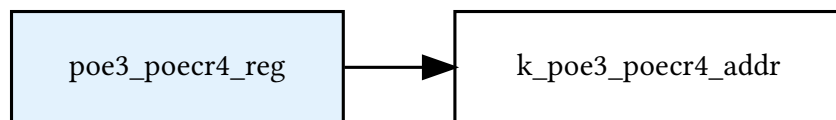


Figure 620: Call graph for poe3_poecr4_reg

Defined in `libs/rx_hal/inc/rx72n_poe3_regs.h`:658

Since Version 1.0.0

—

poe3_poecr5_reg

```
static volatile uint16_t * poe3_poecr5_reg(void)
```

Get pointer to POECR5 register (additional control).

Returns: Volatile pointer to POECR5 register (16-bit)

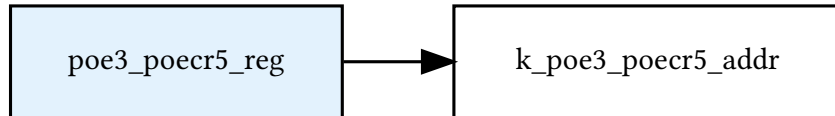


Figure 621: Call graph for poe3_poecr5_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:668_

Since Version 1.0.0

—

poe3_alr1_reg

```
static volatile uint16_t * poe3_alr1_reg(void)
```

Get pointer to ALR1 register (active level).

Returns: Volatile pointer to ALR1 register (16-bit)

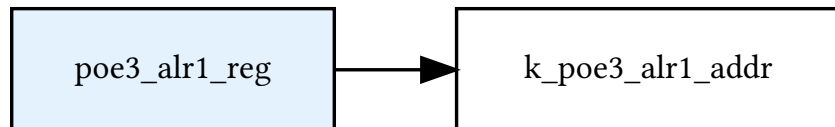


Figure 622: Call graph for poe3_alr1_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:678_

Since Version 1.0.0

—

poe3_m0selr1_reg

```
static volatile uint8_t * poe3_m0selr1_reg(void)
```

Get pointer to M0SELR1 register (MTU0 pin select 1).

Returns: Volatile pointer to M0SELR1 register (8-bit)

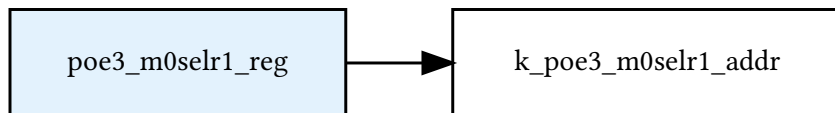


Figure 623: Call graph for poe3_m0selr1_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:688_

Since Version 1.0.0

—

poe3_m0selr2_reg

```
static volatile uint8_t * poe3_m0selr2_reg(void)
```

Get pointer to M0SELR2 register (MTU0 pin select 2).

Returns: Volatile pointer to M0SELR2 register (8-bit)

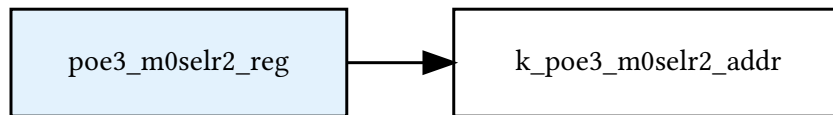


Figure 624: Call graph for poe3_m0selr2_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:698_

Since Version 1.0.0

—

poe3_m3selr_reg

```
static volatile uint8_t * poe3_m3selr_reg(void)
```

Get pointer to M3SELR register (MTU3 pin select).

Returns: Volatile pointer to M3SELR register (8-bit)

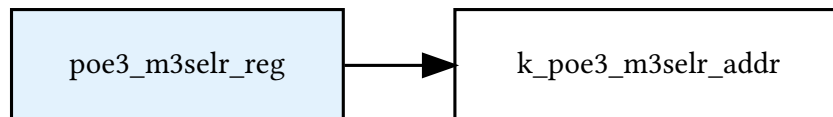


Figure 625: Call graph for poe3_m3selr_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:708_

Since Version 1.0.0

—

poe3_m4selr1_reg

```
static volatile uint8_t * poe3_m4selr1_reg(void)
```

Get pointer to M4SELR1 register (MTU4 pin select 1).

Returns: Volatile pointer to M4SELR1 register (8-bit)

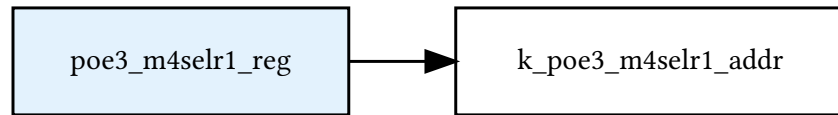


Figure 626: Call graph for poe3_m4selr1_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:718_

Since Version 1.0.0

—

poe3_m4selr2_reg

```
static volatile uint8_t * poe3_m4selr2_reg(void)
```

Get pointer to M4SEL2 register (MTU4 pin select 2).

Returns: Volatile pointer to M4SEL2 register (8-bit)

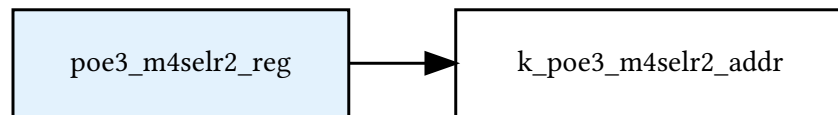


Figure 627: Call graph for poe3_m4selr2_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:728_

Since Version 1.0.0

—

poe3_m6selr_reg

```
static volatile uint8_t * poe3_m6selr_reg(void)
```

Get pointer to M6SEL2 register (MTU6 pin select).

Returns: Volatile pointer to M6SEL2 register (8-bit)

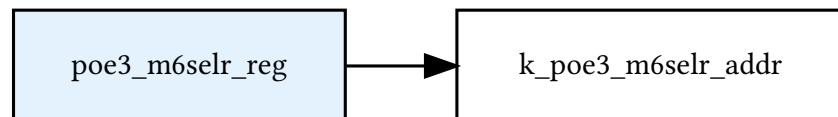


Figure 628: Call graph for poe3_m6selr_reg

_Defined in libs/rx_hal/inc/rx72n_poe3_regs.h:738_

Since Version 1.0.0

—

rx72n_poeg_regs.h

RX72N GPTW Port Output Enable (POEG) Register Definitions.

Register definitions for the POEG module which provides emergency PWM output disable functionality for the General PWM Timer (GPTW). The POEG is critical for motor control safety, allowing immediate PWM shutdown on fault detection.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stdint.h>`
- `<stdint.h>`

rx_poeg_addresses_t

POEG group base addresses.

Base addresses for each POEG group (A-D). Each group controls emergency output disable for associated GPTW channels. Groups are spaced 0x100 bytes apart.

Value	Name	Description
= 0x0009E000	k_poegga_base_addr	POEG Group A base address (0x0009E000).
= 0x0009E100	k_poeggb_base_addr	POEG Group B base address (0x0009E100).
= 0x0009E200	k_poeggc_base_addr	POEG Group C base address (0x0009E200).
= 0x0009E300	k_poeggd_base_addr	POEG Group D base address (0x0009E300).
= 0x100	k_poeg_group_spacing	Group spacing (0x100 bytes between groups).

Note: Manual: Ch27.2.1 - POEG Group n Setting Register] **See also:** `poegga()` Accessor function for Group A

Since Version 1.0.0

—

rx_poegg_bit_shifts_t

POEGGn register bit field positions.

Bit positions for POEGGn register fields. Used to construct field values.

Value	Name	Description
= 0	k_poeg_pidf_shift	PIDF - Port Input Detection Flag (bit 0)
= 1	k_poeg_iocf_shift	IOCF - GPTW Output Stop Flag (bit 1)
= 2	k_poeg_ostpf_shift	OSTPF - Oscillation Stop Flag (bit 2)
= 3	k_poeg_ssf_shift	SSF - Software Stop Flag (bit 3)
= 4	k_poeg_pide_shift	PIDE - Port Input Detection Enable (bit 4)
= 5	k_poeg_ioce_shift	IOCE - GPTW Output Stop Enable (bit 5)
= 6	k_poeg_ostpe_shift	OSTPE - Oscillation Stop Enable (bit 6)
= 16	k_poeg_st_shift	ST - GTETRGN Input Status (bit 16)

= 28	k_poeg_inv_shift	INV - Input Inverting (bit 28)
= 29	k_poeg_nfen_shift	NFEN - Noise Filter Enable (bit 29)
= 30	k_poeg_nfcs_shift	NFCS[1:0] - Noise Filter Clock Select (bits 31:30)

Since Version 1.0.0

—

rx_poegg_bits_t

POEGGn register bit definitions.

Configuration and status bit values for the POEG Group n Setting Register.

Value	Name	Description
= (1U << k_poeg_pidf_shift)	k_poeg_pidf_detected	Port input fault detected
= (1U << k_poeg_iocf_shift)	k_poeg_iocf_detected	GPTW output error detected
= (1U << k_poeg_ostpf_shift)	k_poeg_ostpf_detected	Oscillation stop detected
= (1U << k_poeg_ssf_shift)	k_poeg_ssf_stop	Software emergency stop active
= (1U << k_poeg_pide_shift)	k_poeg_pide_enable	Enable external pin detection
= (1U << k_poeg_ioce_shift)	k_poeg_ioce_enable	Enable GPTW error detection
= (1U << k_poeg_ostpe_shift)	k_poeg_ostpe_enable	Enable oscillation stop detection
= (1U << k_poeg_st_shift)	k_poeg_st_high	GTETRGN input is high

= (1U \<\< k_poeg_inv_shift)	k_poeg_inv_invert	Invert external trigger input
= (1U \<\< k_poeg_nfen_shift)	k_poeg_nfen_enable	Enable digital noise filter
= (0U \<\< k_poeg_nfcs_shift)	k_poeg_nfcs_pclkb_div1	Sample @ PCLKB/1 (50ns)
= (1U \<\< k_poeg_nfcs_shift)	k_poeg_nfcs_pclkb_div8	Sample @ PCLKB/8 (400ns)
= (2U \<\< k_poeg_nfcs_shift)	k_poeg_nfcs_pclkb_div32	Sample @ PCLKB/32 (1.6us)
= (3U \<\< k_poeg_nfcs_shift)	k_poeg_nfcs_pclkb_div128	Sample @ PCLKB/128 (6.4us)
= (k_poeg_pidf_detected k_poeg_iocf_detected k_poeg_ostpf_detected k_poeg_ssf_stop)	k_poeg_all_flags_mask	All flag bits
= (k_poeg_pide_enable k_poeg_ioce_enable k_poeg_ostpe_enable)	k_poeg_all_enable_mask	All enable bits
= (3U \<\< k_poeg_nfcs_shift)	k_poeg_nfcs_mask	NFCS field mask

See also: rx_poegg_regs_t::poeggn Register definition

Since Version 1.0.0

—

rx_poeg_interrupts_t

POEG interrupt vector numbers.

ICU interrupt vector numbers for POEG groups. These interrupts are triggered when PIDF or IOCF flags are set.

Value	Name	Description
-------	------	-------------

= 188	k_poeg_irq_group_a	POEGGAI - Group A interrupt
= 189	k_poeg_irq_group_b	POEGGBI - Group B interrupt
= 190	k_poeg_irq_group_c	POEGGCI - Group C interrupt
= 191	k_poeg_irq_group_d	POEGGDI - Group D interrupt

Note: Manual: Ch27.4 - Interrupt Sources

Since Version 1.0.0

—

poegga

```
static volatile rx_poegg_regs_t * poegga(void)
```

Get pointer to POEG Group A registers.

Returns a volatile pointer to POEG Group A register structure at address 0x0009E000. Use for emergency stop control of motor 0.

Returns: Volatile pointer to POEG Group A register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: GPTW/POEG module enabled (MSTPCRB.MSTPB22 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Example:: poegga()->poeggn|=k_poeg_ssf_stop;

See also: k_poegga_base_addr Base address constant

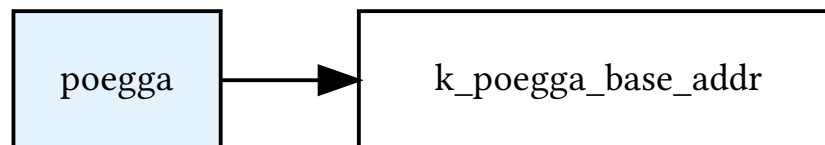


Figure 629: Call graph for poegga

_Defined in libs\rx_hal\inc\rx72n_poeg_regs.h:283_

Since Version 1.0.0

—

poeggb

```
static volatile rx_poegg_regs_t * poeggb(void)
```

Get pointer to POEG Group B registers.

Returns a volatile pointer to POEG Group B register structure at address 0x0009E100. Use for emergency stop control of motor 1.

Returns: Volatile pointer to POEG Group B register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: GPTW/POEG module enabled (MSTPCRB.MSTPB22 = 0)

Post: Pointer valid for lifetime of program execution] **See also:** k_poeggb_base_addr Base address constant

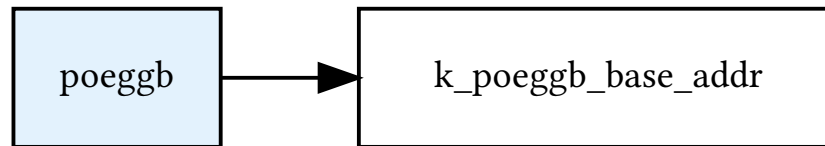


Figure 630: Call graph for poeggb

_Defined in libs/rx_hal/inc/rx72n_poeg_regs.h:304_

Since Version 1.0.0

—

poeggc

```
static volatile rx_poegg_regs_t * poeggc(void)
```

Get pointer to POEG Group C registers.

Returns a volatile pointer to POEG Group C register structure at address 0x0009E200. Use for emergency stop control of motor 2.

Returns: Volatile pointer to POEG Group C register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: GPTW/POEG module enabled (MSTPCRB.MSTPB22 = 0)

Post: Pointer valid for lifetime of program execution] **See also:** k_poeggc_base_addr Base address constant

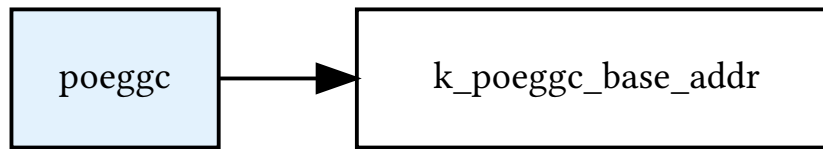


Figure 631: Call graph for poeggc

_Defined in `libs/rx\_hal/inc/rx72n\_poeg\_regs.h:325_`

Since Version 1.0.0

—

poeggd

```
static volatile rx_poegg_regs_t * poeggd(void)
```

Get pointer to POEG Group D registers.

Returns a volatile pointer to POEG Group D register structure at address 0x0009E300. Use for emergency stop control of motor 3.

Returns: Volatile pointer to POEG Group D register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: GPTW/POEG module enabled (MSTPCRB.MSTPB22 = 0)

Post: Pointer valid for lifetime of program execution] **See also:** `k_poeggd_base_addr` Base address constant

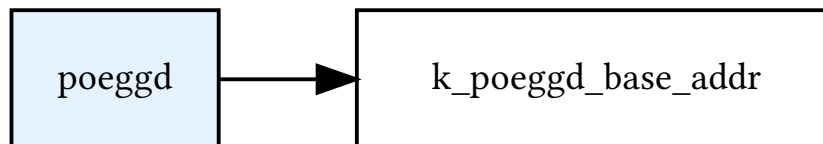


Figure 632: Call graph for poeggd

_Defined in `libs/rx\_hal/inc/rx72n\_poeg\_regs.h:346_`

Since Version 1.0.0

—

rx72n_port_regs.h

RX72N PORT (GPIO) register definitions for digital pin control.

This file provides complete hardware register definitions for the RX72N's General Purpose I/O (GPIO) port peripheral. The PORT module controls digital pin direction, output state, input reading, and special modes like open-drain and pull-up resistors.

This struct provides a PORT-centric view with correct padding to handle the type-grouped hardware layout transparently.

STAR Project Contributors

2026-01-05

1.0.0

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<assert.h>`
- `<stddef.h>`
- `<stdint.h>`

rx72n_ram_regs.h

RX72N RAM/Expansion RAM/ECCRAM register definitions.

This header defines register structures and addresses for the three types of high-speed static RAM on the RX72N MCU:

- RAM (512 KB at 0x00000000-0x0007FFFF): 1-bit parity checking, Memory Bus 1
- Expansion RAM (512 KB at 0x00800000-0x0087FFFF): 1-bit parity checking, Memory Bus 3
- ECCRAM (32 KB at 0x00FF8000-0x00FFFFFF): 1-bit error correction, 2-bit detection, Memory Bus 3

All register addresses are verified against the RX72N User's Manual Chapter 60.

Copyright (c) 2026 STAR Project

Includes:

- `<stddef.h>`
- `<stdint.h>`

rx_ram_region_addr_t

RAM region base addresses and sizes.

Defines the address ranges for all RAM regions on the RX72N. These are from Manual Ch60, Table 60.1.

Value	Name	Description
= 0x00000000	k_rx_ram_base_addr	RAM start address (512 KB, Memory Bus 1).
= 0x0007FFFF	k_rx_ram_end_addr	RAM end address (inclusive).
= 0x00080000	k_rx_ram_size	RAM size in bytes.
= 0x00800000	k_rx_exram_base_addr	Expansion RAM start address (512 KB, Memory Bus 3).
= 0x0087FFFF	k_rx_exram_end_addr	Expansion RAM end address (inclusive).
= 0x00880000	k_rx_exram_size	Expansion RAM size in bytes.
= 0x00FF8000	k_rx_eccram_base_addr	ECCRAM start address (32 KB, Memory Bus 3).
= 0x00FFFFFF	k_rx_eccram_end_addr	ECCRAM end address (inclusive).
= 0x00008000	k_rx_eccram_size	ECCRAM size in bytes.

—

rx_ram_reg_addr_t

RAM control register base addresses.

Defines register base addresses for RAM control, verified against Ch60.

Value	Name	Description
= 0x00081200	k_rx_ram_reg_base_addr	RAM control registers base address.
= 0x00081240	k_rx_exram_reg_base_addr	Expansion RAM control registers base address.
= 0x000812C0	k_rx_eccram_reg_base_addr	ECCRAM control registers base address.

—

rx_ram_reg_offset_t

RAM register offsets from base 0x00081200.

Offsets verified against Manual Ch60, sections 60.2.1-60.2.4.

Value	Name	Description
= 0x00	k_rx_ram_offset_rammode	RAMMODE @ 0x00081200 - RAM Operating Mode Control Register.
= 0x01	k_rx_ram_offset_ramsts	RAMSTS @ 0x00081201 - RAM Error Status Register.
= 0x04	k_rx_ram_offset_ramprcr	RAMPRCR @ 0x00081204 - RAM Protection Register.
= 0x08	k_rx_ram_offset_ramecad	RAMECAD @ 0x00081208 - RAM Error Address Capture Register.

—

rx_exram_reg_offset_t

Expansion RAM register offsets from base 0x00081240.

Offsets verified against Manual Ch60, sections 60.2.5-60.2.8.

Value	Name	Description
= 0x00	k_rx_exram_offset_exrammode	EXRAMMODE @ 0x00081240 - Expansion RAM Operating Mode Control Register.
= 0x01	k_rx_exram_offset_exramsts	EXRAMSTS @ 0x00081241 - Expansion RAM Error Status Register.
= 0x04	k_rx_exram_offset_exramprcr	EXRAMPRCR @ 0x00081244 - Expansion RAM Protection Register.
= 0x08	k_rx_exram_offset_exramecad	EXRAMECAD @ 0x00081248 - Expansion RAM Error Address Capture Register.

—

rx_eccram_reg_offset_t

ECCRAM register offsets from base 0x000812C0.

Offsets verified against Manual Ch60, sections 60.2.9-60.2.17.

Value	Name	Description
= 0x00	k_rx_eccram_offset_eccrammode	ECCRAMMODE @ 0x000812C0 - ECCRAM Operating Mode Control Register.
= 0x01	k_rx_eccram_offset_eccram2sts	ECCRAM2STS @ 0x000812C1 - ECCRAM 2-Bit Error Status Register.
= 0x02	k_rx_eccram_offset_eccram1stsen	ECCRAM1STSEN @ 0x000812C2 - ECCRAM 1-Bit Error Info Update Enable Register.
= 0x03	k_rx_eccram_offset_eccram1sts	ECCRAM1STS @ 0x000812C3 - ECCRAM 1-Bit Error Status Register.
= 0x04	k_rx_eccram_offset_eccramprcr	ECCRAMPRCR @ 0x000812C4 - ECCRAM Protection Register.
= 0x08	k_rx_eccram_offset_eccram2ecad	ECCRAM2ECAD @ 0x000812C8 - ECCRAM 2-Bit Error Address Capture Register.
= 0x0C	k_rx_eccram_offset_eccram1ecad	ECCRAM1ECAD @ 0x000812CC - ECCRAM 1-Bit Error Address Capture Register.
= 0x10	k_rx_eccram_offset_eccramprcr2	ECCRAMPRCR2 @ 0x000812D0 - ECCRAM Protection Register 2.
= 0x14	k_rx_eccram_offset_eccrametst	ECCRAMETST @ 0x000812D4 - ECCRAM Test Control Register.

rx_rammode_bits_t

RAMMODE register bit definitions.

RAM Operating Mode Select bits from Manual Ch60, section 60.2.1.

Value	Name	Description
= 0x00	k_rx_rammode_parity_disabled	Parity checking disabled (RAMMODE[1:0] = 00b).
= 0x01	k_rx_rammode_parity_enabled	Parity checking enabled (RAMMODE[1:0] = 01b).

rx_ramsts_bits_t

RAMSTS register bit definitions.

RAM Error Status Flag from Manual Ch60, section 60.2.2.

Value	Name	Description
= 0	k_rx_ramsts_ramerr_bit	RAM error flag bit position.
= 0x01	k_rx_ramsts_ramerr_mask	RAM error flag mask.

rx_ramprcr_bits_t

RAMPRCR register bit definitions.

RAM Protection Register from Manual Ch60, section 60.2.4. Write 0xF1 to enable writing to RAMMODE register.

Value	Name	Description
= 0	k_rx_ramprcr_ramprcr_bit	RAMPSCR bit position (enable RAMMODE writes).
= 0x01	k_rx_ramprcr_ramprcr_mask	RAMPSCR enable mask.
= 0x78	k_rx_ramprcr_key	Key word for enabling RAMPSCR write (KW[6:0] = 1111000b).
= 0x79	k_rx_ramprcr_unlock	Complete value to enable RAMMODE writes (key + RAMPSCR=1).
= 0x78	k_rx_ramprcr_lock	Complete value to disable RAMMODE writes (key + RAMPSCR=0).

—

rx_exrammode_bits_t

EXRAMMODE register bit definitions.

Expansion RAM Operating Mode Select from Manual Ch60, section 60.2.5.

Value	Name	Description
= 0x00	k_rx_exrammode_parity_disabled	Parity checking disabled (EXRAMMODE[1:0] = 00b).
= 0x01	k_rx_exrammode_parity_enabled	Parity checking enabled (EXRAMMODE[1:0] = 01b).

—

rx_exramsts_bits_t

EXRAMSTS register bit definitions.

Expansion RAM Error Status Flag from Manual Ch60, section 60.2.6.

Value	Name	Description
= 0	k_rx_exramsts_exramerr_bit	Expansion RAM error flag bit position.
= 0x01	k_rx_exramsts_exramerr_mask	Expansion RAM error flag mask.

—

rx_exramprcr_bits_t

EXRAMPRCR register bit definitions.

Expansion RAM Protection Register from Manual Ch60, section 60.2.8. Write 0x79 to enable writing to EXRAMMODE register.

Value	Name	Description
= 0	k_rx_exramprcr_exramprcr_bit	EXRAMPRCR bit position.
= 0x01	k_rx_exramprcr_exramprcr_mask	EXRAMPRCR enable mask.

= 0x78	k_rx_exramprcr_key	Key word for enabling EXRAMPRCR write.
= 0x79	k_rx_exramprcr_unlock	Complete value to enable EXRAMMODE writes.
= 0x78	k_rx_exramprcr_lock	Complete value to disable EXRAMMODE writes.

—

rx_eccrammode_bits_t

ECCRAMMODE register bit definitions.

ECCRAM Operating Mode Select from Manual Ch60, section 60.2.9.

Value	Name	Description
= 0x00	k_rx_eccrammode_ecc_disabled	ECCs disabled (RAMMOD[1:0] = 00b).
= 0x01	k_rx_eccrammode_prohibited	Setting prohibited (RAMMOD[1:0] = 01b) - DO NOT USE.
= 0x02	k_rx_eccrammode_ecc_no_check	ECCs enabled without error checking (RAMMOD[1:0] = 10b).
= 0x03	k_rx_eccrammode_ecc_with_check	ECCs enabled with error checking (RAMMOD[1:0] = 11b).

—

rx_eccram2sts_bits_t

ECCRAM2STS register bit definitions.

ECCRAM 2-Bit Error Status from Manual Ch60, section 60.2.10.

Value	Name	Description
= 0	k_rx_eccram2sts_ecc2err_bit	2-bit ECC error flag bit position
= 0x01	k_rx_eccram2sts_ecc2err_mask	2-bit ECC error flag mask

—

rx_eccram1stsen_bits_t

ECCRAM1STSEN register bit definitions.

ECCRAM 1-Bit Error Info Update Enable from Manual Ch60, section 60.2.11.

Value	Name	Description
= 0	k_rx_eccram1stsen_ecc1stsen_bit	1-bit ECC status enable bit position
= 0x01	k_rx_eccram1stsen_ecc1stsen_mask	1-bit ECC status enable mask

—

rx_eccram1sts_bits_t

ECCRAM1STS register bit definitions.

ECCRAM 1-Bit Error Status from Manual Ch60, section 60.2.12.

Value	Name	Description
= 0	k_rx_eccram1sts_ecc1err_bit	1-bit ECC error flag bit position
= 0x01	k_rx_eccram1sts_ecc1err_mask	1-bit ECC error flag mask

—

rx_eccramprcr_bits_t

ECCRAMPRCR register bit definitions.

ECCRAM Protection Register from Manual Ch60, section 60.2.13. Write 0x79 to enable writing to ECCRAMMODE and ECCRAM1STSEN registers.

Value	Name	Description
= 0	k_rx_eccramprcr_prcr_bit	PRCR bit position.
= 0x01	k_rx_eccramprcr_prcr_mask	PRCR enable mask.
= 0x78	k_rx_eccramprcr_key	Key word for enabling ECCRAMPRCR write.
= 0x79	k_rx_eccramprcr_unlock	Complete value to enable ECCRAMMODE/ ECCRAM1STSEN writes.
= 0x78	k_rx_eccramprcr_lock	Complete value to disable writes.

—

rx_eccramprcr2_bits_t

ECCRAMPRCR2 register bit definitions.

ECCRAM Protection Register 2 from Manual Ch60, section 60.2.16. Write 0x79 to enable writing to ECCRAMETST register.

Value	Name	Description
= 0	k_rx_eccramprcr2_prcr2_bit	PRCR2 bit position.
= 0x01	k_rx_eccramprcr2_prcr2_mask	PRCR2 enable mask.
= 0x78	k_rx_eccramprcr2_key	Key word for enabling ECCRAMPRCR2 write.
= 0x79	k_rx_eccramprcr2_unlock	Complete value to enable ECCRAMETST writes.
= 0x78	k_rx_eccramprcr2_lock	Complete value to disable writes.

—

rx_eccrametst_bits_t

ECCRAMETST register bit definitions.

ECCRAM Test Control Register from Manual Ch60, section 60.2.17. Used for ECC decoder testing.

Value	Name	Description
= 0	k_rx_eccrametst_tstbyp_bit	ECC bypass select bit position.
= 0x01	k_rx_eccrametst_tstbyp_mask	ECC bypass select mask.

—

rx_ram_mstpcrc_bits_t

MSTPCRC register bits for RAM module stop control.

Module stop control from Manual Ch60, section 60.4.1. Setting these bits stops clock supply to respective RAM areas.

Value	Name	Description
= 0	k_rx_mstpcrc_ram_bit	MSTPC0 - RAM module stop bit (stops RAM clock).
= 2	k_rx_mstpcrc_exram_bit	MSTPC2 - Expansion RAM module stop bit.
= 6	k_rx_mstpcrc_eccram_bit	MSTPC6 - ECCRAM module stop bit.

—

ram_regs

```
static rx_ram_regs_t * ram_regs(void)
```

Get pointer to RAM control registers.

Returns a pointer to the RAM control register structure at 0x00081200.

Returns: Pointer to rx_ram_regs_t structure

Example:: ram_regs()->ramprcr=k_rx_ramprcr_unlock; ram_regs()->rammode=k_rx_rammode_parity_enabled; ram_regs()->ramprcr=k_rx_ramprcr_lock;

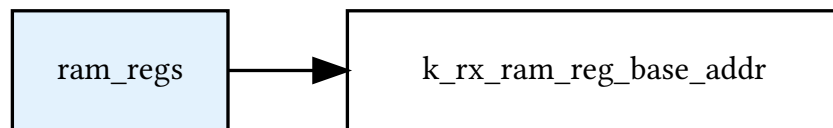


Figure 633: Call graph for ram_regs

_Defined in libs/rx_hal/inc/rx72n\ram_regs.h:491_

—

exram_regs

```
static rx_exram_regs_t * exram_regs(void)
```

Get pointer to Expansion RAM control registers.

Returns a pointer to the Expansion RAM control register structure at 0x00081240.

Returns: Pointer to rx_exram_regs_t structure

Example:: exram_regs()->exramprcr=k_rx_exramprcr_unlock; exram_regs()->exrammode=k_rx_exrammode_parity_enabled; exram_regs()->exramprcr=k_rx_exramprcr_lock;

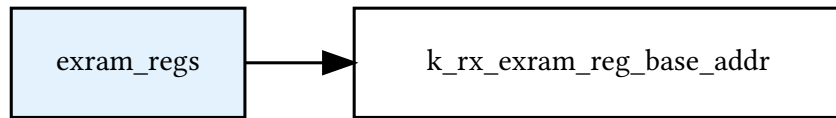


Figure 634: Call graph for exram_regs

_Defined in libs/rx_hal/inc/rx72n_ram_regs.h:512_

—

eccram_regs

```
static rx_eccram_regs_t * eccram_regs(void)
```

Get pointer to ECCRAM control registers.

Returns a pointer to the ECCRAM control register structure at 0x000812C0.

Returns: Pointer to rx_eccram_regs_t structure

Warning: Before enabling ECC with error checking, all ECCRAM must be initialized with valid ECC codes. Use k_rx_eccrammode_ecc_no_check mode first to initialize the memory, then enable error checking.

Example:: eccram_regs()->eccramprcr=k_rx_eccramprcr_unlock; eccram_regs()-
 >eccrammode=k_rx_eccrammode_ecc_with_check; eccram_regs()-
 >eccramprcr=k_rx_eccramprcr_lock;

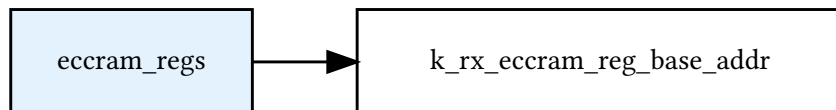


Figure 635: Call graph for eccram_regs

_Defined in libs/rx_hal/inc/rx72n_ram_regs.h:537_

—

rx72n_regs.h

RX72N Hardware Register Definitions - Main Aggregator Header.

Includes:

- <stdint.h>
- "rx72n_system_regs.h"
- "rx72n_port_regs.h"
- "rx72n_adc_regs.h"
- "rx72n_sci_regs.h"
- "rx72n_riic_regs.h"
- "rx72n_rspi_regs.h"
- "rx72n_cmt_regs.h"
- "rx72n_icu_regs.h"
- "rx72n_iwdt_regs.h"

- "rx72n_wdt_regs.h"
- "rx72n_crc_regs.h"
- "rx72n_usb_regs.h"
- "rx72n_mtu_regs.h"
- "rx72n_gptw_regs.h"
- "rx72n_poeg_regs.h"
- "rx72n_tpu_regs.h"
- "rx72n_lvda_regs.h"
- "rx72n_mpc_regs.h"
- "rx72n_flash_regs.h"
- "rx72n_ram_regs.h"
- "rx72n_dmac_regs.h"
- "rx72n_dtc_regs.h"
- "rx72n_mpu_regs.h"
- "rx72n_lpc_regs.h"
- "rx72n_elc_regs.h"
- "rx72n_clock.h"

rx72n_riic_regs.h

RX72N I2C Bus Interface (RIIC) Register Definitions.

Register definitions for the I2C Bus Interface (RIIC) peripheral on the RX72N microcontroller. RIIC provides I2C communication with support for both controller and peripheral modes.

Where:

- fSCL = SCL clock frequency (target bit rate)
- fPCLKB = Peripheral clock B (60 MHz)
- ICBRL = SCL low period setting (5 bits)
- ICBRH = SCL high period setting (5 bits)
- CKS = Internal clock divider (0-7)

2026-01-28

Copyright (c) 2026 STAR Project

Includes:

- <stddef.h>
- <stdint.h>

rx72n_rspi_regs.h

RX72N RSPI Serial Peripheral Interface Register Definitions.

This file defines all 22 RSPI registers per the RX72N Hardware Manual (R01UH0824EJ0120 Rev.1.20, Ch44). All register addresses, offsets, and bit definitions have been verified against the manual.

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx_rspi_addresses_t

RSPI channel base addresses.

Base addresses for the three RSPI channels on the RX72N. Each channel has a 64-byte register space (0x00-0x3F), though only 33 bytes are used. Addresses verified against RX72N Hardware Manual Ch44, Table 44.1.

k_rspi0_base_addr < k_rspi1_base_addr < k_rspi2_base_addr

Value	Name	Description
= 0x000D0100	k_rspi0_base_addr	RSPI0 register base address (0x000D0100).
= 0x000D0140	k_rspi1_base_addr	RSPI1 register base address (0x000D0140).
= 0x000D0300	k_rspi2_base_addr	RSPI2 register base address (0x000D0300).

See also: rspi0(), rspi1(), rspi2() Preferred accessor functions

Since Version 1.0.0

—

rspi_spcr_bits_t

RSPI Control Register (SPCR) Bit Definitions.

Controls RSPI operation including mode selection, interrupt enables, and function enable. The SPCR register must be configured before enabling RSPI with the SPE bit.

Value	Name	Description
= (1 << 7)	k_rspi_spcr_sprie	Receive Interrupt Enable (SPRIE) - Bit 7.
= (1 << 6)	k_rspi_spcr_spe	SPI Function Enable (SPE) - Bit 6.
= (1 << 5)	k_rspi_spcr_sptie	Transmit Interrupt Enable (SPTIE) - Bit 5.
= (1 << 4)	k_rspi_spcr_speie	Error Interrupt Enable (SPEIE) - Bit 4.
= (1 << 3)	k_rspi_spcr_mstr	Controller/Peripheral Mode Select (MSTR) - Bit 3.
= (1 << 2)	k_rspi_spcr_modfe	Mode Fault Error Detection Enable (MODFEN) - Bit 2.
= (1 << 1)	k_rspi_spcr_txmd	Transmit-Only Mode (TXMD) - Bit 1.
= (1 << 0)	k_rspi_spcr_spms	SPI Mode Select (SPMS) - Bit 0.

Warning: SPE must be cleared (0) before changing other bits] **See also:** rx_rspi_regs_t::spcr Register field in structure

Since Version 1.0.0

—

rspi_spsr_bits_t

RSPI Status Register (SPSR) Bit Definitions.

Indicates RSPI status including buffer full/empty flags and error flags. Status flags are used to determine when data can be read/written and to detect error conditions.

Value	Name	Description
-------	------	-------------

= (1 << 7)	k_rspsr_sprf	Receive Buffer Full Flag (SPRF) - Bit 7.
= (1 << 5)	k_rspsr_sptef	Transmit Buffer Empty Flag (SPTEF) - Bit 5.
= (1 << 4)	k_rspsr_udrf	Underrun Error Flag (UDRF) - Bit 4.
= (1 << 3)	k_rspsr_perf	Parity Error Flag (PERF) - Bit 3.
= (1 << 2)	k_rspsr_modf	Mode Fault Error Flag (MODF) - Bit 2.
= (1 << 1)	k_rspsr_idlnf	Idle Flag (IDLNF) - Bit 1.
= (1 << 0)	k_rspsr_ovrf	Overrun Error Flag (OVRF) - Bit 0.

Warning: Error flags must be cleared by writing 0 to the specific bit] **See also:**
 rx_rspsr_regs_t::spsr Register field in structure

Since Version 1.0.0

—

rspsr_sppcr_bits_t

RSPI Pin Control Register (SPPCR) Bit Definitions.

Controls RSPI pin behavior including loopback modes for testing and COPI (Controller Out Peripheral In) idle value control.

Value	Name	Description
= (1 << 6)	k_rspsr_sppcr_moife	COPI Idle Fixed Value Enable (MOIFE) - Bit 6.
= (1 << 5)	k_rspsr_sppcr_moifv	COPI Idle Fixed Value (MOIFV) - Bit 5.
= (1 << 1)	k_rspsr_sppcr_splp2	Loopback Mode 2 (SPLP2) - Bit 1.
= (1 << 0)	k_rspsr_sppcr_splp	Loopback Mode (SPLP) - Bit 0.

See also: rx_rspsr_regs_t::sppcr Register field in structure

Since Version 1.0.0

—

rspsr_spdcr_bits_t

RSPI Data Control Register (SPDCR) Bit Definitions.

Controls data access modes including word/byte access to SPDR register and receive data ready detection timing.

Value	Name	Description
= (1 << 5)	k_rspsr_spdcr_sprtdt	Receive Data Ready Detection (SPRDTD) - Bit 5.
= (1 << 4)	k_rspsr_spdcr_splw	Word Access Mode (SPLW) - Bit 4.

See also: rx_rspsr_regs_t::spdcr Register field in structure

Since Version 1.0.0

—

rspi0

```
static volatile rx_rspi_regs_t * rspi0(void)
```

Get pointer to RSPI0 registers (Raspberry Pi 5 communication).

Returns a volatile pointer to the RSPI0 register structure at address 0x000D0100. RSPI0 is configured as a peripheral for communication with the Raspberry Pi 5 controller at 10 Mbps.

Returns: Volatile pointer to RSPI0 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: None (hardware register always accessible)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Example:: `volatilerx_rspi_regs_t*spi=rspi0(); spi->spr=k_rspi_spr_spe;`

See also: `k_rspi0_base_addr` Base address constant, `rx_spi_comm_init()` Higher-level initialization

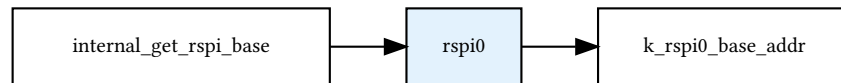


Figure 636: Call graph for rspi0

_Defined in `libs/rx_hal/inc/rx72n_rspi_regs.h:510_`

Since Version 1.0.0

—

rspi1

```
static volatile rx_rspi_regs_t * rspi1(void)
```

Get pointer to RSPI1 registers (not used in current design).

Returns a volatile pointer to the RSPI1 register structure at address 0x000D0140. RSPI1 is not used in the current STAR design.

Returns: Volatile pointer to RSPI1 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: None (hardware register always accessible)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Example:: `volatilerx_rspi_regs_t*spi=rspi1(); spi->spr=k_rspi_spr_spe|k_rspi_spr_mstr;`

See also: `k_rspi1_base_addr` Base address constant, `rspi0()` Primary SPI channel (RPi5 communication)

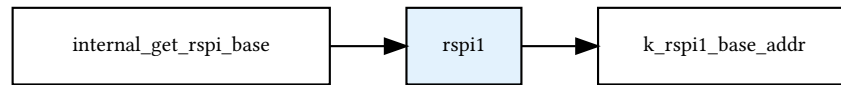


Figure 637: Call graph for rspi1

_Defined in `libs/rx\_hal/inc/rx72n\_rspi\_regs.h:541_`

Since Version 1.0.0

—

rspi2

```
static volatile rx_rspi_regs_t * rspi2(void)
```

Get pointer to RSPi2 registers (not used in current design).

Returns a volatile pointer to the RSPi2 register structure at address 0x000D0300. RSPi2 is not used in the current STAR design.

Returns: Volatile pointer to RSPi2 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: None (hardware register always accessible)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: Currently unused in STAR project] **See also:** `k_rspi2_base_addr` Base address constant

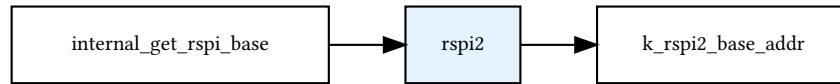


Figure 638: Call graph for rspi2

_Defined in `libs/rx\hal/inc/rx72n\_rspi\_regs.h:565_`

Since Version 1.0.0

—

rx72n_rtc_regs.h

RX72N Real-Time Clock (RTC) Register Definitions.

Minimal RTC register definitions for sub-clock oscillator disable functionality. The RTC module is not used in the STAR project. These definitions exist solely to properly disable the sub-clock oscillator during system initialization.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stdint.h>`

rtc_addresses_t

RTC module register addresses.

Addresses for RTC registers verified against RX72N Hardware Manual Ch33. Only addresses needed for sub-clock disable are defined.

Value	Name	Description
= 0x0008C400U	k_rtc_base_addr	RTC register base address (0x0008C400).
= 0x0008C426U	k_rtc_rcr3_addr	RTC Control Register 3 (RCR3) address.
= 0x0008C422U	k_rtc_rcr1_addr	RTC Control Register 1 (RCR1) address.
= 0x0008C424U	k_rtc_rcr2_addr	RTC Control Register 2 (RCR2) address.
= 0x0008C428U	k_rtc_rcr4_addr	RTC Control Register 4 (RCR4) address.

Note: Addresses verified 2026-01-29 against manual section 33.2

Since Version 1.0.0

—

rtc_offsets_t

RTC register offsets from base address.

Offsets verified against RX72N Hardware Manual Ch33 register table. Used for static assertions to verify struct layout.

Value	Name	Description
-------	------	-------------

= 0x00U	k_rtc_offset_r64cnt	64-Hz Counter offset
= 0x02U	k_rtc_offset_rseccnt	Second Counter offset (note: 0x02, not 0x01)
= 0x04U	k_rtc_offset_rmincnt	Minute Counter offset
= 0x06U	k_rtc_offset_rhrcnt	Hour Counter offset
= 0x08U	k_rtc_offset_rwkcnt	Day-of-Week Counter offset
= 0x0AU	k_rtc_offset_rdaycnt	Day Counter offset
= 0x0CU	k_rtc_offset_rmoncnt	Month Counter offset
= 0x0EU	k_rtc_offset_ryrcnt	Year Counter offset (16-bit)
= 0x10U	k_rtc_offset_rsecar	Second Alarm offset
= 0x12U	k_rtc_offset_rminar	Minute Alarm offset
= 0x14U	k_rtc_offset_rhrar	Hour Alarm offset
= 0x16U	k_rtc_offset_rwkar	Day-of-Week Alarm offset
= 0x18U	k_rtc_offset_rdayar	Day Alarm offset
= 0x1AU	k_rtc_offset_rmonar	Month Alarm offset
= 0x1CU	k_rtc_offset_ryrar	Year Alarm offset (16-bit)
= 0x1EU	k_rtc_offset_ryraren	Year Alarm Enable offset
= 0x22U	k_rtc_offset_rcr1	RTC Control Register 1 offset
= 0x24U	k_rtc_offset_rcr2	RTC Control Register 2 offset
= 0x26U	k_rtc_offset_rcr3	RTC Control Register 3 offset KEY
= 0x28U	k_rtc_offset_rcr4	RTC Control Register 4 offset
= 0x2AU	k_rtc_offset_rfrh	Frequency Register H offset
= 0x2CU	k_rtc_offset_rfrl	Frequency Register L offset
= 0x2EU	k_rtc_offset_radj	Time Error Adjustment offset

Note: Offsets verified 2026-01-29 against manual

Since Version 1.0.0

—

rkr3_bits_t

RTC Control Register 3 (RCR3) Bit Definitions.

Controls sub-clock oscillator input to the RTC module. This is the only RTC register actively used in the STAR project.

Value	Name	Description
= 0x00U	k_rkr3_rtcen_disable	Disable sub-clock input to RTC (RTCEN=0).
= 0x01U	k_rkr3_rtcen_enable	Enable sub-clock input to RTC (RTCEN=1).
= 0x01U	k_rkr3_rtcen_bit	RTCEN bit position (bit 0).

Note: Manual ref: Ch33 section 33.2.11 (page 1637)

Since Version 1.0.0

—

rtc_rcr3_reg

```
static volatile uint8_t * rtc_rcr3_reg(void)
```

Get pointer to RTC Control Register 3 (RCR3).

Returns a volatile pointer to RCR3 at address 0x0008C426. This is the only RTC register used in STAR project (for sub-clock disable).

Returns: Volatile pointer to RCR3 register (8-bit)

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: RTC module stop bit (MSTPCRC.MSTPC27) should be cleared for access

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead] **Address Verification:** Manual: Ch33 section 33.2.11 specifies RCR3 at 0x0008C426 This accessor uses the direct address, NOT struct offset Static assertion verifies address at compile time

Example (Sub-Clock Disable):: *rtc_rcr3_reg()=k_rcr3_rtcen_disable;

See also: k_rtc_rcr3_addr Address constant, rcr3_bits_t RCR3 control bits

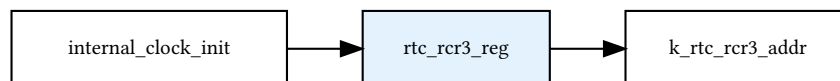


Figure 639: Call graph for rtc_rcr3_reg

_Defined in libs\rx_hal\inc\rx72n_rtc_regs.h:251_

Since Version 1.0.0

—

rtc_rcr1_reg

```
static volatile uint8_t * rtc_rcr1_reg(void)
```

Get pointer to RTC Control Register 1 (RCR1).

Returns a volatile pointer to RCR1 at address 0x0008C422. Not used in STAR project but provided for completeness.

Returns: Volatile pointer to RCR1 register (8-bit)

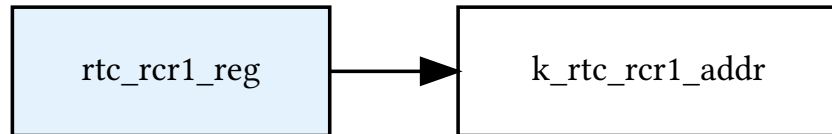


Figure 640: Call graph for rtc_rcr1_reg

_Defined in libs/rx_hal/inc/rx72n_rtc_regs.h:266_

Since Version 1.0.0

—

rtc_rcr2_reg

```
static volatile uint8_t * rtc_rcr2_reg(void)
```

Get pointer to RTC Control Register 2 (RCR2).

Returns a volatile pointer to RCR2 at address 0x0008C424. Not used in STAR project but provided for completeness.

Returns: Volatile pointer to RCR2 register (8-bit)

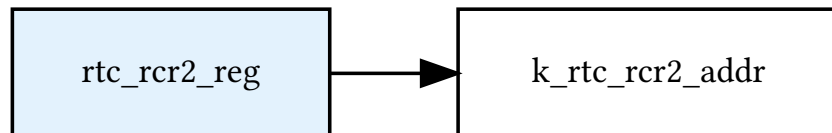


Figure 641: Call graph for rtc_rcr2_reg

_Defined in libs/rx_hal/inc/rx72n_rtc_regs.h:281_

Since Version 1.0.0

—

rtc_rcr4_reg

```
static volatile uint8_t * rtc_rcr4_reg(void)
```

Get pointer to RTC Control Register 4 (RCR4).

Returns a volatile pointer to RCR4 at address 0x0008C428. Not used in STAR project but provided for completeness.

Returns: Volatile pointer to RCR4 register (8-bit)

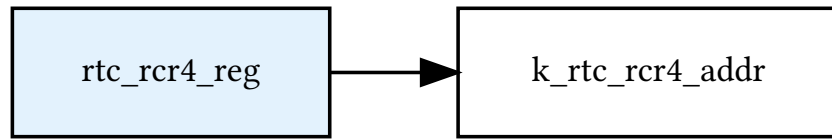


Figure 642: Call graph for rtc_rcr4_reg

_Defined in libs/rx\hal/inc/rx72n_rtc_regs.h:296_

Since Version 1.0.0

—

rx72n_sci_regs.h

RX72N Serial Communication Interface (SCI) register definitions.

This file provides complete hardware register definitions for the RX72N's Serial Communication Interface (SCI) peripheral. The SCI module supports multiple serial communication protocols including UART (asynchronous), clock-synchronous serial, smart card interface, and I2C modes.

Where:

- f_PCLKA = Peripheral clock A (120 MHz for STAR)
- n = SMR.CKS clock select (0, 1, 2, 3)
- B = Desired baud rate

Example: 115200 baud at 120 MHz PCLKA, n=0:

STAR Project Contributors

2026-01-28

1.0.0

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx72n_system_regs.h

RX72N System Control Register Definitions.

System control registers for clock generation, power management, oscillator control, and module stop control on the RX72N microcontroller. This is the foundation for system initialization and power optimization.

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx_system_addresses_t

System register base addresses.

Base addresses for system control registers. Note that system registers are spread across multiple non-contiguous memory regions.

Value	Name	Description
= 0x00080000	k_system_base_addr	System register base address (0x00080000).
= 0x000803FE	k_prcr_addr	PRCR (Protection Register) address (0x000803FE).

Warning: PRCR is NOT in the main system block - use prcr_reg() accessor] **See also:** system_regs() Main system register accessor, prcr_reg() PRCR register accessor

Since Version 1.0.0

—

system_reserved_sizes_t

System register reserved field sizes.

Defines reserved byte counts for gaps in the system register map. These ensure correct structure alignment with hardware.

Value	Name	Description
= 4	k_system_reserved_02_05	Reserved @ 0x02-0x05 (between MDMONR and SYSCR0)
= 2	k_system_reserved_0a_0b	Reserved @ 0x0A-0x0B (between SYSCR1 and SBYCR)
= 2	k_system_reserved_0e_0f	Reserved @ 0x0E-0x0F (between SBYCR and MSTPCRA)
= 5	k_system_reserved_2b_2f	Reserved @ 0x2B-0x2F (between PLLCR2 and BCKCR)
= 1	k_system_reserved_31	Reserved @ 0x31 (between BCKCR and MOSCCR)
= 4	k_system_reserved_38_3b	Reserved @ 0x38-0x3B (between HOCOCCR2 and OSCOVFSR)
= 1	k_system_reserved_3d	Reserved @ 0x3D (between OSCOVFSR and CKOCR)

Since Version 1.0.0

—

mdmonr_bits_t

Mode Monitor Register (MDMONR) bit definitions.

Read-only register indicating MCU mode pin status at reset. Located at address 0x00080000.

Value	Name	Description
= 0x0001U	k_mdmonr_md	MD pin status (0=low/boot, 1=high/single-chip)
= 0x0001U	k_mdmonr_md_mask	MD bit mask

Note: Manual ref: Ch03 section 3.2.1

Since Version 1.0.0

—

syscr0_bits_t

System Control Register 0 (SYSCR0) bit definitions.

Controls on-chip ROM enable and external bus enable. Requires key code 0x5A in upper byte for writes. Located at address 0x00080006.

Value	Name	Description
= 0x0001U	k_syscr0_rome	On-Chip ROM Enable (0=disabled, 1=enabled)
= 0x0002U	k_syscr0_exbe	External Bus Enable (0=disabled, 1=enabled)
= 0x5A00U	k_syscr0_key	Key code for SYSCR0 write (must be in upper byte)
= 0xFF00U	k_syscr0_key_mask	Key code mask
= (0x5A00U 0x0001U)	k_syscr0_rom_enabled	Enable ROM (write value)
= 0x5A00U	k_syscr0_rom_disabled	Disable ROM (write value) - IRREVERSIBLE
= (0x5A00U 0x0003U)	k_syscr0_extbus_enable	Enable ROM + ExtBus (write value)

Note: Manual ref: Ch03 section 3.2.2

Warning: Once ROME is set to 0, it cannot be reverted to 1

Warning: Do not modify EXBE while accessing external bus

Since Version 1.0.0

—

syscr1_bits_t

System Control Register 1 (SYSCR1) bit definitions.

Controls RAM, ECCRAM, and Standby RAM enable. Located at address 0x00080008.

Value	Name	Description
= 0x0001U	k_syscr1_rame	RAM Enable (0=disabled, 1=enabled)
= 0x003EU	k_syscr1_reserved_mask	Reserved bits 5:1 (read/write as 1)
= 0x0040U	k_syscr1_eccrame	ECCRAM Enable (0=disabled, 1=enabled)
= 0x0080U	k_syscr1_sbyrame	Standby RAM Enable (0=disabled, 1=enabled)
= 0x00FFU	k_syscr1_all_ram_enabled	Default: all RAM types enabled
= 0x00BFU	k_syscr1_disable_eccram	Disable ECCRAM (RAME+SBYRAME enabled)

= 0x007FU	k_syscr1_disable_sbyram	Disable StandbyRAM (RAME+ECCRAM enabled)
-----------	-------------------------	--

Note: Manual ref: Ch03 section 3.2.3

Warning: Do not write 0 to RAME/ECCRAM/SBYRAM during access to that memory

Since Version 1.0.0

—

ckocr_bits_t

Clock Output Control Register (CKOCR) bit definitions.

Controls the clock output on CLKOUT pin. Can output various system clocks divided by configurable ratio.

Value	Name	Description
= (0U \<\< 8)	k_ckocr_ckosel_loco	LOCO (low-speed on-chip osc)
= (1U \<\< 8)	k_ckocr_ckosel_hoco	HOCO (high-speed on-chip osc)
= (2U \<\< 8)	k_ckocr_ckosel_main	Main oscillator
= (3U \<\< 8)	k_ckocr_ckosel_subclock	Sub-clock oscillator
= (4U \<\< 8)	k_ckocr_ckosel_pll	PLL output
= (7U \<\< 8)	k_ckocr_ckosel_mask	CKOSEL field mask
= (0U \<\< 12)	k_ckocr_ckodiv_1	Divide by 1
= (1U \<\< 12)	k_ckocr_ckodiv_2	Divide by 2
= (2U \<\< 12)	k_ckocr_ckodiv_4	Divide by 4
= (3U \<\< 12)	k_ckocr_ckodiv_8	Divide by 8
= (4U \<\< 12)	k_ckocr_ckodiv_16	Divide by 16
= (5U \<\< 12)	k_ckocr_ckodiv_32	Divide by 32
= (6U \<\< 12)	k_ckocr_ckodiv_64	Divide by 64
= (7U \<\< 12)	k_ckocr_ckodiv_128	Divide by 128
= (7U \<\< 12)	k_ckocr_ckodiv_mask	CKODIV field mask
= (1U \<\< 15)	k_ckocr_ckostp	Clock output stop (1=stopped, 0=running)

= k_ckocr_ckostp	k_ckocr_output_disabled	Default: output disabled
= (k_ckocr_ckosel_pll k_ckocr_ckodiv_4)	k_ckocr_pll_div4	PLL/4 output (60 MHz @ 240 MHz PLL)

Note: Requires PRCR unlock (PRC0) before modification

Warning: Clock output disabled by default (CKOSTP = 1)] **See also:** Manual Ch09 section 9.2.6

Since Version 1.0.0

—

sckcr_divider_t

Clock divider values for SCKCR fields.

All SCKCR clock divider fields use the same encoding. Value N means divide by 2^N . For example, 0b0010 (2) means divide by 4.

Value	Name	Description
= 0	k_clock_div_1	Divide by 1 (no division)
= 1	k_clock_div_2	Divide by 2
= 2	k_clock_div_4	Divide by 4
= 3	k_clock_div_8	Divide by 8
= 4	k_clock_div_16	Divide by 16
= 5	k_clock_div_32	Divide by 32
= 6	k_clock_div_64	Divide by 64
= 8	k_clock_div_128	Divide by 128 (note: value is 8, not 7!)

Warning: Not all dividers valid for all clocks - check manual] **See also:** Manual Ch09 section 9.2.2

Since Version 1.0.0

—

sckcr_bits_t

System Clock Control Register (SCKCR) bit definitions.

32-bit register controlling all main system clock dividers. Each field is 4 bits wide and uses the encoding from sckcr_divider_t.

Value	Name	Description
= 0	k_sckcr_pckd_shift	

= 4	k_sckcr_pckc_shift	
= 8	k_sckcr_pckb_shift	
= 12	k_sckcr_pcka_shift	
= 16	k_sckcr_bck_shift	
= 24	k_sckcr_ick_shift	
= 28	k_sckcr_fck_shift	
= 0xFUL << k_sckcr_pckd_shift	k_sckcr_pckd_mask	
= 0xFUL << k_sckcr_pckc_shift	k_sckcr_pckc_mask	
= 0xFUL << k_sckcr_pckb_shift	k_sckcr_pckb_mask	
= 0xFUL << k_sckcr_pcka_shift	k_sckcr_pcka_mask	
= 0xFUL << k_sckcr_bck_shift	k_sckcr_bck_mask	
= 0xFUL << k_sckcr_ick_shift	k_sckcr_ick_mask	
= 0xFUL << k_sckcr_fck_shift	k_sckcr_fck_mask	
= (1UL << 22)	k_sckcr_pstop0	PCLKA/ PCLKB stop (1=stopped)
= (1UL << 23)	k_sckcr_pstop1	USB clock stop (1=stopped)
= (((uint32_t)k_clock_div_4 << k_sckcr_pckd_shift) ((uint32_t)k_clock_div_4 << k_sckcr_pckc_shift) ((uint32_t)k_clock_div_4 << k_sckcr_pckb_shift) ((uint32_t)k_clock_div_2 << k_sckcr_pcka_shift) ((uint32_t)k_clock_div_2 << k_sckcr_bck_shift) ((uint32_t)k_clock_div_1 << k_sckcr_ick_shift) ((uint32_t)k_clock_div_4 << k_sckcr_fck_shift) 0UL)	k_sckcr_star_240mhz	Complete SCKCR value for STAR 240 MHz configuration

Note: Requires PRCR unlock (PRC0) before modification

Warning: ICLK > 120 MHz requires MEMWAIT = 1 before changing SCKCR] **See also:** Manual Ch09 section 9.2.2

Since Version 1.0.0

—

sckcr3_cksel_t

System clock source selection (SCKCR3 CKSEL field).

Selects which oscillator/PLL output is used as the system clock source. This is the clock that feeds into SCKCR dividers.

Value	Name	Description
= 0x0000	k_sckcr3_cksel_loco	LOCO (Low-speed On-Chip Oscillator, 240 kHz)
= 0x0100	k_sckcr3_cksel_hoco	HOCO (High-speed On-Chip Oscillator, 16-20 MHz)
= 0x0200	k_sckcr3_cksel_main	Main clock oscillator (STAR: 24 MHz crystal)
= 0x0300	k_sckcr3_cksel_subclock	Sub-clock oscillator (32.768 kHz)
= 0x0400	k_sckcr3_cksel_pll	PLL circuit (STAR: 240 MHz)
= 0x0500	k_sckcr3_cksel_ppll	PPLL circuit (USB: 48 MHz)
= 0x0700	k_sckcr3_cksel_mask	CKSEL field mask (bits 10:8)

Note: Requires PRCR unlock (PRC0) before modification

Warning: Switching clock sources requires waiting for oscillator stabilization] **See also:** Manual Ch09 section 9.2.3

Since Version 1.0.0

—

pllcr_bits_t

PLL Control Register (PLLCR) bit definitions.

Controls PLL input divider, output multiplier, and source selection. PLL output frequency = (Input / PLIDIV) x (STC + 1)

Value	Name	Description
= 0x0000	k_pllcr_plidiv_1	Divide PLL input by 1
= 0x0001	k_pllcr_plidiv_2	Divide PLL input by 2
= 0x0002	k_pllcr_plidiv_4	Divide PLL input by 4
= 0x0003	k_pllcr_plidiv_mask	PLIDIV field mask
= 0x0000	k_pllcr_src_main	Main oscillator

		as PLL source (STAR config)
= 0x0010	k_pllcr_src_hoco	HOCO as PLL source
= 0x0010	k_pllcr_src_mask	PLLSRCSEL field mask
= 8	k_pllcr_stc_shift	
= 0x3F00	k_pllcr_stc_mask	
= (k_pllcr_plidiv_1 k_pllcr_stc_shift)	k_pllcr_src_main (9U \<\<)	STAR: 24 MHz x 10 = 240 MHz
= (k_pllcr_plidiv_1 k_pllcr_stc_shift)	k_pllcr_src_main (19U \<\<)	Alternative: 12 MHz x 20 = 240 MHz

Note: Requires PRCR unlock (PRC0) before modification

Warning: PLL must be stopped (PLLCR2.PLEN = 1) before changing PLLCR] **See also:** Manual Ch09 section 9.2.4

Since Version 1.0.0

—

pllcr2_bits_t

PLL Control Register 2 (PLLCR2) bit definitions.

Controls PLL enable/disable. Note inverted logic: 0 = enabled, 1 = stopped.

Value	Name	Description
-------	------	-------------

= 0x00	k_pllcr2_pll_running	PLL enabled (running)
= 0x01	k_pllcr2_pll_stopped	PLL disabled (stopped)

Note: Must wait for OSCOVFSR.PLOVF before switching clock to PLL

Note: Requires PRCR unlock (PRC0) before modification

Warning: Inverted logic! 0 = PLL running, 1 = PLL stopped] **See also:** Manual Ch09 section 9.2.4

Since Version 1.0.0

—

rx_module_stop_bits_a_t

Module stop bits for MSTPCRA register.

Value	Name	Description
= 14	k_mstpa_cmt23	CMT2/CMT3 module stop bit in MSTPCRA

—

rx_module_stop_bits_b_t

Module stop bits for MSTPCRB register.

Value	Name	Description
= 19	k_mstpb_usb0	USB0 module stop bit in MSTPCRB
= 23	k_mstpb_crc	CRC module stop bit in MSTPCRB

—

rx_rstsr_addresses_t

Reset status register addresses.

IMPORTANT: RSTSR registers are NOT contiguous in memory!

- RSTSR0/1 are adjacent at 0x0008C290-0x0008C291
- RSTSR2 is at a separate address 0x000800C0 Verified against RX72N Group User's Manual Hardware (R01UH0824EJ0120 Rev.1.20)

Value	Name	Description
= 0x0008C290	k_rstsr0_addr	Reset Status Register 0 address (page 286)
= 0x0008C291	k_rstsr1_addr	Reset Status Register 1 address (page 288)
= 0x000800C0	k_rstsr2_addr	Reset Status Register 2 address (page 289)

—

rstsr0_bits_t

Value	Name	Description
= 1U	k_rstsr0_porf	Power-On Reset Detect Flag
= 2U	k_rstsr0_lvd0rf	Voltage-Monitoring 0 Reset Detect Flag
= 4U	k_rstsr0_lvd1rf	Voltage-Monitoring 1 Reset Detect Flag
= 8U	k_rstsr0_lvd2rf	Voltage-Monitoring 2 Reset Detect Flag
= 128U	k_rstsr0_dpsrstf	Deep Software Standby Reset Flag

—

rstsr1_bits_t

Value	Name	Description
= 1U	k_rstsr1_cwsf	Cold/Warm Start Determination Flag

—

rstsr2_bits_t

Value	Name	Description
= 1U	k_rstsr2_iwdtrf	Independent Watchdog Timer Reset Detect Flag
= 2U	k_rstsr2_wdtrf	Watchdog Timer Reset Detect Flag
= 4U	k_rstsr2_swrf	Software Reset Detect Flag

—

rx_swrr_addresses_t

Software Reset Register (SWRR) address.

Writing 0xA501 to this register triggers an immediate software reset. Reference: RX72N manual Ch06 section 6.2.4, page 290.

Value	Name	Description
= 0x000800C2	k_swrr_addr	Software Reset Register address (16-bit)

—

rx_swrr_values_t

Software Reset Register value.

Write this value to SWRR to trigger a software reset. The MCU will reset after the internal reset time (tRESW2) elapses.

Value	Name	Description
= 0xA501	k_swrr_reset_key	Write this value to trigger software reset

Warning: This immediately resets the MCU - ensure all critical state is saved.

—

rx_memwait_addresses_t

MEMWAIT register address.

Memory Wait Cycle Setting Register controls wait cycles for flash and ECCRAM. MANDATORY: Must be set to 1 when ICLK > 120 MHz. Reference: RX72N manual Ch09 section 9.2.2, page 341, line 962.

Value	Name	Description
= 0x0008101C	k_memwait_addr	Memory Wait Cycle Setting Register

—

rx_memwait_bits_t

MEMWAIT register bit values.

- 0: No wait cycle (ICLK ≤ 120 MHz)
- 1: One wait cycle (ICLK > 120 MHz) - REQUIRED for 240 MHz operation

Value	Name	Description
= 0x00	k_memwait_no_wait	No wait cycle (ICLK ≤ 120 MHz)
= 0x01	k_memwait_one_wait	One wait cycle (ICLK > 120 MHz) - MANDATORY

—

rx_ppll_addresses_t

PPLL register addresses.

PPLL (PLL for specific purposes) generates the 48 MHz USB clock (UCLK). Must be configured for USB functionality. Reference: RX72N manual Ch09, PPLL frequency synthesizer section.

Value	Name	Description
= 0x00080048	k_ppllcr_addr	PPLL Control Register @ 0x00080048
= 0x0008004A	k_ppllcr2_addr	PPLL Control Register 2 @ 0x0008004A

—

rx_ppll_config_t

PPLL configuration values.

PPLL generates 48 MHz USB clock from 24 MHz main oscillator. Calculation: 48 MHz = (24 MHz x 8) / 4

- PPLSTBY[1:0] = 01 (divide by 2)
- STC[5:0] = 07h (multiply by 8)
- PPLSRCSEL = 0 (main clock source)

Value	Name	Description
= 0x0703	k_ppll_config_48mhz	PPLLCR: 48MHz from 24MHz main osc

—

rx_ppll_control_t

PPLL control values.

Value	Name	Description
= 0x00	k_pll_enabled	PPLLCR2: Enable PLL
= 0x01	k_pll_disabled	PPLLCR2: Disable PLL

—

rx_pll_flags_t

PPLL stabilization flag in OSCOVFSR.

Value	Name	Description
= 0x08	k_pll_stable_flag	OSCOVFSR: PPLL stabilization flag bit 3

—

system_regs

```
static volatile rx_system_regs_t * system_regs(void)
```

Get pointer to system registers.

Returns: Volatile pointer to system register structure

Note: Named system_regs() instead of system() to avoid naming conflicts

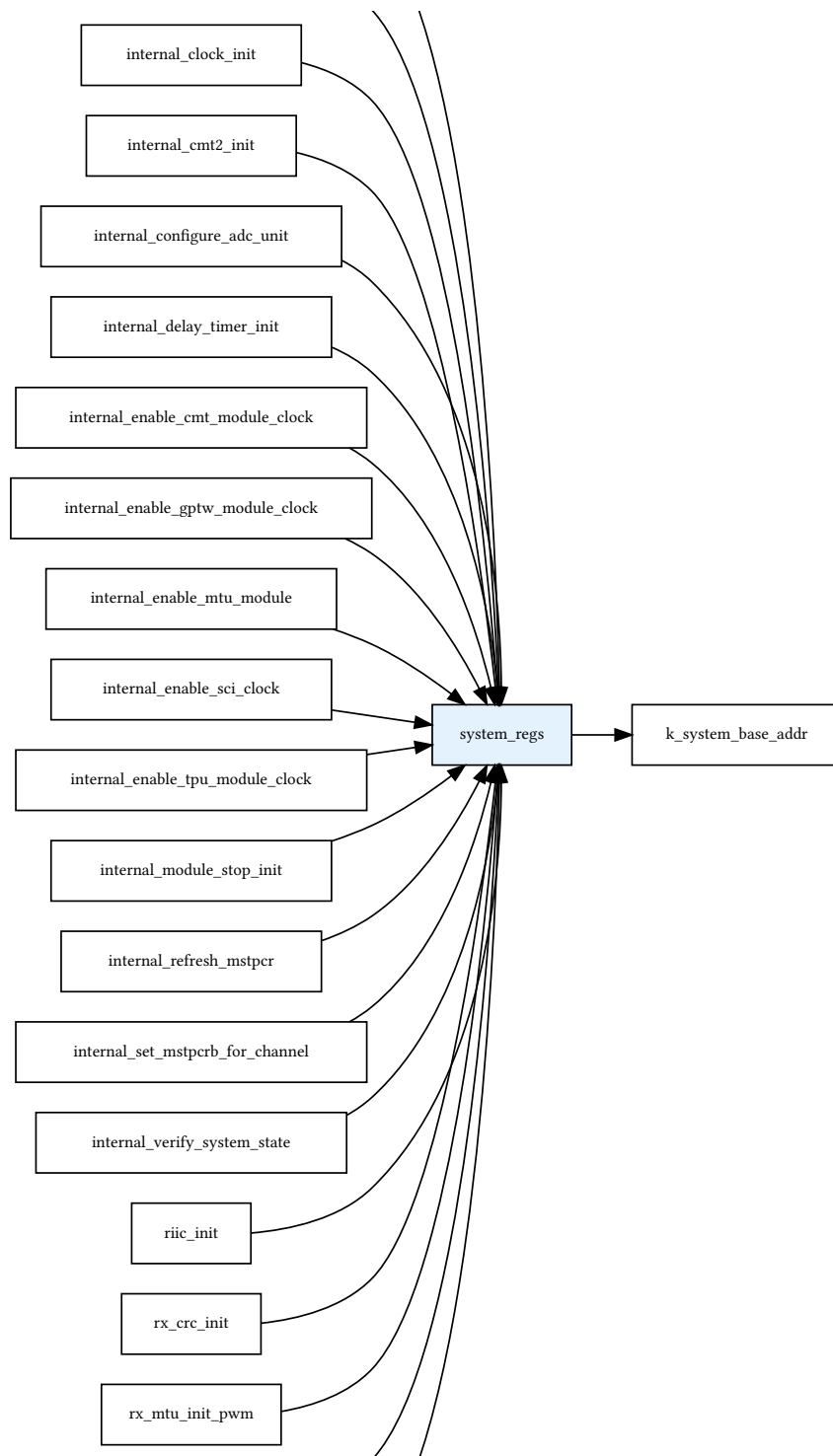


Figure 643: Call graph for system_regs

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:869_

—

prcr_reg

```
static volatile uint16_t * prcr_reg(void)
```

Get pointer to PRCR (Protection Register).

PRCR is at address 0x000803FE, which is NOT contiguous with the main system register block (0x00080000-0x00080041). This accessor provides the correct address for register protection control.

Returns: Volatile pointer to 16-bit PRCR register

Note: Use k_rx_pcr_unlock_* and k_rx_pcr_lock values from rx_register_protection.h for unlock/lock operations.

Manual Reference:: RX72N Group User's Manual: Hardware, Chapter 13 (Register Write Protection) Address: 0x000803FE (verified against manual) Size: 16-bit Key: 0xA5 in upper byte required for any write

Example:: #include "rx_register_protection.h" *prcr_reg()=k_rx_pcr_unlock_prc1; system_regs()->mstpcra&= (1UL<<15); *prcr_reg()=k_rx_pcr_lock;

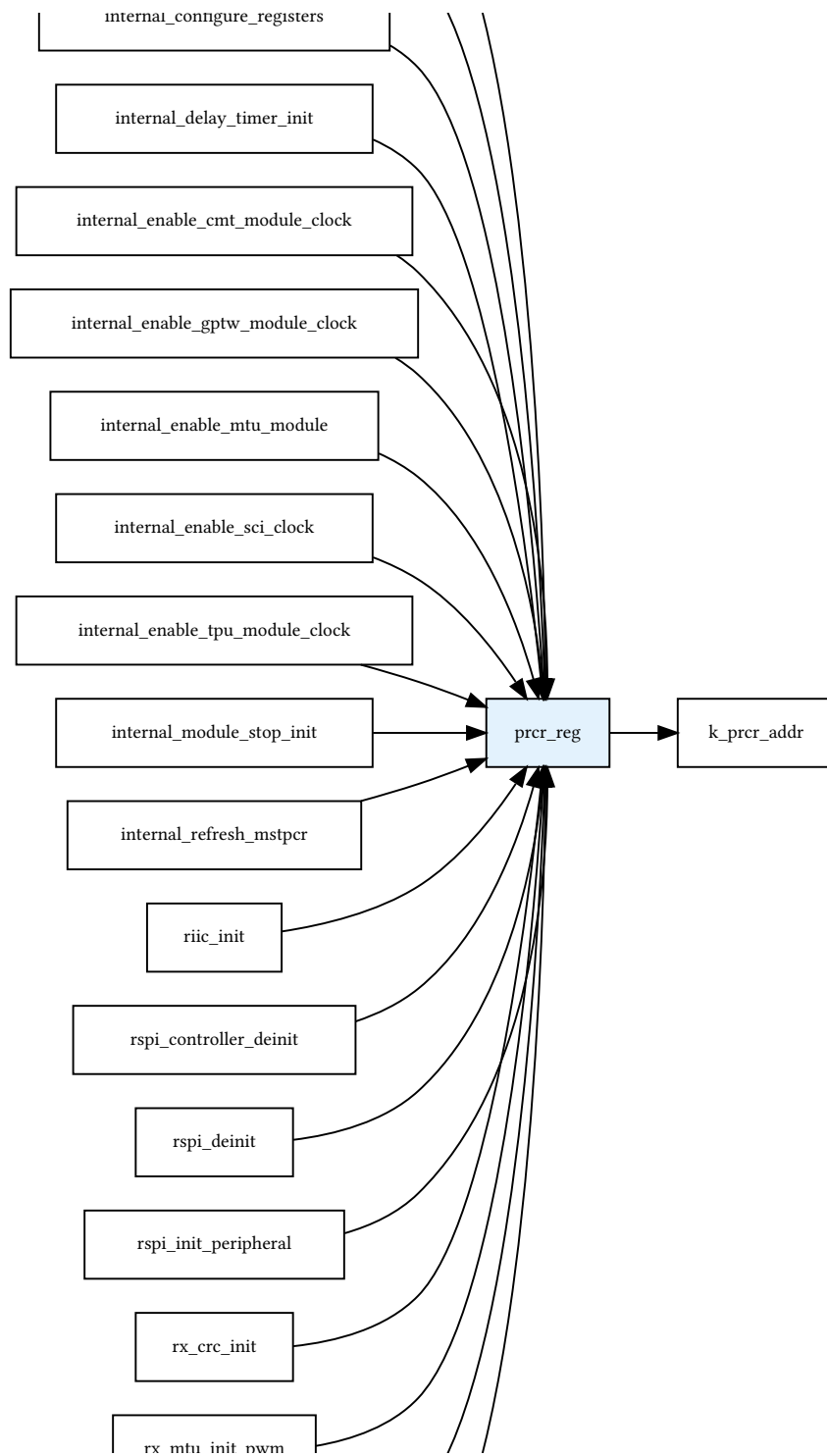


Figure 644: Call graph for prcr_reg

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:906_
—

rstsr01

```
static volatile rx_rstsr01_regs_t * rstsr01(void)
```

Get pointer to RSTSR0/RSTSR1 registers.

Returns: Volatile pointer to RSTSR0/1 register structure

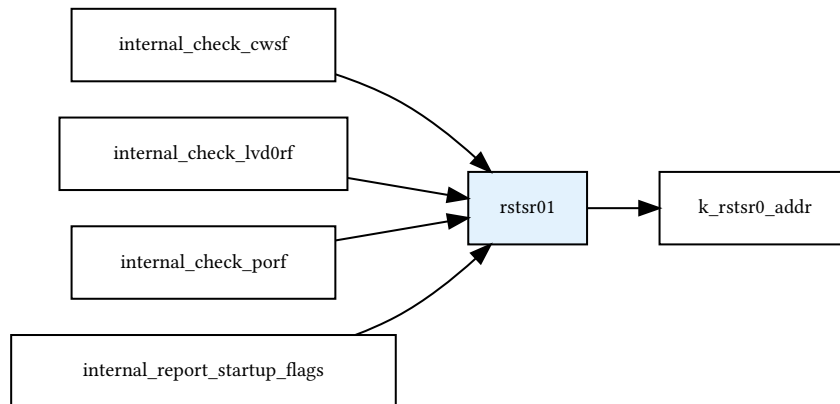


Figure 645: Call graph for `rstsr01`

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:955_
—

rstsr2

```
static volatile uint8_t * rstsr2(void)
```

Get pointer to RSTSR2 register.

Returns: Volatile pointer to RSTSR2 register

Note: RSTSR2 is at a separate address from RSTSR0/1

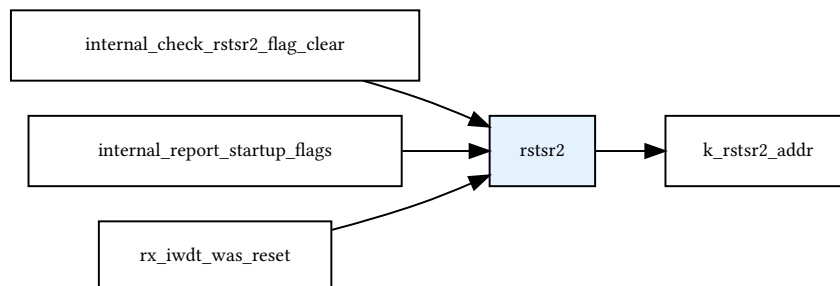


Figure 646: Call graph for `rstsr2`

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:965_
—

swrr_reg

```
static volatile uint16_t * swrr_reg(void)
```

Get pointer to SWRR (Software Reset Register).

SWRR is at address 0x000800C2. Writing 0xA501 triggers a software reset.

Returns: Volatile pointer to 16-bit SWRR register

Warning: Writing to this register causes immediate MCU reset!

Manual Reference:: RX72N Group User's Manual: Hardware, Chapter 6 (Resets), page 290.

Example:: *swrr_reg()=k_swrr_reset_key;

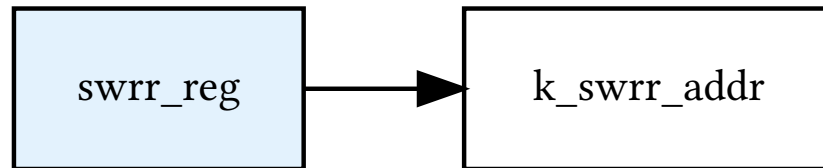


Figure 647: Call graph for `swrr_reg`

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:1036_
—

memwait_reg

```
static volatile uint8_t * memwait_reg(void)
```

Get pointer to MEMWAIT register.

Returns: Volatile pointer to MEMWAIT register

Note: CRITICAL: Must set to 1 before increasing ICLK above 120 MHz

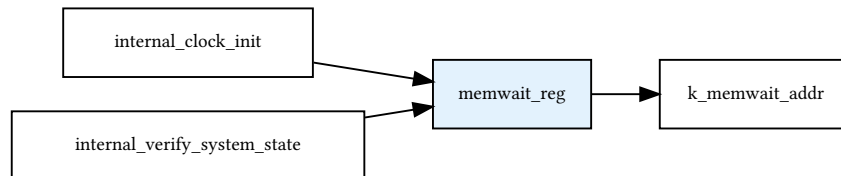


Figure 648: Call graph for `memwait_reg`

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:1073_
—

ppllcr_reg

```
static volatile uint16_t * ppllcr_reg(void)
```

Get pointer to PPLLCR register.

Returns: Volatile pointer to PPLLCR register

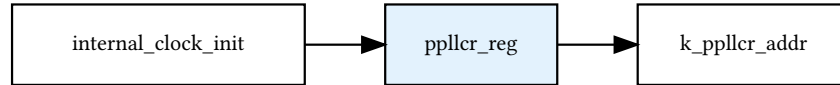


Figure 649: Call graph for ppllcr_reg

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:1127_

—

ppllcr2_reg

```
static volatile uint8_t * ppllcr2_reg(void)
```

Get pointer to PPLLCR2 register.

Returns: Volatile pointer to PPLLCR2 register

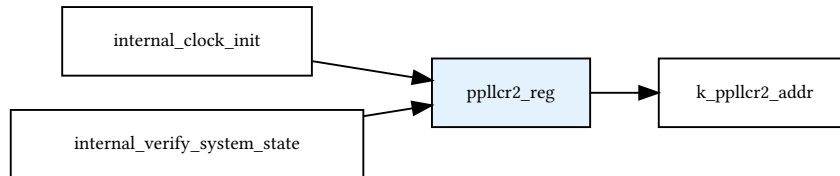


Figure 650: Call graph for ppllcr2_reg

_Defined in libs/rx_hal/inc/rx72n_system_regs.h:1136_

—

rx72n_temps_regs.h

RX72N Temperature Sensor (TEMPS) Register Definitions.

Complete register definitions for the RX72N internal temperature sensor. The temperature sensor outputs a voltage proportional to die temperature, which can be converted to temperature via the 12-bit ADC unit 1.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdint.h>

temps_addresses_t

Temperature Sensor register addresses.

Addresses verified against RX72N Hardware Manual Ch58.

Value	Name	Description
= 0x0008C500U	k_temps_tscr_addr	Temperature Sensor Control Register (TSCR) address.
= 0xFE7F7D7CU	k_temps_tsedr_addr	Temperature Sensor Calibration Data Register (TSEDR) address.

Note: Addresses verified 2026-01-29 against manual section 58.2

Since Version 1.0.0

—

temps_bits_t

Temperature Sensor Control Register (TSCR) bit definitions.

Controls the temperature sensor enable state and output to ADC.

Value	Name	Description
= 0x80U	k_tscr_tsen_bit	TSEN bit position (bit 7).
= 0x10U	k_tscr_tsoe_bit	TSOE bit position (bit 4).
= 0x00U	k_tscr_disabled	Temperature sensor disabled, output disabled (default).
= 0x80U	k_tscr_sensor_only	Temperature sensor enabled, output disabled.
= 0x90U	k_tscr_sensor_with_output	Temperature sensor enabled with output to ADC.
= 0x6FU	k_tscr_reserved_mask	Reserved bits mask (bits 6:5, 3:0 must be 0).

Note: Manual ref: Ch58 section 58.2.1 (page 2964)

Since Version 1.0.0

—

temps_timing_t

Temperature sensor timing constants.

Timing requirements from Ch58 Table 58.2 for proper sensor operation.

Value	Name	Description
= 30U	k_temps_tstbl_us	Reference voltage stabilization wait time (tTSTBL).
= 0U	k_temps_tostbl_us	Output stabilization wait time (tOSTBL).

Note: Manual ref: Ch58 section 58.3.4, Table 58.2

Since Version 1.0.0

—

temps_calibration_t

Temperature sensor calibration constants.

Constants for temperature calculation using factory calibration data.

Value	Name	Description
= 125U	k_temps_cal_temp_celsius	Calibration temperature (125degC).
= 4096U	k_temps_adc_full_scale	ADC full scale value (12-bit = 4096).
= 3300U	k_temps_avcc_mv	ADC reference voltage in millivolts (3300 mV = 3.3V).
= 4100U	k_temps_slope_uv_per_c	Typical temperature slope in uV/degC (absolute value).

Note: Manual ref: Ch58 section 58.3.1

Since Version 1.0.0

—

temps_mstpcr_t

Temperature sensor module stop control bits.

The temperature sensor is controlled by MSTPCRB.MSTPC22. Must be cleared (0) before accessing TEMPS registers.

Value	Name	Description
= (1U \<\< 22U)	k_temps_mstpcrb_bit	MSTPCRB bit for temperature sensor (bit 22).

Note: Manual ref: Ch11 (Low Power Consumption) module stop tables

Since Version 1.0.0

—

temps_tscr_reg

```
static volatile uint8_t * temps_tscr_reg(void)
```

Get pointer to Temperature Sensor Control Register (TSCR).

Returns a volatile pointer to TSCR at address 0x0008C500.

Returns: Volatile pointer to TSCR register (8-bit)

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: Temperature sensor module stop (MSTPCRB.MSTPC22) must be cleared

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Example (Enable Temperature Sensor):: `*temps_tscr_reg()=k_tscr_sensor_only;`
`delay_us(k_temps_tstbl_us);`
`*temps_tscr_reg()=k_tscr_sensor_with_output;`

See also: `k_temps_tscr_addr` Address constant, `tscr_bits_t` Control bits

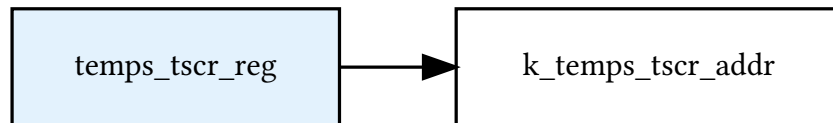


Figure 651: Call graph for `temps_tscr_reg`

_Defined in `libs/rx\_hal/inc/rx72n\_temps\_regs.h:310_`

Since Version 1.0.0

—

temps_tscdr_reg

```
static volatile const uint32_t * temps_tscdr_reg(void)
```

Get pointer to Temperature Sensor Calibration Data Register (TSCDR).

Returns a volatile pointer to TSCDR at address 0xFE7F7D7C. This is a 32-bit read-only register containing factory calibration data.

Returns: Volatile pointer to TSCDR register (32-bit, read-only)

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: On-chip ROM must be enabled (`SYSCR0.ROME = 1`)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Warning: This register is read-only; writes have no effect] **Calibration Data Format:** The lower 12 bits contain the ADC value measured at 125degC and 3.3V: Bits [11:0]: ADC value (0-4095) Bits [31:12]: Reserved (read as 0)

Converting to Voltage: $V1 = 3.3 \times (TSCDR \& 0xFFF) / 4096$ [V]

Example (Read Calibration):: `uint32_t cal125=*temps_tscdr_reg()&0xFFF;`

See also: `k_temps_tscdr_addr` Address constant

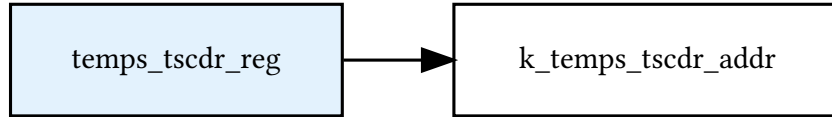


Figure 652: Call graph for `temps_tscdr_reg`

_Defined in `libs/rx_hal/inc/rx72n_temps_regs.h:352_`

Since Version 1.0.0

—

rx72n_tpu_regs.h

RX72N 16-Bit Timer Pulse Unit (TPU) Register Definitions.

Register definitions for the TPU module which provides 6 channels of 16-bit timers with quadrature encoder phase counting capability. In STAR, the TPU is used for rear wheel encoder decoding via hardware phase counting mode.

| (same for B-phase edges with A-phase level swapped) |

STAR Team

2026-02-10

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stddef.h>`
- `<stdint.h>`

rx_tpu_addresses_t

TPU base addresses.

Base addresses for the TPU control block and individual channel register blocks. The control block contains TSTR, TSYP, and NPCR registers shared across all channels. Each channel has a 16-byte register block.

Value	Name	Description
<code>= 0x00088100</code>	<code>k_tpu_control_base_addr</code>	TPU control block base address (0x00088100).
<code>= 0x00088110</code>	<code>k_tpu0_base_addr</code>	TPU0 channel base address (0x00088110) - Extended channel.
<code>= 0x00088120</code>	<code>k_tpu1_base_addr</code>	TPU1 channel base address (0x00088120) - Basic channel.
<code>= 0x00088130</code>	<code>k_tpu2_base_addr</code>	TPU2 channel base address (0x00088130) - Basic channel.
<code>= 0x00088140</code>	<code>k_tpu3_base_addr</code>	TPU3 channel base address (0x00088140) - Extended channel.
<code>= 0x00088150</code>	<code>k_tpu4_base_addr</code>	TPU4 channel base address (0x00088150) - Basic channel.

= 0x00088160	k_tpu5_base_addr	TPU5 channel base address (0x00088160) - Basic channel.
= 0x10	k_tpu_channel_spacing	Channel register block spacing (0x10 bytes between channels).

Note: Manual: Ch28.2 - Register Descriptions] **See also:** `tpu_control()` Accessor for control block, `tpu1()` Accessor for TPU1 channel (rear-left encoder)

Since Version 1.0.0

—

rx_tpu_channel_count_t

TPU channel count constant.

Value	Name	Description
= 6	k_tpu_channel_count	Total TPU channels (TPU0-TPU5)

Since Version 1.0.0

—

rx_tpu_tstr_bits_t

TSTR (Timer Start Register) bit definitions.

Each CSTn bit independently starts (1) or stops (0) the corresponding TPU_n channel counter. Multiple channels can be started simultaneously by OR-ing the appropriate bits.

Value	Name	Description
= (1U << 0)	k_tpu_tstr_cst0	TPU0 counter start (1=run, 0=stop)
= (1U << 1)	k_tpu_tstr_cst1	TPU1 counter start (rear-left encoder)
= (1U << 2)	k_tpu_tstr_cst2	TPU2 counter start (rear-right encoder)
= (1U << 3)	k_tpu_tstr_cst3	TPU3 counter start
= (1U << 4)	k_tpu_tstr_cst4	TPU4 counter start
= (1U << 5)	k_tpu_tstr_cst5	TPU5 counter start

Note: Manual: Ch28.2.8 - Timer Start Register (TSTR)] **See also:** `rx_tpu_control_regs_t::tstr` Register field

Since Version 1.0.0

—

rx_tpu_tsyrr_bits_t

TSYR (Timer Synchronous Register) bit definitions.

Each SYNC_n bit enables (1) or disables (0) synchronous operation for the corresponding TPU_n channel. When multiple channels are set to synchronous, their TCNTs can be written simultaneously.

Value	Name	Description
= (1U \<\< 0)	k_tpu_tsyrc_sync0	TPU0 synchronous operation enable
= (1U \<\< 1)	k_tpu_tsyrc_sync1	TPU1 synchronous operation enable
= (1U \<\< 2)	k_tpu_tsyrc_sync2	TPU2 synchronous operation enable
= (1U \<\< 3)	k_tpu_tsyrc_sync3	TPU3 synchronous operation enable
= (1U \<\< 4)	k_tpu_tsyrc_sync4	TPU4 synchronous operation enable
= (1U \<\< 5)	k_tpu_tsyrc_sync5	TPU5 synchronous operation enable

Note: Manual: Ch28.2.9 - Timer Synchronous Register (TSYR)] **See also:**
rx_tpu_control_regs_t::tsyr Register field

Since Version 1.0.0

—

rx_tpu_tcr_shift_t

TCR (Timer Control Register) bit field positions.

Bit positions for the Timer Control Register fields: prescaler select, clock edge select, and counter clear source select.

Value	Name	Description
= 0	k_tpu_tcr_tpssc_shift	TPSC[2:0] - Timer Prescaler Select (bits 2:0)
= 3	k_tpu_tcr_ckege_shift	CKEG[1:0] - Input Clock Edge Select (bits 4:3)
= 5	k_tpu_tcr_cclr_shift	CCLR[2:0] - Counter Clear Source (bits 7:5)

Note: In phase counting mode, TPSC and CKEG settings are ignored] **See also:**
rx_tpu_tcr_cclr_t Counter clear source values

Since Version 1.0.0

—

rx_tpu_tcr_cclr_t

TCR CCLR[2:0] counter clear source values (for basic channels).

Selects the event that clears TCNT. In phase counting mode, only TGRA and TGRB compare match clearing is available.

Value	Name	Description
= (0U \<\< 5)	k_tpu_tcr_cclr_disabled	TCNT free-running (no clear)
= (1U \<\< 5)	k_tpu_tcr_cclr_tgra_match	Clear on TGRA compare match
= (2U \<\< 5)	k_tpu_tcr_cclr_tgrb_match	Clear on TGRB compare match
= (3U \<\< 5)	k_tpu_tcr_cclr_sync_clear	Synchronous clear

Note: Bit 7 is reserved for TPU1/2/4/5 (basic channels)] **See also:**
 rx_tpu_tcr_shift_t::k_tpu_tcr_cclr_shift Field position

Since Version 1.0.0

—

rx_tpu_tcr_masks_t

TCR register field masks.

Value	Name	Description
= 0x07	k_tpu_tcr_tpsec_mask	TPSEC[2:0] field mask (bits 2:0)
= 0x18	k_tpu_tcr_ckeeg_mask	CKEeg[1:0] field mask (bits 4:3)
= 0xE0	k_tpu_tcr_cclr_mask	CCLR[2:0] field mask (bits 7:5)

Since Version 1.0.0

—

rx_tpu_tmdr_md_t

TMDR MD[3:0] mode select values.

Selects the operating mode for the TPU channel. Phase counting modes (1-4) are only available on TPU1, TPU2, TPU4, and TPU5.

Value	Name	Description
= 0x00	k_tpu_tmdr_md_normal	Normal operation
= 0x02	k_tpu_tmdr_md_pwm1	PWM mode 1
= 0x03	k_tpu_tmdr_md_pwm2	PWM mode 2
= 0x04	k_tpu_tmdr_md_phase_count_1	Phase counting mode 1 (1x)
= 0x05	k_tpu_tmdr_md_phase_count_2	Phase counting mode 2 (1x)
= 0x06	k_tpu_tmdr_md_phase_count_3	Phase counting mode 3 (2x)
= 0x07	k_tpu_tmdr_md_phase_count_4	Phase counting mode 4 (4x) - STAR default

Note: Phase counting mode cannot be set for TPU0 and TPU3

Note: Manual: Ch28.2.2 - Timer Mode Register (TMDR)

Since Version 1.0.0

—

rx_tpu_tmdr_shift_t

TMDR bit field positions.

Value	Name	Description
= 0	k_tpu_tmdr_md_shift	MD[3:0] - Mode Select (bits 3:0)

= 4	k_tpu_tmdr_bfa_shift	BFA - Buffer Operation A (bit 4, TPU0/3 only)
= 5	k_tpu_tmdr_bfb_shift	BFB - Buffer Operation B (bit 5, TPU0/3 only)
= 6	k_tpu_tmdr_icseib_shift	ICSELB - TGRB input capture select (bit 6)
= 7	k_tpu_tmdr_icseld_shift	ICSELD - TGRD input capture select (bit 7, TPU0/3)

Since Version 1.0.0

—

rx_tpu_tmdr_masks_t

TMDR register field masks.

Value	Name	Description
= 0x0F	k_tpu_tmdr_md_mask	MD[3:0] field mask (bits 3:0)

Since Version 1.0.0

—

rx_tpu_tier_bits_t

TIER (Timer Interrupt Enable Register) bit definitions.

Controls interrupt generation for timer compare match, overflow, and underflow events. In phase counting mode, TCIEV (overflow) and TCIEU (underflow) are the most relevant interrupts.

Value	Name	Description
= (1U << 0)	k_tpu_tier_tgiea	TGRA interrupt enable
= (1U << 1)	k_tpu_tier_tgieb	TGRB interrupt enable
= (1U << 2)	k_tpu_tier_tgiec	TGRC interrupt enable (TPU0/3 only)
= (1U << 3)	k_tpu_tier_tgied	TGRD interrupt enable (TPU0/3 only)
= (1U << 4)	k_tpu_tier_tciev	Overflow interrupt enable (all channels)
= (1U << 5)	k_tpu_tier_tcieu	Underflow interrupt enable (TPU1/2/4/5)
= (1U << 7)	k_tpu_tier_ttge	A/D conversion start request enable

Note: TGIEC/TGIED reserved on TPU1/2/4/5 (read as 0, write 0)

Note: TCIEU reserved on TPU0/TPU3 (read as 0, write 0)

Note: TTGE reserved on TPU5 (read as 0, write 0)

Note: Manual: Ch28.2.4 - Timer Interrupt Enable Register (TIER)

Since Version 1.0.0

—

rx_tpu_tsr_bits_t

TSR (Timer Status Register) bit definitions.

Status flags for timer events and counting direction. The TCFD flag is critical for phase counting mode - it indicates whether TCNT is currently counting up (forward) or down (reverse).

Value	Name	Description
= (1U << 0)	k_tpu_tsr_tgfa	TGRA compare match/input capture occurred
= (1U << 1)	k_tpu_tsr_tgfb	TGRB compare match/input capture occurred
= (1U << 2)	k_tpu_tsr_tgfc	TGRC compare match/input capture (TPU0/3)
= (1U << 3)	k_tpu_tsr_tgfd	TGRD compare match/input capture (TPU0/3)
= (1U << 4)	k_tpu_tsr_tcfv	Overflow flag (TCNT wrapped 0xFFFF->0x0000)
= (1U << 5)	k_tpu_tsr_tcfu	Underflow flag (TCNT wrapped 0x0000->0xFFFF)
= (1U << 7)	k_tpu_tsr_tcfv	Counting direction: 1=up, 0=down (read-only)

Note: Flags (bits 0-5) are cleared by writing 0 to them

Note: TCFD is read-only: 1=counting up, 0=counting down

Note: Bit 6 reset value is 1 (must write 1 to preserve)

Note: TGFC/TGFD reserved on TPU1/2/4/5 (read as 0, write 0)

Note: TCFU/TCFD reserved on TPU0/TPU3 (read as 0/1 respectively)

Note: Manual: Ch28.2.5 - Timer Status Register (TSR)

Since Version 1.0.0

—

rx_tpu_tsr_reserved_t

TSR reserved bit value (bit 6 must be written as 1).

Value	Name	Description
= (1U << 6)	k_tpu_tsr_reserved_bit6	Bit 6: read as 1, write 1

Since Version 1.0.0

—

rx_tpu_nfcr_bits_t

NFCR (Noise Filter Control Register) bit definitions.

Controls digital noise filtering on TIOC I/O pins. Each channel has independent filter enable bits for each pin and a shared clock select.

Value	Name	Description
= (1U \<\< 0)	k_tpu_nfcr_nfaen	Noise filter enable for TIOCA pin
= (1U \<\< 1)	k_tpu_nfcr_nfben	Noise filter enable for TIOCB pin
= (1U \<\< 2)	k_tpu_nfcr_nfcen	Noise filter enable for TIOCC (TPU0/3)
= (1U \<\< 3)	k_tpu_nfcr_nfden	Noise filter enable for TIOCD (TPU0/3)

Note: NFCEN/NFDEN are reserved on TPU1/2/4/5 (only TPU0/3 have C/D pins)

Note: Set NFCR only while TCNT is stopped (CSTn = 0)

Note: Manual: Ch28.2.10 - Noise Filter Control Register (NFCR)

Since Version 1.0.0

—

rx_tpu_nfcr_clock_t

NFCR NFCS[1:0] noise filter clock select values.

Selects the sampling clock for the digital noise filter. The filter requires 3 consecutive samples at the selected level to pass.

Value	Name	Description
= (0U \<\< 4)	k_tpu_nfcr_nfcs_pclk_div1	Sample @ PCLKB/1 (50ns)
= (1U \<\< 4)	k_tpu_nfcr_nfcs_pclk_div8	Sample @ PCLKB/8 (400ns)
= (2U \<\< 4)	k_tpu_nfcr_nfcs_pclk_div32	Sample @ PCLKB/32 (1.6us)
= (3U \<\< 4)	k_tpu_nfcr_nfcs_count_clock	Sample @ count clock source

Since Version 1.0.0

—

rx_tpu_nfcr_masks_t

NFCR register field masks.

Value	Name	Description
= 0x30	k_tpu_nfcr_nfcs_mask	NFCS[1:0] field mask (bits 5:4)
= 0x0F	k_tpu_nfcr_enable_mask	All noise filter enable bits (bits 3:0)

Since Version 1.0.0

—

rx_tpu_mstpcra_t

MSTPCRA bit for TPU module stop control.

The TPU module is controlled by MSTPA13 in the Module Stop Control Register A (MSTPCRA). Write 0 to enable, 1 to stop.

Value	Name	Description
= (1U \<\< 13)	k_tpu_mstpcra_mstpa13	MSTPA13: TPU module stop (1=stop)

Since Version 1.0.0

—

rx_tpu_intb_source_t

TPU Software Configurable Interrupt B source numbers.

TPU interrupts are routed through the ICU Software Configurable Interrupt B mechanism. These source numbers are written to SLIBRn registers (n=128-207) to map them to interrupt vector numbers 128-207.

For phase counting mode, the most relevant sources are:

- TCIXV (overflow): TCNT wrapped 0xFFFF -> 0x0000 while counting up
- TCIXU (underflow): TCNT wrapped 0x0000 -> 0xFFFF while counting down

Value	Name	Description
= 15	k_tpu_intb_tgi0a	TPU0 TGRA compare match/input capture
= 16	k_tpu_intb_tgi0b	TPU0 TGRB compare match/input capture
= 17	k_tpu_intb_tgi0c	TPU0 TGRC compare match/input capture
= 18	k_tpu_intb_tgi0d	TPU0 TGRD compare match/input capture
= 19	k_tpu_intb_tci0v	TPU0 overflow
= 20	k_tpu_intb_tgi1a	TPU1 TGRA compare match/input capture
= 21	k_tpu_intb_tgi1b	TPU1 TGRB compare match/input capture
= 22	k_tpu_intb_tci1v	TPU1 overflow (phase counting up wrap)
= 23	k_tpu_intb_tci1u	TPU1 underflow (phase counting down wrap)
= 24	k_tpu_intb_tgi2a	TPU2 TGRA compare match/input capture
= 25	k_tpu_intb_tgi2b	TPU2 TGRB compare match/input capture
= 26	k_tpu_intb_tci2v	TPU2 overflow (phase counting up wrap)
= 27	k_tpu_intb_tci2u	TPU2 underflow (phase counting down wrap)
= 28	k_tpu_intb_tgi3a	TPU3 TGRA compare match/input capture
= 29	k_tpu_intb_tgi3b	TPU3 TGRB compare match/input capture
= 30	k_tpu_intb_tgi3c	TPU3 TGRC compare match/input capture
= 31	k_tpu_intb_tgi3d	TPU3 TGRD compare match/input capture
= 32	k_tpu_intb_tci3v	TPU3 overflow
= 33	k_tpu_intb_tgi4a	TPU4 TGRA compare match/input capture
= 34	k_tpu_intb_tgi4b	TPU4 TGRB compare match/input capture
= 35	k_tpu_intb_tci4v	TPU4 overflow
= 36	k_tpu_intb_tci4u	TPU4 underflow (phase counting)
= 37	k_tpu_intb_tgi5a	TPU5 TGRA compare match/input capture

= 38	k_tpu_intb_tgi5b	TPU5 TGRB compare match/input capture
= 39	k_tpu_intb_tci5v	TPU5 overflow
= 40	k_tpu_intb_tci5u	TPU5 underflow (phase counting)

Note: Manual: Ch15 Table 15.3 - Interrupt Sources for Software Configurable Interrupt B, and Ch28.4 - Interrupt Sources

Warning: These are NOT vector numbers - they are source numbers for SLIBR

Since Version 1.0.0

—

tpu_control

```
static volatile rx_tpu_control_regs_t * tpu_control(void)
```

Get pointer to TPU control block registers.

Returns a volatile pointer to the TPU control register block at address 0x00088100. Contains TSTR (timer start), TSYSR (synchronous), and NFCR (noise filter) registers shared across all channels.

Returns: Volatile pointer to TPU control register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Example:: tpu_control()->tstr|=(k_tpu_tstr_cst1|k_tpu_tstr_cst2);

See also: k_tpu_control_base_addr Base address constant

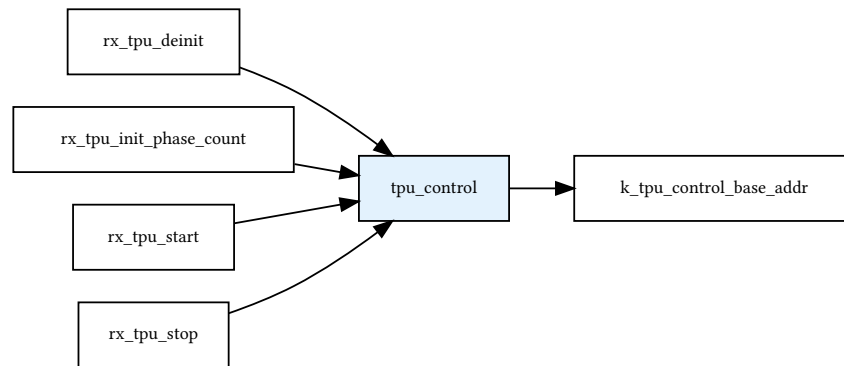


Figure 653: Call graph for tpu_control

_Defined in `libs/rx\hal\inc/rx72n\tpu\_regs.h:502_`

Since Version 1.0.0

—

tpu0

```
static volatile rx_tpu_ext_regs_t * tpu0(void)
```

Get pointer to TPU0 channel registers (extended).

Returns a volatile pointer to TPU0 extended channel registers at address 0x00088110. TPU0 has TIORH/TIORL and TGRA through TGRD.

Returns: Volatile pointer to TPU0 extended register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: TPU0 does NOT support phase counting mode] **See also:** `k_tpu0_base_addr` Base address constant

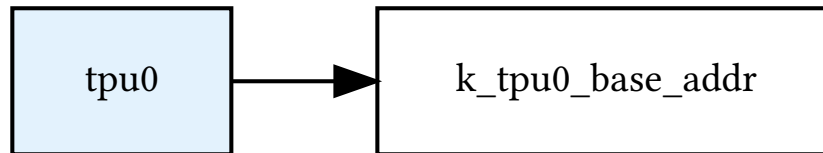


Figure 654: Call graph for tpu0

_Defined in `libs/rx\_hal/inc/rx72n\_tpu\_regs.h:526_`

Since Version 1.0.0

—

tpu1

```
static volatile rx_tpu_regs_t * tpu1(void)
```

Get pointer to TPU1 channel registers (basic, phase counting).

Returns a volatile pointer to TPU1 basic channel registers at address 0x00088120. TPU1 supports phase counting mode using TCLKA (A-phase) and TCLKB (B-phase) external clock pins. Used for rear-left encoder.

Returns: Volatile pointer to TPU1 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Example:: `uint16_tcount=tpu1()->tcnt; boolforward=(tpu1()->tsr&k_tpu_tsr_tcf)!0;`

See also: `k_tpu1_base_addr` Base address constant

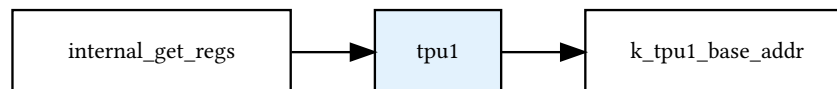


Figure 655: Call graph for tpu1

_Defined in `libs/rx\_hal/inc/rx72n\_tpu\_regs.h:557_`

Since Version 1.0.0

—

tpu2

```
static volatile rx_tpu_regs_t * tpu2(void)
```

Get pointer to TPU2 channel registers (basic, phase counting).

Returns a volatile pointer to TPU2 basic channel registers at address 0x00088130. TPU2 supports phase counting mode using TCLKC (A-phase) and TCLKD (B-phase) external clock pins. Used for rear-right encoder.

Returns: Volatile pointer to TPU2 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address] **See also:** k_tpu2_base_addr
Base address constant

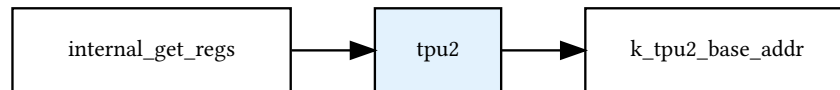


Figure 656: Call graph for tpu2

_Defined in libs\rx_hal\inc\rx72n_tpu_regs.h:581_

Since Version 1.0.0

—

tpu3

```
static volatile rx_tpu_ext_regs_t * tpu3(void)
```

Get pointer to TPU3 channel registers (extended).

Returns a volatile pointer to TPU3 extended channel registers at address 0x00088140. TPU3 has TIORH/TIORL and TGRA through TGRD.

Returns: Volatile pointer to TPU3 extended register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: TPU3 does NOT support phase counting mode] **See also:** k_tpu3_base_addr Base address constant

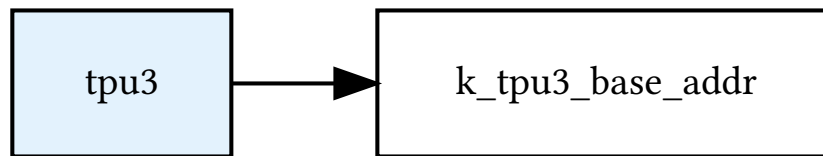


Figure 657: Call graph for tpu3

_Defined in libs\rx_hal\inc\rx72n_tpu_regs.h:605_

Since Version 1.0.0

—

tpu4

```
static volatile rx_tpu_regs_t * tpu4(void)
```

Get pointer to TPU4 channel registers (basic, phase counting).

Returns a volatile pointer to TPU4 basic channel registers at address 0x00088150. TPU4 supports phase counting mode using TCLKC/TCLKD pins (paired with TPU2).

Returns: Volatile pointer to TPU4 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address] **See also:** k_tpu4_base_addr Base address constant

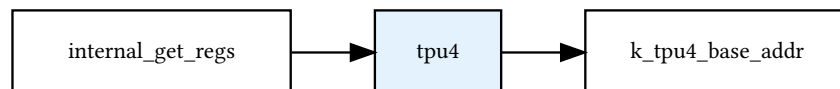


Figure 658: Call graph for tpu4

_Defined in libs\rx_hal\inc\rx72n_tpu_regs.h:629_

Since Version 1.0.0

—

tpu5

```
static volatile rx_tpu_regs_t * tpu5(void)
```

Get pointer to TPU5 channel registers (basic, phase counting).

Returns a volatile pointer to TPU5 basic channel registers at address 0x00088160. TPU5 supports phase counting mode using TCLKA/TCLKB pins (paired with TPU1).

Returns: Volatile pointer to TPU5 register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Post: Pointer valid for lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address] **See also:** k_tpu5_base_addr
Base address constant

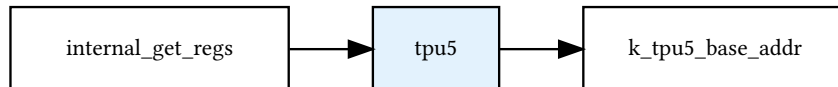


Figure 659: Call graph for tpu5

_Defined in libs/rx_hal/inc/rx72n_tpu_regs.h:653_

Since Version 1.0.0

—

rx72n_usb_regs.h

RX72N USB 2.0 Full-Speed Host/Function Module Register Definitions.

Register definitions for the USB 2.0 Full-Speed Host/Function Module (USB0) verified against RX72N Group User's Manual: Hardware, Chapter 40.

2026-02-03

Copyright (c) 2026 STAR Project

Includes:

- <stddef.h>
- <stdint.h>

usb_register_offsets_t

USB register offsets (verified against manual Chapter 40).

Value	Name	Description
= 0x00	k_usb_offset_syscfg	
= 0x04	k_usb_offset_syssts0	
= 0x08	k_usb_offset_dvstctr0	
= 0x14	k_usb_offset_cfifo	
= 0x18	k_usb_offset_d0fifo	
= 0x1C	k_usb_offset_d1fifo	
= 0x20	k_usb_offset_cfifosel	
= 0x22	k_usb_offset_cfifoctr	
= 0x28	k_usb_offset_d0fifosel	
= 0x2A	k_usb_offset_d0fifocr	
= 0x2C	k_usb_offset_d1fifosel	
= 0x2E	k_usb_offset_d1fifocr	
= 0x30	k_usb_offset_intenb0	
= 0x32	k_usb_offset_intenb1	
= 0x36	k_usb_offset_bradyenb	
= 0x38	k_usb_offset_nrdyenb	
= 0x3A	k_usb_offset_bempenb	
= 0x3C	k_usb_offset_sofcfg	
= 0x40	k_usb_offset_intsts0	
= 0x42	k_usb_offset_intsts1	
= 0x46	k_usb_offset_brdysts	
= 0x48	k_usb_offset_nrdysts	
= 0x4A	k_usb_offset_bempsts	
= 0x4C	k_usb_offset_frmnum	
= 0x4E	k_usb_offset_dvchgr	
= 0x50	k_usb_offset_usbaddr	
= 0x54	k_usb_offset_usbreq	
= 0x56	k_usb_offset_usbval	
= 0x58	k_usb_offset_usbindx	
= 0x5A	k_usb_offset_usbleng	
= 0x5C	k_usb_offset_dcpcfg	
= 0x5E	k_usb_offset_dcpmaxp	
= 0x60	k_usb_offset_dcpctr	
= 0x64	k_usb_offset_pipesel	
= 0x68	k_usb_offset_pipecfg	
= 0x6C	k_usb_offset_pipemaxp	

= 0x6E	k_usb_offset_pipeperi	
= 0x70	k_usb_offset_pipe1ctr	
= 0x80	k_usb_offset_pipe9ctr	
= 0x90	k_usb_offset_pipe1tre	
= 0xA2	k_usb_offset_pipe5trn	
= 0xD0	k_usb_offset_devadd0	
= 0xDA	k_usb_offset_devadd5	
= 0xF0	k_usb_offset_physlew	

—

usb_reserved_sizes_t

Reserved field sizes for USB register structure.

Value	Name	Description
= 2	k_usb_reserved_02_04_bytes	
= 2	k_usb_reserved_06_08_bytes	
= 10	k_usb_reserved_0a_14_bytes	
= 4	k_usb_reserved_24_28_bytes	
= 2	k_usb_reserved_34_36_bytes	
= 2	k_usb_reserved_3e_40_bytes	
= 2	k_usb_reserved_44_46_bytes	
= 2	k_usb_reserved_52_54_bytes	
= 2	k_usb_reserved_62_64_bytes	
= 2	k_usb_reserved_66_68_bytes	
= 2	k_usb_reserved_6a_6c_bytes	
= 14	k_usb_reserved_82_90_bytes	
= 44	k_usb_reserved_a4_d0_bytes	
= 20	k_usb_reserved_dc_f0_bytes	

—

rx_usb_addresses_t

Value	Name	Description
= 0x000A0000	k_usb0_base_addr	

—

usb_syscfg_bits_t

Value	Name	Description
= (1U << 0)	k_usb_syscfg_usbe	
= (1U << 4)	k_usb_syscfg_dprpu	

= (1U \<\< 5)	k_usb_syscfg_drpdcfg	
= (1U \<\< 6)	k_usb_syscfg_dcfm	
= (1U \<\< 10)	k_usb_syscfg_scke	

—

usb_syssts0_bits_t

Value	Name	Description
= 0x0003	k_usb_syssts0_lnst_mask	
= 0x0000	k_usb_syssts0_lnst_se0	
= 0x0001	k_usb_syssts0_lnst_fs_j	
= 0x0002	k_usb_syssts0_lnst_fs_k	
= 0x0003	k_usb_syssts0_lnst_sel	
= (1U \<\< 2)	k_usb_syssts0_idmon	
= (1U \<\< 5)	k_usb_syssts0_sofea	
= (1U \<\< 6)	k_usb_syssts0_htact	
= (3U \<\< 14)	k_usb_syssts0_ovcmon_mask	

—

usb_dvstctr0_bits_t

Value	Name	Description
= 0x0007	k_usb_dvstctr0_rhst_mask	
= 0x0000	k_usb_dvstctr0_rhst_undecided	
= 0x0001	k_usb_dvstctr0_rhst_ls	
= 0x0002	k_usb_dvstctr0_rhst_fs	
= 0x0004	k_usb_dvstctr0_rhst_reset	
= (1U \<\< 4)	k_usb_dvstctr0_uact	
= (1U \<\< 5)	k_usb_dvstctr0_resume	
= (1U \<\< 6)	k_usb_dvstctr0_usbrst	
= (1U \<\< 7)	k_usb_dvstctr0_rwupe	
= (1U \<\< 8)	k_usb_dvstctr0_wkup	

—

usb_intenb0_bits_t

Value	Name	Description
= (1U \<\< 8)	k_usb_intenb0_brbye	
= (1U \<\< 9)	k_usb_intenb0_nrdye	
= (1U \<\< 10)	k_usb_intenb0_bempe	
= (1U \<\< 11)	k_usb_intenb0_ctre	

= (1U \<\< 12)	k_usb_intenb0_dvse	
= (1U \<\< 13)	k_usb_intenb0_sofe	
= (1U \<\< 14)	k_usb_intenb0_rsme	
= (1U \<\< 15)	k_usb_intenb0_vbse	

—

usb_intsts0_bits_t

Value	Name	Description
= 0x0007	k_usb_intsts0_ctsq_mask	
= 0x0000	k_usb_intsts0_ctsq_idle	
= 0x0001	k_usb_intsts0_ctsq_rd_data	
= 0x0002	k_usb_intsts0_ctsq_rd_status	
= 0x0003	k_usb_intsts0_ctsq_wr_data	
= 0x0004	k_usb_intsts0_ctsq_wr_status	
= 0x0005	k_usb_intsts0_ctsq_wr_nd	
= 0x0006	k_usb_intsts0_ctsq_seq_err	
= (1U \<\< 3)	k_usb_intsts0_valid	
= (7U \<\< 4)	k_usb_intsts0_dvsq_mask	
= (0U \<\< 4)	k_usb_intsts0_dvsq_powered	
= (1U \<\< 4)	k_usb_intsts0_dvsq_default	
= (2U \<\< 4)	k_usb_intsts0_dvsq_address	
= (3U \<\< 4)	k_usb_intsts0_dvsq_configured	
= (4U \<\< 4)	k_usb_intsts0_dvsq_suspend	
= (1U \<\< 8)	k_usb_intsts0_brdy	
= (1U \<\< 9)	k_usb_intsts0_nrdy	
= (1U \<\< 10)	k_usb_intsts0_bemp	
= (1U \<\< 11)	k_usb_intsts0_ctrtd	
= (1U \<\< 12)	k_usb_intsts0_dvst	
= (1U \<\< 13)	k_usb_intsts0_sofr	
= (1U \<\< 14)	k_usb_intsts0_resm	
= (1U \<\< 15)	k_usb_intsts0_vbint	

—

usb_dcpcfg_bits_t

Value	Name	Description
= (1U \<\< 4)	k_usb_dcpcfg_dir	
= (1U \<\< 7)	k_usb_dcpcfg_shtnak	

usb_dcpctr_bits_t

Value	Name	Description
= 0x0003	k_usb_dcpctr_pid_mask	
= 0x0000	k_usb_dcpctr_pid_nak	
= 0x0001	k_usb_dcpctr_pid_buf	
= 0x0002	k_usb_dcpctr_pid_stall	
= (1U << 2)	k_usb_dcpctr_ccpl	
= (1U << 5)	k_usb_dcpctr_pbusy	
= (1U << 6)	k_usb_dcpctr_sqmon	
= (1U << 7)	k_usb_dcpctr_sqset	
= (1U << 8)	k_usb_dcpctr_sqclr	
= (1U << 11)	k_usb_dcpctr_sureqclr	
= (1U << 14)	k_usb_dcpctr_sureq	
= (1U << 15)	k_usb_dcpctr_bsts	

usb_pipe_bits_t

USB pipe interrupt control bits.

Bit definitions for BRDYENB/BRDYSTS (buffer ready), BEMPENB/BEMPSTS (buffer empty), and NRDYENB/NRDYSTS (not ready) registers.

These registers control per-pipe interrupt enable/status for all 10 USB pipes (DCP + Pipe 1-9). Each bit corresponds to one pipe:

- Bit 0: DCP (Default Control Pipe)
- Bits 1-9: Data pipes 1-9

Value	Name	Description
= (1U << 0)	k_usb_pipe_bit_0	DCP (Default Control Pipe)
= (1U << 1)	k_usb_pipe_bit_1	Data pipe 1
= (1U << 2)	k_usb_pipe_bit_2	Data pipe 2
= (1U << 3)	k_usb_pipe_bit_3	Data pipe 3
= (1U << 4)	k_usb_pipe_bit_4	Data pipe 4
= (1U << 5)	k_usb_pipe_bit_5	Data pipe 5
= (1U << 6)	k_usb_pipe_bit_6	Data pipe 6
= (1U << 7)	k_usb_pipe_bit_7	Data pipe 7
= (1U << 8)	k_usb_pipe_bit_8	Data pipe 8
= (1U << 9)	k_usb_pipe_bit_9	Data pipe 9
= 0x03FF	k_usb_pipe_all	All 10 pipes (mask for bits 9-0)

Note: RX72N has 10 pipes total: 1 control pipe (DCP) + 9 data pipes

Note: Each CDC-ACM port uses 3 pipes (2 bulk + 1 interrupt)

Note: Maximum 3 CDC-ACM ports supported (9 pipes / 3 = 3 ports)] **See also:** `usb_brdyenb_t` Buffer ready interrupt enable register, `usb_brdysts_t` Buffer ready interrupt status register, `usb_bempenb_t` Buffer empty interrupt enable register, `usb_bempsts_t` Buffer empty interrupt status register, `usb_nrdyenb_t` Buffer not ready interrupt enable register, `usb_nrdysts_t` Buffer not ready interrupt status register

usb_pipecfg_bits_t

Value	Name	Description
= 0x000F	k_usb_pipecfg_epnum_mask	
= (1U << 4)	k_usb_pipecfg_dir	
= (1U << 7)	k_usb_pipecfg_shtnak	
= (1U << 9)	k_usb_pipecfg_dblb	
= (1U << 10)	k_usb_pipecfg_bfre	
= (3U << 14)	k_usb_pipecfg_type_mask	
= (1U << 14)	k_usb_pipecfg_type_bulk	
= (2U << 14)	k_usb_pipecfg_type_int	
= (3U << 14)	k_usb_pipecfg_type_iso	

usb_pipectr_bits_t

Value	Name	Description
= 0x0003	k_usb_pipectr_pid_mask	
= 0x0000	k_usb_pipectr_pid_nak	
= 0x0001	k_usb_pipectr_pid_buf	
= 0x0002	k_usb_pipectr_pid_stall	
= (1U << 5)	k_usb_pipectr_pbusy	
= (1U << 6)	k_usb_pipectr_sqmon	
= (1U << 7)	k_usb_pipectr_sqset	
= (1U << 8)	k_usb_pipectr_sqclr	
= (1U << 9)	k_usb_pipectr_aclrm	
= (1U << 10)	k_usb_pipectr_atrepm	
= (1U << 14)	k_usb_pipectr_inbufm	
= (1U << 15)	k_usb_pipectr_bsts	

—

usb_fifoel_bits_t

Value	Name	Description
= 0x000F	k_usb_fifoel_curpipe_mask	
= 0x0000	k_usb_fifoel_curpipe_dcp	
= (1U << 5)	k_usb_fifoel_isel	
= (1U << 8)	k_usb_fifoel_bigend	
= (3U << 10)	k_usb_fifoel_mbw_mask	
= (0U << 10)	k_usb_fifoel_mbw_8	
= (1U << 10)	k_usb_fifoel_mbw_16	
= (2U << 10)	k_usb_fifoel_mbw_32	
= (1U << 12)	k_usb_fifoel_drege	
= (1U << 13)	k_usb_fifoel_dclrm	
= (1U << 14)	k_usb_fifoel_rcl	
= (1U << 15)	k_usb_fifoel_frdy	

—

usb_fifoctr_bits_t

Value	Name	Description
= 0xFFFF	k_usb_fifoctr_dtlm_mask	
= (1U << 13)	k_usb_fifoctr_frdy	
= (1U << 14)	k_usb_fifoctr_bclr	
= (1U << 15)	k_usb_fifoctr_bval	

—

rx_usb_interrupt_vector_t

Value	Name	Description
= 34	k_vect_usb0_d0fifo	
= 35	k_vect_usb0_d1fifo	
= 36	k_vect_usb0_usbi	
= 90	k_vect_usb0_usbr	

—

usb_cdc_endpoints_t

Value	Name	Description
= 0	k_usb_cdc_ep_ctrl	
= 1	k_usb_cdc_ep_bulk_in	
= 2	k_usb_cdc_ep_bulk_out	

= 3	k_usb_cdc_ep_interrupt_in	
= 64	k_usb_cdc_max_packet_fs	

—

usb0

```
static volatile rx_usb_regs_t * usb0(void)
```

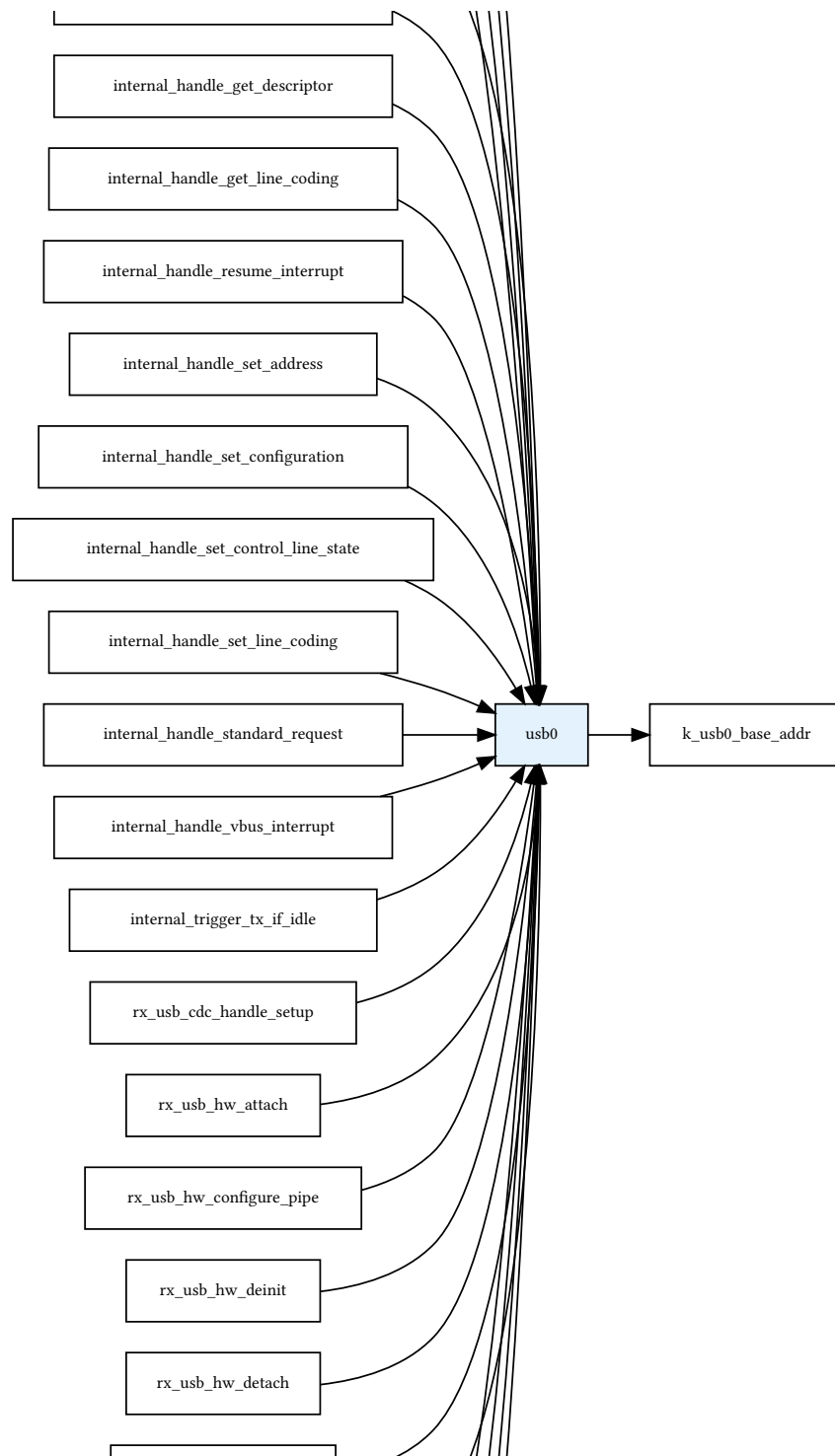


Figure 660: Call graph for usb0

_Defined in libs/rx_hal/inc/rx72n_usb_regs.h:204_

—

rx72n_wdt_regs.h

RX72N WDT Watchdog Timer Register Definitions.

Register definitions for the Watchdog Timer (WDT) module on the RX72N microcontroller. The WDT provides system monitoring to detect software runaway conditions and reset the system if necessary.

Example with PCLKB = 60 MHz, TOPS = 16384, CKS = 8192:

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stddef.h>
- <stdint.h>

rx_wdt_addresses_t

WDT base address constant.

Base address for the Watchdog Timer register block at 0x00088020. The WDT has an 8-byte register space with 4 registers and 2 reserved bytes.

Value	Name	Description
= 0x00088020	k_wdt_base_addr	WDT register base address (0x00088020).

See also: wdt() Accessor function

Since Version 1.0.0

—

wdt_reserved_sizes_t

WDT register reserved field sizes.

Defines reserved byte counts for structure padding to match hardware layout.

Value	Name	Description
= 1	k_wdt_reserved_after_wdtrr_bytes	Reserved byte after WDTRR @ 0x01
= 1	k_wdt_reserved_after_wdtrcr_bytes	Reserved byte after WDTRCR @ 0x07

Since Version 1.0.0

—

wdt_refresh_sequence_t

WDT Refresh Register (WDTRR) Write Sequence Values.

The WDT counter is refreshed by writing a specific two-byte sequence:

1. Write 0x00 (k_wdt_refresh_start)
2. Write 0xFF (k_wdt_refresh_end)

This sequence must complete within the valid refresh window if window protection is enabled, otherwise a refresh error (REFEF) will occur.

Value	Name	Description
= 0x00	k_wdt_refresh_start	First write value for refresh sequence (0x00).
= 0xFF	k_wdt_refresh_end	Second write value for refresh sequence (0xFF).

Warning: Both writes must complete in sequence - partial refresh is invalid

Warning: Do not insert other operations between the two writes] **See also:**
rx_wdt_regs_t::wdtrr Refresh register

Since Version 1.0.0

—

wdt_wdtr_pos_t

WDT Control Register (WDTCR) Bit Field Positions.

Bit positions for WDTCR register fields. Used to construct field values.

Value	Name	Description
= 4	k_wdt_cr_cks_pos	CKS field position (bits 7:4)
= 8	k_wdt_cr_rpes_pos	RPES field position (bits 9:8)
= 12	k_wdt_cr_rpss_pos	RPSS field position (bits 13:12)

Since Version 1.0.0

—

wdt_wdtr_bits_t

WDT Control Register (WDTCR) Bit Field Values.

Configuration values for WDT timeout period, clock division, and window protection. These values can be OR'd together to configure WDTCR.

Value	Name	Description
= 0x0000	k_wdt_tops_1024	1024 cycles timeout (counter loads 0x03FF)
= 0x0001	k_wdt_tops_4096	4096 cycles timeout (counter loads 0x0FFF)
= 0x0002	k_wdt_tops_8192	8192 cycles timeout (counter loads 0x1FFF)
= 0x0003	k_wdt_tops_16384	16384 cycles timeout (counter loads 0x3FFF)
= (0x01 << k_wdt_cr_cks_pos)	k_wdt_cks_div_4	PCLKB divided by 4 (fastest, shortest timeout).

= (0x04 \<\< k_wdt_cr_cks_pos)	k_wdt_cks_div_64	PCLKB divided by 64.
= (0x0F \<\< k_wdt_cr_cks_pos)	k_wdt_cks_div_128	PCLKB divided by 128.
= (0x06 \<\< k_wdt_cr_cks_pos)	k_wdt_cks_div_512	PCLKB divided by 512.
= (0x07 \<\< k_wdt_cr_cks_pos)	k_wdt_cks_div_2048	PCLKB divided by 2048.
= (0x08 \<\< k_wdt_cr_cks_pos)	k_wdt_cks_div_8192	PCLKB divided by 8192 (slowest, longest timeout).
= (0x00 \<\< k_wdt_cr_rpes_pos)	k_wdt_rpes_75	Window closes at 75% counter remaining.
= (0x01 \<\< k_wdt_cr_rpes_pos)	k_wdt_rpes_50	Window closes at 50% counter remaining.
= (0x02 \<\< k_wdt_cr_rpes_pos)	k_wdt_rpes_25	Window closes at 25% counter remaining.
= (0x03 \<\< k_wdt_cr_rpes_pos)	k_wdt_rpes_0	No window end (refresh always allowed until timeout).
= (0x00 \<\< k_wdt_cr_rpss_pos)	k_wdt_rpss_25	Window opens at 25% counter remaining.
= (0x01 \<\< k_wdt_cr_rpss_pos)	k_wdt_rpss_50	Window opens at 50% counter remaining.
= (0x02 \<\< k_wdt_cr_rpss_pos)	k_wdt_rpss_75	Window opens at 75% counter remaining.
= (0x03 \<\< k_wdt_cr_rpss_pos)	k_wdt_rpss_100	No window start (refresh allowed immediately).

Warning: Only 6 CKS settings are valid - others are prohibited] **See also:**
 rx_wdt_regs_t::wdtcr Control register

Since Version 1.0.0

—

wdt_wdtsr_pos_t

WDT Status Register (WDTSR) Bit Field Positions.

Bit positions for status flags in the WDTSR register.

Value	Name	Description
= 14	k_wdt_sr_undff_pos	Underflow flag bit position
= 15	k_wdt_sr_refef_pos	Refresh error flag bit position

Since Version 1.0.0

—

wdt_wdtsr_bits_t

WDT Status Register (WDTSR) Bit Field Values.

Contains the current down-counter value and error flags. The counter value can be read at any time; flags indicate timeout or window violation.

Value	Name	Description
= 0x3FFF	k_wdt_sr_cntval_mask	Counter value mask (bits 13:0).
= (1U << k_wdt_sr_undff_pos)	k_wdt_sr_undff	Underflow flag - Bit 14.
= (1U << k_wdt_sr_refef_pos)	k_wdt_sr_refef	Refresh error flag - Bit 15.

See also: rx_wdt_regs_t::wdtsr Status register

Since Version 1.0.0

—

wdt_wdtrcr_bits_t

WDT Reset Control Register (WDTRCR) Bit Field Values.

Controls the action taken on WDT timeout or window violation. Choice between generating an internal reset or Non-Maskable Interrupt (NMI).

Value	Name	Description
= 7	k_wdt_rcr_rstirqs_pos	RSTIRQS bit position.
= (1U << k_wdt_rcr_rstirqs_pos)	k_wdt_rcr_rstirqs_mask	RSTIRQS bit mask.
= (1U << k_wdt_rcr_rstirqs_pos)	k_wdt_rstirqs_reset	Generate internal reset on timeout (RSTIRQS=1).
= 0x00	k_wdt_rstirqs_nmi	Generate NMI on timeout (RSTIRQS=0).

See also: rx_wdt_regs_t::wdtrcr Reset control register

Since Version 1.0.0

—

wdt

```
static volatile rx_wdt_regs_t * wdt(void)
```

Get pointer to WDT registers.

Returns a volatile pointer to the WDT register structure at address 0x00088020. The WDT is a software-controllable watchdog timer that uses the PCLKB peripheral clock.

Returns: Volatile pointer to WDT register structure

Value	Description
Non-NULL	Always returns valid pointer (hardware address)

Pre: None (hardware register always accessible)

Post: Pointer is valid for the lifetime of program execution

Note: Thread Safety: Safe - returns constant hardware address

Note: This function is always inlined for zero overhead

Example:: `wdt()->wdtrr=k_wdt_refresh_start; wdt()->wdtrr=k_wdt_refresh_end;`
`uint16_tcounter=wdt()->wdtsr&k_wdt_sr_cntval_mask;`

See also: `k_wdt_base_addr` Base address constant, `rx_wdt_init()` Higher-level initialization

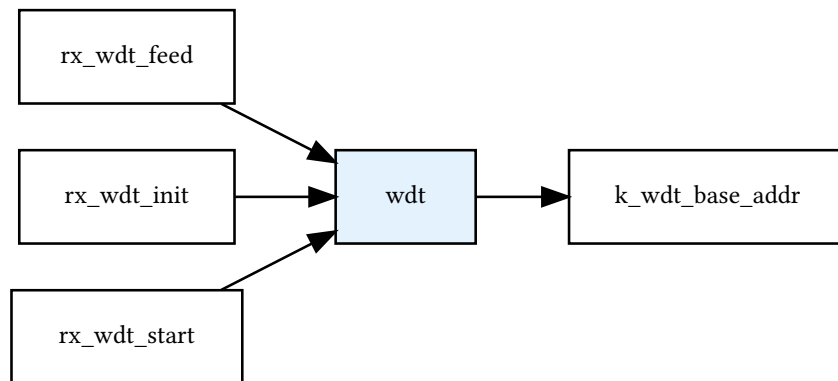


Figure 661: Call graph for wdt

_Defined in `libs/rx\_hal/inc/rx72n\_wdt\_regs.h:350_`

Since Version 1.0.0

—

rx_cmt.h

CMT (Compare Match Timer) Driver API for RX72N - Periodic Interrupt Generation.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`

rx_cmt_channel_t

CMT channel identifiers.

Identifies the 4 available CMT channels. Channels are grouped into units:

- Unit 0: CMT0, CMT1 (shared CMSTR0)
- Unit 1: CMT2, CMT3 (shared CMSTR1)

Value	Name	Description
= 0	<code>k_cmt_channel_0</code>	CMT0 - Reserved for ThreadX system tick (1 kHz)

= 1	k_cmt_channel_1	CMT1 - Motor control loop (user configurable)
= 2	k_cmt_channel_2	CMT2 - Available for general use
= 3	k_cmt_channel_3	CMT3 - Available for general use

—

rx_cmt_callback_t

```
typedef void(* rx_cmt_callback_t) (void *user_data)
```

CMT interrupt callback function type.

User-provided function called from CMT interrupt context on each compare match. Keep callbacks short and non-blocking to avoid interrupt latency issues.

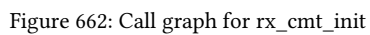
—

rx_cmt_init

```
rx_err_t rx_cmt_init(rx_cmt_channel_t channel, const rx_cmt_config_t *config)
```

Initialize CMT channel for periodic interrupts.

Configures a CMT channel for periodic interrupt generation at the specified frequency. The timer starts immediately after initialization.



Defined in `libs/rx_hal/inc/rx_cmt.h:450`

—

rx_cmt_start

```
rx_err_t rx_cmt_start(rx_cmt_channel_t channel)
```

Start CMT timer.

Starts the timer counter and interrupt generation. The timer is automatically started during `rx_cmt_init()`, so this function is only needed after calling `rx_cmt_stop()` to resume operation.

Start CMT timer.

Sets the appropriate bit in the CMSTR0 or CMSTR1 register to start the CMT counter running. After setting the bit the register is read back to verify the hardware accepted the write. Channels 0/1 share CMSTR0; channels 2/3 share CMSTR1. Channels 0 and 1 use STR0/STR1 bits respectively, as do channels 2 and 3.

Parameters:

Direction	Name	Description
in	channel	CMT channel to start Valid range: k_cmt_channel_0 to k_cmt_channel_3
in	channel	CMT channel to start Valid range: k_cmt_channel_0 through k_cmt_channel_3 Note: Usually called internally by rx_cmt_init()

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Timer started successfully
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized
k_rx_err_hw_error	Hardware failed to start (CMSTR readback failed)
k_rx_ok	Success; counter is running
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized
k_rx_err_hw_error	CMSTR readback did not show bit set

Pre: Channel initialized via `rx_cmt_init()`

Pre: channel must be initialized via `rx_cmt_init()`

Pre: channel must be in range 0, 3

Post: Timer counter incrementing

Post: Interrupts generating at configured frequency

Post: CMSTR bit for channel is set

Post: Counter begins incrementing from its current value

Note: Counter resumes from current value (not reset to 0).

Note: Usually called automatically by `rx_cmt_init()`; call explicitly only after `rx_cmt_stop()`

Note: Performs a readback check to verify the hardware write succeeded

Thread Safety:: Not thread-safe. Call from single thread or protect with mutex.

Thread Safety:: Not thread-safe; do not call concurrently with stop/deinit on the same channel.

Example:: `rx_cmt_stop(k_cmt_channel_1); rx_cmt_start(k_cmt_channel_1);`

See also: `rx_cmt_stop()` Stop timer, `rx_cmt_init()` Initialize channel, `rx_cmt_stop()` Stop the counter, `rx_cmt_init()` Initialize channel (calls start internally)

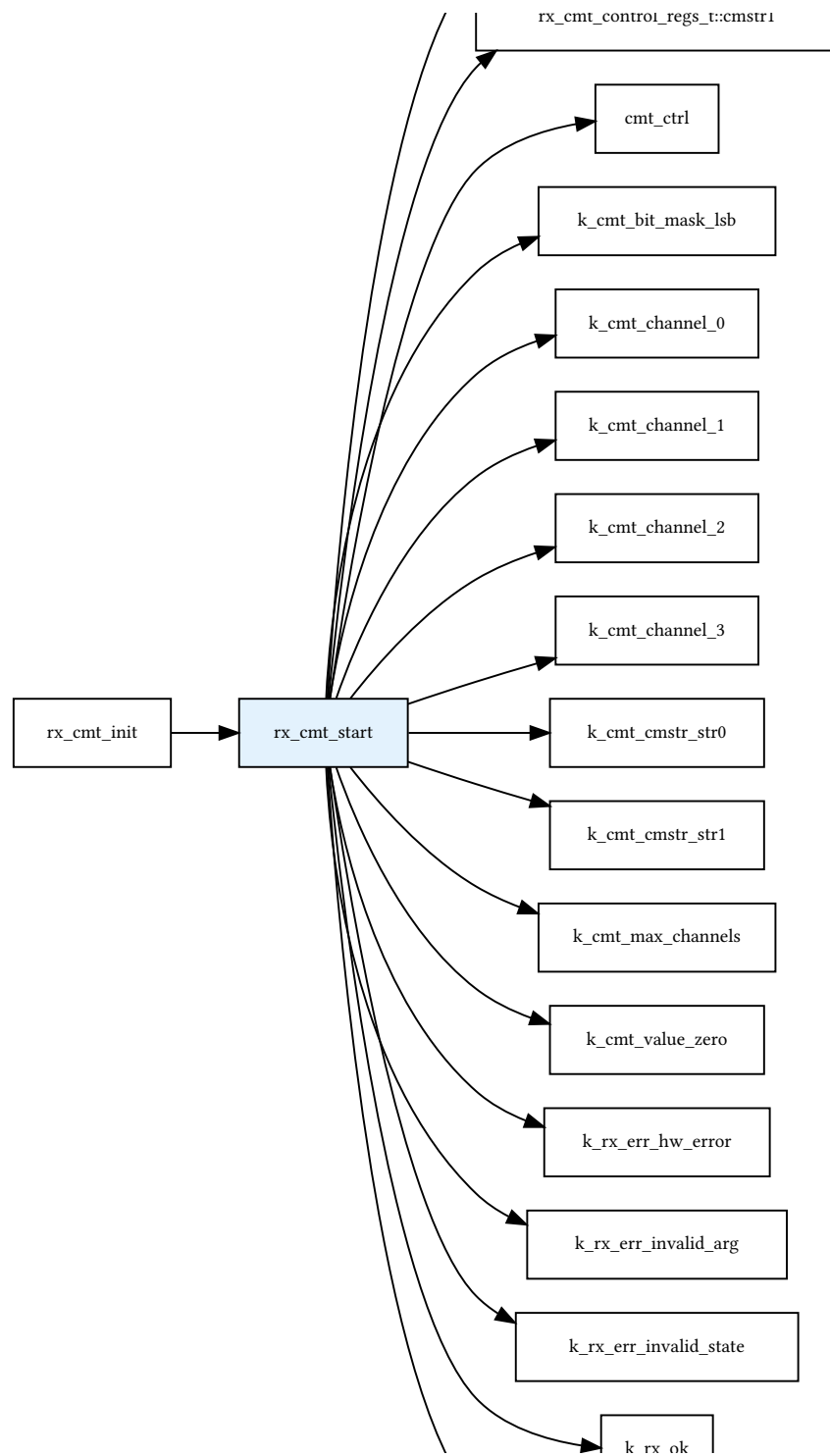


Figure 663: Call graph for rx_cmt_start

Defined in `libs/rx_hal/inc/rx_cmt.h:485`

Since Version 1.0.0

—

rx_cmt_stop

```
rx_err_t rx_cmt_stop(rx_cmt_channel_t channel)
```

Stop CMT timer.

Stops the timer counter and interrupt generation. Counter value is preserved and can be read with `rx_cmt_get_count()`. Use `rx_cmt_start()` to resume.

Stop CMT timer.

Clears the appropriate bit in CMSTR0 or CMSTR1 to halt the CMT counter. After clearing the bit the register is read back to verify the hardware accepted the write. The channel remains initialized; call `rx_cmt_start()` to resume counting.

Note that `rx_cmt_stop()` does NOT require the channel to be initialized (`s_cmt_initialized` check is omitted intentionally so that `rx_cmt_init()` can call it safely before marking the channel as initialized).

Parameters:

Direction	Name	Description
in	channel	CMT channel to stop Valid range: k_cmt_channel_0 to k_cmt_channel_3
in	channel	CMT channel to stop Valid range: k_cmt_channel_0 through k_cmt_channel_3

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
k_rx_ok	Timer stopped successfully
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_hw_error	Hardware failed to stop (CMSTR readback failed)
k_rx_ok	Success; counter halted
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_hw_error	CMSTR readback showed bit still set

Pre: channel must be in range 0, 3

Pre: Interrupts may still fire if one is already pending when stop is called

Post: Timer counter frozen at current value

Post: No further interrupts until `rx_cmt_start()` called

Post: CMSTR bit for channel is cleared

Post: Counter stops incrementing (current CMCNT value preserved)

Note: Counter value preserved. Call rx_cmt_start() to resume counting.

Note: Does not clear the initialized flag - channel can be restarted

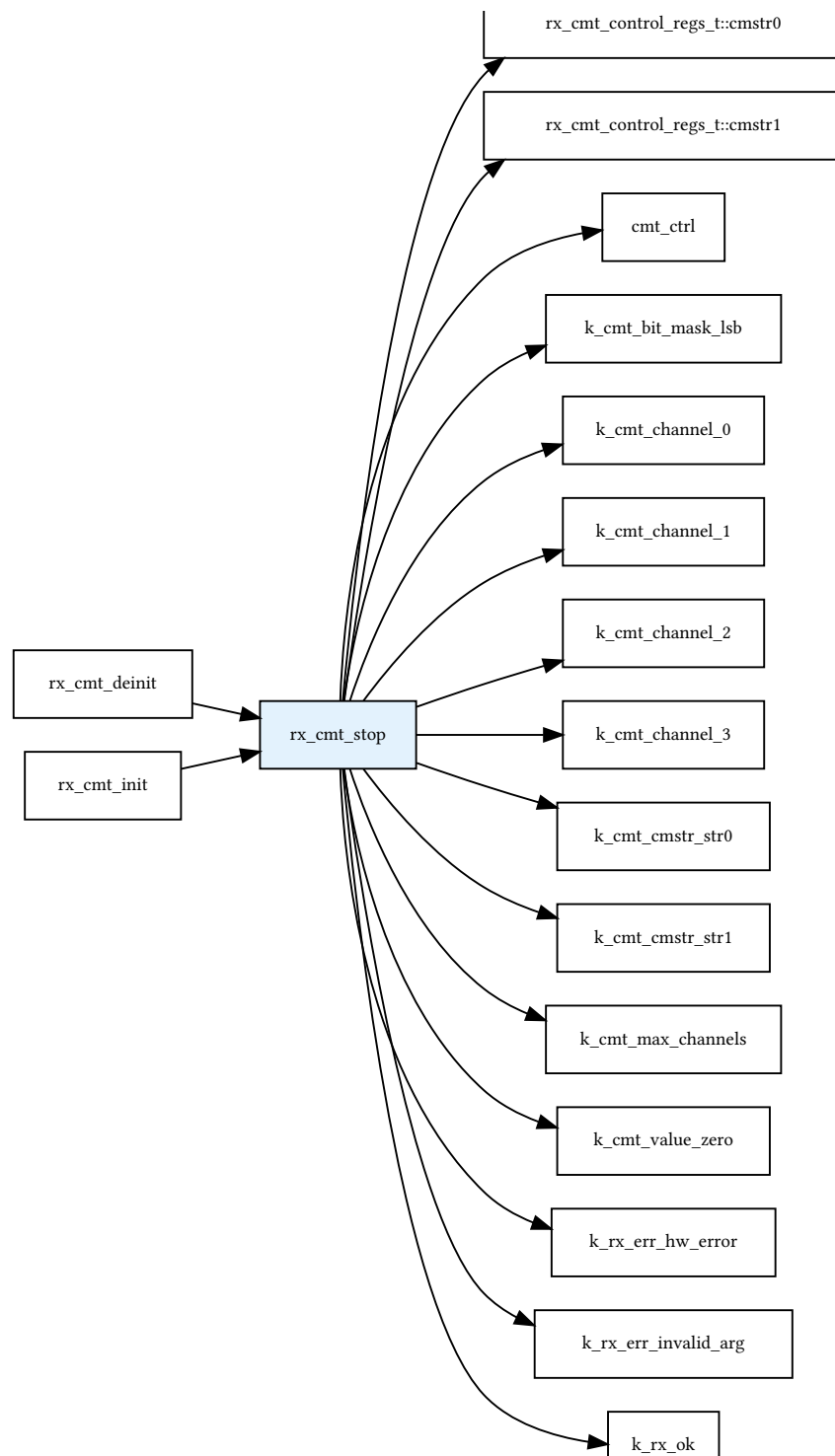
Note: Called by rx_cmt_init() before programming registers (safe on uninitialized channel)

Thread Safety:: Not thread-safe. Call from single thread or protect with mutex.

Thread Safety:: Not thread-safe; do not call concurrently with start/init/deinit on the same channel.

Example:: rx_cmt_stop(k_cmt_channel_2); reconfigure_something();
rx_cmt_start(k_cmt_channel_2);

See also: rx_cmt_start() Resume timer, rx_cmt_get_count() Read frozen counter value,
rx_cmt_start() Resume counting, rx_cmt_deinit() Stop and release channel resources

Figure 664: Call graph for `rx_cmt_stop`

Defined in `libs/rx_hal/inc/rx_cmt.h:516`

Since Version 1.0.0

—

rx_cmt_get_count

```
rx_err_t rx_cmt_get_count(rx_cmt_channel_t channel, uint16_t *count)
```

Get CMT counter value.

Reads the current 16-bit counter value. Useful for timing measurements or debugging. Counter resets to 0 on each compare match.

Get CMT counter value.

Returns the instantaneous value of the 16-bit CMCNT register for the specified channel. The counter increments at PCLKB divided by the configured divider (/8, /32, /128, or /512) and resets to 0 on compare match. Useful for measuring elapsed time within a period.

Parameters:

Direction	Name	Description
in	channel	CMT channel to read Valid range: k_cmt_channel_0 to k_cmt_channel_3
out	count	Pointer to store counter value (0-65535)
in	channel	CMT channel to read (0-3) Valid range: k_cmt_channel_0 through k_cmt_channel_3
out	count	Pointer to store the 16-bit counter value Valid range: Non-nullptr to uint16_t On success: Contains CMCNT value (0 to CMCOR) Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Counter value read successfully
k_rx_err_null_ptr	count pointer is nullptr
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized
k_rx_ok	Success; *count contains current CMCNT value
k_rx_err_null_ptr	count is nullptr
k_rx_err_invalid_state	Channel not initialized or invalid channel
k_rx_err_invalid_arg	Could not obtain register base pointer

Pre: Channel initialized via rx_cmt_init()

Pre: count != nullptr

Pre: Channel must be initialized via `rx_cmt_init()`

Pre: count must point to valid `uint16_t` storage

Post: *count contains current counter value

Post: *count set to CMCNT register value at time of read

Post: Counter continues running (read is non-destructive)

Note: Counter may change immediately after reading if timer running.

Note: Value may change between consecutive reads if counter is running

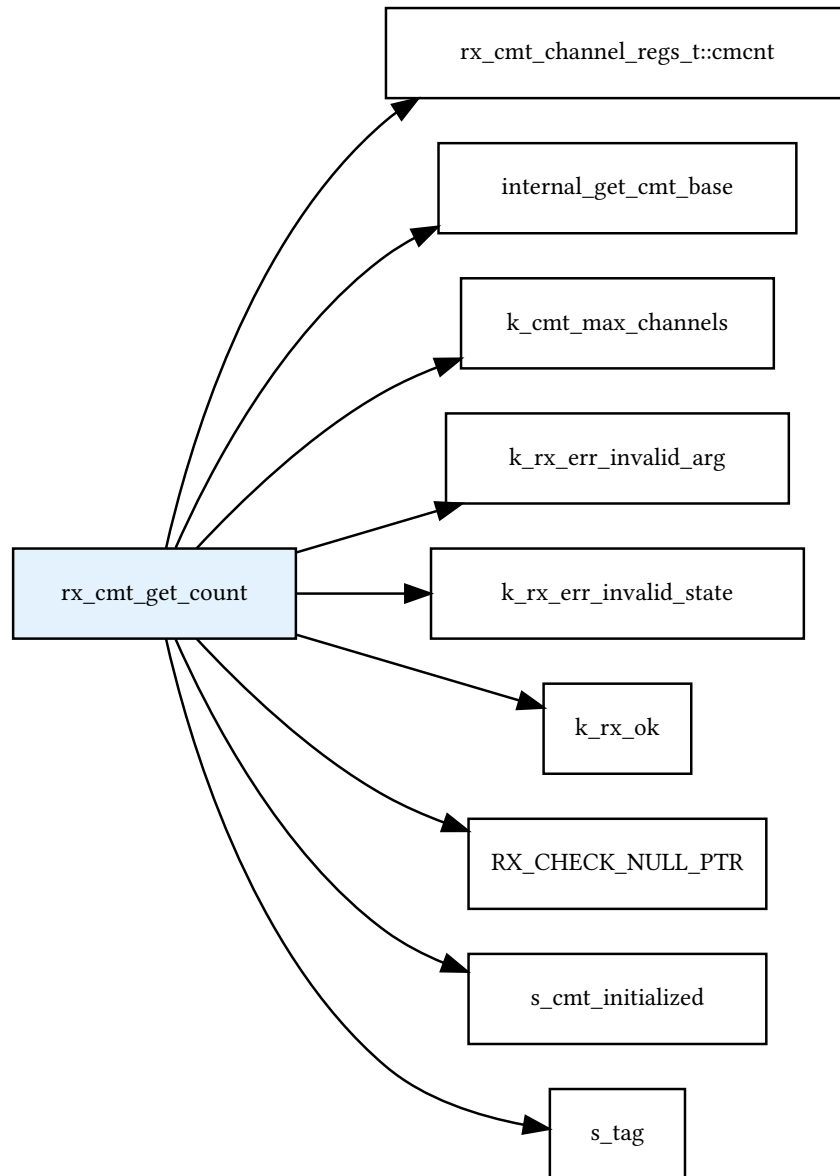
Note: For precise time measurements disable interrupts around consecutive reads

Thread Safety:: Safe to call from multiple threads (atomic 16-bit read).

Thread Safety:: Read-only; safe to call from multiple contexts, but value may be stale.

Example:: `uint16_t ticks=0; rx_err_t err=rx_cmt_get_count(k_cmt_channel_1,&ticks);
if(err!=k_rx_ok){ }`

See also: `rx_cmt_stop()` Stop timer to freeze counter, `rx_cmt_init()` Initialize channel and set frequency, `rx_cmt_start()` Start the counter

Figure 665: Call graph for `rx_cmt_get_count`

Defined in `libs/rx_hal/inc/rx_cmt.h:550`

Since Version 1.0.0

—

rx_cmt_deinit

```
rx_err_t rx_cmt_deinit(rx_cmt_channel_t channel)
```

Deinitialize CMT channel.

Stops timer, disables interrupts, clears callback, and marks channel as uninitialized. Channel can be reinitialized with rx_cmt_init() after this.

Deinitialize CMT channel.

Stops the CMT counter, disables the ICU interrupt for the channel, clears the stored callback and user data, and marks the channel as uninitialized. After this call the channel may be re-initialized with rx_cmt_init().

Algorithm steps:

1. Validate channel range
2. Stop timer via rx_cmt_stop() and propagate any error
3. Compute ICU vector, IER index, and IER bit for the channel
4. Clear the IER bit to disable the interrupt
5. Clear callback, user_data, and initialized flag

Parameters:

Direction	Name	Description
in	channel	CMT channel to deinitialize Valid range: k_cmt_channel_0 to k_cmt_channel_3
in	channel	CMT channel to deinitialize (0-3) Valid range: k_cmt_channel_0 through k_cmt_channel_3

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Channel deinitialized successfully
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_hw_error	Hardware failed to stop
k_rx_ok	Success; channel fully deinitialized
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_hw_error	Timer stop verification failed

Pre: channel must be in range 0, 3

Pre: Caller should ensure no other task is using the channel callback

Post: Timer stopped

Post: Interrupt disabled

Post: Callback cleared

Post: Channel marked as uninitialized

Post: Counter halted (CMSTR bit cleared)

Post: ICU interrupt disabled for channel

Post: s_cmt_initializedchannel set to false

Post: s_cmt_callbackchannel and s_cmt_user_datachannel set to nullptr

Note: Does not re-enable module stop bit (CMT module remains powered).

Note: Safe to call on an uninitialized channel (rx_cmt_stop tolerates it)

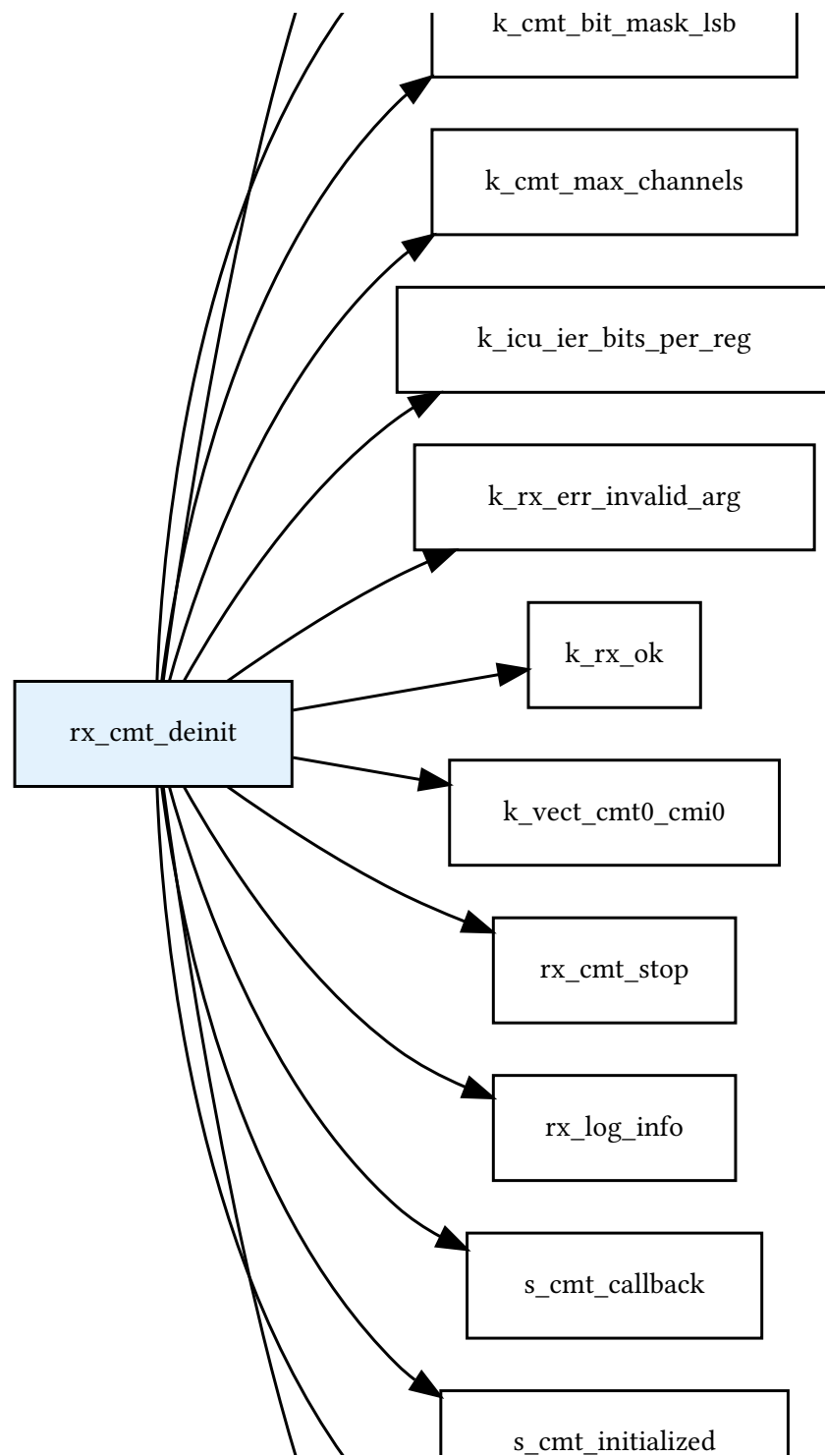
Note: Module clock is NOT re-asserted - hardware remains clocked for re-use

Thread Safety:: Not thread-safe. Ensure no callbacks executing during deinit.

Thread Safety:: Not thread-safe. Ensure no ISR or other thread is using the channel.

Example:: rx_err_t err=rx_cmt_deinit(k_cmt_channel_1); if(err!=k_rx_ok)
{ rx_log_error("APP","CMTdeinitfailed"); }

See also: rx_cmt_init() Reinitialize channel, rx_cmt_init() Re-initialize after deinit,
rx_cmt_stop() Stop without full deinit

Figure 666: Call graph for `rx_cmt_deinit`

Defined in `libs/rx_hal/inc/rx_cmt.h:582`

Since Version 1.0.0

—

rx_gptw.h

GPTW PWM Driver for RX72N Motor Control.

Production-grade PWM driver for the RX72N General PWM Timer (GPTW) peripheral, optimized for brushed DC motor control with H-bridge drivers. This module provides high-resolution 32-bit PWM generation with complementary outputs, deadtime insertion, and phase-staggered operation for 4-motor systems.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_check.h"`
- `"rx_err.h"`

rx_gptw_channel_t

GPTW Channel Identifiers for 4-Motor Control System.

Identifies the four GPTW timer channels used for motor PWM generation. Each channel controls one motor through complementary A/B outputs connected to a DRV8263H H-bridge driver.

Value	Name	Description
= 0	<code>k_gptw_channel_0</code>	GPTW0 channel for Motor 0 (Front-Left). Register base: 0x000C2100. Outputs: PE5/GTIOC0A, PE2/GTIOC0B. Phase offset: 0deg in staggered mode
= 1	<code>k_gptw_channel_1</code>	GPTW1 channel for Motor 1 (Front-Right). Register base: 0x000C2180. Outputs: PE4/GTIOC1A, PE1/GTIOC1B. Phase offset: 90deg in staggered mode
= 2	<code>k_gptw_channel_2</code>	GPTW2 channel for Motor 2 (Rear-Left). Register base: 0x000C2200. Outputs: PE3/GTIOC2A, PE0/GTIOC2B. Phase offset: 180deg in staggered mode
= 3	<code>k_gptw_channel_3</code>	GPTW3 channel for Motor 3 (Rear-Right). Register base: 0x000C2280. Outputs: PE7/GTIOC3A, PE6/GTIOC3B. Phase offset: 270deg in staggered mode

—

rx_gptw_output_t

GPTW Output Channel Selection for Complementary PWM.

Selects between the two outputs (A and B) available on each GPTW channel. Each output has an independent duty cycle and is controlled separately.

Value	Name	Description
-------	------	-------------

= 0	k_gptw_output_a	GTIOCA output pin. Connected to DRV8263H IN2 (direction) in IN2/IN1 mode. Controls motor direction: HIGH=forward, LOW=reverse
= 1	k_gptw_output_b	GTIOCB output pin. Connected to DRV8263H IN1 (PWM) in IN2/IN1 mode. PWM duty cycle controls motor speed (0-100%)

—

rx_gptw_wave_mode_t

GPTW PWM Waveform Mode Selection.

Selects the PWM waveform generation mode. The mode determines counter behavior, output alignment, and duty cycle update timing. Motor control typically uses sawtooth (edge-aligned) or triangle (center-aligned) modes.

Value	Name	Description
= 0	k_gptw_wave_saw_pwm	Sawtooth-wave PWM (edge-aligned). Counter: 0->Period->reset. Single update point at trough. Best for simple PWM applications. GTCR.MD = 0b000
= 1	k_gptw_wave_tri_pwm1	Triangle-wave PWM mode 1 (center-aligned). Counter: 0->Period->0. Update at trough only. Best for motor control with standard update rate. GTCR.MD = 0b001
= 2	k_gptw_wave_tri_pwm2	Triangle-wave PWM mode 2 (center-aligned). Counter: 0->Period->0. Update at both crest and trough. Best for motor control requiring fast duty updates. GTCR.MD = 0b010
= 3	k_gptw_wave_tri_pwm3	Triangle-wave PWM mode 3 (center-aligned). Counter: 0->Period->0. 64-bit buffer transfer at trough. Best for high-precision motor control. GTCR.MD = 0b011

—

rx_gptw_duty_calc_constants_t

Named constants for duty cycle percentage calculations.

Provides named numerators and a common denominator for computing raw duty counts as integer fractions of the period. Using named constants instead of magic literals satisfies NASA Power of 10 Rule 8 (no magic numbers).

All numerator constants must be $\leq$ k_gptw_duty_pct_denom (non-negative)

k_gptw_duty_pct_denom must be non-zero (denominator for percentage calculation)

Value	Name	Description
= 50	k_gptw_duty_50_pct_num	Numerator for 50% duty cycle calculation
= 75	k_gptw_duty_75_pct_num	Numerator for 75% duty cycle calculation
= 100	k_gptw_duty_pct_denom	Denominator for percentage duty calculations

See also: rx_gptw_set_duty_raw() Uses these constants in documentation examples,
rx_gptw_get_period() Provides the period value for the calculation

Since Version 1.0.0

—

rx_gptw_channel_id

```
static rx_gptw_channel_id_t rx_gptw_channel_id(rx_gptw_channel_t value)
```

Helper to construct a GPTW channel wrapper.

Provides type-safe wrapper construction with NASA Power of 10 Rule 5 validation (minimum 2 checks per function):

- Pre-condition: Input value is within valid channel range [0-3]
- Post-condition: Wrapper correctly stores the input value

Parameters:

Direction	Name	Description
in	value	GPTW channel enum value (k_gptw_channel_0 to k_gptw_channel_3)

Returns: Wrapped channel ID for type-safe function calls

Pre: value must be a valid rx_gptw_channel_t in range k_gptw_channel_0, k_gptw_channel_3

Pre: Caller must pass a compile-time known or previously validated channel value

Post: Returned wrapper .value field equals the input value

Post: Wrapper is immediately usable in any API accepting rx_gptw_channel_id_t

Note: Reentrant and lock-free; contains no shared mutable state] **See also:**

rx_gptw_output_id() Companion wrapper constructor for output parameter,
rx_gptw_channel_id_t Wrapper type constructed by this function

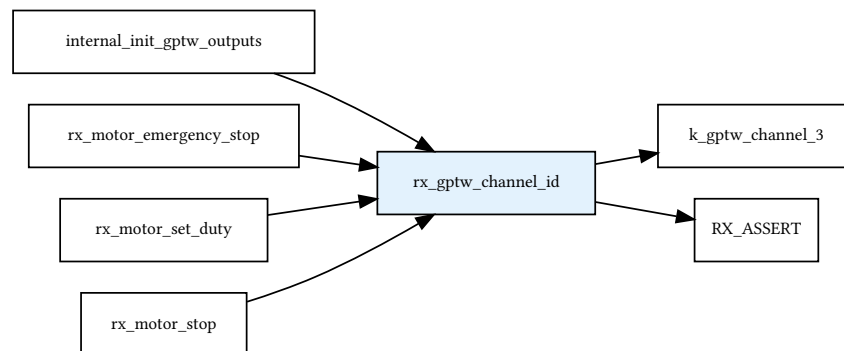


Figure 667: Call graph for rx_gptw_channel_id

Defined in libs/rx_hal/inc/rx_gptw.h:565

Since Version 1.0.0

—

rx_gptw_output_id

```
static rx_gptw_output_id_t rx_gptw_output_id(rx_gptw_output_t value)
```

Helper to construct a GPTW output wrapper.

Provides type-safe wrapper construction with NASA Power of 10 Rule 5 validation (minimum 2 checks per function):

- Pre-condition: Input value is within valid output range [A-B]
- Post-condition: Wrapper correctly stores the input value

Parameters:

Direction	Name	Description
in	value	GPTW output enum value (k_gptw_output_a or k_gptw_output_b)

Returns: Wrapped output ID for type-safe function calls

Pre: value must be a valid rx_gptw_output_t (k_gptw_output_a or k_gptw_output_b)

Pre: Caller must pass a compile-time known or previously validated output value

Post: Returned wrapper .value field equals the input value

Post: Wrapper is immediately usable in any API accepting rx_gptw_output_id_t

Note: Reentrant and lock-free; contains no shared mutable state] **See also:**

rx_gptw_channel_id() Companion wrapper constructor for channel parameter,
rx_gptw_output_id_t Wrapper type constructed by this function

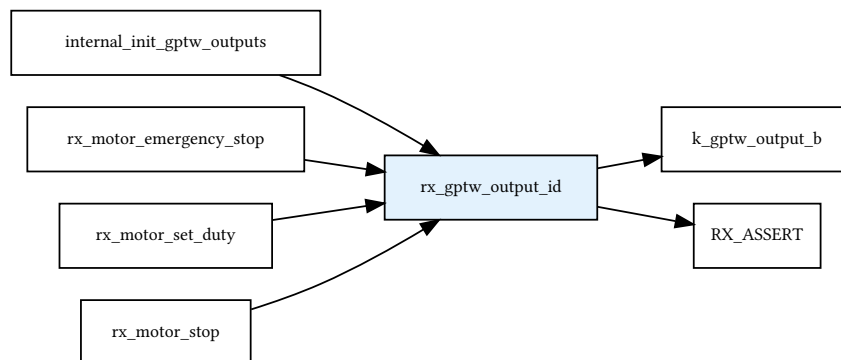


Figure 668: Call graph for rx_gptw_output_id

Defined in `libs/rx_hal/inc/rx_gptw.h:603`

Since Version 1.0.0

—

rx_gptw_init_all_staggered

```
rx_err_t rx_gptw_init_all_staggered(const rx_gptw_config_t *config)
```

Initialize all 4 GPTW channels with 90 degree phase staggering.

Configures GPTW0-3 for synchronized PWM operation with phase offsets that spread current draw across the PWM period. This is the recommended initialization function for 4-motor systems as it reduces peak current and EMI compared to synchronized PWM.

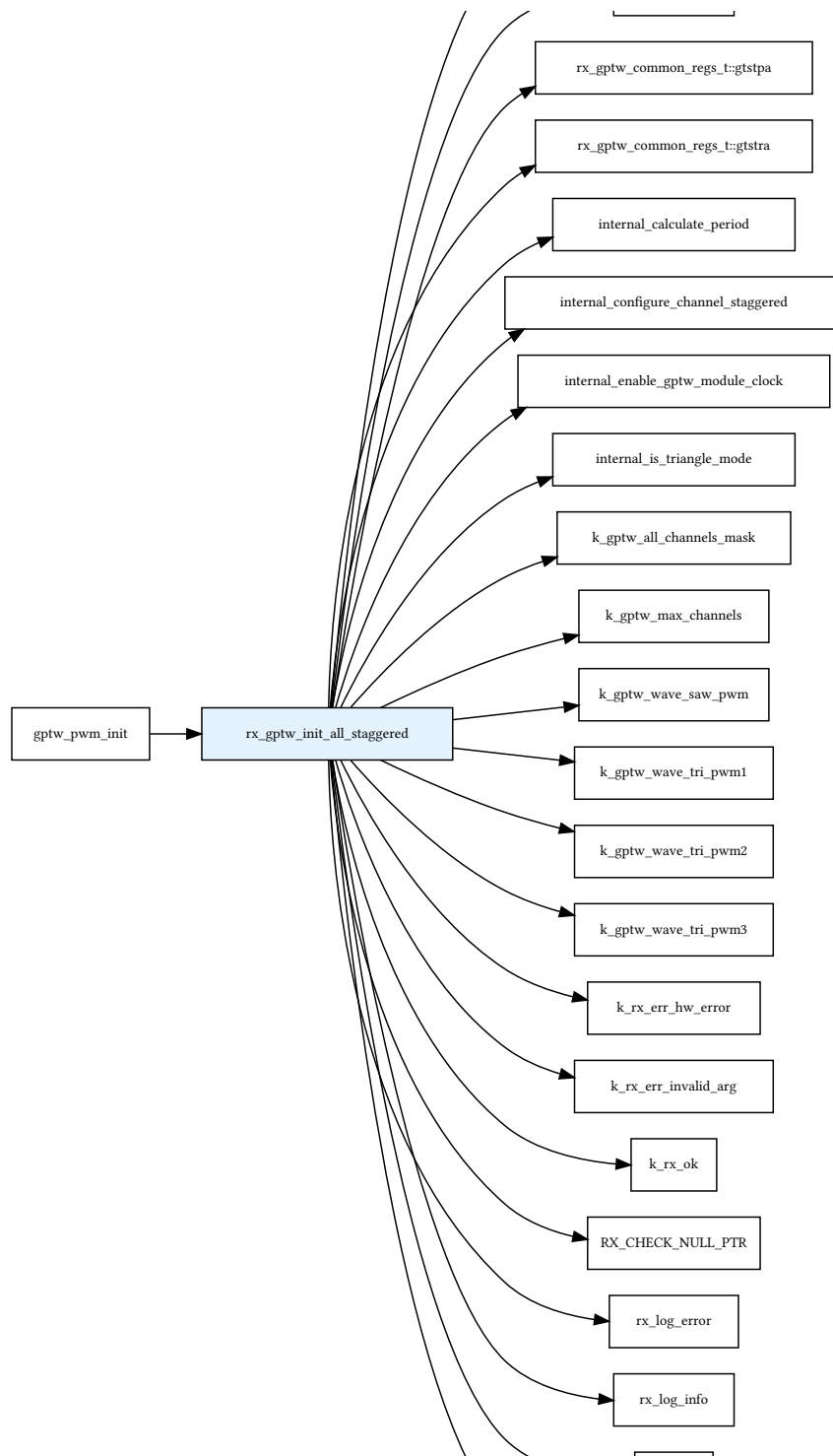


Figure 669: Call graph for rx_gptw_init_all_staggered

Defined in `libs/rx_hal/inc/rx_gptw.h:953`

—

`rx_gptw_init_pwm`

```
rx_err_t rx_gptw_init_pwm(rx_gptw_channel_t channel, const rx_gptw_config_t
*config)
```

Initialize a single GPTW channel for PWM output.

Configures the specified GPTW channel for PWM mode (sawtooth or triangle wave). Automatically configures MPC for the pin alternate functions, sets up dead-time insertion, and starts PWM generation at 0% duty cycle. The GTCNT counter is reset to zero during initialization.

Initialize a single GPTW channel for PWM output.

Configures one GPTW channel for PWM generation. This function performs complete hardware setup including module clock enable, MPC configuration, port pin setup, timer configuration, and automatic timer start. For 4-motor systems, prefer `rx_gptw_init_all_staggered()` instead.

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)
in	config	Pointer to PWM configuration struct (must be non-null)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, channel initialized and timer running
<code>k_rx_err_invalid_arg</code>	Channel out of range or config is null
<code>k_rx_err_invalid_state</code>	GPTW module not enabled in MSTPCR

Pre: GPTW module clock must be enabled

Pre: config must point to a valid `rx_gptw_config_t`

Post: Channel timer running at configured frequency with 0% duty

Post: GTCNT reset to zero (affects phase staggering if called per-channel)

Note: Not thread-safe. Call from single-threaded init context.

Warning: Resets GTCNT to 0. If phase staggering is required, use `rx_gptw_init_all_staggered()` instead, or configure phase offsets after all channels are initialized.] **See also:**
`rx_gptw_init_all_staggered()` Phase-staggered multi-channel init, `rx_gptw_deinit()`
 Release channel resources

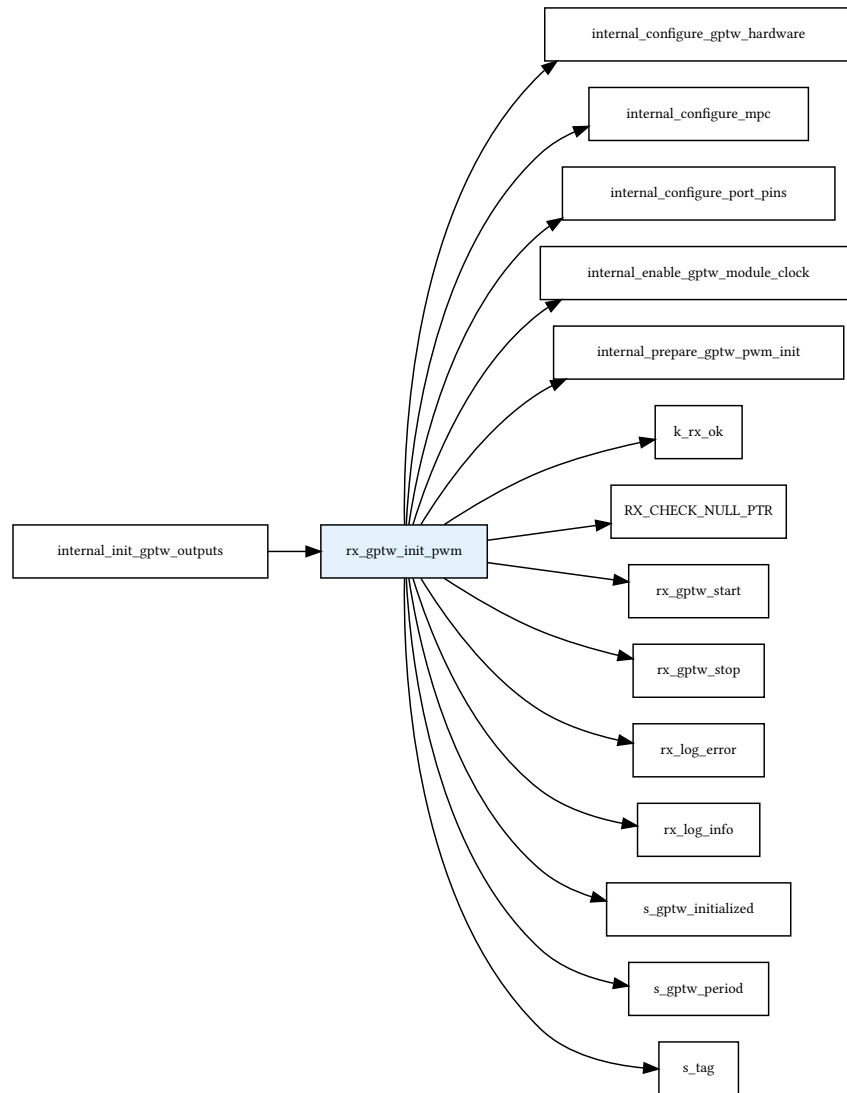


Figure 670: Call graph for rx_gptw_init_pwm

Defined in `libs/rx_hal/inc/rx_gptw.h:987`

Since Version 1.0.0

—

rx_gptw_set_duty

```
rx_err_t rx_gptw_set_duty(rx_gptw_channel_id_t channel, rx_gptw_output_id_t
output, float duty_percent)
```

Set PWM duty cycle (floating-point percentage).

Updates the duty cycle for the specified GPTW channel and output. The new duty cycle is written to the compare register buffer and takes effect on the next PWM period boundary, ensuring glitch-free operation.

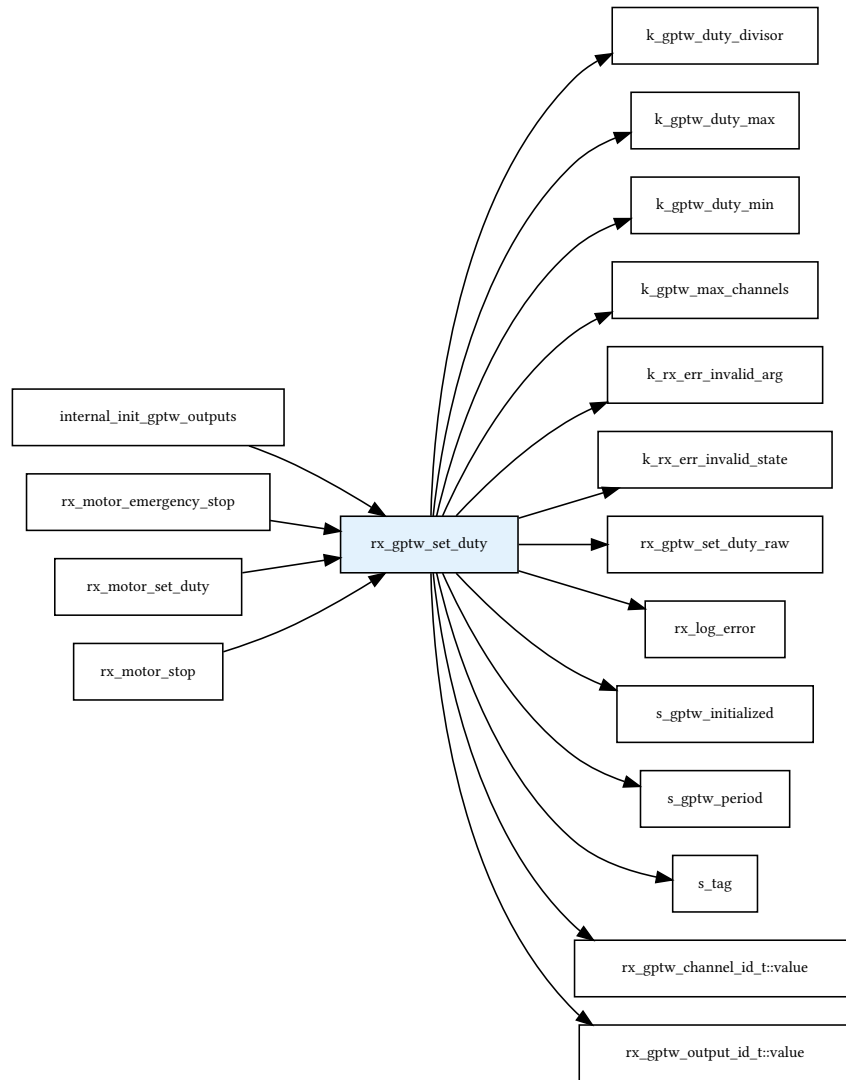


Figure 671: Call graph for `rx_gptw_set_duty`

Defined in `libs/rx_hal/inc/rx_gptw.h:1103`

`rx_gptw_set_duty_raw`

```

rx_err_t rx_gptw_set_duty_raw(rx_gptw_channel_t channel, rx_gptw_output_t output,
uint32_t duty_count)

```

Set PWM duty cycle using raw counter value.

Updates duty cycle using a raw counter value for efficiency in tight control loops where floating-point overhead should be avoided. The count is written to the compare register buffer and takes effect on the next PWM period boundary, ensuring glitch-free operation.

Set PWM duty cycle using raw counter value.

Directly writes duty_count to the compare register (GTCCRA or GTCCRB). Buffer mode (enabled during init via GTBER) ensures the update takes effect on the next PWM period boundary without glitches. Values exceeding the period are clamped to period (100% duty).

Parameters:

Direction	Name	Description
in	channel	GPTW channel (valid: 0-3)
in	output	Output channel (k_gptw_output_a or k_gptw_output_b)
in	duty_count	Duty cycle count (valid range: 0 to period_count)
in	channel	GPTW channel (k_gptw_channel_0 to k_gptw_channel_3)
in	output	Output to update (k_gptw_output_a or k_gptw_output_b)
in	duty_count	Raw count [0, period]; values > period clamped to period

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, duty cycle updated
k_rx_err_invalid_arg	Channel, output, or count out of range
k_rx_err_invalid_state	Channel not initialized via rx_gptw_init_pwm()
k_rx_ok	Duty count written successfully
k_rx_err_invalid_state	Channel out of range or not initialized
k_rx_err_invalid_arg	Could not resolve register base or invalid output

Pre: Channel must be initialized via rx_gptw_init_pwm()

Pre: duty_count must not exceed the period count for the channel

Pre: Channel must be initialized via rx_gptw_init_*)()

Pre: duty_count should be in 0, period for meaningful duty; higher values clamped

Post: Compare register buffer updated with new duty count

Post: Active register updated on next period boundary

Post: GTCCRA or GTCCRB written with clamped duty_count

Post: Change takes effect at next PWM period boundary

Note: Thread-safe: single 32-bit register write is inherently atomic on RX72N

Note: Not thread-safe: concurrent writes to the same register cause undefined behavior

Note: Values exceeding period are silently clamped to period

Performance:: Constant time: single 32-bit register write after validation.

Example::

```
uint32_tperiod=0; rx_err_t err=rx_gptw_get_period(k_gptw_channel_0,&period);
if(err==k_rx_ok){ constuint32_t duty_count=(period*k_gptw_duty_75_pct_num)/
k_gptw_duty_pct_denom;
err=rx_gptw_set_duty_raw(k_gptw_channel_0,k_gptw_output_b,duty_count); }
```

Register Access:: Output A: Writes to GTCCRA Output B: Writes to GTCCRB

Example:

```
uint32_tperiod;
rx_gptw_get_period(k_gptw_channel_0,&period);
rx_gptw_set_duty_raw(k_gptw_channel_0,k_gptw_output_a,period/2U);//50%
```

See also: rx_gptw_set_duty() Percentage-based version with float conversion,
rx_gptw_get_period() Query period count for duty calculation, rx_gptw_set_duty() Set
duty using floating-point percentage, rx_gptw_get_period() Retrieve period count for
duty calculation

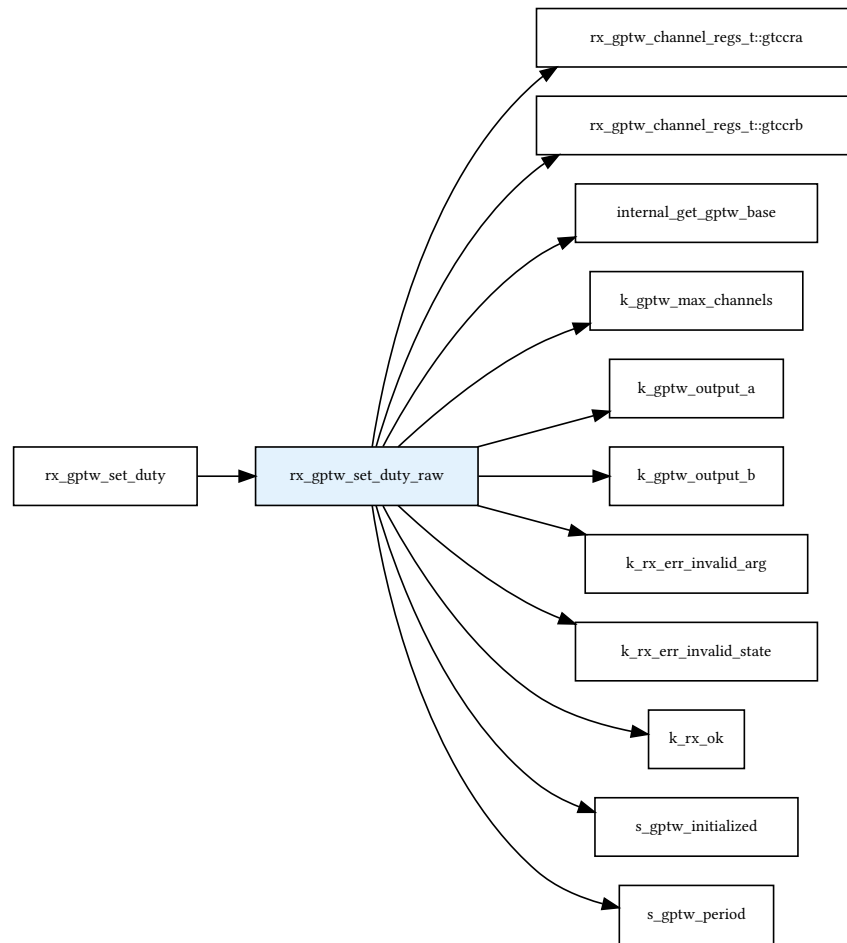


Figure 672: Call graph for rx_gptw_set_duty_raw

Defined in `libs/rx_hal/inc/rx_gptw.h:1150`

Since Version 1.0.0

—

rx_gptw_get_duty

```
rx_err_t rx_gptw_get_duty(rx_gptw_channel_t channel, rx_gptw_output_t output,
float *duty_percent)
```

Get current duty cycle as a floating-point percentage.

Reads the current compare register value for the specified output and converts it to a percentage of the period count. The value reflects the most recently committed duty cycle (may differ from the buffered value pending a period boundary update).

Get current duty cycle as a floating-point percentage.

Reads the current compare register value (GTCCRA or GTCCRB) from hardware and converts it to a percentage using the cached period from `s_gptw_period[]`.

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)
in	output	Output channel (<code>k_gptw_output_a</code> or <code>k_gptw_output_b</code>)
out	duty_percent	Pointer to store duty cycle in percent [0.0, 100.0]
in	channel	GPTW channel (<code>k_gptw_channel_0</code> to <code>k_gptw_channel_3</code>)
in	output	Output to read (<code>k_gptw_output_a</code> or <code>k_gptw_output_b</code>)
out	duty_percent	Pointer to store percentage result [0.0, 100.0] Must be non-NULL

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, <code>duty_percent</code> written
<code>k_rx_err_null_ptr</code>	<code>duty_percent</code> is nullptr
<code>k_rx_err_invalid_arg</code>	Channel or output out of range
<code>k_rx_err_invalid_state</code>	Channel not initialized
<code>k_rx_ok</code>	Percentage written to <code>*duty_percent</code>
<code>k_rx_err_null_ptr</code>	<code>duty_percent</code> is nullptr
<code>k_rx_err_invalid_state</code>	Channel out of range or not initialized
<code>k_rx_err_invalid_arg</code>	Invalid output or register pointer unresolvable

Pre: Channel must be initialized via `rx_gptw_init_pwm()`

Pre: `duty_percent` must be a valid non-null pointer

Pre: Channel must be initialized via `rx_gptw_init_*`()

Pre: `duty_percent` must point to valid float storage

Post: `duty_percent` contains current duty cycle percentage

Post: No hardware state modified (read-only operation)

Post: `*duty_percent` contains duty cycle in 0.0, 100.0 on success

Post: Hardware registers unchanged

Note: Not thread-safe. Caller must provide synchronization.

Note: Returns actual register value; may differ from the last set value if buffer transfer has not yet occurred at period boundary

Note: Not thread-safe: concurrent rx_gptw_set_duty calls may cause torn reads

Example:: floatduty=0.0F;

```
rx_err_t err=rx_gptw_get_duty(k_gptw_channel_0,k_gptw_output_a,&duty); if(err==k_rx_ok){ }
```

Algorithm:: duty_percent = {duty_count x 100}/{period

Example:

```
floatduty;
```

```
rx_err_t err=rx_gptw_get_duty(k_gptw_channel_0,k_gptw_output_a,&duty);
```

See also: rx_gptw_set_duty() Set duty cycle by percentage, rx_gptw_get_period() Get period count for manual calculation, rx_gptw_set_duty() Set duty using floating-point percentage, rx_gptw_get_period() Retrieve period count

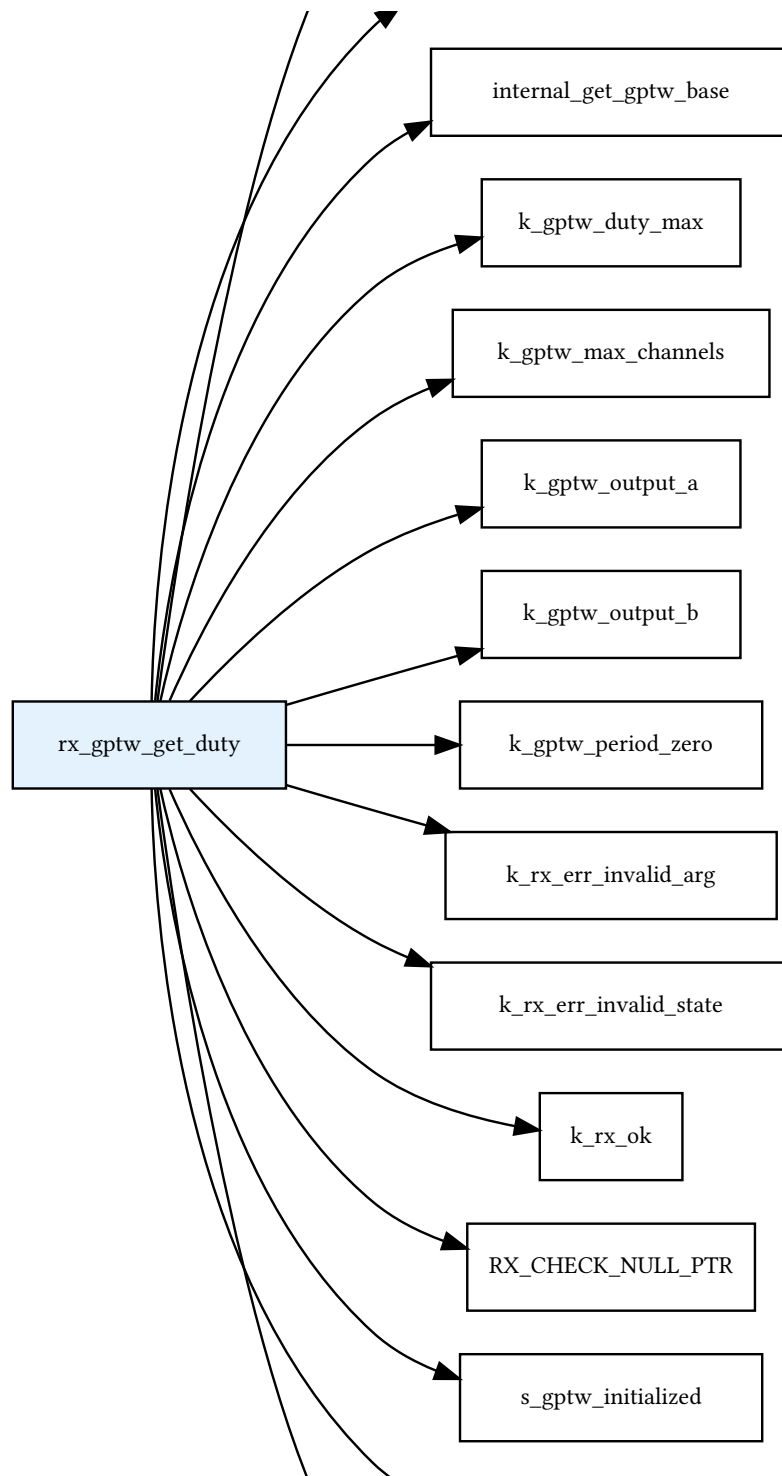


Figure 673: Call graph for rx_gptw_get_duty

Defined in `libs/rx_hal/inc/rx_gptw.h:1193`

Since Version 1.0.0

—

rx_gptw_get_period

```
rx_err_t rx_gptw_get_period(rx_gptw_channel_t channel, uint32_t *period_count)
```

Get the 32-bit PWM period count for a GPTW channel.

Reads the GTPR (General Timer Period Register) value for the specified channel. This value determines the PWM frequency and is needed to calculate raw duty counts: $\text{duty_count} = (\text{duty_percent} / 100) * \text{period_count}$.

Get the 32-bit PWM period count for a GPTW channel.

Implementation of `rx_gptw_get_period()` - see `rx_gptw.h` for complete documentation.

Returns the cached period value from `s_gptw_period[]`, which matches the GTPR register value set during initialization.

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)
out	period_count	Pointer to store 32-bit period count
in	channel	GPTW channel to query Valid range: <code>k_gptw_channel_0</code> to <code>k_gptw_channel_3</code>
out	period_count	Pointer to store period count value Must be non-NULL

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, period_count written
<code>k_rx_err_null_ptr</code>	period_count is nullptr
<code>k_rx_err_invalid_arg</code>	Channel out of range
<code>k_rx_err_invalid_state</code>	Channel not initialized
<code>k_rx_ok</code>	Success, period_count contains valid value
<code>k_rx_err_null_ptr</code>	period_count is nullptr
<code>k_rx_err_invalid_state</code>	Channel not initialized

Pre: Channel must be initialized via `rx_gptw_init_pwm()`

Pre: period_count must be a valid non-null pointer

Pre: Channel initialized via `rx_gptw_init_*`

Pre: period_count points to valid uint32_t storage

Post: period_count contains the GTPR register value

Post: No hardware state modified (read-only operation)

Post: *period_count contains period value on success

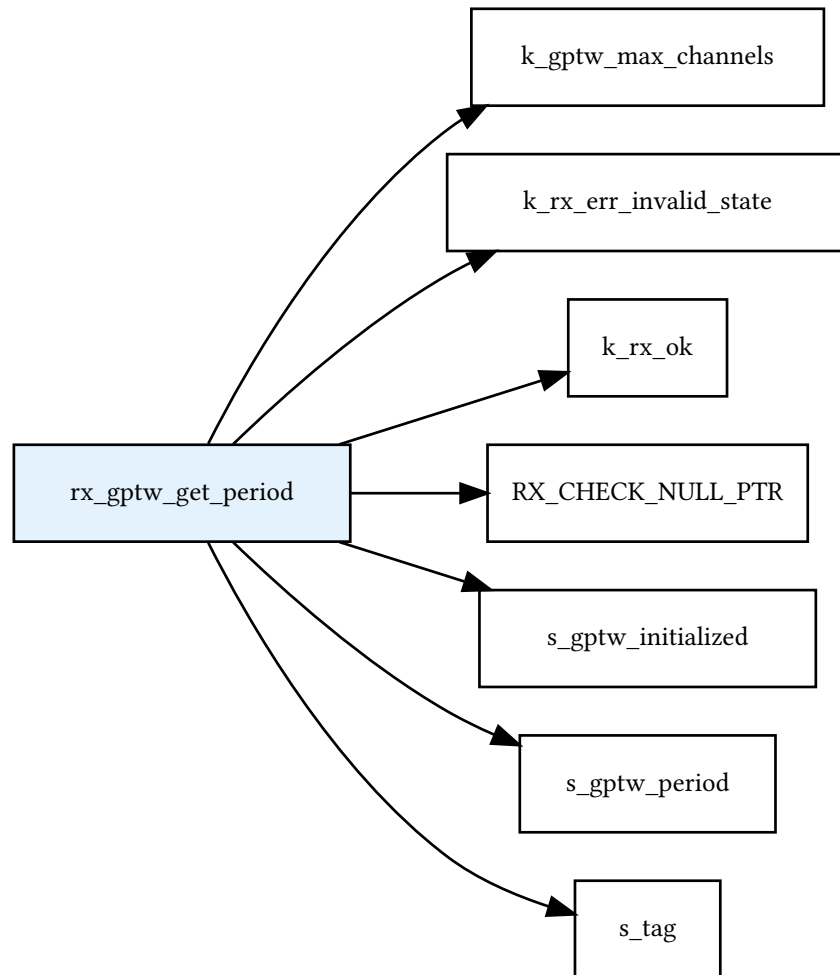
Note: Not thread-safe. Caller must provide synchronization.

Note: Thread-safe: Reads from cached value, not hardware

Note: Useful for calculating raw duty counts from percentages

Example:: uint32_t period=0; rx_err_t err=rx_gptw_get_period(k_gptw_channel_0,&period);
if(err!=k_rx_ok){ uint32_t duty_count=(period*k_gptw_duty_50_pct_num)/
k_gptw_duty_pct_denom; }

See also: rx_gptw_set_duty_raw() Uses period_count for raw duty calculation, rx_gptw.h
Complete API documentation

Figure 674: Call graph for `rx_gptw_get_period`

Defined in `libs/rx_hal/inc/rx_gptw.h:1233`

Since Version 1.0.0

—

rx_gptw_enable_output

```
rx_err_t rx_gptw_enable_output(rx_gptw_channel_t channel, rx_gptw_output_t
output, bool enable)
```

Enable or disable a PWM output pin.

Controls the GTONCR (General Timer Output Negation Control Register) bits that gate the PWM signal to the physical pin. When disabled, the pin is driven to its inactive level (determined by polarity configuration).

Enable or disable a PWM output pin.

Implementation of `rx_gptw_enable_output()` - see `rx_gptw.h` for complete documentation.

Controls the Output Enable bits (OAE/OBE) in the GTIOR register. When disabled, the corresponding output pin is held in its initial state (typically LOW).

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)
in	output	Output channel (<code>k_gptw_output_a</code> or <code>k_gptw_output_b</code>)
in	enable	true to drive PWM on pin, false to hold pin inactive

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, output enable state updated
<code>k_rx_err_invalid_arg</code>	Channel or output out of range
<code>k_rx_err_invalid_state</code>	Channel not initialized

Pre: Channel must be initialized via `rx_gptw_init_pwm()`

Pre: Output must be `k_gptw_output_a` or `k_gptw_output_b`

Post: GTONCR bit updated for the specified output

Post: Pin either drives PWM signal or holds inactive level

Note: Not thread-safe. Caller must provide synchronization.

Example:: `rx_err_t err=rx_gptw_enable_output(k_gptw_channel_0,k_gptw_output_a,true); if(err!=k_rx_ok){ }`

See also: `rx_gptw_start()` Start timer (separate from output enable), `rx_gptw_stop()` Stop timer (separate from output enable)

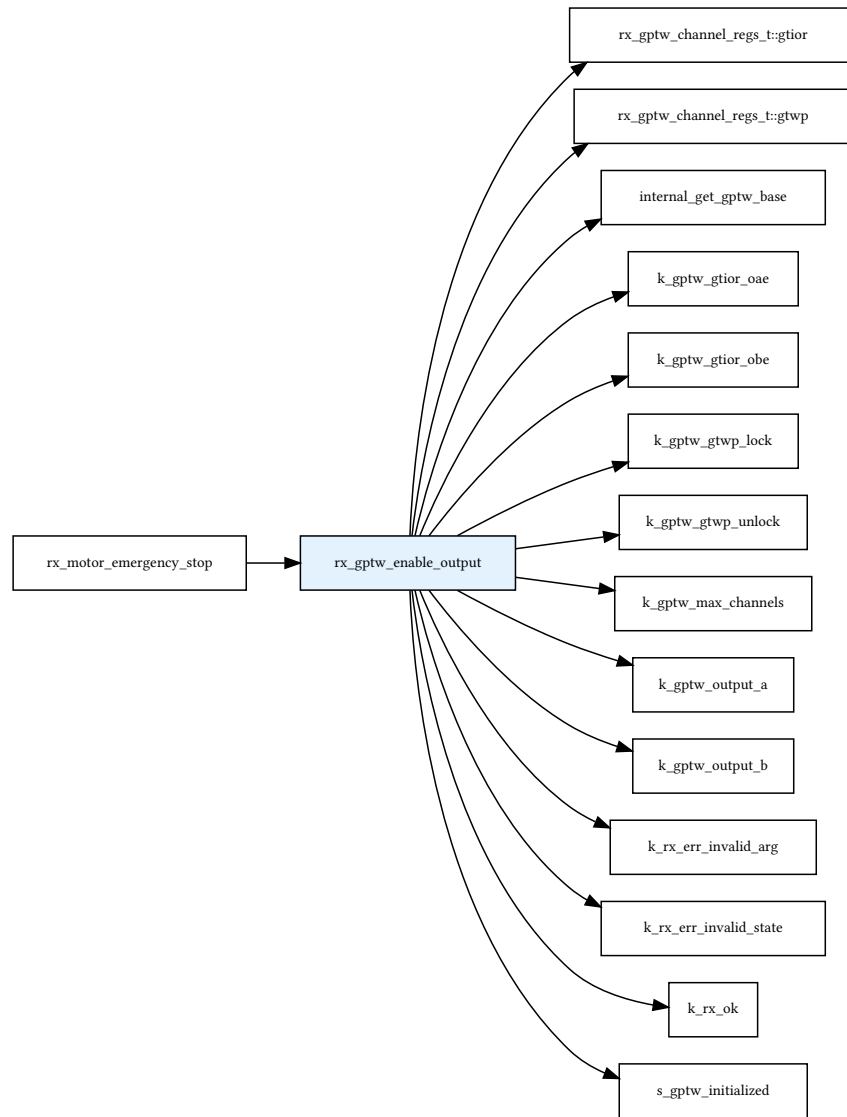


Figure 675: Call graph for rx_gptw_enable_output

Defined in `libs/rx_hal/inc/rx_gptw.h:1274`

Since Version 1.0.0

—

rx_gptw_start

```
rx_err_t rx_gptw_start(rx_gptw_channel_t channel)
```

Start the GPTW timer counter for PWM generation.

Sets the GTCR.CST bit to begin counting. The timer is automatically started during `rx_gptw_init_pwm()`, but can be stopped via `rx_gptw_stop()` and restarted with this function. Counting resumes from the current GTCNT value (not reset to zero).

Start the GPTW timer counter for PWM generation.

Implementation of `rx_gptw_start()` - see `rx_gptw.h` for complete documentation.

Sets the Count Start (CST) bit in GTCR to begin counter operation. The timer will count according to the configured waveform mode (sawtooth or triangle) and generate PWM output on the enabled pins.

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, timer started
<code>k_rx_err_invalid_arg</code>	Channel out of range

Pre: Channel must be initialized via `rx_gptw_init_pwm()`

Pre: Channel must currently be stopped (safe to call if already running)

Post: Timer counter incrementing at peripheral clock rate

Post: PWM outputs active (if enabled via `rx_gptw_enable_output()`)

Note: Not thread-safe. Caller must provide synchronization.

Example:: `rx_err_t err=rx_gptw_start(k_gptw_channel_0); if(err!=k_rx_ok){ }`

See also: `rx_gptw_stop()` Stop the timer counter, `rx_gptw_init_pwm()` Initializes and starts timer automatically

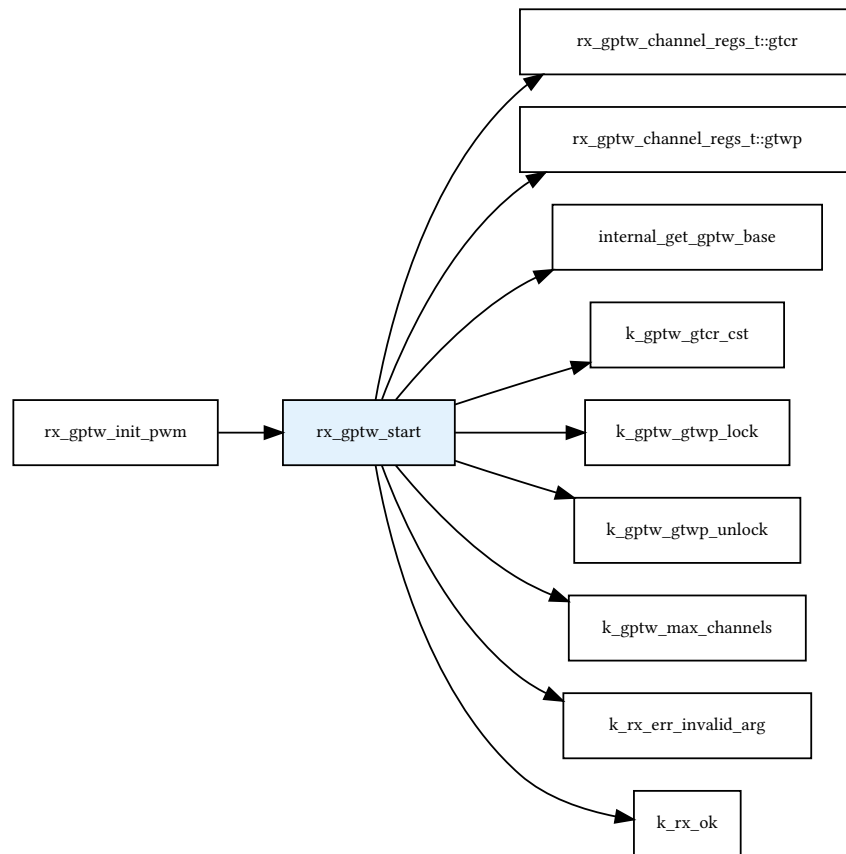


Figure 676: Call graph for `rx_gptw_start`

Defined in `libs/rx_hal/inc/rx_gptw.h:1312`

Since Version 1.0.0

—

rx_gptw_stop

```
rx_err_t rx_gptw_stop(rx_gptw_channel_t channel)
```

Stop the GPTW timer counter.

Clears the GTCR.CST bit to halt counting. PWM outputs hold their last state when the timer stops. The GTCNT value is preserved and counting resumes from that value when `rx_gptw_start()` is called.

Stop the GPTW timer counter.

Implementation of `rx_gptw_stop()` - see `rx_gptw.h` for complete documentation.

Clears the Count Start (CST) bit in GTCR to halt counter operation. The timer counter value (GTCNT) is preserved and PWM outputs hold their current state.

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, timer stopped
k_rx_err_invalid_arg	Channel out of range

Pre: Channel must be initialized via rx_gptw_init_pwm()**Pre:** Channel should be running (safe to call if already stopped)**Post:** Timer counter halted at current GTCNT value**Post:** PWM outputs frozen at last driven state**Note:** Not thread-safe. Caller must provide synchronization.**Warning:** Outputs hold last state; explicitly set duty to 0 before stopping if low output is required.**Example::** rx_gptw_set_duty(rx_gptw_channel_id(k_gptw_channel_0),
rx_gptw_output_id(k_gptw_output_b),0.0F); rx_err_t err=rx_gptw_stop(k_gptw_channel_0);**See also:** rx_gptw_start() Resume timer counter, rx_gptw_deinit() Full shutdown including output disable

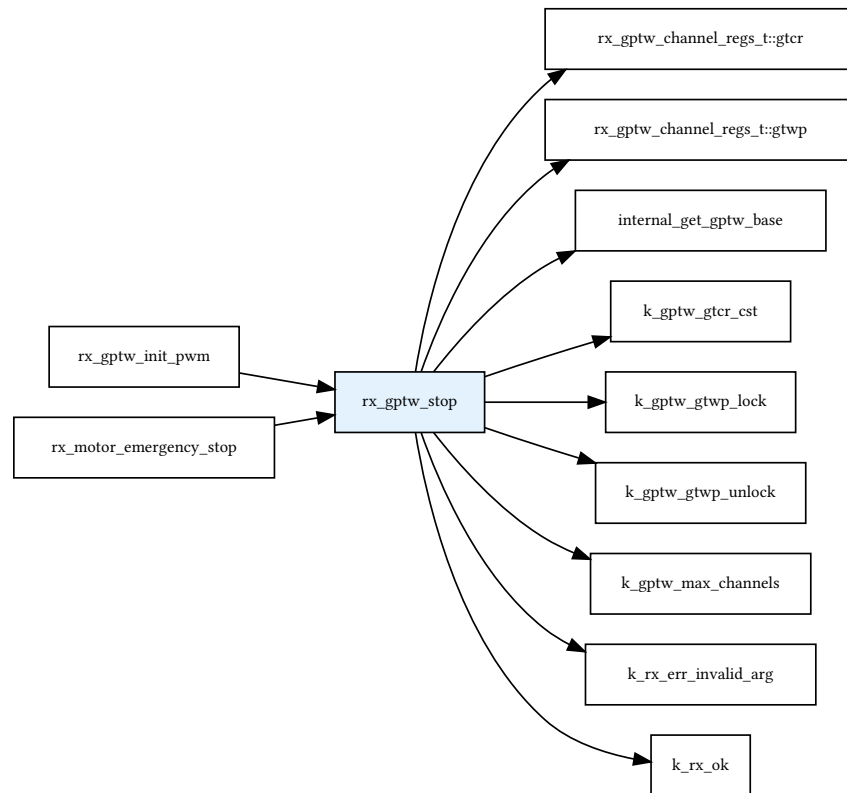


Figure 677: Call graph for rx_gptw_stop

Defined in `libs/rx_hal/inc/rx_gptw.h:1350`

Since Version 1.0.0

—

rx_gptw_deinit

```
rx_err_t rx_gptw_deinit(rx_gptw_channel_t channel)
```

Deinitialize a GPTW channel and release all resources.

Performs a full shutdown sequence: stops the timer counter, disables both PWM outputs, resets the compare and period registers, and marks the channel as uninitialized. After deinit, the channel must be re-initialized via `rx_gptw_init_pwm()` before use.

Deinitialize a GPTW channel and release all resources.

Stops the timer, disables both PWM outputs (A and B), and resets the counter to zero. Call only when motor control is no longer active.

Write protection re-locked (GTWP) on all exit paths

Parameters:

Direction	Name	Description
in	channel	GPTW channel number (valid: 0-3)
in	channel	GPTW channel (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, channel fully deinitialized
k_rx_err_invalid_arg	Channel out of range
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel number

Pre: Channel should be initialized (safe to call if already deinitialized)

Pre: Motor outputs should be in a safe state before deinit

Pre: Motor using this channel must be stopped before calling

Pre: channel must be in range k_gptw_channel_0, k_gptw_channel_3

Post: Timer stopped, outputs disabled, channel marked uninitialized

Post: All channel registers reset to default values

Post: Timer counter stopped (GTCR.CST = 0)

Post: Both outputs disabled (GTIOR.OAE = 0, GTIOR.OBE = 0)

Post: Counter register zeroed (GTCNT = k_gptw_period_zero)

Note: Not thread-safe. Caller must provide synchronization.

Note: Not thread-safe: modifies hardware registers directly

Note: Does not clear s_gptw_initialized or s_gptw_period

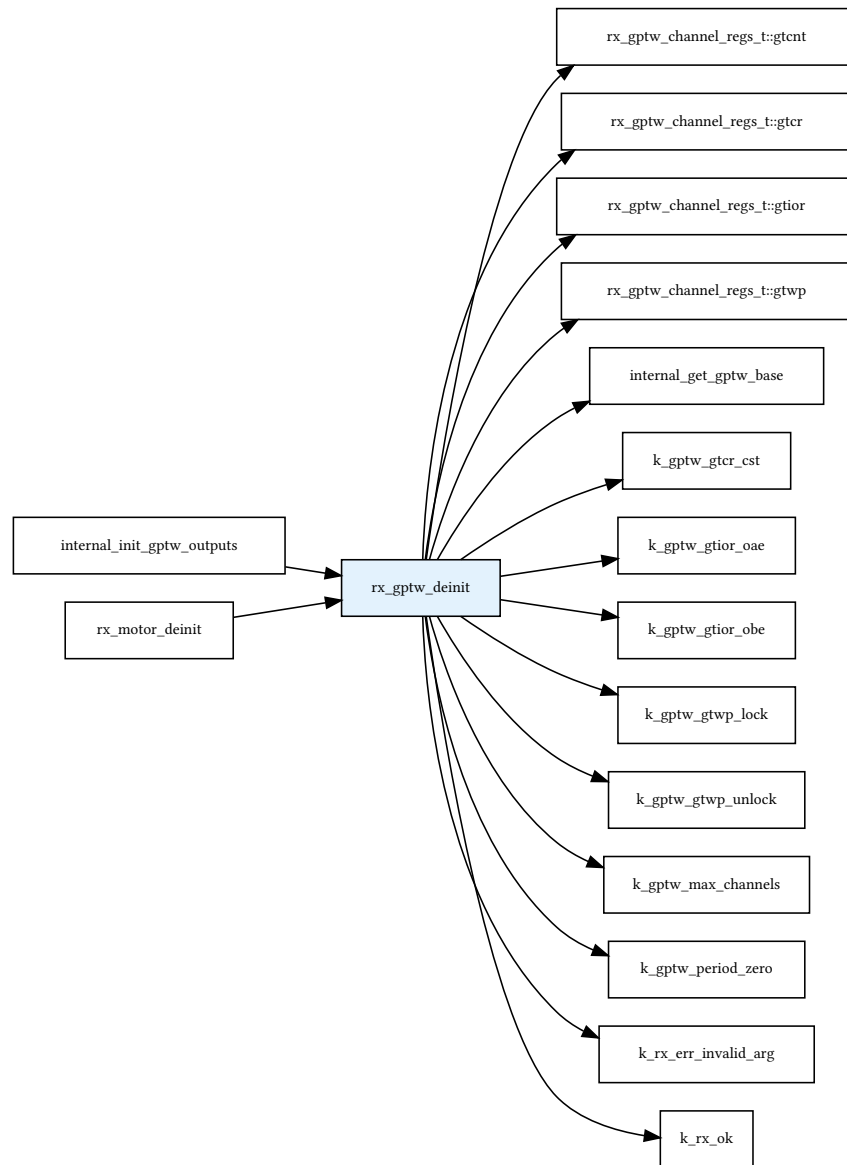
Warning: Ensure motor is at rest before calling to prevent abrupt stop

Example:: rx_err_t err = rx_gptw_deinit(k_gptw_channel_0); if(err != k_rx_ok){ }

Example:

```
rx_err_t err = rx_gptw_deinit(k_gptw_channel_0);
```

See also: `rx_gptw_init_pwm()` Re-initialize after deinit, `rx_gptw_stop()` Stop timer without full deinit, `rx_gptw_init_all()` Re-initialize after deinit, `rx_gptw_stop()` Stop without full teardown

Figure 678: Call graph for `rx_gptw_deinit`

Defined in `libs/rx_hal/inc/rx_gptw.h:1388`

Since Version 1.0.0

—

rx_hal_iwdt.h

Independent Watchdog Timer (IWDT) HAL Driver for RX72N.

Hardware Abstraction Layer for the RX72N Independent Watchdog Timer (IWDT) peripheral. This driver provides low-level register access and task-level monitoring for safety-critical system recovery.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

rx_iwdt_timeout_t

IWDT Timeout Period Options.

Pre-defined timeout periods for the IWDT. These values correspond to hardware divider settings that produce specific timeout durations from the 120 kHz IWDTCLK.

Value	Name	Description
= 128	k_iwdt_timeout_128ms	Minimum timeout (128ms). Use for high-frequency control loops requiring fast failure detection. TOPS=1 divider
= 512	k_iwdt_timeout_512ms	Short timeout (512ms). Use for normal control loops (10-50 Hz). Provides good responsiveness. TOPS=4 divider
= 1000	k_iwdt_timeout_1000ms	Default timeout (1 second). Recommended for typical applications. Good balance between detection speed and tolerance. TOPS 8
= 2048	k_iwdt_timeout_2048ms	Medium timeout (2 seconds). Use for low-frequency tasks that may have variable execution time. TOPS=16 divider
= 8192	k_iwdt_timeout_8192ms	Long timeout (8 seconds). Use for slow background operations or maintenance tasks. TOPS=64 divider
= 16384	k_iwdt_timeout_16384ms	Maximum timeout (16 seconds). Use only for initialization phases or very slow operations. TOPS=128 divider

—

rx_iwdt_reset_cause_t

IWDT Reset Cause Information.

Identifies the specific cause of an IWDT-triggered reset. This information is read from the IWDTSR status register and can be used for post-mortem diagnostics and logging.

Value	Name	Description
-------	------	-------------

= 0	k_iwdt_reset_none	No watchdog reset occurred. System reset was from power-on, external reset pin, or other source. IWDTSR.UNDFF=0, REFEF=0
= 1	k_iwdt_reset_underflow	Reset due to counter underflow. Counter reached zero before rx_hal_iwdt_feed() was called. Indicates task deadlock or infinite loop. IWDTSR.UNDFF=1
= 2	k_iwdt_reset_refresh_error	Reset due to invalid refresh. Either wrong refresh sequence (not 0x00, 0xFF) or refresh during window-prohibited period. IWDTSR.REFEF=1

—

rx_iwdt_limits_t

Maximum number of tasks that can be monitored.

Value	Name	Description
= 8	k_iwdt_max_tasks	Support up to 8 monitored tasks

—

rx_hal_iwdt_init

```
rx_err_t rx_hal_iwdt_init(uint32_t timeout_ms)
```

Initialize the Independent Watchdog Timer.

Configures IWDT with the specified timeout period. Once started, the watchdog cannot be stopped and must be fed periodically to prevent reset.

Configuration:

- Clock: 120 kHz internal IWDT-dedicated oscillator (IWDTCLK)
- Window mode: Disabled (refresh allowed anytime)
- Action: System reset on timeout
- Count stop in sleep: Disabled (continues counting)

Configures and starts the hardware IWDT with the specified timeout period. Once started, the watchdog cannot be stopped - periodic calls to rx_hal_iwdt_feed() are required to prevent system reset.

Parameters:

Direction	Name	Description
in	timeout_ms	Desired timeout in milliseconds. Actual timeout will be rounded up to nearest supported value (128, 512, 1024, 2048, 4096, 8192, or 16384 ms).
in	timeout_ms	Requested timeout period in milliseconds (1-16384) Actual timeout may be slightly longer (next available)

Returns: rx_err_t Error code

Value	Description
-------	-------------

k_rx_ok	IWDT initialized and running
k_rx_err_invalid_state	Already initialized
k_rx_err_invalid_arg	timeout_ms out of valid range
k_rx_err_hw_unmapped	Hardware registers not accessible

Pre: IWDT must not be already initialized

Pre: timeout_ms in range k_iwdt_timeout_min_ms, k_iwdt_timeout_max_ms

Pre: System startup, single-threaded context

Post: IWDT running with selected timeout period

Post: s_iwdt_initialized set to k_iwdt_initialized

Post: Periodic rx_hal_iwdt_feed() calls now required

Note: This function starts the watchdog - rx_hal_iwdt_feed() must be called within the timeout period to prevent reset.

Note: On RX72N, IWDT starts counting after the first refresh operation.

Note: On host builds (non-RX), skips hardware access but sets initialized flag

Note: Actual timeout >= requested (selected from lookup table)

Warning: Once initialized, IWDT cannot be disabled (requires hard reset)

Warning: Must call rx_hal_iwdt_feed() faster than timeout period] **Initialization Sequence:** *
 1. Validate not already initialized * 2. Validate timeout_ms within range * 3. Look up timeout configuration in table * 4. Configure IWDTCR (timeout period, clock divisor, window) * 5. Configure IWDTRCR (reset action - not NMI) * 6. Configure IWDTCSTPR (continue counting in sleep) * 7. Perform first refresh (starts the watchdog) * 8. Set initialized flag *

Point of No Return: * After step 7 (first refresh), the IWDT is running and CANNOT be stopped. *
 Any failure after this point leaves system in watchdog-protected state. *

Example:: rx_err_t err=rx_hal_iwdt_init(2000); if(err!=k_rx_ok){ while(1){ }
 while(1){ rx_hal_iwdt_feed(); tx_thread_sleep(100); }

See also: rx_hal_iwdt_feed() Reset watchdog counter, rx_hal_iwdt.h Public API and configuration details

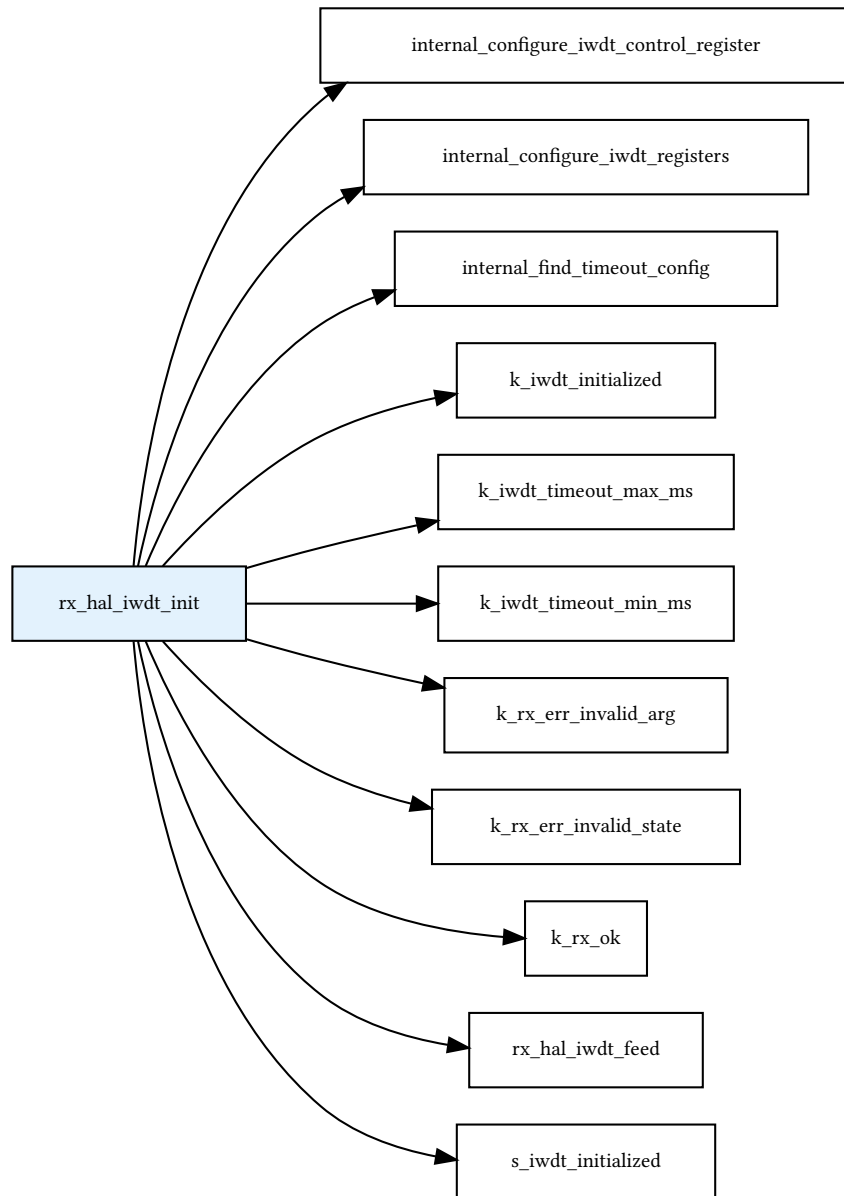


Figure 679: Call graph for `rx_hal_iwdt_init`

Defined in `libs/rx_hal/inc/rx_hal_iwdt.h:317`

Since Version 1.0.0

—

rx_hal_iwdt_feed

```
void rx_hal_iwdt_feed(void)
```

Feed (refresh) the watchdog timer.

Resets the watchdog counter to prevent timeout. This function must be called periodically, within the configured timeout period.

The refresh sequence writes 0x00 followed by 0xFF to the IWDTRR register. This sequence must complete atomically without interruption.

Example placement:

Feed (refresh) the watchdog timer.

Performs the IWDT refresh sequence by writing the magic values 0x00 then 0xFF to the IWDTRR register. This resets the down-counter to its initial value, preventing timeout and system reset.

Pre: IWDT must be initialized via rx_hal_iwdt_init()

Pre: Called at rate faster than configured timeout

Post: IWDT counter reset to initial value

Post: Interrupt state restored to previous value

Note: Call this function from your main control loop.

Note: Calling more frequently than needed has no negative effect.

Note: Do NOT call from interrupt handlers unless you understand the implications for your system's safety.

Note: Safe to call from ISRs (uses atomic disable/restore)

Note: On host builds, function is a no-op

Warning: Incomplete refresh sequence causes refresh error -> system reset

Warning: Missing calls to this function cause timeout -> system reset] **Refresh Sequence (Atomic Operation):** * +-----+ * | 1. Disable interrupts (save PSW) | * | 2. Write 0x00 to IWDTRR | * | 3. Write 0xFF to IWDTRR | * | 4. Restore interrupt state | * +-----+ * **CRITICAL:** Steps 2 and 3 must not be interrupted! *

Timing Requirements: * - Execution time: 2 us including interrupt disable/enable * - Must be called faster than configured timeout period * - Typical call rate: Every 100-500 ms depending on timeout *

Example:: while(1){ rx_err_t err=rx_hal_iwdt_check_tasks(); if(err==k_rx_ok){ rx_hal_iwdt_feed(); } tx_thread_sleep(100); }

Example:

```
//Motorcontrol task at 100Hz (10ms period)
//With 1000ms timeout, we have 100 chances to feed
void motor_control_task_entry(ULONG input){
while(1){
read_sensors();
update_pid();
set_motor_output();
rx_hal_iwdt_feed(); //Feed at end of each iteration
tx_thread_sleep(1); //10ms at 100Hz tick
}
}
```

See also: rx_hal_iwdt_init() Start the watchdog, rx_hal_iwdt_check_tasks() Verify task health before feeding

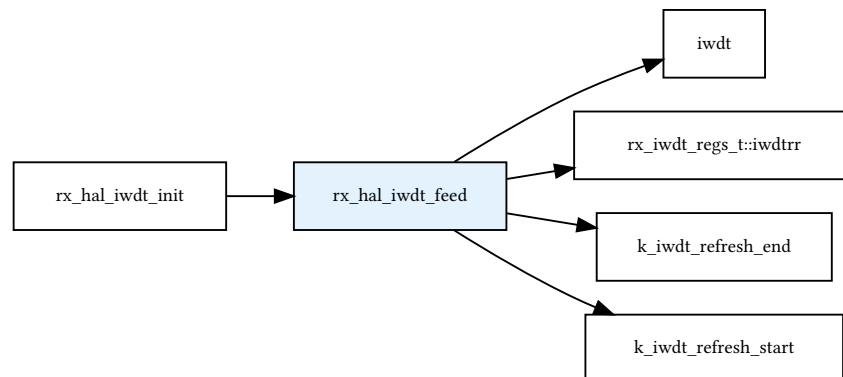


Figure 680: Call graph for rx_hal_iwdt_feed

Defined in libs/rx\hal/inc/rx\hal\iwdt.h:353

Since Version 1.0.0

—

rx_hal_iwdt_was_reset

```
bool rx_hal_iwdt_was_reset(void)
```

Check if last reset was caused by watchdog timeout.

Reads the IWDWT status register to determine if the previous system reset was triggered by the watchdog timer.

Check if last reset was caused by watchdog timeout.

Reads the IWDTSR Status Register (IWDTSR) to determine if the previous system reset was caused by watchdog timeout (underflow) or refresh error. This function should be called early in startup for diagnostics.

Returns: bool Reset cause indication

Value	Description
true	Last reset was caused by watchdog (underflow or refresh error)
false	Last reset was NOT caused by watchdog (power-on, external, etc.)

Pre: Can be called before rx_hal_iwdt_init()

Pre: Status flags preserved across reset (hardware feature)

Post: Status flags NOT cleared (call rx_hal_iwdt_clear_status() to clear)

Note: Call this early in startup (before clearing status flags) to log or handle watchdog-induced resets appropriately.

Note: On host builds, always returns false

Note: Call rx_hal_iwdt_get_reset_cause() for specific cause] **Status Register Flags:** * IWDTSR Bit 14 (UNDF) - Underflow Flag: * - 1: Counter reached 0 (timeout occurred) * - 0: No underflow
* IWDTSR Bit 15 (REFEF) - Refresh Error Flag: * - 1: Invalid refresh sequence detected * - 0: No refresh error *

Example:: if(rx_hal_iwdt_was_reset()){ uart_debug_puts("[WARN]Recoveredfromwatchdogreset!\n"); constchar*failed=rx_hal_iwdt_get_failed_task(); if(failed!=nullptr) { uart_debug_printf("Failedtask:%sn",failed); } }

See also: rx_hal_iwdt_get_reset_cause() Get specific reset cause,
rx_hal_iwdt_clear_status() Clear status flags after reading,
rx_hal_iwdt_get_failed_task() Get name of task that caused timeout

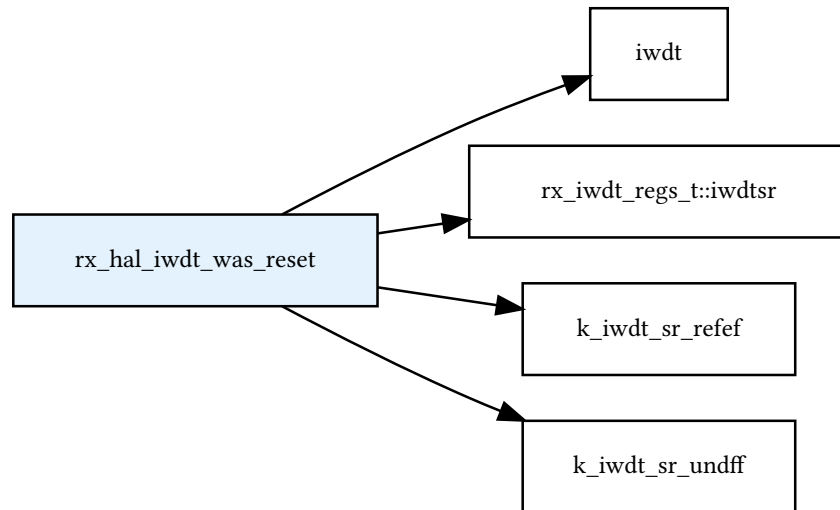


Figure 681: Call graph for rx_hal_iwdt_was_reset

_Defined in libs/rx_hal/inc/rx_hal_iwdt.h:372_

Since Version 1.0.0

—

rx_hal_iwdt_get_reset_cause

```
rx_iwdt_reset_cause_t rx_hal_iwdt_get_reset_cause(void)
```

Get detailed reset cause from IWDT.

Provides more detailed information about what caused a watchdog reset:

- Underflow: Counter reached zero (normal timeout)
- Refresh error: Invalid refresh sequence or window violation

Get detailed reset cause from IWDT.

Reads the IWDT Status Register to determine the specific cause of the last watchdog reset. Refresh errors take priority over underflow since they indicate a software bug in the refresh sequence.

Returns: rx_iwdt_reset_cause_t Specific reset cause

Value	Description
k_iwdt_reset_refresh_error	Invalid refresh sequence detected
k_iwdt_reset_underflow	Counter timeout (feed not called in time)
k_iwdt_reset_none	No watchdog reset (power-on, external, etc.)

Pre: Can be called before rx_hal_iwdt_init()

Pre: Status flags preserved across reset

Post: Status flags NOT cleared (call `rx_hal_iwdt_clear_status()` to clear)

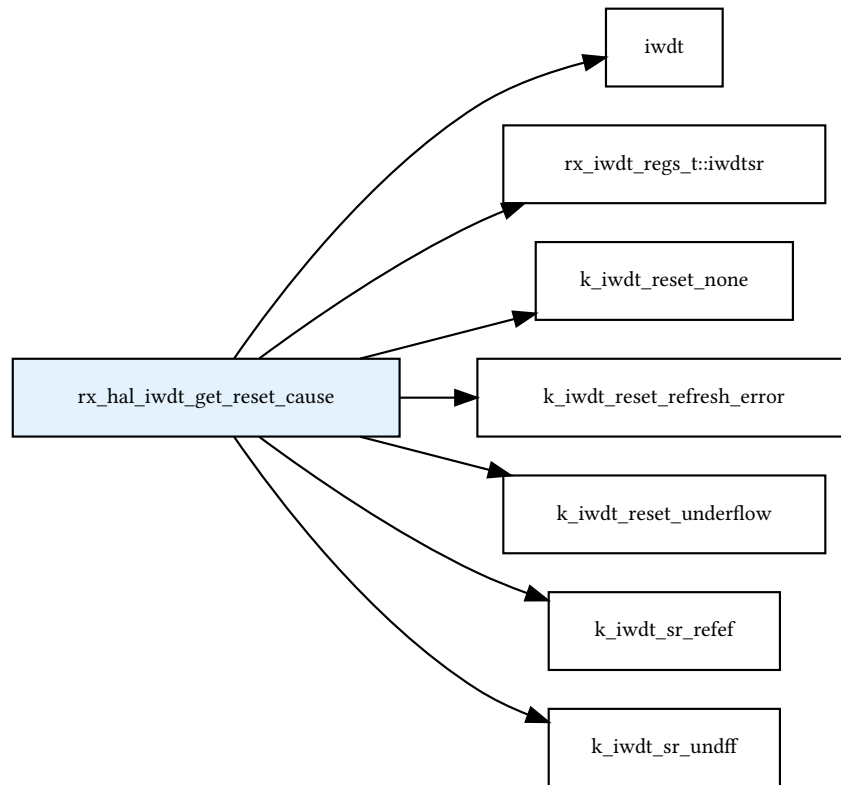
Note: This provides more detail than `rx_hal_iwdt_was_reset()` for diagnostics.

Note: On host builds, always returns `k_iwdt_reset_none`

Note: Refresh error has priority over underflow in return value] **Reset Cause Priority:** * 1. Refresh Error (REFEF=1): Invalid refresh sequence * - Incomplete 0x00/0xFF write * - Write occurred outside window (if window mode enabled) * * 2. Underflow (UNDF=1): Counter reached zero * - `rx_hal_iwdt_feed()` not called within timeout period * - Software hang, deadlock, or infinite loop * * 3. None: No watchdog reset occurred * - Power-on reset, external reset, or other cause *

Example:: `rx_iwdt_reset_cause_t` cause=`rx_hal_iwdt_get_reset_cause()`; switch(cause)
{ case `k_iwdt_reset_refresh_error`: `uart_debug_puts("ERROR:Watchdogrefreshsequenceerror!\n");`
`break`; case `k_iwdt_reset_underflow`: `uart_debug_puts("WARN:Watchdogtimeout-`
`softwarehangdetected\n");` `break`; case `k_iwdt_reset_none`: default:
`uart_debug_puts("Normalstartup\n");` `break`; } `rx_hal_iwdt_clear_status()`;

See also: `rx_hal_iwdt_was_reset()` Quick check for any watchdog reset,
`rx_hal_iwdt_clear_status()` Clear status flags

Figure 682: Call graph for `rx_hal_iwdt_get_reset_cause`

_Defined in `libs/rx_hal/inc/rx_hal_iwdt.h:385_`

Since Version 1.0.0

—

rx_hal_iwdt_clear_status

```
void rx_hal_iwdt_clear_status(void)
```

Clear watchdog status flags.

Clears the underflow and refresh error flags in IWDTSR. Call after reading the reset cause to prepare for next reset detection.

Clears the UNDF (underflow) and REFEF (refresh error) flags in the IWDT Status Register. These flags are “write-0-to-clear” type - writing 0 to a flag clears it, writing 1 has no effect.

Pre: Status has been read via `rx_hal_iwdt_was_reset()` or `rx_hal_iwdt_get_reset_cause()`

Post: UNDF flag cleared

Post: REFEF flag cleared

Note: Flags are typically cleared once at startup after logging the cause.

Note: On host builds, function is a no-op

Note: Call after reading status at startup to prepare for next reset] **Clear Sequence:** * 1. Read IWDTSR (get current status) * 2. AND with (UNDF | REFEF) to clear both flags * 3. Write back to IWDTSR * * Note: Read-modify-write is safe because flags are write-0-to-clear *

Example:: if(rx_hal_iwdt_was_reset()){ rx_iwdt_reset_cause_tcause=rx_hal_iwdt_get_reset_cause(); log_watchdog_reset(cause); } rx_hal_iwdt_clear_status();

See also: rx_hal_iwdt_was_reset() Check reset status, rx_hal_iwdt_get_reset_cause() Get specific cause

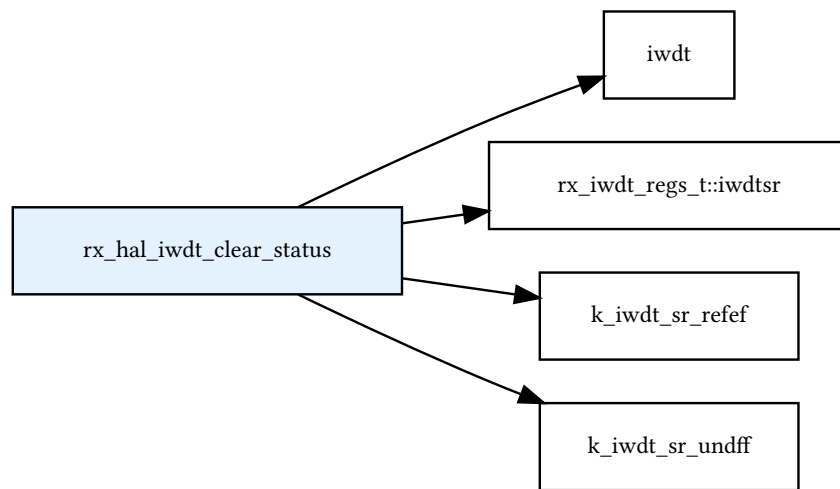


Figure 683: Call graph for rx_hal_iwdt_clear_status

_Defined in `libs/rx_hal/inc/rx_hal_iwdt.h:395_`

Since Version 1.0.0

—

rx_host_irq.h

HOST_IRQ GPIO Output Driver for RPi5 Data-Ready Signaling.

Manages the HOST_IRQ output pin (P67, pin 98) used to signal the Raspberry Pi 5 host controller that the RX72N has data ready for SPI transfer. The signal is active-low (assert LOW to request attention, deassert HIGH when idle).

Includes:

- `<stdbool.h>`

- <stdint.h>
- "rx_err.h"

rx_host_irq_init

```
rx_err_t rx_host_irq_init(void)
```

Initialize HOST_IRQ pin as GPIO output (deasserted / HIGH).

Configures P67 as a GPIO output in CMOS push-pull mode. Sets the initial output level to HIGH (deasserted / idle). The MPC function select is set to GPIO (not IRQ input).

Initialization steps:

1. Clear PMR bit 7 (GPIO mode, not peripheral)
2. Set PODR bit 7 HIGH (deasserted state)
3. Set PDR bit 7 (output direction)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, P67 configured as output
k_rx_err_invalid_state	Already initialized

Pre: P67 not in use by another peripheral (RIIC2 etc.)

Post: P67 configured as GPIO output, driven HIGH

Note: Not thread-safe. Call during system init only.] **See also:** rx_host_irq_deinit() Disable the output

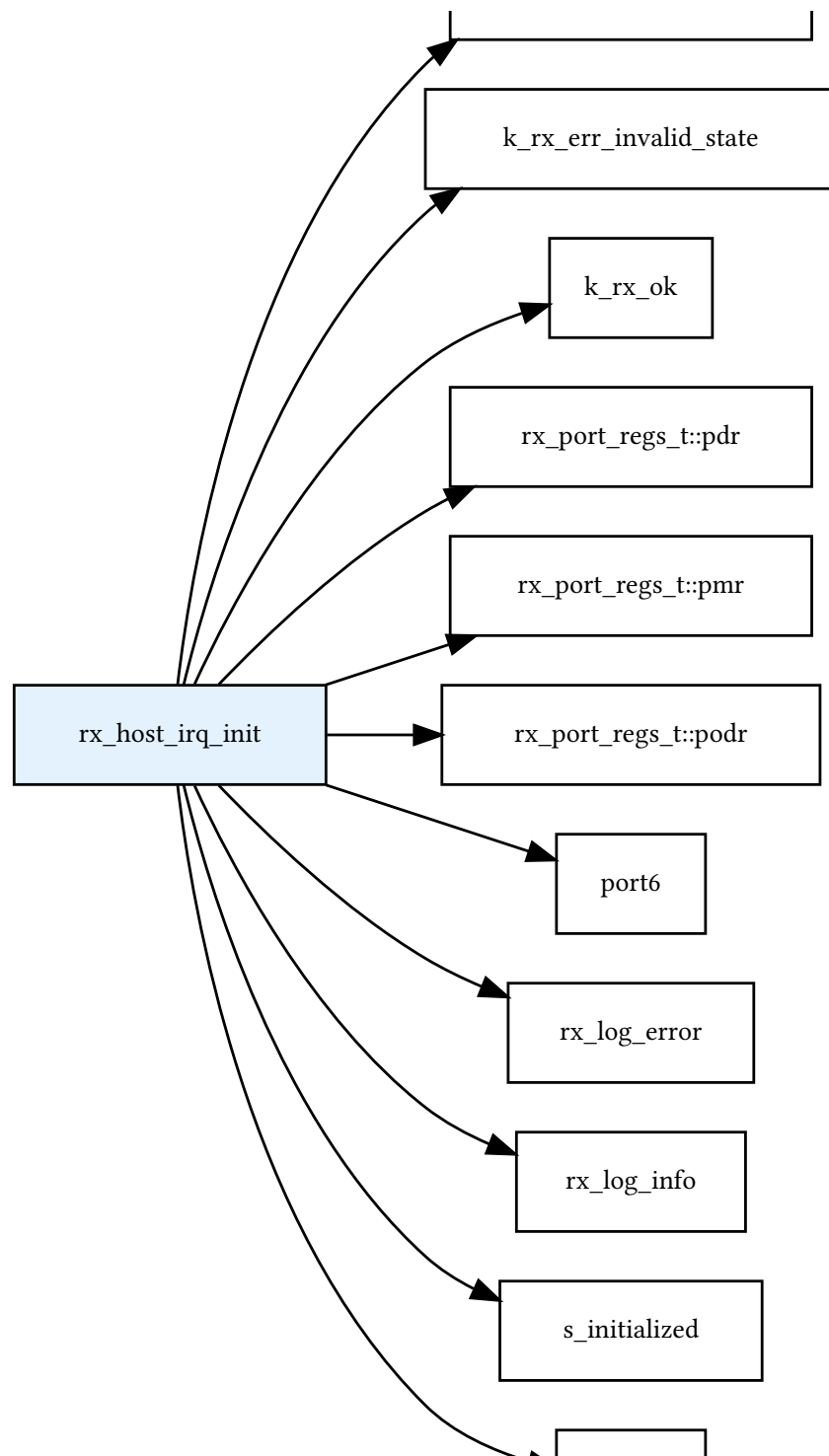


Figure 684: Call graph for rx_host_irq_init

_Defined in libs/rx_hal/inc/rx_host_irq.h:96_

Since Version 1.0.0

—

rx_host_irq_deinit

```
rx_err_t rx_host_irq_deinit(void)
```

Deinitialize HOST_IRQ pin (revert to input).

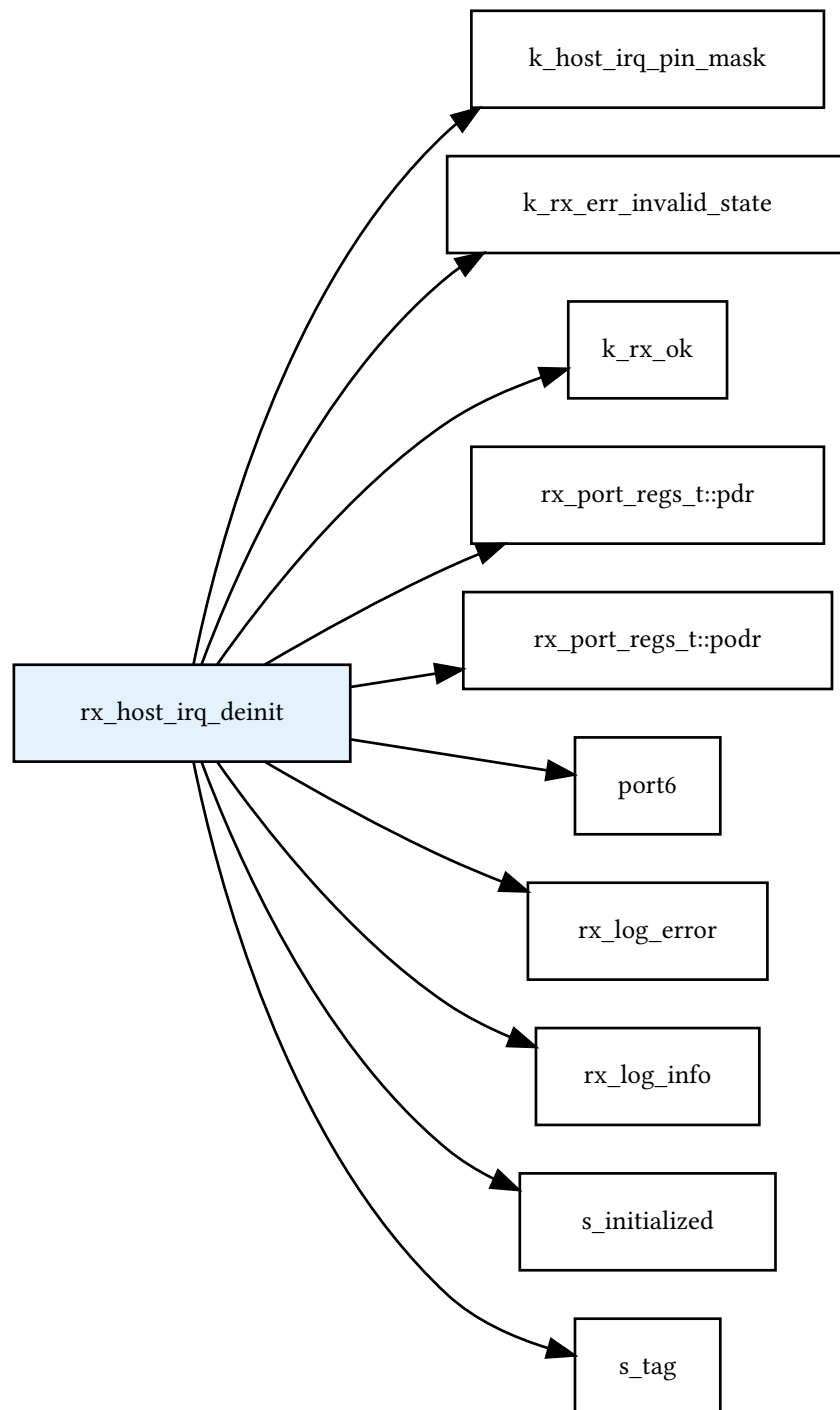
Reverts P67 to input mode (safe default) and deasserts the output.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_state	Not initialized

Pre: rx_host_irq_init() called successfully

Post: P67 reverted to input, no longer driving

Figure 685: Call graph for `rx_host_irq_deinit`

_Defined in libs/rx_hal/inc/rx_host_irq.h:113_

Since Version 1.0.0

—

rx_host_irq_assert

```
rx_err_t rx_host_irq_assert(void)
```

Assert HOST_IRQ (drive LOW to signal data ready).

Drives P67 LOW to signal the RPi5 that the RX72N has data ready for SPI transfer. The RPi5 should respond by initiating an SPI read.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, pin driven LOW
k_rx_err_invalid_state	Not initialized

Pre: rx_host_irq_init() called successfully

Post: P67 driven LOW (active)

Note: Thread-safe: single register write is atomic on RX72N] **See also:** rx_host_irq_deassert()
Release the signal

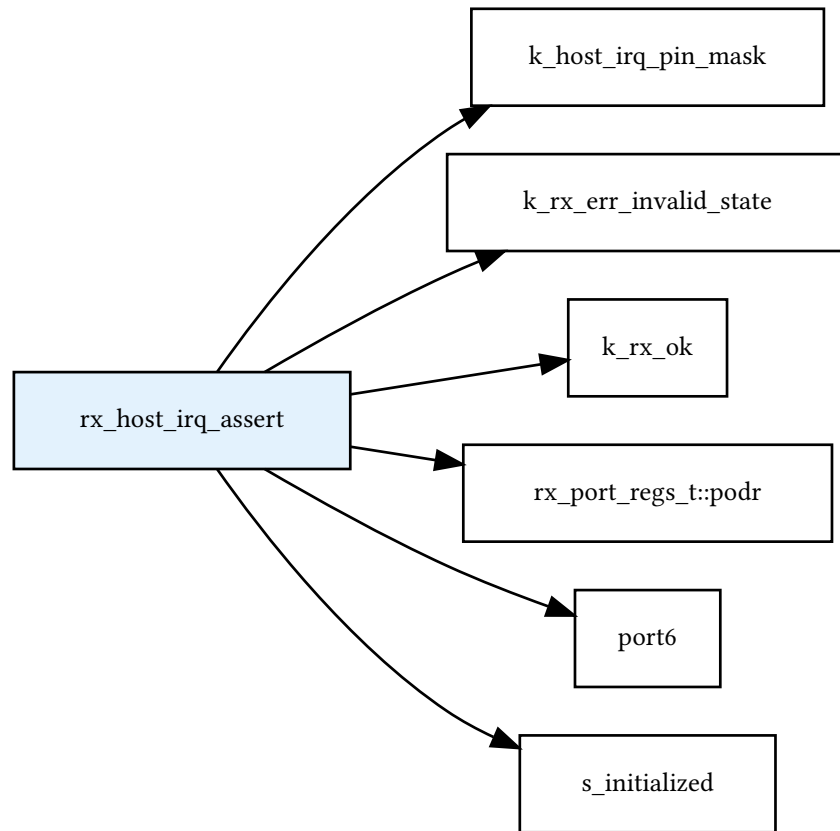


Figure 686: Call graph for rx_host_irq_assert

_Defined in `libs/rx\_hal/inc/rx\_host\_irq.h:134_`

Since Version 1.0.0

—

rx_host_irq_deassert

```
rx_err_t rx_host_irq_deassert(void)
```

Deassert HOST_IRQ (drive HIGH, idle state).

Drives P67 HIGH (idle). Call after the RPi5 has completed its SPI read.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, pin driven HIGH
k_rx_err_invalid_state	Not initialized

Pre: rx_host_irq_init() called successfully

Post: P67 driven HIGH (idle)

Note: Thread-safe: single register write is atomic on RX72N] **See also:** rx_host_irq_assert()
Assert the signal

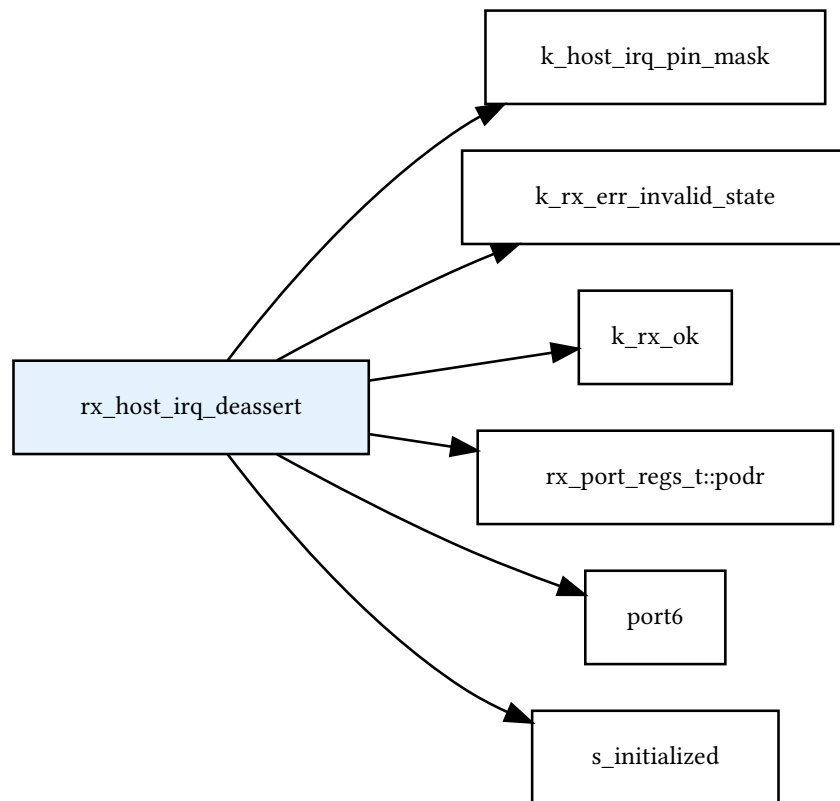


Figure 687: Call graph for rx_host_irq_deassert

_Defined in libs/rx_hal/inc/rx_host_irq.h:154_

Since Version 1.0.0

—

rx_host_irq_is_asserted

```
rx_err_t rx_host_irq_is_asserted(bool *asserted)
```

Check if HOST_IRQ is currently asserted.

Returns the current state of the HOST_IRQ output.

Parameters:

Direction	Name	Description
out	asserted	Pointer to store state (true = asserted / LOW)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	asserted is nullptr

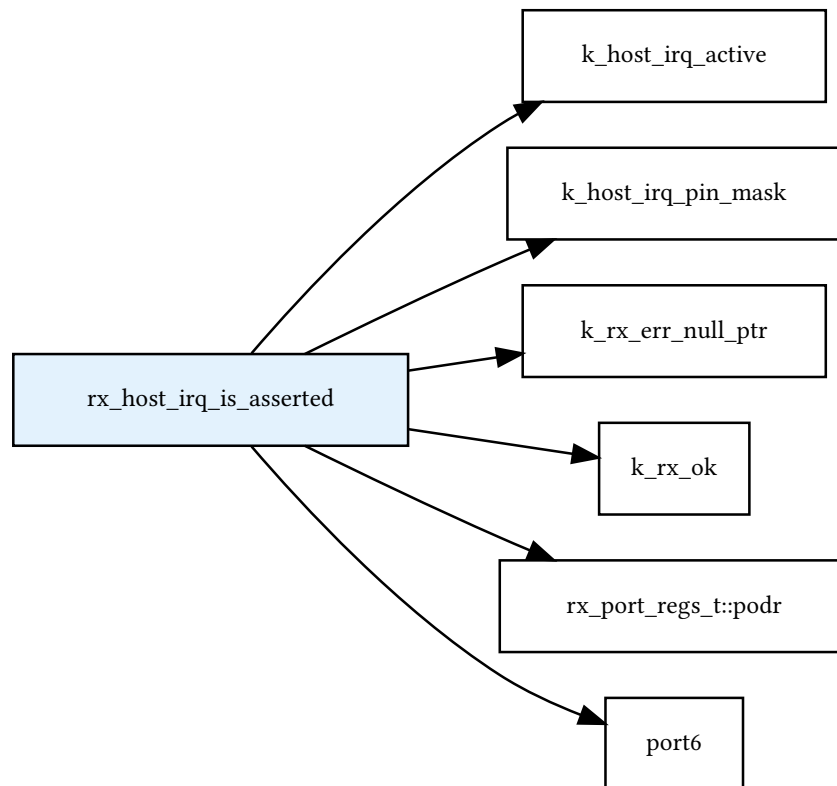
Pre: rx_host_irq_init() called**Post:** asserted set to current output state**Note:** Thread-safe (reads volatile register)

Figure 688: Call graph for rx_host_irq_is_asserted

_Defined in libs/rx_hal/inc/rx_host_irq.h:175_

Since Version 1.0.0

—

rx_irq_filter.h

IRQ Digital Filter Driver API for RX72N.

Provides hardware digital noise filtering for external interrupt (IRQ) pins on the RX72N microcontroller. The filter helps eliminate electrical noise, switch bounce, and transient glitches that could cause spurious interrupts.

The RX72N ICU provides hardware digital filters on all 16 IRQ pins (IRQ0-15). Each filter samples the input at a configurable rate and requires a stable level for 3 consecutive samples to register as a valid transition.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

rx_irq_filter_limits_t

IRQ filter limits for RX72N.

The RX72N ICU supports external IRQ pins IRQ0-IRQ15 (16 total).

Value	Name	Description
= 15	k_rx_irq_max	

—

rx_irq_filter_clk_t

IRQ filter clock divisor options for sampling rate control.

Selects the clock divisor for IRQ digital filter sampling. Higher divisors result in slower sampling, providing more noise rejection but also longer response times to valid transitions.

The digital filter requires 3 consecutive samples at the same level to recognize a valid transition. Total response time = 3 x sample period.

Value	Name	Description
= 0	k_irq_filter_pclk_1	PCLKB/1 - Fastest sampling, minimal filtering.
= 1	k_irq_filter_pclk_8	PCLKB/8 - Fast sampling, light filtering.
= 2	k_irq_filter_pclk_32	PCLKB/32 - Moderate sampling, good filtering.
= 3	k_irq_filter_pclk_64	PCLKB/64 - Slowest sampling, maximum filtering.

Note: Register encoding: 2 bits per IRQ in IRQFLTC register

Warning: Lower divisors provide faster response but less noise immunity] **See also:**
`rx_irq_filter_enable()` Set filter clock divisor, `rx_irq_filter_get_clock()` Read
current divisor setting

Since Version 1.0.0

—

s_irq_max

`const uint8_t s_irq_max`

Maximum valid IRQ number (0-15).

Note: Static const rather than enum due to use in comparison operations

Since Version 1.0.0

—

rx_irq_filter_enable

```
rx_err_t rx_irq_filter_enable(uint8_t irq_num, rx_irq_filter_clk_t filter_clk)
```

Enable hardware digital filter on an IRQ pin.

Enables the ICU digital filter for the specified IRQ pin with the given clock divisor. The filter samples the input and requires 3 consecutive samples at the same level to recognize a valid edge transition.

Configures the ICU filter enable (IRQFLTE) and filter clock (IRQFLTC) registers for the specified IRQ. Sets clock divisor first, then enables the filter to avoid sampling at incorrect rate during transition.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number to configure Valid range: [0, 15] IRQ0-7 use IRQFLTE[0]/IRQFLTC[0] IRQ8-15 use IRQFLTE[1]/IRQFLTC[1]
in	filter_clk	Clock divisor for filter sampling k_irq_filter_pclk_1: Minimal filtering k_irq_filter_pclk_64: Maximum filtering
in	irq_num	IRQ number (0-15)
in	filter_clk	Clock divisor setting

Returns: k_rx_ok on success, k_rx_err_invalid_arg if parameters invalid

Value	Description
k_rx_ok	Filter successfully enabled
k_rx_err_invalid_arg	irq_num > 15
k_rx_err_invalid_arg	filter_clk > k_irq_filter_pclk_64

Pre: PCLKB clock must be running (filter clock source)

Pre: ICU module not in module stop

Post: IRQFLTE bit set for this IRQ

Post: IRQFLTC bits set to specified divisor

Post: Filter immediately active

Note: Filter applies to all edge types (rising, falling, both)

Note: Filter continues operating during software standby

Note: Clock divisor is set before enable to avoid glitches

Note: On host (non-RX) builds, validates parameters but skips register access

Warning: Enabling filter adds latency to interrupt response] **Algorithm Steps:** Validate irq_num is in range [0, 15] Validate filter_clk is valid divisor Calculate register index (0 for IRQ0-7, 1 for IRQ8-15) Set clock divisor in IRQFLTC register Set enable bit in IRQFLTE register

Thread Safety: Not thread-safe. ICU register modification is not atomic.

Performance: Execution time: 200 ns @ 240 MHz No blocking operations

Example - E-STOP Button: rx_err_t err=rx_irq_filter_enable(0,k_irq_filter_pclk_64); if(err!=k_rx_ok){ }

Register Modification Sequence: IRQFLTC: Clear and set 2-bit clock field for this IRQ IRQFLTE: Set enable bit for this IRQ

See also: rx_irq_filter_disable() Disable filter, rx_irq_filter_clk_t Clock divisor options, rx_irq_filter.h Full API documentation

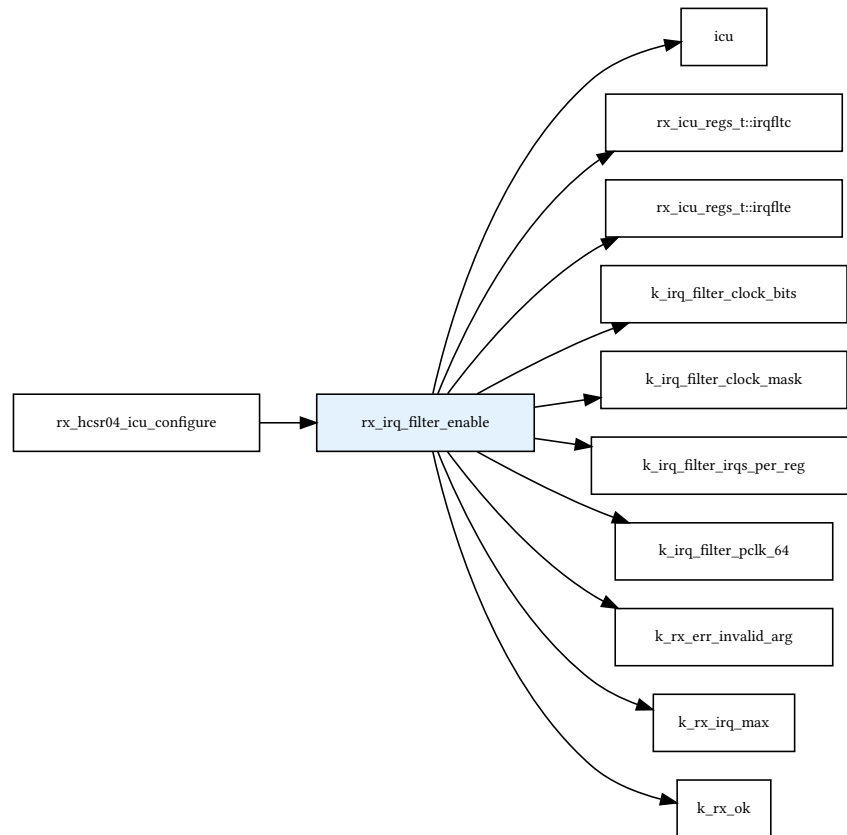


Figure 689: Call graph for rx_irq_filter_enable

_Defined in libs/rx_hal/inc/rx_irq_filter.h:354_

Since Version 1.0.0

—

rx_irq_filter_disable

```
rx_err_t rx_irq_filter_disable(uint8_t irq_num)
```

Disable hardware digital filter on an IRQ pin.

Disables the ICU digital filter for the specified IRQ pin. After disabling, the raw unfiltered input signal is passed through to the interrupt detector.

Clears the filter enable bit in IRQFLTE for the specified IRQ. After disabling, raw unfiltered input passes to interrupt detector.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number to configure Valid range: [0, 15]

in	irq_num	IRQ number (0-15)
----	---------	-------------------

Returns: k_rx_ok on success, k_rx_err_invalid_arg if irq_num > 15

Value	Description
k_rx_ok	Filter successfully disabled
k_rx_err_invalid_arg	irq_num > 15

Pre: None (can be called regardless of current filter state)

Post: IRQFLTE bit cleared for this IRQ

Post: Raw input passed to interrupt detector

Note: Clock divisor setting is preserved (unchanged)

Note: Safe to call even if filter was already disabled

Note: Clock divisor setting preserved for potential re-enable] **Algorithm Steps:** Validate irq_num is in range [0, 15] Calculate register index Clear enable bit in IRQFLTE register

Thread Safety: Not thread-safe. ICU register modification is not atomic.

Example: rx_irq_filter_disable(5);

See also: rx_irq_filter_enable() Enable filter, rx_irq_filter.h Full API documentation

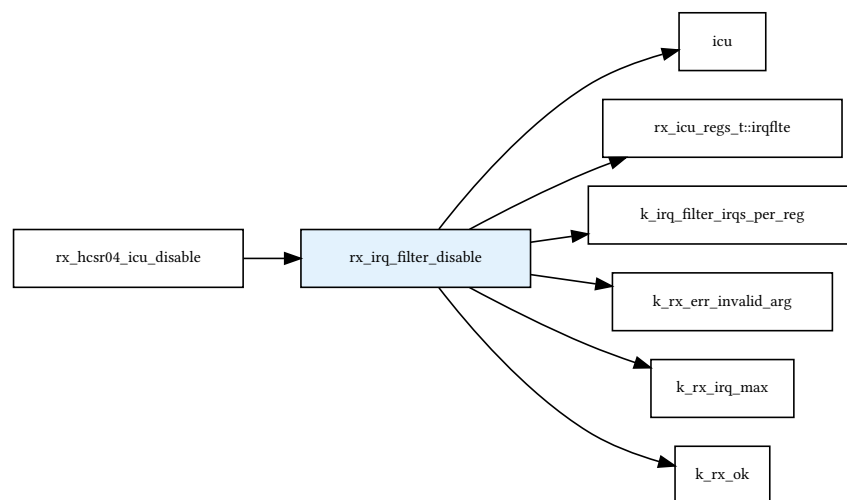


Figure 690: Call graph for rx_irq_filter_disable

_Defined in libs/rx_hal/inc/rx_irq_filter.h:397_

Since Version 1.0.0

—

rx_irq_filter_is_enabled

```
rx_err_t rx_irq_filter_is_enabled(uint8_t irq_num, bool *enabled)
```

Check if digital filter is enabled on an IRQ pin.

Reads the IRQFLTE register to determine if the digital filter is currently enabled for the specified IRQ.

Reads the IRQFLTE register bit for the specified IRQ to determine current filter enable state.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number to query Valid range: [0, 15]
out	enabled	Pointer to receive enable status true: Filter is enabled false: Filter is disabled Must not be nullptr
in	irq_num	IRQ number (0-15)
out	enabled	Pointer to store filter status (true=enabled)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if parameters invalid

Value	Description
k_rx_ok	Status successfully retrieved
k_rx_err_invalid_arg	irq_num > 15
k_rx_err_invalid_arg	enabled pointer is nullptr

Pre: enabled pointer must be non-NULL

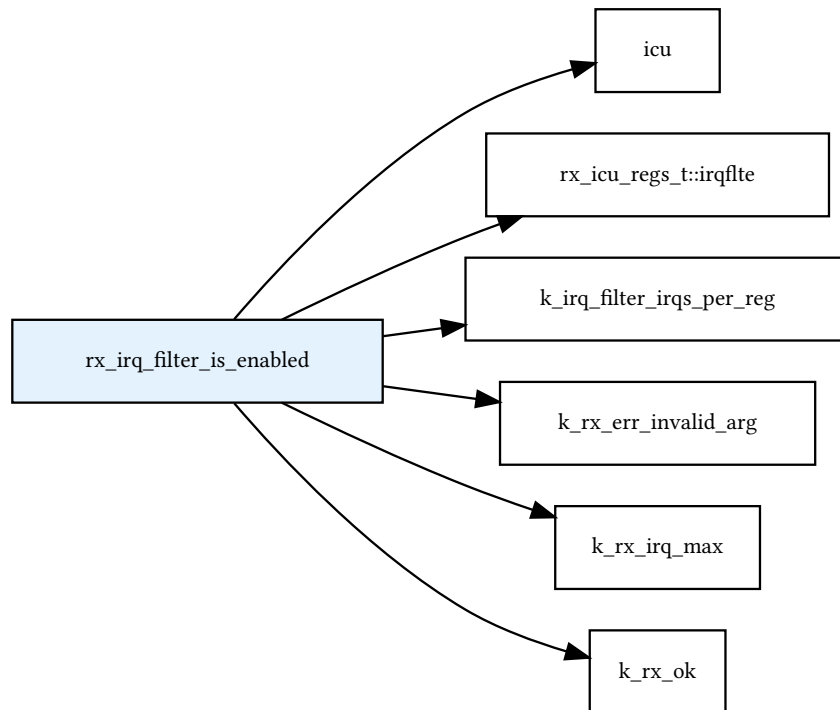
Post: *enabled contains current filter state

Post: No registers modified (read-only operation)

Note: On host builds, always returns false (filter disabled)] **Thread Safety:** Thread-safe for reading. Register read is atomic.

Example: boolfilter_active; rx_err_t err=rx_irq_filter_is_enabled(0,&filter_active);
if(err==k_rx_ok&&filter_active){ }

See also: rx_irq_filter_enable() Enable filter, rx_irq_filter_get_clock() Get clock setting, rx_irq_filter.h Full API documentation

Figure 691: Call graph for `rx_irq_filter_is_enabled`

_Defined in `libs/rx\hal/inc/rx\irq\filter.h:441_`

Since Version 1.0.0

—

`rx_irq_filter_get_clock`

```
rx_err_t rx_irq_filter_get_clock(uint8_t irq_num, rx_irq_filter_clk_t
*filter_clk)
```

Get current filter clock divisor setting for an IRQ.

Reads the IRQFLTC register to retrieve the current clock divisor setting for the specified IRQ's digital filter.

Reads the 2-bit clock divisor field from IRQFLTC register for the specified IRQ. Returns the divisor setting even if filter is disabled.

Parameters:

Direction	Name	Description
in	<code>irq_num</code>	IRQ number to query Valid range: [0, 15]
out	<code>filter_clk</code>	Pointer to receive clock divisor setting Returns <code>rx_irq_filter_clk_t</code> value Must not be nullptr

in	irq_num	IRQ number (0-15)
out	filter_clk	Pointer to store clock divisor (rx_irq_filter_clk_t)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if parameters invalid

Value	Description
k_rx_ok	Clock setting successfully retrieved
k_rx_err_invalid_arg	irq_num > 15
k_rx_err_invalid_arg	filter_clk pointer is nullptr

Pre: filter_clk pointer must be non-NULL

Post: *filter_clk contains current divisor setting

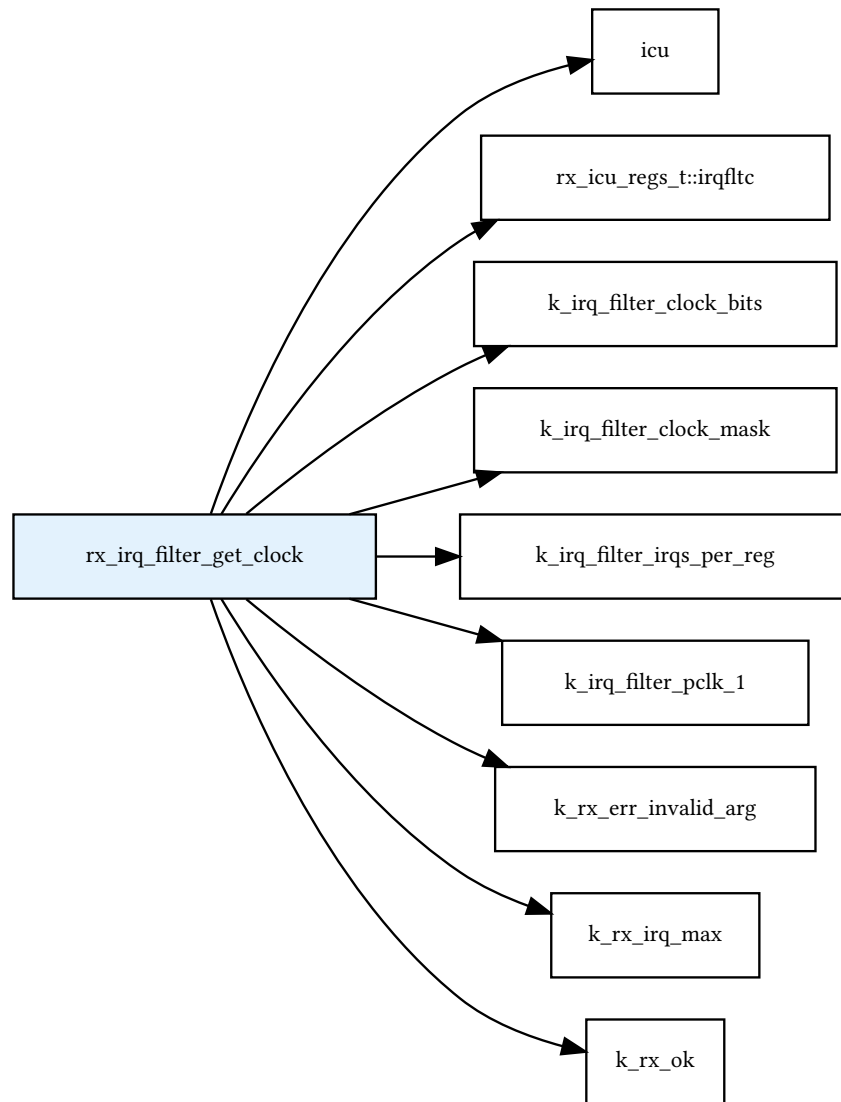
Post: No registers modified (read-only operation)

Note: Returns current setting even if filter is disabled

Note: On host builds, always returns k_irq_filter_pclk_1] **Thread Safety:** Thread-safe for reading. Register read is atomic.

Example: rx_irq_filter_clk_t current_clock; rx_err_t err = rx_irq_filter_get_clock(0, ¤t_clock);
if (err == k_rx_ok) { if (current_clock == k_irq_filter_pclk_64) { } }

See also: rx_irq_filter_enable() Set clock divisor, rx_irq_filter_clk_t Clock divisor values, rx_irq_filter.h Full API documentation

Figure 692: Call graph for `rx_irq_filter_get_clock`

_Defined in `libs/rx_hal/inc/rx_irq_filter.h:488_`

Since Version 1.0.0

—

rx_mpc.h

Multi-Function Pin Controller (MPC) Driver API for RX72N.

High-level driver for configuring RX72N GPIO pin multiplexing. The MPC module controls which peripheral function is connected to each physical pin through Pin Function Select (PFS) registers.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"
- "rx_port_constants.h"

rx_pin_function_t

High-level pin function mode selection.

Specifies whether a pin is configured for GPIO operation or peripheral function mode. This is a conceptual abstraction - the actual hardware configuration is done through PFS register PSEL field and PORT PMR bit.

Value	Name	Description
= 0x00	k_pin_function_gpio	GPIO mode - pin controlled by PORT registers.
= 0x01	k_pin_function_periph	Peripheral function mode - pin connected to peripheral.

Note: This enum is for documentation/API clarity. Actual hardware uses PSEL field values directly.] **See also:** rx_mpc_set_gpio() Configure pin for GPIO mode, rx_mpc_set_peripheral() Configure pin for peripheral mode

Since Version 1.0.0

—

rx_pin_psel_t

Peripheral Select (PSEL) codes for common STAR project functions.

These values are written to the PSEL[4:0] field of PFS registers to select which peripheral function a pin is connected to. Values are pin-specific - these constants represent the most common values used in the STAR project.

Value	Name	Description
= 0x01	k_psel_mtu_ioc	MTU I/O Compare/PWM output (MTIOC).
= 0x02	k_psel_mtu_clk	MTU clock input (MTCLK).
= 0x03	k_psel_mtu_phase	MTU encoder phase counting input.
= 0x0A	k_psel_sci_tx	SCI transmit data output (TXD).
= 0x0A	k_psel_sci_rx	SCI receive data input (RXD).
= 0x0F	k_psel_riic_scl	RIIC clock line (SCL).
= 0x0F	k_psel_riic_sda	RIIC data line (SDA).
= 0x0D	k_psel_rspi_clk	RSPI clock (RSPCK).
= 0x0D	k_psel_rspi_copi	RSPI Controller Out Peripheral In (COPI, formerly MOSI).

= 0x0D	k_psel_rspi_cipo	RSPI Controller In Peripheral Out (CIPO, formerly MISO).
= 0x00	k_psel_adc	ADC analog input mode.
= 0x11	k_psel_usb_vbus	USB VBUS detection input.
= 0x14	k_psel_gptw	GPTW complementary PWM output (GTIOC).

Warning: PSEL values are pin-specific! A value that works for one pin may select a completely different function on another pin. Always verify against the RX72N Hardware Manual pin function tables.] **See also:** rx_mpc_set_peripheral() Use these values with this function, RX72N Hardware Manual, Appendix for complete pin function tables

Since Version 1.0.0

—

rx_mpc_set_gpio

```
rx_err_t rx_mpc_set_gpio(rx_port_pin_t pin)
```

Configure pin for GPIO (General Purpose I/O) function.

Sets the pin to GPIO mode by writing 0x00 to its PFS register (PSEL=0, ISEL=0, ASEL=0). This disconnects the pin from all peripheral functions and allows it to be controlled by PORT PDR/PODR registers.

MPC base address 0x0008C100 remains constant

Other pins' PFS registers are not modified

Sets the specified pin to GPIO mode by writing 0x00 to its PFS register. This disconnects the pin from all peripheral functions and allows it to be controlled by the PORT registers (PDR, PODR, PIDR).

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (rx_port_pin_t from rx_port_constants.h) Encodes both port (A-J) and pin number (0-7) Use k_port_X_pin_Y constants for type safety Example: k_port_e_pin_5 for PE5
in	pin	GPIO pin identifier (rx_port_pin_t from rx_port_constants.h)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for GPIO mode
k_rx_err_invalid_arg	Port number exceeds valid range (> Port J)
k_rx_err_invalid_arg	Pin number exceeds 7
k_rx_err_invalid_arg	Port not available on package (G/H on 144-pin)
k_rx_ok	Pin successfully set to GPIO mode

k_rx_err_invalid_arg	Port number invalid (> Port J)
k_rx_err_invalid_arg	Pin number invalid (> 7)
k_rx_err_invalid_arg	Port not available on package

Pre: PCLKB clock must be running (MPC register access requires it)

Pre: MPC module stop bit must be cleared (MSTPCRA.MSTPA9 = 0)

Pre: PCLKB clock running, MPC not in module stop

Pre: pin must be a valid rx_port_pin_t constant

Post: PFS register for specified pin contains 0x00

Post: Pin is disconnected from all peripheral functions

Post: PWPR is locked (B0WI=1) after operation

Post: PFS register = 0x00 (GPIO mode)

Post: PWPR locked after operation

Note: Not thread-safe. Use mutex protection if calling from multiple threads.

Note: This only sets PFS register. To complete GPIO setup, also configure PORT PMR (peripheral mode = 0) and PDR (direction) registers.

Note: Thread safety: Not thread-safe

Note: Also set PORT PMR bit to 0 for complete GPIO configuration

Warning: Calling on a pin that doesn't exist on the package returns error but does not cause hardware fault.] **Algorithm Steps:** Extract port number and pin number from encoded pin parameter Validate port is in valid range [0, J] Validate pin number is in valid range [0, 7] Calculate PFS register address for this pin Unlock PWPR write protection (B0WI=0, PFSWE=1) Write 0x00 to PFS register (GPIO mode) Lock PWPR write protection (PFSWE=0, B0WI=1) Return success

Performance: Execution time: 1 us @ 240 MHz (PWPR unlock/lock overhead) No loops or blocking operations

Thread Safety: Not thread-safe. The PWPR unlock/lock sequence is non-atomic.

Example - Single Pin: #include "rx_mpc.h" #include "rx_port_constants.h"

```
rx_err_t err = rx_mpc_set_gpio(k_port_e_pin_5); if (err != k_rx_ok)
{ rx_log_error("INIT", "Failed to set PE5 to GPIO mode"); return err; }
```

Example - Error Handling: rx_err_t err = rx_mpc_set_gpio(k_port_g_pin_0);
if (err == k_rx_err_invalid_arg) { rx_log_warn("MPC", "Pin not available on this package"); }

NASA Power of 10 Compliance: Rule 5: [OK] 2 preconditions (clock, module stop), 3
postconditions Rule 7: [OK] All internal function returns checked

Implementation Details: Extract port and pin number from encoded rx_port_pin_t. Validate port is
in range [0, J]. Validate pin is in range [0, 7]. Write 0x00 to PFS register (via internal_write_pfs)

Example:

```
// Configure P30 (port 3, pin 0) for GPIO use
rx_err_t err = rx_mpc_set_gpio(RX_PORT_PIN(3, 0));
if (err != k_rx_ok) {
    // handle error
}
```

See also: rx_mpc_set_peripheral() Configure pin for peripheral function,
rx_port_constants.h Pin enumeration definitions, rx_mpc.h Full API documentation

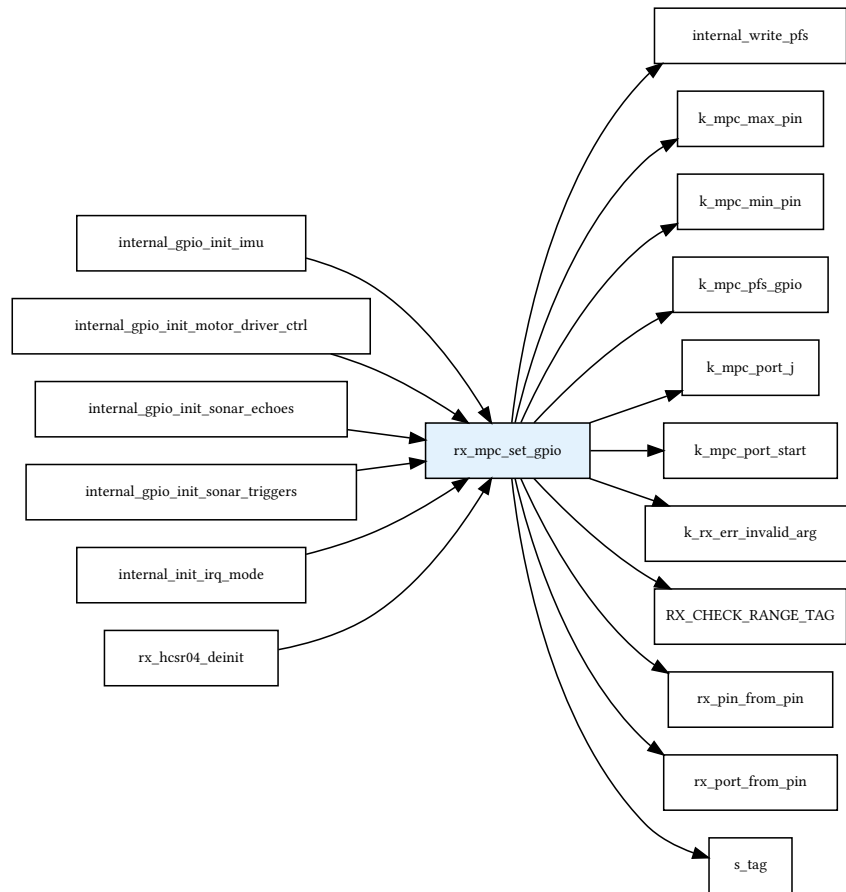


Figure 693: Call graph for rx_mpc_set_gpio

Defined in `libs/rx_hal/inc/rx_mpc.h:700`

Since Version 1.0.0

—

rx_mpc_set_peripheral

```
rx_err_t rx_mpc_set_peripheral(const rx_mpc_peripheral_config_t *config)
```

Configure pin for peripheral function with specified PSEL code.

Core function for pin multiplexer configuration. Writes the specified PSEL value to the pin's PFS register, connecting the pin to a peripheral function. Uses a configuration structure to prevent parameter swapping.

config structure is not modified (const pointer)

Other pins' PFS registers are not modified

Core function for peripheral pin configuration. Writes the specified PSEL value to the pin's PFS register, connecting it to a peripheral.

Parameters:

Direction	Name	Description
in	config	Pointer to MPC peripheral configuration structure config->pin: Target pin (rx_port_pin_t constant) config->psel: PSEL value to write (0-31) Must not be nullptr Must remain valid only during function call
in	config	Pointer to peripheral configuration structure config->pin: Target pin identifier config->psel: PSEL value to write (0-31)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for peripheral function
k_rx_err_invalid_arg	config pointer is nullptr
k_rx_err_invalid_arg	config->psel exceeds 31 (5-bit field max)
k_rx_err_invalid_arg	Port number exceeds valid range
k_rx_err_invalid_arg	Pin number exceeds 7
k_rx_err_invalid_arg	Port not available on package
k_rx_ok	Pin successfully configured
k_rx_err_invalid_arg	config is nullptr
k_rx_err_invalid_arg	PSEL exceeds 31
k_rx_err_invalid_arg	Invalid port or pin

Pre: config pointer must be non-NULL

Pre: config->psel must be in range 0, 31

Pre: config->pin must be a valid rx_port_pin_t value

Pre: PCLKB clock must be running

Pre: config must be non-NULL with valid fields

Pre: PCLKB clock running

Post: PFS register for specified pin contains config->psel value

Post: Pin is connected to peripheral function specified by PSEL

Post: PWPR is locked (B0WI=1) after operation

Post: PFS register contains config->psel

Post: PWPR locked after operation

Note: Not thread-safe. Use mutex protection if calling from multiple threads.

Note: This only sets PFS register. For peripheral operation, also set PORT PMR bit to 1 to enable peripheral mode.

Note: PSEL values are pin-specific. Verify correct value in hardware manual.

Note: Thread safety: Not thread-safe

Note: Also set PORT PMR bit to 1 for peripheral mode

Warning: Wrong PSEL value may connect pin to unexpected peripheral!

Warning: Not all PSEL values are valid for all pins.] **Algorithm Steps:** Validate config pointer is not nullptr Validate PSEL value is in range [0, 31] Extract port number and pin number from config->pin Validate extracted port and pin are in valid ranges Calculate PFS register address for this pin Unlock PWPR write protection (B0WI=0, PFSWE=1) Write config->psel to PFS register Lock PWPR write protection (PFSWE=0, B0WI=1) Return success

Pin Configuration Flow: digraph config_flow { rankdir=TB; node [shape=box, style=rounded];

```
start [label="rx_mpc_set_peripheral()"]; validate_ptr [label="Validate config != nullptr"];
validate_psel [label="Validate PSEL <= 31"]; extract [label="Extract port, pin"]; validate_pin
[label="Validate port, pin range"]; get_pfs [label="Get PFS register address"]; unlock [label="Unlock
PWPR"]; write [label="Write PSEL to PFS"]; lock [label="Lock PWPR"]; success [label="Return
k_rx_ok", fillcolor=lightgreen, style=filled]; error [label="Return k_rx_err_invalid_arg",
fillcolor=lightcoral, style=filled];
```

```
start -> validate_ptr; validate_ptr -> validate_psel [label="OK"]; validate_ptr -> error
[label="NULL"]; validate_psel -> extract [label="OK"]; validate_psel -> error [label="> 31"]; extract -
> validate_pin; validate_pin -> get_pfs [label="OK"]; validate_pin -> error [label="invalid"]; get_pfs
-> unlock; unlock -> write; write -> lock; lock -> success; }
```

Performance: Execution time: 1 us @ 240 MHz No loops or blocking operations

Thread Safety: Not thread-safe. The PWPR unlock/lock sequence is non-atomic.

Example - Motor PWM Configuration: #include "rx_mpc.h" #include "rx_port_constants.h"

```
rx_mpc_peripheral_config_t pwm_config = { .pin = k_port_e_pin_5, .psel = k_psel_gptw };
```

```
rx_err_t err = rx_mpc_set_peripheral(&pwm_config); if(err != k_rx_ok)
{ rx_log_error("MOTOR", "PWM pin configuration failed"); return err; }
```

Example - Batch Configuration:

```
constrx_mpc_peripheral_config_pins[] = { {k_port_e_pin_5, k_psel_gptw},
{k_port_e_pin_4, k_psel_gptw}, {k_port_2_pin_4, 0x02}, {k_port_2_pin_5, 0x02}, };

for(size_t i = 0; i < sizeof(pins)/sizeof(pins[0]); i++) { rx_err_t err = rx_mpc_set_peripheral(&pins[i]); if(err != k_rx_ok) { rx_log_error("MPC", "Pin config failed"); return err; } }
```

NASA Power of 10 Compliance: Rule 5: [OK] 4 preconditions, 3 postconditions Rule 7: [OK] All return values checked

Implementation Details: Validate config pointer is not nullptr Validate PSEL value is in range [0, 31] Extract port and pin from config->pin Validate extracted values Write PSEL to PFS register (via internal_write_pfs)

Example:

```
//Configure PA0 for RSPI clock (PSEL=0x0D)
constrx_mpc_peripheral_config_t cfg = {
    .pin = RX_PORT_PIN(0xA, 0),
    .psel = 0x0D,
};
rx_err_t err = rx_mpc_set_peripheral(&cfg);
if(err != k_rx_ok) {
    //handle error
}
```

See also: rx_mpc_set_gpio() Configure pin for GPIO mode, rx_mpc_peripheral_config_t Configuration structure definition, rx_pin_psel_t Common PSEL value constants, rx_mpc.h Full API documentation

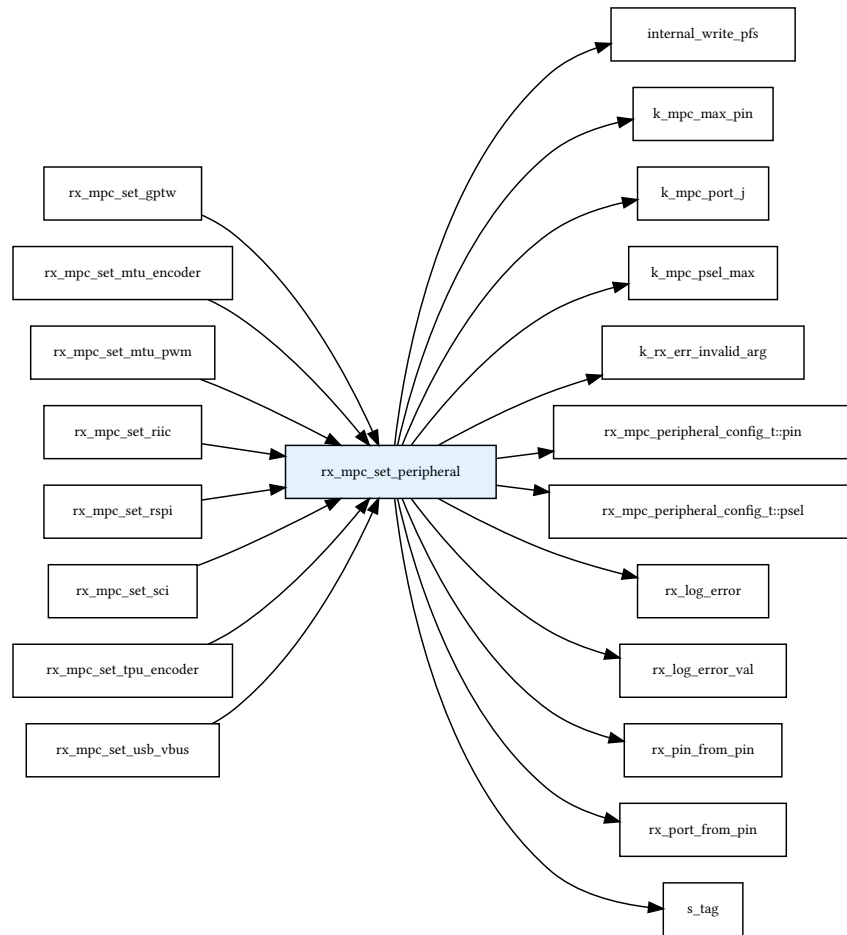


Figure 694: Call graph for rx_mpc_set_peripheral

Defined in `libs/rx_hal/inc/rx_mpc.h:847`

Since Version 1.0.0

—

rx_mpc_set_mtu_pwm

```
rx_err_t rx_mpc_set_mtu_pwm(rx_port_pin_t pin)
```

Configure pin for MTU3a timer PWM output (MTIOC).

Convenience function that configures a pin for MTU3a I/O compare output, used for PWM generation. Internally calls `rx_mpc_set_peripheral()` with `PSEL = k_psel_mtu_ioc (0x01)`.

Convenience wrapper that configures a pin for MTU I/O compare output using `PSEL = k_psel_mtu_ioc (0x01)`. Commonly used for motor PWM.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for MTU PWM output Must be a pin that supports MTIOC function Use k_port_X_pin_Y constants
in	pin	GPIO pin identifier for MTU PWM output

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for MTU PWM
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for MTU PWM
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must support MTU I/O compare function (check hardware manual)

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination supported by MTU MTIOC

Post: PFS register contains PSEL = 0x01 (MTU IOC)

Post: Pin ready for MTU3a compare match output

Post: PFS register contains k_psel_mtu_ioc (0x01) for the specified pin

Post: PWPR locked after operation

Note: Also requires MTU3a timer configuration and PORT PMR=1

Note: STAR project typically uses GPTW for PWM (rx_mpc_set_gptw with PSEL 0x14)

Note: Thread safety: Not thread-safe

Note: STAR project uses GPTW (PSEL=0x07) for motor PWM, not MTU] **Common MTU PWM Pins:** Pin MTU Channel Output Typical Use

P14 MTU3 MTIOC3A Motor 0 Phase A

P15 MTU3 MTIOC3B Motor 0 Phase B

P16 MTU3 MTIOC3C Motor 0 Phase C

P17 MTU3 MTIOC3D Motor 0 Phase D

P24 MTU4 MTIOC4A Motor 1 Phase A

P25 MTU4 MTIOC4B Motor 1 Phase B

P26 MTU4 MTIOC4C Motor 1 Phase C

P27 MTU4 MTIOC4D Motor 1 Phase D

Example: rx_mpc_set_mtu_pwm(k_port_1_pin_4); rx_mpc_set_mtu_pwm(k_port_1_pin_5);
rx_mpc_set_mtu_pwm(k_port_1_pin_6); rx_mpc_set_mtu_pwm(k_port_1_pin_7);

Example:

```
//ConfigureP14forMTU3PWMoutput(MTIOCA)
rx_err_t err=rx_mpc_set_mtu_pwm(RX_PORT_PIN(1,4));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc_set_peripheral() Underlying implementation, rx_mtu.h MTU timer driver,
rx_mpc.h Full API documentation

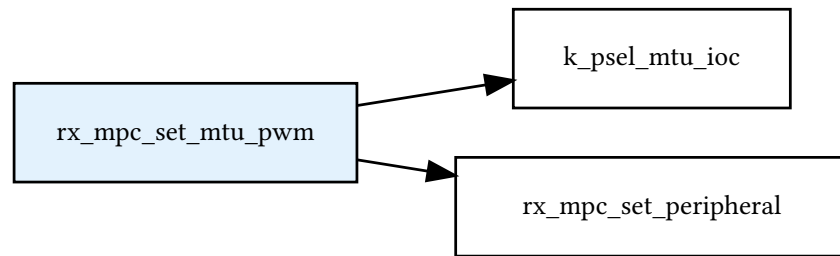


Figure 695: Call graph for rx_mpc_set_mtu_pwm

Defined in `libs/rx_hal/inc/rx_mpc.h:901`

Since Version 1.0.0

—

rx_mpc_set_mtu_encoder

```
rx_err_t rx_mpc_set_mtu_encoder(rx_port_pin_t pin)
```

Configure pin for MTU3a encoder/phase counter input (MTCLK).

Configures a pin for MTU3a phase counting mode input, used with quadrature encoders. Sets PSEL = k_psel_mtu_phase (0x03) for phase counting with direction detection.

Convenience wrapper that configures a pin for MTU phase counting mode using PSEL = k_psel_mtu_phase (0x03). Used for quadrature encoders.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	pin	GPIO pin identifier for encoder input Must be a pin that supports MTCLK function Configure in pairs (A+B) for quadrature
in	pin	GPIO pin identifier for encoder input (e.g., P24/P25)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for encoder input
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for encoder input
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must support MTU clock/phase function

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination supported by MTCLK input

Post: PFS register contains PSEL = 0x03 (MTU phase)

Post: Pin ready for quadrature encoder input

Post: PFS register contains k_psel_mtu_phase (0x03) for the specified pin

Post: PWPR locked after operation

Note: Configure both Phase A and Phase B pins for proper operation

Note: Also requires MTU3a timer configuration in phase counting mode

Note: Thread safety: Not thread-safe

Note: Configure both Phase A and Phase B pins for proper operation] **Quadrature Encoder Interface:** * Encoder Output MTU3a Phase Counting * +-----+ +-----+ * | Phase A |
 ---->| MTCLKA (Count Up) | * | Phase B |---->| MTCLKB (Count Down) | * | (Index Z) |---->|
 (Optional reset) | * +-----+ +-----+ * Counter increments/decrements * based on phase
 relationship *

Common Encoder Pins (STAR Project): Pin Pair MTU Input Encoder

P24/P25 MTCLKA/MTCLKB Motor 0

PC0/PC1 MTCLKC/MTCLKD Motor 1

Example - Quadrature Encoder Setup: rx_err_t err;

```
err=rx_mpc_set_mtu_encoder(k_port_2_pin_4); if(err!=k_rx_ok)return err;
```

```
err=rx_mpc_set_mtu_encoder(k_port_2_pin_5); if(err!=k_rx_ok)return err;
```

Example:

```
//ConfigurePC0andPC1forMTUphasecounting(MTCLKA/B)
rx_err_t err=rx_mpc_set_mtu_encoder(RX_PORT_PIN(0xC,0));
if(err==k_rx_ok){
err=rx_mpc_set_mtu_encoder(RX_PORT_PIN(0xC,1));
}
```

See also: rx_mpc_set_peripheral() Underlying implementation, rx_mtu_encoder.h Encoder driver using MTU phase counting, rx_mpc.h Full API documentation, rx_mtu_encoder.h Encoder driver using MTU phase counting

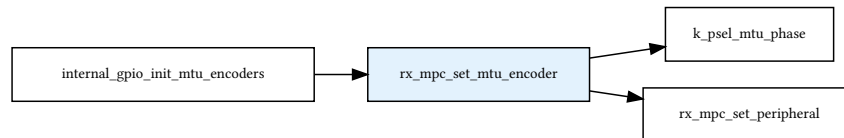


Figure 696: Call graph for rx_mpc_set_mtu_encoder

Defined in `libs/rx_hal/inc/rx_mpc.h:967`

Since Version 1.0.0

—

rx_mpc_set_tpu_encoder

```
rx_err_t rx_mpc_set_tpu_encoder(rx_port_pin_t pin)
```

Configure pin for TPU encoder/phase counter input (TCLK).

Configures a pin for TPU phase counting mode input, used with quadrature encoders on the rear wheels. Sets PSEL = k_psel_mtu_phase (0x03), which is the generic “timer phase counting input” function on the RX72N. The routing to TPU (vs MTU) depends on which timer module has phase counting mode enabled.

This function exists as a separate API from rx_mpc_set_mtu_encoder() for documentation clarity, even though both write the same PSEL value.

Convenience wrapper that configures a pin for TPU phase counting mode using PSEL = k_psel_mtu_phase (0x03). On the RX72N, PSEL 0x03 is the generic “timer phase counting input” function; routing to TPU (vs MTU) depends on which timer module has phase counting mode enabled.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	pin	GPIO pin identifier for TPU encoder input Must be a pin that supports TCLK function Configure in pairs (A+B) for quadrature
in	pin	GPIO pin identifier for TPU encoder input (e.g., PC2/PA3)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for TPU encoder input
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for TPU encoder input
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must support TPU clock/phase function

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination supported by TCLK input

Post: PFS register contains PSEL = 0x03 (timer phase counting)

Post: Pin ready for quadrature encoder input via TPU

Post: PFS register contains k_psel_mtu_phase (0x03) for the specified pin

Post: PWPR locked after operation

Note: Configure both Phase A and Phase B pins for proper operation

Note: Also requires TPU timer configuration in phase counting mode

Note: Thread safety: Not thread-safe

Note: Configure both Phase A and Phase B pins for proper operation] **TPU Encoder Pins (STAR Project - Rear Wheels):** Pin Pair TPU Input Encoder

PC2/PA3 TCLKA/TCLKB Motor 2 (Rear Left)

PC0/PB3 TCLKC/TCLKD Motor 3 (Rear Right)

Example - Rear Wheel Encoder Setup: rx_err_err;

```
err=rx_mpc_set_tpu_encoder(k_rx_pc_2); if(err!=k_rx_ok)returnerr;
err=rx_mpc_set_tpu_encoder(k_rx_pa_3); if(err!=k_rx_ok)returnerr;

err=rx_mpc_set_tpu_encoder(k_rx_pc_0); if(err!=k_rx_ok)returnerr;
err=rx_mpc_set_tpu_encoder(k_rx_pb_3); if(err!=k_rx_ok)returnerr;
```

Example:

```
//ConfigurePC2andPA3forTPUphasecounting(TCLKA/B)
rx_err_t err=rx_mpc_set_tpu_encoder(RX_PORT_PIN(0xC,2));
if(err==k_rx_ok){
err=rx_mpc_set_tpu_encoder(RX_PORT_PIN(0xA,3));
}
```

See also: rx_mpc_set_mtu_encoder() Front wheel encoder pin configuration, rx_mpc_set_peripheral() Underlying implementation, rx_encoder_tpu.h TPU encoder driver using phase counting, rx_mpc.h Full API documentation, rx_encoder_tpu.h TPU encoder driver using phase counting

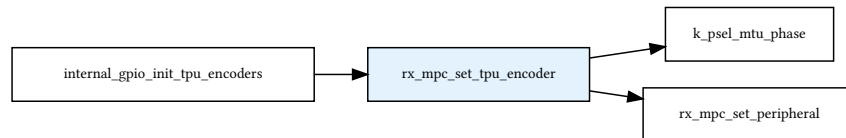


Figure 697: Call graph for rx_mpc_set_tpu_encoder

Defined in `libs/rx_hal/inc/rx_mpc.h:1029`

Since Version 1.1.0

—

rx_mpc_set_adc

```
rx_err_t rx_mpc_set_adc(rx_port_pin_t pin)
```

Configure pin for ADC analog input.

Configures a pin for analog-to-digital converter input by setting the ASEL bit in the PFS register. This disables the digital input buffer to prevent noise and current leakage during analog measurement.

Configures a pin for analog-to-digital conversion by setting the ASEL bit in the PFS register. This disables the digital input buffer to prevent noise during analog measurement.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for ADC input Must be a pin connected to ADC channel Typically P40-P47 for AN000-AN007
in	pin	GPIO pin identifier for ADC (typically P40-P47)

Returns: Error code indicating success or failure

Value	Description
-------	-------------

k_rx_ok	Pin successfully configured for ADC input
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for ADC input
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must be connected to ADC channel (not all pins support ADC)

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with analog capability

Post: PFS register has ASEL = 1 (digital buffer disabled)

Post: Pin ready for analog signal input

Post: PFS register contains k_pfs_asel (0x80) for the specified pin

Post: PWPR locked after operation

Note: Digital input buffer disabled - do not use as GPIO input

Note: Also requires S12ADC configuration for actual conversion

Note: Thread safety: Not thread-safe

Note: Sets ASEL bit (0x80) rather than PSEL field

Note: Also configure S12ADC for actual conversion

Warning: Using a non-ADC pin with this function sets ASEL but has no effect on analog capability (pin hardware determines ADC availability)] **ADC Pin Configuration:** * PFS Register for ADC: * +---+---+---+---+---+---+ * | ASEL | ISEL | Rsvd | PSEL[4:0] | * | 1 | 0 | 0 | 0x00 | * +---+---+---+---+---+---+ * ^ * +- Set to 1 for analog input *

Common ADC Pins (STAR Project): Pin ADC Channel Typical Use

P40 AN000 Motor 0 current sense

P41 AN001 Motor 1 current sense

P42 AN002 Motor 2 current sense

P43 AN003 Motor 3 current sense

P44 AN004 Analog input (unused)

P45 AN005 Temperature sensor

Example - Current Sensing Setup: rx_mpc_set_adc(k_port_4_pin_0);
 rx_mpc_set_adc(k_port_4_pin_1); rx_mpc_set_adc(k_port_4_pin_2);
 rx_mpc_set_adc(k_port_4_pin_3);

Example:

```
//ConfigureP40(AN000)forADCAnaloginput
rx_err_t err=rx_mpc_set_adc(RX_PORT_PIN(4,0));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx72n_adc_regs.h ADC register definitions, rx_mpc.h Full API documentation

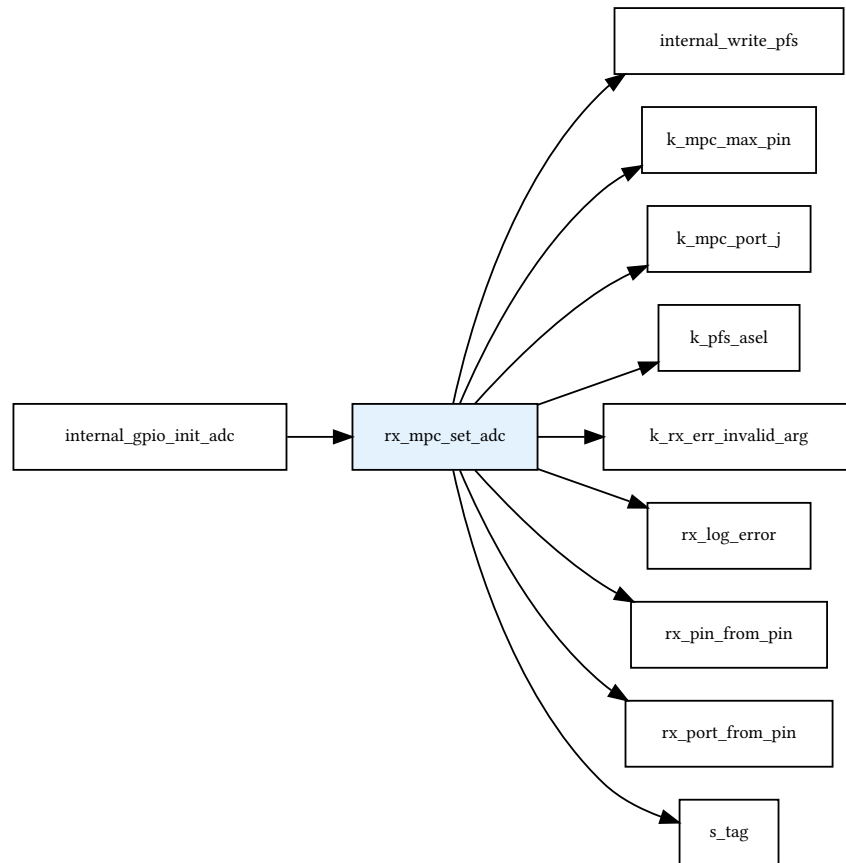


Figure 698: Call graph for rx_mpc_set_adc

Defined in `libs/rx_hal/inc/rx_mpc.h:1097`

Since Version 1.0.0

—

rx_mpc_set_sci

```
rx_err_t rx_mpc_set_sci(rx_port_pin_t pin)
```

Configure pin for SCI (Serial Communication Interface) UART function.

Configures a pin for SCI UART operation by setting PSEL = 0x0A. Works for both TXD (transmit) and RXD (receive) pins - the same PSEL value is used, with the specific function determined by the pin's hardware connection.

Convenience wrapper that configures a pin for SCI UART operation using PSEL = k_psel_sci_tx (0x0A). Works for both TXD and RXD pins.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for SCI function Must be a pin that supports SCI TXD/RXD
in	pin	GPIO pin identifier for SCI (e.g., PB7 for TXD9)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for SCI UART
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for SCI UART
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must support SCI function (check hardware manual)

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with SCI multiplexing

Post: PFS register contains PSEL = 0x0A

Post: Pin ready for UART operation

Post: PFS register contains k_psel_sci_tx (0x0A) for the specified pin

Post: PWPR locked after operation

Note: Configure both TXD and RXD pins for bidirectional UART

Note: Also requires SCI channel configuration and PORT PMR=1

Note: Thread safety: Not thread-safe

Note: Configure both TXD and RXD pins for bidirectional UART] **STAR Project UART Configuration (Debug Console):** Pin SCI Channel Function Notes

PB7 SCI9 TXD9 Debug output

PB6 SCI9 RXD9 Debug input

Example - Debug UART Setup: rx_mpc_set_sci(k_port_b_pin_7);
rx_mpc_set_sci(k_port_b_pin_6);

Example:

```
//ConfigurePB7forSCI9TXD
rx_err_t err=rx_mpc_set_sci(RX_PORT_PIN(0xB,7));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx72n_sci_regs.h SCI register definitions, rx_mpc.h Full API documentation

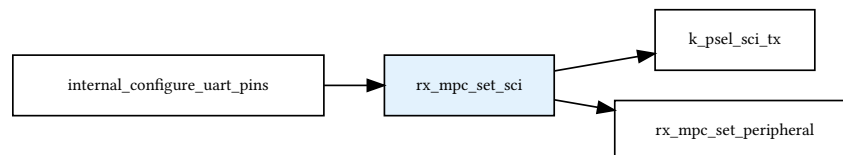


Figure 699: Call graph for rx_mpc_set_sci

Defined in `libs/rx_hal/inc/rx_mpc.h:1145`

Since Version 1.0.0

—

rx_mpc_set_riic

```
rx_err_t rx_mpc_set_riic(rx_port_pin_t pin)
```

Configure pin for RIIC (I2C) bus function.

Configures a pin for RIIC (Renesas I2C) operation by setting PSEL = 0x0F. Works for both SCL (clock) and SDA (data) pins. RIIC hardware automatically configures the open-drain output when the channel is enabled.

Convenience wrapper that configures a pin for RIIC operation using PSEL = `k_psel_riic_scl` (0x0F). Works for both SCL and SDA pins.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for RIIC function Must be a pin that supports RIIC SCL/SDA
in	pin	GPIO pin identifier for RIIC (e.g., P12 for SCL)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured for RIIC I2C
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for RIIC I2C
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must support RIIC function

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with RIIC multiplexing

Post: PFS register contains PSEL = 0x0F

Post: Pin ready for I2C operation

Post: PFS register contains k_psel_riic_scl (0x0F) for the specified pin

Post: PWPR locked after operation

Note: Configure both SCL and SDA pins

Note: External pull-up resistors required (typically 4.7kOhm)

Note: Also requires RIIC channel configuration

Note: Thread safety: Not thread-safe

Note: Configure both SCL and SDA pins; external pull-ups required] **I2C Bus Pins:** Pin RIIC Channel Function Notes

P12 RIIC0 SCL0 Clock line

P13 RIIC0 SDA0 Data line

Example - I2C Bus Setup: rx_mpc_set_riic(k_port_1_pin_2); rx_mpc_set_riic(k_port_1_pin_3);

Example:

```
//ConfigureP12(SCL)andP13(SDA)forRIIC0
rx_err_t err=rx_mpc_set_riic(RX_PORT_PIN(1,2));
if(err==k_rx_ok){
err=rx_mpc_set_riic(RX_PORT_PIN(1,3));
}
```

See also: rx72n_riic_regs.h RIIC register definitions, rx_mpc.h Full API documentation

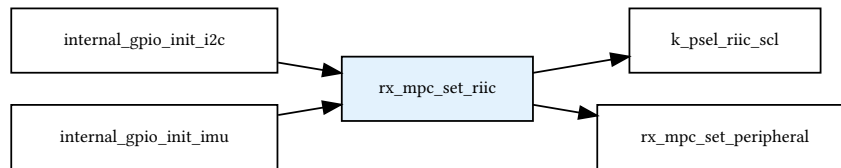


Figure 700: Call graph for rx_mpc_set_riic

Defined in libs/rx_hal/inc/rx_mpc.h:1193

Since Version 1.0.0

—

rx_mpc_set_rspi

```
rx_err_t rx_mpc_set_rspi(rx_port_pin_t pin)
```

Configure pin for RSPI (SPI) bus function.

Configures a pin for RSPI (Renesas SPI) operation by setting PSEL = 0x0D. Works for all SPI signals: RSPCK (clock), COPI (data out), CIPO (data in), and SSLn (chip select).

Convenience wrapper that configures a pin for RSPI operation using PSEL = k_psel_rsapi_clk (0x0D). Works for all SPI signals (CLK, COPI, CIPO, SSL).

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for RSPI function Must be a pin that supports RSPI signals
in	pin	GPIO pin identifier for RSPI (e.g., PA0-PA4)

Returns: Error code indicating success or failure

Value	Description
-------	-------------

k_rx_ok	Pin successfully configured for RSPI SPI
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_ok	Pin configured for RSPI SPI
k_rx_err_invalid_arg	Invalid port or pin

Pre: Pin must support RSPI function

Pre: PCLKB clock must be running

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with RSPI multiplexing

Post: PFS register contains PSEL = 0x0D

Post: Pin ready for SPI operation

Post: PFS register contains k_psel_rspi_clk (0x0D) for the specified pin

Post: PWPR locked after operation

Note: Configure all 4 SPI pins (CLK, COPI, CIPO, CS)

Note: Also requires RSPI channel configuration

Note: Uses OSHWA-approved inclusive terminology (COPI/CIPO)

Note: Thread safety: Not thread-safe

Note: Configure all 4 SPI pins (CLK, COPI, CIPO, CS)

Note: Uses OSHWA-approved inclusive terminology] **STAR Project SPI Configuration (RPI5 Communication):** Note: RSPI0 Signal names (MOSI/MISOA) are RX72N datasheet hardware names. Function names (COPI/CIPO) are project terminology per OSHWA standards.

SPI Bus Topology: * RX72N (Controller) RPi5 (Controller) * +-----+ +-----+ * | RSPI0 | | SPI | * | PA0 (COPI) --+-----+> CIPO | * | PA1 (CIPO) <--+-----+ COPI | * | PA3 (CLK) <--+-----+ CLK | * | PA4 (CS) <--+-----+ CS | * +-----+ +-----+ * Note: RX72N is SPI peripheral, RPi5 is controller *

Example - SPI Bus Setup: rx_mpc_set_rsipi(k_port_a_pin_0); rx_mpc_set_rsipi(k_port_a_pin_1);
rx_mpc_set_rsipi(k_port_a_pin_3); rx_mpc_set_rsipi(k_port_a_pin_4);

Example:

```
//ConfigurePA0-PA4forRSPI0(CLK,COPI,CIP0,SSL0,SSL1)
rx_err_t err=rx_mpc_set_rsipi(RX_PORT_PIN(0xA,0)); //CLK
if(err==k_rx_ok){
err=rx_mpc_set_rsipi(RX_PORT_PIN(0xA,1)); //COPI
}
```

See also: rx72n_rsipi_regs.h RSPI register definitions, rx_spi_comm.h SPI communication driver, rx_mpc.h Full API documentation, rx_spi_comm.h SPI communication driver

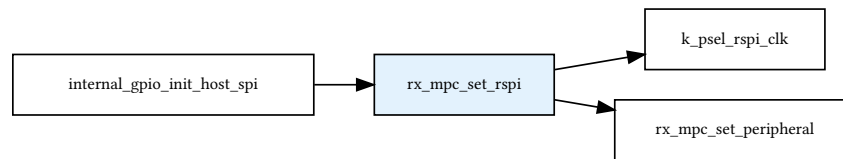


Figure 701: Call graph for rx_mpc_set_rsipi

Defined in `libs/rx_hal/inc/rx_mpc.h:1262`

Since Version 1.0.0

—

rx_mpc_set_gptw

```
rx_err_t rx_mpc_set_gptw(rx_port_pin_t pin)
```

Configure pin for GPTW (General PWM Timer) complementary output.

Configures a pin for GPTW operation by setting PSEL = 0x14. GPTW provides complementary PWM output with dead-time insertion for motor control applications.

GPTW channels support:

- 20 kHz PWM frequency (ultrasonic, inaudible)
- 1 us dead-time (prevents DRV8263H shoot-through)
- 90-degree phase staggering (reduces peak current)
- IN2/IN1 mode (direction and PWM signals per DRV8263H)

Configure pin for GPTW (General PWM Timer) complementary output.

Convenience wrapper that configures a pin for GPTW operation using PSEL = 0x14. GPTW provides complementary PWM output with dead-time insertion for motor control.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for GPTW function Must be a pin that supports GPTW output (see manual)
in	pin	GPIO pin identifier for GPTW function

Returns: Error code from rx_mpc_set_peripheral()

Value	Description
k_rx_ok	Pin successfully configured for GPTW
k_rx_err_null_ptr	Config pointer is NULL (should never happen)
k_rx_err_invalid_arg	Invalid port or pin number (≥ 16)
k_rx_err_hw_error	PWPR write-protect unlock failed
k_rx_ok	Pin configured successfully
k_rx_err_invalid_arg	Invalid port or pin
k_rx_err_hw_error	PWPR unlock failed

Pre: Pin must support GPTW function (check RX72N hardware manual)

Pre: PCLKB clock must be running (MPC registers require clock)

Pre: Must be called during single-threaded initialization

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with GPTW multiplexing

Post: PFS register configured with PSEL = 0x14

Post: PMR bit set (peripheral mode, not GPIO)

Post: PWPR register locked (write protection re-enabled)

Post: Pin ready for GPTW complementary PWM output

Post: PFS register contains k_psel_gptw (0x14) for the specified pin

Post: PWPR locked after operation

Note: Thread Safety: Not thread-safe. Call during initialization only.

Note: Configure both direction (A/IN2) and PWM (B/IN1) pins for each motor

Note: GPTW channel configuration required for actual PWM operation

Note: Thread safety: Not thread-safe

Note: Used for 4 GPTW channels (0-3) with 8 total pins (IN2 + IN1 per motor)

Note: Phase staggering configured separately via rx_gptw driver

Warning: Motor safety: Configure pins before enabling GPTW channels] **STAR Project GPTW Pin Allocation:** Pin GPTW Ch Signal Motor Description

P2.3 GPTW0 GTIOC0A 0 Motor 0 Direction (IN2)

P1.7 GPTW0 GTIOC0B 0 Motor 0 PWM (IN1)

P2.2 GPTW1 GTIOC1A 1 Motor 1 Direction (IN2)

PC.3 GPTW1 GTIOC1B 1 Motor 1 PWM (IN1)

PE.3 GPTW2 GTIOC2A 2 Motor 2 Direction (IN2)

P8.6 GPTW2 GTIOC2B 2 Motor 2 PWM (IN1)

PE.7 GPTW3 GTIOC3A 3 Motor 3 Direction (IN2)

PC.6 GPTW3 GTIOC3B 3 Motor 3 PWM (IN1)

Example - Configure All Motor PWM Pins:

```
constrx_port_pin_tgptw_pins[]={ k_rx_p2_3,k_rx_p1_7, k_rx_p2_2,k_rx_pc_3,
k_rx_pe_3,k_rx_p8_6, k_rx_pe_7,k_rx_pc_6 };
```

```
for(uint8_ti=0;i<8;i++){ rx_err_terr=rx_mpc_set_gptw(gptw_pins[i]); if(err!=k_rx_ok)
{ rx_log_error("GPIO","GPTWpinconfigfailed"); returnerr; } }
```

```
rx_gptw_init_all_staggered(&config);
```

NASA Power of 10 Compliance: Rule 1: [OK] No goto, setjmp, recursion (delegates to rx_mpc_set_peripheral) Rule 3: [OK] No dynamic allocation (stack-only config struct) Rule 4: [OK] Function is 6 lines (well under 60 line limit) Rule 7: [OK] Return value from rx_mpc_set_peripheral checked by caller

Example:

```
//Configuremotor2GPTWIN2andIN1pins(PortE)
rx_err_terr=rx_mpc_set_gptw(RX_PORT_PIN(0xE,3)); //PE3GTIOC2A(IN2)
if(err==k_rx_ok){
err=rx_mpc_set_gptw(RX_PORT_PIN(8,6)); //P8.6GTIOC2B(IN1)
}
```

See also: rx_mpc_set_peripheral() Core pin configuration function,
rx_gptw_init_all_staggered() GPTW timer initialization, RX72N Manual Chapter 23 - Multi-Function Pin Controller, RX72N Manual Chapter 29 - General PWM Timer, rx_mpc.h Full API documentation with usage examples, rx_gptw_init_all_staggered() GPTW channel configuration

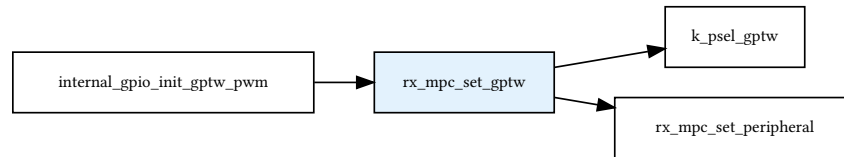


Figure 702: Call graph for rx_mpc_set_gptw

Defined in `libs/rx_hal/inc/rx_mpc.h:1349`

Since Version 1.1.0

—

rx_mpc_set_usb_vbus

```
rx_err_t rx_mpc_set_usb_vbus(rx_port_pin_t pin)
```

Configure pin for USB VBUS detection function.

Configures a pin for USB VBUS detection by setting PSEL = 0x11. VBUS detection is used for USB enumeration and power presence sensing.

The VBUS detect pin monitors the 5V power rail on the USB bus:

- Pin reads HIGH (1) when USB cable connected and 5V present
- Pin reads LOW (0) when USB cable disconnected or unpowered

Required for USB CDC communication with RPi5 host.

Configure pin for USB VBUS detection function.

Convenience wrapper that configures a pin for USB VBUS detection using PSEL = 0x11. Required for USB CDC enumeration and power monitoring.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for USB VBUS detect Typically P1.6 for USB0_VBUS on RX72N
in	pin	GPIO pin identifier for USB VBUS (typically P1.6)

Returns: Error code from rx_mpc_set_peripheral()

Value	Description
k_rx_ok	Pin successfully configured for USB VBUS
k_rx_err_null_ptr	Config pointer is NULL (should never happen)
k_rx_err_invalid_arg	Invalid port or pin number
k_rx_err_hw_error	PWPR write-protect unlock failed
k_rx_ok	Pin configured successfully
k_rx_err_invalid_arg	Invalid port or pin
k_rx_err_hw_error	PWPR unlock failed

Pre: Pin must support USB VBUS function (P1.6 on RX72N)

Pre: PCLKB clock must be running

Pre: Must be called during single-threaded initialization

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with USB VBUS multiplexing

Post: PFS register configured with PSEL = 0x11

Post: PMR bit set (peripheral mode)

Post: PWPR register locked

Post: Pin ready for USB VBUS detection

Post: PFS register contains k_psel_usb_vbus (0x11) for the specified pin

Post: PWPR locked after operation

Note: Thread Safety: Not thread-safe. Call during initialization only.

Note: Active-high detection (pin = 1 when 5V present)

Note: Also requires USB PHY and USB0 controller configuration

Note: Thread safety: Not thread-safe

Note: Active-high VBUS detect (pin = 1 when 5V present)

Note: Also requires USB PHY and controller configuration] **STAR Project USB Configuration:**
Pin Function Description

P1.6 USB0_VBUS VBUS detect (5V presence)

Example - USB VBUS Setup: rx_err_t err=rx_mpc_set_usb_vbus(k_rx_p1_6); if(err!=k_rx_ok)
{ rx_log_error("USB","VBUSpinconfigurationfailed"); returnerr; }
err=usb_init(USB_MODE_CDC);

Example:

```
//ConfigureP16forUSB0_VBUSdetection
rx_err_t rx_mpc_set_usb_vbus(RX_PORT_PIN(1,6));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc_set_peripheral() Core pin configuration function, rx_usb.h USB controller driver, RX72N Manual Chapter 23 - Multi-Function Pin Controller, RX72N Manual Chapter 31 - USB Controller, rx_mpc.h Full API documentation with usage examples, rx_usb.h USB controller driver

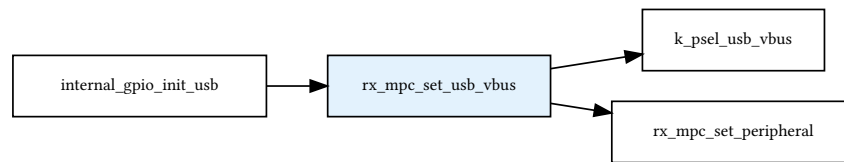


Figure 703: Call graph for rx_mpc_set_usb_vbus

Defined in `libs/rx_hal/inc/rx_mpc.h:1411`

Since Version 1.1.0

—

rx_mpc_set_irq

```
rx_err_t rx_mpc_set_irq(rx_port_pin_t pin)
```

Configure pin for IRQ (external interrupt) function.

Sets pin function to IRQ mode via MPC (Multi-Function Pin Controller). This enables hardware edge detection on IRQ0-IRQ15 pins for use with the Interrupt Controller Unit (ICU).

PFS Register for IRQ Function:

- IRQ pins: ISEL bit (bit 6) = 0x40 written directly to PFS register
- This is NOT a PSEL value; rx_mpc_set_peripheral() is NOT used
- Verify in RX72N Manual Section 20.3 (MPC), PFS register layout

Supported Pins:

- P00-P07: IRQ8-IRQ15
- Other IRQ pins per RX72N Manual Table 20.7

MPC base address remains constant throughout operation

Only target pin's PFS register is modified (other pins' PFS unchanged)

Sets pin function to IRQ mode by writing the ISEL bit (bit 6, value 0x40) directly to the PFS register via `internal_write_pfs()`. This bypasses `rx_mpc_set_peripheral()` because ISEL=0x40 exceeds the 5-bit PSEL field maximum (`k_mpc_psel_max = 31`).

Mirrors the approach used by `rx_mpc_set_adc()` for ASEL (bit 7): both functions write their respective bit directly rather than going through the PSEL validation path.

Algorithm Steps:

1. Extract port number and pin number from encoded pin parameter

2. Validate port in range [0, J]
3. Validate pin in range [0, 7]
4. Write ISEL bit (0x40) to PFS register via `internal_write_pfs()`

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (must support IRQ function) Use <code>k_rx_p0_X</code> constants for IRQ8-15
in	pin	GPIO pin identifier (must support IRQ function, P00-P07)

Returns: Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Pin configured for IRQ function
<code>k_rx_err_invalid_arg</code>	Invalid port or pin number (propagated from <code>internal_write_pfs()</code>)
<code>k_rx_ok</code>	Pin configured for IRQ input mode
<code>k_rx_err_invalid_arg</code>	Invalid port or pin number

Pre: Pin must have IRQ multiplexing capability (P00-P07 for IRQ8-15)

Pre: PCLKB running (MPC requires clock)

Pre: Must be called during single-threaded initialization

Pre: Pin must have IRQ multiplexing capability (P00-P07 for IRQ8-15)

Pre: PCLKB clock must be running

Post: Pin ISEL bit set (0x40) in PFS register (IRQ input mode)

Post: PSEL field remains 0 (no conflicting peripheral function)

Post: PWPR register locked

Post: Pin ready for ICU edge detection configuration

Post: Pin ISEL bit (0x40) set in PFS register

Post: PSEL field remains 0 (no peripheral function conflict)

Note: internal_mpc_unlock() and internal_mpc_lock() do not propagate errors; only internal_write_pfs() can return k_rx_err_invalid_arg on invalid port/pin values.

Note: Thread Safety: Not thread-safe. Call during initialization only.

Note: Call this BEFORE enabling ICU interrupt

Note: Thread safety: Not thread-safe

Note: Typical pins: P00-P07 for IRQ8-IRQ15 on RX72N

Note: Used by HC-SR04 ultrasonic sensors for echo pulse measurement

Warning: Does NOT configure ICU registers (edge detect, priority, etc.)

Warning: Configure ICU separately for edge detection and priority as required] **STAR Project IRQ Usage (HC-SR04 Ultrasonic Sensors):** Pin IRQ Number Sensor Location

P03 IRQ11 Sonar 0 Front-Left

P02 IRQ10 Sonar 1 Front-Right

P01 IRQ9 Sonar 2 Back-Left

P00 IRQ8 Sonar 3 Back-Right

Example - IRQ Pin Setup: rx_err_t err=rx_mpc_set_irq(k_rx_p0_3); if(err!=k_rx_ok)
{ rx_log_error("IRQ","IRQpinconfigurationfailed"); return err; }

NASA Power of 10 Compliance: Rule 5: [OK] 3 preconditions, 4 postconditions Rule 7: [OK] Caller checks return value ([[nodiscard]]) Rule 8: [OK] k_pfs_isel from pfs_bits_t enum (no magic numbers)

Example:

```
//ConfigureP00forIRQ8(HC-SR04echoinput)
rx_err_t err=rx_mpc_set_irq(RX_PORT_PIN(0,0));
if(err!=k_rx_ok){
//handleerror
}
//ThenconfigureICUforrising/fallingedge detection
```

See also: rx_mpc_set_gpio() Set pin back to GPIO mode, RX72N Manual Section 20.3 (MPC), PFS register bit definitions, RX72N Manual Chapter 15 (ICU - Interrupt Controller Unit), rx_mpc_set_adc() Same pattern: writes ASEL bit (0x80) directly, rx_mpc.h Full API documentation with usage examples, rx_hcsr04_icu_configure() Configure ICU after calling this function

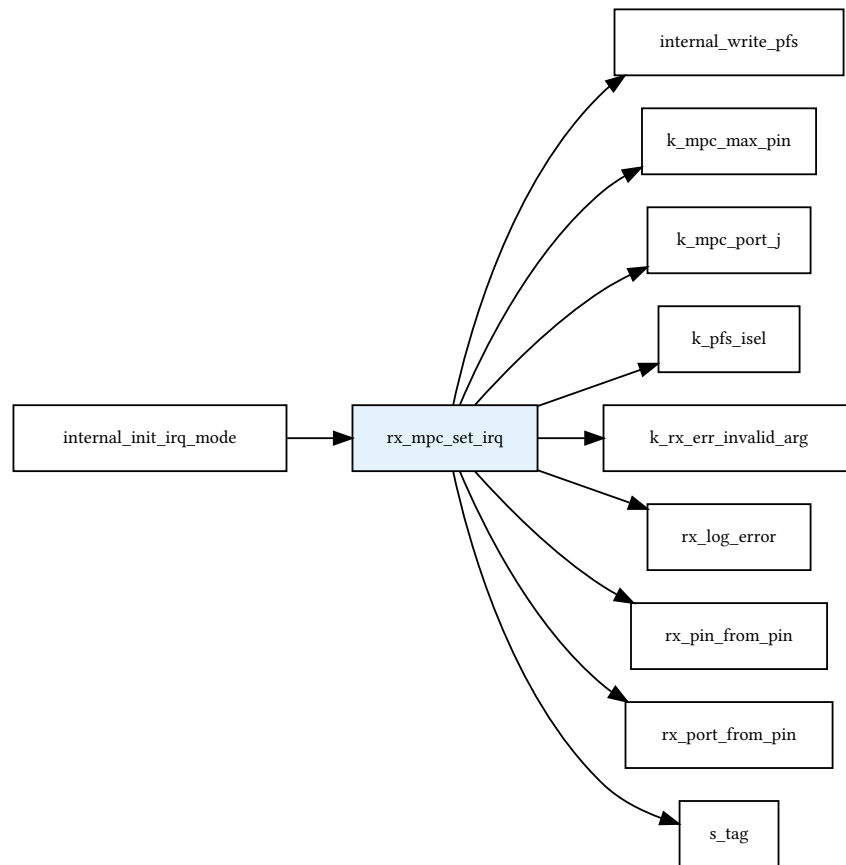


Figure 704: Call graph for rx_mpc_set_irq

Defined in `libs/rx_hal/inc/rx_mpc.h:1492`

Since Version 1.2.0 (Issue 296 - IRQ-based HC-SR04 measurement)

—

rx_mtu.h

MTU PWM Driver for RX72N Multi-Function Timer Unit.

Production-grade PWM driver for the RX72N Multi-Function Timer Unit (MTU3) peripheral. This module provides 16-bit PWM generation for motor control, quadrature encoder input, and general-purpose timing applications.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`

rx_mtu_channel_t

MTU Channel Identifiers.

Identifies the MTU timer channels available on the RX72N. Note that MTU5 does not exist in this MCU family. Channels are grouped for specific functions:

Value	Name	Description
= 0	k_mtu_channel_0	MTU0: General purpose timer. 4 outputs (A,B,C,D). Base: 0x000C1200. Standalone operation
= 1	k_mtu_channel_1	MTU1: Encoder input channel. 2 outputs (A,B). Base: 0x000C1280. Phase counting with MTU2
= 2	k_mtu_channel_2	MTU2: Encoder input channel. 2 outputs (A,B). Base: 0x000C1300. Phase counting with MTU1
= 3	k_mtu_channel_3	MTU3: General purpose timer. 4 outputs (A,B,C,D). Base: 0x000C1100. Pairs with MTU4
= 4	k_mtu_channel_4	MTU4: General purpose timer. 4 outputs (A,B,C,D). Base: 0x000C1100. Pairs with MTU3
= 6	k_mtu_channel_6	MTU6: General purpose timer. 4 outputs (A,B,C,D). Base: 0x000C1A00. Pairs with MTU7
= 7	k_mtu_channel_7	MTU7: General purpose timer. 4 outputs (A,B,C,D). Base: 0x000C1A00. Pairs with MTU6

—

rx_mtu_output_t

MTU Output Channel Selection.

Selects which output pin to configure for PWM or compare match output. MTU channels can have 2 or 4 outputs depending on the channel.

Value	Name	Description
= 0	k_mtu_output_a	MTIOCA output pin. Available on all channels. Compare register: TGRA. Primary PWM output
= 1	k_mtu_output_b	MTIOCB output pin. Available on all channels. Compare register: TGRB. Complementary to A when enabled
= 2	k_mtu_output_c	MTIOCC output pin. MTU0,3,4,6,7 only. Compare register: TGRC. Secondary PWM output
= 3	k_mtu_output_d	MTIOCD output pin. MTU0,3,4,6,7 only. Compare register: TGRD. Complementary to C when enabled

—

rx_mtu_init_pwm

```
rx_err_t rx_mtu_init_pwm(rx_mtu_channel_t channel, const rx_mtu_config_t *config)
```

Initialize MTU channel for PWM output.

Configures MTU channel for PWM mode 1 (center-aligned, triangle wave). Enables timer and starts PWM generation at 0% duty cycle.

Initialize MTU channel for PWM output.

Configures an MTU channel for PWM Mode 1 (triangle wave, center-aligned) operation. Performs complete hardware setup including module clock enable, timer configuration, and automatic start.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	config	PWM configuration

Returns: k_rx_err_invalid_state if MTU module not enabled

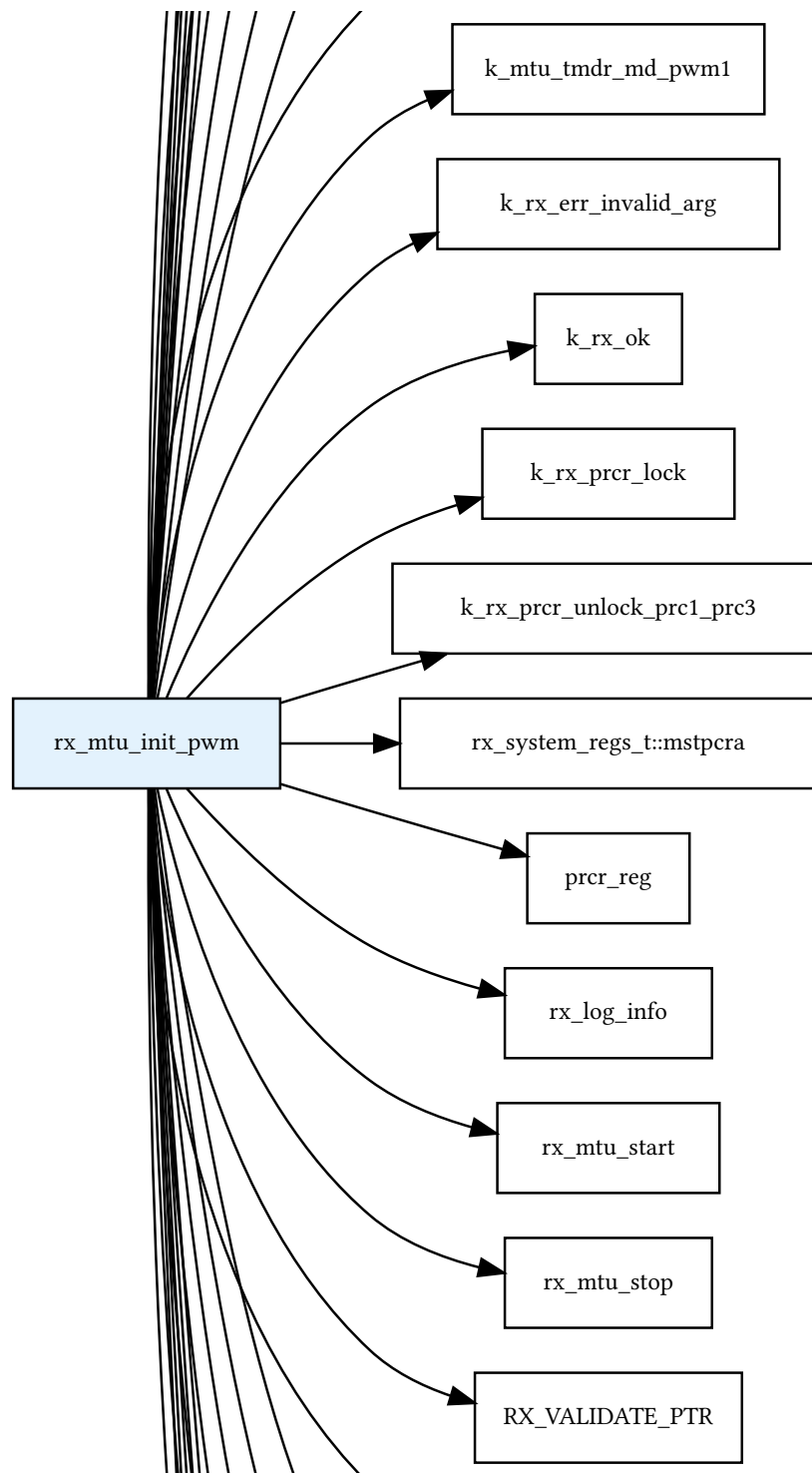


Figure 705: Call graph for rx_mtu_init_pwm

Defined in `libs/rx_hal/inc/rx_mtu.h:469`

—

rx_mtu_set_duty

```
rx_err_t rx_mtu_set_duty(rx_mtu_channel_t channel, rx_mtu_output_t output, float
duty_percent)
```

Set PWM duty cycle.

Updates duty cycle for specified output channel. Change takes effect on next PWM period for glitch-free operation.

Set PWM duty cycle.

Converts a percentage value to raw timer counts using the cached period and writes it to the appropriate TGR register. The update is buffered and takes effect at the next PWM period boundary (glitch-free).

Algorithm:

1. Validate channel and initialization state
2. Validate `duty_percent` in range [0.0, 100.0]
3. Retrieve cached period from `s_mtu_period[channel]`
4. Compute `duty_count = (duty_percent * period) / 100`
5. Write `duty_count` to TGR register via `internal_set_duty_raw_mtu()`

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output channel (A/B/C/D)
in	duty_percent	Duty cycle in percent (0.0 - 100.0)
in	channel	MTU channel (0-4, 6-7)
in	output	Output pin to update (k_mtu_output_a through k_mtu_output_d)
in	duty_percent	Duty cycle percentage [0.0, 100.0] 0.0 = always-off (0% duty) 100.0 = always-on (100% duty)

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	Duty cycle updated successfully
k_rx_err_invalid_arg	Invalid channel or duty_percent out of range
k_rx_err_not_initialized	Channel not initialized via <code>rx_mtu_init_pwm()</code>

Pre: Channel must be initialized via `rx_mtu_init_pwm()`

Pre: `duty_percent` must be in 0.0, 100.0

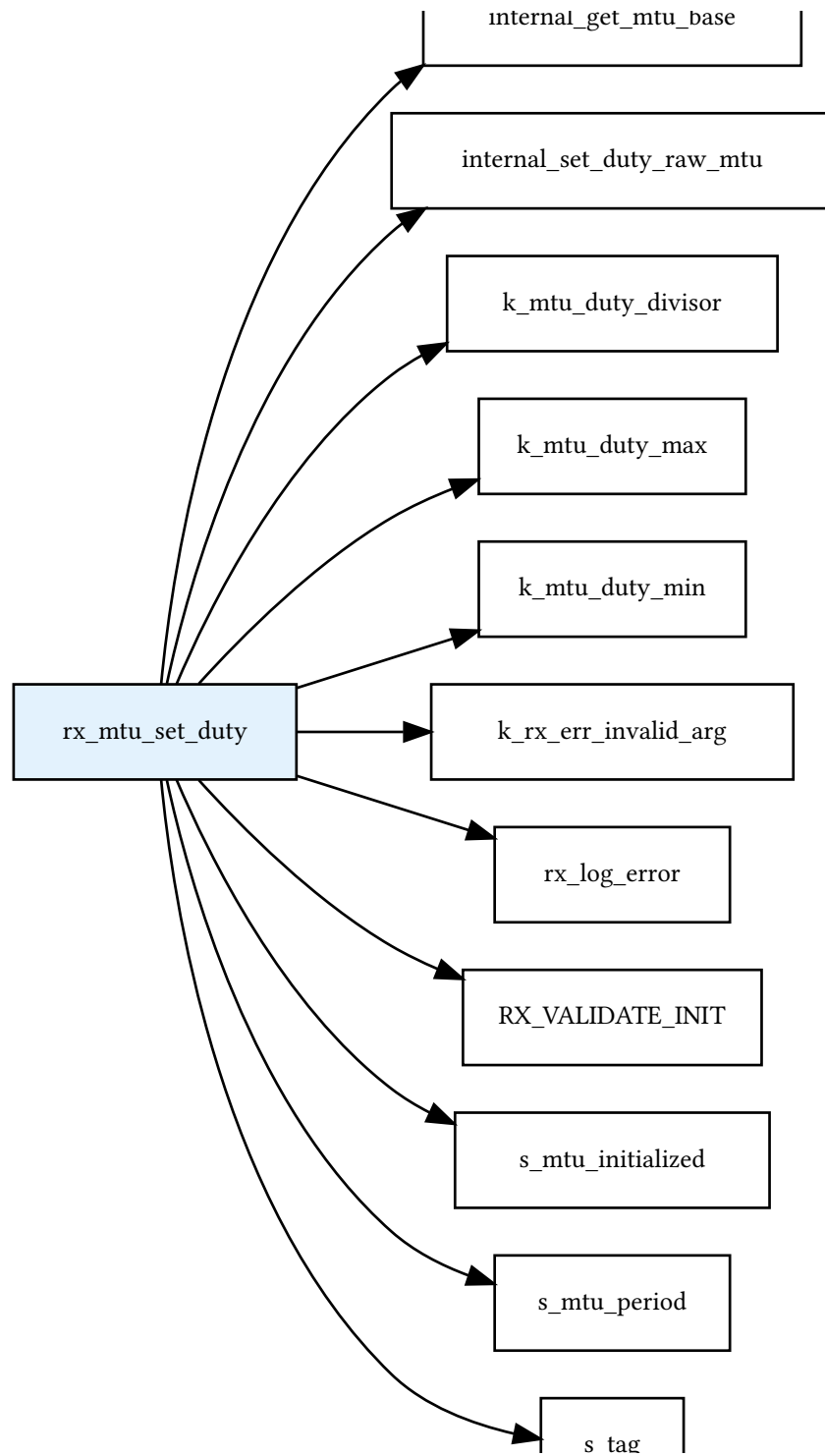
Post: TGR register updated with new duty count

Post: Change takes effect at next PWM period boundary

Note: Not thread-safe: caller must prevent concurrent access] **Example:**

```
rx_err_t err = rx_mtu_set_duty(k_mtu_channel_0, k_mtu_output_b, 50.0f);
```

See also: `rx_mtu_set_duty_raw()` Set duty using raw count value, `rx_mtu_get_duty()` Read current duty cycle

Figure 706: Call graph for `rx_mtu_set_duty`

Defined in `libs/rx_hal/inc/rx_mtu.h:486`

Since Version 1.0.0

—

rx_mtu_set_duty_raw

```
rx_err_t rx_mtu_set_duty_raw(rx_mtu_channel_t channel, rx_mtu_output_t output,
uint16_t duty_count)
```

Set PWM duty cycle (raw count value).

Updates duty cycle using raw counter value for efficiency. Useful in tight control loops where floating-point calculation overhead should be avoided.

Set PWM duty cycle (raw count value).

Directly sets the compare register (TGR) for the specified output to `duty_count` timer ticks. Values larger than the channel period are clamped to the period value (100% duty). The update is buffered.

Use this function when the caller has already converted a duty percentage to counts (e.g. via `rx_mtu_get_period()`) to avoid redundant floating-point multiplication.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output channel (A/B/C/D)
in	duty_count	Duty cycle count (0 - period_count)
in	channel	MTU channel (0-4, 6-7)
in	output	Output pin (k_mtu_output_a through k_mtu_output_d)
in	duty_count	Raw count value in range [0, period] Values > period are automatically clamped to period

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Duty count written successfully
<code>k_rx_err_invalid_arg</code>	Invalid channel or output
<code>k_rx_err_not_initialized</code>	Channel not initialized

Pre: Channel must be initialized via `rx_mtu_init_pwm()`

Pre: `duty_count` should be in 0, period; values above period are clamped

Post: TGR register written with (possibly clamped) `duty_count`

Post: Change takes effect at next PWM period boundary

Note: Not thread-safe: caller must prevent concurrent access] **Example:**

```
uint16_t period;  
rx_mtu_get_period(k_mtu_channel_0, &period);  
rx_mtu_set_duty_raw(k_mtu_channel_0, k_mtu_output_b, period/2U); //50%
```

See also: rx_mtu_set_duty() Set duty using floating-point percentage,
rx_mtu_get_period() Retrieve period for count calculation

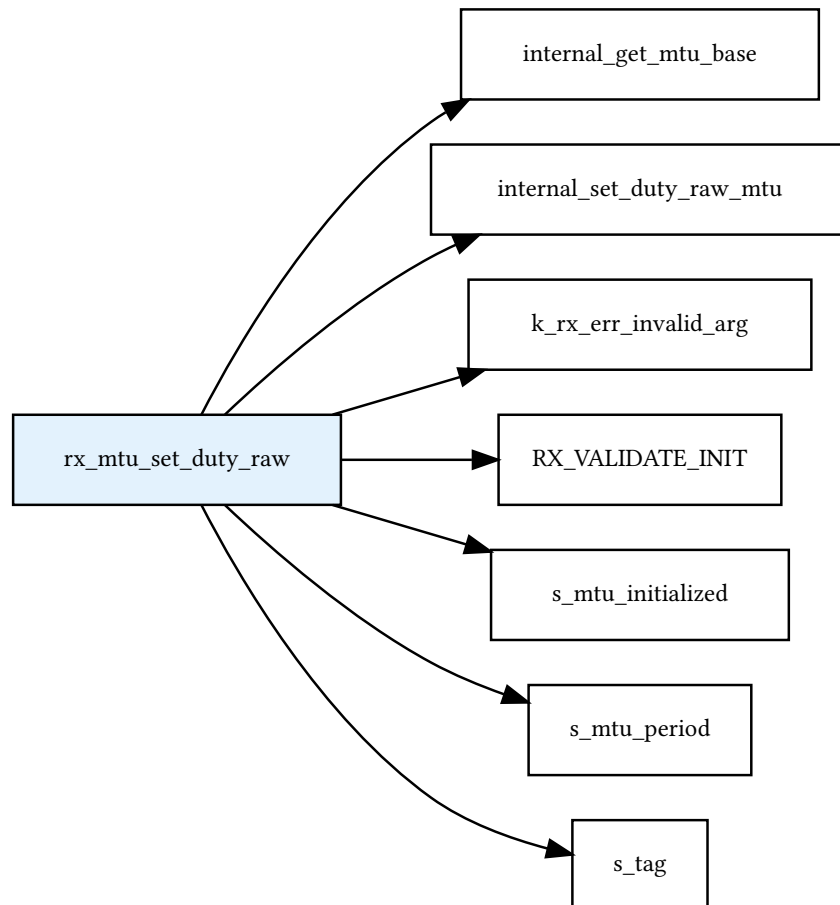


Figure 707: Call graph for rx_mtu_set_duty_raw

Defined in `libs/rx_hal/inc/rx_mtu.h:504`

Since Version 1.0.0

—

rx_mtu_get_duty

```
rx_err_t rx_mtu_get_duty(rx_mtu_channel_t channel, rx_mtu_output_t output, float
*duty_percent)
```

Get current duty cycle.

Get current duty cycle.

Reads the TGR register for the specified output and converts the raw count to a percentage using the cached period value.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output channel (A/B/C/D)
out	duty_percent	Pointer to store duty cycle in percent
in	channel	MTU channel (0-4, 6-7)
in	output	Output to read (k_mtu_output_a through k_mtu_output_d)
out	duty_percent	Pointer to store percentage result [0.0, 100.0] Must be non-NULL

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, duty_percent updated
k_rx_err_null_ptr	duty_percent is nullptr
k_rx_err_invalid_arg	Invalid channel or output
k_rx_err_not_initialized	Channel not initialized

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: duty_percent must point to valid float storage

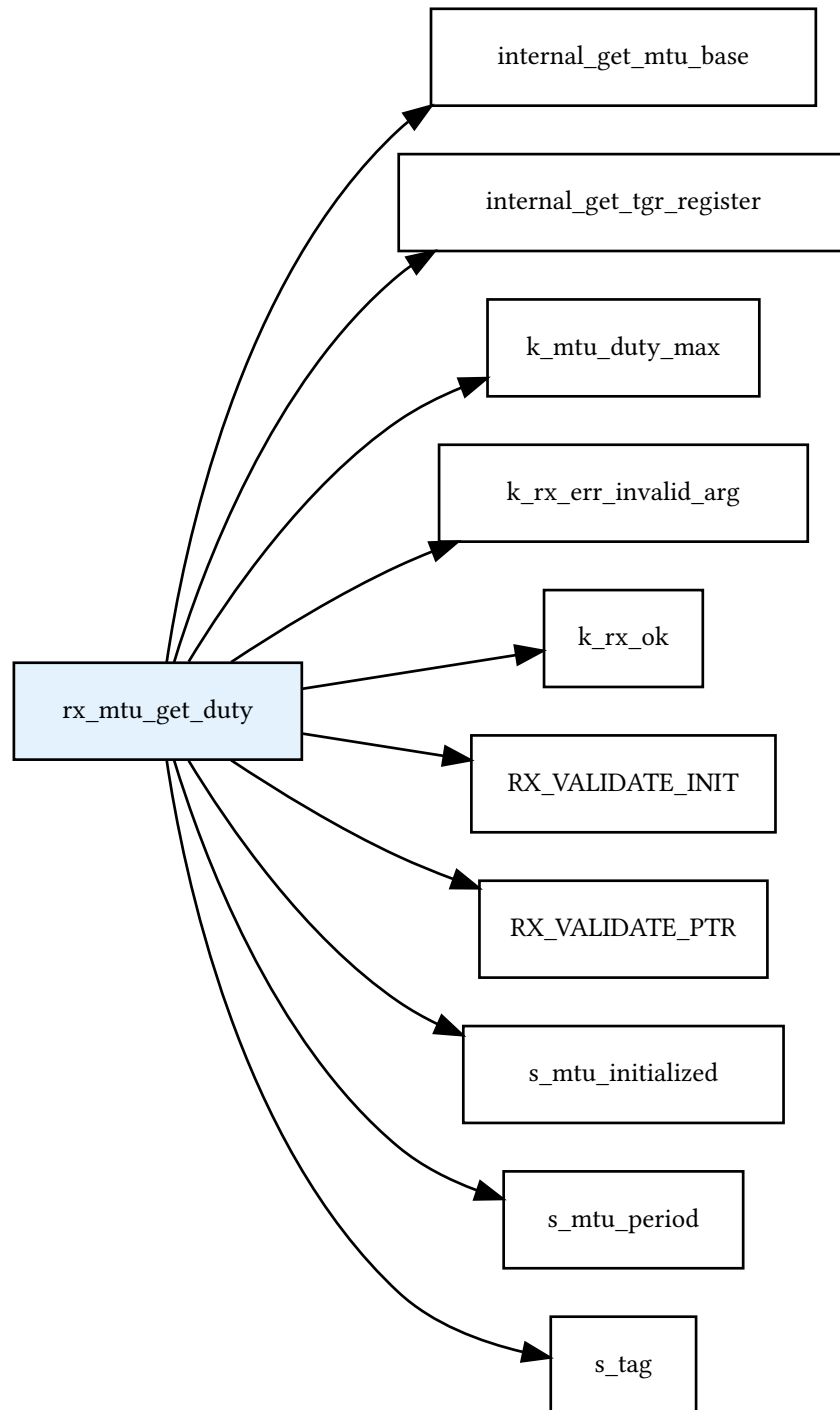
Post: *duty_percent contains duty cycle in 0.0, 100.0 on success

Post: Hardware TGR register unchanged

Note: Not thread-safe: concurrent set_duty calls may cause torn reads] **Example:**

```
float duty;
rx_err_t err = rx_mtu_get_duty(k_mtu_channel_0, k_mtu_output_b, &duty);
```

See also: rx_mtu_set_duty() Set duty using percentage, rx_mtu_get_period() Retrieve period count

Figure 708: Call graph for `rx_mtu_get_duty`

Defined in `libs/rx_hal/inc/rx_mtu.h:519`

Since Version 1.0.0

—

rx_mtu_get_period

```
rx_err_t rx_mtu_get_period(rx_mtu_channel_t channel, uint16_t *period_count)
```

Get PWM period count.

Returns the period register value (useful for raw duty cycle calculations).

Get PWM period count.

Returns the cached period value (TGRA) from initialization. Useful for calculating raw duty counts from percentages without floating-point.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
out	period_count	Pointer to store period count
in	channel	MTU channel to query
out	period_count	Pointer to store period count value

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, period_count contains valid value
k_rx_err_null_ptr	period_count is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_not_initialized	Channel not initialized

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: period_count must point to valid uint16_t storage

Post: *period_count contains the TGRA register value on success

Post: Hardware registers unchanged

Note: Thread-safe: reads from static cache, no hardware access] **Example:**

```
uint16_t period;
rx_err_t err = rx_mtu_get_period(k_mtu_channel_0, &period);
if (err == k_rx_ok) {
```



```
uint16_t half_duty = period / 2U; // 50%
rx_mtu_set_duty_raw(k_mtu_channel_0, k_mtu_output_b, half_duty);
}
```

See also: rx_mtu.h Complete API documentation, rx_mtu_set_duty_raw() Use period count to set duty

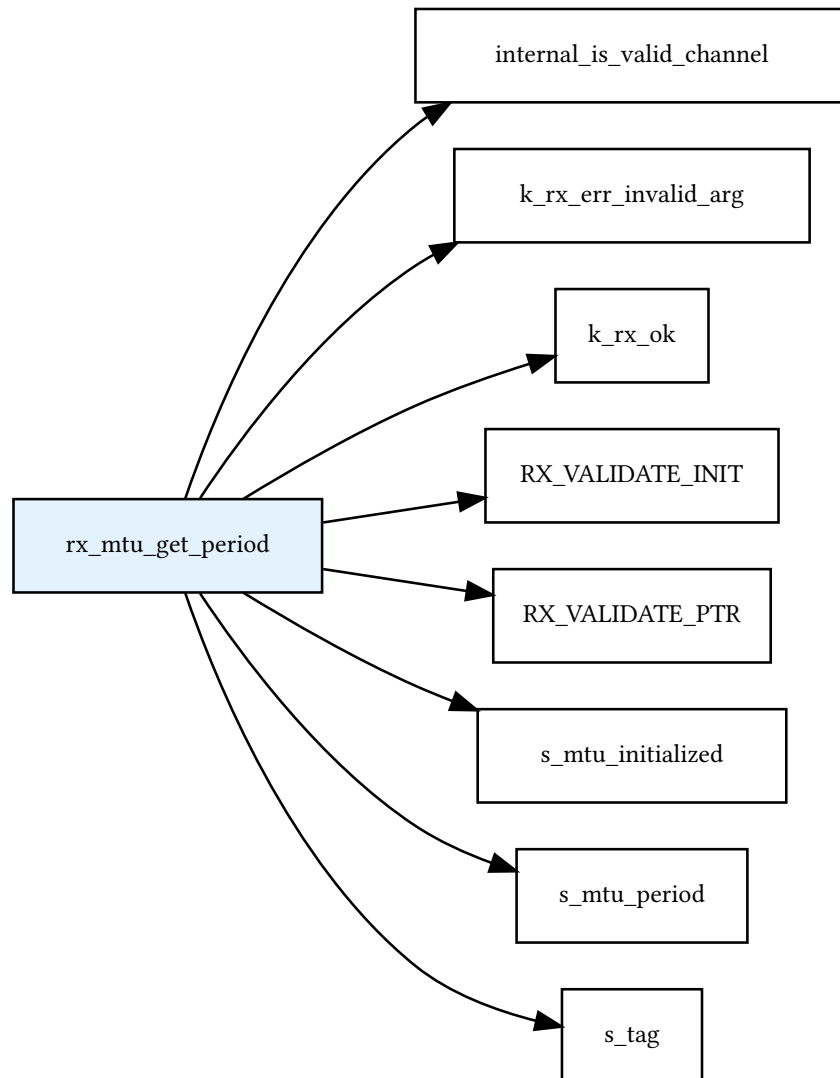


Figure 709: Call graph for rx_mtu_get_period

Defined in `libs/rx_hal/inc/rx_mtu.h:534`

Since Version 1.0.0

—

rx_mtu_enable_output

```
rx_err_t rx_mtu_enable_output(rx_mtu_channel_t channel, rx_mtu_output_t output,
    bool enable)
```

Enable or disable PWM output.

Enable or disable PWM output.

Controls whether the MTIOCA/B/C/D pin actively drives the PWM waveform or is held in its initial state (off). Modifies the TIORH or TIORL register nibble corresponding to the requested output without disturbing other outputs.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output channel (A/B/C/D)
in	enable	True to enable output, false to disable
in	channel	MTU channel (0-4, 6-7)
in	output	Output to control (k_mtu_output_a through k_mtu_output_d)
in	enable	true to enable PWM output, false to disable (hold initial state)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Output state changed successfully
k_rx_err_invalid_arg	Invalid channel or output
k_rx_err_not_initialized	Channel not initialized

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: output must be a valid rx_mtu_output_t enum value

Post: TIORH or TIORL nibble updated; timer continues running

Post: Other outputs on the same channel are unchanged

Note: Not thread-safe: caller must prevent concurrent register access

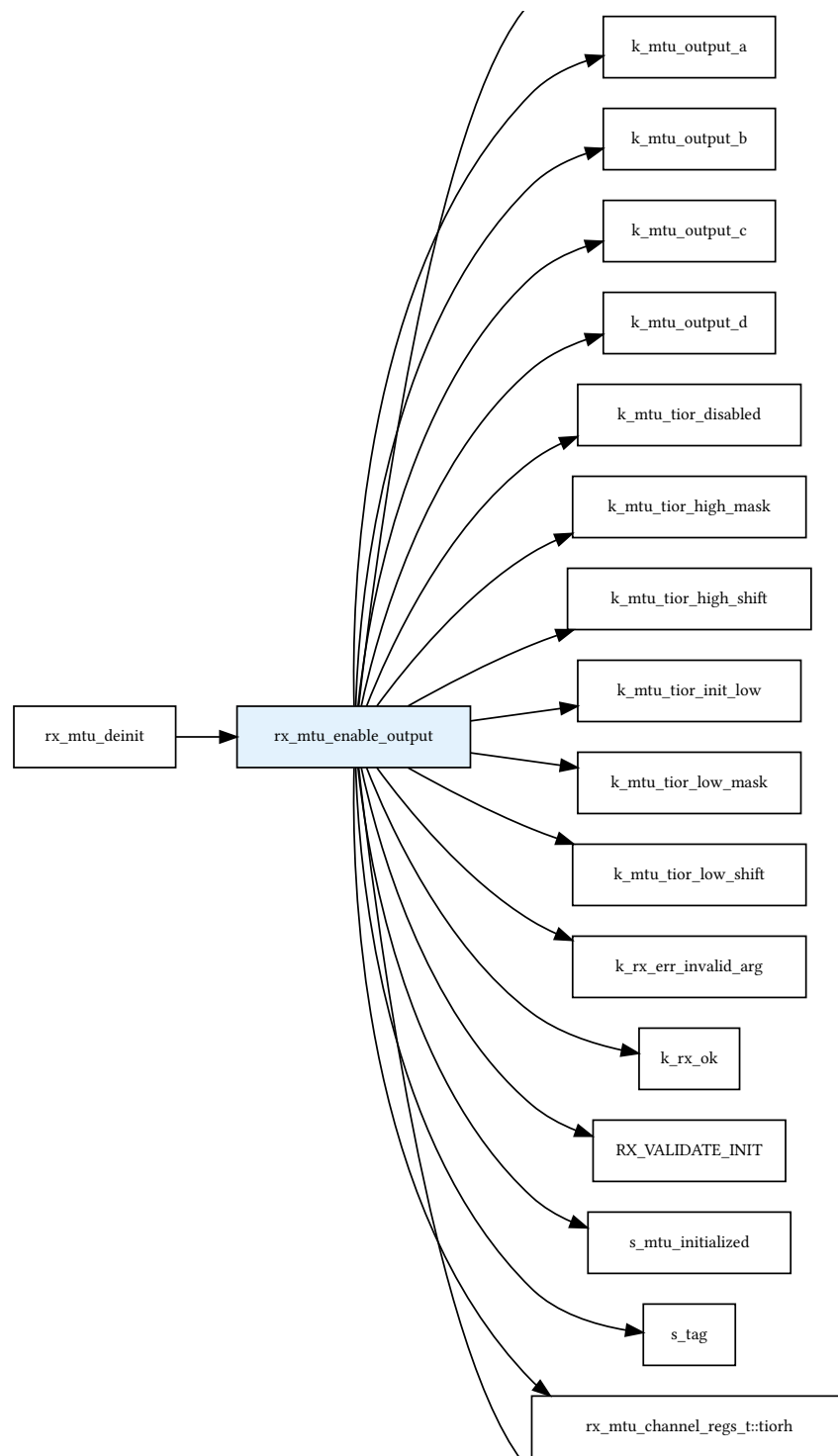
Note: Timer keeps running; only output pin drive changes

Warning: Outputs may remain HIGH when disabled, depending on last compare state]

Example:

```
rx_mtu_enable_output(k_mtu_channel_0,k_mtu_output_a,false);//disable
```

See also: rx_mtu_init_pwm() Initialize channel before calling, rx_mtu_stop() Halt the entire timer

Figure 710: Call graph for `rx_mtu_enable_output`

Defined in `libs/rx_hal/inc/rx_mtu.h:548`

Since Version 1.0.0

—

rx_mtu_start

```
rx_err_t rx_mtu_start(rx_mtu_channel_t channel)
```

Start MTU timer.

Starts the timer counter for PWM generation. Timer is automatically started during init, but can be stopped/restarted.

Start MTU timer.

Sets the Count Start (CSTn) bit in the appropriate TSTR register to begin counter operation. Channels 0-4 use TSTRA, channels 6-7 use TSTRB.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)

Returns: `k_rx_err_invalid_arg` if channel is invalid

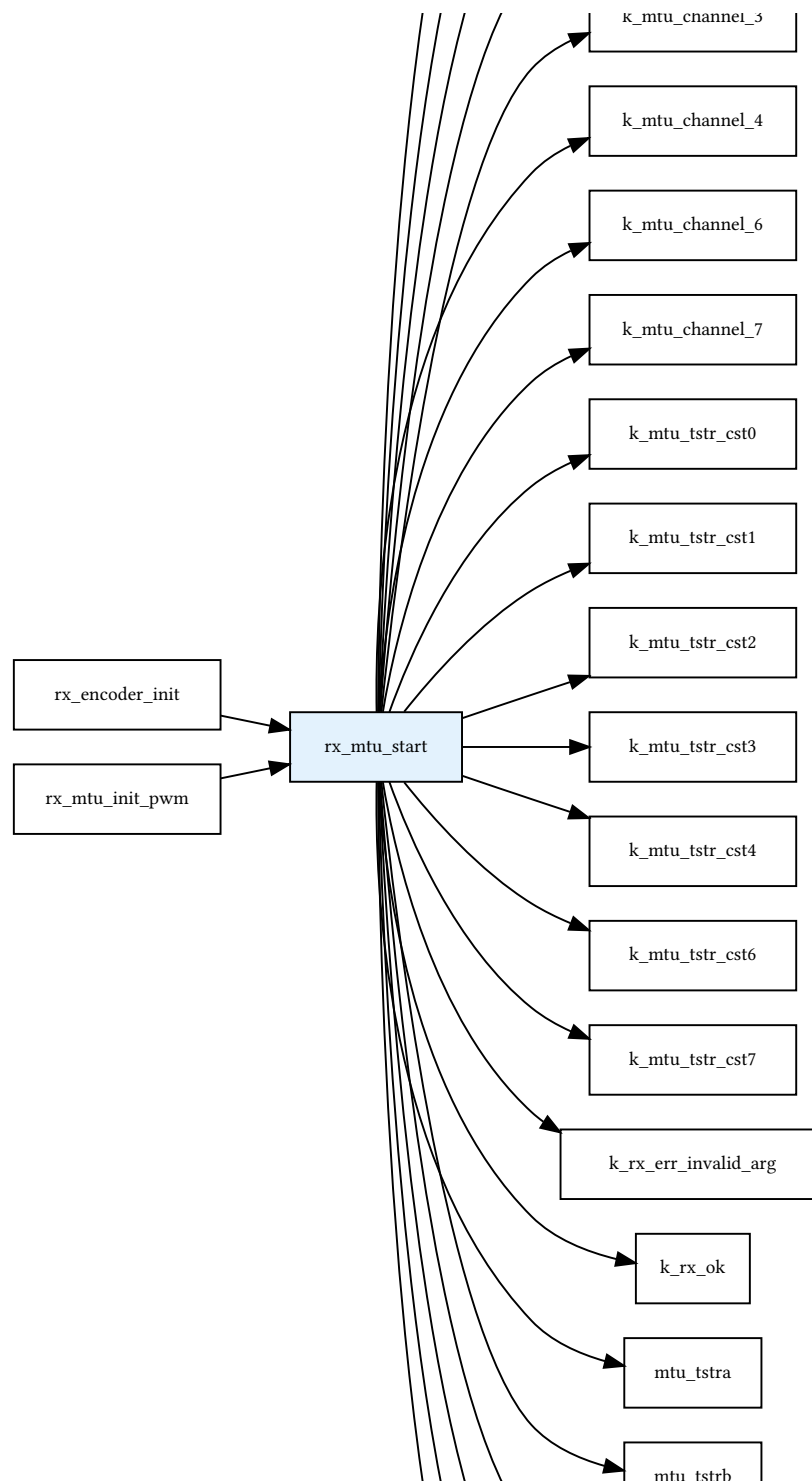


Figure 711: Call graph for rx_mtu_start

Defined in `libs/rx_hal/inc/rx_mtu.h:561`

—

rx_mtu_stop

```
rx_err_t rx_mtu_stop(rx_mtu_channel_t channel)
```

Stop MTU timer.

Stops the timer counter. PWM outputs will hold their last state.

Stop MTU timer.

Clears the Count Start (CSTn) bit in the appropriate TSTR register to halt counter operation. Counter value and output state are preserved.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	channel	MTU channel to stop

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Timer stopped successfully
k_rx_err_invalid_arg	Invalid channel
k_rx_err_hw_error	Failed to clear TSTR bit (hardware issue)

Pre: None (can be called on uninitialized channel)

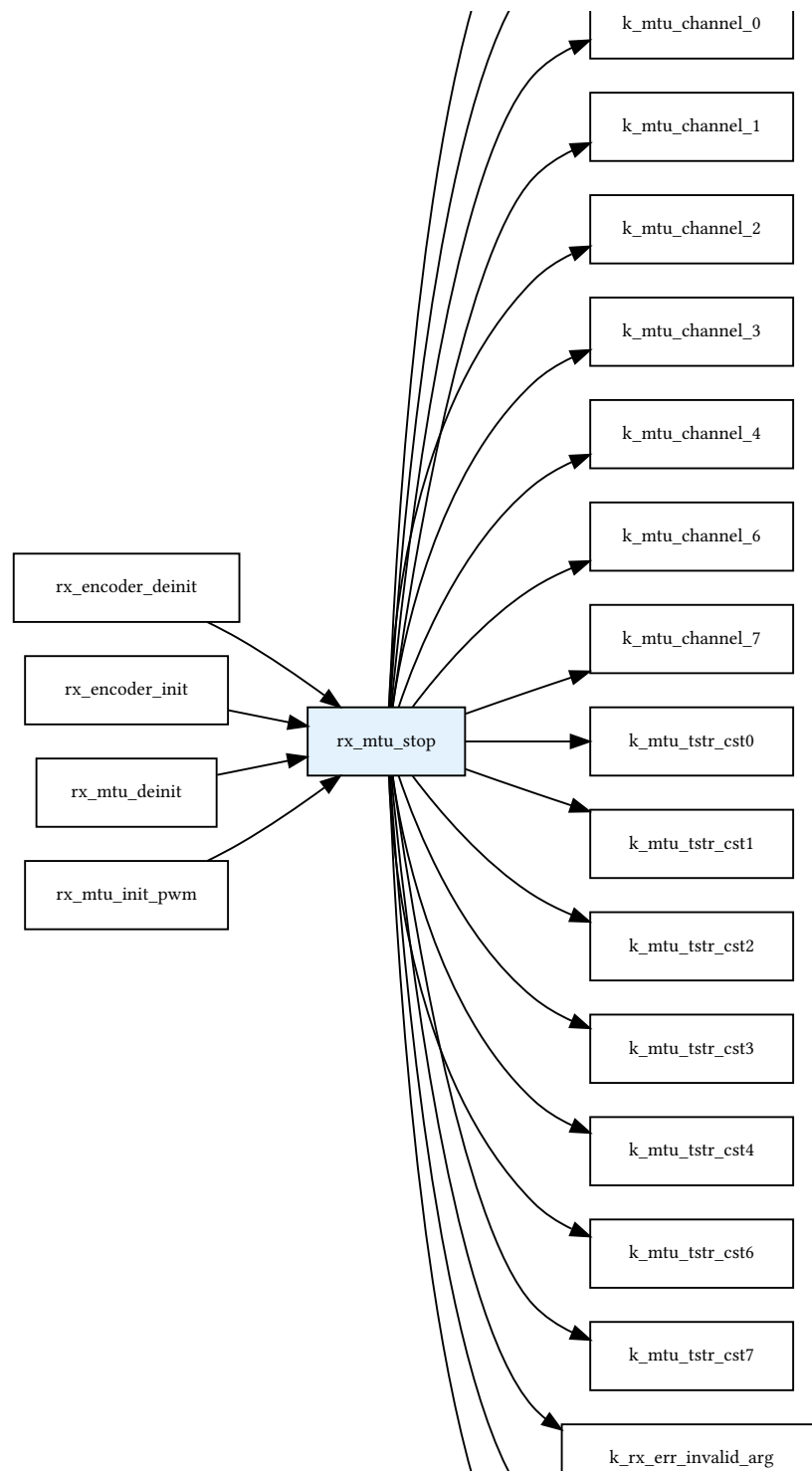
Post: Timer stopped, counter preserved

Post: Output pins hold current state

Note: Does NOT check initialization flag (safe for use during init)

Note: Idempotent: Safe to call when already stopped

Warning: Outputs may remain HIGH - disable outputs for safe state] **See also:** rx_mtu_start()
Resume the timer, rx_mtu_enable_output() Disable outputs

Figure 712: Call graph for `rx_mtu_stop`

Defined in `libs/rx_hal/inc/rx_mtu.h:573`

Since Version 1.0.0

—

rx_mtu_deinit

```
rx_err_t rx_mtu_deinit(rx_mtu_channel_t channel)
```

Deinitialize MTU channel.

Stops timer, disables outputs, and releases resources.

Deinitialize MTU channel.

Performs an orderly shutdown of the specified MTU channel:

1. Stop the timer counter (`rx_mtu_stop()`)
2. Disable all four outputs (A, B, C, D) via `rx_mtu_enable_output()`
3. Clear the cached period to 0
4. Clear the initialized flag

After this call the channel may be safely re-initialized with different parameters via `rx_mtu_init_pwm()`.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	channel	MTU channel to deinitialize (0-4, 6-7)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Channel successfully deinitialized
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_hw_error</code>	Hardware error while stopping timer

Pre: Motor or load driven by this channel must be stopped externally before call

Pre: channel must be a valid MTU channel (0-4, 6-7)

Post: Timer stopped and outputs disabled

Post: `s_mtu_initializedchannel == false`

Post: `s_mtu_periodchannel == k_mtu_period_zero`

Note: Not thread-safe: do not call while another task is accessing this channel

Note: Idempotent with respect to re-initialization (can call init again after)

Warning: Ensure motor is at rest before calling to avoid abrupt stop] **Example:**

```
rx_err_t err = rx_mtu_deinit(k_mtu_channel_0);
```

See also: rx_mtu_init_pwm() Re-initialize after deinit, rx_mtu_stop() Stop timer without full teardown

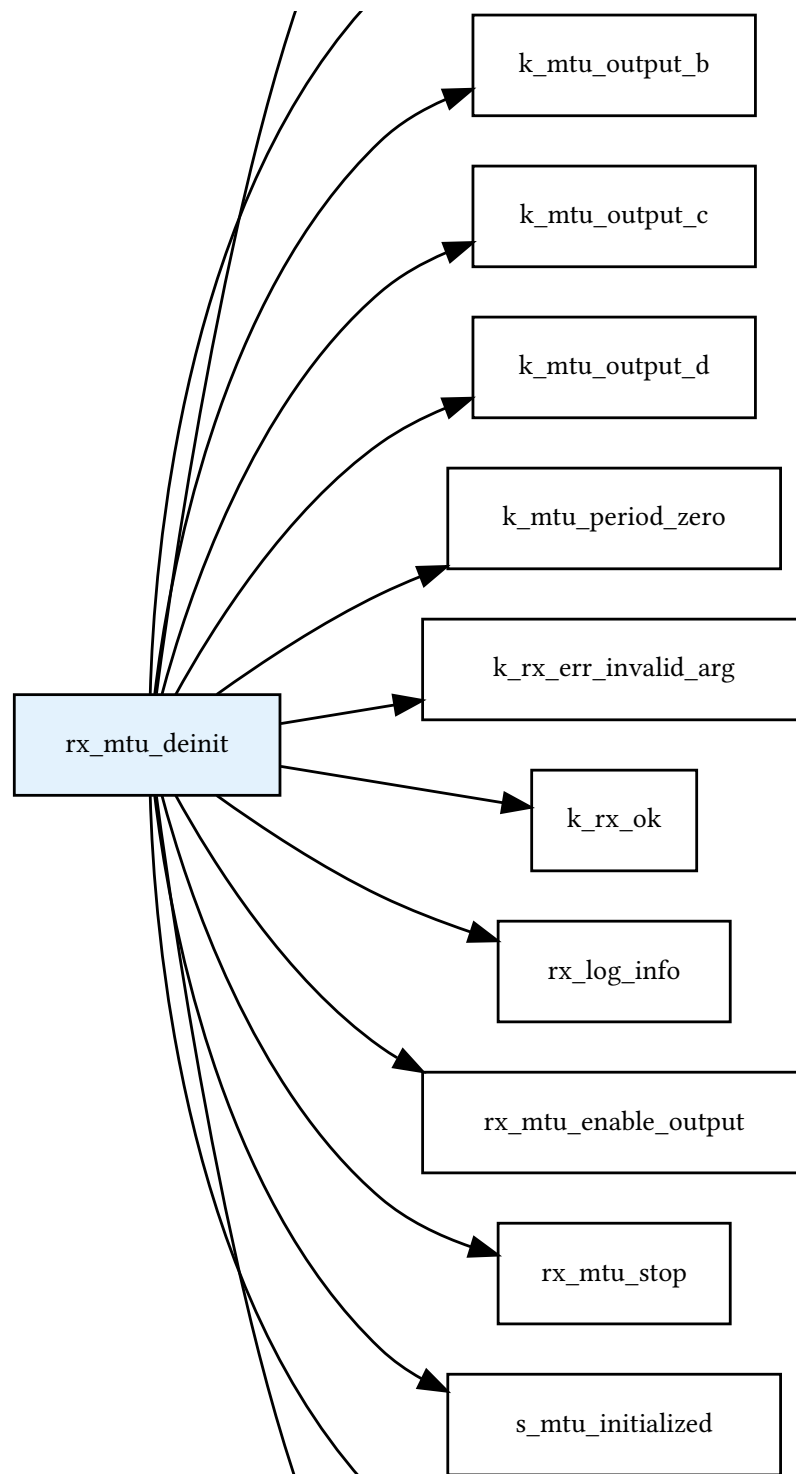


Figure 713: Call graph for rx_mtu_deinit

Defined in `libs/rx_hal/inc/rx_mtu.h:585`

Since Version 1.0.0

—

rx_poeg.h

POEG Motor Fault Protection Driver API.

Provides hardware-level emergency PWM output disable via the POEG module. When a DRV8263H motor driver asserts its nFAULT output (active low), the corresponding GTETRQ pin triggers POEG to immediately disable GPTW PWM outputs, placing the H-bridge in a safe Hi-Z state.

STAR Team

2026-02-10

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`

rx_poeg_motor_count_t

Number of motors with POEG fault protection.

Value	Name	Description
= 4	<code>k_poeg_motor_count</code>	4 motors, one POEG group each (A-D)

Since Version 1.0.0

—

rx_poeg_isr_priority_t

ICU priority for POEG fault interrupts.

Priority 14 (near-maximum) ensures motor faults are handled immediately. Only NMI (level 15) has higher priority.

Value	Name	Description
= 14	<code>k_poeg_isr_priority</code>	High priority for safety-critical fault ISR

Since Version 1.0.0

—

rx_poeg_init

```
rx_err_t rx_poeg_init(void)
```

Initialize POEG fault protection for all 4 motor channels.

Configures POEG Groups A-D for hardware motor fault protection:

1. Enables external pin detection (PIDE) on each group
2. Enables noise filter (NFEN) with PCLKB/8 sampling (400ns)

3. Inverts input (INV) since DRV8263H nFAULT is active-low
4. Links each GPTW channel to its POEG group via GTINTAD
5. Configures ICU interrupts for vectors 188-191 at priority 14

After initialization, a DRV8263H fault will:

- Immediately disable GPTW PWM output (hardware, nanoseconds)
- Fire POEG ISR which sets fault flags in shared_data

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All 4 POEG groups initialized successfully
k_rx_err_hw_init_failed	POEG register write verification failed

Pre: GPTW module enabled (MSTPCRB.MSTPB22 = 0)

Pre: GTETRGR pins configured via MPC (Pin.c)

Pre: shared_data initialized

Post: POEG Groups A-D enabled with pin detection + noise filter

Post: GPTW0-3 GTINTAD linked to POEG Groups A-D respectively

Post: ICU vectors 188-191 enabled at priority 14

Post: Any existing nFAULT assertion will trigger immediately

Note: Thread Safety: Call once during hardware_init, before motor tasks start

Warning: PIDE enable bit is write-once after reset cannot be disabled

NASA Power of 10 Compliance:: Rule 1: No goto, setjmp, recursion Rule 2: Loop bounded by k_poeg_motor_count (4) Rule 3: No dynamic allocation Rule 5: 3 preconditions, 4 postconditions Rule 7: Return value checked Rule 8: All constants as C23 typed enums

See also: rx_poeg_clear_fault() Clear fault after resolution, rx_poeg_get_fault_status() Query fault state

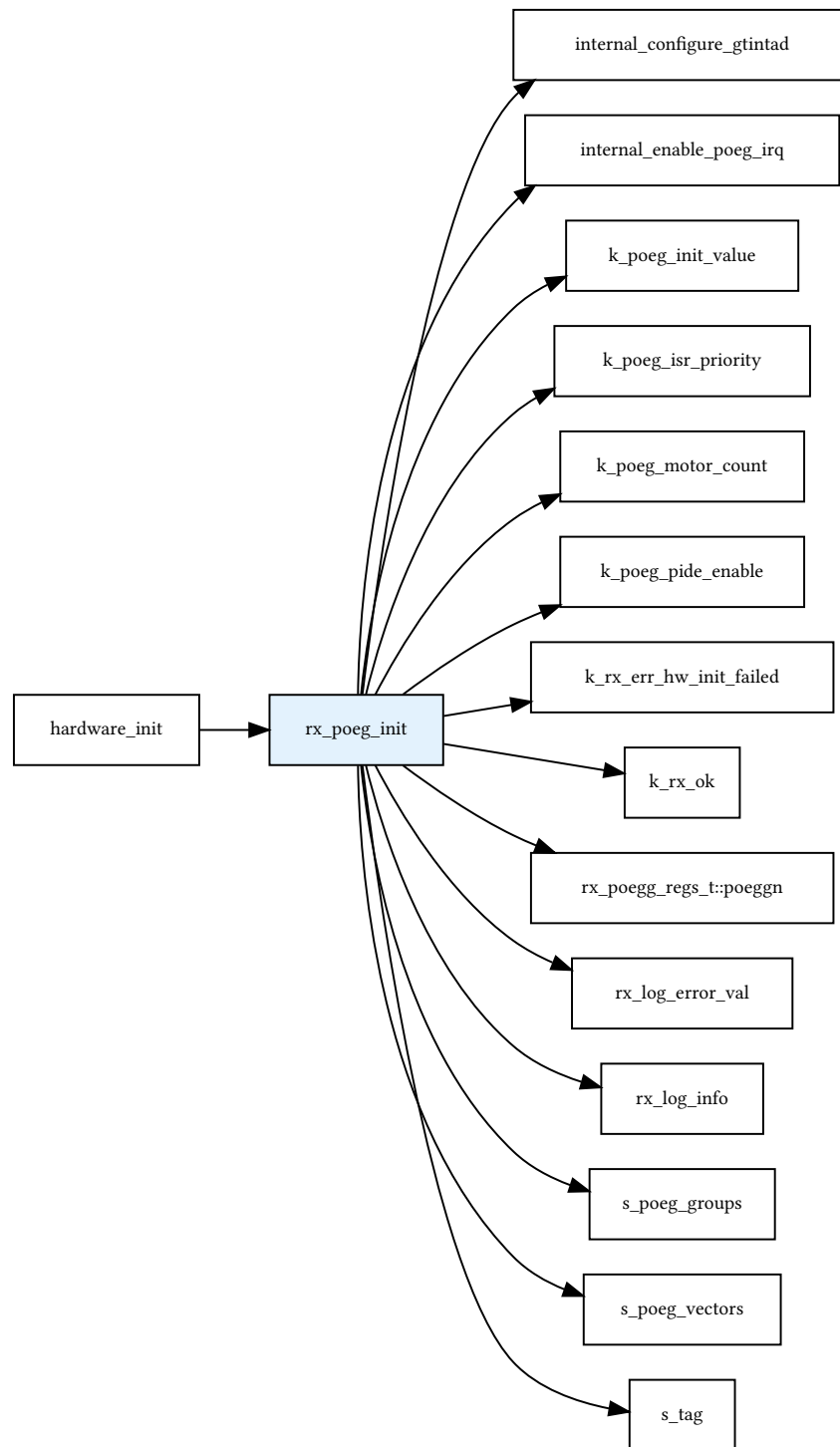


Figure 714: Call graph for rx_poeg_init

Defined in `libs/rx_hal/inc/rx_poeg.h:135`

Since Version 1.0.0

—

rx_poeg_get_fault_status

```
rx_err_t rx_poeg_get_fault_status(uint8_t motor_index, bool *fault_active)
```

Check if a motor has an active POEG fault.

Reads the POEGn register for the specified motor's POEG group and returns whether the PIDF (port input detection flag) is set.

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)
out	fault_active	true if POEG fault is active

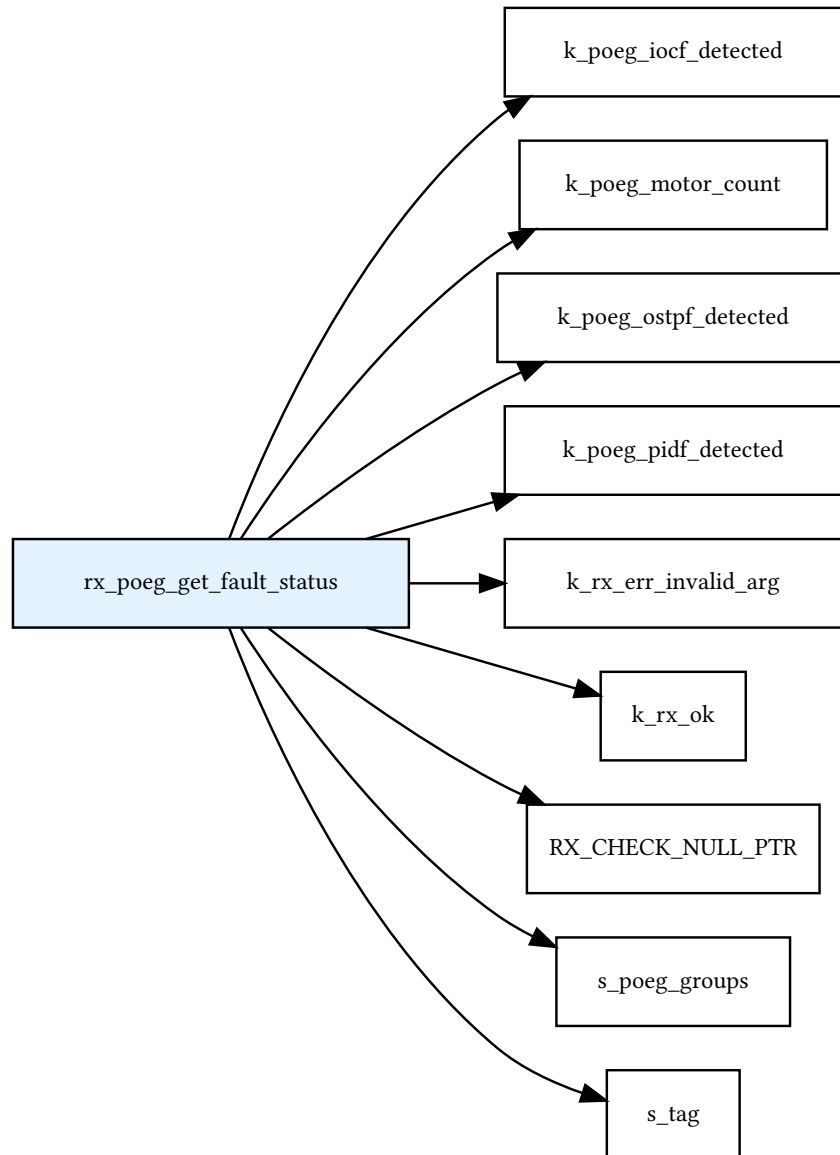
Returns: rx_err_t Error code

Value	Description
k_rx_ok	Status read successfully
k_rx_err_null_ptr	fault_active is nullptr
k_rx_err_invalid_arg	motor_index >= 4

Pre: POEG initialized via rx_poeg_init()

Post: No side effects on hardware state

Note: Thread Safety: Safe to call from any context (read-only register access)

Figure 715: Call graph for `rx_poeg_get_fault_status`

Defined in `libs/rx_hal/inc/rx_poeg.h:159`

Since Version 1.0.0

—

`rx_poeg_clear_fault`

```
rx_err_t rx_poeg_clear_fault(uint8_t motor_index)
```


Clear POEG fault and re-enable motor output.

Attempts to clear the PIDF flag for the specified motor's POEG group. The flag can only be cleared when the underlying fault condition has been resolved (i.e., DRV8263H nFAULT deasserted / GTETRG pin high).

After clearing, GPTW output re-enables at the end of the next GPTW counting cycle (delay of up to 1 PWM period, typically 50us at 20kHz).

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Fault cleared successfully
k_rx_err_invalid_arg	motor_index >= 4
k_rx_err_busy	Fault condition still active (nFAULT still asserted)

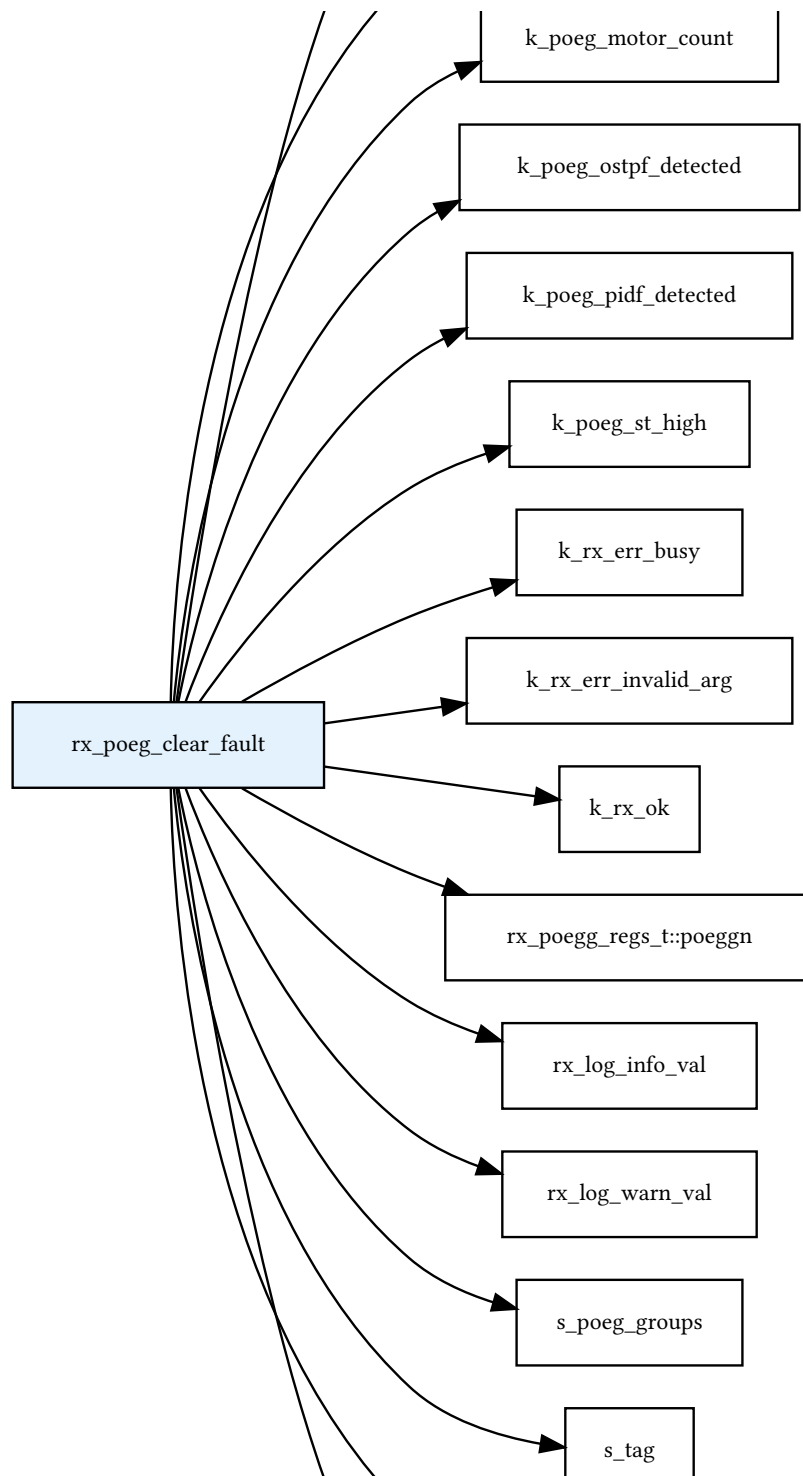
Pre: Underlying fault resolved (DRV8263H nFAULT deasserted)

Post: If successful, PIDF flag cleared and GPTW output will re-enable

Post: shared_data fault_flagsmotor_index cleared

Note: Motor control task should verify estop is cleared before resuming PWM

Warning: Returns k_rx_err_busy if fault source is still active

Figure 716: Call graph for `rx_poeg_clear_fault`

Defined in `libs/rx_hal/inc/rx_poeg.h:188`

Since Version 1.0.0

—

rx_poeg_software_stop

```
rx_err_t rx_poeg_software_stop(uint8_t motor_index)
```

Trigger software emergency stop on a specific motor.

Sets the SSF (Software Stop Flag) in the POEGGn register, immediately disabling GPTW PWM output for the specified motor. This provides a firmware-controlled emergency stop independent of hardware faults.

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Software stop activated
k_rx_err_invalid_arg	motor_index >= 4

Post: GPTW output immediately disabled for specified motor

Post: SSF flag set in POEGGn register] **See also:** rx_poeg_clear_software_stop() Release software stop

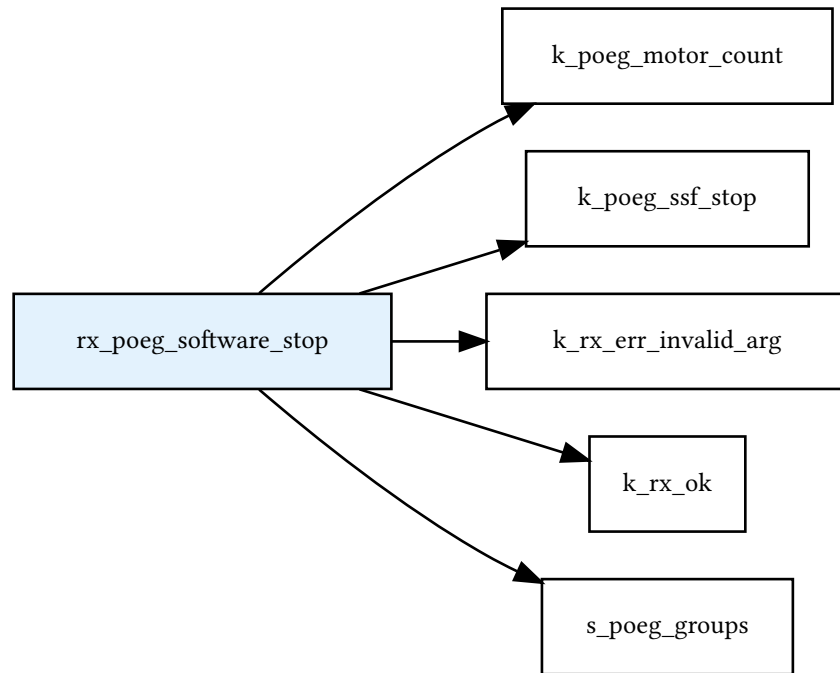


Figure 717: Call graph for rx_poeg_software_stop

Defined in `libs/rx_hal/inc/rx_poeg.h:210`

Since Version 1.0.0

—

rx_poeg_clear_software_stop

```
rx_err_t rx_poeg_clear_software_stop(uint8_t motor_index)
```

Release software emergency stop on a specific motor.

Clears the SSF (Software Stop Flag) in the POEGGn register. GPTW output re-enables at end of next counting cycle if no other fault flags (PIDF, IOCF, OSTPF) are active.

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Software stop released
k_rx_err_invalid_arg	motor_index >= 4

Post: SSF flag cleared in POEGGn register] **See also:** rx_poeg_software_stop() Activate software stop

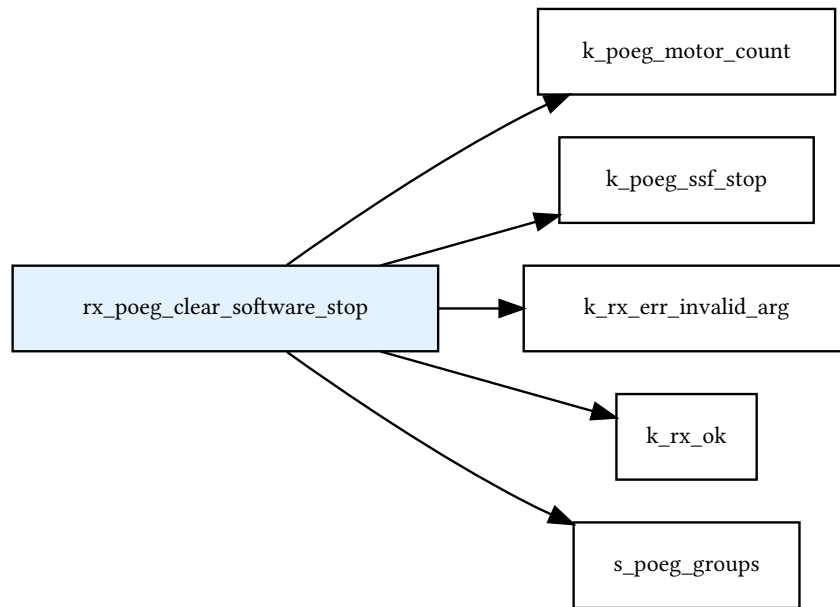


Figure 718: Call graph for rx_poeg_clear_software_stop

Defined in `libs/rx_hal/inc/rx_poeg.h:231`

Since Version 1.0.0

—

rx_port_utils.h

PORT utility functions for RX72N GPIO register access.

This file provides centralized utility functions for working with RX72N PORT (GPIO) registers. The primary function is mapping port numbers to register base addresses, which is critical for all GPIO operations in the STAR system.

Includes:

- "rx72n_port_regs.h"
- "rx_port_constants.h"

rx_port_get_base

```
static volatile rx_port_regs_t * rx_port_get_base(uint8_t port)
```

Map RX72N port number to PORT register base address.

Converts a port number constant (`k_rx_port_*`) to the corresponding volatile PORT register structure pointer for hardware access. This is the single source of truth for port-to-address mapping in the entire STAR firmware.

Algorithm steps:

1. Receive port number as input parameter
2. Switch on port number (compiler optimizes to jump table)
3. For valid port: call corresponding portX() inline accessor
4. For invalid port: return NULL to signal error
5. Caller must validate non-NULL before dereferencing

Implementation details:

- Each case calls an inline accessor (port0(), porta(), etc.) defined in rx72n_port_regs.h which returns the typed register base address
- Compiler optimizes switch to direct jump table (O(1) lookup)
- Function is inline, so entire call optimizes to direct address load
- Invalid port numbers safely return NULL instead of accessing garbage memory

Package-specific behavior: Supports all ports physically available on 144-pin LFQFP package:

- PORT0-9: Available (see rx_port_constants.h for per-port pin counts)
- PORTA-F: Available (PORTF has PF5 only)
- PORTJ: Available (PJ3, PJ5)
- Ports G-I, K-Q: Not available on 144-pin, will return nullptr

Control flow:

Return value is either NULL or valid PORT register base address

Function has no side effects (pure address mapping)

Package-specific: Only supports 144-pin LFQFP. Porting to 176-pin packages requires adding additional case statements for ports G-I, K-Q.

Parameters:

Direction	Name	Description
in	port	Port number constant from rx_port_constants.h Type: uint8_t (C23 typed enum rx_port_num_t) Valid range: k_rx_port_0 through k_rx_port_j Units: N/A (enumerated constant) Constraints: Must use k_rx_port_* constants, not raw integers Invalid values: Any value not in [0-5, 0x0A-0x0E, 0x13] returns nullptr

Returns: Pointer to PORT register base structure, or nullptr if port invalid

Value	Description
Non-NULL	Valid port - pointer to volatile rx_port_regs_t structure with PDR, PODR, PIDR, PMR, etc. registers
NULL	Invalid port number - port not available on 144-pin LFQFP package, or invalid parameter value

Pre: None - function is always safe to call

Pre: Parameter should use k_rx_port_* constants (not validated at runtime)

Post: If return value is non-NULL, pointer is valid volatile register address

Post: If return value is nullptr, caller must not dereference (validation required)

Post: No hardware state modified (read-only address calculation)

Note: Thread Safety: Fully thread-safe and reentrant. No shared state, no hardware access, pure computation. Multiple threads can call simultaneously without synchronization.

Note: Re-entrancy: Fully reentrant. No static variables, no globals. Safe for use in interrupt handlers and multi-threaded contexts.

Note: Performance: Optimizes to 1-2 CPU cycles when inlined. Switch compiles to jump table (O(1) lookup). At 240 MHz: 4-8 nanoseconds.

Note: Memory: Zero stack usage (inlined). Zero heap usage. Zero static storage. Entire function optimizes to immediate address load.

Warning: NULL validation required: Caller MUST check return value before dereferencing. Failing to validate will cause nullptr fault if invalid port number is passed.

Return Value Details::

Example - Basic Usage: #include "rx_port_utils.h" #include "rx_port_constants.h"

```
volatilerx_port_regs_t*port_d=rx_port_get_base(k_rx_port_d); if(port_d!=nullptr){ port_d->
pdr=0xFF; port_d->podr=0x00; }
```

Example - Error Handling: volatilerx_port_regs_t*port_g=rx_port_get_base(0x10);
if(port_g==nullptr){ rx_log_error("GPIO","PortGnotavailableonthispackage");
returnk_rx_err_invalid_arg; }

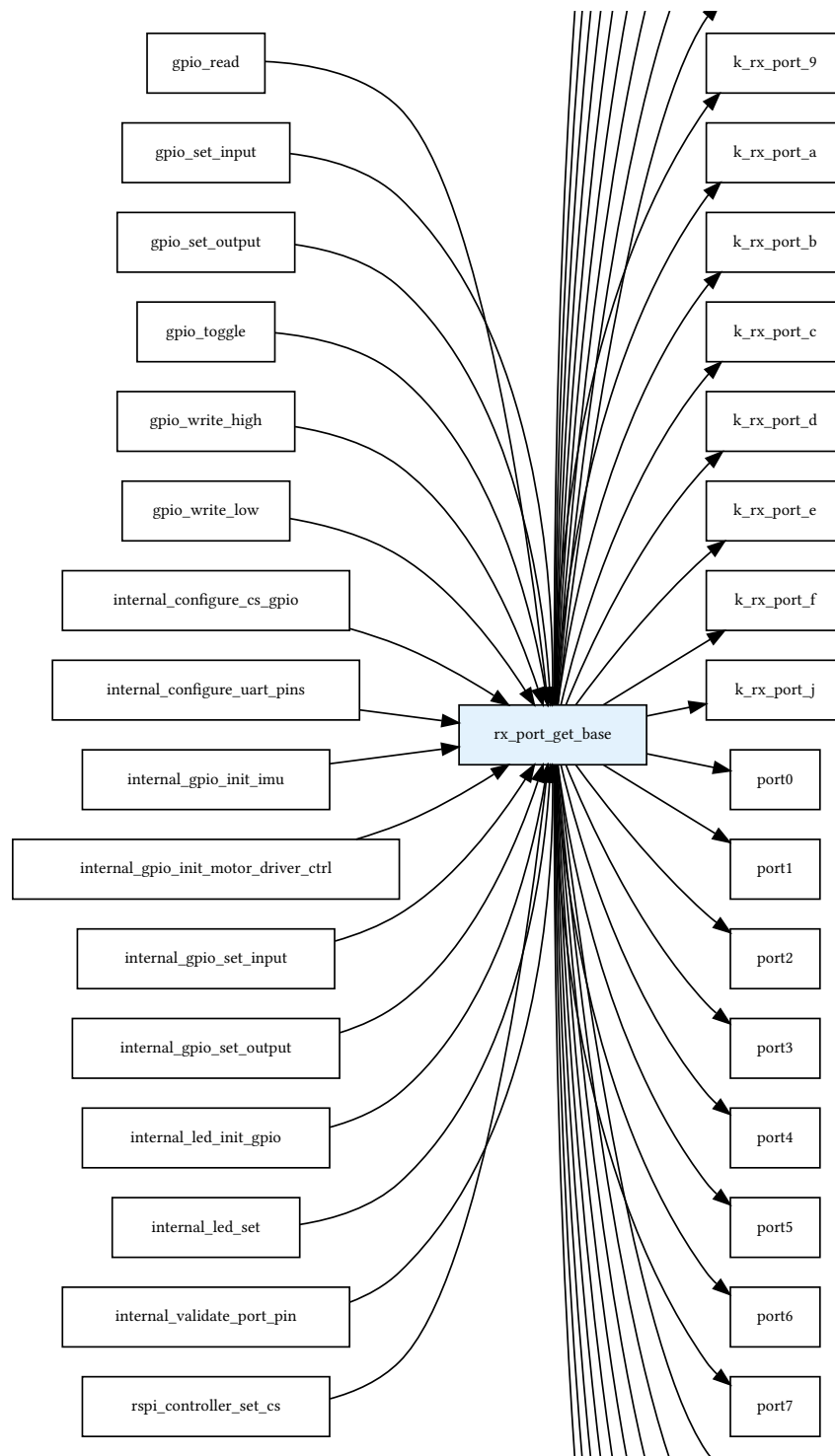
Example - Motor Control Initialization::

```
volatilerx_port_regs_t*port_e=rx_port_get_base(k_rx_port_e); if(port_e!=nullptr){ port_e->
pmr&= 0x0F; port_e->pdr|=0x0F; port_e->podr&= 0x0F; }
```

Example - LED Status Indicator: volatilerx_port_regs_t*port_d=rx_port_get_base(k_rx_port_d);
if(port_d!=nullptr){ port_d->podr^=(1<<0); }

NASA Power of 10 Compliance: Rule 1 [OK] Simple control flow (switch only, no goto/recursion)
Rule 2 [OK] No loops (O(1) switch statement) Rule 3 [OK] No dynamic allocation Rule 4 [OK]
Function is 43 lines (under 60 line limit) Rule 5 [OK] Implicit validation: NULL return signals invalid
input Rule 6 [OK] Minimal scope (single parameter, local to function) Rule 7 [OK] Return value
MUST be validated by caller Rule 8 [OK] Uses C23 typed enums (k_rx_port_*), zero magic numbers
Rule 9 [OK] Single level of pointer dereferencing for register access Rule 10 [OK] Compiles clean
with -Wall -Wextra -Werror

See also: `port0()` through `portj()` Inline register accessors in `rx72n_port_regs.h`, `rx_port_constants.h` Port number constant definitions, `rx_gpio_init()` Higher-level GPIO initialization using this function, `docs/sections/03_hardware_pinout.tex` Complete pin assignment reference

Figure 719: Call graph for `rx_port_get_base`

_Defined in `libs/rx\_hal/inc/rx\_port\_utils.h:321_`

Since Version 1.0.0

—

rx_tpu.h

TPU HAL Driver for RX72N 16-Bit Timer Pulse Unit (Phase Counting).

Hardware abstraction layer for the RX72N TPU peripheral, focused on phase counting mode for quadrature encoder input. This driver provides low-level channel configuration, counter access, and direction reading.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`

rx_tpu_channel_t

TPU channel identifiers for phase counting.

Identifies TPU channels that support phase counting mode. Only TPU1/2/4/5 can do phase counting; TPU0 and TPU3 cannot.

Value	Name	Description
= 1	<code>k_tpu_channel_1</code>	TPU1: rear-left encoder (TCLKA/TCLKB)
= 2	<code>k_tpu_channel_2</code>	TPU2: rear-right encoder (TCLKC/TCLKD)
= 4	<code>k_tpu_channel_4</code>	TPU4: available (TCLKC/TCLKD, paired with TPU2)
= 5	<code>k_tpu_channel_5</code>	TPU5: available (TCLKA/TCLKB, paired with TPU1)

Since Version 1.0.0

—

rx_tpu_phase_mode_t

TPU phase counting mode selection.

Selects the phase counting resolution. Mode 4 (4x) provides the highest resolution by counting all edges of both A and B phases.

Value	Name	Description
= 1	<code>k_tpu_phase_mode_1</code>	1x resolution (A rising only)
= 2	<code>k_tpu_phase_mode_2</code>	1x resolution (B rising only)
= 3	<code>k_tpu_phase_mode_3</code>	2x resolution (A both edges)
= 4	<code>k_tpu_phase_mode_4</code>	4x resolution (all edges) - STAR default

Since Version 1.0.0

—

rx_tpu_init_phase_count

```
rx_err_t rx_tpu_init_phase_count(const rx_tpu_config_t *config)
```

Initialize TPU channel for phase counting mode.

Configures a TPU channel for hardware quadrature encoder phase counting. This function:

1. Enables the TPU module (clears MSTPA13)
2. Stops the channel counter
3. Sets phase counting mode in TMDR
4. Clears the counter to zero
5. Enables overflow/underflow interrupts (if needed later)

After initialization, call rx_tpu_start() to begin counting.

Implementation of rx_tpu_init_phase_count() - see rx_tpu.h for complete API documentation.

Parameters:

Direction	Name	Description
in	config	Phase counting configuration

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, channel configured
k_rx_err_null_ptr	config is nullptr
k_rx_err_invalid_arg	Invalid channel (not 1/2/4/5)
k_rx_err_invalid_arg	Invalid phase mode (not 1-4)
k_rx_err_invalid_state	Channel already initialized

Pre: TPU module clock enabled (or function enables it)

Pre: External clock pins (TCLKA-D) configured as peripheral I/O

Post: Channel configured in phase counting mode

Post: Counter cleared to zero

Post: Counter NOT started (call rx_tpu_start())

Note: Thread Safety: Not thread-safe, call during init only] **Register Configuration Sequence:** Unlock PRCR, clear MSTPCRA.MSTPA13, lock PRCR Clear CSTn in TSTR (stop counter) Write phase counting mode to TMDR.MD Write 0x00 to TCR (free-running, external clock) Write 0x00 to TIER (no interrupts) Write 0x0000 to TCNT (clear counter)

See also: `rx_tpu_start()` Start counting after init, `rx_tpu_deinit()` Release channel resources, `rx_tpu.h` Full API documentation

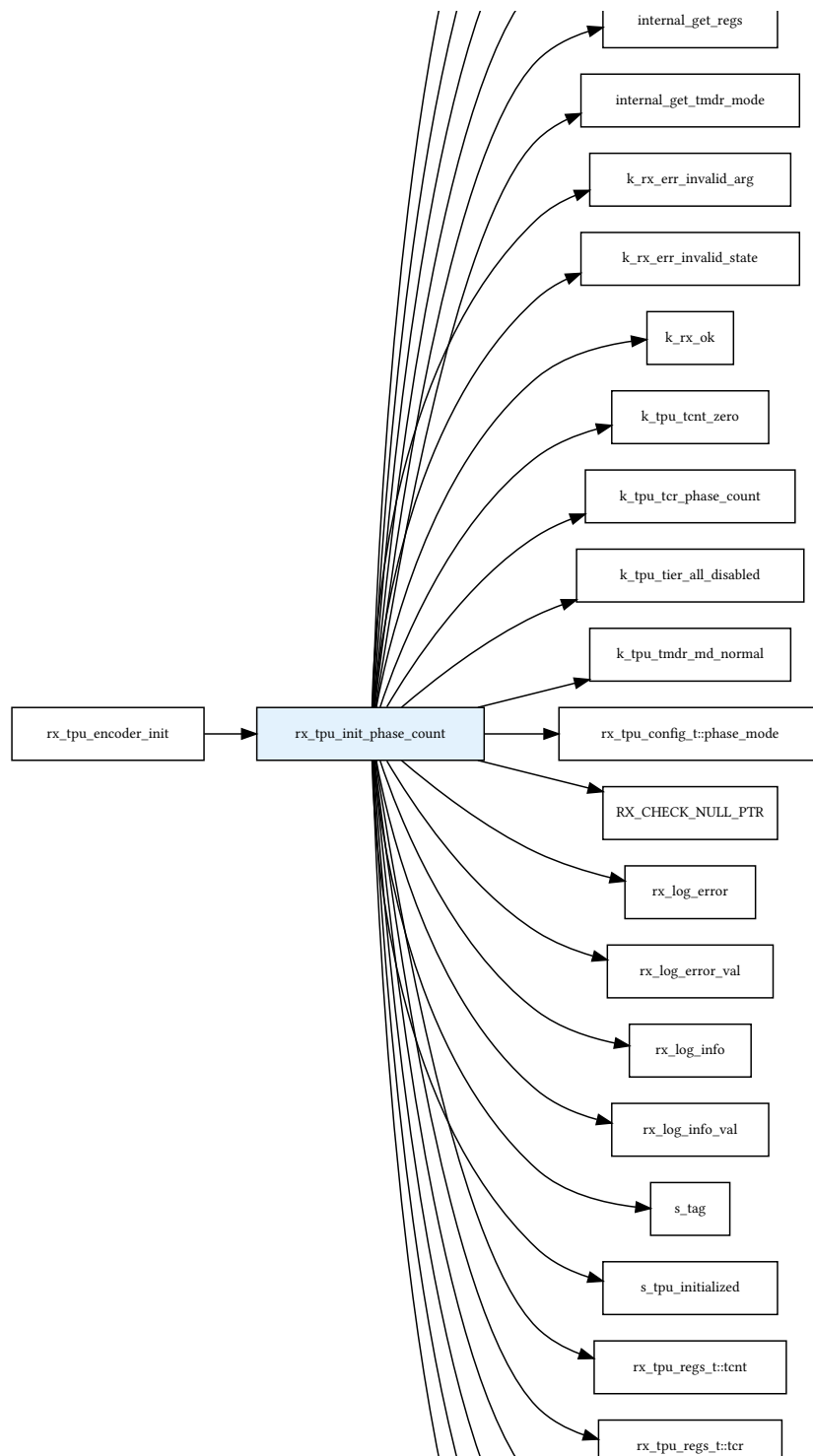


Figure 720: Call graph for rx_tpu_init_phase_count

Defined in `libs/rx_hal/inc/rx_tpu.h:225`

Since Version 1.0.0

—

rx_tpu_start

```
rx_err_t rx_tpu_start(rx_tpu_channel_t channel)
```

Start TPU channel counter.

Sets the CSTn bit in TSTR to begin counter operation. In phase counting mode, the counter increments/decrements based on the external clock phase relationship.

Start TPU channel counter.

Sets the Count Start (CSTn) bit in the TSTR register to begin counter operation. Once started, the TCNT register increments or decrements on each external encoder clock edge according to the configured phase mode.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
in	channel	TPU channel to start (1, 2, 4, or 5) Must have been previously initialized via rx_tpu_init_phase_count()

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, counter started
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Channel not initialized
k_rx_ok	Counter started successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel initialized via rx_tpu_init_phase_count()

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: External encoder clock pins must be connected and valid

Post: CSTn bit set, counter running

Post: TSTR.CSTn bit set, counter begins counting encoder pulses

Post: TCNT updates in real time from encoder edges

Note: Thread Safety: Safe from any context (atomic register write)

Note: Not thread-safe: modifies shared TSTR register

Note: Idempotent: safe to call when already running] **Example:**

```
rx_err_t rx_tpu_start(k_tpu_channel_1);
```

See also: `rx_tpu_stop()` Stop counter, `rx_tpu_stop()` Stop the counter, `rx_tpu_read_count()` Read current counter value

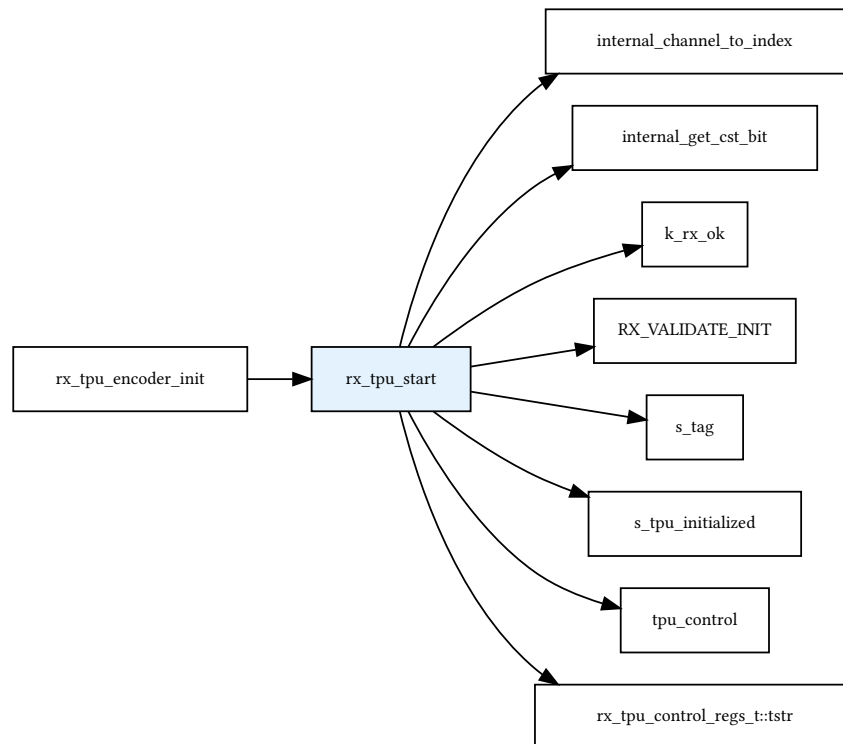


Figure 721: Call graph for `rx_tpu_start`

Defined in `libs/rx_hal/inc/rx_tpu.h:250`

Since Version 1.0.0

—

rx_tpu_stop

```
rx_err_t rx_tpu_stop(rx_tpu_channel_t channel)
```

Stop TPU channel counter.

Clears the CSTn bit in TSTR to stop counter operation. The counter value is preserved.

Stop TPU channel counter.

Clears the Count Start (CSTn) bit in the TSTR register to halt counter operation. The TCNT value is preserved and encoder edges no longer update it. The channel remains initialized and can be restarted with rx_tpu_start().

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
in	channel	TPU channel to stop (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, counter stopped
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Channel not initialized
k_rx_ok	Counter stopped successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: TSTR register must be accessible (module clock enabled)

Post: CSTn bit cleared, counter stopped

Post: Counter value preserved

Post: TSTR.CSTn bit cleared, counter halted

Post: TCNT value preserved at last count

Note: Thread Safety: Safe from any context (atomic register write)

Note: Not thread-safe: modifies shared TSTR register

Note: Idempotent: safe to call when already stopped] **Example:**

```
rx_err_t err = rx_tpu_stop(k_tpu_channel_1);
```

See also: `rx_tpu_start()` Restart counter, `rx_tpu_start()` Resume counter operation, `rx_tpu_reset_count()` Clear counter to zero

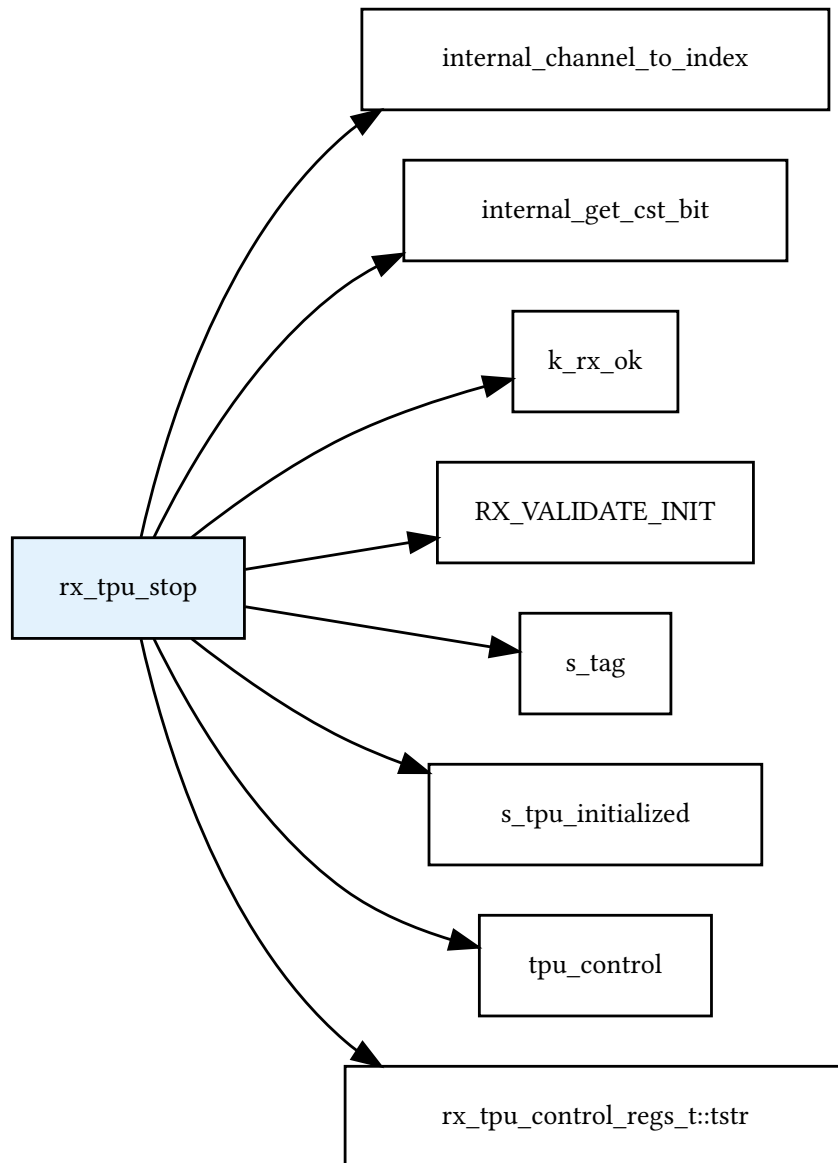


Figure 722: Call graph for `rx_tpu_stop`

Defined in `libs/rx_hal/inc/rx_tpu.h:275`

Since Version 1.0.0

—

rx_tpu_read_count

```
rx_err_t rx_tpu_read_count(rx_tpu_channel_t channel, uint16_t *count)
```

Read TPU channel 16-bit counter value.

Reads the current TCNT register value. In phase counting mode this is an up/down counter that tracks encoder position modulo 65536.

Read TPU channel 16-bit counter value.

Performs a single volatile read of the TCNT register for the specified channel. In 4x phase counting mode, the counter increments or decrements on every edge of both A and B encoder channels.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
out	count	Pointer to store 16-bit counter value (0-65535)
in	channel	TPU channel to read (1, 2, 4, or 5)
out	count	Pointer to store the 16-bit counter value [0, 65535] Must be non-NULL

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, count written
k_rx_err_null_ptr	count is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Channel not initialized
k_rx_ok	Counter value read into *count
k_rx_err_null_ptr	count pointer is nullptr
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel initialized and running

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: count must point to valid uint16_t storage

Post: count contains current TCNT value

Post: *count contains the TCNT register value at the time of the read

Post: Hardware registers unchanged

Note: Thread Safety: Safe - single volatile register read

Note: Not thread-safe: counter may wrap between multiple reads

Note: Wraps naturally at 0 and 65535 (16-bit overflow)] **Example:**

```
uint16_t count;
rx_err_t err = rx_tpu_read_count(k_tpu_channel_1, &count);
```

See also: rx_tpu_read_direction() Read counting direction, rx_tpu_reset_count() Clear counter to zero, rx_tpu_read_direction() Read counting direction

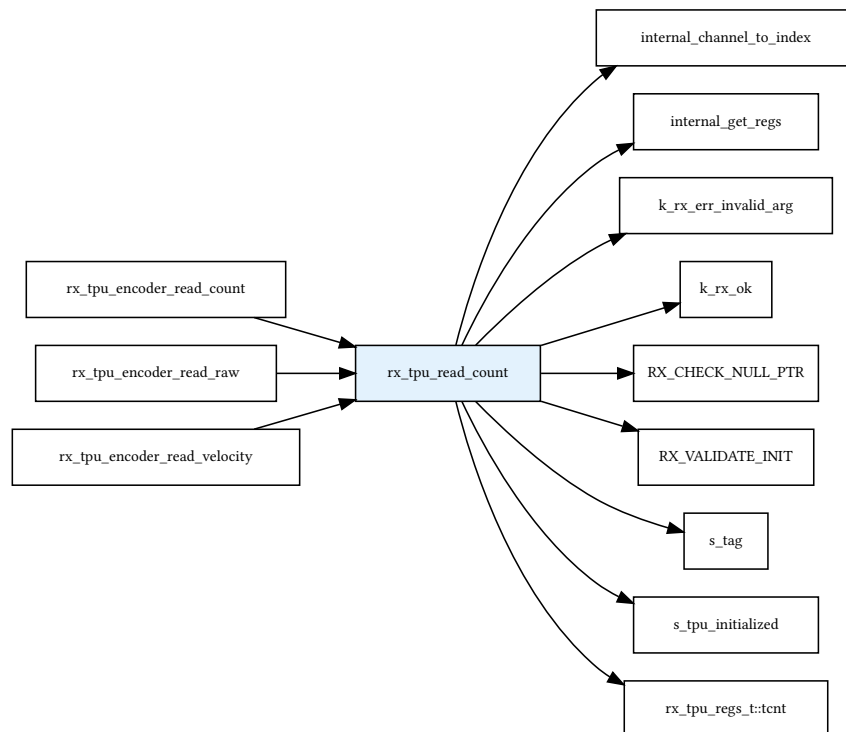


Figure 723: Call graph for rx_tpu_read_count

Defined in `libs/rx_hal/inc/rx_tpu.h:301`

Since Version 1.0.0

—

rx_tpu_read_direction

```
rx_err_t rx_tpu_read_direction(rx_tpu_channel_t channel, bool *counting_up)
```

Read TPU channel counting direction.

Reads the TCFD bit in the TSR register to determine whether the counter is currently counting up (forward) or down (reverse).

Read TPU channel counting direction.

Reads the Timer Counter Flag Direction (TCFD) bit in the TSR register. TCFD reflects the last count event direction:

- TCFD = 1: Counter incremented on the last edge (counting up / forward)
- TCFD = 0: Counter decremented on the last edge (counting down / reverse)

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
out	counting_up	Pointer to store direction flag (true = counting up/forward, false = counting down/reverse)
in	channel	TPU channel to query (1, 2, 4, or 5)
out	counting_up	Pointer to store direction result true = last count was an increment (encoder moving forward) false = last count was a decrement (encoder moving in reverse) Must be non-NULL

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, direction written
k_rx_err_null_ptr	counting_up is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Channel not initialized
k_rx_ok	Direction read into *counting_up
k_rx_err_null_ptr	counting_up pointer is nullptr
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel initialized and running in phase counting mode

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: counting_up must point to valid bool storage

Post: counting_up reflects TSR.TCFD bit state

Post: *counting_up reflects TSR.TCFD at the time of the read

Post: Hardware registers unchanged

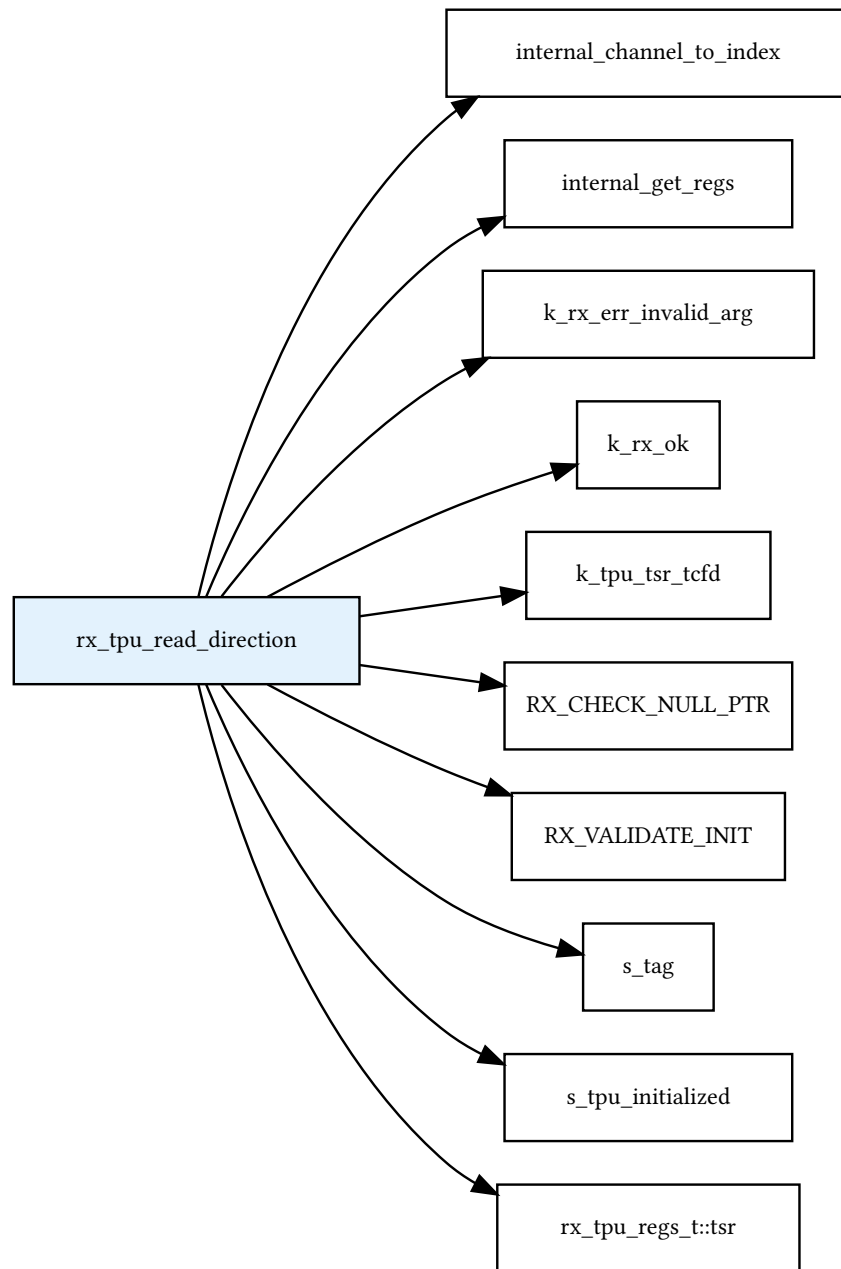
Note: Thread Safety: Safe - single volatile register read

Note: Not thread-safe: value may change between reads if encoder is moving

Note: Direction is unreliable if counter is stationary (no recent edge)] **Example:**

```
bool forward;  
rx_err_t err = rx_tpu_read_direction(k_tpu_channel_1, &forward);
```

See also: rx_tpu_read_count() Read counter value, rx_tpu_read_count() Read counter position, k_tpu_tsr_tcfid TSR direction bit mask

Figure 724: Call graph for `rx_tpu_read_direction`

Defined in `libs/rx_hal/inc/rx_tpu.h:328`

Since Version 1.0.0

—

rx_tpu_reset_count

```
rx_err_t rx_tpu_reset_count(rx_tpu_channel_t channel)
```

Reset TPU channel counter to zero.

Writes zero to the TCNT register. Should be called while the counter is stopped for reliable operation, though the hardware permits writing TCNT while running.

Reset TPU channel counter to zero.

Writes k_tpu_tcmt_zero (0x0000) to the TCNT register, resetting the encoder position counter. The counter continues running from the new zero baseline.

Typical use: zero encoder position at a known reference point (home).

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
in	channel	TPU channel to reset (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, counter zeroed
k_rx_err_invalid_arg	Invalid channel
k_rx_err_invalid_state	Channel not initialized
k_rx_ok	Counter reset to zero successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: Channel should be stopped (rx_tpu_stop()) before resetting for atomic 0

Post: TCNT = 0

Post: TCNT register written to 0x0000

Post: Counter resumes from zero if running

Note: Thread Safety: Not thread-safe (read-modify-write possible issues)

Note: Not thread-safe: write races with live counter updates from encoder

Note: May cause a single spurious velocity estimate if called while running] **Example:**

```
rx_tpu_stop(k_tpu_channel_1);
rx_tpu_reset_count(k_tpu_channel_1);
rx_tpu_start(k_tpu_channel_1);
```

See also: `rx_tpu_stop()` Stop counter before resetting for deterministic result,
`rx_tpu_read_count()` Verify counter after reset

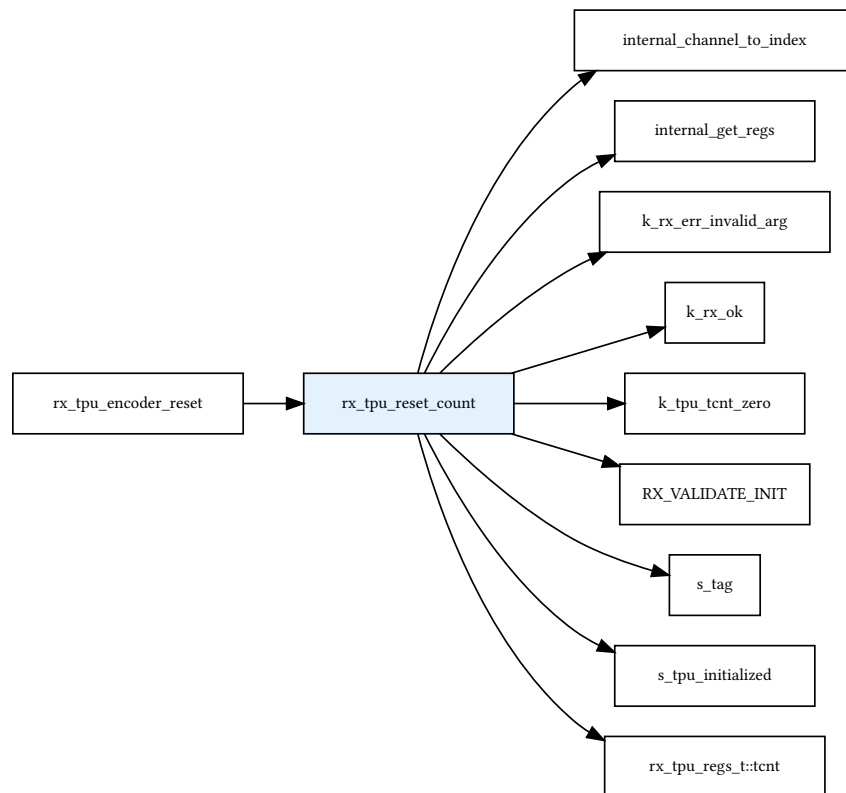


Figure 725: Call graph for `rx_tpu_reset_count`

Defined in `libs/rx_hal/inc/rx_tpu.h:352`

Since Version 1.0.0

—

rx_tpu_deinit

```
rx_err_t rx_tpu_deinit(rx_tpu_channel_t channel)
```

Deinitialize TPU channel.

Stops the channel counter, resets mode to normal, and marks the channel as uninitialized. Does NOT disable the TPU module clock (other channels may still be active).

Deinitialize TPU channel.

Performs an orderly shutdown of the specified TPU channel:

1. Validate channel and get array index
2. Stop counter (clear CSTn in TSTR)
3. Reset TMDR to normal operation mode (0x00)
4. Disable all interrupts (TIER = 0)
5. Clear counter (TCNT = 0)
6. Mark channel as uninitialized in s_tpu_initialized[]

This function does NOT disable the TPU module clock (MSTPCRA.MSTPA13), as other channels may still be in use.

Parameters:

Direction	Name	Description
in	channel	TPU channel (1, 2, 4, or 5)
in	channel	TPU channel to deinitialize (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, channel released
k_rx_err_invalid_arg	Invalid channel
k_rx_ok	Channel deinitialized successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_invalid_arg	Register pointer could not be resolved

Pre: Channel was previously initialized

Pre: channel must be a valid phase-counting TPU channel (1, 2, 4, or 5)

Pre: Motor driven by this encoder must be stopped before calling

Post: Counter stopped and zeroed

Post: Mode register reset to normal operation

Post: Channel marked as uninitialized

Post: Counter stopped and zeroed

Post: TMDR reset to normal mode

Post: `s_tpu_initializedidx == false`

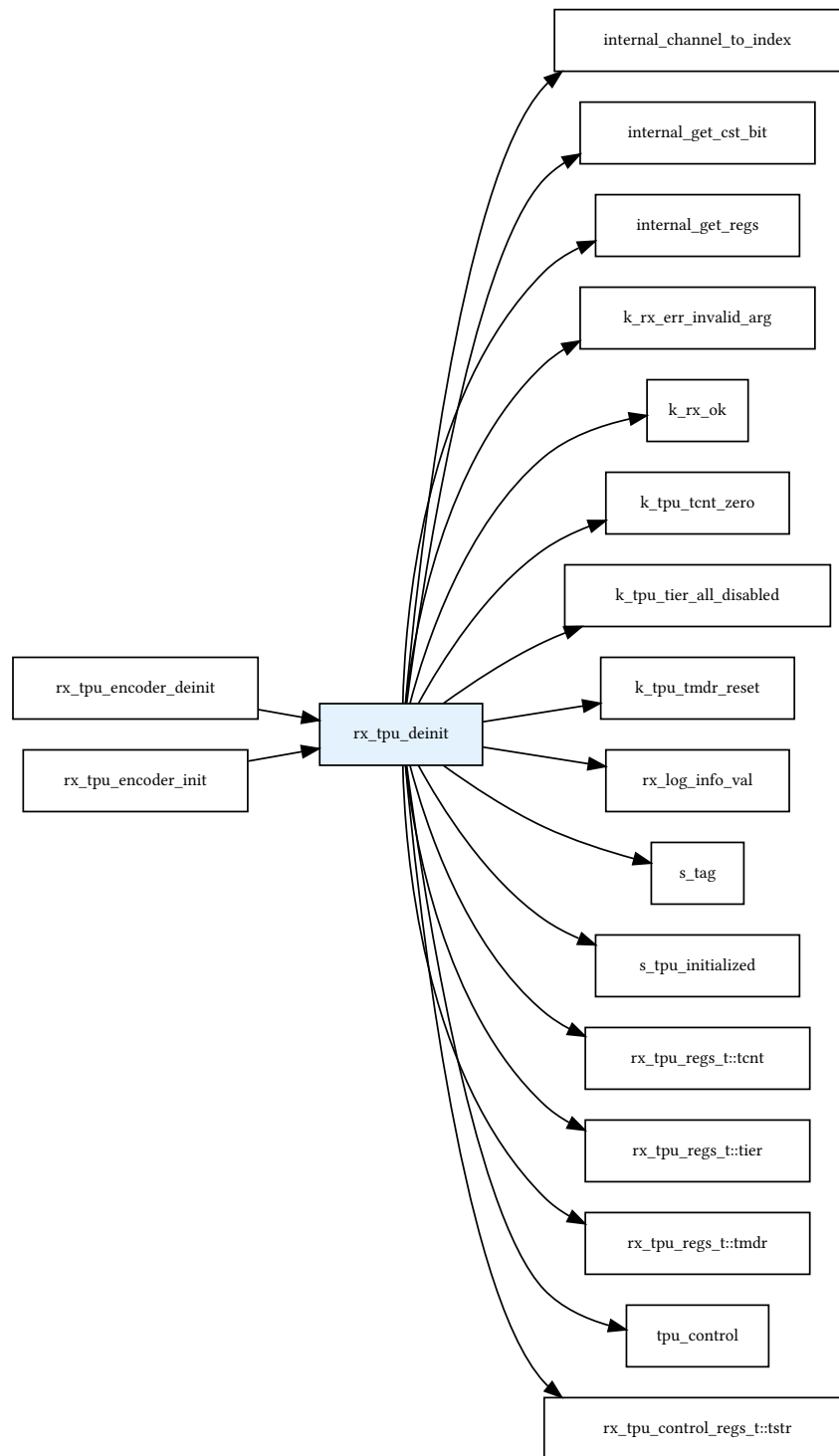
Note: Thread Safety: Not thread-safe, call during shutdown only

Note: Not thread-safe: do not access channel concurrently during deinit

Note: Channel may be partially initialized; deinit succeeds regardless] **Example:**

```
rx_err_t err = rx_tpu_deinit(k_tpu_channel_1);
```

See also: `rx_tpu_init_phase_count()` Re-initialize after deinit,
`rx_tpu_init_phase_count()` Re-initialize after deinit, `rx_tpu_stop()` Stop counter
without full teardown

Figure 726: Call graph for `rx_tpu_deinit`

Defined in *libs/rx_hal/inc/rx_tpu.h:378*

Since Version 1.0.0

—

adc.c

ADC Driver for RX72N S12ADFa - 12-bit Analog-to-Digital Conversion.

Includes:

- <string.h>
- "hardware.h"
- "rx72n_regs.h"
- "rx_register_protection.h"

adc_constants_t

ADC hardware configuration limits.

Defines the hardware limits and configuration constants for the S12ADFa peripheral on the RX72N microcontroller. These values are derived from the RX72N hardware manual Chapter 41 (S12ADFa).

Value	Name	Description
= 2	k_adc_max_units	Number of ADC units (S12AD0, S12AD1)
= 8	k_adc_max_channels	Channels per unit (0-7 for S12AD0, 16-23 for S12AD1)
= 100	k_adc_timeout_ms	Maximum wait time for conversion completion (ms)

See also: `internal_get_adc_base()` Uses `k_adc_max_units` for validation,
`s_adc_unit_initialized` Array sized by `k_adc_max_units`

Since Version 1.0.0

—

adc_bit_constants_t

Bit manipulation constants for ADC register operations.

Single-bit value used for shift operations when setting/clearing individual bits in ADC control and channel selection registers.

Value	Name	Description
= 1U	k_adc_bit_one	Single-bit value for shift operations ($1 \ll n$)

Since Version 1.0.0

—

adc_module_stop_bits_t

ADC module stop bit positions in MSTPCRA register.

Defines the bit positions in the MSTPCRA (Module Stop Control Register A) that control the clock supply to the S12AD0 and S12AD1 modules. These bits must be cleared (clock enabled) before accessing ADC registers.

Value	Name	Description
-------	------	-------------

= 16	k_adc_mstpra_s12ad1	MSTPCRA bit 16: S12AD1 module stop control
= 17	k_adc_mstpra_s12ad0	MSTPCRA bit 17: S12AD0 module stop control

—

adc_adcer_bits_t

ADC Control Extended Register (ADCER) bit definitions.

Defines the bit fields for the ADCER (A/D Control Extended Register) which controls ADC resolution and data format. The ADPRC field (bits 1:0) sets the A/D conversion resolution.

Value	Name	Description
= 3U	k_adc_adcer_adprc_mask	ADPRC field mask (bits 1:0)
= 0U	k_adc_adcer_adprc_shift	ADPRC field bit position (0)
= 0U	k_adc_adcer_adprc_12bit	ADPRC = 00: 12-bit resolution (default)
= 1U	k_adc_adcer_adprc_10bit	ADPRC = 01: 10-bit resolution
= 2U	k_adc_adcer_adprc_8bit	ADPRC = 10: 8-bit resolution

—

adc_adcsr_bits_t

ADC Control/Status Register (ADCSR) bit definitions.

Defines bit values for the ADCSR (A/D Control Status Register) which controls conversion start and indicates conversion status.

Value	Name	Description
= 0U	k_adc_adcsr_idle	Idle state: ADST=0 (conversion complete)
= 4096U	k_adc_adcsr_adst	Start bit: ADST=1 (bit 12 = 0x1000 = 4096)

—

adc_channel_boundaries_t

ADC channel selection register boundaries.

Defines the channel boundaries for ADANSA0 and ADANSA1 registers. Channels 0-15 are selected via ADANSA0, channels 16+ via ADANSA1.

For S12AD0: Physical channels AN0-AN7 (channels 0-7) For S12AD1: Physical channels AN16-AN23 (channels 0-7 mapped)

Value	Name	Description
= 15	k_adc_channel_adansa0_max	Maximum channel for ADANSA0 register
= 16	k_adc_channel_adansa1_base	Base channel for ADANSA1 register

See also: adc_init() Uses these for channel enable logic

Since Version 1.0.0

—

adc_misc_constants_t

ADC timeout and voltage reference constants.

Miscellaneous constants for timeout handling and voltage calculations.

Value	Name	Description
= 1000	k_adc_timeout_multiplier	Loop iterations per ms of timeout
= 0	k_adc_timeout_expired	Sentinel value: timeout counter reached zero
= 3300	k_adc_reference_voltage_mv	ADC reference voltage: 3.3V = 3300 mV

—

adc_raw_value_t

Raw ADC value reset/default constants.

Default raw value used to initialize the raw_value variable before a conversion result is written.

Using a named constant makes the intent explicit and avoids magic numeric literals.

Value	Name	Description
= 0	k_adc_raw_value_reset	Reset/default raw ADC value before conversion

See also: `adc_read_voltage_mv()` Initializes `raw_value` with `k_adc_raw_value_reset`

Since Version 1.0.0

—

s_tag

```
const char* s_tag
```

Log tag for ADC module messages.

—

s_adc_unit_initialized

```
bool s_adc_unit_initialized[k_adc_max_units]
```

Tracks initialization state for each ADC unit.

Warning: NOT thread-safe - initialization should be done from single thread] **See also:** `adc_init()` Sets to true on successful initialization, `adc_read()` Checks before allowing read operation

—

s_adc_unit_resolution

```
adc_resolution_t s_adc_unit_resolution[k_adc_max_units]
```

Stores configured resolution for each ADC unit.

Note: Resolution is set only on first init and cannot be changed] **See also:** `adc_init()` Sets on first initialization, validates on subsequent, `adc_read_voltage_mv()` Uses for voltage calculation

internal_get_adc_base

```
static volatile rx_s12ad_regs_t * internal_get_adc_base(const uint8_t unit)
```

Get ADC register base address from unit number.

Returns a pointer to the register structure for the specified ADC unit. The RX72N has two ADC units with registers at different base addresses:

Parameters:

Direction	Name	Description
in	unit	ADC unit number k_adc_unit_0: S12AD0 k_adc_unit_1: S12AD1

Returns: volatile rx_s12ad_regs_t* Pointer to ADC registers

Value	Description
Non-NULL	Valid register pointer for unit 0 or 1
NULL	Invalid unit number (not 0 or 1)

Pre: None (pure lookup function)

Post: No state changes

Note: Thread-safe: Pure function with no shared state

Note: Performance: O(1), 10 cycles] **See also:** s12ad0() Accessor for S12AD0 registers, s12ad1() Accessor for S12AD1 registers

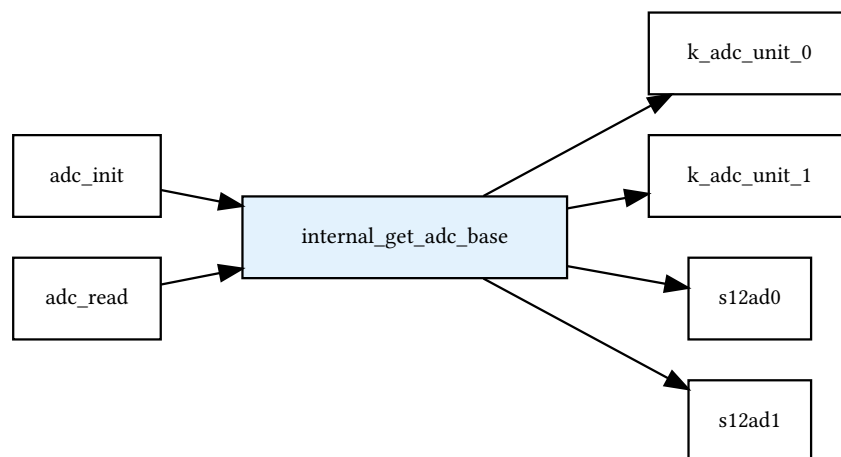


Figure 727: Call graph for `internal_get_adc_base`

_Defined in libs/rx_hal/src/adc.c:444_

Since Version 1.0.0

—

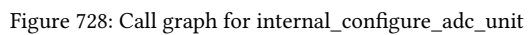
internal_configure_adc_unit

```
static rx_err_t internal_configure_adc_unit(const uint8_t unit, volatile
rx_sl2ad_regs_t *adc, const uint8_t bits)
```

Configure ADC unit control registers and enable module clock.

Performs one-time initialization of an ADC unit:

1. Unlocks PRCR to allow MSTPCRA modification
2. Clears module stop bit in MSTPCRA (enables clock)
3. Locks PRCR
4. Configures ADCER register for desired resolution
5. Marks unit as initialized



_Defined in libs/rx_hal/src/adc.c:508_
 —

internal_validate_unit_channel

```
static rx_err_t internal_validate_unit_channel(const uint8_t unit, uint8_t
channel)
```

Validate ADC unit and channel numbers.

Performs range validation for ADC unit and channel parameters before any hardware access. This prevents undefined behavior from invalid register access.

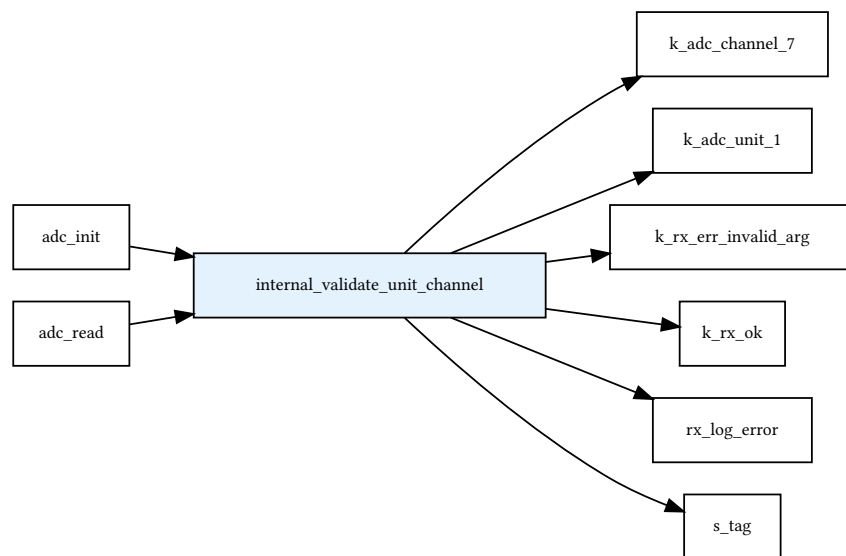


Figure 729: Call graph for internal_validate_unit_channel

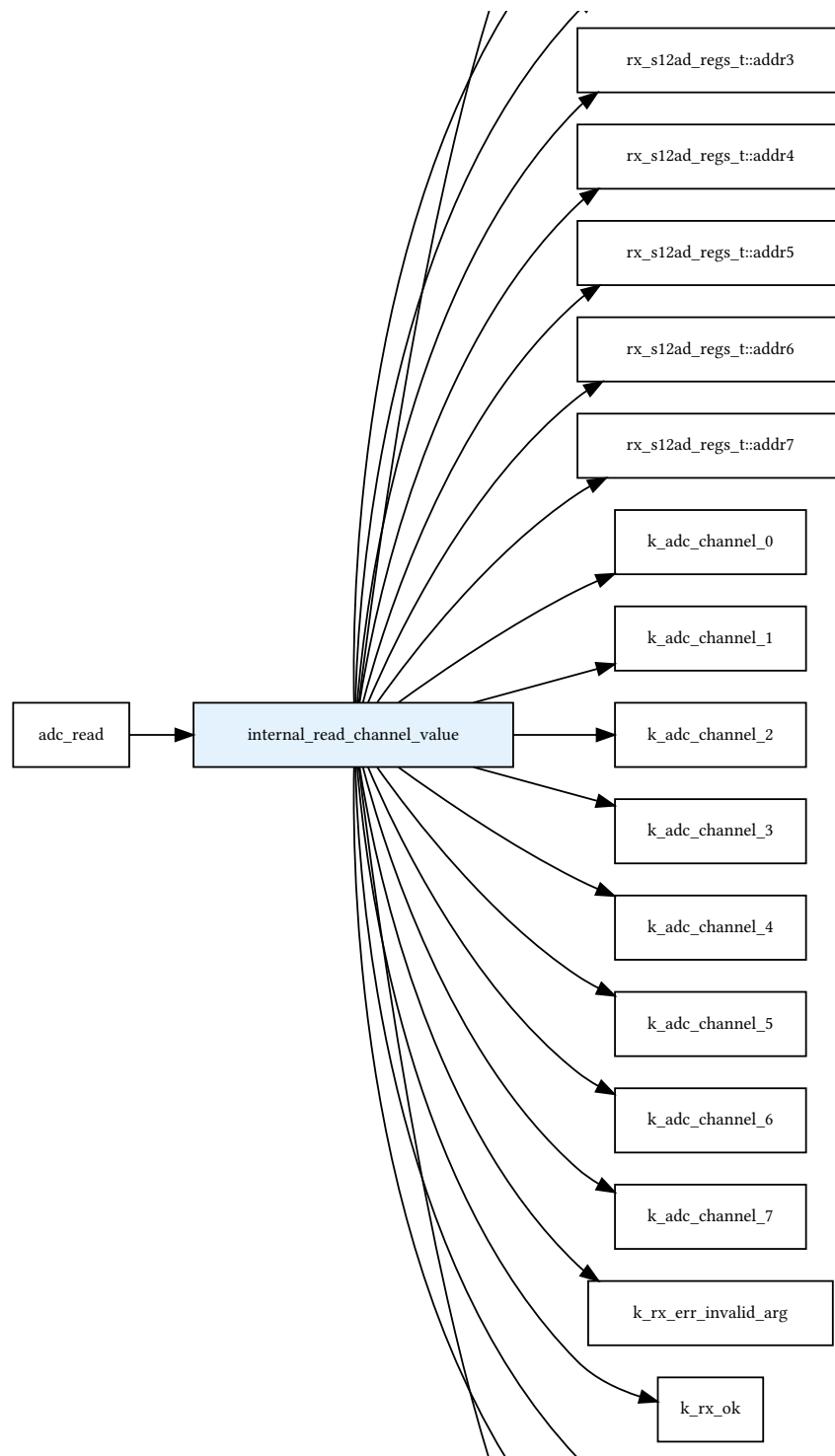
_Defined in libs/rx_hal/src/adc.c:580_
 —

internal_read_channel_value

```
static rx_err_t internal_read_channel_value(volatile rx_s12ad_regs_t *adc, const
adc_channel_t channel, uint16_t *value)
```

Read raw ADC value from a specific channel data register.

Helper function that reads the conversion result from the appropriate ADDRn register based on channel number. Extracted from `adc_read()` to support NASA Power of 10 Rule 4 (functions under 60 lines).

Figure 730: Call graph for `internal_read_channel_value`

_Defined in libs/rx_hal/src/adc.c:640_
—

adc_init

```
rx_err_t adc_init(const adc_unit_t unit, const adc_channel_t channel, const  
adc_resolution_t bits)
```

Initialize ADC unit and enable a specific channel.

Initialize ADC unit and channel.

Configures the S12ADFa peripheral for A/D conversion. On first call for each unit, enables the module clock, configures resolution, and initializes the hardware. Subsequent calls to the same unit enable additional channels.

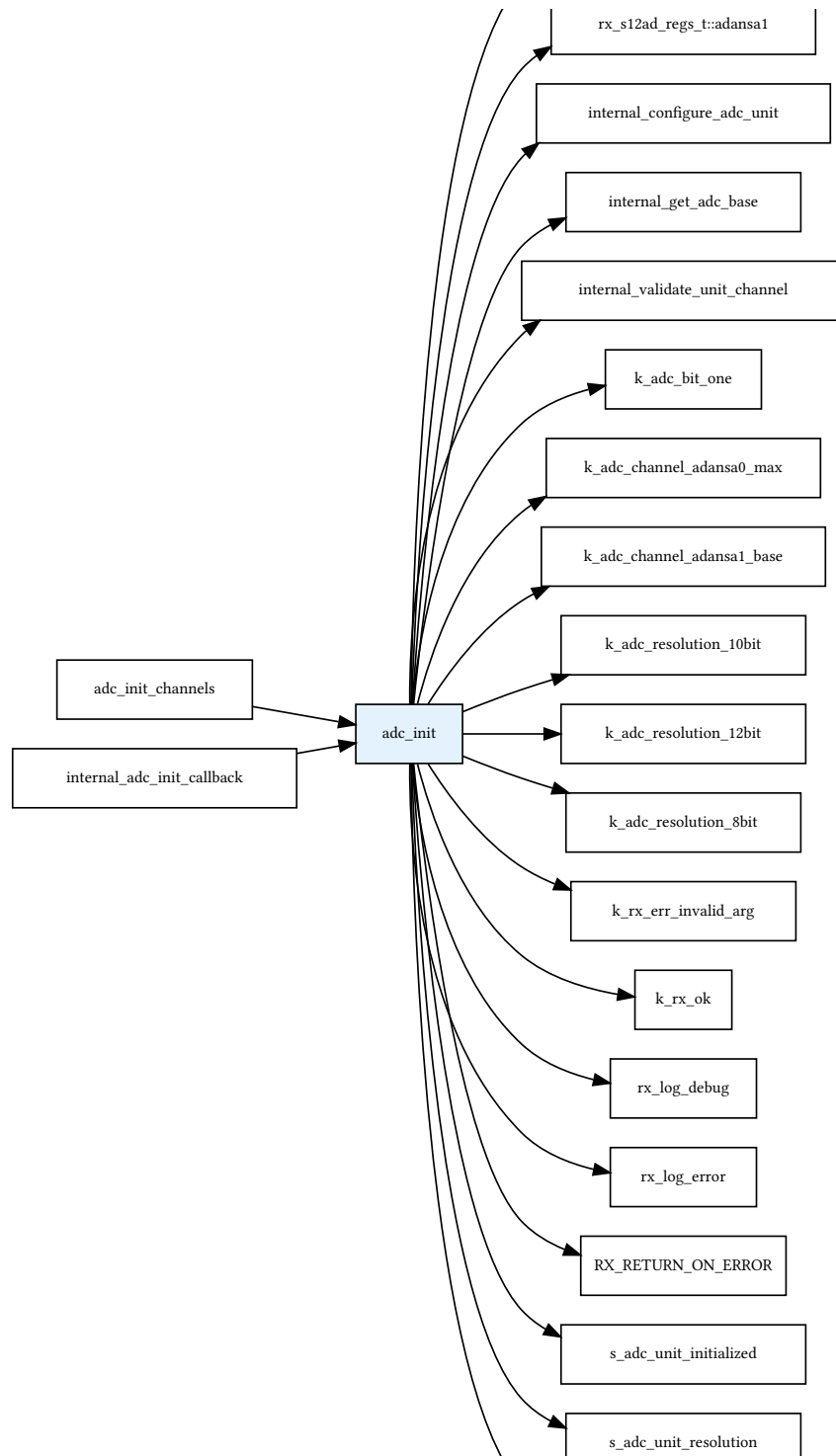


Figure 731: Call graph for `adc_init`

_Defined in libs/rx_hal/src/adc.c:737_
—

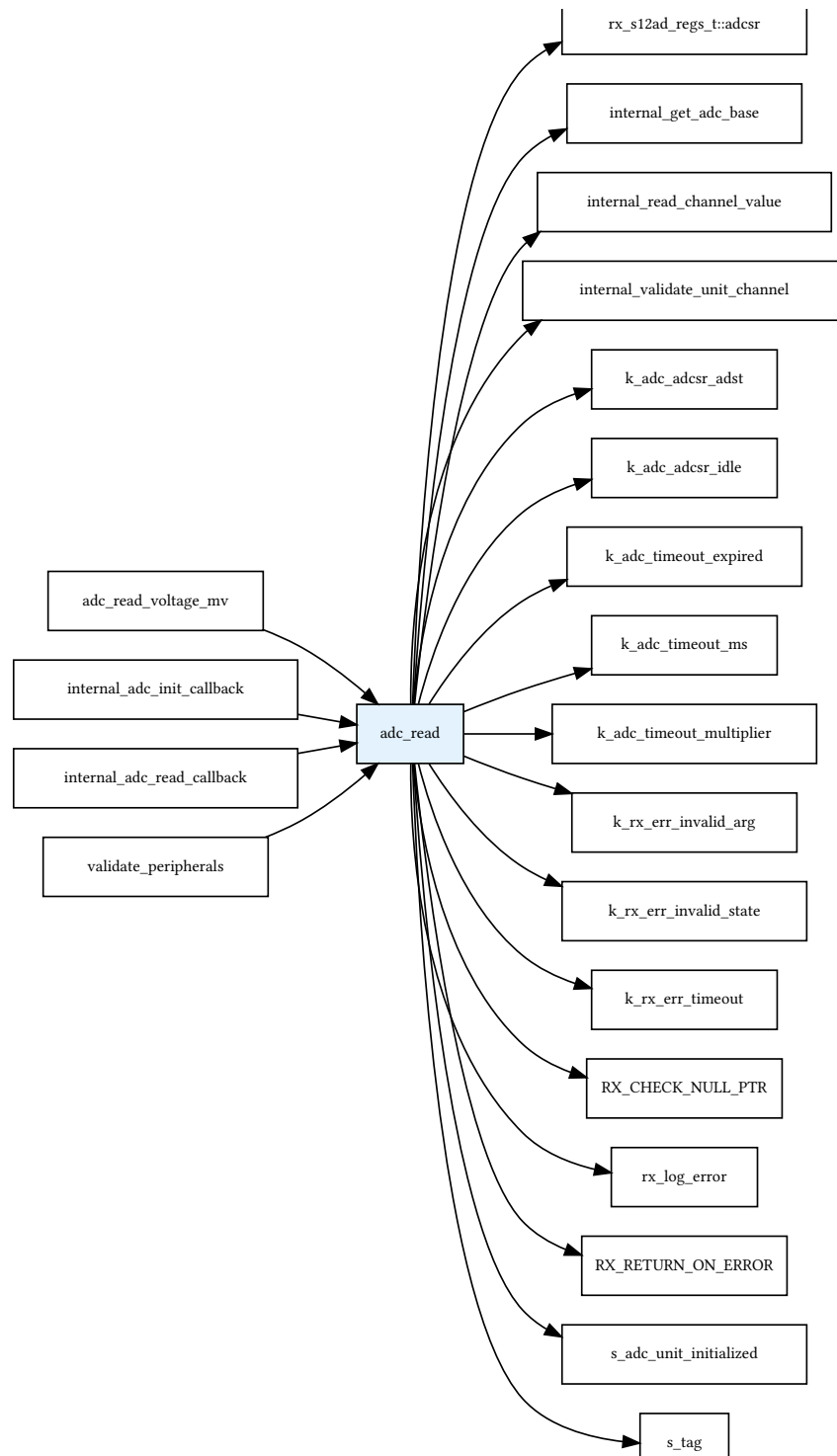
adc_read

```
rx_err_t adc_read(const adc_unit_t unit, const adc_channel_t channel, uint16_t  
*value)
```

Read raw ADC conversion value from specified channel.

Read ADC value.

Performs a single-scan A/D conversion and returns the raw digital value. This is a blocking operation that polls the ADST bit until conversion completes or timeout occurs.

Figure 732: Call graph for `adc_read`

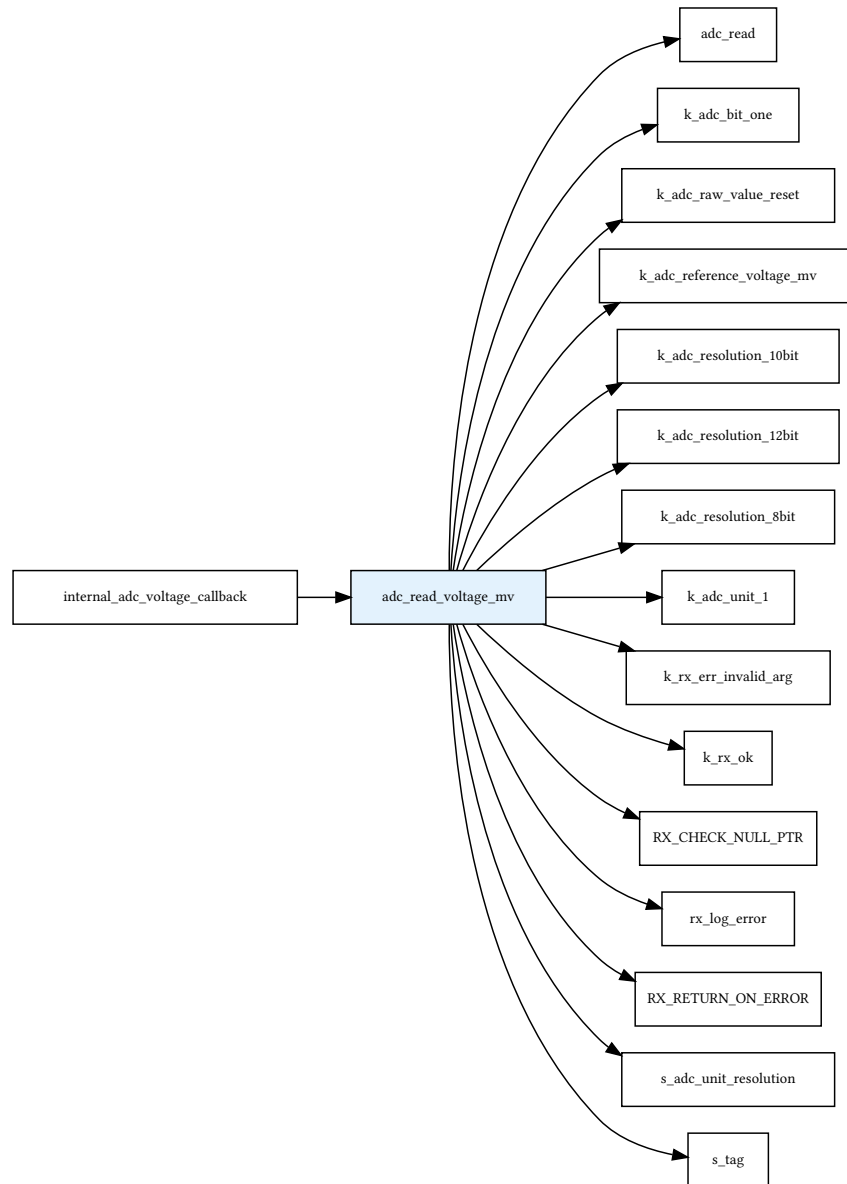
_Defined in libs/rx_hal/src/adc.c:852_
—

adc_read_voltage_mv

```
rx_err_t adc_read_voltage_mv(const adc_unit_t unit, adc_channel_t channel,  
adc_resolution_t bits, uint32_t *voltage_mv)
```

Read ADC value and convert to millivolts.

Performs an A/D conversion and scales the result to millivolts. This is a convenience function that calls `adc_read()` and applies the voltage calculation formula.

Figure 733: Call graph for `adc_read_voltage_mv`

_Defined in `libs/rx\_hal/src/adc.c:956_`

—

gpio.c

GPIO Driver for RX72N - General Purpose Input/Output Control.

Includes:

- <stddef.h>
- "hardware.h"
- "rx72n_regs.h"
- "rx_port_constants.h"
- "rx_port_utils.h"

gpio_bit_constants_t

GPIO register bit manipulation constants.

Defines constants used for bit manipulation in GPIO register operations. Using named constants instead of magic numbers improves code readability and makes the intent of bit operations clear.

Value	Name	Description
= 1	k_gpio_bit_set	Value for setting a single bit (1 < n pattern)
= 0	k_gpio_bit_clear	Value for comparison when checking if bit is cleared

internal_validate_port_pin

```
static rx_err_t internal_validate_port_pin(const uint8_t port, const uint8_t pin)
```

Validate port and pin numbers against RX72N hardware limits.

Performs validation of GPIO port and pin numbers before any register access. This function is called by all public GPIO functions to ensure safe operation and prevent out-of-bounds hardware register access.

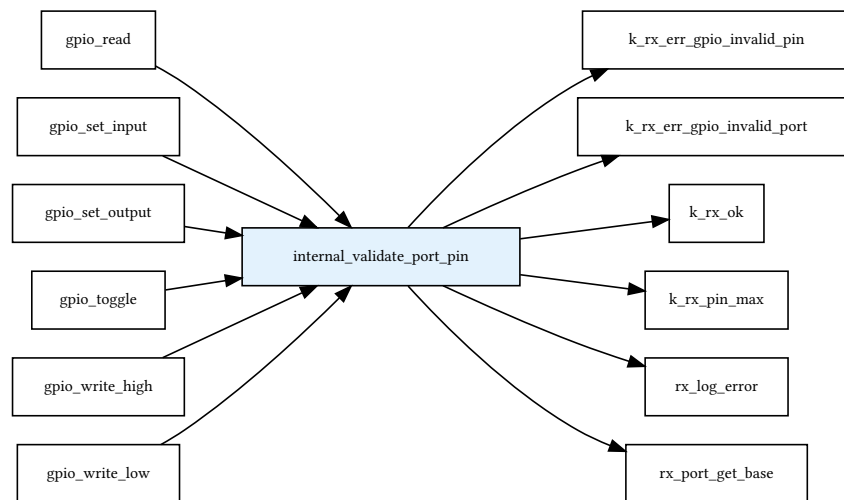


Figure 734: Call graph for internal_validate_port_pin

_Defined in libs/rx_hal/src/gpio.c:284_

gpio_set_output

```
rx_err_t gpio_set_output(const rx_port_pin_t pin)
```

Configure GPIO pin as digital output.

Configures a GPIO pin for digital output mode. This function performs:

1. Parameter validation (port and pin number)
2. Pin reservation via the global pin validator (conflict detection)
3. PMR register clear (select GPIO mode instead of peripheral function)
4. PDR register set (configure as output direction)

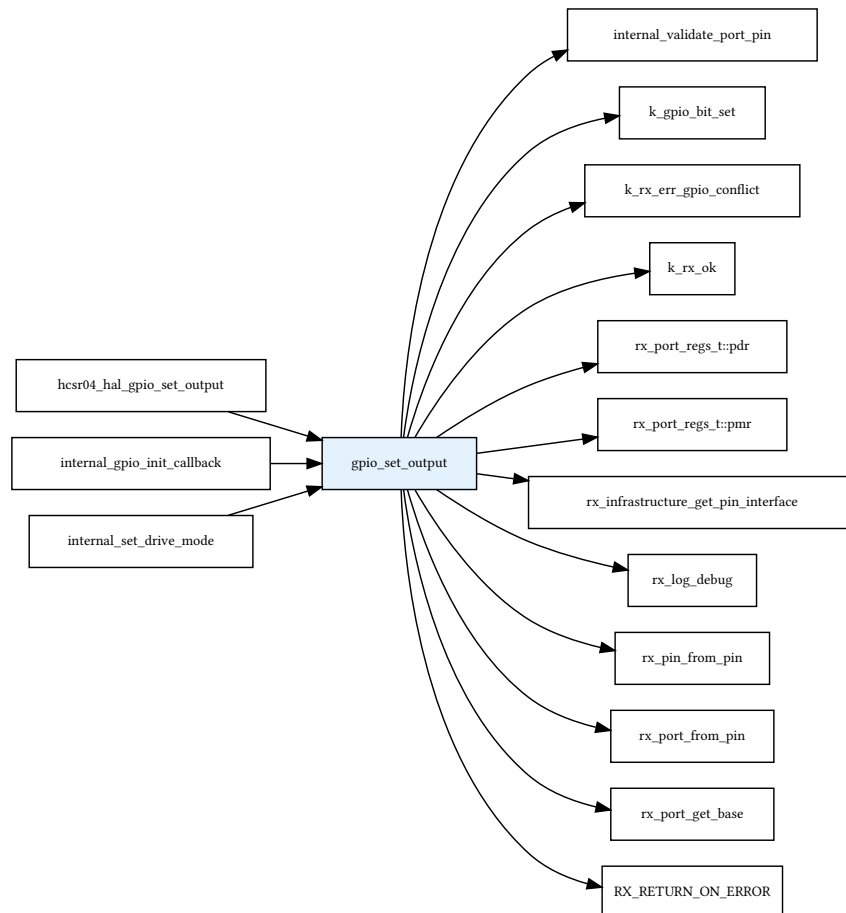


Figure 735: Call graph for `gpio_set_output`

_Defined in `libs/rx_hal/src/gpio.c:387_`

—

gpio_set_input

```
rx_err_t gpio_set_input(const rx_port_pin_t pin)
```

Configure GPIO pin as digital input.

Configures a GPIO pin for digital input mode. This function performs:

1. Parameter validation (port and pin number)
2. Pin reservation via the global pin validator (conflict detection)
3. PMR register clear (select GPIO mode instead of peripheral function)
4. PDR register clear (configure as input direction)

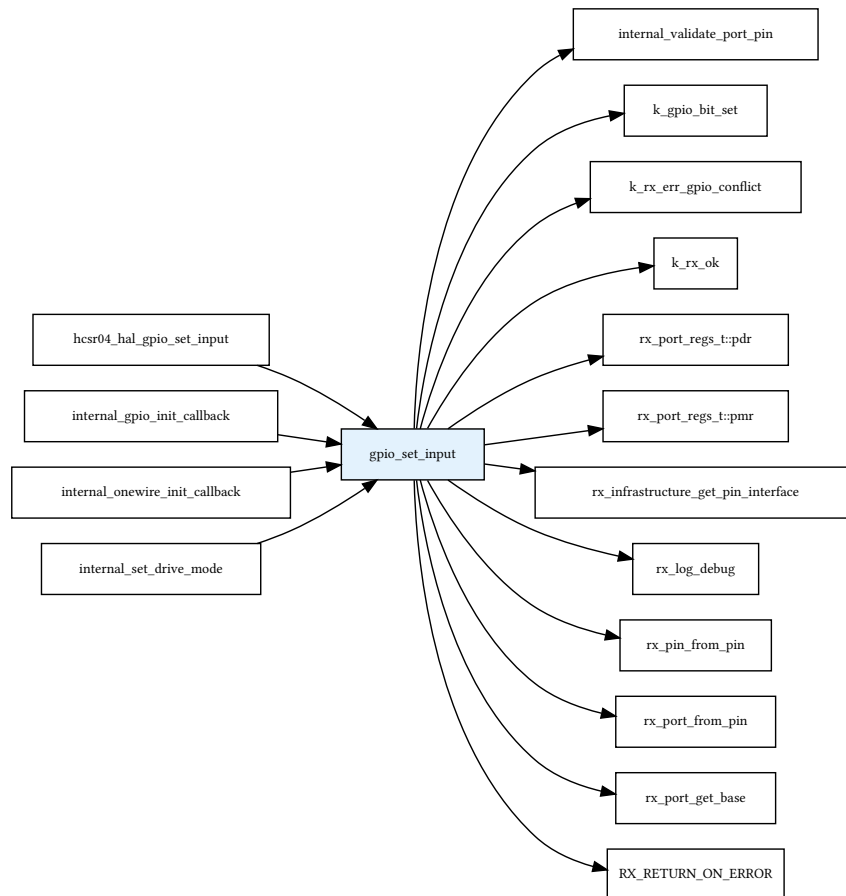


Figure 736: Call graph for `gpio_set_input`

_Defined in `libs/rx\_hal/src/gpio.c:501_`

gpio_write_high

```
rx_err_t gpio_write_high(const rx_port_pin_t pin)
```

Set GPIO output pin to logic high (3.3V).

Set GPIO output pin to logic high (VDD).

Sets a GPIO output pin to logic high level. On the RX72N, this means the pin outputs VCC (typically 3.3V). The pin must be configured as an output before calling this function.

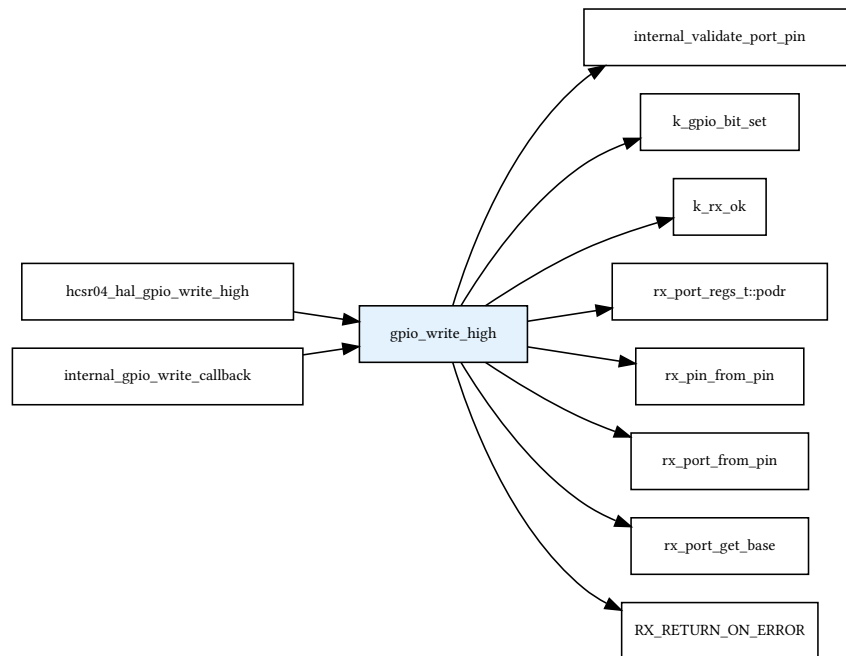


Figure 737: Call graph for `gpio_write_high`

_Defined in `libs/rx_hal/src/gpio.c:591_`

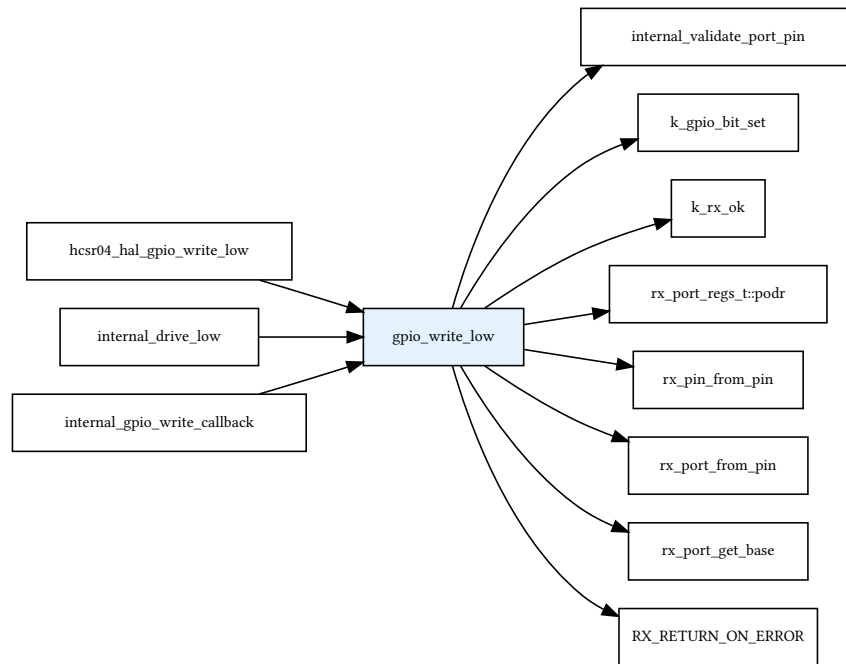
gpio_write_low

```
rx_err_t gpio_write_low(const rx_port_pin_t pin)
```

Set GPIO output pin to logic low (0V).

Set GPIO output pin to logic low (GND).

Sets a GPIO output pin to logic low level. On the RX72N, this means the pin outputs GND (0V). The pin must be configured as an output before calling this function.

Figure 738: Call graph for `gpio_write_low`

_Defined in `libs/rx\_hal/src/gpio.c:666_`

—

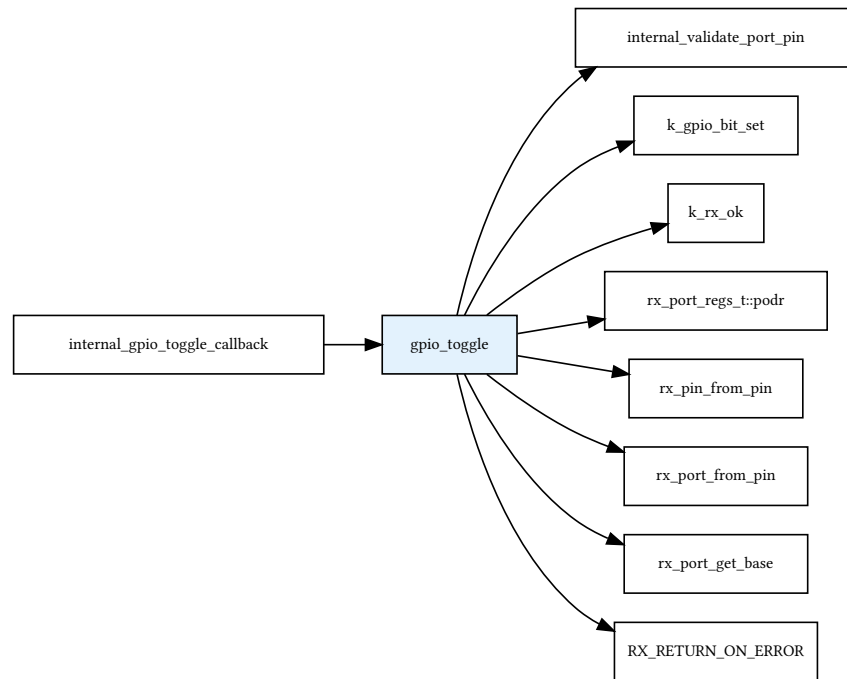
gpio_toggle

```
rx_err_t gpio_toggle(const rx_port_pin_t pin)
```

Toggle GPIO output pin state (high<->low).

Toggle GPIO output pin state (high to low, or low to high).

Inverts the current output state of a GPIO pin. If the pin is currently high, it becomes low. If currently low, it becomes high. Uses XOR operation for efficient toggling.

Figure 739: Call graph for `gpio_toggle`

_Defined in `libs/rx\hal/src/gpio.c:744_`

gpio_read

```
rx_err_t gpio_read(const rx_port_pin_t pin, bool *value)
```

Read GPIO input pin state.

Read current logic level of GPIO pin.

Reads the current logic level of a GPIO pin. Uses the PIDR (Port Input Data Register) which reflects the actual voltage level on the pin, regardless of whether the pin is configured as input or output.

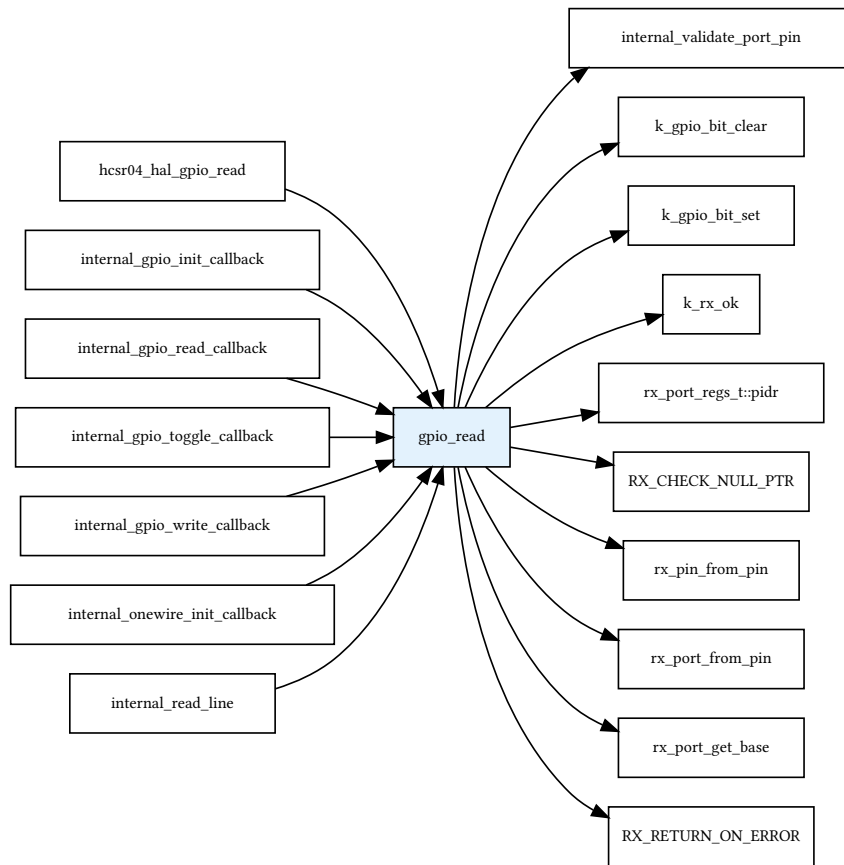


Figure 740: Call graph for gpio_read

_Defined in libs/rx_hal/src/gpio.c:850_

—

riic.c

RIIC (I2C) Driver Implementation for RX72N.

Production-grade implementation of the I2C Bus Interface (RIIC) peripheral driver for the RX72N microcontroller. This driver provides controller-mode I2C communication with support for standard (100 kHz), fast (400 kHz), and fast-plus (1 MHz) modes.

Includes:

- <string.h>
- "hardware.h"
- "rx72n_regs.h"
- "rx_register_protection.h"

riic_constants_t

RIIC channel count, timeout, and transfer limit constants.

General-purpose constants for RIIC driver operation including channel limits, timeout values, and transfer size constraints.

`k_riic_max_channels == 3` (hardware limitation)

`k_riic_max_transfer_length == 256` (single-byte length limit)

Value	Name	Description
= 3	<code>k_riic_max_channels</code>	Total RIIC channels: RIIC0, RIIC1, RIIC2
= 10000	<code>k_riic_timeout_us</code>	10ms timeout for I2C operations (microseconds)
= 0	<code>k_riic_timeout_zero</code>	Timeout counter expired sentinel value
= 0	<code>k_riic_length_zero</code>	Zero-length transfer (invalid) sentinel
= 1	<code>k_riic_last_index_offset</code>	Offset to calculate last byte index from length
= 256	<code>k_riic_max_transfer_length</code>	Maximum bytes per transfer operation

—

riic_channel_num_t

RIIC channel identifiers for switch statement dispatch.

Provides named constants for channel selection in `internal_get_riic_base()`. Using enums instead of raw integers ensures type safety and improves code readability in switch statements.

Values are contiguous starting from 0 and match hardware channel indices

Value	Name	Description
= 0	<code>k_riic_channel_0</code>	RIIC channel 0 (P16/SCL0, P17/SDA0)
= 1	<code>k_riic_channel_1</code>	RIIC channel 1 (P21/SCL1, P20/SDA1)
= 2	<code>k_riic_channel_2</code>	RIIC channel 2 (P66/SCL2, P67/SDA2)

See also: `internal_get_riic_base()` Maps channel number to hardware register pointer, `riic_constants_t` For `k_riic_max_channels` bound used with these values

Example:

```
//Selectchannel0baseaddress
volatile_riic_regs_t*regs=internal_get_riic_base(k_riic_channel_0);
```

—

riic_module_stop_bits_t

RIIC module stop bit positions in MSTPCRB register.

The RX72N uses a module stop mechanism to reduce power consumption. Each peripheral can be stopped/started by setting/clearing its corresponding bit in MSTPCRB. RIIC channels must be enabled (bit cleared) before use.

Value	Name	Description
= 21	<code>k_riic_mstpb_riic0</code>	MSTPCRB bit 21: RIIC0 module stop control
= 20	<code>k_riic_mstpb_riic1</code>	MSTPCRB bit 20: RIIC1 module stop control
= 19	<code>k_riic_mstpb_riic2</code>	MSTPCRB bit 19: RIIC2 module stop control

= 1	k_riic_mstpb_bit_value	Single bit value for bit manipulation
-----	------------------------	---------------------------------------

See also: rx_register_protection.h for PRCR register access

—

riic_frequency_t

Supported I2C bus frequencies (Hz).

I2C standard defines three speed grades. This driver supports all three modes with appropriate bit rate register configuration.

Only these three discrete values are accepted by riic_init()

Value	Name	Description
= 100000	k_riic_freq_100khz	Standard mode: 100 kHz (Sm)
= 400000	k_riic_freq_400khz	Fast mode: 400 kHz (Fm)
= 1000000	k_riic_freq_1mhz	Fast mode plus: 1 MHz (Fm+)

Note: Fast Plus mode requires pull-up resistors < 1kOhm for proper rise time.] **See also:** riic_init() Validates frequency_hz against these enum values, internal_calculate_bit_rate() Uses frequency to compute ICBRL/ICBRH

Example:

```
//Initialize channel at standard (100kHz) mode
riic_channel_tch={.value=0};
rx_err_t err=riic_init(ch,k_riic_freq_100khz);
```

—

riic_bit_rate_t

Bit rate calculation constants for ICBRL/ICBRH registers.

These constants are used in internal_calculate_bit_rate() to compute the correct ICBRL and ICBRH register values for a target frequency.

Where the factor of 3 accounts for:

- SCL high period (ICBRH + 1) cycles
- SCL low period (ICBRL + 1) cycles
- Internal synchronization overhead

Value	Name	Description
= 3	k_riic_brr_divisor	Divisor constant for 50% duty cycle formula
= 1	k_riic_brr_min	Minimum valid divisor (fastest clock)
= 255	k_riic_brr_max	Maximum valid divisor (8-bit register limit)

—

riic_icmr1_values_t

ICMR1 (Mode Register 1) configuration values.

ICMR1 configures the controller/peripheral mode and addressing type. For STAR project, we use controller mode with 7-bit addressing.

Value	Name	Description
= 8	k_riic_icmr1_controller_7bit	CKS=0, controller mode, 7-bit addressing

—

riic_icmr2_values_t

ICMR2 (Mode Register 2) configuration values.

ICMR2 configures hardware timeout detection and SDA output hold-time delay. The STAR project disables hardware timeout (TMOE = 0) because bus-busy detection is handled in software by internal_wait_bus_ready() using the k_riic_timeout_us countdown. The SDA output delay is also left at zero (no additional hold time) since the pull-up resistors on the target PCB provide adequate signal integrity at all supported speeds.

Value	Name	Description
= 0	k_riic_icmr2_default	No hardware timeout, no SDA delay

—

riic_icmr3_bits_t

ICMR3 (Mode Register 3) ACK/NACK control bit definitions.

ICMR3 controls ACK/NACK transmission during receive operations. The key field is ACKBT (Acknowledge Bit), which determines whether the controller sends an ACK or NACK after each received byte.

Value	Name	Description
= 3	k_riic_icmr3_ackbt_pos	ACKBT bit position (bit 3)
= (1U << k_riic_icmr3_ackbt_pos)	k_riic_icmr3_ackbt_mask	ACKBT bit mask (0x08)

—

riic_address_bits_t

I2C 7-bit address formatting constants.

I2C uses a 7-bit address plus a read/write direction bit in the first byte after START. The address is left-shifted by 1 and OR'd with the direction bit.

k_riic_addr_max_7bit == 127 (I2C spec: addresses 0x00-0x7F)

Value	Name	Description
= 1	k_riic_addr_shift	Left shift amount for 7-bit address
= 0	k_riic_addr_write_bit	R/W bit value for write operation
= 1	k_riic_addr_read_bit	R/W bit value for read operation
= 127	k_riic_addr_max_7bit	Maximum valid 7-bit address (0x7F)

See also: `internal_write_byte()` Uses these constants to format address bytes,
`riic_write()` Passes formatted address bytes during write transactions

Example:

```
//Format a write address byte for device at 0x50
riic_device_addr_t dev = {.value = 0x50};
uint8_t addr_byte = (uint8_t)((dev.value << k_riic_addr_shift) | k_riic_addr_write_bit);
```

—

s_tag

```
const char* s_tag
```

Logging tag for RIIC driver messages.

Note: Not thread-safe, but read-only after initialization.

—

s_riic_channel_initialized

```
bool s_riic_channel_initialized[k_riic_max_channels]
```

Channel initialization state tracking array.

Note: Not thread-safe; if multiple threads use RIIC, provide external sync.

Warning: Do not modify directly - only `riic_init()` should set these flags.

—

internal_get_riic_base

```
static volatile rx_riic_regs_t * internal_get_riic_base(const uint8_t channel)
```

Get RIIC channel register base address from channel number.

Maps an RIIC channel number (0-2) to its corresponding hardware register base address using a compile-time-dispatched switch statement. This function is the single point of truth for the channel-to-address mapping and provides a clean abstraction between the logical channel index used throughout the driver and the physical memory-mapped peripheral addresses defined by the RX72N hardware.

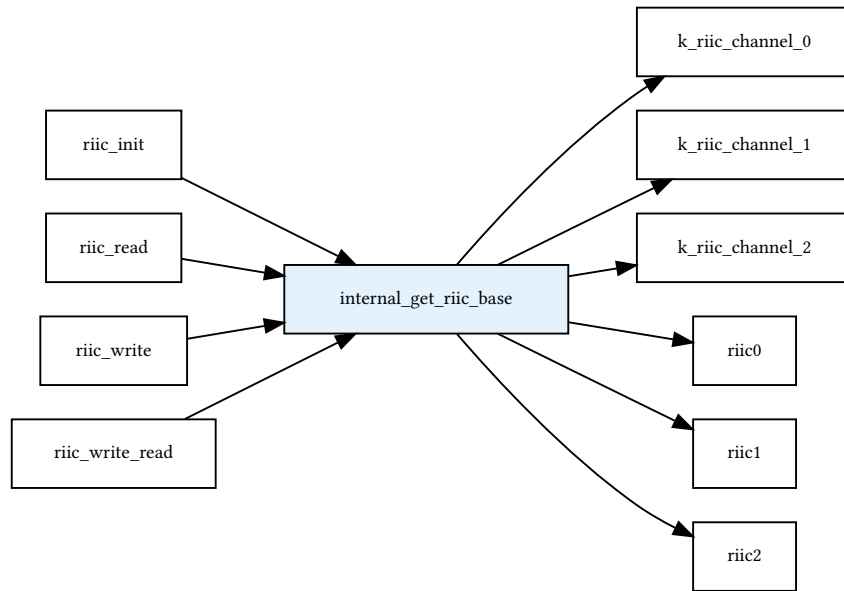


Figure 741: Call graph for internal_get_riic_base

_Defined in libs/rx_hal/src/riic.c:633_

internal_calculate_bit_rate

```
static rx_err_t internal_calculate_bit_rate(const uint32_t frequency_hz, uint8_t
*icbri, uint8_t *icbrh)
```

Calculate RIIC bit rate register values for target frequency.

Computes the ICBRL and ICBRH register values to achieve the desired I2C bus frequency. Uses a simplified calculation assuming 50% duty cycle (equal high and low periods).

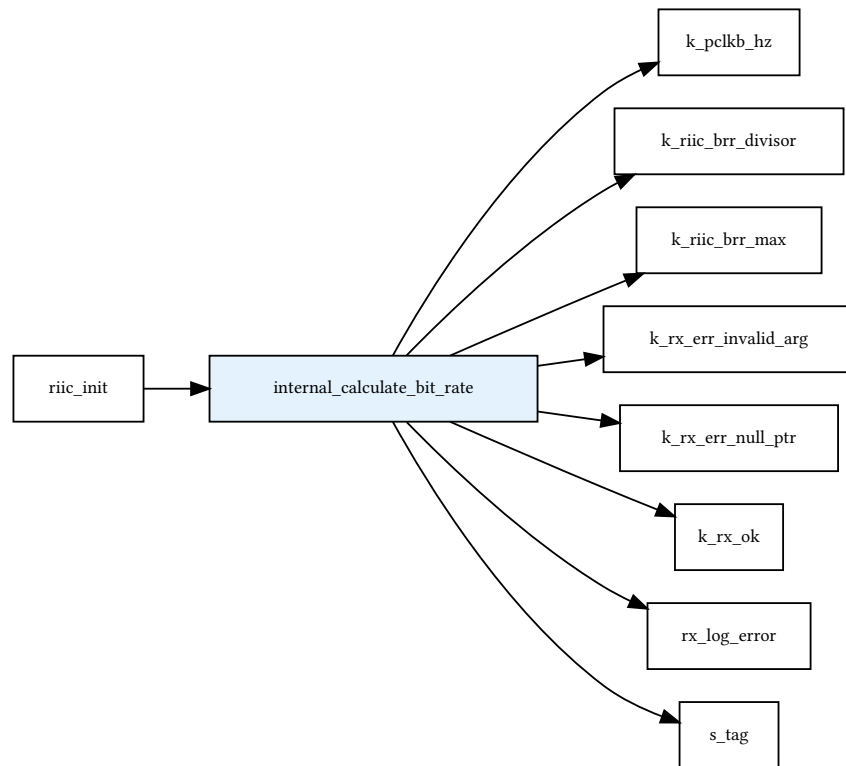


Figure 742: Call graph for internal_calculate_bit_rate

_Defined in `libs/rx\_hal/src/riic.c:705_`

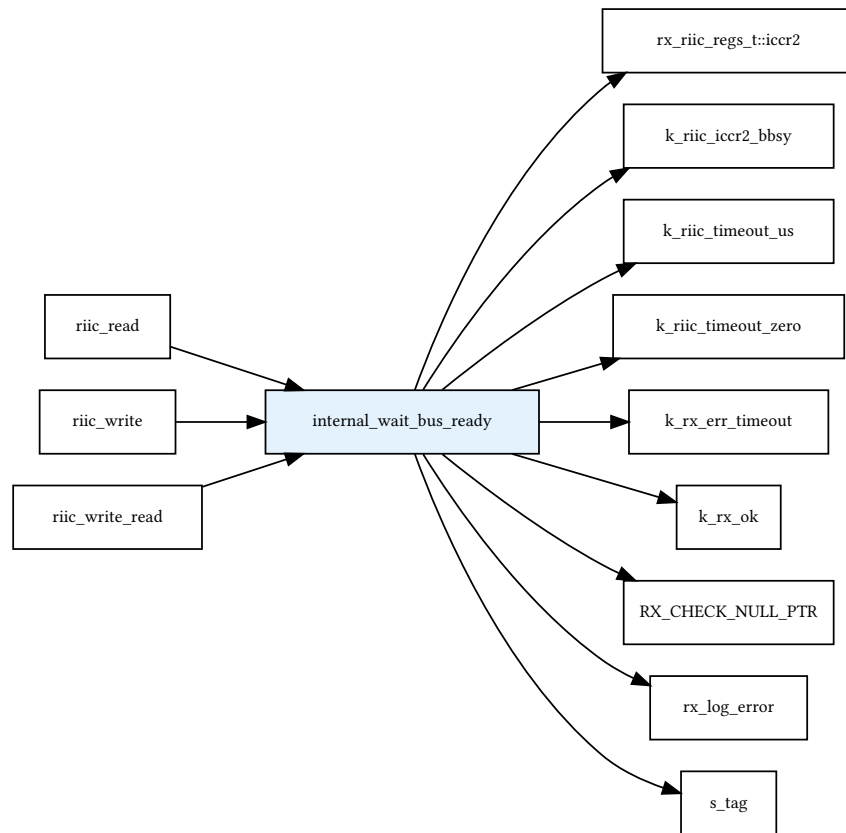
—

internal_wait_bus_ready

```
static rx_err_t internal_wait_bus_ready(const volatile rx_riic_regs_t *riic)
```

Wait for I2C bus to become idle (not busy).

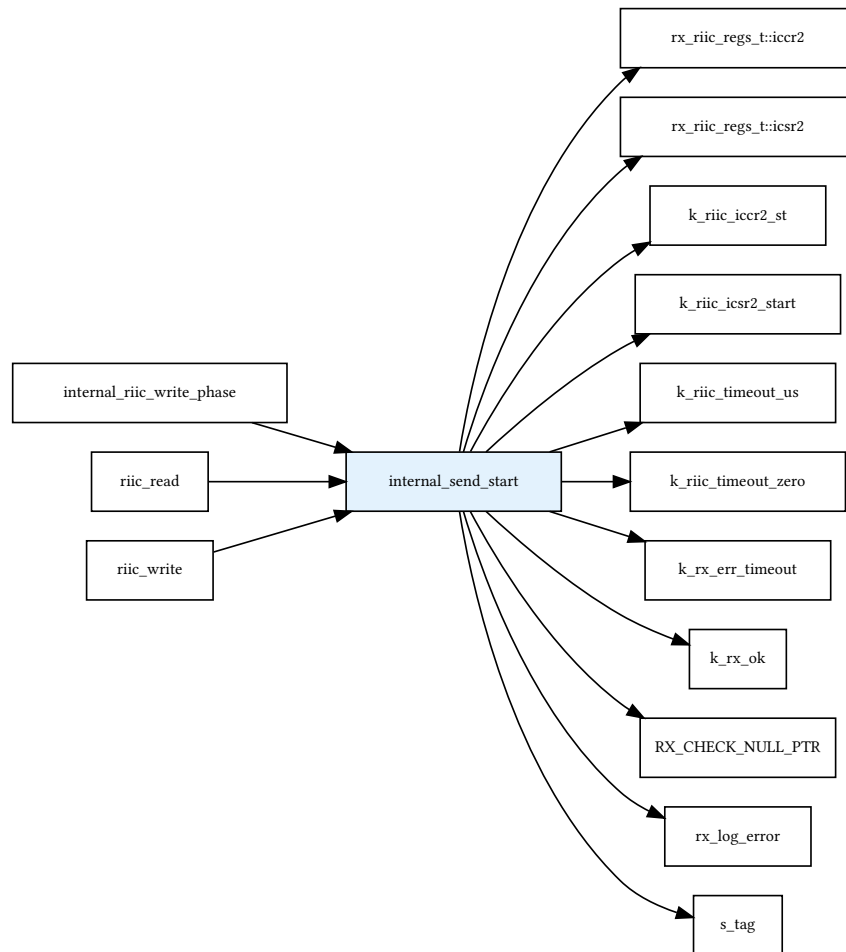
Polls the BBSY (Bus Busy) bit in ICCR2 until the bus is free or timeout occurs. This function must be called before starting a new I2C transaction to ensure no other controller is using the bus (multi-controller arbitration).

Figure 743: Call graph for `internal_wait_bus_ready`_Defined in `libs/rx\hal/src/riic.c:779_`**internal_send_start**

```
static rx_err_t internal_send_start(volatile rx_riic_regs_t *riic)
```

Generate I2C START condition on the bus.

Issues a START condition by setting the ST bit in ICCR2, then waits for hardware confirmation via the START flag in ICSR2. The START condition signals all peripherals that a transaction is beginning.

Figure 744: Call graph for `internal_send_start`_Defined in `libs/rx\hal/src/riic.c:845_`

—

internal_send_stop

```
static rx_err_t internal_send_stop(volatile rx_riic_regs_t *riic)
```

Generate I2C STOP condition on the bus.

Issues a STOP condition by setting the SP bit in ICCR2, then waits for hardware confirmation via the STOP flag in ICSR2. The STOP condition releases the bus for other controllers.

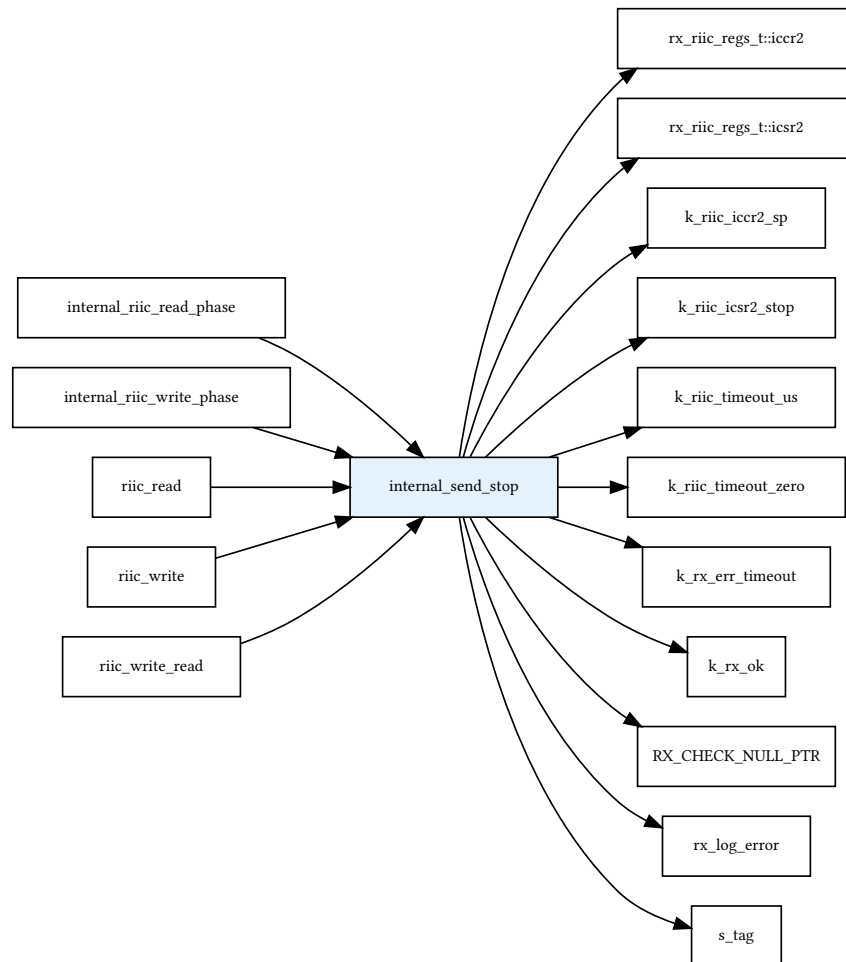


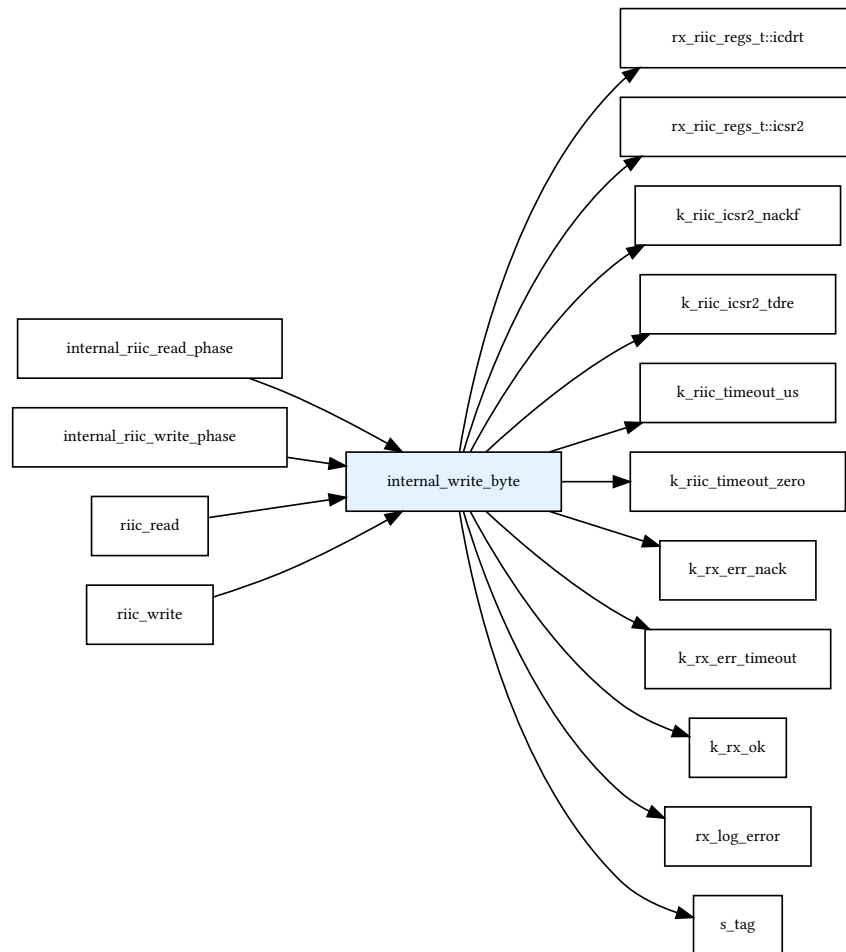
Figure 745: Call graph for internal_send_stop

_Defined in `libs/rx_hal/src/riic.c:931_`**internal_write_byte**

```
static rx_err_t internal_write_byte(volatile rx_riic_regs_t *riic, const uint8_t data)
```

Transmit single byte over I2C bus.

Writes one byte to the I2C bus and waits for acknowledgment from the peripheral. This function handles both address bytes and data bytes.

Figure 746: Call graph for `internal_write_byte`_Defined in `libs/rx\hal/src/riic.c:1015_`

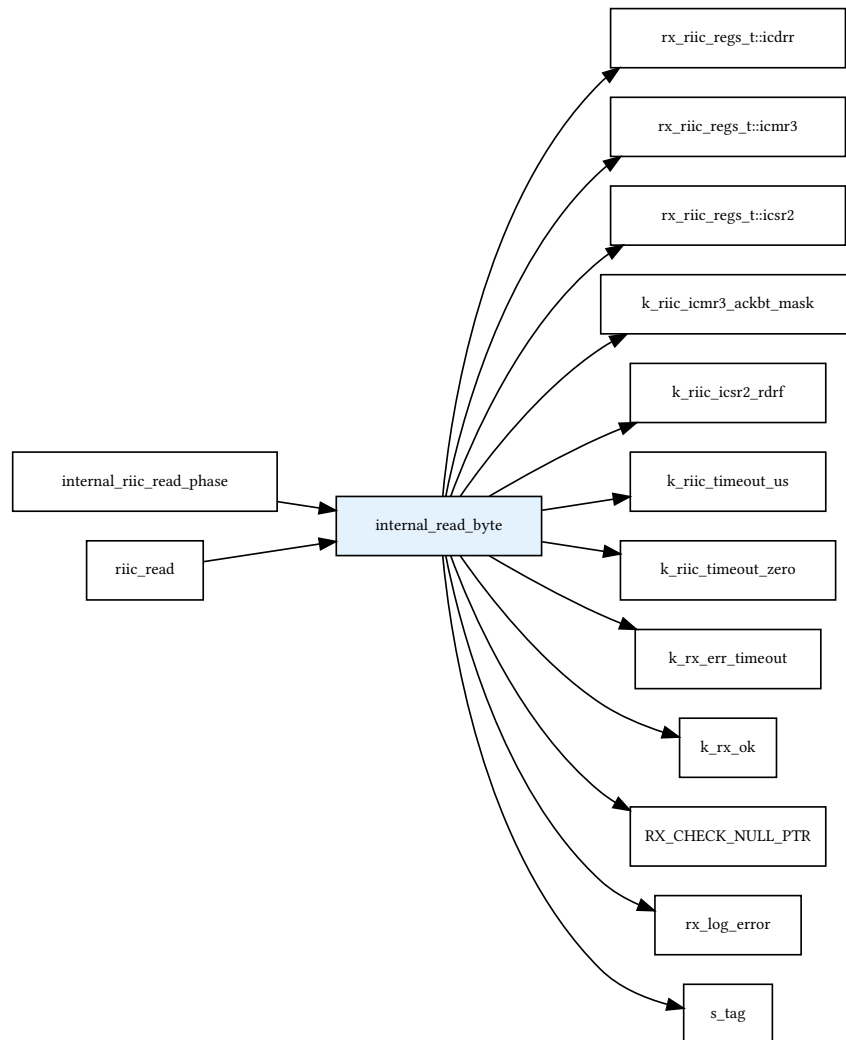
—

internal_read_byte

```
static rx_err_t internal_read_byte(volatile rx_riic_regs_t *riic, uint8_t *data,
const bool send_ack)
```

Receive single byte from I2C bus.

Reads one byte from the I2C bus and sends ACK or NACK as specified. The ACK/NACK response is configured before reading the byte from the receive register.

Figure 747: Call graph for `internal_read_byte`_Defined in `libs/rx\_hal/src/riic.c:1121_`**internal_riic_write_phase**

```

static rx_err_t internal_riic_write_phase(volatile rx_riic_regs_t *riic, const
i2c_device_addr_t device_addr, const uint8_t *write_data, const uint16_t
write_length)

```

Execute I2C write phase for combined write-read transfer.

Performs the write portion of a write-read (register read) transaction. This typically writes a register address to the peripheral, which sets an internal pointer for the subsequent read phase.

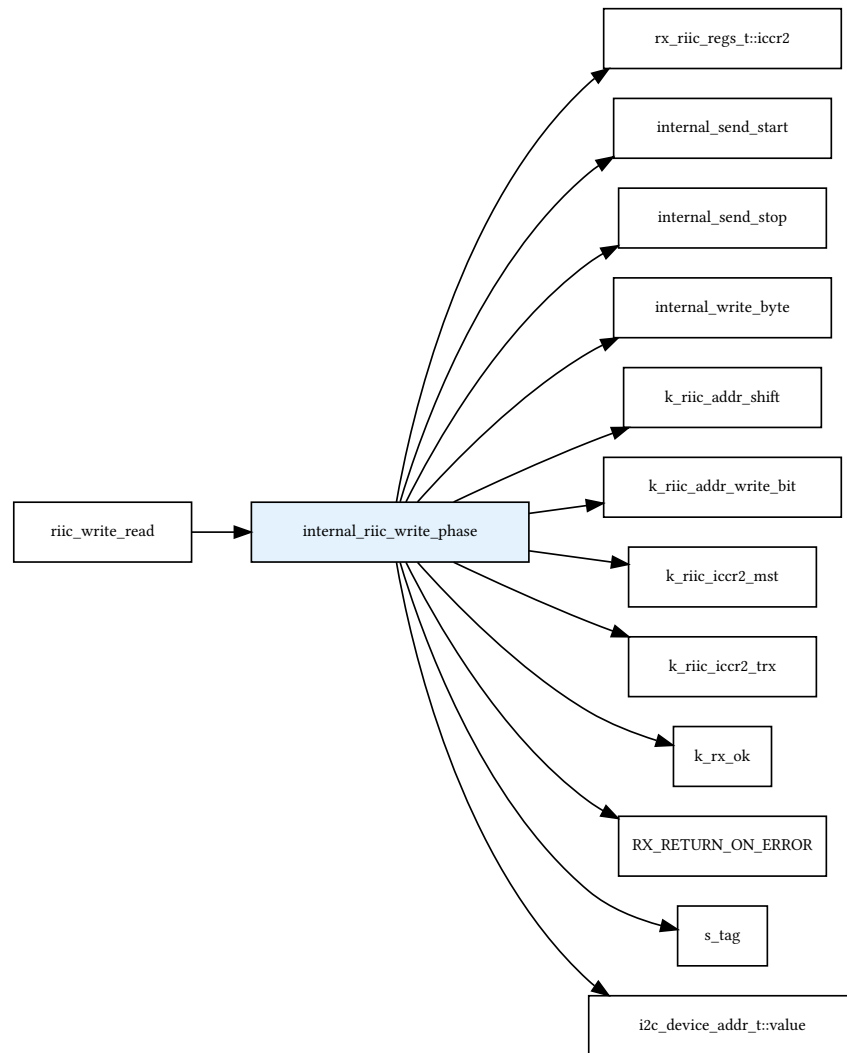


Figure 748: Call graph for internal_riic_write_phase

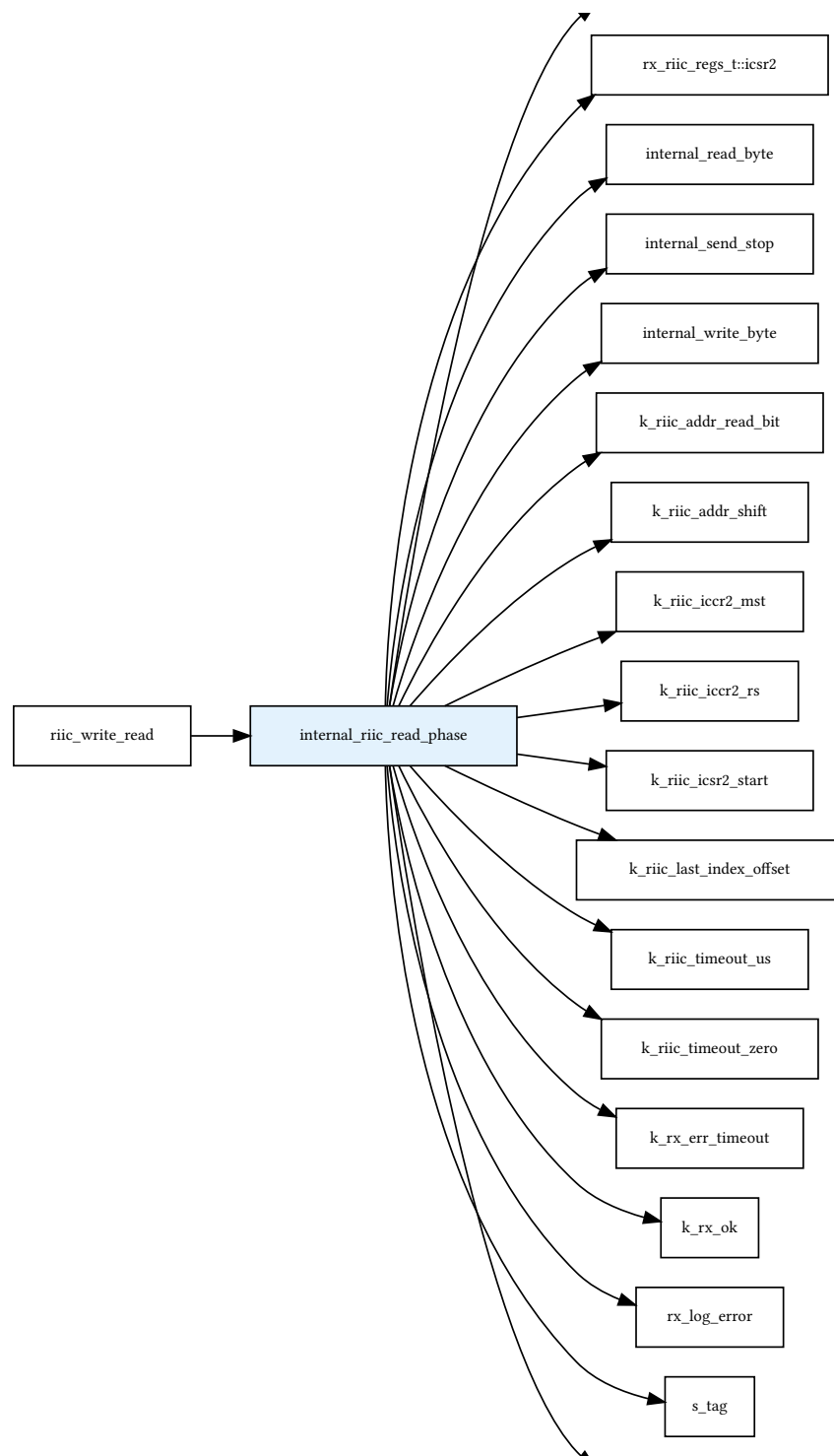
_Defined in libs/rx_hal/src/riic.c:1204_

internal_riic_read_phase

```
static rx_err_t internal_riic_read_phase(volatile rx_riic_regs_t *riic, const
i2c_device_addr_t device_addr, uint8_t *read_data, const uint16_t read_length)
```

Execute I2C read phase for combined write-read transfer.

Performs the read portion of a write-read (register read) transaction. Issues a repeated START condition (without releasing the bus), then reads data bytes from the peripheral.

Figure 749: Call graph for `internal_riic_read_phase`

_Defined in libs/rx_hal/src/riic.c:1301_

—

riic_init

```
rx_err_t riic_init(const riic_channel_t channel, const uint32_t frequency_hz)
```

Initialize RIIC (I2C) channel for controller mode operation.

Initialize RIIC channel for I2C controller communication.

Configures an RIIC channel for I2C controller mode communication at the specified bus frequency.
This function must be called before any I2C transfers on the channel.

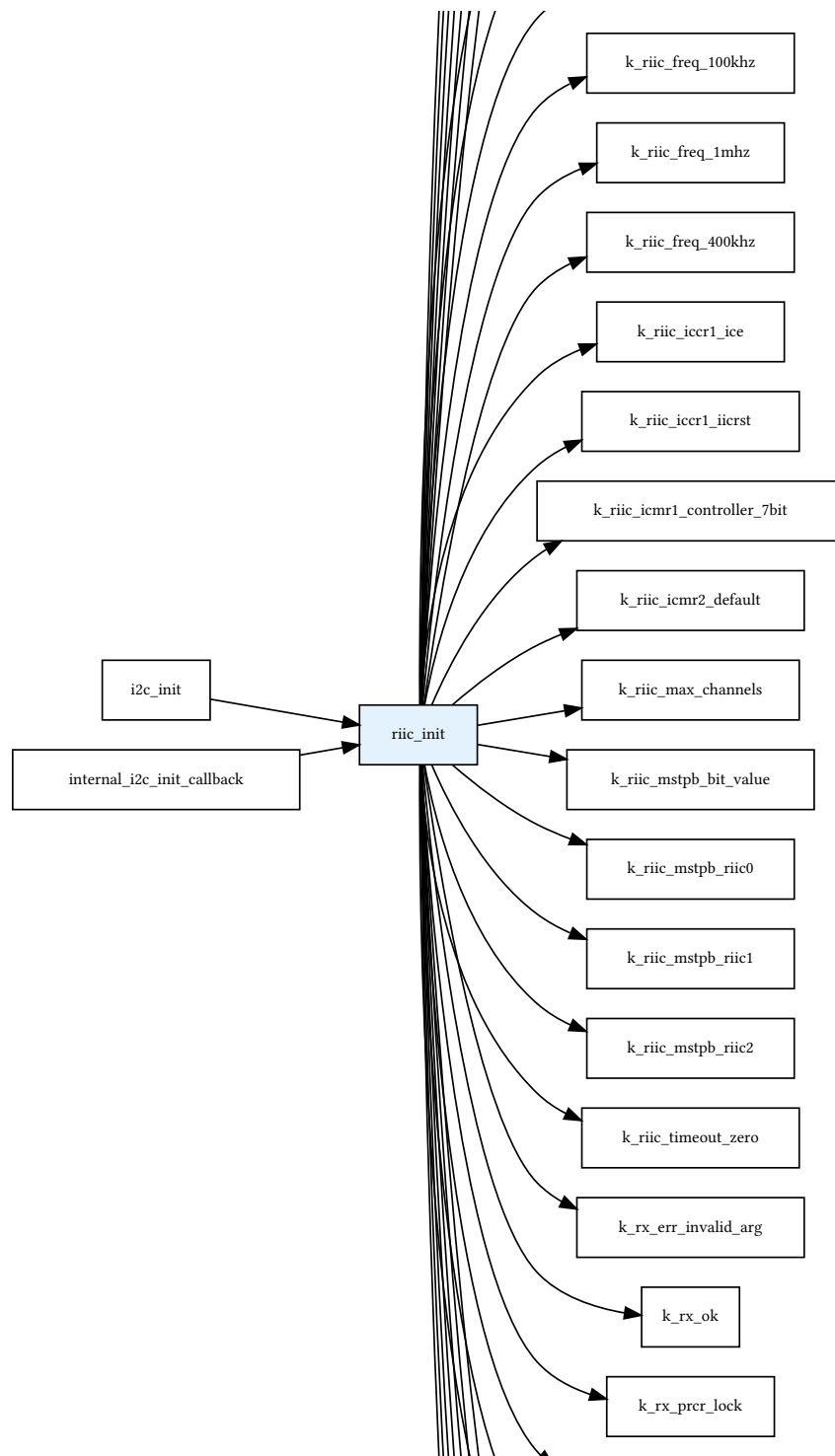


Figure 750: Call graph for riic_init

_Defined in libs/rx_hal/src/riic.c:1431_

—

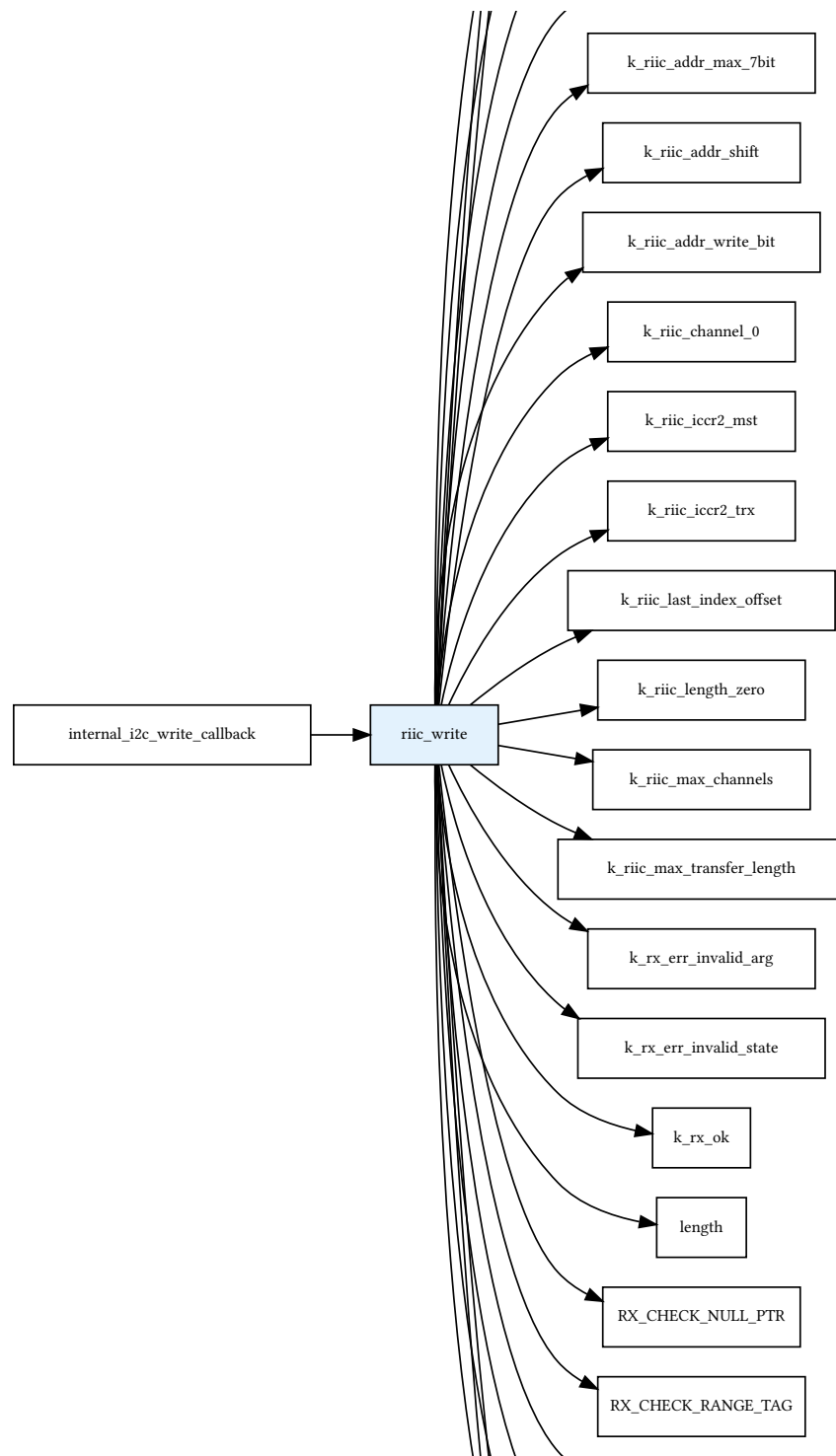
riic_write

```
rx_err_t riic_write(const riic_channel_t channel, const i2c_device_addr_t  
device_addr, const uint8_t *data, const uint16_t length)
```

Write data to an I2C peripheral device.

Write data to I2C device on specified RIIC channel.

Performs a complete I2C write transaction: START, address+W, data bytes, STOP. This function is suitable for writing configuration registers, sending commands, or storing data in EEPROM.

Figure 751: Call graph for `riic_write`

_Defined in libs/rx_hal/src/riic.c:1577_
—

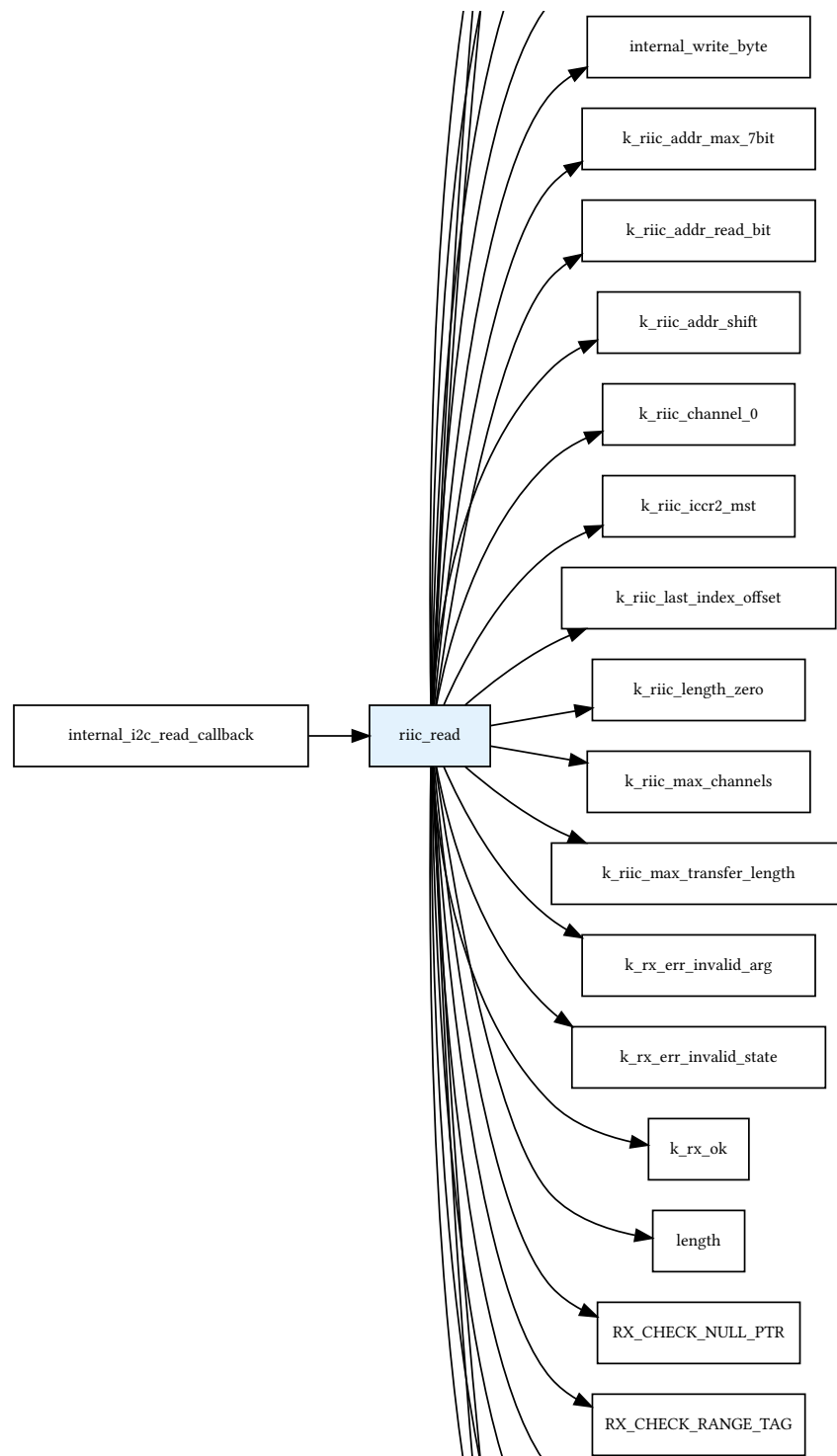
riic_read

```
rx_err_t riic_read(const riic_channel_t channel, const i2c_device_addr_t  
device_addr, uint8_t *data, const uint16_t length)
```

Read data from an I2C peripheral device.

Read data from I2C device on specified RIIC channel.

Performs a complete I2C read transaction: START, address+R, receive data bytes (ACK all except last), NACK last byte, STOP. This function is suitable for reading device status, sensor data, or sequential memory reads where the device auto-increments its internal pointer.

Figure 752: Call graph for `riic_read`

_Defined in libs/rx_hal/src/riic.c:1737_
—

riic_write_read

```
rx_err_t riic_write_read(const riic_channel_t channel, const i2c_device_addr_t  
device_addr, const uint8_t *write_data, const uint16_t write_length, uint8_t  
*read_data, const uint16_t read_length)
```

Combined write-then-read I2C transaction (register read).

Write then read from I2C device using repeated START (combined transaction).

Performs a complete I2C write-read transaction using a repeated START condition. This is the most common pattern for reading registers from I2C peripherals: write the register address, then read the data.



Figure 753: Call graph for `riic_write_read`

_Defined in libs/rx_hal/src/riic.c:1928_

—

rspi.c

RSPI (Renesas Serial Peripheral Interface) Driver for RX72N.

Provides complete SPI driver implementation supporting both peripheral mode (RX72N as SPI peripheral with RPi5 as controller) and controller mode (RX72N as SPI controller for sensor communication). Implements full-duplex communication with configurable clock frequency, SPI modes 0-3, and automatic chip select management.

STAR Team

2026-01-01

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdint.h>
- <string.h>
- "hardware.h"
- "rx72n_regs.h"
- "rx_gpio_constants.h"
- "rx_port_constants.h"
- "rx_port_utils.h"
- "rx_register_protection.h"

rspi_constants_t

RSPI channel count, timeout, and general constants.

Core configuration constants for RSPI driver. Timeout values are calibrated for worst-case SPI peripheral response times at minimum clock frequency.

Value	Name	Description
= 3	k_rspi_max_channels	Number of RSPI channels: RSPI0, RSPI1, RSPI2. Value: 3
= 10000	k_rspi_timeout_us	Timeout for TX/RX wait operations in loop iterations. Approximately 10 ms at typical loop overhead. Value: 10000 iterations (10 ms)
= 0	k_rspi_timeout_zero	Sentinel value indicating timeout has expired. Value: 0
= 0	k_rspi_len_zero	Sentinel for zero-length transfer (invalid). Value: 0
= 0	k_rspi_zero_u32	Zero constant for uint32_t comparisons. Value: 0
= 1	k_rspi_spbr_offset	Offset for SPBR calculation formula. $SPBR = (PCLKB / (2 * freq)) - 1$ Value: 1
= 0	k_rspi_loop_start	Starting index for bounded loops (NASA Rule 2). Value: 0

Since Version 1.0.0

—

rspi_cs_defaults_t

RSPI CS default constants (uninitialized state).

Value	Name	Description
= 0	k_rspi_cs_default_port	Default CS port (unconfigured)
= 0	k_rspi_cs_default_pin	Default CS pin (unconfigured)
= 0	k_rspi_spbr_init	Initial SPBR value

—

rspi_module_stop_bits_t

RSPI module stop bit positions in MSTPCRB.

Value	Name	Description
= 17	k_rspi_mstpb_rspi0	RSPI0 module stop bit
= 16	k_rspi_mstpb_rspi1	RSPI1 module stop bit
= 15	k_rspi_mstpb_rspi2	RSPI2 module stop bit

—

rspi_controller_constants_t

RSPI controller mode constants.

Value	Name	Description
= 60000000	k_rspi_pclkb_hz	PCLKB clock frequency (60 MHz)
= 100000	k_rspi_min_freq_hz	Minimum SPI clock (100 kHz)
= 10000000	k_rspi_max_freq_hz	Maximum SPI clock (10 MHz)
= 255	k_rspi_spbr_max	Maximum SPBR value
= 2	k_rspi_spbr_divisor	SPBR divisor factor
= 10	k_rspi_cs_setup_delay	CS setup delay iterations (300ns)
= 10	k_rspi_cs_hold_delay	CS hold delay iterations (300ns)
= 10000	k_rspi_max_delay_cycles	Sanity limit for timing delays

—

rspi_mode_limits_t

SPI mode limits.

Value	Name	Description
= 0	k_rspi_mode_min	Minimum valid SPI mode (0)
= 3	k_rspi_mode_max	Maximum valid SPI mode (0-3)

—

rspi_spcmd_bits_t

SPCMD register bit positions and masks.

Value	Name	Description
-------	------	-------------

= 1U	k_rsipi_spcmd_cpha_mask	CPHA bit mask (bit 0)
= 2U	k_rsipi_spcmd_cpol_mask	CPOL bit mask (bit 1)
= 1	k_rsipi_spcmd_cpol_pos	CPOL bit position
= 0	k_rsipi_spcmd_cpha_pos	CPHA bit position
= 8	k_rsipi_spcmd_spl_shift	SPL (data length) bit shift
= 7U	k_rsipi_spcmd_8bit	8-bit data length value
= 15U	k_rsipi_spcmd_16bit	16-bit data length value

—

rsipi_spdcr_local_t

SPDCR register bit positions (local).

Value	Name	Description
= 4	k_rsipi_spdcr_splw_pos	SPLW bit position (word access)
= 0U	k_rsipi_spdcr_byte_mode	Byte access mode

—

rsipi_register_defaults_t

RSPI register default values (reset/disabled state).

Values written to RSPI registers during initialization to establish a known baseline configuration. These match the hardware reset defaults documented in the RX72N Hardware Manual Section 38.2. Writing these values before configuring mode-specific settings ensures a deterministic starting state regardless of prior peripheral activity.

All values must match the hardware manual reset state

Value	Name	Description
= 0U	k_rsipi_sppcr_no_loopback	SPPCR: no loopback mode (normal operation)
= 0U	k_rsipi_spcr_disabled	SPCR: SPI disabled (module in reset state)

See also: `rsipi_init_peripheral()` Uses these during peripheral mode init, `internal_configure_registers()` Uses these during controller mode init

Example:

```
//ResetRSPIregisterstodefaultsbeforeconfiguration
rsipi->sppcr=k_rsipi_sppcr_no_loopback;
rsipi->spcr=k_rsipi_spcr_disabled;
```

Since Version 1.0.0

—

rsipi_bit_constants_t

RSPI bit manipulation constants for register field operations.

Provides named constants for single-bit set operations used to construct register masks via left-shift. Using a named constant instead of a literal 1 improves readability and grep-ability of register

manipulation code. All register mask construction in this module uses `k_rspi_bit_set` as the base value to ensure consistent, self-documenting bit manipulation.

`k_rspi_bit_set` must always equal 1

Value	Name	Description
= 1UL	<code>k_rspi_bit_set</code>	Single bit set value (1) for shift-based register mask construction

See also: `internal_set_mstpcrb_for_channel()` Uses for MSTPCRB mask construction, `internal_configure_spcmd()` Uses for CPOL/CPHA bit setting

Example:

```
//Construct a mask for MSTPCRB module stop bit
uint32_t mask = (k_rspi_bit_set << k_rspi_mstpb_rspi0);
system_regs() -> mstpcrb &= ~mask;
```

Since Version 1.0.0

—

rspi_transfer_constants_t

RSPI transfer length and command register constants.

Defines limits and initialization values for the RSPI transfer engine. The maximum transfer length is constrained by the 16-bit DMA counter. SPCMD initial value sets a clean baseline before configuring bit fields for CPOL, CPHA, and data length via bitwise OR operations.

`k_rspi_transfer_len_max` must equal 65535 (uint16_t maximum, 16-bit DMA counter limit)

Value	Name	Description
= 65535U	<code>k_rspi_transfer_len_max</code>	Maximum transfer length (16-bit counter limit)
= 0U	<code>k_rspi_spcmd_init</code>	SPCMD initial value (all fields at reset state)

See also: `rspi_peripheral_transfer()` Uses `k_rspi_transfer_len_max` for length validation, `rspi_init_peripheral()` Uses `k_rspi_spcmd_init` for register setup

Example:

```
//Validate transfer length before starting peripheral transfer
if (length == k_rspi_len_zero || length > k_rspi_transfer_len_max) {
    return k_rx_err_invalid_arg;
}
//Initialize SPCMD before configuring mode-specific bits
uint16_t spcmd = k_rspi_spcmd_init;
```

Since Version 1.0.0

—

s_tag

```
const char* s_tag
```

—

s_rspi_channel_initialized

```
bool s_rspi_channel_initialized[k_rspi_max_channels]
```

s_rspi_controller_initialized

```
bool s_rspi_controller_initialized[k_rspi_max_channels]
```

s_rspi_cs_config

```
rspi_cs_config_t s_rspi_cs_config[k_rspi_max_channels]
```

internal_get_rspi_base

```
static volatile rx_rspi_regs_t * internal_get_rspi_base(const rspi_channel_t
channel)
```

Get RSPI register base address from channel number.

Maps a channel index (0-2) to the corresponding RSPI peripheral register block via the inline accessor functions `rspi0()/rspi1()/rspi2()`. Returns `nullptr` for out-of-range channel numbers to allow callers to detect invalid input without risking a wild pointer dereference.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: 0, 1, or 2)

Returns: Pointer to RSPI register base for the given channel

Value	Description
non-null	Valid register base pointer
<code>nullptr</code>	Invalid channel number

Pre: channel must be in range 0, `k_rspi_max_channels`)

Pre: RSPI hardware must be present (RX72N 144-pin package)

Post: Returned pointer (if non-null) is aligned and points to valid HW registers

Post: No side effects on hardware state

Note: Not thread-safe. Caller must provide synchronization if needed.] **See also:** `rspi0()` Channel 0 register accessor, `rspi1()` Channel 1 register accessor, `rspi2()` Channel 2 register accessor

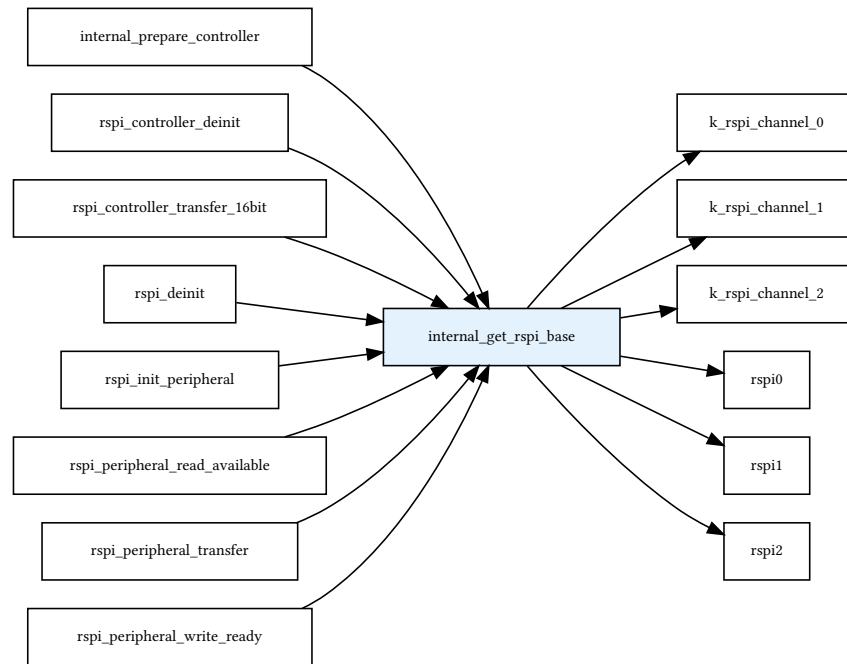


Figure 754: Call graph for internal_get_rspi_base

_Defined in libs/rx_hal/src/rspi.c:394_

Since Version 1.0.0

—

internal_set_mstpcrb_for_channel

```
static void internal_set_mstpcrb_for_channel(const rspi_channel_t channel, const
bool enable)
```

Set MSTPCRB module stop bit for an RSPI channel.

Controls the Module Stop Control Register B (MSTPCRB) bits that gate the clock to each RSPI channel. Clearing a bit enables the module (starts the clock); setting it stops the module (reduces power consumption). The bit positions are defined by k_rspi_mstpb_rspi0/1/2 constants.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (valid: 0, 1, or 2)
in	enable	true to enable module clock (clear MSTPB bit), false to stop module clock (set MSTPB bit)

Pre: system_regs() must return a valid pointer

Pre: channel must be one of k_rspi_channel_0, k_rspi_channel_1, k_rspi_channel_2

Post: MSTPCRB register updated with the appropriate bit set/cleared

Post: Module clock enabled or disabled for the specified channel

Note: Not thread-safe. Caller must hold MSTPCR protection if applicable.

Warning: Disabling a module while it is actively transferring data will cause undefined hardware behavior.] **See also:** internal_get_rspi_base() Maps channel to register base

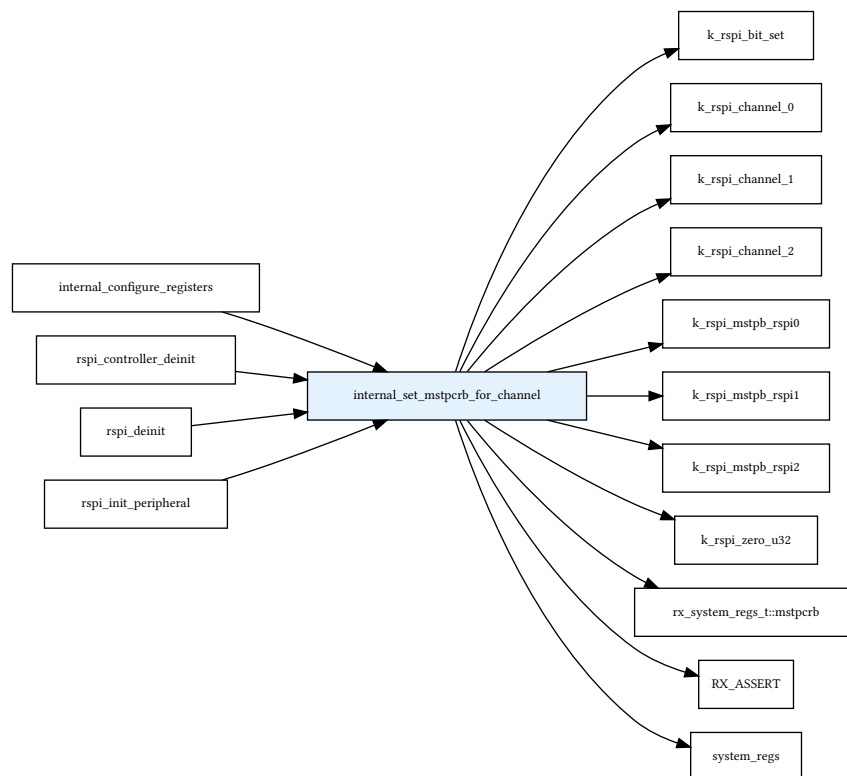


Figure 755: Call graph for internal_set_mstpcrb_for_channel

_Defined in libs/rx_hal/src/rspi.c:438_

Since Version 1.0.0

—

rspi_init_peripheral

```
rx_err_t rspi_init_peripheral(const rspi_channel_t channel, const rspi_config_t
*config)
```

Initialize RSPI channel in peripheral mode.

Initialize RSPI channel in peripheral mode for RPi5 communication.

Configures the specified RSPI channel as a SPI peripheral to communicate with the Raspberry Pi 5 (acting as SPI Controller). Enables the peripheral in the configured SPI mode (0-3) with optional 8-bit or 16-bit data frames. Uses register protection unlock/lock for module stop control.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0-2, corresponding to RSPI0/RSPI1/RSPI2)
in	config	Pointer to rspi_config_t configuration structure config must contain: spi_mode: SPI mode (0-3) controlling CPOL and CPHA bits use_16bit: True for 16-bit data frames, false for 8-bit

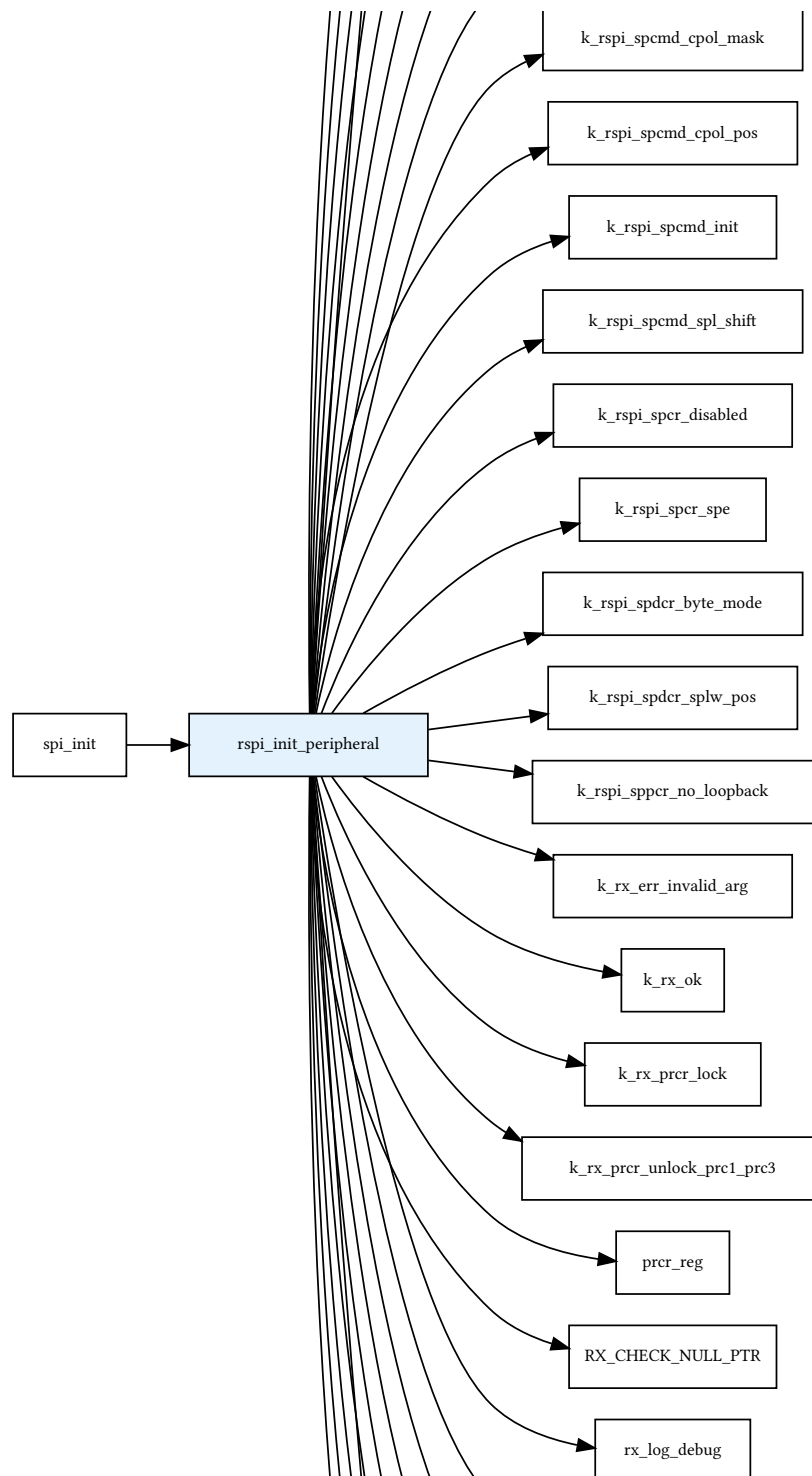
Returns: k_rx_err_invalid_arg if: config pointer is nullptr channel is out of range (≥ 3) config->spi_mode exceeds maximum (> 3) RSPI base address lookup fails for the channel

Pre: config pointer must be non-NULL

Pre: channel must be in range 0, k_rspi_max_channels)

Pre: config->spi_mode must be in range 0, 3

Post: If successful:] RSPI module is enabled (module stop bit cleared via register protection) SPI mode (CPOL/CPHA) is configured per config->spi_mode Data frame length (8-bit or 16-bit) is set per config->use_16bit SPI is enabled in peripheral mode (SPCR.MSTR = 0)
s_rspi_channel_initialized[channel] is set to true Peripheral is ready to receive transfers from SPI controller

Figure 756: Call graph for `rspi_init_peripheral`

_Defined in libs/rx_hal/src/rspi.c:500_

—

internal_wait_tx_ready

```
static rx_err_t internal_wait_tx_ready(volatile rx_rspi_regs_t *rspi)
```

Wait for transmit buffer to become empty (SPTEF flag).

Polls the SPSR.SPTEF (SPI Transmit Buffer Empty Flag) in a busy-wait loop with a configurable timeout. When SPTEF is set, the transmit buffer is empty and ready to accept new data via SPDR. The timeout prevents infinite loops if the SPI peripheral is stuck.

Parameters:

Direction	Name	Description
in	rspi	Pointer to RSPI register block (must be valid and non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Transmit buffer became empty within timeout
k_rx_err_timeout	Timeout expired before SPTEF was set

Pre: rspi must be a valid non-null pointer to initialized RSPI registers

Pre: RSPI module clock must be enabled (MSTPCRB bit cleared)

Post: No hardware state modified (read-only polling of SPSR)

Post: Timeout counter decremented to reflect remaining iterations

Note: Not thread-safe. Caller must provide synchronization.

Warning: Busy-wait loop; do not call from time-critical ISR context.] **See also:** internal_wait_rx_ready() Companion function for receive buffer, k_rspi_timeout_us Timeout constant

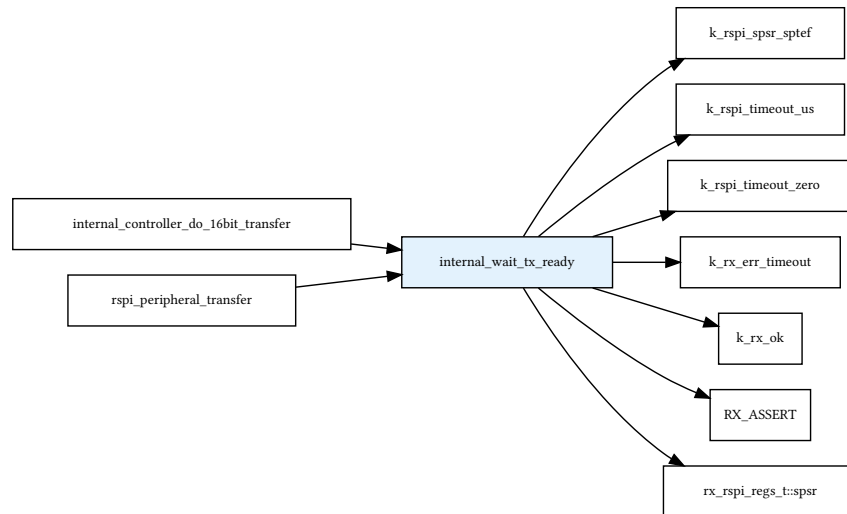


Figure 757: Call graph for internal_wait_tx_ready

_Defined in libs/rx_hal/src/rspi.c:612_

Since Version 1.0.0

—

internal_wait_rx_ready

```
static rx_err_t internal_wait_rx_ready(volatile rx_rspi_regs_t *rspi)
```

Wait for receive buffer to become full (SPRF flag).

Polls the SPSR.SPRF (SPI Receive Buffer Full Flag) in a busy-wait loop with a configurable timeout. When SPRF is set, received data is available in the SPDR register and can be read. The timeout prevents infinite loops if the SPI peripheral is stuck or no clock is being generated.

Parameters:

Direction	Name	Description
in	rspi	Pointer to RSPI register block (must be valid and non-null)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Receive buffer became full within timeout
k_rx_err_timeout	Timeout expired before SPRF was set

Pre: rspi must be a valid non-null pointer to initialized RSPI registers

Pre: A transfer must be in progress or complete for SPRF to set

Post: No hardware state modified (read-only polling of SPSR)

Post: Timeout counter decremented to reflect remaining iterations

Note: Not thread-safe. Caller must provide synchronization.

Warning: Busy-wait loop; do not call from time-critical ISR context.] **See also:** `internal_wait_tx_ready()` Companion function for transmit buffer, `k_rspi_timeout_us` Timeout constant

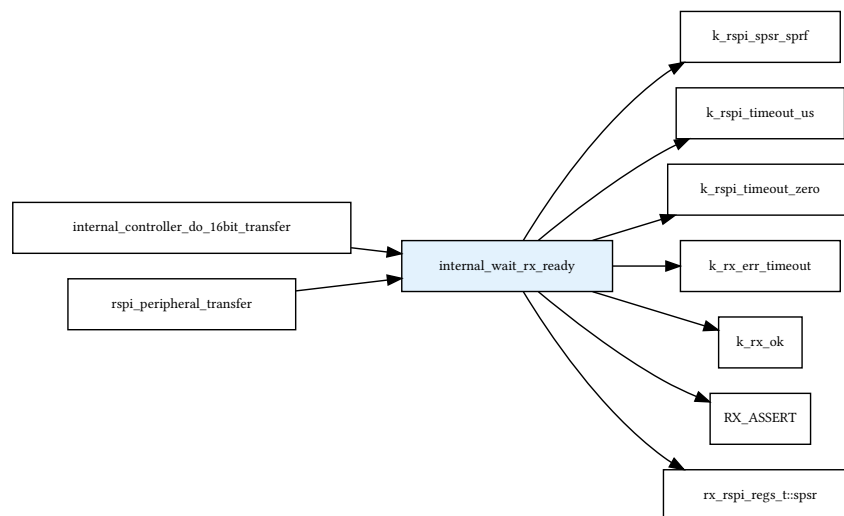


Figure 758: Call graph for `internal_wait_rx_ready`

_Defined in `libs/rx_hal/src/rspi.c:661_`

Since Version 1.0.0

—

`rspi_peripheral_transfer`

```

rx_err_t rspi_peripheral_transfer(const rspi_channel_t channel, const uint8_t
*tx_data, uint8_t *rx_data, const uint16_t length)

```

Perform full-duplex SPI transfer in peripheral mode.

Full-duplex SPI transfer in peripheral mode.

Executes a full-duplex (simultaneous transmit and receive) SPI transfer on the specified RSPI channel operating in peripheral mode. Transfers data byte-by-byte with timeout protection on both transmit buffer ready and receive buffer full operations.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0-2)
in	tx_data	Pointer to transmit data buffer (not nullptr)
out	rx_data	Pointer to receive data buffer (not nullptr)
in	length	Number of bytes to transfer (1 to 65535)

Returns: k_rx_err_timeout if: Transmit buffer does not become ready within timeout Receive buffer does not fill within timeout

Pre: channel must be initialized via `rspi_init_peripheral()` before calling

Pre: tx_data pointer must be non-NULL

Pre: rx_data pointer must be non-NULL

Pre: length must be in range 1, 65535

Post: If successful, rx_data buffer is filled with received bytes corresponding to transmitted data at each byte offset

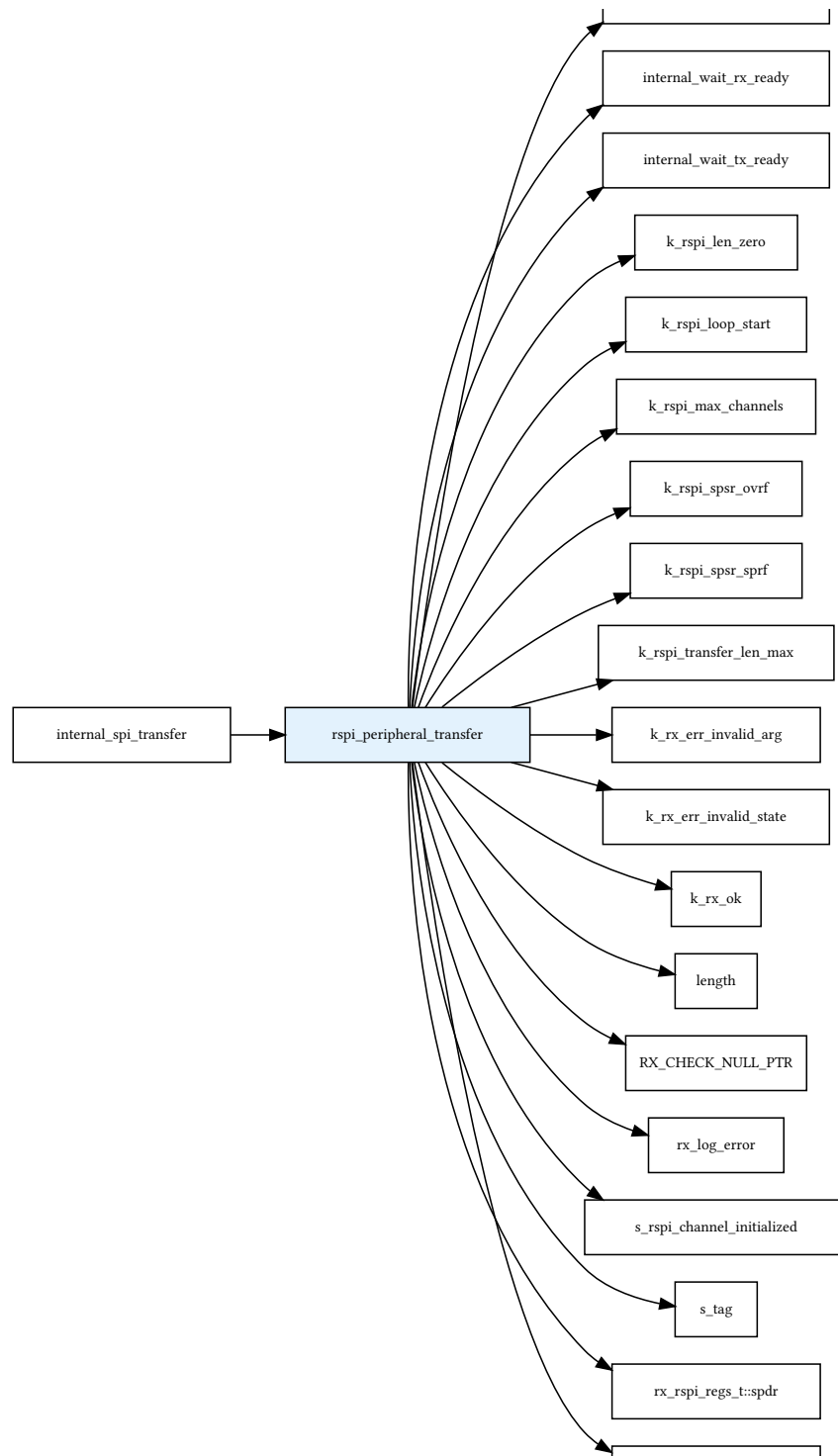


Figure 759: Call graph for `rspi_peripheral_transfer`

_Defined in libs/rx_hal/src/rspi.c:715_

—

rspi_peripheral_read_available

```
rx_err_t rspi_peripheral_read_available(const rspi_channel_t channel, bool
*available)
```

Check if data is available in the RSPI receive buffer.

Check if receive data is available in the RSPI receive buffer.

Polls the RSPI status register to determine if the receive buffer contains data ready for reading.

Returns the status without blocking.

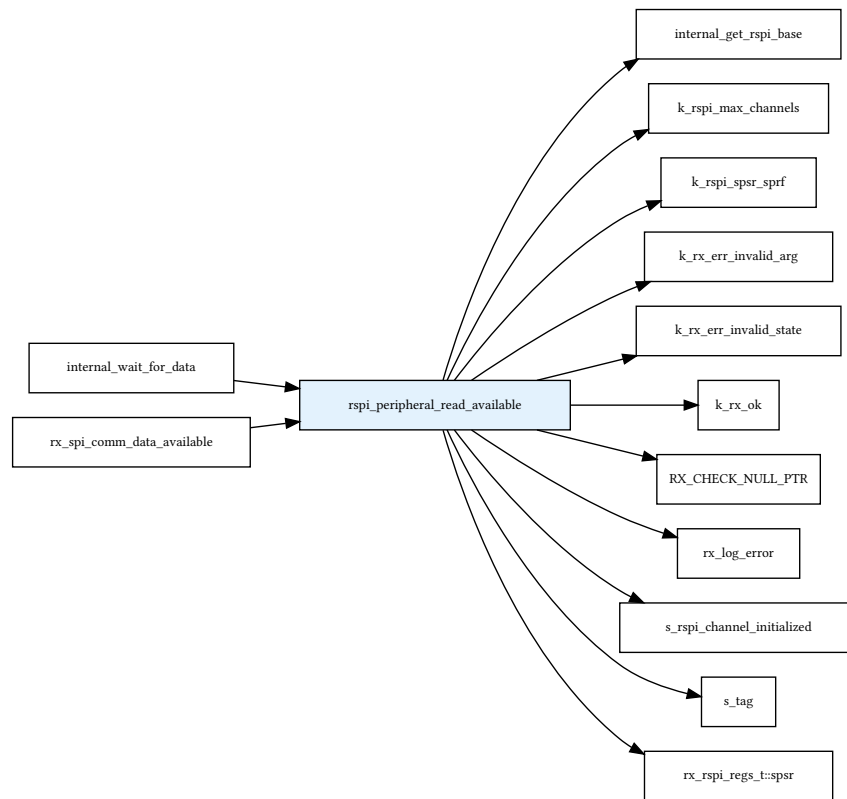
Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0, 1, or 2)
out	available	Pointer to bool flag set to true if data is available, false otherwise

Returns: k_rx_err_invalid_arg if RSPI base address retrieval fails

Pre: Channel must be initialized via rspi_init_peripheral() before calling this function

Pre: available must be a valid non-nullptr

Figure 760: Call graph for `rspi_peripheral_read_available`_Defined in `libs/rx\_hal/src/rspi.c:791_`**rspi_peripheral_write_ready**

```
rx_err_t rspi_peripheral_write_ready(const rspi_channel_t channel, bool *ready)
```

Check if transmit buffer is ready for data.

Check if the RSPI transmit buffer is ready for new data.

Polls the RSPI status register to determine if the transmit buffer is empty and ready to accept new data. Returns the status without blocking.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0-2)
out	ready	Set to true if ready to transmit, false otherwise

Returns: `k_rx_err_invalid_arg` if RSPI base address retrieval fails

Pre: Channel must be initialized via `rspi_init_peripheral()`

Pre: ready must be a valid non-nullptr

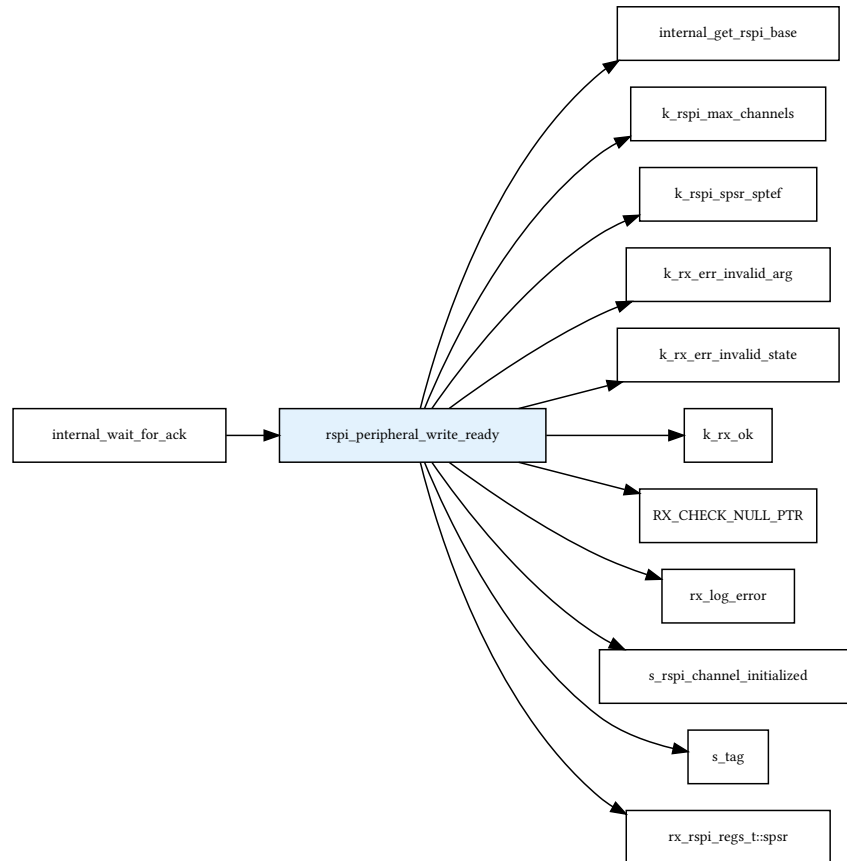


Figure 761: Call graph for `rspi_peripheral_write_ready`

_Defined in `libs/rx\hal/src/rspi.c:831_`

rspi_deinit

```
rx_err_t rspi_deinit(const rspi_channel_t channel)
```

Deinitialize and disable RSPI peripheral.

Deinitialize and disable an RSPI peripheral-mode channel.

Disables the specified RSPI channel, clears the initialization flag, and optionally disables the module (via module stop bit) to reduce power consumption.

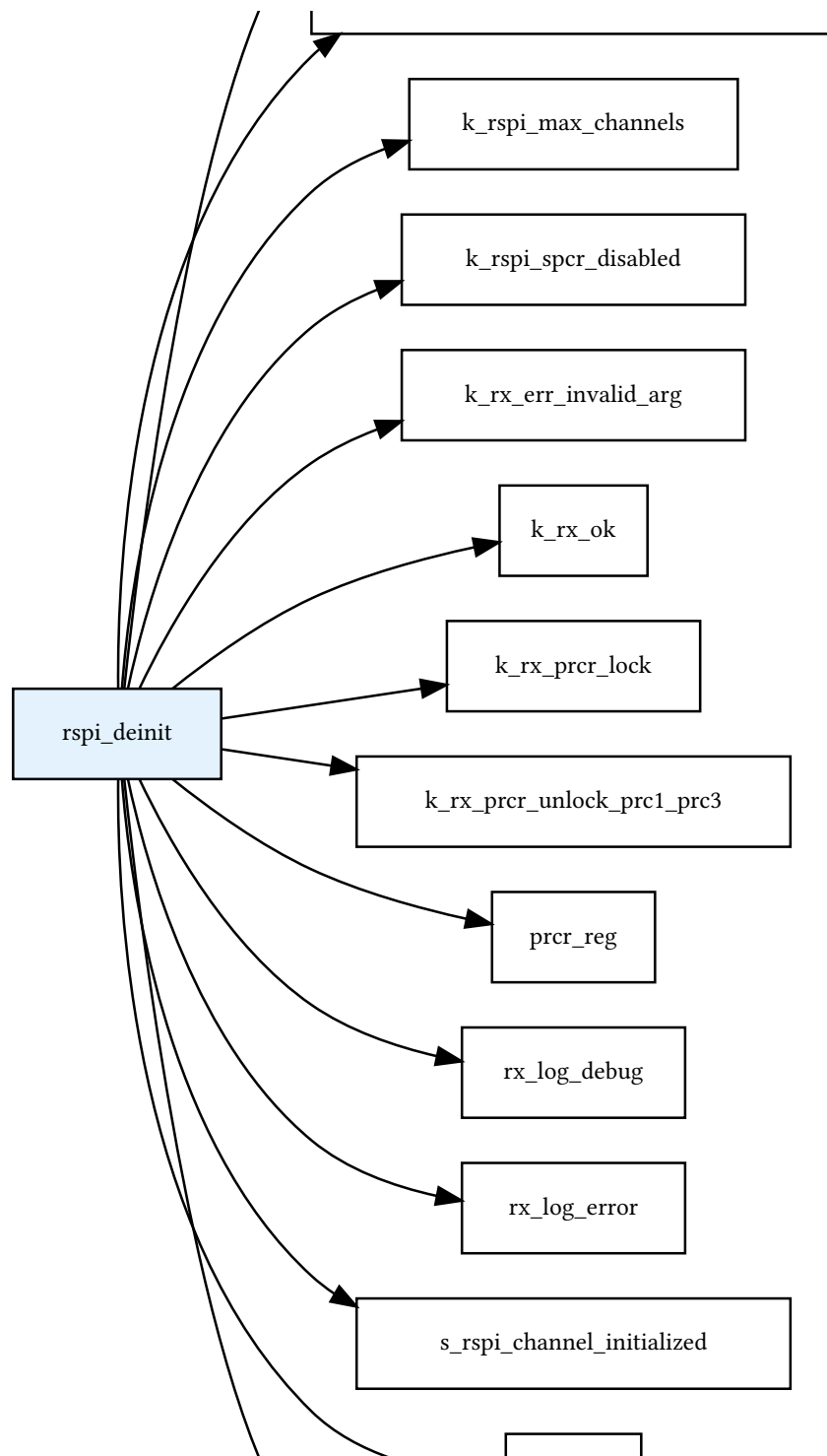
Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0, 1, or 2)

Returns: `k_rx_err_invalid_arg` if: channel is out of range (≥ 3) RSPI base address lookup fails for the channel

Pre: Channel should have been initialized via `rspi_init_peripheral()` previously

Post: If successful:] RSPI peripheral is disabled (`SPCR.SPE = 0`)
`s_rspi_channel_initialized[channel]` is set to false Channel is no longer available for SPI operations

Figure 762: Call graph for `rspi_deinit`

_Defined in libs/rx_hal/src/rspi.c:874_

—

internal_timing_delay

```
static void internal_timing_delay(const uint16_t cycles)
```

Cycle-accurate timing delay using inline NOP instructions.

Produces precise delays using inline assembly NOP operations for SPI timing requirements (CS setup/hold times). Each NOP cycle is 4.17 ns at 240 MHz core clock.

Parameters:

Direction	Name	Description
in	cycles	Number of NOP cycles to execute (must be > 0, <= k_rspi_max_delay_cycles)

Pre: cycles must be non-zero

Pre: cycles must not exceed k_rspi_max_delay_cycles

Post: Minimum delay of (cycles * 4.17 ns) has elapsed

Post: No hardware state modified

Note: Timing is cycle-accurate on RX72N @ 240 MHz (4.17 ns per cycle)

Note: Using inline ASM ensures compiler does not optimize away the loop

Warning: Only accurate when compiled with known optimization level (-O2)] **See also:**
 internal_controller_assert_cs_with_setup() Uses for CS setup time,
 internal_controller_deassert_cs_with_hold() Uses for CS hold time

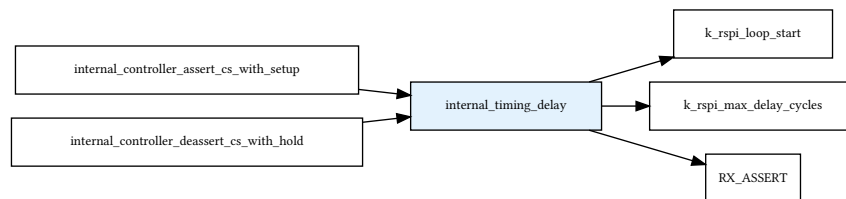


Figure 763: Call graph for internal_timing_delay

_Defined in libs/rx_hal/src/rspi.c:946_

Since Version 1.0.0

—

internal_configure_spcmd

```
static void internal_configure_spcmd(uint16_t *spcmd, const uint8_t spi_mode)
```

Configure SPCMD register for SPI mode (CPOL/CPHA).

Sets CPOL and CPHA bits in an SPCMD register value based on SPI mode (0-3). The SPI mode encodes clock polarity (CPOL) and clock phase (CPHA) per the standard SPI specification. Bits are set via bitwise OR so pre-existing fields in the SPCMD value (such as data length) are preserved.

Parameters:

Direction	Name	Description
inout	spcmd	Pointer to SPCMD register value to modify
in	spi_mode	SPI mode (0-3)

Returns: void

Pre: spcmd must be a valid non-null pointer

Pre: spi_mode must be in range 0, 3 (only bits 0-1 used)

Post: spcmd CPOL/CPHA bits set according to spi_mode

Post: Other spcmd bits remain unchanged

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
rspi_init_controller() Calls this during controller init, rspi_init_peripheral()
Performs equivalent inline configuration

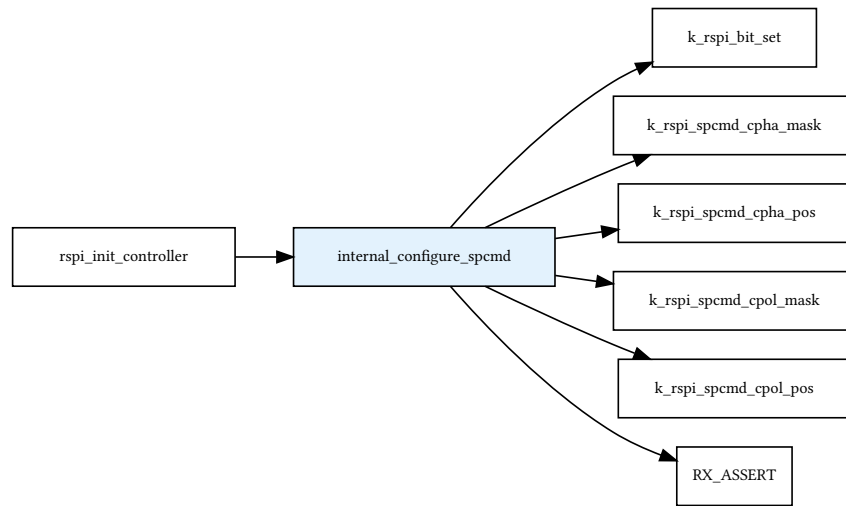


Figure 764: Call graph for internal_configure_spcmd

Defined in libs/rx\hal/src/rspi.c:998

Since Version 1.0.0

—

internal_calculate_spbr

```
static rx_err_t internal_calculate_spbr(const uint32_t freq_hz, uint8_t *spbr)
```

Calculate SPBR value for desired SPI clock frequency.

Computes the RSPI Bit Rate Register (SPBR) value required to achieve the desired SPI clock frequency. Uses the formula: $SPBR = (PCLKB / (2 * freq_hz)) - 1$ with BRDV=0 (no additional divisor). The result is validated against the hardware-supported range [0, 255].

Parameters:

Direction	Name	Description
in	freq_hz	Desired SPI clock frequency in Hz (valid range: k_rspi_min_freq_hz to k_rspi_max_freq_hz)
out	spbr	Pointer to store calculated SPBR value (0-255)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	SPBR calculated and stored successfully
k_rx_err_null_ptr	spbr pointer is nullptr
k_rx_err_invalid_arg	freq_hz outside supported range or SPBR > 255

Pre: spbr must be a valid non-null pointer

Pre: freq_hz must be in range 100000, 10000000 Hz

Post: *spbr contains the calculated SPBR value if k_rx_ok returned

Post: *spbr is unchanged on error

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
 rspi_init_controller() Calls this to configure clock frequency, k_rspi_pclkb_hz
 PCLKB clock frequency used in calculation

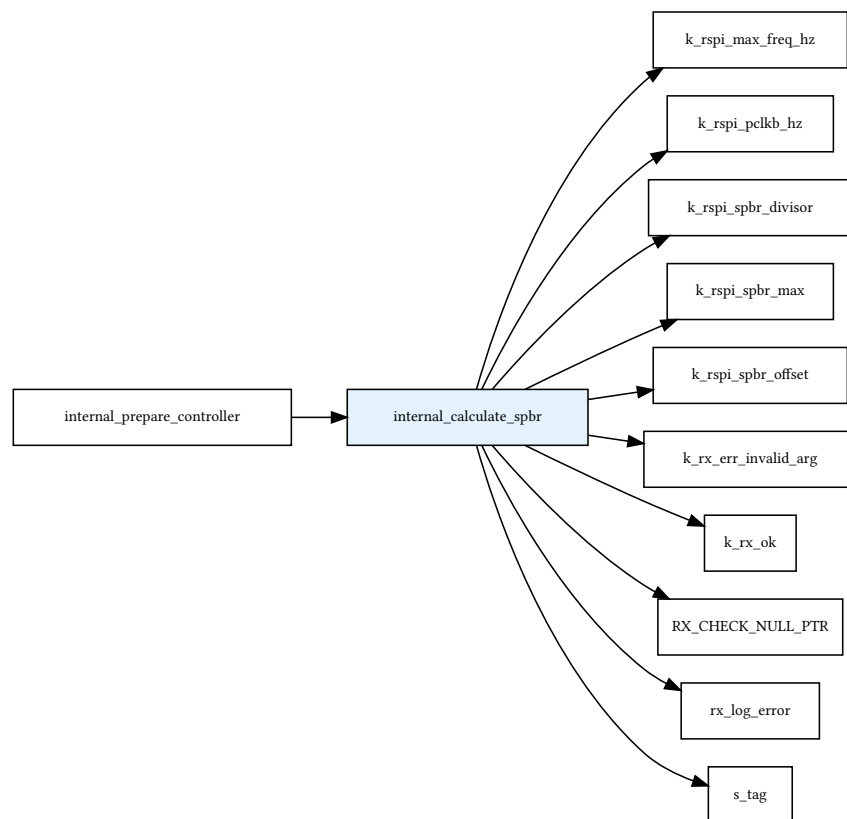


Figure 765: Call graph for internal_calculate_spbr

Defined in libs/rx\hal/src/rspi.c:1045

Since Version 1.0.0

—

internal_configure_cs_gpio

```
static rx_err_t internal_configure_cs_gpio(const rx_port_pin_t pin_config)
```

Configure GPIO pin for chip select output.

Configures a GPIO pin as a digital output for use as an active-low SPI chip select signal. Extracts port and pin numbers from the type-safe `rx_port_pin_t` encoding, sets the pin direction to output via PDR, and drives the pin high (CS inactive) via PODR.

Parameters:

Direction	Name	Description
in	pin_config	GPIO pin (<code>rx_port_pin_t</code> encoding port and pin)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	GPIO configured as CS output and driven high (inactive)
<code>k_rx_err_invalid_arg</code>	Port base lookup failed or pin out of range

Pre: `pin_config` must encode a valid port (0-J) and pin (0-7)

Pre: Port hardware must be available and not module-stopped

Post: GPIO PDR bit set (output direction)

Post: GPIO PODR bit set (CS inactive / high)

Note: Not thread-safe. Caller must provide synchronization for port access.] **See also:**
`rspi_init_controller()` Calls this during controller initialization,
`rx_port_from_pin()` Extracts port number from `rx_port_pin_t`, `rx_pin_from_pin()`
Extracts pin number from `rx_port_pin_t`

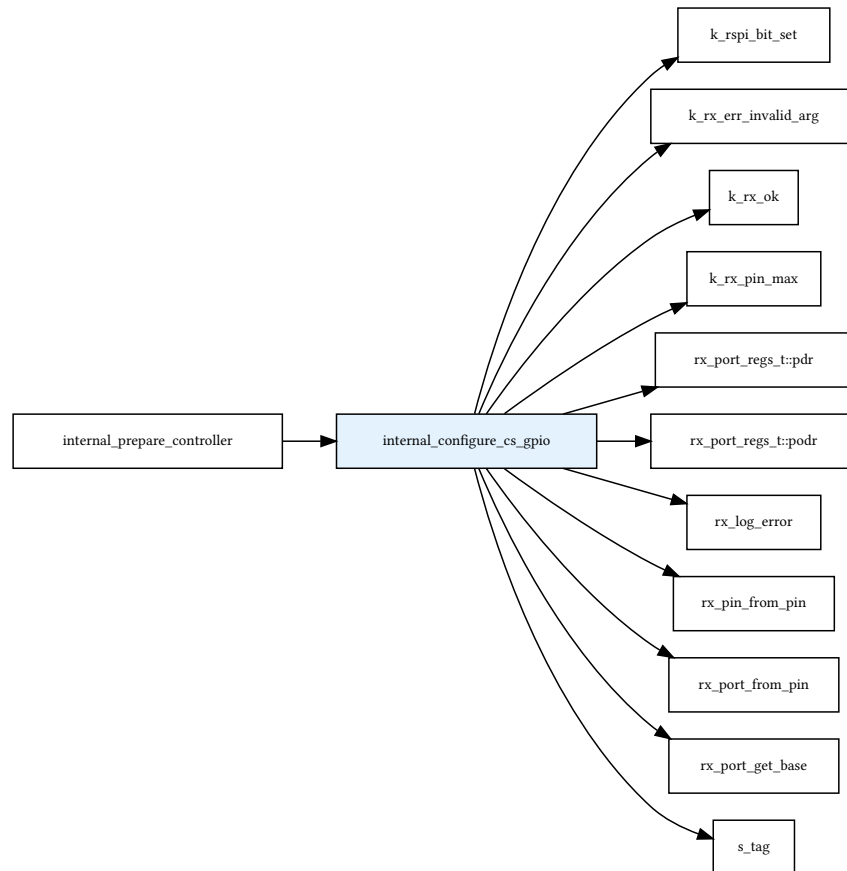


Figure 766: Call graph for internal_configure_cs_gpio

_Defined in libs/rx_hal/src/rspi.c:1099_

Since Version 1.0.0

—

internal_validate_controller_args

```
static rx_err_t internal_validate_controller_args(const rspi_channel_t channel,
const rspi_controller_config_t *config)
```

Validate controller initialization arguments.

Performs comprehensive input validation for rspi_init_controller() before any hardware modification occurs. Checks null pointer, channel range, duplicate initialization, and SPI mode range. Centralizes validation logic to keep the public API function concise.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	channel	RSPI channel number (valid: 0 to k_rspi_max_channels - 1)
in	config	Pointer to controller configuration structure

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All validations passed
k_rx_err_null_ptr	config pointer is nullptr
k_rx_err_invalid_arg	Channel out of range or SPI mode > 3
k_rx_err_invalid_state	Channel already initialized in controller mode

Pre: config may be nullptr (will be detected and rejected)

Pre: channel may be out of range (will be detected and rejected)

Post: No hardware state modified (validation only)

Post: s_rspi_controller_initialized not modified

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
rspi_init_controller() Calls this as first step of initialization,
internal_prepare_controller() Called after this validation passes

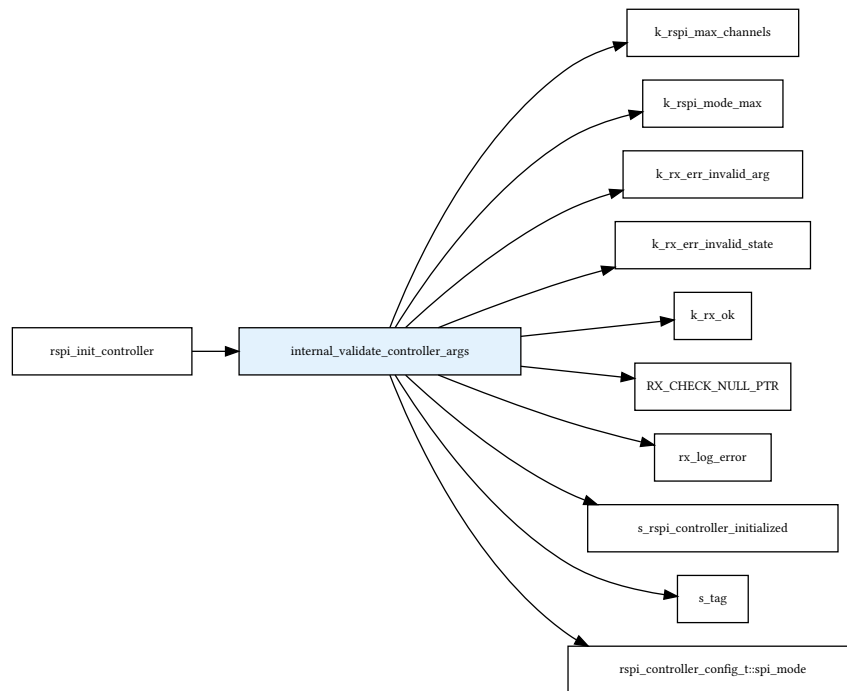


Figure 767: Call graph for internal_validate_controller_args

Defined in libs/rx\hal/src/rspi.c:1157

Since Version 1.0.0

—

internal_prepare_controller

```
static rx_err_t internal_prepare_controller(const rspi_channel_t channel, const
rspi_controller_config_t *config, volatile rx_rspi_regs_t **out_rspi, uint8_t
*out_spbr)
```

Prepare controller hardware resources.

Calculates the SPBR register value for the requested clock frequency, validates the RSPI register base address for the given channel, and configures the CS GPIO pin as an active-low output. Operations are ordered so that a failure in SPBR calculation avoids unnecessary GPIO configuration, and RSPI base validation occurs before GPIO setup to avoid leaving GPIO configured if the channel is invalid.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0 to k_rspi_max_channels - 1)
in	config	Pointer to controller configuration (freq_hz, cs pin)
out	out_rspi	Pointer to receive RSPI register base address

out	out_spbr	Pointer to receive calculated SPBR value
-----	----------	--

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Hardware resources prepared successfully
k_rx_err_null_ptr	out_spbr is nullptr (via internal_calculate_spbr)
k_rx_err_invalid_arg	Frequency out of range, invalid channel, or bad CS pin

Pre: config must be validated by internal_validate_controller_args() first

Pre: channel must be in range 0, k_rspi_max_channels)

Post: *out_rspi points to valid RSPI register base on success

Post: *out_spbr contains calculated SPBR value on success

Post: CS GPIO pin configured as output, driven high (inactive) on success

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
 internal_validate_controller_args() Must be called before this function,
 internal_configure_registers() Called after this to program hardware,
 internal_calculate_spbr() Called internally for frequency calculation

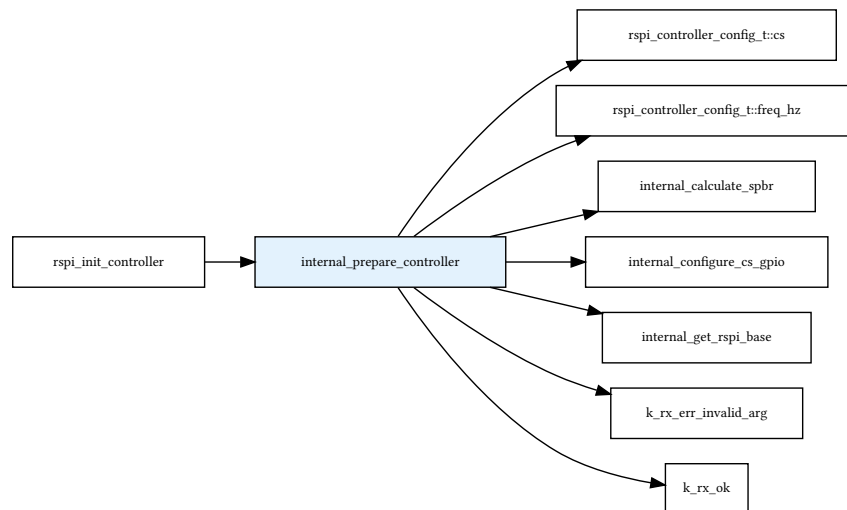


Figure 768: Call graph for internal_prepare_controller

_Defined in libs/rx/_hal/src/rspi.c:1218_

Since Version 1.0.0

—

internal_configure_registers

```
static rx_err_t internal_configure_registers(const rspi_channel_t channel,
volatile rx_rspi_regs_t *rspi, uint8_t spbr, uint16_t spcmd, const
rspi_controller_config_t *config)
```

Configure RSPI hardware registers for controller mode.

Performs the complete RSPI hardware register configuration sequence for controller mode. Enables the module clock, disables SPI for safe register modification, programs bit rate, command, data control and pin control registers, then enables SPI in controller mode. Also stores the CS pin configuration and marks the channel as initialized. The register write order follows the RX72N Hardware Manual Section 38.3.6 initialization procedure.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0 to k_rspi_max_channels - 1)
in	rspi	Pointer to RSPI register base (must be valid, non-null)
in	spbr	Calculated SPBR value for clock frequency (0-255)
in	spcmd	Pre-configured SPCMD register value (CPOL/CPHA/data length)
in	config	Pointer to controller configuration (used for CS pin info)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Registers configured and channel marked as initialized

Pre: rspi must be a valid non-null pointer from internal_get_rspi_base()

Pre: spbr must be pre-calculated by internal_calculate_spbr()

Pre: spcmd must be pre-configured with CPOL/CPHA and data length bits

Post: RSPI module clock enabled, registers configured, SPI enabled

Post: s_rspi_cs_configchannel stores CS port and pin

Post: s_rspi_controller_initializedchannel set to true

Note: Not thread-safe. Caller must provide synchronization.

Warning: PRCR register protection is temporarily unlocked during this call.] **See also:**
`internal_prepare_controller()` Provides `rspi`, `spbr`, and validates channel,
`rspi_init_controller()` Public API that orchestrates the full init flow

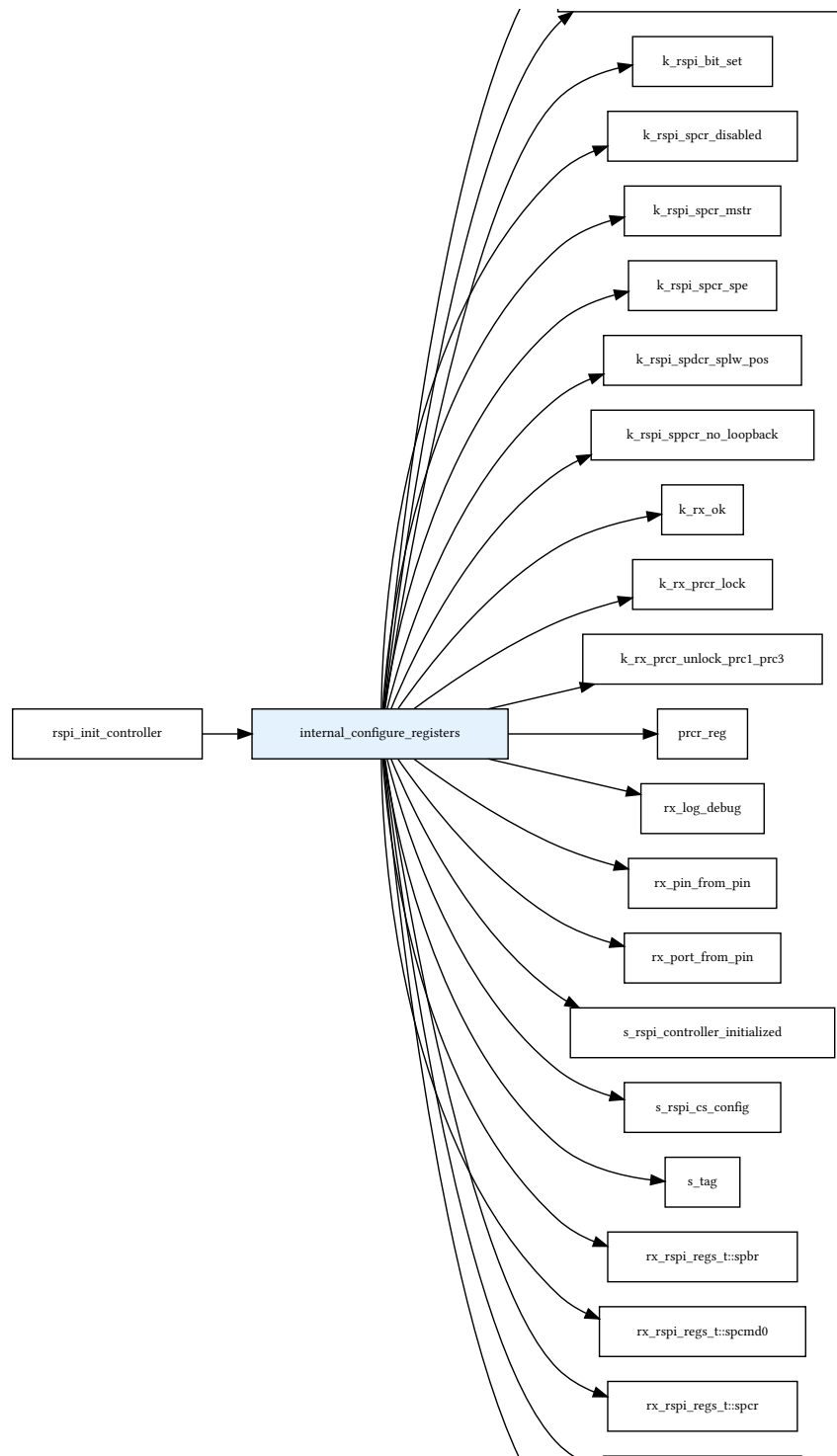


Figure 769: Call graph for `internal_configure_registers`

_Defined in libs/rx_hal/src/rspi.c:1281_

Since Version 1.0.0

—

rspi_init_controller

```
rx_err_t rspi_init_controller(const rspi_channel_t channel, const
rspi_controller_config_t *config)
```

Initialize RSPI channel in controller mode.

Initialize RSPI channel in controller mode for external device communication.

Configures the specified RSPI channel as a SPI controller to communicate with SPI peripheral devices. Enables the module, configures clock frequency, SPI mode, and chip select GPIO pin.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0-2)
in	config	Pointer to controller configuration structure containing: freq_hz: SPI clock frequency (100 kHz to 10 MHz) spi_mode: SPI mode (0-3) for CPOL/CPHA configuration cs: GPIO pin for chip select (type-safe rx_port_pin_t)

Returns: k_rx_err_invalid_state if channel is already initialized

Pre: config must be non-NULL

Pre: channel < k_rspi_max_channels

Pre: channel must not be already initialized

Pre: CS GPIO port and pin must be valid and available

Post: If successful:] RSPI module is enabled via internal_set_mstpcrb_for_channel() *prcr_reg() is unlocked/locked for register protection RSPI registers configured: spbr, spcmd0, spcr, spdcr, sppcr CS GPIO configured as output, driven high (inactive) s_rspi_cs_config[channel] stores CS port/pin s_rspi_controller_initialized[channel] = true

Post: On error:] Hardware state is unchanged Channel remains uninitialized

Note: Bit rate calculation: $\text{freq} = \text{PCLKB} / (2 * (\text{SPBR} + 1))$

Note: Always configures 16-bit data frames with word access mode

Note: CS is active-low (asserted=0, deasserted=1)

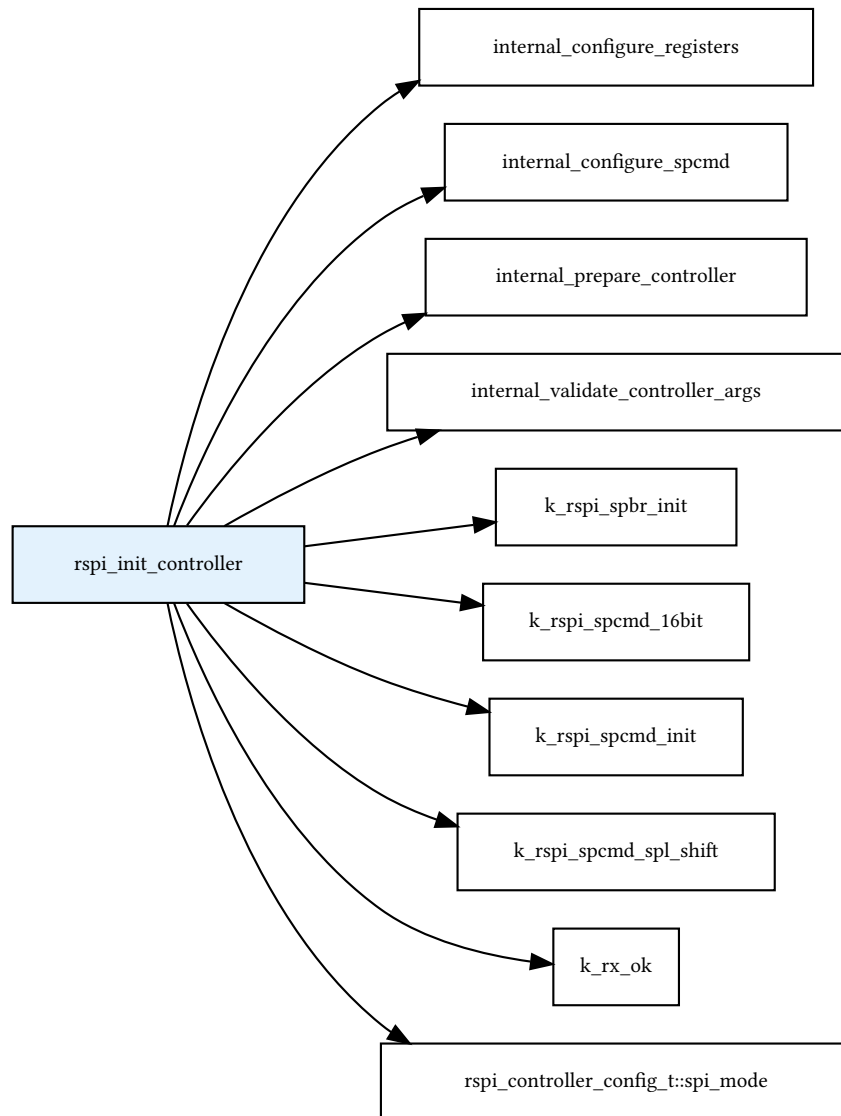


Figure 770: Call graph for `rspi_init_controller`

_Defined in `libs/rx_hal/src/rspi.c:1380_`

`rspi_controller_set_cs`

```
rx_err_t rspi_controller_set_cs(const rspi_channel_t channel, const bool active)
```

Set chip select line state for RSPI controller channel.

Set chip select line state for an RSPI controller-mode channel.

Controls the GPIO chip select pin for the specified RSPI channel operating in controller mode. CS is active-low (asserted low, deasserted high).

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0-2)
in	active	True to assert CS (drive low), false to deassert CS (drive high)

Returns: k_rx_err_invalid_arg if GPIO port base lookup fails

Pre: Channel must be initialized via rspi_init_controller()

Pre: channel < k_rspi_max_channels

Pre: s_rspi_controller_initializedchannel == true

Post: GPIO port_regs->podr is modified to assert/deassert CS pin

Note: Reads s_rspi_cs_configchannel for port and pin configuration

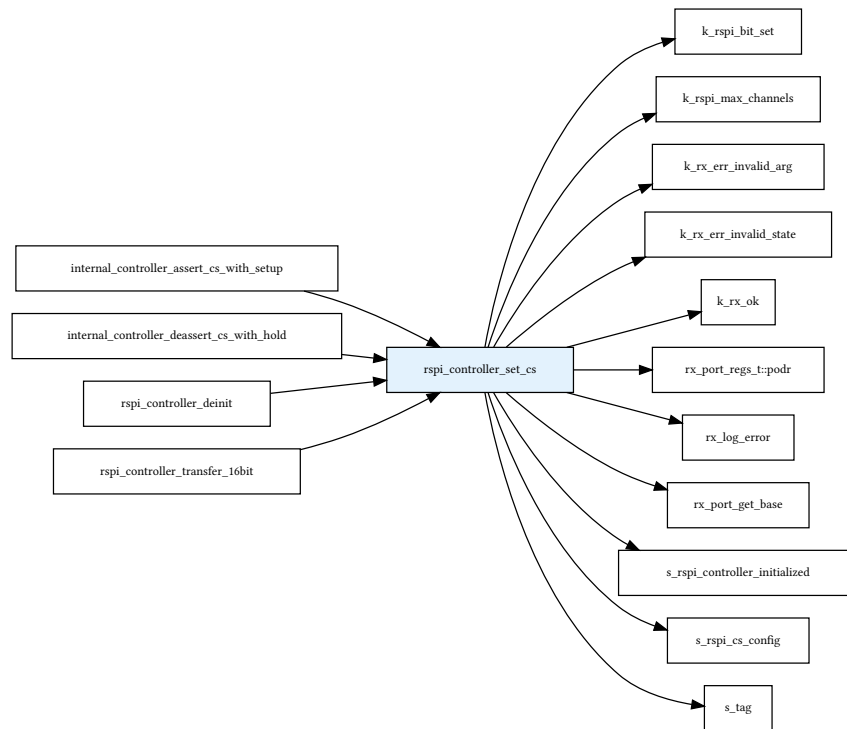


Figure 771: Call graph for rspi_controller_set_cs

_Defined in libs/rx_hal/src/rspi.c:1428_

internal_controller_assert_cs_with_setup

```
static rx_err_t internal_controller_assert_cs_with_setup(const rspi_channel_t
channel)
```

Assert CS with setup delay.

Asserts the chip select line (drives low for active-low CS) and then applies a cycle-accurate setup time delay using NOP instructions. The setup delay ensures the SPI peripheral has sufficient time to recognize CS assertion before clock transitions begin. Delay is calibrated for 300ns at 240 MHz core clock.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0 to k_rspi_max_channels - 1)

Returns: rx_err_t Error code

Value	Description
-------	-------------

k_rx_ok	CS asserted and setup delay completed
k_rx_err_invalid_state	Channel not initialized
k_rx_err_invalid_arg	GPIO port lookup failed

Pre: Channel must be initialized via `rspi_init_controller()`

Pre: `s_rspi_controller_initializedchannel` must be true

Post: CS GPIO pin driven low (active)

Post: Minimum setup delay of 300ns has elapsed

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
`internal_controller_deassert_cs_with_hold()` Companion deassert function,
`internal_timing_delay()` Provides cycle-accurate delay, `k_rspi_cs_setup_delay` Setup
delay iteration count

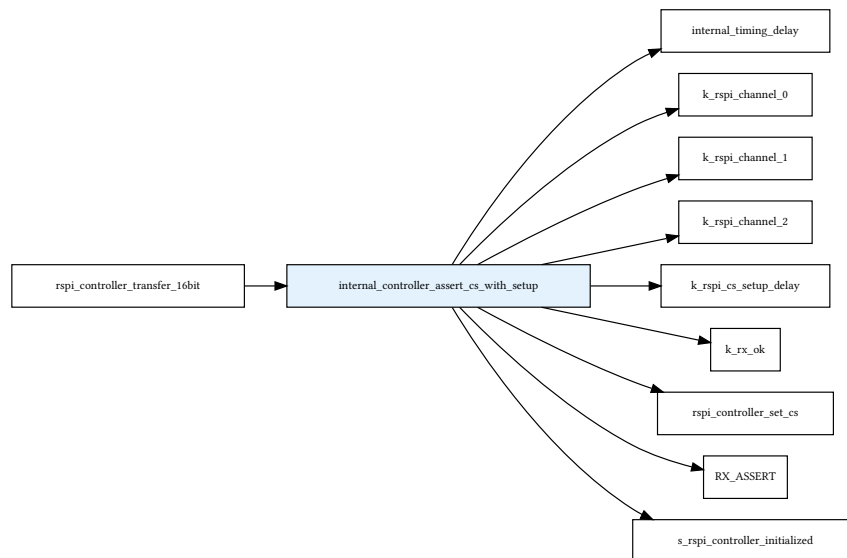


Figure 772: Call graph for `internal_controller_assert_cs_with_setup`

Defined in `libs/rx\hal/src/rspi.c:1488`

Since Version 1.0.0

internal_controller_deassert_cs_with_hold

```
static rx_err_t internal_controller_deassert_cs_with_hold(const rspi_channel_t
channel)
```

Deassert CS with hold delay.

Applies a cycle-accurate hold time delay using NOP instructions, then deasserts the chip select line (drives high for active-low CS). The hold delay ensures the SPI peripheral has sufficient time to latch the final data bit before CS is released. Delay is calibrated for 300ns at 240 MHz core clock.

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0 to k_rspi_max_channels - 1)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Hold delay completed and CS deasserted
k_rx_err_invalid_state	Channel not initialized
k_rx_err_invalid_arg	GPIO port lookup failed

Pre: Channel must be initialized via rspi_init_controller()

Pre: s_rspi_controller_initializedchannel must be true

Post: Minimum hold delay of 300ns has elapsed

Post: CS GPIO pin driven high (inactive)

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
internal_controller_assert_cs_with_setup() Companion assert function,
internal_timing_delay() Provides cycle-accurate delay, k_rspi_cs_hold_delay Hold
delay iteration count

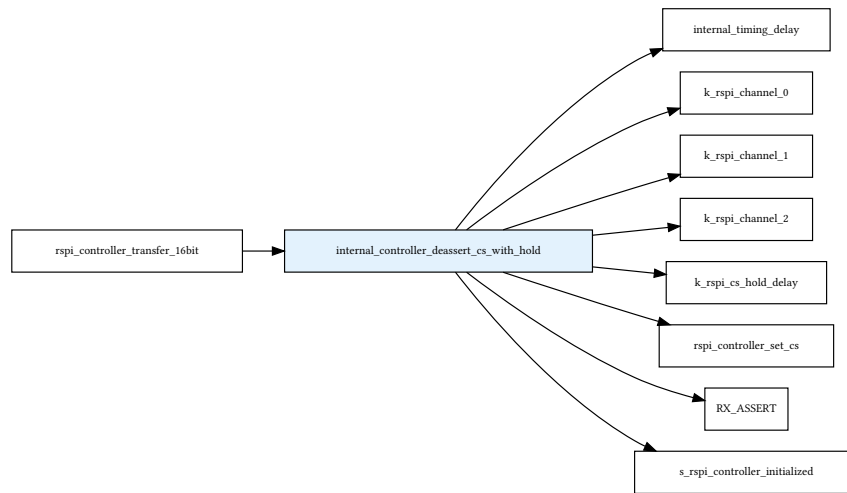


Figure 773: Call graph for internal_controller_deassert_cs_with_hold

_Defined in libs/rx_hal/src/rspi.c:1544_

Since Version 1.0.0

—

internal_controller_do_16bit_transfer

```
static rx_err_t internal_controller_do_16bit_transfer(volatile rx_rspi_regs_t
*rspi, uint16_t tx_data, uint16_t *rx_data)
```

Perform core 16-bit SPI transfer operation.

Executes a single full-duplex 16-bit SPI transfer without chip select management. Waits for the transmit buffer to become empty, writes 16-bit data to SPDR, waits for the receive buffer to fill, reads the 16-bit response from SPDR, and clears SPRF and OVRF status flags. This is the inner transfer primitive used by rspi_controller_transfer_16bit() after CS assertion and before CS deassertion.

Parameters:

Direction	Name	Description
in	rspi	Pointer to RSPI register base (must be valid, non-null)
in	tx_data	16-bit data word to transmit via SPDR
out	rx_data	Pointer to store received 16-bit response from SPDR

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Transfer completed successfully, rx_data contains response
k_rx_err_null_ptr	rspi or rx_data pointer is nullptr
k_rx_err_timeout	TX buffer did not empty or RX buffer did not fill

Pre: rspi must be a valid non-null pointer to initialized RSPI registers

Pre: SPI must be enabled (SPCR.SPE=1) before calling

Post: *rx_data contains the 16-bit value received during the transfer

Post: SPSR flags SPRF and OVRF are cleared

Note: Not thread-safe. Caller must provide synchronization.] **See also:**
 rspi_controller_transfer_16bit() Public API that wraps this with CS handling,
 internal_wait_tx_ready() TX buffer polling with timeout, internal_wait_rx_ready()
 RX buffer polling with timeout

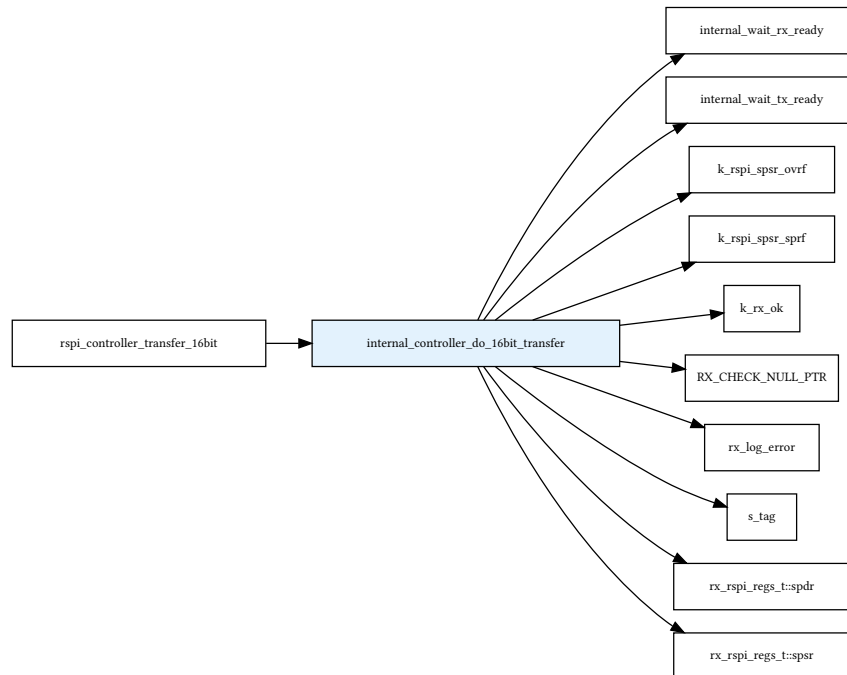


Figure 774: Call graph for internal_controller_do_16bit_transfer

Defined in libs/rx\hal/src/rspi.c:1598

Since Version 1.0.0

rspi_controller_transfer_16bit

```

rx_err_t rspi_controller_transfer_16bit(const rspi_channel_t channel, const
uint16_t tx_data, uint16_t *const rx_data)

```

Perform 16-bit SPI transfer in controller mode with CS handling.

Perform a 16-bit full-duplex SPI transfer in controller mode with automatic CS.

Executes a full-duplex 16-bit SPI transfer with automatic chip select assertion/deassertion and timing delays.

Sequence:

1. Assert CS (active low) with setup delay (300ns)
2. Wait for TX buffer ready, then transmit 16-bit data
3. Wait for RX buffer full, then read 16-bit response
4. Hold CS for minimum hold time (300ns)
5. Deassert CS (inactive high)
6. Clear status flags

Parameters:

Direction	Name	Description
in	channel	RSPI channel number (0-2)
in	tx_data	16-bit data to transmit
out	rx_data	Pointer to receive 16-bit response (must be non-NULL)

Returns: Other errors from `rspi_controller_set_cs()` on CS control failure

Pre: Channel must be initialized via `rspi_init_controller()`

Pre: `rx_data` must be a valid non-nullptr

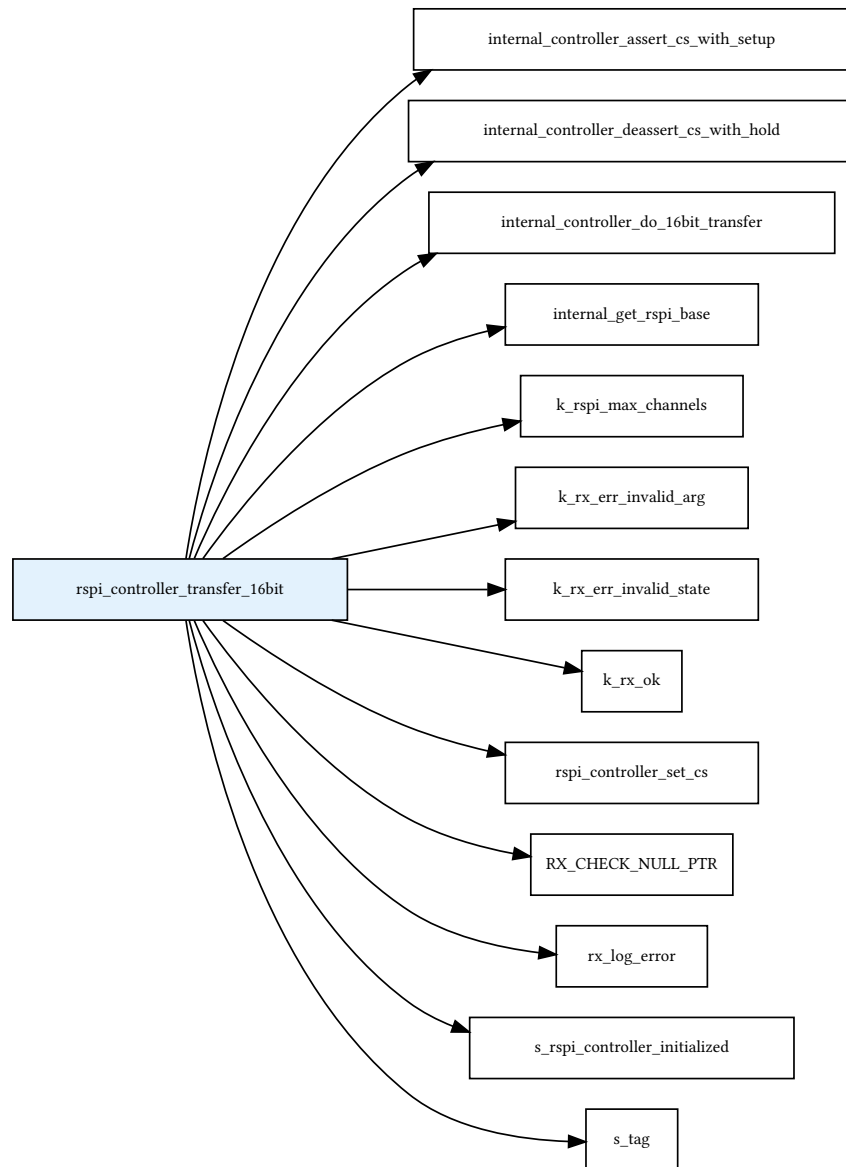
Post: If successful:] `rx_data` is filled with received 16-bit value Status flags (SPRF, OVRF) are cleared CS is deasserted (high)

Post: On error:] CS is deasserted if possible `rx_data` contents are undefined

Note: CS setup delay: `k_rspi_cs_setup_delay` NOP loops (300ns)

Note: CS hold delay: `k_rspi_cs_hold_delay` NOP loops (300ns)

Note: Uses inline assembly NOP for precise timing control

Figure 775: Call graph for `rspi_controller_transfer_16bit`_Defined in `libs/rx_hal/src/rspi.c:1677_`

—

`rspi_controller_deinit`

```
rx_err_t rspi_controller_deinit(const rspi_channel_t channel)
```

Deinitialize RSPI controller mode.

Deinitialize and disable an RSPI controller-mode channel.

Disables the RSPI controller, deasserts chip select, clears configuration, and stops the module to reduce power consumption.

Parameters:

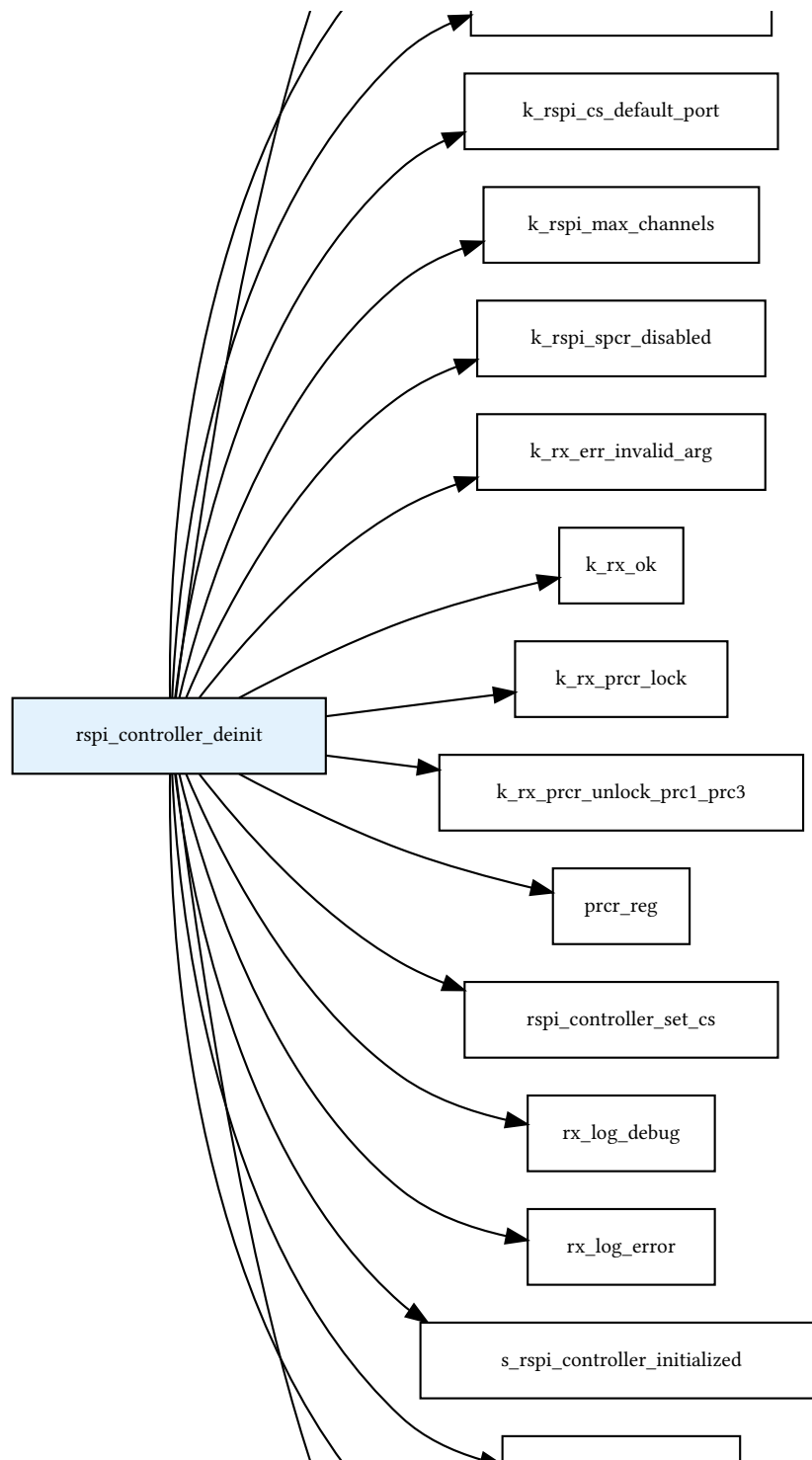
Direction	Name	Description
in	channel	RSPI channel number (0-2)

Returns: k_rx_err_invalid_arg if channel is out of range or base address lookup fails

Pre: Should be called after `rspi_init_controller()` to properly clean up

Post: If successful:] CS is deasserted via `rspi_controller_set_cs(channel, false)` RSPI is disabled (`rspi->spr = k_rspi_spr_disabled`) Module clock is stopped via `internal_set_mstpcrb_for_channel()` `s_rspi_cs_config[channel]` is cleared (`port=0, pin=0`) `s_rspi_controller_initialized[channel] = false` Register protection is unlocked/locked via `*prcr_reg()`

Note: Thread-safe only if caller ensures no concurrent access to the same channel

Figure 776: Call graph for `rspi_controller_deinit`

_Defined in libs/rx_hal/src/rspi.c:1736_

—

rx_cmt.c

CMT (Compare Match Timer) Driver Implementation - Periodic Interrupt Generation.

Includes:

- "rx_cmt.h"
- <stddef.h>
- "rx72n_regs.h"
- "rx_check.h"
- "rx_log.h"
- "rx_register_protection.h"

cmt_constants_t

CMT channel and divider configuration constants.

Defines the channel count, divider settings, and loop bounds for CMT configuration. These values correspond to RX72N hardware capabilities.

Value	Name	Description
= 4	k_cmt_max_channels	Total CMT channels: CMT0, CMT1, CMT2, CMT3
= 0	k_cmt_divider_start	Loop start index for divider iteration (Rule 2)
= 0	k_cmt_div_8	CKS=0b00: PCLKB/8 divider (7.5 MHz @ 60 MHz PCLKB)
= 1	k_cmt_div_32	CKS=0b01: PCLKB/32 divider (1.875 MHz @ 60 MHz PCLKB)
= 2	k_cmt_div_128	CKS=0b10: PCLKB/128 divider (468.75 kHz @ 60 MHz PCLKB)
= 3	k_cmt_div_512	CKS=0b11: PCLKB/512 divider (117.19 kHz @ 60 MHz PCLKB)
= 4	k_cmt_num_dividers	Loop bound for divider iteration (Rule 2 compliance)

—

cmt_divider_values_t

CMT divider values (actual divisor values).

Value	Name	Description
= 8	k_cmt_divider_val_8	Divide by 8
= 32	k_cmt_divider_val_32	Divide by 32
= 128	k_cmt_divider_val_128	Divide by 128
= 512	k_cmt_divider_val_512	Divide by 512

—

cmt_module_stop_bits_t

CMT module stop bit positions in MSTPCRB.

Value	Name	Description
= 15	k_cmt_mstpb_cmt	CMT0-CMT3 module stop bit

—

cmt_cmcr_bits_t

CMCR register bit positions.

Value	Name	Description
= 0	k_cmt_cmcr_cks_shift	CKS (clock select) bit shift
= 6	k_cmt_cmcr_cmie_pos	CMIE (interrupt enable) bit position

—

cmt_period_constants_t

Period calculation constants.

Value	Name	Description
= 0	k_cmt_period_min	Minimum valid period
= 0xFFFF	k_cmt_period_max	Maximum valid period (16-bit)
= 1	k_cmt_period_adj	Period adjustment (period - 1)

—

cmt_cmstr_bits_t

CMSTR register bit positions.

Value	Name	Description
= 0	k_cmt_cmstr_str0	CMT0/CMT2 start bit
= 1	k_cmt_cmstr_str1	CMT1/CMT3 start bit

—

cmt_bit_constants_t

CMT bit manipulation constants.

Value	Name	Description
= 1	k_cmt_bit_mask_lsb	Single bit set for shifts
= 0	k_cmt_value_zero	Zero value for comparisons

—

s_tag`const char* s_tag`

—

s_cmt_initialized`bool s_cmt_initialized[k_cmt_max_channels]`

Per-channel initialization status flags.

Note: Used to prevent operations on uninitialized channels.

Warning: Do not modify directly; managed by init/deinit functions.

Since Version 1.0.0

—

s_cmt_callback

`rx_cmt_callback_t s_cmt_callback[k_cmt_max_channels]`

Per-channel interrupt callback function pointers.

Note: NULL callbacks are valid - interrupt fires but no action taken.

Warning: Callback executes in interrupt context. Keep it fast!] **See also:** `rx_cmt_callback_t`
Callback function type

Since Version 1.0.0

—

s_cmt_user_data

`void* s_cmt_user_data[k_cmt_max_channels]`

Per-channel user data for callbacks.

Note: May be NULL if callback doesn't need user data.

Since Version 1.0.0

—

cmt1_isr

```
void cmt1_isr(void)
```

CMT1 (CMT channel 1) compare-match interrupt service routine.

Invoked by the RX72N hardware whenever the CMT1 counter (CMCNT) reaches the value stored in CMCOR. The ISR checks whether a callback has been registered in `s_cmt_callback[k_cmt_channel_1]` and, if so, calls it with the associated user data pointer from `s_cmt_user_data[k_cmt_channel_1]`.

The compare-match interrupt flag is automatically cleared by the RX72N hardware upon entry; no explicit IR register write is needed.

Note: Executes in interrupt context; all code in this ISR and any registered callback must be ISR-safe (no blocking, no long operations)

Note: Not thread-safe; the callback executes with the CMT1 interrupt masked

Warning: Do not call any ThreadX API that is not interrupt-safe from the registered callback (e.g., `tx_thread_sleep()` is forbidden; use ISR-safe variants such as `tx_semaphore_put_from_isr()`)

See also: `rx_cmt_init()` Registers the callback invoked by this ISR, `rx_cmt_deinit()` Clears the callback and disables this interrupt, `s_cmt_callback` Per-channel callback function pointer array

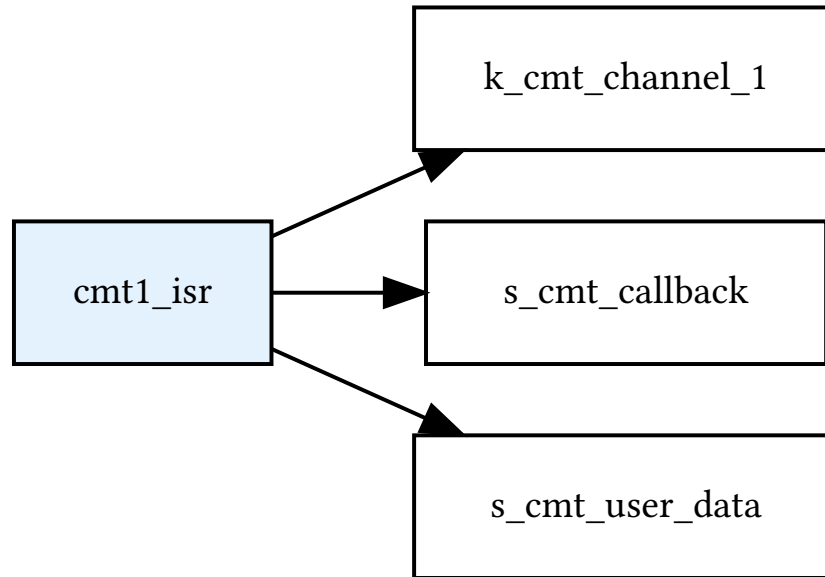


Figure 777: Call graph for `cmt1_isr`

Defined in `libs/rx_hal/src/rx_cmt.c:764`

Since Version 1.0.0

—

cmt2_isr

```
void cmt2_isr(void)
```

CMT2 (CMT channel 2) compare-match interrupt service routine.

Invoked by the RX72N hardware whenever the CMT2 counter (CMCNT) reaches the value stored in CMCOR. The ISR checks whether a callback has been registered in `s_cmt_callback[k_cmt_channel_2]` and, if so, calls it with the associated user data pointer from `s_cmt_user_data[k_cmt_channel_2]`.

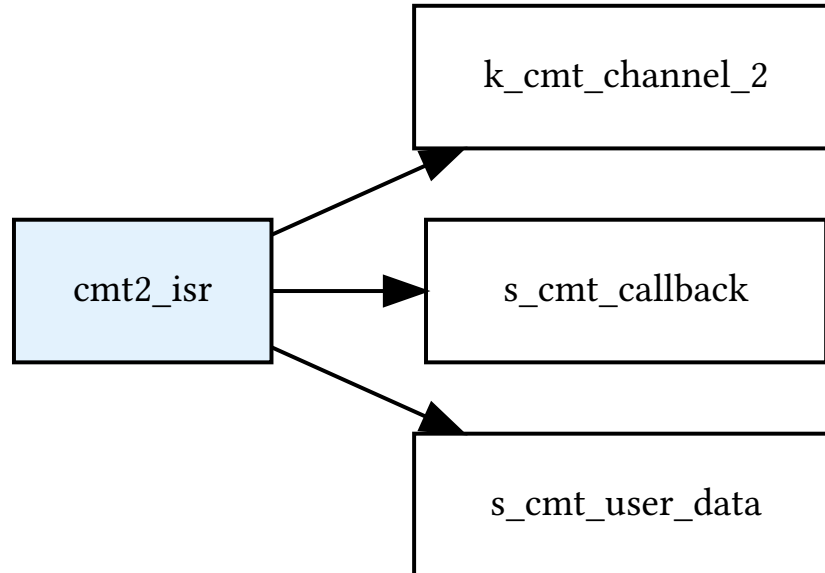
The compare-match interrupt flag is automatically cleared by the RX72N hardware upon entry; no explicit IR register write is needed.

Note: Executes in interrupt context; all code in this ISR and any registered callback must be ISR-safe (no blocking, no long operations)

Note: Not thread-safe; the callback executes with the CMT2 interrupt masked

Warning: Do not call any ThreadX API that is not interrupt-safe from the registered callback (e.g., `tx_thread_sleep()` is forbidden; use ISR-safe variants such as `tx_semaphore_put_from_isr()`)

See also: `rx_cmt_init()` Registers the callback invoked by this ISR, `rx_cmt_deinit()` Clears the callback and disables this interrupt, `s_cmt_callback` Per-channel callback function pointer array

Figure 778: Call graph for `cmt2_isr`

Defined in `libs/rx_hal/src/rx_cmt.c:800`

Since Version 1.0.0

—

cmt3_isr

```
void cmt3_isr(void)
```

CMT3 (CMT channel 3) compare-match interrupt service routine.

Invoked by the RX72N hardware whenever the CMT3 counter (CMCNT) reaches the value stored in CMCOR. The ISR checks whether a callback has been registered in `s_cmt_callback[k_cmt_channel_3]` and, if so, calls it with the associated user data pointer from `s_cmt_user_data[k_cmt_channel_3]`.

The compare-match interrupt flag is automatically cleared by the RX72N hardware upon entry; no explicit IR register write is needed.

Note: Executes in interrupt context; all code in this ISR and any registered callback must be ISR-safe (no blocking, no long operations)

Note: Not thread-safe; the callback executes with the CMT3 interrupt masked

Warning: Do not call any ThreadX API that is not interrupt-safe from the registered callback (e.g., `tx_thread_sleep()` is forbidden; use ISR-safe variants such as `tx_semaphore_put_from_isr()`)
See also: `rx_cmt_init()` Registers the callback invoked by this ISR, `rx_cmt_deinit()` Clears the callback and disables this interrupt, `s_cmt_callback` Per-channel callback function pointer array

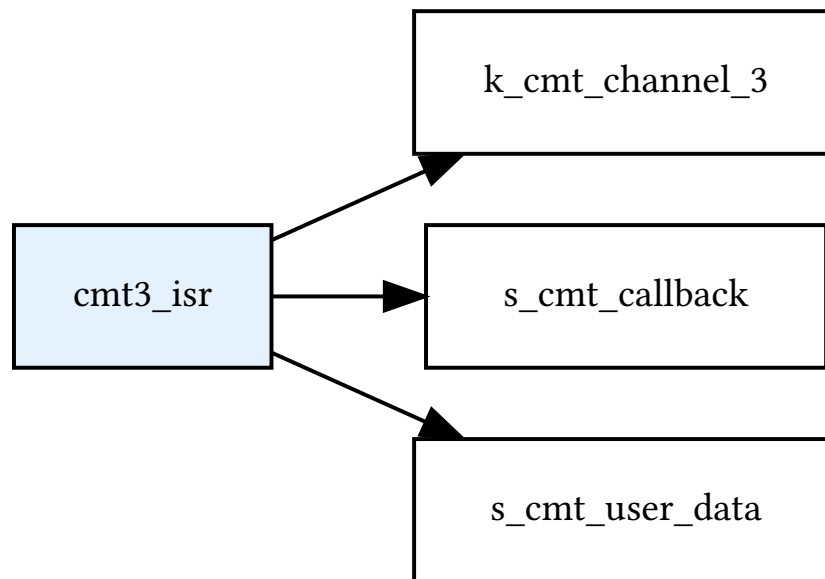


Figure 779: Call graph for `cmt3_isr`

Defined in `libs/rx_hal/src/rx_cmt.c:833`

Since Version 1.0.0

—

internal_get_cmt_base

```
static volatile rx_cmt_channel_regs_t * internal_get_cmt_base(const
rx_cmt_channel_t channel)
```

Map a CMT channel enum to its hardware register base pointer.

Uses a switch statement to convert a logical channel identifier into a pointer to the corresponding volatile `rx_cmt_channel_regs_t` register block. The four CMT channels (CMT0-CMT3) each have a distinct base address in the RX72N memory map; the inline accessor functions `cmt0()`-`cmt3()` encode those addresses in a type-safe manner.

The default case returns `nullptr`, which every caller is expected to check immediately after the call (NASA Rule 7 compliance).

Parameters:

Direction	Name	Description
in	channel	CMT channel to resolve Valid range: k_cmt_channel_0 through k_cmt_channel_3 Any other value results in a nullptr return

Returns: Pointer to the volatile CMT channel register block

Value	Description
Non-null	Pointer to rx_cmt_channel_regs_t for the requested channel
nullptr	channel is outside the valid range [0, 3]

Pre: channel is a member of rx_cmt_channel_t (0-3)

Pre: Hardware register addresses must be accessible (clocks enabled before use)

Post: Return value, if non-null, points to the live hardware register block

Post: Caller must validate the return value before dereferencing (Rule 7)

Note: Not thread-safe; the hardware registers are globally shared

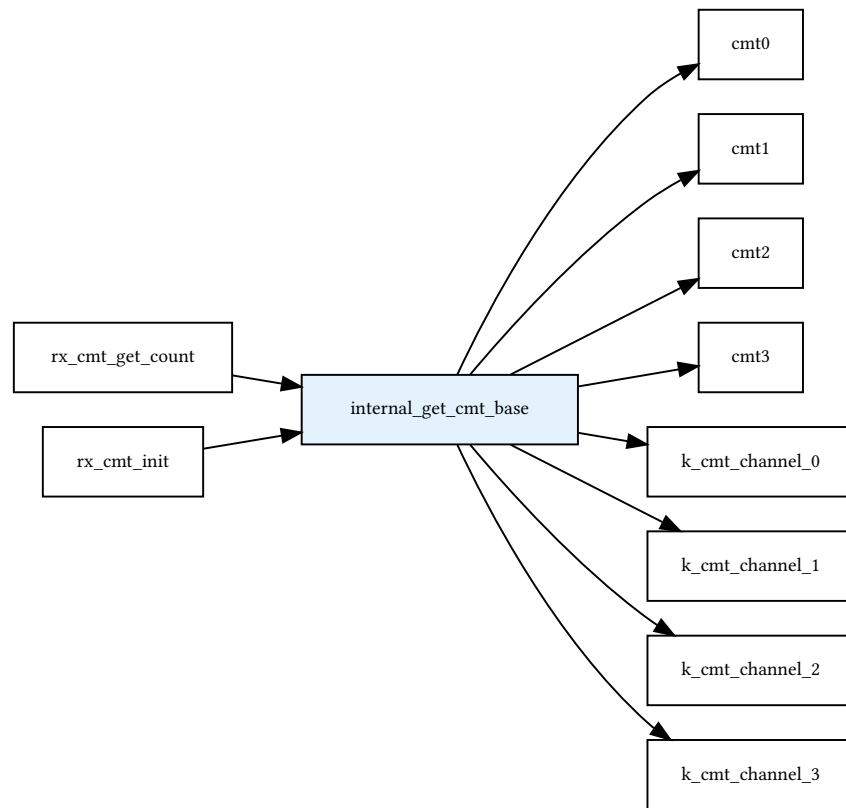
Note: This function never writes to hardware; it only computes an address

Thread Safety:: Read-only address computation; no shared mutable state is accessed.

Performance:: Execution time: 1 cycle (single branch/address load with -O2)

Example:: `volatilerx_cmt_channel_regs_t*cmt=internal_get_cmt_base(k_cmt_channel_2);
if(cmt==nullptr){ returnk_rx_err_invalid_arg; } cmt->cment=0;`

See also: `cmt0()` Inline accessor for CMT0 base address, `cmt1()` Inline accessor for CMT1 base address, `cmt2()` Inline accessor for CMT2 base address, `cmt3()` Inline accessor for CMT3 base address

Figure 780: Call graph for `internal_get_cmt_base`

Defined in `libs/rx_hal/src/rx_cmt.c:384`

Since Version 1.0.0

—

internal_calculate_cmt_params

```
static rx_err_t internal_calculate_cmt_params(const uint32_t frequency_hz,
uint8_t *divider, uint16_t *cmcor)
```

Calculate the optimal CMT clock divider and compare-match value for a target frequency.

Iterates the four available PCLKB clock dividers (/8, /32, /128, /512) in ascending order and selects the first divider whose resulting 16-bit period value satisfies:

The chosen divider index is returned in `*divider` (corresponding to the CKS field of CMCR: 0=/8, 1=/32, 2=/128, 3=/512). The compare-match register value is returned in `*cmcor`; the caller must subtract the adjustment constant `k_cmt_period_adj` when writing to the hardware register.

Algorithm steps:

1. Validate output pointers are non-null
2. Reject `frequency_hz == 0` or `frequency_hz > PCLKB/8` (maximum achievable)

3. Loop i from k_cmt_divider_start to k_cmt_num_dividers - 1
4. Compute period_calc = (PCLKB / dividers[i]) / frequency_hz
5. If $0 < \text{period_calc} \leq 65535$, store i and period_calc, return k_rx_ok
6. If no divider fits, return k_rx_err_invalid_arg

Parameters:

Direction	Name	Description
in	frequency_hz	Target interrupt frequency in Hz Valid range: 1 to PCLKB/8 (typically 1 to 7,500,000 Hz) Constraint: Must produce a period that fits in 16 bits for at least one of the four clock dividers
out	divider	CKS field value to write into CMCR.CKS (0-3) On success: Set to the index of the chosen divider On failure: Undefined
out	cmcor	Raw period count before the -1 adjustment On success: Set to (PCLKB / divider_value) / frequency_hz On failure: Undefined

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Divider and compare-match value computed successfully
k_rx_err_null_ptr	divider or cmcor is nullptr
k_rx_err_invalid_arg	frequency_hz is 0, exceeds PCLKB/8, or no divider produces a period fitting in 16 bits

Pre: divider must point to a valid uint8_t storage location

Pre: cmcor must point to a valid uint16_t storage location

Post: On k_rx_ok: *divider holds the CKS value (0-3) for CMCR

Post: On k_rx_ok: *cmcor holds the period count; caller subtracts 1 before writing CMCR

Note: Not thread-safe; pure computation with no shared state modification

Note: Uses a bounded loop (k_cmt_num_dividers iterations) NASA Rule 2 compliant

Thread Safety:: Pure computation with no global or static state; safe to call from any context.

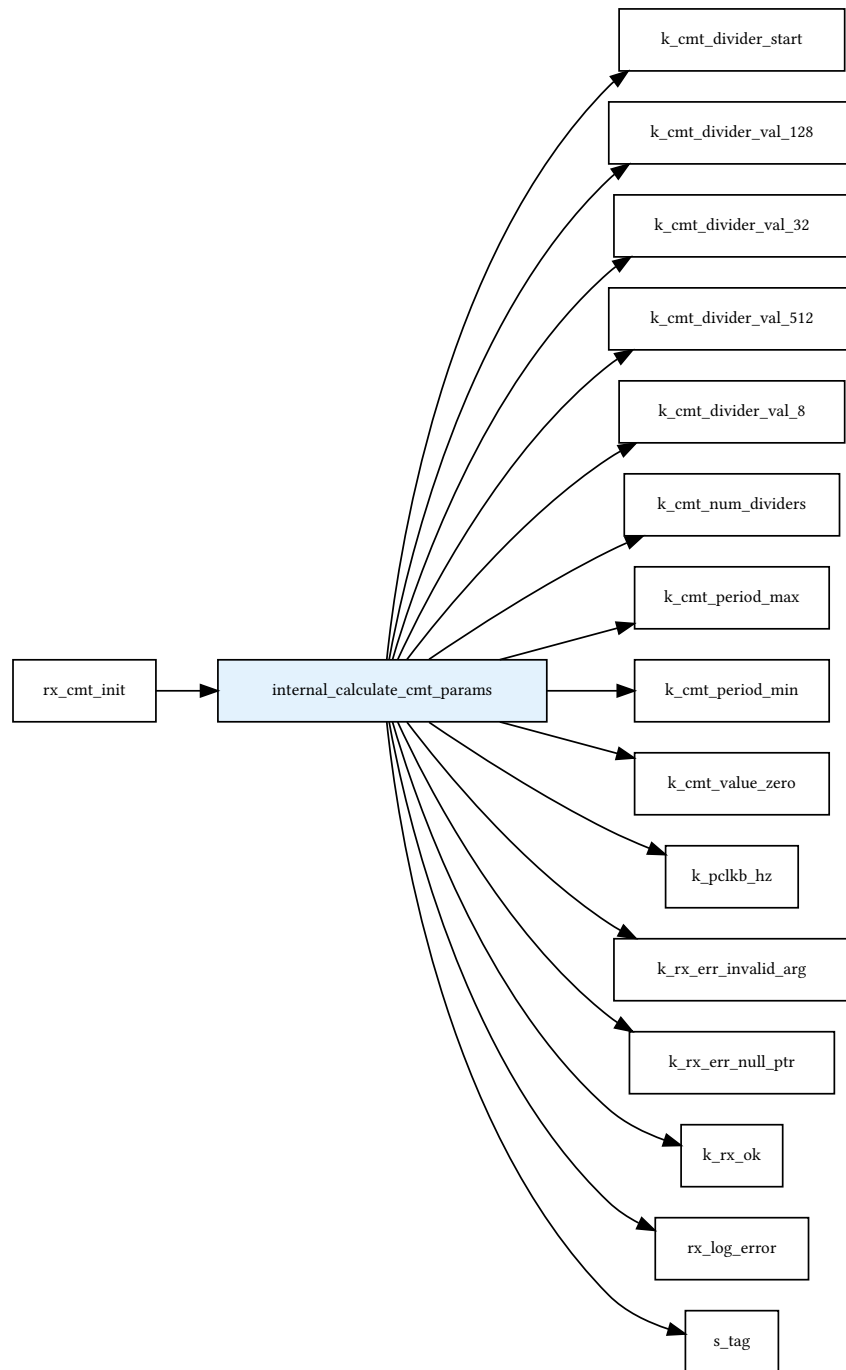
Performance:: Execution time: 5 cycles (loop unrolled at -O2 for 4 divider steps)

Example:: uint8_tdivider=0; uint16_tcmcor=0;
rx_err_terr=internal_calculate_cmt_params(1000U,÷r,&cmcor); if(err==k_rx_ok){ }

Example:

```
period=(PCLKB/divider)/frequency_hz
0<period<=65535
```

See also: `internal_configure_cmt_timer_registers()` Consumes the output of this function, `cmt_constants_t` Divider index constants (`k_cmt_div_8` ... `k_cmt_div_512`), `cmt_divider_values_t` Actual divisor values (8, 32, 128, 512)

Figure 781: Call graph for `internal_calculate_cmt_params`*Defined in `libs/rx_hal/src/rx_cmt.c:475`*

Since Version 1.0.0

—

internal_enable_cmt_module_clock

```
static void internal_enable_cmt_module_clock(void)
```

Enable the CMT module clock by clearing the MSTPCRB module-stop bit.

Removes the CMT0-CMT3 peripheral from module-stop state so that the module clock is supplied and its registers become accessible. The sequence is:

1. Unlock the write-protect register (PRCR) for PRC1 and PRC3 bits
2. Clear bit k_cmt_mstpb_cmt (bit 15) in the MSTPCRB register
3. Re-lock the PRCR to prevent accidental modification

This function must be called before any CMT register access; reading or writing CMT registers while the module clock is stopped produces undefined hardware behaviour on the RX72N.

The function is idempotent: calling it when the module is already enabled simply clears an already-clear bit, which is harmless.

Returns: void This function does not return an error code. Hardware register access failures are not detectable at this level; the caller relies on subsequent register reads for verification.

Pre: The system clock (PCLKB) must be running before calling this function

Pre: No other task or ISR must be concurrently modifying MSTPCRB

Post: MSTPCRB bit 15 is cleared; CMT module clock is active

Post: PRCR is locked upon return (prevents accidental MSTPCRB changes)

Note: Not thread-safe; PRCR unlock/lock is a non-atomic read-modify-write

Note: Called once per system lifetime; subsequent calls are benign

Thread Safety:: Not thread-safe. Call during system initialization before RTOS scheduler starts, or with interrupts disabled.

Performance:: Execution time: 4 register writes; negligible overhead

Example:: internal_enable_cmt_module_clock();

See also: rx_register_protection.h PRCR lock/unlock constants, rx_cmt_init() Sole caller; invoked before register programming

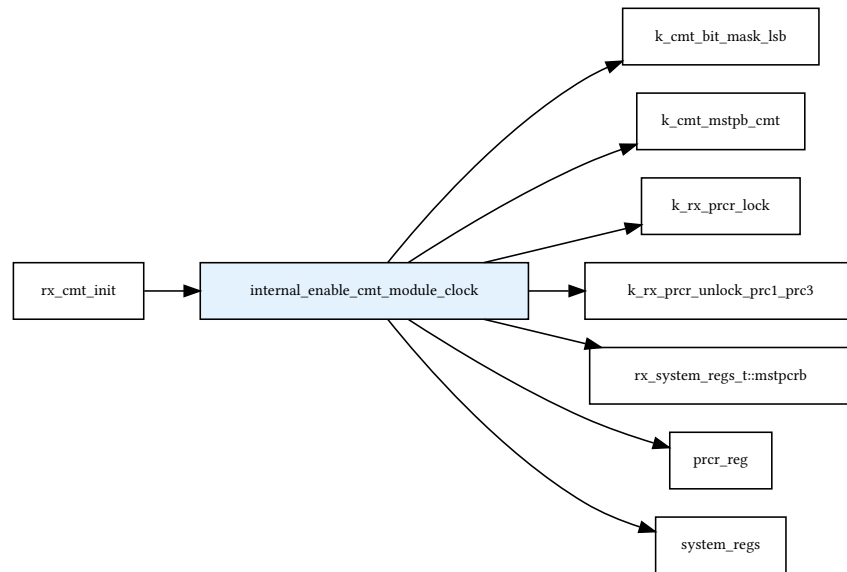


Figure 782: Call graph for internal_enable_cmt_module_clock

Defined in `libs/rx_hal/src/rx_cmt.c:558`

Since Version 1.0.0

internal_configure_cmt_timer_registers

```
static void internal_configure_cmt_timer_registers(volatile rx_cmt_channel_regs_t
*cmt, const uint8_t divider, const uint16_t cmcor)
```

Write clock divider, interrupt-enable, compare-match, and counter registers.

Programs the three CMT hardware registers for a single channel in the following order:

1. CMCR - Sets the CKS (clock-select) field from divider and enables the compare-match interrupt (CMIE bit).
2. CMCOR - Loads the compare-match value after subtracting the adjustment constant `k_cmt_period_adj (-1)` so the hardware fires at the correct period.
3. CMCNT - Clears the counter to 0 so the first interrupt occurs exactly one full period after the timer is started.

The timer must be stopped (CMSTR bit cleared) before calling this function to avoid race conditions while writing to CMCOR and CMCNT.

Parameters:

Direction	Name	Description
in	cmt	Pointer to the volatile CMT channel register block Constraint: Must not be nullptr (asserted)

in	divider	CKS clock-select field value (0-3) produced by internal_calculate_cmt_params() 0 = PCLKB/8, 1 = PCLKB/32, 2 = PCLKB/128, 3 = PCLKB/512
in	cmcor	Period count (before adjustment) produced by internal_calculate_cmt_params(); the -1 adjustment is applied internally before writing to hardware

Returns: void This function does not return an error code; the assert on cmt is the only validation gate.

Pre: cmt must not be nullptr (enforced by RX_ASSERT)

Pre: The CMT timer must be stopped before calling this function

Post: cmt->cmcr = (divider << CKS_SHIFT) | CMIE_BIT

Post: cmt->cmcor = cmcor - k_cmt_period_adj

Post: cmt->cmcnt = 0

Note: Not thread-safe; caller must ensure the timer is stopped

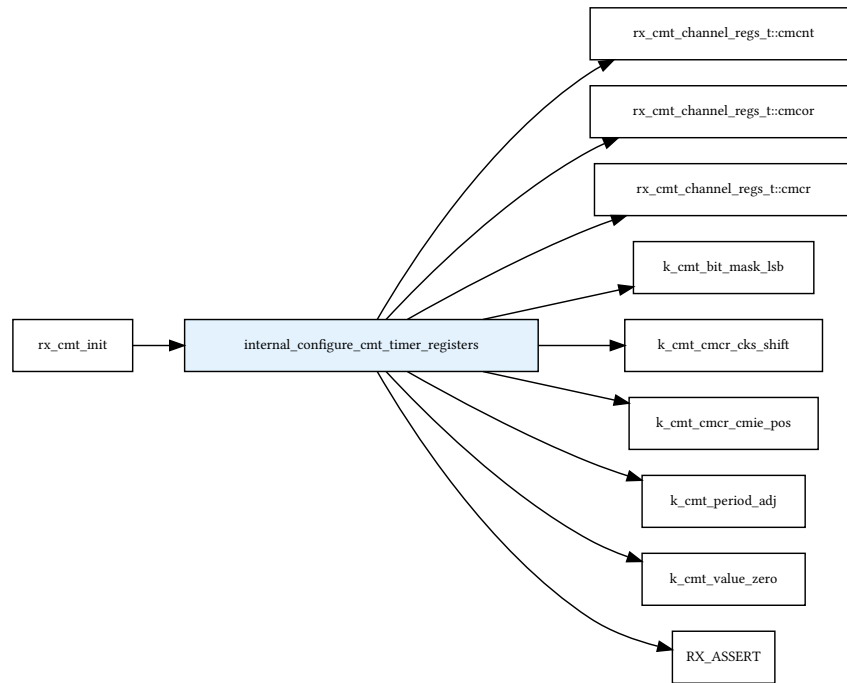
Note: RX_ASSERT fires in debug builds if cmt is nullptr

Thread Safety:: Not thread-safe. Must be called with the timer stopped and no concurrent ISR activity on this channel.

Performance:: Execution time: 3 register writes; negligible overhead

Example:: uint8_tdivider=0; uint16_tcmcor=0;
 (void)internal_calculate_cmt_params(1000U,÷r,&cmcor);
 volatilerx_cmt_channel_regs_t*cmt=internal_get_cmt_base(k_cmt_channel_1);
 internal_configure_cmt_timer_registers(cmt,divider,cmcor);

See also: internal_calculate_cmt_params() Produces the divider and cmcor arguments,
 rx_cmt_stop() Must be called before this function, cmt_cmcr_bits_t CKS shift and CMIE
 bit position constants

Figure 783: Call graph for `internal_configure_cmt_timer_registers`

Defined in `libs/rx_hal/src/rx_cmt.c:626`

Since Version 1.0.0

—

internal_configure_cmt_interrupt

```
static rx_err_t internal_configure_cmt_interrupt(const rx_cmt_interrupt_config_t config)
```

Configure the ICU interrupt vector priority and enable for a CMT channel.

Programs the Interrupt Controller Unit (ICU) to route and enable the compare-match interrupt (CMIn) for the specified CMT channel. The three ICU registers modified are:

1. IPR[vector] - Sets the interrupt priority level for the channel's CMIn vector to `config.priority`.
2. IER[index] - Sets the enable bit for the channel's interrupt vector within the IER register array.
The bit index and byte index are derived from the vector number.
3. IR[vector] - Clears any pending interrupt flag before enabling, so a stale flag does not fire immediately after enable.

The vector number is computed as `k_vect_cmt0_cmi0 + config.channel`, which yields CMI0 for channel 0, CMI1 for channel 1, and so on.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	config	Interrupt routing and priority specification (passed by value) config.channel: CMT channel (0-3); mapped to ICU vector config.priority: ICU priority (k_ipr_level_min to k_ipr_level_max)
----	--------	---

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	ICU programmed successfully
k_rx_err_invalid_arg	config.channel >= k_cmt_max_channels, or config.priority outside [k_ipr_level_min, k_ipr_level_max]

Pre: config.channel must be in the range 0, k_cmt_max_channels)

Pre: config.priority must be in k_ipr_level_min, k_ipr_level_max

Post: ICU IPR register for the channel's vector set to config.priority

Post: ICU IER bit for the channel's vector enabled

Post: ICU IR flag for the channel's vector cleared

Note: Not thread-safe; concurrent ICU modification from another context is unsafe

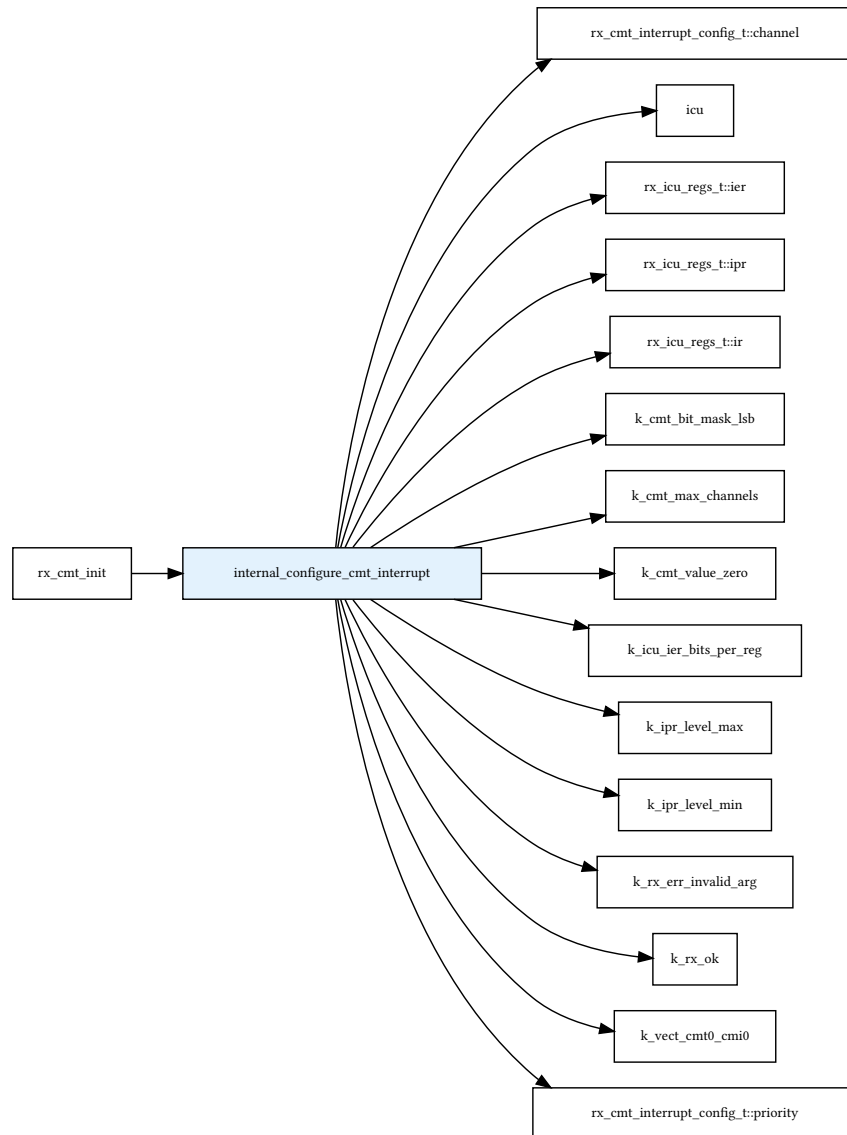
Note: Clears IR before enabling to prevent a spurious first interrupt

Thread Safety:: Not thread-safe. Must be called during initialization with interrupts disabled or before the RTOS scheduler starts.

Performance:: Execution time: 4 register accesses; negligible overhead

Example:: rx_cmt_interrupt_config_tirq_cfg={ .channel=k_cmt_channel_1, .priority=5, };
rx_err_t err=internal_configure_cmt_interrupt(irq_cfg); if(err!=k_rx_ok){ return err; }

See also: rx_cmt_interrupt_config_t Configuration structure passed to this function,
rx_cmt_deinit() Clears the IER bit to disable the interrupt, rx_cmt_init() Constructs
config and calls this function

Figure 784: Call graph for `internal_configure_cmt_interrupt`

Defined in `libs/rx_hal/src/rx_cmt.c:710`

Since Version 1.0.0

`internal_validate_cmt_init_params`

```
static rx_err_t internal_validate_cmt_init_params(const rx_cmt_channel_t channel,
const rx_cmt_config_t *config)
```

Validate channel and configuration pointer before CMT initialisation.

Performs all parameter checking required before `rx_cmt_init()` proceeds to hardware configuration.

Validation is performed in this strict order:

1. Verify config pointer is non-null (`k_rx_err_null_ptr`)
2. Verify channel is within `[0, k_cmt_max_channels)` (`k_rx_err_invalid_arg`)
3. Reject channel 0, which is reserved for the ThreadX system-tick timer (`k_rx_err_conflict`)
4. Verify interrupt priority is within `[k_ipr_level_min, k_ipr_level_max]` (`k_rx_err_invalid_arg`)

Centralising these checks in a dedicated helper keeps the main `rx_cmt_init()` function concise and ensures that every check is logged before returning.

Parameters:

Direction	Name	Description
in	channel	CMT channel to validate Valid range: <code>k_cmt_channel_1</code> through <code>k_cmt_channel_3</code> <code>k_cmt_channel_0</code> is rejected (reserved for ThreadX)
in	config	Pointer to the channel configuration structure Null handling: Returns <code>k_rx_err_null_ptr</code> immediately if nullptr priority field: Must be in <code>[k_ipr_level_min, k_ipr_level_max]</code>

Returns: `rx_err_t` Error code indicating success or the first validation failure

Value	Description
<code>k_rx_ok</code>	All parameters are valid
<code>k_rx_err_null_ptr</code>	config is nullptr
<code>k_rx_err_invalid_arg</code>	channel $\geq$ <code>k_cmt_max_channels</code> , or config->priority out of <code>[min, max]</code>
<code>k_rx_err_conflict</code>	channel == <code>k_cmt_channel_0</code> (ThreadX reserved)

Pre: config must be a pointer to a caller-owned `rx_cmt_config_t` (may be nullptr)

Pre: channel must be a member of `rx_cmt_channel_t`

Post: On `k_rx_ok`: channel and *config are safe to use for hardware programming

Post: On error: an error message has been emitted via `rx_log_error()`

Note: Not thread-safe; pure validation with no shared state modification

Note: The lower-bound check `channel  $\geq$  k_cmt_channel_0` is intentionally omitted to avoid -Wtype-limits: channel is `uint8_t` and `k_cmt_channel_0 == 0`, making the comparison always true

Thread Safety:: Pure validation; no shared mutable state is modified. Safe to call from any context, but the result may be stale if state changes after the call.

Performance:: Execution time: 4 comparisons; negligible overhead

Example:: rx_err_t err=internal_validate_cmt_init_params(k_cmt_channel_1,&config); if(err!=k_rx_ok){ return err; }

See also: rx_cmt_init() Sole caller; uses this to guard hardware programming, rx_cmt_config_t Configuration structure whose fields are validated here

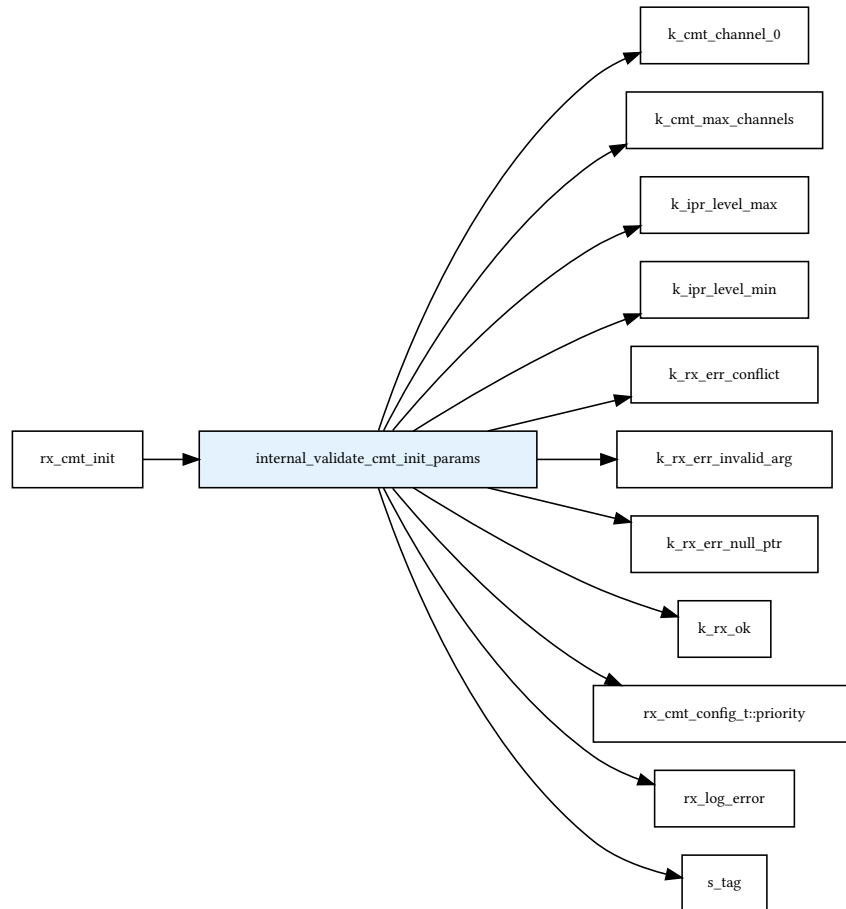


Figure 785: Call graph for internal_validate_cmt_init_params

Defined in `libs/rx_hal/src/rx_cmt.c:907`

Since Version 1.0.0

—

internal_save_cmt_callback

```
static void internal_save_cmt_callback(const rx_cmt_channel_t channel, const
rx_cmt_config_t *config)
```

Persist the user callback, user-data pointer, and channel initialized flag.

Copies the callback function pointer and user-data pointer from the caller's configuration structure into the module-level static arrays, then marks the channel as initialized. After this call the ISR for the channel will invoke the registered callback on every compare-match event.

This function is the final step in the `rx_cmt_init()` sequence. It must only be called after all hardware registers have been programmed and the interrupt has been configured in the ICU, so that the ISR never encounters a partially initialised channel state.

Parameters:

Direction	Name	Description
in	channel	CMT channel whose callback state is to be saved Valid range: <code>k_cmt_channel_1</code> through <code>k_cmt_channel_3</code> Caller (<code>rx_cmt_init</code>) has already validated the channel
in	config	Pointer to the CMT configuration structure <code>config->callback</code> : Function pointer (may be nullptr; ISR is a no-op if null) <code>config->user_data</code> : Opaque pointer passed verbatim to callback (may be nullptr) Null handling: Caller guarantees config is non-null

Returns: void This function does not return an error code; all preconditions are enforced by the caller before this function is invoked.

Pre: channel must be valid and in range 1, `k_cmt_max_channels`) (validated by caller)

Pre: config must be non-null (validated by `internal_validate_cmt_init_params`)

Post: `s_cmt_callbackchannel = config->callback`

Post: `s_cmt_user_datachannel = config->user_data`

Post: `s_cmt_initializedchannel = true`

Note: Not thread-safe; must be the last step of `rx_cmt_init()` with interrupts either disabled or not yet fired for this channel

Note: After this call the channel is considered "live"; the ISR may fire at any point

Thread Safety:: Not thread-safe. The three static array writes are not atomic. Call only during initialization with the channel interrupt not yet active.

Performance:: Execution time: 3 stores; negligible overhead

Example:: `internal_save_cmt_callback(channel,config);`

See also: `rx_cmt_init()` Sole caller; calls this as the last initialisation step, `rx_cmt_deinit()` Clears the state written by this function, `s_cmt_callback` Destination array for `config->callback`, `s_cmt_user_data` Destination array for `config->user_data`, `s_cmt_initialized` Destination array for the initialized flag

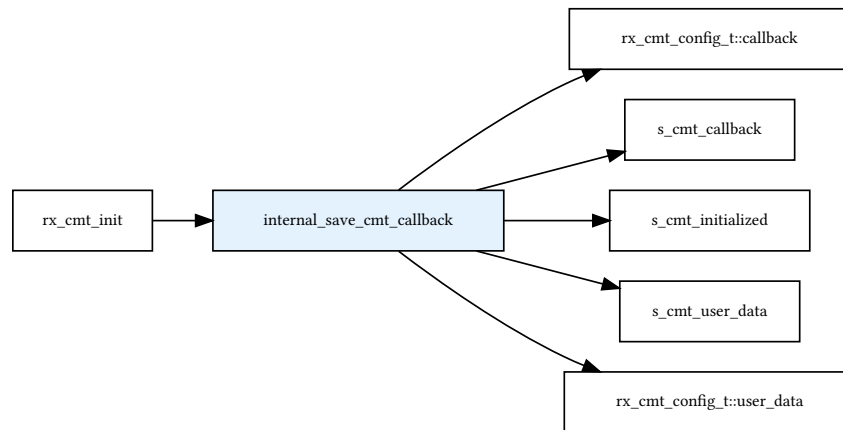


Figure 786: Call graph for internal_save_cmt_callback

Defined in `libs/rx_hal/src/rx_cmt.c:995`

Since Version 1.0.0

—

rx_cmt_init

```
rx_err_t rx_cmt_init(const rx_cmt_channel_t channel, const rx_cmt_config_t
*config)
```

Initialize a CMT channel for periodic interrupt generation.

Initialize CMT channel for periodic interrupts.

Configures a CMT (Compare Match Timer) channel to fire a user callback at the requested frequency. Validates parameters, selects the optimal clock divider, enables the module clock, stops the timer for safe configuration, programs timer registers, configures the ICU interrupt routing, saves the callback, and starts the timer.

Algorithm steps:

1. Validate channel and config parameters via `internal_validate_cmt_init_params()`
2. Obtain CMT channel register base via `internal_get_cmt_base()`
3. Calculate optimal divider and CMCOR value via `internal_calculate_cmt_params()`
4. Enable CMT module clock (clear MSTPCRB bit) via `internal_enable_cmt_module_clock()`
5. Stop the timer to allow safe register writes via `rx_cmt_stop()`
6. Program CMCR (divider + CMIE) and CMCOR via `internal_configure_cmt_timer_registers()`
7. Configure ICU interrupt vector priority and enable via `internal_configure_cmt_interrupt()`
8. Save callback pointer and mark channel initialized via `internal_save_cmt_callback()`
9. Start the timer via `rx_cmt_start()`

Clock divider selection: The driver iterates dividers /8, /32, /128, /512 and picks the first that produces a period fitting in 16 bits:

Parameters:

Direction	Name	Description
-----------	------	-------------

in	channel	CMT channel to initialize Valid range: k_cmt_channel_1, k_cmt_channel_2, k_cmt_channel_3 Reserved: k_cmt_channel_0 is reserved for ThreadX system tick
in	config	Pointer to channel configuration structure frequency_hz: Desired interrupt frequency in Hz (> 0, <= PCLKB/8) priority: ICU interrupt priority (k_ipr_level_min to k_ipr_level_max) callback: Function called from interrupt context on each match user_data: Opaque pointer passed to callback (may be nullptr) Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success; timer running and callback registered
k_rx_err_null_ptr	config is nullptr
k_rx_err_invalid_arg	Invalid channel, frequency out of range, or invalid priority
k_rx_err_conflict	channel is k_cmt_channel_0 (reserved for ThreadX)
k_rx_err_hw_error	Timer start verification failed (CMSTR readback mismatch)

Pre: config must point to a valid rx_cmt_config_t structure

Pre: config->frequency_hz must be > 0 and <= PCLKB / 8 (7.5 MHz at 60 MHz PCLKB)

Pre: channel must not be k_cmt_channel_0

Pre: System clocks must be initialized (PCLKB = 60 MHz)

Post: Timer running at requested frequency

Post: config->callback registered and called from ISR at each compare match

Post: s_cmt_initializedchannel set to true

Post: ICU interrupt for the channel enabled at specified priority

Note: Not thread-safe - call once per channel during system initialization

Note: Callback executes in interrupt context; keep it short and non-blocking

Warning: CMT0 is reserved for ThreadX; do not use k_cmt_channel_0

Thread Safety:: Not thread-safe. Call once per channel during initialization before starting the RTOS scheduler or with interrupts disabled.

Performance:: Execution time: 5 us (mostly register writes) Stack usage: 48 bytes

Example::

```
static void my_timer_cb(void* user_data) { (void) user_data; toggle_led(); }  
const rx_cmt_config_t cfg = { .frequency_hz = 1000, .priority = 5, .callback = my_timer_cb, .user_data = nullptr };  
rx_err_t err = rx_cmt_init(k_cmt_channel_1, &cfg); if (err != k_rx_ok)  
{ rx_log_error("APP", "CMT init failed"); }
```

Example:

```
period = (PCLKB / divider) / frequency_hz  
if (0 < period <= 65535) : use this divider
```

See also: rx_cmt_deinit() Stop and release channel, rx_cmt_start() Start a previously stopped channel, rx_cmt_stop() Stop the timer without deinitializing, rx_cmt_config_t Configuration structure definition



Defined in `libs/rx_hal/src/rx_cmt.c:1106`

Since Version 1.0.0

—

rx_cmt_start

```
rx_err_t rx_cmt_start(const rx_cmt_channel_t channel)
```

Start CMT counter for a previously initialized channel.

Start CMT timer.

Sets the appropriate bit in the CMSTR0 or CMSTR1 register to start the CMT counter running. After setting the bit the register is read back to verify the hardware accepted the write. Channels 0/1 share CMSTR0; channels 2/3 share CMSTR1. Channels 0 and 1 use STR0/STR1 bits respectively, as do channels 2 and 3.

Parameters:

Direction	Name	Description
in	channel	CMT channel to start Valid range: k_cmt_channel_0 through k_cmt_channel_3 Note: Usually called internally by rx_cmt_init()

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success; counter is running
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized
k_rx_err_hw_error	CMSTR readback did not show bit set

Pre: channel must be initialized via rx_cmt_init()

Pre: channel must be in range 0, 3

Post: CMSTR bit for channel is set

Post: Counter begins incrementing from its current value

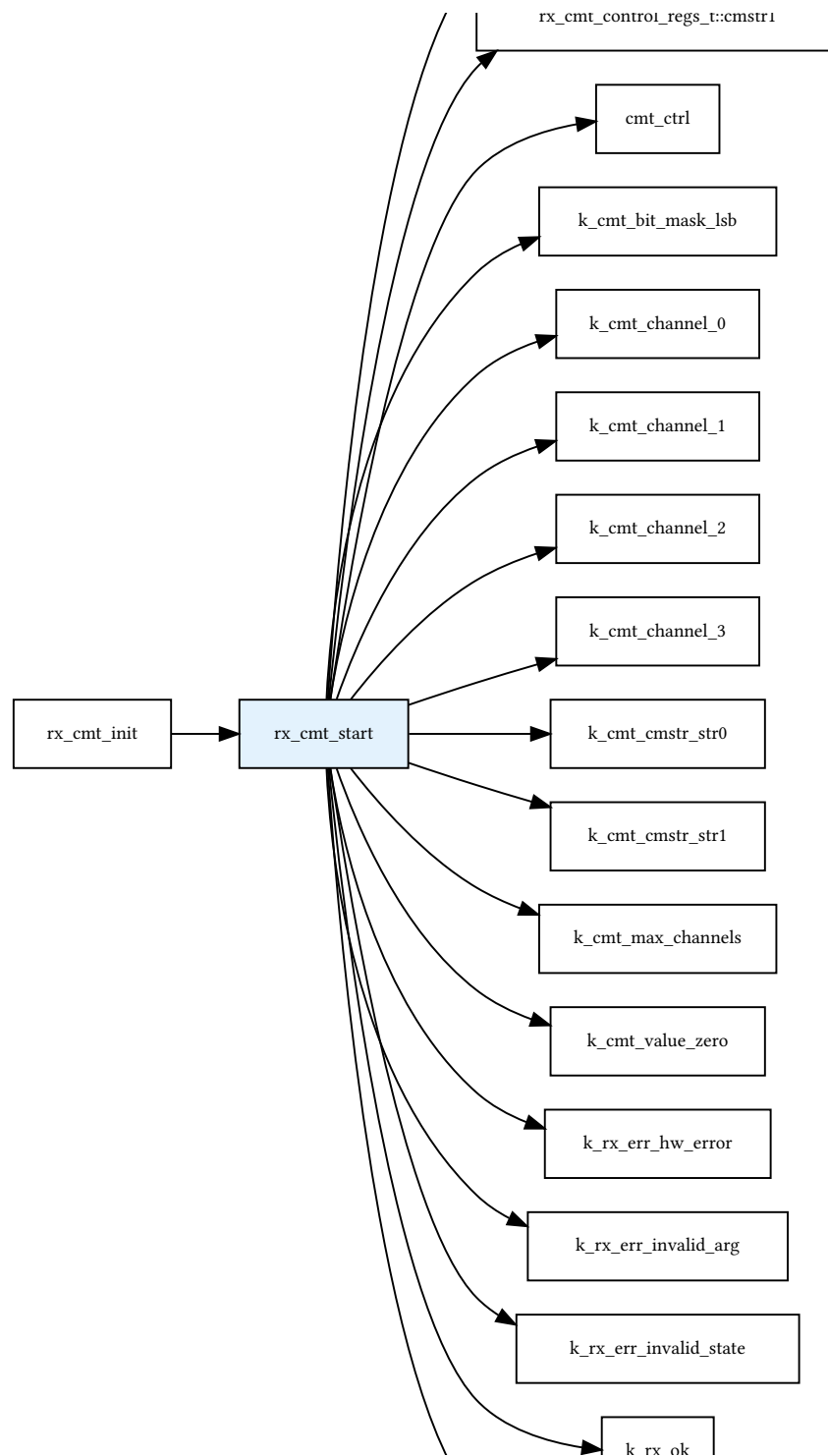
Note: Usually called automatically by rx_cmt_init(); call explicitly only after rx_cmt_stop()

Note: Performs a readback check to verify the hardware write succeeded

Thread Safety:: Not thread-safe; do not call concurrently with stop/deinit on the same channel.

Example:: rx_cmt_stop(k_cmt_channel_1); rx_cmt_start(k_cmt_channel_1);

See also: `rx_cmt_stop()` Stop the counter, `rx_cmt_init()` Initialize channel (calls start internally)

Figure 788: Call graph for `rx_cmt_start`

Defined in `libs/rx_hal/src/rx_cmt.c:1205`

Since Version 1.0.0

—

rx_cmt_stop

```
rx_err_t rx_cmt_stop(const rx_cmt_channel_t channel)
```

Stop CMT counter without deinitializing the channel.

Stop CMT timer.

Clears the appropriate bit in CMSTR0 or CMSTR1 to halt the CMT counter. After clearing the bit the register is read back to verify the hardware accepted the write. The channel remains initialized; call `rx_cmt_start()` to resume counting.

Note that `rx_cmt_stop()` does NOT require the channel to be initialized (`s_cmt_initialized` check is omitted intentionally so that `rx_cmt_init()` can call it safely before marking the channel as initialized).

Parameters:

Direction	Name	Description
in	channel	CMT channel to stop Valid range: <code>k_cmt_channel_0</code> through <code>k_cmt_channel_3</code>

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success; counter halted
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_hw_error</code>	CMSTR readback showed bit still set

Pre: channel must be in range 0, 3

Pre: Interrupts may still fire if one is already pending when stop is called

Post: CMSTR bit for channel is cleared

Post: Counter stops incrementing (current CMCNT value preserved)

Note: Does not clear the initialized flag - channel can be restarted

Note: Called by `rx_cmt_init()` before programming registers (safe on uninitialized channel)

Thread Safety:: Not thread-safe; do not call concurrently with start/init/deinit on the same channel.

Example:: rx_cmt_stop(k_cmt_channel_2); reconfigure_something();
rx_cmt_start(k_cmt_channel_2);

See also: rx_cmt_start() Resume counting, rx_cmt_deinit() Stop and release channel resources

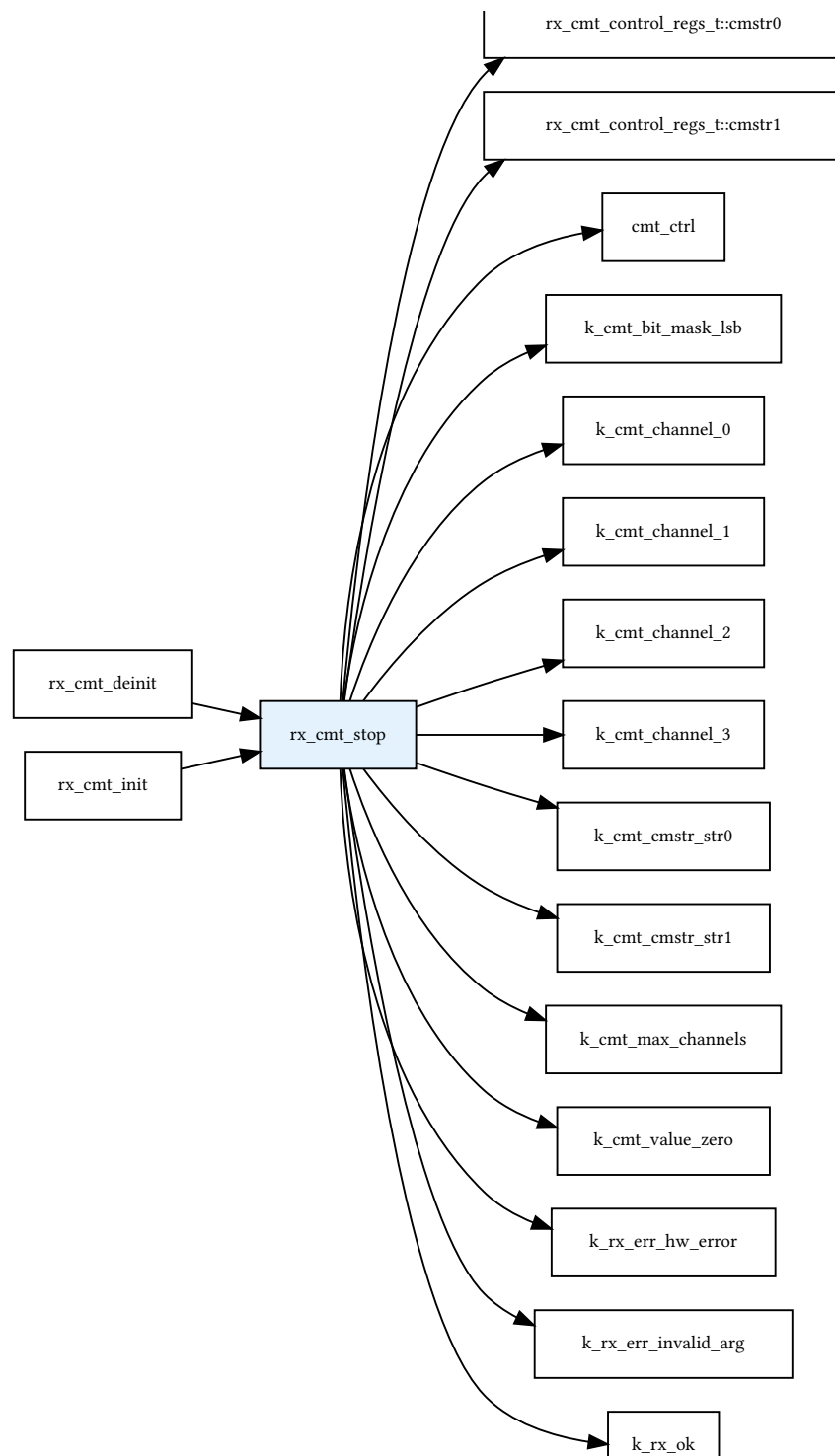


Figure 789: Call graph for `rx_cmt_stop`

Defined in `libs/rx_hal/src/rx_cmt.c:1306`

Since Version 1.0.0

—

rx_cmt_get_count

```
rx_err_t rx_cmt_get_count(const rx_cmt_channel_t channel, uint16_t *count)
```

Read current CMT counter (CMCNT) value.

Get CMT counter value.

Returns the instantaneous value of the 16-bit CMCNT register for the specified channel. The counter increments at PCLKB divided by the configured divider (/8, /32, /128, or /512) and resets to 0 on compare match. Useful for measuring elapsed time within a period.

Parameters:

Direction	Name	Description
in	channel	CMT channel to read (0-3) Valid range: k_cmt_channel_0 through k_cmt_channel_3
out	count	Pointer to store the 16-bit counter value Valid range: Non-nullptr to uint16_t On success: Contains CMCNT value (0 to CMCOR) Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success; *count contains current CMCNT value
k_rx_err_null_ptr	count is nullptr
k_rx_err_invalid_state	Channel not initialized or invalid channel
k_rx_err_invalid_arg	Could not obtain register base pointer

Pre: Channel must be initialized via rx_cmt_init()

Pre: count must point to valid uint16_t storage

Post: *count set to CMCNT register value at time of read

Post: Counter continues running (read is non-destructive)

Note: Value may change between consecutive reads if counter is running

Note: For precise time measurements disable interrupts around consecutive reads

Thread Safety:: Read-only; safe to call from multiple contexts, but value may be stale.

Example:: uint16_ticks=0; rx_err_terr=rx_cmt_get_count(k_cmt_channel_1,&ticks);
if(err!=k_rx_ok){ }

See also: rx_cmt_init() Initialize channel and set frequency, rx_cmt_start() Start the counter

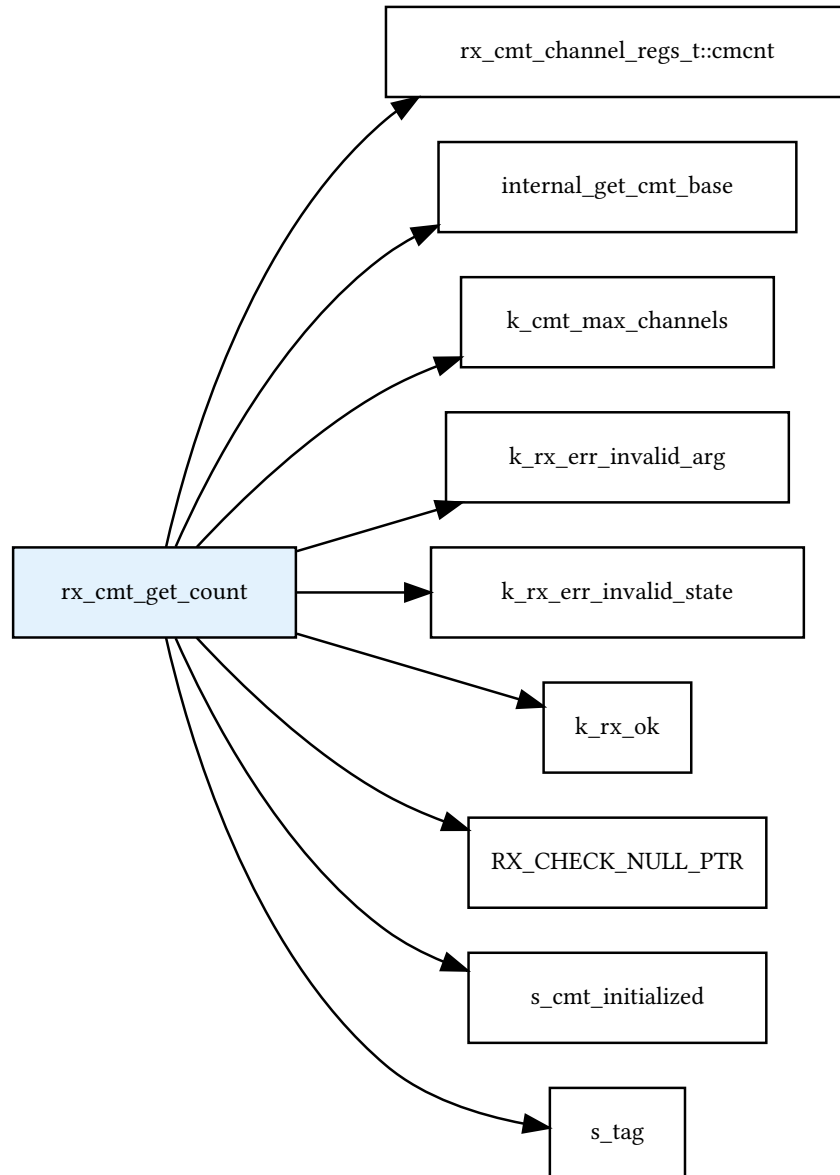


Figure 790: Call graph for rx_cmt_get_count

Defined in `libs/rx_hal/src/rx_cmt.c:1406`

Since Version 1.0.0

rx_cmt_deinit

```
rx_err_t rx_cmt_deinit(const rx_cmt_channel_t channel)
```

Deinitialize CMT channel and release all resources.

Deinitialize CMT channel.

Stops the CMT counter, disables the ICU interrupt for the channel, clears the stored callback and user data, and marks the channel as uninitialized. After this call the channel may be re-initialized with rx_cmt_init().

Algorithm steps:

1. Validate channel range
2. Stop timer via rx_cmt_stop() and propagate any error
3. Compute ICU vector, IER index, and IER bit for the channel
4. Clear the IER bit to disable the interrupt
5. Clear callback, user_data, and initialized flag

Parameters:

Direction	Name	Description
in	channel	CMT channel to deinitialize (0-3) Valid range: k_cmt_channel_0 through k_cmt_channel_3

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success; channel fully deinitialized
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_hw_error	Timer stop verification failed

Pre: channel must be in range 0, 3

Pre: Caller should ensure no other task is using the channel callback

Post: Counter halted (CMSTR bit cleared)

Post: ICU interrupt disabled for channel

Post: s_cmt_initializedchannel set to false

Post: s_cmt_callbackchannel and s_cmt_user_datachannel set to nullptr

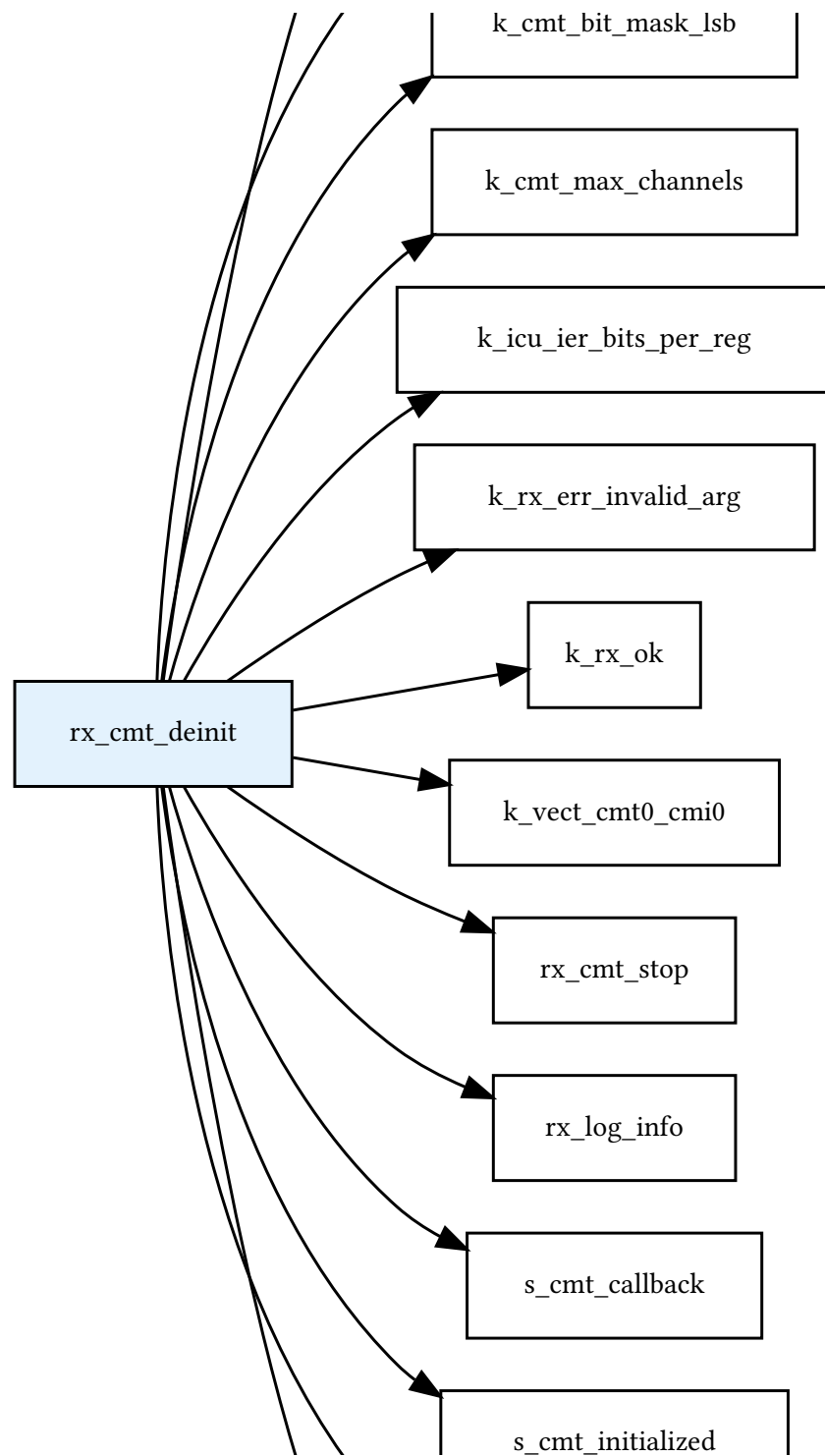
Note: Safe to call on an uninitialized channel (rx_cmt_stop tolerates it)

Note: Module clock is NOT re-asserted - hardware remains clocked for re-use

Thread Safety:: Not thread-safe. Ensure no ISR or other thread is using the channel.

Example:: `rx_err_t err=rx_cmt_deinit(k_cmt_channel_1); if(err!=k_rx_ok)
{ rx_log_error("APP","CMTdeinitfailed"); }`

See also: `rx_cmt_init()` Re-initialize after deinit, `rx_cmt_stop()` Stop without full deinit

Figure 791: Call graph for `rx_cmt_deinit`

Defined in `libs/rx_hal/src/rx_cmt.c:1476`

Since Version 1.0.0

—

rx_gptw.c

GPTW PWM Driver Implementation for Motor Control.

Production-grade implementation of the General PWM Timer (GPTW) driver for RX72N motor control applications. This file contains all internal helper functions and public API implementations for GPTW peripheral management.

Includes:

- `"rx_gptw.h"`
- `<stddef.h>`
- `"rx72n_regs.h"`
- `"rx_check.h"`
- `"rx_log.h"`
- `"rx_register_protection.h"`

gptw_constants_t

GPTW general constants.

Value	Name	Description
= 4	<code>k_gptw_max_channels</code>	GPTW0-GPTW3
= 2	<code>k_gptw_outputs_per_channel</code>	GTIOCA and GTIOCB

—

gptw_period_constants_t

Value	Name	Description
= 1	<code>k_gptw_period_divisor_saw</code>	Sawtooth: period = PCLKA / freq
= 2	<code>k_gptw_period_divisor_tri</code>	Triangle: period = PCLKA / (2 * freq)
= 10	<code>k_gptw_period_min</code>	Minimum valid period
= 0	<code>k_gptw_period_zero</code>	Zero period value
= 0	<code>k_gptw_deadtime_disabled</code>	No dead time

—

gptw_duty_constants_t

Duty cycle calculation constants.

Value	Name	Description
= 0	<code>k_gptw_duty_min</code>	Minimum duty cycle (0%)
= 100	<code>k_gptw_duty_max</code>	Maximum duty cycle (100%)
= 100	<code>k_gptw_duty_divisor</code>	Divisor for percentage conversion

—

mpc_pwpr_constants_t

MPC configuration constants.

Value	Name	Description
= 0x00	k_mpc_pwpr_b0wi_clear	Clear B0WI to enable PFSWE write
= 0x40	k_mpc_pwpr_pfswe_set	Set PFSWE to enable PFS write
= 0x80	k_mpc_pwpr_lock	Set B0WI to lock PFS

—

mpc_pfs_offset_constants_t

MPC PFS register offset constants.

Value	Name	Description
= 0x40	k_mpc_pfs_base_offset	Base offset for PFS registers
= 0x0E	k_mpc_port_e_index	Port E index
= 8	k_mpc_pfs_port_stride	Bytes between port PFS groups

—

gptw_bit_constants_t

Single-bit value for shift operations.

Value	Name	Description
= 1	k_gptw_bit_set	Value for single bit in shift operations

—

gptw_channel_mask_t

GPTW channel mask for start/stop operations.

Value	Name	Description
= 0x0F	k_gptw_all_channels_mask	Bitmask for all 4 GPTW channels (Ch0-3)

—

gptw_direction_constants_t

GPTW count direction values for GTUDDTYC register.

Value	Name	Description
= 0	k_gptw_gtuddtyc_ud_down	Count direction down (UD bit = 0)

—

porte_gptw_pin_t

Port E pin indices for GPTW motor outputs.

Value	Name	Description
-------	------	-------------

= 5	k_porte_gptw0_a	PE5/GTIOC0A - Motor 0 phase
= 2	k_porte_gptw0_b	PE2/GTIOC0B - Motor 0 enable
= 4	k_porte_gptw1_a	PE4/GTIOC1A - Motor 1 phase
= 1	k_porte_gptw1_b	PE1/GTIOC1B - Motor 1 enable
= 3	k_porte_gptw2_a	PE3/GTIOC2A - Motor 2 phase
= 0	k_porte_gptw2_b	PE0/GTIOC2B - Motor 2 enable
= 7	k_porte_gptw3_a	PE7/GTIOC3A - Motor 3 phase
= 6	k_porte_gptw3_b	PE6/GTIOC3B - Motor 3 enable

—

gptw_phase_divisor_t

Phase divisors for staggered PWM initialization.

Divisor constants used to calculate the initial GTCNT counter value for each GPTW channel during staggered initialization. Each channel is offset by 90 degrees from the previous one to spread switching edges across the PWM period.

For a given period P the offsets are:

- Ch0: 0 (0/4 of P)
- Ch1: P / 4 (quarter divisor)
- Ch2: P / 2 (half divisor)
- Ch3: 3 * P / 4 (three-quarter: multiply by 3, then divide by 4)

All divisor values are ≥ 1 , preventing division-by-zero in `internal_calculate_phase_offset()`

Value	Name	Description
= 1	k_phase_divisor_none	0 deg: no offset (safe default, never causes div-by-zero)
= 4	k_phase_divisor_quarter	90 deg = period/4
= 2	k_phase_divisor_half	180 deg = period/2
= 4	k_phase_divisor_three_quarter	270 deg = 3*period/4 (multiplied by 3 then divided)

See also: `internal_calculate_phase_offset()` Uses these divisors for GTCNT seeding, `rx_gptw_init_all_staggered()` Top-level staggered initialization function

Example:

```
//Example:derive90-degreeoffsetforaperiodof3000counts
uint32_toffset_ch1=3000U/k_phase_divisor_quarter;//==750
uint32_toffset_ch3=(3000U*3U)/k_phase_divisor_three_quarter;//==2250
```

—

s_tag

```
const char* s_tag
```

—

s_gptw_period_max

```
const uint32_t s_gptw_period_max
```

Period calculation constants.

—

s_gptw_ns_per_second

```
const uint64_t s_gptw_ns_per_second
```

—

s_gptw_initialized

```
bool s_gptw_initialized[k_gptw_max_channels]
```

Track initialized channels.

—

s_gptw_period

```
uint32_t s_gptw_period[k_gptw_max_channels]
```

Period values for each channel.

—

internal_get_gptw_base

```
static volatile rx_gptw_channel_regs_t * internal_get_gptw_base(const  
rx_gptw_channel_t channel)
```

Get GPTW channel register base address.

Maps a GPTW channel enumeration value to its corresponding hardware register base address. Each GPTW channel (0-3) has a dedicated register block at a fixed memory offset.

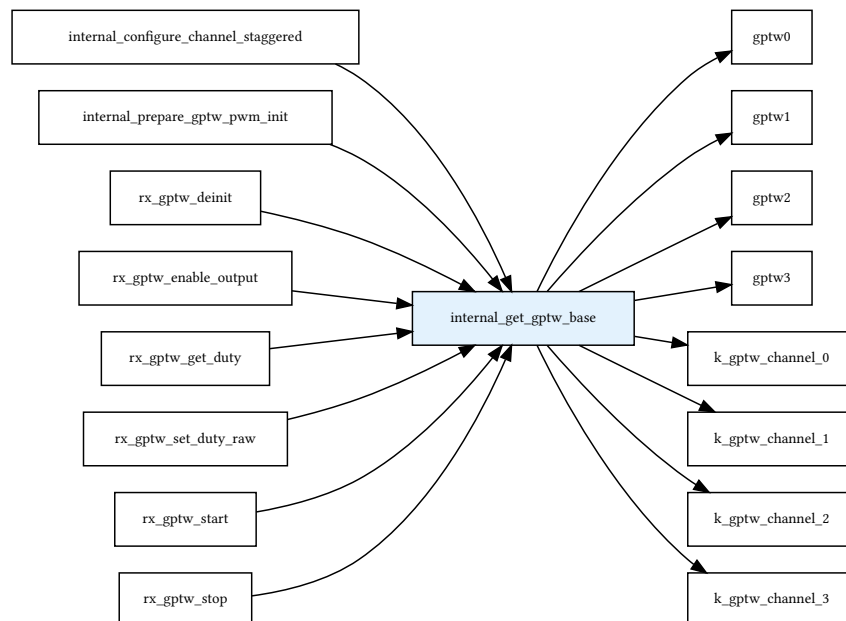


Figure 792: Call graph for `internal_get_gptw_base`

Defined in *libs/rx_hal/src/rx_gptw.c:252*

—

internal_is_triangle_mode

```
static bool internal_is_triangle_mode(const rx_gptw_wave_mode_t mode)
```

Check if wave mode is triangle (center-aligned).

Determines whether the specified waveform mode uses triangle wave (center-aligned) PWM generation. Triangle modes count up to period, then down to zero, resulting in symmetric PWM pulses centered within each period.

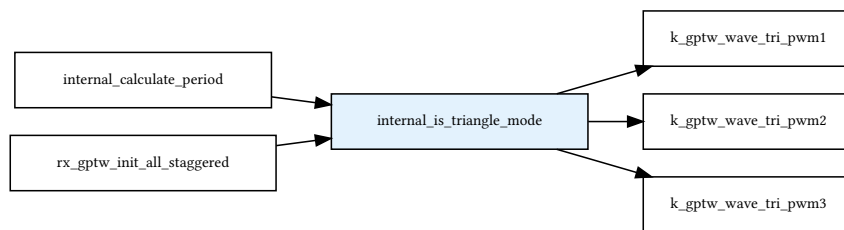


Figure 793: Call graph for `internal_is_triangle_mode`

Defined in *libs/rx_hal/src/rx_gptw.c:310*

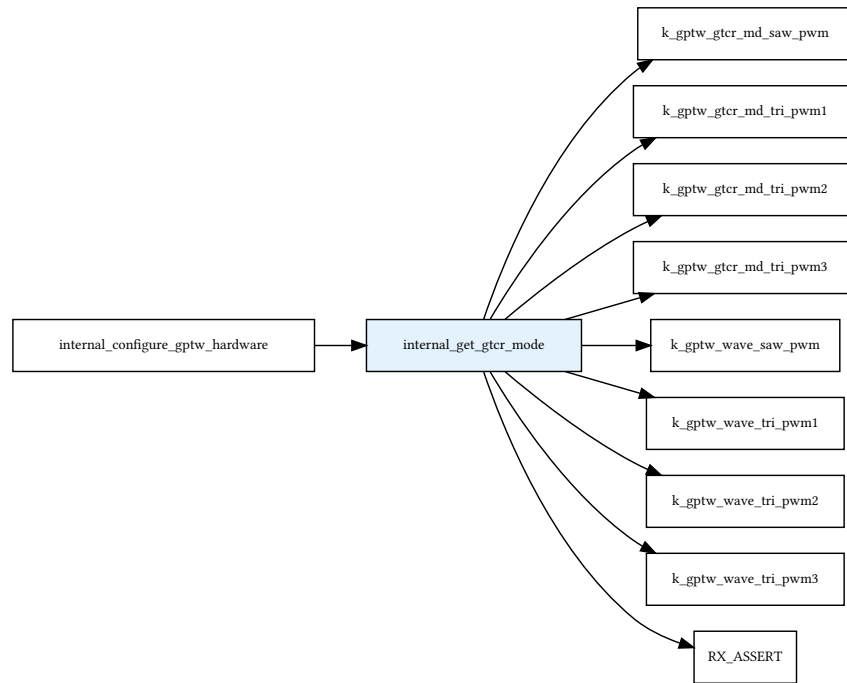
—

internal_get_gtcr_mode

```
static uint32_t internal_get_gtcr_mode(const rx_gptw_wave_mode_t mode)
```

Get GTCR mode bits for wave mode.

Converts a waveform mode enumeration to the corresponding GTCR.MD register bit field value. The MD field (bits 18:16) in GTCR controls the timer's counting mode and PWM generation behavior.

Figure 794: Call graph for `internal_get_gtcr_mode`

Defined in `libs/rx_hal/src/rx_gptw.c:369`

internal_calculate_period

```
static rx_err_t internal_calculate_period(const uint32_t frequency_hz, const
rx_gptw_wave_mode_t wave_mode, uint32_t *period)
```

Calculate period register value from frequency.

Converts a desired PWM frequency to the GTPR (period register) count value. The calculation differs based on waveform mode:

Figure 795: Call graph for `internal_calculate_period`

Defined in `libs/rx_hal/src/rx_gptw.c:473`

—

internal_configure_mpc

```
static rx_err_t internal_configure_mpc(const rx_gptw_channel_t channel)
```

Configure MPC for GPTW output pins.

Configures the Multi-Function Pin Controller (MPC) to route GPTW timer outputs to the appropriate Port E pins. Each GPTW channel uses two pins for complementary PWM outputs (GTIOCA and GTIOCB).



Figure 796: Call graph for `internal_configure_mpc`

Defined in `libs/rx_hal/src/rx_gptw.c:580`

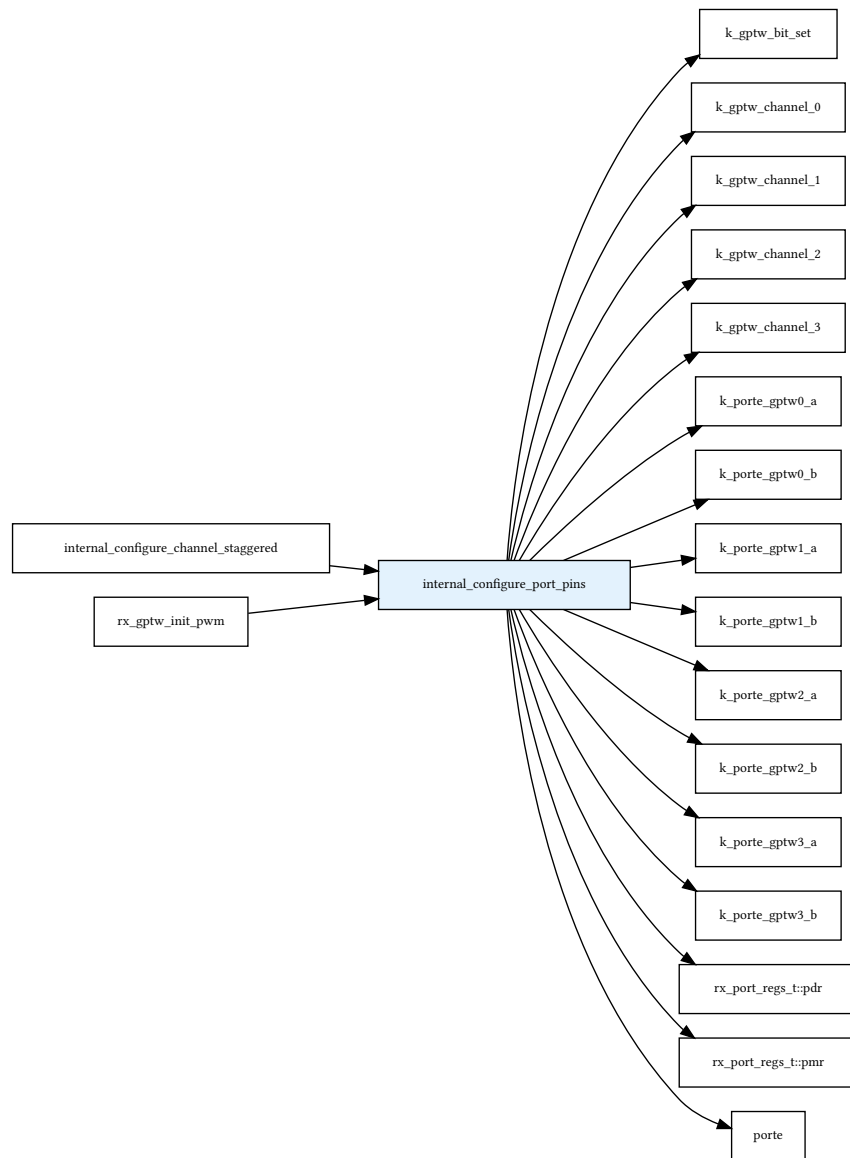
—

internal_configure_port_pins

```
static void internal_configure_port_pins(const rx_gptw_channel_t channel)
```

Configure Port E pins as peripheral outputs.

Configures the Port E GPIO pins for GPTW peripheral output mode. Sets both the Peripheral Mode Register (PMR) and Port Direction Register (PDR) to enable GPTW timer outputs.

Figure 797: Call graph for `internal_configure_port_pins`

Defined in `libs/rx_hal/src/rx_gptw.c:681`

`internal_enable_gptw_module_clock`

```
static void internal_enable_gptw_module_clock(void)
```

Enable GPTW module clock.

Enables the GPTW peripheral by clearing its module stop bit in MSTPCRC. The GPTW module is controlled by MSTPCRC.MSTPC10 (bit 10). When set, the module clock is stopped and registers are inaccessible.

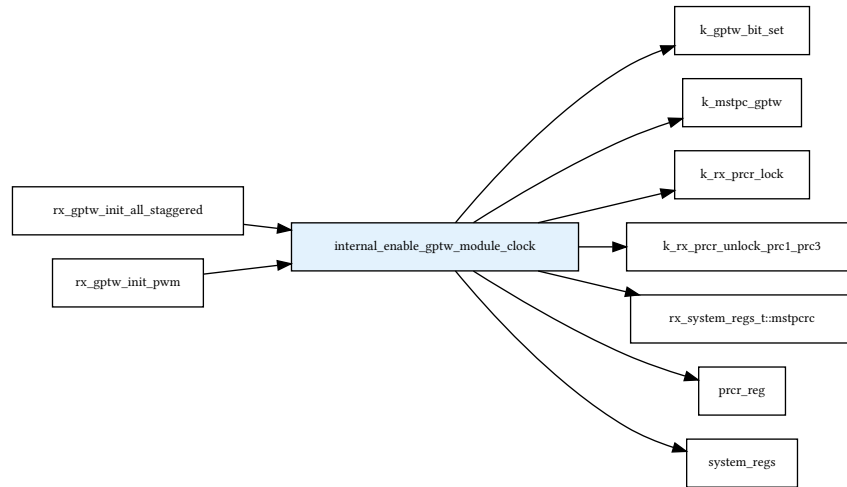


Figure 798: Call graph for `internal_enable_gptw_module_clock`

Defined in `libs/rx_hal/src/rx_gptw.c:753`

internal_configure_gptw_hardware

```
static rx_err_t internal_configure_gptw_hardware(volatile rx_gptw_channel_regs_t
*gptw, const rx_gptw_config_t *config, const uint32_t period)
```

Configure GPTW hardware registers.

Performs the core hardware configuration of a GPTW channel. This function writes all necessary registers to set up PWM operation including control mode, I/O configuration, period, duty cycle initialization, buffer mode, and deadline.

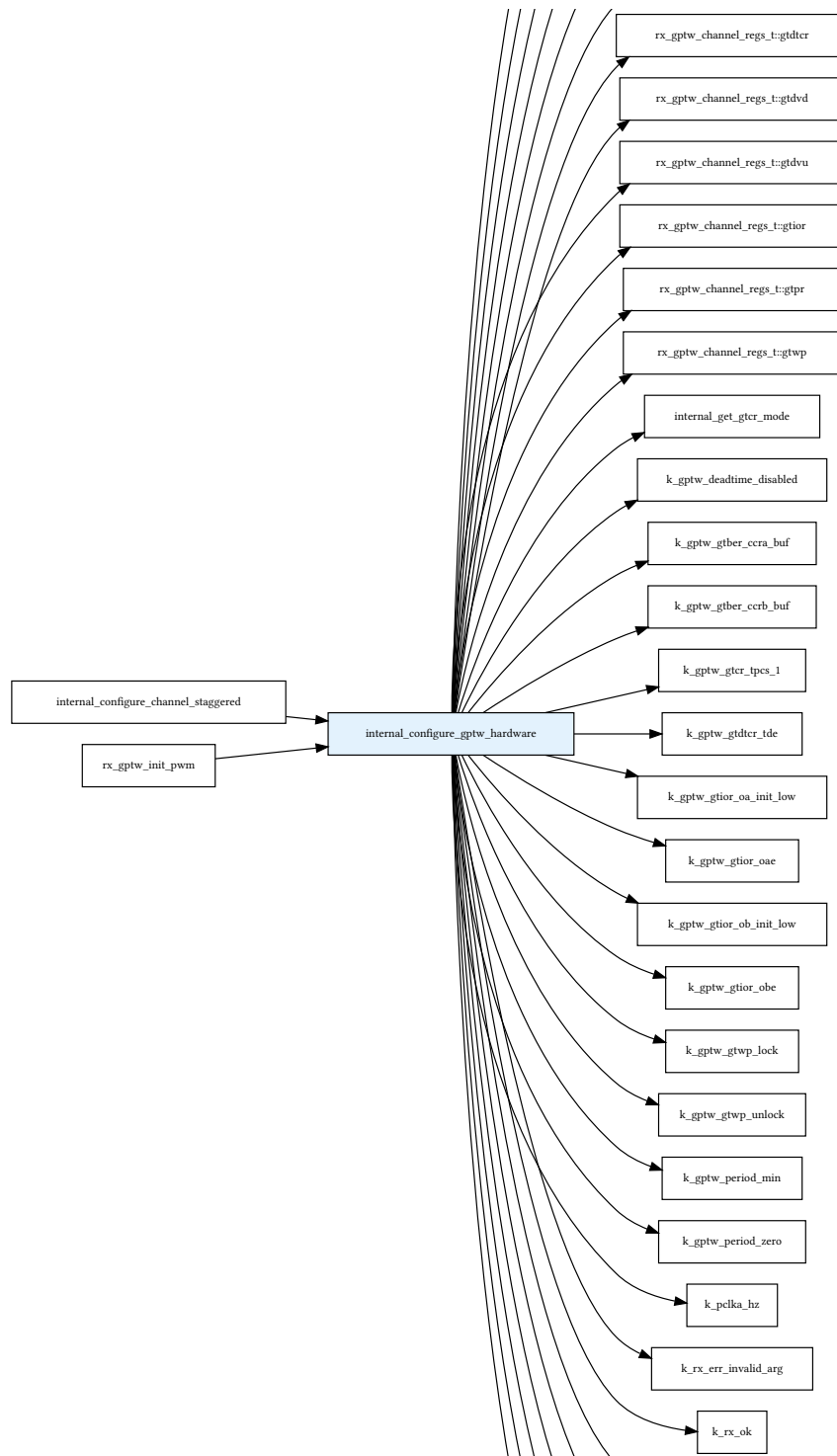


Figure 799: Call graph for `internal_configure_gptw_hardware`

Defined in `libs/rx_hal/src/rx_gptw.c:838`

—

internal_prepare_gptw_pwm_init

```
static rx_err_t internal_prepare_gptw_pwm_init(const rx_gptw_channel_t channel,
const rx_gptw_config_t *config, volatile rx_gptw_channel_regs_t **gptw_out,
uint32_t *period_out)
```

Validate channel and compute period for PWM initialization.

Shared pre-initialization helper used by `rx_gptw_init_pwm()`. Performs two actions in a single function to reduce code duplication:

1. Validate channel range and resolve register base pointer
2. Calculate the GTPR period value from frequency and wave mode

Parameters:

Direction	Name	Description
in	channel	GPTW channel to initialize Valid range: <code>k_gptw_channel_0</code> to <code>k_gptw_channel_3</code>
in	config	Configuration structure (caller guarantees non-NULL) <code>config->frequency_hz</code> and <code>config->wave_mode</code> are used
out	gptw_out	Pointer-to-pointer to receive the channel register base Must be non-NULL; <code>*gptw_out</code> set on success
out	period_out	Pointer to receive the calculated period count Must be non-NULL; <code>*period_out</code> set on success

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Validation and period calculation succeeded
<code>k_rx_err_invalid_arg</code>	channel out of range or register base unresolvable
<code>k_rx_err_invalid_arg</code>	frequency_hz is zero, wave_mode invalid, or period out of range

Pre: `gptw_out` and `period_out` must be non-NULL (caller's responsibility)

Pre: `config` must be non-NULL (caller's responsibility)

Post: `*gptw_out` points to valid GPTW channel registers on success

Post: `*period_out` contains a value in `k_gptw_period_min`, `s_gptw_period_max` on success

Note: Pure helper: does not modify hardware registers or module state

Note: Not thread-safe: relies on config content stability during call] **See also:**
 internal_calculate_period() Period calculation details, rx_gptw_init_pwm() Primary caller

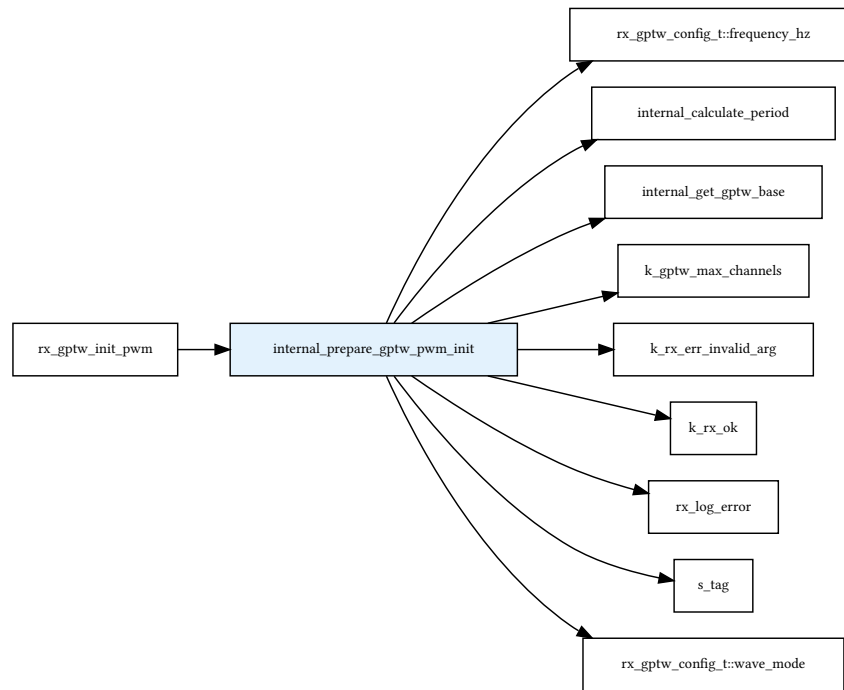


Figure 800: Call graph for `internal_prepare_gptw_pwm_init`

Defined in `libs/rx_hal/src/rx_gptw.c:935`

Since Version 1.0.0

rx_gptw_init_pwm

```
rx_err_t rx_gptw_init_pwm(const rx_gptw_channel_t channel, const rx_gptw_config_t
*config)
```

Initialize a single GPTW channel for PWM operation.

Initialize a single GPTW channel for PWM output.

Configures one GPTW channel for PWM generation. This function performs complete hardware setup including module clock enable, MPC configuration, port pin setup, timer configuration, and automatic timer start. For 4-motor systems, prefer `rx_gptw_init_all_staggered()` instead.

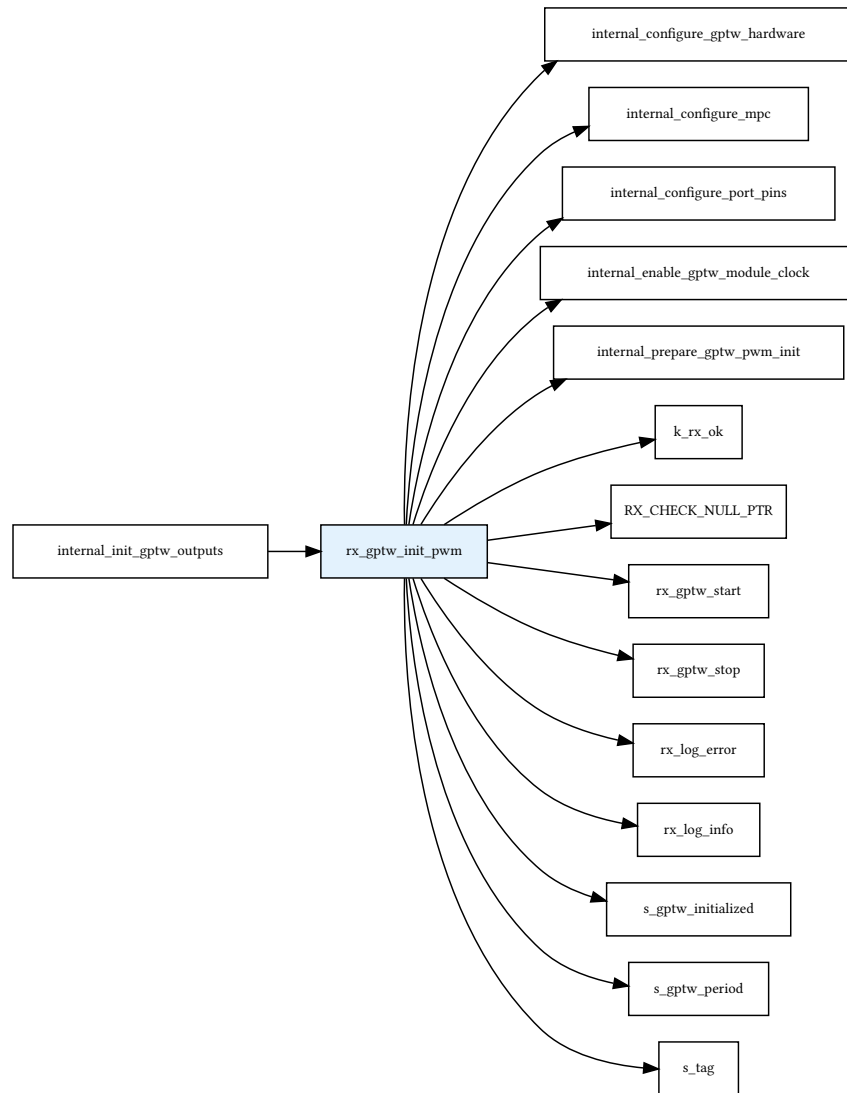


Figure 801: Call graph for rx_gptw_init_pwm

Defined in `libs/rx_hal/src/rx_gptw.c:1048`

—

rx_gptw_set_duty

```
rx_err_t rx_gptw_set_duty(const rx_gptw_channel_id_t channel, const
rx_gptw_output_id_t output, const float duty_percent)
```

Set PWM duty cycle using floating-point percentage.

Set PWM duty cycle (floating-point percentage).

Converts floating-point duty percentage [0.0, 100.0] to raw count value and delegates to `rx_gptw_set_duty_raw()` for hardware register update. The compare register (GTCCRA or GTCCRB) is written and the change takes effect on the next PWM period boundary (glitch-free via buffer mode).

Parameters:

Direction	Name	Description
in	channel	Typed channel identifier (use <code>rx_gptw_channel_id()</code>)
in	output	Typed output identifier (use <code>rx_gptw_output_id()</code>)
in	duty_percent	Duty cycle [0.0, 100.0] percent 0.0 = always-off, 100.0 = always-on

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Duty cycle updated successfully
<code>k_rx_err_invalid_state</code>	Channel out of range or not initialized
<code>k_rx_err_invalid_arg</code>	<code>duty_percent</code> outside [0.0, 100.0] or invalid output

Pre: Channel must be initialized via `rx_gptw_init_*`()

Pre: `duty_percent` must be in 0.0, 100.0

Post: Compare register written; change takes effect next period boundary

Post: Motor output drive updated on next PWM cycle

Note: Not thread-safe: concurrent writes to same output cause undefined behavior

Algorithm:: `duty_count = leftlfloor {duty_percent x period}{100 rightrfloor`

Example:

```
rx_gptw_set_duty(rx_gptw_channel_id(k_gptw_channel_0),
rx_gptw_output_id(k_gptw_output_a), 75.0f);
```

See also: `rx_gptw_set_duty_raw()` Set duty using raw count value, `rx_gptw_get_duty()` Read current duty cycle

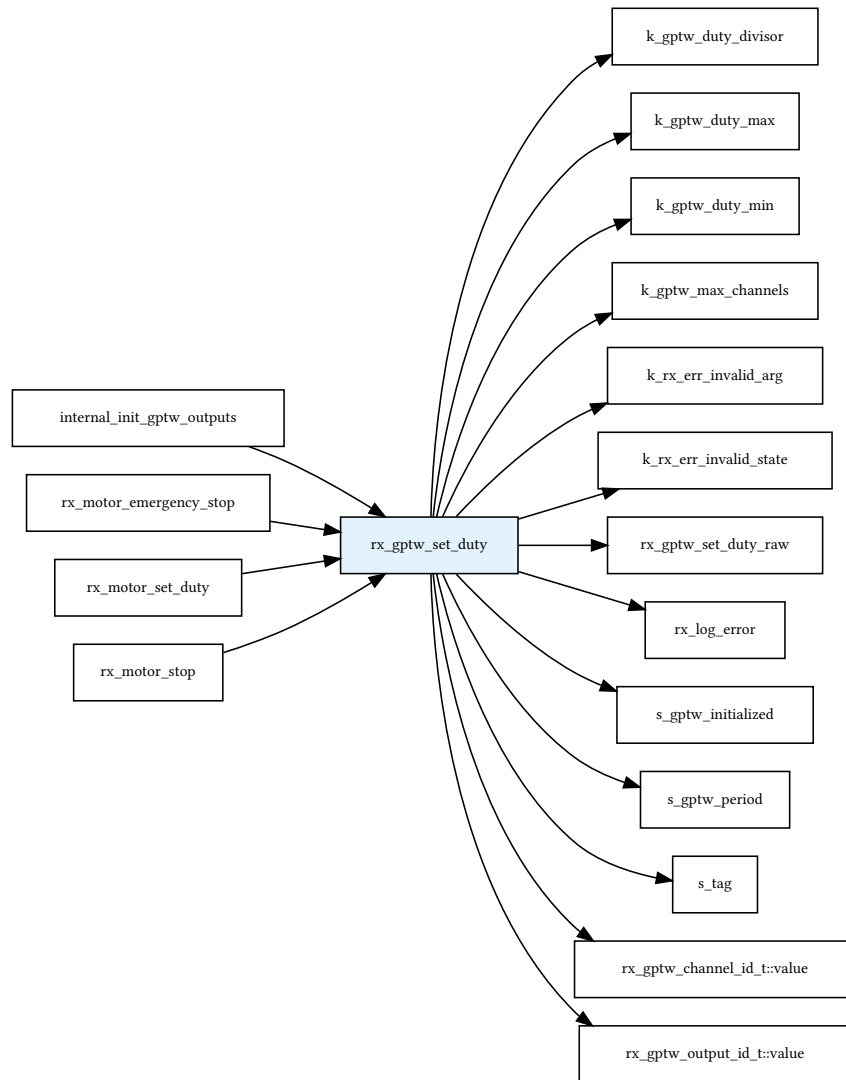


Figure 802: Call graph for `rx_gptw_set_duty`

Defined in `libs/rx_hal/src/rx_gptw.c:1146`

Since Version 1.0.0

—

`rx_gptw_set_duty_raw`

```
rx_err_t rx_gptw_set_duty_raw(const rx_gptw_channel_t channel, rx_gptw_output_t
output, uint32_t duty_count)
```

Set PWM duty cycle using a raw 32-bit timer count.

Set PWM duty cycle using raw counter value.

Directly writes duty_count to the compare register (GTCCRA or GTCCRB). Buffer mode (enabled during init via GTBER) ensures the update takes effect on the next PWM period boundary without glitches. Values exceeding the period are clamped to period (100% duty).

Parameters:

Direction	Name	Description
in	channel	GPTW channel (k_gptw_channel_0 to k_gptw_channel_3)
in	output	Output to update (k_gptw_output_a or k_gptw_output_b)
in	duty_count	Raw count [0, period]; values > period clamped to period

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Duty count written successfully
k_rx_err_invalid_state	Channel out of range or not initialized
k_rx_err_invalid_arg	Could not resolve register base or invalid output

Pre: Channel must be initialized via rx_gptw_init_*)

Pre: duty_count should be in 0, period for meaningful duty; higher values clamped

Post: GTCCRA or GTCCRB written with clamped duty_count

Post: Change takes effect at next PWM period boundary

Note: Not thread-safe: concurrent writes to the same register cause undefined behavior

Note: Values exceeding period are silently clamped to period

Register Access:: Output A: Writes to GTCCRA Output B: Writes to GTCCRB

Example:

```
uint32_t period;
rx_gptw_get_period(k_gptw_channel_0, &period);
rx_gptw_set_duty_raw(k_gptw_channel_0, k_gptw_output_a, period/2U); //50%
```

See also: rx_gptw_set_duty() Set duty using floating-point percentage,
rx_gptw_get_period() Retrieve period count for duty calculation

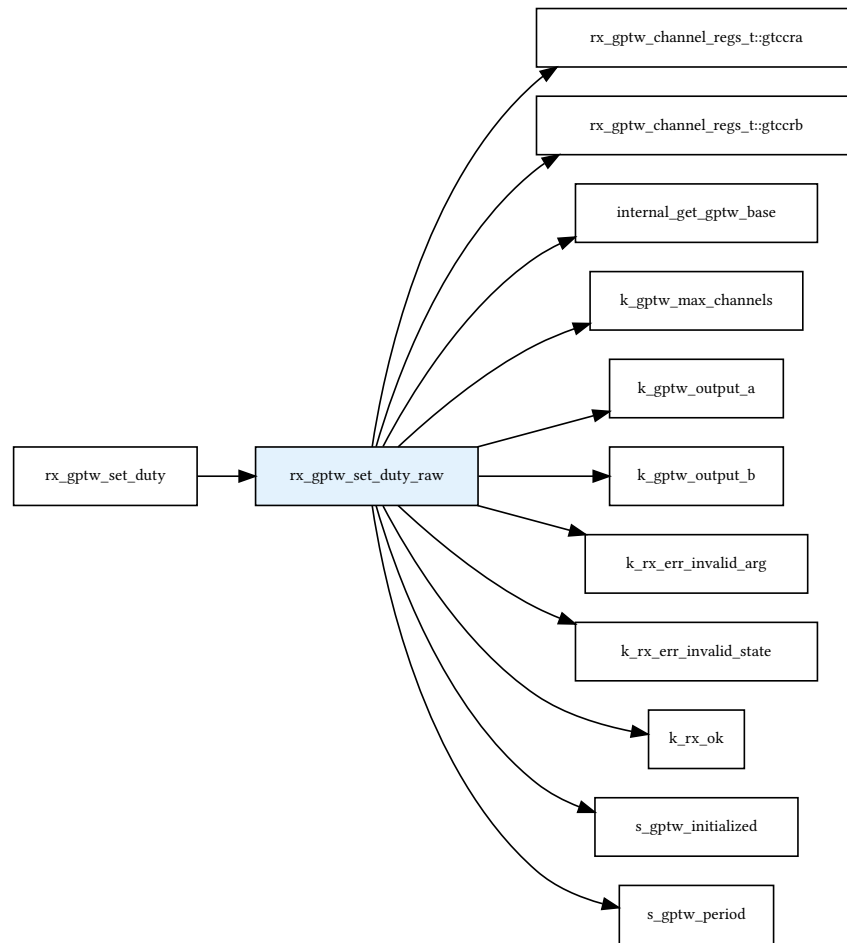


Figure 803: Call graph for rx_gptw_set_duty_raw

Defined in `libs/rx_hal/src/rx_gptw.c:1215`

Since Version 1.0.0

—

rx_gptw_get_duty

```
rx_err_t rx_gptw_get_duty(const rx_gptw_channel_t channel, const rx_gptw_output_t
output, float *duty_percent)
```

Read the current PWM duty cycle as a floating-point percentage.

Get current duty cycle as a floating-point percentage.

Reads the current compare register value (GTCCRA or GTCCRB) from hardware and converts it to a percentage using the cached period from `s_gptw_period[]`.

Parameters:

Direction	Name	Description
in	channel	GPTW channel (k_gptw_channel_0 to k_gptw_channel_3)
in	output	Output to read (k_gptw_output_a or k_gptw_output_b)
out	duty_percent	Pointer to store percentage result [0.0, 100.0] Must be non-NULL

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Percentage written to *duty_percent
k_rx_err_null_ptr	duty_percent is nullptr
k_rx_err_invalid_state	Channel out of range or not initialized
k_rx_err_invalid_arg	Invalid output or register pointer unresolvable

Pre: Channel must be initialized via rx_gptw_init_*

Pre: duty_percent must point to valid float storage

Post: *duty_percent contains duty cycle in 0.0, 100.0 on success

Post: Hardware registers unchanged

Note: Returns actual register value; may differ from the last set value if buffer transfer has not yet occurred at period boundary

Note: Not thread-safe: concurrent rx_gptw_set_duty calls may cause torn reads

Algorithm:: duty_percent = {duty_count x 100/period

Example:

```
float duty;
rx_err_t err = rx_gptw_get_duty(k_gptw_channel_0, k_gptw_output_a, &duty);
```

See also: rx_gptw_set_duty() Set duty using floating-point percentage,
rx_gptw_get_period() Retrieve period count

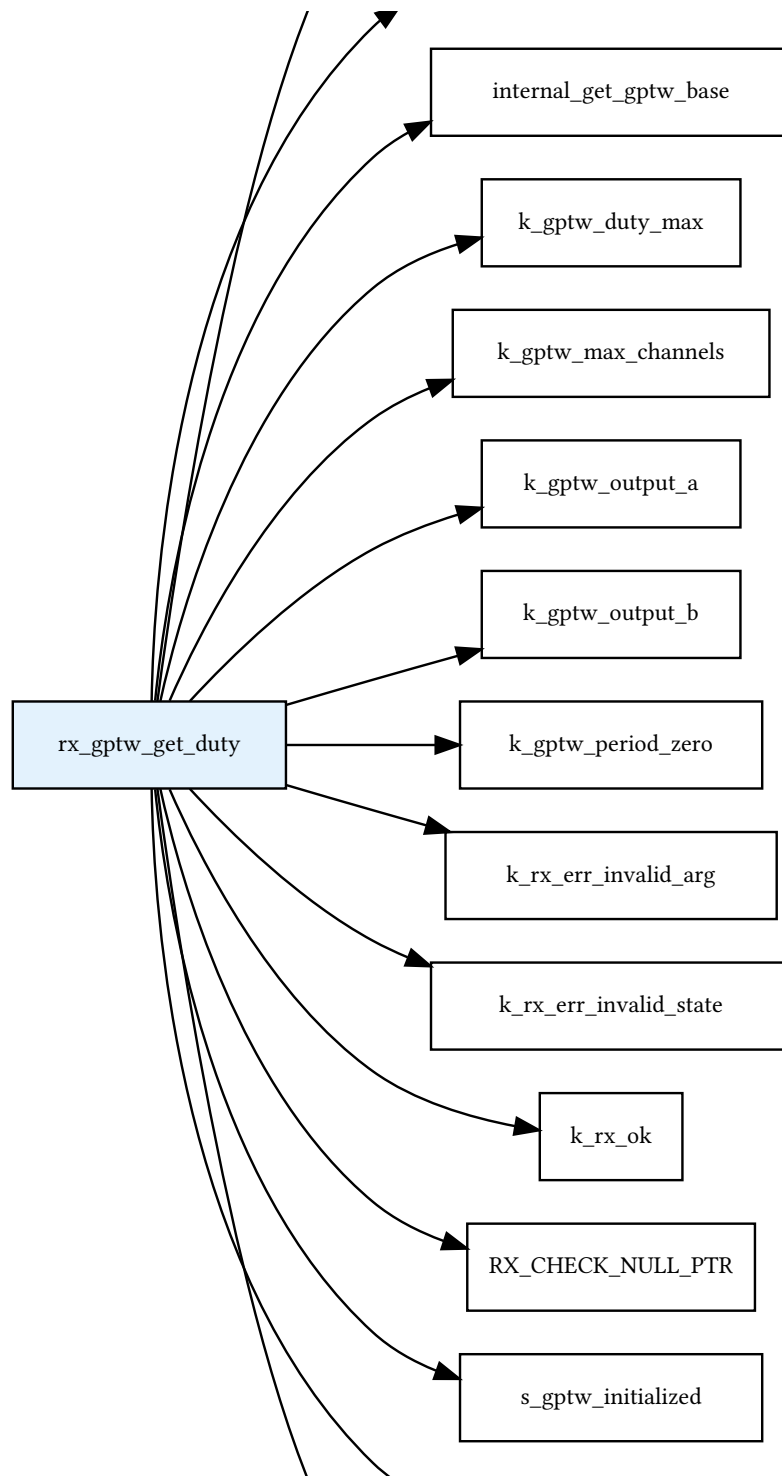


Figure 804: Call graph for rx_gptw_get_duty

Defined in `libs/rx_hal/src/rx_gptw.c:1290`

Since Version 1.0.0

—

rx_gptw_get_period

```
rx_err_t rx_gptw_get_period(const rx_gptw_channel_t channel, uint32_t
*period_count)
```

Get PWM period count for GPTW channel.

Get the 32-bit PWM period count for a GPTW channel.

Implementation of `rx_gptw_get_period()` - see `rx_gptw.h` for complete documentation.

Returns the cached period value from `s_gptw_period[]`, which matches the GTPR register value set during initialization.

Parameters:

Direction	Name	Description
in	channel	GPTW channel to query Valid range: <code>k_gptw_channel_0</code> to <code>k_gptw_channel_3</code>
out	period_count	Pointer to store period count value Must be non-NULL

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, <code>period_count</code> contains valid value
<code>k_rx_err_null_ptr</code>	<code>period_count</code> is nullptr
<code>k_rx_err_invalid_state</code>	Channel not initialized

Pre: Channel initialized via `rx_gptw_init_*`()

Pre: `period_count` points to valid `uint32_t` storage

Post: `*period_count` contains period value on success

Note: Thread-safe: Reads from cached value, not hardware

Note: Useful for calculating raw duty counts from percentages] **See also:** `rx_gptw.h` Complete API documentation

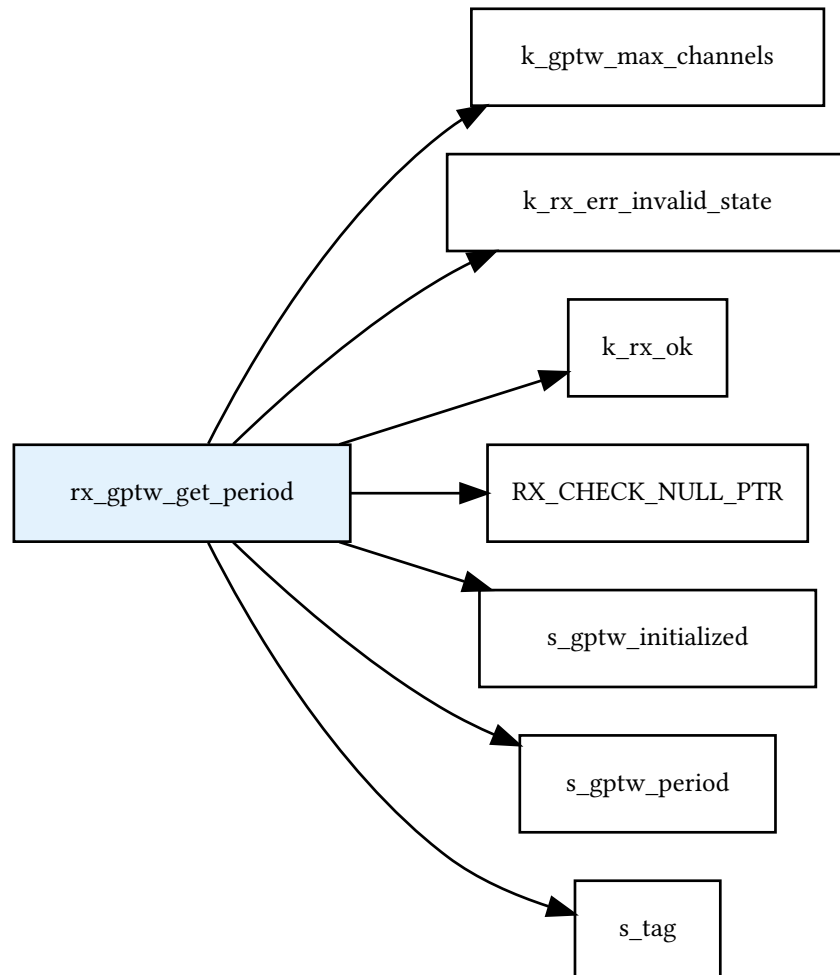


Figure 805: Call graph for rx_gptw_get_period

Defined in `libs/rx_hal/src/rx_gptw.c:1353`

—

rx_gptw_enable_output

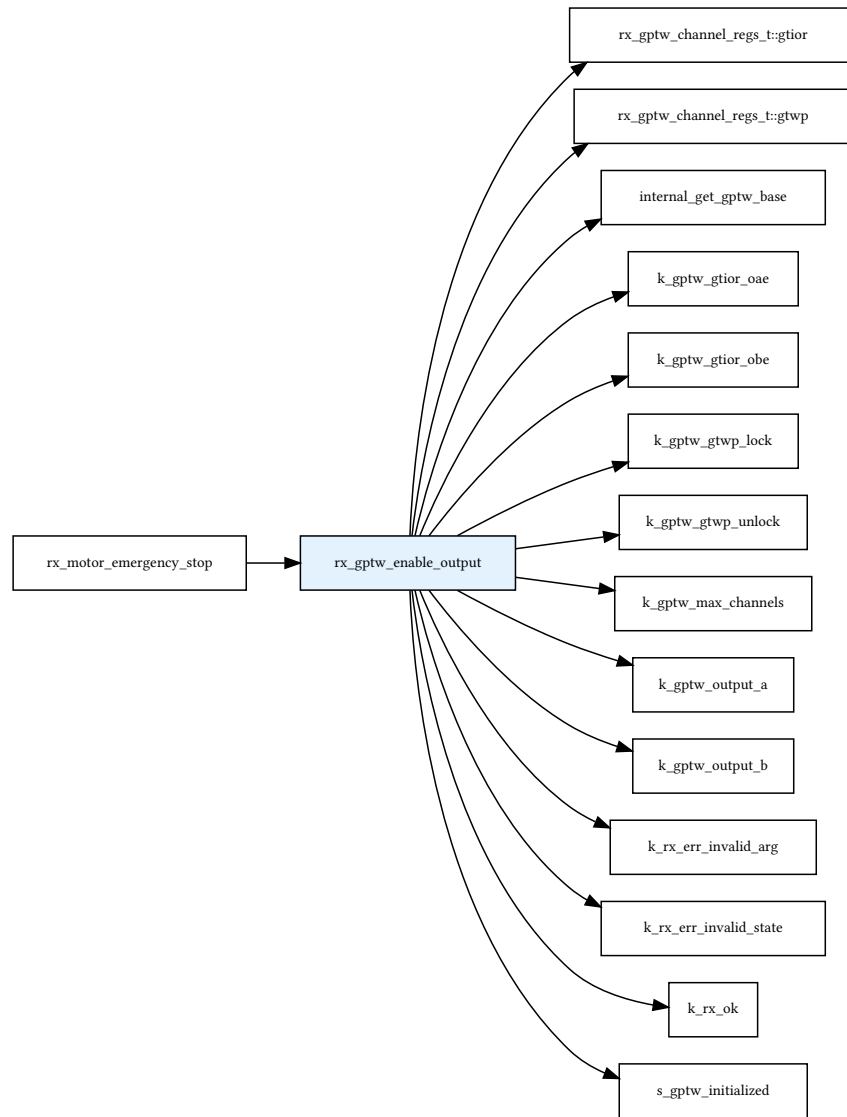
```
rx_err_t rx_gptw_enable_output(const rx_gptw_channel_t channel, const
rx_gptw_output_t output, bool enable)
```

Enable or disable GPTW output pin.

Enable or disable a PWM output pin.

Implementation of rx_gptw_enable_output() - see rx_gptw.h for complete documentation.

Controls the Output Enable bits (OAE/OBE) in the GTIOR register. When disabled, the corresponding output pin is held in its initial state (typically LOW).

Figure 806: Call graph for `rx_gptw_enable_output`

Defined in `libs/rx_hal/src/rx_gptw.c:1403`

rx_gptw_start

```
rx_err_t rx_gptw_start(const rx_gptw_channel_t channel)
```

Start GPTW counter (begin PWM generation).

Start the GPTW timer counter for PWM generation.

Implementation of `rx_gptw_start()` - see `rx_gptw.h` for complete documentation.

Sets the Count Start (CST) bit in GTCR to begin counter operation. The timer will count according to the configured waveform mode (sawtooth or triangle) and generate PWM output on the enabled pins.

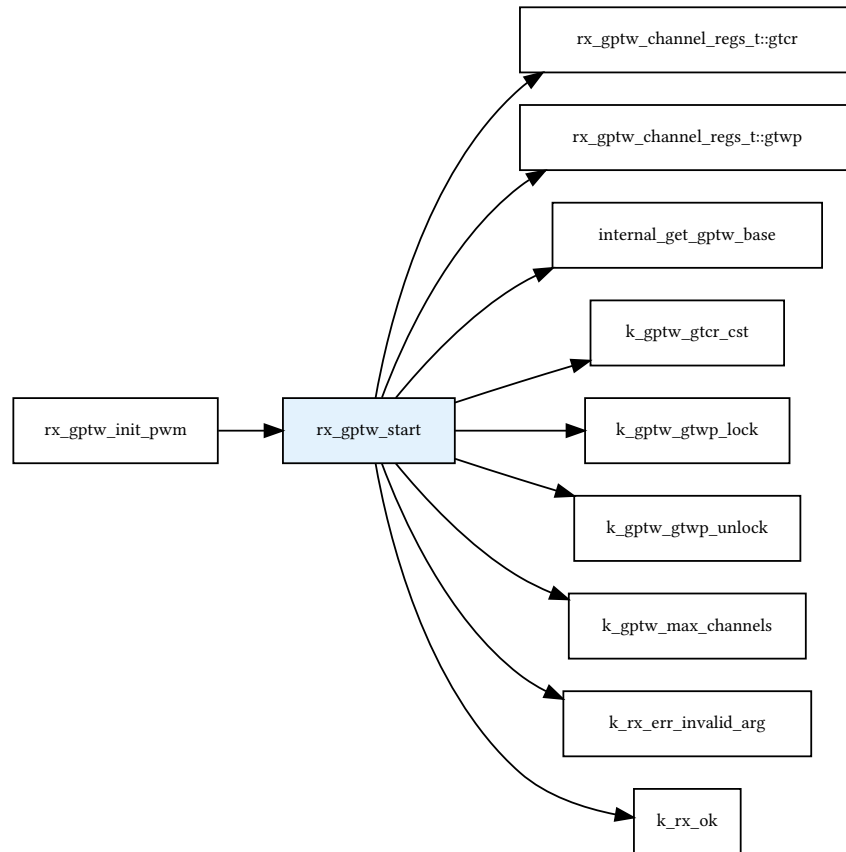


Figure 807: Call graph for `rx_gptw_start`

Defined in `libs/rx_hal/src/rx_gptw.c:1478`

rx_gptw_stop

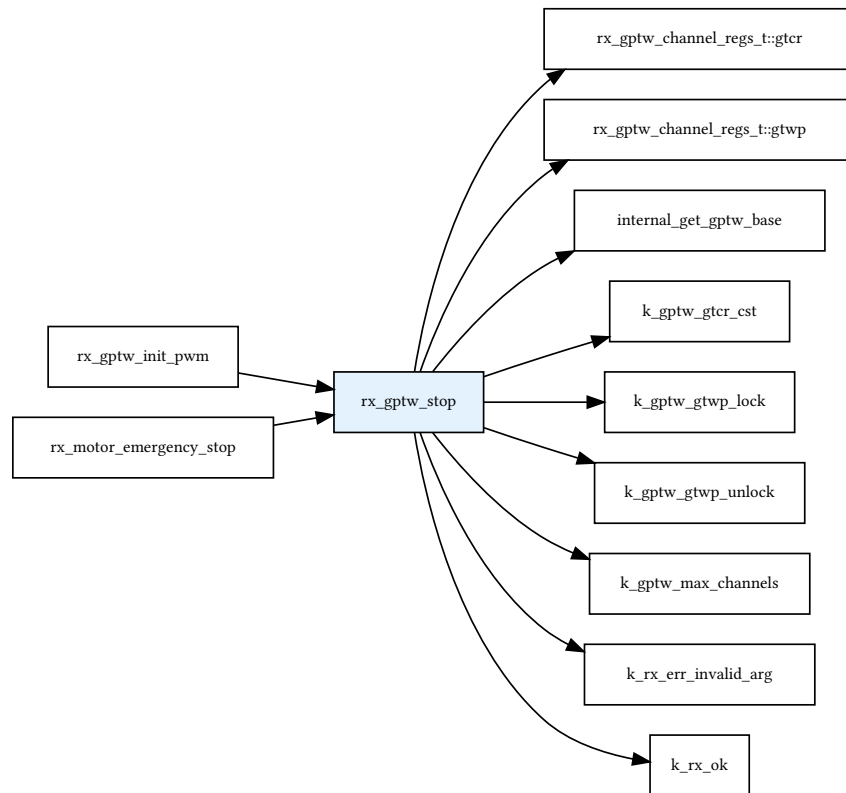
```
rx_err_t rx_gptw_stop(const rx_gptw_channel_t channel)
```

Stop GPTW counter (halt PWM generation).

Stop the GPTW timer counter.

Implementation of `rx_gptw_stop()` - see `rx_gptw.h` for complete documentation.

Clears the Count Start (CST) bit in GTCR to halt counter operation. The timer counter value (GTCNT) is preserved and PWM outputs hold their current state.

Figure 808: Call graph for `rx_gptw_stop`

Defined in `libs/rx_hal/src/rx_gptw.c:1535`

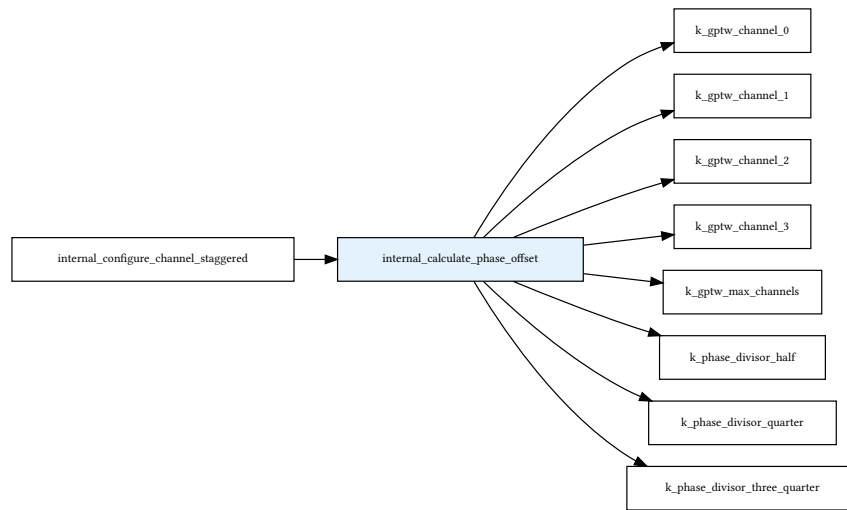
—

internal_calculate_phase_offset

```
static void internal_calculate_phase_offset(const rx_gptw_channel_t channel,
const uint32_t period, const bool is_triangle, uint32_t *init_count, bool
*init_dir_up)
```

Calculate initial counter value and direction for phase staggering.

Computes the initial counter value (GTCNT) and counting direction for a GPTW channel to achieve the desired phase offset. Phase staggering spreads PWM edges across the period to reduce peak current draw and EMI.

Figure 809: Call graph for `internal_calculate_phase_offset`

Defined in `libs/rx_hal/src/rx_gptw.c:1651`

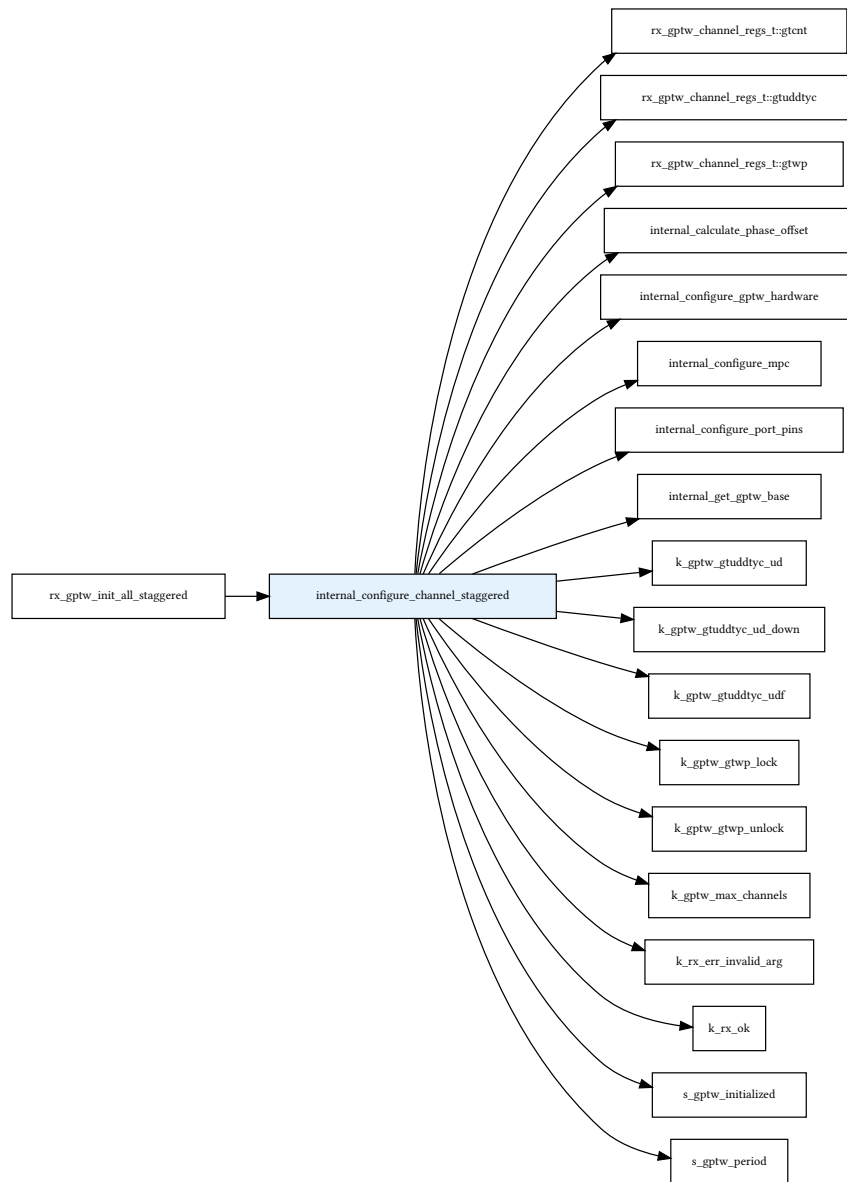
—

internal_configure_channel_staggered

```
static rx_err_t internal_configure_channel_staggered(const rx_gptw_channel_t  
channel, const rx_gptw_config_t *config, const uint32_t period, const bool  
is_triangle)
```

Configure a single GPTW channel for staggered PWM operation.

Performs complete hardware configuration for one GPTW channel including base timer setup, MPC pin function selection, port configuration, and phase staggering initialization.

Figure 810: Call graph for `internal_configure_channel_staggered`

Defined in `libs/rx_hal/src/rx_gptw.c:1804`

`rx_gptw_init_all_staggered`

```
rx_err_t rx_gptw_init_all_staggered(const rx_gptw_config_t *config)
```

Initialize all 4 GPTW channels with 90-degree phase staggering.

Initialize all 4 GPTW channels with 90 degree phase staggering.

Implementation of `rx_gptw_init_all_staggered()` - see `rx_gptw.h` for complete documentation with diagrams, examples, and NASA compliance notes.

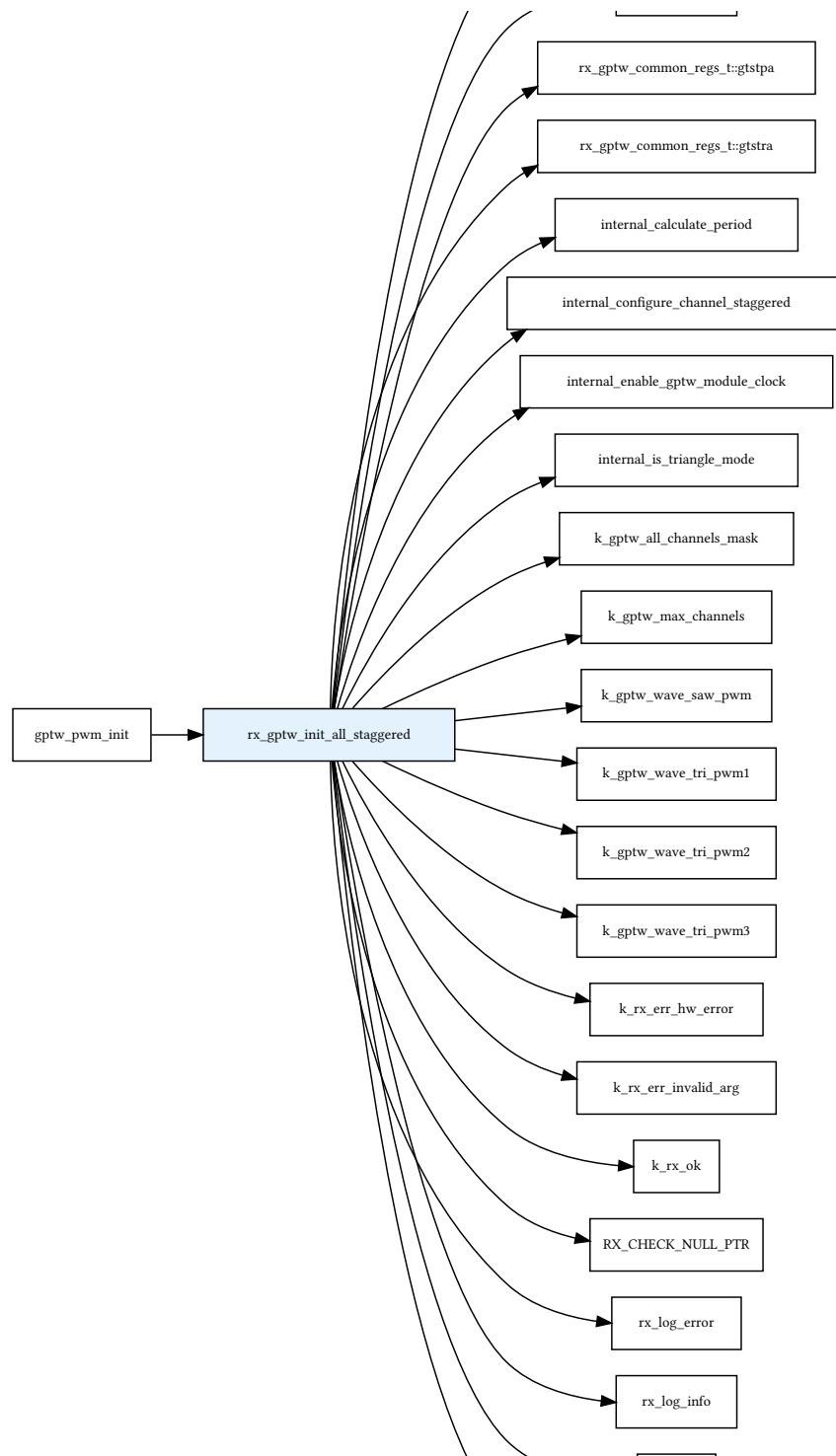


Figure 811: Call graph for rx_gptw_init_all_staggered

Defined in `libs/rx_hal/src/rx_gptw.c:1906`

—

rx_gptw_deinit

```
rx_err_t rx_gptw_deinit(const rx_gptw_channel_t channel)
```

Deinitialize a GPTW channel.

Deinitialize a GPTW channel and release all resources.

Stops the timer, disables both PWM outputs (A and B), and resets the counter to zero. Call only when motor control is no longer active.

Write protection re-locked (GTWP) on all exit paths

Parameters:

Direction	Name	Description
in	channel	GPTW channel (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid channel number

Pre: Motor using this channel must be stopped before calling

Pre: channel must be in range k_gptw_channel_0, k_gptw_channel_3

Post: Timer counter stopped (GTCR.CST = 0)

Post: Both outputs disabled (GTIOR.OAE = 0, GTIOR.OBE = 0)

Post: Counter register zeroed (GTCNT = k_gptw_period_zero)

Note: Not thread-safe: modifies hardware registers directly

Note: Does not clear s_gptw_initialized or s_gptw_period

Warning: Ensure motor is at rest before calling to prevent abrupt stop] **Example:**

```
rx_err_t err = rx_gptw_deinit(k_gptw_channel_0);
```

See also: rx_gptw_init_all() Re-initialize after deinit, rx_gptw_stop() Stop without full teardown

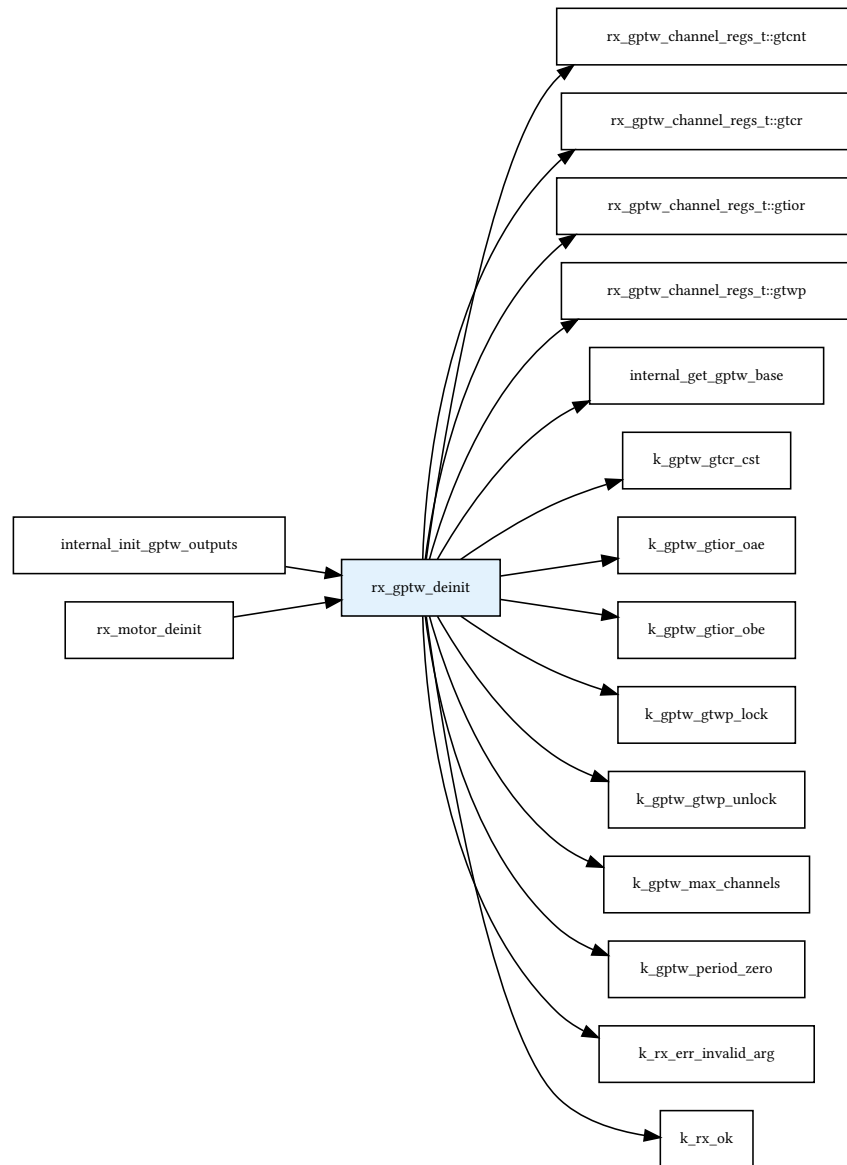


Figure 812: Call graph for rx_gptw_deinit

Defined in `libs/rx_hal/src/rx_gptw.c:2005`

Since Version 1.0.0

—

rx_host_irq.c

HOST_IRQ GPIO Output Driver Implementation.

Configures P67 (PORT6 pin 7) as a GPIO output for signaling the RPi5 host controller that the RX72N has data ready for SPI transfer. The pin is active-low: assert LOW = data ready, deassert HIGH = idle.

Includes:

- "rx_host_irq.h"
- "rx72n_port_regs.h"
- "rx_log.h"

host_irq_gpio_constants_t

GPIO configuration constants for HOST_IRQ on P67.

Value	Name	Description
= 7	k_host_irq_pin_bit	Pin 7 within PORT6
= 0x80	k_host_irq_pin_mask	Bit mask for pin 7 ($1 \ll 7$)
= 0	k_host_irq_active	Active level: LOW
= 1	k_host_irq_idle	Idle level: HIGH

—

s_tag

`const char* s_tag`

—

s_initialized

`bool s_initialized`

Initialization flag.

—

rx_host_irq_init

`rx_err_t rx_host_irq_init(void)`

Initialize HOST_IRQ pin as GPIO output (deasserted / HIGH).

Configures P67 as a GPIO output in CMOS push-pull mode. Sets the initial output level to HIGH (deasserted / idle). The MPC function select is set to GPIO (not IRQ input).

Initialization steps:

1. Clear PMR bit 7 (GPIO mode, not peripheral)
2. Set PODR bit 7 HIGH (deasserted state)
3. Set PDR bit 7 (output direction)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, P67 configured as output
k_rx_err_invalid_state	Already initialized

Pre: P67 not in use by another peripheral (RIIC2 etc.)

Post: P67 configured as GPIO output, driven HIGH

Note: Not thread-safe. Call during system init only.] **See also:** `rx_host_irq_deinit()` Disable the output

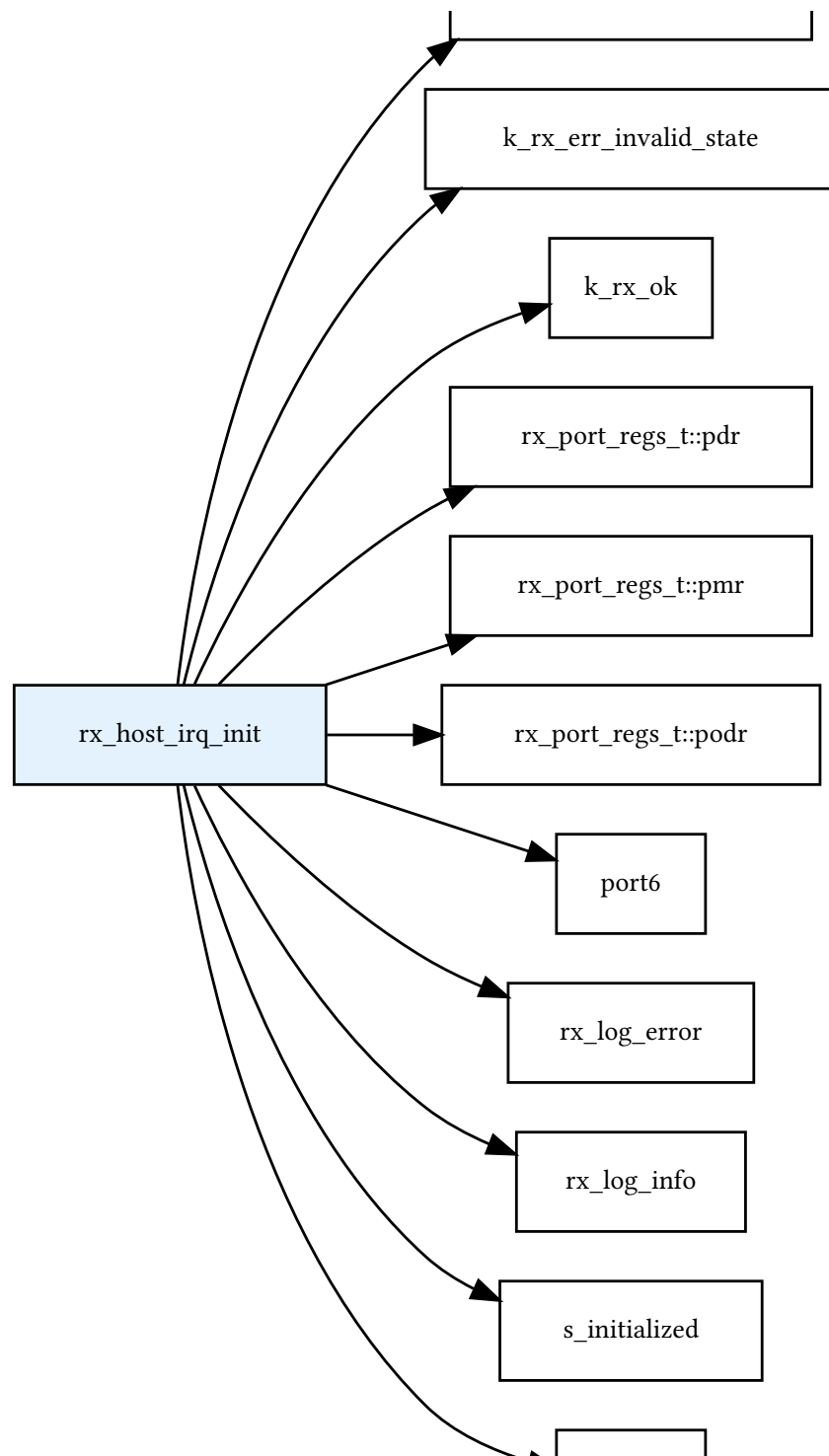


Figure 813: Call graph for rx_host_irq_init

_Defined in libs/rx_hal/src/rx_host_irq.c:78_

Since Version 1.0.0

—

rx_host_irq_deinit

```
rx_err_t rx_host_irq_deinit(void)
```

Deinitialize HOST_IRQ pin (revert to input).

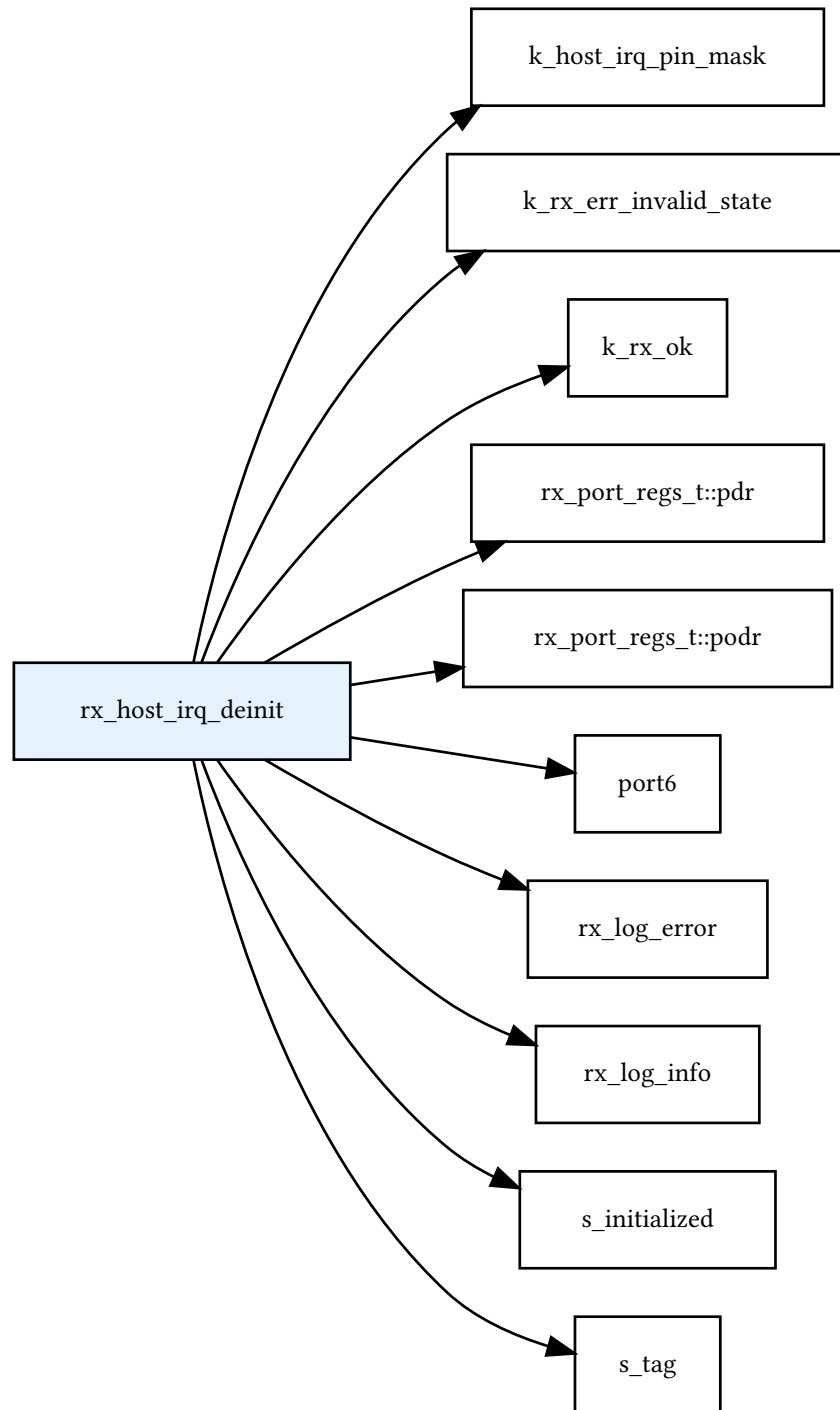
Reverts P67 to input mode (safe default) and deasserts the output.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_state	Not initialized

Pre: rx_host_irq_init() called successfully

Post: P67 reverted to input, no longer driving

Figure 814: Call graph for `rx_host_irq_deinit`

_Defined in `libs/rx\_hal/src/rx\_host\_irq.c:102_`

Since Version 1.0.0

—

rx_host_irq_assert

```
rx_err_t rx_host_irq_assert(void)
```

Assert HOST_IRQ (drive LOW to signal data ready).

Drives P67 LOW to signal the RPi5 that the RX72N has data ready for SPI transfer. The RPi5 should respond by initiating an SPI read.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, pin driven LOW
k_rx_err_invalid_state	Not initialized

Pre: rx_host_irq_init() called successfully

Post: P67 driven LOW (active)

Note: Thread-safe: single register write is atomic on RX72N] **See also:** rx_host_irq_deassert()
Release the signal

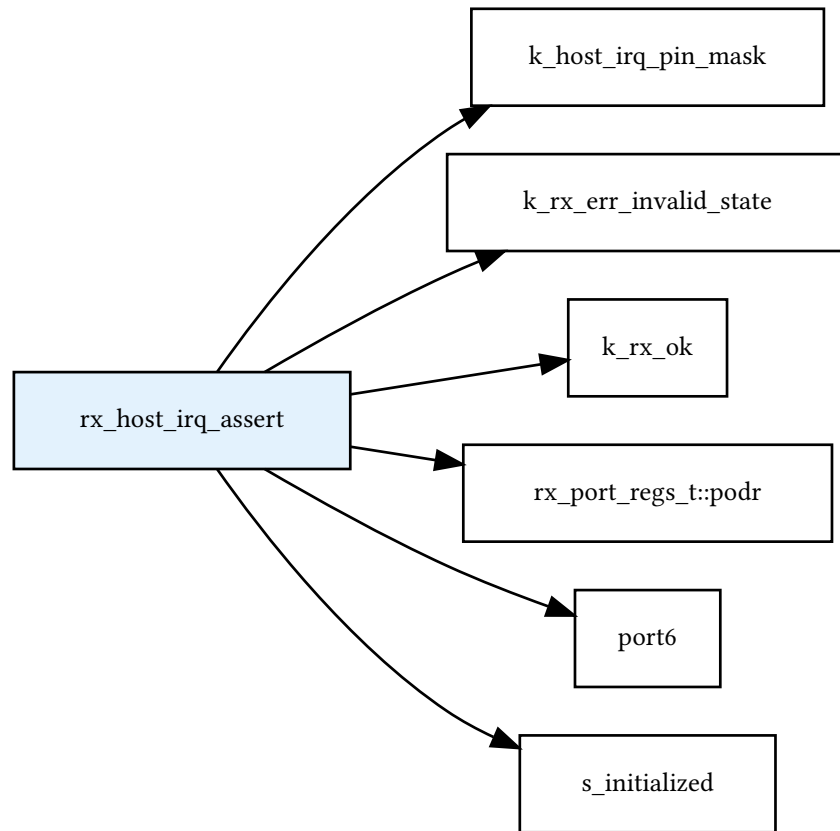


Figure 815: Call graph for rx_host_irq_assert

_Defined in `libs/rx\_hal/src/rx\_host\_irq.c:123_`

Since Version 1.0.0

—

rx_host_irq_deassert

```
rx_err_t rx_host_irq_deassert(void)
```

Deassert HOST_IRQ (drive HIGH, idle state).

Drives P67 HIGH (idle). Call after the RPi5 has completed its SPI read.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, pin driven HIGH
k_rx_err_invalid_state	Not initialized

Pre: rx_host_irq_init() called successfully

Post: P67 driven HIGH (idle)

Note: Thread-safe: single register write is atomic on RX72N] **See also:** rx_host_irq_assert()
Assert the signal

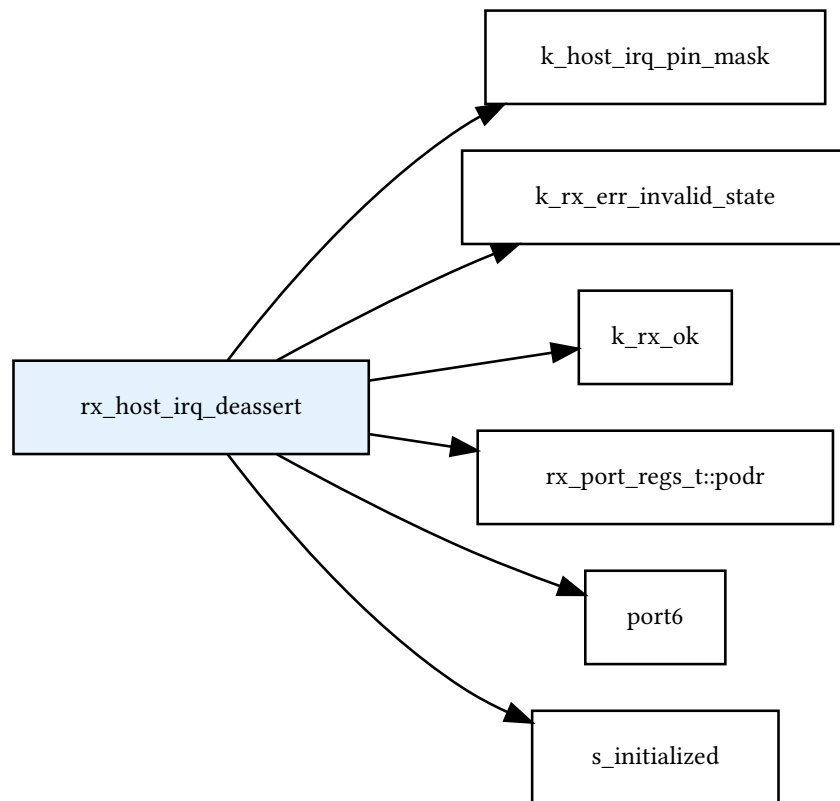


Figure 816: Call graph for rx_host_irq_deassert

_Defined in libs/rx_hal/src/rx_host_irq.c:135_

Since Version 1.0.0

—

rx_host_irq_is_asserted

```
rx_err_t rx_host_irq_is_asserted(bool *asserted)
```

Check if HOST_IRQ is currently asserted.

Returns the current state of the HOST_IRQ output.

Parameters:

Direction	Name	Description
out	asserted	Pointer to store state (true = asserted / LOW)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	asserted is nullptr

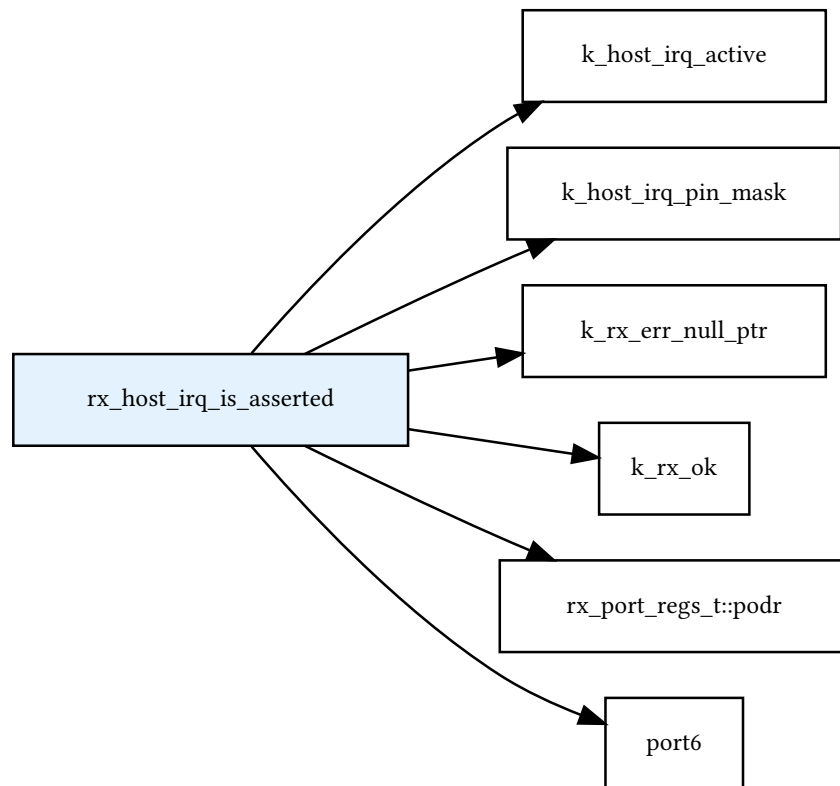
Pre: rx_host_irq_init() called**Post:** asserted set to current output state**Note:** Thread-safe (reads volatile register)

Figure 817: Call graph for rx_host_irq_is_asserted

_Defined in libs/rx_hal/src/rx_host_irq.c:147_

Since Version 1.0.0

—

rx_irq_filter.c

IRQ Digital Filter Driver Implementation for RX72N.

Implements hardware digital filtering on IRQ pins using the ICU (Interrupt Controller Unit) peripheral. The filter provides noise rejection by requiring 3 consecutive samples at the same level before recognizing a valid transition.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_irq_filter.h"
- "rx72n_regs.h"

irq_filter_constants_t

IRQ filter register organization and bit field constants.

Defines constants for navigating the IRQFLTE and IRQFLTC register arrays. The ICU organizes filter controls into two registers each:

- Index 0: IRQ0-7 (lower 8 IRQs)
- Index 1: IRQ8-15 (upper 8 IRQs)

Value	Name	Description
= 8	k_irq_filter_irqs_per_reg	Number of IRQs controlled by each register pair.
= 2	k_irq_filter_clock_bits	Number of bits per clock divisor setting in IRQFLTC.
= 0x03	k_irq_filter_clock_mask	Bit mask for extracting/setting clock divisor value.

Since Version 1.0.0

—

rx_irq_filter_enable

```
rx_err_t rx_irq_filter_enable(uint8_t irq_num, rx_irq_filter_clk_t filter_clk)
```

Enable hardware digital filter on an IRQ pin.

Configures the ICU filter enable (IRQFLTE) and filter clock (IRQFLTC) registers for the specified IRQ. Sets clock divisor first, then enables the filter to avoid sampling at incorrect rate during transition.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (0-15)
in	filter_clk	Clock divisor setting

Returns: k_rx_ok on success, k_rx_err_invalid_arg if parameters invalid

Note: On host (non-RX) builds, validates parameters but skips register access] **Register Modification Sequence:** IRQFLTC: Clear and set 2-bit clock field for this IRQ IRQFLTE: Set enable bit for this IRQ

See also: rx_irq_filter.h Full API documentation

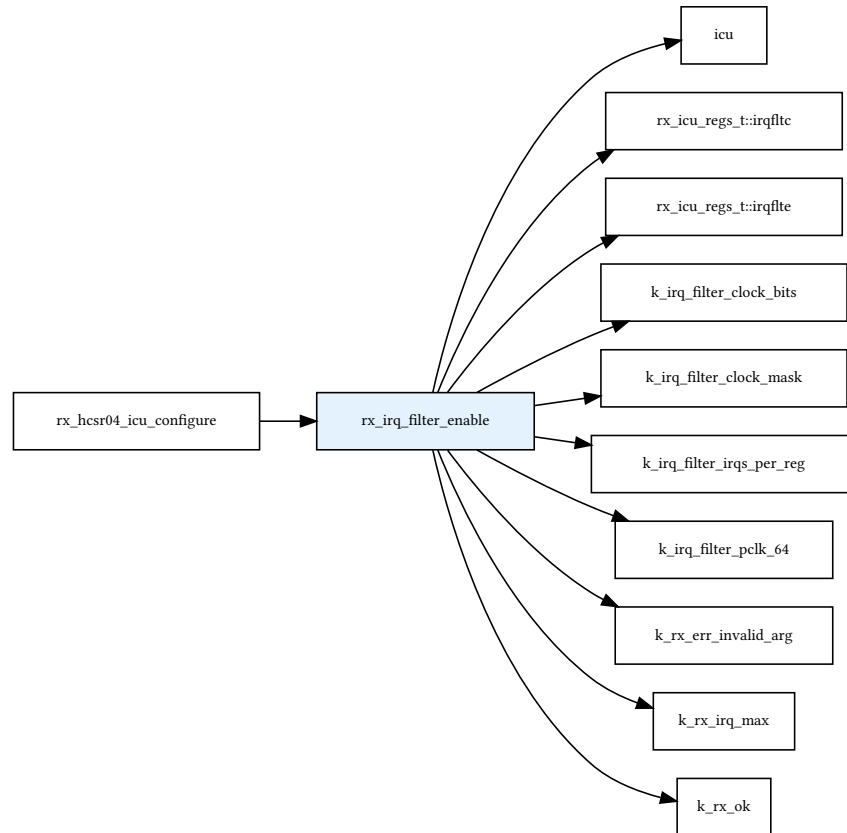


Figure 818: Call graph for rx_irq_filter_enable

_Defined in `libs/rx_hal/src/rx_irq_filter.c:174_`

Since Version 1.0.0

—

rx_irq_filter_disable

```
rx_err_t rx_irq_filter_disable(uint8_t irq_num)
```

Disable hardware digital filter on an IRQ pin.

Clears the filter enable bit in IRQFLTE for the specified IRQ. After disabling, raw unfiltered input passes to interrupt detector.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (0-15)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if irq_num > 15

Note: Clock divisor setting preserved for potential re-enable] **See also:** rx_irq_filter.h Full API documentation

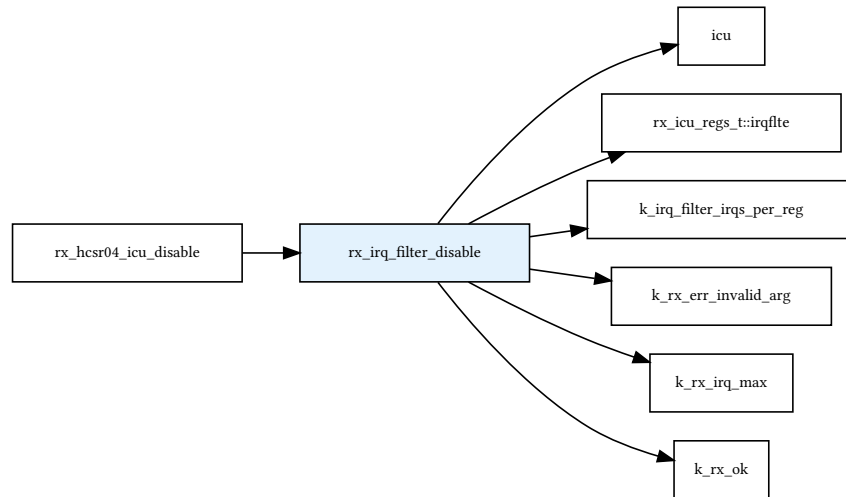


Figure 819: Call graph for rx_irq_filter_disable

_Defined in libs/rx_hal/src/rx_irq_filter.c:242_

Since Version 1.0.0

—

rx_irq_filter_is_enabled

```
rx_err_t rx_irq_filter_is_enabled(uint8_t irq_num, bool *enabled)
```

Check if digital filter is enabled on an IRQ pin.

Reads the IRQFLTE register bit for the specified IRQ to determine current filter enable state.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (0-15)
out	enabled	Pointer to store filter status (true=enabled)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if parameters invalid

Note: On host builds, always returns false (filter disabled)] **See also:** `rx_irq_filter.h` Full API documentation

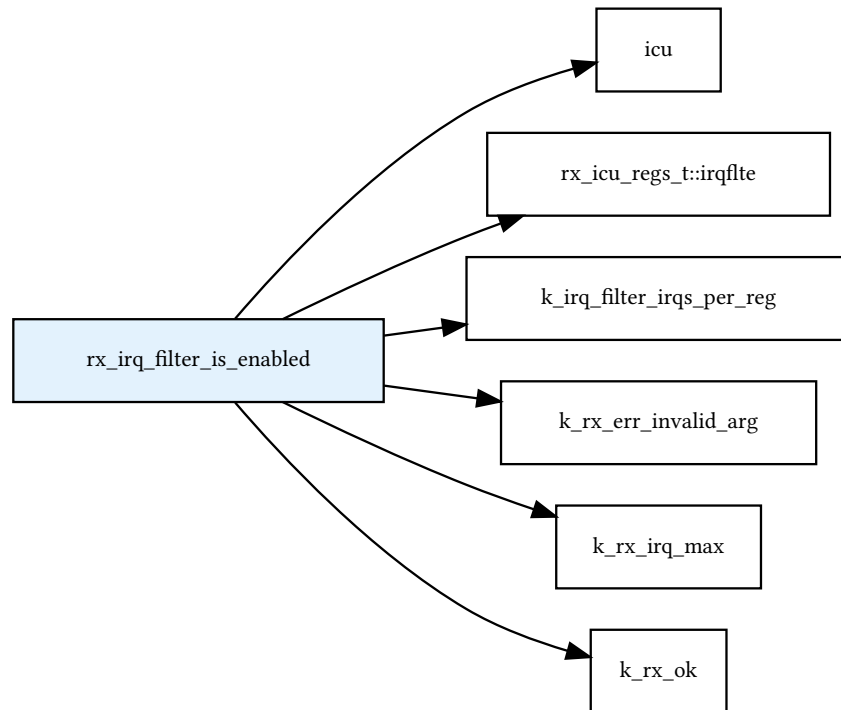


Figure 820: Call graph for `rx_irq_filter_is_enabled`

_Defined in `libs/rx_hal/src/rx_irq_filter.c:277_`

Since Version 1.0.0

—

rx_irq_filter_get_clock

```
rx_err_t rx_irq_filter_get_clock(uint8_t irq_num, rx_irq_filter_clk_t
*filter_clk)
```

Get current filter clock divisor setting for an IRQ.

Reads the 2-bit clock divisor field from IRQFLTC register for the specified IRQ. Returns the divisor setting even if filter is disabled.

Parameters:

Direction	Name	Description
in	<code>irq_num</code>	IRQ number (0-15)
out	<code>filter_clk</code>	Pointer to store clock divisor (<code>rx_irq_filter_clk_t</code>)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if parameters invalid

Note: On host builds, always returns k_irq_filter_pclk_1] **See also:** rx_irq_filter.h Full API documentation

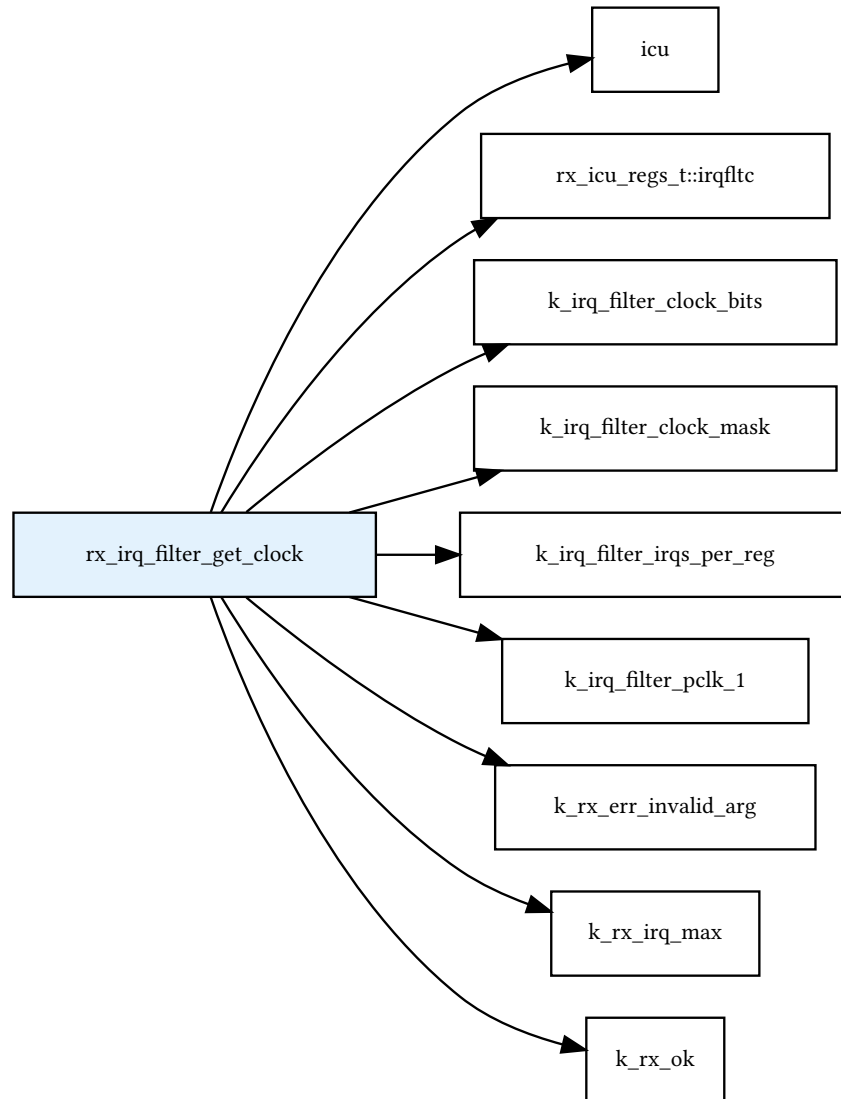


Figure 821: Call graph for rx_irq_filter_get_clock

_Defined in libs/rx_hal/src/rx_irq_filter.c:317_

Since Version 1.0.0

—

rx_mpc.c

Multi-Function Pin Controller (MPC) Driver Implementation.

Implementation of the MPC driver for RX72N pin multiplexer configuration. This module provides the core functionality for connecting GPIO pins to peripheral functions through the PFS (Pin Function Select) registers.

STAR Team

2026-01-29

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_mpc.h"
- <stddef.h>
- "rx72n_regs.h"
- "rx_check.h"
- "rx_log.h"
- "rx_port_constants.h"

mpc_pin_constants_t

Pin number validation constants for MPC driver.

Defines the valid range for pin numbers within a port. RX72N GPIO ports support up to 8 pins per port (pins 0-7), though not all pins are available on all ports or packages.

Value	Name	Description
= 0	k_mpc_min_pin	Minimum valid pin number (always 0)
= k_rx_pin_max	k_mpc_max_pin	Maximum valid pin number (7 for 8 pins per port)

See also: rx_port_constants.h Source of k_rx_pin_max definition

Since Version 1.0.0

—

mpc_pfs_offset_t

PFS register base offset from MPC structure start.

The PFS registers start at offset 0x40 from MPC base (0x0008C100), which is address 0x0008C140. However, when accessing via the rx_mpc_regs_t struct, we need to account for the structure layout.

This offset is used with pointer arithmetic to calculate PFS register addresses from the mpc() accessor.

Value	Name	Description
= 33	k_mpc_pfs_base_offset	Byte offset from mpc() to P00PFS register

Since Version 1.0.0

—

mpc_port_number_t

Port number aliases for MPC driver validation.

Maps the generic `rx_port_*` constants to MPC-specific names for use in the switch statement in `internal_get_pfs_register()`. This provides compile-time verification that all ports are handled.

Value	Name	Description
<code>= k_rx_port_0</code>	<code>k_mpc_port_start</code>	First valid port number for range check
<code>= k_rx_port_0</code>	<code>k_mpc_port_0</code>	Port 0 (GPIO P00-P07)
<code>= k_rx_port_1</code>	<code>k_mpc_port_1</code>	Port 1 (GPIO P10-P17)
<code>= k_rx_port_2</code>	<code>k_mpc_port_2</code>	Port 2 (GPIO P20-P27, encoder inputs)
<code>= k_rx_port_3</code>	<code>k_mpc_port_3</code>	Port 3 (GPIO P30-P34, P35=NMI)
<code>= k_rx_port_4</code>	<code>k_mpc_port_4</code>	Port 4 (GPIO P40-P47, ADC inputs)
<code>= k_rx_port_5</code>	<code>k_mpc_port_5</code>	Port 5 (GPIO P50-P57)
<code>= k_rx_port_6</code>	<code>k_mpc_port_6</code>	Port 6 (GPIO P60-P67)
<code>= k_rx_port_7</code>	<code>k_mpc_port_7</code>	Port 7 (GPIO P71-P77, P70 N/A)
<code>= k_rx_port_8</code>	<code>k_mpc_port_8</code>	Port 8 (GPIO P80-P87)
<code>= k_rx_port_9</code>	<code>k_mpc_port_9</code>	Port 9 (GPIO P90-P97)
<code>= k_rx_port_a</code>	<code>k_mpc_port_a</code>	Port A (GPIO PA0-PA7, SPI pins)
<code>= k_rx_port_b</code>	<code>k_mpc_port_b</code>	Port B (GPIO PB0-PB7, UART pins)
<code>= k_rx_port_c</code>	<code>k_mpc_port_c</code>	Port C (GPIO PC0-PC7, encoder inputs)
<code>= k_rx_port_d</code>	<code>k_mpc_port_d</code>	Port D (GPIO PD0-PD7)
<code>= k_rx_port_e</code>	<code>k_mpc_port_e</code>	Port E (GPIO PE0-PE7, PWM outputs)
<code>= k_rx_port_f</code>	<code>k_mpc_port_f</code>	Port F (partial on 144-pin: PF0-PF2, PF5)
<code>= k_rx_port_g</code>	<code>k_mpc_port_g</code>	Port G (not on 144-pin package)
<code>= k_rx_port_j</code>	<code>k_mpc_port_j</code>	Port J (partial: PJ3, PJ5 on 144-pin)

See also: `rx_port_constants.h` Source definitions

Since Version 1.0.0

—

mpc_port_offset_t

PFS register byte offsets for each port.

PFS registers are laid out contiguously in memory with 8 bytes allocated per port (one byte per pin), even if not all pins are physically present. These offsets are added to the PFS base address to reach each port's registers.

Value	Name	Description
<code>= 0</code>	<code>k_mpc_port0_offset</code>	Port 0 offset: 0x00 (P00-P07)
<code>= 8</code>	<code>k_mpc_port1_offset</code>	Port 1 offset: 0x08 (P10-P17)
<code>= 16</code>	<code>k_mpc_port2_offset</code>	Port 2 offset: 0x10 (P20-P27)
<code>= 24</code>	<code>k_mpc_port3_offset</code>	Port 3 offset: 0x18 (P30-P34)
<code>= 32</code>	<code>k_mpc_port4_offset</code>	Port 4 offset: 0x20 (P40-P47, ADC)

= 40	k_mpc_port5_offset	Port 5 offset: 0x28 (P50-P57)
= 48	k_mpc_port6_offset	Port 6 offset: 0x30 (P60-P67)
= 56	k_mpc_port7_offset	Port 7 offset: 0x38 (P71-P77)
= 64	k_mpc_port8_offset	Port 8 offset: 0x40 (P80-P87)
= 72	k_mpc_port9_offset	Port 9 offset: 0x48 (P90-P97)
= 80	k_mpc_porta_offset	Port A offset: 0x50 (PA0-PA7, SPI)
= 88	k_mpc_portb_offset	Port B offset: 0x58 (PB0-PB7, UART)
= 96	k_mpc_portc_offset	Port C offset: 0x60 (PC0-PC7, MTU)
= 104	k_mpc_portd_offset	Port D offset: 0x68 (PD0-PD7)
= 112	k_mpc_porte_offset	Port E offset: 0x70 (PE0-PE7, PWM)
= 120	k_mpc_portf_offset	Port F offset: 0x78 (partial on 144-pin)
= 128	k_mpc_portg_offset	Port G offset: 0x80 (not on 144-pin)
= 136	k_mpc_porth_offset	Port H offset: 0x88 (not on 144-pin)
= 144	k_mpc_portj_offset	Port J offset: 0x90 (PJ3, PJ5 on 144-pin)

Note: Each port occupies 8 bytes regardless of actual pin count] **See also:** `internal_get_pfs_register()` Uses these offsets

Since Version 1.0.0

—

mpc_psel_constants_t

PSEL field validation constants.

The PSEL field in PFS registers is 5 bits wide, allowing values 0-31. This constant defines the maximum valid value for input validation.

Value	Name	Description
= 31	k_mpc_psel_max	Maximum PSEL value (5 bits = 0x1F)

See also: `rx_mpc_set_peripheral()` Validates PSEL against this maximum

Since Version 1.0.0

—

mpc_pfs_gpio_t

PFS register value for GPIO mode.

GPIO mode requires all bits in PFS register to be 0:

- PSEL[4:0] = 0 (no peripheral function)
- ISEL = 0 (no interrupt input)
- ASEL = 0 (digital mode, not analog)

Value	Name	Description
= 0	k_mpc_pfs_gpio	GPIO mode: all zeros (PSEL=0, ISEL=0, ASEL=0)

See also: `rx_mpc_set_gpio()` Uses this value

Since Version 1.0.0

—

mpc_pwpr_t

PWPR register control values.

Values used to control the PWPR (Write Protect Register) during the unlock/lock sequence. Clearing PWPR is the first step in both sequences.

Value	Name	Description
= 0	<code>k_mpc_pwpr_clear</code>	Clear all PWPR bits (first step of unlock/lock)

See also: `internal_mpc_unlock()` Uses `k_mpc_pwpr_clear` then `k_mpc_pwpr_pfswe`, `internal_mpc_lock()` Uses `k_mpc_pwpr_clear` then `k_mpc_pwpr_b0wi`

Since Version 1.0.0

—

s_tag

`const char* s_tag`

Logging tag for MPC module.

Note: Statically allocated, never modified

Since Version 1.0.0

—

internal_get_pfs_register

```
static volatile uint8_t * internal_get_pfs_register(const uint8_t port, uint8_t pin)
```

Calculate PFS register address for a given port and pin.

Computes the memory address of the Pin Function Select (PFS) register for the specified port and pin combination. Uses a lookup table (switch) to get the port offset, then adds the pin number.

Parameters:

Direction	Name	Description
in	<code>port</code>	Port number (0-9, A-G, J encoded as <code>uint8_t</code>) Valid range: [<code>k_mpc_port_0</code> , <code>k_mpc_port_j</code>] Ports G, H return NULL on 144-pin package
in	<code>pin</code>	Pin number within the port Valid range: [0, 7] Some ports have fewer than 8 pins; caller must verify

Returns: Pointer to the PFS register for the specified pin

Value	Description
-------	-------------

non-NULL	Valid volatile pointer to PFS register
NULL	Invalid port (out of range or unavailable on package)
NULL	Invalid pin number (> 7)

Pre: PCLKB clock must be running for MPC register access

Pre: Caller has verified channel is in valid range for the target package

Post: No registers modified (read-only address calculation)

Note: Internal function - not exported in header

Note: Port G, H availability depends on package (not on 144-pin LFQFP)

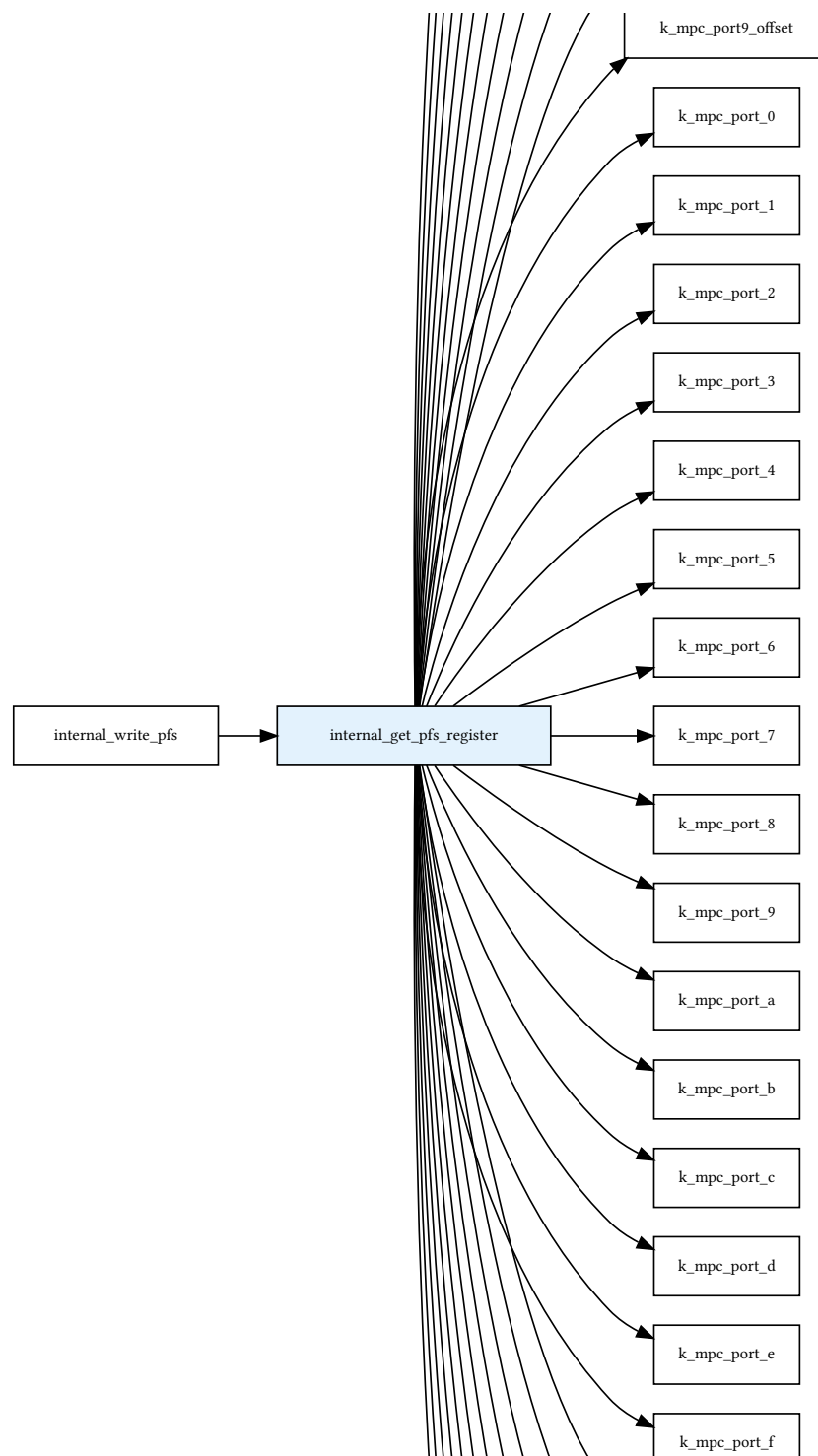
Warning: Returned pointer is volatile - must dereference with care

Warning: Caller must still unlock PWPR before writing to the register] **Algorithm Steps:**
 Validate pin number is in range [0, 7] Get base address of PFS registers from mpc() accessor Look
 up port offset in switch statement If port invalid, log error and return nullptr Return base + offset
 + pin

Address Calculation: * PFS_addr = (mpc() + k_mpc_pfs_base_offset) + port_offset + pin *
 Example: PB7 (Port B, Pin 7) * = (0x0008C100 + 33) + 88 + 7 * = 0x0008C121 + 88 + 7 * = 0x0008C1D0
 (incorrect - see note) * * Note: The actual calculation uses struct-based addressing which * handles
 the non-contiguous reserved regions correctly. *

Performance: Execution time: 200 ns @ 240 MHz (switch lookup + pointer math) Stack usage: 8
 bytes (local variables)

See also: mpc() MPC register accessor, mpc_port_offset_t Port offset constants

Figure 822: Call graph for `internal_get_pfs_register`

Defined in `libs/rx_hal/src/rx_mpc.c:442`

Since Version 1.0.0

—

internal_mpc_unlock

```
static void internal_mpc_unlock(void)
```

Unlock MPC PFS register write protection.

Executes the two-step unlock sequence required before writing to any PFS register. The PWPR (Write Protect Register) guards against accidental modification of pin functions.

Pre: MPC module must be active (not in module stop)

Post: PWPR.B0WI = 0 (PFSWE modifiable)

Post: PWPR.PFSWE = 1 (PFS registers writable)

Note: Must call `internal_mpc_lock()` after PFS modification!

Note: Internal function - not exported in header

Warning: Not thread-safe - window between unlock and lock is vulnerable

Warning: Failing to lock after modification leaves PFS unprotected] **Unlock Sequence (per RX72N Hardware Manual Ch20):** Write 0x00 to PWPR (clears B0WI bit, allows PFSWE modification) Write 0x40 to PWPR (sets PFSWE bit, enables PFS register writes)

Timing: Two 8-bit register writes No wait states required between writes Total time: 100 ns @ 240 MHz

Example:: `internal_mpc_unlock(); *pfs_reg=k_psel_sci_tx; internal_mpc_lock();`

See also: `internal_mpc_lock()` Must be called after PFS writes, `k_mpc_pwpr_pfswe` PFSWE enable constant (0x40)

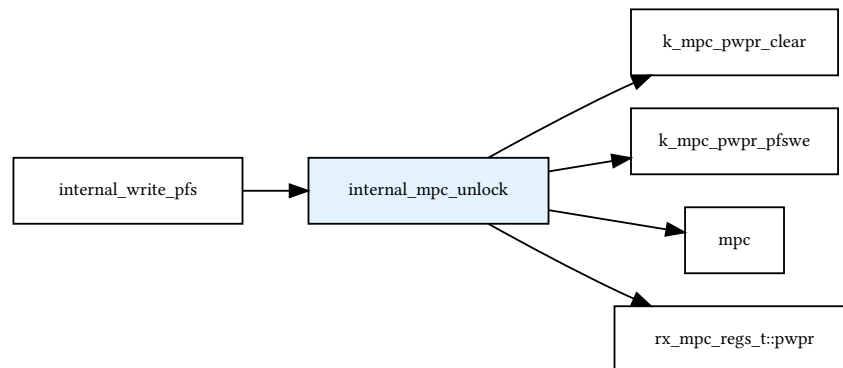


Figure 823: Call graph for internal_mpc_unlock

Defined in `libs/rx_hal/src/rx_mpc.c:561`

Since Version 1.0.0

internal_mpc_lock

```
static void internal_mpc_lock(void)
```

Lock MPC PFS register write protection.

Executes the two-step lock sequence to protect PFS registers after modification. This restores write protection and prevents accidental changes to pin functions.

Pre: `internal_mpc_unlock()` was called previously

Pre: PFS modifications are complete

Post: `PWPR.PFSWE = 0` (PFS registers protected)

Post: `PWPR.B0WI = 1` (PFSWE bit protected)

Note: Internal function - not exported in header] **Lock Sequence (per RX72N Hardware Manual Ch20):** Write 0x00 to PWPR (clears PFSWE bit, protects PFS registers) Write 0x80 to PWPR (sets B0WI bit, prevents PFSWE modification)

Timing: Two 8-bit register writes No wait states required between writes Total time: 100 ns @ 240 MHz

Example:: `internal_mpc_unlock(); *pfs_reg=k_psel_rspl_clk; internal_mpc_lock();`

See also: `internal_mpc_unlock()` Must be called before PFS writes, `k_mpc_pwpr_b0wi` B0WI enable constant (0x80)

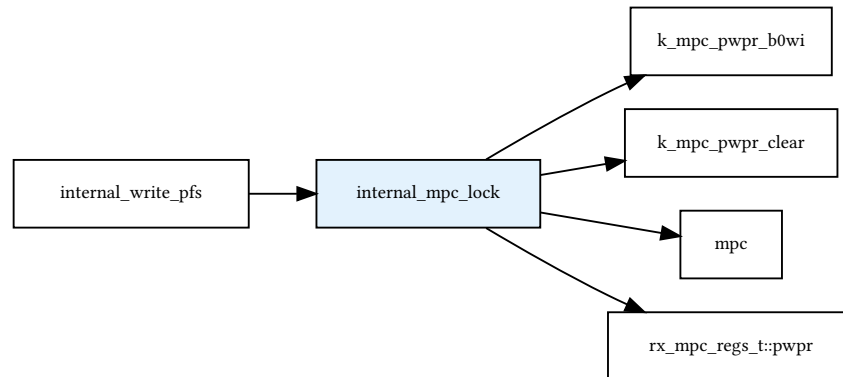


Figure 824: Call graph for internal_mpc_lock

Defined in `libs/rx_hal/src/rx_mpc.c:609`

Since Version 1.0.0

internal_write_pfs

```
static rx_err_t internal_write_pfs(const uint8_t port, uint8_t pin, uint8_t value)
```

Write value to PFS register with automatic unlock/lock.

Complete PFS modification sequence: calculates register address, unlocks protection, writes the value, and re-locks protection. This function handles the entire write-protected register access pattern.

Parameters:

Direction	Name	Description
in	port	Port number (0-9, A-G, J) Validated by internal_get_pfs_register()
in	pin	Pin number within port Valid range: [0, 7]
in	value	Value to write to PFS register Typically PSEL code (0-31) or ASEL/ISEL bits

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	PFS register successfully written
k_rx_err_invalid_arg	Invalid port or pin (register address NULL)

Pre: PCLKB clock must be running

Pre: Value should be a valid PFS configuration

Post: PFS register contains specified value

Post: PWPR is locked (B0WI=1)

Note: Internal function - not exported in header

Note: Lock is always executed even on error path (defensive coding)

Warning: Not thread-safe - the unlock/write/lock sequence is non-atomic] **Algorithm Steps:**
 Call internal_get_pfs_register() to get register address If address is nullptr, return error (no unlock performed) Call internal_mpc_unlock() to enable PFS writes Write value to PFS register Call internal_mpc_lock() to protect PFS registers Return success

Execution Flow: digraph write_pfs { rankdir=TB; node [shape=box, style=rounded];

```
start [label="internal_write_pfs()"]; get_reg [label="internal_get_pfs_register()"]; check
[label="pfs_reg == nullptr?", shape=diamond]; error [label="return k_rx_err_invalid_arg",
fillcolor=lightcoral, style=filled]; unlock [label="internal_mpc_unlock()"]; write [label="*pfs_reg =
value"]; lock [label="internal_mpc_lock()"]; success [label="return k_rx_ok", fillcolor=lightgreen,
style=filled];
```

```
start -> get_reg; get_reg -> check; check -> error [label="yes"]; check -> unlock [label="no"]; unlock
-> write; write -> lock; lock -> success; }
```

Performance: Execution time: 500 ns @ 240 MHz Stack usage: 16 bytes

See also: internal_get_pfs_register() Address calculation, internal_mpc_unlock()
 Protection unlock, internal_mpc_lock() Protection lock

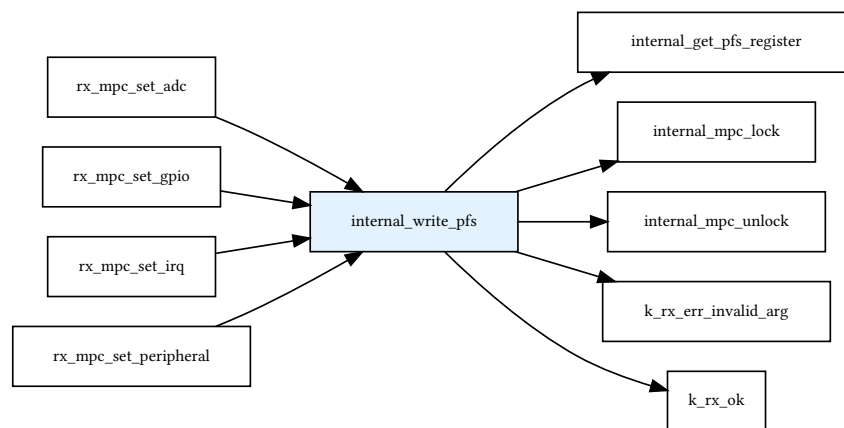


Figure 825: Call graph for internal_write_pfs

Defined in libs/rx_hal/src/rx_mpc.c:692

Since Version 1.0.0

—

rx_mpc_set_gpio

```
rx_err_t rx_mpc_set_gpio(const rx_port_pin_t pin)
```

Configure pin for GPIO (General Purpose I/O) function.

Sets the specified pin to GPIO mode by writing 0x00 to its PFS register. This disconnects the pin from all peripheral functions and allows it to be controlled by the PORT registers (PDR, PODR, PIDR).

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (rx_port_pin_t from rx_port_constants.h)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully set to GPIO mode
k_rx_err_invalid_arg	Port number invalid (> Port J)
k_rx_err_invalid_arg	Pin number invalid (> 7)
k_rx_err_invalid_arg	Port not available on package

Pre: PCLKB clock running, MPC not in module stop

Pre: pin must be a valid rx_port_pin_t constant

Post: PFS register = 0x00 (GPIO mode)

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Also set PORT PMR bit to 0 for complete GPIO configuration] **Implementation Details:** Extract port and pin number from encoded rx_port_pin_t Validate port is in range [0, J] Validate pin is in range [0, 7] Write 0x00 to PFS register (via internal_write_pfs)

Example:

```
//ConfigureP30(port3,pin0) forGPIOuse
rx_err_t err=rx_mpc_set_gpio(RX_PORT_PIN(3,0));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc.h Full API documentation



Figure 826: Call graph for rx_mpc_set_gpio

Defined in `libs/rx_hal/src/rx_mpc.c:754`

Since Version 1.0.0

—

rx_mpc_set_peripheral

```
rx_err_t rx_mpc_set_peripheral(const rx_mpc_peripheral_config_t *config)
```

Configure pin for peripheral function with specified PSEL code.

Core function for peripheral pin configuration. Writes the specified PSEL value to the pin's PFS register, connecting it to a peripheral.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	config	Pointer to peripheral configuration structure config->pin: Target pin identifier config->psel: PSEL value to write (0-31)
----	--------	---

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin successfully configured
k_rx_err_invalid_arg	config is nullptr
k_rx_err_invalid_arg	PSEL exceeds 31
k_rx_err_invalid_arg	Invalid port or pin

Pre: config must be non-NULL with valid fields

Pre: PCLKB clock running

Post: PFS register contains config->psel

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Also set PORT PMR bit to 1 for peripheral mode] **Implementation Details:** Validate config pointer is not nullptr Validate PSEL value is in range [0, 31] Extract port and pin from config->pin Validate extracted values Write PSEL to PFS register (via internal_write_pfs)

Example:

```
//ConfigurePA0forRSPIClock(PSEL=0x0D)
constrx_mpc_peripheral_config_tcfg={
    .pin=RX_PORT_PIN(0xA,0),
    .psel=0x0D,
};
rx_err_terr=rx_mpc_set_peripheral(&cfg);
if(err!=k_rx_ok){
    //handleerror
}
```

See also: rx_mpc.h Full API documentation

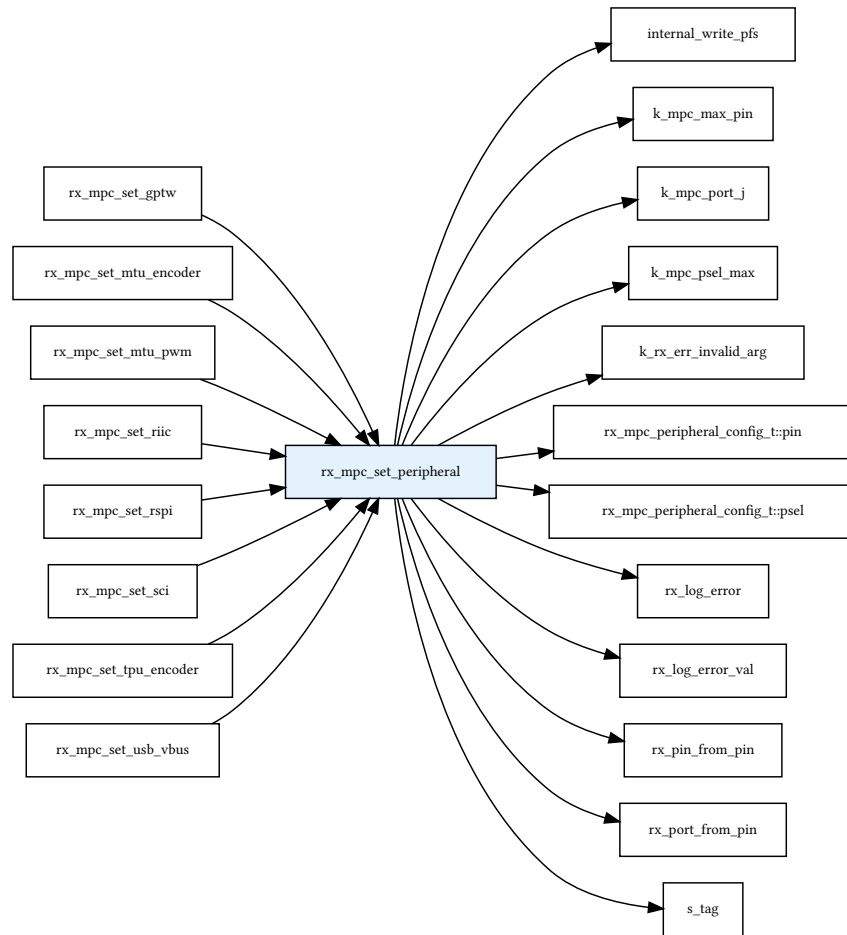


Figure 827: Call graph for rx_mpc_set_peripheral

Defined in `libs/rx_hal/src/rx_mpc.c:816`

Since Version 1.0.0

—

rx_mpc_set_mtu_pwm

```
rx_err_t rx_mpc_set_mtu_pwm(const rx_port_pin_t pin)
```

Configure pin for MTU3a timer PWM output (MTIOC).

Convenience wrapper that configures a pin for MTU I/O compare output using PSEL = k_psel_mtu_ioc (0x01). Commonly used for motor PWM.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	pin	GPIO pin identifier for MTU PWM output
----	-----	--

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for MTU PWM
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination supported by MTU MTIOC

Post: PFS register contains k_psel_mtu_ioc (0x01) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: STAR project uses GPTW (PSEL=0x07) for motor PWM, not MTU] **Example:**

```
//ConfigureP14forMTU3PWMoutput(MTIOCA)
rx_err_t err=rx_mpc_set_mtu_pwm(RX_PORT_PIN(1,4));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc.h Full API documentation

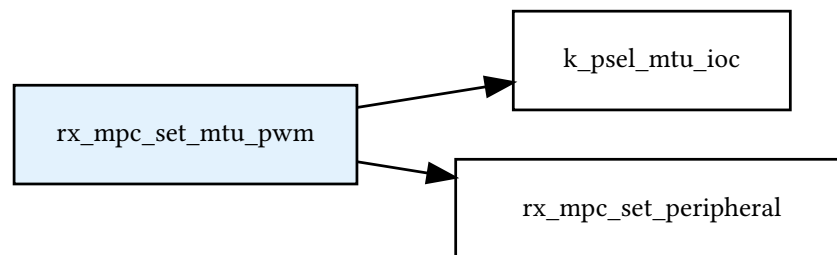


Figure 828: Call graph for rx_mpc_set_mtu_pwm

Defined in `libs/rx_hal/src/rx_mpc.c:879`

Since Version 1.0.0

—

rx_mpc_set_mtu_encoder

```
rx_err_t rx_mpc_set_mtu_encoder(const rx_port_pin_t pin)
```

Configure pin for MTU3a encoder/phase counter input (MTCLK).

Convenience wrapper that configures a pin for MTU phase counting mode using PSEL = k_psel_mtu_phase (0x03). Used for quadrature encoders.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for encoder input (e.g., P24/P25)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for encoder input
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination supported by MTCLK input

Post: PFS register contains k_psel_mtu_phase (0x03) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Configure both Phase A and Phase B pins for proper operation] **Example:**

```
//ConfigurePC0andPC1forMTUphasecounting(MTCLKA/B)
rx_err_t err=rx_mpc_set_mtu_encoder(RX_PORT_PIN(0xC,0));
if(err==k_rx_ok){
err=rx_mpc_set_mtu_encoder(RX_PORT_PIN(0xC,1));
}
```

See also: rx_mpc.h Full API documentation, rx_mtu_encoder.h Encoder driver using MTU phase counting

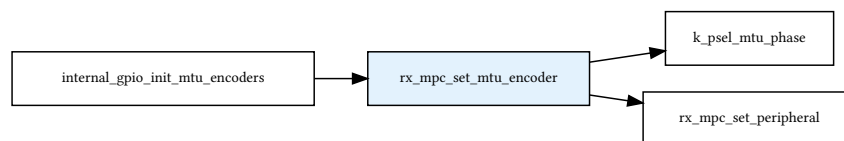


Figure 829: Call graph for rx_mpc_set_mtu_encoder

Defined in `libs/rx_hal/src/rx_mpc.c:925`

Since Version 1.0.0

rx_mpc_set_tpu_encoder

```
rx_err_t rx_mpc_set_tpu_encoder(const rx_port_pin_t pin)
```

Configure pin for TPU encoder/phase counter input (TCLK).

Convenience wrapper that configures a pin for TPU phase counting mode using PSEL = k_psel_mtu_phase (0x03). On the RX72N, PSEL 0x03 is the generic “timer phase counting input” function; routing to TPU (vs MTU) depends on which timer module has phase counting mode enabled.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for TPU encoder input (e.g., PC2/PA3)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for TPU encoder input
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination supported by TCLK input

Post: PFS register contains k_psel_mtu_phase (0x03) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Configure both Phase A and Phase B pins for proper operation] **Example:**

```
//ConfigurePC2andPA3forTPUphasecounting(TCLKA/B)
rx_err_t err=rx_mpc_set_tpu_encoder(RX_PORT_PIN(0xC,2));
if(err==k_rx_ok){
err=rx_mpc_set_tpu_encoder(RX_PORT_PIN(0xA,3));
}
```

See also: rx_mpc.h Full API documentation, rx_encoder_tpu.h TPU encoder driver using phase counting

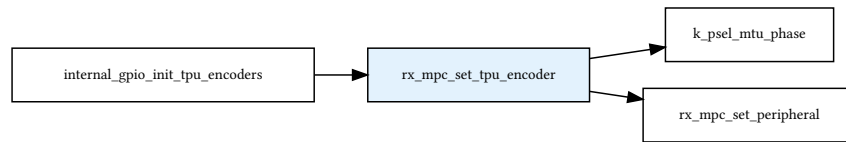


Figure 830: Call graph for rx_mpc_set_tpu_encoder

Defined in `libs/rx_hal/src/rx_mpc.c:975`

Since Version 1.1.0

—

rx_mpc_set_adc

```
rx_err_t rx_mpc_set_adc(const rx_port_pin_t pin)
```

Configure pin for ADC analog input.

Configures a pin for analog-to-digital conversion by setting the ASEL bit in the PFS register. This disables the digital input buffer to prevent noise during analog measurement.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for ADC (typically P40-P47)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for ADC input
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with analog capability

Post: PFS register contains k_pfs_asel (0x80) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Sets ASEL bit (0x80) rather than PSEL field

Note: Also configure S12ADC for actual conversion] **Example:**

```
//ConfigureP40(AN000)forADCAnaloginput
rx_err_t rx_mpc_set_adc(RX_PORT_PIN(4,0));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc.h Full API documentation

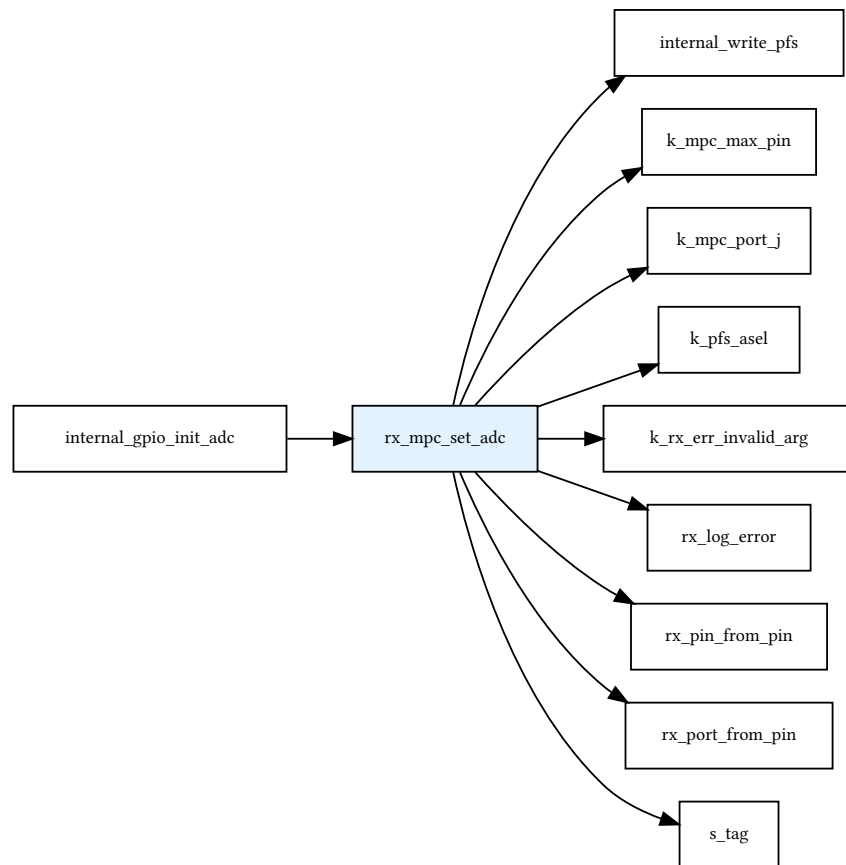


Figure 831: Call graph for rx_mpc_set_adc

Defined in `libs/rx_hal/src/rx_mpc.c:1022`

Since Version 1.0.0

—

rx_mpc_set_sci

```
rx_err_t rx_mpc_set_sci(const rx_port_pin_t pin)
```

Configure pin for SCI (Serial Communication Interface) UART function.

Convenience wrapper that configures a pin for SCI UART operation using PSEL = `k_psel_sci_tx` (0x0A). Works for both TXD and RXD pins.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for SCI (e.g., PB7 for TXD9)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for SCI UART
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with SCI multiplexing

Post: PFS register contains k_psel_sci_tx (0x0A) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Configure both TXD and RXD pins for bidirectional UART] **Example:**

```
//ConfigurePB7forSCI9TXD
rx_err_t rx_mpc_set_sci(RX_PORT_PIN(0xB,7));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc.h Full API documentation

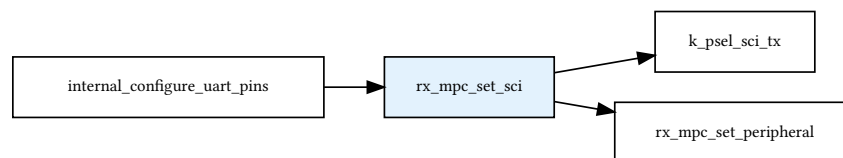


Figure 832: Call graph for rx_mpc_set_sci

Defined in `libs/rx_hal/src/rx_mpc.c:1073`

Since Version 1.0.0

—

rx_mpc_set_riic

```
rx_err_t rx_mpc_set_riic(const rx_port_pin_t pin)
```

Configure pin for RIIC (I2C) bus function.

Convenience wrapper that configures a pin for RIIC operation using PSEL = k_psel_riic_scl (0x0F).

Works for both SCL and SDA pins.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for RIIC (e.g., P12 for SCL)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for RIIC I2C
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with RIIC multiplexing

Post: PFS register contains k_psel_riic_scl (0x0F) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Configure both SCL and SDA pins; external pull-ups required] **Example:**

```
//ConfigureP12(SCL)andP13(SDA)forRIIC0
rx_err_t err=rx_mpc_set_riic(RX_PORT_PIN(1,2));
if(err==k_rx_ok){
err=rx_mpc_set_riic(RX_PORT_PIN(1,3));
}
```

See also: rx_mpc.h Full API documentation

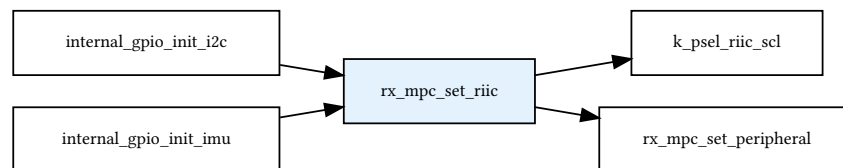


Figure 833: Call graph for rx_mpc_set_riic

Defined in `libs/rx_hal/src/rx_mpc.c:1116`

Since Version 1.0.0

rx_mpc_set_rspi

```
rx_err_t rx_mpc_set_rspi(const rx_port_pin_t pin)
```

Configure pin for RSPI (SPI) bus function.

Convenience wrapper that configures a pin for RSPI operation using PSEL = k_psel_rspi_clk (0x0D). Works for all SPI signals (CLK, COPI, CIPO, SSL).

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for RSPI (e.g., PA0-PA4)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for RSPI SPI
k_rx_err_invalid_arg	Invalid port or pin

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with RSPI multiplexing

Post: PFS register contains k_psel_rspi_clk (0x0D) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Configure all 4 SPI pins (CLK, COPI, CIPO, CS)

Note: Uses OSHWA-approved inclusive terminology] **Example:**

```
//ConfigurePA0-PA4forRSPI0(CLK,COPI,CIPO,SSL0,SSL1)
rx_err_t err=rx_mpc_set_rspi(RX_PORT_PIN(0xA,0));//CLK
if(err==k_rx_ok){
err=rx_mpc_set_rspi(RX_PORT_PIN(0xA,1));//COPI
}
```

See also: rx_mpc.h Full API documentation, rx_spi_comm.h SPI communication driver

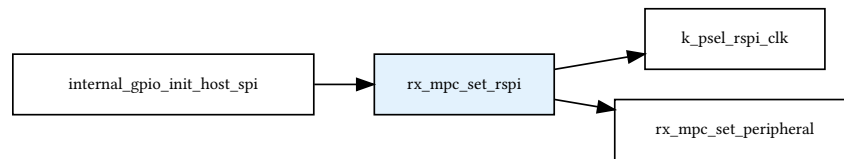


Figure 834: Call graph for rx_mpc_set_rspi

Defined in `libs/rx_hal/src/rx_mpc.c:1162`

Since Version 1.0.0

—

rx_mpc_set_gptw

```
rx_err_t rx_mpc_set_gptw(const rx_port_pin_t pin)
```

Configure pin for GPTW complementary PWM output.

Configure pin for GPTW (General PWM Timer) complementary output.

Convenience wrapper that configures a pin for GPTW operation using PSEL = 0x14. GPTW provides complementary PWM output with dead-time insertion for motor control.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for GPTW function

Returns: Error code from `rx_mpc_set_peripheral()`

Value	Description
k_rx_ok	Pin configured successfully
k_rx_err_invalid_arg	Invalid port or pin
k_rx_err_hw_error	PWPR unlock failed

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with GPTW multiplexing

Post: PFS register contains `k_psel_gptw` (0x14) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Used for 4 GPTW channels (0-3) with 8 total pins (IN2 + IN1 per motor)

Note: Phase staggering configured separately via rx_gptw driver] **Example:**

```
//Configure motor2GPTWIN2 and IN1 pins (PortE)
rx_err_t err = rx_mpc_set_gptw(RX_PORT_PIN(0xE, 3)); //PE3GTIOC2A(IN2)
if(err == k_rx_ok){
    err = rx_mpc_set_gptw(RX_PORT_PIN(8, 6)); //P8.6GTIOC2B(IN1)
}
```

See also: rx_mpc.h Full API documentation with usage examples,
rx_gptw_init_all_staggered() GPTW channel configuration

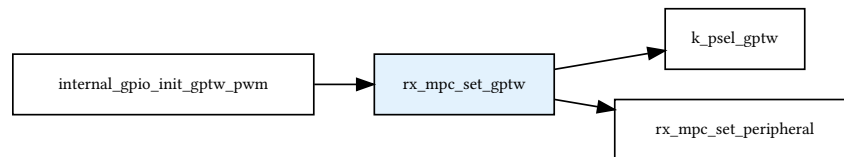


Figure 835: Call graph for rx_mpc_set_gptw

Defined in `libs/rx_hal/src/rx_mpc.c:1210`

Since Version 1.1.0

rx_mpc_set_usb_vbus

```
rx_err_t rx_mpc_set_usb_vbus(const rx_port_pin_t pin)
```

Configure pin for USB VBUS detection.

Configure pin for USB VBUS detection function.

Convenience wrapper that configures a pin for USB VBUS detection using PSEL = 0x11. Required for USB CDC enumeration and power monitoring.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier for USB VBUS (typically P1.6)

Returns: Error code from rx_mpc_set_peripheral()

Value	Description
k_rx_ok	Pin configured successfully
k_rx_err_invalid_arg	Invalid port or pin
k_rx_err_hw_error	PWPR unlock failed

Pre: PCLKB clock must be running, MPC not in module stop

Pre: pin must encode a valid port/pin combination with USB VBUS multiplexing

Post: PFS register contains k_psel_usb_vbus (0x11) for the specified pin

Post: PWPR locked after operation

Note: Thread safety: Not thread-safe

Note: Active-high VBUS detect (pin = 1 when 5V present)

Note: Also requires USB PHY and controller configuration] **Example:**

```
//ConfigureP16forUSB0_VBUSdetection
rx_err_t rx_mpc_set_usb_vbus(RX_PORT_PIN(1,6));
if(err!=k_rx_ok){
//handleerror
}
```

See also: rx_mpc.h Full API documentation with usage examples, rx_usb.h USB controller driver

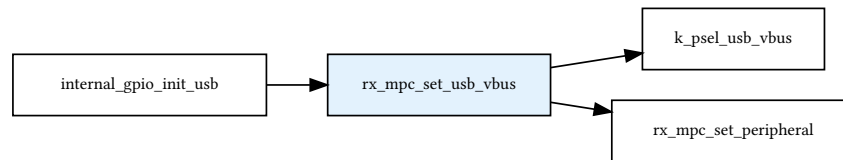


Figure 836: Call graph for rx_mpc_set_usb_vbus

Defined in `libs/rx_hal/src/rx_mpc.c:1259`

Since Version 1.1.0

—

rx_mpc_set_irq

```
rx_err_t rx_mpc_set_irq(const rx_port_pin_t pin)
```

Configure pin for IRQ (external interrupt) function.

Sets pin function to IRQ mode by writing the ISEL bit (bit 6, value 0x40) directly to the PFS register via `internal_write_pfs()`. This bypasses `rx_mpc_set_peripheral()` because ISEL=0x40 exceeds the 5-bit PSEL field maximum (`k_mpc_psel_max = 31`).

Mirrors the approach used by `rx_mpc_set_adc()` for ASEL (bit 7): both functions write their respective bit directly rather than going through the PSEL validation path.

Algorithm Steps:

1. Extract port number and pin number from encoded pin parameter
2. Validate port in range [0, J]
3. Validate pin in range [0, 7]
4. Write ISEL bit (0x40) to PFS register via `internal_write_pfs()`

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (must support IRQ function, P00-P07)

Returns: Error code indicating success or failure

Value	Description
k_rx_ok	Pin configured for IRQ input mode
k_rx_err_invalid_arg	Invalid port or pin number

Pre: Pin must have IRQ multiplexing capability (P00-P07 for IRQ8-15)

Pre: PCLKB clock must be running

Post: Pin ISEL bit (0x40) set in PFS register

Post: PSEL field remains 0 (no peripheral function conflict)

Note: Thread safety: Not thread-safe

Note: Typical pins: P00-P07 for IRQ8-IRQ15 on RX72N

Note: Used by HC-SR04 ultrasonic sensors for echo pulse measurement] **Example:**

```
//ConfigureP00forIRQ8(HC-SR04echoinput)
rx_err_t err=rx_mpc_set_irq(RX_PORT_PIN(0,0));
if(err!=k_rx_ok){
//handleerror
}
//ThenconfigureICUforrising/fallingedge detection
```

See also: rx_mpc_set_adc() Same pattern: writes ASEL bit (0x80) directly, rx_mpc.h Full API documentation with usage examples, rx_hcsr04_icu_configure() Configure ICU after calling this function

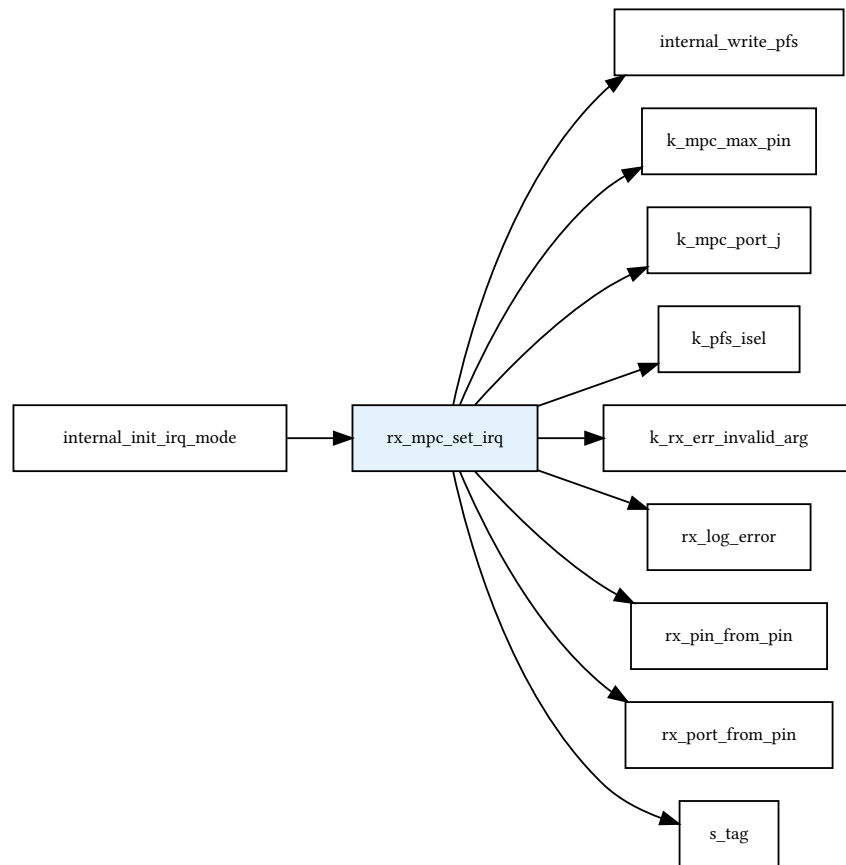


Figure 837: Call graph for rx_mpc_set_irq

Defined in `libs/rx_hal/src/rx_mpc.c:1321`

Since Version 1.2.0 (Issue 296 - IRQ-based HC-SR04 measurement)

—

rx_mtu.c

MTU PWM Driver Implementation for RX72N.

Production-grade implementation of the Multi-Function Timer Unit (MTU) peripheral driver for PWM generation on RX72N. The MTU provides 16-bit timer resolution with multiple outputs per channel.

Includes:

- `"rx_mtu.h"`
- `<stddef.h>`
- `"rx72n_regs.h"`
- `"rx_check.h"`
- `"rx_log.h"`
- `"rx_register_protection.h"`

mtu_constants_t

MTU general constants.

Value	Name	Description
= 8	k_mtu_max_channels	MTU0-MTU4, MTU6-MTU7 (sparse indexing, max index is 7)

—

mtu_module_stop_bits_t

MTU module stop bit positions in MSTPCRA.

Value	Name	Description
= 9	k_mtu_mstopa_mtu0_4	MTU0-MTU4 module stop bit
= 8	k_mtu_mstopa_mtu6_7	MTU6-MTU7 module stop bit

—

mtu_bit_constants_t

Bit manipulation constants.

Value	Name	Description
= 1UL	k_mtu_bit_one	Single bit value for shifts

—

mtu_period_constants_t

Period calculation constants.

Value	Name	Description
= 2	k_mtu_period_divisor	Triangle wave period divisor
= 0xFFFF	k_mtu_period_max	Maximum valid period (16-bit)
= 10	k_mtu_period_min	Minimum valid period
= 0	k_mtu_period_zero	Zero period value

—

mtu_tior_shift_t

TIOR register shift positions.

Value	Name	Description
= 0	k_mtu_tior_low_shift	Low nibble shift (MTIOCA/MTIOCC)
= 4	k_mtu_tior_high_shift	High nibble shift (MTIOCB/MTIOCD)

—

mtu_tior_mask_t

TIOR register mask values.

Value	Name	Description
= 0xF0	k_mtu_tior_low_mask	Mask for low nibble
= 0x0F	k_mtu_tior_high_mask	Mask for high nibble

—

mtu_tior_disabled_t

TIOR output disabled value.

Value	Name	Description
= 0x00	k_mtu_tior_disabled	Output disabled

—

mtu_duty_constants_t

Duty cycle calculation constants.

Value	Name	Description
= 0	k_mtu_duty_min	Minimum duty cycle (0%)
= 100	k_mtu_duty_max	Maximum duty cycle (100%)
= 100	k_mtu_duty_divisor	Divisor for percentage conversion

—

s_tag`const char* s_tag`

—

s_mtu_initialized`bool s_mtu_initialized[k_mtu_max_channels]`

Track which MTU channels have been initialized.

Note: Indexed directly by rx_mtu_channel_t value (sparse: indices 0-4, 6-7)**Warning:** Do not modify directly - use the public API functions*Since Version 1.0.0*

—

s_mtu_period`uint16_t s_mtu_period[k_mtu_max_channels]`

Cached PWM period register values for each MTU channel.

Note: Indexed directly by rx_mtu_channel_t value (sparse: indices 0-4, 6-7)**Warning:** Do not modify directly - value must match hardware TGRA register

Since Version 1.0.0

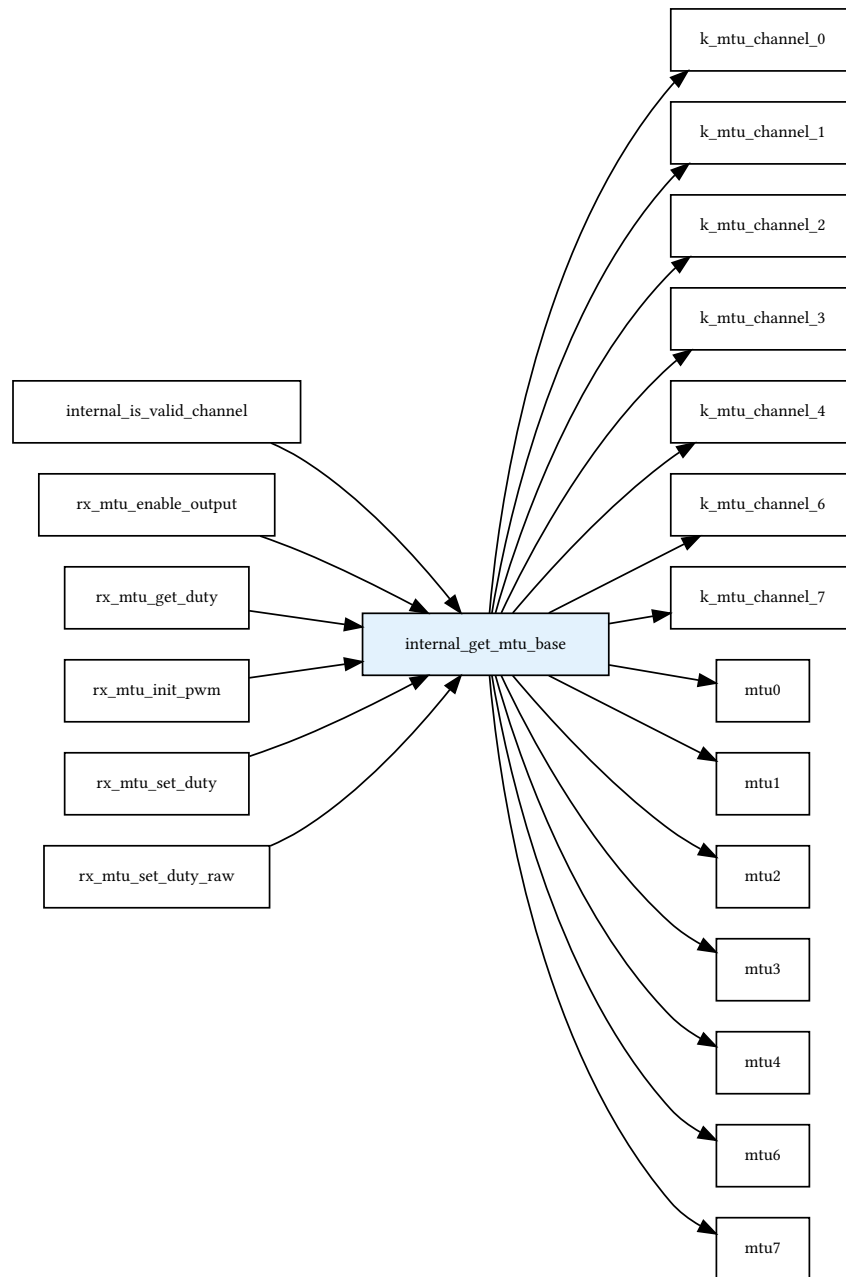
—

internal_get_mtu_base

```
static volatile void * internal_get_mtu_base(const rx_mtu_channel_t channel)
```

Get MTU channel register base address.

Maps an MTU channel enumeration to its hardware register base address. The MTU peripheral has sparse channel numbering (0-4, 6-7 with no 5).

Figure 838: Call graph for `internal_get_mtu_base`

Defined in `libs/rx_hal/src/rx_mtu.c:252`

internal_is_valid_channel

```
static bool internal_is_valid_channel(const rx_mtu_channel_t channel)
```

Check if MTU channel is valid.

Validates an MTU channel by checking if it has a valid register base address. MTU channels 0-4 and 6-7 are valid; channel 5 does not exist.

Parameters:

Direction	Name	Description
in	channel	MTU channel to validate

Returns: bool Validation result

Value	Description
true	Channel is valid (0-4, 6-7)
false	Channel is invalid (5 or out of range)

Note: Thread-safe: Pure function

Note: Delegates to internal_get_mtu_base() for actual validation] **See also:** internal_get_mtu_base() Underlying validation logic

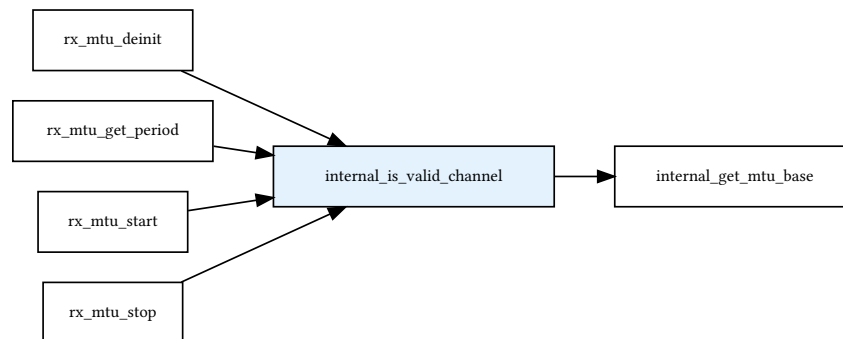


Figure 839: Call graph for internal_is_valid_channel

Defined in `libs/rx_hal/src/rx_mtu.c:294`

Since Version 1.0.0

—

internal_clear_tstr_bit

```
static rx_err_t internal_clear_tstr_bit(volatile rx_mtu_tstr_regs_t *tstr, const uint8_t mask)
```

Clear a counter-start bit in a TSTR register and verify it cleared.

Performs a read-modify-write on the TSTR register to clear the specified bit mask, then reads back the register to confirm the bit was cleared. Returns a hardware error if the bit is still set after the write, which can indicate a peripheral fault.

Parameters:

Direction	Name	Description
in	tstr	Pointer to the MTU timer-start register structure (TSTRA or TSTRB) Must be non-NULL
in	mask	Bitmask of the bit(s) to clear (e.g. k_mtu_tstr_cst0) Must correspond to a valid CSTn bit for the chosen register

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Bit cleared successfully and verified
k_rx_err_invalid_arg	tstr pointer is nullptr
k_rx_err_hw_error	Bit still set after write (hardware fault)

Pre: tstr points to a valid TSTRA or TSTRB hardware register

Pre: mask is a valid CSTn bit mask from mtu_tstr_bits_t

Post: Specified CSTn bit is cleared (counter stopped) on success

Post: tstr register unchanged on error

Note: Not thread-safe: caller must ensure exclusive access to TSTR register

Note: Readback verification adds one extra register read cycle] **See also:** rx_mtu_stop()
Primary caller of this function, k_mtu_tstr_cst0 Counter-start bit constants

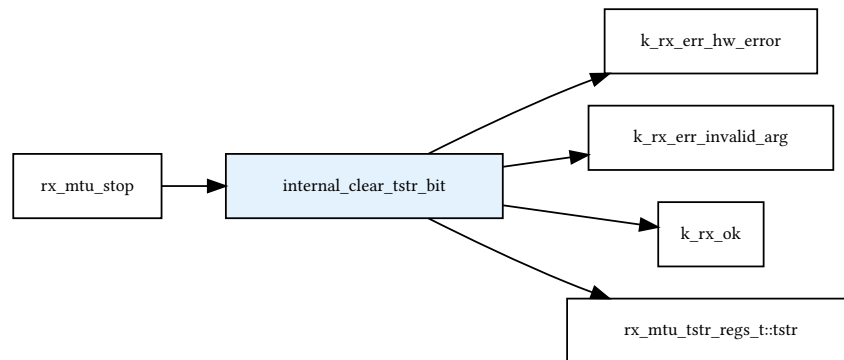


Figure 840: Call graph for internal_clear_tstr_bit

Defined in `libs/rx_hal/src/rx_mtu.c:332`

Since Version 1.0.0

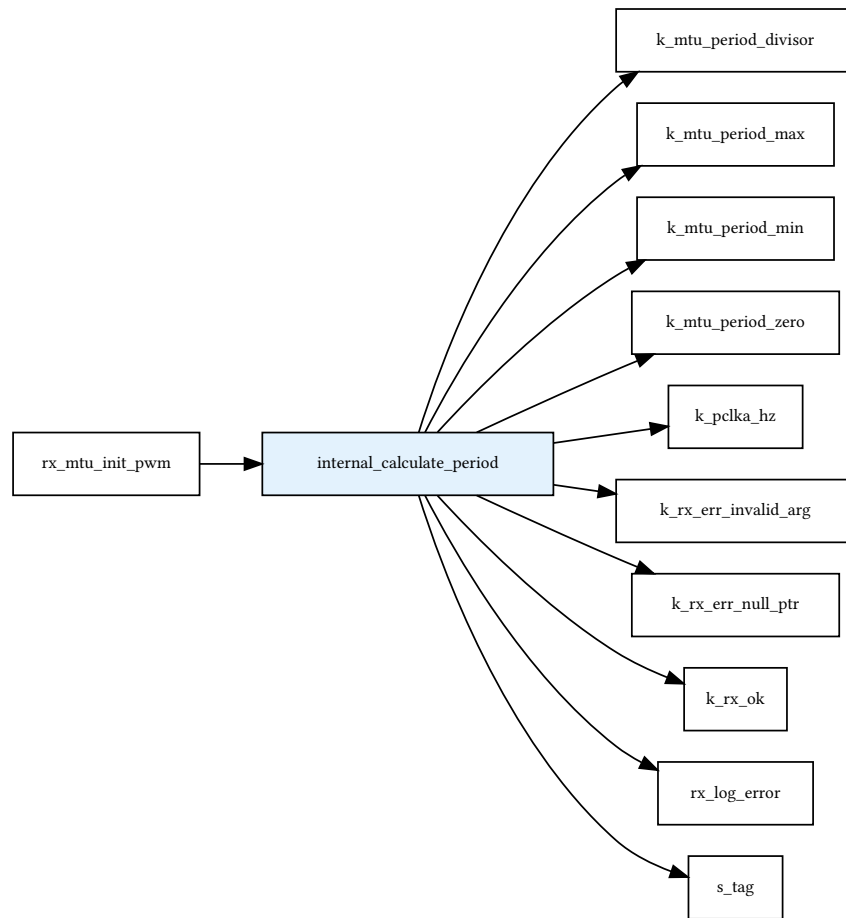
—

internal_calculate_period

```
static rx_err_t internal_calculate_period(const uint32_t frequency_hz, uint16_t *period)
```

Calculate period register value from frequency.

Converts desired PWM frequency to the TGRA (period) register value for PWM Mode 1 (triangle wave, center-aligned).

Figure 841: Call graph for `internal_calculate_period`

Defined in `libs/rx_hal/src/rx_mtu.c:394`

`internal_get_tgr_register`

```
static volatile uint16_t * internal_get_tgr_register(volatile
rx_mtu_channel_regs_t *mtu, const rx_mtu_output_t output)
```

Get TGR register pointer for output channel.

Parameters:

Direction	Name	Description
in	mtu	MTU base pointer
in	output	Output channel

Returns: Pointer to TGR register, or nullptr if invalid

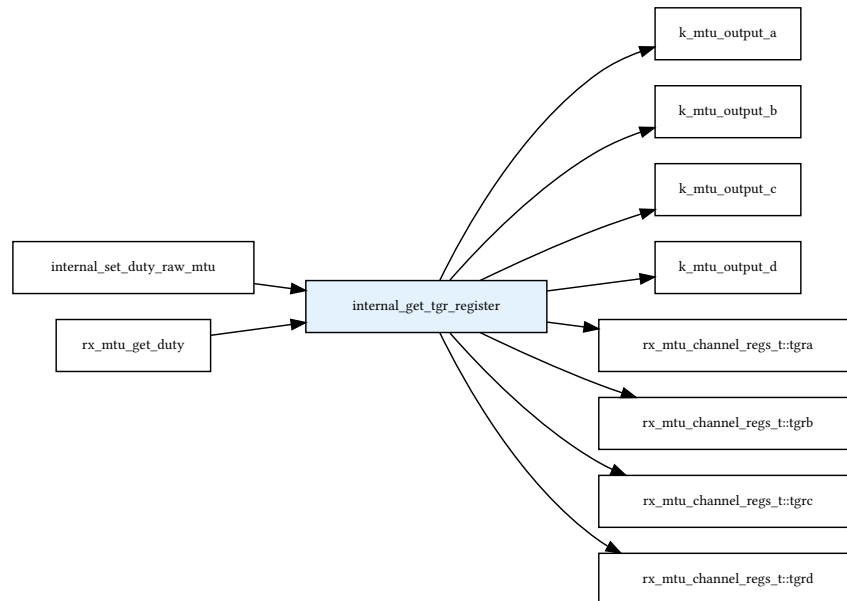


Figure 842: Call graph for internal_get_tgr_register

Defined in `libs/rx_hal/src/rx_mtu.c:434`

internal_set_duty_raw_mtu

```
static rx_err_t internal_set_duty_raw_mtu(volatile rx_mtu_channel_regs_t *mtu,
const rx_mtu_output_t output, const uint16_t duty_count)
```

Write a raw duty count to the TGR register for the specified output.

Resolves the TGR register pointer for the requested output via `internal_get_tgr_register()` and writes `duty_count` to it. The write is buffered by hardware and takes effect at the next PWM period boundary, producing a glitch-free duty cycle update.

Algorithm:

1. Validate mtu pointer is non-NULL
2. Resolve TGR register pointer for output via `internal_get_tgr_register()`
3. Validate resolved TGR pointer is non-NULL
4. Write `duty_count` to TGR register

Parameters:

Direction	Name	Description
in	mtu	Pointer to MTU channel register structure Must be non-NULL Must point to a valid, clock-enabled MTU channel register block
in	output	Output channel whose TGR register to update Valid values: <code>k_mtu_output_a</code> , <code>k_mtu_output_b</code> , <code>k_mtu_output_c</code> , <code>k_mtu_output_d</code>

in	duty_count	Raw count value to write to the TGR register Caller is responsible for clamping to [0, period] before calling
----	------------	---

Returns: rx_err_t Error code

Value	Description
k_rx_ok	duty_count written to TGR register successfully
k_rx_err_invalid_arg	mtu is nullptr, or output has no corresponding TGR register

Pre: mtu points to a valid, initialized MTU channel register block

Pre: duty_count has been validated or clamped by the caller to 0, period

Post: TGR register written with duty_count

Post: Change takes effect at next PWM period boundary (buffered by hardware)

Note: Not thread-safe: caller must ensure exclusive access to the TGR register

Note: Duty count is not bounds-checked; caller must clamp before calling] **Example:**

```
volatilerx_mtu_channel_regs_t*mtu=
(volatilerx_mtu_channel_regs_t*)internal_get_mtu_base(k_mtu_channel_0);
rx_err_terr=internal_set_duty_raw_mtu(mtu,k_mtu_output_b,1500U);
```

See also: internal_get_tgr_register() Resolves TGR pointer from output selector,
rx_mtu_set_duty_raw() Public API caller that clamps duty_count first

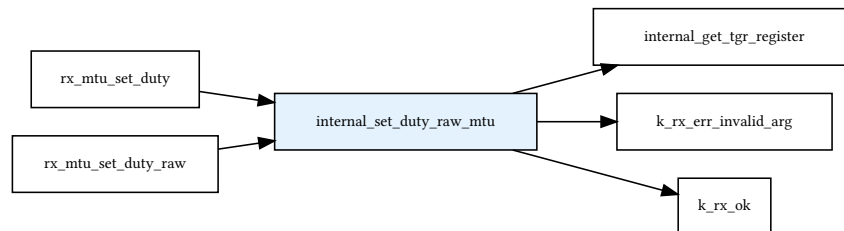


Figure 843: Call graph for internal_set_duty_raw_mtu

Defined in `libs/rx_hal/src/rx_mtu.c:502`

Since Version 1.0.0

rx_mtu_init_pwm

```
rx_err_t rx_mtu_init_pwm(const rx_mtu_channel_t channel, const rx_mtu_config_t
*config)
```

Initialize MTU channel for PWM operation.

Initialize MTU channel for PWM output.

Configures an MTU channel for PWM Mode 1 (triangle wave, center-aligned) operation. Performs complete hardware setup including module clock enable, timer configuration, and automatic start.

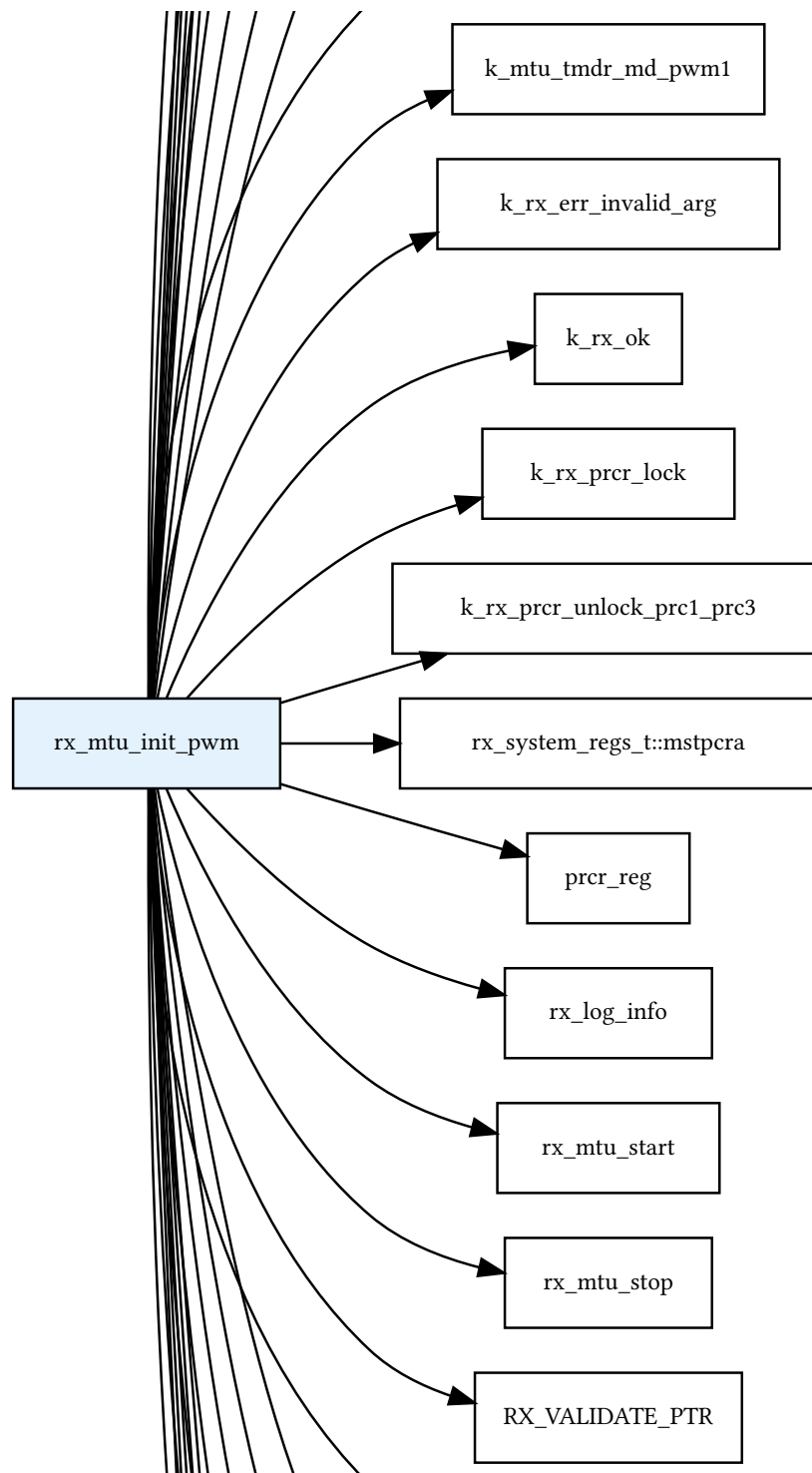


Figure 844: Call graph for rx_mtu_init_pwm

Defined in `libs/rx_hal/src/rx_mtu.c:592`

—

rx_mtu_set_duty

```
rx_err_t rx_mtu_set_duty(const rx_mtu_channel_t channel, rx_mtu_output_t output,
                        const float duty_percent)
```

Set PWM duty cycle as a floating-point percentage.

Set PWM duty cycle.

Converts a percentage value to raw timer counts using the cached period and writes it to the appropriate TGR register. The update is buffered and takes effect at the next PWM period boundary (glitch-free).

Algorithm:

1. Validate channel and initialization state
2. Validate duty_percent in range [0.0, 100.0]
3. Retrieve cached period from s_mtu_period[channel]
4. Compute duty_count = (duty_percent * period) / 100
5. Write duty_count to TGR register via internal_set_duty_raw_mtu()

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output pin to update (k_mtu_output_a through k_mtu_output_d)
in	duty_percent	Duty cycle percentage [0.0, 100.0] 0.0 = always-off (0% duty) 100.0 = always-on (100% duty)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Duty cycle updated successfully
k_rx_err_invalid_arg	Invalid channel or duty_percent out of range
k_rx_err_not_initialized	Channel not initialized via rx_mtu_init_pwm()

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: duty_percent must be in 0.0, 100.0

Post: TGR register updated with new duty count

Post: Change takes effect at next PWM period boundary

Note: Not thread-safe: caller must prevent concurrent access] **Example:**

```
rx_err_t err = rx_mtu_set_duty(k_mtu_channel_0, k_mtu_output_b, 50.0f);
```

See also: `rx_mtu_set_duty_raw()` Set duty using raw count value, `rx_mtu_get_duty()` Read current duty cycle

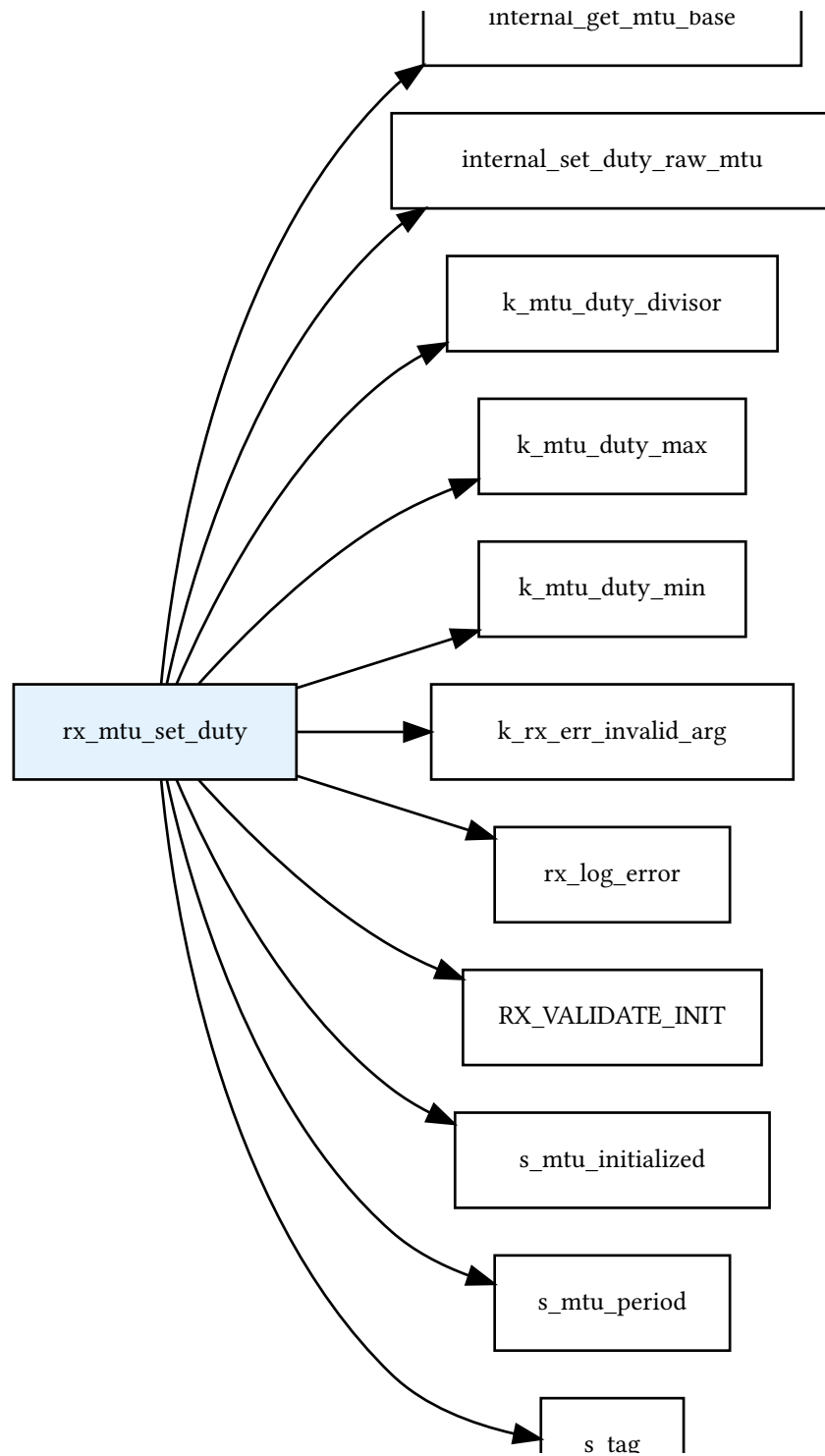


Figure 845: Call graph for rx_mtu_set_duty

Defined in `libs/rx_hal/src/rx_mtu.c:719`

Since Version 1.0.0

—

rx_mtu_set_duty_raw

```
rx_err_t rx_mtu_set_duty_raw(const rx_mtu_channel_t channel, rx_mtu_output_t
output, uint16_t duty_count)
```

Set PWM duty cycle using a raw 16-bit timer count.

Set PWM duty cycle (raw count value).

Directly sets the compare register (TGR) for the specified output to `duty_count` timer ticks. Values larger than the channel period are clamped to the period value (100% duty). The update is buffered.

Use this function when the caller has already converted a duty percentage to counts (e.g. via `rx_mtu_get_period()`) to avoid redundant floating-point multiplication.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output pin (<code>k_mtu_output_a</code> through <code>k_mtu_output_d</code>)
in	duty_count	Raw count value in range <code>[0, period]</code> Values <code>> period</code> are automatically clamped to period

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Duty count written successfully
<code>k_rx_err_invalid_arg</code>	Invalid channel or output
<code>k_rx_err_not_initialized</code>	Channel not initialized

Pre: Channel must be initialized via `rx_mtu_init_pwm()`

Pre: `duty_count` should be in `[0, period]`; values above period are clamped

Post: TGR register written with (possibly clamped) `duty_count`

Post: Change takes effect at next PWM period boundary

Note: Not thread-safe: caller must prevent concurrent access] **Example:**

```
uint16_t period;
rx_mtu_get_period(k_mtu_channel_0, &period);
rx_mtu_set_duty_raw(k_mtu_channel_0, k_mtu_output_b, period/2U); //50%
```

See also: `rx_mtu_set_duty()` Set duty using floating-point percentage,
`rx_mtu_get_period()` Retrieve period for count calculation

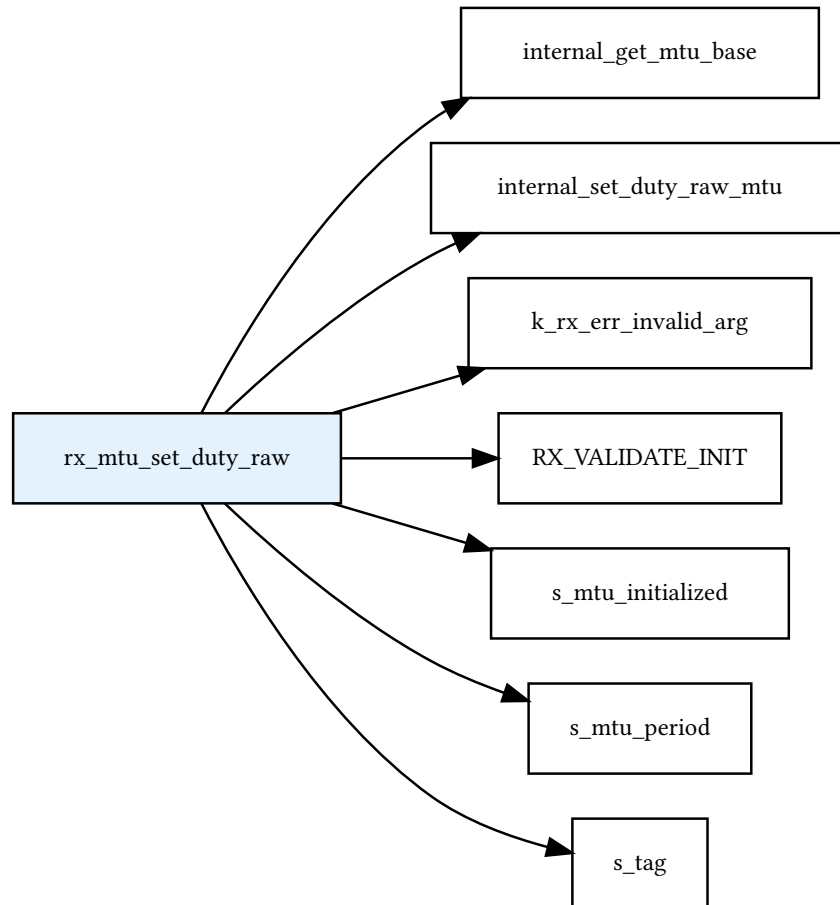


Figure 846: Call graph for `rx_mtu_set_duty_raw`

Defined in `libs/rx_hal/src/rx_mtu.c:783`

Since Version 1.0.0

—

`rx_mtu_get_duty`

```
rx_err_t rx_mtu_get_duty(const rx_mtu_channel_t channel, const rx_mtu_output_t
output, float *duty_percent)
```

Read the current PWM duty cycle as a floating-point percentage.

Get current duty cycle.

Reads the TGR register for the specified output and converts the raw count to a percentage using the cached period value.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output to read (k_mtu_output_a through k_mtu_output_d)
out	duty_percent	Pointer to store percentage result [0.0, 100.0] Must be non-NULL

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, duty_percent updated
k_rx_err_null_ptr	duty_percent is nullptr
k_rx_err_invalid_arg	Invalid channel or output
k_rx_err_not_initialized	Channel not initialized

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: duty_percent must point to valid float storage

Post: *duty_percent contains duty cycle in 0.0, 100.0 on success

Post: Hardware TGR register unchanged

Note: Not thread-safe: concurrent set_duty calls may cause torn reads] **Example:**

```
floatduty;  
rx_err_t err=rx_mtu_get_duty(k_mtu_channel_0,k_mtu_output_b,&duty);
```

See also: rx_mtu_set_duty() Set duty using percentage, rx_mtu_get_period() Retrieve period count

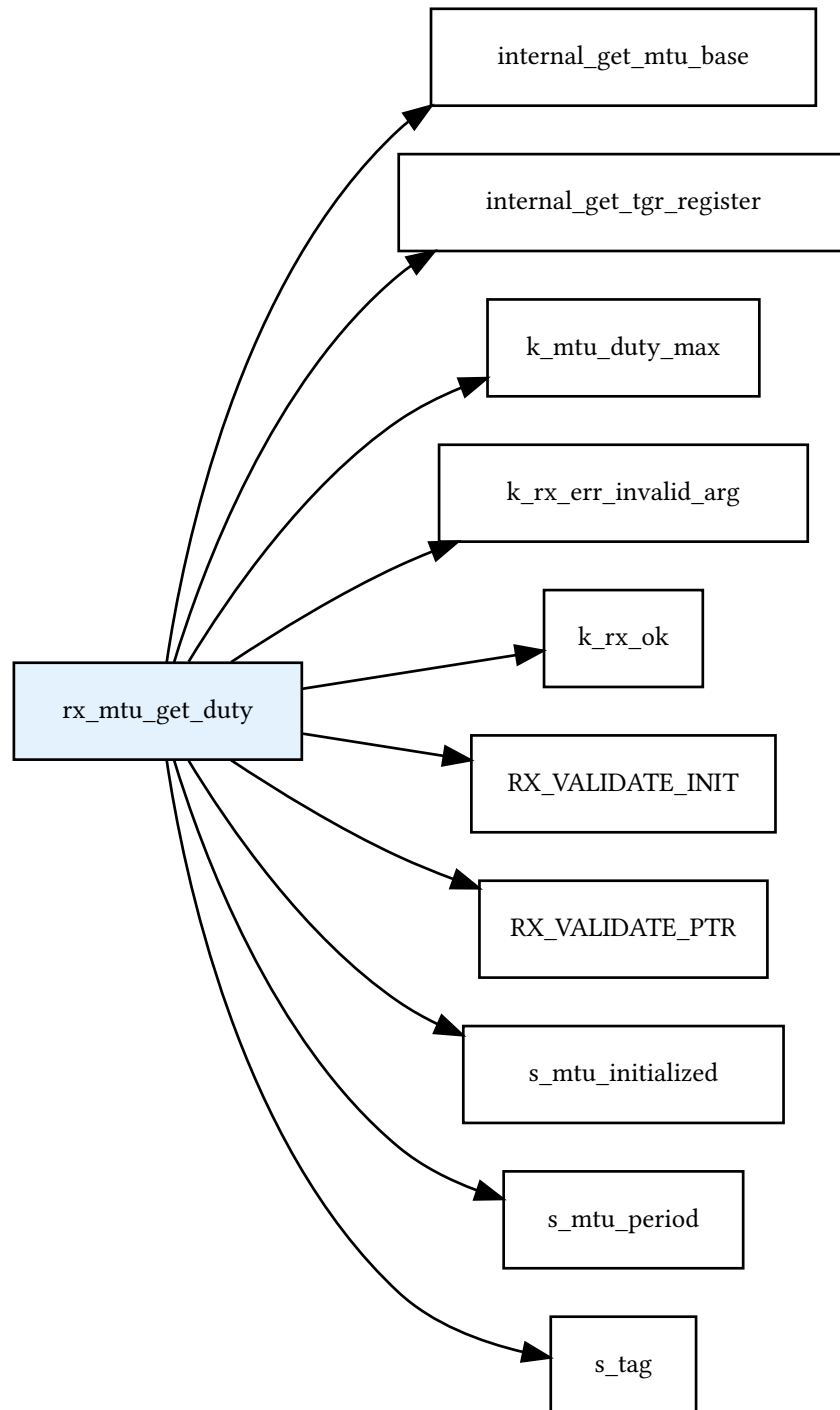


Figure 847: Call graph for rx_mtu_get_duty

Defined in `libs/rx_hal/src/rx_mtu.c:843`

Since Version 1.0.0

—

rx_mtu_get_period

```
rx_err_t rx_mtu_get_period(const rx_mtu_channel_t channel, uint16_t
*period_count)
```

Get PWM period count for MTU channel.

Get PWM period count.

Returns the cached period value (TGRA) from initialization. Useful for calculating raw duty counts from percentages without floating-point.

Parameters:

Direction	Name	Description
in	channel	MTU channel to query
out	period_count	Pointer to store period count value

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, period_count contains valid value
k_rx_err_null_ptr	period_count is nullptr
k_rx_err_invalid_arg	Invalid channel
k_rx_err_not_initialized	Channel not initialized

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: period_count must point to valid uint16_t storage

Post: *period_count contains the TGRA register value on success

Post: Hardware registers unchanged

Note: Thread-safe: reads from static cache, no hardware access] **Example:**

```
uint16_t period;
rx_err_t err = rx_mtu_get_period(k_mtu_channel_0, &period);
if (err == k_rx_ok) {
    uint16_t half_duty = period / 2U; // 50%
    rx_mtu_set_duty_raw(k_mtu_channel_0, k_mtu_output_b, half_duty);
}
```

See also: rx_mtu.h Complete API documentation, rx_mtu_set_duty_raw() Use period count to set duty

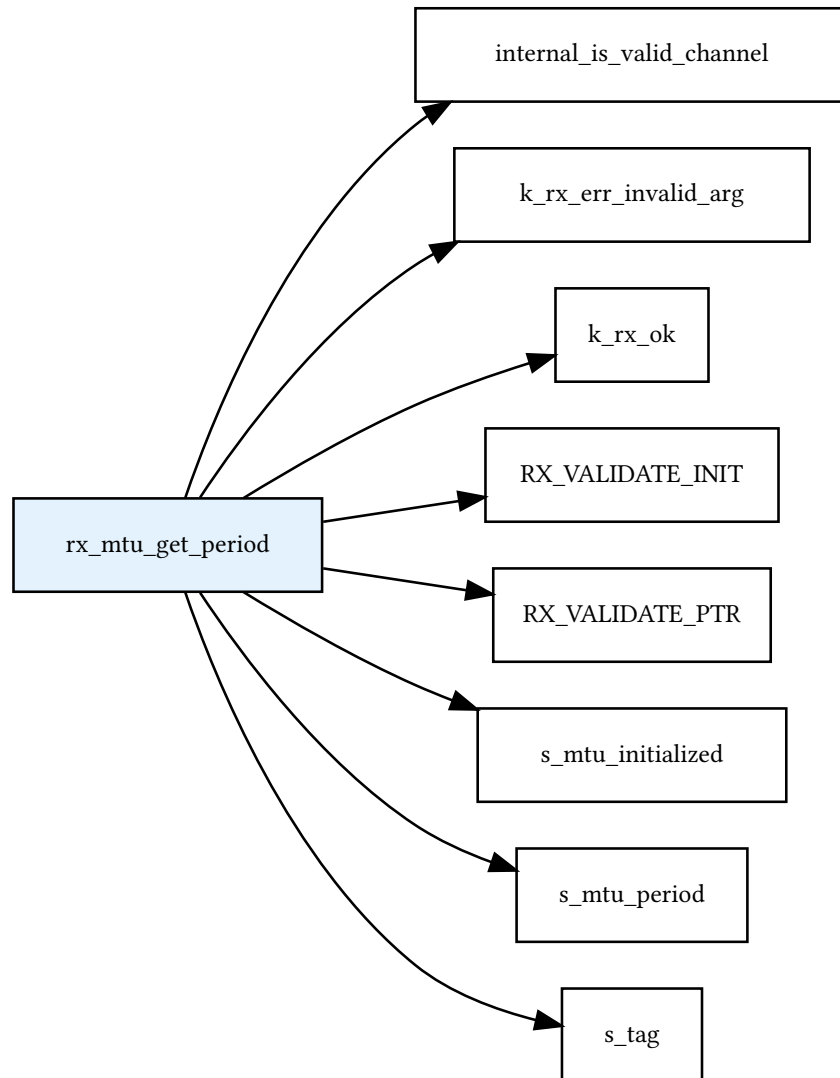


Figure 848: Call graph for rx_mtu_get_period

Defined in `libs/rx_hal/src/rx_mtu.c:906`

Since Version 1.0.0

—

rx_mtu_enable_output

```
rx_err_t rx_mtu_enable_output(const rx_mtu_channel_t channel, rx_mtu_output_t output, const bool enable)
```

Enable or disable a single MTU PWM output pin.

Enable or disable PWM output.

Controls whether the MTIOCA/B/C/D pin actively drives the PWM waveform or is held in its initial state (off). Modifies the TIORH or TIORL register nibble corresponding to the requested output without disturbing other outputs.

Parameters:

Direction	Name	Description
in	channel	MTU channel (0-4, 6-7)
in	output	Output to control (k_mtu_output_a through k_mtu_output_d)
in	enable	true to enable PWM output, false to disable (hold initial state)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Output state changed successfully
k_rx_err_invalid_arg	Invalid channel or output
k_rx_err_not_initialized	Channel not initialized

Pre: Channel must be initialized via rx_mtu_init_pwm()

Pre: output must be a valid rx_mtu_output_t enum value

Post: TIORH or TIORL nibble updated; timer continues running

Post: Other outputs on the same channel are unchanged

Note: Not thread-safe: caller must prevent concurrent register access

Note: Timer keeps running; only output pin drive changes

Warning: Outputs may remain HIGH when disabled, depending on last compare state]

Example:

```
rx_mtu_enable_output(k_mtu_channel_0,k_mtu_output_a,false);//disable
```

See also: rx_mtu_init_pwm() Initialize channel before calling, rx_mtu_stop() Halt the entire timer

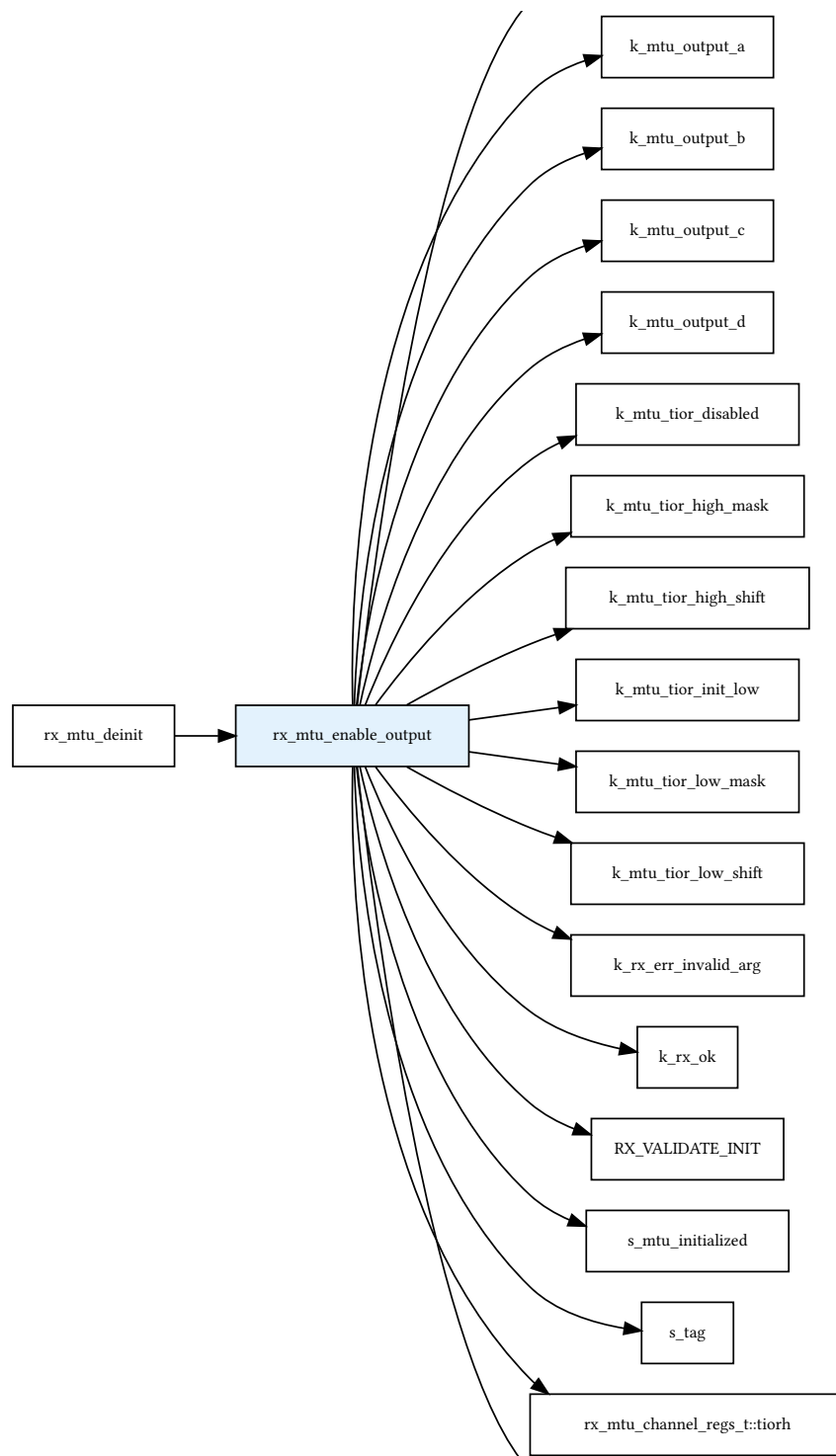


Figure 849: Call graph for rx_mtu_enable_output

Defined in `libs/rx_hal/src/rx_mtu.c:965`

Since Version 1.0.0

—

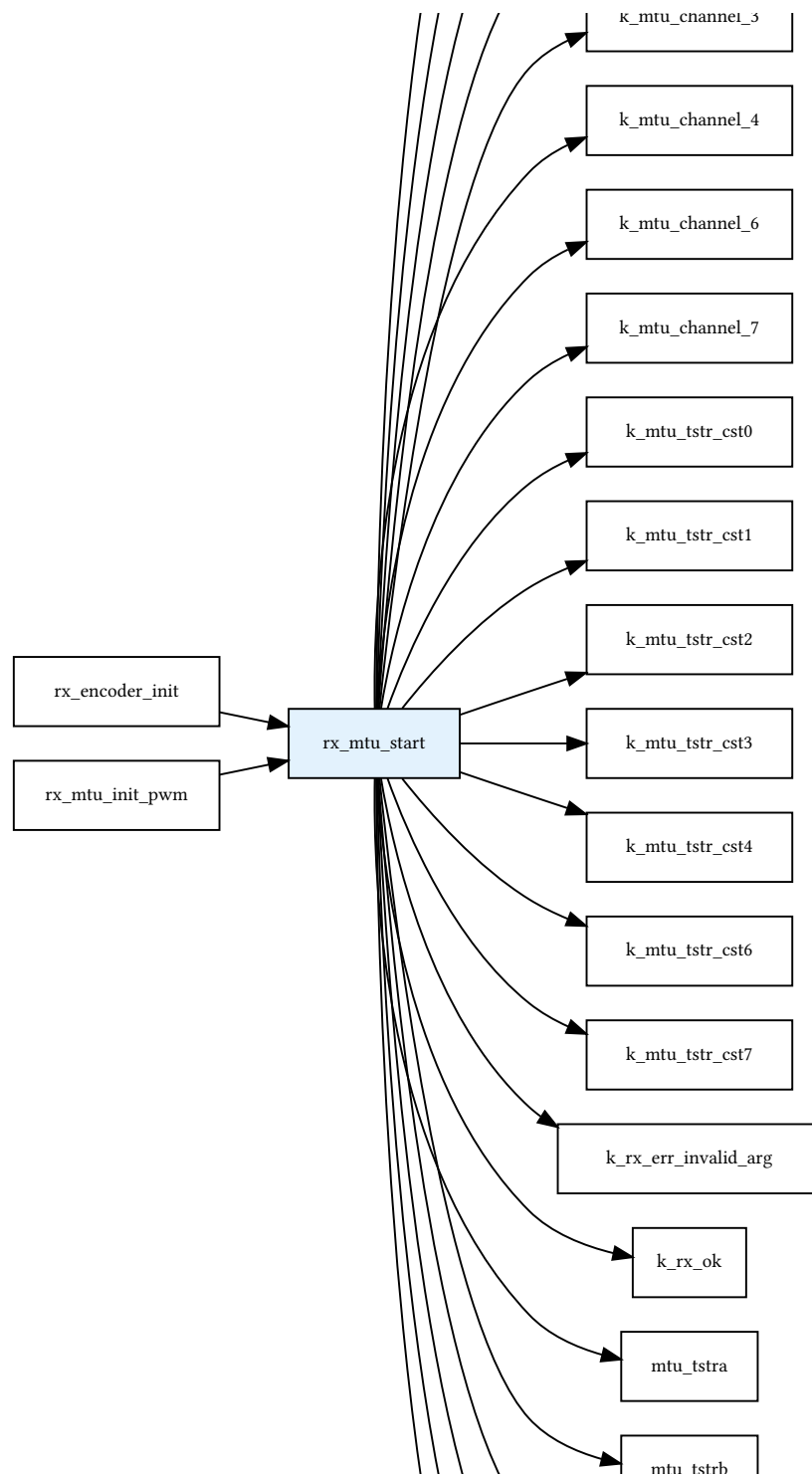
rx_mtu_start

```
rx_err_t rx_mtu_start(const rx_mtu_channel_t channel)
```

Start MTU counter (begin PWM generation).

Start MTU timer.

Sets the Count Start (CSTn) bit in the appropriate TSTR register to begin counter operation.
Channels 0-4 use TSTRA, channels 6-7 use TSTRB.

Figure 850: Call graph for `rx_mtu_start`

Defined in `libs/rx_hal/src/rx_mtu.c:1035`

—

rx_mtu_stop

```
rx_err_t rx_mtu_stop(const rx_mtu_channel_t channel)
```

Stop MTU counter (halt PWM generation).

Stop MTU timer.

Clears the Count Start (CSTn) bit in the appropriate TSTR register to halt counter operation. Counter value and output state are preserved.

Parameters:

Direction	Name	Description
in	channel	MTU channel to stop

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Timer stopped successfully
k_rx_err_invalid_arg	Invalid channel
k_rx_err_hw_error	Failed to clear TSTR bit (hardware issue)

Pre: None (can be called on uninitialized channel)

Post: Timer stopped, counter preserved

Post: Output pins hold current state

Note: Does NOT check initialization flag (safe for use during init)

Note: Idempotent: Safe to call when already stopped

Warning: Outputs may remain HIGH - disable outputs for safe state] **See also:** rx_mtu_start()
Resume the timer, rx_mtu_enable_output() Disable outputs

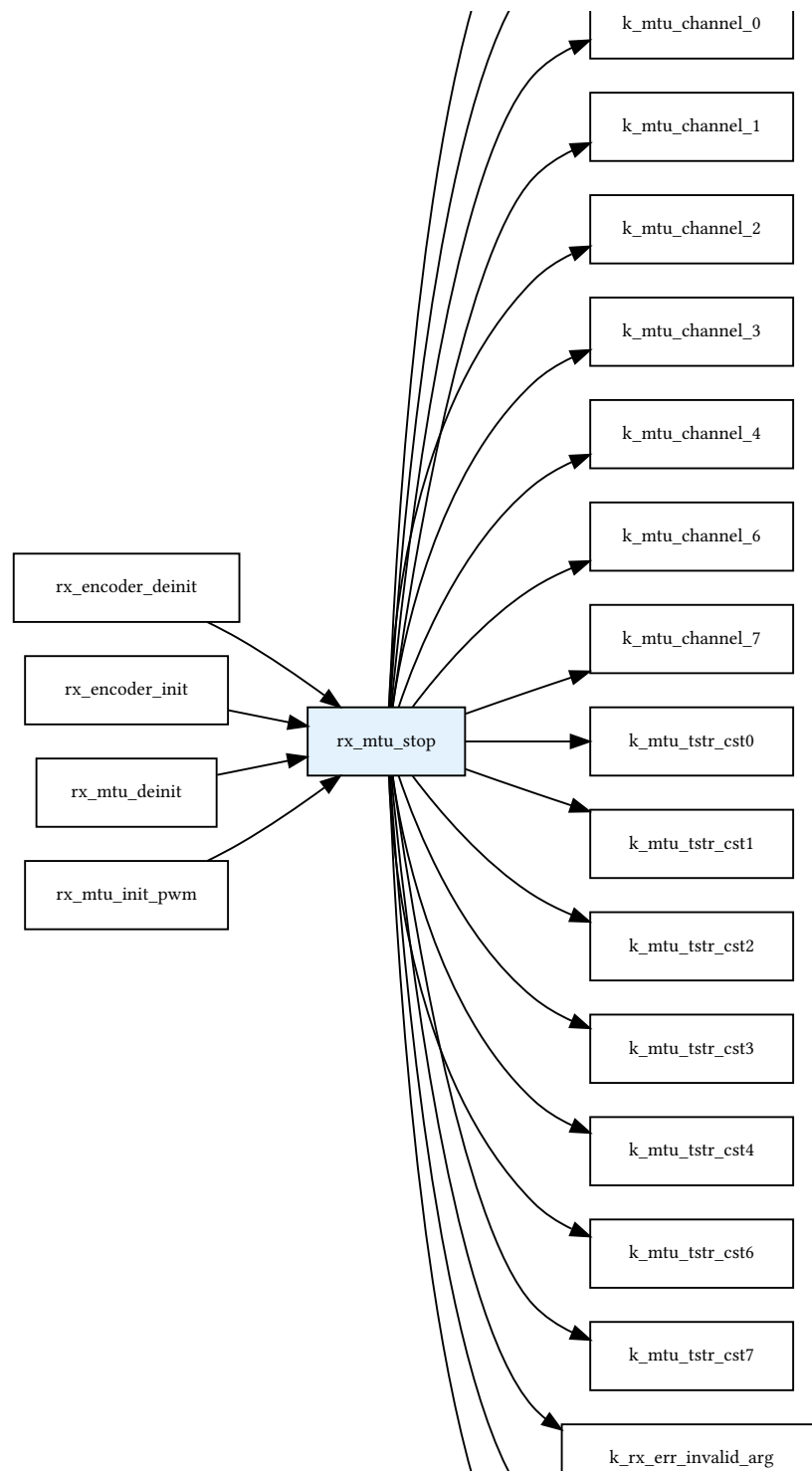


Figure 851: Call graph for rx_mtu_stop

Defined in `libs/rx_hal/src/rx_mtu.c:1102`

Since Version 1.0.0

—

rx_mtu_deinit

```
rx_err_t rx_mtu_deinit(const rx_mtu_channel_t channel)
```

Deinitialize an MTU PWM channel.

Deinitialize MTU channel.

Performs an orderly shutdown of the specified MTU channel:

1. Stop the timer counter (`rx_mtu_stop()`)
2. Disable all four outputs (A, B, C, D) via `rx_mtu_enable_output()`
3. Clear the cached period to 0
4. Clear the initialized flag

After this call the channel may be safely re-initialized with different parameters via `rx_mtu_init_pwm()`.

Parameters:

Direction	Name	Description
in	channel	MTU channel to deinitialize (0-4, 6-7)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Channel successfully deinitialized
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_hw_error</code>	Hardware error while stopping timer

Pre: Motor or load driven by this channel must be stopped externally before call

Pre: channel must be a valid MTU channel (0-4, 6-7)

Post: Timer stopped and outputs disabled

Post: `s_mtu_initializedchannel == false`

Post: `s_mtu_periodchannel == k_mtu_period_zero`

Note: Not thread-safe: do not call while another task is accessing this channel

Note: Idempotent with respect to re-initialization (can call init again after)

Warning: Ensure motor is at rest before calling to avoid abrupt stop] **Example:**

```
rx_err_t err = rx_mtu_deinit(k_mtu_channel_0);
```

See also: rx_mtu_init_pwm() Re-initialize after deinit, rx_mtu_stop() Stop timer without full teardown

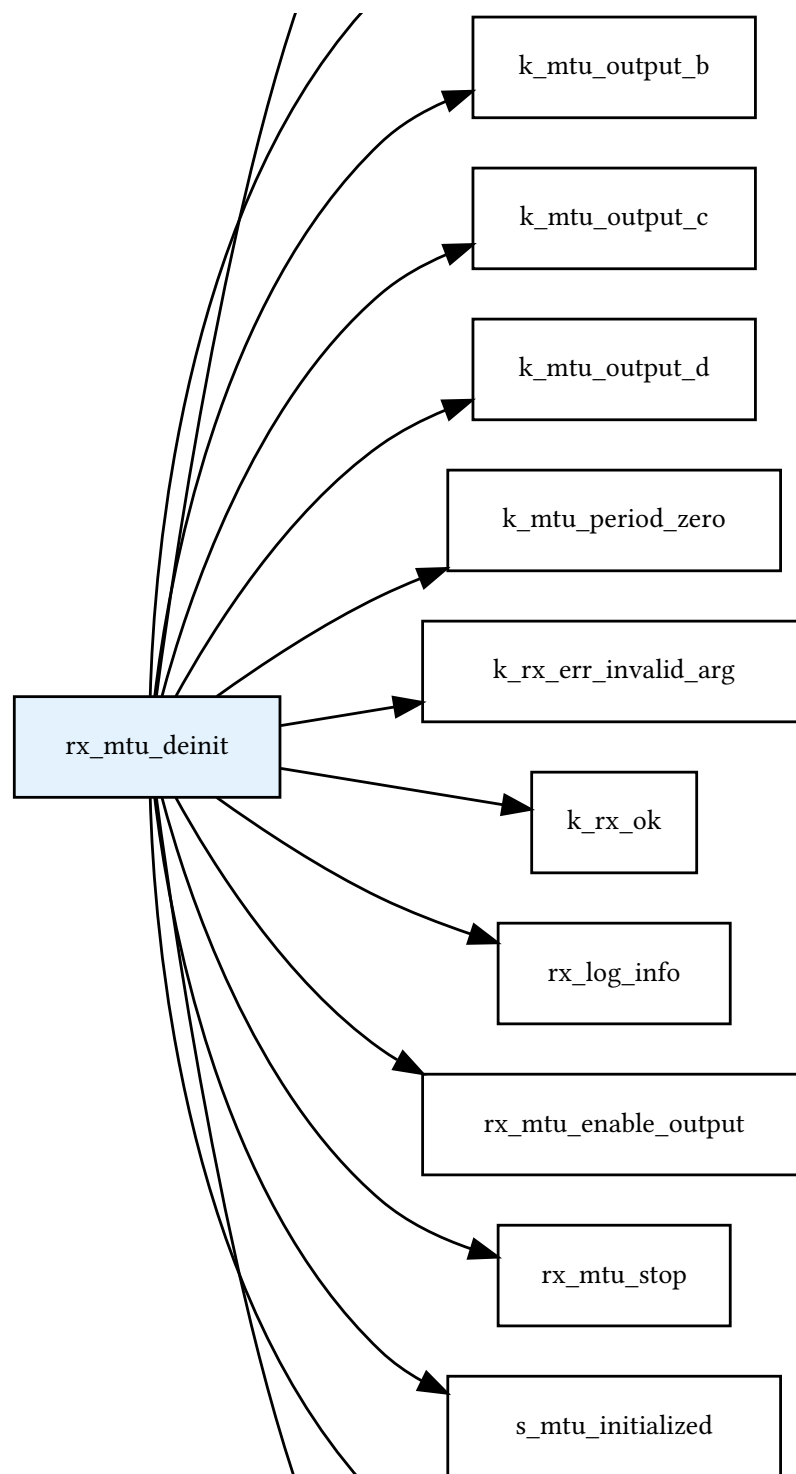


Figure 852: Call graph for rx_mtu_deinit

Defined in `libs/rx_hal/src/rx_mtu.c:1180`

Since Version 1.0.0

—

rx_poeg.c

POEG Motor Fault Protection Driver Implementation.

Implements hardware-level emergency PWM output disable for the STAR motor controller. Each of the 4 DRV8263H motor drivers has its nFAULT output connected to a GTETRQ pin, which feeds a POEG group. When a fault is detected, POEG immediately disables GPTW PWM output (Hi-Z).

The ISR handlers (vectors 188-191) run at priority 14 and:

1. Clear the ICU interrupt request flag
2. Set fault flags in the shared_data motor state
3. Trigger e-stop via shared_data
4. Log the fault for diagnostics

STAR Team

2026-02-10

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_poeg.h"
- "rx72n_gptw_regs.h"
- "rx72n_icu_regs.h"
- "rx72n_poeg_regs.h"
- "rx_check.h"
- "rx_err.h"
- "rx_log.h"
- "shared_data.h"

poeg_icu_constants_t

ICU configuration constants for POEG interrupts.

Value	Name	Description
= 0	k_icu_ir_clear	Write 0 to clear IR flag
= 1	k_ier_bit_enable_base	Base for IER bit shift
= 8	k_ier_bits_per_reg	8 bits per IER register

Since Version 1.0.0

—

poeg_init_config_t

POEG initialization configuration values.

Combined register value for each POEG group initialization:

- PIDE = 1 (enable pin detection)
- INV = 1 (invert for active-low nFAULT)
- NFEN = 1 (enable noise filter)

- NFCS = 01 (PCLKB/8, 400ns sampling)

Value	Name	Description
= (k_poeg_pide_enable k_poeg_inv_invert k_poeg_nfen_enable k_poeg_nfcs_pclkb_div8)	k_poeg_init_value	

Since Version 1.0.0

—

poeg_gtintad_values_t

Pre-computed GTINTAD values for each GPTW-POEG link.

Each GPTW channel's GTINTAD register is configured to:

- Select the corresponding POEG group (A/B/C/D)
- Enable dead-time error detection (GRPDTE)
- Enable simultaneous low output detection (GRPABL)

Value	Name	Description
= (k_gptw_gtintad_grp_a k_gptw_gtintad_grpdte k_gptw_gtintad_grpabl)	k_gtintad_motor_0	
= (k_gptw_gtintad_grp_b k_gptw_gtintad_grpdte k_gptw_gtintad_grpabl)	k_gtintad_motor_1	
= (k_gptw_gtintad_grp_c k_gptw_gtintad_grpdte k_gptw_gtintad_grpabl)	k_gtintad_motor_2	
= (k_gptw_gtintad_grp_d k_gptw_gtintad_grpdte k_gptw_gtintad_grpabl)	k_gtintad_motor_3	

Since Version 1.0.0

—

s_tag

```
const char* s_tag
```

Module log tag.

—

s_poeg_groups

```
volatile rx_poegg_regs_t* const s_poeg_groups[k_poeg_motor_count]
```

POEG group register accessor lookup table (indexed by motor).

—

s_gptw_channels

```
volatile rx_gptw_channel_regs_t* const s_gptw_channels[k_poeg_motor_count]
```

GPTW channel register accessor lookup table (indexed by motor).

—

s_gtintad_values

```
const uint32_t s_gtintad_values[k_poeg_motor_count]
```

GTINTAD values lookup table (indexed by motor).

—

s_poeg_vectors

```
const uint16_t s_poeg_vectors[k_poeg_motor_count]
```

POEG interrupt vector numbers (indexed by motor).

—

internal_enable_poeg_irq

```
static void internal_enable_poeg_irq(uint16_t vector, uint8_t priority)
```

Configure ICU for a single POEG interrupt vector.

Parameters:

Direction	Name	Description
in	vector	ICU vector number (188-191)
in	priority	IPR priority level (1-15)

Pre: ICU registers accessible

Post: Vector enabled at specified priority

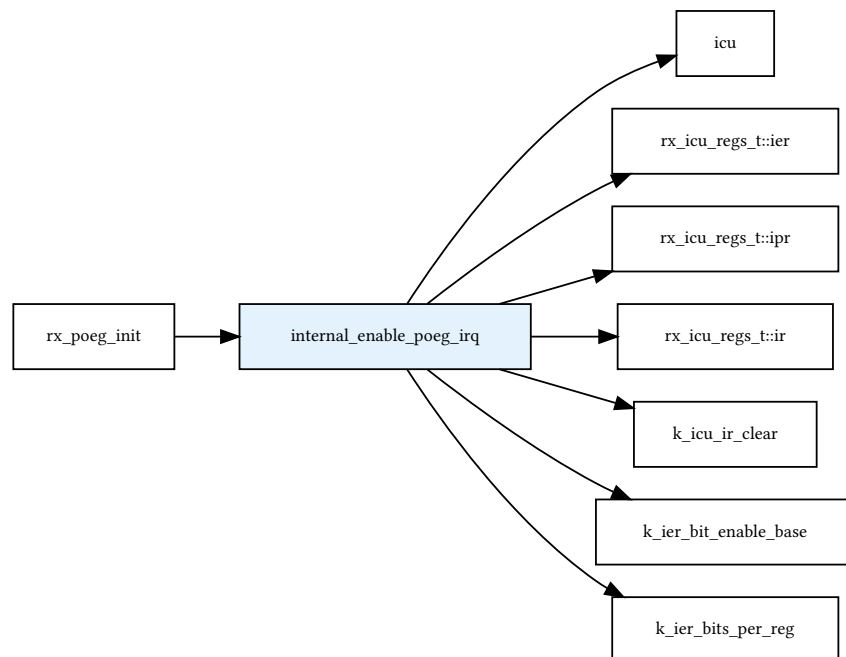


Figure 853: Call graph for `internal_enable_poeg_irq`

Defined in `libs/rx_hal/src/rx_poeg.c:150`

Since Version 1.0.0

—

internal_configure_gtintad

```
static void internal_configure_gtintad(uint8_t motor_index)
```

Configure GPTW GTINTAD for POEG group linkage.

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)

Pre: GPTW channel initialized and outputs not active during GRP write

Post: GTINTAD configured with POEG group and detection enables

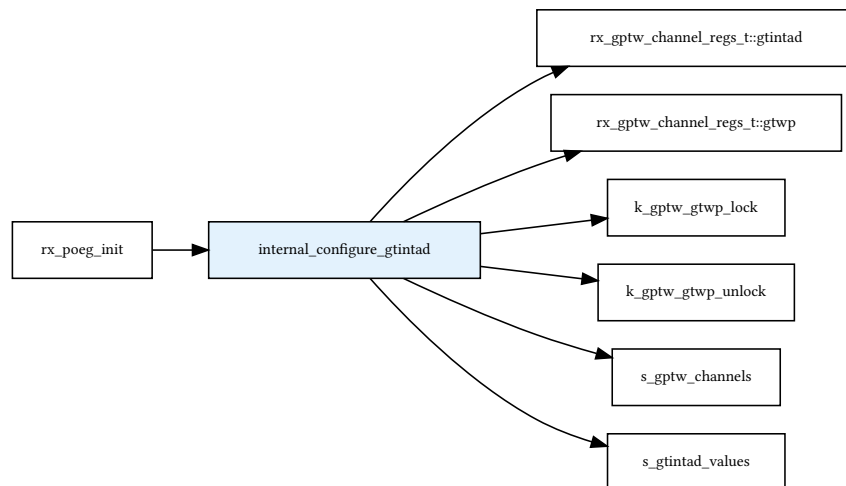


Figure 854: Call graph for `internal_configure_gtintad`

Defined in `libs/rx_hal/src/rx_poeg.c:177`

Since Version 1.0.0

—

internal_poeg_isr_handler

```
static void internal_poeg_isr_handler(uint8_t motor_index, uint16_t vector)
```

Common POEG ISR handler for all groups.

Called from each group-specific ISR. Reads the fault register, identifies the fault source, and updates `shared_data`.

Parameters:

Direction	Name	Description
in	<code>motor_index</code>	Motor index (0-3) identifying which group faulted
in	<code>vector</code>	ICU vector number for clearing IR flag

Pre: Called from ISR context only

Post: `shared_data` motor fault flags updated

Post: IR flag cleared

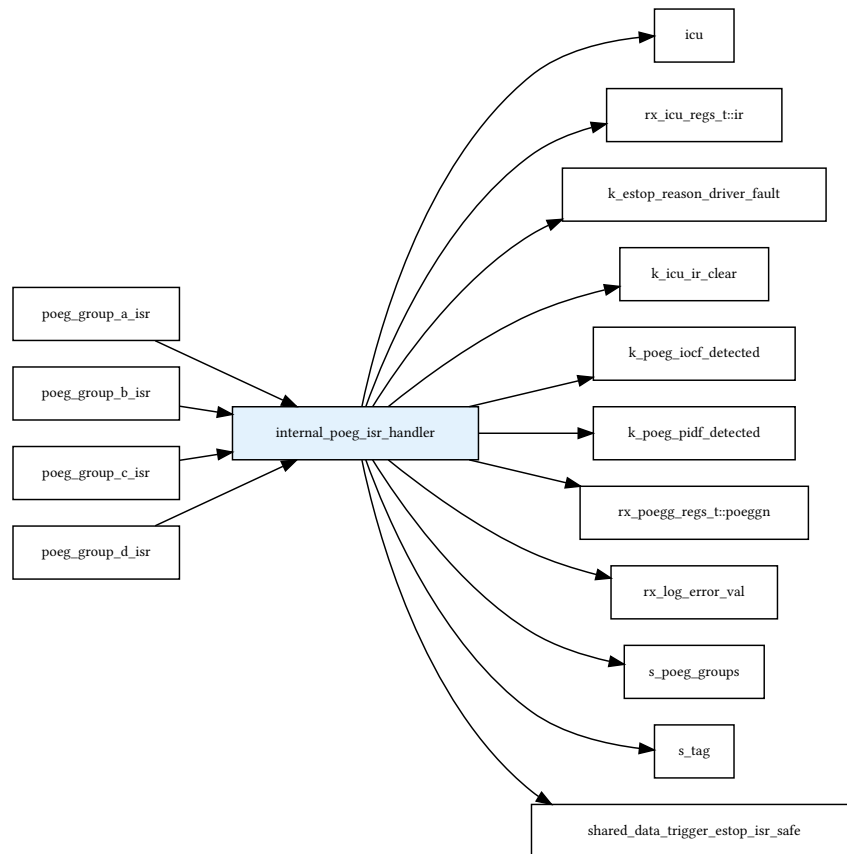


Figure 855: Call graph for `internal_poeg_isr_handler`

Defined in `libs/rx_hal/src/rx_poeg.c:207`

Since Version 1.0.0

—

poeg_group_a_isr

```
void poeg_group_a_isr(void)
```

POEG Group A ISR (Motor 0, vector 188).

Handles fault interrupt from GTETRGA (P15 / DRV8263H Motor 0 nFAULT). GPTW0 PWM output has already been disabled by hardware before this ISR runs.

Pre: POEG Group A initialized via rx_poeg_init()

Post: Motor 0 fault logged and e-stop triggered

Note: Thread Safety: ISR context, runs at priority 14

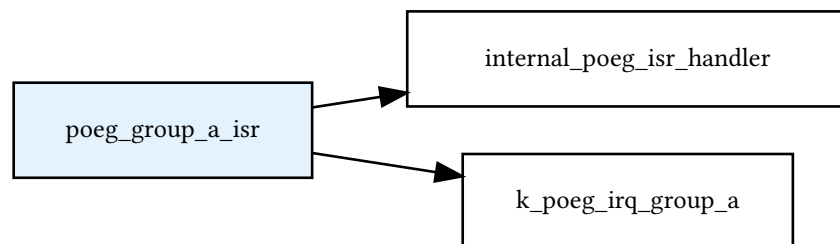


Figure 856: Call graph for poeg_group_a_isr

Defined in `libs/rx_hal/src/rx_poeg.c:247`

Since Version 1.0.0

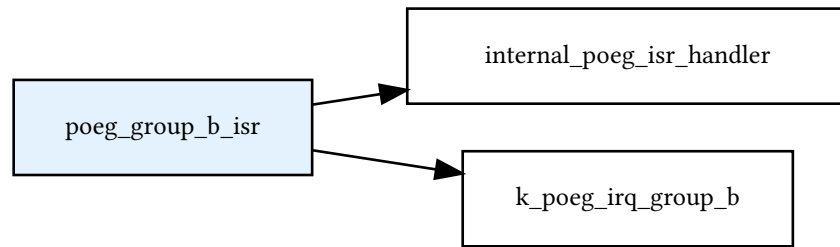
—

poeg_group_b_isr

```
void poeg_group_b_isr(void)
```

POEG Group B ISR (Motor 1, vector 189).

See also: `poeg_group_a_isr()` for full documentation

Figure 857: Call graph for `poeg_group_b_isr`

Defined in `libs/rx_hal/src/rx_poeg.c:258`

Since Version 1.0.0

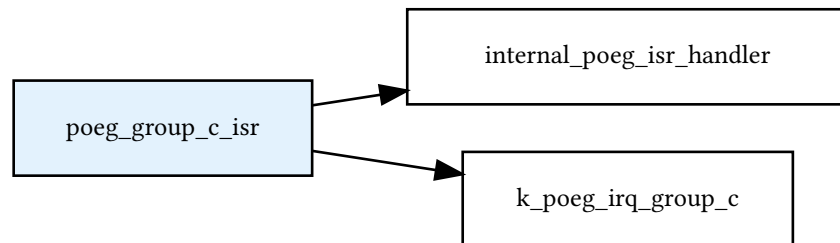
—

`poeg_group_c_isr`

```
void poeg_group_c_isr(void)
```

POEG Group C ISR (Motor 2, vector 190).

See also: `poeg_group_a_isr()` for full documentation

Figure 858: Call graph for `poeg_group_c_isr`

Defined in `libs/rx_hal/src/rx_poeg.c:269`

Since Version 1.0.0

—

`poeg_group_d_isr`

```
void poeg_group_d_isr(void)
```

POEG Group D ISR (Motor 3, vector 191).

See also: `poeg_group_a_isr()` for full documentation

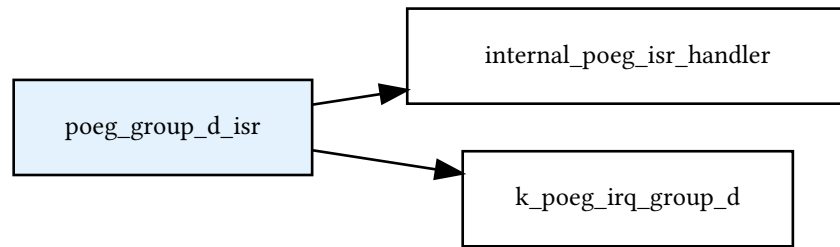


Figure 859: Call graph for poeg_group_d_isr

Defined in `libs/rx_hal/src/rx_poeg.c:280`

Since Version 1.0.0

—

rx_poeg_init

```
rx_err_t rx_poeg_init(void)
```

Initialize POEG fault protection for all 4 motor channels.

Configures POEG Groups A-D for hardware motor fault protection:

1. Enables external pin detection (PIDE) on each group
2. Enables noise filter (NFEN) with PCLKB/8 sampling (400ns)
3. Inverts input (INV) since DRV8263H nFAULT is active-low
4. Links each GPTW channel to its POEG group via GTINTAD
5. Configures ICU interrupts for vectors 188-191 at priority 14

After initialization, a DRV8263H fault will:

- Immediately disable GPTW PWM output (hardware, nanoseconds)
- Fire POEG ISR which sets fault flags in shared_data

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All 4 POEG groups initialized successfully
k_rx_err_hw_init_failed	POEG register write verification failed

Pre: GPTW module enabled (MSTPCRB.MSTPB22 = 0)

Pre: GTETRQ pins configured via MPC (Pin.c)

Pre: shared_data initialized

Post: POEG Groups A-D enabled with pin detection + noise filter

Post: GPTW0-3 GTINTAD linked to POEG Groups A-D respectively

Post: ICU vectors 188-191 enabled at priority 14

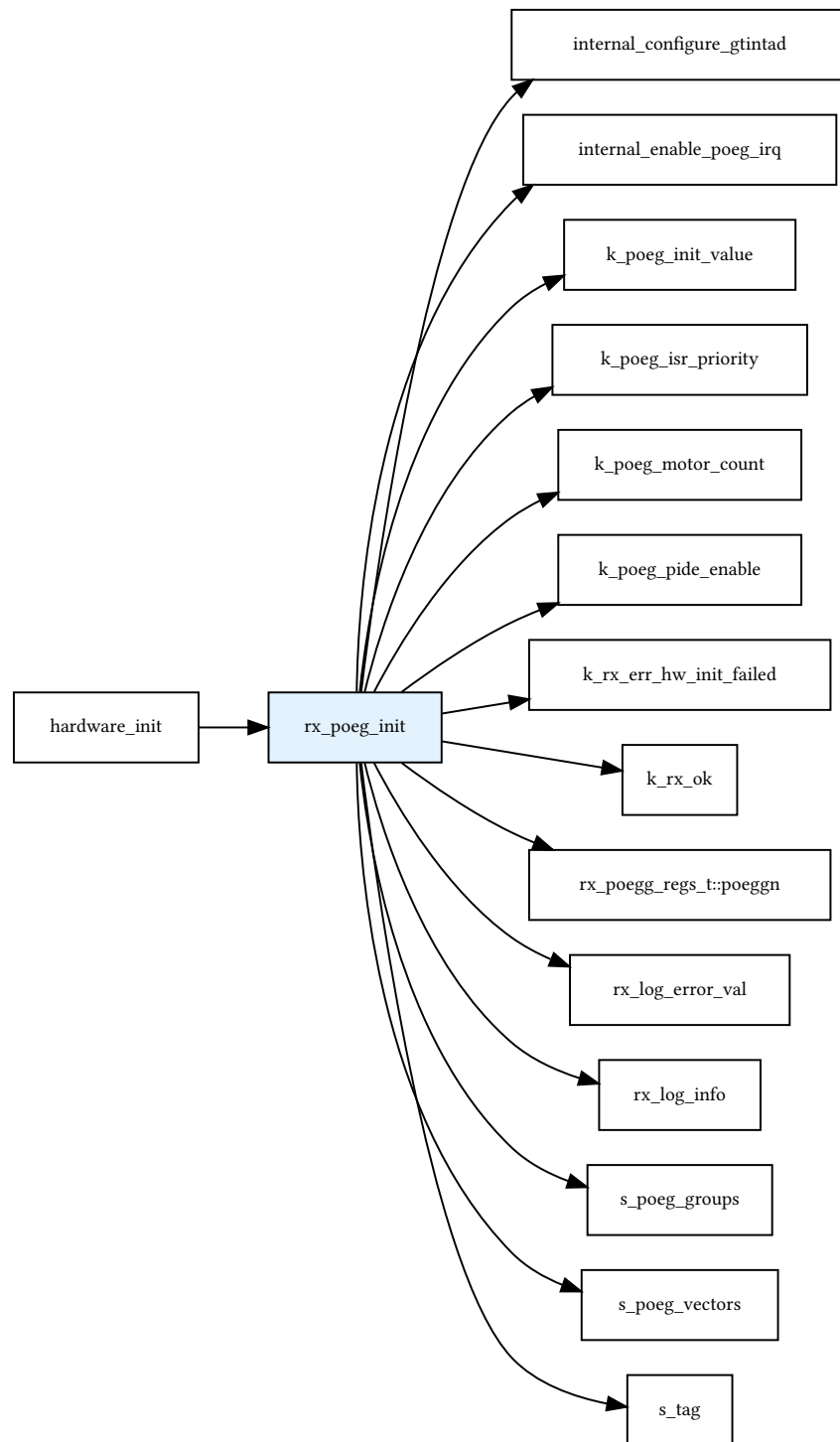
Post: Any existing nFAULT assertion will trigger immediately

Note: Thread Safety: Call once during hardware_init, before motor tasks start

Warning: PIDE enable bit is write-once after reset cannot be disabled

NASA Power of 10 Compliance:: Rule 1: No goto, setjmp, recursion Rule 2: Loop bounded by k_poeg_motor_count (4) Rule 3: No dynamic allocation Rule 5: 3 preconditions, 4 postconditions Rule 7: Return value checked Rule 8: All constants as C23 typed enums

See also: rx_poeg_clear_fault() Clear fault after resolution, rx_poeg_get_fault_status() Query fault state

Figure 860: Call graph for `rx_poeg_init`

Defined in `libs/rx_hal/src/rx_poeg.c:290`

Since Version 1.0.0

—

rx_poeg_get_fault_status

```
rx_err_t rx_poeg_get_fault_status(uint8_t motor_index, bool *fault_active)
```

Check if a motor has an active POEG fault.

Reads the POEGGn register for the specified motor's POEG group and returns whether the PIDF (port input detection flag) is set.

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)
out	fault_active	true if POEG fault is active

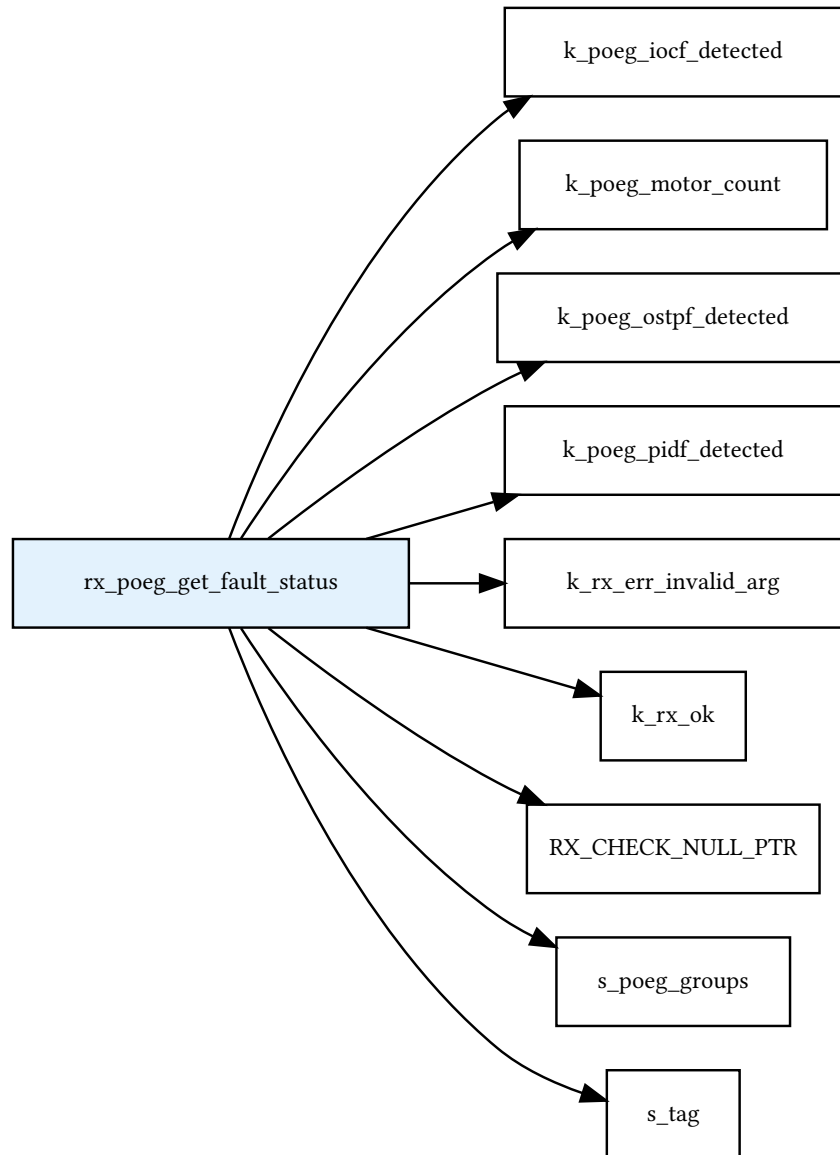
Returns: rx_err_t Error code

Value	Description
k_rx_ok	Status read successfully
k_rx_err_null_ptr	fault_active is nullptr
k_rx_err_invalid_arg	motor_index >= 4

Pre: POEG initialized via rx_poeg_init()

Post: No side effects on hardware state

Note: Thread Safety: Safe to call from any context (read-only register access)

Figure 861: Call graph for `rx_poeg_get_fault_status`

Defined in `libs/rx_hal/src/rx_poeg.c:316`

Since Version 1.0.0

—

`rx_poeg_clear_fault`

```
rx_err_t rx_poeg_clear_fault(uint8_t motor_index)
```

Clear POEG fault and re-enable motor output.

Attempts to clear the PIDF flag for the specified motor's POEG group. The flag can only be cleared when the underlying fault condition has been resolved (i.e., DRV8263H nFAULT deasserted / GTETRG pin high).

After clearing, GPTW output re-enables at the end of the next GPTW counting cycle (delay of up to 1 PWM period, typically 50us at 20kHz).

Parameters:

Direction	Name	Description
in	motor_index	Motor index (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Fault cleared successfully
k_rx_err_invalid_arg	motor_index >= 4
k_rx_err_busy	Fault condition still active (nFAULT still asserted)

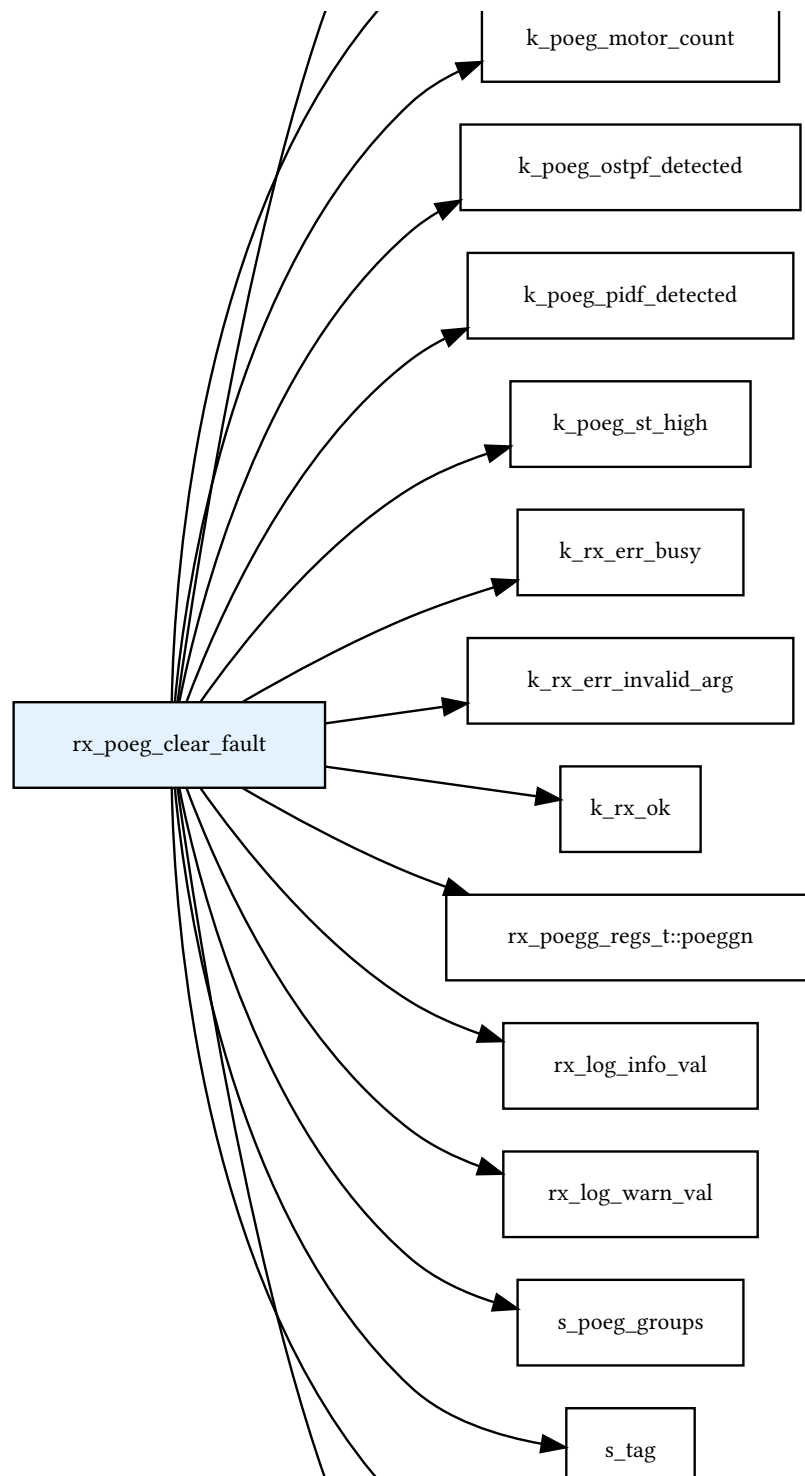
Pre: Underlying fault resolved (DRV8263H nFAULT deasserted)

Post: If successful, PIDF flag cleared and GPTW output will re-enable

Post: shared_data fault_flagsmotor_index cleared

Note: Motor control task should verify estop is cleared before resuming PWM

Warning: Returns k_rx_err_busy if fault source is still active

Figure 862: Call graph for `rx_poeg_clear_fault`

Defined in `libs/rx_hal/src/rx_poeg.c:331`

Since Version 1.0.0

—

rx_poeg_software_stop

```
rx_err_t rx_poeg_software_stop(uint8_t motor_index)
```

Trigger software emergency stop on a specific motor.

Sets the SSF (Software Stop Flag) in the POEGGn register, immediately disabling GPTW PWM output for the specified motor. This provides a firmware-controlled emergency stop independent of hardware faults.

Parameters:

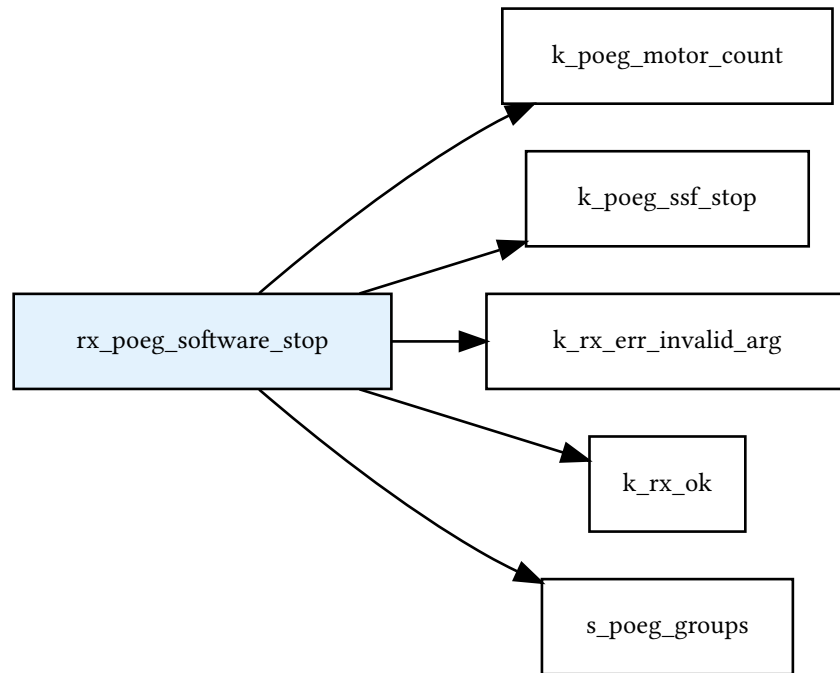
Direction	Name	Description
in	motor_index	Motor index (0-3)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Software stop activated
k_rx_err_invalid_arg	motor_index >= 4

Post: GPTW output immediately disabled for specified motor

Post: SSF flag set in POEGGn register] **See also:** rx_poeg_clear_software_stop() Release software stop

Figure 863: Call graph for `rx_poeg_software_stop`

Defined in `libs/rx_hal/src/rx_poeg.c:360`

Since Version 1.0.0

—

rx_poeg_clear_software_stop

```
rx_err_t rx_poeg_clear_software_stop(uint8_t motor_index)
```

Release software emergency stop on a specific motor.

Clears the SSF (Software Stop Flag) in the POEGGn register. GPTW output re-enables at end of next counting cycle if no other fault flags (PIDF, IOCF, OSTPF) are active.

Parameters:

Direction	Name	Description
in	<code>motor_index</code>	Motor index (0-3)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Software stop released
<code>k_rx_err_invalid_arg</code>	<code>motor_index >= 4</code>

Post: SSF flag cleared in POEGGn register] **See also:** rx_poeg_software_stop() Activate software stop

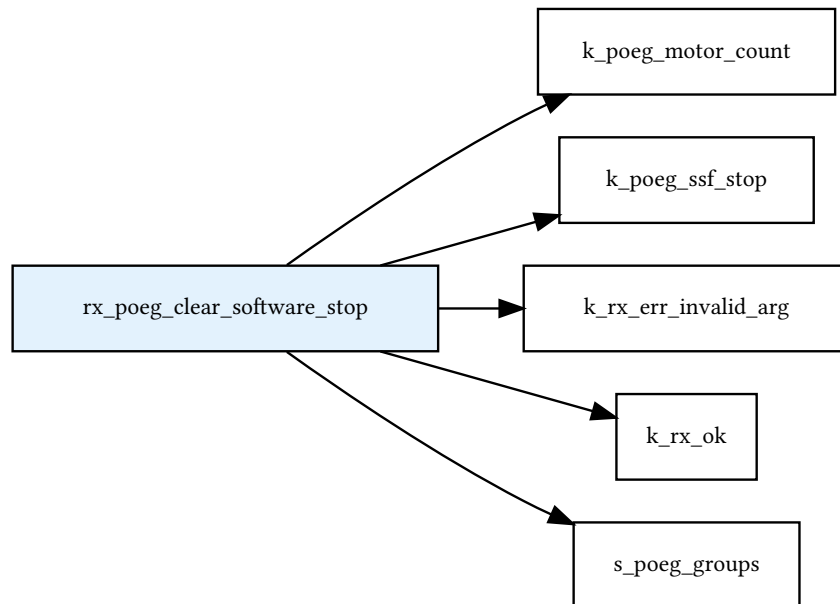


Figure 864: Call graph for rx_poeg_clear_software_stop

Defined in `libs/rx_hal/src/rx_poeg.c:370`

Since Version 1.0.0

—

rx_tpu.c

TPU HAL Driver Implementation for Phase Counting Mode.

Implementation of the TPU hardware abstraction layer for RX72N quadrature encoder phase counting. This driver configures TPU channels 1, 2, 4, and 5 for hardware-based encoder position tracking with 4x resolution.

Includes:

- "rx_tpu.h"
- <stddef.h>
- "rx72n_regs.h"
- "rx_check.h"
- "rx_log.h"
- "rx_register_protection.h"

tpu_phase_channel_count_t

Number of phase-counting-capable TPU channels.

TPU1, TPU2, TPU4, and TPU5 support phase counting mode. TPU0 and TPU3 do NOT support phase counting.

Value	Name	Description
= 4	k_tpu_phase_channel_count	Channels 1, 2, 4, 5

Since Version 1.0.0

—

tpu_channel_index_t

Array index mapping for phase-counting channels.

Maps the non-contiguous channel numbers (1, 2, 4, 5) to contiguous array indices (0-3) for the s_tpu_initialized[] tracking array.

Value	Name	Description
= 0	k_tpu_idx_ch1	TPU1 -> index 0 (rear-left encoder)
= 1	k_tpu_idx_ch2	TPU2 -> index 1 (rear-right encoder)
= 2	k_tpu_idx_ch4	TPU4 -> index 2 (available)
= 3	k_tpu_idx_ch5	TPU5 -> index 3 (available)

Since Version 1.0.0

—

tpu_tmdr_reset_t

TMDR register reset value (normal mode).

Value	Name	Description
= 0x00	k_tpu_tmdr_reset	Normal operation mode (reset default)

Since Version 1.0.0

—

tpu_tcr_phase_count_t

TCR register value for phase counting mode.

In phase counting mode, the TPSC and CKEG fields in TCR are ignored because the external clock pins provide the counting signal. CCLR is set to disabled (free-running counter).

Value	Name	Description
= 0x00	k_tpu_tcr_phase_count	Free-running: TPSC=0, CKEG=0, CCLR=0

Since Version 1.0.0

—

tpu_tier_disable_t

TIER register value with all interrupts disabled.

Value	Name	Description
= 0x00	k_tpu_tier_all_disabled	All interrupt sources disabled

Since Version 1.0.0

—

tpu_tcmt_zero_t

TCNT register zero value.

Value	Name	Description
= 0x0000	k_tpu_tcmt_zero	Counter cleared to zero

Since Version 1.0.0

—

s_tag

`const char* s_tag`

Logging tag for TPU module.

Note: Statically allocated, never modified

Since Version 1.0.0

—

s_tpu_initialized

`bool s_tpu_initialized[k_tpu_phase_channel_count]`

Track which phase-counting channels are initialized.

Warning: Do not modify directly - use public API functions

Since Version 1.0.0

—

internal_channel_to_index

```
static rx_err_t internal_channel_to_index(const rx_tpu_channel_t channel, uint8_t *index)
```

Map TPU channel number to array index.

Converts the non-contiguous channel enumeration (1, 2, 4, 5) to a contiguous array index (0-3) for use with `s_tpu_initialized[]`.

Parameters:

Direction	Name	Description
in	channel	TPU channel identifier
out	index	Pointer to store the array index

Returns: `rx_err_t` Error code

Value	Description
-------	-------------

k_rx_ok	Valid channel, index written
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5

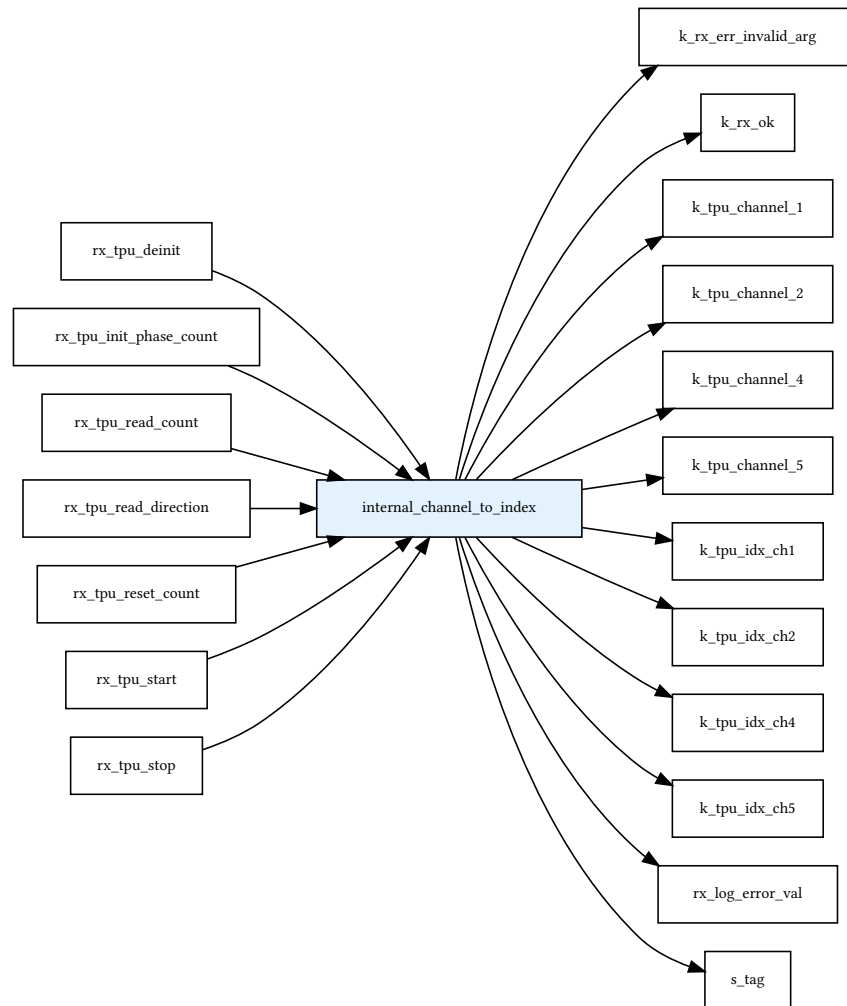
Pre: index must be non-NULL (caller responsibility)

Pre: channel must be a valid rx_tpu_channel_t enumeration value

Post: *index contains valid array index on success

Post: *index is unchanged on k_rx_err_invalid_arg

Note: Thread Safety: Safe - pure function with no shared state

Figure 865: Call graph for `internal_channel_to_index`

Defined in `libs/rx_hal/src/rx_tpu.c:249`

Since Version 1.0.0

—

internal_get_regs

```
static volatile rx_tpu_regs_t * internal_get_regs(const rx_tpu_channel_t channel)
```

Get register pointer for a phase-counting TPU channel.

Maps a TPU channel identifier to the corresponding hardware register base address pointer. Only returns pointers for phase-counting-capable channels (1, 2, 4, 5).

Parameters:

Direction	Name	Description
in	channel	TPU channel identifier (1, 2, 4, or 5)

Returns: Volatile pointer to TPU channel register structure

Value	Description
Non-NULL	Valid pointer for channels 1, 2, 4, 5
NULL	Invalid channel (not phase-counting capable)

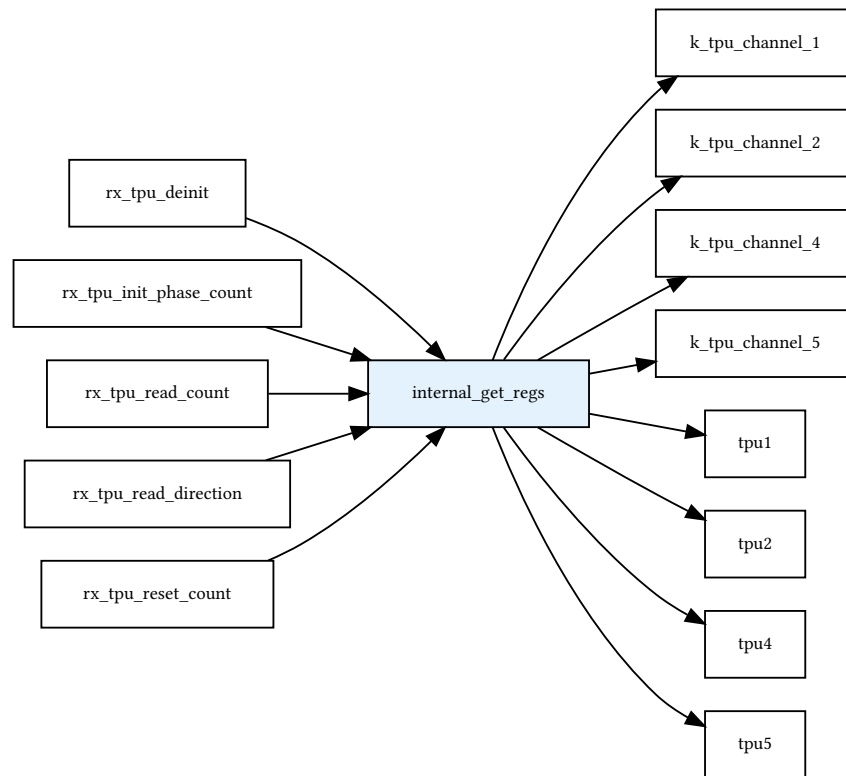
Pre: TPU module enabled (MSTPCRA.MSTPA13 = 0)

Pre: channel must be a valid rx_tpu_channel_t enumeration value

Post: No registers modified

Post: Returned pointer (if non-NULL) points to valid hardware register block

Note: Thread Safety: Safe - returns constant hardware address] **See also:** tpu1(), tpu2(), tpu4(), tpu5() Underlying accessor functions

Figure 866: Call graph for `internal_get_regs`

Defined in `libs/rx_hal/src/rx_tpu.c:294`

Since Version 1.0.0

—

`internal_get_cst_bit`

```
static uint8_t internal_get_cst_bit(const rx_tpu_channel_t channel)
```

Get TSTR CST bit mask for a TPU channel.

Returns the counter start bit mask in TSTR for the specified channel. This bit is set to start counting and cleared to stop counting. Each phase-counting-capable channel has a dedicated bit position in the shared TSTR register: CST1, CST2, CST4, and CST5.

Parameters:

Direction	Name	Description
in	channel	TPU channel identifier (1, 2, 4, or 5)

Returns: TSTR bit mask for the channel

Value	Description
Non-zero	Valid CST bit mask
0	Invalid channel

Pre: channel must be one of k_tpu_channel_1, _2, _4, or _5

Pre: Caller validates that a non-zero return indicates a valid channel

Post: No hardware registers modified

Post: Returned value is safe to OR/AND into TSTR register

Note: Thread Safety: Safe - pure function

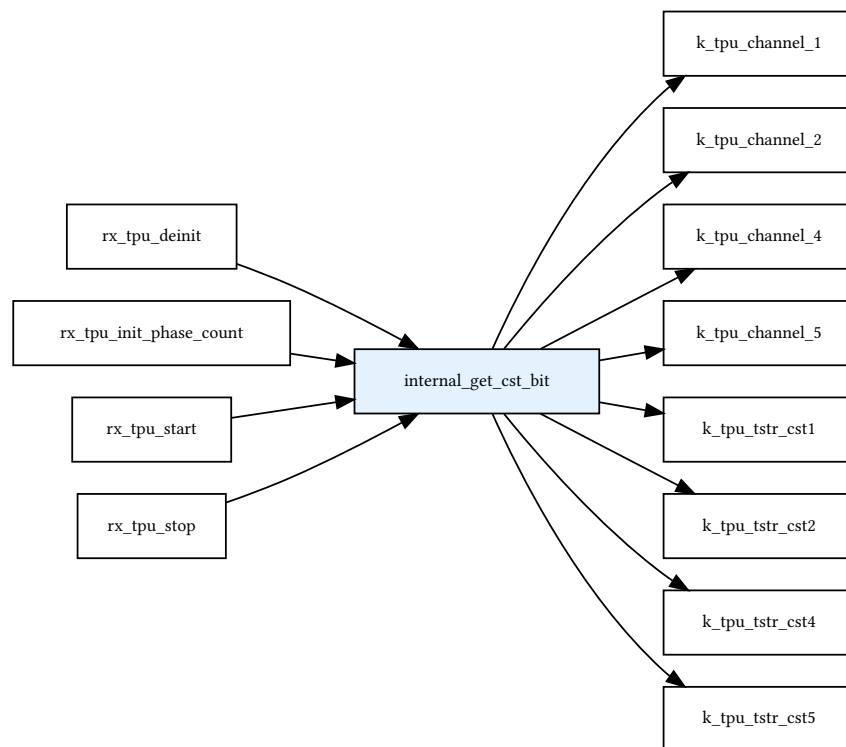


Figure 867: Call graph for `internal_get_cst_bit`

Defined in `libs/rx_hal/src/rx_tpu.c:334`

Since Version 1.0.0

internal_get_tmdr_mode

```
static uint8_t internal_get_tmdr_mode(const rx_tpu_phase_mode_t mode)
```

Map rx_tpu_phase_mode_t to TMDR.MD register value.

Converts the phase mode enumeration (1-4) to the corresponding TMDR register value for phase counting mode selection. The caller must check whether the return value equals k_tpu_tmdr_md_normal (0x00) to detect an invalid mode argument before writing to hardware.

Parameters:

Direction	Name	Description
in	mode	Phase counting mode (1-4)

Returns: TMDR.MD register value

Value	Description
0x04 - 0x07	Valid phase counting mode values
0x00	Invalid mode (normal operation fallback)

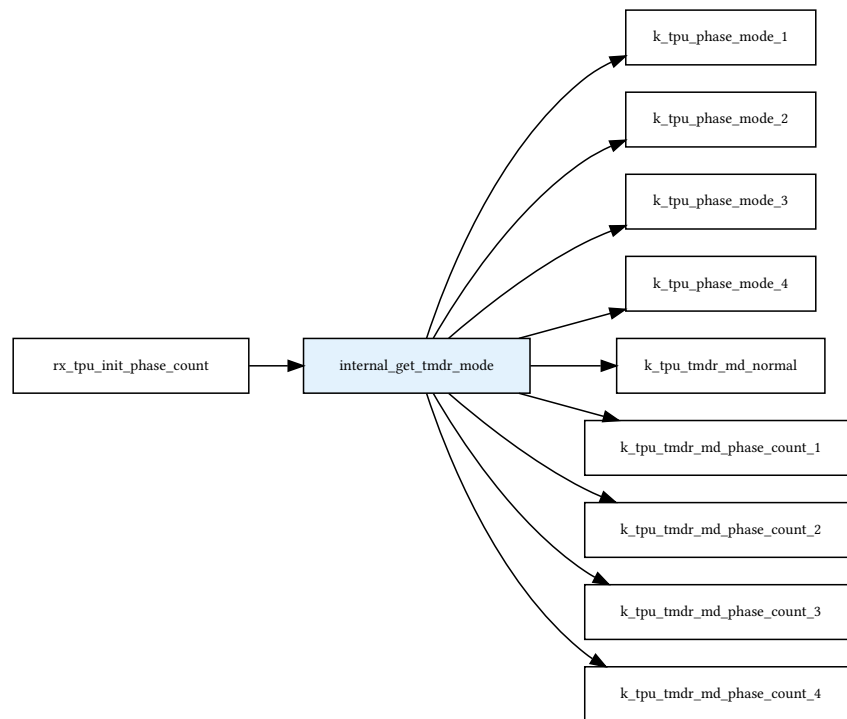
Pre: mode must be a valid rx_tpu_phase_mode_t enumeration value

Pre: Caller must check return != k_tpu_tmdr_md_normal before hardware write

Post: No hardware registers modified

Post: Returned value is safe to write directly to TMDR.MD field

Note: Thread Safety: Safe - pure function] **See also:** rx_tpu_tmdr_md_t TMDR mode register values

Figure 868: Call graph for `internal_get_tmdr_mode`

Defined in `libs/rx_hal/src/rx_tpu.c:375`

Since Version 1.0.0

—

`internal_enable_tpu_module_clock`

```
static void internal_enable_tpu_module_clock(void)
```

Enable TPU module clock by clearing MSTPCRA.MSTPA13.

Enables the TPU peripheral by clearing its module stop bit in MSTPCRA. Uses the PRCR register protection unlock/lock sequence. Idempotent - safe to call multiple times.

Pre: System clock configured

Pre: PRCR register accessible (system not in protected mode that blocks PRC1)

Post: TPU module clock enabled (MSTPCRA.MSTPA13 = 0)

Post: PRCR protection re-enabled

Note: Thread Safety: Not thread-safe (modifies system-wide registers)

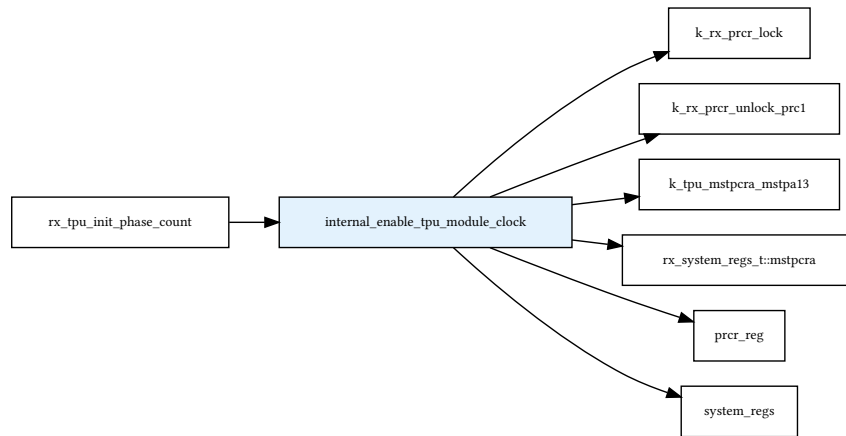


Figure 869: Call graph for `internal_enable_tpu_module_clock`

Defined in `libs/rx_hal/src/rx_tpu.c:408`

Since Version 1.0.0

—

rx_tpu_init_phase_count

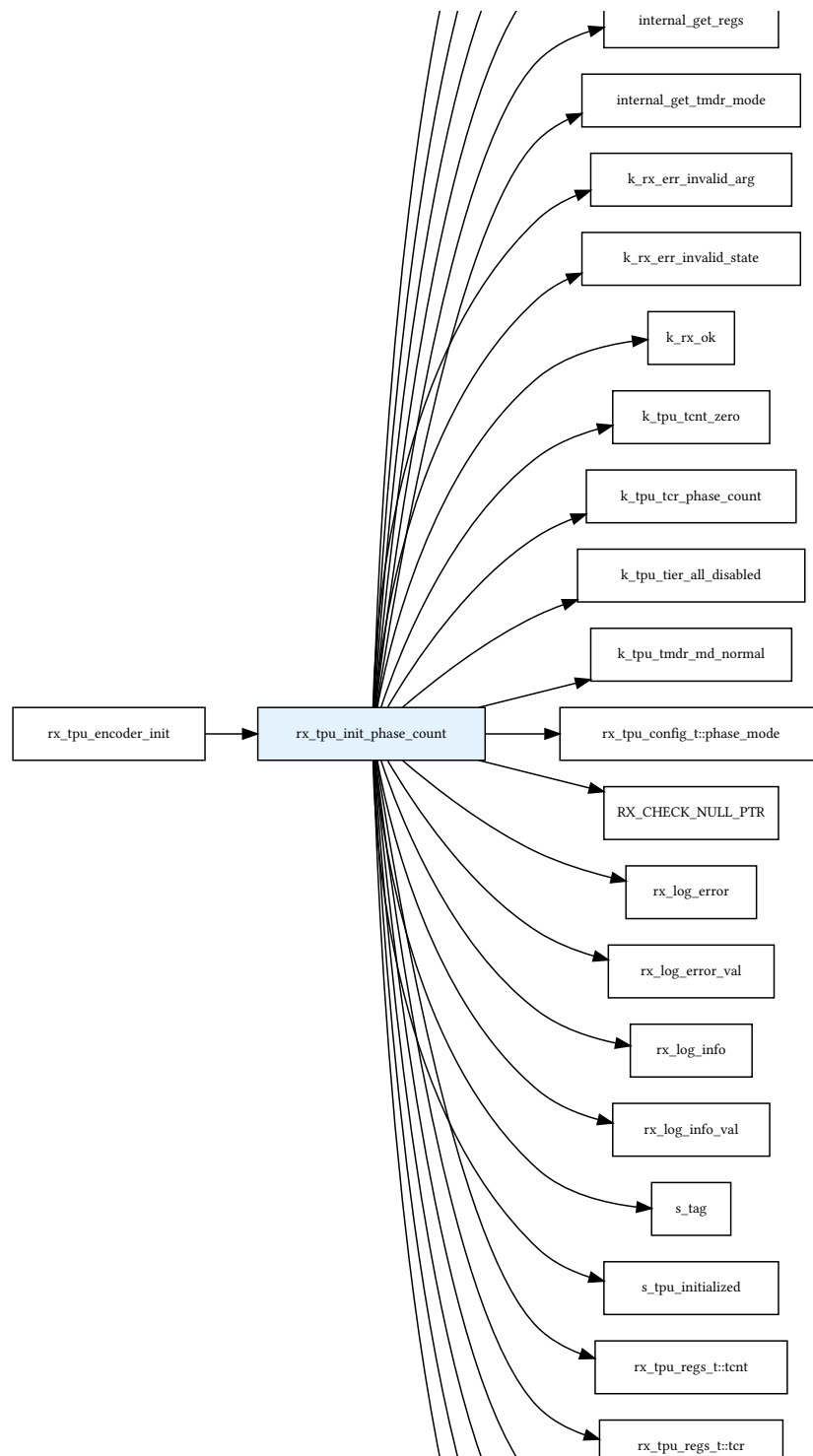
```
rx_err_t rx_tpu_init_phase_count(const rx_tpu_config_t *config)
```

Initialize TPU channel for phase counting mode.

Implementation of `rx_tpu_init_phase_count()` - see `rx_tpu.h` for complete API documentation.

Register Configuration Sequence: Unlock PRPCR, clear MSTPCRA.MSTPA13, lock PRPCR Clear CSTn in TSTR (stop counter) Write phase counting mode to TMDR.MD Write 0x00 to TCR (free-running, external clock) Write 0x00 to TIER (no interrupts) Write 0x0000 to TCNT (clear counter)

See also: `rx_tpu.h` Full API documentation

Figure 870: Call graph for `rx_tpu_init_phase_count`

Defined in `libs/rx_hal/src/rx_tpu.c:438`

Since Version 1.0.0

—

rx_tpu_start

```
rx_err_t rx_tpu_start(const rx_tpu_channel_t channel)
```

Start the TPU channel counter (begin phase counting).

Start TPU channel counter.

Sets the Count Start (CSTn) bit in the TSTR register to begin counter operation. Once started, the TCNT register increments or decrements on each external encoder clock edge according to the configured phase mode.

Parameters:

Direction	Name	Description
in	channel	TPU channel to start (1, 2, 4, or 5) Must have been previously initialized via rx_tpu_init_phase_count()

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Counter started successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: External encoder clock pins must be connected and valid

Post: TSTR.CSTn bit set, counter begins counting encoder pulses

Post: TCNT updates in real time from encoder edges

Note: Not thread-safe: modifies shared TSTR register

Note: Idempotent: safe to call when already running] **Example:**

```
rx_err_t err = rx_tpu_start(k_tpu_channel_1);
```

See also: rx_tpu_stop() Stop the counter, rx_tpu_read_count() Read current counter value

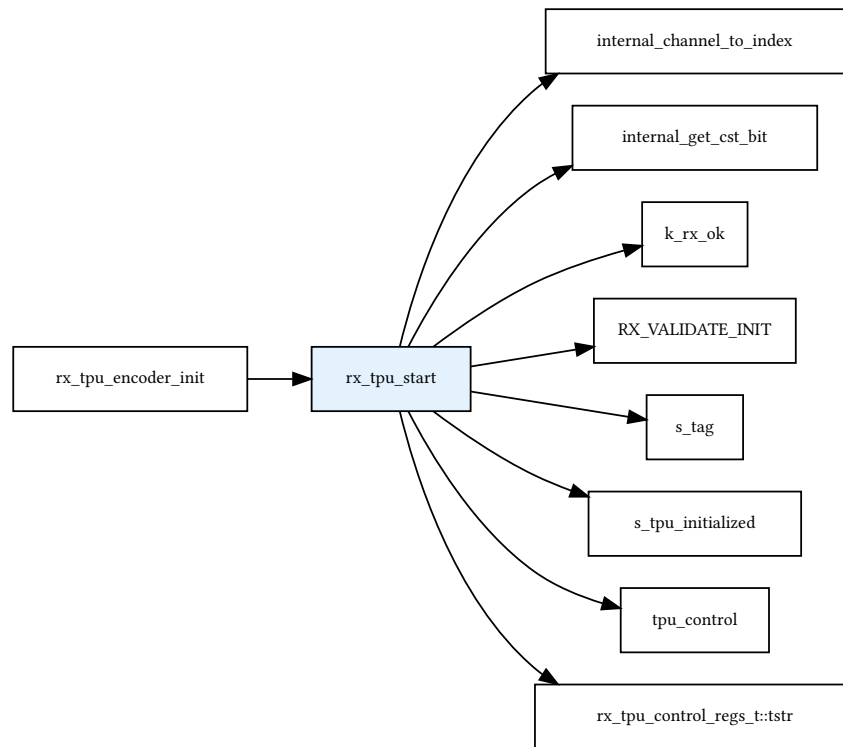


Figure 871: Call graph for rx_tpu_start

Defined in `libs/rx_hal/src/rx_tpu.c:532`

Since Version 1.0.0

—

rx_tpu_stop

```
rx_err_t rx_tpu_stop(const rx_tpu_channel_t channel)
```

Stop the TPU channel counter (halt phase counting).

Stop TPU channel counter.

Clears the Count Start (CSTn) bit in the TSTR register to halt counter operation. The TCNT value is preserved and encoder edges no longer update it. The channel remains initialized and can be restarted with `rx_tpu_start()`.

Parameters:

Direction	Name	Description
in	channel	TPU channel to stop (1, 2, 4, or 5)

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	Counter stopped successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: TSTR register must be accessible (module clock enabled)

Post: TSTR.CSTn bit cleared, counter halted

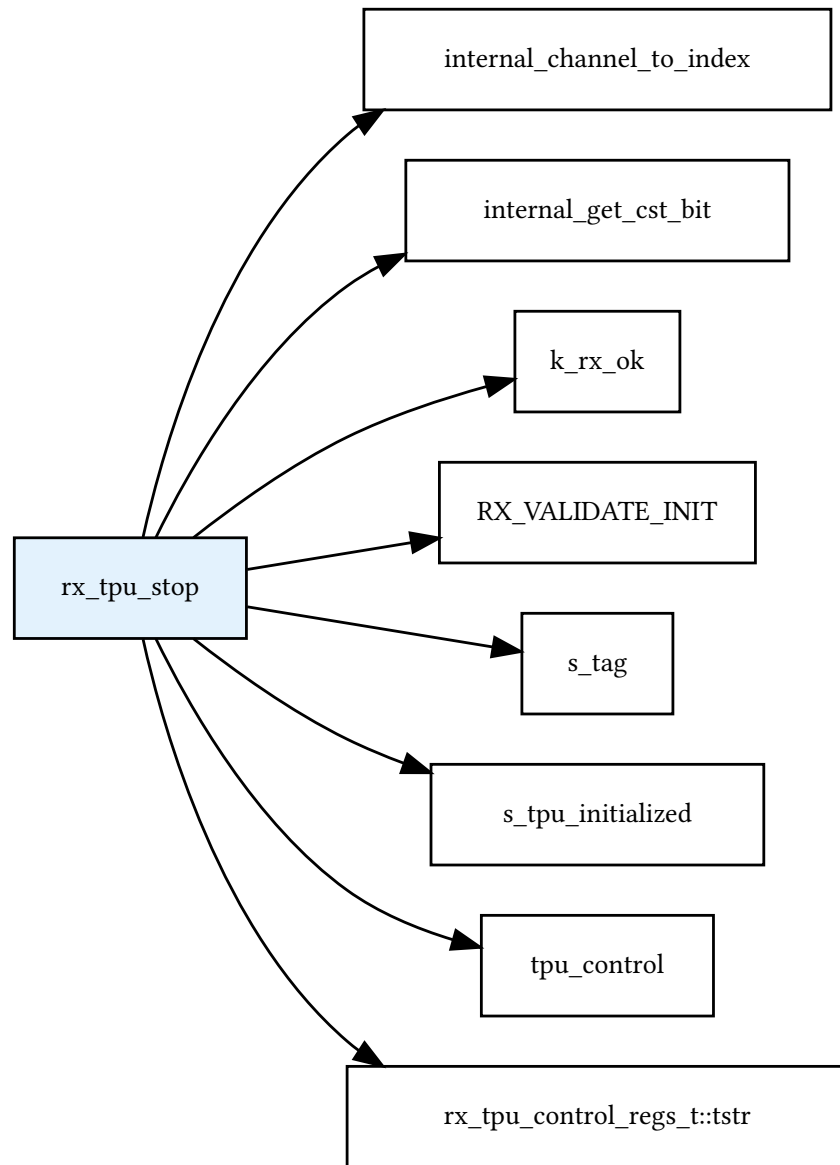
Post: TCNT value preserved at last count

Note: Not thread-safe: modifies shared TSTR register

Note: Idempotent: safe to call when already stopped] **Example:**

```
rx_err_t err = rx_tpu_stop(k_tpu_channel_1);
```

See also: rx_tpu_start() Resume counter operation, rx_tpu_reset_count() Clear counter to zero

Figure 872: Call graph for `rx_tpu_stop`

Defined in `libs/rx_hal/src/rx_tpu.c:584`

Since Version 1.0.0

—

`rx_tpu_read_count`

```
rx_err_t rx_tpu_read_count(const rx_tpu_channel_t channel, uint16_t *count)
```

Read the current 16-bit encoder counter value.

Read TPU channel 16-bit counter value.

Performs a single volatile read of the TCNT register for the specified channel. In 4x phase counting mode, the counter increments or decrements on every edge of both A and B encoder channels.

Parameters:

Direction	Name	Description
in	channel	TPU channel to read (1, 2, 4, or 5)
out	count	Pointer to store the 16-bit counter value [0, 65535] Must be non-NULL

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Counter value read into *count
k_rx_err_null_ptr	count pointer is nullptr
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: count must point to valid uint16_t storage

Post: *count contains the TCNT register value at the time of the read

Post: Hardware registers unchanged

Note: Not thread-safe: counter may wrap between multiple reads

Note: Wraps naturally at 0 and 65535 (16-bit overflow)] **Example:**

```
uint16_t count;  
rx_err_t err = rx_tpu_read_count(k_tpu_channel_1, &count);
```

See also: rx_tpu_reset_count() Clear counter to zero, rx_tpu_read_direction() Read counting direction

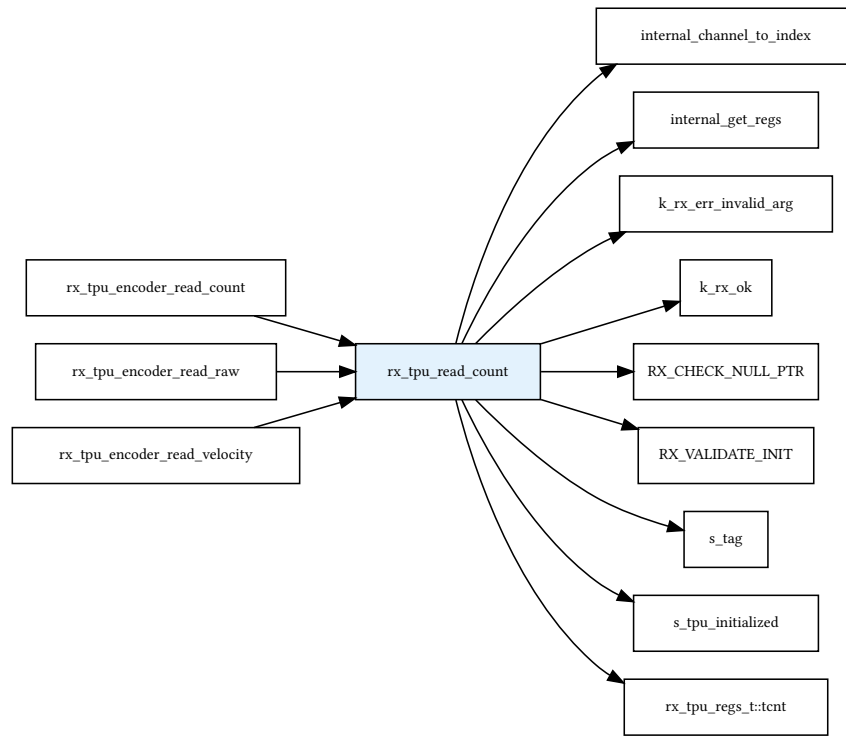


Figure 873: Call graph for rx_tpu_read_count

Defined in `libs/rx_hal/src/rx_tpu.c:640`

Since Version 1.0.0

—

rx_tpu_read_direction

```
rx_err_t rx_tpu_read_direction(const rx_tpu_channel_t channel, bool *counting_up)
```

Read the current encoder counting direction.

Read TPU channel counting direction.

Reads the Timer Counter Flag Direction (TCFD) bit in the TSR register. TCFD reflects the last count event direction:

- TCFD = 1: Counter incremented on the last edge (counting up / forward)
- TCFD = 0: Counter decremented on the last edge (counting down / reverse)

Parameters:

Direction	Name	Description
in	channel	TPU channel to query (1, 2, 4, or 5)

out	counting_up	Pointer to store direction result true = last count was an increment (encoder moving forward) false = last count was a decrement (encoder moving in reverse) Must be non-NULL
-----	-------------	---

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Direction read into *counting_up
k_rx_err_null_ptr	counting_up pointer is nullptr
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: counting_up must point to valid bool storage

Post: *counting_up reflects TSR.TCFD at the time of the read

Post: Hardware registers unchanged

Note: Not thread-safe: value may change between reads if encoder is moving

Note: Direction is unreliable if counter is stationary (no recent edge)] **Example:**

```
bool forward;  
rx_err_t err = rx_tpu_read_direction(k_tpu_channel_1, &forward);
```

See also: rx_tpu_read_count() Read counter position, k_tpu_tsr_tcfD TSR direction bit mask

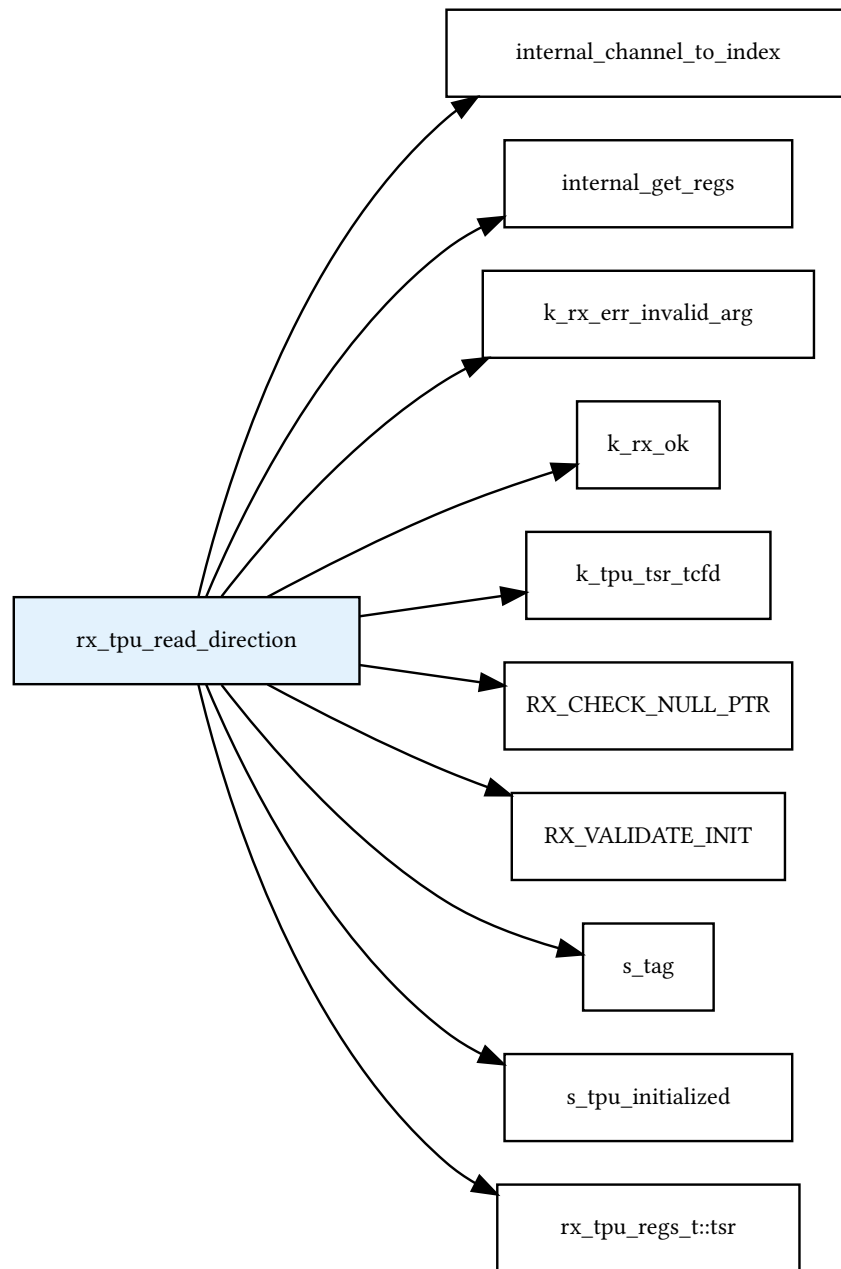


Figure 874: Call graph for rx_tpu_read_direction

Defined in `libs/rx_hal/src/rx_tpu.c:706`

Since Version 1.0.0

—

rx_tpu_reset_count

```
rx_err_t rx_tpu_reset_count(const rx_tpu_channel_t channel)
```

Reset the TPU channel encoder counter to zero.

Reset TPU channel counter to zero.

Writes k_tpu_tcmt_zero (0x0000) to the TCNT register, resetting the encoder position counter. The counter continues running from the new zero baseline.

Typical use: zero encoder position at a known reference point (home).

Parameters:

Direction	Name	Description
in	channel	TPU channel to reset (1, 2, 4, or 5)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Counter reset to zero successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_not_initialized	Channel not yet initialized

Pre: Channel must be initialized via rx_tpu_init_phase_count()

Pre: Channel should be stopped (rx_tpu_stop()) before resetting for atomic 0

Post: TCNT register written to 0x0000

Post: Counter resumes from zero if running

Note: Not thread-safe: write races with live counter updates from encoder

Note: May cause a single spurious velocity estimate if called while running] **Example:**

```
rx_tpu_stop(k_tpu_channel_1);  
rx_tpu_reset_count(k_tpu_channel_1);  
rx_tpu_start(k_tpu_channel_1);
```

See also: rx_tpu_stop() Stop counter before resetting for deterministic result,
rx_tpu_read_count() Verify counter after reset

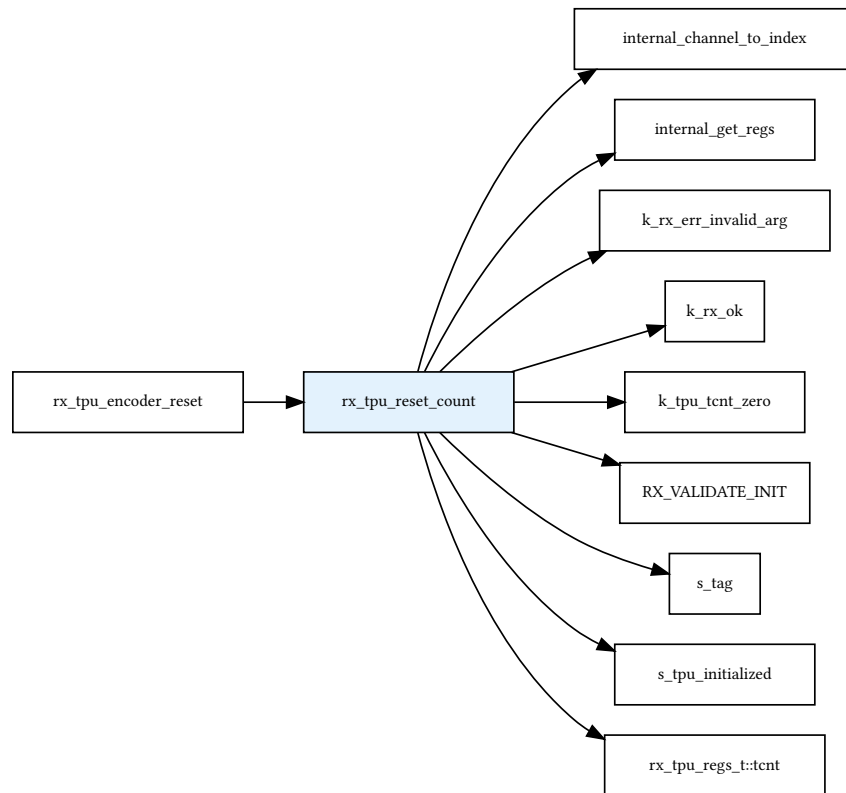


Figure 875: Call graph for rx_tpu_reset_count

Defined in `libs/rx_hal/src/rx_tpu.c:769`

Since Version 1.0.0

—

rx_tpu_deinit

```
rx_err_t rx_tpu_deinit(const rx_tpu_channel_t channel)
```

Deinitialize a TPU phase-counting channel.

Deinitialize TPU channel.

Performs an orderly shutdown of the specified TPU channel:

1. Validate channel and get array index
2. Stop counter (clear CSTn in TSTR)
3. Reset TMDR to normal operation mode (0x00)
4. Disable all interrupts (TIER = 0)
5. Clear counter (TCNT = 0)
6. Mark channel as uninitialized in `s_tpu_initialized[]`

This function does NOT disable the TPU module clock (MSTPCRA.MSTPA13), as other channels may still be in use.

Parameters:

Direction	Name	Description
in	channel	TPU channel to deinitialize (1, 2, 4, or 5)

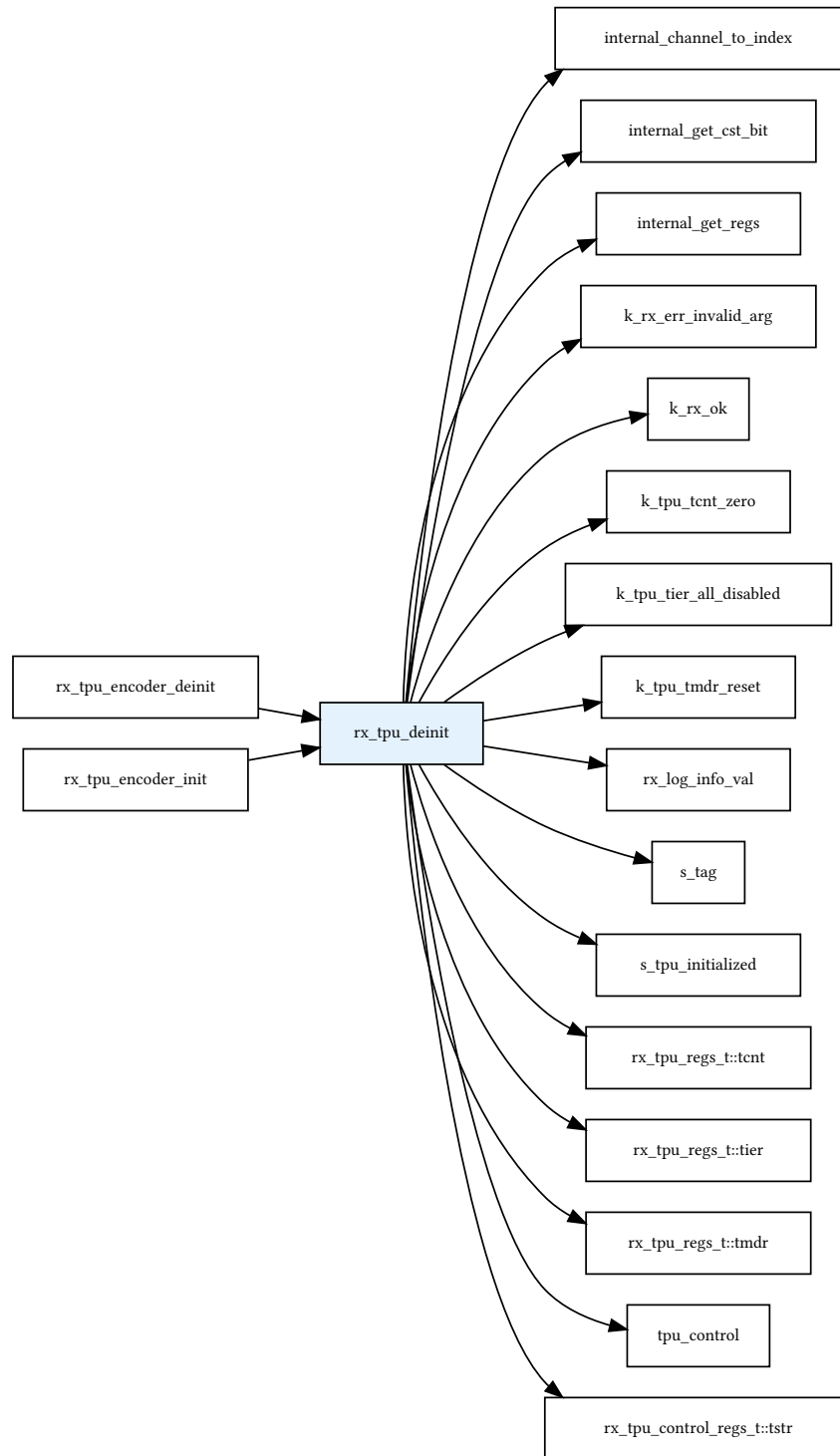
Returns: rx_err_t Error code

Value	Description
k_rx_ok	Channel deinitialized successfully
k_rx_err_invalid_arg	Channel is not 1, 2, 4, or 5
k_rx_err_invalid_arg	Register pointer could not be resolved

Pre: channel must be a valid phase-counting TPU channel (1, 2, 4, or 5)**Pre:** Motor driven by this encoder must be stopped before calling**Post:** Counter stopped and zeroed**Post:** TMDR reset to normal mode**Post:** s_tpu_initializedidx == false**Note:** Not thread-safe: do not access channel concurrently during deinit**Note:** Channel may be partially initialized; deinit succeeds regardless] **Example:**

```
rx_err_t err = rx_tpu_deinit(k_tpu_channel_1);
```

See also: rx_tpu_init_phase_count() Re-initialize after deinit, rx_tpu_stop() Stop counter without full teardown

Figure 876: Call graph for `rx_tpu_deinit`

Defined in `libs/rx_hal/src/rx_tpu.c:833`

Since Version 1.0.0

—

timer.c

ThreadX system tick timer implementation using RX72N CMT0.

This file implements the system tick timer for the ThreadX RTOS using the RX72N's CMT0 (Compare Match Timer 0) peripheral. The timer generates precise periodic interrupts at 100 Hz (10 ms period) to drive the ThreadX scheduler, time-slice preemption, and system time tracking.

Includes:

- `<stdint.h>`
- `"hardware.h"`
- `"rx72n_regs.h"`
- `"rx_check.h"`
- `"tx_api.h"`

cmt0_config_t

CMT0 timer configuration constants for 100 Hz ThreadX system tick.

Configuration values for CMT0 hardware timer to achieve precise 100.00 Hz periodic interrupts for ThreadX RTOS scheduling. Values are derived from clock tree analysis and interrupt priority allocation.

Clock calculation derivation:

Priority allocation rationale:

- Priority 3/15 balances responsiveness with critical motor interrupts
- Higher than application tasks (default priority 0)
- Lower than motor control (priority 5) and fault handlers (priority 7)

Value	Name	Description
= 4687	<code>k_cmt0_compare_match</code>	Compare match value for 100.00 Hz tick rate.
= 3	<code>k_cmt0_irq_priority</code>	CMT0 interrupt priority level in ICU.

See also: `timer_init()` Initialization function using these constants, RX72N User's Manual Section 23.2.3 for CMCOR calculation

—

cmt0_cmcr_values_t

CMT0 Control Register (CMCR) configuration values.

Bit field values for CMT0.CMCR register configuration. CMCR controls clock source, divider, and interrupt enable.

Bit breakdown for 0x0042:

- Bit 7 (CMIE) = 1: Compare match interrupt enabled
- Bits 1-0 (CKS) = 10: Clock source = PCLKB/128
- All other bits = 0: Default/reserved

Value	Name	Description
= 0x0042	k_cmt0_cmcr_config	CMCR register configuration for 100 Hz tick with interrupts.

See also: RX72N User's Manual Section 23.2.1 - CMCR register description

—

cmt0_cmstr_bits_t

CMT Start Register (CMSTR0) bit masks.

Bit positions for CMT0.CMSTR0 register, which controls timer start/stop. CMSTR0 bit 0 starts/stops CMT0 counter.

Value	Name	Description
= 0x01	k_cmt0_cmstr_start_bit	CMT0 start/stop control bit in CMSTR0 register.

See also: RX72N User's Manual Section 23.2.2 - CMSTR0 register

—

cmt0_ier_constants_t

ICU Interrupt Enable Register (IER) calculation constants.

Constants for calculating IER bit position. IER registers are byte arrays where each bit enables a specific interrupt vector.

Value	Name	Description
= 1	k_ier_bit_enable_base	Base value for IER bit enable shift calculation.

—

cmt0_counter_values_t

CMT0 Counter (CMCNT) register values.

Standard values for CMT0.CMCNT counter register. Counter increments every clock cycle and resets to 0 on compare match.

Value	Name	Description
= 0	k_cmt0_counter_init	Counter initialization value (reset state).

—

icu_ir_values_t

ICU Interrupt Request (IR) flag values.

Values for clearing ICU interrupt request flags. IR flags must be cleared in ISR before processing interrupt to prevent spurious re-triggering.

Value	Name	Description
= 0	k_icu_ir_clear	Value to clear ICU IR interrupt request flag.

See also: RX72N User's Manual Chapter 13 - ICU (Interrupt Controller Unit)

—

cmt0_isr

```
void cmt0_isr(void)
```

CMT0 compare match interrupt service routine (ISR).

Hardware interrupt handler called by ICU when CMT0 counter matches CMCOR value (every 10.000 ms). This ISR clears the interrupt flag and delegates to ThreadX kernel to handle timer processing and thread scheduling.

Execution context: Interrupt context (cannot block, minimal stack)

Call frequency: 100 Hz (every 10 ms)

Algorithm steps:

1. Hardware triggers interrupt when CMCNT == CMCOR
2. CPU vectors to this ISR via vector table entry VECT_CMT0_CMI0
3. Clear ICU IR flag to acknowledge interrupt
4. Call ThreadX _tx_timer_interrupt() to process timers and scheduling
5. Return from interrupt (hardware restores context)

ThreadX processing (inside _tx_timer_interrupt()):

- Increment system time counter
- Check timer expiration list
- Wake sleeping threads whose timeout expired
- Perform time-slice round-robin if enabled
- Trigger context switch if higher-priority thread ready

****IR flag must be cleared first****: Clearing IR flag before processing prevents race condition where new interrupt triggers while still processing previous one.

Pre: CMT0 must be initialized and running via timer_init()

Pre: ICU interrupt must be enabled (IER bit set)

Pre: Global interrupts must be enabled (PSW I-flag set)

Post: ICU IR flag cleared (prevents spurious re-trigger)

Post: ThreadX system time incremented by 1 tick

Post: Expired timers processed, blocked threads potentially woken

Post: Context switch may occur if higher-priority thread ready

Note: Thread Safety: Runs in interrupt context. Must not call blocking ThreadX APIs (tx_mutex_get, tx_thread_sleep). Only non-blocking operations allowed.

Note: Re-entrancy: Not reentrant (interrupt priority protects). CMT0 interrupt cannot nest on itself. Higher-priority interrupts (motor faults) can preempt this ISR.

Note: Critical Timing: This ISR must complete quickly (<10 us target) to maintain system responsiveness. Excessive ISR time causes scheduler latency and motor control jitter.

Warning: ****No blocking operations****: Never call tx_thread_sleep, tx_mutex_get, tx_queue_receive, or any blocking API from ISR. These will cause system crash.

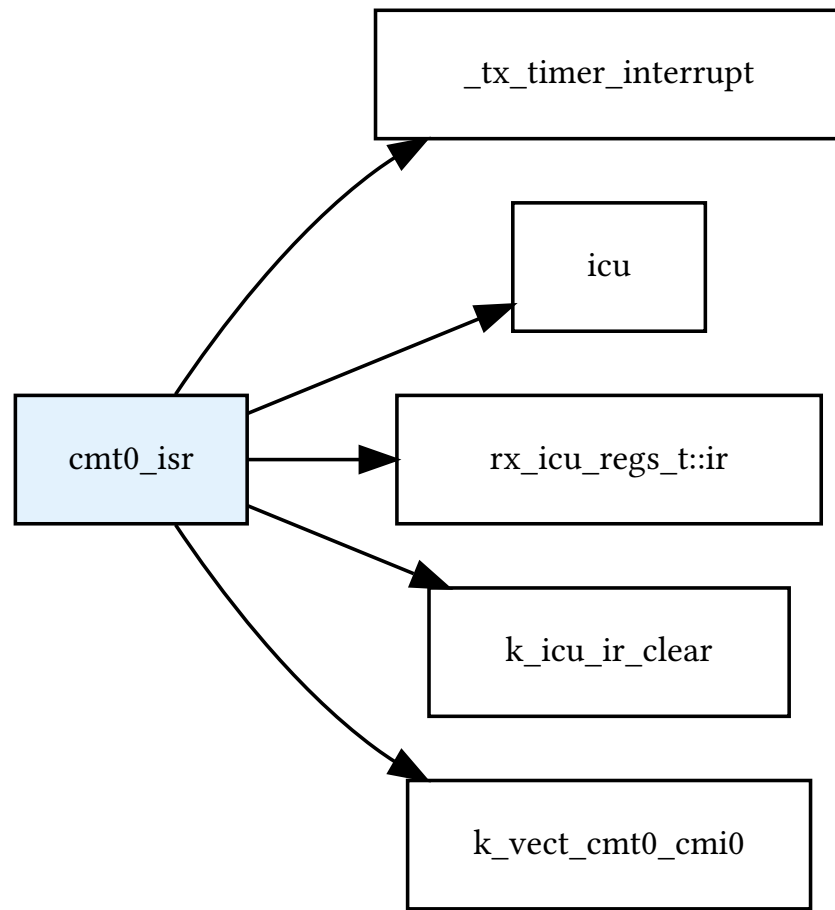
Performance:: Execution time: 2-5 us @ 240 MHz (measured) CPU overhead: 0.02% - 0.05% (5 us x 100 Hz / 1 second) Stack usage: 64 bytes (interrupt frame + _tx_timer_interrupt) Jitter: <100 ns (hardware interrupt latency)

Control Flow Diagram:: digraph cmt0_isr_flow { rankdir=TB; node [shape=box, style=rounded]; hw_match [label="Hardware:nCMCNT == CMCOR", fillcolor=lightyellow, style=filled]; icu_trigger [label="ICU:nTrigger IRQ", fillcolor=lightyellow, style=filled]; isr_entry [label="ISR Entry:nSave context"]; clear_ir [label="Clear IR flag"]; call_tx [label="Call _tx_timer_interrupt()"]; tx_process [label="ThreadX:nUpdate timersnSchedule threads", fillcolor=lightblue, style=filled]; isr_exit [label="ISR Exit:nRestore context"]; resume [label="Resumeninterrupted code"]; hw_match -> icu_trigger; icu_trigger -> isr_entry; isr_entry -> clear_ir; clear_ir -> call_tx; call_tx -> tx_process; tx_process -> isr_exit; isr_exit -> resume; }

Example - System Timeline:: t=0.000ms:Applicationthreadrunning
t=10.000ms:CMT0comparematch->Hardwareinterrupt t=10.001ms:ISRentry(CPUcontextsave: 500ns)
t=10.002ms:ClearIRflag(registerwrite: 100ns) t=10.003ms:Call_tx_timer_interrupt()(2usprocessing)
t=10.005ms:ISRexit(CPUcontextrestore: 500ns)
t=10.006ms:Resumeapplicationthread(orswitchtohigherpriority)

NASA Power of 10 Compliance: Rule 1 [OK] No goto, setjmp, recursion Rule 2 [OK] No loops Rule 3 [OK] No dynamic allocation Rule 4 [OK] Function is 7 lines (under 60 line limit) Rule 5 [OK] 3 preconditions, 4 postconditions Rule 8 [OK] Uses C23 typed enum (k_icu_ir_clear) Rule 9 [OK] Single level of dereferencing

See also: _tx_timer_interrupt() ThreadX kernel timer processing function, timer_init() CMT0 initialization that enables this ISR, RX72N User's Manual Section 23.3 - CMT Interrupt Operation, ThreadX User Guide Section 3.4 - Interrupt Handling

Figure 877: Call graph for `cmt0_isr`

Defined in `libs/rx\_hal/src/timer.c:500`

Since Version 1.0.0

—

`_tx_timer_interrupt`

```
void _tx_timer_interrupt(void)
```

Figure 878: Call graph for `_tx_timer_interrupt`

Defined in `libs/rx\_hal/src/timer.c:356`

timer_init

```
rx_err_t timer_init(void)
```

Initialize CMT0 timer for ThreadX system tick at 100 Hz.

Initialize CMT0 for ThreadX system tick (100 Hz).

Configures the RX72N CMT0 (Compare Match Timer 0) peripheral to generate precise periodic interrupts at 100.00 Hz for ThreadX RTOS scheduling. This function must be called during system initialization, before starting ThreadX kernel.

Algorithm steps:

1. Validate CMT0 and CMT_CTRL hardware accessibility
2. Stop CMT0 if currently running (safe reconfiguration)
3. Configure CMCR register: PCLKB/128 clock, interrupt enabled
4. Set CMCOR compare match value: 4687 (for 100 Hz)
5. Reset CMCNT counter to 0 (known initial state)
6. Configure ICU: clear pending interrupt, set priority, enable interrupt
7. Start CMT0 timer
8. Enable global interrupts (PSW I-flag)
9. Verify timer started successfully

Configuration summary:

- Clock source: PCLKB = 60 MHz
- Divider: /128 -> 468.75 kHz
- Compare value: 4687 -> 100.00 Hz
- Interrupt priority: 3/15 (higher than app tasks, lower than motor control)
- Tick period: 10.000 ms (exact, zero drift)

Hardware configuration sequence:

CMT0 compare value = 4687 (100.00 Hz)

CMT0 interrupt priority = 3/15

Global interrupt enable: This function enables global interrupts via “setpsw i”. If interrupts were intentionally disabled, they will be enabled after this call.

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success - CMT0 configured and running at 100 Hz
k_rx_err_invalid_state	Hardware registers NULL (clock not configured)
k_rx_err_hw_error	Timer failed to start (hardware fault)

Pre: System clocks must be configured (PCLKB = 60 MHz)

Pre: CMT0 must not be in use by another module

Pre: Must be called before tx_kernel_enter()

Pre: Interrupts can be disabled (function enables at end)

Post: CMT0 running at 100 Hz, generating periodic interrupts

Post: cmt0_isr() will be called every 10 ms

Post: ThreadX system tick operational

Post: Global interrupts enabled (PSW I-flag set)

Note: Thread Safety: Not thread-safe. Must be called during single-threaded initialization before ThreadX kernel starts. Do not call from running RTOS tasks.

Note: Re-entrancy: Not reentrant. Calling multiple times without timer_stop() is undefined behavior (timer will be reconfigured).

Note: Performance: Initialization takes 10 us @ 240 MHz (multiple register writes). This is one-time startup cost, not runtime overhead.

Note: Memory: Zero heap allocation. All configuration is register I/O.

Warning: Call order critical: Must call timer_init() BEFORE tx_kernel_enter(). ThreadX depends on system tick being operational. Calling after kernel start causes undefined behavior.

Warning: Clock dependency: Assumes PCLKB = 60 MHz. If system clock configuration changes, k_cmt0_compare_match must be recalculated. Incorrect clock frequency causes incorrect tick rate.

Return Value Details:

Example - System Initialization:: #include "timer.h" #include "tx_api.h"

```
intmain(void) { hardware_init();
```

```
rx_err_t err = timer_init(); if(err != k_rx_ok) { rx_log_error("MAIN", "System tick init failed: %d", err);
while(1) {} }
```

```
tx_application_define(nullptr);
```

```
tx_kernel_enter(); }
```

Example - Error Handling:: rx_err_t err = timer_init(); switch(err) { case k_rx_ok: rx_log_info("MAIN", "System tick running at 100Hz"); break; case k_rx_err_invalid_state: rx_log_error("MAIN", "Clock not configured - check hardware_init()"); return err;

```
casek_rx_err_hw_error: rx_log_error("MAIN","CMT0hardwarefault"); returnerr; default:  
rx_log_error("MAIN","Unexpectederror:%d",err); returnerr; }
```

Example - Verification:: timer_init();

```
uint16_tcount1,count2; timer_get_count(&count1); tx_thread_sleep(50); timer_get_count(&count2);  
rx_log_info("MAIN","Tickverified:countchangedfrom%uto%u",count1,count2);
```

NASA Power of 10 Compliance: Rule 1 [OK] No goto, setjmp, recursion (sequential register writes) Rule 2 [OK] No loops (straight-line initialization code) Rule 3 [OK] No dynamic allocation (all register I/O) Rule 4 [OK] Function is 47 lines (under 60 line limit) Rule 5 [OK] 4 preconditions, 4 postconditions documented Rule 6 [OK] Minimal scope (no local variables) Rule 7 [OK] All register accessor return values validated Rule 8 [OK] C23 typed enums for ALL constants (k_cmt0_*) Rule 9 [OK] Single level of dereferencing for registers Rule 10 [OK] Compiles clean with -Wall -Wextra -Werror

See also: timer_stop() Stop system tick timer, timer_get_count() Read current counter value, cmt0_isr() Interrupt handler called every 10 ms, tx_kernel_enter() ThreadX kernel entry point, RX72N User's Manual Chapter 23 - Compare Match Timer

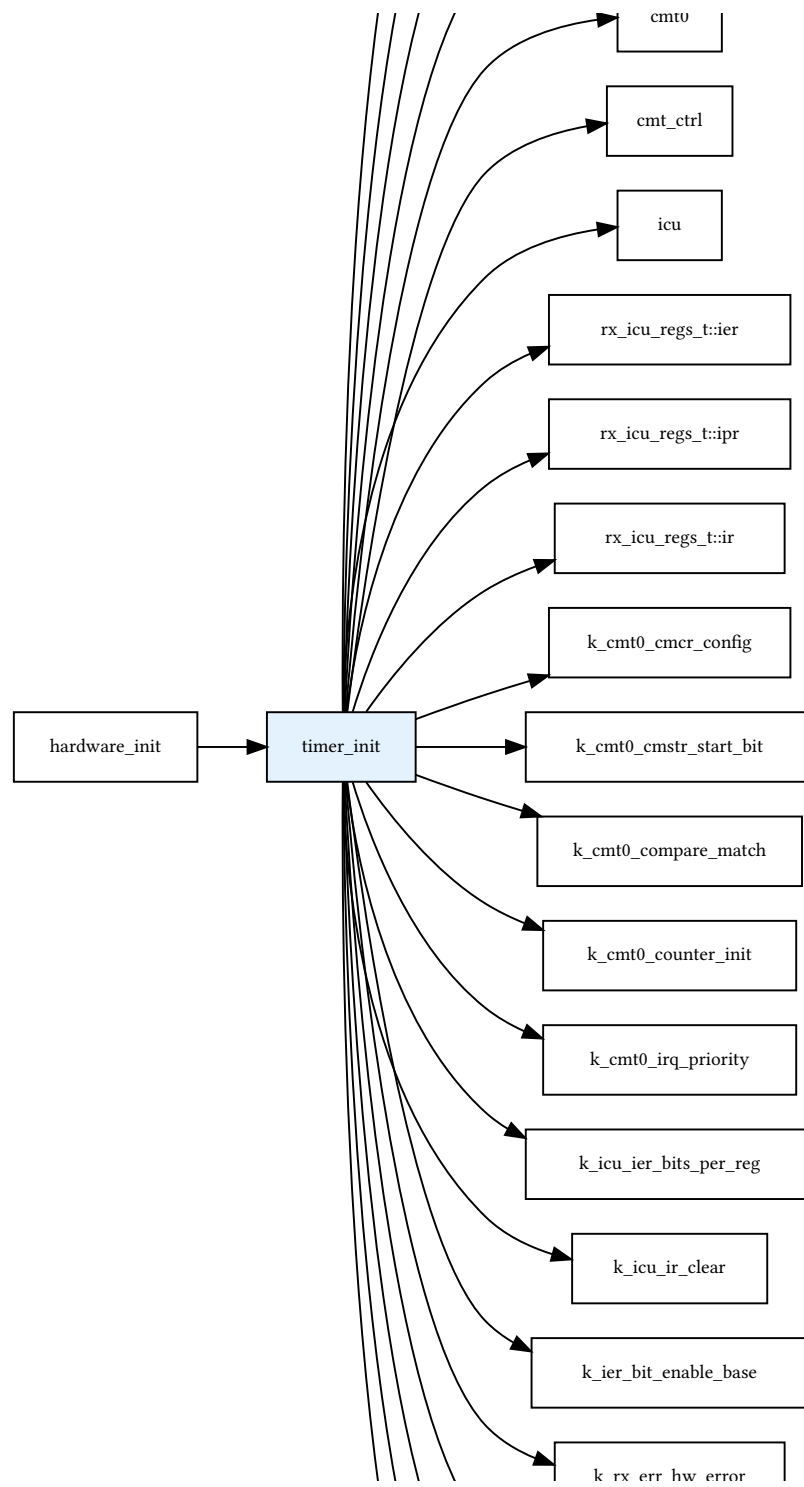


Figure 879: Call graph for timer_init

_Defined in libs/rx_hal/src/timer.c:690_

Since Version 1.0.0

—

timer_stop

```
rx_err_t timer_stop(void)
```

Stop CMT0 system tick timer.

Stop CMT0 timer and disable compare match interrupts.

Halts the CMT0 timer by clearing the start bit in CMSTR0 register. This stops the system tick interrupts, effectively freezing ThreadX scheduling and time tracking. Use only during system shutdown or for testing.

Algorithm steps:

1. Validate CMT_CTRL hardware accessibility
2. Check if timer is currently running
3. Clear CMSTR0 start bit to stop counter
4. Verify timer stopped successfully

Use cases:

- System shutdown sequence
- Power management (entering low-power mode)
- Testing and diagnostics
- Emergency stop during fault condition

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success - CMT0 stopped, no more interrupts
k_rx_err_hw_unmapped	CMT_CTRL register inaccessible
k_rx_err_invalid_state	Timer already stopped
k_rx_err_hw_error	Failed to stop timer (hardware fault)

Pre: timer_init() must have been called successfully

Pre: CMT0 should be running (otherwise returns k_rx_err_invalid_state)

Post: CMT0 timer stopped, CMCNT frozen

Post: No more cmt0_isr() interrupts will occur

Post: ThreadX time tracking frozen (system time no longer increments)

Note: Thread Safety: Not thread-safe. Should only be called from single-threaded shutdown sequence or with ThreadX suspended.

Note: Performance: Executes in 1 us @ 240 MHz (register read/write).

Warning: Stopping system tick will freeze ThreadX scheduler. All time-based operations (tx_thread_sleep, timeouts) will hang. Only call during controlled shutdown or with ThreadX suspended.

Example - Controlled Shutdown:: tx_thread_suspend(&all_app_threads);

```
rx_err_t err=timer_stop(); if(err!=k_rx_ok){ rx_log_error("SHUTDOWN","Failed to stop timer: %d",err); }
```

See also: timer_init() Initialize and start timer, timer_get_count() Read current counter value

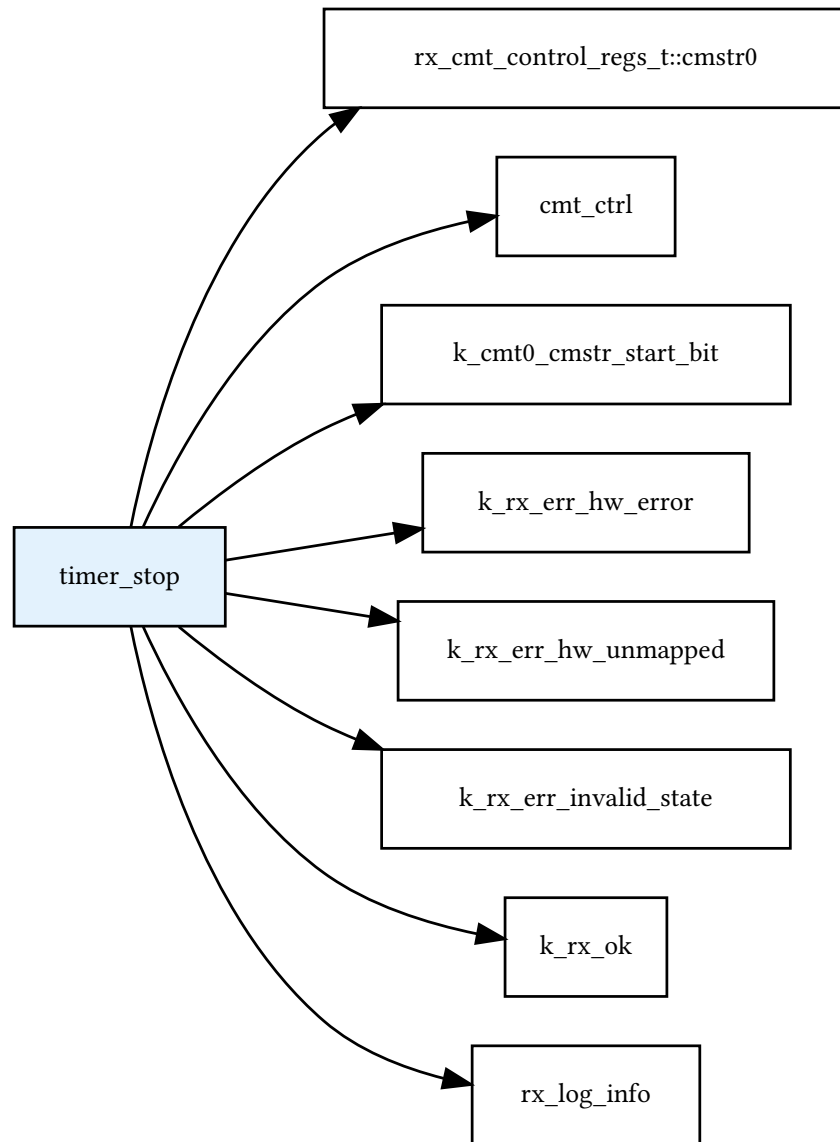


Figure 880: Call graph for timer_stop

_Defined in libs/rx_hal/src/timer.c:804_

Since Version 1.0.0

—

timer_get_count

```
rx_err_t timer_get_count(uint16_t *count)
```

Read current CMT0 counter value.

Get current CMT0 counter value.

Reads the current value of CMT0.CMCNT register, which increments from 0 to CMCOR (4687) at 468.75 kHz. This provides sub-millisecond timing resolution for diagnostics, performance measurement, and jitter analysis.

Algorithm steps:

1. Validate count pointer is not nullptr
2. Validate CMT0 hardware accessibility
3. Read CMCNT register (atomic 16-bit read)
4. Validate count is within expected range (0-4687)
5. Store count value to output parameter

Counter behavior:

- Increments every $1/(468.75 \text{ kHz}) = 2.133 \text{ us}$
- Resets to 0 when reaching CMCOR (4687)
- Wraps every 10 ms (one system tick)
- Can be read at any time without affecting timer operation

Timing resolution:

Maximum readable value:

Return value k_rx_ok guarantees *count in [0, 4687]

Parameters:

Direction	Name	Description
out	count	Pointer to uint16_t to store current counter value Type: uint16_t* Valid range: Must be non-NULL Output range: 0 to k_cmt0_compare_match (4687) Units: Timer counts (1 count = 2.133 us) Null handling: Returns k_rx_err_null_ptr if nullptr Lifetime: Caller must ensure pointer valid until return

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success - counter value written to *count
k_rx_err_null_ptr	count parameter is nullptr
k_rx_err_hw_unmapped	CMT0 register inaccessible
k_rx_err_hw_error	Counter value exceeds CMCOR (hardware fault)

Pre: timer_init() must have been called successfully

Pre: count pointer must be valid and non-NULL

Post: If k_rx_ok: *count contains current CMCNT value (0-4687)

Post: If error: *count unchanged (nullptr) or contains invalid value (hw_error)

Post: Timer operation not affected (read-only access)

Note: Thread Safety: Thread-safe for reading. Multiple threads can call simultaneously. CMCNT is atomic 16-bit register on RX72N. No need for mutual exclusion.

Note: Re-entrancy: Fully reentrant. Safe to call from ISRs and tasks concurrently. No shared state modified.

Note: Performance: Executes in 500 ns @ 240 MHz (single register read). Suitable for high-frequency polling.

Note: Memory: No heap allocation. Stack usage: 4 bytes (local variable).

Warning: Counter wraps: Counter resets to 0 every 10 ms. For measuring intervals longer than 10 ms, use ThreadX system time (tx_time_get) instead.

Warning: Race condition: Counter value is snapshot at time of read. By the time caller processes value, hardware counter has advanced. Don't assume static value.

Return Value Details::

Example - Basic Usage:: uint16_tcount; rx_err_t err = timer_get_count(&count); if(err == k_rx_ok) { float us = count * 2.133f; rx_log_info("TIMER", "Current time tick: %.1fus", us); }

Example - Performance Measurement:: uint16_t start, end; timer_get_count(&start);
motor_update();
timer_get_count(&end);
uint16_t elapsed; if(end >= start) { elapsed = end - start; } else { elapsed = (4687 - start) + end; }
float us = elapsed * 2.133f; rx_log_info("PERF", "motor_update took %.1fus", us);

Example - Jitter Analysis:: static uint16_t last_count = 0;
void analyze_jitter(void) { uint16_t current; timer_get_count(¤t);
int16_t jitter = current - last_count; if(abs(jitter) > 10) { rx_log_warn("TIMER", "High jitter detected: %d counts", jitter); }
last_count = current; }

Example - Error Handling:: uint16_t count; rx_err_t err = timer_get_count(&count); switch(err) { case k_rx_ok: break; case k_rx_err_null_ptr: rx_log_error("TIMER", "Invalid pointer"); return err; case k_rx_err_hw_unmapped: rx_log_error("TIMER", "CMT0 not initialized"); return err; case k_rx_err_hw_error: rx_log_error("TIMER", "Counter overflow: %u", count); return err; }

NASA Power of 10 Compliance: Rule 1 [OK] No goto, setjmp, recursion Rule 2 [OK] No loops Rule 3 [OK] No dynamic allocation Rule 4 [OK] Function is 13 lines (under 60 line limit) Rule 5 [OK] 2 preconditions, 3 postconditions Rule 6 [OK] Minimal scope (single output parameter) Rule 7 [OK] All return values validated (RX_CHECK_NULL_PTR) Rule 8 [OK] Uses C23 typed enums (k_cmt0_compare_match) Rule 9 [OK] Single level of pointer dereferencing Rule 10 [OK] Compiles with -Wall -Wextra -Werror

See also: timer_init() Initialize timer before reading, tx_time_get() For millisecond-resolution system time, RX72N User's Manual Section 23.2.5 - CMCNT Register

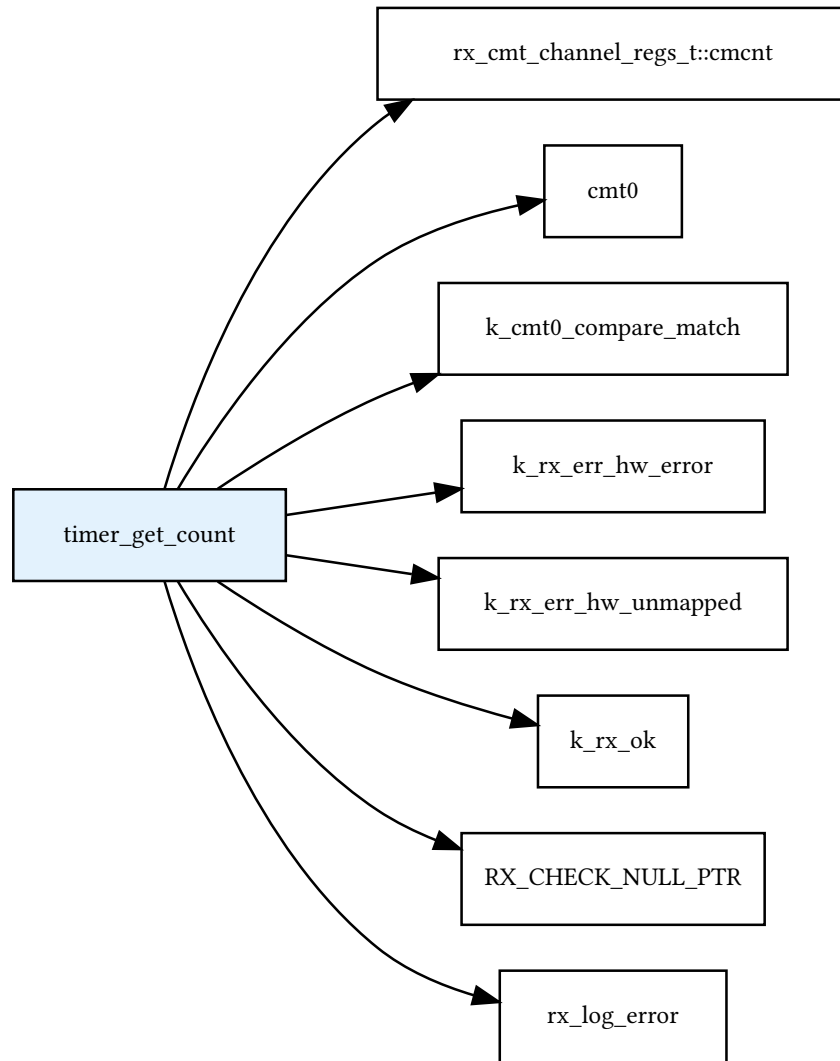


Figure 881: Call graph for timer_get_count

_Defined in libs/rx_hal/src/timer.c:1003_

Since Version 1.0.0

—

uart.c

Multi-Channel UART Driver for RX72N SCI Peripherals.

Includes:

- <stdbool.h>
- <stdint.h>
- "hardware.h"
- "rx72n_clock.h"
- "rx72n_regs.h"
- "rx_mpc.h"
- "rx_port_utils.h"
- "rx_register_protection.h"
- "rx_simulator_config.h"

uart_config_constants_t

UART configuration constants.

Value	Name	Description
= 115200	k_uart_default_baudrate	Default baud rate: 115200 bps
= 1000	k_uart_bit_time_delay_cycles	Bit time delay (8.68us at 115200 bps, >520 cycles)

—

sci_register_values_t

SCI register values.

Value	Name	Description
= 0x00	k_sci_scr_disabled	SCR: All functions disabled
= 0x20	k_sci_scr_tx_enabled	SCR: Transmit enabled (TE=1)
= 0x10	k_sci_scr_rx_enabled	SCR: Receive enabled (RE=1)
= 0x30	k_sci_scr_txrx_enabled	SCR: TX + RX enabled (TE=1, RE=1)
= 0x00	k_sci_smr_async_8n1	SMR: Async mode, 8 data bits, no parity, 1 stop bit, PCLK/1
= 0x00	k_sci_semr_default	SEMR: Default extended mode
= 0x80	k_sci_ssr_tdre_flag	SSR: Transmit data register empty flag
= 0x40	k_sci_ssr_rdrf_flag	SSR: Receive data register full flag
= 0x20	k_sci_ssr_orer_flag	SSR: Overrun error flag
= 0x10	k_sci_ssr_fer_flag	SSR: Framing error flag
= 0x08	k_sci_ssr_per_flag	SSR: Parity error flag
= 0x38	k_sci_ssr_error_mask	SSR: All error flags mask

—

uart_int_constants_t

Integer to string buffer constants.

Value	Name	Description
= 12	k_uart_int_buffer_size	Buffer size for int32 to string (enough for -2147483648)
= 10	k_uart_base_10	Base 10 for decimal conversion

—

uart_hex_constants_t

Hex digit constants.

Value	Name	Description
= 8	k_uart_hex_max_digits	Maximum hex digits to print (32-bit value)
= 1	k_uart_hex_min_digits	Minimum hex digits to print
= 0	k_uart_hex_zero_digits	Zero digits value
= 4	k_uart_hex_nibble_bits	Bits per hex nibble
= 0x0F	k_uart_hex_nibble_mask	Mask for hex nibble

—

brr_constants_t

BRR calculation constants.

Value	Name	Description
= 32	k_brr_divisor_n0	Divisor for n=0 (CKS=00): $64 * 2^{(2n-1)} = 32$
= 4	k_brr_multiplier	Multiplier per CKS increment (2^2)
= 255	k_brr_max_value	Maximum BRR register value
= 0	k_brr_min_value	Minimum BRR register value
= 1	k_brr_formula_offset	BRR formula subtract-1 offset: $BRR = (PCLKB/(32*B)) - 1$

—

uart_mstpcrb_constants_t

MSTPCRB register bit manipulation constants.

Value	Name	Description
= 1UL	k_uart_mstpcrb_bit_set	Single-bit mask for MSTPCRB bit-clear operations

—

uart_internal_constants_t

Maximum SCI channels (array size, must be enum for compile-time constant).

Value	Name	Description
= 0	k_uart_channel_min	Minimum UART channel (SCI0)
= 13	k_uart_array_size	Array size for s_channel_initialized
= 11	k_uart_max_mstpb_channel	Maximum channel in MSTPCRB (SCI12 uses MSTPCRC)

= 13	k_uart_max_channels	Maximum valid channel value (SCI channels 0-12)
------	---------------------	---

—

uart_validation_limits_t

Baud rate validation bounds for uart_init_channel() parameter checking.

Defines the closed interval [k_uart_baudrate_min, k_uart_baudrate_max] that every requested baud rate must satisfy before the driver attempts to program the BRR register.

The maximum is derived directly from the BRR formula for n=0 (CKS=00): A BRR of 0 corresponds to the fastest achievable baud rate at the current PCLKB frequency, so requesting anything higher would underflow the register.

The minimum is 1 bps, which prevents a divide-by-zero in the BRR formula. In practice, any baud rate below 9600 is impractical on a 60 MHz PCLKB, but the lower bound is kept permissive to avoid false negatives during testing.

k_uart_baudrate_min must be > 0 to prevent division by zero in internal_calculate_brr().

k_uart_baudrate_max must equal k_pclkb_hz / k_brr_divisor_n0 so that the computed BRR is always >= 0 (i.e., no register underflow).

Value	Name	Description
= 1	k_uart_baudrate_min	Minimum valid baud rate (bps); must be > 0 to avoid divide-by-zero in BRR formula
= (k_pclkb_hz / k_brr_divisor_n0)	k_uart_baudrate_max	Maximum valid baud rate (bps); BRR = 0 at this rate, higher values would underflow

See also: uart_init_channel() Uses these bounds to validate the baudrate field, internal_calculate_brr() Computes the actual BRR register value, brr_constants_t BRR formula divisor constants

Since Version 1.0.0

—

uart_timeout_t

UART timeout constants.

Value	Name	Description
= 100000	k_uart_tx_timeout	Transmit buffer wait timeout (prevents infinite loop)
= 0	k_uart_timeout_expired	Timeout counter expired value
= 1	k_uart_timeout_decrement	Timeout counter decrement value

—

uart_flag_constants_t

UART flag comparison constants.

Value	Name	Description
= 0	k_uart_flag_clear	Flag bit is not set (comparison result)

—

uart_buffer_constants_t

UART buffer and string size limits.

Value	Name	Description
= 256	k_uart_max_str_len	Maximum string length for uart_puts_channel

—

sci_mstpb_bits_t

SCI module stop bit positions in MSTPCRB.

Value	Name	Description
= 31	k_sci_mstpb_sci0	SCI0 module stop bit
= 30	k_sci_mstpb_sci1	SCI1 module stop bit
= 29	k_sci_mstpb_sci2	SCI2 module stop bit
= 28	k_sci_mstpb_sci3	SCI3 module stop bit
= 27	k_sci_mstpb_sci4	SCI4 module stop bit
= 26	k_sci_mstpb_sci5	SCI5 module stop bit
= 25	k_sci_mstpb_sci6	SCI6 module stop bit
= 24	k_sci_mstpb_sci7	SCI7 module stop bit
= 23	k_sci_mstpb_sci8	SCI8 module stop bit
= 22	k_sci_mstpb_sci9	SCI9 module stop bit
= 21	k_sci_mstpb_sci10	SCI10 module stop bit
= 20	k_sci_mstpb_sci11	SCI11 module stop bit

—

uart_debug_pins_t

Debug UART pins (SCI9 on RX72N).

Value	Name	Description
= k_rx_pb_7	k_uart_debug_tx_gpio	PB7 = TXD9
= k_rx_pb_6	k_uart_debug_rx_gpio	PB6 = RXD9

—

uart_gpio_constants_t

GPIO register bit manipulation constants for UART pin configuration.

Provides the single-bit seed value used when constructing per-pin bitmasks for the PDR (Port Direction Register) and PMR (Port Mode Register) during UART TX/RX pin setup. A bit mask for a specific pin is formed by shifting this value left by the pin index obtained from `rx_pin_from_pin()`.

`k_uart_gpio_bit_set` must equal 1 so that left-shifting by a pin index produces an isolated single-bit mask.

Value	Name	Description
= 1	<code>k_uart_gpio_bit_set</code>	Seed value for constructing a single-pin bitmask via left-shift

See also: `internal_configure_uart_pins()` Only consumer of this constant, `uart_init_channel()` Top-level function that triggers pin configuration

Since Version 1.0.0

—

s_channel_initialized

`bool s_channel_initialized[k_uart_array_size]`

Per-channel initialization state.

—

internal_calculate_brr

```
static uint8_t internal_calculate_brr(const uint32_t baudrate)
```

Calculate the 8-bit BRR register value for a target baud rate.

Computes the value to be written to the SCI Bit Rate Register (BRR) so that the SCI peripheral generates the requested baud rate from PCLKB. The general RX72N SCI baud rate formula for asynchronous mode is:

This driver always uses `n=0` (CKS bits = 0b00, clock source = PCLK/1), which simplifies to:

Algorithm steps:

1. Guard against `baudrate == 0` (return `k_brr_max_value` as a safe sentinel).
2. Guard against `baudrate > k_uart_baudrate_max` (return `k_brr_min_value`).
3. Compute `divisor_result = PCLKB / (32 * B)` using integer arithmetic.
4. Guard against `divisor_result <= 1` to prevent underflow from the `-1` offset.
5. Compute `BRR = divisor_result - 1`.
6. Clamp the result to `k_brr_max_value` (255) if the integer exceeds the 8-bit range (indicates a baud rate too slow for this clock).
7. Assert post-condition and return the clamped 8-bit result.

Baud rate error: Integer truncation introduces a small positive frequency error. At PCLKB = 60 MHz the worst-case error is +1.73 % (at 115 200 bps, BRR = 15), which is within the +/-2 % tolerance of the RS-232/UART standard.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	baudrate	Target baud rate in bps Valid range: 1 to k_uart_baudrate_max (k_pclkb_hz / k_brr_divisor_n0) Special case: 0 returns k_brr_max_value (255) as a safe sentinel Units: bits per second
----	----------	---

Returns: uint8_t BRR register value to program into sci->brr

Value	Description
0..254	Computed BRR for the requested baud rate
0	(k_brr_min_value) Returned when baudrate > k_uart_baudrate_max or divisor_result underflows the formula offset
255	(k_brr_max_value) Returned when baudrate == 0 or the computed value exceeds 255 (baud rate too low for n=0 divisor)

Pre: baudrate should be validated against k_uart_baudrate_min, k_uart_baudrate_max by the caller before invoking this function

Pre: k_pclkb_hz and k_brr_divisor_n0 must be non-zero compile-time constants

Post: Return value is always in 0, 255; no register overflow is possible

Post: Caller must write the returned value to sci->brr before enabling TX/RX

Note: This function performs only integer arithmetic; no floating-point is used, making it suitable for the RX72N toolchain with FPU disabled

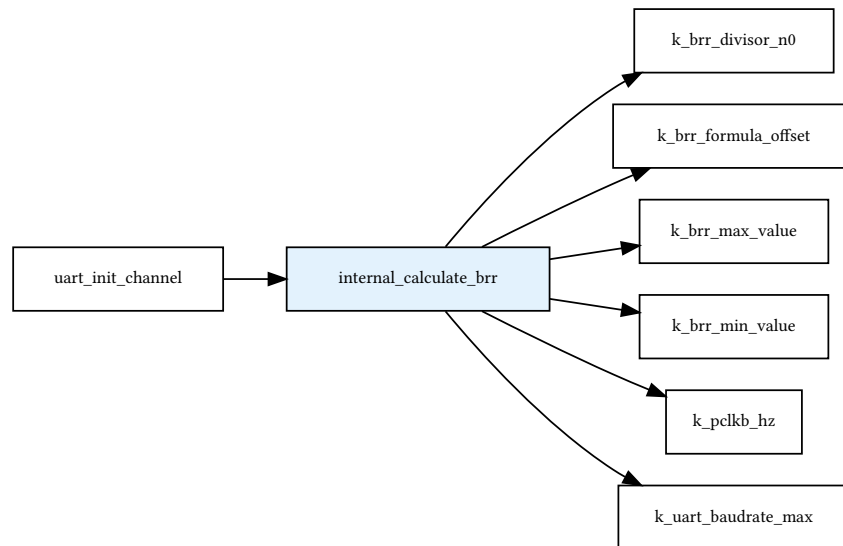
Note: Always uses n=0 (CKS=00); support for n=1..3 is not implemented

Thread Safety:: Stateless pure function; safe to call from any context including ISR.

Performance:: Execution time: 5 cycles (two integer multiplications and a comparison) Stack usage: 8 bytes (one local uint32_t)

Example:: sci->brr=internal_calculate_brr(115200U);

See also: uart_init_channel() Caller that validates baudrate and writes sci->brr, brr_constants_t Constants used in the BRR formula, uart_validation_limits_t Baud rate bounds checked before this call

Figure 882: Call graph for `internal_calculate_brr`

_Defined in `libs/rx\_hal/src/uart.c:486_`

Since Version 1.0.0

—

internal_clear_errors

```
static void internal_clear_errors(volatile rx_sci_regs_t *sci)
```

Clear receive error flags in the SCI Serial Status Register (SSR).

Reads the SSR register and writes it back with the three receive error flag bits masked out, which clears any pending ORER (Overrun Error), FER (Framing Error), and PER (Parity Error) conditions on the SCI channel.

The RX72N SCI peripheral requires a read-modify-write sequence to clear error flags: the hardware only allows clearing a flag by writing 0 to it while keeping the rest of the register unchanged. Writing 1 to an already- set flag has no effect (the write is ignored).

Algorithm steps:

1. Guard: return immediately if `sci` is nullptr (NASA Power of 10 Rule 5).
2. Read the current value of `sci->ssr` into a local volatile variable.
3. Cast-to-void the read result to suppress the unused-variable warning while still ensuring the volatile read is not optimised away.
4. Write `sci->ssr = (ssr & k_sci_ssr_error_mask)` to clear bits [5:3] (ORER, FER, PER) while preserving all other SSR bits.

Error flag bit positions in SSR:

Parameters:

Direction	Name	Description
-----------	------	-------------

in	sci	Pointer to the SCI register block for the target channel Valid range: Non-nullptr pointer obtained from sci_get_channel() Null handling: Returns silently if nullptr (no-op; no error return)
----	-----	---

Pre: sci must point to a valid, hardware-mapped rx_sci_regs_t register block

Pre: The SCI module clock must be enabled (MSTPCRB bit cleared) before any SSR access; undefined behaviour otherwise

Post: ORER, FER, and PER bits in sci->ssr are cleared (written to 0)

Post: All other SSR bits (TDRE, RDRF, TEND, MPB, MPBT) are preserved

Note: This function is intentionally void-returning; the caller (uart_getc_channel) checks for errors before calling and does not need a return code

Note: The intermediate volatile read prevents the compiler from merging the read and write into a single store, which would miss the read requirement

Thread Safety:: Not thread-safe. SSR is a read-modify-write target; concurrent access from two threads on the same channel can corrupt flag state. External mutex protection is required if multiple threads share a channel.

Performance:: Execution time: 3 cycles (volatile read + mask + volatile write) Stack usage: 4 bytes (one volatile uint8_t local)

Example:: `volatilerx_sci_regs_t*sci=sci_get_channel(k_uart_channel_9); if(sci!=nullptr) { internal_clear_errors(sci); constchardata=(char)sci->rdr; }`

See also: `uart_getc_channel()` Caller that invokes this before reading RDR, `sci_register_values_t k_sci_ssr_error_mask` bitmask definition, `uart_rx_available()` Non-destructive receive availability check

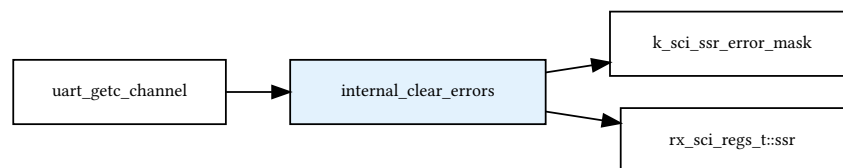


Figure 883: Call graph for internal_clear_errors

_Defined in libs/rx_hal/src/uart.c:587_

Since Version 1.0.0

—

internal_get_mstpb_bit

```
static int8_t internal_get_mstpb_bit(const uint8_t channel)
```

Return the MSTPCRB bit position that controls the module-stop clock gate for a given SCI channel.

The RX72N Module Stop Control Register B (MSTPCRB) contains one bit per SCI channel (SCI0-SCI11). Clearing a bit removes the module from the stopped state, allowing its clock to run. The bit layout is contiguous and descending: SCI0 occupies bit 31, SCI1 occupies bit 30, and so on down to SCI11 at bit 20.

SCI12 is controlled by a different register (MSTPCRC bit 4) and is therefore outside the scope of this function; channel 12 returns -1 to signal the caller that a different mechanism is needed.

Algorithm steps:

1. Check whether channel > k_uart_max_mstpb_channel (11); return -1 if so.
2. Compute bit position as k_sci_mstpb_sci0 (31) minus channel index.
3. Cast to int8_t and return.

Bit position table (MSTPCRB):

Parameters:

Direction	Name	Description
in	channel	SCI channel index Valid range: 0 to k_uart_max_mstpb_channel (11) Out-of-range: 12 or above returns -1 (SCI12 is in MSTPCRC) Units: dimensionless channel index

Returns: int8_t MSTPCRB bit position for the given channel

Value	Description
20..31	Valid bit position (MSTPCRB bit index)
-1	Channel is not in MSTPCRB (channel >= 12); caller must use an alternative register (MSTPCRC) or treat as an error

Pre: channel is obtained from a validated uart_channel_t value

Pre: k_sci_mstpb_sci0 must equal 31 so that the descending formula is correct

Post: Return value, if >= 0, is a valid bit index in 20, 31

Post: Caller is responsible for performing the register unlock/lock sequence around the MSTPCRB write

Note: Returns int8_t (signed) so that -1 can be used as an unambiguous sentinel without consuming any valid bit position in 0, 31

Note: SCI12 support requires a separate call path using MSTPCRC; this function deliberately does not handle it

Thread Safety:: Stateless pure function; safe to call from any context including ISR.

Performance:: Execution time: 2 cycles (one comparison, one subtraction) Stack usage: 0 bytes (no locals beyond return register)

Example:: `constint8_tbit=internal_get_mstpbit(9U); if(bit>=0){ system_regs()->mstpcrb&= (1UL<<(uint8_t)bit); }`

See also: `internal_enable_sci_clock()` Only caller; uses the returned bit to clear the module-stop gate in MSTPCRB, `sci_mstpbit_t` Enum defining all SCI MSTPCRB bit positions, `uart_internal_constants_t` `k_uart_max_mstpbit_channel` boundary constant

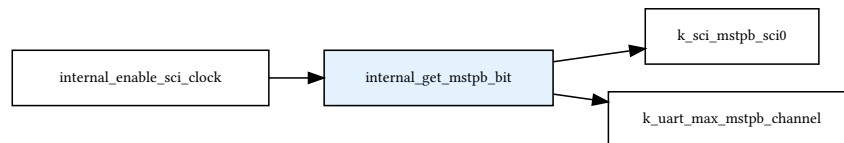


Figure 884: Call graph for `internal_get_mstpbit`

_Defined in `libs/rx\_hal/src/uart.c:677_`

Since Version 1.0.0

—

internal_enable_sci_clock

```
static rx_err_t internal_enable_sci_clock(const uint8_t channel)
```

Enable the peripheral clock for an SCI channel by clearing its Module Stop Control Register B (MSTPCRB) bit.

On reset, all SCI modules are held in the module-stop state (their clock gates are closed) to minimize power consumption. Before any SCI register can be accessed, the corresponding MSTPCRB bit must be cleared. Clearing the bit opens the clock gate and allows the SCI peripheral to operate.

The MSTPCRB register is write-protected by the Protect Register (PRCR). This function performs the required unlock/modify/lock sequence:

Algorithm steps:

1. Call `internal_get_mstpbit(channel)` to obtain the MSTPCRB bit index. Return `k_rx_err_invalid_arg` immediately if the result is negative (channel 12 or out of range).
2. Write `k_rx_prcr_unlock_all` to `*prcr_reg()` to remove write protection.
3. Clear the target bit in `system_regs()->mstpcrb` using a read-modify-write with the single-bit mask `k_uart_mstpcrb_bit_set` shifted left by `mstpbit`.
4. Write `k_rx_prcr_lock` to `*prcr_reg()` to restore write protection.
5. Return `k_rx_ok`.

Register sequence:

Parameters:

Direction	Name	Description
in	channel	SCI channel index to enable Valid range: 0 to k_uart_max_mstpb_channel (11) Invalid: 12 or above SCI12 uses MSTPCRC (not supported here) Units: dimensionless channel index

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success MSTPCRB bit cleared, SCI clock enabled
k_rx_err_invalid_arg	channel >= 12 (not in MSTPCRB); MSTPCRB is not modified and the channel clock remains gated

Pre: channel must be in range 0, k_uart_max_mstpb_channel (i.e., 0-11)

Pre: PRCR register must be accessible; this function does not check for nested unlock attempts

Post: On success, the MSTPCRB bit for the specified channel is cleared and the SCI module clock is running

Post: On success, the PRCR register is returned to its locked state

Note: This function does not re-assert module stop on error or deinit; the clock is left enabled for the lifetime of the MCU session

Warning: Do not call while another thread or ISR is modifying MSTPCRB or any other PRCR-protected register; the unlock/lock window is not atomic

Thread Safety:: Not thread-safe. The PRCR unlock-MSTPCRB write-PRCR lock sequence must execute atomically. Call only during single-threaded initialization before ThreadX starts.

Performance:: Execution time: 5 cycles (two PRCR writes + one RMW on MSTPCRB) Stack usage: 8 bytes (one int8_t local for mstpb_bit)

Example:: rx_err_t err=internal_enable_sci_clock(9U); if(err!=k_rx_ok){ return err; }

Example:

```
PRCR=0xA50B; //Unlock
MSTPCRB&=~(1UL<<bit); //Clear module-stopbit
PRCR=0xA500; //Lock
```

See also: internal_get_mstpb_bit() Helper that maps channel -> MSTPCRB bit index,
uart_init_channel() Caller that invokes this as part of channel setup,
uart_mstpcrb_constants_t k_uart_mstpcrb_bit_set single-bit mask seed

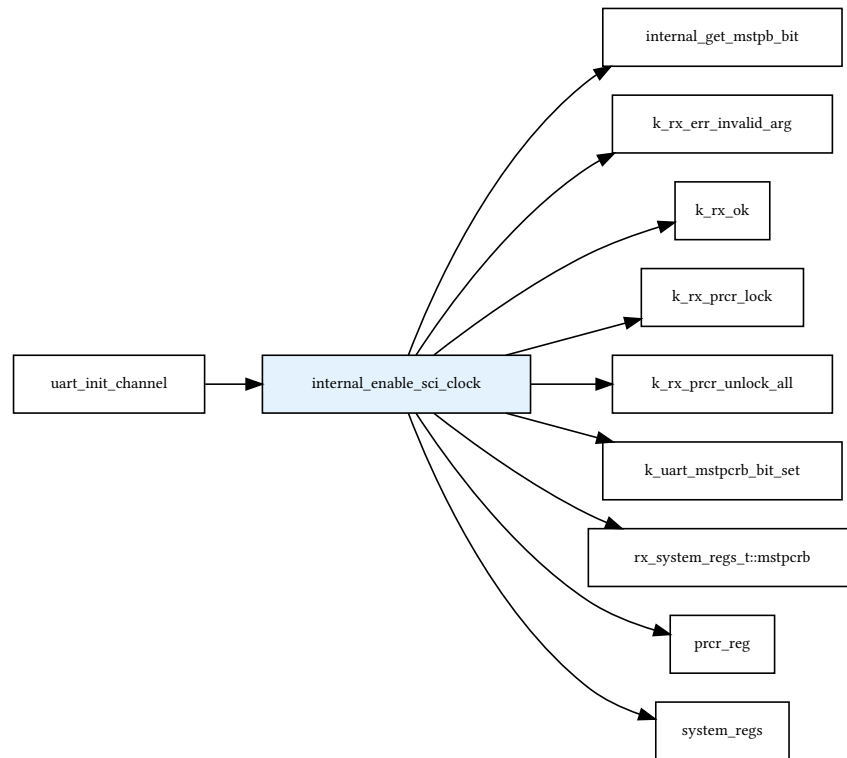


Figure 885: Call graph for internal_enable_sci_clock

Defined in `libs/rx\_hal/src/uart.c:766`

Since Version 1.0.0

—

internal_configure_uart_pins

```
static rx_err_t internal_configure_uart_pins(const rx_port_pin_t tx_gpio,
rx_port_pin_t rx_gpio)
```

Configure the GPIO and MPC pin-mux settings for a UART TX/RX pin pair.

Prepares the two GPIO pins required for SCI UART operation by programming the Multi-Function Pin Controller (MPC) and the Port Direction/Mode registers in the correct order. The RX72N hardware requires that:

- MPC is configured before switching a pin to peripheral mode (PMR)
- TX pin is driven as an output; RX pin is configured as an input
- Both pins are switched to peripheral mode (PMR bit = 1) last

Algorithm steps:

1. Extract the port number and pin number from each `rx_port_pin_t` using `rx_port_from_pin()` and `rx_pin_from_pin()`.

2. Validate that both pin numbers are $\leq k_rx_pin_max$ (7); return `k_rx_err_invalid_arg` if out of range. (Lower-bound check is omitted because pin numbers are `uint8_t` and `k_rx_pin_min == 0`, which would trigger -Wtype-limits.)
3. Obtain volatile port register base pointers via `rx_port_get_base()`; return `k_rx_err_invalid_arg` if either is `nullptr` (invalid port number).
4. Call `rx_mpc_set_sci(tx_gpio)` to set the TX pin's PFS register to the SCI TXD function; propagate any error immediately.
5. Call `rx_mpc_set_sci(rx_gpio)` to set the RX pin's PFS register to the SCI RXD function; propagate any error immediately.
6. Build per-pin bitmasks (`k_uart_gpio_bit_set << pin_number`).
7. Set PDR bit for TX pin (output direction) and PMR bit for TX pin (peripheral mode).
8. Clear PDR bit for RX pin (input direction) and set PMR bit for RX pin (peripheral mode).
9. Return `k_rx_ok`.

MPC write protection: `rx_mpc_set_sci()` internally handles the PFSWE unlock/lock sequence, so this function does not need to touch the PFSWE bit directly.

Pin direction and mode summary:

Parameters:

Direction	Name	Description
in	<code>tx_gpio</code>	TX pin encoded as <code>rx_port_pin_t</code> Valid range: Any <code>rx_port_pin_t</code> with port in $[0, k_rx_port_max]$ and pin in $[0, k_rx_pin_max]$ Typical value: <code>k_uart_debug_tx_gpio (k_rx_pb_7)</code> for SCI9 Encoding: Use <code>k_rx_p{port}_{pin}</code> constants from <code>rx_port_constants.h</code>
in	<code>rx_gpio</code>	RX pin encoded as <code>rx_port_pin_t</code> Valid range: Same constraints as <code>tx_gpio</code> ; must be distinct from <code>tx_gpio</code> Typical value: <code>k_uart_debug_rx_gpio (k_rx_pb_6)</code> for SCI9

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success both pins configured for SCI UART operation
<code>k_rx_err_invalid_arg</code>	<code>tx_pin > k_rx_pin_max</code> , <code>rx_pin > k_rx_pin_max</code> , or <code>rx_port_get_base()</code> returned <code>nullptr</code> for either port number
<code>k_rx_err_invalid_arg</code>	Propagated from <code>rx_mpc_set_sci()</code> if the MPC mapping is not supported for the given pin

Pre: `tx_gpio` and `rx_gpio` must correspond to physically valid RX72N port pins

Pre: The SCI module clock must already be enabled (MSTPCRB bit cleared) so that the SCI TXD/RXD MPC function codes are accepted

Post: TX pin: PDR bit set (output), PMR bit set (peripheral function)

Post: RX pin: PDR bit cleared (input), PMR bit set (peripheral function)

Post: Both pins' PFS registers configured for SCI TX/RX function via MPC

Note: Pin validation only checks upper bound ($>k_rx_pin_max$); lower bound (0) is omitted to avoid the -Wtype-limits compiler warning triggered by comparing an unsigned type to 0

Warning: Passing the same pin for both TX and RX produces undefined hardware behaviour; no duplicate-pin check is performed

Thread Safety:: Not thread-safe. PDR/PMR are read-modify-write targets shared with all GPIO operations on the same port. Call only during single-threaded initialization.

Performance:: Execution time: 10 us (dominated by two `rx_mpc_set_sci()` calls) Stack usage: 32 bytes (four `uint8_t` locals + two pointer locals)

Example:: `rx_err_t err = internal_configure_uart_pins(rx_port_pin_t)k_uart_debug_tx_gpio, (rx_port_pin_t)k_uart_debug_rx_gpio); if(err != k_rx_ok){ return err; }`

See also: `uart_init_channel()` Caller that passes validated `tx_gpio/rx_gpio`, `rx_mpc_set_sci()` MPC pin-function assignment (with PFSWE unlock), `rx_port_get_base()` Port register base address lookup, `uart_gpio_constants_t k_uart_gpio_bit_set` used to build pin bitmasks

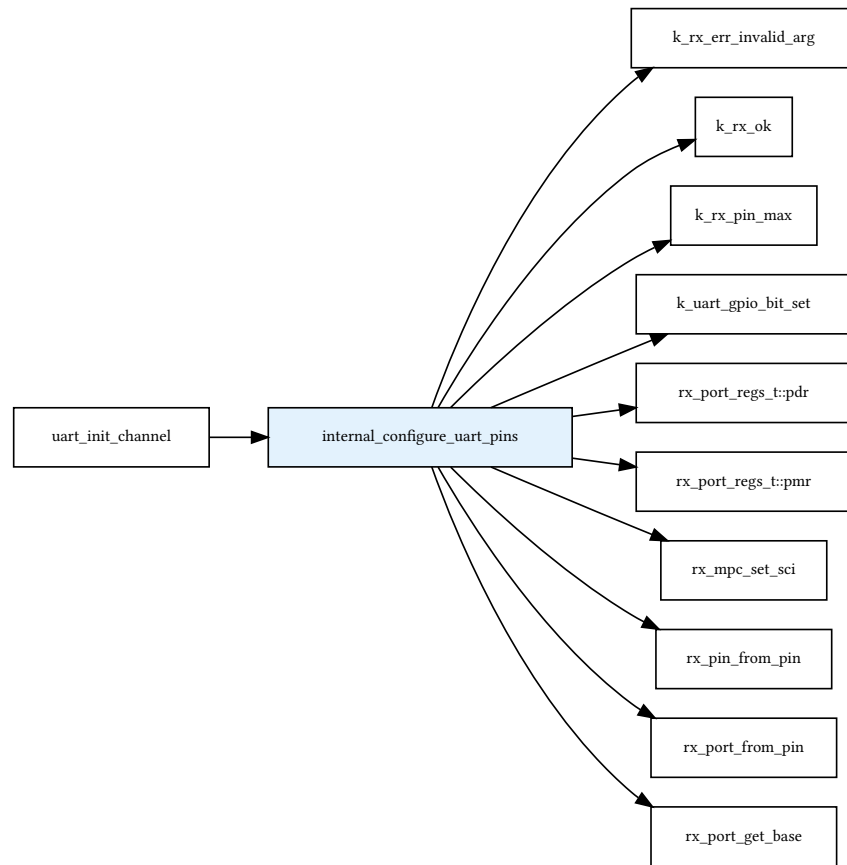


Figure 886: Call graph for internal_configure_uart_pins

_Defined in `libs/rx\hal/src/uart.c:884_`

Since Version 1.0.0

—

uart_init_channel

```
rx_err_t uart_init_channel(const uart_channel_config_t *config)
```

Initialize UART channel with specified configuration.

Initialize SCI channel for UART communication.

Configures and enables an SCI peripheral for asynchronous UART operation. Handles module clock enable, pin mux configuration, and SCI register setup.

Algorithm steps:

1. Validate configuration parameters (NULL check, channel range, baud rate)
2. Get SCI register base address for channel
3. Check if channel is already initialized (prevent double-init)

4. Enable SCI module clock (clear MSTPB bit)
5. Configure TX/RX pins via MPC and GPIO registers
6. Disable TX/RX during configuration (SCR = 0)
7. Set serial mode: async, 8N1, PCLK/1 (SMR)
8. Calculate and set baud rate divisor (BRR)
9. Wait for 1 bit time (synchronization)
10. Enable TX and RX (SCR = 0x30)
11. Mark channel as initialized

Parameters:

Direction	Name	Description
in	config	Pointer to channel configuration structure. channel: SCI channel (0-12) baudrate: Target baud rate (1 to PCLKB/32) tx_gpio: TX pin (rx_port_pin_t) rx_gpio: RX pin (rx_port_pin_t) Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, channel initialized and ready
k_rx_err_null_ptr	config parameter is nullptr
k_rx_err_invalid_arg	Invalid channel (>=13) or invalid baud rate
k_rx_err_invalid_state	Channel already initialized

Pre: config must point to valid uart_channel_config_t structure

Pre: config->channel must be in range 0, 12

Pre: config->baudrate must be in valid range for PCLKB

Pre: Channel must not be already initialized

Post: Channel configured for 8N1 async operation

Post: TX and RX enabled and ready for use

Post: s_channel_initializedchannel set to true

Post: TX/RX pins configured for peripheral function

Note: Not thread-safe - call once during initialization

Note: Includes 1ms delay for baud rate synchronization

Warning: Does not validate pin assignments against hardware

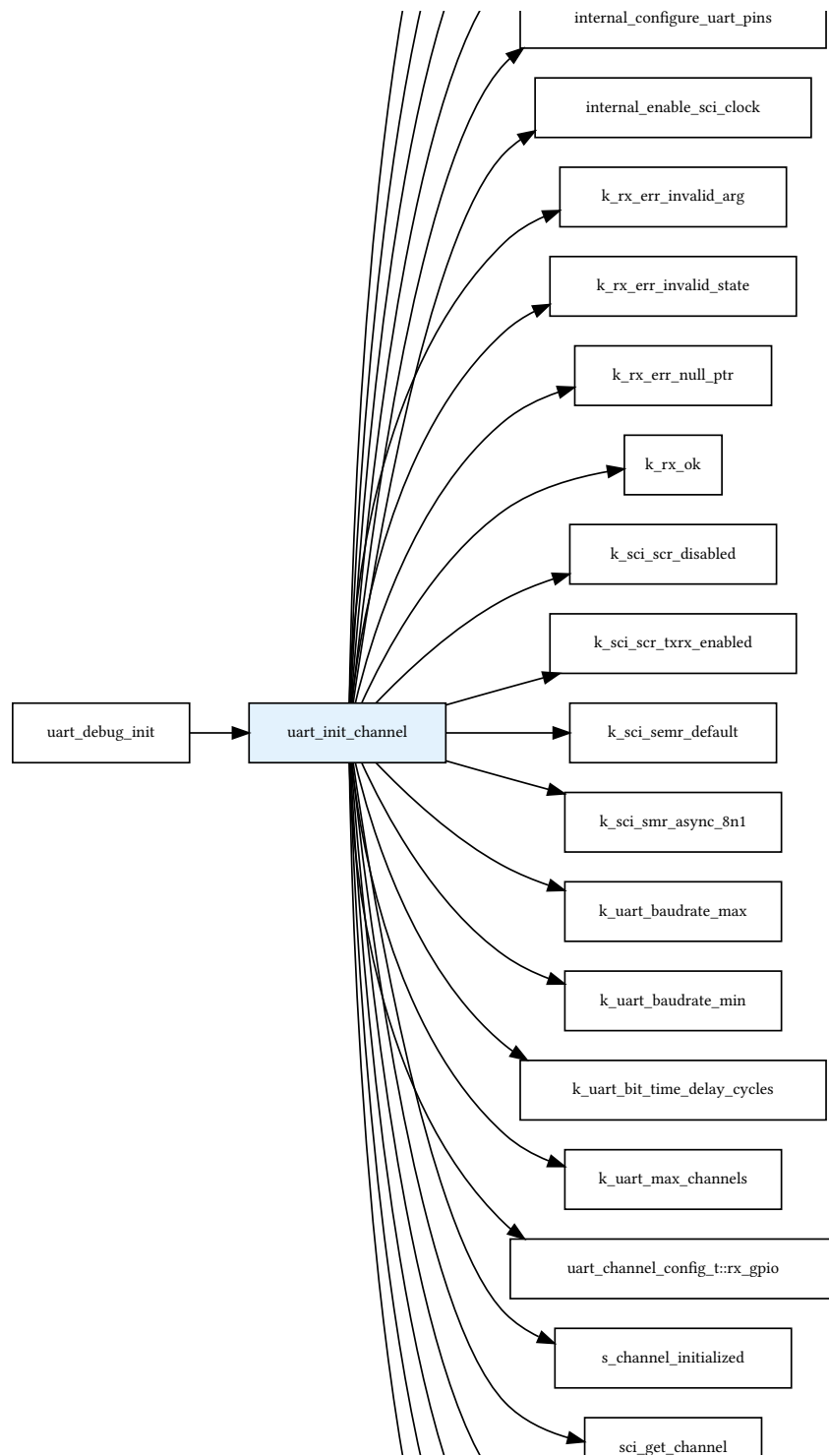
Thread Safety:: Not thread-safe. Call once per channel during system initialization.

Performance:: Execution time: 50 us (includes MPC and delay) Stack usage: 48 bytes

Example::

```
const uart_channel_config_t config = { .channel = k_uart_channel_9, .baudrate = 115200, .tx_gpio = k_rx_pb_7, .rx_gpio = k_
rx_err_t err = uart_init_channel(&config); if(err != k_rx_ok){ }
```

See also: `uart_deinit_channel()` Deinitialize channel, `uart_putc_channel()` Transmit single character, `uart_channel_config_t` Configuration structure

Figure 887: Call graph for `uart_init_channel`

_Defined in libs/rx_hal/src/uart.c:1047_

Since Version 1.0.0

—

uart_deinit_channel

```
rx_err_t uart_deinit_channel(const uart_channel_t channel)
```

Deinitialize UART channel and disable TX/RX.

Deinitialize SCI channel.

Disables transmit and receive on the specified SCI channel and marks it as uninitialized. The module clock is left running (safe to re-initialize without re-enabling). After this call the channel may be re-initialized with `uart_init_channel()`.

Algorithm steps:

1. Validate channel number (0-12)
2. Get SCI register base address
3. Write SCR = 0 to disable TX and RX
4. Clear `s_channel_initialized[channel]`

Parameters:

Direction	Name	Description
in	channel	UART channel to deinitialize (0-12) Valid range: 0 to 12 (SCI0 through SCI12) Recommended: Use <code>k_uart_channel_X</code> constants

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, channel TX/RX disabled
<code>k_rx_err_invalid_arg</code>	Invalid channel number (≥ 13) or invalid register pointer

Pre: channel must be in range 0, 12

Pre: Channel should be initialized before deinitializing (safe to call on uninit channel)

Post: TX and RX disabled (SCR = 0)

Post: `s_channel_initializedchannel` set to false

Note: Not thread-safe - call during shutdown, not during normal operation

Note: Module stop clock is NOT re-asserted (peripheral clock left enabled)

Thread Safety:: Not thread-safe. Do not call while another thread is using the channel.

Performance:: Execution time: 1 us Stack usage: 16 bytes

Example:: rx_err_t err=uart_deinit_channel(k_uart_channel_9); if(err!=k_rx_ok){ }

See also: uart_init_channel() Initialize channel, uart_debug_init() Initialize debug channel (SCI9)

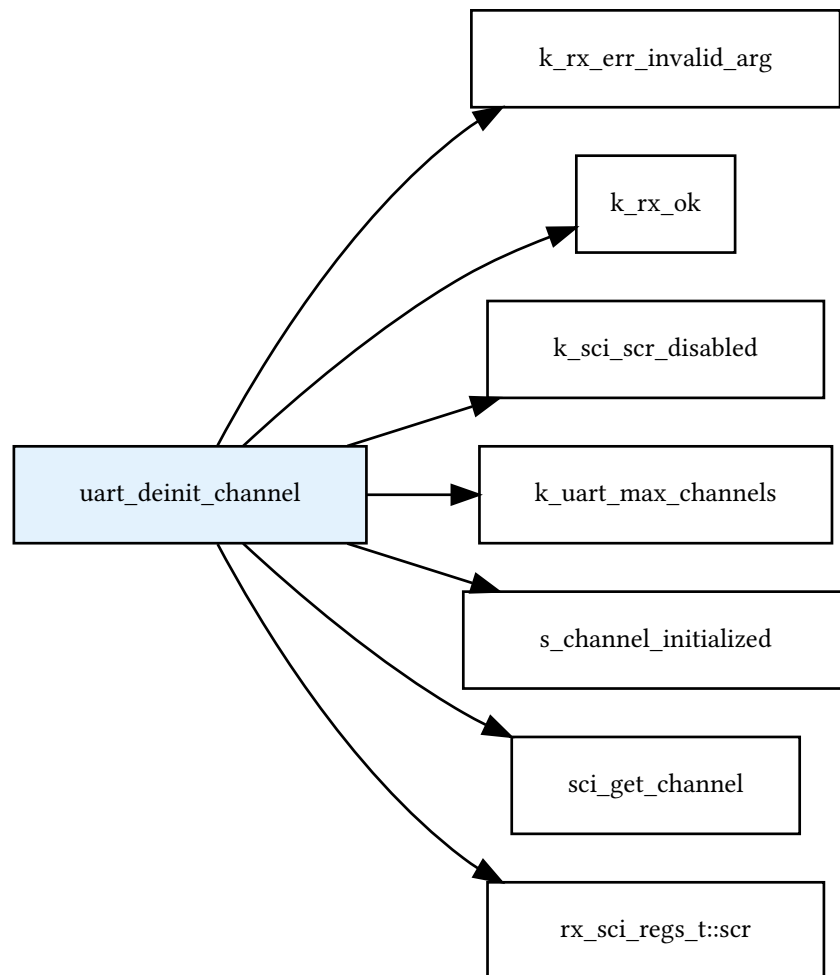


Figure 888: Call graph for `uart_deinit_channel`

_Defined in `libs/rx\hal/src/uart.c:1166_`

Since Version 1.0.0

—

uart_putc_channel

```
rx_err_t uart_putc_channel(const uart_channel_t channel, const char data)
```

Transmit single character on UART channel.

Transmit a single character on specified channel.

Transmits a single byte on the specified SCI channel. Waits for the transmit data register to become empty (TDRE flag) with timeout protection, then writes the data to TDR. Uses polling mode (no interrupts).

Algorithm steps:

1. Validate channel number (0-12)
2. Check channel is initialized
3. Get SCI register base address
4. Wait for TDRE flag with timeout (k_uart_tx_timeout iterations)
5. Write data byte to TDR register
6. Clear TDRE flag (read SSR, write back with TDRE=0)

Character transmission timing at 115200 baud:

- Start bit: 8.68 us
- 8 data bits: 69.44 us
- Stop bit: 8.68 us
- Total: 86.8 us per character

Parameters:

Direction	Name	Description
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12) Recommended: Use k_uart_channel_X constants
in	data	Character to transmit Valid range: 0x00 to 0xFF (any byte value) Special handling: ‘ ‘ not converted (use uart_puts_channel for conversion)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, character transmitted
k_rx_err_invalid_arg	Invalid channel number (>=13)
k_rx_err_invalid_state	Channel not initialized
k_rx_err_timeout	Transmit buffer did not become empty within timeout

Pre: Channel must be initialized via uart_init_channel()

Pre: UART TX must be enabled (happens during init)

Post: Character written to TDR and transmission started

Post: TDRE flag cleared (TDR occupied)

Note: Blocking call - waits for TDRE flag

Note: Timeout prevents infinite wait if hardware fails

Warning: May block for up to 100ms if TX line is held low

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 90 us @ 115200 baud (one character time) Stack usage: 16 bytes

Example:: `rx_err_t err=uart_putc_channel(k_uart_channel_9,'A'); if(err!=k_rx_ok){ }
uart_putc_channel(k_uart_channel_9,'r'); uart_putc_channel(k_uart_channel_9,'n');`

See also: `uart_puts_channel()` Transmit string with newline conversion,
`uart_write_channel()` Transmit binary data

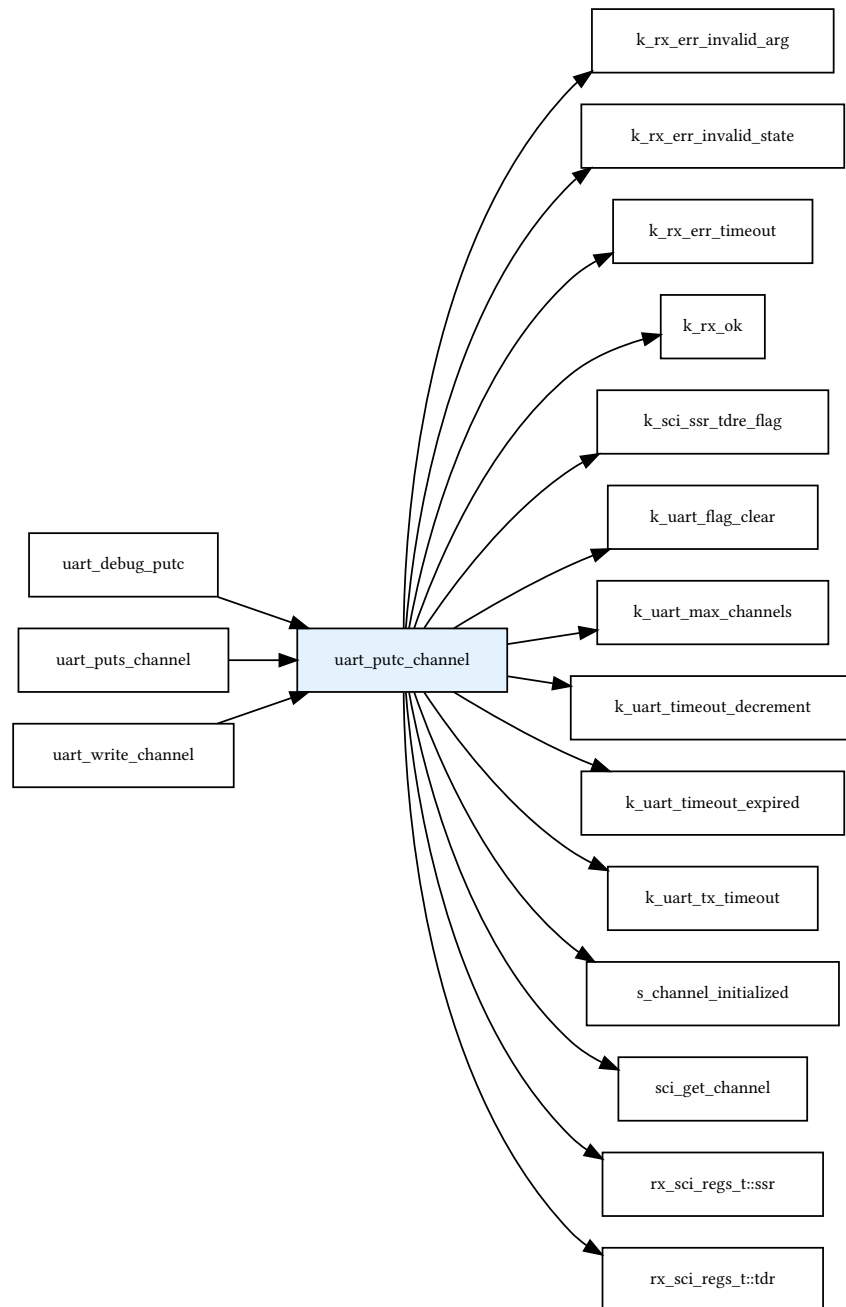


Figure 889: Call graph for `uart_putc_channel`

_Defined in `libs/rx_hal/src/uart.c:1259_`

Since Version 1.0.0

uart_puts_channel

```
rx_err_t uart_puts_channel(const uart_channel_t channel, const char *str)
```

Transmit null-terminated string on UART channel with newline conversion.

Transmit a null-terminated string on specified channel.

Transmits a null-terminated string with automatic LF to CR+LF conversion for terminal compatibility. Enforces maximum string length (k_uart_max_str_len = 256) to comply with NASA Power of 10 Rule 2 (bounded loops).

Algorithm steps:

1. Validate string pointer (NULL check)
2. Validate channel number and initialization state
3. For each character up to k_uart_max_str_len: a. Check for null terminator (success exit) b. If ‘
’, send ‘r’ first (CR+LF conversion) c. Transmit character via uart_putc_channel() d. Propagate any errors immediately
4. If limit reached without terminator, return k_rx_err_invalid_size

Newline conversion:

- Input ‘
(LF) -> Output ‘r’ (CR+LF)
- Required for Windows terminals (e.g., PuTTY, TeraTerm)
- Ensures cursor returns to column 0 before line feed

Parameters:

Direction	Name	Description
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
in	str	Pointer to null-terminated string Valid range: Non-nullptr to valid memory Maximum length: 256 characters (k_uart_max_str_len) Null handling: Returns k_rx_err_null_ptr if nullptr Encoding: ASCII (extended characters passed through)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, entire string transmitted
k_rx_err_null_ptr	str parameter is nullptr
k_rx_err_invalid_arg	Invalid channel number
k_rx_err_invalid_state	Channel not initialized
k_rx_err_invalid_size	String exceeds 256 characters
k_rx_err_timeout	Hardware timeout during transmission

Pre: Channel must be initialized via uart_init_channel()

Pre: str must point to null-terminated string in valid memory

Post: All characters transmitted including converted newlines

Post: On error, partial string may have been transmitted

Note: Blocking call - waits for each character to transmit

Note: String length limited to 256 characters for safety

Warning: Long strings may take significant time (256 chars 22ms @ 115200)

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes Example:
100-char string 9 ms

Example:: `uart_puts_channel(k_uart_channel_9, "Hello,World!\n");`

`uart_puts_channel(k_uart_channel_9, "Line1\nLine2\nLine3\n");`

`rx_err_t err = uart_puts_channel(k_uart_channel_9, msg); if (err != k_rx_ok) { }`

See also: `uart_putc_channel()` Transmit single character, `uart_write_channel()` Transmit binary data (no conversion), `uart_debug_puts()` Simplified debug wrapper

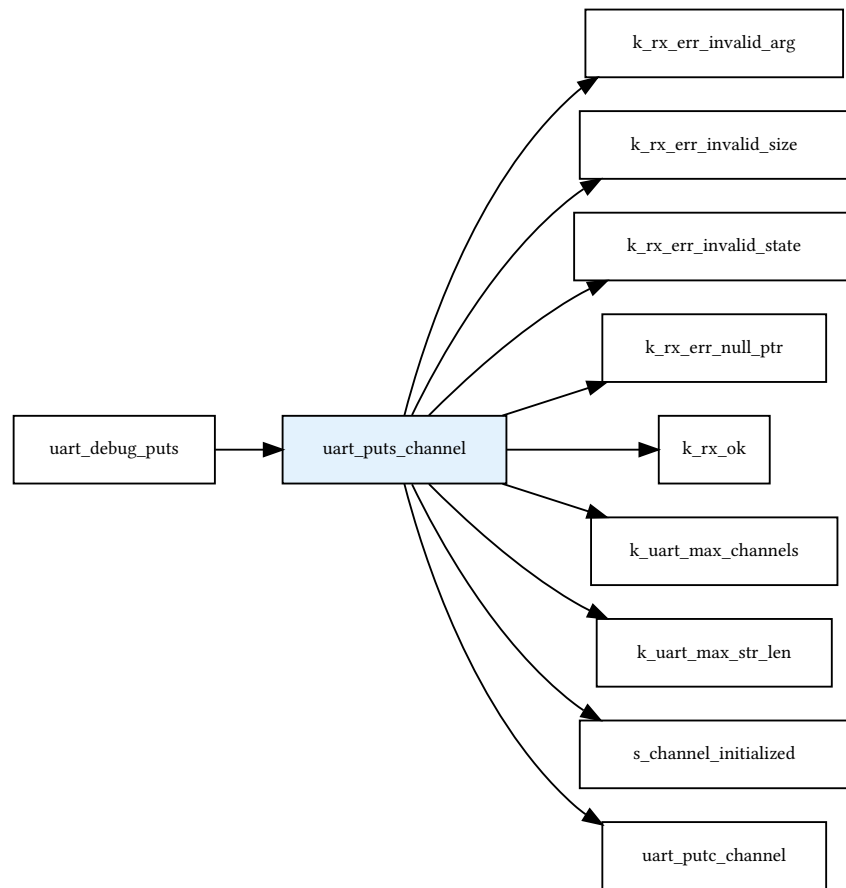


Figure 890: Call graph for uart_puts_channel

Defined in libs/rx\hal/src/uart.c:1380

Since Version 1.0.0

—

uart_write_channel

```
rx_err_t uart_write_channel(const uart_channel_t channel, const uint8_t *data,
uint16_t length)
```

Write binary data to UART channel without newline conversion.

Write buffer to specified channel.

Transmits a block of raw bytes on the specified SCI channel using `uart_putc_channel()` for each byte. No LF-to-CR+LF conversion is performed, making this function suitable for binary protocol data.

Algorithm steps:

1. Validate data pointer (NULL check)
2. Validate channel number and initialization state

3. For each byte in [0, length): call `uart_putc_channel()` and propagate errors

Parameters:

Direction	Name	Description
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
in	data	Pointer to buffer containing bytes to transmit Valid range: Non-nullptr pointing to at least length bytes Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr
in	length	Number of bytes to write Valid range: 0 to <code>UINT16_MAX</code> ; 0 returns <code>k_rx_ok</code> immediately

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, all bytes transmitted
<code>k_rx_err_null_ptr</code>	data parameter is nullptr
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_invalid_state</code>	Channel not initialized
<code>k_rx_err_timeout</code>	Hardware timeout during transmission

Pre: Channel must be initialized via `uart_init_channel()`

Pre: data must point to at least length bytes of valid memory

Post: All length bytes transmitted in order

Post: On error, partial data may have been transmitted

Note: Blocking call - waits for each byte to be accepted by TDR

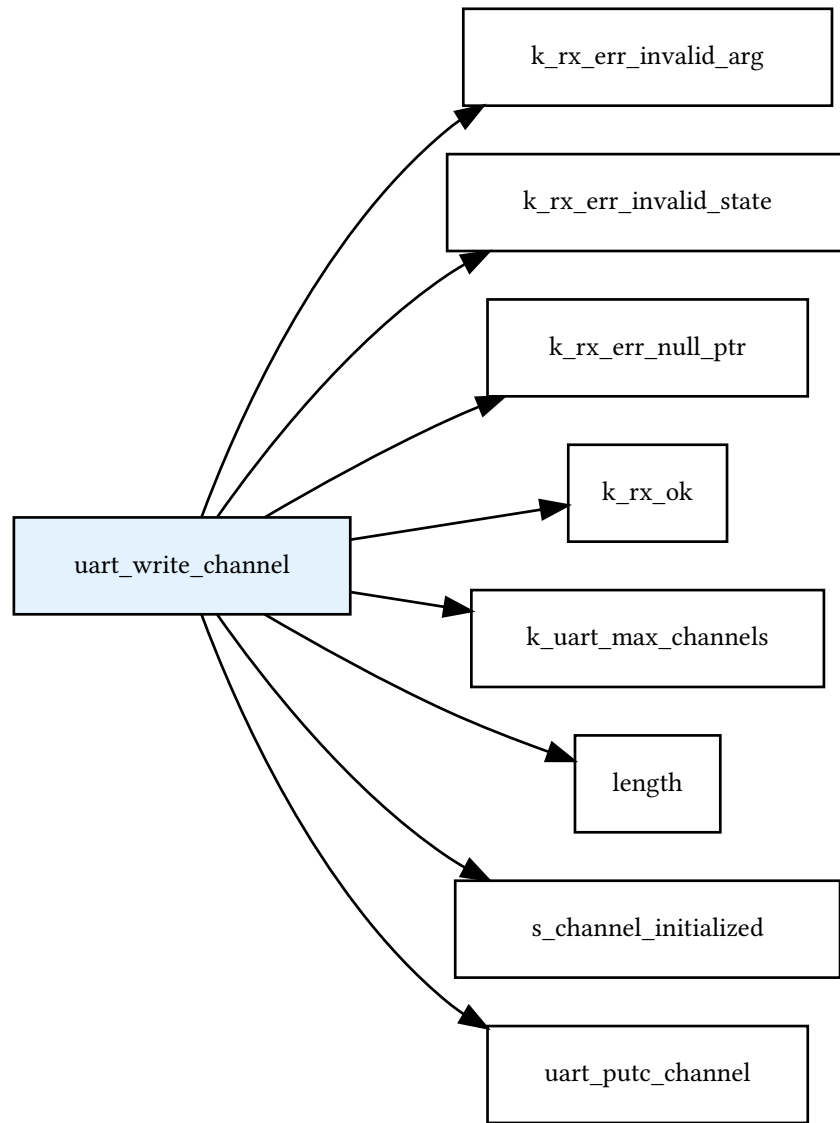
Note: No newline conversion - use `uart_puts_channel()` for text output

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 90 us per byte @ 115200 baud Stack usage: 24 bytes

Example:: `constuint8_tframe[]={0xAA,0x55,0x01,0x02,0x03};
rx_err_t err=uart_write_channel(k_uart_channel_9,frame,sizeof(frame)); if(err!=k_rx_ok){ }`

See also: `uart_puts_channel()` Transmit text string with newline conversion,
`uart_read_channel()` Read binary data

Figure 891: Call graph for `uart_write_channel`

_Defined in `libs/rx_hal/src/uart.c:1476_`

Since Version 1.0.0

—

uart_getc_channel

```
rx_err_t uart_getc_channel(const uart_channel_t channel, char *data)
```

Receive single character from UART channel (non-blocking).

Receive a single character from specified channel (non-blocking).

Attempts to read a single byte from the specified SCI channel's receive data register (RDR). This is a non-blocking operation - returns immediately with `k_rx_err_empty` if no data is available.

Algorithm steps:

1. Validate data pointer and channel number
2. Check channel initialization state
3. Get SCI register base address
4. Clear any error flags (ORER, FER, PER)
5. Check RDRF flag (Receive Data Register Full)

If not set, return `k_rx_err_empty` immediately

6. Read data byte from RDR register
7. Clear RDRF flag (read SSR, write back with RDRF=0)

Error flag handling:

- ORER (Overrun Error): Previous data overwritten before read
- FER (Framing Error): Stop bit not detected
- PER (Parity Error): Parity mismatch (not used in 8N1 mode)

Parameters:

Direction	Name	Description
in	channel	UART channel to use (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
out	data	Pointer to store received character Valid range: Non-nullptr to char On success: Contains received byte (0x00-0xFF) On error: Content undefined Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, received character stored in *data
<code>k_rx_err_null_ptr</code>	data parameter is nullptr
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_invalid_state</code>	Channel not initialized
<code>k_rx_err_empty</code>	No data available (RDRF not set)

Pre: Channel must be initialized via `uart_init_channel()`

Pre: data must point to valid memory for single char

Post: On success, *data contains received byte

Post: RDRF flag cleared (ready for next byte)

Post: Error flags cleared if any were set

Note: Non-blocking - returns immediately if no data

Note: Clears error flags automatically before read

Note: Single byte buffer - call frequently to avoid overrun

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 5 us (no wait) Stack usage: 16 bytes

Example (Polling Loop):: charc; rx_err_t err;

```
do{ err=uart_getc_channel(k_uart_channel_9,&c); if(err==k_rx_ok){ process_character(c); } }  
while(err!=k_rx_ok);
```

Example (Timeout Loop):: charc; uint32_t timeout=100000;

```
while(timeout>0){ rx_err_t err=uart_getc_channel(k_uart_channel_9,&c); if(err==k_rx_ok){ break; }  
timeout--;
```

```
if(timeout==0){ }
```

See also: `uart_read_channel()` Read multiple bytes, `uart_rx_available()` Check if data is available

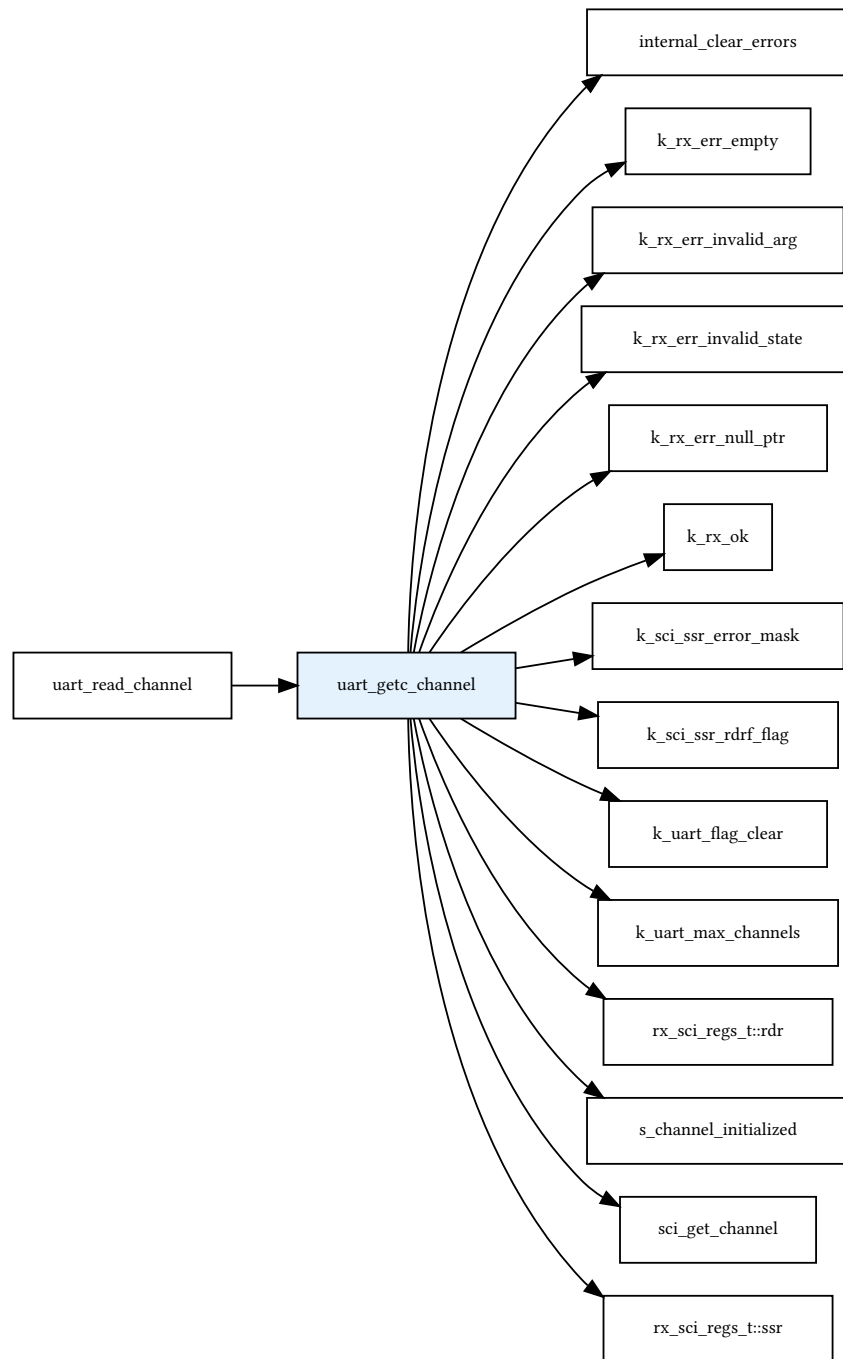


Figure 892: Call graph for `uart_getc_channel`

_Defined in `libs/rx_hal/src/uart.c:1597_`

Since Version 1.0.0

—

uart_read_channel

```
rx_err_t uart_read_channel(const uart_channel_t channel, uint8_t *data, uint16_t
length, uint16_t *bytes_read)
```

Read available bytes from UART channel (non-blocking).

Read available data from specified channel (non-blocking).

Reads up to length bytes from the specified SCI channel using `uart_getc_channel()`. Stops immediately when no more data is available (RDRF flag not set) rather than waiting. Actual bytes received is reported via `bytes_read`.

Algorithm steps:

1. Validate data and bytes_read pointers (NULL check)
2. Validate channel number and initialization state
3. Set `*bytes_read = 0`
4. For each slot in `[0, length)`: a. Call `uart_getc_channel()`; if `k_rx_err_empty`, break (done) b. On other error, propagate immediately c. Store byte and increment `*bytes_read`
5. Return `k_rx_ok` (even if zero bytes were read)

Parameters:

Direction	Name	Description
in	channel	UART channel to read from (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
out	data	Pointer to buffer to store received bytes Valid range: Non-nullptr pointing to at least length bytes Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr
in	length	Maximum number of bytes to read Valid range: 0 to <code>UINT16_MAX</code>
out	bytes_read	Pointer to store actual number of bytes read Valid range: Non-nullptr to <code>uint16_t</code> On success: Set to number of bytes actually received (0..length) Null handling: Returns <code>k_rx_err_null_ptr</code> if nullptr

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success; <code>*bytes_read</code> contains actual count (may be 0)
<code>k_rx_err_null_ptr</code>	data or bytes_read is nullptr
<code>k_rx_err_invalid_arg</code>	Invalid channel number
<code>k_rx_err_invalid_state</code>	Channel not initialized

Pre: Channel must be initialized via `uart_init_channel()`

Pre: data must point to at least length bytes of writable memory

Post: *bytes_read set to actual number of bytes received

Post: data[0..bytes_read-1] contain the received bytes

Note: Non-blocking - returns immediately with whatever data is available

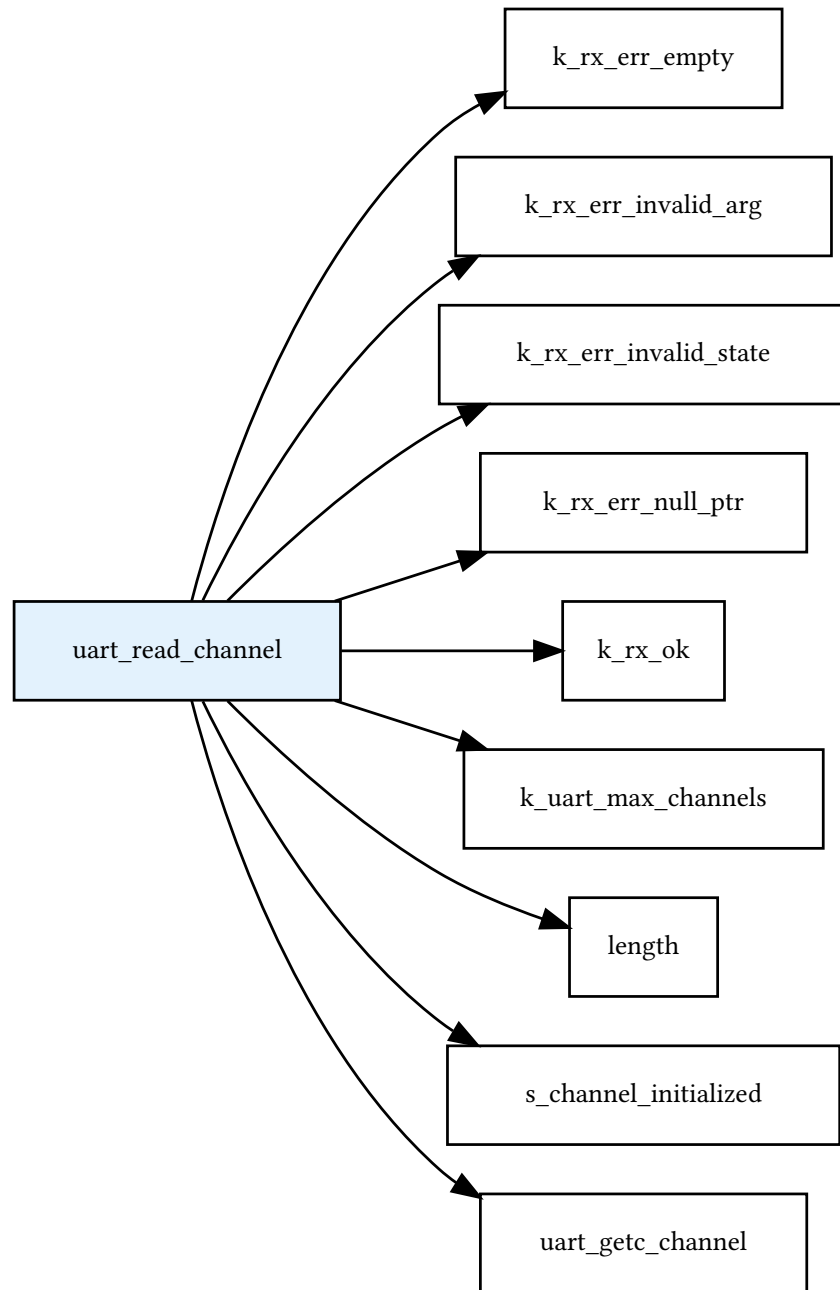
Note: k_rx_ok with *bytes_read == 0 means no data was available

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 5 us per byte received + 5 us when no data Stack usage: 24 bytes

Example:: uint8_t buf[64]; uint16_t count=0;
rx_err_t err=uart_read_channel(k_uart_channel_9,buf,sizeof(buf),&count);
if(err==k_rx_ok&&count>0){ process_bytes(buf,count); }

See also: uart_getc_channel() Read single character, uart_rx_available() Check if data is available, uart_write_channel() Write binary data

Figure 893: Call graph for `uart_read_channel`

_Defined in `libs/rx_hal/src/uart.c:1711_`

Since Version 1.0.0

—

uart_rx_available

```
rx_err_t uart_rx_available(const uart_channel_t channel, bool *available)
```

Check if receive data is available on UART channel.

Check if receive data is available on channel.

Checks the RDRF (Receive Data Register Full) flag in the SSR register of the specified SCI channel. Provides a non-destructive peek at receive availability without consuming any data from the buffer.

Algorithm steps:

1. Validate available pointer (NULL check)
2. Validate channel number and initialization state
3. Get SCI register base address
4. Read SSR and test RDRF bit; store boolean result in *available

Parameters:

Direction	Name	Description
in	channel	UART channel to check (0-12) Valid range: 0 to 12 (SCI0 through SCI12)
out	available	Pointer to store result On success: true if RDRF flag set (data ready), false otherwise On error: value undefined Null handling: Returns k_rx_err_null_ptr if nullptr

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success; *available set to data availability status
k_rx_err_null_ptr	available is nullptr
k_rx_err_invalid_arg	Invalid channel number or invalid register pointer
k_rx_err_invalid_state	Channel not initialized

Pre: Channel must be initialized via uart_init_channel()

Pre: available must point to valid bool storage

Post: *available reflects current RDRF state

Post: No data is consumed from the receive buffer

Note: Non-destructive - does not read RDR or clear any flags

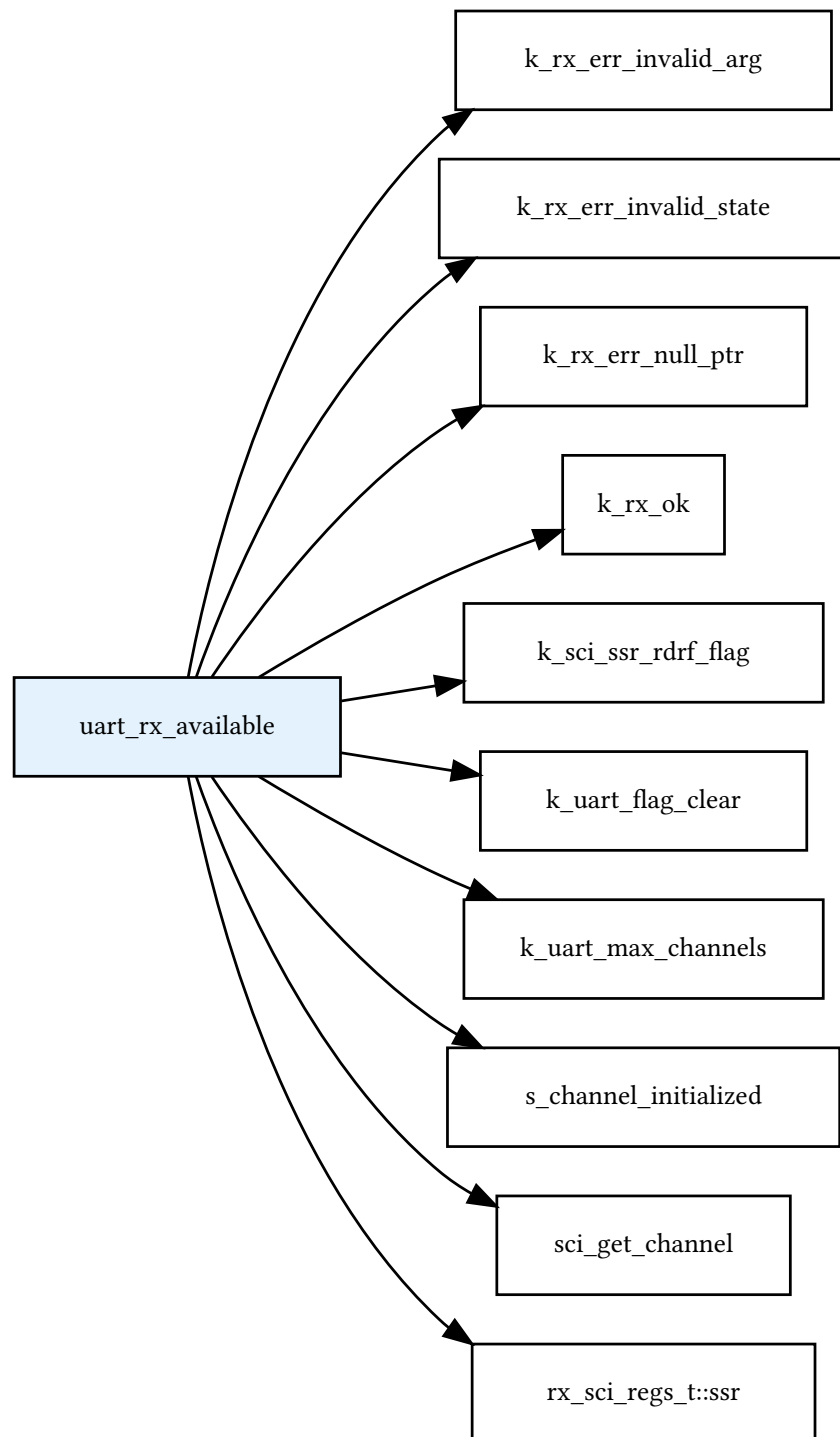
Note: RDRF can be set again immediately after reading if UART is receiving

Thread Safety:: Thread-safe for different channels. Not safe for same channel without mutex.

Performance:: Execution time: 2 us Stack usage: 16 bytes

Example:: boolready=false; if(uart_rx_available(k_uart_channel_9,&ready)==k_rx_ok&&ready)
{ charc; uart_getc_channel(k_uart_channel_9,&c); }

See also: uart_getc_channel() Read single character, uart_read_channel() Read multiple bytes

Figure 894: Call graph for `uart_rx_available`

_Defined in libs/rx_hal/src/uart.c:1806_

Since Version 1.0.0

—

uart_debug_init

```
rx_err_t uart_debug_init(void)
```

Initialize default debug UART channel (SCI9 at 115200 baud).

Initialize debug UART (SCI9, 115200 bps, 8N1).

Convenience function to initialize SCI9 with default debug settings:

- Channel: SCI9
- Baud rate: 115200 bps
- TX pin: PB7 (TXD9)
- RX pin: PB6 (RXD9)
- Format: 8N1 (8 data bits, no parity, 1 stop bit)

SCI9 is connected to the CY7C65213 USB-UART bridge on the STAR PCB, providing a virtual COM port when connected via USB.

Algorithm steps:

1. Create `uart_channel_config_t` with default values
2. Call `uart_init_channel()` with the configuration
3. Return result

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, debug UART ready
<code>k_rx_err_invalid_state</code>	SCI9 already initialized

Pre: System clocks must be initialized (PCLKB = 60 MHz)

Pre: SCI9 must not be already initialized

Post: SCI9 configured and ready for TX/RX at 115200 baud

Post: `uart_debug_puts()`, `uart_debug_putc()` etc. are functional

Note: Call once during early system initialization

Note: Call before any other debug output functions

Warning: Must be called before ThreadX starts if debug output is needed during init

Thread Safety:: Not thread-safe. Call once during startup before ThreadX.

Performance:: Execution time: 50 us Stack usage: 64 bytes

Example:: voidsystem_init(void) { rx_clock_init();
rx_err_t err=uart_debug_init(); if(err!=k_rx_ok){ return; }
uart_debug_puts("[INFO]Systemstarting...n"); }

See also: uart_init_channel() Generic channel initialization, uart_debug_puts() Debug string output, uart_debug_putc() Debug character output

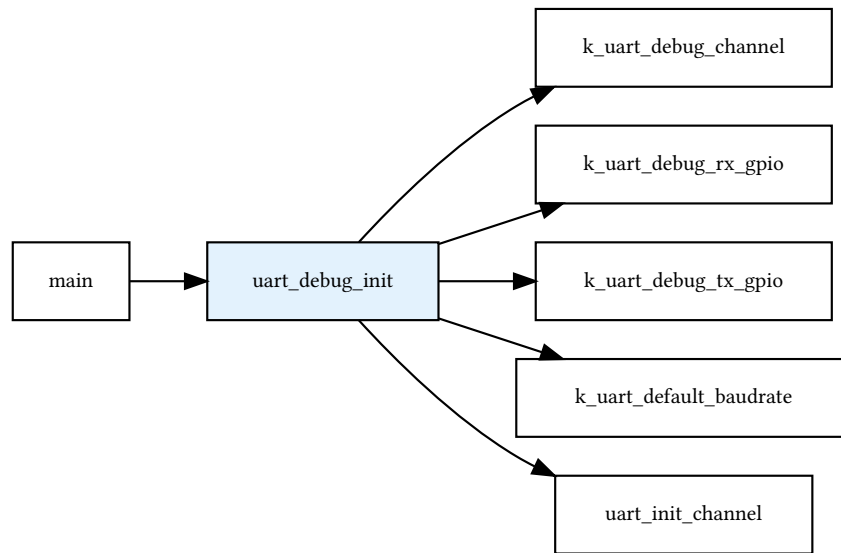


Figure 895: Call graph for `uart_debug_init`

_Defined in `libs/rx\_hal/src/uart.c:1904_`

Since Version 1.0.0

—

uart_debug_putc

```
void uart_debug_putc(const char data)
```

Transmit single character on debug UART (SCI9).

Transmit a single character on debug UART (SCI9).

Convenience wrapper around `uart_putc_channel()` for the fixed debug channel (SCI9). Errors are silently ignored to allow use in early initialization contexts where error propagation is not yet possible.

Algorithm steps:

1. Call `uart_putc_channel(k_uart_debug_channel, data)`
2. Cast return value to (void) - errors discarded

Parameters:

Direction	Name	Description
in	data	Character to transmit (0x00-0xFF) Special handling: ‘ ‘ is NOT converted; use uart_debug_puts() for text

Pre: uart_debug_init() must have been called successfully

Pre: SCI9 must be initialized and TX enabled

Post: Character written to SCI9 TDR and transmission started

Post: Any errors are silently discarded

Note: Error return from uart_putc_channel() is intentionally discarded

Note: For error-checked output use uart_putc_channel() directly

Warning: Only available when RX_IS_SIMULATOR is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us @ 115200 baud Stack usage: 16 bytes

Example:: uart_debug_putc('A'); uart_debug_putc('r'); uart_debug_putc('n');

See also: uart_debug_puts() Transmit string with newline conversion, uart_putc_channel() Error-checked single character transmit

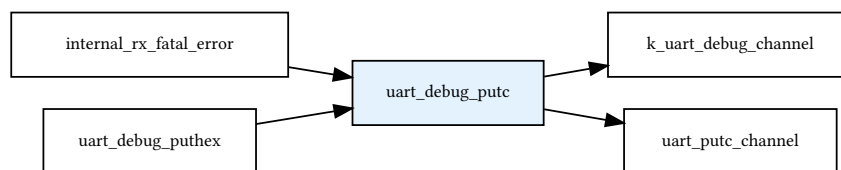


Figure 896: Call graph for uart_debug_putc

Defined in libs/rx\hal/src/uart.c:1969

Since Version 1.0.0

—

uart_debug_puts

```
void uart_debug_puts(const char *str)
```

Transmit null-terminated string on debug UART with newline conversion.

Transmit a null-terminated string on debug UART (SCI9).

Transmits a null-terminated string to SCI9 with automatic LF-to-CR+LF conversion for terminal compatibility. Enforces a maximum length of `k_uart_max_str_len` (256) characters per NASA Power of 10 Rule 2. Silently returns on NULL pointer to allow safe use in early init.

Algorithm steps:

1. Return immediately if `str` is `nullptr` (defensive, no error return)
2. For each character up to `k_uart_max_str_len`: a. Stop at null terminator b. If ‘
‘, send ‘r’ first c. Send character via `uart_debug_putc()`

Parameters:

Direction	Name	Description
in	<code>str</code>	Pointer to null-terminated ASCII string Maximum length: 256 characters (<code>k_uart_max_str_len</code>) Null handling: Returns silently if <code>nullptr</code> Encoding: ASCII; ‘ ‘ converted to ‘r’

Pre: `uart_debug_init()` must have been called successfully

Pre: `str` should point to a null-terminated string in valid memory

Post: All characters up to null terminator transmitted to SCI9

Post: Each ‘]’ replaced by ‘r’ in the transmitted output

Note: No return value - errors from `uart_debug_putc()` are silently discarded

Note: String truncated at `k_uart_max_str_len` (256) characters

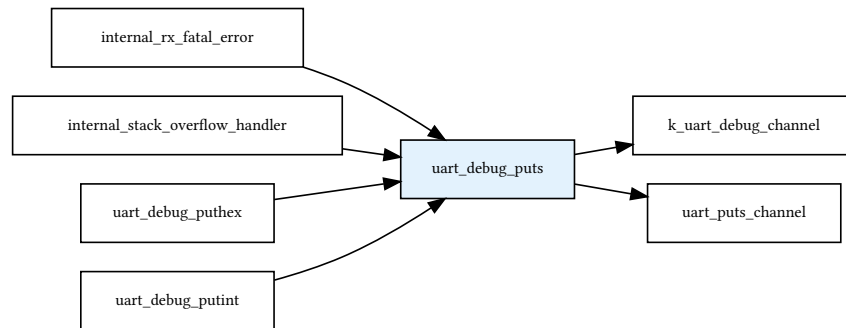
Warning: Only available when `RX_IS_SIMULATOR` is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes

Example:: `uart_debug_puts("Hello,World!\n");` `uart_debug_puts("[INFO]Bootcompleten");`

See also: `uart_debug_putc()` Transmit single character, `uart_debug_putint()` Transmit decimal integer, `uart_puts_channel()` Error-checked string transmit

Figure 897: Call graph for `uart_debug_puts`

_Defined in `libs/rx\_hal/src/uart.c:2025_`

Since Version 1.0.0

—

uart_debug_putint

```
void uart_debug_putint(const int32_t value)
```

Transmit signed 32-bit integer as decimal string on debug UART.

Print a signed integer on debug UART (SCI9).

Converts a signed 32-bit integer to a decimal ASCII string and transmits it on SCI9. Handles INT32_MIN correctly by using `int64_t` for the negation. Uses a fixed-size stack buffer (`k_uart_int_buffer_size = 12`) built in reverse then passed to `uart_debug_puts()`.

Algorithm steps:

1. Determine sign; compute `abs_value` as `uint32_t`
2. Null-terminate the buffer end
3. Build digits right-to-left (statically bounded by `k_uart_int_buffer_size`)
4. Prepend '-' if negative
5. Call `uart_debug_puts()` with the resulting substring pointer

Parameters:

Direction	Name	Description
in	value	Signed 32-bit integer to print Valid range: INT32_MIN (-2147483648) to INT32_MAX (2147483647) INT32_MIN handling: Correctly handled via <code>int64_t</code> cast

Pre: `uart_debug_init()` must have been called successfully

Pre: `uart_debug_puts()` must be functional

Post: Decimal representation of value transmitted on SCI9

Post: Leading zeros suppressed; negative values prefixed with '-'

Note: No return value - output errors are silently discarded

Note: Buffer is stack-allocated; safe for re-entrant calls on different tasks

Warning: Only available when RX_IS_SIMULATOR is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per digit @ 115200 baud + digit-loop overhead Stack usage: 32 bytes (buffer + locals)

Example:: `uart_debug_puts("Value:"); uart_debug_putint(42); uart_debug_putint(-2147483648);
uart_debug_putc('n');`

See also: `uart_debug_puthex()` Print value in hexadecimal, `uart_debug_puts()` Underlying string transmit

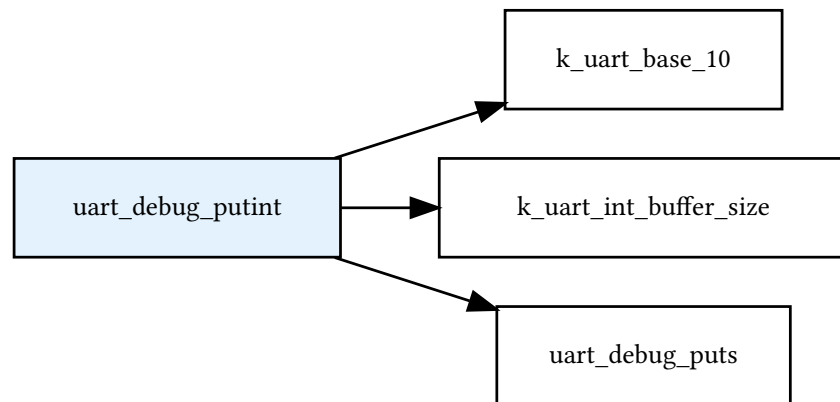


Figure 898: Call graph for `uart_debug_putint`

_Defined in `libs/rx\hal/src/uart.c:2086_`

Since Version 1.0.0

—

uart_debug_puthex

```
void uart_debug_puthex(const uint32_t value, uint8_t digits)
```

Transmit 32-bit unsigned integer as hexadecimal string on debug UART.

Print a hexadecimal value on debug UART (SCI9).

Transmits "0x" prefix followed by the specified number of uppercase hex digits of value on SCI9. Digits are always printed most-significant first. The `digits` parameter is clamped to `[k_uart_hex_min_digits, k_uart_hex_max_digits]` (1-8) before use.

Algorithm steps:

1. Send "0x" prefix via `uart_debug_puts()`
2. Clamp digits to [1, 8]
3. Iterate nibbles from MSN to LSN (statically bounded by `k_uart_hex_max_digits`)
4. For each nibble index < digits, extract nibble and print uppercase hex char

Parameters:

Direction	Name	Description
in	value	Unsigned 32-bit value to display in hexadecimal Valid range: 0x00000000 to 0xFFFFFFFF
in	digits	Number of hex digits to print (1-8) Valid range: 1 to 8 (clamped; 0 -> 1, >8 -> 8) Common values: 2 for byte, 4 for word, 8 for full 32-bit

Pre: `uart_debug_init()` must have been called successfully

Pre: `uart_debug_puts()` and `uart_debug_putc()` must be functional

Post: "0x" followed by digits uppercase hex characters transmitted on SCI9

Post: digits clamped to 1, 8 if out of range

Note: Uses static lookup table `s_hex` for digit-to-character conversion

Note: Always prefixes output with "0x"

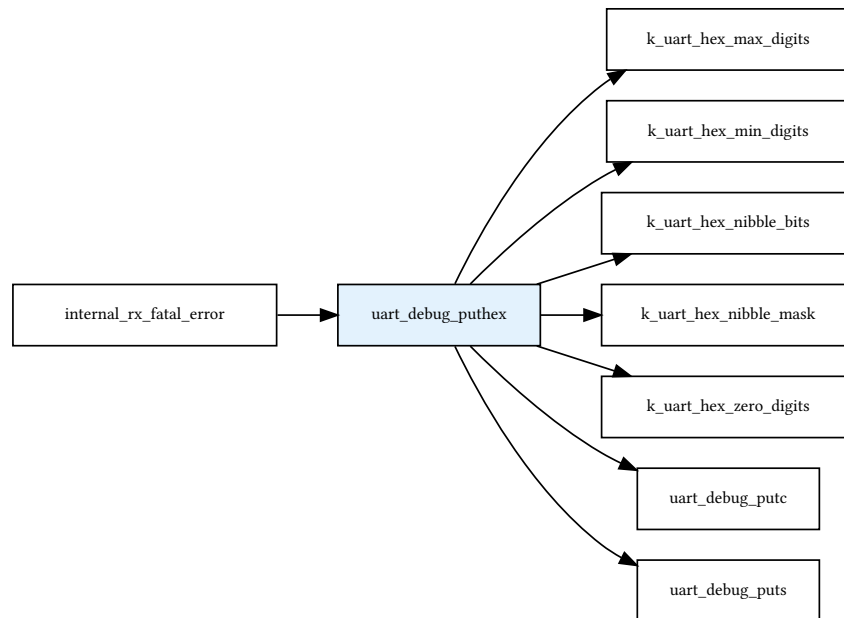
Warning: Only available when `RX_IS_SIMULATOR` is 0 (hardware builds)

Thread Safety:: Not safe for concurrent access on SCI9 without external mutex.

Performance:: Execution time: 90 us per character @ 115200 baud Stack usage: 24 bytes

Example:: `uart_debug_puts("Addr:"); uart_debug_puthex(0xDEADBEEF,8);
uart_debug_puthex(0x42,2); uart_debug_putc('n');`

See also: `uart_debug_putint()` Print signed decimal value, `uart_debug_puts()` Underlying string transmit

Figure 899: Call graph for `uart_debug_puthex`

_Defined in `libs/rx\hal/src/uart.c:2171_`

Since Version 1.0.0

—

rx_harq.h

Hybrid Automatic Repeat Request (HARQ) Protocol for RX72N.

Implements Chase Combining HARQ Type I for reliable communication over lossy channels. Combines FEC (Forward Error Correction) with ARQ (Automatic Repeat Request) to achieve both error correction and detection.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_fec.h"`

rx_harq_params_t

HARQ protocol parameters.

Value	Name	Description
= 1024	<code>k_harq_max_payload</code>	Maximum payload size in bytes
= 3	<code>k_harq_default_retries</code>	Default maximum retries

= 3	k_harq_default_combines	Default max combining attempts
= 16400	k_harq_soft_buffer_size	Max soft buffer size for Chase Combining (1024*8+6)*2 = 16396 bits, rounded to 16400
= k_harq_soft_buffer_size / 2U	k_harq_survivors_buffer_size	Survivors buffer size

—

rx_harq_state_t

HARQ state machine states.

Value	Name	Description
= 0	k_harq_state_idle	Ready for new transmission
= 1	k_harq_state_waiting_ack	Waiting for ACK
= 2	k_harq_state_combining	Combining retransmissions
= 3	k_harq_state_error	Unrecoverable error

—

rx_chase_combiner_init

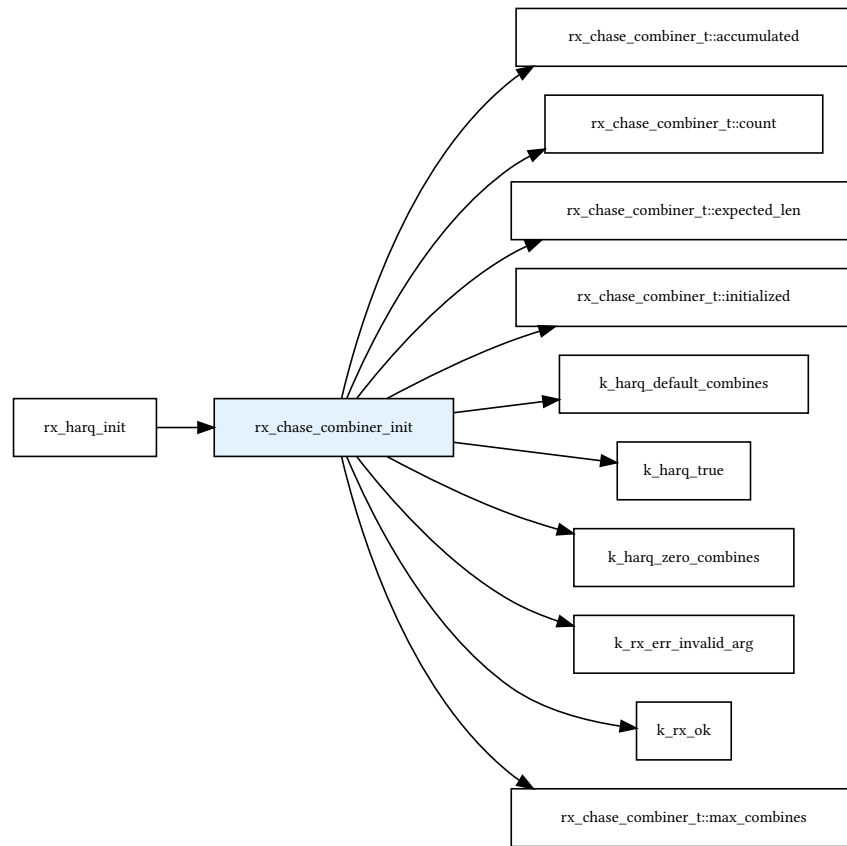
```
rx_err_t rx_chase_combiner_init(rx_chase_combiner_t *combiner, uint8_t max_combines)
```

Initialize Chase Combiner.

Parameters:

Direction	Name	Description
out	combiner	Pointer to combiner handle
in	max_combines	Maximum number of combining attempts (0 = default)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr

Figure 900: Call graph for `rx_chase_combiner_init`

Defined in `libs/rx_harq/inc/rx_harq.h:219`

—

rx_chase_combiner_deinit

```
rx_err_t rx_chase_combiner_deinit(rx_chase_combiner_t *combiner)
```

Deinitialize Chase Combiner.

Parameters:

Direction	Name	Description
inout	combiner	Pointer to combiner handle

Returns: `k_rx_ok` on success

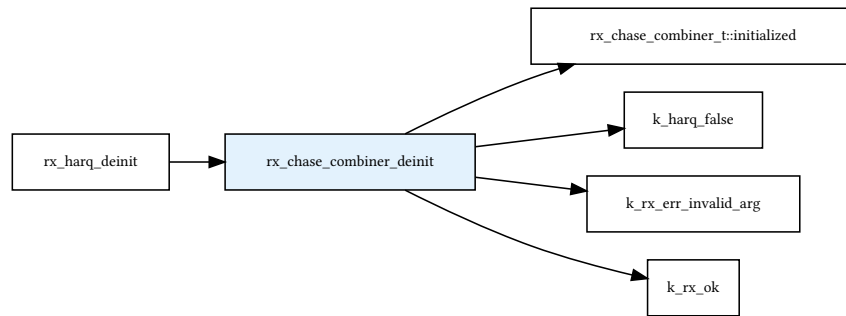


Figure 901: Call graph for rx_chase_combiner_deinit

Defined in `libs/rx_harq/inc/rx_harq.h:227`

—

rx_chase_combiner_add

```
rx_err_t rx_chase_combiner_add(rx_chase_combiner_t *combiner, const rx_soft_bit_t
*soft_bits, uint32_t len)
```

Add soft bits from a transmission attempt.

Accumulates soft bits element-wise. First call sets expected length.

Parameters:

Direction	Name	Description
inout	combiner	Initialized combiner handle
in	soft_bits	Received soft bits
in	len	Number of soft bits

Returns: `k_rx_err_busy` if max combines reached

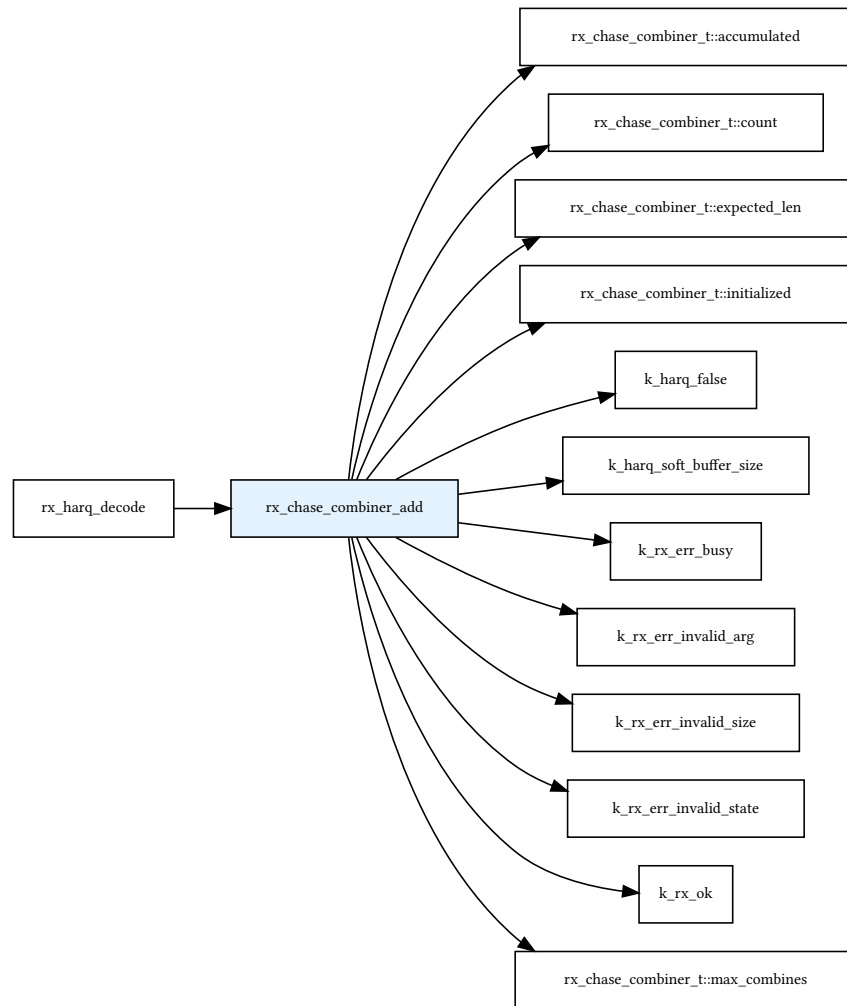


Figure 902: Call graph for rx_chase_combiner_add

Defined in `libs/rx_harq/inc/rx_harq.h:244`

—

rx_chase_combiner_combined

```
rx_err_t rx_chase_combiner_combined(const rx_chase_combiner_t *combiner,
rx_soft_bit_t *output, uint32_t *len)
```

Get combined soft bits for decoding.

Returns accumulated values clamped to SoftBit range $[-127, +127]$.

Parameters:

Direction	Name	Description
in	combiner	Initialized combiner handle
out	output	Output buffer for combined soft bits
out	len	Number of combined soft bits

Returns: k_rx_err_invalid_state if no soft bits have been added

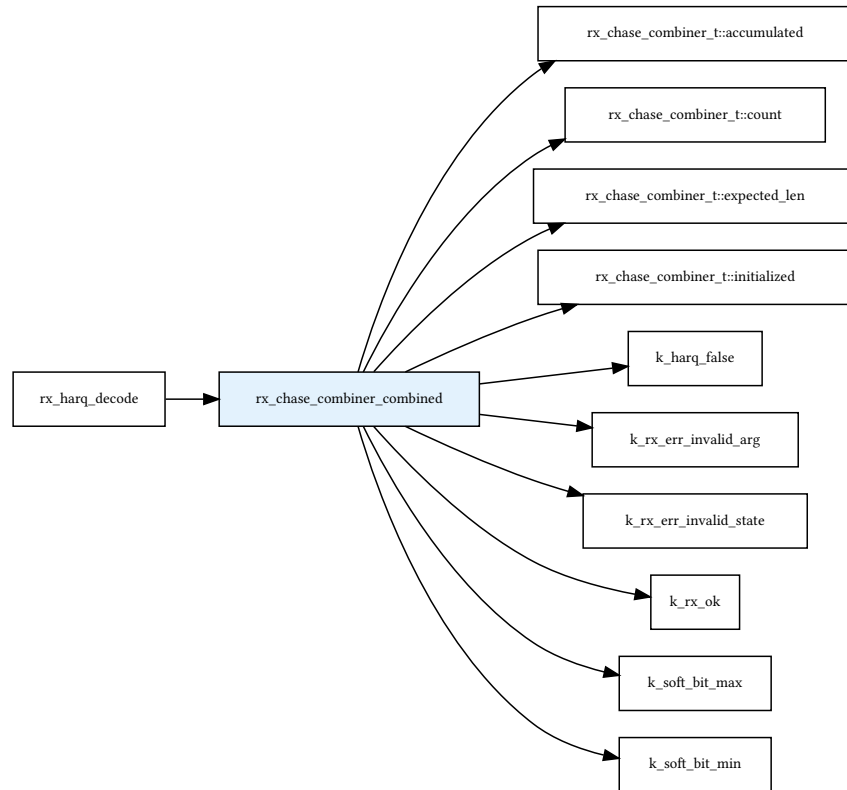


Figure 903: Call graph for `rx_chase_combiner_combined`

Defined in `libs/rx_harq/inc/rx_harq.h:259`

—

rx_chase_combiner_reset

```
rx_err_t rx_chase_combiner_reset(rx_chase_combiner_t *combiner)
```

Reset combiner for new frame.

Clears accumulated buffer and count.

Parameters:

Direction	Name	Description
-----------	------	-------------

inout	combiner	Pointer to combiner handle
-------	----------	----------------------------

Returns: k_rx_err_invalid_state if combiner is not initialized

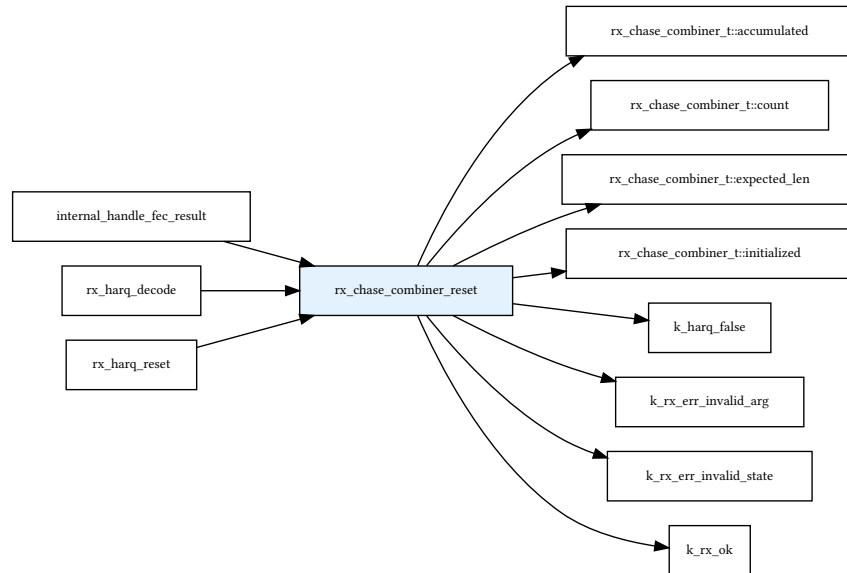


Figure 904: Call graph for `rx_chase_combiner_reset`

Defined in `libs/rx_harq/inc/rx_harq.h:274`

rx_chase_combiner_can_add

```
bool rx_chase_combiner_can_add(const rx_chase_combiner_t *combiner)
```

Check if more transmissions can be combined.

Check if more transmissions can be combined.

Parameters:

Direction	Name	Description
in	combiner	Pointer to combiner handle
in	combiner	Pointer to Chase combiner instance

Returns: true if more transmissions can be added, false otherwise

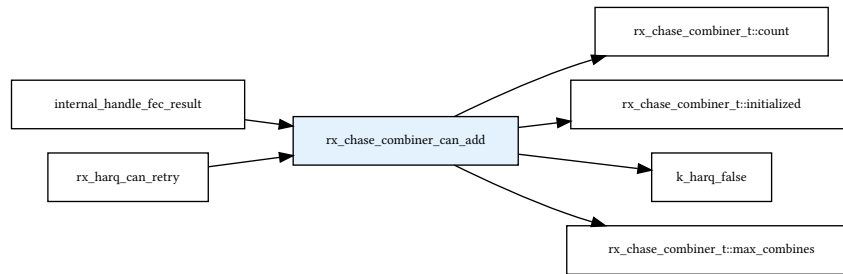


Figure 905: Call graph for rx_chase_combiner_can_add

Defined in `libs/rx_harq/inc/rx_harq.h:282`

rx_chase_combiner_count

```
uint8_t rx_chase_combiner_count(const rx_chase_combiner_t *combiner)
```

Get number of combined transmissions.

Get number of combined transmissions.

Parameters:

Direction	Name	Description
in	combiner	Pointer to combiner handle
in	combiner	Pointer to Chase combiner instance

Returns: Number of combined transmissions (0 if combiner is nullptr)

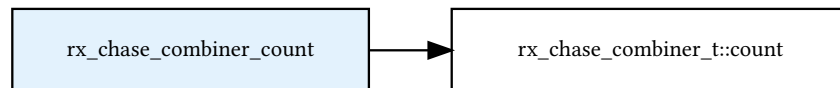


Figure 906: Call graph for rx_chase_combiner_count

Defined in `libs/rx_harq/inc/rx_harq.h:290`

rx_harq_init

```
rx_err_t rx_harq_init(rx_harq_handle_t *harq, const rx_harq_config_t *config)
```

Initialize HARQ handle.

Parameters:

Direction	Name	Description
out	harq	Pointer to HARQ handle
in	config	Configuration (NULL for defaults)

Returns: k_rx_err_invalid_arg if harq is nullptr

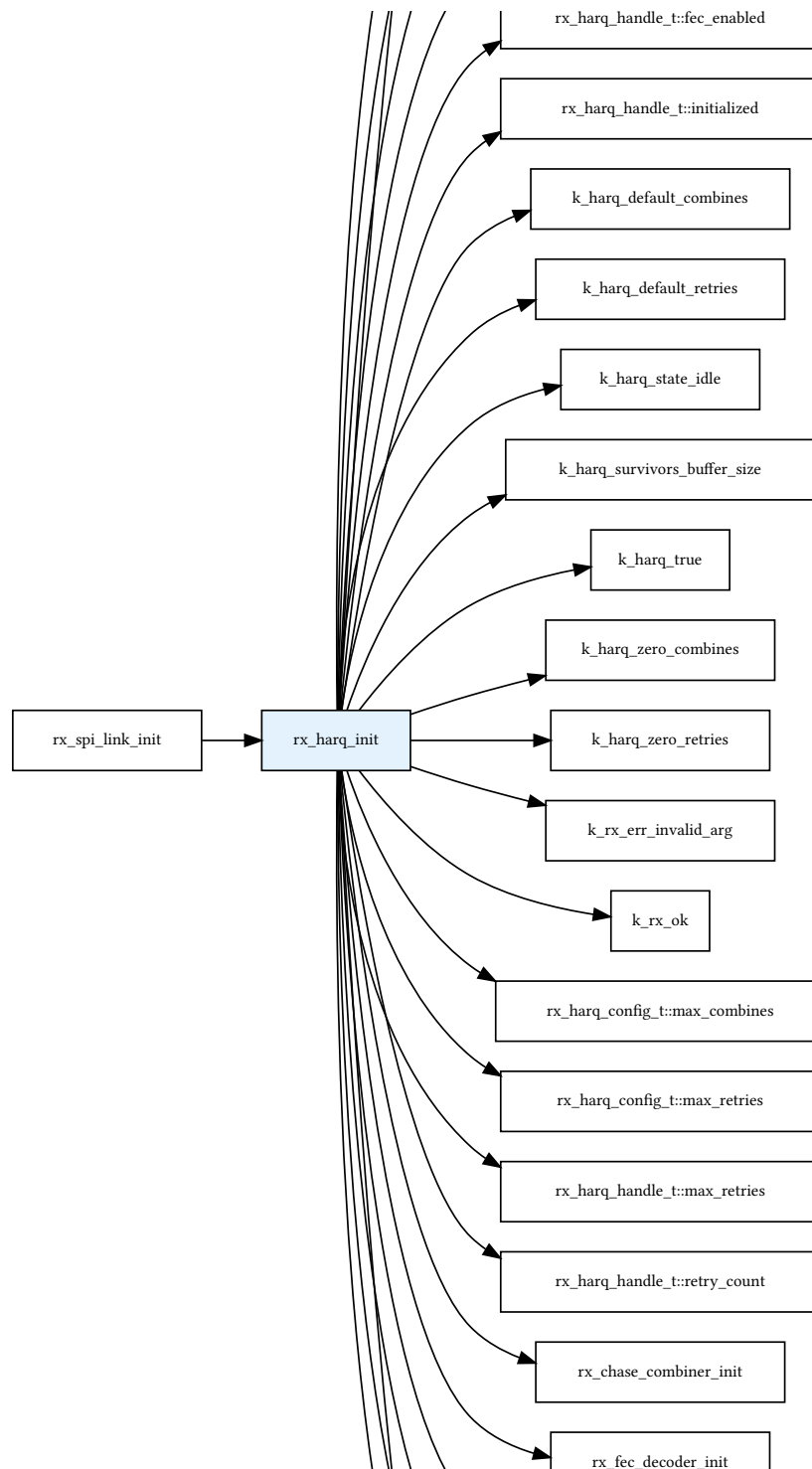


Figure 907: Call graph for rx_harq_init

Defined in `libs/rx_harq/inc/rx_harq.h:345`

—

rx_harq_deinit

```
rx_err_t rx_harq_deinit(rx_harq_handle_t *harq)
```

Deinitialize HARQ handle.

Deinitialize HARQ handle.

Parameters:

Direction	Name	Description
inout	harq	Pointer to HARQ handle
in	harq	Pointer to HARQ handle

Returns: `k_rx_ok` on success, `k_rx_err_invalid_arg` if `harq` is `nullptr`

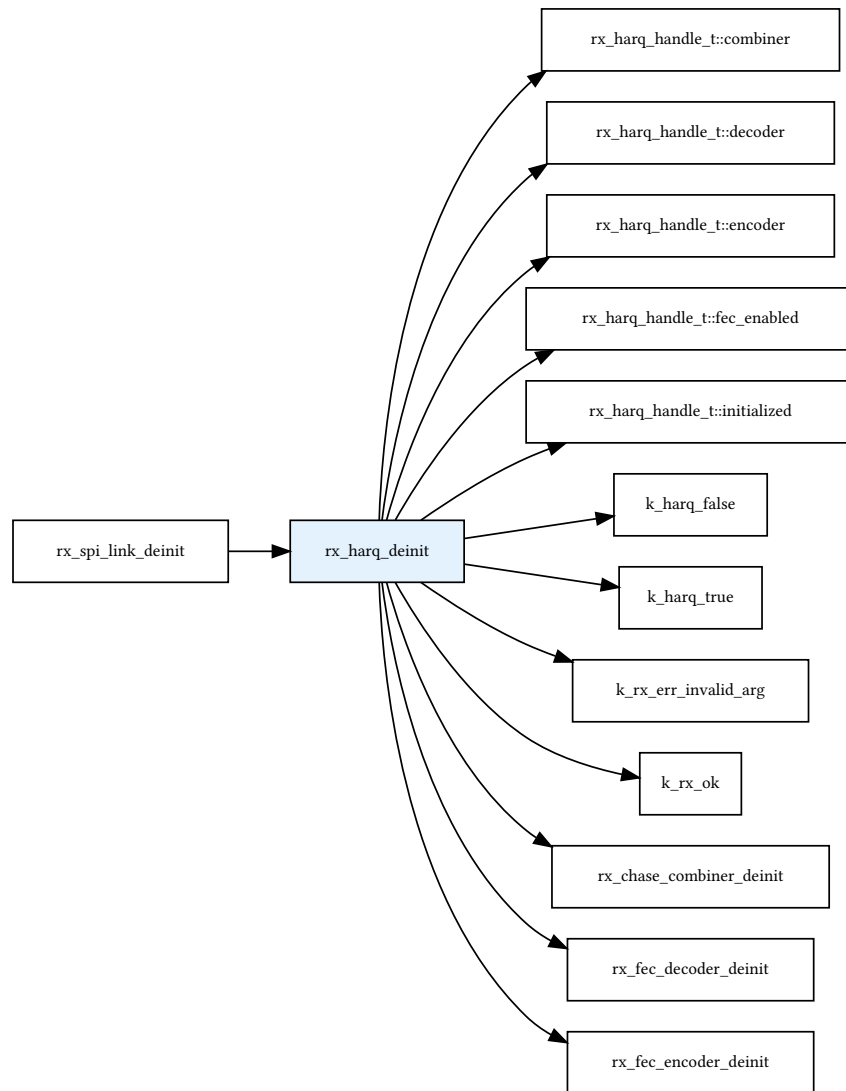


Figure 908: Call graph for rx_harq_deinit

Defined in `libs/rx_harq/inc/rx_harq.h:353`

—

rx_harq_get_state

```
rx_harq_state_t rx_harq_get_state(const rx_harq_handle_t *harq)
```

Get current HARQ state.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle
in	harq	Pointer to HARQ handle

Returns: Current state, or k_harq_state_error if harq is nullptr

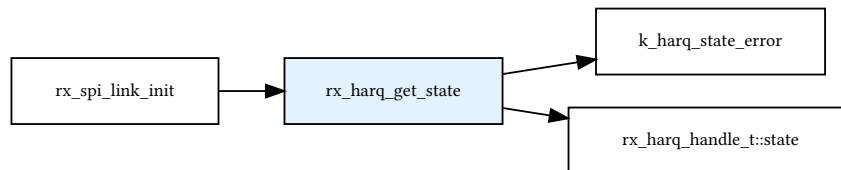


Figure 909: Call graph for rx_harq_get_state

Defined in *libs/rx_harq/inc/rx_harq.h:361*

—

rx_harq_reset

```
rx_err_t rx_harq_reset(rx_harq_handle_t *harq)
```

Reset HARQ for new transaction.

Resets state to idle and clears combiner.

Reset HARQ for new transaction.

Parameters:

Direction	Name	Description
inout	harq	Pointer to HARQ handle
in	harq	Pointer to HARQ handle

Returns: k_rx_err_invalid_state if not initialized

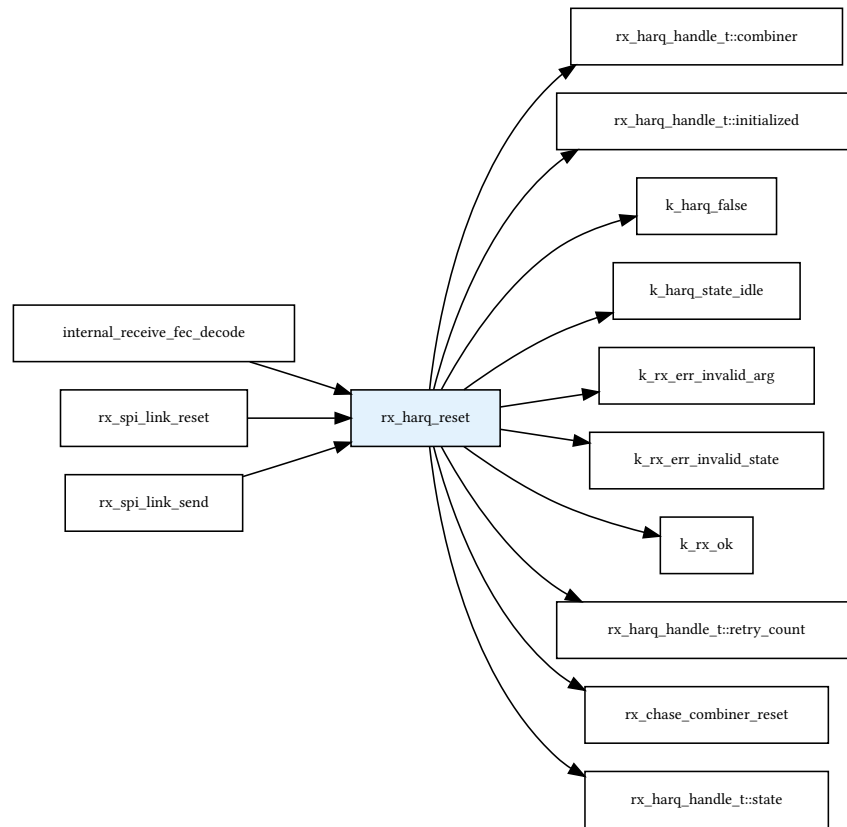


Figure 910: Call graph for rx_harq_reset

Defined in `libs/rx_harq/inc/rx_harq.h:374`

—

rx_harq_encode

```

rx_err_t rx_harq_encode(const rx_harq_handle_t *harq, const uint8_t *payload,
uint32_t payload_len, uint8_t *output, uint32_t output_size, uint32_t
*output_len)

```

Encode payload with FEC (if enabled).

Parameters:

Direction	Name	Description
in	harq	HARQ handle
in	payload	Input payload
in	payload_len	Payload length
out	output	Output buffer (must be large enough for encoded data)

in	output_size	Size of output buffer in bytes
out	output_len	Actual output length

Returns: k_rx_err_invalid_state if not initialized

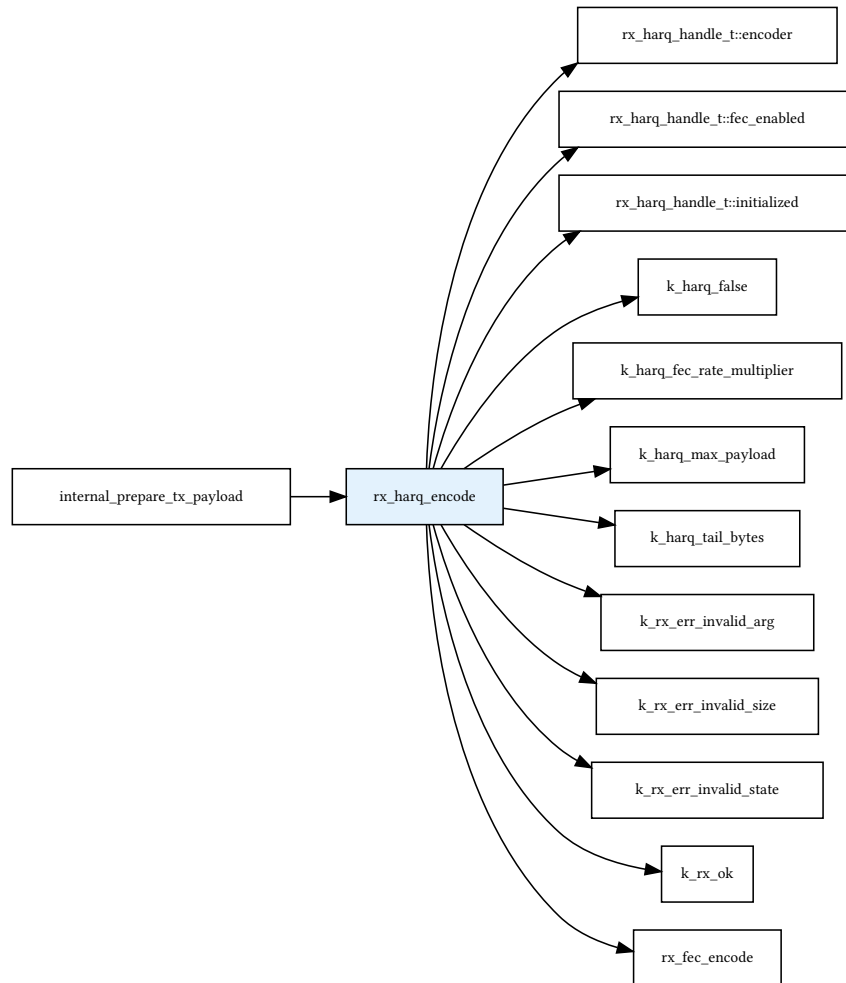


Figure 911: Call graph for rx_harq_encode

Defined in `libs/rx_harq/inc/rx_harq.h:391`

rx_harq_decode

```
rx_err_t rx_harq_decode(rx_harq_handle_t *harq, const rx_harq_decode_params_t
*params, uint8_t *output, uint32_t *output_len)
```

Decode soft bits with combining and FEC.

Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.

Parameters:

Direction	Name	Description
in	harq	HARQ handle
in	params	Decode parameters
out	output	Decoded output buffer
out	output_len	Actual decoded length

Returns: k_rx_err_invalid_state if not initialized



Figure 912: Call graph for rx_harq_decode

Defined in `libs/rx_harq/inc/rx_harq.h:413`

—

rx_harq_get_retry_count

```
uint8_t rx_harq_get_retry_count(const rx_harq_handle_t *harq)
```

Get current retry count.

Get current retry count.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle
in	harq	Pointer to HARQ handle

Returns: Number of retries (0 if harq is nullptr)

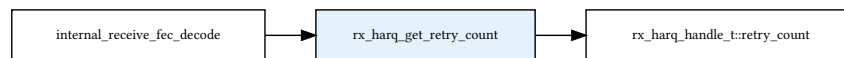


Figure 913: Call graph for `rx_harq_get_retry_count`

Defined in `libs/rx_harq/inc/rx_harq.h:424`

—

rx_harq_can_retry

```
bool rx_harq_can_retry(const rx_harq_handle_t *harq)
```

Check if more retries are available.

Check if more retries are available.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle
in	harq	Pointer to HARQ handle

Returns: true if retry is possible, false otherwise

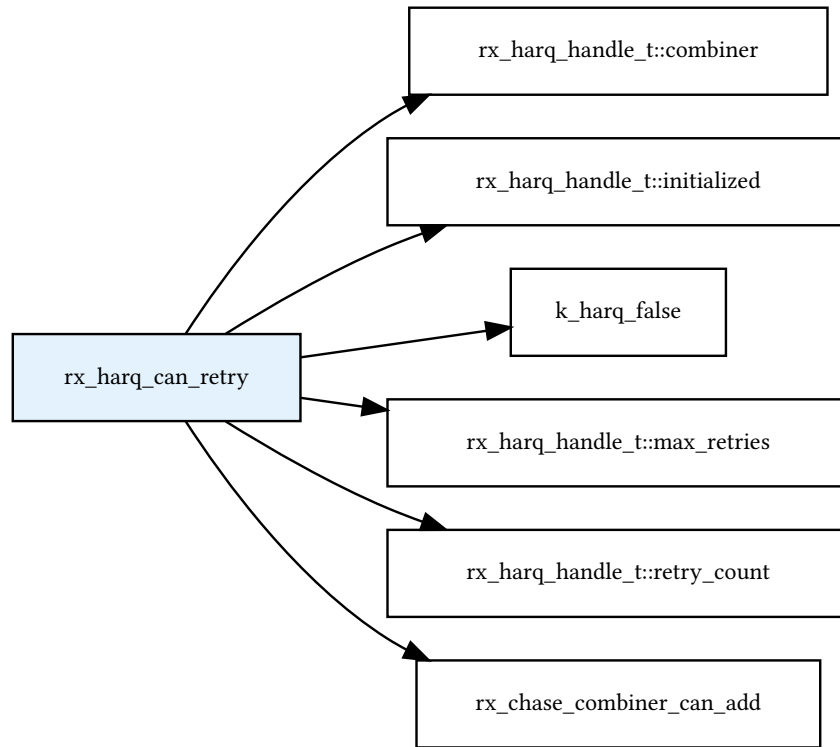


Figure 914: Call graph for rx_harq_can_retry

Defined in `libs/rx_harq/inc/rx_harq.h:432`

—

rx_harq.c

Hybrid Automatic Repeat Request (HARQ) Protocol Implementation.

Implements Chase Combining HARQ Type I combining FEC with soft bit accumulation across retransmissions. This is a C port of `star-gateway/internal/harq/` and `fec/combiner.go` for bit-exact compatibility between RX72N and RPi5.

Includes:

- `"rx_harq.h"`
- `<string.h>`
- `"rx_bit_constants.h"`

harq_fec_constants_t

HARQ FEC size constants.

Value	Name	Description
= 2	<code>k_harq_fec_rate_multiplier</code>	Rate 1/2: encoded length multiplier
= 2	<code>k_harq_tail_bytes</code>	Tail overhead in bytes

harq_cfg_sentinel_t

HARQ config validation sentinels zero values for “not configured, use default”.

Used to detect unconfigured (zero) values for max_combines and max_retries fields in HARQ configuration, to fall back to project defaults.

k_harq_zero_combines and k_harq_zero_retries equal zero, signifying “use default config” these sentinels must never be changed to non-zero values.

Value	Name	Description
= 0	k_harq_zero_combines	Sentinel: no max_combines configured -> fall back to default
= 0	k_harq_zero_retries	Sentinel: no max_retries configured -> fall back to default

See also: rx_harq_config_t HARQ configuration structure containing max_combines/max_retries, rx_harq_init() Applies these sentinels during initialization

Example:

```
//Checkingsentinelswhenvalidatingconfigfields:
const rx_harq_config_t config = {.max_combines=0, .max_retries=0};
//max_combines==k_harq_zero_combines->usek_harq_default_combines
//max_retries==k_harq_zero_retries->usek_harq_default_retries
uint8_t tcombines = (config.max_combines > k_harq_zero_combines)
? config.max_combines : k_harq_default_combines;
uint8_t tretries = (config.max_retries > k_harq_zero_retries)
? config.max_retries : k_harq_default_retries;
```

Since Version 1.0.0

harq_bool_t

Value	Name	Description
= 0U	k_harq_false	
= 1U	k_harq_true	

bit_constants_t

Bit position constants for soft-to-hard conversion.

Value	Name	Description
= k_rx_bits_per_byte - 1U	k_msb_position	Most significant bit position
= k_rx_bits_per_byte - 1U	k_rounding_adjustment	Ceiling division: (n + 7) / 8
= 0	k_soft_bit_zero_thresh	Threshold for soft-to-hard: >= 0 is bit 1

rx_chase_combiner_init

```
rx_err_t rx_chase_combiner_init(rx_chase_combiner_t *combiner, const uint8_t
max_combines)
```

Initialize Chase Combiner.

Parameters:

Direction	Name	Description
out	combiner	Pointer to combiner handle
in	max_combines	Maximum number of combining attempts (0 = default)

Returns: k_rx_ok on success, k_rx_err_invalid_arg if combiner is nullptr

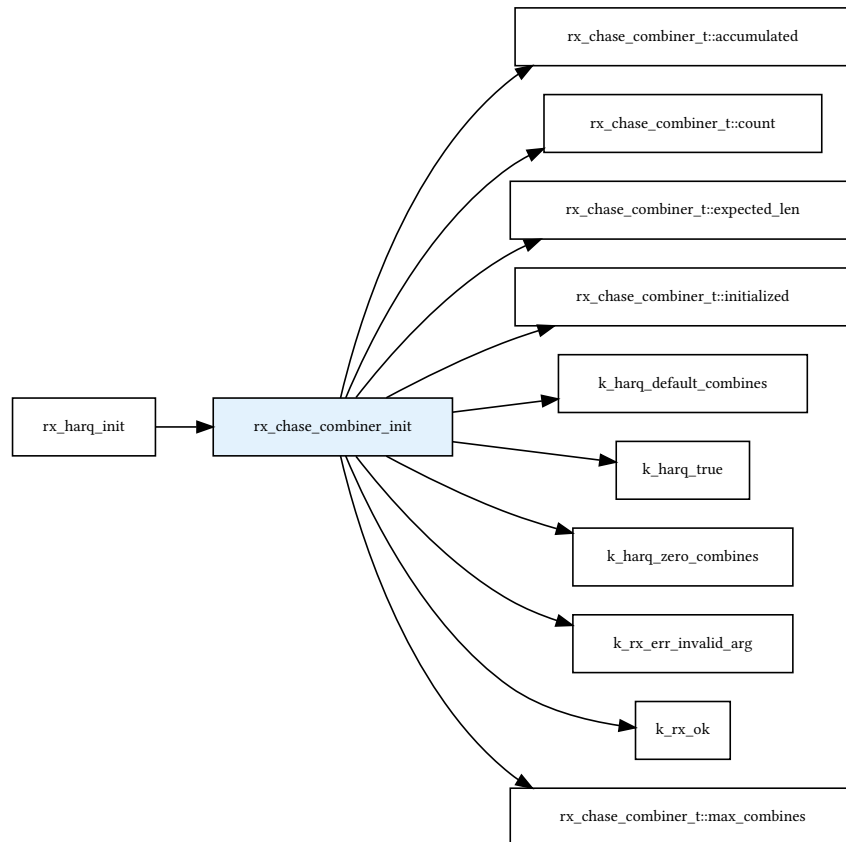


Figure 915: Call graph for `rx_chase_combiner_init`

Defined in `libs/rx_harq/src/rx_harq.c:156`

rx_chase_combiner_deinit

```
rx_err_t rx_chase_combiner_deinit(rx_chase_combiner_t *combiner)
```

Deinitialize Chase Combiner.

Parameters:

Direction	Name	Description
inout	combiner	Pointer to combiner handle

Returns: k_rx_ok on success

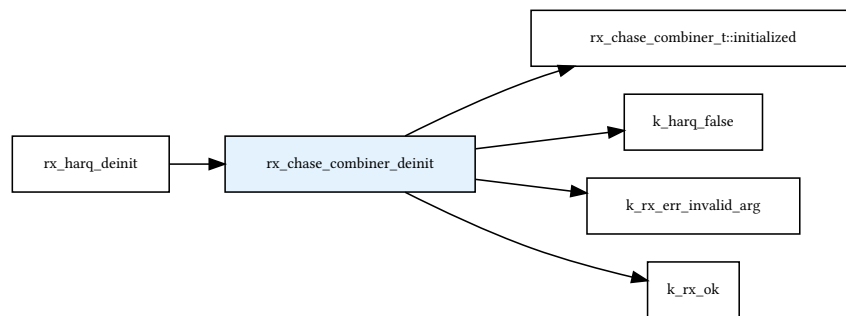


Figure 916: Call graph for rx_chase_combiner_deinit

Defined in `libs/rx_harq/src/rx_harq.c:177`

—

rx_chase_combiner_add

```
rx_err_t rx_chase_combiner_add(rx_chase_combiner_t *combiner, const rx_soft_bit_t *soft_bits, uint32_t len)
```

Add soft bits from a transmission attempt.

Accumulates soft bits element-wise. First call sets expected length.

Parameters:

Direction	Name	Description
inout	combiner	Initialized combiner handle
in	soft_bits	Received soft bits
in	len	Number of soft bits

Returns: k_rx_err_busy if max combines reached

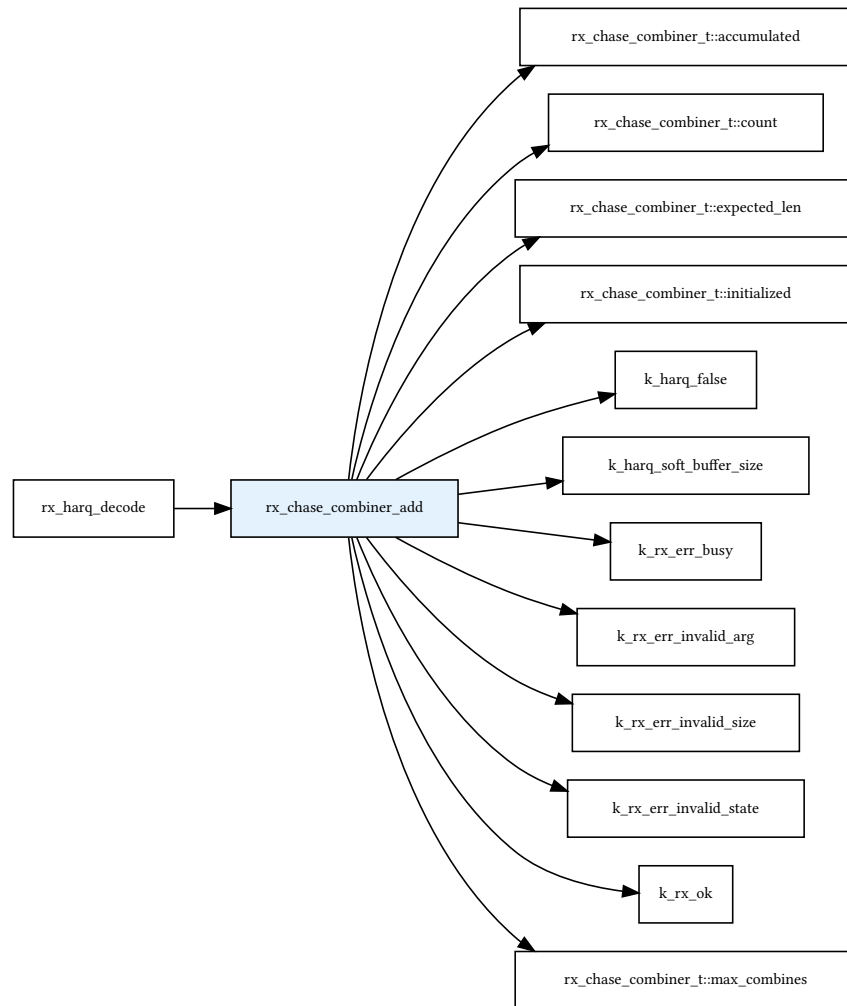


Figure 917: Call graph for rx_chase_combiner_add

Defined in `libs/rx_harq/src/rx_harq.c:188`

—

rx_chase_combiner_combined

```
rx_err_t rx_chase_combiner_combined(const rx_chase_combiner_t *combiner,
rx_soft_bit_t *output, uint32_t *len)
```

Get combined soft bits for decoding.

Returns accumulated values clamped to SoftBit range $[-127, +127]$.

Parameters:

Direction	Name	Description
in	combiner	Initialized combiner handle
out	output	Output buffer for combined soft bits
out	len	Number of combined soft bits

Returns: k_rx_err_invalid_state if no soft bits have been added

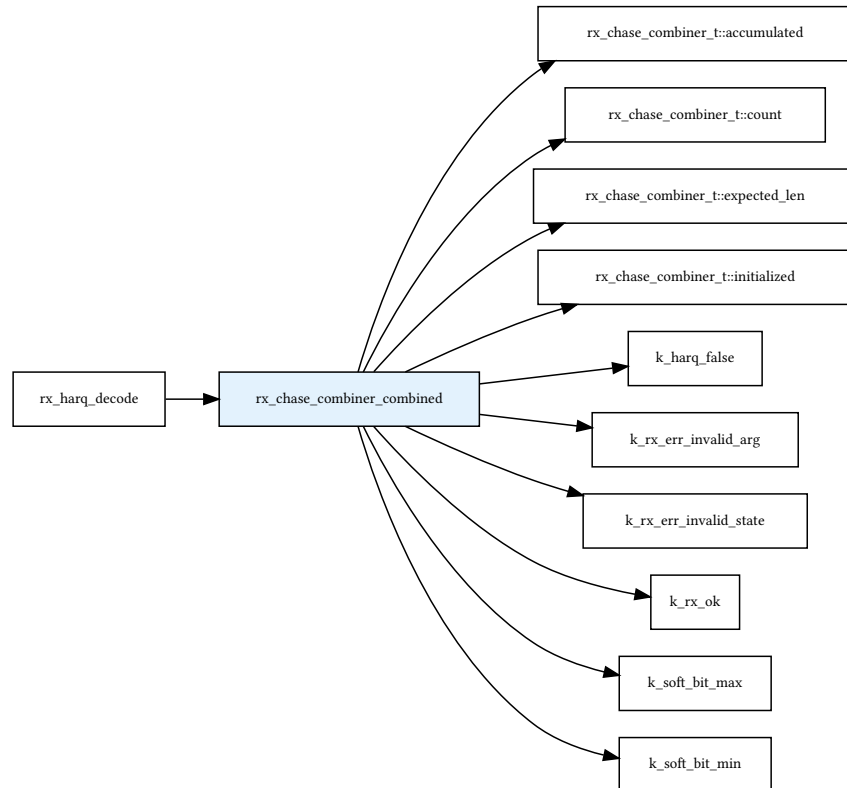


Figure 918: Call graph for `rx_chase_combiner_combined`

Defined in `libs/rx_harq/src/rx_harq.c:235`

—

rx_chase_combiner_reset

```
rx_err_t rx_chase_combiner_reset(rx_chase_combiner_t *combiner)
```

Reset combiner for new frame.

Clears accumulated buffer and count.

Parameters:

Direction	Name	Description
-----------	------	-------------

inout	combiner	Pointer to combiner handle
-------	----------	----------------------------

Returns: k_rx_err_invalid_state if combiner is not initialized

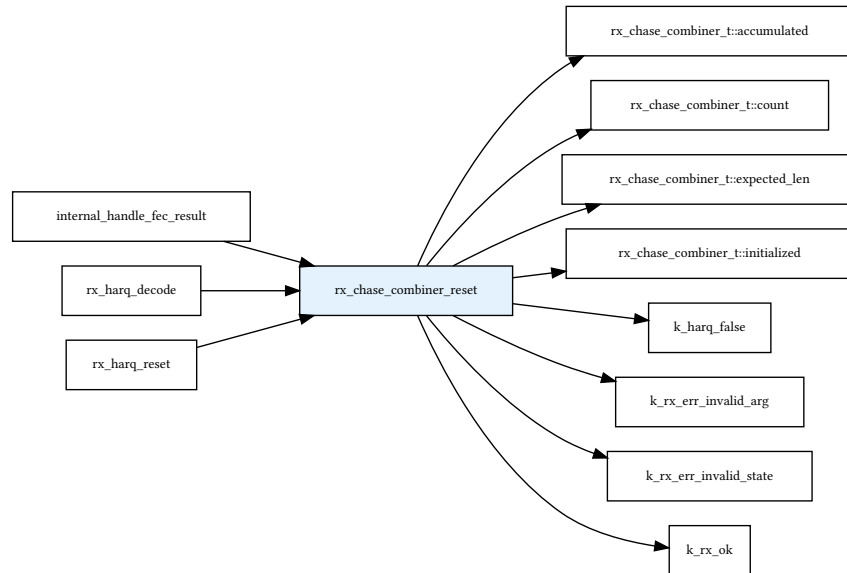


Figure 919: Call graph for `rx_chase_combiner_reset`

Defined in `libs/rx_harq/src/rx_harq.c:276`

`rx_chase_combiner_can_add`

```
bool rx_chase_combiner_can_add(const rx_chase_combiner_t *combiner)
```

Check if combiner can accept more transmissions.

Check if more transmissions can be combined.

Parameters:

Direction	Name	Description
in	combiner	Pointer to Chase combiner instance

Returns: true if more transmissions can be added, false otherwise

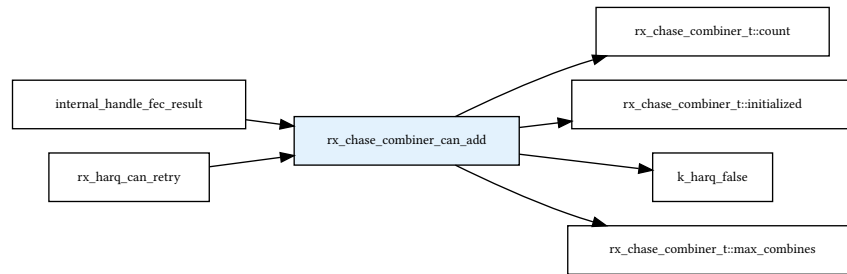


Figure 920: Call graph for rx_chase_combiner_can_add

Defined in `libs/rx_harq/src/rx_harq.c:301`

rx_chase_combiner_count

```
uint8_t rx_chase_combiner_count(const rx_chase_combiner_t *combiner)
```

Get number of transmissions combined so far.

Get number of combined transmissions.

Parameters:

Direction	Name	Description
in	combiner	Pointer to Chase combiner instance

Returns: Number of combined transmissions (0 if combiner is nullptr)

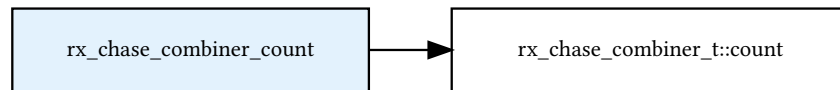


Figure 921: Call graph for rx_chase_combiner_count

Defined in `libs/rx_harq/src/rx_harq.c:316`

rx_harq_init

```
rx_err_t rx_harq_init(rx_harq_handle_t *harq, const rx_harq_config_t *config)
```

Initialize HARQ handle.

Parameters:

Direction	Name	Description
out	harq	Pointer to HARQ handle
in	config	Configuration (NULL for defaults)

Returns: k_rx_err_invalid_arg if harq is nullptr

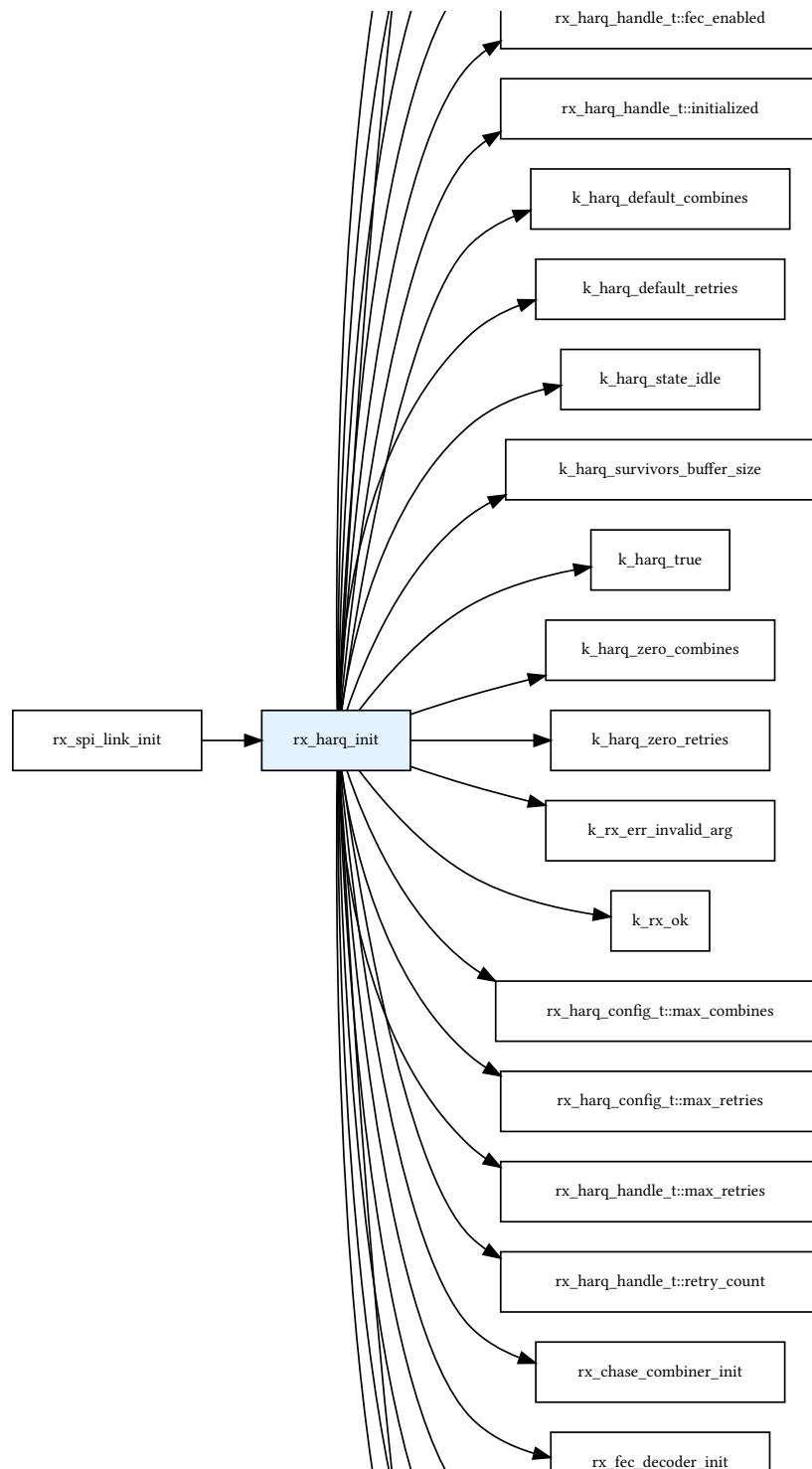


Figure 922: Call graph for rx_harq_init

Defined in `libs/rx_harq/src/rx_harq.c:329`

—

rx_harq_deinit

```
rx_err_t rx_harq_deinit(rx_harq_handle_t *harq)
```

Deinitialize HARQ handle and release resources.

Deinitialize HARQ handle.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle

Returns: k_rx_ok on success, k_rx_err_invalid_arg if harq is nullptr

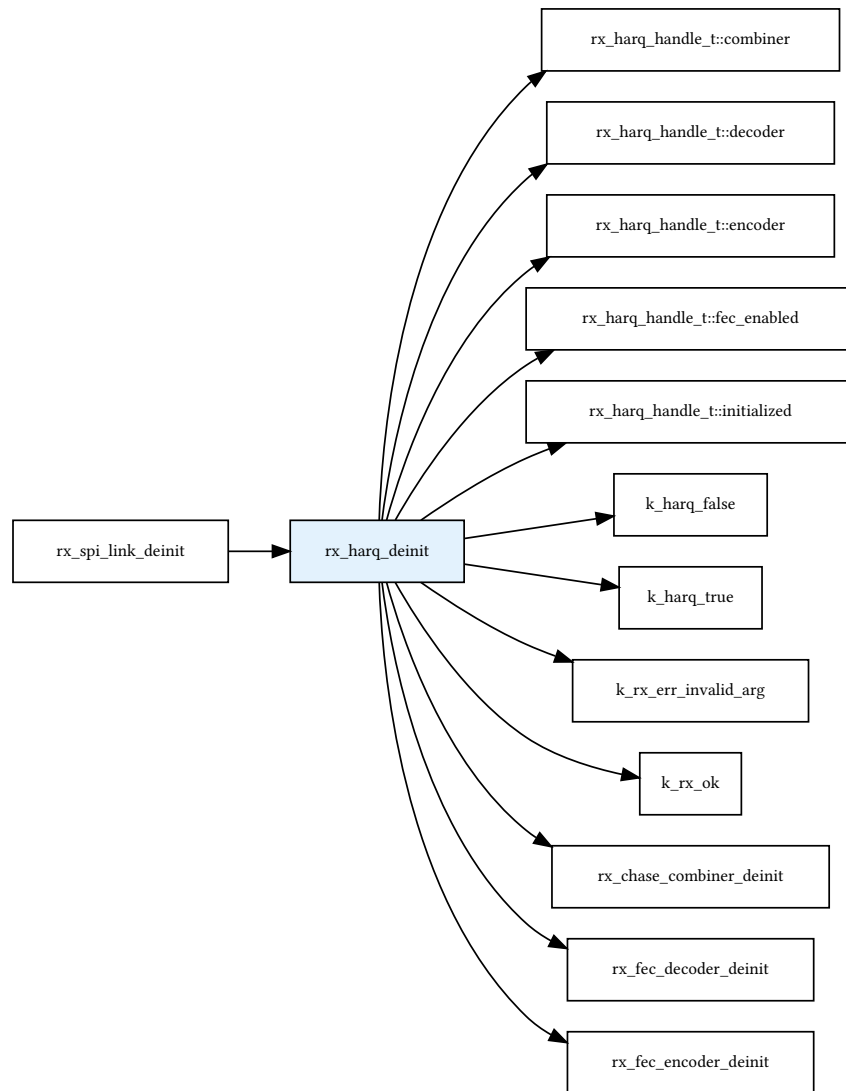


Figure 923: Call graph for rx_harq_deinit

Defined in `libs/rx_harq/src/rx_harq.c:389`

—

rx_harq_get_state

```
rx_harq_state_t rx_harq_get_state(const rx_harq_handle_t *harq)
```

Get current HARQ state.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle

Returns: Current state, or k_harq_state_error if harq is nullptr

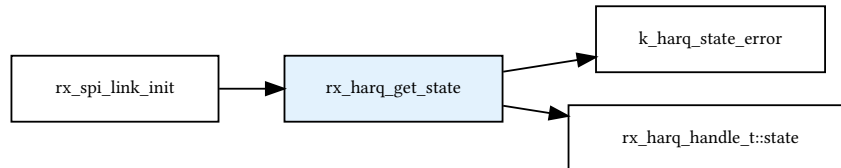


Figure 924: Call graph for rx_harq_get_state

Defined in `libs/rx_harq/src/rx_harq.c:424`

—

rx_harq_reset

```
rx_err_t rx_harq_reset(rx_harq_handle_t *harq)
```

Reset HARQ state for new transmission.

Reset HARQ for new transaction.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle

Returns: k_rx_err_invalid_state if not initialized

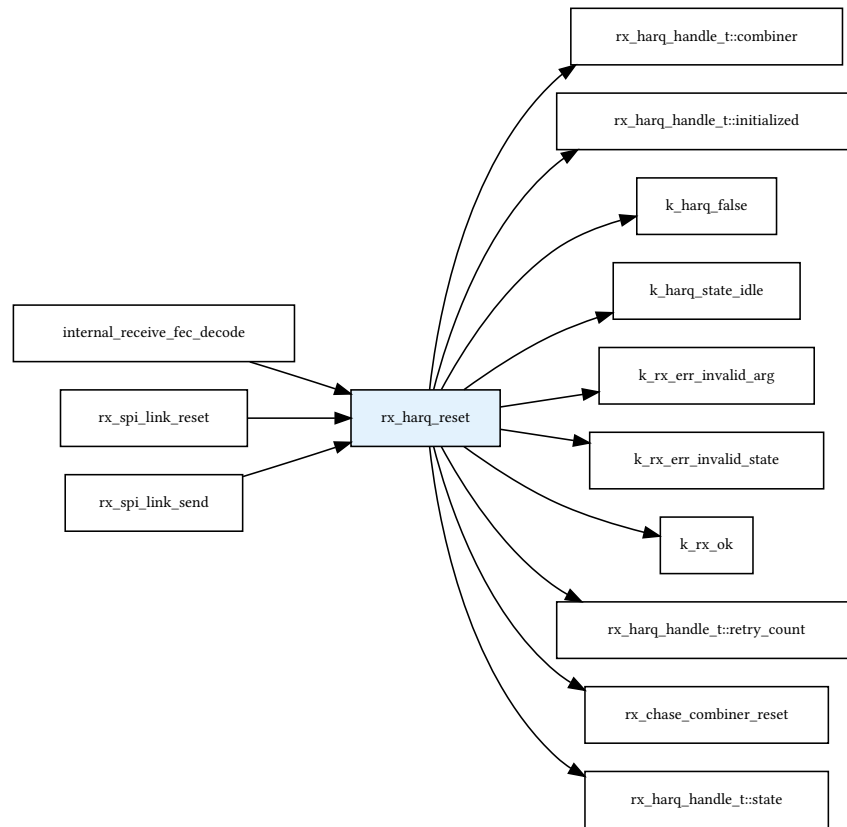


Figure 925: Call graph for rx_harq_reset

Defined in `libs/rx_harq/src/rx_harq.c:441`

—

rx_harq_encode

```

rx_err_t rx_harq_encode(const rx_harq_handle_t *harq, const uint8_t *payload,
const uint32_t payload_len, uint8_t *output, const uint32_t output_size, uint32_t
*output_len)

```

Encode payload with FEC (if enabled).

Parameters:

Direction	Name	Description
in	harq	HARQ handle
in	payload	Input payload
in	payload_len	Payload length
out	output	Output buffer (must be large enough for encoded data)

in	output_size	Size of output buffer in bytes
out	output_len	Actual output length

Returns: k_rx_err_invalid_state if not initialized

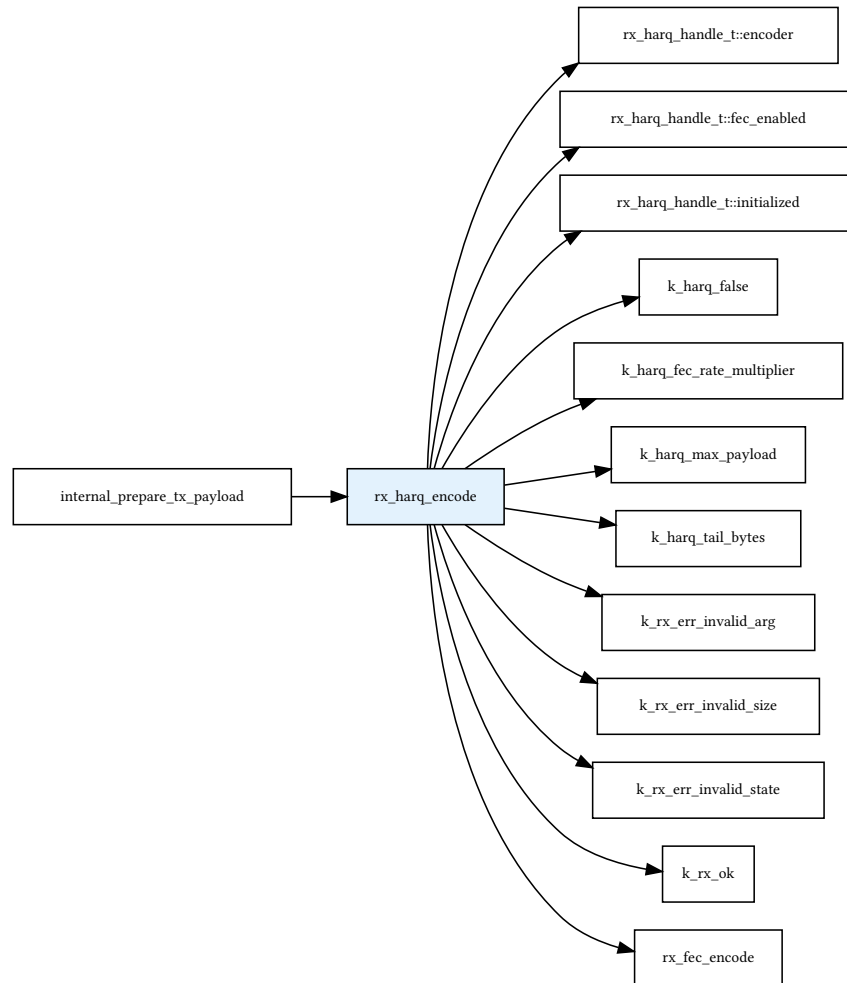


Figure 926: Call graph for rx_harq_encode

Defined in `libs/rx_harq/src/rx_harq.c:462`

internal_soft_to_hard

```

static rx_err_t internal_soft_to_hard(const rx_soft_bit_t *soft_bits, const
uint32_t soft_bit_count, const uint32_t max_output_bytes, uint8_t *output,
uint32_t *output_len)

```

Convert combined soft bits to hard bits (non-FEC path).

Parameters:

Direction	Name	Description
in	soft_bits	Combined soft bits
in	soft_bit_count	Number of soft bits
in	max_output_bytes	Maximum output length (bytes, must be > 0)
out	output	Output hard bits (packed bytes)
out	output_len	Actual output length

Returns: k_rx_err_invalid_arg if any pointer is nullptr

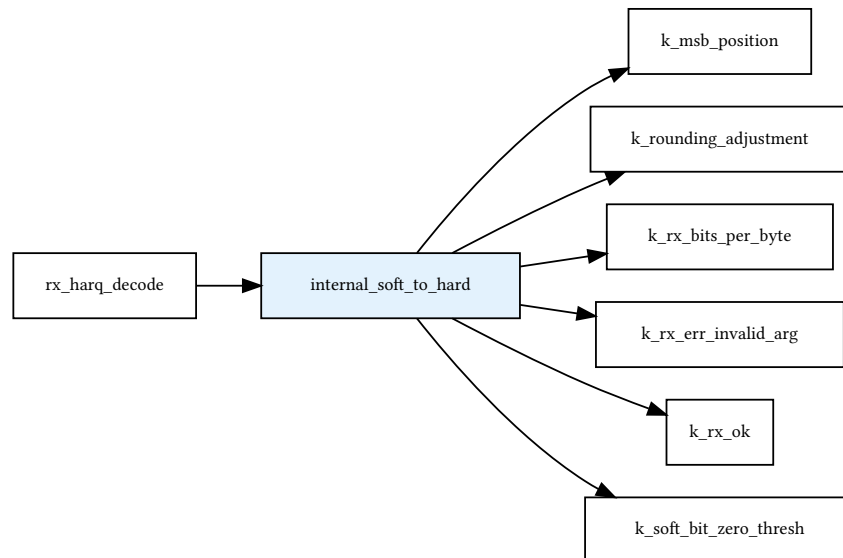


Figure 927: Call graph for internal_soft_to_hard

Defined in `libs/rx_harq/src/rx_harq.c:532`

internal_handle_fec_result

```
static rx_err_t internal_handle_fec_result(rx_harq_handle_t *harq, const rx_err_t result)
```

Handle FEC decode result and update HARQ state.

Parameters:

Direction	Name	Description
inout	harq	HARQ handle (must not be NULL)
in	result	Decode result from FEC decoder

Returns: result error code if max retries reached

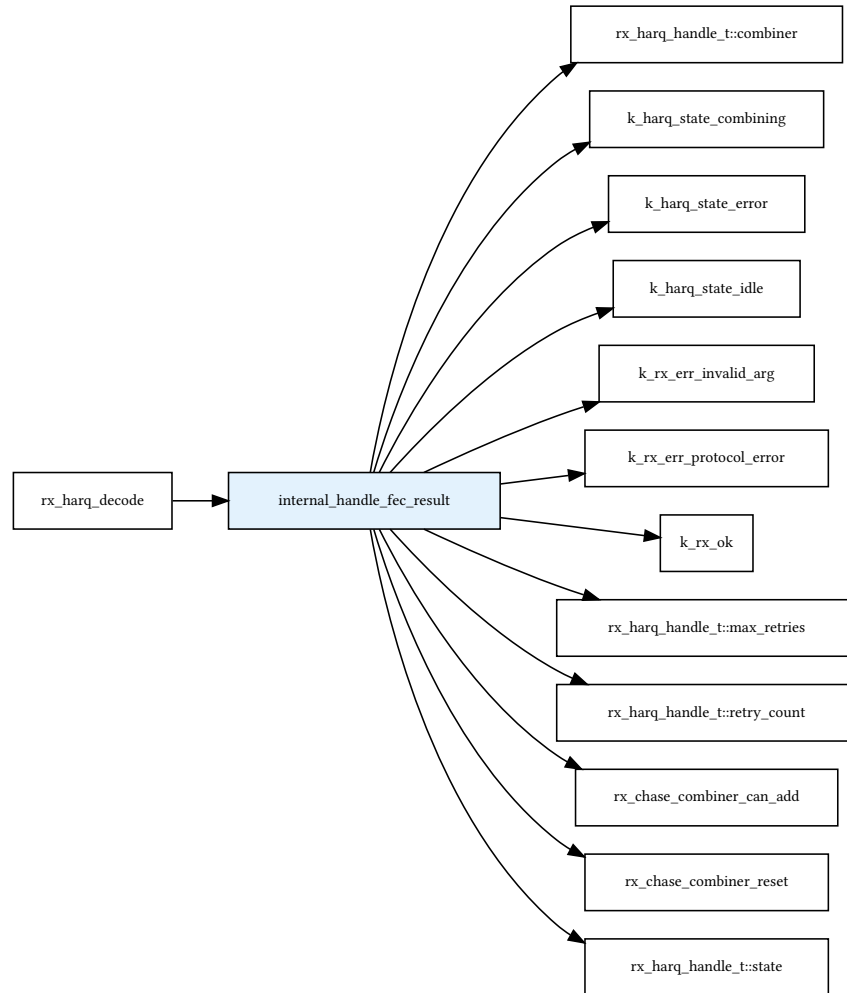


Figure 928: Call graph for `internal_handle_fec_result`

Defined in `libs/rx_harq/src/rx_harq.c:581`

rx_harq_decode

```
rx_err_t rx_harq_decode(rx_harq_handle_t *harq, const rx_harq_decode_params_t
*params, uint8_t *output, uint32_t *output_len)
```

Decode soft bits with combining and FEC.

Adds soft bits to combiner, attempts decode. On failure, keeps accumulating.

Parameters:

Direction	Name	Description
in	harq	HARQ handle
in	params	Decode parameters
out	output	Decoded output buffer
out	output_len	Actual decoded length

Returns: `k_rx_err_invalid_state` if not initialized



Figure 929: Call graph for rx_harq_decode

Defined in `libs/rx_harq/src/rx_harq.c:614`

—

rx_harq_get_retry_count

```
uint8_t rx_harq_get_retry_count(const rx_harq_handle_t *harq)
```

Get number of retries attempted.

Get current retry count.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle

Returns: Number of retries (0 if harq is nullptr)

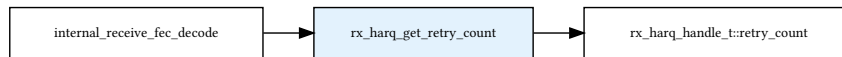


Figure 930: Call graph for `rx_harq_get_retry_count`

Defined in `libs/rx_harq/src/rx_harq.c:680`

—

rx_harq_can_retry

```
bool rx_harq_can_retry(const rx_harq_handle_t *harq)
```

Check if more retries are allowed.

Check if more retries are available.

Parameters:

Direction	Name	Description
in	harq	Pointer to HARQ handle

Returns: true if retry is possible, false otherwise

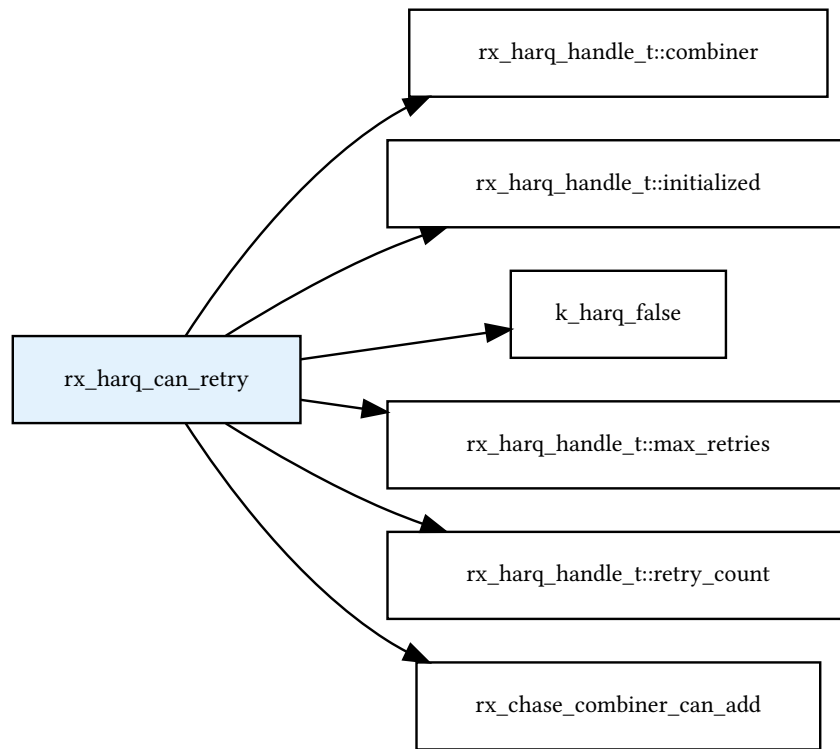


Figure 931: Call graph for rx_harq_can_retry

Defined in `libs/rx_harq/src/rx_harq.c:695`

—

rx_hcsr04.h

HC-SR04 Ultrasonic Distance Sensor Driver for RX72N.

Production-ready driver for HC-SR04 ultrasonic distance sensors with blocking and asynchronous measurement modes, temperature compensation, and comprehensive error handling. Designed for obstacle detection and collision avoidance in autonomous robotics applications.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_port_constants.h"`

rx_hcsr04_timing_t

HC-SR04 timing constants based on ultrasonic physics.

Timing parameters derived from HC-SR04 datasheet and ultrasonic propagation. All values calibrated for speed of sound at 20degC (343 m/s).

Physics Background:

- Speed of sound at 20degC: 343 m/s = 34,300 cm/s = 0.0343 cm/us

- Roundtrip distance: sound travels to object and back
- Time per cm (roundtrip): $1 / 0.0343 = 29.15 \text{ us/cm}$ (one-way)
- Time per cm (total): $29.15 * 2 = 58.3 \text{ us/cm}$ 58 us/cm

Distance Calculation:

Trigger Pulse Timing:

Echo Pulse Timing:

- Minimum (2cm): $116 \text{ us} = 2 \text{ cm} * 58 \text{ us/cm}$
- Maximum (400cm): $23,200 \text{ us} = 400 \text{ cm} * 58 \text{ us/cm}$
- Timeout: 30,000 us (400cm + 30% margin)

Measurement Rate Limit:

- HC-SR04 needs 60ms recovery time between measurements
- Faster polling causes echo interference/false readings
- Maximum rate: 1000ms / 60ms 16 Hz

Value	Name	Description
= 2	k_hcsr04_trigger_settle_us	Pre-trigger settle time (ensure clean LOW)
= 10	k_hcsr04_trigger_pulse_us	Trigger pulse width (10us min per datasheet)
= 30000	k_hcsr04_echo_timeout_us	Echo timeout (30ms = 400cm + margin)
= 116	k_hcsr04_min_echo_us	Minimum valid echo (2cm * 58 us/cm)
= 23200	k_hcsr04_max_echo_us	Maximum valid echo (400cm * 58 us/cm)
= 58	k_hcsr04_us_per_cm_roundtrip	Microseconds per cm roundtrip at 20degC
= 60	k_hcsr04_measurement_gap_ms	Minimum gap between measurements (per datasheet)

Note: Temperature affects speed of sound (0.17%/degC) - use temp compensation] **See also:**
 rx_hcsr04_set_temperature() For temperature compensation,
 rx_hcsr04_get_speed_of_sound() Speed of sound calculation

Example:

```
distance_cm=echo_time_us/k_hcsr04_us_per_cm_roundtrip
distance_cm=echo_time_us/58
```

Example:

```
Time(us)Signal
0TRIG-+
2TRIG| (settletime)
2TRIG|
12TRIG-+ (10uspulse)
12+Waitforecho...
```

—

rx_hcsr04_range_t

HC-SR04 measurement range limits.

Valid distance range for HC-SR04 sensor per datasheet specifications. Measurements outside this range are rejected as k_rx_err_out_of_range.

Range Characteristics:

- Minimum (2cm): Blind zone - echo overlaps trigger pulse
- Maximum (400cm): Signal too weak for reliable detection
- Optimal: 10cm - 200cm (best accuracy and reliability)

Factors Affecting Maximum Range:

- Object size: Larger objects detectable at greater distance
- Object material: Hard surfaces reflect better than soft
- Surface angle: Perpendicular surfaces give strongest echo
- Ambient noise: Ultrasonic interference reduces range

Dead Zone (<2cm):

- Sensor cannot distinguish echo from trigger pulse
- Use IR sensor for closer range

Value	Name	Description
= 2	k_hcsr04_min_distance_cm	Minimum measurable distance (blind zone limit)
= 400	k_hcsr04_max_distance_cm	Maximum measurable distance (detection limit)

Note: Practical max range often 300cm due to real-world conditions] **See also:** rx_hcsr04_measure() Returns k_rx_err_out_of_range if outside limits

—

rx_hcsr04_echo_mode_t

Echo pulse measurement mode selection.

Determines how the HC-SR04 echo pulse is measured:

- Polling: Software reads GPIO pin state in a loop (simple, portable)
- IRQ: Hardware captures edges via ICU interrupt (precise, low CPU overhead)

Mode Comparison:

Recommended Usage:

- Use polling for: debugging, single sensor, simple applications
- Use IRQ for: production code, multiple sensors, CPU efficiency

value is one of k_hcsr04_echo_polling or k_hcsr04_echo_irq

Value	Name	Description
= 0	k_hcsr04_echo_polling	Software polling mode (default).
= 1	k_hcsr04_echo_irq	Hardware IRQ edge capture mode.

See also: rx_hcsr04_config_t Configuration structure using this enum, rx_hcsr04_icu_configure() ICU setup required for IRQ mode

Example:

```
//SelectIRQmodeandconfigureICU
rx_hcsr04_config_tcfg={
    .trigger_pin=k_rx_pf_5,
    .echo_pin=k_rx_p0_3,
    .timeout_us=k_hcsr04_echo_timeout_us,
```

```
.echo_mode=k_hcsr04_echo_irq,//Hardwareedgecapture
.echo_irq=k_hcsr04_irq_11,//P03->IRQ11
.irq_priority=k_hcsr04_irq_priority_default,
};
rx_hcsr04_tsensor;
rx_err_t err=rx_hcsr04_init(&sensor,&cfg);
//ICUisconfiguredautomaticallyinsiderx_hcsr04_init()forIRQmode
```

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_irq_t

IRQ number selection for HC-SR04 echo pin (IRQ mode).

Type-safe enumeration of the valid IRQ numbers for HC-SR04 echo pulse measurement. Use in the `echo_irq` field of `rx_hcsr04_config_t` when `echo_mode == k_hcsr04_echo_irq`. The IRQ number must match the physical echo pin: P00 -> IRQ8, P01 -> IRQ9, P02 -> IRQ10, P03 -> IRQ11.

Only values `k_hcsr04_irq_8` through `k_hcsr04_irq_11` are currently used in the STAR project (one per sonar sensor)

Value	Name	Description
= 0	<code>k_hcsr04_irq_none</code>	Sentinel: no IRQ assigned (polling mode); zero so zero-init is safe
= 8	<code>k_hcsr04_irq_8</code>	IRQ8 - maps to P00 (Sonar 3 Back-Right)
= 9	<code>k_hcsr04_irq_9</code>	IRQ9 - maps to P01 (Sonar 2 Back-Left)
= 10	<code>k_hcsr04_irq_10</code>	IRQ10 - maps to P02 (Sonar 1 Front-Right)
= 11	<code>k_hcsr04_irq_11</code>	IRQ11 - maps to P03 (Sonar 0 Front-Left)
= 12	<code>k_hcsr04_irq_12</code>	IRQ12 - reserved; not accepted by <code>rx_hcsr04_init()</code>
= 13	<code>k_hcsr04_irq_13</code>	IRQ13 - reserved; not accepted by <code>rx_hcsr04_init()</code>
= 14	<code>k_hcsr04_irq_14</code>	IRQ14 - reserved; not accepted by <code>rx_hcsr04_init()</code>
= 15	<code>k_hcsr04_irq_15</code>	IRQ15 - reserved; not accepted by <code>rx_hcsr04_init()</code>

See also: `rx_hcsr04_config_t` Configuration structure using this type, RX72N Manual Chapter 15 - ICU, Table listing IRQ pin assignments

Example:

```
rx_hcsr04_config_t config={
.echo_pin=k_rx_p0_3,
.echo_mode=k_hcsr04_echo_irq,
.echo_irq=k_hcsr04_irq_11,//P03->IRQ11
};
```

Since Version 1.2.0 (Issue 296 - IRQ mode added)

—

rx_hcsr04_irq_priority_t

Named constants for IRQ interrupt priority configuration.

Provides named values for the `irq_priority` field in `rx_hcsr04_config_t`. Use

`k_hcsr04_irq_priority_unset (0)` to let `rx_hcsr04_init()` select the default priority, or provide an explicit value 1-15.

`k_hcsr04_irq_priority_unset (0)` means “not set” (`rx_hcsr04_init()` selects the default); valid explicit priorities are 1-15; any other value is rejected by `rx_hcsr04_icu_configure()`

Value	Name	Description
= 0	<code>k_hcsr04_irq_priority_unset</code>	Sentinel: use default priority in <code>rx_hcsr04_init()</code>
= 1	<code>k_hcsr04_irq_priority_min</code>	Minimum valid ICU interrupt priority
= 10	<code>k_hcsr04_irq_priority_default</code>	Default ICU interrupt priority (balances latency and preemption)
= 15	<code>k_hcsr04_irq_priority_max</code>	Maximum valid ICU interrupt priority

See also: `rx_hcsr04_config_t.irq_priority` Configure sensor priority

Example:

```
//Usesentineltoletrx_hcsr04_init()pickthedefaultpriority
rx_hcsr04_config_tcfg={
    .trigger_pin=k_rx_pf_5,
    .echo_pin=k_rx_p0_3,
    .timeout_us=k_hcsr04_echo_timeout_us,
    .echo_mode=k_hcsr04_echo_irq,
    .echo_irq=k_hcsr04_irq_11,
    .irq_priority=k_hcsr04_irq_priority_unset,//
    rx_hcsr04_init()appliesk_hcsr04_irq_priority_default
};
rx_hcsr04_tsensor;
rx_err_terr=rx_hcsr04_init(&sensor,&cfg);
```

Since Version 1.2.0 (Issue 296)

—

`rx_hcsr04_sensor_index_t`

Named sensor slot indices for ISR registration and dispatch.

Maps each physical sensor position to a zero-based array index. Use in the `sensor_index` field of `rx_hcsr04_config_t` when `echo_mode == k_hcsr04_echo_irq`. The ISR dispatch table uses this index to route echo timestamps to the correct sensor handle.

STAR Hardware Mapping:

`k_hcsr04_sensor_count` is the exclusive upper bound used for range validation inside `rx_hcsr04_init()`: a `sensor_index` value must satisfy `(uint8_t)sensor_index < (uint8_t)k_hcsr04_sensor_count`.

Values are zero-based consecutive integers [0, 3]; `k_hcsr04_sensor_count (4)` is the exclusive upper bound

Value	Name	Description
= 0	<code>k_hcsr04_sensor_front_left</code>	Sonar 0 Front-Left (IRQ11, P03)
= 1	<code>k_hcsr04_sensor_front_right</code>	Sonar 1 Front-Right (IRQ10, P02)

= 2	k_hcsr04_sensor_back_left	Sonar 2 Back-Left (IRQ9, P01)
= 3	k_hcsr04_sensor_back_right	Sonar 3 Back-Right (IRQ8, P00)
= 4	k_hcsr04_sensor_count	Total number of sensors; used as exclusive upper-bound for range validation

Note: k_hcsr04_sensor_count is used as the exclusive upper-bound for range validation; it does not represent a valid sensor slot] **See also:** rx_hcsr04_config_t.sensor_index Field using this enum, rx_hcsr04_isr_register() Receives this value during init

Example:

```
//Configure front-left sensor in IRQ mode
rx_hcsr04_config_t cfg = {
    .trigger_pin = k_rx_pf_5,
    .echo_pin = k_rx_p0_3, //P03->IRQ11
    .timeout_us = k_hcsr04_echo_timeout_us,
    .echo_mode = k_hcsr04_echo_irq,
    .echo_irq = k_hcsr04_irq_11,
    .irq_priority = k_hcsr04_irq_priority_default,
    .sensor_index = k_hcsr04_sensor_front_left, //Slot0
};
```

Since Version 1.3.0 (Issue 336 - promoted from internal ISR header)

rx_hcsr04_callback_t

```
typedef void (* rx_hcsr04_callback_t) (rx_hcsr04_t *handle, const rx_hcsr04_result_t
*result, void *user_data)
```

Async measurement callback function type.

Called when async measurement completes or times out.

rx_hcsr04_init

```
rx_err_t rx_hcsr04_init(rx_hcsr04_t *handle, const rx_hcsr04_config_t *config)
```

Initialize HC-SR04 sensor.

Configures GPIO pins for trigger (output) and echo (input). In IRQ mode, also configures the MPC pin function and ICU for edge detection. Validates that the echo_pin matches the echo_irq mapping (P00->IRQ8, P01->IRQ9, P02->IRQ10, P03->IRQ11) before calling rx_mpc_set_irq().

Initialize HC-SR04 sensor.

Configures GPIO pins for trigger and echo, initializes the sensor handle with default settings, and verifies GPIO configuration.

Parameters:

Direction	Name	Description
out	handle	Handle to initialize (caller-allocated)

in	config	Configuration with pin assignments and mode
out	handle	Sensor handle to initialize
in	config	Configuration parameters (pins, timeout)

Returns: Error code from HAL GPIO operations

Value	Description
k_rx_ok	Success
k_rx_err_null_ptr	handle or config is nullptr
k_rx_err_invalid_arg	port/pin values are invalid, IRQ number out of range, or echo_pin/echo_irq mismatch
k_rx_err_gpio_conflict	pins already reserved
k_rx_err_invalid_state	handle already initialized

Pre: handle must not be NULL

Pre: config must not be NULL with valid port/pin values

Pre: If echo_mode == k_hcsr04_echo_irq: echo_pin and echo_irq must match hardware mapping (P00<->IRQ8, P01<->IRQ9, P02<->IRQ10, P03<->IRQ11)

Pre: If echo_mode == k_hcsr04_echo_irq: echo_irq must be in range k_hcsr04_irq_8..k_hcsr04_irq_11 (only IRQ8-11 have ISR handlers and ICU support in this firmware)

Post: handle->initialized == true on success

Post: Trigger pin configured as GPIO output (low)

Post: Echo pin configured as GPIO input (polling) or IRQ input (IRQ mode)

Post: handle->irq_priority reflects the effective priority used (IRQ mode only)

Note: Not thread-safe; caller must synchronize if called from multiple threads

Note: For IRQ mode, irq_priority of k_hcsr04_irq_priority_unset (0) selects default (k_hcsr04_irq_priority_default)

Warning: echo_pin and echo_irq MUST match the hardware mapping (P00<->IRQ8, P01<->IRQ9, P02<->IRQ10, P03<->IRQ11)] **Example: Initialize Polling Mode Sensor:**

```
rx_hcsr04_tsensor={0};
rx_hcsr04_config_tcfg={.trigger_pin=k_rx_pf_5,.echo_pin=k_rx_p0_3,.timeout_us=k_hcsr04_echo_timeout_us,.ec
rx_err_tterr=rx_hcsr04_init(&sensor,&cfg); if(err!=k_rx_ok){ rx_log_error("SONAR","Initfailed");
returnerr; }
```

Example: Initialize IRQ Mode Sensor: rx_hcsr04_tsensor={0};
rx_hcsr04_config_tcfg={.trigger_pin=k_rx_pf_5,.echo_pin=k_rx_p0_3,.timeout_us=k_hcsr04_echo_timeout_us,.ech
rx_err_tterr=rx_hcsr04_init(&sensor,&cfg); if(err!=k_rx_ok){ rx_log_error("SONAR","IRQinitfailed");
returnerr; }

See also: rx_hcsr04_irq_t Type-safe IRQ number enum, rx_hcsr04_config_t.irq_priority
Configurable interrupt priority, rx_hcsr04_irq_priority_t Named priority constants

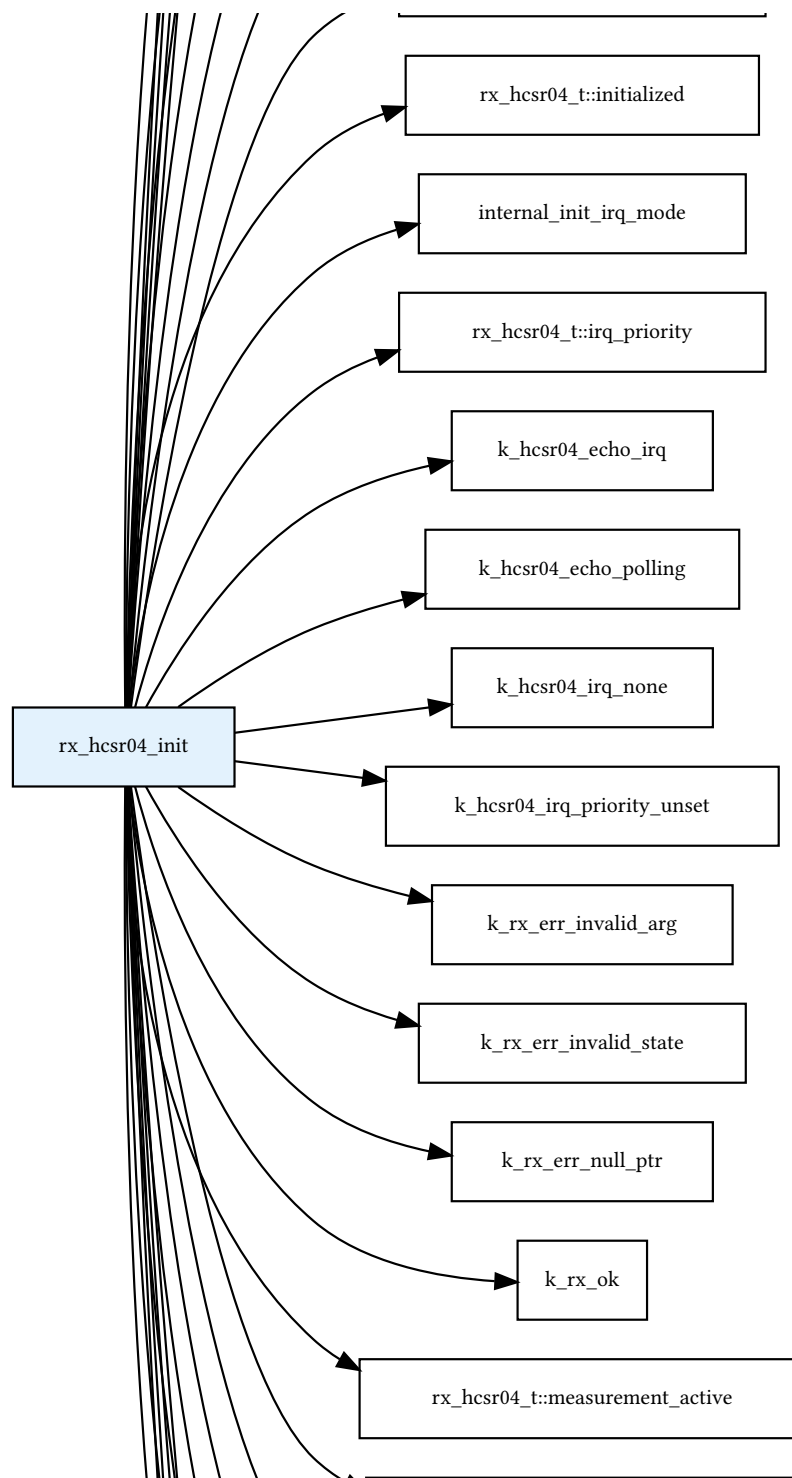


Figure 932: Call graph for rx_hcsr04_init

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:849`

Since Version 1.0.0 (IRQ mode added in Version 1.2.0)

—

rx_hcsr04_deinit

```
rx_err_t rx_hcsr04_deinit(rx_hcsr04_t *handle)
```

Deinitialize HC-SR04 sensor.

Releases GPIO pin reservations and resets handle state.

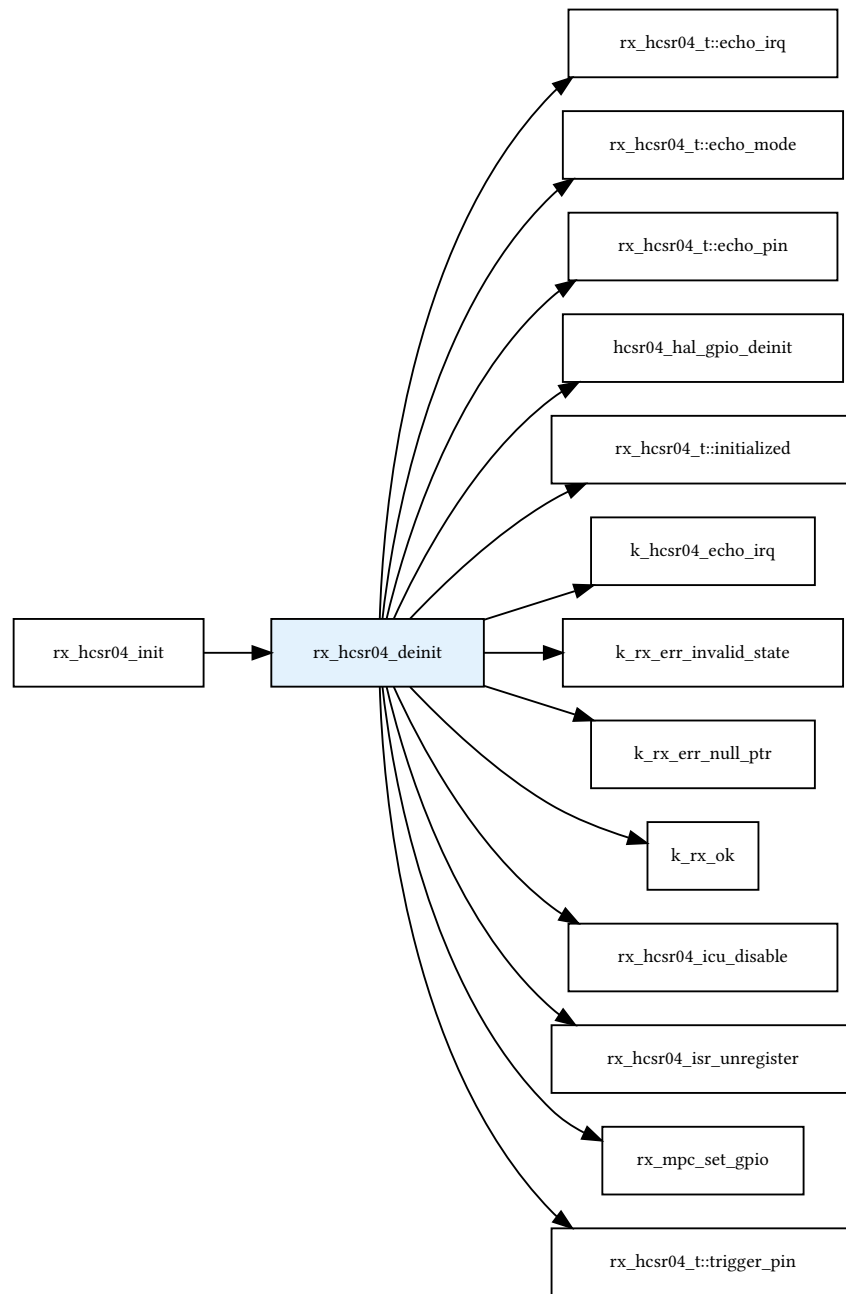
Deinitialize HC-SR04 sensor.

Releases GPIO pins and clears the sensor handle state.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle
inout	handle	Sensor handle to deinitialize

Returns: k_rx_err_invalid_state if not initialized

Figure 933: Call graph for `rx_hcsr04_deinit`

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:862`

rx_hcsr04_worker_init

```
rx_err_t rx_hcsr04_worker_init(void)
```

Initialize HC-SR04 async worker thread.

Creates a ThreadX worker thread for non-blocking async measurements. Must be called before using rx_hcsr04_measure_async() for true async operation.

The worker thread:

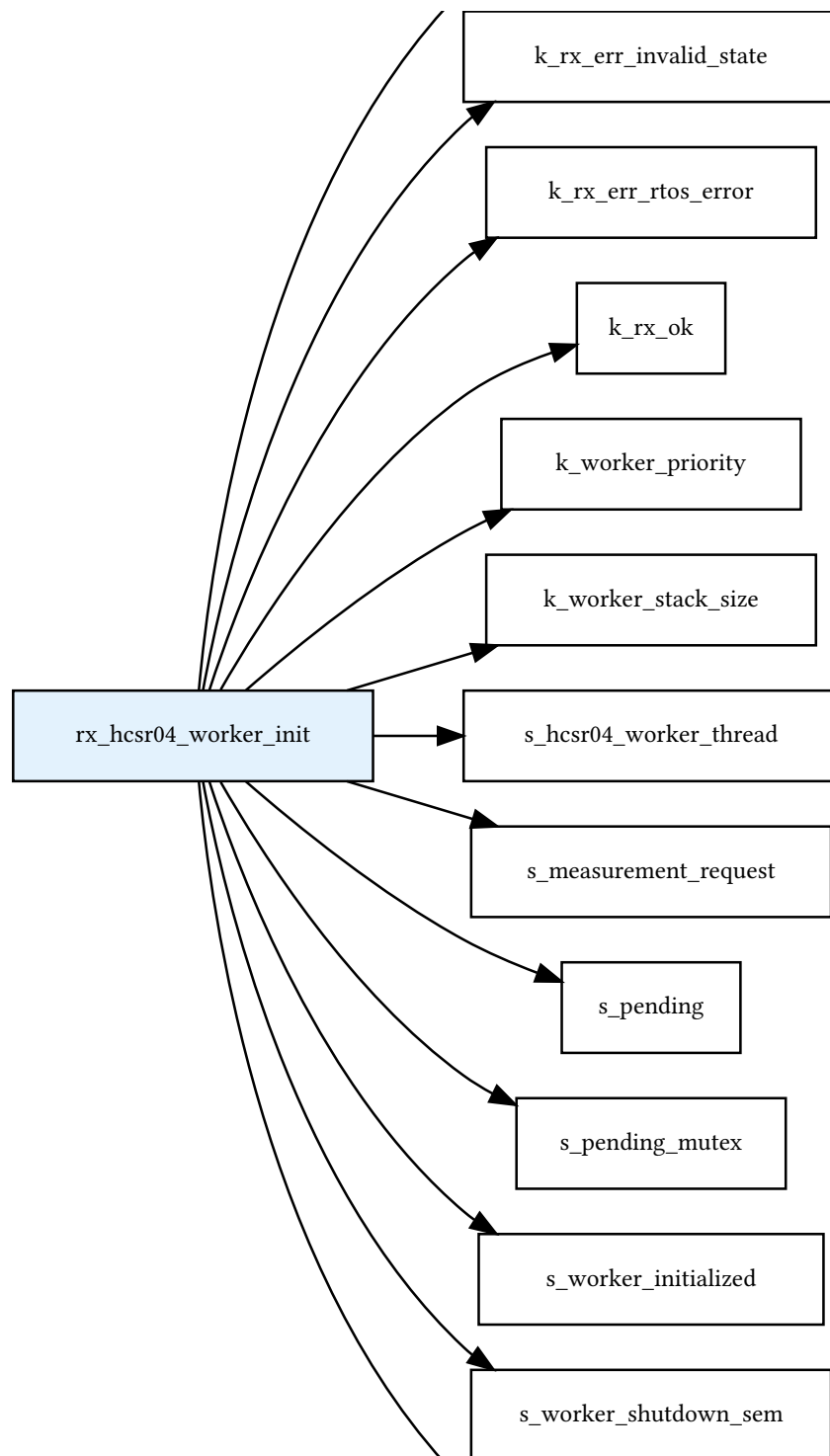
- Runs at priority 10
- Uses 1KB stack
- Processes measurement requests from event flags
- Invokes callbacks in worker thread context

Initialize HC-SR04 async worker thread.

Creates the worker thread, mutex, event flags, and semaphore needed for asynchronous measurement operations. Must be called before using rx_hcsr04_measure_async().

Returns: k_rx_err_rtos_error if ThreadX resource creation fails

Note: This is optional. If not called, rx_hcsr04_measure_async() will fall back to synchronous operation (callback invoked before return).

Figure 934: Call graph for `rx_hcsr04_worker_init`

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:883`

—

rx_hcsr04_worker_deinit

```
rx_err_t rx_hcsr04_worker_deinit(void)
```

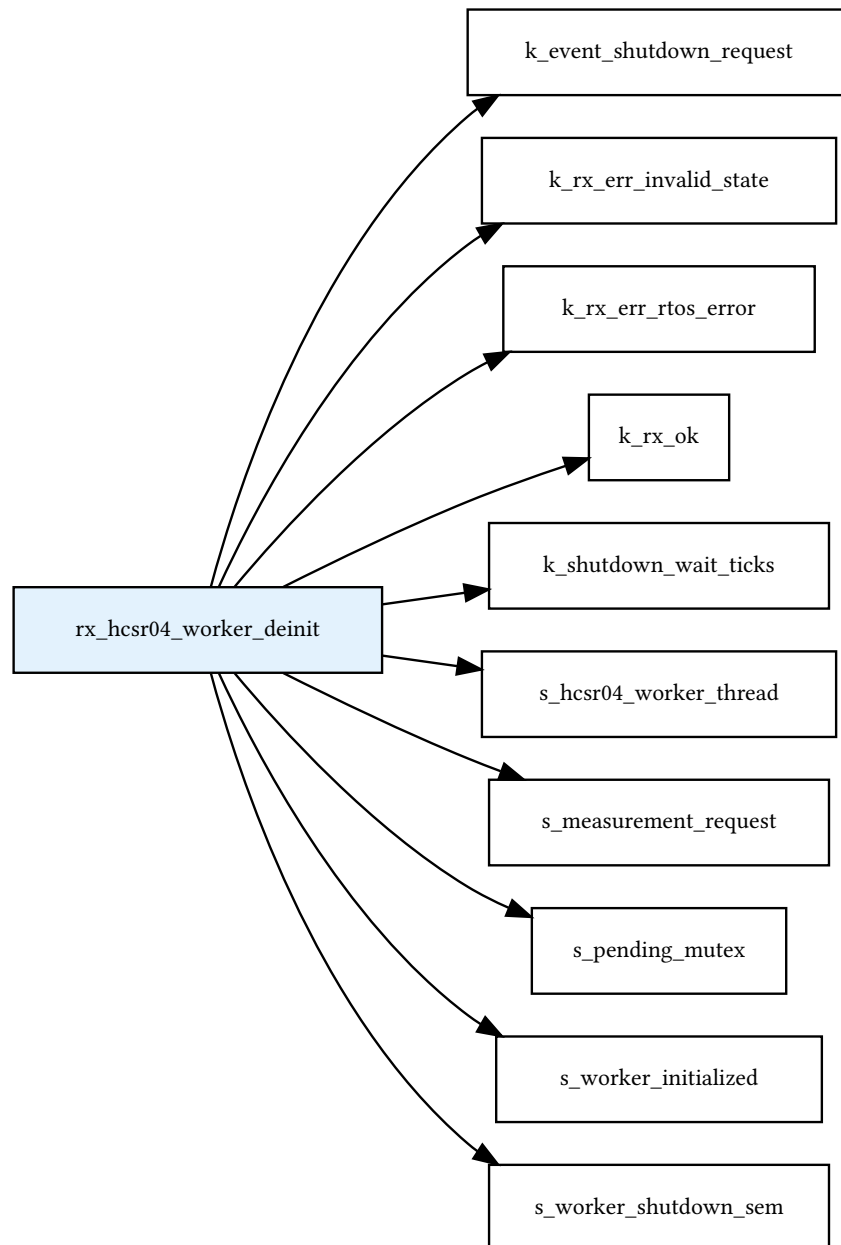
Deinitialize HC-SR04 async worker thread.

Terminates the worker thread and releases resources. Any pending async measurements will be cancelled.

Deinitialize HC-SR04 async worker thread.

Gracefully shuts down the worker thread and releases all ThreadX resources. Waits for the worker thread to complete any in-progress measurement before terminating.

Returns: `k_rx_err_rtos_error` if ThreadX cleanup fails

Figure 935: Call graph for `rx_hcsr04_worker_deinit`

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:894`

rx_hcsr04_measure_blocking

```
rx_err_t rx_hcsr04_measure_blocking(rx_hcsr04_t *handle, float *distance_cm)
```

Measure distance (blocking).

Performs a complete measurement cycle:

1. Send 10us trigger pulse
2. Wait for echo pulse start
3. Measure echo pulse duration
4. Convert to distance

Blocks for up to timeout_us (default 30ms).

Measure distance (blocking).

Triggers the sensor, waits for echo pulse, and converts to distance in centimeters. Blocks until measurement completes or times out.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
out	distance_cm	Measured distance in centimeters
in	handle	Sensor handle
out	distance_cm	Measured distance in centimeters

Returns: k_rx_err_out_of_range if distance outside sensor range (2-400 cm)

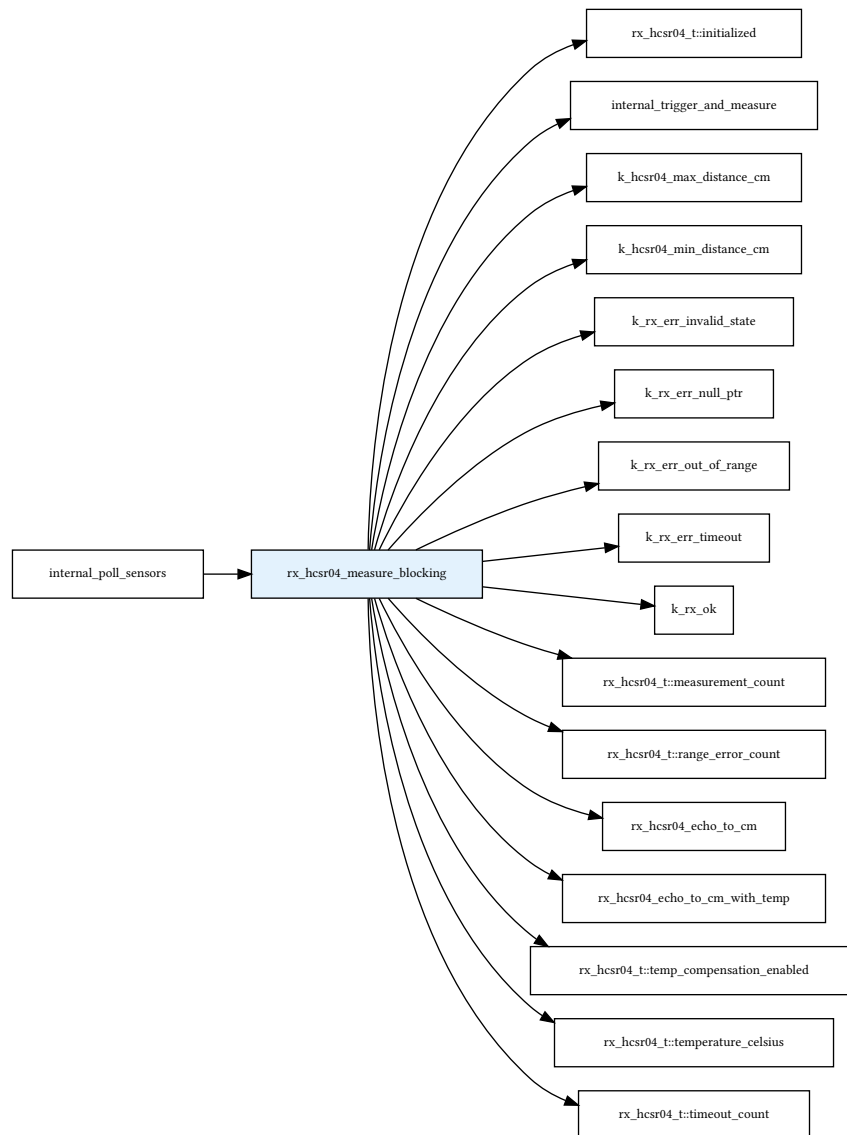


Figure 936: Call graph for rx_hcsr04_measure_blocking

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:921`

rx_hcsr04_measure

```
rx_err_t rx_hcsr04_measure(rx_hcsr04_t *handle, rx_hcsr04_result_t *result)
```

Measure distance with full result (blocking).

Same as `rx_hcsr04_measure_blocking()` but returns complete result including both distance units and raw timing.

Measure distance with full result (blocking).

Triggers the sensor, measures echo pulse, and populates result structure with distance in both cm and inches, echo time, and status code.

Parameters:

Direction	Name	Description
in	<code>handle</code>	Sensor handle
out	<code>result</code>	Complete measurement result
in	<code>handle</code>	Sensor handle
out	<code>result</code>	Measurement result structure

Returns: `k_rx_err_out_of_range` if distance outside sensor range (2-400 cm)

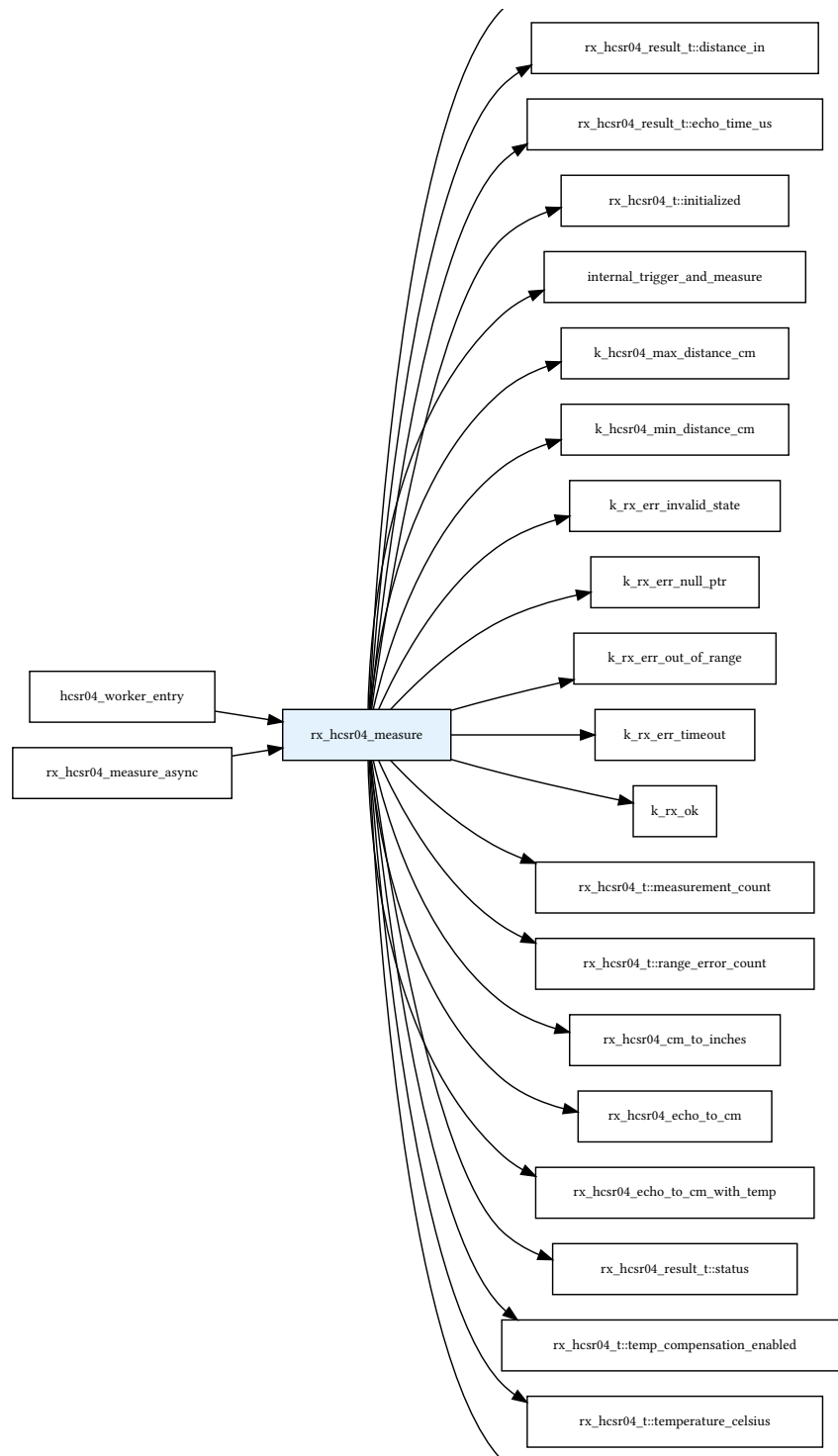


Figure 937: Call graph for `rx_hcsr04_measure`

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:938`

—

rx_hcsr04_measure_async

```
rx_err_t rx_hcsr04_measure_async(rx_hcsr04_t *handle, rx_hcsr04_callback_t  
callback, void *user_data)
```

Start asynchronous measurement.

Initiates measurement and invokes callback with result.

Operating Modes:

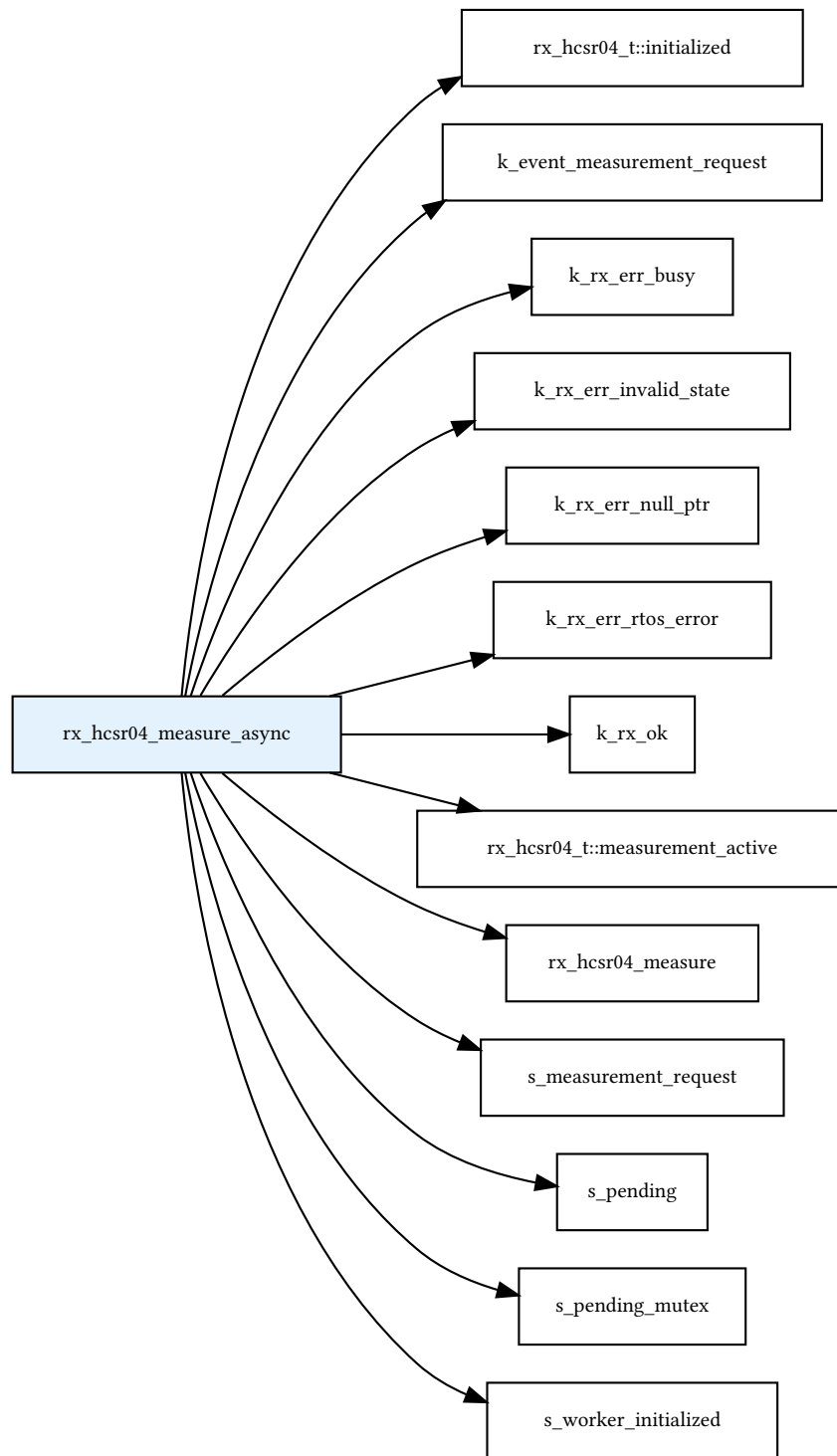
- Worker initialized (`rx_hcsr04_worker_init()` called): Returns immediately, callback invoked from worker thread when complete. True non-blocking operation.
- Worker not initialized: Performs synchronous measurement, callback invoked before return. Caller blocks for up to `timeout_us` (default 30ms).

Parameters:

Direction	Name	Description
in	handle	Sensor handle
in	callback	Completion callback (required)
in	user_data	User context passed to callback (can be NULL)

Returns: `k_rx_err_busy` if measurement already in progress

Note: Use `rx_hcsr04_cancel()` to abort in-progress async measurements. Cancellation only works when worker thread is active.

Figure 938: Call graph for `rx_hcsr04_measure_async`

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:967`

—

rx_hcsr04_is_busy

```
bool rx_hcsr04_is_busy(const rx_hcsr04_t *handle)
```

Check if async measurement is in progress.

Check if async measurement is in progress.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
in	handle	Sensor handle

Returns: true if measurement is active, false otherwise

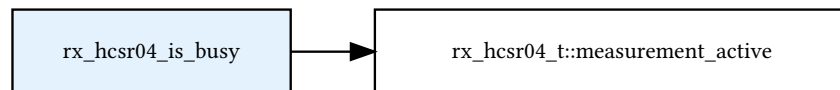


Figure 939: Call graph for rx_hcsr04_is_busy

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:977`

—

rx_hcsr04_cancel

```
rx_err_t rx_hcsr04_cancel(rx_hcsr04_t *handle)
```

Cancel async measurement.

Cancels in-progress async measurement. Callback will NOT be invoked.

Cancel async measurement.

Sets a cancel flag that will be checked during echo pulse waiting, causing the measurement to abort with `k_rx_err_cancelled`.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
inout	handle	Sensor handle

Returns: `k_rx_err_invalid_state` if no measurement is active

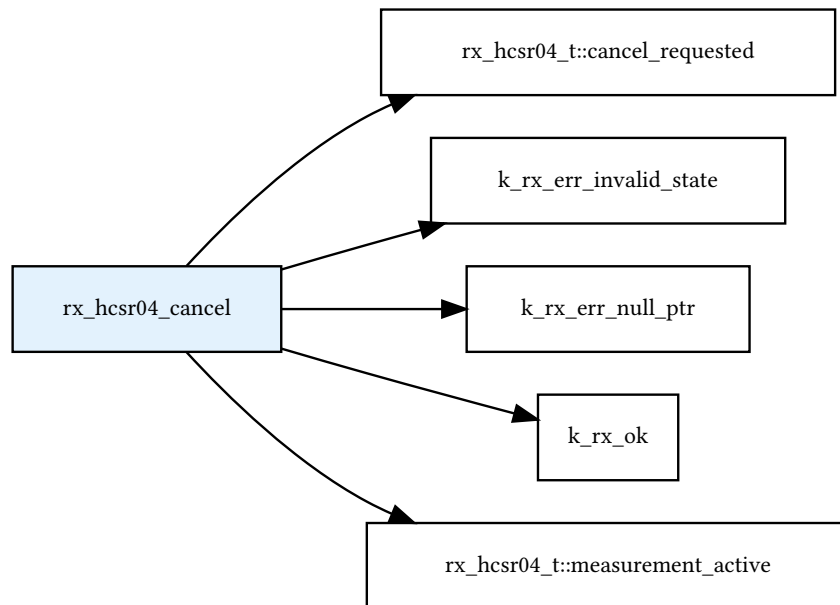


Figure 940: Call graph for rx_hcsr04_cancel

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:990`

—

rx_hcsr04_set_temperature

```
rx_err_t rx_hcsr04_set_temperature(rx_hcsr04_t *handle, float temp_celsius)
```

Set ambient temperature for automatic distance compensation.

Enables temperature compensation and updates the temperature used for all subsequent distance measurements. The speed of sound varies with temperature (0.17% per degC), affecting measurement accuracy.

When enabled, all measurement functions (`rx_hcsr04_measure_blocking`, `rx_hcsr04_measure`, `rx_hcsr04_measure_async`) will automatically use temperature-compensated distance calculations.

Temperature can be updated at any time, including between measurements. Typical usage: read from DS18B20 sensor periodically and update.

Set ambient temperature for automatic distance compensation.

Enables temperature compensation and sets the ambient temperature used for calculating the speed of sound in distance conversions.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle

in	temp_celsius	Ambient temperature in degrees Celsius Valid range: -40degC to +85degC
inout	handle	Sensor handle
in	temp_celsius	Ambient temperature in degrees Celsius (-40 to +85degC)

Returns: k_rx_err_invalid_arg if temperature outside valid range

Note: To disable temperature compensation, call rx_hcsr04_disable_temp_compensation()] **See also:** rx_hcsr04_disable_temp_compensation(), rx_hcsr04_echo_to_cm_with_temp()

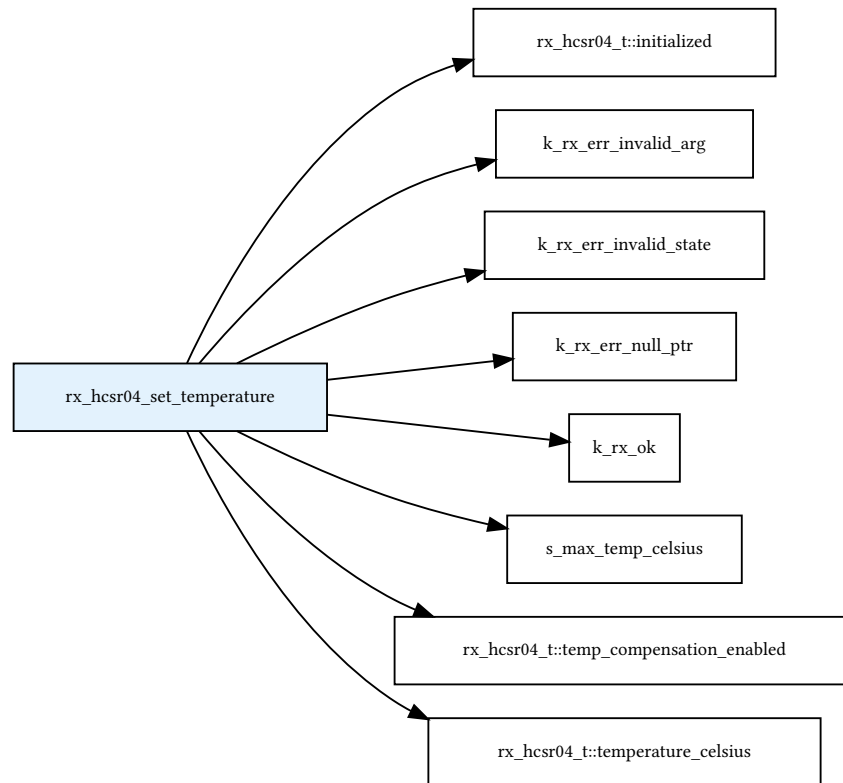


Figure 941: Call graph for rx_hcsr04_set_temperature

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1025`

—

rx_hcsr04_disable_temp_compensation

```
rx_err_t rx_hcsr04_disable_temp_compensation(rx_hcsr04_t *handle)
```

Disable automatic temperature compensation.

Disables temperature compensation and reverts to assuming 20degC for all distance calculations. This is the default behavior after initialization.

Disable automatic temperature compensation.

Disables temperature compensation and uses default 20degC speed of sound for distance calculations.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle
inout	handle	Sensor handle

Returns: k_rx_err_invalid_state if not initialized

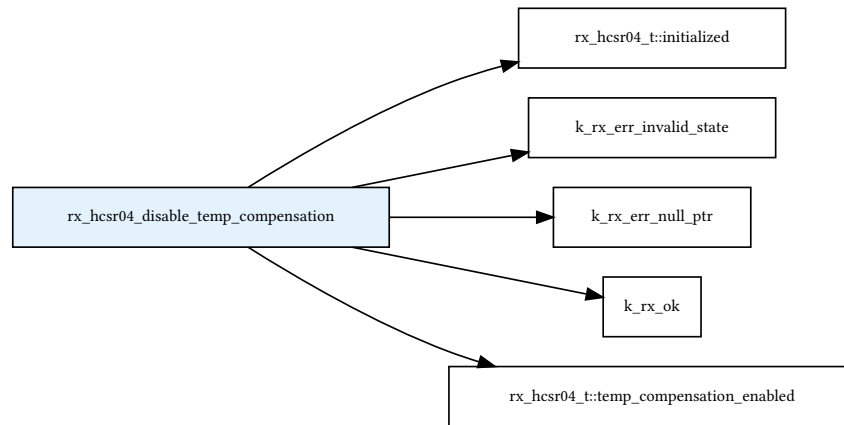


Figure 942: Call graph for `rx_hcsr04_disable_temp_compensation`

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1039`

—

rx_hcsr04_is_temp_compensation_enabled

```
bool rx_hcsr04_is_temp_compensation_enabled(const rx_hcsr04_t *handle)
```

Check if temperature compensation is enabled.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
in	handle	Sensor handle

Returns: true if temperature compensation is enabled, false otherwise

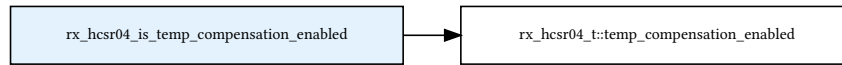


Figure 943: Call graph for rx_hcsr04_is_temp_compensation_enabled

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1049`

—

rx_hcsr04_get_temperature

```
rx_err_t rx_hcsr04_get_temperature(const rx_hcsr04_t *handle, float
*temp_celsius)
```

Get current temperature setting.

Returns the currently configured temperature value used for compensation. If compensation is disabled, still returns the last set value (or 20.0degC default).

Retrieves the temperature currently used for speed of sound compensation.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
out	temp_celsius	Current temperature setting
in	handle	Sensor handle
out	temp_celsius	Current temperature in degrees Celsius

Returns: k_rx_err_invalid_state if not initialized

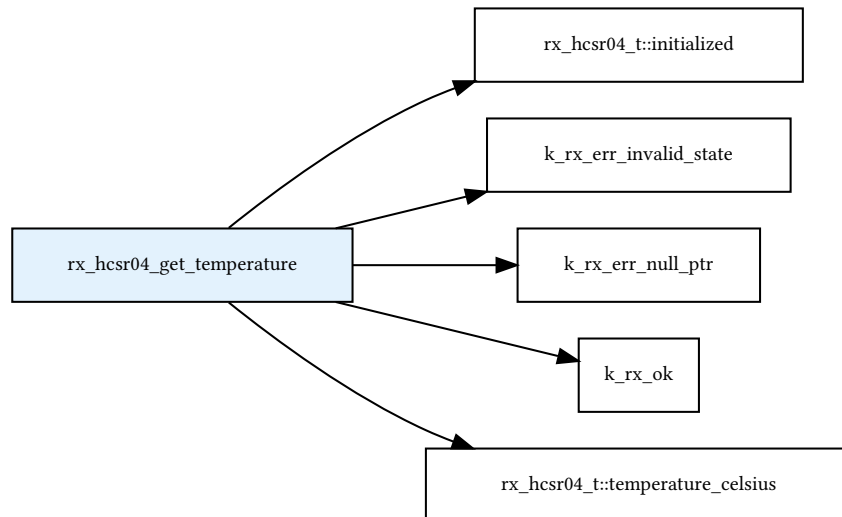


Figure 944: Call graph for rx_hcsr04_get_temperature

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1064`

rx_hcsr04_cm_to_inches

```
float rx_hcsr04_cm_to_inches(float distance_cm)
```

Convert distance from centimeters to inches.

Parameters:

Direction	Name	Description
in	distance_cm	Distance in centimeters

Returns: Distance in inches

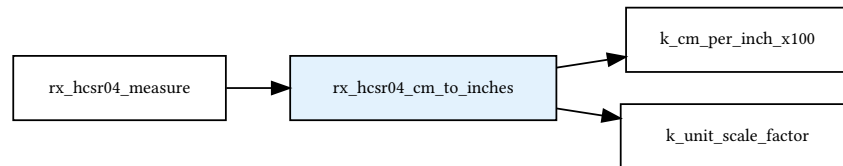


Figure 945: Call graph for rx_hcsr04_cm_to_inches

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1078`

rx_hcsr04_echo_to_cm

```
float rx_hcsr04_echo_to_cm(uint32_t echo_time_us)
```

Convert echo time to distance in centimeters.

Uses formula: $\text{distance} = (\text{echo_us} * \text{speed_of_sound}) / 2$ Speed of sound at 20C = 343 m/s = 0.0343 cm/us

Parameters:

Direction	Name	Description
in	echo_time_us	Echo pulse duration in microseconds

Returns: Distance in centimeters

Note: This function assumes 20degC. For temperature compensation, use `rx_hcsr04_echo_to_cm_with_temp()` instead.

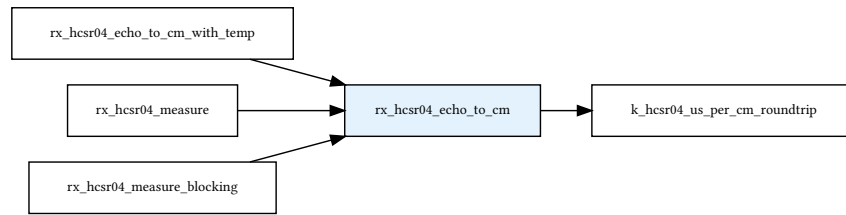


Figure 946: Call graph for rx_hcsr04_echo_to_cm

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1093`

—

rx_hcsr04_echo_to_cm_with_temp

```
float rx_hcsr04_echo_to_cm_with_temp(uint32_t echo_time_us, float temp_celsius)
```

Convert echo time to distance with temperature compensation.

Calculates distance using temperature-compensated speed of sound. Speed of sound varies with temperature: $v = 331.3 + (0.606 * \text{temp_c})$ m/s

Temperature compensation improves accuracy by 0.17% per degC deviation from 20degC.

Example: At 10degC, speed is 337 m/s vs 343 m/s at 20degC (1.75% difference)

Calculates distance using temperature-compensated speed of sound.

Parameters:

Direction	Name	Description
in	echo_time_us	Echo pulse duration in microseconds
in	temp_celsius	Ambient temperature in degrees Celsius
in	echo_time_us	Echo pulse duration in microseconds
in	temp_celsius	Ambient temperature in degrees Celsius (-40 to +85degC)

Returns: Distance in centimeters, or 0.0 if input is invalid

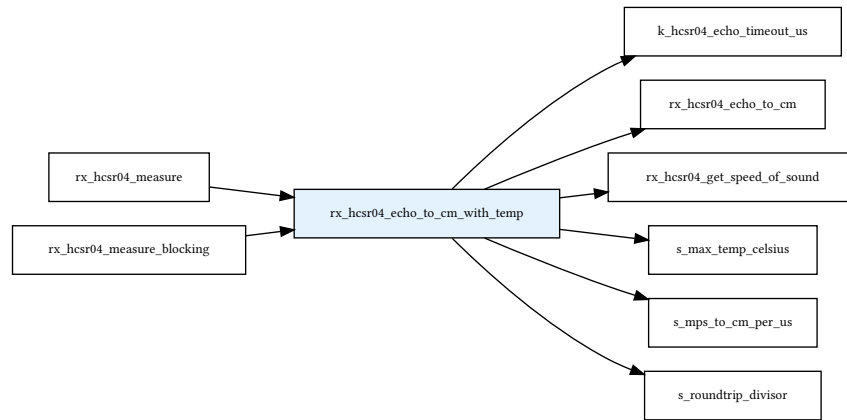


Figure 947: Call graph for rx_hcsr04_echo_to_cm_with_temp

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1110`

rx_hcsr04_get_speed_of_sound

```
float rx_hcsr04_get_speed_of_sound(float temp_celsius)
```

Calculate speed of sound at given temperature.

Uses formula: $v = 331.3 + (0.606 * \text{temp_c})$ m/s

Valid temperature range: -40degC to +85degC (DS18B20 operating range)

Parameters:

Direction	Name	Description
in	temp_celsius	Ambient temperature in degrees Celsius

Returns: Speed of sound in m/s

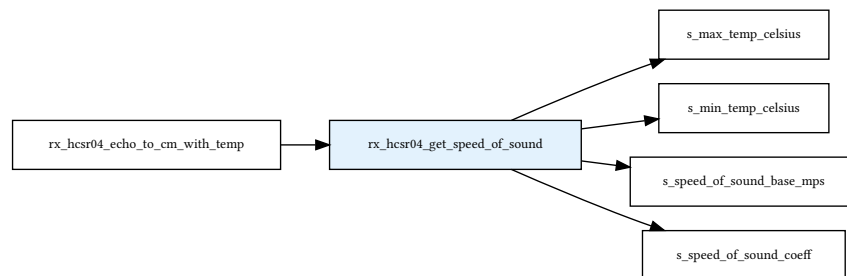


Figure 948: Call graph for rx_hcsr04_get_speed_of_sound

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1123`

rx_hcsr04_get_stats

```
rx_err_t rx_hcsr04_get_stats(const rx_hcsr04_t *handle, uint32_t
*measurement_count, uint32_t *timeout_count, uint32_t *range_error_count)
```

Get sensor statistics.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
out	measurement_count	Total measurements (can be NULL)
out	timeout_count	Timeout count (can be NULL)
out	range_error_count	Range error count (can be NULL)

Returns: k_rx_err_invalid_state if not initialized

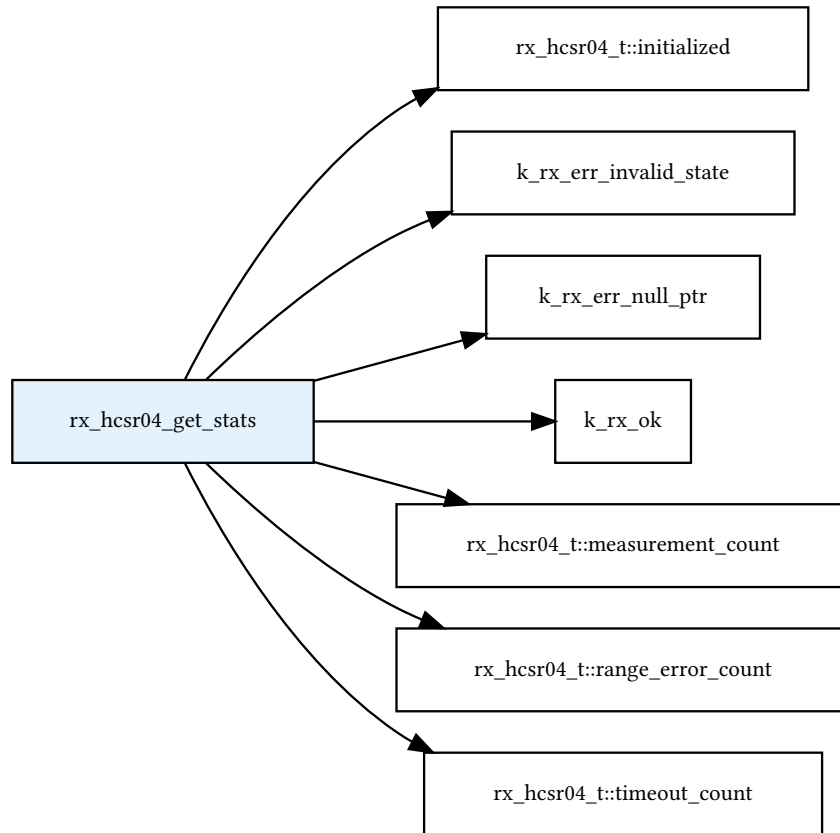


Figure 949: Call graph for rx_hcsr04_get_stats

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1137`

—

rx_hcsr04_reset_stats

```
rx_err_t rx_hcsr04_reset_stats(rx_hcsr04_t *handle)
```

Reset sensor statistics.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle

Returns: k_rx_err_invalid_state if not initialized

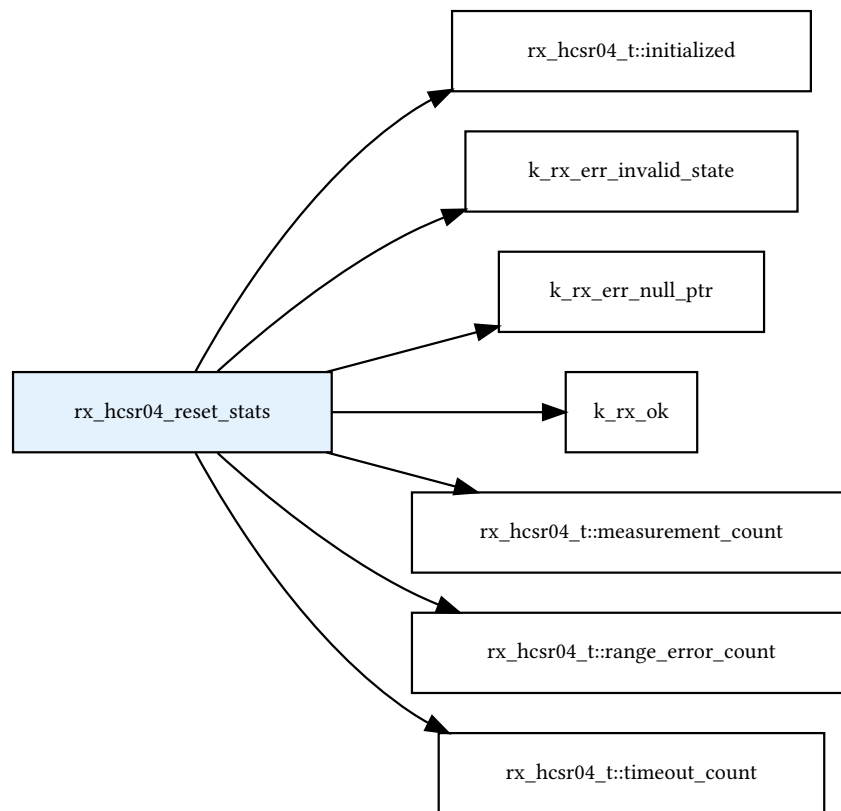


Figure 950: Call graph for rx_hcsr04_reset_stats

Defined in `libs/rx_hcsr04/inc/rx_hcsr04.h:1151`

—

rx_hcsr04.c

HC-SR04 Ultrasonic Distance Sensor Driver Implementation.

Includes:

- "rx_hcsr04.h"

- <stddef.h>
- "rx_check.h"
- "rx_hcsr04_hal.h"
- "rx_hcsr04_icu.h"
- "rx_hcsr04_isr.h"
- "rx_log.h"
- "rx_mpc.h"
- "tx_api.h"

rx_hcsr04_worker_priority_t

Worker thread priority configuration.

ThreadX priority level for async measurement worker thread. Lower numeric value = higher priority (ThreadX convention).

Priority 10 chosen for medium-priority sensor operations:

- Higher than telemetry (15)
- Lower than motor control (5)
- Same as general sensor polling

Value	Name	Description
= 10	k_worker_priority	Medium priority (0=highest, 31=lowest in ThreadX)

—

rx_hcsr04_worker_stack_t

Worker thread stack size configuration.

Stack allocated for HC-SR04 async worker thread.

Stack Usage Analysis:

- Function call depth: 4 (worker -> measure -> measure_pulse -> HAL)
- Local variables: 200 bytes (result struct, counters, status)
- ThreadX overhead: 256 bytes
- Safety margin: 2x (568 bytes)
- Total: 1024 bytes (conservative, allows future expansion)

Verified by stack checking: Maximum observed usage: 600 bytes (40% headroom remaining).

Value	Name	Description
= 1024	k_worker_stack_size	1 KB stack (600 bytes typical usage)

Example:

```
#define TX_ENABLE_STACK_CHECKING // Intx_user.h
```

—

rx_hcsr04_event_flags_t

Worker thread event flag definitions.

Bit flags used with ThreadX event flags group to signal worker thread. Supports OR-based waiting (thread wakes on either event).

Event Processing:

- `k_event_measurement_request`: Start async measurement
- `k_event_shutdown_request`: Gracefully terminate worker thread

Priority: Shutdown checked first to ensure clean exit.

Value	Name	Description
= 1U	<code>k_event_measurement_request</code>	Bit 0: Measurement request (normal operation)
= 2U	<code>k_event_shutdown_request</code>	Bit 1: Shutdown request (cleanup path)

—

`rx_hcsr04_conversion_t`

Unit conversion constants.

Value	Name	Description
= 254	<code>k_cm_per_inch_x100</code>	Centimeters per inch * 100 (2.54 * 100)

—

`rx_hcsr04_scale_t`

Scaling factor for integer-based unit conversion.

Value	Name	Description
= 100	<code>k_unit_scale_factor</code>	Scale factor for fixed-point conversion

—

`rx_hcsr04_shutdown_t`

Shutdown wait time in ThreadX ticks.

Value	Name	Description
= 5	<code>k_shutdown_wait_ticks</code>	50ms at 100 Hz tick rate

—

`rx_hcsr04_poll_limits_t`

Echo polling loop bounds.

Value	Name	Description
= 30000	<code>k_echo_poll_max_iterations</code>	Max polling iterations

—

`rx_hcsr04_irq_range_t`

Valid IRQ number range for P00-P07 echo pins.

Named bounds for the IRQ number range accepted by the HC-SR04 IRQ mode. P00-P07 map to IRQ8-IRQ15 respectively. The downstream ICU and ISR layers further restrict to IRQ8-11 (only those have ISR handlers registered).

`k_irq_range_min` <= `config->echo_irq` <= `k_irq_range_max` for valid configs

Value	Name	Description
= 8	k_irq_range_min	Minimum valid IRQ number (IRQ8 = P00)
= 15	k_irq_range_max	Maximum valid IRQ number (IRQ15 = P07)

See also: rx_hcsr04_init() Uses these bounds for initial IRQ validation, internal_init_irq_mode() Performs the actual range check

Example:

```
//Validateecho_irqininternal_init_irq_mode:
if((uint8_t)config->echo_irq<k_irq_range_min||
(uint8_t)config->echo_irq>k_irq_range_max){
returnk_rx_err_invalid_arg;
}
```

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_irq_port_t

Port number for P00-P07 IRQ echo pins.

Identifies the GPIO port that maps to external IRQ pins. Only Port 0 pins (P00-P03) are used for HC-SR04 echo sensing with IRQ8-IRQ11.

k_irq_p0_port == 0 (Port 0 is the only port supporting IRQ for HC-SR04)

Value	Name	Description
= 0	k_irq_p0_port	Port 0 (P00-P07) maps to IRQ8-IRQ15

See also: internal_init_irq_mode() Uses this to validate echo_pin port

Example:

```
//VerifyechopinisonPort0forIRQmapping:
constuint8_tpin_port=rx_port_from_pin(config->echo_pin);
if(pin_port!=k_irq_p0_port){returnk_rx_err_invalid_arg;}
```

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_irq_yield_t

ThreadX sleep duration for IRQ polling yield.

Number of ThreadX ticks to sleep per iteration in the IRQ echo wait loop. At the default 100 Hz tick rate, 1 tick = 10 ms.

k_irq_yield_ticks >= 1 (must yield at least 1 tick to avoid busy-wait)

Value	Name	Description
= 1	k_irq_yield_ticks	ThreadX ticks to yield per IRQ poll iteration (10ms at 100Hz)

See also: internal_measure_echo_pulse_irq() Uses this in the polling loop

Example:

```
//YieldCPUbetweenISRcompletionpolls:
tx_thread_sleep(k_irq_yield_ticks);
```

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_irq_poll_limits_t

Bounded loop iteration cap for IRQ echo wait.

Maximum iterations for the IRQ measurement polling loop in `internal_measure_echo_pulse_irq()`. Prevents unbounded busy-wait per NASA Power of 10 Rule 2. Each iteration calls `tx_thread_sleep(1)`, which yields for 10ms at the default 100 Hz ThreadX tick rate. 100 iterations provides a 1 second upper bound as a safety net; the actual timeout is governed by `handle->timeout_us` checked each iteration.

`k_irq_poll_max_iterations > 0` (at least one poll attempt)

`k_irq_poll_max_iterations * 10ms >> handle->timeout_us` (safety margin)

Value	Name	Description
= 100	<code>k_irq_poll_max_iterations</code>	Max iterations (10ms/iter at 100Hz, 1s safety bound)

See also: `internal_measure_echo_pulse_irq()` Uses this as the loop bound,
`k_irq_yield_ticks` Sleep duration per iteration

Example:

```
//Boundedpollingloopininternal_measure_echo_pulse_irq:
for(uint32_ti=0;i<k_irq_poll_max_iterations;i++){
  rx_err_t err=rx_hcsr04_isr_get_duration(irq,&duration);
  if(err==k_rx_ok){returnk_rx_ok;}
  tx_thread_sleep(k_irq_yield_ticks);
}
```

Since Version 1.2.0 (Issue 296)

—

s_speed_of_sound_base_mps

```
const float s_speed_of_sound_base_mps
```

Speed of sound base constant (m/s at 0degC).

—

s_speed_of_sound_coeff

```
const float s_speed_of_sound_coeff
```

Speed of sound temperature coefficient (m/s per degC).

—

s_default_temperature_celsius

```
const float s_default_temperature_celsius
```

Default temperature for distance calculations (degC).

—

s_min_temp_celsius

```
const float s_min_temp_celsius
```

Minimum valid temperature (degC) - DS18B20 sensor lower limit.

—

s_max_temp_celsius

```
const float s_max_temp_celsius
```

Maximum valid temperature (degC) - DS18B20 sensor upper limit.

—

s_mps_to_cm_per_us

```
const float s_mps_to_cm_per_us
```

Conversion factor from m/s to cm/us (1 m/s = 0.0001 cm/us).

—

s_roundtrip_divisor

```
const float s_roundtrip_divisor
```

Roundtrip divisor (echo travels to target and back).

—

s_hcsr04_worker_thread

```
TX_THREAD s_hcsr04_worker_thread
```

ThreadX worker thread control block for async measurements.

Warning: NOT thread-safe - only access via ThreadX APIs] **See also:** hcsr04_worker_entry
Worker thread main loop, rx_hcsr04_worker_init Thread creation,
rx_hcsr04_worker_deinit Thread deletion

—

s_worker_stack

```
uint8_t s_worker_stack[k_worker_stack_size]
```

Statically allocated stack memory for worker thread.

Note: Increase size if adding deep call chains or large local buffers

Warning: Stack overflow results in undefined behavior (memory corruption)

—

s_measurement_request

```
TX_EVENT_FLAGS_GROUP s_measurement_request
```

ThreadX event flags group for worker thread signaling.

Warning: NOT thread-safe - only access via ThreadX APIs] **See also:**

`rx_hcsr04_event_flags_t` Event flag definitions, `hcsr04_worker_entry` Worker event processing

—

s_worker_shutdown_sem

TX_SEMAPHORE `s_worker_shutdown_sem`

ThreadX semaphore for synchronizing graceful worker shutdown.

Warning: NOT thread-safe - only access via ThreadX APIs] **See also:**

`rx_hcsr04_worker_deinit` Graceful shutdown implementation, `hcsr04_worker_entry` Worker exit path

—

s_pending_mutex

TX_MUTEX `s_pending_mutex`

ThreadX mutex protecting shared `s_pending` measurement context.

Warning: Must hold mutex when reading or writing `s_pending`] **See also:** `s_pending` Shared measurement context

Example:

```
ThreadAThreadB(worker)
-----
if(s_pending.handle!=nullptr)//false
s_pending.handle=nullptr;//Cleared
s_pending.handle=&sensor_A;//Overwrites!
```

—

s_pending

pending_measurement_t `s_pending`

Shared async measurement request context.

Warning: NEVER access without holding `s_pending_mutex`

Warning: Direct modification outside async subsystem causes race conditions] **See also:**

`s_pending_mutex` Synchronization primitive, `pending_measurement_t` Structure definition

Example:

```
IDLE(handle==nullptr)
v[measure_asynccalled]
QUEUED(handle!=nullptr,workernotyetstarted)
v[workerwakes,startsmeasurement]
ACTIVE(handle!=nullptr,measurementinprogress)
```



```
v[measurementcompletes,callbackinvoked]
IDLE(handle=NULLptr,readyfornextrequest)
```

—

s_worker_initialized

```
bool s_worker_initialized
```

Worker thread subsystem initialization flag.

Note: If false, measure_async() falls back to synchronous operation

Warning: NOT thread-safe - caller must ensure init/deinit not concurrent] **Example:**

```
if(!s_worker_initialized){
    rx_hcsr04_worker_init();//Firsttimesetup
}
rx_hcsr04_measure_async(...);//Safetocallnow
```

—

internal_send_trigger_pulse

```
static rx_err_t internal_send_trigger_pulse(const rx_hcsr04_t *handle)
```

Send 10us trigger pulse to initiate HC-SR04 measurement.

Generates the HC-SR04 trigger pulse sequence per datasheet:

1. Ensure trigger pin LOW (baseline)
2. Delay 2us (settle time)
3. Set trigger pin HIGH
4. Delay 10us (pulse width)
5. Set trigger pin LOW (return to idle)

This pulse sequence causes the HC-SR04 to transmit 8 ultrasonic pulses at 40 kHz and begin listening for echo.

Timing Diagram:

Algorithm:

Parameters:

Direction	Name	Description
in	handle	Sensor handle (must be initialized)

Returns: k_rx_err_hw_operation_failed if GPIO write fails

Pre: handle != NULLptr

Pre: handle->initialized == true

Pre: handle->trigger_pin is configured as output

Post: Trigger pulse sent (10us HIGH pulse completed)

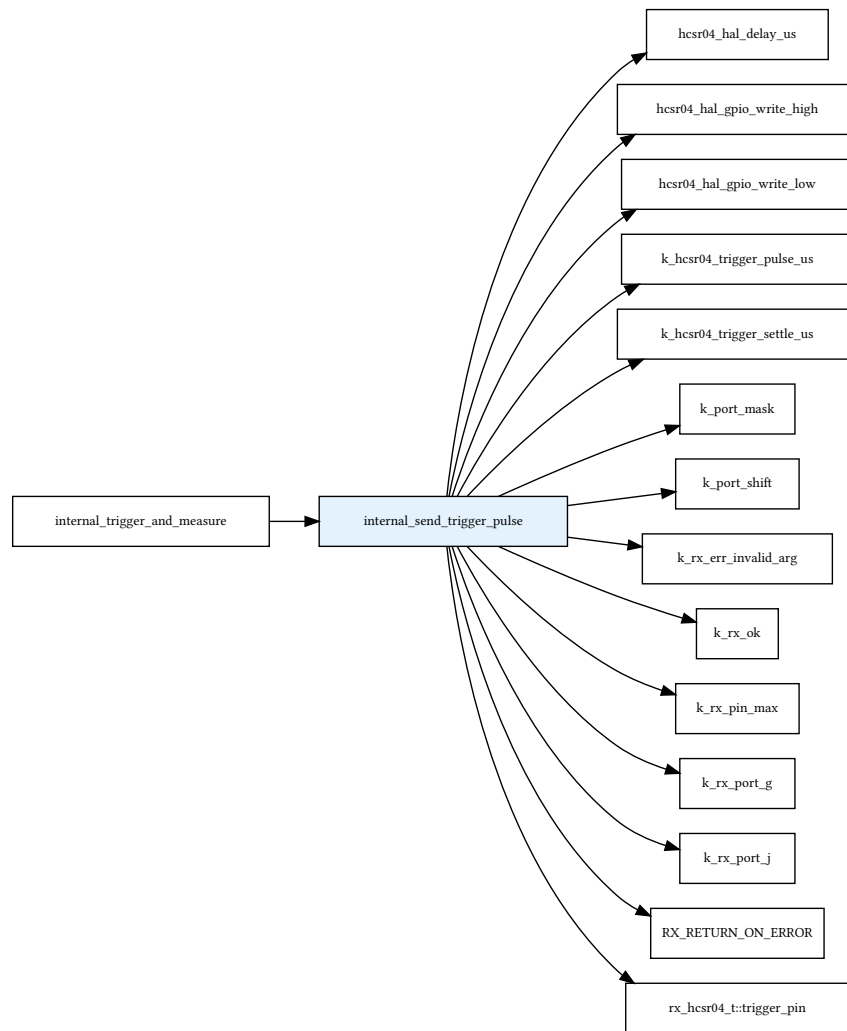
Post: Trigger pin returned to LOW state

Note: Not thread-safe - concurrent calls corrupt trigger timing

Warning: Do not call faster than 60ms (HC-SR04 minimum cycle time)] **Example:**

```
+-----+
TRIG|10us|
--++-----
^^
|+-ReturntoLOW
+-Send8x40kHzburst
```

See also: `hcsr04_hal_gpio_write_high` HAL GPIO write function, `hcsr04_hal_delay_us` HAL microsecond delay, `k_hcsr04_trigger_pulse_us` Trigger pulse duration constant

Figure 951: Call graph for `internal_send_trigger_pulse`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1064`

Since Version 1.0.0

—

internal_wait_for_echo

```
static rx_err_t internal_wait_for_echo(rx_hcsr04_t *handle, const bool
target_state, uint32_t timeout_us)
```

Wait for echo pin to transition to target state with timeout and cancellation.

Polls echo pin until it matches target state or timeout expires. Supports cancellation via `handle->cancel_requested` flag.

Polling Strategy:

- Bounded loop: Max 30,000 iterations (k_echo_poll_max_iterations)
- Time-based exit: Checks elapsed time each iteration
- Cancellation: Checks flag each iteration

Use Cases:

1. Wait for echo HIGH (pulse start): internal_wait_for_echo(handle, true, timeout_us)
2. Wait for echo LOW (pulse end): internal_wait_for_echo(handle, false, timeout_us)

Timing Analysis:

- Near object (2cm): 116us (2 iterations @ 60us/iter)
- Far object (400cm): 23.2ms (387 iterations)
- Timeout (no object): 30ms (500 iterations)

State Diagram:**Flow Diagram:**

Loop iterations $\leq$ k_echo_poll_max_iterations (30000)

Cancellation only checked during polling (not retroactive)

Parameters:

Direction	Name	Description
inout	handle	Sensor handle (cancel_requested cleared on cancellation)
in	target_state	Desired pin state (true = HIGH, false = LOW)
in	timeout_us	Maximum wait time in microseconds

Returns: k_rx_err_hw_operation_failed GPIO read failed

Pre: handle != nullptr

Pre: handle->echo_pin configured as input

Pre: timeout_us > 0 and $\leq$ k_hcsr04_echo_timeout_us

Post: If cancelled: handle->cancel_requested = false

Post: Pin state == target_state (if k_rx_ok returned)

Note: Not thread-safe - concurrent access to handle causes race conditions

Warning: Long timeouts block caller (no RTOS yielding in tight loop)

Performance: Per-iteration time: 1-2us (GPIO read + comparison) Typical iterations: 10-400 (depends on object distance) CPU usage: 100% during polling (busy-wait)

Thread Safety:: NOT thread-safe. Concurrent calls corrupt handle->cancel_requested. Use separate handles or external locking.

See also: internal_measure_echo_pulse Caller function, hcsr04_hal_gpio_read HAL GPIO read function, k_echo_poll_max_iterations Loop bound constant

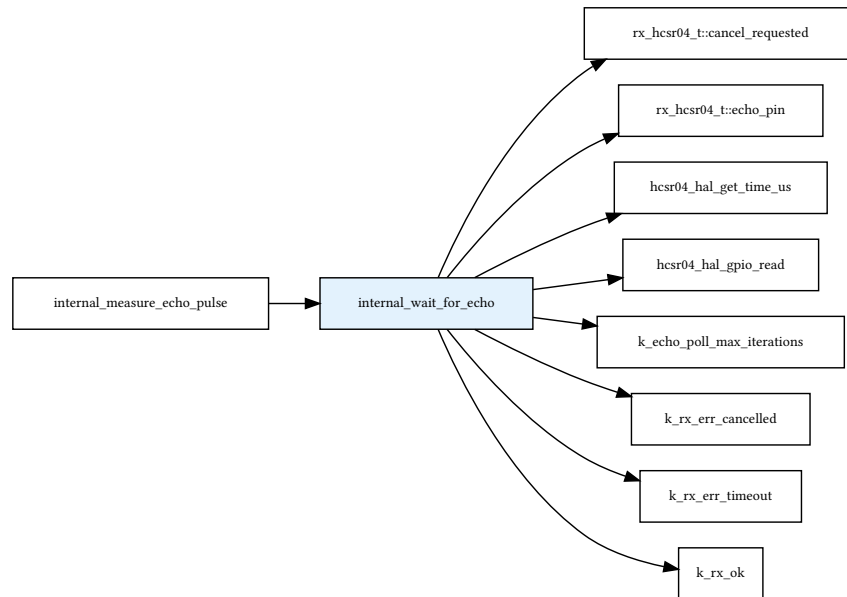


Figure 952: Call graph for internal_wait_for_echo

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1187`

Since Version 1.0.0

—

internal_measure_echo_pulse

```
static rx_err_t internal_measure_echo_pulse(rx_hcsr04_t *handle, uint32_t
*duration_us)
```

Measure HC-SR04 echo pulse width with microsecond precision.

Captures rising and falling edges of echo pulse to compute duration. Echo pulse width is proportional to object distance (58us per cm roundtrip).

Measurement Sequence:

Algorithm Steps:

1. Wait for echo pin to go HIGH (pulse start) with timeout
2. Capture timestamp (pulse_start)
3. Wait for echo pin to go LOW (pulse end) with timeout
4. Capture timestamp (pulse_end)
5. Calculate duration = pulse_end - pulse_start

Timing Examples:

Error Conditions:

- No rising edge: Object too far (>400cm) or sensor fault
- No falling edge: Sensor stuck HIGH (hardware fault)
- Duration too short: Object inside dead zone (<2cm)
- Duration too long: Echo timeout (>30ms)

duration_us in range [0, timeout_us]

Does NOT validate distance range (caller's responsibility)

Parameters:

Direction	Name	Description
inout	handle	Sensor handle (supports cancellation)
out	duration_us	Measured pulse width in microseconds

Returns: k_rx_err_hw_operation_failed GPIO read failed

Pre: handle != nullptr

Pre: duration_us != nullptr

Pre: handle->echo_pin configured as input

Pre: Echo pin initially LOW (idle state)

Post: *duration_us contains measured pulse width (if k_rx_ok)

Post: Echo pin returned to LOW state (pulse complete)

Note: Not thread-safe - concurrent measurements corrupt timing

Warning: Blocks caller until pulse completes or timeout (1-30ms typical)

Performance:: Measurement time: 1-30ms (depends on object distance) Timing resolution: 133ns (CMT2 timer on RX72N @ 7.5 MHz) CPU usage: 100% during pulse (busy-wait polling)

Example Usage:: uint32_tpulse_us=0;
 rx_err_t err=internal_measure_echo_pulse(&sensor,&pulse_us);
 if(err==k_rx_ok){ float distance_cm=(float)pulse_us/58.0f; printf("Distance: %.1fcm(pulse: %luus)\n", distance_cm, pulse_us); }elseif(err==k_rx_err_timeout)
 { printf("No object detected (>400cm or no echo)\n"); }

See also: internal_wait_for_echo Polling function for edge detection,
 hcsr04_hal_get_time_us HAL microsecond timer, rx_hcsr04_echo_to_cm Distance conversion function

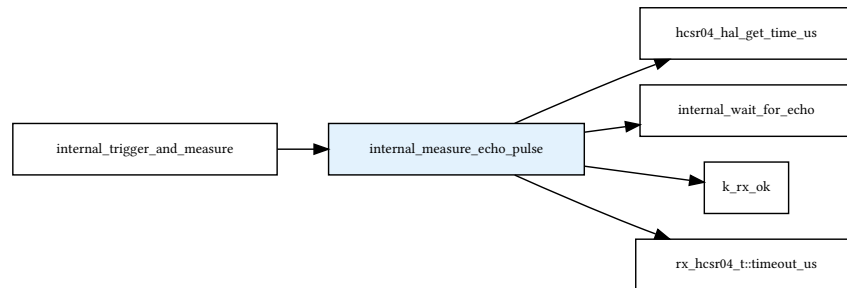


Figure 953: Call graph for internal_measure_echo_pulse

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1300`

Since Version 1.0.0

internal_measure_echo_pulse_irq

```
static rx_err_t internal_measure_echo_pulse_irq(rx_hcsr04_t *handle, uint32_t
*duration_us)
```

Measure echo pulse duration using IRQ (hardware edge capture) mode.

Waits for an ISR-driven echo pulse measurement to complete. The ISR (INT_IRQ8-11) captures rising and falling edge timestamps automatically with microsecond precision via the ICU. This function polls ISR completion state in a bounded loop, yielding the CPU each iteration via `tx_thread_sleep(k_irq_yield_ticks)` to reduce busy-wait overhead.

Algorithm:

1. Record start time for timeout tracking
2. Poll `rx_hcsr04_isr_get_duration()` in bounded loop (max `k_irq_poll_max_iterations`)
3. Each iteration: on `k_rx_ok` return duration; on `k_rx_err_timeout` continue polling; on any other error propagate immediately
4. Check cancellation and timeout each iteration
5. Yield CPU via `tx_thread_sleep(k_irq_yield_ticks)`

Parameters:

Direction	Name	Description
in	handle	Sensor handle configured for IRQ mode (must not be NULL)
out	duration_us	Pointer to store echo pulse duration in microseconds (must not be NULL; valid range 150-8700 us for 2-150 cm in IRQ mode)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Measurement complete, duration written to <code>*duration_us</code>
<code>k_rx_err_null_ptr</code>	handle or duration_us is NULL
<code>k_rx_err_timeout</code>	No echo captured within handle->timeout_us

k_rx_err_cancelled	Measurement cancelled via handle->cancel_requested
k_rx_err_invalid_arg	Propagated from rx_hcsr04_isr_get_duration() (invalid IRQ)

Pre: handle must be initialized with echo_mode == k_hcsr04_echo_irq

Pre: ICU must be configured (done during rx_hcsr04_init())

Pre: rx_hcsr04_isr_start() must have been called by the caller BEFORE sending the trigger pulse (arming first prevents missing the rising echo edge)

Post: *duration_us contains echo pulse width on k_rx_ok

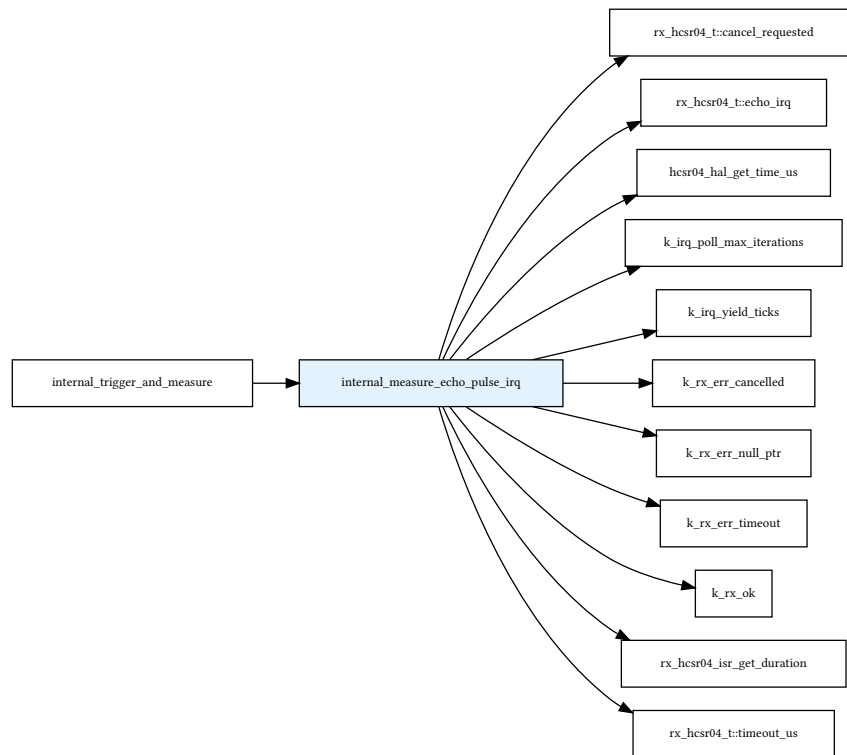
Post: ISR state machine is idle on return (active=false)

Note: Not thread-safe; must not be called from ISR context (uses tx_thread_sleep)

Note: Safe only when caller coordinates ISR/trigger sequencing

Note: Bounded loop: max k_irq_poll_max_iterations iterations] **Example: Caller Sequence:**
 rx_hcsr04_isr_start((uint8_t)handle->echo_irq); internal_send_trigger_pulse(handle);
 uint32_t duration_us; rx_err_t err = internal_measure_echo_pulse_irq(handle, &duration_us);

See also: rx_hcsr04_isr_start() Must be called by caller before trigger pulse,
 rx_hcsr04_isr_get_duration() Reads ISR-captured timestamps, tx_thread_sleep() ThreadX
 yield called each iteration, k_irq_poll_max_iterations Bounded loop iteration cap

Figure 954: Call graph for `internal_measure_echo_pulse_irq`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1381`

Since Version 1.2.0 (Issue 296 - IRQ-based HC-SR04 measurement)

hcsr04_worker_entry

```
static void hcsr04_worker_entry(const ULONG input)
```

HC-SR04 async worker thread main loop.

Dedicated thread for processing asynchronous HC-SR04 measurement requests. Waits on event flags for measurement or shutdown requests, processes measurements serially (FIFO order), and invokes callbacks from worker context.

Thread Lifecycle:

Event Processing State Machine:

Measurement Flow:

Error Handling:

- Measurement errors (timeout, out-of-range): Passed to callback via `result.status`
- ThreadX errors (event flag get): Logged, loop continues
- Callback exceptions: Not handled (callbacks must be robust)

Synchronization:

- `s_pending_mutex`: Protects access to `s_pending` context
- `s_measurement_request`: Event flags for signaling
- `s_worker_shutdown_sem`: Semaphore for shutdown sync

Performance:

- Idle power: Near-zero CPU (blocked on event flag)
- Active time: 1-30ms per measurement (same as blocking mode)
- Context switch overhead: 10us (wake + dispatch)

Infinite loop with bounded iterations per measurement (complies with NASA Rule 2)

Parameters:

Direction	Name	Description
in	input	ThreadX thread input parameter (unused, reserved for future use)

Returns: void (never returns normally, exits via break on shutdown)

Pre: Worker thread created by `rx_hcsr04_worker_init()`

Pre: `s_pending_mutex`, `s_measurement_request`, `s_worker_shutdown_sem` initialized

Post: Thread exits cleanly on shutdown (no resource leaks)

Post: `s_worker_shutdown_sem` signaled before exit

Note: Runs at priority 10 (medium) - below motor control, above telemetry

Note: Stack size: 1024 bytes (verified sufficient via `TX_ENABLE_STACK_CHECKING`)

Warning: Do NOT call directly - managed by ThreadX scheduler

Warning: Callbacks invoked from worker thread context (not caller's thread)

Thread Safety:: Safe: Multiple sensors can queue requests (FIFO processing) Unsafe: Concurrent shutdown (caller must ensure single deinit call)

Shutdown Sequence:: `rx_hcsr04_worker_deinit()` sets shutdown event flag Worker detects flag, breaks from loop Worker signals `s_worker_shutdown_sem` `deinit()` waits on semaphore, then deletes thread

Example Callback:: `voidmy_callback(rx_hcsr04_t*sensor, constrx_hcsr04_result_t*result, void*user_data) { if(result->status==k_rx_ok){ printf("[Worker]Distance:%.1fcmn",result->distance_cm); } }`

See also: `rx_hcsr04_worker_init` Worker thread creation, `rx_hcsr04_worker_deinit` Graceful shutdown, `rx_hcsr04_measure_async` Async measurement API, `s_pending` Shared measurement context

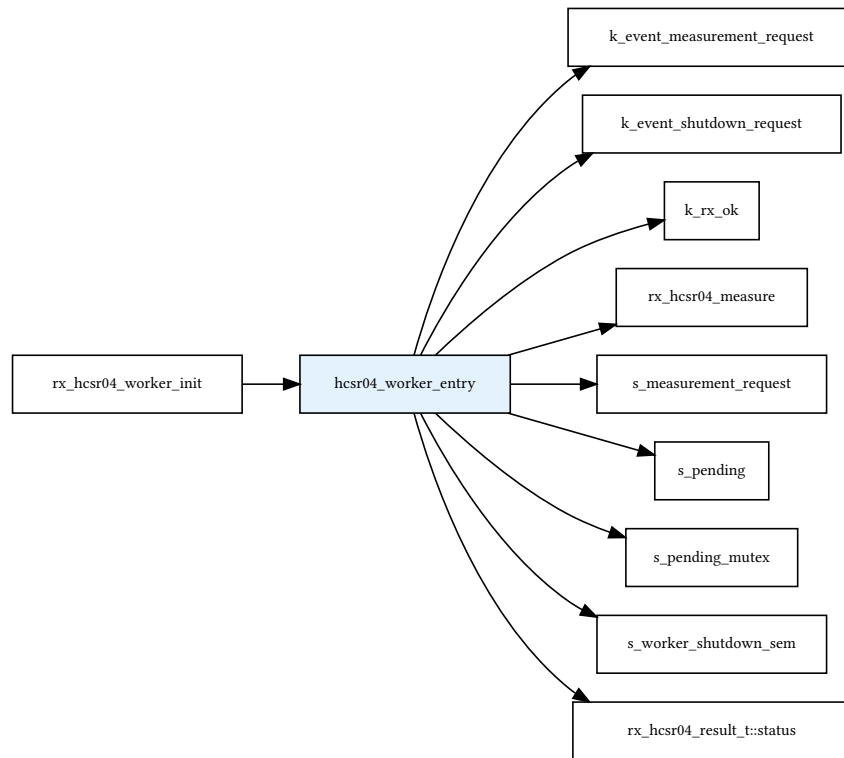


Figure 955: Call graph for hcsr04_worker_entry

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1546`

Since Version 1.0.0

—

rx_hcsr04_worker_init

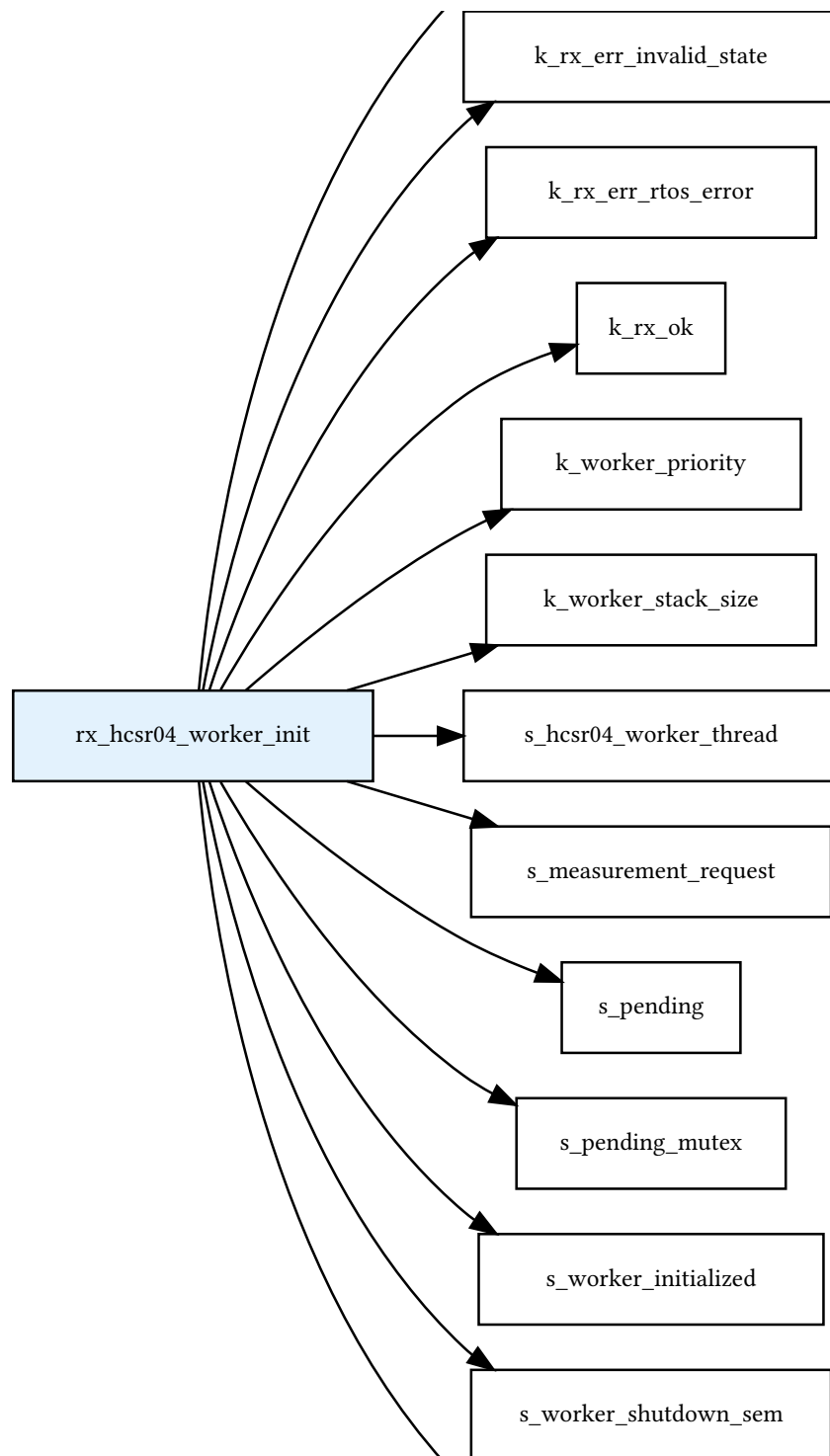
```
rx_err_t rx_hcsr04_worker_init(void)
```

Initialize the HC-SR04 worker thread for async measurements.

Initialize HC-SR04 async worker thread.

Creates the worker thread, mutex, event flags, and semaphore needed for asynchronous measurement operations. Must be called before using `rx_hcsr04_measure_async()`.

Returns: `k_rx_err_rtos_error` if ThreadX resource creation fails

Figure 956: Call graph for `rx_hcsr04_worker_init`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1608`

—

rx_hcsr04_worker_deinit

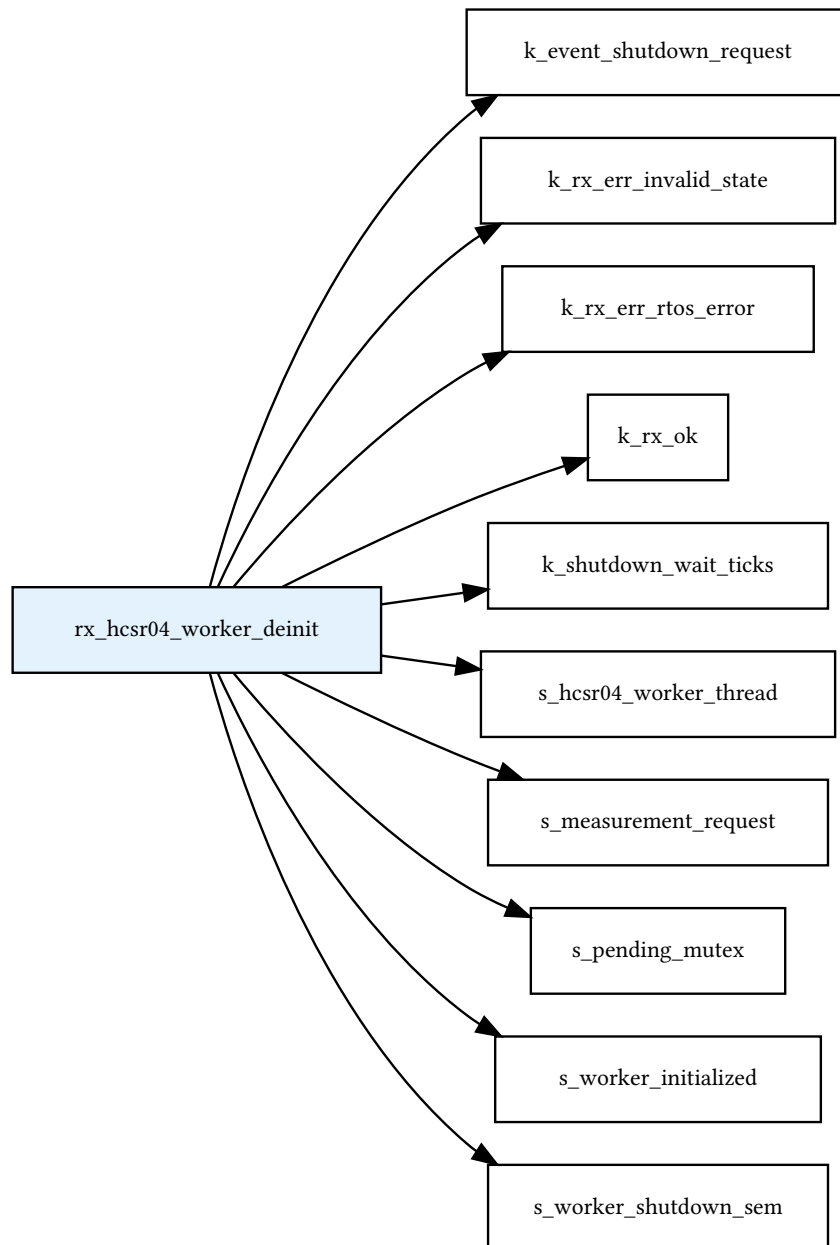
```
rx_err_t rx_hcsr04_worker_deinit(void)
```

Deinitialize the HC-SR04 worker thread.

Deinitialize HC-SR04 async worker thread.

Gracefully shuts down the worker thread and releases all ThreadX resources. Waits for the worker thread to complete any in-progress measurement before terminating.

Returns: k_rx_err_rtos_error if ThreadX cleanup fails

Figure 957: Call graph for `rx_hcsr04_worker_deinit`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1676`

internal_init_irq_mode

```
static rx_err_t internal_init_irq_mode(const rx_hcsr04_config_t *config, uint8_t
*out_priority)
```

Configure IRQ mode for echo pin (MPC + ICU + ISR registration).

Extracted from rx_hcsr04_init() to keep that function under 60 lines (NASA Power of 10 Rule 4). Performs all IRQ-specific configuration: validates the IRQ number range, checks the echo_pin/echo_irq hardware mapping, configures MPC ISEL, sets up ICU edge detection, and registers the sensor with the ISR handler layer.

On any failure, this function reverses its own partial changes (MPC reset, ICU disable) before returning. The caller (rx_hcsr04_init) is responsible for releasing the trigger pin if this function returns an error.

Parameters:

Direction	Name	Description
in	config	Sensor configuration (echo_pin, echo_irq, irq_priority)
out	out_priority	Effective ICU priority used (caller stores in handle)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All IRQ configuration applied successfully
k_rx_err_null_ptr	config or out_priority is NULL
k_rx_err_invalid_arg	echo_irq out of range, echo_pin/irq mismatch, or sensor_index >= k_hcsr04_sensor_count
Error	from rx_mpc_set_irq(), rx_hcsr04_icu_configure(), or rx_hcsr04_isr_register()

Pre: config->echo_mode == k_hcsr04_echo_irq

Pre: Trigger pin already configured as output by caller

Pre: config->sensor_index < k_hcsr04_sensor_count

Post: On success: echo pin in IRQ mode, ICU armed, ISR registered with sensor_index

Post: On failure: any partial changes reversed (MPC, ICU)

Note: Not thread-safe; call only during single-threaded initialization. Must not be called from ISR context (uses rx_log_error which may block).] **Example:** Called from rx_hcsr04_init():
uint8_t effective_priority=k_hcsr04_irq_priority_unset;

```
rx_err_t err = internal_init_irq_mode(config, &effective_priority); if (err != k_rx_ok) { return err; }  
handle->irq_priority = effective_priority;
```

See also: `rx_hcsr04_init()` Sole caller handles trigger pin cleanup on error

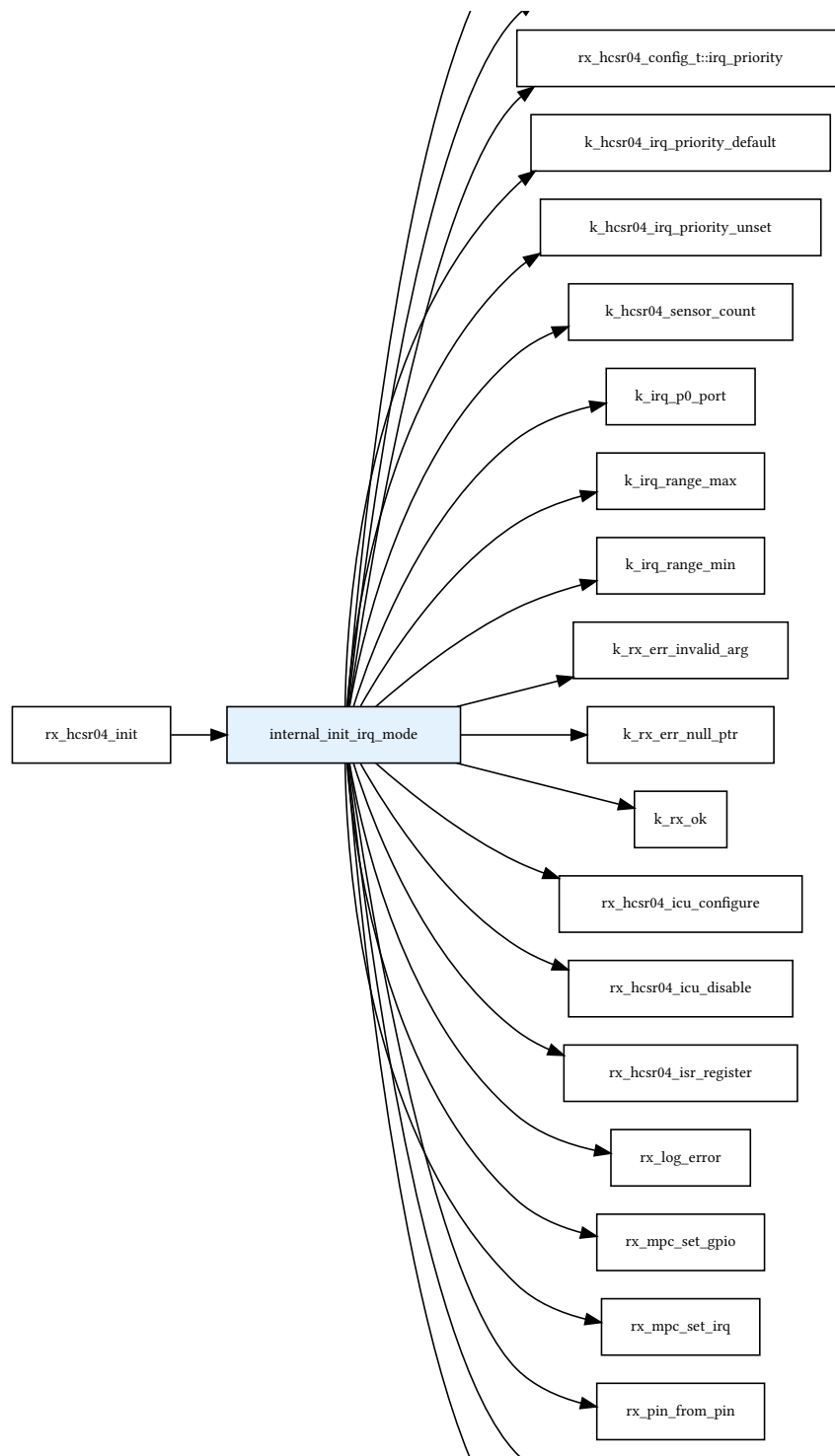


Figure 958: Call graph for internal_init_irq_mode

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1782`

_Since Version 1.3.0 (Issue 336 - sensor_index validated and forwarded to ISR)_

—

rx_hcsr04_init

```
rx_err_t rx_hcsr04_init(rx_hcsr04_t *handle, const rx_hcsr04_config_t *config)
```

Initialize an HC-SR04 sensor handle.

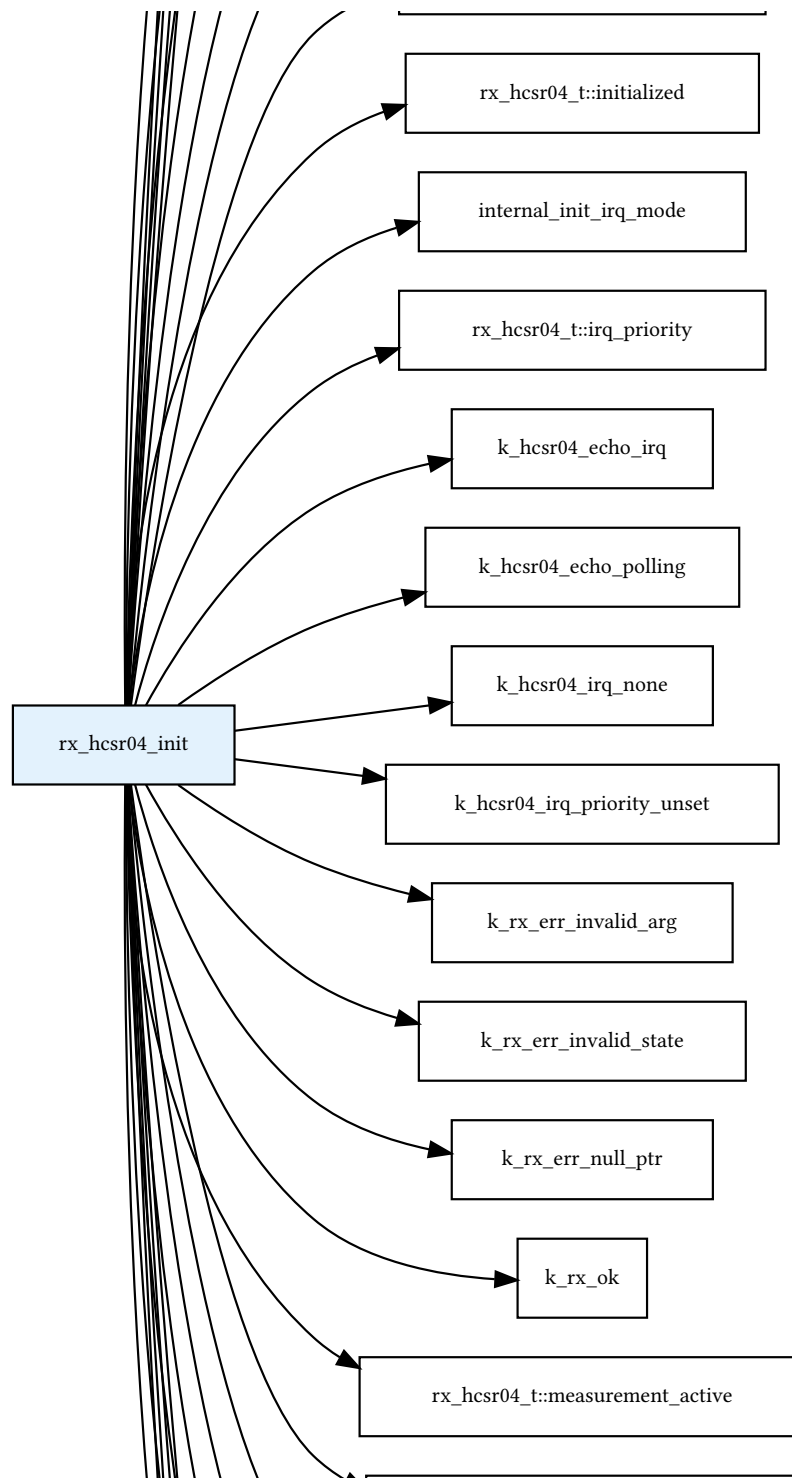
Initialize HC-SR04 sensor.

Configures GPIO pins for trigger and echo, initializes the sensor handle with default settings, and verifies GPIO configuration.

Parameters:

Direction	Name	Description
out	handle	Sensor handle to initialize
in	config	Configuration parameters (pins, timeout)

Returns: Error code from HAL GPIO operations

Figure 959: Call graph for `rx_hcsr04_init`

Defined in libs/rx_hcsr04/src/rx_hcsr04.c:1852

—

rx_hcsr04_deinit

```
rx_err_t rx_hcsr04_deinit(rx_hcsr04_t *handle)
```

Deinitialize an HC-SR04 sensor handle.

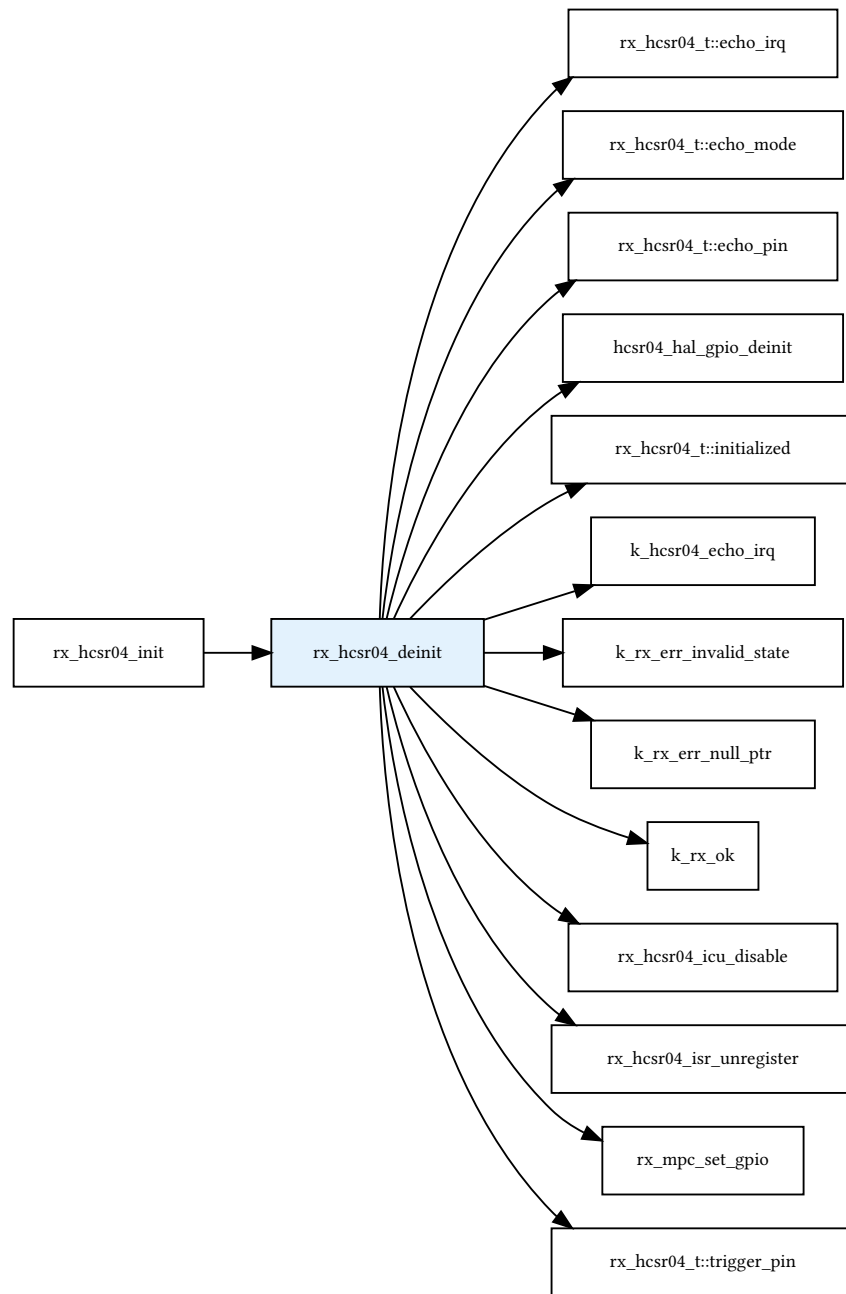
Deinitialize HC-SR04 sensor.

Releases GPIO pins and clears the sensor handle state.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle to deinitialize

Returns: k_rx_err_invalid_state if not initialized

Figure 960: Call graph for `rx_hcsr04_deinit`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:1932`

internal_trigger_and_measure

```
static rx_err_t internal_trigger_and_measure(rx_hcsr04_t *handle, uint32_t
*out_echo_us)
```

Arm ISR, send trigger pulse, and measure echo (common helper).

Consolidates the shared arm-ISR/trigger/dispatch sequence used by both `rx_hcsr04_measure_blocking()` and `rx_hcsr04_measure()`. In IRQ mode, arms the ISR before the trigger pulse and disarms on trigger failure. Dispatches to the appropriate echo measurement function based on `handle->echo_mode`.

Parameters:

Direction	Name	Description
inout	handle	Initialized sensor handle
out	out_echo_us	Measured echo pulse duration in microseconds

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Measurement complete, *out_echo_us written
k_rx_err_null_ptr	handle or out_echo_us is NULL
k_rx_err_invalid_arg	Propagated from rx_hcsr04_isr_start() (bad IRQ number)
k_rx_err_invalid_state	Propagated when ISR state is inconsistent (echo_mode/ISR mismatch)
k_rx_err_timeout	No echo within timeout
k_rx_err_cancelled	Measurement cancelled
k_rx_err_hw_operation_failed	Trigger pulse GPIO failure

Pre: handle->initialized == true

Pre: handle->echo_mode set to polling or IRQ

Post: *out_echo_us contains echo pulse width on k_rx_ok

Post: ISR disarmed on trigger failure (IRQ mode)

Note: Not thread-safe; caller must synchronize

Example:: uint32_t echo_us; rx_err_t err = internal_trigger_and_measure(handle, &echo_us); if (err != k_rx_ok) { return err; } float distance_cm = rx_hcsr04_echo_to_cm(echo_us);

See also: rx_hcsr04_measure_blocking() Primary caller, rx_hcsr04_measure() Primary caller

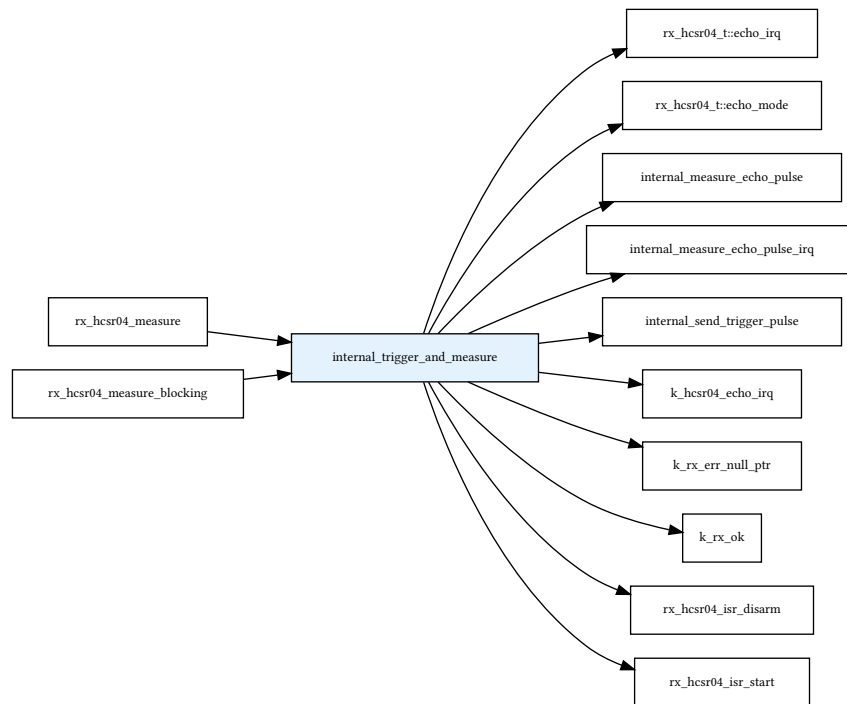


Figure 961: Call graph for internal_trigger_and_measure

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2026`

Since Version 1.2.0

—

rx_hcsr04_measure_blocking

```
rx_err_t rx_hcsr04_measure_blocking(rx_hcsr04_t *handle, float *distance_cm)
```

Perform a blocking distance measurement.

Measure distance (blocking).

Triggers the sensor, waits for echo pulse, and converts to distance in centimeters. Blocks until measurement completes or times out.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
out	distance_cm	Measured distance in centimeters

Returns: `k_rx_err_out_of_range` if distance outside sensor range (2-400 cm)

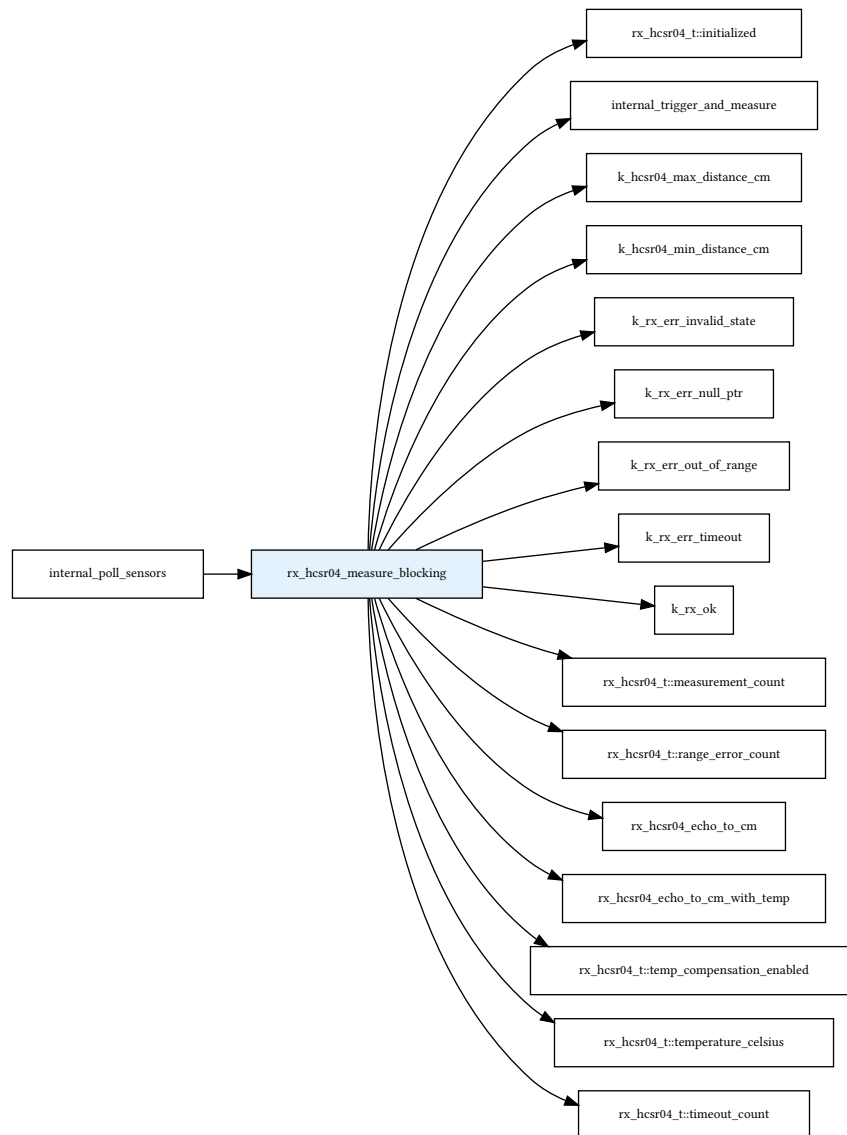


Figure 962: Call graph for rx_hcsr04_measure_blocking

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2077`

rx_hcsr04_measure

```
rx_err_t rx_hcsr04_measure(rx_hcsr04_t *handle, rx_hcsr04_result_t *result)
```

Perform a blocking distance measurement with full result data.

Measure distance with full result (blocking).

Triggers the sensor, measures echo pulse, and populates result structure with distance in both cm and inches, echo time, and status code.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
out	result	Measurement result structure

Returns: `k_rx_err_out_of_range` if distance outside sensor range (2-400 cm)

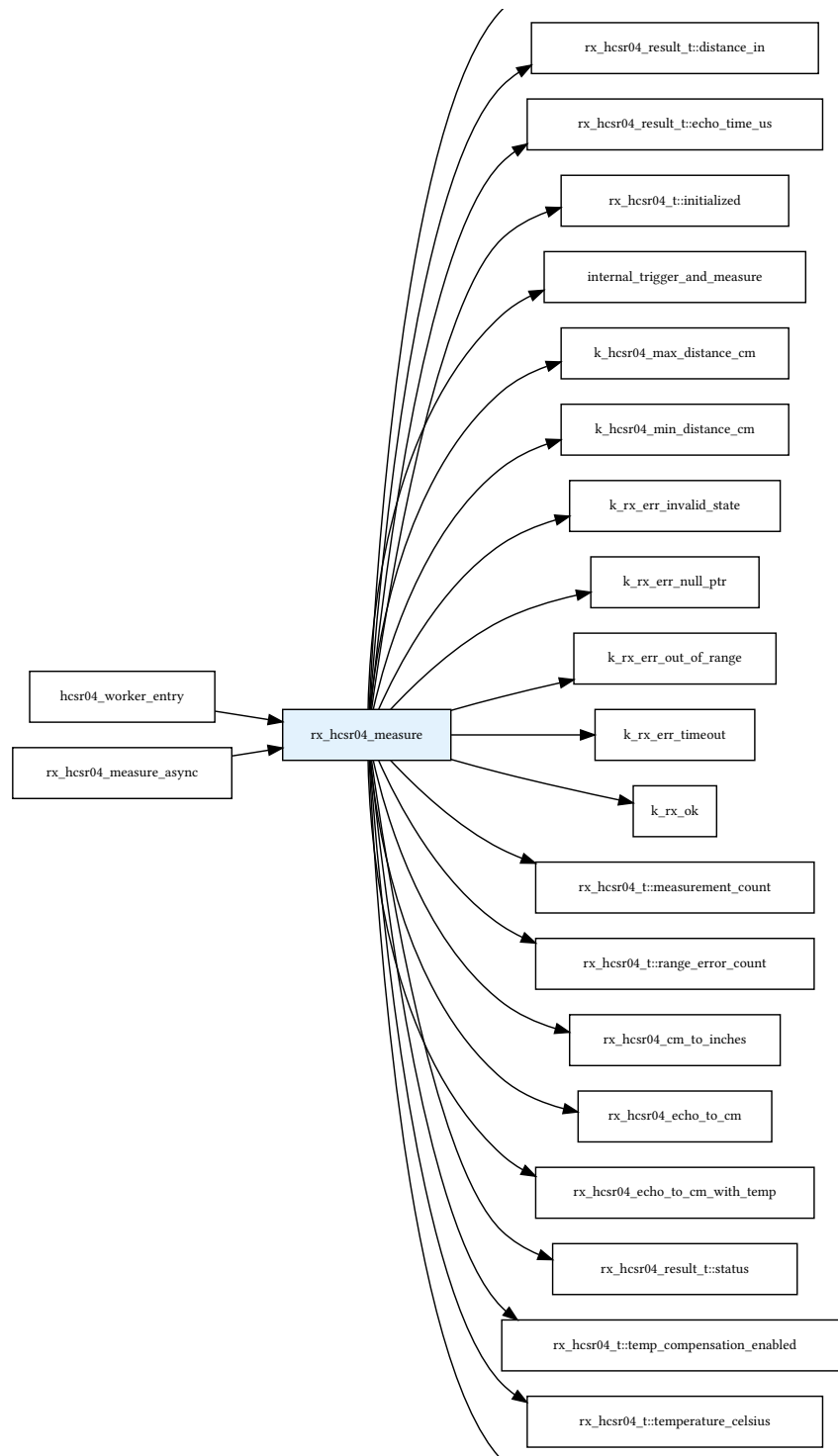


Figure 963: Call graph for rx_hcsr04_measure

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2135`

—

rx_hcsr04_measure_async

```
rx_err_t rx_hcsr04_measure_async(rx_hcsr04_t *handle, const rx_hcsr04_callback_t
callback, void *user_data)
```

Start asynchronous measurement.

Initiates measurement and invokes callback with result.

Operating Modes:

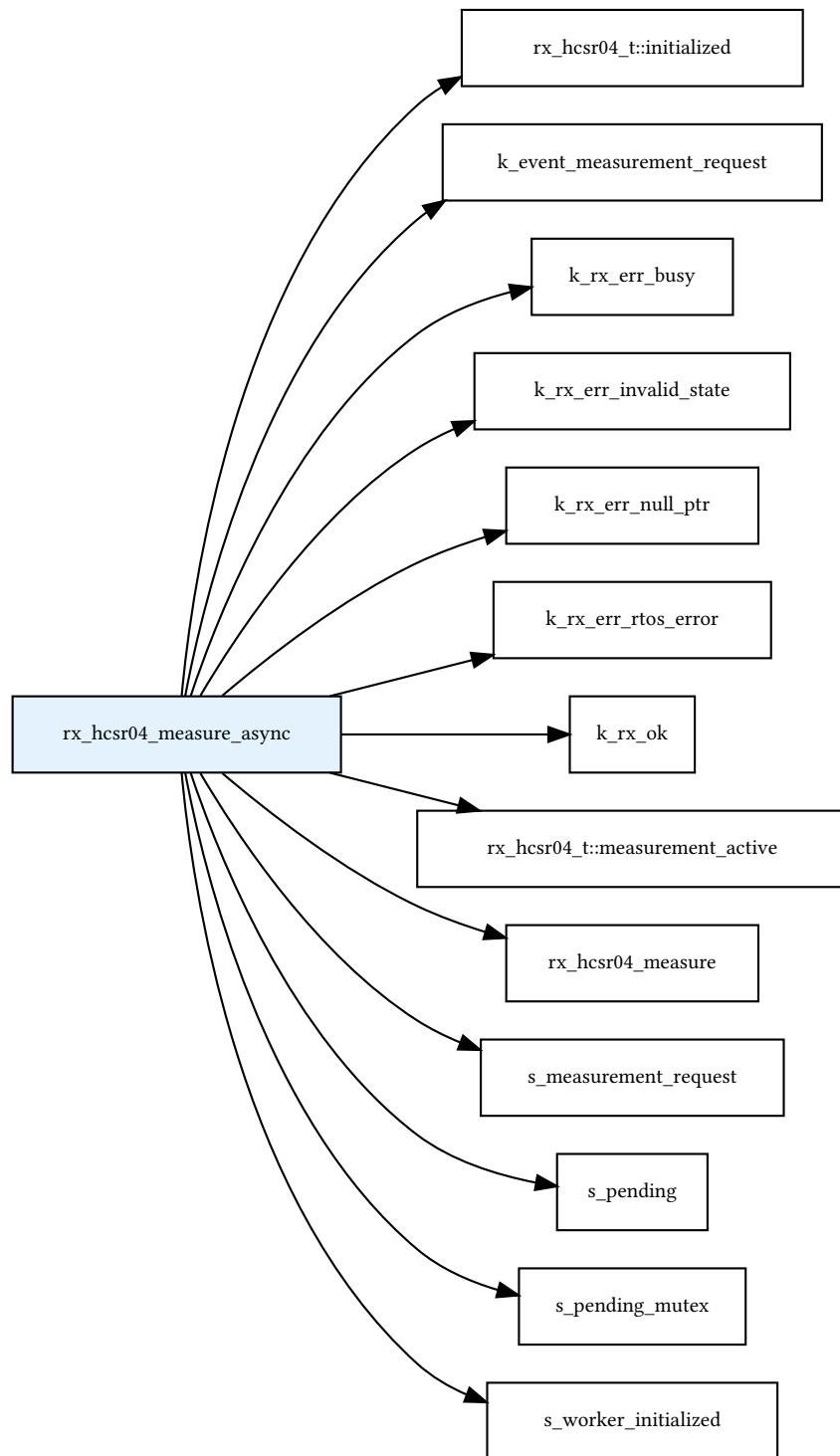
- Worker initialized (`rx_hcsr04_worker_init()` called): Returns immediately, callback invoked from worker thread when complete. True non-blocking operation.
- Worker not initialized: Performs synchronous measurement, callback invoked before return. Caller blocks for up to `timeout_us` (default 30ms).

Parameters:

Direction	Name	Description
in	handle	Sensor handle
in	callback	Completion callback (required)
in	user_data	User context passed to callback (can be NULL)

Returns: `k_rx_err_busy` if measurement already in progress

Note: Use `rx_hcsr04_cancel()` to abort in-progress async measurements. Cancellation only works when worker thread is active.

Figure 964: Call graph for `rx_hcsr04_measure_async`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2190`

—

rx_hcsr04_is_busy

```
bool rx_hcsr04_is_busy(const rx_hcsr04_t *handle)
```

Check if a measurement is currently in progress.

Check if async measurement is in progress.

Parameters:

Direction	Name	Description
in	handle	Sensor handle

Returns: true if measurement is active, false otherwise

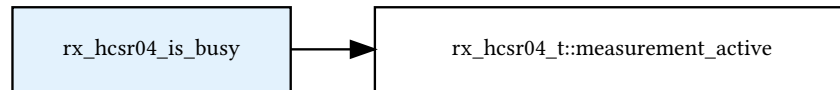


Figure 965: Call graph for rx_hcsr04_is_busy

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2265`

—

rx_hcsr04_cancel

```
rx_err_t rx_hcsr04_cancel(rx_hcsr04_t *handle)
```

Cancel an in-progress measurement.

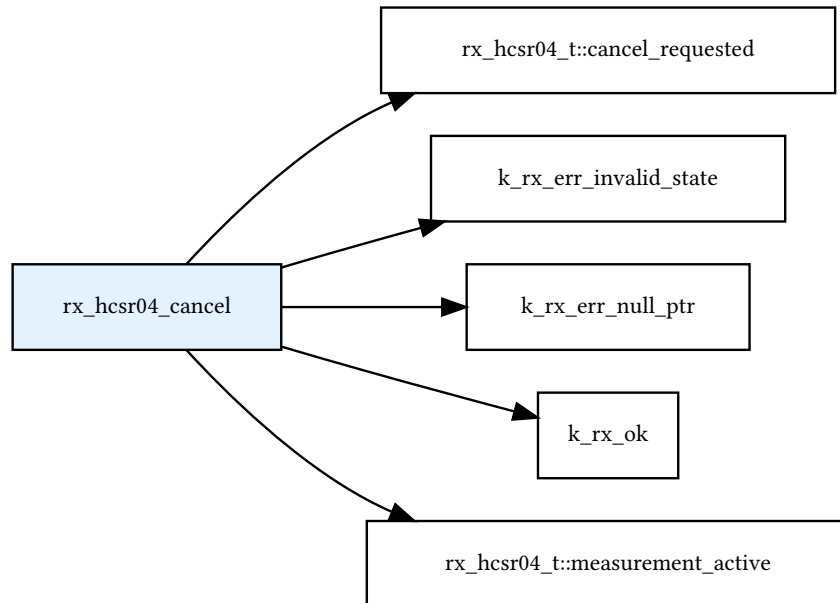
Cancel async measurement.

Sets a cancel flag that will be checked during echo pulse waiting, causing the measurement to abort with `k_rx_err_cancelled`.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle

Returns: `k_rx_err_invalid_state` if no measurement is active

Figure 966: Call graph for `rx_hcsr04_cancel`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2286`

—

rx_hcsr04_set_temperature

```
rx_err_t rx_hcsr04_set_temperature(rx_hcsr04_t *handle, const float temp_celsius)
```

Set temperature for speed of sound compensation.

Set ambient temperature for automatic distance compensation.

Enables temperature compensation and sets the ambient temperature used for calculating the speed of sound in distance conversions.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle
in	temp_celsius	Ambient temperature in degrees Celsius (-40 to +85degC)

Returns: `k_rx_err_invalid_arg` if temperature outside valid range

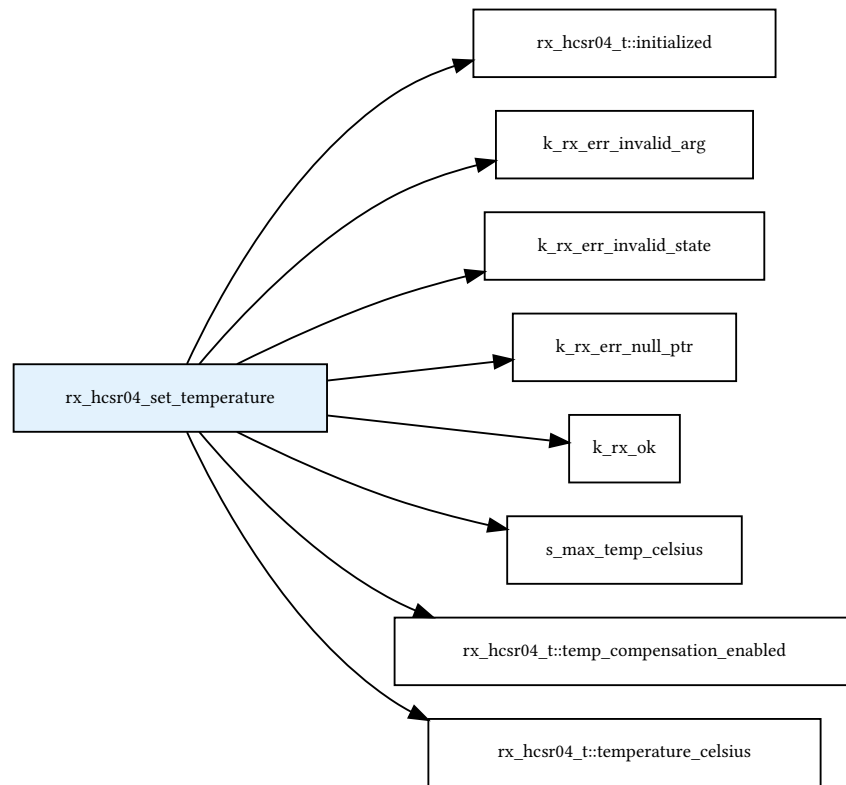


Figure 967: Call graph for rx_hcsr04_set_temperature

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2324`

—

rx_hcsr04_disable_temp_compensation

```
rx_err_t rx_hcsr04_disable_temp_compensation(rx_hcsr04_t *handle)
```

Disable temperature compensation.

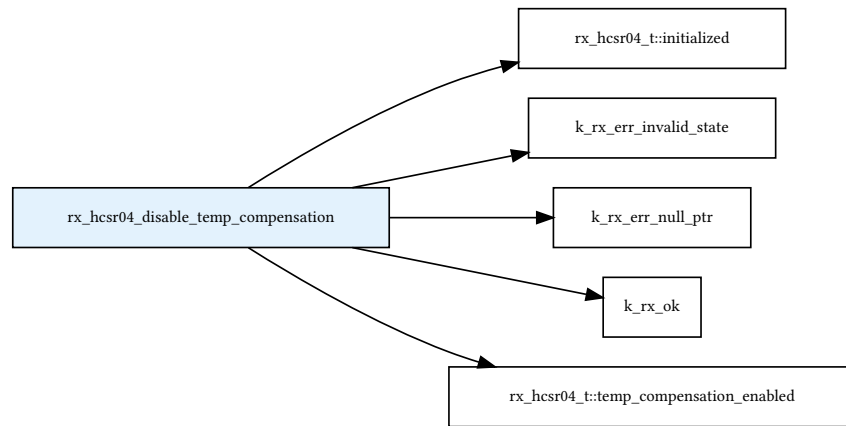
Disable automatic temperature compensation.

Disables temperature compensation and uses default 20degC speed of sound for distance calculations.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle

Returns: `k_rx_err_invalid_state` if not initialized

Figure 968: Call graph for `rx_hcsr04_disable_temp_compensation`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2358`

—

`rx_hcsr04_is_temp_compensation_enabled`

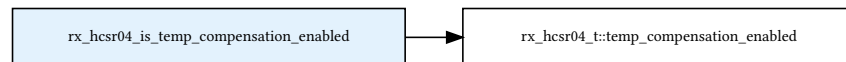
```
bool rx_hcsr04_is_temp_compensation_enabled(const rx_hcsr04_t *handle)
```

Check if temperature compensation is enabled.

Parameters:

Direction	Name	Description
in	handle	Sensor handle

Returns: true if temperature compensation is enabled, false otherwise

Figure 969: Call graph for `rx_hcsr04_is_temp_compensation_enabled`

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2380`

—

`rx_hcsr04_get_temperature`

```
rx_err_t rx_hcsr04_get_temperature(const rx_hcsr04_t *handle, float *temp_celsius)
```

Get current temperature setting.

Retrieves the temperature currently used for speed of sound compensation.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	handle	Sensor handle
out	temp_celsius	Current temperature in degrees Celsius

Returns: k_rx_err_invalid_state if not initialized

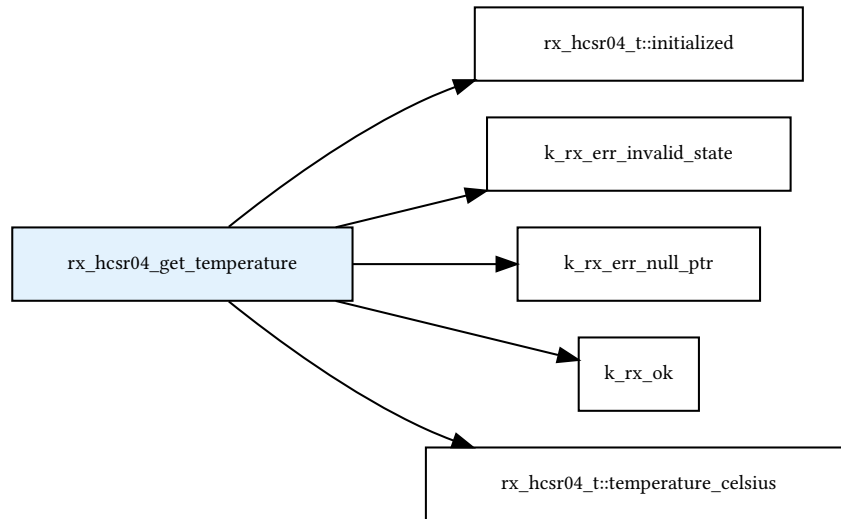


Figure 970: Call graph for rx_hcsr04_get_temperature

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2401`

rx_hcsr04_cm_to_inches

```
float rx_hcsr04_cm_to_inches(const float distance_cm)
```

Convert distance from centimeters to inches.

Parameters:

Direction	Name	Description
in	distance_cm	Distance in centimeters

Returns: Distance in inches

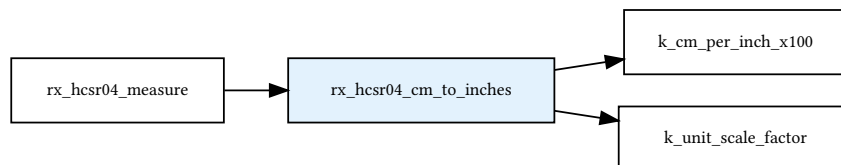


Figure 971: Call graph for rx_hcsr04_cm_to_inches

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2428`

rx_hcsr04_echo_to_cm

```
float rx_hcsr04_echo_to_cm(const uint32_t echo_time_us)
```

Convert echo time to distance in centimeters.

Uses formula: distance = (echo_us * speed_of_sound) / 2 Speed of sound at 20C = 343 m/s = 0.0343 cm/us

Parameters:

Direction	Name	Description
in	echo_time_us	Echo pulse duration in microseconds

Returns: Distance in centimeters

Note: This function assumes 20degC. For temperature compensation, use rx_hcsr04_echo_to_cm_with_temp() instead.

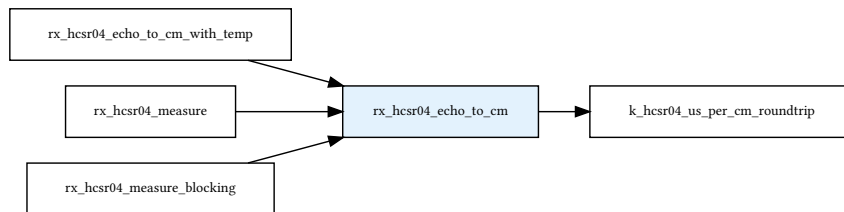


Figure 972: Call graph for rx_hcsr04_echo_to_cm

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2434`

rx_hcsr04_get_speed_of_sound

```
float rx_hcsr04_get_speed_of_sound(float temp_celsius)
```

Calculate speed of sound at given temperature.

Uses formula: $v = 331.3 + (0.606 * \text{temp}_c)$ m/s

Valid temperature range: -40degC to +85degC (DS18B20 operating range)

Parameters:

Direction	Name	Description
in	temp_celsius	Ambient temperature in degrees Celsius

Returns: Speed of sound in m/s

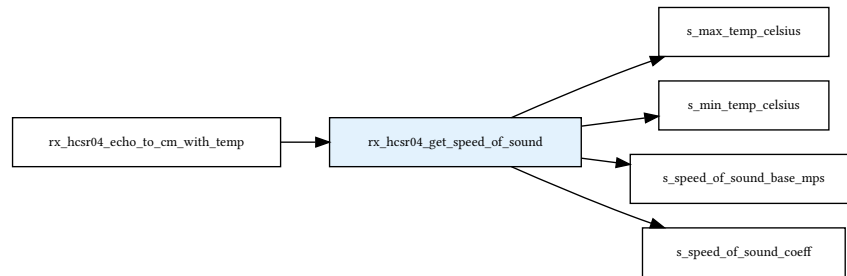


Figure 973: Call graph for rx_hcsr04_get_speed_of_sound

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2446`

rx_hcsr04_echo_to_cm_with_temp

```
float rx_hcsr04_echo_to_cm_with_temp(const uint32_t echo_time_us, float temp_celsius)
```

Convert echo time to distance with temperature compensation.

Calculates distance using temperature-compensated speed of sound.

Parameters:

Direction	Name	Description
in	echo_time_us	Echo pulse duration in microseconds
in	temp_celsius	Ambient temperature in degrees Celsius (-40 to +85degC)

Returns: Distance in centimeters, or 0.0 if input is invalid

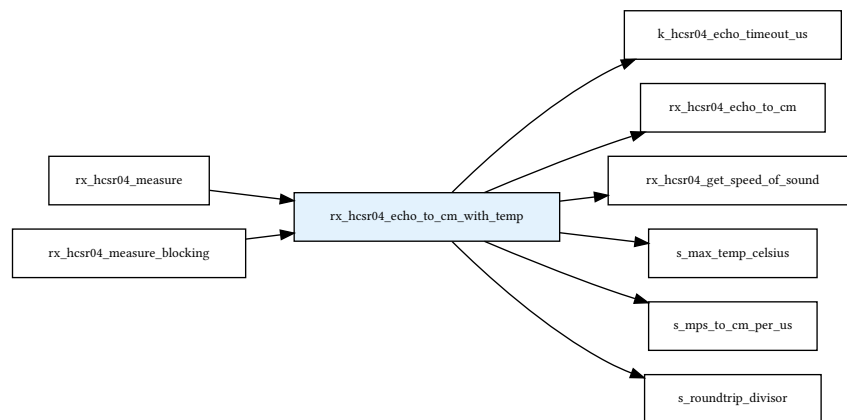


Figure 974: Call graph for rx_hcsr04_echo_to_cm_with_temp

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2475`

rx_hcsr04_get_stats

```
rx_err_t rx_hcsr04_get_stats(const rx_hcsr04_t *handle, uint32_t
*measurement_count, uint32_t *timeout_count, uint32_t *range_error_count)
```

Get sensor statistics.

Parameters:

Direction	Name	Description
in	handle	Sensor handle
out	measurement_count	Total measurements (can be NULL)
out	timeout_count	Timeout count (can be NULL)
out	range_error_count	Range error count (can be NULL)

Returns: k_rx_err_invalid_state if not initialized

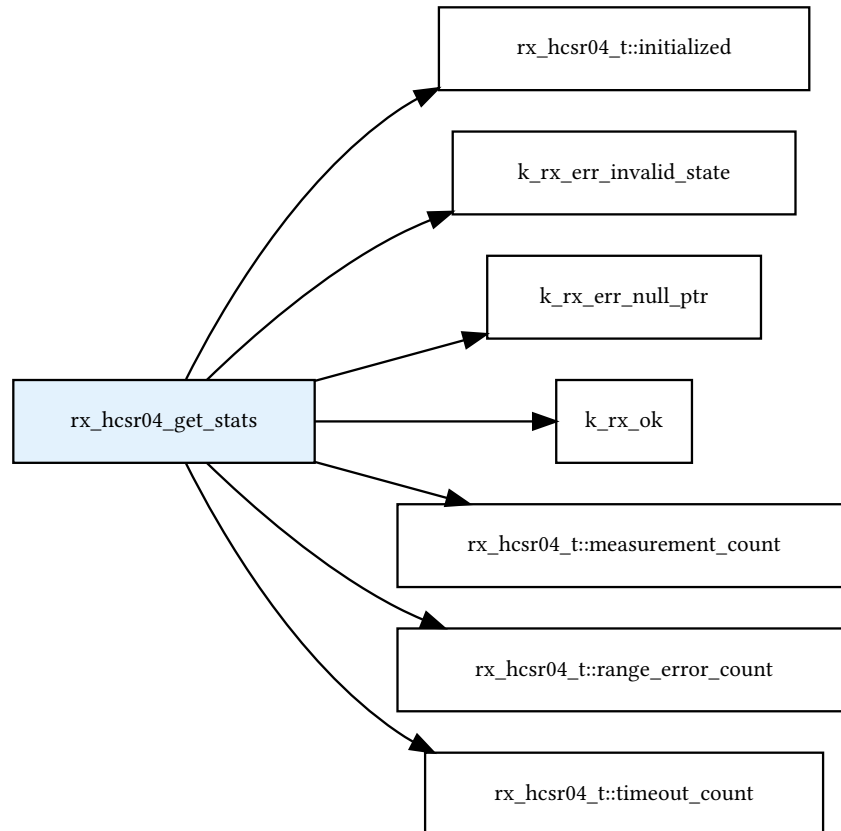


Figure 975: Call graph for rx_hcsr04_get_stats

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2510`

rx_hcsr04_reset_stats

```
rx_err_t rx_hcsr04_reset_stats(rx_hcsr04_t *handle)
```

Reset sensor statistics.

Parameters:

Direction	Name	Description
inout	handle	Sensor handle

Returns: k_rx_err_invalid_state if not initialized

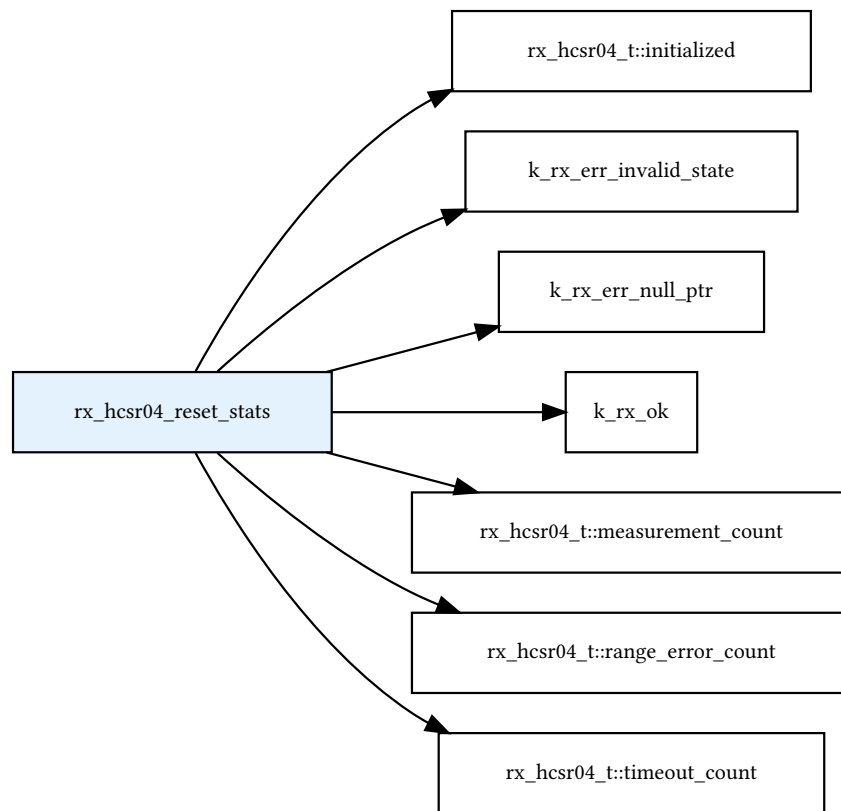


Figure 976: Call graph for rx_hcsr04_reset_stats

Defined in `libs/rx_hcsr04/src/rx_hcsr04.c:2538`

—

rx_hcsr04_hal.h

HC-SR04 Hardware Abstraction Layer (HAL) Interface.

Includes:

- `<stdbool.h>`

- <stdint.h>
- "rx_err.h"
- "rx_port_constants.h"

hcsr04_hal_gpio_set_output

```
rx_err_t hcsr04_hal_gpio_set_output(rx_port_pin_t pin)
```

Configure GPIO pin as output for HC-SR04 trigger signal.

Configures the specified GPIO pin as an output in push-pull mode for driving the HC-SR04 trigger input. On RX72N, this sets the PDR (Port Direction Register) bit to 1 for the specified pin.

Hardware Configuration (RX72N):

- PDR bit = 1 (output mode)
- Initial state: Undefined (caller should write LOW immediately)
- Drive strength: Standard CMOS (sufficient for HC-SR04 trigger)

Mock Implementation:

- Tracks pin configuration in memory
- No actual hardware access

Configure GPIO pin as output for HC-SR04 trigger signal.

Validates the port/pin combination and then configures the specified GPIO pin as a push-pull digital output via the `gpio_set_output()` hardware abstraction layer. Used to configure the HC-SR04 trigger pin before initiating ultrasonic measurement pulses.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()` (checks port and pin indices)
2. Call `gpio_set_output(pin)` to set the data direction register bit

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (type-safe enum from <code>rx_port_constants.h</code>)
in	pin	GPIO port/pin descriptor specifying the trigger GPIO; must be a valid <code>rx_port_pin_t</code> with legal port and pin values

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin successfully configured as output
<code>k_rx_err_invalid_arg</code>	pin specifies an invalid port or pin number

Pre: Pin must be a valid GPIO (not used by peripherals)

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: GPIO peripheral clock must be enabled before calling

Post: Pin configured as output, ready for write operations

Post: Pin state undefined until first write

Post: The specified GPIO pin direction register bit is set to output mode

Post: Pin output level is undefined; call `hcsr04_hal_gpio_write_low()` to initialize

Note: Call once during sensor initialization

Note: Not thread-safe; GPIO direction configuration should occur during initialization

Warning: Multiple calls safe (idempotent) but unnecessary

Example:: `rx_err_t err=hcsr04_hal_gpio_set_output(config->trigger_pin); if(err!=k_rx_ok) { rx_log_error("hcsr04","Failed to configure trigger pin as output"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, clock enabled), 2 postconditions

See also: `hcsr04_hal_gpio_write_high` Set pin HIGH, `hcsr04_hal_gpio_write_low` Set pin LOW, `rx_port_constants.h` GPIO pin definitions, `hcsr04_hal_gpio_write_high()` Drive trigger pin high to start pulse, `hcsr04_hal_gpio_write_low()` Drive trigger pin low, `hcsr04_hal_gpio_set_input()` Configure echo pin as input

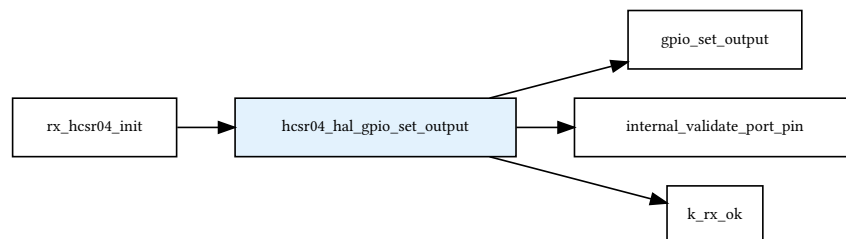


Figure 977: Call graph for `hcsr04_hal_gpio_set_output`

_Defined in `libs/rx\hcsr04/src/rx\hcsr04\_hal.h:321_`

Since Version 1.0.0

—

hcsr04_hal_gpio_set_input

```
rx_err_t hcsr04_hal_gpio_set_input(rx_port_pin_t pin)
```

Configure GPIO pin as input for HC-SR04 echo signal.

Configures the specified GPIO pin as a high-impedance input for reading the HC-SR04 echo pulse. On RX72N, this sets the PDR (Port Direction Register) bit to 0 for the specified pin.

Hardware Configuration (RX72N):

- PDR bit = 0 (input mode)
- Input buffer: Enabled (Schmitt trigger for noise immunity)
- Pull-up/down: Disabled (HC-SR04 drives the line)

Voltage Levels:

- RX72N GPIO: 3.3V logic
- HC-SR04 output: 5V logic (may need level shifter or voltage divider)
- Check RX72N datasheet for 5V tolerance on specific pins

Mock Implementation:

- Tracks pin configuration in memory
- Simulates input state for testing

Configure GPIO pin as input for HC-SR04 echo signal.

Validates the port/pin combination and then configures the specified GPIO pin as a digital input via the `gpio_set_input()` hardware abstraction layer. Used to configure the HC-SR04 echo pin so that the rising and falling edges of the echo pulse can be measured.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()` (checks port and pin indices)
2. Call `gpio_set_input(pin)` to clear the data direction register bit

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (type-safe enum from <code>rx_port_constants.h</code>)
in	pin	GPIO port/pin descriptor specifying the echo GPIO; must be a valid <code>rx_port_pin_t</code> with legal port and pin values

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin successfully configured as input
<code>k_rx_err_invalid_arg</code>	pin specifies an invalid port or pin number

Pre: Pin must be a valid GPIO (not used by peripherals)

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: GPIO peripheral clock must be enabled before calling

Post: Pin configured as input, ready for read operations

Post: Input buffer enabled, pull-up/down disabled

Post: The specified GPIO pin direction register bit is cleared to input mode

Post: Pin can now be read to detect echo pulse edges

Note: Call once during sensor initialization

Note: Not thread-safe; GPIO direction configuration should occur during initialization

Warning: Ensure HC-SR04 echo output compatible with RX72N input levels

Example:: `rx_err_t rx_hcsr04_hal_gpio_set_input(config->echo_pin); if(err!=k_rx_ok)`
`{ rx_log_error("hcsr04","Failed to configure echo pin as input"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, clock enabled), 2 postconditions

See also: `hcsr04_hal_gpio_read` Read pin state, `rx_port_constants.h` GPIO pin definitions, `hcsr04_hal_gpio_set_output()` Configure trigger pin as output, `hcsr04_hal_gpio_write_high()` Drive trigger pin high

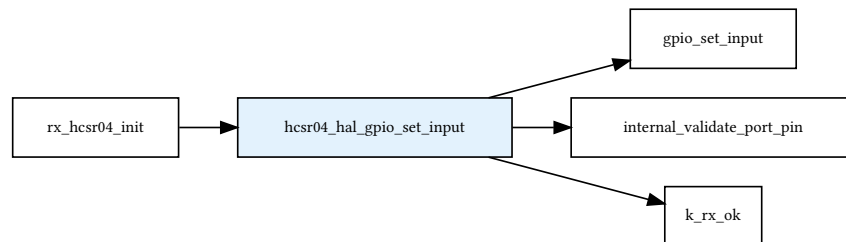


Figure 978: Call graph for `hcsr04_hal_gpio_set_input`

_Defined in `libs/rx\hcsr04/src/rx\hcsr04\_hal.h:363_`

Since Version 1.0.0

—

hcsr04_hal_gpio_write_high

```
rx_err_t hcsr04_hal_gpio_write_high(rx_port_pin_t pin)
```

Set GPIO pin to HIGH (3.3V) level.

Drives the GPIO pin to logic HIGH (VCC level, typically 3.3V on RX72N). Used to generate the rising edge of the HC-SR04 trigger pulse.

Hardware Operation (RX72N):

- Sets PODR (Port Output Data Register) bit to 1
- Output voltage: VCC (3.3V +/-10%)
- Rise time: 5 ns (typical for RX72N GPIO)
- Drive current: +/-4 mA (standard drive strength)

Timing Constraints:

- HC-SR04 requires >=10us HIGH pulse
- This function executes in 200 ns

- Caller must add delay between `write_high()` and `write_low()`

Mock Implementation:

- Sets simulated pin state to true
- Can trigger simulated echo response

Set GPIO pin to HIGH (3.3V) level.

Validates the port/pin combination and drives the specified output GPIO pin to a logic high (VDD) level via the `gpio_write_high()` hardware abstraction layer. In HC-SR04 operation, this is used to begin the 10 us trigger pulse that initiates an ultrasonic ranging measurement.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()`
2. Call `gpio_write_high(pin)` to set the output data register bit

Parameters:

Direction	Name	Description
in	<code>pin</code>	GPIO pin identifier (must be configured as output)
in	<code>pin</code>	GPIO port/pin descriptor specifying the output pin to drive high; must be a valid <code>rx_port_pin_t</code> already configured as output

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin output register bit successfully set high
<code>k_rx_err_invalid_arg</code>	<code>pin</code> specifies an invalid port or pin number

Pre: Pin configured as output via `hcsr04_hal_gpio_set_output()`

Pre: `pin` must encode a valid port/pin combination supported by the RX72N

Pre: Pin must have been configured as output via `hcsr04_hal_gpio_set_output()`

Post: Pin driven to HIGH level (VCC)

Post: HC-SR04 trigger input sees rising edge

Post: The GPIO pin output data register bit is set; pin drives logic high

Post: Follow with `hcsr04_hal_delay_us(10)` then `hcsr04_hal_gpio_write_low()` for trigger pulse

Note: Execution time: 200 ns on RX72N @ 240 MHz

Note: Not thread-safe; concurrent GPIO writes to the same pin produce undefined behavior

Warning: Pin must be output mode (undefined behavior if input)

Example:: `hcsr04_hal_gpio_write_high(trigger_pin); hcsr04_hal_delay_us(10);`
`hcsr04_hal_gpio_write_low(trigger_pin);`

Example:: `hcsr04_hal_gpio_write_high(config->trigger_pin); hcsr04_hal_delay_us(10);`
`hcsr04_hal_gpio_write_low(config->trigger_pin);`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, configured as output), 2 postconditions

See also: `hcsr04_hal_gpio_write_low` Set pin LOW, `hcsr04_hal_gpio_set_output` Configure pin as output, `hcsr04_hal_gpio_write_low()` Drive pin low to end trigger pulse, `hcsr04_hal_gpio_set_output()` Configure pin as output first, `hcsr04_hal_delay_us()` Delay for trigger pulse duration

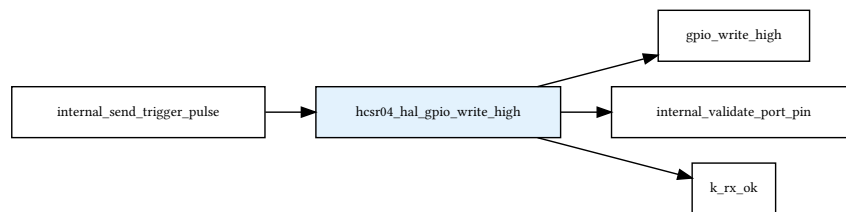


Figure 979: Call graph for `hcsr04_hal_gpio_write_high`

_Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_hal.h:413_`

Since Version 1.0.0

—

hcsr04_hal_gpio_write_low

```
rx_err_t hcsr04_hal_gpio_write_low(rx_port_pin_t pin)
```

Set GPIO pin to LOW (GND) level.

Drives the GPIO pin to logic LOW (ground level, 0V). Used to complete the HC-SR04 trigger pulse and return to idle state.

Hardware Operation (RX72N):

- Clears PODR (Port Output Data Register) bit to 0
- Output voltage: GND (0V +/-0.3V)
- Fall time: 5 ns (typical for RX72N GPIO)
- Sink current: +/-4 mA (standard drive strength)

HC-SR04 Behavior:

- Falling edge of trigger pulse ends the 10us sequence
- Sensor begins ultrasonic burst transmission
- Echo pin will go HIGH when echo starts

Mock Implementation:

- Sets simulated pin state to false
- Completes simulated trigger sequence

Set GPIO pin to LOW (GND) level.

Validates the port/pin combination and drives the specified output GPIO pin to a logic low (GND) level via the `gpio_write_low()` hardware abstraction layer. In HC-SR04 operation, this is used after a 10 us high pulse to complete the trigger signal, after which the sensor emits ultrasonic bursts and the echo pin goes high for a period proportional to measured distance.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()`
2. Call `gpio_write_low(pin)` to clear the output data register bit

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (must be configured as output)
in	pin	GPIO port/pin descriptor specifying the output pin to drive low; must be a valid <code>rx_port_pin_t</code> already configured as output

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin output register bit successfully cleared low
<code>k_rx_err_invalid_arg</code>	pin specifies an invalid port or pin number

Pre: Pin configured as output via `hcsr04_hal_gpio_set_output()`

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: Pin must have been configured as output via `hcsr04_hal_gpio_set_output()`

Post: Pin driven to LOW level (GND)

Post: HC-SR04 trigger input sees falling edge

Post: The GPIO pin output data register bit is cleared; pin drives logic low

Post: HC-SR04 trigger sequence is complete; echo pin will pulse proportional to distance

Note: Execution time: 200 ns on RX72N @ 240 MHz

Note: Not thread-safe; concurrent GPIO writes to the same pin produce undefined behavior

Warning: Pin must be output mode (undefined behavior if input)

Example:: `hcsr04_hal_gpio_set_output(trigger_pin); hcsr04_hal_gpio_write_low(trigger_pin);`

Example:: `hcsr04_hal_gpio_write_high(config->trigger_pin); hcsr04_hal_delay_us(10);
hcsr04_hal_gpio_write_low(config->trigger_pin); uint32_techo_start=hcsr04_hal_get_time_us();`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, configured as output), 2 postconditions

See also: `hcsr04_hal_gpio_write_high` Set pin HIGH, `hcsr04_hal_gpio_set_output` Configure pin as output, `hcsr04_hal_gpio_write_high()` Drive pin high to start trigger pulse, `hcsr04_hal_gpio_set_output()` Configure pin as output first, `hcsr04_hal_get_time_us()` Capture echo start timestamp after trigger

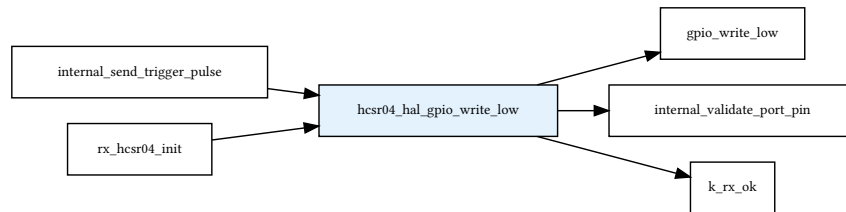


Figure 980: Call graph for `hcsr04_hal_gpio_write_low`

_Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_hal.h:462_`

Since Version 1.0.0

—

hcsr04_hal_gpio_read

```
rx_err_t hcsr04_hal_gpio_read(rx_port_pin_t pin, bool *value)
```

Read current logic level of GPIO pin.

Reads the current state of the GPIO pin's input buffer. Used to detect rising and falling edges of the HC-SR04 echo pulse for timing measurement.

Hardware Operation (RX72N):

- Reads PIDR (Port Input Data Register) bit
- Input threshold: 0.7xVCC (HIGH), 0.3xVCC (LOW)
- Schmitt trigger: 0.2V hysteresis (noise immunity)
- Sample time: 150 ns (single register read)

Return Values:

- `*value = true`: Pin at HIGH level ($\geq 2.3V$ on 3.3V system)
- `*value = false`: Pin at LOW level ($\leq 1.0V$ on 3.3V system)

Mock Implementation:

- Returns simulated pin state from memory
- Can inject specific pulse widths for testing

Typical Usage Pattern:

Read current logic level of GPIO pin.

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (must be configured as input)
out	value	Pointer to receive pin state (true=HIGH, false=LOW)
in	pin	GPIO pin to read
out	value	Pointer to receive pin state (true=high, false=low)

Returns: k_rx_err_invalid_arg if pin is invalid

Pre: Pin configured as input via hcsr04_hal_gpio_set_input()

Pre: value != nullptr

Post: *value contains current pin state (if k_rx_ok returned)

Post: Pin configuration unchanged

Note: Execution time: 150 ns on RX72N @ 240 MHz

Note: Read is non-blocking (immediate return)

Warning: Pin must be input mode for reliable readings

Warning: Floating pins may read unpredictable values

Performance:: Single register read (PIDR) No interrupt overhead Suitable for tight polling loops

Example:

```
//WaitforechotogoHIGH(pulsestart)
boolecho_state=false;
while(!echo_state){
    hcsr04_hal_gpio_read(echo_pin,&echo_state);
}
uint32_tstart_time=hcsr04_hal_get_time_us();

//WaitforechotogoLOW(pulseend)
while(echo_state){
    hcsr04_hal_gpio_read(echo_pin,&echo_state);
}
uint32_tend_time=hcsr04_hal_get_time_us();
uint32_tpulse_width=end_time-start_time;
```

See also: hcsr04_hal_gpio_set_input Configure pin as input, hcsr04_hal_get_time_us Capture timestamps

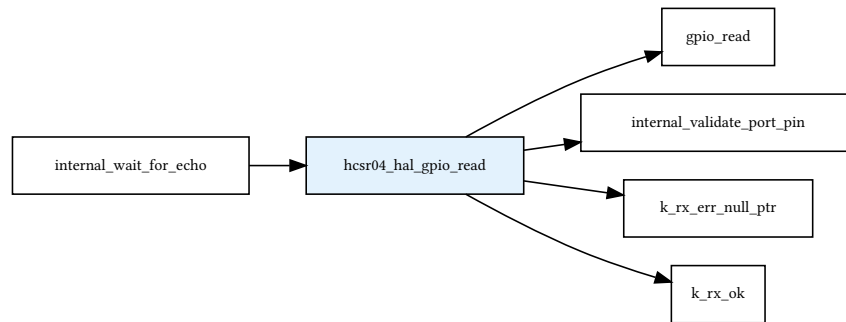


Figure 981: Call graph for hcsr04_hal_gpio_read

_Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_hal.h:530_`

Since Version 1.0.0

—

hcsr04_hal_gpio_deinit

```
rx_err_t hcsr04_hal_gpio_deinit(rx_port_pin_t pin)
```

Release GPIO pin resources (cleanup).

Optionally resets GPIO pin to a safe state or releases allocated resources. On RX72N, this is typically a no-op since GPIO pins are static resources (no dynamic allocation).

Hardware Implementation (RX72N):

- No-op: GPIO pins don't require deallocation
- Pin configuration persists until next reconfiguration
- Provided for API completeness and future platforms

Mock Implementation:

- Clears simulated pin state
- Marks pin as uninitialized for test validation

When to Call:

- During sensor deinitialization (`rx_hcsr04_deinit`)
- On error paths during init (cleanup)
- Safe to skip on RX72N (but recommended for portability)

Release GPIO pin resources (cleanup).

This is intentionally a no-op as RX72N GPIO pins are static resources.

Parameters:

Direction	Name	Description
in	<code>pin</code>	GPIO pin identifier to deinitialize
in	<code>pin</code>	GPIO pin to deinitialize

Returns: `k_rx_err_invalid_arg` if pin is invalid

Pre: Pin previously initialized via `set_output()` or `set_input()`

Post: Pin configuration undefined (platform-specific)

Post: Future read/write operations may fail or return undefined values

Note: On RX72N: Always returns `k_rx_ok` (no actual operation)

Note: On mock HAL: Resets pin state for test isolation

Warning: Do not use pin after deinit without reconfiguring

Example:: `rx_err_trx_hcsr04_deinit(rx_hcsr04_t*handle){ hcsr04_hal_gpio_deinit(handle->trigger_pin); hcsr04_hal_gpio_deinit(handle->echo_pin); handle->initialized=false; return k_rx_ok; }`

See also: `hcsr04_hal_gpio_set_output` Initialize as output, `hcsr04_hal_gpio_set_input` Initialize as input, `rx_hcsr04_deinit` Sensor cleanup function

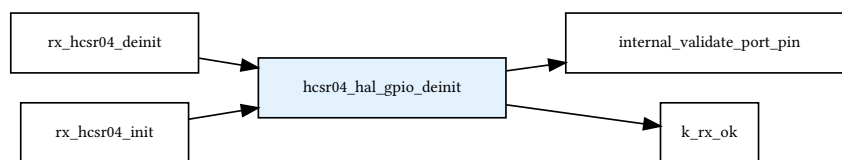


Figure 982: Call graph for `hcsr04_hal_gpio_deinit`

_Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_hal.h:584_`

Since Version 1.0.0

—

hcsr04_hal_delay_us

```
void hcsr04_hal_delay_us(uint32_t us)
```

Blocking delay for specified microseconds.

Busy-wait delay with microsecond precision. Blocks the calling thread for the specified duration. Used for HC-SR04 trigger pulse timing (10us pulse, 2us settle).

Hardware Implementation (RX72N):

- Uses CMT2 (Compare Match Timer 2)
- Clock: PCLKB 60 MHz / 8 = 7.5 MHz
- Resolution: 133 ns per tick
- Accuracy: +/-5% (clock variation + loop overhead)
- Method: Spin-wait on hardware counter

Delay Ranges:

Mock Implementation:

- Uses host OS sleep functions (nanosleep, usleep, Sleep)
- Accuracy: +/-10% (OS scheduler variance)
- Non-blocking: May yield CPU (unlike hardware version)

Performance Characteristics:

- CPU usage: 100% during delay (busy-wait, no RTOS yield)
- Power: Full active power (no sleep mode)
- Interrupts: Enabled (delay can be interrupted)

Blocking delay for specified microseconds.

Produces a blocking busy-wait delay by polling the CMT2 Compare Match Timer counter. CMT2 is automatically initialized on the first call via `internal_cmt2_init()`. For large delays (exceeding `k_timer_counter_max` ticks per iteration) the wait is split into multiple smaller segments.

CMT2 is driven from `PCLKB / k_cmt2_divider`, yielding a tick frequency of `k_pclkb_hz / k_cmt2_divider` Hz. The requested microsecond count is converted to ticks using this frequency before entering the polling loop.

Safety bounds:

- `us == 0`: returns immediately (`k_no_delay` check)
- `ticks < k_min_ticks`: clamped up to `k_min_ticks`
- `ticks > k_max_delay_ticks`: no-op in release; assertion in unit tests
- `iteration_count >= k_max_delay_iterations`: exits loop (prevents infinite spin)

Algorithm steps:

1. Return immediately if `us == k_no_delay`
2. Call `internal_cmt2_init()` to ensure CMT2 is running
3. Compute ticks from `us` and timer frequency; clamp to `[k_min_ticks, k_max_delay_ticks]`
4. Loop: poll CMT2 counter until `wait_ticks` have elapsed; decrement remaining ticks
5. Exit when `ticks == 0` or `iteration_count` reaches `k_max_delay_iterations`

Parameters:

Direction	Name	Description
in	<code>us</code>	Delay duration in microseconds (0 = no delay, max 500,000)
in	<code>us</code>	Delay duration in microseconds (0 = return immediately; values exceeding <code>k_max_delay_ticks</code> converted to ticks are silently no-op)

Returns: void

Pre: CMT2 timer initialized (automatic on first call)

Pre: CMT2 hardware is accessible (initialized automatically on first call)

Pre: Must NOT be called from a context requiring very long delays ($> k_max_delay_ticks$ ticks)

Post: Caller blocked for approximately `us` microseconds

Post: No side effects (timer state unchanged)

Post: Approximately us microseconds have elapsed (accuracy depends on PCLKB frequency)

Post: CMT2 has been initialized (`s_cmt2_initialized == true`)

Note: NOT thread-safe: Concurrent calls may interfere on mock HAL

Note: Does NOT yield to RTOS: Use `tx_thread_sleep()` for long delays

Note: Not thread-safe; CMT2 is a shared hardware resource; concurrent callers will both observe the same free-running counter and race on initialization

Warning: Delays >10ms should use RTOS sleep (avoid busy-wait)

Warning: Accuracy degrades for very short delays (<2us) due to function call overhead

Warning: This is a BLOCKING busy-wait; avoid in latency-sensitive interrupt contexts

Warning: Accuracy degrades for large us values due to 16-bit counter overflow splitting

Example:: `hcsr04_hal_gpio_write_high(trig_pin); hcsr04_hal_delay_us(10);`
`hcsr04_hal_gpio_write_low(trig_pin);`

Performance:: Function call overhead: 500 ns Loop overhead: 100 ns per iteration Effective minimum delay: 1 us

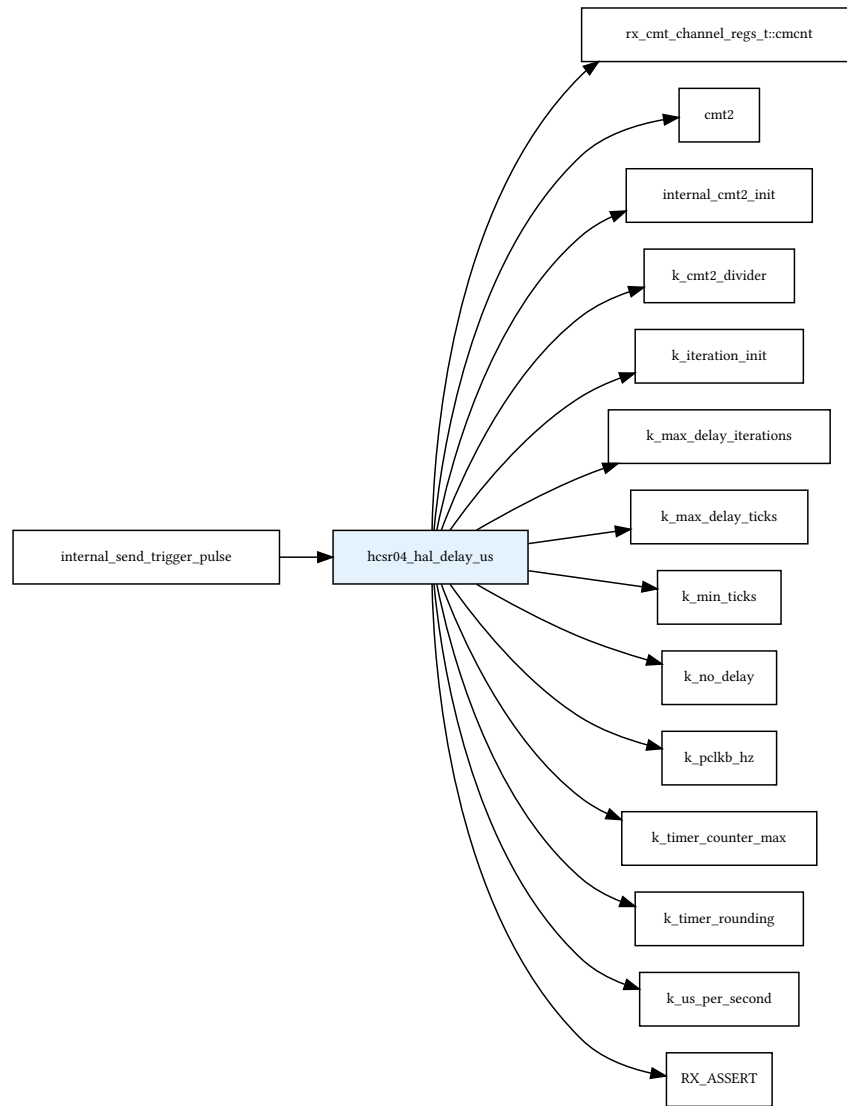
Thread Safety:: Hardware HAL: Safe (uses dedicated timer, no shared state) Mock HAL: NOT safe (simulated delays may overlap)

Performance:: Overhead: 3-5 us minimum due to init check and loop setup @ 240 MHz

Example:: `hcsr04_hal_gpio_write_high(trigger_pin); hcsr04_hal_delay_us(10);`
`hcsr04_hal_gpio_write_low(trigger_pin);`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (CMT2 accessible, delay within bounds), 2 postconditions

See also: `hcsr04_hal_get_time_us` Get current timestamp, `k_hcsr04_trigger_pulse_us` Trigger pulse duration constant, `hcsr04_hal_get_time_us()` Get current timestamp without blocking, `hcsr04_hal_gpio_write_high()` Drive trigger pin high before delay

Figure 983: Call graph for `hcsr04_hal_delay_us`

_Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_hal.h:661_`

Since Version 1.0.0

—

`hcsr04_hal_get_time_us`

```
uint32_t hcsr04_hal_get_time_us(void)
```

Get monotonic timestamp in microseconds.

Returns a continuously increasing timestamp with microsecond resolution. Used to measure HC-SR04 echo pulse duration by capturing start and end timestamps. Guaranteed monotonic (never decreases, even across overflows).

Hardware Implementation (RX72N):

- Timer: CMT2 (Compare Match Timer 2)
- Clock: PCLKB 60 MHz / 8 = 7.5 MHz
- Native resolution: 133 ns per tick
- Counter width: 16-bit (0-65535)
- Overflow period: 8.7 ms
- Software extension: 32-bit via overflow tracking
- Range: 0 to 571 seconds (uint32_t microseconds)

Overflow Handling:

- Hardware counter: Wraps every 8.7 ms (65536 ticks)
- Software tracks overflows in static variable
- Mutex-protected for thread safety
- Result: 32-bit timestamp = (overflow_count << 16) | counter

Mock Implementation:

- Uses host monotonic clock:

Linux/macOS: `clock_gettime(CLOCK_MONOTONIC)` Windows: `QueryPerformanceCounter()`

- Resolution: Platform-dependent (100 ns to 1 us)

Monotonicity Guarantee:

- Time never goes backward
- Immune to system clock adjustments (NTP, user changes)
- Safe for duration calculations across overflows

Typical Usage Pattern:

Returned value never decreases (monotonic property)

Resolution ≥ 1 us (meets HC-SR04 timing requirements)

Get monotonic timestamp in microseconds.

Returns a microsecond-resolution timestamp derived from the CMT2 free-running 16-bit counter. Tracks 16-bit overflow events using a static overflow counter so that timestamps remain monotonically increasing beyond the 8.7 ms CMT2 wrap period (65536 ticks at PCLKB/8 = 7.5 MHz).

Thread Safety: A ThreadX mutex (`s_time_mutex`) protects the static `overflow_count` and `last_counter` variables. If mutex initialization fails (`internal_time_mutex_init()`), the function falls back to returning the raw CMT2 counter without overflow tracking. If `tx_mutex_get()` fails at runtime, the same unprotected fallback is used.

Overflow detection algorithm:

- If `current_counter < last_counter`, the 16-bit counter has wrapped; increment `overflow_count`
- Total ticks = (`overflow_count << k_timer_counter_bits`) | `current_counter`
- Microseconds = (`total_ticks * k_us_per_second`) / `timer_hz`

Algorithm steps:

1. Call `internal_cmt2_init()` to ensure CMT2 is running
2. Attempt `internal_time_mutex_init()`; on failure return raw counter (no overflow tracking)

3. Acquire s_time_mutex (TX_WAIT_FOREVER); on failure return raw counter
4. Read current_counter from cmt2()->cmcnt
5. If current_counter < last_counter: increment overflow_count
6. Update last_counter = current_counter
7. Compute total_ticks and convert to microseconds
8. Release s_time_mutex
9. Return microsecond result

Returns: uint32_t Monotonic timestamp in microseconds

Value	Description
[0	Microseconds since CMT2 was first initialized; wraps at UINT32_MAX (71 minutes); value increases monotonically between calls (assuming no > 71 minute gap between calls)

Pre: CMT2 timer initialized (automatic on first call)

Pre: CMT2 hardware is accessible (initialized automatically on first call)

Pre: ThreadX kernel must be running for mutex-protected overflow tracking

Post: No side effects (read-only operation)

Post: CMT2 has been initialized (s_cmt2_initialized == true after first call)

Post: overflow_count is updated if 16-bit counter wrapped since last call

Note: Thread-safe on RX72N (mutex-protected overflow counter)

Note: Zero cost when idle (no active counting overhead)

Note: Thread-safe when mutex acquisition succeeds; falls back to unsafe mode otherwise

Warning: Wraps at 2^{32} us (71 minutes) - caller must handle wrap-around

Warning: First call initializes CMT2 (5 us overhead)

Warning: Overflow tracking accuracy requires calls more frequent than 8.7 ms; if called less frequently the overflow count may be incorrect

Performance:: Read time: 1-2 us (register read + conversion) Mutex overhead: 5 us (if overflow tracking needed) Call frequency: Unlimited (no rate limiting)

Thread Safety:: Hardware HAL: SAFE - overflow counter protected by mutex Mock HAL: SAFE - system monotonic clock is thread-safe

Accuracy:: RX72N: +/-0.1% (crystal tolerance) Mock: Platform-dependent (typically +/-0.01%)

Example: Echo Pulse Measurement: boolecho=false; while(!echo)
{ hcsr04_hal_gpio_read(echo_pin,&echo); } uint32_tpulse_start=hcsr04_hal_get_time_us();
while(echo){ hcsr04_hal_gpio_read(echo_pin,&echo); } uint32_tpulse_end=hcsr04_hal_get_time_us();
uint32_tpulse_us=pulse_end-pulse_start; floatdistance_cm=(float)pulse_us/58.0f;

Overflow Handling Example:: uint32_tstart=0xFFFFFFFF; uint32_tend=0x00000004;
uint32_tdur=end-start;

Performance:: Execution time: 5-10 us with mutex @ 240 MHz (ThreadX overhead)

Example:: uint32_tstart=hcsr04_hal_get_time_us(); uint32_tend=hcsr04_hal_get_time_us();
uint32_techo_us=end-start; floatdistance_cm=(float)echo_us/58.0f;

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (CMT2 accessible, ThreadX running for mutex), 2 postconditions

Example:

```
//Measureechopulseduration  
uint32_tstart=hcsr04_hal_get_time_us();  
//...waitforecho...  
uint32_tend=hcsr04_hal_get_time_us();  
uint32_tduration=end-start; //Correctevenifoverflowoccurred
```

See also: hcsr04_hal_delay_us Blocking delay function, internal_measure_echo_pulse Echo timing function, hcsr04_hal_get_time_us_isr() ISR-safe variant (no mutex), hcsr04_hal_delay_us() Blocking microsecond delay using same CMT2 timer

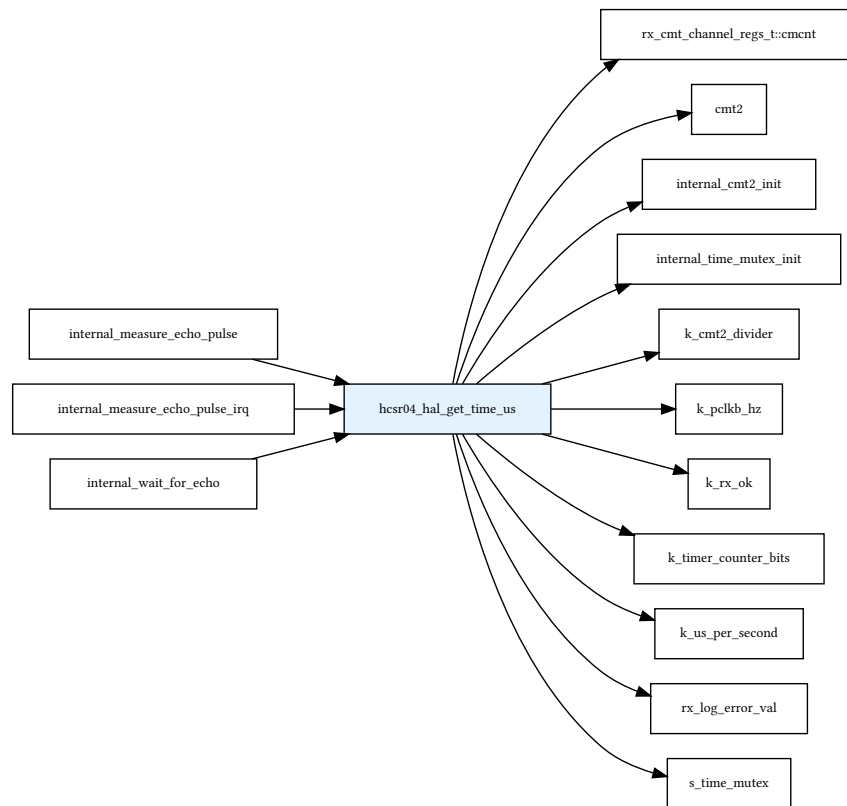


Figure 984: Call graph for hcsr04_hal_get_time_us

_Defined in libs/rx_hcsr04/src/rx_hcsr04_hal.h:768_

Since Version 1.0.0

—

hcsr04_hal_get_time_us_isr

```
uint32_t hcsr04_hal_get_time_us_isr(void)
```

Get monotonic timestamp in microseconds ISR-safe (no mutex).

Returns the raw CMT2 counter value converted to microseconds. Unlike hcsr04_hal_get_time_us(), this function does NOT acquire the ThreadX mutex and does NOT track 16-bit counter overflows. It is safe to call from interrupt context and provides sub-microsecond latency.

Hardware Constraint 8.7 ms Maximum Echo Duration: The CMT2 timer is a 16-bit counter clocked at PCLKB/8 = 7.5 MHz, which wraps every 8.7 ms. HC-SR04 echo pulses can be up to 23 ms (400 cm x 58 us/cm). If the echo spans more than one counter period, the naive end_us - start_us subtraction will be incorrect.

Supported range in IRQ mode: ≤ 150 cm (8.7 ms echo pulse) For longer ranges, use polling mode with `hcsr04_hal_get_time_us()` (which tracks overflows via a software overflow counter protected by a mutex).

Reads the CMT2 counter directly without acquiring the ThreadX mutex. Safe to call from interrupt context. Does NOT track 16-bit overflows. Because the 16-bit CMT2 counter wraps every 8.7 ms (65536 ticks at $PCLKB/8 = 7.5$ MHz), this function is only valid for echo windows shorter than the CMT2 wrap period. Rising and falling edge timestamps separated by more than 8.7 ms produce invalid durations. HC-SR04 IRQ mode is therefore limited to 150 cm maximum range.

Returns: CMT2 counter converted to microseconds (no overflow tracking)

Value	Description
uint32_t	Raw CMT2 counter converted to microseconds; wraps at 8.7 ms
uint32_t	Value in range [0, 8700) us; wraps every 8.7 ms with CMT2 period

Pre: CMT2 timer initialized (call `hcsr04_hal_get_time_us()` at least once at startup)

Pre: Called from ISR context (no ThreadX blocking calls permitted)

Pre: CMT2 timer initialized (call `hcsr04_hal_get_time_us()` once at startup)

Pre: Called from interrupt context (no ThreadX blocking calls permitted)

Post: No side effects (read-only, no mutex acquired)

Post: Returned value is in range 0, 8700) us (wraps with CMT2 period)

Post: No side effects (read-only, no mutex acquired)

Post: Returned value equals $(\text{cmcnt} * 1000000) / \text{timer_hz}$ for current CMT2 count

Note: ISR-safe: does NOT call `tx_mutex_get()` or any blocking ThreadX API

Note: For task context, prefer `hcsr04_hal_get_time_us()` (overflow tracking)

Note: ISR-safe: does NOT call `tx_mutex_get()` or any blocking ThreadX API

Warning: Does not track 16-bit overflow; do not use for durations > 8.7 ms

Warning: HC-SR04 IRQ mode limited to 150 cm due to 8.7 ms CMT2 wrap period

Warning: Does not track 16-bit counter overflow; do not use for intervals > 8.7 ms

Warning: HC-SR04 IRQ mode limited to 150 cm (8.7 ms echo) due to CMT2 wrap period

Example: Edge Capture in ISR: `void INT_IRQ11(void) { internal_irq_handler(k_irq_num_11); }`

Example: Edge Timestamp in ISR: `uint32_t ts=hcsr04_hal_get_time_us_isr();
s_irq_state[idx].start_us=ts; s_irq_state[idx].end_us=ts;`

See also: `hcsr04_hal_get_time_us()` Thread-safe variant with overflow tracking for task context, `hcsr04_hal_get_time_us()` Thread-safe variant with overflow tracking for task context

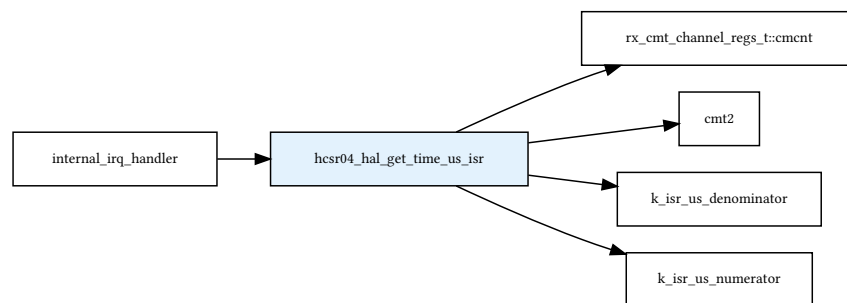


Figure 985: Call graph for `hcsr04_hal_get_time_us_isr`

_Defined in `libs/rx\hcsr04/src/rx\hcsr04_hal.h:818_`

Since Version 1.2.0 (Issue 296 - ISR-mode HC-SR04 measurement)

rx_hcsr04_hal_hw.c

HC-SR04 Hardware Abstraction Layer Implementation for Renesas RX72N.

Includes:

- `<stdbool.h>`
- `"hardware.h"`
- `"rx72n_clock.h"`
- `"rx72n_cmt_regs.h"`
- `"rx72n_system_regs.h"`
- `"rx_check.h"`
- `"rx_hcsr04_hal.h"`
- `"rx_log.h"`
- `"tx_api.h"`

cmt2_timing_constants_t

CMT2 timer configuration and timing calculation constants.

Constants for configuring the CMT2 (Compare Match Timer Unit 2) and performing microsecond timing calculations. All values derived from RX72N clock configuration and CMT2 hardware specifications.

Value	Name	Description
= 8	k_cmt2_divider	Clock divider: PCLKB / 8 = 7.5 MHz
= 0x0000	k_cmt2_divider_bits	CMCR.CKS[1:0] = 00 for /8
= 0xFFFF	k_timer_counter_max	16-bit max value (65535 ticks)
= 16	k_timer_counter_bits	Counter width for overflow tracking
= 1000000	k_us_per_second	us to seconds conversion
= 500000	k_timer_rounding	Add 0.5 for round-to-nearest
= 100	k_max_delay_iterations	NASA Rule 2: Loop bound (max 873 ms)
= k_timer_counter_max * k_max_delay_iterations	k_max_delay_ticks	6.55M ticks
= 0	k_no_delay	Delay sentinel (no-op)
= 1	k_min_ticks	Minimum delay to prevent zero-wait
= k_rx72n_cmstr1_cmt2_enable	k_cmstr1_cmt2_enable_bit	CMSTR1.STR2 bit (bit 0)
= 0	k_counter_reset	Counter initial value
= 0	k_iteration_init	Iteration counter

		initial value
= 2	k_isr_us_numerator	ISR timestamp ratio numerator: 1000000 / (60000000/8) = 2/15
= 15	k_isr_us_denominator	ISR timestamp ratio denominator: avoids 64-bit division in ISR

—

s_cmt2_initialized`bool s_cmt2_initialized`

CMT2 timer initialization state flag.

Note: Once true, never cleared (CMT2 runs for entire program lifetime)

Warning: NOT thread-safe - caller must ensure single initialization

—

s_time_mutex`TX_MUTEX s_time_mutex`

ThreadX mutex protecting overflow counter in get_time_us().

Note: Mutex overhead: 5 us per get_time_us() call

Warning: Only access via ThreadX APIs (tx_mutex_get/put)] **Example:**

```
tx_mutex_get(&s_time_mutex, TX_WAIT_FOREVER);
//Readcurrent_counter
//Detectoverflow:if(current_counter<last_counter)overflow_count++
//Updatelast_counter=current_counter
//Computetimestampfromoverflow_countandcurrent_counter
tx_mutex_put(&s_time_mutex);
```

—

s_time_mutex_initialized

```
bool s_time_mutex_initialized
```

Mutex initialization state flag.

Note: Once true, never cleared (mutex exists for entire program lifetime)

Warning: NOT thread-safe - caller must ensure single initialization

internal_validate_port_pin

```
static rx_err_t internal_validate_port_pin(const rx_port_pin_t pin, uint8_t
*port, uint8_t *pin_num)
```

Validate GPIO pin number and extract port/pin components.

Validates that the pin identifier is within valid RX72N GPIO range and optionally extracts the port and pin number components for register access.

RX72N GPIO Organization:

- Pins encoded as: (port << 8) | pin_num
- Example: Port B Pin 2 = (0x01 << 8) | 0x02 = 0x0102
- Valid ports: 0-7, A-G, J (NOT H, I - gaps in RX72N GPIO map)
- Valid pin numbers: 0-7 (8 pins per port)

Validation Checks:

1. Pin value <= k_rx_pj_7 (highest valid GPIO)
2. Extracted port number <= k_rx_port_j
3. Extracted pin number <= k_rx_pin_max (7)

Why Validation Needed:

- Prevents register access to non-existent ports (hardware fault)
- Catches configuration errors early (at HAL layer, not deep in GPIO driver)
- Consistent error handling across HC-SR04 HAL

Example Usage:

RX72N has gaps in GPIO map (ports H, I don't exist)

Parameters:

Direction	Name	Description
in	pin	GPIO pin identifier (type-safe enum from rx_port_constants.h)
out	port	Pointer to receive port number (0-10), or nullptr if not needed
out	pin_num	Pointer to receive pin number (0-7), or nullptr if not needed

Returns: k_rx_err_invalid_arg Pin out of valid range

Pre: Pin must be from rx_port_constants.h (type-safe enum)

Post: If k_rx_ok: *port and *pin_num contain valid values (if non-NULL pointers)

Post: If error: *port and *pin_num unchanged

Note: Output pointers can be NULL (validation-only mode)

Note: Lower bound checks omitted (unsigned types, minimums are 0)

Warning: Does NOT check if pin is already allocated to peripheral

RX72N GPIO Map:: Port 0-7: General-purpose I/O Port A-G: Additional GPIO Port J: Highest port Port H, I: NOT PRESENT (gaps in addressing)

Example:

```
uint8_tport_num=0;
uint8_tpin_num=0;
rx_err_terr=internal_validate_port_pin(k_gpio_pb2,&port_num,&pin_num);
if(err==k_rx_ok){
//port_num=1(PortB),pin_num=2
PORT[port_num].PDR|=(1<<pin_num);//Safetoaccess
}
```

See also: rx_port_from_pin Extract port number from pin, rx_pin_from_pin Extract pin number from pin, rx_port_constants.h GPIO pin definitions

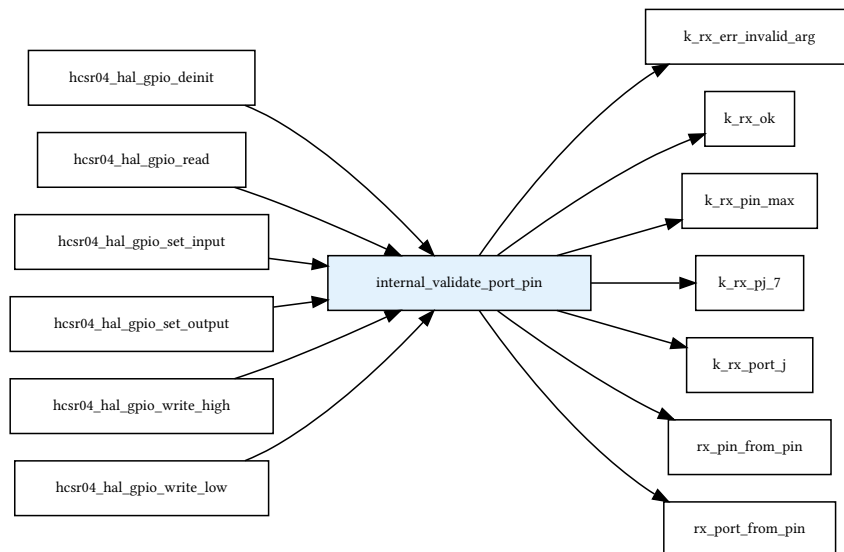


Figure 986: Call graph for internal_validate_port_pin

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:509`

Since Version 1.0.0

hcsr04_hal_gpio_set_output

```
rx_err_t hcsr04_hal_gpio_set_output(const rx_port_pin_t pin)
```

Configure a GPIO pin as a digital output for HC-SR04 trigger control.

Configure GPIO pin as output for HC-SR04 trigger signal.

Validates the port/pin combination and then configures the specified GPIO pin as a push-pull digital output via the `gpio_set_output()` hardware abstraction layer. Used to configure the HC-SR04 trigger pin before initiating ultrasonic measurement pulses.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()` (checks port and pin indices)
2. Call `gpio_set_output(pin)` to set the data direction register bit

Parameters:

Direction	Name	Description
in	pin	GPIO port/pin descriptor specifying the trigger GPIO; must be a valid <code>rx_port_pin_t</code> with legal port and pin values

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin successfully configured as output
<code>k_rx_err_invalid_arg</code>	pin specifies an invalid port or pin number

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: GPIO peripheral clock must be enabled before calling

Post: The specified GPIO pin direction register bit is set to output mode

Post: Pin output level is undefined; call `hcsr04_hal_gpio_write_low()` to initialize

Note: Not thread-safe; GPIO direction configuration should occur during initialization

Example:: `rx_err_t err = hcsr04_hal_gpio_set_output(config->trigger_pin); if(err != k_rx_ok) { rx_log_error("hcsr04", "Failed to configure trigger pin as output"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, clock enabled), 2 postconditions

See also: `hcsr04_hal_gpio_write_high()` Drive trigger pin high to start pulse, `hcsr04_hal_gpio_write_low()` Drive trigger pin low, `hcsr04_hal_gpio_set_input()` Configure echo pin as input

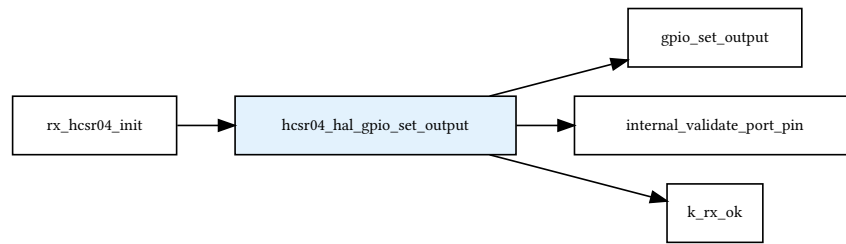


Figure 987: Call graph for hcsr04_hal_gpio_set_output

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:588`

Since Version 1.0.0

—

hcsr04_hal_gpio_set_input

```
rx_err_t hcsr04_hal_gpio_set_input(const rx_port_pin_t pin)
```

Configure a GPIO pin as a digital input for HC-SR04 echo reception.

Configure GPIO pin as input for HC-SR04 echo signal.

Validates the port/pin combination and then configures the specified GPIO pin as a digital input via the `gpio_set_input()` hardware abstraction layer. Used to configure the HC-SR04 echo pin so that the rising and falling edges of the echo pulse can be measured.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()` (checks port and pin indices)
2. Call `gpio_set_input(pin)` to clear the data direction register bit

Parameters:

Direction	Name	Description
in	pin	GPIO port/pin descriptor specifying the echo GPIO; must be a valid <code>rx_port_pin_t</code> with legal port and pin values

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin successfully configured as input
<code>k_rx_err_invalid_arg</code>	pin specifies an invalid port or pin number

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: GPIO peripheral clock must be enabled before calling

Post: The specified GPIO pin direction register bit is cleared to input mode

Post: Pin can now be read to detect echo pulse edges

Note: Not thread-safe; GPIO direction configuration should occur during initialization

Example:: `rx_err_t rx_hcsr04_hal_gpio_set_input(config->echo_pin); if(err!=k_rx_ok) { rx_log_error("hcsr04","Failed to configure echo pin as input"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, clock enabled), 2 postconditions

See also: `hcsr04_hal_gpio_set_output()` Configure trigger pin as output, `hcsr04_hal_gpio_write_high()` Drive trigger pin high

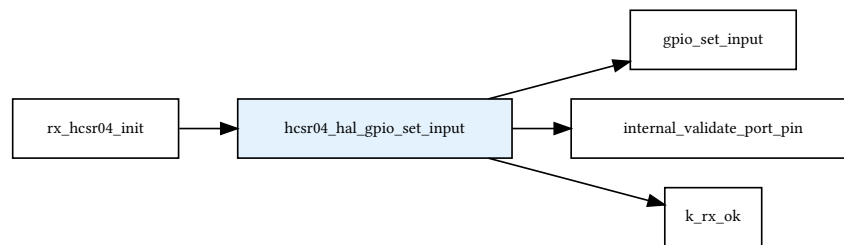


Figure 988: Call graph for hcsr04_hal_gpio_set_input

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:642`

Since Version 1.0.0

—

hcsr04_hal_gpio_write_high

```
rx_err_t hcsr04_hal_gpio_write_high(const rx_port_pin_t pin)
```

Drive a GPIO output pin to logic high to initiate HC-SR04 trigger pulse.

Set GPIO pin to HIGH (3.3V) level.

Validates the port/pin combination and drives the specified output GPIO pin to a logic high (VDD) level via the `gpio_write_high()` hardware abstraction layer. In HC-SR04 operation, this is used to begin the 10 us trigger pulse that initiates an ultrasonic ranging measurement.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()`
2. Call `gpio_write_high(pin)` to set the output data register bit

Parameters:

Direction	Name	Description
in	pin	GPIO port/pin descriptor specifying the output pin to drive high; must be a valid <code>rx_port_pin_t</code> already configured as output

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	Pin output register bit successfully set high
k_rx_err_invalid_arg	pin specifies an invalid port or pin number

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: Pin must have been configured as output via `hcsr04_hal_gpio_set_output()`

Post: The GPIO pin output data register bit is set; pin drives logic high

Post: Follow with `hcsr04_hal_delay_us(10)` then `hcsr04_hal_gpio_write_low()` for trigger pulse

Note: Not thread-safe; concurrent GPIO writes to the same pin produce undefined behavior

Example:: `hcsr04_hal_gpio_write_high(config->trigger_pin); hcsr04_hal_delay_us(10);`
`hcsr04_hal_gpio_write_low(config->trigger_pin);`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, configured as output), 2 postconditions

See also: `hcsr04_hal_gpio_write_low()` Drive pin low to end trigger pulse,
`hcsr04_hal_gpio_set_output()` Configure pin as output first, `hcsr04_hal_delay_us()`
Delay for trigger pulse duration

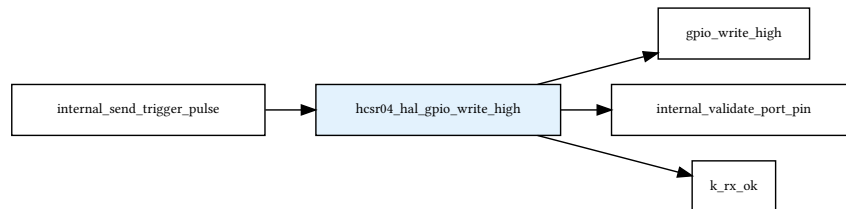


Figure 989: Call graph for `hcsr04_hal_gpio_write_high`

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:697`

Since Version 1.0.0

—

hcsr04_hal_gpio_write_low

```
rx_err_t hcsr04_hal_gpio_write_low(const rx_port_pin_t pin)
```

Drive a GPIO output pin to logic low to complete HC-SR04 trigger pulse.

Set GPIO pin to LOW (GND) level.

Validates the port/pin combination and drives the specified output GPIO pin to a logic low (GND) level via the `gpio_write_low()` hardware abstraction layer. In HC-SR04 operation, this is used after a

10 us high pulse to complete the trigger signal, after which the sensor emits ultrasonic bursts and the echo pin goes high for a period proportional to measured distance.

Algorithm steps:

1. Validate the pin via `internal_validate_port_pin()`
2. Call `gpio_write_low(pin)` to clear the output data register bit

Parameters:

Direction	Name	Description
in	pin	GPIO port/pin descriptor specifying the output pin to drive low; must be a valid <code>rx_port_pin_t</code> already configured as output

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Pin output register bit successfully cleared low
<code>k_rx_err_invalid_arg</code>	pin specifies an invalid port or pin number

Pre: pin must encode a valid port/pin combination supported by the RX72N

Pre: Pin must have been configured as output via `hcsr04_hal_gpio_set_output()`

Post: The GPIO pin output data register bit is cleared; pin drives logic low

Post: HC-SR04 trigger sequence is complete; echo pin will pulse proportional to distance

Note: Not thread-safe; concurrent GPIO writes to the same pin produce undefined behavior

Example:: `hcsr04_hal_gpio_write_high(config->trigger_pin); hcsr04_hal_delay_us(10);`
`hcsr04_hal_gpio_write_low(config->trigger_pin); uint32_t echo_start = hcsr04_hal_get_time_us();`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (valid pin, configured as output), 2 postconditions

See also: `hcsr04_hal_gpio_write_high()` Drive pin high to start trigger pulse,
`hcsr04_hal_gpio_set_output()` Configure pin as output first, `hcsr04_hal_get_time_us()`
 Capture echo start timestamp after trigger

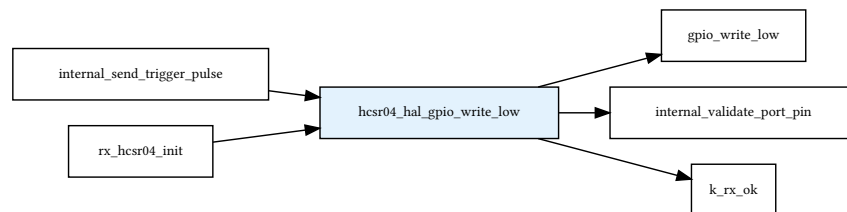


Figure 990: Call graph for `hcsr04_hal_gpio_write_low`

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:754`

Since Version 1.0.0

—

hcsr04_hal_gpio_read

```
rx_err_t hcsr04_hal_gpio_read(const rx_port_pin_t pin, bool *value)
```

Read current state of GPIO pin.

Read current logic level of GPIO pin.

Parameters:

Direction	Name	Description
in	pin	GPIO pin to read
out	value	Pointer to receive pin state (true=high, false=low)

Returns: k_rx_err_invalid_arg if pin is invalid

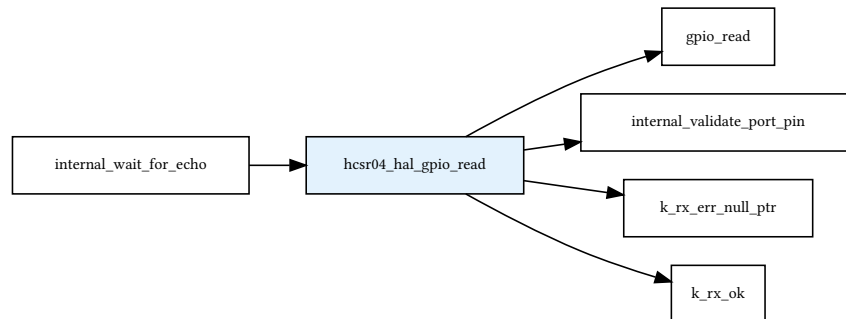


Figure 991: Call graph for `hcsr04_hal_gpio_read`

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:775`

—

hcsr04_hal_gpio_deinit

```
rx_err_t hcsr04_hal_gpio_deinit(const rx_port_pin_t pin)
```

Deinitialize GPIO pin (no-op on RX72N).

Release GPIO pin resources (cleanup).

This is intentionally a no-op as RX72N GPIO pins are static resources.

Parameters:

Direction	Name	Description
in	pin	GPIO pin to deinitialize

Returns: k_rx_err_invalid_arg if pin is invalid

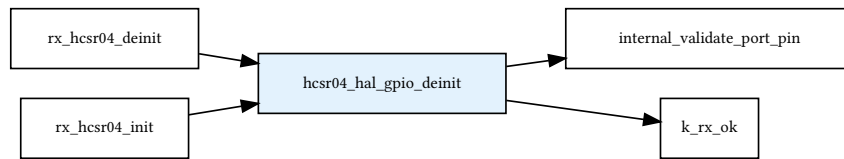


Figure 992: Call graph for hcsr04_hal_gpio_deinit

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:799`

internal_time_mutex_init

```
static rx_err_t internal_time_mutex_init(void)
```

Initialize ThreadX mutex for overflow counter protection (idempotent).

Creates the ThreadX mutex used by `hcsr04_hal_get_time_us()` to protect overflow counter variables. Safe to call multiple times (checks initialized flag).

Created Resource:

- Name: “TimeMutex”
- Type: TX_MUTEX
- Priority inheritance: TX_NO_INHERIT (avoids priority inversion complexity)

Initialization Flow:

Failure Handling: If creation fails (rare), caller falls back to unprotected mode:

Returns: `k_rx_err_hw_init_failed` `tx_mutex_create()` failed

Pre: ThreadX kernel initialized (`tx_kernel_enter()` called)

Post: `s_time_mutex` ready for use (if `k_rx_ok` returned)

Post: `s_time_mutex_initialized` = true (if `k_rx_ok` returned)

Note: Idempotent: Safe to call multiple times

Note: First call: 50 us (mutex creation overhead)

Note: Subsequent calls: 100 ns (flag check only)

Warning: Do NOT call before ThreadX kernel starts

Warning: Failure leaves overflow tracking disabled (degraded accuracy)

Thread Safety:: NOT thread-safe during initialization. Concurrent first calls may attempt double-initialization. In practice, first call occurs from single thread during startup.

Example:

```

if(internal_time_mutex_init()!=k_rx_ok){
//Fallback:Returncurrentcounterwithoutoverflowtracking
return(uint32_t)((cmt2()->cmcnt*1000000)/timer_hz);
}

```

See also: s_time_mutex Mutex object, s_time_mutex_initialized Initialization flag, hcsr04_hal_get_time_us User of this mutex

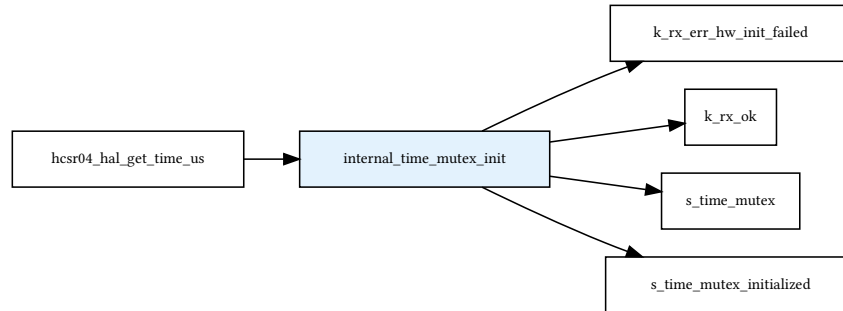


Figure 993: Call graph for `internal_time_mutex_init`

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:1070`

Since Version 1.0.0

—

internal_cmt2_init

```
static void internal_cmt2_init(void)
```

Initialize CMT2 hardware timer for microsecond timing operations (idempotent).

Configures CMT2 (Compare Match Timer Unit 2) as a free-running timer for microsecond-precision delays and timestamps. Safe to call multiple times (checks initialized flag).

Hardware Configuration Sequence:

1. Enable CMT2 module clock

Clear MSTPCRA bit 14 (CMT2/3 module stop) Provides PCLKB clock to CMT2 peripheral

2. Stop CMT2

Clear CMSTR1 bit 0 (CMT2 start bit) Required before configuration changes

3. Configure clock divider

Set CMCR.CKS[1:0] = 00 (/8 divider) Result: 60 MHz / 8 = 7.5 MHz timer clock

4. Reset counter

Write CMCNT = 0 (start from zero)

5. Set compare match value

Write CMCOR = 0xFFFF (maximum, free-running mode) Timer counts 0 -> 65535, then wraps to 0 (no interrupt)

6. Start CMT2

Set CMSTR1 bit 0 (CMT2 start bit) Counter begins incrementing at 7.5 MHz

Register Access:

Free-Running Mode:

- Counter increments continuously: 0 -> 1 -> ... -> 65535 -> 0 -> 1 -> ...
- No compare match interrupt (CMIE = 0)
- Overflow period: $65536 / 7.5 \text{ MHz} = 8.738 \text{ ms}$
- Software extends to 32-bit via overflow tracking

Timing Characteristics:

Returns: void (never fails, idempotent)

Pre: PCLKB = 60 MHz (system clock configured)

Post: CMT2 running at 7.5 MHz

Post: CMCNT incrementing continuously

Post: s_cmt2_initialized = true

Note: Idempotent: Safe to call multiple times (flag prevents reconfig)

Note: First call: 5 us (register writes + module enable)

Note: Subsequent calls: 50 ns (flag check only)

Note: CMT2 runs forever after init (never stopped)

Warning: Do NOT modify CMT2 registers after initialization

Warning: CMT2 used exclusively by HC-SR04 HAL (don't share with other modules)

Thread Safety:: NOT thread-safe during initialization. Concurrent first calls may reconfigure timer mid-operation. In practice, first call occurs from single thread during first delay_us() or get_time_us().

Example:

```
MSTPCRA(0x80028000):
```

```
Bit14:CMT2/3modulestop(0=running,1=stopped)
```

```
CMSTR1(0x00088010):
```

```
Bit0:CMT2start(0=stopped,1=running)
```

```
CMT2.CMCR(0x00088012):
```

```
Bits[7:6]:Reserved(mustbe0)
```

```
Bits[5:4]:Reserved(mustbe0)
```

```
Bits[3:2]:Reserved(mustbe0)
```

```
Bits[1:0]:CKS[1:0]=00(PCLK/8)
```

```
CMT2.CMCNT(0x00088014):  
16-bitcounter(read-onlyduringrun,writewhenstopped)
```

```
CMT2.CMCOR(0x00088016):  
16-bitcomparematchvalue(0xFFFF=free-running)
```

See also: s_cmt2_initialized Initialization flag, hcsr04_hal_delay_us Delay function
using CMT2, hcsr04_hal_get_time_us Timestamp function using CMT2, cmt2() CMT2 register
accessor

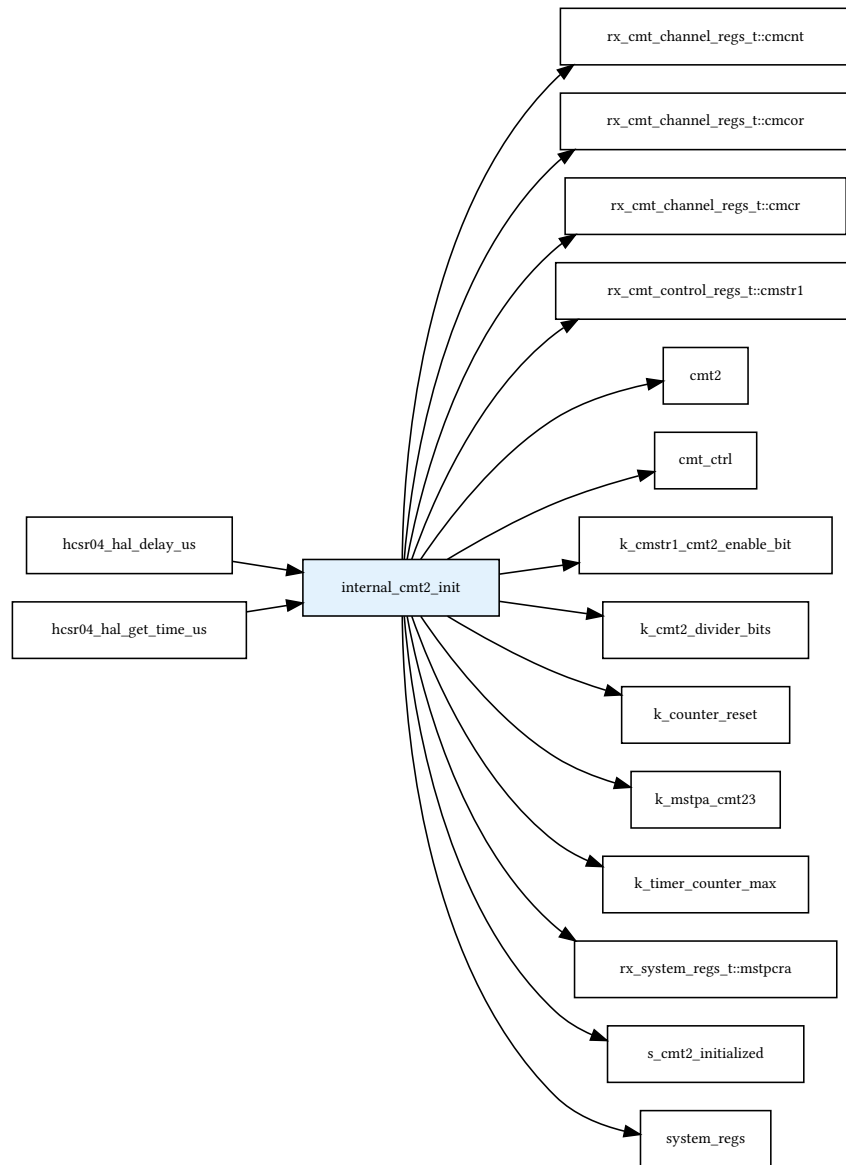


Figure 994: Call graph for `internal_cmt2_init`

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:1180`

Since Version 1.0.0

`hcsr04_hal_delay_us`

```
void hcsr04_hal_delay_us(uint32_t us)
```


Delay execution by a specified number of microseconds using CMT2 hardware timer.

Blocking delay for specified microseconds.

Produces a blocking busy-wait delay by polling the CMT2 Compare Match Timer counter. CMT2 is automatically initialized on the first call via `internal_cmt2_init()`. For large delays (exceeding `k_timer_counter_max` ticks per iteration) the wait is split into multiple smaller segments.

CMT2 is driven from `PCLKB / k_cmt2_divider`, yielding a tick frequency of `k_pclkb_hz / k_cmt2_divider` Hz. The requested microsecond count is converted to ticks using this frequency before entering the polling loop.

Safety bounds:

- `us == 0`: returns immediately (`k_no_delay` check)
- `ticks < k_min_ticks`: clamped up to `k_min_ticks`
- `ticks > k_max_delay_ticks`: no-op in release; assertion in unit tests
- `iteration_count >= k_max_delay_iterations`: exits loop (prevents infinite spin)

Algorithm steps:

1. Return immediately if `us == k_no_delay`
2. Call `internal_cmt2_init()` to ensure CMT2 is running
3. Compute ticks from `us` and timer frequency; clamp to `[k_min_ticks, k_max_delay_ticks]`
4. Loop: poll CMT2 counter until `wait_ticks` have elapsed; decrement remaining ticks
5. Exit when `ticks == 0` or `iteration_count` reaches `k_max_delay_iterations`

Parameters:

Direction	Name	Description
in	<code>us</code>	Delay duration in microseconds (0 = return immediately; values exceeding <code>k_max_delay_ticks</code> converted to ticks are silently no-op)

Returns: void

Pre: CMT2 hardware is accessible (initialized automatically on first call)

Pre: Must NOT be called from a context requiring very long delays ($> k_max_delay_ticks$ ticks)

Post: Approximately `us` microseconds have elapsed (accuracy depends on PCLKB frequency)

Post: CMT2 has been initialized (`s_cmt2_initialized == true`)

Note: Not thread-safe; CMT2 is a shared hardware resource; concurrent callers will both observe the same free-running counter and race on initialization

Warning: This is a BLOCKING busy-wait; avoid in latency-sensitive interrupt contexts

Warning: Accuracy degrades for large `us` values due to 16-bit counter overflow splitting

Performance: Overhead: 3-5 us minimum due to init check and loop setup @ 240 MHz

Example:: hcsr04_hal_gpio_write_high(trigger_pin); hcsr04_hal_delay_us(10);
hcsr04_hal_gpio_write_low(trigger_pin);

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (CMT2 accessible, delay within bounds),
2 postconditions

See also: hcsr04_hal_get_time_us() Get current timestamp without blocking,
hcsr04_hal_gpio_write_high() Drive trigger pin high before delay

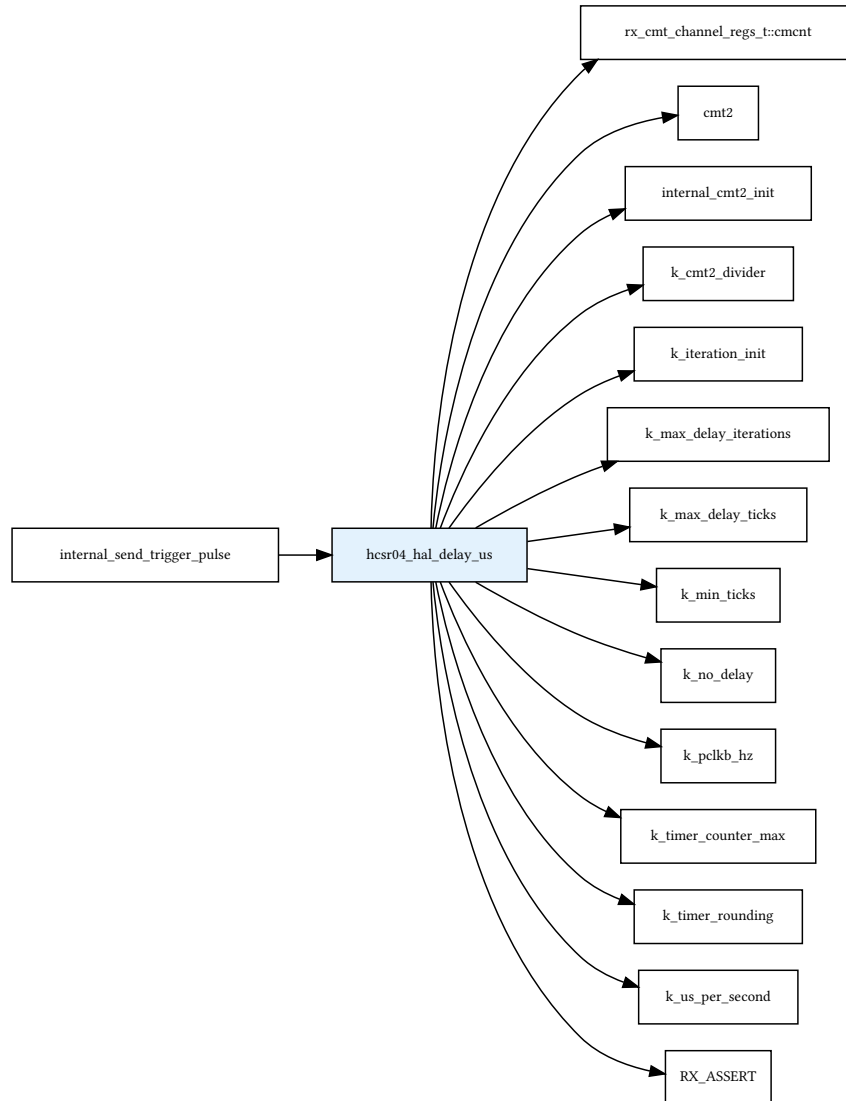


Figure 995: Call graph for hcsr04_hal_delay_us

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:1270`

Since Version 1.0.0

hcsr04_hal_get_time_us

```
uint32_t hcsr04_hal_get_time_us(void)
```

Get current monotonic timestamp in microseconds using CMT2 hardware timer.

Get monotonic timestamp in microseconds.

Returns a microsecond-resolution timestamp derived from the CMT2 free-running 16-bit counter. Tracks 16-bit overflow events using a static overflow counter so that timestamps remain monotonically increasing beyond the 8.7 ms CMT2 wrap period (65536 ticks at PCLKB/8 = 7.5 MHz).

Thread Safety: A ThreadX mutex (s_time_mutex) protects the static overflow_count and last_counter variables. If mutex initialization fails (internal_time_mutex_init()), the function falls back to returning the raw CMT2 counter without overflow tracking. If tx_mutex_get() fails at runtime, the same unprotected fallback is used.

Overflow detection algorithm:

- If current_counter < last_counter, the 16-bit counter has wrapped; increment overflow_count
- Total ticks = (overflow_count << k_timer_counter_bits) | current_counter
- Microseconds = (total_ticks * k_us_per_second) / timer_hz

Algorithm steps:

1. Call internal_cmt2_init() to ensure CMT2 is running
2. Attempt internal_time_mutex_init(); on failure return raw counter (no overflow tracking)
3. Acquire s_time_mutex (TX_WAIT_FOREVER); on failure return raw counter
4. Read current_counter from cmt2()->cmcnt
5. If current_counter < last_counter: increment overflow_count
6. Update last_counter = current_counter
7. Compute total_ticks and convert to microseconds
8. Release s_time_mutex
9. Return microsecond result

Returns: uint32_t Monotonic timestamp in microseconds

Value	Description
[0	Microseconds since CMT2 was first initialized; wraps at UINT32_MAX (71 minutes); value increases monotonically between calls (assuming no > 71 minute gap between calls)

Pre: CMT2 hardware is accessible (initialized automatically on first call)

Pre: ThreadX kernel must be running for mutex-protected overflow tracking

Post: CMT2 has been initialized (s_cmt2_initialized == true after first call)

Post: overflow_count is updated if 16-bit counter wrapped since last call

Note: Thread-safe when mutex acquisition succeeds; falls back to unsafe mode otherwise

Warning: Overflow tracking accuracy requires calls more frequent than 8.7 ms; if called less frequently the overflow count may be incorrect

Performance:: Execution time: 5-10 us with mutex @ 240 MHz (ThreadX overhead)

Example:: `uint32_tstart=hcsr04_hal_get_time_us(); uint32_tend=hcsr04_hal_get_time_us();
uint32_techo_us=end-start; floatdistance_cm=(float)echo_us/58.0f;`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (CMT2 accessible, ThreadX running for mutex), 2 postconditions

See also: `hcsr04_hal_delay_us()` Blocking microsecond delay using same CMT2 timer



Figure 996: Call graph for `hcsr04_hal_get_time_us`

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:1367`

Since Version 1.0.0

hcsr04_hal_get_time_us_isr

```
uint32_t hcsr04_hal_get_time_us_isr(void)
```

Get monotonic timestamp in microseconds ISR-safe (no mutex).

Reads the CMT2 counter directly without acquiring the ThreadX mutex. Safe to call from interrupt context. Does NOT track 16-bit overflows. Because the 16-bit CMT2 counter wraps every 8.7 ms (65536 ticks at PCLKB/8 = 7.5 MHz), this function is only valid for echo windows shorter than the CMT2 wrap period. Rising and falling edge timestamps separated by more than 8.7 ms produce invalid durations. HC-SR04 IRQ mode is therefore limited to 150 cm maximum range.

Returns: CMT2 counter converted to microseconds (no overflow tracking)

Value	Description
uint32_t	Value in range [0, 8700) us; wraps every 8.7 ms with CMT2 period

Pre: CMT2 timer initialized (call hcsr04_hal_get_time_us() once at startup)

Pre: Called from interrupt context (no ThreadX blocking calls permitted)

Post: No side effects (read-only, no mutex acquired)

Post: Returned value equals (cmcnt * 1000000) / timer_hz for current CMT2 count

Note: ISR-safe: does NOT call tx_mutex_get() or any blocking ThreadX API

Warning: Does not track 16-bit counter overflow; do not use for intervals > 8.7 ms

Warning: HC-SR04 IRQ mode limited to 150 cm (8.7 ms echo) due to CMT2 wrap period

Example: Edge Timestamp in ISR: uint32_tts=hcsr04_hal_get_time_us_isr();
s_irq_state[idx].start_us=ts; s_irq_state[idx].end_us=ts;

See also: hcsr04_hal_get_time_us() Thread-safe variant with overflow tracking for task context

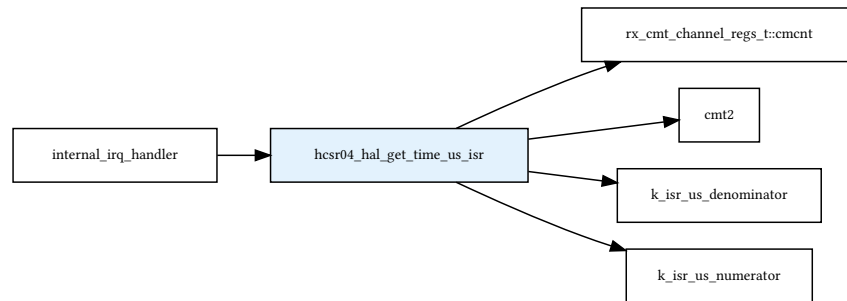


Figure 997: Call graph for hcsr04_hal_get_time_us_isr

Defined in `libs/rx_hcsr04/src/rx_hcsr04_hal_hw.c:1449`

Since Version 1.2.0 (Issue 296 - ISR-mode HC-SR04 measurement)

—

rx_hcsr04_icu.c

HC-SR04 ICU (Interrupt Controller Unit) Configuration Implementation.

Implements ICU configuration for HC-SR04 ultrasonic sensors operating in IRQ mode with hardware edge detection.

Configuration Steps:

1. Validate IRQ number and priority
2. Configure IRQCR for both-edge detection
3. Enable digital filter (PCLK/8 = 133ns @ 60MHz)
4. Clear pending interrupt flag
5. Set interrupt priority
6. Enable interrupt in IER register

STAR Team

2026-02-16

Copyright (c) 2026 STAR Project. MIT License.

1.2.0

Includes:

- "rx_hcsr04_icu.h"
- "rx72n_icu_regs.h"
- "rx_check.h"
- "rx_irq_filter.h"
- "rx_log.h"

icu_constants_t

ICU configuration constants for HC-SR04 edge detection.

Named constants for ICU (Interrupt Controller Unit) register configuration. All magic numbers in register operations must reference this enum per the project "No Magic Numbers" policy.

Register Background (RX72N Manual Chapter 15):

- IRQCR[n] controls edge sensitivity for IRQn

- IER[byte][bit] enables interrupt; byte = vector/8, bit = vector%8
- IR[vector] is the pending-flag register; write 0 to clear
- IPR[vector] sets interrupt priority 1-15 (0 = disabled)

k_irq_min <= all valid IRQ numbers <= k_irq_max

k_priority_min <= all valid priorities <= k_priority_max

k_vector_base + k_irq_max == 75 (last HC-SR04 vector; IRQ11 = vector 75)

Value	Name	Description
= 8	k_irq_min	Minimum IRQ number (IRQ8 = P00)
= 11	k_irq_max	Maximum IRQ number (IRQ11 = P03; only ISR handlers IRQ8-11 exist)
= 1	k_priority_min	Minimum interrupt priority
= 15	k_priority_max	Maximum interrupt priority
= 0x08	k_irqcr_both_edges	Both edges trigger interrupt (IRQCR[n] value)
= 64	k_vector_base	IRQ vector base (IRQ0 = vector 64)
= 8	k_bits_per_ier	IER register is 8 bits wide
= 1	k_bit_base	Bit value 1 for IER enable mask construction
= 0	k_ir_flag_clear	Write 0 to IR register to clear pending flag
= 0	k_ier_bit_clear	IER bit value when interrupt is disabled (not enabled)

See also: rx_hcsr04_icu_configure() Uses all these constants, rx_hcsr04_icu_disable()
Uses k_vector_base, k_bits_per_ier, k_ir_flag_clear, k_ier_bit_clear

Example:

```
//Access all constants in this enum via named identifiers:
const uint8_t vector = k_vector_base + irq_num; //e.g. 72-75 (IRQ8-11)
const uint8_t tier_index = vector / k_bits_per_ier;
const uint8_t tier_bit = vector % k_bits_per_ier;
icu()->ier[tier_index] |= (k_bit_base << tier_bit);
icu()->ir[vector] = k_ir_flag_clear;
```

Since Version 1.2.0 (Issue 296)

—

s_tag

```
const char* const s_tag
```

—

rx_hcsr04_icu_configure

```
rx_err_t rx_hcsr04_icu_configure(const uint8_t irq_num, const uint8_t priority)
```

Configure ICU for HC-SR04 echo IRQ measurement.

Configures IRQ edge detection, digital filter, interrupt priority, and enables the interrupt in the IER register. Must be called after rx_mpc_set_irq() has put the pin in IRQ input mode.

Algorithm Steps:

1. Validate IRQ number in range [k_irq_min, k_irq_max]
2. Validate priority in range [k_priority_min, k_priority_max]
3. Write k_irqcr_both_edges to IRQCR[irq_num]
4. Enable digital filter via rx_irq_filter_enable()
5. Calculate vector = k_vector_base + irq_num
6. Clear IR[vector] flag (discard any pending interrupt)
7. Write priority to IPR[vector]
8. Set IER[vector/8] bit (vector%8) to enable interrupt

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11 for P00-P03; ISR handlers exist for IRQ8-11 only)
in	priority	Interrupt priority (1-15; recommend 10)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	ICU configured successfully
k_rx_err_invalid_arg	irq_num not in [8, 11] or priority not in [1, 15]
k_rx_err_invalid_arg	irq_num not valid for digital filter (propagated from rx_irq_filter_enable())

Pre: Pin already configured for IRQ function via rx_mpc_set_irq()**Pre:** ICU module not in module stop (MSTPCRA bit cleared)**Post:** IRQCRirq_num set for both-edge detection**Post:** Digital filter enabled at PCLK/8**Post:** IRvector cleared (no stale pending interrupt)**Post:** IPRvector contains priority**Post:** IERvector/8 bit set (interrupt enabled)**Note:** Not thread-safe. Call during initialization only.

Warning: Do not call while a measurement is in progress.] **Example:**
 rx_err_t err=rx_mpc_set_irq(k_rx_p0_3); if(err!=k_rx_ok){return err;}


```
err=rx_hcsr04_icu_configure((uint8_t)k_hcsr04_irq_11,k_hcsr04_irq_priority_default); if(err!=k_rx_ok){returnerr;}
```

See also: `rx_mpc_set_irq()` Configure pin before calling this, `rx_hcsr04_icu_disable()`
Reverse this configuration

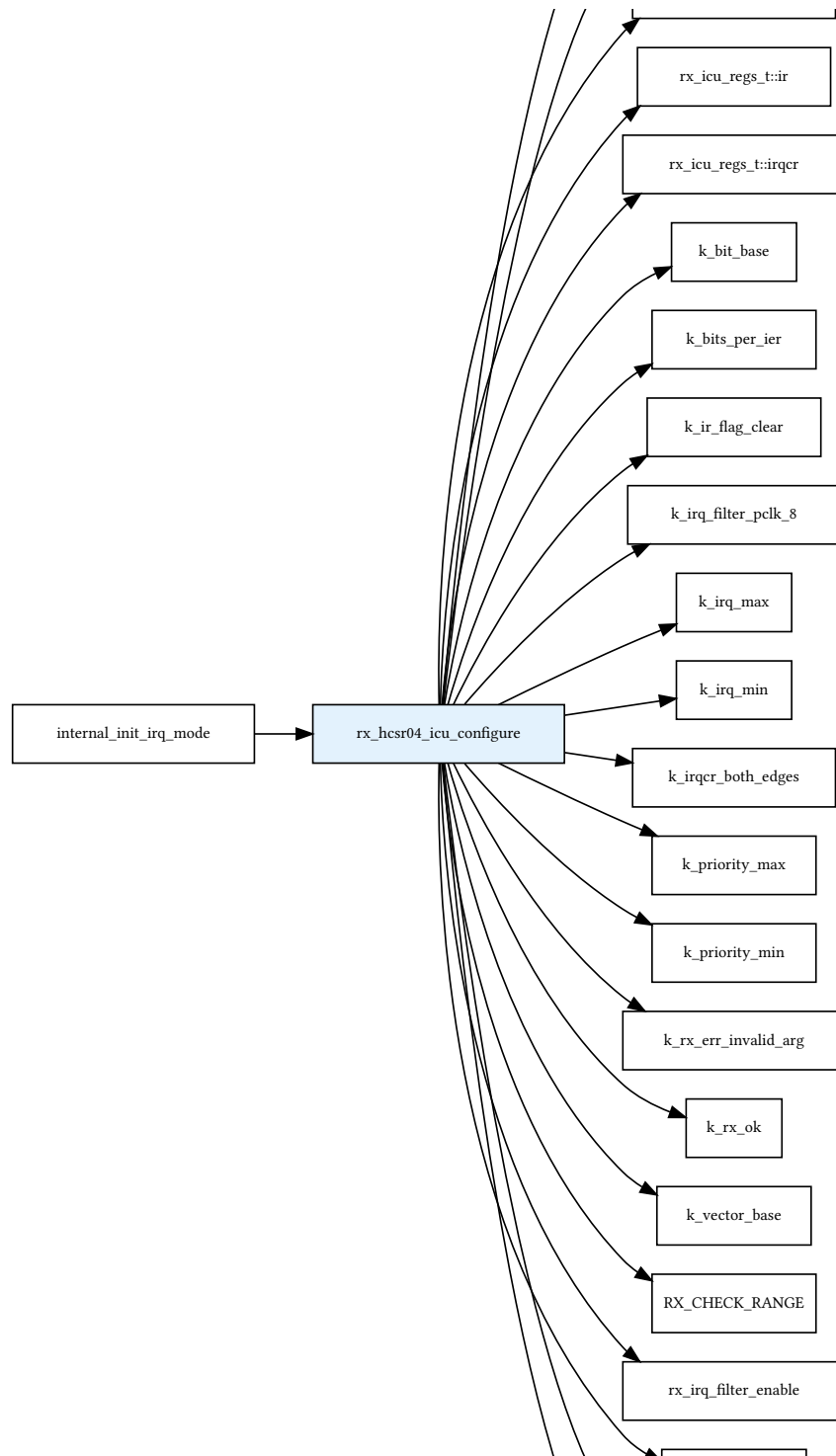


Figure 998: Call graph for rx_hcsr04_icu_configure

_Defined in libs/rx_hcsr04/src/rx_hcsr04_icu.c:160_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_icu_disable

```
rx_err_t rx_hcsr04_icu_disable(const uint8_t irq_num)
```

Disable HC-SR04 echo IRQ in ICU.

Disables the specified IRQ interrupt and digital filter. Performs all cleanup steps then returns the first error encountered (if any).

Algorithm Steps:

1. Validate IRQ number in range [k_irq_min, k_irq_max]
2. Calculate vector = k_vector_base + irq_num
3. Clear IER[vector/8] bit (vector%8) to disable interrupt
4. Write k_ir_flag_clear to IR[vector] to clear any pending flag
5. Disable digital filter via rx_irq_filter_disable() and return its error

Parameters:

Direction	Name	Description
in	irq_num	IRQ number to disable (8-11)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	IRQ and digital filter both disabled successfully
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	IRQ was not enabled in IER (rx_hcsr04_icu_configure() not called)
k_rx_err_invalid_arg	irq_num not valid for digital filter (propagated from rx_irq_filter_disable())

Pre: IRQ was previously enabled via rx_hcsr04_icu_configure() (IER bit must be set)

Pre: No measurement in progress on this IRQ

Post: IER bit cleared (interrupt will not fire)

Post: IR flag cleared (no stale pending interrupt)

Post: Digital filter disabled

Post: ISR will not trigger on pin edges

Note: Only safe to call after `rx_hcsr04_icu_configure()` has enabled the IRQ

Note: Does NOT reconfigure pin back to GPIO (use `rx_mpc_set_gpio` separately)

Warning: Do not call while measurement in progress

Warning: Returns `k_rx_err_invalid_state` if IRQ was not enabled; call only after `configure()`]

Example: `rx_err_t err=rx_hcsr04_icu_disable((uint8_t)k_hcsr04_irq_11); if(err!=k_rx_ok)
{rx_log_warn("SONAR","Failed to disable IRQ11");}`

See also: `rx_hcsr04_icu_configure()` Enable IRQ, `rx_mpc_set_gpio()` Reconfigure pin back to GPIO mode

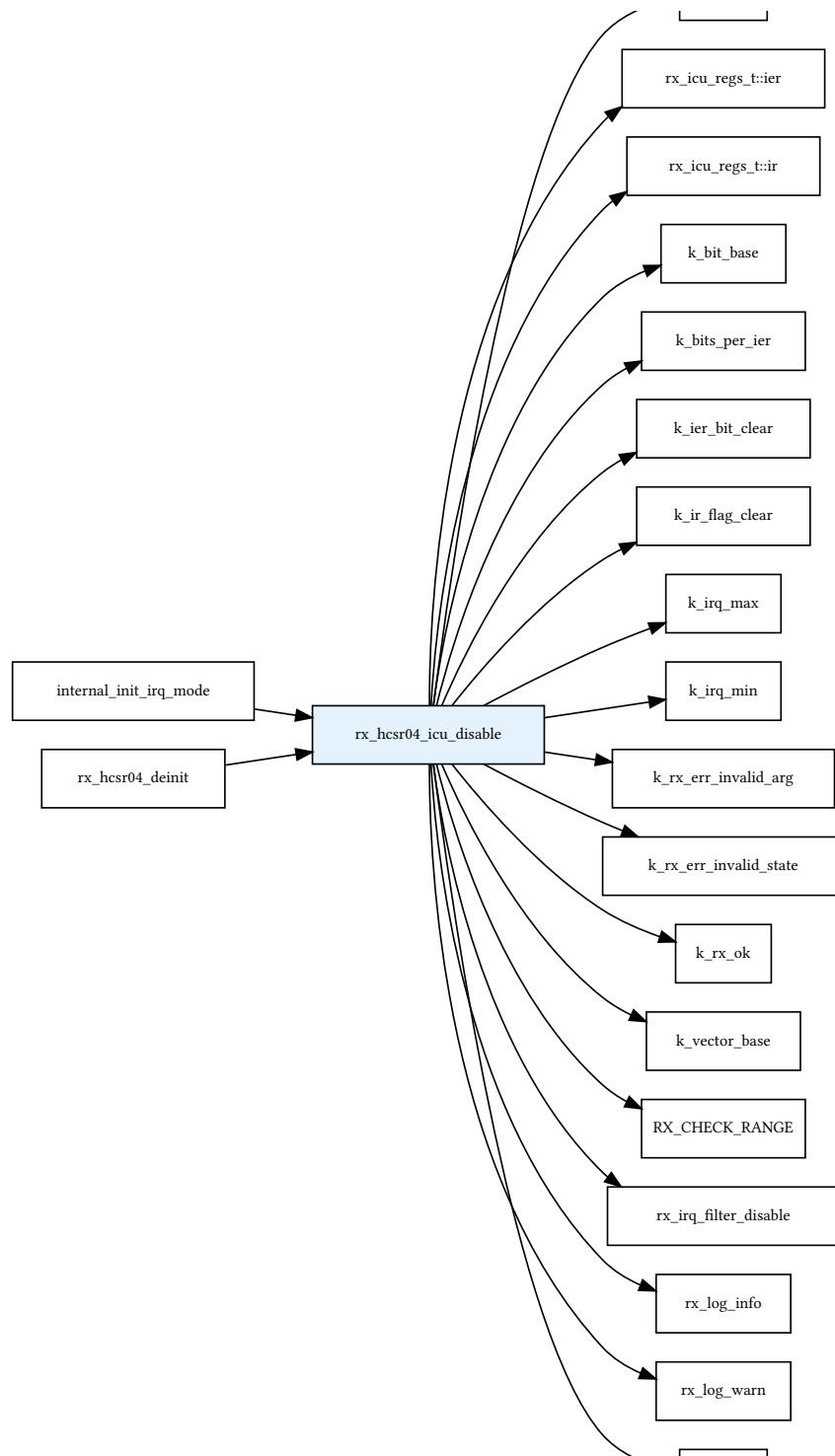


Figure 999: Call graph for rx_hcsr04_icu_disable

_Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_icu.c:246_`

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_icu.h

HC-SR04 ICU (Interrupt Controller Unit) Configuration Layer.

Provides ICU configuration functions for HC-SR04 ultrasonic sensors operating in IRQ mode. Configures external interrupt pins (IRQ8-11) for hardware edge detection with microsecond-precision timing.

Responsibilities:

- Configure IRQCR for edge detection (both edges)
- Enable digital filtering to debounce mechanical noise
- Set interrupt priority (typical: 10)
- Enable/disable IRQ in ICU IER register

Usage Flow:

1. Configure pin for IRQ function via `rx_mpc_set_irq()`
2. Call `rx_hcsr04_icu_configure()` to set up ICU
3. ISR captures rising and falling edges automatically
4. Call `rx_hcsr04_icu_disable()` to clean up

STAR Team

2026-02-16

Copyright (c) 2026 STAR Project. MIT License.

1.2.0

Includes:

- `<stdint.h>`
- `"rx_err.h"`

rx_hcsr04_icu_configure

```
rx_err_t rx_hcsr04_icu_configure(uint8_t irq_num, uint8_t priority)
```

Configure ICU for HC-SR04 echo IRQ measurement.

Sets up IRQ edge detection, digital filter, priority, and enables interrupt. Uses both-edge detection to capture rising (start) and falling (end) edges of the echo pulse automatically.

Configuration Applied:

- `IRQCR[n] = k_irqcr_both_edges` (both edges trigger interrupt)
- Digital filter enabled ($PCLK/8 = 133\text{ns}$ @ 60MHz PCLKB)
- Interrupt priority set to specified value (recommend 10)
- IR flag cleared (any pending interrupt discarded)
- IER bit set (interrupt enabled)

Edge Detection:

- Rising edge (echo HIGH): ISR captures start timestamp
- Falling edge (echo LOW): ISR captures end timestamp
- Duration = end - start (microseconds)

Configures IRQ edge detection, digital filter, interrupt priority, and enables the interrupt in the IER register. Must be called after `rx_mpc_set_irq()` has put the pin in IRQ input mode.

Algorithm Steps:

1. Validate IRQ number in range `[k_irq_min, k_irq_max]`
2. Validate priority in range `[k_priority_min, k_priority_max]`
3. Write `k_irqcr_both_edges` to `IRQCR[irq_num]`
4. Enable digital filter via `rx_irq_filter_enable()`
5. Calculate vector = `k_vector_base + irq_num`
6. Clear `IR[vector]` flag (discard any pending interrupt)
7. Write priority to `IPR[vector]`
8. Set `IER[vector/8]` bit (`(vector%8)` to enable interrupt

Parameters:

Direction	Name	Description
in	<code>irq_num</code>	IRQ number (8-11 for P00-P03; ISR handlers exist for IRQ8-11 only) IRQ8 (P00) - Sonar 3 (Back-Right) IRQ9 (P01) - Sonar 2 (Back-Left) IRQ10 (P02) - Sonar 1 (Front-Right) IRQ11 (P03) - Sonar 0 (Front-Left)
in	<code>priority</code>	Interrupt priority (1-15) 1 = Lowest priority 10 = Recommended (balances latency and preemption) 15 = Highest priority (use sparingly)
in	<code>irq_num</code>	IRQ number (8-11 for P00-P03; ISR handlers exist for IRQ8-11 only)
in	<code>priority</code>	Interrupt priority (1-15; recommend 10)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	ICU configured successfully
<code>k_rx_err_invalid_arg</code>	<code>irq_num</code> not in range <code>[8, 11]</code>
<code>k_rx_err_invalid_arg</code>	priority not in range <code>[1, 15]</code>
<code>k_rx_err_invalid_arg</code>	<code>irq_num</code> not valid for digital filter (propagated from <code>rx_irq_filter_enable()</code>)
<code>k_rx_ok</code>	ICU configured successfully
<code>k_rx_err_invalid_arg</code>	<code>irq_num</code> not in <code>[8, 11]</code> or priority not in <code>[1, 15]</code>
<code>k_rx_err_invalid_arg</code>	<code>irq_num</code> not valid for digital filter (propagated from <code>rx_irq_filter_enable()</code>)

Pre: Pin already configured for IRQ function via `rx_mpc_set_irq()`

Pre: ICU module not in module stop (MSTPCRA bit cleared)

Pre: No other driver using this IRQ number

Pre: Pin already configured for IRQ function via `rx_mpc_set_irq()`

Pre: ICU module not in module stop (MSTPCRA bit cleared)

Post: IRQCRirq_num configured for both-edge detection

Post: Digital filter enabled (PCLK/8)

Post: Interrupt priority set

Post: ISR will trigger on rising and falling edges

Post: IRQCRirq_num set for both-edge detection

Post: Digital filter enabled at PCLK/8

Post: IRRvector cleared (no stale pending interrupt)

Post: IPRvector contains priority

Post: IERvector/8 bit set (interrupt enabled)

Note: Call this AFTER rx_mpc_set_irq() (pin must be in IRQ mode first)

Note: Thread-safe: Can be called during initialization only

Note: Digital filter provides 133ns noise immunity @ 60MHz PCLKB

Note: Not thread-safe. Call during initialization only.

Warning: Do not call while measurement in progress

Warning: Ensure ISR handler is registered before enabling interrupt

Warning: Do not call while a measurement is in progress.

Example: Configure All 4 HC-SR04 Sensors: `constrx_hcsr04_irq_tirq_nums[]={ k_hcsr04_irq_11, k_hcsr04_irq_10, k_hcsr04_irq_9, k_hcsr04_irq_8, }; constuint8_tsensor_count=(uint8_t)(sizeof(irq_nums)/sizeof(irq_nums[0])); constuint8_tpriority=k_hcsr04_irq_priority_default;`


```
for(uint8_ti=0;i<sensor_count;i++)  
{ rx_err_t err=rx_hcsr04_icu_configure((uint8_t)irq_nums[i],priority); if(err!=k_rx_ok)  
{ rx_log_error("SONAR","ICUconfigfailed"); return err; } }
```

NASA Power of 10 Compliance: Rule 5: [OK] 3 preconditions, 4 postconditions Rule 7: [OK] All register writes validated Rule 8: [OK] Typed enums for all constants

Example: rx_err_t err=rx_mpc_set_irq(k_rx_p0_3); if(err!=k_rx_ok){return err;}
err=rx_hcsr04_icu_configure((uint8_t)k_hcsr04_irq_11,k_hcsr04_irq_priority_default); if(err!=k_rx_ok){return err;}

See also: rx_mpc_set_irq() Must be called first to configure pin,
rx_hcsr04_icu_disable() Disable IRQ when done, rx_hcsr04_isr.h ISR handler receiving edge events, RX72N Manual Chapter 15.2 - External Interrupts (IRQ), rx_mpc_set_irq()
Configure pin before calling this, rx_hcsr04_icu_disable() Reverse this configuration

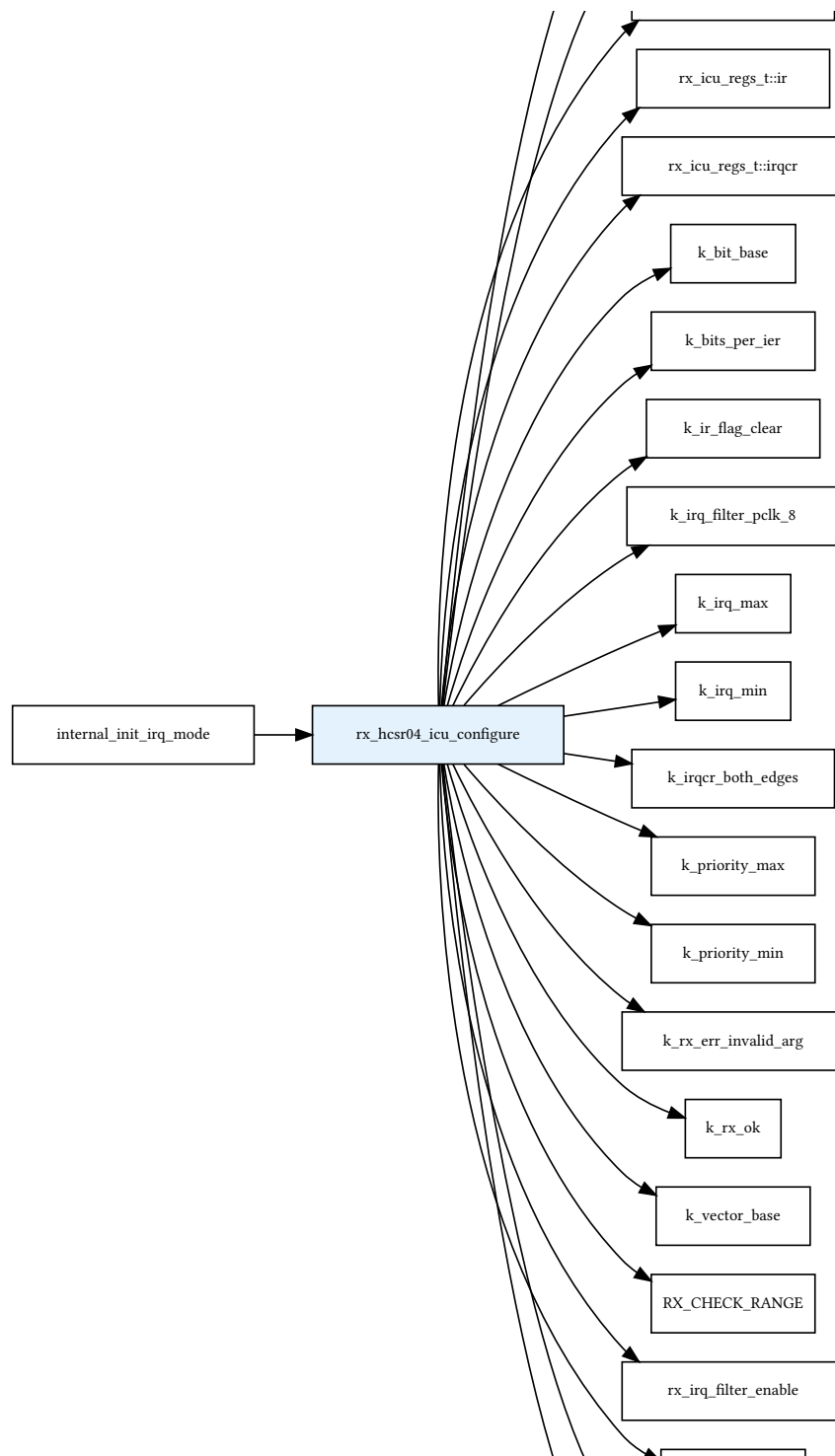


Figure 1000: Call graph for rx_hcsr04_icu_configure

_Defined in libs/rx_hcsr04/src/rx_hcsr04_icu.h:171_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_icu_disable

```
rx_err_t rx_hcsr04_icu_disable(uint8_t irq_num)
```

Disable HC-SR04 echo IRQ in ICU.

Disables the specified IRQ interrupt and digital filter. Use this during sensor deinitialization or when switching from IRQ mode back to polling mode.

Operations Performed:

- Clear IER bit (interrupt disabled)
- Clear IR flag (discard any pending interrupt)
- Disable digital filter
- IRQCR left unchanged (safe state)

Disables the specified IRQ interrupt and digital filter. Performs all cleanup steps then returns the first error encountered (if any).

Algorithm Steps:

1. Validate IRQ number in range [k_irq_min, k_irq_max]
2. Calculate vector = k_vector_base + irq_num
3. Clear IER[vector/8] bit (vector%8) to disable interrupt
4. Write k_ir_flag_clear to IR[vector] to clear any pending flag
5. Disable digital filter via rx_irq_filter_disable() and return its error

Parameters:

Direction	Name	Description
in	irq_num	IRQ number to disable (8-11)
in	irq_num	IRQ number to disable (8-11)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	IRQ and digital filter both disabled successfully
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	IRQ was not enabled in IER (rx_hcsr04_icu_configure() not called first)
k_rx_err_invalid_arg	irq_num not valid for digital filter (propagated from rx_irq_filter_disable())
k_rx_ok	IRQ and digital filter both disabled successfully
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	IRQ was not enabled in IER (rx_hcsr04_icu_configure() not called)

k_rx_err_invalid_arg	irq_num not valid for digital filter (propagated from rx_irq_filter_disable())
Pre: IRQ was previously enabled via rx_hcsr04_icu_configure() (IER bit must be set)	
Pre: No measurement in progress on this IRQ	
Pre: IRQ was previously enabled via rx_hcsr04_icu_configure() (IER bit must be set)	
Pre: No measurement in progress on this IRQ	
Post: IER bit cleared (interrupt will not fire)	
Post: IR flag cleared (no stale pending interrupt)	
Post: Digital filter disabled	
Post: ISR will not trigger on pin edges	
Post: IER bit cleared (interrupt will not fire)	
Post: IR flag cleared (no stale pending interrupt)	
Post: Digital filter disabled	
Post: ISR will not trigger on pin edges	
Note: Call only after rx_hcsr04_icu_configure() has enabled the IRQ	
Note: Does NOT reconfigure pin back to GPIO (use rx_mpc_set_gpio separately)	
Note: Only safe to call after rx_hcsr04_icu_configure() has enabled the IRQ	
Note: Does NOT reconfigure pin back to GPIO (use rx_mpc_set_gpio separately)	
Warning: Do not call while measurement in progress	

Warning: Returns `k_rx_err_invalid_state` if the IRQ IER bit is not set; always call `configure()` first

Warning: Do not call while measurement in progress

Warning: Returns `k_rx_err_invalid_state` if IRQ was not enabled; call only after `configure()`

Example: Clean Up Sensor: `rx_err_t err=rx_hcsr04_icu_disable(k_hcsr04_irq_11); if(err!=k_rx_ok){ rx_log_warn("SONAR","Failed to disable IRQ11"); }`

```
(void)rx_mpc_set_gpio(k_rx_p0_3);
```

NASA Power of 10 Compliance: Rule 5: [OK] 2 preconditions (IER enabled + no measurement in progress), 4 postconditions Rule 7: [OK] IER state validated before clearing; `rx_irq_filter_disable()` return propagated Rule 8: [OK] Typed enums for all constants (`icu_constants_t`)

Example: `rx_err_t err=rx_hcsr04_icu_disable((uint8_t)k_hcsr04_irq_11); if(err!=k_rx_ok){ rx_log_warn("SONAR","Failed to disable IRQ11"); }`

See also: `rx_hcsr04_icu_configure()` Enable IRQ, `rx_mpc_set_gpio()` Reconfigure pin back to GPIO mode, `rx_hcsr04_icu_configure()` Enable IRQ, `rx_mpc_set_gpio()` Reconfigure pin back to GPIO mode

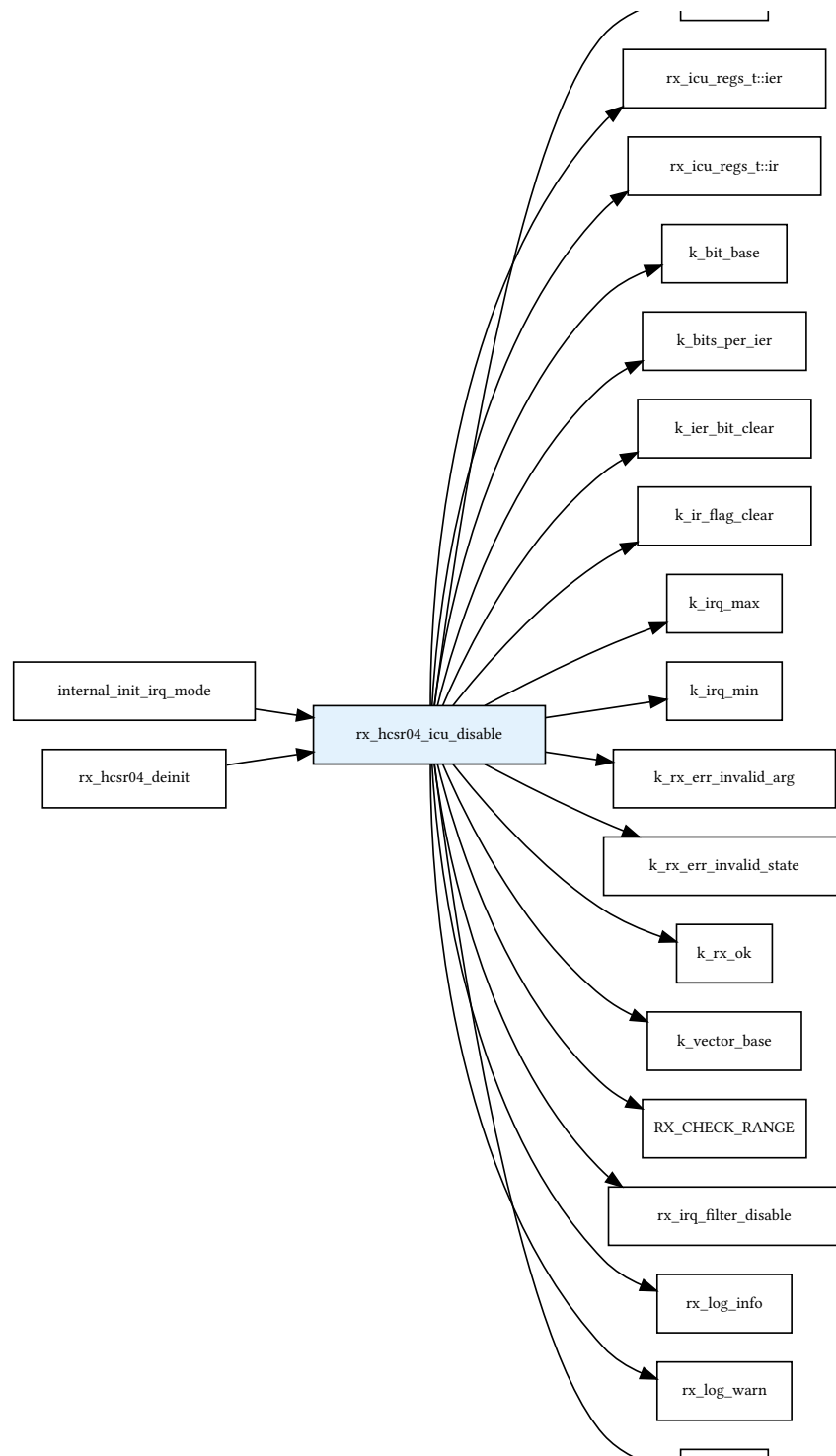


Figure 1001: Call graph for rx_hcsr04_icu_disable

_Defined in libs/rx_hcsr04/src/rx_hcsr04_icu.h:231_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr.c

HC-SR04 ISR (Interrupt Service Routine) Handler Implementation.

Implements ISR handlers for HC-SR04 ultrasonic sensors operating in IRQ mode. Captures rising and falling edge timestamps with microsecond precision.

ISR Design:

- Keep ISR execution time minimal (< 5us)
- No blocking operations in ISR
- No ThreadX calls in ISR (use TX_INTERRUPT_SAVE_AREA if needed)
- Clear interrupt flag before exit

STAR Team

2026-02-16

Copyright (c) 2026 STAR Project. MIT License.

1.2.0

Includes:

- "rx_hcsr04_isr.h"
- "rx72n_icu_regs.h"
- "rx72n_port_regs.h"
- "rx_check.h"
- "rx_hcsr04_hal.h"

isr_constants_t

ISR configuration and state machine constants.

Named constants for ISR (Interrupt Service Routine) logic. All magic literals in this module must reference this enum.

Sensor Map: s_sensor_map[] has k_sensor_map_size entries. Entries initialized to k_sensor_unused and set via rx_hcsr04_isr_register(). Only indices [k_irq_min..k_irq_max] (8-11) are valid IRQ numbers.

IRQ Handler: Pin state is read from PORT0->PIDR register; the pin bit is extracted by shifting right by (irq_num - k_irq_min), then masking with k_pin_state_mask.

k_sensor_map_size > k_irq_max (map covers all valid IRQ indices)

k_irq_count == k_irq_max - k_irq_min + 1 == 4

Value	Name	Description
= 8	k_irq_min	Minimum IRQ number (IRQ8 = P00)
= 11	k_irq_max	Maximum IRQ number (IRQ11 = P03; only ISR handlers IRQ8-11 exist)
= k_irq_max - k_irq_min + 1	k_irq_count	Number of supported IRQs (4: IRQ8-11)

= 64	k_vector_base	IRQ vector base (IRQ0 = vector 64)
= 0xFF	k_sensor_unused	Sentinel: sensor slot not assigned
= 16	k_sensor_map_size	Total entries in s_sensor_map (IRQ0-15)
= 0x01	k_pin_state_mask	Bit mask to extract single pin state
= 0	k_ir_flag_clear	Write 0 to IR register to clear pending interrupt flag

See also: internal_irq_handler() Uses k_vector_base, k_irq_min, k_pin_state_mask, rx_hcsr04_isr_register() Uses k_sensor_map_size as array bound

Example:

```
//TypicalISRhandlerusingtheseconstants:
voidINT_IRQ11(void)
{
    internal_irq_handler(k_irq_max); //IRQ11=maxIRQ
}

//ValidateIRQnumberbeforeindexing:
if(irq_num<k_irq_min||irq_num>k_irq_max){returnk_rx_err_invalid_arg;}
constuint8_tidx=irq_num-k_irq_min; //IRQ8->0, IRQ11->3
```

Since Version 1.2.0 (Issue 296)

—

isr_timer_constants_t

CMT2 timer wrap period for ISR timestamp correction.

The CMT2 16-bit counter wraps every 65536 ticks at PCLKB/8 = 7.5 MHz. Wrap period = 65536 / 7500000 s = 8738 us. hcsr04_hal_get_time_us_isr() returns values in [0, 8738]. When the falling edge timestamp wraps around to a value smaller than the rising edge timestamp, this constant is added to correct the duration calculation.

Clock dependency: This constant assumes PCLKB = 60 MHz (k_pelkb_hz in rx72n_clock.h) and CMT2 divider = 8 (k_cmt2_divider in rx_hcsr04_hal_hw.c). If either value changes, k_cmt2_wrap_us MUST be recalculated: k_cmt2_wrap_us = (65536 * 1000000) / (PCLKB / divider)

k_cmt2_wrap_us == (65536 * 1000000) / (60000000 / 8) == 8738

Assumes PCLKB = 60 MHz and CMT2 divider = 8; update if clock config changes

Value	Name	Description
= 8738	k_cmt2_wrap_us	CMT2 16-bit counter wrap period in us (65536 / 7.5 MHz)

See also: hcsr04_hal_get_time_us_isr() Returns values that wrap at this period, rx_hcsr04_isr_get_duration() Uses this for wrap correction

Example:

```
//Wrapcorrectioninget_duration:
if(end_us<start_us){
    duration=end_us+k_cmt2_wrap_us-start_us;
}
```


Since Version 1.2.0 (Issue 296)

—

isr_irq_numbers_t

Named IRQ number constants for ISR wrapper functions.

Provides named constants for the literal IRQ numbers passed to `internal_irq_handler()` from each ISR wrapper. Replaces magic literals 8, 9, 10, 11 per the project “No Magic Numbers” policy.

Values are fixed 8..11, matching P00..P03 pin assignments; each value must equal `k_irq_min + pin_number` (P0x -> IRQ(8+x))

Value	Name	Description
= 8	<code>k_irq_num_8</code>	IRQ8 - maps to P00 (Sonar 3 Back-Right)
= 9	<code>k_irq_num_9</code>	IRQ9 - maps to P01 (Sonar 2 Back-Left)
= 10	<code>k_irq_num_10</code>	IRQ10 - maps to P02 (Sonar 1 Front-Right)
= 11	<code>k_irq_num_11</code>	IRQ11 - maps to P03 (Sonar 0 Front-Left)

See also: `isr_constants_t` Range bounds (`k_irq_min`, `k_irq_max`) and state constants, `internal_irq_handler()` Common ISR logic receiving these constants

Example:

```
//EachISRwrapperpassesitsnamedconstanttothecommonhandler:
voidINT_IRQ8(void){internal_irq_handler(k_irq_num_8);}
voidINT_IRQ11(void){internal_irq_handler(k_irq_num_11);}
```

Since Version 1.2.0 (Issue 296)

—

s_irq_state

```
volatile rx_hcsr04_irq_state_t s_irq_state[k_irq_count]
```

Per-IRQ echo measurement state (IRQ8-11 -> array indices 0-3).

Note: Access pattern: ISR writes volatile fields; application reads under timeout loop

Warning: Never access outside of `rx_hcsr04_isr_start()`, `internal_irq_handler()`, and `rx_hcsr04_isr_get_duration()`; direct access bypasses state validation

Since Version 1.2.0 (Issue 296)

—

s_sensor_map

```
uint8_t s_sensor_map[k_sensor_map_size]
```

Sensor index mapping table (IRQ number -> sensor array index).

Note: Only indices `k_irq_min..k_irq_max` (8-11) are used by HC-SR04 sensors

Note: Indices 0..7 and 12..15 are always k_sensor_unused for HC-SR04 use

Note: Issue #336 (resolved): sensor_index is now correctly set per-sensor via rx_hcsr04_config_t.sensor_index (no longer hardcoded to slot 0). s_sensor_map is written with the correct index but not yet read in the ISR; future work will use this mapping to dispatch per-sensor callbacks from internal_irq_handler() when multi-sensor callback support is implemented.

Warning: Direct modification outside rx_hcsr04_isr_register/unregister is forbidden; use the public API to maintain consistent IRQ-to-sensor mappings

Since Version 1.2.0 (Issue 336)

—

internal_irq_handler

```
static void internal_irq_handler(const uint8_t irq_num)
```

Internal IRQ handler (common logic for all IRQs).

Shared ISR logic called by INT_IRQ8 through INT_IRQ11. Determines edge type (rising or falling) by reading GPIO pin state, then captures timestamp.

Algorithm:

1. Clear interrupt flag (acknowledge hardware)
2. Compute array index: idx = irq_num - k_irq_min
3. Check if measurement is active (ignore spurious interrupts)
4. Read GPIO pin state: pin_state = (PORT0.PIDR >> idx) & k_pin_state_mask
5. Rising edge (HIGH): Capture start_us timestamp
6. Falling edge (LOW): Capture end_us timestamp, set complete=true, active=false

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11)

Returns: void (ISR context - no return value)

Pre: irq_num must be in range k_irq_min, k_irq_max

Pre: ICU configured via rx_hcsr04_icu_configure()

Post: IR flag cleared for this vector

Post: If active and rising edge: start_us timestamp captured

Post: If active and falling edge: end_us captured, complete=true, active=false

Note: Called from ISR context - keep execution time minimal (< 5us)

Note: Assumes PORT0 pins (P00-P07) map to IRQ8-15

Warning: Do not call from task context] **See also:** rx_hcsr04_isr_start() Sets active=true before trigger pulse, rx_hcsr04_isr_get_duration() Reads complete flag and timestamps

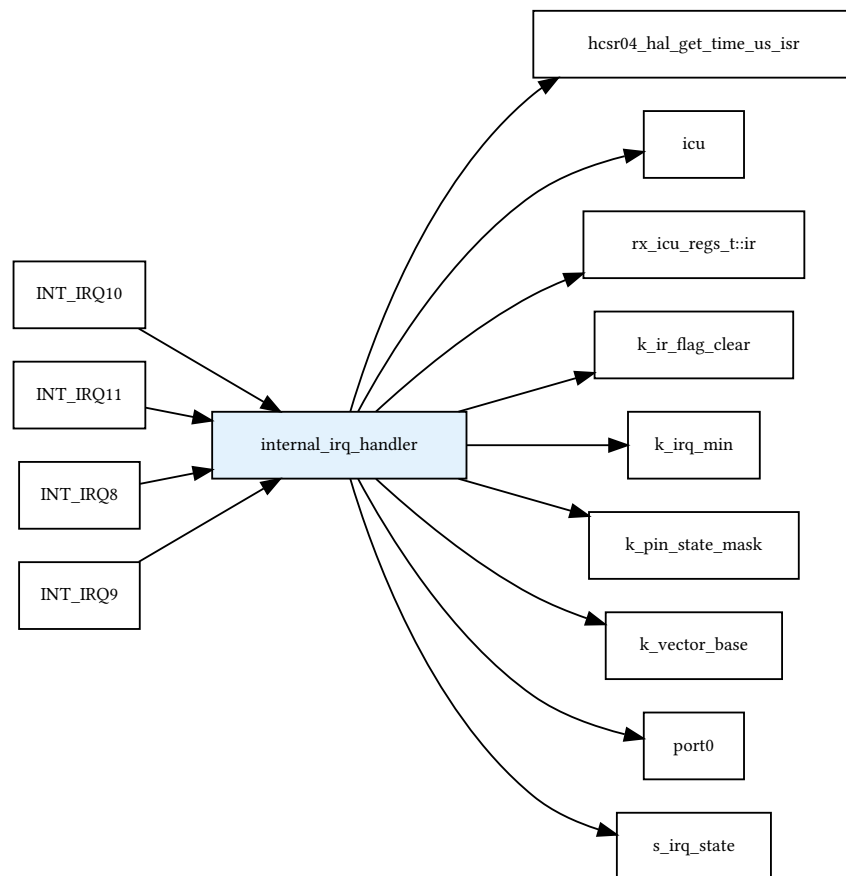


Figure 1002: Call graph for internal_irq_handler

_Defined in libs/rx_hcsr04/src/rx_hcsr04_isr.c:241_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_register

```
rx_err_t rx_hcsr04_isr_register(const uint8_t irq_num, const
rx_hcsr04_sensor_index_t sensor_index)
```

Register HC-SR04 sensor for IRQ echo measurement.

Maps an IRQ number to a sensor index. Validates both IRQ number range and sensor_index range before storing the mapping. Used to identify which sensor triggered a given interrupt.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11 for P00-P03)
in	sensor_index	Sensor array index (0-3 for 4 HC-SR04 sensors)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Registration successful
k_rx_err_invalid_arg	irq_num not in [k_irq_min, k_irq_max]
k_rx_err_invalid_arg	sensor_index >= k_hcsr04_sensor_count

Pre: IRQ configured via rx_hcsr04_icu_configure()

Pre: sensor_index must be a valid sensor array index

Post: s_sensor_mapirq_num updated with sensor_index

Post: ISR can identify sensor on next interrupt

Note: Call once during sensor initialization

Note: Not thread-safe; call during initialization only] **See also:** rx_hcsr04_isr_start() Arm ISR before sending trigger pulse

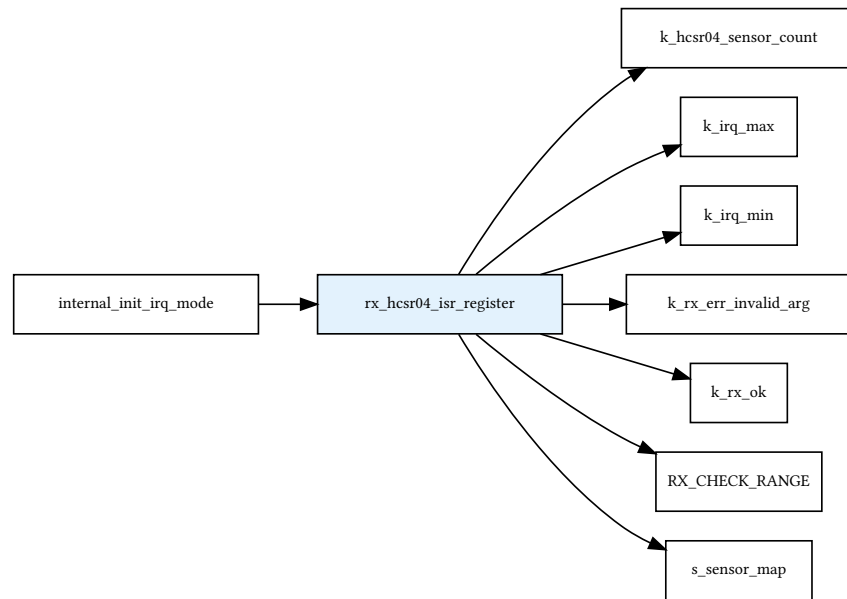


Figure 1003: Call graph for rx_hcsr04_isr_register

_Defined in libs/rx_hcsr04/src/rx_hcsr04_isr.c:306_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_unregister

```
rx_err_t rx_hcsr04_isr_unregister(const uint8_t irq_num)
```

Unregister HC-SR04 sensor from IRQ echo measurement.

Clears the sensor mapping for the given IRQ number, restoring it to the k_sensor_unused sentinel. Call this during sensor deinitialization to prevent stale ISR callbacks after the sensor handle is invalidated.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11) to unregister

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Unregistration successful
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	IRQ slot was not registered (s_sensor_map entry is k_sensor_unused)

Pre: IRQ was previously registered via rx_hcsr04_isr_register()

Pre: No measurement currently active on this IRQ

Post: s_sensor_mapirq_num = k_sensor_unused

Post: ISR will ignore subsequent interrupts on this IRQ

Note: Call during sensor deinitialization, after rx_hcsr04_icu_disable()

Note: Not thread-safe; call during single-threaded cleanup only] **See also:**
 rx_hcsr04_isr_register() Register sensor, rx_hcsr04_deinit() Calls this during cleanup in IRQ mode

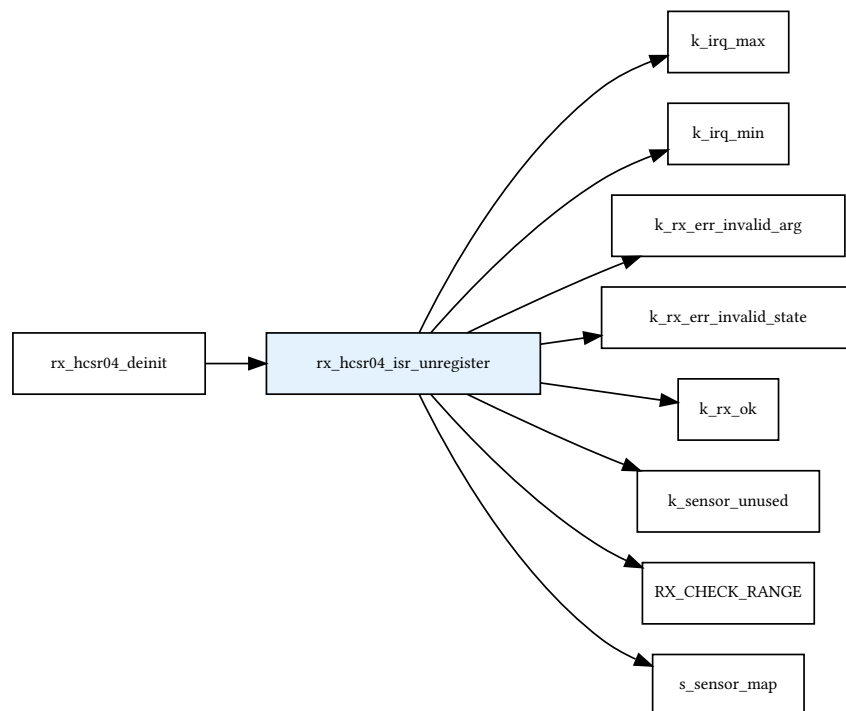


Figure 1004: Call graph for rx_hcsr04_isr_unregister

_Defined in libs/rx\hcsr04/src/rx\hcsr04_isr.c:351_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_start

```
rx_err_t rx_hcsr04_isr_start(const uint8_t irq_num)
```

Start new echo measurement (arm ISR before sending trigger pulse).

Start new echo measurement (call before trigger pulse).

Prepares ISR state for a new measurement. Validates that the IRQ is registered, then sets complete=false first (to prevent a stale complete flag being read), then sets active=true to allow ISR to capture edges. Order matters: complete must be cleared before active is set to prevent a race where ISR fires between the two writes.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	ISR armed successfully
k_rx_err_invalid_arg	irq_num not in valid range [k_irq_min, k_irq_max]
k_rx_err_invalid_state	irq_num slot not registered (s_sensor_map[irq_num] == k_sensor_unused)

Pre: rx_hcsr04_isr_register() called for this IRQ number

Pre: No measurement currently active on this IRQ

Post: s_irq_stateidx.complete = false (on k_rx_ok)

Post: s_irq_stateidx.active = true (on k_rx_ok)

Post: ISR is armed to capture rising edge (on k_rx_ok)

Note: Always call BEFORE sending trigger pulse (ordering is critical)

Note: complete cleared BEFORE active set (avoids race condition)] **See also:**
rx_hcsr04_isr_get_duration() Poll for completion after this call

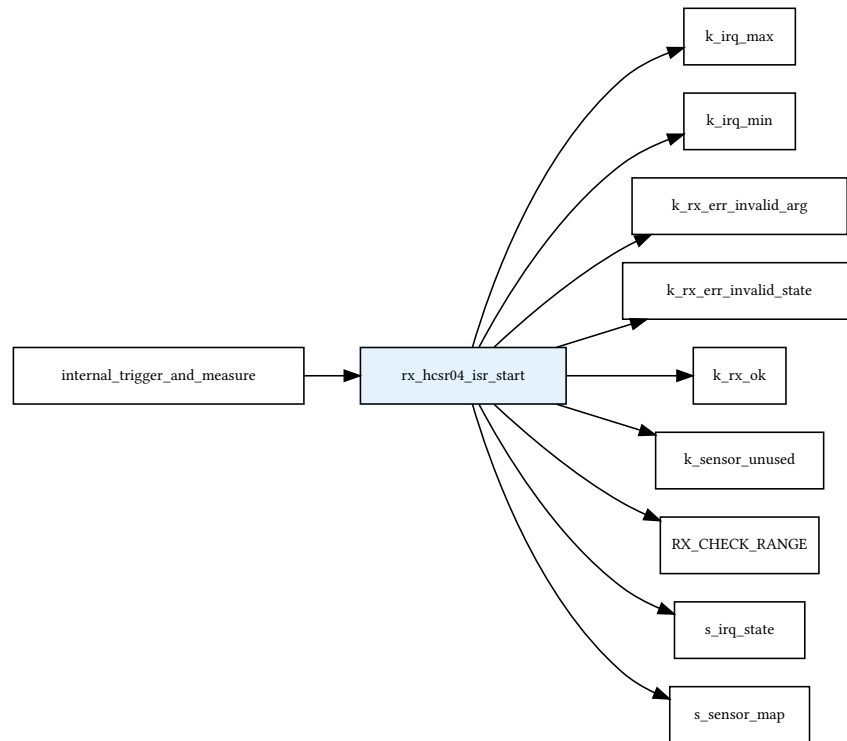


Figure 1005: Call graph for rx_hcsr04_isr_start

Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_isr.c:396`

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_disarm

```
rx_err_t rx_hcsr04_isr_disarm(const uint8_t irq_num)
```

Disarm ISR state machine (cancel armed measurement on trigger failure).

Disarm ISR state machine (call on trigger failure after arm).

Clears the active flag so the ISR will ignore subsequent edge interrupts. Call this in the error path when `rx_hcsr04_isr_start()` succeeded but the trigger pulse failed, to prevent the ISR from remaining armed indefinitely.

Parameters:

Direction	Name	Description
in	<code>irq_num</code>	IRQ number (8-11) to disarm

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	ISR disarmed successfully
k_rx_err_invalid_arg	irq_num not in range [k_irq_min, k_irq_max]
k_rx_err_invalid_state	irq_num not registered via rx_hcsr04_isr_register()

Pre: rx_hcsr04_isr_start() called successfully for this IRQ

Pre: irq_num registered via rx_hcsr04_isr_register() (s_sensor_mapirq_num != k_sensor_unused)

Post: s_irq_stateidx.active = false

Post: s_irq_stateidx.complete = false

Post: ISR will ignore subsequent interrupts on this IRQ

Note: Best-effort cleanup; always call in error path after successful isr_start] **See also:** rx_hcsr04_isr_start() Arm ISR state before trigger pulse

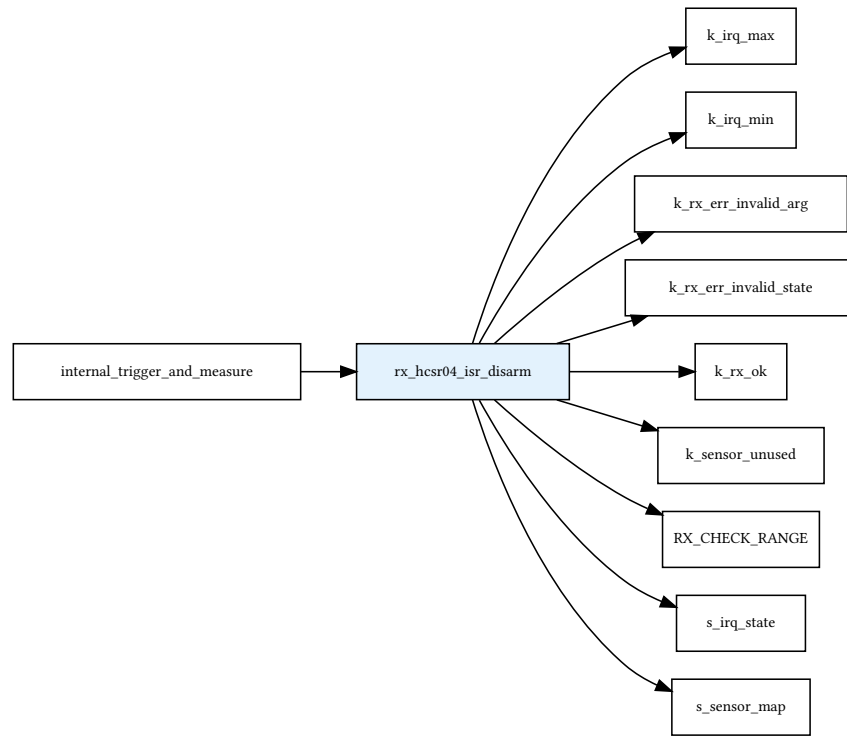


Figure 1006: Call graph for rx_hcsr04_isr_disarm

_Defined in libs/rx_hcsr04/src/rx_hcsr04_isr.c:443_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_get_duration

```
rx_err_t rx_hcsr04_isr_get_duration(const uint8_t irq_num, uint32_t *const
duration_us)
```

Get echo pulse duration from ISR state.

Reads the captured timestamps and calculates echo pulse duration. Returns k_rx_err_timeout (not yet ready) if the ISR has not captured both edges. On success, clears the complete flag to prepare for the next measurement.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11)
out	duration_us	Pointer to store pulse duration in microseconds (valid range: 150-8700 us for 2-150 cm in IRQ mode; CMT2 wrap handled automatically for durations < k_cmt2_wrap_us)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Duration available, *duration_us written, complete flag cleared
k_rx_err_timeout	Both edges not yet captured
k_rx_err_null_ptr	duration_us is NULL
k_rx_err_invalid_arg	irq_num not in [k_irq_min, k_irq_max]
k_rx_err_range_check_failed	Computed duration exceeds k_cmt2_wrap_us (invalid measurement)

Pre: rx_hcsr04_isr_start() called before trigger pulse

Pre: ISR handlers registered in ICU (INT_IRQ8-11)

Post: On k_rx_ok: *duration_us = end_us - start_us

Post: On k_rx_ok: complete flag cleared (ready for next measurement)

Note: Call repeatedly until k_rx_ok or caller's timeout expires

Note: Not ISR-safe; call from task context only] **See also:** rx_hcsr04_isr_start() Call before sending trigger pulse

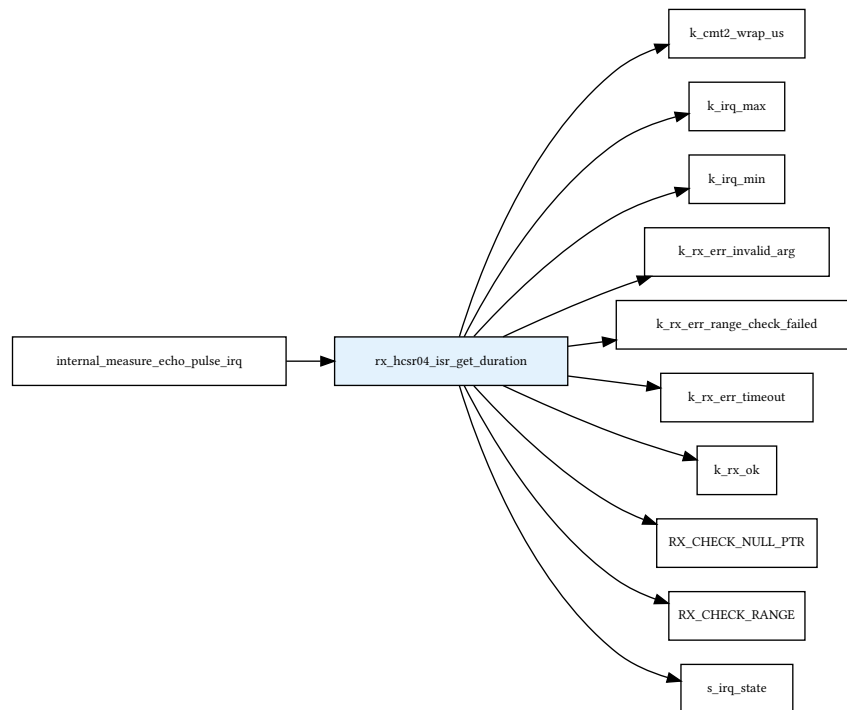


Figure 1007: Call graph for rx_hcsr04_isr_get_duration

_Defined in libs/rx_hcsr04/src/rx_hcsr04_isr.c:493_

Since Version 1.2.0 (Issue 296)

—

INT_IRQ8

```
void INT_IRQ8(void)
```

ISR for IRQ8 (P00 - Sonar 3 Back-Right).

Thin wrapper that forwards IRQ number constant k_irq_num_8 to internal_irq_handler(). The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ8 via rx_hcsr04_icu_configure() with k_hcsr04_irq_8

Pre: Interrupts globally enabled (PSW.I = 1)

Post: IRvector 72 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state0` if measurement active

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `internal_irq_handler()` Core ISR logic, `k_irq_num_8` IRQ number constant passed to the handler

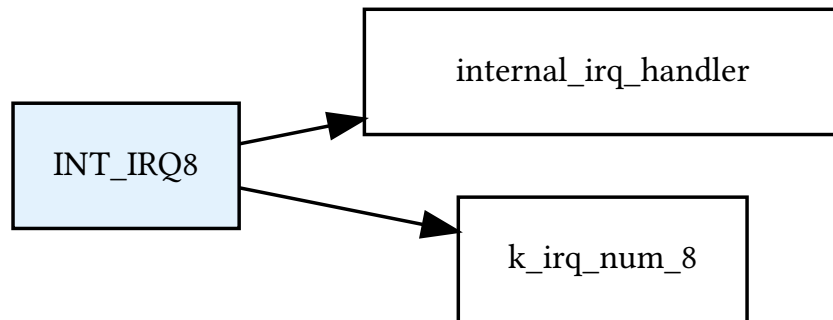


Figure 1008: Call graph for INT_IRQ8

_Defined in `libs/rx\hcsr04/src/rx\hcsr04_isr.c:588_`

Since Version 1.2.0 (Issue 296)

—

INT_IRQ9

```
void INT_IRQ9(void)
```

ISR for IRQ9 (P01 - Sonar 2 Back-Left).

Thin wrapper that forwards IRQ number constant `k_irq_num_9` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ9 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_9`

Pre: Interrupts globally enabled (PSW.I = 1)

Post: IRvector 73 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state1` if measurement active

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `internal_irq_handler()` Core ISR logic, `k_irq_num_9` IRQ number constant passed to the handler

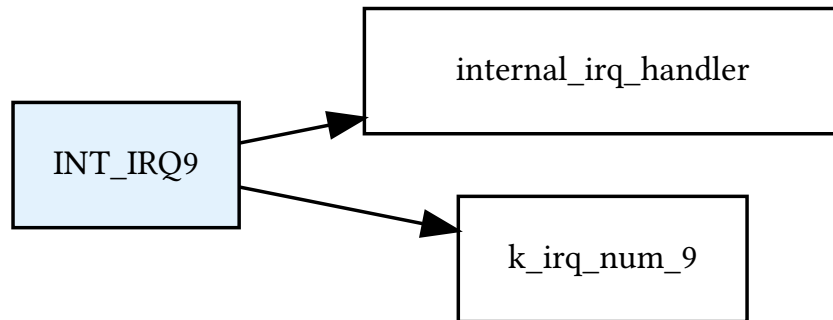


Figure 1009: Call graph for `INT_IRQ9`

_Defined in `libs/rx\hcsr04/src/rx\hcsr04_isr.c:617_`

Since Version 1.2.0 (Issue 296)

—

INT_IRQ10

```
void INT_IRQ10(void)
```

ISR for IRQ10 (P02 - Sonar 1 Front-Right).

Thin wrapper that forwards IRQ number constant `k_irq_num_10` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ10 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_10`

Pre: Interrupts globally enabled (PSW.I = 1)

Post: IRvector 74 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state2` if measurement active

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `internal_irq_handler()` Core ISR logic, `k_irq_num_10` IRQ number constant passed to the handler

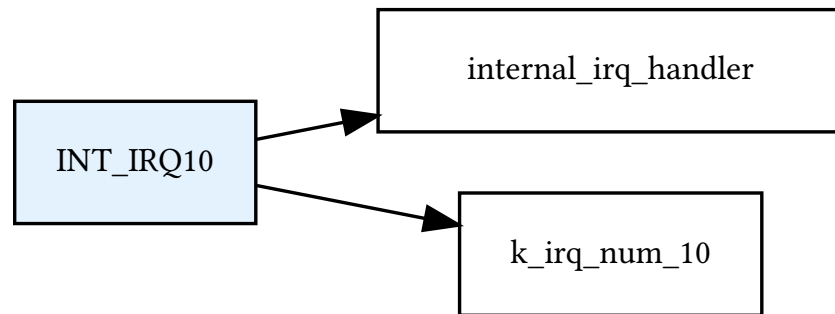


Figure 1010: Call graph for INT_IRQ10

_Defined in libs/rx\hcsr04/src/rx\hcsr04_isr.c:646_

Since Version 1.2.0 (Issue 296)

—

INT_IRQ11

```
void INT_IRQ11(void)
```

ISR for IRQ11 (P03 - Sonar 0 Front-Left).

Thin wrapper that forwards IRQ number constant `k_irq_num_11` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ11 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_11`

Pre: Interrupts globally enabled (`PSW.I = 1`)

Post: IRvector 75 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state3` if measurement active

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `internal_irq_handler()` Core ISR logic, `k_irq_num_11` IRQ number constant passed to the handler

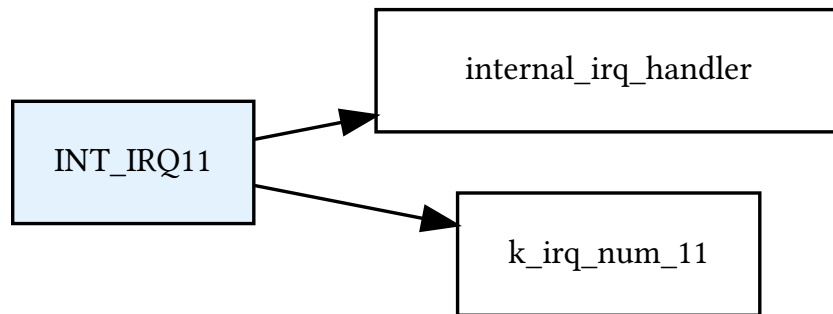


Figure 1011: Call graph for INT_IRQ11

_Defined in `libs/rx\hcsr04/src/rx\hcsr04_isr.c:675_`

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr.h

HC-SR04 ISR (Interrupt Service Routine) Handler Layer.

Provides ISR handlers and state management for HC-SR04 ultrasonic sensors operating in IRQ mode. Captures rising and falling edge timestamps with microsecond precision for accurate distance measurement.

Responsibilities:

- ISR functions for IRQ8-11 (INT_IRQ8 through INT_IRQ11, for 4 HC-SR04 sensors)
- Per-IRQ state management (start/end timestamps, completion flag)
- Edge type detection (rising vs falling)
- Timestamp capture via hardware abstraction layer

Operation:

1. Application calls `rx_hcsr04_isr_start()` before trigger pulse
2. ISR fires on rising edge -> captures start timestamp
3. ISR fires on falling edge -> captures end timestamp, sets complete flag
4. Application calls `rx_hcsr04_isr_get_duration()` to retrieve pulse width

STAR Team

2026-02-16

Copyright (c) 2026 STAR Project. MIT License.

1.2.0

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"rx_hcsr04.h"`

s_hcsr04_us_per_cm

`const float s_hcsr04_us_per_cm`

Microseconds per centimeter (roundtrip) for HC-SR04 distance conversion.

Note: Each translation unit that includes this header gets its own copy (static linkage)

Warning: Do not use for production distance calculations in rx_hcsr04.c; use the k_hcsr04_us_per_cm_roundtrip enum constant from rx_hcsr04.h instead

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_register

```
rx_err_t rx_hcsr04_isr_register(uint8_t irq_num, rx_hcsr04_sensor_index_t
sensor_index)
```

Register HC-SR04 sensor for IRQ echo measurement.

Maps an IRQ number to a sensor index for future use. This allows the ISR to identify which sensor triggered the interrupt.

Maps an IRQ number to a sensor index. Validates both IRQ number range and sensor_index range before storing the mapping. Used to identify which sensor triggered a given interrupt.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11 for P00-P03)
in	sensor_index	Sensor position index (rx_hcsr04_sensor_index_t) k_hcsr04_sensor_front_left (0): IRQ11 / P03 k_hcsr04_sensor_front_right (1): IRQ10 / P02 k_hcsr04_sensor_back_left (2): IRQ9 / P01 k_hcsr04_sensor_back_right (3): IRQ8 / P00
in	irq_num	IRQ number (8-11 for P00-P03)
in	sensor_index	Sensor array index (0-3 for 4 HC-SR04 sensors)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Registration successful
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_arg	sensor_index >= k_hcsr04_sensor_count (4)
k_rx_ok	Registration successful
k_rx_err_invalid_arg	irq_num not in [k_irq_min, k_irq_max]
k_rx_err_invalid_arg	sensor_index >= k_hcsr04_sensor_count

Pre: IRQ configured via rx_hcsr04_icu_configure()

Pre: sensor_index must be a valid sensor array index (< k_hcsr04_sensor_count)

Pre: IRQ configured via rx_hcsr04_icu_configure()

Pre: sensor_index must be a valid sensor array index

Post: s_sensor_mapirq_num = sensor_index

Post: ISR can identify sensor on interrupt

Post: s_sensor_mapirq_num updated with sensor_index

Post: ISR can identify sensor on next interrupt

Note: Call once during sensor initialization

Note: Not thread-safe; call during single-threaded initialization only

Note: Call once during sensor initialization

Note: Not thread-safe; call during initialization only] **Example:**

```
rx_hcsr04_isr_register((uint8_t)k_hcsr04_irq_11,k_hcsr04_sensor_front_left);  
rx_hcsr04_isr_register((uint8_t)k_hcsr04_irq_10,k_hcsr04_sensor_front_right);  
rx_hcsr04_isr_register((uint8_t)k_hcsr04_irq_9,k_hcsr04_sensor_back_left);  
rx_hcsr04_isr_register((uint8_t)k_hcsr04_irq_8,k_hcsr04_sensor_back_right);
```

See also: rx_hcsr04_isr_unregister() Cleanup counterpart for deinitialization,
rx_hcsr04_isr_start() Arm ISR before sending trigger pulse

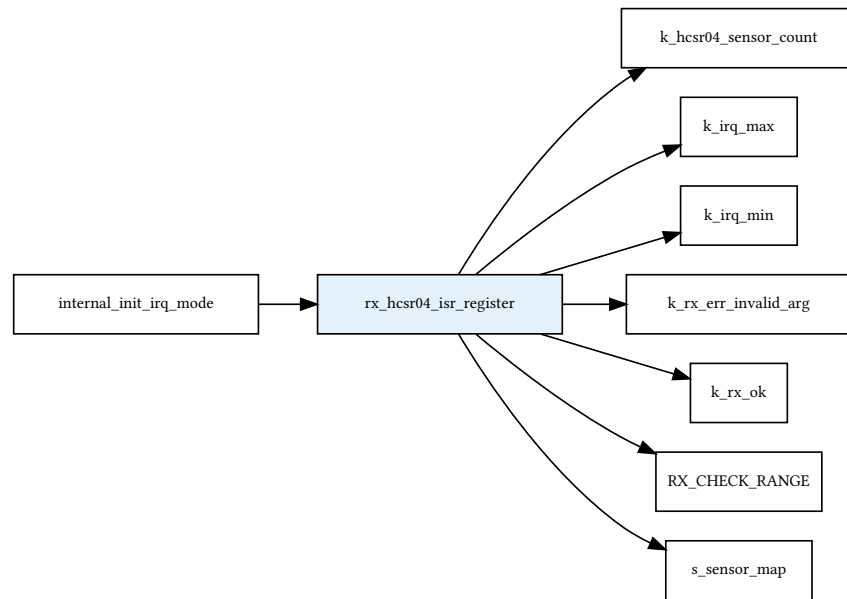


Figure 1012: Call graph for rx_hcsr04_isr_register

_Defined in libs/rx_hcsr04/src/rx_hcsr04_isr.h:220_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_unregister

```
rx_err_t rx_hcsr04_isr_unregister(uint8_t irq_num)
```

Unregister HC-SR04 sensor from IRQ echo measurement.

Clears the sensor mapping for the given IRQ number, restoring the slot to the k_sensor_unused sentinel. Call during sensor deinitialization after rx_hcsr04_icu_disable() to prevent stale ISR callbacks.

Clears the sensor mapping for the given IRQ number, restoring it to the k_sensor_unused sentinel. Call this during sensor deinitialization to prevent stale ISR callbacks after the sensor handle is invalidated.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11) to unregister
in	irq_num	IRQ number (8-11) to unregister

Returns: rx_err_t Error code

Value	Description
-------	-------------

k_rx_ok	Unregistration successful
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	IRQ slot was not registered (already unset)
k_rx_ok	Unregistration successful
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	IRQ slot was not registered (s_sensor_map entry is k_sensor_unused)

Pre: IRQ was previously registered via rx_hcsr04_isr_register()

Pre: No measurement currently active on this IRQ

Pre: IRQ was previously registered via rx_hcsr04_isr_register()

Pre: No measurement currently active on this IRQ

Post: s_sensor_mapirq_num reset to k_sensor_unused

Post: ISR will ignore subsequent interrupts on this IRQ

Post: s_sensor_mapirq_num = k_sensor_unused

Post: ISR will ignore subsequent interrupts on this IRQ

Note: Call during sensor deinitialization, after rx_hcsr04_icu_disable()

Note: Call during sensor deinitialization, after rx_hcsr04_icu_disable()

Note: Not thread-safe; call during single-threaded cleanup only] **Example: Sensor Deinitialization:** rx_err_t err=rx_hcsr04_icu_disable((uint8_t)k_hcsr04_irq_11);
err=rx_hcsr04_isr_unregister((uint8_t)k_hcsr04_irq_11); if(err!=k_rx_ok)
{ rx_log_warn("SONAR","Failed to unregister IRQ11"); }

See also: rx_hcsr04_isr_register() Register sensor mapping, rx_hcsr04_deinit() Calls this during IRQ-mode cleanup, rx_hcsr04_isr_register() Register sensor, rx_hcsr04_deinit() Calls this during cleanup in IRQ mode

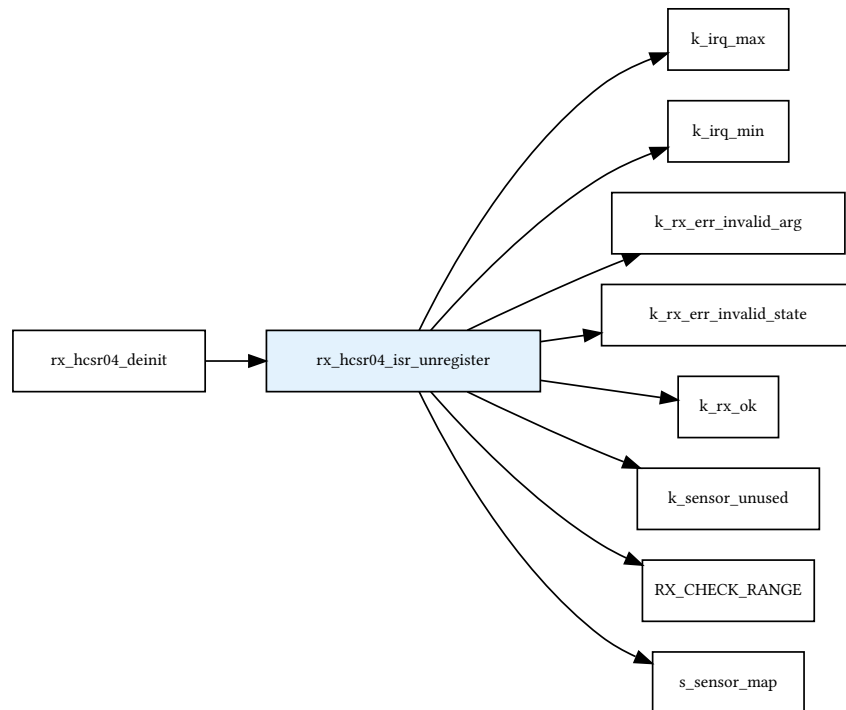


Figure 1013: Call graph for rx_hcsr04_isr_unregister

_Defined in libs/rx_hcsr04/src/rx_hcsr04_isr.h:262_

Since Version 1.2.0 (Issue 296)

—

rx_hcsr04_isr_get_duration

```
rx_err_t rx_hcsr04_isr_get_duration(uint8_t irq_num, uint32_t *duration_us)
```

Get echo pulse duration from ISR state.

Reads the captured timestamps and calculates pulse duration. Returns error if measurement is not complete yet.

Reads the captured timestamps and calculates echo pulse duration. Returns k_rx_err_timeout (not yet ready) if the ISR has not captured both edges. On success, clears the complete flag to prepare for the next measurement.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11)

out	duration_us	Pointer to store pulse duration (microseconds) Valid range: 150us - 8700us (2cm - 150cm in IRQ mode) CMT2 16-bit wrap handled automatically Must not be NULL
in	irq_num	IRQ number (8-11)
out	duration_us	Pointer to store pulse duration in microseconds (valid range: 150-8700 us for 2-150 cm in IRQ mode; CMT2 wrap handled automatically for durations < k_cmt2_wrap_us)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Duration available, written to *duration_us
k_rx_err_timeout	Measurement not complete yet
k_rx_err_null_ptr	duration_us pointer is NULL
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_range_check_failed	Computed duration exceeds CMT2 wrap period (invalid)
k_rx_ok	Duration available, *duration_us written, complete flag cleared
k_rx_err_timeout	Both edges not yet captured
k_rx_err_null_ptr	duration_us is NULL
k_rx_err_invalid_arg	irq_num not in [k_irq_min, k_irq_max]
k_rx_err_range_check_failed	Computed duration exceeds k_cmt2_wrap_us (invalid measurement)

Pre: rx_hcsr04_isr_start() called before trigger pulse

Pre: Both rising and falling edges captured by ISR

Pre: rx_hcsr04_isr_start() called before trigger pulse

Pre: ISR handlers registered in ICU (INT_IRQ8-11)

Post: On k_rx_ok: *duration_us contains echo pulse width in microseconds

Post: On k_rx_ok: complete flag cleared (ready for next measurement via start())

Post: On k_rx_ok: *duration_us = end_us - start_us

Post: On k_rx_ok: complete flag cleared (ready for next measurement)

Note: Does NOT block; returns immediately with k_rx_err_timeout if not ready

Note: Call repeatedly in a bounded loop (caller must enforce timeout)

Note: Call repeatedly until k_rx_ok or caller's timeout expires

Note: Not ISR-safe; call from task context only] **Example: Bounded Polling for Completion:**
 uint32_t start_time = get_time_us(); uint32_t duration_us; rx_err_t err = k_rx_err_timeout;

```
for(uint32_t i=0; i<k_hcsr04_echo_timeout_us; i++)
{ err=rx_hcsr04_isr_get_duration((uint8_t)k_hcsr04_irq_11,&duration_us); if(err==k_rx_ok){ break; }
if((get_time_us()-start_time)>k_hcsr04_echo_timeout_us){ break; } } if(err==k_rx_ok)
{ float distance_cm=(float)duration_us/s_hcsr04_us_per_cm; }
```

See also: rx_hcsr04_isr_start() Must be called before trigger pulse to arm ISR,
 rx_hcsr04_isr_start() Call before sending trigger pulse

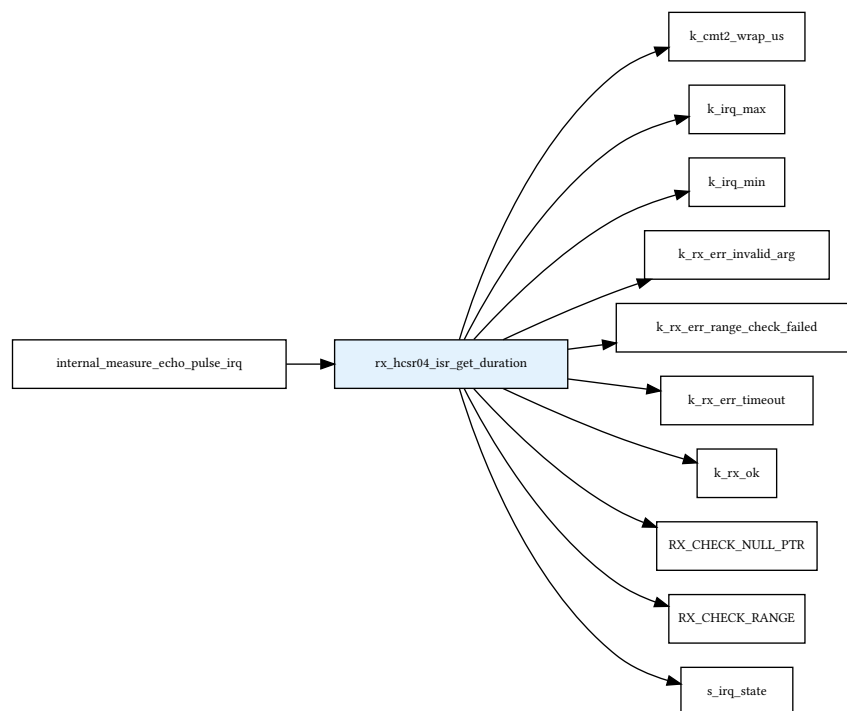


Figure 1014: Call graph for rx_hcsr04_isr_get_duration

_Defined in libs/rx\hcsr04/src/rx\hcsr04_isr.h:318_

Since Version 1.2.0 (Issue 296)

rx_hcsr04_isr_start

```
rx_err_t rx_hcsr04_isr_start(uint8_t irq_num)
```

Start new echo measurement (call before trigger pulse).

Prepares ISR state for a new measurement. Validates that the IRQ number is registered in the sensor map, then clears the completion flag and sets the active flag. Must be called before sending trigger pulse.

Start new echo measurement (call before trigger pulse).

Prepares ISR state for a new measurement. Validates that the IRQ is registered, then sets complete=false first (to prevent a stale complete flag being read), then sets active=true to allow ISR to capture edges. Order matters: complete must be cleared before active is set to prevent a race where ISR fires between the two writes.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (k_hcsr04_irq_8 through k_hcsr04_irq_11)
in	irq_num	IRQ number (8-11)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	ISR armed successfully
k_rx_err_invalid_arg	irq_num not in valid range
k_rx_err_invalid_state	irq_num not registered via rx_hcsr04_isr_register()
k_rx_ok	ISR armed successfully
k_rx_err_invalid_arg	irq_num not in valid range [k_irq_min, k_irq_max]
k_rx_err_invalid_state	irq_num slot not registered (s_sensor_map[irq_num] == k_sensor_unused)

Pre: ISR registered via rx_hcsr04_isr_register()

Pre: No measurement currently active

Pre: rx_hcsr04_isr_register() called for this IRQ number

Pre: No measurement currently active on this IRQ

Post: State.active = true (on k_rx_ok)

Post: State.complete = false (on k_rx_ok)

Post: Ready to capture rising edge (on k_rx_ok)

Post: s_irq_stateidx.complete = false (on k_rx_ok)

Post: s_irq_stateidx.active = true (on k_rx_ok)

Post: ISR is armed to capture rising edge (on k_rx_ok)

Note: Always call before sending trigger pulse

Note: Do not call while previous measurement active

Note: Always call BEFORE sending trigger pulse (ordering is critical)

Note: complete cleared BEFORE active set (avoids race condition)] **Example: Typical Measurement Sequence:** rx_err_t err=rx_hcsr04_isr_start((uint8_t)k_hcsr04_irq_11); if(err!=k_rx_ok){ return err; }

```
gpio_set_high(trigger_pin); delay_us(k_hcsr04_trigger_pulse_us); gpio_set_low(trigger_pin);
uint32_t duration_us; err=k_rx_err_timeout; for(uint32_t i=0; i<k_hcsr04_echo_timeout_us; i++)
{ err=rx_hcsr04_isr_get_duration((uint8_t)k_hcsr04_irq_11, &duration_us); if(err==k_rx_ok)
{ break; } } if(err!=k_rx_ok){ }
```

See also: rx_hcsr04_isr_disarm() Disarm ISR on trigger failure after successful start,
rx_hcsr04_isr_get_duration() Poll for completion after this call

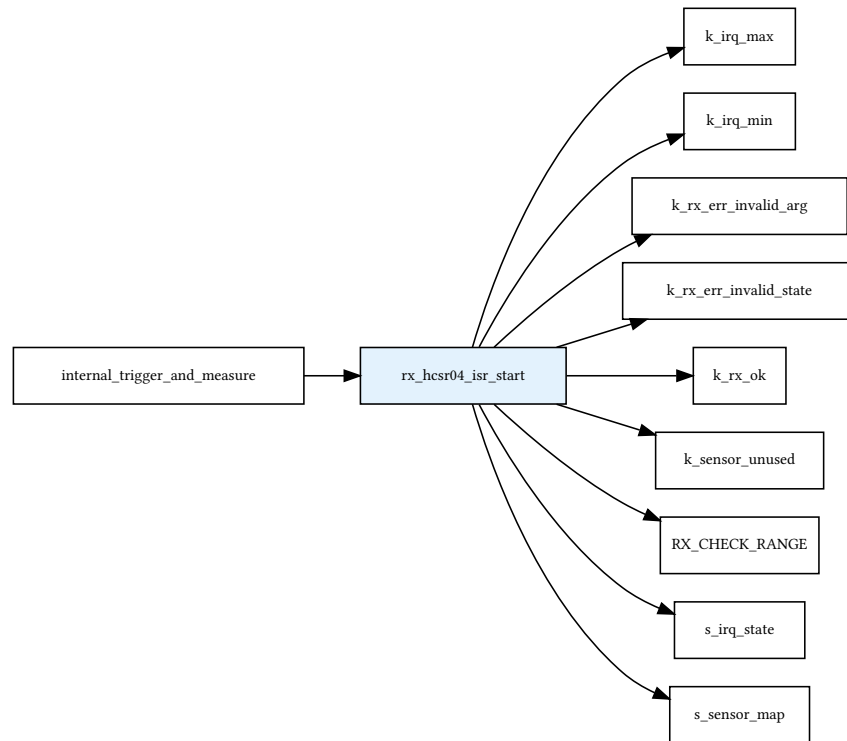


Figure 1015: Call graph for rx_hcsr04_isr_start

Defined in `libs/rx\_hcsr04/src/rx\_hcsr04\_isr.h:376`

Since Version 1.2.0 (Issue 296)

rx_hcsr04_isr_disarm

```
rx_err_t rx_hcsr04_isr_disarm(uint8_t irq_num)
```

Disarm ISR state machine (call on trigger failure after arm).

Clears the active flag in the per-IRQ state structure to cancel a previously armed measurement. Must be called when `rx_hcsr04_isr_start()` has succeeded but the subsequent trigger pulse fails, to prevent the ISR state machine from remaining armed indefinitely.

Use Case:

Disarm ISR state machine (call on trigger failure after arm).

Clears the active flag so the ISR will ignore subsequent edge interrupts. Call this in the error path when `rx_hcsr04_isr_start()` succeeded but the trigger pulse failed, to prevent the ISR from remaining armed indefinitely.

Parameters:

Direction	Name	Description
in	irq_num	IRQ number (8-11) to disarm
in	irq_num	IRQ number (8-11) to disarm

Returns: rx_err_t Error code

Value	Description
k_rx_ok	ISR disarmed successfully
k_rx_err_invalid_arg	irq_num not in range [8, 11]
k_rx_err_invalid_state	irq_num not registered via rx_hcsr04_isr_register()
k_rx_ok	ISR disarmed successfully
k_rx_err_invalid_arg	irq_num not in range [k_irq_min, k_irq_max]
k_rx_err_invalid_state	irq_num not registered via rx_hcsr04_isr_register()

Pre: rx_hcsr04_isr_start() called successfully for this IRQ

Pre: irq_num registered via rx_hcsr04_isr_register() (sensor map entry != k_sensor_unused)

Pre: rx_hcsr04_isr_start() called successfully for this IRQ

Pre: irq_num registered via rx_hcsr04_isr_register() (s_sensor_mapirq_num != k_sensor_unused)

Post: s_irq_stateidx.active = false

Post: s_irq_stateidx.complete = false

Post: ISR will ignore subsequent interrupts on this IRQ

Post: s_irq_stateidx.active = false

Post: s_irq_stateidx.complete = false

Post: ISR will ignore subsequent interrupts on this IRQ

Note: Call only in the error path, after a successful rx_hcsr04_isr_start()

Note: Ignores return value of disarm in error path (best-effort cleanup)

Note: Best-effort cleanup; always call in error path after successful isr_start] **Example:**

```
err=rx_hcsr04_isr_start((uint8_t)handle->echo_irq);//ArmISR
if(err!=k_rx_ok){returnerr;}

err=internal_send_trigger_pulse(handle);//Sendtrigger
if(err!=k_rx_ok){
//Triggerfailed--disarmISRtorestoreconsistentstate
(void)rx_hcsr04_isr_disarm((uint8_t)handle->echo_irq);
returnerr;
}
```

See also: rx_hcsr04_isr_start() Arm ISR state before trigger pulse,
rx_hcsr04_isr_start() Arm ISR state before trigger pulse

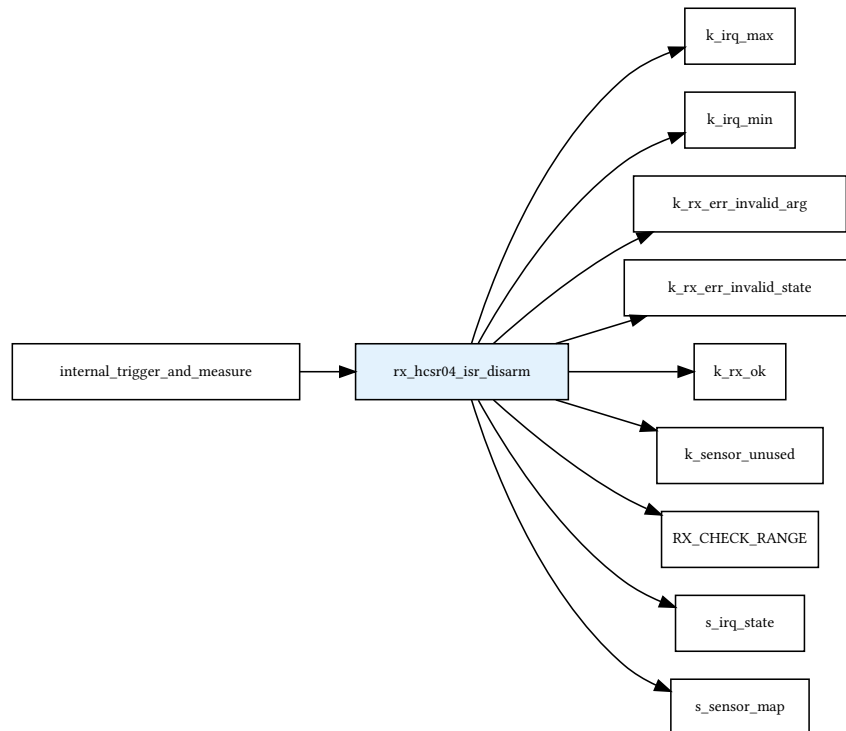


Figure 1016: Call graph for rx_hcsr04_isr_disarm

_Defined in libs/rx\hcsr04/src/rx\hcsr04_isr.h:421_

Since Version 1.2.0 (Issue 296)

—

INT_IRQ8

```
void INT_IRQ8(void)
```

ISR for IRQ8 (P00 - Sonar 3 Back-Right).

Hardware-triggered ISR on both edges of the HC-SR04 echo pulse from P00. Delegates to `internal_irq_handler(8)`. Clears `IR[72]`, detects rising/falling edge from `PORT0 PIDR` bit 0, and captures `hcsr04_hal_get_time_us_isr()` timestamp.

Thin wrapper that forwards IRQ number constant `k_irq_num_8` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ8 via `rx_hcsr04_icu_configure()`

Pre: PIN P00 configured for IRQ function via `rx_mpc_set_irq()`

Pre: ICU configured for IRQ8 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_8`

Pre: Interrupts globally enabled (`PSW.I = 1`)

Post: IR72 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state0` if measurement active

Post: IRvector 72 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state0` if measurement active

Note: Called by hardware on both rising and falling edges; execution time < 5us

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `rx_hcsr04_isr_start()` Arms ISR state before trigger pulse, `rx_hcsr04_isr_get_duration()` Reads captured timestamps, `internal_irq_handler()` Core ISR logic, `k_irq_num_8` IRQ number constant passed to the handler

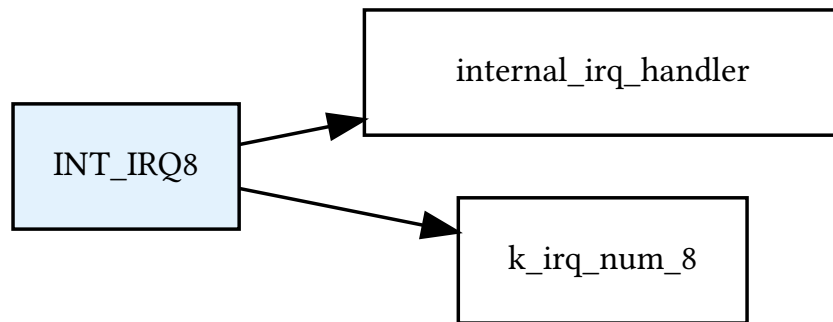


Figure 1017: Call graph for INT_IRQ8

_Defined in `libs/rx\hcsr04/src/rx\hcsr04_isr.h:451_`

Since Version 1.2.0 (Issue 296)

—

INT_IRQ9

```
void INT_IRQ9(void)
```

ISR for IRQ9 (P01 - Sonar 2 Back-Left).

Hardware-triggered ISR on both edges of the HC-SR04 echo pulse from P01. Delegates to `internal_irq_handler(9)`. Clears `IR[73]`, detects rising/falling edge from `PORT0 PIDR bit 1`, and captures `hcsr04_hal_get_time_us_isr()` timestamp.

Thin wrapper that forwards IRQ number constant `k_irq_num_9` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ9 via `rx_hcsr04_icu_configure()`

Pre: PIN P01 configured for IRQ function via `rx_mpc_set_irq()`

Pre: ICU configured for IRQ9 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_9`

Pre: Interrupts globally enabled (`PSW.I = 1`)

Post: IR73 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state1` if measurement active

Post: IRvector 73 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state1` if measurement active

Note: Called by hardware on both rising and falling edges; execution time < 5us

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `rx_hcsr04_isr_start()` Arms ISR state before trigger pulse, `rx_hcsr04_isr_get_duration()` Reads captured timestamps, `internal_irq_handler()` Core ISR logic, `k_irq_num_9` IRQ number constant passed to the handler

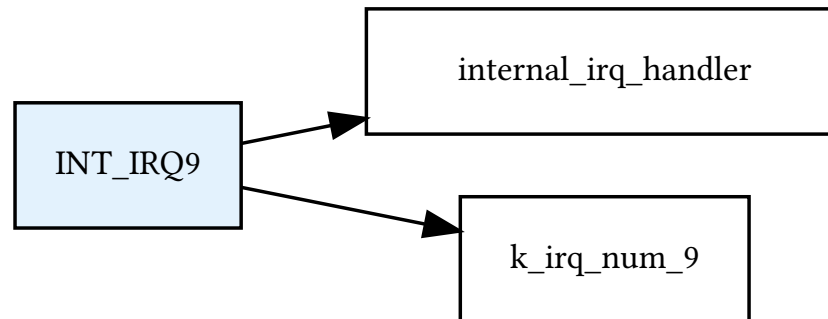


Figure 1018: Call graph for INT_IRQ9

_Defined in `libs/rx_hcsr04/src/rx_hcsr04_isr.h:476_`

Since Version 1.2.0 (Issue 296)

—

INT_IRQ10

```
void INT_IRQ10(void)
```

ISR for IRQ10 (P02 - Sonar 1 Front-Right).

Hardware-triggered ISR on both edges of the HC-SR04 echo pulse from P02. Delegates to `internal_irq_handler(10)`. Clears `IR[74]`, detects rising/falling edge from `PORT0 PIDR` bit 2, and captures `hcsr04_hal_get_time_us_isr()` timestamp.

Thin wrapper that forwards IRQ number constant `k_irq_num_10` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ10 via `rx_hcsr04_icu_configure()`

Pre: PIN P02 configured for IRQ function via `rx_mpc_set_irq()`

Pre: ICU configured for IRQ10 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_10`

Pre: Interrupts globally enabled (`PSW.I = 1`)

Post: IR74 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state2` if measurement active

Post: IRvector 74 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state2` if measurement active

Note: Called by hardware on both rising and falling edges; execution time < 5us

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `rx_hcsr04_isr_start()` Arms ISR state before trigger pulse, `rx_hcsr04_isr_get_duration()` Reads captured timestamps, `internal_irq_handler()` Core ISR logic, `k_irq_num_10` IRQ number constant passed to the handler

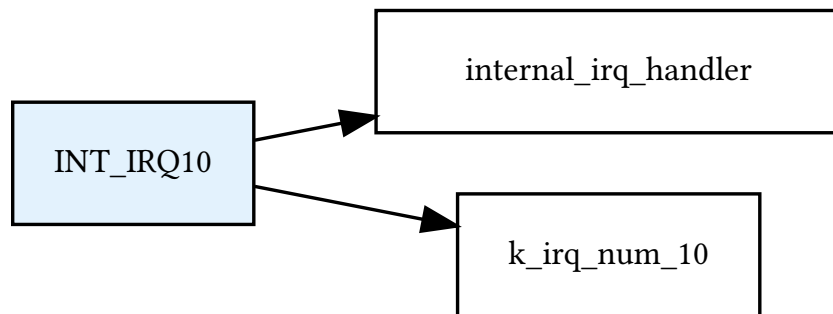


Figure 1019: Call graph for INT_IRQ10

_Defined in `libs/rx_hcsr04/src/rx_hcsr04_isr.h:501_`

Since Version 1.2.0 (Issue 296)

—

INT_IRQ11

```
void INT_IRQ11(void)
```

ISR for IRQ11 (P03 - Sonar 0 Front-Left).

Hardware-triggered ISR on both edges of the HC-SR04 echo pulse from P03. Delegates to `internal_irq_handler(11)`. Clears `IR[75]`, detects rising/falling edge from `PORT0 PIDR bit 3`, and captures `hcsr04_hal_get_time_us_isr()` timestamp.

Thin wrapper that forwards IRQ number constant `k_irq_num_11` to `internal_irq_handler()`. The handler clears the IR flag, reads the GPIO pin state, and captures the rising or falling edge timestamp.

Returns: void

Pre: ICU configured for IRQ11 via `rx_hcsr04_icu_configure()`

Pre: PIN P03 configured for IRQ function via `rx_mpc_set_irq()`

Pre: ICU configured for IRQ11 via `rx_hcsr04_icu_configure()` with `k_hcsr04_irq_11`

Pre: Interrupts globally enabled (`PSW.I = 1`)

Post: IR75 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state3` if measurement active

Post: IRvector 75 flag cleared (interrupt acknowledged)

Post: Echo edge timestamp captured in `s_irq_state3` if measurement active

Note: Called by hardware on both rising and falling edges; execution time < 5us

Note: Thin wrapper; all logic is in `internal_irq_handler()`

Note: Keep ISR execution time minimal (< 5 us)] **See also:** `rx_hcsr04_isr_start()` Arms ISR state before trigger pulse, `rx_hcsr04_isr_get_duration()` Reads captured timestamps, `internal_irq_handler()` Core ISR logic, `k_irq_num_11` IRQ number constant passed to the handler

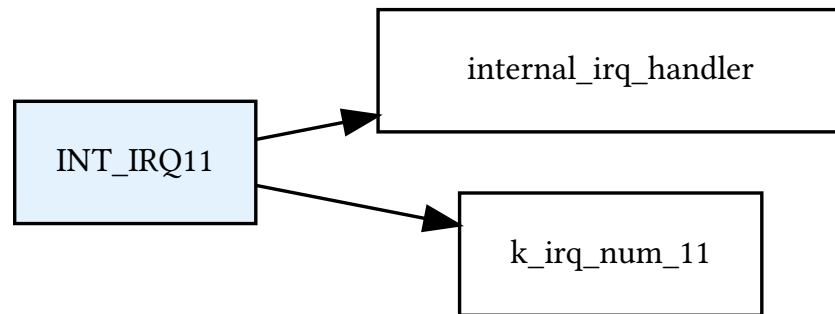


Figure 1020: Call graph for INT_IRQ11

_Defined in libs/rx\hcsr04/src/rx\hcsr04_isr.h:526_

Since Version 1.2.0 (Issue 296)

—

rx_motor.h

Brushed DC Motor Control API using RX72N GPTW PWM with H-Bridge Driver Support.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"
- "rx_gptw.h"

rx_motor_init

```
rx_err_t rx_motor_init(rx_motor_handle_t *handle, const rx_motor_config_t
*config)
```

Initialize motor controller with GPTW PWM.

Configures the RX72N GPTW peripheral for H-bridge motor control. Initializes PWM outputs with specified frequency and dead-time.

Initialize motor controller with GPTW PWM.

Performs complete initialization of brushed DC motor control system including GPTW PWM peripheral configuration, output pin setup, and handle state initialization. This function must be called before any other motor control operations.

Initialization Sequence:

1. Input validation - Verify handle and config pointers, check initialization state
2. Parameter validation - Validate PWM frequency and dead-time against hardware limits
3. GPTW configuration - Prepare GPTW config struct with frequency, dead-time, polarity
4. GPTW initialization - Initialize PWM peripheral and set outputs to 0% (safe state)
5. Handle setup - Save configuration parameters to handle structure
6. Post-condition check - Verify handle was properly initialized (NASA Rule 5)

Safety Features:

- All outputs initialized to 0% duty (motor stopped/coast) before function returns

- Input parameter validation prevents invalid configurations from reaching hardware
- Handle marked initialized only after successful setup (prevents use before init)
- Post-condition verification catches initialization errors

Configuration Parameters:

- `pwm_freq_hz`: PWM frequency [1 kHz, 50 kHz], typical 20 kHz for inaudible operation
- `dead_time_ns`: Dead-time insertion [100 ns, 10 us], typical 1 us for shoot-through protection
- `channel`: GPTW channel to use (`k_gptw_channel_0` through `k_gptw_channel_7`)
- `output_a`: IN2 (direction) signal pin for direction control
- `output_b`: IN1 (PWM/enable) signal pin for speed PWM
- `invert_pwm`: Polarity inversion (true = inverted, false = normal)

If function returns `k_rx_ok`, `handle->initialized == true`

If function returns error, `handle->initialized == false`

PWM frequency and dead-time cannot be changed without deinitialization

Multiple motors can share same GPTW channel if outputs are different

1.0.0 Initial implementation with IN2/IN1 mode support

Parameters:

Direction	Name	Description
out	<code>handle</code>	Pointer to motor handle structure. Must not be nullptr.
in	<code>config</code>	Pointer to motor configuration. Must not be nullptr.
out	<code>handle</code>	Pointer to motor handle structure to initialize Must not be nullptr Must not already be initialized (checked via <code>handle->initialized</code> flag) Will be populated with configuration on success Structure should be allocated by caller (typically stack allocation) Contents undefined on error
in	<code>config</code>	Pointer to motor configuration parameters Must not be nullptr <code>pwm_freq_hz</code> : [1000, 50000] Hz (validated) <code>dead_time_ns</code> : [100, 10000] ns (validated) <code>channel</code> : Valid GPTW channel with available outputs <code>output_a</code> , <code>output_b</code> : Valid GPTW outputs, must be different <code>invert_pwm</code> : true (inverted) or false (normal) Configuration is copied to handle (config can be stack-allocated)

Returns: `rx_err_t` Error code indicating success or failure reason

Value	Description
<code>k_rx_ok</code>	Success, motor initialized and ready for use
<code>k_rx_err_null_ptr</code>	<code>handle</code> or <code>config</code> pointer is nullptr
<code>k_rx_err_invalid_state</code>	handle already initialized (must call <code>rx_motor_deinit</code> first)
<code>k_rx_err_invalid_arg</code>	<code>pwm_freq_hz</code> out of range [1 kHz, 50 kHz]
<code>k_rx_err_invalid_arg</code>	<code>dead_time_ns</code> out of range [100 ns, 10 us]
<code>k_rx_err_invalid_arg</code>	<code>output_a</code> or <code>output_b</code> invalid GPTW output
<code>k_rx_err_invalid_arg</code>	<code>output_a == output_b</code> (same pin used twice)
<code>k_rx_err_invalid_state</code>	Post-condition check failed (initialization verification)

k_rx_err_*	Propagated from rx_gptw_init_pwm() or rx_gptw_set_duty()
------------	--

Pre: handle must point to valid, uninitialized rx_motor_handle_t structure

Pre: config must point to valid rx_motor_config_t with parameters in valid ranges

Pre: GPTW peripheral must be available (not used by other motor instance)

Pre: handle->initialized must be false (or uninitialized memory)

Post: On success: handle->initialized = true, motor ready for commands

Post: On success: Motor in coast state (both outputs 0%, high impedance)

Post: On success: handle contains copy of all config parameters

Post: On failure: handle->initialized remains false, GPTW not initialized

Note: Not thread-safe, do not call simultaneously on same handle

Note: Configuration is copied to handle, original config can be freed/reused

Note: Motor remains stopped after initialization (call rx_motor_set_duty to move)

Note: Can only initialize once per handle (call rx_motor_deinit to reinitialize)

Warning: Must be called before any other motor control functions

Warning: Do not modify handle contents directly after initialization

Warning: Ensure handle remains in scope for entire motor lifetime

Thread Safety:: Not thread-safe. If multiple tasks need motor access, use ThreadX mutex around all motor operations or allocate one motor handle per task.

Performance:: Execution time: 18 us @ 240 MHz (includes GPTW initialization) One-time overhead, called once per motor at system startup Not performance-critical (initialization phase)

Example - Single Motor::

```
rx_motor_config_t motor0_config = { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b }
```

```
rx_motor_handle_t motor0; rx_err_t err = rx_motor_init(&motor0, &motor0_config); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to initialize motor 0: %d", err); return err; }
```

Example - Four Motors (STAR Platform):: rx_motor_handle_t motors[4];

```
const rx_motor_config_t motor_configs[4] = { { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 100, .pwm_duty = 50 },
{ .channel = k_gptw_channel_1, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 100, .pwm_duty = 50 },
{ .channel = k_gptw_channel_2, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 100, .pwm_duty = 50 },
{ .channel = k_gptw_channel_3, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 100, .pwm_duty = 50 } };

for (uint8_t i = 0; i < 4; i++) { rx_err_t err = rx_motor_init(&motors[i], &motor_configs[i]); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to initialize motor %d", i); for (uint8_t j = 0; j < i; j++)
{ (void) rx_motor_deinit(&motors[j]); } return err; } }

rx_log_info("APP", "All 4 motors initialized successfully");
```

Example - Error Handling::

```
rx_motor_config_t config = { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 100, .pwm_duty = 50 };
rx_motor_handle_t motor; rx_err_t err = rx_motor_init(&motor, &config);

switch (err) { case k_rx_ok: break; case k_rx_err_invalid_arg:
rx_log_error("APP", "Invalid config (check freq, dead-time, outputs)"); break; case k_rx_err_invalid_state:
rx_log_error("APP", "Motor already initialized or init verification failed"); break; default:
rx_log_error("APP", "Unexpected error: %d", err); break; }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential error checking
Rule 3: [OK] No dynamic memory allocation (handle passed by caller) Rule 4: [OK] Function length: 61 lines (within 60-line guideline) Rule 5: [OK] 4 preconditions (NULL checks, state check, range validation) Rule 5: [OK] 2 postconditions (initialized flag set, config copied) Rule 7: [OK] All return values checked (internal_init_gptw_outputs) Rule 8: [OK] C23 typed enums for validation constants

See also: rx_motor_deinit() Cleanup function, must call to reinitialize,
rx_motor_set_duty() Set motor speed/direction after initialization, rx_motor_config_t Configuration structure definition, rx_motor_handle_t Handle structure definition,
rx_gptw_init_pwm() Underlying GPTW initialization



Figure 1021: Call graph for rx_motor_init

Defined in `libs/rx_motor/inc/rx_motor.h:519`

Since Version 1.0.0

—

rx_motor_deinit

```
rx_err_t rx_motor_deinit(rx_motor_handle_t *handle)
```

Deinitialize motor controller and release resources.

Stops motor, disables PWM outputs, and releases GPTW peripheral.

Deinitialize motor controller and release resources.

Performs orderly shutdown of motor control system by stopping motor outputs and releasing GPTW peripheral resources. After deinitialization, handle can be reinitialized via `rx_motor_init()` with different configuration parameters.

Shutdown Sequence:

1. Validate handle pointer (NULL check)
2. Verify handle is initialized (prevent double-deinit)
3. Stop motor outputs (coast mode, 0% duty on both pins)
4. Deinitialize GPTW peripheral (release hardware resources)
5. Clear initialized flag (mark handle as deinitialized)

Use Cases:

- System shutdown: Release hardware before power-down
- Reconfiguration: Change PWM frequency or dead-time (requires deinit -> init)
- Error recovery: Clean up after hardware fault
- Resource sharing: Free GPTW channel for other use

Failure to deinit before program exit may leave GPTW in active state

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized motor handle. Must not be nullptr.
inout	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) Will be marked uninitialized (initialized = false) on success Contents remain valid but motor operations will fail until reinitialized

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, motor deinitialized and GPTW released
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	Motor not initialized (already deinitialized)
<code>k_rx_err_*</code>	Propagated from <code>rx_gptw_deinit()</code> (hardware cleanup failed)

Pre: handle must be initialized via rx_motor_init()

Pre: No other tasks are currently accessing this motor handle

Post: handle->initialized = false (marked as deinitialized)

Post: Motor outputs disabled (high impedance, 0% duty)

Post: GPTW peripheral deinitialized (channel available for reuse)

Post: Handle can be reinitialized with rx_motor_init()

Note: If rx_motor_stop() fails, warning logged but deinit continues

Note: GPTW deinit affects entire channel (may impact other outputs on same channel)

Note: Thread-safe only with external synchronization

Warning: Ensure motor is not needed before calling (deactivates all control)

Thread Safety:: Not thread-safe. Ensure no other tasks are using motor handle during deinit.

Performance:: Execution time: 8 us @ 240 MHz (includes GPTW peripheral cleanup) Called once per motor during shutdown (not performance-critical)

Example - Clean Shutdown:: rx_motor_handle_tmotor;

```
rx_err_t err=rx_motor_deinit(&motor); if(err!=k_rx_ok){ rx_log_error("APP","Failed to deinit motor: %d",err); }
```

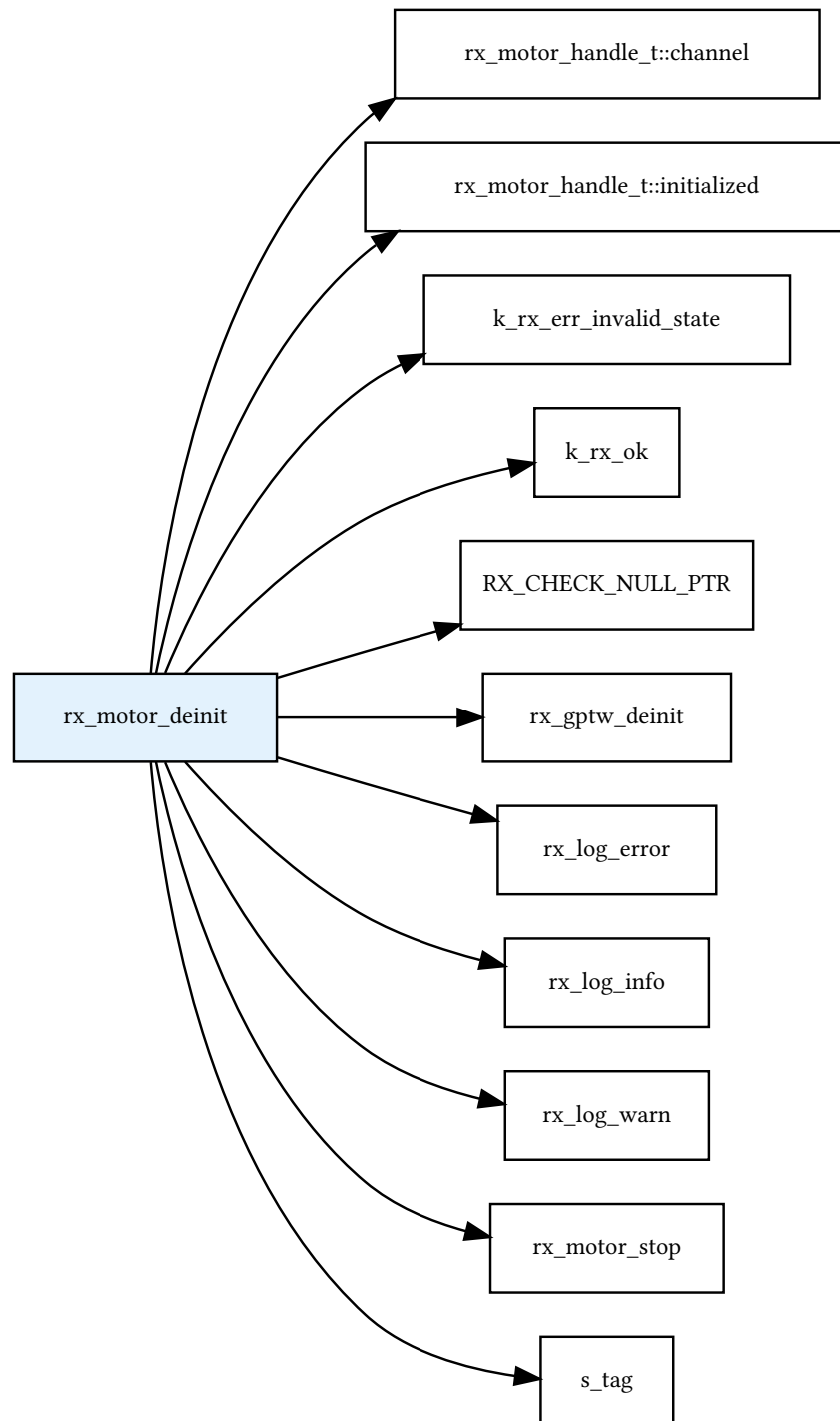
Example - Reconfiguration:: rx_motor_handle_tmotor;

```
rx_err_t err=rx_motor_deinit(&motor); if(err!=k_rx_ok){ return err; }
```

```
rx_motor_config_t new_config={ .channel=k_gptw_channel_0, .output_a=k_gptw_output_a, .output_b=k_gptw_output_b };
err=rx_motor_init(&motor,&new_config);
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (NULL check, initialized check) Rule 5: [OK] 1 postcondition (initialized flag cleared) Rule 7: [OK] Return values checked (rx_motor_stop logged, rx_gptw_deinit propagated)

See also: rx_motor_init() Initialize motor (required before reuse after deinit), rx_motor_stop() Stop motor without deinitializing, rx_motor_emergency_stop() Emergency shutdown (also deinitializes), rx_gptw_deinit() Underlying GPTW peripheral cleanup

Figure 1022: Call graph for `rx_motor_deinit`

Defined in `libs/rx_motor/inc/rx_motor.h:532`

Since Version 1.0.0

—

rx_motor_set_duty

```
rx_err_t rx_motor_set_duty(rx_motor_handle_t *handle, float duty)
```

Set motor duty cycle.

Sets the motor PWM duty cycle. Positive values drive forward, negative values drive reverse. The duty cycle is clamped to $[-100.0, +100.0]$.

Set motor duty cycle.

Primary motor control function for setting speed and direction with a single signed duty cycle value. Positive values drive motor forward, negative values drive motor in reverse, and zero stops the motor (coast mode). Internally converts signed duty cycle into IN2/IN1 signal pair for H-bridge control.

Control Mapping (IN2/IN1 Mode):

- duty > 0 (Forward):

IN2 (output_a) = HIGH (100% duty) -> Direction = forward IN1 (output_b) = |duty| PWM -> Speed = magnitude Example: duty = +75% -> IN2 = HIGH, IN1 = 75% PWM

- duty < 0 (Reverse):

IN2 (output_a) = LOW (0% duty) -> Direction = reverse IN1 (output_b) = |duty| PWM -> Speed = magnitude Example: duty = -50% -> IN2 = LOW, IN1 = 50% PWM

- duty = 0 (Coast):

IN2 (output_a) = LOW (0% duty) IN1 (output_b) = LOW (0% duty) Motor free-wheels (high impedance)

Algorithm:

Safety Features:

- Duty cycle clamping prevents hardware over-drive ($|duty| \leq 100\%$)
- NaN/Inf detection prevents undefined hardware register values
- Initialization check prevents operation on uninitialized peripheral
- Post-condition verification catches update failures (NASA Rule 5)

Performance:

- Typical case: 3 us @ 240 MHz (2 GPTW register writes)
- Suitable for 100 Hz - 10 kHz control loops
- STAR platform: Called at 100 Hz from PID velocity controller

If function returns `k_rx_ok`, `handle->current_duty == commanded_duty`

Motor duty always in range $[-100.0, +100.0]$ after function call

If `invert_pwm` was configured, duty is inverted internally

Post-condition failure indicates serious hardware fault (investigate immediately)

1.0.0 Initial implementation with IN2/IN1 mode

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized motor handle. Must not be nullptr.
in	duty	Duty cycle percentage (-100.0 to +100.0). Positive: Forward rotation (output_a active) Negative: Reverse rotation (output_b active) Zero: Coast (both outputs low)
in	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized via rx_motor_init() (validated) Handle state will be updated with new duty value current_duty field reflects actual commanded duty after return
in	duty	Signed duty cycle percentage (speed and direction) Valid range: [-100.0, +100.0] Values outside range are clamped (no error returned) Units: Percent (%) Positive: Forward rotation, negative: Reverse rotation Zero: Motor coast (high impedance) NaN/Inf: Rejected with k_rx_err_invalid_arg Resolution: Float precision (7 decimal digits)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, motor duty updated, outputs configured
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	Motor not initialized (call rx_motor_init first)
k_rx_err_invalid_arg	duty is NaN or Inf (invalid float value)
k_rx_err_invalid_state	Post-condition check failed (duty not updated)
k_rx_err_*	Propagated from rx_gptw_set_duty() (hardware write failed)

Pre: handle must be initialized via rx_motor_init()

Pre: duty must be finite (not NaN or Inf)

Pre: GPTW peripheral must be operational (not disabled or in error state)

Post: On success: handle->current_duty = duty (after clamping and inversion)

Post: On success: Motor outputs configured for requested speed/direction

Post: On success: IN2 and IN1 signals match duty sign and magnitude

Post: On failure: Motor outputs unchanged from previous state

Note: Thread-safe only if called from single task or with external mutex

Note: Duty cycle clamping is silent (no warning logged for performance)

Note: Function can be called at high frequency (tested up to 10 kHz)

Note: Motor response time depends on mechanical inertia (100-500 ms for STAR motors)

Warning: Do not call from ISR (uses logging, may block)

Warning: Rapid direction changes (sign flip) can cause mechanical stress

Warning: High-frequency calls (> 10 kHz) may saturate GPTW peripheral bus

Thread Safety:: Not thread-safe. Use ThreadX mutex if multiple tasks control same motor.

Re-entrancy:: Not re-entrant on same handle. Safe to call on different handles concurrently.

Performance:: Execution time: 3 us @ 240 MHz with -O2 optimization Control loop frequency:
Tested up to 10 kHz (100 us period) Typical usage: 100 Hz (10 ms period) for velocity control
Memory: 16 bytes stack (float variables, error codes)

Example - Basic Usage:: rx_motor_handle_tmotor;

```
rx_err_t err = rx_motor_set_duty(&motor, 50.0F); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to set motor duty"); }

err = rx_motor_set_duty(&motor, -75.0F);

err = rx_motor_set_duty(&motor, 0.0F);
```

Example - Velocity Control Loop (100 Hz):: void motor_control_task(void* arg)

```
{ rx_motor_handle_t* motor = (rx_motor_handle_t*)arg; rx_pid_handle_t pid;

while(1) { float measured_velocity = rx_encoder_get_velocity_mps(&encoder);

float pid_output; rx_err_t err = rx_pid_compute(&pid, target_velocity,
measured_velocity, 0.01F, &pid_output);

err = rx_motor_set_duty(motor, pid_output); if (err != k_rx_ok)
{ (void)rx_motor_emergency_stop(motor); break; }

tx_thread_sleep(10); }
```

Example - Gradual Acceleration:: rx_motor_handle_tmotor;

```
const float ramp_time_s = 2.0F; const float dt_s = 0.01F; const float max_duty = 100.0F;
const float duty_increment = max_duty / (ramp_time_s / dt_s);

float current_duty = 0.0F; while (current_duty < max_duty)
{ rx_err_t err = rx_motor_set_duty(&motor, current_duty); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed during ramp"); break; } current_duty += duty_increment;
tx_thread_sleep(10); }

tx_thread_sleep(5000);
```

```
while(current_duty>0.0F){ rx_motor_set_duty(&motor,current_duty); current_duty-  
=duty_increment; tx_thread_sleep(10); }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential if-else logic Rule 3: [OK] No dynamic memory allocation Rule 4: [WARN] Function length: 82 lines (exceeds 60-line guideline, justified below) Rule 5: [OK] 3 preconditions (NULL check, init check, finite value check) Rule 5: [OK] 2 postconditions (duty updated, outputs configured) Rule 7: [OK] All return values checked (rx_gptw_set_duty)

Rule 4 Justification:: Function exceeds 60-line guideline (82 total) due to comprehensive error checking and logging required for NASA Rule 5 and Rule 7 compliance. Function represents a single logical operation (set motor duty) and cannot be meaningfully decomposed without harming code clarity and maintainability. Splitting would require passing internal state between helper functions, increasing coupling and reducing readability.

See also: rx_motor_init() Must call before using this function, rx_motor_stop() Alternative for stopping motor (supports brake mode), rx_motor_get_duty() Query current duty cycle, rx_motor_emergency_stop() Immediate shutdown for fault conditions, rx_pid_compute() PID controller for closed-loop velocity control, internal_clamp_duty() Internal duty cycle clamping function



Figure 1023: Call graph for rx_motor_set_duty

Defined in `libs/rx_motor/inc/rx_motor.h:550`

Since Version 1.0.0

—

rx_motor_stop

```
rx_err_t rx_motor_stop(rx_motor_handle_t *handle, bool brake)
```

Stop motor.

Stops the motor by either coasting (both outputs low) or braking (both outputs high, creating a short circuit brake).

Stop motor.

Stops motor rotation by setting both outputs to 0% duty cycle (coast mode). In IN2/IN1 control mode, active braking (short-circuit brake) is not supported - brake parameter is ignored and motor always coasts. Coast mode allows motor to free-wheel (high impedance), with deceleration due to friction and back-EMF.

IN2/IN1 Mode Behavior:

- Coast (brake=false): IN2 = LOW, IN1 = LOW -> Motor in high impedance
- Brake (brake=true): [FAIL] NOT SUPPORTED -> Falls back to coast, warning logged

Coast vs. Brake:

- Coast: Both outputs LOW -> H-bridge in high impedance -> Motor free-wheels
- Brake: Both outputs HIGH -> H-bridge shorts motor terminals -> Active braking
- Note: Brake mode requires IN/IN control mode (not available in IN2/IN1)

Use Cases:

- Normal stop: Gradual deceleration via friction
- Power saving: Disable PWM when motor not needed
- Direction change preparation: Stop before reversing

Motor deceleration is passive (friction only, no active braking)

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized motor handle. Must not be nullptr.
in	brake	If true, apply brake (both outputs high). If false, coast (both outputs low).
inout	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) current_duty field will be set to 0.0 on success
in	brake	Brake mode request (ignored in IN2/IN1 mode) true: Request active braking (NOT SUPPORTED, falls back to coast) false: Coast mode (supported, motor free-wheels) Warning logged if brake=true (user informed of fallback)

Returns: rx_err_t Error code indicating success or failure

Value	Description
-------	-------------

k_rx_ok	Success, motor stopped (coast mode)
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	Motor not initialized
k_rx_err_*	Propagated from rx_gptw_set_duty() (hardware write failed)

Pre: handle must be initialized via rx_motor_init()

Pre: GPTW peripheral must be operational

Post: On success: handle->current_duty = 0.0

Post: On success: Both outputs (IN2 and IN1) at 0% duty

Post: On success: Motor in coast mode (high impedance)

Post: Motor deceleration time depends on mechanical friction (100-500 ms for STAR)

Note: Thread-safe only with external synchronization

Note: Motor may continue spinning briefly after function returns (inertia)

Note: For immediate stop, use rx_motor_emergency_stop() (disables outputs)

Warning: Brake mode NOT supported in IN2/IN1 configuration

Warning: If brake=true, warning logged but function succeeds with coast

Performance:: Execution time: 2 us @ 240 MHz (2 GPTW register writes to zero)

Example - Normal Stop:: rx_motor_handle_tmotor;

```
rx_err_t err = rx_motor_stop(&motor, false); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to stop motor"); }
```

Example - Brake Request (Falls Back to Coast):: rx_err_t err = rx_motor_stop(&motor, true);

Example - Stop Before Direction Change:: rx_motor_set_duty(&motor, 75.0F);

```
tx_thread_sleep(5000);
```

```
rx_motor_stop(&motor, false); tx_thread_sleep(500);
```

```
rx_motor_set_duty(&motor, -75.0F);
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (NULL check, initialized check) Rule 5: [OK] 2 postconditions (current_duty = 0, outputs at 0%) Rule 7: [OK] Return values checked (rx_gptw_set_duty)

See also: `rx_motor_set_duty()` Set motor duty to 0 for same effect,
`rx_motor_emergency_stop()` Immediate stop with output disable, `rx_motor_deinit()` Stop
and release resources



Figure 1024: Call graph for `rx_motor_stop`

Defined in `libs/rx_motor/inc/rx_motor.h:566`

Since Version 1.0.0

—

rx_motor_get_duty

```
rx_err_t rx_motor_get_duty(const rx_motor_handle_t *handle, float *out_duty)
```

Get current motor duty cycle.

Get current motor duty cycle.

Retrieves the most recently commanded duty cycle from motor handle. Returns the signed duty cycle value that was set via rx_motor_set_duty() or rx_motor_stop(). This is the commanded duty cycle (what was requested), not the actual motor speed (which requires encoder feedback).

Returned Value Meaning:

- Positive: Forward direction (0 to +100%)
- Negative: Reverse direction (0 to -100%)
- Zero: Stopped/coast (0%)
- Range: [-100.0, +100.0]

Use Cases:

- Monitor current command state
- Logging and telemetry
- State verification after commands
- PID controller feedback (open-loop command, not actual velocity)

Note: This returns commanded duty, not actual motor velocity. For actual velocity, use encoder feedback (rx_encoder_get_velocity_mps).

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized motor handle. Must not be nullptr.
out	out_duty	Pointer to store current duty cycle. Must not be nullptr.
in	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) Handle state is not modified (const qualified)
out	out_duty	Pointer to float variable to receive duty cycle Must not be NULL (validated) Will be set to current duty cycle [-100.0, +100.0] Value remains unchanged if function returns error

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, duty cycle written to *out_duty
k_rx_err_null_ptr	handle or out_duty is nullptr
k_rx_err_invalid_state	Motor not initialized

Pre: handle must be initialized via rx_motor_init()

Pre: out_duty must point to valid float variable

Post: On success: *out_duty contains current commanded duty cycle

Post: On failure: *out_duty unchanged

Note: Thread-safe for read-only access (handle is const)

Note: Returns commanded duty, not actual motor velocity

Note: Very fast operation (0.5 us) - just a memory read

Performance:: Execution time: 0.5 us @ 240 MHz (single memory read) Safe to call at high frequency (tested > 10 kHz)

Example - Query Current State:: rx_motor_handle_t motor;

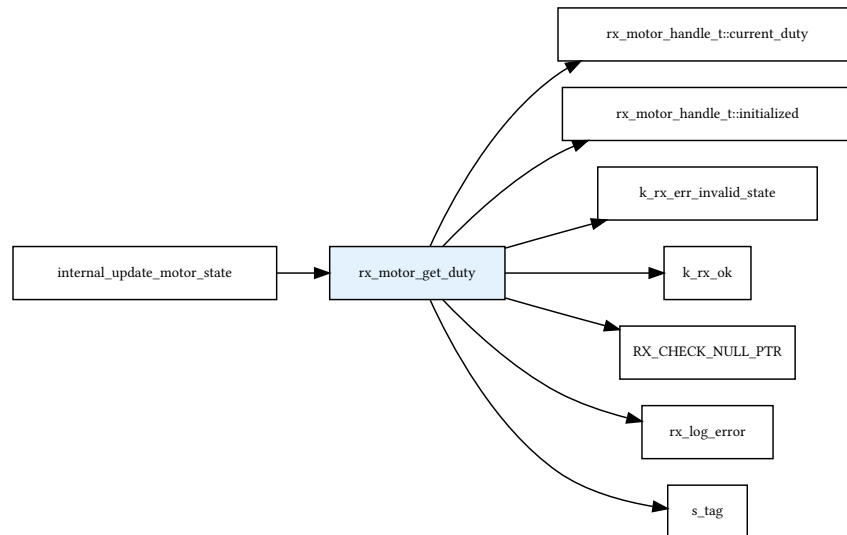
```
float current_duty; rx_err_t err = rx_motor_get_duty(&motor, &current_duty); if (err == k_rx_ok)
{ rx_log_info("APP", "Motor duty: %.1f%%", current_duty); if (current_duty > 0) { } elseif (current_duty < 0) { }
else { } }
```

Example - Telemetry Logging:: void telemetry_task(void* arg)

```
{ rx_motor_handle_t* motor = (rx_motor_handle_t*) arg; while(1) { float duty;
if (rx_motor_get_duty(motor, &duty) == k_rx_ok) { telemetry_log("motor_duty_pct", duty); }
tx_thread_sleep(100); } }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (NULL checks for handle and out_duty) Rule 5: [OK] 1 postcondition (out_duty contains current duty on success)

See also: rx_motor_set_duty() Set duty cycle, rx_encoder_get_velocity_mps() Get actual motor velocity from encoder

Figure 1025: Call graph for `rx_motor_get_duty`

Defined in `libs/rx_motor/inc/rx_motor.h:578`

Since Version 1.0.0

—

rx_motor_emergency_stop

```
rx_err_t rx_motor_emergency_stop(rx_motor_handle_t *handle)
```

Emergency stop - disable GPTW outputs immediately.

Disables GPTW peripheral outputs at hardware level for fail-safe operation. Motor CANNOT be re-enabled without calling `rx_motor_init()` again.

This function provides hardware-level safety cutoff for emergency situations such as control loop failures, sensor faults, or safety violations.

Safety features:

- Immediately sets duty to 0%
- Disables GPTW outputs at hardware level
- Stops timer to prevent glitches
- Marks handle as uninitialized (requires re-init to use)

Emergency stop - disable GPTW outputs immediately.

Performs emergency shutdown of motor control with multiple layers of safety:

1. Set duty to 0% - Software command to stop PWM
2. Disable outputs - Hardware-level output disable at GPTW peripheral
3. Stop timer - Halt GPTW timer to prevent glitches
4. Mark uninitialized - Prevent further use until reinitialization

This is the most aggressive stop mechanism, intended for fault conditions, safety shutdowns, or emergency stop button presses. Unlike `rx_motor_stop()`, this function disables outputs at the

hardware level and marks the handle as uninitialized, requiring `rx_motor_init()` before motor can be used again.

Emergency Stop Sequence:

Error Handling:

- Best-effort approach: Continues through all steps even if some fail
- First error is saved and returned to caller
- All errors logged with “E-STOP:” prefix for visibility
- Motor is maximally disabled even on partial failures

Difference from `rx_motor_stop()`:

When to Use:

- Hardware fault detected (overcurrent, overvoltage, etc.)
- Emergency stop button pressed
- Watchdog timeout or safety violation
- Unrecoverable software error
- System shutdown or panic

Even on error, motor is maximally disabled (safe state)

If function returns, `handle->initialized == false`

Affects entire GPTW channel (may impact other motors on same channel)

Parameters:

Direction	Name	Description
in	handle	Pointer to motor handle. Must not be nullptr.
inout	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via <code>handle->initialized</code> flag) Will be marked uninitialized (<code>initialized = false</code>) on completion Requires <code>rx_motor_init()</code> before motor can be used again

Returns: `rx_err_t` Error code indicating success or first failure encountered

Value	Description
<code>k_rx_ok</code>	Success, all emergency stop steps completed without errors
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	Motor not initialized (already in emergency stop state)
<code>k_rx_err_*</code>	First error encountered during shutdown sequence (duty, disable, or stop)

Pre: handle must be initialized via `rx_motor_init()`

Pre: No other tasks are accessing this motor handle

Post: `handle->initialized = false` (marked as uninitialized)

Post: handle->current_duty = 0.0 (internal state cleared)

Post: Motor outputs disabled at hardware level (high impedance)

Post: GPTW timer stopped (no PWM generation possible)

Post: Motor cannot be used until rx_motor_init() called again

Note: After emergency stop, motor requires rx_motor_init() to operate again

Note: Best-effort shutdown - continues through all steps even if some fail

Note: First error encountered is returned, but subsequent steps still execute

Note: All errors logged with "E-STOP:" prefix for immediate visibility

Note: Motor requires reinitialization (rx_motor_init) before reuse

Warning: This is a safety function - use for emergency conditions only

Warning: This is a DESTRUCTIVE operation - handle cannot be reused without reinit

Warning: Do not use for normal stops - use rx_motor_stop() instead

Thread Safety:: Not thread-safe. Ensure no other tasks are accessing motor during emergency stop.

Performance:: Execution time: 8 us @ 240 MHz (multiple hardware operations) Not performance-critical (emergency/fault scenario)

Example - Fault Handler:: rx_motor_handle_tmotor;

```
if(motor_current_ma>k_max_current_ma){ rx_log_error("SAFETY","Overcurrentdetected:
%dmA",motor_current_ma);

rx_err_t err=rx_motor_emergency_stop(&motor); if(err!=k_rx_ok){ rx_log_error("SAFETY","E-
STOPfailed:%d(motormaximallydisabled)",err); }

}
```

Example - Emergency Stop Button:: void emergency_stop_isr(void)

```
{ g_emergency_stop_requested=true; }

void motor_control_task(void*arg){ rx_motor_handle_t* motors=(rx_motor_handle_t*)arg;

while(1){ if(g_emergency_stop_requested){ rx_log_warn("SAFETY","Emergency stop button pressed!");

for(uint8_t i=0;i<4;i++){ (void)rx_motor_emergency_stop(&motors[i]); }
```

```
g_emergency_stop_requested=false;
wait_for_estop_reset();
for(uint8_ti=0;i<4;i++){ rx_motor_init(&motors[i],&motor_configs[i]); }
tx_thread_sleep(10); }
```

Example - Watchdog Timeout Handler:: void watchdog_timeout_callback(void)
{ rx_log_error("WDT","Watchdogtimeout-emergencystopallmotors");

extern rx_motor_handle_tg_motors[4]; for(uint8_ti=0;i<4;i++)
{ (void)rx_motor_emergency_stop(&g_motors[i]); }

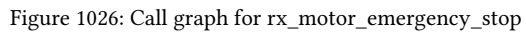
}

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential error handling
Rule 5: [OK] 2 preconditions (NULL check, initialized check) Rule 5: [OK] 3 postconditions
(initialized flag cleared, duty cleared, outputs disabled) Rule 7: [OK] All return values checked (best-effort error collection)

Example:

```
1.Setoutput_aduty->0%(IN2signalLOW)
2.Setoutput_bduty->0%(IN1signalLOW)
3.Disableoutput_aathardwarelevel(GPTWperipheral)
4.Disableoutput_bathardwarelevel(GPTWperipheral)
5.StopGPTWtimer(preventanyfurtherPWMgeneration)
6.Clearinitializedflag(handlenowinvaliduntilreinitialized)
7.Setcurrent_duty=0(internalstateconsistency)
```

See also: rx_motor_stop() Normal stop (does not require reinit), rx_motor_deinit()
Orderly shutdown (stop + cleanup), rx_motor_init() Required to reinitialize after
emergency stop



Defined in `libs/rx_motor/inc/rx_motor.h:604`

Since Version 1.0.0

—

rx_motor.c

Brushed DC Motor Control Implementation for RX72N GPTW PWM with DRV8263H H-Bridge.

Includes:

- `"rx_motor.h"`
- `<math.h>`
- `"rx_check.h"`
- `"rx_log.h"`

motor_duty_limits_t

Motor control duty cycle constants for DRV8263H IN2/IN1 mode operation.

Defines valid duty cycle ranges and special values for IN2/IN1 control mode. In IN2/IN1 mode, the duty cycle sign determines direction (+ = forward, - = reverse) and magnitude determines speed (0-100%). These constants provide named values for duty cycle limits and direction encoding for the IN2 (direction) signal.

Usage Context:

- Duty cycle input validation in `rx_motor_set_duty()`
- IN2 signal encoding (HIGH=forward, LOW=reverse)
- IN1 signal magnitude extraction (absolute value of duty)

Value	Name	Description
= -100	<code>k_motor_duty_min</code>	Minimum duty cycle (-100% = full reverse).
= 100	<code>k_motor_duty_max</code>	Maximum duty cycle (+100% = full forward).
= 0	<code>k_motor_duty_zero</code>	Zero duty cycle (0% = stopped/coast).

See also: `internal_clamp_duty()` Clamps duty to valid range, `rx_motor_set_duty()` Uses these constants for input validation

—

motor_in2_signal_t

IN2 direction signal values for DRV8263H H-bridge.

Encodes the IN2 pin duty percentage that determines motor rotation direction in the DRV8263H IN2/IN1 control mode. IN2 is driven as a static duty level (100% HIGH or 0% LOW) while IN1 carries the speed PWM.

Values must be either 0 (LOW) or 100 (HIGH) - no intermediate duty levels are valid for the IN2 direction pin in DRV8263H IN2/IN1 control mode.

Value	Name	Description
= 100	<code>k_motor_in2_high</code>	IN2 forward direction (100% = HIGH). Sets output_a HIGH for forward rotation. H-bridge: high-side A ON, low-side B ON

= 0	k_motor_in2_low	IN2 reverse direction (0% = LOW). Sets output_a LOW for reverse rotation. H-bridge: high-side B ON, low-side A ON
-----	-----------------	---

See also: rx_motor_set_duty() Uses these values to set direction

Example:

```
//Setforwarddirection:IN2=HIGH(100%)
rx_err_t err=rx_gptw_set_duty(rx_gptw_channel_id(channel),
rx_gptw_output_id(output_a),
(float)k_motor_in2_high);

//Setreversedirection:IN2=LOW(0%)
err=rx_gptw_set_duty(rx_gptw_channel_id(channel),
rx_gptw_output_id(output_a),
(float)k_motor_in2_low);

//Coastmode:IN2=LOW, IN1=LOW(highimpedance)
err=rx_gptw_set_duty(rx_gptw_channel_id(channel),
rx_gptw_output_id(output_a),
(float)k_motor_in2_low);
err=rx_gptw_set_duty(rx_gptw_channel_id(channel),
rx_gptw_output_id(output_b),
(float)k_motor_duty_zero);
```

Since Version 1.0.0

—

motor_validation_limits_t

Configuration parameter validation limits for NASA Rule 5 compliance.

Defines valid ranges for motor configuration parameters (PWM frequency and dead-time). These limits ensure safe H-bridge operation and prevent hardware damage from invalid configurations. All values are hardware-constrained based on:

- GPTW timer capabilities (1 kHz - 50 kHz practical range)
- FET switching characteristics (500-800 ns turn-off time)
- Motor inductance and back-EMF characteristics
- EMI considerations and PCB layout constraints

NASA Rule 5 Compliance: All input parameters validated against these limits in rx_motor_init() to prevent undefined behavior and hardware damage.

Value	Name	Description
= 1000	k_motor_min_pwm_freq	Minimum PWM frequency (1 kHz).
= 50000	k_motor_max_pwm_freq	Maximum PWM frequency (50 kHz).
= 100	k_motor_min_dead_time	Minimum dead-time (100 ns).
= 10000	k_motor_max_dead_time	Maximum dead-time (10 us).

See also: rx_motor_init() Validates config parameters against these limits,
rx_motor_config_t Configuration structure with these fields

—

```
s_tag
const char s_tag[]
```

—

internal_clamp_duty

```
static float internal_clamp_duty(const float duty)
```

Clamp duty cycle to valid range $[-100, +100]$ with NaN/Inf protection.

Bounds-checking function that ensures duty cycle input remains within valid range for motor control. Provides additional safety by detecting invalid IEEE 754 float values (NaN, Inf) which could occur from arithmetic errors or uninitialized variables.

Algorithm:

1. Check for NaN (Not-a-Number) or Inf (Infinity) -> return 0 (safe default)
2. If duty > +100 -> clamp to +100 (full forward)
3. If duty < -100 -> clamp to -100 (full reverse)
4. Otherwise -> return duty unchanged

Rationale for NaN/Inf check:

- Prevents propagation of invalid float values to hardware registers
- Returns safe default (0 = motor stopped) rather than undefined behavior
- NASA Rule 5 compliance: validate all inputs before use

Return value in $[-100.0, +100.0]$

Parameters:

Direction	Name	Description
in	duty	Duty cycle percentage to clamp Valid range: $[-100.0, +100.0]$ Units: Percent (%) Special values: NaN/Inf -> treated as 0.0 Type: float (IEEE 754 single precision)

Returns: Clamped duty cycle percentage

Value	Description
$[-100.0]$	Valid duty cycle (clamped to range)
0.0	If input was NaN or Inf (safe default)

Pre: None (this function handles all input values safely)

Post: Return value is always in range $-100.0, +100.0$ or exactly 0.0

Post: Return value is always a valid, finite float (never NaN/Inf)

Note: Pure function with no side effects (safe to call multiple times)

Note: Thread-safe (no shared state access)

Note: Does NOT log clamping events (called frequently in control loop)

Performance:: Execution time: 0.2 us @ 240 MHz with -O2 optimization Best case: 3 comparisons (normal range) Worst case: 5 comparisons (NaN/Inf check + clamping) No branches misprediction overhead (predictable for valid inputs)

Example:: floatduty_cmd=150.0F; floatsafe_duty=internal_clamp_duty(duty_cmd);

floatinvalid=0.0F/0.0F; floatsafe=internal_clamp_duty(invalid);

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, simple if-else logic Rule 5: [OK] Pre-condition: none (accepts all float values) Rule 5: [OK] Post-condition: output in valid range [-100, +100]

See also: rx_motor_set_duty() Calls this function to validate user input, k_motor_duty_min Minimum duty cycle constant (-100), k_motor_duty_max Maximum duty cycle constant (+100), k_motor_duty_zero Zero duty cycle constant (0)

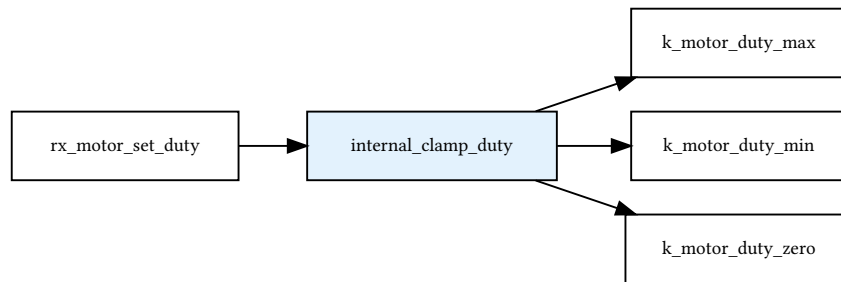


Figure 1027: Call graph for internal_clamp_duty

Defined in `libs/rx_motor/src/rx_motor.c:505`

Since Version 1.0.0

—

internal_init_gptw_outputs

```
static rx_err_t internal_init_gptw_outputs(const rx_gptw_channel_t channel, const
rx_gptw_output_pair_t outputs, const rx_gptw_config_t *gptw_config)
```

Initialize GPTW PWM peripheral and set both outputs to safe state (0% duty).

Internal helper function that performs three critical initialization steps for motor control:

1. Validate output configuration - Ensure outputs A and B are valid and different
2. Initialize GPTW PWM peripheral - Configure timer frequency, dead-time, polarity
3. Set outputs to safe state - Both outputs to 0% duty (motor coast/stopped)

Safety-Critical Behavior:

- Both outputs initialized to 0% duty BEFORE enabling PWM (prevents motor startup surge)
- Validation prevents same pin assigned to both IN2 and IN1 (would prevent direction control)
- On any error, partially initialized state is cleaned up (GPTW deinitialized)

Algorithm:

1. Validate gptw_config pointer (NULL check)
2. Validate outputs.a and outputs.b are valid GPTW outputs
3. Validate outputs.a != outputs.b (different pins)
4. Initialize GPTW peripheral with frequency, dead-time, polarity settings
5. Set output_a to 0% duty (IN2 signal starts LOW)
6. Set output_b to 0% duty (IN1 signal starts LOW)
7. On error during steps 5-6: deinitialize GPTW and propagate error

Error Handling:

- If rx_gptw_init_pwm() fails -> return error immediately (no cleanup needed)
- If rx_gptw_set_duty() fails -> deinitialize GPTW before returning error
- Ensures no partial initialization state remains after failure

If function returns k_rx_ok, GPTW peripheral is initialized and outputs are at 0%

On error, GPTW is deinitialized to prevent partial initialization state

Parameters:

Direction	Name	Description
in	channel	GPTW timer channel to use for PWM generation Valid values: k_gptw_channel_0 through k_gptw_channel_7 Must support PWM mode and dead-time insertion Must have unused/available outputs for motor control
in	outputs	Output pin pair for IN2/IN1 mode control outputs.a: IN2 (direction) signal, direction control outputs.b: IN1 (PWM/enable) signal, speed PWM Must be from same GPTW channel Must be different (a != b)
in	gptw_config	GPTW peripheral configuration parameters Must not be nullptr frequency_hz: PWM frequency [1 kHz, 50 kHz] deadtime_ns: Dead-time insertion [100 ns, 10 us] enable_complementary: Should be false for IN2/IN1 mode invert_polarity: true if H-bridge needs inverted PWM

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, GPTW initialized and outputs set to 0% duty
k_rx_err_null_ptr	gptw_config pointer is nullptr
k_rx_err_invalid_arg	outputs.a or outputs.b not valid GPTW output
k_rx_err_invalid_arg	outputs.a == outputs.b (same pin assigned twice)
k_rx_err_*	Propagated from rx_gptw_init_pwm() (GPTW initialization failed)
k_rx_err_*	Propagated from rx_gptw_set_duty() (duty cycle write failed)

Pre: channel must be a valid GPTW channel with available outputs

Pre: outputs.a and outputs.b must be valid GPTW outputs from same channel

Pre: outputs.a != outputs.b (different physical pins)

Pre: gptw_config parameters must be within valid ranges

Post: On success: GPTW initialized, both outputs at 0% duty, PWM disabled

Post: On failure: GPTW deinitialized (no partial initialization remains)

Post: Motor remains in safe state (stopped/coast) after function returns

Note: Not thread-safe, caller must ensure exclusive access

Note: Called only from rx_motor_init(), not exposed in public API

Note: Errors are logged with detailed messages for debugging

Warning: Do not call directly - use rx_motor_init() instead

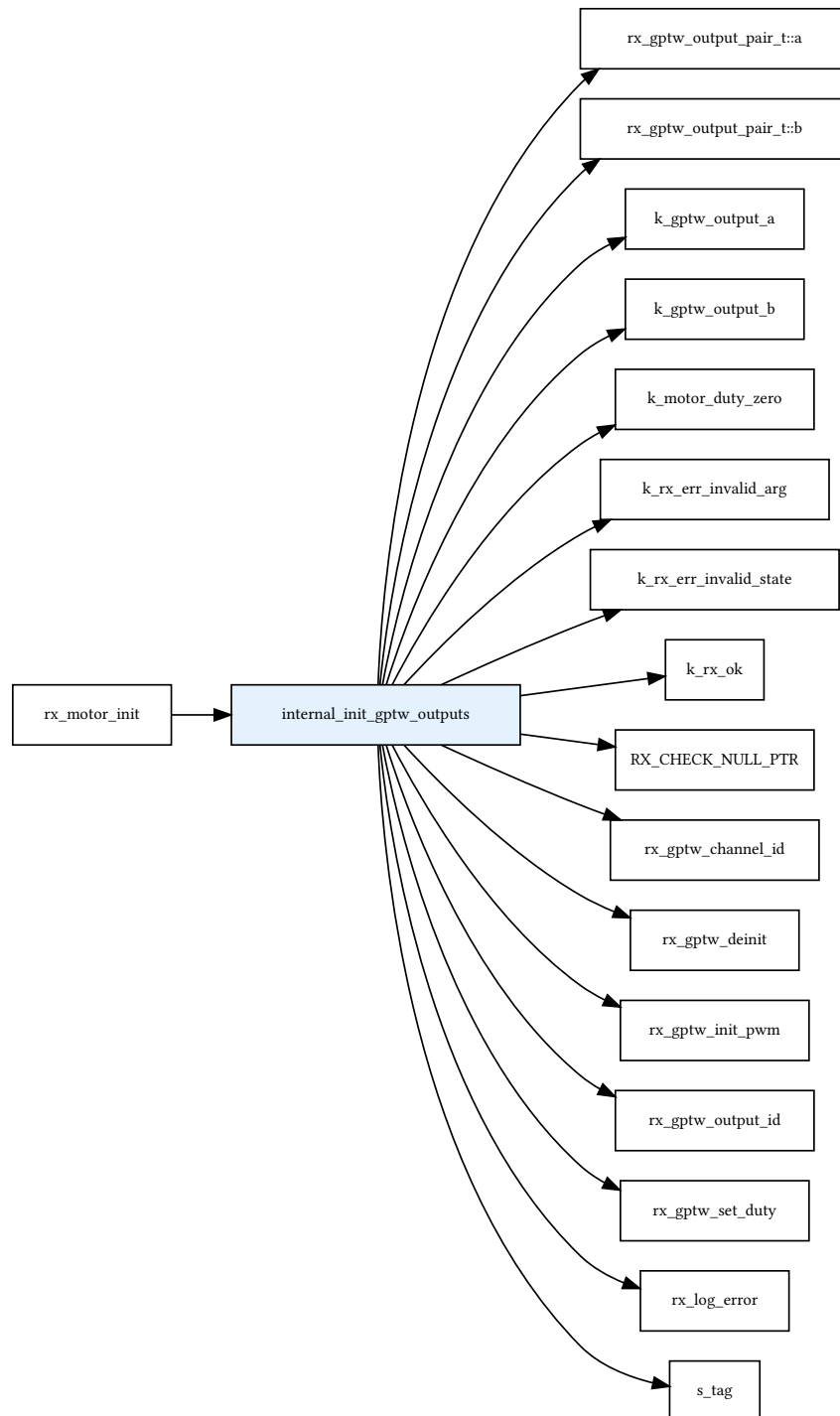
Performance:: Execution time: 12 us @ 240 MHz (includes GPTW register writes) Called once per motor during initialization (not performance-critical)

Example (Internal Usage)::

```
rx_gptw_config_tgptw_config={.frequency_hz=20000,.deadtime_ns=1000,.enable_complementary=false,.invert_pol:
rx_gptw_output_pair_toutputs={.a=k_gptw_output_a,.b=k_gptw_output_b,};
rx_err_terr=internal_init_gptw_outputs(k_gptw_channel_0,outputs,&gptw_config); if(err!=k_rx_ok)
{ returnerr; }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential error checking
Rule 5: [OK] 3 preconditions (channel valid, outputs valid, config non-NULL) Rule 5: [OK] 2
postconditions (GPTW initialized on success, deinitialized on failure) Rule 7: [OK] All return values
checked (rx_gptw_init_pwm, rx_gptw_set_duty)

See also: rx_motor_init() Calls this function during motor initialization,
rx_gptw_init_pwm() GPTW peripheral initialization, rx_gptw_set_duty() Set individual
output duty cycle, rx_gptw_deinit() Cleanup on error, rx_gptw_output_pair_t Output pair
structure definition

Figure 1028: Call graph for `internal_init_gptw_outputs`

Defined in `libs/rx_motor/src/rx_motor.c:673`

Since Version 1.0.0

—

rx_motor_init

```
rx_err_t rx_motor_init(rx_motor_handle_t *handle, const rx_motor_config_t
*config)
```

Initialize motor controller with H-bridge configuration for bidirectional control.

Initialize motor controller with GPTW PWM.

Performs complete initialization of brushed DC motor control system including GPTW PWM peripheral configuration, output pin setup, and handle state initialization. This function must be called before any other motor control operations.

Initialization Sequence:

1. Input validation - Verify handle and config pointers, check initialization state
2. Parameter validation - Validate PWM frequency and dead-time against hardware limits
3. GPTW configuration - Prepare GPTW config struct with frequency, dead-time, polarity
4. GPTW initialization - Initialize PWM peripheral and set outputs to 0% (safe state)
5. Handle setup - Save configuration parameters to handle structure
6. Post-condition check - Verify handle was properly initialized (NASA Rule 5)

Safety Features:

- All outputs initialized to 0% duty (motor stopped/coast) before function returns
- Input parameter validation prevents invalid configurations from reaching hardware
- Handle marked initialized only after successful setup (prevents use before init)
- Post-condition verification catches initialization errors

Configuration Parameters:

- `pwm_freq_hz`: PWM frequency [1 kHz, 50 kHz], typical 20 kHz for inaudible operation
- `dead_time_ns`: Dead-time insertion [100 ns, 10 us], typical 1 us for shoot-through protection
- `channel`: GPTW channel to use (`k_gptw_channel_0` through `k_gptw_channel_7`)
- `output_a`: IN2 (direction) signal pin for direction control
- `output_b`: IN1 (PWM/enable) signal pin for speed PWM
- `invert_pwm`: Polarity inversion (true = inverted, false = normal)

If function returns `k_rx_ok`, `handle->initialized == true`

If function returns error, `handle->initialized == false`

PWM frequency and dead-time cannot be changed without deinitialization

Multiple motors can share same GPTW channel if outputs are different

1.0.0 Initial implementation with IN2/IN1 mode support

Parameters:

Direction	Name	Description
out	handle	Pointer to motor handle structure to initialize Must not be nullptr Must not already be initialized (checked via <code>handle->initialized</code> flag) Will be

		populated with configuration on success Structure should be allocated by caller (typically stack allocation) Contents undefined on error
in	config	Pointer to motor configuration parameters Must not be nullptr pwm_freq_hz: [1000, 50000] Hz (validated) dead_time_ns: [100, 10000] ns (validated) channel: Valid GPTW channel with available outputs output_a, output_b: Valid GPTW outputs, must be different invert_pwm: true (inverted) or false (normal) Configuration is copied to handle (config can be stack-allocated)

Returns: rx_err_t Error code indicating success or failure reason

Value	Description
k_rx_ok	Success, motor initialized and ready for use
k_rx_err_null_ptr	handle or config pointer is nullptr
k_rx_err_invalid_state	handle already initialized (must call rx_motor_deinit first)
k_rx_err_invalid_arg	pwm_freq_hz out of range [1 kHz, 50 kHz]
k_rx_err_invalid_arg	dead_time_ns out of range [100 ns, 10 us]
k_rx_err_invalid_arg	output_a or output_b invalid GPTW output
k_rx_err_invalid_arg	output_a == output_b (same pin used twice)
k_rx_err_invalid_state	Post-condition check failed (initialization verification)
k_rx_err_*	Propagated from rx_gptw_init_pwm() or rx_gptw_set_duty()

Pre: handle must point to valid, uninitialized rx_motor_handle_t structure

Pre: config must point to valid rx_motor_config_t with parameters in valid ranges

Pre: GPTW peripheral must be available (not used by other motor instance)

Pre: handle->initialized must be false (or uninitialized memory)

Post: On success: handle->initialized = true, motor ready for commands

Post: On success: Motor in coast state (both outputs 0%, high impedance)

Post: On success: handle contains copy of all config parameters

Post: On failure: handle->initialized remains false, GPTW not initialized

Note: Not thread-safe, do not call simultaneously on same handle

Note: Configuration is copied to handle, original config can be freed/reused

Note: Motor remains stopped after initialization (call rx_motor_set_duty to move)

Note: Can only initialize once per handle (call rx_motor_deinit to reinitialize)

Warning: Must be called before any other motor control functions

Warning: Do not modify handle contents directly after initialization

Warning: Ensure handle remains in scope for entire motor lifetime

Thread Safety:: Not thread-safe. If multiple tasks need motor access, use ThreadX mutex around all motor operations or allocate one motor handle per task.

Performance:: Execution time: 18 us @ 240 MHz (includes GPTW initialization) One-time overhead, called once per motor at system startup Not performance-critical (initialization phase)

Example - Single Motor::

```
rx_motor_config_t motor0_config = { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 1000 };
rx_motor_handle_t motor0; rx_err_t err = rx_motor_init(&motor0, &motor0_config); if (err != k_rx_ok) { rx_log_error("APP", "Failed to initialize motor 0: %d", err); return err; }
```

Example - Four Motors (STAR Platform):: rx_motor_handle_t motors[4];

```
const rx_motor_config_t motor_configs[4] = { { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 1000 },
{ .channel = k_gptw_channel_1, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 1000 },
{ .channel = k_gptw_channel_2, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 1000 },
{ .channel = k_gptw_channel_3, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 1000 } };
for (uint8_t i = 0; i < 4; i++) { rx_err_t err = rx_motor_init(&motors[i], &motor_configs[i]); if (err != k_rx_ok) { rx_log_error("APP", "Failed to initialize motor %d", i); for (uint8_t j = 0; j < i; j++) { (void)rx_motor_deinit(&motors[j]); } return err; } }
rx_log_info("APP", "All 4 motors initialized successfully");
```

Example - Error Handling::

```
rx_motor_config_t config = { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b, .pwm_freq_hz = 20000, .dead_time_us = 1000 };
rx_motor_handle_t motor; rx_err_t err = rx_motor_init(&motor, &config);
switch (err) { case k_rx_ok: break; case k_rx_err_invalid_arg: rx_log_error("APP", "Invalid config (check freq, dead-time, outputs)"); break; case k_rx_err_invalid_state: rx_log_error("APP", "Motor already initialized or init verification failed"); break; default: rx_log_error("APP", "Unexpected error: %d", err); break; }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential error checking Rule 3: [OK] No dynamic memory allocation (handle passed by caller) Rule 4: [OK] Function length: 61 lines (within 60-line guideline) Rule 5: [OK] 4 preconditions (NULL checks, state check, range validation) Rule 5: [OK] 2 postconditions (initialized flag set, config copied) Rule 7: [OK] All return values checked (internal_init_gptw_outputs) Rule 8: [OK] C23 typed enums for validation constants

See also: `rx_motor_deinit()` Cleanup function, must call to reinitialize,
`rx_motor_set_duty()` Set motor speed/direction after initialization, `rx_motor_config_t`
Configuration structure definition, `rx_motor_handle_t` Handle structure definition,
`rx_gptw_init_pwm()` Underlying GPTW initialization

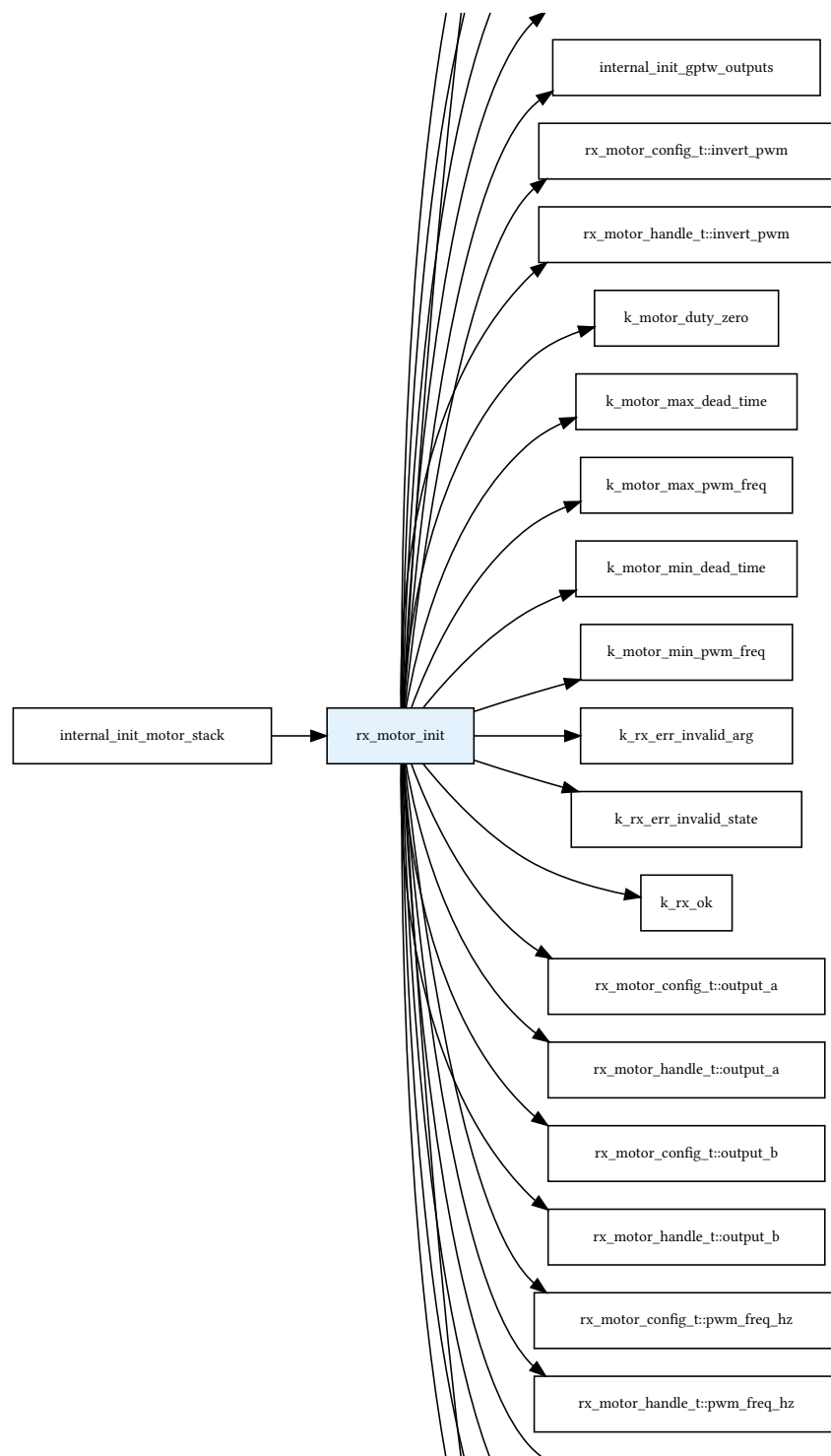


Figure 1029: Call graph for `rx_motor_init`

Defined in `libs/rx_motor/src/rx_motor.c:924`

Since Version 1.0.0

—

rx_motor_deinit

```
rx_err_t rx_motor_deinit(rx_motor_handle_t *handle)
```

Deinitialize motor controller and release GPTW peripheral resources.

Deinitialize motor controller and release resources.

Performs orderly shutdown of motor control system by stopping motor outputs and releasing GPTW peripheral resources. After deinitialization, handle can be reinitialized via `rx_motor_init()` with different configuration parameters.

Shutdown Sequence:

1. Validate handle pointer (NULL check)
2. Verify handle is initialized (prevent double-deinit)
3. Stop motor outputs (coast mode, 0% duty on both pins)
4. Deinitialize GPTW peripheral (release hardware resources)
5. Clear initialized flag (mark handle as deinitialized)

Use Cases:

- System shutdown: Release hardware before power-down
- Reconfiguration: Change PWM frequency or dead-time (requires deinit -> init)
- Error recovery: Clean up after hardware fault
- Resource sharing: Free GPTW channel for other use

Failure to deinit before program exit may leave GPTW in active state

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) Will be marked uninitialized (initialized = false) on success Contents remain valid but motor operations will fail until reinitialized

Returns: `rx_err_t` Error code indicating success or failure

Value	Description
<code>k_rx_ok</code>	Success, motor deinitialized and GPTW released
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	Motor not initialized (already deinitialized)
<code>k_rx_err_*</code>	Propagated from <code>rx_gptw_deinit()</code> (hardware cleanup failed)

Pre: handle must be initialized via `rx_motor_init()`

Pre: No other tasks are currently accessing this motor handle

Post: handle->initialized = false (marked as deinitialized)

Post: Motor outputs disabled (high impedance, 0% duty)

Post: GPTW peripheral deinitialized (channel available for reuse)

Post: Handle can be reinitialized with rx_motor_init()

Note: If rx_motor_stop() fails, warning logged but deinit continues

Note: GPTW deinit affects entire channel (may impact other outputs on same channel)

Note: Thread-safe only with external synchronization

Warning: Ensure motor is not needed before calling (deactivates all control)

Thread Safety:: Not thread-safe. Ensure no other tasks are using motor handle during deinit.

Performance:: Execution time: 8 us @ 240 MHz (includes GPTW peripheral cleanup) Called once per motor during shutdown (not performance-critical)

Example - Clean Shutdown:: rx_motor_handle_tmotor;

```
rx_err_t err = rx_motor_deinit(&motor); if(err != k_rx_ok) { rx_log_error("APP", "Failed to deinit motor: %d", err); }
```

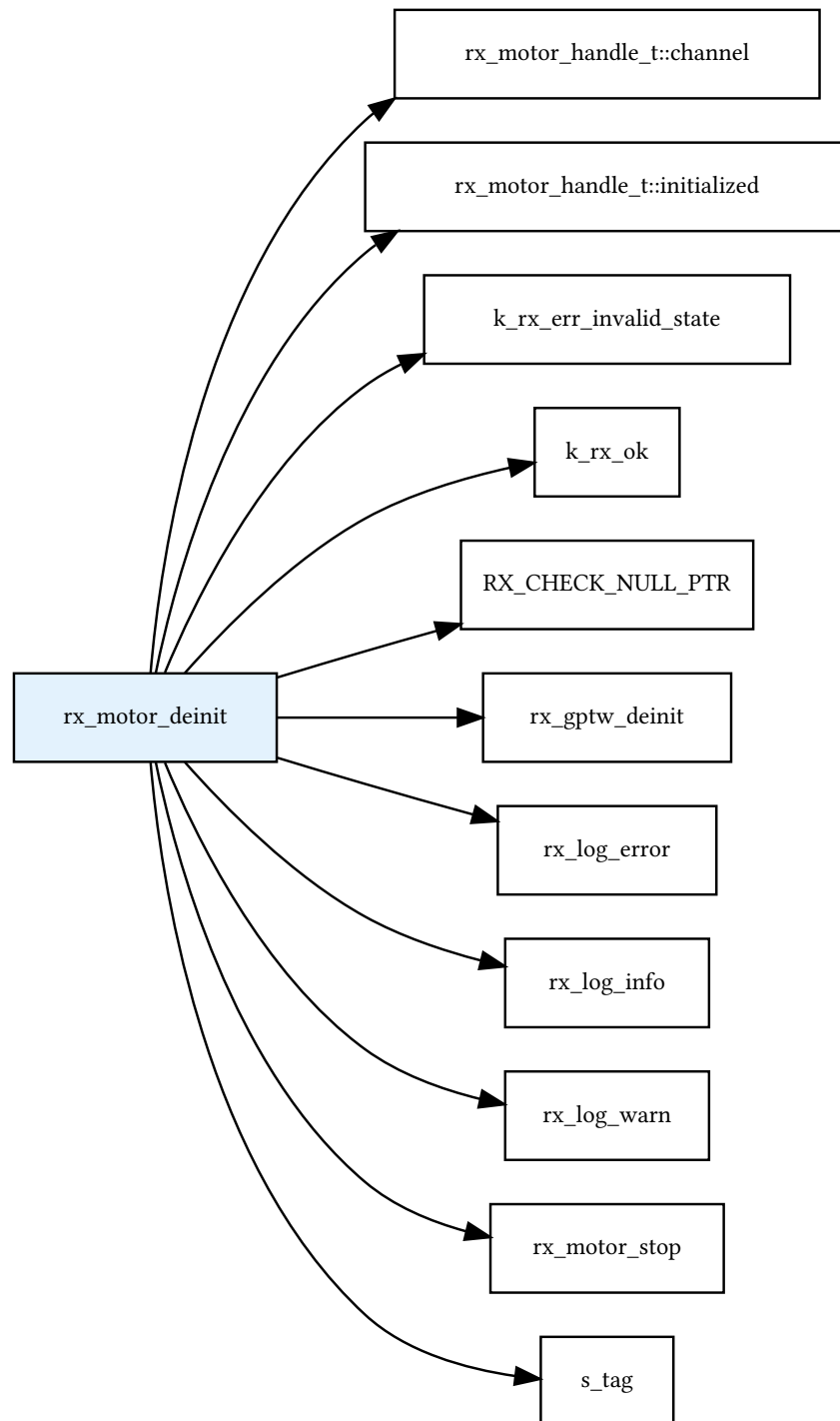
Example - Reconfiguration:: rx_motor_handle_tmotor;

```
rx_err_t err = rx_motor_deinit(&motor); if(err != k_rx_ok) { return err; }
```

```
rx_motor_config_t new_config = { .channel = k_gptw_channel_0, .output_a = k_gptw_output_a, .output_b = k_gptw_output_b };
err = rx_motor_init(&motor, &new_config);
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (NULL check, initialized check) Rule 5: [OK] 1 postcondition (initialized flag cleared) Rule 7: [OK] Return values checked (rx_motor_stop logged, rx_gptw_deinit propagated)

See also: rx_motor_init() Initialize motor (required before reuse after deinit), rx_motor_stop() Stop motor without deinitializing, rx_motor_emergency_stop() Emergency shutdown (also deinitializes), rx_gptw_deinit() Underlying GPTW peripheral cleanup

Figure 1030: Call graph for `rx_motor_deinit`

Defined in `libs/rx_motor/src/rx_motor.c:1093`

Since Version 1.0.0

—

rx_motor_set_duty

```
rx_err_t rx_motor_set_duty(rx_motor_handle_t *handle, float duty)
```

Set motor speed and direction via signed duty cycle (-100% to +100%).

Set motor duty cycle.

Primary motor control function for setting speed and direction with a single signed duty cycle value. Positive values drive motor forward, negative values drive motor in reverse, and zero stops the motor (coast mode). Internally converts signed duty cycle into IN2/IN1 signal pair for H-bridge control.

Control Mapping (IN2/IN1 Mode):

- duty > 0 (Forward):

IN2 (output_a) = HIGH (100% duty) -> Direction = forward IN1 (output_b) = |duty| PWM -> Speed = magnitude Example: duty = +75% -> IN2 = HIGH, IN1 = 75% PWM

- duty < 0 (Reverse):

IN2 (output_a) = LOW (0% duty) -> Direction = reverse IN1 (output_b) = |duty| PWM -> Speed = magnitude Example: duty = -50% -> IN2 = LOW, IN1 = 50% PWM

- duty = 0 (Coast):

IN2 (output_a) = LOW (0% duty) IN1 (output_b) = LOW (0% duty) Motor free-wheels (high impedance)

Algorithm:

Safety Features:

- Duty cycle clamping prevents hardware over-drive ($|duty| \leq 100\%$)
- NaN/Inf detection prevents undefined hardware register values
- Initialization check prevents operation on uninitialized peripheral
- Post-condition verification catches update failures (NASA Rule 5)

Performance:

- Typical case: 3 μ s @ 240 MHz (2 GPTW register writes)
- Suitable for 100 Hz - 10 kHz control loops
- STAR platform: Called at 100 Hz from PID velocity controller

If function returns `k_rx_ok`, `handle->current_duty == commanded_duty`

Motor duty always in range $[-100.0, +100.0]$ after function call

If `invert_pwm` was configured, duty is inverted internally

Post-condition failure indicates serious hardware fault (investigate immediately)

1.0.0 Initial implementation with IN2/IN1 mode

Parameters:

Direction	Name	Description
-----------	------	-------------

in	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized via rx_motor_init() (validated) Handle state will be updated with new duty value current_duty field reflects actual commanded duty after return
in	duty	Signed duty cycle percentage (speed and direction) Valid range: [−100.0, +100.0] Values outside range are clamped (no error returned) Units: Percent (%) Positive: Forward rotation, negative: Reverse rotation Zero: Motor coast (high impedance) NaN/Inf: Rejected with k_rx_err_invalid_arg Resolution: Float precision (7 decimal digits)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, motor duty updated, outputs configured
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	Motor not initialized (call rx_motor_init first)
k_rx_err_invalid_arg	duty is NaN or Inf (invalid float value)
k_rx_err_invalid_state	Post-condition check failed (duty not updated)
k_rx_err_*	Propagated from rx_gptw_set_duty() (hardware write failed)

Pre: handle must be initialized via rx_motor_init()

Pre: duty must be finite (not NaN or Inf)

Pre: GPTW peripheral must be operational (not disabled or in error state)

Post: On success: handle->current_duty = duty (after clamping and inversion)

Post: On success: Motor outputs configured for requested speed/direction

Post: On success: IN2 and IN1 signals match duty sign and magnitude

Post: On failure: Motor outputs unchanged from previous state

Note: Thread-safe only if called from single task or with external mutex

Note: Duty cycle clamping is silent (no warning logged for performance)

Note: Function can be called at high frequency (tested up to 10 kHz)

Note: Motor response time depends on mechanical inertia (100-500 ms for STAR motors)

Warning: Do not call from ISR (uses logging, may block)

Warning: Rapid direction changes (sign flip) can cause mechanical stress

Warning: High-frequency calls (> 10 kHz) may saturate GPTW peripheral bus

Thread Safety:: Not thread-safe. Use ThreadX mutex if multiple tasks control same motor.

Re-entrancy:: Not re-entrant on same handle. Safe to call on different handles concurrently.

Performance:: Execution time: 3 us @ 240 MHz with -O2 optimization Control loop frequency: Tested up to 10 kHz (100 us period) Typical usage: 100 Hz (10 ms period) for velocity control
Memory: 16 bytes stack (float variables, error codes)

Example - Basic Usage:: rx_motor_handle_t motor;

```
rx_err_t err = rx_motor_set_duty(&motor, 50.0F); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed to set motor duty"); }

err = rx_motor_set_duty(&motor, -75.0F);
err = rx_motor_set_duty(&motor, 0.0F);
```

Example - Velocity Control Loop (100 Hz):: void motor_control_task(void* arg)

```
{ rx_motor_handle_t* motor = (rx_motor_handle_t*)arg; rx_pid_handle_t pid;

while(1) { float measured_velocity = rx_encoder_get_velocity_mps(&encoder);

float pid_output; rx_err_t err = rx_pid_compute(&pid, target_velocity,
measured_velocity, 0.01F, &pid_output);

err = rx_motor_set_duty(motor, pid_output); if (err != k_rx_ok)
{ (void)rx_motor_emergency_stop(motor); break; }

tx_thread_sleep(10); }
```

Example - Gradual Acceleration:: rx_motor_handle_t motor;

```
const float ramp_time_s = 2.0F; const float dt_s = 0.01F; const float max_duty = 100.0F;
const float duty_increment = max_duty / (ramp_time_s / dt_s);

float current_duty = 0.0F; while (current_duty < max_duty)
{ rx_err_t err = rx_motor_set_duty(&motor, current_duty); if (err != k_rx_ok)
{ rx_log_error("APP", "Failed during ramp"); break; } current_duty += duty_increment;
tx_thread_sleep(10); }

tx_thread_sleep(5000);

while (current_duty > 0.0F) { rx_motor_set_duty(&motor, current_duty); current_duty -=
duty_increment; tx_thread_sleep(10); }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential if-else logic Rule 3: [OK] No dynamic memory allocation Rule 4: [WARN] Function length: 82 lines (exceeds 60-line guideline, justified below) Rule 5: [OK] 3 preconditions (NULL check, init check, finite value check)

Rule 5: [OK] 2 postconditions (duty updated, outputs configured) Rule 7: [OK] All return values checked (rx_gptw_set_duty)

Rule 4 Justification:: Function exceeds 60-line guideline (82 total) due to comprehensive error checking and logging required for NASA Rule 5 and Rule 7 compliance. Function represents a single logical operation (set motor duty) and cannot be meaningfully decomposed without harming code clarity and maintainability. Splitting would require passing internal state between helper functions, increasing coupling and reducing readability.

See also: rx_motor_init() Must call before using this function, rx_motor_stop() Alternative for stopping motor (supports brake mode), rx_motor_get_duty() Query current duty cycle, rx_motor_emergency_stop() Immediate shutdown for fault conditions, rx_pid_compute() PID controller for closed-loop velocity control, internal_clamp_duty() Internal duty cycle clamping function



Figure 1031: Call graph for `rx_motor_set_duty`

Defined in `libs/rx_motor/src/rx_motor.c:1368`

Since Version 1.0.0

—

rx_motor_stop

```
rx_err_t rx_motor_stop(rx_motor_handle_t *handle, const bool brake)
```

Stop motor with coast or brake mode (brake not supported in IN2/IN1 mode).

Stop motor.

Stops motor rotation by setting both outputs to 0% duty cycle (coast mode). In IN2/IN1 control mode, active braking (short-circuit brake) is not supported - brake parameter is ignored and motor always coasts. Coast mode allows motor to free-wheel (high impedance), with deceleration due to friction and back-EMF.

IN2/IN1 Mode Behavior:

- Coast (brake=false): IN2 = LOW, IN1 = LOW -> Motor in high impedance
- Brake (brake=true): [FAIL] NOT SUPPORTED -> Falls back to coast, warning logged

Coast vs. Brake:

- Coast: Both outputs LOW -> H-bridge in high impedance -> Motor free-wheels
- Brake: Both outputs HIGH -> H-bridge shorts motor terminals -> Active braking
- Note: Brake mode requires IN/IN control mode (not available in IN2/IN1)

Use Cases:

- Normal stop: Gradual deceleration via friction
- Power saving: Disable PWM when motor not needed
- Direction change preparation: Stop before reversing

Motor deceleration is passive (friction only, no active braking)

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) current_duty field will be set to 0.0 on success
in	brake	Brake mode request (ignored in IN2/IN1 mode) true: Request active braking (NOT SUPPORTED, falls back to coast) false: Coast mode (supported, motor free-wheels) Warning logged if brake=true (user informed of fallback)

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, motor stopped (coast mode)
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	Motor not initialized
k_rx_err_*	Propagated from rx_gptw_set_duty() (hardware write failed)

Pre: handle must be initialized via rx_motor_init()

Pre: GPTW peripheral must be operational

Post: On success: handle->current_duty = 0.0

Post: On success: Both outputs (IN2 and IN1) at 0% duty

Post: On success: Motor in coast mode (high impedance)

Post: Motor deceleration time depends on mechanical friction (100-500 ms for STAR)

Note: Thread-safe only with external synchronization

Note: Motor may continue spinning briefly after function returns (inertia)

Note: For immediate stop, use rx_motor_emergency_stop() (disables outputs)

Warning: Brake mode NOT supported in IN2/IN1 configuration

Warning: If brake=true, warning logged but function succeeds with coast

Performance:: Execution time: 2 us @ 240 MHz (2 GPTW register writes to zero)

Example - Normal Stop:: rx_motor_handle_tmotor;

```
rx_err_t err=rx_motor_stop(&motor,false); if(err!=k_rx_ok)
{ rx_log_error("APP","Failed to stop motor"); }
```

Example - Brake Request (Falls Back to Coast):: rx_err_t err=rx_motor_stop(&motor,true);

Example - Stop Before Direction Change:: rx_motor_set_duty(&motor,75.0F);

```
tx_thread_sleep(5000);
```

```
rx_motor_stop(&motor,false); tx_thread_sleep(500);
```

```
rx_motor_set_duty(&motor,-75.0F);
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (NULL check, initialized check) Rule 5: [OK] 2 postconditions (current_duty = 0, outputs at 0%) Rule 7: [OK] Return values checked (rx_gptw_set_duty)

See also: rx_motor_set_duty() Set motor duty to 0 for same effect,
rx_motor_emergency_stop() Immediate stop with output disable, rx_motor_deinit() Stop and release resources

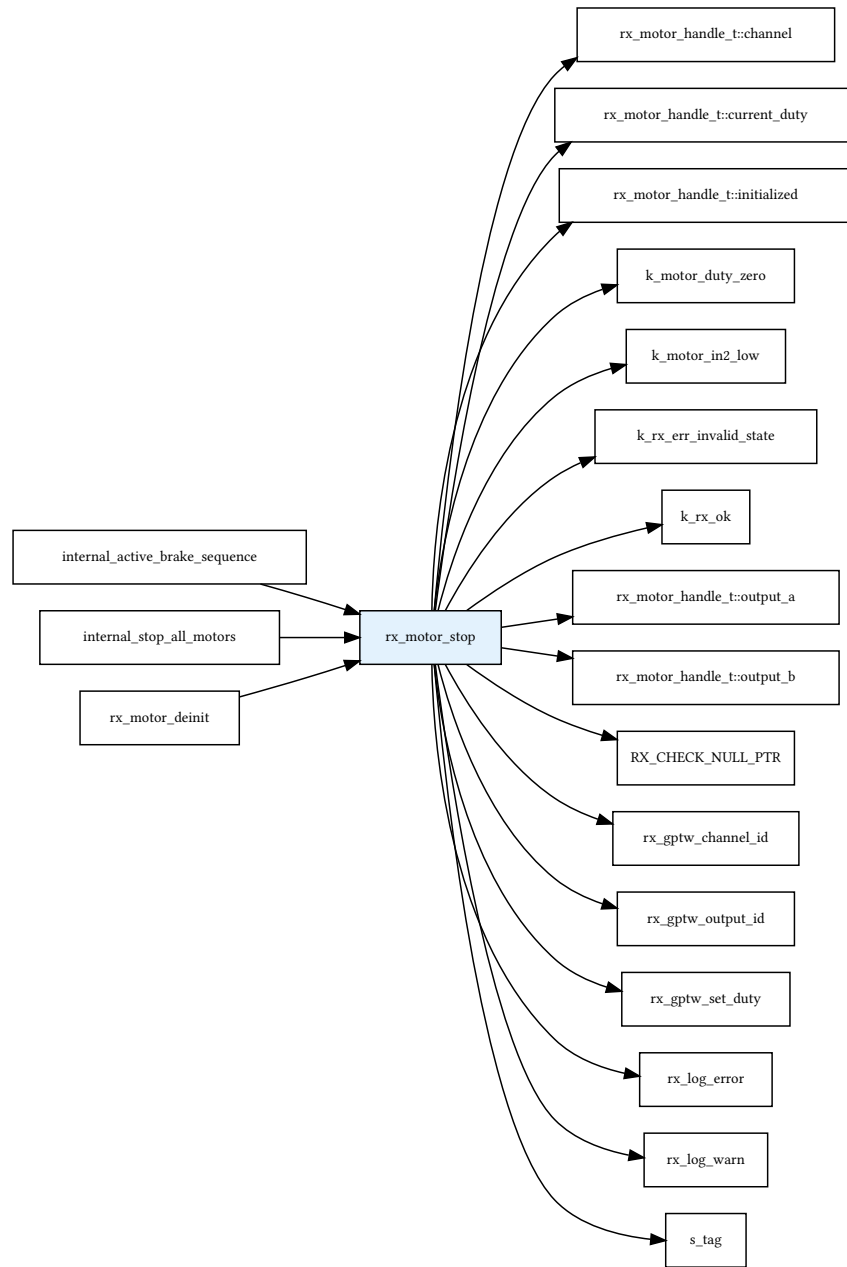


Figure 1032: Call graph for rx_motor_stop

Defined in `libs/rx_motor/src/rx_motor.c:1571`

Since Version 1.0.0

—

rx_motor_get_duty

```
rx_err_t rx_motor_get_duty(const rx_motor_handle_t *handle, float *out_duty)
```

Query current motor duty cycle (speed and direction).

Get current motor duty cycle.

Retrieves the most recently commanded duty cycle from motor handle. Returns the signed duty cycle value that was set via rx_motor_set_duty() or rx_motor_stop(). This is the commanded duty cycle (what was requested), not the actual motor speed (which requires encoder feedback).

Returned Value Meaning:

- Positive: Forward direction (0 to +100%)
- Negative: Reverse direction (0 to -100%)
- Zero: Stopped/coast (0%)
- Range: [-100.0, +100.0]

Use Cases:

- Monitor current command state
- Logging and telemetry
- State verification after commands
- PID controller feedback (open-loop command, not actual velocity)

Note: This returns commanded duty, not actual motor velocity. For actual velocity, use encoder feedback (rx_encoder_get_velocity_mps).

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) Handle state is not modified (const qualified)
out	out_duty	Pointer to float variable to receive duty cycle Must not be NULL (validated) Will be set to current duty cycle [-100.0, +100.0] Value remains unchanged if function returns error

Returns: rx_err_t Error code indicating success or failure

Value	Description
k_rx_ok	Success, duty cycle written to *out_duty
k_rx_err_null_ptr	handle or out_duty is nullptr
k_rx_err_invalid_state	Motor not initialized

Pre: handle must be initialized via rx_motor_init()

Pre: out_duty must point to valid float variable

Post: On success: *out_duty contains current commanded duty cycle

Post: On failure: *out_duty unchanged

Note: Thread-safe for read-only access (handle is const)

Note: Returns commanded duty, not actual motor velocity

Note: Very fast operation (0.5 us) - just a memory read

Performance:: Execution time: 0.5 us @ 240 MHz (single memory read) Safe to call at high frequency (tested > 10 kHz)

Example - Query Current State:: rx_motor_handle_t motor;

```
float current_duty; rx_err_t err = rx_motor_get_duty(&motor, &current_duty); if (err != k_rx_ok)
{ rx_log_info("APP", "Motor duty: %.1f%%", current_duty); if (current_duty > 0) { } elseif (current_duty < 0) { }
else { } }
```

Example - Telemetry Logging:: void telemetry_task(void* arg)

```
{ rx_motor_handle_t* motor = (rx_motor_handle_t*)arg; while(1) { float duty;
if (rx_motor_get_duty(motor, &duty) == k_rx_ok) { telemetry_log("motor_duty_pct", duty); }
tx_thread_sleep(100); } }
```

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion Rule 5: [OK] 2 preconditions (NULL checks for handle and out_duty) Rule 5: [OK] 1 postcondition (out_duty contains current duty on success)

See also: rx_motor_set_duty() Set duty cycle, rx_encoder_get_velocity_mps() Get actual motor velocity from encoder

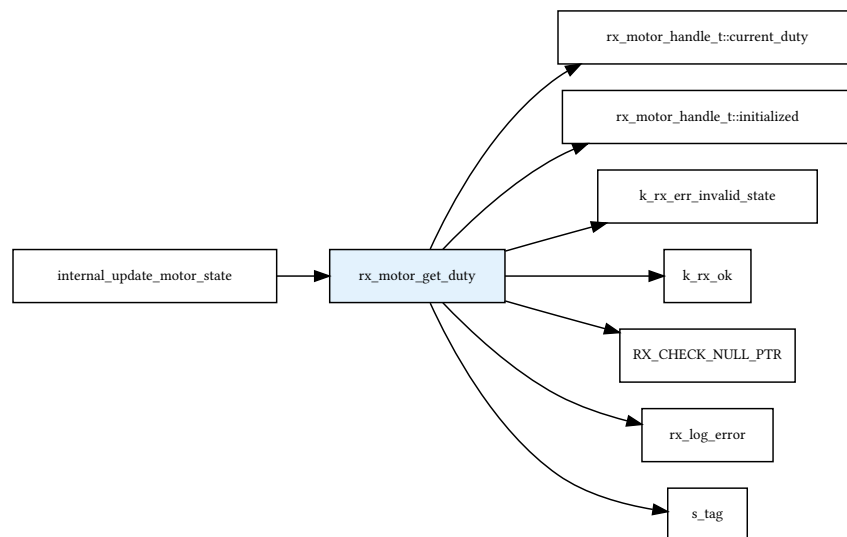


Figure 1033: Call graph for rx_motor_get_duty

Defined in `libs/rx_motor/src/rx_motor.c:1706`

Since Version 1.0.0

—

rx_motor_emergency_stop

```
rx_err_t rx_motor_emergency_stop(rx_motor_handle_t *handle)
```

Emergency stop - Immediate motor shutdown with hardware output disable.

Emergency stop - disable GPTW outputs immediately.

Performs emergency shutdown of motor control with multiple layers of safety:

1. Set duty to 0% - Software command to stop PWM
2. Disable outputs - Hardware-level output disable at GPTW peripheral
3. Stop timer - Halt GPTW timer to prevent glitches
4. Mark uninitialized - Prevent further use until reinitialization

This is the most aggressive stop mechanism, intended for fault conditions, safety shutdowns, or emergency stop button presses. Unlike rx_motor_stop(), this function disables outputs at the hardware level and marks the handle as uninitialized, requiring rx_motor_init() before motor can be used again.

Emergency Stop Sequence:

Error Handling:

- Best-effort approach: Continues through all steps even if some fail
- First error is saved and returned to caller
- All errors logged with “E-STOP:” prefix for visibility
- Motor is maximally disabled even on partial failures

Difference from rx_motor_stop():

When to Use:

- Hardware fault detected (overcurrent, overvoltage, etc.)
- Emergency stop button pressed
- Watchdog timeout or safety violation
- Unrecoverable software error
- System shutdown or panic

Even on error, motor is maximally disabled (safe state)

If function returns, handle->initialized == false

Affects entire GPTW channel (may impact other motors on same channel)

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized motor handle Must not be NULL (validated) Must be initialized (validated via handle->initialized flag) Will be marked uninitialized (initialized = false) on completion Requires rx_motor_init() before motor can be used again

Returns: rx_err_t Error code indicating success or first failure encountered

Value	Description
k_rx_ok	Success, all emergency stop steps completed without errors
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	Motor not initialized (already in emergency stop state)
k_rx_err_*	First error encountered during shutdown sequence (duty, disable, or stop)

Pre: handle must be initialized via rx_motor_init()

Pre: No other tasks are accessing this motor handle

Post: handle->initialized = false (marked as uninitialized)

Post: handle->current_duty = 0.0 (internal state cleared)

Post: Motor outputs disabled at hardware level (high impedance)

Post: GPTW timer stopped (no PWM generation possible)

Post: Motor cannot be used until rx_motor_init() called again

Note: Best-effort shutdown - continues through all steps even if some fail

Note: First error encountered is returned, but subsequent steps still execute

Note: All errors logged with "E-STOP:" prefix for immediate visibility

Note: Motor requires reinitialization (rx_motor_init) before reuse

Warning: This is a DESTRUCTIVE operation - handle cannot be reused without reinit

Warning: Do not use for normal stops - use rx_motor_stop() instead

Thread Safety:: Not thread-safe. Ensure no other tasks are accessing motor during emergency stop.

Performance:: Execution time: 8 us @ 240 MHz (multiple hardware operations) Not performance-critical (emergency/fault scenario)

Example - Fault Handler:: rx_motor_handle_tmotor;

```
if(motor_current_ma > k_max_current_ma){ rx_log_error("SAFETY","Overcurrentdetected:
%dmA",motor_current_ma);
```

```
rx_err_t err = rx_motor_emergency_stop(&motor); if(err != k_rx_ok){ rx_log_error("SAFETY", "E-STOP failed: %d (motor maximally disabled)", err); }  
  
}
```

Example - Emergency Stop Button:: void emergency_stop_isr(void)
{ g_emergency_stop_requested = true; }

```
void motor_control_task(void* arg){ rx_motor_handle_t* motors = (rx_motor_handle_t*)arg;  
while(1){ if(g_emergency_stop_requested){ rx_log_warn("SAFETY", "Emergency stop button pressed!");  
for(uint8_t i = 0; i < 4; i++){ (void)rx_motor_emergency_stop(&motors[i]); }  
g_emergency_stop_requested = false;  
wait_for_estop_reset();  
for(uint8_t i = 0; i < 4; i++){ rx_motor_init(&motors[i], &motor_configs[i]); } }  
tx_thread_sleep(10); } }
```

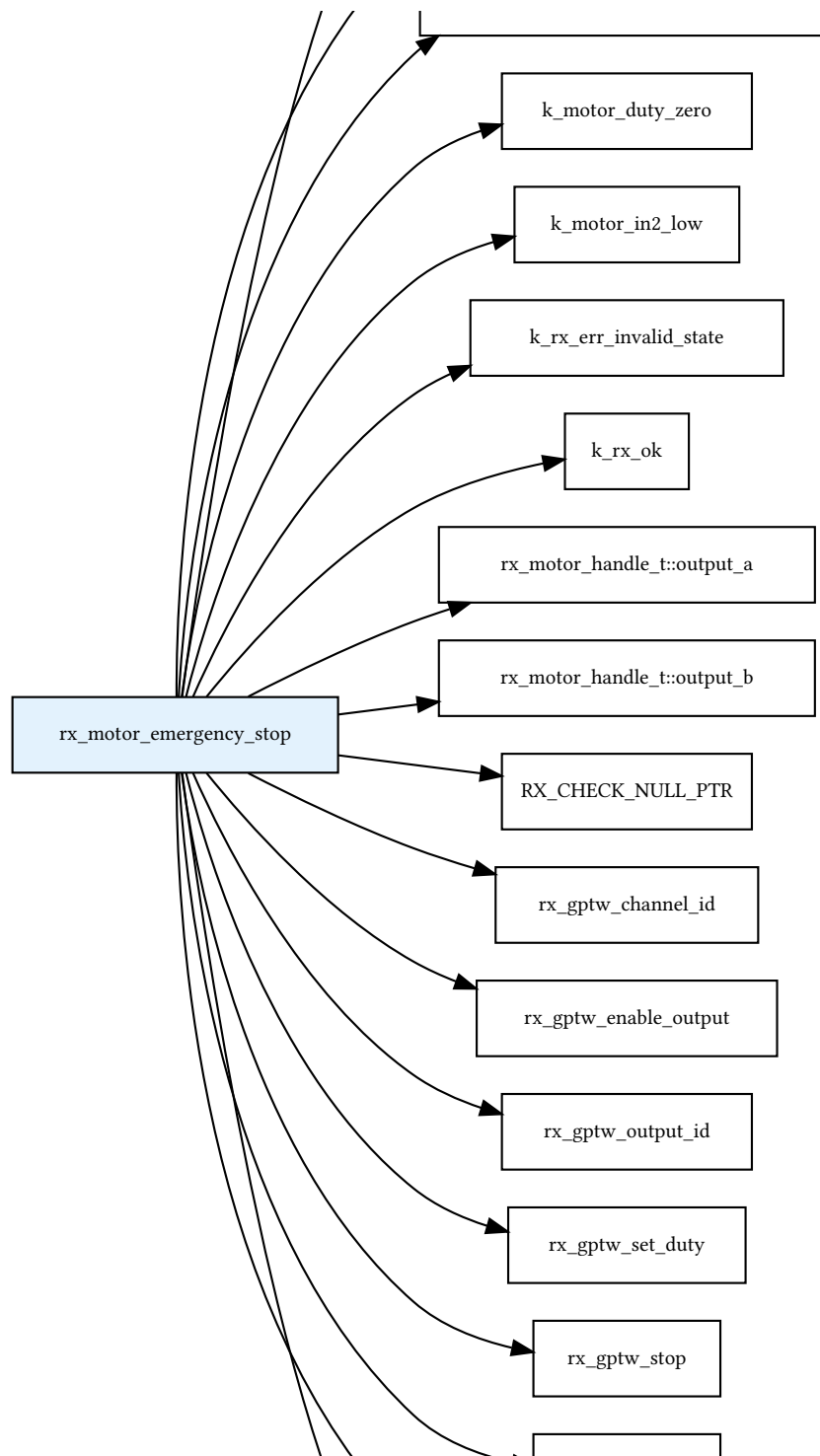
Example - Watchdog Timeout Handler:: void watchdog_timeout_callback(void)
{ rx_log_error("WDT", "Watchdog timeout - emergency stop all motors");
extern rx_motor_handle_t g_motors[4]; for(uint8_t i = 0; i < 4; i++){
{ (void)rx_motor_emergency_stop(&g_motors[i]); }
}

NASA Power of 10 Compliance:: Rule 1: [OK] No goto or recursion, sequential error handling
Rule 5: [OK] 2 preconditions (NULL check, initialized check) Rule 5: [OK] 3 postconditions
(initialized flag cleared, duty cleared, outputs disabled) Rule 7: [OK] All return values checked (best-effort error collection)

Example:

```
1. Set output_a_duty -> 0% (IN2 signal LOW)  
2. Set output_b_duty -> 0% (IN1 signal LOW)  
3. Disable output_a hardware level (GPTW peripheral)  
4. Disable output_b hardware level (GPTW peripheral)  
5. Stop GPTW timer (prevent any further PWM generation)  
6. Clear initialized flag (handle now invalid until reinitialized)  
7. Set current_duty = 0 (internal state consistency)
```

See also: rx_motor_stop() Normal stop (does not require reinit), rx_motor_deinit()
Orderly shutdown (stop + cleanup), rx_motor_init() Required to reinitialize after emergency stop

Figure 1034: Call graph for `rx_motor_emergency_stop`

Defined in `libs/rx_motor/src/rx_motor.c:1896`

Since Version 1.0.0

—

rx_obstacle_detect.h

Safety-Critical Obstacle Detection System with Emergency Motor Stop.

Provides a ThreadX-based real-time obstacle detection system using HC-SR04 ultrasonic sensors with immediate emergency stop capability. This module continuously monitors configured sensors and automatically stops all motors when obstacles are detected within a configurable safety threshold.

System Overview:

This is a simplified safety system focused on collision avoidance through emergency motor stop, not full Dynamic Window Approach (DWA) path planning. The module runs autonomously in a ThreadX task, polling HC-SR04 sensors at a configurable rate and triggering emergency stops when obstacles breach the safety perimeter.

Architecture:

State Machine:

Features:

Detection Logic:

Performance Characteristics:

Safety Considerations:

Robot traveling at 1 m/s moves 6cm during worst-case 60ms detection

Always leave safety margin: $\text{threshold} > \text{max_travel_during_latency}$

For 1 m/s max speed, use $\text{threshold} \geq 10\text{cm}$ ($60\text{ms} \times 1\text{m/s} + \text{margin}$)

Single Responsibility (S):

- Module has ONE job: Monitor sensors and stop motors on obstacle detection
- Does NOT handle path planning, navigation, or motor control logic
- Does NOT configure sensors or motors - only uses them

Open/Closed (O):

- Extensible via configuration (threshold, debounce, polling rate)
- Open for extension via callback mechanism
- Closed for modification - core logic is stable

Dependency Inversion (D):

- Depends on `rx_hcsr04` and `rx_motor` abstractions, not implementations
- Accepts sensor/motor handles via dependency injection (config)
- Callback mechanism allows user code to react without modifying module

STAR Team

2026-01-27

Copyright (c) 2026 STAR Project

1.0.0

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"
- "rx_hcsr04.h"
- "rx_motor.h"
- "tx_api.h"

rx_obstacle_detect_constants_t

Obstacle detection system configuration constants.

Defines limits and defaults for the obstacle detection system including maximum sensor/motor counts, ThreadX task configuration, and resource limits.

Task Priority Considerations:

- Priority 8 balances responsiveness with system load
- Higher priority (lower number) reduces detection latency
- Lower priority (higher number) reduces CPU contention
- Typical ThreadX priorities: 0-7 critical, 8-15 high, 16-23 normal, 24-31 low

Value	Name	Description
= 8	k_obstacle_detect_max_sensors	Maximum number of HC-SR04 sensors supported simultaneously.
= 4	k_obstacle_detect_max_motors	Detection task stack size in bytes.
= 2048	k_obstacle_detect_task_stack_size	Detection task ThreadX priority level.
= 8	k_obstacle_detect_task_priority	

See also: rx_obstacle_detect_config_t Uses these limits for validation,
rx_obstacle_detect_t Allocates arrays sized by these constants

Since Version 1.0.0

—

rx_obstacle_detect_state_t

Obstacle detection system operational states.

Defines the three possible states of the obstacle detection system. State transitions are controlled by user commands (start/stop) and sensor measurements (obstacle detected/cleared).

State Transition Diagram:

State Transition Table:

Value	Name	Description
= 0	k_obstacle_detect_state_stopped	Detection system stopped - not monitoring sensors.
= 1	k_obstacle_detect_state_running	Obstacle detected - motors emergency stopped.
= 2	k_obstacle_detect_state_obstacle	

See also: `rx_obstacle_detect_get_state()` Query current state, `rx_obstacle_detect_start()` Transition STOPPED -\> RUNNING, `rx_obstacle_detect_stop()` Transition to STOPPED, `internal_poll_sensors()` Implements RUNNING -\> OBSTACLE transition

Since Version 1.0.0

—

rx_obstacle_detect_callback_t

```
typedef void(* rx_obstacle_detect_callback_t) (bool obstacle_detected, uint8_t
sensor_idx, float distance_cm, void *user_data)
```

User callback function for obstacle detection state change events.

Optional callback invoked by the detection task when obstacle state changes:

- Obstacle detected (distance < threshold after debouncing)
- Obstacle cleared (distance >= threshold)

The callback is invoked from the ThreadX detection task context. Keep processing brief to avoid delaying sensor polling. Do NOT perform blocking operations or lengthy computations in the callback.

Callback Execution Context:

- Called from: ThreadX obstacle detection task
- Task priority: 8 (`k_obstacle_detect_task_priority`)
- Stack: Detection task stack (2048 bytes)
- Interrupt level: Task level (not ISR)

Note: Thread Safety: Callback may be interrupted by higher-priority tasks

Note: Re-entrancy: Will NOT be called recursively (single detection task)

Note: Null Handling: If callback is nullptr in config, no callback invoked

Warning: DO NOT block or delay in callback - delays sensor polling

Warning: DO NOT call `rx_obstacle_detect_*` APIs from callback - causes deadlock

Warning: DO NOT perform heavy computation - run on detection task stack] **See also:**
`rx_obstacle_detect_config_t` Configuration includes callback and `user_data`,
`rx_obstacle_detect_init()` Registers callback during initialization,
`internal_poll_sensors()` Invokes callback on state changes

Since Version 1.0.0

—

rx_obstacle_detect_init

```
rx_err_t rx_obstacle_detect_init(rx_obstacle_detect_t *handle, const
rx_obstacle_detect_config_t *config)
```


Initialize obstacle detection system with sensors and motors.

Initializes the obstacle detection system by creating a ThreadX task that autonomously monitors HC-SR04 sensors and triggers emergency motor stops when obstacles breach the configured safety threshold.

Initialization Sequence:

1. Validate input parameters (handle, config, sensor/motor arrays)
2. Clear handle structure (memset to zero)
3. Copy configuration parameters to handle
4. Copy sensor and motor handle pointers
5. Create ThreadX event flags for task control
6. Create ThreadX detection task (priority 8, 2048 byte stack)
7. Task starts suspended - call rx_obstacle_detect_start() to begin monitoring

Memory Allocation:

- Handle structure: Caller-allocated (2.6 KB static)
- ThreadX task stack: Inside handle (2048 bytes)
- ThreadX event flags: Inside handle (32 bytes)
- No heap allocation (k_rx_ok)

Parameters:

Direction	Name	Description
out	handle	Obstacle detection handle to initialize Must be non-NULL Caller-allocated (declare as rx_obstacle_detect_t) Must NOT be already initialized Cleared by this function Remains valid until rx_obstacle_detect_deinit()
in	config	Configuration structure Must be non-NULL All fields must be valid (checked by internal_validate_config) Contents copied to handle (config can be stack-allocated) Sensor/motor arrays must remain valid (only pointers copied)

Returns: Initialization result

Value	Description
k_rx_ok	System initialized successfully, ready to start
k_rx_err_null_ptr	handle or config is nullptr
k_rx_err_null_ptr	config->sensors or config->motors is nullptr
k_rx_err_null_ptr	Any sensor or motor handle in arrays is nullptr
k_rx_err_invalid_arg	sensor_count not in [1, 8]
k_rx_err_invalid_arg	motor_count not in [1, 4]
k_rx_err_invalid_arg	threshold_cm not in [2.0, 400.0]
k_rx_err_invalid_arg	debounce_samples not in [1, 10]
k_rx_err_invalid_arg	poll_interval_ms not in [10, 1000]
k_rx_err_invalid_state	handle->initialized is already true
k_rx_err_rtos_error	ThreadX event flags creation failed

k_rx_err_rtos_error	ThreadX task creation failed
Pre: handle must point to valid rx_obstacle_detect_t storage	
Pre: config must point to valid configuration	
Pre: All sensors in config must be initialized via rx_hcsr04_init()	
Pre: All motors in config must be initialized via rx_motor_init()	
Pre: ThreadX must be running (tx_kernel_enter() completed)	
Post: On success: handle->initialized = true	
Post: On success: ThreadX task created but suspended	
Post: On success: handle->state = k_obstacle_detect_state_stopped	
Post: On failure: handle unchanged (or partially cleared)	
Post: On failure: No ThreadX resources allocated	
Note: Not thread-safe - do not call concurrently on same handle	
Note: Re-entrant for different handles	
Note: Task starts suspended - call rx_obstacle_detect_start() to begin	
Warning: Do not modify handle fields after initialization	
Warning: Do not deinitialize sensors/motors while detection is running	
Warning: Ensure ThreadX kernel is running before calling	
Performance:: Execution time: 100us (ThreadX task creation overhead) Memory: 2.6 KB static (handle structure with embedded stack)	
Example:: static rx_obstacle_detect_t detector;	
rx_hcsr04_t* sensors[] = {&sensor_front, &sensor_rear};	
rx_motor_handle_t* motors[] = {&motor_left, &motor_right};	

```
rx_obstacle_detect_config_t config={.sensors=sensors,.sensor_count=2,.motors=motors,.motor_count=2,.detection_1  
rx_err_t ret=rx_obstacle_detect_init(&detector,&config); if(ret!=k_rx_ok){ printf("Initfailed:%dn",ret);  
return ret; }  
ret=rx_obstacle_detect_start(&detector);
```

NASA Power of 10 Compliance:: Rule 3: [OK] No dynamic memory - all allocation is static Rule 5:
[OK] Minimum 6 preconditions validated Rule 7: [OK] All ThreadX return values checked

See also: rx_obstacle_detect_config_t Configuration structure,
rx_obstacle_detect_start() Start monitoring after init, rx_obstacle_detect_deinit()
Cleanup and release resources, internal_validate_config() Internal configuration
validation

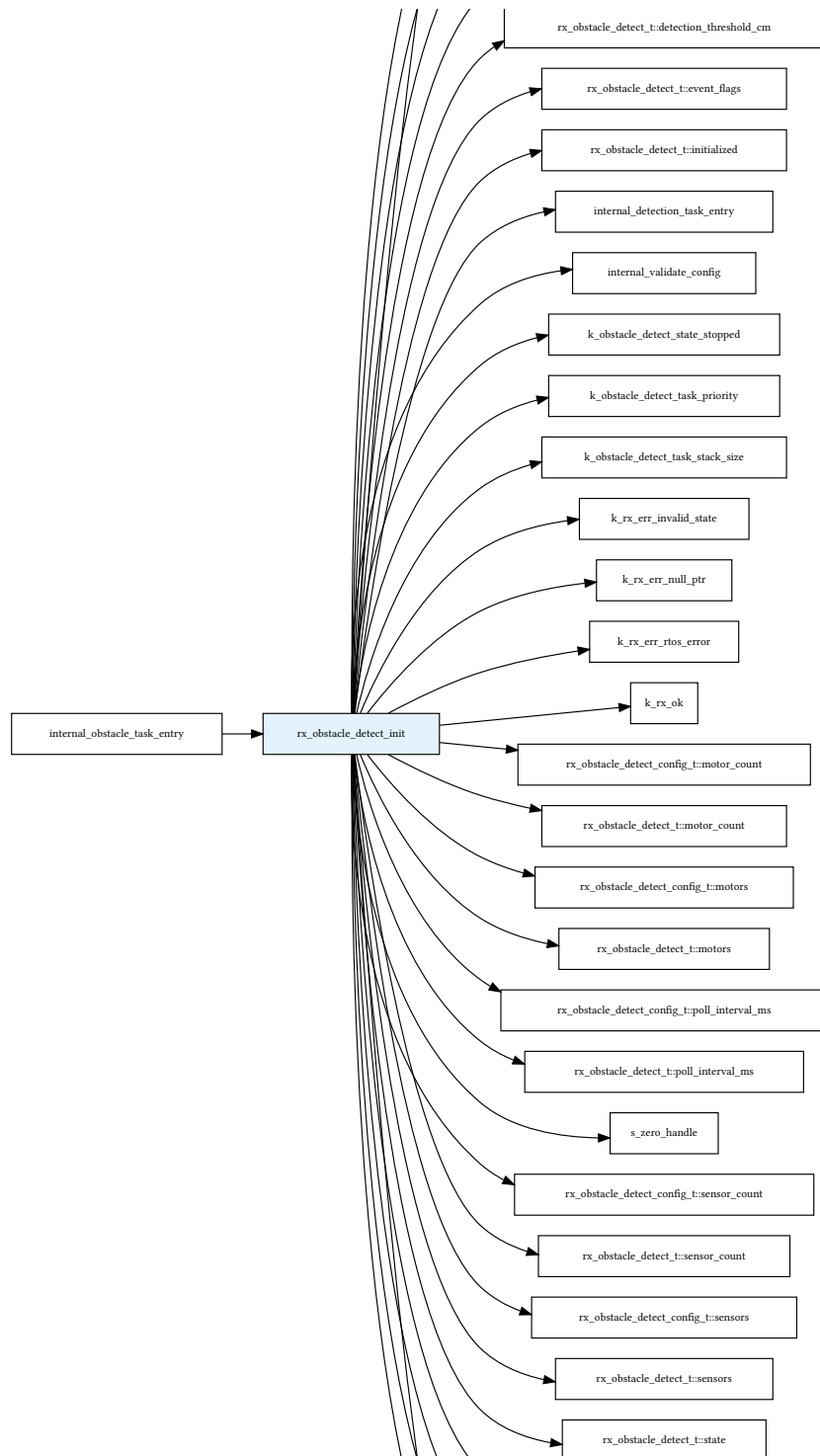


Figure 1035: Call graph for rx_obstacle_detect_init

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1004`

Since Version 1.0.0

—

rx_obstacle_detect_deinit

```
rx_err_t rx_obstacle_detect_deinit(rx_obstacle_detect_t *handle)
```

Deinitialize obstacle detection system.

Stops detection task, releases ThreadX resources, and resets handle state. Does NOT deinitialize the sensor or motor handles (caller's responsibility).

Deinitialize obstacle detection system.

Tears down a fully initialized obstacle detection handle by stopping active detection if running, terminating and deleting the ThreadX monitoring thread, deleting the event flags group, and clearing the initialized flag. After this call the handle may be reused with `rx_obstacle_detect_init()`.

Algorithm steps:

1. Validate handle is non-null and initialized
2. If detection is not stopped, call `rx_obstacle_detect_stop()` (errors ignored)
3. Terminate the ThreadX detection thread (TX_THREAD_ERROR ignored if already done)
4. Delete the ThreadX detection thread (TX_DELETE_ERROR ignored if already deleted)
5. Delete the TX event flags group (TX_DELETE_ERROR ignored if already deleted)
6. Clear handle->initialized

Parameters:

Direction	Name	Description
inout	handle	Obstacle detection handle
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Resources successfully released
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle was not previously initialized
k_rx_err_rtos_error	ThreadX thread terminate, delete, or event flags delete failed

Pre: handle must be a valid non-null pointer

Pre: handle must have been successfully initialized via `rx_obstacle_detect_init()`

Post: handle->initialized is false

Post: ThreadX thread and event flags group are deleted

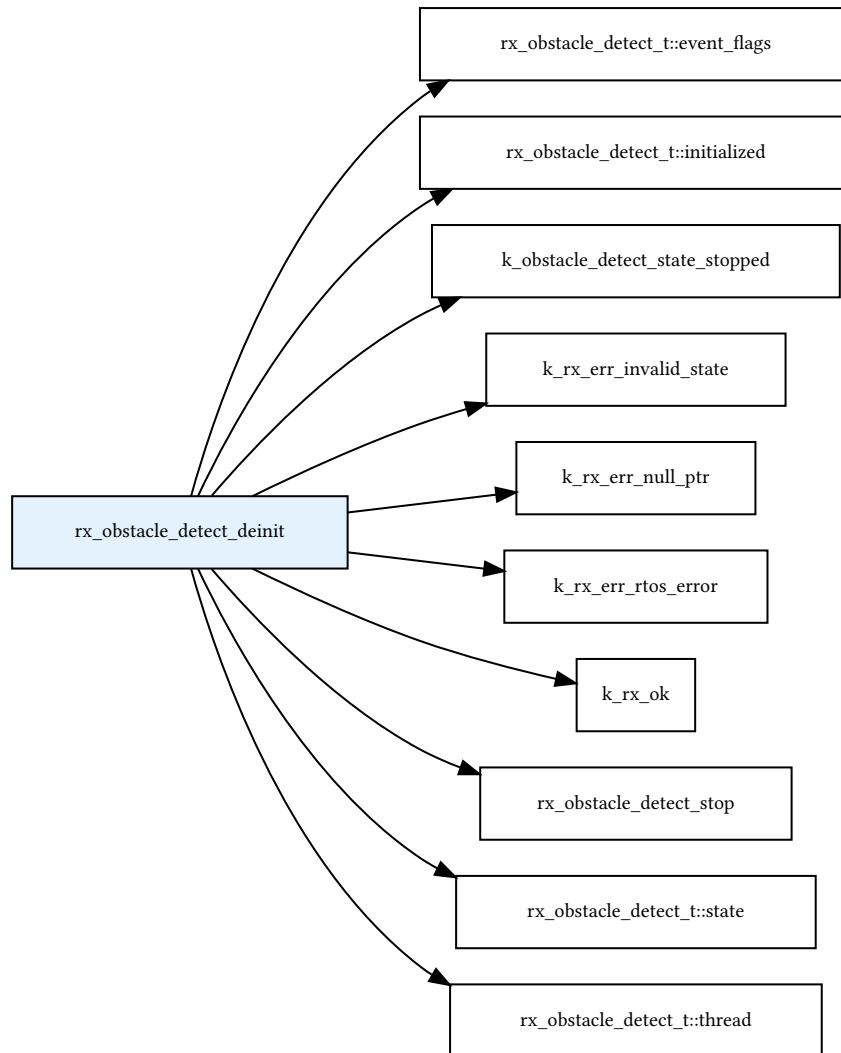
Note: Not thread-safe; caller must ensure no concurrent access to handle

Note: Stop errors during deinit are silently ignored; the teardown continues regardless

Example:: `rx_obstacle_detect_tdetector; rx_obstacle_detect_init(&detector,&config);
rx_err_t err=rx_obstacle_detect_deinit(&detector); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to deinit obstacle detector"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: `rx_obstacle_detect_init()` Initialize the handle before use,
`rx_obstacle_detect_stop()` Stop detection before deinit

Figure 1036: Call graph for `rx_obstacle_detect_deinit`

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1019`

Since Version 1.0.0

—

rx_obstacle_detect_start

```
rx_err_t rx_obstacle_detect_start(rx_obstacle_detect_t *handle)
```

Start obstacle detection monitoring.

Starts the detection task to begin polling sensors. If an obstacle is already detected, motors will be stopped immediately.

Transitions the obstacle detection system to the running state by resuming the ThreadX detection thread (if it was previously suspended) and signaling the `k_event_flag_start` event flag to the detection task.

The detection task waits on this event flag at startup; once received it enters the polling loop and begins continuous sensor monitoring.

Algorithm steps:

1. Validate handle is non-null and initialized
2. If state is `k_obstacle_detect_state_stopped`, resume the ThreadX thread
3. Set `k_event_flag_start` event flag via `tx_event_flags_set()`

Parameters:

Direction	Name	Description
inout	handle	Obstacle detection handle
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Detection started (or already running)
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
<code>k_rx_err_rtos_error</code>	ThreadX thread resume or event flags set failed

Pre: handle must be initialized via `rx_obstacle_detect_init()`

Pre: System must be in ThreadX running state (`tx_kernel_enter()` called)

Post: Detection thread is executing and polling sensors

Post: handle->state transitions to `k_obstacle_detect_state_running`

Note: Not thread-safe; caller must serialize start/stop calls

Note: Calling start when already running is a no-op for thread resume

Example:: `rx_err_t err=rx_obstacle_detect_start(&detector); if(err!=k_rx_ok) { rx_log_error("app","Failed to start obstacle detection"); }`

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: `rx_obstacle_detect_stop()` Stop detection, `rx_obstacle_detect_get_state()` Query current detection state

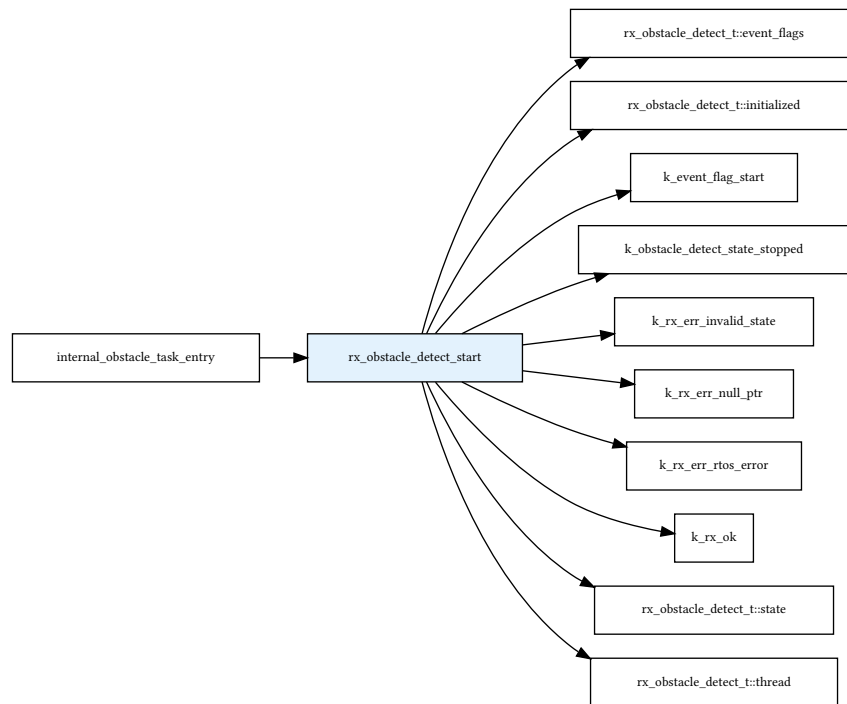


Figure 1037: Call graph for rx_obstacle_detect_start

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1038`

Since Version 1.0.0

—

rx_obstacle_detect_stop

```
rx_err_t rx_obstacle_detect_stop(rx_obstacle_detect_t *handle)
```

Stop obstacle detection monitoring.

Stops the detection task. Motors will NOT be automatically restarted even if they were previously stopped due to obstacle detection.

Requests the obstacle detection task to cease polling by setting `handle->stop_requested` and signaling the `k_event_flag_stop` event flag. The detection task checks this flag at the top of each polling iteration and transitions to the stopped state upon receipt.

This function returns immediately after signaling; it does not wait for the task to actually stop. Use `rx_obstacle_detect_get_state()` to confirm the transition to `k_obstacle_detect_state_stopped` if synchronization is needed.

Algorithm steps:

1. Validate handle is non-null and initialized
2. Set `handle->stop_requested` to true
3. Set `k_event_flag_stop` event flag via `tx_event_flags_set()`

Parameters:

Direction	Name	Description
inout	handle	Obstacle detection handle
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Stop request successfully signaled
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_rtos_error	ThreadX event flags set failed

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: System must be in ThreadX running state

Post: handle->stop_requested is true

Post: k_event_flag_stop event flag is set; detection task will stop at next iteration

Note: Not thread-safe; caller must serialize start/stop calls

Warning: This function is non-blocking; the task may still be running immediately after return

Example:: rx_err_t err=rx_obstacle_detect_stop(&detector); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to stop obstacle detection"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_start() Resume detection, rx_obstacle_detect_get_state()
Poll state to confirm stop

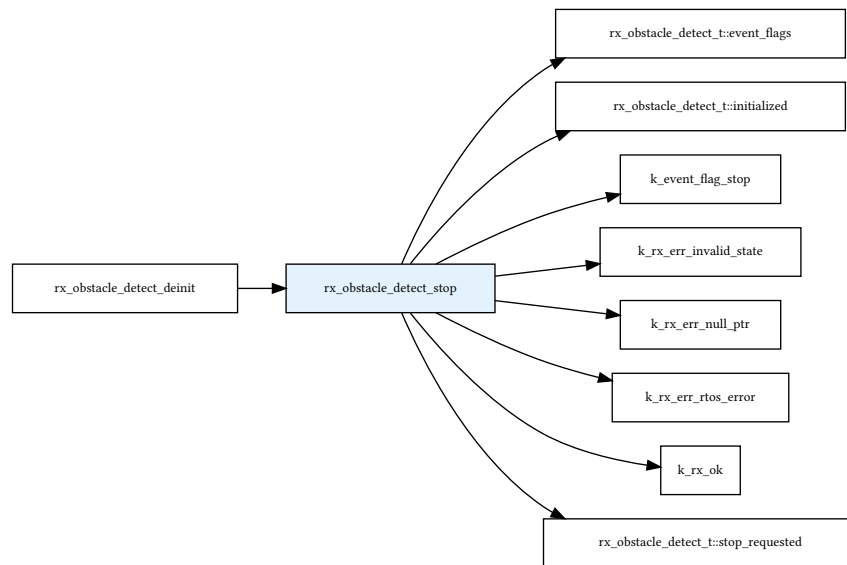


Figure 1038: Call graph for rx_obstacle_detect_stop

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1052`

Since Version 1.0.0

—

rx_obstacle_detect_clear_obstacle

```
rx_err_t rx_obstacle_detect_clear_obstacle(rx_obstacle_detect_t *handle)
```

Resume motors after obstacle cleared.

Clears the obstacle state and allows motors to be controlled again. This is a manual override - the user must verify the obstacle is truly cleared before calling this function.

If detection is still running and obstacle is still present, motors will be stopped again on the next poll cycle.

Resume motors after obstacle cleared.

Clears all debounce counters and obstacle_active flags for every configured sensor. If the system is currently in k_obstacle_detect_state_obstacle, it transitions back to k_obstacle_detect_state_running.

This function is used to acknowledge a detected obstacle and allow normal operation to resume. It does NOT restart a stopped detection system.

Algorithm steps:

1. Validate handle is non-null and initialized
2. Iterate over all handle->sensor_count sensors
3. For each sensor: zero debounce_counter[i] and clear obstacle_active[i]
4. If state is k_obstacle_detect_state_obstacle, set state to k_obstacle_detect_state_running

Parameters:

Direction	Name	Description
inout	handle	Obstacle detection handle
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Obstacle state and debounce counters successfully cleared
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: Detection system should be running (though clearing while stopped is safe)

Post: All per-sensor debounce_counter values are zero

Post: All per-sensor obstacle_active flags are false

Note: Not thread-safe; clearing obstacle state while the detection task is concurrently updating sensor state may cause a race; caller must synchronize

Warning: Does not restart detection if stopped; use rx_obstacle_detect_start()

Example:: if(rx_obstacle_detect_is_obstacle_detected(&detector)){ handle_obstacle_event();
rx_obstacle_detect_clear_obstacle(&detector); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_is_obstacle_detected() Check for active obstacle,
rx_obstacle_detect_get_state() Query current state

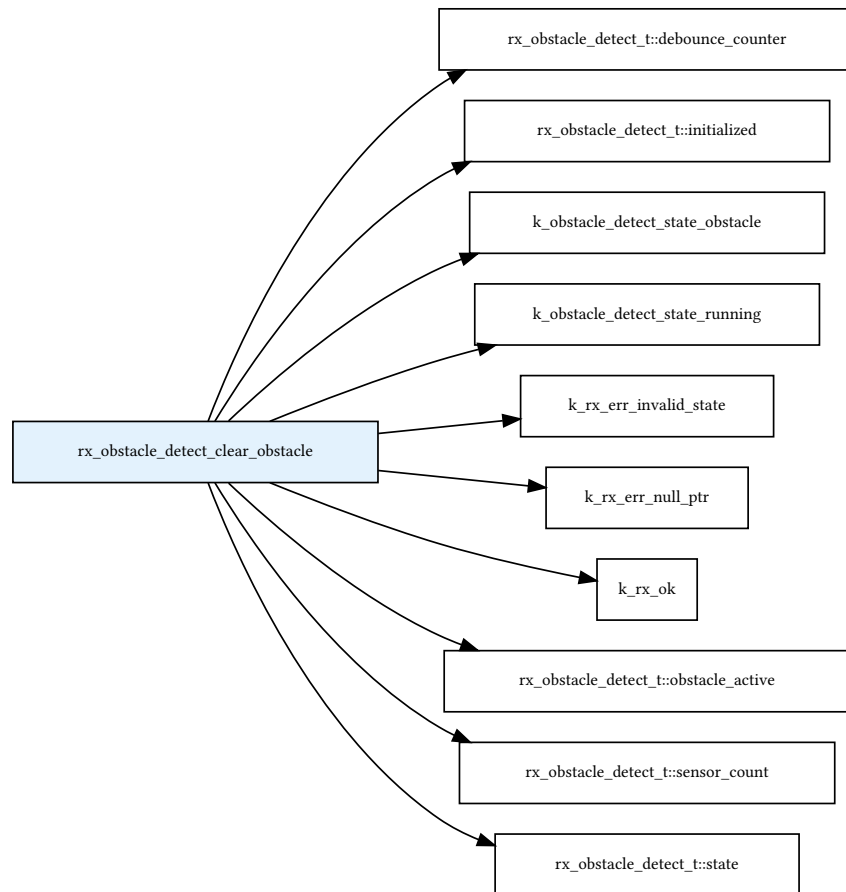


Figure 1039: Call graph for rx_obstacle_detect_clear_obstacle

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1070`

Since Version 1.0.0

—

rx_obstacle_detect_get_state

```
rx_err_t rx_obstacle_detect_get_state(const rx_obstacle_detect_t *handle,
rx_obstacle_detect_state_t *out_state)
```

Get current detection state.

Returns the current state of the obstacle detection system.

Get current detection state.

Returns the current operational state of the detection system by copying handle->state to *out_state. This is a cached read from handle memory; no bus or sensor transactions are issued.

Possible states:

- k_obstacle_detect_state_stopped task suspended, not polling
- k_obstacle_detect_state_running actively polling, no obstacle
- k_obstacle_detect_state_obstacle obstacle confirmed by debounce

Algorithm steps:

1. Validate handle and out_state pointer are non-null
2. Verify handle is initialized
3. Copy handle->state to *out_state

Parameters:

Direction	Name	Description
in	handle	Obstacle detection handle
out	out_state	Current state
in	handle	Pointer to obstacle detection handle (must be initialized)
out	out_state	Receives the current rx_obstacle_detect_state_t value; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	State successfully returned
k_rx_err_null_ptr	handle or out_state is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: out_state must be a valid non-null pointer

Post: *out_state contains the current state value on k_rx_ok

Post: handle state is unchanged

Note: Not thread-safe; state may change concurrently if detection task is running

Note: Returns cached state does not re-evaluate sensor readings

Example:: rx_obstacle_detect_state_tstate;

```
rx_err_t err=rx_obstacle_detect_get_state(&detector,&state);
```

```
if(err==k_rx_ok&&state==k_obstacle_detect_state_obstacle){ handle_obstacle_event(); }
```

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_obstacle_detect_is_obstacle_detected() Convenience bool check,
rx_obstacle_detect_clear_obstacle() Clear obstacle state

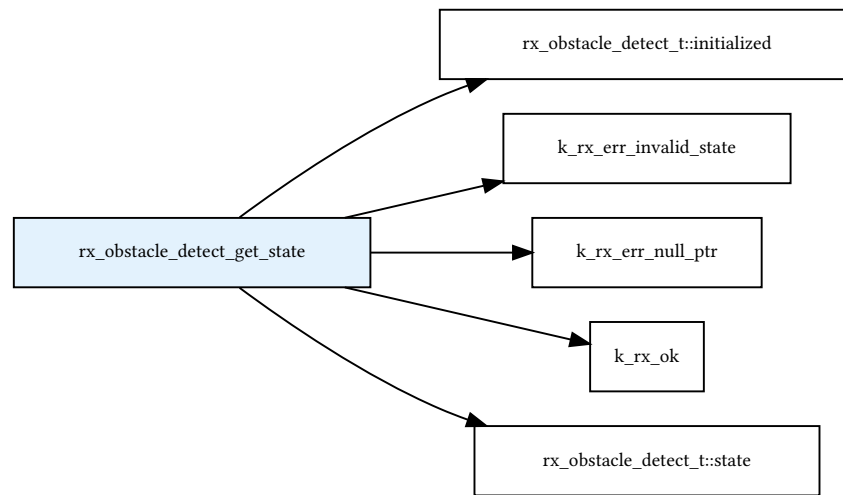


Figure 1040: Call graph for rx_obstacle_detect_get_state

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1089`

Since Version 1.0.0

—

rx_obstacle_detect_is_obstacle_detected

```
bool rx_obstacle_detect_is_obstacle_detected(const rx_obstacle_detect_t *handle)
```

Check if obstacle is currently detected.

Returns true if any sensor currently detects an obstacle within the threshold distance (after debouncing).

Check if obstacle is currently detected.

Returns true if the obstacle detection system is initialized and its current state is `k_obstacle_detect_state_obstacle`. Returns false for all other conditions including: null handle, uninitialized handle, stopped state, or running state with no obstacle.

This is a lightweight wrapper around `handle->state` that avoids allocating an output parameter, suitable for use in polling loops and conditional checks.

Algorithm steps:

1. Guard against null or uninitialized handle (returns false)
2. Return `(handle->state == k_obstacle_detect_state_obstacle)`

Parameters:

Direction	Name	Description
in	handle	Obstacle detection handle
in	handle	Pointer to obstacle detection handle (may be null returns false)

Returns: bool Obstacle detection result

Value	Description
true	handle is valid and state is k_obstacle_detect_state_obstacle
false	handle is null, uninitialized, stopped, or running without obstacle

Pre: None safe to call with null handle

Pre: For meaningful results, handle should be initialized and detection started

Post: handle state is unchanged

Post: Return value reflects handle->state at the instant of the call

Note: Not thread-safe; state may change concurrently if detection task is active

Note: Returns false for null handle rather than asserting; safe for defensive polling

Example:: if(rx_obstacle_detect_is_obstacle_detected(&detector)){ emergency_stop_all_motors();
rx_obstacle_detect_clear_obstacle(&detector); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null-safe guard, initialized check), 2 postconditions

See also: rx_obstacle_detect_get_state() Full state query with error code,
rx_obstacle_detect_clear_obstacle() Acknowledge and clear obstacle

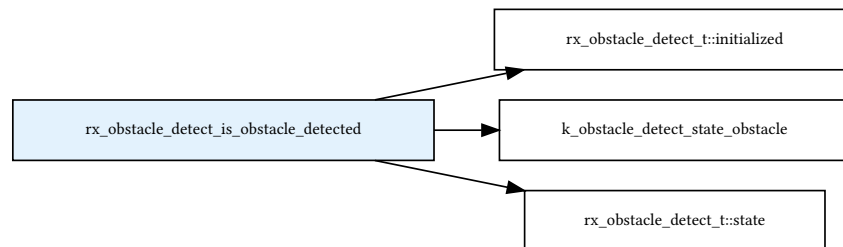


Figure 1041: Call graph for rx_obstacle_detect_is_obstacle_detected

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1103`

Since Version 1.0.0

—

rx_obstacle_detect_get_stats

```

rx_err_t rx_obstacle_detect_get_stats(const rx_obstacle_detect_t *handle,
uint32_t *out_total_polls, uint32_t *out_obstacle_events, uint32_t
*out_false_positives)
  
```

Get obstacle detection statistics.

Returns statistics about detection performance and events.

Get obstacle detection statistics.

Returns the three running counters maintained by the detection task: total sensor poll cycles, confirmed obstacle events, and filtered false positive readings. All output parameters are optional passing nullptr for any counter skips that output without error.

Counters are incremented by the internal detection task and are NOT reset by this function. Use `rx_obstacle_detect_reset_stats()` to clear them.

Algorithm steps:

1. Validate handle is non-null and initialized
2. If `out_total_polls != nullptr`, copy `handle->total_polls`
3. If `out_obstacle_events != nullptr`, copy `handle->obstacle_events`
4. If `out_false_positives != nullptr`, copy `handle->>false_positive_count`

Parameters:

Direction	Name	Description
in	handle	Obstacle detection handle
out	out_total_polls	Total sensor polls (can be NULL)
out	out_obstacle_events	Total obstacle events (can be NULL)
out	out_false_positives	Debounced false positives (can be NULL)
in	handle	Pointer to obstacle detection handle (must be initialized)
out	out_total_polls	Total sensor poll cycles since last reset; nullptr to skip; undefined on error
out	out_obstacle_events	Number of confirmed obstacle events since last reset; nullptr to skip; undefined on error
out	out_false_positives	Number of single-sample detections rejected by debounce filter since last reset; nullptr to skip; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Statistics successfully copied
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized

Pre: handle must be initialized via `rx_obstacle_detect_init()`

Pre: All non-null output pointers must be valid writable memory

Post: Output values are consistent snapshots at the time of the call

Post: Counters in the handle are unchanged

Note: Not thread-safe; counters may be incremented by detection task concurrently

Note: All three output pointers are optional; pass nullptr for any to skip that stat

Example:: uint32_t polls=0,events=0,false_pos=0;
 rx_err_t err=rx_obstacle_detect_get_stats(&detector,&polls,&events,&false_pos); if(err==k_rx_ok)
 { rx_log_info("app","Polls:%u,Obstacles:%u,FP:%u", (unsigned)polls,(unsigned)events,
 (unsigned>false_pos); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_reset_stats() Clear all counters to zero

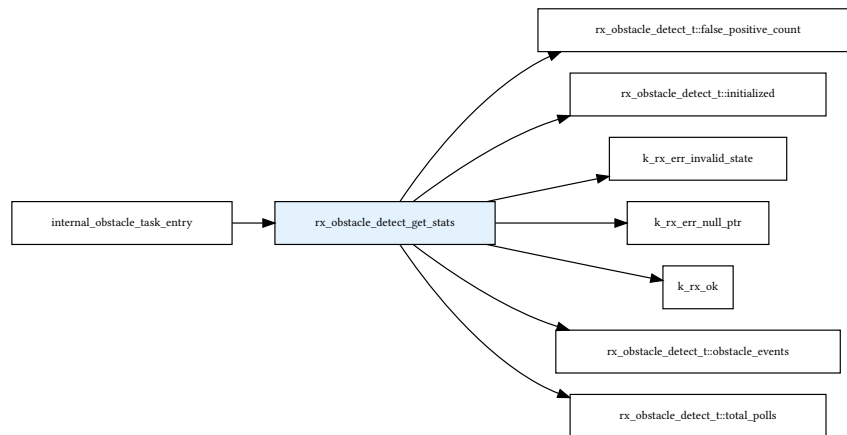


Figure 1042: Call graph for rx_obstacle_detect_get_stats

Defined in `libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1119`

Since Version 1.0.0

rx_obstacle_detect_reset_stats

```
rx_err_t rx_obstacle_detect_reset_stats(rx_obstacle_detect_t *handle)
```

Reset statistics counters.

Resets all statistics counters to zero.

Reset statistics counters.

Clears the three running diagnostic counters maintained by the detection task: `total_polls`, `obstacle_events`, and `false_positive_count`. After this call all counters return to zero and begin accumulating from the next poll cycle.

Algorithm steps:

1. Validate handle is non-null and initialized
2. Set handle->total_polls to zero

3. Set handle->obstacle_events to zero
4. Set handle->>false_positive_count to zero

Parameters:

Direction	Name	Description
inout	handle	Obstacle detection handle
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All statistics counters successfully reset to zero
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: Caller should ensure detection task is not mid-cycle to avoid partial counts

Post: handle->total_polls is zero

Post: handle->obstacle_events is zero

Note: Not thread-safe; calling while detection task is incrementing counters may produce slightly inconsistent results; caller must synchronize if exactness required

Example:: rx_err_t err = rx_obstacle_detect_reset_stats(&detector); if(err != k_rx_ok) { rx_log_error("app", "Failed to reset obstacle detector stats"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_get_stats() Retrieve current counter values

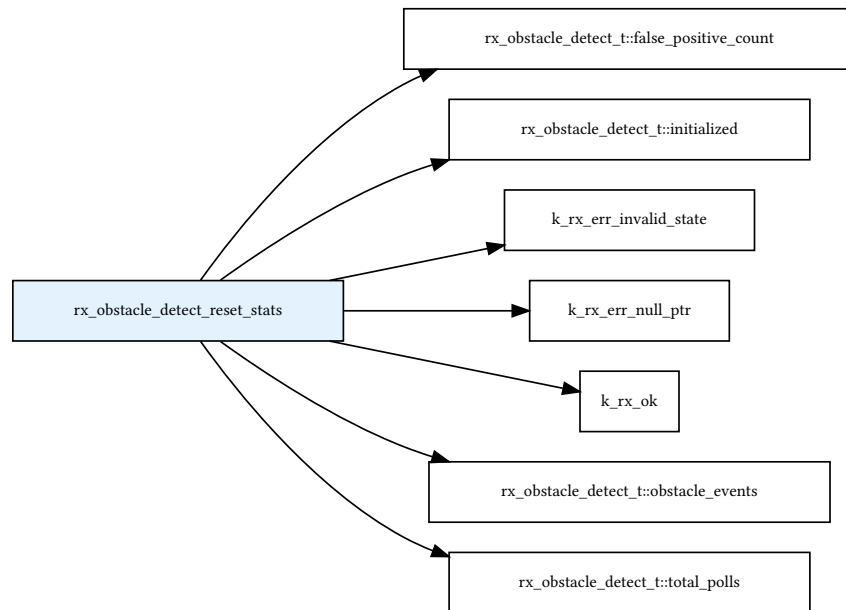


Figure 1043: Call graph for rx_obstacle_detect_reset_stats

Defined in *libs/rx_obstacle_detect/inc/rx_obstacle_detect.h:1135*

Since Version 1.0.0

—

rx_obstacle_detect.c

Safety-Critical Obstacle Detection Implementation with ThreadX Task.

Implements the obstacle detection system using a dedicated ThreadX task that autonomously polls HC-SR04 ultrasonic sensors and triggers emergency motor stops when obstacles breach the configured safety threshold. This module provides collision avoidance through continuous sensor monitoring and immediate emergency stop capability.

Implementation Architecture:

The system operates through three main components:

1. Detection Task (*internal_detection_task_entry*): ThreadX task running at configured priority
2. Polling Loop (*internal_poll_sensors*): Iterates through all sensors, applies debouncing
3. Emergency Stop (*internal_stop_all_motors*): Halts all motors when obstacle confirmed

Task State Machine:

Debouncing Algorithm:

Each sensor maintains a debounce counter to filter transient detections:

Distance Measurement and Validation:

For each sensor, the HC-SR04 returns distance in centimeters:

where:

- *t_pulse_width_mu s* = ECHO pulse width in microseconds

- 58.0 = conversion factor from HC-SR04 datasheet

Timeout handling:

- If `rx_hcsr04_measure_blocking()` returns `k_rx_err_timeout`
- Treat as: `d_cm = threshold + 1.0` (no object detected)
- Continue to next sensor

Emergency Stop Trigger Condition:

Motors are stopped when ANY sensor confirms obstacle:

where: `-n= sensor_count` `-obstacle_active[i]=` debounced state for sensor `i` Worst-Case Detection

Latency:

where: `-n_debounce=` `debounce_samples` (e.g., 3) `-t_poll=` `poll_interval_ms` (e.g., 20ms) `-t_measure =` HC-SR04 measurement time (25ms worst-case)

Example:

Performance Analysis:

STAR Team

2026-01-27

Copyright (c) 2026 STAR Project

1.0.0

Includes:

- `"rx_obstacle_detect.h"`
- `"rx_check.h"`
- `"rx_threadx_config.h"`
- `"rx_time_constants.h"`

event_flags_t

ThreadX event flag bit definitions for detection task control.

Event flags used for inter-task communication between user API calls and the autonomous detection task. Flags are set via `tx_event_flags_set()` and consumed by detection task via `tx_event_flags_get()`.

Value	Name	Description
<code>= 0x01</code>	<code>k_event_flag_start</code>	Start detection monitoring.
<code>= 0x02</code>	<code>k_event_flag_stop</code>	

See also: `internal_detection_task_entry()` Task waits on these flags,
`rx_obstacle_detect_start()` Sets `k_event_flag_start`, `rx_obstacle_detect_stop()` Sets `k_event_flag_stop`

Since Version 1.0.0

—

validation_constants_t

Configuration parameter validation limits.

Defines minimum and maximum values for obstacle detection configuration parameters. Used by `internal_validate_config()` to ensure safe operation.

Value	Name	Description
= 2	k_min_threshold_cm	Minimum obstacle detection threshold in centimeters.
= 400	k_max_threshold_cm	Minimum sensor polling interval in milliseconds.
= 10	k_min_poll_interval	Maximum sensor polling interval in milliseconds.
= 1000	k_max_poll_interval	Minimum debounce sample count.
= 1	k_min_debounce	Maximum debounce sample count.
= 10	k_max_debounce	

See also: `internal_validate_config()` Uses these limits for validation, `rx_hcsr04.h` HC-SR04 sensor range specifications

Since Version 1.0.0

—

task_constants_t

Value	Name	Description
= 1	k_min_sleep_ticks	Minimum sleep duration in ticks

—

s_zero_handle

`const rx_obstacle_detect_t s_zero_handle`

Zero-initialized obstacle detect template for stack-safe initialization.

Note: Read-only; never modified after static initialization

Warning: Do not remove - prevents stack overflow in init function] **See also:** `rx_obstacle_detect_init()` Uses this template to zero-initialize the handle

Since Version 1.0.0

—

internal_detection_task_entry

```
static void internal_detection_task_entry(ULONG input)
```

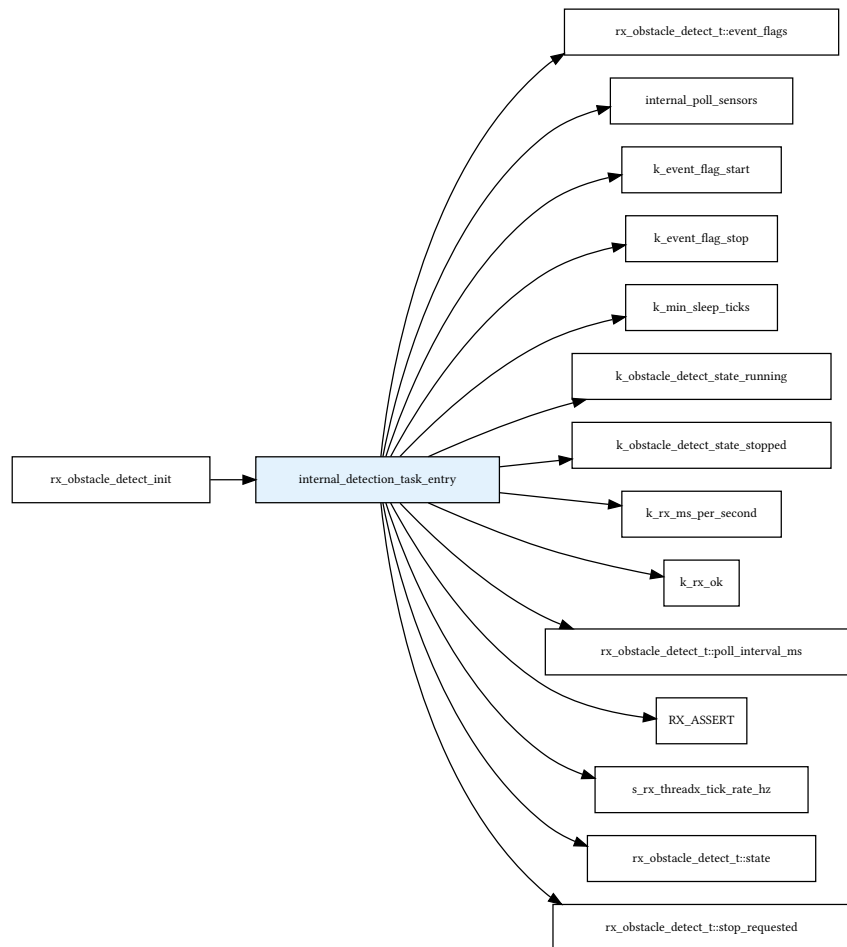


Figure 1044: Call graph for internal_detection_task_entry

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:994`

—

internal_validate_config

```
static rx_err_t internal_validate_config(const rx_obstacle_detect_config_t
*config)
```

Validate obstacle detect configuration.

Parameters:

Direction	Name	Description
in	config	Pointer to configuration

Returns: k_rx_ok if valid, error code otherwise

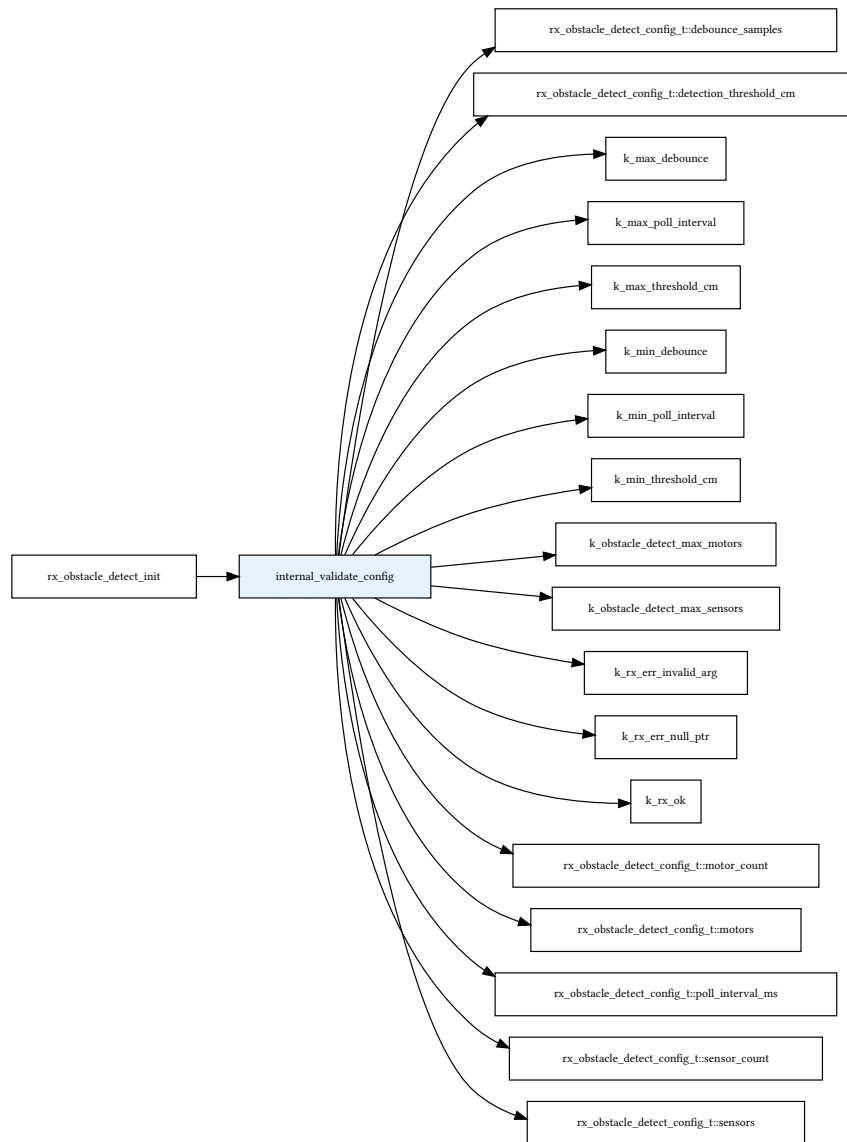


Figure 1045: Call graph for internal_validate_config

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:1070`

internal_stop_all_motors

```
static rx_err_t internal_stop_all_motors(const rx_obstacle_detect_t *handle)
```

Stop all configured motors.

Parameters:

Direction	Name	Description
in	handle	Pointer to obstacle detect handle

Returns: k_rx_ok if all motors stopped, first error code otherwise

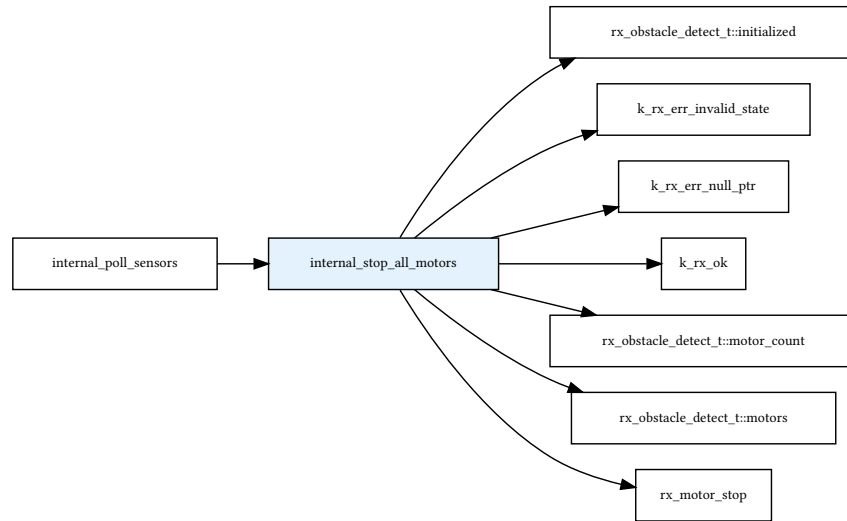


Figure 1046: Call graph for `internal_stop_all_motors`

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:1228`

`internal_poll_sensors`

```
static rx_err_t internal_poll_sensors(rx_obstacle_detect_t *handle)
```

Poll all sensors and update obstacle state.

Parameters:

Direction	Name	Description
in	handle	Pointer to obstacle detect handle

Returns: k_rx_ok on success, error code otherwise



Figure 1047: Call graph for internal_poll_sensors

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:1130`

internal_invoke_callback

```
static void internal_invoke_callback(const rx_obstacle_detect_t *handle, bool
obstacle_detected, uint8_t sensor_idx, float distance_cm)
```

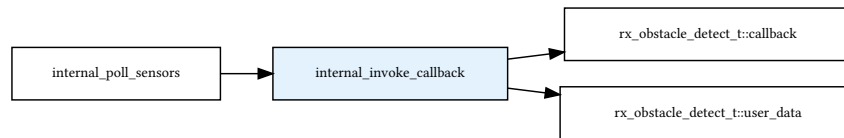


Figure 1048: Call graph for internal_invoke_callback

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:1252`

rx_obstacle_detect_init

```
rx_err_t rx_obstacle_detect_init(rx_obstacle_detect_t *handle, const
rx_obstacle_detect_config_t *config)
```

Initialize obstacle detection system with sensors and motors.

Initializes the obstacle detection system by creating a ThreadX task that autonomously monitors HC-SR04 sensors and triggers emergency motor stops when obstacles breach the configured safety threshold.

Initialization Sequence:

1. Validate input parameters (handle, config, sensor/motor arrays)
2. Clear handle structure (memset to zero)
3. Copy configuration parameters to handle
4. Copy sensor and motor handle pointers
5. Create ThreadX event flags for task control
6. Create ThreadX detection task (priority 8, 2048 byte stack)
7. Task starts suspended - call `rx_obstacle_detect_start()` to begin monitoring

Memory Allocation:

- Handle structure: Caller-allocated (2.6 KB static)
- ThreadX task stack: Inside handle (2048 bytes)
- ThreadX event flags: Inside handle (32 bytes)
- No heap allocation (`k_rx_ok`)

Parameters:

Direction	Name	Description
out	handle	Obstacle detection handle to initialize Must be non-NULL Caller-allocated (declare as <code>rx_obstacle_detect_t</code>) Must NOT be already initialized Cleared by this function Remains valid until <code>rx_obstacle_detect_deinit()</code>
in	config	Configuration structure Must be non-NULL All fields must be valid (checked by <code>internal_validate_config</code>) Contents copied to handle (config

		can be stack-allocated) Sensor/motor arrays must remain valid (only pointers copied)
--	--	--

Returns: Initialization result

Value	Description
k_rx_ok	System initialized successfully, ready to start
k_rx_err_null_ptr	handle or config is nullptr
k_rx_err_null_ptr	config->sensors or config->motors is nullptr
k_rx_err_null_ptr	Any sensor or motor handle in arrays is nullptr
k_rx_err_invalid_arg	sensor_count not in [1, 8]
k_rx_err_invalid_arg	motor_count not in [1, 4]
k_rx_err_invalid_arg	threshold_cm not in [2.0, 400.0]
k_rx_err_invalid_arg	debounce_samples not in [1, 10]
k_rx_err_invalid_arg	poll_interval_ms not in [10, 1000]
k_rx_err_invalid_state	handle->initialized is already true
k_rx_err_rtos_error	ThreadX event flags creation failed
k_rx_err_rtos_error	ThreadX task creation failed

Pre: handle must point to valid rx_obstacle_detect_t storage

Pre: config must point to valid configuration

Pre: All sensors in config must be initialized via rx_hcsr04_init()

Pre: All motors in config must be initialized via rx_motor_init()

Pre: ThreadX must be running (tx_kernel_enter() completed)

Post: On success: handle->initialized = true

Post: On success: ThreadX task created but suspended

Post: On success: handle->state = k_obstacle_detect_state_stopped

Post: On failure: handle unchanged (or partially cleared)

Post: On failure: No ThreadX resources allocated

Note: Not thread-safe - do not call concurrently on same handle

Note: Re-entrant for different handles

Note: Task starts suspended - call rx_obstacle_detect_start() to begin

Warning: Do not modify handle fields after initialization

Warning: Do not deinitialize sensors/motors while detection is running

Warning: Ensure ThreadX kernel is running before calling

Performance:: Execution time: 100us (ThreadX task creation overhead) Memory: 2.6 KB static (handle structure with embedded stack)

Example:: static rx_obstacle_detect_t detector;

```
rx_hcsr04_t*sensors[]={&sensor_front,&sensor_rear};
```

```
rx_motor_handle_t*motors[]={&motor_left,&motor_right};
```

```
rx_obstacle_detect_config_t config={.sensors=sensors,.sensor_count=2,.motors=motors,.motor_count=2,.detection_t
```

```
rx_err_t ret=rx_obstacle_detect_init(&detector,&config); if(ret!=k_rx_ok){ printf("Init failed:%dn",ret);  
return ret; }
```

```
ret=rx_obstacle_detect_start(&detector);
```

NASA Power of 10 Compliance:: Rule 3: [OK] No dynamic memory - all allocation is static Rule 5: [OK] Minimum 6 preconditions validated Rule 7: [OK] All ThreadX return values checked

See also: rx_obstacle_detect_config_t Configuration structure,

rx_obstacle_detect_start() Start monitoring after init, rx_obstacle_detect_deinit()

Cleanup and release resources, internal_validate_config() Internal configuration validation

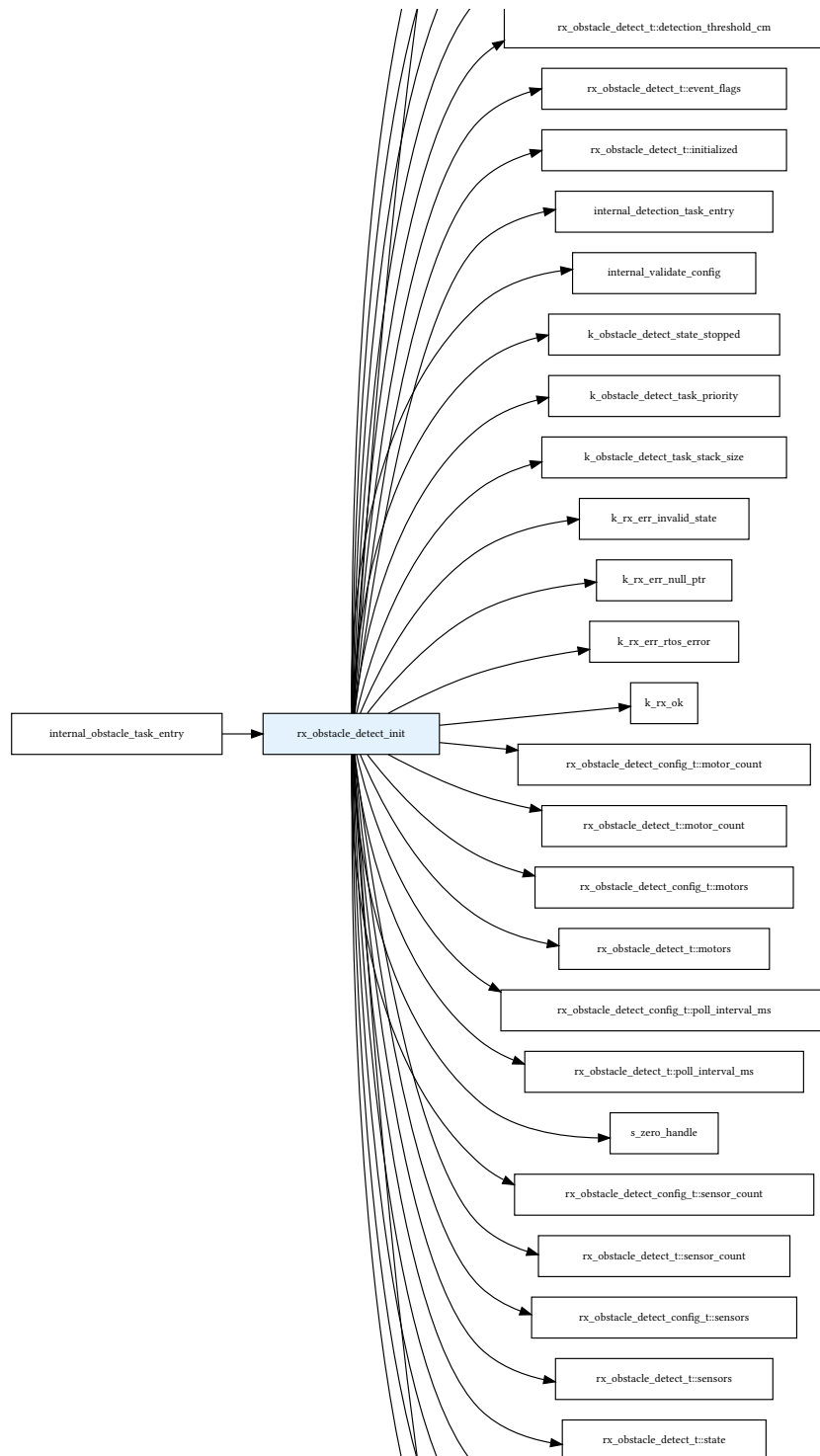


Figure 1049: Call graph for rx_obstacle_detect_init

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:312`

Since Version 1.0.0

—

rx_obstacle_detect_deinit

```
rx_err_t rx_obstacle_detect_deinit(rx_obstacle_detect_t *handle)
```

Deinitialize obstacle detection system and release all resources.

Deinitialize obstacle detection system.

Tears down a fully initialized obstacle detection handle by stopping active detection if running, terminating and deleting the ThreadX monitoring thread, deleting the event flags group, and clearing the initialized flag. After this call the handle may be reused with `rx_obstacle_detect_init()`.

Algorithm steps:

1. Validate handle is non-null and initialized
2. If detection is not stopped, call `rx_obstacle_detect_stop()` (errors ignored)
3. Terminate the ThreadX detection thread (TX_THREAD_ERROR ignored if already done)
4. Delete the ThreadX detection thread (TX_DELETE_ERROR ignored if already deleted)
5. Delete the TX event flags group (TX_DELETE_ERROR ignored if already deleted)
6. Clear handle->initialized

Parameters:

Direction	Name	Description
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Resources successfully released
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle was not previously initialized
k_rx_err_rtos_error	ThreadX thread terminate, delete, or event flags delete failed

Pre: handle must be a valid non-null pointer

Pre: handle must have been successfully initialized via `rx_obstacle_detect_init()`

Post: handle->initialized is false

Post: ThreadX thread and event flags group are deleted

Note: Not thread-safe; caller must ensure no concurrent access to handle

Note: Stop errors during deinit are silently ignored; the teardown continues regardless

Example:: rx_obstacle_detect_tdetector; rx_obstacle_detect_init(&detector,&config);
 rx_err_t err=rx_obstacle_detect_deinit(&detector); if(err!=k_rx_ok)
 { rx_log_error("app","Failed to deinit obstacle detector"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_init() Initialize the handle before use,
 rx_obstacle_detect_stop() Stop detection before deinit

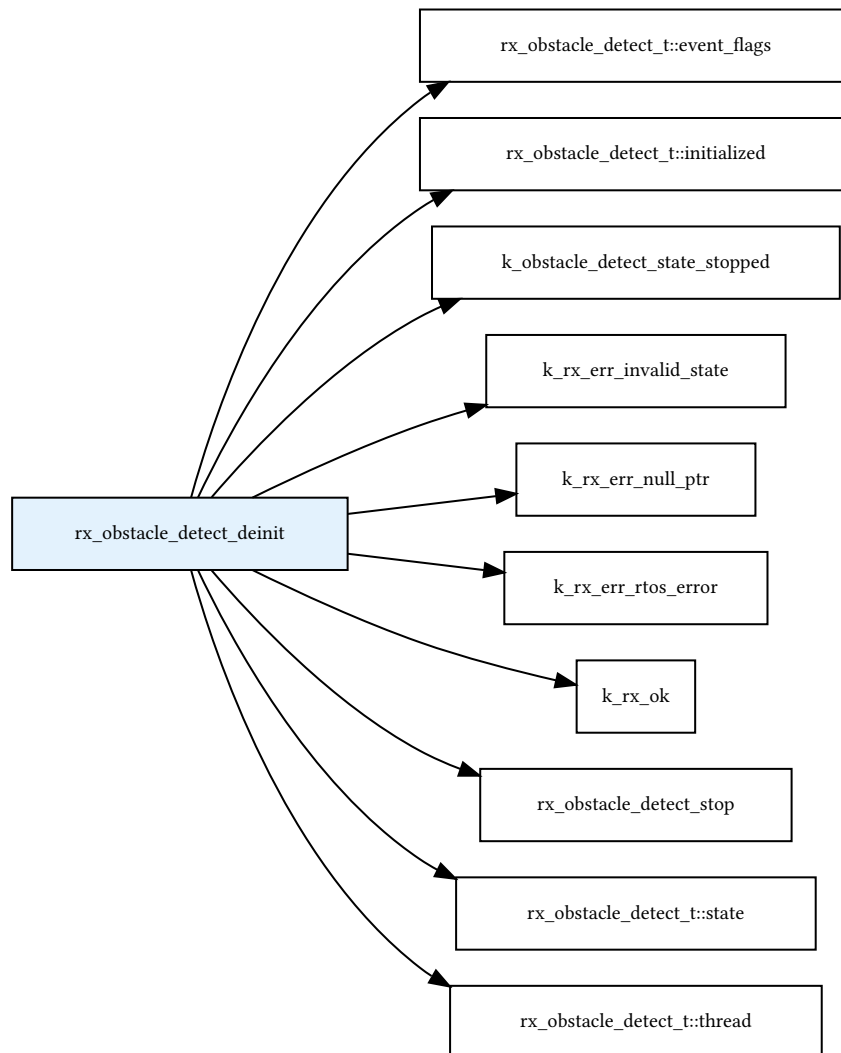


Figure 1050: Call graph for rx_obstacle_detect_deinit

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:434`

Since Version 1.0.0

—

rx_obstacle_detect_start

```
rx_err_t rx_obstacle_detect_start(rx_obstacle_detect_t *handle)
```

Start obstacle detection monitoring.

Transitions the obstacle detection system to the running state by resuming the ThreadX detection thread (if it was previously suspended) and signaling the k_event_flag_start event flag to the detection task.

The detection task waits on this event flag at startup; once received it enters the polling loop and begins continuous sensor monitoring.

Algorithm steps:

1. Validate handle is non-null and initialized
2. If state is k_obstacle_detect_state_stopped, resume the ThreadX thread
3. Set k_event_flag_start event flag via tx_event_flags_set()

Parameters:

Direction	Name	Description
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Detection started (or already running)
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_rtos_error	ThreadX thread resume or event flags set failed

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: System must be in ThreadX running state (tx_kernel_enter() called)

Post: Detection thread is executing and polling sensors

Post: handle->state transitions to k_obstacle_detect_state_running

Note: Not thread-safe; caller must serialize start/stop calls

Note: Calling start when already running is a no-op for thread resume

Example:: rx_err_t rx_obstacle_detect_start(&detector); if(err!=k_rx_ok)
 { rx_log_error("app","Failed to start obstacle detection"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_stop() Stop detection, rx_obstacle_detect_get_state() Query current detection state

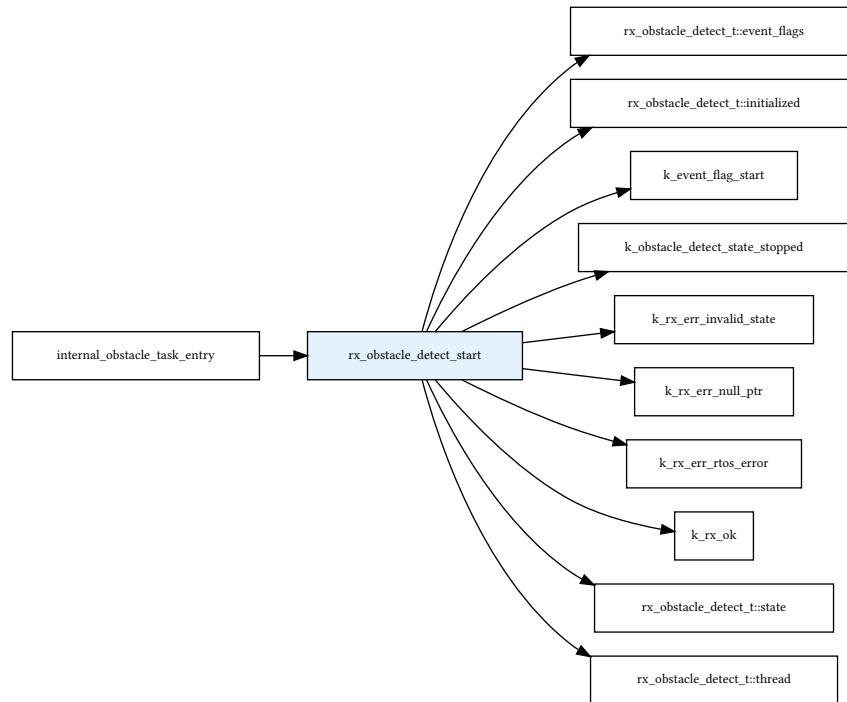


Figure 1051: Call graph for rx_obstacle_detect_start

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:530`

Since Version 1.0.0

rx_obstacle_detect_stop

```
rx_err_t rx_obstacle_detect_stop(rx_obstacle_detect_t *handle)
```

Stop obstacle detection monitoring.

Requests the obstacle detection task to cease polling by setting `handle->stop_requested` and signaling the `k_event_flag_stop` event flag. The detection task checks this flag at the top of each polling iteration and transitions to the stopped state upon receipt.

This function returns immediately after signaling; it does not wait for the task to actually stop. Use `rx_obstacle_detect_get_state()` to confirm the transition to `k_obstacle_detect_state_stopped` if synchronization is needed.

Algorithm steps:

1. Validate handle is non-null and initialized
2. Set handle->stop_requested to true
3. Set k_event_flag_stop event flag via tx_event_flags_set()

Parameters:

Direction	Name	Description
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Stop request successfully signaled
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_rtos_error	ThreadX event flags set failed

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: System must be in ThreadX running state

Post: handle->stop_requested is true

Post: k_event_flag_stop event flag is set; detection task will stop at next iteration

Note: Not thread-safe; caller must serialize start/stop calls

Warning: This function is non-blocking; the task may still be running immediately after return

Example:: rx_err_t err=rx_obstacle_detect_stop(&detector); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to stop obstacle detection"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_start() Resume detection, rx_obstacle_detect_get_state()
Poll state to confirm stop

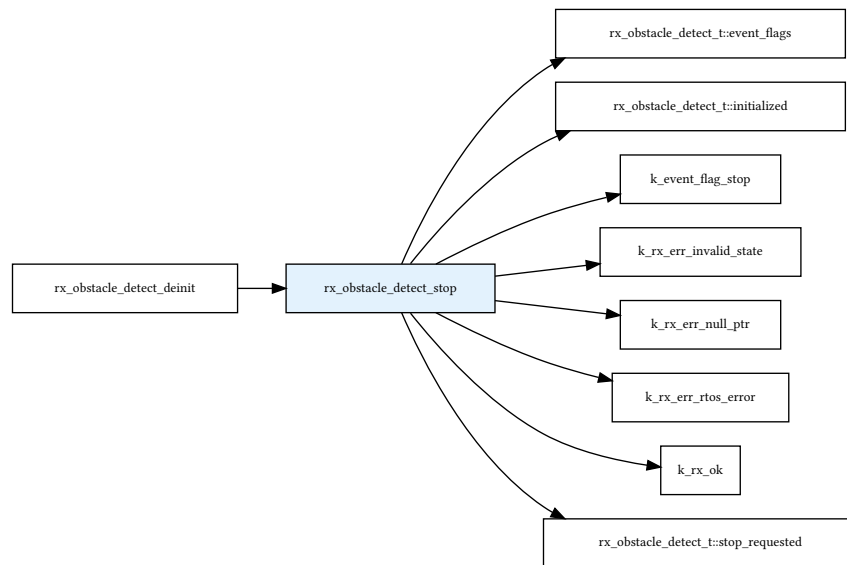


Figure 1052: Call graph for rx_obstacle_detect_stop

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:609`

Since Version 1.0.0

—

rx_obstacle_detect_clear_obstacle

```
rx_err_t rx_obstacle_detect_clear_obstacle(rx_obstacle_detect_t *handle)
```

Clear active obstacle state and reset per-sensor debounce counters.

Resume motors after obstacle cleared.

Clears all debounce counters and obstacle_active flags for every configured sensor. If the system is currently in k_obstacle_detect_state_obstacle, it transitions back to k_obstacle_detect_state_running.

This function is used to acknowledge a detected obstacle and allow normal operation to resume. It does NOT restart a stopped detection system.

Algorithm steps:

1. Validate handle is non-null and initialized
2. Iterate over all handle->sensor_count sensors
3. For each sensor: zero debounce_counter[i] and clear obstacle_active[i]
4. If state is k_obstacle_detect_state_obstacle, set state to k_obstacle_detect_state_running

Parameters:

Direction	Name	Description
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Obstacle state and debounce counters successfully cleared
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: Detection system should be running (though clearing while stopped is safe)

Post: All per-sensor debounce_counter values are zero

Post: All per-sensor obstacle_active flags are false

Note: Not thread-safe; clearing obstacle state while the detection task is concurrently updating sensor state may cause a race; caller must synchronize

Warning: Does not restart detection if stopped; use rx_obstacle_detect_start()

Example:: if(rx_obstacle_detect_is_obstacle_detected(&detector)){ handle_obstacle_event();
rx_obstacle_detect_clear_obstacle(&detector); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_is_obstacle_detected() Check for active obstacle,
rx_obstacle_detect_get_state() Query current state

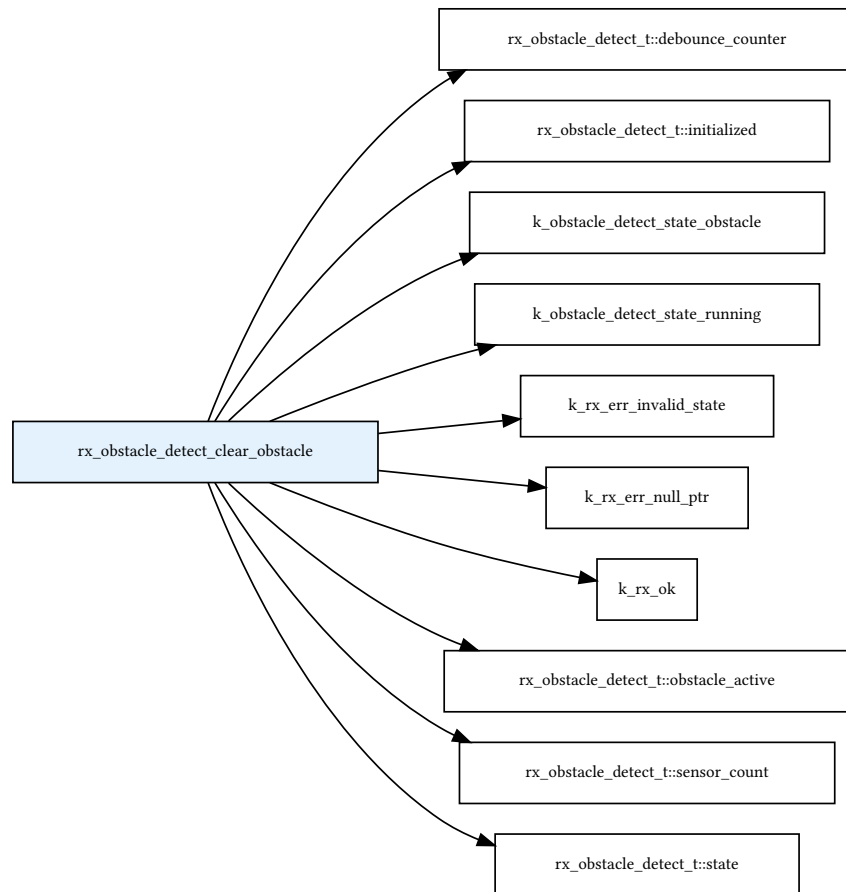


Figure 1053: Call graph for rx_obstacle_detect_clear_obstacle

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:681`

Since Version 1.0.0

—

rx_obstacle_detect_get_state

```
rx_err_t rx_obstacle_detect_get_state(const rx_obstacle_detect_t *handle,
rx_obstacle_detect_state_t *out_state)
```

Get the current obstacle detection state.

Get current detection state.

Returns the current operational state of the detection system by copying `handle->state` to `*out_state`. This is a cached read from handle memory; no bus or sensor transactions are issued.

Possible states:

- `k_obstacle_detect_state_stopped` task suspended, not polling
- `k_obstacle_detect_state_running` actively polling, no obstacle

- k_obstacle_detect_state_obstacle obstacle confirmed by debounce

Algorithm steps:

1. Validate handle and out_state pointer are non-null
2. Verify handle is initialized
3. Copy handle->state to *out_state

Parameters:

Direction	Name	Description
in	handle	Pointer to obstacle detection handle (must be initialized)
out	out_state	Receives the current rx_obstacle_detect_state_t value; undefined on error

Returns: rx_err_t Error code

Value	Description
k_rx_ok	State successfully returned
k_rx_err_null_ptr	handle or out_state is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: out_state must be a valid non-null pointer

Post: *out_state contains the current state value on k_rx_ok

Post: handle state is unchanged

Note: Not thread-safe; state may change concurrently if detection task is running

Note: Returns cached state does not re-evaluate sensor readings

Example:: rx_obstacle_detect_state_tstate;

```
rx_err_t err=rx_obstacle_detect_get_state(&detector,&state);
```

```
if(err==k_rx_ok&&state==k_obstacle_detect_state_obstacle){ handle_obstacle_event(); }
```

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null checks, initialized state), 2 postconditions

See also: rx_obstacle_detect_is_obstacle_detected() Convenience bool check,
rx_obstacle_detect_clear_obstacle() Clear obstacle state

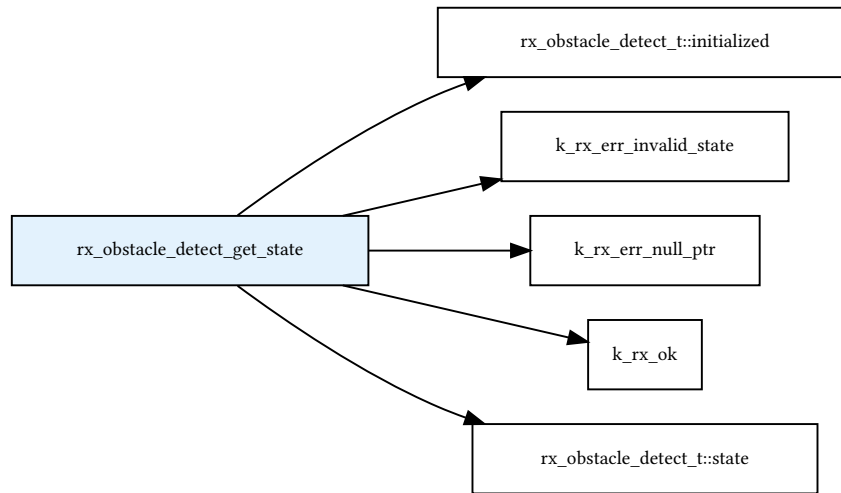


Figure 1054: Call graph for rx_obstacle_detect_get_state

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:763`

Since Version 1.0.0

—

rx_obstacle_detect_is_obstacle_detected

```
bool rx_obstacle_detect_is_obstacle_detected(const rx_obstacle_detect_t *handle)
```

Check whether an obstacle is currently detected (boolean convenience).

Check if obstacle is currently detected.

Returns true if the obstacle detection system is initialized and its current state is `k_obstacle_detect_state_obstacle`. Returns false for all other conditions including: null handle, uninitialized handle, stopped state, or running state with no obstacle.

This is a lightweight wrapper around `handle->state` that avoids allocating an output parameter, suitable for use in polling loops and conditional checks.

Algorithm steps:

1. Guard against null or uninitialized handle (returns false)
2. Return `(handle->state == k_obstacle_detect_state_obstacle)`

Parameters:

Direction	Name	Description
in	handle	Pointer to obstacle detection handle (may be null returns false)

Returns: bool Obstacle detection result

Value	Description
true	handle is valid and state is <code>k_obstacle_detect_state_obstacle</code>

false	handle is null, uninitialized, stopped, or running without obstacle
-------	---

Pre: None safe to call with null handle

Pre: For meaningful results, handle should be initialized and detection started

Post: handle state is unchanged

Post: Return value reflects handle->state at the instant of the call

Note: Not thread-safe; state may change concurrently if detection task is active

Note: Returns false for null handle rather than asserting; safe for defensive polling

Example:: if(rx_obstacle_detect_is_obstacle_detected(&detector)){ emergency_stop_all_motors();
rx_obstacle_detect_clear_obstacle(&detector); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null-safe guard, initialized check), 2 postconditions

See also: rx_obstacle_detect_get_state() Full state query with error code,
rx_obstacle_detect_clear_obstacle() Acknowledge and clear obstacle

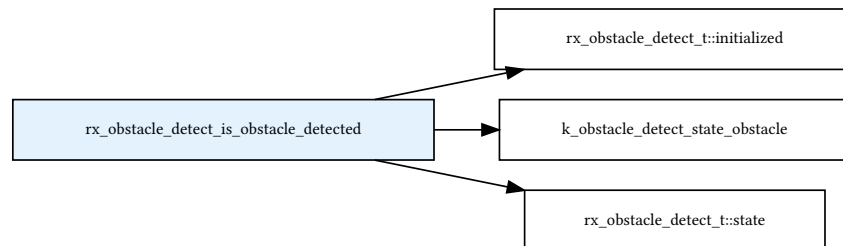


Figure 1055: Call graph for rx_obstacle_detect_is_obstacle_detected

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:826`

Since Version 1.0.0

rx_obstacle_detect_get_stats

```

rx_err_t rx_obstacle_detect_get_stats(const rx_obstacle_detect_t *handle,
uint32_t *out_total_polls, uint32_t *out_obstacle_events, uint32_t
*out_false_positives)
  
```

Retrieve accumulated diagnostic statistics from obstacle detection system.

Get obstacle detection statistics.

Returns the three running counters maintained by the detection task: total sensor poll cycles, confirmed obstacle events, and filtered false positive readings. All output parameters are optional passing nullptr for any counter skips that output without error.

Counters are incremented by the internal detection task and are NOT reset by this function. Use `rx_obstacle_detect_reset_stats()` to clear them.

Algorithm steps:

1. Validate handle is non-null and initialized
2. If `out_total_polls != nullptr`, copy `handle->total_polls`
3. If `out_obstacle_events != nullptr`, copy `handle->obstacle_events`
4. If `out_false_positives != nullptr`, copy `handle->>false_positive_count`

Parameters:

Direction	Name	Description
in	handle	Pointer to obstacle detection handle (must be initialized)
out	out_total_polls	Total sensor poll cycles since last reset; nullptr to skip; undefined on error
out	out_obstacle_events	Number of confirmed obstacle events since last reset; nullptr to skip; undefined on error
out	out_false_positives	Number of single-sample detections rejected by debounce filter since last reset; nullptr to skip; undefined on error

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Statistics successfully copied
<code>k_rx_err_null_ptr</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized

Pre: handle must be initialized via `rx_obstacle_detect_init()`

Pre: All non-null output pointers must be valid writable memory

Post: Output values are consistent snapshots at the time of the call

Post: Counters in the handle are unchanged

Note: Not thread-safe; counters may be incremented by detection task concurrently

Note: All three output pointers are optional; pass nullptr for any to skip that stat

Example:: `uint32_t polls=0, events=0, false_pos=0;`
`rx_err_t err=rx_obstacle_detect_get_stats(&detector, &polls, &events, &false_pos); if(err==k_rx_ok)`

```
{ rx_log_info("app","Polls:%u,Obstacles:%u,FP:%u", (unsigned)polls,(unsigned)events,
(unsigned>false_pos); }
```

NASA Power of 10 Compliance: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: `rx_obstacle_detect_reset_stats()` Clear all counters to zero

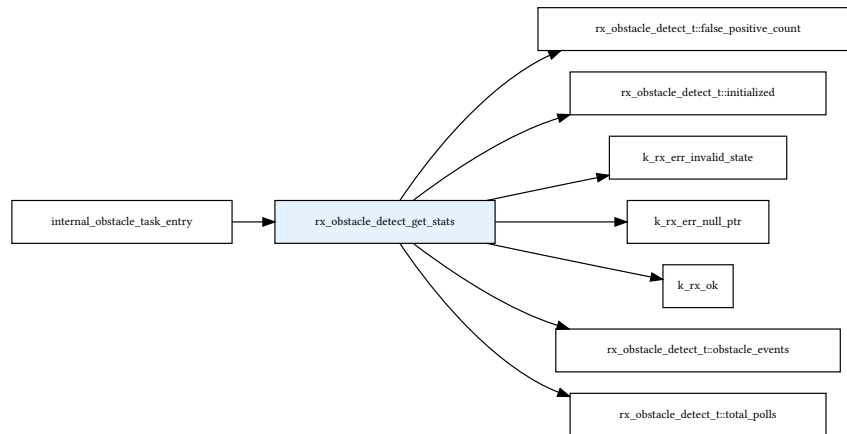


Figure 1056: Call graph for `rx_obstacle_detect_get_stats`

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:891`

Since Version 1.0.0

rx_obstacle_detect_reset_stats

```
rx_err_t rx_obstacle_detect_reset_stats(rx_obstacle_detect_t *handle)
```

Reset all accumulated diagnostic statistics counters to zero.

Reset statistics counters.

Clears the three running diagnostic counters maintained by the detection task: `total_polls`, `obstacle_events`, and `false_positive_count`. After this call all counters return to zero and begin accumulating from the next poll cycle.

Algorithm steps:

1. Validate handle is non-null and initialized
2. Set `handle->total_polls` to zero
3. Set `handle->obstacle_events` to zero
4. Set `handle->>false_positive_count` to zero

Parameters:

Direction	Name	Description
inout	handle	Pointer to obstacle detection handle (must be initialized)

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	All statistics counters successfully reset to zero
k_rx_err_null_ptr	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_obstacle_detect_init()

Pre: Caller should ensure detection task is not mid-cycle to avoid partial counts

Post: handle->total_polls is zero

Post: handle->obstacle_events is zero

Note: Not thread-safe; calling while detection task is incrementing counters may produce slightly inconsistent results; caller must synchronize if exactness required

Example:: rx_err_t err=rx_obstacle_detect_reset_stats(&detector); if(err!=k_rx_ok)
{ rx_log_error("app","Failed to reset obstacle detector stats"); }

NASA Power of 10 Compliance:: Rule 5: 2 preconditions (null check, initialized check), 2 postconditions

See also: rx_obstacle_detect_get_stats() Retrieve current counter values

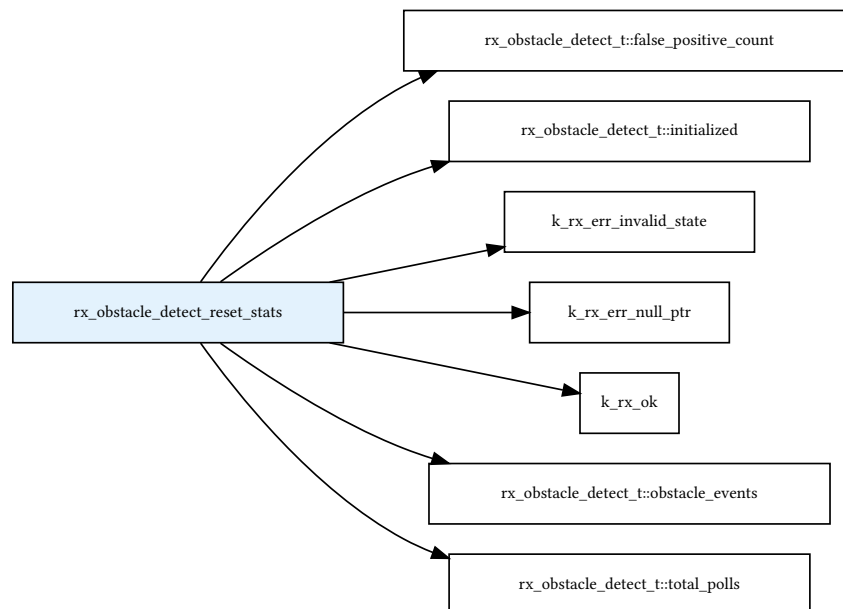


Figure 1057: Call graph for rx_obstacle_detect_reset_stats

Defined in `libs/rx_obstacle_detect/src/rx_obstacle_detect.c:966`

Since Version 1.0.0

—

rx_pid.h

PID Controller API for Closed-Loop Motor Control with Anti-Windup.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`

rx_pid_init

```
rx_err_t rx_pid_init(rx_pid_handle_t *handle, const rx_pid_config_t *config)
```

Initialize a PID controller instance.

Initialize a PID controller instance.

Configures a PID controller handle with gains, output limits, and integral anti-windup limits. This function must be called before any other PID operations can be performed. Initializes internal state (integral, prev_error) to zero for a clean start.

Initialization Steps:

1. Validate handle and config pointers (NULL checks)
2. Check if already initialized (prevent double-initialization)
3. Validate configuration constraints:

output_max > output_min integral_max > integral_min

4. Zero out handle structure (clear any previous state)
5. Copy configuration parameters to handle
6. Initialize state variables (integral = 0, prev_error = 0)
7. Set initialized flag to true

After successful initialization: handle->output_max > handle->output_min

After successful initialization: handle->integral_max > handle->integral_min

Gain Validation: Function does NOT validate gain values (kp, ki, kd) - negative or NaN/Inf gains will be accepted but cause incorrect behavior in `rx_pid_compute()`. Use `rx_pid_set_gains()` for validated gain updates.

1.0.0

Parameters:

Direction	Name	Description
out	handle	Pointer to PID handle structure. Must not be nullptr.
in	config	Pointer to PID configuration. Must not be nullptr.
out	handle	Pointer to PID controller handle structure Must not be NULL (validated) Will be zeroed and initialized with config parameters Must remain valid

		for lifetime of controller use Typically statically allocated: static rx_pid_handle_t motor_pid;
in	config	Pointer to PID configuration structure Must not be NULL (validated) Can be stack-allocated (copied to handle, not retained) Fields validated: output_max > output_min, integral_max > integral_min See rx_pid_config_t documentation for field details

Returns: rx_err_t Error code indicating initialization result

Value	Description
k_rx_ok	Initialization successful, controller ready for use
k_rx_err_null_ptr	handle or config pointer is nullptr
k_rx_err_invalid_state	Controller already initialized (call rx_pid_deinit first)
k_rx_err_invalid_arg	Configuration validation failed: output_max <= output_min, OR integral_max <= integral_min

Pre: handle must point to allocated rx_pid_handle_t structure

Pre: config must point to valid rx_pid_config_t with properly set fields

Pre: handle must not be already initialized (or call rx_pid_deinit first)

Pre: config->output_max > config->output_min (strictly greater)

Pre: config->integral_max > config->integral_min (strictly greater)

Post: handle->initialized == true (on success)

Post: handle->integral == 0.0f (clean initial state)

Post: handle->prev_error == 0.0f (clean initial state)

Post: Configuration parameters copied to handle

Post: Controller ready for rx_pid_compute() calls

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Memory: No dynamic allocation - all state in provided handle structure

Note: Re-initialization: Must call rx_pid_deinit() before re-initializing same handle

Note: Performance: 625 ns @ 240 MHz (150 cycles) - one-time cost

Warning: State Persistence: Handle must remain valid for entire controller lifetime

Warning: Configuration Lifetime: Config can be stack-allocated (copied, not retained)

Warning: Double Init: Attempting to init already-initialized handle returns error] **Example:**
Motor Velocity Control Initialization: #include "rx_pid.h"

```
static rx_pid_handle_t motor_pid;

void motor_init(void)
{ rx_pid_config_t config = { .kp=0.286f, .ki=8.01f, .kd=0.0f, .output_min=-100.0f, .output_max=100.0f, .integral_min=-50.0f, .integral_max=50.0f };
  rx_err_t err = rx_pid_init(&motor_pid, &config); if (err == k_rx_ok)
  { rx_log_info("MOTOR", "PID initialized successfully"); }
  else { rx_log_error_val("MOTOR", "PID init failed", err); } }
```

Example: Error Handling: rx_pid_handle_t pid; rx_pid_config_t config = {...};

```
rx_err_t err = rx_pid_init(&pid, &config); switch(err) { case k_rx_ok: break; case k_rx_err_null_ptr:
rx_log_error("PID", "null ptr init"); break; case k_rx_err_invalid_state:
rx_log_warn("PID", "Already initialized-deinit first"); rx_pid_deinit(&pid); rx_pid_init(&pid, &config);
break; case k_rx_err_invalid_arg: rx_log_error("PID", "Invalid config (check output/integral limits)");
break; }
```

Example: Configuration Validation Failures:

```
rx_pid_config_t bad_config1 = { .output_min=100.0f, .output_max=50.0f, };
rx_pid_init(&pid, &bad_config1);

rx_pid_config_t bad_config2 = { .output_min=-100.0f, .output_max=100.0f, .integral_min=50.0f, .integral_max=50.0f, };
rx_pid_init(&pid, &bad_config2);
```

Performance Analysis:: Execution time: 625 ns @ 240 MHz (150 cycles) Memory: 40 bytes (sizeof(rx_pid_handle_t)) Stack usage: 0 bytes (parameters passed by pointer) Called: Once per controller instance at startup Critical path: No (one-time initialization, not in control loop)

Parameter Validation Table:: Parameter Validation Error Code

```
handle != nullptr k_rx_err_null_ptr
config != nullptr k_rx_err_null_ptr
handle->initialized
```

false

```
k_rx_err_invalid_state
```

```
config->output_max > config->output_min k_rx_err_invalid_arg
```

```
config->integral_max > config->integral_min k_rx_err_invalid_arg
```

NASA Power of 10 Compliance:: Rule 4: Function is 29 lines (well under 60-line limit) Rule 5: 5 validation checks (NULLx2, initialized, output limits, integral limits) Rule 7: All return values must be checked by caller

See also: `rx_pid_config_t` for configuration structure details, `rx_pid_deinit()` to deinitialize and allow re-initialization, `rx_pid_compute()` to perform PID computation after initialization, `rx_pid_reset()` to clear state without deinitialization, `rx_pid_set_gains()` to update gains after initialization, `matlab/pid_design_velocity.m` for gain tuning methodology

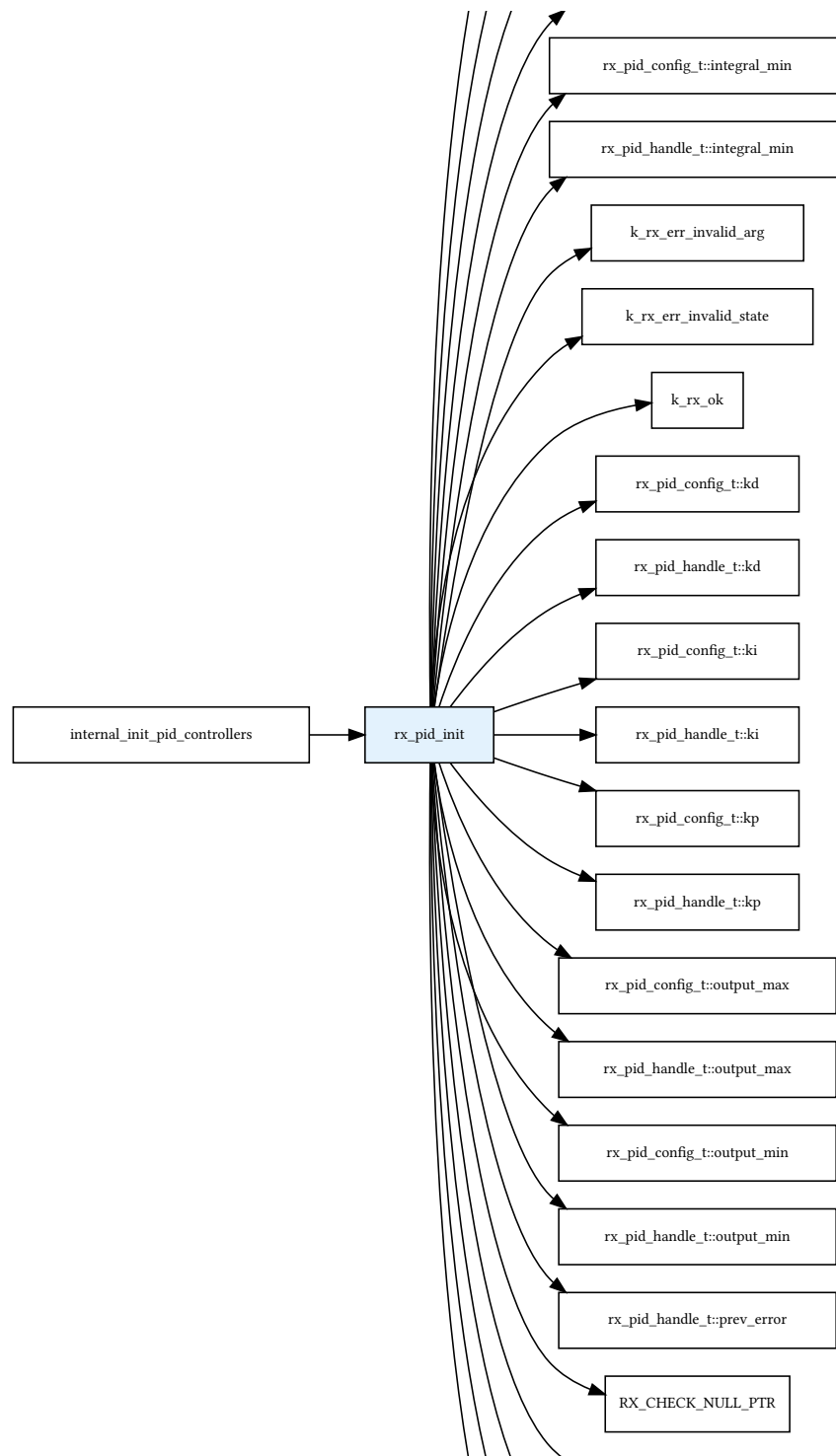


Figure 1058: Call graph for rx_pid_init

Defined in `libs/rx_pid/inc/rx_pid.h:406`

Since Version 1.0.0

—

rx_pid_deinit

```
rx_err_t rx_pid_deinit(rx_pid_handle_t *handle)
```

Deinitialize PID controller and clear state.

Deinitialize PID controller and clear state.

Deinitializes a PID controller handle, clearing all configuration parameters and internal state. After deinitialization, the handle can be reinitialized with different parameters or safely deallocated (if dynamically allocated, though not recommended).

Deinitialization Steps:

1. Validate handle pointer (NULL check)
2. Check if currently initialized (prevent double-deinitialization)
3. Zero entire handle structure (memset to 0)
4. Log deinitialization event

Use Cases:

- Controller shutdown before system reset
- Switching between different control modes
- Clearing state before reconfiguration with different gains/limits
- Testing/debugging (force clean state)

1.0.0

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized PID handle. Must not be nullptr.
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be currently initialized (validated) Will be zeroed on success (all fields set to 0) After deinitialization: handle->initialized == false

Returns: rx_err_t Error code indicating deinitialization result

Value	Description
k_rx_ok	Deinitialization successful, handle cleared and ready for re-init
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (already deinitialized or never initialized)

Pre: handle must not be nullptr

Pre: handle->initialized must be true (controller must be initialized first)

Post: handle->initialized == false (on success)

Post: All handle fields zeroed: gains, limits, integral, prev_error == 0

Post: Handle can be safely reinitialized with rx_pid_init()

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 125 ns @ 240 MHz (30 cycles) - memset is fast

Note: Re-initialization: After deinit, handle can be reinitialized with rx_pid_init()

Note: Memory Safety: Does not free memory (handle is caller-managed)

Warning: Active Control Loops: Do not deinitialize while control loop is active - stop control loop first

Warning: State Loss: All PID state (integral, prev_error) is lost - cannot be recovered

Example: Clean Shutdown: motor_control_active=false; tx_thread_sleep(10);

```
rx_err_t err = rx_pid_deinit(&motor_pid); if(err != k_rx_ok)
{ rx_log_info("MOTOR", "PID deinitialized successfully"); }
```

Example: Reconfiguration Pattern: rx_pid_deinit(&pid);

```
rx_pid_config_t position_config = { .kp=1.5f, .ki=0.2f, .kd=0.1f, }; rx_pid_init(&pid, &position_config);
```

Example: Error Handling: rx_err_t err = rx_pid_deinit(&pid); switch(err){ case k_rx_ok: break; case k_rx_err_null_ptr: rx_log_error("PID", "Cannot deinit null ptr handle"); break; case k_rx_err_invalid_state: rx_log_warn("PID", "Already deinitialized or never initialized"); break; }

Performance Analysis:: Execution time: 125 ns @ 240 MHz (30 cycles) Memory: Clears 40 bytes (sizeof(rx_pid_handle_t)) Stack usage: 0 bytes (parameters passed by pointer) Called: Once per controller instance at shutdown or reconfiguration

NASA Power of 10 Compliance:: Rule 4: Function is 12 lines (well under 60-line limit) Rule 5: 2 validation checks (nullptr, initialization state)

See also: rx_pid_init() to (re)initialize controller after deinitialization, rx_pid_reset() to clear state WITHOUT deinitialization (preserves configuration)

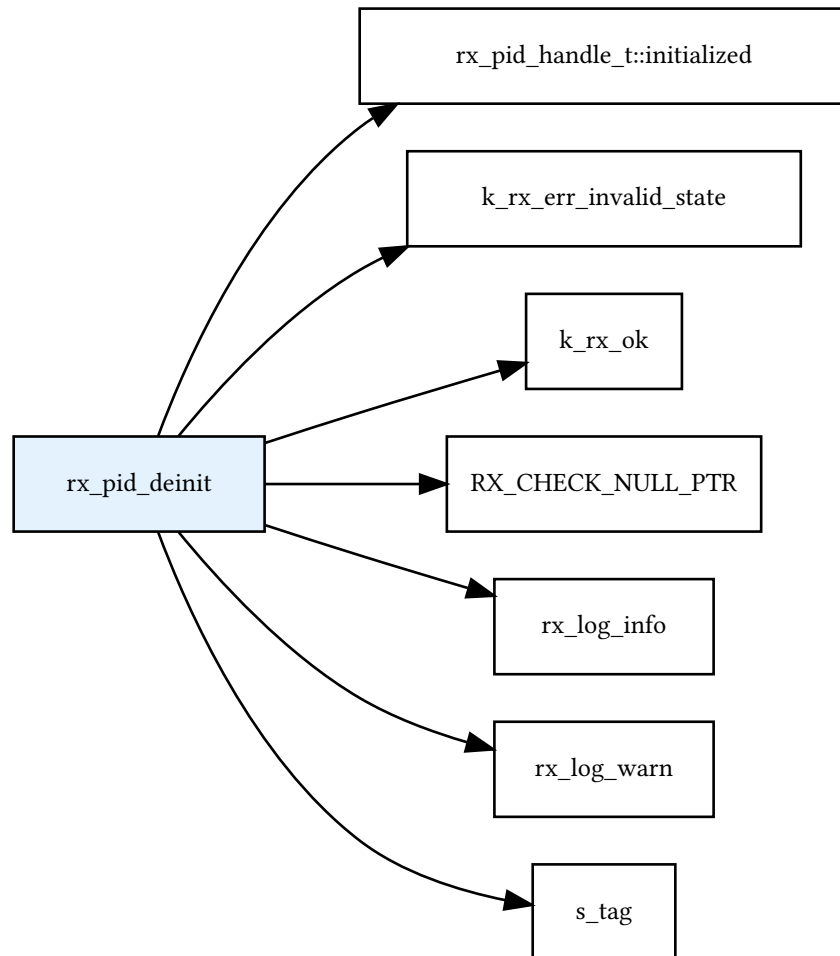


Figure 1059: Call graph for rx_pid_deinit

Defined in `libs/rx_pid/inc/rx_pid.h:417`

Since Version 1.0.0

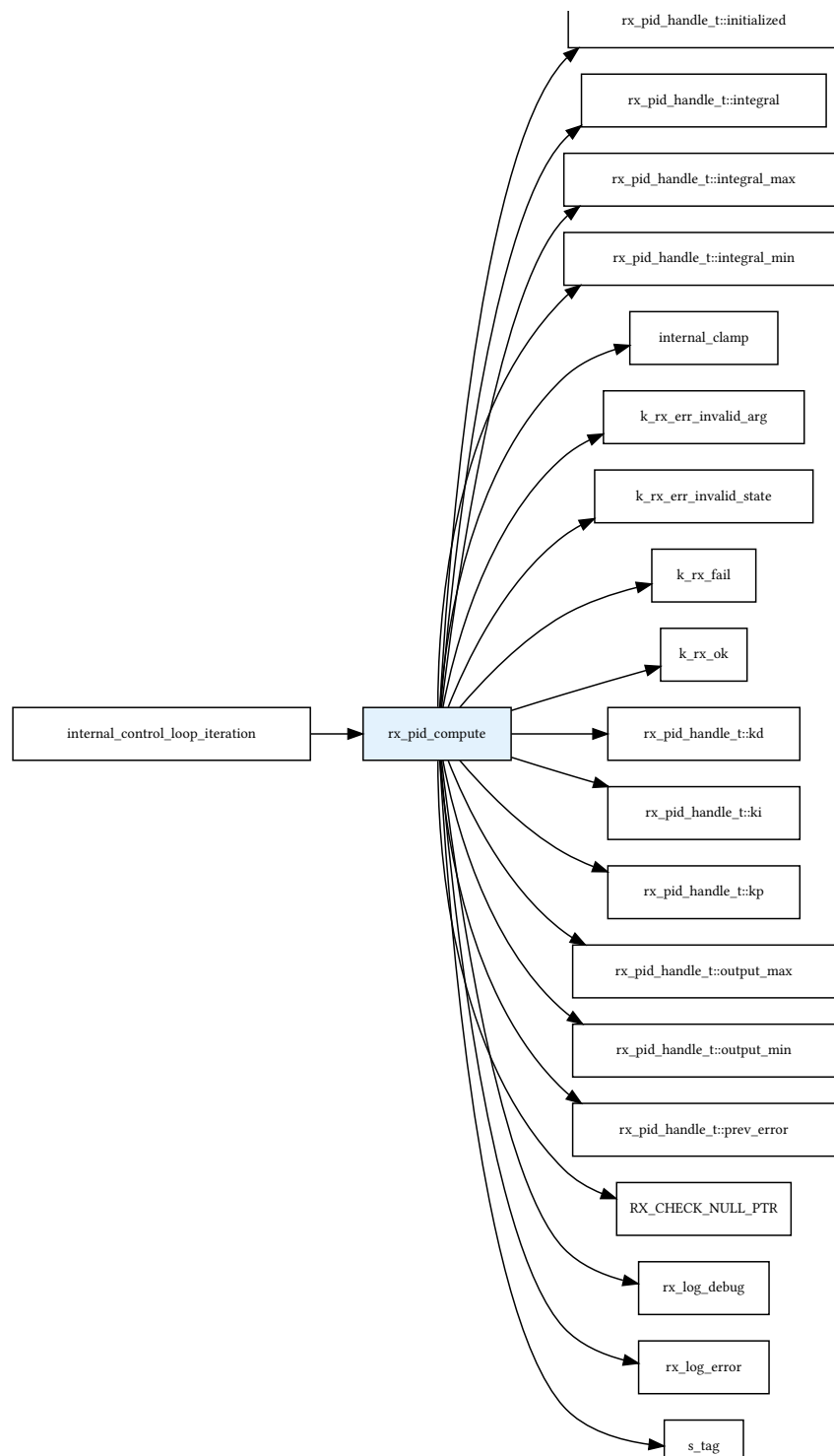
—

rx_pid_compute

```
rx_err_t rx_pid_compute(rx_pid_handle_t *handle, float setpoint, float measured,
float dt, float *output)
```

Compute PID control output for current sample.

Core PID computation function implementing the discrete parallel-form PID algorithm with integral anti-windup and output saturation. Call this function at a fixed sample rate (e.g., 250 Hz for motor control) to generate control outputs.

Figure 1060: Call graph for `rx_pid_compute`

Defined in `libs/rx_pid/inc/rx_pid.h:594`

—

rx_pid_reset

```
rx_err_t rx_pid_reset(rx_pid_handle_t *handle)
```

Reset PID controller internal state.

Clears the integral accumulator and previous error. Useful when changing setpoints or recovering from disturbances to prevent integral windup.

Reset PID controller internal state.

Clears the integral accumulator and previous error state while preserving all configuration parameters (gains, limits). This is useful when changing setpoints, recovering from disturbances, or preventing integral windup after prolonged saturation.

Reset Operations:

1. Validate handle pointer (NULL check)
2. Check initialization state
3. Clear integral accumulator (integral = 0.0f)
4. Clear previous error (prev_error = 0.0f)
5. Log reset event

Configuration Preserved (NOT cleared):

- Gains: kp, ki, kd
- Output limits: output_min, output_max
- Integral limits: integral_min, integral_max
- Initialization flag: initialized

When to Use:

- Large setpoint changes: Prevents integral windup from old error
- Mode transitions: Switching between position/velocity control
- After saturation: Clear accumulated error after prolonged output limiting
- Disturbance recovery: Reset after external disturbances (collisions, stalls)
- System startup: Ensure clean initial state before control begins

Gains remain unchanged: kp, ki, kd

Limits remain unchanged: output_min/max, integral_min/max

Initialization state remains true

1.0.0

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized PID handle. Must not be nullptr.
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) integral and prev_error fields will be zeroed Configuration (gains, limits) remains unchanged

Returns: rx_err_t Error code indicating reset result

Value	Description
k_rx_ok	State reset successful, controller ready with clean state
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Post: handle->integral == 0.0f (on success)

Post: handle->prev_error == 0.0f (on success)

Post: Configuration parameters unchanged (gains, limits)

Post: Controller ready for rx_pid_compute() with clean state

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 125 ns @ 240 MHz (30 cycles) - simple field assignment

Note: Next Computation: First rx_pid_compute() after reset will have derivative = 0

Note: Difference from Deinit: Preserves configuration, only clears state

Warning: Active Control: Do not reset during critical control phases - causes output discontinuity

Warning: Derivative Spike: Reset causes prev_error = 0, so next derivative term may spike if error is large] **Example: Setpoint Change:** #include "rx_pid.h"

```
void change_target_speed(float new_target_rpm){ rx_err_t err = rx_pid_reset(&motor_pid); if(err != k_rx_ok){ rx_log_error_val("MOTOR", "PID reset failed", err); return; }
target_rpm = new_target_rpm; rx_log_info_val("MOTOR", "New target RPM", (uint32_t) new_target_rpm);
}
```

Example: Disturbance Recovery: if(motor_current > k_stall_threshold_ma)
{ motor_set_duty_cycle(0.0f);
rx_pid_reset(&motor_pid);

```
tx_thread_sleep(100);  
motor_control_active=true; }
```

Example: Startup Sequence: void motor_system_init(void){ rx_pid_config_t config={...};
rx_pid_init(&motor_pid,&config);
rx_pid_reset(&motor_pid);
start_motor_control_task(); }

Example: Mode Switching: void motor_idle(void){ rx_pid_reset(&motor_pid);
target_rpm=0.0f;
}

Performance Analysis:: Execution time: 125 ns @ 240 MHz (30 cycles) Memory: Modifies 8 bytes (2 floats: integral, prev_error) Stack usage: 0 bytes (parameters passed by pointer) Called: As needed (setpoint changes, disturbances, mode transitions)

Reset vs. Deinit Comparison:: Operation rx_pid_reset() rx_pid_deinit()

Clear integral YES YES

Clear prev_error YES YES

Clear gains NO YES

Clear limits NO YES

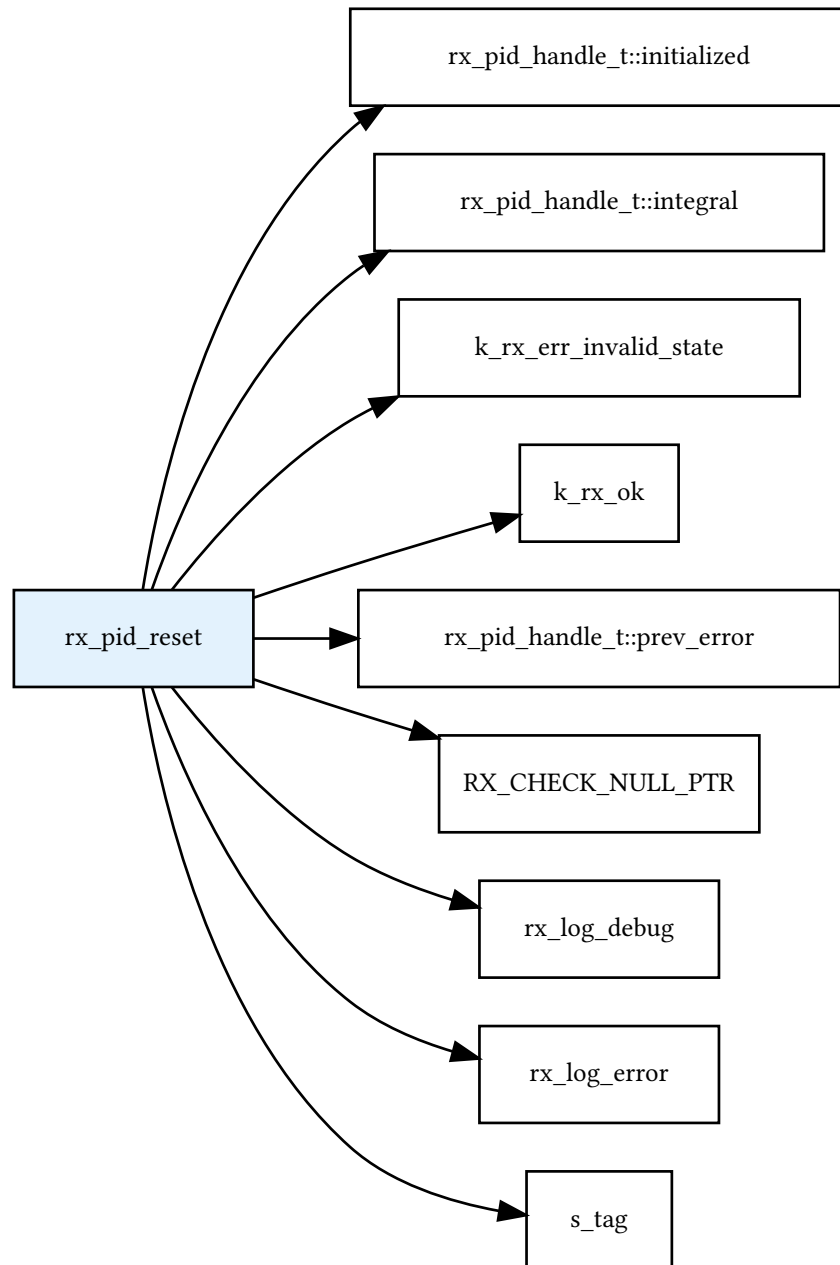
Clear initialized flag NO YES

Can compute after YES NO (must reinit)

Use case State cleanup Full shutdown

NASA Power of 10 Compliance:: Rule 4: Function is 12 lines (well under 60-line limit) Rule 5: 2 validation checks (nullptr, initialization state)

See also: rx_pid_init() to initialize controller, rx_pid_deinit() to fully deinitialize (clears configuration too), rx_pid_compute() to perform computation after reset, rx_pid_set_gains() to change gains without resetting state

Figure 1061: Call graph for `rx_pid_reset`

Defined in `libs/rx_pid/inc/rx_pid.h:608`

Since Version 1.0.0

—

rx_pid_set_gains

```
rx_err_t rx_pid_set_gains(rx_pid_handle_t *handle, const float kp, const float
ki, const float kd)
```

Update PID gains at runtime.

Allows tuning of PID gains without reinitializing the controller. The internal state (integral, prev_error) is preserved.

Update PID gains at runtime.

Allows runtime modification of PID gains (Kp, Ki, Kd) without reinitializing the controller. Internal state (integral, prev_error) is preserved, enabling smooth gain transitions during operation. Useful for adaptive control, auto-tuning, and gain scheduling applications.

Validation Steps:

1. nullptr check (handle)
2. Initialization state check
3. Validate gains are non-negative (kp >= 0, ki >= 0, kd >= 0)
4. Validate gains are finite (not NaN or Inf)
5. Store new gains
6. Verify storage success (post-condition check per NASA Rule 5)

State Preserved:

- integral accumulator (NOT cleared)
- prev_error (NOT cleared)
- Output limits (output_min, output_max)
- Integral limits (integral_min, integral_max)

handle->integral unchanged (state preserved for smooth transition)

handle->prev_error unchanged

handle->output_min/max unchanged

handle->integral_min/max unchanged

Auto-Tuning: When implementing auto-tuners, validate new gains before applying

1.0.0

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized PID handle. Must not be nullptr.
in	kp	New proportional gain (must be >= 0 and finite).
in	ki	New integral gain (must be >= 0 and finite).
in	kd	New derivative gain (must be >= 0 and finite).
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) Gains will be updated on success State (integral, prev_error) preserved
in	kp	New proportional gain (K_p) Must be >= 0.0 (validated - negative gains cause instability) Must be finite (validated - NaN/Inf causes undefined

		behavior) Units: output / error Example: 0.286 for motor velocity control Higher = faster response, risk of overshoot
in	ki	New integral gain (K_i) Must be ≥ 0.0 (validated) Must be finite (validated) Units: output / (error * s) Example: 8.01 for motor velocity control Higher = faster steady-state error elimination, risk of windup
in	kd	New derivative gain (K_d) Must be ≥ 0.0 (validated) Must be finite (validated) Units: output / (error/s) Example: 0.0 (disabled for noisy encoder feedback) Higher = more damping, risk of noise amplification

Returns: rx_err_t Error code indicating gain update result

Value	Description
k_rx_ok	Gains updated successfully, controller ready with new gains
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)
k_rx_err_invalid_arg	One or more gains are negative or non-finite (NaN/Inf)
k_rx_fail	Gain storage verification failed (should never happen)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Pre: $kp \geq 0.0$ and $isfinite(kp)$

Pre: $ki \geq 0.0$ and $isfinite(ki)$

Pre: $kd \geq 0.0$ and $isfinite(kd)$

Post: handle->kp == kp (on success)

Post: handle->ki == ki (on success)

Post: handle->kd == kd (on success)

Post: Internal state preserved (integral, prev_error unchanged)

Post: Controller ready for rx_pid_compute() with new gains

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 250 ns @ 240 MHz (60 cycles) - includes validation

Note: Smooth Transition: Preserving state enables bumpless gain changes

Note: Negative Gains: Not allowed - would cause positive feedback (instability)

Warning: Stability: Large gain changes during operation may cause transient oscillations

Warning: Integral Term: Existing integral may be incompatible with new Ki - consider rx_pid_reset() if changing Ki significantly

Warning: Zero Gains: $k_p = k_i = k_d = 0$ results in zero output (valid but useless)] **Example:**
Manual Tuning at Runtime: #include "rx_pid.h"

```
void increase_responsiveness(void){ float kp=motor_pid.kp; float ki=motor_pid.ki;
float kd=motor_pid.kd;

kp*=1.1f;

rx_err_tterr=rx_pid_set_gains(&motor_pid,kp,ki,kd); if(err==k_rx_ok)
{ rx_log_info("TUNE","Kp increased successfully"); }
else{ rx_log_error_val("TUNE","Gain update failed",err); } }

Example: Ziegler-Nichols Auto-Tuner: void auto_tune_pid(rx_pid_handle_t*pid)
{ float ku=measure_critical_gain(); float pu=measure_oscillation_period();

float kp=0.6f*ku; float ki=1.2f*ku/pu; float kd=0.075f*ku*pu;

if(kp>0.0f&&ki>0.0f&&kd>0.0f){ rx_err_tterr=rx_pid_set_gains(pid,kp,ki,kd); if(err==k_rx_ok)
{ rx_log_info("TUNE","Auto-tuned gains applied"); } }else{ rx_log_error("TUNE","Invalid auto-
tuned gains"); } }
```

Example: Gain Scheduling (Velocity-Dependent):

```
void update_gains_for_velocity(float current_rpm){ float kp,ki,kd;

if(current_rpm<50.0f){ kp=0.4f; ki=10.0f; kd=0.0f; }else{ kp=0.2f; ki=5.0f; kd=0.0f; }

rx_pid_set_gains(&motor_pid,kp,ki,kd); }
```

Example: Error Handling: rx_err_tterr=rx_pid_set_gains(&pid,0.5f,2.0f,0.1f);

```
switch(err){ case k_rx_ok: rx_log_info("PID","Gains updated successfully"); break;
case k_rx_err_null_ptr: rx_log_error("PID","null ptr handle pointer"); break;
case k_rx_err_invalid_state: rx_log_error("PID","PID not initialized"); break; case k_rx_err_invalid_arg:
rx_log_error("PID","Invalid gains (negative or NaN/Inf)"); break; case k_rx_fail:
rx_log_error("PID","Gain storage verification failed"); break; }
```

Performance Analysis:: Execution time: 250 ns @ 240 MHz (60 cycles) Memory: Modifies 12 bytes (3 floats: kp, ki, kd) Stack usage: 0 bytes (parameters passed by value/pointer) Validation overhead: 4 checks (nullptr, init, range, finite)

Gain Validation Table:: Validation Condition Error Code

nullptr handle != nullptr k_rx_err_null_ptr

Initialized handle->initialized == true k_rx_err_invalid_state

Non-negative kp >= 0 && ki >= 0 && kd >= 0 k_rx_err_invalid_arg

Finite isfinite(kp/ki/kd) k_rx_err_invalid_arg

Storage handle->kp == kp (etc.) k_rx_fail

NASA Power of 10 Compliance:: Rule 4: Function is 29 lines (well under 60-line limit) Rule 5: 4 precondition checks + 1 postcondition check (exceeds minimum 2)

See also: rx_pid_init() to initialize controller with initial gains, rx_pid_reset() to clear state if changing Ki significantly, rx_pid_compute() uses the updated gains in next computation, matlab/pid_design_velocity.m for gain tuning methodology

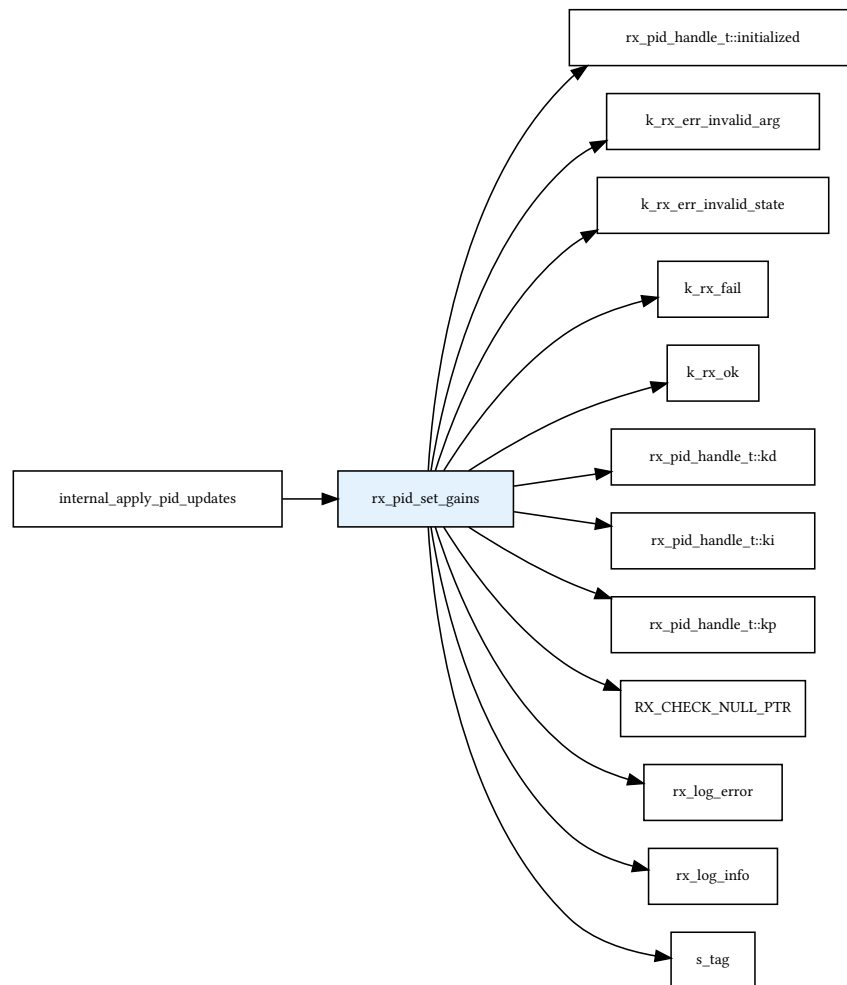


Figure 1062: Call graph for rx_pid_set_gains

Defined in `libs/rx_pid/inc/rx_pid.h:628`

Since Version 1.0.0

—

rx_pid_set_output_limits

```
rx_err_t rx_pid_set_output_limits(rx_pid_handle_t *handle, float output_min,
float output_max)
```

Update PID output limits at runtime.

Allows changing the output saturation limits without reinitializing.

Update PID output limits at runtime.

Modifies the output saturation limits (output_min, output_max) without reinitializing the controller. The output limits define the range to which the PID output is clamped before being applied to the actuator. Useful for adapting to different operating modes or actuator constraints.

Update Steps:

1. Validate handle pointer (NULL check)
2. Check initialization state
3. Validate output_max > output_min (strict inequality)
4. Store new limits
5. Log update event

Note: This function does NOT clamp the current output or state. The new limits take effect on the next call to rx_pid_compute().

handle->integral unchanged

handle->kp, ki, kd unchanged

handle->integral_min/max unchanged

Integral Limits: Consider updating integral_min/max proportionally when changing output limits

1.0.0

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized PID handle. Must not be nullptr.
in	output_min	New minimum output limit.
in	output_max	New maximum output limit.
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) output_min and output_max fields will be updated State and gains preserved
in	output_min	New minimum output limit (lower saturation bound) Must be < output_max (validated - strict inequality) Units depend on application (% duty cycle, voltage, current, etc.) Example: -100.0f for full reverse motor duty cycle Can be negative, zero, or positive
in	output_max	New maximum output limit (upper saturation bound) Must be > output_min (validated - strict inequality) Units must match

		output_min Example: +100.0f for full forward motor duty cycle Must be strictly greater than output_min
--	--	--

Returns: rx_err_t Error code indicating limit update result

Value	Description
k_rx_ok	Output limits updated successfully
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)
k_rx_err_invalid_arg	output_max <= output_min (not strictly greater)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Pre: output_max > output_min (strictly greater, not equal)

Post: handle->output_min == output_min (on success)

Post: handle->output_max == output_max (on success)

Post: New limits take effect on next rx_pid_compute() call

Post: Gains and state unchanged (kp, ki, kd, integral, prev_error)

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 200 ns @ 240 MHz (50 cycles) - includes validation

Note: Immediate Effect: New limits apply starting with next rx_pid_compute() call

Note: State Preserved: Integral and derivative state not affected

Warning: Symmetric vs Asymmetric: Ensure limits match actuator capabilities (e.g., motor can reverse)

Warning: No Auto-Clamping: Existing integral state is NOT clamped to new limits - only future outputs] **Example: Reduce Speed for Safety Mode:** #include "rx_pid.h"

```
void enter_safety_mode(void){ rx_err_t err=rx_pid_set_output_limits(&motor_pid,-50.0f,50.0f);
if(err!=k_rx_ok){ rx_log_info("MOTOR","Safety mode: output limited to +/-50%"); } }

void exit_safety_mode(void){ rx_pid_set_output_limits(&motor_pid,-100.0f,100.0f);
rx_log_info("MOTOR","Normal mode: full output range restored"); }
```

Example: Asymmetric Limits (Brake-Only Mode): void enable_brake_only_mode(void)
{ rx_pid_set_output_limits(&motor_pid,-100.0f,0.0f); rx_log_info("MOTOR","Brake-only mode active"); }

Example: Runtime Limit Adjustment Based on Temperature:
void adjust_limits_for_temperature(float temperature_celsius){ const float k_max_temp=80.0f;
const float k_warn_temp=60.0f; const float k_max_duty=100.0f;
float scaled_max=k_max_duty; if(temperature_celsius>k_warn_temp)
{ scaled_max=k_max_duty*(1.0f-(temperature_celsius-k_warn_temp)/(k_max_temp-
k_warn_temp)); }
if(scaled_max>k_max_duty)scaled_max=k_max_duty; if(scaled_max<20.0f)scaled_max=20.0f;
rx_pid_set_output_limits(&motor_pid,-scaled_max,scaled_max); }

Example: Error Handling: rx_err_t err=rx_pid_set_output_limits(&pid,-100.0f,100.0f);
switch(err){ case k_rx_ok: rx_log_info("PID","Output limits updated"); break; case k_rx_err_null_ptr:
rx_log_error("PID","null ptr handle pointer"); break; case k_rx_err_invalid_state:
rx_log_error("PID","PID not initialized"); break; case k_rx_err_invalid_arg:
rx_log_error("PID","Invalid limits (max <= min)"); break; }

Performance Analysis:: Execution time: 200 ns @ 240 MHz (50 cycles) Memory: Modifies 8 bytes
(2 floats: output_min, output_max) Stack usage: 0 bytes (parameters passed by value/pointer)
Validation overhead: 3 checks (nullptr, init, max > min)

Limit Validation Table:: Validation Condition Error Code

nullptr handle != nullptr k_rx_err_null_ptr

Initialized handle->initialized == true k_rx_err_invalid_state

Strictly greater output_max > output_min k_rx_err_invalid_arg

NASA Power of 10 Compliance:: Rule 4: Function is 15 lines (well under 60-line limit) Rule 5: 3
validation checks (nullptr, init, range)

See also: rx_pid_init() to initialize controller with initial output limits,
rx_pid_set_integral_limits() to update anti-windup limits (usually proportional to
output limits), rx_pid_compute() applies the output limits via clamping

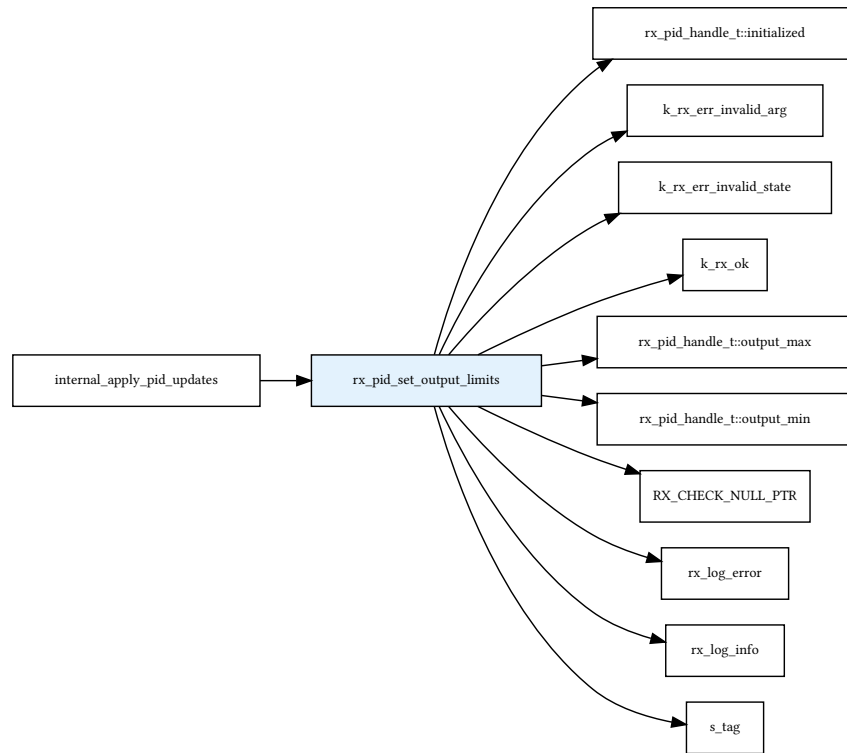


Figure 1063: Call graph for rx_pid_set_output_limits

Defined in `libs/rx_pid/inc/rx_pid.h:645`

Since Version 1.0.0

—

rx_pid_set_integral_limits

```
rx_err_t rx_pid_set_integral_limits(rx_pid_handle_t *handle, float integral_min,
float integral_max)
```

Update PID integral limits at runtime (anti-windup).

Allows changing the integral term saturation limits without reinitializing.

Update PID integral limits at runtime (anti-windup).

Modifies the integral accumulator saturation limits (`integral_min`, `integral_max`) without reinitializing the controller. These limits prevent integral windup during prolonged actuator saturation. Important: This function IMMEDIATELY clamps the current integral value to the new limits, unlike `rx_pid_set_output_limits()`.

Update Steps:

1. Validate handle pointer (NULL check)
2. Check initialization state
3. Validate `integral_max > integral_min` (strict inequality)

4. Store new integral limits
5. Immediately clamp current integral to new limits (key difference from output limits)
6. Log update event

Anti-Windup Strategy: Integral windup occurs when the controller output saturates but the integral term continues to accumulate error. This causes overshoot and sluggish response. By limiting the integral accumulator to [integral_min, integral_max], windup is prevented.

Typical Settings:

- Rule of Thumb: Set integral limits to 50-80% of output range
- Symmetric: integral_min = -integral_max (most common for bidirectional actuators)
- Asymmetric: Different limits for forward/reverse (e.g., heating vs. cooling)

After successful call: integral_min <= handle->integral <= integral_max

handle->kp, ki, kd unchanged

handle->output_min/max unchanged

handle->prev_error unchanged

Recommended Practice: Update integral limits when output limits change to maintain 50-80% ratio

1.0.0

Parameters:

Direction	Name	Description
in	handle	Pointer to initialized PID handle. Must not be nullptr.
in	integral_min	New minimum integral limit.
in	integral_max	New maximum integral limit.
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) integral_min and integral_max fields will be updated IMPORTANT: Current integral value will be clamped to new limits
in	integral_min	New minimum integral accumulator limit (anti-windup lower bound) Must be < integral_max (validated - strict inequality) Units same as output (since I_term = Ki x integral) Example: -50.0f for motor control (50% of output range) Prevents excessive negative integral buildup
in	integral_max	New maximum integral accumulator limit (anti-windup upper bound) Must be > integral_min (validated - strict inequality) Units same as output Example: +50.0f for motor control (50% of output range) Prevents excessive positive integral buildup

Returns: rx_err_t Error code indicating limit update result

Value	Description
k_rx_ok	Integral limits updated successfully, current integral clamped
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)

k_rx_err_invalid_arg	integral_max <= integral_min (not strictly greater)
----------------------	---

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Pre: integral_max > integral_min (strictly greater, not equal)

Post: handle->integral_min == integral_min (on success)

Post: handle->integral_max == integral_max (on success)

Post: handle->integral in integral_min, integral_max (immediately clamped)

Post: Gains and output limits unchanged (kp, ki, kd, output_min/max)

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 220 ns @ 240 MHz (55 cycles) - includes validation and clamping

Note: Immediate Clamping: Unlike rx_pid_set_output_limits(), this immediately clamps integral

Note: Windup Prevention: Limits should be 50-80% of output range for effective anti-windup

Warning: Integral Clamping: Existing integral value is IMMEDIATELY clamped to new limits - may cause transient in next output

Warning: Proportional to Output: Integral limits should scale with output limits for consistent behavior] **Example: Update Integral Limits with Output Limits:** #include "rx_pid.h"

```
void enter_safety_mode(void){ const float k_safety_output_limit=50.0f;
const float k_safety_integral_limit=k_safety_output_limit*0.6f;

rx_pid_set_output_limits(&motor_pid,-k_safety_output_limit,k_safety_output_limit);
rx_pid_set_integral_limits(&motor_pid,-k_safety_integral_limit,k_safety_integral_limit);
rx_log_info("MOTOR","Safety mode: limits reduced"); }

Example: Aggressive Anti-Windup for Fast Response: void reduce_overshoot(void)
{ const float k_tight_integral_limit=30.0f;

rx_err_t err=rx_pid_set_integral_limits(&motor_pid,-k_tight_integral_limit,k_tight_integral_limit);
if(err==k_rx_ok){ rx_log_info("PID","Tighter anti-windup limits applied"); } }
```

Example: Asymmetric Limits (Heater Control): void configure_heater_pid(void)

```
{ rx_pid_set_output_limits(&heater_pid, 0.0f, 100.0f);
```

```
rx_pid_set_integral_limits(&heater_pid, 0.0f, 50.0f); }
```

Example: Dynamic Limit Adjustment: void adjust_integral_limits_for_load(float load_percentage)

```
{ float integral_limit;
```

```
if (load_percentage > 80.0f) { integral_limit = 30.0f; } else { integral_limit = 60.0f; }
```

```
rx_pid_set_integral_limits(&motor_pid, -integral_limit, integral_limit); }
```

Example: Error Handling: rx_err_t err = rx_pid_set_integral_limits(&pid, -40.0f, 40.0f);

```
switch (err) { case k_rx_ok: rx_log_info("PID", "Integral limits updated and current integral clamped");
```

```
break; case k_rx_err_null_ptr: rx_log_error("PID", "null ptr handle pointer"); break;
```

```
case k_rx_err_invalid_state: rx_log_error("PID", "PID not initialized"); break; case k_rx_err_invalid_arg:
```

```
rx_log_error("PID", "Invalid limits (max <= min)"); break; }
```

Performance Analysis:: Execution time: 220 ns @ 240 MHz (55 cycles) Memory: Modifies 12 bytes (3 floats: integral_min, integral_max, integral) Stack usage: 0 bytes (parameters passed by value/pointer) Validation overhead: 3 checks + 1 clamp operation

Anti-Windup Effectiveness Table:: Integral Limit (% of Output Range) Windup Prevention

Response Speed Overshoot

30-40% Aggressive Slow Minimal

50-60% Moderate (recommended) Balanced Low

70-80% Light Fast Moderate

90-100% Minimal Very Fast High

Limit Validation Table:: Validation Condition Error Code

nullptr handle != nullptr k_rx_err_null_ptr

Initialized handle->initialized == true k_rx_err_invalid_state

Strictly greater integral_max > integral_min k_rx_err_invalid_arg

Key Difference from rx_pid_set_output_limits(): Feature rx_pid_set_output_limits()

rx_pid_set_integral_limits()

Clamps current value? NO (affects next computation only) YES (immediately clamps integral)

When to use? Change actuator constraints Tune anti-windup behavior

Typical ratio to output N/A (defines actuator range) 50-80% of output range

NASA Power of 10 Compliance:: Rule 4: Function is 19 lines (well under 60-line limit) Rule 5: 3 validation checks (nullptr, init, range) + 1 postcondition (clamping)

See also: rx_pid_init() to initialize controller with initial integral limits, rx_pid_set_output_limits() to update output saturation limits (usually updated together), rx_pid_compute() uses integral limits to prevent windup during computation, internal_clamp() helper function used to clamp integral



Figure 1064: Call graph for rx_pid_set_integral_limits

Defined in `libs/rx_pid/inc/rx_pid.h:662`

Since Version 1.0.0

—

rx_pid.c

PID Controller Implementation for Closed-Loop Motor Control.

Includes:

- `"rx_pid.h"`
- `<math.h>`
- `<string.h>`
- `"rx_check.h"`
- `"rx_log.h"`

s_tag

```
const char* s_tag
```

—

internal_clamp

```
static float internal_clamp(const float value, const float min, const float max)
```

Clamp floating-point value to specified min/max range (saturation function).

Implements a saturation function that constrains a value to a specified range. This is a fundamental building block for anti-windup and output limiting in PID control.

Algorithm Steps:

1. If value < min, return min (lower saturation)
2. If value > max, return max (upper saturation)
3. Otherwise, return value unchanged (linear region)

Transfer Function:

For all valid inputs: $\min \leq \text{clamp}(\text{value}, \min, \max) \leq \max$

Parameters:

Direction	Name	Description
in	value	Value to be clamped Can be any finite floating-point value NaN/Inf handling: NaN propagates, Inf may saturate to min/max
in	min	Minimum allowed value (lower saturation limit) Must be $\leq \max$ for correct operation Example: -100.0f for motor reverse limit
in	max	Maximum allowed value (upper saturation limit) Must be $\geq \min$ for correct operation Example: +100.0f for motor forward limit

Returns: float Clamped value in range [min, max]

Value	Description
min	if input value < min (lower saturation active)
max	if input value > max (upper saturation active)
value	if $\min \leq \text{value} \leq \max$ (linear pass-through)

Pre: $\min \leq \max$ (caller responsibility - not validated for performance)

Pre: value should be finite (NaN/Inf not explicitly handled)

Post: Return value in min, max (guaranteed if preconditions met)

Note: Performance: Inline function, 3 comparisons worst-case (15 cycles @ 240 MHz = 62 ns)

Note: Thread Safety: Reentrant - no shared state, pure function

Note: Numerical Stability: Direct comparisons, no arithmetic (no rounding errors)

Warning: NaN Handling: NaN comparisons always return false, so NaN input may not saturate correctly

Warning: min > max: Results are undefined if min > max (saturates to min)

Example: Basic Usage: `floatduty=internal_clamp(raw_output,-100.0f,100.0f);`

Example: Anti-Windup Integral Limiting: `handle->integral+=error*dt; handle->integral=internal_clamp(handle->integral, handle->integral_min, handle->integral_max);`

Example: Edge Cases: `internal_clamp(50.0f,-100.0f,100.0f); internal_clamp(-150.0f,-100.0f,100.0f); internal_clamp(200.0f,-100.0f,100.0f); internal_clamp(0.0f,0.0f,0.0f);`

Performance Analysis:: Best case: 1 comparison (value already in range) Average case: 2 comparisons Worst case: 2 comparisons Execution time: 62 ns @ 240 MHz (FPU comparison latency) Stack usage: 0 bytes (parameters in registers)

NASA Power of 10 Compliance:: Rule 4: Function is 8 lines (well under 60-line limit) Rule 5: Preconditions documented (min <= max) Rule 9: No pointer dereferencing (value types only)

See also: `rx_pid_compute()` Uses this for integral and output clamping, `rx_pid_set_integral_limits()` Immediately applies clamping to current integral

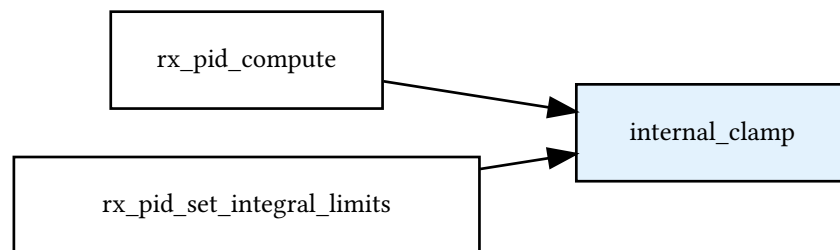


Figure 1065: Call graph for internal_clamp

Defined in `libs/rx_pid/src/rx_pid.c:289`

Since Version 1.0.0

—

rx_pid_init

```
rx_err_t rx_pid_init(rx_pid_handle_t *handle, const rx_pid_config_t *config)
```

Initialize a PID controller instance with configuration parameters.

Initialize a PID controller instance.

Configures a PID controller handle with gains, output limits, and integral anti-windup limits. This function must be called before any other PID operations can be performed. Initializes internal state (integral, prev_error) to zero for a clean start.

Initialization Steps:

1. Validate handle and config pointers (NULL checks)
2. Check if already initialized (prevent double-initialization)

3. Validate configuration constraints:

`output_max > output_min` `integral_max > integral_min`

4. Zero out handle structure (clear any previous state)

5. Copy configuration parameters to handle

6. Initialize state variables (`integral = 0`, `prev_error = 0`)

7. Set initialized flag to true

After successful initialization: `handle->output_max > handle->output_min`

After successful initialization: `handle->integral_max > handle->integral_min`

Gain Validation: Function does NOT validate gain values (`kp`, `ki`, `kd`) - negative or NaN/Inf gains will be accepted but cause incorrect behavior in `rx_pid_compute()`. Use `rx_pid_set_gains()` for validated gain updates.

1.0.0

Parameters:

Direction	Name	Description
out	handle	Pointer to PID controller handle structure Must not be NULL (validated) Will be zeroed and initialized with config parameters Must remain valid for lifetime of controller use Typically statically allocated: <code>static rx_pid_handle_t motor_pid;</code>
in	config	Pointer to PID configuration structure Must not be NULL (validated) Can be stack-allocated (copied to handle, not retained) Fields validated: <code>output_max > output_min</code> , <code>integral_max > integral_min</code> See <code>rx_pid_config_t</code> documentation for field details

Returns: `rx_err_t` Error code indicating initialization result

Value	Description
<code>k_rx_ok</code>	Initialization successful, controller ready for use
<code>k_rx_err_null_ptr</code>	handle or config pointer is nullptr
<code>k_rx_err_invalid_state</code>	Controller already initialized (call <code>rx_pid_deinit</code> first)
<code>k_rx_err_invalid_arg</code>	Configuration validation failed: <code>output_max <= output_min</code> , OR <code>integral_max <= integral_min</code>

Pre: handle must point to allocated `rx_pid_handle_t` structure

Pre: config must point to valid `rx_pid_config_t` with properly set fields

Pre: handle must not be already initialized (or call `rx_pid_deinit` first)

Pre: `config->output_max > config->output_min` (strictly greater)

Pre: `config->integral_max > config->integral_min` (strictly greater)

Post: handle->initialized == true (on success)

Post: handle->integral == 0.0f (clean initial state)

Post: handle->prev_error == 0.0f (clean initial state)

Post: Configuration parameters copied to handle

Post: Controller ready for rx_pid_compute() calls

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Memory: No dynamic allocation - all state in provided handle structure

Note: Re-initialization: Must call rx_pid_deinit() before re-initializing same handle

Note: Performance: 625 ns @ 240 MHz (150 cycles) - one-time cost

Warning: State Persistence: Handle must remain valid for entire controller lifetime

Warning: Configuration Lifetime: Config can be stack-allocated (copied, not retained)

Warning: Double Init: Attempting to init already-initialized handle returns error] **Example:**
Motor Velocity Control Initialization: #include "rx_pid.h"

```
static rx_pid_handle_t motor_pid;

void motor_init(void)
{ rx_pid_config_t config = { .kp=0.286f, .ki=8.01f, .kd=0.0f, .output_min=-100.0f, .output_max=100.0f, .integral_min=-50.0f, .integral_max=50.0f };
  rx_err_t err = rx_pid_init(&motor_pid, &config); if (err == k_rx_ok)
  { rx_log_info("MOTOR", "PID initialized successfully"); }
  else { rx_log_error_val("MOTOR", "PID init failed", err); } }
```

Example: Error Handling: rx_pid_handle_t pid; rx_pid_config_t config = {...};

```
rx_err_t err = rx_pid_init(&pid, &config); switch(err) { case k_rx_ok: break; case k_rx_err_null_ptr:
rx_log_error("PID", "null ptr init"); break; case k_rx_err_invalid_state:
rx_log_warn("PID", "Already initialized-deinit first"); rx_pid_deinit(&pid); rx_pid_init(&pid, &config);
break; case k_rx_err_invalid_arg: rx_log_error("PID", "Invalid config (check output/integral limits)");
break; }
```

Example: Configuration Validation Failures:

```
rx_pid_config_t bad_config1 = { .output_min=100.0f, .output_max=50.0f, };
rx_pid_init(&pid, &bad_config1);
```

```
rx_pid_config_tbad_config2={.output_min=-100.0f,.output_max=100.0f,.integral_min=50.0f,.integral_max=50.0f,};  
rx_pid_init(&pid,&bad_config2);
```

Performance Analysis:: Execution time: 625 ns @ 240 MHz (150 cycles) Memory: 40 bytes (sizeof(rx_pid_handle_t)) Stack usage: 0 bytes (parameters passed by pointer) Called: Once per controller instance at startup Critical path: No (one-time initialization, not in control loop)

Parameter Validation Table:: Parameter Validation Error Code

```
handle != nullptr k_rx_err_null_ptr
```

```
config != nullptr k_rx_err_null_ptr
```

```
handle->initialized
```

false

```
k_rx_err_invalid_state
```

```
config->output_max > config->output_min k_rx_err_invalid_arg
```

```
config->integral_max > config->integral_min k_rx_err_invalid_arg
```

NASA Power of 10 Compliance:: Rule 4: Function is 29 lines (well under 60-line limit) Rule 5: 5 validation checks (NULLx2, initialized, output limits, integral limits) Rule 7: All return values must be checked by caller

See also: rx_pid_config_t for configuration structure details, rx_pid_deinit() to deinitialize and allow re-initialization, rx_pid_compute() to perform PID computation after initialization, rx_pid_reset() to clear state without deinitialization, rx_pid_set_gains() to update gains after initialization, matlab/pid_design_velocity.m for gain tuning methodology

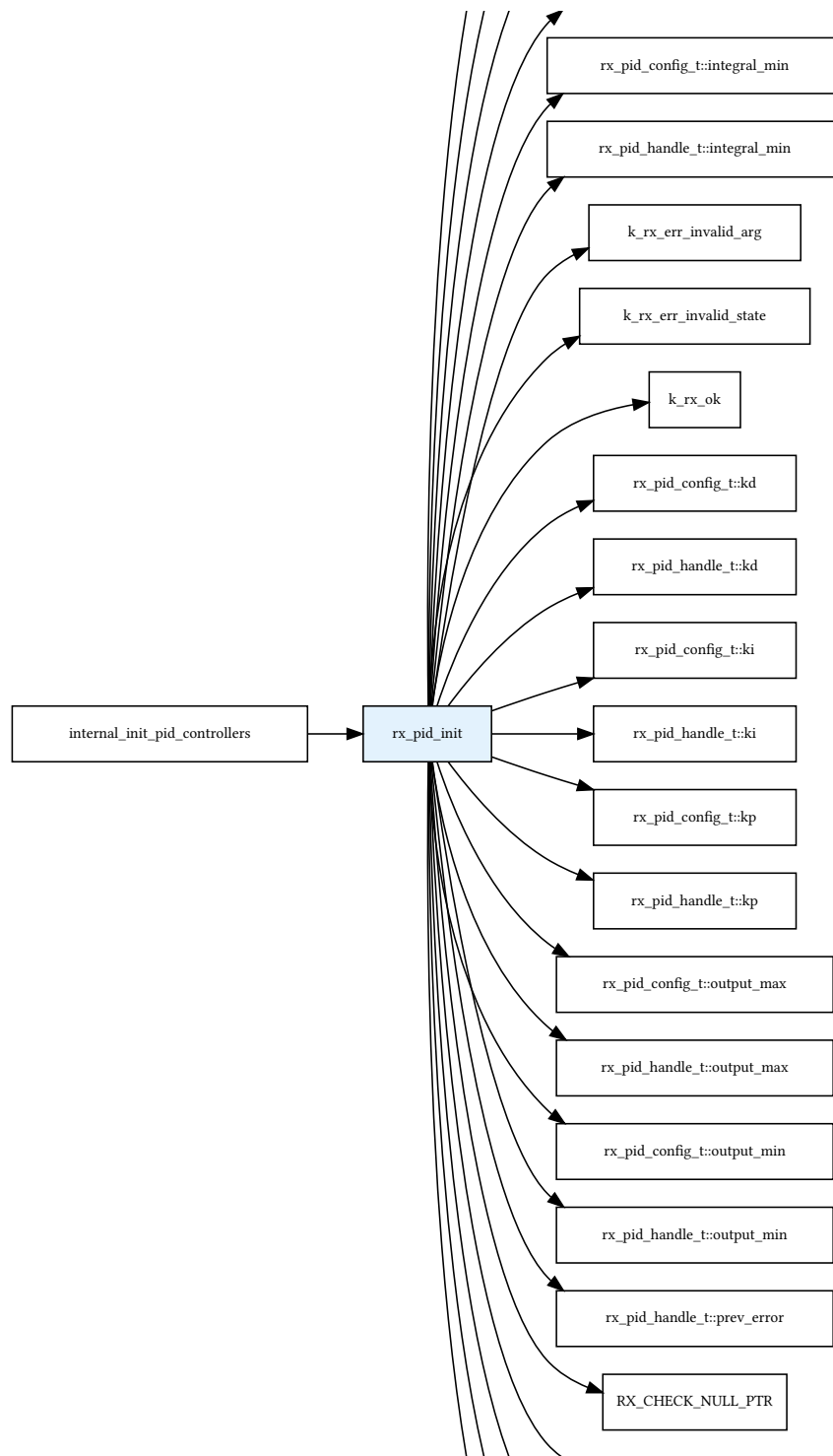


Figure 1066: Call graph for rx_pid_init

Defined in `libs/rx_pid/src/rx_pid.c:511`

Since Version 1.0.0

—

rx_pid_deinit

```
rx_err_t rx_pid_deinit(rx_pid_handle_t *handle)
```

Deinitialize PID controller and clear all state.

Deinitialize PID controller and clear state.

Deinitializes a PID controller handle, clearing all configuration parameters and internal state. After deinitialization, the handle can be reinitialized with different parameters or safely deallocated (if dynamically allocated, though not recommended).

Deinitialization Steps:

1. Validate handle pointer (NULL check)
2. Check if currently initialized (prevent double-deinitialization)
3. Zero entire handle structure (memset to 0)
4. Log deinitialization event

Use Cases:

- Controller shutdown before system reset
- Switching between different control modes
- Clearing state before reconfiguration with different gains/limits
- Testing/debugging (force clean state)

1.0.0

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be currently initialized (validated) Will be zeroed on success (all fields set to 0) After deinitialization: handle->initialized == false

Returns: rx_err_t Error code indicating deinitialization result

Value	Description
k_rx_ok	Deinitialization successful, handle cleared and ready for re-init
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (already deinitialized or never initialized)

Pre: handle must not be nullptr

Pre: handle->initialized must be true (controller must be initialized first)

Post: handle->initialized == false (on success)

Post: All handle fields zeroed: gains, limits, integral, prev_error == 0

Post: Handle can be safely reinitialized with rx_pid_init()

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 125 ns @ 240 MHz (30 cycles) - memset is fast

Note: Re-initialization: After deinit, handle can be reinitialized with rx_pid_init()

Note: Memory Safety: Does not free memory (handle is caller-managed)

Warning: Active Control Loops: Do not deinitialize while control loop is active - stop control loop first

Warning: State Loss: All PID state (integral, prev_error) is lost - cannot be recovered

Example: Clean Shutdown: motor_control_active=false; tx_thread_sleep(10);

```
rx_err_t err=rx_pid_deinit(&motor_pid); if(err!=k_rx_ok)
{ rx_log_info("MOTOR","PIDdeinitializedsuccessfully"); }
```

Example: Reconfiguration Pattern: rx_pid_deinit(&pid);

```
rx_pid_config_t position_config={ .kp=1.5f, .ki=0.2f, .kd=0.1f, }; rx_pid_init(&pid,&position_config);
```

Example: Error Handling: rx_err_t err=rx_pid_deinit(&pid); switch(err){ case k_rx_ok: break; case k_rx_err_null_ptr: rx_log_error("PID","Cannot deinit null ptr handle"); break; case k_rx_err_invalid_state: rx_log_warn("PID","Already deinitialized or never initialized"); break; }

Performance Analysis:: Execution time: 125 ns @ 240 MHz (30 cycles) Memory: Clears 40 bytes (sizeof(rx_pid_handle_t)) Stack usage: 0 bytes (parameters passed by pointer) Called: Once per controller instance at shutdown or reconfiguration

NASA Power of 10 Compliance:: Rule 4: Function is 12 lines (well under 60-line limit) Rule 5: 2 validation checks (nullptr, initialization state)

See also: rx_pid_init() to (re)initialize controller after deinitialization, rx_pid_reset() to clear state WITHOUT deinitialization (preserves configuration)

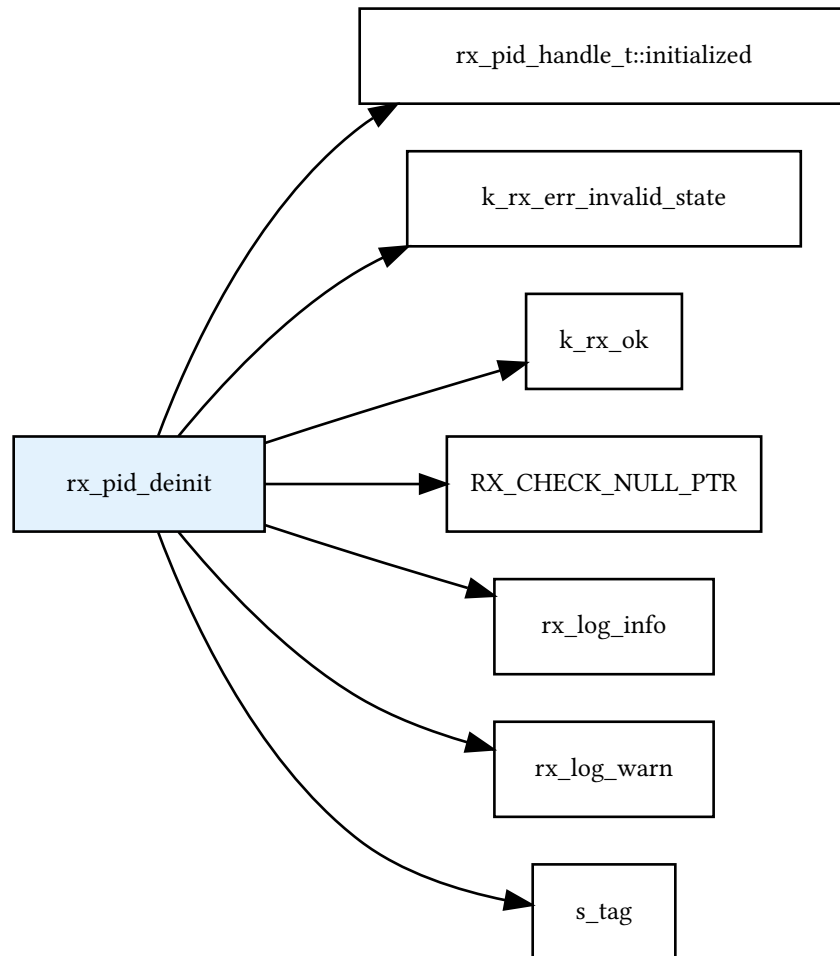


Figure 1067: Call graph for rx_pid_deinit

Defined in `libs/rx_pid/src/rx_pid.c:661`

Since Version 1.0.0

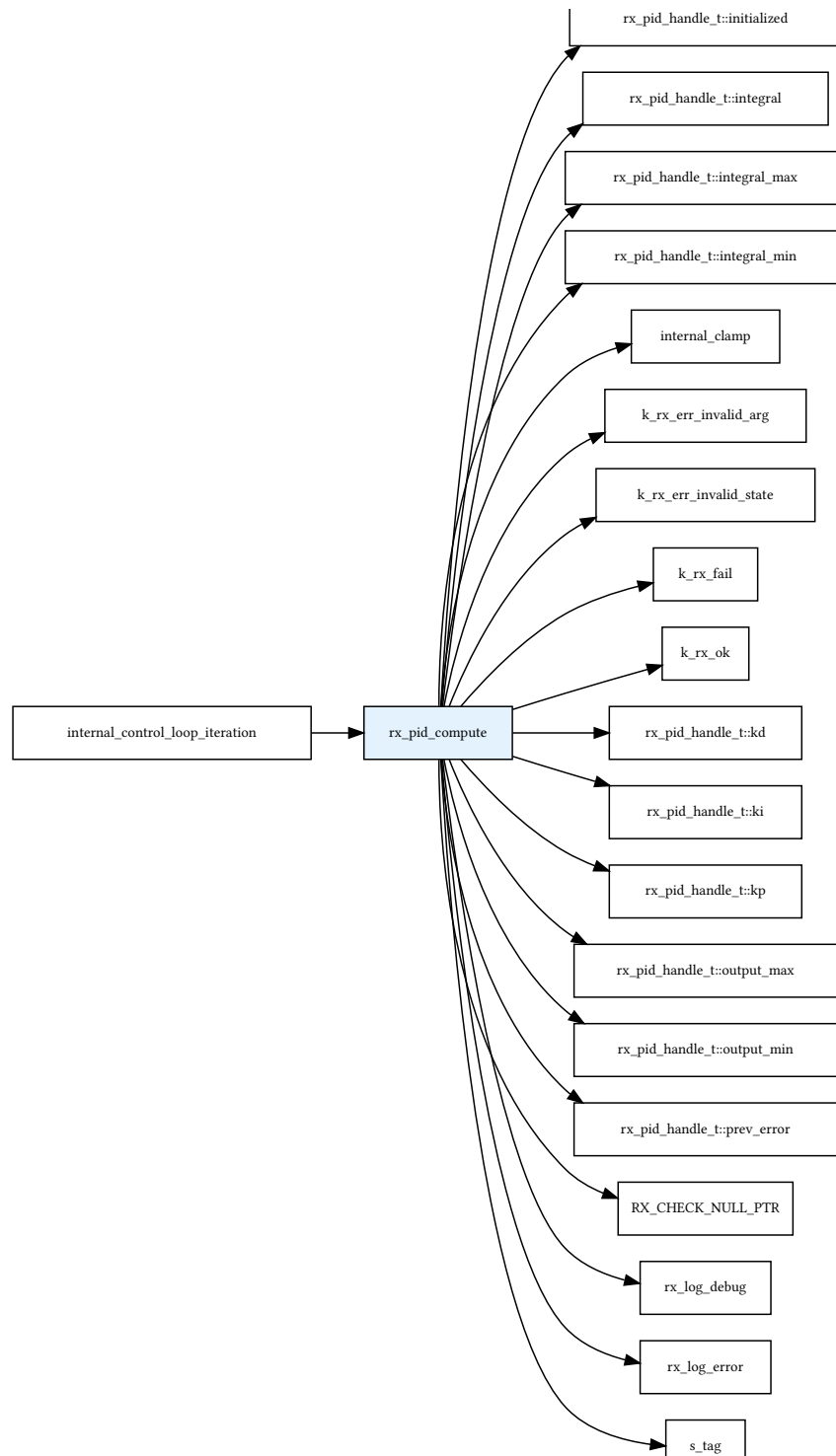
—

rx_pid_compute

```
rx_err_t rx_pid_compute(rx_pid_handle_t *handle, const float setpoint, const
float measured, const float dt, float *output)
```

Compute PID control output for current sample.

Core PID computation function implementing the discrete parallel-form PID algorithm with integral anti-windup and output saturation. Call this function at a fixed sample rate (e.g., 250 Hz for motor control) to generate control outputs.

Figure 1068: Call graph for `rx_pid_compute`

Defined in `libs/rx_pid/src/rx_pid.c:678`

—

rx_pid_reset

```
rx_err_t rx_pid_reset(rx_pid_handle_t *handle)
```

Reset PID controller internal state without clearing configuration.

Reset PID controller internal state.

Clears the integral accumulator and previous error state while preserving all configuration parameters (gains, limits). This is useful when changing setpoints, recovering from disturbances, or preventing integral windup after prolonged saturation.

Reset Operations:

1. Validate handle pointer (NULL check)
2. Check initialization state
3. Clear integral accumulator (integral = 0.0f)
4. Clear previous error (prev_error = 0.0f)
5. Log reset event

Configuration Preserved (NOT cleared):

- Gains: kp, ki, kd
- Output limits: output_min, output_max
- Integral limits: integral_min, integral_max
- Initialization flag: initialized

When to Use:

- Large setpoint changes: Prevents integral windup from old error
- Mode transitions: Switching between position/velocity control
- After saturation: Clear accumulated error after prolonged output limiting
- Disturbance recovery: Reset after external disturbances (collisions, stalls)
- System startup: Ensure clean initial state before control begins

Gains remain unchanged: kp, ki, kd

Limits remain unchanged: output_min/max, integral_min/max

Initialization state remains true

1.0.0

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) integral and prev_error fields will be zeroed Configuration (gains, limits) remains unchanged

Returns: rx_err_t Error code indicating reset result

Value	Description
k_rx_ok	State reset successful, controller ready with clean state

k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Post: handle->integral == 0.0f (on success)

Post: handle->prev_error == 0.0f (on success)

Post: Configuration parameters unchanged (gains, limits)

Post: Controller ready for rx_pid_compute() with clean state

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 125 ns @ 240 MHz (30 cycles) - simple field assignment

Note: Next Computation: First rx_pid_compute() after reset will have derivative = 0

Note: Difference from Deinit: Preserves configuration, only clears state

Warning: Active Control: Do not reset during critical control phases - causes output discontinuity

Warning: Derivative Spike: Reset causes prev_error = 0, so next derivative term may spike if error is large] **Example: Setpoint Change:** #include "rx_pid.h"

```
void change_target_speed(float new_target_rpm) { rx_err_t err = rx_pid_reset(&motor_pid); if (err != k_rx_ok) { rx_log_error_val("MOTOR", "PID reset failed", err); return; }
```

```
target_rpm = new_target_rpm; rx_log_info_val("MOTOR", "New target RPM", (uint32_t) new_target_rpm);
```

```
}
```

Example: Disturbance Recovery: if (motor_current > k_stall_threshold_ma)

```
{ motor_set_duty_cycle(0.0f);
```

```
rx_pid_reset(&motor_pid);
```

```
tx_thread_sleep(100);
```

```
motor_control_active = true; }
```

Example: Startup Sequence: `void motor_system_init(void){ rx_pid_config_t config={...};
rx_pid_init(&motor_pid,&config);
rx_pid_reset(&motor_pid);
start_motor_control_task(); }`

Example: Mode Switching: `void motor_idle(void){ rx_pid_reset(&motor_pid);
target_rpm=0.0f;
}`

Performance Analysis:: Execution time: 125 ns @ 240 MHz (30 cycles) Memory: Modifies 8 bytes (2 floats: integral, prev_error) Stack usage: 0 bytes (parameters passed by pointer) Called: As needed (setpoint changes, disturbances, mode transitions)

Reset vs. Deinit Comparison:: Operation rx_pid_reset() rx_pid_deinit()

Clear integral YES YES

Clear prev_error YES YES

Clear gains NO YES

Clear limits NO YES

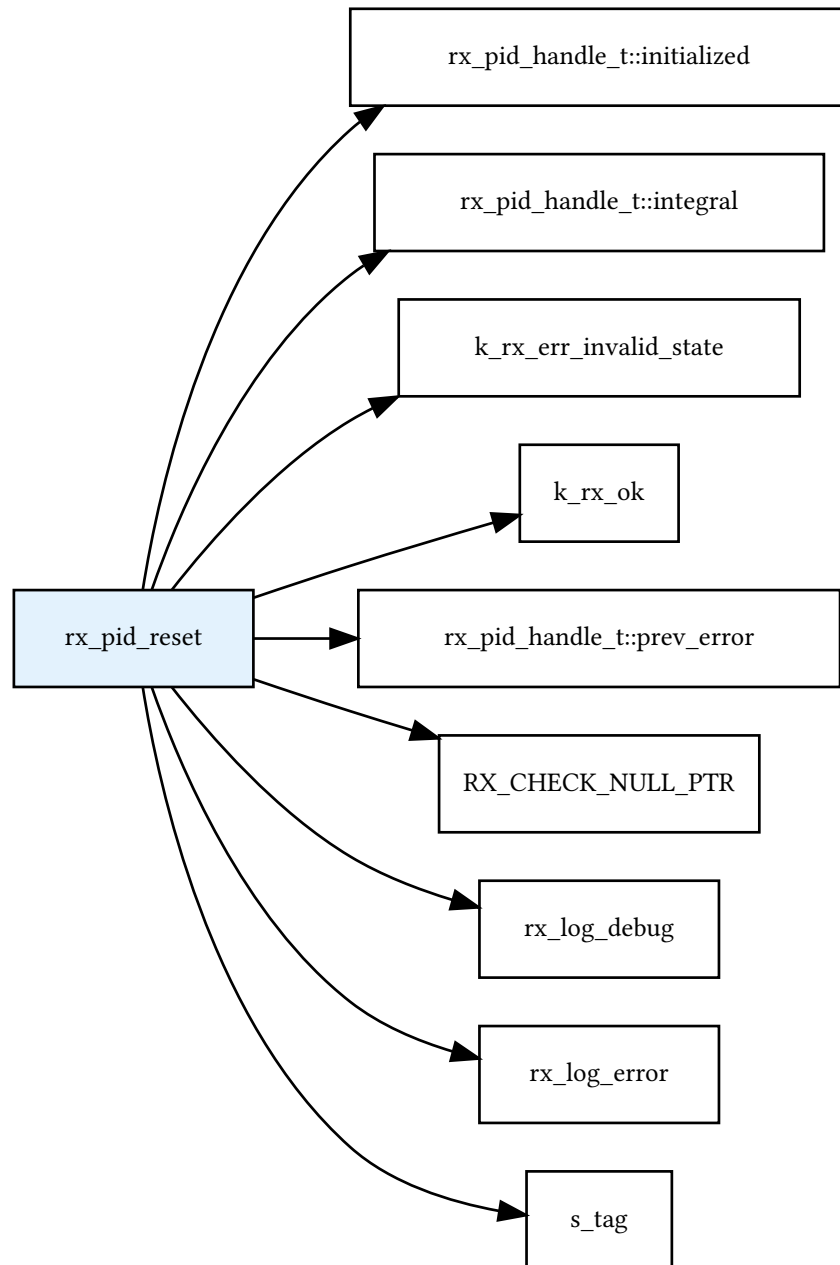
Clear initialized flag NO YES

Can compute after YES NO (must reinit)

Use case State cleanup Full shutdown

NASA Power of 10 Compliance:: Rule 4: Function is 12 lines (well under 60-line limit) Rule 5: 2 validation checks (nullptr, initialization state)

See also: rx_pid_init() to initialize controller, rx_pid_deinit() to fully deinitialize (clears configuration too), rx_pid_compute() to perform computation after reset, rx_pid_set_gains() to change gains without resetting state

Figure 1069: Call graph for `rx_pid_reset`

Defined in `libs/rx_pid/src/rx_pid.c:903`

Since Version 1.0.0

—

rx_pid_set_gains

```
rx_err_t rx_pid_set_gains(rx_pid_handle_t *handle, const float kp, const float
ki, const float kd)
```

Update PID gains at runtime without clearing state.

Update PID gains at runtime.

Allows runtime modification of PID gains (Kp, Ki, Kd) without reinitializing the controller. Internal state (integral, prev_error) is preserved, enabling smooth gain transitions during operation. Useful for adaptive control, auto-tuning, and gain scheduling applications.

Validation Steps:

1. nullptr check (handle)
2. Initialization state check
3. Validate gains are non-negative (kp >= 0, ki >= 0, kd >= 0)
4. Validate gains are finite (not NaN or Inf)
5. Store new gains
6. Verify storage success (post-condition check per NASA Rule 5)

State Preserved:

- integral accumulator (NOT cleared)
- prev_error (NOT cleared)
- Output limits (output_min, output_max)
- Integral limits (integral_min, integral_max)

handle->integral unchanged (state preserved for smooth transition)

handle->prev_error unchanged

handle->output_min/max unchanged

handle->integral_min/max unchanged

Auto-Tuning: When implementing auto-tuners, validate new gains before applying

1.0.0

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) Gains will be updated on success State (integral, prev_error) preserved
in	kp	New proportional gain (K _p) Must be >= 0.0 (validated - negative gains cause instability) Must be finite (validated - NaN/Inf causes undefined behavior) Units: output / error Example: 0.286 for motor velocity control Higher = faster response, risk of overshoot
in	ki	New integral gain (K _i) Must be >= 0.0 (validated) Must be finite (validated) Units: output / (error * s) Example: 8.01 for motor velocity control Higher = faster steady-state error elimination, risk of windup

in	kd	New derivative gain (K_d) Must be ≥ 0.0 (validated) Must be finite (validated) Units: output / (error/s) Example: 0.0 (disabled for noisy encoder feedback) Higher = more damping, risk of noise amplification
----	----	---

Returns: rx_err_t Error code indicating gain update result

Value	Description
k_rx_ok	Gains updated successfully, controller ready with new gains
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)
k_rx_err_invalid_arg	One or more gains are negative or non-finite (NaN/Inf)
k_rx_fail	Gain storage verification failed (should never happen)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Pre: $kp \geq 0.0$ and isfinite(kp)

Pre: $ki \geq 0.0$ and isfinite(ki)

Pre: $kd \geq 0.0$ and isfinite(kd)

Post: handle->kp == kp (on success)

Post: handle->ki == ki (on success)

Post: handle->kd == kd (on success)

Post: Internal state preserved (integral, prev_error unchanged)

Post: Controller ready for rx_pid_compute() with new gains

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 250 ns @ 240 MHz (60 cycles) - includes validation

Note: Smooth Transition: Preserving state enables bumpless gain changes

Note: Negative Gains: Not allowed - would cause positive feedback (instability)

Warning: Stability: Large gain changes during operation may cause transient oscillations

Warning: Integral Term: Existing integral may be incompatible with new Ki - consider rx_pid_reset() if changing Ki significantly

Warning: Zero Gains: $kp = ki = kd = 0$ results in zero output (valid but useless)] **Example:**
Manual Tuning at Runtime: #include "rx_pid.h"

```
voidincrease_responsiveness(void){ floatkp=motor_pid.kp; floatki=motor_pid.ki;
floatkd=motor_pid.kd;
```

```
kp*=1.1f;
```

```
rx_err_terr=rx_pid_set_gains(&motor_pid,kp,ki,kd); if(err!=k_rx_ok)
{ rx_log_info("TUNE","Kpincreasedsuccessfully"); }
else{ rx_log_error_val("TUNE","Gainupdatefailed",err); } }
```

Example: Ziegler-Nichols Auto-Tuner: voidauto_tune_pid(rx_pid_handle_t*pid)
{ floatku=measure_critical_gain(); floatpu=measure_oscillation_period();

```
floatkp=0.6f*ku; floatki=1.2f*ku/pu; floatkd=0.075f*ku*pu;
```

```
if(kp>0.0f&&ki>0.0f&&kd>=0.0f){ rx_err_terr=rx_pid_set_gains(pid,kp,ki,kd); if(err!=k_rx_ok)
{ rx_log_info("TUNE","Auto-tunedgainsapplied"); } }else{ rx_log_error("TUNE","Invalidauto-
tunedgains"); } }
```

Example: Gain Scheduling (Velocity-Dependent):

```
voidupdate_gains_for_velocity(floatcurrent_rpm){ floatkp,ki,kd;
```

```
if(current_rpm<50.0f){ kp=0.4f; ki=10.0f; kd=0.0f; }else{ kp=0.2f; ki=5.0f; kd=0.0f; }
```

```
rx_pid_set_gains(&motor_pid,kp,ki,kd); }
```

Example: Error Handling: rx_err_terr=rx_pid_set_gains(&pid,0.5f,2.0f,0.1f);

```
switch(err){ casek_rx_ok: rx_log_info("PID","Gainsupdatedsuccessfully"); break;
```

```
casek_rx_err_null_ptr: rx_log_error("PID","nullptrhandlepointer"); break;
```

```
casek_rx_err_invalid_state: rx_log_error("PID","PIDnotinitialized"); break; casek_rx_err_invalid_arg:
```

```
rx_log_error("PID","Invalidgains(negativeorNaN/Inf"); break; casek_rx_fail:
```

```
rx_log_error("PID","Gainstorageverificationfailed"); break; }
```

Performance Analysis:: Execution time: 250 ns @ 240 MHz (60 cycles) Memory: Modifies 12 bytes
(3 floats: kp, ki, kd) Stack usage: 0 bytes (parameters passed by value/pointer) Validation overhead: 4
checks (nullptr, init, range, finite)

Gain Validation Table:: Validation Condition Error Code

```
nullptr handle != nullptr k_rx_err_null_ptr
```

```
Initialized handle->initialized == true k_rx_err_invalid_state
```

```
Non-negative kp >= 0 && ki >= 0 && kd >= 0 k_rx_err_invalid_arg
```

```
Finite isfinite(kp/ki/kd) k_rx_err_invalid_arg
```

Storage handle->kp == kp (etc.) k_rx_fail

NASA Power of 10 Compliance: Rule 4: Function is 29 lines (well under 60-line limit) Rule 5: 4 precondition checks + 1 postcondition check (exceeds minimum 2)

See also: rx_pid_init() to initialize controller with initial gains, rx_pid_reset() to clear state if changing Ki significantly, rx_pid_compute() uses the updated gains in next computation, matlab/pid_design_velocity.m for gain tuning methodology

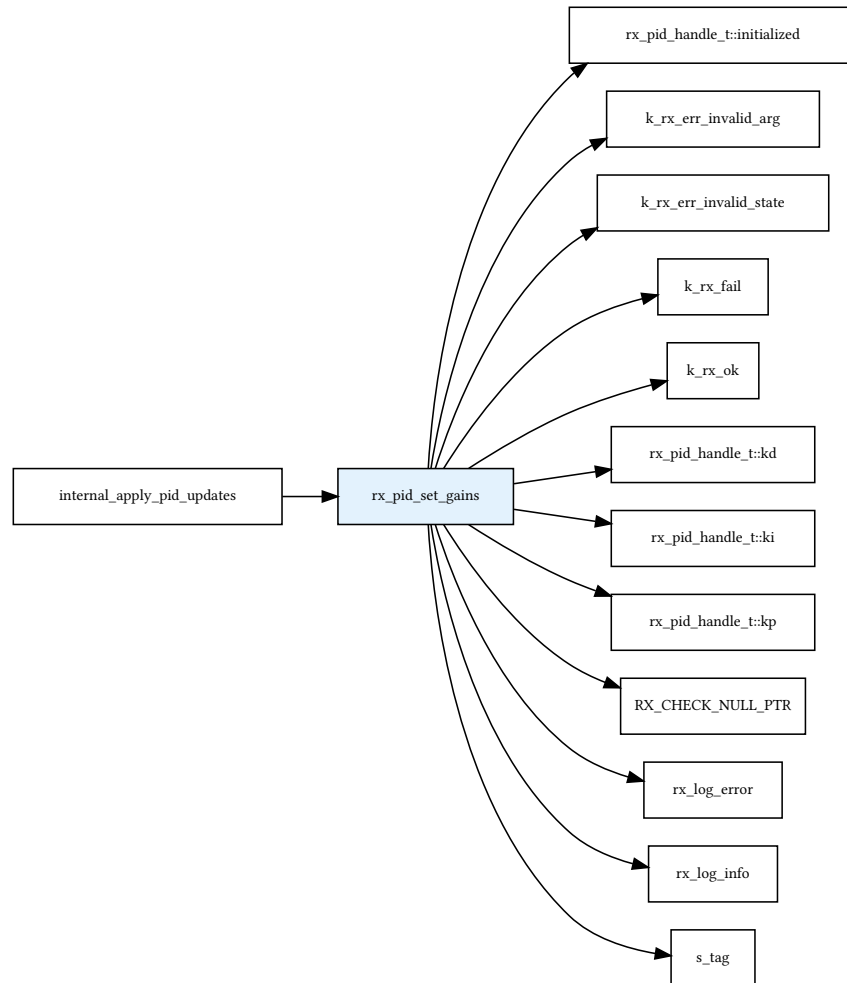


Figure 1070: Call graph for rx_pid_set_gains

Defined in `libs/rx_pid/src/rx_pid.c:1131`

Since Version 1.0.0

—

rx_pid_set_output_limits

```
rx_err_t rx_pid_set_output_limits(rx_pid_handle_t *handle, const float
output_min, const float output_max)
```

Update PID output saturation limits at runtime.

Update PID output limits at runtime.

Modifies the output saturation limits (output_min, output_max) without reinitializing the controller. The output limits define the range to which the PID output is clamped before being applied to the actuator. Useful for adapting to different operating modes or actuator constraints.

Update Steps:

1. Validate handle pointer (NULL check)
2. Check initialization state
3. Validate output_max > output_min (strict inequality)
4. Store new limits
5. Log update event

Note: This function does NOT clamp the current output or state. The new limits take effect on the next call to rx_pid_compute().

handle->integral unchanged

handle->kp, ki, kd unchanged

handle->integral_min/max unchanged

Integral Limits: Consider updating integral_min/max proportionally when changing output limits

1.0.0

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) output_min and output_max fields will be updated State and gains preserved
in	output_min	New minimum output limit (lower saturation bound) Must be < output_max (validated - strict inequality) Units depend on application (% duty cycle, voltage, current, etc.) Example: -100.0f for full reverse motor duty cycle Can be negative, zero, or positive
in	output_max	New maximum output limit (upper saturation bound) Must be > output_min (validated - strict inequality) Units must match output_min Example: +100.0f for full forward motor duty cycle Must be strictly greater than output_min

Returns: rx_err_t Error code indicating limit update result

Value	Description
k_rx_ok	Output limits updated successfully
k_rx_err_null_ptr	handle pointer is nullptr

k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)
k_rx_err_invalid_arg	output_max <= output_min (not strictly greater)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Pre: output_max > output_min (strictly greater, not equal)

Post: handle->output_min == output_min (on success)

Post: handle->output_max == output_max (on success)

Post: New limits take effect on next rx_pid_compute() call

Post: Gains and state unchanged (kp, ki, kd, integral, prev_error)

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 200 ns @ 240 MHz (50 cycles) - includes validation

Note: Immediate Effect: New limits apply starting with next rx_pid_compute() call

Note: State Preserved: Integral and derivative state not affected

Warning: Symmetric vs Asymmetric: Ensure limits match actuator capabilities (e.g., motor can reverse)

Warning: No Auto-Clamping: Existing integral state is NOT clamped to new limits - only future outputs
Example: Reduce Speed for Safety Mode: #include "rx_pid.h"

```
void enter_safety_mode(void){ rx_err_t err=rx_pid_set_output_limits(&motor_pid,-50.0f,50.0f);
if(err==k_rx_ok){ rx_log_info("MOTOR","Safety mode: output limited to +/-50%"); } }

void exit_safety_mode(void){ rx_pid_set_output_limits(&motor_pid,-100.0f,100.0f);
rx_log_info("MOTOR","Normal mode: full output range restored"); }
```

Example: Asymmetric Limits (Brake-Only Mode): void enable_brake_only_mode(void)
{ rx_pid_set_output_limits(&motor_pid,-100.0f,0.0f); rx_log_info("MOTOR","Brake-only mode active"); }

Example: Runtime Limit Adjustment Based on Temperature:

```
voidadjust_limits_for_temperature(floattemperature_celsius){ constfloatk_max_temp=80.0f;
constfloatk_warn_temp=60.0f; constfloatk_max_duty=100.0f;

floatscaled_max=k_max_duty; if(temperature_celsius>k_warn_temp)
{ scaled_max=k_max_duty*(1.0f-(temperature_celsius-k_warn_temp)/(k_max_temp-
k_warn_temp)); }

if(scaled_max>k_max_duty)scaled_max=k_max_duty; if(scaled_max<20.0f)scaled_max=20.0f;

rx_pid_set_output_limits(&motor_pid,-scaled_max,scaled_max); }
```

Example: Error Handling: rx_err_terr=rx_pid_set_output_limits(&pid,-100.0f,100.0f);

```
switch(err){ casek_rx_ok: rx_log_info("PID","Outputlimitsupdated"); break; casek_rx_err_null_ptr:
rx_log_error("PID","nullptrhandlepointer"); break; casek_rx_err_invalid_state:
rx_log_error("PID","PIDnotinitialized"); break; casek_rx_err_invalid_arg:
rx_log_error("PID","Invalidlimits(max<=min)"); break; }
```

Performance Analysis:: Execution time: 200 ns @ 240 MHz (50 cycles) Memory: Modifies 8 bytes
(2 floats: output_min, output_max) Stack usage: 0 bytes (parameters passed by value/pointer)
Validation overhead: 3 checks (nullptr, init, max > min)

Limit Validation Table:: Validation Condition Error Code

nullptr handle != nullptr k_rx_err_null_ptr

Initialized handle->initialized == true k_rx_err_invalid_state

Strictly greater output_max > output_min k_rx_err_invalid_arg

NASA Power of 10 Compliance:: Rule 4: Function is 15 lines (well under 60-line limit) Rule 5: 3
validation checks (nullptr, init, range)

See also: rx_pid_init() to initialize controller with initial output limits,
rx_pid_set_integral_limits() to update anti-windup limits (usually proportional to
output limits), rx_pid_compute() applies the output limits via clamping

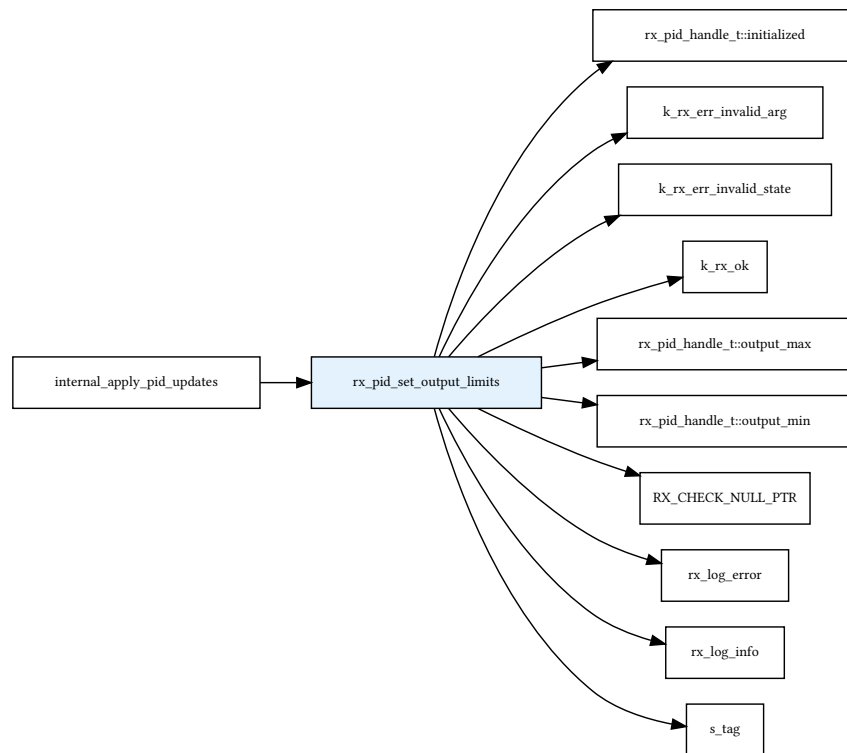


Figure 1071: Call graph for rx_pid_set_output_limits

Defined in `libs/rx_pid/src/rx_pid.c:1336`

Since Version 1.0.0

—

rx_pid_set_integral_limits

```
rx_err_t rx_pid_set_integral_limits(rx_pid_handle_t *handle, const float
integral_min, const float integral_max)
```

Update PID integral anti-windup limits at runtime.

Update PID integral limits at runtime (anti-windup).

Modifies the integral accumulator saturation limits (`integral_min`, `integral_max`) without reinitializing the controller. These limits prevent integral windup during prolonged actuator saturation. Important: This function IMMEDIATELY clamps the current integral value to the new limits, unlike `rx_pid_set_output_limits()`.

Update Steps:

1. Validate handle pointer (NULL check)
2. Check initialization state
3. Validate `integral_max > integral_min` (strict inequality)
4. Store new integral limits

5. Immediately clamp current integral to new limits (key difference from output limits)
6. Log update event

Anti-Windup Strategy: Integral windup occurs when the controller output saturates but the integral term continues to accumulate error. This causes overshoot and sluggish response. By limiting the integral accumulator to [integral_min, integral_max], windup is prevented.

Typical Settings:

- Rule of Thumb: Set integral limits to 50-80% of output range
- Symmetric: integral_min = -integral_max (most common for bidirectional actuators)
- Asymmetric: Different limits for forward/reverse (e.g., heating vs. cooling)

After successful call: integral_min <= handle->integral <= integral_max

handle->kp, ki, kd unchanged

handle->output_min/max unchanged

handle->prev_error unchanged

Recommended Practice: Update integral limits when output limits change to maintain 50-80% ratio

1.0.0

Parameters:

Direction	Name	Description
inout	handle	Pointer to initialized PID controller handle Must not be NULL (validated) Must be initialized (validated) integral_min and integral_max fields will be updated IMPORTANT: Current integral value will be clamped to new limits
in	integral_min	New minimum integral accumulator limit (anti-windup lower bound) Must be < integral_max (validated - strict inequality) Units same as output (since I_term = Ki x integral) Example: -50.0f for motor control (50% of output range) Prevents excessive negative integral buildup
in	integral_max	New maximum integral accumulator limit (anti-windup upper bound) Must be > integral_min (validated - strict inequality) Units same as output Example: +50.0f for motor control (50% of output range) Prevents excessive positive integral buildup

Returns: rx_err_t Error code indicating limit update result

Value	Description
k_rx_ok	Integral limits updated successfully, current integral clamped
k_rx_err_null_ptr	handle pointer is nullptr
k_rx_err_invalid_state	Controller not initialized (call rx_pid_init first)
k_rx_err_invalid_arg	integral_max <= integral_min (not strictly greater)

Pre: handle must not be nullptr

Pre: handle->initialized must be true

Pre: integral_max > integral_min (strictly greater, not equal)

Post: handle->integral_min == integral_min (on success)

Post: handle->integral_max == integral_max (on success)

Post: handle->integral in integral_min, integral_max (immediately clamped)

Post: Gains and output limits unchanged (kp, ki, kd, output_min/max)

Note: Thread Safety: NOT thread-safe - do not call concurrently for same handle

Note: Performance: 220 ns @ 240 MHz (55 cycles) - includes validation and clamping

Note: Immediate Clamping: Unlike rx_pid_set_output_limits(), this immediately clamps integral

Note: Windup Prevention: Limits should be 50-80% of output range for effective anti-windup

Warning: Integral Clamping: Existing integral value is IMMEDIATELY clamped to new limits - may cause transient in next output

Warning: Proportional to Output: Integral limits should scale with output limits for consistent behavior] **Example: Update Integral Limits with Output Limits:** #include "rx_pid.h"

```
voidenter_safety_mode(void){ constfloatk_safety_output_limit=50.0f;
constfloatk_safety_integral_limit=k_safety_output_limit*0.6f;

rx_pid_set_output_limits(&motor_pid,-k_safety_output_limit,k_safety_output_limit);
rx_pid_set_integral_limits(&motor_pid,-k_safety_integral_limit,k_safety_integral_limit);
rx_log_info("MOTOR","Safetymode:limitsreduced"); }

Example: Aggressive Anti-Windup for Fast Response: voidreduce_overshoot(void)
{ constfloatk_tight_integral_limit=30.0f;

rx_err_terr=rx_pid_set_integral_limits(&motor_pid, -k_tight_integral_limit, k_tight_integral_limit);
if(err==k_rx_ok){ rx_log_info("PID","Tighteranti-winduplimitsapplied"); } }

Example: Asymmetric Limits (Heater Control): voidconfigure_heater_pid(void)
{ rx_pid_set_output_limits(&heater_pid,0.0f,100.0f);
rx_pid_set_integral_limits(&heater_pid,0.0f,50.0f); }
```

Example: Dynamic Limit Adjustment: void adjust_integral_limits_for_load(float load_percentage)
{ float integral_limit;

if(load_percentage > 80.0f){ integral_limit = 30.0f; } else{ integral_limit = 60.0f; }

rx_pid_set_integral_limits(&motor_pid, integral_limit, integral_limit); }

Example: Error Handling: rx_err_t err = rx_pid_set_integral_limits(&pid, -40.0f, 40.0f);

switch(err){ case k_rx_ok: rx_log_info("PID", "Integral limits updated and current integral clamped");

break; case k_rx_err_null_ptr: rx_log_error("PID", "null ptr handle pointer"); break;

case k_rx_err_invalid_state: rx_log_error("PID", "PID not initialized"); break; case k_rx_err_invalid_arg:

rx_log_error("PID", "Invalid limits (max <= min)"); break; }

Performance Analysis:: Execution time: 220 ns @ 240 MHz (55 cycles) Memory: Modifies 12 bytes
(3 floats: integral_min, integral_max, integral) Stack usage: 0 bytes (parameters passed by value/
pointer) Validation overhead: 3 checks + 1 clamp operation

Anti-Windup Effectiveness Table:: Integral Limit (% of Output Range) Windup Prevention
Response Speed Overshoot

30-40% Aggressive Slow Minimal

50-60% Moderate (recommended) Balanced Low

70-80% Light Fast Moderate

90-100% Minimal Very Fast High

Limit Validation Table:: Validation Condition Error Code

nullptr handle != nullptr k_rx_err_null_ptr

Initialized handle->initialized == true k_rx_err_invalid_state

Strictly greater integral_max > integral_min k_rx_err_invalid_arg

Key Difference from rx_pid_set_output_limits(): Feature rx_pid_set_output_limits()

rx_pid_set_integral_limits()

Clamps current value? NO (affects next computation only) YES (immediately clamps integral)

When to use? Change actuator constraints Tune anti-windup behavior

Typical ratio to output N/A (defines actuator range) 50-80% of output range

NASA Power of 10 Compliance:: Rule 4: Function is 19 lines (well under 60-line limit) Rule 5: 3
validation checks (nullptr, init, range) + 1 postcondition (clamping)

See also: rx_pid_init() to initialize controller with initial integral limits,
rx_pid_set_output_limits() to update output saturation limits (usually updated
together), rx_pid_compute() uses integral limits to prevent windup during computation,
internal_clamp() helper function used to clamp integral



Figure 1072: Call graph for rx_pid_set_integral_limits

Defined in `libs/rx_pid/src/rx_pid.c:1562`

Since Version 1.0.0

—

rx_session.h

Shared session state for cross-transport sequence continuity.

Provides a single, shared session state (TX and RX sequence counters) that is used by BOTH USB CDC and SPI transport layers. When the active transport switches (e.g., USB at seq=105 to SPI), SPI continues at seq=106 because both transports reference the same session state.

This module mirrors the Go gateway's `manager/session.go` implementation:

- `NextTxSequence()` -> `rx_session_next_tx()`
- `ValidateRxSequence()` -> `rx_session_validate_rx()`
- `Reset()` -> `rx_session_reset()`

Includes:

- `<stdbool.h>`

- <stdint.h>
- "rx_err.h"

rx_session_constants_t

Session state configuration constants.

Configuration parameters for the session state module. These match the Go gateway's constants in `manager/session.go`.

Value	Name	Description
= 0	<code>k_session_initial_sequence</code>	Initial TX/RX sequence value after init or reset.
= 0	<code>k_session_diff_exact_match</code>	Exact-match diff value for sequence validation.
= 10	<code>k_session_max_gap_tolerance</code>	Maximum allowable gap between expected and received sequence numbers.
= 0xFFFF	<code>k_session_seq_wrap_mask</code>	Sequence number wraparound mask (16-bit unsigned).

See also: `star-gateway/internal/manager/session.go` Go reference values

Since Version 1.0.0

—

rx_session_validate_result_t

Result codes for sequence validation.

Returned by `rx_session_validate_rx()` to indicate the validation outcome. This provides more information than a simple bool, allowing callers to distinguish between exact matches, gap acceptance, and rejection.

Value	Name	Description
= 0	<code>k_session_validate_ok</code>	Exact sequence match (most common case)
= 1	<code>k_session_validate_gap</code>	Accepted with gap (packet loss detected)
= 2	<code>k_session_validate_fail</code>	Rejected (large gap or duplicate)

Since Version 1.0.0

—

rx_session_init

```
rx_err_t rx_session_init(rx_session_state_t *state)
```

Initialize session state with zeroed sequences.

Sets both TX and RX sequences to 0 and creates the internal ThreadX mutex for thread-safe access. Must be called before any other `rx_session_*`() function.

In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since the ThreadX kernel is not running.

Sets both TX and RX sequences to `k_session_initial_sequence` and creates the internal ThreadX mutex for thread-safe access. Must be called before any other `rx_session_*`() function.

In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since the ThreadX kernel is not running.

Parameters:

Direction	Name	Description
inout	state	Session state to initialize (must not be NULL)
inout	state	Session state to initialize (must not be NULL)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, session ready for use
k_rx_err_null_ptr	state is nullptr
k_rx_err_rtos_mutex	ThreadX mutex creation failed
k_rx_ok	Success, session ready for use
k_rx_err_null_ptr	state is nullptr
k_rx_err_rtos_mutex	ThreadX mutex creation failed

Pre: state must point to valid, allocated memory

Pre: state must not already be initialized (call rx_session_deinit() first)

Pre: state must point to valid, allocated memory

Pre: state must not already be initialized (call rx_session_deinit() first)

Post: state->tx_sequence == 0

Post: state->rx_sequence == 0

Post: state->initialized == true

Post: state->tx_sequence == k_session_initial_sequence

Post: state->rx_sequence == k_session_initial_sequence

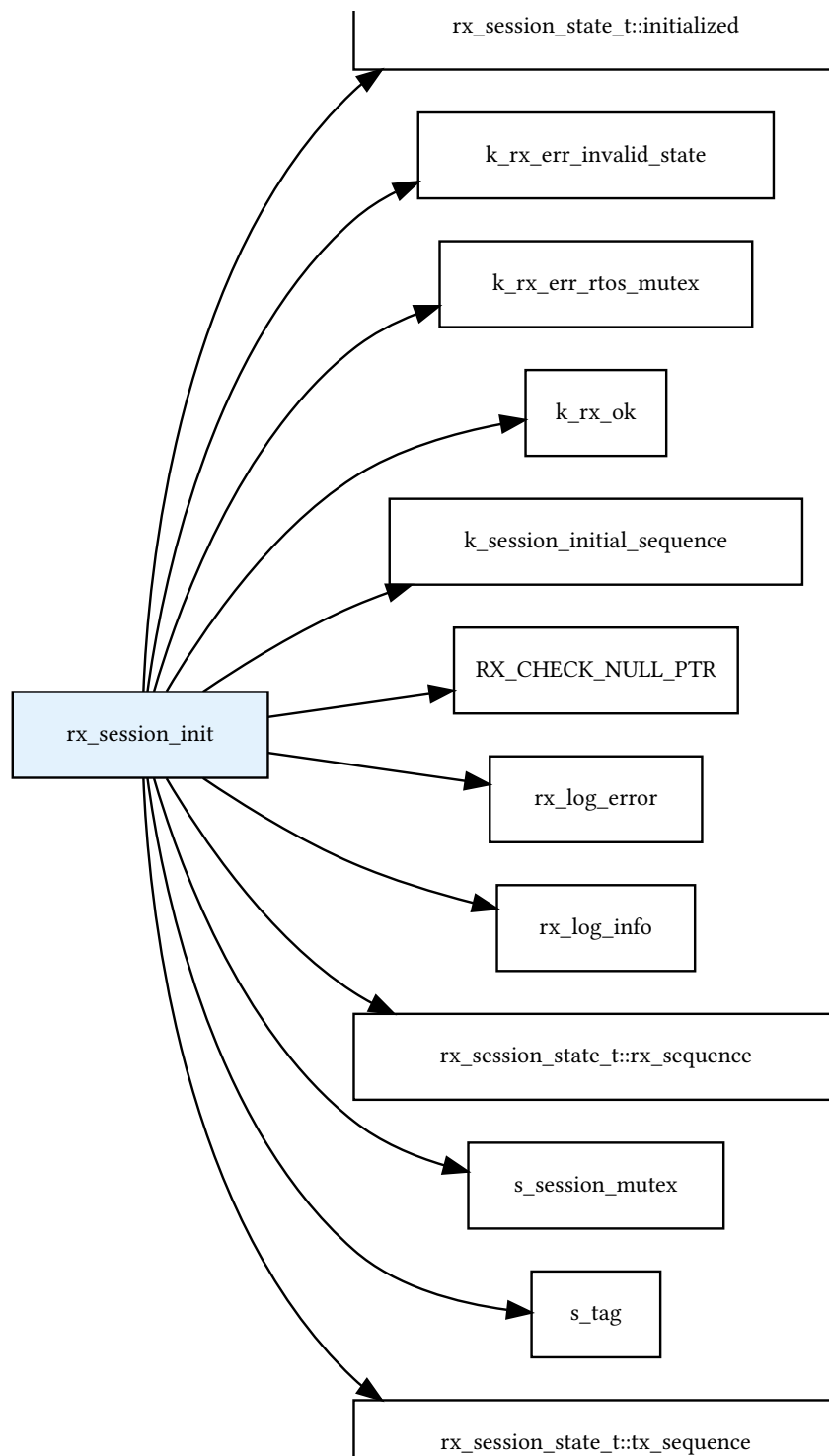
Post: state->initialized == true

Note: Thread-safe: No mutex needed during init (single-threaded startup)

Note: Thread-safe: No mutex needed during init (single-threaded startup)

Example:: `static rx_session_state_tg_session; rx_err_t err=rx_session_init(&g_session);
assert(err==k_rx_ok);`

See also: `rx_session_deinit()` Cleanup counterpart, `rx_session_deinit()` Cleanup counterpart

Figure 1073: Call graph for `rx_session_init`

Defined in `libs/rx_session/inc/rx_session.h:246`

Since Version 1.0.0

—

rx_session_deinit

```
rx_err_t rx_session_deinit(rx_session_state_t *state)
```

Deinitialize session state and release resources.

Deletes the internal ThreadX mutex and marks the session as uninitialized. After this call, no other `rx_session_*`() function may be called until `rx_session_init()` is called again.

Deletes the internal ThreadX mutex and marks the session as uninitialized. After this call, no other `rx_session_*`() function may be called until `rx_session_init()` is called again.

In simulator builds, mutex deletion is skipped since it was never created.

Parameters:

Direction	Name	Description
inout	state	Session state to deinitialize (must not be NULL)
inout	state	Session state to deinitialize (must not be NULL)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, resources released
<code>k_rx_err_null_ptr</code>	state is nullptr
<code>k_rx_err_not_initialized</code>	state was not initialized
<code>k_rx_ok</code>	Success, resources released
<code>k_rx_err_null_ptr</code>	state is nullptr
<code>k_rx_err_not_initialized</code>	state was not initialized

Pre: state must be initialized via `rx_session_init()`

Pre: state != nullptr

Pre: state->initialized == true

Post: state->initialized == false

Post: Internal mutex deleted

Post: state->initialized == false

Post: Internal mutex deleted (on non-simulator builds)

Note: Thread-safe: Caller must ensure no other threads are using the session

Note: Thread-safe: Caller must ensure no other threads are using the session] **See also:**
rx_session_init() Initialization counterpart, rx_session_init() Initialization counterpart

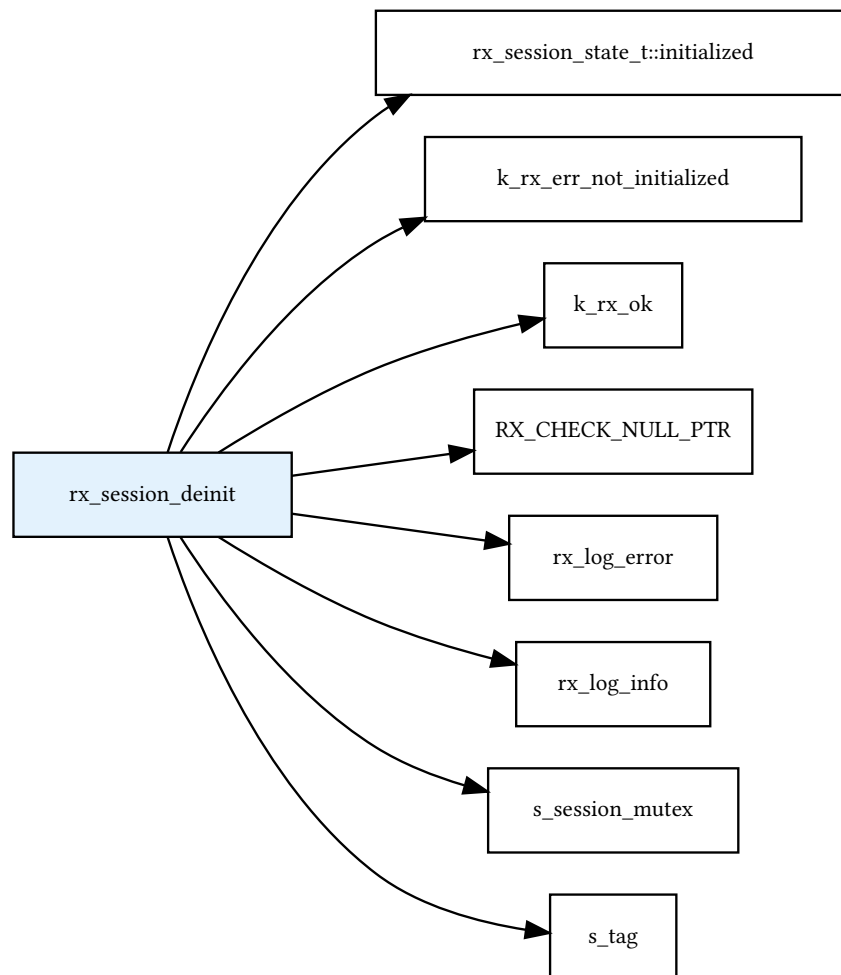


Figure 1074: Call graph for rx_session_deinit

Defined in `libs/rx_session/inc/rx_session.h:273`

Since Version 1.0.0

rx_session_next_tx

```
rx_err_t rx_session_next_tx(rx_session_state_t *state, uint16_t *sequence)
```

Get next TX sequence number and increment the counter.

Atomically reads the current TX sequence, increments it (with wraparound at 65535), and returns the pre-increment value. This is used by both USB and SPI transport layers when sending frames.

Mirrors Go gateway's `SessionState.NextTxSequence()`.

Atomically reads the current TX sequence, increments it (with wraparound at `k_session_seq_wrap_mask`), and returns the pre-increment value. This is used by both USB and SPI transport layers when sending frames.

Mirrors Go gateway's `SessionState.NextTxSequence()`.

Parameters:

Direction	Name	Description
inout	state	Session state (must be initialized)
out	sequence	Pointer to receive the sequence number
inout	state	Session state (must be initialized)
out	sequence	Pointer to receive the sequence number

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, sequence written
<code>k_rx_err_null_ptr</code>	state or sequence is nullptr
<code>k_rx_err_not_initialized</code>	state not initialized
<code>k_rx_ok</code>	Success, sequence written
<code>k_rx_err_null_ptr</code>	state or sequence is nullptr
<code>k_rx_err_not_initialized</code>	state not initialized
<code>k_rx_err_rtos_mutex</code>	Mutex acquisition failed

Pre: state must be initialized via `rx_session_init()`

Pre: sequence must point to valid memory

Pre: state must be initialized via `rx_session_init()`

Pre: sequence must point to valid memory

Post: state->tx_sequence incremented by 1 (with wraparound)

Post: *sequence contains the pre-increment value

Post: state->tx_sequence incremented by 1 (with wraparound)

Post: *sequence contains the pre-increment value

Post: *sequence <= k_session_seq_wrap_mask

Note: Thread-safe: Protected by internal mutex

Note: Thread-safe: Protected by internal mutex

Example:: `uint16_t seq; rx_err_t err = rx_session_next_tx(&g_session, &seq); if (err == k_rx_ok) { frame.header.sequence = seq; }`

See also: `rx_session_get_tx()` Read without incrementing, `rx_session_get_tx()` Read without incrementing

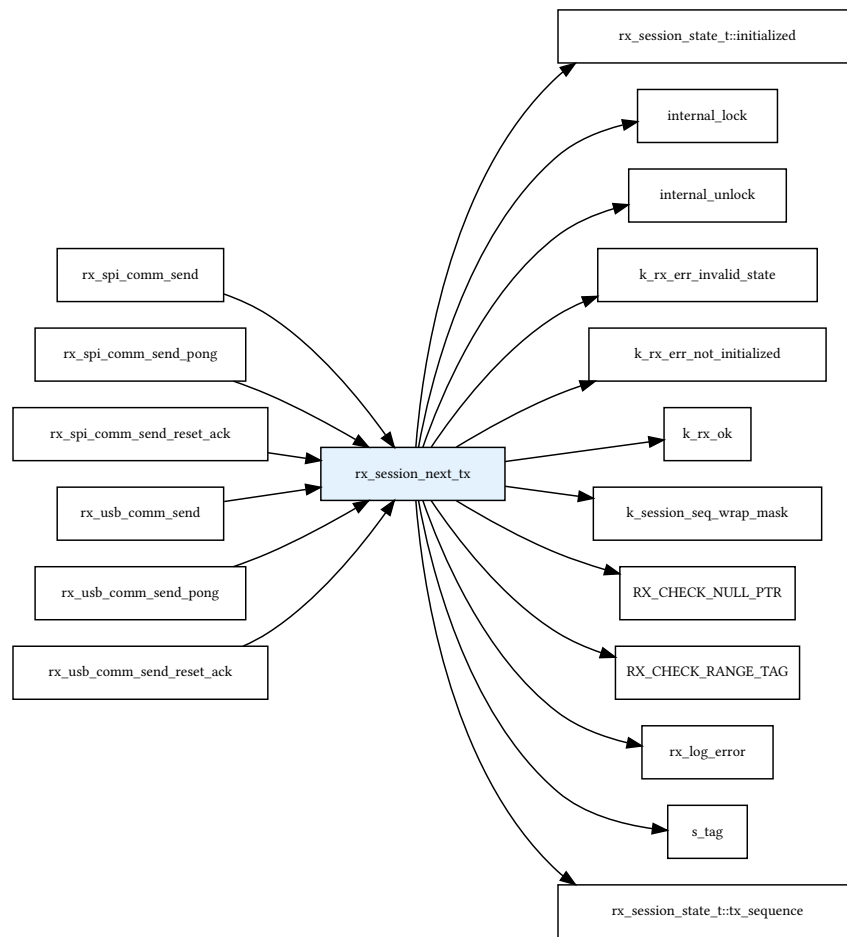


Figure 1075: Call graph for rx_session_next_tx

Defined in `libs/rx_session/inc/rx_session.h:313`

Since Version 1.0.0

—

rx_session_validate_rx

```

rx_err_t rx_session_validate_rx(rx_session_state_t *state, uint16_t received_seq,
rx_session_validate_result_t *result)

```

Validate a received sequence number.

Checks whether the received sequence number is acceptable based on the expected RX sequence. Implements gap tolerance matching the Go gateway's `SessionState.ValidateRxSequence()`.

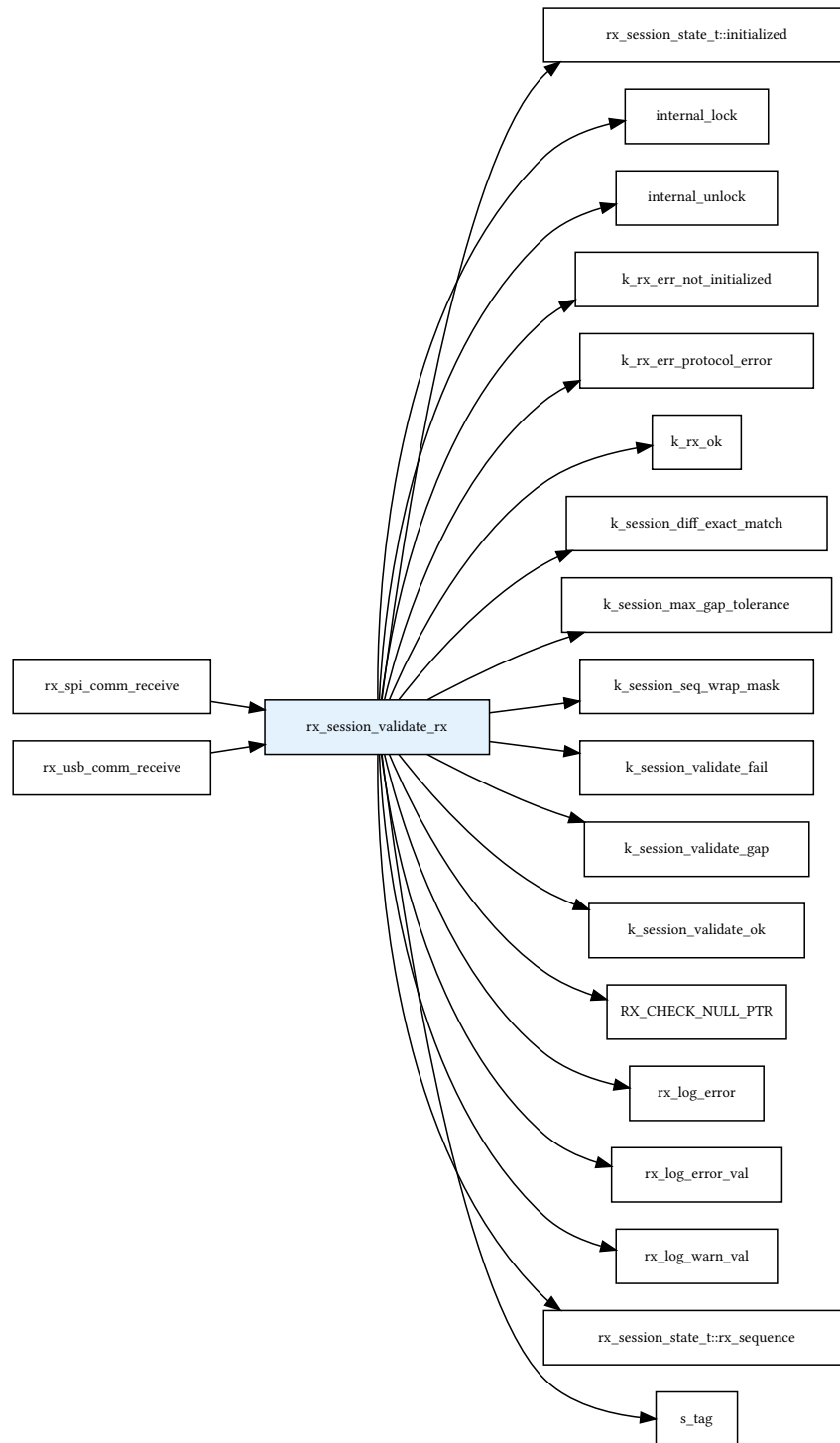


Figure 1076: Call graph for rx_session_validate_rx

Defined in `libs/rx_session/inc/rx_session.h:367`

—

rx_session_reset

```
rx_err_t rx_session_reset(rx_session_state_t *state)
```

Reset both TX and RX sequences to zero.

Called after a RESET/RESET_ACK handshake completes to synchronize sequences between firmware and gateway. Both counters are atomically set to 0.

Mirrors Go gateway's `SessionState.Reset()`.

Reset both TX and RX sequences to zero.

Called after a RESET/RESET_ACK handshake completes to synchronize sequences between firmware and gateway. Both counters are atomically set to `k_session_initial_sequence`.

Mirrors Go gateway's `SessionState.Reset()`.

Parameters:

Direction	Name	Description
inout	state	Session state (must be initialized)
inout	state	Session state (must be initialized)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, sequences reset
<code>k_rx_err_null_ptr</code>	state is nullptr
<code>k_rx_err_not_initialized</code>	state not initialized
<code>k_rx_ok</code>	Success, sequences reset
<code>k_rx_err_null_ptr</code>	state is nullptr
<code>k_rx_err_not_initialized</code>	state not initialized
<code>k_rx_err_rtos_mutex</code>	Mutex acquisition failed

Pre: state must be initialized via `rx_session_init()`

Pre: state must be initialized via `rx_session_init()`

Post: `state->tx_sequence == 0`

Post: `state->rx_sequence == 0`

Post: `state->tx_sequence == k_session_initial_sequence`

Post: state->rx_sequence == k_session_initial_sequence

Note: Thread-safe: Protected by internal mutex

Note: Thread-safe: Protected by internal mutex] **See also:**

rx_frame_type_t::k_frame_type_reset RESET frame triggers this,
 rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset,
 rx_frame_type_t::k_frame_type_reset RESET frame triggers this,
 rx_frame_type_t::k_frame_type_reset_ack RESET_ACK confirms reset

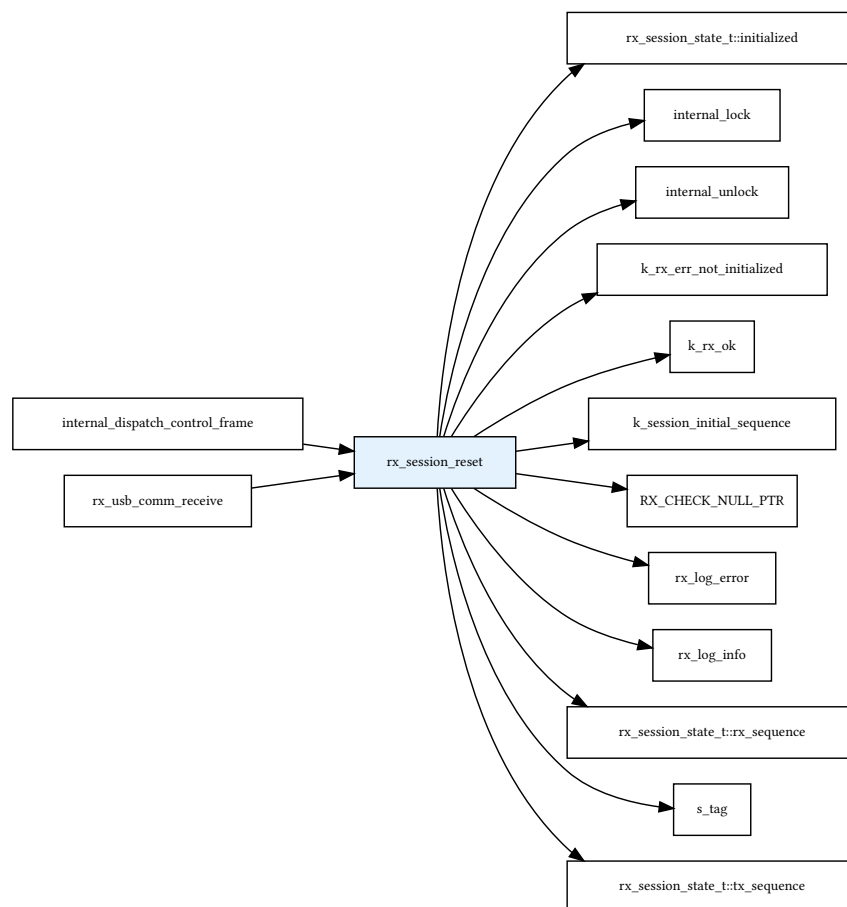


Figure 1077: Call graph for rx_session_reset

Defined in `libs/rx_session/inc/rx_session.h:398`

Since Version 1.0.0

rx_session_get_tx

```
rx_err_t rx_session_get_tx(const rx_session_state_t *state, uint16_t *sequence)
```

Get current TX sequence without incrementing.

Returns the current TX sequence counter value for diagnostic purposes. Does not modify the counter.

Returns the current TX sequence counter value for diagnostic purposes. Does not modify the counter.

Parameters:

Direction	Name	Description
in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current TX sequence
in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current TX sequence

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: state must be initialized via rx_session_init()

Pre: state must be initialized via rx_session_init()

Pre: sequence must point to valid memory

Post: state unchanged

Post: state unchanged

Post: *sequence <= k_session_seq_wrap_mask

Note: Thread-safe: Protected by internal mutex

Note: Thread-safe: Protected by internal mutex] **See also:** `rx_session_next_tx()` Read and increment, `rx_session_next_tx()` Read and increment

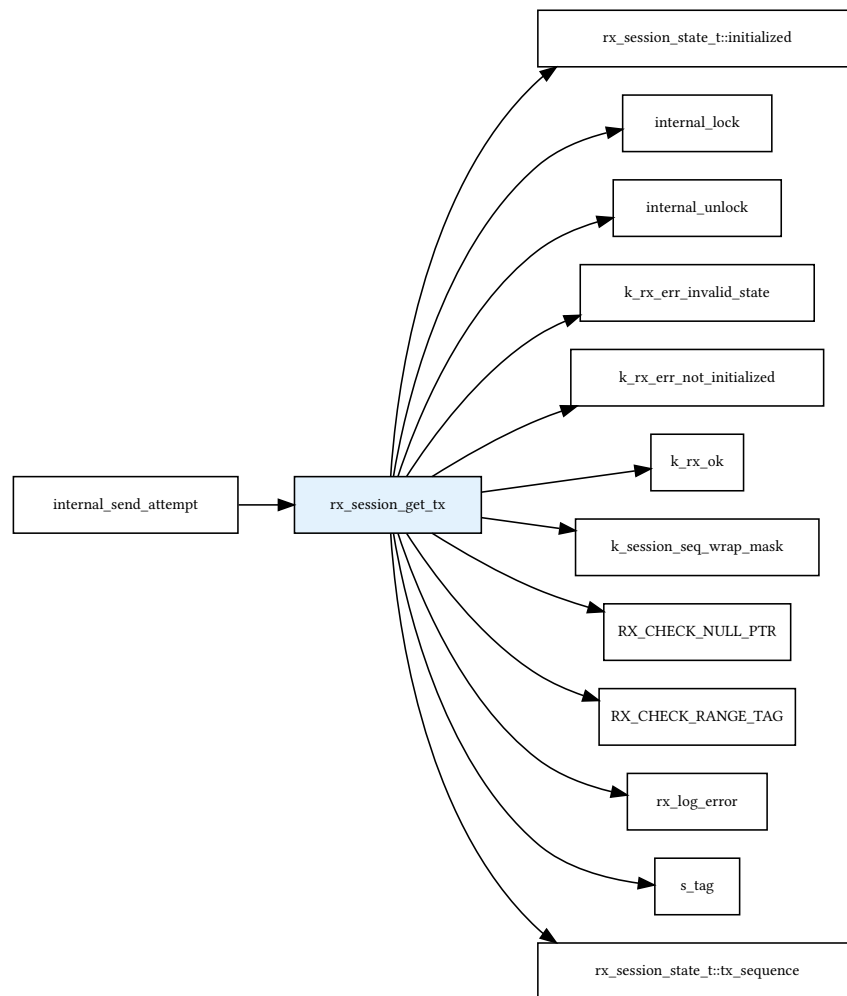


Figure 1078: Call graph for `rx_session_get_tx`

Defined in `libs/rx_session/inc/rx_session.h:424`

Since Version 1.0.0

rx_session_get_rx

```
rx_err_t rx_session_get_rx(const rx_session_state_t *state, uint16_t *sequence)
```

Get current expected RX sequence.

Returns the next expected RX sequence counter value for diagnostic purposes. Does not modify the counter.

Get current expected RX sequence.

Returns the next expected RX sequence counter value for diagnostic purposes. Does not modify the counter.

Parameters:

Direction	Name	Description
in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current RX sequence
in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current RX sequence

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: state must be initialized via rx_session_init()

Pre: state must be initialized via rx_session_init()

Pre: sequence must point to valid memory

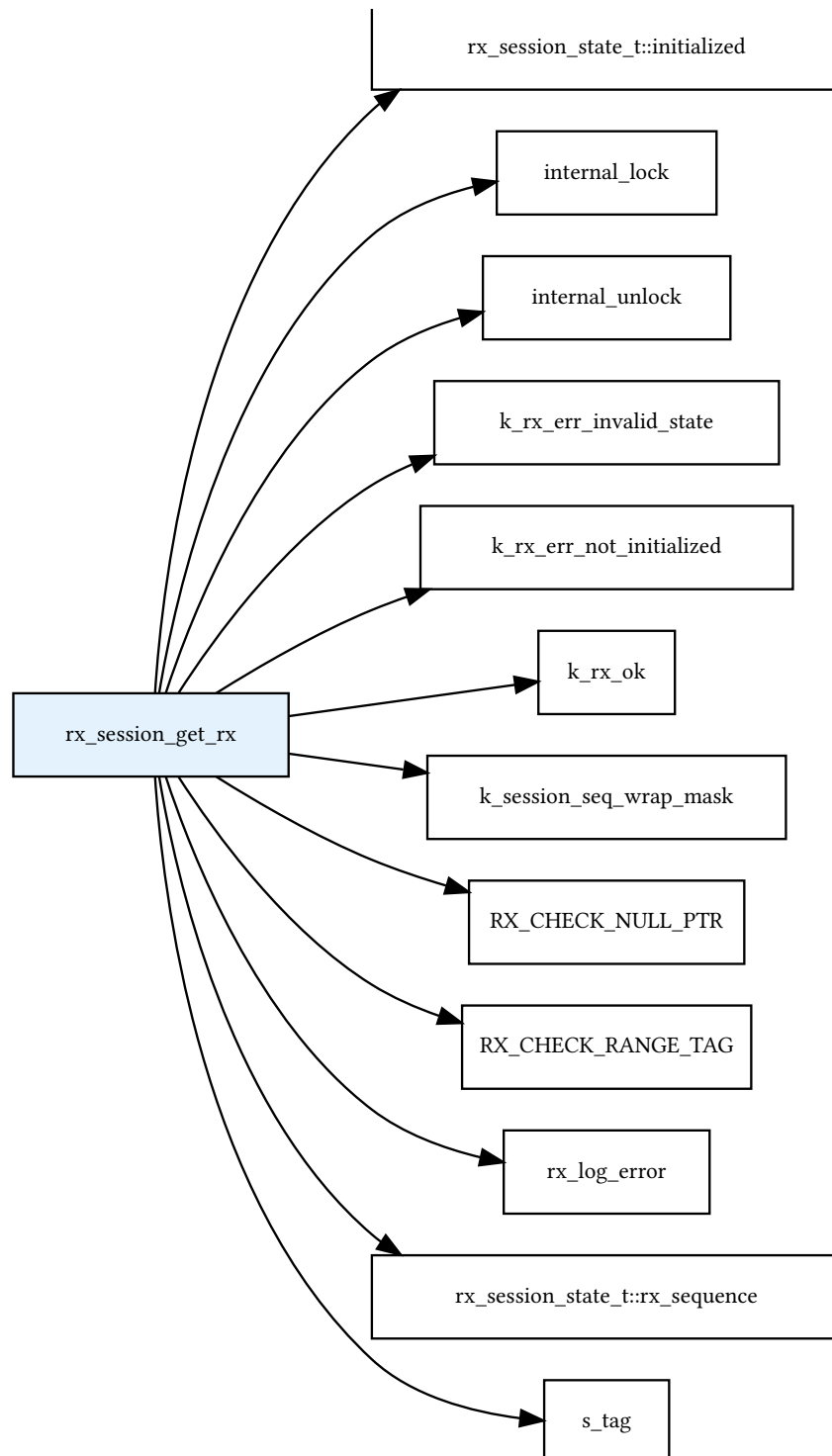
Post: state unchanged

Post: state unchanged

Post: *sequence <= k_session_seq_wrap_mask

Note: Thread-safe: Protected by internal mutex

Note: Thread-safe: Protected by internal mutex] **See also:** rx_session_validate_rx()
Validate and update, rx_session_validate_rx() Validate and update

Figure 1079: Call graph for `rx_session_get_rx`

Defined in `libs/rx_session/inc/rx_session.h:450`

Since Version 1.0.0

—

rx_session.c

Shared session state implementation for cross-transport sequence continuity.

Implements the cross-transport shared session state module. Provides thread-safe TX/RX sequence tracking with gap tolerance, mirroring the Go gateway's `manager/session.go` implementation.

Both USB CDC and SPI transports share a single session state instance (owned by `rx_comm_manager_t`), ensuring sequence continuity when the active transport switches.

Includes:

- `"rx_session.h"`
- `"rx_check.h"`
- `"rx_log.h"`
- `"rx_simulator_config.h"`
- `"tx_api.h"`

s_tag

```
const char s_tag[]
```

Logging tag for session state messages.

Note: Used with `rx_log_*`() macros for consistent log filtering

Since Version 1.0.0

—

s_session_mutex

```
TX_MUTEX s_session_mutex
```

ThreadX mutex for protecting session state access.

Warning: Do not access directly. Use `internal_lock()` / `internal_unlock()`.

Since Version 1.0.0

—

internal_lock

```
static rx_err_t internal_lock(void)
```

Acquire the session mutex.

Locks the internal mutex with `TX_WAIT_FOREVER`. In simulator mode, this is a no-op since ThreadX is not running.

Returns: `rx_err_t`

Value	Description
-------	-------------

k_rx_ok	Mutex acquired
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: Mutex must have been created via rx_session_init()

Post: Mutex is held by calling thread

Note: Thread-safe: This IS the synchronization primitive

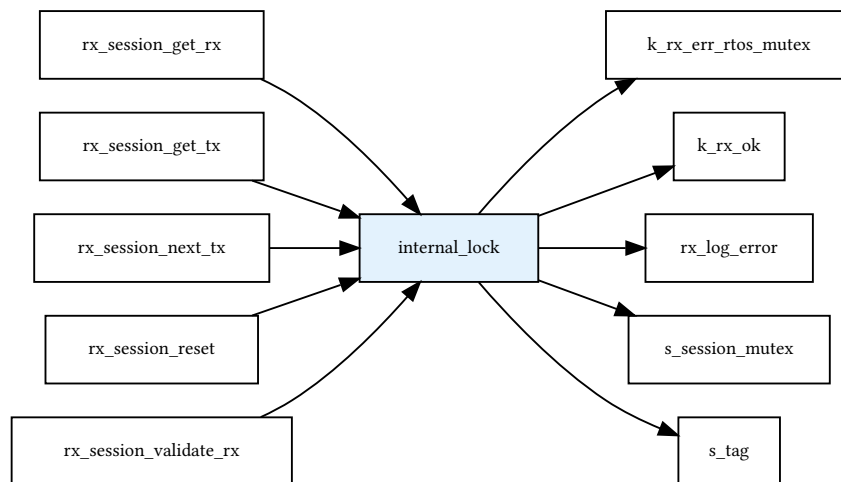


Figure 1080: Call graph for internal_lock

Defined in `libs/rx_session/src/rx_session.c:114`

Since Version 1.0.0

—

internal_unlock

```
static void internal_unlock(void)
```

Release the session mutex.

Unlocks the internal mutex. In simulator mode, this is a no-op.

Pre: Mutex must be held by calling thread

Post: Mutex is released

Note: Thread-safe: This IS the synchronization primitive

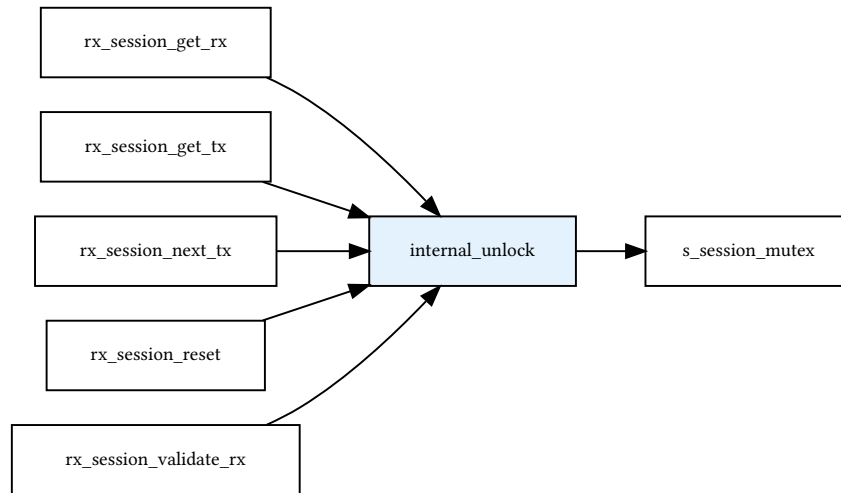


Figure 1081: Call graph for internal_unlock

Defined in `libs/rx_session/src/rx_session.c:138`

Since Version 1.0.0

—

rx_session_init

```
rx_err_t rx_session_init(rx_session_state_t *state)
```

Initialize session state with zeroed sequences.

Sets both TX and RX sequences to `k_session_initial_sequence` and creates the internal ThreadX mutex for thread-safe access. Must be called before any other `rx_session_*`() function.

In simulator builds (RX_SIMULATOR_MODE), mutex creation is skipped since the ThreadX kernel is not running.

Parameters:

Direction	Name	Description
inout	state	Session state to initialize (must not be NULL)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, session ready for use
k_rx_err_null_ptr	state is nullptr
k_rx_err_rtos_mutex	ThreadX mutex creation failed

Pre: state must point to valid, allocated memory

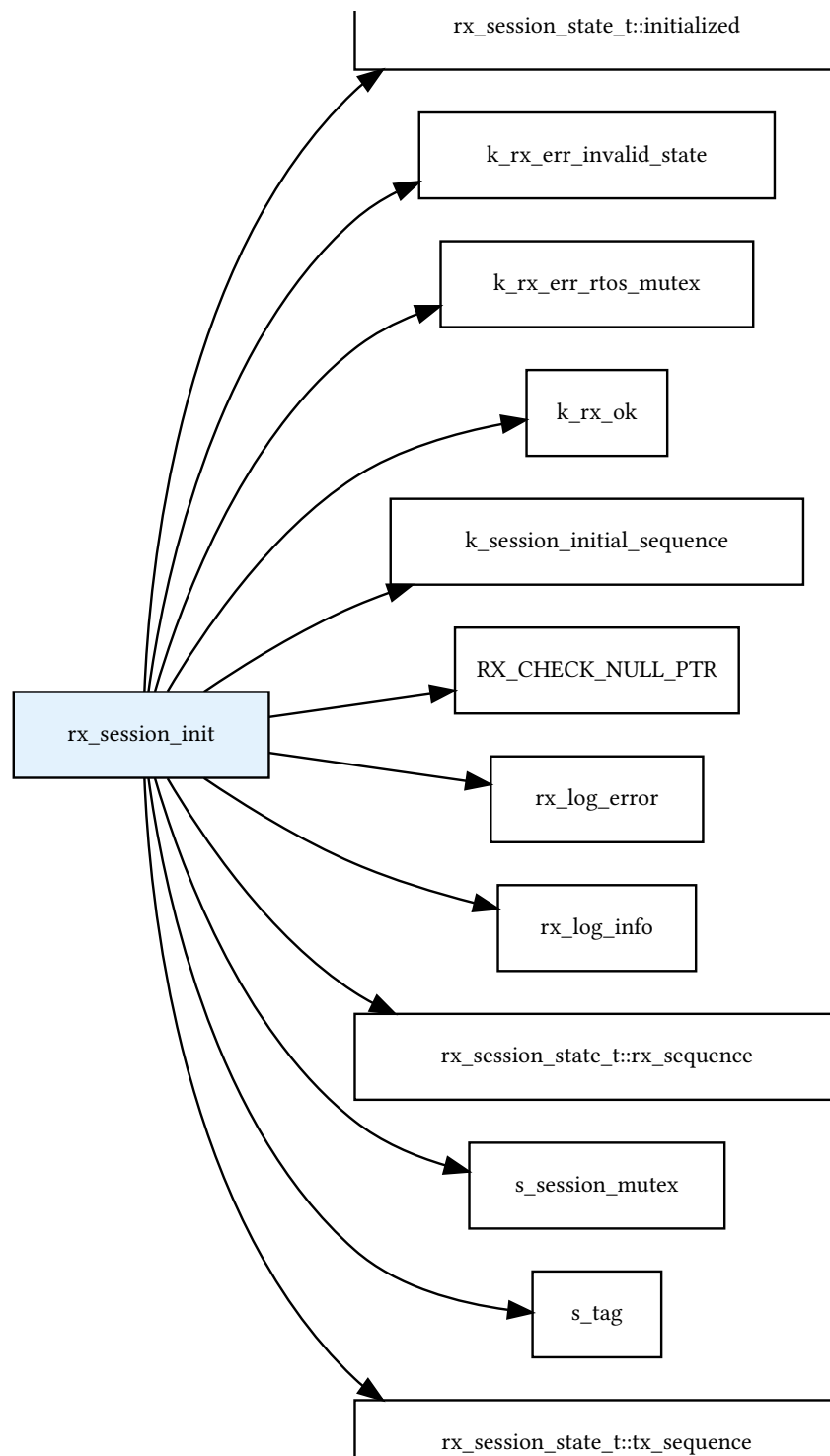
Pre: state must not already be initialized (call rx_session_deinit() first)

Post: state->tx_sequence == k_session_initial_sequence

Post: state->rx_sequence == k_session_initial_sequence

Post: state->initialized == true

Note: Thread-safe: No mutex needed during init (single-threaded startup)] **See also:**
rx_session_deinit() Cleanup counterpart

Figure 1082: Call graph for `rx_session_init`

Defined in `libs/rx_session/src/rx_session.c:178`

Since Version 1.0.0

—

rx_session_deinit

```
rx_err_t rx_session_deinit(rx_session_state_t *state)
```

Deinitialize session state and release resources.

Deletes the internal ThreadX mutex and marks the session as uninitialized. After this call, no other `rx_session_*`() function may be called until `rx_session_init()` is called again.

In simulator builds, mutex deletion is skipped since it was never created.

Parameters:

Direction	Name	Description
inout	state	Session state to deinitialize (must not be NULL)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, resources released
<code>k_rx_err_null_ptr</code>	state is nullptr
<code>k_rx_err_not_initialized</code>	state was not initialized

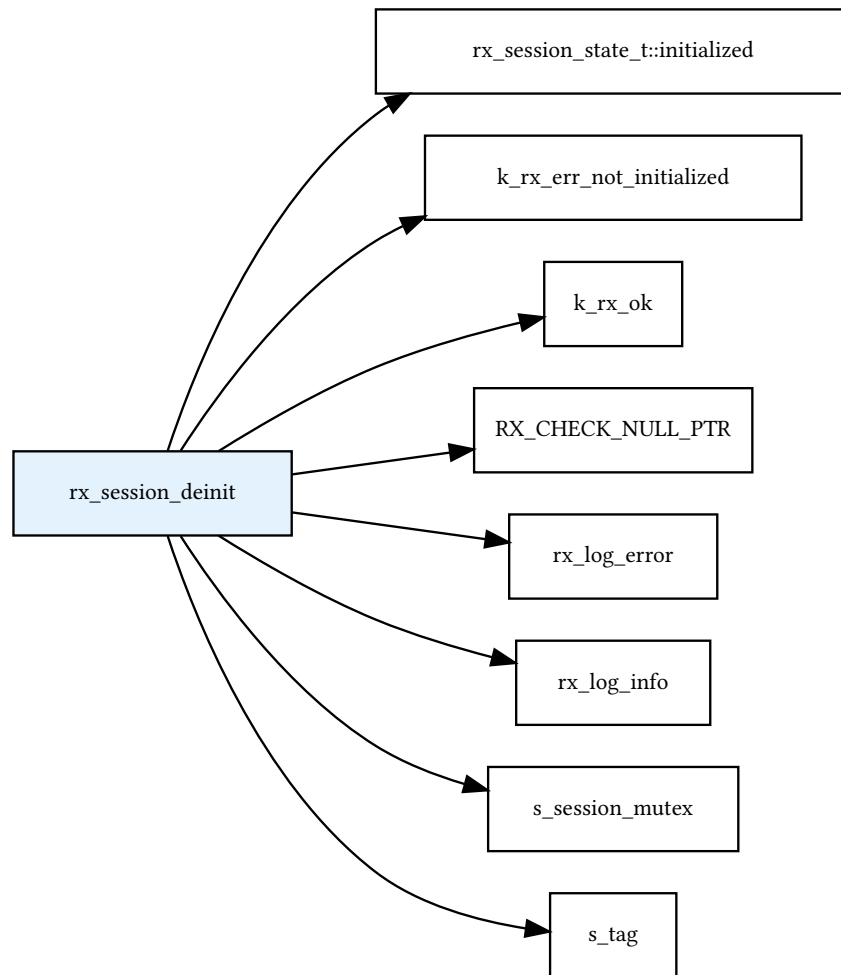
Pre: state != nullptr

Pre: state->initialized == true

Post: state->initialized == false

Post: Internal mutex deleted (on non-simulator builds)

Note: Thread-safe: Caller must ensure no other threads are using the session] **See also:** `rx_session_init()` Initialization counterpart

Figure 1083: Call graph for `rx_session_deinit`

Defined in `libs/rx_session/src/rx_session.c:232`

Since Version 1.0.0

—

rx_session_next_tx

```
rx_err_t rx_session_next_tx(rx_session_state_t *state, uint16_t *sequence)
```

Get next TX sequence number and increment the counter.

Atomically reads the current TX sequence, increments it (with wraparound at `k_session_seq_wrap_mask`), and returns the pre-increment value. This is used by both USB and SPI transport layers when sending frames.

Mirrors Go gateway's `SessionState.NextTxSequence()`.

Parameters:

Direction	Name	Description
inout	state	Session state (must be initialized)
out	sequence	Pointer to receive the sequence number

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: state must be initialized via rx_session_init()**Pre:** sequence must point to valid memory**Post:** state->tx_sequence incremented by 1 (with wraparound)**Post:** *sequence contains the pre-increment value**Post:** *sequence <= k_session_seq_wrap_mask**Note:** Thread-safe: Protected by internal mutex] **See also:** rx_session_get_tx() Read without incrementing

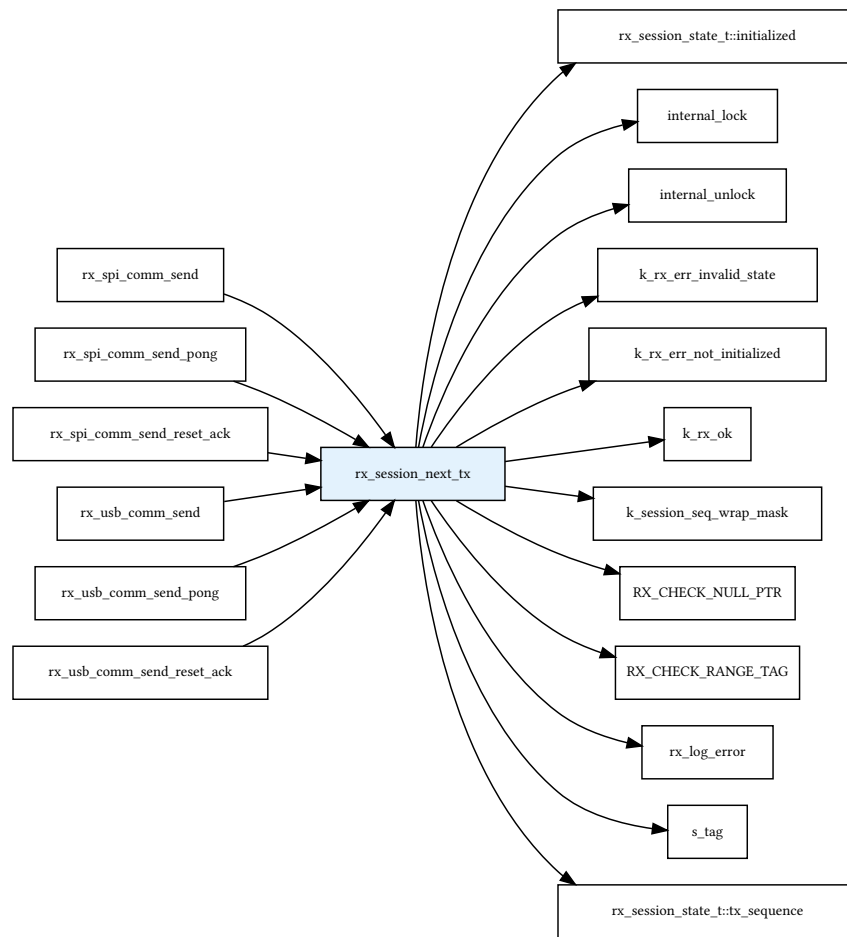


Figure 1084: Call graph for rx_session_next_tx

Defined in `libs/rx_session/src/rx_session.c:283`

Since Version 1.0.0

—

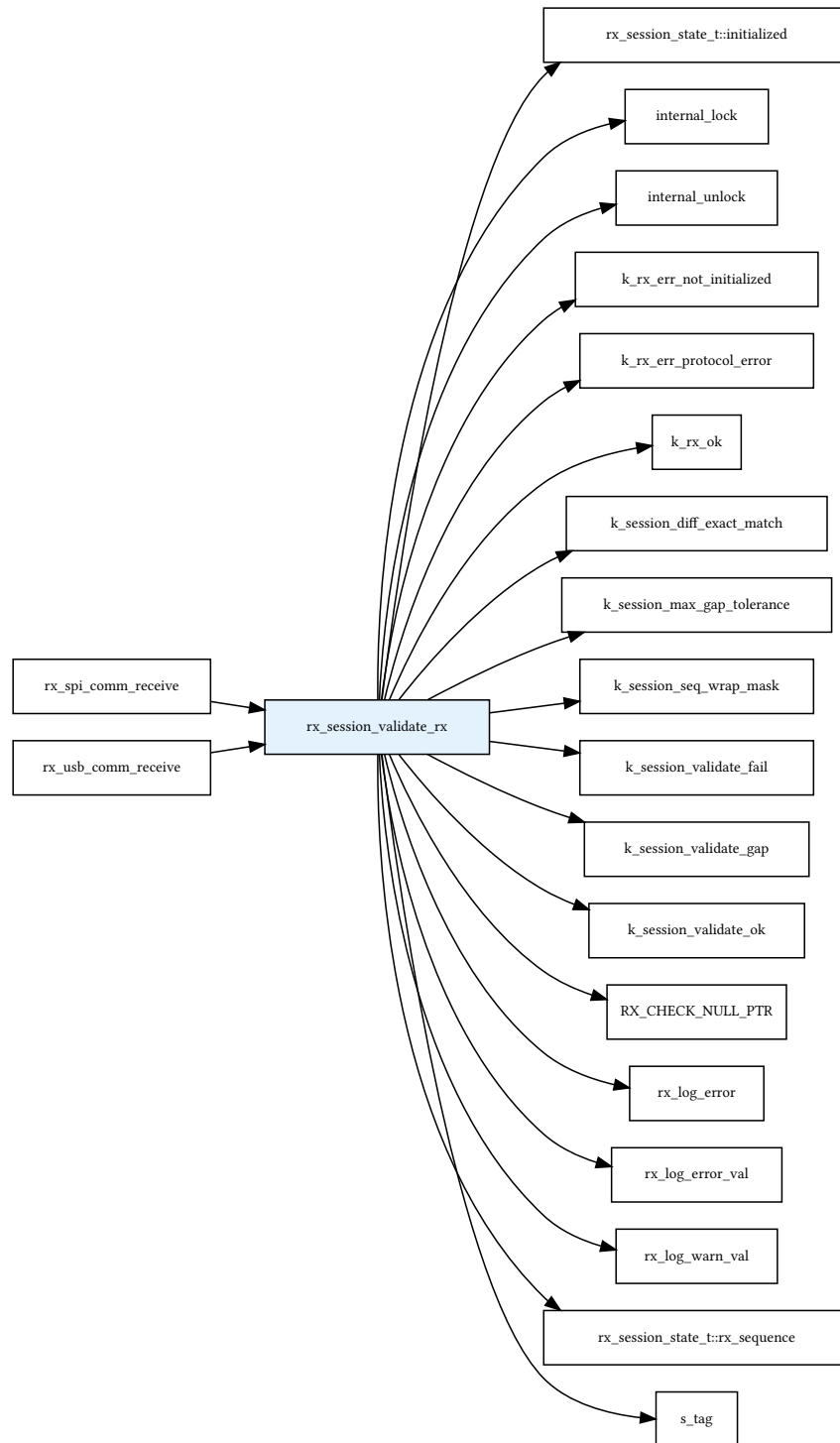
rx_session_validate_rx

```
rx_err_t rx_session_validate_rx(rx_session_state_t *state, uint16_t received_seq,
rx_session_validate_result_t *result)
```

Validate a received sequence number against expected RX sequence.

Validate a received sequence number.

Checks whether the received sequence number is acceptable based on the expected RX sequence. Implements gap tolerance matching the Go gateway's `SessionState.ValidateRxSequence()`.

Figure 1085: Call graph for `rx_session_validate_rx`

Defined in `libs/rx_session/src/rx_session.c:344`

—

rx_session_reset

```
rx_err_t rx_session_reset(rx_session_state_t *state)
```

Reset both TX and RX sequences to initial values.

Reset both TX and RX sequences to zero.

Called after a RESET/RESET_ACK handshake completes to synchronize sequences between firmware and gateway. Both counters are atomically set to `k_session_initial_sequence`.

Mirrors Go gateway's `SessionState.Reset()`.

Parameters:

Direction	Name	Description
inout	state	Session state (must be initialized)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, sequences reset
<code>k_rx_err_null_ptr</code>	state is nullptr
<code>k_rx_err_not_initialized</code>	state not initialized
<code>k_rx_err_rtos_mutex</code>	Mutex acquisition failed

Pre: state must be initialized via `rx_session_init()`

Post: `state->tx_sequence == k_session_initial_sequence`

Post: `state->rx_sequence == k_session_initial_sequence`

Note: Thread-safe: Protected by internal mutex] **See also:**
`rx_frame_type_t::k_frame_type_reset` RESET frame triggers this,
`rx_frame_type_t::k_frame_type_reset_ack` RESET_ACK confirms reset

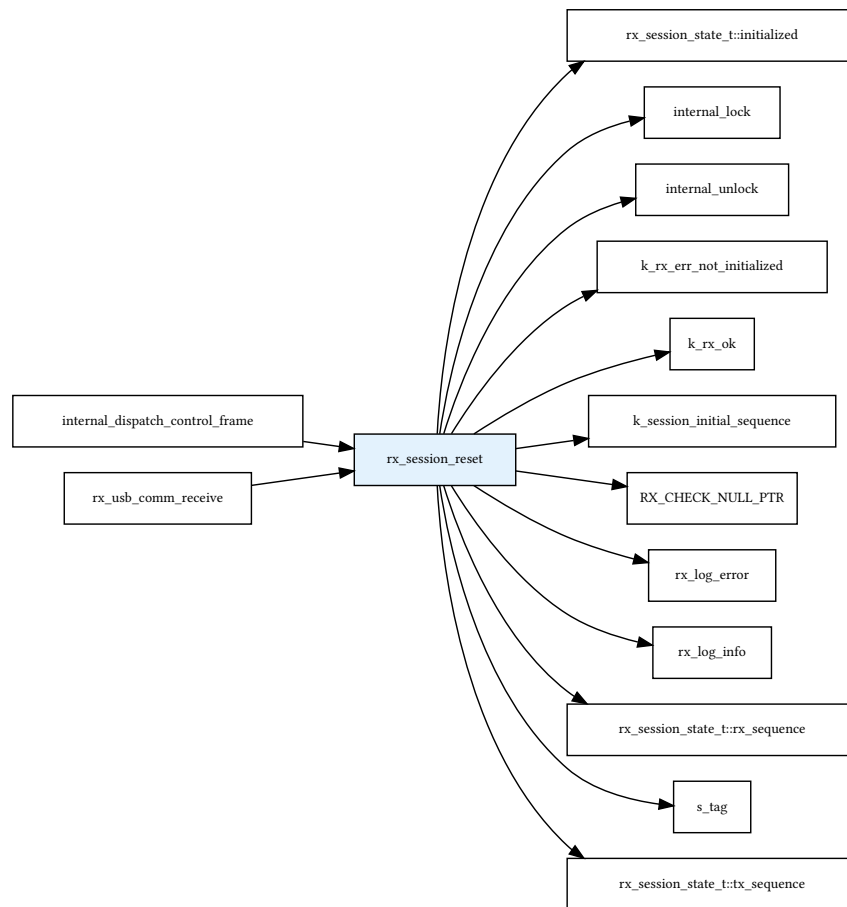


Figure 1086: Call graph for rx_session_reset

Defined in `libs/rx_session/src/rx_session.c:424`

Since Version 1.0.0

—

rx_session_get_tx

```
rx_err_t rx_session_get_tx(const rx_session_state_t *state, uint16_t *sequence)
```

Get current TX sequence without incrementing.

Returns the current TX sequence counter value for diagnostic purposes. Does not modify the counter.

Parameters:

Direction	Name	Description
in	state	Session state (must be initialized)

out	sequence	Pointer to receive the current TX sequence
-----	----------	--

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

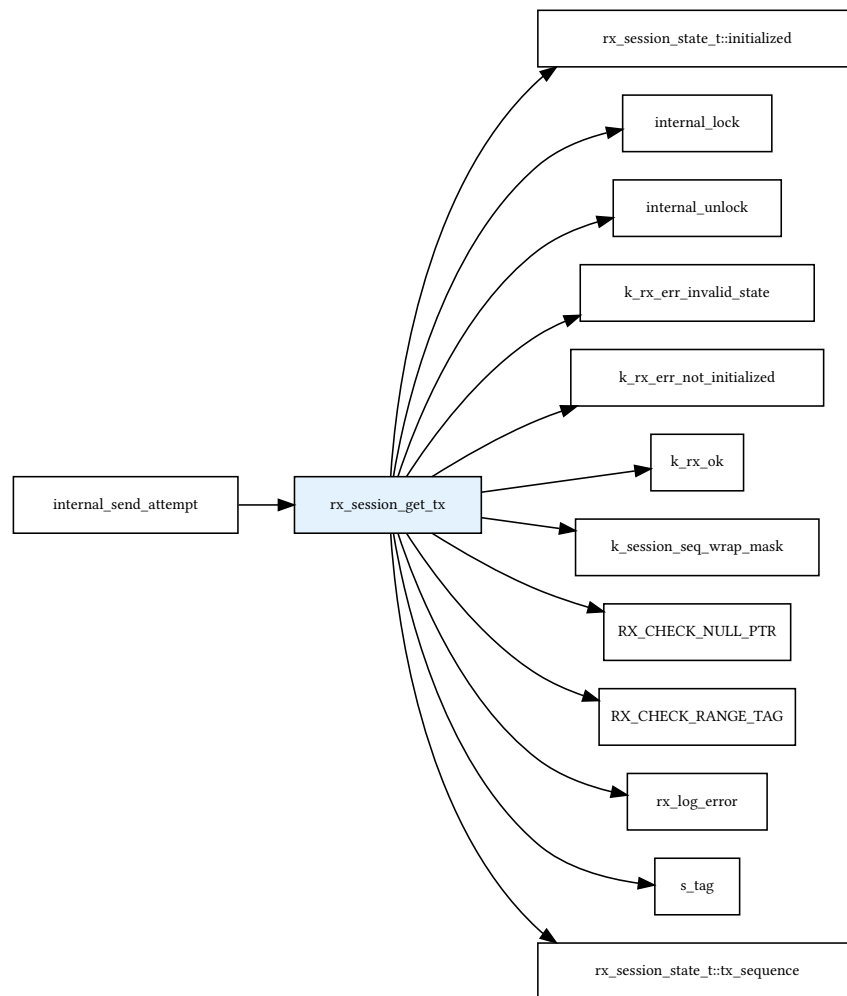
Pre: state must be initialized via rx_session_init()

Pre: sequence must point to valid memory

Post: state unchanged

Post: *sequence <= k_session_seq_wrap_mask

Note: Thread-safe: Protected by internal mutex] **See also:** rx_session_next_tx() Read and increment

Figure 1087: Call graph for `rx_session_get_tx`

Defined in `libs/rx_session/src/rx_session.c:472`

Since Version 1.0.0

—

rx_session_get_rx

```
rx_err_t rx_session_get_rx(const rx_session_state_t *state, uint16_t *sequence)
```

Get current expected RX sequence without modifying.

Get current expected RX sequence.

Returns the next expected RX sequence counter value for diagnostic purposes. Does not modify the counter.

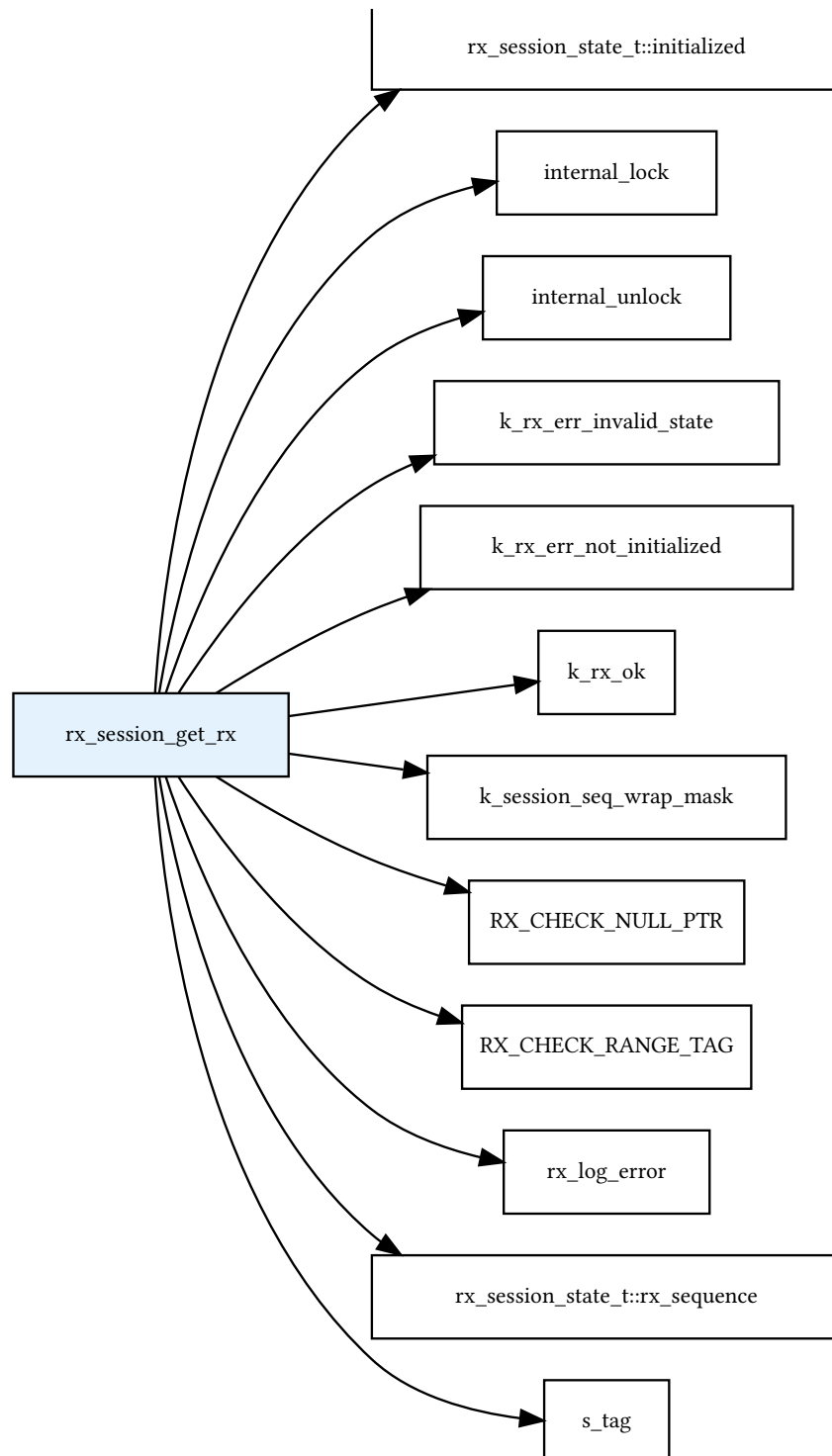
Parameters:

Direction	Name	Description
in	state	Session state (must be initialized)
out	sequence	Pointer to receive the current RX sequence

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, sequence written
k_rx_err_null_ptr	state or sequence is nullptr
k_rx_err_not_initialized	state not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: state must be initialized via rx_session_init()**Pre:** sequence must point to valid memory**Post:** state unchanged**Post:** *sequence <= k_session_seq_wrap_mask**Note:** Thread-safe: Protected by internal mutex] **See also:** rx_session_validate_rx()
Validate and update

Figure 1088: Call graph for `rx_session_get_rx`

Defined in `libs/rx_session/src/rx_session.c:522`

Since Version 1.0.0

—

rx_spi_comm.h

High-Level SPI Frame Protocol Communication Layer for RX72N Peripheral Mode.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"hardware.h"`
- `"rx_err.h"`
- `"rx_frame.h"`
- `"rx_session.h"`

rx_spi_comm_constants_t

SPI communication configuration constants and buffer sizes.

Defines compile-time constants for SPI channel selection, buffer sizes, and default timeout values. All sizes chosen to accommodate maximum frame size (`k_frame_max_total_size` = 263 bytes) with safety margin.

Design Rationale:

- 2048-byte buffers: Allows 7 maximum-size frames to be buffered, providing tolerance for burst traffic and preventing frame loss.
- 10 MHz SPI clock: Maximum reliable speed for RPi5 spidev with 1m cable.
- 1 second timeout: Conservative default for reliable operation.

Value	Name	Description
= 0	<code>k_spi_comm_default_mode</code>	Default SPI mode (CPOL=0, CPHA=0).
= 2048	<code>k_spi_comm_rx_buffer_size</code>	RX staging buffer size in bytes.
= 2048	<code>k_spi_comm_tx_buffer_size</code>	TX staging buffer size in bytes.
= 1000	<code>k_spi_comm_default_timeout</code>	Default timeout in milliseconds for blocking operations.

Since Version 1.0.0

—

rx_retransmit_defaults_retries_t

Default retry count for automatic retransmission.

Maximum number of retransmission attempts before giving up. Applied when `auto_retransmit` is enabled but `max_retries` is zero.

Value	Name	Description
= 3	<code>k_retransmit_default_max_retries</code>	Default max retransmit attempts

Since Version 1.1.0

—

rx_retransmit_defaults_timing_t

Default timing values for automatic retransmission.

Timing parameters for ACK timeout and exponential backoff cap. Applied when auto_retransmit is enabled but timing values are zero.

Value	Name	Description
= 50	k_retransmit_default_ack_timeout_ms	Initial ACK wait in ms
= 400	k_retransmit_default_max_backoff_ms	Max exponential backoff cap in ms

Since Version 1.1.0

—

rx_spi_comm_init

```
rx_err_t rx_spi_comm_init(rx_spi_comm_handle_t *handle, const
rx_spi_comm_config_t *config)
```

Initialize SPI communication handle with frame protocol support.

Performs complete initialization of SPI communication handle including frame encoder/decoder setup, sequence number initialization, and optional FEC configuration. This function MUST be called before any send/receive operations.

IMPORTANT: This function does NOT initialize SPI hardware. Caller must initialize RSPi peripheral via `rspi_init_peripheral()` BEFORE calling this function.

Algorithm:

1. Validate parameters: Check handle pointer for nullptr (NASA Rule 5)
2. Clear handle: Zero-fill entire structure (4120 bytes)
3. Apply configuration: Copy config fields if non-NULL, else use defaults
4. Initialize frame encoder: Setup CRC-32 and header encoding
5. Initialize frame decoder: Setup CRC-32 validation and header parsing
6. Initialize sequences: Set tx_sequence = 0, rx_sequence = 0
7. Mark initialized: Set handle->initialized = true

Configuration Options (config is REQUIRED, not optional):

- session: REQUIRED, pointer to initialized rx_session_state_t
- channel: 0-2 (RSPi0/1/2), default 0
- spi_mode: 0-3 (SPI modes), default 0
- fec_enabled: true/false, default false

Initialize SPI communication handle with frame protocol support.

Initializes SPI communication handle for frame-based communication between RX72N (peripheral) and RPi5 (controller). Sets up frame encoder/decoder, clears staging buffers, and configures channel and FEC settings.

Initialization Steps:

1. Validate handle: Check handle pointer non-nullptr
2. Zero handle: memset entire structure to clear old state
3. Apply configuration: Use config values or defaults (RSPi0, no FEC)
4. Initialize encoder: Call rx_frame_encoder_init() for TX encoding

5. Initialize decoder: Call `rx_frame_decoder_init()` for RX decoding
6. Reset sequence counters: Set TX/RX sequence to 0
7. Mark initialized: Set `handle->initialized = true`

Configuration (config must be non-nullptr):

- Channel: Must specify RSPI channel (e.g., `k_rspi_channel_2` for host)
- FEC: `fec_enabled` flag (true/false)
- Passing `config == nullptr` returns `k_rx_err_invalid_arg`

Memory Allocation: This function does NOT allocate memory (NASA Rule 3 compliance). Caller must provide pre-allocated handle (typically static or on task stack).

Parameters:

Direction	Name	Description
out	handle	Pointer to SPI communication handle to initialize Valid range: Non-nullptr to allocated <code>rx_spi_comm_handle_t</code> (4120 bytes) Constraints: Must be allocated by caller (typically static or global) Side effects: Entire structure zero-filled, then populated from config Lifetime: Must remain valid until <code>rx_spi_comm_deinit()</code> called
in	config	Required configuration structure (must not be NULL) Valid range: Non-nullptr to valid <code>rx_spi_comm_config_t</code> Constraints: session must be non-NULL, channel must be 0-2, <code>spi_mode</code> must be 0-3 Lifetime: Not stored - values copied into handle (except session ptr)
out	handle	SPI communication handle to initialize (4120 bytes) Valid range: Non-nullptr pointer to <code>rx_spi_comm_handle_t</code> Allocation: Caller-allocated (static, stack, or global) Side effects: Entire structure zeroed and re-initialized
in	config	Configuration parameters (must not be nullptr) Valid range: Non-nullptr pointer to <code>rx_spi_comm_config_t</code> Required fields: session (non-nullptr), channel (0-2), <code>fec_enabled</code> (true/false)

Returns: `rx_err_t` Error code indicating initialization result

Value	Description
<code>k_rx_ok</code>	Success - handle fully initialized and ready for operation
<code>k_rx_err_invalid_arg</code>	handle is nullptr
<code>k_rx_err_invalid_arg</code>	config is nullptr
<code>k_rx_err_invalid_arg</code>	config->session is nullptr
<code>k_rx_err_invalid_arg</code>	config->channel > 2 (invalid RSPI channel)
<code>k_rx_err_invalid_arg</code>	config->spi_mode > 3 (invalid SPI mode)
<code>k_rx_ok</code>	Initialization successful, handle ready for use
<code>k_rx_err_invalid_arg</code>	handle pointer is nullptr
(encoder	errors) <code>rx_frame_encoder_init()</code> failed (rare, internal error)
(decoder	errors) <code>rx_frame_decoder_init()</code> failed (rare, internal error)

Pre: handle must point to allocated memory (uninitialized content OK)

Pre: RSPI hardware must already be initialized via `rspi_init_peripheral()`

Pre: handle must point to valid `rx_spi_comm_handle_t` storage (4120 bytes)

Pre: `config != nullptr`

Pre: `config->session != nullptr`

Pre: `config->channel` must be a valid `rspi_channel_t` value (0-2)

Post: `handle->initialized = true` on success

Post: `handle->session` points to `config->session`

Post: All buffers zero-filled (`rx_buffer`, `tx_buffer`)

Post: On success, `handle->initialized = true`

Post: On success, `handle->tx_sequence = 0`, `handle->rx_sequence = 0`

Post: On success, `handle->encoder` and `handle->decoder` initialized

Post: On failure (encoder init), handle state is zeroed but not usable

Post: On failure (decoder init), encoder is cleaned up, handle unusable

Note: This function does NOT initialize SPI hardware - caller responsible

Note: On success, handle is ready for `rx_spi_comm_send/receive` operations

Note: Thread-safe ONLY if each thread uses a different handle

Note: Call this ONCE per handle before any send/receive operations

Note: This function is NOT thread-safe (single-threaded init only)

Warning: Must call `rspi_init_peripheral()` BEFORE this function

Warning: Do not call multiple times without `rx_spi_comm_deinit()` first

Warning: RSPI peripheral hardware (rspi HAL) must be initialized separately

Performance:: Execution time: 10 us @ 240 MHz Dominated by memset (4120 bytes)

Example:: `static rx_spi_comm_handle_tg_spi_handle; static rx_session_state_tg_session;`
`rx_err_t err = rx_session_init(&g_session); if(err != k_rx_ok) return err;`
`rx_spi_comm_config_t comm_cfg = { .session = &g_session, .channel = 0, .spi_mode = 0, .fec_enabled = false };`
`err = rx_spi_comm_init(&g_spi_handle, &comm_cfg); if(err != k_rx_ok) { log_error("SPIcomm init failed: %d", err); return err; }`

Performance:: Execution time: 50 us @ 240 MHz (memset + encoder/decoder init)

Example - Basic Configuration:: `static rx_spi_comm_handle_tg_spi_handle;`
`rx_spi_comm_config_t config = { .session = &session, .channel = k_rspi_channel_0, .fec_enabled = false, };`
`rx_err_t err = rx_spi_comm_init(&g_spi_handle, &config); if(err != k_rx_ok)`
`{ rx_log_error("init", "SPIcomm init failed"); return; }`
`rx_spi_comm_send(&g_spi_handle, k_frame_type_data, 0, payload, len);`

Example - FEC Enabled:: `static rx_spi_comm_handle_tg_spi_handle;`
`rx_spi_comm_config_t config = { .channel = 0, .fec_enabled = true, };`
`rx_err_t err = rx_spi_comm_init(&g_spi_handle, &config); if(err == k_rx_ok)`
`{ rx_log_info("init", "SPIcomm initialized with FEC"); }`

Example - Multiple RSPI Channels:: `static rx_spi_comm_handle_tg_spi1_handle;`
`rx_spi_comm_config_t config = { .channel = 1, .fec_enabled = false, };`
`rx_spi_comm_init(&g_spi1_handle, &config);`

See also: `rx_spi_comm_deinit()` Cleanup and resource release, `rx_spi_comm_send()` Send frame, `rx_spi_comm_receive()` Receive frame, `rspi_init_peripheral()` Initialize SPI hardware (must call first), `rx_spi_comm_deinit()` Clean up resources when done, `rx_spi_comm_config_t` Configuration structure definition, `rx_frame_encoder_init()` Frame encoder initialization, `rx_frame_decoder_init()` Frame decoder initialization

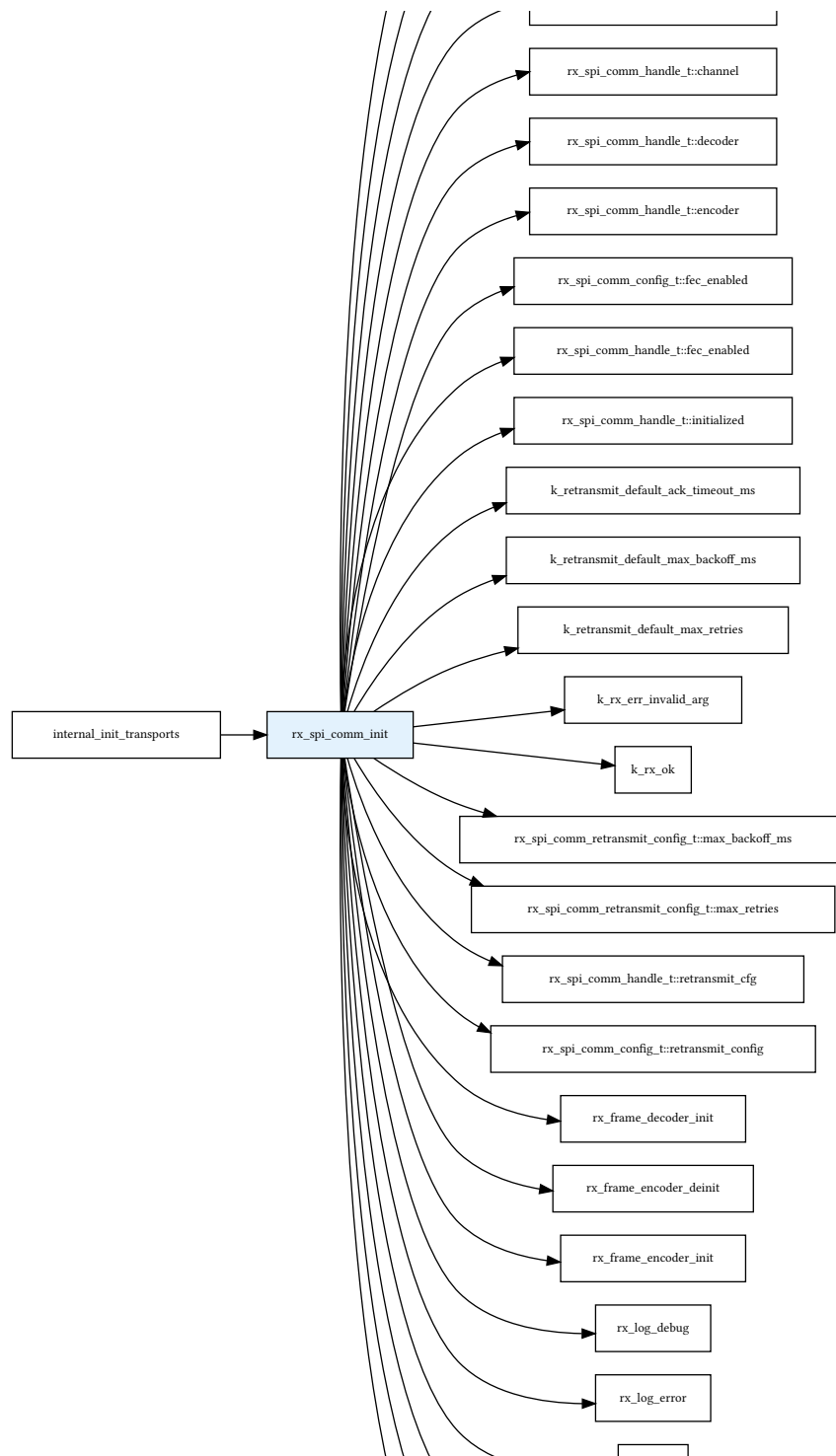


Figure 1089: Call graph for rx_spi_comm_init

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:923`

Since Version 1.0.0

—

rx_spi_comm_deinit

```
rx_err_t rx_spi_comm_deinit(rx_spi_comm_handle_t *handle)
```

Deinitialize SPI communication handle and release resources.

Marks SPI communication handle as uninitialized, preventing further operations. This function does NOT deinitialize RSPI hardware - caller must separately call `rspi_deinit()` if needed.

Algorithm:

1. Validate handle pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated in handle structure). Function primarily serves as safety guard to prevent use-after-deinit.

Deinitialize SPI communication handle and release resources.

Tears down the encoder and decoder, clearing internal state. Saves the first error encountered from sub-deinit calls and returns it after completing all teardown steps.

Parameters:

Direction	Name	Description
inout	handle	Pointer to SPI communication handle Valid range: Non-nullptr to initialized handle Constraints: Must have been initialized via <code>rx_spi_comm_init()</code> Side effects: Clears <code>handle->initialized</code> flag
inout	handle	SPI communication handle to deinitialize

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success - handle deinitialized
<code>k_rx_err_invalid_arg</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle was not initialized
<code>k_rx_ok</code>	Deinit completed successfully
<code>k_rx_err_invalid_arg</code>	handle is nullptr
(other)	First error from encoder or decoder deinit

Pre: handle must be non-NULL

Pre: handle must be initialized (`handle->initialized == true`)

Pre: handle != nullptr

Pre: handle was previously initialized via rx_spi_comm_init()

Post: handle->initialized = false

Post: Subsequent calls to rx_spi_comm_send/receive will return k_rx_err_invalid_state

Post: handle->initialized == false

Post: Encoder and decoder resources released

Note: Does NOT free memory (handle is statically allocated)

Note: Does NOT deinitialize RSPI hardware

Note: Thread-safe if no concurrent operations on same handle

Note: Not thread-safe; caller must ensure no concurrent SPI operations.

Example:: rx_err_t err=rx_spi_comm_deinit(&g_spi_handle); if(err==k_rx_ok){
rspi_deinit(g_spi_handle.channel); }

See also: rx_spi_comm_init() Initialization function, rspi_deinit() Deinitialize RSPI hardware, rx_spi_comm_init() Corresponding initialization function

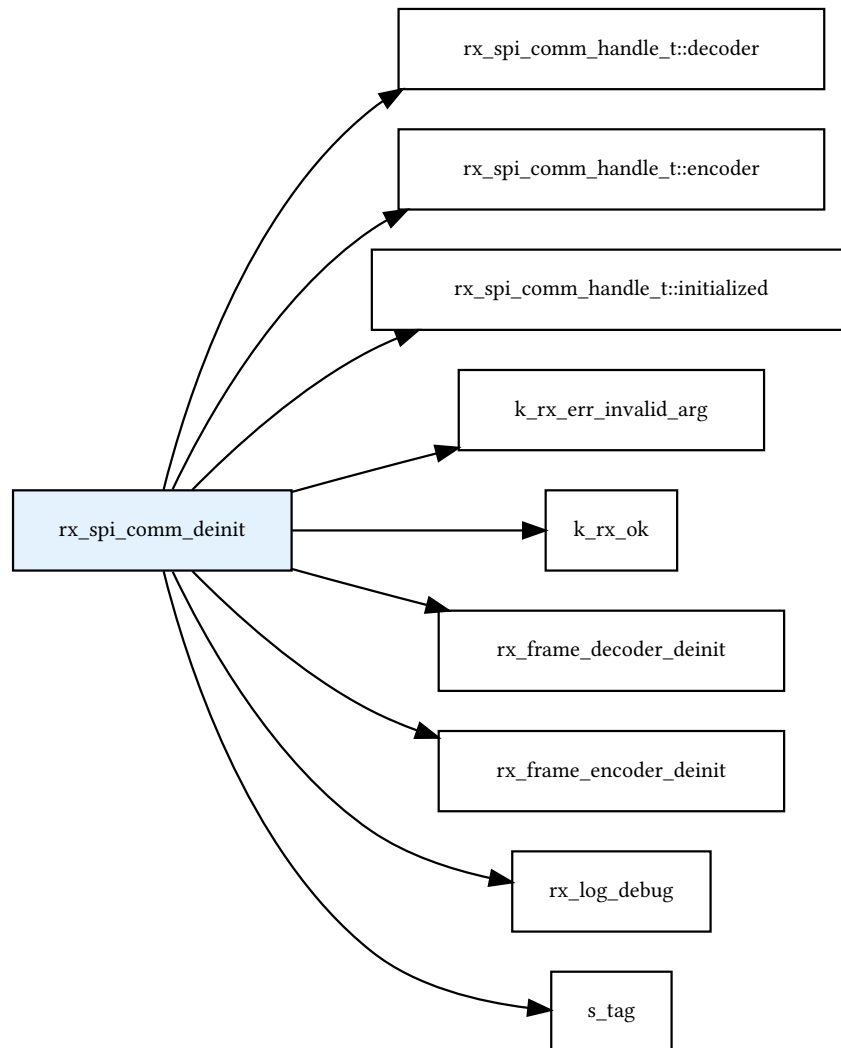


Figure 1090: Call graph for rx_spi_comm_deinit

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:979`

Since Version 1.0.0

—

rx_spi_comm_send

```
rx_err_t rx_spi_comm_send(rx_spi_comm_handle_t *handle, rx_frame_type_t type,
uint8_t flags, const uint8_t *payload, uint32_t payload_len)
```

Send frame on SPI with automatic sequencing and CRC.

Encodes payload into frame format with automatic sequence number assignment, CRC-32 calculation, and optional FEC encoding. Transmits frame to RPi5 via SPI hardware. This is the primary send function for all frame types.

Algorithm:

1. Validate parameters: Check handle, payload, payload_len (NASA Rule 5)
2. Assign sequence: Get next TX sequence from shared rx_session_state_t
3. Build frame: Create frame header (type, flags, length, sequence)
4. Encode frame: Call frame encoder to add CRC-32
5. Optional FEC: If enabled, apply FEC encoding
6. SPI transfer: Transmit encoded frame via RSPI peripheral
7. Update state: TX sequence already incremented by rx_session_next_tx()

Performance:

- Small payload (16 bytes): 150 us
- Medium payload (128 bytes): 300 us
- Large payload (255 bytes): 500 us
- FEC overhead: +50% time

Send frame on SPI with automatic sequencing and CRC.

Transmits application payload to RPi5 by building frame structure, encoding to wire format with CRC-32, waiting for host ready signal, performing SPI transfer, and incrementing TX sequence counter. This is the primary transmit function for all data frames.

Algorithm:

1. Pre-condition 1: Validate handle pointer non-nullptr
2. Pre-condition 2: Validate handle initialized
3. Build frame: Call internal_build_frame() to populate rx_frame_t

Sets sequence number from handle->tx_sequence Copies payload (0-1024 bytes) Sets type, flags, applies FEC flag if configured

4. Encode frame: Call rx_frame_encode() to create wire buffer

Adds sync word (0xAA55) Encodes header fields (little-endian) Computes CRC-32 over header + payload Appends CRC-32 (little-endian)

5. Post-condition 1: Validate encoded length in valid range (14-279 bytes)
6. Transfer via SPI: Call internal_spi_transfer()

Waits for RPi5 ready signal (50ms timeout) Performs DMA/IRQ-based SPI transfer

7. Post-condition 2: Increment TX sequence counter

handle->tx_sequence++ (wraps at 65536) Validates increment successful (NASA Rule 5)

Frame Format on Wire (transmitted bytes):

Sequence Number Behavior:

- Auto-incremented after each successful send
- Starts at 0, wraps to 0 after 65535
- Receiver uses sequence to detect duplicates, out-of-order, missing frames

Parameters:

Direction	Name	Description
-----------	------	-------------

inout	handle	Initialized SPI communication handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_spi_comm_init() Side effects: Increments shared session TX sequence via rx_session_next_tx()
in	type	Frame type (command, request, response, etc.) Valid range: Any rx_frame_type_t value Common values: k_frame_type_response (most common for RX72N) Units: Enumeration value
in	flags	Frame flags (ACK, NACK, etc.) Valid range: Bitmask of rx_frame_flag_t values Common values: k_frame_flag_none (0x00) Units: Bitmask
in	payload	Payload data to send Valid range: Non-NULL if payload_len > 0, may be NULL if payload_len == 0 Constraints: Must contain valid data for payload_len bytes Lifetime: Copied immediately, does not need to persist Max size: k_frame_max_payload (255 bytes)
in	payload_len	Payload length in bytes Valid range: [0, k_frame_max_payload] (0-255 bytes) Units: Bytes Constraints: Must be <= k_frame_max_payload
inout	handle	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Side effects: handle->tx_sequence incremented on success Usage: Uses handle->tx_buffer for staging (2048 bytes)
in	type	Frame type from rx_frame_type_t enum Valid values: k_frame_type_data, k_frame_type_telemetry, k_frame_type_command Common: k_frame_type_data (general purpose) Note: Use rx_spi_comm_send_ack/nack() for ACK/NACK frames
in	flags	Frame flags bitmask Valid bits: k_frame_flag_fec_enabled, k_frame_flag_harq_enabled Typical: 0 (no special flags, FEC auto-added if configured) Auto-set: k_frame_flag_fec_enabled added if handle->fec_enabled
in	payload	Application data to send (or nullptr if no payload) Valid range: nullptr or pointer to payload_len bytes nullptr semantics: Zero-length frame (14 bytes on wire) Constraints: Must contain payload_len valid bytes if non-nullptr
in	payload_len	Payload length in bytes Valid range: 0-1024 bytes (k_frame_max_payload) Typical: 8-64 (telemetry), 100-200 (bulk data) Zero: Valid for ping/keepalive frames

Returns: rx_err_t Error code indicating send result

Value	Description
k_rx_ok	Frame sent successfully, tx_sequence incremented
k_rx_err_invalid_arg	handle or payload is nullptr
k_rx_err_invalid_arg	payload_len > k_frame_max_payload (255 bytes)
k_rx_err_invalid_arg	payload is nullptr but payload_len > 0

k_rx_err_invalid_state	handle not initialized
k_rx_err_hw	SPI hardware error (RSPI transfer failed)
k_rx_err_timeout	SPI transfer timeout
k_rx_ok	Frame sent successfully, TX sequence incremented
k_rx_err_invalid_arg	handle pointer is nullptr
k_rx_err_invalid_state	handle not initialized (call rx_spi_comm_init first)
k_rx_err_invalid_size	payload_len > 1024 (protocol violation)
k_rx_err_invalid_size	Encoded frame length invalid (internal error)
k_rx_err_timeout	RPi5 ready signal timeout (host not responding)
(encoder	errors) rx_frame_encode() failed (rare, internal error)
(HAL	errors) RSPI peripheral transfer failure (overflow, mode fault)

Pre: handle must be non-NULL and initialized

Pre: If payload_len > 0, payload must point to valid data

Pre: RSPI hardware must be initialized and ready

Pre: handle must be initialized via rx_spi_comm_init()

Pre: If payload != nullptr, payload_len must be > 0 and <= 1024

Pre: If payload == nullptr, payload_len must be 0

Pre: RSPI peripheral hardware must be initialized and enabled

Pre: RPi5 controller must be ready to receive (CS asserted, clock provided)

Post: Shared session TX sequence incremented (wraps at 65535)

Post: Frame transmitted to RPi5 if successful

Post: On success, handle->tx_sequence incremented (wraps at 65536)

Post: On success, frame transmitted to RPi5

Post: On failure, TX sequence NOT incremented (safe to retry)

Post: On timeout, RPi5 may be disconnected or busy

Note: Sequence number managed automatically - caller does not specify

Note: CRC-32 calculated automatically by frame encoder

Note: FEC applied automatically if handle->fec_enabled = true

Note: This function blocks until transfer complete or timeout (1-50 ms)

Note: TX sequence auto-increments, no manual management needed

Warning: Ensure RPi5 is ready to receive (chip select asserted)

Warning: Not thread-safe - use external mutex if multiple threads send

Warning: Do not modify payload buffer during call (memcpy in progress)

Example - Send Telemetry Response:: uint8_ttelemetry[16];

```
rx_err_tterr=rx_spi_comm_send(&g_spi_handle, k_frame_type_response, k_frame_flag_none,
telemetry, sizeof(telemetry)); if(err==k_rx_ok){ }else{ log_error("Sendfailed:%d",err); }
```

Example - Send Empty ACK (No Payload):: rx_err_tterr=rx_spi_comm_send(&g_spi_handle, k_frame_type_response, k_frame_flag_ack, NULL, 0);

Performance:: Execution time: 150-500 us @ 10 MHz SPI + 0-50 ms host ACK wait 10-byte payload: 150 us (10 us build + 20 us encode + 120 us SPI) 100-byte payload: 300 us (15 us build + 100 us encode + 185 us SPI) 1024-byte payload: 800 us (20 us build + 250 us encode + 530 us SPI) Host ACK wait: Add 0-50 ms if RPi5 not immediately ready

Example - Send Telemetry Data:: uint8_ttelemetry[32]; telemetry[0]=temperature_celsius&0xFF; telemetry[1]=(temperature_celsius>>8)&0xFF;

```
rx_err_tterr=rx_spi_comm_send(&g_spi_handle, k_frame_type_telemetry, 0, telemetry,
sizeof(telemetry)); if(err==k_rx_ok)
{ rx_log_debug("spi", "Telemetrysent,seq=%d",g_spi_handle.tx_sequence-1); }
elseif(err==k_rx_err_timeout){ rx_log_warn("spi", "RPi5notresponding"); }
```

Example - Send Command with Retry:: uint8_tcmd[]={0x01,0x02,0x03};

```
for(uint8_tretry=0;retry<3;retry++){ rx_err_tterr=rx_spi_comm_send(&g_spi_handle,
k_frame_type_command, 0, cmd, sizeof(cmd)); if(err==k_rx_ok){ break; }
elseif(err==k_rx_err_timeout){ rx_log_warn("spi", "Timeout,retry%d/3",retry+1);
tx_thread_sleep(10); }else{ rx_log_error("spi", "Sendfailed:%d",err); break; } }
```

Example - Zero-Length Frame (Ping):: rx_err_tterr=rx_spi_comm_send(&g_spi_handle, k_frame_type_data, 0, nullptr, 0);

Example:

```
+-----+-----+-----+-----+-----+-----+
| SYNC | Sequence | Length | Type | Flags | Payload | CRC-32 |
| 2B   | 2B       | 2B     | 1B   | 1B   | 0-1024B | 4B   |
| 0x55AA | LE | LE | | | LE |
+-----+-----+-----+-----+-----+-----+
```

See also: rx_spi_comm_send_ack() Convenience wrapper for ACK frames,
rx_spi_comm_send_nack() Convenience wrapper for NACK frames, rx_spi_comm_receive()
Receive frames from RPi5, rx_frame.h Frame protocol definitions, rx_spi_comm_send_ack()
Send acknowledgment frame, rx_spi_comm_send_nack() Send negative acknowledgment,
rx_spi_comm_receive() Receive frames from RPi5, internal_build_frame() Frame building
implementation, internal_spi_transfer() SPI transfer with flow control

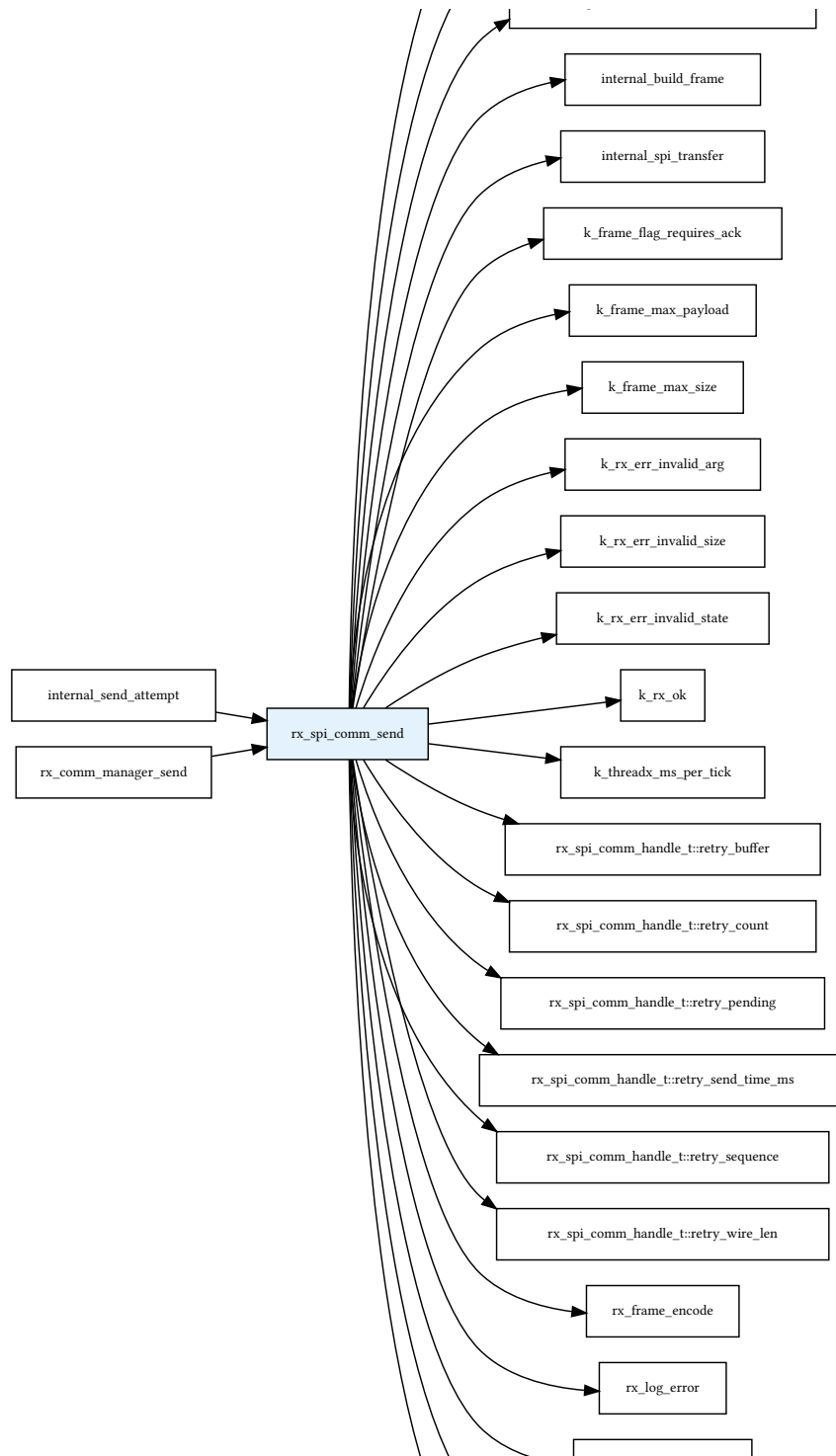


Figure 1091: Call graph for rx_spi_comm_send

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1089`

Since Version 1.0.0

—

rx_spi_comm_send_ack

```
rx_err_t rx_spi_comm_send_ack(rx_spi_comm_handle_t *handle, uint16_t sequence)
```

Send ACK frame for received frame (convenience wrapper).

Convenience function for sending acknowledgment frames. Automatically sets frame type to `k_frame_type_response` and flags to `k_frame_flag_ack`. Includes the sequence number being acknowledged in the frame.

Use Case: Acknowledge successful receipt of a frame from RPi5.

Algorithm:

1. Build ACK frame with type=response, flags=ack
2. Include sequence number in payload (2 bytes, little-endian)
3. Call `rx_spi_comm_send()` with constructed frame

Performance: 50 us (minimal frame with 2-byte payload)

Send ACK frame for received frame (convenience wrapper).

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via <code>rx_spi_comm_init()</code>
in	sequence	Sequence number to acknowledge Valid range: [0, 65535] Units: Sequence number (uint16_t) Purpose: Tells RPi5 which frame is being acknowledged
inout	handle	SPI communication handle
in	sequence	Sequence number to acknowledge

Returns: Error code from frame encoding or SPI transfer on failure

Value	Description
<code>k_rx_ok</code>	ACK sent successfully
<code>k_rx_err_invalid_arg</code>	handle is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized
(other)	Errors propagated from <code>rx_spi_comm_send()</code>

Pre: Same preconditions as `rx_spi_comm_send()`

Post: ACK frame transmitted with specified sequence number

Note: Equivalent to calling `rx_spi_comm_send()` with `type=response`, `flags=ack`

Note: This is a thin wrapper - no additional validation performed

Example:: `rx_frame_tframe; rx_err_t err=rx_spi_comm_receive(&g_spi_handle,&frame,0);`
`if(err!=k_rx_ok){ handle_frame(&frame);`
`err=rx_spi_comm_send_ack(&g_spi_handle,frame.header.sequence); }`

See also: `rx_spi_comm_send()` Full send function, `rx_spi_comm_send_nack()` Send NACK for errors

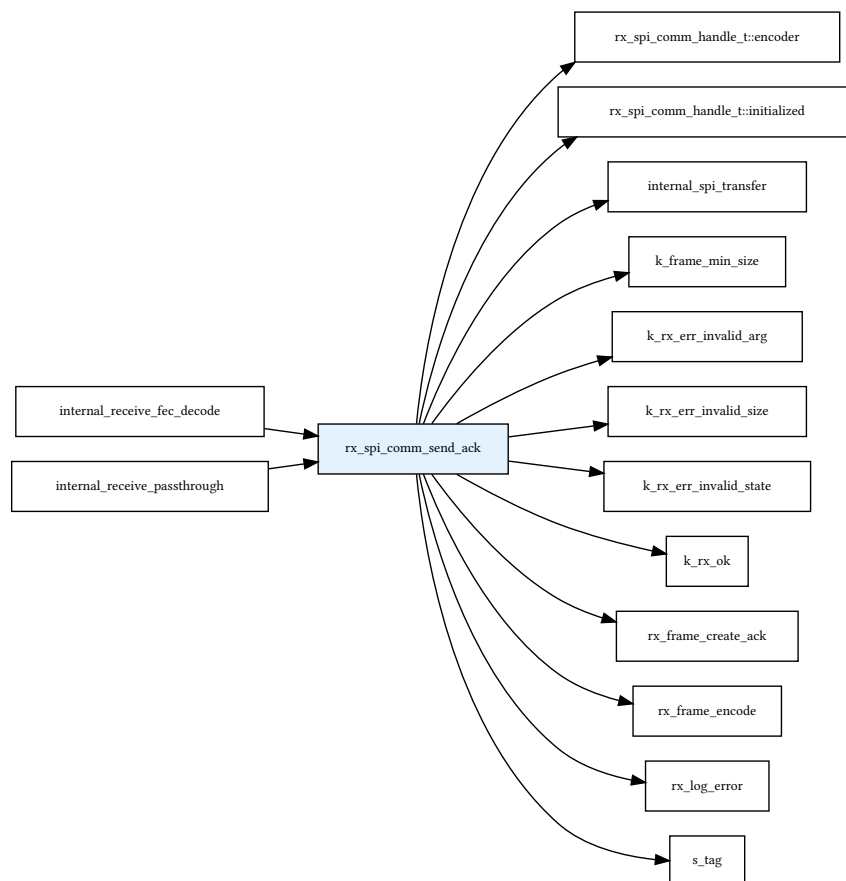


Figure 1092: Call graph for `rx_spi_comm_send_ack`

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1152`

Since Version 1.0.0

rx_spi_comm_send_nack

```
rx_err_t rx_spi_comm_send_nack(rx_spi_comm_handle_t *handle, uint16_t sequence,
uint8_t flags)
```

Send NACK frame for received frame with error indication.

Convenience function for sending negative acknowledgment frames. Used to request retransmission when frame reception fails (CRC error, corruption, etc.).

Use Cases:

- CRC validation failure (hard NACK)
- FEC decoding failure (soft NACK if HARQ enabled)
- Out-of-order sequence detection
- Protocol violation

Algorithm:

1. Build NACK frame with type=response, flags=nack | additional_flags
2. Include sequence number in payload (2 bytes, little-endian)
3. Call rx_spi_comm_send() with constructed frame

Performance: 50 us (minimal frame with 2-byte payload)

Send NACK frame for received frame with error indication.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_spi_comm_init()
in	sequence	Sequence number being NACKed Valid range: [0, 65535] Units: Sequence number (uint16_t) Purpose: Tells RPi5 which frame failed and needs retransmission
in	flags	Additional flags (e.g., k_frame_flag_soft_nack) Valid range: Bitmask of rx_frame_flag_t values Common values: k_frame_flag_none (hard NACK), k_frame_flag_soft_nack (HARQ) Units: Bitmask
inout	handle	SPI communication handle
in	sequence	Sequence number to negatively acknowledge
in	flags	NACK flags (e.g., k_frame_flag_soft_nack)

Returns: Error code from frame encoding or SPI transfer on failure

Value	Description
k_rx_ok	NACK sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
(other)	Errors propagated from rx_spi_comm_send()

Pre: Same preconditions as rx_spi_comm_send()

Post: NACK frame transmitted with specified sequence number and flags

Note: RPi5 should retransmit frame with specified sequence number

Note: Use k_frame_flag_soft_nack for FEC soft-combining (HARQ)

Example - Hard NACK on CRC Error:: rx_frame_tframe;
rx_err_t err=rx_spi_comm_receive(&g_spi_handle,&frame,0); if(err==k_rx_err_crc_mismatch)
{ rx_spi_comm_send_nack(&g_spi_handle, frame.header.sequence, k_frame_flag_none); }

Example - Soft NACK for HARQ:: if(fec_decode_failed&&harq_enabled)
{ rx_spi_comm_send_nack(&g_spi_handle, frame.header.sequence, k_frame_flag_soft_nack); }

See also: rx_spi_comm_send() Full send function, rx_spi_comm_send_ack() Send ACK for success, rx_frame.h Frame flag definitions

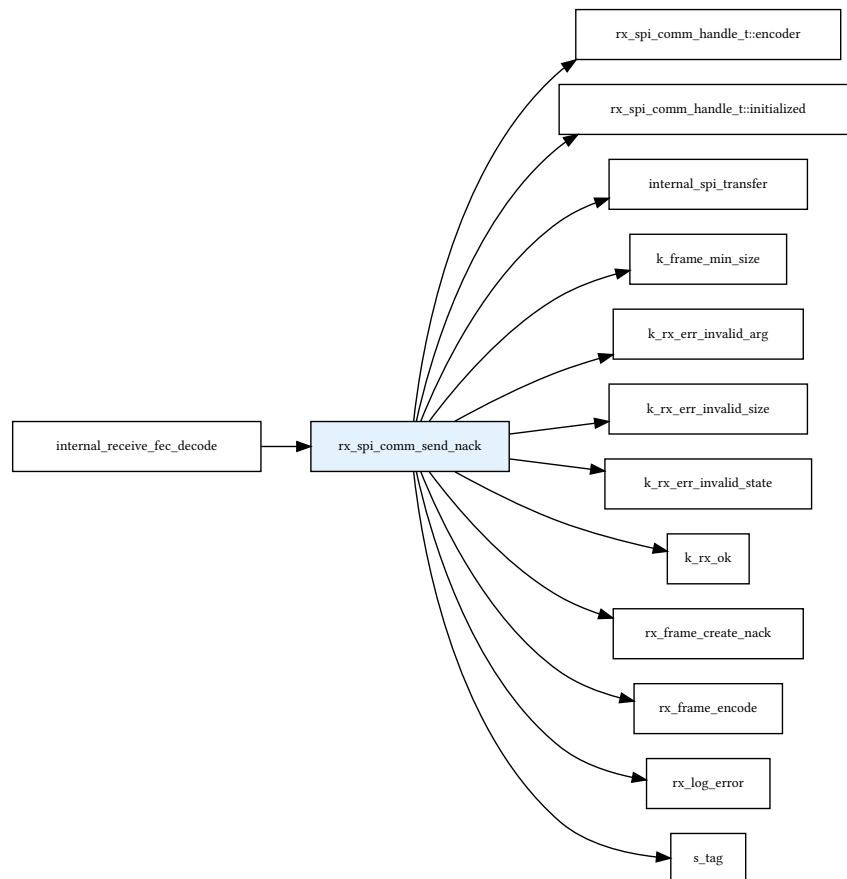


Figure 1093: Call graph for rx_spi_comm_send_nack

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1229`

Since Version 1.0.0

—

rx_spi_comm_receive

```
rx_err_t rx_spi_comm_receive(rx_spi_comm_handle_t *handle, rx_frame_t *frame,
uint32_t timeout_ms)
```

Receive and decode frame from RPi5 with CRC validation.

Reads raw bytes from SPI hardware, validates CRC-32, decodes frame header and payload, and optionally applies FEC decoding with HARQ soft-combining. Returns complete decoded frame to caller.

Algorithm:

1. Validate parameters: Check handle, frame pointer (NASA Rule 5)
2. Poll SPI hardware: Read available bytes (non-blocking if timeout=0)

3. Optional FEC decode: If enabled, apply FEC decoding with HARQ
4. Decode frame: Parse header, extract payload
5. Validate CRC: Compute CRC-32 and compare with frame CRC
6. Validate sequence: Check if sequence matches expected rx_sequence
7. Update state: Increment rx_sequence on success
8. Return frame: Copy decoded frame to output parameter

Blocking Behavior:

- timeout_ms = 0: Non-blocking poll (returns immediately)
- timeout_ms > 0: Blocks up to timeout_ms waiting for data
- timeout_ms = k_spi_comm_default_timeout: Use default (1 second)

Performance:

- Small frame (16 bytes): 100 us
- Medium frame (128 bytes): 250 us
- Large frame (255 bytes): 400 us
- FEC overhead: +100 us

Receive and decode frame from RPi5 with CRC validation.

Receives complete frame from RPi5 by polling for data availability, reading header, reading payload + CRC, validating frame integrity, and updating RX sequence counter. This is the primary receive function for all frame types.

Algorithm:

1. Pre-condition 1: Validate handle and frame pointers non-nullptr
2. Pre-condition 2: Validate handle initialized
3. Wait for data: Call internal_wait_for_data() with timeout

Polls RSPI peripheral for incoming data (yields CPU every 10ms) Returns k_rx_err_timeout if no data within timeout_ms

4. Read frame header: Call internal_read_frame_header()

Reads 10 bytes (SYNC + sequence + length + type + flags) Validates sync word (0xAA55) Extracts payload length (0-1024)

5. Read payload + CRC: Call internal_spi_transfer() for remaining bytes

Reads payload_len + 4 bytes (CRC-32) Stores in handle->rx_buffer

6. Decode frame: Call internal_decode_frame()

Parses header fields into frame->header Copies payload to frame->payload Validates CRC-32 (returns k_rx_err_crc_mismatch if corrupt)

7. Update RX sequence: Set handle->rx_sequence = frame->sequence + 1

Tracks expected next sequence number Caller can detect out-of-order/missing frames

Blocking Behavior:

- timeout_ms = 0: Non-blocking, returns immediately if no data
- timeout_ms > 0: Blocks up to timeout_ms milliseconds waiting for data
- ThreadX sleep: Yields CPU every 10ms tick (not busy-wait)

Timeout Semantics:

- k_rx_err_timeout is NOT an error in polling loops (means “no data yet”)
- Typical polling: Call with timeout_ms=100 in loop, handle timeout normally

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via rx_spi_comm_init() Side effects: Updates shared session RX sequence via rx_session_validate_rx()
out	frame	Pointer to frame structure to receive decoded frame Valid range: Non-nullptr to allocated rx_frame_t Constraints: Must have sufficient space (263 bytes) Lifetime: Filled by this function, owned by caller afterward
in	timeout_ms	Timeout in milliseconds Valid range: [0, 60000] (0 = non-blocking, up to 1 minute) Units: Milliseconds Common values: 0 (poll), k_spi_comm_default_timeout (1 second)
inout	handle	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Side effects: handle->rx_sequence updated on success Usage: Uses handle->rx_buffer for staging (2048 bytes)
out	frame	Frame structure to populate with received data Valid range: Non-nullptr pointer to rx_frame_t (304 bytes) Output: frame->header and frame->payload populated on success Output: frame->crc contains validated CRC-32 value
in	timeout_ms	Maximum wait time in milliseconds Valid range: 0-65535 ms 0: Non-blocking (return immediately if no data) Typical: 100-1000 ms (polling loop timeout) Precision: +/-10 ms (ThreadX tick granularity)

Returns: rx_err_t Error code indicating receive result

Value	Description
k_rx_ok	Frame received and decoded successfully, rx_sequence incremented
k_rx_err_invalid_arg	handle or frame is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_timeout	No data received within timeout (not an error if timeout=0)
k_rx_err_crc_mismatch	CRC validation failed (frame corrupted)
k_rx_err_protocol_error	Frame format invalid (bad sync, length, etc.)
k_rx_err_sequence_error	Unexpected sequence number (out-of-order)
k_rx_err_hw	SPI hardware error
k_rx_ok	Frame received successfully, frame populated, RX sequence updated
k_rx_err_invalid_arg	handle or frame pointer is nullptr
k_rx_err_invalid_state	handle not initialized (call rx_spi_comm_init first)
k_rx_err_timeout	No data available within timeout_ms (normal in polling)
k_rx_err_protocol_error	Sync word invalid (not 0xAA55, resync needed)
k_rx_err_invalid_size	Payload length > k_frame_max_payload (protocol violation)
k_rx_err_crc_mismatch	CRC validation failed (corruption, send NACK)
(HAL errors)	RSPI peripheral read failure

Pre: handle must be non-NULL and initialized

Pre: frame must be non-NULL and allocated

Pre: RSPi hardware must be initialized and ready

Pre: handle must be initialized via `rx_spi_comm_init()`

Pre: frame must point to valid `rx_frame_t` storage (304 bytes)

Pre: RSPi peripheral hardware must be initialized and enabled

Pre: RPi5 controller must have transmitted complete frame

Post: On `k_rx_ok`: frame filled with decoded data, `rx_sequence` incremented

Post: On timeout: frame unchanged, `rx_sequence` unchanged

Post: On CRC error: frame may be partially filled, `rx_sequence` unchanged

Post: On success, frame contains validated header, payload, CRC

Post: On success, `handle->rx_sequence = frame->header.sequence + 1`

Post: On timeout, frame contents undefined, RX sequence unchanged

Post: On CRC mismatch, frame partially populated, RX sequence unchanged

Post: On protocol error, frame undefined, may need resync

Note: `timeout=0` is recommended for polling loops (non-blocking)

Note: CRC errors should trigger NACK and retransmission request

Note: Sequence errors indicate lost frames or out-of-order delivery

Note: This function blocks (with ThreadX sleep) if data not immediately available

Note: timeout_ms=0 provides non-blocking behavior for polling loops

Warning: Must call rx_spi_comm_send_ack() or rx_spi_comm_send_nack() after receive

Warning: Not thread-safe - use external mutex if multiple threads receive

Warning: Do not process frame if return value != k_rx_ok (data invalid)

Warning: Send NACK if k_rx_err_crc_mismatch to request retransmit

Example - Non-Blocking Poll:: rx_frame_tframe;

```
rx_err_t err = rx_spi_comm_receive(&g_spi_handle, &frame, 0);
if (err == k_rx_ok) { handle_frame(&frame);
rx_spi_comm_send_ack(&g_spi_handle, frame.header.sequence); }
else if (err == k_rx_err_timeout) { }
else if (err == k_rx_err_crc_mismatch)
{ rx_spi_comm_send_nack(&g_spi_handle, frame.header.sequence, k_frame_flag_none); }
else { log_error("Receive error: %d", err); }
```

Example - Blocking Receive with Timeout:: rx_frame_tframe;

```
rx_err_t err = rx_spi_comm_receive(&g_spi_handle, &frame, 1000);
if (err == k_rx_ok) { process_frame(&frame);
rx_spi_comm_send_ack(&g_spi_handle, frame.header.sequence); }
else if (err == k_rx_err_timeout) { log_warn("No frame received within timeout"); }
```

Performance:: Execution time: 200-600 us @ 10 MHz SPI + 0-timeout_ms wait 14-byte frame (ACK): 200 us (10 us wait + 80 us read + 110 us decode) 100-byte frame: 400 us (10 us wait + 200 us read + 190 us decode) 279-byte frame (max): 600 us (10 us wait + 300 us read + 290 us decode) Wait time: Add 0-timeout_ms if data not immediately available

Example - Polling Loop (Typical Usage):: while(1) { rx_frame_tframe;

```
rx_err_t err = rx_spi_comm_receive(&g_spi_handle, &frame, 100);
if (err == k_rx_ok) { if (frame.header.type == k_frame_type_data) { process_data_frame(&frame);
rx_spi_comm_send_ack(&g_spi_handle, frame.header.sequence); } }
else if (err == k_rx_err_timeout) { tx_thread_sleep(1); }
else if (err == k_rx_err_crc_mismatch) { rx_spi_comm_send_nack(&g_spi_handle, frame.header.sequence, 0); }
else { rx_log_error("spi", "Receive error: %d", err); } }
```

Example - Non-Blocking Check:: rx_frame_tframe;

```
rx_err_t err = rx_spi_comm_receive(&g_spi_handle, &frame, 0);
if (err == k_rx_ok) { rx_log_debug("spi", "Frame received: seq=%d, len=%d",
frame.header.sequence, frame.header.length); }
else if (err == k_rx_err_timeout) { }
```

Example - Sequence Number Checking:: static uint16_t expected_seq = 0; rx_frame_tframe;

```
rx_err_t err = rx_spi_comm_receive(&g_spi_handle, &frame, 1000);
if (err == k_rx_ok) { if (frame.header.sequence != expected_seq) { rx_log_warn("spi", "Out-of-order: expected %d, got %d",
expected_seq, frame.header.sequence); } expected_seq = frame.header.sequence + 1; }
```


See also: `rx_spi_comm_send()` Send frames to RPi5, `rx_spi_comm_send_ack()` Send ACK for received frame, `rx_spi_comm_send_nack()` Send NACK on error, `rx_spi_comm_data_available()` Check if data waiting, `rx_frame.h` Frame structure definition, `rx_spi_comm_send()` Send frames to RPi5, `rx_spi_comm_data_available()` Check data availability without timeout, `rx_spi_comm_send_ack()` Send acknowledgment after successful receive, `rx_spi_comm_send_nack()` Send NACK after CRC mismatch, `internal_wait_for_data()` Wait for data with timeout, `internal_decode_frame()` Frame decoding and CRC validation

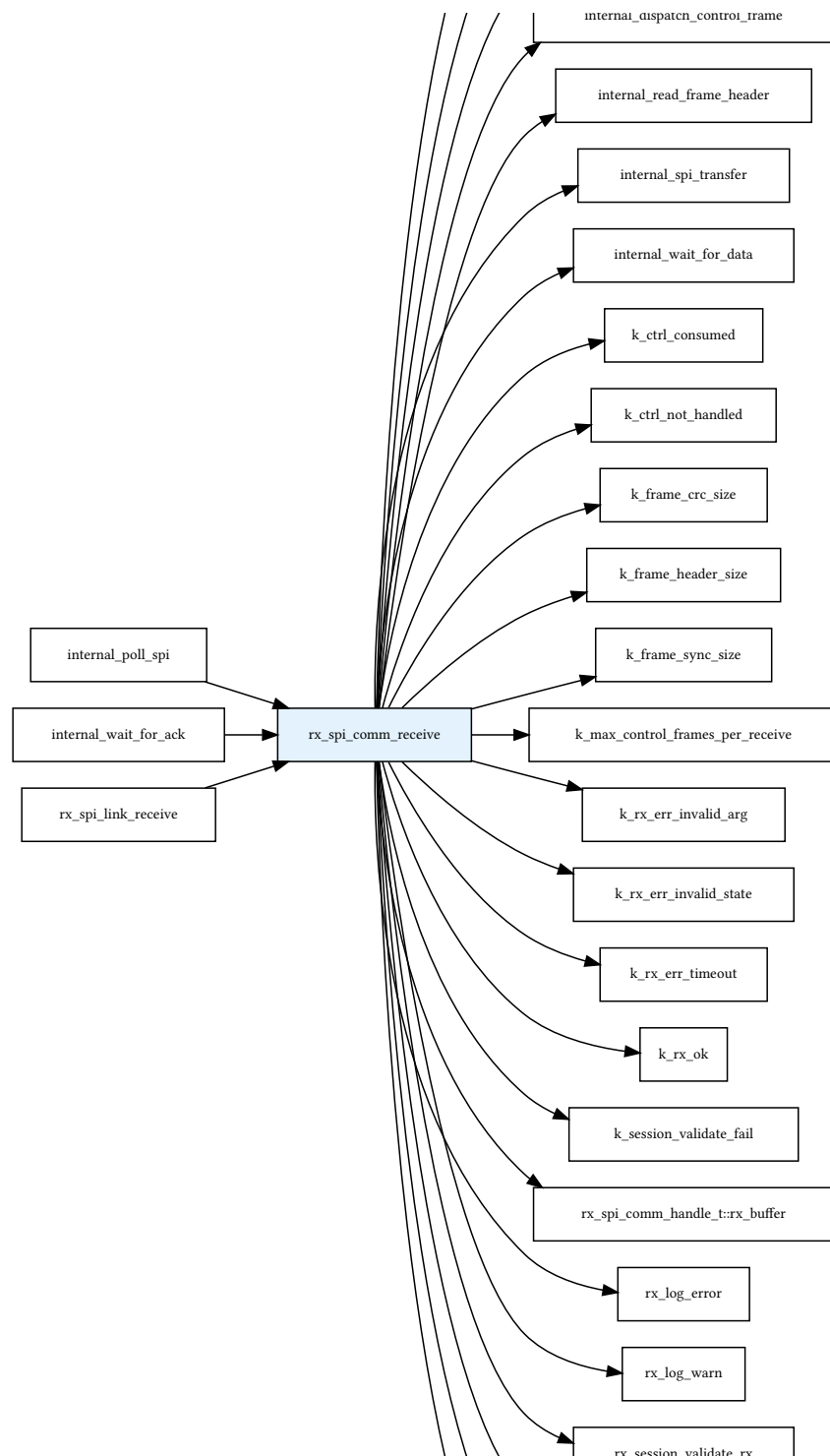


Figure 1094: Call graph for rx_spi_comm_receive

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1350`

Since Version 1.0.0

—

rx_spi_comm_data_available

```
rx_err_t rx_spi_comm_data_available(const rx_spi_comm_handle_t *handle, bool
*available)
```

Check if data is available for reading (non-blocking query).

Queries SPI hardware to determine if data is available in receive buffer without actually reading the data. Useful for polling loops to avoid unnecessary receive attempts.

Use Case: Optimize polling by checking availability before calling `rx_spi_comm_receive()`, avoiding timeout overhead.

Algorithm:

1. Validate handle and available pointer
2. Query RSPI hardware for RX buffer status
3. Set `*available` = true if data waiting, false otherwise

Performance: 2 us (fast hardware register read)

Check if data is available for reading (non-blocking query).

Parameters:

Direction	Name	Description
in	handle	Initialized SPI communication handle Valid range: Non-nullptr to initialized handle Constraints: Must be initialized via <code>rx_spi_comm_init()</code>
out	available	Pointer to receive availability status Valid range: Non-nullptr to bool Output values: true (data available), false (no data) On error: Set to false
in	handle	SPI communication handle
out	available	Set to true if data available, false otherwise

Returns: Error code from SPI peripheral read availability check on failure

Value	Description
<code>k_rx_ok</code>	Query successful, available flag updated
<code>k_rx_err_invalid_arg</code>	handle or available is nullptr
<code>k_rx_err_invalid_state</code>	handle not initialized

Pre: handle must be non-NULL and initialized

Pre: available must be non-NULL

Post: *available set to true or false based on hardware state

Post: On error, *available set to false (safe default)

Note: This function does NOT read or modify receive buffer

Note: Availability can change between check and actual receive

Note: Fast operation - suitable for high-frequency polling

Example - Optimized Polling Loop::

```
voidpoll_spi(void) { boolavailable=false;
rx_err_terr=rx_spi_comm_data_available(&g_spi_handle,&available);

if(err==k_rx_ok&&available){ rx_frame_tframe;
err=rx_spi_comm_receive(&g_spi_handle,&frame,0); if(err==k_rx_ok){ handle_frame(&frame); } }
```

See also: rx_spi_comm_receive() Receive frame

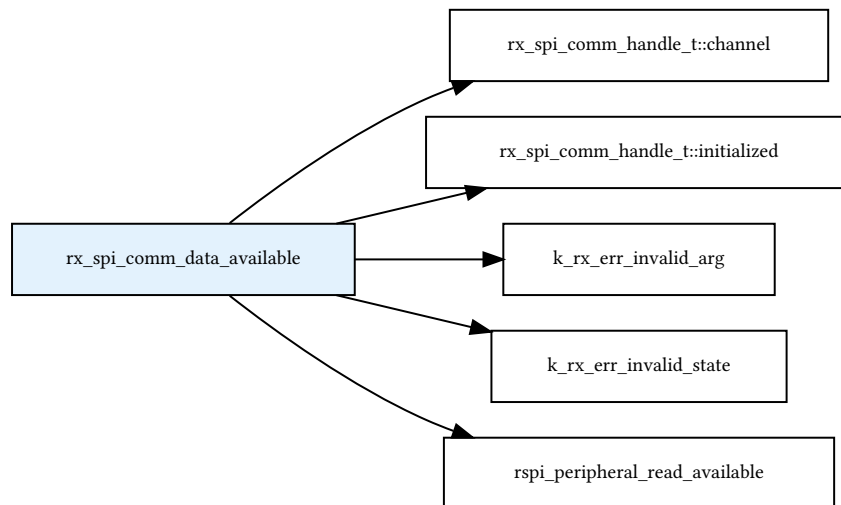


Figure 1095: Call graph for rx_spi_comm_data_available

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1415`

Since Version 1.0.0

—

rx_spi_comm_set_control_callbacks

```
rx_err_t rx_spi_comm_set_control_callbacks(rx_spi_comm_handle_t *handle,
void(*on_ping_cb)(const rx_frame_t *frame, void *ctx), void(*on_reset_cb)(const
rx_frame_t *frame, void *ctx), void *cb_ctx)
```

Register callbacks for PING and RESET control frames.

Sets callback functions that are invoked when `rx_spi_comm_receive()` encounters PING or RESET control frames. Control frames are handled internally (auto PONG / auto RESET_ACK) and callbacks provide notification to the application layer.

Stores function pointers that the receive path invokes when it decodes a PING or RESET frame. Passing NULL for a callback disables notification for that frame type while still performing the automatic PONG / RESET_ACK response.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle
in	on_ping_cb	Callback for PING frames (may be NULL to disable)
in	on_reset_cb	Callback for RESET frames (may be NULL to disable)
in	cb_ctx	User context pointer passed to callbacks
inout	handle	SPI communication handle (must be initialized)
in	on_ping_cb	Callback invoked on PING receipt (NULL to disable)
in	on_reset_cb	Callback invoked on RESET receipt (NULL to disable)
in	cb_ctx	Opaque context forwarded to control callbacks

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Callbacks registered successfully
<code>k_rx_err_invalid_arg</code>	handle is nullptr
<code>k_rx_ok</code>	Callbacks registered
<code>k_rx_err_invalid_arg</code>	handle is nullptr

Pre: handle must be non-NULL

Pre: handle != nullptr

Pre: handle->initialized == true

Post: on_ping_cb, on_reset_cb, control_cb_ctx stored in handle

Post: Callbacks stored; invoked on next matching control frame

Post: Previous callbacks overwritten

Note: Callbacks are invoked from within `rx_spi_comm_receive()` context

Note: Not thread-safe, set callbacks before starting receive loop

Note: Not thread-safe; register before starting the receive loop.] **See also:**

`rx_spi_comm_receive()` Invokes callbacks on control frames, `rx_spi_comm_send_pong()` Auto-response sent on PING, `rx_spi_comm_send_reset_ack()` Auto-response sent on RESET

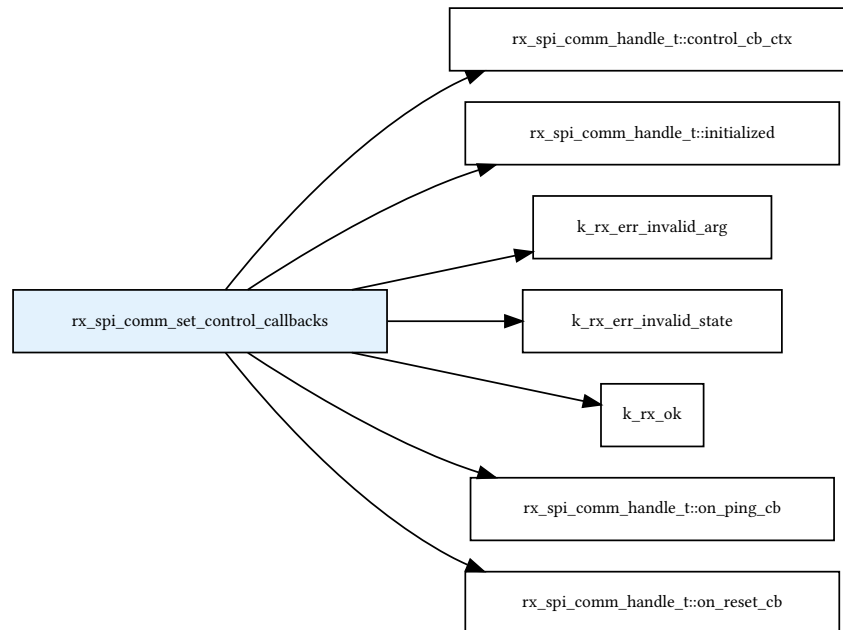


Figure 1096: Call graph for `rx_spi_comm_set_control_callbacks`

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1458`

Since Version 1.0.0

—

rx_spi_comm_send_pong

```
rx_err_t rx_spi_comm_send_pong(rx_spi_comm_handle_t *handle, const uint8_t
*payload, uint32_t payload_len)
```

Send PONG frame with payload (response to PING).

Constructs and transmits a PONG frame echoing the provided payload. Used internally by `rx_spi_comm_receive()` for auto-PONG, but also available for manual PONG sending.

Send PONG frame with payload (response to PING).

Constructs a PONG frame echoing the given payload, acquires the next TX sequence number from the shared session, encodes the frame, and transfers it over SPI. Typically called automatically by the receive path in response to a PING.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle
in	payload	Payload data to echo (may be NULL if payload_len is 0)
in	payload_len	Length of payload in bytes
inout	handle	SPI communication handle
in	payload	Payload to echo (NULL when payload_len is 0)
in	payload_len	Length in bytes [0, k_frame_max_payload]

Returns: rx_err_t Error code

Value	Description
k_rx_ok	PONG frame sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
(other)	Errors propagated from frame creation or SPI transfer
k_rx_ok	PONG transmitted successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	Handle not initialized
k_rx_err_invalid_size	payload_len exceeds k_frame_max_payload
(other)	Error from session, encode, or SPI transfer

Pre: handle must be non-NULL and initialized**Pre:** handle != nullptr && handle->initialized**Pre:** payload != NULL when payload_len > 0**Post:** PONG frame transmitted with echoed payload**Post:** TX sequence incremented on success**Post:** PONG frame sent over SPI**Note:** Gets next TX sequence from shared session state for the PONG frame

Note: Not thread-safe; called from the SPI receive context.] **See also:** `rx_frame_create_pong()`
 Frame creation for PONG, `rx_spi_comm_receive()` Auto-sends PONG on PING reception,
`rx_frame_create_pong()` Frame construction helper

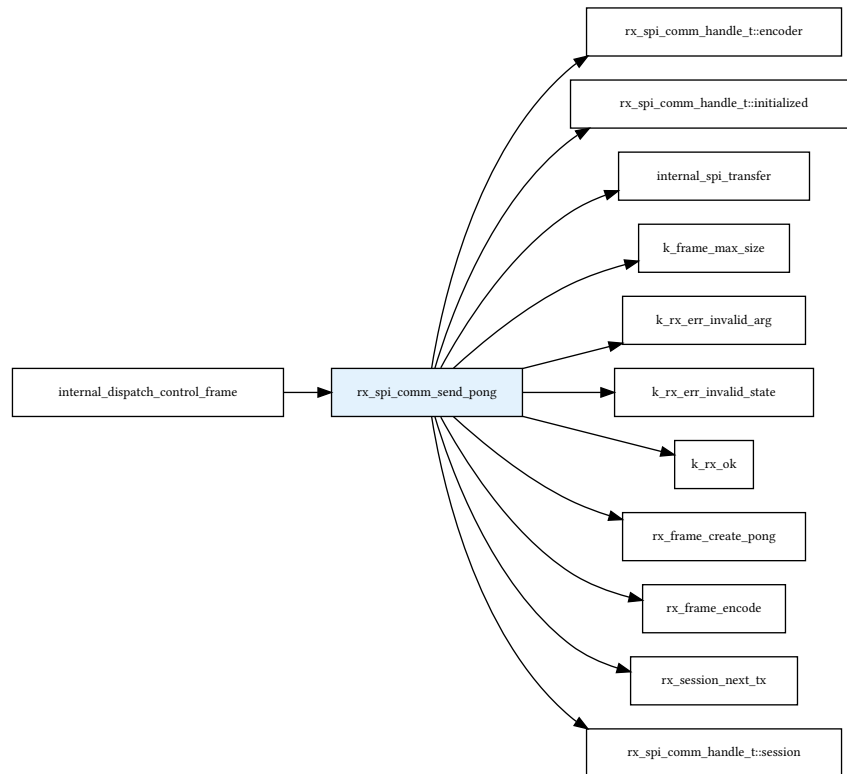


Figure 1097: Call graph for `rx_spi_comm_send_pong`

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1492`

Since Version 1.0.0

—

`rx_spi_comm_send_reset_ack`

```
rx_err_t rx_spi_comm_send_reset_ack(rx_spi_comm_handle_t *handle)
```

Send RESET_ACK frame (response to RESET).

Constructs and transmits a RESET_ACK frame. Used internally by `rx_spi_comm_receive()` for auto-RESET_ACK, but also available for manual sending.

Send RESET_ACK frame (response to RESET).

Constructs a RESET_ACK frame (empty payload), acquires the next TX sequence number, encodes, and transfers over SPI. Called automatically by the receive path after processing a RESET frame. The

RESET_ACK itself does NOT reset sequence numbers; that is handled by the session layer in response to the RESET.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle
inout	handle	SPI communication handle

Returns: rx_err_t Error code

Value	Description
k_rx_ok	RESET_ACK frame sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
(other)	Errors propagated from frame creation or SPI transfer
k_rx_ok	RESET_ACK transmitted successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	Handle not initialized
(other)	Error from session, encode, or SPI transfer

Pre: handle must be non-NULL and initialized

Pre: handle != nullptr && handle->initialized

Pre: A RESET frame was received and session state cleared

Post: RESET_ACK frame transmitted

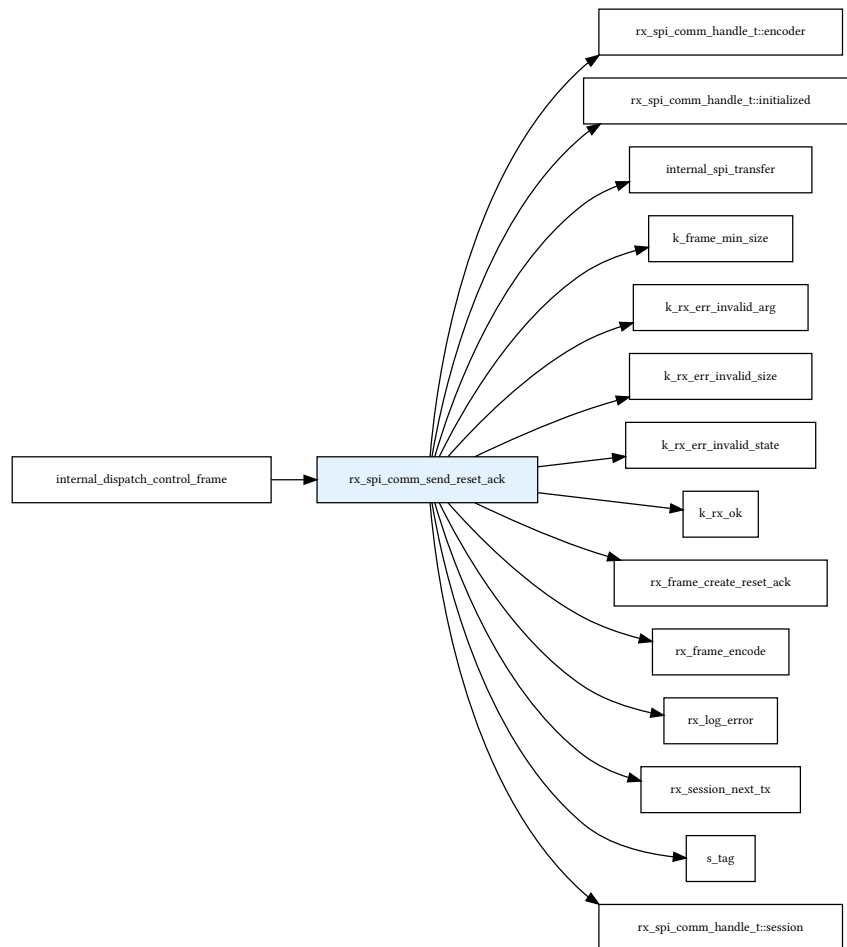
Post: TX sequence incremented on success

Post: RESET_ACK frame sent over SPI

Note: Does NOT reset sequence numbers (caller must do that if desired)

Note: Not thread-safe; called from the SPI receive context.] **See also:**

rx_frame_create_reset_ack() Frame creation for RESET_ACK, rx_spi_comm_receive()
Auto-sends RESET_ACK on RESET reception, rx_frame_create_reset_ack() Frame
construction helper, rx_spi_comm_set_control_callbacks() Register RESET
notification

Figure 1098: Call graph for `rx_spi_comm_send_reset_ack`

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1520`

Since Version 1.0.0

—

rx_spi_comm_process_retransmits

```
rx_err_t rx_spi_comm_process_retransmits(rx_spi_comm_handle_t *handle, uint32_t
current_time_ms)
```

Check and process automatic retransmissions.

Checks whether an ACK timeout has elapsed for the most recent sent frame. If `auto_retransmit` is enabled and a retransmit is pending, resends the buffered frame (up to `max_retries` with exponential backoff).

Check and process automatic retransmissions.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle
in	current_time_ms	Current system time in milliseconds
inout	handle	SPI communication handle
in	current_time_ms	Current system time in milliseconds

Returns: rx_err_t Error code

Value	Description
k_rx_ok	No action needed or retransmit sent
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_ok	No action needed or retransmit sent
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_retry_limit	Max retries exceeded

Pre: handle must be non-NULL and initialized**Pre:** handle must be non-NULL and initialized**Post:** retry_count incremented on retransmit, pending cleared on limit**Post:** retry_count incremented on retransmit, pending cleared on limit**Note:** Safe to call when auto_retransmit is disabled (no-op)**Note:** Safe to call when auto_retransmit is disabled (no-op)] **See also:**
rx_spi_comm_set_auto_retransmit() Enable/disable retransmission

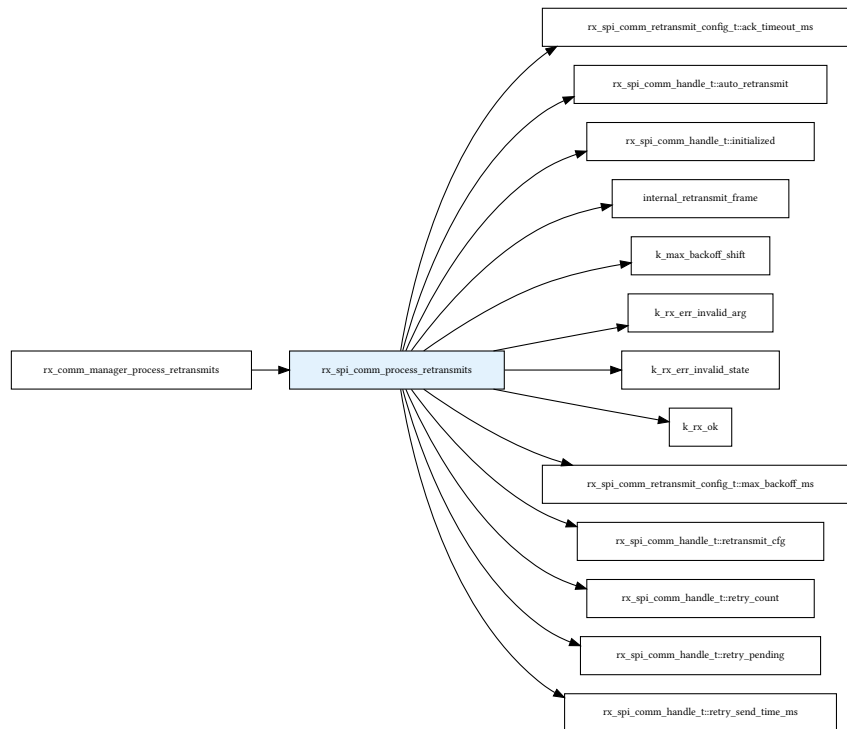


Figure 1099: Call graph for rx_spi_comm_process_retransmits

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1547`

Since Version 1.1.0

rx_spi_comm_set_auto_retransmit

```
rx_err_t rx_spi_comm_set_auto_retransmit(rx_spi_comm_handle_t *handle, bool
enabled, const rx_spi_comm_retransmit_config_t *config)
```

Enable or disable automatic retransmission at runtime.

Allows the RPi5 to configure retransmission behavior via protobuf command. When disabling, any pending retry state is cleared.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle
in	enabled	true to enable, false to disable
in	config	Retransmit configuration (may be NULL to keep current or use defaults)
inout	handle	SPI communication handle
in	enabled	true to enable, false to disable

in	config	Retransmit config (NULL to keep current/defaults)
----	--------	---

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Configuration applied successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_ok	Configuration applied successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be non-NULL and initialized

Pre: handle must be non-NULL and initialized

Post: auto_retransmit flag and config updated

Post: If disabling, pending retry state cleared

Post: auto_retransmit flag and config updated

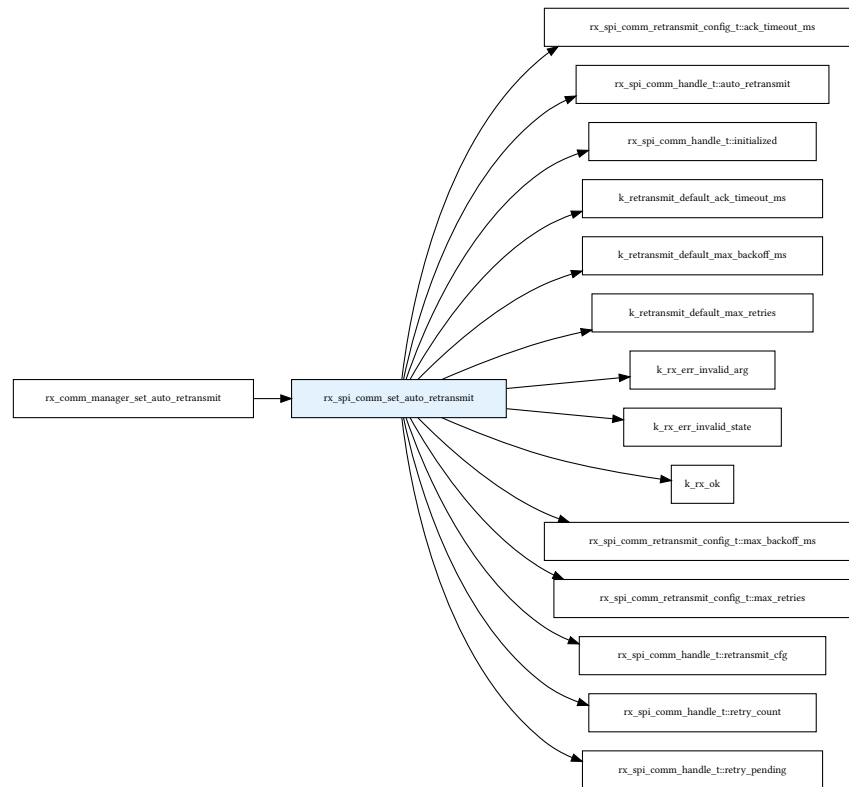


Figure 1100: Call graph for rx_spi_comm_set_auto_retransmit

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1573`

Since Version 1.1.0

—

rx_spi_comm_set_retransmit_callbacks

```

rx_err_t rx_spi_comm_set_retransmit_callbacks(rx_spi_comm_handle_t *handle,
void(*on_ack_cb)(uint16_t sequence, void *ctx), void(*on_nack_cb)(uint16_t
sequence, void *ctx), void *ctx)

```

Register ACK/NACK notification callbacks for retransmission.

Sets optional callbacks invoked when ACK or NACK control frames are consumed during `rx_spi_comm_receive()`. Useful for telemetry and diagnostics.

Register ACK/NACK notification callbacks for retransmission.

Parameters:

Direction	Name	Description
inout	handle	Initialized SPI communication handle

in	on_ack_cb	Callback for ACK frames (may be NULL to disable)
in	on_nack_cb	Callback for NACK frames (may be NULL to disable)
in	ctx	User context pointer passed to callbacks
inout	handle	SPI communication handle
in	on_ack_cb	ACK callback (NULL to disable)
in	on_nack_cb	NACK callback (NULL to disable)
in	ctx	User context passed to callbacks

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Callbacks registered
k_rx_err_invalid_arg	handle is nullptr
k_rx_ok	Callbacks registered
k_rx_err_invalid_arg	handle is nullptr

Pre: handle must be non-NULL

Post: Callback pointers stored in handle

Note: Callbacks invoked from within rx_spi_comm_receive() context

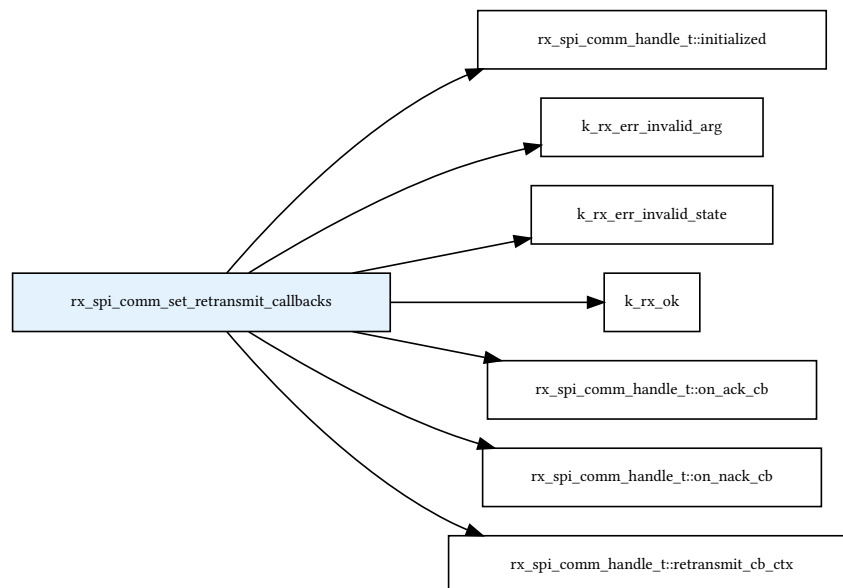


Figure 1101: Call graph for rx_spi_comm_set_retransmit_callbacks

Defined in `libs/rx_spi_comm/inc/rx_spi_comm.h:1601`

Since Version 1.1.0

—

rx_spi_comm.c

High-Level SPI Frame Protocol Communication Layer Implementation.

Includes:

- `"rx_spi_comm.h"`
- `<string.h>`
- `"hardware.h"`
- `"rx_crc.h"`
- `"rx_threadx_config.h"`
- `"rx_time_constants.h"`
- `"tx_api.h"`

backoff_limit_t

Maximum left-shift for exponential backoff (prevents UB on uint32_t).

Value	Name	Description
= 31	<code>k_max_backoff_shift</code>	Clamp <code>retry_count</code> before shifting to avoid UB

—

frame_header_offset_t

Byte offsets within frame header buffer (SYNC + header fields).

Value	Name	Description
= 0	<code>k_frame_offset_init</code>	Initial frame parse offset (start of buffer)
= 0	<code>k_hdr_sync_low</code>	SYNC word low byte (LE: LSB first)
= 1	<code>k_hdr_sync_high</code>	SYNC word high byte (LE: MSB second)
= 2	<code>k_hdr_seq_low</code>	Sequence number low byte (LE)
= 3	<code>k_hdr_seq_high</code>	Sequence number high byte (LE)
= 4	<code>k_hdr_len_low</code>	Payload length low byte (LE)
= 5	<code>k_hdr_len_high</code>	Payload length high byte (LE)
= 6	<code>k_hdr_type</code>	Frame type
= 7	<code>k_hdr_flags</code>	Frame flags

—

poll_timing_t

Sleep duration for polling loop.

Value	Name	Description
= 1	<code>k_poll_sleep_ticks</code>	1 ThreadX tick between polls

—

ack_wait_t

ACK/ready wait timing constants.

Value	Name	Description
= 50	k_ack_wait_timeout_ms	Abort if host doesn't ACK within 50 ms

—

receive_bounds_t

Receive loop bounds (NASA Power of 10 Rule 2).

Value	Name	Description
= 16	k_max_control_frames_per_receive	Max control frames before returning error

—

polling_limits_t

Polling loop iteration limits.

Value	Name	Description
= 1000	k_max_poll_iterations	Maximum polling iterations (safety bound)

—

ctrl_dispatch_result_t

Result of control-frame dispatch within the receive loop.

Value	Name	Description
= 0	k_ctrl_not_handled	Frame is not a control frame; return to caller
= 1	k_ctrl_consumed	Control frame consumed; continue receive loop
= 2	k_ctrl_error	Error occurred during control-frame handling

—

s_tag

```
const char s_tag[]
```

—

s_zero_handle

```
const rx_spi_comm_handle_t s_zero_handle
```

Zero-initialized SPI comm handle template for stack-safe initialization.

Note: Read-only; never modified after static initialization

Warning: Do not remove - prevents stack overflow in init function] **See also:**
rx_spi_comm_init() Uses this template to zero-initialize the comm handle

Since Version 1.0.0

—

internal_retransmit_frame

```
static rx_err_t internal_retransmit_frame(rx_spi_comm_handle_t *handle)
```

Retransmit the buffered frame via SPI.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle with retry_buffer populated

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Retransmit sent successfully
k_rx_err_retry_limit	Max retries exceeded

Pre: handle->retry_pending == true

Pre: handle->retry_buffer contains valid encoded frame

Post: retry_count incremented on success, retry_pending cleared on limit

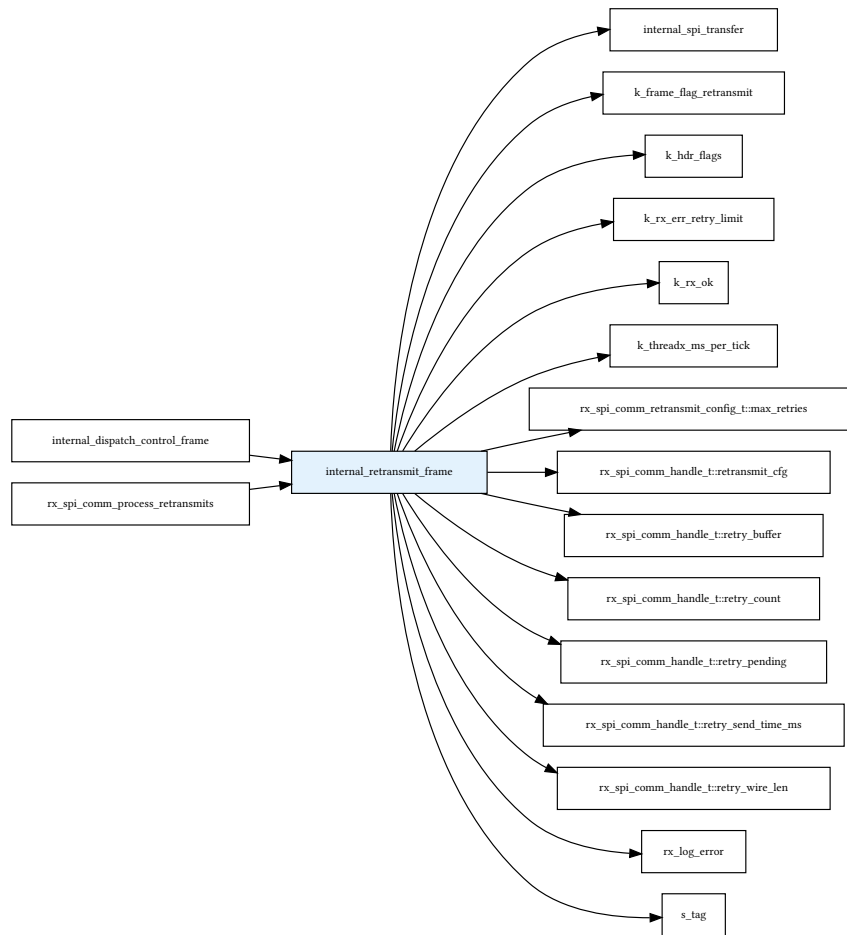


Figure 1102: Call graph for internal_retransmit_frame

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2389`

Since Version 1.1.0

internal_decode_header

```
static rx_err_t internal_decode_header(const uint8_t *data, const uint32_t
data_len, rx_frame_t *frame, uint32_t *offset_out)
```

Decode and validate SPI frame header from raw wire buffer.

Parses the frame header fields from raw received SPI data buffer, validating sync word and payload length constraints. Extracts sequence number, length, type, and flags into `rx_frame_t` structure.

Algorithm:

1. Validate inputs: Check data and frame pointers, minimum buffer size
2. Parse sync word: Read little-endian 0x55AA, validate against `k_frame_sync_word`

3. Parse sequence: Read little-endian 16-bit sequence number
4. Parse length: Read little-endian 16-bit payload length, validate ≤ 1024
5. Validate total size: Check buffer contains header + payload + CRC
6. Parse type and flags: Extract frame type and flags bytes
7. Return payload offset: Optionally return offset to payload start

Frame Header Format (10 bytes):

Parameters:

Direction	Name	Description
in	data	Raw frame data buffer from SPI receive Valid range: Non-nullptr pointer to buffer of size data_len Constraints: Must contain at least k_frame_min_size (14) bytes Format: Little-endian header fields
in	data_len	Length of data buffer in bytes Valid range: ≥ 14 (minimum frame: header + 0 payload + CRC) Typical range: 14-1038 bytes (0-1024 payload + overhead)
out	frame	Frame structure to populate with parsed header Valid range: Non-nullptr pointer to rx_frame_t Side effects: frame->header fields populated, payload NOT copied
out	offset_out	Optional pointer to receive payload offset Valid range: nullptr or pointer to uint32_t Output: Offset to payload start (8 if header parsed successfully) Usage: Caller uses offset to locate payload and CRC in data buffer

Returns: rx_err_t Error code indicating parse result

Value	Description
k_rx_ok	Header parsed successfully, frame->header populated
k_rx_err_invalid_arg	data or frame pointer is nullptr
k_rx_err_invalid_size	data_len < 14 (too small for minimum frame)
k_rx_err_invalid_size	Payload length > k_frame_max_payload (protocol violation)
k_rx_err_invalid_size	Buffer too small for declared payload + CRC
k_rx_err_protocol_error	Sync word != 0x55AA (not a valid frame)

Pre: data must point to valid buffer of size data_len

Pre: frame must point to valid rx_frame_t structure

Post: On success, frame->header contains parsed fields

Post: On success, offset_out (if non-nullptr) contains payload offset

Post: On failure, frame contents are undefined

Note: This function does NOT copy payload or validate CRC

Note: Use `internal_verify_crc()` separately to validate CRC-32

Warning: Do not assume frame->payload is valid after this call

Performance:: Execution time: 15 us @ 240 MHz (byte-level parsing, no memcpy)

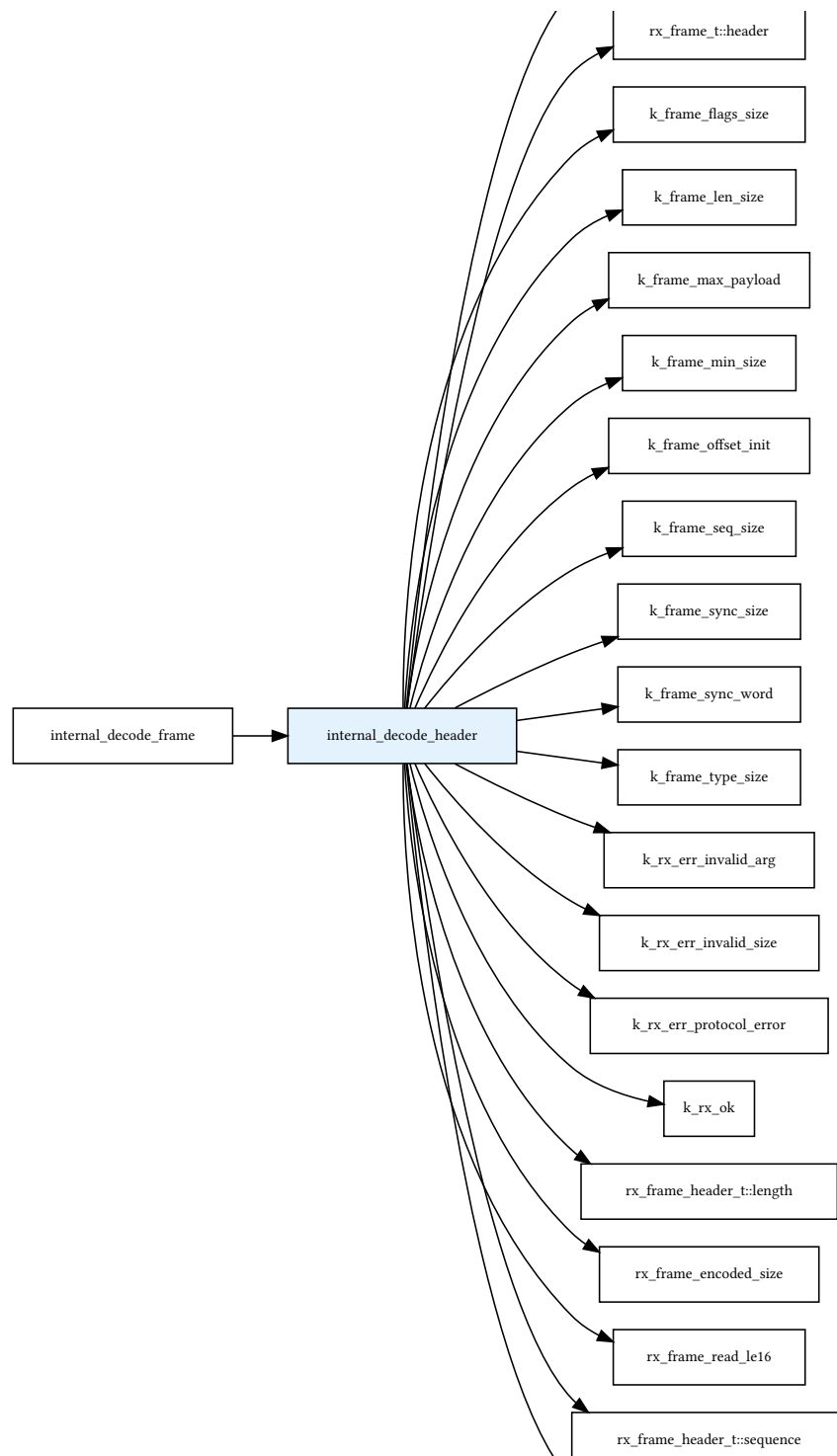
Example Usage:: `uint8_trx_buf[279]; rx_frame_tframe; uint32_tpayload_offset;`

```
rx_err_terr=internal_decode_header(rx_buf,sizeof(rx_buf),&frame,&payload_offset);
if(err==k_rx_ok){ if(frame.header.length>0)
{ memcpy(frame.payload,&rx_buf[payload_offset],frame.header.length); } }
```

Example:

Offset	Size	Field	Format	Notes
0	2	SYNC	LEuint16	Always 0x55AA (wire: 0xAA, 0x55)
2	2	Sequence	LEuint16	0-65535, wraps
4	2	Length	LEuint16	Payloadbytes (0-1024)
6	1	Type	uint8	k_frame_type_tenum
7	1	Flags	uint8	Bitmask (FEC, ACK, NACK)
8	N	Payload	bytes	N=Length (parsed above)
8+N	4	CRC-32	LEuint32	IEEECRC-32

See also: `internal_verify_crc()` Validate CRC-32 after header parse,
`internal_decode_frame()` Higher-level decode (header + payload + CRC),
`rx_frame_read_le16()` Little-endian uint16 parser

Figure 1103: Call graph for `internal_decode_header`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:440`

Since Version 1.0.0

—

internal_verify_crc

```
static rx_err_t internal_verify_crc(const uint8_t *data, uint32_t offset,
uint32_t *crc_out)
```

Verify CRC-32 for a received SPI frame.

Validates frame integrity by computing CRC-32 over header + payload and comparing with the CRC-32 field at the end of the frame. Uses IEEE polynomial 0x04C11DB7 (same as Ethernet, ZIP, PNG).

Algorithm:

1. Extract received CRC: Read 4-byte little-endian CRC from data[offset]
2. Compute expected CRC: Calculate CRC-32 over data[0..offset-1]
3. Compare: If received == calculated, return k_rx_ok
4. Optionally return CRC: Copy received CRC to crc_out if non-nullptr

CRC Computation Details:

- Algorithm: CRC-32/IEEE (polynomial 0x04C11DB7, init 0xFFFFFFFF, final XOR 0xFFFFFFFF)
- Input: All bytes from sync word through end of payload (excludes CRC itself)
- Output: 32-bit checksum stored in little-endian at frame end
- Implementation: rx_crc32_ieee() (hardware CRC peripheral on RX72N)

Parameters:

Direction	Name	Description
in	data	Raw frame data buffer containing header + payload + CRC Valid range: Non-nullptr pointer to complete frame buffer Layout: [SYNC(2)][Header(6)][Payload(0-1024)][CRC(4)] Constraints: Must contain offset + 4 bytes minimum
in	offset	Offset to CRC field in data buffer (bytes to hash) Valid range: 8-1032 (header=8, max payload=1024, CRC at offset+4) Typical: 8 (ACK/NACK, no payload) to 1032 (1024-byte payload) Usage: CRC computed over data[0..offset-1], compared with data[offset..offset+3]
out	crc_out	Optional pointer to receive the received CRC value Valid range: nullptr (ignore) or pointer to uint32_t Output: Received CRC from frame (useful for logging/debugging) Note: Always set, even if CRC mismatch occurs

Returns: rx_err_t Error code indicating CRC validation result

Value	Description
k_rx_ok	CRC valid (received == calculated), frame intact
k_rx_err_invalid_arg	data pointer is nullptr
k_rx_err_crc_mismatch	CRC invalid (corruption detected)

Pre: data must point to valid buffer containing offset + 4 bytes

Pre: offset must be the byte position where CRC-32 field begins

Post: On k_rx_ok, frame integrity guaranteed

Post: On k_rx_err_crc_mismatch, frame is corrupted (send NACK)

Post: crc_out (if non-nullptr) contains received CRC value

Note: Thread-safe if caller serializes access to data buffer

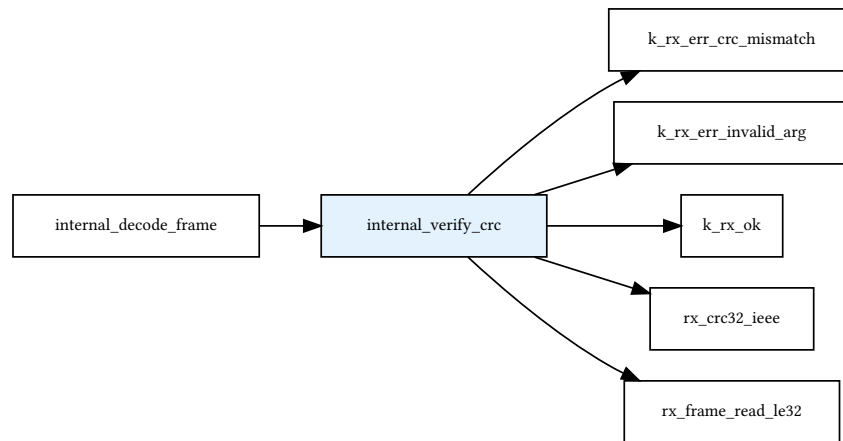
Warning: This function does NOT modify the data buffer

Warning: CRC mismatch indicates transmission error, do NOT process frame

Performance:: Execution time: 40-250 us @ 240 MHz (depends on offset = bytes to hash) 10-byte frame (ACK): 40 us 100-byte frame: 130 us 263-byte frame (max): 250 us Hardware CRC: If RX_CRC32_USE_HARDWARE defined, 4x faster

CRC Mismatch Handling:: uint8_trx_buf[279]; uint32_tpayload_offset=8;
uint32_tcrc_offset=payload_offset+payload_len; uint32_treceived_crc;
rx_err_terr=internal_verify_crc(rx_buf, crc_offset, &received_crc); if(err==k_rx_err_crc_mismatch)
{ rx_log_error("spi_comm", "CRCmismatch:received=0x%08X", received_crc);
rx_spi_comm_send_nack(handle, frame_seq, 0); }

See also: rx_crc32_ieee() CRC-32 computation implementation, rx_frame_read_le32()
Little-endian uint32 reader, internal_decode_frame() Uses this function for CRC
validation

Figure 1104: Call graph for `internal_verify_crc`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:566`

Since Version 1.0.0

internal_wait_for_ack

```
static rx_err_t internal_wait_for_ack(const rx_spi_comm_handle_t *handle,
uint32_t timeout_ms)
```

Wait for RPi5 controller ready signal before transmitting data.

Implements flow control by polling RSPI peripheral for host ready signal before initiating SPI transfer. Prevents peripheral from blocking indefinitely if controller never asserts chip select or clock.

Purpose: RX72N (SPI peripheral) cannot initiate transfers - it must wait for RPi5 (SPI controller) to assert CS and provide clock. This function polls the RSPI hardware for the “ready to write” condition with a timeout to detect host failures.

Algorithm:

1. Get start time: Capture current ThreadX tick count (or iteration counter in tests)
2. Poll loop:

Call `rspi_peripheral_write_ready()` to check hardware status
 If ready: Return `k_rx_ok` immediately
 If error: Return HAL error code
 If not ready: Sleep 1 tick (10ms) and retry

3. Timeout check: If elapsed time $\geq$ `timeout_ms`, return `k_rx_err_timeout`
4. Thread yield: `tx_thread_sleep()` yields CPU to other tasks (not busy-wait)

Platform Differences:

- RX72N (RX defined): Uses ThreadX `tx_time_get()` for precise timing
- Host builds (tests): Uses iteration counter to simulate time (no ThreadX)

Parameters:

Direction	Name	Description
-----------	------	-------------

in	handle	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Usage: handle->channel selects which RSPI peripheral to poll
in	timeout_ms	Maximum wait time in milliseconds Valid range: 1-65535 ms Typical: 50 ms (k_ack_wait_timeout_ms constant) Rationale: 50ms allows 5 retries @ 10ms/tick, detects dead host quickly

Returns: rx_err_t Error code indicating wait result

Value	Description
k_rx_ok	Host ready signal detected, safe to transmit
k_rx_err_timeout	Timeout expired, host not ready (dead/busy)
(HAL errors)	RSPI peripheral error (mode fault, hardware failure)

Pre: handle must be initialized via rx_spi_comm_init()

Pre: timeout_ms must be > 0

Post: On k_rx_ok, RSPI is ready for rspi_peripheral_transfer()

Post: On k_rx_err_timeout, host may be dead/disconnected

Post: On HAL error, RSPI peripheral may require reset

Note: Yields CPU via tx_thread_sleep(1) every iteration (cooperative multitasking)

Warning: Do NOT call with timeout_ms = 0 (will return timeout immediately on RX72N)

Warning: Timeout does NOT guarantee exact millisecond precision (+/-10ms due to tick granularity)

Performance:: Best case: 10 us (host ready immediately, 1 HAL call) Typical: 10-50 ms (host ready within 1-5 polling iterations) Worst case: timeout_ms (50 ms default, host never ready)

ThreadX Timing:: Tick rate: 100 Hz (k_threadx_ms_per_tick = 10 ms) Sleep duration: 1 tick = 10 ms per iteration Timeout conversion: (timeout_ms + 9) / 10 ticks (round up)

Example - Normal Transmission:: rx_err_t err=internal_wait_for_ack(handle,50);
if(err==k_rx_ok){ rspi_peripheral_transfer(handle->channel,tx_buf,rx_buf,len); }
elseif(err==k_rx_err_timeout){ rx_log_error("spi","HostACKtimeout-RPi5maybedown"); }

Example - Timeout Handling:: for(uint8_t retry=0;retry<3;retry++)
{ rx_err_t err=internal_wait_for_ack(handle,50); if(err==k_rx_ok)break;

```
if(err==k_rx_err_timeout){ rx_log_warn("spi","Hostnotready,retry%d/3",retry+1);
tx_thread_sleep(10); }else{ returnerr; }}
```

See also: `rspi_peripheral_write_ready()` RSPI HAL function for ready status,
`internal_spi_transfer()` Calls this function before TX operations,
`k_ack_wait_timeout_ms` Default timeout constant (50ms)

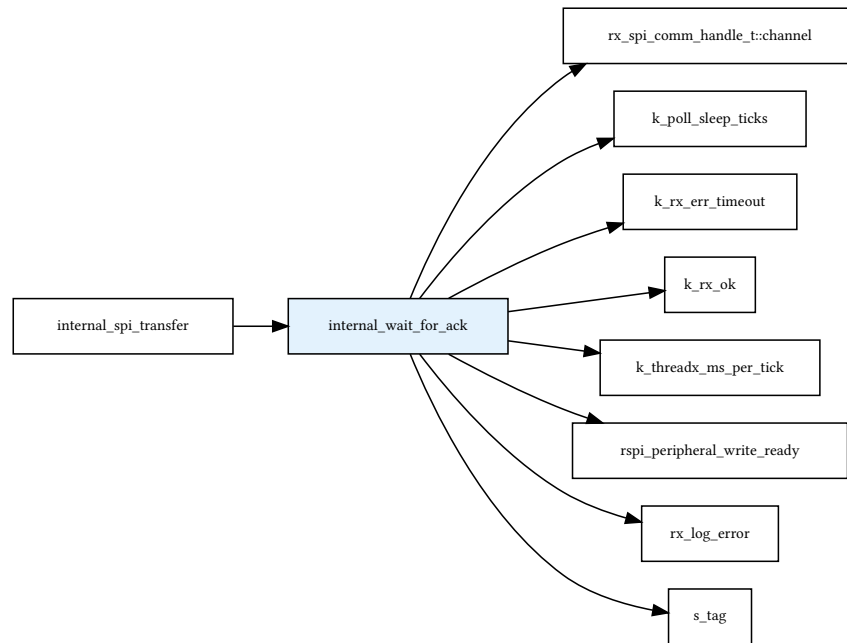


Figure 1105: Call graph for `internal_wait_for_ack`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:709`

Since Version 1.0.0

—

internal_spi_transfer

```
static rx_err_t internal_spi_transfer(rx_spi_comm_handle_t *handle, const uint8_t
*tx_data, const uint32_t tx_len, uint8_t *rx_data, const uint32_t rx_len)
```

Perform raw SPI full-duplex transfer using handle's staging buffers.

Low-level SPI transfer wrapper that uses handle's internal TX/RX staging buffers (2048 bytes each) to perform full-duplex communication with RPi5 controller. Implements flow control by waiting for host ready signal before transmit operations.

Why Staging Buffers? RSPI peripheral DMA requires contiguous aligned buffers. Using handle's pre-allocated staging buffers avoids dynamic allocation and ensures proper alignment for DMA transfers.

Algorithm:

1. Pre-condition 1: Validate handle pointer (`RX_CHECK_NULL_PTR`)

2. Pre-condition 2: Calculate `transfer_len = max(tx_len, rx_len)`
3. Pre-condition 3: Validate `transfer_len <= k_spi_comm_tx_buffer_size (2048)`
4. Pre-condition 4: Validate TX data pointer consistent with `tx_len`
5. Pre-condition 5: Validate RX data pointer consistent with `rx_len`
6. Flow control: If transmitting (`tx_data != nullptr`), wait for host ready via `internal_wait_for_ack()`
7. Prepare TX buffer: Zero-fill staging buffer, copy `tx_data` if present
8. SPI transfer: Call `rspi_peripheral_transfer()` with staging buffers
9. Post-condition: Copy RX staging buffer to `rx_data` if requested

Full-Duplex Behavior:

- SPI is inherently full-duplex (simultaneous TX and RX)
- If `tx_len < rx_len`: TX buffer zero-padded, RX receives full `rx_len` bytes
- If `tx_len > rx_len`: RX buffer receives `tx_len` bytes, caller extracts `rx_len`
- Transfer length = `max(tx_len, rx_len)` to accommodate both directions

Parameters:

Direction	Name	Description
inout	<code>handle</code>	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Usage: <code>handle->tx_buffer</code> and <code>handle->rx_buffer</code> used for staging Channel: <code>handle->channel</code> selects RSPI peripheral (0-2)
in	<code>tx_data</code>	Transmit data buffer (or nullptr for RX-only) Valid range: nullptr or pointer to <code>tx_len</code> bytes nullptr semantics: Receive-only operation (TX buffer zero-filled) Constraints: If non-nullptr, must contain <code>tx_len</code> valid bytes
in	<code>tx_len</code>	Transmit data length in bytes Valid range: 0-2048 bytes (<code>k_spi_comm_tx_buffer_size</code>) Usage: Bytes to copy from <code>tx_data</code> to staging buffer Zero semantics: No TX data, RX-only operation
out	<code>rx_data</code>	Receive data buffer (or nullptr if discarding RX) Valid range: nullptr or pointer to <code>rx_len</code> bytes nullptr semantics: Transmit-only, discard received data Output: Populated with received bytes from SPI transfer
in	<code>rx_len</code>	Receive data length in bytes Valid range: 0-2048 bytes (<code>k_spi_comm_rx_buffer_size</code>) Usage: Bytes to copy from staging buffer to <code>rx_data</code> Zero semantics: TX-only operation, discard RX data

Returns: `rx_err_t` Error code indicating transfer result

Value	Description
<code>k_rx_ok</code>	Transfer completed successfully
<code>k_rx_err_null_ptr</code>	handle pointer is nullptr
<code>k_rx_err_invalid_size</code>	<code>transfer_len > 2048</code> (exceeds buffer capacity)
<code>k_rx_err_invalid_arg</code>	nullptr <code>tx_data</code> with non-zero <code>tx_len</code>
<code>k_rx_err_invalid_arg</code>	nullptr <code>rx_data</code> with non-zero <code>rx_len</code>
<code>k_rx_err_timeout</code>	Host ACK timeout (if transmitting, see <code>internal_wait_for_ack</code>)
(HAL)	errors) RSPI peripheral transfer failure (overrun, mode fault, etc.)

Pre: handle must be initialized via rx_spi_comm_init()

Pre: If tx_data != nullptr, tx_len must be > 0 and <= 2048

Pre: If rx_data != nullptr, rx_len must be > 0 and <= 2048

Pre: tx_data and rx_data must not alias (undefined behavior if overlapping)

Post: On k_rx_ok, rx_data (if non-nullptr) contains received bytes

Post: On k_rx_ok, handle->tx_buffer and handle->rx_buffer modified

Post: On error, rx_data contents are undefined

Note: This function uses handle's staging buffers (not caller's buffers directly)

Note: Transfer is synchronous (blocks until complete or timeout)

Warning: Do NOT pass overlapping tx_data and rx_data buffers

Warning: This function is NOT thread-safe (use external mutex if multi-threaded)

Performance:: Execution time: 80-400 us @ 10 MHz SPI (depends on transfer_len) 10-byte transfer (ACK/NACK): 80 us (8 us SPI + 72 us overhead) 100-byte transfer: 180 us (80 us SPI + 100 us overhead) 1038-byte transfer (max frame): 1000 us (830 us SPI + 170 us overhead) Host ACK wait: Add 0-50 ms if transmitting (see internal_wait_for_ack)

Example - Transmit Only:: uint8_t tx_frame[14]={0xAA,0x55,...};
rx_err_t err=internal_spi_transfer(handle,tx_frame,14,nullptr,0); if(err==k_rx_ok)
{ rx_log_debug("spi","ACKsent successfully"); }

Example - Receive Only:: uint8_t rx_header[10];
rx_err_t err=internal_spi_transfer(handle,nullptr,0,rx_header,10); if(err==k_rx_ok)
{ uint16_t sync=(rx_header[0]<<8)|rx_header[1]; if(sync==0xAA55){ }

Example - Full-Duplex:: uint8_t tx_buf[50]={...}; uint8_t rx_buf[50];
rx_err_t err=internal_spi_transfer(handle,tx_buf,50,rx_buf,50); if(err==k_rx_ok){ }

See also: internal_wait_for_ack() Flow control for TX operations,
rspi_peripheral_transfer() RSPI HAL transfer function, rx_spi_comm_handle_t Handle
structure with staging buffers

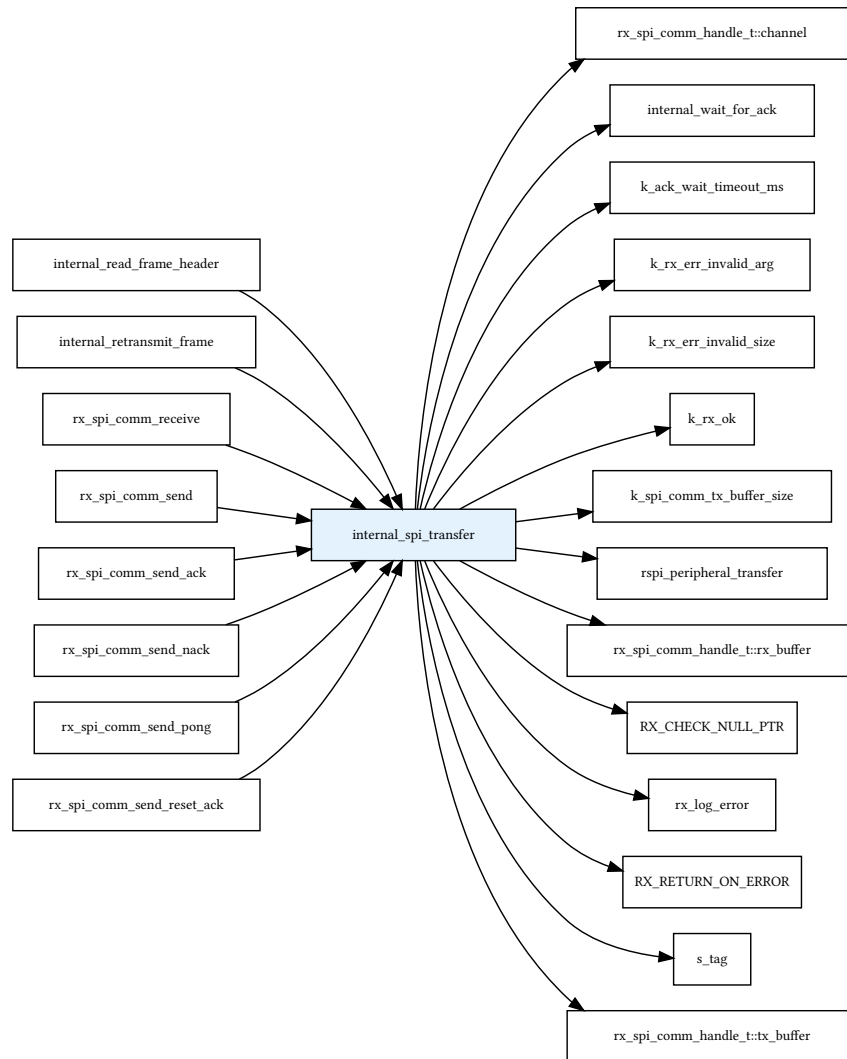


Figure 1106: Call graph for internal_spi_transfer

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:874`

Since Version 1.0.0

—

rx_spi_comm_init

```
rx_err_t rx_spi_comm_init(rx_spi_comm_handle_t *handle, const
rx_spi_comm_config_t *config)
```

Initialize SPI communication layer and allocate resources.

Initialize SPI communication handle with frame protocol support.

Initializes SPI communication handle for frame-based communication between RX72N (peripheral) and RPi5 (controller). Sets up frame encoder/decoder, clears staging buffers, and configures channel and FEC settings.

Initialization Steps:

1. Validate handle: Check handle pointer non-nullptr
2. Zero handle: memset entire structure to clear old state
3. Apply configuration: Use config values or defaults (RSPi0, no FEC)
4. Initialize encoder: Call rx_frame_encoder_init() for TX encoding
5. Initialize decoder: Call rx_frame_decoder_init() for RX decoding
6. Reset sequence counters: Set TX/RX sequence to 0
7. Mark initialized: Set handle->initialized = true

Configuration (config must be non-nullptr):

- Channel: Must specify RSPi channel (e.g., k_rspi_channel_2 for host)
- FEC: fec_enabled flag (true/false)
- Passing config == nullptr returns k_rx_err_invalid_arg

Memory Allocation: This function does NOT allocate memory (NASA Rule 3 compliance). Caller must provide pre-allocated handle (typically static or on task stack).

Parameters:

Direction	Name	Description
out	handle	SPI communication handle to initialize (4120 bytes) Valid range: Non-nullptr pointer to rx_spi_comm_handle_t Allocation: Caller-allocated (static, stack, or global) Side effects: Entire structure zeroed and re-initialized
in	config	Configuration parameters (must not be nullptr) Valid range: Non-nullptr pointer to rx_spi_comm_config_t Required fields: session (non-nullptr), channel (0-2), fec_enabled (true/false)

Returns: rx_err_t Error code indicating initialization result

Value	Description
k_rx_ok	Initialization successful, handle ready for use
k_rx_err_invalid_arg	handle pointer is nullptr
(encoder	errors) rx_frame_encoder_init() failed (rare, internal error)
(decoder	errors) rx_frame_decoder_init() failed (rare, internal error)

Pre: handle must point to valid rx_spi_comm_handle_t storage (4120 bytes)

Pre: config != nullptr

Pre: config->session != nullptr

Pre: config->channel must be a valid rspi_channel_t value (0-2)

Post: On success, handle->initialized = true

Post: On success, handle->tx_sequence = 0, handle->rx_sequence = 0

Post: On success, handle->encoder and handle->decoder initialized

Post: On failure (encoder init), handle state is zeroed but not usable

Post: On failure (decoder init), encoder is cleaned up, handle unusable

Note: Call this ONCE per handle before any send/receive operations

Note: This function is NOT thread-safe (single-threaded init only)

Warning: Do not call multiple times without rx_spi_comm_deinit() first

Warning: RSPI peripheral hardware (rspi HAL) must be initialized separately

Performance:: Execution time: 50 us @ 240 MHz (memset + encoder/decoder init)

Example - Basic Configuration:: static rx_spi_comm_handle_tg_spi_handle;

```
rx_spi_comm_config_t config = { .session = &session, .channel = k_rspi_channel_0, .fec_enabled = false, };
rx_err_t err = rx_spi_comm_init(&g_spi_handle, &config); if (err != k_rx_ok)
{ rx_log_error("init", "SPIcomm init failed"); return; }
```

```
rx_spi_comm_send(&g_spi_handle, k_frame_type_data, 0, payload, len);
```

Example - FEC Enabled:: static rx_spi_comm_handle_tg_spi_handle;

```
rx_spi_comm_config_t config = { .channel = 0, .fec_enabled = true, };
rx_err_t err = rx_spi_comm_init(&g_spi_handle, &config); if (err != k_rx_ok)
{ rx_log_info("init", "SPIcomm initialized with FEC"); }
```

Example - Multiple RSPI Channels:: static rx_spi_comm_handle_tg_spi1_handle;

```
rx_spi_comm_config_t config = { .channel = 1, .fec_enabled = false, };
rx_spi_comm_init(&g_spi1_handle, &config);
```

See also: rx_spi_comm_deinit() Clean up resources when done, rx_spi_comm_config_t Configuration structure definition, rx_frame_encoder_init() Frame encoder initialization, rx_frame_decoder_init() Frame decoder initialization

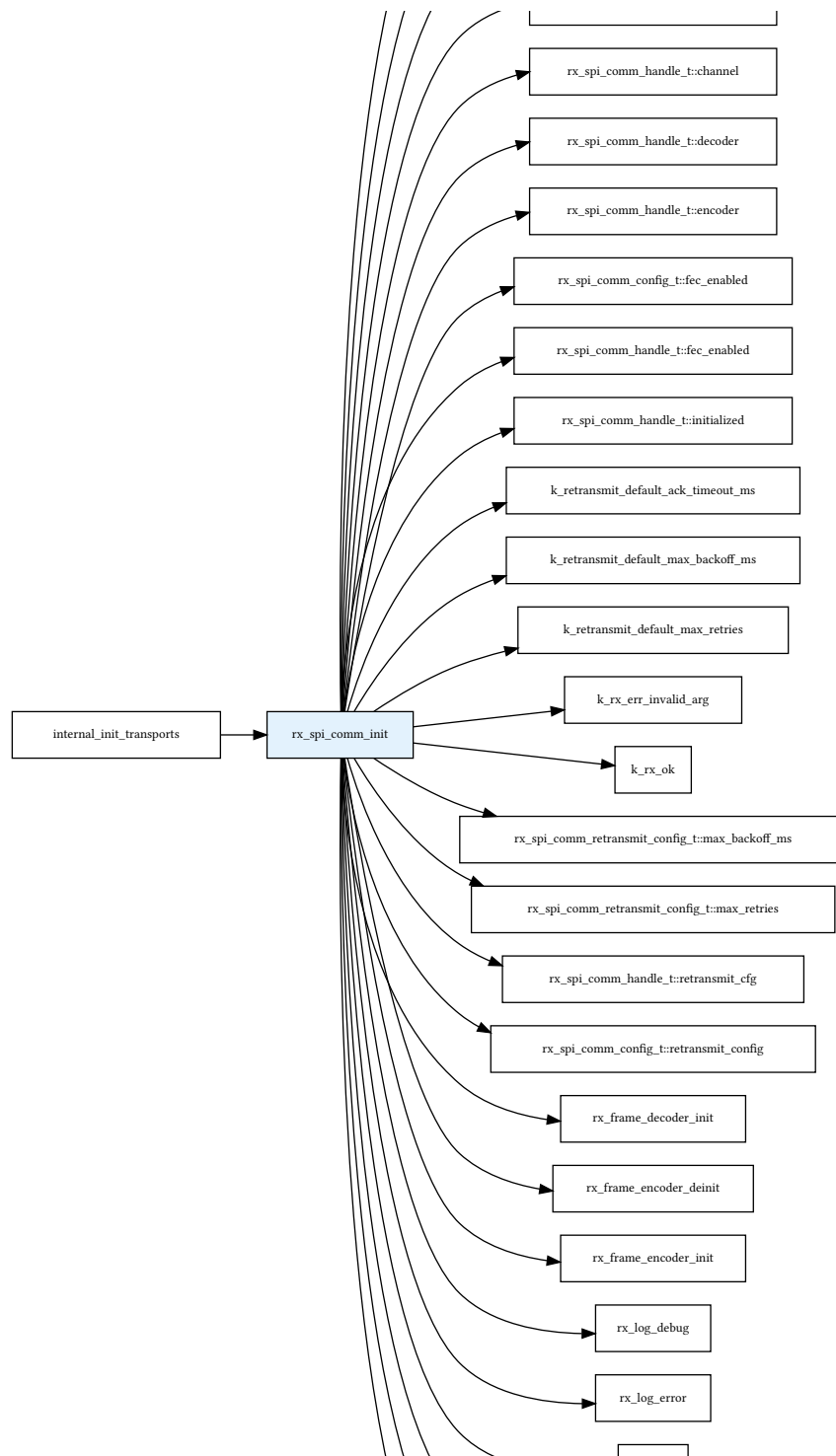


Figure 1107: Call graph for rx_spi_comm_init

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1069`

Since Version 1.0.0

—

rx_spi_comm_deinit

```
rx_err_t rx_spi_comm_deinit(rx_spi_comm_handle_t *handle)
```

Deinitialize SPI communication layer.

Deinitialize SPI communication handle and release resources.

Tears down the encoder and decoder, clearing internal state. Saves the first error encountered from sub-deinit calls and returns it after completing all teardown steps.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle to deinitialize

Value	Description
k_rx_ok	Deinit completed successfully
k_rx_err_invalid_arg	handle is nullptr
(other)	First error from encoder or decoder deinit

Pre: handle != nullptr

Pre: handle was previously initialized via `rx_spi_comm_init()`

Post: handle->initialized == false

Post: Encoder and decoder resources released

Note: Not thread-safe; caller must ensure no concurrent SPI operations.] **See also:** `rx_spi_comm_init()` Corresponding initialization function

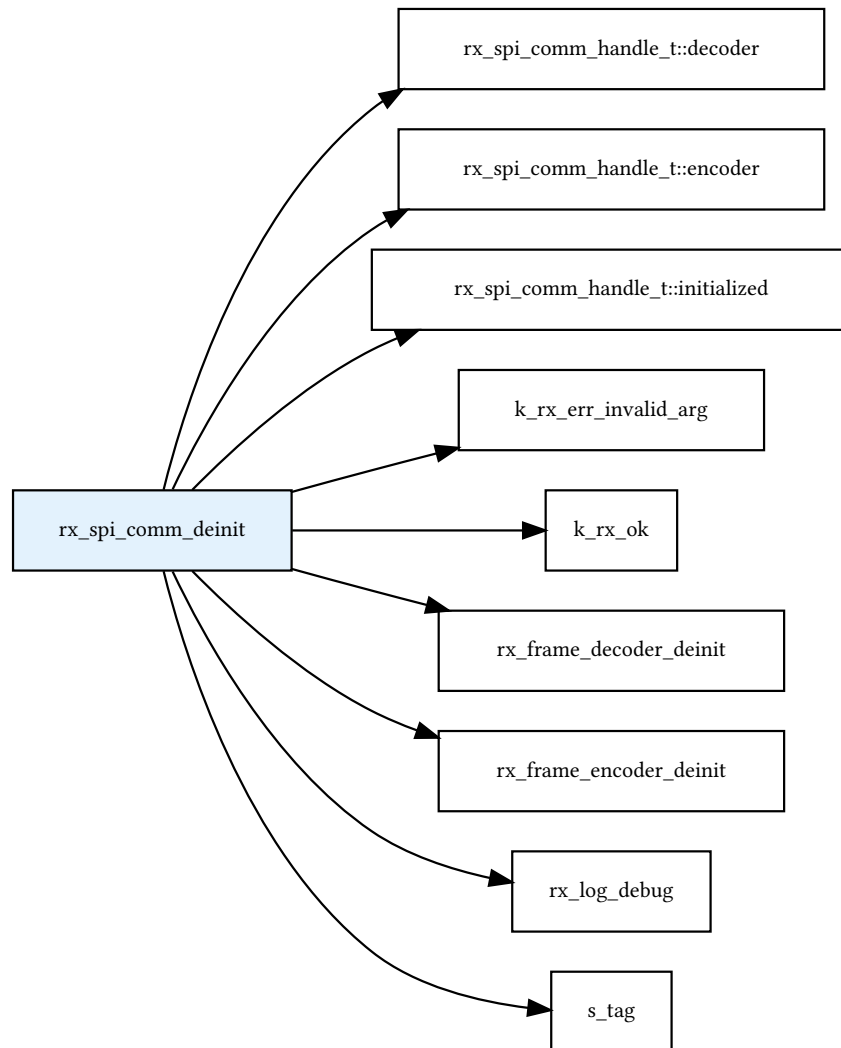


Figure 1108: Call graph for rx_spi_comm_deinit

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1147`

Since Version 1.0.0

—

internal_build_frame

```
static rx_err_t internal_build_frame(const rx_spi_comm_handle_t *handle, const
uint16_t sequence, const rx_frame_type_t type, const uint8_t flags, const uint8_t
*payload, const uint32_t payload_len, rx_frame_t *frame)
```

Build frame structure from application payload data.

Populates `rx_frame_t` structure with header fields (sequence, length, type, flags) and payload data, preparing it for encoding via `rx_frame_encode()`. Automatically sets sequence number from handle's TX counter and applies FEC flag if enabled in configuration.

Algorithm:

1. Validate inputs: Check handle, frame, payload pointer consistency
2. Validate payload size: Ensure `payload_len` <= 1024 bytes (protocol limit)
3. Zero frame: Clear `rx_frame_t` to ensure padding bytes are zero
4. Set sequence: Use `handle->tx_sequence` (auto-incremented by sender)
5. Set length: `payload_len` (0-1024)
6. Set type: Frame type (data, ACK, NACK, telemetry, etc.)
7. Set flags: User-provided flags
8. Copy payload: `memcpy` `payload_len` bytes to `frame->payload` if present
9. Apply FEC flag: Set `k_frame_flag_fec_enabled` if `handle->fec_enabled`

Frame Types Supported:

- `k_frame_type_data`: General data payload (0-1024 bytes)
- `k_frame_type_ack`: Acknowledgment (0 bytes payload, handled separately)
- `k_frame_type_nack`: Negative acknowledgment (0 bytes, handled separately)
- `k_frame_type_telemetry`: Sensor/status data
- `k_frame_type_command`: Motor control commands

Parameters:

Direction	Name	Description
in	<code>handle</code>	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Usage: <code>handle->tx_sequence</code> used for frame sequence number Usage: <code>handle->fec_enabled</code> determines if FEC flag is set
in	<code>type</code>	Frame type from <code>rx_frame_type_t</code> enum Valid values: <code>k_frame_type_data</code> , <code>k_frame_type_telemetry</code> , <code>k_frame_type_command</code> Note: ACK/NACK should use <code>rx_frame_create_ack/nack()</code> instead
in	<code>flags</code>	Frame flags bitmask Valid bits: <code>k_frame_flag_fec_enabled</code> , <code>k_frame_flag_harq_enabled</code> Auto-set: <code>k_frame_flag_fec_enabled</code> added if <code>handle->fec_enabled</code> true Usage: Caller can pass 0 or specific flags, FEC auto-added
in	<code>payload</code>	Application payload data (or nullptr if no payload) Valid range: nullptr or pointer to <code>payload_len</code> bytes nullptr semantics: Zero-length payload (e.g., ping frames) Constraints: Must contain <code>payload_len</code> valid bytes if non-nullptr
in	<code>payload_len</code>	Payload length in bytes Valid range: 0-1024 bytes (<code>k_frame_max_payload</code>) Typical: 0 (ACK/NACK), 8-64 (telemetry), 100-1024 (bulk data) Constraints: Must be <= 1024 (protocol specification limit)
out	<code>frame</code>	Frame structure to populate Valid range: Non-nullptr pointer to <code>rx_frame_t</code> (304 bytes) Output: <code>frame->header</code> and <code>frame->payload</code> populated Note: <code>frame->crc</code> NOT set (added by <code>rx_frame_encode()</code>)

Returns: rx_err_t Error code indicating build result

Value	Description
k_rx_ok	Frame built successfully, ready for encoding
k_rx_err_invalid_arg	handle or frame pointer is nullptr
k_rx_err_invalid_arg	payload is nullptr but payload_len > 0
k_rx_err_invalid_size	payload_len > 1024 (exceeds protocol limit)

Pre: handle must be initialized via rx_spi_comm_init()

Pre: If payload != nullptr, payload_len must be > 0 and <= 1024

Pre: If payload == nullptr, payload_len must be 0

Post: On success, frame->header populated with sequence, length, type, flags

Post: On success, frame->payload contains payload_len bytes from payload

Post: On success, frame ready for rx_frame_encode()

Post: On failure, frame contents are undefined

Note: This function does NOT increment handle->tx_sequence (caller does that)

Note: This function does NOT compute CRC (rx_frame_encode() does that)

Warning: Do not modify frame after build and before encode (breaks CRC)

Performance:: Execution time: 5-20 us @ 240 MHz (depends on payload_len memcpy)

Example - Data Frame:: uint8_t telemetry[32]={...}; rx_frame_t frame;

```
rx_err_t err=internal_build_frame(handle, k_frame_type_telemetry, 0, telemetry, sizeof(telemetry),
&frame); if(err==k_rx_ok){ uint8_t wire_buf[279]; uint32_t wire_len; rx_frame_encode(&handle-
>encoder,&frame,wire_buf,&wire_len); }
```

Example - Zero-Length Payload:: rx_frame_t ping_frame;

```
rx_err_t err=internal_build_frame(handle, k_frame_type_data, 0, nullptr, 0, &ping_frame);
```

See also: rx_spi_comm_send() Uses this function to build frames before sending,
rx_frame_encode() Encodes frame to wire format after build, rx_frame_create_ack()
Special builder for ACK frames, rx_frame_create_nack() Special builder for NACK frames

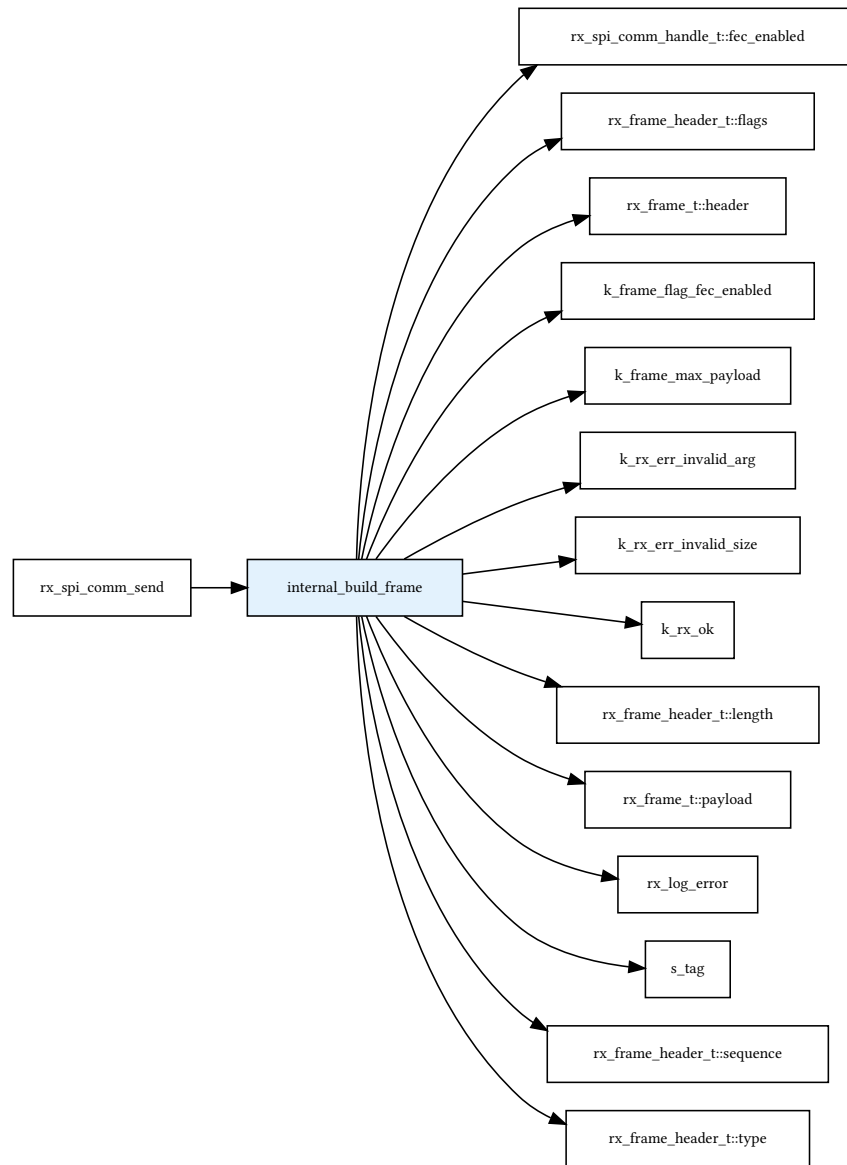


Figure 1109: Call graph for `internal_build_frame`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1291`

Since Version 1.0.0

—

rx_spi_comm_send

```
rx_err_t rx_spi_comm_send(rx_spi_comm_handle_t *handle, const rx_frame_type_t
type, const uint8_t flags, const uint8_t *payload, const uint32_t payload_len)
```

Send data frame to RPi5 controller via SPI.

Send frame on SPI with automatic sequencing and CRC.

Transmits application payload to RPi5 by building frame structure, encoding to wire format with CRC-32, waiting for host ready signal, performing SPI transfer, and incrementing TX sequence counter. This is the primary transmit function for all data frames.

Algorithm:

1. Pre-condition 1: Validate handle pointer non-nullptr
2. Pre-condition 2: Validate handle initialized
3. Build frame: Call internal_build_frame() to populate rx_frame_t

Sets sequence number from handle->tx_sequence Copies payload (0-1024 bytes) Sets type, flags, applies FEC flag if configured

4. Encode frame: Call rx_frame_encode() to create wire buffer

Adds sync word (0xAA55) Encodes header fields (little-endian) Computes CRC-32 over header + payload Appends CRC-32 (little-endian)

5. Post-condition 1: Validate encoded length in valid range (14-279 bytes)
6. Transfer via SPI: Call internal_spi_transfer()

Waits for RPi5 ready signal (50ms timeout) Performs DMA/IRQ-based SPI transfer

7. Post-condition 2: Increment TX sequence counter

handle->tx_sequence++ (wraps at 65536) Validates increment successful (NASA Rule 5)

Frame Format on Wire (transmitted bytes):

Sequence Number Behavior:

- Auto-incremented after each successful send
- Starts at 0, wraps to 0 after 65535
- Receiver uses sequence to detect duplicates, out-of-order, missing frames

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Side effects: handle->tx_sequence incremented on success Usage: Uses handle->tx_buffer for staging (2048 bytes)
in	type	Frame type from rx_frame_type_t enum Valid values: k_frame_type_data, k_frame_type_telemetry, k_frame_type_command Common: k_frame_type_data (general purpose) Note: Use rx_spi_comm_send_ack/nack() for ACK/NACK frames
in	flags	Frame flags bitmask Valid bits: k_frame_flag_fec_enabled, k_frame_flag_harq_enabled Typical: 0 (no special flags, FEC auto-added if configured) Auto-set: k_frame_flag_fec_enabled added if handle->fec_enabled

in	payload	Application data to send (or nullptr if no payload) Valid range: nullptr or pointer to payload_len bytes nullptr semantics: Zero-length frame (14 bytes on wire) Constraints: Must contain payload_len valid bytes if non-nullptr
in	payload_len	Payload length in bytes Valid range: 0-1024 bytes (k_frame_max_payload) Typical: 8-64 (telemetry), 100-200 (bulk data) Zero: Valid for ping/keepalive frames

Returns: rx_err_t Error code indicating send result

Value	Description
k_rx_ok	Frame sent successfully, TX sequence incremented
k_rx_err_invalid_arg	handle pointer is nullptr
k_rx_err_invalid_state	handle not initialized (call rx_spi_comm_init first)
k_rx_err_invalid_size	payload_len > 1024 (protocol violation)
k_rx_err_invalid_size	Encoded frame length invalid (internal error)
k_rx_err_timeout	RPi5 ready signal timeout (host not responding)
(encoder	errors) rx_frame_encode() failed (rare, internal error)
(HAL	errors) RSPI peripheral transfer failure (overrun, mode fault)

Pre: handle must be initialized via rx_spi_comm_init()

Pre: If payload != nullptr, payload_len must be > 0 and <= 1024

Pre: If payload == nullptr, payload_len must be 0

Pre: RSPI peripheral hardware must be initialized and enabled

Pre: RPi5 controller must be ready to receive (CS asserted, clock provided)

Post: On success, handle->tx_sequence incremented (wraps at 65536)

Post: On success, frame transmitted to RPi5

Post: On failure, TX sequence NOT incremented (safe to retry)

Post: On timeout, RPi5 may be disconnected or busy

Note: This function blocks until transfer complete or timeout (1-50 ms)

Note: TX sequence auto-increments, no manual management needed

Warning: Not thread-safe - use external mutex if multiple threads send

Warning: Do not modify payload buffer during call (memcpy in progress)

Performance:: Execution time: 150-500 us @ 10 MHz SPI + 0-50 ms host ACK wait 10-byte payload: 150 us (10 us build + 20 us encode + 120 us SPI) 100-byte payload: 300 us (15 us build + 100 us encode + 185 us SPI) 1024-byte payload: 800 us (20 us build + 250 us encode + 530 us SPI) Host ACK wait: Add 0-50 ms if RPi5 not immediately ready

Example - Send Telemetry Data:: uint8_t telemetry[32]; telemetry[0]=temperature_celsius&0xFF; telemetry[1]=(temperature_celsius>>8)&0xFF;

```
rx_err_t err = rx_spi_comm_send(&g_spi_handle, k_frame_type_telemetry, 0, telemetry,
sizeof(telemetry)); if(err==k_rx_ok)
{ rx_log_debug("spi", "Telemetry sent, seq=%d", g_spi_handle.tx_sequence-1); }
elseif(err==k_rx_err_timeout){ rx_log_warn("spi", "RPi5 not responding"); }
```

Example - Send Command with Retry:: uint8_t cmd[]={0x01,0x02,0x03};

```
for(uint8_t retry=0; retry<3; retry++){ rx_err_t err = rx_spi_comm_send(&g_spi_handle,
k_frame_type_command, 0, cmd, sizeof(cmd)); if(err==k_rx_ok){ break; }
elseif(err==k_rx_err_timeout){ rx_log_warn("spi", "Timeout, retry%d/3", retry+1);
tx_thread_sleep(10); }else{ rx_log_error("spi", "Send failed: %d", err); break; } }
```

Example - Zero-Length Frame (Ping):: rx_err_t err = rx_spi_comm_send(&g_spi_handle, k_frame_type_data, 0, nullptr, 0);

Example:

```
+-----+-----+-----+-----+-----+-----+
| SYNC | Sequence | Length | Type | Flags | Payload | CRC-32 |
| 2B | 2B | 2B | 1B | 1B | 0-1024B | 4B |
| 0x55AA | LE | LE | | | LE |
+-----+-----+-----+-----+-----+-----+-----+
```

See also: rx_spi_comm_send_ack() Send acknowledgment frame, rx_spi_comm_send_nack() Send negative acknowledgment, rx_spi_comm_receive() Receive frames from RPi5, internal_build_frame() Frame building implementation, internal_spi_transfer() SPI transfer with flow control

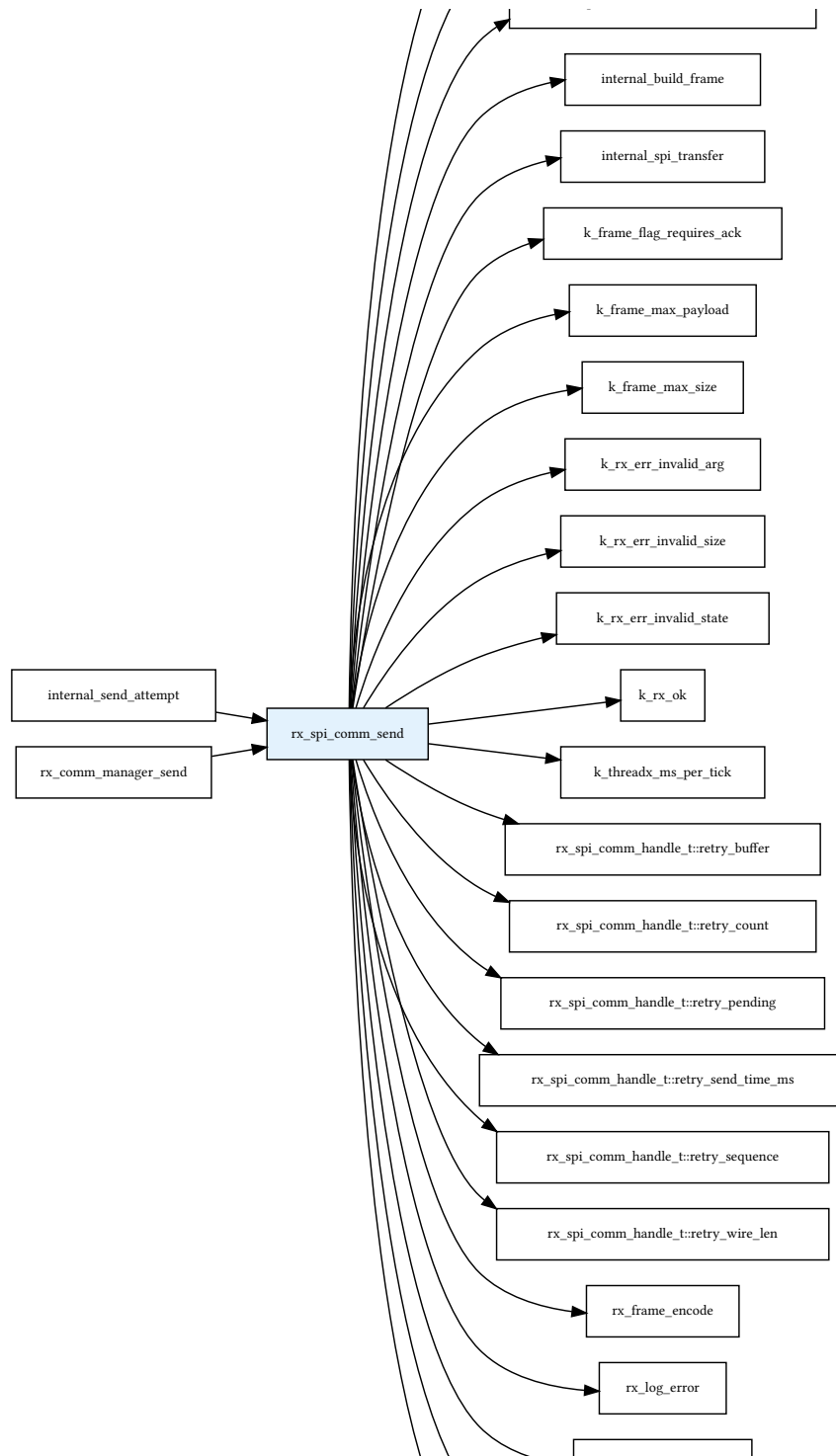


Figure 1110: Call graph for rx_spi_comm_send

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1492`

Since Version 1.0.0

—

rx_spi_comm_send_ack

```
rx_err_t rx_spi_comm_send_ack(rx_spi_comm_handle_t *handle, const uint16_t
sequence)
```

Send ACK frame for specified sequence number.

Send ACK frame for received frame (convenience wrapper).

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle
in	sequence	Sequence number to acknowledge

Returns: Error code from frame encoding or SPI transfer on failure

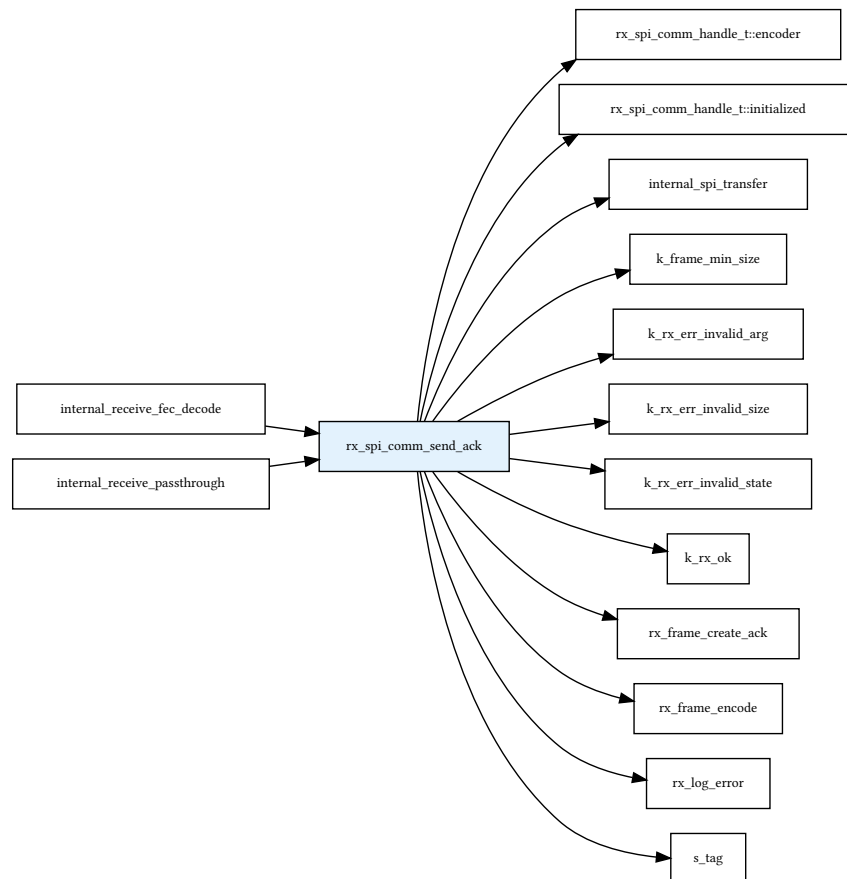


Figure 1111: Call graph for rx_spi_comm_send_ack

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1582`

rx_spi_comm_send_nack

```
rx_err_t rx_spi_comm_send_nack(rx_spi_comm_handle_t *handle, const uint16_t
sequence, uint8_t flags)
```

Send NACK frame for specified sequence number.

Send NACK frame for received frame with error indication.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle
in	sequence	Sequence number to negatively acknowledge
in	flags	NACK flags (e.g., k_frame_flag_soft_nack)

Returns: Error code from frame encoding or SPI transfer on failure

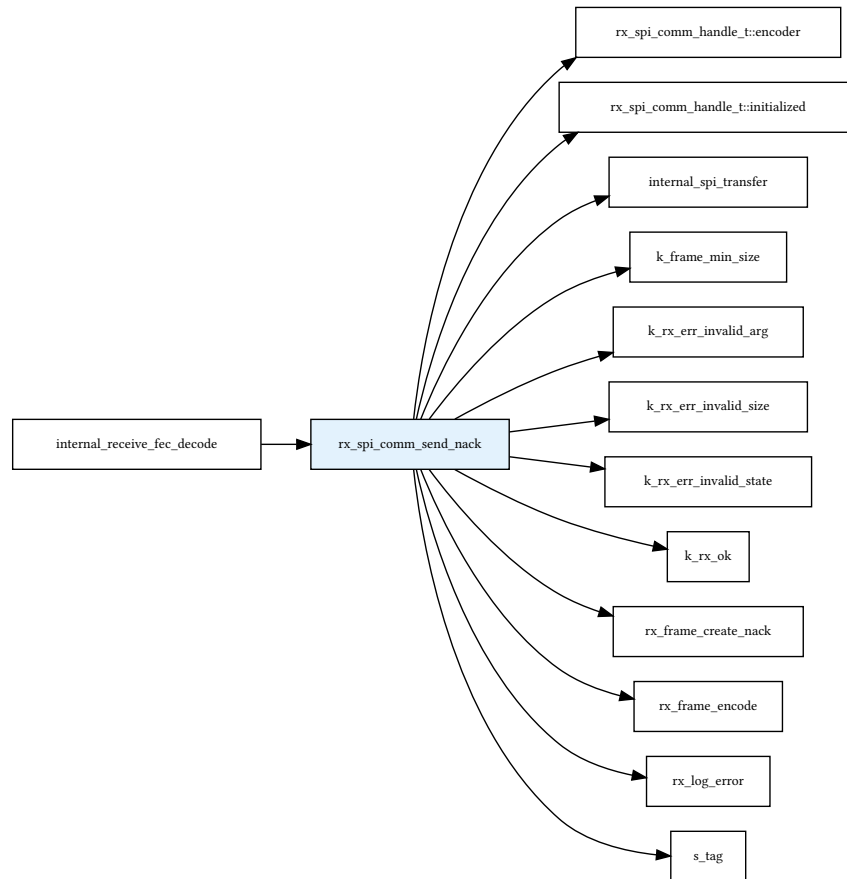


Figure 1112: Call graph for `rx_spi_comm_send_nack`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1629`

`internal_wait_for_data`

```
static rx_err_t internal_wait_for_data(const rx_spi_comm_handle_t *handle, const
uint32_t timeout_ms)
```

Wait for data to be available on SPI channel.

Parameters:

Direction	Name	Description
in	<code>handle</code>	SPI communication handle
in	<code>timeout_ms</code>	Maximum time to wait in milliseconds (0 = no wait)

Returns: `k_rx_ok` if data available, `k_rx_err_timeout` if timeout expired

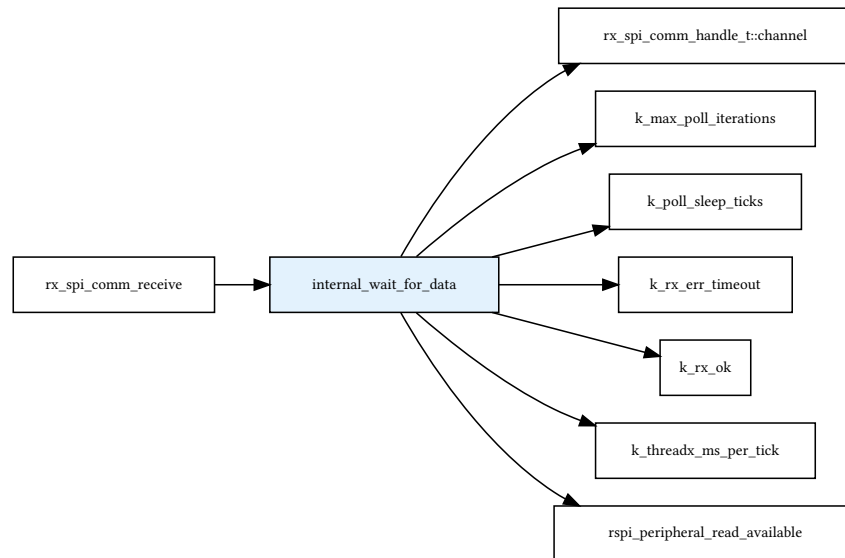


Figure 1113: Call graph for internal_wait_for_data

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1677`

internal_read_frame_header

```
static rx_err_t internal_read_frame_header(rx_spi_comm_handle_t *handle, uint8_t
*header_buf, uint16_t *payload_len)
```

Read and validate frame header from SPI.

Parameters:

Direction	Name	Description
in	handle	SPI communication handle
in	header_buf	Output buffer for header (must be k_frame_sync_size + k_frame_header_size)
in	payload_len	Output pointer for extracted payload length

Returns: k_rx_ok on success, error code on failure

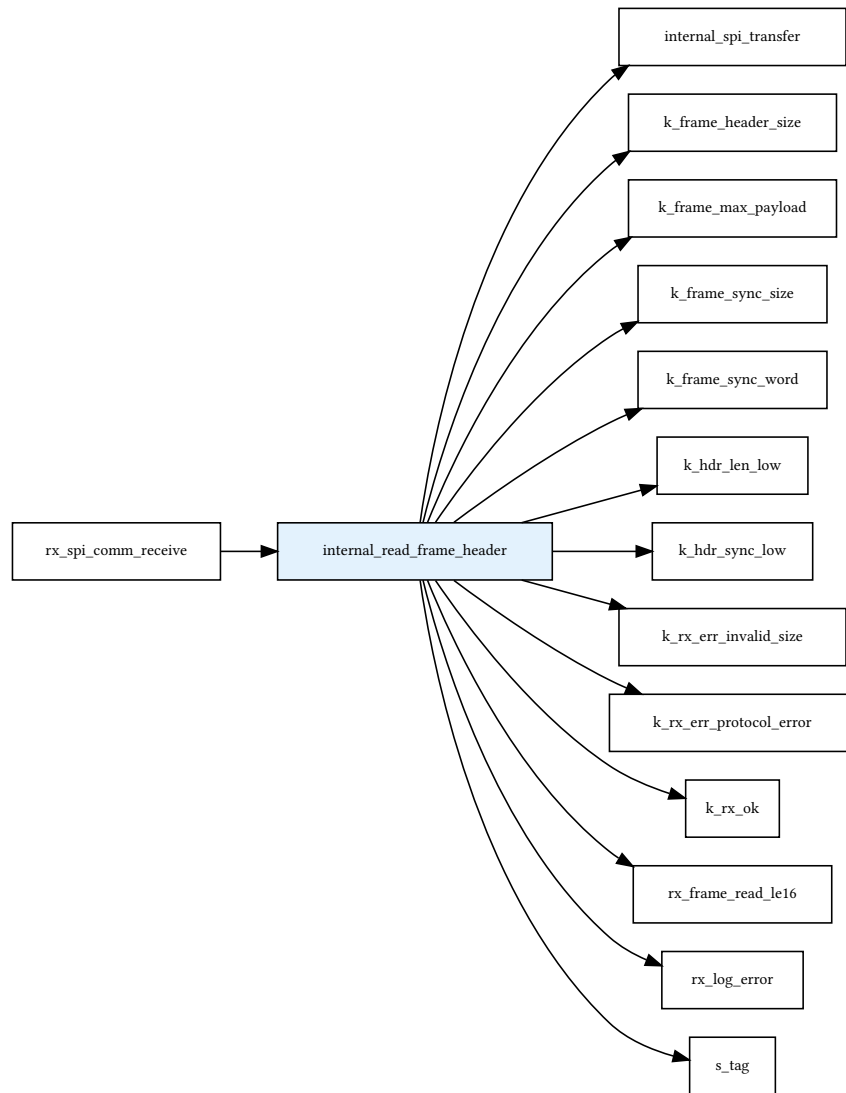


Figure 1114: Call graph for internal_read_frame_header

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1731`

internal_decode_frame

```
static rx_err_t internal_decode_frame(const rx_spi_comm_handle_t *handle,
rx_frame_t *frame, const uint32_t total_size)
```

Decode and verify a received SPI frame.

Parameters:

Direction	Name	Description
in	handle	SPI communication handle
out	frame	Frame to populate (header, payload, CRC)
in	total_size	Total frame size in bytes (header + payload + CRC)

Returns: k_rx_err_crc_mismatch from internal_verify_crc

Note: Delegates header validation to internal_decode_header and CRC validation to internal_verify_crc.

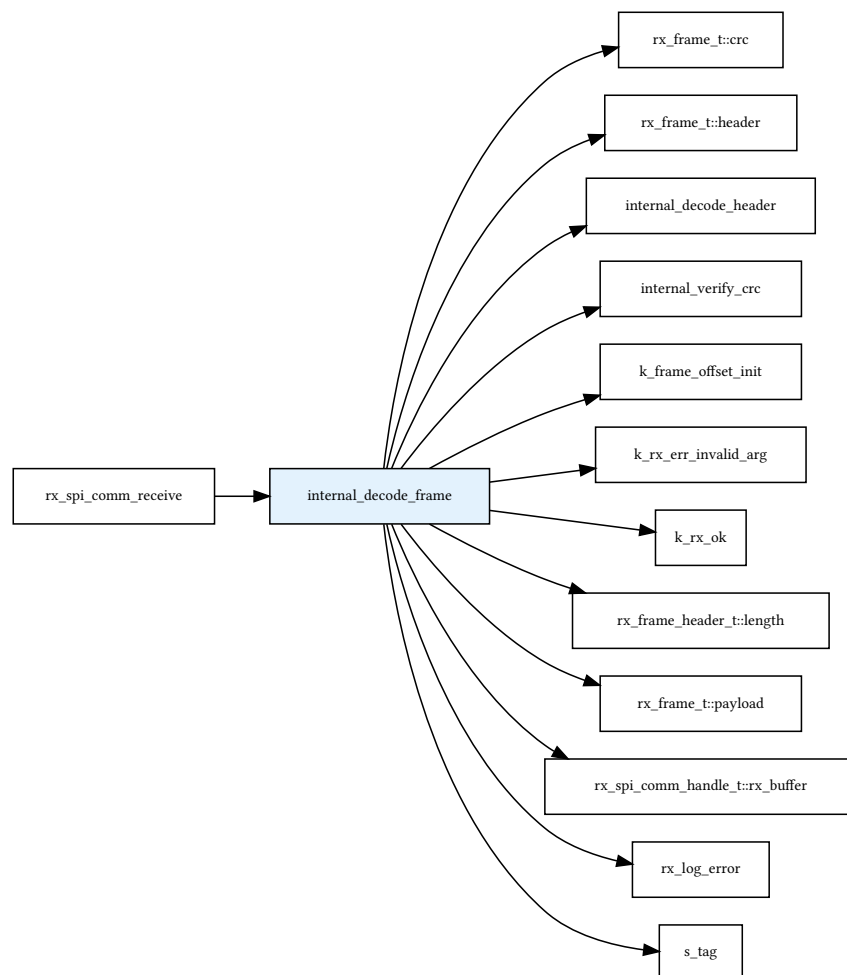


Figure 1115: Call graph for internal_decode_frame

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1774`

internal_dispatch_control_frame

```
static rx_err_t internal_dispatch_control_frame(rx_spi_comm_handle_t *handle,  
rx_frame_t *frame, ctrl_dispatch_result_t *result)
```

Dispatch control frames (PING, RESET, ACK/NACK) during receive.

Handles PING (auto-PONG + callback), RESET (auto-RESET_ACK + session reset

- callback), and ACK/NACK (retransmit subsystem dispatch). Data frames are not consumed and signal the caller to return to the application.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle
in	frame	Decoded frame to inspect
out	result	Set to consumed/not-handled/error

Returns: rx_err_t k_rx_ok unless a fatal error occurred

Pre: handle and frame must be non-nullptr and valid

Post: Control-frame side effects applied (PONG sent, session reset, etc.)

Note: Called only from rx_spi_comm_receive(); not part of the public API.

Figure 1116: Call graph for `internal_dispatch_control_frame`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:1835`

Since Version 1.2.0

—

rx_spi_comm_receive

```
rx_err_t rx_spi_comm_receive(rx_spi_comm_handle_t *handle, rx_frame_t *frame,
    const uint32_t timeout_ms)
```

Receive data frame from RPi5 controller via SPI (blocking with timeout).

Receive and decode frame from RPi5 with CRC validation.

Receives complete frame from RPi5 by polling for data availability, reading header, reading payload + CRC, validating frame integrity, and updating RX sequence counter. This is the primary receive function for all frame types.

Algorithm:

1. Pre-condition 1: Validate handle and frame pointers non-nullptr
2. Pre-condition 2: Validate handle initialized
3. Wait for data: Call `internal_wait_for_data()` with timeout

Polls RSPi peripheral for incoming data (yields CPU every 10ms) Returns `k_rx_err_timeout` if no data within `timeout_ms`

4. Read frame header: Call `internal_read_frame_header()`

Reads 10 bytes (SYNC + sequence + length + type + flags) Validates sync word (0xAA55) Extracts payload length (0-1024)

5. Read payload + CRC: Call `internal_spi_transfer()` for remaining bytes

Reads `payload_len + 4` bytes (CRC-32) Stores in `handle->rx_buffer`

6. Decode frame: Call `internal_decode_frame()`

Parses header fields into `frame->header` Copies payload to `frame->payload` Validates CRC-32 (returns `k_rx_err_crc_mismatch` if corrupt)

7. Update RX sequence: Set `handle->rx_sequence = frame->sequence + 1`

Tracks expected next sequence number Caller can detect out-of-order/missing frames

Blocking Behavior:

- `timeout_ms = 0`: Non-blocking, returns immediately if no data
- `timeout_ms > 0`: Blocks up to `timeout_ms` milliseconds waiting for data
- ThreadX sleep: Yields CPU every 10ms tick (not busy-wait)

Timeout Semantics:

- `k_rx_err_timeout` is NOT an error in polling loops (means “no data yet”)
- Typical polling: Call with `timeout_ms=100` in loop, handle timeout normally

Parameters:

Direction	Name	Description
inout	<code>handle</code>	SPI communication handle Valid range: Non-nullptr pointer to initialized handle Side effects: <code>handle->rx_sequence</code> updated on success Usage: Uses <code>handle->rx_buffer</code> for staging (2048 bytes)

out	frame	Frame structure to populate with received data Valid range: Non-nullptr pointer to rx_frame_t (304 bytes) Output: frame->header and frame->payload populated on success Output: frame->crc contains validated CRC-32 value
in	timeout_ms	Maximum wait time in milliseconds Valid range: 0-65535 ms 0: Non-blocking (return immediately if no data) Typical: 100-1000 ms (polling loop timeout) Precision: +/-10 ms (ThreadX tick granularity)

Returns: rx_err_t Error code indicating receive result

Value	Description
k_rx_ok	Frame received successfully, frame populated, RX sequence updated
k_rx_err_invalid_arg	handle or frame pointer is nullptr
k_rx_err_invalid_state	handle not initialized (call rx_spi_comm_init first)
k_rx_err_timeout	No data available within timeout_ms (normal in polling)
k_rx_err_protocol_error	Sync word invalid (not 0xAA55, resync needed)
k_rx_err_invalid_size	Payload length > k_frame_max_payload (protocol violation)
k_rx_err_crc_mismatch	CRC validation failed (corruption, send NACK)
(HAL	errors) RSPI peripheral read failure

Pre: handle must be initialized via rx_spi_comm_init()

Pre: frame must point to valid rx_frame_t storage (304 bytes)

Pre: RSPI peripheral hardware must be initialized and enabled

Pre: RPi5 controller must have transmitted complete frame

Post: On success, frame contains validated header, payload, CRC

Post: On success, handle->rx_sequence = frame->header.sequence + 1

Post: On timeout, frame contents undefined, RX sequence unchanged

Post: On CRC mismatch, frame partially populated, RX sequence unchanged

Post: On protocol error, frame undefined, may need resync

Note: This function blocks (with ThreadX sleep) if data not immediately available

Note: timeout_ms=0 provides non-blocking behavior for polling loops

Warning: Not thread-safe - use external mutex if multiple threads receive

Warning: Do not process frame if return value != k_rx_ok (data invalid)

Warning: Send NACK if k_rx_err_crc_mismatch to request retransmit

Performance:: Execution time: 200-600 us @ 10 MHz SPI + 0-timeout_ms wait 14-byte frame (ACK): 200 us (10 us wait + 80 us read + 110 us decode) 100-byte frame: 400 us (10 us wait + 200 us read + 190 us decode) 279-byte frame (max): 600 us (10 us wait + 300 us read + 290 us decode) Wait time: Add 0-timeout_ms if data not immediately available

Example - Polling Loop (Typical Usage):: while(1){ rx_frame_tframe;
rx_err_terr=rx_spi_comm_receive(&g_spi_handle,&frame,100);

if(err==k_rx_ok){ if(frame.header.type==k_frame_type_data){ process_data_frame(&frame);
rx_spi_comm_send_ack(&g_spi_handle,frame.header.sequence); } }elseif(err==k_rx_err_timeout)
{ tx_thread_sleep(1); }elseif(err==k_rx_err_crc_mismatch)
{ rx_spi_comm_send_nack(&g_spi_handle,frame.header.sequence,0); }
else{ rx_log_error("spi","Receiveerror:%d",err); } }

Example - Non-Blocking Check:: rx_frame_tframe;
rx_err_terr=rx_spi_comm_receive(&g_spi_handle,&frame,0);

if(err==k_rx_ok){ rx_log_debug("spi","Framereceived:seq=%d,len=%d",
frame.header.sequence,frame.header.length); }elseif(err==k_rx_err_timeout){ }

Example - Sequence Number Checking:: staticuint16_texpected_seq=0; rx_frame_tframe;
rx_err_terr=rx_spi_comm_receive(&g_spi_handle,&frame,1000); if(err==k_rx_ok)
{ if(frame.header.sequence!=expected_seq){ rx_log_warn("spi","Out-of-order:expected%d,got%d",
expected_seq,frame.header.sequence); } expected_seq=frame.header.sequence+1; }

See also: rx_spi_comm_send() Send frames to RPi5, rx_spi_comm_data_available() Check data availability without timeout, rx_spi_comm_send_ack() Send acknowledgment after successful receive, rx_spi_comm_send_nack() Send NACK after CRC mismatch, internal_wait_for_data() Wait for data with timeout, internal_decode_frame() Frame decoding and CRC validation

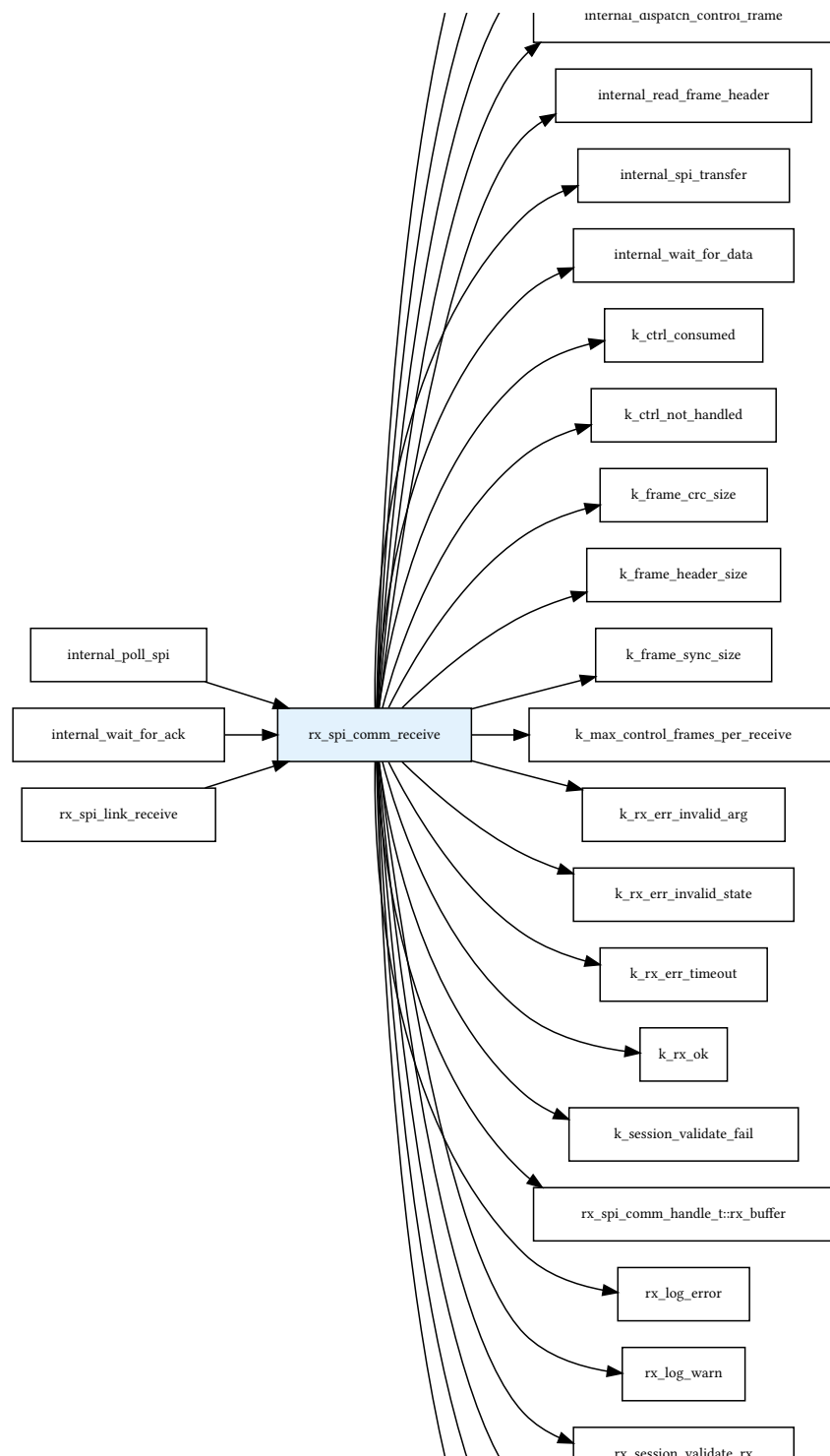


Figure 1117: Call graph for rx_spi_comm_receive

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2064`

Since Version 1.0.0

rx_spi_comm_data_available

```
rx_err_t rx_spi_comm_data_available(const rx_spi_comm_handle_t *handle, bool
*available)
```

Check if data is available to receive on SPI.

Check if data is available for reading (non-blocking query).

Parameters:

Direction	Name	Description
in	handle	SPI communication handle
out	available	Set to true if data available, false otherwise

Returns: Error code from SPI peripheral read availability check on failure

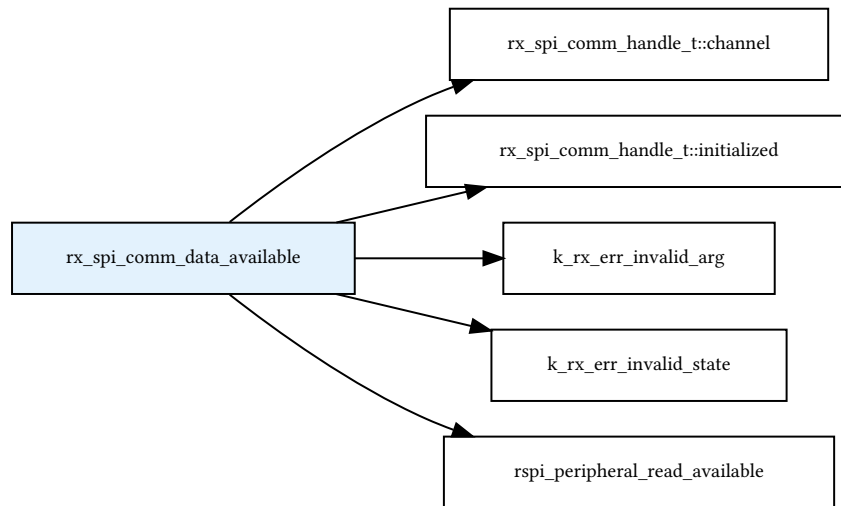


Figure 1118: Call graph for `rx_spi_comm_data_available`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2157`

rx_spi_comm_set_control_callbacks

```
rx_err_t rx_spi_comm_set_control_callbacks(rx_spi_comm_handle_t *handle,
void(*on_ping_cb)(const rx_frame_t *frame, void *ctx), void(*on_reset_cb)(const
rx_frame_t *frame, void *ctx), void *cb_ctx)
```

Register callbacks for PING and RESET control frames.

Stores function pointers that the receive path invokes when it decodes a PING or RESET frame. Passing NULL for a callback disables notification for that frame type while still performing the automatic PONG / RESET_ACK response.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle (must be initialized)
in	on_ping_cb	Callback invoked on PING receipt (NULL to disable)
in	on_reset_cb	Callback invoked on RESET receipt (NULL to disable)
in	cb_ctx	Opaque context forwarded to control callbacks

Value	Description
k_rx_ok	Callbacks registered
k_rx_err_invalid_arg	handle is nullptr

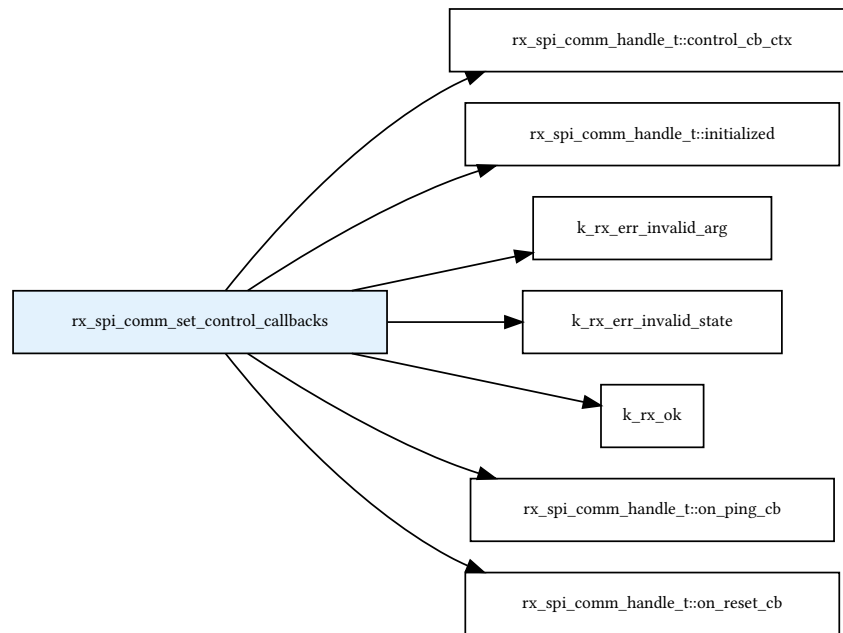
Pre: handle != nullptr

Pre: handle->initialized == true

Post: Callbacks stored; invoked on next matching control frame

Post: Previous callbacks overwritten

Note: Not thread-safe; register before starting the receive loop.] **See also:**
rx_spi_comm_send_pong() Auto-response sent on PING, rx_spi_comm_send_reset_ack()
Auto-response sent on RESET

Figure 1119: Call graph for `rx_spi_comm_set_control_callbacks`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2213`

Since Version 1.0.0

—

rx_spi_comm_send_pong

```
rx_err_t rx_spi_comm_send_pong(rx_spi_comm_handle_t *handle, const uint8_t
*payload, uint32_t payload_len)
```

Send PONG frame with echoed payload.

Send PONG frame with payload (response to PING).

Constructs a PONG frame echoing the given payload, acquires the next TX sequence number from the shared session, encodes the frame, and transfers it over SPI. Typically called automatically by the receive path in response to a PING.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle
in	payload	Payload to echo (NULL when payload_len is 0)
in	payload_len	Length in bytes [0, k_frame_max_payload]

Value	Description
-------	-------------

k_rx_ok	PONG transmitted successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	Handle not initialized
k_rx_err_invalid_size	payload_len exceeds k_frame_max_payload
(other)	Error from session, encode, or SPI transfer

Pre: handle != nullptr && handle->initialized

Pre: payload != NULL when payload_len > 0

Post: TX sequence incremented on success

Post: PONG frame sent over SPI

Note: Not thread-safe; called from the SPI receive context.] **See also:** rx_frame_create_pong()
Frame construction helper

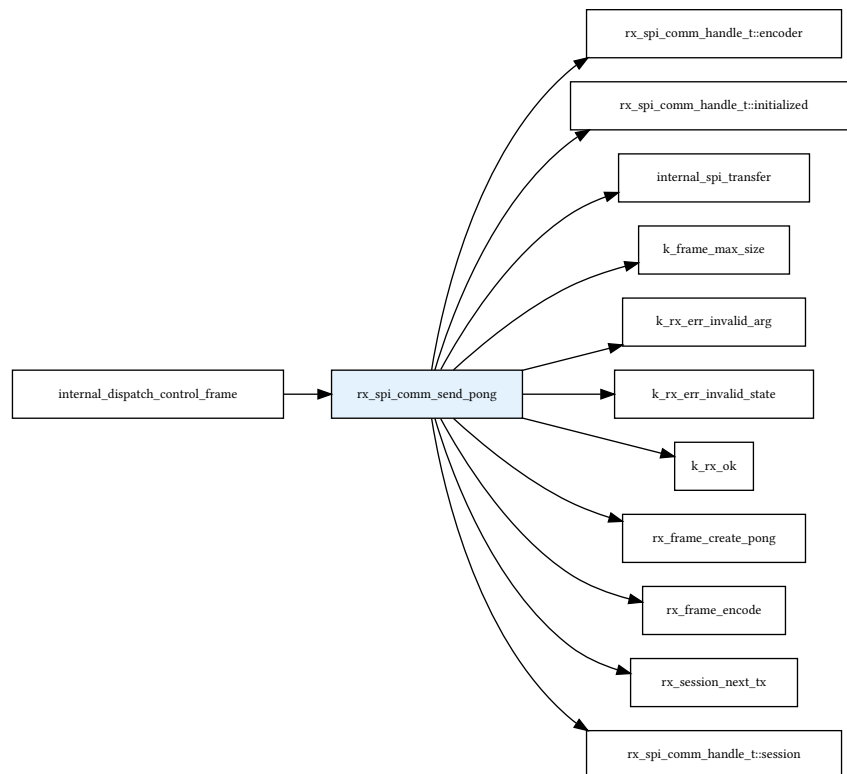


Figure 1120: Call graph for rx_spi_comm_send_pong

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2262`

Since Version 1.0.0

—

rx_spi_comm_send_reset_ack

```
rx_err_t rx_spi_comm_send_reset_ack(rx_spi_comm_handle_t *handle)
```

Send RESET_ACK frame confirming a reset request.

Send RESET_ACK frame (response to RESET).

Constructs a RESET_ACK frame (empty payload), acquires the next TX sequence number, encodes, and transfers over SPI. Called automatically by the receive path after processing a RESET frame. The RESET_ACK itself does NOT reset sequence numbers; that is handled by the session layer in response to the RESET.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle

Value	Description
k_rx_ok	RESET_ACK transmitted successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	Handle not initialized
(other)	Error from session, encode, or SPI transfer

Pre: handle != nullptr && handle->initialized

Pre: A RESET frame was received and session state cleared

Post: TX sequence incremented on success

Post: RESET_ACK frame sent over SPI

Note: Not thread-safe; called from the SPI receive context.] **See also:**

`rx_frame_create_reset_ack()` Frame construction helper,
`rx_spi_comm_set_control_callbacks()` Register RESET notification

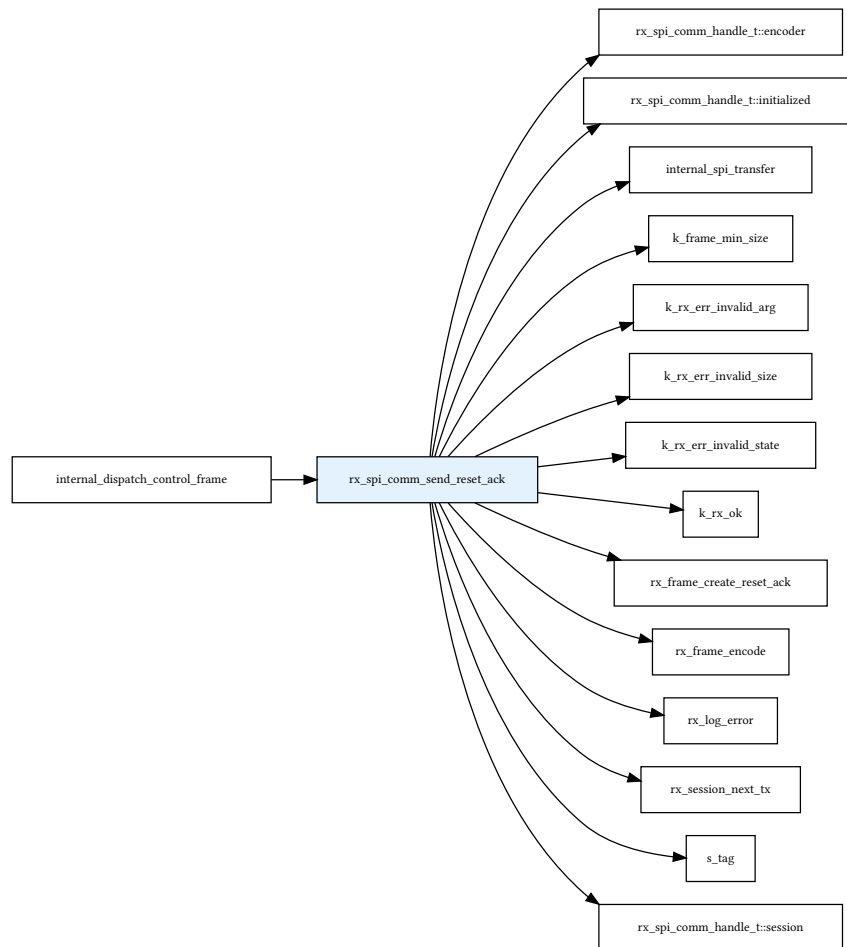


Figure 1121: Call graph for rx_spi_comm_send_reset_ack

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2326`

Since Version 1.0.0

—

rx_spi_comm_process_retransmits

```
rx_err_t rx_spi_comm_process_retransmits(rx_spi_comm_handle_t *handle, const
uint32_t current_time_ms)
```

Process pending retransmissions based on timeout.

Check and process automatic retransmissions.

Parameters:

Direction	Name	Description
-----------	------	-------------

inout	handle	SPI communication handle
in	current_time_ms	Current system time in milliseconds

Returns: rx_err_t Error code

Value	Description
k_rx_ok	No action needed or retransmit sent
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_err_retry_limit	Max retries exceeded

Pre: handle must be non-NULL and initialized

Post: retry_count incremented on retransmit, pending cleared on limit

Note: Safe to call when auto_retransmit is disabled (no-op)

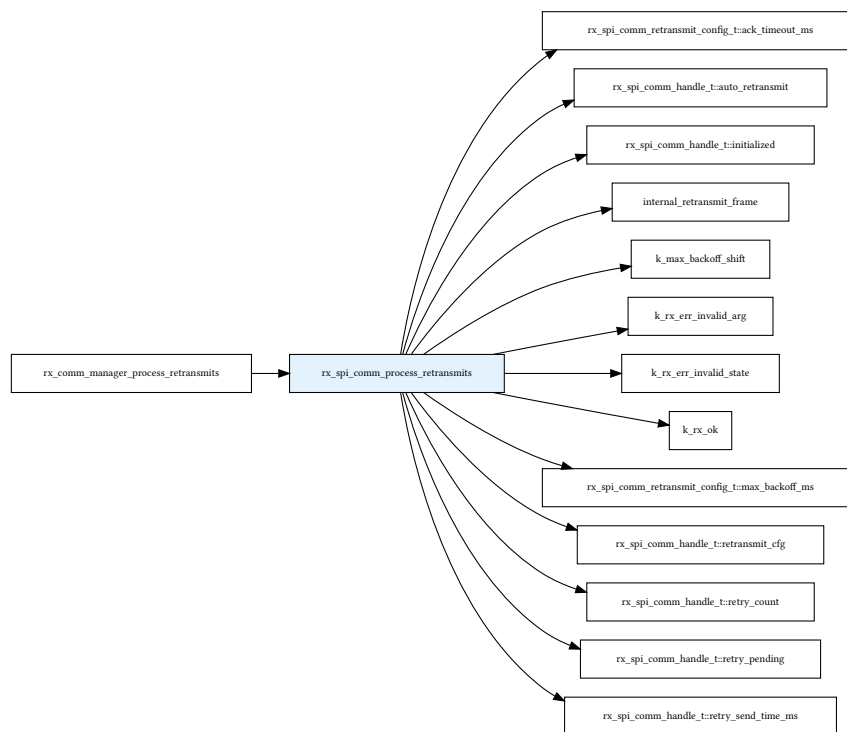


Figure 1122: Call graph for rx_spi_comm_process_retransmits

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2446`

Since Version 1.1.0

—

rx_spi_comm_set_auto_retransmit

```
rx_err_t rx_spi_comm_set_auto_retransmit(rx_spi_comm_handle_t *handle, const bool
enabled, const rx_spi_comm_retransmit_config_t *config)
```

Enable or disable automatic retransmission at runtime.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle
in	enabled	true to enable, false to disable
in	config	Retransmit config (NULL to keep current/defaults)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Configuration applied successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be non-NULL and initialized

Post: auto_retransmit flag and config updated

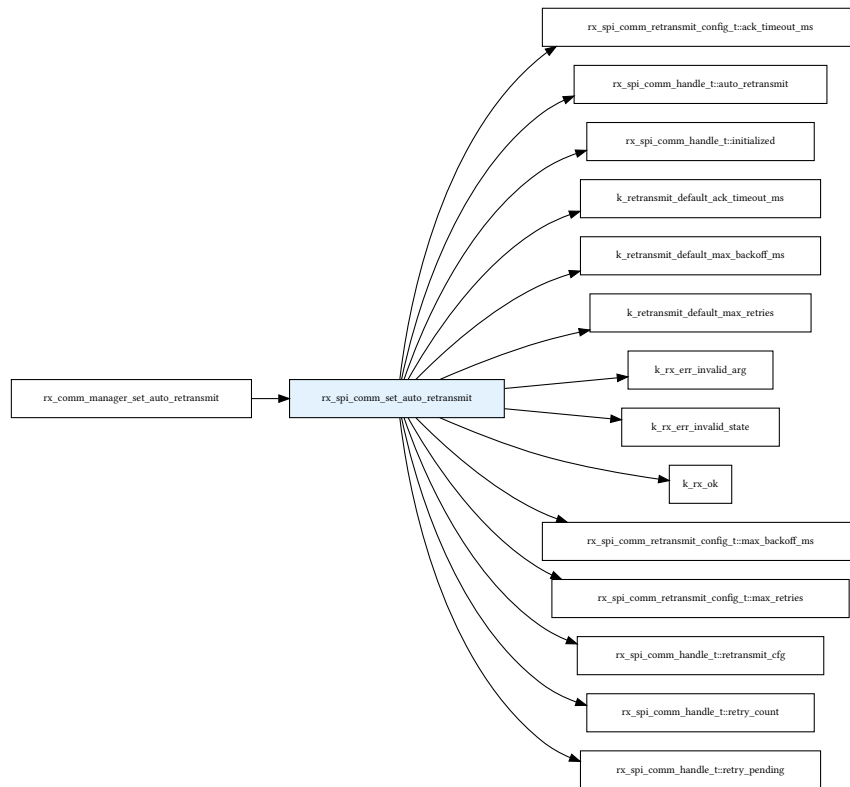


Figure 1123: Call graph for rx_spi_comm_set_auto_retransmit

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2503`

Since Version 1.1.0

—

rx_spi_comm_set_retransmit_callbacks

```

rx_err_t rx_spi_comm_set_retransmit_callbacks(rx_spi_comm_handle_t *handle,
void(*on_ack_cb)(uint16_t sequence, void *ctx), void(*on_nack_cb)(uint16_t
sequence, void *ctx), void *ctx)

```

Register ACK/NACK notification callbacks.

Register ACK/NACK notification callbacks for retransmission.

Parameters:

Direction	Name	Description
inout	handle	SPI communication handle
in	on_ack_cb	ACK callback (NULL to disable)
in	on_nack_cb	NACK callback (NULL to disable)

in	ctx	User context passed to callbacks
----	-----	----------------------------------

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Callbacks registered
k_rx_err_invalid_arg	handle is nullptr

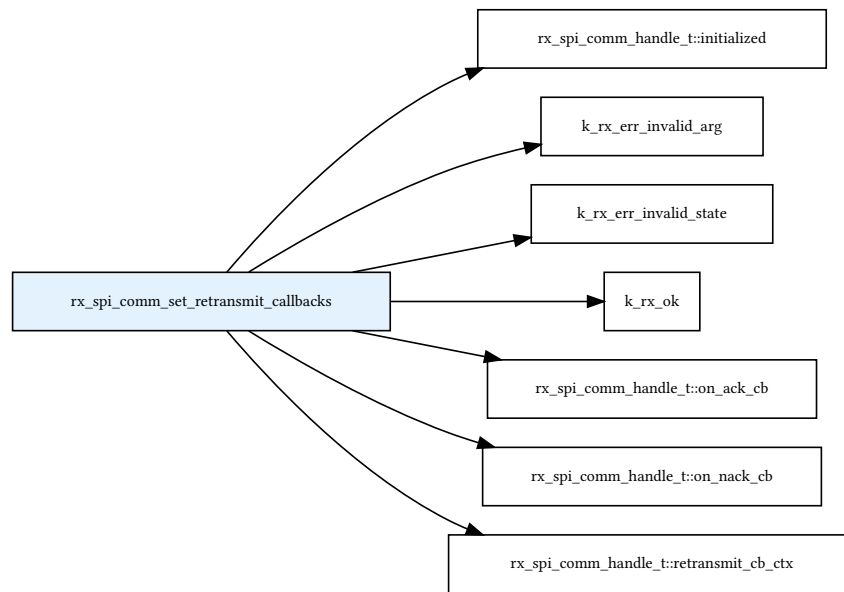


Figure 1124: Call graph for `rx_spi_comm_set_retransmit_callbacks`

Defined in `libs/rx_spi_comm/src/rx_spi_comm.c:2556`

Since Version 1.1.0

—

rx_spi_link.h

SPI Link Layer with HARQ.

Implements a reliability layer between the communication manager and the raw SPI transport. This module mirrors the Go gateway's `internal/link/spi.go`, providing:

- FEC encoding on TX (convolutional K=7, rate 1/2)
- FEC decoding on RX (soft-decision Viterbi)
- Chase Combining across retransmissions (soft bit accumulation)
- ACK/NACK handshake with sequence tracking
- Configurable retries (default 3)

Includes:

- `<stdbool.h>`
- `<stdint.h>`

- "rx_err.h"
- "rx_frame.h"
- "rx_harq.h"
- "rx_spi_comm.h"

rx_spi_link_constants_t

SPI link layer compile-time constants.

Protocol timing and retry parameters matching the Go gateway's internal/link/spi.go constants.

max_retries must be in range [0, 255], where 0 means use default value of 3

fec_enabled is boolean (0 or 1)

Value	Name	Description
= 3	k_spi_link_default_max_retries	Default maximum transmission attempts (matches Go maxRetries=3). After 3 failed attempts (including Chase Combining), the link returns an error and the comm_manager may fail over to USB.
= 1	k_spi_link_default_fec_enabled	Default FEC enabled state (enabled by default, aligns with Go HARQ). FEC/HARQ is enabled by default for reliability. Can be disabled at runtime via config or test modes for raw SPI operation.

See also: star-gateway/internal/link/spi.go Go reference constants (maxRetries=3), rx_spi_link_config_t Configuration structure using these defaults

Example:

```
//Example:Usingdefaultconstants
rx_spi_link_config_tconfig={
    .spi_handle=&spi_handle,
    .fec_enabled=k_spi_link_default_fec_enabled,//1(enabled)
    .max_retries=k_spi_link_default_max_retries,//3attempts
};
```

Since Version 1.1.0

—

rx_spi_link_timing_t

SPI link layer timing constants.

Timeout values for ACK waiting and buffer sizing. Matches Go gateway's ackTimeout = 10ms and FEC encoding buffer requirements.

ack_timeout_ms must be > 0 and < 65535 milliseconds

max_encoded_payload = 2 * k_harq_max_payload + k_fec_tail_bits (2050 bytes for 1024-byte max payload)

Value	Name	Description
= 10	k_spi_link_ack_timeout_ms	ACK/NACK wait timeout in milliseconds (matches Go ackTimeout=10ms). How long to wait for ACK/NACK

	after sending a frame. If no response arrives within this window, the frame is retransmitted.
--	---

See also: star-gateway/internal/link/spi.go Go ackTimeout constant (10ms), rx_fec.h FEC encoding parameters (rate 1/2, tail bits)

Example:

```
//Example: Using timing constants for ACK wait
rx_err_t err = rx_spi_comm_receive(spi_handle, &ack_frame, k_spi_link_ack_timeout_ms);
if(err == k_rx_err_timeout){
//No ACK received within 10ms, retry transmission
}
```

Since Version 1.1.0

—

rx_spi_link_buffer_t

Buffer size constants for SPI link layer.

Defines buffer sizes for FEC-encoded payloads. Sized to accommodate the maximum payload after FEC encoding (rate 1/2) plus tail bits.

Formula Derivation:

- FEC rate 1/2: Each input byte produces 2 output bytes
- k_harq_max_payload = 1024 bytes
- FEC encoded size = $2 * k_harq_max_payload = 2 * 1024 = 2048$ bytes
- k_fec_tail_bits = 6 bits (trellis termination, K=7 constraint length)
- Tail bytes = $\text{ceiling}(k_fec_tail_bits / 8) = \text{ceiling}(6 / 8) = 1$ byte
- Total = $2048 + 1 = 2049$ bytes

Value	Name	Description
= 2049	k_spi_link_max_encoded_payload	Maximum FEC-encoded payload size in bytes. Formula: bytes = $(2 * k_harq_max_payload) + \text{ceiling}(k_fec_tail_bits / 8) = (2 * 1024) + \text{ceiling}(6 / 8) = 2048 + 1 = 2049$. FEC rate 1/2 doubles the payload size, and tail bits (6 bits = 1 byte after ceiling) are appended for trellis termination.

See also: rx_harq.h HARQ protocol maximum payload size, rx_fec.h FEC encoding parameters (rate 1/2, tail bits)

Since Version 1.1.0

—

rx_spi_link_state_t

SPI link layer state machine states.

Mirrors the Go gateway's harq.State enum. Tracks whether the link is idle, waiting for acknowledgment, retransmitting, or in error.

Value	Name	Description
-------	------	-------------

= 0	k_spi_link_state_idle	Ready for new send/receive
= 1	k_spi_link_state_waiting_ack	Frame sent, waiting for ACK/NACK
= 2	k_spi_link_state_retransmitting	Retransmitting after NACK/timeout
= 3	k_spi_link_state_error	Unrecoverable error (max retries)

See also: rx_spi_link_get_state() Query current state, rx_spi_link_reset() Reset to Idle state

Since Version 1.1.0

—

rx_spi_link_init

```
rx_err_t rx_spi_link_init(rx_spi_link_t *link, const rx_spi_link_config_t
*config)
```

Initialize SPI link layer.

Initializes the HARQ handle (including FEC encoder/decoder and Chase Combiner) and stores configuration. The underlying SPI transport must already be initialized.

Parameters:

Direction	Name	Description
out	link	Link handle to initialize
in	config	Configuration (spi_handle is REQUIRED)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	link or config is nullptr
k_rx_err_invalid_arg	config->spi_handle is nullptr

Pre: link must point to valid, allocated memory (86 KB)

Pre: config->spi_handle must be initialized via rx_spi_comm_init()

Post: link->initialized == true

Post: link->harq initialized with FEC if fec_enabled

Note: Thread Safety: Not thread-safe. Call from single thread during init.] **See also:** rx_spi_link_deinit() Cleanup counterpart, rx_spi_link.h for full documentation

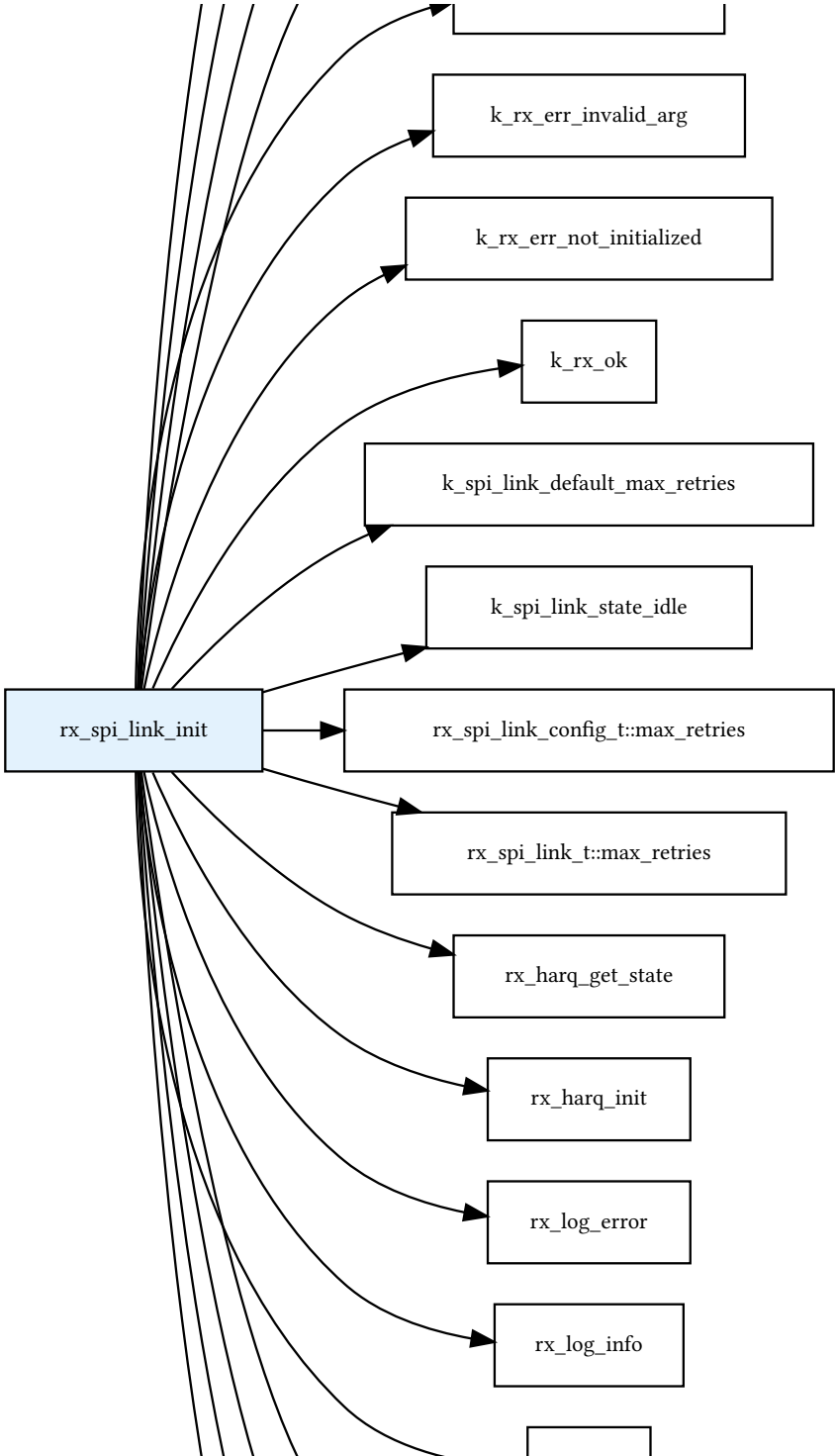


Figure 1125: Call graph for rx_spi_link_init

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:425`

Since Version 1.1.0

—

rx_spi_link_deinit

```
rx_err_t rx_spi_link_deinit(rx_spi_link_t *link)
```

Deinitialize SPI link layer.

Deinitializes the HARQ handle and marks the link as uninitialized. Does NOT deinitialize the underlying SPI transport.

Parameters:

Direction	Name	Description
inout	link	Link handle to deinitialize

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	link is nullptr
k_rx_err_invalid_state	link not initialized

Pre: link must be non-NULL

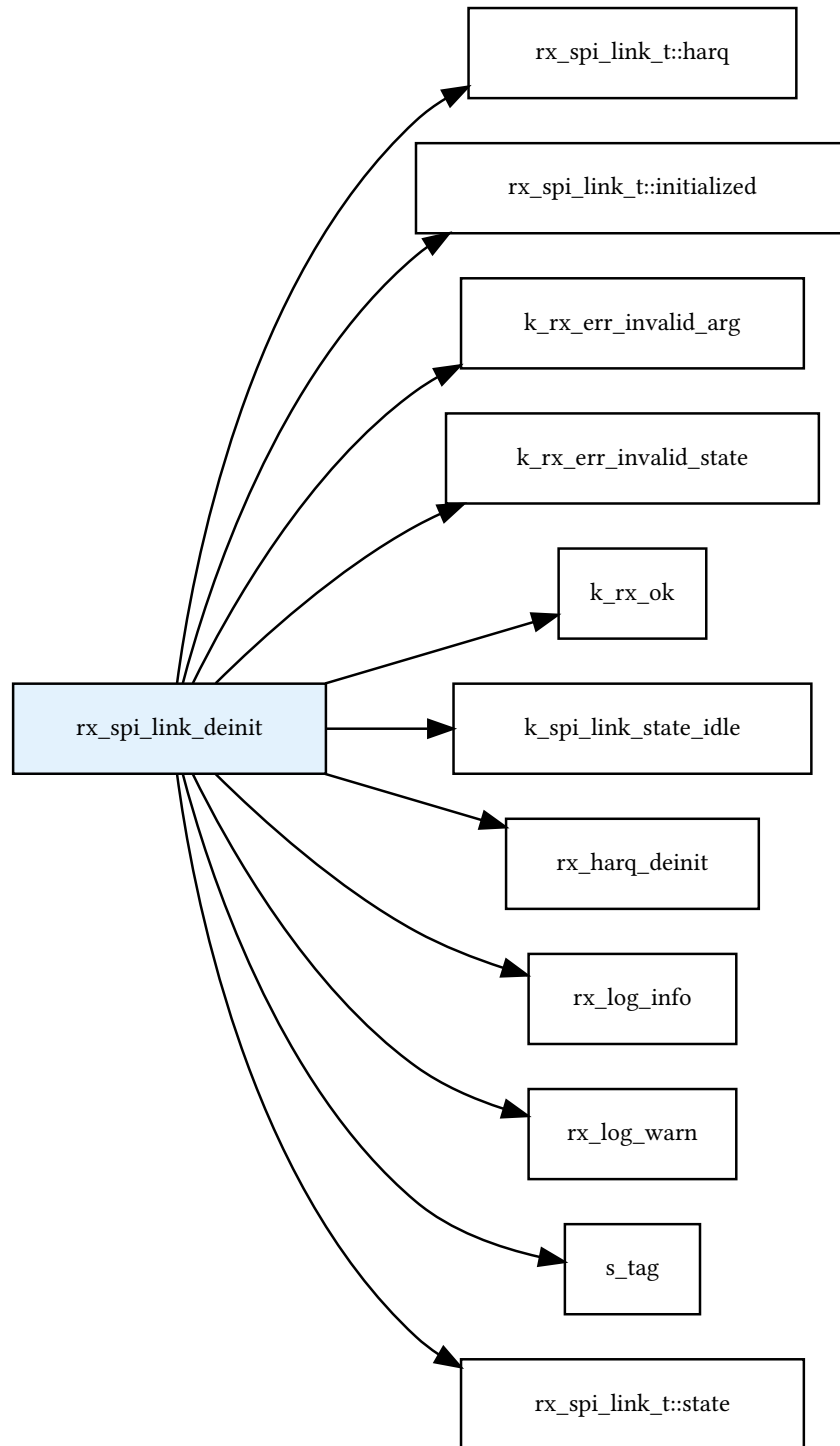
Pre: link must be initialized (link->initialized == true)

Post: link->initialized == false

Post: link->harq internal state deinitialized (HARQ handle invalid)

Note: Does NOT deinitialize the underlying SPI transport handle] **See also:**

`rx_spi_link_init()` Initialization counterpart, `rx_harq_deinit()` Internal HARQ deinitialization, `rx_spi_link.h` for full documentation

Figure 1126: Call graph for `rx_spi_link_deinit`

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:452`

Since Version 1.1.0

—

rx_spi_link_send

```
rx_err_t rx_spi_link_send(rx_spi_link_t *link, rx_frame_type_t type, const
uint8_t *payload, uint32_t payload_len)
```

Send payload with HARQ reliability over SPI.

Encodes payload with FEC (if enabled), sends via SPI transport, and waits for ACK/NACK. On NACK or timeout, retransmits up to `max_retries`.

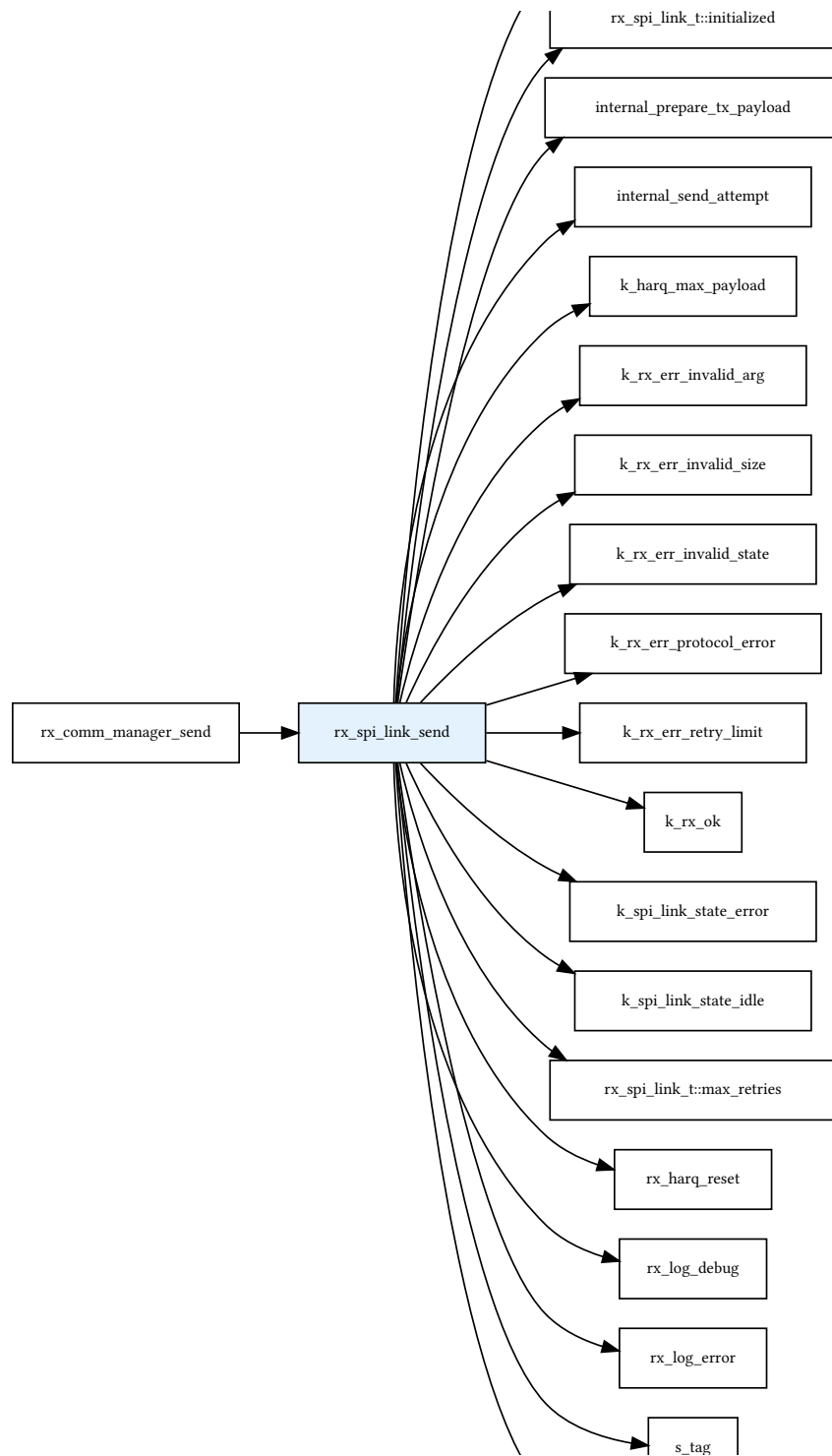


Figure 1127: Call graph for rx_spi_link_send

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:501`

—

rx_spi_link_receive

```
rx_err_t rx_spi_link_receive(rx_spi_link_t *link, rx_spi_link_receive_result_t
*result, uint32_t timeout_ms)
```

Receive and decode a frame from SPI with HARQ.

Receives a frame from the SPI transport, handles ACK/NACK dispatch, and performs FEC decoding with Chase Combining if applicable.

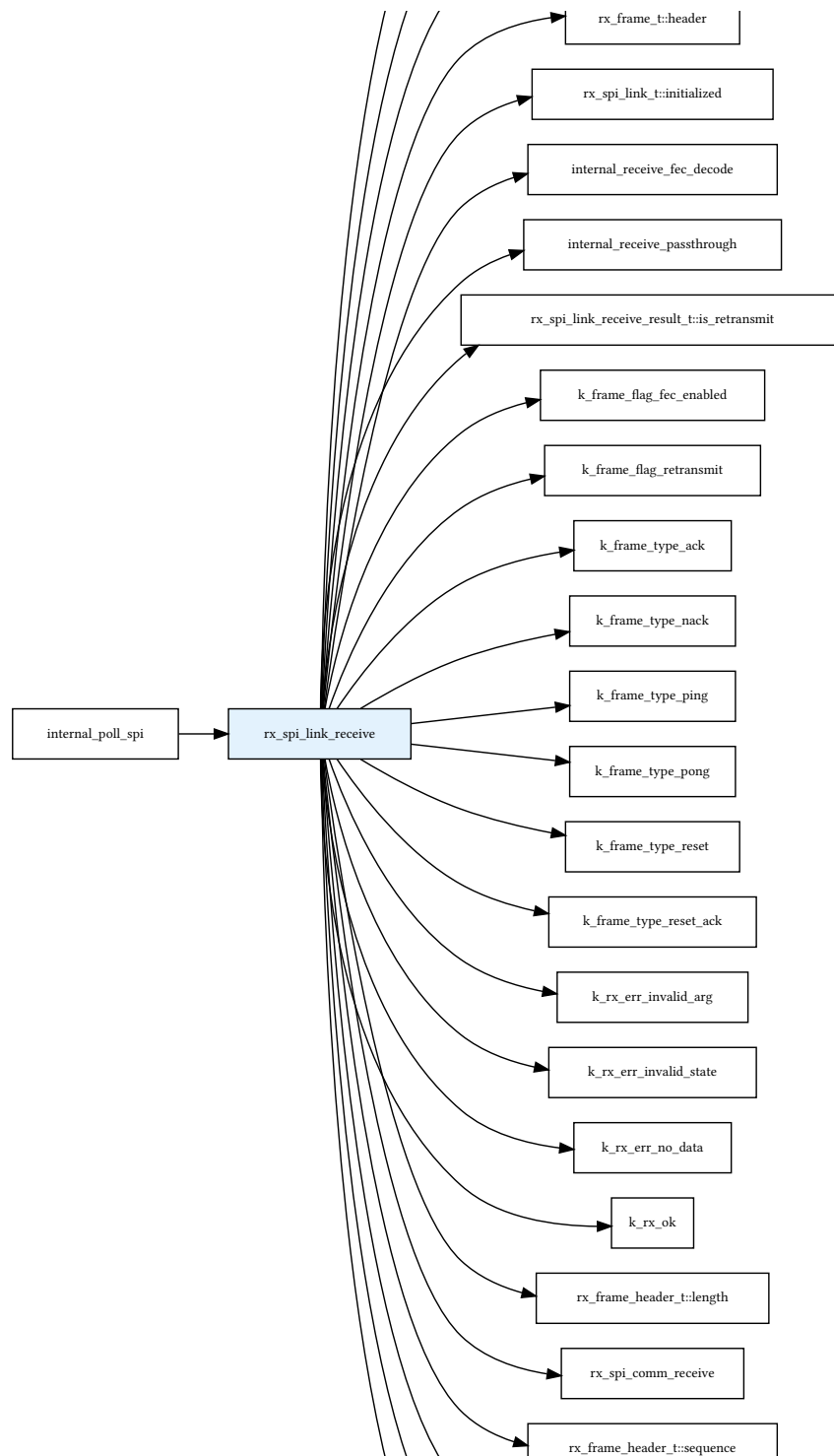


Figure 1128: Call graph for rx_spi_link_receive

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:553`

—

rx_spi_link_reset

```
rx_err_t rx_spi_link_reset(rx_spi_link_t *link)
```

Reset SPI link layer for new session.

Resets the HARQ state machine, clears the Chase Combiner, and returns the link to idle state. Does NOT reset the shared session state (sequence numbers are managed by rx_session).

Parameters:

Direction	Name	Description
inout	link	Initialized link handle

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	link is nullptr
k_rx_err_invalid_state	link not initialized

Pre: link must be non-NULL

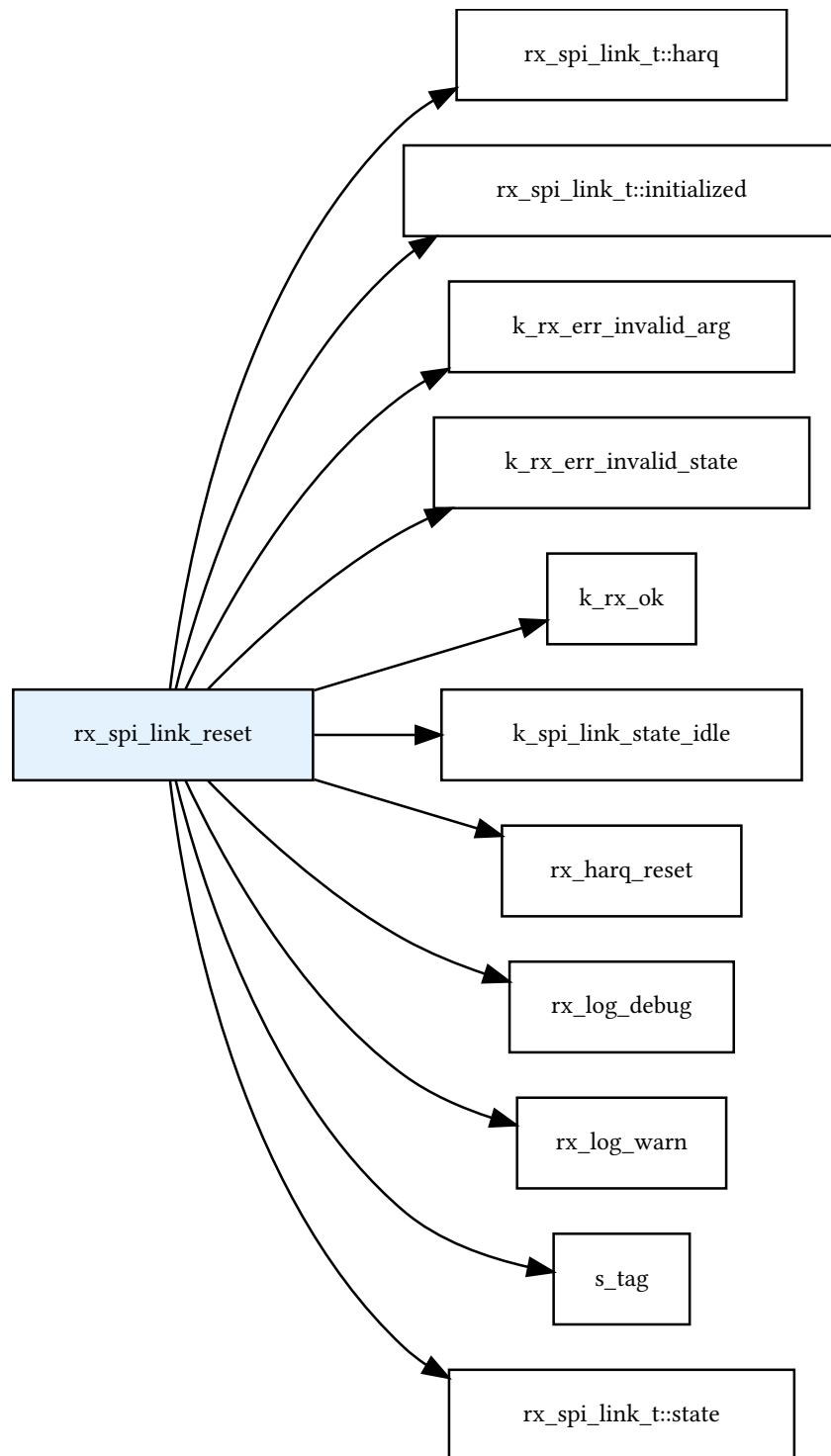
Pre: link must be initialized (link->initialized == true)

Post: link->state == k_spi_link_state_idle

Post: Chase Combiner buffers cleared (all soft-decision history discarded)

Note: Does NOT reset TX/RX sequence numbers (managed externally by rx_session)

Note: Safe to call after error state to recover link operation] **See also:** rx_harq_reset()
Internal HARQ reset, rx_spi_link_get_state() Query current state, rx_spi_link.h for full documentation

Figure 1129: Call graph for `rx_spi_link_reset`

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:586`

Since Version 1.1.0

—

rx_spi_link_get_state

```
rx_spi_link_state_t rx_spi_link_get_state(const rx_spi_link_t *link)
```

Get current SPI link state.

Returns the current state of the SPI link state machine. Safe to call with nullptr (returns error state). Does not modify link state.

Parameters:

Direction	Name	Description
in	link	Link handle (may be NULL)

Returns: rx_spi_link_state_t Current link state

Value	Description
k_spi_link_state_idle	Link is ready for new send/receive
k_spi_link_state_waiting_ack	Waiting for ACK/NACK response
k_spi_link_state_retransmitting	Retransmitting after NACK/timeout
k_spi_link_state_error	Unrecoverable error (also returned if link is NULL)

Pre: link may be NULL (function handles gracefully)

Pre: If link is non-NULL, may be in any state (initialized or not)

Post: No state change (pure query function)

Post: Return value reflects state at time of call (may change immediately after)

Note: Thread-safe: Read-only operation, no synchronization needed

Note: NULL-safe: Returns k_spi_link_state_error when link is nullptr

Note: Does not distinguish between “link is NULL” vs “link is in error state”] **See also:** rx_spi_link_t Link handle structure, rx_spi_link_state_t State enumeration with all possible values, rx_spi_link_reset() Reset link to idle state, rx_spi_link.h for full documentation

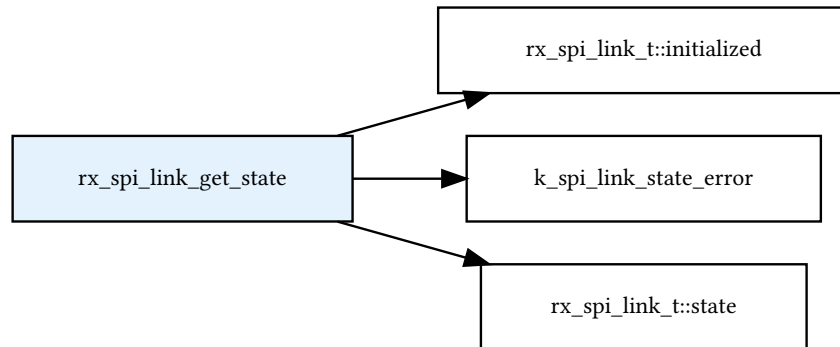


Figure 1130: Call graph for rx_spi_link_get_state

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:618`

Since Version 1.1.0

—

rx_spi_link_fec_enabled

```
bool rx_spi_link_fec_enabled(const rx_spi_link_t *link)
```

Check if FEC is enabled on this link.

Queries the FEC/HARQ enabled flag. Returns false if link is NULL. Does not modify link state. FEC setting is configured at init time via `rx_spi_link_config_t.fec_enabled` and cannot be changed at runtime.

Parameters:

Direction	Name	Description
in	link	Link handle (may be NULL)

Returns: bool FEC enabled status

Value	Description
true	FEC encoding/decoding is active (Chase Combining, Viterbi)
false	FEC disabled (raw payload transmission) or link is nullptr

Pre: link may be NULL (function handles gracefully by returning false)

Pre: If link is non-NULL, may be in any state (initialized or not)

Post: No state change (pure query function)

Post: Return value reflects FEC setting at time of call (constant for link lifetime)

Note: Thread-safe: Read-only operation, no synchronization needed

Note: NULL-safe: Returns false when link is nullptr

Note: FEC setting is immutable after initialization (cannot be changed at runtime)

Note: When FEC is enabled, rx_spi_link_send/receive use HARQ with Chase Combining] **See also:** rx_spi_link_config_t Configuration structure with fec_enabled field, rx_spi_link_init() Initialization function that sets FEC mode, k_spi_link_default_fec_enabled Default FEC setting (enabled = 1), rx_spi_link.h for full documentation

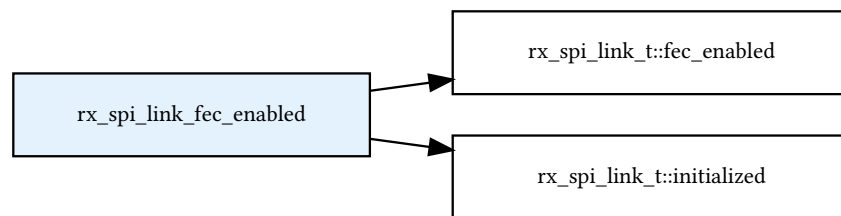


Figure 1131: Call graph for rx_spi_link_fec_enabled

Defined in `libs/rx_spi_link/inc/rx_spi_link.h:650`

Since Version 1.1.0

—

rx_spi_link.c

SPI Link Layer with HARQ Implementation.

Implements the SPI link layer that sits between the communication manager and the raw SPI transport (rx_spi_comm). This is a C port of the Go gateway's `internal/link/spi.go`, providing:

- FEC encoding on TX (convolutional K=7, rate 1/2)
- Soft-decision Viterbi decoding on RX
- Chase Combining across retransmissions
- ACK/NACK handshake with configurable retries

Includes:

- `"rx_spi_link.h"`
- `<string.h>`
- `"rx_fec.h"`
- `"rx_log.h"`

internal_soft_bit_values_t

Soft bit conversion constants.

Used when converting received hard bytes to soft bits for the Viterbi decoder. Each bit becomes +127 (confident 1) or -127 (confident 0). Matches Go's `bytesToSoftBits()` in `internal/link/spi.go`.

Value	Name	Description
= 127	k_internal_soft_bit_one	Hard bit 1 -> soft confidence +127
= -127	k_internal_soft_bit_zero	Hard bit 0 -> soft confidence -127

Since Version 1.1.0

—

internal_bit_constants_t

Bit manipulation constants for byte-to-soft-bit conversion.

Value	Name	Description
= 8	k_spi_link_bits_per_byte	Number of bits in a byte
= 7	k_spi_link_msb_shift	Shift to extract MSB
= 0x01	k_spi_link_bit_mask	Mask to extract single bit

Since Version 1.1.0

—

s_tag

```
const char s_tag[]
```

Log tag for SPI link layer messages.

—

s_zero_link

```
const rx_spi_link_t s_zero_link
```

Zero-initialized SPI link template for stack-safe initialization.

Note: Read-only; never modified after static initialization

Warning: Do not remove - prevents stack overflow in init function] **See also:**
rx_spi_link_init() Uses this template to zero-initialize the link handle

Since Version 1.1.0

—

internal_bytes_to_soft_bits

```
static rx_err_t internal_bytes_to_soft_bits(const uint8_t *data, uint32_t
data_len, rx_soft_bit_t *soft, uint32_t soft_size, uint32_t *soft_len)
```

Convert payload bytes to soft bits for Viterbi decoder.

Each byte is expanded to 8 soft bits, MSB-first. Bit value 1 becomes +127, bit value 0 becomes -127. This matches the Go gateway's bytesToSoftBits() function.

Parameters:

Direction	Name	Description
in	data	Input hard bytes
in	data_len	Number of input bytes
out	soft	Output soft bits (must hold data_len * 8 elements)
in	soft_size	Size of soft buffer in elements
out	soft_len	Actual number of soft bits written

Returns: k_rx_ok on success, k_rx_err_invalid_size if buffer too small

Pre: data and soft must be non-NULL

Pre: soft_size >= data_len * k_spi_link_bits_per_byte

Post: soft_len == data_len * k_spi_link_bits_per_byte

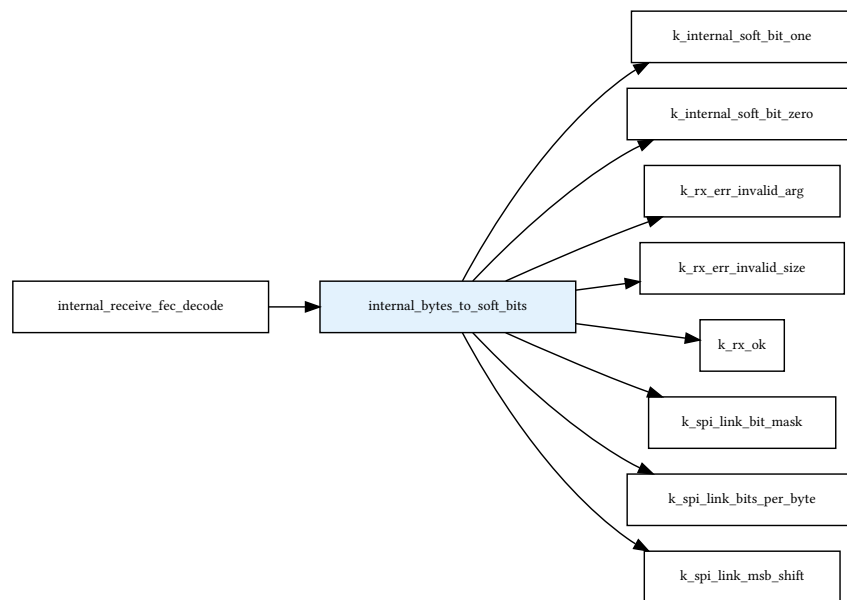


Figure 1132: Call graph for internal_bytes_to_soft_bits

Defined in `libs/rx_spi_link/src/rx_spi_link.c:157`

Since Version 1.1.0

internal_wait_for_ack

```
static rx_err_t internal_wait_for_ack(rx_spi_link_t *link, uint16_t expected_seq)
```

Wait for ACK/NACK response from gateway.

Polls `rx_spi_comm_receive()` looking for an ACK or NACK frame matching the expected sequence number. Non-matching frames are logged and ignored.

Parameters:

Direction	Name	Description
inout	<code>link</code>	Link handle
in	<code>expected_seq</code>	Expected ACK/NACK sequence

Returns: `k_rx_err_timeout` if no ACK/NACK within timeout

Pre: link must be initialized

Pre: `expected_seq` matches the last sent frame's sequence

Post: No side effects on timeout

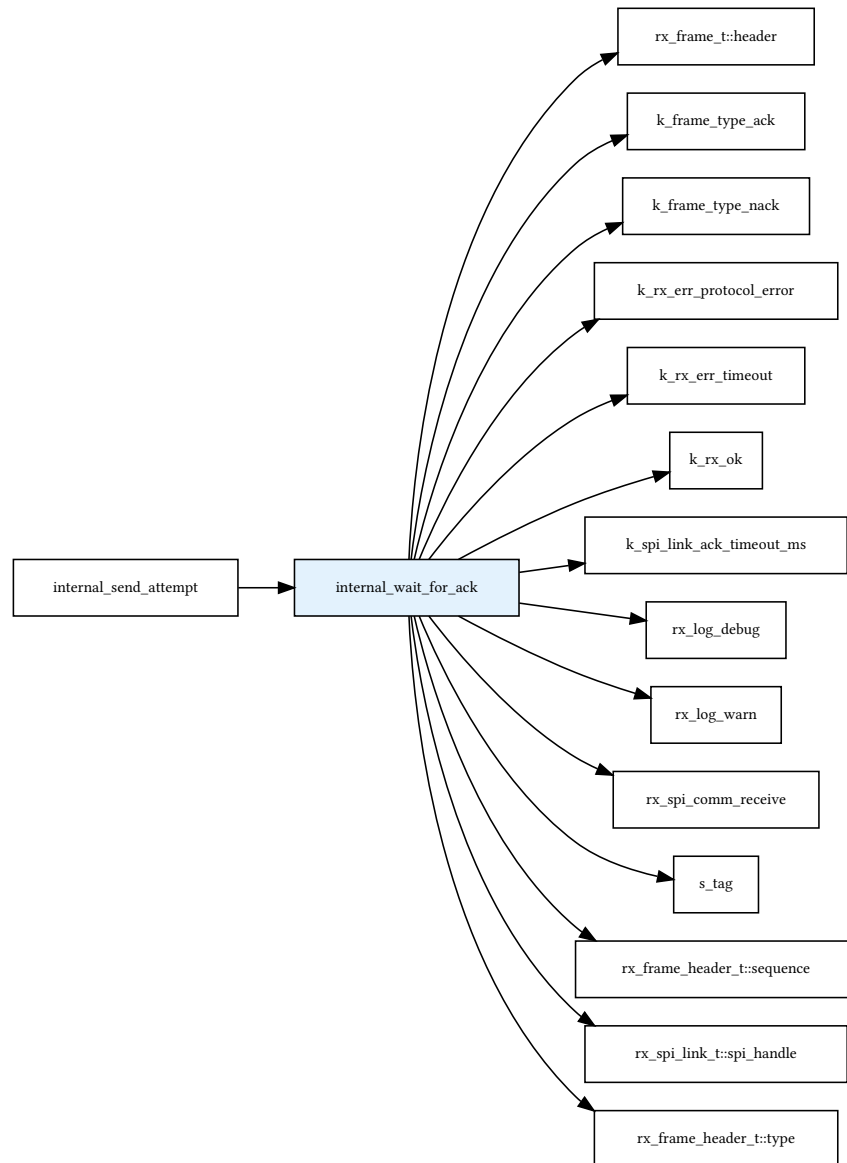


Figure 1133: Call graph for internal_wait_for_ack

Defined in `libs/rx_spi_link/src/rx_spi_link.c:229`

Since Version 1.1.0

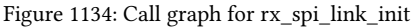
—

rx_spi_link_init

```
rx_err_t rx_spi_link_init(rx_spi_link_t *link, const rx_spi_link_config_t
*config)
```

Initialize SPI link layer.

See also: rx_spi_link.h for full documentation



Defined in `libs/rx_spi_link/src/rx_spi_link.c:284`

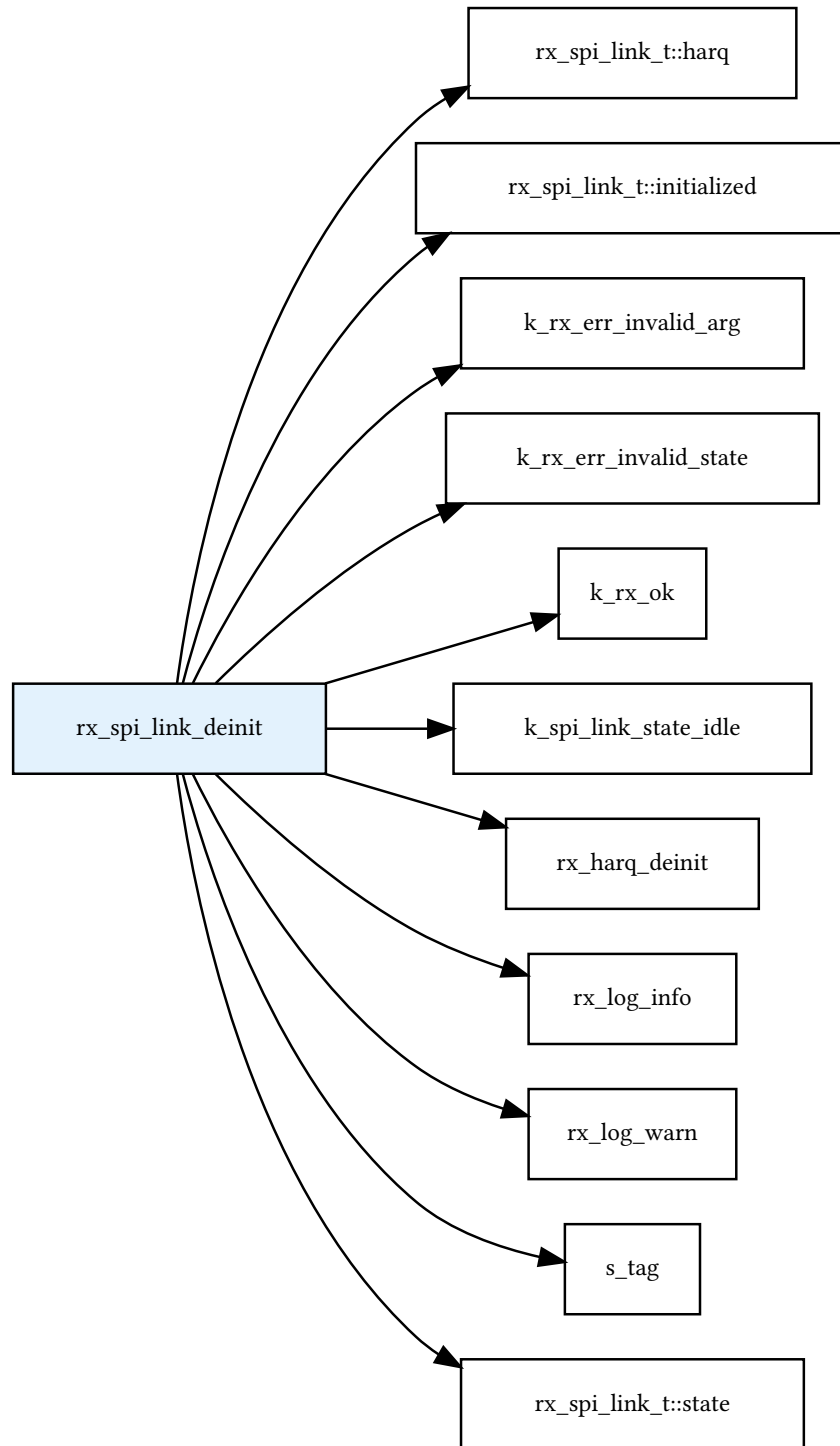
—

rx_spi_link_deinit

```
rx_err_t rx_spi_link_deinit(rx_spi_link_t *link)
```

Deinitialize SPI link layer.

See also: `rx_spi_link.h` for full documentation

Figure 1135: Call graph for `rx_spi_link_deinit`

Defined in `libs/rx_spi_link/src/rx_spi_link.c:337`

—

internal_send_attempt

```
static rx_err_t internal_send_attempt(rx_spi_link_t *link, rx_frame_type_t type,
uint8_t base_flags, const uint8_t *tx_payload, uint32_t tx_len, uint8_t attempt)
```

Attempt a single transmission with ACK/NACK handling.

Sends frame via SPI transport, waits for ACK/NACK, and handles retransmission logic. Updates link state for retransmit/waiting_ack. Gets TX sequence BEFORE sending to avoid race conditions with external sequence management.

Parameters:

Direction	Name	Description
inout	link	Link handle (must be initialized)
in	type	Frame type (command, telemetry, etc.)
in	base_flags	Base flags (fec_enabled, requires_ack)
in	tx_payload	Payload to transmit (may be FEC-encoded)
in	tx_len	Payload length in bytes
in	attempt	Attempt number (0 = first, >0 = retransmit)

Returns: rx_err_t Result of attempt

Value	Description
k_rx_ok	ACK received (success)
k_rx_err_protocol_error	NACK received (retry needed)
k_rx_err_timeout	Timeout (retry needed)
Other	SPI send errors

Note: Parameter ordering convention: type, base_flags, tx_payload, tx_len, attempt. This ordering groups frame metadata (type, flags) before payload data (payload, len), then retry state (attempt). While adjacent integer parameters may appear swappable, the semantic grouping is intentional for code clarity.

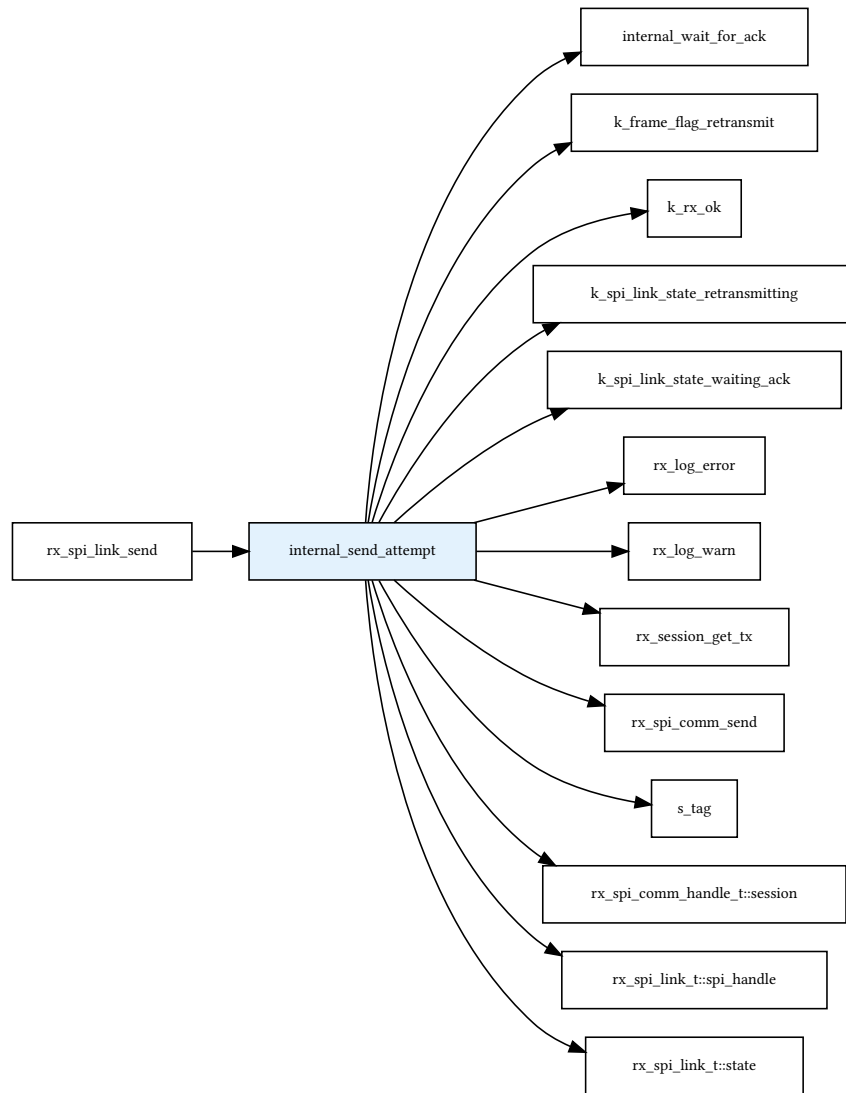


Figure 1136: Call graph for internal_send_attempt

Defined in `libs/rx_spi_link/src/rx_spi_link.c:393`

internal_prepare_tx_payload

```
static rx_err_t internal_prepare_tx_payload(rx_spi_link_t *link, const uint8_t
*payload, uint32_t payload_len, const uint8_t **out_payload, uint32_t *out_len,
uint8_t *out_flags)
```

Prepare payload for transmission (FEC encode or passthrough).

If FEC is enabled and payload is non-empty, encodes via `rx_harq_encode()` and sets FEC flag. Otherwise, passes payload through unchanged. Sets `base_flags` to include `k_frame_flag_requires_ack` and optionally `k_frame_flag_fec_enabled`.

Parameters:

Direction	Name	Description
inout	link	Link handle (must be initialized, <code>fec_enabled</code> checked)
in	payload	Original payload to send
in	payload_len	Original payload length
out	out_payload	Pointer to prepared payload (original or encoded buffer)
out	out_len	Prepared payload length
out	out_flags	Base flags for transmission

Returns: `rx_err_t` Error code

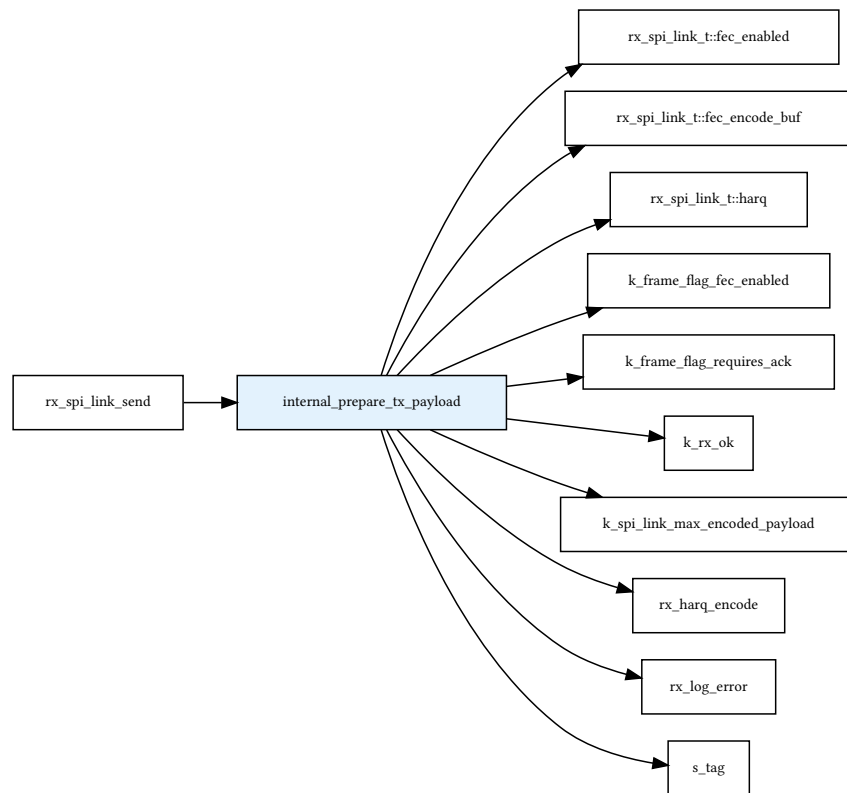
Value	Description
<code>k_rx_ok</code>	Preparation successful
<code>k_rx_err_*</code>	FEC encoding error

Pre: `link->fec_enabled` indicates whether FEC should be used

Pre: `link->fec_encode_buf` sized for `k_spi_link_max_encoded_payload`

Post: If FEC enabled: `out_payload` points to `link->fec_encode_buf`, `out_flags` has FEC flag

Post: If FEC disabled: `out_payload == payload`, `out_len == payload_len`

Figure 1137: Call graph for `internal_prepare_tx_payload`

Defined in `libs/rx_spi_link/src/rx_spi_link.c:462`

—

rx_spi_link_send

```
rx_err_t rx_spi_link_send(rx_spi_link_t *link, rx_frame_type_t type, const
uint8_t *payload, uint32_t payload_len)
```

Send payload with HARQ reliability over SPI.

See also: `rx_spi_link.h` for full documentation

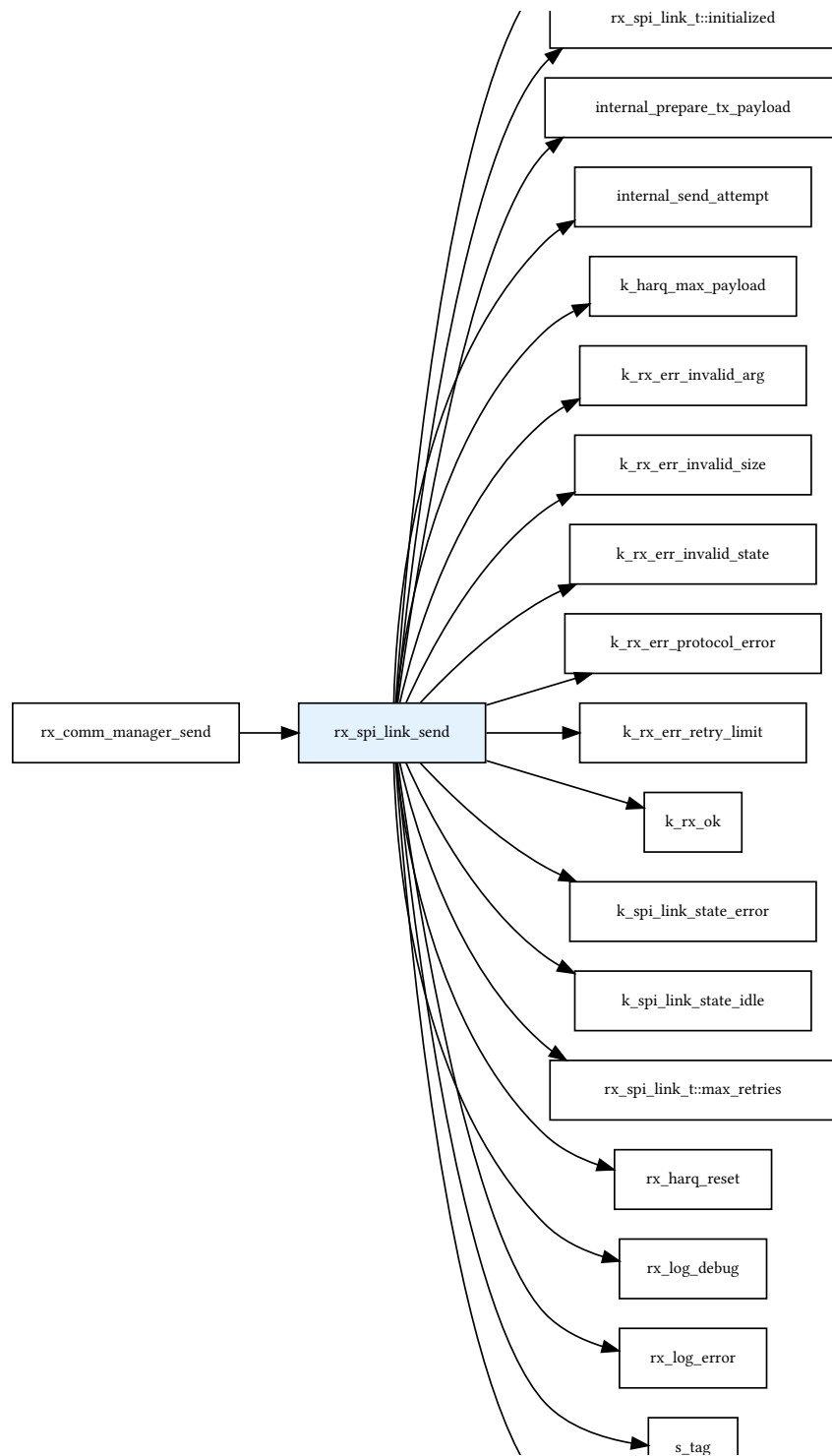


Figure 1138: Call graph for rx_spi_link_send

Defined in `libs/rx_spi_link/src/rx_spi_link.c:498`

—

internal_receive_fec_decode

```
static rx_err_t internal_receive_fec_decode(rx_spi_link_t *link, const rx_frame_t
*frame, rx_spi_link_receive_result_t *result)
```

Process received frame with FEC decoding and Chase Combining.

Converts hard bits to soft bits, performs Viterbi decoding via HARQ, and sends ACK/NACK based on decode success. On success, resets HARQ combiner. On failure, sends NACK for retransmission.

Algorithm:

1. Convert received bytes to soft bits (hard decision: 1 -> +127, 0 -> -127)
2. Pass soft bits to HARQ decoder (Chase Combining + Viterbi)
3. If decode succeeds: send ACK, reset combiner, return payload
4. If decode fails: send NACK, accumulate soft bits for next retransmit

Parameters:

Direction	Name	Description
inout	link	Link handle (must be initialized)
in	frame	Received frame with FEC-encoded payload
out	result	Decode result (payload, length, metadata)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Decode success, ACK sent
k_rx_err_protocol_error	Decode failed, NACK sent

Pre: link->initialized == true (link must be initialized)

Pre: frame->payload contains FEC-encoded data (FEC flag set)

Pre: result->payload buffer >= k_harq_max_payload (1024 bytes minimum)

Pre: Single-threaded context (SPI receive path protected by state machine)

Post: On success: result->fec_decoded == true, result->payload filled

Post: On success: ACK sent to sender, HARQ combiner reset

Post: On failure: NACK sent, soft bits accumulated in combiner

Post: result->combining_count reflects number of transmissions combined

Note: Thread safety: This function is NOT thread-safe. Must be called from single-threaded SPI receive context (enforced by link state machine).

State Machine Invariant:: This function is called only from rx_spi_link_receive(), which serializes access via the link state machine. Multiple concurrent calls would corrupt the Chase Combiner state and static soft bit buffer.

Static Buffer Lifetime:: Uses static s_soft_bits[k_harq_soft_buffer_size] (16,400 bytes) to avoid stack overflow. Safe because function is never called concurrently. Buffer is overwritten on each call (no persistent state).

Example (FEC Decode Flow):: rx_frame_tframe; rx_spi_comm_receive(spi,&frame,timeout);
rx_spi_link_receive_result_tresult; rx_err_terr=internal_receive_fec_decode(link,&frame,&result);
if(err==k_rx_ok){ process_decoded_data(result.payload,result.payload_len); }else{ }



Figure 1139: Call graph for internal_receive_fec_decode

Defined in `libs/rx_spi_link/src/rx_spi_link.c:635`

—

internal_receive_passthrough

```
static rx_err_t internal_receive_passthrough(rx_spi_link_t *link, const
rx_frame_t *frame, rx_spi_link_receive_result_t *result)
```

Process received frame without FEC (passthrough mode).

Copies payload directly to result buffer and sends ACK if required by frame flags. No FEC decoding or Chase Combining applied.

Parameters:

Direction	Name	Description
inout	link	Link handle (must be initialized)
in	frame	Received frame with raw payload
out	result	Receive result (payload, length, metadata)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, payload copied and ACK sent (if required)



Figure 1140: Call graph for internal_receive_passthrough

Defined in `libs/rx_spi_link/src/rx_spi_link.c:712`

rx_spi_link_receive

```
rx_err_t rx_spi_link_receive(rx_spi_link_t *link, rx_spi_link_receive_result_t
*result, uint32_t timeout_ms)
```

Receive and decode a frame from SPI with HARQ.

See also: `rx_spi_link.h` for full documentation

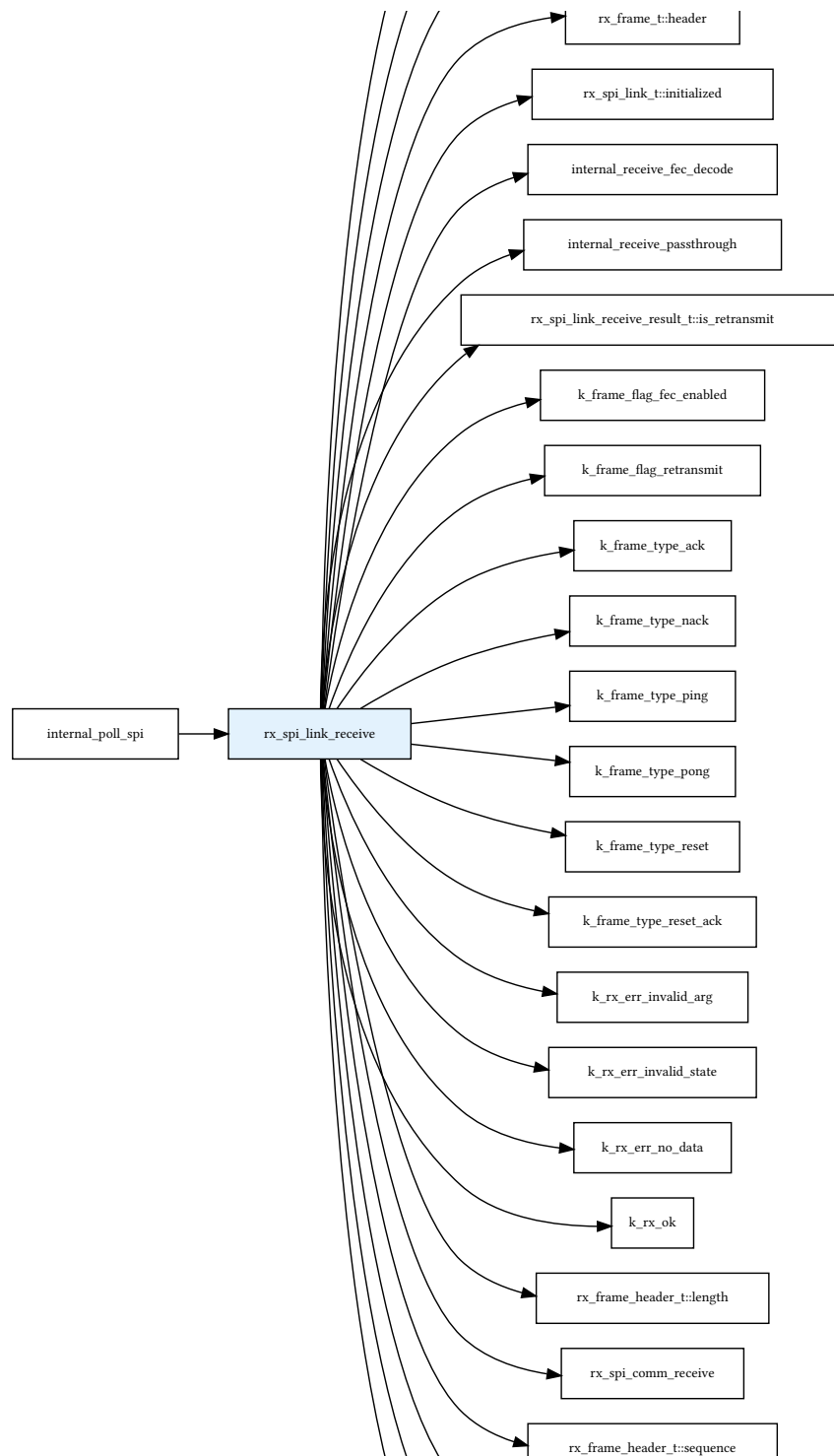


Figure 1141: Call graph for rx_spi_link_receive

Defined in `libs/rx_spi_link/src/rx_spi_link.c:748`

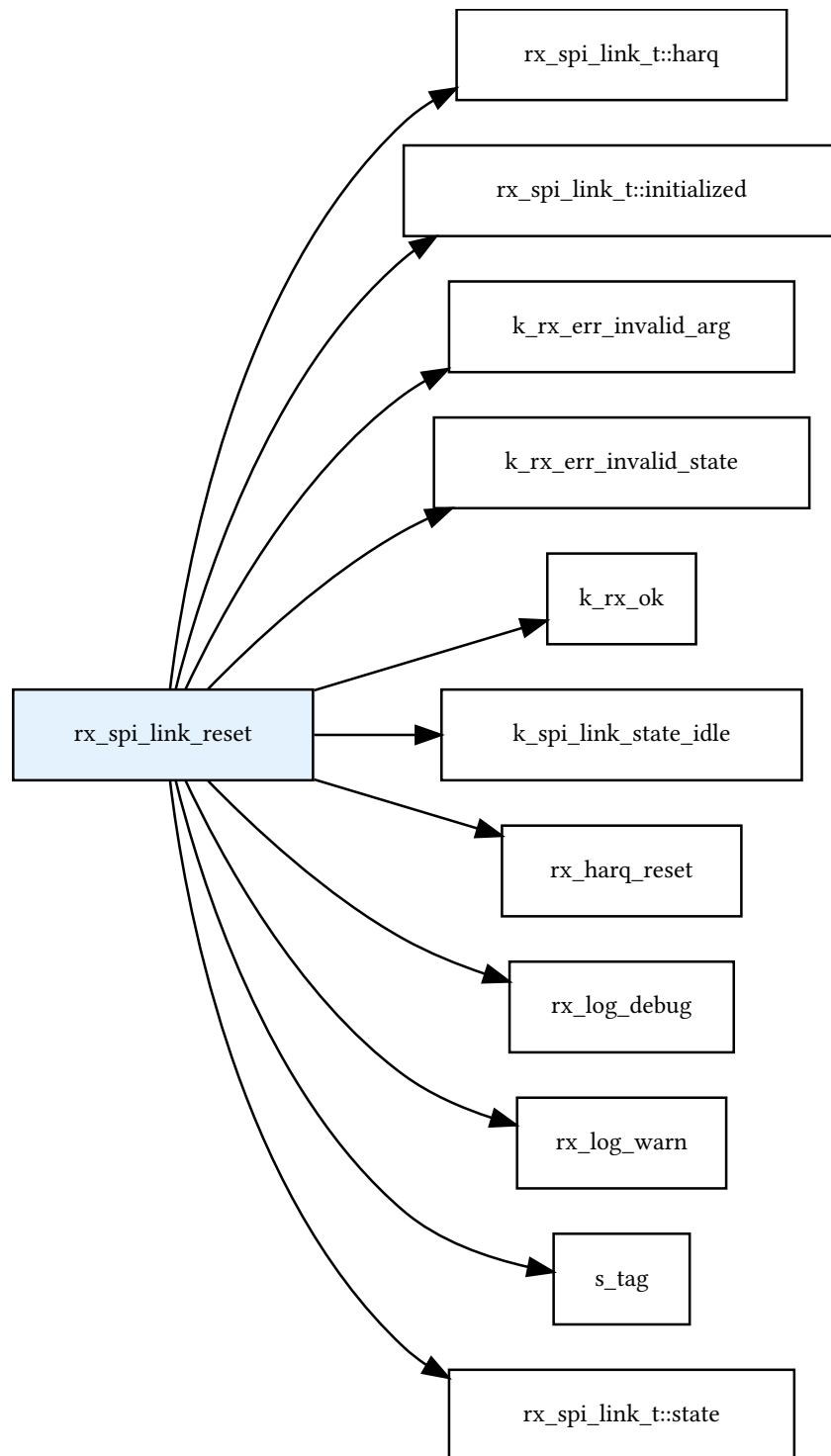
—

rx_spi_link_reset

```
rx_err_t rx_spi_link_reset(rx_spi_link_t *link)
```

Reset SPI link layer for new session.

See also: `rx_spi_link.h` for full documentation

Figure 1142: Call graph for `rx_spi_link_reset`

Defined in `libs/rx_spi_link/src/rx_spi_link.c:809`

—

rx_spi_link_get_state

```
rx_spi_link_state_t rx_spi_link_get_state(const rx_spi_link_t *link)
```

Get current SPI link state.

See also: `rx_spi_link.h` for full documentation

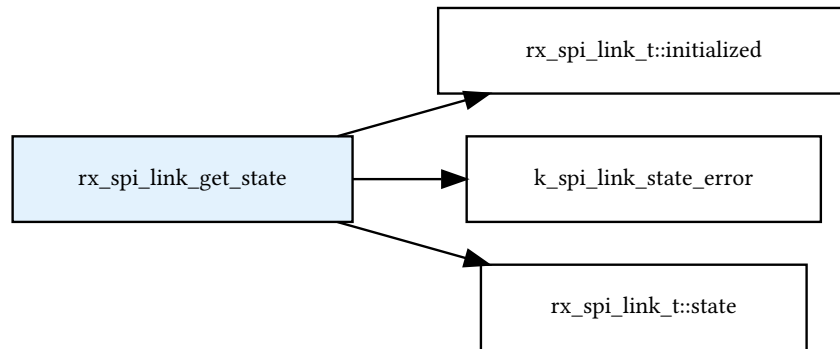


Figure 1143: Call graph for `rx_spi_link_get_state`

Defined in `libs/rx_spi_link/src/rx_spi_link.c:838`

—

rx_spi_link_fec_enabled

```
bool rx_spi_link_fec_enabled(const rx_spi_link_t *link)
```

Check if FEC is enabled on this link.

See also: `rx_spi_link.h` for full documentation

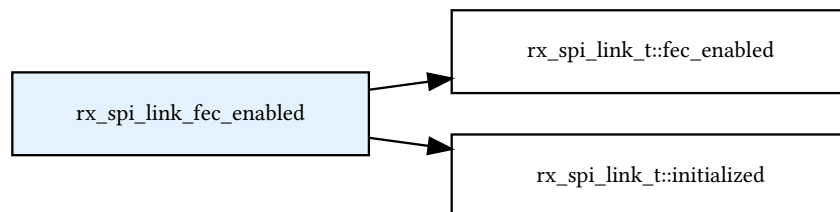


Figure 1144: Call graph for `rx_spi_link_fec_enabled`

Defined in `libs/rx_spi_link/src/rx_spi_link.c:850`

—

rx_usb.h

Multi-Port USB CDC-ACM Composite Device Driver API for RX72N.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"

rx_usb_port_id_t

USB CDC-ACM virtual serial port identifiers.

Defines the 3 independent CDC-ACM virtual serial ports provided by the USB composite device. Each port has dedicated TX/RX ring buffers, endpoints, and appears as a separate /dev/ttyACM* device on Linux hosts.

Design Rationale:

- 3 ports: Separates protocol, debug, and logging traffic for clarity
- Port 0 (Protocol): High bandwidth, binary data, critical path
- Port 1 (Decoded): Medium bandwidth, ASCII text, development/debugging
- Port 2 (Log): High TX, low RX, one-way logging output
- Hardware limit: RX72N USB0 has 9 pipes; each CDC needs 3 (1 interrupt + 2 bulk)

Host Device Mapping (Linux):

Port Assignment Example:

Value	Name	Description
= 0	k_usb_port_proto	Protocol port for binary communication (nanopb frames).
= 1	k_usb_port_decoded	Decoded port for ASCII frame dumps and debugging.
= 2	k_usb_port_log	Log port for system logging (mirrored to UART).
= 3	k_usb_port_count	Number of active USB CDC-ACM ports.
= 3	k_usb_port_max	Maximum number of ports supported by hardware.

See also: rx_usb_write() Send data to a port, rx_usb_read() Receive data from a port, rx_usb_is_configured() Check if port is ready

Example:

RX72NPort->HostDeviceTypicalUsage

```
=====
k_usb_port_proto->/dev/ttyACM0nanopbprotocol,motorcommands
k_usb_port_decoded->/dev/ttyACM1ASCIIframedumps,debugging
k_usb_port_log->/dev/ttyACM2Logoutput,diagnostics
```

Example:

```
//Sendmotorcommandonprotocolport
uint8_tcmd_frame[64];
encode_motor_command(cmd_frame,...);
rx_usb_write(k_usb_port_proto,cmd_frame,64);

//SendASCIIdebuginfoondecodedport
rx_usb_puts(k_usb_port_decoded,"Frame:type=0x01seq=123\r\n");

//Sendlogmessageonlogport
rx_usb_puts(k_usb_port_log,"[INF0]Motorinitialized\r\n");
```

Since Version 1.0.0

—

rx_usb_event_t

USB event types delivered to user callback function.

Defines all asynchronous USB events that can trigger the user-provided callback function (rx_usb_callback_t). Events are generated by the USB interrupt handler and delivered from ISR context.

Event Categories:

- Connection events: Attached, Detached (cable physical state)
- Enumeration events: Reset, Configured (USB state machine transitions)
- Power events: Suspended, Resumed (host power management)
- Data events: Data RX, TX Complete (transfer completion)

Event Timing:

Callback Constraints:

- ISR context: Callback runs in USB interrupt, keep fast (<10 us)
- No blocking: Do not call tx_thread_sleep(), tx_mutex_get(), etc.
- Safe operations: Set flags, post semaphores, queue messages

Value	Name	Description
= 0	k_usb_event_attached	USB cable attached (VBUS detected).
= 1	k_usb_event_detached	USB cable detached (VBUS lost).
= 2	k_usb_event_reset	USB bus reset received from host.
= 3	k_usb_event_configured	Port configured by host (ready for data transfer).
= 4	k_usb_event_suspended	Host suspended the device (USB idle >3ms).
= 5	k_usb_event_resumed	Host resumed the device (USB activity detected).
= 6	k_usb_event_data_rx	Data received from host (RX buffer has new data).
= 7	k_usb_event_tx_complete	TX buffer drained (all queued data sent to host).

See also: rx_usb_callback_t Callback function type, rx_usb_set_callback() Register callback, rx_usb_config_t Configuration with callback

Example:

```
CablePlug->Attached(immediate)
->Reset(20-100ms)
->Configured(100-200ms)
->DataRX/TX(ongoing)
->Suspended(hostidle>3ms)
->Resumed(hostactivity)
CableUnplug->Detached(immediate)
```

Since Version 1.0.0

—

rx_usb_endpoints.h

USB CDC Endpoint Address Definitions for RX72N Composite Device.

Includes:

- <stdint.h>

usb_endpoint_addresses_t

USB CDC endpoint addresses for three-port composite device.

Defines the USB endpoint addresses for all nine endpoints used by the STAR motor controller's USB CDC composite device. Each of the three virtual serial ports requires three endpoints:

- Bulk IN: Device-to-host data transfer (0x8N format)
- Bulk OUT: Host-to-device data transfer (0x0N format)
- Interrupt IN: Asynchronous notifications (0x8N format)

Value	Name	Description
= 0x81	k_port0_ep_bulk_in	Port 0 bulk IN endpoint for data transmission to host.
= 0x02	k_port0_ep_bulk_out	Port 0 bulk OUT endpoint for data reception from host.
= 0x83	k_port0_ep_int_in	Port 0 interrupt IN endpoint for CDC notifications.
= 0x84	k_port1_ep_bulk_in	Port 1 bulk IN endpoint for decoded ASCII output.
= 0x05	k_port1_ep_bulk_out	Port 1 bulk OUT endpoint for decoded port input.
= 0x86	k_port1_ep_int_in	Port 1 interrupt IN endpoint for CDC notifications.
= 0x87	k_port2_ep_bulk_in	Port 2 bulk IN endpoint for log output.
= 0x08	k_port2_ep_bulk_out	Port 2 bulk OUT endpoint for log port input.
= 0x89	k_port2_ep_int_in	Port 2 interrupt IN endpoint for CDC notifications.

—

rx_usb_private.h

Private types and declarations for USB driver internal testing.

Includes:

- <stdint.h>
- "rx_usb.h"

rx_usb.c

Multi-Port USB CDC-ACM Composite Device Driver Implementation for RX72N.

Includes:

- "rx_usb.h"
- "rx_time_constants.h"
- "rx_usb_endpoints.h"
- "rx_usb_internal.h"
- "rx72n_regs.h"
- "rx_log.h"
- "rx_threadx_config.h"
- "tx_api.h"

flush_loop_bounds_t

USB flush timing and iteration constants.

For NASA Rule 2 compliance, flush operations use compile-time bounded loops. Maximum flush timeout is 10000ms with 10ms poll interval = 1000 max iterations.

Value	Name	Description
= 10000	k_flush_max_timeout_ms	Maximum flush timeout (10 seconds)
= 1000	k_flush_max_iterations	Maximum flush loop iterations (10000ms / 10ms)

—

ring_buffer_constants_t

Ring buffer operation constants.

Value	Name	Description
= 0	k_ring_buffer_empty	Empty buffer count
= 1	k_ring_buffer_increment	Increment value for buffer indices
= 0	k_min_transfer_size	Minimum data transfer size (no data)
= 1	k_min_sleep_ticks	Minimum sleep duration (1 tick)

—

port_config_constants_t

Port configuration constants.

Value	Name	Description
= 2	k_port_interfaces_per_cdc	Each CDC function has 2 interfaces (control + data)
= 3	k_port_pipes_per_cdc	Each CDC function uses 3 pipes

—

port_pipe_assignments_t

Port pipe assignments.

Value	Name	Description
= 1	k_port0_pipe_bulk_in	Pipe 1
= 2	k_port0_pipe_bulk_out	Pipe 2
= 3	k_port0_pipe_int_in	Pipe 3
= 4	k_port1_pipe_bulk_in	Pipe 4
= 5	k_port1_pipe_bulk_out	Pipe 5
= 6	k_port1_pipe_int_in	Pipe 6
= 7	k_port2_pipe_bulk_in	Pipe 7
= 8	k_port2_pipe_bulk_out	Pipe 8
= 9	k_port2_pipe_int_in	Pipe 9
= 1	k_data_pipe_min	Minimum data pipe number (first port pipe)
= 9	k_data_pipe_max	Maximum data pipe number (last port pipe)

—

int_conversion_constants_t

Constants for integer-to-string conversion.

Value	Name	Description
= 1	k_single_byte_count	Byte count for single character (putc)
= 12	k_int_buffer_size	Max digits for int32_t (-2147483648) + null
= 9	k_hex_buffer_size	Max 8 hex digits + null
= 10	k_decimal_base	Base 10 for decimal conversion
= 16	k_hex_base	Base 16 for hex conversion
= 8	k_max_hex_digits	Maximum hex digits (32-bit value)
= 4	k_bits_per_hex_nibble	Bits per hexadecimal nibble
= 0x0F	k_hex_nibble_mask	Mask to extract lowest nibble
= 10	k_hex_letter_offset	Offset for hex letters (A-F)

—

s_tag`const char* s_tag`

—

s_flush_poll_interval_ms`const uint16_t s_flush_poll_interval_ms`

USB flush poll interval.

—

s_port_configs`const rx_usb_port_hw_config_t s_port_configs[k_usb_port_count]`

Port configuration table (const, compile-time).

—

s_port0_rx_buf`uint8_t s_port0_rx_buf[k_usb_port_proto_rx_size]`

—

s_port0_tx_buf`uint8_t s_port0_tx_buf[k_usb_port_proto_tx_size]`

—

s_port1_rx_buf`uint8_t s_port1_rx_buf[k_usb_port_decoded_rx_size]`

—

s_port1_tx_buf`uint8_t s_port1_tx_buf[k_usb_port_decoded_tx_size]`

```
—  
  
s_port2_rx_buf  
uint8_t s_port2_rx_buf[k_usb_port_log_rx_size]  
  
—  
  
s_port2_tx_buf  
uint8_t s_port2_tx_buf[k_usb_port_log_tx_size]  
  
—  
  
s_usb  
usb_driver_t s_usb  
  
—  
  
internal_port_is_valid  
  
static bool internal_port_is_valid(const rx_usb_port_id_t port)
```

Check if port ID is valid.

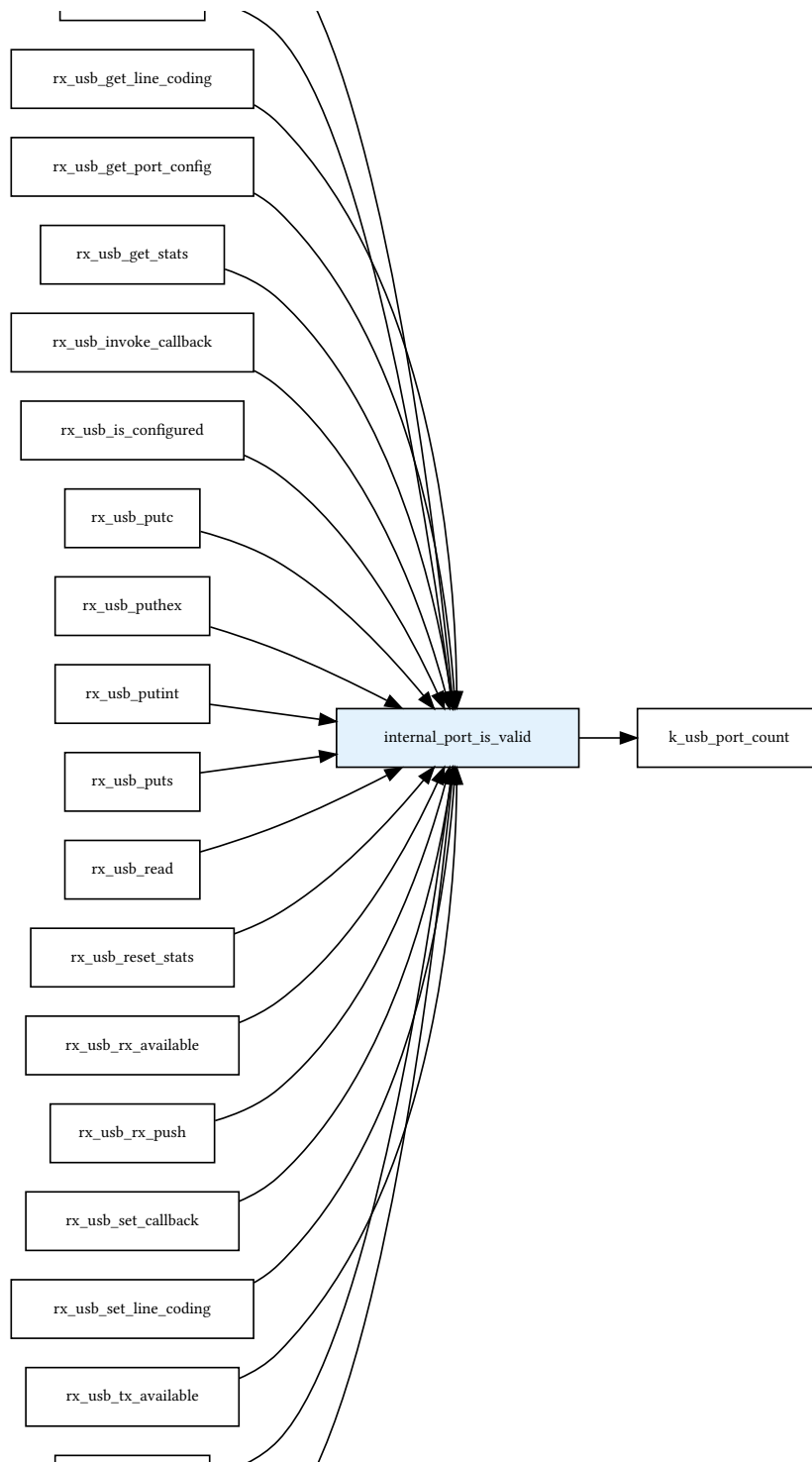


Figure 1145: Call graph for internal_port_is_valid

Defined in `libs/rx_usb/src/rx_usb.c:659`

—

internal_state_is_valid

```
static bool internal_state_is_valid(const rx_usb_state_t state)
```

Check if USB state is valid.

Validates USB state enum value. Assumes `rx_usb_state_t` enums are sequential with `k_usb_state_suspended` as the highest valid value.

Parameters:

Direction	Name	Description
in	state	USB state to validate

Returns: false if state is invalid

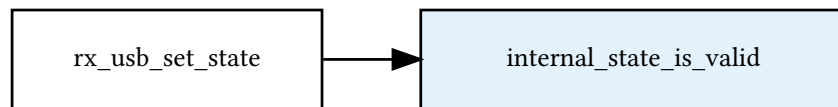


Figure 1146: Call graph for `internal_state_is_valid`

Defined in `libs/rx_usb/src/rx_usb.c:674`

—

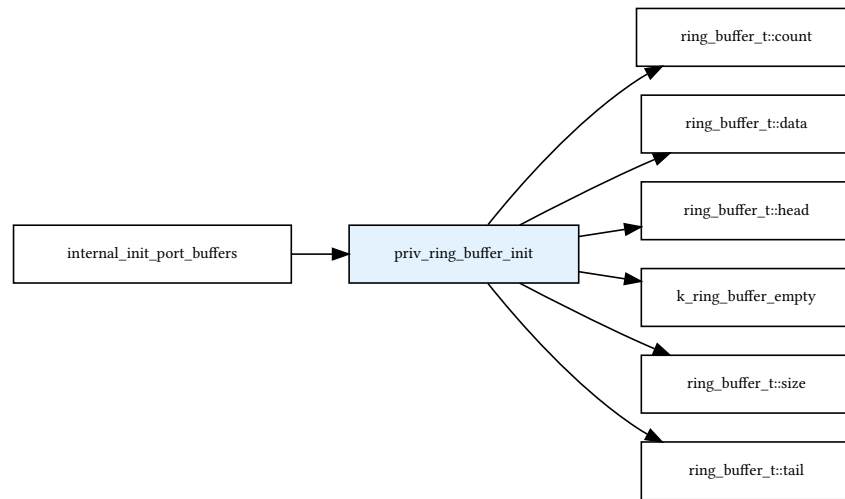
priv_ring_buffer_init

```
void priv_ring_buffer_init(ring_buffer_t *buf, uint8_t *data, const uint32_t size)
```

Initialize a ring buffer with backing storage.

Parameters:

Direction	Name	Description
inout	buf	Ring buffer to initialize
in	data	Backing storage buffer
in	size	Backing buffer size in bytes

Figure 1147: Call graph for `priv_ring_buffer_init`

Defined in `libs/rx_usb/src/rx_usb.c:700`

priv_ring_buffer_available

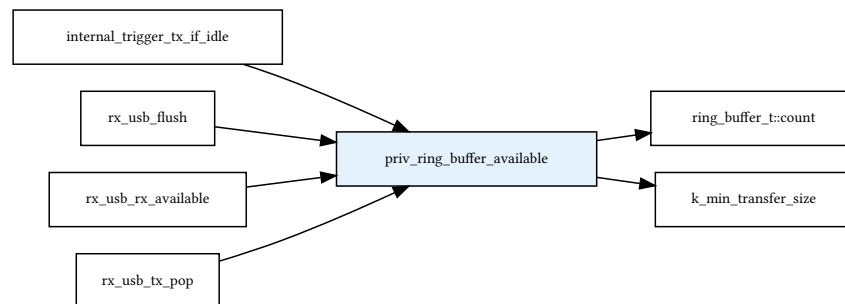
```
uint32_t priv_ring_buffer_available(const ring_buffer_t *buf)
```

Get number of bytes available to read from ring buffer.

Parameters:

Direction	Name	Description
in	buf	Ring buffer to query

Returns: Number of bytes available, or 0 if buffer is nullptr

Figure 1148: Call graph for `priv_ring_buffer_available`

Defined in `libs/rx_usb/src/rx_usb.c:720`

priv_ring_buffer_free

```
uint32_t priv_ring_buffer_free(const ring_buffer_t *buf)
```

Get number of free bytes available for writing to ring buffer.

Parameters:

Direction	Name	Description
in	buf	Ring buffer to query

Returns: Number of free bytes, or 0 if buffer is nullptr

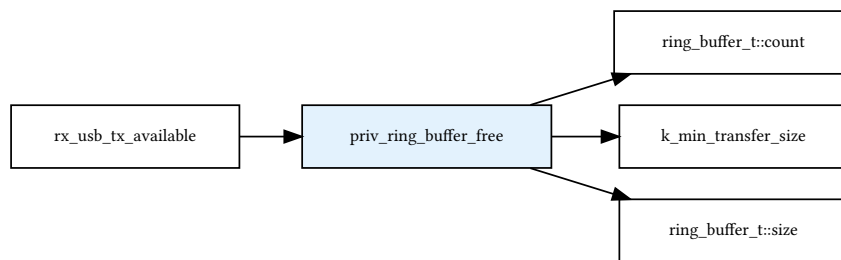


Figure 1149: Call graph for `priv_ring_buffer_free`

Defined in `libs/rx_usb/src/rx_usb.c:736`

—

priv_ring_buffer_write

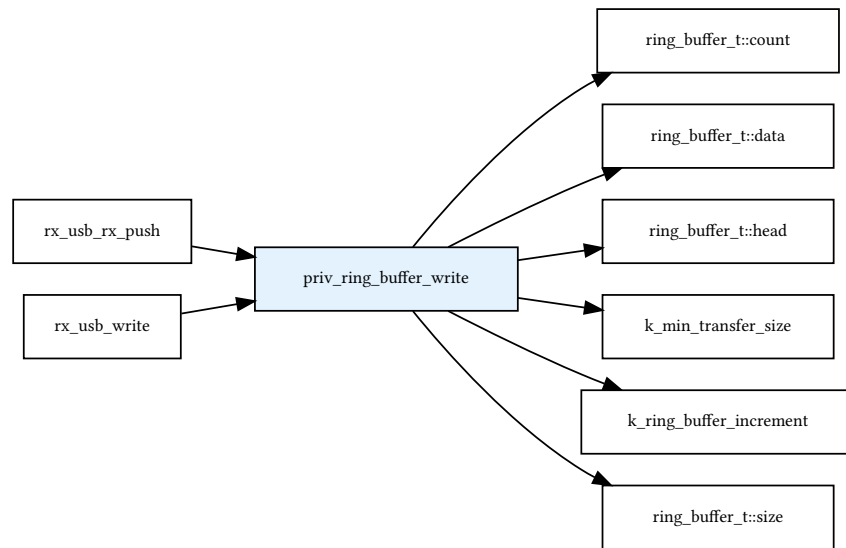
```
uint32_t priv_ring_buffer_write(ring_buffer_t *buf, const uint8_t *data, const
uint32_t len)
```

Write bytes into the ring buffer.

Parameters:

Direction	Name	Description
inout	buf	Ring buffer to write to
in	data	Source data
in	len	Number of bytes to write

Returns: Number of bytes written

Figure 1150: Call graph for `priv_ring_buffer_write`

Defined in `libs/rx_usb/src/rx_usb.c:754`

—

priv_ring_buffer_read

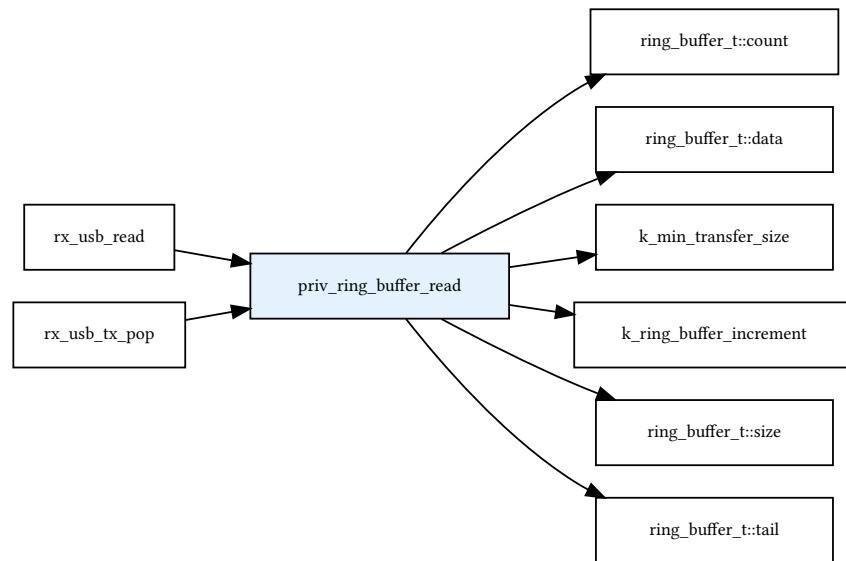
```
uint32_t priv_ring_buffer_read(ring_buffer_t *buf, uint8_t *data, const uint32_t
max_len)
```

Read bytes from the ring buffer.

Parameters:

Direction	Name	Description
inout	buf	Ring buffer to read from
out	data	Destination buffer
in	max_len	Max bytes to read

Returns: Number of bytes read

Figure 1151: Call graph for `priv_ring_buffer_read`

Defined in `libs/rx_usb/src/rx_usb.c:785`

—

internal_trigger_tx_if_idle

```
static void internal_trigger_tx_if_idle(const rx_usb_port_id_t port)
```

Trigger USB transmission for a port if pipe is not busy.

Parameters:

Direction	Name	Description
in	port	Port to trigger transmission for

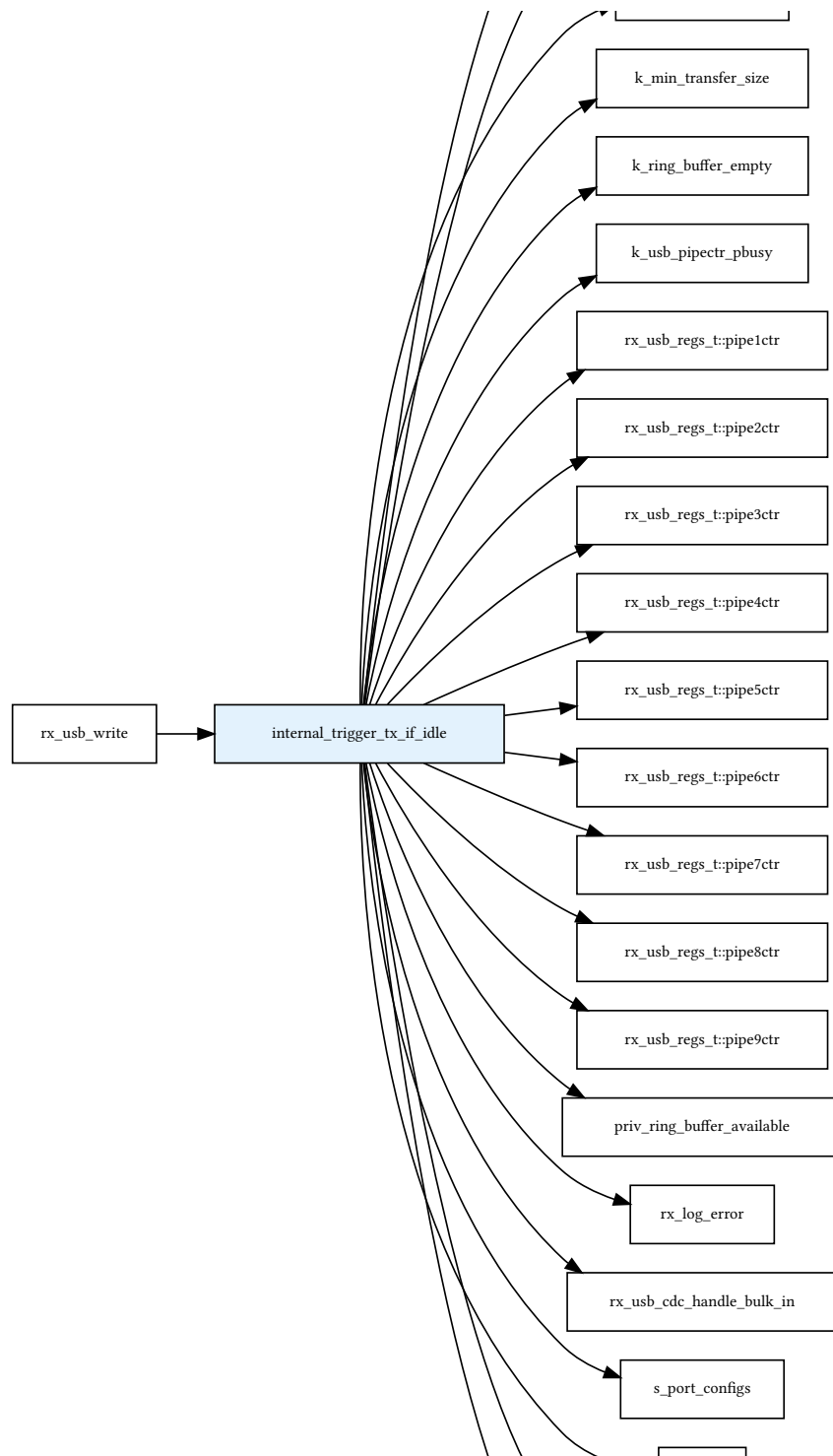


Figure 1152: Call graph for internal_trigger_tx_if_idle

Defined in `libs/rx_usb/src/rx_usb.c:818`

—

internal_init_port_buffers

```
static void internal_init_port_buffers(void)
```

Initialize all port buffers.

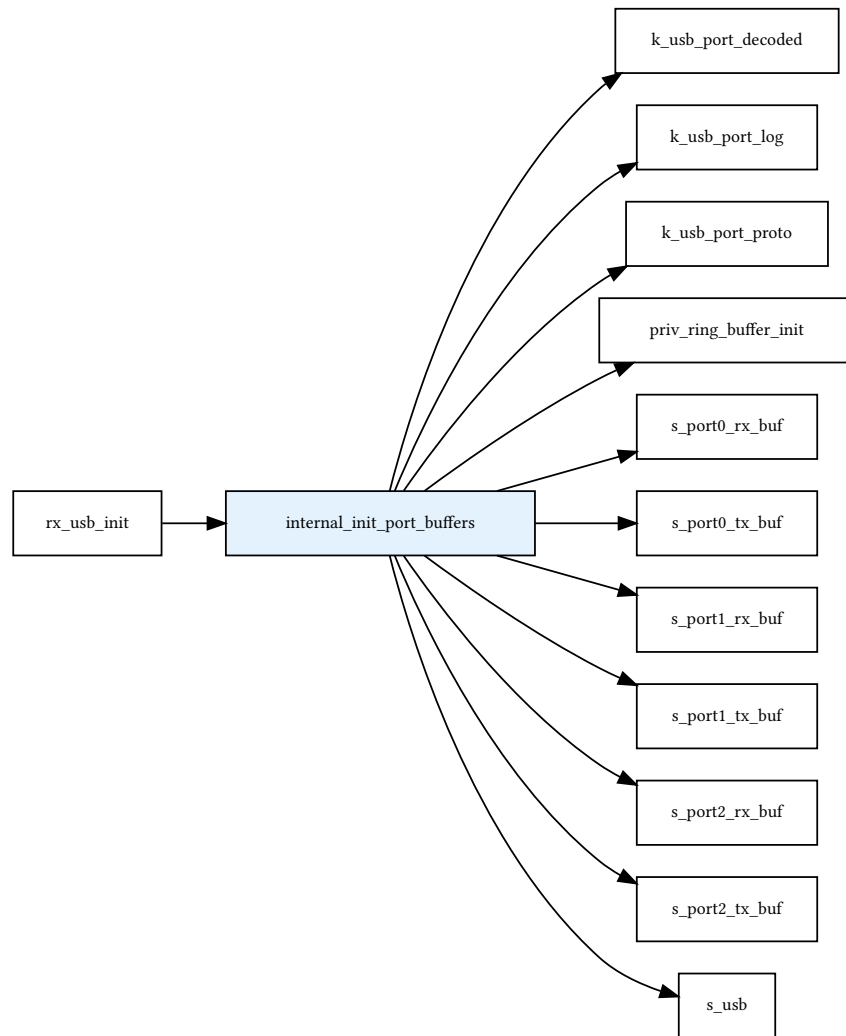


Figure 1153: Call graph for `internal_init_port_buffers`

Defined in `libs/rx_usb/src/rx_usb.c:867`

—

internal_init_line_coding

```
static void internal_init_line_coding(void)
```

Initialize default line coding for all ports.

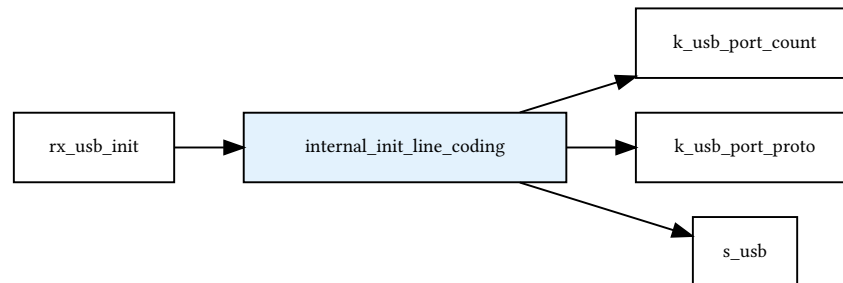


Figure 1154: Call graph for internal_init_line_coding

Defined in `libs/rx_usb/src/rx_usb.c:897`

—

rx_usb_init

```
rx_err_t rx_usb_init(const rx_usb_config_t *config)
```

Initialize USB CDC composite device driver.

Initializes the USB CDC-ACM composite device driver with 3 independent virtual serial ports. This function must be called once during system initialization before any other USB operations.

Initialization Sequence:

1. Validate not already initialized (prevent double-init)
2. Clear driver state (memset global state to zero)
3. Store global callback (optional, for device-level events)
4. Initialize ring buffers (RX/TX for all 3 ports, statically allocated)
5. Initialize line coding (default 115200 baud, 8N1)
6. Initialize hardware layer (`rx_usb_hw_init()`) - configure USB0 peripheral)
7. Initialize CDC class (`rx_usb_cdc_init()`) - setup descriptors, endpoints)
8. Attach to USB bus (`rx_usb_hw_attach()`) - enable D+ pull-up)
9. Mark initialized (set global flag, `device_state = attached`)

Post-Initialization State:

- Device state: `k_usb_state_attached` (waiting for host enumeration)
- Port states: Not configured (ports not yet usable)
- Ring buffers: Empty, ready for data
- Line coding: 115200 baud, 8 data bits, 1 stop bit, no parity
- Hardware: USB0 peripheral enabled, D+ pull-up active, interrupts enabled

Timeline to Ready State:

Configuration Structure:

Global Callback vs. Per-Port Callback:

- Global callback (this init function): Device-level events (bus reset, suspend)
- Per-port callback (rx_usb_set_callback()): Port-specific events (data RX, TX complete)
- Both can be used simultaneously

Parameters:

Direction	Name	Description
in	config	Optional configuration structure NULL allowed: No global callback, default configuration used Non-NULL: Global callback registered, invoked on device events

Returns: rx_err_t Initialization result

Value	Description
k_rx_ok	Initialization successful, device attached to bus
k_rx_err_invalid_state	Already initialized (double-init prevented)
k_rx_err_hardware	Hardware initialization failed (USB0 peripheral fault)
k_rx_err_timeout	CDC initialization timeout

Pre: System clock (ICLK, PCLKB) must be configured and stable

Pre: USB0 peripheral power domain must be enabled

Pre: Interrupts must be enabled globally (ICU configured)

Post: On success, USB driver ready, device attached to bus (D+ pull-up active)

Post: On failure, driver state unchanged, hardware not modified

Post: Ring buffers cleared and ready for data

Post: All 3 ports in unconfigured state (will configure during enumeration)

Note: Call this ONCE during system initialization (thread-safe: single-threaded init)

Note: Typical initialization time: 5 ms

Note: Host enumeration takes additional 50-100 ms after init

Warning: Do not call from interrupt context

Warning: If init fails, do not call any other USB functions

Performance:: Execution time: 5 ms @ 240 MHz (hardware init dominates) Memory impact: 7668 bytes static RAM (all buffers allocated) Stack usage: 48 bytes (locals + function calls)

Example - Basic Initialization:: voidsystem_init(void) { rx_err_t err=rx_usb_init(nullptr);

if(err==k_rx_ok){ rx_log_info("USB","USBinitialized,waitingforhost"); }

else{ rx_log_error("USB","USBinitfailed:%d",err); }

while(!rx_usb_is_configured(k_usb_port_proto)){ tx_thread_sleep(10); }

rx_log_info("USB","USBport0configuredandready"); }

Example - Initialization with Global Callback:: voidusb_device_callback(rx_usb_port_id_t port, rx_usb_event_t event, void* context) { (void)port;

switch(event){ casek_usb_event_reset: rx_log_warn("USB","Busresetdetected"); break;

casek_usb_event_suspended: rx_log_info("USB","Enteringsuspend(hostidle)"); break;

casek_usb_event_resumed: rx_log_info("USB","Resumedfromsuspend"); break; default: break; }

voidsystem_init(void) { rx_usb_config_t config={ .callback=usb_device_callback, .ctx=nullptr; }

rx_err_t err=rx_usb_init(&config); if(err!=k_rx_ok){ }

Example - Error Handling:: rx_err_t init_usb_with_retry(void) { constuint8_t max_retries=3;

for(uint8_t i=0; i<max_retries; i++){ rx_err_t err=rx_usb_init(nullptr);

if(err==k_rx_ok){ returnk_rx_ok; }

if(err==k_rx_err_invalid_state){ returnk_rx_ok; }

rx_log_warn("USB","Initfailed(attempt%u/%u):%d",i+1,max_retries,err); tx_thread_sleep(100); }

returnk_rx_err_hardware; }

Example:

T+0ms: rx_usb_init() called

T+5ms: USBAttach(D+pull-upenabled)

T+10ms: Hostdetectsdevice(busreset)

T+50ms: Deviceenumeration(descriptorsexchanged)

T+100ms: SET_CONFIGURATION(portsbecomeconfigured)

T+100ms: All3portsreadyforrx_usb_write()/rx_usb_read()

Example:

typedefstruct{

rx_usb_callback_t callback; //Optionalglobalcallbackfordeviceevents

void* ctx; //Optionalcontextforcallback

} rx_usb_config_t;

See also: rx_usb_deinit() Shutdown USB driver, rx_usb_is_configured() Check if port ready for communication, rx_usb_set_callback() Register per-port callback, rx_usb_hw.c Hardware layer implementation, rx_usb_cdc.c CDC class implementation

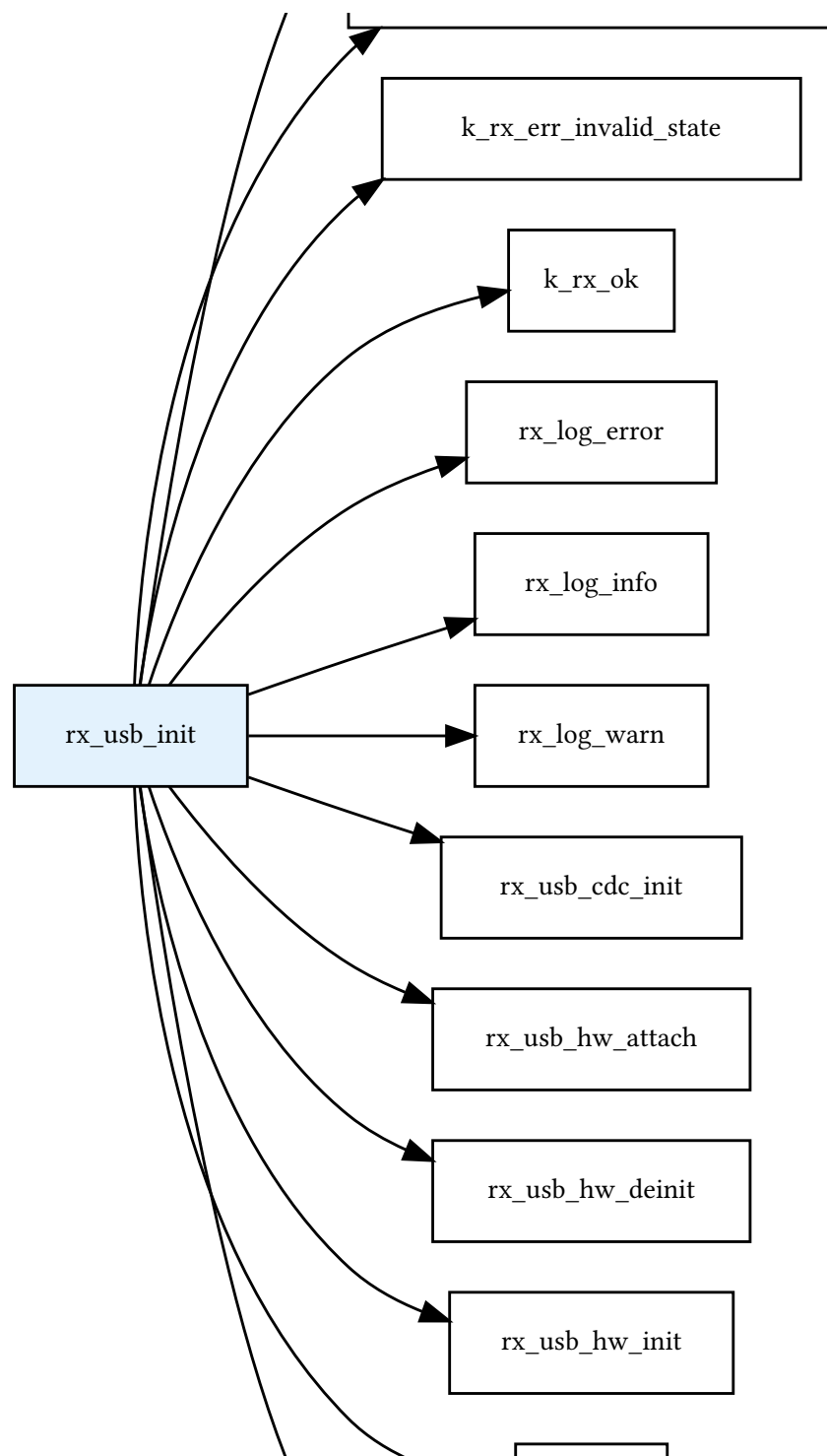


Figure 1155: Call graph for rx_usb_init

Defined in `libs/rx_usb/src/rx_usb.c:1087`

Since Version 1.0.0

—

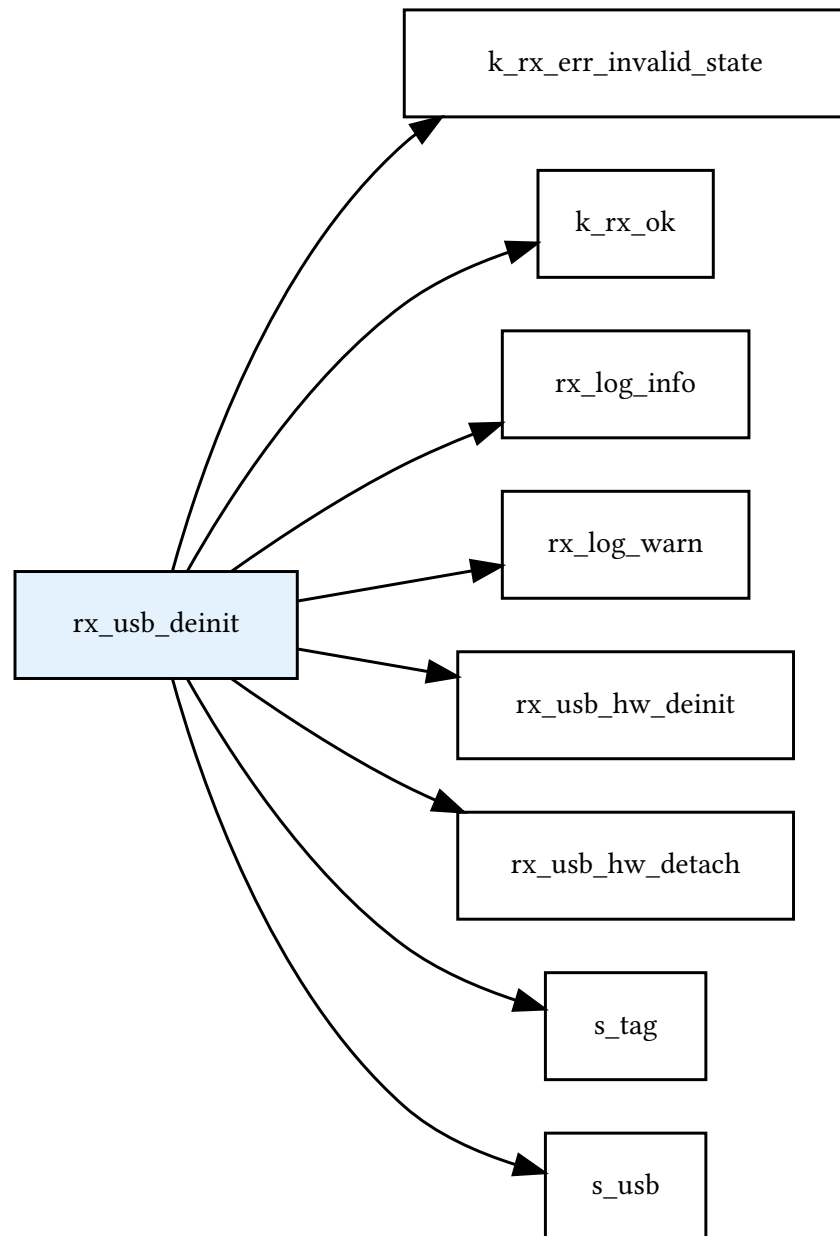
rx_usb_deinit

```
rx_err_t rx_usb_deinit(void)
```

Deinitialize USB CDC composite driver.

Detaches from USB bus, deinitializes hardware, and clears driver state.

Returns: `k_rx_err_invalid_state` if driver not initialized

Figure 1156: Call graph for `rx_usb_deinit`

Defined in `libs/rx_usb/src/rx_usb.c:1158`

—

rx_usb_is_configured

```
bool rx_usb_is_configured(const rx_usb_port_id_t port)
```

Check if USB port is configured by host.

Parameters:

Direction	Name	Description
in	port	Port ID to check

Returns: false if port is invalid or not configured

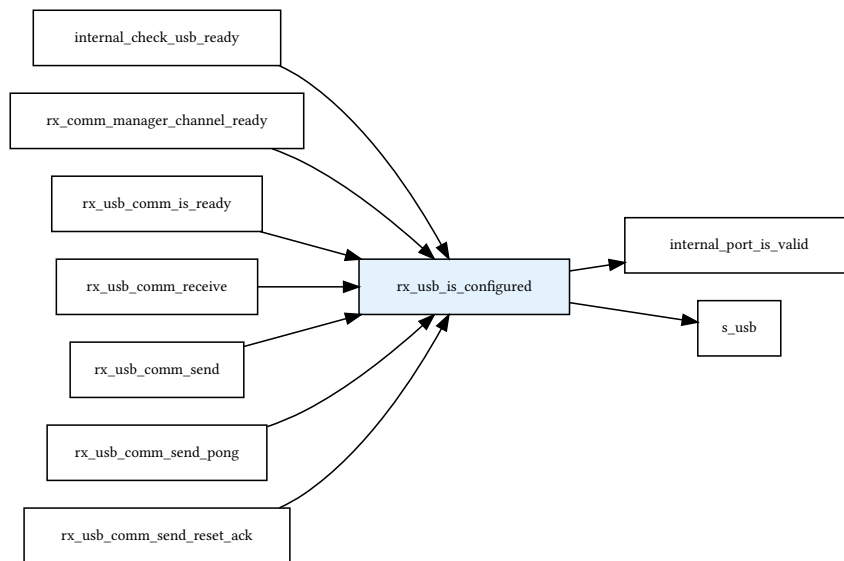


Figure 1157: Call graph for rx_usb_is_configured

Defined in `libs/rx_usb/src/rx_usb.c:1192`

—

rx_usb_get_state

```
rx_usb_state_t rx_usb_get_state(void)
```

Get current USB device state.

Returns: Current USB device state (attached, configured, suspended, etc.)

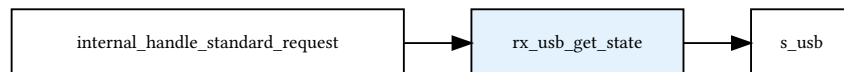


Figure 1158: Call graph for rx_usb_get_state

Defined in `libs/rx_usb/src/rx_usb.c:1206`

—

rx_usb_write

```
rx_err_t rx_usb_write(const rx_usb_port_id_t port, const uint8_t *data, const
uint32_t len)
```

Write data to USB CDC port (transmit to host).

Writes data to the specified port's TX ring buffer and initiates transmission to the USB host if the hardware pipe is idle. This function returns immediately after copying data to the ring buffer - actual USB transmission happens asynchronously via ISR context.

Operation Flow:

1. Validate parameters (port ID, data pointer, driver state)
2. Check configured state (port must be configured by host)
3. Copy data to TX ring buffer (priv_ring_buffer_write)
4. Update statistics (bytes_tx, tx_underruns if buffer full)
5. Trigger transmission (if pipe idle, start first transfer)
6. Return immediately (USB transmission continues in ISR)

Transmission Timeline:

Ring Buffer Behavior:

- Capacity: Port 0 (1024 bytes), Port 1 (512 bytes), Port 2 (2048 bytes)
- Full buffer: Returns k_rx_err_busy, increments tx_underruns
- Partial write: If buffer has 100 bytes free and len=200, writes 100 and returns busy
- Available space: Query via rx_usb_tx_available() before writing

Blocking vs Non-Blocking:

- This function is non-blocking (returns immediately after buffer copy)
- To wait for transmission completion, call rx_usb_flush() after this function
- Example: rx_usb_write(port, data, len); rx_usb_flush(port, 100);

Thread Safety:

- Per-port safe: Multiple tasks can write to different ports concurrently
- Not multi-writer safe: Only one task should write to each port
- ISR interaction: ISR reads from TX buffer, no locking needed (producer/consumer)

Parameters:

Direction	Name	Description
in	port	Port ID to write to Valid values: k_usb_port_proto (0), k_usb_port_decoded (1), k_usb_port_log (2) Invalid values: Returns k_rx_err_invalid_arg
in	data	Data buffer to transmit Valid range: Pointer to buffer of size len NULL handling: Returns k_rx_err_null_ptr Alignment: No alignment requirement (byte-addressable) Lifetime: Data copied to ring buffer, source buffer can be freed immediately
in	len	Number of bytes to write Valid range: 0 - 4294967295 Typical: 1-1024 bytes per call Zero handling: len=0 is valid (no-op, returns k_rx_ok) Large writes: If len > buffer capacity, partial write + k_rx_err_busy

Returns: rx_err_t Write result

Value	Description
k_rx_ok	Success, all data written to TX buffer
k_rx_err_invalid_arg	Port ID invalid (port >= k_usb_port_count)
k_rx_err_null_ptr	data pointer is nullptr (len > 0)
k_rx_err_invalid_state	Driver not initialized via rx_usb_init()
k_rx_err_invalid_state	Port not configured by host (call rx_usb_is_configured())
k_rx_err_busy	TX buffer full, partial write occurred (check tx_underruns stat)

Pre: Driver must be initialized via rx_usb_init()

Pre: Port must be configured by host (rx_usb_is_configured() returns true)

Pre: data must point to valid buffer of size len (if len > 0)

Post: On k_rx_ok, all data copied to TX ring buffer

Post: On k_rx_err_busy, partial data copied, tx_underruns incremented

Post: USB transmission initiated (if pipe was idle)

Post: bytes_tx statistic updated with bytes written

Note: This function is non-blocking (returns before USB transmission completes)

Note: Use rx_usb_flush() to wait for transmission completion

Note: If called from ISR context, keep data size small (<64 bytes)

Warning: Calling with unconfigured port returns error (wait for enumeration first)

Warning: Large writes may fill buffer - check return value and handle k_rx_err_busy

Performance: **Execution time**: 1-5 us @ 240 MHz (ring buffer copy + trigger) ***Throughput***: 30 MB/s (ring buffer copy via memcpy) ****USB bandwidth****: 1.2 MB/s (actual transmission to host, USB Full-Speed limit) ****Stack usage****: 32 bytes (locals + ring buffer function call)

Example - Basic Write: conrx_usb_port_id_tport=k_usb_port_proto;
 constchar*msg="HelloUSB!\n";
 while(!rx_usb_is_configured(port)){ tx_thread_sleep(10); }

```
rx_err_t err=rx_usb_write(port,(uint8_t*)msg,strlen(msg)); if(err==k_rx_ok)
{ rx_log_debug("USB","Writesuccessful"); }elseif(err==k_rx_err_busy)
{ rx_log_warn("USB","TXbufferfull,datatruncated"); }
```

Example - Write with Flush:: rx_err_t err=rx_usb_write(k_usb_port_decoded,data,len);
if(err==k_rx_ok){ rx_usb_flush(k_usb_port_decoded,100); }

Example - Check Buffer Space:: uint32_t available; rx_usb_tx_available(port,&available);
if(available>=len){ rx_usb_write(port,data,len); }else{ rx_log_warn("USB","TXbufferfull(%u/
%u free)",available,len); }

Example - Handle Busy (Retry):: rx_err_t err=rx_usb_write(port,data,len);
if(err==k_rx_err_busy){ tx_thread_sleep(10);
err=rx_usb_write(port,data,len); if(err!=k_rx_ok){ rx_log_error("USB","Writefailedafterretry:
%d",err); } }

Example - Streaming Write (Chunked):: const uint32_t chunk_size=512; uint32_t offset=0;
while(offset<total_len){ uint32_t remaining=total_len-offset;
uint32_t to_write=(remaining<chunk_size)?remaining:chunk_size;
rx_err_t err=rx_usb_write(port,&data[offset],to_write); if(err==k_rx_ok){ offset+=to_write; }
elseif(err==k_rx_err_busy){ tx_thread_sleep(5); }else{ break; } }

Example:

```
T+0us: rx_usb_write() called by application
T+1us: Data copied to TX ring buffer (~30MB/s memcpy)
T+2us: internal_trigger_tx_if_idle() called
T+3us: rx_usb_cdc_handle_bulk_in() loads USB FIFO
T+5us: rx_usb_write() returns k_rx_ok to application
T+50us: USB host polls endpoint, receives first packet
T+1ms: ISR: BEMP interrupt, send next packet from ring buffer
T+2ms: ISR: BEMP interrupt, send next packet
... (continues until ring buffer empty)
```

See also: rx_usb_flush() Wait for transmission completion, rx_usb_tx_available() Query
free TX buffer space, rx_usb_is_configured() Check if port ready, rx_usb_get_stats()
Query transmission statistics, priv_ring_buffer_write() Internal ring buffer
implementation

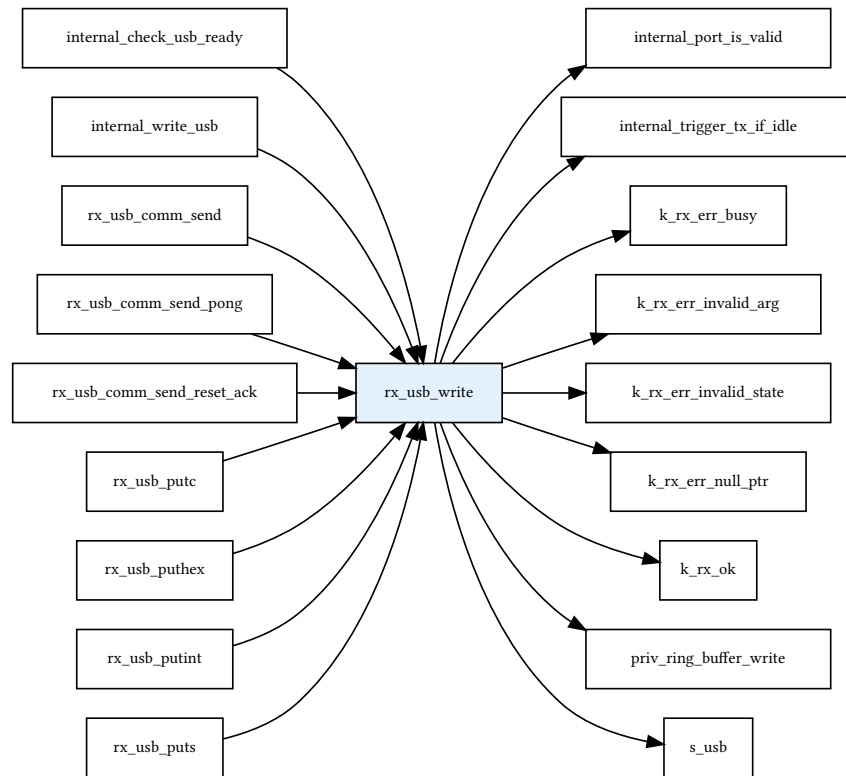


Figure 1159: Call graph for rx_usb_write

Defined in `libs/rx_usb/src/rx_usb.c:1391`

Since Version 1.0.0

—

rx_usb_read

```
rx_err_t rx_usb_read(const rx_usb_port_id_t port, uint8_t *data, const uint32_t
max_len, uint32_t *actual_len)
```

Read data from USB CDC port (receive from host).

Reads data from the specified port's RX ring buffer. This function returns immediately with whatever data is currently buffered - it does not block waiting for data. The actual number of bytes read is returned via the `actual_len` output parameter.

Operation Flow:

1. Validate parameters (port ID, data pointer, `actual_len` pointer)
2. Check driver initialized (does NOT require configured state)
3. Read from RX ring buffer (`priv_ring_buffer_read`, non-blocking)
4. Set `actual_len` (bytes actually read, may be 0 if buffer empty)
5. Return `k_rx_ok` (even if no data available, check `actual_len`)

Reception Timeline:

Ring Buffer Behavior:

- Capacity: Port 0 (1024 bytes), Port 1 (512 bytes), Port 2 (2048 bytes)
- Empty buffer: Returns `k_rx_ok` with `actual_len = 0`
- Partial read: If buffer has 50 bytes and `max_len=100`, reads 50 and returns
- Available data: Query via `rx_usb_rx_available()` to know how much to read

Blocking vs Non-Blocking:

- This function is non-blocking (returns immediately with available data)
- To wait for data, poll `rx_usb_rx_available()` or use callback notification
- Example polling: `while (available == 0) { rx_usb_rx_available(&available); tx_thread_sleep(1); }`

Configured State Requirement:

- Unlike `rx_usb_write()`, this function does NOT require port to be configured
- Rationale: Buffered data may exist from before enumeration completion
- Allows reading residual data during disconnection/reconnection

Thread Safety:

- Per-port safe: Multiple tasks can read from different ports concurrently
- Not multi-reader safe: Only one task should read from each port
- ISR interaction: ISR writes to RX buffer, no locking needed (producer/consumer)

Parameters:

Direction	Name	Description
in	port	Port ID to read from Valid values: <code>k_usb_port_proto</code> (0), <code>k_usb_port_decoded</code> (1), <code>k_usb_port_log</code> (2) Invalid values: Returns <code>k_rx_err_invalid_arg</code>
out	data	Destination buffer for received data Valid range: Pointer to buffer of size <code>max_len</code> NULL handling: Returns <code>k_rx_err_null_ptr</code> Alignment: No alignment requirement (byte-addressable) Modification: Filled with up to <code>max_len</code> bytes from RX buffer
in	max_len	Maximum bytes to read (size of data buffer) Valid range: 0 - 4294967295 Typical: Match buffer capacity (1024, 512, or 2048) Zero handling: <code>max_len=0</code> is valid (no data copied, <code>actual_len=0</code>)
out	actual_len	Pointer to store number of bytes actually read Valid range: NULL not allowed, returns <code>k_rx_err_null_ptr</code> Output range: 0 - <code>max_len</code> Zero output: <code>actual_len=0</code> means RX buffer was empty

Returns: `rx_err_t` Read result

Value	Description
<code>k_rx_ok</code>	Success, <code>actual_len</code> contains bytes read (may be 0)
<code>k_rx_err_invalid_arg</code>	Port ID invalid (port >= <code>k_usb_port_count</code>)
<code>k_rx_err_null_ptr</code>	<code>data</code> or <code>actual_len</code> pointer is nullptr
<code>k_rx_err_invalid_state</code>	Driver not initialized via <code>rx_usb_init()</code>

Pre: Driver must be initialized via rx_usb_init()

Pre: data must point to valid buffer of size max_len

Pre: actual_len must point to valid uint32_t variable

Post: On k_rx_ok, *actual_len contains bytes read (0 if buffer empty)

Post: On k_rx_ok, data buffer filled with up to max_len bytes

Post: Ring buffer read index advanced by *actual_len

Note: This function is non-blocking (returns immediately with available data)

Note: Returns k_rx_ok even if no data available (check actual_len)

Note: Does not require port to be configured (can read buffered data)

Warning: Always check actual_len - may be less than max_len or zero

Warning: Do not assume data received - check actual_len before processing

Performance:: Execution time: 1-3 us @ 240 MHz (ring buffer copy) Throughput: 30 MB/s (ring buffer copy via memcpy) Stack usage: 32 bytes (locals + ring buffer function call)

Example - Basic Read:: constrx_usb_port_id_tport=k_usb_port_proto; uint8_tbuffer[128];
uint32_tbytes_read;

```
rx_err_t err=rx_usb_read(port,buffer,sizeof(buffer),&bytes_read);
```

```
if(err==k_rx_ok){ if(bytes_read>0){ rx_log_debug("USB","Received%ubytes",bytes_read); }  
else{ rx_log_debug("USB","Nodataavailable"); } }
```

Example - Poll for Data:: constuint32_ttimeout_ms=1000; uint32_telapsed_ms=0;
uint32_tavailable=0;

```
while(available==0&&elapsed_ms<timeout_ms){ rx_usb_rx_available(port,&available);  
if(available==0){ tx_thread_sleep(10); elapsed_ms+=10; } }
```

```
if(available>0){ uint8_tbuffer[256]; uint32_tto_read=(available<sizeof(buffer))?available:sizeof(buffer);  
uint32_tbytes_read;
```

```
rx_usb_read(port,buffer,to_read,&bytes_read); }
```

Example - Read with Callback:: voidusb_rx_callback(rx_usb_port_id_tport, rx_usb_event_tevent,
void*context) { if(event==k_usb_event_data_rx)


```

{ TX_EVENT_FLAGS_GROUP*flags=(TX_EVENT_FLAGS_GROUP*)context;
tx_event_flags_set(flags,(1<<port),TX_OR); } }

ULONGactual_flags;
tx_event_flags_get(&usb_flags,0x07,TX_OR_CLEAR,&actual_flags,TX_WAIT_FOREVER);

for(rx_usb_port_id_tport=0;port<k_usb_port_count;port++){ if(actual_flags&(1<<port))
{ uint8_tbuffer[512]; uint32_tbytes_read; rx_usb_read(port,buffer,sizeof(buffer),&bytes_read); } }

Example - Read All Available Data:: uint32_tavailable; rx_usb_rx_available(port,&available);

if(available>0){ uint8_t*buffer=malloc(available); uint32_tbytes_read;

rx_usb_read(port,buffer,available,&bytes_read);

if(bytes_read==available){ }else{ }

free(buffer); }

```

```

Example - Line-Based Read (Scan for Newline):: charline_buffer[128]; uint32_tline_pos=0;

while(line_pos<sizeof(line_buffer)-1){ uint8_tbyte; uint32_tbytes_read;

rx_usb_read(port,&byte,1,&bytes_read);

if(bytes_read==0){ tx_thread_sleep(1); continue; }

line_buffer[line_pos++]=byte;

if(byte=='n'){ line_buffer[line_pos]='0'; rx_log_info("USB","Line:%s",line_buffer); break; } }

```

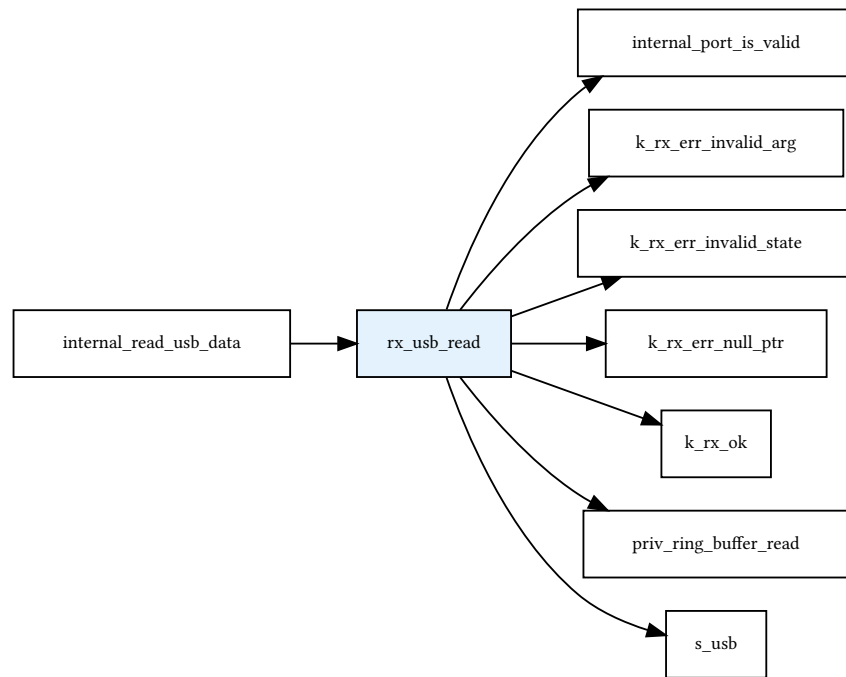
Example:

```

T+0ms:HostsendsdataviaUSBBulkOUT
T+0ms:USB0peripheralreceivesdatainFIFO
T+0ms:ISR:BRDYinterruptfires
T+0ms:ISR:rx_usb_cdc_handle_bulk_out()copiesFIFOtoRXringbuffer
T+0ms:ISR:Invokesportcallback(k_usb_event_data_rx)
T+1ms:Application:rx_usb_read()called
T+1ms:Application:Datacopiedfromringbuffertoapplicationbuffer
T+1ms:rx_usb_read()returnsk_rx_okwithactual_len

```

See also: rx_usb_write() Transmit data to host, rx_usb_rx_available() Query available RX data, rx_usb_set_callback() Register callback for data arrival, priv_ring_buffer_read() Internal ring buffer implementation

Figure 1160: Call graph for `rx_usb_read`

Defined in `libs/rx_usb/src/rx_usb.c:1648`

Since Version 1.0.0

—

rx_usb_rx_available

```
rx_err_t rx_usb_rx_available(const rx_usb_port_id_t port, uint32_t *available)
```

Get number of bytes available to read from RX buffer.

Parameters:

Direction	Name	Description
in	port	Port ID to query
out	available	Pointer to store number of available bytes

Returns: `k_rx_err_null_ptr` if `available` is nullptr

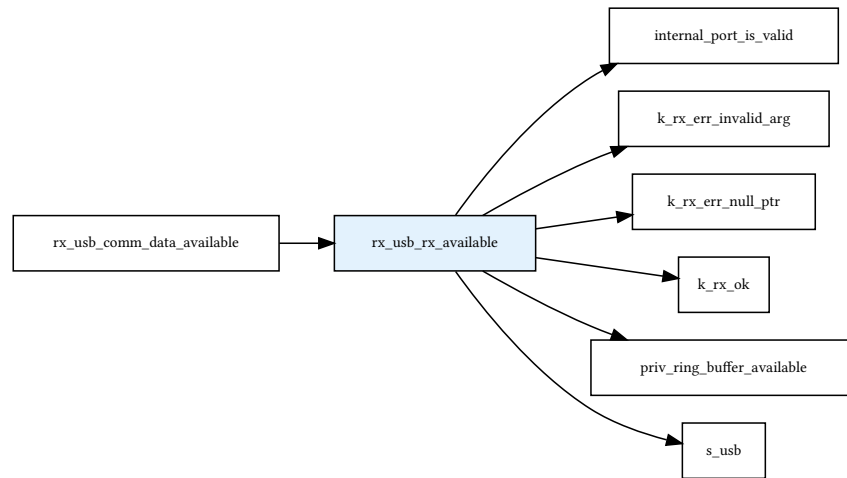


Figure 1161: Call graph for rx_usb_rx_available

Defined in `libs/rx_usb/src/rx_usb.c:1681`

rx_usb_tx_available

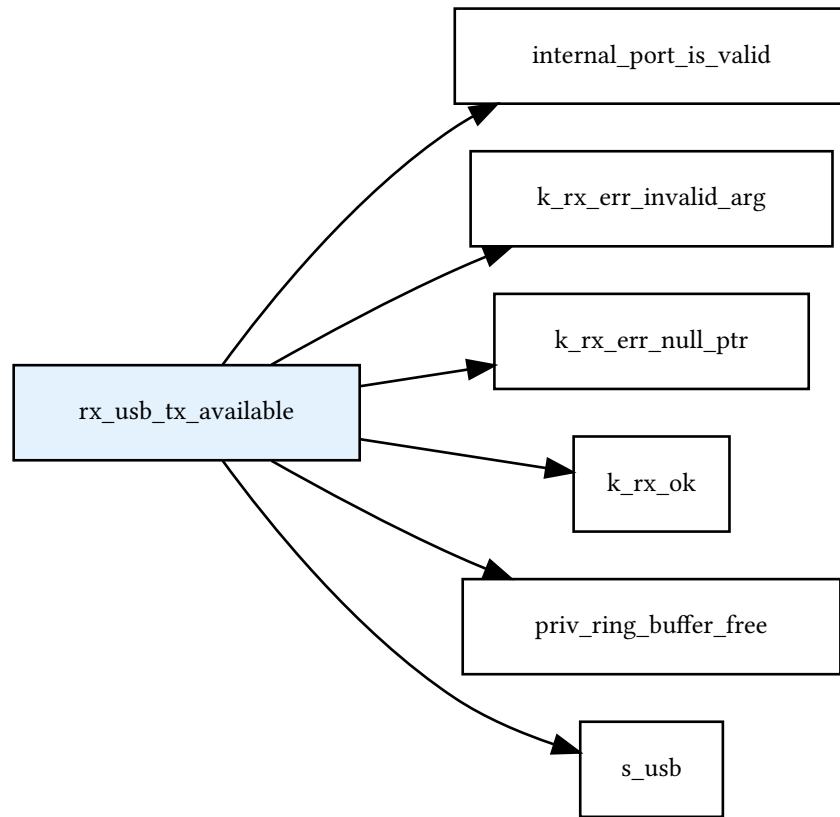
```
rx_err_t rx_usb_tx_available(const rx_usb_port_id_t port, uint32_t *available)
```

Get number of free bytes in TX buffer.

Parameters:

Direction	Name	Description
in	port	Port ID to query
out	available	Pointer to store number of free bytes

Returns: `k_rx_err_null_ptr` if available is nullptr

Figure 1162: Call graph for `rx_usb_tx_available`

Defined in `libs/rx_usb/src/rx_usb.c:1705`

—

rx_usb_flush

```
rx_err_t rx_usb_flush(const rx_usb_port_id_t port, const uint32_t timeout_ms)
```

Flush TX buffer by waiting for transmission to complete.

Blocks until TX buffer is empty or timeout expires. If `timeout_ms` is 0, checks once and returns immediately.

Parameters:

Direction	Name	Description
in	<code>port</code>	Port ID to flush
in	<code>timeout_ms</code>	Timeout in milliseconds (0 = non-blocking check)

Returns: `k_rx_err_timeout` if buffer not empty after timeout

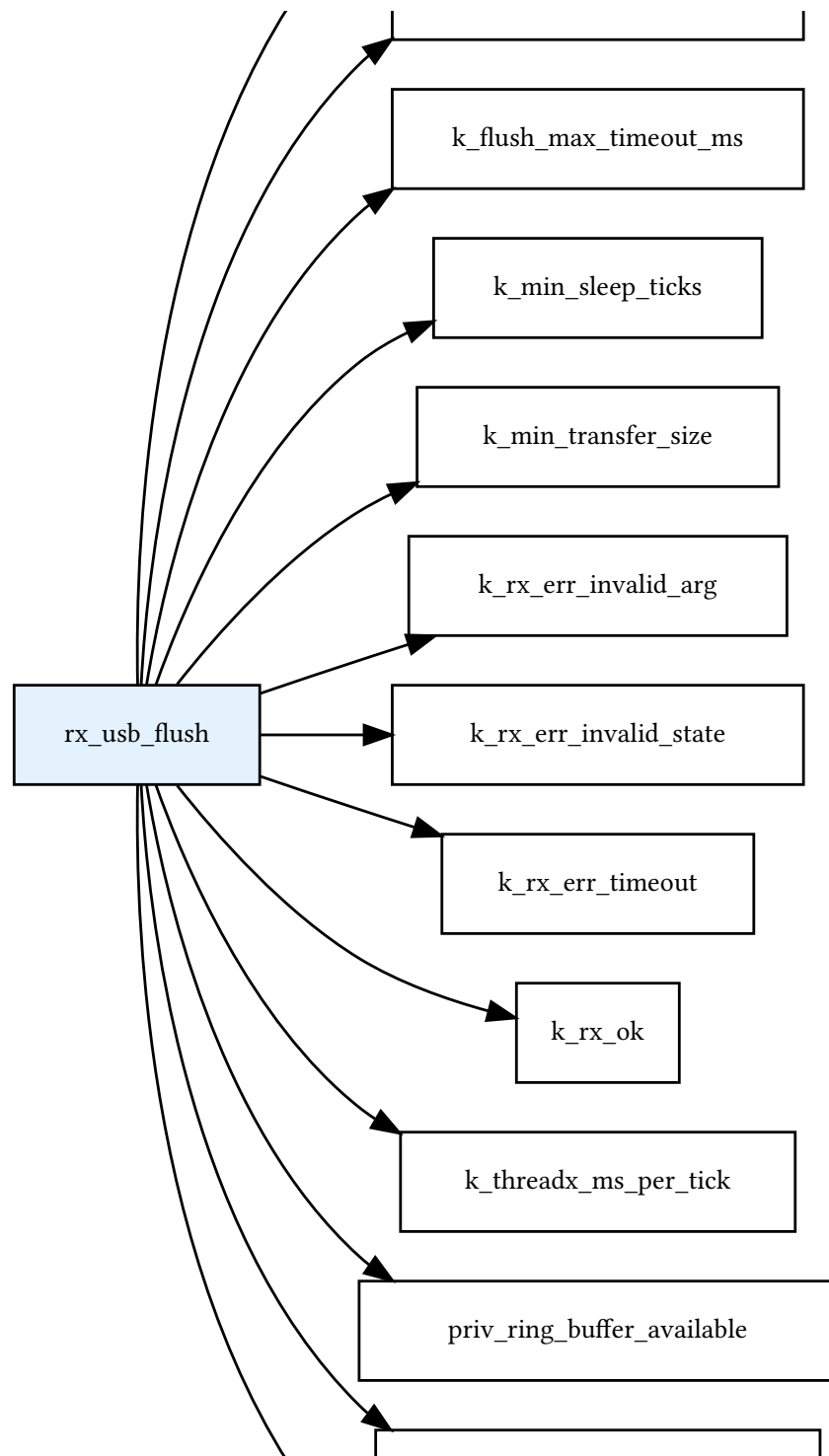


Figure 1163: Call graph for `rx_usb_flush`

Defined in `libs/rx_usb/src/rx_usb.c:1733`

—

rx_usb_set_callback

```
rx_err_t rx_usb_set_callback(const rx_usb_port_id_t port, rx_usb_callback_t
callback, void *ctx)
```

Register callback for USB events on specified port.

Parameters:

Direction	Name	Description
in	port	USB port identifier (k_usb_port_proto or k_usb_port_decoded)
in	callback	Callback function to invoke on USB events (may be NULL to unregister)
in	ctx	User context pointer passed to callback (may be NULL)

Returns: k_rx_err_invalid_arg if port is invalid

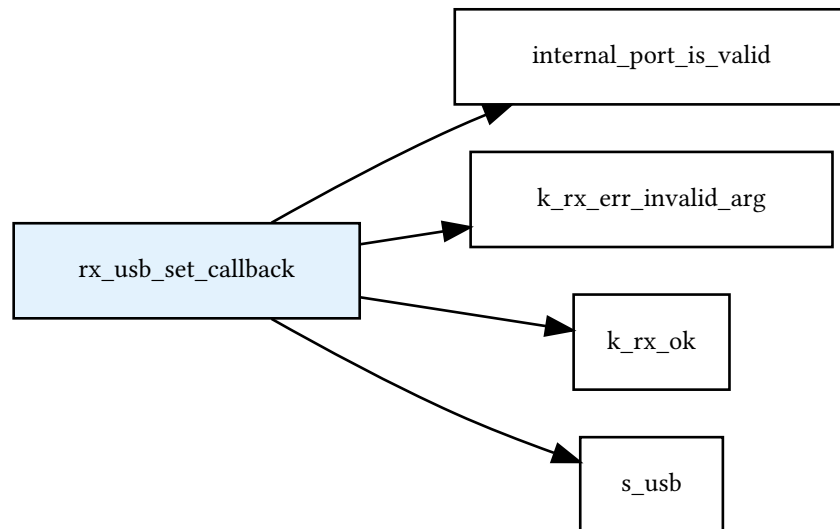


Figure 1164: Call graph for rx_usb_set_callback

Defined in `libs/rx_usb/src/rx_usb.c:1795`

—

rx_usb_get_line_coding

```
rx_err_t rx_usb_get_line_coding(const rx_usb_port_id_t port, rx_usb_line_coding_t
*line_coding)
```

Get CDC line coding for a port.

Returns the current line coding (baud rate, stop bits, parity, data bits) as set by the host via CDC SET_LINE_CODING request.

Parameters:

Direction	Name	Description
in	port	Port ID to query
out	line_coding	Pointer to store line coding

Returns: k_rx_err_null_ptr if line_coding is nullptr

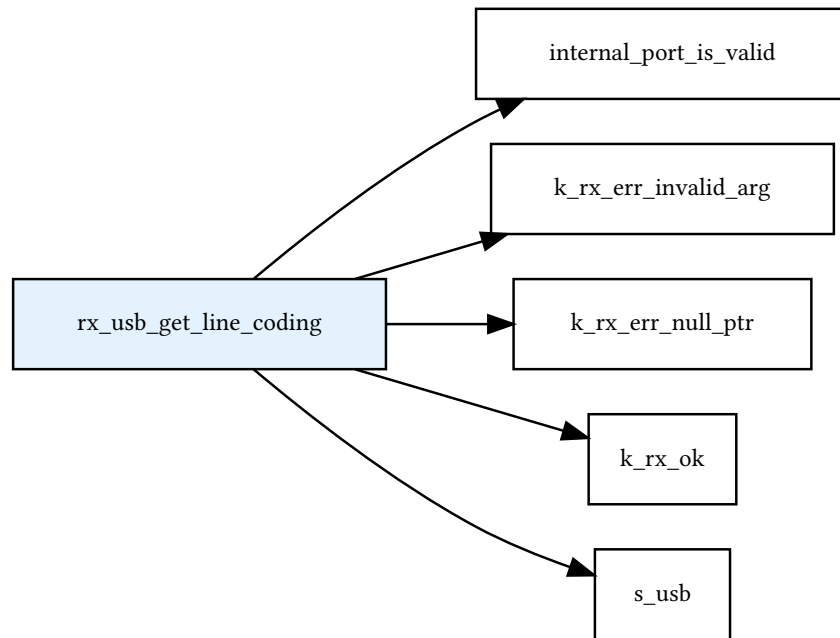


Figure 1165: Call graph for rx_usb_get_line_coding

Defined in `libs/rx_usb/src/rx_usb.c:1819`

—

rx_usb_get_stats

```
rx_err_t rx_usb_get_stats(const rx_usb_port_id_t port, rx_usb_stats_t *stats)
```

Get USB port statistics.

Returns statistics including bytes TX/RX, buffer overruns/underruns, bus resets, and suspend events.

Parameters:

Direction	Name	Description
in	port	Port ID to query

out	stats	Pointer to store statistics
-----	-------	-----------------------------

Returns: k_rx_err_null_ptr if stats is nullptr

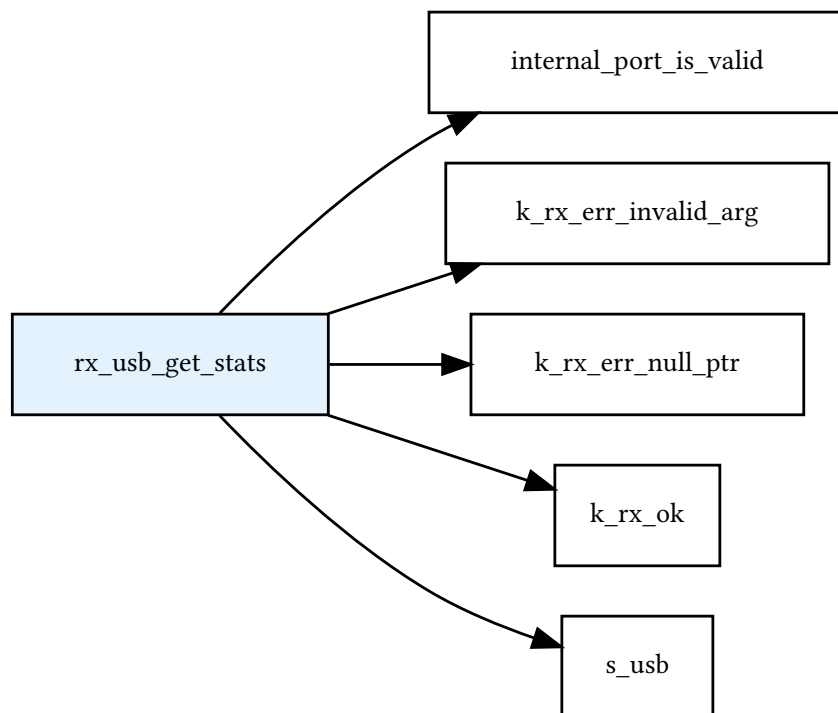


Figure 1166: Call graph for rx_usb_get_stats

Defined in `libs/rx_usb/src/rx_usb.c:1846`

—

rx_usb_reset_stats

```
void rx_usb_reset_stats(const rx_usb_port_id_t port)
```

Reset USB port statistics to zero.

Clears all statistics counters for the specified port.

Parameters:

Direction	Name	Description
in	port	Port ID to reset statistics for

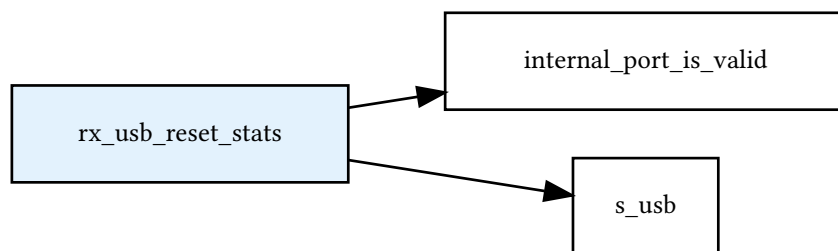


Figure 1167: Call graph for rx_usb_reset_stats

Defined in `libs/rx_usb/src/rx_usb.c:1868`

—

rx_usb_putc

```
rx_err_t rx_usb_putc(const rx_usb_port_id_t port, const char c)
```

Transmit single character to USB port.

Parameters:

Direction	Name	Description
in	port	USB port identifier (k_usb_port_proto or k_usb_port_decoded)
in	c	Character to transmit

Returns: k_rx_err_busy if TX buffer is full

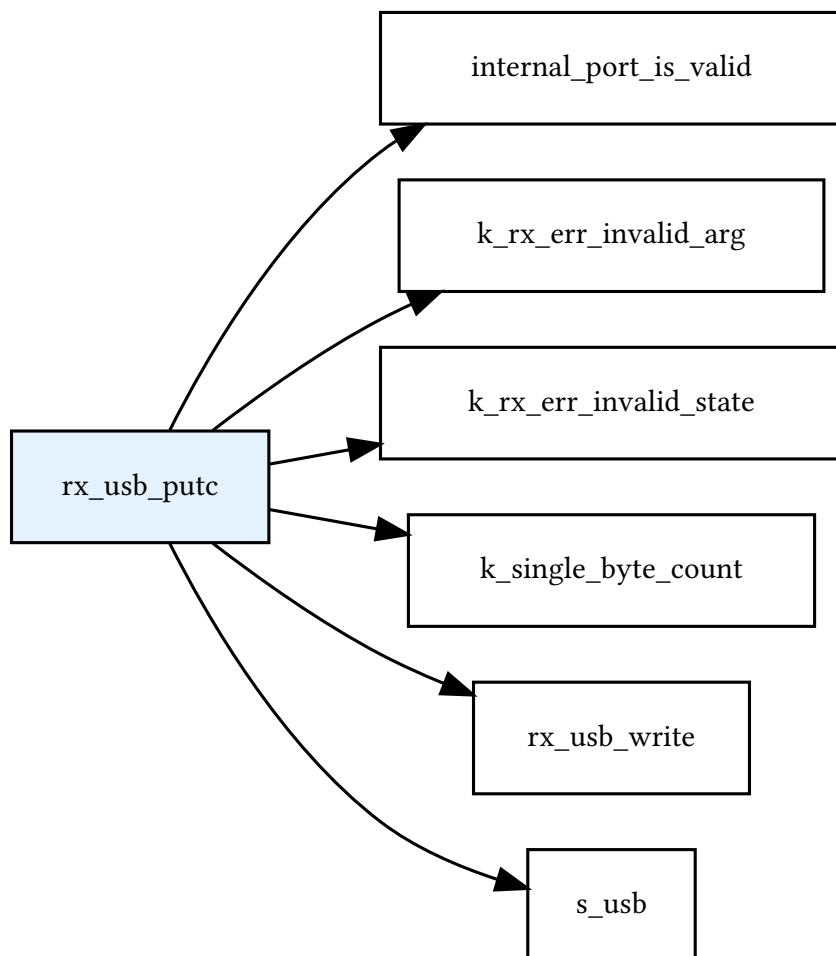


Figure 1168: Call graph for rx_usb_putc

Defined in `libs/rx_usb/src/rx_usb.c:1906`

—

rx_usb_puts

```
rx_err_t rx_usb_puts(const rx_usb_port_id_t port, const char *str)
```

Transmit null-terminated string to USB port.

Parameters:

Direction	Name	Description
in	port	USB port identifier (k_usb_port_proto or k_usb_port_decoded)
in	str	Null-terminated string to transmit (must not be NULL)

Returns: k_rx_err_busy if TX buffer is full

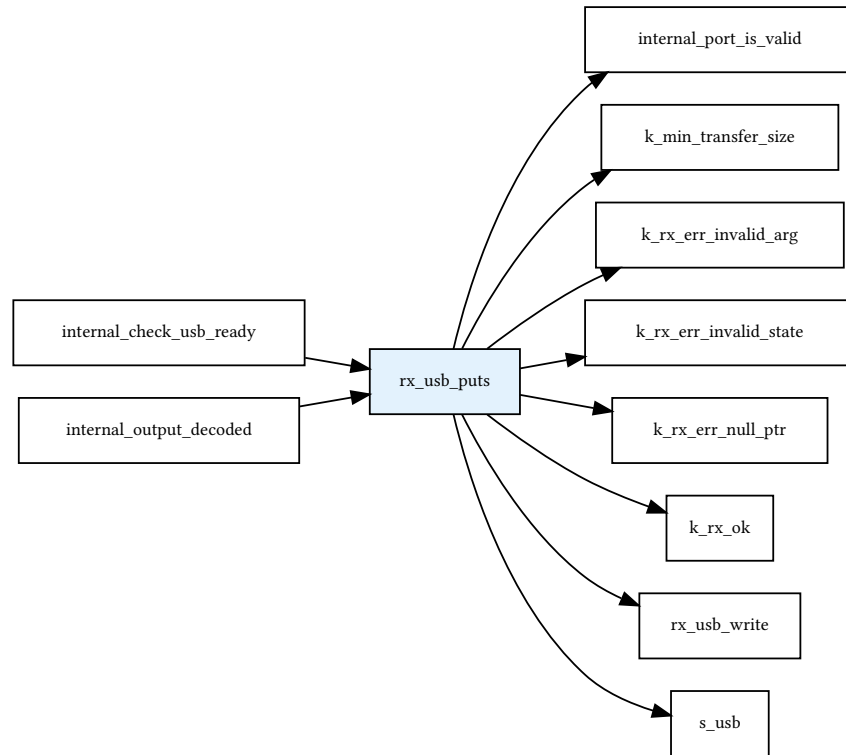


Figure 1169: Call graph for rx_usb_puts

Defined in `libs/rx_usb/src/rx_usb.c:1934`

—

rx_usb_putint

```
rx_err_t rx_usb_putint(const rx_usb_port_id_t port, int32_t value)
```

Write signed integer as decimal string to USB port.

Converts the integer to a decimal ASCII string and writes it to the USB port. Handles negative numbers and INT32_MIN correctly.

Parameters:

Direction	Name	Description
in	port	Port ID to write to
in	value	Signed 32-bit integer to write

Returns: k_rx_err_busy if TX buffer is full

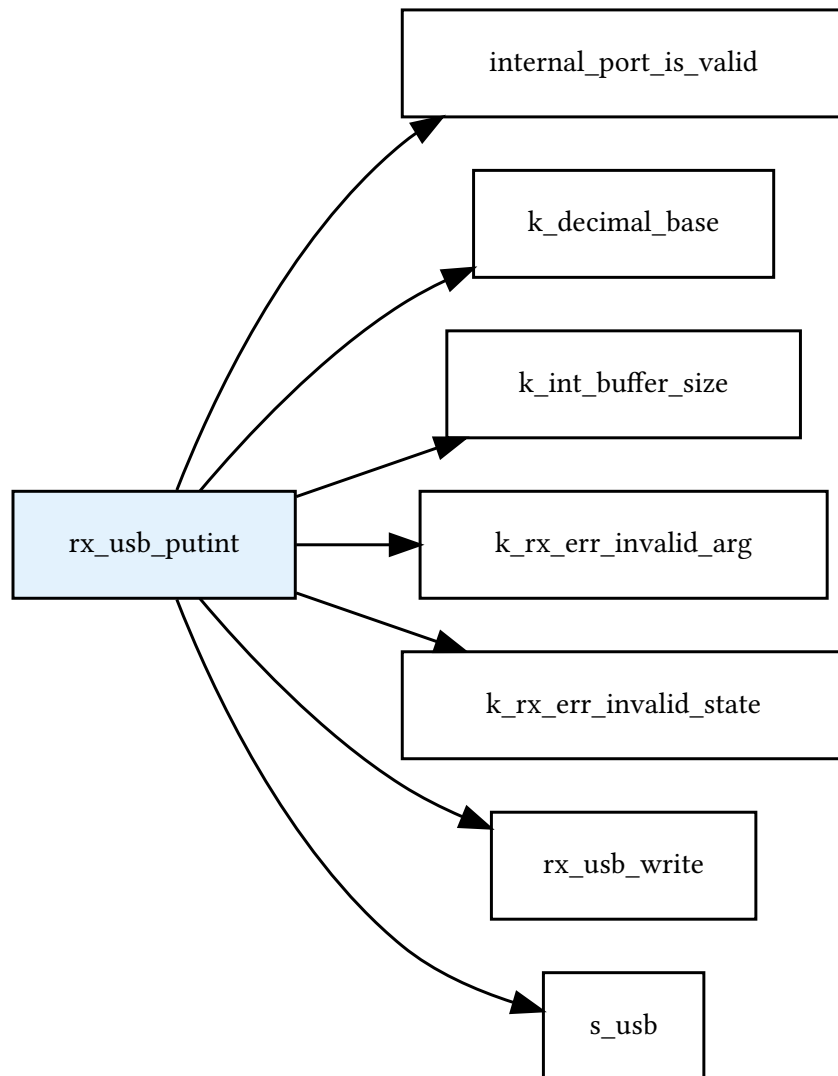


Figure 1170: Call graph for rx_usb_putint

Defined in `libs/rx_usb/src/rx_usb.c:1983`

—

rx_usb_puthex

```
rx_err_t rx_usb_puthex(const rx_usb_port_id_t port, uint32_t value, uint8_t digits)
```

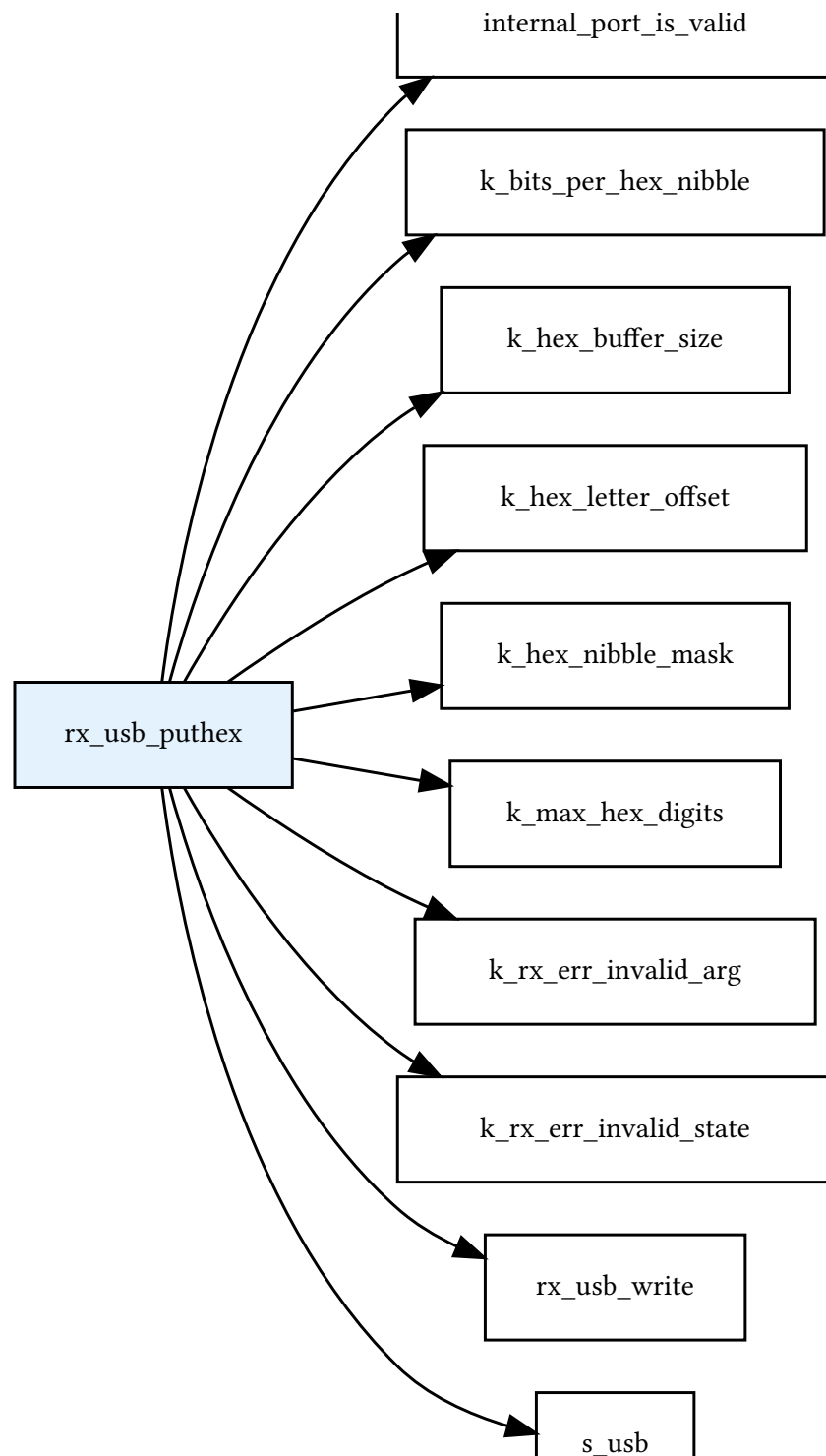
Write unsigned integer as hexadecimal string to USB port.

Converts the value to a zero-padded hexadecimal ASCII string (uppercase A-F) and writes it to the USB port. Digits are clamped to range [1, 8].

Parameters:

Direction	Name	Description
in	port	Port ID to write to
in	value	Unsigned 32-bit integer to write
in	digits	Number of hex digits to output (1-8, zero-padded)

Returns: k_rx_err_busy if TX buffer is full

Figure 1171: Call graph for `rx_usb_puthex`

Defined in `libs/rx_usb/src/rx_usb.c:2051`

—

rx_usb_set_state

```
void rx_usb_set_state(const rx_usb_state_t state)
```

Set USB device state (called by hardware/CDC modules).

Set global USB device state.

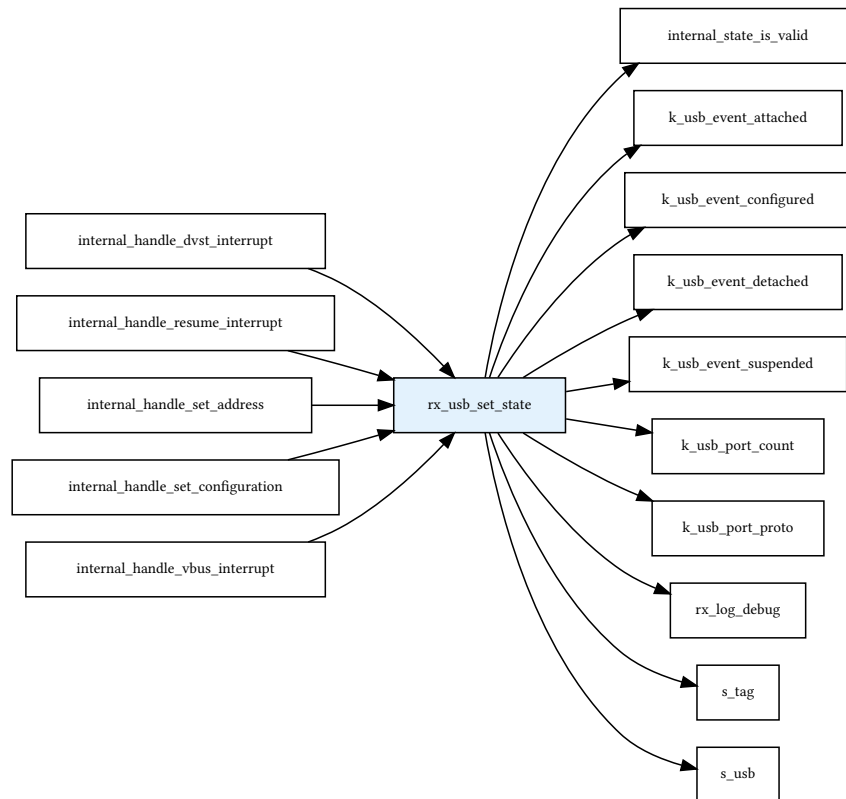


Figure 1172: Call graph for `rx_usb_set_state`

Defined in `libs/rx_usb/src/rx_usb.c:2097`

—

rx_usb_set_line_coding

```
void rx_usb_set_line_coding(const rx_usb_port_id_t port, const
rx_usb_line_coding_t *coding)
```

Set line coding for a port (called by CDC module on SET_LINE_CODING request).

Set CDC line coding for a port.

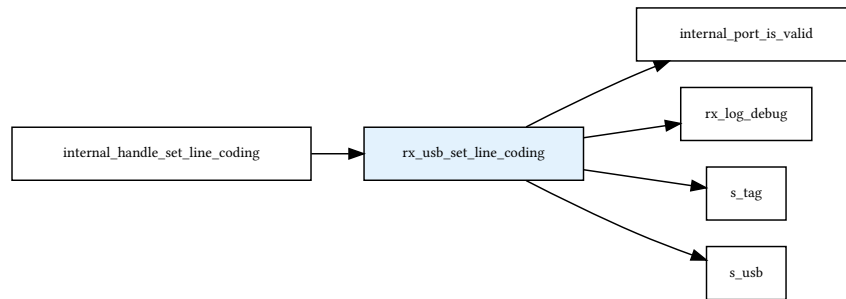


Figure 1173: Call graph for rx_usb_set_line_coding

Defined in `libs/rx_usb/src/rx_usb.c:2160`

—

rx_usb_rx_push

```
uint32_t rx_usb_rx_push(const rx_usb_port_id_t port, const uint8_t *data, const
uint32_t len)
```

Add received data to a port's RX buffer (called by CDC/ISR).

Push data into a port's RX ring buffer.

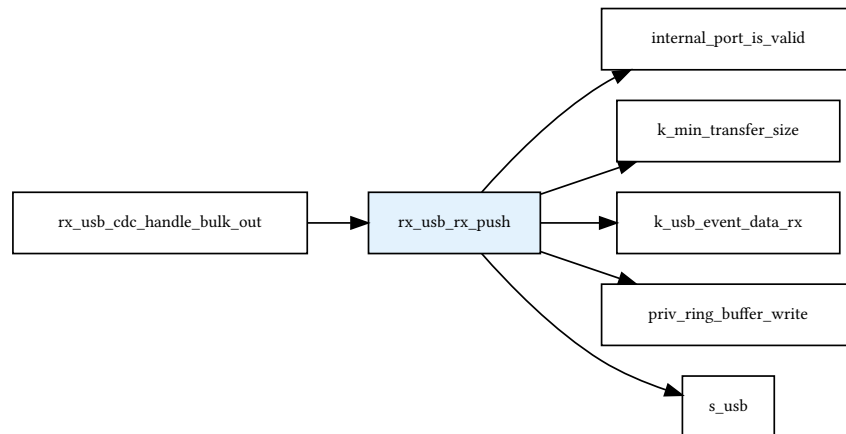


Figure 1174: Call graph for rx_usb_rx_push

Defined in `libs/rx_usb/src/rx_usb.c:2173`

—

rx_usb_tx_pop

```
uint32_t rx_usb_tx_pop(const rx_usb_port_id_t port, uint8_t *data, const uint32_t
max_len)
```

Get data from a port's TX buffer for transmission (called by CDC/ISR).

Pop data from a port's TX ring buffer.

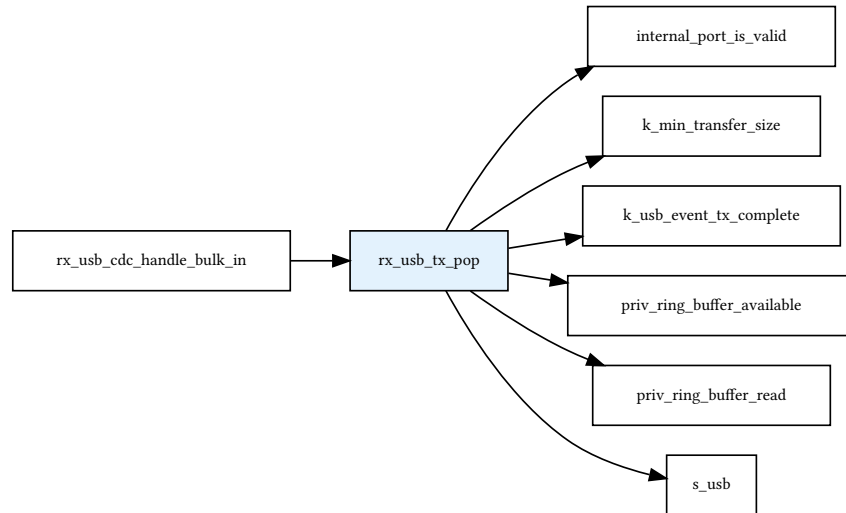


Figure 1175: Call graph for rx_usb_tx_pop

Defined in `libs/rx_usb/src/rx_usb.c:2198`

—

rx_usb_count_bus_reset

```
void rx_usb_count_bus_reset(void)
```

Increment bus reset counter.

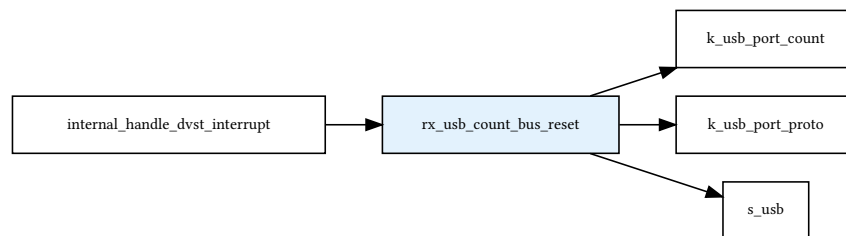


Figure 1176: Call graph for rx_usb_count_bus_reset

Defined in `libs/rx_usb/src/rx_usb.c:2219`

—

rx_usb_count_suspend

```
void rx_usb_count_suspend(void)
```

Increment suspend counter.

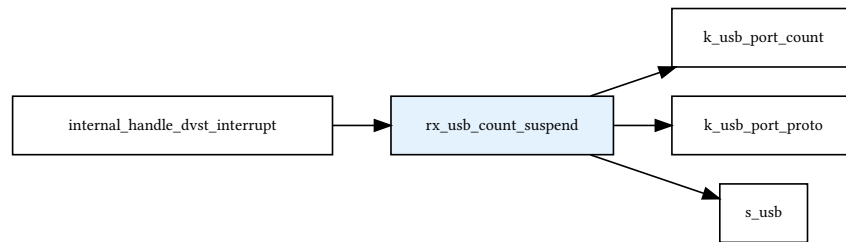


Figure 1177: Call graph for rx_usb_count_suspend

Defined in `libs/rx_usb/src/rx_usb.c:2229`

rx_usb_get_port_config

```
const rx_usb_port_hw_config_t * rx_usb_get_port_config(const rx_usb_port_id_t port)
```

Get port configuration (for CDC module).

Get port hardware configuration (shared-internal).

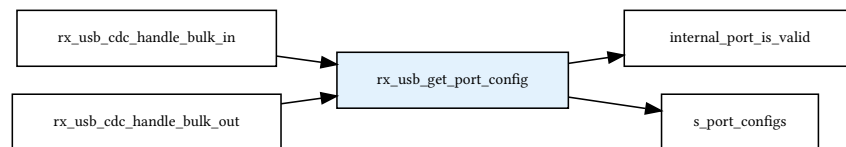


Figure 1178: Call graph for rx_usb_get_port_config

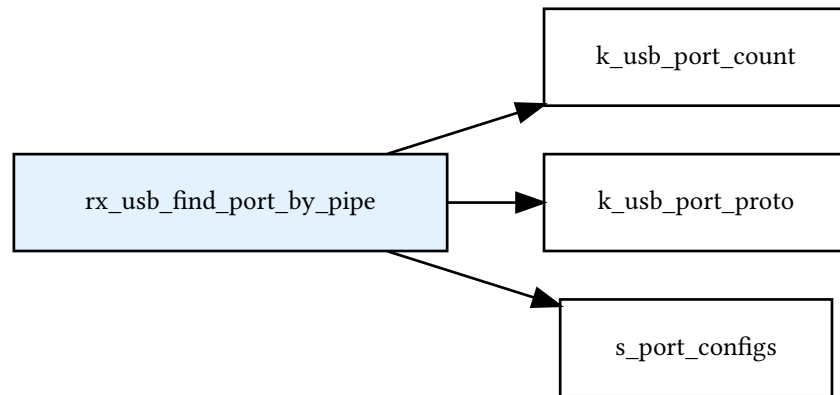
Defined in `libs/rx_usb/src/rx_usb.c:2239`

rx_usb_find_port_by_pipe

```
rx_usb_port_id_t rx_usb_find_port_by_pipe(const uint8_t pipe)
```

Find port by pipe number (for ISR routing).

Find port by pipe number.

Figure 1179: Call graph for `rx_usb_find_port_by_pipe`

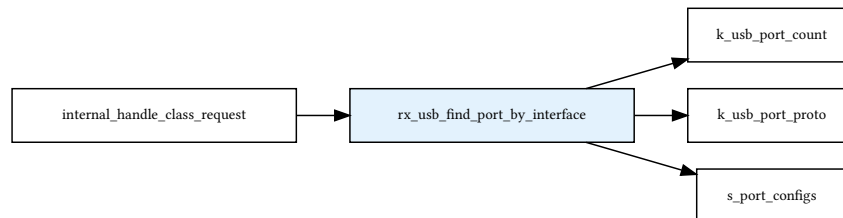
Defined in `libs/rx_usb/src/rx_usb.c:2250`

rx_usb_find_port_by_interface

```
rx_usb_port_id_t rx_usb_find_port_by_interface(const uint8_t interface)
```

Find port by interface number (for CDC class requests).

Find port by interface number.

Figure 1180: Call graph for `rx_usb_find_port_by_interface`

Defined in `libs/rx_usb/src/rx_usb.c:2264`

rx_usb_invoke_callback

```
void rx_usb_invoke_callback(const rx_usb_port_id_t port, const rx_usb_event_t event)
```

Invoke callback for a specific port and event.

Invoke user callback for a port event.

Called by ISR and CDC handlers to notify the application of USB events.

Parameters:

Direction	Name	Description
in	port	Port ID
in	event	Event type

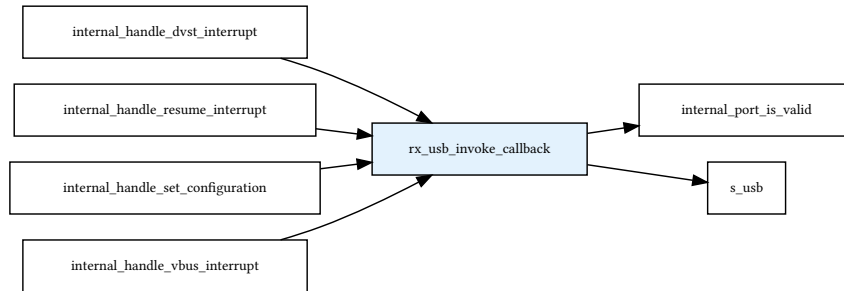


Figure 1181: Call graph for rx_usb_invoke_callback

Defined in `libs/rx_usb/src/rx_usb.c:2283`

—

rx_usb_cdc.c

USB CDC-ACM Composite Device Implementation for RX72N Multi-Port Serial.

Includes:

- `<string.h>`
- `"rx72n_regs.h"`
- `"rx_check.h"`
- `"rx_log.h"`
- `"rx_usb.h"`
- `"rx_usb_endpoints.h"`
- `"rx_usb_internal.h"`

usb_descriptor_defaults_t

USB Descriptor Field Default Values.

Value	Name	Description
= 0x00	k_usb_subclass_none	No subclass
= 0x00	k_usb_protocol_none	No protocol
= 6	k_usb_num_interfaces_composite	Total interfaces: 3 CDC x 2 interfaces each
= 1	k_usb_config_value_default	Default configuration value
= 0	k_usb_alternate_setting_default	Default alternate setting
= 0	k_usb_string_index_none	No string descriptor
= 1	k_usb_num_configurations	Number of configurations
= 1	k_usb_num_endpoints_control	Control interface has 1 endpoint (interrupt)
= 2	k_usb_num_endpoints_data	Data interface has 2 endpoints (bulk in/out)
= 0x00	k_cdc_call_mgmt_cap_none	No call management capabilities

= 0	k_min_transfer_size	Minimum data transfer size (no data)
-----	---------------------	--------------------------------------

—

usb_descriptor_type_t

USB Descriptor Types.

Value	Name	Description
= 0x01	k_usb_desc_type_device	Device descriptor
= 0x02	k_usb_desc_type_configuration	Configuration descriptor
= 0x03	k_usb_desc_type_string	String descriptor
= 0x04	k_usb_desc_type_interface	Interface descriptor
= 0x05	k_usb_desc_type_endpoint	Endpoint descriptor
= 0x0B	k_usb_desc_type_interface_association	Interface Association Descriptor
= 0x24	k_usb_desc_type_cs_interface	Class-specific interface descriptor

—

usb_class_code_t

USB Class Codes.

Value	Name	Description
= 0xEF	k_usb_class_misc	Miscellaneous Device Class (for IAD)
= 0x02	k_usb_class_cdc	Communications Device Class
= 0x0A	k_usb_class_cdc_data	CDC Data Class
= 0x02	k_usb_subclass_common	Common Class subclass (for IAD)
= 0x02	k_usb_subclass_acm	Abstract Control Model subclass
= 0x01	k_usb_protocol_iad	Interface Association Descriptor protocol
= 0x01	k_usb_protocol_at	AT command protocol

—

cdc_request_code_t

CDC Class Request Codes.

Value	Name	Description
= 0x20	k_cdc_set_line_coding	Set line coding (baud rate, parity, etc.)
= 0x21	k_cdc_get_line_coding	Get current line coding
= 0x22	k_cdc_set_control_line_state	Set control line state (DTR/RTS)

—

usb_request_code_t

USB Standard Request Codes.

Value	Name	Description
= 0x00	k_usb_req_get_status	Get device/interface/endpoint status
= 0x01	k_usb_req_clear_feature	Clear feature
= 0x03	k_usb_req_set_feature	Set feature
= 0x05	k_usb_req_set_address	Set device address
= 0x06	k_usb_req_get_descriptor	Get descriptor
= 0x07	k_usb_req_set_descriptor	Set descriptor
= 0x08	k_usb_req_get_configuration	Get configuration
= 0x09	k_usb_req_set_configuration	Set configuration
= 0x0A	k_usb_req_get_interface	Get interface
= 0x0B	k_usb_req_set_interface	Set interface

—

usb_device_id_t

Vendor and Product IDs.

Value	Name	Description
= 0x045B	k_usb_vid	Vendor ID: Renesas Electronics (test VID)
= 0x0235	k_usb_pid	Product ID: CDC Composite device (test PID, changed from 0x0234)

—

usb_version_t

USB Version Numbers (BCD format).

Value	Name	Description
= 0x0200	k_usb_version_2_0	USB 2.0 specification
= 0x0110	k_usb_version_1_1	USB 1.1 specification (for CDC)
= 0x0100	k_device_version	Device release version 1.0
= 0x0409	k_usb_langid_en_us	Language ID: English (United States)

—

cdc_descriptor_subtype_t

CDC Functional Descriptor Subtypes.

Value	Name	Description
= 0x00	k_cdc_subtype_header	Header functional descriptor
= 0x01	k_cdc_subtype_call_management	Call management functional descriptor
= 0x02	k_cdc_subtype_acm	ACM functional descriptor
= 0x06	k_cdc_subtype_union	Union functional descriptor

—

usb_control_endpoint_t

USB Control Endpoint Addresses.

Value	Name	Description
= 0x00	k_usb_ep0_out	Control endpoint 0 OUT
= 0x80	k_usb_ep0_in	Control endpoint 0 IN

—

usb_endpoint_type_t

USB Endpoint Transfer Types (bmAttributes).

Value	Name	Description
= 0x00	k_usb_ep_type_control	Control transfer
= 0x01	k_usb_ep_type_isochronous	Isochronous transfer
= 0x02	k_usb_ep_type_bulk	Bulk transfer
= 0x03	k_usb_ep_type_interrupt	Interrupt transfer

—

usb_config_attributes_t

USB Configuration Attributes.

Value	Name	Description
= 0x80	k_usb_cfg_attr_bus_powered	Bus-powered device
= 0xC0	k_usb_cfg_attr_self_powered	Self-powered device

—

usb_max_power_t

USB Power Consumption (in 2mA units).

Value	Name	Description
= 50	k_usb_max_power_100ma	100mA (50 * 2mA)
= 250	k_usb_max_power_500ma	500mA (250 * 2mA)

—

cdc_acm_capabilities_t

CDC Capabilities.

Value	Name	Description
= 0x02	k_cdc_acm_cap_line_coding	Supports SET/GET_LINE_CODING and serial state

—

usb_string_index_t

USB Descriptor Field Indices.

Value	Name	Description
= 0	k_usb_string_index_langid	Language ID string index
= 1	k_usb_string_index_manufacturer	Manufacturer string index
= 2	k_usb_string_index_product	Product string index
= 3	k_usb_string_index_serial_number	Serial number string index

—

usb_packet_size_t

USB Endpoint Packet Sizes.

Value	Name	Description
= 64	k_usb_ep0_packet_size	Control endpoint max packet size
= 64	k_usb_bulk_packet_size	Bulk endpoint max packet size (Full-Speed)
= 8	k_usb_interrupt_packet_size	Interrupt endpoint max packet size

—

usb_polling_interval_t

USB Polling Intervals.

Value	Name	Description
= 0	k_usb_bulk_interval	Bulk endpoints don't use polling
= 10	k_usb_interrupt_interval	Interrupt endpoint polling interval (10ms)

—

usb_string_size_t

USB String Descriptor Sizes.

Value	Name	Description
= 2	k_usb_string_header_size	bLength + bDescriptorType
= 2	k_usb_string_char_size	UTF-16LE character size (2 bytes)
= 1	k_usb_string_langid_chars	Language ID is 1 uint16_t
= 7	k_usb_string_mfr_chars	"Renesas" = 7 characters
= 14	k_usb_string_product_chars	"STAR RX72N CDC" = 14 characters
= 8	k_usb_string_serial_chars	"00000001" = 8 characters

—

bit_shift_t

Bit Shift Values for Multi-Byte Fields.

Value	Name	Description
= 0	k_bit_shift_byte_0	Shift for byte 0 (LSB)

= 8	k_bit_shift_byte_1	Shift for byte 1
= 16	k_bit_shift_byte_2	Shift for byte 2
= 24	k_bit_shift_byte_3	Shift for byte 3 (MSB for 32-bit)

—

byte_mask_t

Byte Masks.

Value	Name	Description
= 0xFF	k_byte_mask	Mask for extracting a single byte
= 0x7F	k_usb_address_mask	USB address mask (7 bits, 0-127)

—

usb_bit_constants_t

Bit Manipulation Constants.

Value	Name	Description
= 1	k_usb_bit_mask	Single bit mask for shift operations (interrupt enable/disable)

—

usb_control_line_constants_t

USB Control Line State Constants.

Value	Name	Description
= 0	k_usb_control_line_cleared	Control line state cleared (DTR/RTS both off)

—

usb_request_type_mask_t

USB Request Type Field Masks (bmRequestType).

Value	Name	Description
= 0x60	k_usb_req_type_mask	Mask for bits 5-6 (request type)
= 0x00	k_usb_req_type_standard	Standard request
= 0x20	k_usb_req_type_class	Class-specific request
= 0x40	k_usb_req_type_vendor	Vendor-specific request

—

cdc_line_coding_index_t

CDC Line Coding Structure Byte Indices.

Value	Name	Description
= 0	k_line_coding_baud_rate_byte_0	Baud rate byte 0 (LSB)

= 1	k_line_coding_baud_rate_byte_1	Baud rate byte 1
= 2	k_line_coding_baud_rate_byte_2	Baud rate byte 2
= 3	k_line_coding_baud_rate_byte_3	Baud rate byte 3 (MSB)
= 4	k_line_coding_stop_bits_index	Stop bits byte index
= 5	k_line_coding_parity_index	Parity byte index
= 6	k_line_coding_data_bits_index	Data bits byte index
= 7	k_line_coding_size	Total line coding structure size

—

usb_pipe_number_t

USB Pipe Numbers for multi-port configuration.

Value	Name	Description
= 0	k_usb_pipe_dcp	Default Control Pipe
= 1	k_usb_port0_pipe_bulk_in	Port 0: Bulk IN pipe
= 2	k_usb_port0_pipe_bulk_out	Port 0: Bulk OUT pipe
= 3	k_usb_port0_pipe_int_in	Port 0: Interrupt IN pipe
= 4	k_usb_port1_pipe_bulk_in	Port 1: Bulk IN pipe
= 5	k_usb_port1_pipe_bulk_out	Port 1: Bulk OUT pipe
= 6	k_usb_port1_pipe_int_in	Port 1: Interrupt IN pipe
= 7	k_usb_port2_pipe_bulk_in	Port 2: Bulk IN pipe
= 8	k_usb_port2_pipe_bulk_out	Port 2: Bulk OUT pipe
= 9	k_usb_port2_pipe_int_in	Port 2: Interrupt IN pipe

—

usb_config_value_t

USB Configuration Values.

Value	Name	Description
= 0	k_usb_config_unconfigured	Device not configured
= 1	k_usb_config_value_1	Configuration 1

—

iad_constants_t

IAD descriptor constants.

Value	Name	Description
= 2	k_iad_interface_count	Each CDC function has 2 interfaces
= k_usb_class_cdc	k_iad_function_class	CDC class
= k_usb_subclass_acm	k_iad_function_subclass	ACM subclass

= k_usb_protocol_at	k_iad_function_protocol	AT command protocol
---------------------	-------------------------	---------------------

—

composite_config_size_t

Expected total configuration descriptor size.

Value	Name	Description
= 207	k_composite_config_size	

—

s_tag

```
const char* s_tag
```

—

s_device_desc

```
const usb_device_descriptor_t s_device_desc
```

Device Descriptor - Composite Device with IAD support.

—

s_config_desc

```
const usb_composite_config_descriptor_t s_config_desc
```

Configuration Descriptor Instance.

—

length

```
uint8_t length
```

—

descriptor_type

```
uint8_t descriptor_type
```

—

langid

```
uint16_t langid
```

—

s_string_langid

```
const struct @134333355310322200204036303054067355376065172022 s_string_langid
```

String Descriptor 0: Language ID (English US).

—

string

```
uint16_t string[k_usb_string_serial_chars]
```

—

s_string_manufacturer

```
const struct @221301111046113066167373310006321114167153030020 s_string_manufacturer
```

String Descriptor 1: Manufacturer ("Renesas").

—

s_string_product

```
const struct @172275054327074270375303002012173075035072202315 s_string_product
```

String Descriptor 2: Product ("STAR RX72N CDC").

—

s_string_serial

```
const struct @3132450570562206052322053265022171313315133333 s_string_serial
```

String Descriptor 3: Serial Number ("00000001").

—

s_cdc_initialized

```
bool s_cdc_initialized
```

—

s_line_coding

```
rx_usb_line_coding_t s_line_coding[k_usb_port_count]
```

—

s_control_line_state

```
uint16_t s_control_line_state[k_usb_port_count]
```

—

internal_send_descriptor

```
static void internal_send_descriptor(const uint8_t *desc, uint16_t desc_len,
uint16_t requested_len)
```

Send descriptor in response to GET_DESCRIPTOR request.

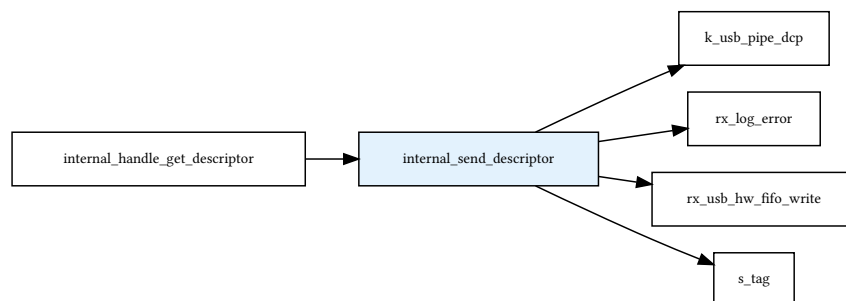


Figure 1182: Call graph for `internal_send_descriptor`

_Defined in `libs/rx_usb/src/rx_usb_cdc.c:1481_`

—

internal_handle_get_descriptor

```
static void internal_handle_get_descriptor(const uint16_t usb_value, const
uint16_t usb_length)
```

Handle GET_DESCRIPTOR request.

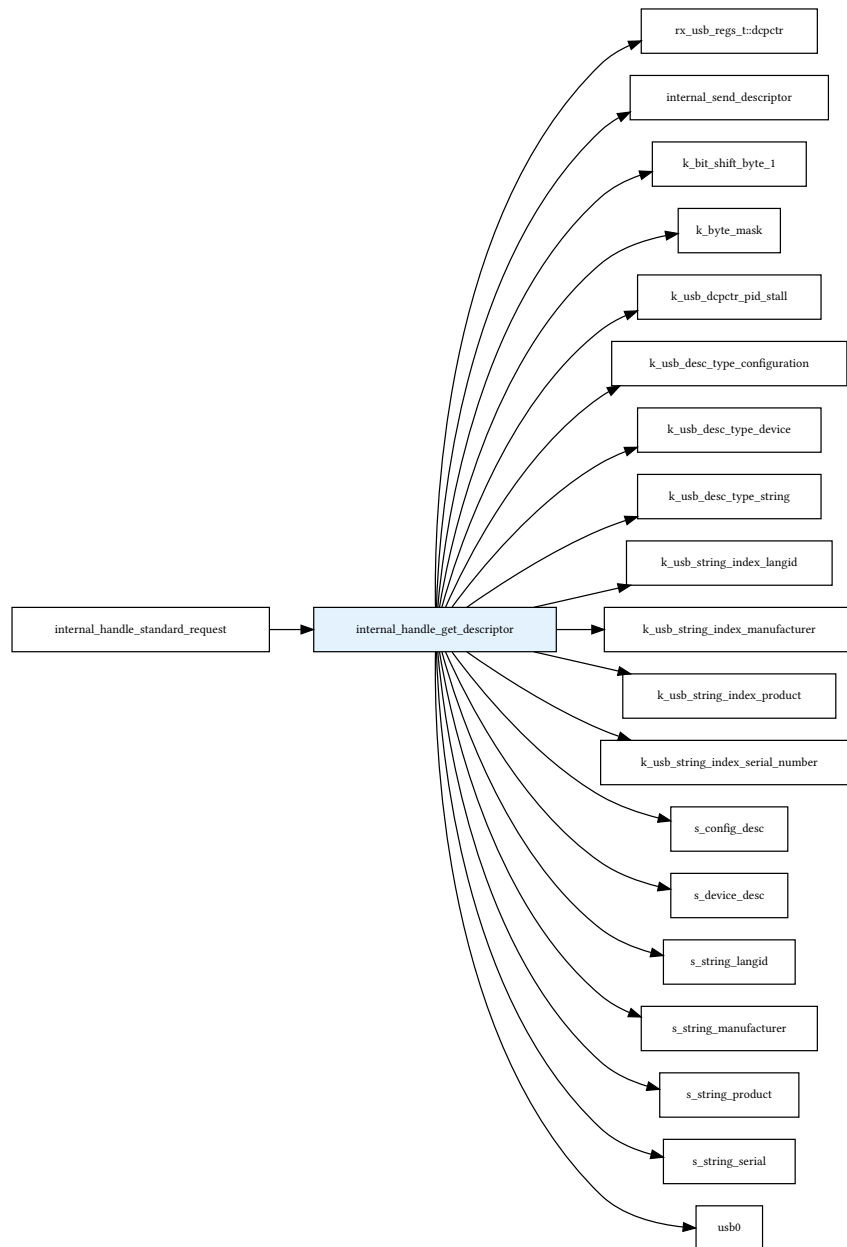


Figure 1183: Call graph for internal_handle_get_descriptor

_Defined in libs/rx_usb/src/rx_usb_cdc.c:1494_

internal_handle_set_address

```
static void internal_handle_set_address(const uint16_t usb_value)
```

Handle SET_ADDRESS request.

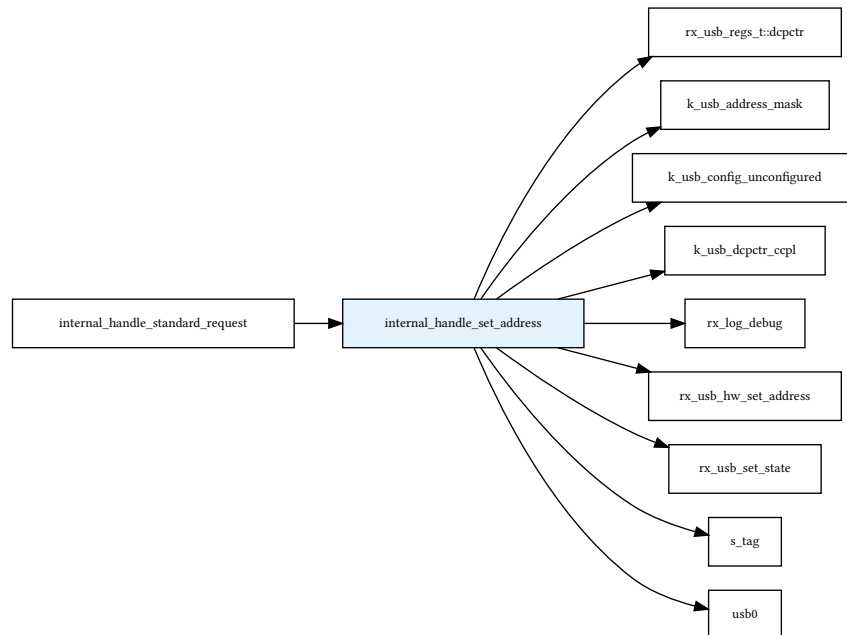


Figure 1184: Call graph for `internal_handle_set_address`

_Defined in `libs/rx_usb/src/rx_usb_cdc.c:1544_`

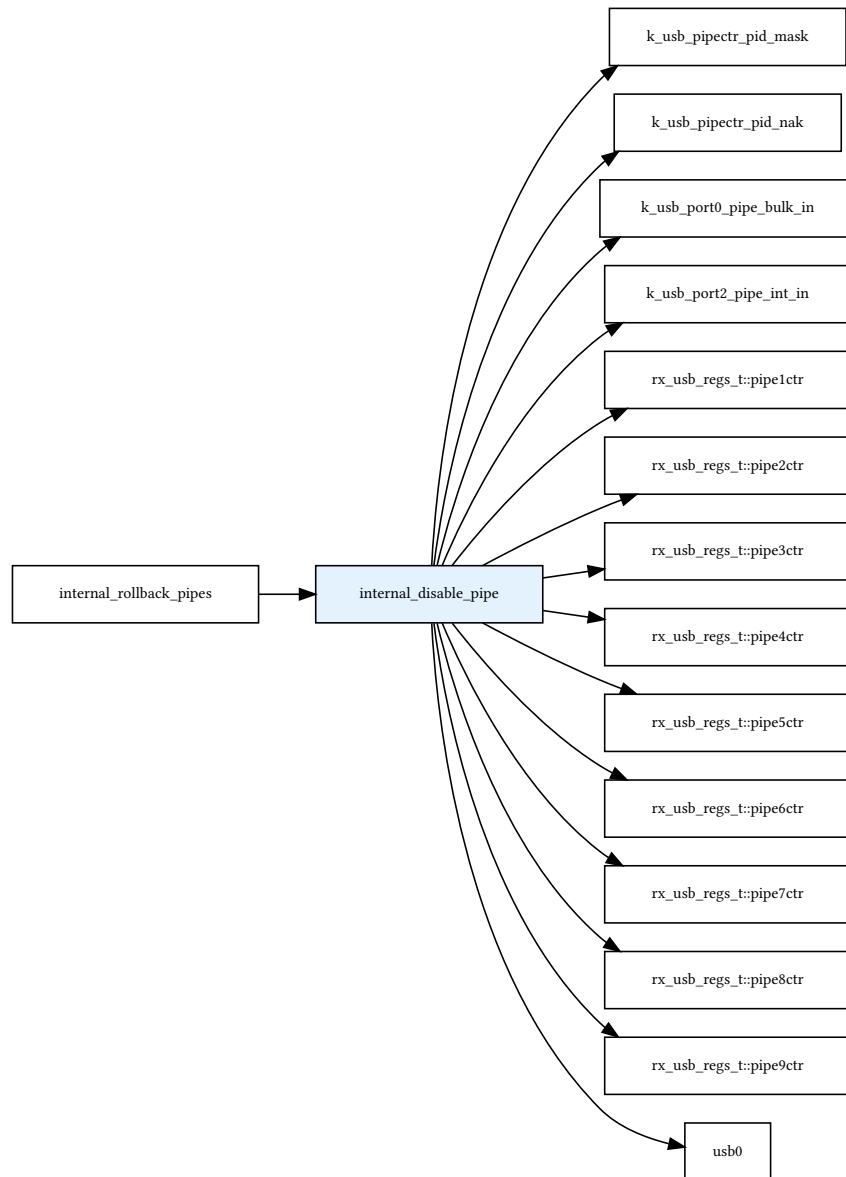
internal_disable_pipe

```
static void internal_disable_pipe(uint8_t pipe)
```

Disable a single USB pipe by setting PID to NAK.

Parameters:

Direction	Name	Description
in	pipe	Pipe number (1-9) to disable

Figure 1185: Call graph for `internal_disable_pipe`

_Defined in `libs/rx_usb/src/rx_usb_cdc.c:1566_`

`internal_rollback_pipes`

```
static void internal_rollback_pipes(uint8_t failed_pipe)
```

Rollback configured pipes on configuration failure.

Disables all pipes from the first configured pipe up to (but not including) the pipe that failed configuration.

Parameters:

Direction	Name	Description
in	failed_pipe	The pipe number that failed (pipes 1 to failed_pipe-1 are disabled)

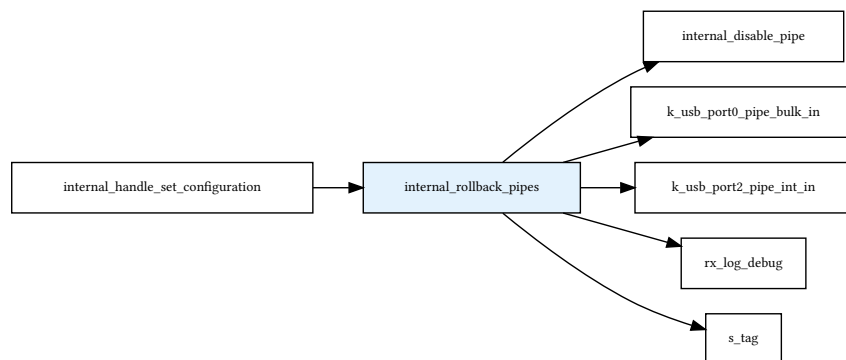


Figure 1186: Call graph for internal_rollback_pipes

_Defined in `libs/rx_usb/src/rx_usb_cdc.c:1596_`

internal_handle_set_configuration

```
static void internal_handle_set_configuration(const uint16_t usb_value)
```

Handle SET_CONFIGURATION request for composite device.

Configures all pipes for both CDC ports.

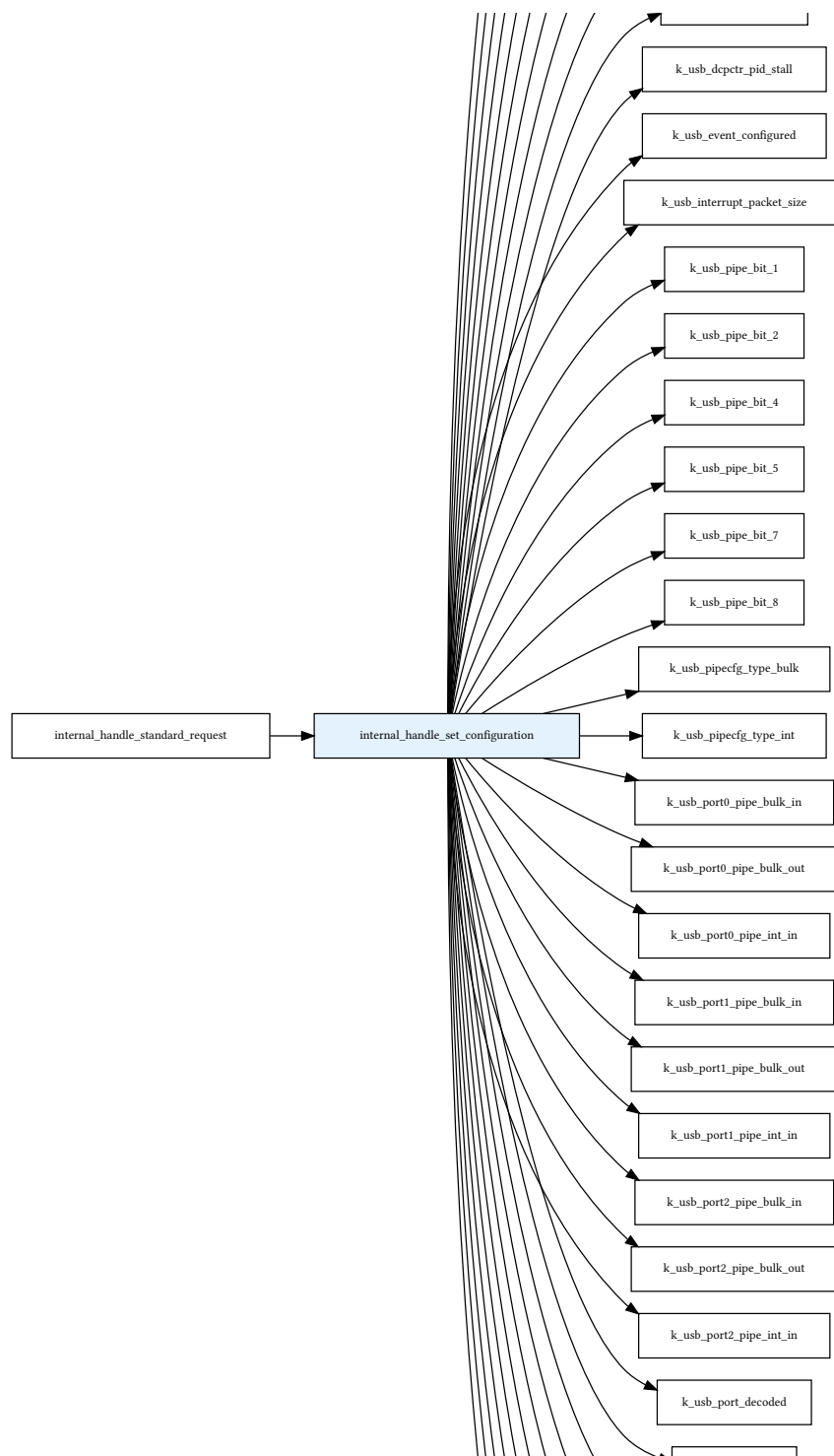


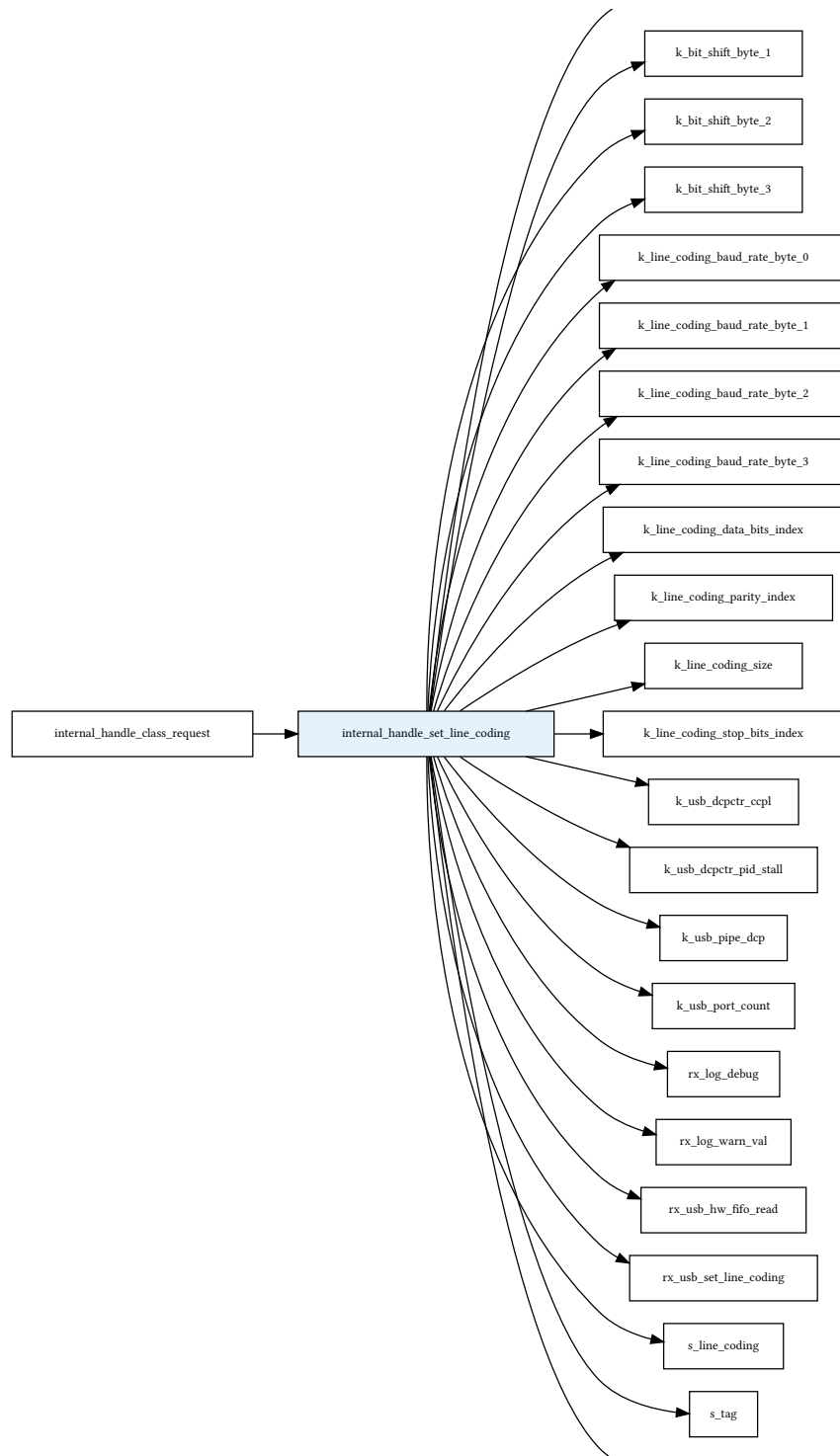
Figure 1187: Call graph for internal_handle_set_configuration

_Defined in libs/rx_usb/src/rx_usb_cdc.c:1613_
—

internal_handle_set_line_coding

```
static void internal_handle_set_line_coding(const rx_usb_port_id_t port)
```

Handle CDC SET_LINE_CODING request for specific port.

Figure 1188: Call graph for `internal_handle_set_line_coding`

_Defined in libs/rx\usb/src/rx\usb_cdc.c:1786_
—

internal_handle_get_line_coding

```
static void internal_handle_get_line_coding(const rx_usb_port_id_t port)
```

Handle CDC GET_LINE_CODING request for specific port.



Figure 1189: Call graph for `internal_handle_get_line_coding`

_Defined in libs/rx_usb/src/rx_usb_cdc.c:1821_
—

internal_handle_set_control_line_state

```
static void internal_handle_set_control_line_state(const rx_usb_port_id_t port,
const uint16_t usb_value)
```

Handle CDC SET_CONTROL_LINE_STATE request for specific port.

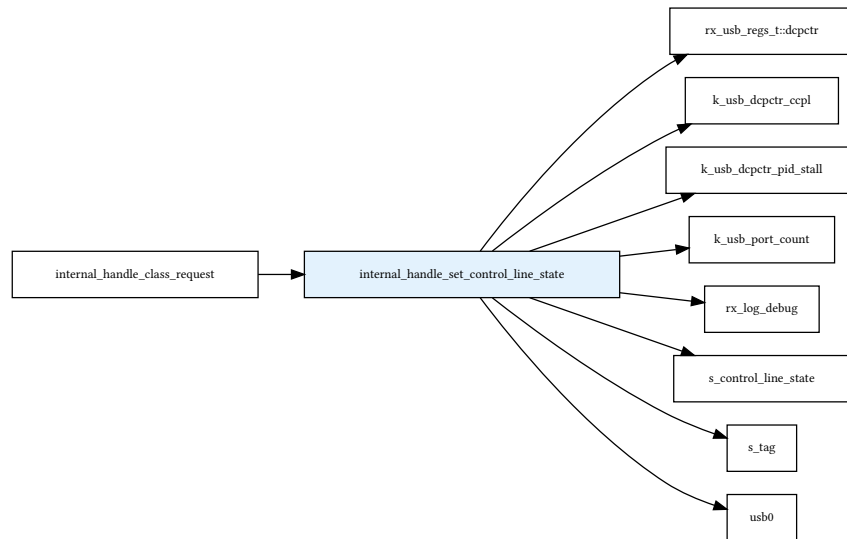


Figure 1190: Call graph for internal_handle_set_control_line_state

_Defined in libs/rx_usb/src/rx_usb_cdc.c:1851_
—

rx_usb_cdc_init

```
rx_err_t rx_usb_cdc_init(void)
```

Initialize USB CDC-ACM class layer for 3-port composite device.

Initialize CDC class (descriptors, state).

Initializes the CDC-ACM class protocol handler for all 3 virtual COM ports. Must be called once during USB subsystem initialization, before enabling USB interrupts and after hardware initialization (`rx_usb_hw_init()`).

Initialization sequence:

1. Check if already initialized (idempotent operation)
2. Reset line coding to default values for all 3 ports:

Baud rate: 115200 bps Stop bits: 1 Parity: None Data bits: 8

3. Clear control line state for all ports (DTR/RTS = 0)
4. Set initialization flag

What this does NOT do:

- Does NOT configure USB hardware registers (done by `rx_usb_hw_init()`)
- Does NOT enable USB interrupts (done by `rx_usb_init()`)
- Does NOT configure pipes (done during SET_CONFIGURATION request)
- Does NOT start USB enumeration (started by VBUS attach)

Default line coding (all ports):

These defaults match common serial terminal settings (minicom, PuTTY, screen). Host can override via SET_LINE_CODING request after enumeration.

Control line state (all ports):

Host will set DTR=1 when opening the port (e.g., `screen /dev/ttyACM0 115200`).

Parameters:

Direction	Name	Description
in	None	

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, CDC class initialized

Pre: USB hardware initialized via `rx_usb_hw_init()`

Pre: USB core initialized via `rx_usb_init()`

Post: `s_cdc_initialized = true`

Post: All ports have default line coding (115200 8N1)

Post: All control line states cleared (DTR=0, RTS=0)

Note: Thread-safe: Must be called from main thread only, before ISR enabled

Note: Idempotent: Safe to call multiple times (returns `k_rx_ok` immediately)

Note: No side effects if already initialized

Warning: Do NOT call after USB interrupts enabled (race condition with ISR)

Performance: Execution time: 2 us @ 240 MHz (simple memory initialization)

Example: `rx_err_t err;`

```
err=rx_usb_hw_init(); if(err!=k_rx_ok){return err;}
```



```
err=rx_usb_init(); if(err!=k_rx_ok){returnerr;}
```

```
err=rx_usb_cdc_init(); if(err!=k_rx_ok){returnerr;}
```

NASA Power of 10 Compliance:: Rule 2: [OK] Fixed loop bound (k_usb_port_count = 3) Rule 3: [OK] No dynamic memory allocation Rule 5: [OK] 2 preconditions, 3 postconditions Rule 6: [OK] Variables declared at minimal scope

SOLID Principles:: S: Single responsibility (initialize CDC class state only) O: Open/closed (adding ports doesn't require code changes) D: Depends on abstraction (rx_usb_port_count constant, not hardcoded 3)

Example:

```
{
.baud_rate=115200, //StandardUSB-CDCdefault
.stop_bits=0, //1stopbit
.parity=0, //Noparity
.data_bits=8 //8databits
}
```

Example:

```
{
.dtr=0, //DataTerminalReady(bit0)
.rts=0 //RequestToSend(bit1)
}
```

See also: rx_usb_init() Core USB initialization (ring buffers, state machine),
rx_usb_hw_init() Hardware layer initialization (registers, clocks),
rx_usb_cdc_handle_setup() Called by ISR after initialization

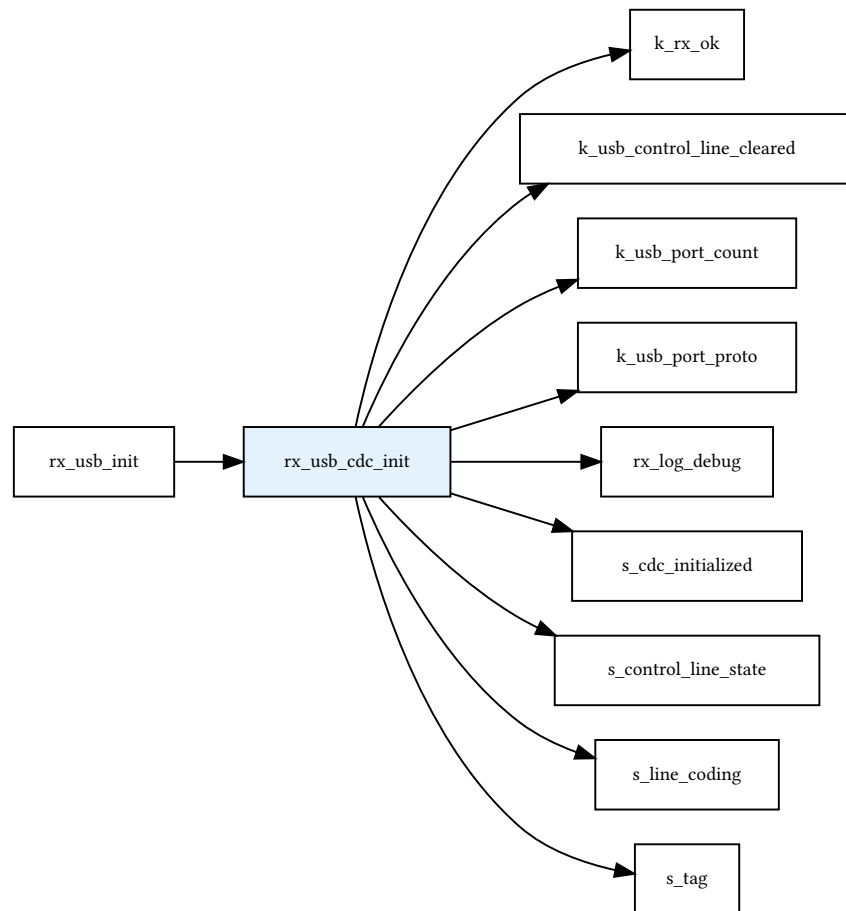


Figure 1191: Call graph for rx_usb_cdc_init

_Defined in `libs/rx_usb/src/rx_usb_cdc.c:1975_`

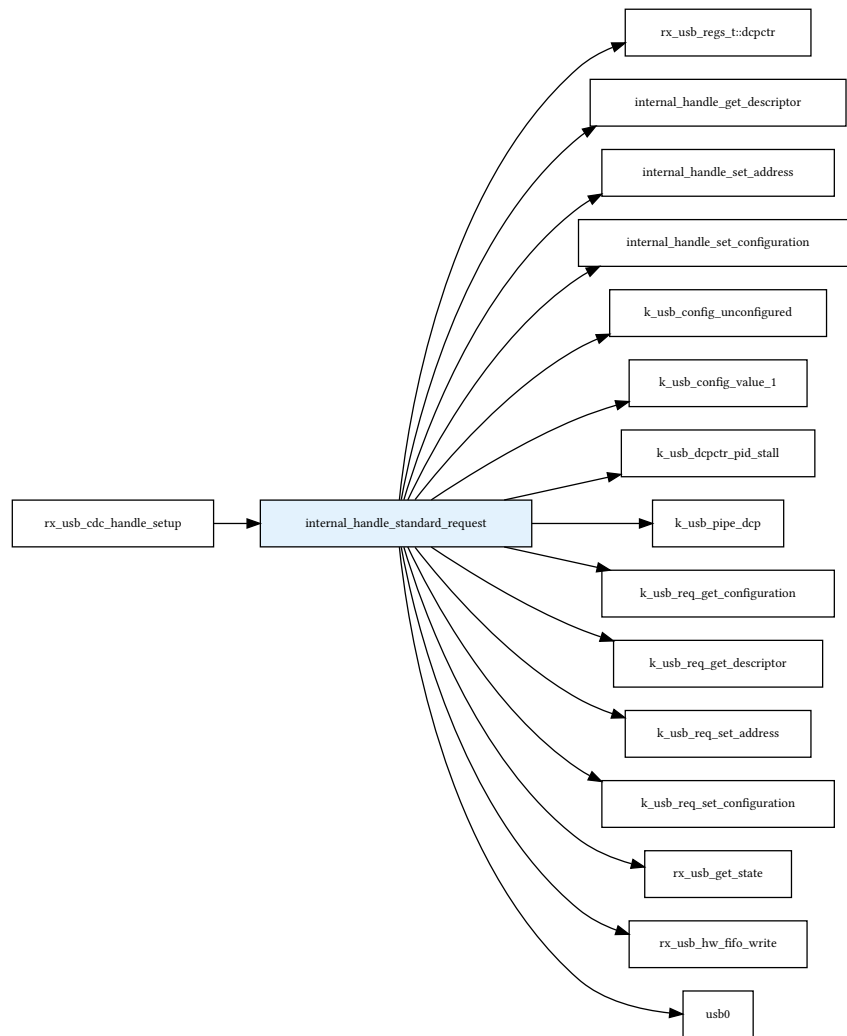
Since Version 1.5.0 (added 3rd port support)

—

internal_handle_standard_request

```
static void internal_handle_standard_request(const uint8_t usb_request, uint16_t
usb_value, uint16_t usb_length)
```

Handle standard USB requests.

Figure 1192: Call graph for `internal_handle_standard_request`

_Defined in `libs/rx\usb/src/rx\usb\_cdc.c:2001_`

internal_handle_class_request

```
static void internal_handle_class_request(const uint8_t usb_request, const
uint16_t usb_value, const uint16_t usb_index)
```

Handle CDC class-specific requests.

Routes the request to the correct port based on the interface number in `usb_index`.

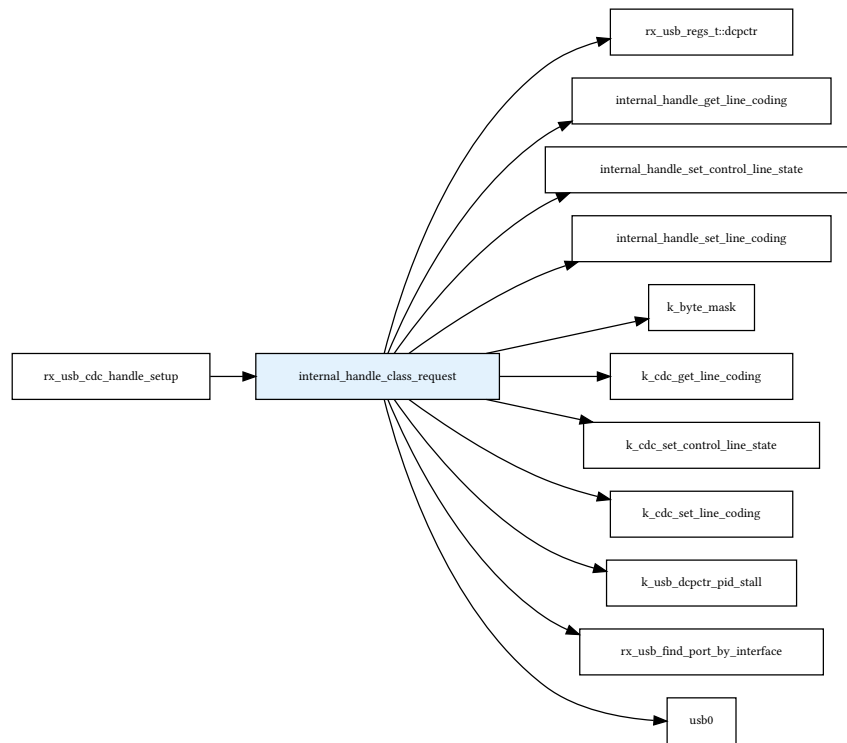


Figure 1193: Call graph for internal_handle_class_request

_Defined in libs/rx_usb/src/rx_usb_cdc.c:2036_

rx_usb_cdc_handle_setup

```
void rx_usb_cdc_handle_setup(void)
```

Handle USB control transfer SETUP packet for CDC composite device.

Handle USB control transfer (SETUP stage).

Processes USB SETUP packets received on endpoint 0 (default control pipe) during the control transfer protocol. This is the main entry point for all USB host requests (standard, class-specific, vendor). Called from USB ISR when CTRT (Control Transfer) interrupt fires.

SETUP Packet Structure (8 bytes):

Request Processing Flow:

1. Read SETUP packet from USB registers (USBREQ, USBVAL, USBINDX, USBLENG)
2. Extract request type from bmRequestType bits [6:5]
3. Route to appropriate handler:

Standard requests (0b00) -> internal_handle_standard_request() Class requests (0b01) -> internal_handle_class_request() Vendor requests (0b10) -> STALL (not supported)

4. Handler sends data/status or STALL

Standard Requests Supported:

Class Requests Supported (CDC-ACM per port):

SETUP Packet Examples:

Example 1: GET_DESCRIPTOR(Device)

Example 2: SET_CONFIGURATION(1)

Example 3: SET_LINE_CODING (Port 0)

Error Handling:

- Unsupported request type -> Send STALL (DCPCTR.PID = STALL)
- Unknown descriptor -> Send STALL
- Invalid interface number -> Send STALL
- Pipe configuration failure -> Rollback + STALL

STALL response tells host “request not supported” - host may retry or continue with defaults.

Control Transfer Timing:

Execution Context: USB ISR (interrupt priority 5)

- [OK] No blocking operations
- [OK] Minimal execution time (<500 us worst-case)
- [OK] Re-entrant safe (no shared mutable state between SETUP packets)

Parameters:

Direction	Name	Description
in	None	(reads SETUP packet from USB registers)

Returns: void (no return value)

Pre: USB ISR context (called from usb0_usbi_isr())

Pre: CTRT interrupt flag set in INTSTS0

Pre: USB in Default, Addressed, or Configured state

Post: SETUP packet processed (data sent, status sent, or STALL sent)

Post: USB state may transition (Default->Addressed->Configured)

Post: Pipes may be configured (if SET_CONFIGURATION)

Note: NOT thread-safe - must only be called from USB ISR

Note: Reads from USB registers directly (USBREQ, USBVAL, USBINDX, USBLENG)

Note: Modifies USB registers (DCPCTR for status/STALL)

Warning: Do NOT call from application code (ISR-only function)

Warning: Do NOT block inside this function (breaks ISR timing)

Performance:: Typical: 10-100 us (descriptor fetch) Worst-case: 500 us (SET_CONFIGURATION with 9 pipes) Frequency: 10 Hz during enumeration, <1 Hz after configured

State Machine Impact:: GET_DESCRIPTOR -> No state change SET_ADDRESS -> Default -> Addressed SET_CONFIGURATION -> Addressed -> Configured Class requests -> No state change

Example Usage (from ISR):: voidusb0_usbi_isr(void){ uint16_tintsts0=usb0()->intsts0; if(intsts0&k_usb_intsts0_ctr){ usb0()->intsts0= k_usb_intsts0_ctr; rx_usb_cdc_handle_setup(); } }

Example Host Trace (lsusb -v):: #EnumerationsequencecapturedbyUSBPcap:

```
1.SETUP[8006000100001200]GET_DESCRIPTOR(Device) <-
DATA[12010002EF0201405B043502000101020301]

2.SETUP[0005070000000000]SET_ADDRESS(7) <-STATUS(zero-length)

3.SETUP[800600020000CF00]GET_DESCRIPTOR(Config) <-
DATA[207bytes:config+3IADs+6interfaces+9endpoints]

4.SETUP[0009010000000000]SET_CONFIGURATION(1) <-STATUS(zero-length)
[Devicenowreadyfordatatransfer]
```

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (if/switch, no goto) Rule 4: [OK] Short function (25 lines) Rule 5: [OK] 3 preconditions, 3 postconditions Rule 7: [OK] All return values checked or explicitly ignored

SOLID Principles:: S: Single responsibility (route SETUP packets only) O: Open/closed (adding request types extends handlers, not this function) I: Interface segregation (minimal entry point, delegates to specialized handlers)

Example:

```
Byte0:bmRequestType[D7=direction,D6:5=type,D4:0=recipient]
Byte1:bRequest[specificrequestcode]
Bytes2-3:wValue[request-specificparameter]
Bytes4-5:wIndex[interface/endpointnumber]
Bytes6-7:wLength[datastagelength]
```

Example:

```
bmRequestType=0x80[IN,Standard,Device]
bRequest=0x06[GET_DESCRIPTOR]
wValue=0x0100[Type=Device(01),Index=0]
wIndex=0x0000
wLength=0x0012[18bytesrequested]
->Response:Sends_device_desc(18bytes)
```

Example:

```
bmRequestType=0x00[OUT,Standard,Device]
bRequest=0x09[SET_CONFIGURATION]
```

```
wValue=0x0001[Configuration1]
wIndex=0x0000
wLength=0x0000[Nodatastage]
->Action:Configure9pipes,enableBRDY/BEMPinterrupts
->Response:Sendzero-lengthstatuspacket(CCPL)
```

Example:

```
bmRequestType=0x21[OUT,Class,Interface]
bRequest=0x20[SET_LINE_CODING]
wValue=0x0000
wIndex=0x0000[Interface0=Port0]
wLength=0x0007[7bytes]
Data:[00C20100][00][00][08]
^115200bps^^8databits
1stopNoparity
->Action:Updates_line_coding[0]
->Response:Sendzero-lengthstatuspacket(CCPL)
```

See also: `internal_handle_standard_request()` Processes standard USB requests,
`internal_handle_class_request()` Processes CDC class requests, `rx_usb_cdc_init()` Must
be called before this function, `usb0_usbi_isr()` ISR that calls this function

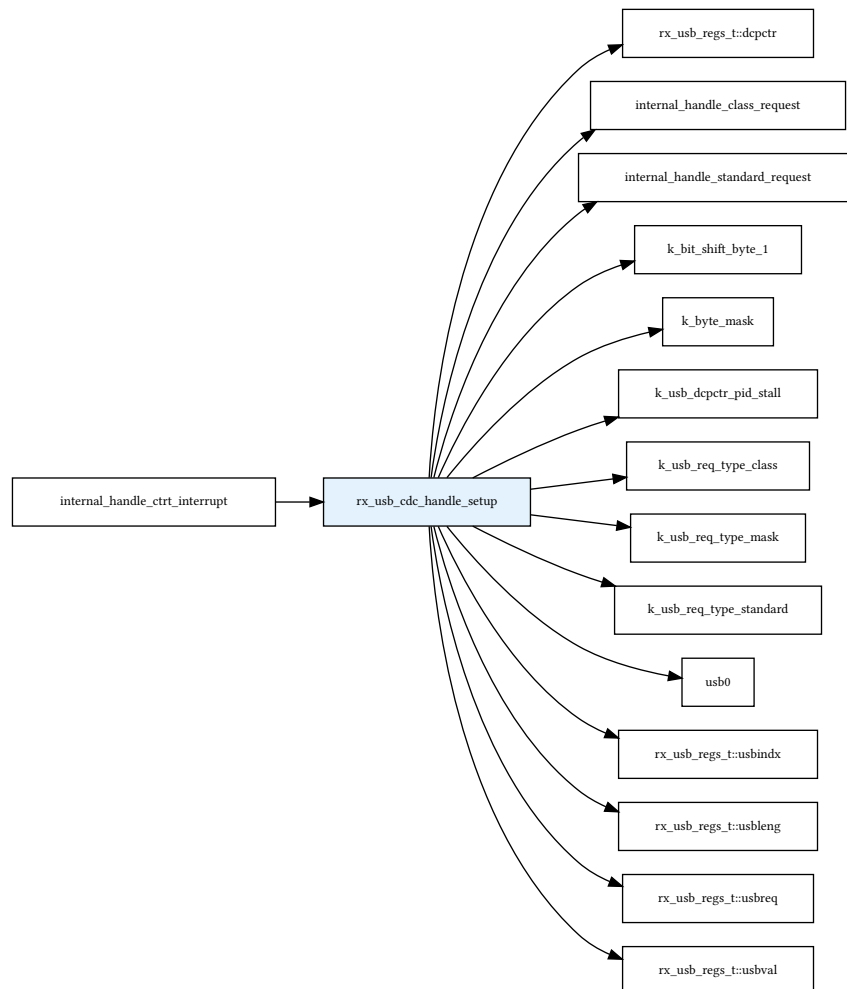


Figure 1194: Call graph for rx_usb_cdc_handle_setup

_Defined in `libs/rx\usb/src/rx\usb\_cdc.c:2239_`

Since Version 1.5.0 (added 3rd port support)

—

rx_usb_cdc_handle_bulk_out

```
void rx_usb_cdc_handle_bulk_out(const rx_usb_port_id_t port)
```

Handle USB bulk OUT transfer (data received from host -> device).

Handle bulk OUT data received for a port.

Processes bulk OUT transfers for a specific CDC port when data arrives from the host computer. Called from USB ISR when BRDY (Buffer Ready) interrupt fires for a bulk OUT pipe. This is the reception path for all serial data sent by the host.

Reception Flow (Host -> RX72N):

Pipe-to-Port Mapping (Bulk OUT):

Execution Sequence:

1. Validate port ID (0-2)
2. Get pipe number from port configuration table
3. Read data from USB FIFO (up to 64 bytes)
4. If len > 0, push to RX ring buffer
5. Ring buffer invokes RX_AVAILABLE callback if registered

Ring Buffer Behavior:

- Port 0: 1024-byte RX buffer (for binary protocol frames)
- Port 1: 512-byte RX buffer (for decoded commands)
- Port 2: 2048-byte RX buffer (for log messages)
- If buffer full, new data discarded (logged by rx_usb_rx_push())
- Application must read promptly to avoid overflow

Zero-Length Packet Handling:

- If len == 0, no data pushed to ring buffer
- No callback invoked for zero-length packets
- This is normal USB behavior (ZLP for transaction termination)

Error Cases:

- Invalid port ID -> Silent return (defensive check)
- nullptr config pointer -> Silent return (should never happen)
- FIFO read incomplete -> Logged by rx_usb_hw_fifo_read()
- Ring buffer full -> Logged by rx_usb_rx_push(), data lost

Performance Characteristics:

Interrupt Frequency:

- Idle: 0 Hz (no host traffic)
- Low traffic: 1-10 Hz (occasional commands)
- High traffic: Up to 18 kHz (1000 frames/sec x 18 packets/frame = saturated USB)
- CPU load: 0.1% idle, up to 12% saturated (7 us x 18 kHz)

Execution Context: USB ISR (interrupt priority 5)

- [OK] No blocking operations
- [OK] Fast execution (<10 us typical)
- [OK] Re-entrant safe (per-port state isolation)
- [OK] Callback invoked from ISR context (callback must be ISR-safe)

Parameters:

Direction	Name	Description
in	port	Port ID that received data (k_usb_port_proto, k_usb_port_decoded, k_usb_port_log)

Returns: void (no return value)

Pre: USB ISR context (called from usb0_usbi_isr())

Pre: BRDY interrupt for bulk OUT pipe

Pre: USB in Configured state

Pre: port < k_usb_port_count (0-2)

Post: Data read from USB FIFO

Post: Data pushed to RX ring buffer (if len > 0)

Post: RX_AVAILABLE callback invoked (if registered and buffer not empty)

Note: NOT thread-safe - must only be called from USB ISR

Note: Callback (if invoked) executes in ISR context

Note: Zero-length packets are valid and handled gracefully

Warning: Do NOT call from application code (ISR-only function)

Warning: Callback must be ISR-safe (no blocking, minimal execution time)

Warning: Ring buffer overflow causes data loss (application must read promptly)

Performance:: Typical: 5-7 us per 64-byte packet Worst-case: 10 us (with callback overhead)
Frequency: Up to 18 kHz @ full bandwidth CPU load: 0.1-12% (depends on traffic)

Example Usage (from ISR):: voidusb0_usbi_isr(void){ uint16_tbrdysts=usb0()->brdysts;
if(brdysts&(1<<2)){ usb0()->brdysts= (1<<2); rx_usb_cdc_handle_bulk_out(k_usb_port_proto); }
if(brdysts&(1<<5)){ usb0()->brdysts= (1<<5); rx_usb_cdc_handle_bulk_out(k_usb_port_decoded); }
if(brdysts&(1<<8)){ usb0()->brdysts= (1<<8); rx_usb_cdc_handle_bulk_out(k_usb_port_log); } }

Example Application Usage:: voidmy_task(void){ uint8_tbuffer[256]; uint32_tlen;
len=rx_usb_read(k_usb_port_proto,buffer,sizeof(buffer)); if(len>0){ process_command(buffer,len); } }
voidmy_rx_callback(rx_usb_port_id_tport,rx_usb_event_tevent,void*ctx)
{ if(event==k_usb_event_rx_available){ tx_event_flags_set(&usb_events,(1<<port),TX_OR); } }

Example Host Command:: #Linux:SendcommandtoProtocolport(Port0) echo-n"HelloRX72N">/dev/ttyACM0

#This triggers: #1.Host sends bulk OUT packet: "HelloRX72N" (11 bytes) #2.BRDY interrupt fires for pipe 2
 #3.rx_usb_cdc_handle_bulk_out(0) reads 11 bytes from FIFO #4.Data pushed to Port 0 RX ring buffer
 #5.RX_AVAILABLE callback invoked (if registered) #6.Application reads via rx_usb_read(0,...)

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (no recursion, no goto) Rule 4: [OK] Short function (20 lines) Rule 5: [OK] 4 preconditions, 3 postconditions Rule 7: [OK] rx_usb_rx_push() return value explicitly ignored (errors logged internally)

SOLID Principles:: S: Single responsibility (read USB FIFO, push to ring buffer) D: Depends on abstractions (rx_usb_hw_fifo_read, rx_usb_rx_push interfaces)

Example:

```
1.Host sends bulk OUT packet (upto 64 bytes)
v
2.USB hardware receives packet into FIFO
v
3.BRDY interrupt fires (pipe-specific bit set)
v
4.ISR calls rx_usb_cdc_handle_bulk_out(port) <- This function
v
5.Read data from USB FIFO via rx_usb_hw_fifo_read()
v
6.Push data to RX ring buffer via rx_usb_rx_push()
v
7.If RX_AVAILABLE callback registered, invoke callback
v
8.Application calls rx_usb_read(port) to consume data
```

See also: rx_usb_cdc_handle_bulk_in() Transmission counterpart (device -> host), rx_usb_read() Application function to consume received data, rx_usb_rx_push() Internal function that pushes data to ring buffer, rx_usb_hw_fifo_read() Hardware layer function that reads USB FIFO, usb0_usbi_isr() ISR that calls this function

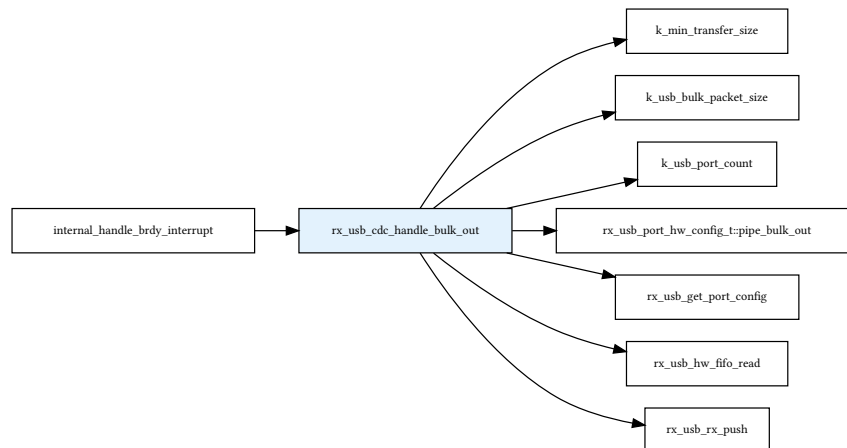


Figure 1195: Call graph for rx_usb_cdc_handle_bulk_out

_Defined in `libs/rx\usb/src/rx\usb\_cdc.c:2447_`

Since Version 1.5.0 (added Port 2 support)

rx_usb_cdc_handle_bulk_in

```
void rx_usb_cdc_handle_bulk_in(const rx_usb_port_id_t port)
```

Handle USB bulk IN transfer (data transmitted from device -> host).

Handle bulk IN transfer completion for a port.

Processes bulk IN transfers for a specific CDC port when the USB hardware is ready to transmit more data to the host computer. Called from USB ISR when BEMP (Buffer Empty) interrupt fires for a bulk IN pipe. This is the transmission path for all serial data sent to the host.

Transmission Flow (RX72N -> Host):

Pipe-to-Port Mapping (Bulk IN):

Execution Sequence:

1. Validate port ID (0-2)
2. Get pipe number from port configuration table
3. Pop data from TX ring buffer (up to 64 bytes)
4. If len > 0, write to USB FIFO
5. Hardware sends packet to host
6. If TX buffer now empty, invoke TX_COMPLETE callback

Ring Buffer Behavior:

- Port 0: 1024-byte TX buffer (for binary protocol responses)
- Port 1: 512-byte TX buffer (for decoded status messages)
- Port 2: 2048-byte TX buffer (for log streams)
- BEMP interrupts continue until TX buffer completely drained
- After last packet, TX_COMPLETE callback invoked (if registered)

Zero-Length Packet Handling:

- If TX buffer empty (len == 0), no data written to FIFO
- USB hardware automatically handles ZLP for transfers ending on packet boundary
- Example: 64-byte transfer needs ZLP to signal end (USB spec requirement)

Transmission Completion Detection:

Error Cases:

- Invalid port ID -> Silent return (defensive check)
- nullptr config pointer -> Silent return (should never happen)
- FIFO write incomplete -> Logged by rx_usb_hw_fifo_write()
- No error state persists (next BEMP retry continues transmission)

Performance Characteristics:

Interrupt Frequency:

- Idle: 0 Hz (no application TX, no BEMP interrupts)
- Burst TX: Up to 18 kHz (1000 frames/sec x 18 packets/frame = saturated USB)
- CPU load: 0% idle, up to 12% saturated (7 us x 18 kHz)

Typical Transmission Timeline:

Execution Context: USB ISR (interrupt priority 5)

- [OK] No blocking operations

- [OK] Fast execution (<10 us typical)
- [OK] Re-entrant safe (per-port state isolation)
- [OK] Callback invoked from ISR context (callback must be ISR-safe)

Parameters:

Direction	Name	Description
in	port	Port ID to transmit from (k_usb_port_proto, k_usb_port_decoded, k_usb_port_log)

Returns: void (no return value)

Pre: USB ISR context (called from usb0_usbi_isr())

Pre: BEMP interrupt for bulk IN pipe

Pre: USB in Configured state

Pre: port < k_usb_port_count (0-2)

Post: Data popped from TX ring buffer (if available)

Post: Data written to USB FIFO (if len > 0)

Post: TX_COMPLETE callback invoked (if TX buffer now empty and callback registered)

Note: NOT thread-safe - must only be called from USB ISR

Note: Callback (if invoked) executes in ISR context

Note: Zero-length pop (empty buffer) is normal and handled gracefully

Warning: Do NOT call from application code (ISR-only function)

Warning: Callback must be ISR-safe (no blocking, minimal execution time)

Warning: High-frequency BEMP interrupts can saturate CPU (12% @ full bandwidth)

Performance:: Typical: 5-7 us per 64-byte packet Worst-case: 10 us (with callback overhead)
Frequency: Up to 18 kHz @ full bandwidth CPU load: 0% idle, 12% saturated

Example Usage (from ISR):: voidusb0_usbi_isr(void){ uint16_tbempsts=usb0()->bempsts;
if(bempsts&(1<<1)){ usb0()->bempsts= (1<<1); rx_usb_cdc_handle_bulk_in(k_usb_port_proto); }

```

if(bempsts&(1<<4)){ usb0()->bempsts= (1<<4); rx_usb_cdc_handle_bulk_in(k_usb_port_decoded); }
if(bempsts&(1<<7)){ usb0()->bempsts= (1<<7); rx_usb_cdc_handle_bulk_in(k_usb_port_log); }

```

Example Application Usage:: voidsend_response(constuint8_t*data,uint32_tlen)
{ rx_err_tterr=rx_usb_write(k_usb_port_proto,data,len); if(err==k_rx_ok){ } }

```

voidmy_tx_callback(rx_usb_port_id_tport,rx_usb_event_tevent,void*ctx)
{ if(event==k_usb_event_tx_complete)
{ tx_event_flags_set(&usb_events,TX_DONE_FLAG,TX_OR); } }

```

Example Host Read:: #Linux:ReadresponsefromProtocolport(Port0) cat/dev/ttyACM0

```

#This triggers(onRX72Nside): #1.Application:rx_usb_write(0,"Respsen",9)
#2.DatacopiedtoPort0TXringbuffer #3.Firstpacket(9bytes)loadedtoFIFO
#4.USBhardwaresendsbulkINpacket #5.BEMPinterruptfires
#6.rx_usb_cdc_handle_bulk_in(0)popsnextpacket(noneleft) #7.TX_COMPLETEcallbackinvoked #
#Hostsees:"Respsen"

```

Large Transfer Example:: uint8_tdata[1024]; memset(data,'A',sizeof(data));
rx_usb_write(k_usb_port_log,data,1024);

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (no recursion, no goto) Rule 4:
[OK] Short function (18 lines) Rule 5: [OK] 4 preconditions, 3 postconditions Rule 7: [OK]
rx_usb_hw_fifo_write() return value explicitly ignored (errors logged internally)

SOLID Principles:: S: Single responsibility (pop from ring buffer, write to USB FIFO) D: Depends on
abstractions (rx_usb_tx_pop, rx_usb_hw_fifo_write interfaces)

Example:

```

1.Applicationcallsrx_usb_write(port,data,len)
v
2.DatacopiedtoTXringbuffer(rx_usb.c)
v
3.FirstpacketloadedintoUSBFIFO
v
4.USBhardwaresendspackettohost(upto64bytes)
v
5.BEMPinterruptfires(bufferempty,readyformore)
v
6.ISRcallsrx_usb_cdc_handle_bulk_in(port)<-Thisfunction
v
7.PopnextpacketfromTXringbufferviarx_usb_tx_pop()
v
8.WritetoUSBFIFOviarx_usb_hw_fifo_write()
v
9.Repeatsteps4-8untilTXbufferempty
v
10.IfTXbufferempty,invokeTX_COMPLETEcallback

```

Example:

```

//Afterlastpacketsent:
rx_usb_tx_pop(port,...)returns0(bufferempty)
v
NodataawrittentoFIFO
v
BEMPinterruptsstop(nomoredatatototend)

```

v
TX_COMPLETEcallbackinvoked(byrx_usb_tx_pop())

Example:

T+0ms: rx_usb_write(port, 200bytes)
T+0.1ms: First 64 bytes loaded to FIFO, packets sent
T+1.0ms: BEMP interrupt, load bytes 64-127
T+2.0ms: BEMP interrupt, load bytes 128-191
T+3.0ms: BEMP interrupt, load bytes 192-199 (final 8 bytes)
T+4.0ms: BEMP interrupt, TX buffer empty, TX_COMPLETE callback

See also: rx_usb_cdc_handle_bulk_out() Reception counterpart (host -\> device),
rx_usb_write() Application function to queue data for transmission, rx_usb_tx_pop()
Internal function that pops data from ring buffer, rx_usb_hw_fifo_write() Hardware
layer function that writes USB FIFO, usb0_usbi_isr() ISR that calls this function

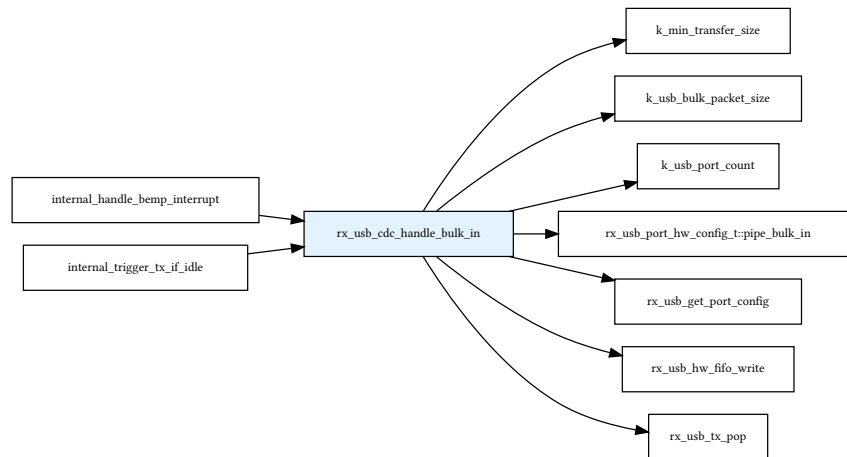


Figure 1196: Call graph for rx_usb_cdc_handle_bulk_in

_Defined in libs/rx\usb/src/rx\usb_cdc.c:2697_

Since Version 1.5.0 (added Port 2 support)

—

rx_usb_hw.c

USB0 Hardware Abstraction Layer (HAL) for RX72N Full-Speed Module.

Includes:

- <stddef.h>
- "rx72n_regs.h"
- "rx_log.h"
- "rx_register_protection.h"
- "rx_threadx_config.h"
- "rx_time_constants.h"
- "rx_usb.h"
- "rx_usb_internal.h"
- "tx_api.h"

usb_hw_timing_t

USB timing constants for initialization delays.

Value	Name	Description
= 10	k_usb_pll_stabilization_ms	USB PLL stabilization time (10ms)
= 10	k_usb_clock_stabilization_ms	USB clock stabilization time (10ms)
= 1	k_min_sleep_ticks	Minimum sleep duration (1 tick)
= 0	k_min_transfer_size	Minimum data transfer size (no data)

—

usb_syscfg_value_t

USB SYSCFG register values.

Value	Name	Description
= 0x0000	k_usb_syscfg_disabled	USB module disabled (all bits clear)

—

usb_fifo_constants_t

FIFO operation timeouts and masks.

Value	Name	Description
= 1000	k_usb_fifo_timeout_iterations	FIFO ready timeout (busy-wait iterations)
= 0	k_usb_fifo_timeout_expired	Timeout counter expired
= 0xFF	k_usb_fifo_byte_mask	Byte mask for 8-bit FIFO read

—

usb_address_mask_t

USB address mask.

Value	Name	Description
= 0x7F	k_usb_address_mask_hw	USB address mask (7 bits, 0-127)

—

usb_icu_config_t

Interrupt Controller (ICU) configuration constants.

Value	Name	Description
= 8	k_icu_bits_per_ier_register	Number of interrupt enable bits per IER register
= 6	k_usb_interrupt_priority	USB interrupt priority (moderate, below motor control)

—

usb_validation_limits_t

USB pipe and endpoint validation limits.

Value	Name	Description
= 0	k_usb_pipe_min	Minimum pipe number (DCP)
= 9	k_usb_pipe_max	Maximum pipe number
= 15	k_usb_endpoint_max	Maximum endpoint number (0-15)
= 512	k_usb_max_packet_size_max	Maximum packet size (512 bytes for FS)

—

s_tag

```
const char* s_tag
```

—

s_hw_initialized

```
bool s_hw_initialized
```

—

rx_usb_hw_init

```
rx_err_t rx_usb_hw_init(void)
```

Initialize USB0 hardware.

Initialize USB0 hardware peripheral.

This function:

1. Enables the USB0 module clock (clears module stop bit)
2. Configures USB0 for function (peripheral) mode
3. Sets up the USB clock (48 MHz from PLL)
4. Configures interrupts

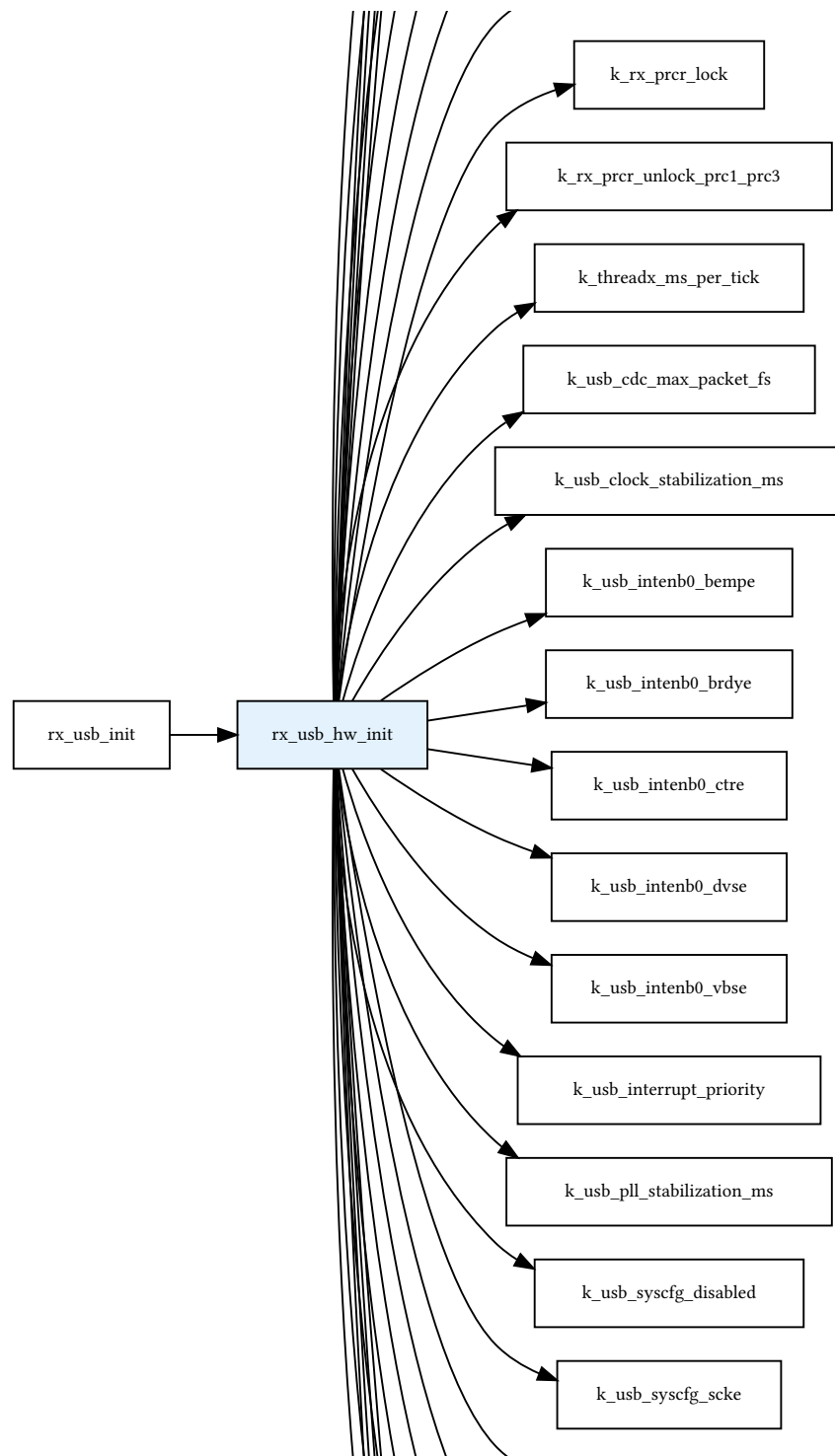


Figure 1197: Call graph for rx_usb_hw_init

_Defined in libs/rx_usb/src/rx_usb_hw.c:552_

—

rx_usb_hw_deinit

```
rx_err_t rx_usb_hw_deinit(void)
```

Deinitialize USB0 hardware.

Deinitialize USB0 hardware peripheral.

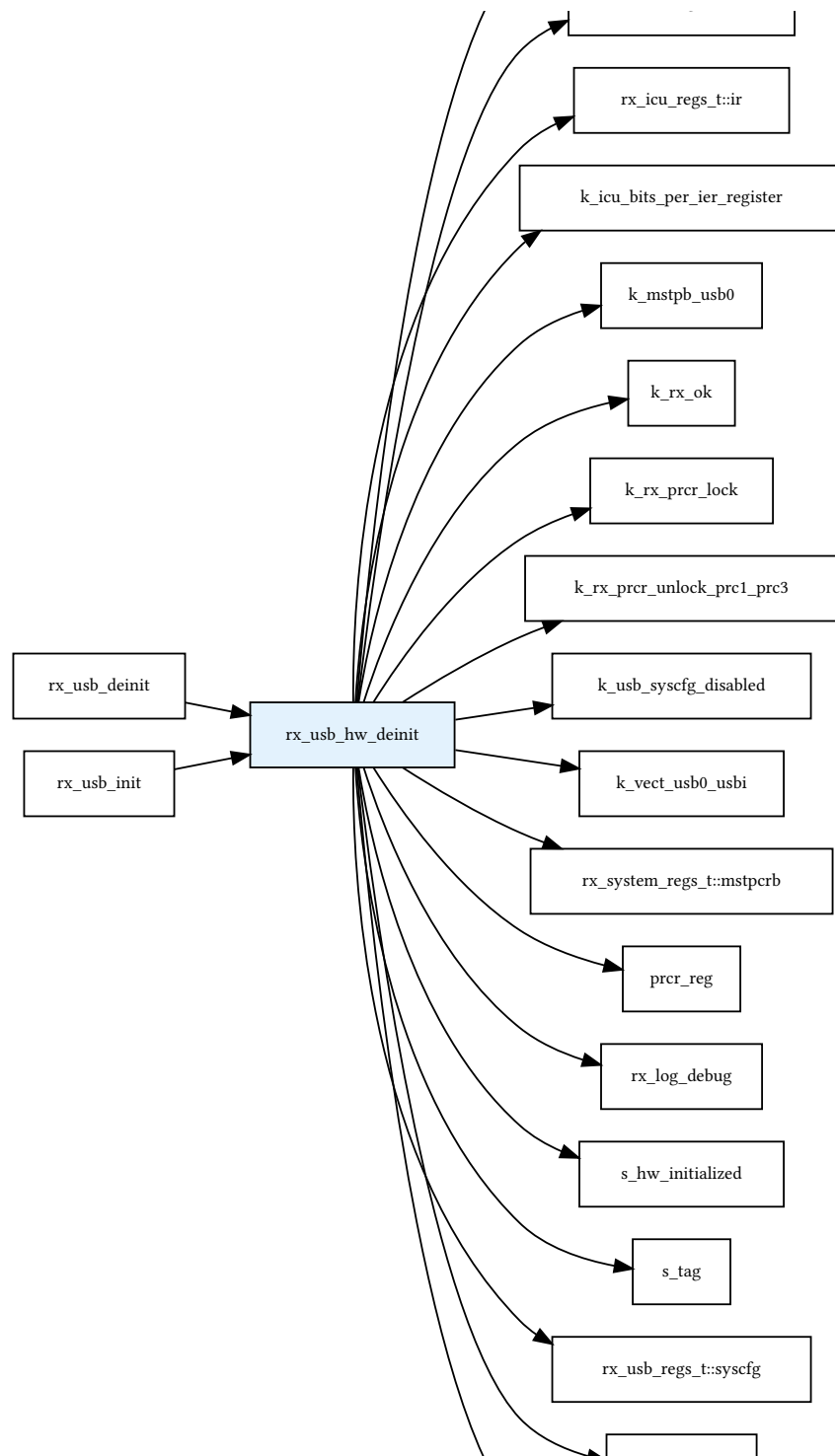


Figure 1198: Call graph for rx_usb_hw_deinit

_Defined in libs/rx_usb/src/rx_usb_hw.c:631_
—

rx_usb_hw_attach

```
rx_err_t rx_usb_hw_attach(void)
```

Attach to USB bus (enable D+ pull-up).

This signals to the host that a device is connected.

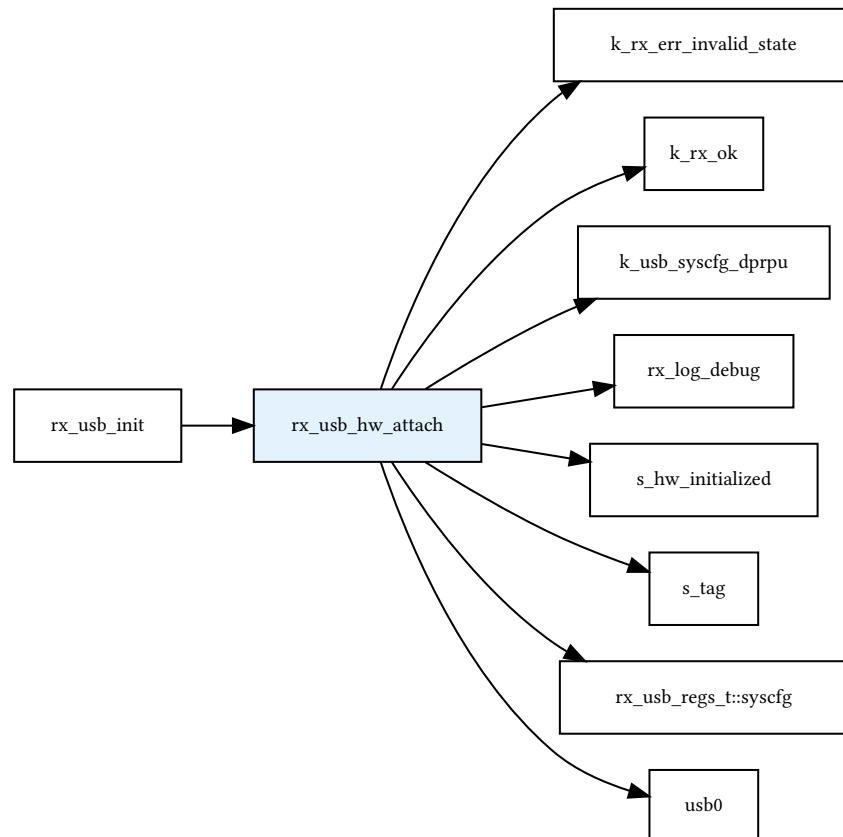


Figure 1199: Call graph for rx_usb_hw_attach

_Defined in libs/rx_usb/src/rx_usb_hw.c:663_
—

rx_usb_hw_detach

```
rx_err_t rx_usb_hw_detach(void)
```

Detach from USB bus (disable D+ pull-up).

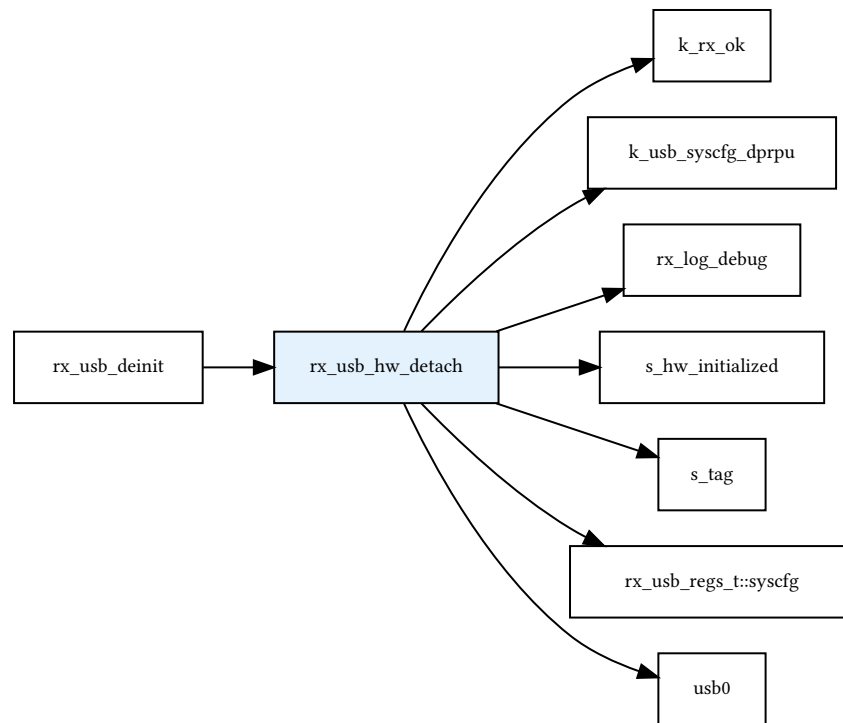


Figure 1200: Call graph for rx_usb_hw_detach

_Defined in libs/rx_usb/src/rx_usb_hw.c:680_

rx_usb_hw_fifo_read

```
uint32_t rx_usb_hw_fifo_read(uint8_t pipe, uint8_t *data, uint32_t max_len)
```

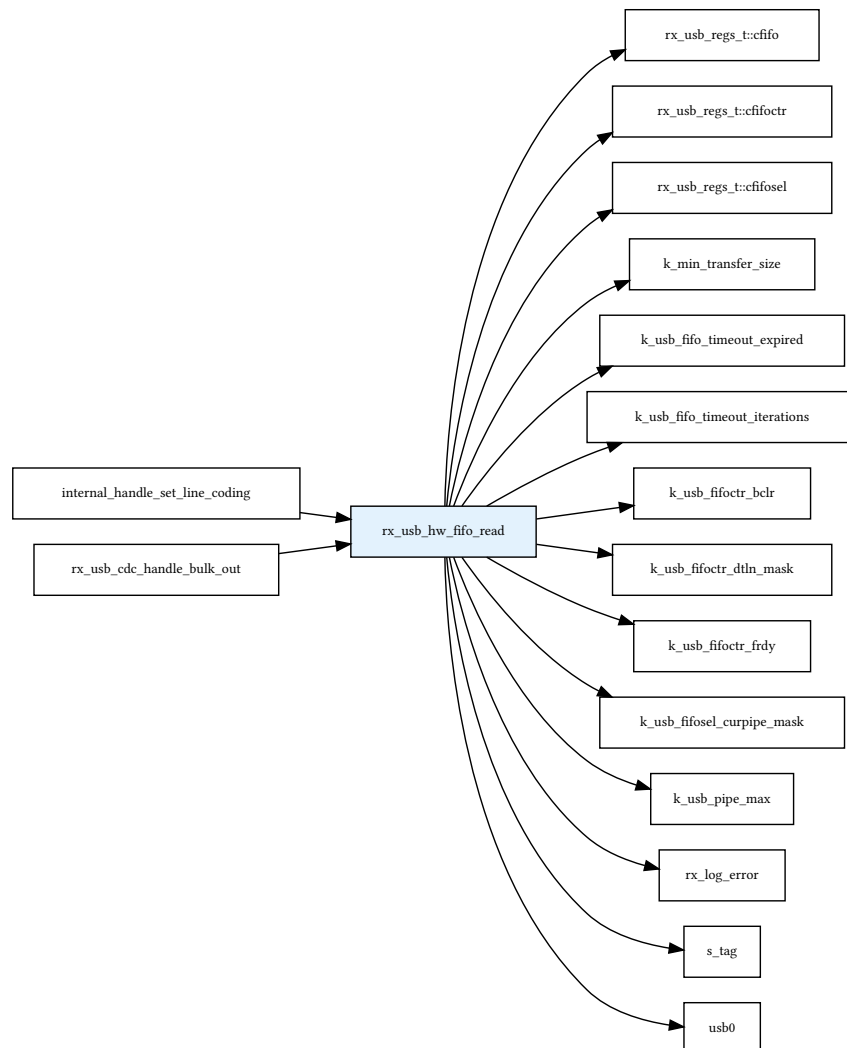
Read data from USB FIFO.

Read data from a USB pipe's FIFO.

Parameters:

Direction	Name	Description
in	pipe	Pipe number (0 = DCP, 1-9 = data pipes)
in	data	Output buffer
in	max_len	Maximum bytes to read

Returns: Number of bytes read

Figure 1201: Call graph for `rx_usb_hw_fifo_read`

_Defined in `libs/rx_usb/src/rx_usb_hw.c:702_`

`rx_usb_hw_fifo_write`

```
uint32_t rx_usb_hw_fifo_write(uint8_t pipe, const uint8_t *data, uint32_t len)
```

Write data to USB FIFO.

Write data to a USB pipe's FIFO.

Parameters:

Direction	Name	Description
in	pipe	Pipe number (0 = DCP, 1-9 = data pipes)
in	data	Input buffer
in	len	Number of bytes to write

Returns: Number of bytes written



Figure 1202: Call graph for `rx_usb_hw_fifo_write`

_Defined in `libs/rx_usb/src/rx_usb_hw.c:767_`

rx_usb_hw_get_bus_state

```
rx_usb_state_t rx_usb_hw_get_bus_state(void)
```

Get current USB bus state from hardware.

Get current USB bus state from hardware registers.

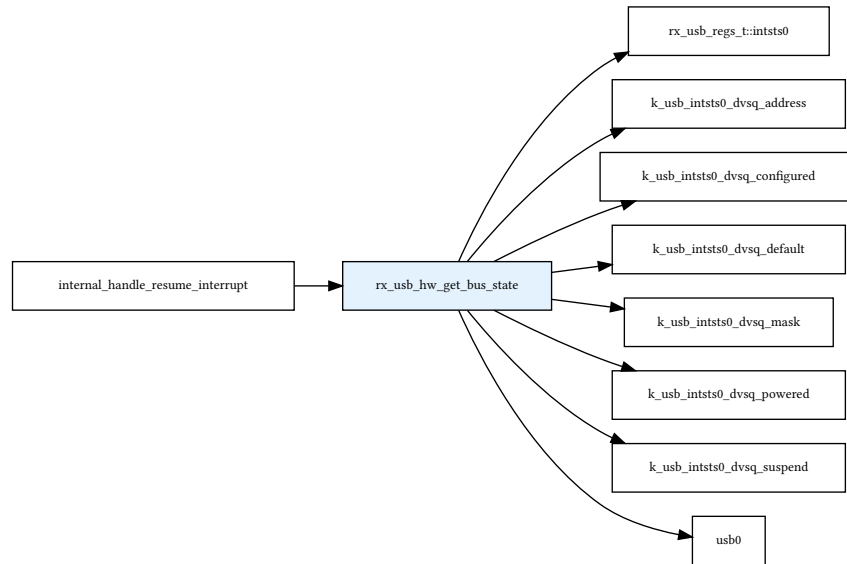


Figure 1203: Call graph for rx_usb_hw_get_bus_state

_Defined in libs/rx_usb/src/rx_usb_hw.c:829_

—

rx_usb_hw_set_address

```
void rx_usb_hw_set_address(const uint8_t address)
```

Set USB address (called during enumeration).

Set USB device address during enumeration.

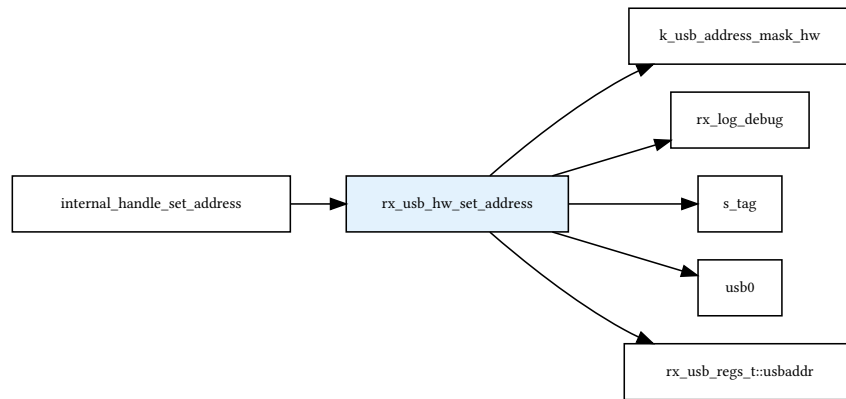


Figure 1204: Call graph for rx_usb_hw_set_address

Defined in libs/rx\usb/src/rx\usb\hw.c:852

rx_usb_hw_configure_pipe

```
rx_err_t rx_usb_hw_configure_pipe(const uint8_t pipe, const uint8_t endpoint,  
const bool is_in, const uint16_t type, const uint16_t max_packet)
```

Configure a pipe for bulk/interrupt transfer.

Configure a USB pipe for bulk/interrupt transfer.

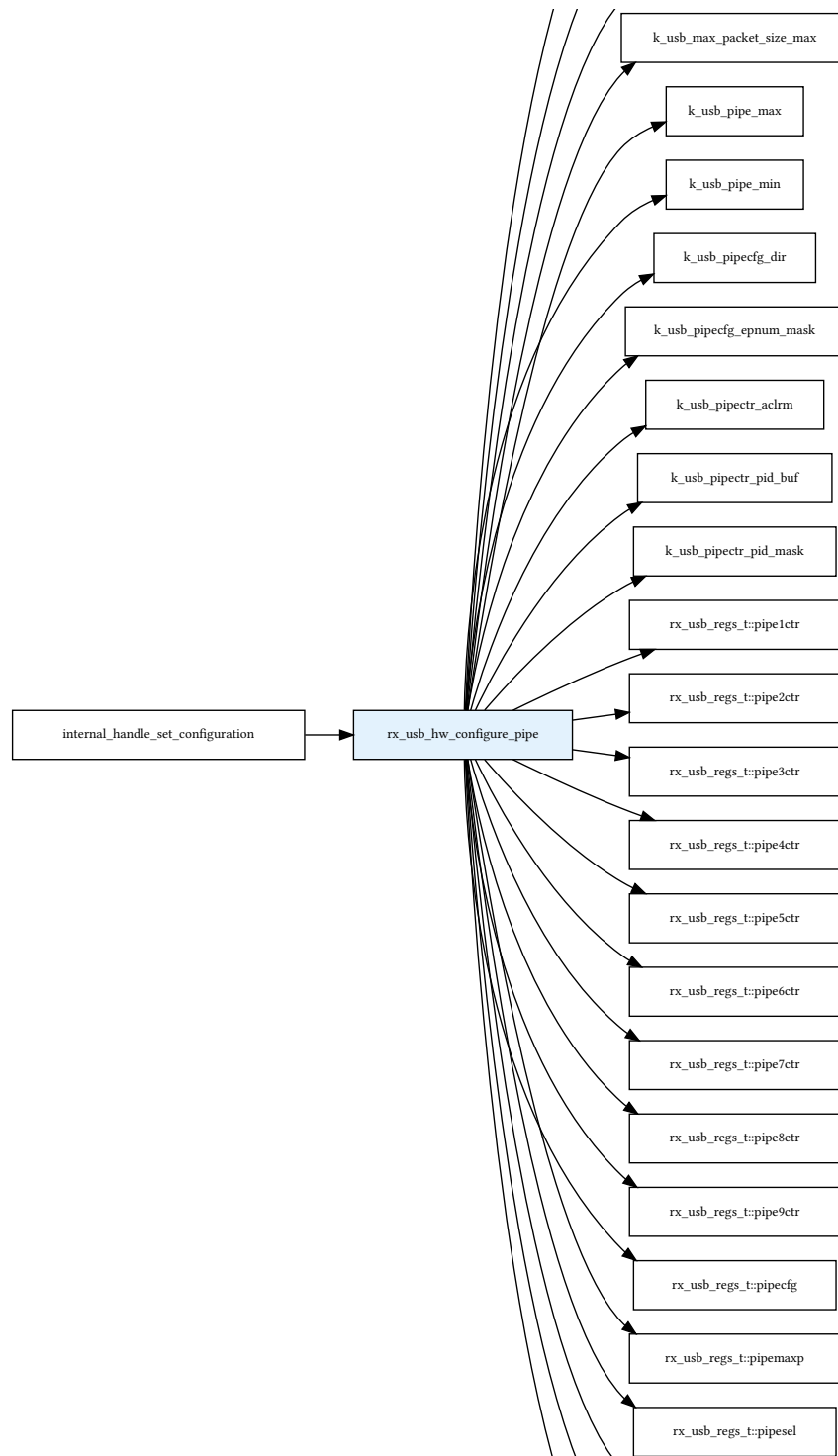


Figure 1205: Call graph for rx_usb_hw_configure_pipe

_Defined in libs/rx_usb/src/rx_usb_hw.c:861_

—

rx_usb_internal.h

Internal shared definitions for USB CDC driver implementation.

This private header provides internal types and function declarations shared between rx_usb.c and rx_usb_cdc.c. Not for public use.

2026-01-27

Copyright (c) 2026 STAR Project

Includes:

- <stdint.h>
- "rx_usb.h"

usb_interface_number_t

USB interface number assignments for CDC composite device.

Defines the interface numbers for each CDC port in the composite device. Shared between rx_usb.c (port configuration) and rx_usb_cdc.c (descriptor construction).

Value	Name	Description
= 0	k_intf_port0_control	Port 0 (Protocol) CDC control interface
= 1	k_intf_port0_data	Port 0 (Protocol) CDC data interface
= 2	k_intf_port1_control	Port 1 (Decoded) CDC control interface
= 3	k_intf_port1_data	Port 1 (Decoded) CDC data interface
= 4	k_intf_port2_control	Port 2 (Log) CDC control interface
= 5	k_intf_port2_data	Port 2 (Log) CDC data interface

—

rx_usb_get_port_config

```
const rx_usb_port_hw_config_t * rx_usb_get_port_config(rx_usb_port_id_t port)
```

Get port hardware configuration (shared-internal).

Returns the hardware configuration for the specified port. Used by rx_usb_cdc.c to construct descriptors. Defined in rx_usb.c.

Get port hardware configuration (shared-internal).

Parameters:

Direction	Name	Description
in	port	Port ID

Returns: Pointer to port configuration, or nullptr if port is invalid

Note: This is a shared-internal API, not part of the public interface. The `rx_usb_` prefix is used for internal functions shared across multiple implementation files within this module.

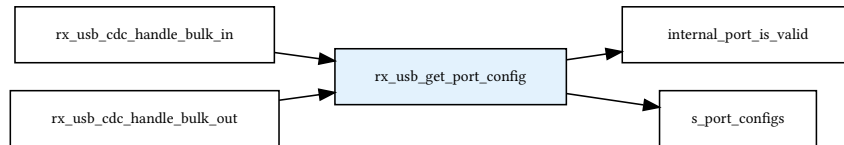


Figure 1206: Call graph for `rx_usb_get_port_config`

_Defined in `libs/rx_usb/src/rx_usb_internal.h:88_`

`rx_usb_hw_fifo_read`

```
uint32_t rx_usb_hw_fifo_read(uint8_t pipe, uint8_t *data, uint32_t max_len)
```

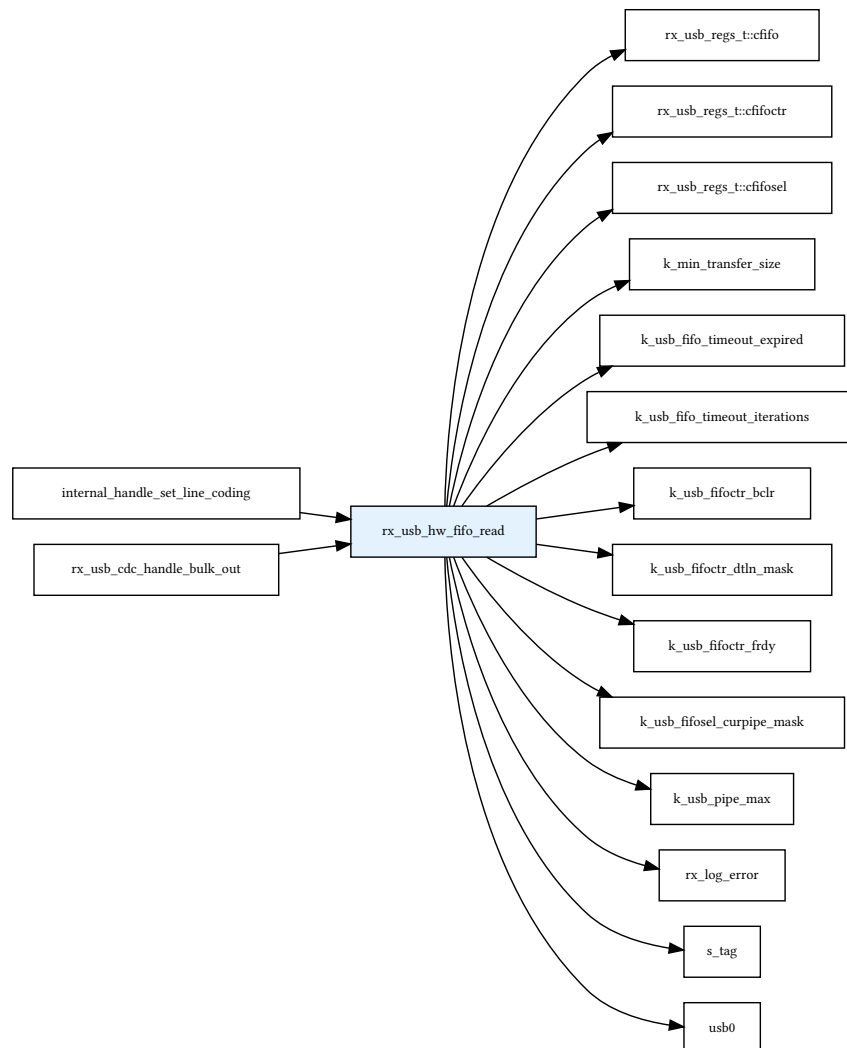
Read data from a USB pipe's FIFO.

Read data from a USB pipe's FIFO.

Parameters:

Direction	Name	Description
in	<code>pipe</code>	Pipe number to read from
out	<code>data</code>	Buffer to store read data
in	<code>max_len</code>	Maximum bytes to read
in	<code>pipe</code>	Pipe number (0 = DCP, 1-9 = data pipes)
in	<code>data</code>	Output buffer
in	<code>max_len</code>	Maximum bytes to read

Returns: Number of bytes read

Figure 1207: Call graph for `rx_usb_hw_fifo_read`

_Defined in `libs/rx_usb/src/rx_usb_internal.h:103_`

`rx_usb_hw_fifo_write`

```
uint32_t rx_usb_hw_fifo_write(uint8_t pipe, const uint8_t *data, uint32_t len)
```

Write data to a USB pipe's FIFO.

Write data to a USB pipe's FIFO.

Parameters:

Direction	Name	Description
in	pipe	Pipe number to write to
in	data	Data to write
in	len	Number of bytes to write
in	pipe	Pipe number (0 = DCP, 1-9 = data pipes)
in	data	Input buffer
in	len	Number of bytes to write

Returns: Number of bytes written



Figure 1208: Call graph for rx_usb_hw_fifo_write

_Defined in libs/rx_usb/src/rx_usb_internal.h:113_

rx_usb_hw_set_address

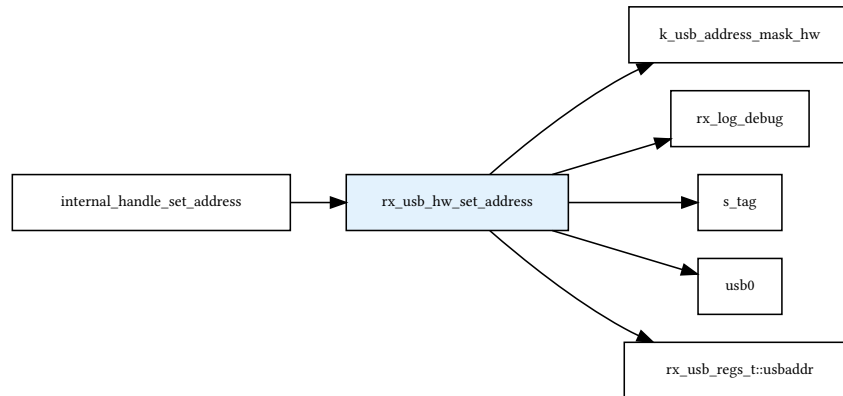
```
void rx_usb_hw_set_address(uint8_t address)
```

Set USB device address during enumeration.

Set USB device address during enumeration.

Parameters:

Direction	Name	Description
in	address	USB address assigned by host (0-127)

Figure 1209: Call graph for `rx_usb_hw_set_address`

_Defined in `libs/rx_usb/src/rx_usb_internal.h:120_`

rx_usb_hw_configure_pipe

```
rx_err_t rx_usb_hw_configure_pipe(uint8_t pipe, uint8_t endpoint, bool is_in,
uint16_t type, uint16_t max_packet)
```

Configure a USB pipe for bulk/interrupt transfer.

Configure a USB pipe for bulk/interrupt transfer.

Parameters:

Direction	Name	Description
in	pipe	Pipe number (1-9)
in	endpoint	Endpoint number (0-15)
in	is_in	True for IN (device to host), false for OUT
in	type	Pipe type (bulk, interrupt, isochronous)
in	max_packet	Maximum packet size

Returns: `k_rx_ok` on success, error code on failure

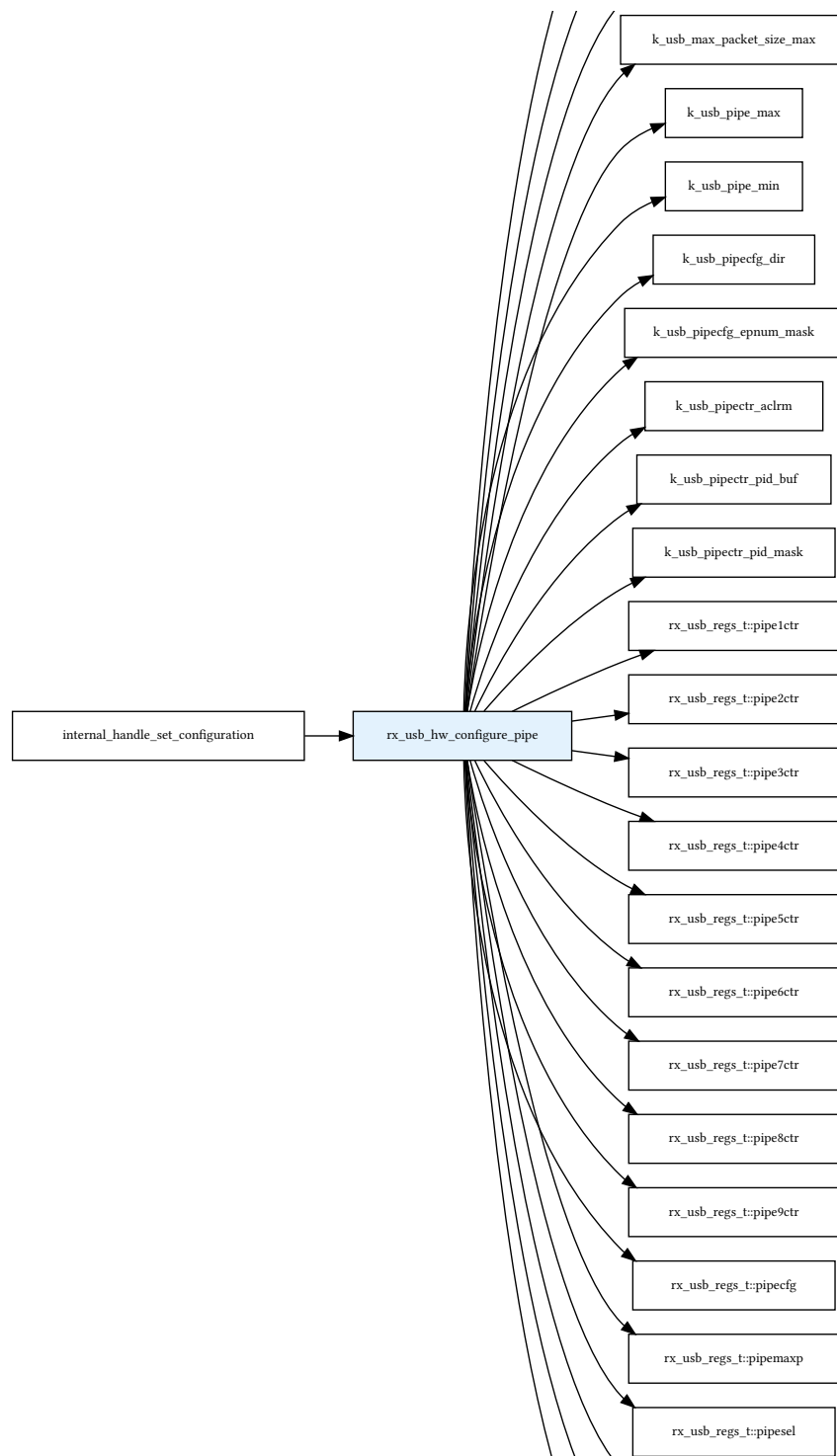


Figure 1210: Call graph for rx_usb_hw_configure_pipe

_Defined in libs/rx_usb/src/rx_usb_internal.h:132_

—

rx_usb_hw_init

```
rx_err_t rx_usb_hw_init(void)
```

Initialize USB0 hardware peripheral.

Initialize USB0 hardware peripheral.

This function:

1. Enables the USB0 module clock (clears module stop bit)
2. Configures USB0 for function (peripheral) mode
3. Sets up the USB clock (48 MHz from PLL)
4. Configures interrupts

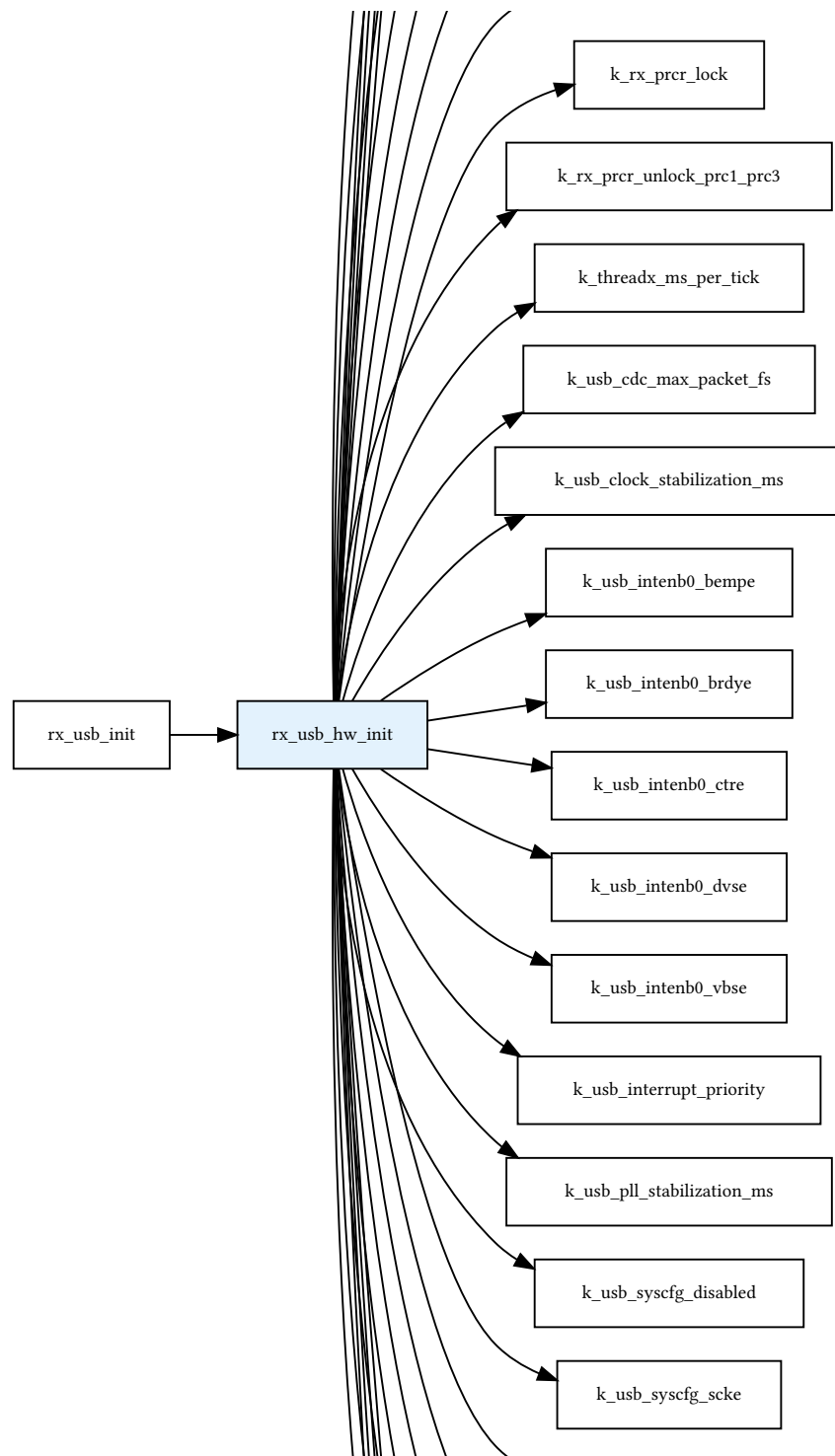


Figure 1211: Call graph for rx_usb_hw_init

_Defined in libs/rx_usb/src/rx_usb_internal.h:139_

—

rx_usb_hw_deinit

```
rx_err_t rx_usb_hw_deinit(void)
```

Deinitialize USB0 hardware peripheral.

Deinitialize USB0 hardware peripheral.

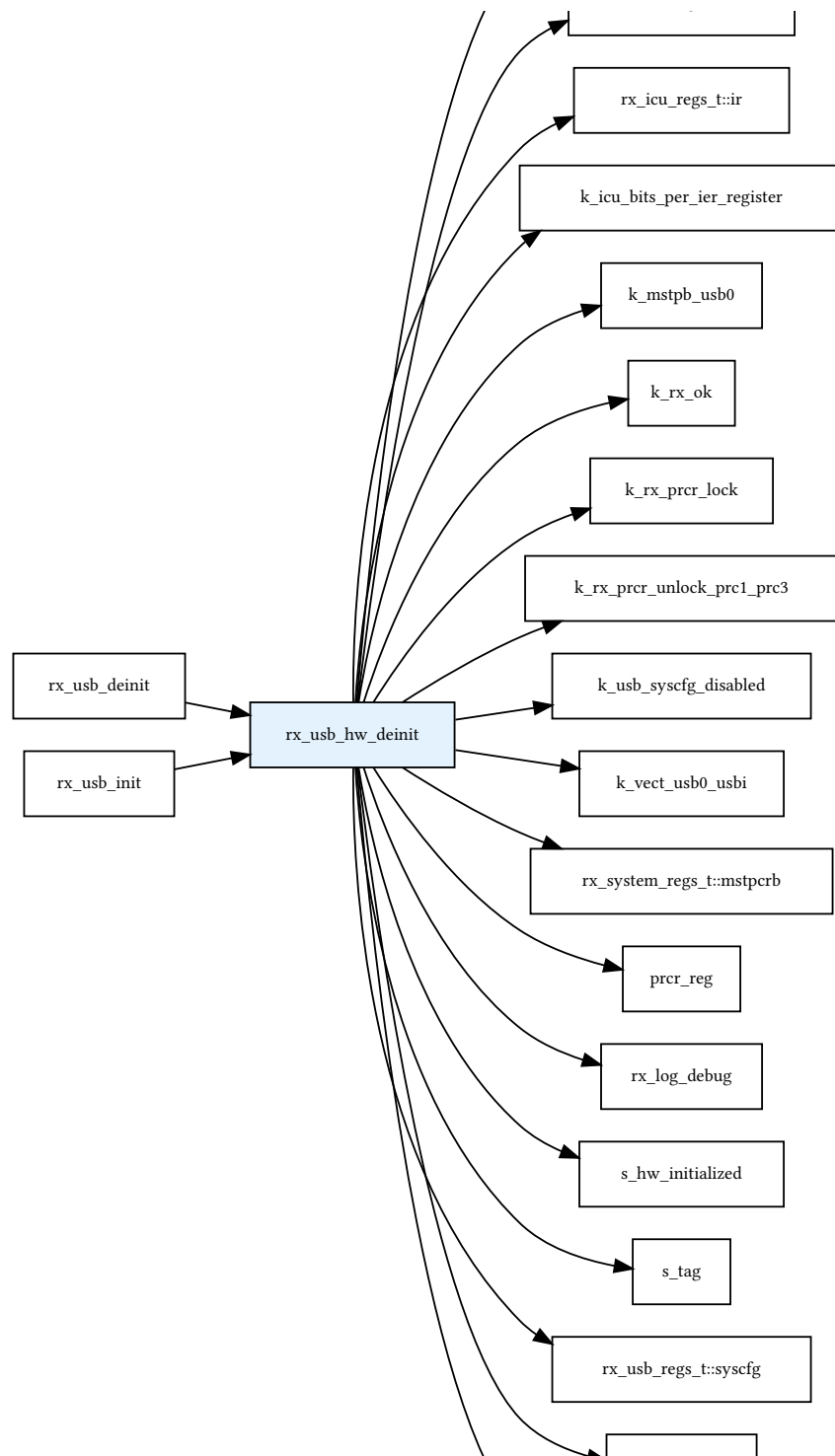


Figure 1212: Call graph for rx_usb_hw_deinit

_Defined in libs/rx_usb/src/rx_usb_internal.h:142_

—

rx_usb_hw_attach

```
rx_err_t rx_usb_hw_attach(void)
```

Attach to USB bus (enable D+ pull-up).

This signals to the host that a device is connected.

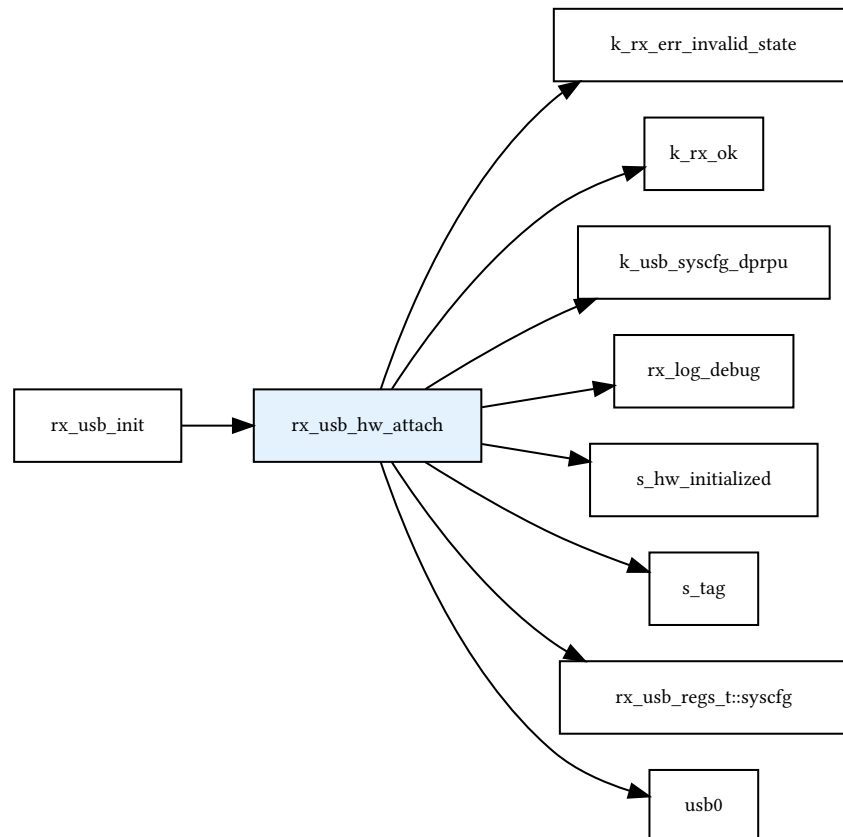


Figure 1213: Call graph for rx_usb_hw_attach

_Defined in libs/rx_usb/src/rx_usb_internal.h:145_

—

rx_usb_hw_detach

```
rx_err_t rx_usb_hw_detach(void)
```

Detach from USB bus (disable D+ pull-up).

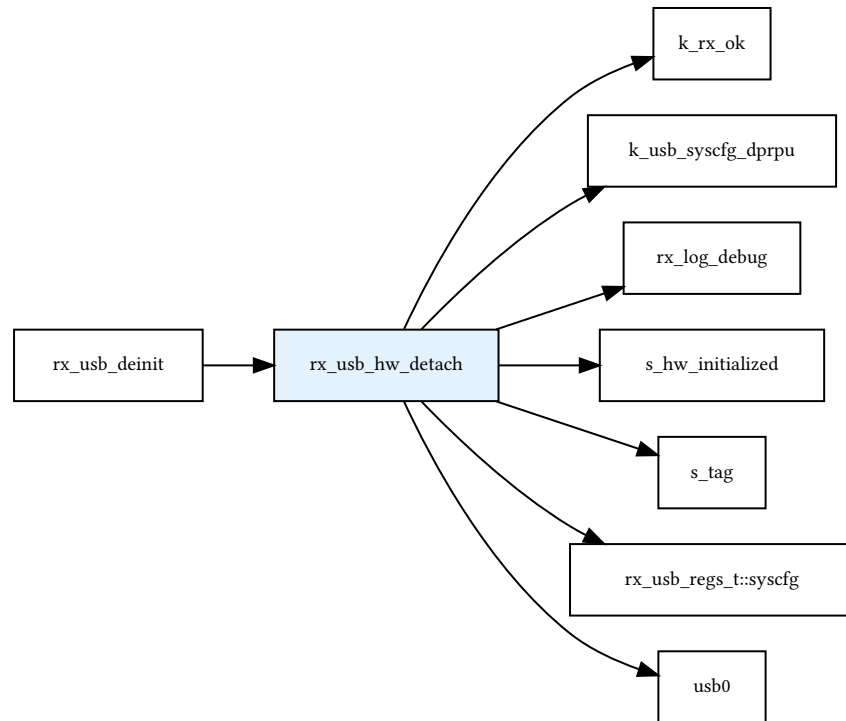


Figure 1214: Call graph for rx_usb_hw_detach

_Defined in `libs/rx_usb/src/rx_usb_internal.h:148_`

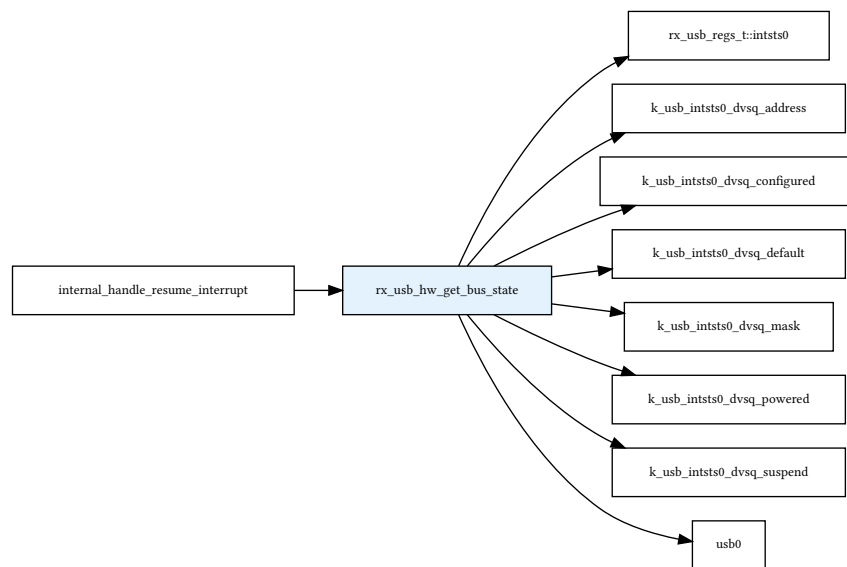
—

rx_usb_hw_get_bus_state

```
rx_usb_state_t rx_usb_hw_get_bus_state(void)
```

Get current USB bus state from hardware registers.

Get current USB bus state from hardware registers.

Figure 1215: Call graph for `rx_usb_hw_get_bus_state`

_Defined in `libs/rx\usb/src/rx\usb\_internal.h:151_`

rx_usb_set_state

```
void rx_usb_set_state(rx_usb_state_t state)
```

Set global USB device state.

Set global USB device state.

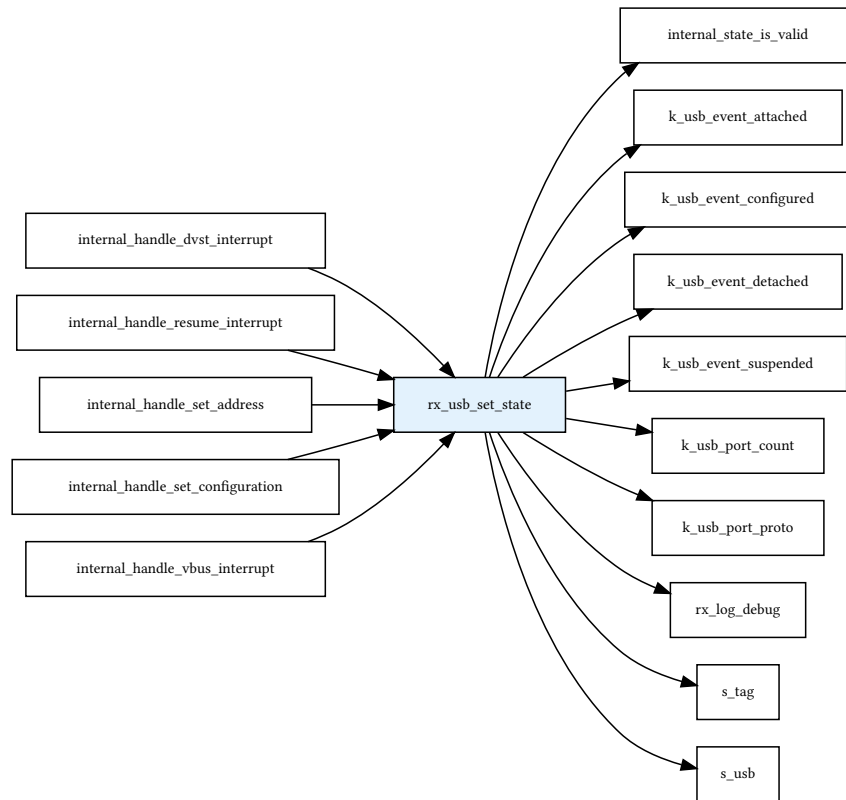


Figure 1216: Call graph for rx_usb_set_state

_Defined in libs/rx_usb/src/rx_usb_internal.h:159_

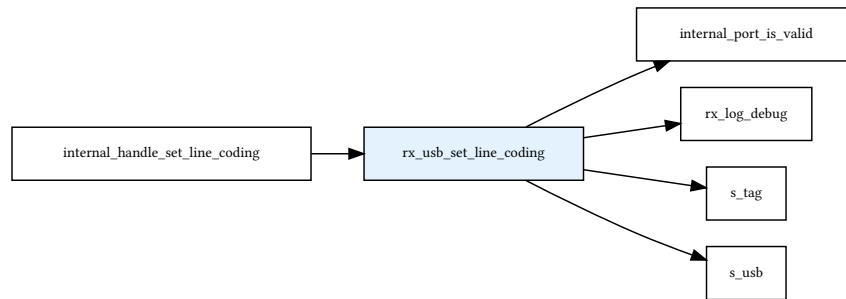
—

rx_usb_set_line_coding

```
void rx_usb_set_line_coding(rx_usb_port_id_t port, const rx_usb_line_coding_t
*coding)
```

Set CDC line coding for a port.

Set CDC line coding for a port.

Figure 1217: Call graph for `rx_usb_set_line_coding`_Defined in `libs/rx\usb/src/rx\usb\_internal.h:162_`

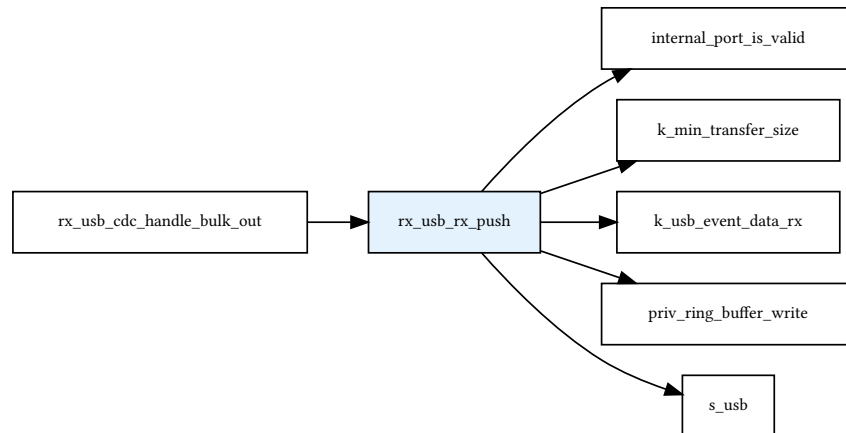
—

rx_usb_rx_push

```
uint32_t rx_usb_rx_push(rx_usb_port_id_t port, const uint8_t *data, uint32_t len)
```

Push data into a port's RX ring buffer.

Push data into a port's RX ring buffer.

Figure 1218: Call graph for `rx_usb_rx_push`_Defined in `libs/rx\usb/src/rx\usb\_internal.h:165_`

—

rx_usb_tx_pop

```
uint32_t rx_usb_tx_pop(rx_usb_port_id_t port, uint8_t *data, uint32_t max_len)
```

Pop data from a port's TX ring buffer.

Pop data from a port's TX ring buffer.

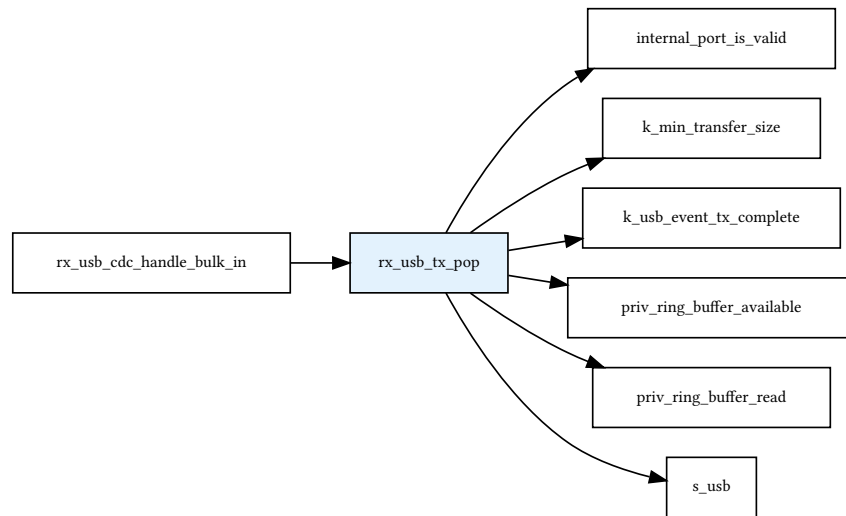


Figure 1219: Call graph for rx_usb_tx_pop

_Defined in libs/rx\usb/src/rx\usb_internal.h:168_

—

rx_usb_count_bus_reset

```
void rx_usb_count_bus_reset(void)
```

Increment bus reset counter.

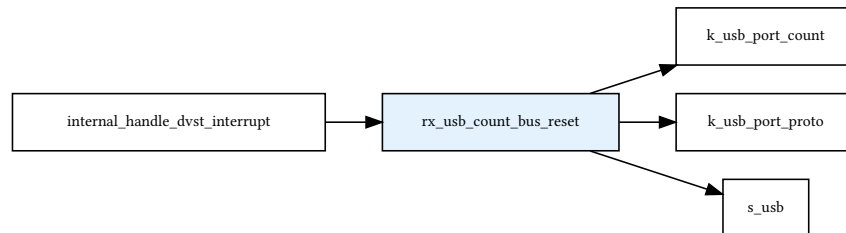


Figure 1220: Call graph for rx_usb_count_bus_reset

_Defined in libs/rx\usb/src/rx\usb_internal.h:171_

—

rx_usb_count_suspend

```
void rx_usb_count_suspend(void)
```

Increment suspend counter.

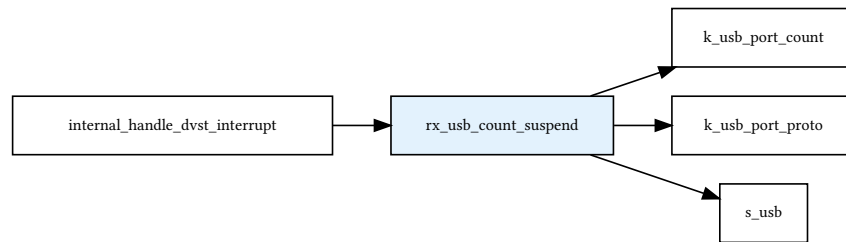


Figure 1221: Call graph for rx_usb_count_suspend

_Defined in libs/rx\usb/src/rx\usb_internal.h:174_
 —

rx_usb_find_port_by_pipe

```
rx_usb_port_id_t rx_usb_find_port_by_pipe(uint8_t pipe)
```

Find port by pipe number.

Find port by pipe number.

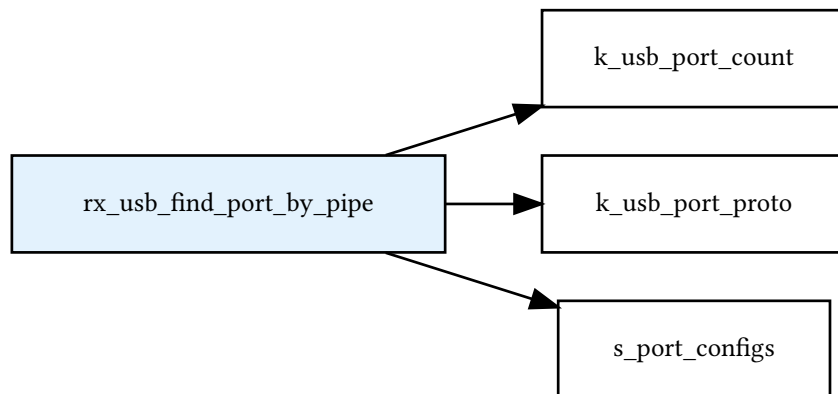


Figure 1222: Call graph for rx_usb_find_port_by_pipe

_Defined in libs/rx\usb/src/rx\usb_internal.h:177_
 —

rx_usb_find_port_by_interface

```
rx_usb_port_id_t rx_usb_find_port_by_interface(uint8_t interface)
```

Find port by interface number.

Find port by interface number.

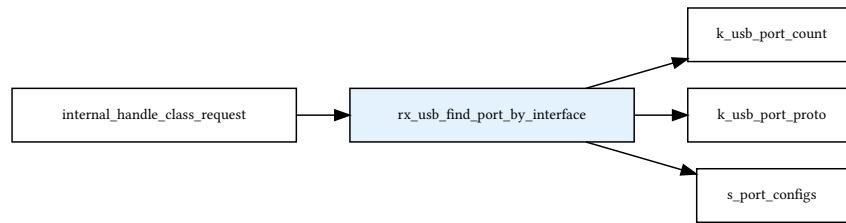


Figure 1223: Call graph for rx_usb_find_port_by_interface

Defined in libs/rx\usb/src/rx\usb\internal.h:180

—

rx_usb_invoke_callback

```
void rx_usb_invoke_callback(rx_usb_port_id_t port, rx_usb_event_t event)
```

Invoke user callback for a port event.

Invoke user callback for a port event.

Called by ISR and CDC handlers to notify the application of USB events.

Parameters:

Direction	Name	Description
in	port	Port ID
in	event	Event type

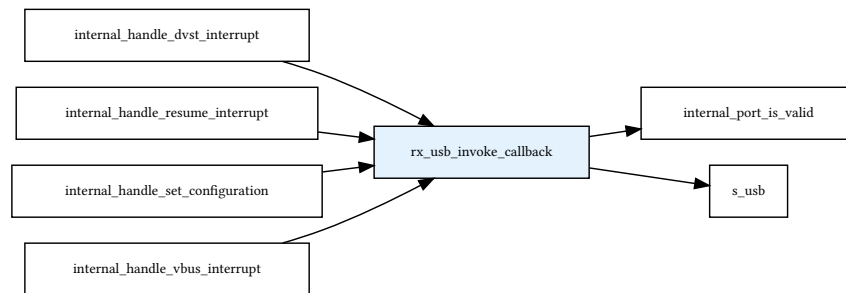


Figure 1224: Call graph for rx_usb_invoke_callback

Defined in libs/rx\usb/src/rx\usb\internal.h:183

—

rx_usb_cdc_init

```
rx_err_t rx_usb_cdc_init(void)
```

Initialize CDC class (descriptors, state).

Initialize CDC class (descriptors, state).

Initializes the CDC-ACM class protocol handler for all 3 virtual COM ports. Must be called once during USB subsystem initialization, before enabling USB interrupts and after hardware initialization (`rx_usb_hw_init()`).

Initialization sequence:

1. Check if already initialized (idempotent operation)
2. Reset line coding to default values for all 3 ports:

Baud rate: 115200 bps Stop bits: 1 Parity: None Data bits: 8

3. Clear control line state for all ports (DTR/RTS = 0)
4. Set initialization flag

What this does NOT do:

- Does NOT configure USB hardware registers (done by `rx_usb_hw_init()`)
- Does NOT enable USB interrupts (done by `rx_usb_init()`)
- Does NOT configure pipes (done during SET_CONFIGURATION request)
- Does NOT start USB enumeration (started by VBUS attach)

Default line coding (all ports):

These defaults match common serial terminal settings (minicom, PuTTY, screen). Host can override via SET_LINE_CODING request after enumeration.

Control line state (all ports):

Host will set DTR=1 when opening the port (e.g., `screen /dev/ttyACM0 115200`).

Parameters:

Direction	Name	Description
in	None	

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success, CDC class initialized

Pre: USB hardware initialized via `rx_usb_hw_init()`

Pre: USB core initialized via `rx_usb_init()`

Post: `s_cdc_initialized = true`

Post: All ports have default line coding (115200 8N1)

Post: All control line states cleared (DTR=0, RTS=0)

Note: Thread-safe: Must be called from main thread only, before ISR enabled

Note: Idempotent: Safe to call multiple times (returns k_rx_ok immediately)

Note: No side effects if already initialized

Warning: Do NOT call after USB interrupts enabled (race condition with ISR)

Performance:: Execution time: 2 us @ 240 MHz (simple memory initialization)

Example:: rx_err_t err;

```
err=rx_usb_hw_init(); if(err!=k_rx_ok){return err;}
```

```
err=rx_usb_init(); if(err!=k_rx_ok){return err;}
```

```
err=rx_usb_cdc_init(); if(err!=k_rx_ok){return err;}
```

NASA Power of 10 Compliance:: Rule 2: [OK] Fixed loop bound (k_usb_port_count = 3) Rule 3: [OK] No dynamic memory allocation Rule 5: [OK] 2 preconditions, 3 postconditions Rule 6: [OK] Variables declared at minimal scope

SOLID Principles:: S: Single responsibility (initialize CDC class state only) O: Open/closed (adding ports doesn't require code changes) D: Depends on abstraction (rx_usb_port_count constant, not hardcoded 3)

Example:

```
{
    .baud_rate=115200, //StandardUSB-CDCdefault
    .stop_bits=0, //1stopbit
    .parity=0, //Noparity
    .data_bits=8 //8databits
}
```

Example:

```
{
    .dtr=0, //DataTerminalReady(bit0)
    .rts=0 //RequestToSend(bit1)
}
```

See also: rx_usb_init() Core USB initialization (ring buffers, state machine),
rx_usb_hw_init() Hardware layer initialization (registers, clocks),
rx_usb_cdc_handle_setup() Called by ISR after initialization

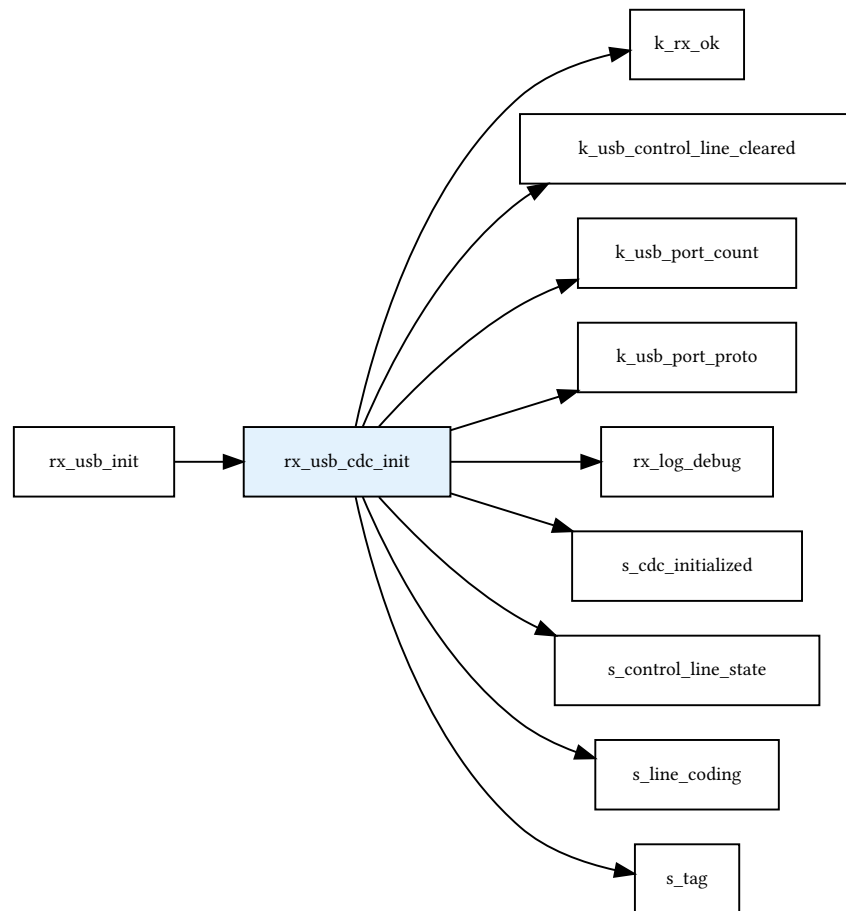


Figure 1225: Call graph for rx_usb_cdc_init

_Defined in `libs/rx_usb/src/rx_usb_internal.h:191_`

Since Version 1.5.0 (added 3rd port support)

—

rx_usb_cdc_handle_setup

```
void rx_usb_cdc_handle_setup(void)
```

Handle USB control transfer (SETUP stage).

Handle USB control transfer (SETUP stage).

Processes USB SETUP packets received on endpoint 0 (default control pipe) during the control transfer protocol. This is the main entry point for all USB host requests (standard, class-specific, vendor). Called from USB ISR when CTRT (Control Transfer) interrupt fires.

SETUP Packet Structure (8 bytes):

Request Processing Flow:

1. Read SETUP packet from USB registers (USBREQ, USBVAL, USBINDX, USBLENG)
2. Extract request type from bmRequestType bits [6:5]
3. Route to appropriate handler:

Standard requests (0b00) -> internal_handle_standard_request() Class requests (0b01) -> internal_handle_class_request() Vendor requests (0b10) -> STALL (not supported)

4. Handler sends data/status or STALL

Standard Requests Supported:

Class Requests Supported (CDC-ACM per port):

SETUP Packet Examples:

Example 1: GET_DESCRIPTOR(Device)

Example 2: SET_CONFIGURATION(1)

Example 3: SET_LINE_CODING (Port 0)

Error Handling:

- Unsupported request type -> Send STALL (DCPCTR.PID = STALL)
- Unknown descriptor -> Send STALL
- Invalid interface number -> Send STALL
- Pipe configuration failure -> Rollback + STALL

STALL response tells host "request not supported" - host may retry or continue with defaults.

Control Transfer Timing:

Execution Context: USB ISR (interrupt priority 5)

- [OK] No blocking operations
- [OK] Minimal execution time (<500 us worst-case)
- [OK] Re-entrant safe (no shared mutable state between SETUP packets)

Parameters:

Direction	Name	Description
in	None	(reads SETUP packet from USB registers)

Returns: void (no return value)

Pre: USB ISR context (called from usb0_usbi_isr())

Pre: CTRT interrupt flag set in INTSTS0

Pre: USB in Default, Addressed, or Configured state

Post: SETUP packet processed (data sent, status sent, or STALL sent)

Post: USB state may transition (Default->Addressed->Configured)

Post: Pipes may be configured (if SET_CONFIGURATION)

Note: NOT thread-safe - must only be called from USB ISR

Note: Reads from USB registers directly (USBREQ, USBVAL, USBINDX, USBLENG)

Note: Modifies USB registers (DCPCTR for status/STALL)

Warning: Do NOT call from application code (ISR-only function)

Warning: Do NOT block inside this function (breaks ISR timing)

Performance:: Typical: 10-100 us (descriptor fetch) Worst-case: 500 us (SET_CONFIGURATION with 9 pipes) Frequency: 10 Hz during enumeration, <1 Hz after configured

State Machine Impact:: GET_DESCRIPTOR -> No state change SET_ADDRESS -> Default -> Addressed SET_CONFIGURATION -> Addressed -> Configured Class requests -> No state change

Example Usage (from ISR):: voidusb0_usb0_isr(void){ uint16_tintsts0=usb0()->intsts0; if(intsts0&k_usb_intsts0_ctr){ usb0()->intsts0= k_usb_intsts0_ctr; rx_usb_cdc_handle_setup(); } }

Example Host Trace (lsusb -v):: #EnumerationsequencecapturedbyUSBPcap:

```
1.SETUP[8006000100001200]GET_DESCRIPTOR(Device) <-
DATA[12010002EF0201405B043502000101020301]

2.SETUP[0005070000000000]SET_ADDRESS(7) <-STATUS(zero-length)

3.SETUP[800600020000CF00]GET_DESCRIPTOR(Config) <-
DATA[207bytes:config+3IADs+6interfaces+9endpoints]

4.SETUP[0009010000000000]SET_CONFIGURATION(1) <-STATUS(zero-length)
[Devicenowreadyfordatatransfer]
```

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (if/switch, no goto) Rule 4: [OK] Short function (25 lines) Rule 5: [OK] 3 preconditions, 3 postconditions Rule 7: [OK] All return values checked or explicitly ignored

SOLID Principles:: S: Single responsibility (route SETUP packets only) O: Open/closed (adding request types extends handlers, not this function) I: Interface segregation (minimal entry point, delegates to specialized handlers)

Example:

```
Byte0:bmRequestType[D7=direction,D6:5=type,D4:0=recipient]
Byte1:bRequest[specificrequestcode]
Bytes2-3:wValue[request-specificparameter]
Bytes4-5:wIndex[interface/endpointnumber]
Bytes6-7:wLength[datastagelength]
```

Example:

```
bmRequestType=0x80[IN,Standard,Device]
bRequest=0x06[GET_DESCRIPTOR]
```

```
wValue=0x0100[Type=Device(01),Index=0]  
wIndex=0x0000  
wLength=0x0012[18bytesrequested]  
->Response:Sends_device_desc(18bytes)
```

Example:

```
bmRequestType=0x00[OUT,Standard,Device]  
bRequest=0x09[SET_CONFIGURATION]  
wValue=0x0001[Configuration1]  
wIndex=0x0000  
wLength=0x0000[Nodatastage]  
->Action:Configure9pipes,enableBRDY/BEMPinterrupts  
->Response:Sendzero-lengthstatuspacket(CCPL)
```

Example:

```
bmRequestType=0x21[OUT,Class,Interface]  
bRequest=0x20[SET_LINE_CODING]  
wValue=0x0000  
wIndex=0x0000[Interface0=Port0]  
wLength=0x0007[7bytes]  
Data:[00C20100][00][00][08]  
^115200bps^^8databits  
1stopNoparity  
->Action:Updates_line_coding[0]  
->Response:Sendzero-lengthstatuspacket(CCPL)
```

See also: `internal_handle_standard_request()` Processes standard USB requests,
`internal_handle_class_request()` Processes CDC class requests, `rx_usb_cdc_init()` Must
be called before this function, `usb0_usbi_isr()` ISR that calls this function

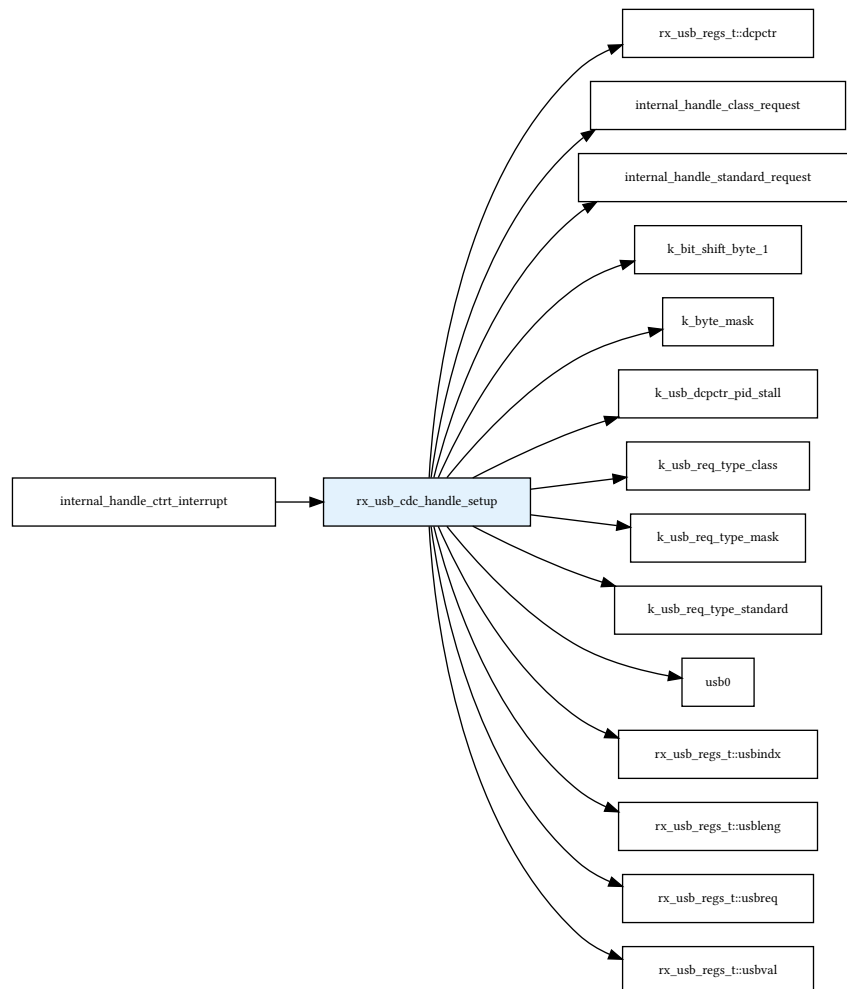


Figure 1226: Call graph for rx_usb_cdc_handle_setup

_Defined in libs/rx_usb/src/rx_usb_internal.h:194_

Since Version 1.5.0 (added 3rd port support)

—

rx_usb_cdc_handle_bulk_in

```
void rx_usb_cdc_handle_bulk_in(rx_usb_port_id_t port)
```

Handle bulk IN transfer completion for a port.

Handle bulk IN transfer completion for a port.

Processes bulk IN transfers for a specific CDC port when the USB hardware is ready to transmit more data to the host computer. Called from USB ISR when BEMP (Buffer Empty) interrupt fires for a bulk IN pipe. This is the transmission path for all serial data sent to the host.

Transmission Flow (RX72N -> Host):

Pipe-to-Port Mapping (Bulk IN):

Execution Sequence:

1. Validate port ID (0-2)
2. Get pipe number from port configuration table
3. Pop data from TX ring buffer (up to 64 bytes)
4. If len > 0, write to USB FIFO
5. Hardware sends packet to host
6. If TX buffer now empty, invoke TX_COMPLETE callback

Ring Buffer Behavior:

- Port 0: 1024-byte TX buffer (for binary protocol responses)
- Port 1: 512-byte TX buffer (for decoded status messages)
- Port 2: 2048-byte TX buffer (for log streams)
- BEMP interrupts continue until TX buffer completely drained
- After last packet, TX_COMPLETE callback invoked (if registered)

Zero-Length Packet Handling:

- If TX buffer empty (len == 0), no data written to FIFO
- USB hardware automatically handles ZLP for transfers ending on packet boundary
- Example: 64-byte transfer needs ZLP to signal end (USB spec requirement)

Transmission Completion Detection:

Error Cases:

- Invalid port ID -> Silent return (defensive check)
- nullptr config pointer -> Silent return (should never happen)
- FIFO write incomplete -> Logged by rx_usb_hw_fifo_write()
- No error state persists (next BEMP retry continues transmission)

Performance Characteristics:

Interrupt Frequency:

- Idle: 0 Hz (no application TX, no BEMP interrupts)
- Burst TX: Up to 18 kHz (1000 frames/sec x 18 packets/frame = saturated USB)
- CPU load: 0% idle, up to 12% saturated (7 us x 18 kHz)

Typical Transmission Timeline:

Execution Context: USB ISR (interrupt priority 5)

- [OK] No blocking operations
- [OK] Fast execution (<10 us typical)
- [OK] Re-entrant safe (per-port state isolation)
- [OK] Callback invoked from ISR context (callback must be ISR-safe)

Parameters:

Direction	Name	Description
in	port	Port ID to transmit from (k_usb_port_proto, k_usb_port_decoded, k_usb_port_log)

Returns: void (no return value)

Pre: USB ISR context (called from `usb0_usbi_isr()`)

Pre: BEMP interrupt for bulk IN pipe

Pre: USB in Configured state

Pre: `port < k_usb_port_count` (0-2)

Post: Data popped from TX ring buffer (if available)

Post: Data written to USB FIFO (if `len > 0`)

Post: TX_COMPLETE callback invoked (if TX buffer now empty and callback registered)

Note: NOT thread-safe - must only be called from USB ISR

Note: Callback (if invoked) executes in ISR context

Note: Zero-length pop (empty buffer) is normal and handled gracefully

Warning: Do NOT call from application code (ISR-only function)

Warning: Callback must be ISR-safe (no blocking, minimal execution time)

Warning: High-frequency BEMP interrupts can saturate CPU (12% @ full bandwidth)

Performance:: Typical: 5-7 us per 64-byte packet Worst-case: 10 us (with callback overhead)
Frequency: Up to 18 kHz @ full bandwidth CPU load: 0% idle, 12% saturated

Example Usage (from ISR):: `voidusb0_usbi_isr(void){ uint16_tbempsts=usb0()->bempsts;
if(bempsts&(1<<1)){ usb0()->bempsts= (1<<1); rx_usb_cdc_handle_bulk_in(k_usb_port_proto); }
if(bempsts&(1<<4)){ usb0()->bempsts= (1<<4); rx_usb_cdc_handle_bulk_in(k_usb_port_decoded); }
if(bempsts&(1<<7)){ usb0()->bempsts= (1<<7); rx_usb_cdc_handle_bulk_in(k_usb_port_log); } }`

Example Application Usage:: `voidsend_response(constuint8_t*data,uint32_tlen)
{ rx_err_terr=rx_usb_write(k_usb_port_proto,data,len); if(err!=k_rx_ok){ } }

voidmy_tx_callback(rx_usb_port_id_tport,rx_usb_event_tevent,void*ctx)
{ if(event==k_usb_event_tx_complete)
{ tx_event_flags_set(&usb_events,TX_DONE_FLAG,TX_OR); } }`

Example Host Read:: `#Linux:ReadresponsefromProtocolport(Port0) cat/dev/ttyACM0`

#This triggers (on RX72N side): #1. Application: rx_usb_write(0, "Responser", 9)
 #2. Data copied to Port0 TX ring buffer #3. First packet (9 bytes) loaded to FIFO
 #4. USB hardware sends bulk IN packet #5. BEMP interrupt fires
 #6. rx_usb_cdc_handle_bulk_in(0) pops next packet (none left) #7. TX_COMPLETE callback invoked #
 #Host sees: "Responser"

Large Transfer Example:: uint8_t data[1024]; memset(data, 'A', sizeof(data));
 rx_usb_write(k_usb_port_log, data, 1024);

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (no recursion, no goto) Rule 4:
 [OK] Short function (18 lines) Rule 5: [OK] 4 preconditions, 3 postconditions Rule 7: [OK]
 rx_usb_hw_fifo_write() return value explicitly ignored (errors logged internally)

SOLID Principles:: S: Single responsibility (pop from ring buffer, write to USB FIFO) D: Depends on
 abstractions (rx_usb_tx_pop, rx_usb_hw_fifo_write interfaces)

Example:

```
1. Application calls rx_usb_write(port, data, len)
v
2. Data copied to TX ring buffer (rx_usb.c)
v
3. First packet loaded into USB FIFO
v
4. USB hardware sends packet to host (up to 64 bytes)
v
5. BEMP interrupt fires (buffer empty, ready for more)
v
6. ISR calls rx_usb_cdc_handle_bulk_in(port) <- This function
v
7. Pops next packet from TX ring buffer via rx_usb_tx_pop()
v
8. Writes to USB FIFO via rx_usb_hw_fifo_write()
v
9. Repeats steps 4-8 until TX buffer empty
v
10. If TX buffer empty, invoke TX_COMPLETE callback
```

Example:

```
// After last packet sent:
rx_usb_tx_pop(port, ...) returns 0 (buffer empty)
v
No data written to FIFO
v
BEMP interrupt stops (no more data to send)
v
TX_COMPLETE callback invoked (by rx_usb_tx_pop())
```

Example:

```
T+0ms: rx_usb_write(port, 200 bytes)
T+0.1ms: First 64 bytes loaded to FIFO, packet sent
T+1.0ms: BEMP interrupt, load bytes 64-127
T+2.0ms: BEMP interrupt, load bytes 128-191
T+3.0ms: BEMP interrupt, load bytes 192-199 (final 8 bytes)
T+4.0ms: BEMP interrupt, TX buffer empty, TX_COMPLETE callback
```


See also: `rx_usb_cdc_handle_bulk_out()` Reception counterpart (host -> device), `rx_usb_write()` Application function to queue data for transmission, `rx_usb_tx_pop()` Internal function that pops data from ring buffer, `rx_usb_hw_fifo_write()` Hardware layer function that writes USB FIFO, `usb0_usbi_isr()` ISR that calls this function

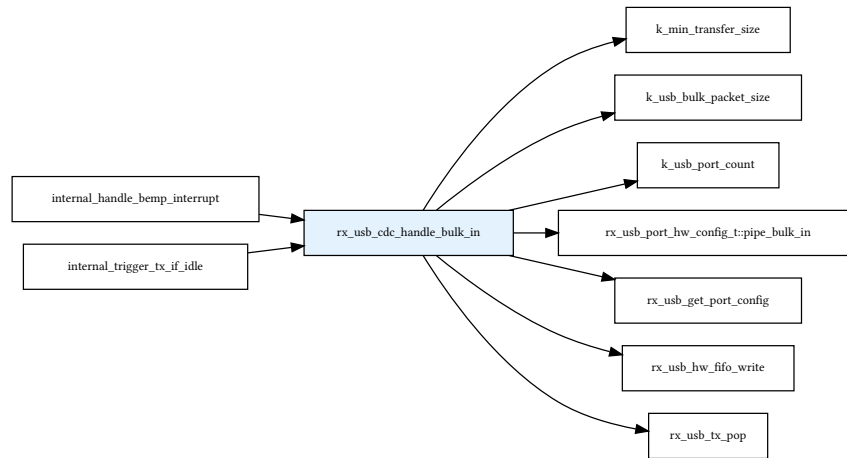


Figure 1227: Call graph for `rx_usb_cdc_handle_bulk_in`

_Defined in `libs/rx\usb/src/rx\usb\_internal.h:197_`

Since Version 1.5.0 (added Port 2 support)

—

`rx_usb_cdc_handle_bulk_out`

```
void rx_usb_cdc_handle_bulk_out(rx_usb_port_id_t port)
```

Handle bulk OUT data received for a port.

Handle bulk OUT data received for a port.

Processes bulk OUT transfers for a specific CDC port when data arrives from the host computer. Called from USB ISR when BRDY (Buffer Ready) interrupt fires for a bulk OUT pipe. This is the reception path for all serial data sent by the host.

Reception Flow (Host -> RX72N):

Pipe-to-Port Mapping (Bulk OUT):

Execution Sequence:

1. Validate port ID (0-2)
2. Get pipe number from port configuration table
3. Read data from USB FIFO (up to 64 bytes)
4. If len > 0, push to RX ring buffer
5. Ring buffer invokes `RX_AVAILABLE` callback if registered

Ring Buffer Behavior:

- Port 0: 1024-byte RX buffer (for binary protocol frames)
- Port 1: 512-byte RX buffer (for decoded commands)

- Port 2: 2048-byte RX buffer (for log messages)
- If buffer full, new data discarded (logged by rx_usb_rx_push())
- Application must read promptly to avoid overflow

Zero-Length Packet Handling:

- If len == 0, no data pushed to ring buffer
- No callback invoked for zero-length packets
- This is normal USB behavior (ZLP for transaction termination)

Error Cases:

- Invalid port ID -> Silent return (defensive check)
- nullptr config pointer -> Silent return (should never happen)
- FIFO read incomplete -> Logged by rx_usb_hw_fifo_read()
- Ring buffer full -> Logged by rx_usb_rx_push(), data lost

Performance Characteristics:

Interrupt Frequency:

- Idle: 0 Hz (no host traffic)
- Low traffic: 1-10 Hz (occasional commands)
- High traffic: Up to 18 kHz (1000 frames/sec x 18 packets/frame = saturated USB)
- CPU load: 0.1% idle, up to 12% saturated (7 us x 18 kHz)

Execution Context: USB ISR (interrupt priority 5)

- [OK] No blocking operations
- [OK] Fast execution (<10 us typical)
- [OK] Re-entrant safe (per-port state isolation)
- [OK] Callback invoked from ISR context (callback must be ISR-safe)

Parameters:

Direction	Name	Description
in	port	Port ID that received data (k_usb_port_proto, k_usb_port_decoded, k_usb_port_log)

Returns: void (no return value)

Pre: USB ISR context (called from usb0_usbi_isr())

Pre: BRDY interrupt for bulk OUT pipe

Pre: USB in Configured state

Pre: port < k_usb_port_count (0-2)

Post: Data read from USB FIFO

Post: Data pushed to RX ring buffer (if len > 0)

Post: RX_AVAILABLE callback invoked (if registered and buffer not empty)

Note: NOT thread-safe - must only be called from USB ISR

Note: Callback (if invoked) executes in ISR context

Note: Zero-length packets are valid and handled gracefully

Warning: Do NOT call from application code (ISR-only function)

Warning: Callback must be ISR-safe (no blocking, minimal execution time)

Warning: Ring buffer overflow causes data loss (application must read promptly)

Performance:: Typical: 5-7 us per 64-byte packet Worst-case: 10 us (with callback overhead)

Frequency: Up to 18 kHz @ full bandwidth CPU load: 0.1-12% (depends on traffic)

Example Usage (from ISR):: voidusb0_usbi_isr(void){ uint16_tbrdysts=usb0()->brdysts;

if(brdysts&(1<<2)){ usb0()->brdysts= (1<<2); rx_usb_cdc_handle_bulk_out(k_usb_port_proto); }

if(brdysts&(1<<5)){ usb0()->brdysts= (1<<5); rx_usb_cdc_handle_bulk_out(k_usb_port_decoded); }

if(brdysts&(1<<8)){ usb0()->brdysts= (1<<8); rx_usb_cdc_handle_bulk_out(k_usb_port_log); } }

Example Application Usage:: voidmy_task(void){ uint8_tbuffer[256]; uint32_tlen;

len=rx_usb_read(k_usb_port_proto,buffer,sizeof(buffer)); if(len>0){ process_command(buffer,len); } }

voidmy_rx_callback(rx_usb_port_id_tport,rx_usb_event_tevent,void*ctx)

{ if(event==k_usb_event_rx_available){ tx_event_flags_set(&usb_events,(1<<port),TX_OR); } }

Example Host Command:: #Linux:SendcommandtoProtocolport(Port0) echo-n"HelloRX72N">/dev/ttyACM0

#This triggers: #1.HostsendsbulkOUTpacket:"HelloRX72N"(11bytes) #2.BRDYinterruptfiresforpipe2

#3.rx_usb_cdc_handle_bulk_out(0)reads11bytesfromFIFO #4.DatapushedtoPort0RXringbuffer

#5.RX_AVAILABLEcallbackinvoked(ifregistered) #6.Applicationreadsviarx_usb_read(0,...)

NASA Power of 10 Compliance:: Rule 1: [OK] Simple control flow (no recursion, no goto) Rule 4:

[OK] Short function (20 lines) Rule 5: [OK] 4 preconditions, 3 postconditions Rule 7: [OK]

rx_usb_rx_push() return value explicitly ignored (errors logged internally)

SOLID Principles:: S: Single responsibility (read USB FIFO, push to ring buffer) D: Depends on

abstractions (rx_usb_hw_fifo_read, rx_usb_rx_push interfaces)

Example:

1.HostsendsbulkOUTpacket(upto64bytes)

v

2.USBhardwarereceivespacketintoFIFO

v

3.BRDYinterruptfires(pipe-specificbitset)

```

v
4.ISRcallsrx_usb_cdc_handle_bulk_out(port)<-Thisfunction
v
5.ReaddatafromUSBFI0viarx_usb_hw_fifo_read()
v
6.PushdatatoRXringbufferviarx_usb_rx_push()
v
7.IfRX_AVAILABLEcallbackregistered,invokecallback
v
8.Applicationcallsrx_usb_read(port)toconsumedata

```

See also: rx_usb_cdc_handle_bulk_in() Transmission counterpart (device -> host), rx_usb_read() Application function to consume received data, rx_usb_rx_push() Internal function that pushes data to ring buffer, rx_usb_hw_fifo_read() Hardware layer function that reads USB FIFO, usb0_usbi_isr() ISR that calls this function

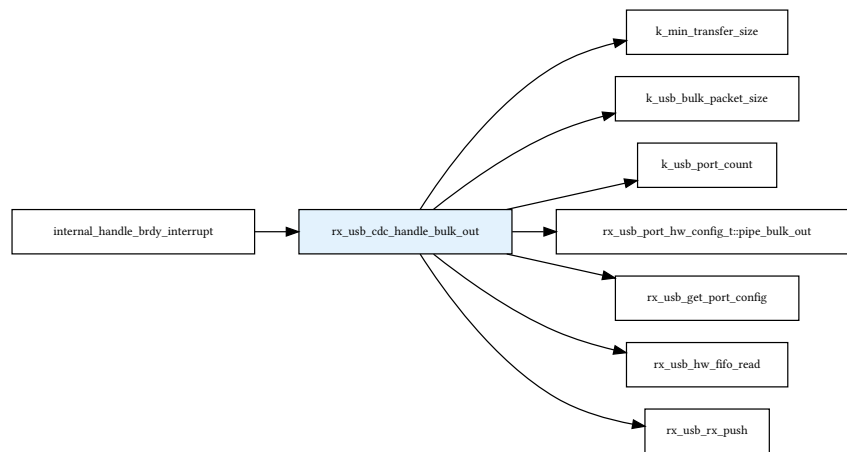


Figure 1228: Call graph for rx_usb_cdc_handle_bulk_out

_Defined in `libs/rx_usb/src/rx_usb_internal.h:200_`

Since Version 1.5.0 (added Port 2 support)

—

rx_usb_isr.c

USB0 Interrupt Service Routine for RX72N Multi-Port CDC Composite Device.

Includes:

- `"rx72n_regs.h"`
- `"rx_log.h"`
- `"rx_usb.h"`
- `"rx_usb_internal.h"`

usb_pipe_numbers_t

USB pipe number constants.

Value	Name	Description
-------	------	-------------

= 0	k_usb_pipe_dcp	Default Control Pipe (DCP) number
= 9	k_usb_pipe_max	Maximum pipe number (pipes 0-9)

—

usb_pipe_assignment_t

Pipe assignments for multi-port CDC.

Value	Name	Description
= 1	k_port0_pipe_bulk_in	Port 0: Bulk IN pipe
= 2	k_port0_pipe_bulk_out	Port 0: Bulk OUT pipe
= 3	k_port0_pipe_int_in	Port 0: Interrupt IN pipe
= 4	k_port1_pipe_bulk_in	Port 1: Bulk IN pipe
= 5	k_port1_pipe_bulk_out	Port 1: Bulk OUT pipe
= 6	k_port1_pipe_int_in	Port 1: Interrupt IN pipe
= 7	k_port2_pipe_bulk_in	Port 2: Bulk IN pipe
= 8	k_port2_pipe_bulk_out	Port 2: Bulk OUT pipe
= 9	k_port2_pipe_int_in	Port 2: Interrupt IN pipe

—

s_tag

```
const char* s_tag
```

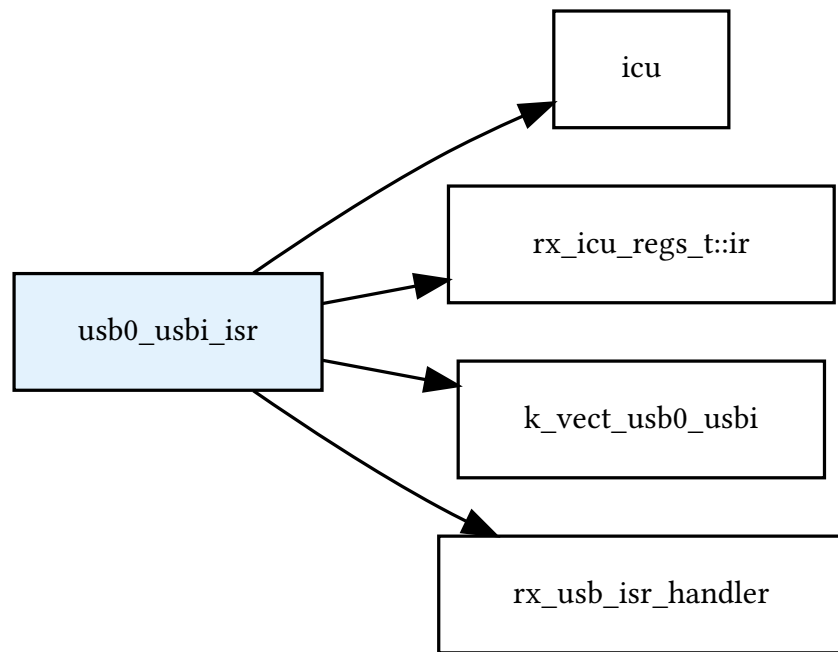
—

usb0_usbi_isr

```
void usb0_usbi_isr(void)
```

USB0 USBI Interrupt Handler.

This is the actual interrupt handler that is registered in the vector table. It calls the main ISR handler.

Figure 1229: Call graph for `usb0_usbi_isr`

_Defined in `libs/rx_usb/src/rx_usb_isr.c:624_`

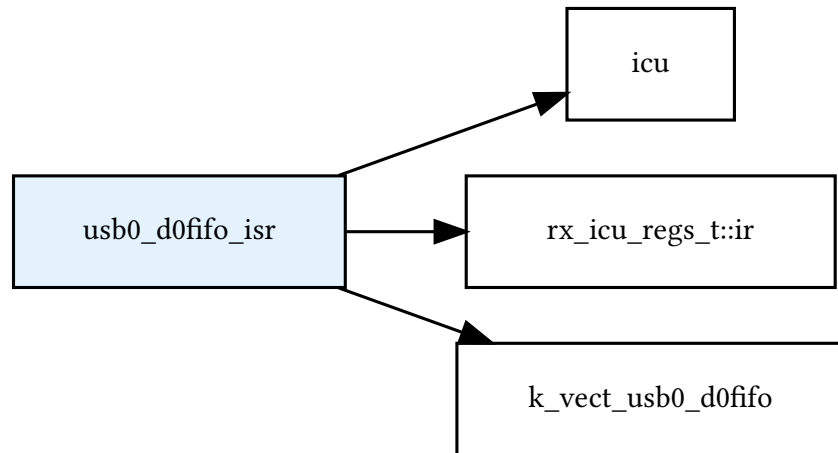
—

usb0_d0fifo_isr

```
void usb0_d0fifo_isr(void)
```

USB0 D0FIFO Interrupt Handler.

DMA-related interrupt for D0FIFO (not used in this implementation).

Figure 1230: Call graph for `usb0_d0fifo_isr`

Defined in `libs/rx_usb/src/rx_usb_isr.c:638`

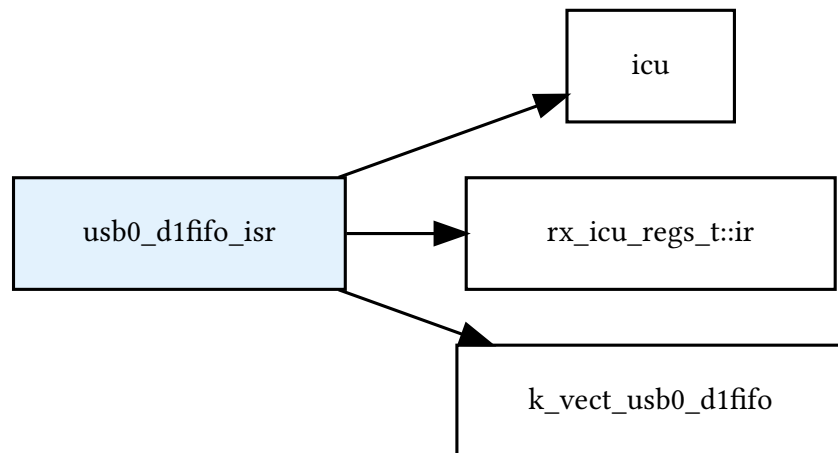
—

usb0_d1fifo_isr

```
void usb0_d1fifo_isr(void)
```

USB0 D1FIFO Interrupt Handler.

DMA-related interrupt for D1FIFO (not used in this implementation).

Figure 1231: Call graph for `usb0_d1fifo_isr`

Defined in `libs/rx_usb/src/rx_usb_isr.c:651`

—

usb0_usbr_isr

```
void usb0_usbr_isr(void)
```

USB0 Resume Interrupt Handler.

Separate resume interrupt for wakeup from low-power modes.

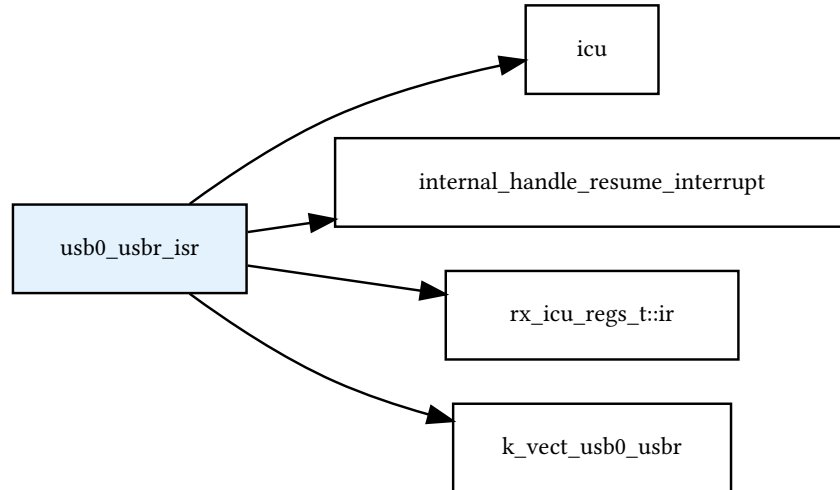


Figure 1232: Call graph for `usb0_usbr_isr`

_Defined in `libs/rx\usb/src/rx\usb_isr.c:664_`

—

internal_handle_vbus_interrupt

```
static void internal_handle_vbus_interrupt(void)
```

Handle VBUS interrupt (cable connect/disconnect).

Figure 1233: Call graph for `internal_handle_vbus_interrupt`

_Defined in `libs/rx\usb/src/rx\usb_isr.c:357_`

internal_handle_dvst_interrupt

```
static void internal_handle_dvst_interrupt(void)
```

Handle device state transition interrupt.

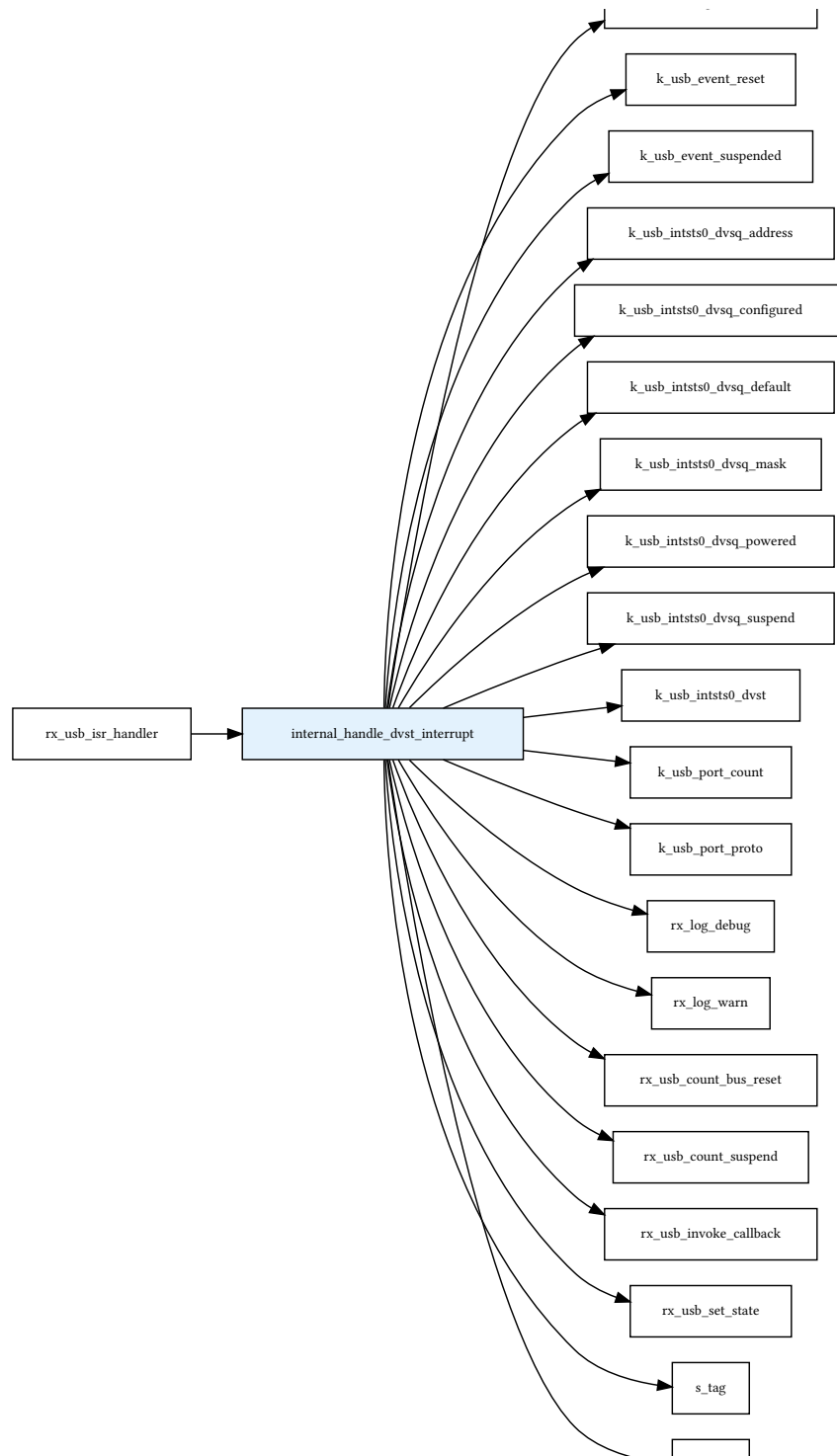


Figure 1234: Call graph for internal_handle_dvst_interrupt

_Defined in libs/rx_usb/src/rx_usb_isr.c:384_

—

internal_handle_ctr_t_interrupt

```
static void internal_handle_ctr_t_interrupt(void)
```

Handle control transfer stage transition interrupt.

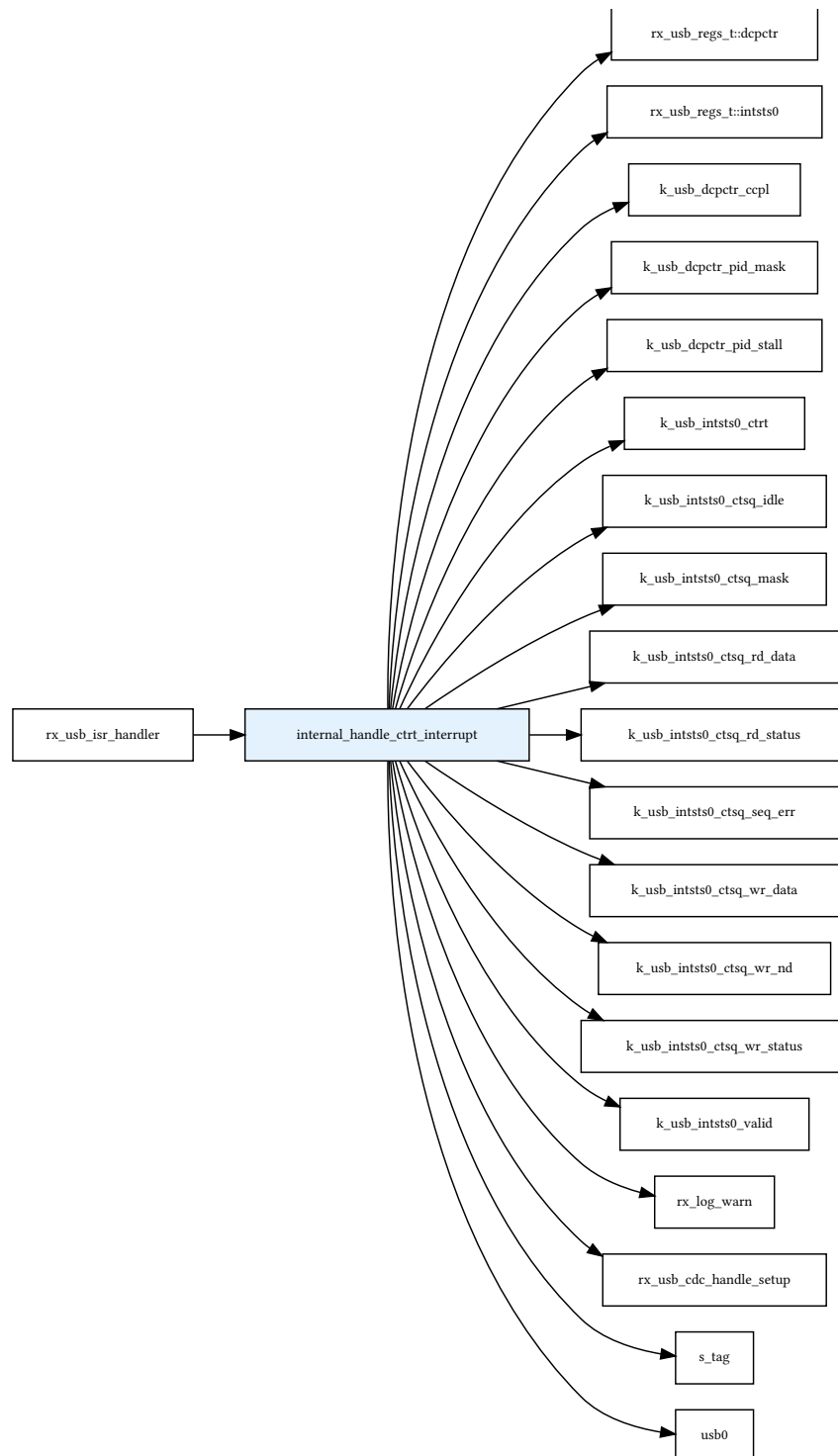


Figure 1235: Call graph for internal_handle_crt_interrupt

_Defined in libs/rx_usb/src/rx_usb_isr.c:433_
—

internal_handle_brdy_interrupt

```
static void internal_handle_brdy_interrupt(void)
```

Handle buffer ready interrupt (data received) for multi-port CDC.

Routes the interrupt to the correct port based on pipe number.

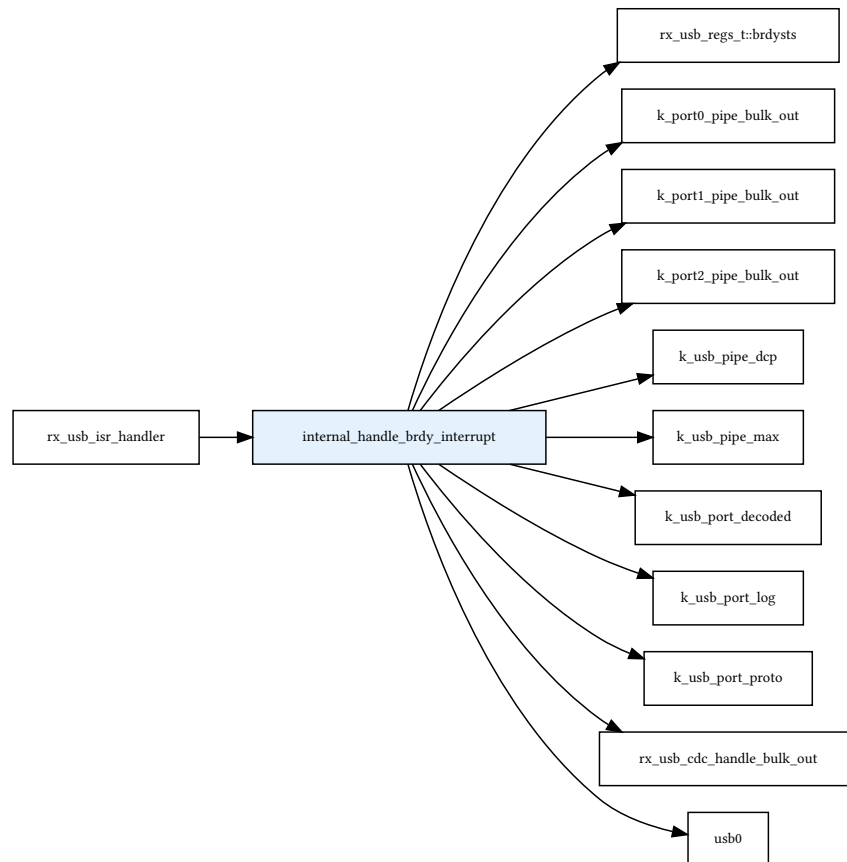


Figure 1236: Call graph for internal_handle_brdy_interrupt

_Defined in libs/rx_usb/src/rx_usb_isr.c:488_
—

internal_handle_bemp_interrupt

```
static void internal_handle_bemp_interrupt(void)
```

Handle buffer empty interrupt (transmission complete) for multi-port CDC.

Routes the interrupt to the correct port based on pipe number.

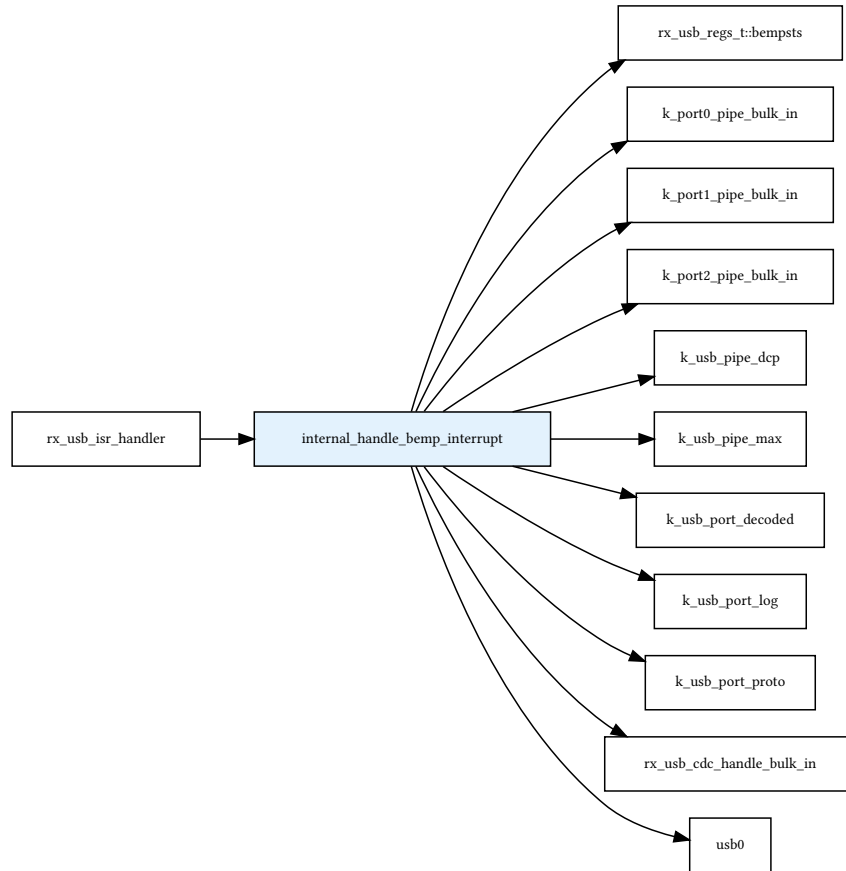


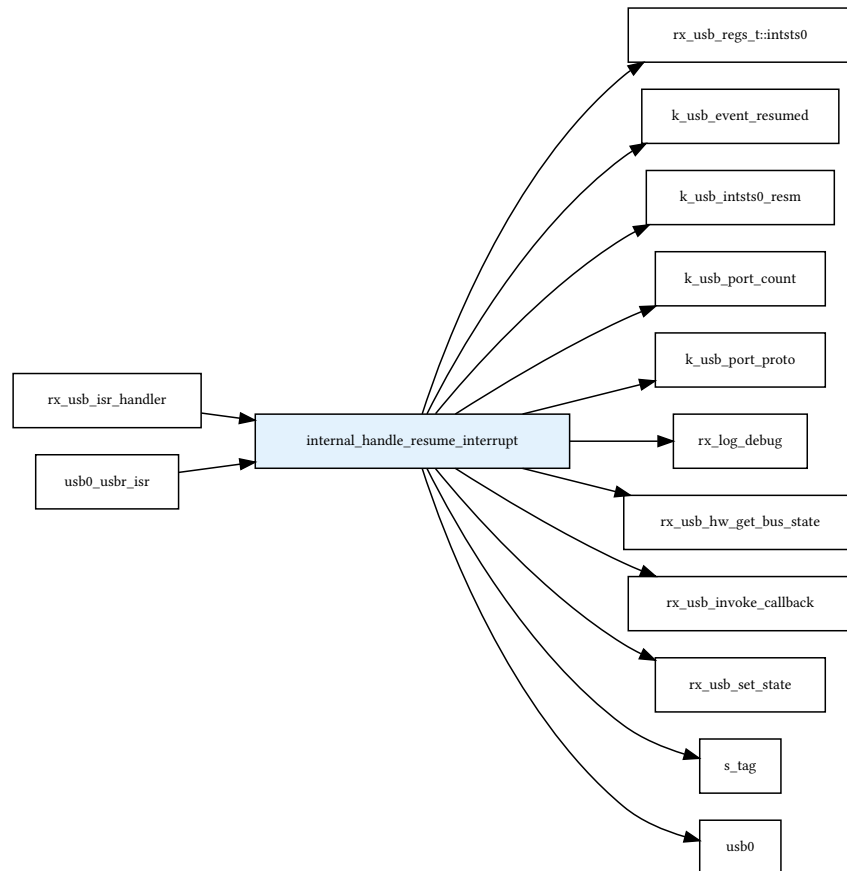
Figure 1237: Call graph for `internal_handle_bemp_interrupt`

_Defined in `libs/rx\usb/src/rx\usb\_isr.c:520_`

internal_handle_resume_interrupt

```
static void internal_handle_resume_interrupt(void)
```

Handle resume interrupt.

Figure 1238: Call graph for `internal_handle_resume_interrupt`

_Defined in `libs/rx_usb/src/rx_usb_isr.c:550_`

rx_usb_isr_handler

```
void rx_usb_isr_handler(void)
```

USB0 Interrupt Service Routine.

This function is called from the vector table when a USB interrupt occurs. It reads the interrupt status register and dispatches to the appropriate handler. For data pipe interrupts, it routes to the correct port.

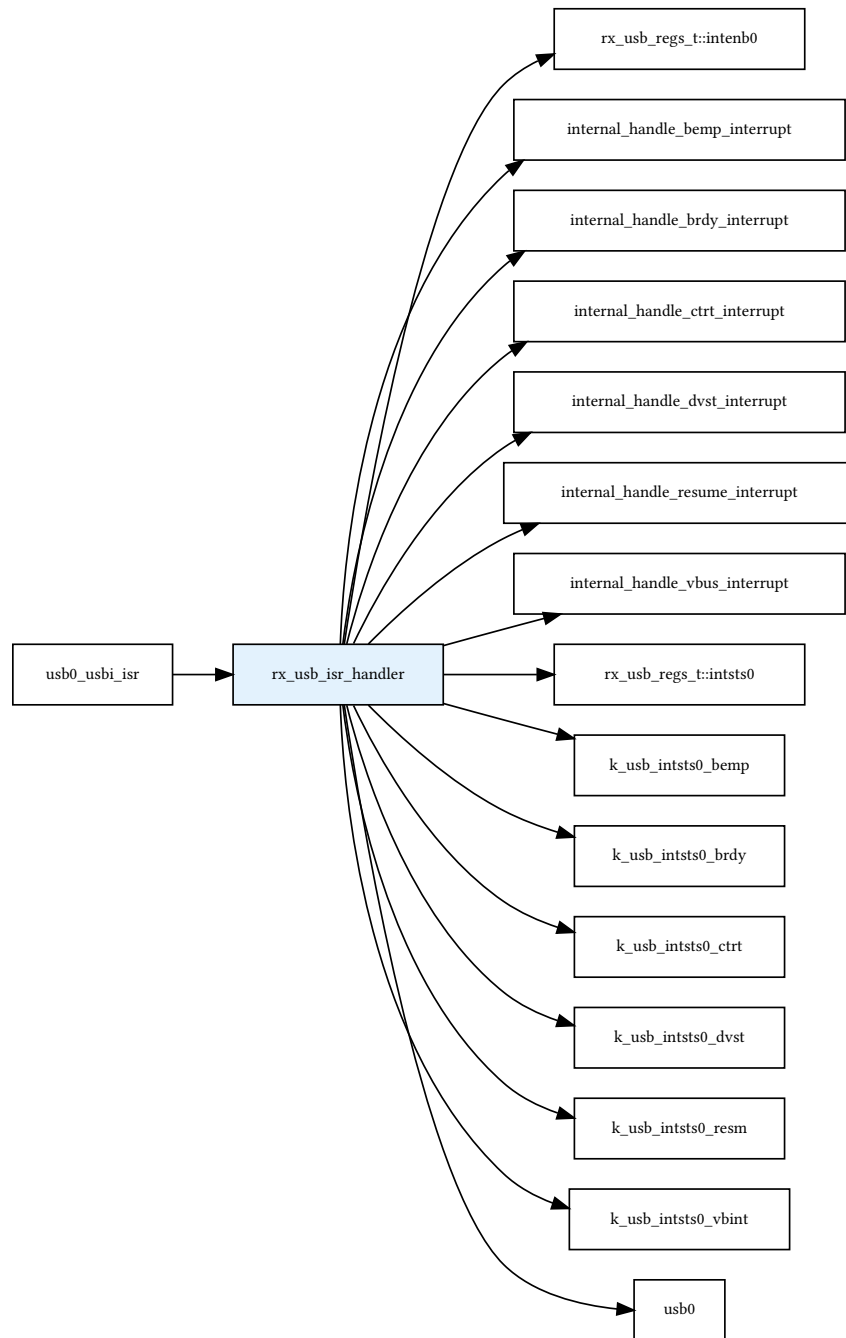


Figure 1239: Call graph for rx_usb_isr_handler

_Defined in libs/rx_usb/src/rx_usb_isr.c:579_

rx_usb_comm.h

High-Level USB CDC Communication Layer for RX72N.

Integrates Frame Layer and USB CDC for reliable communication with the RPi5 controller. Provides the same frame-based protocol as SPI but over USB CDC-ACM virtual serial port. This module enables debug and control communication without requiring the SPI interface.

STAR Team

2026-01-28

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- <stdbool.h>
- <stdint.h>
- "rx_err.h"
- "rx_frame.h"
- "rx_session.h"
- "rx_time_interface.h"

rx_usb_comm_constants_t

USB communication configuration constants.

Compile-time constants for USB communication buffer sizes, timeouts, and loop bounds. These values are tuned for the RX72N USB Full Speed implementation with typical control message sizes.

Buffer sizes must be $\geq$ maximum frame size

k_usb_comm_max_receive_iterations bounds all receive loops

Value	Name	Description
= 2048	k_usb_comm_rx_buffer_size	RX staging buffer size in bytes.
= 2048	k_usb_comm_tx_buffer_size	TX staging buffer size in bytes.
= 1000	k_usb_comm_default_timeout	Default timeout for blocking operations (milliseconds).
= 1000	k_usb_comm_max_receive_iterations	Maximum iterations for receive loop (NASA Rule 2).

Since Version 1.0.0

—

rx_usb_comm_mode_t

USB communication operating mode.

Controls whether the USB CDC communication layer operates in binary protocol mode (for robot control via protobuf) or debug text mode (for terminal access via screen/minicom). Mode can be changed at runtime via rx_usb_comm_set_mode().

Value	Name	Description
= 0xFF	k_usb_comm_mode_invalid	Invalid/uninitialized mode sentinel.
= 0	k_usb_comm_mode_binary	Binary protocol mode (default after init).

= 1	k_usb_comm_mode_debug	Debug text mode for terminal access.
-----	-----------------------	--------------------------------------

Note: Mode only affects this communication layer, not the underlying USB driver] **See also:** rx_usb_comm_set_mode() Change mode at runtime, rx_usb_comm_get_mode() Query current mode

Since Version 1.0.0

—

rx_usb_comm_init

```
rx_err_t rx_usb_comm_init(rx_usb_comm_handle_t *handle, const
rx_usb_comm_config_t *config)
```

Initialize USB communication handle.

Sets up frame encoder/decoder, clears internal buffers, and prepares the handle for communication. The USB driver must be initialized separately via rx_usb_init() before this handle can successfully send/receive data.

Initialize USB communication handle.

Initializes the USB communication handle for framed communication. Sets up the frame encoder/decoder, configures FEC and time interface, and prepares buffer state. Must be called before any other rx_usb_comm functions.

Parameters:

Direction	Name	Description
out	handle	Pointer to handle to initialize Must be valid non-nullptr All fields will be overwritten Caller maintains ownership
in	config	Configuration options (must not be NULL) session: Required shared session state pointer time_iface: Optional time interface (NULL for defaults) Structure is copied, need not persist
out	handle	USB communication handle to initialize
in	config	Configuration parameters (must not be NULL) session: Required shared session state pointer time_iface: Time interface for sleep operations (nullptr = no sleep)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Handle initialized successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_ok	Success, handle ready for use
k_rx_err_invalid_arg	nullptr handle pointer
k_rx_err_invalid_state	Post-condition validation failed

Other	errors from rx_frame_encoder_init() or rx_frame_decoder_init()
Pre: handle points to allocated rx_usb_comm_handle_t	
Pre: USB driver initialized via rx_usb_init() (for actual communication)	
Pre: handle != nullptr	
Pre: handle memory writable (sizeof(rx_usb_comm_handle_t) bytes)	
Post: handle->initialized == 1	
Post: handle->mode == k_usb_comm_mode_binary	
Post: Sequence counters reset to 0	
Post: Buffers cleared	
Post: handle->initialized == true on success	
Post: handle->mode == k_usb_comm_mode_binary on success	
Post: handle->session points to shared session state	
Note: This handle operates on Port 0 (k_usb_port_proto) for protocol	
Note: Safe to call multiple times (reinitializes)	
Note: Not thread-safe during initialization	
Note: USB hardware must be initialized separately via rx_usb_init() Algorithm: Validate handle pointer Clear handle memory (memset to 0) Initialize frame encoder and decoder Copy configuration (or use defaults if nullptr) Set mode to k_usb_comm_mode_binary Set initialized flag	

Example: static rx_usb_comm_handle_t usb_comm;

void init(void) { rx_usb_init();

rx_err_t err = rx_usb_comm_init(&usb_comm, NULL); if (err != k_rx_ok) { }

Performance: 50-100 us (buffer clearing + encoder/decoder init)

Algorithm:: Validate handle pointer Clear handle to zero state Apply configuration (or defaults if nullptr) Initialize frame encoder Initialize frame decoder (cleanup encoder on failure) Initialize sequence counters to 0 Initialize buffer state Set mode to binary (default) Mark handle as initialized Validate post-conditions

Example:: rx_usb_comm_handle_thandle;
rx_usb_comm_config_tconfig={ .session=&g_session, .time_iface=&time_interface };
rx_err_t err=rx_usb_comm_init(&handle,&config); if(err!=k_rx_ok)
{ rx_log_error("USB","Initfailed"); }

See also: rx_usb_comm_deinit() Cleanup resources, rx_usb_init() USB driver initialization (prerequisite), rx_usb_comm_deinit() Cleanup function, rx_usb_init() USB hardware initialization

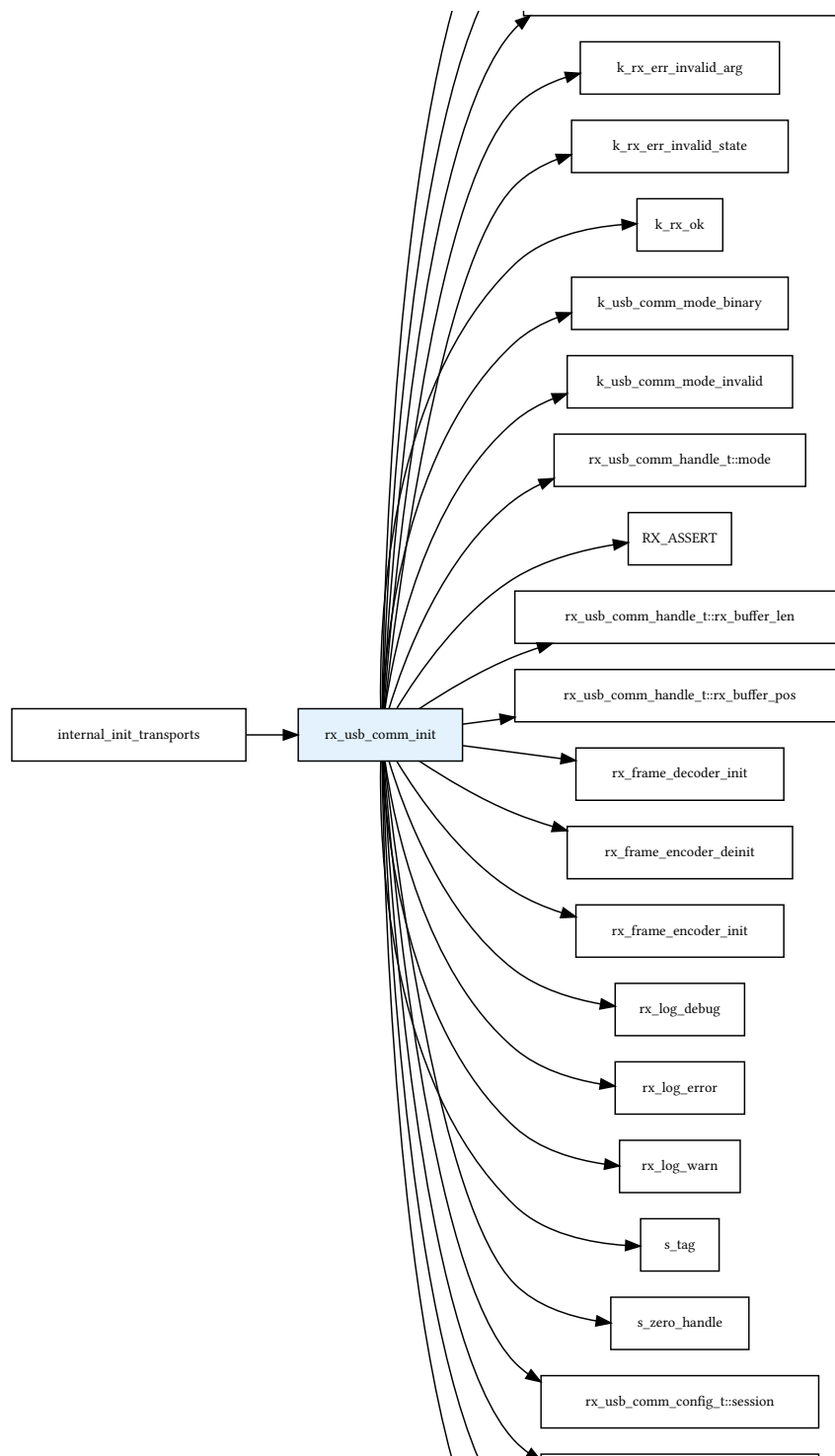


Figure 1240: Call graph for rx_usb_comm_init

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:650`

Since Version 1.0.0

—

rx_usb_comm_deinit

```
rx_err_t rx_usb_comm_deinit(rx_usb_comm_handle_t *handle)
```

Deinitialize USB communication handle.

Deinitialize USB communication handle.

Releases resources associated with the USB communication handle. Deinitializes both frame encoder and decoder. Safe to call on uninitialized handle (logs assertion but doesn't crash).

Parameters:

Direction	Name	Description
inout	handle	Pointer to handle
inout	handle	USB communication handle to deinitialize

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success (or handle was not initialized)
k_rx_err_invalid_arg	nullptr handle pointer
Other	errors from encoder/decoder deinit (first error returned)

Pre: handle != nullptr

Pre: handle->initialized == true (asserted, not enforced)

Post: handle->initialized == false

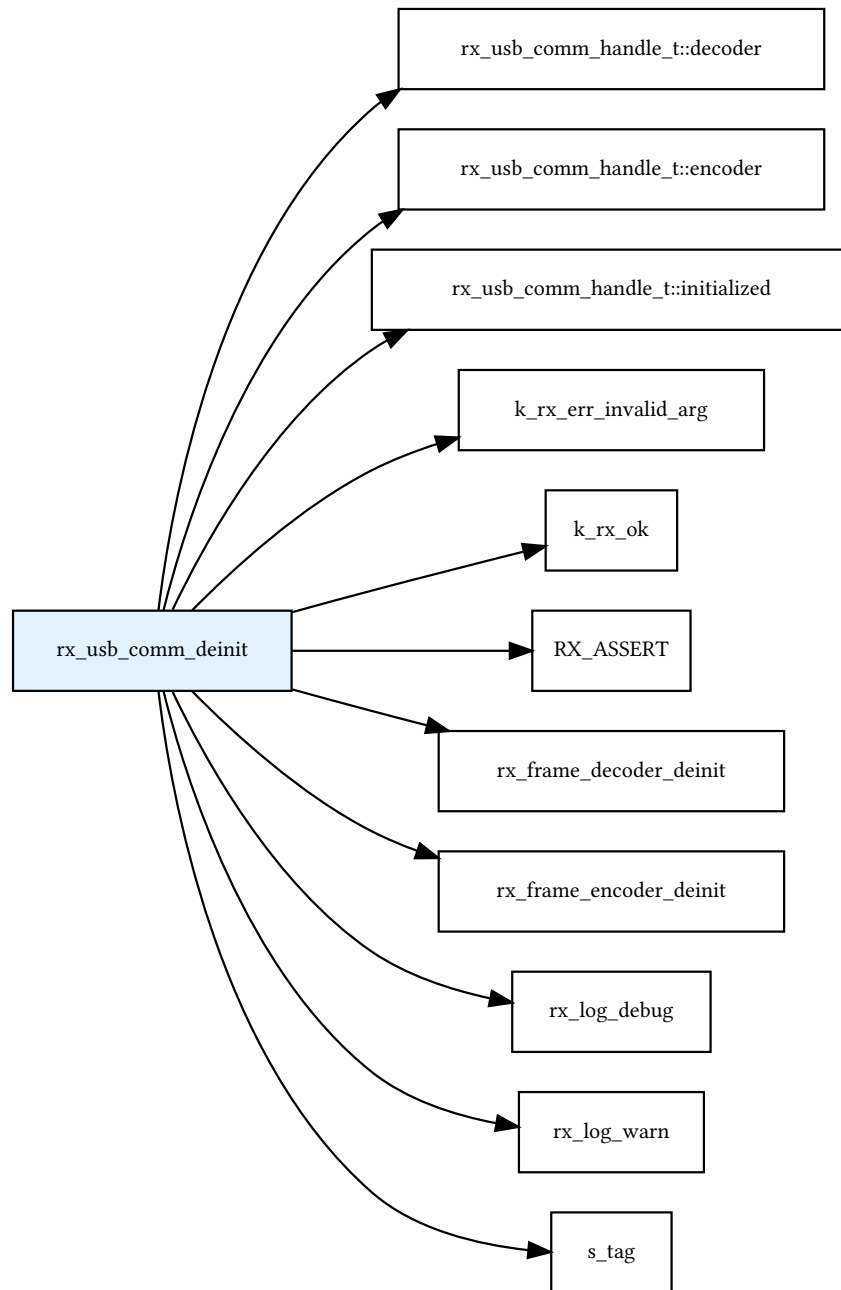
Post: Frame encoder/decoder resources released

Note: Safe to call multiple times (idempotent after first call)

Note: Logs warning but continues if sub-deinit fails

Algorithm:: Validate handle pointer Assert handle was initialized (catches programming errors) If initialized, deinit encoder (log warning on failure) Deinit decoder (log warning on failure) Mark handle as not initialized Return first error encountered (or k_rx_ok)

See also: rx_usb_comm_init() Initialize before use

Figure 1241: Call graph for `rx_usb_comm_deinit`

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:659`

rx_usb_comm_send

```
rx_err_t rx_usb_comm_send(rx_usb_comm_handle_t *handle, rx_frame_type_t type,
uint8_t flags, const uint8_t *payload, uint32_t payload_len)
```

Send a response frame over USB.

Encodes the payload into a frame with automatic sequence number assignment and CRC-32 calculation. The frame is queued to the USB TX buffer for transmission. This operation is non-blocking - it returns as soon as the data is queued (not when transmission completes).

Parameters:

Direction	Name	Description
inout	handle	Initialized handle Must be initialized via rx_usb_comm_init() Session TX sequence incremented on success
in	type	Frame type (typically k_frame_type_response) See rx_frame_type_t for valid types
in	flags	Frame flags (ORed together) See rx_frame_flags_t for valid flags 0 for no flags
in	payload	Pointer to payload data Must not be nullptr (even for zero-length payload, use dummy) Copied to internal buffer (no ownership transfer)
in	payload_len	Payload length in bytes Max: k_usb_comm_tx_buffer_size - frame overhead (2036 bytes) 0 is valid (empty payload)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Frame queued successfully
k_rx_err_invalid_arg	handle or payload is nullptr
k_rx_err_invalid_state	Not initialized, wrong mode, or USB not configured
k_rx_err_invalid_size	Payload exceeds maximum frame size
k_rx_err_busy	USB TX buffer full, try again later

Pre: handle initialized via rx_usb_comm_init()

Pre: handle->mode == k_usb_comm_mode_binary

Pre: USB driver initialized and configured

Pre: payload !=nullptr

Post: On success: session TX sequence incremented

Post: On success: frame queued for transmission

Post: On failure: no state changes

Note: Non-blocking: returns immediately after queueing

Note: Sends on Port 0 (k_usb_port_proto)

Note: Thread safety: caller must provide external synchronization

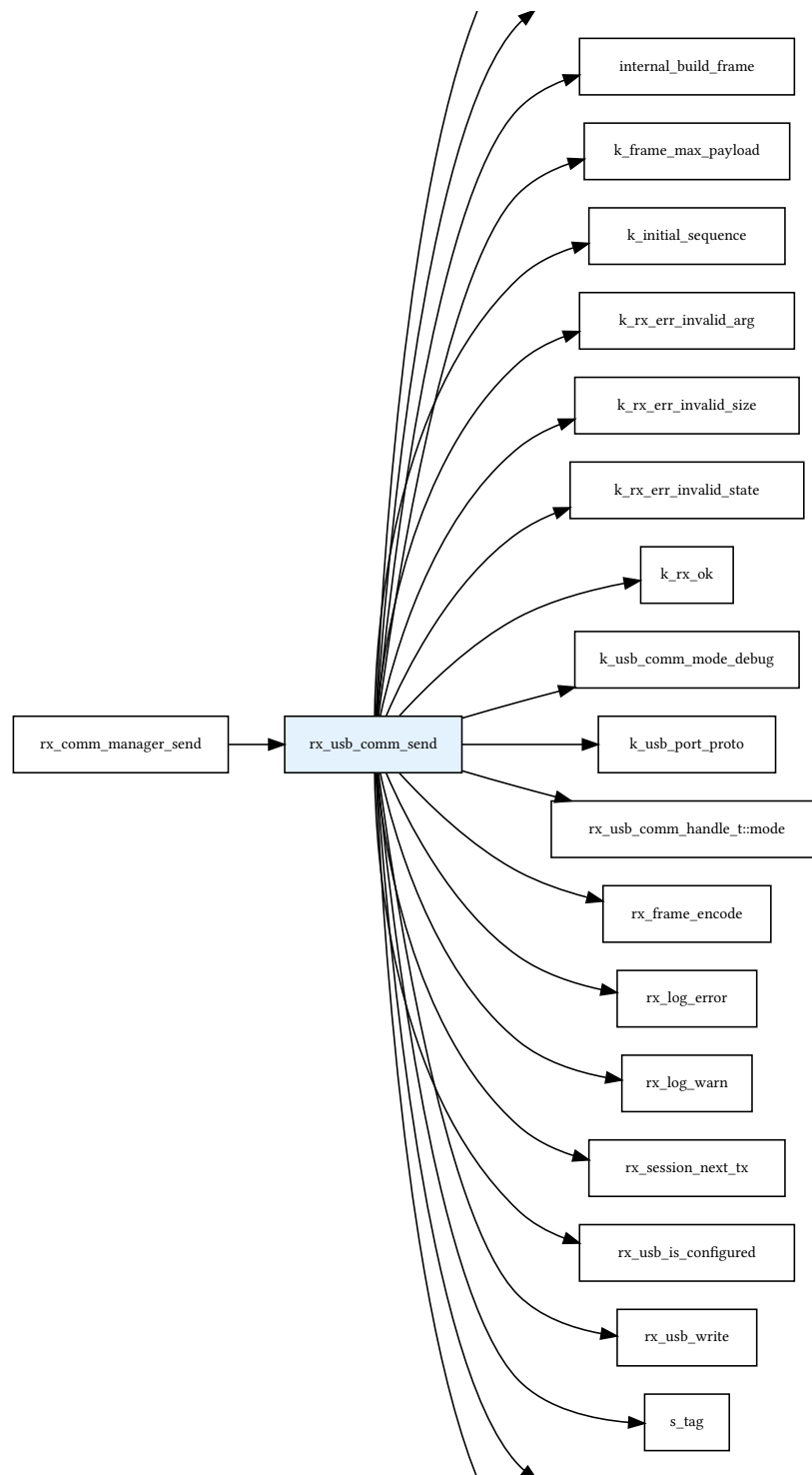
Warning: In debug mode, returns k_rx_err_invalid_state] **Frame Structure (Generated):** +---
 +---+---+---+---+---+---+---+ |SYNC|TYPE|FLAG|SEQ|LENGTH|PAYLOAD|CRC32| |0xAA|
 type|flags|auto|len|data|auto| +---+---+---+---+---+---+---+ +

Algorithm: Validate handle, payload pointers Check initialization and mode (must be binary mode)
 Verify USB is configured and ready Check payload fits in buffer Build frame header with current
 sequence number Copy payload to frame Calculate and append CRC-32 Queue frame to USB TX
 buffer Increment sequence counter

Example: uint8_tpb_buffer[256];
 size_tpb_len=encode_protobuf_response(pb_buffer,sizeof(pb_buffer));
 rx_err_terr=rx_usb_comm_send(&handle, k_frame_type_response, 0, pb_buffer, (uint32_t)pb_len);
 if(err==k_rx_err_busy){ tx_thread_sleep(1); err=rx_usb_comm_send(...); }

Performance: 10-50 us (encoding + USB queue)

See also: rx_usb_comm_receive() Receive frames from peer, rx_frame_type_t Frame type
 definitions

Figure 1242: Call graph for `rx_usb_comm_send`

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:760`

Since Version 1.0.0

—

rx_usb_comm_receive

```
rx_err_t rx_usb_comm_receive(rx_usb_comm_handle_t *handle, rx_frame_t *frame,
uint32_t timeout_ms)
```

Receive and decode a frame from USB.

Reads data from the USB CDC buffer, feeds it through the frame decoder, validates CRC-32, and extracts the payload. The function can operate in blocking mode (with timeout) or polling mode (timeout_ms = 0).

Parameters:

Direction	Name	Description
inout	handle	Initialized handle Must be initialized via rx_usb_comm_init() Internal buffers and decoder state modified
out	frame	Pointer to frame structure for output Must be valid non-nullptr All fields populated on success
in	timeout_ms	Timeout in milliseconds 0: Poll once, return immediately if no frame >0: Block up to timeout_ms waiting for complete frame

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Complete frame received and decoded
k_rx_err_invalid_arg	handle or frame is nullptr
k_rx_err_invalid_state	Not initialized or USB not configured
k_rx_err_timeout	No complete frame within timeout (timeout_ms > 0)
k_rx_err_no_data	No data available (timeout_ms == 0)
k_rx_err_crc_mismatch	CRC-32 validation failed
k_rx_err_protocol_error	Invalid frame format (bad sync, length)

Pre: handle initialized via rx_usb_comm_init()

Pre: USB driver initialized and configured

Pre: frame points to valid rx_frame_t

Post: On success: frame contains decoded data

Post: On success: session RX sequence updated

Post: On CRC error: frame may be partially populated

Note: Receives from Port 0 (k_usb_port_proto)

Note: Loop iterations bounded by k_usb_comm_max_receive_iterations

Note: Thread safety: caller must provide external synchronization] **Algorithm:** Validate handle and frame pointers Check initialization and USB ready state Loop until frame complete or timeout: a. Read available bytes from USB buffer b. Feed bytes to frame decoder c. Check if complete frame received d. If not complete and timeout_ms > 0, wait briefly On complete frame: validate CRC, check sequence Copy decoded frame to output Update expected sequence number

Example: rx_frame_tframe;

```
rx_err_t err = rx_usb_comm_receive(&handle, &frame, 100);
```

```
switch(err) { case k_rx_ok: process_frame(&frame); break; case k_rx_err_timeout: break;
```

```
case k_rx_err_crc_mismatch: rx_log_warn("USB", "CRC mismatch"); break; default:
```

```
rx_log_error("USB", "Receive error"); break; }
```

Performance: Polling (timeout_ms=0): 5-20 us With timeout: depends on data availability

See also: rx_usb_comm_send() Send frames to peer, rx_usb_comm_data_available() Check if data waiting, rx_frame_t Frame structure definition

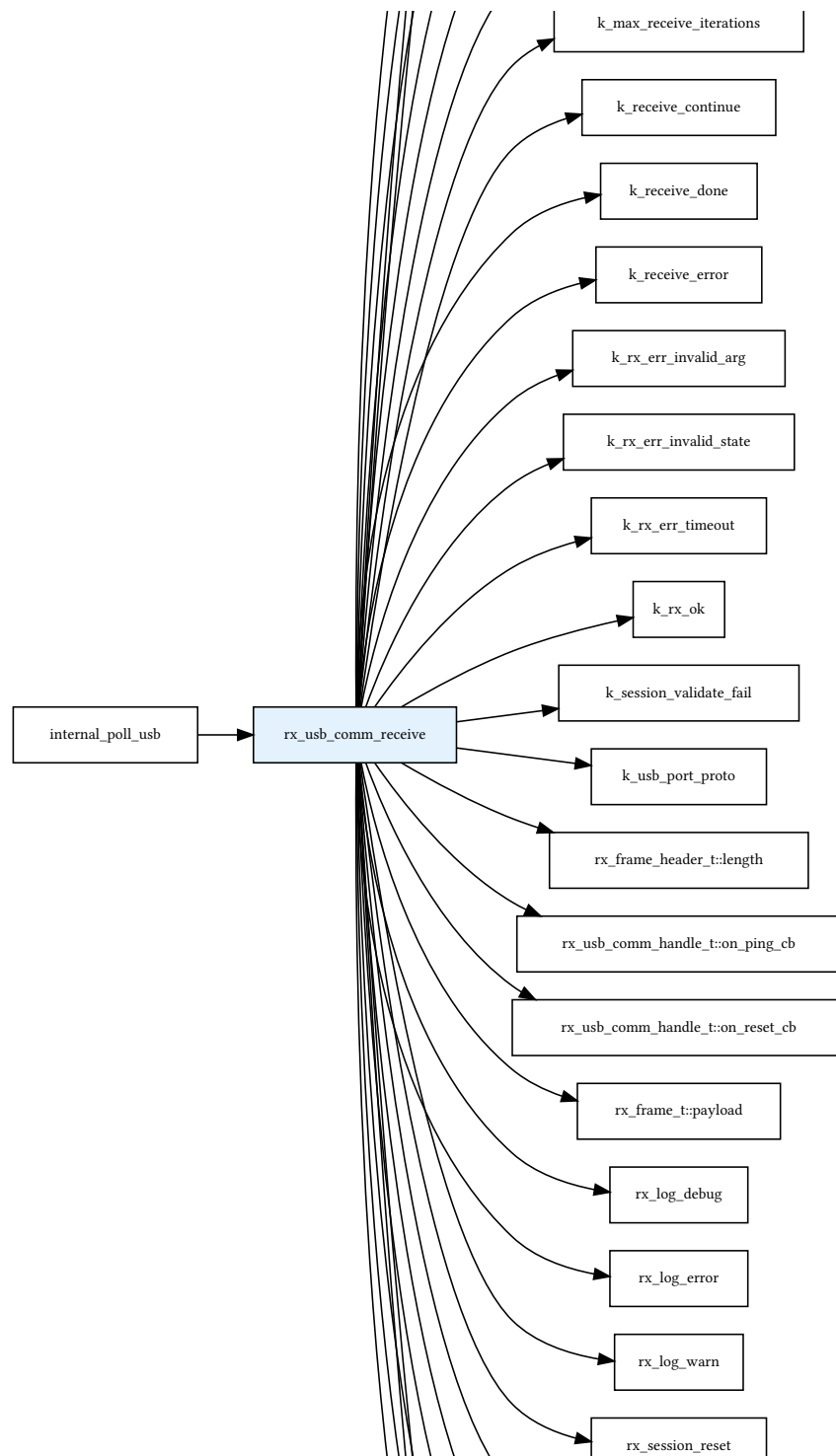


Figure 1243: Call graph for rx_usb_comm_receive

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:890`

Since Version 1.0.0

—

rx_usb_comm_data_available

```
rx_err_t rx_usb_comm_data_available(const rx_usb_comm_handle_t *handle, bool
*available)
```

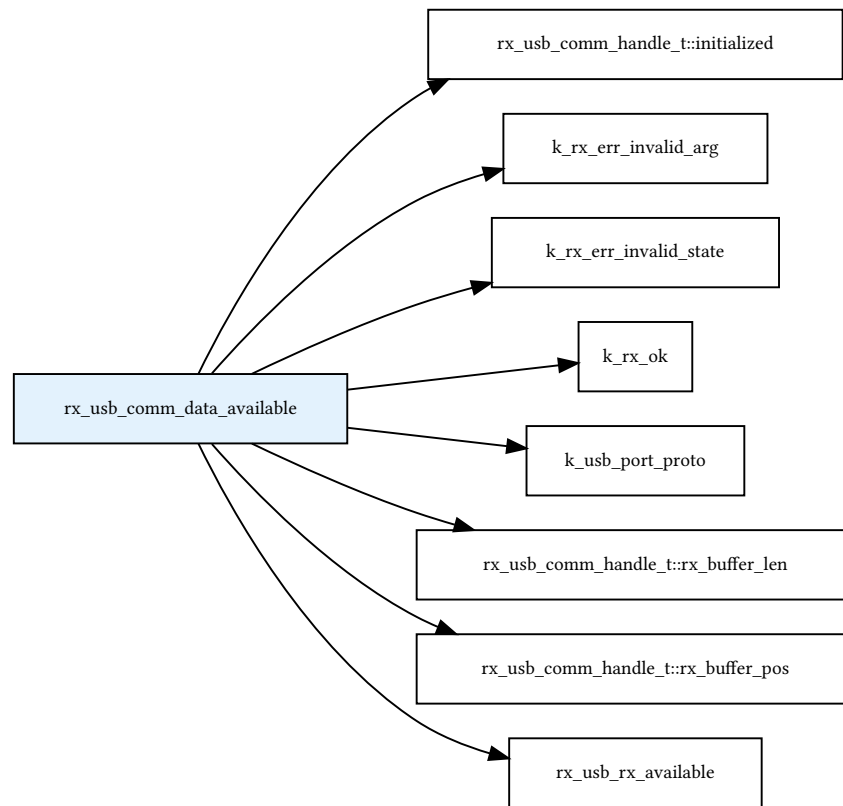
Check if data is available for reading.

Check if data is available for reading.

Parameters:

Direction	Name	Description
in	handle	Initialized handle
out	available	True if at least one byte is waiting
in	handle	USB communication handle
out	available	Set to true if data available, false otherwise

Returns: Error code from USB RX availability check on failure

Figure 1244: Call graph for `rx_usb_comm_data_available`

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:900`

—

`rx_usb_comm_is_ready`

```
rx_err_t rx_usb_comm_is_ready(const rx_usb_comm_handle_t *handle, bool *ready)
```

Check if USB is connected and ready.

Check if USB is connected and ready.

Parameters:

Direction	Name	Description
in	<code>handle</code>	Initialized handle
out	<code>ready</code>	True if USB is configured and ready for communication
in	<code>handle</code>	USB communication handle
out	<code>ready</code>	Set to true if ready, false otherwise

Returns: `k_rx_err_invalid_state` if handle not initialized

Note: Checks Port 0 (k_usb_port_proto) status.

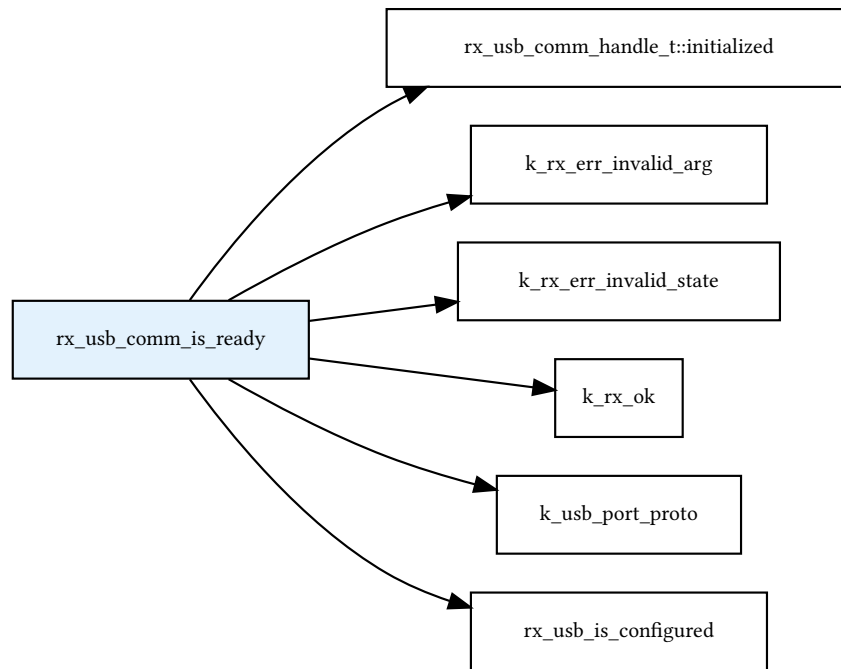


Figure 1245: Call graph for rx_usb_comm_is_ready

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:913`

rx_usb_comm_flush_rx

```
void rx_usb_comm_flush_rx(rx_usb_comm_handle_t *handle)
```

Flush internal receive buffer.

Discards any partially received data.

Flush internal receive buffer.

Resets the internal RX buffer position and length to zero, effectively discarding any partially received or buffered data. This is useful after error recovery or mode switching to ensure a clean receive state.

Parameters:

Direction	Name	Description
inout	handle	Pointer to handle
inout	handle	USB communication handle (nullptr-safe: no-op if nullptr)

Returns: void

Pre: handle should be initialized, but nullptr is safely handled

Post: handle->rx_buffer_len == 0

Post: handle->rx_buffer_pos == 0

Note: Not thread-safe: caller must serialize access to handle

Warning: Any partially received frame data will be lost] **See also:** rx_usb_comm_receive()
Receive path that populates the buffer

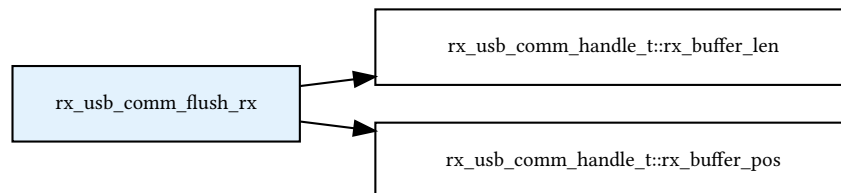


Figure 1246: Call graph for rx_usb_comm_flush_rx

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:927`

Since Version 1.0.0

—

rx_usb_comm_set_mode

```
rx_err_t rx_usb_comm_set_mode(rx_usb_comm_handle_t *handle, rx_usb_comm_mode_t mode)
```

Set USB communication mode.

Switches between binary protocol mode and debug text mode at runtime. This allows the same USB connection to be used for robot control (binary) or interactive debugging (text) depending on the development phase.

Set USB communication mode.

Parameters:

Direction	Name	Description
inout	handle	Initialized handle Mode field will be updated
in	mode	New mode to set k_usb_comm_mode_binary or k_usb_comm_mode_debug k_usb_comm_mode_invalid is rejected
inout	handle	USB communication handle
in	mode	Mode to set (k_usb_comm_mode_binary or k_usb_comm_mode_debug)

Returns: k_rx_err_invalid_state if handle not initialized

Value	Description
k_rx_ok	Mode changed successfully
k_rx_err_invalid_arg	handle is nullptr or mode is invalid
k_rx_err_invalid_state	Handle not initialized

Pre: handle initialized via rx_usb_comm_init()

Post: handle->mode == mode (on success)

Note: Does not flush buffers - pending data remains

Note: Thread safety: caller must provide external synchronization] **Mode Behaviors:** Binary mode (default): Frame-based protocol for robot control. Use rx_usb_comm_send/receive for framed data transfer. Debug mode: Plain ASCII text output for debugging. Use rx_usb_puts(k_usb_port_debug, ...) for text output. rx_usb_comm_send() returns k_rx_err_invalid_state in this mode.

Example: rx_usb_comm_set_mode(&handle,k_usb_comm_mode_debug);
rx_usb_puts(k_usb_port_debug,"Debugmodeactive\r\n");
rx_usb_comm_set_mode(&handle,k_usb_comm_mode_binary);

See also: rx_usb_comm_get_mode() Query current mode, rx_usb_comm_mode_t Mode definitions

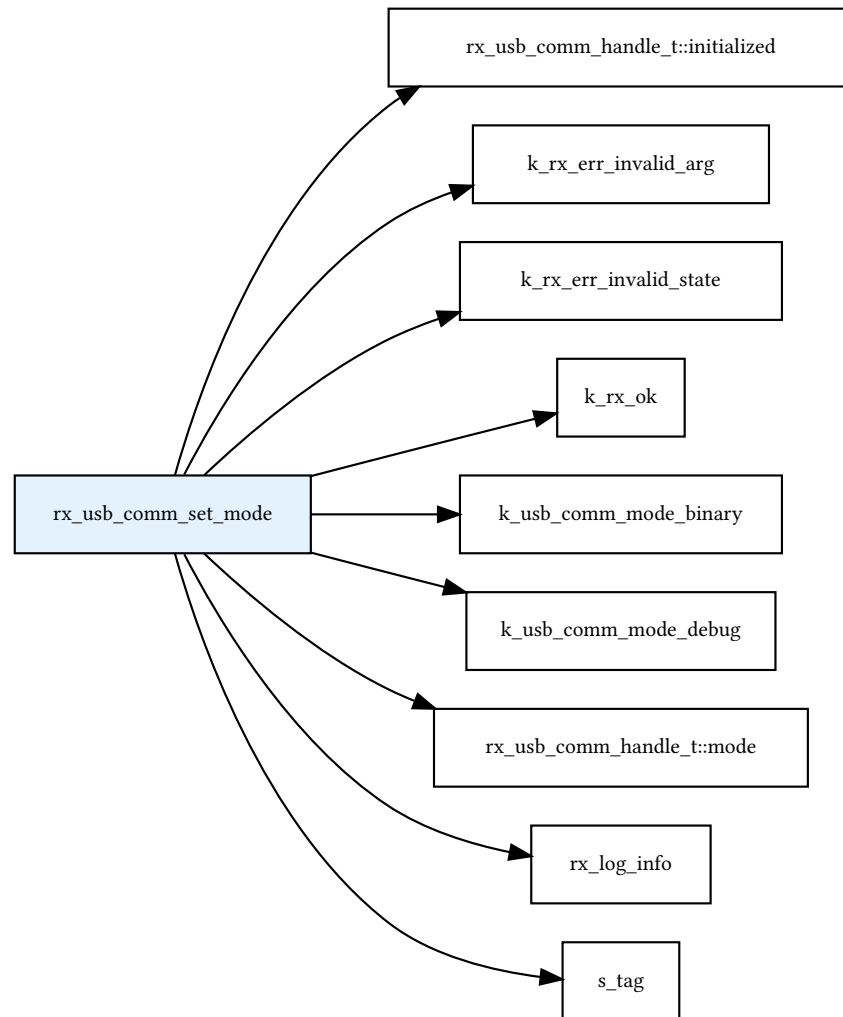


Figure 1247: Call graph for rx_usb_comm_set_mode

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:982`

Since Version 1.0.0

—

rx_usb_comm_get_mode

```
rx_usb_comm_mode_t rx_usb_comm_get_mode(const rx_usb_comm_handle_t *handle)
```

Get current USB communication mode.

Parameters:

Direction	Name	Description
-----------	------	-------------

in	handle	Initialized handle
in	handle	USB communication handle

Returns: Current mode (k_usb_comm_mode_invalid if handle is nullptr or not initialized)

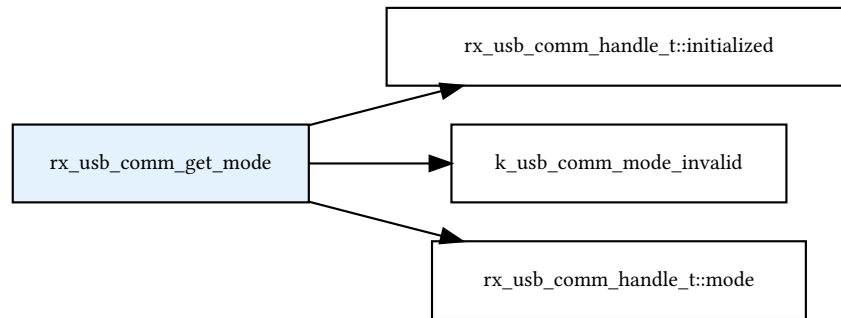


Figure 1248: Call graph for rx_usb_comm_get_mode

Defined in *libs/rx_usb_comm/inc/rx_usb_comm.h:990*

rx_usb_comm_set_control_callbacks

```

rx_err_t rx_usb_comm_set_control_callbacks(rx_usb_comm_handle_t *handle,
void(*on_ping_cb)(const rx_frame_t *frame, void *ctx), void(*on_reset_cb)(const
rx_frame_t *frame, void *ctx), void *cb_ctx)

```

Register callbacks for control frame notifications.

Sets optional callbacks invoked when PING or RESET control frames are received. The auto-response (PONG or RESET_ACK) is always sent regardless of whether callbacks are registered.

Register callbacks for control frame notifications.

Sets application-level callbacks invoked after control frame auto-response. PING callback fires after PONG is sent; RESET callback fires after RESET_ACK is sent and session is reset. Either callback may be NULL to disable notification for that frame type.

Parameters:

Direction	Name	Description
inout	handle	Initialized USB communication handle
in	on_ping_cb	Callback for PING frames (may be NULL to disable)
in	on_reset_cb	Callback for RESET frames (may be NULL to disable)
in	cb_ctx	User context pointer passed to callbacks
inout	handle	USB communication handle (must be initialized)
in	on_ping_cb	Callback for PING frames (NULL to disable)
in	on_reset_cb	Callback for RESET frames (NULL to disable)
in	cb_ctx	Opaque context pointer forwarded to callbacks

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, callbacks registered
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized
k_rx_ok	Callbacks registered successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle is not initialized

Pre: handle must be initialized via rx_usb_comm_init()

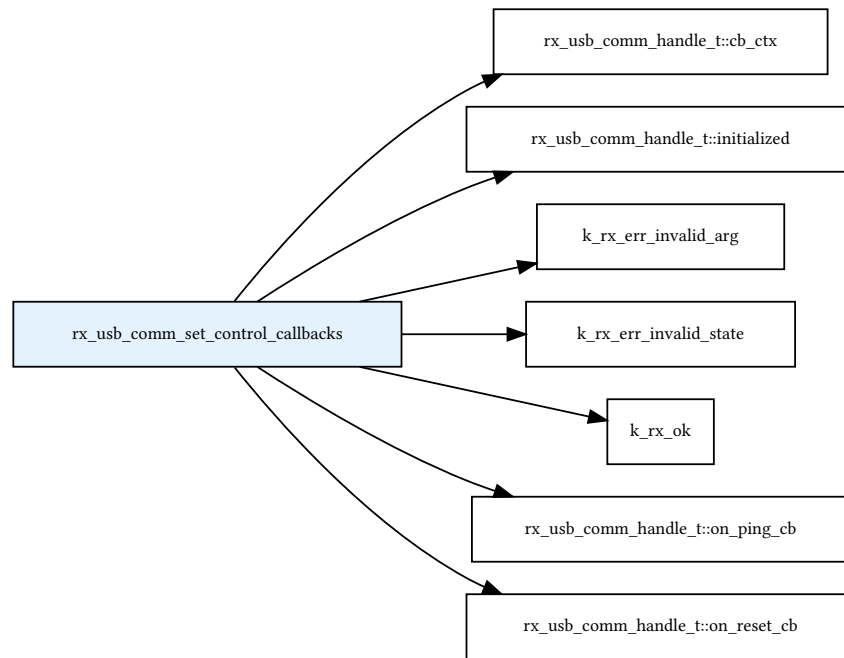
Pre: handle must be initialized via rx_usb_comm_init()

Post: on_ping_cb, on_reset_cb, cb_ctx stored in handle

Post: handle->on_ping_cb, on_reset_cb, cb_ctx updated

Note: Not thread-safe: call during initialization or from single thread

Note: Callback ownership: caller retains ownership of cb_ctx] **See also:**
rx_usb_comm_receive() Control frames handled automatically during receive,
rx_usb_comm_send_pong() Auto-response before PING callback,
rx_usb_comm_send_reset_ack() Auto-response before RESET callback

Figure 1249: Call graph for `rx_usb_comm_set_control_callbacks`

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:1023`

Since Version 1.0.0

—

`rx_usb_comm_send_pong`

```
rx_err_t rx_usb_comm_send_pong(rx_usb_comm_handle_t *handle, const uint8_t
*payload, uint16_t payload_len)
```

Send a PONG response frame.

Constructs and sends a PONG frame echoing the provided payload. Called automatically by the receive path when a PING frame is received. Can also be called manually if needed.

Send a PONG response frame.

Constructs and sends a PONG frame echoing the PING payload (typically a 4-byte counter). Uses the shared session for TX sequence assignment.

Parameters:

Direction	Name	Description
inout	<code>handle</code>	Initialized USB communication handle
in	<code>payload</code>	Payload to echo (from received PING, may be NULL)
in	<code>payload_len</code>	Length of payload in bytes

inout	handle	USB communication handle (must be initialized)
in	payload	PING payload to echo (may be NULL if payload_len == 0)
in	payload_len	Length of payload in bytes

Returns: rx_err_t Error code

Value	Description
k_rx_ok	PONG sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized or USB not ready
k_rx_ok	PONG sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized via rx_usb_comm_init()

Pre: handle must be initialized

Post: TX sequence incremented via shared session

Post: TX sequence incremented by 1

Note: Called automatically by rx_usb_comm_receive() on PING frames] **See also:**
rx_usb_comm_receive() Calls this automatically for PING frames,
rx_frame_create_pong() Frame construction, rx_session_next_tx() Sequence assignment

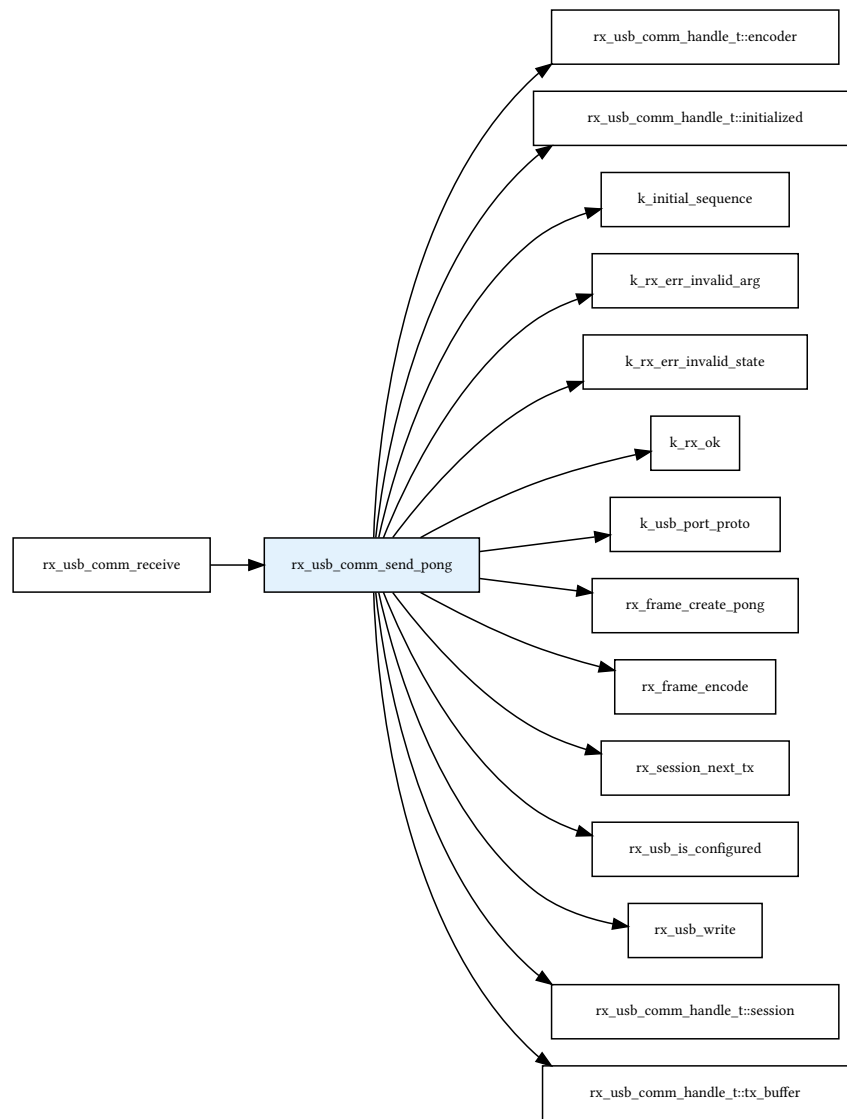


Figure 1250: Call graph for rx_usb_comm_send_pong

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:1053`

Since Version 1.0.0

—

rx_usb_comm_send_reset_ack

```
rx_err_t rx_usb_comm_send_reset_ack(rx_usb_comm_handle_t *handle)
```

Send a RESET_ACK response frame.

Constructs and sends a RESET_ACK frame (no payload). Called automatically by the receive path when a RESET frame is received.

Send a RESET_ACK response frame.

Constructs and sends a RESET_ACK frame (no payload) to acknowledge a session reset request. Uses the shared session for TX sequence assignment.

Parameters:

Direction	Name	Description
inout	handle	Initialized USB communication handle
inout	handle	USB communication handle (must be initialized)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	RESET_ACK sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized or USB not ready
k_rx_ok	RESET_ACK sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized

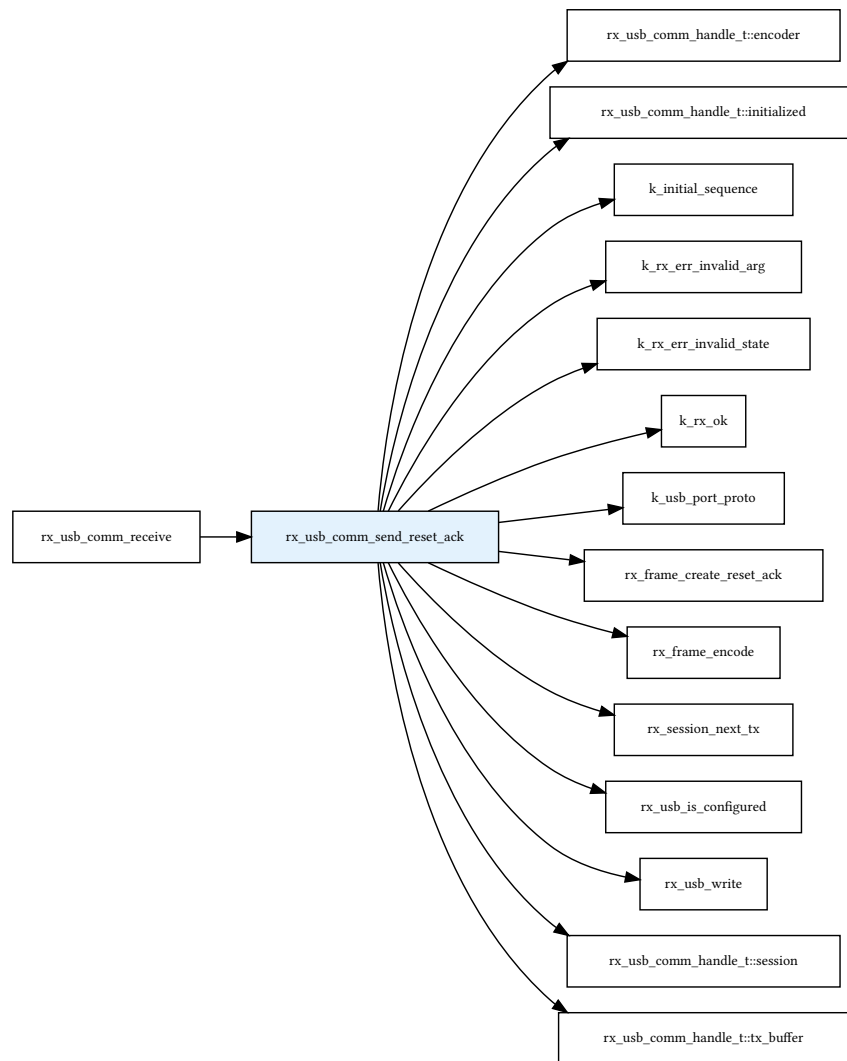
Pre: handle must be initialized via rx_usb_comm_init()

Pre: handle must be initialized

Post: TX sequence incremented via shared session

Post: TX sequence incremented by 1

Note: Called automatically by rx_usb_comm_receive() on RESET frames] **See also:**
rx_usb_comm_receive() Calls this automatically for RESET frames,
rx_frame_create_reset_ack() Frame construction, rx_session_reset() Session reset
after ACK

Figure 1251: Call graph for `rx_usb_comm_send_reset_ack`

Defined in `libs/rx_usb_comm/inc/rx_usb_comm.h:1076`

Since Version 1.0.0

—

rx_usb_comm.c

High-Level USB CDC Communication Layer Implementation.

Implements reliable USB communication by integrating the frame protocol layer (`rx_frame`) with the USB CDC driver (`rx_usb`). Handles frame encoding/decoding, CRC-32 validation, sequence number management, and receive buffer handling.

STAR Team

2026-01-01

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_usb_comm.h"
- <string.h>
- "rx_check.h"
- "rx_crc.h"
- "rx_log.h"
- "rx_usb.h"

rx_usb_comm_header_size_t

Total header size including sync word: SYNC(2) + SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1).

Value	Name	Description
= k_frame_sync_size + k_frame_seq_size + k_frame_len_size + k_frame_type_size + k_frame_flags_size	k_frame_header_total	

—

rx_usb_comm_sleep_t

Sleep interval for receive polling (ms).

Value	Name	Description
= 10	k_sleep_interval_ms	

—

rx_usb_comm_sequence_constants_t

Sequence number constants.

Value	Name	Description
= 0	k_initial_sequence	Initial TX/RX sequence number

—

rx_usb_comm_sync_constants_t

Sync search constants.

Value	Name	Description
= 1	k_sync_second_byte_offset	Offset from current byte to second sync byte

—

rx_usb_comm_loop_limits_t

Receive loop bounds for NASA Power of 10 Rule 2 compliance.

Value	Name	Description
= 100	k_max_receive_iterations	Max receive loop attempts

—

frame_header_offset_t

Byte offsets within frame header buffer.

Value	Name	Description
= 4	k_hdr_len_offset	Payload length field offset

—

rx_usb_comm_sync_result_t

Sync word not found sentinel value.

Value	Name	Description
= -1	k_sync_not_found	Sync word not found in buffer

—

rx_receive_result_t

Receive iteration result codes.

Value	Name	Description
= 0	k_receive_continue	Continue to next iteration
= 1	k_receive_done	Frame received successfully
= 2	k_receive_error	Error occurred, check err output

—

s_tag

```
const char* s_tag
```

—

s_zero_handle

```
const rx_usb_comm_handle_t s_zero_handle
```

Zero-initialized USB comm handle template for stack-safe initialization.

Note: Read-only; never modified after static initialization

Warning: Do not remove - prevents stack overflow in init function] **See also:**

rx_usb_comm_init() Uses this template to zero-initialize the comm handle

Since Version 1.0.0

—

internal_decode_header

```
static rx_err_t internal_decode_header(const uint8_t *data, const uint32_t
data_len, rx_frame_t *frame, uint32_t *offset_out)
```

Decode frame header from raw wire data.

Parses the frame header fields from a raw byte buffer. Validates sync word, extracts sequence number, payload length, type, and flags.

Parameters:

Direction	Name	Description
in	data	Raw frame data buffer (must not be nullptr)
in	data_len	Length of data buffer in bytes
out	frame	Frame structure to populate (must not be nullptr)
out	offset_out	Offset past header (optional, can be nullptr)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, frame header decoded
k_rx_err_invalid_arg	nullptr pointer in data or frame
k_rx_err_invalid_size	Data too short for header
k_rx_err_protocol_error	Invalid sync word

Pre: data != nullptr

Pre: frame != nullptr

Pre: data_len >= k_frame_min_size for valid frame

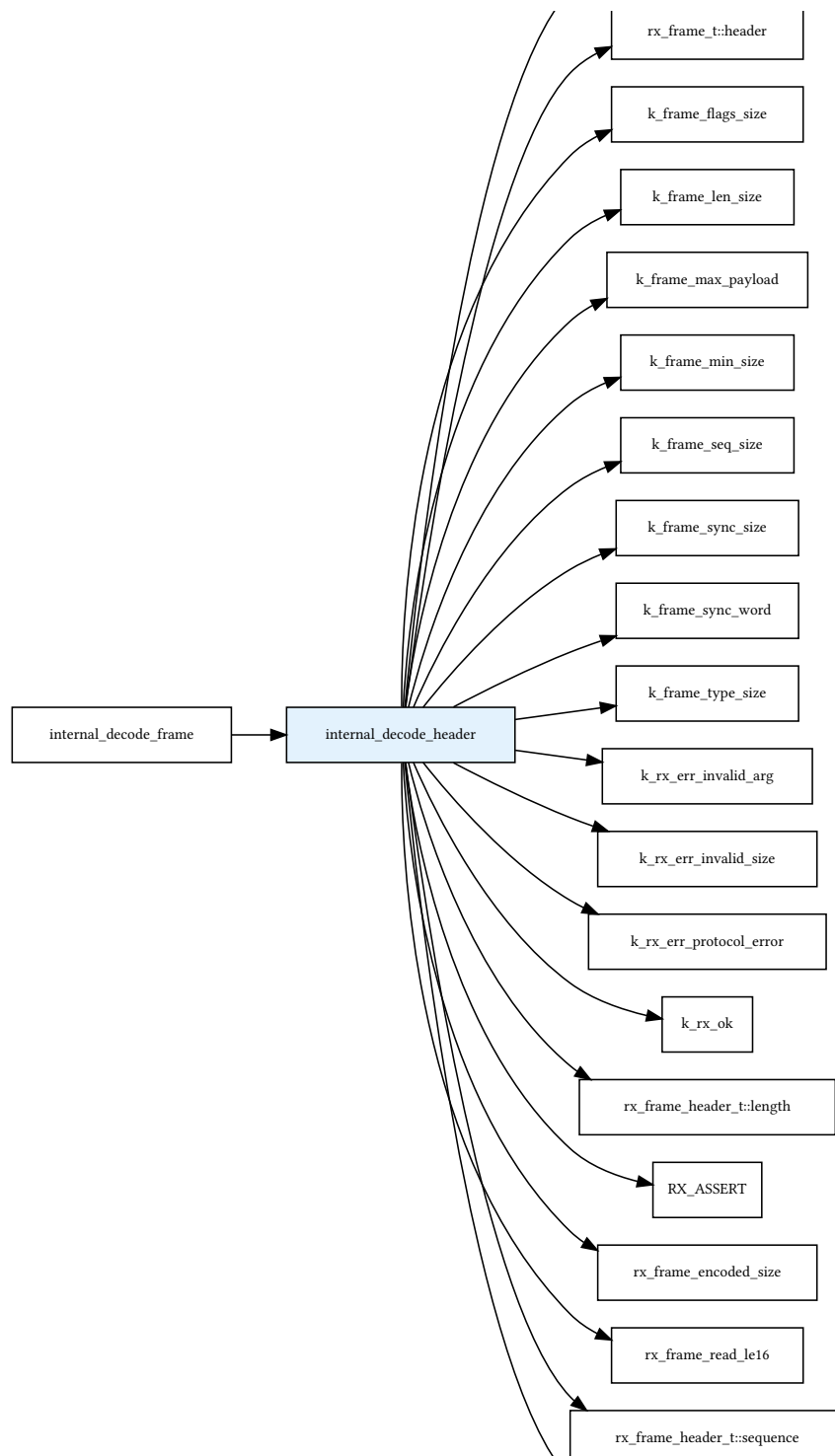
Post: frame->header fields populated on success

Post: offset_out contains byte offset past header (if not nullptr)

Note: Not thread-safe, but stateless (safe for concurrent calls with different data)

Algorithm:: Validate input pointers (RX_ASSERT + runtime check) Verify minimum data length for header Read and validate sync word (0x55AA little-endian) Extract sequence number (LE16) Extract and validate payload length (LE16) Extract frame type and flags Return offset past header for payload access

See also: internal_verify_crc() Called after this to verify frame CRC

Figure 1252: Call graph for `internal_decode_header`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:124`

—

internal_verify_crc

```
static rx_err_t internal_verify_crc(const uint8_t *data, uint32_t offset,
uint32_t *crc_out)
```

Verify CRC-32 IEEE checksum for received frame.

Computes CRC-32 IEEE 802.3 checksum over the frame data (excluding CRC field) and compares against the CRC stored in the frame. Uses little-endian CRC storage as per protocol specification.

Parameters:

Direction	Name	Description
in	data	Raw frame data buffer containing header + payload + CRC
in	offset	Byte offset to CRC field (= header_size + payload_len)
out	crc_out	Optional pointer to receive verified CRC value (can be nullptr)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	CRC valid, frame integrity confirmed
k_rx_err_invalid_arg	nullptr data or offset too small
k_rx_err_crc_mismatch	Calculated CRC doesn't match stored CRC

Pre: data != nullptr

Pre: offset >= (k_frame_min_size - k_frame_crc_size)

Post: crc_out contains verified CRC value on success (if not nullptr)

Post: Frame integrity validated on k_rx_ok return

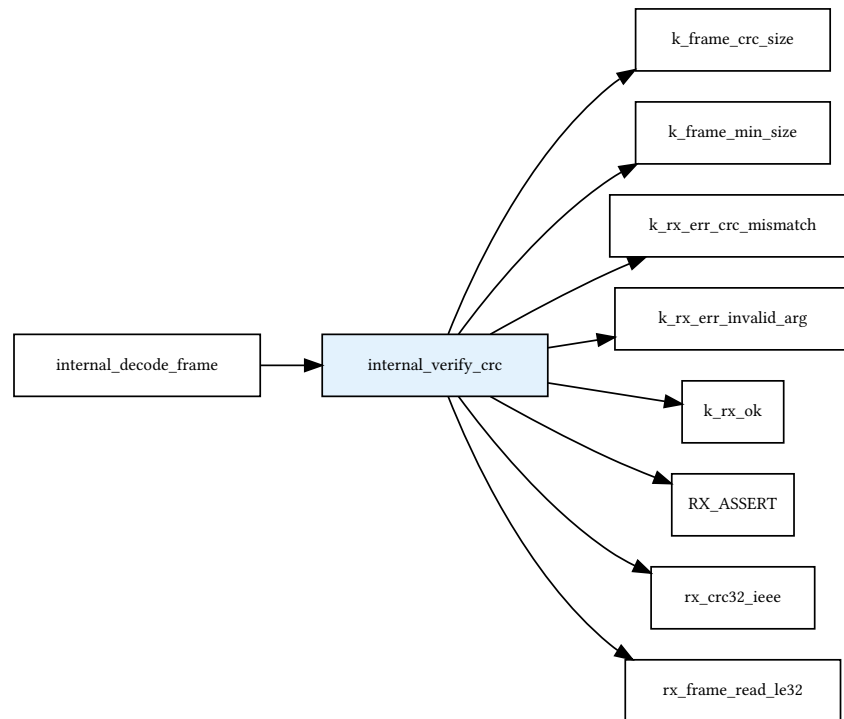
Note: Uses rx_crc32_ieee() - CRC-32 IEEE 802.3 polynomial

Note: CRC stored in little-endian format in frame

Algorithm:: Validate data pointer (RX_ASSERT + runtime check) Validate offset is valid for CRC position Read stored CRC from data[offset] (LE32) Calculate CRC-32 over data[0..offset-1] Compare and return result

Performance:: CRC calculation: 10 us for 64-byte payload @ 240 MHz

See also: rx_crc32_ieee() CRC calculation function, internal_decode_header() Called before this to parse header

Figure 1253: Call graph for `internal_verify_crc`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:212`

internal_read_usb_data

```
static rx_err_t internal_read_usb_data(rx_usb_comm_handle_t *handle)
```

Read available data from USB CDC into staging buffer.

Reads any available data from USB Port 0 (protocol port) into the handle's `rx_buffer`. Appends to existing data without overwriting. Returns immediately if buffer is full or no data available.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle with <code>rx_buffer</code>

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success (0 or more bytes read)
<code>k_rx_err_invalid_arg</code>	nullptr handle
<code>k_rx_err_invalid_state</code>	Buffer length exceeds maximum

Other	errors from rx_usb_read()
-------	---------------------------

Pre: handle != nullptr

Pre: handle->rx_buffer_len <= k_usb_comm_rx_buffer_size

Post: handle->rx_buffer_len <= k_usb_comm_rx_buffer_size

Post: Data appended to handle->rx_bufferold_len..new_len

Note: Non-blocking: returns immediately with available data

Note: Uses Port 0 (k_usb_port_proto) exclusively

Algorithm:: Validate handle pointer Validate buffer length within bounds Calculate available space in buffer Return early if buffer full Call rx_usb_read() to get available data Update buffer length Validate post-condition

See also: internal_compact_rx_buffer() Call after consuming data

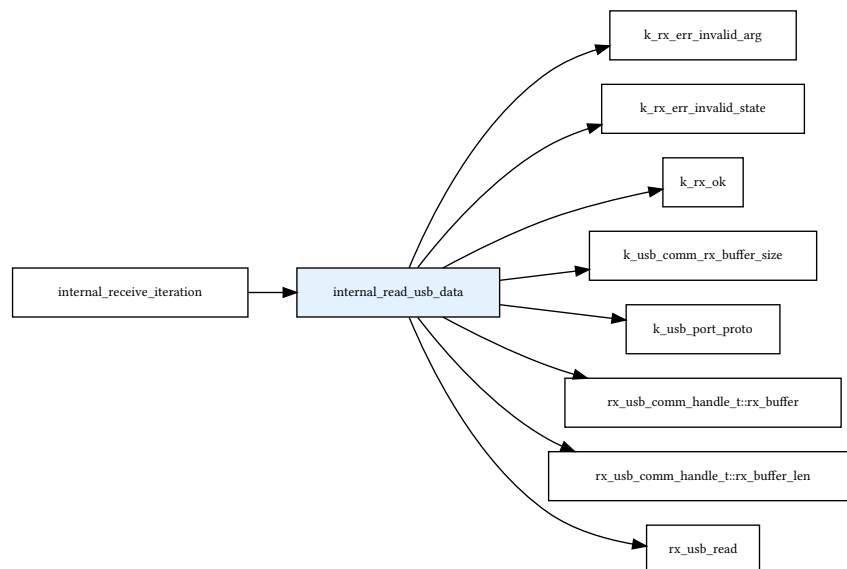


Figure 1254: Call graph for internal_read_usb_data

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:319`

—

internal_compact_rx_buffer

```
static rx_err_t internal_compact_rx_buffer(rx_usb_comm_handle_t *handle)
```

Compact receive buffer by removing consumed data.

Removes data from the beginning of rx_buffer that has already been processed (bytes 0 to rx_buffer_pos-1). Remaining data is moved to the start of the buffer using memmove() to handle overlapping regions safely.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle with rx_buffer

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, buffer compacted
k_rx_err_invalid_arg	nullptr handle
k_rx_err_invalid_state	Position exceeds length

Pre: handle != nullptr

Pre: handle->rx_buffer_pos <= handle->rx_buffer_len

Post: handle->rx_buffer_pos == 0

Post: Remaining data preserved at buffer start

Note: Uses memmove() for safe overlapping copy

Note: O(n) where n = remaining bytes

Algorithm:: Validate handle pointer Validate position <= length invariant Return early if position is 0 (nothing to compact) If all data consumed, reset both counters to 0 Otherwise, memmove remaining data to start Update length and reset position to 0 Validate post-condition

See also: internal_read_usb_data() Fills buffer before compaction

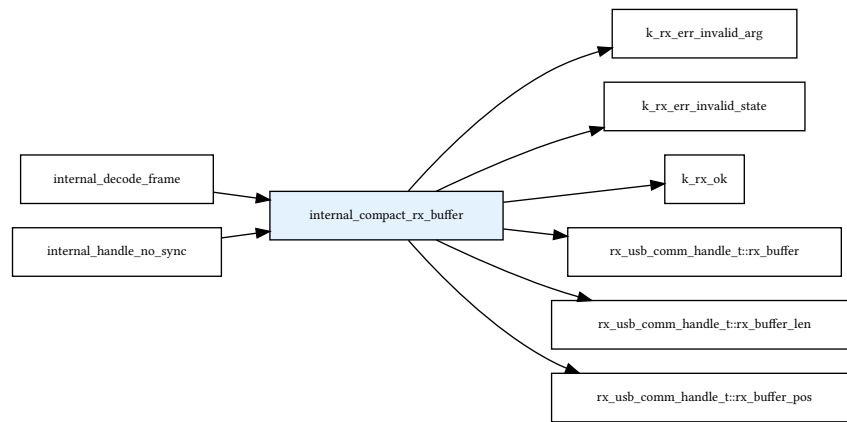


Figure 1255: Call graph for internal_compact_rx_buffer

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:390`

internal_find_sync

```
static rx_err_t internal_find_sync(const rx_usb_comm_handle_t *handle, int32_t
*sync_pos)
```

Search for frame sync word in receive buffer.

Scans the receive buffer starting from rx_buffer_pos for the frame sync word (0x55AA) stored in little-endian format. Returns the position of the first occurrence found.

Parameters:

Direction	Name	Description
in	handle	USB communication handle with rx_buffer
out	sync_pos	Output position of sync word (k_sync_not_found if not found)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Sync word found, position in sync_pos
k_rx_err_invalid_arg	nullptr handle or sync_pos
k_rx_err_invalid_state	Buffer position exceeds length
k_rx_err_not_found	Sync word not in buffer

Pre: handle != nullptr

Pre: sync_pos != nullptr

Pre: `handle->rx_buffer_pos <= handle->rx_buffer_len`

Post: `sync_pos` contains position or `k_sync_not_found`

Post: Buffer state unchanged

Note: $O(n)$ linear scan where n = buffer length

Note: Read-only operation, does not modify buffer

Algorithm:: Validate handle and output pointers Validate buffer position $\leq$ length Extract sync word bytes (0xAA low, 0x55 high) Linear scan from `rx_buffer_pos` to end-1 Return position if found, `k_sync_not_found` otherwise

Sync Word Format:: The sync word 0x55AA is stored little-endian in the frame: Byte 0: 0xAA (low byte, LSB first) Byte 1: 0x55 (high byte, MSB second)

See also: `k_frame_sync_word` Sync word constant (0x55AA)

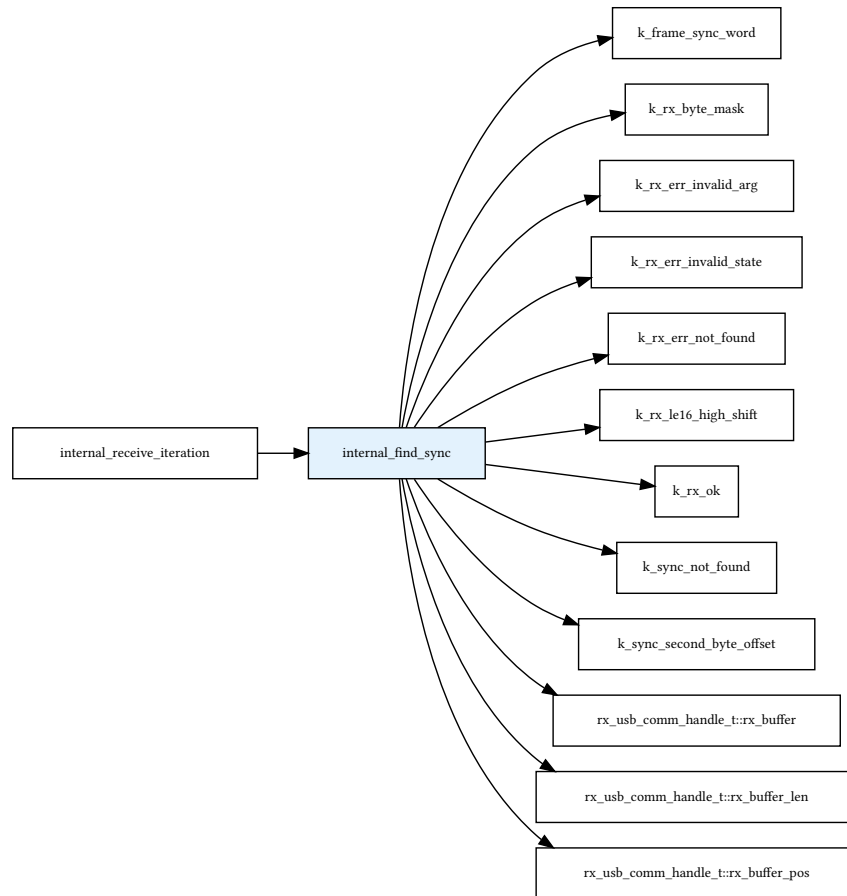


Figure 1256: Call graph for internal_find_sync

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:466`

internal_sleep_and_advance

```
static bool internal_sleep_and_advance(const rx_usb_comm_handle_t *handle,
uint32_t *elapsed_ms)
```

Sleep and advance elapsed time if time interface available.

Parameters:

Direction	Name	Description
in	handle	USB communication handle
inout	elapsed_ms	Pointer to elapsed time counter

Returns: true if sleep was performed, false if no time interface

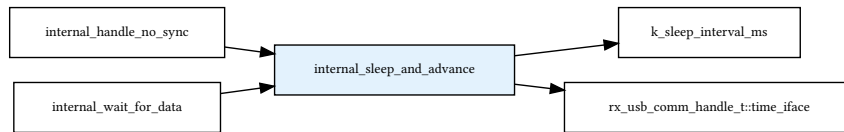


Figure 1257: Call graph for internal_sleep_and_advance

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:508`

internal_is_timed_out

```
static bool internal_is_timed_out(const uint32_t timeout_ms, uint32_t elapsed_ms)
```

Check if timeout has been reached.

Parameters:

Direction	Name	Description
in	timeout_ms	Timeout value in milliseconds (0 = immediate)
in	elapsed_ms	Elapsed time in milliseconds

Returns: true if timed out, false otherwise

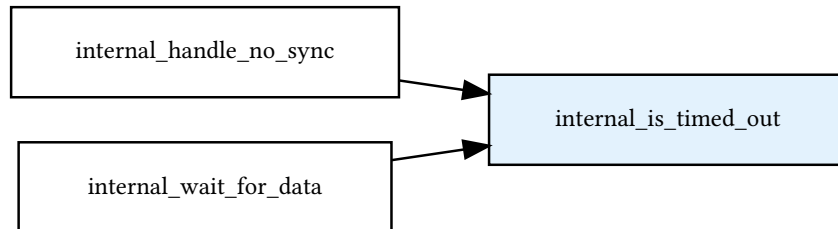


Figure 1258: Call graph for internal_is_timed_out

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:528`

internal_handle_no_sync

```
static rx_err_t internal_handle_no_sync(rx_usb_comm_handle_t *handle, const uint32_t timeout_ms, uint32_t *elapsed_ms)
```

Handle case when no sync word is found in buffer.

Advances buffer position if full, compacts buffer, and handles timeout/sleep.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle
in	timeout_ms	Timeout value in milliseconds

inout	elapsed_ms	Pointer to elapsed time counter
-------	------------	---------------------------------

Returns: k_rx_err_timeout if timed out or no time interface

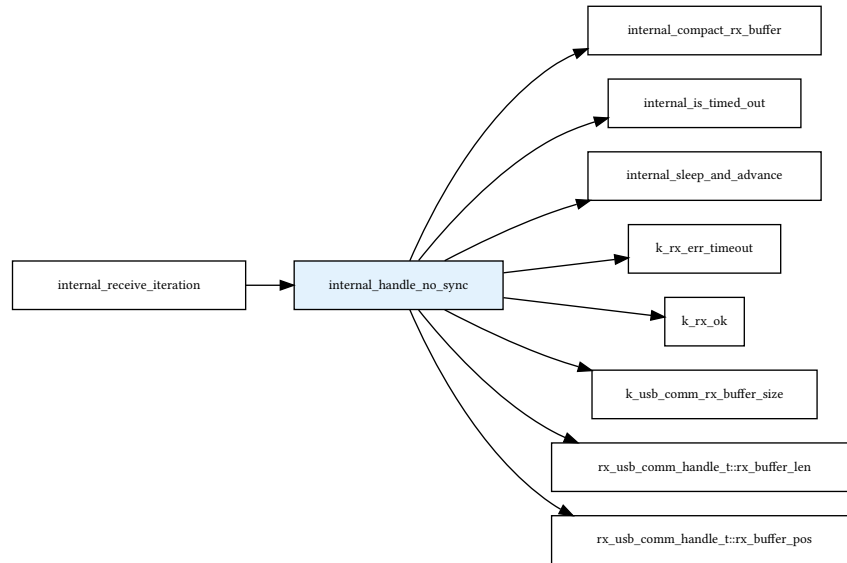


Figure 1259: Call graph for internal_handle_no_sync

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:544`

—

internal_wait_for_data

```
static rx_err_t internal_wait_for_data(const rx_usb_comm_handle_t *handle, const
uint32_t timeout_ms, uint32_t *elapsed_ms)
```

Wait for more data with timeout handling.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle
in	timeout_ms	Timeout value in milliseconds
inout	elapsed_ms	Pointer to elapsed time counter

Returns: k_rx_err_timeout if timed out or no time interface

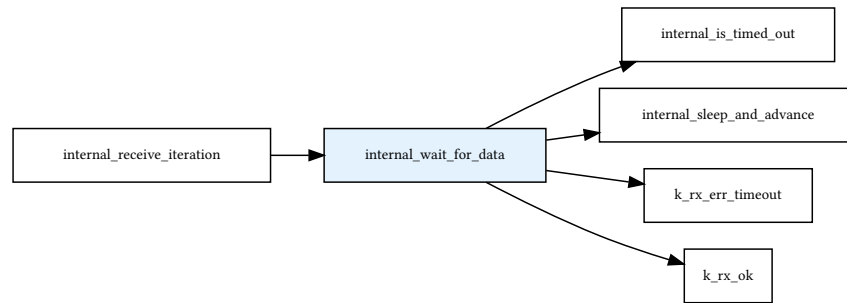


Figure 1260: Call graph for internal_wait_for_data

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:580`

—

internal_decode_frame

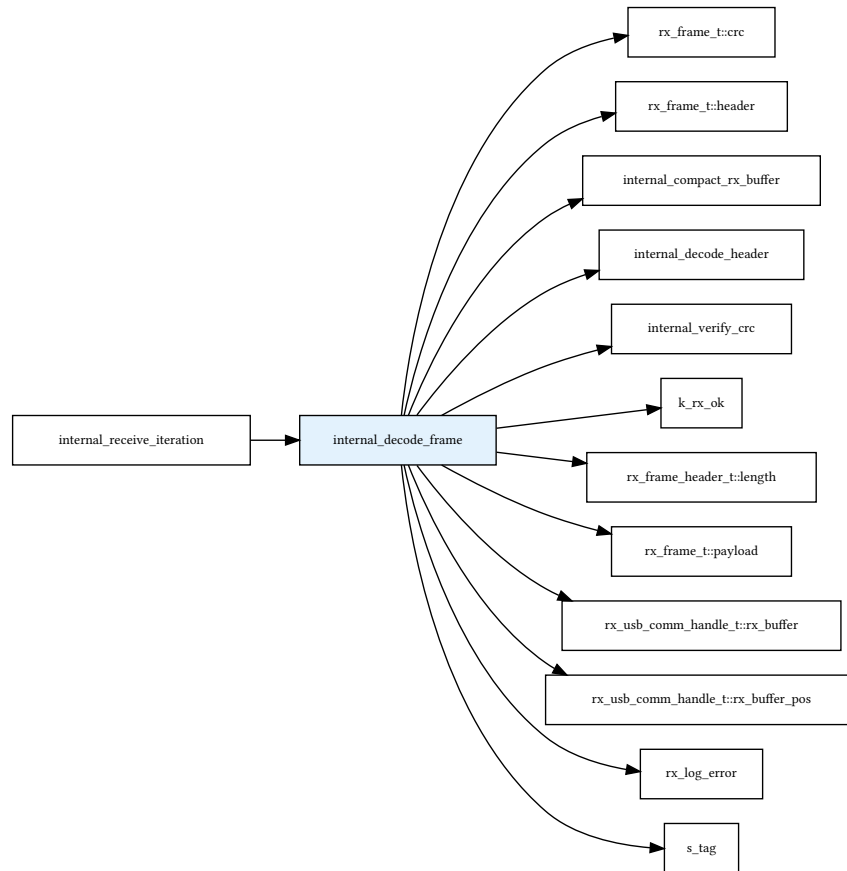
```
static rx_err_t internal_decode_frame(rx_usb_comm_handle_t *handle, rx_frame_t  
*frame, const uint32_t total_size)
```

Decode a complete frame from buffer.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle
out	frame	Decoded frame output
in	total_size	Total frame size in bytes

Returns: Error code from `rx_frame_decode` on failure

Figure 1261: Call graph for `internal_decode_frame`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:606`

`internal_align_to_sync`

```
static void internal_align_to_sync(rx_usb_comm_handle_t *handle, const int32_t
sync_pos)
```

Align buffer position to sync word if found.

Parameters:

Direction	Name	Description
inout	<code>handle</code>	USB communication handle
in	<code>sync_pos</code>	Position of sync word in buffer

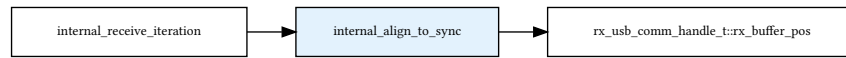


Figure 1262: Call graph for internal_align_to_sync

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:659`

internal_parse_header

```
static bool internal_parse_header(rx_usb_comm_handle_t *handle, uint16_t
*payload_len, uint32_t *total_size)
```

Parse frame header and validate payload length.

Parameters:

Direction	Name	Description
in	handle	USB communication handle
out	payload_len	Output payload length
out	total_size	Output total frame size

Returns: true if header is valid, false if invalid payload length

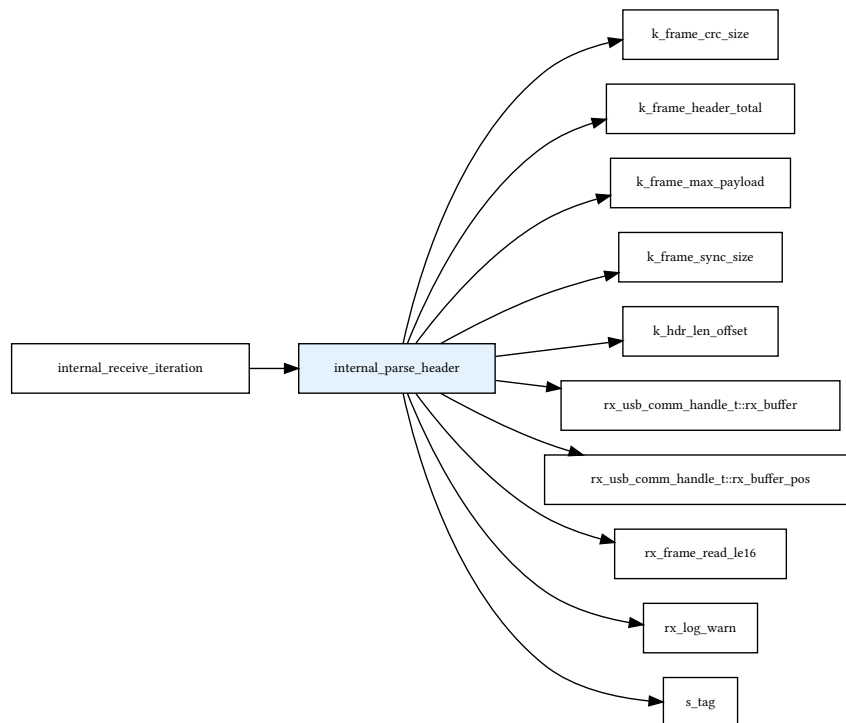


Figure 1263: Call graph for internal_parse_header

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:675`

—

internal_receive_iteration

```
static rx_receive_result_t internal_receive_iteration(rx_usb_comm_handle_t
*handle, rx_frame_t *frame, const uint32_t timeout_ms, uint32_t *elapsed_ms,
rx_err_t *err)
```

Process one iteration of the frame receive loop.

Performs a single pass of the receive state machine. Attempts to read USB data, find sync word, parse header, and decode complete frame. Returns result code indicating whether to continue, success, or error.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle
out	frame	Decoded frame output (valid only if k_receive_done)
in	timeout_ms	Timeout value in milliseconds
inout	elapsed_ms	Elapsed time counter (updated on sleep)
out	err	Error code (valid only if k_receive_error)

Returns: rx_receive_result_t Result code

Value	Description
k_receive_continue	Continue to next iteration
k_receive_done	Frame successfully received
k_receive_error	Error occurred, check err parameter

Pre: handle != nullptr and initialized

Pre: frame != nullptr

Pre: elapsed_ms != nullptr

Pre: err != nullptr

Post: On k_receive_done: frame contains valid decoded frame

Post: On k_receive_error: err contains error code

Note: Called in loop by rx_usb_comm_receive()

State Machine:: digraph receive_iteration { rankdir=TB; read [label="Read USB Data"]; find_sync [label="Find Sync"]; check_header [label="Header Complete?"]; parse_header [label="Parse Header"]; check_frame [label="Frame Complete?"]; decode [label="Decode Frame"]; done [label="DONE"]; continue_state [label="CONTINUE"]; error [label="ERROR"];

read -> find_sync [label="ok"]; read -> error [label="fail"]; find_sync -> check_header [label="found"]; find_sync -> continue_state [label="not found"]; check_header -> parse_header [label="yes"]; check_header -> continue_state [label="no"]; parse_header -> check_frame [label="ok"]; parse_header -> continue_state [label="invalid"]; check_frame -> decode [label="yes"]; check_frame -> continue_state [label="no"]; decode -> done [label="ok"]; decode -> error [label="fail"]; }

See also: rx_usb_comm_receive() Main receive API that calls this

Figure 1264: Call graph for `internal_receive_iteration`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:749`

rx_usb_comm_init

```
rx_err_t rx_usb_comm_init(rx_usb_comm_handle_t *handle, const
rx_usb_comm_config_t *config)
```

Initialize USB communication layer.

Initialize USB communication handle.

Initializes the USB communication handle for framed communication. Sets up the frame encoder/decoder, configures FEC and time interface, and prepares buffer state. Must be called before any other rx_usb_comm functions.

Parameters:

Direction	Name	Description
out	handle	USB communication handle to initialize
in	config	Configuration parameters (must not be NULL) session: Required shared session state pointer time_iface: Time interface for sleep operations (nullptr = no sleep)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, handle ready for use
k_rx_err_invalid_arg	nullptr handle pointer
k_rx_err_invalid_state	Post-condition validation failed
Other	errors from rx_frame_encoder_init() or rx_frame_decoder_init()

Pre: handle != nullptr

Pre: handle memory writable (sizeof(rx_usb_comm_handle_t) bytes)

Post: handle->initialized == true on success

Post: handle->mode == k_usb_comm_mode_binary on success

Post: handle->session points to shared session state

Note: Not thread-safe during initialization

Note: USB hardware must be initialized separately via rx_usb_init()

Algorithm:: Validate handle pointer Clear handle to zero state Apply configuration (or defaults if nullptr) Initialize frame encoder Initialize frame decoder (cleanup encoder on failure) Initialize

sequence counters to 0 Initialize buffer state Set mode to binary (default) Mark handle as initialized
Validate post-conditions

Example:: rx_usb_comm_handle_thandle;
rx_usb_comm_config_tconfig={ .session=&g_session, .time_iface=&time_interface };
rx_err_t err=rx_usb_comm_init(&handle,&config); if(err!=k_rx_ok)
{ rx_log_error("USB","Initfailed"); }

See also: rx_usb_comm_deinit() Cleanup function, rx_usb_init() USB hardware initialization



Figure 1265: Call graph for rx_usb_comm_init

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:873`

—

rx_usb_comm_deinit

```
rx_err_t rx_usb_comm_deinit(rx_usb_comm_handle_t *handle)
```

Deinitialize USB communication layer.

Deinitialize USB communication handle.

Releases resources associated with the USB communication handle. Deinitializes both frame encoder and decoder. Safe to call on uninitialized handle (logs assertion but doesn't crash).

Parameters:

Direction	Name	Description
inout	handle	USB communication handle to deinitialize

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success (or handle was not initialized)
k_rx_err_invalid_arg	nullptr handle pointer
Other	errors from encoder/decoder deinit (first error returned)

Pre: handle != nullptr

Pre: handle->initialized == true (asserted, not enforced)

Post: handle->initialized == false

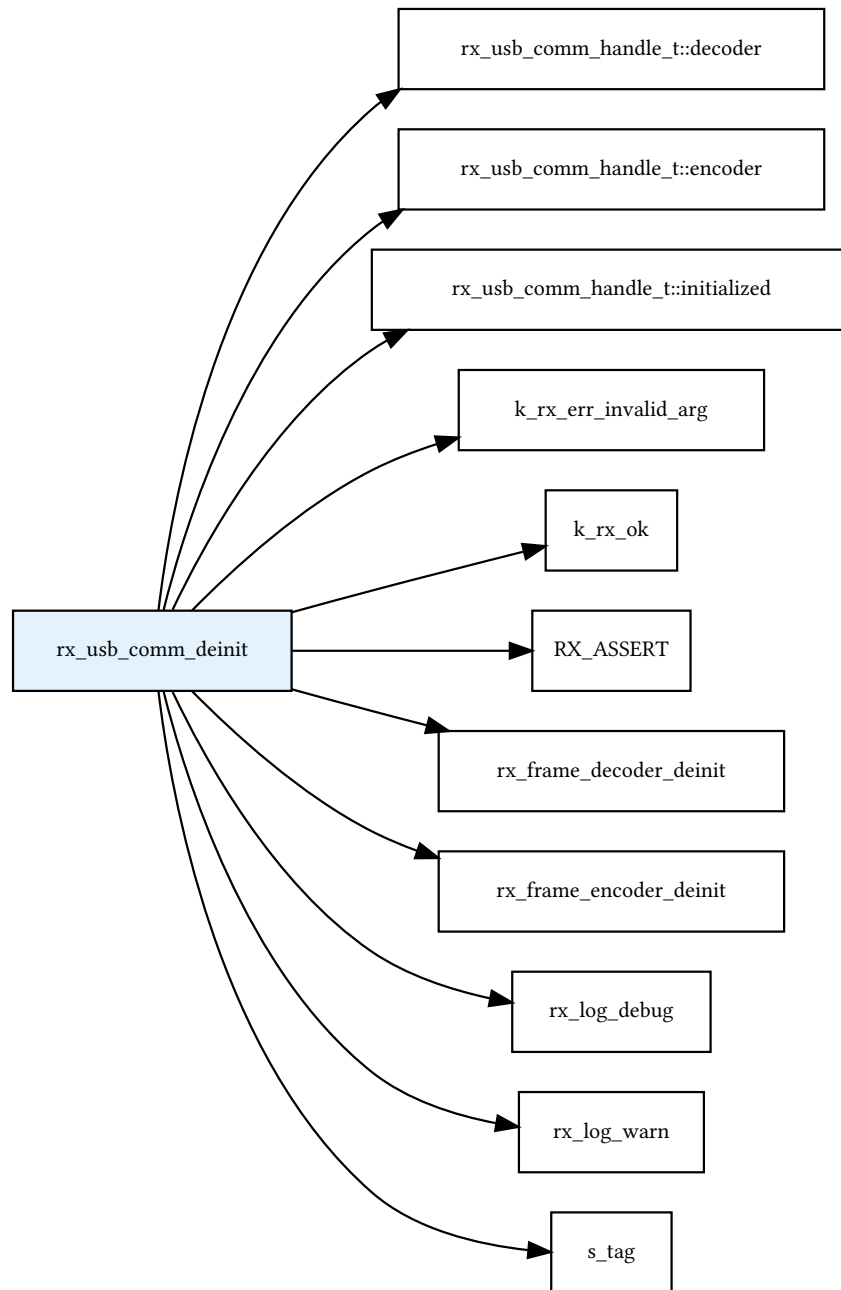
Post: Frame encoder/decoder resources released

Note: Safe to call multiple times (idempotent after first call)

Note: Logs warning but continues if sub-deinit fails

Algorithm:: Validate handle pointer Assert handle was initialized (catches programming errors) If initialized, deinit encoder (log warning on failure) Deinit decoder (log warning on failure) Mark handle as not initialized Return first error encountered (or k_rx_ok)

See also: rx_usb_comm_init() Initialize before use

Figure 1266: Call graph for `rx_usb_comm_deinit`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:963`

internal_build_frame

```
static rx_err_t internal_build_frame(rx_frame_t *frame, const uint16_t sequence,
const rx_frame_type_t type, const uint8_t flags, const uint8_t *payload, const
uint32_t payload_len)
```

Build frame structure from individual parameters.

Constructs an rx_frame_t structure by populating header fields and copying payload data. Does not compute CRC - that is done during encoding.

Parameters:

Direction	Name	Description
out	frame	Output frame structure to populate
in	sequence	Sequence number for frame (0-65535)
in	type	Frame type (command, response, ack, etc.)
in	flags	Frame flags bitmap
in	payload	Payload data (can be nullptr if payload_len is 0)
in	payload_len	Payload length in bytes (0 to k_frame_max_payload)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success, frame populated
k_rx_err_invalid_arg	nullptr frame or (nullptr payload with len > 0)

Pre: frame != nullptr

Pre: payload != nullptr || payload_len == 0

Post: frame->header fields populated

Post: frame->payload contains copy of input payload

Note: Does not compute CRC - use rx_frame_encode() for that

Note: Payload is copied, caller can free original after call

Algorithm:: Validate frame pointer (RX_ASSERT + runtime check) Validate payload pointer if length > 0 Populate header fields (sequence, length, type, flags) Copy payload data if present

See also: rx_usb_comm_send() Uses this to build frames for transmission

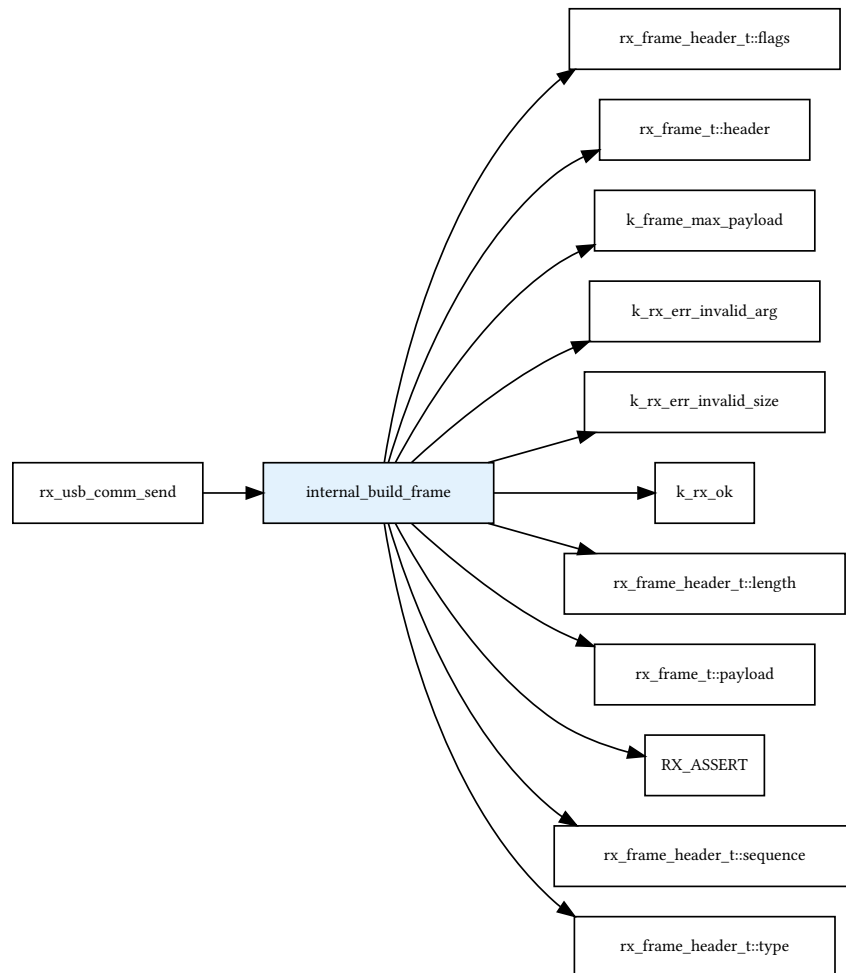


Figure 1267: Call graph for internal_build_frame

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1036`

rx_usb_comm_send

```
rx_err_t rx_usb_comm_send(rx_usb_comm_handle_t *handle, const rx_frame_type_t
type, const uint8_t flags, const uint8_t *payload, const uint32_t payload_len)
```

Send a response frame over USB.

Encodes the payload into a frame with automatic sequence number assignment and CRC-32 calculation. The frame is queued to the USB TX buffer for transmission. This operation is non-blocking - it returns as soon as the data is queued (not when transmission completes).

Parameters:

Direction	Name	Description
inout	handle	Initialized handle Must be initialized via rx_usb_comm_init() Session TX sequence incremented on success
in	type	Frame type (typically k_frame_type_response) See rx_frame_type_t for valid types
in	flags	Frame flags (ORed together) See rx_frame_flags_t for valid flags 0 for no flags
in	payload	Pointer to payload data Must not be nullptr (even for zero-length payload, use dummy) Copied to internal buffer (no ownership transfer)
in	payload_len	Payload length in bytes Max: k_usb_comm_tx_buffer_size - frame overhead (2036 bytes) 0 is valid (empty payload)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Frame queued successfully
k_rx_err_invalid_arg	handle or payload is nullptr
k_rx_err_invalid_state	Not initialized, wrong mode, or USB not configured
k_rx_err_invalid_size	Payload exceeds maximum frame size
k_rx_err_busy	USB TX buffer full, try again later

Pre: handle initialized via rx_usb_comm_init()

Pre: handle->mode == k_usb_comm_mode_binary

Pre: USB driver initialized and configured

Pre: payload !=nullptr

Post: On success: session TX sequence incremented

Post: On success: frame queued for transmission

Post: On failure: no state changes

Note: Non-blocking: returns immediately after queueing

Note: Sends on Port 0 (k_usb_port_proto)

Note: Thread safety: caller must provide external synchronization

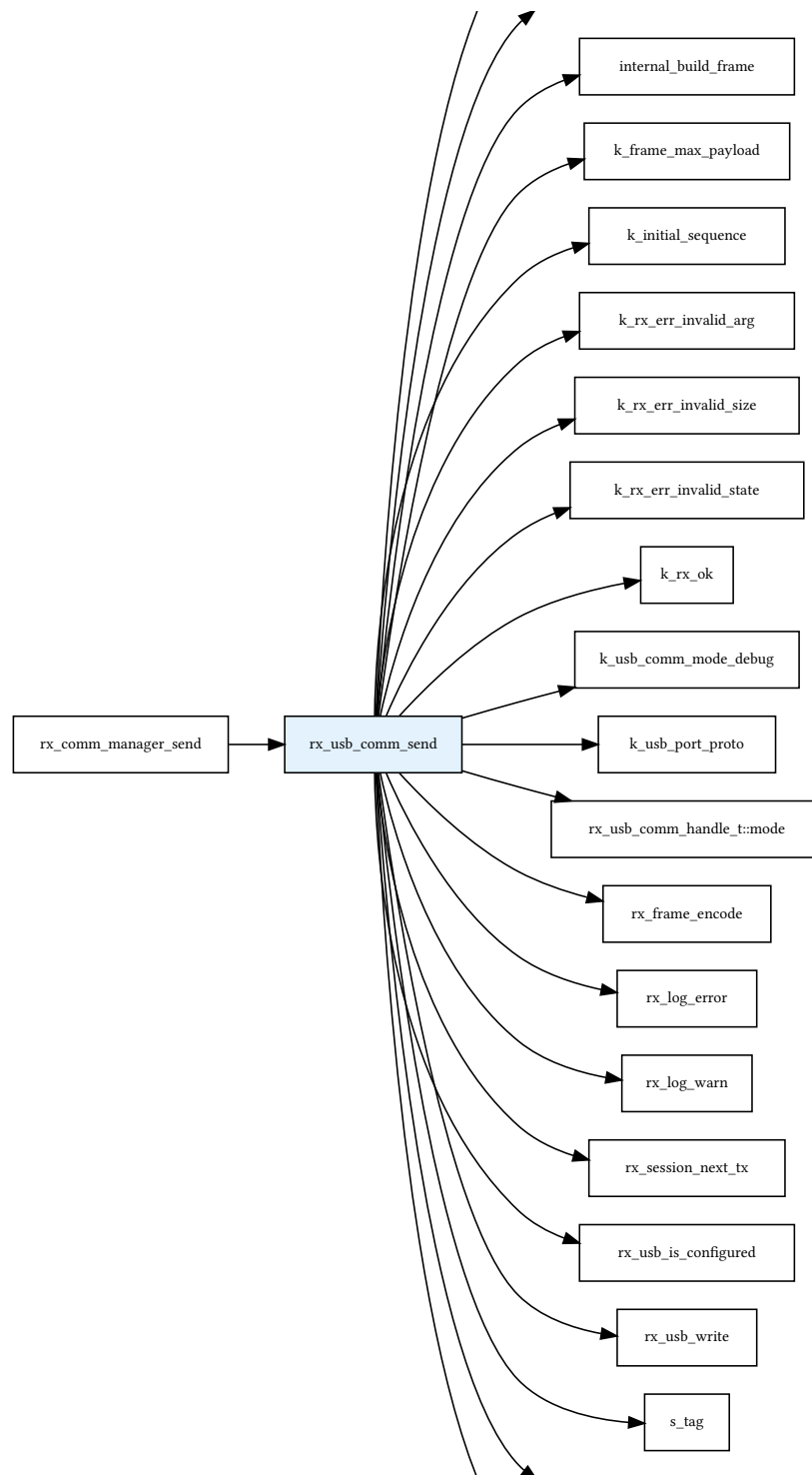
Warning: In debug mode, returns k_rx_err_invalid_state] **Frame Structure (Generated):** +---
 +---+---+---+---+---+---+---+ |SYNC|TYPE|FLAG|SEQ|LENGTH|PAYLOAD|CRC32| |0xAA|
 type|flags|auto|len|data|auto| +---+---+---+---+---+---+---+ +

Algorithm: Validate handle, payload pointers Check initialization and mode (must be binary mode)
 Verify USB is configured and ready Check payload fits in buffer Build frame header with current
 sequence number Copy payload to frame Calculate and append CRC-32 Queue frame to USB TX
 buffer Increment sequence counter

Example: uint8_tpb_buffer[256];
 size_tpb_len=encode_protobuf_response(pb_buffer,sizeof(pb_buffer));
 rx_err_terr=rx_usb_comm_send(&handle, k_frame_type_response, 0, pb_buffer, (uint32_t)pb_len);
 if(err==k_rx_err_busy){ tx_thread_sleep(1); err=rx_usb_comm_send(...); }

Performance: 10-50 us (encoding + USB queue)

See also: rx_usb_comm_receive() Receive frames from peer, rx_frame_type_t Frame type
 definitions

Figure 1268: Call graph for `rx_usb_comm_send`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1072`

Since Version 1.0.0

—

rx_usb_comm_receive

```
rx_err_t rx_usb_comm_receive(rx_usb_comm_handle_t *handle, rx_frame_t *frame,  
    const uint32_t timeout_ms)
```

Receive and decode a frame from USB.

Reads data from the USB CDC buffer, feeds it through the frame decoder, validates CRC-32, and extracts the payload. The function can operate in blocking mode (with timeout) or polling mode (timeout_ms = 0).

Parameters:

Direction	Name	Description
inout	handle	Initialized handle Must be initialized via rx_usb_comm_init() Internal buffers and decoder state modified
out	frame	Pointer to frame structure for output Must be valid non-nullptr All fields populated on success
in	timeout_ms	Timeout in milliseconds 0: Poll once, return immediately if no frame >0: Block up to timeout_ms waiting for complete frame

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Complete frame received and decoded
k_rx_err_invalid_arg	handle or frame is nullptr
k_rx_err_invalid_state	Not initialized or USB not configured
k_rx_err_timeout	No complete frame within timeout (timeout_ms > 0)
k_rx_err_no_data	No data available (timeout_ms == 0)
k_rx_err_crc_mismatch	CRC-32 validation failed
k_rx_err_protocol_error	Invalid frame format (bad sync, length)

Pre: handle initialized via rx_usb_comm_init()

Pre: USB driver initialized and configured

Pre: frame points to valid rx_frame_t

Post: On success: frame contains decoded data

Post: On success: session RX sequence updated

Post: On CRC error: frame may be partially populated

Note: Receives from Port 0 (k_usb_port_proto)

Note: Loop iterations bounded by k_usb_comm_max_receive_iterations

Note: Thread safety: caller must provide external synchronization] **Algorithm:** Validate handle and frame pointers Check initialization and USB ready state Loop until frame complete or timeout: a. Read available bytes from USB buffer b. Feed bytes to frame decoder c. Check if complete frame received d. If not complete and timeout_ms > 0, wait briefly On complete frame: validate CRC, check sequence Copy decoded frame to output Update expected sequence number

Example: rx_frame_tframe;

```
rx_err_t err=rx_usb_comm_receive(&handle,&frame,100);
```

```
switch(err){ casek_rx_ok: process_frame(&frame); break; casek_rx_err_timeout: break;
```

```
casek_rx_err_crc_mismatch: rx_log_warn("USB","CRCmismatch"); break; default:
```

```
rx_log_error("USB","Receiveerror"); break; }
```

Performance: Polling (timeout_ms=0): 5-20 us With timeout: depends on data availability

See also: rx_usb_comm_send() Send frames to peer, rx_usb_comm_data_available() Check if data waiting, rx_frame_t Frame structure definition

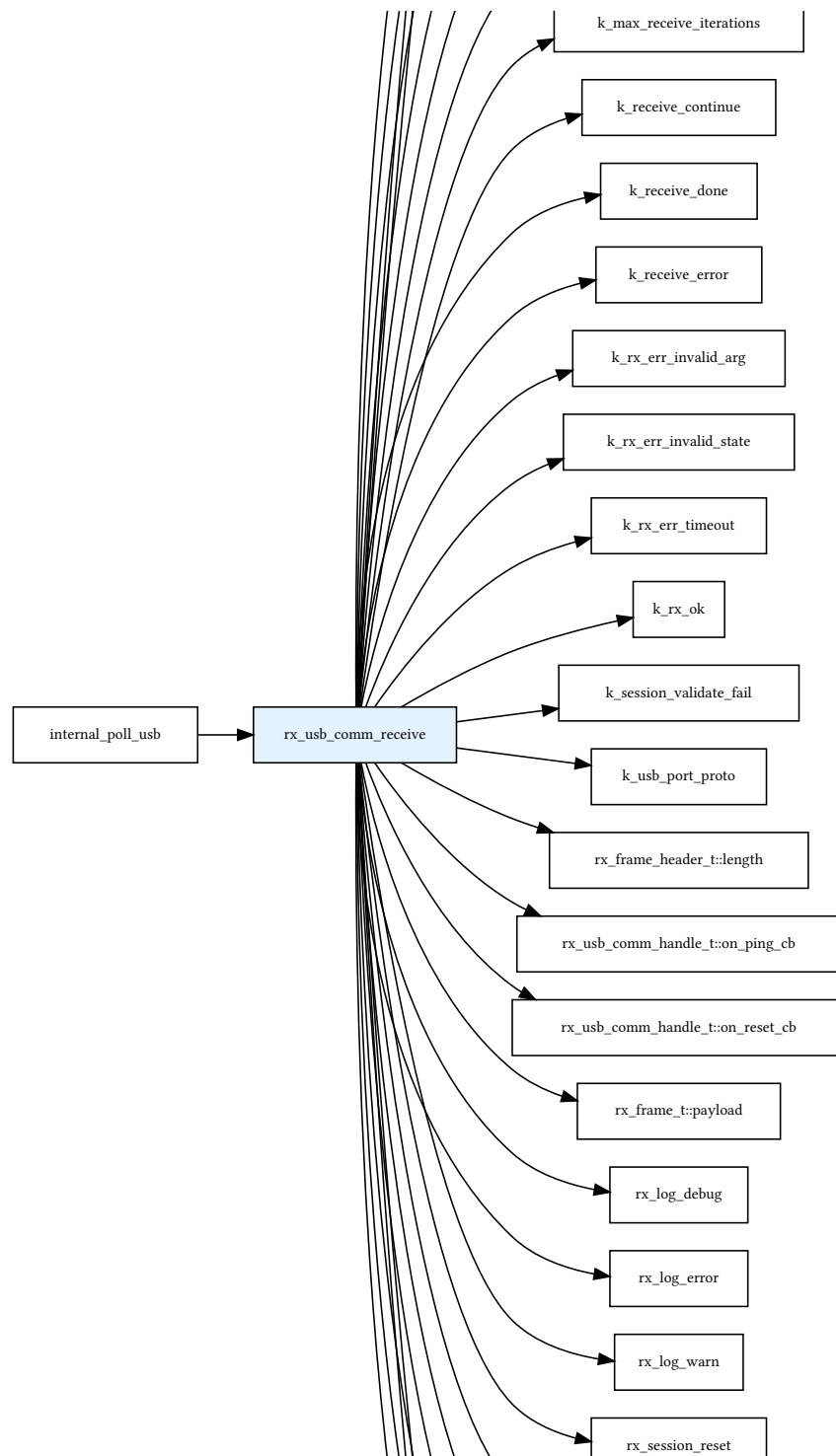


Figure 1269: Call graph for rx_usb_comm_receive

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1149`

Since Version 1.0.0

rx_usb_comm_data_available

```
rx_err_t rx_usb_comm_data_available(const rx_usb_comm_handle_t *handle, bool
*available)
```

Check if data is available to receive.

Check if data is available for reading.

Parameters:

Direction	Name	Description
in	handle	USB communication handle
out	available	Set to true if data available, false otherwise

Returns: Error code from USB RX availability check on failure

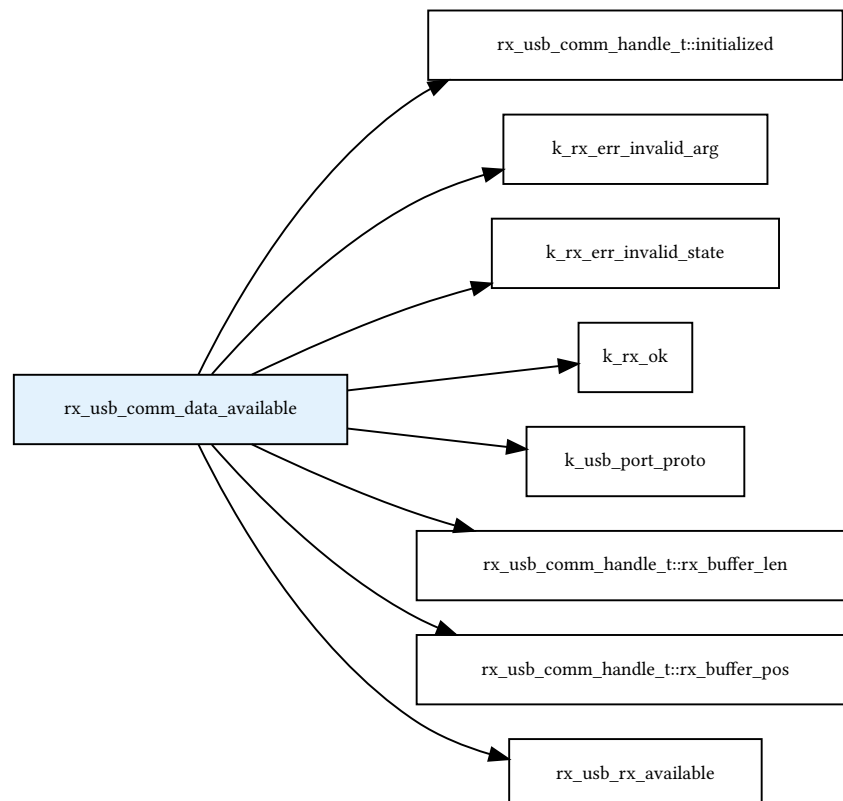


Figure 1270: Call graph for `rx_usb_comm_data_available`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1264`

—

rx_usb_comm_is_ready

```
rx_err_t rx_usb_comm_is_ready(const rx_usb_comm_handle_t *handle, bool *ready)
```

Check if USB communication is ready to send/receive.

Check if USB is connected and ready.

Parameters:

Direction	Name	Description
in	handle	USB communication handle
out	ready	Set to true if ready, false otherwise

Returns: `k_rx_err_invalid_state` if handle not initialized

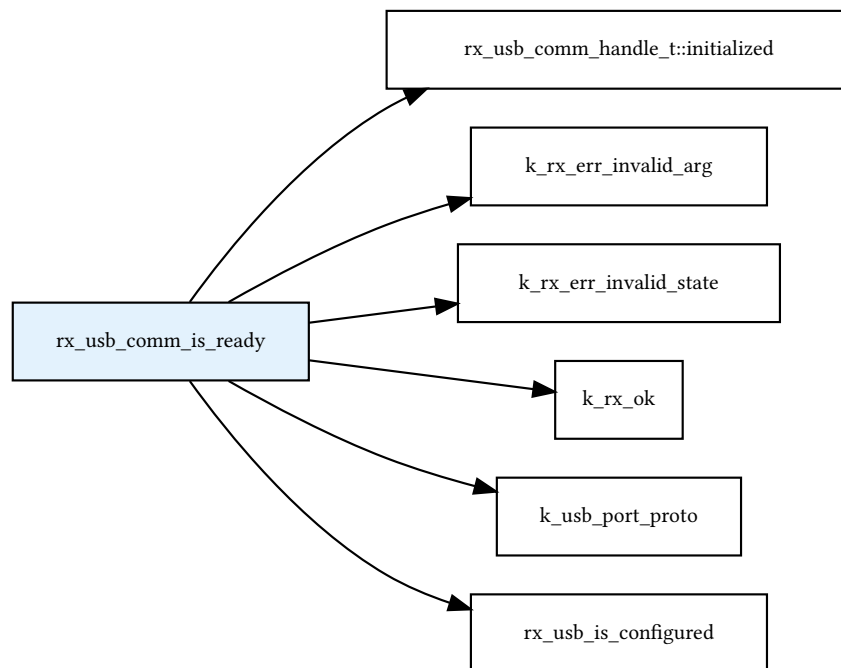


Figure 1271: Call graph for `rx_usb_comm_is_ready`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1296`

—

rx_usb_comm_flush_rx

```
void rx_usb_comm_flush_rx(rx_usb_comm_handle_t *handle)
```

Flush the USB receive buffer, discarding all pending data.

Flush internal receive buffer.

Resets the internal RX buffer position and length to zero, effectively discarding any partially received or buffered data. This is useful after error recovery or mode switching to ensure a clean receive state.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle (nullptr-safe: no-op if nullptr)

Returns: void

Pre: handle should be initialized, but nullptr is safely handled

Post: handle->rx_buffer_len == 0

Post: handle->rx_buffer_pos == 0

Note: Not thread-safe: caller must serialize access to handle

Warning: Any partially received frame data will be lost] **See also:** rx_usb_comm_receive()
Receive path that populates the buffer

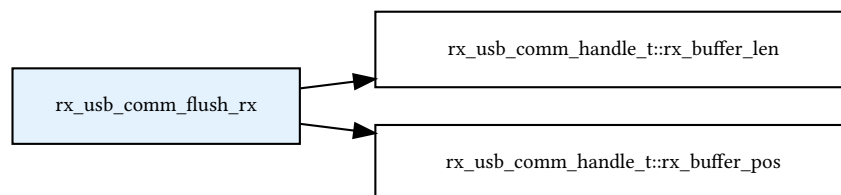


Figure 1272: Call graph for rx_usb_comm_flush_rx

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1338`

Since Version 1.0.0

rx_usb_comm_set_control_callbacks

```

rx_err_t rx_usb_comm_set_control_callbacks(rx_usb_comm_handle_t *handle,
void(*on_ping_cb)(const rx_frame_t *frame, void *ctx), void(*on_reset_cb)(const
rx_frame_t *frame, void *ctx), void *cb_ctx)
  
```

Register callbacks for PING and RESET control frames.

Register callbacks for control frame notifications.

Sets application-level callbacks invoked after control frame auto-response. PING callback fires after PONG is sent; RESET callback fires after RESET_ACK is sent and session is reset. Either callback may be NULL to disable notification for that frame type.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle (must be initialized)
in	on_ping_cb	Callback for PING frames (NULL to disable)
in	on_reset_cb	Callback for RESET frames (NULL to disable)
in	cb_ctx	Opaque context pointer forwarded to callbacks

Returns: rx_err_t Error code

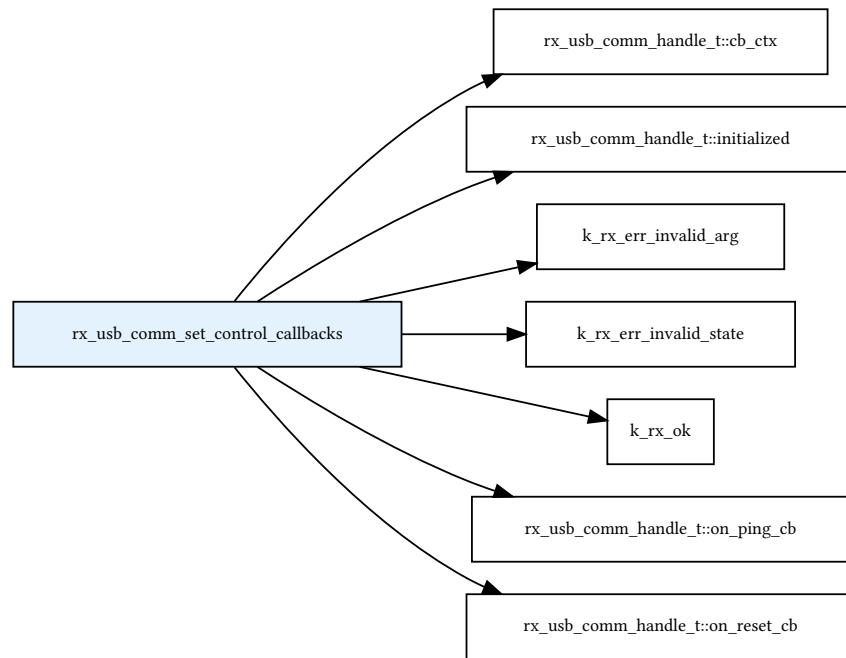
Value	Description
k_rx_ok	Callbacks registered successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle is not initialized

Pre: handle must be initialized via rx_usb_comm_init()

Post: handle->on_ping_cb, on_reset_cb, cb_ctx updated

Note: Not thread-safe: call during initialization or from single thread

Note: Callback ownership: caller retains ownership of cb_ctx] **See also:**
rx_usb_comm_send_pong() Auto-response before PING callback,
rx_usb_comm_send_reset_ack() Auto-response before RESET callback

Figure 1273: Call graph for `rx_usb_comm_set_control_callbacks`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1380`

Since Version 1.0.0

—

`rx_usb_comm_send_pong`

```
rx_err_t rx_usb_comm_send_pong(rx_usb_comm_handle_t *handle, const uint8_t
*payload, const uint16_t payload_len)
```

Send a PONG frame in response to a received PING.

Send a PONG response frame.

Constructs and sends a PONG frame echoing the PING payload (typically a 4-byte counter). Uses the shared session for TX sequence assignment.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle (must be initialized)
in	payload	PING payload to echo (may be NULL if payload_len == 0)
in	payload_len	Length of payload in bytes

Returns: `rx_err_t` Error code

Value	Description
k_rx_ok	PONG sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized

Post: TX sequence incremented by 1

Note: Called automatically by rx_usb_comm_receive() on PING frames] **See also:**
rx_frame_create_pong() Frame construction, rx_session_next_tx() Sequence assignment

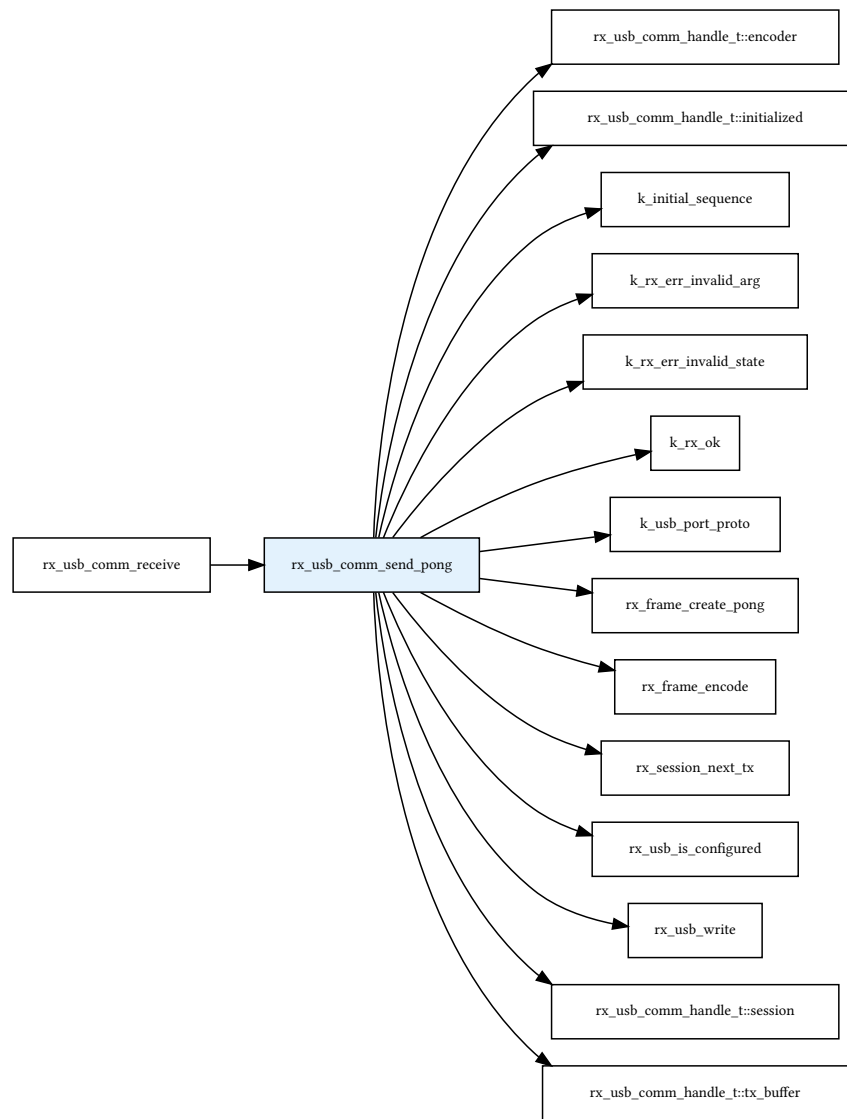


Figure 1274: Call graph for rx_usb_comm_send_pong

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1425`

Since Version 1.0.0

—

rx_usb_comm_send_reset_ack

```
rx_err_t rx_usb_comm_send_reset_ack(rx_usb_comm_handle_t *handle)
```

Send a RESET_ACK frame in response to a received RESET.

Send a RESET_ACK response frame.

Constructs and sends a RESET_ACK frame (no payload) to acknowledge a session reset request.

Uses the shared session for TX sequence assignment.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle (must be initialized)

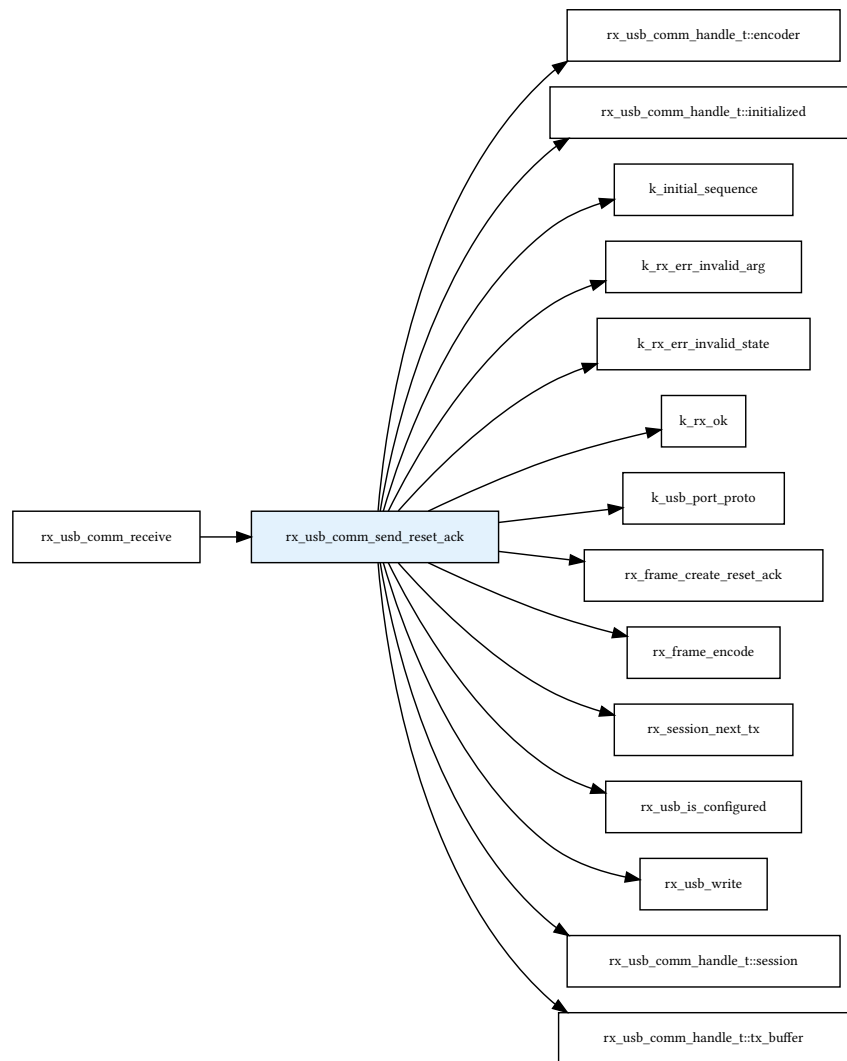
Returns: rx_err_t Error code

Value	Description
k_rx_ok	RESET_ACK sent successfully
k_rx_err_invalid_arg	handle is nullptr
k_rx_err_invalid_state	handle not initialized

Pre: handle must be initialized

Post: TX sequence incremented by 1

Note: Called automatically by rx_usb_comm_receive() on RESET frames] **See also:**
rx_frame_create_reset_ack() Frame construction, rx_session_reset() Session reset
after ACK

Figure 1275: Call graph for `rx_usb_comm_send_reset_ack`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1488`

Since Version 1.0.0

—

rx_usb_comm_set_mode

```
rx_err_t rx_usb_comm_set_mode(rx_usb_comm_handle_t *handle, const
rx_usb_comm_mode_t mode)
```

Set USB communication mode (binary protocol or debug text).

Set USB communication mode.

Parameters:

Direction	Name	Description
inout	handle	USB communication handle
in	mode	Mode to set (k_usb_comm_mode_binary or k_usb_comm_mode_debug)

Returns: k_rx_err_invalid_state if handle not initialized

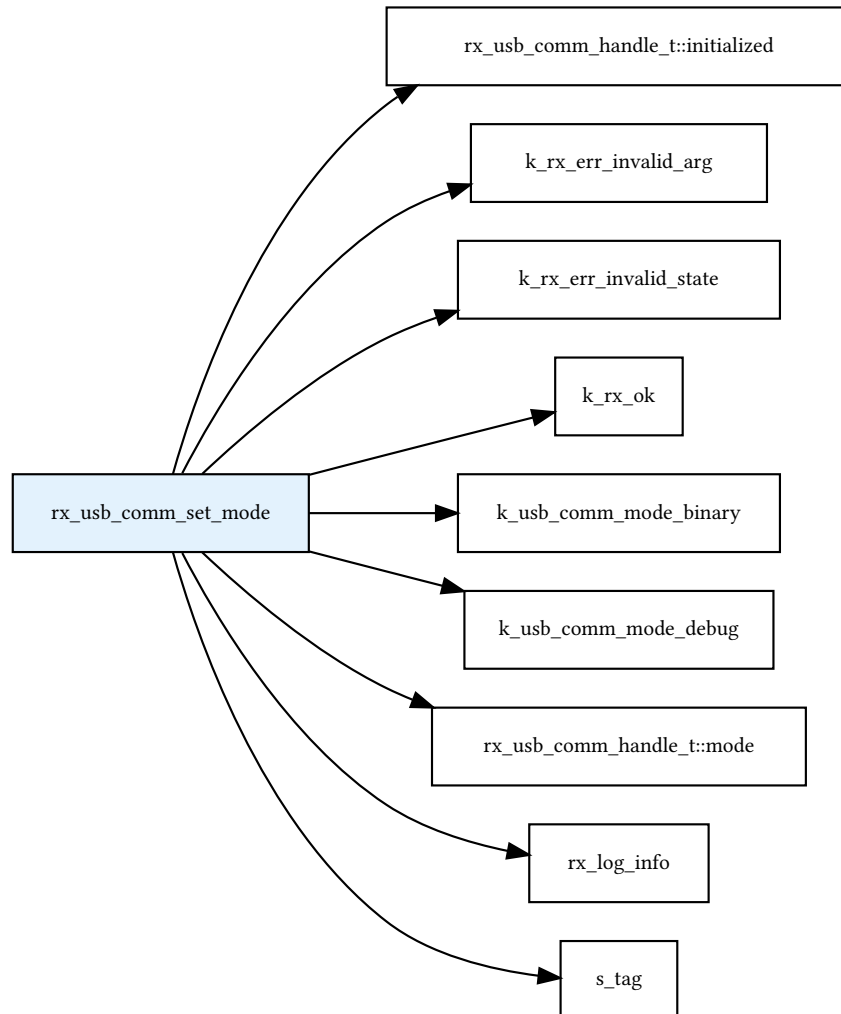


Figure 1276: Call graph for `rx_usb_comm_set_mode`

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1539`

rx_usb_comm_get_mode

```
rx_usb_comm_mode_t rx_usb_comm_get_mode(const rx_usb_comm_handle_t *handle)
```

Get current USB communication mode.

Parameters:

Direction	Name	Description
in	handle	USB communication handle

Returns: Current mode (k_usb_comm_mode_invalid if handle is nullptr or not initialized)

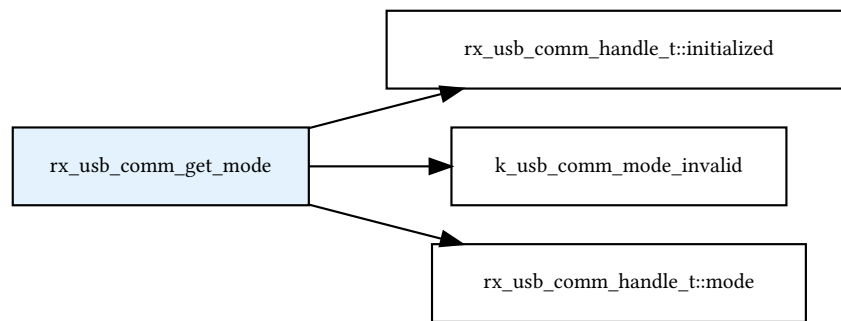


Figure 1277: Call graph for rx_usb_comm_get_mode

Defined in `libs/rx_usb_comm/src/rx_usb_comm.c:1572`

—

default_interrupt_handlers.c

Default exception and interrupt handlers for RX72N boot sequence.

Provides weak default implementations of exception handlers referenced by vecttbl.c. These can be overridden by user implementations if needed.

Default behavior: Infinite loop (halt on exception). This is the safest default for unhandled exceptions in safety-critical embedded systems.

Copyright (c) 2026 STAR Project

Includes:

- "platform.h"

boot_common.h

Boot support header providing common definitions for SMC-generated boot files.

Provides minimal definitions needed by boot files moved from smc_gen/. Currently provides the INTERNAL_NOT_USED macro needed by SMC-generated boot files.

Copyright (c) 2026 STAR Project

INTERNAL_NOT_USED

```
#define INTERNAL_NOT_USED ((void)(p))
```

Suppress “unused parameter” warnings for API-required parameters.

Since Version 1.0.0

—

platform.h

Boot platform header providing BSP definitions for boot file compilation.

Provides all necessary BSP definitions for boot file compilation without Smart Configurator. The boot files (reset_program.S, resetprg.c, lowsrc.c, lowlvl.c, vecttbl.c, dbsct.c) are SMC-generated code with extensive dependencies on Renesas BSP headers. This header routes to our minimal r_bsp.h which includes only what boot actually needs:

- Standard C types (stdint.h, stdbool.h)
- R_BSP_* macros (r_rx_compiler.h)
- BSP configuration (r_bsp_config.h)
- INTERNAL_NOT_USED macro (boot_common.h)

Copyright (c) 2026 STAR Project

Includes:

- "r_bsp.h"

r_bsp.h

Boot-compatible minimal replacement for full Renesas SMC r_bsp.h.

Provides only the headers and macros required by boot sequence files (resetprg.c, vecttbl.c, lowsrc.c, etc.) without requiring the full Renesas Smart Configurator BSP package.

Includes:

- Standard C types (stdint.h, stdbool.h)
- R_BSP_* compiler macros (r_rx_compiler.h)
- BSP configuration (r_bsp_config.h)
- INTERNAL_NOT_USED macro (boot_common.h)
- Function prototypes for lowlvl/lowsrc

Copyright (c) 2026 STAR Project

Includes:

- <stdbool.h>
- <stddef.h>
- <stdint.h>
- <stdio.h>
- "boot_common.h"
- "mcu_info.h"
- "r_bsp_config.h"
- "r_rx_compiler.h"
- "r_rx_intrinsic_functions.h"
- "lowlvl.h"
- "lowsrc.h"
- "r_rtos.h"

r_rx_intrinsic_functions.c

Implementation of RX intrinsic functions for GNUC and ICCRX compilers.

Provides implementations for RX architecture intrinsic functions that are built-in to the Renesas CC-RX compiler but not available in GCC (GNURX) or IAR ICCRX compilers. These functions bridge compiler differences to enable portable BSP code across all three supported toolchains.

Implementation Techniques:

- Inline Assembly: Register access, special instructions, TFU operations
- C Fallbacks: Arithmetic operations (max/min, rotations)
- Algorithm Emulation: Multiply-accumulate for compilers lacking intrinsics

Function Categories:

- Arithmetic: Max/min, multiply-accumulate operations
- Bit Manipulation: Rotation with/without carry, bit set/clear/reverse
- Register Access: PSW, ACC, BPC, BPSW, EXTB, INTB, etc.
- CPU Control: Mode switching, interrupt priority
- Trigonometry: TFU (Trigonometric Function Unit) wrappers
- Double-Precision FPU: DPFPU control registers (if DPFPU defined)

Compiler-Specific Implementation:

- CC-RX: Uses native intrinsics - this file not used
- GNUC: Most functions implemented here with inline assembly
- ICCRX: Some functions implemented here, others use IAR intrinsics

Inline Assembly Usage (GNUC): Many functions use inline assembly with `R_BSP_ATTRIB_INLINE_ASM` attribute to directly execute RX instructions not exposed by GCC built-ins. Assembly syntax follows GCC inline asm conventions for RX architecture.

Renesas Electronics Corporation (original), STAR Project/Locked, Inc. (modifications)

2019-02-28 (original), 2026-02-13 (STAR modifications)

1.05

Copyright (C) 2019 Renesas Electronics Corporation. Modified by Locked, Inc.

Includes:

- "r_rx_intrinsic_functions.h"
- "r_rx_compiler.h"

bsp_get_bpsw

```
R_BSP_ATTRIB_STATIC_INLINE_ASM void bsp_get_bpsw(uint32_t *data)
```

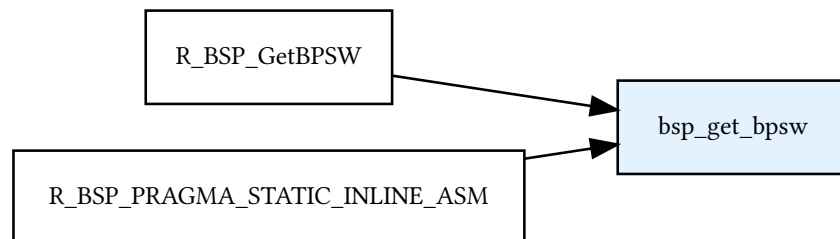


Figure 1278: Call graph for `bsp_get_bpsw`

Defined in `src/boot/r_bsp/r_rx_intrinsic_functions.c:148`

bsp_get_bpc

```
R_BSP_ATTRIB_STATIC_INLINE_ASM void bsp_get_bpc(uint32_t *data)
```

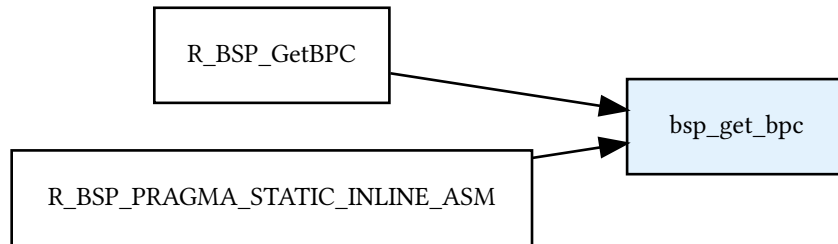


Figure 1279: Call graph for bsp_get_bpc

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:149*

bsp_move_from_acc_hi_long

```
R_BSP_ATTRIB_STATIC_INLINE_ASM void bsp_move_from_acc_hi_long(uint32_t *data)
```

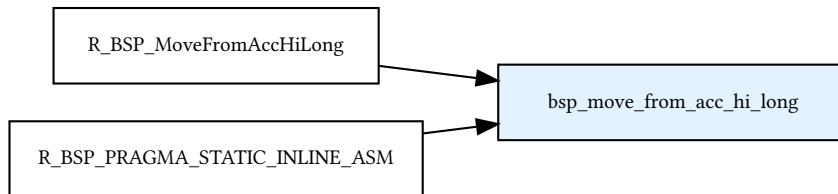


Figure 1280: Call graph for bsp_move_from_acc_hi_long

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:153*

bsp_move_from_acc_mi_long

```
void bsp_move_from_acc_mi_long(uint32_t *data)
```

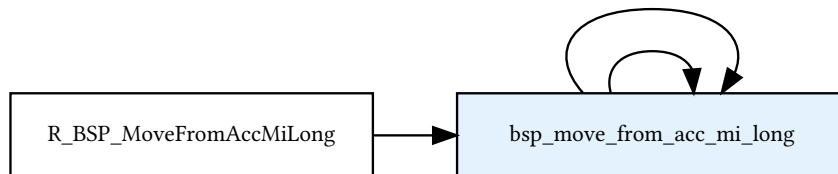


Figure 1281: Call graph for bsp_move_from_acc_mi_long

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:938*

R_BSP_ChangeToUserMode

```
void R_BSP_ChangeToUserMode(void)
```

Get maximum of two signed long values.

Selects the greater of two input values. GNUC implementation using C comparison operator (ternary conditional).

CC-RX and ICCRX use compiler intrinsics (max()) and MAX respectively). This function provides equivalent functionality for GNUC.

Get minimum of two signed long values

Selects the smaller of two input values. GNUC implementation using C comparison operator (ternary conditional).

CC-RX and ICCRX use compiler intrinsics (min()) and MIN respectively). This function provides equivalent functionality for GNUC.

Multiply-and-accumulate operation on signed char arrays

Performs multiply-and-accumulate (MAC) operation on byte-sized arrays: result = init + sum(addr1[i] * addr2[i]) for i=0 to count-1

GNUC implementation uses C loop since RMPA.B instruction not available as GCC built-in. CC-RX uses rmpab() intrinsic for hardware acceleration.

Algorithm:

1. Initialize accumulator with init value
2. For each element: accumulator += addr1[i] * addr2[i]
3. Return lower 64 bits of accumulated result

Multiply-and-accumulate operation on short arrays

Performs multiply-and-accumulate (MAC) operation on word-sized arrays: result = init + sum(addr1[i] * addr2[i]) for i=0 to count-1

GNUC implementation uses C loop since RMPA.W instruction not available as GCC built-in. CC-RX uses rmpaw() intrinsic for hardware acceleration.

Algorithm:

1. Initialize accumulator with init value
2. For each element: accumulator += addr1[i] * addr2[i]
3. Return lower 64 bits of accumulated result

Parameters:

Direction	Name	Description
in	data1	First value to compare
in	data2	Second value to compare
in	data1	First value to compare
in	data2	Second value to compare
in	init	Initial accumulator value (64-bit)
in	count	Number of elements to process (must be > 0)

in	addr1	Pointer to first array (signed char, count elements)
in	addr2	Pointer to second array (signed char, count elements)
in	init	Initial accumulator value (64-bit)
in	count	Number of elements to process (must be > 0)
in	addr1	Pointer to first array (short, count elements)
in	addr2	Pointer to second array (short, count elements)

Returns: long long Lower 64 bits of: $\text{init} + \text{sum}(\text{addr1}[n] * \text{addr2}[n])$

Value	Description
data1	If $\text{data1} > \text{data2}$
data2	If $\text{data2} \geq \text{data1}$
data1	If $\text{data1} < \text{data2}$
data2	If $\text{data2} \leq \text{data1}$

Pre: None

Pre: None

Pre: addr1 points to valid array of at least count elements

Pre: addr2 points to valid array of at least count elements

Pre: count > 0 (loop bounds checked per NASA Rule 2)

Pre: addr1 points to valid array of at least count elements

Pre: addr2 points to valid array of at least count elements

Pre: count > 0 (loop bounds checked per NASA Rule 2)

Post: Return value is greater than or equal to both inputs

Post: Return value is less than or equal to both inputs

Post: Accumulator contains sum of products plus initial value

Post: Accumulator contains sum of products plus initial value

Note: GNUC-specific implementation

Note: Thread-safe (pure function with no side effects)

Note: Typically used via `R_BSP_MAX` macro

Note: GNUC-specific implementation

Note: Thread-safe (pure function with no side effects)

Note: Typically used via `R_BSP_MIN` macro

Note: GNUC-specific implementation (C fallback)

Note: Not thread-safe if `addr1` or `addr2` arrays are shared

Note: Typically used via `R_BSP_RMPAB` macro

Note: GNUC-specific implementation (C fallback)

Note: Not thread-safe if `addr1` or `addr2` arrays are shared

Note: Typically used via `R_BSP_RMPAW` macro] **See also:** `R_BSP_Min()` Corresponding minimum function, `R_BSP_MAX` Portable macro wrapper, `R_BSP_Max()` Corresponding maximum function, `R_BSP_MIN` Portable macro wrapper, `R_BSP_MulAndAccOperation_W()` Word-sized variant, `R_BSP_MulAndAccOperation_L()` Long-sized variant, `R_BSP_RMPAB` Portable macro wrapper, `R_BSP_MulAndAccOperation_B()` Byte-sized variant, `R_BSP_MulAndAccOperation_L()` Long-sized variant, `R_BSP_RMPAW` Portable macro wrapper

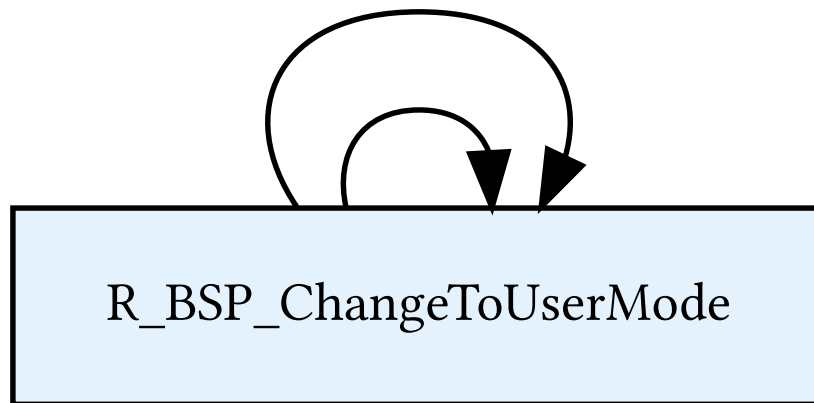


Figure 1282: Call graph for R_BSP_ChangeToUserMode

Defined in `src/boot/r_bsp/r_rx_intrinsic_functions.c:474`

Since Version 1.0.0

—

R_BSP_SetBPSW

```
void R_BSP_SetBPSW(uint32_t data)
```

Multiply-and-accumulate operation on 16-bit arrays with middle accumulator.

Performs multiply-and-accumulate (MAC) operation on 16-bit arrays and returns the middle 32 bits of the 48-bit accumulator result. Uses DSP instructions (MULLO, MACLO, MACHI) for hardware acceleration on CC-RX, with C fallback for GNUC.

Algorithm:

1. Clear accumulator (MULLO with 0,0)
2. Process pairs: $ACC += data1[i] * data2[i] + data1[i+1] * data2[i+1]$
3. Process remaining odd element if count is odd
4. Extract middle 32 bits via MVFACMI

The function processes 16-bit elements in pairs (as 32-bit longs) for efficiency, then handles any remaining odd element separately.

Multiply-and-accumulate with RACW #1 rounding

Performs multiply-and-accumulate (MAC) operation on 16-bit arrays with RACW #1 rounding applied before extracting high accumulator word. Uses DSP instructions (MULLO, MACLO, MACHI, RACW, MVFACHI) for hardware acceleration on CC-RX, with C fallback for GNUC.

Algorithm:

1. Clear accumulator (MULLO with 0,0)
2. Process pairs: $ACC += data1[i] * data2[i] + data1[i+1] * data2[i+1]$
3. Process remaining odd element if count is odd
4. Apply RACW #1 rounding (round to nearest, ties away from zero)
5. Extract high 16 bits via MVFACHI

RACW #1 performs: $ACC = (ACC + 2^0) \gg 1$ (round and shift right 1 bit)

Multiply-and-accumulate with RACW #2 rounding

Performs multiply-and-accumulate (MAC) operation on 16-bit arrays with RACW #2 rounding applied before extracting high accumulator word. Uses DSP instructions (MULLO, MACLO, MACHI, RACW, MVFACHI) for hardware acceleration on CC-RX, with C fallback for GNUC.

Algorithm:

1. Clear accumulator (MULLO with 0,0)
2. Process pairs: $ACC += data1[i] * data2[i] + data1[i+1] * data2[i+1]$
3. Process remaining odd element if count is odd
4. Apply RACW #2 rounding (round to nearest, ties away from zero)
5. Extract high 16 bits via MVFACHI

RACW #2 performs: $ACC = (ACC + 2^1) \gg 2$ (round and shift right 2 bits)

Parameters:

Direction	Name	Description
in	data1	Pointer to first 16-bit array (count elements)
in	data2	Pointer to second 16-bit array (count elements)
in	count	Number of 16-bit elements to process (must be > 0)
in	data1	Pointer to first 16-bit array (count elements)
in	data2	Pointer to second 16-bit array (count elements)
in	count	Number of 16-bit elements to process (must be > 0)
in	data1	Pointer to first 16-bit array (count elements)
in	data2	Pointer to second 16-bit array (count elements)
in	count	Number of 16-bit elements to process (must be > 0)

Returns: short High 16 bits of accumulator after RACW #2 rounding

Pre: data1 points to valid array of at least count 16-bit elements

Pre: data2 points to valid array of at least count 16-bit elements

Pre: count > 0 (loop bounds checked per NASA Rule 2)

Pre: Arrays properly aligned for 32-bit access (when processing pairs)

Pre: data1 points to valid array of at least count 16-bit elements

Pre: data2 points to valid array of at least count 16-bit elements

Pre: count > 0 (loop bounds checked per NASA Rule 2)

Pre: Arrays properly aligned for 32-bit access (when processing pairs)

Pre: data1 points to valid array of at least count 16-bit elements

Pre: data2 points to valid array of at least count 16-bit elements

Pre: count > 0 (loop bounds checked per NASA Rule 2)

Pre: Arrays properly aligned for 32-bit access (when processing pairs)

Post: ACC contains sum of products

Post: Result is middle 32 bits of 48-bit accumulator

Post: ACC contains rounded sum of products

Post: Result is high 16 bits of rounded accumulator

Post: ACC contains rounded sum of products

Post: Result is high 16 bits of rounded accumulator

Note: GNUC-specific implementation (C fallback with GCC built-ins)

Note: Not thread-safe if ACC is shared

Note: Used for fixed-point DSP operations

Note: GNUC-specific implementation (C fallback with GCC built-ins)

Note: Not thread-safe if ACC is shared

Note: Used for fixed-point DSP with rounding

Note: GNUC-specific implementation (C fallback with GCC built-ins)

Note: Not thread-safe if ACC is shared

Note: Used for fixed-point DSP with rounding] **See also:**

R_BSP_MulAndAccOperation_FixedPoint1() Variant with RACW \#1 rounding,
 R_BSP_MulAndAccOperation_FixedPoint2() Variant with RACW \#2 rounding,
 R_BSP_MulAndAccOperation_2byte() Variant without rounding,
 R_BSP_MulAndAccOperation_FixedPoint2() Variant with RACW \#2 rounding,
 R_BSP_MulAndAccOperation_2byte() Variant without rounding,
 R_BSP_MulAndAccOperation_FixedPoint1() Variant with RACW \#1 rounding

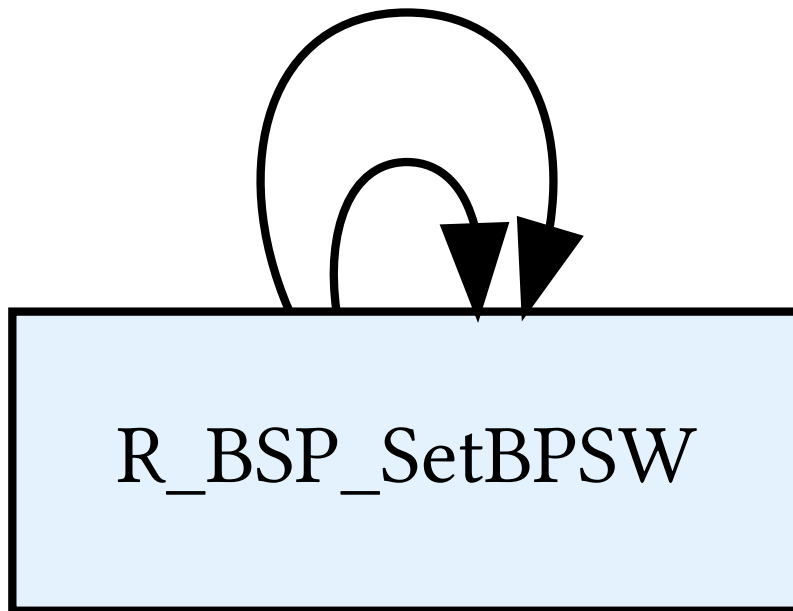


Figure 1283: Call graph for R_BSP_SetBPSW

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:743*

Since Version 1.0.0

—

R_BSP_PRAGMA_STATIC_INLINE_ASM

```
R_BSP_PRAGMA_STATIC_INLINE_ASM(bsp_get_bpsw)
```

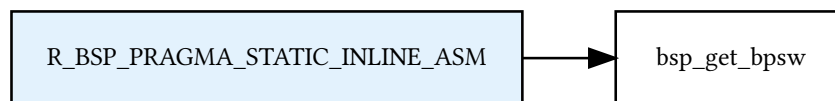


Figure 1284: Call graph for R_BSP_PRAGMA_STATIC_INLINE_ASM

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:754*

—

R_BSP_GetBPSW

```
uint32_t R_BSP_GetBPSW(void)
```

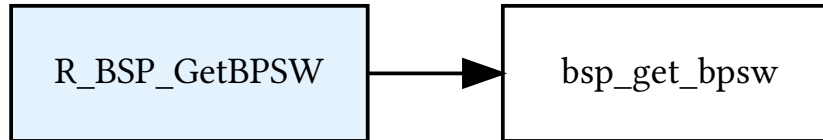


Figure 1285: Call graph for R_BSP_GetBPSW

Defined in `src/boot/r_bsp/r_rx_intrinsic_functions.c:768`

—

R_BSP_SetBPC

```
void R_BSP_SetBPC(void *data)
```

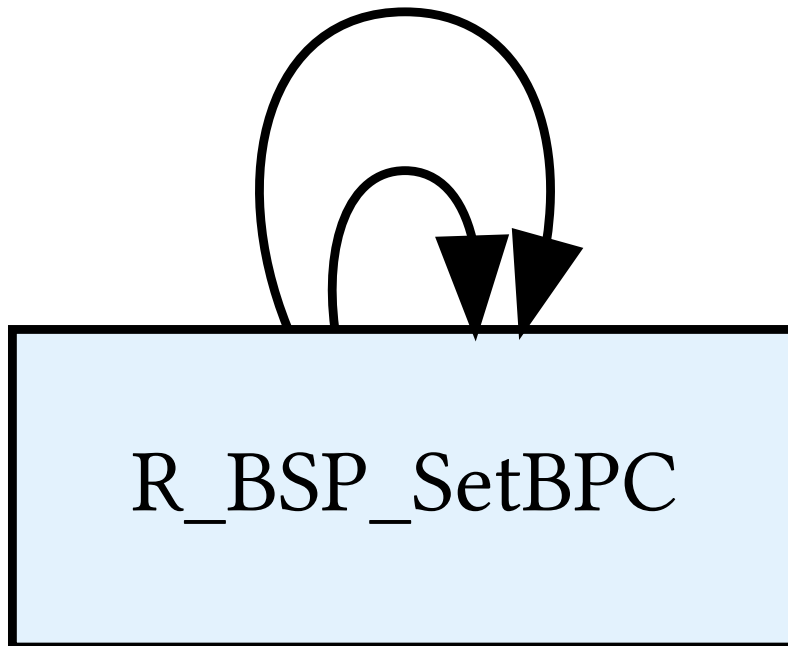


Figure 1286: Call graph for R_BSP_SetBPC

Defined in `src/boot/r_bsp/r_rx_intrinsic_functions.c:784`

—

R_BSP_PRAGMA_STATIC_INLINE_ASM

```
R_BSP_PRAGMA_STATIC_INLINE_ASM(bsp_get_bpc)
```

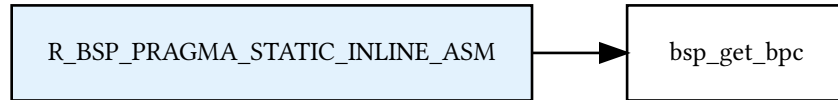


Figure 1287: Call graph for R_BSP_PRAGMA_STATIC_INLINE_ASM

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:795*

R_BSP_GetBPC

```
void * R_BSP_GetBPC(void)
```

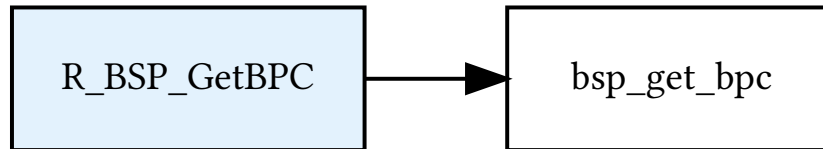


Figure 1288: Call graph for R_BSP_GetBPC

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:815*

R_BSP_MoveToAccHiLong

```
void R_BSP_MoveToAccHiLong(int32_t data)
```

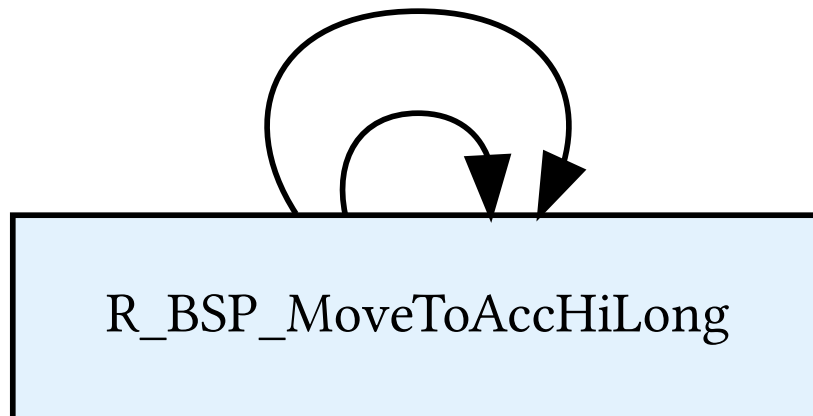


Figure 1289: Call graph for R_BSP_MoveToAccHiLong

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:884*

R_BSP_PRAGMA_INLINE_ASM

```
R_BSP_PRAGMA_INLINE_ASM(R_BSP_MoveToAccLoLong)
```

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:895*

R_BSP_PRAGMA_STATIC_INLINE_ASM

```
R_BSP_PRAGMA_STATIC_INLINE_ASM(bsp_move_from_acc_hi_long)
```

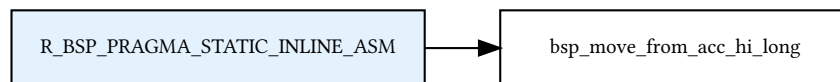


Figure 1290: Call graph for R_BSP_PRAGMA_STATIC_INLINE_ASM

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:907*

R_BSP_MoveFromAccHiLong

```
int32_t R_BSP_MoveFromAccHiLong(void)
```

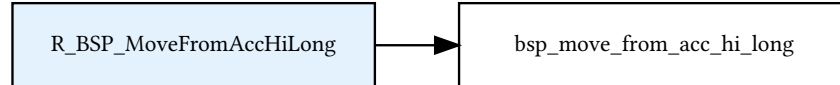


Figure 1291: Call graph for R_BSP_MoveFromAccHiLong

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:922*

R_BSP_MoveFromAccMiLong

```
int32_t R_BSP_MoveFromAccMiLong(void)
```

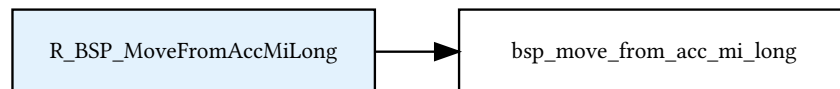


Figure 1292: Call graph for R_BSP_MoveFromAccMiLong

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:952*

R_BSP_BitSet

```
void R_BSP_BitSet(uint8_t *data, uint32_t bit)
```

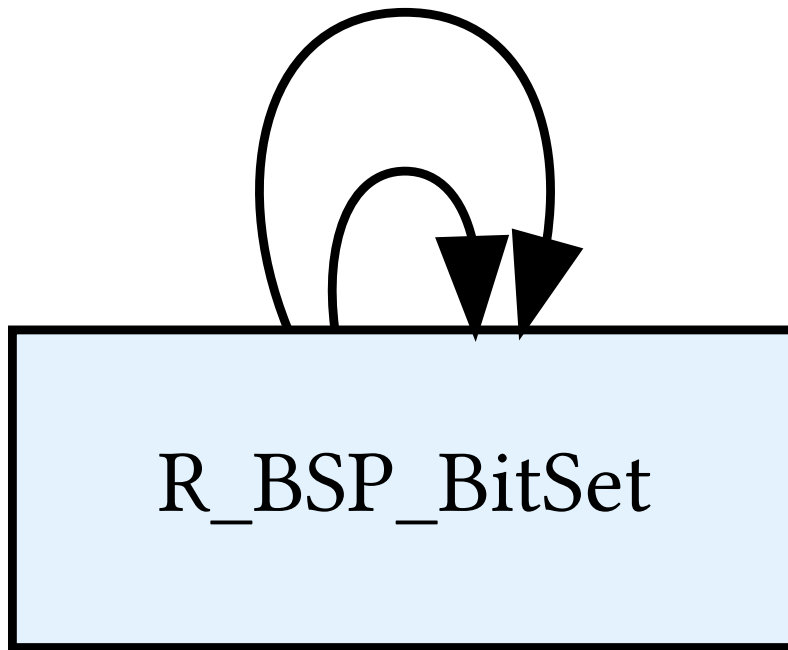


Figure 1293: Call graph for R_BSP_BitSet

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:969*

—

R_BSP_PRAGMA_INLINE_ASM

```
R_BSP_PRAGMA_INLINE_ASM(R_BSP_BitClear)
```

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:981*

—

R_BSP_PRAGMA_INLINE_ASM

```
R_BSP_PRAGMA_INLINE_ASM(R_BSP_BitReverse)
```

Defined in *src/boot/r_bsp/r_rx_intrinsic_functions.c:993*

—

hardware_init.c

Application-Specific Hardware Initialization - Motor Control, Sensors, Communication.

Includes:

- "hardware_init.h"
- "hardware.h"
- "rx72n_sci_regs.h"

- "rx72n_system_regs.h"
- "rx_check.h"
- "rx_err.h"
- "rx_gptw.h"
- "rx_mpc.h"
- "rx_poeg.h"
- "rx_port_utils.h"
- "rx_simulator_config.h"

rx_mpc_pin_t

Port pin identifiers for MPC configuration (rx_port_pin_t values).

Value	Name	Description
= k_rx_p1_2	k_pin_host_scl0	P1.2 - SCL0 (host I2C clock)
= k_rx_p1_3	k_pin_host_sda0	P1.3 - SDA0 (host I2C data)
= k_rx_p1_6	k_pin_usb_vbus	P1.6 - USB0_VBUS (USB VBUS detect)
= k_rx_pf_5	k_pin_sonar_trig0	PF.5 - Sonar 0 trigger (pin 9)
= k_rx_pj_5	k_pin_sonar_trig1	PJ.5 - Sonar 1 trigger (pin 11)
= k_rx_pj_3	k_pin_sonar_trig2	PJ.3 - Sonar 2 trigger (pin 13)
= k_rx_p3_3	k_pin_sonar_trig3	P3.3 - Sonar 3 trigger (pin 26)
= k_rx_p0_3	k_pin_sonar_echo0	P0.3 - Sonar 0 echo (pin 4, IRQ11)
= k_rx_p0_2	k_pin_sonar_echo1	P0.2 - Sonar 1 echo (pin 6, IRQ10)
= k_rx_p0_1	k_pin_sonar_echo2	P0.1 - Sonar 2 echo (pin 7, IRQ9)
= k_rx_p0_0	k_pin_sonar_echo3	P0.0 - Sonar 3 echo (pin 8, IRQ8)
= k_rx_pd_1	k_pin_host_copi	Host SPI (RSPI2 peripheral mode on PORTD) PD1 (MOSIC): COPI - Controller Out, Peripheral In (data from RPi5) PD2 (MISOC): CIPO - Controller In, Peripheral Out (data to RPi5) PD.1 - COPI/MOSIC (RPi5 -> RX72N)
= k_rx_pd_2	k_pin_host_cipo	PD.2 - CIPO/MISOC (RX72N -> RPi5)
= k_rx_pd_3	k_pin_host_sclk	PD.3 - SCLK (RSPI2 clock from RPi5)
= k_rx_pd_4	k_pin_host_cs0	PD.4 - CS0 (chip select from RPi5)
= k_rx_p2_4	k_pin_enc0_pha	P2.4 - MTCLKA (encoder 0 phase A)
= k_rx_p2_5	k_pin_enc0_phb	P2.5 - MTCLKB (encoder 0 phase B)
= k_rx_pa_1	k_pin_enc1_pha	PA.1 - MTCLKC (encoder 1 phase A)
= k_rx_pc_5	k_pin_enc1_phb	PC.5 - MTCLKD (encoder 1 phase B)
= k_rx_pc_2	k_pin_enc2_pha	PC.2 - TCLKA (encoder 2 phase A)
= k_rx_pa_3	k_pin_enc2_phb	PA.3 - TCLKB (encoder 2 phase B)
= k_rx_pc_0	k_pin_enc3_pha	PC.0 - TCLKC (encoder 3 phase A)
= k_rx_pb_3	k_pin_enc3_phb	PB.3 - TCLKD (encoder 3 phase B)
= k_rx_p2_3	k_pin_motor0_in2	P2.3 - GTIOC0A (motor 0 IN2/direction, pin 34)

= k_rx_p1_7	k_pin_motor0_in1	P1.7 - GTIOC0B (motor 0 IN1/PWM, pin 38)
= k_rx_p2_2	k_pin_motor1_in2	P2.2 - GTIOC1A (motor 1 IN2/direction, pin 35)
= k_rx_pc_3	k_pin_motor1_in1	PC.3 - GTIOC1B (motor 1 IN1/PWM, pin 67)
= k_rx_pe_3	k_pin_motor2_in2	PE.3 - GTIOC2A (motor 2 IN2/direction, pin 108)
= k_rx_p8_6	k_pin_motor2_in1	P8.6 - GTIOC2B (motor 2 IN1/PWM, pin 41)
= k_rx_pe_7	k_pin_motor3_in2	PE.7 - GTIOC3A (motor 3 IN2/direction, pin 101)
= k_rx_pc_6	k_pin_motor3_in1	PC.6 - GTIOC3B (motor 3 IN1/PWM, pin 61)
= k_rx_p2_1	k_pin_imu_scl	P2.1 - SCL1 (IMU I2C clock, pin 36)
= k_rx_p2_0	k_pin_imu_sda	P2.0 - SDA1 (IMU I2C data, pin 37)
= k_rx_p3_2	k_pin_imu_int	P3.2 - IMU interrupt (active-low, pin 27)
= k_rx_p8_3	k_pin_imu_rst	P8.3 - IMU reset (active-low, pin 58)
= k_rx_p6_1	k_pin_motor0_drvofff	P6.1 - Motor 0 DRVOFF (pin 115)
= k_rx_p6_3	k_pin_motor1_drvofff	P6.3 - Motor 1 DRVOFF (pin 113)
= k_rx_pe_0	k_pin_motor2_drvofff	PE.0 - Motor 2 DRVOFF (pin 111)
= k_rx_pe_2	k_pin_motor3_drvofff	PE.2 - Motor 3 DRVOFF (pin 109)
= k_rx_p6_0	k_pin_motor0_nsleee	P6.0 - Motor 0 nSLEEP (pin 117)
= k_rx_p6_2	k_pin_motor1_nsleee	P6.2 - Motor 1 nSLEEP (pin 114)
= k_rx_p6_4	k_pin_motor2_nsleee	P6.4 - Motor 2 nSLEEP (pin 112)
= k_rx_pe_1	k_pin_motor3_nsleee	PE.1 - Motor 3 nSLEEP (pin 110)
= k_rx_p4_4	k_pin_adc_an004	P4.4 - AN004 (motor 3 current)
= k_rx_p4_5	k_pin_adc_an005	P4.5 - AN005 (motor 2 current)
= k_rx_p4_6	k_pin_adc_an006	P4.6 - AN006 (motor 1 current)
= k_rx_p4_7	k_pin_adc_an007	P4.7 - AN007 (motor 0 current)

gpio_pin_counts_t

Static array-size and loop-bound constants for GPIO pin groups.

Provides compile-time upper bounds for all GPIO pin arrays. Used as loop limits in `internal_gpio_init_*`() helpers and as array-size declarators.

All values must fit in `uint8_t` (loop counter type)

Value	Name	Description
= 8	k_gptw_pin_count	4 motors x 2 pins (IN2 + IN1)
= 4	k_motor_count	4 DRV8263H motor drivers
= 4	k_sonar_count	4 HC-SR04 ultrasonic sensors

See also: `internal_gpio_init_gptw_pwm()` Uses `k_gptw_pin_count`,
`internal_gpio_init_motor_driver_ctrl()` Uses `k_motor_count`

Example:

```
constrx_port_pin_tpins[k_gptw_pin_count]={...};  
for(uint8_ti=0;i<k_gptw_pin_count;i++){  
  rx_mpc_set_gptw(pins[i]);  
}
```

Since Version 1.1.0

—

gptw_freq_t

GPTW PWM frequency constant.

Value	Name	Description
= 20000	k_gptw_pwm_freq_hz	20 kHz PWM frequency

—

gptw_deadtime_t

GPTW dead-time constant.

Value	Name	Description
= 1000	k_gptw_deadtime_ns	1 us dead-time between complementary outputs

—

i2c_channel_t

I2C channel assignments.

Value	Name	Description
= 0	k_i2c_channel_host	RIIC0 = host I2C (RPi5 comms)
= 1	k_i2c_channel_1	RIIC1 (reserved)

—

i2c_freq_t

I2C bus frequency constants.

Value	Name	Description
= 400000	k_i2c_host_freq_hz	400 kHz fast mode for host

—

adc_count_t

Number of motor current ADC channels.

Value	Name	Description
= 4	k_motor_adc_count	4 motors = 4 current sense channels

—

gpio_reg_constants_t

Constants for GPIO register bit manipulation.

Provides named constants for single-bit operations on 8-bit GPIO port registers (PMR, PDR, PODR). Eliminates magic number 1U in bit-shift expressions and documents the hardware-imposed pin count limit.

`k_gpio_single_bit_mask == 1` (exactly one bit for shift operations)

`k_gpio_max_pin_number == k_pins_per_port - 1`

Value	Name	Description
= 1U	<code>k_gpio_single_bit_mask</code>	Single-bit mask shifted by pin number for register RMW
= 7U	<code>k_gpio_max_pin_number</code>	Maximum valid pin index (0-7, 8 pins per port)

See also: `internal_gpio_set_output()` Uses these for register bit manipulation, `internal_gpio_set_input()` Uses these for register bit manipulation, `k_pins_per_port` From `rx_gpio_constants.h` (hardware 8-pin limit)

Example:

```
//Setpin5asoutputinPDRregister(read-modify-write)
port->PDR|=(uint8_t)(k_gpio_single_bit_mask<<k_bit_led);

//Validatepinnumberbeforeaccess
if(pin_number<=k_gpio_max_pin_number){
port->PODR|=(uint8_t)(k_gpio_single_bit_mask<<pin_number);
}
```

Since Version 1.1.0

—

s_sckcr3_reset_state

`const uint8_t s_sckcr3_reset_state`

SCKCR3 reset/unconfigured state value (before clock initialization).

—

internal_gpio_init_i2c

```
static rx_err_t internal_gpio_init_i2c(void)
```

Configure I2C bus pins (RIIC0/RIIC1).

Configures MPC pin multiplexing for Host I2C (RIIC0) on P1.2/P1.3 and RIIC host I2C on P2.0/P2.1. Sets PSEL registers to enable I2C peripheral function for SCL and SDA pins.

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	All pins configured successfully
<code>k_rx_err_hw_init_failed</code>	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: P1.2 configured as SCL0, P1.3 configured as SDA0

Post: P2.1 configured as SCL1, P2.0 configured as SDA1

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

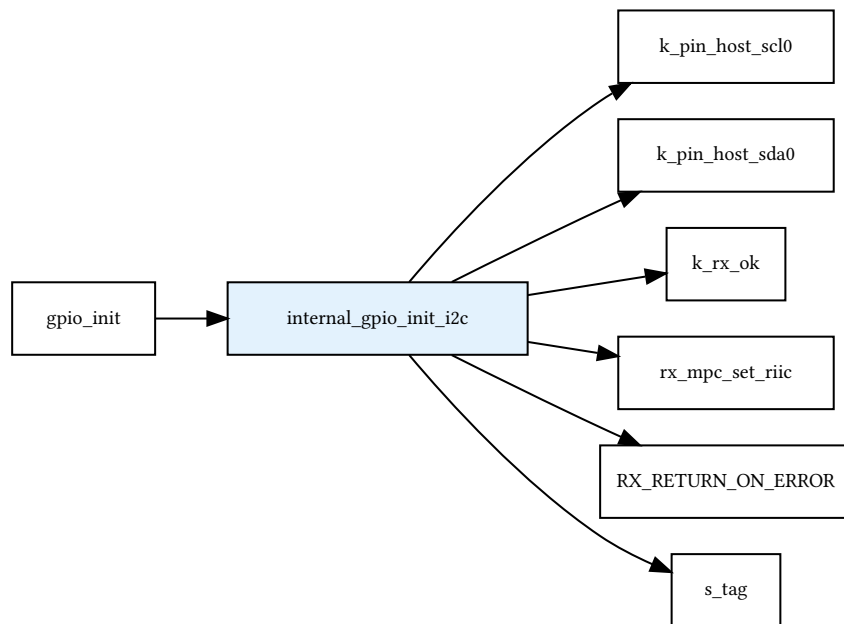


Figure 1294: Call graph for `internal_gpio_init_i2c`

_Defined in `src/hardware/_init.c:403_`

Since Version 1.0.0

—

internal_gpio_init_host_spi

```
static rx_err_t internal_gpio_init_host_spi(void)
```

Configure host SPI pins (RSPI2).

Configures MPC pin multiplexing for RSPI2 host SPI interface on PD.1-PD.4. Sets PSEL registers to enable RSPI2 peripheral function for COPI, CIPO, SCLK, and CS0 pins used for RPi5 communication.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: PD.1-PD.4 configured as RSPI2 COPI/CIPO/SCLK/CS0

Post: RSPI2 pins ready for host communication

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

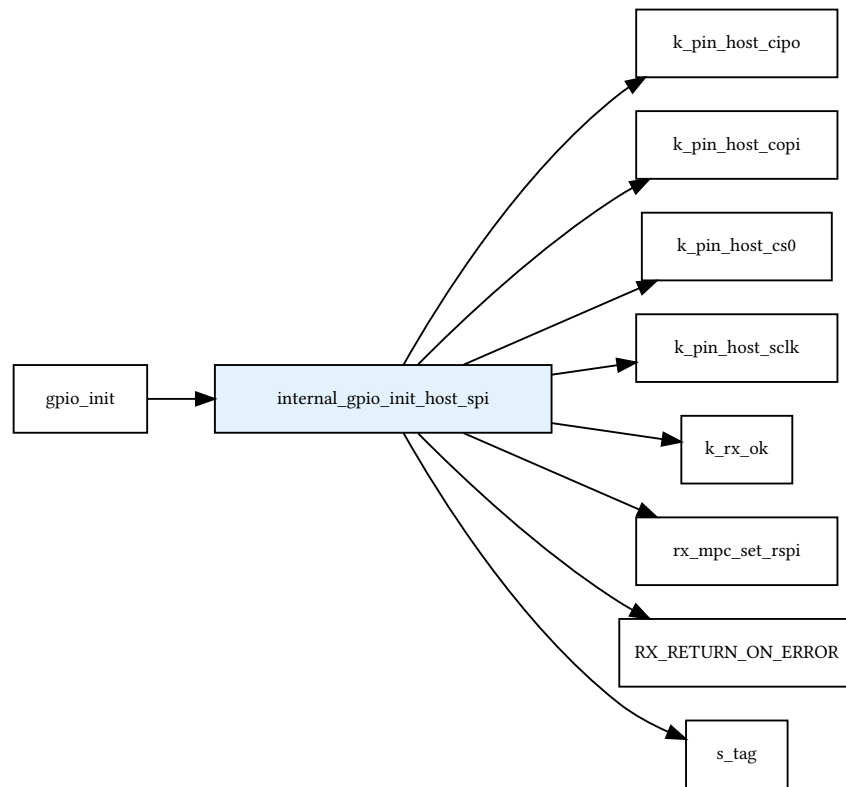


Figure 1295: Call graph for internal_gpio_init_host_spi

_Defined in src/hardware_init.c:439_

Since Version 1.0.0

—

internal_gpio_init_mtu_encoders

```
static rx_err_t internal_gpio_init_mtu_encoders(void)
```

Configure MTU encoder input pins (front wheels).

Configures MPC pin multiplexing for MTU external clock inputs MTCLKA-MTCLKD on P2.4/P2.5/PA.1/PC.5. Enables pulse counter mode for quadrature encoder inputs from front wheel motors.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: P2.4/P2.5 configured as MTCLKA/MTCLKB

Post: PA.1/PC.5 configured as MTCLKC/MTCLKD

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

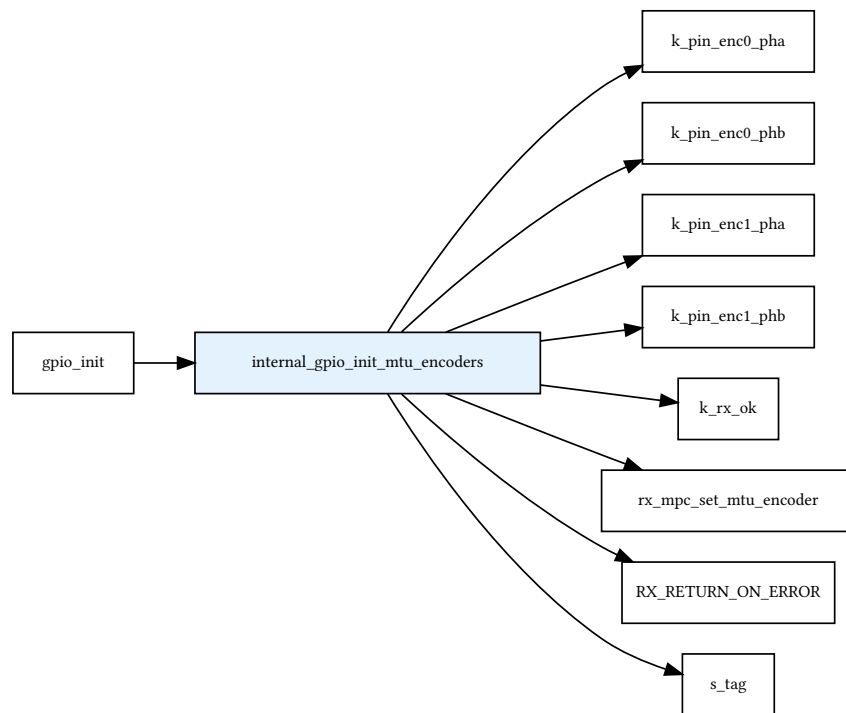


Figure 1296: Call graph for `internal_gpio_init_mtu_encoders`

_Defined in `src/hardware/_init.c:480_`

Since Version 1.0.0

internal_gpio_init_tpu_encoders

```
static rx_err_t internal_gpio_init_tpu_encoders(void)
```

Configure TPU encoder input pins (rear wheels).

Configures MPC pin multiplexing for TPU external clock inputs TCLKA-TCLKD on PC.2/PA.3/PC.0/PB.3. Enables pulse counter mode for quadrature encoder inputs from rear wheel motors.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: PC.2/PA.3 configured as TCLKA/TCLKB

Post: PC.0/PB.3 configured as TCLKC/TCLKD

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

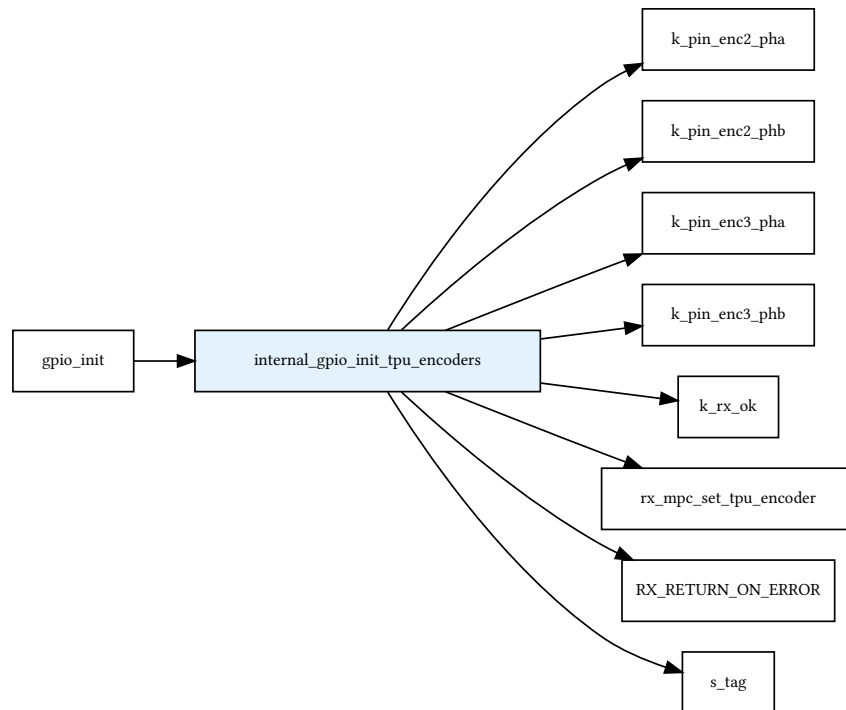


Figure 1297: Call graph for internal_gpio_init_tpu_encoders

_Defined in src/hardware/_init.c:521_

Since Version 1.0.0

—

internal_gpio_init_gptw_pwm

```
static rx_err_t internal_gpio_init_gptw_pwm(void)
```

Configure GPTW PWM output pins (4 motors).

Configures MPC pin multiplexing for GPTW0-GPTW3 PWM outputs distributed across PORT2, PORT1, PORT8, PORTC, and PORTE. Sets PSEL for all 8 pins (4 motors x IN2+IN1 per motor) for DRV8263H motor driver control.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

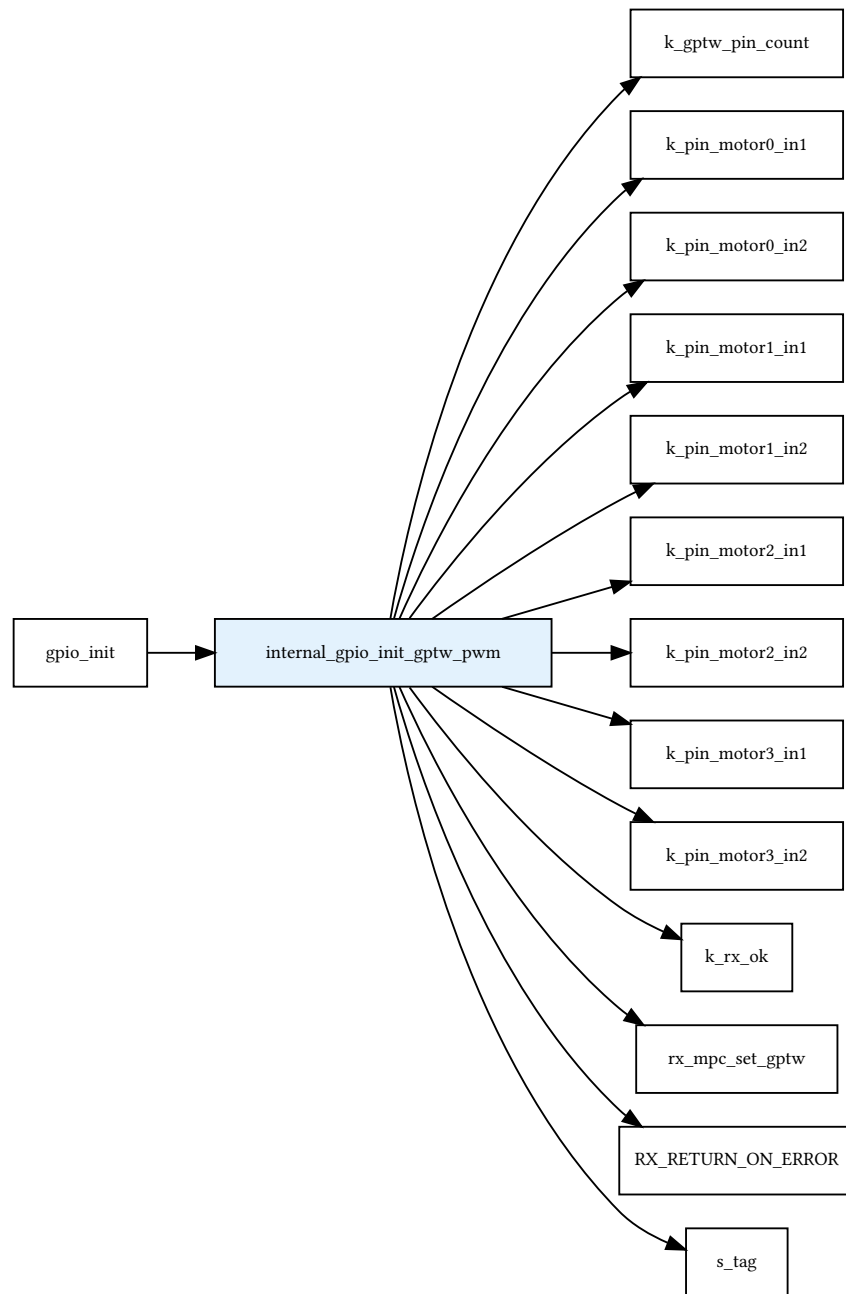
Pre: Pins not in use by other peripherals

Post: P23/P17, P22/PC3, PE3/P86, PE7/PC6 configured as GPTW outputs

Post: All 8 GPTW PWM pins ready for motor control

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

Figure 1298: Call graph for `internal_gpio_init_gptw_pwm`

_Defined in `src/hardware/_init.c:562_`

Since Version 1.0.0

—

internal_gpio_set_output

```
static void internal_gpio_set_output(rx_port_pin_t port_pin, bool initial_high)
```

Configure a single GPIO pin as output with initial level.

Sets the given pin to GPIO mode (PMR cleared), drives the specified initial logic level on PODR, and configures PDR as output. This is a common helper used by DRVOFF, nSLEEP, sonar trigger, and IMU reset pin initialization.

Register write order is critical for glitch-free startup:

1. Clear PMR (switch from peripheral to GPIO mode)
2. Set PODR (drive the desired initial logic level)
3. Set PDR (enable output driver last to avoid glitch)

Parameters:

Direction	Name	Description
in	port_pin	Encoded port/pin value from rx_mpc_pin_t (valid range: any value decodable by rx_port_from_pin() and rx_pin_from_pin() yielding pin 0-7)
in	initial_high	true to drive HIGH on startup, false for LOW

Returns: void (asserts on invalid input does not return on failure)

Pre: Pin must have MPC already configured via rx_mpc_set_gpio()

Pre: Port base address must be valid (rx_port_get_base() != nullptr)

Post: Pin PMR cleared (GPIO mode), PDR set (output), PODR at requested level

Post: Pin is actively driving the requested logic level

Note: Not thread-safe. Performs non-atomic RMW on 8-bit port registers.

Note: Called only during single-threaded initialization before RTOS start.

Warning: Do not call after RTOS start without external synchronization.] **See also:**
 internal_gpio_set_input() Complementary function for input pins,
 internal_gpio_init_motor_driver_ctrl() Primary caller for DRVOFF/nSLEEP,
 internal_gpio_init_sonar_triggers() Primary caller for sonar trigger pins

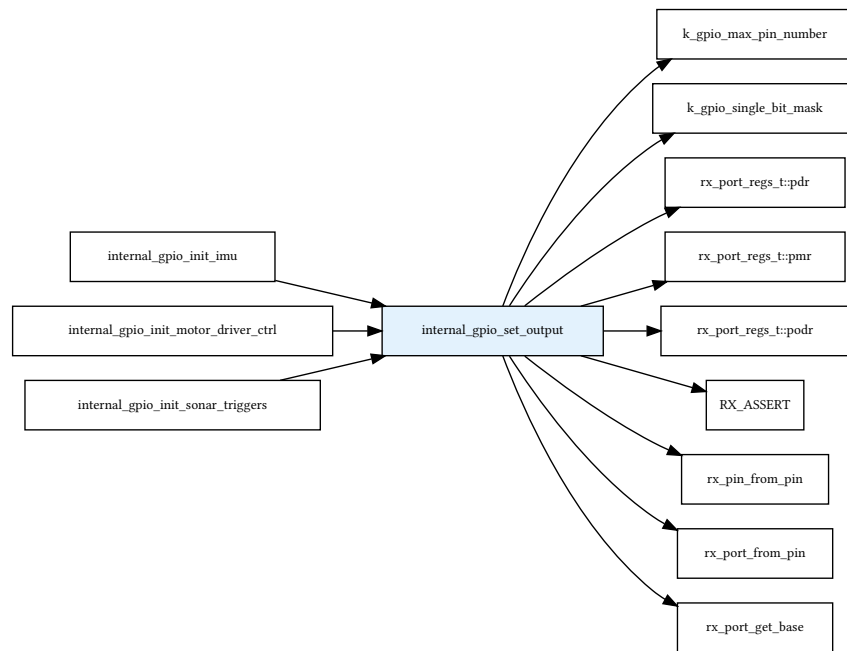


Figure 1299: Call graph for internal_gpio_set_output

_Defined in src/hardware/_init.c:620_

Since Version 1.1.0

—

internal_gpio_set_input

```
static void internal_gpio_set_input(rx_port_pin_t port_pin)
```

Configure a single GPIO pin as input.

Sets the given pin to GPIO mode (PMR cleared) and configures PDR as input (direction bit cleared). Used by IMU interrupt and sonar echo pin initialization.

Register write order:

1. Clear PMR (switch from peripheral to GPIO mode)
2. Clear PDR (configure as input direction)

Parameters:

Direction	Name	Description
in	port_pin	Encoded port/pin value from rx_mpc_pin_t (valid range: any value decodable by rx_port_from_pin() and rx_pin_from_pin() yielding pin 0-7)

Returns: void (asserts on invalid input does not return on failure)

Pre: Pin must have MPC already configured via `rx_mpc_set_gpio()`

Pre: Port base address must be valid (`rx_port_get_base() != nullptr`)

Post: Pin PMR cleared (GPIO mode) and PDR cleared (input direction)

Post: Pin is in high-impedance input state

Note: Not thread-safe. Performs non-atomic RMW on 8-bit port registers.

Note: Called only during single-threaded initialization before RTOS start.

Warning: Do not call after RTOS start without external synchronization.] **See also:**
`internal_gpio_set_output()` Complementary function for output pins,
`internal_gpio_init_imu()` Primary caller for IMU interrupt pin,
`internal_gpio_init_sonar_echoes()` Primary caller for sonar echo pins

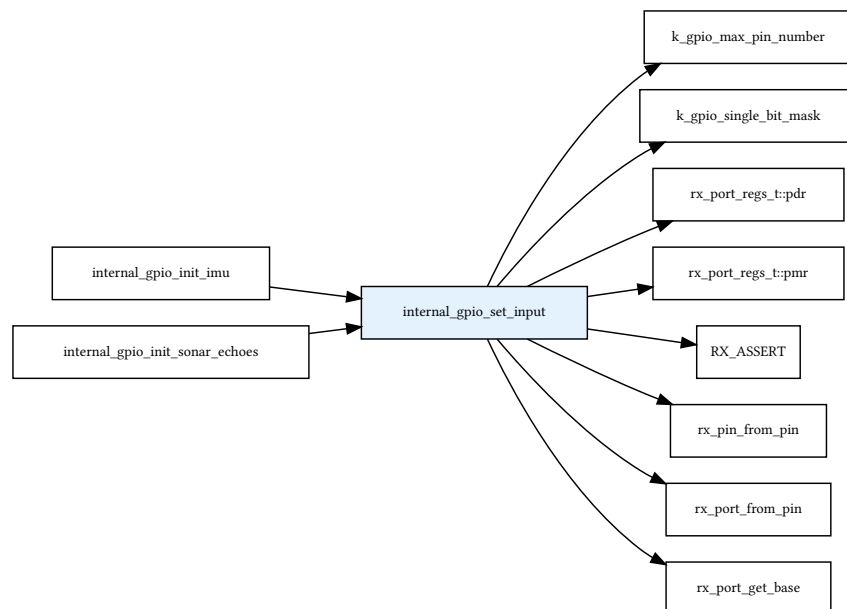


Figure 1300: Call graph for `internal_gpio_set_input`

_Defined in `src/hardware/_init.c:670_`

Since Version 1.1.0

—

internal_gpio_init_imu

```
static rx_err_t internal_gpio_init_imu(void)
```

Configure IMU pins (RIIC1 I2C + GPIO interrupt and reset).

Configures 4 pins for the inertial measurement unit:

- P2.1/SCL1 and P2.0/SDA1: RIIC1 I2C bus for IMU communication
- P3.2: IMU interrupt input (active-low from IMU INT pin)
- P8.3: IMU reset output (active-low, initialized HIGH = not in reset)

Pin configuration order:

1. I2C bus pins (SCL1, SDA1) via RIIC MPC
2. Interrupt input via GPIO MPC + input direction
3. Reset output via GPIO MPC + output HIGH (de-asserted)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All 4 pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: P2.1 configured as SCL1, P2.0 configured as SDA1 (RIIC1 peripheral mode)

Post: P3.2 configured as GPIO input, P8.3 configured as GPIO output HIGH

Note: Not thread-safe. Performs non-atomic RMW on port registers.

Note: Called only during single-threaded initialization before RTOS start.] **See also:**
internal_gpio_set_input() Used for IMU interrupt pin, internal_gpio_set_output()
Used for IMU reset pin, rx_mpc_set_riic() MPC configuration for I2C pins

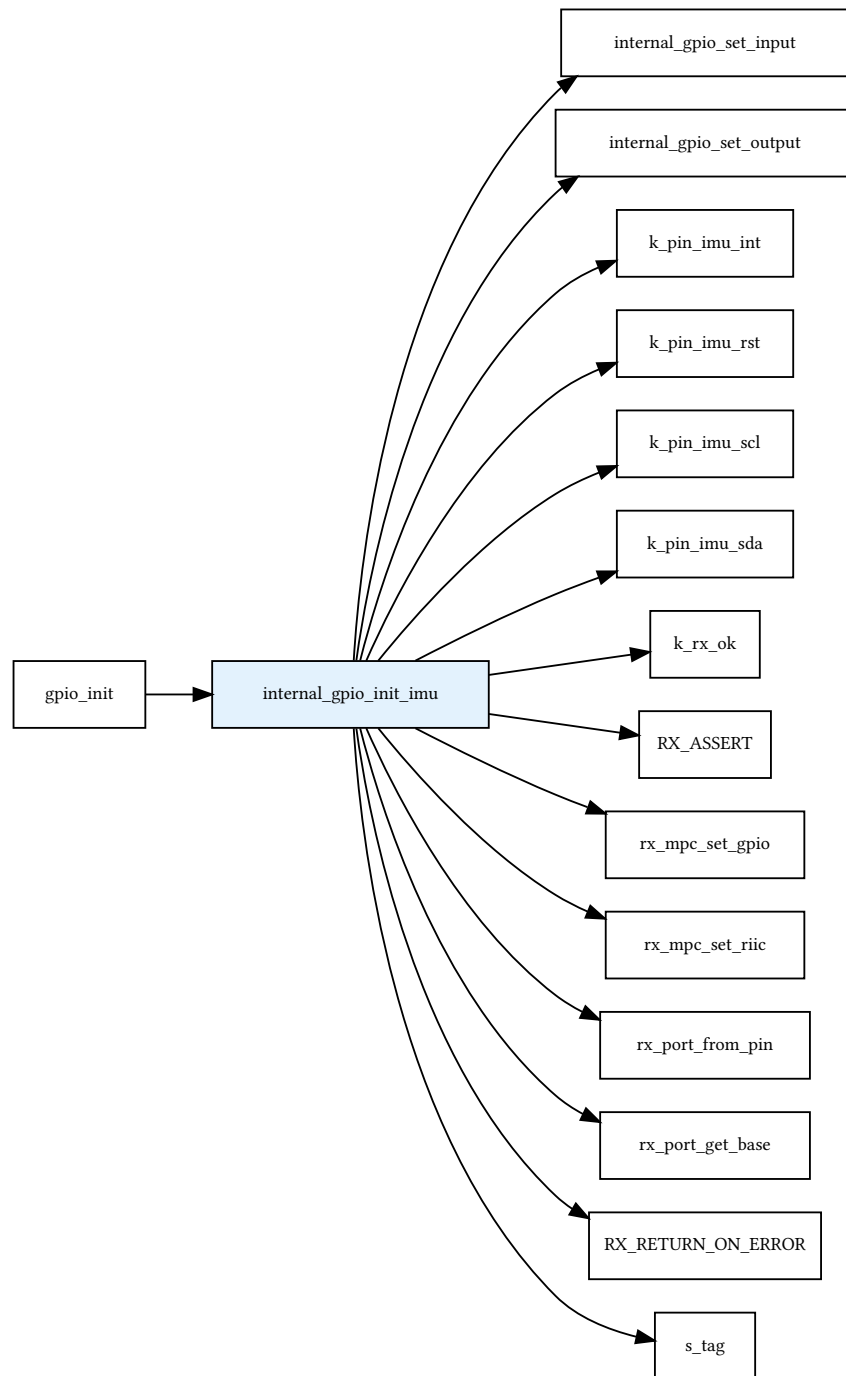


Figure 1301: Call graph for `internal_gpio_init_imu`
_Defined in `src/hardware/_init.c:713_`

Since Version 1.1.0

—

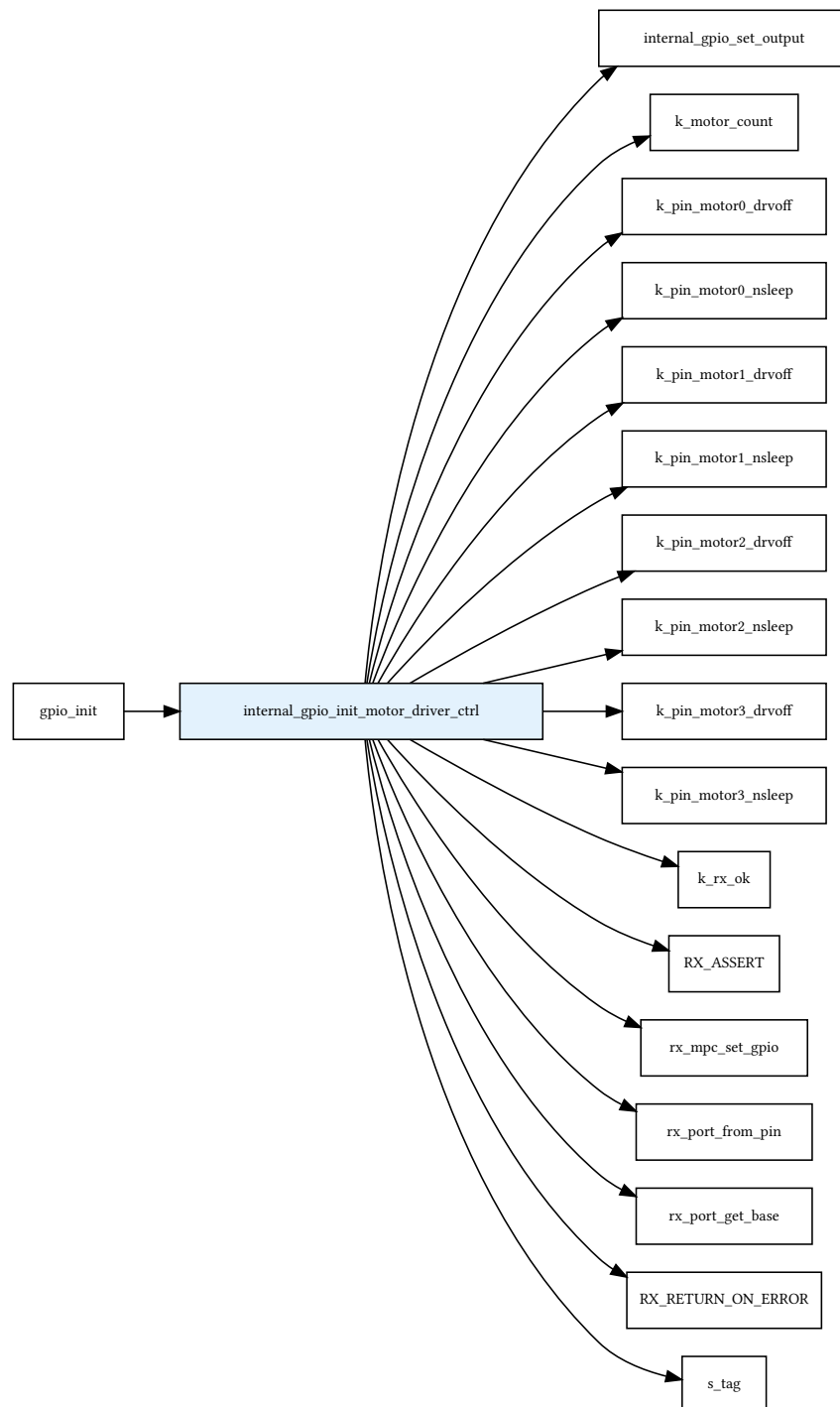
internal_gpio_init_motor_driver_ctrl

```
static rx_err_t internal_gpio_init_motor_driver_ctrl(void)
```

Configure DRV8263H motor driver control pins (DRVOFF + nSLEEP).

Configures 8 GPIO output pins for DRV8263H motor driver control:

- 4x DRVOFF pins: Output HIGH (driver outputs disabled for safe startup)
- 4x nSLEEP pins: Output HIGH (driver awake, not in sleep mode)

Figure 1302: Call graph for `internal_gpio_init_motor_driver_ctrl`

_Defined in src/hardware/_init.c:790_

—

internal_gpio_init_adc

```
static rx_err_t internal_gpio_init_adc(void)
```

Configure ADC current sense input pins.

Configures MPC pin multiplexing for S12AD0 analog inputs AN004-AN007 on P4.4-P4.7. Disables digital input buffers and sets pins to analog mode for motor current sensing.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: P4.4-P4.7 configured as AN004-AN007 analog inputs

Post: ADC pins ready for current sensing

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

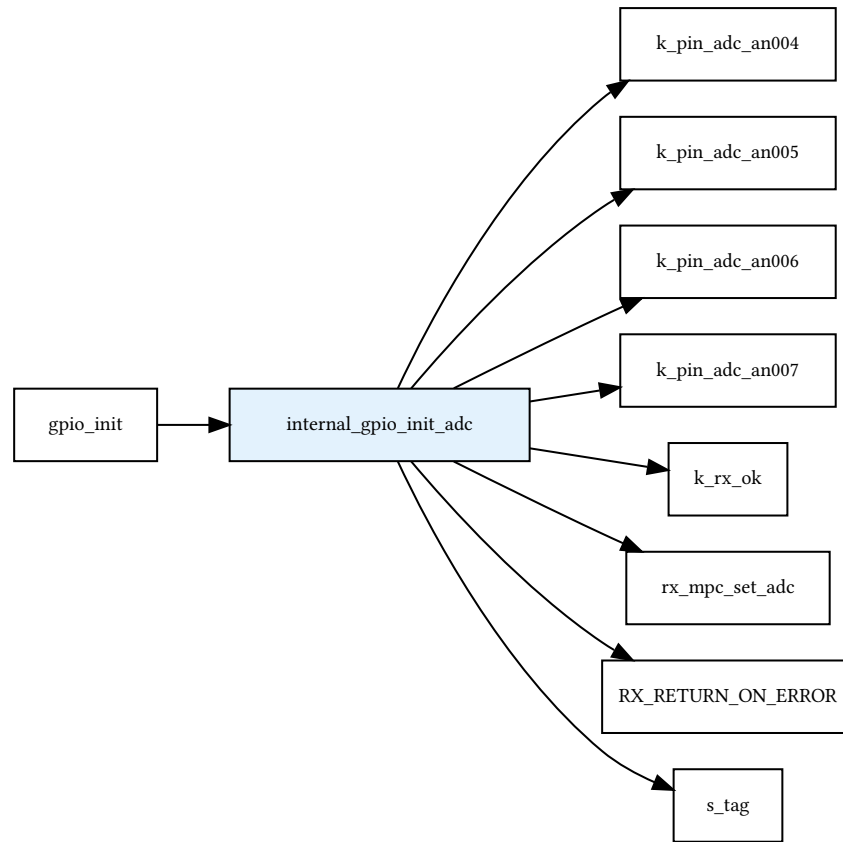


Figure 1303: Call graph for internal_gpio_init_adc

_Defined in src/hardware/_init.c:858_

Since Version 1.0.0

—

internal_gpio_init_usb

```
static rx_err_t internal_gpio_init_usb(void)
```

Configure USB VBUS detection pin.

Configures MPC pin multiplexing for USB0_VBUS detection on P1.6. Enables USB peripheral to detect host connection via VBUS voltage level.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: P1.6 configured as USB0_VBUS input

Post: USB VBUS detection enabled

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

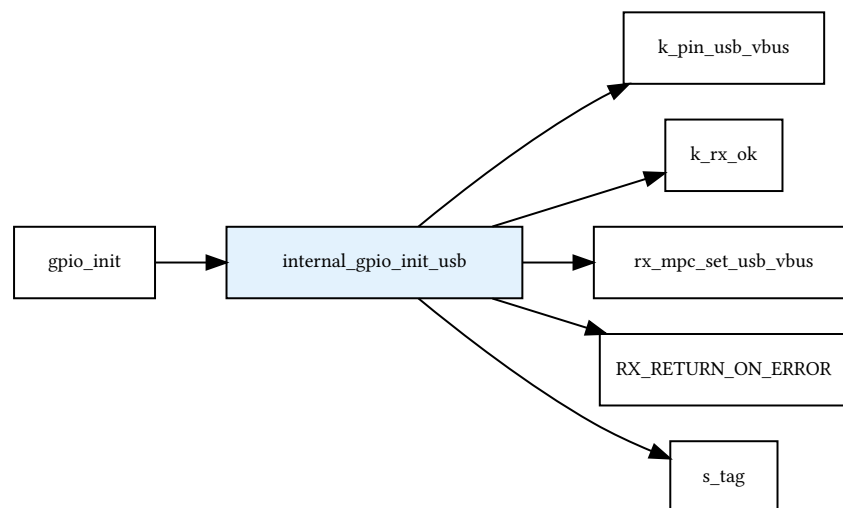


Figure 1304: Call graph for internal_gpio_init_usb

_Defined in src/hardware/_init.c:898_

Since Version 1.0.0

internal_gpio_init_sonar_triggers

```
static rx_err_t internal_gpio_init_sonar_triggers(void)
```

Configure HC-SR04 sonar trigger pins (GPIO outputs).

Configures 4 sonar trigger pins as GPIO outputs with initial LOW state. Pins: PF5, PJ5, PJ3, P33. Used to trigger ultrasonic pulse transmission on HC-SR04 distance sensors.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: All 4 sonar trigger pins configured as GPIO outputs

Post: Trigger pins initialized to LOW (idle state)

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.

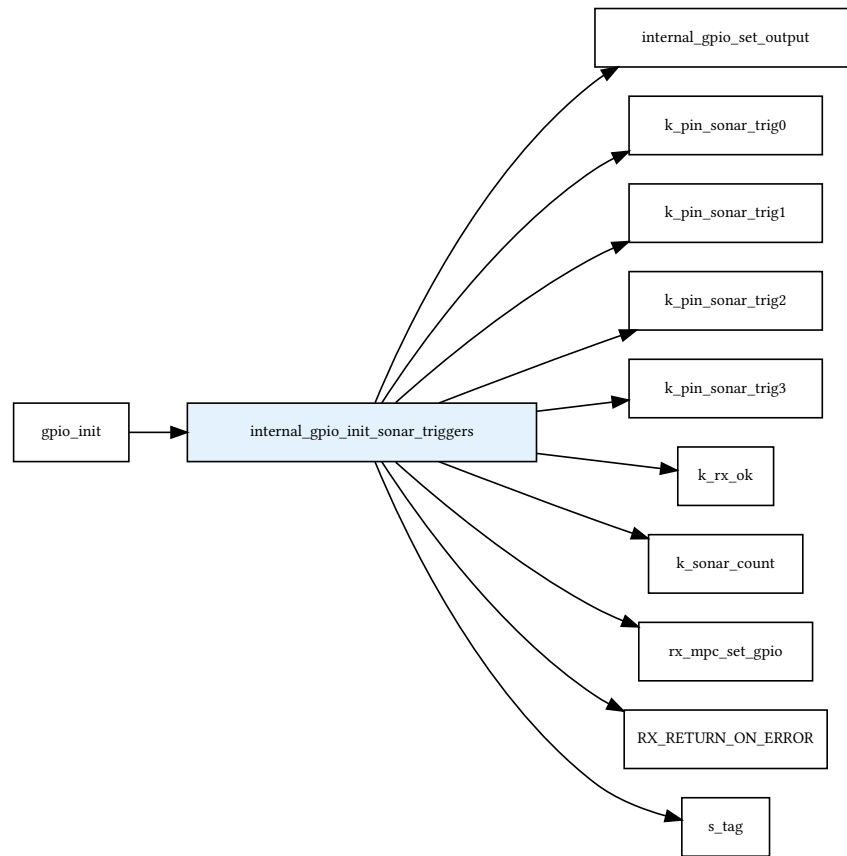


Figure 1305: Call graph for internal_gpio_init_sonar_triggers

_Defined in src/hardware_init.c:930_

Since Version 1.0.0

—

internal_gpio_init_sonar_echoes

```
static rx_err_t internal_gpio_init_sonar_echoes(void)
```

Configure HC-SR04 sonar echo pins (GPIO inputs).

Configures 4 sonar echo pins as GPIO inputs for pulse width measurement. Pins: P0.3, P0.2, P0.1, P0.0 (also IRQ11-IRQ8). Echo pulse duration measured by IRQ timing to determine distance.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All pins configured successfully
k_rx_err_hw_init_failed	MPC configuration failed for one or more pins

Pre: MPC write protection disabled (PWPR.B0WI=0, PWPR.PFSWE=1)

Pre: Pins not in use by other peripherals

Post: All 4 sonar echo pins configured as GPIO inputs

Post: Echo pins ready for IRQ-based pulse measurement

Note: Thread-safe. No shared state modified.

Note: Called only during system initialization.



Figure 1306: Call graph for `internal_gpio_init_sonar_echoes`

_Defined in `src/hardware/_init.c:970_`

Since Version 1.0.0

—

gpio_init

```
static rx_err_t gpio_init(void)
```

Initialize GPIO pins for motor control, I2C, and USB communication.

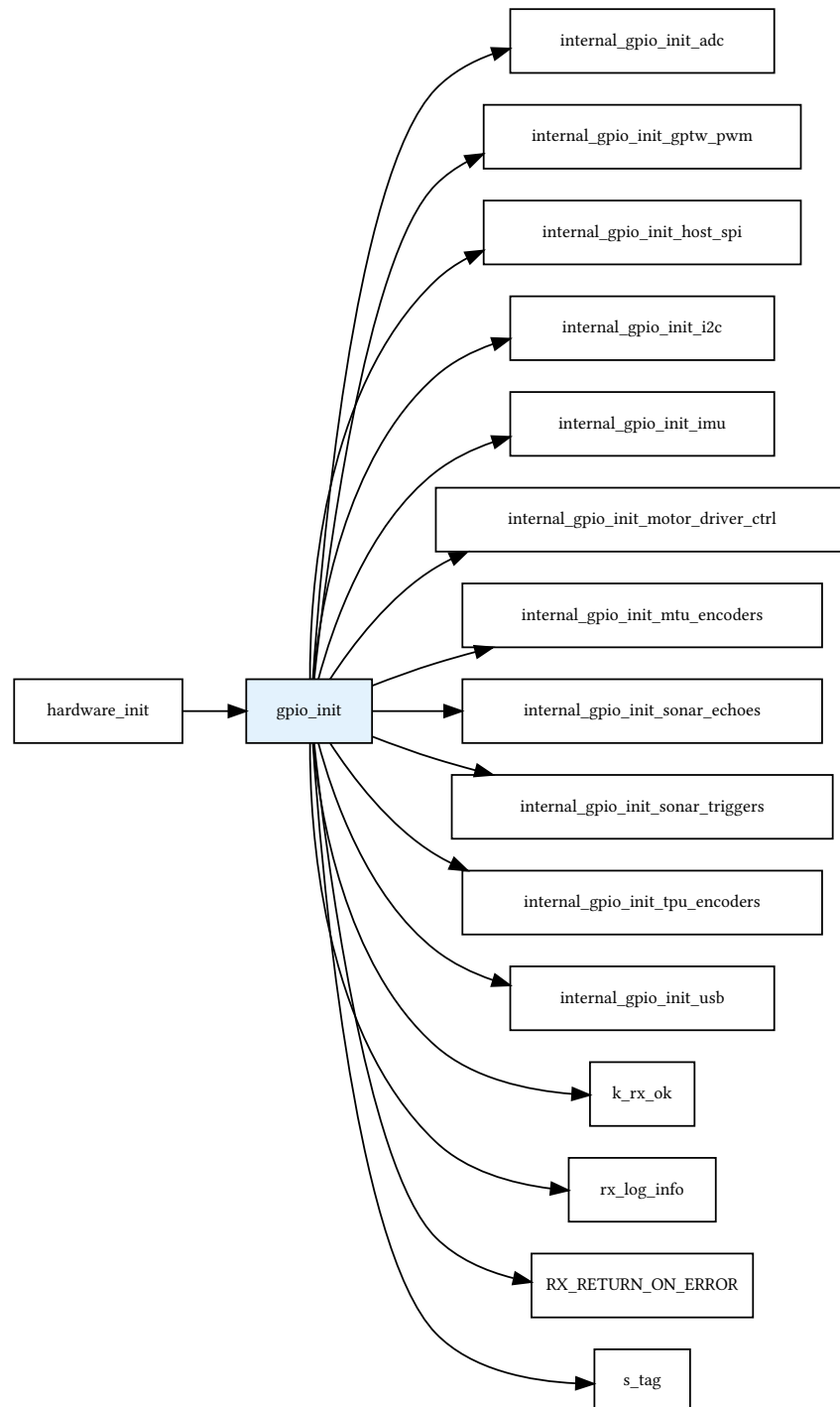
Configures Multi-Function Pin Controller (MPC) for 8 critical pins used in STAR robot application. All pins are configured for peripheral mode (not GPIO), with specific PSEL values determined by hardware function.

Pin configuration:

- 8x GPTW PWM pins - Motor control GPTW PWM outputs (PORT2/1/8/C/E)
- 2x I2C pins - Sensor communication bus (SCL/SDA)
- 1x USB pin - USB VBUS detection for CDC debug interface

Algorithm steps:

1. Validate all pin identifiers are within valid range
2. Configure GPTW PWM pins for 4-motor DRV8263H IN1/IN2 control
3. Configure I2C pins (PSEL = 0x0F) for sensor bus
4. Configure USB pin (PSEL = 0x11) for USB VBUS detect
5. All operations use rx_mpc API with write-protect handling

Figure 1307: Call graph for `gpio_init`

_Defined in src/hardware/_init.c:1106_

—

gptw_pwm_init

```
static rx_err_t gptw_pwm_init(void)
```

Initialize GPTW PWM for 4 motor channels with 90-degree phase staggering.

Configures GPTW channels 0-3 for complementary PWM output at 20 kHz with 1 us dead-time. Channels are phase-staggered by 90 degrees to reduce peak current draw and EMI.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All 4 GPTW channels initialized successfully
k_rx_err_hw_init_failed	GPTW staggered init failed

Pre: GPIO pins for GPTW configured via gpio_init() (PSEL = 0x1E)

Pre: PCLKA clock running at 120 MHz

Post: 4 GPTW channels configured for 20 kHz complementary PWM

Post: Phase staggering active (0, 90, 180, 270 degrees)

Note: Not thread-safe. Call during single-threaded initialization only.] **See also:** rx_gptw_init_all_staggered() HAL function for staggered PWM init

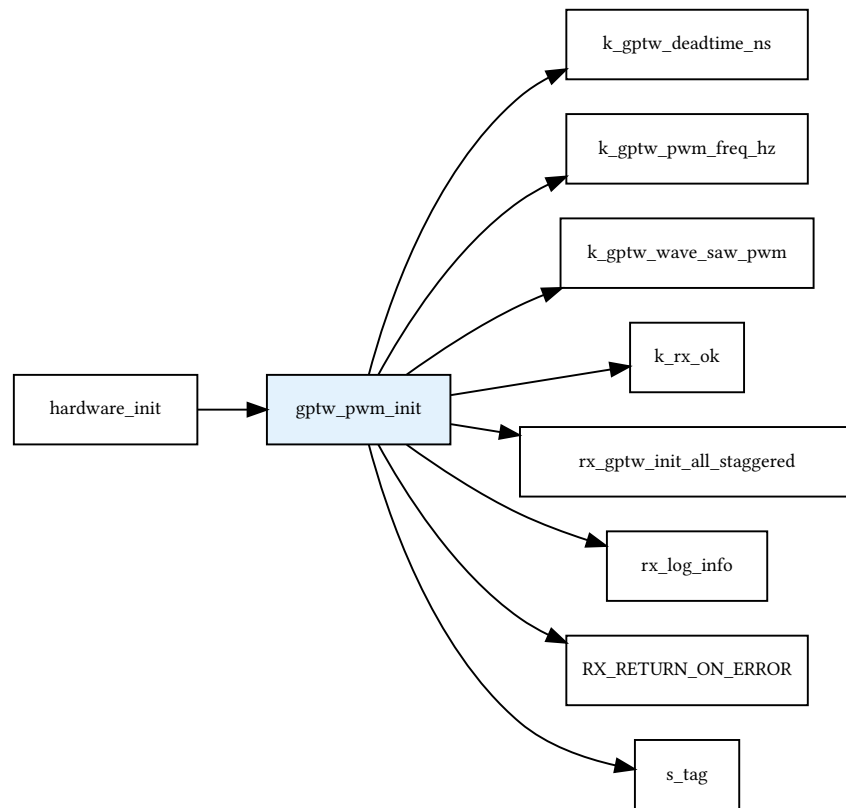


Figure 1308: Call graph for gptw_pwm_init

_Defined in src/hardware_init.c:1174_

Since Version 1.1.0

—

spi_init

```
static rx_err_t spi_init(void)
```

Initialize RSPI2 as SPI peripheral for RPi5 host communication.

Configures RSPI2 in peripheral mode for receiving commands from the Raspberry Pi 5 host controller. Uses SPI mode 0 (CPOL=0, CPHA=0) with 8-bit transfers.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	RSPI2 peripheral initialized successfully
k_rx_err_hw_init_failed	RSPI2 init failed

Pre: GPIO pins for RSPi2 configured via `gpio_init()`

Pre: PCLKA clock running

Post: RSPi2 ready to receive SPI transactions from RPi5

Post: SPI mode 0, 8-bit transfers configured

Note: Not thread-safe. Call during single-threaded initialization only.] **See also:** `rspi_init_peripheral()` HAL function for RSPi peripheral mode

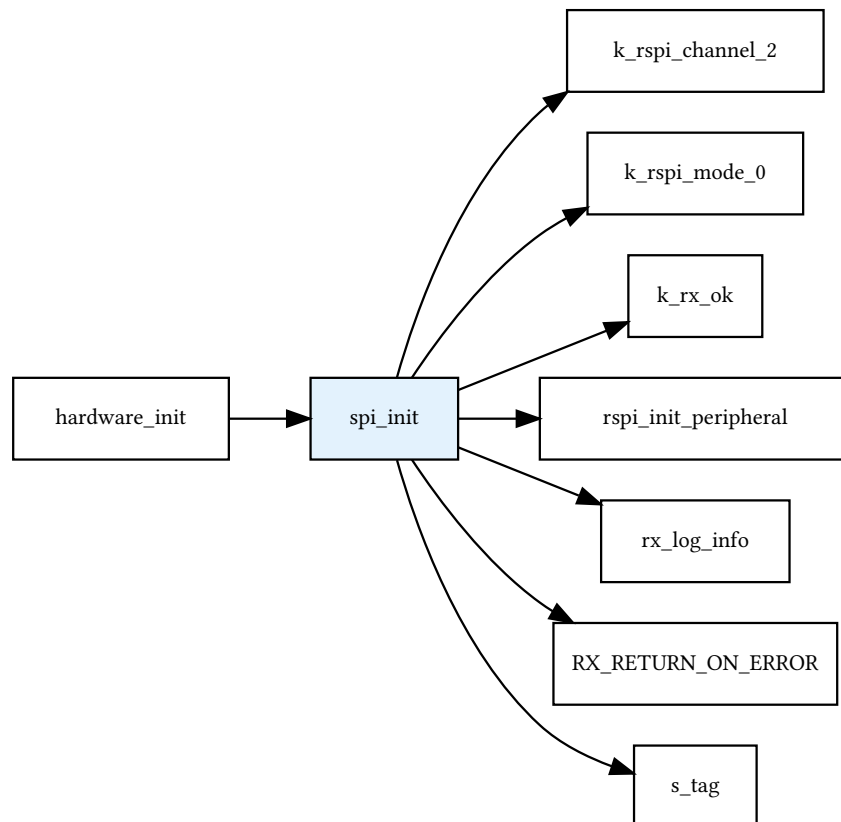


Figure 1309: Call graph for `spi_init`

_Defined in `src/hardware/_init.c:1215_`

Since Version 1.1.0

—

i2c_init

```
static rx_err_t i2c_init(void)
```

Initialize I2C buses for host communication.

Configures two RIIC channels:

- RIIC0 at 400 kHz (fast mode) for RPi5 host I2C

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Both RIIC channels initialized successfully
k_rx_err_hw_init_failed	RIIC init failed

Pre: GPIO pins for RIIC0/RIIC1 configured via gpio_init()

Pre: PCLKB clock running at 60 MHz

Post: RIIC0 ready for host I2C at 400 kHz

Note: Not thread-safe. Call during single-threaded initialization only.] **See also:** riic_init() HAL function for RIIC channel init

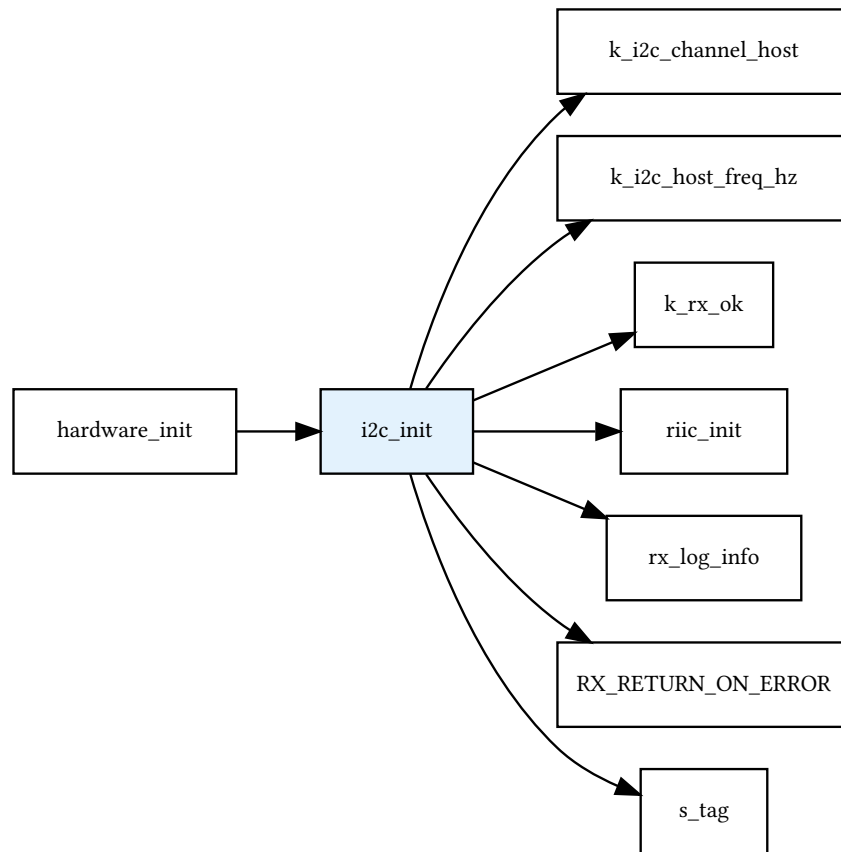


Figure 1310: Call graph for i2c_init

_Defined in src/hardware/_init.c:1250_

Since Version 1.1.0

—

adc_init_channels

```
static rx_err_t adc_init_channels(void)
```

Initialize ADC channels for motor current sensing.

Configures S12AD0 channels AN004-AN007 at 12-bit resolution for reading motor current via analog sense inputs.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All 4 ADC channels initialized
k_rx_err_hw_init_failed	ADC channel init failed

Pre: GPIO pins for ADC configured via `gpio_init()`

Pre: PCLKB/PCLKD clock running

Post: S12AD0 channels AN004-AN007 ready for conversion

Post: 12-bit resolution configured

Note: Not thread-safe. Call during single-threaded initialization only.] **See also:** `adc_init()` HAL function for ADC channel init

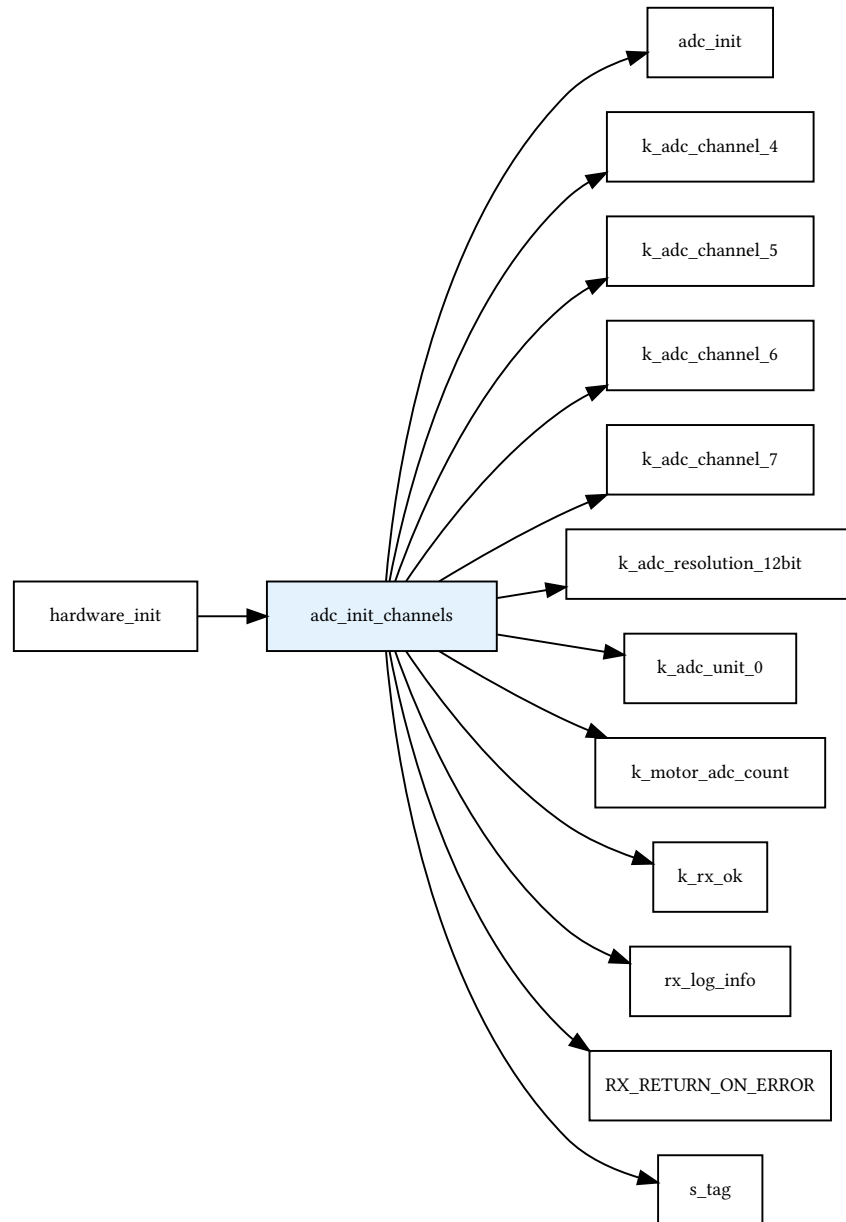


Figure 1311: Call graph for `adc_init_channels`

`_Defined in src/hardware/_init.c:1292_`

Since Version 1.1.0

—

validate_peripherals

```
static void validate_peripherals(void)
```

Non-fatal peripheral validation after initialization.

Performs a test ADC conversion to verify the ADC subsystem is operational. Logs warnings on failure but never halts - all checks are informational.

Pre: All peripheral init functions have completed

Post: Validation results logged

Post: No state changes (read-only checks)

Note: Not thread-safe. Call during single-threaded initialization only.

Note: Non-fatal: failures are logged as warnings, boot continues.

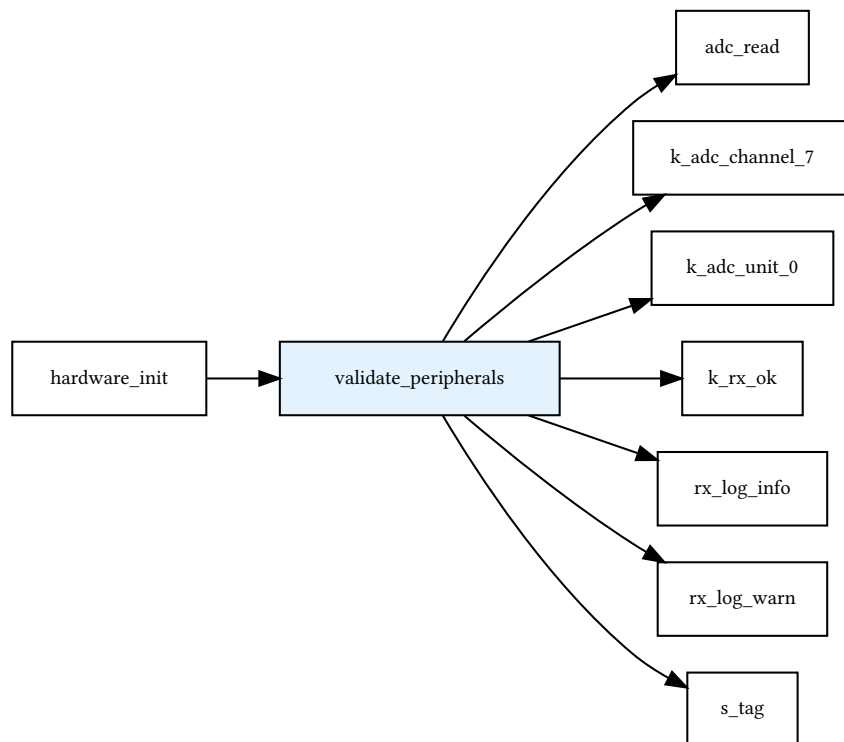


Figure 1312: Call graph for `validate_peripherals`

_Defined in `src/hardware/_init.c:1329_`

Since Version 1.1.0

—

hardware_init

```
rx_err_t hardware_init(void)
```

Initialize all application-specific hardware peripherals for STAR motor controller.

Initialize all application hardware peripherals.

Configures seven categories of peripherals required by the STAR robot application:

1. GPIO ([PASS] implemented) - Motor control pins, LEDs, sensor chip selects
2. Timers ([PASS] implemented) - CMT0 for ThreadX tick at 1 kHz
3. UART ([PASS] implemented) - SCI9 for debug console at 115200 baud
4. SPI ([PASS] implemented) - Host SPI peripheral (RSPI2), sensor bus
5. I2C (planned) - IMU, temperature, pressure sensors
6. ADC (planned) - Current sensing

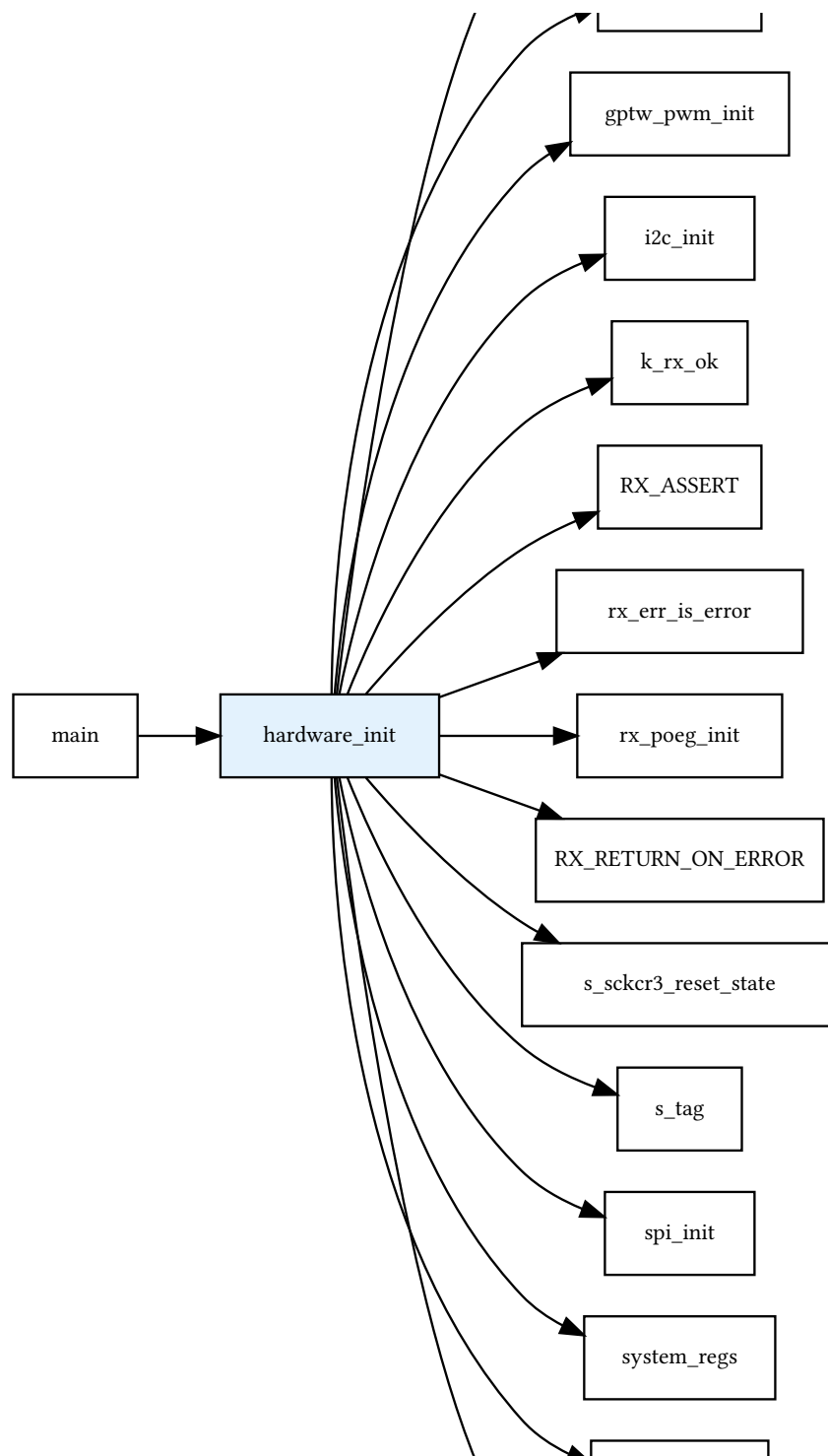


Figure 1313: Call graph for hardware_init

_Defined in src/hardware/_init.c:1538_

—

hardware_config.h

Hardware Pin Configuration Constants for STAR RX72N Platform (144-pin LQFP).

This file defines all hardware pin assignments for the STAR robot platform using the 144-pin LQFP package (R5F572NNHxFB). Pin assignments are verified against pinout.txt (repository root, 57/144 pins used).

CRITICAL: These pin assignments MUST match the PCB design. Any changes to this file require corresponding updates to pinout.txt and vice versa.

Includes:

- <stdint.h>

hardware_init.h

Application-Specific Hardware Initialization.

Declares the hardware initialization function that configures all application peripherals after system clock setup.

Includes:

- "rx_err.h"

hardware_init

```
rx_err_t hardware_init(void)
```

Initialize all application hardware peripherals.

Initializes GPIO, timers, UART, SPI, I2C, ADC in the correct order. Must be called after rx_clock_power_init() and before tx_kernel_enter().

Initialize all application hardware peripherals.

Configures seven categories of peripherals required by the STAR robot application:

1. GPIO ([PASS] implemented) - Motor control pins, LEDs, sensor chip selects
2. Timers ([PASS] implemented) - CMT0 for ThreadX tick at 1 kHz
3. UART ([PASS] implemented) - SCI9 for debug console at 115200 baud
4. SPI ([PASS] implemented) - Host SPI peripheral (RSPI2), sensor bus
5. I2C (planned) - IMU, temperature, pressure sensors
6. ADC (planned) - Current sensing

Returns: rx_err_t Error code

Value	Description
k_rx_ok	All peripherals initialized successfully
k_rx_err_*	Peripheral initialization failed

Pre: System clocks configured (rx_clock_power_init called)

Post: All peripherals ready for use] **See also:** `rx_clock_power_init()` Must be called first,
`main()` Calls this function during boot

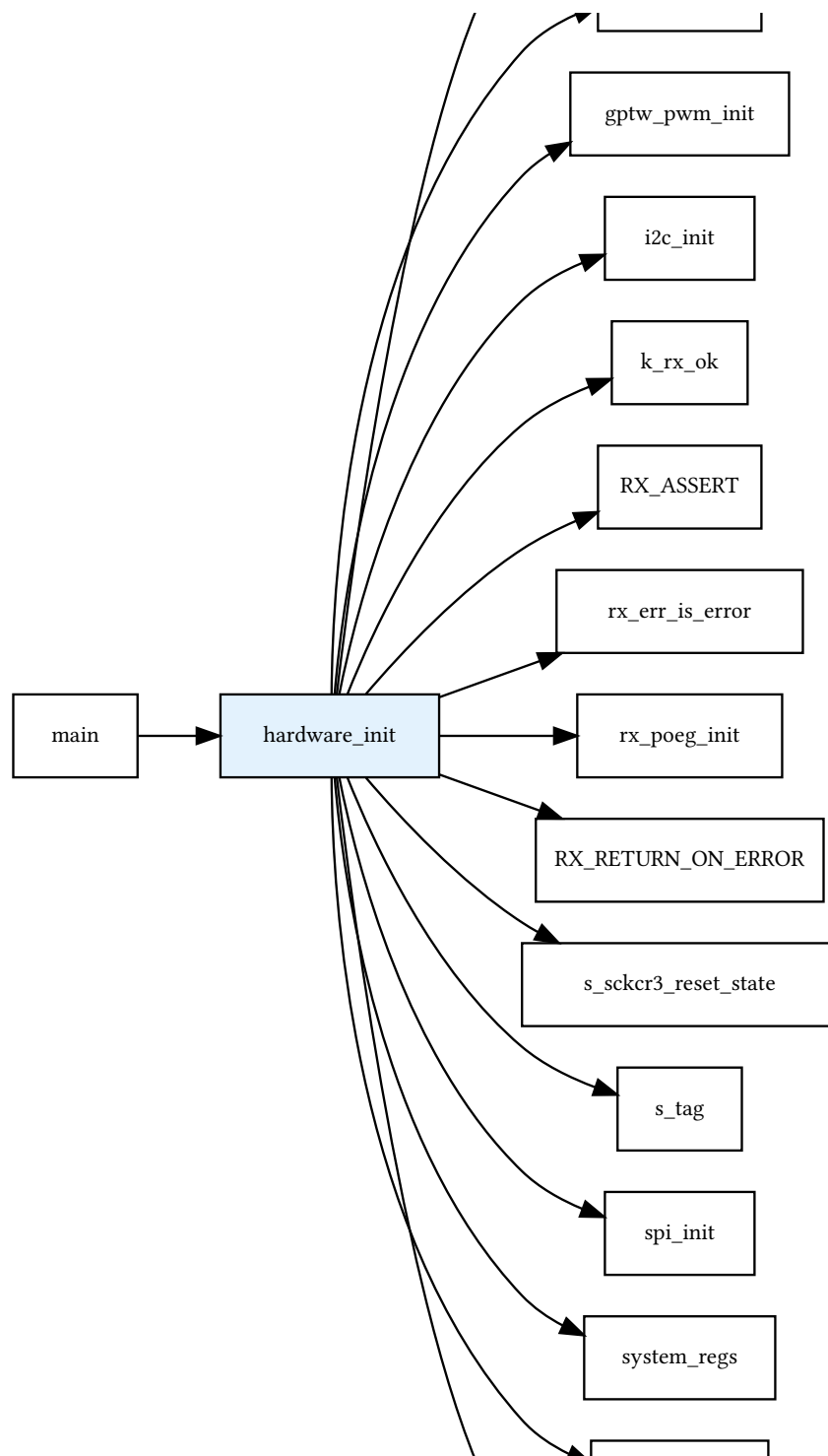


Figure 1314: Call graph for hardware_init

_Defined in src/inc/hardware_init.h:33_

—

rx_clock_power_init.h

RX72N System Clock and Power Management Initialization.

Declares the clock and power initialization function that configures the RX72N for 240 MHz operation with PLL.

Includes:

- "rx_err.h"

rx_clock_power_init

```
rx_err_t rx_clock_power_init(void)
```

Initialize system clocks and power management.

Configures the RX72N clock tree for maximum performance:

- ICLK (CPU): 240 MHz
- PCLKA (Timers): 120 MHz
- PCLKB (UART/SPI): 60 MHz
- FCLK (Flash): 60 MHz

Must be called early in main() before any peripheral initialization.

Initialize system clocks and power management.

Configures the complete clock tree and module power control for the RX72N MCU. This is the FIRST initialization function called from main() after reset, before any peripheral setup or RTOS startup.

Call sequence: Reset -> main() -> rx_clock_power_init() -> hardware_init() -> tx_kernel_enter()

Returns: rx_err_t Error code

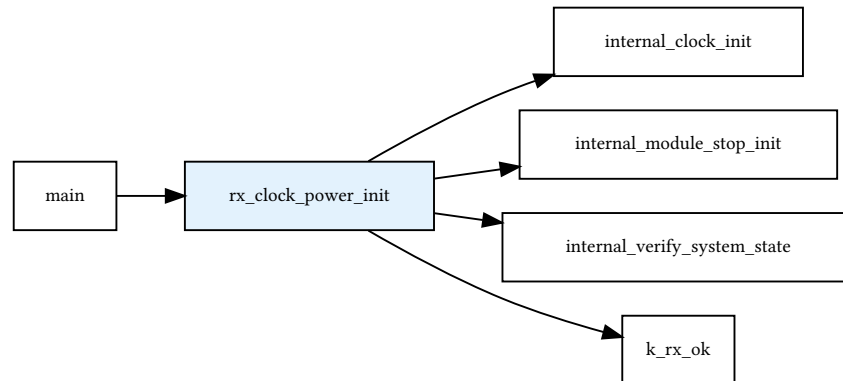
Value	Description
k_rx_ok	Clock configuration successful
k_rx_err_timeout	PLL failed to lock

Pre: CPU reset completed

Pre: C runtime initialized

Post: System running at 240 MHz

Post: All clock domains configured] **See also:** main() Calls this function during boot, hardware_init() Called after this function

Figure 1315: Call graph for `rx_clock_power_init`

_Defined in `src/inc/rx\_clock\_power\_init.h:40_`

rx_stack_monitor.h

ThreadX Stack Overflow Detection and High-Water Mark Monitoring.

Provides the application-level ThreadX stack overflow handler and related stack monitoring utilities for safety-critical RX72N firmware.

Includes:

- "rx_err.h"
- "tx_api.h"

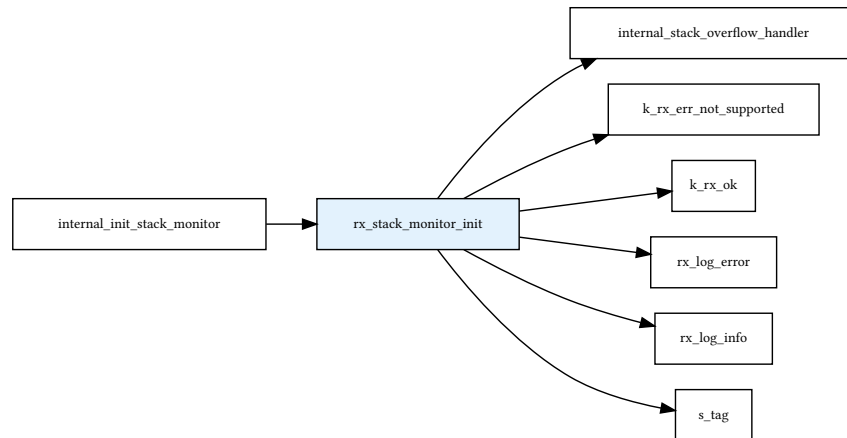
rx_stack_monitor_init

```
rx_err_t rx_stack_monitor_init(void)
```

Register the stack overflow handler with ThreadX.

Calls `tx_thread_stack_error_notify()` to register `internal_stack_overflow_handler()` as the application callback invoked by ThreadX whenever a corrupted stack sentinel is detected at a context switch.

Must be called from `tx_application_define()`, after all threads are created and before the scheduler starts running those threads.

Figure 1316: Call graph for `rx_stack_monitor_init`

Defined in `src/inc/rx_stack_monitor.h:136`

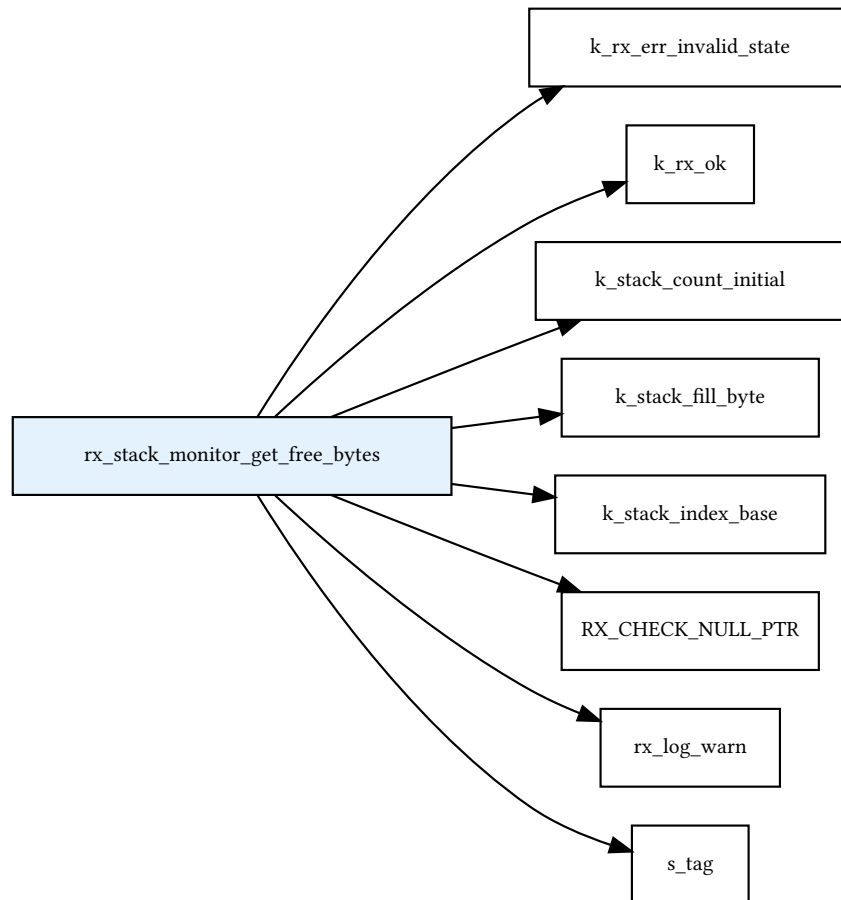
`rx_stack_monitor_get_free_bytes`

```
rx_err_t rx_stack_monitor_get_free_bytes(const TX_THREAD *thread_ptr, uint32_t
*free_bytes)
```

Return the number of unused (free) bytes in a thread's stack.

Scans the thread's stack memory for the 0xEF fill pattern placed by ThreadX stack filling (enabled when `TX_DISABLE_STACK_FILLING` is NOT defined). The count of consecutive 0xEF bytes from the stack base toward the stack pointer is returned as the free-byte estimate.

This value represents the minimum remaining headroom since the scan starts from the low-address end of the stack (the direction into which the RX stack grows).

Figure 1317: Call graph for `rx_stack_monitor_get_free_bytes`

Defined in `src/inc/rx_stack_monitor.h:201`

—

main.c

STAR RX72N Firmware Entry Point - System Initialization and ThreadX Bootstrap.

Includes:

- "hardware.h"
- "hardware_init.h"
- "rx72n_system_regs.h"
- "rx_bus_config.h"
- "rx_bus_manager.h"
- "rx_bus_types.h"
- "rx_check.h"
- "rx_clock_power_init.h"
- "rx_err.h"

- "rx_infrastructure.h"
- "rx_port_utils.h"
- "tx_api.h"
- "comm_task.h"
- "led_status_task.h"
- "motor_control_task.h"
- "obstacle_detect_task.h"
- "shared_data.h"
- "telemetry_task.h"
- "temp_sensor_task.h"
- "watchdog_monitor_task.h"
- "rx_iwdt.h"
- "rx_stack_monitor.h"

main_ret_t

Main function return codes.

Note: main() should never return in this firmware as ThreadX takes over. These codes exist for completeness and static analysis tools.

Value	Name	Description
= 0	k_main_ret_success	Successful completion (should never be reached)

—

riic_channel_num_t

RIIC (I2C) channel number constants for RX72N.

RX72N has 3 RIIC channels (0-2). These constants are used with uint8_t parameters in bus configuration functions.

Value	Name	Description
= 0	k_riic_channel_0	RIIC channel 0
= 1	k_riic_channel_1	RIIC channel 1
= 2	k_riic_channel_2	RIIC channel 2

Note: hardware.h defines riic_channel_t as a struct wrapper, but bus configuration functions use raw uint8_t for channel numbers.] **See also:** rx_bus_config_init_i2c() Uses uint8_t channel parameter

—

i2c_frequency_t

Standard I2C bus frequencies for RIIC configuration.

Standard I2C frequency constants for bus configuration. Values in Hz for clarity and type safety.

Value	Name	Description
= 100000	k_i2c_frequency_100khz	Standard mode: 100 kHz (default)

= 400000	k_i2c_frequency_400khz	Fast mode: 400 kHz
----------	------------------------	--------------------

See also: `rx_bus_config_init_i2c()` Uses `frequency_hz` parameter

—

iwdt_hardware_timeout_ms_t

IWDT hardware watchdog timeout constants.

Defines the hardware Independent Watchdog Timer (IWDT) timeout period. This is the maximum time the system can run without feeding the watchdog before triggering an automatic hardware reset. The timeout is configured via the IWDT Clock Select (CKS) register setting.

Hardware Configuration:

- CKS = 10 (PCLKB/8192 divider)
- PCLKB = 60 MHz
- Timeout period = 2048ms (+/-10% tolerance)

Usage: Set as `default_timeout_ms` in `rx_iwdt_config_t` during initialization. Watchdog monitor task feeds the hardware watchdog at 100 Hz (10ms period).

All values in milliseconds. Valid range: 128-16384ms per hardware limits. Hardware timeout must exceed longest task timeout to prevent spurious resets.

Value	Name	Description
= 2048	k_iwdt_hw_timeout_ms	Hardware IWDT timeout in milliseconds (CKS=10, 2048ms +/-10%). Must exceed all task timeouts to prevent spurious resets. Valid range: 128-16384ms per hardware limits.

Note: Hardware timeout must be longer than longest task timeout to prevent false resets] **See also:** `rx_iwdt_config_t` Configuration structure using this constant, `watchdog_monitor_task.c` Task that feeds the hardware watchdog

Example:

```
//ConfigureIWDTwithhardwaretimeout
staticconst rx_iwdt_config_t s_iwdt_config={
    .default_timeout_ms=k_iwdt_hw_timeout_ms,//2048mshardwaretimeout
    .enable_task_monitoring=true,
    .reset_on_timeout=true,
};
rx_err_t err=rx_iwdt_init(&s_iwdt_config);
```

Since Version 1.0.0

—

iwdt_state_timeout_ms_t

IWDT state-dependent timeout constants.

Defines software-level watchdog timeouts for each system state. These timeouts determine how long the system can remain in a given state without task heartbeats before the watchdog monitor triggers a reset. State-specific timeouts allow longer grace periods during initialization and error recovery while maintaining tight timing during normal operation.

State Timeout Strategy:

- Init/Idle: 5s - Accommodates slow peripheral initialization (I2C, USB, flash)
- Running/Motor/Comm: 2s - Standard operation with 100 Hz watchdog feeding
- Error: 10s - Extended timeout for diagnostics, logging, and recovery attempts

Timeout Hierarchy: State timeout > Task timeout > Heartbeat interval (prevents false positives)

All values in milliseconds. State timeouts < hardware IWDG timeout (2048ms for running states).

Init/idle/error states may exceed hardware timeout (watchdog not yet started or in recovery). Valid ranges per state: init/idle (2000-10000ms), running/motor/comm (1000-2000ms), error (5000-16000ms).

Value	Name	Description
= 5000	k_iwdt_timeout_init_ms	Init state timeout (5000ms). Allows slow startup: clock init, peripheral init, task creation. Exceeds hardware timeout (watchdog not yet started). Valid range: 2000-10000ms.
= 5000	k_iwdt_timeout_idle_ms	Idle state timeout (5000ms). System initialized but no critical operations running. Same as init for consistency. Valid range: 2000-10000ms.
= 2000	k_iwdt_timeout_running_ms	Running state timeout (2000ms). Default operational state. Matches hardware IWDG timeout. Must be fed at 100 Hz (10ms) to prevent reset. Valid range: 1000-2000ms.
= 2000	k_iwdt_timeout_motor_active_ms	Motor active state timeout (2000ms). Motors running with closed-loop control at 100 Hz. Same as running state (motor control is time-critical). Valid range: 1000-2000ms.
= 2000	k_iwdt_timeout_comm_active_ms	Communication active state timeout (2000ms). SPI communication in progress. Same as running (comm task runs at 100 Hz). Valid range: 1000-2000ms.
= 10000	k_iwdt_timeout_error_ms	Error state timeout (10000ms). Extended timeout for error logging, diagnostics, and recovery attempts before reset. Allows thorough post-mortem. Valid range: 5000-16000ms.

Note: All timeouts are in milliseconds (ms)

Note: State timeouts must be less than hardware IWDG timeout (2048ms for running states)] **See also:** `system_state_t` System state enumeration, `rx_iwdt_config_t.state_timeouts_ms` Array indexed by `system_state_t`, `rx_iwdt_set_state()` Function to transition between states

Example:

```
//Configure state-dependent timeouts
static const rx_iwdt_config_ts_iwdt_config = {
    .default_timeout_ms = k_iwdt_hw_timeout_ms,
    .state_timeouts_ms = {
        [k_system_state_init] = k_iwdt_timeout_init_ms, //5s
        [k_system_state_idle] = k_iwdt_timeout_idle_ms, //5s
        [k_system_state_running] = k_iwdt_timeout_running_ms, //2s
        [k_system_state_motor_active] = k_iwdt_timeout_motor_active_ms, //2s
        [k_system_state_comm_active] = k_iwdt_timeout_comm_active_ms, //2s
        [k_system_state_error] = k_iwdt_timeout_error_ms, //10s
    }
};
```

Since Version 1.0.0

—

iwdt_task_timeout_ms_t

IWDT per-task heartbeat timeout constants.

Defines individual watchdog timeout periods for each ThreadX task. Each task must call `rx_iwdt_task_heartbeat()` within its timeout period or the watchdog monitor will detect the failure and stop feeding the hardware IWDT, triggering a system reset after 2 seconds.

Timeout Calculation Strategy: Task timeout = 3x task period (allows 2 missed heartbeats before failure detection)

Timeout Categories:

- Fast tasks (10ms period): 30ms timeout - Motor control, communication, watchdog
- Medium tasks (50ms period): 150ms timeout - Telemetry, LED status
- Slow tasks (1000ms period): 3000ms timeout - temperature sensor
- Variable tasks: Custom timeout based on worst-case execution time

Failure Detection Flow:

1. Task misses heartbeat deadline
2. Watchdog monitor detects timeout via `rx_iwdt_check_tasks()`
3. Watchdog monitor stops feeding hardware IWDT
4. Hardware IWDT triggers reset after 2048ms
5. System reboots, failed task name preserved in logs

All values in milliseconds. Task timeouts < state timeouts < hardware IWDT timeout. Heartbeat intervals must be < task timeouts (typically timeout/3 for 2 missed beats margin). Valid ranges: fast tasks (20-50ms), medium tasks (100-300ms), slow tasks (2000-5000ms).

Value	Name	Description
= 150	<code>k_iwdt_task_timeout_telemetry_ms</code>	Telemetry task timeout (150ms). Task period: 50ms @ 20 Hz. Timeout = 3x period. Heartbeat called every 50ms. Valid range: 100-300ms. If exceeded: telemetry stops, system reset after 2s.
= 150	<code>k_iwdt_task_timeout_ledstatus_ms</code>	LED Status task timeout (150ms). Task period: 50ms @ 20 Hz. Timeout = 3x period. Heartbeat called every 50ms. Valid range:

		100-300ms. If exceeded: LED updates stop, system reset after 2s.
= 3000	k_iwdt_task_timeout_tempsensor_ms	Temperature Sensor task timeout (3000ms). Task period: 1000ms @ 1 Hz. Timeout = 3x period. Heartbeat called every 1s. Valid range: 2000-5000ms. If exceeded: temp compensation stops, system reset after 2s.
= 60	k_iwdt_task_timeout_obstdetect_ms	Obstacle Detection task timeout (60ms). Task heartbeat: 20ms @ 50 Hz. Timeout = 3x 20ms period. Heartbeat called every 20ms. Valid range: 50-150ms. If exceeded: collision avoidance stops, system reset after 2s.
= 30	k_iwdt_task_timeout_motorctrl_ms	Motor Control task timeout (30ms). Task period: 10ms @ 100 Hz. Timeout = 3x period. Heartbeat called every 10ms. Valid range: 20-50ms. If exceeded: motor control stops, system reset after 2s. CRITICAL TASK.
= 30	k_iwdt_task_timeout_commtask_ms	Communication task timeout (30ms). Task period: 10ms @ 100 Hz. Timeout = 3x period. Heartbeat called every 10ms. Valid range: 20-50ms. If exceeded: SPI comm stops, system reset after 2s. CRITICAL TASK.
= 30	k_iwdt_task_timeout_watchdog_ms	Watchdog Monitor task timeout (30ms). Task period: 10ms @ 100 Hz. Timeout = 3x period. Self-monitoring via own heartbeat. Valid range: 20-50ms. If exceeded: watchdog stops feeding IWDT, system reset after 2s. CRITICAL TASK.

Note: All timeouts are in milliseconds (ms)

Note: Task timeouts must be less than state timeout to allow proper detection

Note: Heartbeat interval must be less than timeout (typically timeout/3)] **See also:** rx_iwdt_register_task() Register task with timeout, rx_iwdt_task_heartbeat() Report task liveness, watchdog_monitor_task.c Monitors all task heartbeats at 100 Hz

Example:

```
//Register tasks with individual timeouts
rx_err_t err;
err = rx_iwdt_register_task("MotorCtrl", k_iwdt_task_timeout_motorctrl_ms); //30ms
```

```
err=rx_iwdt_register_task("CommTask",k_iwdt_task_timeout_commtask_ms);//30ms  
err=rx_iwdt_register_task("Telemetry",k_iwdt_task_timeout_telemetry_ms);//150ms
```

Since Version 1.0.0

—

s_owewire0_config

rx_bus_config_t s_owewire0_config

1-Wire configuration for DS18B20 temperature sensor

Note: Static allocation follows NASA Power of 10 Rule 3 (no dynamic memory).

Note: Registered with bus manager in tx_application_define() before task creation.

Note: Requires 4.7 kOhm pullup resistor on P51 (see schematic).

Note: 1-Wire protocol timing is critical - interrupts may cause timing violations.] **See also:**
rx_bus_config_init_owewire() Bus configuration function, temp_sensor_task.c Task
that uses this bus for temperature monitoring

Since Version 1.0.0

—

s_gpio_config

rx_bus_config_t s_gpio_config

Generic GPIO bus configuration for motor driver control.

Note: Static allocation follows NASA Power of 10 Rule 3 (no dynamic memory).

Note: Registered with bus manager in tx_application_define() before task creation.

Note: This is a generic bus abstraction - one "gpio" bus serves all 4 motor drivers.

Note: Each driver operation specifies the actual pin to access.] **See also:**
rx_bus_config_init_gpio() Bus configuration function, motor_control_task.c Task
that uses this bus for motor control, rx_bus_gpio_write() Runtime GPIO write with
pin specification, rx_bus_gpio_read() Runtime GPIO read with pin specification

Since Version 1.0.0

—

s_adc0_config

rx_bus_config_t s_adc0_config

ADC configuration for motor current sensing.

Note: Static allocation follows NASA Power of 10 Rule 3 (no dynamic memory).

Note: Registered with bus manager in tx_application_define() before task creation.

Note: This is a generic bus abstraction - one “adc0” bus serves all 4 motor drivers.

Note: Each current reading specifies the actual ADC channel to sample.

Note: 12-bit resolution provides 1 mV per count with 3.3V reference.] **See also:** rx_bus_config_init_adc() Bus configuration function, motor_control_task.c Task that uses this bus for current monitoring, rx_bus_adc_read_voltage_mv() Runtime ADC read with channel specification

Since Version 1.0.0

—

s_iwdt_config

```
const rx_iwdt_config_t s_iwdt_config
```

IWDT configuration for system watchdog monitoring.

Note: Configuration is immutable after rx_iwdt_init()

Warning: Configuration is immutable after rx_iwdt_init()] **See also:** rx_iwdt_init() Initialization function using this config, system_state_t State definitions for state_timeouts_ms array

Since Version 1.0.0

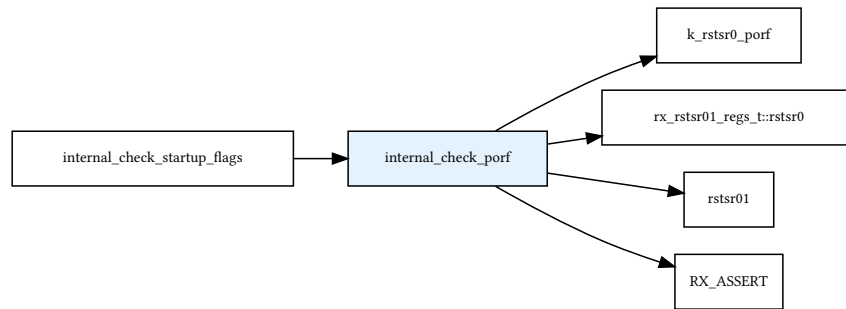
—

internal_check_porf

```
static bool internal_check_porf(void)
```

Check Power-On Reset Detect Flag (PORF) in RSTSR0 register.

Reads the RSTSR0 register to determine if the current boot was initiated by a power-on reset (fresh power application, not warm boot or software reset).

Figure 1318: Call graph for `internal_check_porf`

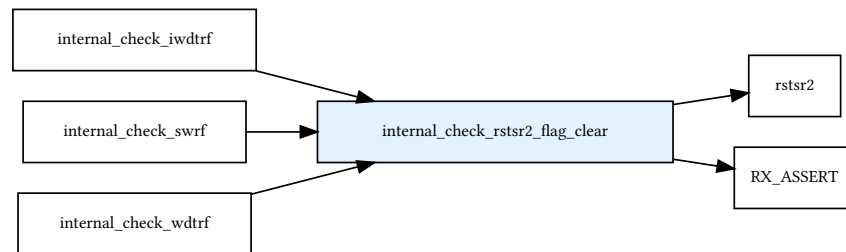
Defined in `src/main.c:456`

internal_check_rstsr2_flag_clear

```
static bool internal_check_rstsr2_flag_clear(const uint8_t flag_mask)
```

Helper function to check if a specific RSTSR2 flag is clear (not set).

Generic RSTSR2 flag checker used by multiple reset detection functions to reduce code duplication. Reads the RSTSR2 register and tests if a specific bit (identified by `flag_mask`) is clear (0).

Figure 1319: Call graph for `internal_check_rstsr2_flag_clear`

Defined in `src/main.c:565`

internal_check_iwdtrf

```
static bool internal_check_iwdtrf(void)
```

Check Independent Watchdog Timer Reset Detect Flag (IWDTRF) - CRITICAL safety check.

Reads the RSTSR2.1 (IWDTRF) bit to detect if the current boot was caused by an Independent Watchdog Timer (IWDG) timeout. IWDG timeout is a critical firmware error indicating the prior execution hung, deadlocked, or failed to refresh the watchdog.

CRITICAL: This is the most important reset flag check. If IWDTRF=1, the firmware MUST halt immediately (via `RX_ASSERT` in caller) to prevent cascading failures.

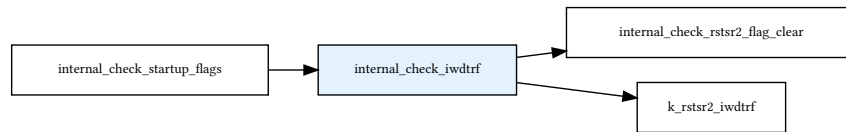


Figure 1320: Call graph for internal_check_iwdtrf

Defined in *src/main.c*:693

internal_check_wdtrf

```
static bool internal_check_wdtrf(void)
```

Check Watchdog Timer Reset Detect Flag (WDTRF).

Reads the RSTSR2.0 (WDTRF) bit to detect if the current boot was caused by a Watchdog Timer (WDT) timeout. Unlike IWDTRF (critical), WDTRF may be intentional in some scenarios (bootloader timeout, intentional watchdog reset).

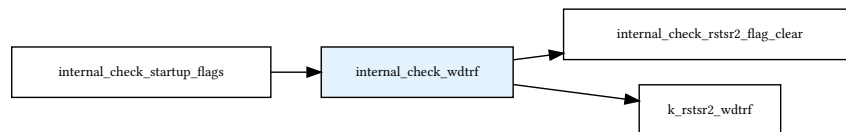


Figure 1321: Call graph for internal_check_wdtrf

Defined in *src/main.c*:750

internal_check_swrf

```
static bool internal_check_swrf(void)
```

Check Software Reset Detect Flag (SWRF).

Reads the RSTSR2.2 (SWRF) bit to detect if the current boot was caused by a software-initiated reset (via SWRR register write). Software resets are often intentional (bootloader entry, firmware update mode, debug reset) but may also indicate error recovery.

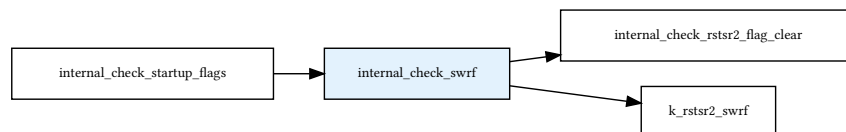


Figure 1322: Call graph for internal_check_swrf

Defined in *src/main.c*:829

internal_check_lvd0rf

```
static bool internal_check_lvd0rf(void)
```

Check Voltage-Monitoring 0 Reset Detect Flag (LVD0RF) - Brownout detection.

Reads the RSTSR0.1 (LVD0RF) bit to detect if the current boot was caused by a Low-Voltage Detect (LVD) reset. LVD0RF=1 indicates the supply voltage (VCC) dropped below the configured threshold, triggering a protective reset (brownout condition).

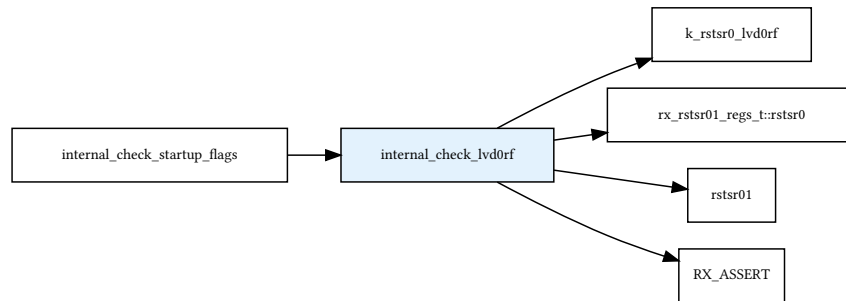


Figure 1323: Call graph for internal_check_lvd0rf

Defined in `src/main.c:921`

—

internal_check_cwsf

```
static bool internal_check_cwsf(void)
```

Check Cold/Warm Start Determination Flag (CWSF) - Boot type classification.

Reads the RSTSR1.7 (CWSF) bit to classify the boot type as cold start (power-on, voltage drop) or warm start (software reset, watchdog, debugger). This is informational only - both cold and warm starts are valid boot conditions.

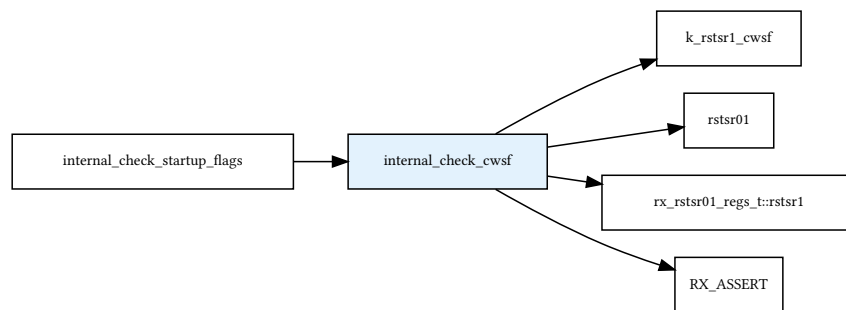


Figure 1324: Call graph for internal_check_cwsf

Defined in `src/main.c:1031`

—

internal_report_startup_flags

```
static void internal_report_startup_flags(void)
```

Report startup flags to UART console after early UART initialization.

Re-reads reset status registers and logs diagnostic information now that UART is available. This function is called after `uart_debug_init()` in `main()` to provide visibility into boot conditions that occurred before UART was initialized.

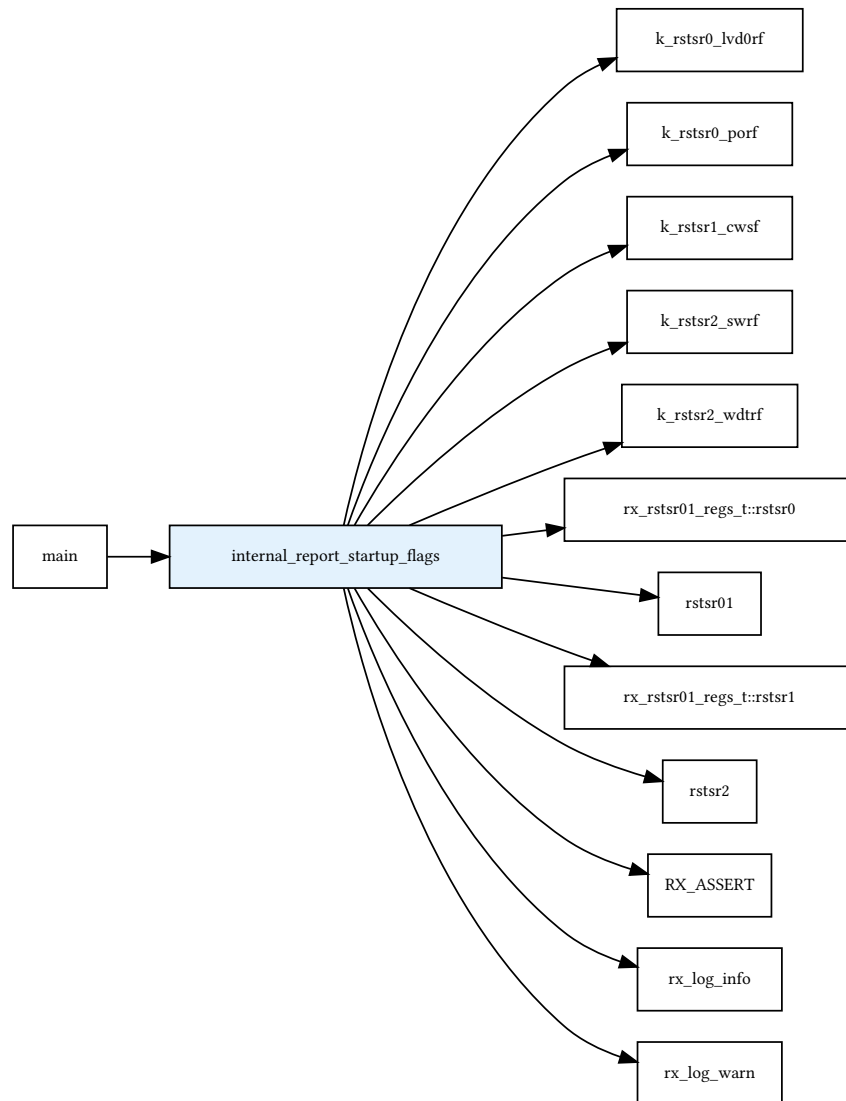


Figure 1325: Call graph for `internal_report_startup_flags`

Defined in `src/main.c:1132`

internal_check_startup_flags

```
static rx_err_t internal_check_startup_flags(void)
```

Validate all reset status flags to detect abnormal boot conditions.

Orchestrates six reset status register checks to detect abnormal reset conditions (watchdog timeout, brownout, software reset, etc.). Implements a fail-fast strategy for critical flags (IWDTRF) while logging non-critical warnings.

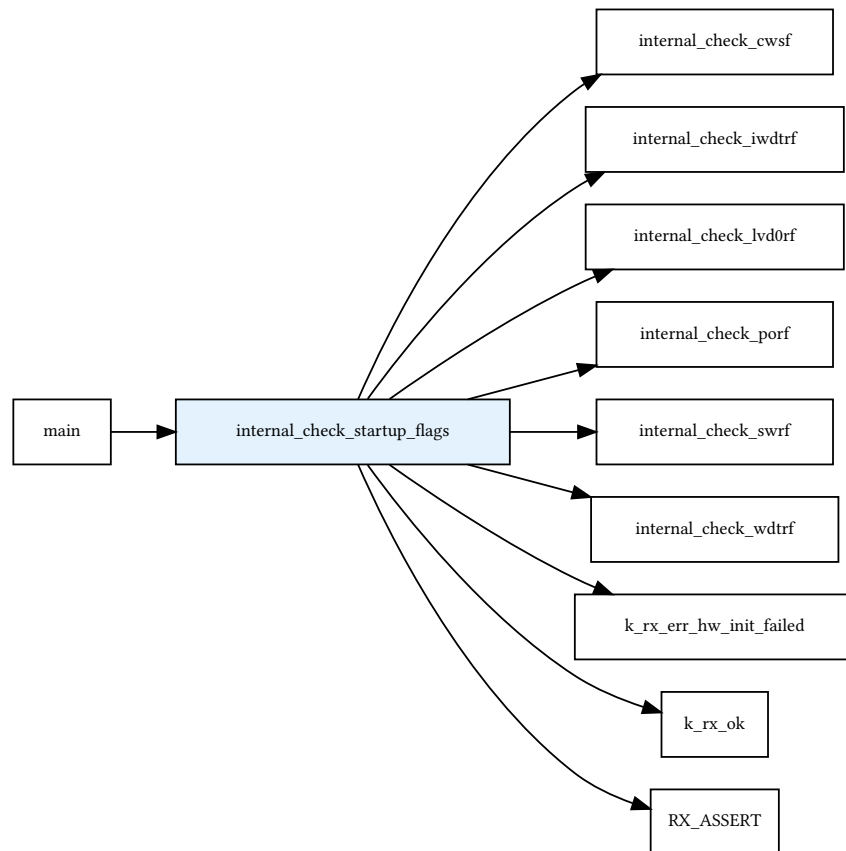


Figure 1326: Call graph for `internal_check_startup_flags`

Defined in `src/main.c:1316`

internal_init_bus_manager

```
static void internal_init_bus_manager(void)
```

Initialize the global bus manager with infrastructure interfaces.

Creates the bus manager singleton that all tasks use to access hardware buses. The bus manager requires a ThreadX mutex internally, so it must be initialized inside `tx_application_define` (after the ThreadX kernel is ready but before the scheduler starts).

Pre: `rx_infrastructure_init()` completed successfully in `main()`

Pre: ThreadX kernel initialized (mutex creation available)

Post: `g_bus_manager` initialized and ready for `rx_bus_manager_add_bus()` calls

Post: Infrastructure error and pin interfaces wired into the bus manager

Note: Executes in single-threaded context (scheduler not started). No synchronization needed.]

See also: `rx_bus_manager_init()` Underlying initialization function,
`internal_register_system_buses()` Registers individual buses after this call

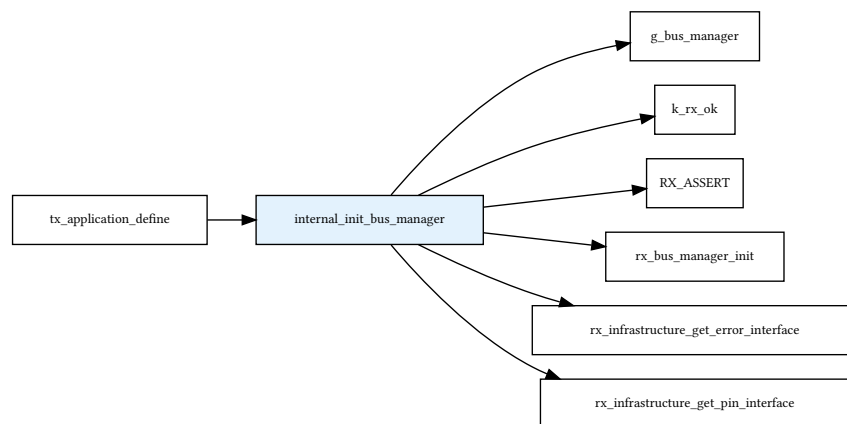


Figure 1327: Call graph for `internal_init_bus_manager`

Defined in `src/main.c:1586`

Since Version 1.0.0

—

internal_register_system_buses

```
static void internal_register_system_buses(void)
```

Register all hardware buses (1-Wire, GPIO, ADC) with the bus manager.

Initializes static bus configuration structs and registers them with the global bus manager. Each bus is available to tasks via `rx_bus_manager_get_interface()`.

Registered buses:

- onewire0 (P51): DS18B20 temperature sensor, 1-Wire bit-banging

- gpio (P00): Generic GPIO for motor driver control signals
- adc0 (S12AD0, ch0): Motor current sensing, 12-bit resolution

Pre: internal_init_bus_manager() completed successfully

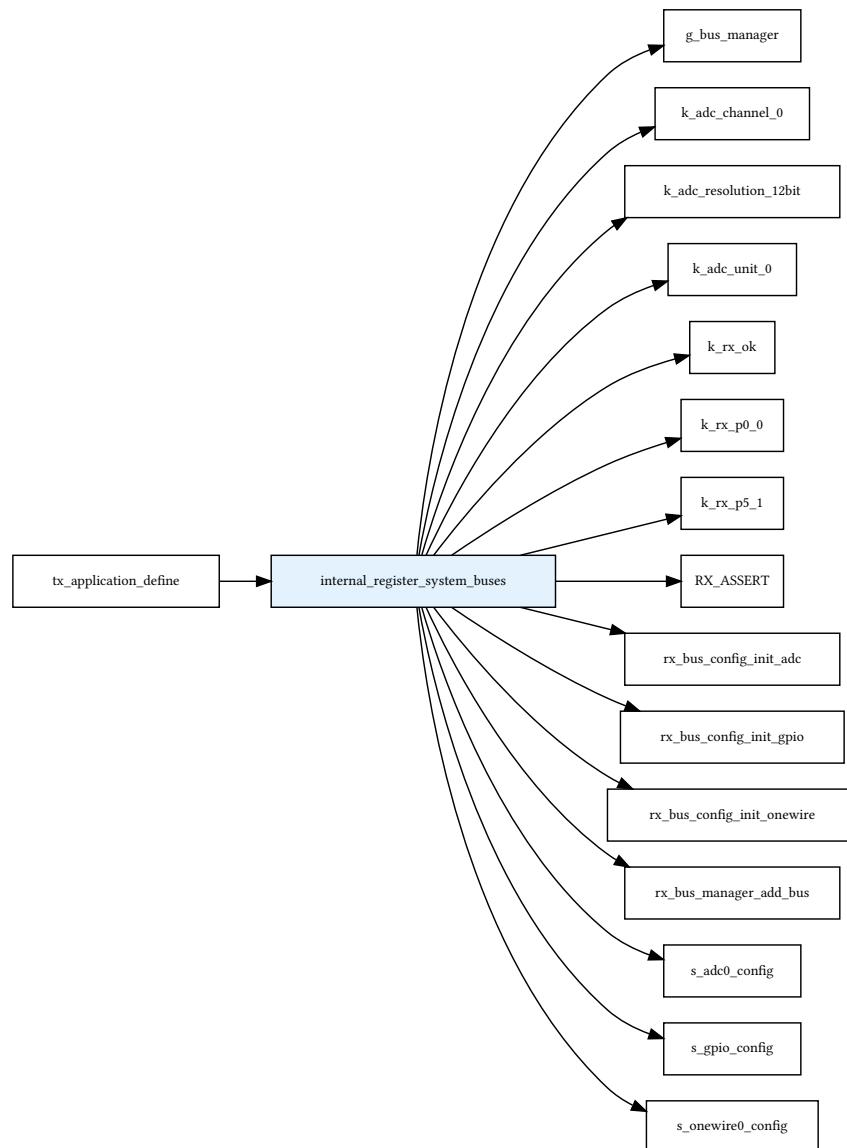
Pre: Static bus config structs (s_onewire0_config, s_gpio_config, s_adc0_config) are zeroed (BSS)

Post: All three buses registered and accessible via bus manager

Post: Static config structs populated with bus parameters

Note: Executes in single-threaded context (scheduler not started). No synchronization needed.]

See also: internal_init_bus_manager() Must be called first,
rx_bus_config_init_onewire() 1-Wire bus configuration, rx_bus_config_init_gpio()
GPIO bus configuration, rx_bus_config_init_adc() ADC bus configuration

Figure 1328: Call graph for `internal_register_system_buses`

Defined in `src/main.c:1624`

Since Version 1.0.0

—

internal_init_shared_data_and_watchdog

```
static void internal_init_shared_data_and_watchdog(void)
```

Initialize shared data module and hardware watchdog timer.

Sets up inter-task communication (ThreadX mutexes and event flags via `shared_data_init`) and initializes the Independent Watchdog Timer with state-dependent timeouts.

Pre: `internal_register_system_buses()` completed (buses available for tasks)

Pre: ThreadX kernel initialized (mutex/event-flag creation available)

Post: Shared data mutexes and event flags created and ready for task use

Post: IWDT hardware configured with `s_iwdt_config` parameters

Note: Executes in single-threaded context (scheduler not started). No synchronization needed.]
See also: `shared_data_init()` Inter-task communication setup, `rx_iwdt_init()` Hardware watchdog initialization

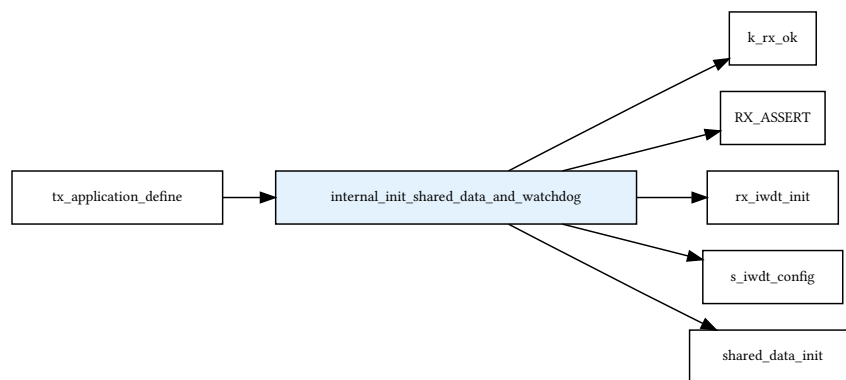


Figure 1329: Call graph for `internal_init_shared_data_and_watchdog`

Defined in `src/main.c:1675`

Since Version 1.0.0

internal_register_iwdt_tasks

```
static void internal_register_iwdt_tasks(void)
```

Register all seven tasks for IWDT heartbeat monitoring and set initial state.

Registers each application task with the IWDT module so that missed heartbeats are detected. Timeouts are set to 3x the nominal task period to allow two missed heartbeats before a timeout is declared.

Task timeout mapping:

- Telemetry (50ms period) -> 150ms timeout

- LED Status (50ms period) -> 150ms timeout
- Temp Sensor (1000ms period) -> 3000ms timeout
- Obstacle Detect (20ms period) -> 60ms timeout
- Motor Control (10ms period) -> 30ms timeout
- Communication (10ms period) -> 30ms timeout
- Watchdog Monitor (10ms period) -> 30ms timeout

After all tasks are registered the system state is set to `k_system_state_init` so the IWDT uses the 5-second init-phase timeout.

Pre: `internal_init_shared_data_and_watchdog()` completed (IWDT initialized)

Pre: No tasks registered yet (first call after `rx_iwdt_init`)

Post: All seven tasks registered with per-task heartbeat timeouts

Post: IWDT system state set to `k_system_state_init` (5s timeout)

Note: Executes in single-threaded context (scheduler not started). No synchronization needed.]

See also: `rx_iwdt_register_task()` Registers a single task for monitoring,
`rx_iwdt_set_state()` Sets system state for state-dependent timeouts



Figure 1330: Call graph for internal_register_iwdt_tasks

Defined in `src/main.c:1719`

Since Version 1.0.0

—

internal_create_system_tasks

```
static void internal_create_system_tasks(void)
```

Create all seven application tasks and transition to running state.

Creates all seven ThreadX tasks in lowest-priority-first order. Tasks do not begin executing until `tx_application_define()` returns and the scheduler starts. After all tasks are created, the IWDT system state transitions to `k_system_state_running`.

Task creation order (lowest priority first):

1. Telemetry (priority 18)
2. LED Status (priority 17)
3. Temperature Sensor (priority 15)
4. Obstacle Detection (priority 12)

5. Motor Control (priority 8)
6. Communication (priority 5)
7. Watchdog Monitor (priority 6)

Pre: `internal_register_iwdt_tasks()` completed (all tasks registered for monitoring)

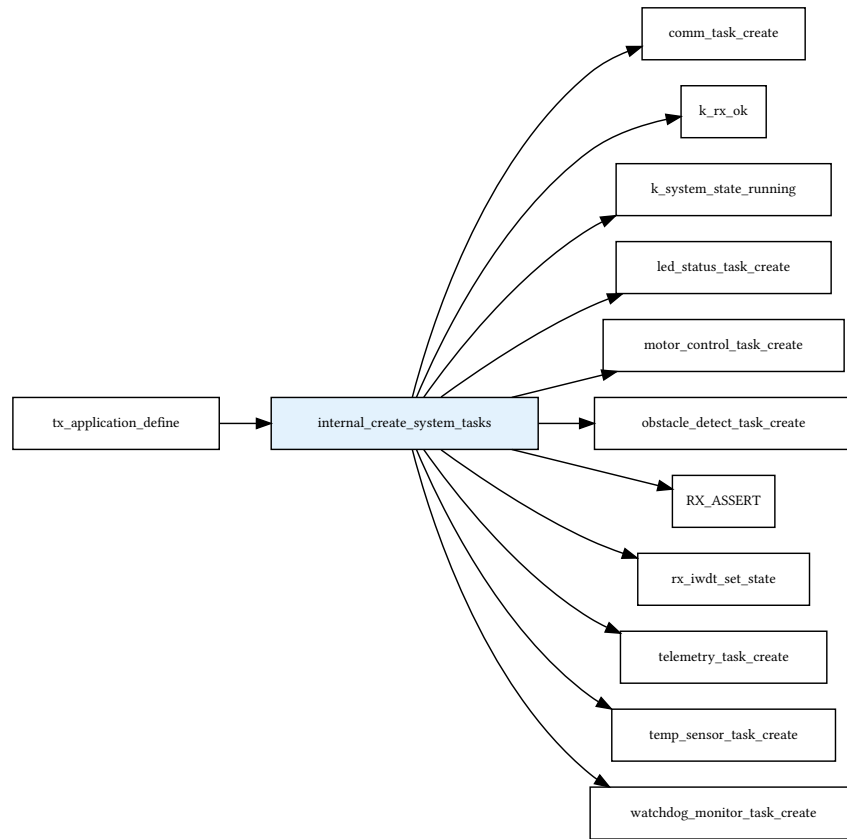
Pre: Static task stacks allocated in each task module

Post: All seven ThreadX threads created in READY state

Post: IWDT system state set to `k_system_state_running` (2s timeout)

Note: Executes in single-threaded context (scheduler not started). No synchronization needed.

Note: Tasks do not begin executing until `tx_application_define()` returns.] **See also:** `telemetry_task_create()` through `watchdog_monitor_task_create()`, `rx_iwdt_set_state()`
Transitions to running state after task creation

Figure 1331: Call graph for `internal_create_system_tasks`

Defined in `src/main.c:1779`

Since Version 1.0.0

—

internal_init_stack_monitor

```
static void internal_init_stack_monitor(void)
```

Register ThreadX stack overflow detection handler.

Installs the `rx_stack_monitor` callback that ThreadX invokes when any thread exceeds its allocated stack. Requires `TX_ENABLE_STACK_CHECKING` to be defined in the ThreadX build configuration.

Pre: All application tasks created via `internal_create_system_tasks()`

Pre: ThreadX `TX_ENABLE_STACK_CHECKING` enabled at build time

Post: Stack overflow handler registered with ThreadX kernel

Post: Any future stack overflow triggers rx_stack_monitor callback

Note: Executes in single-threaded context (scheduler not started). No synchronization needed.]

See also: rx_stack_monitor_init() Underlying registration function

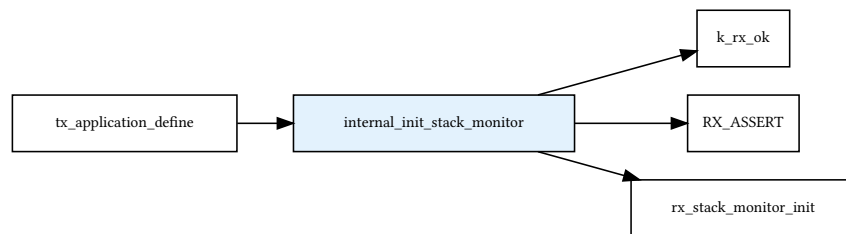


Figure 1332: Call graph for internal_init_stack_monitor

Defined in `src/main.c:1834`

Since Version 1.0.0

—

tx_application_define

```
void tx_application_define(void *first_unused_memory)
```

ThreadX application definition callback - Create application threads and kernel objects.

This function is called by the ThreadX kernel immediately after `tx_kernel_enter()`. It provides the application with an entry point to create threads, semaphores, message queues, mutexes, and other RTOS kernel objects.

ThreadX startup sequence:

Example:

```
main()->tx_kernel_enter()->[ThreadXinternalinit]->tx_application_define()->[schedulerstart]
```

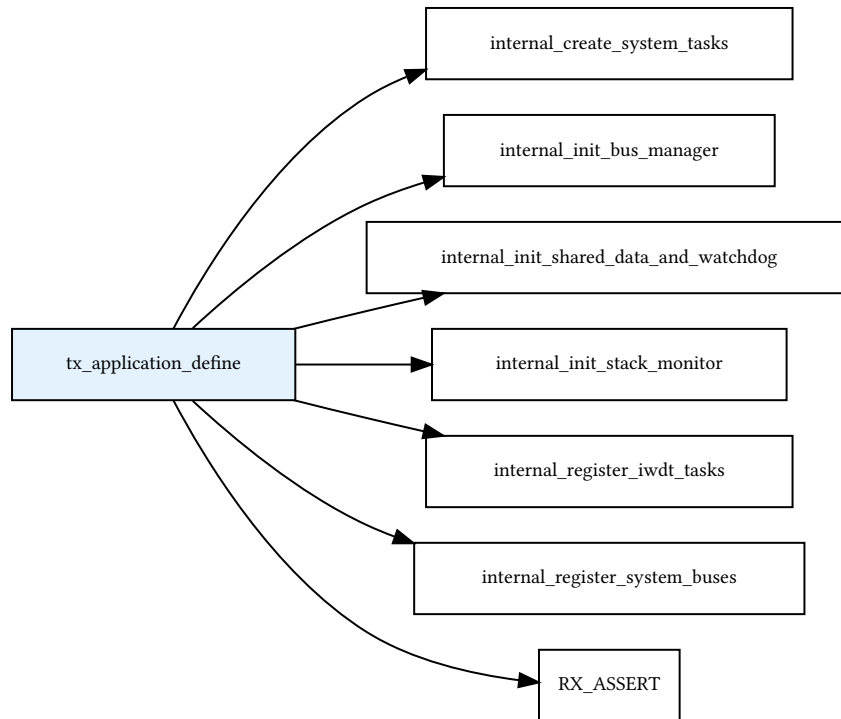


Figure 1333: Call graph for tx_application_define

Defined in *src/main.c*:1950

—

main

```
int main(void)
```

Main entry point for STAR RX72N motor controller firmware.

Implements the three-stage bootstrap sequence for the embedded system:

1. Stage 1: Startup Validation (10 us)

Read reset status registers (RSTSR0, RSTSR1, RSTSR2) Detect abnormal reset conditions (watchdog timeout, brownout, etc.) Assert-halt on critical flags (IWDTRF = firmware bug)

2. Stage 2: System Initialization (250 us)

Configure PLL for 240 MHz operation (rx_clock_power_init) Initialize UART for early error logging (uart_debug_init) Report startup flags to console (internal_report_startup_flags) Initialize infrastructure (error handler, pin validator) (rx_infrastructure_init) Initialize hardware (motor drivers, encoders, USB CDC) (hardware_init)

3. Stage 3: ThreadX Bootstrap (never returns)

Call tx_kernel_enter() to start RTOS scheduler ThreadX calls tx_application_define() callback
Application threads created (comm, motor, sensors, watchdog, telemetry, LED)

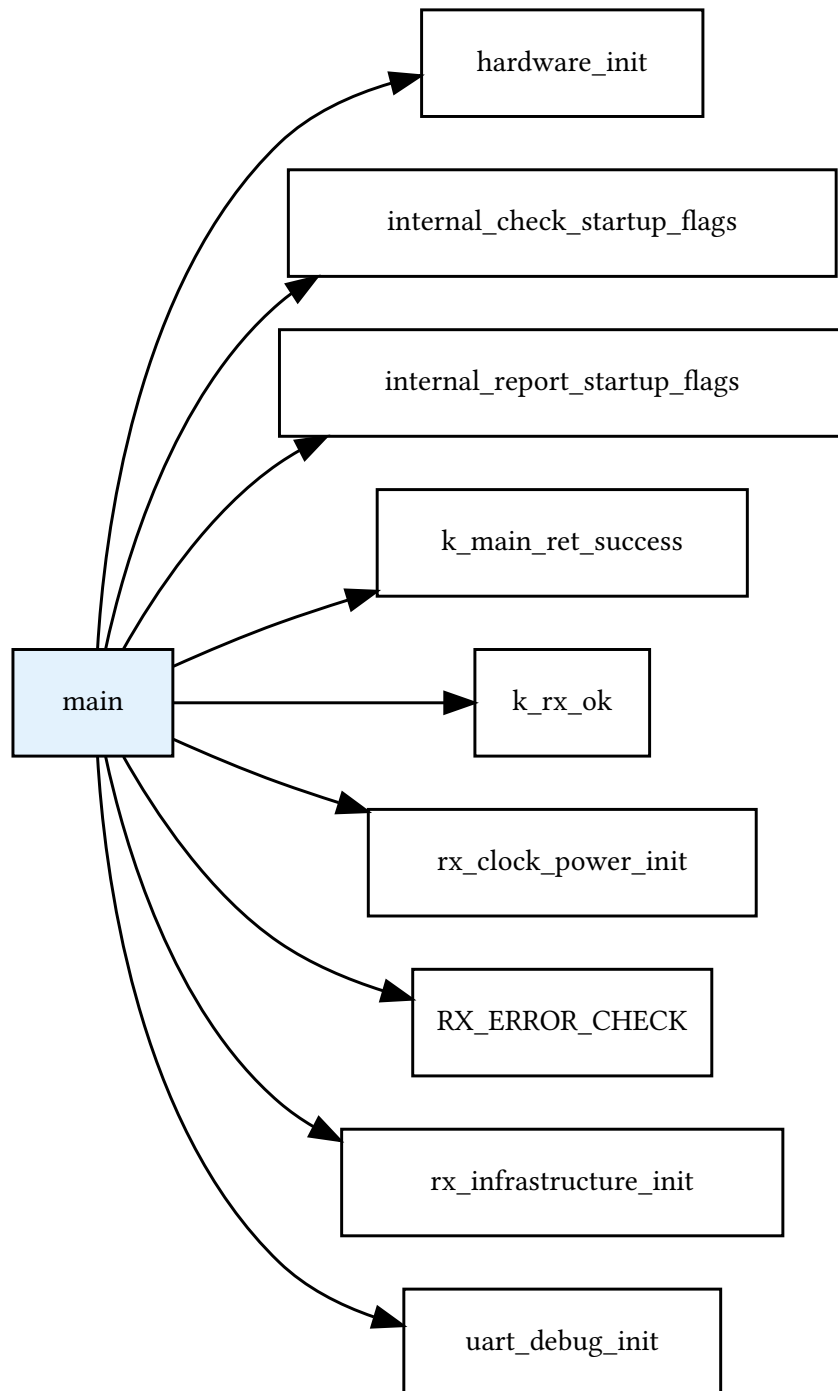


Figure 1334: Call graph for main

Defined in *src/main.c:2174*

—

rx_clock_power_init.c

RX72N System Clock and Power Management - 240 MHz PLL Configuration.

Includes:

- "rx_clock_power_init.h"
- <stdint.h>
- "rx72n_regs.h"
- "rx72n_rtc_regs.h"
- "rx_check.h"
- "rx_register_protection.h"
- "rx_simulator_config.h"

oscillator_timing_t

Oscillator stabilization timing constants.

Value	Name	Description
= 2400000	k_main_osc_stabilization_cycles	Main oscillator delay (10ms at 240MHz)
= 1000000	k_pll_stabilization_timeout	PLL stabilization max wait iterations
= 0	k_pll_stabilization_timeout_expired	PLL timeout expiration value

—

oscillator_control_t

Oscillator control register values.

Value	Name	Description
= 0x01	k_sub_clock_stopped	SOSCCR: Stop sub-clock oscillator
= 0x00	k_main_osc_enabled	MOSCCR: Enable main oscillator
= 0x00	k_pll_enabled	PLLCR2: Enable PLL

—

pll_config_t

PLL configuration values.

Value	Name	Description
= 0x1300	k_pll_multiplier_10_div_1	PLLCR: 10x multiplier, divide by 1 (240MHz from 24MHz)
= 0x04	k_pll_stable_flag	OSCOVFSR: PLL stabilization flag bit
= 0	k_pll_stable_flag_unset	PLL stable flag not yet asserted
= 0	k_ppll_stable_flag_unset	PPLL stable flag not yet asserted

—

system_clock_config_t

System clock configuration.

Value	Name	Description
= 0x21C21211	k_system_clock_dividers	SCKCR: ICLK=240MHz, PCLKA=120MHz, others=60MHz
= 0x0400	k_system_clock_source_pll	SCKCR3: Select PLL as system clock source

—

mstpcra_bits_t

Module stop bit positions in MSTPCRA.

Value	Name	Description
= 15	k_mstpcra_cmt	CMT0, CMT1 module stop bit
= 9	k_mstpcra_mtu	MTU module stop bit

—

mstpcrb_bits_t

Module stop bit positions in MSTPCRB.

Value	Name	Description
= 17	k_mstpcrb_rspi0	RSPI0 module stop bit
= 16	k_mstpcrb_rspi1	RSPI1 module stop bit

—

mstpcrc_bits_t

Module stop bit positions in MSTPCRC.

Value	Name	Description
= 17	k_mstpcrc_s12ad	S12AD module stop bit

—

module_init_config_t

Module initialization retry configuration.

Value	Name	Description
= 3	k_retry_count_module_stop	Number of retries for module stop register verification

—

s_tag`const char* s_tag`

—

internal_clock_init

```
static rx_err_t internal_clock_init(void)
```

Initialize clock system to 240 MHz.

Configures the full RX72N clock tree starting from the 24 MHz external crystal:

- Unlocks PRCR for clock register access
- Stops sub-clock oscillator (not used)
- Starts main oscillator and waits for stabilization
- Configures PLL (24 MHz x 10 / 1 = 240 MHz) and waits for PLL lock
- Configures PPLL (24 MHz x 8 / 4 = 48 MHz USB) and waits for PPLL lock
- Sets MEMWAIT=1 for safe flash access at 240 MHz (applied AFTER PLL and PPLL lock)
- Sets system clock dividers (ICLK=240, PCLKA=120, PCLKB/C/D=60, FCLK=60 MHz)
- Switches clock source to PLL
- Relocks PRCR

Must be called before any peripheral initialization or RTOS startup.

Returns: rx_err_t Initialization result

Value	Description
k_rx_ok	All oscillators locked and clocks configured correctly
k_rx_err_hw_timeout	PLL or PPLL failed to lock within stabilization limit

Pre: External 24 MHz crystal is connected and oscillating

Pre: VCC supply voltage is within RX72N operating range

Post: System clock running at 240 MHz (ICLK) sourced from PLL

Post: MEMWAIT=1 set for safe flash access at 240 MHz

Post: PRCR relocked to protect clock registers

Note: Thread safety: Must be called from single-threaded context before ThreadX starts

Note: Blocking: Waits for crystal and PLL stabilization (bounded by enum timeout constants)

NASA Power of 10 Compliance: Rule 5: [OK] 2 preconditions, 3 postconditions

See also: internal_verify_system_state() Post-initialization verification,
rx_clock_power_init() Public entry point that calls this function

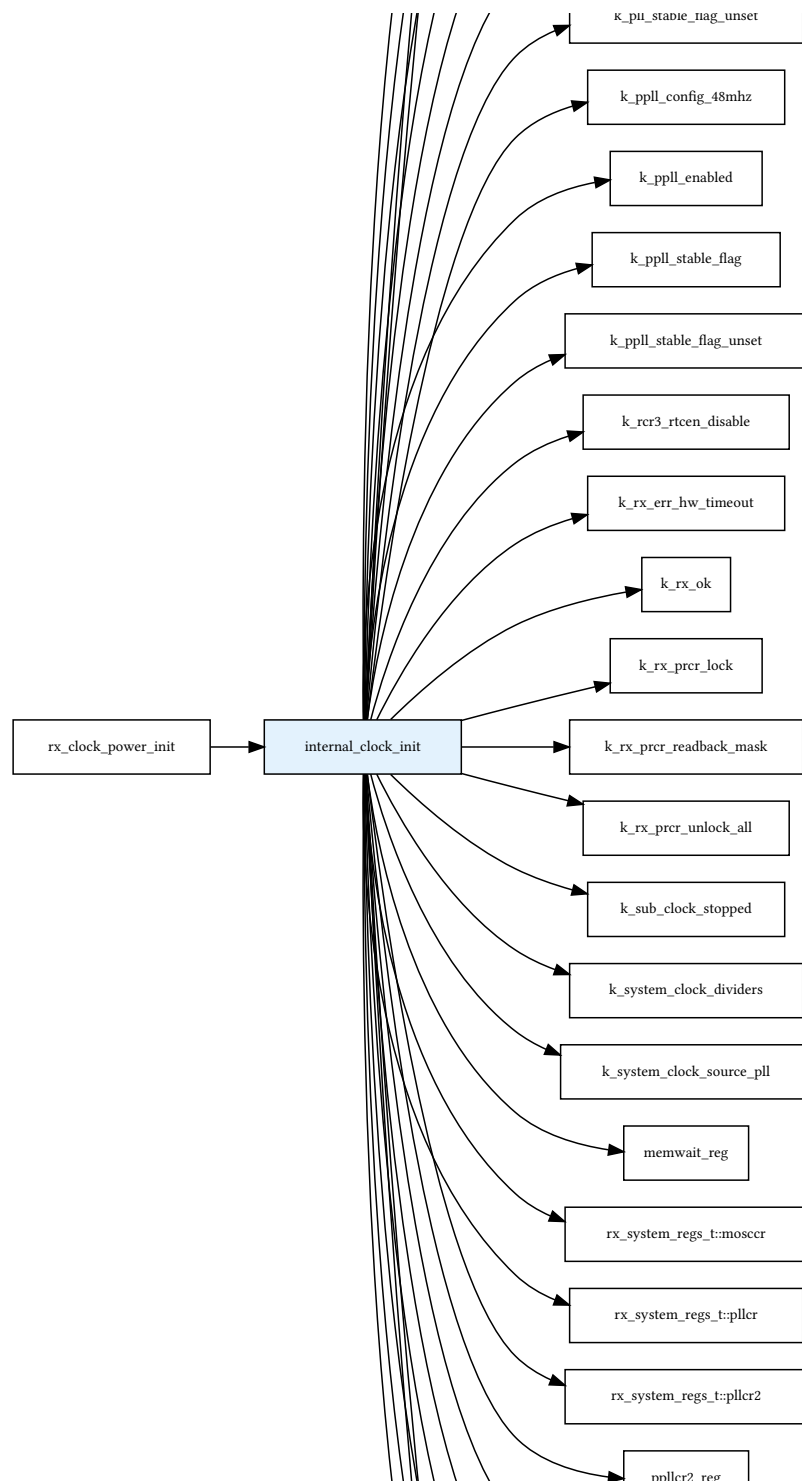


Figure 1335: Call graph for `internal_clock_init`

_Defined in src/rx_clock_power_init.c:352_

Since Version 1.0.0

—

internal_module_stop_init

```
static rx_err_t internal_module_stop_init(void)
```

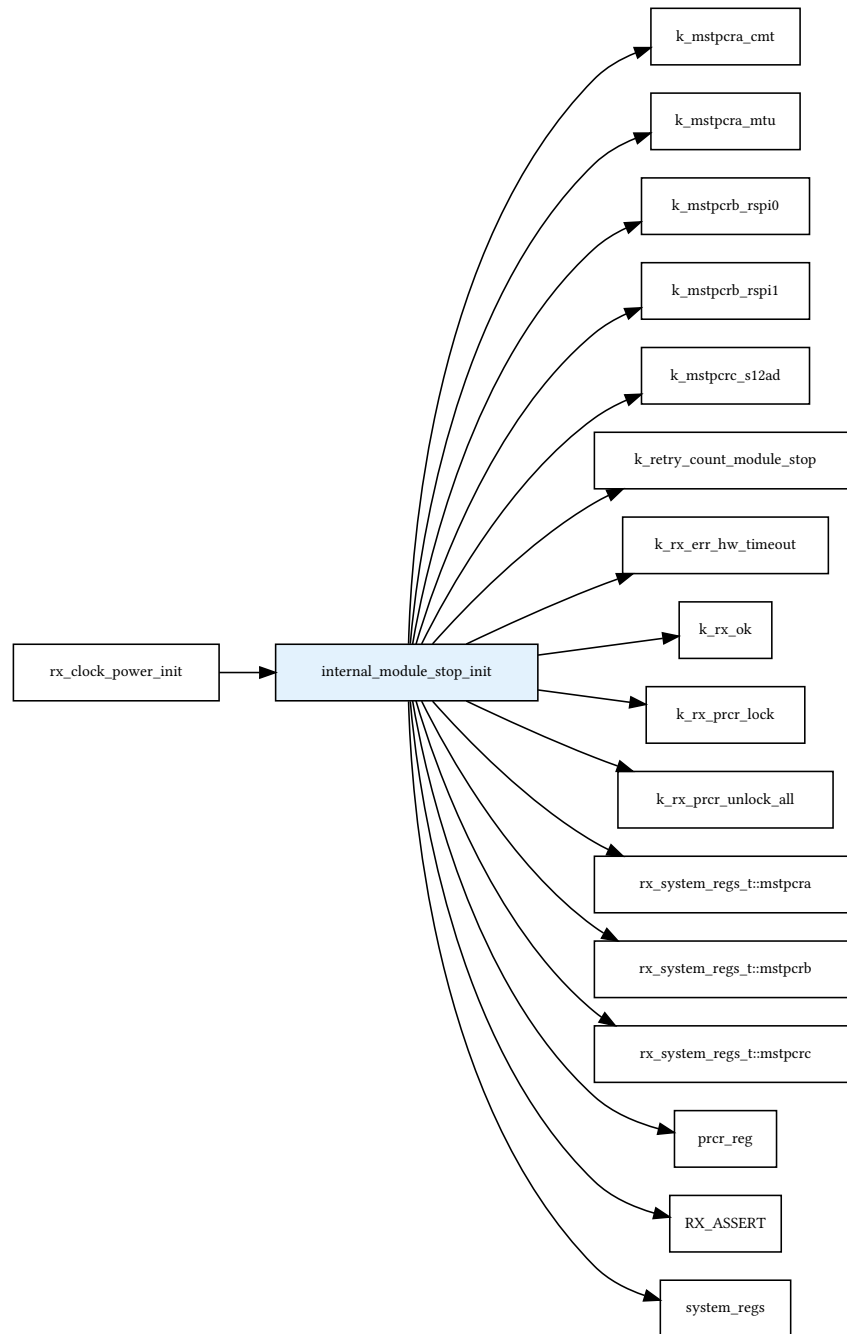
Enable peripheral modules.

Disables module stop for peripherals we'll use:

- CMT0 (system tick timer)
- MTU (motor PWM)
- S12AD (ADC)

Note: SCI modules are enabled per-channel in `uart_init_channel()`

Returns: `k_rx_ok` on success, `k_rx_err` if verification fails after retries

Figure 1336: Call graph for `internal_module_stop_init`

_Defined in `src/rx\_clock\_power\_init.c:512_`

internal_verify_system_state

```
static rx_err_t internal_verify_system_state(void)
```

Verify system clock configuration is correct.

Performs post-initialization validation of the RX72N clock subsystem by reading back hardware registers. Checks that:

- System register pointer is valid
- PLL is enabled (pllcr2 register)
- System clock dividers match expected values (sckcr register)
- PLL is selected as the system clock source (sckcr3 register)
- PPLL is enabled for USB clock (ppllcr2 register)
- PPLL oscillator is stable (oscovfsr register, hardware only)
- Flash wait state is configured for 240 MHz operation (memwait register)

Module-stop register verification (MSTPCRA, MSTPCRB) is performed by `internal_module_stop_init()`, not by this function.

Intended to be called immediately after `internal_clock_init()` to confirm that all register writes took effect.

Returns: rx_err_t Verification result

Value	Description
k_rx_ok	All clock configuration registers match expected values
k_rx_err_null_ptr	system_regs() returned nullptr
k_rx_err_hw_init_failed	One or more clock registers do not match expected values

Pre: `internal_clock_init()` completed successfully

Pre: Hardware registers are accessible (not in reset or low-power mode)

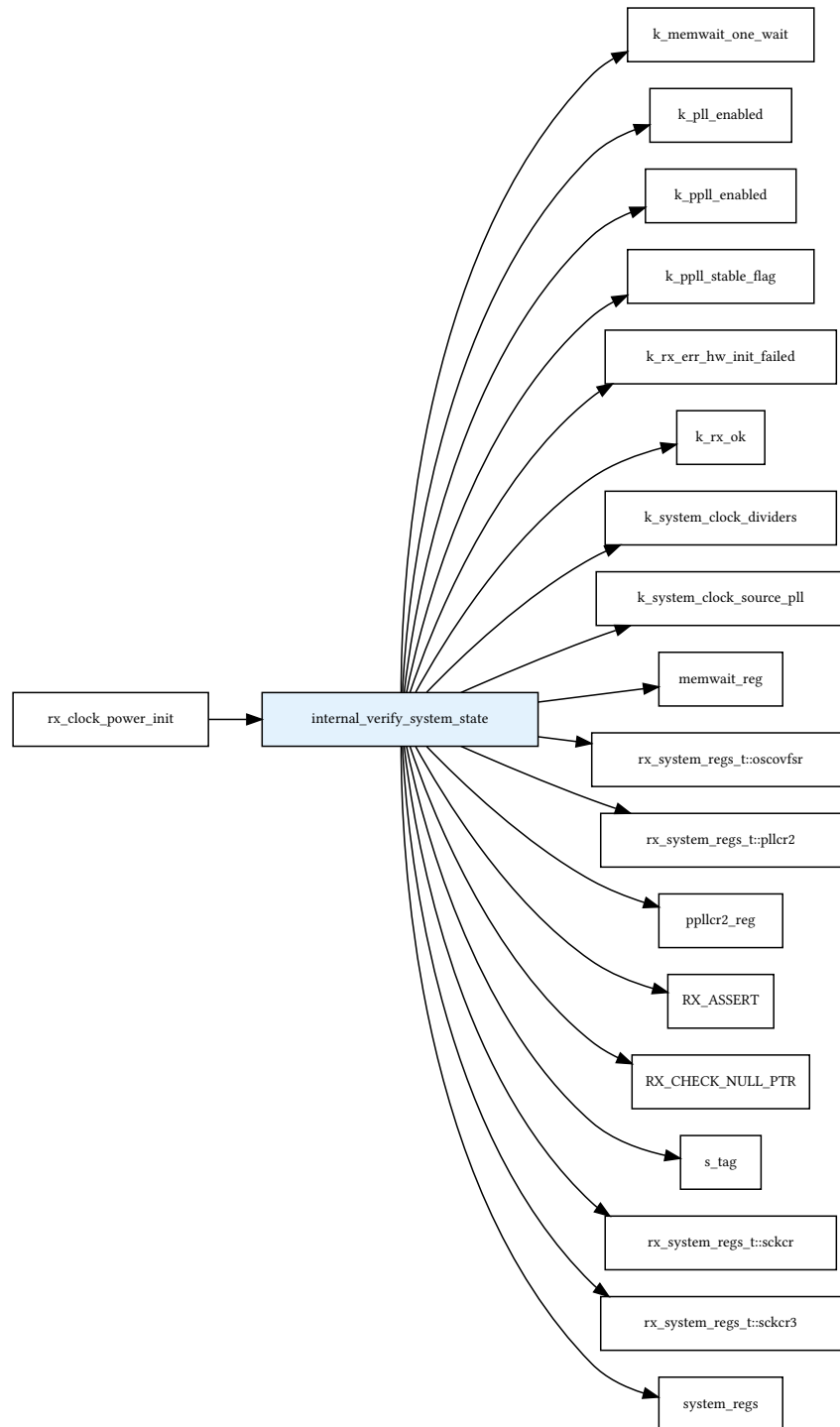
Post: No hardware state is modified (read-only verification)

Post: Return value reliably reflects whether system is in expected clock state

Note: Thread safety: Must be called from single-threaded context before ThreadX starts

NASA Power of 10 Compliance:: Rule 5: [OK] 2 preconditions, 2 postconditions

See also: `internal_clock_init()` Clock configuration that this function verifies, `rx_clock_power_init()` Public entry point

Figure 1337: Call graph for `internal_verify_system_state`

_Defined in src/rx_clock_power_init.c:595_

Since Version 1.0.0

—

rx_clock_power_init

```
rx_err_t rx_clock_power_init(void)
```

Initialize RX72N system clock and power management - Public entry point.

Initialize system clocks and power management.

Configures the complete clock tree and module power control for the RX72N MCU. This is the FIRST initialization function called from main() after reset, before any peripheral setup or RTOS startup.

Call sequence: Reset -> main() -> rx_clock_power_init() -> hardware_init() -> tx_kernel_enter()

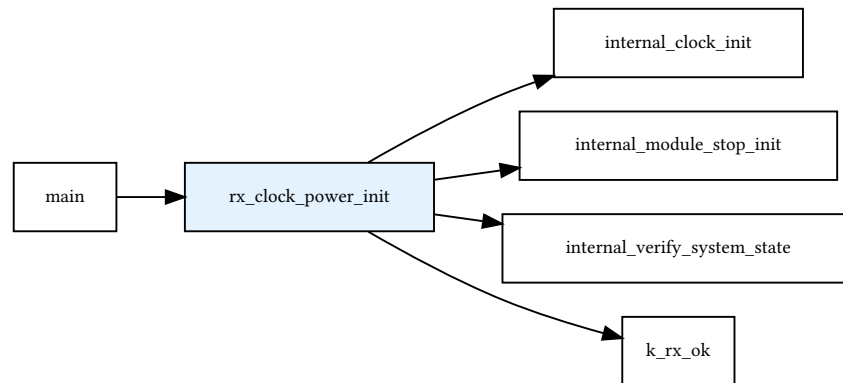


Figure 1338: Call graph for rx_clock_power_init

_Defined in src/rx_clock_power_init.c:888_

—

rx_stack_monitor.c

ThreadX Stack Overflow Detection and High-Water Mark Monitoring.

Implements the application-level ThreadX stack overflow handler required when `TX_ENABLE_STACK_CHECKING` is defined in `tx_user.h`. At every context switch ThreadX verifies the `0xEF` sentinel bytes placed by `TX_STACK_FILLING`; if those bytes are corrupted it calls the handler registered via `tx_thread_stack_error_notify()`.

Includes:

- "rx_stack_monitor.h"
- "rx_check.h"
- "rx_err.h"
- "rx_log.h"
- "tx_api.h"

stack_fill_constants_t

Stack fill pattern constants for high-water mark scanning.

ThreadX fills unused stack bytes with 0xEF when TX_DISABLE_STACK_FILLING is not defined. The 32-bit word pattern 0xEFEFEFEF is used by ThreadX internally (TX_STACK_FILL) and is replicated here for byte-level scanning.

k_stack_fill_byte == 0xEF (matches ThreadX fill byte)

Value	Name	Description
= 0xEF	k_stack_fill_byte	Byte value placed in unused stack memory by ThreadX
= 0U	k_stack_index_base	Index of the first (base) byte in the stack buffer
= 0U	k_stack_count_initial	Initial free-byte counter before scanning begins

See also: TX_STACK_FILL definition in tx_api.h

Example:

```
//Checkwhetherthefirststackbyteholdsthefillpattern:
if(stack_base[k_stack_index_base]==k_stack_fill_byte){
//patternpresent--scanningisvalid
}
```

Since Version 1.0.0

—

s_tag

```
const char* const s_tag
```

Log tag for stack monitor module.

Note: Read-only after initialisation (effectively constant).

Warning: Do not modify at runtime.

Since Version 1.0.0

—

internal_stack_overflow_handler

```
static void internal_stack_overflow_handler(TX_THREAD *thread_ptr)
```

Application-level ThreadX stack overflow handler.

ThreadX calls this function (via the registered callback pointer) whenever it detects that the 0xEF sentinel pattern at the bottom of a thread's stack has been corrupted during a context switch.

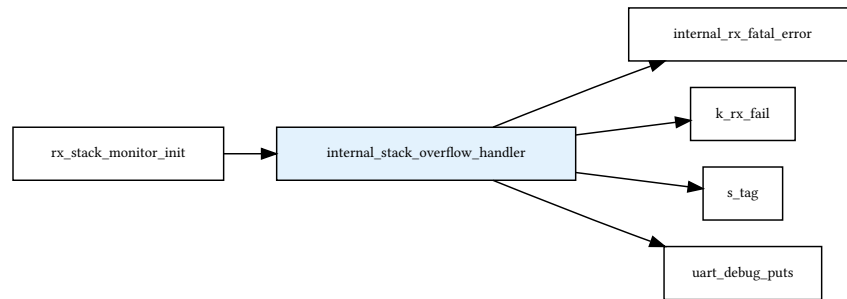


Figure 1339: Call graph for internal_stack_overflow_handler

Defined in *src/rx_stack_monitor.c:171*

—

rx_stack_monitor_init

```
rx_err_t rx_stack_monitor_init(void)
```

Register the stack overflow handler with ThreadX.

Calls tx_thread_stack_error_notify() to install internal_stack_overflow_handler() as the application callback. ThreadX invokes this callback at every context switch when a corrupted stack sentinel is detected.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Handler registered successfully
k_rx_err_not_supported	ThreadX returned TX_FEATURE_NOT_ENABLED, meaning TX_ENABLE_STACK_CHECKING was not defined at build time

Pre: TX_ENABLE_STACK_CHECKING is defined via USE_TX_ENABLE_STACK_CHECKING=1

Pre: Called from tx_application_define() (single-threaded context)

Post: _tx_thread_application_stack_error_handler set to the STAR handler

Post: Stack sentinel violations will call internal_stack_overflow_handler()

Note: Thread safety: single-threaded context in tx_application_define().

Warning: Returns k_rx_err_not_supported if built without stack checking.

NASA Power of 10 Compliance:: Rule 5: [PASS] 2 preconditions, 2 postconditions Rule 7: [PASS] tx_thread_stack_error_notify() return value checked and mapped

See also: `tx_thread_stack_error_notify()` Underlying ThreadX API,
`rx_stack_monitor_get_free_bytes()` Query per-thread headroom after init

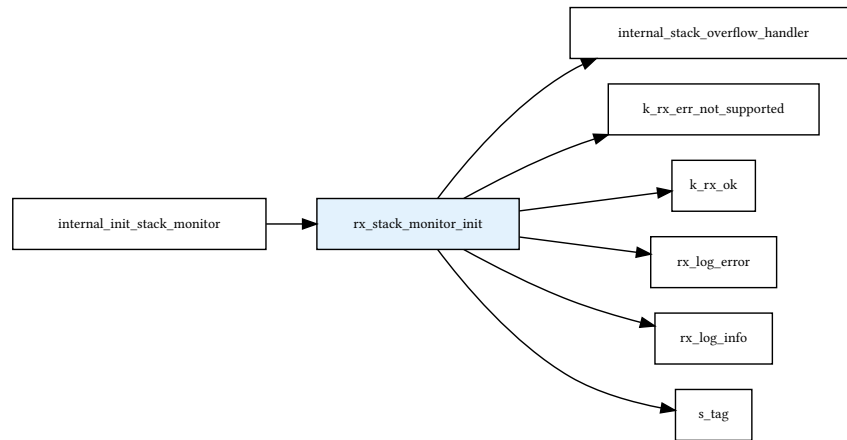


Figure 1340: Call graph for `rx_stack_monitor_init`

Defined in `src/rx_stack_monitor.c:234`

Since Version 1.0.0

`rx_stack_monitor_get_free_bytes`

```

rx_err_t rx_stack_monitor_get_free_bytes(const TX_THREAD *thread_ptr, uint32_t
*free_bytes)

```

Return the number of unused (free) bytes in a thread's stack.

Scans the thread's stack memory from the base (lowest address, where the fill pattern is placed first) toward higher addresses, counting consecutive 0xEF bytes. The result is the number of bytes that have never been written by the thread (i.e., the high-water mark complement).

Parameters:

Direction	Name	Description
in	<code>thread_ptr</code>	Non-NULL pointer to an initialised TX_THREAD.
out	<code>free_bytes</code>	Receives the free-byte count on <code>k_rx_ok</code> .

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Scan succeeded, <code>*free_bytes</code> is valid
<code>k_rx_err_null_ptr</code>	<code>thread_ptr</code> or <code>free_bytes</code> is NULL
<code>k_rx_err_invalid_state</code>	Fill pattern absent (stack checking disabled or stack completely overrun)

Pre: thread_ptr is a valid, initialised TX_THREAD (not NULL)

Pre: free_bytes points to a writable uint32_t (not NULL)

Post: *free_bytes contains the count of 0xEF bytes at stack base

Post: Thread state is not modified (read-only scan)

Note: Thread safety: snapshot only; result may change immediately after return.

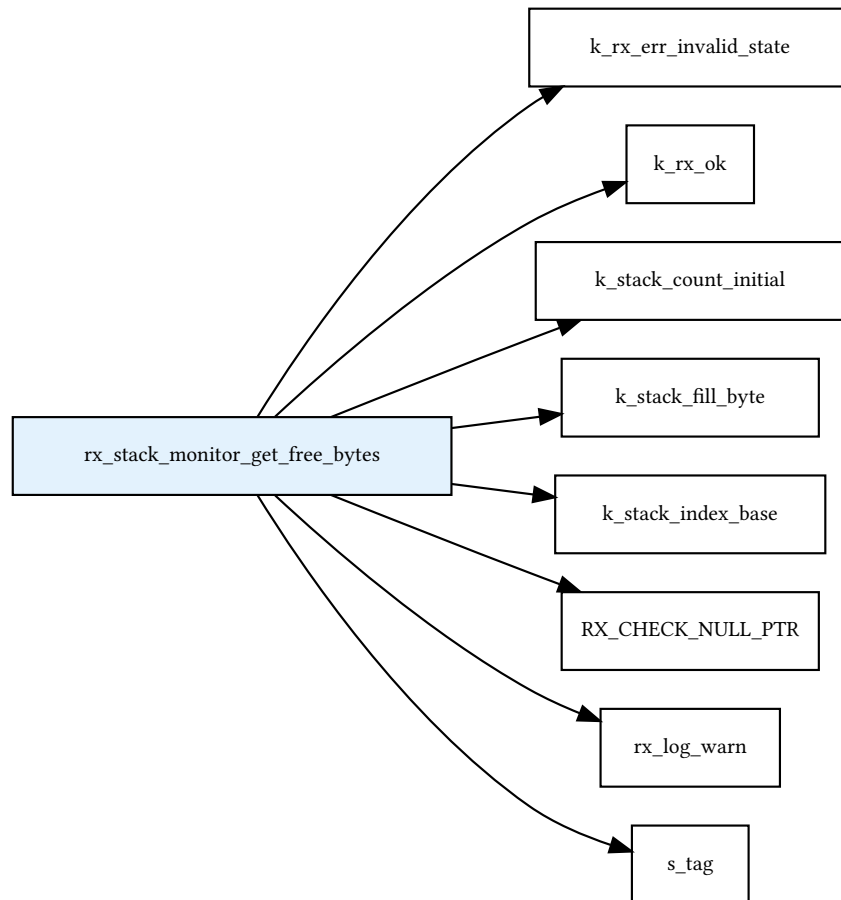
Warning: TX_DISABLE_STACK_FILLING must not be defined; otherwise the fill pattern is absent and this function returns k_rx_err_invalid_state.

NASA Power of 10 Compliance:: Rule 2: [PASS] Loop bound = tx_thread_stack_size (statically known at entry) Rule 5: [PASS] 2 preconditions (NULL checks), 2 postconditions

Example:

```
//Seerx_stack_monitor.hforcompleteusageexample.
```

See also: rx_stack_monitor_init() Must be called first to enable overflow detection

Figure 1341: Call graph for `rx_stack_monitor_get_free_bytes`

Defined in `src/rx_stack_monitor.c:293`

Since Version 1.0.0

—

shared_data.c

Thread-Safe Shared Data Infrastructure for Multi-Task Motor Control Architecture.

Includes:

- "shared_data.h"
- <string.h>
- "rx_bus_types.h"
- "rx_check.h"

shared_data_internal_constants_t

Internal constants for shared_data module implementation.

Constants used internally by the shared data module. Not exposed in public API.

Value	Name	Description
= 1	k_mutex_inherit	TX_INHERIT for mutex creation (priority inheritance enabled)
= 10	k_ms_per_tick	ThreadX milliseconds per tick (100 Hz tick rate = 10ms/tick)

—

s_default_pid_kp

```
const float s_default_pid_kp
```

Default PID proportional gain $K_p = 0.286$.

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_default_pid_ki

```
const float s_default_pid_ki
```

Default PID integral gain $K_i = 8.01$.

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_default_pid_kd

```
const float s_default_pid_kd
```

Default PID derivative gain $K_d = 0.0$ (not used).

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_default_pid_output_min

```
const float s_default_pid_output_min
```

Default PID output lower limit (% duty cycle).

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_default_pid_output_max

`const float s_default_pid_output_max`

Default PID output upper limit (% duty cycle).

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_default_pid_integral_min

`const float s_default_pid_integral_min`

Default PID integral lower limit (anti-windup).

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_default_pid_integral_max

`const float s_default_pid_integral_max`

Default PID integral upper limit (anti-windup).

Note: File-scope only; read via `shared_data_get_pid_gains()`, written via `shared_data_set_pid_gains()`

Warning: Do not modify directly; use `shared_data_set_pid_gains()` for runtime updates

Since Version 1.0.0

—

s_tag

```
const char* const s_tag
```

Log tag for this module (used by RX_CHECK_NULL_PTR).

—

s_estop_pending_from_isr

```
volatile bool s_estop_pending_from_isr
```

ISR-safe e-stop trigger flag (set by ISR, cleared by task).

Note: Volatile ensures ISR writes are not optimized away

Warning: RESTRICTED ACCESS: ISRs may ONLY SET this variable to true via `shared_data_trigger_estop_isr_safe()`. Motor control task is the ONLY context that may READ and CLEAR this variable via `shared_data_commit_isr_estop()`. All other access is FORBIDDEN. Non-ISR/non-task access will cause race conditions and undefined behavior.

Since Version 1.1.0

—

s_pending_estop_reason

```
volatile estop_reason_t s_pending_estop_reason
```

ISR-triggered e-stop reason code (volatile for ISR access).

Note: Volatile ensures ISR writes are visible to task

Warning: RESTRICTED ACCESS: ISRs may ONLY WRITE this variable via `shared_data_trigger_estop_isr_safe()`. Motor control task is the ONLY context that may READ this variable via `shared_data_commit_isr_estop()`. RACE CONDITION BEHAVIOR: If multiple ISRs fire simultaneously, last writer wins (acceptable for safety - any e-stop reason triggers shutdown). All other access is FORBIDDEN.

Since Version 1.1.0

—

g_bus_manager

```
rx_bus_manager_t g_bus_manager
```

Global bus manager instance for I2C/SPI/1-Wire communication.

Note: Do NOT access this variable directly from tasks. Use `rx_bus_manager_get_bus()`.] **See also:** `rx_bus_manager_init()` Initialize bus manager (in `hardware_init.c`), `rx_bus_types.h` Bus abstraction API

Example:

```
rx_bus_interface_t*i2c=rx_bus_manager_get_bus(&g_bus_manager,k_rx_bus_type_i2c);
i2c->read(i2c->ctx,imu_addr,data,len);
```

—

g_shared_data

shared_data_t [g_shared_data](#)

Global shared data instance accessed by all tasks.

Warning: Direct access bypasses mutex protection and causes data races!] **See also:** [shared_data_init\(\)](#) Create mutexes and event flags, [shared_data_t](#) Structure definition (in [shared_data.h](#))

—

shared_data_init

```
rx_err_t shared\_data\_init(void)
```

Initialize the shared data module.

Initialize shared data module.

Creates all ThreadX synchronization primitives and sets default values for shared state. Must be called exactly once from [tx_application_define\(\)](#) before any tasks start.

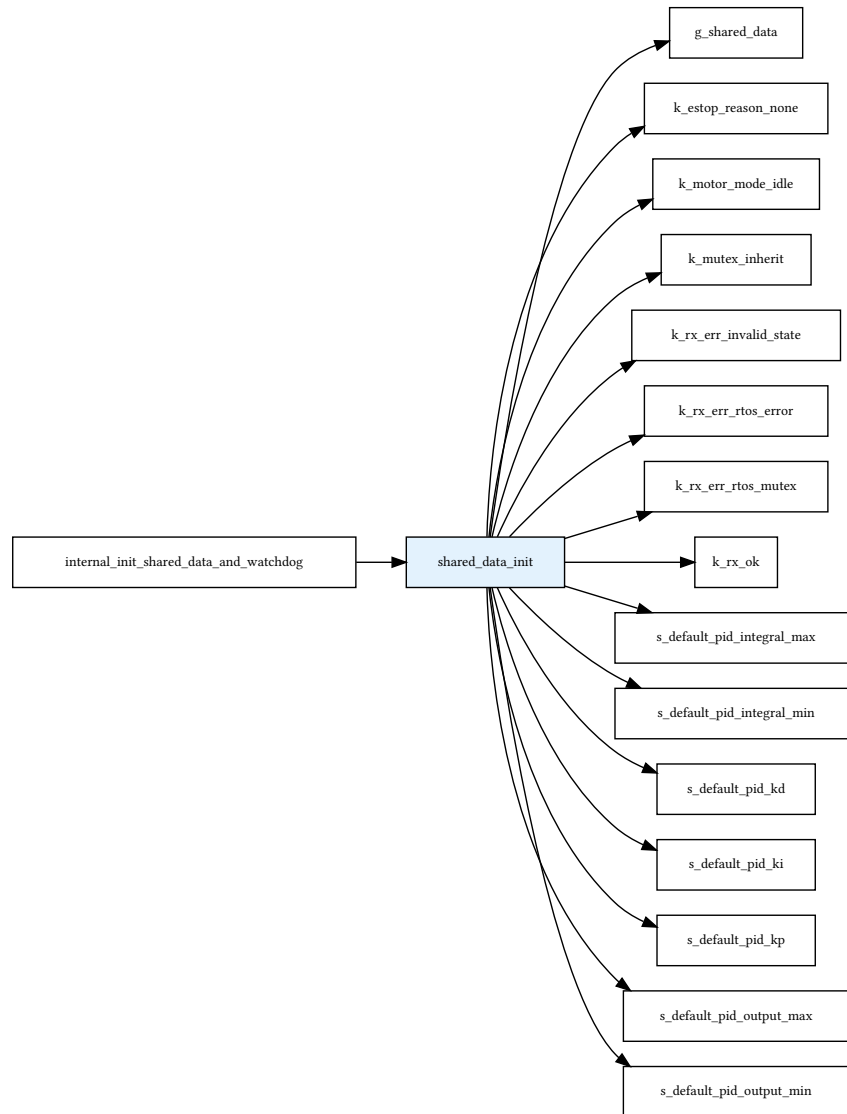


Figure 1342: Call graph for `shared_data_init`

_Defined in `src/shared/shared_data.c:693_`

—

`shared_data_set_motor_command`

```
rx_err_t shared_data_set_motor_command(const motor_command_t *cmd)
```

Set motor velocity command (called by Communication Task).

Set motor velocity command.

Stores a new motor velocity command and updates the communication timestamp for timeout detection. Sets the `k_event_motor_command_updated` event flag to wake the motor control task.

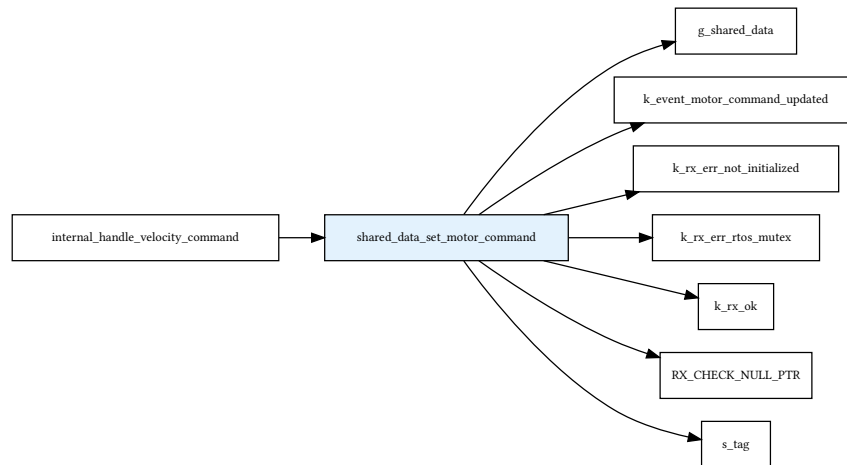


Figure 1343: Call graph for `shared_data_set_motor_command`

_Defined in `src/shared/shared_data.c:870_`

`shared_data_get_motor_command`

```
rx_err_t shared_data_get_motor_command(motor_command_t *out_cmd)
```

Get current motor velocity command (called by Motor Control Task).

Get motor velocity command.

Retrieves the latest motor velocity command for the motor control loop. Called at 250 Hz by motor task to fetch target velocities.

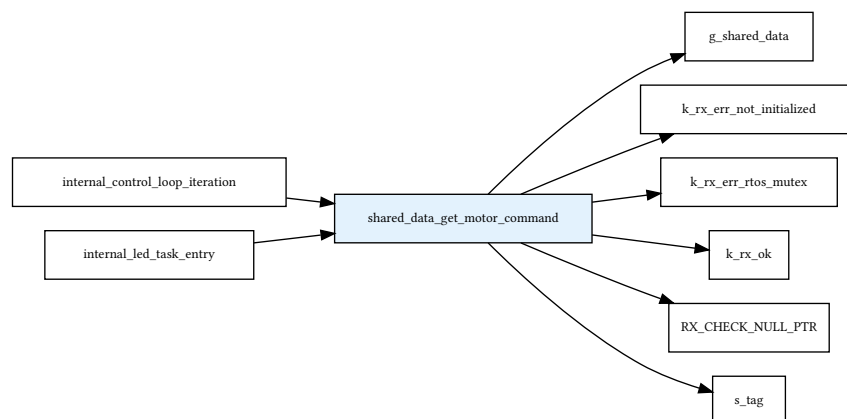


Figure 1344: Call graph for `shared_data_get_motor_command`

_Defined in `src/shared/shared_data.c:968_`

shared_data_update_motor_state

```
rx_err_t shared_data_update_motor_state(const motor_state_t *state)
```

Update motor state (called by Motor Control Task).

Update motor state telemetry.

Stores current motor telemetry (velocities, duty cycles, currents, faults) for the telemetry task to read. Called at 250 Hz after each control loop iteration.

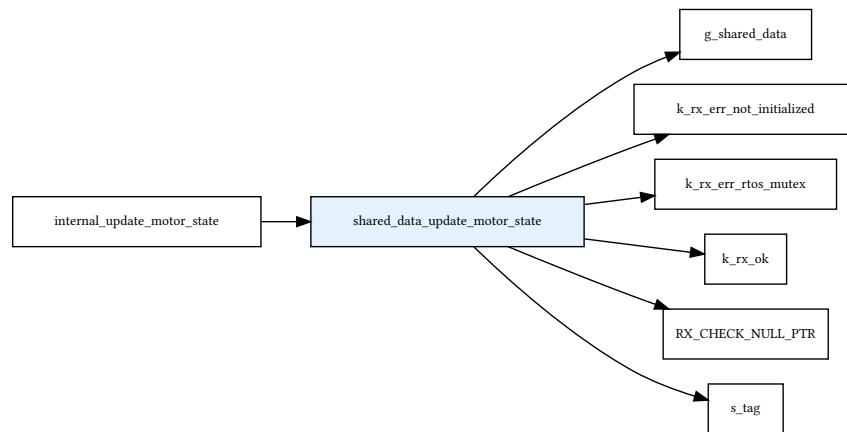


Figure 1345: Call graph for `shared_data_update_motor_state`

_Defined in `src/shared/shared_data.c:1058_`

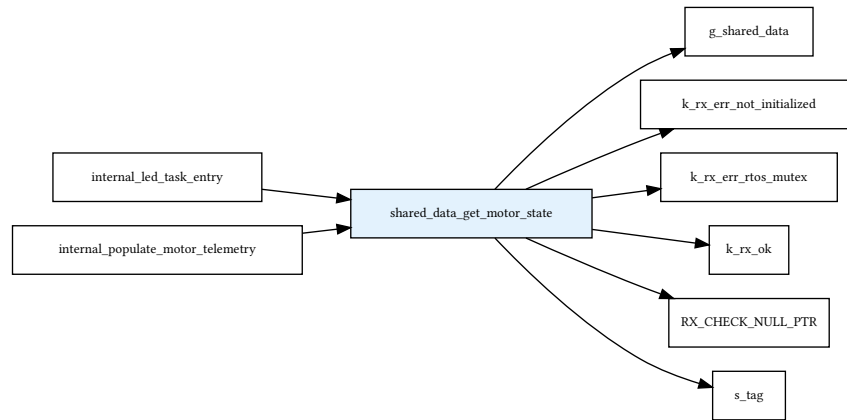
shared_data_get_motor_state

```
rx_err_t shared_data_get_motor_state(motor_state_t *out_state)
```

Get motor state (called by Telemetry Task).

Get motor state telemetry.

Retrieves current motor telemetry for transmission to ROS2 gateway. Called at 20 Hz by telemetry task.

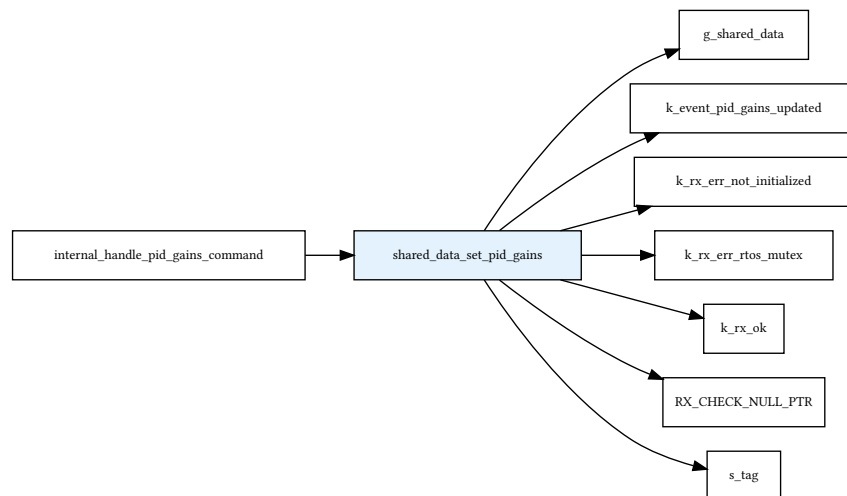
Figure 1346: Call graph for `shared_data_get_motor_state`_Defined in `src/shared/shared_data.c:1132_`**shared_data_set_pid_gains**

```
rx_err_t shared_data_set_pid_gains(const pid_gains_t *gains)
```

Set PID gains (called by Communication Task on SetPIDGainsRequest).

Set PID controller gains.

Updates PID controller gains at runtime for performance tuning. Sets the `update_pending` flag and signals `k_event_pid_gains_updated` event so motor task can apply new gains to all 4 PID controllers.

Figure 1347: Call graph for `shared_data_set_pid_gains`_Defined in `src/shared/shared_data.c:1231_`

shared_data_get_pid_gains

```
rx_err_t shared_data_get_pid_gains(pid_gains_t *out_gains)
```

Get PID gains (called by Motor Control Task).

Get PID controller gains.

Retrieves current PID gains for motor control. Called when motor task needs to apply updated gains to PID controllers.

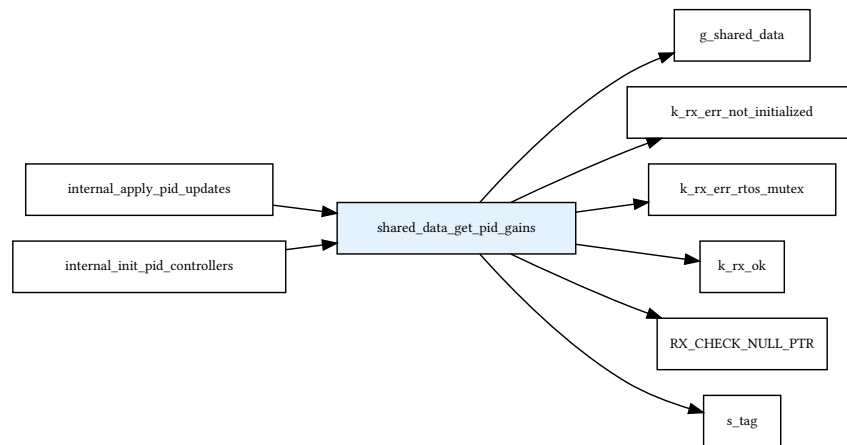


Figure 1348: Call graph for `shared_data_get_pid_gains`

_Defined in `src/shared/shared_data.c:1317_`

shared_data_pid_update_pending

```
bool shared_data_pid_update_pending(void)
```

Check if PID gains update is pending.

Check if PID update is pending.

Returns whether new PID gains have been set but not yet applied to controllers. Motor task polls this at 250 Hz to detect when gains need updating.

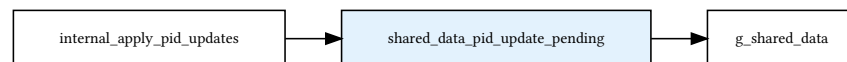


Figure 1349: Call graph for `shared_data_pid_update_pending`

_Defined in `src/shared/shared_data.c:1385_`

shared_data_clear_pid_update_flag

```
void shared_data_clear_pid_update_flag(void)
```

Clear PID update pending flag (called by Motor Task after applying gains).

Clear PID update flag.

Clears the update_pending flag after motor task has applied new PID gains to all controllers.
Prevents re-application of same gains.

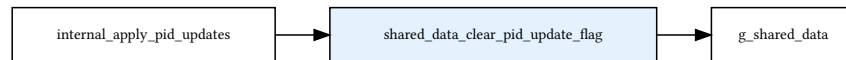


Figure 1350: Call graph for shared_data_clear_pid_update_flag

_Defined in src/shared/shared_data.c:1446_

shared_data_trigger_estop

```
rx_err_t shared_data_trigger_estop(estop_reason_t reason)
```

Trigger emergency stop.

Activates emergency stop with specified reason code. Sets the estop_active flag and signals k_event_estop_triggered event to immediately halt all motors.

Can be called from any task when dangerous condition detected:

- Communication Task: timeout, manual request
- Obstacle Task: collision imminent
- Motor Task: overcurrent, driver fault

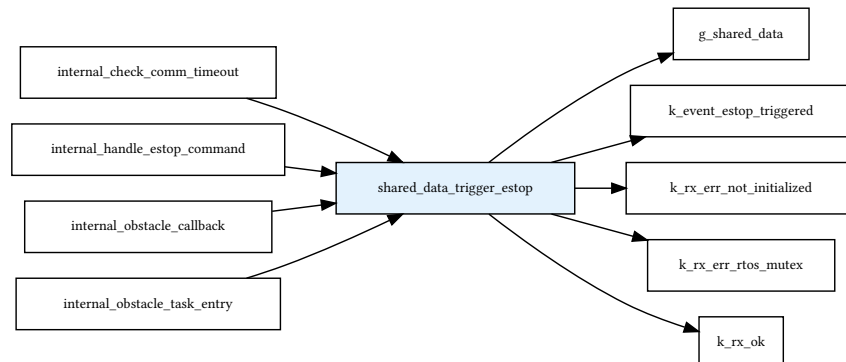


Figure 1351: Call graph for shared_data_trigger_estop

_Defined in src/shared/shared_data.c:1560_

shared_data_trigger_estop_isr_safe

```
void shared_data_trigger_estop_isr_safe(estop_reason_t reason)
```

Trigger emergency stop from ISR context (ISR-safe, no blocking).

ISR-safe version of `shared_data_trigger_estop()` that DOES NOT block on mutex. Sets a volatile flag and event flag only. Motor control task commits the pending e-stop to mutex-protected state via `shared_data_commit_isr_estop()`.

CRITICAL: This function is specifically designed for ISR context and MUST be used instead of `shared_data_trigger_estop()` when called from interrupt handlers.

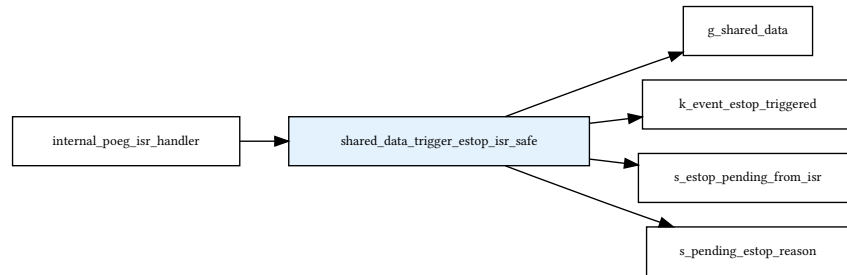


Figure 1352: Call graph for `shared_data_trigger_estop_isr_safe`

_Defined in `src/shared/shared_data.c:1641_`

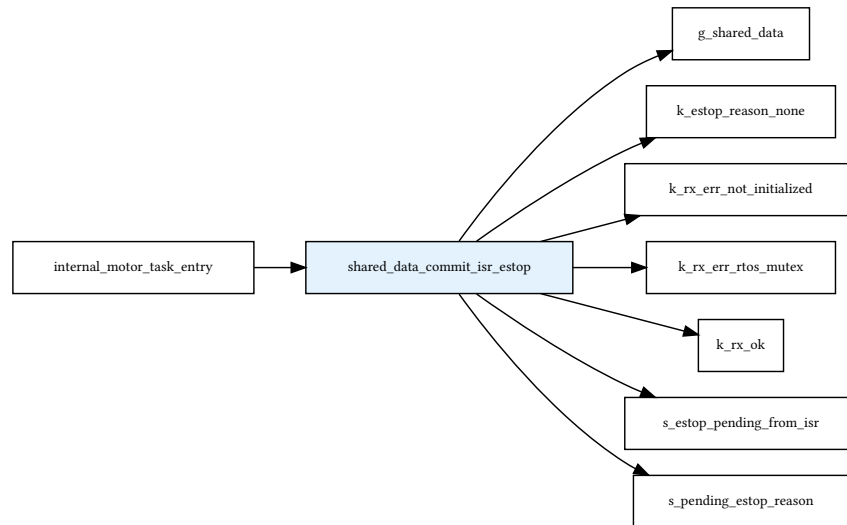
shared_data_commit_isr_estop

```
rx_err_t shared_data_commit_isr_estop(void)
```

Commit ISR-triggered e-stop to mutex-protected state (task context).

Transfers ISR-triggered e-stop from volatile flags to mutex-protected shared state. Called by motor control task at start of each 4ms iteration (250 Hz).

Why this is needed: ISRs cannot block on mutexes, so they set a volatile flag. This function runs in task context where mutex acquisition is safe.

Figure 1353: Call graph for `shared_data_commit_isr_estop`

_Defined in `src/shared/shared_data.c:1723_`

shared_data_clear_estop

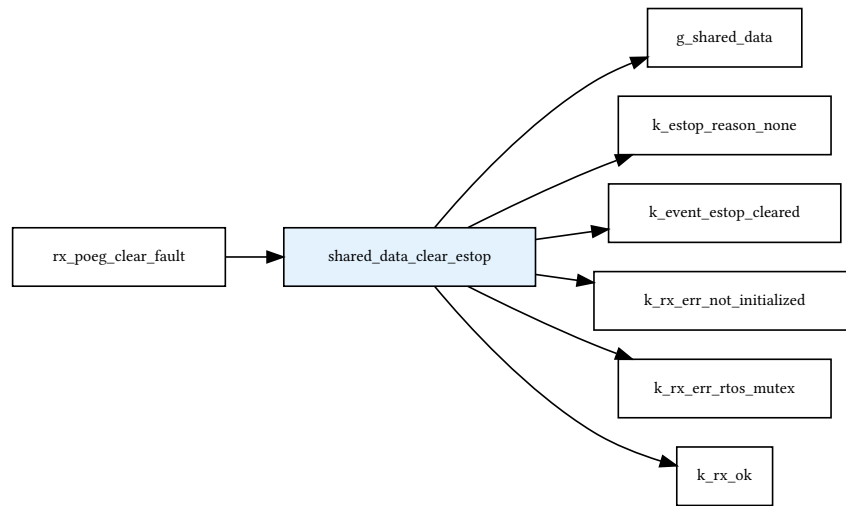
```
rx_err_t shared_data_clear_estop(void)
```

Clear emergency stop.

Clears the emergency stop flag after fault condition has been resolved. Allows system to resume normal operation. Sets `k_event_estop_cleared` event.

Should only be called after:

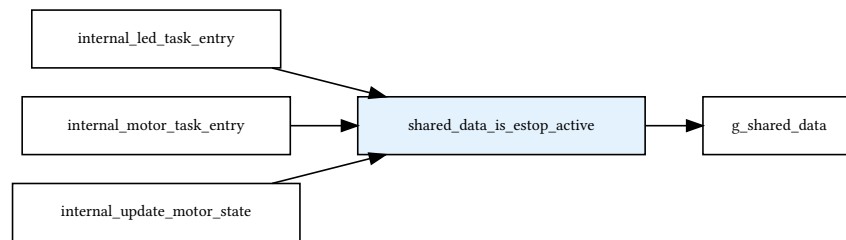
- Fault condition verified resolved
- System returned to safe state
- Manual operator approval received

Figure 1354: Call graph for `shared_data_clear_estop`_Defined in `src/shared/shared_data.c:1827_`**shared_data_is_estop_active**

```
bool shared_data_is_estop_active(void)
```

Check if emergency stop is active.

Returns current emergency stop status. Motor task checks this at 250 Hz to immediately halt outputs when e-stop triggered.

Figure 1355: Call graph for `shared_data_is_estop_active`_Defined in `src/shared/shared_data.c:1900_`**shared_data_get_estop_reason**

```
estop_reason_t shared_data_get_estop_reason(void)
```

Get emergency stop reason.

Get emergency stop reason code.

Returns the reason code for why emergency stop was triggered. Used for diagnostics, logging, and displaying fault information to operator.

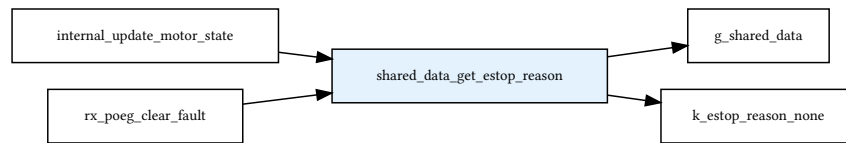


Figure 1356: Call graph for shared_data_get_estop_reason

_Defined in src/shared/shared_data.c:1976_

shared_data_update_temp

```
rx_err_t shared_data_update_temp(const temp_sensor_state_t *state)
```

Update temperature sensor state (called by Temperature Task).

Update temperature sensor state.

Stores temperature readings from DS18B20 1-Wire sensors. Called at 1 Hz by temperature task. Used for ambient temperature compensation of HC-SR04.

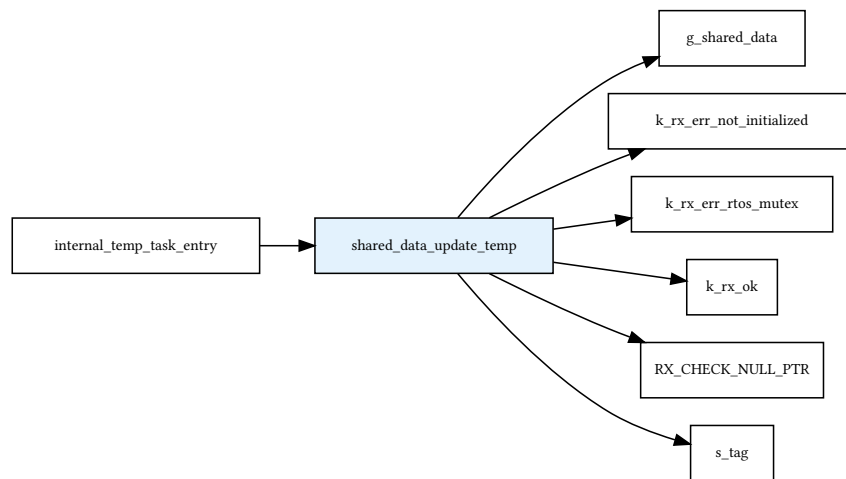


Figure 1357: Call graph for shared_data_update_temp

_Defined in src/shared/shared_data.c:2055_

shared_data_get_temp

```
rx_err_t shared_data_get_temp(temp_sensor_state_t *out_state)
```

Get temperature sensor state (called by Telemetry Task).

Get temperature sensor state.

Retrieves temperature readings for telemetry reporting. Called at 20 Hz by telemetry task.

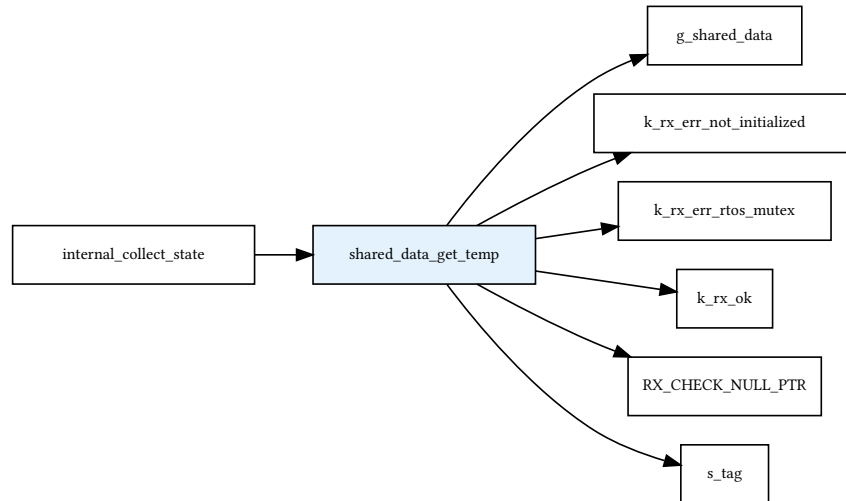


Figure 1358: Call graph for `shared_data_get_temp`

_Defined in `src/shared/shared_data.c:2130_`

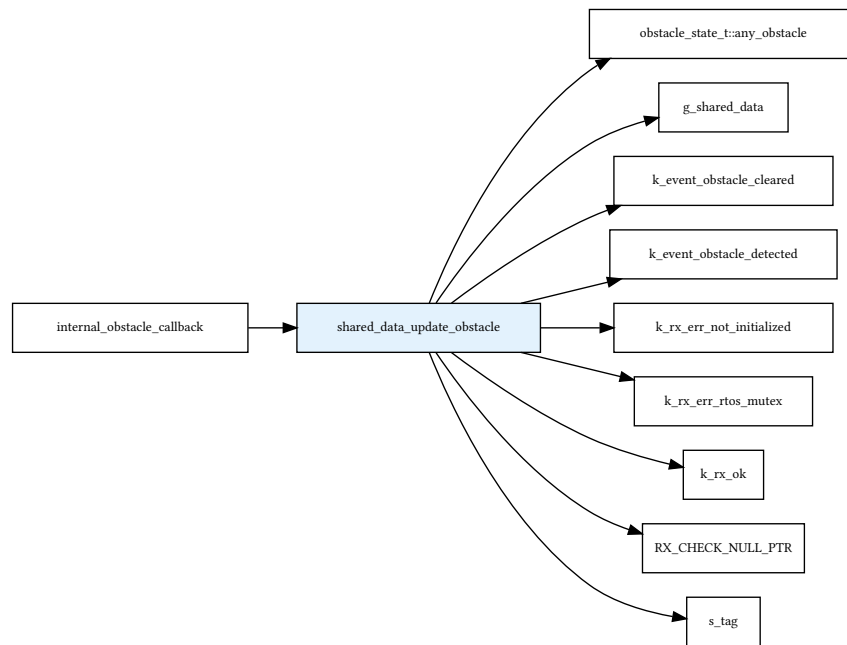
shared_data_update_obstacle

```
rx_err_t shared_data_update_obstacle(const obstacle_state_t *state)
```

Update obstacle detection state (called by Obstacle Task).

Update obstacle detection state.

Stores distance readings from HC-SR04 ultrasonic sensors. Called when new measurements available (typically 10-20 Hz). Automatically sets obstacle detected/cleared events based on distance thresholds.

Figure 1359: Call graph for `shared_data_update_obstacle`

_Defined in `src/shared/shared_data.c:2227_`

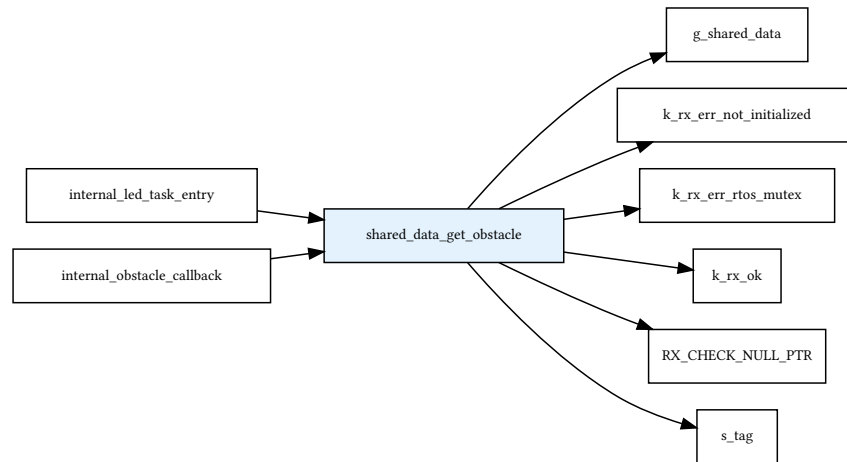
`shared_data_get_obstacle`

```
rx_err_t shared_data_get_obstacle(obstacle_state_t *out_state)
```

Get obstacle detection state (called by Telemetry Task).

Get obstacle detection state.

Retrieves obstacle detection data for telemetry reporting. Called at 20 Hz by telemetry task.

Figure 1360: Call graph for `shared_data_get_obstacle`

_Defined in `src/shared/shared\_data.c:2307_`

shared_data_is_comm_timeout

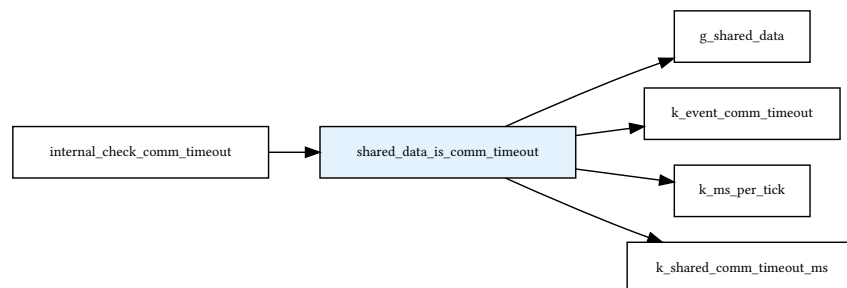
```
bool shared_data_is_comm_timeout(void)
```

Check if communication has timed out.

Check if communication timeout occurred.

Determines if motor commands are stale by comparing current time to last command timestamp. Returns true if no valid command received within 500ms. Called at 250 Hz by motor task for safety monitoring.

Safety feature: Prevents motors from running with outdated commands if communication link lost (USB CDC disconnect, ROS2 crash, RPi5 failure).

Figure 1361: Call graph for `shared_data_is_comm_timeout`

_Defined in `src/shared/shared\_data.c:2427_`

shared_data_update_last_comm_tick

```
void shared_data_update_last_comm_tick(void)
```

Update last communication timestamp.

Manually refreshes the communication watchdog timestamp. Normally updated automatically by `shared_data_set_motor_command()`, but this function allows explicit refresh if needed (e.g., heartbeat messages, non-motor commands).

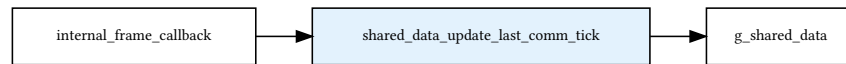


Figure 1362: Call graph for `shared_data_update_last_comm_tick`

_Defined in `src/shared/shared\_data.c:2499_`

shared_data_set_event

```
rx_err_t shared_data_set_event(shared_event_flags_t flags)
```

Set event flag(s).

Set event flags for inter-task signaling.

Raises one or more event flags to signal events to other tasks. Used for efficient inter-task communication without polling. Flags can be OR'd together.

Lock-free operation: Event flags use ThreadX atomic operations, safe to call from any context (tasks or ISRs) without mutex.

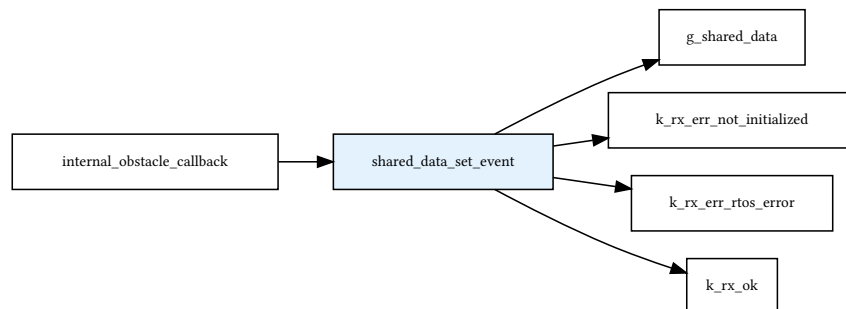


Figure 1363: Call graph for `shared_data_set_event`

_Defined in `src/shared/shared\_data.c:2582_`

shared_data_wait_event

```
rx_err_t shared_data_wait_event(shared_event_flags_t flags, uint32_t wait_option,
uint32_t *out_actual_flags)
```

Wait for event flag(s).

Wait for event flags with optional timeout.

Blocks until one or more event flags are set. Uses TX_OR_CLEAR mode to automatically clear flags after retrieval (consume events). Efficient alternative to polling.

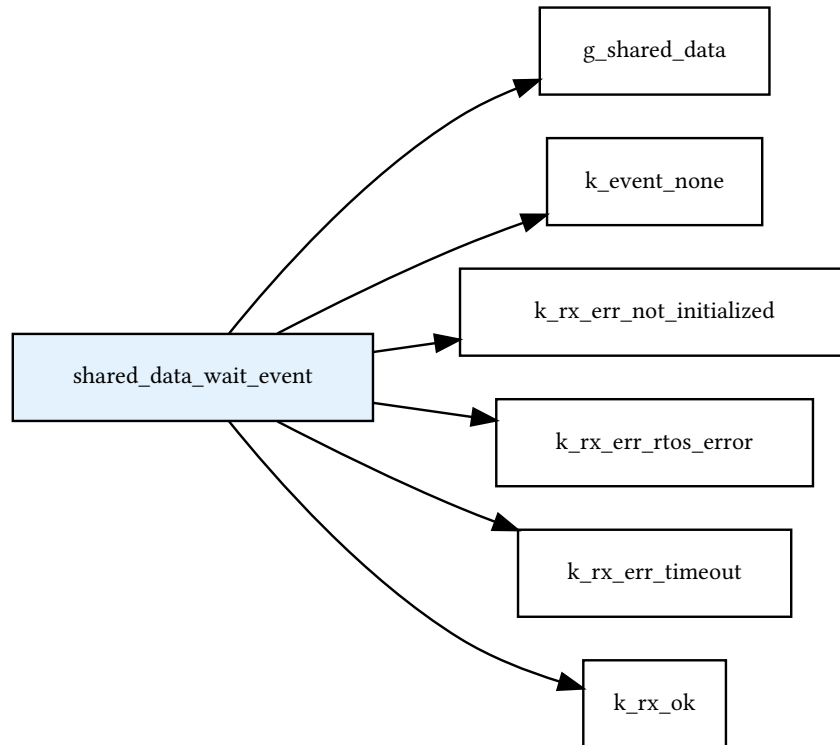


Figure 1364: Call graph for `shared_data_wait_event`

_Defined in `src/shared/shared\_data.c:2690_`

shared_data.h

Thread-Safe Shared Data Infrastructure for Multi-Task Communication.

Declares all shared data types and accessor functions for inter-task communication in the STAR firmware. All access is mutex-protected.

Includes:

- `<stdbool.h>`
- `<stdint.h>`
- `"rx_err.h"`
- `"tx_api.h"`

estop_reason_t

Emergency stop reason codes.

Value	Name	Description
= 0	k_estop_reason_none	No e-stop active
= 1	k_estop_reason_comm_timeout	Communication timeout
= 2	k_estop_reason_obstacle	Obstacle too close
= 3	k_estop_reason_driver_fault	Motor driver hardware fault
= 4	k_estop_reason_overcurrent	Motor overcurrent
= 5	k_estop_reason_manual	Manual request

motor_mode_t

Motor control mode.

Value	Name	Description
= 0	k_motor_mode_idle	Motors idle, no control active
= 1	k_motor_mode_velocity	Velocity control mode
= 2	k_motor_mode_estop	Emergency stop mode

shared_event_flags_t

Event flags for inter-task signaling.

One-hot bitmask constants used to signal asynchronous events between RTOS tasks via a ThreadX TX_EVENT_FLAGS_GROUP. Flags are OR-combined when raised and remain set (sticky) until explicitly cleared by the consuming task. Multiple flags may be set simultaneously; consumers should test each bit independently.

k_event_none == 0 (no-op / cleared state; safe as a default value)

All non-zero members are distinct powers of two (one-hot bit fields)

Value	Name	Description
= 0x00000000	k_event_none	No events pending (cleared state)
= 0x00000001	k_event_motor_command_updated	New motor command available
= 0x00000002	k_event_estop_triggered	E-stop activated
= 0x00000004	k_event_pid_gains_updated	PID gains changed
= 0x00000010	k_event_obstacle_detected	Obstacle detected
= 0x00000020	k_event_obstacle_cleared	Obstacle cleared
= 0x00000040	k_event_estop_cleared	E-stop cleared
= 0x00000080	k_event_comm_timeout	Communication timeout

See also: shared_data_set_event() Raises one or more flags, shared_data.c ThreadX event-flags group used internally

Example:


```
//Raise two flags at once:
(void)shared_data_set_event(k_event_comm_timeout|k_event_estop_triggered);

//Test for a specific flag in the consumer task:
shared_event_flags_t flags;
(void)tx_event_flags_get(&g_event_flags, k_event_comm_timeout,
TX_OR_CLEAR, (ULONG*)&flags, TX_WAIT_FOREVER);
```

Since Version 1.0.0

—

shared_data_constants_t

Shared data module timing constants.

Communication timing thresholds used by the shared data module for timeout detection and watchdog monitoring. Values fit in uint32_t because they represent millisecond durations that exceed uint16_t range at higher values.

k_shared_comm_timeout_ms > 0

Value	Name	Description
= 500	k_shared_comm_timeout_ms	Communication timeout threshold in milliseconds

See also: shared_data_is_comm_timeout() Communication timeout check,
shared_data_update_last_comm_tick() Update communication watchdog

Example:

```
if((current_tick-last_tick)*k_tick_period_ms>k_shared_comm_timeout_ms){
    shared_data_trigger_estop(k_estop_reason_comm_timeout);
}
```

Since Version 1.0.0

—

g_shared_data

shared_data_t g_shared_data

Global shared data instance (do not access directly!).

Warning: Direct access bypasses mutex protection and causes data races!] **See also:**
shared_data_init() Create mutexes and event flags, shared_data_t Structure
definition (in shared_data.h)

—

shared_data_init

```
rx_err_t shared_data_init(void)
```

Initialize shared data module.

Initialize shared data module.

Creates all ThreadX synchronization primitives and sets default values for shared state. Must be called exactly once from tx_application_define() before any tasks start.

Returns: rx_err_t Error code



Figure 1365: Call graph for `shared_data_init`

_Defined in `src/shared/shared_data.h:220_`

`shared_data_set_motor_command`

```
rx_err_t shared_data_set_motor_command(const motor_command_t *cmd)
```

Set motor velocity command.

Set motor velocity command.

Stores a new motor velocity command and updates the communication timestamp for timeout detection. Sets the `k_event_motor_command_updated` event flag to wake the motor control task.

Parameters:

Direction	Name	Description
in	cmd	Motor command

Returns: `rx_err_t` Error code

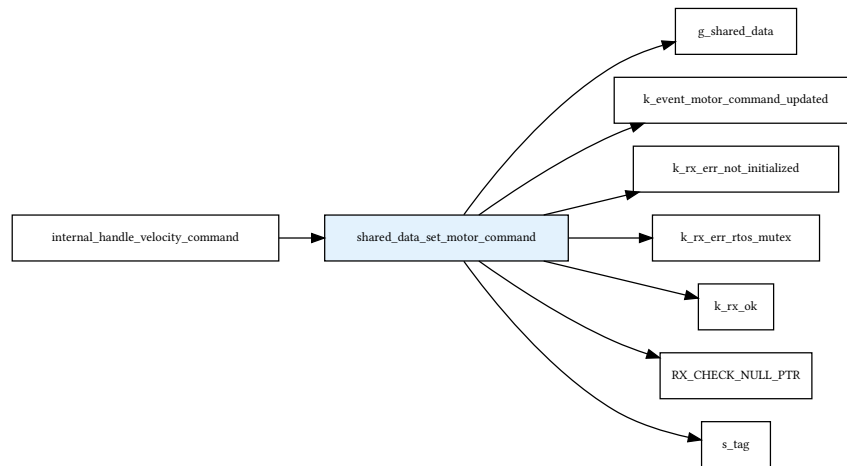


Figure 1366: Call graph for `shared_data_set_motor_command`

_Defined in `src/shared/shared\_data.h:227_`

shared_data_get_motor_command

```
rx_err_t shared_data_get_motor_command(motor_command_t *out_cmd)
```

Get motor velocity command.

Get motor velocity command.

Retrieves the latest motor velocity command for the motor control loop. Called at 250 Hz by motor task to fetch target velocities.

Parameters:

Direction	Name	Description
in	out_cmd	Output motor command

Returns: `rx_err_t` Error code

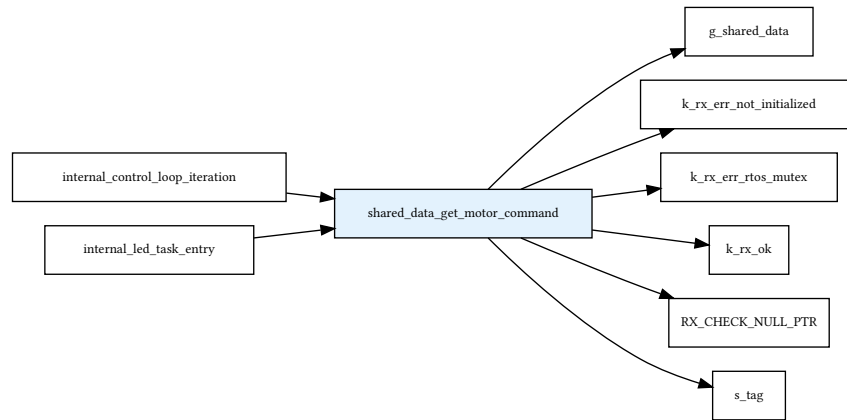


Figure 1367: Call graph for shared_data_get_motor_command

_Defined in src/shared/shared_data.h:234_

shared_data_update_motor_state

```
rx_err_t shared_data_update_motor_state(const motor_state_t *state)
```

Update motor state telemetry.

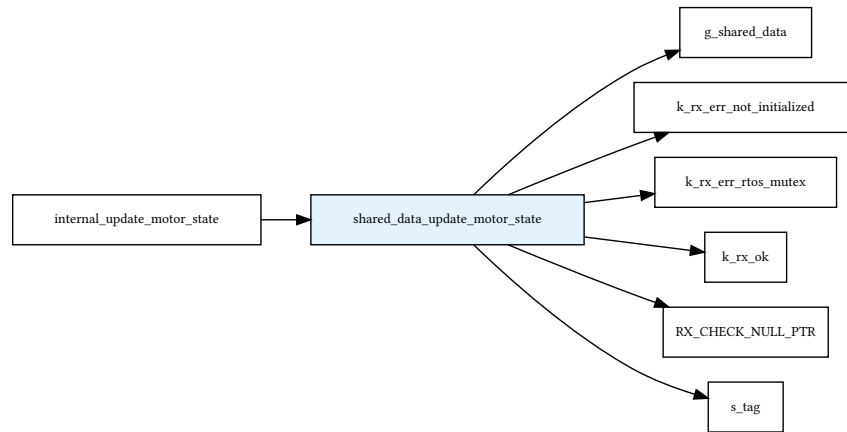
Update motor state telemetry.

Stores current motor telemetry (velocities, duty cycles, currents, faults) for the telemetry task to read. Called at 250 Hz after each control loop iteration.

Parameters:

Direction	Name	Description
in	state	Motor state

Returns: rx_err_t Error code

Figure 1368: Call graph for `shared_data_update_motor_state`

_Defined in `src/shared/shared\_data.h:241_`

shared_data_get_motor_state

```
rx_err_t shared_data_get_motor_state(motor_state_t *out_state)
```

Get motor state telemetry.

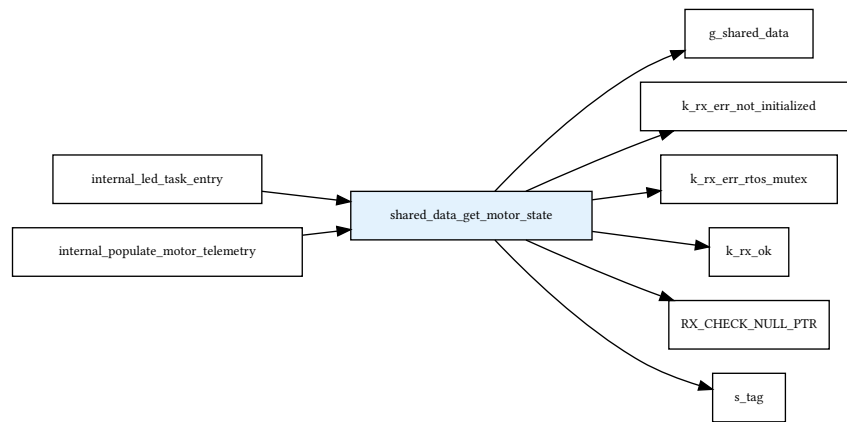
Get motor state telemetry.

Retrieves current motor telemetry for transmission to ROS2 gateway. Called at 20 Hz by telemetry task.

Parameters:

Direction	Name	Description
in	out_state	Output motor state

Returns: `rx_err_t` Error code

Figure 1369: Call graph for `shared_data_get_motor_state`

_Defined in `src/shared/shared\_data.h:248_`

`shared_data_set_pid_gains`

```
rx_err_t shared_data_set_pid_gains(const pid_gains_t *gains)
```

Set PID controller gains.

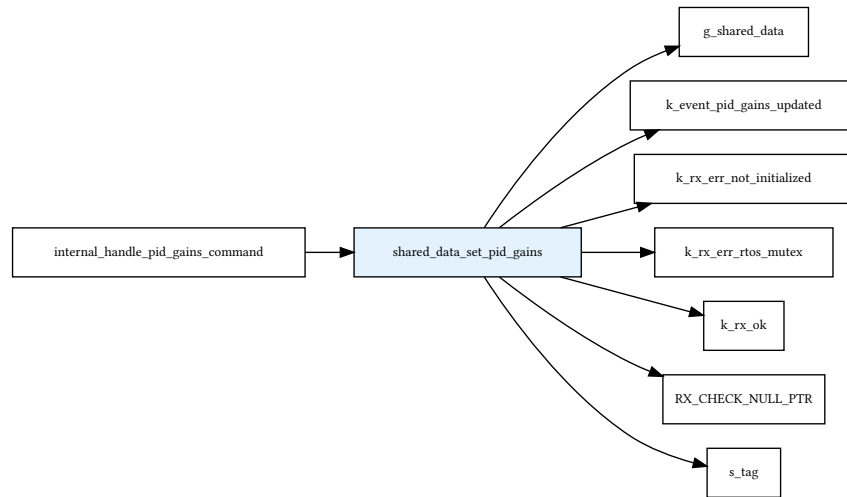
Set PID controller gains.

Updates PID controller gains at runtime for performance tuning. Sets the `update_pending` flag and signals `k_event_pid_gains_updated` event so motor task can apply new gains to all 4 PID controllers.

Parameters:

Direction	Name	Description
in	<code>gains</code>	PID gains

Returns: `rx_err_t` Error code

Figure 1370: Call graph for `shared_data_set_pid_gains`

_Defined in `src/shared/shared\_data.h:255_`

`shared_data_get_pid_gains`

```
rx_err_t shared_data_get_pid_gains(pid_gains_t *out_gains)
```

Get PID controller gains.

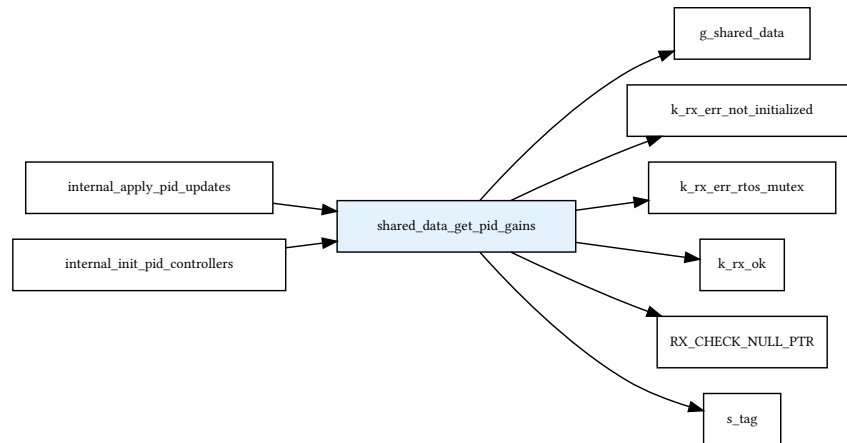
Get PID controller gains.

Retrieves current PID gains for motor control. Called when motor task needs to apply updated gains to PID controllers.

Parameters:

Direction	Name	Description
in	out_gains	Output PID gains

Returns: `rx_err_t` Error code

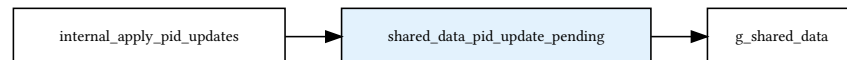
Figure 1371: Call graph for `shared_data_get_pid_gains`_Defined in `src/shared/shared_data.h:262_`**shared_data_pid_update_pending**

```
bool shared_data_pid_update_pending(void)
```

Check if PID update is pending.

Check if PID update is pending.

Returns whether new PID gains have been set but not yet applied to controllers. Motor task polls this at 250 Hz to detect when gains need updating.

Returns: true if update pendingFigure 1372: Call graph for `shared_data_pid_update_pending`_Defined in `src/shared/shared_data.h:268_`**shared_data_clear_pid_update_flag**

```
void shared_data_clear_pid_update_flag(void)
```

Clear PID update flag.

Clear PID update flag.

Clears the `update_pending` flag after motor task has applied new PID gains to all controllers. Prevents re-application of same gains.

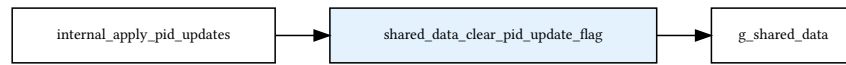


Figure 1373: Call graph for shared_data_clear_pid_update_flag

_Defined in src/shared/shared_data.h:273_

shared_data_trigger_estop

```
rx_err_t shared_data_trigger_estop(estop_reason_t reason)
```

Trigger emergency stop.

Activates emergency stop with specified reason code. Sets the estop_active flag and signals k_event_estop_triggered event to immediately halt all motors.

Can be called from any task when dangerous condition detected:

- Communication Task: timeout, manual request
- Obstacle Task: collision imminent
- Motor Task: overcurrent, driver fault

Parameters:

Direction	Name	Description
in	reason	E-stop reason code

Returns: rx_err_t Error code

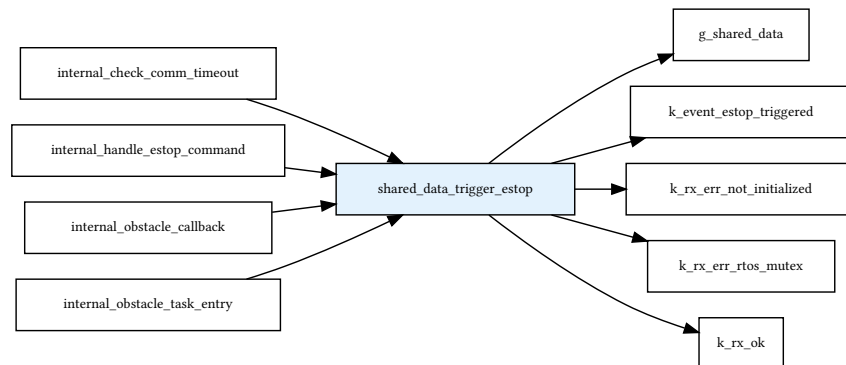


Figure 1374: Call graph for shared_data_trigger_estop

_Defined in src/shared/shared_data.h:280_

shared_data_trigger_estop_isr_safe

```
void shared_data_trigger_estop_isr_safe(estop_reason_t reason)
```

Trigger emergency stop from ISR context (ISR-safe, no blocking).

ISR-safe version of `shared_data_trigger_estop()` that DOES NOT block on mutex. Sets volatile flag `s_estop_pending_from_isr` and `s_pending_estop_reason`, then sets `k_event_estop_triggered` event flag. Motor control task commits pending ISR e-stop via `shared_data_commit_isr_estop()` within 4ms (250 Hz loop).

CRITICAL: This function MUST be used instead of `shared_data_trigger_estop()` when called from interrupt handlers (POEG ISRs).

Algorithm: Write `s_pending_estop_reason` first, then `s_estop_pending_from_isr` (ensures reason always valid when flag is set), then set event flag via `tx_event_flags_set(&g_shared_data.event_flags, k_event_estop_triggered, TX_OR)`.

ISR-safe version of `shared_data_trigger_estop()` that DOES NOT block on mutex. Sets a volatile flag and event flag only. Motor control task commits the pending e-stop to mutex-protected state via `shared_data_commit_isr_estop()`.

CRITICAL: This function is specifically designed for ISR context and MUST be used instead of `shared_data_trigger_estop()` when called from interrupt handlers.

Parameters:

Direction	Name	Description
in	reason	E-stop reason code (<code>driver_fault</code> , <code>overcurrent</code> , etc.)

Returns: void (no return value)

Value	Description
N/A	Always succeeds (void return, no error cases, side-effect only)

Pre: Called from ISR context only (POEG motor fault ISRs)

Pre: `shared_data_init()` completed and `g_shared_data.event_flags` initialized

Post: `s_estop_pending_from_isr == true` (volatile flag set)

Post: `s_pending_estop_reason == reason` (volatile reason stored)

Post: `k_event_estop_triggered` set via `tx_event_flags_set(&g_shared_data.event_flags, k_event_estop_triggered, TX_OR)`

Post: Motor task will commit within 4ms via `shared_data_commit_isr_estop()`

Note: ISR-safe: No blocking calls, no mutex usage (1 us execution)

Note: Thread Safety: Volatile writes are atomic on RX72N

Note: Multiple ISRs may race - last reason wins (acceptable for safety)

Warning: ONLY call from ISR context. For task context, use `shared_data_trigger_estop()`

Warning: Race condition: If multiple ISRs fire, last reason wins (acceptable)] **Example:**

```
//POEGmotorfaultISRexample
void __attribute__((interrupt)) poeg_group_a_isr(void){
icu()->ir[k_poeg_irq_group_a]=0;//ClearIRflag
rx_log_error("POEG","nFAULTmotor0");
shared_data_trigger_estop_isr_safe(k_estop_reason_driver_fault);//ISR-safe
}
```

See also: `shared_data_commit_isr_estop()` Commit function (motor task),
`shared_data_trigger_estop()` Task-context version (uses mutex),
`shared_data_is_estop_active()` Check if e-stop active

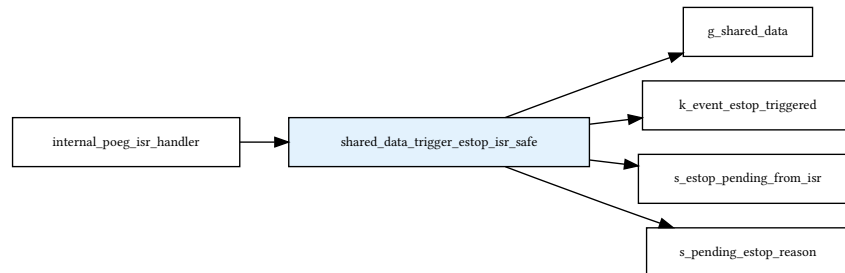


Figure 1375: Call graph for `shared_data_trigger_estop_isr_safe`

_Defined in `src/shared/shared_data.h:332`

Since Version 1.1.0

shared_data_commit_isr_estop

```
rx_err_t shared_data_commit_isr_estop(void)
```

Commit ISR-triggered e-stop to mutex-protected state (task context).

Transfers ISR-triggered e-stop from volatile flags (`s_estop_pending_from_isr`, `s_pending_estop_reason`) to mutex-protected shared state (`g_shared_data.estop_active`, `g_shared_data.estop_reason`). Called by motor control task at start of each 4ms iteration (250 Hz).

Why this is needed: ISRs cannot block on mutexes, so they set volatile flags. This function runs in task context where mutex acquisition is safe.

Algorithm: Uses ThreadX critical section (`tx_interrupt_control`) to atomically check-and-clear `s_estop_pending_from_isr` flag, preventing race with ISRs. If pending, acquires `estop_mutex` and commits to shared state.

Transfers ISR-triggered e-stop from volatile flags to mutex-protected shared state. Called by motor control task at start of each 4ms iteration (250 Hz).

Why this is needed: ISRs cannot block on mutexes, so they set a volatile flag. This function runs in task context where mutex acquisition is safe.

Parameters:

Direction	Name	Description
in	None	

Returns: rx_err_t Operation status

Value	Description
k_rx_ok	E-stop committed successfully or no pending e-stop
k_rx_err_not_initialized	Module not initialized
k_rx_err_rtos_mutex	Mutex acquisition failed

Pre: Module initialized via shared_data_init()

Pre: Called from task context only (motor control task, NOT ISR)

Post: If pending e-stop: s_estop_pending_from_isr cleared, g_shared_data.estop_active set, g_shared_data.estop_reason updated

Post: If no pending e-stop: State unchanged, estop_mutex not acquired

Post: estop_mutex released (if acquired) or state unchanged if none pending

Note: Thread Safety: Protected by estop_mutex and critical section

Note: Performance: 2.5 us when pending, 0.5 us when not pending

Note: Frequency: Called every 4ms by motor task (250 Hz)

Warning: ONLY call from task context (motor control task). Uses blocking mutex.

Warning: DO NOT call from ISR context (will cause deadlock/fault)] **Example:**

```
//Motorcontroltaskmainloop
while(true){
//CommitanyISR-triggerede-stopfirst
(void)shared_data_commit_isr_estop();

//Checke-stopstatus(willseecommittedISR-e-stop)
if(shared_data_is_estop_active()){
internal_active_brake_sequence();
```

```
tx_thread_sleep(1); //4ms
continue;
}

//Normalcontrolloop...
}
```

See also: `shared_data_trigger_estop_isr_safe()` ISR-safe e-stop trigger,
`shared_data_trigger_estop()` Task-context e-stop trigger, `shared_data_is_estop_active()`
 Check if e-stop active

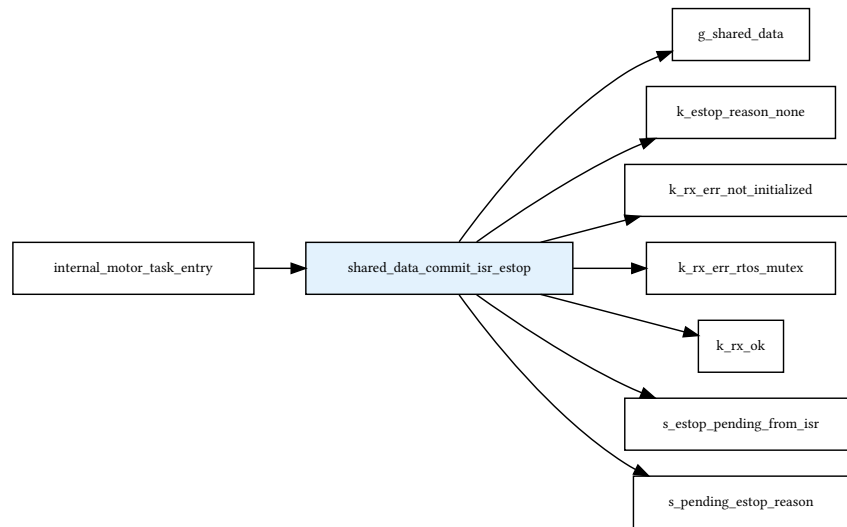


Figure 1376: Call graph for `shared_data_commit_isr_estop`

Defined in `src/shared/shared_data.h:393`

Since Version 1.1.0

—

shared_data_clear_estop

```
rx_err_t shared_data_clear_estop(void)
```

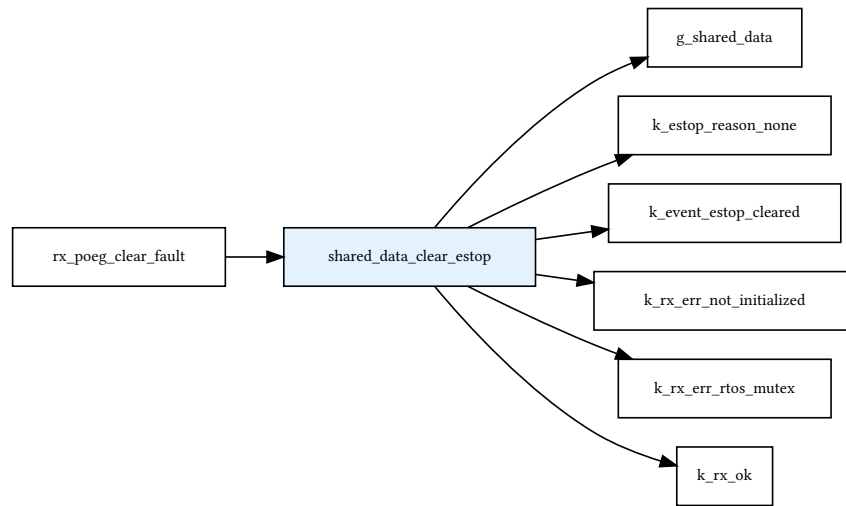
Clear emergency stop.

Clears the emergency stop flag after fault condition has been resolved. Allows system to resume normal operation. Sets `k_event_estop_cleared` event.

Should only be called after:

- Fault condition verified resolved
- System returned to safe state
- Manual operator approval received

Returns: `rx_err_t` Error code

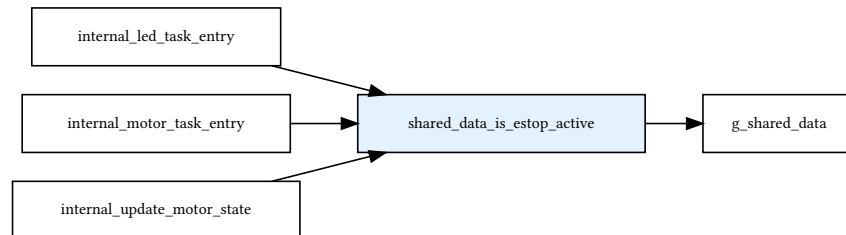
Figure 1377: Call graph for `shared_data_clear_estop`_Defined in `src/shared/shared_data.h:399_`**shared_data_is_estop_active**

```
bool shared_data_is_estop_active(void)
```

Check if emergency stop is active.

Returns current emergency stop status. Motor task checks this at 250 Hz to immediately halt outputs when e-stop triggered.

Returns: true if e-stop active

Figure 1378: Call graph for `shared_data_is_estop_active`_Defined in `src/shared/shared_data.h:405_`**shared_data_get_estop_reason**

```
estop_reason_t shared_data_get_estop_reason(void)
```

Get emergency stop reason code.

Get emergency stop reason code.

Returns the reason code for why emergency stop was triggered. Used for diagnostics, logging, and displaying fault information to operator.

Returns: `estop_reason_t` Reason code for current e-stop

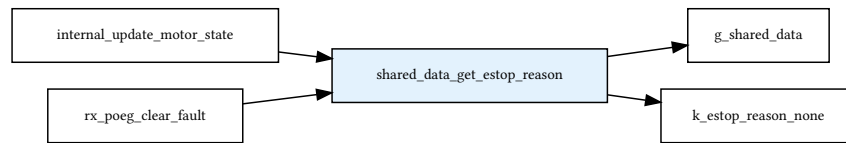


Figure 1379: Call graph for `shared_data_get_estop_reason`

_Defined in `src/shared/shared\_data.h:411_`

—

`shared_data_update_temp`

```
rx_err_t shared_data_update_temp(const temp_sensor_state_t *state)
```

Update temperature sensor state.

Update temperature sensor state.

Stores temperature readings from DS18B20 1-Wire sensors. Called at 1 Hz by temperature task. Used for ambient temperature compensation of HC-SR04.

Parameters:

Direction	Name	Description
in	state	Temperature state

Returns: `rx_err_t` Error code

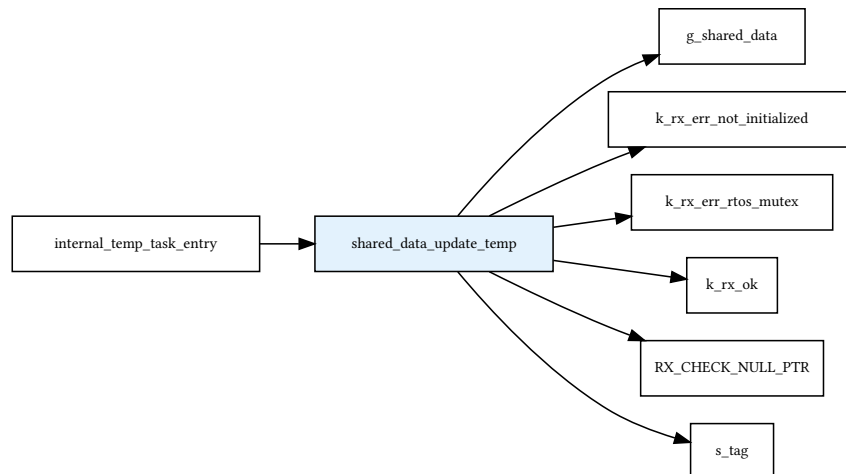


Figure 1380: Call graph for `shared_data_update_temp`

_Defined in src/shared/shared_data.h:419_

—

shared_data_get_temp

```
rx_err_t shared_data_get_temp(temp_sensor_state_t *out_state)
```

Get temperature sensor state.

Get temperature sensor state.

Retrieves temperature readings for telemetry reporting. Called at 20 Hz by telemetry task.

Parameters:

Direction	Name	Description
in	out_state	Output temperature state

Returns: rx_err_t Error code

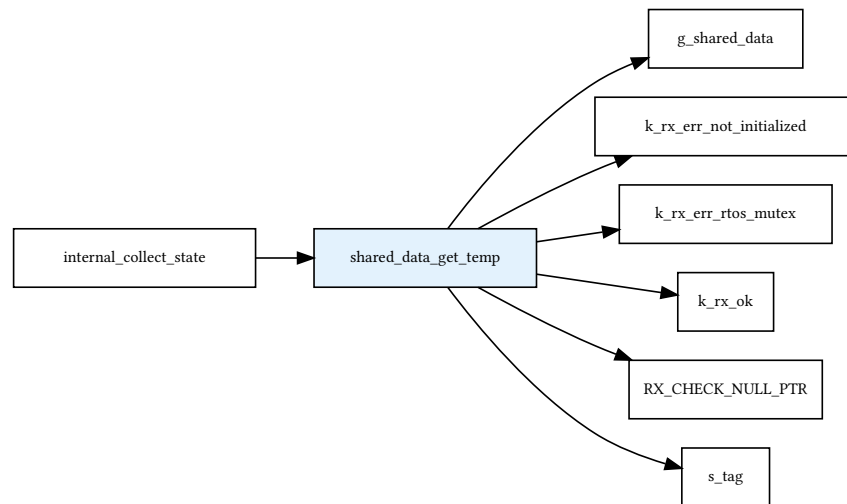


Figure 1381: Call graph for shared_data_get_temp

_Defined in src/shared/shared_data.h:426_

—

shared_data_update_obstacle

```
rx_err_t shared_data_update_obstacle(const obstacle_state_t *state)
```

Update obstacle detection state.

Update obstacle detection state.

Stores distance readings from HC-SR04 ultrasonic sensors. Called when new measurements available (typically 10-20 Hz). Automatically sets obstacle detected/cleared events based on distance thresholds.

Parameters:

Direction	Name	Description
in	state	Obstacle state

Returns: rx_err_t Error code

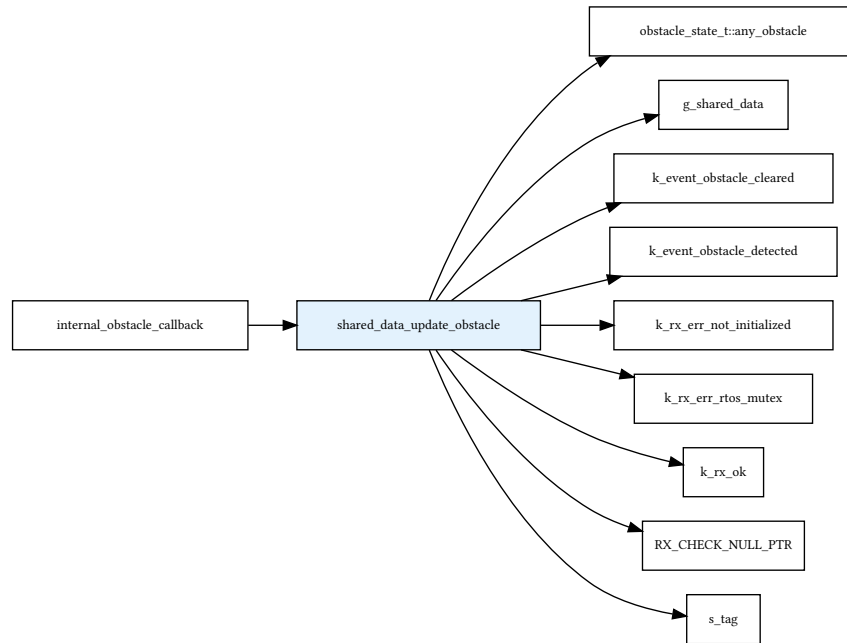


Figure 1382: Call graph for `shared_data_update_obstacle`

_Defined in `src/shared/shared\_data.h:433_`

shared_data_get_obstacle

```
rx_err_t shared_data_get_obstacle(obstacle_state_t *out_state)
```

Get obstacle detection state.

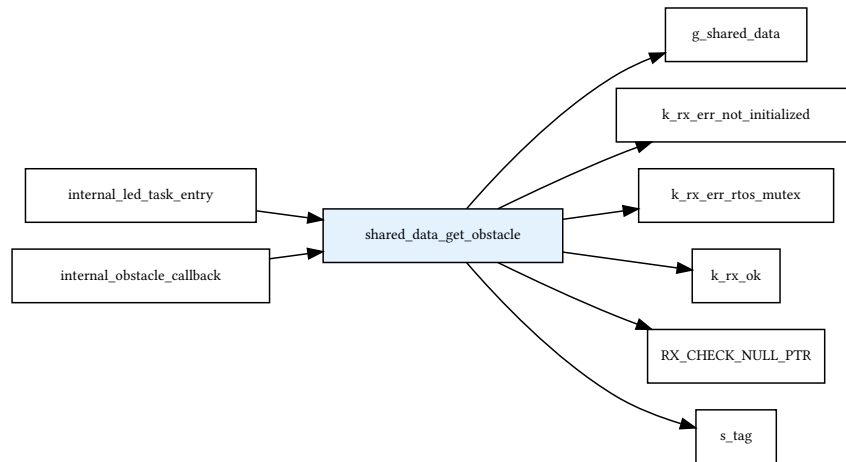
Get obstacle detection state.

Retrieves obstacle detection data for telemetry reporting. Called at 20 Hz by telemetry task.

Parameters:

Direction	Name	Description
in	out_state	Output obstacle state

Returns: rx_err_t Error code

Figure 1383: Call graph for `shared_data_get_obstacle`

Defined in `src/shared/shared\_data.h:440`

shared_data_is_comm_timeout

```
bool shared_data_is_comm_timeout(void)
```

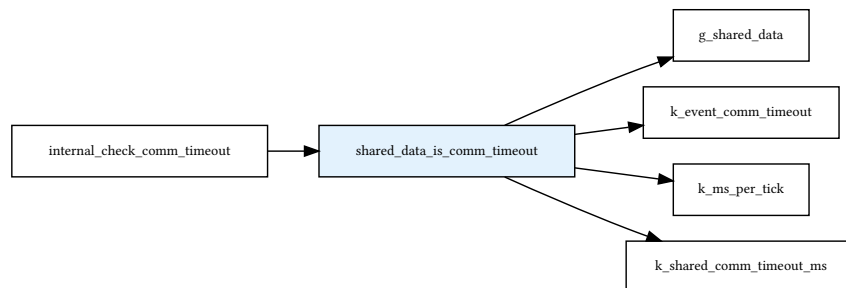
Check if communication timeout occurred.

Check if communication timeout occurred.

Determines if motor commands are stale by comparing current time to last command timestamp. Returns true if no valid command received within 500ms. Called at 250 Hz by motor task for safety monitoring.

Safety feature: Prevents motors from running with outdated commands if communication link lost (USB CDC disconnect, ROS2 crash, RPi5 failure).

Returns: true if timeout (>500ms since last command)

Figure 1384: Call graph for `shared_data_is_comm_timeout`

Defined in `src/shared/shared\_data.h:446`

shared_data_update_last_comm_tick

```
void shared_data_update_last_comm_tick(void)
```

Update last communication timestamp.

Manually refreshes the communication watchdog timestamp. Normally updated automatically by `shared_data_set_motor_command()`, but this function allows explicit refresh if needed (e.g., heartbeat messages, non-motor commands).

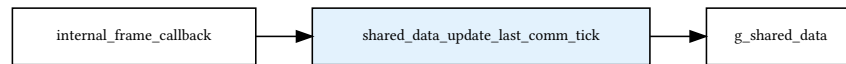


Figure 1385: Call graph for `shared_data_update_last_comm_tick`

_Defined in `src/shared/shared\_data.h:451_`

shared_data_set_event

```
rx_err_t shared_data_set_event(shared_event_flags_t flags)
```

Set event flags for inter-task signaling.

Set event flags for inter-task signaling.

Raises one or more event flags to signal events to other tasks. Used for efficient inter-task communication without polling. Flags can be OR'd together.

Lock-free operation: Event flags use ThreadX atomic operations, safe to call from any context (tasks or ISRs) without mutex.

Parameters:

Direction	Name	Description
in	flags	Event flags to set

Returns: `rx_err_t` Error code

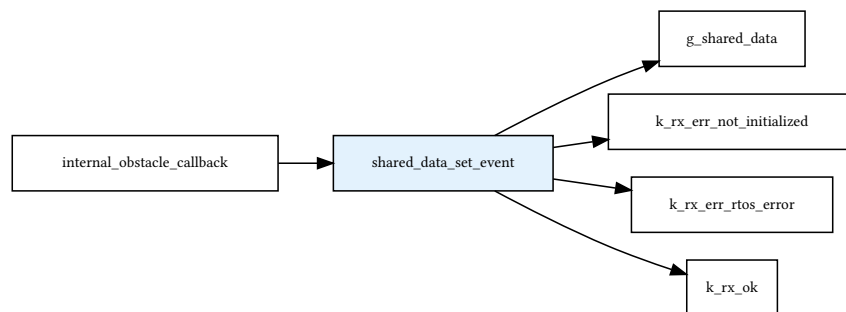


Figure 1386: Call graph for `shared_data_set_event`

_Defined in `src/shared/shared\_data.h:458_`

shared_data_wait_event

```
rx_err_t shared_data_wait_event(shared_event_flags_t flags, uint32_t wait_option,  
uint32_t *out_actual_flags)
```

Wait for event flags with optional timeout.

Wait for event flags with optional timeout.

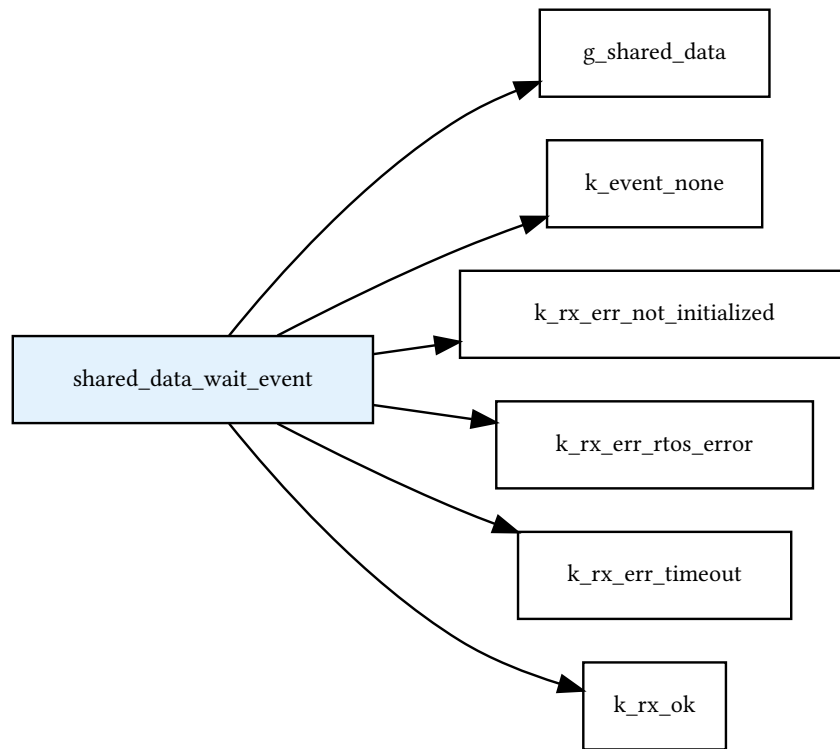
Blocks until one or more event flags are set. Uses TX_OR_CLEAR mode to automatically clear flags after retrieval (consume events). Efficient alternative to polling.

Parameters:

Direction	Name	Description
in	flags	Event flags to wait for (OR combination)
in	wait_option	ThreadX wait option (TX_WAIT_FOREVER, tick count, TX_NO_WAIT)
out	out_actual_flags	Actual flags that were set (may be NULL)

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Flags received
k_rx_err_timeout	Wait timed out (TX_NO_EVENTS)
k_rx_err_not_initialized	Shared data not initialized
k_rx_err_rtos_error	ThreadX error

Figure 1387: Call graph for `shared_data_wait_event`

_Defined in `src/shared/shared\_data.h:471_`

—

comm_task.c

Communication Task Implementation - HIGHEST PRIORITY Protocol Handling for RPi5 Interface.

Includes:

- `"comm_task.h"`
- `<string.h>`
- `"rx_check.h"`
- `"rx_comm_manager.h"`
- `"rx_frame.h"`
- `"rx_iwtdt.h"`
- `"rx_log.h"`
- `"rx_nanopb.h"`
- `"rx_spi_comm.h"`
- `"rx_time_constants.h"`
- `"rx_usb_comm.h"`
- `"shared_data.h"`
- `"tx_api.h"`

comm_task_constants_t

Communication task configuration constants.

Value	Name	Description
= 2048	k_comm_task_stack_size	Stack size in bytes
= 5	k_comm_task_priority	ThreadX priority (highest app priority)
= 1	k_comm_task_sleep_ticks	Sleep period (10ms = 100 Hz)
= 0	k_comm_task_input	Thread entry input parameter

—

comm_motor_constants_t

Motor index constants.

Value	Name	Description
= 0	k_motor_idx_front_left	Front left motor index
= 1	k_motor_idx_front_right	Front right motor index
= 2	k_motor_idx_back_left	Back left motor index
= 3	k_motor_idx_back_right	Back right motor index

—

retransmit_retries_limits_t

Valid range limits for max_retries in SetRetransmitConfigRequest.

Bounds used to validate the max_retries field before narrowing from uint32_t to uint8_t. Prevents undefined behaviour on out-of-range protobuf values.

k_retransmit_max_retries_min <= k_retransmit_max_retries_max

Value	Name	Description
= 1	k_retransmit_max_retries_min	Minimum allowed max_retries value
= 10	k_retransmit_max_retries_max	Maximum allowed max_retries value

See also: retransmit_timing_limits_t Companion enum for timing field limits

Since Version 1.0.0

—

retransmit_timing_limits_t

Valid range limits for timing fields in SetRetransmitConfigRequest.

Bounds used to validate ack_timeout_ms and max_backoff_ms before narrowing from uint32_t to uint16_t. Prevents undefined behaviour on out-of-range protobuf values.

k_retransmit_ack_timeout_min_ms <= k_retransmit_ack_timeout_max_ms

k_retransmit_backoff_min_ms <= k_retransmit_backoff_max_ms

Value	Name	Description
-------	------	-------------

= 10	k_retransmit_ack_timeout_min_ms	Minimum ACK timeout (ms)
= 1000	k_retransmit_ack_timeout_max_ms	Maximum ACK timeout (ms)
= 50	k_retransmit_backoff_min_ms	Minimum backoff delay (ms)
= 5000	k_retransmit_backoff_max_ms	Maximum backoff delay (ms)

See also: `retransmit_retries_limits_t` Companion enum for retry count limits

Since Version 1.0.0

—

s_comm_thread

TX_THREAD `s_comm_thread`

ThreadX thread control block.

—

s_comm_stack

`uint8_t` `s_comm_stack[k_comm_task_stack_size]`

Static thread stack (no dynamic allocation).

—

s_comm_created

`bool` `s_comm_created`

Task creation guard flag.

—

g_comm_manager

`rx_comm_manager_t` `g_comm_manager`

Communication manager handle (global for `telemetry_task` access).

Note: Declared extern - definition is in `comm_task.c`

Note: Shared across tasks (`comm_task`, `telemetry_task`, `motor_control_task`)

Warning: Do NOT call `rx_comm_manager_send()` before `comm_task_create()` completes] **See also:** `rx_comm_manager.h` Communication manager API, `comm_task.c` Initializes and manages `g_comm_manager`

Since Version 1.0.0

—

s_tag

`const char*` `const` `s_tag`

Log tag for this module.

—

s_session_state`rx_session_state_t s_session_state`

Shared session state for USB and SPI transports.

Note: File-scoped static - not accessible outside comm_task.c

Warning: Do not modify directly - managed by rx_usb_comm and rx_spi_comm layers

Since Version 1.0.0

—

s_usb_comm_handle`rx_usb_comm_handle_t s_usb_comm_handle`

USB CDC communication layer handle.

Note: File-scoped static - not accessible outside comm_task.c

Warning: Do not access internal fields directly - use rx_usb_comm API functions] **See also:**
rx_usb_comm_init() Initialization function

Since Version 1.0.0

—

s_spi_comm_handle`rx_spi_comm_handle_t s_spi_comm_handle`

SPI communication layer handle (RSPI2 peripheral mode).

Note: File-scoped static - not accessible outside comm_task.c

Warning: Do not access internal fields directly - use rx_spi_comm API functions] **See also:**
rx_spi_comm_init() Initialization function

Since Version 1.0.0

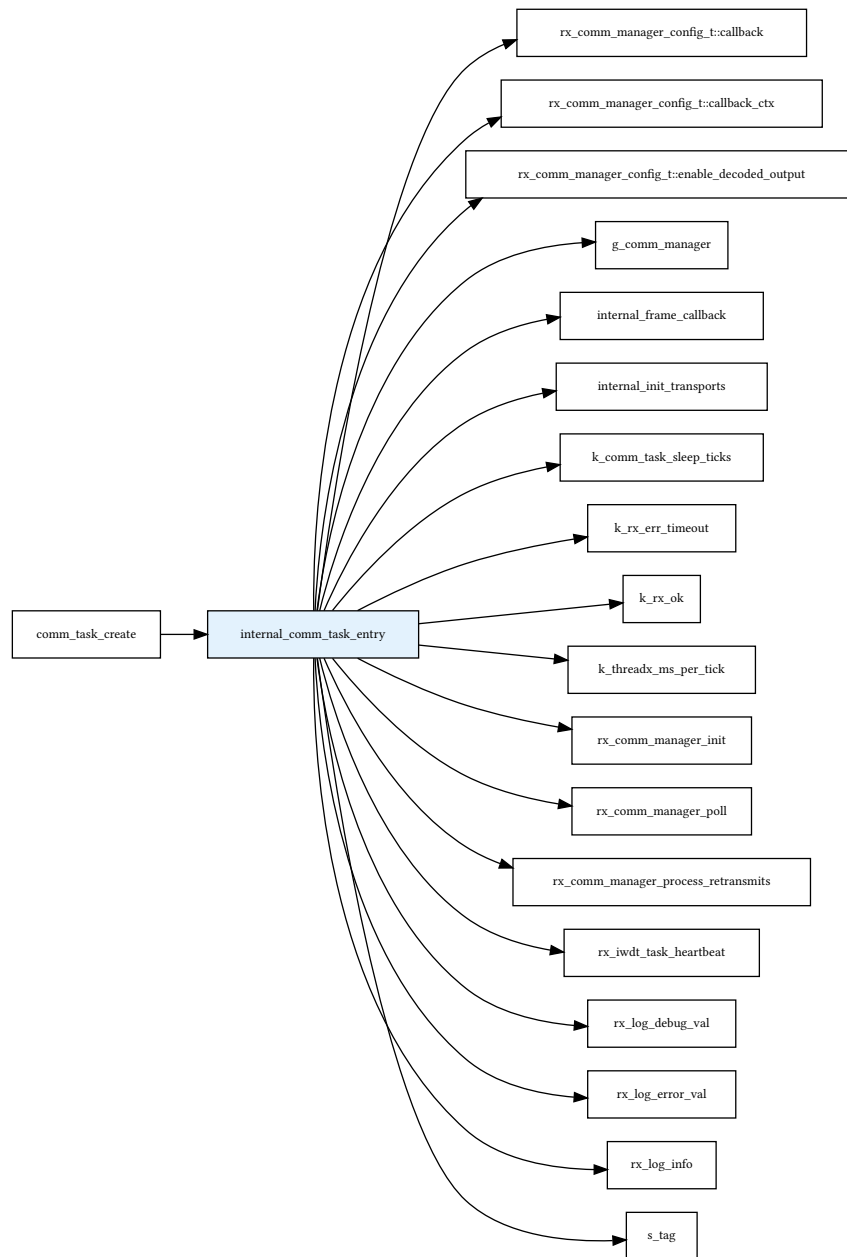
—

internal_comm_task_entry

```
static void internal_comm_task_entry(ULONG input)
```

Communication task entry point - infinite loop polling rx_comm_manager at 100 Hz.

This is the main task loop that executes after comm_task_create() starts the thread. The function never returns and runs continuously at 100 Hz polling rate until power-down or emergency stop. It is the HIGHEST PRIORITY application task (Priority 5) to minimize command-to-motor latency.

Figure 1388: Call graph for `internal_comm_task_entry`

_Defined in `src/tasks/comm\_task.c:1267_`

internal_init_transports

```
static void internal_init_transports(rx_comm_manager_config_t *config)
```

Initialize USB and SPI transport layers and wire into comm manager config.

Performs complete initialization of USB CDC and SPI communication transport layers, including handle initialization, error handling, and validation logging. This function populates the `rx_comm_manager_config_t` structure with either real transport handles (on success) or nullptr (on failure) to enable graceful degradation.

Algorithm Steps:

1. Initialize USB transport: Call `rx_usb_comm_init()` with shared session state
2. Handle USB failure: Log error if init fails, set handle to nullptr
3. Initialize SPI transport: Call `rx_spi_comm_init()` with shared session state
4. Handle SPI failure: Log error if init fails, set handle to nullptr
5. Wire handles: Assign valid or nullptr handles to config structure
6. Validate transports: Log critical error if both transports failed
7. Log success: Log info for each successfully initialized transport

Graceful Degradation:

- If one transport fails: Other transport continues (logged warning)
- If both transports fail: Config wired with nullptr handles (logged critical error)
- Comm manager will return `k_rx_err_timeout` on poll for nullptr handles

Parameters:

Direction	Name	Description
out	config	Comm manager configuration to populate Valid range: Non-nullptr to <code>rx_comm_manager_config_t</code> Constraints: Must be zeroed before calling this function Side effects: <code>usb_handle</code> and <code>spi_handle</code> fields populated

Returns: void This function does not return a value

Pre: config must be non-NULL and zero-initialized (memset)

Pre: s_session_state must be zero-initialized (static storage)

Pre: RSP12 peripheral must be initialized before calling this function

Post: config->usb_handle set to &s_usb_comm_handle or nullptr

Post: config->spi_handle set to &s_spi_comm_handle or nullptr

Post: At least one transport handle set on success (best effort)

Post: All init failures logged via `rx_log_error`

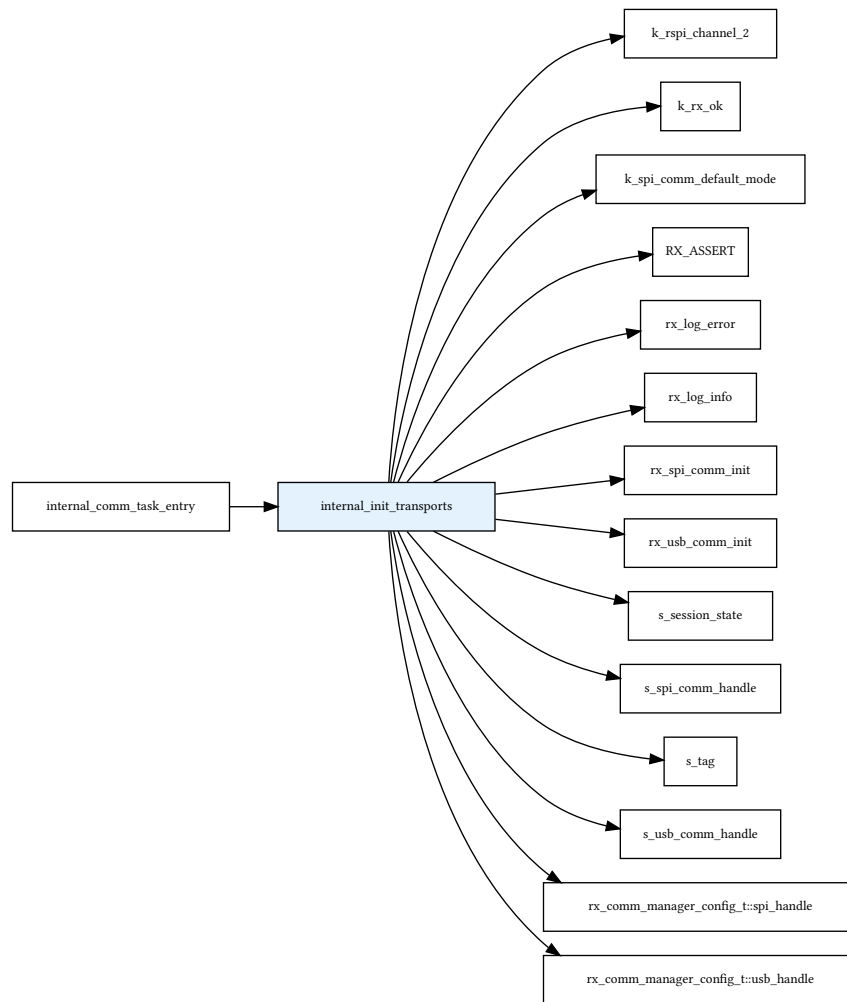
Note: This function does NOT initialize RSPI hardware - caller must ensure RSPi2 peripheral is initialized before calling

Note: Called once during single-threaded task initialization; not thread-safe

Warning: Function has no return value - check config->usb_handle and config->spi_handle for nullptr to detect complete failure

Thread Safety:: This function is not thread-safe. It must be called only once during task initialization in a single-threaded context. The function modifies file-scoped static variables (s_usb_comm_handle, s_spi_comm_handle) and should not be called concurrently from multiple threads.

See also: rx_usb_comm_init() USB transport layer initialization, rx_spi_comm_init() SPI transport layer initialization

Figure 1389: Call graph for `internal_init_transports`

Defined in `src/tasks/comm\_task.c:929`

Since Version 1.0.0

—

internal_frame_callback

```
static void internal_frame_callback(rx_comm_channel_t channel, const rx_frame_t
*frame, void *ctx)
```

Frame received callback - dispatches frames to appropriate handlers.

This callback is invoked by `rx_comm_manager` when a complete valid frame has been received and validated (CRC-32 passed, header parsed). The function runs in the Communication Task context (Priority 5), NOT in ISR context.

Critical Safety Feature: Updates communication watchdog timestamp on EVERY valid frame reception (regardless of type) to prevent communication timeout emergency stop.



Figure 1390: Call graph for `internal_frame_callback`

_Defined in `src/tasks/comm/_task.c:1613_`

internal_handle_command_frame

```
static rx_err_t internal_handle_command_frame(const rx_frame_t *frame)
```

Handle command frame by delegating to per-message-type helper functions.

Dispatches a COMMAND-type frame to the appropriate per-message handler using a try-each-in-order cascade. Each helper attempts to decode the frame as its specific protobuf message type and returns true if it handled the message, false otherwise. The first handler that returns true wins; if none match the frame is an unknown type.

Decode Priority (most frequent first):

1. SetVelocityRequest (100 Hz)
2. EmergencyStopRequest (rare, on demand)
3. SetPIDGainsRequest (very rare, config only)
4. SetRetransmitConfigRequest (very rare, config only)
5. Unknown (log warning, return error)

Function executes in Communication Task context (Priority 5), not ISR

Unknown message types do NOT crash firmware (log warning, return error)

Parameters:

Direction	Name	Description
in	frame	Frame containing the command payload Type: const rx_frame_t* Must NOT be NULL frame->header.type must be k_frame_type_command frame->payload contains nanopb-encoded protobuf message frame->header.length is protobuf message size in bytes

Returns: rx_err_t Processing status

Value	Description
k_rx_ok	Message decoded and handled by one of the helpers
k_rx_err_null_ptr	frame is nullptr
k_rx_err_invalid_arg	No helper recognised the payload (unknown message type)

Pre: frame must not be nullptr

Pre: frame->header.type == k_frame_type_command (enforced by caller)

Post: If k_rx_ok: shared_data updated (motor command, e-stop, PID gains, or retransmit cfg)

Post: If k_rx_err_invalid_arg: warning logged, no shared_data changes

Note: Not thread-safe; single-threaded callback invocation guaranteed by rx_comm_manager

NASA Power of 10 Compliance:: Rule 4: [PASS] Function body is 12 lines (well under 60 LOC target) Rule 5: [PASS] 2 preconditions, 2 postconditions documented

See also: internal_handle_velocity_command() SetVelocityRequest handler,
internal_handle_estop_command() EmergencyStopRequest handler,
internal_handle_pid_gains_command() SetPIDGainsRequest handler,
internal_handle_retransmit_config_command() SetRetransmitConfigRequest handler

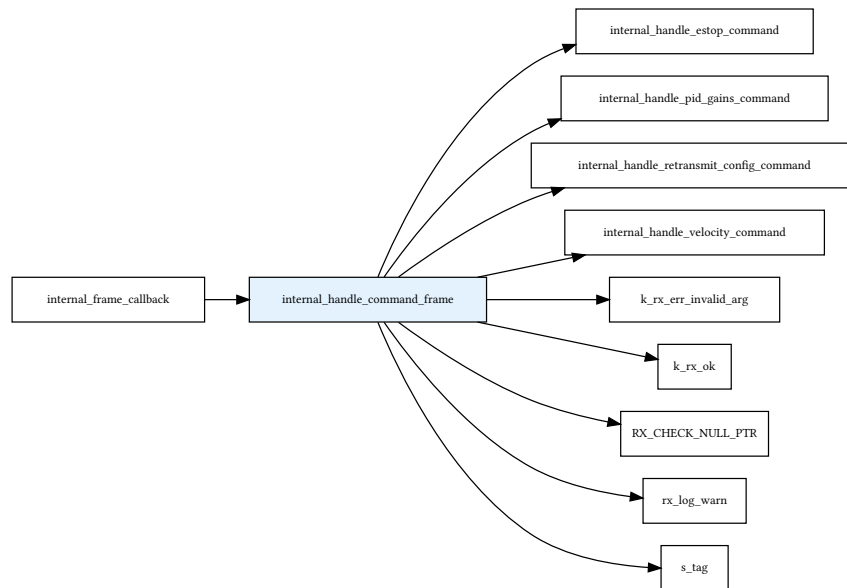


Figure 1391: Call graph for internal_handle_command_frame

_Defined in src/tasks/comm_task.c:1733_

Since Version 1.0.0

—

internal_handle_velocity_command

```
static bool internal_handle_velocity_command(const rx_frame_t *frame)
```

Attempt to decode and handle a SetVelocityRequest from a command frame.

Tries to decode the frame payload as a star_v1_SetVelocityRequest protobuf message. If decoding succeeds and the required command sub-message is present, converts the four motor velocities (double -> float) into a motor_command_t and writes it to shared_data for consumption by the Motor Control Task.

Algorithm:

1. Attempt nanopb decode via rx_nanopb_decode_velocity_request()
2. Verify has_command flag is set (sub-message present)
3. Build motor_command_t: copy 4 velocities, sequence number, timestamp, valid=true
4. Write to shared_data via shared_data_set_motor_command()
5. Log result and return true

Parameters:

Direction	Name	Description
in	frame	COMMAND frame to decode Must NOT be nullptr (caller guarantees) frame->payload and frame->header.length are used for decode

Returns: bool Whether this handler recognised and consumed the frame

Value	Description
true	Frame decoded as SetVelocityRequest; motor command written to shared_data
false	Decode failed; frame is not a SetVelocityRequest (try next handler)

Pre: frame must not be nullptr

Pre: shared_data_init() must have been called

Post: On true: shared_data motor command updated with new velocities

Post: On true: k_event_new_motor_cmd event set (wakes Motor Control Task)

Note: Not thread-safe; executes in Communication Task context (Priority 5)

Note: double -> float velocity conversion: +/-0.0001 m/s precision loss acceptable

NASA Power of 10 Compliance:: Rule 4: [PASS] Function body is under 30 lines Rule 5: [PASS] 2 preconditions, 2 postconditions documented Rule 7: [PASS] All return values checked

See also: rx_nanopb_decode_velocity_request() nanopb decode function,
shared_data_set_motor_command() Thread-safe motor command write,
internal_handle_command_frame() Caller (cascade dispatcher)

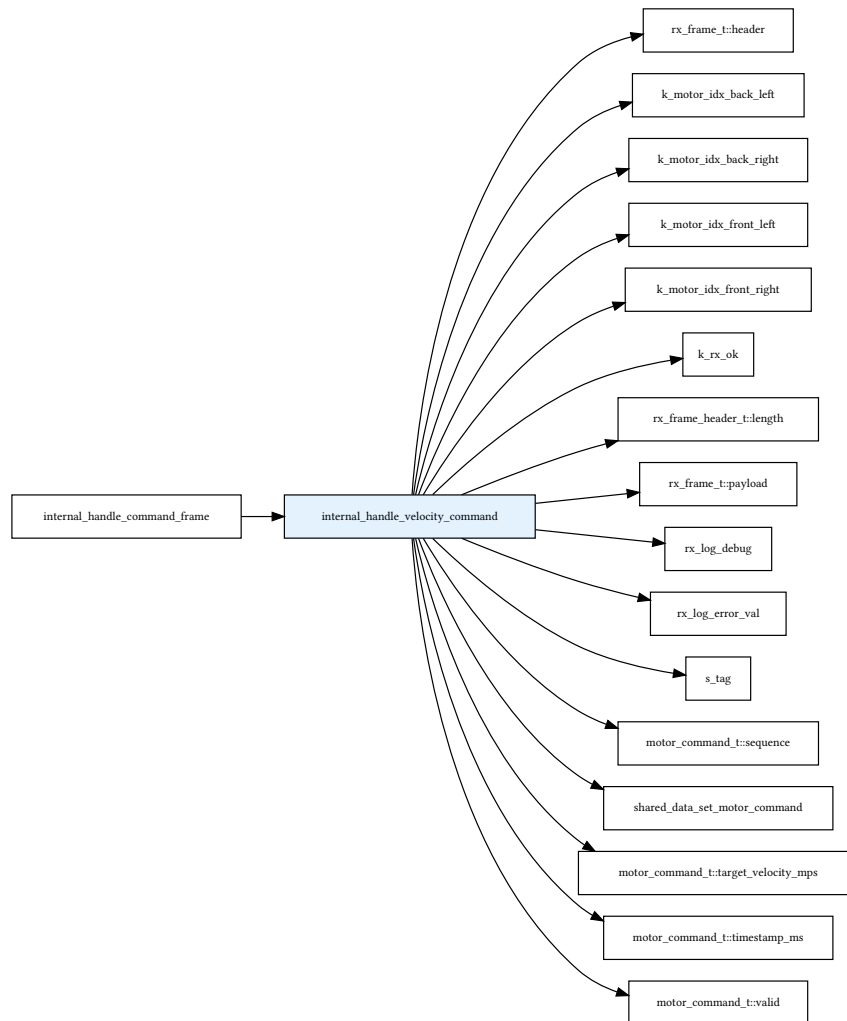


Figure 1392: Call graph for internal_handle_velocity_command

_Defined in `src/tasks/comm\_task.c:1788_`

Since Version 1.0.0

internal_handle_estop_command

```
static bool internal_handle_estop_command(const rx_frame_t *frame)
```

Attempt to decode and handle an EmergencyStopRequest from a command frame.

Tries to decode the frame payload as a `star_v1_EmergencyStopRequest` protobuf message. If decoding succeeds, triggers an emergency stop via `shared_data` with reason `k_estop_reason_manual`. This immediately disables all motor outputs in the Motor Control Task.

Algorithm:

1. Attempt nanopb decode via rx_nanopb_decode_estop_request()
2. Log warning ("E-Stop request received")
3. Call shared_data_trigger_estop(k_estop_reason_manual)
4. Log any error and return true

Parameters:

Direction	Name	Description
in	frame	COMMAND frame to decode Must NOT be nullptr (caller guarantees) frame->payload and frame->header.length are used for decode

Returns: bool Whether this handler recognised and consumed the frame

Value	Description
true	Frame decoded as EmergencyStopRequest; emergency stop triggered
false	Decode failed; frame is not an EmergencyStopRequest (try next handler)

Pre: frame must not be nullptr

Pre: shared_data_init() must have been called

Post: On true: estop_active == true in shared_data

Post: On true: k_event_estop_triggered event set (wakes Motor Control Task)

Note: Not thread-safe; executes in Communication Task context (Priority 5)

Warning: E-stop overrides any pending velocity commands; motors disabled immediately

NASA Power of 10 Compliance:: Rule 4: [PASS] Function body is under 20 lines Rule 5: [PASS] 2 preconditions, 2 postconditions documented Rule 7: [PASS] All return values checked

See also: rx_nanopb_decode_estop_request() nanopb decode function,
shared_data_trigger_estop() Thread-safe emergency stop trigger,
internal_handle_command_frame() Caller (cascade dispatcher)

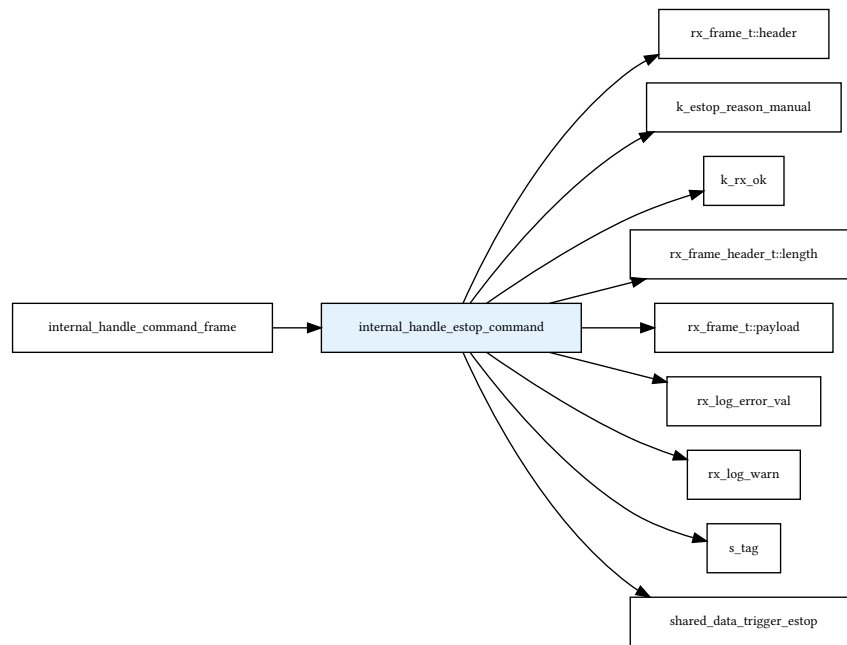


Figure 1393: Call graph for internal_handle_estop_command

_Defined in src/tasks/comm_task.c:1861_

Since Version 1.0.0

—

internal_handle_pid_gains_command

```
static bool internal_handle_pid_gains_command(const rx_frame_t *frame)
```

Attempt to decode and handle a SetPIDGainsRequest from a command frame.

Tries to decode the frame payload as a star_v1_SetPIDGainsRequest protobuf message. If decoding succeeds and the required pid_config sub-message is present, converts all gain fields (double -> float) into a pid_gains_t structure and writes it to shared_data. The Motor Control Task applies the gains on the next control cycle.

Algorithm:

1. Attempt nanopb decode via rx_nanopb_decode_pid_gains_request()
2. Verify has_pid_config flag is set (sub-message present)
3. Build pid_gains_t: convert kp, ki, kd, limits, set update_pending=true
4. Write to shared_data via shared_data_set_pid_gains()
5. Log result and return true

Parameters:

Direction	Name	Description
-----------	------	-------------

in	frame	COMMAND frame to decode Must NOT be nullptr (caller guarantees) frame->payload and frame->header.length are used for decode
----	-------	--

Returns: bool Whether this handler recognised and consumed the frame

Value	Description
true	Frame decoded as SetPIDGainsRequest; PID gains written to shared_data
false	Decode failed or has_pid_config not set (try next handler)

Pre: frame must not be nullptr

Pre: shared_data_init() must have been called

Post: On true: shared_data PID gains updated with new values

Post: On true: k_event_pid_gains_updated event set (Motor Control Task will apply)

Note: Not thread-safe; executes in Communication Task context (Priority 5)

Note: double -> float gain conversion: precision loss negligible for PID tuning

NASA Power of 10 Compliance:: Rule 4: [PASS] Function body is under 30 lines Rule 5: [PASS] 2 preconditions, 2 postconditions documented Rule 7: [PASS] All return values checked

See also: rx_nanopb_decode_pid_gains_request() nanopb decode function,
shared_data_set_pid_gains() Thread-safe PID gains write,
internal_handle_command_frame() Caller (cascade dispatcher)

Figure 1394: Call graph for `internal_handle_pid_gains_command`_Defined in `src/tasks/comm\_task.c:1922_`*Since Version 1.0.0***internal_handle_retransmit_config_command**

```
static bool internal_handle_retransmit_config_command(const rx_frame_t *frame)
```

Attempt to decode and handle a `SetRetransmitConfigRequest` from a command frame.

Tries to decode the frame payload as a `star_v1_SetRetransmitConfigRequest` protobuf message. If decoding succeeds and the required `retransmit_config` sub-message is present, validates all fields against safe bounds before narrowing to their firmware types, then calls `rx_comm_manager_set_auto_retransmit()` to update the HARQ retransmission parameters.

Algorithm:

1. Attempt nanopb decode via `rx_nanopb_decode_retransmit_config_request()`
2. Verify `has_retransmit_config` flag is set (sub-message present)
3. Extract fields into `const uint32_t` locals for bounds checking
4. Validate each field against typed enum limits; log error and return false on violation
5. Build `rx_spi_comm_retransmit_config_t` with safe narrowing casts
6. Call `rx_comm_manager_set_auto_retransmit()`
7. Log result and return true

Bounds validated:

- `max_retries`: [1, 10]
- `ack_timeout_ms`: [10, 1000]
- `max_backoff_ms`: [50, 5000]

Parameters:

Direction	Name	Description
in	<code>frame</code>	COMMAND frame to decode Must NOT be nullptr (caller guarantees) <code>frame->payload</code> and <code>frame->header.length</code> are used for decode

Returns: bool Whether this handler recognised and consumed the frame

Value	Description
true	Frame decoded as <code>SetRetransmitConfigRequest</code> ; retransmit config updated
false	Decode failed, <code>has_retransmit_config</code> not set, or fields out of range

Pre: frame must not be nullptr

Pre: `g_comm_manager` must be initialised (comm manager init called)

Post: On true: `rx_comm_manager` HARQ retransmit parameters updated

Post: On true: retransmit config is within safe validated bounds

Note: Not thread-safe; executes in Communication Task context (Priority 5)

Warning: Out-of-range protobuf values are rejected (logged error, returns false)

NASA Power of 10 Compliance: Rule 4: [PASS] Function body is under 40 lines Rule 5: [PASS] 2 preconditions, 2 postconditions documented Rule 7: [PASS] All return values checked

See also: rx_nanopb_decode_retransmit_config_request() nanopb decode function,
 rx_comm_manager_set_auto_retransmit() Update HARQ retransmission config,
 retransmit_retries_limits_t Bounds for max_retries, retransmit_timing_limits_t Bounds
 for timing fields, internal_handle_command_frame() Caller (cascade dispatcher)

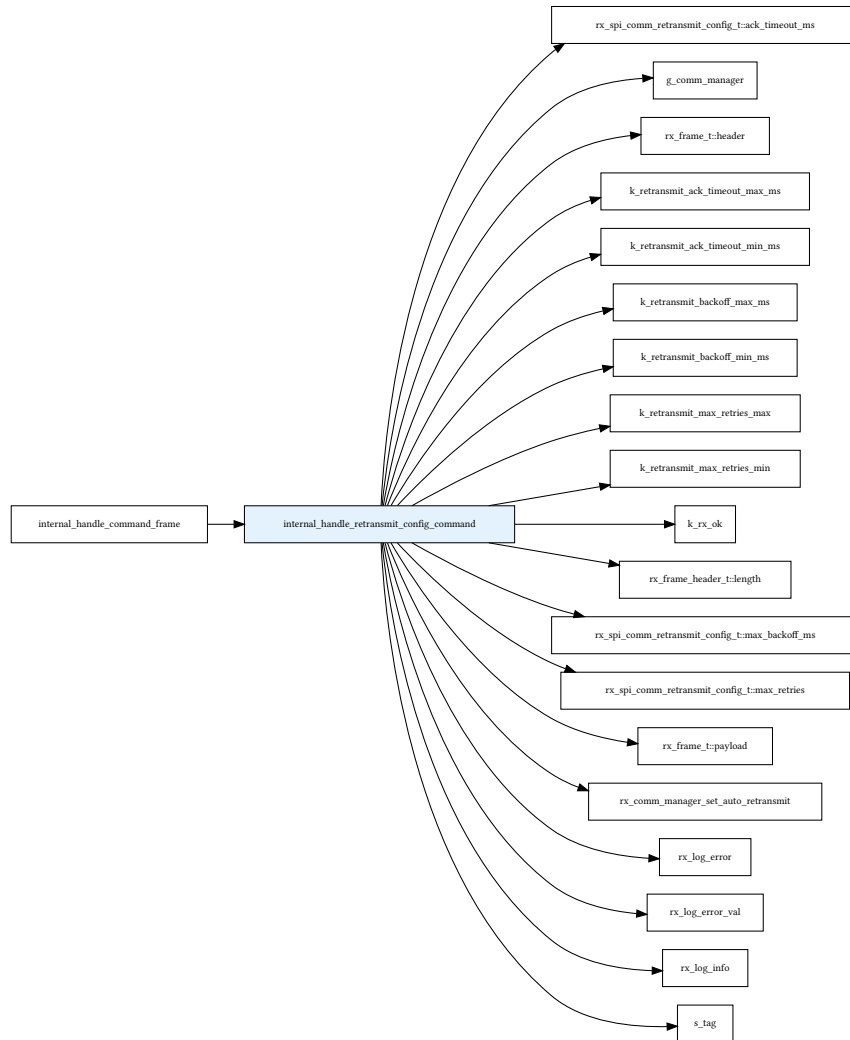


Figure 1395: Call graph for `internal_handle_retransmit_config_command`

_Defined in `src/tasks/comm\_task.c:2005_`

Since Version 1.0.0

`comm_task_create`

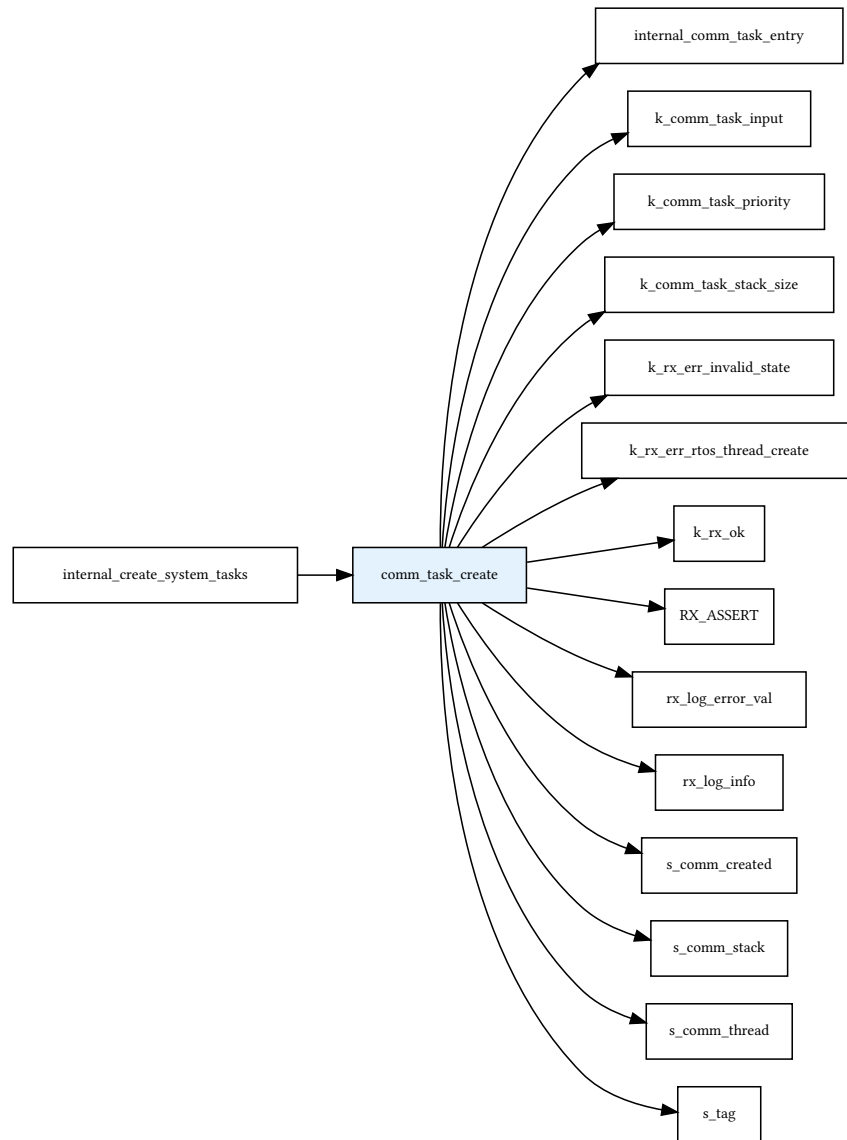
```
rx_err_t comm_task_create(void)
```

Create the Communication task (HIGHEST PRIORITY application task).

Create and start the communication task.

Creates and starts the Communication ThreadX task with HIGHEST application priority (Priority 5) for low-latency protocol command processing at 100 Hz. This function allocates the thread control block, assigns a static stack, and registers the task with ThreadX.

Critical Design Decision: Priority 5 ensures this task preempts ALL other application tasks (motor control, telemetry, sensors) to minimize command-to-motor latency. Only system-level tasks (ThreadX scheduler, ISRs) have higher priority.

Figure 1396: Call graph for `comm_task_create`

_Defined in `src/tasks/comm\_task.c:839_`

comm_task.h

Communication Task - SPI Protocol Handler for Raspberry Pi 5.

Declares the communication task responsible for handling Protocol Buffers communication over SPI with the Raspberry Pi 5 main controller.

Responsibilities:

- Receive command frames from RPi5 via SPI
- Parse and validate Protocol Buffer messages (nanopb)
- Dispatch commands to appropriate handlers (motor control, telemetry requests)
- Send response frames with telemetry data
- Handle communication errors and timeouts

Communication Protocol:

- Physical: SPI peripheral (RX72N) <- SPI controller (RPi5)
- Data link: Custom frame protocol with CRC-32
- Application: Protocol Buffers (nanopb) messages
- Commands: MotorCommand, EmergencyStop, TelemetryRequest
- Responses: MotorStatus, SensorData

Copyright (c) 2026 STAR Project

Includes:

- "rx_err.h"

comm_task_create

```
rx_err_t comm_task_create(void)
```

Create and start the communication task.

Creates the CommTask ThreadX task for SPI protocol handling. This task receives commands from the Raspberry Pi 5 and dispatches them to the appropriate system components.

Task Configuration:

- Priority: 8 (high priority - responsive to RPi5 commands)
- Stack: 3072 bytes (Protocol Buffers decoding requires stack space)
- Trigger: Event-driven (SPI RX interrupt)
- Auto-start: Enabled

Create and start the communication task.

Creates and starts the Communication ThreadX task with HIGHEST application priority (Priority 5) for low-latency protocol command processing at 100 Hz. This function allocates the thread control block, assigns a static stack, and registers the task with ThreadX.

Critical Design Decision: Priority 5 ensures this task preempts ALL other application tasks (motor control, telemetry, sensors) to minimize command-to-motor latency. Only system-level tasks (ThreadX scheduler, ISRs) have higher priority.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_*	ThreadX task creation failed

Pre: ThreadX kernel running

Pre: SPI peripheral initialized

Pre: Protocol Buffers decoder initialized

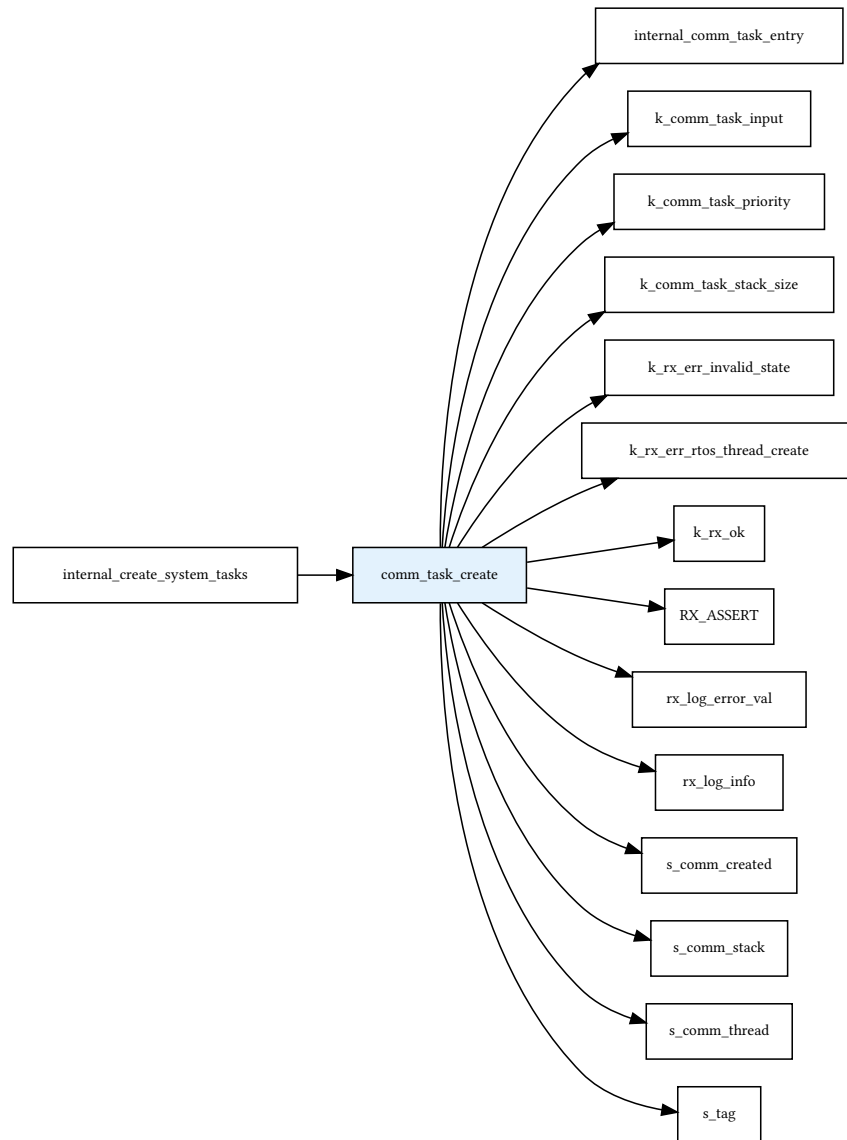
Pre: Shared data initialized

Post: CommTask created and waiting for SPI frames

Post: Commands from RPi5 will be processed

Note: Call from tx_application_define() in main.c after SPI initialization

Note: Task blocks on event flag - woken by SPI RX interrupt] **See also:** comm_task.c
Implementation details, main.c Task creation in tx_application_define()

Figure 1397: Call graph for `comm_task_create`

_Defined in `src/tasks/inc/comm\_task.h:68_`

Since *STAR v1.0.0*

—

led_status_task.h

LED Status Indicator Task - Visual System Health Feedback.

Declares the LED status task responsible for driving 6 status LEDs based on system state read from `shared_data`. Provides visual feedback for heartbeat, errors, motor status, communication activity, obstacle detection, and boot/debug indicators.

LED Assignments:

Copyright (c) 2026 STAR Project. MIT License.

Includes:

- "rx_err.h"

led_status_task_create

```
rx_err_t led_status_task_create(void)
```

Create and start the LED status indicator task.

Creates the LEDTask ThreadX task for visual system health feedback. This task reads system state from `shared_data` and drives 6 status LEDs at 20 Hz to provide real-time visual indication of system health.

Task Configuration:

- Priority: 17 (low priority - visual feedback only)
- Stack: 512 bytes (minimal - GPIO writes only)
- Period: 50 ms (20 Hz LED update rate)
- Auto-start: Enabled

Create and start the LED status indicator task.

Creates and starts the LED ThreadX task with low priority (17) for visual system health feedback at 20 Hz. GPIO pins are initialized as outputs inside the task entry to keep creation lightweight.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_invalid_state	Task already created
k_rx_err_rtos_thread_create	ThreadX task creation failed
k_rx_ok	Task created successfully
k_rx_err_invalid_state	Already created
k_rx_err_rtos_thread_create	ThreadX error

Pre: ThreadX kernel running

Pre: shared_data initialized

Pre: LED GPIO pins configured as outputs (hardware_init)

Pre: ThreadX kernel running

Pre: `shared_data_init()` called

Post: LEDTask created and running at 20 Hz

Post: 6 status LEDs actively driven based on system state

Post: LEDTask created and running at 20 Hz

Post: `s_led_created` set to true

Note: Call from `tx_application_define` after `shared_data_init`

Note: Low priority - visual feedback only, never preempts control tasks

Note: Thread Safety: Not thread-safe, call from `tx_application_define` only] **See also:** `led_status_task.c` Implementation details, `tx_application_define()` Task creation coordinator

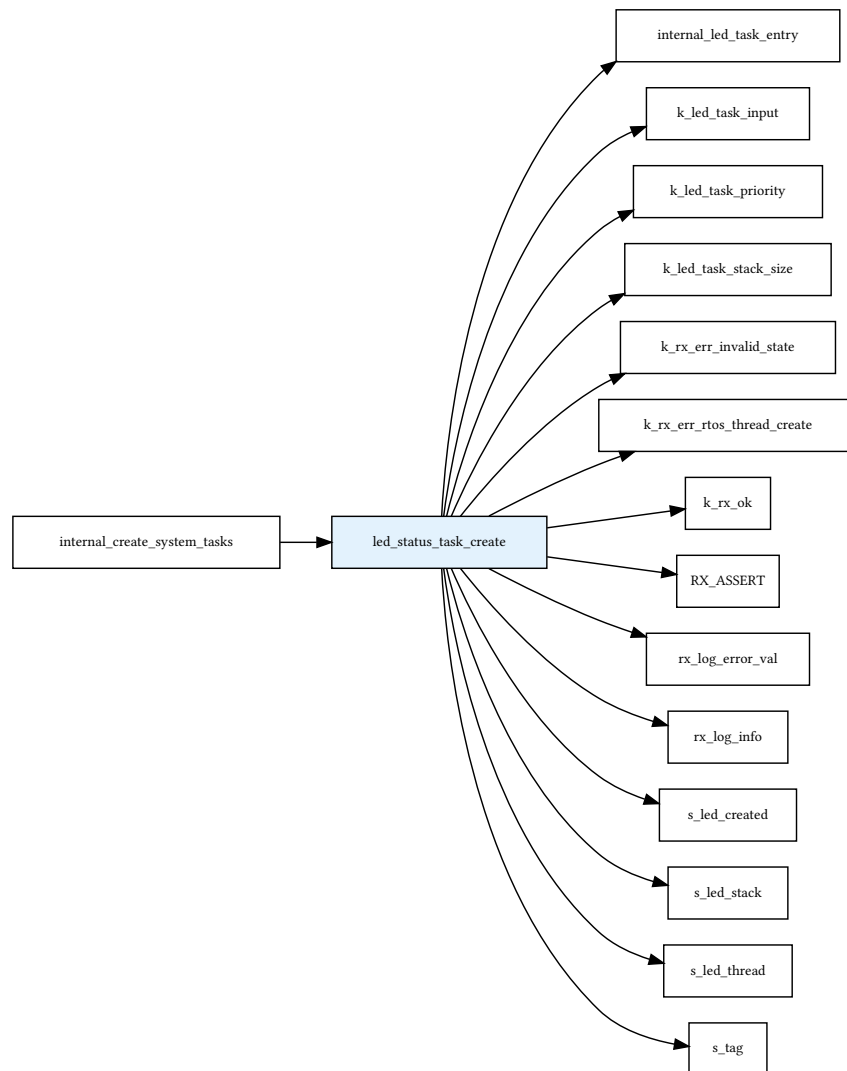


Figure 1398: Call graph for `led_status_task_create`

Defined in `src/tasks/inc/led_status_task.h:67`

Since Version 1.0.0

—

motor_control_task.h

Motor Control Task - 250 Hz PID-Based Velocity Control.

Declares the motor control task responsible for closed-loop velocity control of four brushed DC gearmotors using PID controllers.

Responsibilities:

- Run PID control loop at 250 Hz (4 ms period)

- Read encoder feedback from MTU quadrature inputs
- Compute PID output for each motor
- Command DRV8263H drivers via PWM (IN2/IN1 mode)
- Monitor motor faults (overcurrent, stall detection)
- Handle emergency stop conditions

Control Architecture:

Copyright (c) 2026 STAR Project

Includes:

- "rx_err.h"
- "rx_motor.h"

motor_count_t

Motor count sentinels for motor_control_task_get_motors().

Named constants for motor count return values. Used to avoid magic number 0 in callers checking whether motors are ready. Callers should compare the out_count value returned by motor_control_task_get_motors() against k_motor_count_none to determine whether motors are available.

k_motor_count_none == 0 (sentinel value, never a valid motor count)

Value	Name	Description
= 0	k_motor_count_none	Zero motors available motor_control_task_create() has not yet been called

See also: motor_control_task_get_motors() Returns k_motor_count_none when not ready

Example:

```
uint8_t motor_count=k_motor_count_none;
rx_motor_handle_t**motors= motor_control_task_get_motors(&motor_count);
if(motors==nullptr||motor_count==k_motor_count_none){
//Motortasknotyetcreated--motorsnotavailable
}
```

Since STAR v1.0.0

—

motor_control_task_create

```
rx_err_t motor_control_task_create(void)
```

Create and start the motor control task.

Creates the MotorTask ThreadX task for high-frequency motor control. This is the highest-priority application task to ensure deterministic control loop timing.

Task Configuration:

- Priority: 6 (highest application priority after AppMain)
- Stack: 3072 bytes (PID computations require stack)
- Period: 4 ms (250 Hz control loop)
- Auto-start: Enabled

Create and start the motor control task.

Creates and starts the Motor Control ThreadX task with high priority (Priority 8) for real-time 100 Hz PID velocity control of 4 DC gearmotors. This is the most critical task in the firmware - all robot motion control depends on this task executing correctly within its 10ms timing budget.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_*	ThreadX task creation failed

Pre: ThreadX kernel running

Pre: Motor drivers (DRV8263H PWM) initialized

Pre: Encoders (MTU) configured

Pre: PID controllers tuned

Pre: Shared data initialized

Post: MotorTask created and running at 250 Hz

Post: Motors ready to respond to velocity commands

Note: NOT thread-safe. Must be called from single-threaded init context (tx_application_define() in main.c) after motor hardware initialization

Note: Highest priority task - runs before all other application tasks] **See also:** motor_control_task.c Implementation details, main.c Task creation in tx_application_define()

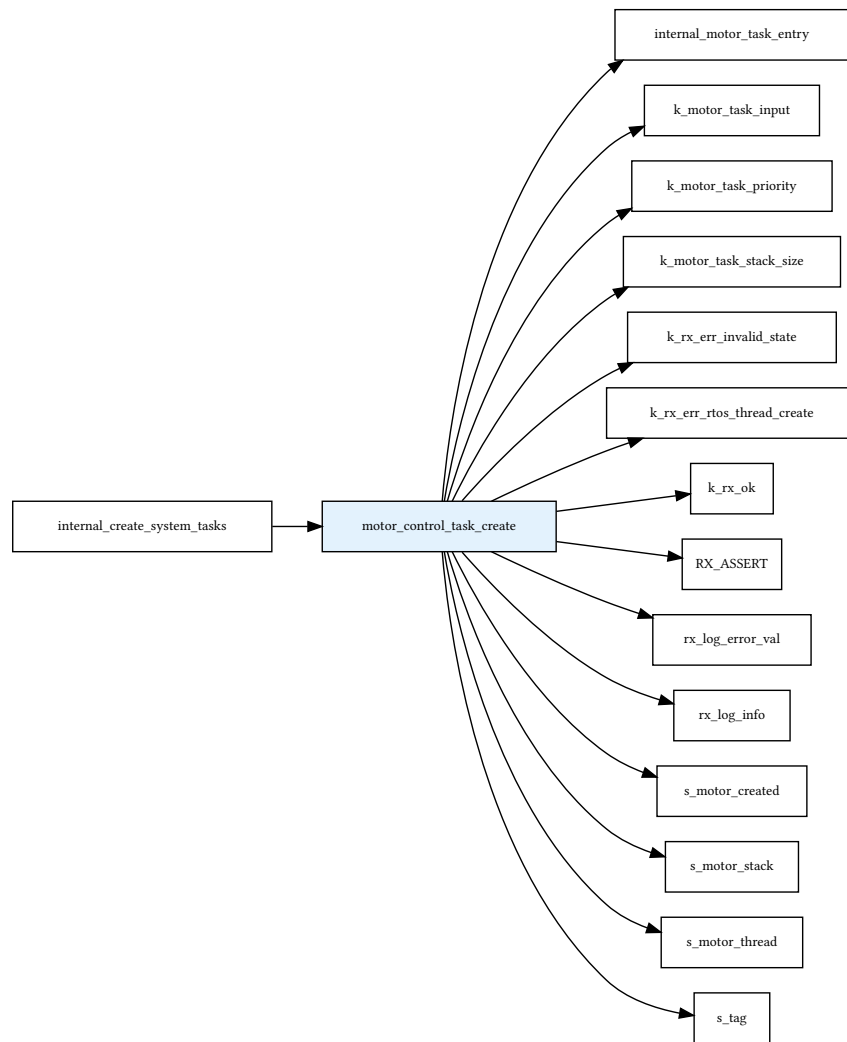


Figure 1399: Call graph for `motor_control_task_create`

Defined in `src/tasks/inc/motor_control_task.h:105`

Since STAR v1.0.0

motor_control_task_get_motors

```
rx_motor_handle_t ** motor_control_task_get_motors(uint8_t *out_count)
```

Get motor handle array for obstacle detection emergency stop.

Returns pointers to the 4 initialized motor handles for use by the obstacle detection system. These handles allow the obstacle detection task to execute emergency motor stops when obstacles are detected.

Usage Pattern:

Motor Array:

- Index 0: Front-left motor
- Index 1: Front-right motor
- Index 2: Rear-left motor
- Index 3: Rear-right motor

Returns pointers to the 4 initialized motor handles. The obstacle detection system uses these handles to execute emergency motor stops when obstacles breach the safety perimeter.

Three possible outcomes:

1. Motor stack initialized + out_count non-null -> returns s_motor_ptrs, sets *out_count = k_motor_count
2. Motor stack not yet initialized (s_motor_stack_initialized == false) -> returns nullptr, sets *out_count = k_motor_count_none
3. out_count is nullptr -> returns nullptr immediately, out_count unchanged

Guard condition (s_motor_stack_initialized vs s_motor_created): s_motor_created is set immediately after tx_thread_create() before the thread has executed. s_motor_stack_initialized is set only when internal_init_motor_stack() returns k_rx_ok inside the thread. This function checks s_motor_stack_initialized to ensure handles are fully initialized before returning them.

Parameters:

Direction	Name	Description
out	out_count	Pointer to receive motor count Set to k_motor_count (4) on success, or k_motor_count_none (0) on failure Must be non-NULL
out	out_count	Pointer to receive motor count. Set to k_motor_count (4) when motor stack is initialized; set to k_motor_count_none (0) when not yet initialized. Must be non-NULL (nullptr returns nullptr without writing).

Returns: rx_motor_handle_t** Pointer to array of motor handle pointers

Value	Description
Non-null	pointer to static array of k_motor_count (4) rx_motor_handle_t*, with out_count set to k_motor_count returned when motors are initialized and out_count is non-NULL
nullptr	with out_count set to k_motor_count_none (0) returned when motor_control_task_create() has not yet been called (motors not ready)
nullptr	(out_count unchanged) returned when out_count argument is nullptr
s_motor_ptrs	Valid pointer to static array of k_motor_count (4) rx_motor_handle_t*, *out_count set to k_motor_count motor stack fully initialized
nullptr	with *out_count = k_motor_count_none internal_init_motor_stack() has not yet completed successfully
nullptr	(out_count unchanged) out_count argument was nullptr

Pre: motor_control_task_create() has been called successfully

Pre: out_count != nullptr (NULL pointer returns nullptr immediately)

Pre: motor_control_task_create() called (otherwise s_motor_stack_initialized is never set)

Pre: out_count != nullptr (nullptr out_count causes immediate nullptr return)

Post: out_count set to k_motor_count (4) on success

Post: Returns valid pointer to motor handle array on success

Post: *out_count == k_motor_count when return value is non-null

Post: *out_count == k_motor_count_none when motor stack not yet initialized

Note: Thread-safe: Returns pointer to static memory

Note: Lifetime: Motor handles valid until program termination

Note: Thread-safe: Returns pointer to static memory (no shared mutable state)

Note: Lifetime: Returned pointer valid until program termination (static allocation)

Note: Returns nullptr gracefully if called before motor stack init completes (no assert)

Warning: Do NOT call before motor_control_task initialization

Warning: Do NOT modify returned motor handles directly

NASA Power of 10 Compliance:: Rule 9 Exception: Returns rx_motor_handle_t** (double indirection) Standard Rule 9: Maximum one level of pointer dereferencing Justification: Enables zero-copy access to motor handles for emergency stop Safety: Returns pointer to static array (no dynamic allocation, lifetime guaranteed) Alternative rejected: Copy-out API would require 128 bytes stack per call (4 handles x 32 bytes) Usage pattern: Similar to argv in main(int argc, char** argv) - read-only array access Verification: Static analysis confirms single dereference in obstacle_detect_task.c

Example:

```
uint8_t motor_count=0;
rx_motor_handle_t** motors= motor_control_task_get_motors(&motor_count);
```

```
config.motors=motors;
config.motor_count=motor_count;
```

Example:

```
//Typical usage from obstacle_detect_task(after motor task is running):
uint8_t motor_count=k_motor_count_none;
rx_motor_handle_t**motors=motor_control_task_get_motors(&motor_count);
if(motors==nullptr||motor_count==k_motor_count_none){
//Motor stack not yet initialized--treat as fatal
}
config.motors=motors;
config.motor_count=motor_count;
```

See also: rx_motor.h Motor handle operations, obstacle_detect_task.c Uses these handles for emergency stop, motor_control_task_create() Must be called before this function, internal_init_motor_stack() Sets s_motor_stack_initialized on success, s_motor_ptrs Underlying static array returned on success, s_motor_stack_initialized Guard flag checked before returning handles

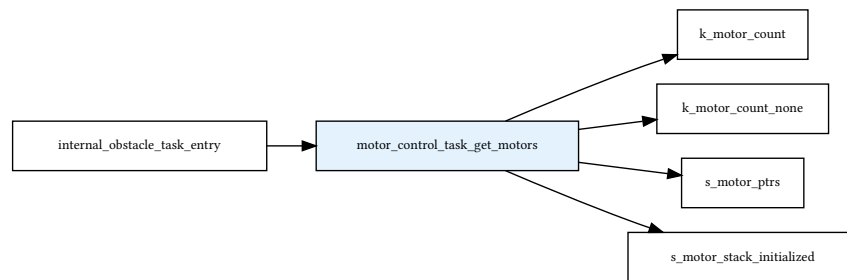


Figure 1400: Call graph for motor_control_task_get_motors

Defined in `src/tasks/inc/motor_control_task.h:166`

Since STAR v1.0.0

obstacle_detect_task.h

Obstacle Detection Task - HC-SR04 Ultrasonic Sensor Monitoring.

Declares the obstacle detection task responsible for monitoring ultrasonic sensors to detect obstacles and trigger emergency stops if needed.

Responsibilities:

- Trigger and read HC-SR04 ultrasonic sensors
- Calculate distance to obstacles
- Monitor critical zones (e.g., < 10 cm)
- Trigger emergency stop on imminent collision
- Update shared obstacle data for path planning

Sensor Configuration:

- Sensors: 4x HC-SR04 ultrasonic rangefinders
- Range: 2 cm to 400 cm
- Trigger: GPTW timer for pulse generation
- Echo: MTU input capture for timing

- Update rate: 50 Hz (20 ms period)

Copyright (c) 2026 STAR Project

Includes:

- "rx_err.h"

obstacle_detect_task_create

```
rx_err_t obstacle_detect_task_create(void)
```

Create and start the obstacle detection task.

Creates the ObstacleTask ThreadX task for ultrasonic sensor monitoring. This task periodically triggers sensors and detects obstacles.

Task Configuration:

- Priority: 12 (medium-low priority - safety-critical but not time-critical)
- Stack: 2048 bytes
- Period: 20 ms (50 Hz obstacle detection)
- Auto-start: Enabled

Create and start the obstacle detection task.

Initializes the obstacle detection system by creating a ThreadX task at medium priority (12) with a static 1024-byte stack. The task initializes 4 HC-SR04 ultrasonic sensors via the rx_obstacle_detect library and registers a callback to trigger emergency stop when obstacles are detected within 30 cm.

Architecture Pattern: Wrapper Task with Callback-Driven Library

- This task creates a ThreadX thread for monitoring/statistics
- rx_obstacle_detect library creates its own internal polling task
- Main work done by library (50 Hz polling, debouncing, distance calculation)
- This task's role: initialization, callback handling, statistics logging

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_*	ThreadX task creation failed

Pre: ThreadX kernel running

Pre: HC-SR04 sensors initialized

Pre: GPTW/MTU timers configured

Pre: Shared data initialized

Post: ObstacleTask created and running at 50 Hz

Post: Obstacle distances updated in shared memory

Post: Emergency stop triggered on close obstacles

Note: Call from tx_application_define() in main.c after sensor initialization

Note: Safety-critical - obstacles closer than 10 cm trigger emergency stop] **See also:** obstacle_detect_task.c Implementation details, main.c Task creation in tx_application_define()

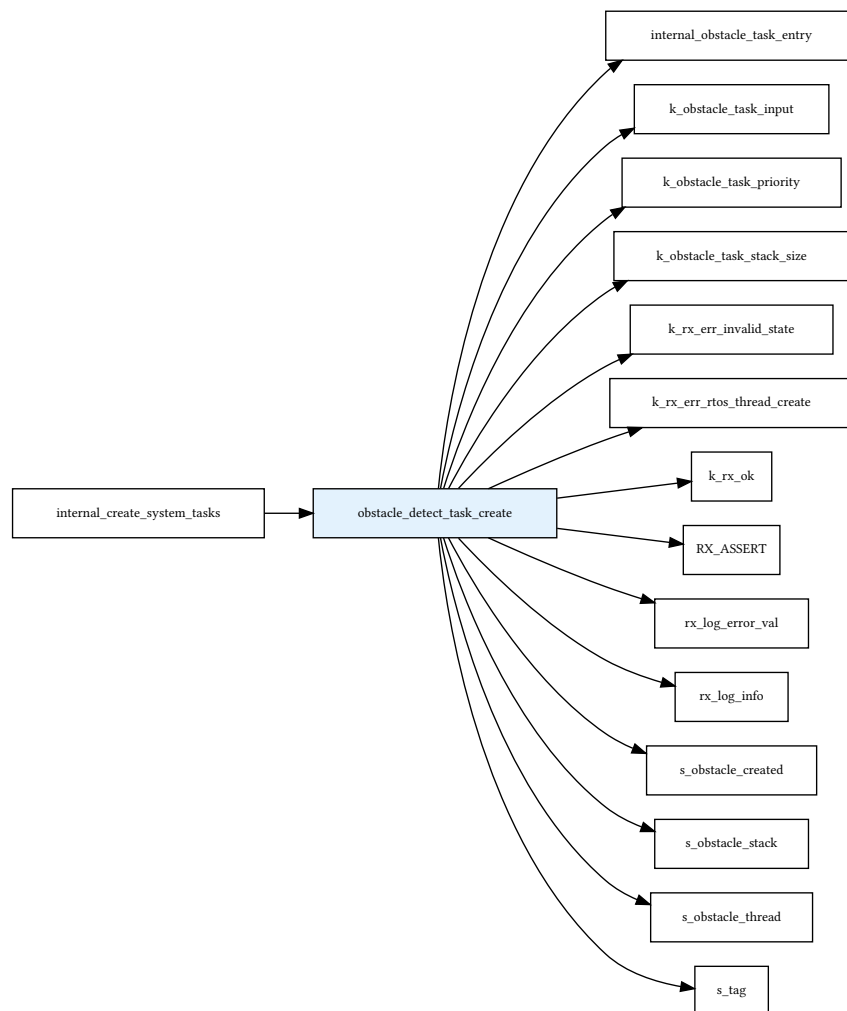


Figure 1401: Call graph for obstacle_detect_task_create

Defined in src/tasks/inc/obstacle_detect_task.h:68

Since STAR v1.0.0

—

telemetry_task.h

Telemetry Aggregation Task - System Status Collection and Reporting.

Declares the telemetry task responsible for aggregating system status data from all subsystems and sending it to the Raspberry Pi 5.

Responsibilities:

- Collect data from shared memory (motor status, sensors)
- Aggregate into Protocol Buffer telemetry messages
- Send periodic status updates to RPi5 via SPI
- Monitor system health and diagnostic counters
- Generate system health reports

Telemetry Data:

- Motor: Velocity, current, encoder position, faults
- Sensors: Temperature, obstacle distances
- System: CPU usage, task status, error counters

Copyright (c) 2026 STAR Project

Includes:

- "rx_err.h"

telemetry_task_create

```
rx_err_t telemetry_task_create(void)
```

Create and start the telemetry aggregation task.

Creates the TelemetryTask ThreadX task for system status reporting. This task periodically sends telemetry data to the Raspberry Pi 5.

Task Configuration:

- Priority: 14 (low priority - best-effort telemetry)
- Stack: 3072 bytes (Protocol Buffers encoding requires stack)
- Period: 50 ms (20 Hz telemetry rate)
- Auto-start: Enabled

Initializes the telemetry task thread and starts it with auto-start enabled. The task immediately begins running after this function returns (unless a higher-priority task is ready to run).

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_*	ThreadX task creation failed

Pre: ThreadX kernel running

Pre: SPI communication initialized

Pre: Protocol Buffers encoder initialized

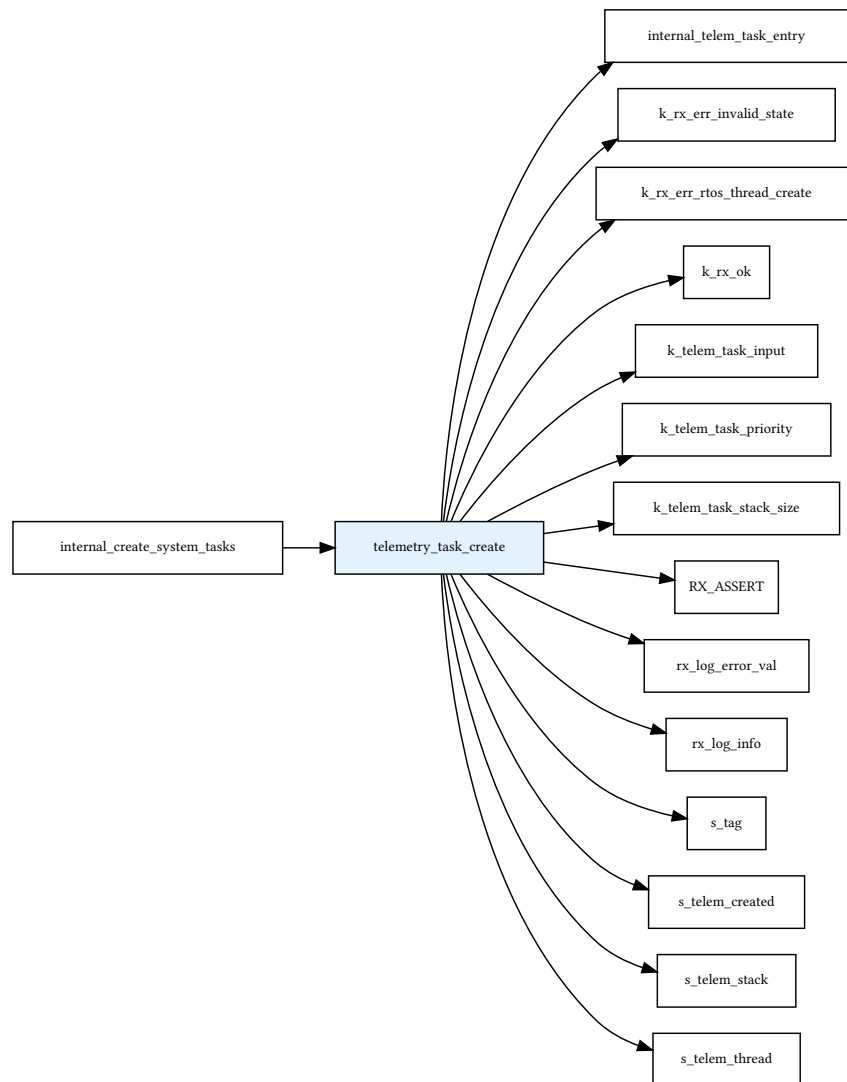
Pre: Shared data initialized

Post: TelemetryTask created and running at 20 Hz

Post: RPi5 receives periodic status updates

Note: Call from tx_application_define() in main.c after communication initialization

Note: Low priority - can be preempted by control tasks] **See also:** telemetry_task.c
Implementation details, main.c Task creation in tx_application_define()

Figure 1402: Call graph for `telemetry_task_create`

_Defined in `src/tasks/inc/telemetry\_task.h:65_`

Since *STAR v1.0.0*

—

temp_sensor_task.h

Temperature Sensor Task - DS18B20 Digital Thermometer Monitoring.

Declares the temperature sensor task responsible for reading DS18B20 1-Wire digital temperature sensors for thermal monitoring.

Responsibilities:

- Read temperature from DS18B20 sensors
- Monitor critical temperatures (motor, ambient)
- Trigger warnings on high temperature conditions
- Update shared temperature data
- Detect sensor faults (CRC errors, disconnects)

Sensor Configuration:

- Sensors: DS18B20 digital thermometers
- Interface: 1-Wire protocol
- Resolution: 12-bit (0.0625degC precision)
- Range: -55degC to +125degC
- Update rate: 10 Hz (100 ms period)

Copyright (c) 2026 STAR Project

Includes:

- "rx_err.h"

temp_sensor_task_create

```
rx_err_t temp_sensor_task_create(void)
```

Create and start the temperature sensor task.

Creates the TempSensorTask ThreadX task for temperature monitoring. This task periodically reads DS18B20 sensors and detects thermal issues.

Task Configuration:

- Priority: 11 (medium-low priority - thermal monitoring not time-critical)
- Stack: 2048 bytes
- Period: 100 ms (10 Hz temperature monitoring)
- Auto-start: Enabled

Create and start the temperature sensor task.

Creates and starts the temperature sensor ThreadX task with low priority (Priority 15) for non-critical ambient temperature monitoring at 1 Hz. This function allocates the thread control block, assigns a static stack, and registers the task with ThreadX.

The temperature data is used by the obstacle detection task to compensate for air temperature variations in HC-SR04 ultrasonic sensor readings. Accurate temperature measurement is essential for precise distance calculation, as the speed of sound in air varies by 0.6 m/s per degC.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_*	ThreadX task creation failed

Pre: ThreadX kernel running

Pre: DS18B20 sensors initialized

Pre: 1-Wire bus configured

Pre: Shared data initialized

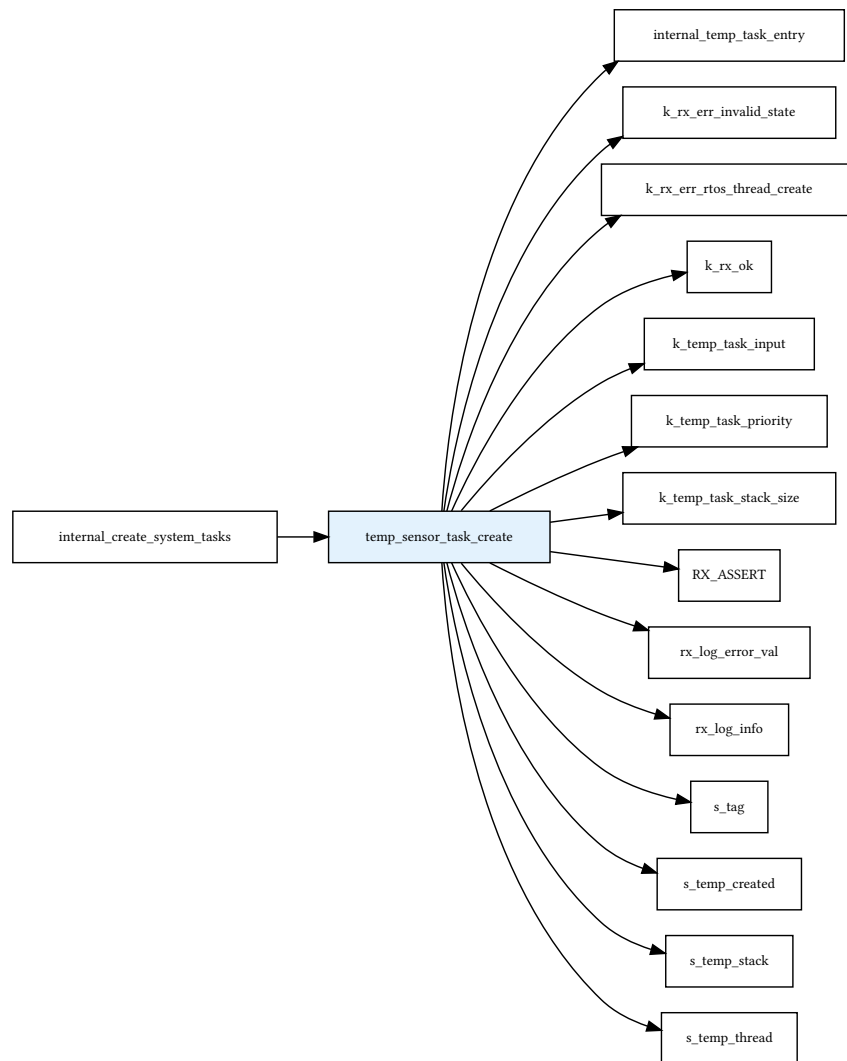
Post: TempSensorTask created and running at 10 Hz

Post: Temperature data updated in shared memory

Post: Warnings triggered on high temperatures

Note: Call from tx_application_define() in main.c after sensor initialization

Note: Thermal monitoring prevents motor overheating] **See also:** temp_sensor_task.c
Implementation details, main.c Task creation in tx_application_define()

Figure 1403: Call graph for `temp_sensor_task_create`

Defined in `src/tasks/inc/temp_sensor_task.h:68`

Since STAR v1.0.0

—

watchdog_monitor_task.h

Watchdog Monitor Task - IWDT supervision and system health monitoring.

Dedicated high-priority task that ensures system liveness by:

1. Checking all registered tasks for heartbeat timeouts
2. Feeding the hardware Independent Watchdog Timer (IWDT)
3. Reporting own heartbeat for self-monitoring

Includes:

- "rx_err.h"

watchdog_monitor_task_create

```
rx_err_t watchdog_monitor_task_create(void)
```

Create and start the watchdog monitor task.

Creates a dedicated task that runs at 100 Hz to supervise system health. Must be called AFTER rx_iwdt_init() and task registration.

Task configuration:

- Name: "WatchdogMon"
- Priority: 6 (high priority, below comm, above motor control)
- Stack: 512 bytes
- Period: 10ms (100 Hz)

Main loop:

Create and start the watchdog monitor task.

Creates and starts the watchdog monitor ThreadX task with high priority (6) for system health supervision at 100 Hz.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_invalid_state	Task already created
k_rx_err_rtos_thread_create	ThreadX task creation failed
k_rx_ok	Task created successfully
k_rx_err_invalid_state	Already created
k_rx_err_rtos_thread_create	ThreadX error

Pre: IWDT initialized via rx_iwdt_init()

Pre: All tasks registered via rx_iwdt_register_task()

Pre: ThreadX kernel running

Pre: rx_iwdt_init() called successfully

Pre: All tasks registered via rx_iwdt_register_task()

Post: Watchdog monitor running at Priority 6, 100 Hz

Post: WatchdogMon thread created and in READY/RUNNING state

Post: WatchdogMon created and running at 100 Hz

Post: s_watchdog_created set to true

Note: Must be called from tx_application_define() during system init

Note: Thread Safety: Not thread-safe, call from tx_application_define only

Warning: Task creation failure is fatal - RX_ASSERT in caller

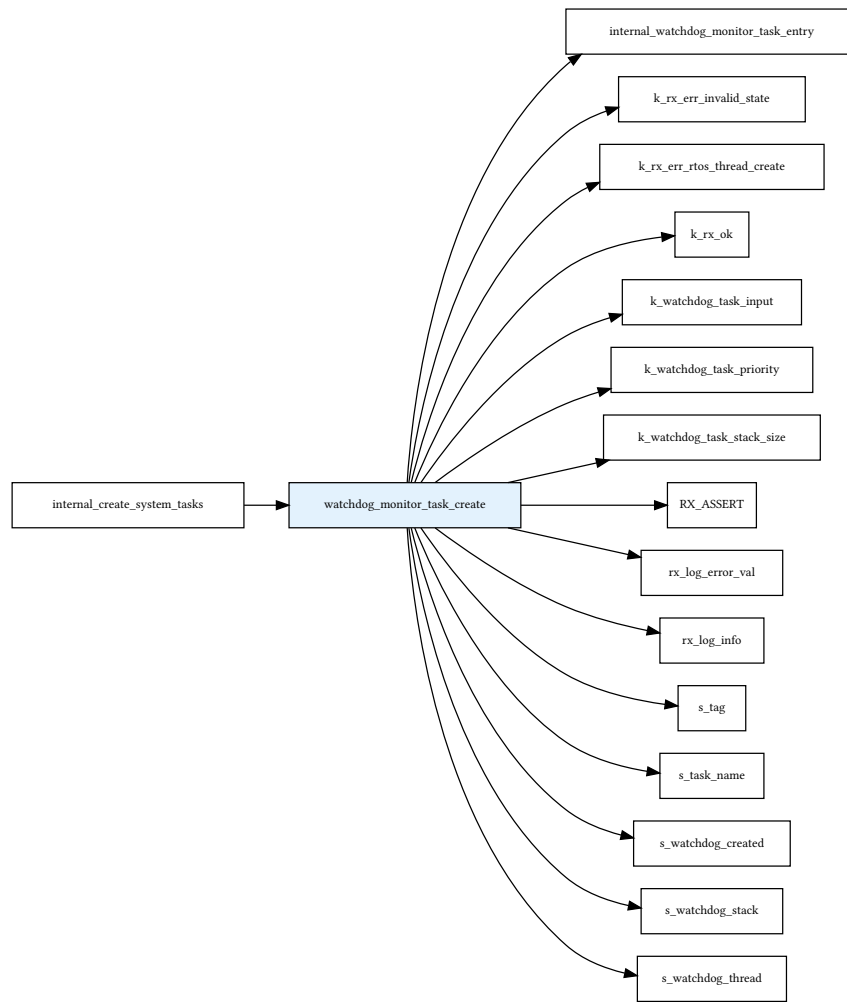
Usage Example:: rx_err_t err;

```
err=rx_iwdt_init(&s_iwdt_config); RX_ASSERT(err==k_rx_ok,"IWDTinitfailed");
err=rx_iwdt_register_task("MotorCtrl",30); RX_ASSERT(err==k_rx_ok,"Taskregistrationfailed");
err=rx_iwdt_set_state(k_system_state_init); RX_ASSERT(err==k_rx_ok,"Setstatefailed");
err=watchdog_monitor_task_create(); RX_ASSERT(err==k_rx_ok,"Watchdogtaskcreationfailed");
```

Example:

```
while(1){
    rx_iwdt_check_tasks();//Detecttasktimeouts
    rx_iwdt_feed();//Feedhardwarewatchdog
    rx_iwdt_task_heartbeat("WatchdogMon");//Self-report
    tx_thread_sleep(1);//10ms@100Hz
}
```

See also: rx_iwdt_init() Initialize IWDT before calling this, rx_iwdt_register_task()
Register tasks before creating monitor

Figure 1404: Call graph for `watchdog_monitor_task_create`

Defined in `src/tasks/inc/watchdog_monitor_task.h:162`

Since Version 1.0.0

—

led_status_task.c

LED Status Indicator Task Implementation.

Implements a low-priority ThreadX task that drives 6 status LEDs at 20 Hz to provide visual system health feedback. Each LED is mapped to a specific system function and driven by state read from `shared_data`.

Includes:

- "led_status_task.h"
- "hardware_config.h"

- "rx_check.h"
- "rx_iwdt.h"
- "rx_log.h"
- "rx_poeg.h"
- "rx_port_utils.h"
- "shared_data.h"
- "tx_api.h"

led_task_constants_t

LED status task configuration constants.

Value	Name	Description
= 512	k_led_task_stack_size	Stack size in bytes (GPIO writes only)
= 17	k_led_task_priority	ThreadX priority (low - visual only)
= 0	k_led_task_input	Thread entry input parameter
= 5	k_led_task_period_ticks	50ms period = 20 Hz (5 ticks @ 100 Hz)

Since Version 1.0.0

—

led_timing_constants_t

LED blink timing constants in task ticks (50ms per tick).

Value	Name	Description
= 10	k_led_heartbeat_half_period	10 ticks x 50ms = 500ms half-period (1 Hz)
= 3	k_led_error_half_period	3 ticks x 50ms = 150ms half-period (fast blink)
= 2	k_led_comm_pulse_duration	2 ticks x 50ms = 100ms comm activity pulse

Since Version 1.0.0

—

led_timing_divisor_t

Divisor constants for LED blink period calculations.

Provides named constants for arithmetic performed on LED timing values. Using a named divisor instead of a bare literal prevents magic numbers and makes the intent of half-period calculations self-documenting.

k_led_half_period_divisor must equal 2 (half-period = full/2)

Value	Name	Description
= 2	k_led_half_period_divisor	Divisor to convert full period to half period (full / 2)

See also: led_timing_constants_t Full-period and half-period tick counts

Example:

```
//Convert a full-period counter to half-period comparison:
bool led_on = (s_heartbeat_counter < (k_led_heartbeat_half_period
/k_led_half_period_divisor));
```

Since Version 1.0.0

—

led_index_t

LED index assignments for clarity.

Value	Name	Description
= 0	k_led_idx_heartbeat	LED 0: System heartbeat
= 1	k_led_idx_error	LED 1: Error indicator
= 2	k_led_idx_motor	LED 2: Motor active
= 3	k_led_idx_comm	LED 3: Communication activity
= 4	k_led_idx_obstacle	LED 4: Obstacle detected
= 5	k_led_idx_estop	LED 5: E-stop active

Since Version 1.0.0

—

s_led_thread

TX_THREAD [s_led_thread](#)

ThreadX thread control block.

—

s_led_stack

[uint8_t](#) s_led_stack[k_led_task_stack_size]

Static thread stack (no dynamic allocation).

—

s_led_created

[bool](#) [s_led_created](#)

Task creation guard flag.

—

s_tag

[const char*](#) [const](#) [s_tag](#)

Log tag for this module.

—

s_led_ports

[const](#) [uint8_t](#) s_led_ports[k_led_count]

LED port numbers lookup table (indexed by LED number).

—

s_led_pins

[const](#) [uint8_t](#) s_led_pins[k_led_count]

LED pin numbers lookup table (indexed by LED number).

—

s_heartbeat_counter

`uint8_t s_heartbeat_counter`

Heartbeat blink counter (incremented each task tick).

—

s_error_counter

`uint8_t s_error_counter`

Error blink counter.

—

s_comm_pulse_remaining

`uint8_t s_comm_pulse_remaining`

Communication pulse countdown (ticks remaining).

—

s_last_comm_sequence

`uint32_t s_last_comm_sequence`

Last seen motor command sequence number for comm activity detection.

—

internal_led_task_entry

```
static void internal_led_task_entry(ULONG input)
```

LED status task entry point - infinite loop updating LEDs at 20 Hz.

Reads system state from `shared_data` each tick and updates 6 LEDs:

- LED 0: Heartbeat toggle at 1 Hz (10 ticks on, 10 ticks off)
- LED 1: Fast blink when any motor fault active
- LED 2: Solid on when any motor has nonzero duty cycle
- LED 3: 100ms pulse when new motor command received
- LED 4: Solid on when any obstacle detected
- LED 5: Solid on when e-stop is active

Parameters:

Direction	Name	Description
in	input	Thread input parameter (unused)

Pre: `led_status_task_create()` called successfully

Pre: ThreadX scheduler started

Pre: shared_data initialized

Post: Infinite loop - never returns

Post: LEDs updated at 20 Hz based on system state

Note: Thread Safety: Reads shared_data via thread-safe APIs

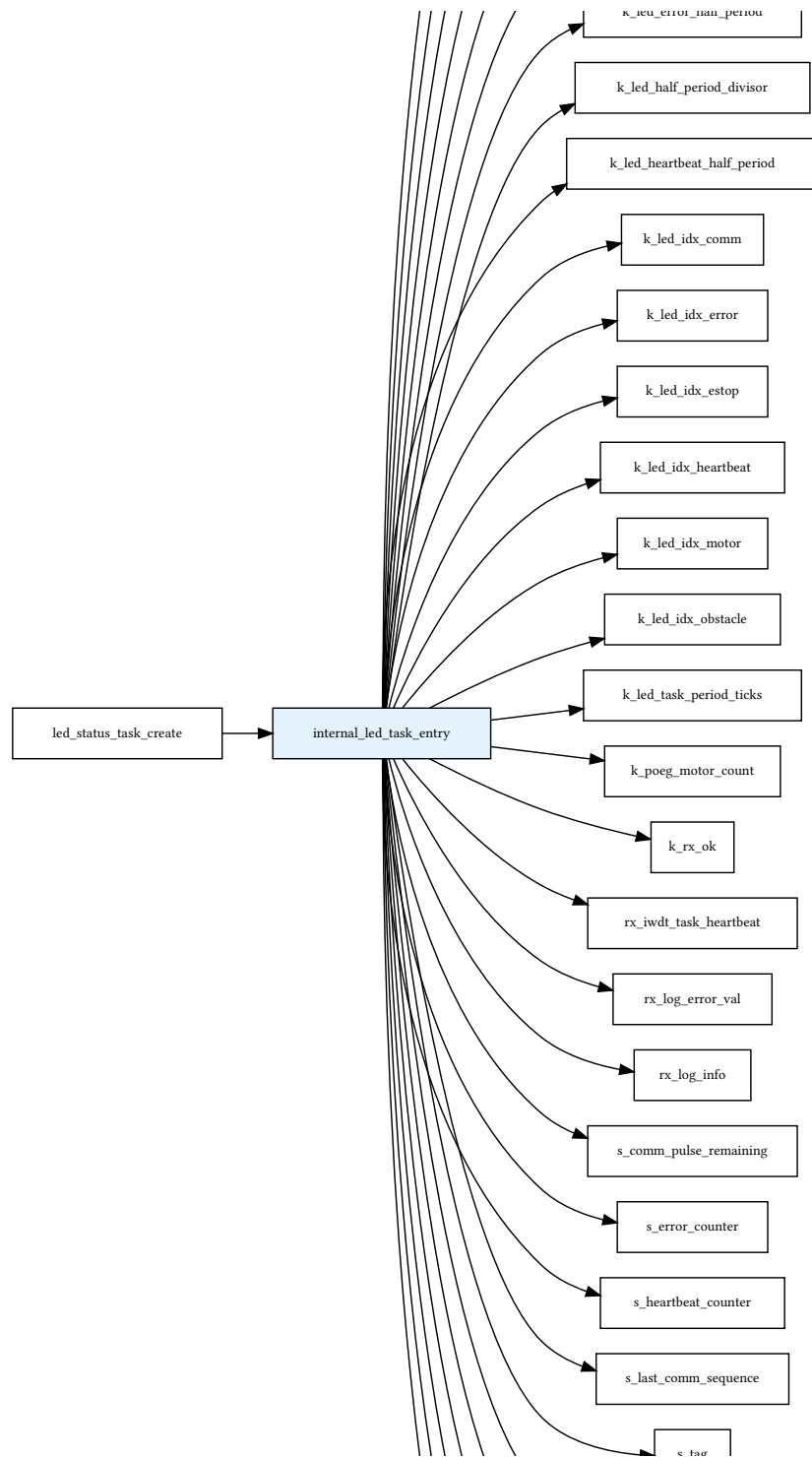


Figure 1405: Call graph for internal_led_task_entry

Defined in `src/tasks/led_status_task.c:391`

Since Version 1.0.0

—

internal_led_set

```
static void internal_led_set(uint8_t led_index, bool on)
```

Set an LED on or off by writing to its PORT output data register.

Drives a single LED high (on) or low (off) by writing to the PODR bit of the corresponding PORT register. Out-of-range `led_index` values or invalid port pointers return silently to prevent crashes from defensive callers.

Algorithm:

1. Bounds-check `led_index` against `k_led_count`; return silently if invalid
2. Retrieve PORT register base from `s_led_ports[led_index]`
3. Return silently if port is nullptr (invalid port configuration)
4. Compute `pin_mask = (1U << s_led_pins[led_index])`
5. Set (`on=true`) or clear (`on=false`) the PODR bit for the pin

Parameters:

Direction	Name	Description
in	<code>led_index</code>	LED index (0 to <code>k_led_count - 1</code> , inclusive) 0: Heartbeat LED 1: Error LED 2: Motor active LED 3: Communication activity LED 4: Obstacle detected LED 5: E-stop active LED
in	<code>on</code>	true to turn LED on (PODR bit set), false to turn off

Pre: LED GPIO initialized via `internal_led_init_gpio()`

Pre: `led_index < k_led_count` (silently ignored if out of range)

Post: LED PODR bit set or cleared to match the requested on/off state

Post: No change if `led_index` is out of range or port is invalid

Note: Thread Safety: Safe from single task context; do not call from multiple tasks without external synchronization] **Example:**

```
//TurnheartbeatLED on:
internal_led_set(k_led_idx_heartbeat, true);

//TurnerrorLED off:
internal_led_set(k_led_idx_error, false);
```

See also: `internal_led_init_gpio()` Must be called first to configure pins as outputs, `led_index_t` Named constants for `led_index` values, `internal_led_task_entry()` Main loop that calls this function at 20 Hz

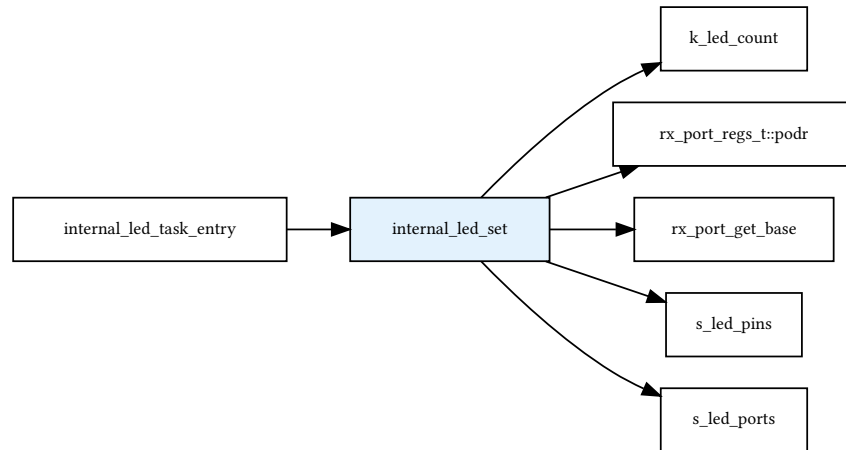


Figure 1406: Call graph for `internal_led_set`

Defined in `src/tasks/led_status_task.c:346`

Since Version 1.0.0

—

internal_led_init_gpio

```
static void internal_led_init_gpio(void)
```

Initialize LED GPIO pins as outputs with all LEDs off.

Iterates over all `k_led_count` LED entries in the `s_led_ports` and `s_led_pins` lookup tables and configures each pin as a GPIO output set low (LED off). For each pin the function:

1. Retrieves the PORT register base via `rx_port_get_base()`
2. Clears the PMR bit to set the pin to GPIO mode
3. Sets the PDR bit to configure the pin as an output
4. Clears the PODR bit to drive the pin low (LED off)

Invalid port entries (`rx_port_get_base` returns `nullptr`) are skipped with an error log; remaining valid pins continue to be initialized.

Pre: PORT register space must be accessible (clock and power initialized)

Pre: `s_led_ports` and `s_led_pins` must contain valid PORT/pin values

Post: All valid LED pins are configured as GPIO outputs, driven low

Post: Invalid LED port entries are skipped (error logged, not fatal)

Note: Thread Safety: Call once during task initialization only; not re-entrant due to read-modify-write on PORT registers] **Example:**

```
//CalledinternallyduringLEDtaskstartup:
staticvoidinternal_led_task_entry(ULONGinput){
(void)input;
internal_led_init_gpio();//AllLEDsoffafterthis
while(true){...} //Mainblinkloop
}
```

See also: internal_led_task_entry() Only caller of this function, internal_led_set() Runtime LED state control after initialization, rx_port_get_base() PORT register accessor used for each pin

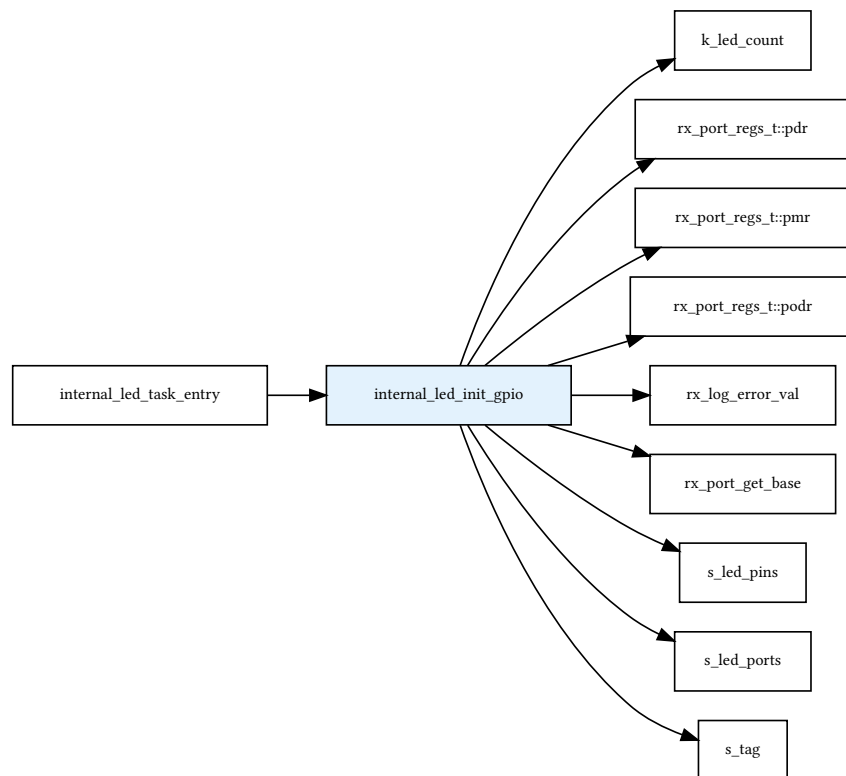


Figure 1407: Call graph for internal_led_init_gpio

Defined in `src/tasks/led_status_task.c:277`

Since Version 1.0.0

led_status_task_create

```
rx_err_t led_status_task_create(void)
```


Create the LED status indicator task.

Create and start the LED status indicator task.

Creates and starts the LED ThreadX task with low priority (17) for visual system health feedback at 20 Hz. GPIO pins are initialized as outputs inside the task entry to keep creation lightweight.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_invalid_state	Already created
k_rx_err_rtos_thread_create	ThreadX error

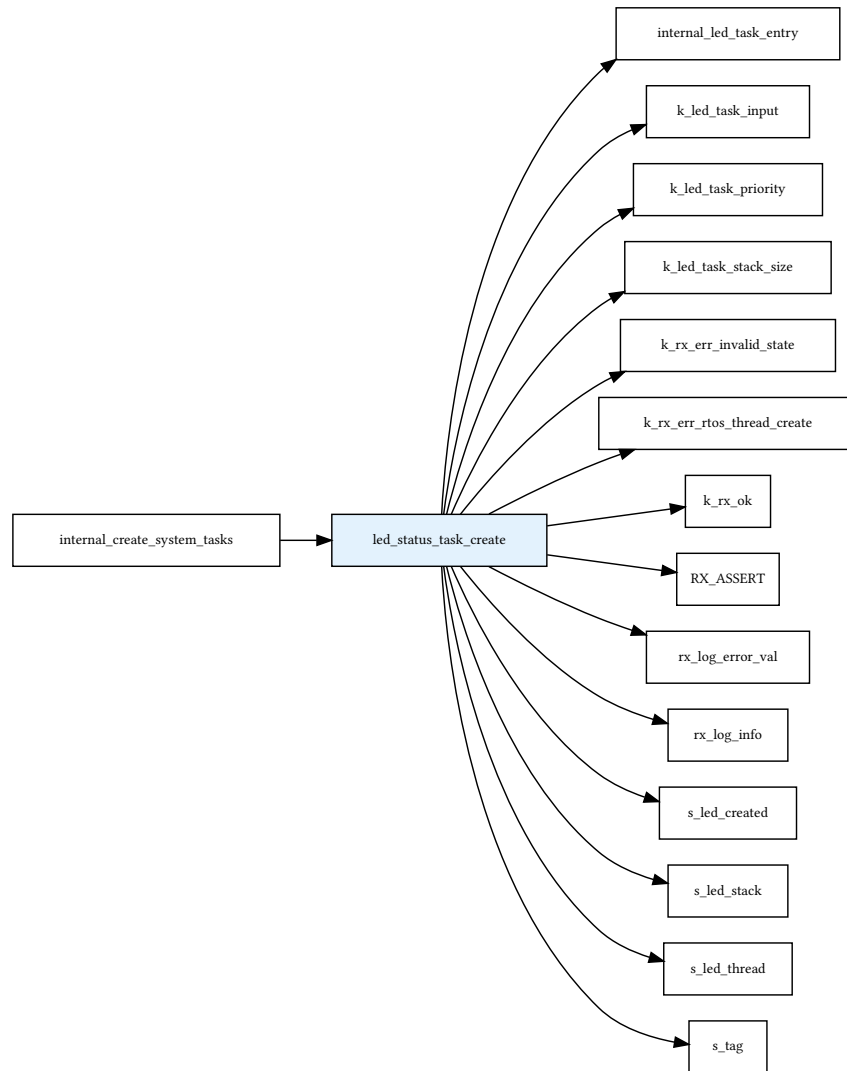
Pre: ThreadX kernel running

Pre: shared_data_init() called

Post: LEDTask created and running at 20 Hz

Post: s_led_created set to true

Note: Thread Safety: Not thread-safe, call from tx_application_define only

Figure 1408: Call graph for `led_status_task_create`

Defined in `src/tasks/led_status_task.c:202`

Since Version 1.0.0

—

motor_control_task.c

Motor Control Task - 4-Motor PID Velocity Control @ 100 Hz.

Includes:

- "motor_control_task.h"
- "hardware_config.h"
- "rx_bus_manager.h"

- "rx_check.h"
- "rx_encoder_tpu.h"
- "rx_iwdt.h"
- "rx_log.h"
- "rx_motor.h"
- "rx_mtu_encoder.h"
- "rx_pid.h"
- "shared_data.h"
- "tx_api.h"

motor_task_constants_t

Motor control task configuration constants.

Defines ThreadX task parameters for the motor control task, which runs the closed-loop PID velocity controller at 100 Hz. The task executes at priority 8, between the watchdog monitor (6) and lower-priority tasks, ensuring deterministic control loop timing for all four motors.

`k_motor_task_stack_size` must be sufficient for PID computation local variables and call stack (verified via stack high-water mark)

`k_motor_task_priority` must be lower (higher number) than the watchdog monitor priority (6) to allow watchdog preemption

Value	Name	Description
= 2048	<code>k_motor_task_stack_size</code>	Stack size in bytes: sufficient for PID locals and HAL calls
= 8	<code>k_motor_task_priority</code>	ThreadX priority: 8 (below watchdog=6, comm=5)
= 1	<code>k_motor_task_sleep_ticks</code>	Sleep period: 1 tick = 10ms at 100 Hz system tick rate
= 0	<code>k_motor_task_input</code>	Thread entry input parameter: unused (ThreadX convention)

See also: `motor_control_task_create()` Function that uses these constants, `motor_control_constants_t` Algorithm constants for the control loop

Example:

```
//Usedinternallybymotor_control_task_create():
tx_thread_create(&s_motor_thread,
"MotorCtrl",
internal_motor_task_entry,
k_motor_task_input,
s_motor_stack,
k_motor_task_stack_size,
k_motor_task_priority,
k_motor_task_priority,
TX_NO_TIME_SLICE,
TX_AUTO_START);
```

Since Version 1.0.0

—

motor_control_constants_t

Motor control algorithm constants.

Defines physical and algorithmic parameters for the 4-wheel motor control system. These constants are derived from hardware specifications and MATLAB system identification results. The control period matches the ThreadX tick rate to ensure the PID dt parameter is accurate.

The encoder count of 1364 is derived from the Hall encoder specification: 341 pulses per revolution (PPR) x 4 (quadrature decoding edges) = 1364 counts per revolution. This provides 0.26deg angular resolution.

k_motor_count must match the number of H-bridge channels physically wired on the PCB (4 channels fixed)

k_control_period_us must match the ThreadX tick period in microseconds (10 ticks/s x 1000 us/tick = 10000 us)

Value	Name	Description
= 4	k_motor_count	Number of motors: 4 wheels (front-left, front-right, back-left, back-right)
= 10000	k_control_period_us	Control period: 10000 us = 10ms = 100 Hz control loop rate
= 50	k_active_brake_ms	Active brake duration: 50ms short-circuit braking before coast
= 30	k_active_brake_duty	Active brake PWM duty: 30% duty cycle during braking phase
= 20000	k_pwm_frequency_hz	PWM frequency: 20 kHz (above human hearing, reduces audible noise)
= 1364	k_encoder_counts_per_rev	Encoder resolution: 341 PPR x 4 (quadrature) = 1364 counts/rev

See also: motor_hw_constants_t Hardware-level PWM and current parameters,
motor_task_constants_t Task scheduling constants

Example:

```
//Computingvelocityfromencodercounts:
constfloatdt_sec=(float)k_control_period_us/1000000.0F;
constfloatrad_per_count=(2.0F*M_PI)/(float)k_encoder_counts_per_rev;
floatvelocity_rps=(float)delta_counts*rad_per_count/dt_sec;
```

Since Version 1.0.0

—

motor_index_t

Motor array indices for the four-wheel differential drive platform.

Maps logical motor positions to array indices used throughout the motor control subsystem. The index order (front-left=0, front-right=1, back-left=2, back-right=3) is consistent across all motor arrays: motor handles, encoder handles, PID controllers, and shared_data motor state arrays.

The physical motor layout viewed from above:

Values must be contiguous starting at 0 so they can be used as array indices into k_motor_count-sized arrays

`k_motor_back_right` must equal `k_motor_count - 1`

Value	Name	Description
= 0	<code>k_motor_front_left</code>	Front-left motor: array index 0
= 1	<code>k_motor_front_right</code>	Front-right motor: array index 1
= 2	<code>k_motor_back_left</code>	Back-left motor: array index 2
= 3	<code>k_motor_back_right</code>	Back-right motor: array index 3

See also: `motor_control_constants_t` `k_motor_count` defines the array size, `encoder_limits_t` Encoder count distribution (front vs rear)

Example:

Front
`[0][1]`
`[2][3]`
 Rear

Example:

```
//Iteratingoverallmotors:
for(uint8_ti=k_motor_front_left;i<k_motor_count;i++){
  rx_pid_compute(&s_pid[i],setpoint[i],measured[i],dt,&output[i]);
}
```

Since Version 1.0.0

—

encoder_limits_t

Encoder channel counts per encoder type for initialization loops.

The RX72N uses two different hardware timer peripherals to read the four Hall effect quadrature encoders. Front motors use the Multi-Function Timer Unit (MTU) and rear motors use the Timer Pulse Unit (TPU). This enum defines the count of channels for each peripheral so initialization loops can be bounded at compile time.

Channel assignment:

- MTU channels 0-1: front-left, front-right encoders
- TPU channels 0-1: back-left, back-right encoders

`k_front_encoder_count + k_rear_encoder_count` must equal `k_motor_count`

Value	Name	Description
= 2	<code>k_front_encoder_count</code>	Front encoder channels: 2 MTU channels (motors 0 and 1)
= 2	<code>k_rear_encoder_count</code>	Rear encoder channels: 2 TPU channels (motors 2 and 3)

See also: `motor_index_t` Motor array index assignments, `motor_control_constants_t` `k_motor_count` (total motor count)

Example:

```
//InitializefrontencodersviaMTU
for(uint8_ti=0;i<k_front_encoder_count;i++){
  rx_encoder_mtu_init(&s_encoders[k_motor_front_left+i],mtu_ch[i]);
}
```

```

}
//InitializearencodersviaTPU
for(uint8_ti=0;i<k_rear_encoder_count;i++){
  rx_encoder_tpu_init(&s_encoders[k_motor_back_left+i],tpu_ch[i]);
}

```

Since Version 1.0.0

—

motor_hw_constants_t

Motor hardware configuration constants for H-bridge drivers.

Defines hardware-level parameters that configure the H-bridge motor drivers. These values are derived from hardware specifications and system power budget analysis.

- PWM frequency: 20 kHz chosen to be inaudible to humans while keeping switching losses acceptable for the 6V brushed DC motors
- Dead-time: 1 us prevents shoot-through in the H-bridge during high/low-side switching transitions
- Current limit: 2A software limit protects motor windings

k_motor_pwm_freq_hz must be supported by the RX72N MTU/GPT at the configured PCLK frequency (120 MHz -> minimum 1.8 kHz)

k_motor_dead_time_ns must exceed the H-bridge propagation delay (typically 100-500 ns) to prevent shoot-through

Value	Name	Description
= 20000	k_motor_pwm_freq_hz	PWM frequency: 20 kHz (inaudible, low switching loss)
= 1000	k_motor_dead_time_ns	H-bridge dead-time: 1000 ns = 1 us (prevents shoot-through)
= 2000	k_motor_current_limit_ma	Software current limit: 2000 mA = 2A per motor channel
= 10	k_threadx_tick_interval_ms	ThreadX tick period: 10 ms at 100 Hz tick rate

See also: motor_control_constants_t Algorithm-level constants (PID period, encoder PPR), motor_task_constants_t Task scheduling constants (stack, priority)

Example:

```

//ConfiguringmotorPWM:
rx_motor_config_tcfg={
  .pwm_freq_hz=k_motor_pwm_freq_hz,
  .dead_time_ns=k_motor_dead_time_ns,
};
rx_err_t err=rx_motor_init(&s_motors[i],&cfg);

```

Since Version 1.0.0

—

g_bus_manager

rx_bus_manager_t g_bus_manager

Global bus manager instance (defined in `shared_data.c`).

Note: Do NOT access this variable directly from tasks. Use `rx_bus_manager_get_bus()`.] **See also:** `rx_bus_manager_init()` Initialize bus manager (in `hardware_init.c`), `rx_bus_types.h` Bus abstraction API

Example:

```
rx_bus_interface_t*i2c=rx_bus_manager_get_bus(&g_bus_manager,k_rx_bus_type_i2c);
i2c->read(i2c->ctx,imu_addr,data,len);
```

—

s_pid_kp_default

```
const float s_pid_kp_default
```

Default PID proportional gain (MATLAB-tuned).

—

s_pid_ki_default

```
const float s_pid_ki_default
```

Default PID integral gain (MATLAB-tuned).

—

s_pid_kd_default

```
const float s_pid_kd_default
```

Default PID derivative gain.

—

s_pid_output_min_percent

```
const float s_pid_output_min_percent
```

PID output minimum (percent duty).

—

s_pid_output_max_percent

```
const float s_pid_output_max_percent
```

PID output maximum (percent duty).

—

s_pid_integral_min_percent

```
const float s_pid_integral_min_percent
```

PID integral minimum (percent, anti-windup clamp).

—

s_pid_integral_max_percent

```
const float s_pid_integral_max_percent
```

PID integral maximum (percent, anti-windup clamp).

—

s_velocity_near_stopped_mps

```
const float s_velocity_near_stopped_mps
```

Velocity threshold below which motor is considered stopped (m/s).

—

s_motor_thread

```
TX_THREAD s_motor_thread
```

ThreadX thread control block.

—

s_motor_stack

```
uint8_t s_motor_stack[k_motor_task_stack_size]
```

Static thread stack (no dynamic allocation).

—

s_motor_created

```
bool s_motor_created
```

Task creation guard flag.

—

s_motor_stack_initialized

```
bool s_motor_stack_initialized
```

Motor stack initialization complete flag.

Note: Set inside `internal_motor_task_entry()` only when `internal_init_motor_stack()` returns `k_rx_ok` not at thread creation time

Warning: Do NOT use `s_motor_created` as a proxy for this flag; `s_motor_created` is set by `motor_control_task_create()` immediately after `tx_thread_create()`, before the thread has run or initialized any motor hardware

Since STAR v1.0.0

—

s_motors

```
rx_motor_handle_t s_motors[k_motor_count]
```

Motor handles for each motor.

—

s_motor_ptrs

```
rx_motor_handle_t* s_motor_ptrs[k_motor_count]
```

Externally accessible array of `rx_motor_handle_t` pointers (one per motor).

Note: Read-only for external callers do NOT modify pointed-to handles

Warning: Handles are uninitialized until `internal_init_motor_stack()` completes

Since STAR v1.0.0

—

s_encoder_state

`rx_encoder_state_t s_encoder_state[k_motor_count]`

Encoder state for each motor.

—

s_pids

`rx_pid_handle_t s_pids[k_motor_count]`

PID controller handles for each motor.

—

s_dt_sec

`const float s_dt_sec`

Control loop dt (seconds).

—

s_timeout_estop_triggered

`bool s_timeout_estop_triggered`

Flag to track if timeout e-stop was already triggered.

—

s_active_brake_in_progress

`bool s_active_brake_in_progress`

Flag to track if currently in active brake sequence.

—

s_tag

`const char* const s_tag`

Log tag for this module.

—

s_task_name

`const char* const s_task_name`

IWDT task name for heartbeat registration and reporting.

Note: IWDT task name (“MotorCtrl”) intentionally differs from ThreadX thread name (“MotorTask”)

Note: ThreadX name kept short for thread list display; IWDT name more descriptive

Warning: Do not modify - IWDT registration depends on exact string match] **See also:**
rx_iwdt_register_task() Task registration using this name, rx_iwdt_task_heartbeat()
Heartbeat reporting using this name

Since Version 1.0.0

—

internal_motor_task_entry

```
static void internal_motor_task_entry(ULONG input)
```

Motor control task entry point - infinite 100 Hz PID control loop.

This is the main task loop that executes after motor_control_task_create() starts the thread. The function never returns and runs continuously at 100 Hz (10ms period) controlling 4 DC motors with PID velocity control until power-down or emergency stop.

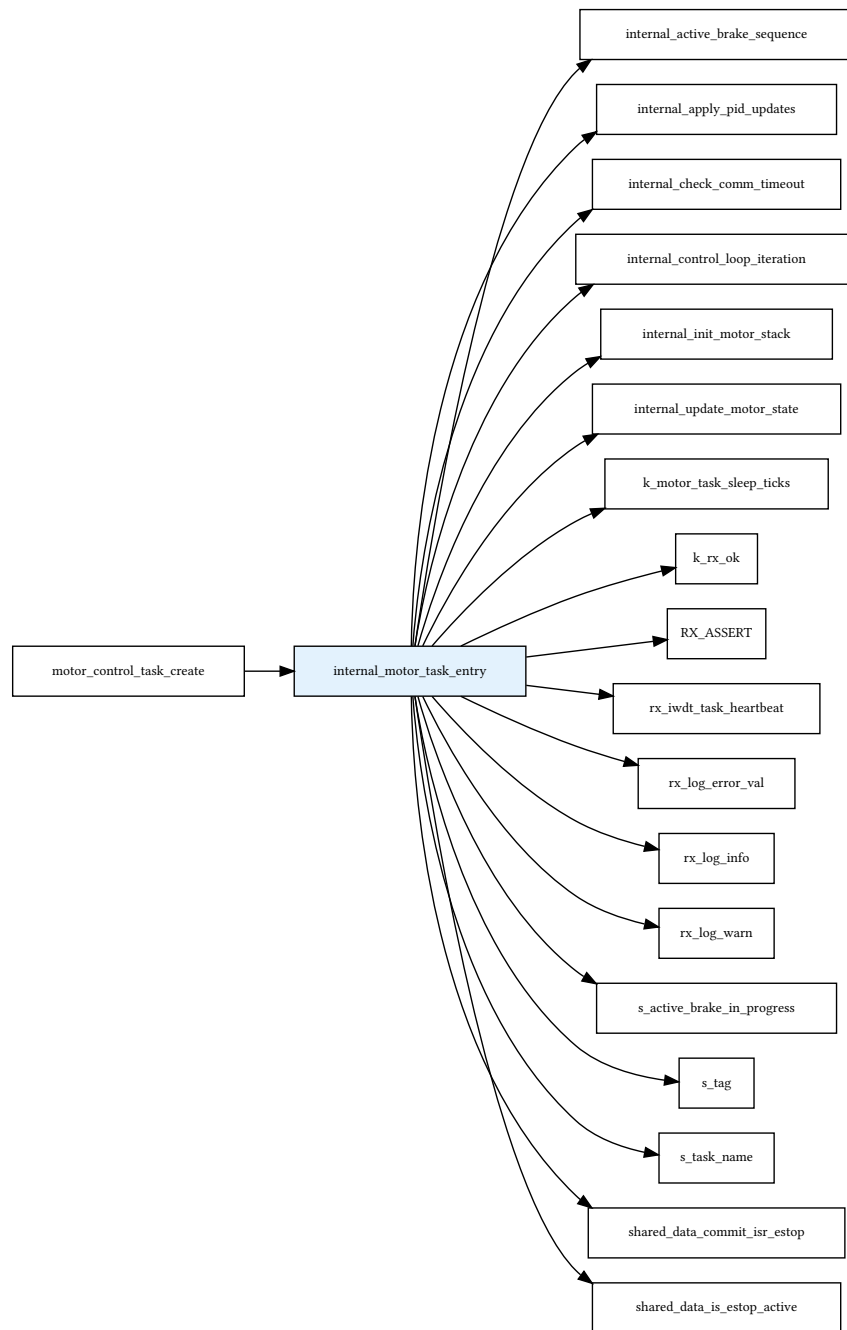


Figure 1409: Call graph for `internal_motor_task_entry`

Defined in `src/tasks/motor_control_task.c:1458`

internal_init_motor_stack

```
static rx_err_t internal_init_motor_stack(void)
```

Initialize motor control stack (4 motors, encoders, PIDs, drivers).

Initializes the complete motor control system including all hardware drivers and software controllers for 4 DC gearmotors. This function retrieves MATLAB-tuned PID gains from `shared_data` and configures all 4 PID controllers identically.

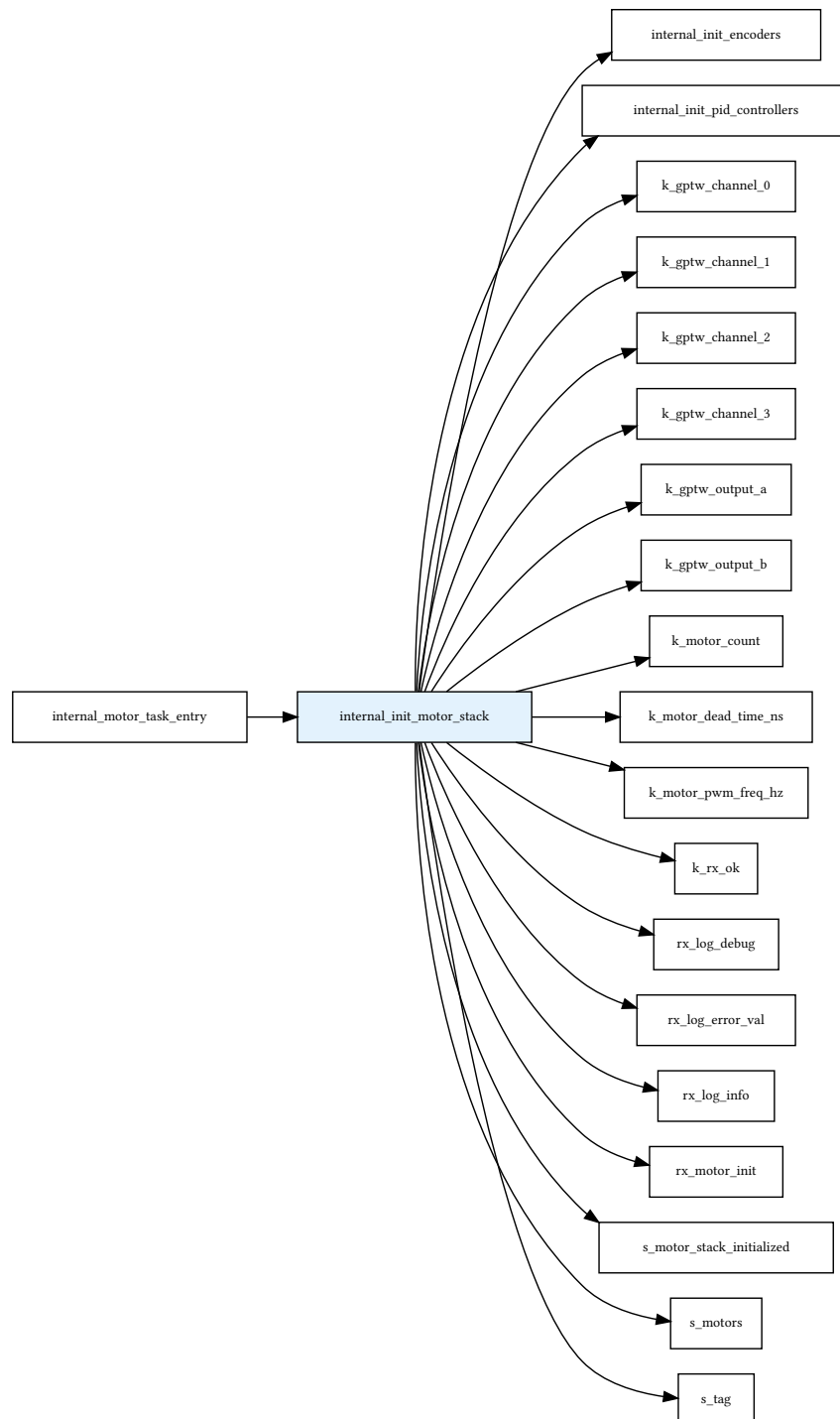


Figure 1410: Call graph for `internal_init_motor_stack`

Defined in *src/tasks/motor_control_task.c:1698*

—

internal_init_pid_controllers

```
static rx_err_t internal_init_pid_controllers(void)
```

Initialize PID controllers for all 4 motors.

Loads PID gains from `shared_data` (set by comm task) or falls back to MATLAB-tuned default gains. Initializes all 4 PID controller handles.

Returns: `rx_err_t` Initialization status

Value	Description
<code>k_rx_ok</code>	All 4 PID controllers initialized
<code>k_rx_err_null_ptr</code>	nullptr in <code>rx_pid_init</code> (internal error)
<code>k_rx_err_invalid_arg</code>	Invalid PID configuration

Pre: `s_pids` array allocated (static memory)

Pre: `shared_data_init()` called

Post: `s_pids0-3` initialized with gains

Note: Not thread-safe. Called once during task startup.



Figure 1411: Call graph for `internal_init_pid_controllers`

Defined in src/tasks/motor_control_task.c:1767

Since Version 1.0.0

—

internal_init_encoders

```
static rx_err_t internal_init_encoders(void)
```

Initialize all 4 quadrature encoders (MTU front + TPU rear).

Initializes front-left (MTU1) and front-right (MTU2) encoders via the MTU encoder driver, and rear-left (TPU1) and rear-right (TPU2) encoders via the TPU encoder driver. All four encoders use 4x quadrature decoding with 1364 counts per revolution (341 PPR * 4).

Returns: rx_err_t Initialization status

Value	Description
k_rx_ok	All 4 encoders initialized
k_rx_err_*	Error from rx_encoder_init() or rx_tpu_encoder_init()

Pre: MTU and TPU peripheral clocks enabled

Pre: Encoder pins configured via MPC

Post: All 4 encoder channels ready for velocity measurement

Note: Not thread-safe. Called once during task startup.

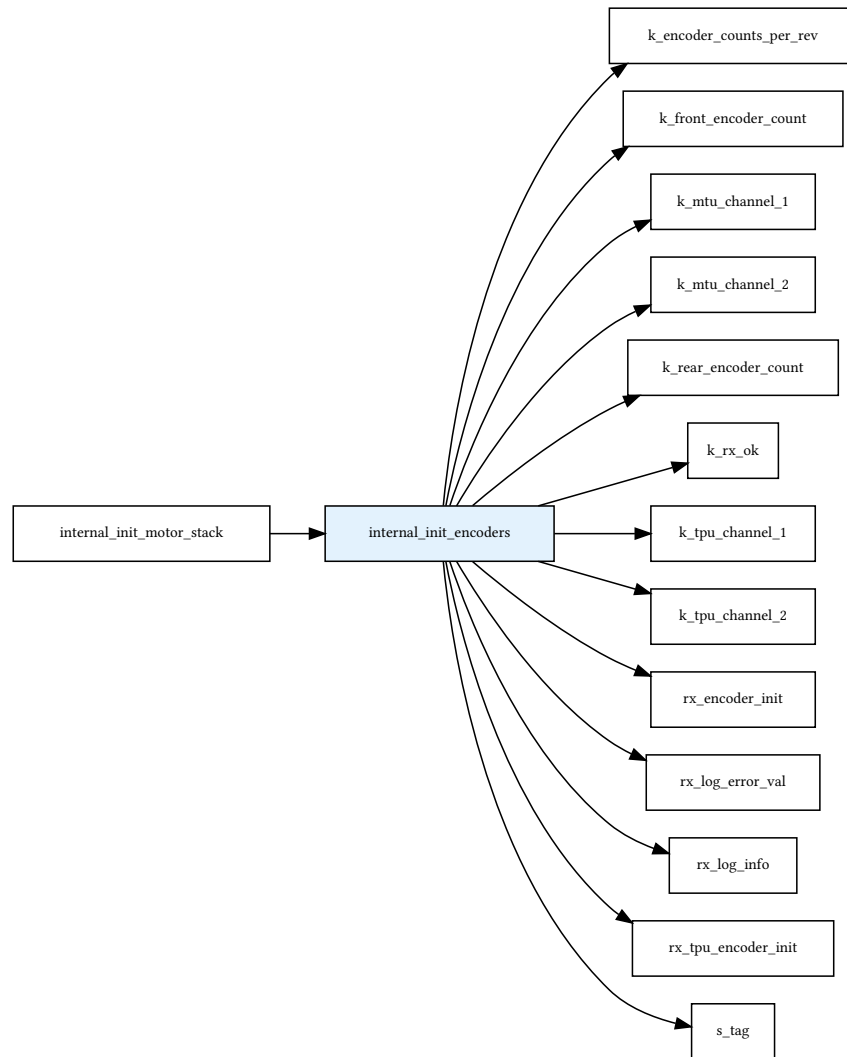


Figure 1412: Call graph for `internal_init_encoders`

Defined in `src/tasks/motor_control_task.c:1827`

Since Version 1.0.0

—

internal_control_loop_iteration

```
static void internal_control_loop_iteration(void)
```

Execute one iteration of the 100 Hz control loop (10ms cycle for 4 motors).

This is the core motor control function that executes one complete control cycle for all 4 motors. It reads encoder velocities, retrieves target velocities from shared_data, computes PID outputs, applies PWM duties, and checks for driver faults.

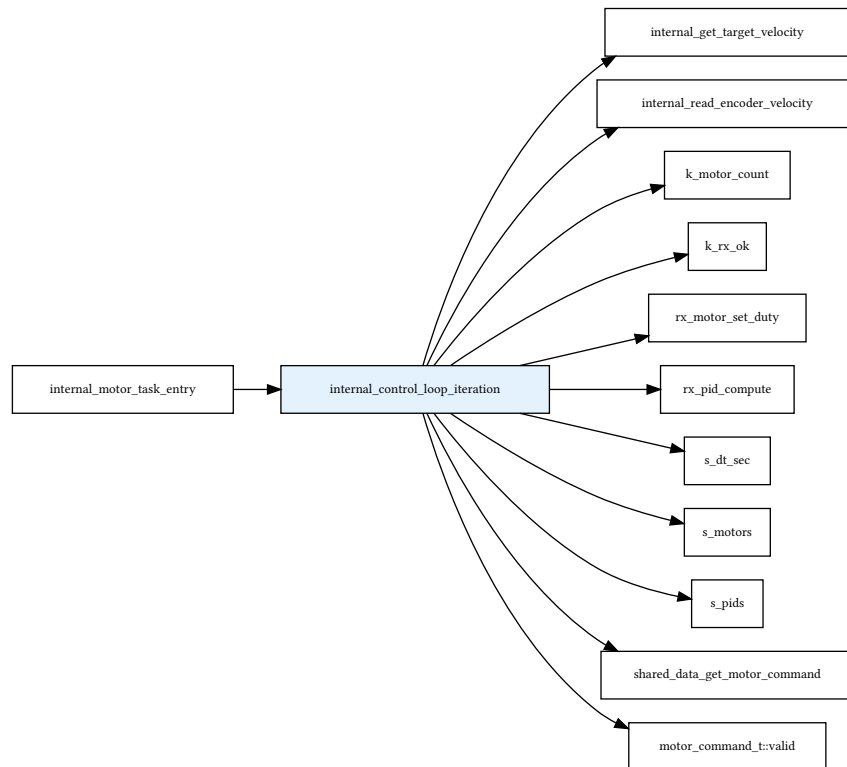


Figure 1413: Call graph for `internal_control_loop_iteration`

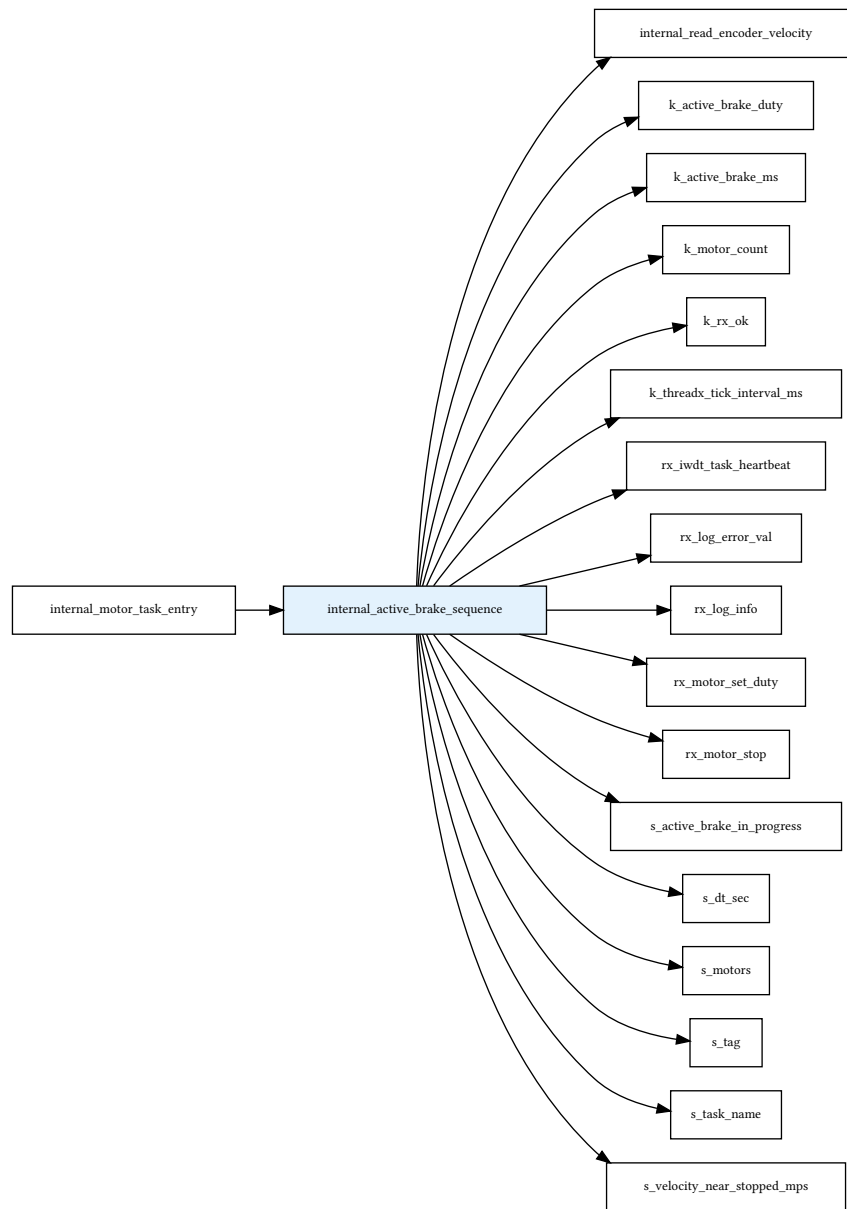
Defined in `src/tasks/motor_control_task.c:2109`

internal_active_brake_sequence

```
static void internal_active_brake_sequence(void)
```

Execute active braking sequence (50ms reverse PWM @ 30% to stop motors).

Active braking is an emergency stop technique that applies reverse PWM for a brief period to quickly dissipate motor kinetic energy, followed by passive regenerative braking to lock motors in place. This achieves faster stopping than coast-down or passive braking alone.

Figure 1414: Call graph for `internal_active_brake_sequence`

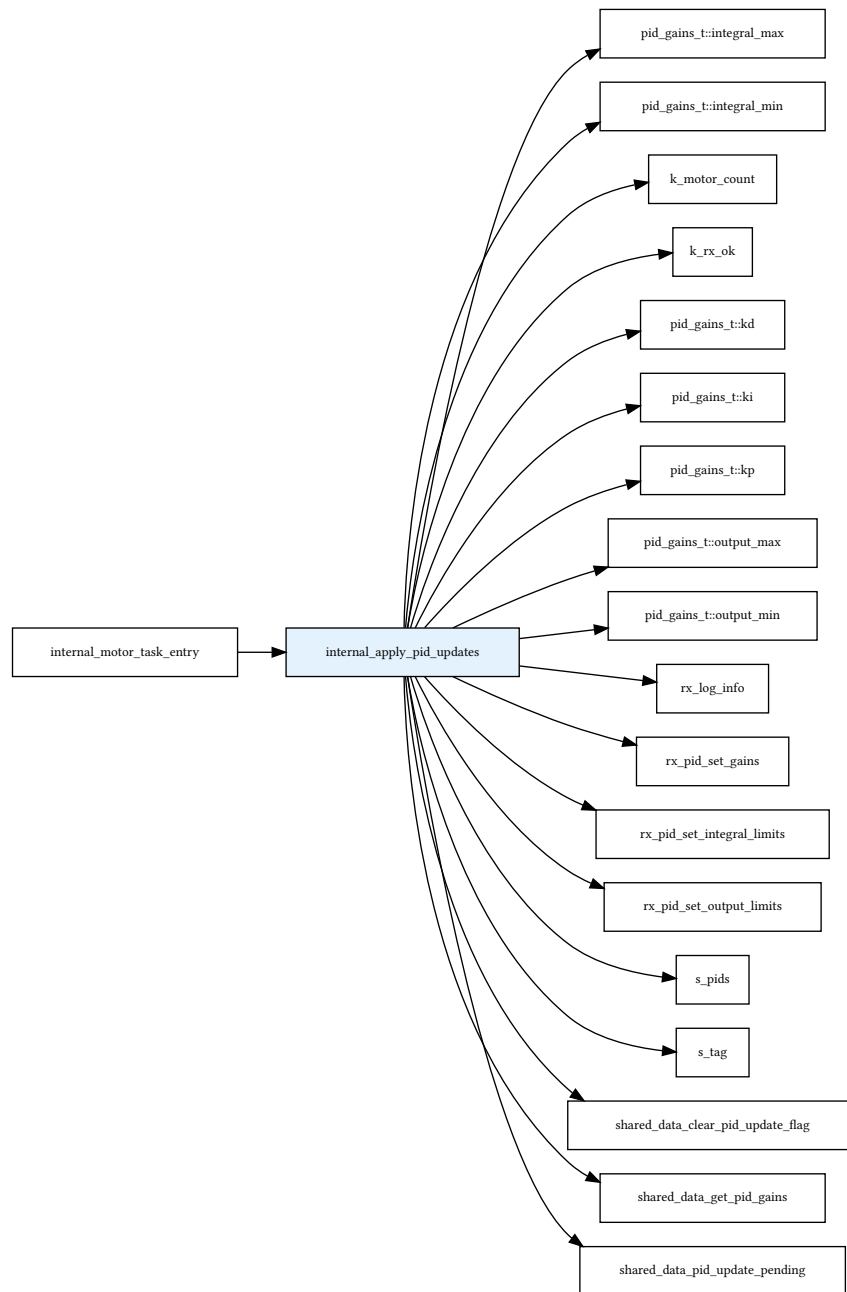
Defined in `src/tasks/motor_control_task.c:2366`

`internal_apply_pid_updates`

```
static void internal_apply_pid_updates(void)
```

Apply pending PID gain updates from `shared_data` to all 4 PID controllers.

Checks if new PID gains were received via SPI Protocol Buffer command (`SetPIDGainsRequest`) and applies them to all 4 motor PID controllers at runtime. This allows tuning PID gains without recompiling or reflashing firmware.

Figure 1415: Call graph for `internal_apply_pid_updates`

Defined in `src/tasks/motor_control_task.c:2488`

internal_check_comm_timeout

```
static void internal_check_comm_timeout(void)
```

Check for communication timeout and trigger e-stop (500ms watchdog).

Monitors the age of the last received motor command. If no valid command has been received within 500ms, triggers emergency stop to prevent runaway motors. This is a critical safety feature that prevents uncontrolled robot motion if communication with the Raspberry Pi 5 is lost.

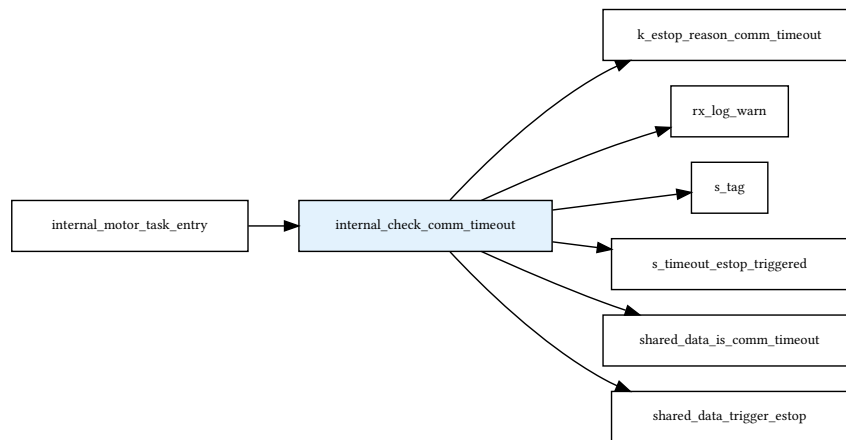


Figure 1416: Call graph for `internal_check_comm_timeout`

Defined in `src/tasks/motor_control_task.c:2580`

internal_read_encoder_velocity

```
static rx_err_t internal_read_encoder_velocity(float *velocity_rps, const float dt_sec, const uint8_t motor_idx)
```

Read encoder velocity for any motor (dispatches MTU or TPU).

Motors 0-1 (front) use MTU encoder, motors 2-3 (rear) use TPU encoder. This function dispatches to the correct encoder API based on motor index.

Parameters:

Direction	Name	Description
out	<code>velocity_rps</code>	Pointer to store velocity (rev/sec)
in	<code>dt_sec</code>	Time interval in seconds
in	<code>motor_idx</code>	Motor index (0-3)

Returns: `rx_err_t` Error code

Value	Description
<code>k_rx_ok</code>	Success

k_rx_err_invalid_arg	Invalid motor index
----------------------	---------------------

Pre: Encoders initialized via internal_init_encoders()

Post: velocity_rps updated with current velocity

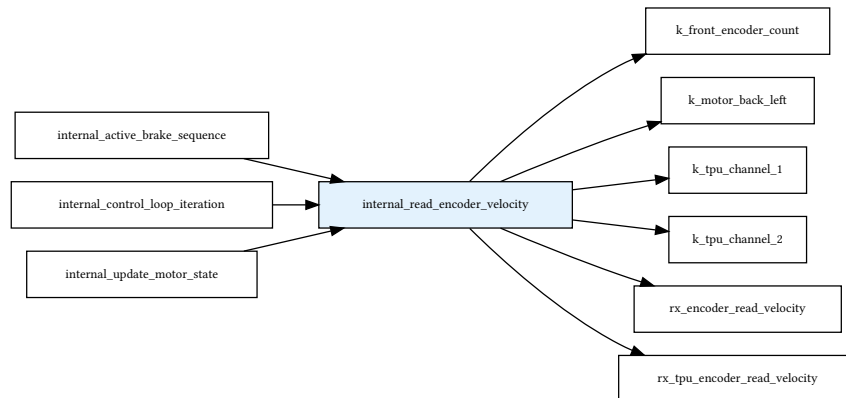


Figure 1417: Call graph for internal_read_encoder_velocity

Defined in `src/tasks/motor_control_task.c:1897`

Since Version 1.0.0

internal_read_encoder_count

```
static rx_err_t internal_read_encoder_count(const uint8_t motor_idx,
rx_encoder_state_t *state)
```

Read encoder count for any motor (dispatches MTU or TPU).

Motors 0-1 (front) use MTU encoder, motors 2-3 (rear) use TPU encoder. This function dispatches to the correct encoder API based on motor index.

Parameters:

Direction	Name	Description
in	motor_idx	Motor index (0-3)
out	state	Pointer to encoder state structure

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Success
k_rx_err_invalid_arg	Invalid motor index

Pre: Encoders initialized via `internal_init_encoders()`

Post: state updated with current encoder count and position

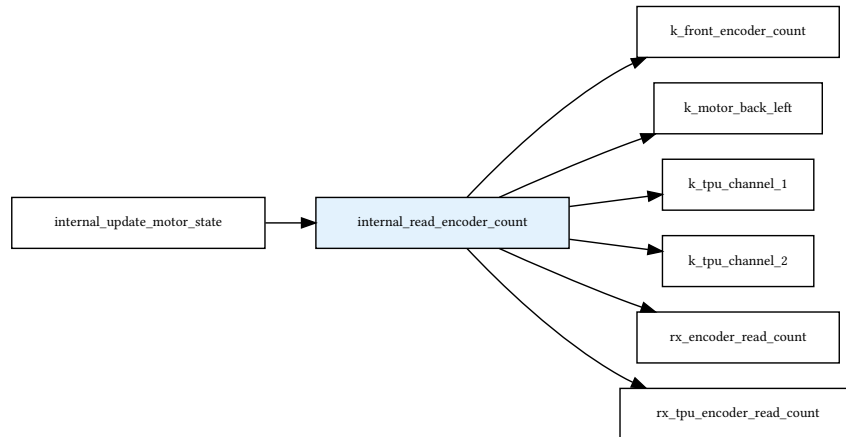


Figure 1418: Call graph for `internal_read_encoder_count`

Defined in `src/tasks/motor_control_task.c:1928`

Since Version 1.0.0

—

internal_update_motor_state

```
static void internal_update_motor_state(void)
```

Update motor state in `shared_data` for telemetry (aggregate 4-motor state).

Collects current state from all 4 motors (velocity, duty, current, encoder counts, faults) and aggregates into a single `motor_state_t` structure for consumption by the telemetry task. This function is called every control loop iteration (100 Hz) to provide real-time motor state to the Raspberry Pi 5 via SPI.

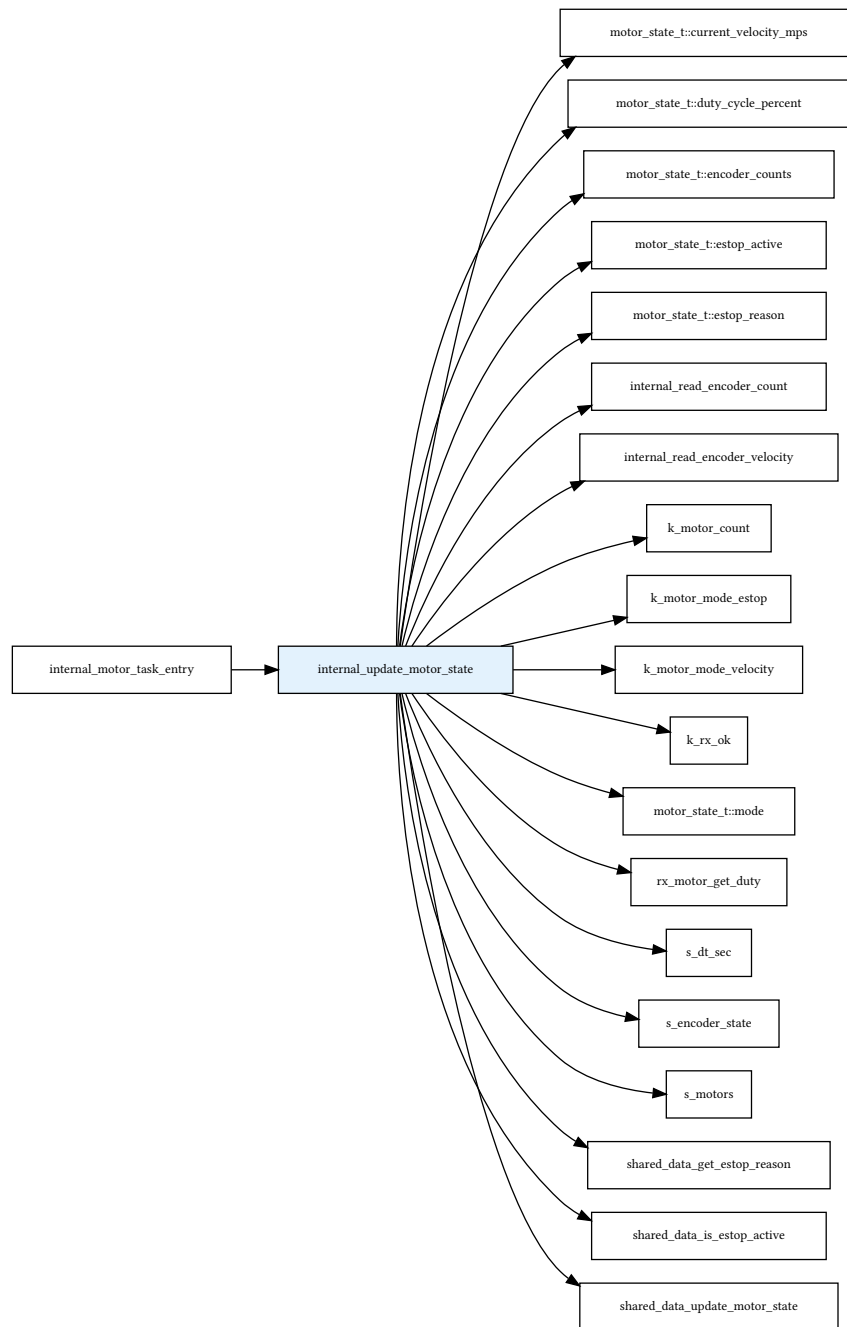


Figure 1419: Call graph for `internal_update_motor_state`

Defined in `src/tasks/motor_control_task.c:2644`

internal_get_target_velocity

```
static float internal_get_target_velocity(const motor_command_t *cmd, uint8_t
motor_idx)
```

Get target velocity for a specific motor from command structure.

Extracts the target velocity for a specific motor from the motor_command_t structure received via SPI. This is a simple accessor function with bounds checking.

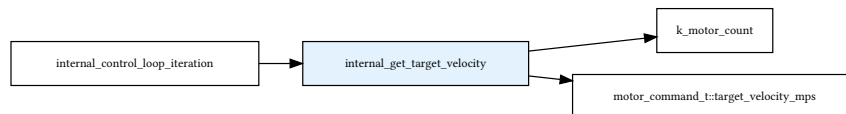


Figure 1420: Call graph for internal_get_target_velocity

Defined in `src/tasks/motor_control_task.c:2759`

—

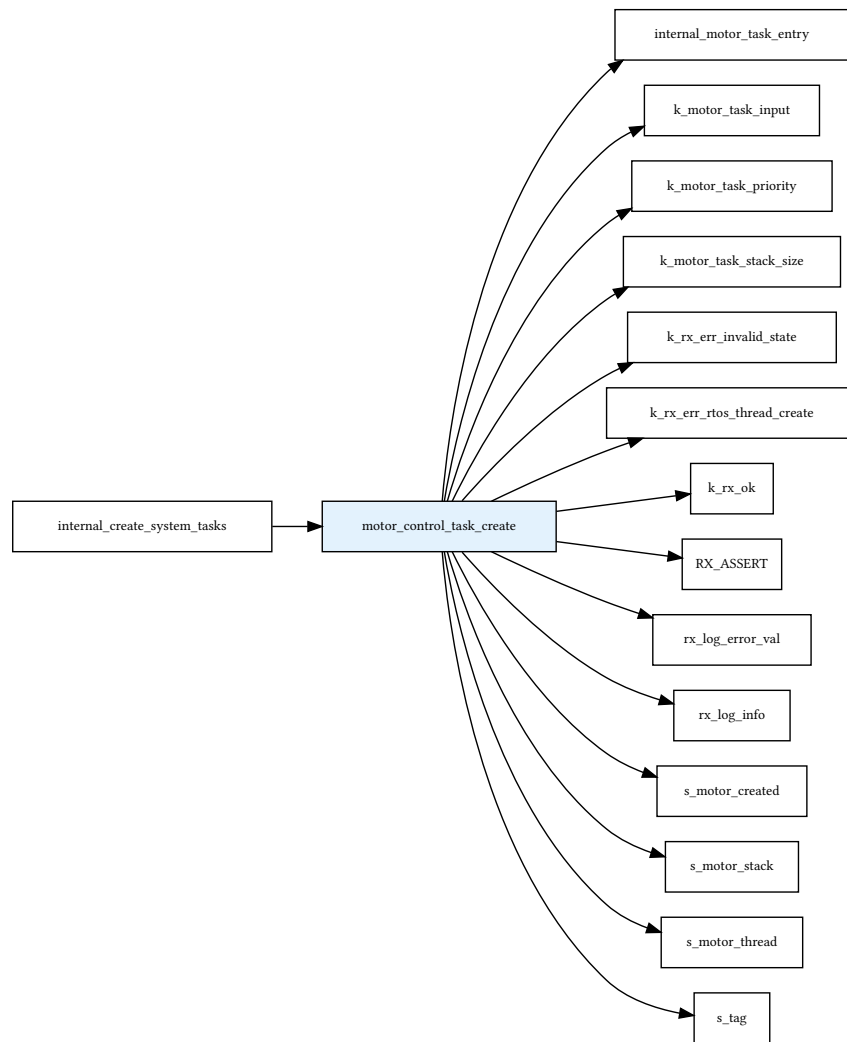
motor_control_task_create

```
rx_err_t motor_control_task_create(void)
```

Create the Motor Control task (4-motor PID velocity control @ 100 Hz).

Create and start the motor control task.

Creates and starts the Motor Control ThreadX task with high priority (Priority 8) for real-time 100 Hz PID velocity control of 4 DC gearmotors. This is the most critical task in the firmware - all robot motion control depends on this task executing correctly within its 10ms timing budget.

Figure 1421: Call graph for `motor_control_task_create`

Defined in `src/tasks/motor_control_task.c:1032`

motor_control_task_get_motors

```
rx_motor_handle_t ** motor_control_task_get_motors(uint8_t *out_count)
```

Get motor handle array for obstacle detection emergency stop.

Returns pointers to the 4 initialized motor handles. The obstacle detection system uses these handles to execute emergency motor stops when obstacles breach the safety perimeter.

Three possible outcomes:

1. Motor stack initialized + out_count non-null -> returns s_motor_ptrs, sets *out_count = k_motor_count
2. Motor stack not yet initialized (s_motor_stack_initialized == false) -> returns nullptr, sets *out_count = k_motor_count_none
3. out_count is nullptr -> returns nullptr immediately, out_count unchanged

Guard condition (s_motor_stack_initialized vs s_motor_created): s_motor_created is set immediately after tx_thread_create() before the thread has executed. s_motor_stack_initialized is set only when internal_init_motor_stack() returns k_rx_ok inside the thread. This function checks s_motor_stack_initialized to ensure handles are fully initialized before returning them.

Parameters:

Direction	Name	Description
out	out_count	Pointer to receive motor count. Set to k_motor_count (4) when motor stack is initialized; set to k_motor_count_none (0) when not yet initialized. Must be non-NULL (nullptr returns nullptr without writing).

Returns: rx_motor_handle_t** Pointer to array of motor handle pointers

Value	Description
s_motor_ptrs	Valid pointer to static array of k_motor_count (4) rx_motor_handle_t*, *out_count set to k_motor_count motor stack fully initialized
nullptr	with *out_count = k_motor_count_none internal_init_motor_stack() has not yet completed successfully
nullptr	(out_count unchanged) out_count argument was nullptr

Pre: motor_control_task_create() called (otherwise s_motor_stack_initialized is never set)

Pre: out_count != nullptr (nullptr out_count causes immediate nullptr return)

Post: *out_count == k_motor_count when return value is non-null

Post: *out_count == k_motor_count_none when motor stack not yet initialized

Note: Thread-safe: Returns pointer to static memory (no shared mutable state)

Note: Lifetime: Returned pointer valid until program termination (static allocation)

Note: Returns nullptr gracefully if called before motor stack init completes (no assert)]

Example:

```
//Typicalusagefromobstacle_detect_task(aftermotortaskisrunning):
uint8_tmotor_count=k_motor_count_none;
rx_motor_handle_t**motors=motor_control_task_get_motors(&motor_count);
if(motors==nullptr||motor_count==k_motor_count_none){
```

```
//Motorstacknotyetinitialized--treatasfatal
}
config.motors=motors;
config.motor_count=motor_count;
```

See also: `motor_control_task_create()` Must be called before this function,
`internal_init_motor_stack()` Sets `s_motor_stack_initialized` on success, `s_motor_ptr`
Underlying static array returned on success, `s_motor_stack_initialized` Guard flag
checked before returning handles

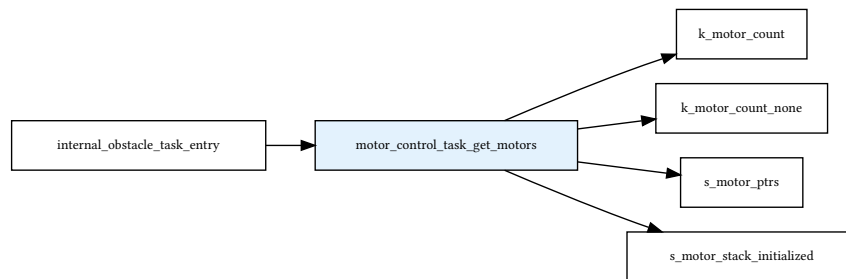


Figure 1422: Call graph for `motor_control_task_get_motors`

Defined in `src/tasks/motor_control_task.c:1121`

Since STAR v1.0.0

—

obstacle_detect_task.c

Obstacle Detection Task Implementation - HC-SR04 Ultrasonic Collision Avoidance.

Includes:

- "obstacle_detect_task.h"
- "motor_control_task.h"
- "rx_check.h"
- "rx_hcsr04.h"
- "rx_iwdt.h"
- "rx_log.h"
- "rx_obstacle_detect.h"
- "rx_port_utils.h"
- "shared_data.h"
- "tx_api.h"

obstacle_task_constants_t

Obstacle detection task configuration constants.

Defines task-level constants for ThreadX thread creation and monitoring. All values chosen to balance responsiveness, CPU usage, and memory footprint.

Value	Name	Description
= 1024	<code>k_obstacle_task_stack_size</code>	Stack size in bytes (static allocation)
= 12	<code>k_obstacle_task_priority</code>	ThreadX priority (1=highest, 31=lowest)

= 2	k_obstacle_heartbeat_ticks	IWDT heartbeat interval: 2 ticks = 20ms @ 100 Hz (3x safety margin for 60ms timeout)
= 50	k_obstacle_stats_log_interval	Log stats every 50 heartbeats (50 x 20ms = 1s)
= 0	k_obstacle_task_input	Thread entry input parameter (unused)

Since Version 1.0.0

—

obstacle_sensor_constants_t

HC-SR04 obstacle sensor configuration parameters.

Defines sensor-specific constants for obstacle detection behavior. These values are passed to the rx_obstacle_detect library during initialization.

Threshold Calculation (30 cm):

- Max robot speed: 0.5 m/s
- Detection + brake latency: 100 ms
- Distance during latency: 0.5 m/s x 0.1 s = 5 cm
- Braking distance at 1 m/s² deceleration: $v^2/(2a) = 12.5$ cm
- Total stopping distance: 17.5 cm
- Safety margin: 30 - 17.5 = 12.5 cm (71%)

Debounce Rationale (3 samples):

- False positive rate without debounce: 10% (acoustic glitches)
- 3-sample confirmation: <1% false positive rate (measured)
- Confirmation latency: 3 x 20 ms = 60 ms (acceptable)
- Distance traveled during confirmation: 0.5 m/s x 0.06 s = 3 cm

Value	Name	Description
= 4	k_obstacle_sensor_count	Number of HC-SR04 sensors (perimeter coverage)
= 30	k_obstacle_threshold_cm	Obstacle detection threshold (30 cm = 71% margin)
= 3	k_obstacle_debounce_samples	Consecutive readings to confirm state change
= 20	k_obstacle_poll_interval_ms	Polling interval: 20 ms = 50 Hz per sensor

Since Version 1.0.0

—

obstacle_sensor_idx_t

Named indices for the 4 HC-SR04 sensors in s_sensors and s_sensor_ptrs.

Provides self-documenting array indices for sensor access, eliminating magic numbers in initializers and any future direct array access. Maps to physical sensor positions around the robot perimeter.

Exactly k_obstacle_sensor_count (4) values defined

Values are contiguous starting at 0 (suitable as array indices)

Value	Name	Description
-------	------	-------------

= 0	k_sensor_front_left	Front-left sensor: TRIG=PF5 (k_rx_pf_5), ECHO=P03/IRQ11 (k_rx_p0_3)
= 1	k_sensor_front_right	Front-right sensor: TRIG=PJ5 (k_rx_pj_5), ECHO=P02/IRQ10 (k_rx_p0_2)
= 2	k_sensor_back_left	Back-left sensor: TRIG=PJ3 (k_rx_pj_3), ECHO=P01/IRQ9 (k_rx_p0_1)
= 3	k_sensor_back_right	Back-right sensor: TRIG=P33 (k_rx_p3_3), ECHO=P00/IRQ8 (k_rx_p0_0)

See also: s_sensors HC-SR04 handle array indexed by these values, s_sensor_ptrs Pointer array indexed by these values, internal_init_sensors() Uses these indices to initialize sensors in order

Example:

```
//Accessaspecificsensorhandlebypositionname
rx_hcsr04_t*front_left=&s_sensors[k_sensor_front_left];

//Iterateoverallsensors
for(uint8_ti=k_sensor_front_left;i<k_obstacle_sensor_count;i++){
rx_hcsr04_config_tcfg={.trigger_pin=trigger_pins[i],...};
}
```

Since STAR v1.0.0

—

obstacle_motor_count_t

Motor count sentinel for motor handle availability checks.

Named constant to replace magic number 0 when checking whether motor_control_task_get_motors() returned a valid (non-empty) handle array. Compare the out_count value against k_obstacle_motor_count_none to decide whether to proceed or enter the fatal fail-safe path.

k_obstacle_motor_count_none == 0 (sentinel, never a valid motor count)

Value	Name	Description
= 0	k_obstacle_motor_count_none	Zero motors available motor_control_task_create() not yet called

See also: motor_control_task_get_motors() Returns this sentinel when motors not ready

Example:

```
uint8_tmotor_count=k_obstacle_motor_count_none;
rx_motor_handle_t**motors=motor_control_task_get_motors(&motor_count);
if(motors==nullptr||motor_count==k_obstacle_motor_count_none){
//Fatal:motorhandlesnotyetavailable
}
```

Since STAR v1.0.0

—

s_obstacle_threadTX_THREAD [s_obstacle_thread](#)

ThreadX thread control block for obstacle detection task.

Note: Never access directly - use ThreadX API (tx_thread_*)*Since Version 1.0.0*

—

s_obstacle_stack

uint8_t s_obstacle_stack[k_obstacle_task_stack_size]

Static thread stack for obstacle detection task.

Note: No dynamic allocation - all stack space pre-reserved at link time**Warning:** Stack overflow detection enabled (TX_ENABLE_STACK_CHECKING)*Since Version 1.0.0*

—

s_obstacle_createdbool [s_obstacle_created](#)

Task creation guard flag (prevents double-creation).

See also: obstacle_detect_task_create() Sets this flag on successful creation*Since Version 1.0.0*

—

s_obstacle_handlerx_obstacle_detect_t [s_obstacle_handle](#)

Obstacle detection system handle (library state).

Note: Treat as opaque - access only via rx_obstacle_detect_* API**Warning:** Do NOT access fields directly (breaks encapsulation)*Since Version 1.0.0*

—

s_tagconst char* const [s_tag](#)

Log tag for rx_log output (UART debug messages).

See also: rx_log.h UART logging macros**Example:**


```
[OBSTACLE]ObstacleDetectedBySensor1
[OBSTACLE]Distance(cm)29
[OBSTACLE]ObstacleDetectionRunning
```

Since Version 1.0.0

—

s_sensors

```
rx_hcsr04_t s_sensors[k_obstacle_sensor_count]
```

HC-SR04 sensor handle array (4 sensors for 360deg coverage).

Note: Statically allocated (no malloc) per NASA Power of 10 Rule 3

Note: Handles uninitialized until `internal_init_sensors()` completes successfully

Warning: Do NOT access handles from ISR context `rx_hcsr04_t` is not ISR-safe

Warning: Do NOT share handles across tasks without external synchronization

Since STAR v1.0.0

—

s_sensor_ptrs

```
rx_hcsr04_t* s_sensor_ptrs[k_obstacle_sensor_count]
```

Sensor handle pointers passed to `rx_obstacle_detect_init()`.

Note: Pointers reference `s_sensors` elements valid for program lifetime

Note: Array size matches `k_obstacle_sensor_count` (4) exactly

Warning: Do NOT modify pointer targets after `rx_obstacle_detect_init()` is called; the library retains these pointers for the duration of the polling task

Warning: Do NOT use after module teardown; there is no teardown path in this firmware

Since STAR v1.0.0

—

internal_obstacle_task_entry

```
static void internal_obstacle_task_entry(ULONG input)
```

Obstacle detection task entry point.

Main task loop for obstacle detection monitoring. This function executes in the obstacle detection task context (Priority 12) and performs three main activities:

1. Initialization Phase:

Configure rx_obstacle_detect library (4 sensors, 30 cm threshold, 3-sample debounce) Register internal_obstacle_callback() for obstacle events Start library polling (library creates internal task @ 50 Hz)

2. Monitoring Loop:

Retrieve statistics from rx_obstacle_detect library (total_polls, obstacle_events) Log statistics via UART for debugging/health monitoring (1 Hz rate) Sleep for 1 second (100 ticks @ 100 Hz tick rate)

3. Callback Handling (Parallel):

Callback executes in rx_obstacle internal task (Priority 10) Triggers emergency stop on obstacle detection Updates shared_data with obstacle state for telemetry

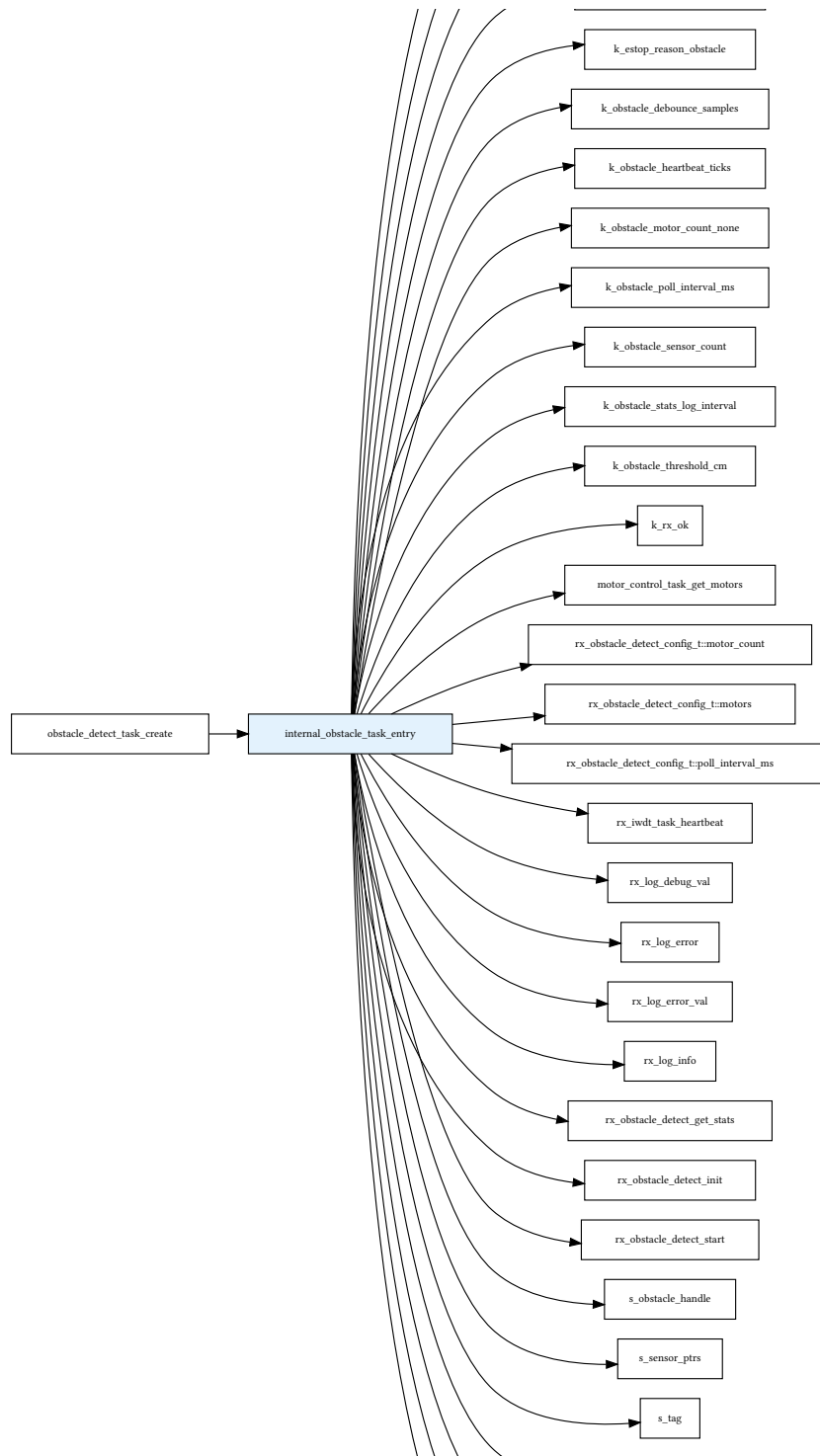


Figure 1423: Call graph for internal_obstacle_task_entry

Defined in src/tasks/obstacle_detect_task.c:1783

—

internal_obstacle_callback

```
static void internal_obstacle_callback(bool obstacle_detected, uint8_t
sensor_idx, float distance_cm, void *user_data)
```

Obstacle detection callback handler.

Callback function invoked by the rx_obstacle_detect library when an obstacle is detected (distance < 30 cm after 3-sample debounce) or cleared (distance > 30 cm). This callback executes in the rx_obstacle internal task context (Priority 10), NOT in the obstacle_detect_task context (Priority 12).

Callback Responsibilities:

1. Emergency Stop: Trigger e-stop via shared_data_trigger_estop() on detection
2. Event Signaling: Set k_event_obstacle_detected or k_event_obstacle_cleared
3. State Storage: Update obstacle_state_t in shared_data for telemetry
4. Logging: UART debug output (sensor index, distance)

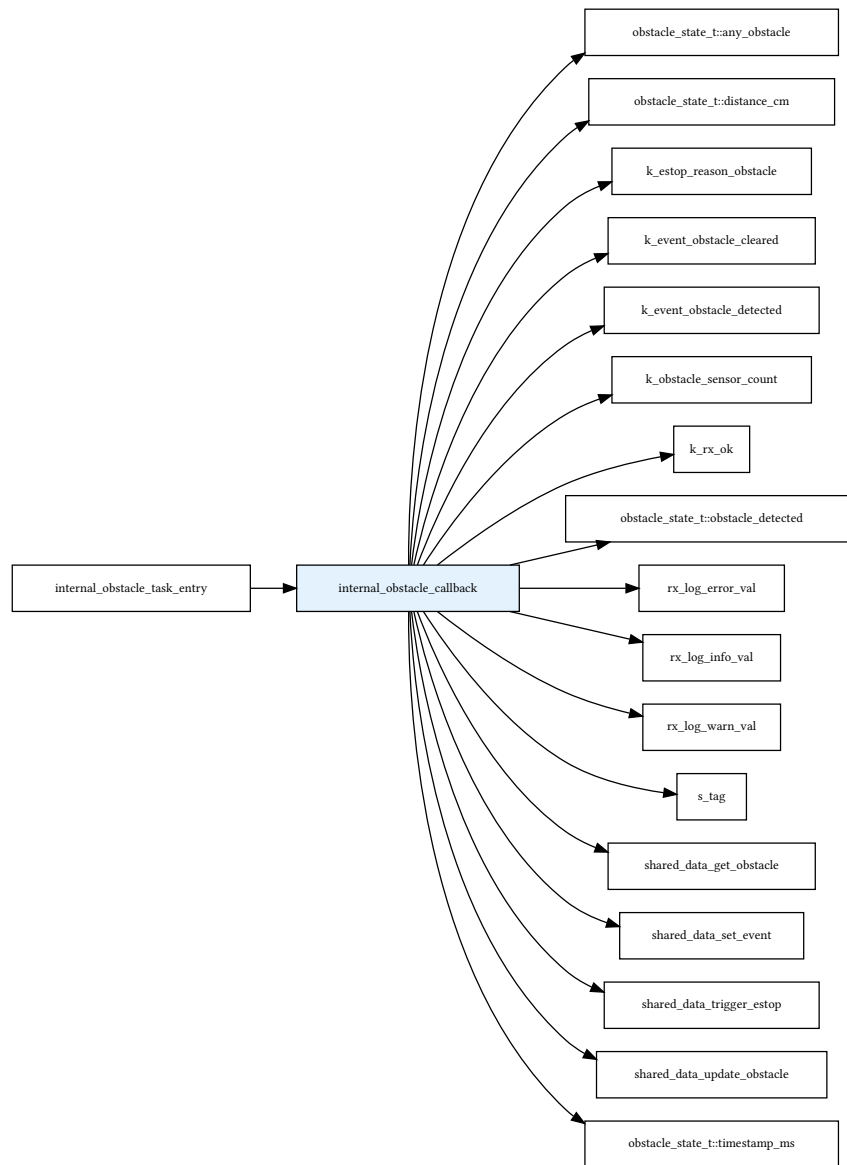
Callback Trigger Conditions:

- obstacle_detected = true:

Distance falls below 30 cm threshold 3 consecutive samples confirm detection (debouncing) Called once per detection event (not continuously while obstacle present)

- obstacle_detected = false:

Distance rises above 30 cm threshold 3 consecutive samples confirm clearance (debouncing) E-stop remains active (manual reset required)

Figure 1424: Call graph for `internal_obstacle_callback`

Defined in `src/tasks/obstacle_detect_task.c:2147`

—

obstacle_detect_task_create

```
rx_err_t obstacle_detect_task_create(void)
```

Create the obstacle detection task.

Create and start the obstacle detection task.

Initializes the obstacle detection system by creating a ThreadX task at medium priority (12) with a static 1024-byte stack. The task initializes 4 HC-SR04 ultrasonic sensors via the rx_obstacle_detect library and registers a callback to trigger emergency stop when obstacles are detected within 30 cm.

Architecture Pattern: Wrapper Task with Callback-Driven Library

- This task creates a ThreadX thread for monitoring/statistics
- rx_obstacle_detect library creates its own internal polling task
- Main work done by library (50 Hz polling, debouncing, distance calculation)
- This task's role: initialization, callback handling, statistics logging

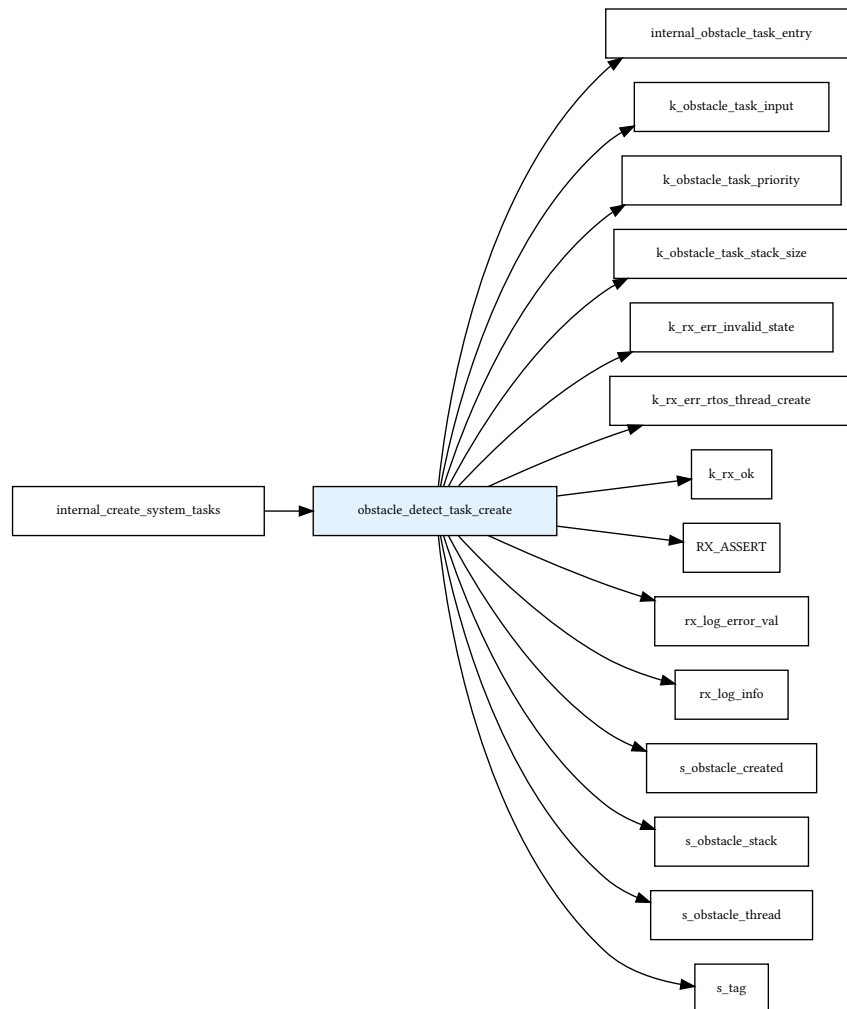


Figure 1425: Call graph for `obstacle_detect_task_create`

Defined in `src/tasks/obstacle_detect_task.c:1418`

telemetry_task.c

Telemetry Aggregation Task - Robot System State Broadcasting @ 20 Hz.

Includes:

- "telemetry_task.h"
- <string.h>
- "rx_check.h"
- "rx_comm_manager.h"
- "rx_frame.h"
- "rx_iwdt.h"
- "rx_log.h"
- "rx_nanopb.h"
- "shared_data.h"
- "tx_api.h"

telemetry_task_constants_t

Telemetry task configuration constants.

Defines all compile-time configuration parameters for the telemetry task. These values control task priority, timing, and buffer allocation.

All constants use explicit C23 typed enums with underlying type `uint16_t` for predictable memory layout and ABI stability (NASA Power of 10 Rule 8).

Value	Name	Description
= 2048	<code>k_telem_task_stack_size</code>	Stack size for telemetry task thread in bytes.
= 18	<code>k_telem_task_priority</code>	ThreadX task priority (18 = LOWEST application task).
= 5	<code>k_telem_task_sleep_ticks</code>	Sleep period in ThreadX ticks (5 ticks = 50ms @ 100 Hz).
= 0	<code>k_telem_task_input</code>	Thread entry input parameter (unused, always 0).
= 512	<code>k_telem_buffer_size</code>	Telemetry protobuf encode buffer size in bytes.

Since Version 1.0.0

—

telemetry_time_constants_t

Time conversion constants for timestamp calculation.

ThreadX system timer runs at 100 Hz (10ms per tick, configured in `tx_user.h`). Protocol Buffers telemetry schema requires timestamps in microseconds (us) for compatibility with ROS2 builtin_interfaces/Time (nanosecond precision).

Conversion formula:

Example:

- ThreadX ticks = 12345 (123.45 seconds uptime)
- `timestamp_us` = 12345 x 10000 = 123450000 us = 123.45 seconds

Value	Name	Description
= 10000	<code>k_telem_us_per_tick</code>	Microseconds per ThreadX tick (10000 us = 10 ms @ 100 Hz).

Note: All constants use C23 typed enums with explicit underlying type (NASA Rule 8)]

Example:

```
timestamp_us=tx_time_get()*k_telem_us_per_tick  
=ThreadX_ticks*10000
```

Since Version 1.0.0

—

telem_motor_idx_t

Motor indices for telemetry encoder data.

Value	Name	Description
= 0	k_telem_motor_front_left	Front left motor index
= 1	k_telem_motor_front_right	Front right motor index
= 2	k_telem_motor_back_left	Back left motor index
= 3	k_telem_motor_back_right	Back right motor index

Since Version 1.0.0

—

telem_fault_shift_t

Bit shift offsets for packing per-motor fault flags into uint32_t.

Value	Name	Description
= 0	k_fault_shift_front_left	Front left motor fault flags at bits [7:0]
= 8	k_fault_shift_front_right	Front right motor fault flags at bits [15:8]
= 16	k_fault_shift_back_left	Back left motor fault flags at bits [23:16]
= 24	k_fault_shift_back_right	Back right motor fault flags at bits [31:24]

Since Version 1.0.0

—

telem_sensor_idx_t

Temperature sensor indices for telemetry.

Value	Name	Description
= 0	k_telem_sensor_ambient	DS18B20 ambient temperature sensor index

Since Version 1.0.0

—

telemetry_transport_t

Active transport channel for telemetry transmission.

Identifies which physical communication channel is currently selected for telemetry broadcast. The selection algorithm prefers USB CDC when it is ready (development convenience and higher bandwidth) and falls back to SPI when USB is unavailable (production RPi5 connection).

Exactly one value selected per telemetry cycle

Value	Name	Description
= 0	k_telemetry_transport_usb	USB CDC channel (primary, preferred for development).
= 1	k_telemetry_transport_spi	SPI channel (fallback for production with RPi5).
= 2	k_telemetry_transport_none	No transport available (both USB and SPI unavailable).

See also: `internal_select_transport()` Returns this type,
`internal_build_and_send_telemetry()` Consumer of this type

Since Version 1.1.0

—

s_telem_thread

TX_THREAD `s_telem_thread`

ThreadX thread control block for telemetry task.

Note: Size: 140 bytes (TX_THREAD struct from ThreadX source)

Note: Statically allocated (NASA Power of 10 Rule 3 - no malloc)

Warning: NEVER modify this struct after `tx_thread_create()` - undefined behavior

Since Version 1.0.0

—

s_telem_stack

`uint8_t s_telem_stack[k_telem_task_stack_size]`

Static thread stack for telemetry task (2048 bytes).

Note: Size: 2048 bytes (5x safety margin over peak usage 400 bytes)

Note: Alignment: ThreadX automatically aligns to 8-byte boundary

Warning: Stack overflow causes memory corruption (nearby variables overwritten)

Warning: If peak usage exceeds 1500 bytes, increase `k_telem_task_stack_size` **Example:**

```
s_telem_stack[2047]<-Stackbase(initialSP)
|
v(growsdownduringfunctioncalls)
```

```
[...functionframes...]  
|  
s_telem_stack[0]<-Stacklimit(overflowifSPreacheshere)
```

Since Version 1.0.0

—

s_telem_created

```
bool s_telem_created
```

Task creation guard flag (prevents double-creation).

Note: This is a single-writer variable (only telemetry_task_create() writes)

Note: No mutex needed (task creation happens once at boot, single-threaded)

Warning: Do NOT manually set to false - task cannot be recreated after destruction

Since Version 1.0.0

—

s_telem_buffer

```
uint8_t s_telem_buffer[k_telem_buffer_size]
```

Telemetry protobuf encode buffer (512 bytes, static allocation).

Note: Statically allocated (NASA Power of 10 Rule 3 - no malloc)

Note: Single writer (telemetry task only), no mutex needed

Note: Zero-copy design: Buffer passed by pointer to comm_manager (no memcpy)

Warning: If encoding fails with k_rx_err_buffer_too_small, increase k_telem_buffer_size] **See also:** rx_nanopb_encode_telemetry() Encodes TelemetryData to this buffer, rx_comm_manager_send() Transmits buffer contents

Since Version 1.0.0

—

g_comm_manager

```
rx_comm_manager_t g_comm_manager
```

Communication manager handle (global for telemetry_task access).

Note: Declared extern - definition is in comm_task.c

Note: Shared across tasks (comm_task, telemetry_task, motor_control_task)

Warning: Do NOT call `rx_comm_manager_send()` before `comm_task_create()` completes] **See also:** `rx_comm_manager.h` Communication manager API, `comm_task.c` Initializes and manages `g_comm_manager`

Since Version 1.0.0

—

s_tag

`const char* const s_tag`

Log tag for telemetry task (used by `rx_log` macros).

Note: String stored in `.rodata` (flash), not RAM

Note: Total size: 8 bytes ("TELEM" + null terminator + padding)] **Example:**

```
#Viewonlytelemetrylogs
cat/dev/ttyUSB0|grep"$ TELEM $"
```

Since Version 1.0.0

—

s_sequence

`uint32_t s_sequence`

Telemetry message sequence counter (auto-incrementing).

Note: Single writer (telemetry task only), no mutex needed

Note: Wraps around every 49.7 days ($2^{32} / 20 \text{ Hz} / 86400 \text{ s/day}$)

Note: Host must handle wraparound: (`current_seq < last_seq`) -> wraparound detected]

Example:

```
if(current_seq!=last_seq+1){
uint32_tdropped=current_seq-last_seq-1;
printf("Dropped%utelemetrymessages\n",dropped);
}
```

Since Version 1.0.0

—

s_cdegc_per_degree

`const float s_cdegc_per_degree`

Conversion factor: millivolts per volt.

—

internal_telem_task_entry

```
static void internal_telem_task_entry(ULONG input)
```

Telemetry aggregation task entry point (infinite 20 Hz loop).

This is the main loop for the telemetry task. It executes continuously at 20 Hz, collecting robot system state from all data sources, encoding to Protocol Buffers, and transmitting to the host via USB CDC or SPI.

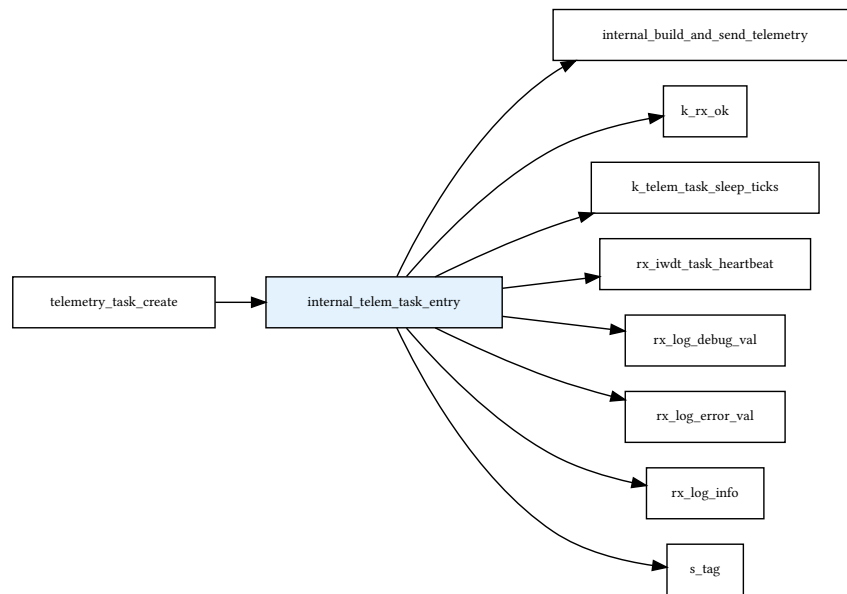


Figure 1426: Call graph for internal_telem_task_entry

_Defined in src/tasks/telemetry_task.c:1476_

internal_build_and_send_telemetry

```
static rx_err_t internal_build_and_send_telemetry(void)
```

Build and send complete telemetry message (4-phase aggregation orchestrator).

Orchestrates the complete telemetry aggregation pipeline across 4 focused helpers:

1. Timestamp + sequence - Set timestamp_us and increment s_sequence
2. internal_collect_state() - Phase 1+2: populate motor and temperature fields
3. internal_encode_telemetry() - Phase 3: nanopb encode to s_telem_buffer
4. internal_select_transport() - Phase 4a: USB preferred, SPI fallback
5. internal_send_via_channel() - Phase 4b: transmit via comm manager

Encoding failure is fatal for this cycle (message not sent, retry next cycle). Send failure is non-critical (message lost, host detects via sequence gap). No-transport drops the encoded message silently (both channels unavailable).

Partial messages are acceptable (Protocol Buffers optional fields)

Returns: rx_err_t Operation status

Value	Description
k_rx_ok	Telemetry sent successfully (all phases completed)
k_rx_err_encoding_failed	nanopb encoding failed (buffer too small, invalid message)
k_rx_err_invalid_state	No transport available (both channels unavailable)
k_rx_err_usb_tx_fail	USB CDC transmission failed (host disconnected)
k_rx_err_spi_tx_fail	SPI transmission failed (RPi5 not responding)
k_rx_err_timeout	Communication manager send timeout

Pre: Shared data initialized (shared_data_init() called)

Pre: Communication manager initialized (rx_comm_manager_init() called)

Pre: nanopb initialized (compile-time, no init function)

Pre: Task created and running (telemetry_task_create() called)

Post: Telemetry message sent via USB CDC (primary) or SPI (fallback)

Post: s_sequence incremented exactly once per call

Post: s_telem_buffer overwritten with latest encoded protobuf

Note: Called every 50ms by internal_telem_task_entry() (20 Hz)

Note: Execution time: 120 us (all phases combined)

Note: Data collection is non-blocking (graceful degradation if mutex locked)

Note: Transport channel selected dynamically each cycle (USB preferred)

Warning: If encoding fails, message is NOT sent (retry next cycle)

Warning: If transmission fails, message is lost (host detects via sequence gap)

NASA Power of 10 Rule 4 Compliance:: This function is now a 30-line orchestrator. Each phase delegated to a focused helper function of <=60 lines, all individually verifiable.

NASA Power of 10 Rule 5 Compliance:: Precondition 1: Shared data initialized Precondition 2: Comm manager initialized Postcondition 1: s_sequence incremented Postcondition 2: Returns error code for any fatal phase failure

See also: `internal_collect_state()` Phase 1+2 - Collect and populate all robot state,
`internal_populate_motor_telemetry()` Phase 2 - Motor encoder field population,
`internal_encode_telemetry()` Phase 3 - nanopb protobuf encoding,
`internal_select_transport()` Phase 4a - Select active transport channel,
`internal_send_via_channel()` Phase 4b - Transmit via comm manager

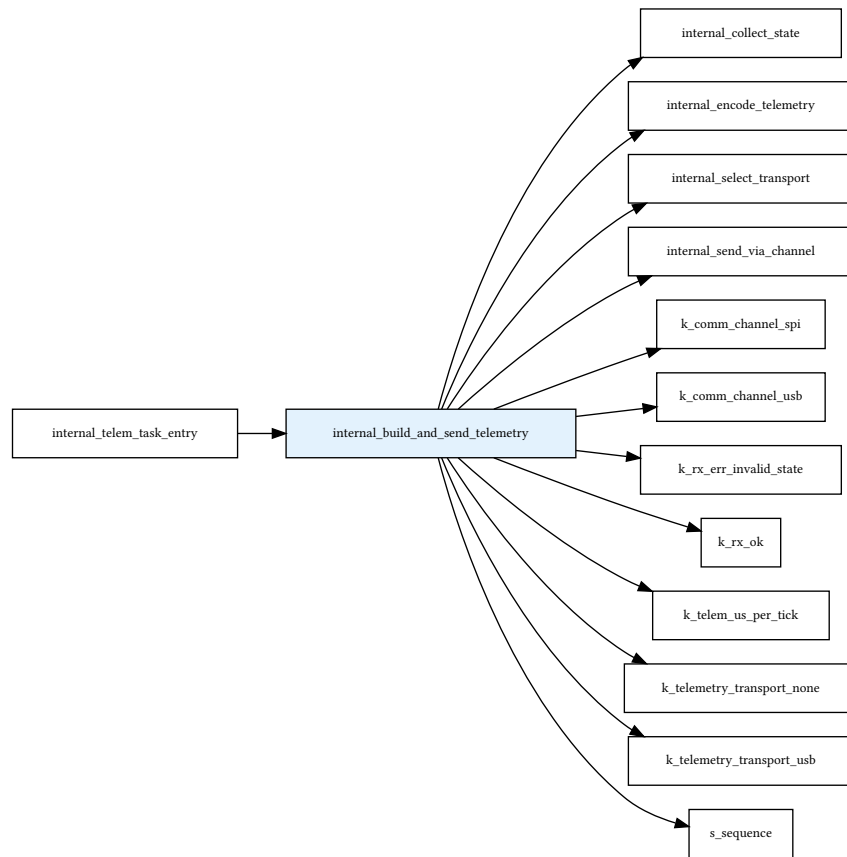


Figure 1427: Call graph for `internal_build_and_send_telemetry`

_Defined in `src/tasks/telemetry/task.c:1949`

Since Version 1.0.0 (refactored in v1.1.0 to delegate to focused helpers)

—

internal_select_transport

```
static telemetry_transport_t internal_select_transport(void)
```

Select the active transport channel for this telemetry cycle.

Queries the communication manager to determine which physical channel is ready for transmission. Implements the USB-preferred, SPI-fallback policy:

1. Query USB CDC readiness via `rx_comm_manager_channel_ready()`
2. If USB is ready, return `k_telemetry_transport_usb`
3. Otherwise query SPI readiness
4. If SPI is ready, return `k_telemetry_transport_spi` (and log the failover once per transition)
5. If neither is ready, return `k_telemetry_transport_none`

The function uses a static flag to detect the USB->SPI transition and emits a warning log exactly once per failover event (not once per cycle) to avoid log flooding at 20 Hz.

Returns: `telemetry_transport_t` Selected transport

Value	Description
<code>k_telemetry_transport_usb</code>	USB CDC ready, use as primary
<code>k_telemetry_transport_spi</code>	USB not ready, SPI available as fallback
<code>k_telemetry_transport_none</code>	Both channels unavailable, drop message

Pre: `g_comm_manager` must be initialized via `rx_comm_manager_init()`

Pre: `rx_comm_manager_channel_ready()` must be callable (non-blocking)

Post: Returned transport reflects real-time channel readiness

Post: Transition warning logged at most once per USB->SPI failover event

Note: Called every 50 ms from `internal_build_and_send_telemetry()` (20 Hz)

Note: Non-blocking: `rx_comm_manager_channel_ready()` never blocks

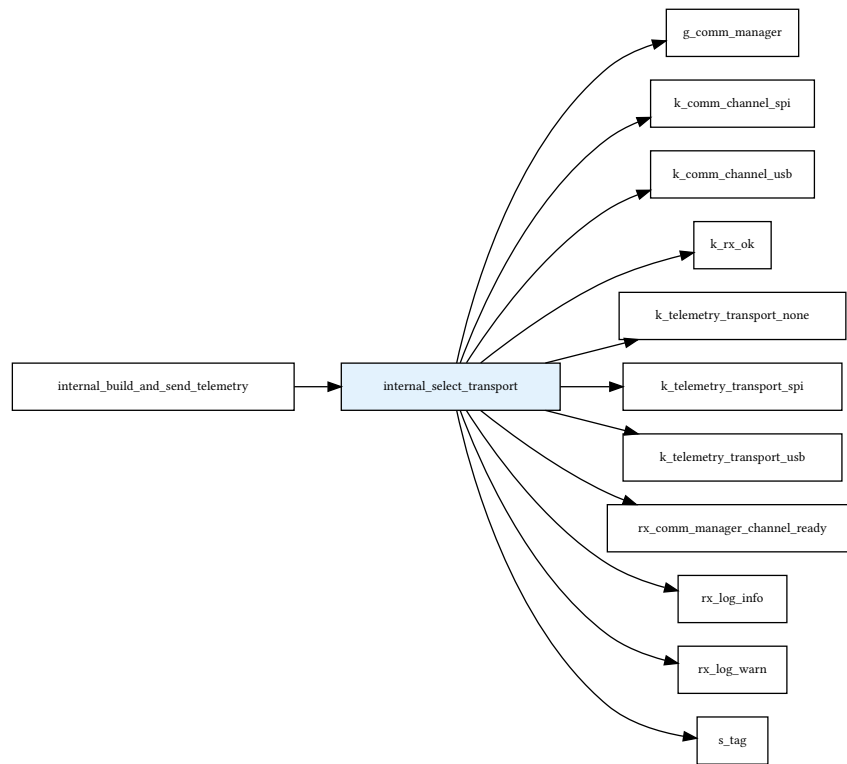
Note: Thread-safe: reads only `s_usb_was_active` (single writer: this task)

Warning: Returns `k_telemetry_transport_none` if `g_comm_manager` not initialized

Channel selection state machine::

NASA Power of 10 Rule 5 Compliance:: Precondition 1: `g_comm_manager` initialized (checked via `channel_ready` return) Precondition 2: channel argument within valid enum range Postcondition 1: Returns a valid `telemetry_transport_t` value Postcondition 2: Transition logged if `s_usb_was_active` changes

See also: `internal_build_and_send_telemetry()` Caller - maps transport to channel, `rx_comm_manager_channel_ready()` Readiness query API, `rx_comm_manager_channel_name()` Human-readable channel name for logs

Figure 1428: Call graph for `internal_select_transport`

_Defined in `src/tasks/telemetry\_task.c:1573_`

Since Version 1.1.0

—

internal_populate_motor_telemetry

```
static rx_err_t internal_populate_motor_telemetry(star_v1_TelemetryData
*telemetry)
```

Populate `TelemetryData` motor fields from shared motor state.

Reads the current motor state from `shared_data` and populates all motor-related fields in the telemetry protobuf message. If `shared_data` is unavailable (mutex locked), the motor fields are left as their zero-initialized defaults and the error is propagated to the caller for non-fatal handling.

Populated fields:

- `emergency_stop` - E-stop active flag
- `fault_flags` - 4 motor fault bytes packed into `uint32_t` bitfield
- `has_encoder_*` / `encoder_*` - 4 encoder submessages (`motor_id`, `ticks`, `velocity_mps`, `timestamp_us`) for `front_left`, `front_right`, `back_left`, `back_right`

Parameters:

Direction	Name	Description
inout	telemetry	TelemetryData struct to populate (must not be NULL)

Returns: rx_err_t Result from shared_data_get_motor_state()

Value	Description
k_rx_ok	Motor state read and fields populated
k_rx_err_timeout	Shared data mutex timed out (stale data, partial msg)
k_rx_err_null_ptr	NULL pointer passed (programming error)

Pre: telemetry must point to a valid star_v1_TelemetryData struct

Pre: telemetry->timestamp_us must already be set (used for encoder timestamps)

Post: If k_rx_ok: all motor encoder fields populated, has_encoder_* = true

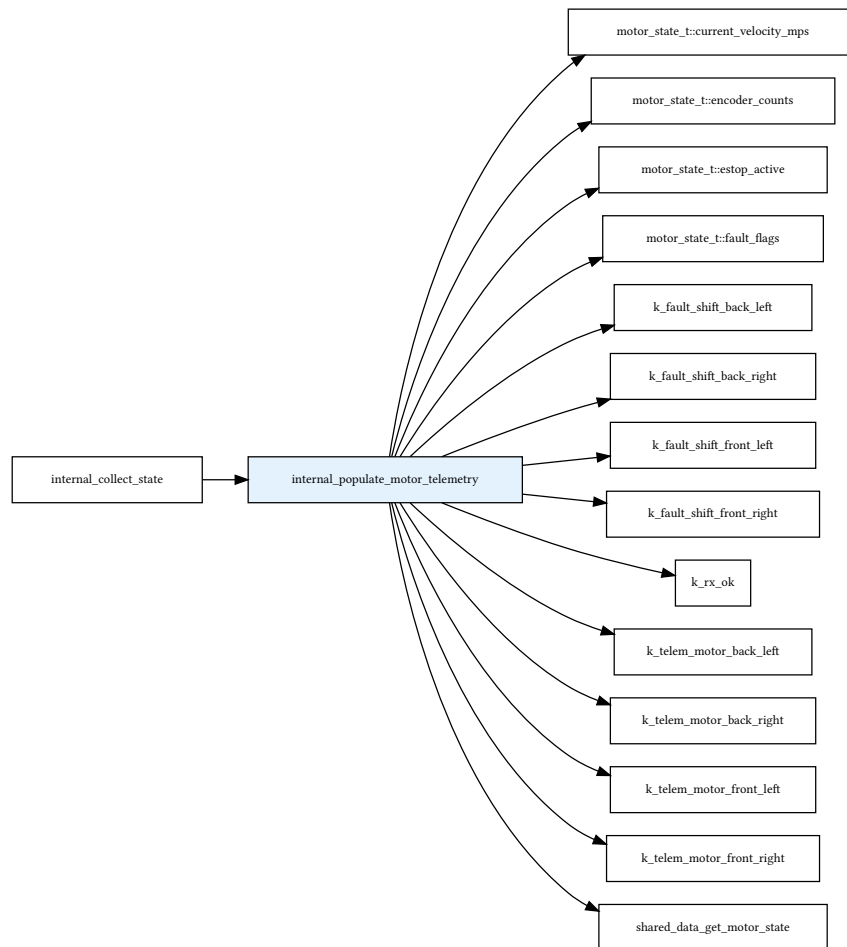
Post: If error: motor fields left at zero-init defaults, caller handles non-fatally

Note: Non-blocking: shared_data_get_motor_state() uses a non-blocking mutex try

Note: Error return is non-fatal for callers - partial telemetry is acceptable

NASA Power of 10 Rule 5 Compliance:: Precondition 1: telemetry != NULL Precondition 2: timestamp_us already set before call Postcondition 1: Returns shared_data_get_motor_state() result directly Postcondition 2: On k_rx_ok, has_encoder_* flags are true for all 4 encoders

See also: internal_collect_state() Caller - treats non-ok return as non-fatal, shared_data_get_motor_state() Shared data accessor

Figure 1429: Call graph for `internal_populate_motor_telemetry`_Defined in `src/tasks/telemetry\_task.c:1676_`*Since Version 1.1.0*

—

internal_collect_state

```
static void internal_collect_state(star_v1_TelemetryData *telemetry)
```

Collect all robot system state into the telemetry message (Phase 1 + Phase 2).

Reads motor and temperature state from `shared_data` and populates the corresponding fields in telemetry. All reads are non-blocking and non-fatal: if a data source is unavailable, the corresponding fields are left at zero-init defaults and collection continues for remaining sources.

Data collected:

1. Motor state (via `internal_populate_motor_telemetry()`): E-stop flag, `fault_flags` bitfield, 4 encoder submessages
2. Temperature state: `temperature_celsius` (`cdegC`->`degC`)

Parameters:

Direction	Name	Description
inout	<code>telemetry</code>	TelemetryData struct to populate (must not be NULL)

Pre: `telemetry` must point to a valid, zero-initialized `star_v1_TelemetryData` struct

Pre: `telemetry->timestamp_us` must already be set before this call

Post: `telemetry` fields populated for all available data sources

Post: Missing data sources leave their fields at zero-init defaults (graceful degradation)

Note: Always returns; partial population is intentional and acceptable

Note: Non-blocking: all `shared_data` accessors use non-blocking mutex try

Note: Thread-safe: `shared_data` accessors are mutex-protected

NASA Power of 10 Rule 5 Compliance:: Precondition 1: `telemetry != NULL` Precondition 2: `timestamp_us` set before call (for encoder sub-timestamps) Postcondition 1: Motor fields populated if `shared_data_get_motor_state() == k_rx_ok` Postcondition 2: Temp fields populated if `shared_data_get_temp() == k_rx_ok` && valid

See also: `internal_populate_motor_telemetry()` Motor field population helper, `shared_data_get_temp()` Temperature state accessor, `internal_build_and_send_telemetry()`
Caller - sets `timestamp` before calling

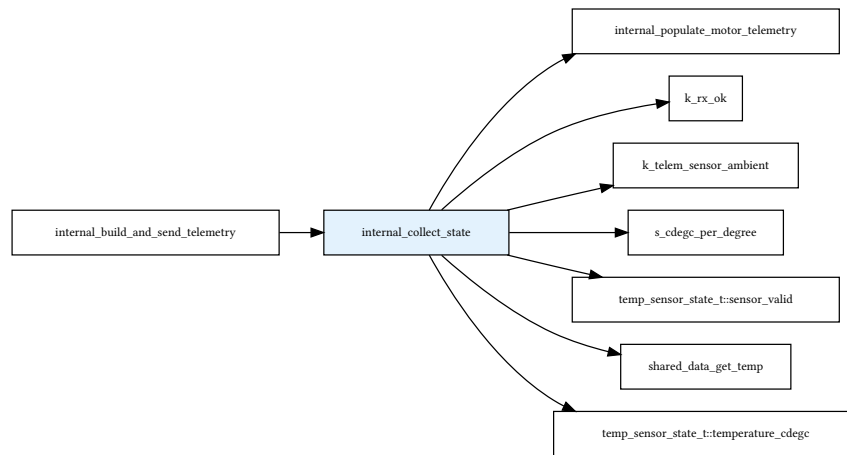


Figure 1430: Call graph for internal_collect_state

_Defined in src/tasks/telemetry_task.c:1768_

Since Version 1.1.0

—

internal_encode_telemetry

```
static rx_err_t internal_encode_telemetry(const star_v1_TelemetryData *telemetry,
uint32_t *out_encoded_len)
```

Encode TelemetryData protobuf message into the static transmit buffer (Phase 3).

Encodes the populated star_v1_TelemetryData struct into binary protobuf format using nanopb and writes to the static s_telem_buffer. The encoded length is returned via out_encoded_len for use by the transport layer.

Parameters:

Direction	Name	Description
in	telemetry	Populated TelemetryData message to encode
out	out_encoded_len	Encoded byte count written to s_telem_buffer

Returns: rx_err_t Encoding result

Value	Description
k_rx_ok	Encoding succeeded; s_telem_buffer contains *out_encoded_len valid bytes
k_rx_err_encoding_failed	nanopb encoding failed (buffer too small or invalid msg)

Pre: telemetry must point to a valid, populated star_v1_TelemetryData struct

Pre: out_encoded_len must not be NULL

Post: On k_rx_ok: s_telem_buffer[0..out_encoded_len-1] contains encoded protobuf

Post: On error: s_telem_buffer contents are undefined, error logged

Note: Zero-copy: passes s_telem_buffer pointer to nanopb (no intermediate copy)

Note: Not thread-safe: s_telem_buffer is exclusively owned by telemetry task

Warning: If encoding fails, message is NOT sent (caller returns error, retry next cycle)

NASA Power of 10 Rule 5 Compliance:: Precondition 1: telemetry != NULL Precondition 2: out_encoded_len != NULL Postcondition 1: Returns k_rx_err_encoding_failed on failure (error logged) Postcondition 2: *out_encoded_len valid only when k_rx_ok returned

See also: rx_nanopb_encode_telemetry() Underlying nanopb encoder, internal_build_and_send_telemetry() Orchestrator - calls this after collect_state

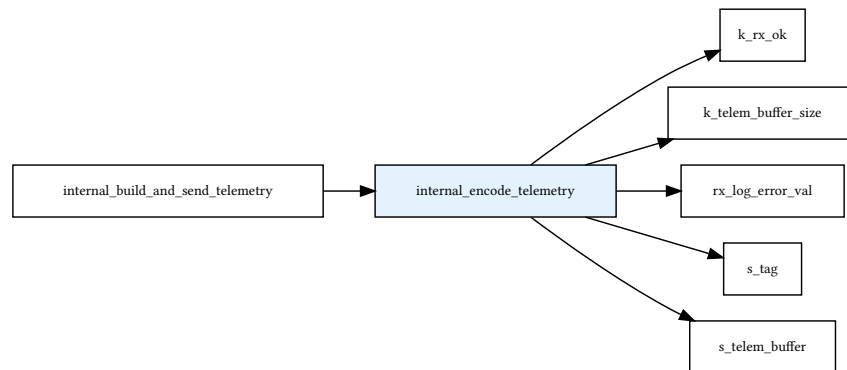


Figure 1431: Call graph for internal_encode_telemetry

_Defined in src/tasks/telemetry_task.c:1822_

Since Version 1.1.0

—

internal_send_via_channel

```
static rx_err_t internal_send_via_channel(rx_comm_channel_t channel, uint32_t
encoded_len)
```

Send the encoded telemetry buffer via the specified comm manager channel (Phase 4).

Builds `rx_comm_send_params_t` with `k_frame_type_response` (RX72N -> host data), `s_telem_buffer` as payload, and calls `rx_comm_manager_send()`. The channel argument is the caller-selected transport (USB or SPI) returned by `internal_select_transport()`.

Parameters:

Direction	Name	Description
in	channel	Comm manager channel to send on (k_comm_channel_usb or k_comm_channel_spi)
in	encoded_len	Number of bytes in s_telem_buffer to transmit

Returns: rx_err_t Send result from rx_comm_manager_send()

Value	Description
k_rx_ok	Message queued for transmission
k_rx_err_usb_tx_fail	USB CDC transmission failed (host disconnected mid-cycle)
k_rx_err_spi_tx_fail	SPI transmission failed (RPi5 not responding)
k_rx_err_timeout	Communication manager send queue full

Pre: channel must be k_comm_channel_usb or k_comm_channel_spi

Pre: encoded_len must be > 0 and <= k_telem_buffer_size

Pre: s_telem_buffer must contain valid encoded protobuf (internal_encode_telemetry() called)

Pre: g_comm_manager must be initialized (rx_comm_manager_init() called)

Post: On k_rx_ok: bytes queued in comm manager TX queue for DMA/interrupt transfer

Post: On error: message lost this cycle (host detects via sequence gap)

Note: Send failure is non-critical: telemetry task logs and continues to next cycle

Note: CRC added by comm_manager (not included in encoded_len)

Note: Zero-copy: s_telem_buffer passed by pointer, no intermediate memcpy

NASA Power of 10 Rule 5 Compliance:: Precondition 1: channel is a valid rx_comm_channel_t value Precondition 2: encoded_len > 0 (caller verified encoding succeeded) Postcondition 1: Returns rx_comm_manager_send() result directly Postcondition 2: s_telem_buffer remains valid after call (no modification)

See also: internal_build_and_send_telemetry() Caller - provides channel from select_transport, rx_comm_manager_send() Underlying send implementation, internal_select_transport() Channel selection (USB preferred, SPI fallback)

Figure 1432: Call graph for `internal_send_via_channel`

Defined in `src/tasks/telemetry\_task.c:1876`

Since Version 1.1.0

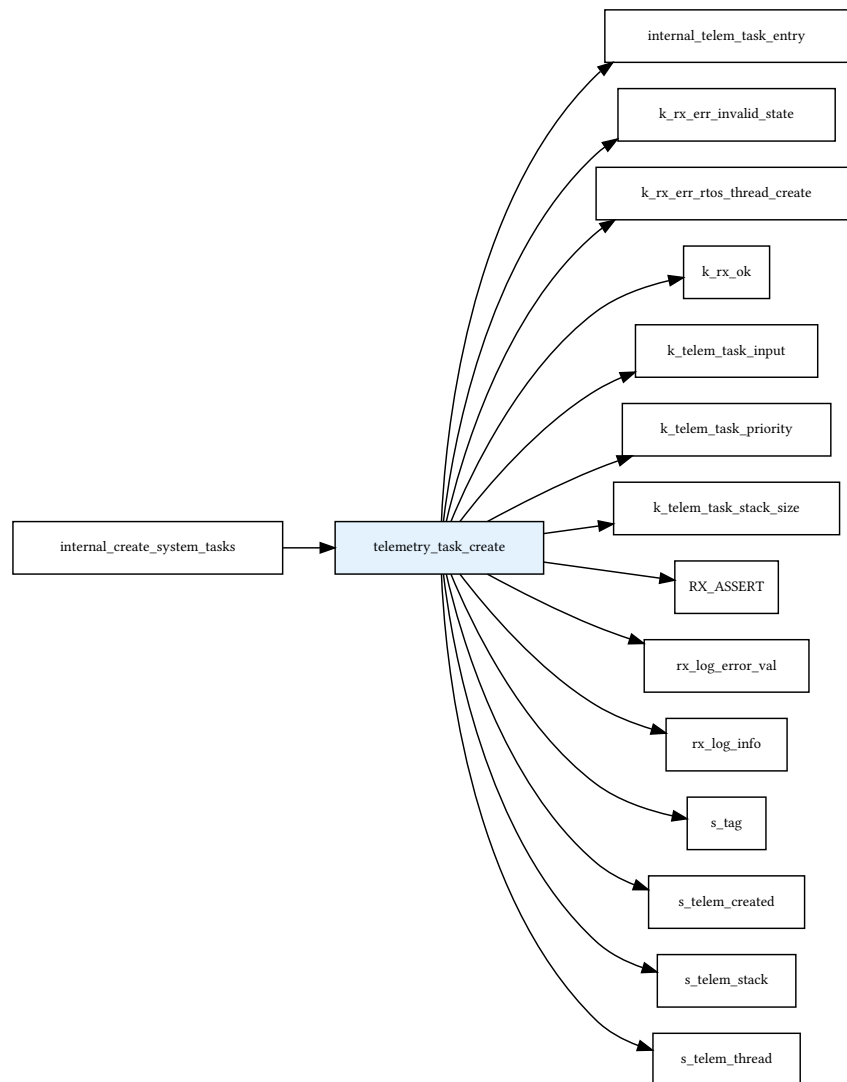
—

telemetry_task_create

```
rx_err_t telemetry_task_create(void)
```

Create and start the telemetry aggregation task.

Initializes the telemetry task thread and starts it with auto-start enabled. The task immediately begins running after this function returns (unless a higher-priority task is ready to run).

Figure 1433: Call graph for `telemetry_task_create`_Defined in `src/tasks/telemetry\_task.c:1276_`**temp_sensor_task.c**

Temperature Sensor Task Implementation - DS18B20 Ambient Temperature for Ultrasonic Compensation.

Includes:

- `"temp_sensor_task.h"`
- `"rx_check.h"`
- `"rx_ds18b20.h"`

- "rx_iwdt.h"
- "rx_log.h"
- "shared_data.h"
- "tx_api.h"

temp_task_constants_t

Temperature sensor task configuration constants.

Value	Name	Description
= 1024	k_temp_task_stack_size	Stack size in bytes
= 15	k_temp_task_priority	ThreadX priority (low)
= 0	k_temp_task_input	Thread entry input parameter
= 80	k_temp_conversion_ticks	800ms for 12-bit conversion
= 20	k_temp_remaining_ticks	200ms remaining in 1s period

—

temp_sensor_constants_t

Temperature sensor configuration.

Value	Name	Description
= 1	k_temp_sensor_count	Number of DS18B20 sensors
= 0	k_temp_sensor_idx	Primary sensor index

—

s_temp_thread

TX_THREAD [s_temp_thread](#)

ThreadX thread control block.

—

s_temp_stack

[uint8_t](#) [s_temp_stack](#)[k_temp_task_stack_size]

Static thread stack (no dynamic allocation).

—

s_temp_created

[bool](#) [s_temp_created](#)

Task creation guard flag.

—

s_ds18b20

[rx_ds18b20_handle_t](#) [s_ds18b20](#)

DS18B20 sensor handle.

—

s_tag

```
const char* const s_tag
```

Log tag for this module.

—

g_bus_manager

```
rx_bus_manager_t g_bus_manager
```

Bus manager handle (assumed initialized elsewhere).

Note: Do NOT access this variable directly from tasks. Use `rx_bus_manager_get_bus()`.] **See also:** `rx_bus_manager_init()` Initialize bus manager (in `hardware_init.c`), `rx_bus_types.h` Bus abstraction API

Example:

```
rx_bus_interface_t*i2c=rx_bus_manager_get_bus(&g_bus_manager,k_rx_bus_type_i2c);
i2c->read(i2c->ctx,imu_addr,data,len);
```

—

s_owewire_bus_name

```
const char* const s_owewire_bus_name
```

1-Wire bus name for DS18B20

—

s_cdegc_per_degree

```
const float s_cdegc_per_degree
```

Conversion factor: centi-degrees Celsius per degree Celsius.

—

internal_send_iwdt_heartbeat

```
static void internal_send_iwdt_heartbeat(void)
```

Send IWDT task heartbeat to prevent watchdog timeout.

Calls `rx_iwdt_task_heartbeat()` with the “TempSensor” task identifier and logs any failure. Called once per temperature monitoring cycle from `internal_temp_task_entry()`. Extracted to keep `internal_temp_task_entry()` under the 60-line NASA Rule 4 target.

Pre: IWDT subsystem initialized

Post: Heartbeat sent if IWDT operational

Post: Error logged if heartbeat fails (watchdog monitor will detect timeout)

Note: Not thread-safe; called only from `internal_temp_task_entry()`] **See also:**
`rx_iwdt_task_heartbeat()` IWDT heartbeat API, `internal_temp_task_entry()` Caller

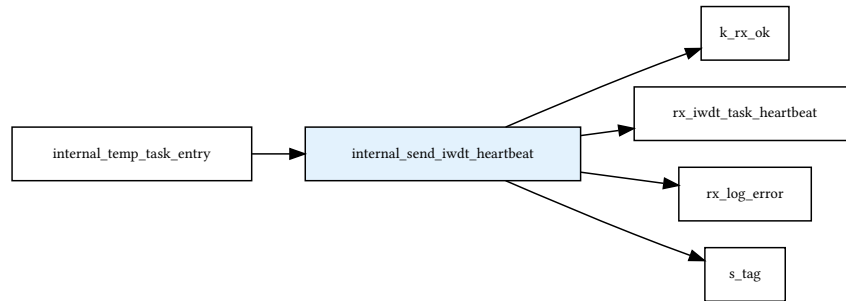


Figure 1434: Call graph for `internal_send_iwdt_heartbeat`

Defined in `src/tasks/temp_sensor_task.c:749`

Since Version 1.0.0

—

internal_temp_task_entry

```
static void internal_temp_task_entry(ULONG input)
```

Temperature sensor task entry point - infinite loop polling DS18B20 at 1 Hz.

This is the main task loop that executes after `temp_sensor_task_create()` starts the thread. The function never returns and runs continuously at 1 Hz polling rate until power-down or emergency stop.

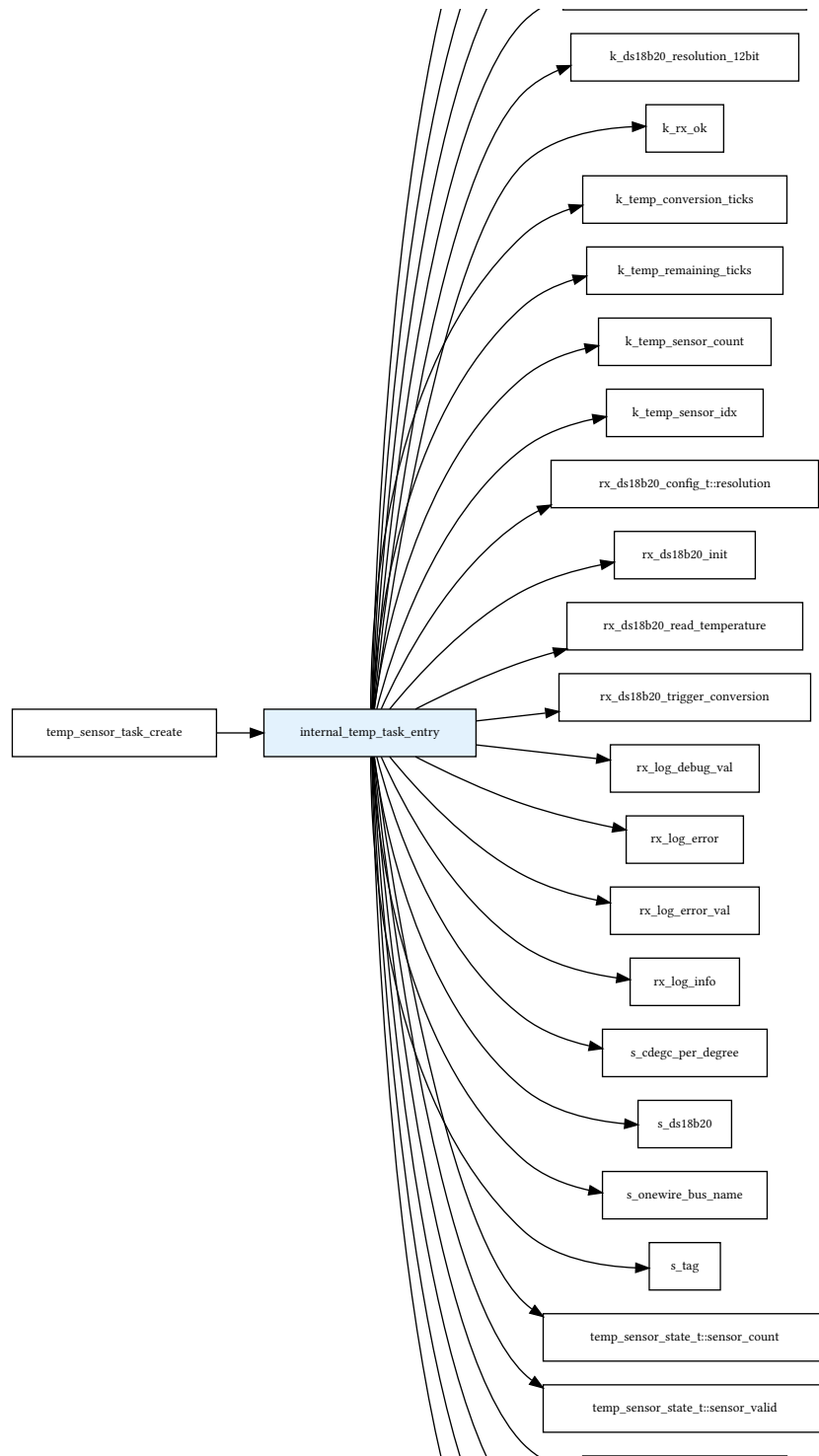


Figure 1435: Call graph for internal_temp_task_entry

Defined in src/tasks/temp_sensor_task.c:1171

—

temp_sensor_task_create

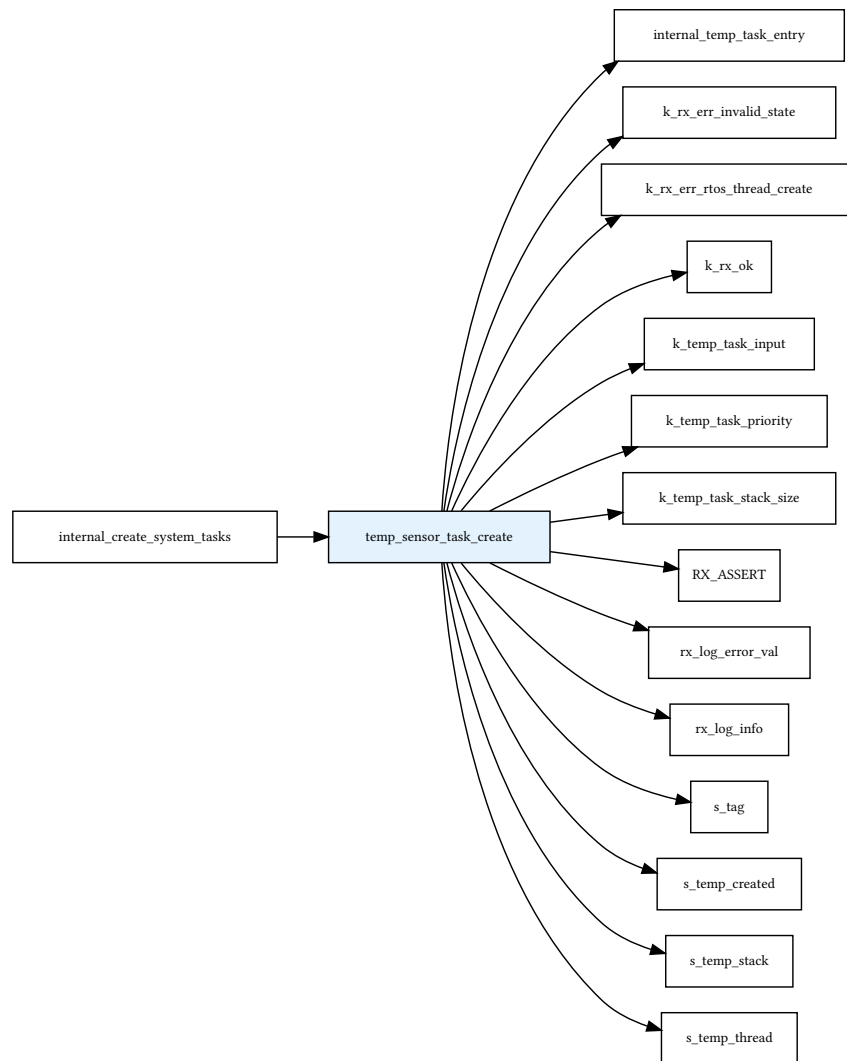
```
rx_err_t temp_sensor_task_create(void)
```

Create the temperature sensor task for DS18B20 ambient temperature monitoring.

Create and start the temperature sensor task.

Creates and starts the temperature sensor ThreadX task with low priority (Priority 15) for non-critical ambient temperature monitoring at 1 Hz. This function allocates the thread control block, assigns a static stack, and registers the task with ThreadX.

The temperature data is used by the obstacle detection task to compensate for air temperature variations in HC-SR04 ultrasonic sensor readings. Accurate temperature measurement is essential for precise distance calculation, as the speed of sound in air varies by 0.6 m/s per degC.

Figure 1436: Call graph for `temp_sensor_task_create`

Defined in `src/tasks/temp_sensor_task.c:693`

watchdog_monitor_task.c

Watchdog Monitor Task Implementation.

Implements a high-priority ThreadX task that provides system-level watchdog supervision by monitoring all registered tasks for liveness and feeding the hardware Independent Watchdog Timer (IWDI).

Includes:

- "watchdog_monitor_task.h"

- "rx_check.h"
- "rx_iwdt.h"
- "rx_log.h"
- "tx_api.h"

watchdog_task_constants_t

Watchdog monitor task configuration constants.

Defines ThreadX task parameters for the watchdog monitor task, which supervises all registered tasks by checking heartbeats and feeding the hardware Independent Watchdog Timer (IWDT) at 100 Hz. The task runs at high priority (6) between Communication (5) and Motor Control (8) to ensure timely watchdog feeding.

Task Characteristics:

- Stack: 512 bytes (minimal - no complex logic, just checks and feeds)
- Priority: 6 (high - ensures IWDT fed even under heavy load)
- Period: 10ms (100 Hz - provides 200x safety margin for 2048ms IWDT timeout)
- CPU Utilization: < 0.01% (extremely lightweight)

Design Rationale:

- Small stack sufficient for heartbeat checks and IWDT feed operations
- High priority prevents preemption during safety-critical watchdog operations
- 100 Hz rate ensures hardware watchdog fed with 200x margin (2048ms / 10ms)

Value	Name	Description
= 512	k_watchdog_task_stack_size	Stack size (512 bytes). Minimal allocation for lightweight supervision loop. No recursion, no large locals. Valid range: 256-1024 bytes.
= 6	k_watchdog_task_priority	ThreadX priority (6). High priority ensures timely IWDT feeding. Between Communication (5) and Motor Control (8). Valid range: 1-15 (1=highest).
= 0	k_watchdog_task_input	Thread entry input parameter (0). Unused by watchdog monitor task. ThreadX convention for parameterless tasks.
= 1	k_watchdog_task_period_ticks	Task period (1 tick = 10ms @ 100 Hz). Watchdog check and feed rate. Provides 200x safety margin for 2048ms hardware IWDT timeout. Valid range: 1-10 ticks (10-100ms).

Note: All size values in bytes, periods in ThreadX ticks (10ms @ 100 Hz)] **See also:** `watchdog_monitor_task_create()` Task creation using these constants, `internal_watchdog_monitor_task_entry()` Main loop running at configured period

Since Version 1.0.0

—

s_watchdog_thread

TX_THREAD [s_watchdog_thread](#)

ThreadX thread control block for watchdog monitor task.

Note: Internal ThreadX structure - do not modify directly

Warning: Must remain in scope for task lifetime (static storage)] **See also:** watchdog_monitor_task_create() Initializes this control block

Since Version 1.0.0

—

s_watchdog_stack

`uint8_t s_watchdog_stack[k_watchdog_task_stack_size]`

Static thread stack for watchdog monitor task (512 bytes).

Note: NASA Power of 10 Rule 3: No dynamic memory allocation (static array)

Warning: Stack overflow triggers undefined behavior - monitor in debug builds

Since Version 1.0.0

—

s_watchdog_created

`bool s_watchdog_created`

Task creation guard flag - prevents duplicate task creation.

Note: Not thread-safe - watchdog_monitor_task_create() must only be called from tx_application_define() (single-threaded initialization context)

Since Version 1.0.0

—

s_tag

`const char* const s_tag`

Log tag for watchdog monitor module ("IWDT_MON").

Note: Stored in .rodata section (const string literal)] **See also:** rx_log_info() Log informational messages with this tag, rx_log_error() Log error messages with this tag

Since Version 1.0.0

—

s_task_name

`const char* const s_task_name`

IWDT task name for thread creation and heartbeat registration.

Note: String must match IWDT registration for heartbeat correlation

Warning: Do not modify - IWDT system depends on exact string match] **See also:**
watchdog_monitor_task_create() Thread creation using this name,
internal_watchdog_monitor_task_entry() Heartbeat reporting using this name

Since Version 1.0.0

—

internal_watchdog_monitor_task_entry

```
static void internal_watchdog_monitor_task_entry(ULONG input)
```

Watchdog monitor task entry point - infinite loop at 100 Hz.

Main supervision loop that executes three critical operations each iteration:

1. Check all registered tasks for heartbeat timeouts
2. Feed hardware watchdog (prevents 2s timeout reset)
3. Report own heartbeat (self-monitoring)

Failure Handling:

- If task timeout detected: Log error, STOP feeding watchdog
- System will reset after 2s (hardware IWDT timeout)
- Failed task name preserved for post-mortem analysis

Timing:

- 100 Hz execution rate (10ms period)
- Feed margin: 2000ms / 10ms = 200x safety factor
- Can tolerate 200 missed iterations before reset

Parameters:

Direction	Name	Description
in	input	Thread input parameter (unused, always 0)

Returns: void Function never returns (infinite loop)

Pre: watchdog_monitor_task_create() called successfully

Pre: ThreadX scheduler started

Pre: IWDT initialized

Post: Infinite loop - never returns

Post: Hardware watchdog fed at 100 Hz

Post: Task timeouts detected and logged

Note: Thread Safety: Safe from single task context

Usage:: tx_thread_create(&s_watchdog_thread, "WatchdogMon",
internal_watchdog_monitor_task_entry, k_watchdog_task_input, s_watchdog_stack,
k_watchdog_task_stack_size, k_watchdog_task_priority, k_watchdog_task_priority,
TX_NO_TIME_SLICE, TX_AUTO_START);

See also: watchdog_monitor_task_create() Creates and starts this task entry,
rx_iwtdt_check_tasks() Heartbeat timeout detection called each iteration,
rx_iwtdt_feed() Hardware watchdog feed called when all tasks healthy,
rx_iwtdt_task_heartbeat() Self-monitoring heartbeat reported each iteration



Figure 1437: Call graph for `internal_watchdog_monitor_task_entry`

Defined in `src/tasks/watchdog_monitor_task.c:404`

Since Version 1.0.0

—

watchdog_monitor_task_create

```
rx_err_t watchdog_monitor_task_create(void)
```

Create the watchdog monitor task.

Create and start the watchdog monitor task.

Creates and starts the watchdog monitor ThreadX task with high priority (6) for system health supervision at 100 Hz.

Returns: rx_err_t Error code

Value	Description
k_rx_ok	Task created successfully
k_rx_err_invalid_state	Already created
k_rx_err_rtos_thread_create	ThreadX error

Pre: ThreadX kernel running

Pre: rx_iwdt_init() called successfully

Pre: All tasks registered via rx_iwdt_register_task()

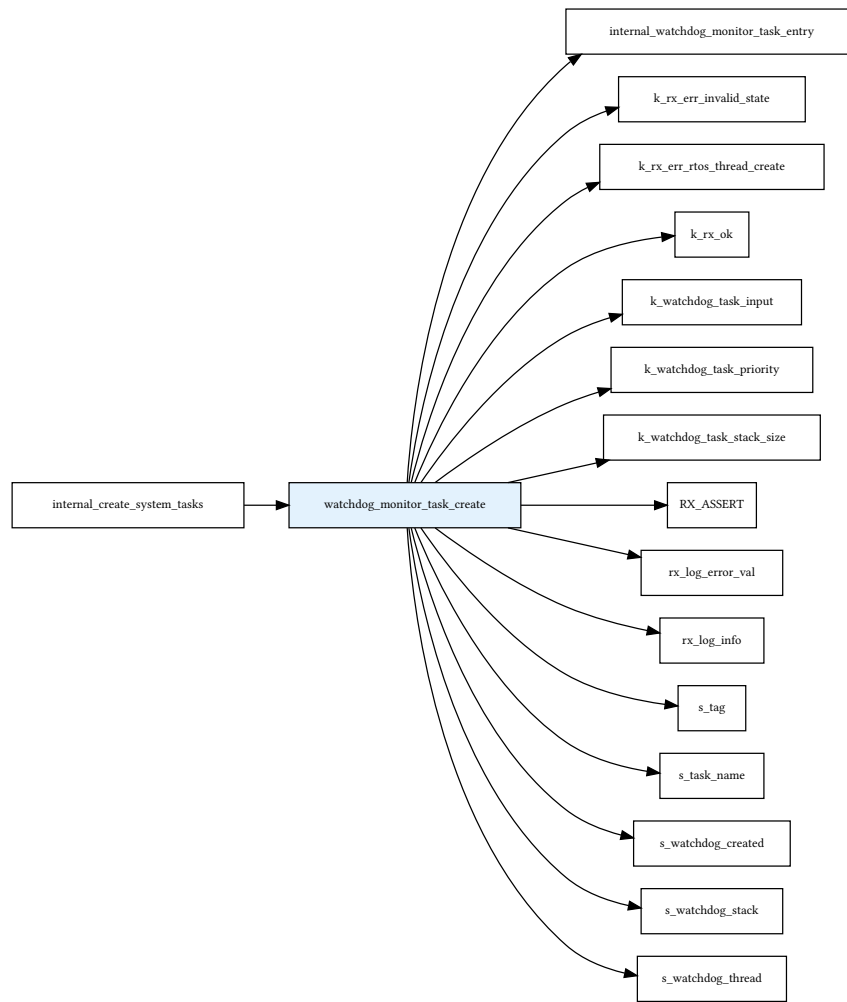
Post: WatchdogMon created and running at 100 Hz

Post: s_watchdog_created set to true

Note: Thread Safety: Not thread-safe, call from tx_application_define only

Usage Example:: rx_err_t err;

```
err=rx_iwdt_init(&s_iwdt_config); RX_ASSERT(err==k_rx_ok,"IWDTinitfailed");  
err=rx_iwdt_register_task("MotorCtrl",30); RX_ASSERT(err==k_rx_ok,"Taskregistrationfailed");  
err=rx_iwdt_set_state(k_system_state_init); RX_ASSERT(err==k_rx_ok,"Setstatefailed");  
err=watchdog_monitor_task_create(); RX_ASSERT(err==k_rx_ok,"Watchdogtaskcreationfailed");
```

Figure 1438: Call graph for `watchdog_monitor_task_create`

Defined in `src/tasks/watchdog_monitor_task.c:313`

Since Version 1.0.0

—

Index

—

- `_tx_timer_interrupt` (function, timer.c)

A

- `ack_wait_t` (enum, rx_spi_comm.c)
- `adc_adcer_bits_t` (enum, adc.c)
- `adc_adcsr_bits_t` (enum, adc.c)

- `adc_bit_constants_t` (enum, `adc.c`)
- `adc_channel_boundaries_t` (enum, `adc.c`)
- `adc_channel_t` (enum, `hardware.h`)
- `adc_constants_t` (enum, `adc.c`)
- `adc_count_t` (enum, `hardware_init.c`)
- `adc_init` (function, `hardware.h`)
- `adc_init` (function, `adc.c`)
- `adc_init_channels` (function, `hardware_init.c`)
- `adc_misc_constants_t` (enum, `adc.c`)
- `adc_module_stop_bits_t` (enum, `adc.c`)
- `adc_raw_value_t` (enum, `adc.c`)
- `adc_read` (function, `hardware.h`)
- `adc_read` (function, `adc.c`)
- `adc_read_voltage_mv` (function, `hardware.h`)
- `adc_read_voltage_mv` (function, `adc.c`)
- `adc_resolution_t` (typedef, `hardware.h`)
- `adc_unit_t` (enum, `hardware.h`)
- `adc_voltage_limits_t` (enum, `rx_bus_adc.c`)
- `alr1_bits_t` (enum, `rx72n_poe3_regs.h`)

B

- `backoff_limit_t` (enum, `rx_spi_comm.c`)
- `bit_constants_t` (enum, `rx_harq.c`)
- `bit_masks_t` (enum, `rx_bus_types.h`)
- `bit_shift_t` (enum, `rx_usb_cdc.c`)
- `brr_constants_t` (enum, `uart.c`)
- `bsp_get_bpc` (function, `r_rx_intrinsic_functions.c`)
- `bsp_get_bpsw` (function, `r_rx_intrinsic_functions.c`)
- `bsp_move_from_acc_hi_long` (function, `r_rx_intrinsic_functions.c`)
- `bsp_move_from_acc_mi_long` (function, `r_rx_intrinsic_functions.c`)
- `buffer_size_constants_t` (enum, `rx_frame_ascii.c`)
- `bus_manager_limits_t` (enum, `rx_bus_types.h`)
- `bus_manager_timeout_t` (enum, `rx_bus_types.h`)
- `byte_mask_t` (enum, `rx_usb_cdc.c`)
- `byte_shift_t` (enum, `rx_frame.c`)

C

- `cdc_acm_capabilities_t` (enum, `rx_usb_cdc.c`)
- `cdc_descriptor_subtype_t` (enum, `rx_usb_cdc.c`)
- `cdc_line_coding_index_t` (enum, `rx_usb_cdc.c`)
- `cdc_request_code_t` (enum, `rx_usb_cdc.c`)
- `CHECK_DS18B20_HANDLE` (macro, `rx_ds18b20.c`)
- `CHECK_ONEWIRE_BUS` (macro, `rx_bus_onewire.c`)
- `ckocr_bits_t` (enum, `rx72n_system_regs.h`)
- `cmt0_cmcr_values_t` (enum, `timer.c`)
- `cmt0_cmstr_bits_t` (enum, `timer.c`)
- `cmt0_config_t` (enum, `timer.c`)
- `cmt0_counter_values_t` (enum, `timer.c`)
- `cmt0_ier_constants_t` (enum, `timer.c`)

- `cmt0_isr` (function, `timer.c`)
- `cmt1_isr` (function, `rx_cmt.c`)
- `cmt2_isr` (function, `rx_cmt.c`)
- `cmt2_timing_constants_t` (enum, `rx_hcsr04_hal_hw.c`)
- `cmt3_isr` (function, `rx_cmt.c`)
- `cmt_bit_constants_t` (enum, `rx_cmt.c`)
- `cmt_cmcr_bits_t` (enum, `rx_cmt.c`)
- `cmt_cmstr_bits_t` (enum, `rx_cmt.c`)
- `cmt_constants_t` (enum, `rx_cmt.c`)
- `cmt_divider_values_t` (enum, `rx_cmt.c`)
- `cmt_module_stop_bits_t` (enum, `rx_cmt.c`)
- `cmt_period_constants_t` (enum, `rx_cmt.c`)
- `comm_motor_constants_t` (enum, `comm_task.c`)
- `comm_task_constants_t` (enum, `comm_task.c`)
- `comm_task_create` (function, `comm_task.c`)
- `comm_task_create` (function, `comm_task.h`)
- `composite_config_size_t` (enum, `rx_usb_cdc.c`)
- `crc_regs` (function, `rx72n_crc_regs.h`)
- `ctrl_dispatch_result_t` (enum, `rx_spi_comm.c`)

D

- `decimal_buf_constants_t` (enum, `rx_frame_ascii.c`)
- `descriptor_type` (variable, `rx_usb_cdc.c`)
- `dmac0` (function, `rx72n_dmac_regs.h`)
- `dmac1` (function, `rx72n_dmac_regs.h`)
- `dmac2` (function, `rx72n_dmac_regs.h`)
- `dmac3` (function, `rx72n_dmac_regs.h`)
- `dmac4` (function, `rx72n_dmac_regs.h`)
- `dmac5` (function, `rx72n_dmac_regs.h`)
- `dmac6` (function, `rx72n_dmac_regs.h`)
- `dmac7` (function, `rx72n_dmac_regs.h`)
- `dmac_addresses_t` (enum, `rx72n_dmac_regs.h`)
- `dmac_channel` (function, `rx72n_dmac_regs.h`)
- `dmac_channel_t` (enum, `rx72n_dmac_regs.h`)
- `dmac_dmast_reg` (function, `rx72n_dmac_regs.h`)
- `dmac_dmist_reg` (function, `rx72n_dmac_regs.h`)
- `dmac_mstpcr_t` (enum, `rx72n_dmac_regs.h`)
- `dmac_offsets_t` (enum, `rx72n_dmac_regs.h`)
- `dmamd_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmast_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmcnt_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmcs1_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmint_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmist_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmreq_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmsts_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dmtmd_bits_t` (enum, `rx72n_dmac_regs.h`)
- `dps_regs` (function, `rx72n_lpc_regs.h`)
- `dpsbkr` (function, `rx72n_lpc_regs.h`)

- ds18b20_command_t (enum, rx_ds18b20.h)
- ds18b20_config_bits_t (enum, rx_ds18b20.h)
- ds18b20_conversion_constants_t (enum, rx_ds18b20.h)
- ds18b20_conversion_time_ms_t (enum, rx_ds18b20.h)
- ds18b20_init_values_t (enum, rx_ds18b20.h)
- ds18b20_power_mode_t (enum, rx_ds18b20.h)
- ds18b20_resolution_t (enum, rx_ds18b20.h)
- ds18b20_scratchpad_index_t (enum, rx_ds18b20.h)
- ds18b20_temp_mask_t (enum, rx_ds18b20.h)
- ds18b20_write_idx_t (enum, rx_ds18b20.h)
- dtc_addresses_t (enum, rx72n_dtc_regs.h)
- dtc_dtcadmod_reg (function, rx72n_dtc_regs.h)
- dtc_dtccr_reg (function, rx72n_dtc_regs.h)
- dtc_dtcdisp_reg (function, rx72n_dtc_regs.h)
- dtc_dtcibr_reg (function, rx72n_dtc_regs.h)
- dtc_dtcpr_reg (function, rx72n_dtc_regs.h)
- dtc_dtcscqe_reg (function, rx72n_dtc_regs.h)
- dtc_dtcst_reg (function, rx72n_dtc_regs.h)
- dtc_dtcsts_reg (function, rx72n_dtc_regs.h)
- dtc_dtcvbr_reg (function, rx72n_dtc_regs.h)
- dtc_mra_bits_t (enum, rx72n_dtc_regs.h)
- dtc_mrb_bits_t (enum, rx72n_dtc_regs.h)
- dtc_mstpcr_t (enum, rx72n_dtc_regs.h)
- dtc_offsets_t (enum, rx72n_dtc_regs.h)
- dtcadmod_bits_t (enum, rx72n_dtc_regs.h)
- dtccr_bits_t (enum, rx72n_dtc_regs.h)
- dtcor_bits_t (enum, rx72n_dtc_regs.h)
- dtcsqe_bits_t (enum, rx72n_dtc_regs.h)
- dtcst_bits_t (enum, rx72n_dtc_regs.h)
- dtcsts_bits_t (enum, rx72n_dtc_regs.h)

E

- eccram_regs (function, rx72n_ram_regs.h)
- eepfclk_reg (function, rx72n_flash_regs.h)
- elc_addresses_t (enum, rx72n_elc_regs.h)
- elc_elcr_reg (function, rx72n_elc_regs.h)
- elc_elopa_reg (function, rx72n_elc_regs.h)
- elc_elopb_reg (function, rx72n_elc_regs.h)
- elc_elsegr_reg (function, rx72n_elc_regs.h)
- elc_elsr0_reg (function, rx72n_elc_regs.h)
- elc_elsr15_reg (function, rx72n_elc_regs.h)
- elc_elsr45_reg (function, rx72n_elc_regs.h)
- elc_elsr48_reg (function, rx72n_elc_regs.h)
- elc_elsr_reg (function, rx72n_elc_regs.h)
- elc_mstpcr_t (enum, rx72n_elc_regs.h)
- elc_pdbf1_reg (function, rx72n_elc_regs.h)
- elc_pdbf2_reg (function, rx72n_elc_regs.h)
- elc_pel_reg (function, rx72n_elc_regs.h)
- elc_pgc1_reg (function, rx72n_elc_regs.h)

- `elc_pgc2_reg` (function, `rx72n_elc_regs.h`)
- `elc_pgr1_reg` (function, `rx72n_elc_regs.h`)
- `elc_pgr2_reg` (function, `rx72n_elc_regs.h`)
- `elcr_bits_t` (enum, `rx72n_elc_regs.h`)
- `elop_timer_op_t` (enum, `rx72n_elc_regs.h`)
- `elopa_bits_t` (enum, `rx72n_elc_regs.h`)
- `elopb_bits_t` (enum, `rx72n_elc_regs.h`)
- `elopc_bits_t` (enum, `rx72n_elc_regs.h`)
- `elopd_bits_t` (enum, `rx72n_elc_regs.h`)
- `elsegr_bits_t` (enum, `rx72n_elc_regs.h`)
- `elsr_event_t` (enum, `rx72n_elc_regs.h`)
- `encoder_calc_constants_t` (enum, `rx_mtu_encoder.c`)
- `encoder_constants_t` (enum, `rx_mtu_encoder.c`)
- `encoder_init_values_t` (enum, `rx_mtu_encoder.c`)
- `encoder_limits_t` (enum, `motor_control_task.c`)
- `encoder_velocity_limits_t` (enum, `rx_mtu_encoder.c`)
- `error_handler_backoff_constants_t` (enum, `rx_error_handler.c`)
- `error_handler_deinit` (function, `rx_error_handler.h`)
- `error_handler_deinit` (function, `rx_error_handler.c`)
- `error_handler_get_interface` (function, `rx_error_handler.h`)
- `error_handler_get_interface` (function, `rx_error_handler.c`)
- `error_handler_init` (function, `rx_error_handler.h`)
- `error_handler_init` (function, `rx_error_handler.c`)
- `error_handler_limits_t` (enum, `rx_error_handler.h`)
- `estop_reason_t` (enum, `shared_data.h`)
- `event_flags_t` (enum, `rx_obstacle_detect.c`)
- `excep_access_isr` (function, `default_interrupt_handlers.c`)
- `excep_access_isr` (function, `r_bsp.h`)
- `excep_address_isr` (function, `default_interrupt_handlers.c`)
- `excep_address_isr` (function, `r_bsp.h`)
- `excep_floating_point_isr` (function, `default_interrupt_handlers.c`)
- `excep_floating_point_isr` (function, `r_bsp.h`)
- `excep_supervisor_inst_isr` (function, `default_interrupt_handlers.c`)
- `excep_supervisor_inst_isr` (function, `r_bsp.h`)
- `excep_undefined_inst_isr` (function, `default_interrupt_handlers.c`)
- `excep_undefined_inst_isr` (function, `r_bsp.h`)
- `exram_regs` (function, `rx72n_ram_regs.h`)

F

- `faci_cmd_area` (function, `rx72n_flash_regs.h`)
- `faeint_reg` (function, `rx72n_flash_regs.h`)
- `fastat_reg` (function, `rx72n_flash_regs.h`)
- `fawmon_reg` (function, `rx72n_flash_regs.h`)
- `fbccnt_reg` (function, `rx72n_flash_regs.h`)
- `fbccstat_reg` (function, `rx72n_flash_regs.h`)
- `fcmdr_reg` (function, `rx72n_flash_regs.h`)
- `fcpsr_reg` (function, `rx72n_flash_regs.h`)
- `feaddr_reg` (function, `rx72n_flash_regs.h`)
- `fentryr_reg` (function, `rx72n_flash_regs.h`)

- flush_loop_bounds_t (enum, rx_usb.c)
- format_constants_t (enum, rx_frame_ascii.c)
- fpckar_reg (function, rx72n_flash_regs.h)
- fpsaddr_reg (function, rx72n_flash_regs.h)
- frame_header_offset_t (enum, rx_spi_comm.c)
- frame_header_offset_t (enum, rx_usb_comm.c)
- frame_offset_t (enum, rx_frame.c)
- frame_resync_discarded_t (enum, rx_frame.c)
- frdyie_reg (function, rx72n_flash_regs.h)
- fsaddr_reg (function, rx72n_flash_regs.h)
- fstatr_reg (function, rx72n_flash_regs.h)
- fsuacr_reg (function, rx72n_flash_regs.h)
- fsuinitr_reg (function, rx72n_flash_regs.h)
- fwepror_reg (function, rx72n_flash_regs.h)

G

- g_bus_manager (variable, shared_data.c)
- g_bus_manager (variable, motor_control_task.c)
- g_bus_manager (variable, temp_sensor_task.c)
- g_comm_manager (variable, comm_task.c)
- g_comm_manager (variable, telemetry_task.c)
- g_shared_data (variable, shared_data.c)
- g_shared_data (variable, shared_data.h)
- gpio_bit_constants_t (enum, gpio.c)
- gpio_init (function, hardware_init.c)
- gpio_pin_counts_t (enum, hardware_init.c)
- gpio_read (function, hardware.h)
- gpio_read (function, gpio.c)
- gpio_reg_constants_t (enum, hardware_init.c)
- gpio_set_input (function, hardware.h)
- gpio_set_input (function, gpio.c)
- gpio_set_output (function, hardware.h)
- gpio_set_output (function, gpio.c)
- gpio_toggle (function, hardware.h)
- gpio_toggle (function, gpio.c)
- gpio_write_high (function, hardware.h)
- gpio_write_high (function, gpio.c)
- gpio_write_low (function, hardware.h)
- gpio_write_low (function, gpio.c)
- gptw_bit_constants_t (enum, rx_gptw.c)
- gptw_channel_mask_t (enum, rx_gptw.c)
- gptw_constants_t (enum, rx_gptw.c)
- gptw_deadtime_t (enum, hardware_init.c)
- gptw_direction_constants_t (enum, rx_gptw.c)
- gptw_duty_constants_t (enum, rx_gptw.c)
- gptw_freq_t (enum, hardware_init.c)
- gptw_period_constants_t (enum, rx_gptw.c)
- gptw_phase_divisor_t (enum, rx_gptw.c)
- gptw_pwm_init (function, hardware_init.c)

H

- hardware_init (function, hardware_init.c)
- hardware_init (function, hardware_init.h)
- hardware_setup (function, r_bsp.h)
- harq_bool_t (enum, rx_harq.c)
- harq_cfg_sentinel_t (enum, rx_harq.c)
- harq_fec_constants_t (enum, rx_harq.c)
- hcsr04_hal_delay_us (function, rx_hcsr04_hal.h)
- hcsr04_hal_delay_us (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_get_time_us (function, rx_hcsr04_hal.h)
- hcsr04_hal_get_time_us (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_get_time_us_isr (function, rx_hcsr04_hal.h)
- hcsr04_hal_get_time_us_isr (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_gpio_deinit (function, rx_hcsr04_hal.h)
- hcsr04_hal_gpio_deinit (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_gpio_read (function, rx_hcsr04_hal.h)
- hcsr04_hal_gpio_read (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_gpio_set_input (function, rx_hcsr04_hal.h)
- hcsr04_hal_gpio_set_input (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_gpio_set_output (function, rx_hcsr04_hal.h)
- hcsr04_hal_gpio_set_output (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_gpio_write_high (function, rx_hcsr04_hal.h)
- hcsr04_hal_gpio_write_high (function, rx_hcsr04_hal_hw.c)
- hcsr04_hal_gpio_write_low (function, rx_hcsr04_hal.h)
- hcsr04_hal_gpio_write_low (function, rx_hcsr04_hal_hw.c)
- hcsr04_worker_entry (function, rx_hcsr04.c)
- host_irq_gpio_constants_t (enum, rx_host_irq.c)

I

- i2c_channel_t (enum, hardware_init.c)
- i2c_freq_t (enum, hardware_init.c)
- i2c_frequency_t (enum, main.c)
- i2c_init (function, hardware_init.c)
- i2c_validation_constants_t (enum, rx_bus_i2c.c)
- iad_constants_t (enum, rx_usb_cdc.c)
- icsr6_bits_t (enum, rx72n_poe3_regs.h)
- icsr_bits_t (enum, rx72n_poe3_regs.h)
- icsr_poemode_t (enum, rx72n_poe3_regs.h)
- icu_constants_t (enum, rx_hcsr04_icu.c)
- icu_ir_values_t (enum, timer.c)
- impl_clear_all_reservations (function, rx_pin_validator.c)
- impl_clear_errors (function, rx_error_handler.c)
- impl_get_backoff_delay (function, rx_error_handler.c)
- impl_get_component_error_count (function, rx_error_handler.c)
- impl_get_error_count (function, rx_error_handler.c)
- impl_get_ms (function, rx_time_threadx.c)
- impl_get_pin_function (function, rx_pin_validator.c)
- impl_is_elapsed (function, rx_time_threadx.c)
- impl_is_pin_reserved (function, rx_pin_validator.c)

- `impl_is_retry_limit_reached` (function, `rx_error_handler.c`)
- `impl_release_pin` (function, `rx_pin_validator.c`)
- `impl_report_error` (function, `rx_error_handler.c`)
- `impl_reserve_pin` (function, `rx_pin_validator.c`)
- `impl_reset_retry_counter` (function, `rx_error_handler.c`)
- `impl_sleep_ms` (function, `rx_time_threadx.c`)
- `impl_validate_pin` (function, `rx_pin_validator.c`)
- `init_state_t` (enum, `rx_frame.c`)
- `int_conversion_constants_t` (enum, `rx_usb.c`)
- `INT_IRQ10` (function, `rx_hcsr04_isr.c`)
- `INT_IRQ10` (function, `rx_hcsr04_isr.h`)
- `INT_IRQ11` (function, `rx_hcsr04_isr.c`)
- `INT_IRQ11` (function, `rx_hcsr04_isr.h`)
- `INT_IRQ8` (function, `rx_hcsr04_isr.c`)
- `INT_IRQ8` (function, `rx_hcsr04_isr.h`)
- `INT_IRQ9` (function, `rx_hcsr04_isr.c`)
- `INT_IRQ9` (function, `rx_hcsr04_isr.h`)
- `internal_acquire_state` (function, `rx_bus_onewire.c`)
- `internal_active_brake_sequence` (function, `motor_control_task.c`)
- `internal_adc_init_callback` (function, `rx_bus_adc.c`)
- `internal_adc_read_callback` (function, `rx_bus_adc.c`)
- `internal_adc_voltage_callback` (function, `rx_bus_adc.c`)
- `internal_align_to_sync` (function, `rx_usb_comm.c`)
- `internal_apply_pid_updates` (function, `motor_control_task.c`)
- `internal_bit_constants_t` (enum, `rx_spi_link.c`)
- `internal_branch_metric` (function, `rx_fec.c`)
- `internal_buffer_boot_log` (function, `rx_log_usb.c`)
- `internal_build_and_send_telemetry` (function, `telemetry_task.c`)
- `internal_build_frame` (function, `rx_spi_comm.c`)
- `internal_build_frame` (function, `rx_usb_comm.c`)
- `internal_bytes_to_soft_bits` (function, `rx_spi_link.c`)
- `internal_calculate_bit_rate` (function, `riic.c`)
- `internal_calculate_brr` (function, `uart.c`)
- `internal_calculate_cmt_params` (function, `rx_cmt.c`)
- `internal_calculate_period` (function, `rx_gptw.c`)
- `internal_calculate_period` (function, `rx_mtu.c`)
- `internal_calculate_phase_offset` (function, `rx_gptw.c`)
- `internal_calculate_spbr` (function, `rspi.c`)
- `internal_capture_mstpctr` (function, `rx_register_guard.c`)
- `internal_capture_pdr` (function, `rx_register_guard.c`)
- `internal_channel_to_index` (function, `rx_tpu.c`)
- `internal_check_comm_timeout` (function, `motor_control_task.c`)
- `internal_check_cwsf` (function, `main.c`)
- `internal_check_heartbeat` (function, `rx_comm_manager.c`)
- `internal_check_iwdtrf` (function, `main.c`)
- `internal_check_lvd0rf` (function, `main.c`)
- `internal_check_porf` (function, `main.c`)
- `internal_check_rstsr2_flag_clear` (function, `main.c`)
- `internal_check_startup_flags` (function, `main.c`)

- `internal_check_swrf` (function, `main.c`)
- `internal_check_usb_ready` (function, `rx_log_usb.c`)
- `internal_check_wdtrf` (function, `main.c`)
- `internal_clamp` (function, `rx_pid.c`)
- `internal_clamp_duty` (function, `rx_motor.c`)
- `internal_clear_errors` (function, `uart.c`)
- `internal_clear_tstr_bit` (function, `rx_mtu.c`)
- `internal_clock_init` (function, `rx_clock_power_init.c`)
- `internal_cmt2_init` (function, `rx_hcsr04_hal_hw.c`)
- `internal_collect_state` (function, `telemetry_task.c`)
- `internal_comm_task_entry` (function, `comm_task.c`)
- `internal_compact_rx_buffer` (function, `rx_usb_comm.c`)
- `internal_configure_adc_unit` (function, `adc.c`)
- `internal_configure_channel_staggered` (function, `rx_gptw.c`)
- `internal_configure_cmt_interrupt` (function, `rx_cmt.c`)
- `internal_configure_cmt_timer_registers` (function, `rx_cmt.c`)
- `internal_configure_cs_gpio` (function, `rspi.c`)
- `internal_configure_encoder_timer` (function, `rx_mtu_encoder.c`)
- `internal_configure_gptw_hardware` (function, `rx_gptw.c`)
- `internal_configure_gtintad` (function, `rx_poeg.c`)
- `internal_configure_iwdt_control_register` (function, `rx_iwdt.c`)
- `internal_configure_iwdt_registers` (function, `rx_iwdt.c`)
- `internal_configure_mpc` (function, `rx_gptw.c`)
- `internal_configure_port_pins` (function, `rx_gptw.c`)
- `internal_configure_registers` (function, `rspi.c`)
- `internal_configure_spcmd` (function, `rspi.c`)
- `internal_configure_uart_pins` (function, `uart.c`)
- `internal_constants_t` (enum, `rx_comm_manager.c`)
- `internal_control_loop_iteration` (function, `motor_control_task.c`)
- `internal_controller_assert_cs_with_setup` (function, `rspi.c`)
- `internal_controller_deassert_cs_with_hold` (function, `rspi.c`)
- `internal_controller_do_16bit_transfer` (function, `rspi.c`)
- `internal_create_system_tasks` (function, `main.c`)
- `internal_decode_frame` (function, `rx_spi_comm.c`)
- `internal_decode_frame` (function, `rx_usb_comm.c`)
- `internal_decode_header` (function, `rx_frame.c`)
- `internal_decode_header` (function, `rx_spi_comm.c`)
- `internal_decode_header` (function, `rx_usb_comm.c`)
- `internal_delay_timer_init` (function, `rx_bus_onewire.c`)
- `internal_delay_us` (function, `rx_bus_onewire.c`)
- `internal_detection_task_entry` (function, `rx_obstacle_detect.c`)
- `internal_disable_pipe` (function, `rx_usb_cdc.c`)
- `internal_dispatch_control_frame` (function, `rx_spi_comm.c`)
- `internal_drive_low` (function, `rx_bus_onewire.c`)
- `internal_ds18b20_get_temp_mask` (function, `rx_ds18b20.c`)
- `internal_ds18b20_raw_to_celsius` (function, `rx_ds18b20.c`)
- `internal_ds18b20_read_scratchpad_raw` (function, `rx_ds18b20.c`)
- `internal_ds18b20_resolution_to_config` (function, `rx_ds18b20.c`)
- `internal_ds18b20_select_device` (function, `rx_ds18b20.c`)

- `internal_ds18b20_validate_config` (function, `rx_ds18b20.c`)
- `internal_ds18b20_verify_device_presence` (function, `rx_ds18b20.c`)
- `internal_ds18b20_write_scratchpad` (function, `rx_ds18b20.c`)
- `internal_enable_cmt_module_clock` (function, `rx_cmt.c`)
- `internal_enable_gptw_module_clock` (function, `rx_gptw.c`)
- `internal_enable_mtu_module` (function, `rx_mtu_encoder.c`)
- `internal_enable_poeg_irq` (function, `rx_poeg.c`)
- `internal_enable_sci_clock` (function, `uart.c`)
- `internal_enable_tpu_module_clock` (function, `rx_tpu.c`)
- `internal_encode_bit` (function, `rx_fec.c`)
- `internal_encode_telemetry` (function, `telemetry_task.c`)
- `internal_execute_command_callback` (function, `rx_bus_manager.c`)
- `internal_find_component` (function, `rx_error_handler.c`)
- `internal_find_free_slot` (function, `rx_iwtd.c`)
- `internal_find_or_create_component` (function, `rx_error_handler.c`)
- `internal_find_sync` (function, `rx_usb_comm.c`)
- `internal_find_sync_offset` (function, `rx_frame.c`)
- `internal_find_task` (function, `rx_iwtd.c`)
- `internal_find_timeout_config` (function, `rx_iwtd.c`)
- `internal_frame_callback` (function, `comm_task.c`)
- `internal_get_adc_base` (function, `adc.c`)
- `internal_get_bit` (function, `rx_fec.c`)
- `internal_get_cmt_base` (function, `rx_cmt.c`)
- `internal_get_cst_bit` (function, `rx_tpu.c`)
- `internal_get_gptw_base` (function, `rx_gptw.c`)
- `internal_get_gtcr_mode` (function, `rx_gptw.c`)
- `internal_get_mstpb_bit` (function, `uart.c`)
- `internal_get_mtu_base` (function, `rx_mtu_encoder.c`)
- `internal_get_mtu_base` (function, `rx_mtu.c`)
- `internal_get_pfs_register` (function, `rx_mpc.c`)
- `internal_get_regs` (function, `rx_tpu.c`)
- `internal_get_riic_base` (function, `riic.c`)
- `internal_get_rspi_base` (function, `rspi.c`)
- `internal_get_state` (function, `rx_bus_1wire.c`)
- `internal_get_target_velocity` (function, `motor_control_task.c`)
- `internal_get_tgr_register` (function, `rx_mtu.c`)
- `internal_get_tick_count` (function, `rx_iwtd.c`)
- `internal_get_time_ms` (function, `rx_comm_manager.c`)
- `internal_get_tmdr_mode` (function, `rx_tpu.c`)
- `internal_gpio_init_adc` (function, `hardware_init.c`)
- `internal_gpio_init_callback` (function, `rx_bus_gpio.c`)
- `internal_gpio_init_gptw_pwm` (function, `hardware_init.c`)
- `internal_gpio_init_host_spi` (function, `hardware_init.c`)
- `internal_gpio_init_i2c` (function, `hardware_init.c`)
- `internal_gpio_init_imu` (function, `hardware_init.c`)
- `internal_gpio_init_motor_driver_ctrl` (function, `hardware_init.c`)
- `internal_gpio_init_mtu_encoders` (function, `hardware_init.c`)
- `internal_gpio_init_sonar_echoes` (function, `hardware_init.c`)
- `internal_gpio_init_sonar_triggers` (function, `hardware_init.c`)

- `internal_gpio_init_tpu_encoders` (function, `hardware_init.c`)
- `internal_gpio_init_usb` (function, `hardware_init.c`)
- `internal_gpio_read_callback` (function, `rx_bus_gpio.c`)
- `internal_gpio_set_input` (function, `hardware_init.c`)
- `internal_gpio_set_output` (function, `hardware_init.c`)
- `internal_gpio_toggle_callback` (function, `rx_bus_gpio.c`)
- `internal_gpio_write_callback` (function, `rx_bus_gpio.c`)
- `internal_halt_cpu` (function, `rx_exception.c`)
- `internal_handle_bemp_interrupt` (function, `rx_usb_isr.c`)
- `internal_handle_brdy_interrupt` (function, `rx_usb_isr.c`)
- `internal_handle_class_request` (function, `rx_usb_cdc.c`)
- `internal_handle_command_frame` (function, `comm_task.c`)
- `internal_handle_ctr_t_interrupt` (function, `rx_usb_isr.c`)
- `internal_handle_dvst_interrupt` (function, `rx_usb_isr.c`)
- `internal_handle_estop_command` (function, `comm_task.c`)
- `internal_handle_fec_result` (function, `rx_harq.c`)
- `internal_handle_frame` (function, `rx_comm_manager.c`)
- `internal_handle_get_descriptor` (function, `rx_usb_cdc.c`)
- `internal_handle_get_line_coding` (function, `rx_usb_cdc.c`)
- `internal_handle_no_sync` (function, `rx_usb_comm.c`)
- `internal_handle_pid_gains_command` (function, `comm_task.c`)
- `internal_handle_resume_interrupt` (function, `rx_usb_isr.c`)
- `internal_handle_retransmit_config_command` (function, `comm_task.c`)
- `internal_handle_set_address` (function, `rx_usb_cdc.c`)
- `internal_handle_set_configuration` (function, `rx_usb_cdc.c`)
- `internal_handle_set_control_line_state` (function, `rx_usb_cdc.c`)
- `internal_handle_set_line_coding` (function, `rx_usb_cdc.c`)
- `internal_handle_standard_request` (function, `rx_usb_cdc.c`)
- `internal_handle_vbus_interrupt` (function, `rx_usb_isr.c`)
- `internal_handle_velocity_command` (function, `comm_task.c`)
- `internal_i2c_init_callback` (function, `rx_bus_i2c.c`)
- `internal_i2c_read_callback` (function, `rx_bus_i2c.c`)
- `internal_i2c_write_callback` (function, `rx_bus_i2c.c`)
- `internal_i2c_write_read_callback` (function, `rx_bus_i2c.c`)
- `internal_init_branch_table` (function, `rx_fec.c`)
- `internal_init_bus_manager` (function, `main.c`)
- `internal_init_default_config` (function, `rx_iwdt.c`)
- `internal_init_encoders` (function, `motor_control_task.c`)
- `internal_init_gptw_outputs` (function, `rx_motor.c`)
- `internal_init_irq_mode` (function, `rx_hcsr04.c`)
- `internal_init_line_coding` (function, `rx_usb.c`)
- `internal_init_motor_stack` (function, `motor_control_task.c`)
- `internal_init_mutex_once` (function, `rx_log_usb.c`)
- `internal_init_pid_controllers` (function, `motor_control_task.c`)
- `internal_init_port_buffers` (function, `rx_usb.c`)
- `internal_init_shared_data_and_watchdog` (function, `main.c`)
- `internal_init_stack_monitor` (function, `main.c`)
- `internal_init_transports` (function, `comm_task.c`)
- `internal_initialize_encoder_state` (function, `rx_mtu_encoder.c`)

- `internal_invoke_callback` (function, `rx_obstacle_detect.c`)
- `internal_irq_handler` (function, `rx_hcsr04_isr.c`)
- `internal_is_timed_out` (function, `rx_usb_comm.c`)
- `internal_is_triangle_mode` (function, `rx_gptw.c`)
- `internal_is_valid_channel` (function, `rx_encoder_tpu.c`)
- `internal_is_valid_channel` (function, `rx_mtu_encoder.c`)
- `internal_is_valid_channel` (function, `rx_mtu.c`)
- `internal_led_init_gpio` (function, `led_status_task.c`)
- `internal_led_set` (function, `led_status_task.c`)
- `internal_led_task_entry` (function, `led_status_task.c`)
- `internal_lock` (function, `rx_session.c`)
- `internal_log_header` (function, `rx_log.h`)
- `internal_measure_echo_pulse` (function, `rx_hcsr04.c`)
- `internal_measure_echo_pulse_irq` (function, `rx_hcsr04.c`)
- `internal_module_stop_init` (function, `rx_clock_power_init.c`)
- `internal_motor_task_entry` (function, `motor_control_task.c`)
- `internal_mpc_lock` (function, `rx_mpc.c`)
- `internal_mpc_unlock` (function, `rx_mpc.c`)
- `INTERNAL_NOT_USED` (macro, `boot_common.h`)
- `internal_obstacle_callback` (function, `obstacle_detect_task.c`)
- `internal_obstacle_task_entry` (function, `obstacle_detect_task.c`)
- `internal_owewire_deinit_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_init_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_match_rom_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_read_bit_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_read_buffer_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_read_byte_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_read_rom_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_reset_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_search_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_skip_rom_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_write_bit_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_write_buffer_callback` (function, `rx_bus_owewire.c`)
- `internal_owewire_write_byte_callback` (function, `rx_bus_owewire.c`)
- `internal_output_decoded` (function, `rx_comm_manager.c`)
- `internal_parity` (function, `rx_fec.c`)
- `internal_parse_header` (function, `rx_usb_comm.c`)
- `internal_poeg_isr_handler` (function, `rx_poeg.c`)
- `internal_poll_sensors` (function, `rx_obstacle_detect.c`)
- `internal_poll_spi` (function, `rx_comm_manager.c`)
- `internal_poll_usb` (function, `rx_comm_manager.c`)
- `internal_populate_motor_telemetry` (function, `telemetry_task.c`)
- `internal_port_is_valid` (function, `rx_usb.c`)
- `internal_port_to_index` (function, `rx_pin_validator.c`)
- `internal_prepare_controller` (function, `rspi.c`)
- `internal_prepare_gptw_pwm_init` (function, `rx_gptw.c`)
- `internal_prepare_tx_payload` (function, `rx_spi_link.c`)
- `internal_process_event_queue` (function, `rx_comm_manager.c`)
- `internal_process_exception` (function, `rx_exception.c`)

- `internal_read_bit` (function, `rx_bus_owewire.c`)
- `internal_read_byte` (function, `rx_bus_owewire.c`)
- `internal_read_byte` (function, `riic.c`)
- `internal_read_channel_value` (function, `adc.c`)
- `internal_read_encoder_count` (function, `motor_control_task.c`)
- `internal_read_encoder_velocity` (function, `motor_control_task.c`)
- `internal_read_frame_header` (function, `rx_spi_comm.c`)
- `internal_read_le32` (function, `rx_frame.c`)
- `internal_read_line` (function, `rx_bus_owewire.c`)
- `internal_read_usb_data` (function, `rx_usb_comm.c`)
- `internal_receive_fec_decode` (function, `rx_spi_link.c`)
- `internal_receive_iteration` (function, `rx_usb_comm.c`)
- `internal_receive_passthrough` (function, `rx_spi_link.c`)
- `internal_refresh_mstpcr` (function, `rx_register_guard.c`)
- `internal_refresh_pdr` (function, `rx_register_guard.c`)
- `internal_register_iwdt_tasks` (function, `main.c`)
- `internal_register_system_buses` (function, `main.c`)
- `internal_release_line` (function, `rx_bus_owewire.c`)
- `internal_release_state` (function, `rx_bus_owewire.c`)
- `internal_report_startup_flags` (function, `main.c`)
- `internal_reset_pulse` (function, `rx_bus_owewire.c`)
- `internal_reset_search_state` (function, `rx_bus_owewire.c`)
- `internal_retransmit_frame` (function, `rx_spi_comm.c`)
- `internal_riic_read_phase` (function, `riic.c`)
- `internal_riic_write_phase` (function, `riic.c`)
- `internal_rollback_pipes` (function, `rx_usb_cdc.c`)
- `internal_rx_fatal_error` (function, `rx_check.h`)
- `internal_rx_log_debug` (function, `rx_log.h`)
- `internal_rx_log_debug_err` (function, `rx_log.h`)
- `internal_rx_log_debug_hex` (function, `rx_log.h`)
- `internal_rx_log_debug_i32` (function, `rx_log.h`)
- `internal_rx_log_debug_str` (function, `rx_log.h`)
- `internal_rx_log_debug_u16` (function, `rx_log.h`)
- `internal_rx_log_debug_u32` (function, `rx_log.h`)
- `internal_rx_log_debug_u8` (function, `rx_log.h`)
- `internal_rx_log_error` (function, `rx_log.h`)
- `internal_rx_log_error_err` (function, `rx_log.h`)
- `internal_rx_log_error_hex` (function, `rx_log.h`)
- `internal_rx_log_error_i32` (function, `rx_log.h`)
- `internal_rx_log_error_str` (function, `rx_log.h`)
- `internal_rx_log_error_u16` (function, `rx_log.h`)
- `internal_rx_log_error_u32` (function, `rx_log.h`)
- `internal_rx_log_error_u8` (function, `rx_log.h`)
- `internal_rx_log_info` (function, `rx_log.h`)
- `internal_rx_log_info_err` (function, `rx_log.h`)
- `internal_rx_log_info_hex` (function, `rx_log.h`)
- `internal_rx_log_info_i32` (function, `rx_log.h`)
- `internal_rx_log_info_str` (function, `rx_log.h`)
- `internal_rx_log_info_u16` (function, `rx_log.h`)

- `internal_rx_log_info_u32` (function, `rx_log.h`)
- `internal_rx_log_info_u8` (function, `rx_log.h`)
- `internal_rx_log_verbose` (function, `rx_log.h`)
- `internal_rx_log_verbose_err` (function, `rx_log.h`)
- `internal_rx_log_verbose_hex` (function, `rx_log.h`)
- `internal_rx_log_verbose_i32` (function, `rx_log.h`)
- `internal_rx_log_verbose_str` (function, `rx_log.h`)
- `internal_rx_log_verbose_u16` (function, `rx_log.h`)
- `internal_rx_log_verbose_u32` (function, `rx_log.h`)
- `internal_rx_log_verbose_u8` (function, `rx_log.h`)
- `internal_rx_log_warn` (function, `rx_log.h`)
- `internal_rx_log_warn_err` (function, `rx_log.h`)
- `internal_rx_log_warn_hex` (function, `rx_log.h`)
- `internal_rx_log_warn_i32` (function, `rx_log.h`)
- `internal_rx_log_warn_str` (function, `rx_log.h`)
- `internal_rx_log_warn_u16` (function, `rx_log.h`)
- `internal_rx_log_warn_u32` (function, `rx_log.h`)
- `internal_rx_log_warn_u8` (function, `rx_log.h`)
- `internal_save_cmt_callback` (function, `rx_cmt.c`)
- `internal_search_bits` (function, `rx_bus_owewire.c`)
- `internal_search_iteration` (function, `rx_bus_owewire.c`)
- `internal_search_step` (function, `rx_bus_owewire.c`)
- `internal_select_transport` (function, `telemetry_task.c`)
- `internal_send_attempt` (function, `rx_spi_link.c`)
- `internal_send_descriptor` (function, `rx_usb_cdc.c`)
- `internal_send_iwdt_heartbeat` (function, `temp_sensor_task.c`)
- `internal_send_start` (function, `riic.c`)
- `internal_send_stop` (function, `riic.c`)
- `internal_send_trigger_pulse` (function, `rx_hcsr04.c`)
- `internal_send_via_channel` (function, `telemetry_task.c`)
- `internal_set_drive_mode` (function, `rx_bus_owewire.c`)
- `internal_set_duty_raw_mtu` (function, `rx_mtu.c`)
- `internal_set_mstpcrb_for_channel` (function, `rspi.c`)
- `internal_set_output_bit` (function, `rx_fec.c`)
- `internal_sleep_and_advance` (function, `rx_usb_comm.c`)
- `internal_soft_bit_values_t` (enum, `rx_spi_link.c`)
- `internal_soft_to_hard` (function, `rx_harq.c`)
- `internal_spi_transfer` (function, `rx_spi_comm.c`)
- `internal_stack_overflow_handler` (function, `rx_stack_monitor.c`)
- `internal_state_is_valid` (function, `rx_usb.c`)
- `internal_stop_all_motors` (function, `rx_obstacle_detect.c`)
- `internal_telem_task_entry` (function, `telemetry_task.c`)
- `internal_temp_task_entry` (function, `temp_sensor_task.c`)
- `internal_time_mutex_init` (function, `rx_hcsr04_hal_hw.c`)
- `internal_timing_delay` (function, `rspi.c`)
- `internal_trigger_and_measure` (function, `rx_hcsr04.c`)
- `internal_trigger_tx_if_idle` (function, `rx_usb.c`)
- `internal_unlock` (function, `rx_session.c`)
- `internal_update_motor_state` (function, `motor_control_task.c`)

- `internal_update_state` (function, `rx_encoder_tpu.c`)
- `internal_update_state_from_count` (function, `rx_mtu_encoder.c`)
- `internal_validate_cmt_init_params` (function, `rx_cmt.c`)
- `internal_validate_config` (function, `rx_obstacle_detect.c`)
- `internal_validate_controller_args` (function, `rspi.c`)
- `internal_validate_decode_params` (function, `rx_fec.c`)
- `internal_validate_pin` (function, `rx_pin_validator.c`)
- `internal_validate_port` (function, `rx_pin_validator.c`)
- `internal_validate_port_pin` (function, `rx_bus_config.c`)
- `internal_validate_port_pin` (function, `gpio.c`)
- `internal_validate_port_pin` (function, `rx_hcsr04_hal_hw.c`)
- `internal_validate_unit_channel` (function, `adc.c`)
- `internal_verify_crc` (function, `rx_frame.c`)
- `internal_verify_crc` (function, `rx_spi_comm.c`)
- `internal_verify_crc` (function, `rx_usb_comm.c`)
- `internal_verify_system_state` (function, `rx_clock_power_init.c`)
- `internal_verify_timer_counting` (function, `rx_mtu_encoder.c`)
- `internal_viterbi_forward_pass` (function, `rx_fec.c`)
- `internal_viterbi_process_symbol` (function, `rx_fec.c`)
- `internal_viterbi_traceback` (function, `rx_fec.c`)
- `internal_wait_bus_ready` (function, `riic.c`)
- `internal_wait_for_ack` (function, `rx_spi_comm.c`)
- `internal_wait_for_ack` (function, `rx_spi_link.c`)
- `internal_wait_for_data` (function, `rx_spi_comm.c`)
- `internal_wait_for_data` (function, `rx_usb_comm.c`)
- `internal_wait_for_echo` (function, `rx_hcsr04.c`)
- `internal_wait_rx_ready` (function, `rspi.c`)
- `internal_wait_tx_ready` (function, `rspi.c`)
- `internal_watchdog_monitor_task_entry` (function, `watchdog_monitor_task.c`)
- `internal_write_bit` (function, `rx_bus_onewire.c`)
- `internal_write_byte` (function, `rx_bus_onewire.c`)
- `internal_write_byte` (function, `riic.c`)
- `internal_write_le32` (function, `rx_frame.c`)
- `internal_write_pfs` (function, `rx_mpc.c`)
- `internal_write_usb` (function, `rx_log_usb.c`)
- `irq_filter_constants_t` (enum, `rx_irq_filter.c`)
- `isr_constants_t` (enum, `rx_hcsr04_isr.c`)
- `isr_irq_numbers_t` (enum, `rx_hcsr04_isr.c`)
- `isr_timer_constants_t` (enum, `rx_hcsr04_isr.c`)
- `iwdt` (function, `rx72n_iwdt_regs.h`)
- `iwdt_buffer_size_constants_t` (enum, `rx_iwdt.c`)
- `iwdt_constants_t` (enum, `rx_iwdt.h`)
- `iwdt_hardware_timeout_ms_t` (enum, `main.c`)
- `iwdt_init_state_t` (enum, `rx_iwdt.c`)
- `iwdt_iwdtcr_bits_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_iwdtcr_shifts_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_iwdtcstpr_bits_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_iwdtcstpr_shifts_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_iwdtrcr_bits_t` (enum, `rx72n_iwdt_regs.h`)

- `iwdt_iwdtsr_bits_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_refresh_sequence_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_reserved_sizes_t` (enum, `rx72n_iwdt_regs.h`)
- `iwdt_reset_reason_t` (enum, `rx_iwdt.h`)
- `iwdt_state_timeout_ms_t` (enum, `main.c`)
- `iwdt_task_timeout_ms_t` (enum, `main.c`)
- `iwdt_timeout_limits_t` (enum, `rx_iwdt.c`)
- `iwdt_timeout_period_t` (enum, `rx_iwdt.h`)

K

- `k_encoder_initial_position_deg` (variable, `rx_mtu_encoder.c`)
- `k_max_delta_time_s` (variable, `rx_mtu_encoder.c`)
- `k_min_delta_time_s` (variable, `rx_mtu_encoder.c`)
- `k_tpu_enc_initial_position_deg` (variable, `rx_encoder_tpu.c`)
- `k_tpu_enc_max_delta_time_s` (variable, `rx_encoder_tpu.c`)
- `k_tpu_enc_min_delta_time_s` (variable, `rx_encoder_tpu.c`)

L

- `langid` (variable, `rx_usb_cdc.c`)
- `led_index_t` (enum, `led_status_task.c`)
- `led_status_task_create` (function, `led_status_task.h`)
- `led_status_task_create` (function, `led_status_task.c`)
- `led_task_constants_t` (enum, `led_status_task.c`)
- `led_timing_constants_t` (enum, `led_status_task.c`)
- `led_timing_divisor_t` (enum, `led_status_task.c`)
- `length` (variable, `rx_usb_cdc.c`)
- `log_format_constants_t` (enum, `rx_log.h`)
- `LOG_LEVEL` (macro, `rx_log.h`)
- `log_level_t` (enum, `rx_log.h`)
- `LOG_PUTC` (macro, `rx_log.h`)
- `LOG_PUTHEX` (macro, `rx_log.h`)
- `LOG_PUTINT` (macro, `rx_log.h`)
- `LOG_PUTS` (macro, `rx_log.h`)
- `lvcmpcr_reg` (function, `rx72n_lvda_regs.h`)
- `lvd1cr0_reg` (function, `rx72n_lvda_regs.h`)
- `lvd1cr1_reg` (function, `rx72n_lvda_regs.h`)
- `lvd1sr_reg` (function, `rx72n_lvda_regs.h`)
- `lvd2cr0_reg` (function, `rx72n_lvda_regs.h`)
- `lvd2cr1_reg` (function, `rx72n_lvda_regs.h`)
- `lvd2sr_reg` (function, `rx72n_lvda_regs.h`)
- `lvd1vlr_reg` (function, `rx72n_lvda_regs.h`)

M

- `main` (function, `main.c`)
- `main_ret_t` (enum, `main.c`)
- `mdmonr_bits_t` (enum, `rx72n_system_regs.h`)
- `memwait_reg` (function, `rx72n_system_regs.h`)
- `mhitd_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mhitl_bits_t` (enum, `rx72n_mpu_regs.h`)

- `module_init_config_t` (enum, `rx_clock_power_init.c`)
- `motor_control_constants_t` (enum, `motor_control_task.c`)
- `motor_control_task_create` (function, `motor_control_task.h`)
- `motor_control_task_create` (function, `motor_control_task.c`)
- `motor_control_task_get_motors` (function, `motor_control_task.h`)
- `motor_control_task_get_motors` (function, `motor_control_task.c`)
- `motor_count_t` (enum, `motor_control_task.h`)
- `motor_duty_limits_t` (enum, `rx_motor.c`)
- `motor_hw_constants_t` (enum, `motor_control_task.c`)
- `motor_in2_signal_t` (enum, `rx_motor.c`)
- `motor_index_t` (enum, `motor_control_task.c`)
- `motor_mode_t` (enum, `shared_data.h`)
- `motor_task_constants_t` (enum, `motor_control_task.c`)
- `motor_validation_limits_t` (enum, `rx_motor.c`)
- `mpbac_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mpc_pfs_gpio_t` (enum, `rx_mpc.c`)
- `mpc_pfs_offset_constants_t` (enum, `rx_gptw.c`)
- `mpc_pfs_offset_t` (enum, `rx_mpc.c`)
- `mpc_pin_constants_t` (enum, `rx_mpc.c`)
- `mpc_port_number_t` (enum, `rx_mpc.c`)
- `mpc_port_offset_t` (enum, `rx_mpc.c`)
- `mpc_psel_constants_t` (enum, `rx_mpc.c`)
- `mpc_pwpr_constants_t` (enum, `rx_gptw.c`)
- `mpc_pwpr_t` (enum, `rx_mpc.c`)
- `mpeclr_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mpen_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mpests_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mpopi_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mpops_bits_t` (enum, `rx72n_mpu_regs.h`)
- `mpu_addr_to_page` (function, `rx72n_mpu_regs.h`)
- `mpu_addresses_t` (enum, `rx72n_mpu_regs.h`)
- `mpu_mhitd_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mhiti_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpbac_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpdea_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpeclr_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpen_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpests_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpopi_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpops_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_mpsa_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_page_t` (enum, `rx72n_mpu_regs.h`)
- `mpu_page_to_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_region` (function, `rx72n_mpu_regs.h`)
- `mpu_region_t` (enum, `rx72n_mpu_regs.h`)
- `mpu_repage_reg` (function, `rx72n_mpu_regs.h`)
- `mpu_rspage_reg` (function, `rx72n_mpu_regs.h`)
- `mstpcra_bits_t` (enum, `rx_clock_power_init.c`)
- `mstpcrb_bits_t` (enum, `rx_clock_power_init.c`)

- mstpcrc_bits_t (enum, rx_clock_power_init.c)
- mtu_bit_constants_t (enum, rx_mtu.c)
- mtu_constants_t (enum, rx_mtu.c)
- mtu_duty_constants_t (enum, rx_mtu.c)
- mtu_module_stop_bits_t (enum, rx_mtu.c)
- mtu_period_constants_t (enum, rx_mtu.c)
- mtu_tior_disabled_t (enum, rx_mtu.c)
- mtu_tior_mask_t (enum, rx_mtu.c)
- mtu_tior_shift_t (enum, rx_mtu.c)

N

- non_maskable_isr (function, default_interrupt_handlers.c)
- non_maskable_isr (function, r_bsp.h)

O

- obstacle_detect_task_create (function, obstacle_detect_task.h)
- obstacle_detect_task_create (function, obstacle_detect_task.c)
- obstacle_motor_count_t (enum, obstacle_detect_task.c)
- obstacle_sensor_constants_t (enum, obstacle_detect_task.c)
- obstacle_sensor_idx_t (enum, obstacle_detect_task.c)
- obstacle_task_constants_t (enum, obstacle_detect_task.c)
- ocsr_bits_t (enum, rx72n_poe3_regs.h)
- onewire_delay_hw_constants_t (enum, rx_bus_onewire.c)
- onewire_delay_tick_constants_t (enum, rx_bus_onewire.c)
- onewire_driver_constants_t (enum, rx_bus_onewire.c)
- onewire_rom_command_t (enum, rx_bus_onewire.h)
- onewire_rom_size_t (enum, rx_bus_onewire.h)
- onewire_timing_us_t (enum, rx_bus_onewire.h)
- opccr_reg (function, rx72n_lpc_regs.h)
- oscillator_control_t (enum, rx_clock_power_init.c)
- oscillator_timing_t (enum, rx_clock_power_init.c)

P

- pel_bits_t (enum, rx72n_elc_regs.h)
- pgc_bits_t (enum, rx72n_elc_regs.h)
- pin_interface_limits_t (enum, rx_pin_interface.h)
- pin_validator_deinit (function, rx_pin_validator.h)
- pin_validator_deinit (function, rx_pin_validator.c)
- pin_validator_get_interface (function, rx_pin_validator.h)
- pin_validator_get_interface (function, rx_pin_validator.c)
- pin_validator_init (function, rx_pin_validator.h)
- pin_validator_init (function, rx_pin_validator.c)
- pin_validator_limits_t (enum, rx_pin_validator.h)
- pll_config_t (enum, rx_clock_power_init.c)
- pllcr2_bits_t (enum, rx72n_system_regs.h)
- pllcr_bits_t (enum, rx72n_system_regs.h)
- poe3_addresses_t (enum, rx72n_poe3_regs.h)
- poe3_alr1_reg (function, rx72n_poe3_regs.h)
- poe3_icsr1_reg (function, rx72n_poe3_regs.h)

- poe3_icsr2_reg (function, rx72n_poe3_regs.h)
- poe3_icsr3_reg (function, rx72n_poe3_regs.h)
- poe3_icsr4_reg (function, rx72n_poe3_regs.h)
- poe3_icsr5_reg (function, rx72n_poe3_regs.h)
- poe3_icsr6_reg (function, rx72n_poe3_regs.h)
- poe3_m0selr1_reg (function, rx72n_poe3_regs.h)
- poe3_m0selr2_reg (function, rx72n_poe3_regs.h)
- poe3_m3selr_reg (function, rx72n_poe3_regs.h)
- poe3_m4selr1_reg (function, rx72n_poe3_regs.h)
- poe3_m4selr2_reg (function, rx72n_poe3_regs.h)
- poe3_m6selr_reg (function, rx72n_poe3_regs.h)
- poe3_mstpcr_t (enum, rx72n_poe3_regs.h)
- poe3_ocr1_reg (function, rx72n_poe3_regs.h)
- poe3_ocr2_reg (function, rx72n_poe3_regs.h)
- poe3_offsets_t (enum, rx72n_poe3_regs.h)
- poe3_poecr1_reg (function, rx72n_poe3_regs.h)
- poe3_poecr2_reg (function, rx72n_poe3_regs.h)
- poe3_poecr4_reg (function, rx72n_poe3_regs.h)
- poe3_poecr5_reg (function, rx72n_poe3_regs.h)
- poe3_spoer_reg (function, rx72n_poe3_regs.h)
- poecr1_bits_t (enum, rx72n_poe3_regs.h)
- poecr2_bits_t (enum, rx72n_poe3_regs.h)
- poecr4_bits_t (enum, rx72n_poe3_regs.h)
- poecr5_bits_t (enum, rx72n_poe3_regs.h)
- poeg_group_a_isr (function, rx_poeg.c)
- poeg_group_b_isr (function, rx_poeg.c)
- poeg_group_c_isr (function, rx_poeg.c)
- poeg_group_d_isr (function, rx_poeg.c)
- poeg_gtintad_values_t (enum, rx_poeg.c)
- poeg_icu_constants_t (enum, rx_poeg.c)
- poeg_init_config_t (enum, rx_poeg.c)
- poegga (function, rx72n_poeg_regs.h)
- poeggb (function, rx72n_poeg_regs.h)
- poeggc (function, rx72n_poeg_regs.h)
- poeggd (function, rx72n_poeg_regs.h)
- poll_timing_t (enum, rx_spi_comm.c)
- polling_limits_t (enum, rx_spi_comm.c)
- port_config_constants_t (enum, rx_usb.c)
- port_encoding_t (enum, rx_port_constants.h)
- port_pipe_assignments_t (enum, rx_usb.c)
- porte_gptw_pin_t (enum, rx_gptw.c)
- PowerON_Reset_PC (function, r_bsp.h)
- ppllcr2_reg (function, rx72n_system_regs.h)
- ppllcr_reg (function, rx72n_system_regs.h)
- prcr_reg (function, rx72n_system_regs.h)
- priv_ring_buffer_available (function, rx_usb.c)
- priv_ring_buffer_free (function, rx_usb.c)
- priv_ring_buffer_init (function, rx_usb.c)
- priv_ring_buffer_read (function, rx_usb.c)

- `priv_ring_buffer_write` (function, `rx_usb.c`)

R

- `R_BSP_BitSet` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_ChangeToUserMode` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_GetBPC` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_GetBPSW` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_MoveFromAccHiLong` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_MoveFromAccMiLong` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_MoveToAccHiLong` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_PRAGMA_INLINE_ASM` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_PRAGMA_INLINE_ASM` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_PRAGMA_INLINE_ASM` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_PRAGMA_STATIC_INLINE_ASM` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_PRAGMA_STATIC_INLINE_ASM` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_PRAGMA_STATIC_INLINE_ASM` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_SetBPC` (function, `r_rx_intrinsic_functions.c`)
- `R_BSP_SetBPSW` (function, `r_rx_intrinsic_functions.c`)
- `ram_regs` (function, `rx72n_ram_regs.h`)
- `rcr3_bits_t` (enum, `rx72n_rtc_regs.h`)
- `receive_bounds_t` (enum, `rx_spi_comm.c`)
- `register_guard_defaults_t` (enum, `rx_register_guard.c`)
- `repage_bits_t` (enum, `rx72n_mpu_regs.h`)
- `retransmit_retries_limits_t` (enum, `comm_task.c`)
- `retransmit_timing_limits_t` (enum, `comm_task.c`)
- `riic_address_bits_t` (enum, `riic.c`)
- `riic_bit_rate_t` (enum, `riic.c`)
- `riic_channel_num_t` (enum, `riic.c`)
- `riic_channel_num_t` (enum, `main.c`)
- `riic_constants_t` (enum, `riic.c`)
- `riic_frequency_t` (enum, `riic.c`)
- `riic_icmr1_values_t` (enum, `riic.c`)
- `riic_icmr2_values_t` (enum, `riic.c`)
- `riic_icmr3_bits_t` (enum, `riic.c`)
- `riic_init` (function, `hardware.h`)
- `riic_init` (function, `riic.c`)
- `riic_module_stop_bits_t` (enum, `riic.c`)
- `riic_read` (function, `hardware.h`)
- `riic_read` (function, `riic.c`)
- `riic_write` (function, `hardware.h`)
- `riic_write` (function, `riic.c`)
- `riic_write_read` (function, `hardware.h`)
- `riic_write_read` (function, `riic.c`)
- `ring_buffer_constants_t` (enum, `rx_usb.c`)
- `romce_reg` (function, `rx72n_flash_regs.h`)
- `romciv_reg` (function, `rx72n_flash_regs.h`)
- `rspage_bits_t` (enum, `rx72n_mpu_regs.h`)
- `rspi0` (function, `rx72n_rspi_regs.h`)
- `rspi1` (function, `rx72n_rspi_regs.h`)

- `rspi2` (function, `rx72n_rspi_regs.h`)
- `rspi_bit_constants_t` (enum, `rspi.c`)
- `rspi_channel_t` (enum, `hardware.h`)
- `rspi_constants_t` (enum, `rspi.c`)
- `rspi_controller_constants_t` (enum, `rspi.c`)
- `rspi_controller_deinit` (function, `hardware.h`)
- `rspi_controller_deinit` (function, `rspi.c`)
- `rspi_controller_set_cs` (function, `hardware.h`)
- `rspi_controller_set_cs` (function, `rspi.c`)
- `rspi_controller_transfer_16bit` (function, `hardware.h`)
- `rspi_controller_transfer_16bit` (function, `rspi.c`)
- `rspi_cs_defaults_t` (enum, `rspi.c`)
- `rspi_deinit` (function, `hardware.h`)
- `rspi_deinit` (function, `rspi.c`)
- `rspi_freq_constants_t` (enum, `hardware.h`)
- `rspi_init_controller` (function, `hardware.h`)
- `rspi_init_controller` (function, `rspi.c`)
- `rspi_init_peripheral` (function, `hardware.h`)
- `rspi_init_peripheral` (function, `rspi.c`)
- `rspi_mode_limits_t` (enum, `rspi.c`)
- `rspi_mode_t` (enum, `hardware.h`)
- `rspi_module_stop_bits_t` (enum, `rspi.c`)
- `rspi_peripheral_read_available` (function, `hardware.h`)
- `rspi_peripheral_read_available` (function, `rspi.c`)
- `rspi_peripheral_transfer` (function, `hardware.h`)
- `rspi_peripheral_transfer` (function, `rspi.c`)
- `rspi_peripheral_write_ready` (function, `hardware.h`)
- `rspi_peripheral_write_ready` (function, `rspi.c`)
- `rspi_register_defaults_t` (enum, `rspi.c`)
- `rspi_spcmd_bits_t` (enum, `rspi.c`)
- `rspi_spcr_bits_t` (enum, `rx72n_rspi_regs.h`)
- `rspi_spdcr_bits_t` (enum, `rx72n_rspi_regs.h`)
- `rspi_spdcr_local_t` (enum, `rspi.c`)
- `rspi_sppcr_bits_t` (enum, `rx72n_rspi_regs.h`)
- `rspi_spsr_bits_t` (enum, `rx72n_rspi_regs.h`)
- `rspi_transfer_constants_t` (enum, `rspi.c`)
- `rstckcr_reg` (function, `rx72n_lpc_regs.h`)
- `rstsr01` (function, `rx72n_system_regs.h`)
- `rstsr0_bits_t` (enum, `rx72n_system_regs.h`)
- `rstsr1_bits_t` (enum, `rx72n_system_regs.h`)
- `rstsr2` (function, `rx72n_system_regs.h`)
- `rstsr2_bits_t` (enum, `rx72n_system_regs.h`)
- `rtc_addresses_t` (enum, `rx72n_rtc_regs.h`)
- `rtc_offsets_t` (enum, `rx72n_rtc_regs.h`)
- `rtc_rcr1_reg` (function, `rx72n_rtc_regs.h`)
- `rtc_rcr2_reg` (function, `rx72n_rtc_regs.h`)
- `rtc_rcr3_reg` (function, `rx72n_rtc_regs.h`)
- `rtc_rcr4_reg` (function, `rx72n_rtc_regs.h`)
- `rx_adc_channel_limits_t` (enum, `rx_bus_types.h`)

- rx_adc_limits_t (enum, rx_bus_types.h)
- rx_adc_resolution_t (enum, rx_bus_types.h)
- RX_ASSERT (macro, rx_check.h)
- rx_bit_shifts_t (enum, rx_bit_constants.h)
- rx_bit_sizes_t (enum, rx_bit_constants.h)
- rx_bus_adc_init (function, rx_bus_adc.h)
- rx_bus_adc_init (function, rx_bus_adc.c)
- rx_bus_adc_read (function, rx_bus_adc.h)
- rx_bus_adc_read (function, rx_bus_adc.c)
- rx_bus_adc_read_voltage_mv (function, rx_bus_adc.h)
- rx_bus_adc_read_voltage_mv (function, rx_bus_adc.c)
- rx_bus_callback_t (typedef, rx_bus_manager.h)
- rx_bus_command_execute_fn (typedef, rx_bus_command.h)
- rx_bus_command_init (function, rx_bus_command.h)
- rx_bus_config_init_adc (function, rx_bus_config.h)
- rx_bus_config_init_adc (function, rx_bus_config.c)
- rx_bus_config_init_gpio (function, rx_bus_config.h)
- rx_bus_config_init_gpio (function, rx_bus_config.c)
- rx_bus_config_init_i2c (function, rx_bus_config.h)
- rx_bus_config_init_i2c (function, rx_bus_config.c)
- rx_bus_config_init_onewire (function, rx_bus_config.h)
- rx_bus_config_init_onewire (function, rx_bus_config.c)
- rx_bus_config_init_uart (function, rx_bus_config.h)
- rx_bus_config_init_uart (function, rx_bus_config.c)
- rx_bus_config_t (typedef, rx_bus_command.h)
- rx_bus_gpio_init (function, rx_bus_gpio.h)
- rx_bus_gpio_init (function, rx_bus_gpio.c)
- rx_bus_gpio_read (function, rx_bus_gpio.h)
- rx_bus_gpio_read (function, rx_bus_gpio.c)
- rx_bus_gpio_toggle (function, rx_bus_gpio.h)
- rx_bus_gpio_toggle (function, rx_bus_gpio.c)
- rx_bus_gpio_write (function, rx_bus_gpio.h)
- rx_bus_gpio_write (function, rx_bus_gpio.c)
- rx_bus_i2c_init (function, rx_bus_i2c.h)
- rx_bus_i2c_init (function, rx_bus_i2c.c)
- rx_bus_i2c_read (function, rx_bus_i2c.h)
- rx_bus_i2c_read (function, rx_bus_i2c.c)
- rx_bus_i2c_write (function, rx_bus_i2c.h)
- rx_bus_i2c_write (function, rx_bus_i2c.c)
- rx_bus_i2c_write_read (function, rx_bus_i2c.h)
- rx_bus_i2c_write_read (function, rx_bus_i2c.c)
- rx_bus_manager_add_bus (function, rx_bus_manager.h)
- rx_bus_manager_add_bus (function, rx_bus_manager.c)
- rx_bus_manager_deinit (function, rx_bus_manager.h)
- rx_bus_manager_deinit (function, rx_bus_manager.c)
- rx_bus_manager_execute_command (function, rx_bus_manager.h)
- rx_bus_manager_execute_command (function, rx_bus_manager.c)
- rx_bus_manager_find_bus (function, rx_bus_manager.h)
- rx_bus_manager_find_bus (function, rx_bus_manager.c)

- rx_bus_manager_init (function, rx_bus_manager.h)
- rx_bus_manager_init (function, rx_bus_manager.c)
- rx_bus_manager_remove_bus (function, rx_bus_manager.h)
- rx_bus_manager_remove_bus (function, rx_bus_manager.c)
- rx_bus_manager_with_bus (function, rx_bus_manager.h)
- rx_bus_manager_with_bus (function, rx_bus_manager.c)
- rx_bus_owewire_deinit (function, rx_bus_owewire.h)
- rx_bus_owewire_deinit (function, rx_bus_owewire.c)
- rx_bus_owewire_init (function, rx_bus_owewire.h)
- rx_bus_owewire_init (function, rx_bus_owewire.c)
- rx_bus_owewire_match_rom (function, rx_bus_owewire.h)
- rx_bus_owewire_match_rom (function, rx_bus_owewire.c)
- rx_bus_owewire_read (function, rx_bus_owewire.h)
- rx_bus_owewire_read (function, rx_bus_owewire.c)
- rx_bus_owewire_read_bit (function, rx_bus_owewire.h)
- rx_bus_owewire_read_bit (function, rx_bus_owewire.c)
- rx_bus_owewire_read_byte (function, rx_bus_owewire.h)
- rx_bus_owewire_read_byte (function, rx_bus_owewire.c)
- rx_bus_owewire_read_rom (function, rx_bus_owewire.h)
- rx_bus_owewire_read_rom (function, rx_bus_owewire.c)
- rx_bus_owewire_reset (function, rx_bus_owewire.h)
- rx_bus_owewire_reset (function, rx_bus_owewire.c)
- rx_bus_owewire_search (function, rx_bus_owewire.h)
- rx_bus_owewire_search (function, rx_bus_owewire.c)
- rx_bus_owewire_skip_rom (function, rx_bus_owewire.h)
- rx_bus_owewire_skip_rom (function, rx_bus_owewire.c)
- rx_bus_owewire_write (function, rx_bus_owewire.h)
- rx_bus_owewire_write (function, rx_bus_owewire.c)
- rx_bus_owewire_write_bit (function, rx_bus_owewire.h)
- rx_bus_owewire_write_bit (function, rx_bus_owewire.c)
- rx_bus_owewire_write_byte (function, rx_bus_owewire.h)
- rx_bus_owewire_write_byte (function, rx_bus_owewire.c)
- rx_bus_type_t (enum, rx_bus_types.h)
- rx_byte_order_constants_t (enum, rx_frame.h)
- rx_chase_combiner_add (function, rx_harq.h)
- rx_chase_combiner_add (function, rx_harq.c)
- rx_chase_combiner_can_add (function, rx_harq.h)
- rx_chase_combiner_can_add (function, rx_harq.c)
- rx_chase_combiner_combined (function, rx_harq.h)
- rx_chase_combiner_combined (function, rx_harq.c)
- rx_chase_combiner_count (function, rx_harq.h)
- rx_chase_combiner_count (function, rx_harq.c)
- rx_chase_combiner_deinit (function, rx_harq.h)
- rx_chase_combiner_deinit (function, rx_harq.c)
- rx_chase_combiner_init (function, rx_harq.h)
- rx_chase_combiner_init (function, rx_harq.c)
- rx_chase_combiner_reset (function, rx_harq.h)
- rx_chase_combiner_reset (function, rx_harq.c)
- RX_CHECK_NULL_PTR (macro, rx_check.h)

- RX_CHECK_RANGE (macro, rx_check.h)
- RX_CHECK_RANGE_TAG (macro, rx_check.h)
- rx_clock_frequencies_t (enum, rx72n_clock.h)
- rx_clock_power_init (function, rx_clock_power_init.h)
- rx_clock_power_init (function, rx_clock_power_init.c)
- rx_cmt_callback_t (typedef, rx_cmt.h)
- rx_cmt_channel_t (enum, rx_cmt.h)
- rx_cmt_deinit (function, rx_cmt.h)
- rx_cmt_deinit (function, rx_cmt.c)
- rx_cmt_get_count (function, rx_cmt.h)
- rx_cmt_get_count (function, rx_cmt.c)
- rx_cmt_init (function, rx_cmt.h)
- rx_cmt_init (function, rx_cmt.c)
- rx_cmt_start (function, rx_cmt.h)
- rx_cmt_start (function, rx_cmt.c)
- rx_cmt_stop (function, rx_cmt.h)
- rx_cmt_stop (function, rx_cmt.c)
- rx_comm_channel_t (enum, rx_comm_manager.h)
- rx_comm_event_retry_constants_t (enum, rx_comm_manager.h)
- rx_comm_frame_callback_t (typedef, rx_comm_manager.h)
- rx_comm_heartbeat_constants_t (enum, rx_comm_manager.h)
- rx_comm_link_status_callback_t (typedef, rx_comm_manager.h)
- rx_comm_link_status_t (enum, rx_comm_manager.h)
- rx_comm_manager_channel_name (function, rx_comm_manager.h)
- rx_comm_manager_channel_name (function, rx_comm_manager.c)
- rx_comm_manager_channel_ready (function, rx_comm_manager.h)
- rx_comm_manager_channel_ready (function, rx_comm_manager.c)
- rx_comm_manager_constants_t (enum, rx_comm_manager.h)
- rx_comm_manager_deinit (function, rx_comm_manager.h)
- rx_comm_manager_deinit (function, rx_comm_manager.c)
- rx_comm_manager_event_send (function, rx_comm_manager.h)
- rx_comm_manager_event_send (function, rx_comm_manager.c)
- rx_comm_manager_init (function, rx_comm_manager.h)
- rx_comm_manager_init (function, rx_comm_manager.c)
- rx_comm_manager_link_status (function, rx_comm_manager.h)
- rx_comm_manager_link_status (function, rx_comm_manager.c)
- rx_comm_manager_poll (function, rx_comm_manager.h)
- rx_comm_manager_poll (function, rx_comm_manager.c)
- rx_comm_manager_process_retransmits (function, rx_comm_manager.h)
- rx_comm_manager_process_retransmits (function, rx_comm_manager.c)
- rx_comm_manager_respond (function, rx_comm_manager.h)
- rx_comm_manager_respond (function, rx_comm_manager.c)
- rx_comm_manager_send (function, rx_comm_manager.h)
- rx_comm_manager_send (function, rx_comm_manager.c)
- rx_comm_manager_set_auto_retransmit (function, rx_comm_manager.h)
- rx_comm_manager_set_auto_retransmit (function, rx_comm_manager.c)
- rx_comm_manager_stream_send (function, rx_comm_manager.h)
- rx_comm_manager_stream_send (function, rx_comm_manager.c)
- rx_comm_sim_time_t (enum, rx_comm_manager.c)

- rx_crc32_ieee (function, rx_crc.h)
- rx_crc32_ieee (function, rx_crc32.c)
- rx_crc32_ieee_impl (function, rx_crc_internal.h)
- rx_crc32_ieee_impl (function, rx_crc32_hw.c)
- rx_crc32_sw_constants_t (enum, rx_crc32_sw.c)
- rx_crc32_update (function, rx_crc.h)
- rx_crc32_update (function, rx_crc32.c)
- rx_crc32_update_impl (function, rx_crc_internal.h)
- rx_crc32_update_impl (function, rx_crc32_hw.c)
- rx_crc32_update_sw (function, rx_crc_internal.h)
- rx_crc32_update_sw (function, rx_crc32_sw.c)
- RX_CRC32_USE_HARDWARE (macro, rx_crc_internal.h)
- rx_crc8_constants_t (enum, rx_crc8.c)
- rx_crc8_maxim (function, rx_crc.h)
- rx_crc8_maxim (function, rx_crc8.c)
- rx_crc_addresses_t (enum, rx72n_crc_regs.h)
- rx_crc_crccr_bits_t (enum, rx72n_crc_regs.h)
- rx_crc_crccr_masks_t (enum, rx72n_crc_regs.h)
- rx_crc_crccr_shifts_t (enum, rx72n_crc_regs.h)
- rx_crc_deinit (function, rx_crc.h)
- rx_crc_deinit (function, rx_crc_internal.h)
- rx_crc_deinit (function, rx_crc32_hw.c)
- rx_crc_hw_constants_t (enum, rx_crc32_hw.c)
- rx_crc_hw_loop_constants_t (enum, rx_crc32_hw.c)
- rx_crc_init (function, rx_crc.h)
- rx_crc_init (function, rx_crc_internal.h)
- rx_crc_init (function, rx_crc32_hw.c)
- rx_crc_reserved_sizes_t (enum, rx72n_crc_regs.h)
- rx_crc_shared_constants_t (enum, rx_crc_internal.h)
- rx_dpsbycr_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsiegr0_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsiegr1_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsiegr2_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsiegr3_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsier0_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsier1_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsier2_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsier3_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsifr0_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsifr1_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsifr2_bits_t (enum, rx72n_lpc_regs.h)
- rx_dpsifr3_bits_t (enum, rx72n_lpc_regs.h)
- rx_ds18b20_deinit (function, rx_ds18b20.h)
- rx_ds18b20_deinit (function, rx_ds18b20.c)
- rx_ds18b20_get_conversion_time_ms (function, rx_ds18b20.h)
- rx_ds18b20_get_conversion_time_ms (function, rx_ds18b20.c)
- rx_ds18b20_get_resolution (function, rx_ds18b20.h)
- rx_ds18b20_get_resolution (function, rx_ds18b20.c)
- rx_ds18b20_init (function, rx_ds18b20.h)

- rx_ds18b20_init (function, rx_ds18b20.c)
- rx_ds18b20_read_power_mode (function, rx_ds18b20.h)
- rx_ds18b20_read_power_mode (function, rx_ds18b20.c)
- rx_ds18b20_read_scratchpad (function, rx_ds18b20.h)
- rx_ds18b20_read_scratchpad (function, rx_ds18b20.c)
- rx_ds18b20_read_temperature (function, rx_ds18b20.h)
- rx_ds18b20_read_temperature (function, rx_ds18b20.c)
- rx_ds18b20_read_temperature_raw (function, rx_ds18b20.h)
- rx_ds18b20_read_temperature_raw (function, rx_ds18b20.c)
- rx_ds18b20_recall_config (function, rx_ds18b20.h)
- rx_ds18b20_recall_config (function, rx_ds18b20.c)
- rx_ds18b20_save_config (function, rx_ds18b20.h)
- rx_ds18b20_save_config (function, rx_ds18b20.c)
- rx_ds18b20_set_resolution (function, rx_ds18b20.h)
- rx_ds18b20_set_resolution (function, rx_ds18b20.c)
- rx_ds18b20_trigger_conversion (function, rx_ds18b20.h)
- rx_ds18b20_trigger_conversion (function, rx_ds18b20.c)
- rx_eccram1sts_bits_t (enum, rx72n_ram_regs.h)
- rx_eccram1stsen_bits_t (enum, rx72n_ram_regs.h)
- rx_eccram2sts_bits_t (enum, rx72n_ram_regs.h)
- rx_eccram_reg_offset_t (enum, rx72n_ram_regs.h)
- rx_eccrametst_bits_t (enum, rx72n_ram_regs.h)
- rx_eccrammode_bits_t (enum, rx72n_ram_regs.h)
- rx_eccramprcr2_bits_t (enum, rx72n_ram_regs.h)
- rx_eccramprcr_bits_t (enum, rx72n_ram_regs.h)
- rx_eepfclk_addresses_t (enum, rx72n_flash_regs.h)
- rx_encoder_deinit (function, rx_mtu_encoder.h)
- rx_encoder_deinit (function, rx_mtu_encoder.c)
- rx_encoder_init (function, rx_mtu_encoder.h)
- rx_encoder_init (function, rx_mtu_encoder.c)
- rx_encoder_read_count (function, rx_mtu_encoder.h)
- rx_encoder_read_count (function, rx_mtu_encoder.c)
- rx_encoder_read_raw (function, rx_mtu_encoder.h)
- rx_encoder_read_raw (function, rx_mtu_encoder.c)
- rx_encoder_read_velocity (function, rx_mtu_encoder.h)
- rx_encoder_read_velocity (function, rx_mtu_encoder.c)
- rx_encoder_reset (function, rx_mtu_encoder.h)
- rx_encoder_reset (function, rx_mtu_encoder.c)
- rx_encoder_set_count (function, rx_mtu_encoder.h)
- rx_encoder_set_count (function, rx_mtu_encoder.c)
- rx_err_codes_t (enum, rx_err.h)
- rx_err_from_threadx (function, rx_err.h)
- rx_err_is_error (function, rx_err.h)
- rx_err_is_ok (function, rx_err.h)
- rx_err_t (typedef, rx_err.h)
- RX_ERROR_CHECK (macro, rx_check.h)
- RX_ERROR_CHECK_WITHOUT_ABORT (macro, rx_check.h)
- rx_error_clear_fn (typedef, rx_error_interface.h)
- rx_error_get_backoff_delay_fn (typedef, rx_error_interface.h)

- rx_error_get_component_count_fn (typedef, rx_error_interface.h)
- rx_error_get_count_fn (typedef, rx_error_interface.h)
- rx_error_handler_defaults_t (enum, rx_gpio_constants.h)
- rx_error_interface_validate (function, rx_error_interface.h)
- rx_error_is_retry_limit_reached_fn (typedef, rx_error_interface.h)
- rx_error_report_fn (typedef, rx_error_interface.h)
- rx_error_reset_retry_fn (typedef, rx_error_interface.h)
- rx_exc_access_c_handler (function, rx_exception.c)
- rx_exc_access_handler (function, rx_exception.h)
- rx_exc_address_c_handler (function, rx_exception.c)
- rx_exc_address_handler (function, rx_exception.h)
- rx_exc_floating_point_c_handler (function, rx_exception.c)
- rx_exc_floating_point_handler (function, rx_exception.h)
- rx_exc_nmi_c_handler (function, rx_exception.c)
- rx_exc_nmi_handler (function, rx_exception.h)
- rx_exc_privileged_instruction_c_handler (function, rx_exception.c)
- rx_exc_privileged_instruction_handler (function, rx_exception.h)
- rx_exc_undefined_instruction_c_handler (function, rx_exception.c)
- rx_exc_undefined_instruction_handler (function, rx_exception.h)
- rx_exception_get_name (function, rx_exception.h)
- rx_exception_get_name (function, rx_exception.c)
- rx_exception_get_stats (function, rx_exception.h)
- rx_exception_get_stats (function, rx_exception.c)
- rx_exception_init (function, rx_exception.h)
- rx_exception_init (function, rx_exception.c)
- rx_exception_type_t (enum, rx_exception.h)
- rx_exception_vector_offset_t (enum, rx_exception.h)
- rx_extram_reg_offset_t (enum, rx72n_ram_regs.h)
- rx_extrammode_bits_t (enum, rx72n_ram_regs.h)
- rx_extramprcr_bits_t (enum, rx72n_ram_regs.h)
- rx_exramsts_bits_t (enum, rx72n_ram_regs.h)
- rx_faci_addresses_t (enum, rx72n_flash_regs.h)
- rx_faci_commands_t (enum, rx72n_flash_regs.h)
- rx_faeint_bits_t (enum, rx72n_flash_regs.h)
- rx_fastat_bits_t (enum, rx72n_flash_regs.h)
- rx_fbcnt_bits_t (enum, rx72n_flash_regs.h)
- rx_fbcstat_bits_t (enum, rx72n_flash_regs.h)
- rx_fcldr_bits_t (enum, rx72n_flash_regs.h)
- rx_fec_buffer_limits_t (enum, rx_fec.h)
- rx_fec_decode_hard (function, rx_fec.h)
- rx_fec_decode_hard (function, rx_fec.c)
- rx_fec_decode_soft (function, rx_fec.h)
- rx_fec_decode_soft (function, rx_fec.c)
- rx_fec_decoder_deinit (function, rx_fec.h)
- rx_fec_decoder_deinit (function, rx_fec.c)
- rx_fec_decoder_init (function, rx_fec.h)
- rx_fec_decoder_init (function, rx_fec.c)
- rx_fec_encode (function, rx_fec.h)
- rx_fec_encode (function, rx_fec.c)

- rx_fec_encoded_len (function, rx_fec.h)
- rx_fec_encoded_len (function, rx_fec.c)
- rx_fec_encoder_deinit (function, rx_fec.h)
- rx_fec_encoder_deinit (function, rx_fec.c)
- rx_fec_encoder_init (function, rx_fec.h)
- rx_fec_encoder_init (function, rx_fec.c)
- rx_fec_hard_to_soft (function, rx_fec.h)
- rx_fec_impl_constants_t (enum, rx_fec.c)
- rx_fec_output_index_t (enum, rx_fec.c)
- rx_fec_params_t (enum, rx_fec.h)
- rx_fec_soft_to_hard (function, rx_fec.h)
- rx_fentryr_bits_t (enum, rx72n_flash_regs.h)
- rx_flash_memory_addresses_t (enum, rx72n_flash_regs.h)
- rx_fpckar_bits_t (enum, rx72n_flash_regs.h)
- rx_frame_ascii_constants_t (enum, rx_frame_ascii.h)
- rx_frame_ascii_flags_str (function, rx_frame_ascii.h)
- rx_frame_ascii_flags_str (function, rx_frame_ascii.c)
- rx_frame_ascii_format (function, rx_frame_ascii.h)
- rx_frame_ascii_format (function, rx_frame_ascii.c)
- rx_frame_ascii_type_name (function, rx_frame_ascii.h)
- rx_frame_ascii_type_name (function, rx_frame_ascii.c)
- rx_frame_constants_t (enum, rx_frame.h)
- rx_frame_create_ack (function, rx_frame.h)
- rx_frame_create_ack (function, rx_frame.c)
- rx_frame_create_nack (function, rx_frame.h)
- rx_frame_create_nack (function, rx_frame.c)
- rx_frame_create_ping (function, rx_frame.h)
- rx_frame_create_ping (function, rx_frame.c)
- rx_frame_create_pong (function, rx_frame.h)
- rx_frame_create_pong (function, rx_frame.c)
- rx_frame_create_reset (function, rx_frame.h)
- rx_frame_create_reset (function, rx_frame.c)
- rx_frame_create_reset_ack (function, rx_frame.h)
- rx_frame_create_reset_ack (function, rx_frame.c)
- rx_frame_decode (function, rx_frame.h)
- rx_frame_decode (function, rx_frame.c)
- rx_frame_decode_with_resync (function, rx_frame.h)
- rx_frame_decode_with_resync (function, rx_frame.c)
- rx_frame_decoder_deinit (function, rx_frame.h)
- rx_frame_decoder_deinit (function, rx_frame.c)
- rx_frame_decoder_init (function, rx_frame.h)
- rx_frame_decoder_init (function, rx_frame.c)
- rx_frame_encode (function, rx_frame.h)
- rx_frame_encode (function, rx_frame.c)
- rx_frame_encoded_size (function, rx_frame.h)
- rx_frame_encoder_deinit (function, rx_frame.h)
- rx_frame_encoder_deinit (function, rx_frame.c)
- rx_frame_encoder_init (function, rx_frame.h)
- rx_frame_encoder_init (function, rx_frame.c)

- rx_frame_flags_t (enum, rx_frame.h)
- rx_frame_read_le16 (function, rx_frame.h)
- rx_frame_read_le32 (function, rx_frame.h)
- rx_frame_type_t (enum, rx_frame.h)
- rx_frame_type_valid (function, rx_frame.h)
- rx_frame_write_le16 (function, rx_frame.h)
- rx_frdyie_bits_t (enum, rx72n_flash_regs.h)
- rx_fstatr_bits_t (enum, rx72n_flash_regs.h)
- rx_fsunitr_bits_t (enum, rx72n_flash_regs.h)
- rx_fwepror_addresses_t (enum, rx72n_flash_regs.h)
- rx_fwepror_bits_t (enum, rx72n_flash_regs.h)
- rx_gpio_constants_t (enum, rx_gpio_constants.h)
- rx_gptw_channel_id (function, rx_gptw.h)
- rx_gptw_channel_t (enum, rx_gptw.h)
- rx_gptw_deinit (function, rx_gptw.h)
- rx_gptw_deinit (function, rx_gptw.c)
- rx_gptw_duty_calc_constants_t (enum, rx_gptw.h)
- rx_gptw_enable_output (function, rx_gptw.h)
- rx_gptw_enable_output (function, rx_gptw.c)
- rx_gptw_get_duty (function, rx_gptw.h)
- rx_gptw_get_duty (function, rx_gptw.c)
- rx_gptw_get_period (function, rx_gptw.h)
- rx_gptw_get_period (function, rx_gptw.c)
- rx_gptw_init_all_staggered (function, rx_gptw.h)
- rx_gptw_init_all_staggered (function, rx_gptw.c)
- rx_gptw_init_pwm (function, rx_gptw.h)
- rx_gptw_init_pwm (function, rx_gptw.c)
- rx_gptw_output_id (function, rx_gptw.h)
- rx_gptw_output_t (enum, rx_gptw.h)
- rx_gptw_set_duty (function, rx_gptw.h)
- rx_gptw_set_duty (function, rx_gptw.c)
- rx_gptw_set_duty_raw (function, rx_gptw.h)
- rx_gptw_set_duty_raw (function, rx_gptw.c)
- rx_gptw_start (function, rx_gptw.h)
- rx_gptw_start (function, rx_gptw.c)
- rx_gptw_stop (function, rx_gptw.h)
- rx_gptw_stop (function, rx_gptw.c)
- rx_gptw_wave_mode_t (enum, rx_gptw.h)
- rx_hal_iwdt_clear_status (function, rx_iwdt.c)
- rx_hal_iwdt_clear_status (function, rx_hal_iwdt.h)
- rx_hal_iwdt_feed (function, rx_iwdt.c)
- rx_hal_iwdt_feed (function, rx_hal_iwdt.h)
- rx_hal_iwdt_get_reset_cause (function, rx_iwdt.c)
- rx_hal_iwdt_get_reset_cause (function, rx_hal_iwdt.h)
- rx_hal_iwdt_init (function, rx_iwdt.c)
- rx_hal_iwdt_init (function, rx_hal_iwdt.h)
- rx_hal_iwdt_was_reset (function, rx_iwdt.c)
- rx_hal_iwdt_was_reset (function, rx_hal_iwdt.h)
- rx_harq_can_retry (function, rx_harq.h)

- rx_harq_can_retry (function, rx_harq.c)
- rx_harq_decode (function, rx_harq.h)
- rx_harq_decode (function, rx_harq.c)
- rx_harq_deinit (function, rx_harq.h)
- rx_harq_deinit (function, rx_harq.c)
- rx_harq_encode (function, rx_harq.h)
- rx_harq_encode (function, rx_harq.c)
- rx_harq_get_retry_count (function, rx_harq.h)
- rx_harq_get_retry_count (function, rx_harq.c)
- rx_harq_get_state (function, rx_harq.h)
- rx_harq_get_state (function, rx_harq.c)
- rx_harq_init (function, rx_harq.h)
- rx_harq_init (function, rx_harq.c)
- rx_harq_params_t (enum, rx_harq.h)
- rx_harq_reset (function, rx_harq.h)
- rx_harq_reset (function, rx_harq.c)
- rx_harq_state_t (enum, rx_harq.h)
- rx_hcsr04_callback_t (typedef, rx_hcsr04.h)
- rx_hcsr04_cancel (function, rx_hcsr04.h)
- rx_hcsr04_cancel (function, rx_hcsr04.c)
- rx_hcsr04_cm_to_inches (function, rx_hcsr04.h)
- rx_hcsr04_cm_to_inches (function, rx_hcsr04.c)
- rx_hcsr04_conversion_t (enum, rx_hcsr04.c)
- rx_hcsr04_deinit (function, rx_hcsr04.h)
- rx_hcsr04_deinit (function, rx_hcsr04.c)
- rx_hcsr04_disable_temp_compensation (function, rx_hcsr04.h)
- rx_hcsr04_disable_temp_compensation (function, rx_hcsr04.c)
- rx_hcsr04_echo_mode_t (enum, rx_hcsr04.h)
- rx_hcsr04_echo_to_cm (function, rx_hcsr04.h)
- rx_hcsr04_echo_to_cm (function, rx_hcsr04.c)
- rx_hcsr04_echo_to_cm_with_temp (function, rx_hcsr04.h)
- rx_hcsr04_echo_to_cm_with_temp (function, rx_hcsr04.c)
- rx_hcsr04_event_flags_t (enum, rx_hcsr04.c)
- rx_hcsr04_get_speed_of_sound (function, rx_hcsr04.h)
- rx_hcsr04_get_speed_of_sound (function, rx_hcsr04.c)
- rx_hcsr04_get_stats (function, rx_hcsr04.h)
- rx_hcsr04_get_stats (function, rx_hcsr04.c)
- rx_hcsr04_get_temperature (function, rx_hcsr04.h)
- rx_hcsr04_get_temperature (function, rx_hcsr04.c)
- rx_hcsr04_icu_configure (function, rx_hcsr04_icu.c)
- rx_hcsr04_icu_configure (function, rx_hcsr04_icu.h)
- rx_hcsr04_icu_disable (function, rx_hcsr04_icu.c)
- rx_hcsr04_icu_disable (function, rx_hcsr04_icu.h)
- rx_hcsr04_init (function, rx_hcsr04.h)
- rx_hcsr04_init (function, rx_hcsr04.c)
- rx_hcsr04_irq_poll_limits_t (enum, rx_hcsr04.c)
- rx_hcsr04_irq_port_t (enum, rx_hcsr04.c)
- rx_hcsr04_irq_priority_t (enum, rx_hcsr04.h)
- rx_hcsr04_irq_range_t (enum, rx_hcsr04.c)

- rx_hcsr04_irq_t (enum, rx_hcsr04.h)
- rx_hcsr04_irq_yield_t (enum, rx_hcsr04.c)
- rx_hcsr04_is_busy (function, rx_hcsr04.h)
- rx_hcsr04_is_busy (function, rx_hcsr04.c)
- rx_hcsr04_is_temp_compensation_enabled (function, rx_hcsr04.h)
- rx_hcsr04_is_temp_compensation_enabled (function, rx_hcsr04.c)
- rx_hcsr04_isr_disarm (function, rx_hcsr04_isr.c)
- rx_hcsr04_isr_disarm (function, rx_hcsr04_isr.h)
- rx_hcsr04_isr_get_duration (function, rx_hcsr04_isr.c)
- rx_hcsr04_isr_get_duration (function, rx_hcsr04_isr.h)
- rx_hcsr04_isr_register (function, rx_hcsr04_isr.c)
- rx_hcsr04_isr_register (function, rx_hcsr04_isr.h)
- rx_hcsr04_isr_start (function, rx_hcsr04_isr.c)
- rx_hcsr04_isr_start (function, rx_hcsr04_isr.h)
- rx_hcsr04_isr_unregister (function, rx_hcsr04_isr.c)
- rx_hcsr04_isr_unregister (function, rx_hcsr04_isr.h)
- rx_hcsr04_measure (function, rx_hcsr04.h)
- rx_hcsr04_measure (function, rx_hcsr04.c)
- rx_hcsr04_measure_async (function, rx_hcsr04.h)
- rx_hcsr04_measure_async (function, rx_hcsr04.c)
- rx_hcsr04_measure_blocking (function, rx_hcsr04.h)
- rx_hcsr04_measure_blocking (function, rx_hcsr04.c)
- rx_hcsr04_poll_limits_t (enum, rx_hcsr04.c)
- rx_hcsr04_range_t (enum, rx_hcsr04.h)
- rx_hcsr04_reset_stats (function, rx_hcsr04.h)
- rx_hcsr04_reset_stats (function, rx_hcsr04.c)
- rx_hcsr04_scale_t (enum, rx_hcsr04.c)
- rx_hcsr04_sensor_index_t (enum, rx_hcsr04.h)
- rx_hcsr04_set_temperature (function, rx_hcsr04.h)
- rx_hcsr04_set_temperature (function, rx_hcsr04.c)
- rx_hcsr04_shutdown_t (enum, rx_hcsr04.c)
- rx_hcsr04_timing_t (enum, rx_hcsr04.h)
- rx_hcsr04_worker_deinit (function, rx_hcsr04.h)
- rx_hcsr04_worker_deinit (function, rx_hcsr04.c)
- rx_hcsr04_worker_init (function, rx_hcsr04.h)
- rx_hcsr04_worker_init (function, rx_hcsr04.c)
- rx_hcsr04_worker_priority_t (enum, rx_hcsr04.c)
- rx_hcsr04_worker_stack_t (enum, rx_hcsr04.c)
- rx_host_irq_assert (function, rx_host_irq.h)
- rx_host_irq_assert (function, rx_host_irq.c)
- rx_host_irq_deassert (function, rx_host_irq.h)
- rx_host_irq_deassert (function, rx_host_irq.c)
- rx_host_irq_deinit (function, rx_host_irq.h)
- rx_host_irq_deinit (function, rx_host_irq.c)
- rx_host_irq_init (function, rx_host_irq.h)
- rx_host_irq_init (function, rx_host_irq.c)
- rx_host_irq_is_asserted (function, rx_host_irq.h)
- rx_host_irq_is_asserted (function, rx_host_irq.c)
- rx_i2c_constants_t (enum, rx_bus_types.h)

- rx_infrastructure_deinit (function, rx_infrastructure.h)
- rx_infrastructure_deinit (function, rx_infrastructure.c)
- rx_infrastructure_get_error_interface (function, rx_infrastructure.h)
- rx_infrastructure_get_error_interface (function, rx_infrastructure.c)
- rx_infrastructure_get_pin_interface (function, rx_infrastructure.h)
- rx_infrastructure_get_pin_interface (function, rx_infrastructure.c)
- rx_infrastructure_init (function, rx_infrastructure.h)
- rx_infrastructure_init (function, rx_infrastructure.c)
- rx_irq_filter_clk_t (enum, rx_irq_filter.h)
- rx_irq_filter_disable (function, rx_irq_filter.h)
- rx_irq_filter_disable (function, rx_irq_filter.c)
- rx_irq_filter_enable (function, rx_irq_filter.h)
- rx_irq_filter_enable (function, rx_irq_filter.c)
- rx_irq_filter_get_clock (function, rx_irq_filter.h)
- rx_irq_filter_get_clock (function, rx_irq_filter.c)
- rx_irq_filter_is_enabled (function, rx_irq_filter.h)
- rx_irq_filter_is_enabled (function, rx_irq_filter.c)
- rx_irq_filter_limits_t (enum, rx_irq_filter.h)
- RX_IS_SIMULATOR (macro, rx_simulator_config.h)
- rx_iwdt_addresses_t (enum, rx72n_iwdt_regs.h)
- rx_iwdt_check_tasks (function, rx_iwdt.h)
- rx_iwdt_check_tasks (function, rx_iwdt.c)
- rx_iwdt_feed (function, rx_iwdt.h)
- rx_iwdt_feed (function, rx_iwdt.c)
- rx_iwdt_get_status (function, rx_iwdt.h)
- rx_iwdt_get_status (function, rx_iwdt.c)
- rx_iwdt_init (function, rx_iwdt.h)
- rx_iwdt_init (function, rx_iwdt.c)
- rx_iwdt_limits_t (enum, rx_hal_iwdt.h)
- rx_iwdt_register_task (function, rx_iwdt.h)
- rx_iwdt_register_task (function, rx_iwdt.c)
- rx_iwdt_reset_cause_t (enum, rx_hal_iwdt.h)
- rx_iwdt_set_state (function, rx_iwdt.h)
- rx_iwdt_set_state (function, rx_iwdt.c)
- rx_iwdt_set_timeout_for_state (function, rx_iwdt.h)
- rx_iwdt_set_timeout_for_state (function, rx_iwdt.c)
- rx_iwdt_task_heartbeat (function, rx_iwdt.h)
- rx_iwdt_task_heartbeat (function, rx_iwdt.c)
- rx_iwdt_timeout_t (enum, rx_hal_iwdt.h)
- rx_iwdt_was_reset (function, rx_iwdt.h)
- rx_iwdt_was_reset (function, rx_iwdt.c)
- rx_le16_byte_idx_t (enum, rx_frame.h)
- rx_le32_byte_idx_t (enum, rx_frame.h)
- rx_le32_shift_t (enum, rx_frame.h)
- rx_log_debug (macro, rx_log.h)
- rx_log_debug_hex (macro, rx_log.h)
- rx_log_debug_str (macro, rx_log.h)
- rx_log_debug_val (macro, rx_log.h)
- rx_log_error (macro, rx_log.h)

- rx_log_error_hex (macro, rx_log.h)
- rx_log_error_str (macro, rx_log.h)
- rx_log_error_val (macro, rx_log.h)
- rx_log_info (macro, rx_log.h)
- rx_log_info_hex (macro, rx_log.h)
- rx_log_info_str (macro, rx_log.h)
- rx_log_info_val (macro, rx_log.h)
- rx_log_usb_get_stats (function, rx_log.h)
- rx_log_usb_get_stats (function, rx_log_usb.c)
- rx_log_usb_notify_ready (function, rx_log.h)
- rx_log_usb_notify_ready (function, rx_log_usb.c)
- rx_log_usb_putc (function, rx_log.h)
- rx_log_usb_putc (function, rx_log_usb.c)
- rx_log_usb_puthex (function, rx_log.h)
- rx_log_usb_puthex (function, rx_log_usb.c)
- rx_log_usb_putint (function, rx_log.h)
- rx_log_usb_putint (function, rx_log_usb.c)
- rx_log_usb_puts (function, rx_log.h)
- rx_log_usb_puts (function, rx_log_usb.c)
- RX_LOG_VAL_IMPL (macro, rx_log.h)
- rx_log_verbose (macro, rx_log.h)
- rx_log_verbose_hex (macro, rx_log.h)
- rx_log_verbose_str (macro, rx_log.h)
- rx_log_verbose_val (macro, rx_log.h)
- rx_log_warn (macro, rx_log.h)
- rx_log_warn_hex (macro, rx_log.h)
- rx_log_warn_str (macro, rx_log.h)
- rx_log_warn_val (macro, rx_log.h)
- rx_lpc_addresses_t (enum, rx72n_lpc_regs.h)
- rx_lpc_sizes_t (enum, rx72n_lpc_regs.h)
- rx_lvcmpcr_bits_t (enum, rx72n_lvda_regs.h)
- rx_lvd_cr0_bits_t (enum, rx72n_lvda_regs.h)
- rx_lvd_cr1_bits_t (enum, rx72n_lvda_regs.h)
- rx_lvd_interrupts_t (enum, rx72n_lvda_regs.h)
- rx_lvd_sr_bits_t (enum, rx72n_lvda_regs.h)
- rx_lvda_addresses_t (enum, rx72n_lvda_regs.h)
- rx_lvdlvlr_bits_t (enum, rx72n_lvda_regs.h)
- rx_memwait_addresses_t (enum, rx72n_system_regs.h)
- rx_memwait_bits_t (enum, rx72n_system_regs.h)
- rx_module_stop_bits_a_t (enum, rx72n_system_regs.h)
- rx_module_stop_bits_b_t (enum, rx72n_system_regs.h)
- rx_motor_deinit (function, rx_motor.h)
- rx_motor_deinit (function, rx_motor.c)
- rx_motor_emergency_stop (function, rx_motor.h)
- rx_motor_emergency_stop (function, rx_motor.c)
- rx_motor_get_duty (function, rx_motor.h)
- rx_motor_get_duty (function, rx_motor.c)
- rx_motor_init (function, rx_motor.h)
- rx_motor_init (function, rx_motor.c)

- rx_motor_set_duty (function, rx_motor.h)
- rx_motor_set_duty (function, rx_motor.c)
- rx_motor_stop (function, rx_motor.h)
- rx_motor_stop (function, rx_motor.c)
- rx_mpc_pin_t (enum, hardware_init.c)
- rx_mpc_set_adc (function, rx_mpc.h)
- rx_mpc_set_adc (function, rx_mpc.c)
- rx_mpc_set_gpio (function, rx_mpc.h)
- rx_mpc_set_gpio (function, rx_mpc.c)
- rx_mpc_set_gptw (function, rx_mpc.h)
- rx_mpc_set_gptw (function, rx_mpc.c)
- rx_mpc_set_irq (function, rx_mpc.h)
- rx_mpc_set_irq (function, rx_mpc.c)
- rx_mpc_set_mtu_encoder (function, rx_mpc.h)
- rx_mpc_set_mtu_encoder (function, rx_mpc.c)
- rx_mpc_set_mtu_pwm (function, rx_mpc.h)
- rx_mpc_set_mtu_pwm (function, rx_mpc.c)
- rx_mpc_set_peripheral (function, rx_mpc.h)
- rx_mpc_set_peripheral (function, rx_mpc.c)
- rx_mpc_set_riic (function, rx_mpc.h)
- rx_mpc_set_riic (function, rx_mpc.c)
- rx_mpc_set_rspi (function, rx_mpc.h)
- rx_mpc_set_rspi (function, rx_mpc.c)
- rx_mpc_set_sci (function, rx_mpc.h)
- rx_mpc_set_sci (function, rx_mpc.c)
- rx_mpc_set_tpu_encoder (function, rx_mpc.h)
- rx_mpc_set_tpu_encoder (function, rx_mpc.c)
- rx_mpc_set_usb_vbus (function, rx_mpc.h)
- rx_mpc_set_usb_vbus (function, rx_mpc.c)
- rx_mtu_channel_t (enum, rx_mtu.h)
- rx_mtu_deinit (function, rx_mtu.h)
- rx_mtu_deinit (function, rx_mtu.c)
- rx_mtu_enable_output (function, rx_mtu.h)
- rx_mtu_enable_output (function, rx_mtu.c)
- rx_mtu_get_duty (function, rx_mtu.h)
- rx_mtu_get_duty (function, rx_mtu.c)
- rx_mtu_get_period (function, rx_mtu.h)
- rx_mtu_get_period (function, rx_mtu.c)
- rx_mtu_init_pwm (function, rx_mtu.h)
- rx_mtu_init_pwm (function, rx_mtu.c)
- rx_mtu_output_t (enum, rx_mtu.h)
- rx_mtu_set_duty (function, rx_mtu.h)
- rx_mtu_set_duty (function, rx_mtu.c)
- rx_mtu_set_duty_raw (function, rx_mtu.h)
- rx_mtu_set_duty_raw (function, rx_mtu.c)
- rx_mtu_start (function, rx_mtu.h)
- rx_mtu_start (function, rx_mtu.c)
- rx_mtu_stop (function, rx_mtu.h)
- rx_mtu_stop (function, rx_mtu.c)

- rx_obstacle_detect_callback_t (typedef, rx_obstacle_detect.h)
- rx_obstacle_detect_clear_obstacle (function, rx_obstacle_detect.h)
- rx_obstacle_detect_clear_obstacle (function, rx_obstacle_detect.c)
- rx_obstacle_detect_constants_t (enum, rx_obstacle_detect.h)
- rx_obstacle_detect_deinit (function, rx_obstacle_detect.h)
- rx_obstacle_detect_deinit (function, rx_obstacle_detect.c)
- rx_obstacle_detect_get_state (function, rx_obstacle_detect.h)
- rx_obstacle_detect_get_state (function, rx_obstacle_detect.c)
- rx_obstacle_detect_get_stats (function, rx_obstacle_detect.h)
- rx_obstacle_detect_get_stats (function, rx_obstacle_detect.c)
- rx_obstacle_detect_init (function, rx_obstacle_detect.h)
- rx_obstacle_detect_init (function, rx_obstacle_detect.c)
- rx_obstacle_detect_is_obstacle_detected (function, rx_obstacle_detect.h)
- rx_obstacle_detect_is_obstacle_detected (function, rx_obstacle_detect.c)
- rx_obstacle_detect_reset_stats (function, rx_obstacle_detect.h)
- rx_obstacle_detect_reset_stats (function, rx_obstacle_detect.c)
- rx_obstacle_detect_start (function, rx_obstacle_detect.h)
- rx_obstacle_detect_start (function, rx_obstacle_detect.c)
- rx_obstacle_detect_state_t (enum, rx_obstacle_detect.h)
- rx_obstacle_detect_stop (function, rx_obstacle_detect.h)
- rx_obstacle_detect_stop (function, rx_obstacle_detect.c)
- rx_opccr_bits_t (enum, rx72n_lpc_regs.h)
- rx_pid_compute (function, rx_pid.h)
- rx_pid_compute (function, rx_pid.c)
- rx_pid_deinit (function, rx_pid.h)
- rx_pid_deinit (function, rx_pid.c)
- rx_pid_init (function, rx_pid.h)
- rx_pid_init (function, rx_pid.c)
- rx_pid_reset (function, rx_pid.h)
- rx_pid_reset (function, rx_pid.c)
- rx_pid_set_gains (function, rx_pid.h)
- rx_pid_set_gains (function, rx_pid.c)
- rx_pid_set_integral_limits (function, rx_pid.h)
- rx_pid_set_integral_limits (function, rx_pid.c)
- rx_pid_set_output_limits (function, rx_pid.h)
- rx_pid_set_output_limits (function, rx_pid.c)
- rx_pin_clear_all_fn (typedef, rx_pin_interface.h)
- rx_pin_from_pin (function, rx_port_constants.h)
- rx_pin_function_t (enum, rx_mpc.h)
- rx_pin_get_function_fn (typedef, rx_pin_interface.h)
- rx_pin_interface_validate (function, rx_pin_interface.h)
- rx_pin_is_reserved_fn (typedef, rx_pin_interface.h)
- rx_pin_number_t (enum, rx_port_constants.h)
- rx_pin_psel_t (enum, rx_mpc.h)
- rx_pin_release_fn (typedef, rx_pin_interface.h)
- rx_pin_reserve_fn (typedef, rx_pin_interface.h)
- rx_pin_t (typedef, rx_bus_config.c)
- rx_pin_validate_fn (typedef, rx_pin_interface.h)
- rx_poeg_addresses_t (enum, rx72n_poeg_regs.h)

- rx_poeg_clear_fault (function, rx_poeg.h)
- rx_poeg_clear_fault (function, rx_poeg.c)
- rx_poeg_clear_software_stop (function, rx_poeg.h)
- rx_poeg_clear_software_stop (function, rx_poeg.c)
- rx_poeg_get_fault_status (function, rx_poeg.h)
- rx_poeg_get_fault_status (function, rx_poeg.c)
- rx_poeg_init (function, rx_poeg.h)
- rx_poeg_init (function, rx_poeg.c)
- rx_poeg_interrupts_t (enum, rx72n_poeg_regs.h)
- rx_poeg_isr_priority_t (enum, rx_poeg.h)
- rx_poeg_motor_count_t (enum, rx_poeg.h)
- rx_poeg_software_stop (function, rx_poeg.h)
- rx_poeg_software_stop (function, rx_poeg.c)
- rx_poegg_bit_shifts_t (enum, rx72n_poeg_regs.h)
- rx_poegg_bits_t (enum, rx72n_poeg_regs.h)
- rx_port_from_pin (function, rx_port_constants.h)
- rx_port_get_base (function, rx_port_utils.h)
- rx_port_number_t (enum, rx_port_constants.h)
- rx_port_pin_t (enum, rx_port_constants.h)
- rx_port_t (typedef, rx_bus_config.c)
- rx_ppll_addresses_t (enum, rx72n_system_regs.h)
- rx_ppll_config_t (enum, rx72n_system_regs.h)
- rx_ppll_control_t (enum, rx72n_system_regs.h)
- rx_ppll_flags_t (enum, rx72n_system_regs.h)
- rx_prcr_constants_t (enum, rx_gpio_constants.h)
- rx_prcr_values_t (enum, rx_register_protection.h)
- rx_ram_mstpcrc_bits_t (enum, rx72n_ram_regs.h)
- rx_ram_reg_addr_t (enum, rx72n_ram_regs.h)
- rx_ram_reg_offset_t (enum, rx72n_ram_regs.h)
- rx_ram_region_addr_t (enum, rx72n_ram_regs.h)
- rx_rammode_bits_t (enum, rx72n_ram_regs.h)
- rx_ramprcr_bits_t (enum, rx72n_ram_regs.h)
- rx_ramsts_bits_t (enum, rx72n_ram_regs.h)
- rx_receive_result_t (enum, rx_usb_comm.c)
- rx_register_guard_config_t (enum, rx_register_guard.h)
- rx_register_guard_get_correction_count (function, rx_register_guard.h)
- rx_register_guard_get_correction_count (function, rx_register_guard.c)
- rx_register_guard_init (function, rx_register_guard.h)
- rx_register_guard_init (function, rx_register_guard.c)
- rx_register_guard_is_initialized (function, rx_register_guard.h)
- rx_register_guard_is_initialized (function, rx_register_guard.c)
- rx_register_guard_refresh (function, rx_register_guard.h)
- rx_register_guard_refresh (function, rx_register_guard.c)
- rx_register_guard_reset_count (function, rx_register_guard.h)
- rx_register_guard_reset_count (function, rx_register_guard.c)
- RX_RELEASE_PIN (macro, rx_infrastructure.h)
- RX_REPORT_ERROR (macro, rx_infrastructure.h)
- RX_RESERVE_PIN (macro, rx_infrastructure.h)
- rx_retransmit_defaults_retries_t (enum, rx_spi_comm.h)

- rx_retransmit_defaults_timing_t (enum, rx_spi_comm.h)
- RX_RETURN_NULL_ON_ERROR (macro, rx_check.h)
- RX_RETURN_ON_ERROR (macro, rx_check.h)
- RX_RETURN_VOID_ON_ERROR (macro, rx_check.h)
- rx_riic_limits_t (enum, rx_bus_types.h)
- rx_rom_cache_addresses_t (enum, rx72n_flash_regs.h)
- rx_romce_bits_t (enum, rx72n_flash_regs.h)
- rx_romciv_bits_t (enum, rx72n_flash_regs.h)
- rx_rspi_addresses_t (enum, rx72n_rspi_regs.h)
- rx_rspi_limits_t (enum, rx_bus_types.h)
- rx_rstckcr_bits_t (enum, rx72n_lpc_regs.h)
- rx_rstsr_addresses_t (enum, rx72n_system_regs.h)
- rx_sci_limits_t (enum, rx_bus_types.h)
- rx_session_constants_t (enum, rx_session.h)
- rx_session_deinit (function, rx_session.h)
- rx_session_deinit (function, rx_session.c)
- rx_session_get_rx (function, rx_session.h)
- rx_session_get_rx (function, rx_session.c)
- rx_session_get_tx (function, rx_session.h)
- rx_session_get_tx (function, rx_session.c)
- rx_session_init (function, rx_session.h)
- rx_session_init (function, rx_session.c)
- rx_session_next_tx (function, rx_session.h)
- rx_session_next_tx (function, rx_session.c)
- rx_session_reset (function, rx_session.h)
- rx_session_reset (function, rx_session.c)
- rx_session_validate_result_t (enum, rx_session.h)
- rx_session_validate_rx (function, rx_session.h)
- rx_session_validate_rx (function, rx_session.c)
- rx_soft_bit_limits_t (enum, rx_fec.h)
- rx_soft_bit_t (typedef, rx_fec.h)
- rx_spi_comm_constants_t (enum, rx_spi_comm.h)
- rx_spi_comm_data_available (function, rx_spi_comm.h)
- rx_spi_comm_data_available (function, rx_spi_comm.c)
- rx_spi_comm_deinit (function, rx_spi_comm.h)
- rx_spi_comm_deinit (function, rx_spi_comm.c)
- rx_spi_comm_init (function, rx_spi_comm.h)
- rx_spi_comm_init (function, rx_spi_comm.c)
- rx_spi_comm_process_retransmits (function, rx_spi_comm.h)
- rx_spi_comm_process_retransmits (function, rx_spi_comm.c)
- rx_spi_comm_receive (function, rx_spi_comm.h)
- rx_spi_comm_receive (function, rx_spi_comm.c)
- rx_spi_comm_send (function, rx_spi_comm.h)
- rx_spi_comm_send (function, rx_spi_comm.c)
- rx_spi_comm_send_ack (function, rx_spi_comm.h)
- rx_spi_comm_send_ack (function, rx_spi_comm.c)
- rx_spi_comm_send_nack (function, rx_spi_comm.h)
- rx_spi_comm_send_nack (function, rx_spi_comm.c)
- rx_spi_comm_send_pong (function, rx_spi_comm.h)

- rx_spi_comm_send_pong (function, rx_spi_comm.c)
- rx_spi_comm_send_reset_ack (function, rx_spi_comm.h)
- rx_spi_comm_send_reset_ack (function, rx_spi_comm.c)
- rx_spi_comm_set_auto_retransmit (function, rx_spi_comm.h)
- rx_spi_comm_set_auto_retransmit (function, rx_spi_comm.c)
- rx_spi_comm_set_control_callbacks (function, rx_spi_comm.h)
- rx_spi_comm_set_control_callbacks (function, rx_spi_comm.c)
- rx_spi_comm_set_retransmit_callbacks (function, rx_spi_comm.h)
- rx_spi_comm_set_retransmit_callbacks (function, rx_spi_comm.c)
- rx_spi_link_buffer_t (enum, rx_spi_link.h)
- rx_spi_link_constants_t (enum, rx_spi_link.h)
- rx_spi_link_deinit (function, rx_spi_link.h)
- rx_spi_link_deinit (function, rx_spi_link.c)
- rx_spi_link_fec_enabled (function, rx_spi_link.h)
- rx_spi_link_fec_enabled (function, rx_spi_link.c)
- rx_spi_link_get_state (function, rx_spi_link.h)
- rx_spi_link_get_state (function, rx_spi_link.c)
- rx_spi_link_init (function, rx_spi_link.h)
- rx_spi_link_init (function, rx_spi_link.c)
- rx_spi_link_receive (function, rx_spi_link.h)
- rx_spi_link_receive (function, rx_spi_link.c)
- rx_spi_link_reset (function, rx_spi_link.h)
- rx_spi_link_reset (function, rx_spi_link.c)
- rx_spi_link_send (function, rx_spi_link.h)
- rx_spi_link_send (function, rx_spi_link.c)
- rx_spi_link_state_t (enum, rx_spi_link.h)
- rx_spi_link_timing_t (enum, rx_spi_link.h)
- rx_stack_monitor_get_free_bytes (function, rx_stack_monitor.h)
- rx_stack_monitor_get_free_bytes (function, rx_stack_monitor.c)
- rx_stack_monitor_init (function, rx_stack_monitor.h)
- rx_stack_monitor_init (function, rx_stack_monitor.c)
- rx_swrr_addresses_t (enum, rx72n_system_regs.h)
- rx_swrr_values_t (enum, rx72n_system_regs.h)
- rx_system_addresses_t (enum, rx72n_system_regs.h)
- rx_time_ceil_offset_t (enum, rx_time_threadx.c)
- rx_time_conversion_t (enum, rx_time_constants.h)
- rx_time_get_ms_fn (typedef, rx_time_interface.h)
- rx_time_interface_validate (function, rx_time_interface.h)
- rx_time_is_elapsed_fn (typedef, rx_time_interface.h)
- rx_time_sleep_ms_fn (typedef, rx_time_interface.h)
- rx_time_threadx_get_interface (function, rx_time_interface.h)
- rx_time_threadx_get_interface (function, rx_time_threadx.c)
- rx_time_threadx_zero_t (enum, rx_time_threadx.c)
- rx_tpu_addresses_t (enum, rx72n_tpu_regs.h)
- rx_tpu_channel_count_t (enum, rx72n_tpu_regs.h)
- rx_tpu_channel_t (enum, rx_tpu.h)
- rx_tpu_deinit (function, rx_tpu.h)
- rx_tpu_deinit (function, rx_tpu.c)
- rx_tpu_encoder_deinit (function, rx_encoder_tpu.h)

- rx_tpu_encoder_deinit (function, rx_encoder_tpu.c)
- rx_tpu_encoder_init (function, rx_encoder_tpu.h)
- rx_tpu_encoder_init (function, rx_encoder_tpu.c)
- rx_tpu_encoder_read_count (function, rx_encoder_tpu.h)
- rx_tpu_encoder_read_count (function, rx_encoder_tpu.c)
- rx_tpu_encoder_read_raw (function, rx_encoder_tpu.h)
- rx_tpu_encoder_read_raw (function, rx_encoder_tpu.c)
- rx_tpu_encoder_read_velocity (function, rx_encoder_tpu.h)
- rx_tpu_encoder_read_velocity (function, rx_encoder_tpu.c)
- rx_tpu_encoder_reset (function, rx_encoder_tpu.h)
- rx_tpu_encoder_reset (function, rx_encoder_tpu.c)
- rx_tpu_encoder_set_count (function, rx_encoder_tpu.h)
- rx_tpu_encoder_set_count (function, rx_encoder_tpu.c)
- rx_tpu_init_phase_count (function, rx_tpu.h)
- rx_tpu_init_phase_count (function, rx_tpu.c)
- rx_tpu_intb_source_t (enum, rx72n_tpu_regs.h)
- rx_tpu_mstpcra_t (enum, rx72n_tpu_regs.h)
- rx_tpu_nfcr_bits_t (enum, rx72n_tpu_regs.h)
- rx_tpu_nfcr_clock_t (enum, rx72n_tpu_regs.h)
- rx_tpu_nfcr_masks_t (enum, rx72n_tpu_regs.h)
- rx_tpu_phase_mode_t (enum, rx_tpu.h)
- rx_tpu_read_count (function, rx_tpu.h)
- rx_tpu_read_count (function, rx_tpu.c)
- rx_tpu_read_direction (function, rx_tpu.h)
- rx_tpu_read_direction (function, rx_tpu.c)
- rx_tpu_reset_count (function, rx_tpu.h)
- rx_tpu_reset_count (function, rx_tpu.c)
- rx_tpu_start (function, rx_tpu.h)
- rx_tpu_start (function, rx_tpu.c)
- rx_tpu_stop (function, rx_tpu.h)
- rx_tpu_stop (function, rx_tpu.c)
- rx_tpu_tcr_clr_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tcr_masks_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tcr_shift_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tier_bits_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tmdr_masks_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tmdr_md_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tmdr_shift_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tsr_bits_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tsr_reserved_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tstr_bits_t (enum, rx72n_tpu_regs.h)
- rx_tpu_tsyrr_bits_t (enum, rx72n_tpu_regs.h)
- rx_uidr_addresses_t (enum, rx72n_flash_regs.h)
- rx_usb_addresses_t (enum, rx72n_usb_regs.h)
- rx_usb_cdc_handle_bulk_in (function, rx_usb_cdc.c)
- rx_usb_cdc_handle_bulk_in (function, rx_usb_internal.h)
- rx_usb_cdc_handle_bulk_out (function, rx_usb_cdc.c)
- rx_usb_cdc_handle_bulk_out (function, rx_usb_internal.h)
- rx_usb_cdc_handle_setup (function, rx_usb_cdc.c)

- rx_usb_cdc_handle_setup (function, rx_usb_internal.h)
- rx_usb_cdc_init (function, rx_usb_cdc.c)
- rx_usb_cdc_init (function, rx_usb_internal.h)
- rx_usb_comm_constants_t (enum, rx_usb_comm.h)
- rx_usb_comm_data_available (function, rx_usb_comm.h)
- rx_usb_comm_data_available (function, rx_usb_comm.c)
- rx_usb_comm_deinit (function, rx_usb_comm.h)
- rx_usb_comm_deinit (function, rx_usb_comm.c)
- rx_usb_comm_flush_rx (function, rx_usb_comm.h)
- rx_usb_comm_flush_rx (function, rx_usb_comm.c)
- rx_usb_comm_get_mode (function, rx_usb_comm.h)
- rx_usb_comm_get_mode (function, rx_usb_comm.c)
- rx_usb_comm_header_size_t (enum, rx_usb_comm.c)
- rx_usb_comm_init (function, rx_usb_comm.h)
- rx_usb_comm_init (function, rx_usb_comm.c)
- rx_usb_comm_is_ready (function, rx_usb_comm.h)
- rx_usb_comm_is_ready (function, rx_usb_comm.c)
- rx_usb_comm_loop_limits_t (enum, rx_usb_comm.c)
- rx_usb_comm_mode_t (enum, rx_usb_comm.h)
- rx_usb_comm_receive (function, rx_usb_comm.h)
- rx_usb_comm_receive (function, rx_usb_comm.c)
- rx_usb_comm_send (function, rx_usb_comm.h)
- rx_usb_comm_send (function, rx_usb_comm.c)
- rx_usb_comm_send_pong (function, rx_usb_comm.h)
- rx_usb_comm_send_pong (function, rx_usb_comm.c)
- rx_usb_comm_send_reset_ack (function, rx_usb_comm.h)
- rx_usb_comm_send_reset_ack (function, rx_usb_comm.c)
- rx_usb_comm_sequence_constants_t (enum, rx_usb_comm.c)
- rx_usb_comm_set_control_callbacks (function, rx_usb_comm.h)
- rx_usb_comm_set_control_callbacks (function, rx_usb_comm.c)
- rx_usb_comm_set_mode (function, rx_usb_comm.h)
- rx_usb_comm_set_mode (function, rx_usb_comm.c)
- rx_usb_comm_sleep_t (enum, rx_usb_comm.c)
- rx_usb_comm_sync_constants_t (enum, rx_usb_comm.c)
- rx_usb_comm_sync_result_t (enum, rx_usb_comm.c)
- rx_usb_count_bus_reset (function, rx_usb.c)
- rx_usb_count_bus_reset (function, rx_usb_internal.h)
- rx_usb_count_suspend (function, rx_usb.c)
- rx_usb_count_suspend (function, rx_usb_internal.h)
- rx_usb_deinit (function, rx_usb.c)
- rx_usb_event_t (enum, rx_usb.h)
- rx_usb_find_port_by_interface (function, rx_usb.c)
- rx_usb_find_port_by_interface (function, rx_usb_internal.h)
- rx_usb_find_port_by_pipe (function, rx_usb.c)
- rx_usb_find_port_by_pipe (function, rx_usb_internal.h)
- rx_usb_flush (function, rx_usb.c)
- rx_usb_get_line_coding (function, rx_usb.c)
- rx_usb_get_port_config (function, rx_usb.c)
- rx_usb_get_port_config (function, rx_usb_internal.h)

- rx_usb_get_state (function, rx_usb.c)
- rx_usb_get_stats (function, rx_usb.c)
- rx_usb_hw_attach (function, rx_usb_hw.c)
- rx_usb_hw_attach (function, rx_usb_internal.h)
- rx_usb_hw_configure_pipe (function, rx_usb_hw.c)
- rx_usb_hw_configure_pipe (function, rx_usb_internal.h)
- rx_usb_hw_deinit (function, rx_usb_hw.c)
- rx_usb_hw_deinit (function, rx_usb_internal.h)
- rx_usb_hw_detach (function, rx_usb_hw.c)
- rx_usb_hw_detach (function, rx_usb_internal.h)
- rx_usb_hw_fifo_read (function, rx_usb_hw.c)
- rx_usb_hw_fifo_read (function, rx_usb_internal.h)
- rx_usb_hw_fifo_write (function, rx_usb_hw.c)
- rx_usb_hw_fifo_write (function, rx_usb_internal.h)
- rx_usb_hw_get_bus_state (function, rx_usb_hw.c)
- rx_usb_hw_get_bus_state (function, rx_usb_internal.h)
- rx_usb_hw_init (function, rx_usb_hw.c)
- rx_usb_hw_init (function, rx_usb_internal.h)
- rx_usb_hw_set_address (function, rx_usb_hw.c)
- rx_usb_hw_set_address (function, rx_usb_internal.h)
- rx_usb_init (function, rx_usb.c)
- rx_usb_interrupt_vector_t (enum, rx72n_usb_regs.h)
- rx_usb_invoke_callback (function, rx_usb.c)
- rx_usb_invoke_callback (function, rx_usb_internal.h)
- rx_usb_is_configured (function, rx_usb.c)
- rx_usb_isr_handler (function, rx_usb_isr.c)
- rx_usb_port_id_t (enum, rx_usb.h)
- rx_usb_putc (function, rx_usb.c)
- rx_usb_puthex (function, rx_usb.c)
- rx_usb_putint (function, rx_usb.c)
- rx_usb_puts (function, rx_usb.c)
- rx_usb_read (function, rx_usb.c)
- rx_usb_reset_stats (function, rx_usb.c)
- rx_usb_rx_available (function, rx_usb.c)
- rx_usb_rx_push (function, rx_usb.c)
- rx_usb_rx_push (function, rx_usb_internal.h)
- rx_usb_set_callback (function, rx_usb.c)
- rx_usb_set_line_coding (function, rx_usb.c)
- rx_usb_set_line_coding (function, rx_usb_internal.h)
- rx_usb_set_state (function, rx_usb.c)
- rx_usb_set_state (function, rx_usb_internal.h)
- rx_usb_tx_available (function, rx_usb.c)
- rx_usb_tx_pop (function, rx_usb.c)
- rx_usb_tx_pop (function, rx_usb_internal.h)
- rx_usb_write (function, rx_usb.c)
- RX_VALIDATE_INIT (macro, rx_check.h)
- RX_VALIDATE_PTR (macro, rx_check.h)
- rx_wdt_addresses_t (enum, rx72n_wdt_regs.h)
- rx_wdt_feed (function, rx_wdt.h)

- rx_wdt_feed (function, rx_wdt.c)
- rx_wdt_init (function, rx_wdt.h)
- rx_wdt_init (function, rx_wdt.c)
- rx_wdt_start (function, rx_wdt.h)
- rx_wdt_start (function, rx_wdt.c)
- rx_wdt_stop (function, rx_wdt.h)
- rx_wdt_stop (function, rx_wdt.c)

S

- s_active_brake_in_progress (variable, motor_control_task.c)
- s_adc0_config (variable, main.c)
- s_adc_unit_initialized (variable, adc.c)
- s_adc_unit_resolution (variable, adc.c)
- s_boot_buffer (variable, rx_log_usb.c)
- s_cdc_initialized (variable, rx_usb_cdc.c)
- s_cdegc_per_degree (variable, telemetry_task.c)
- s_cdegc_per_degree (variable, temp_sensor_task.c)
- s_channel_initialized (variable, uart.c)
- s_cmt2_initialized (variable, rx_hcsr04_hal_hw.c)
- s_cmt_callback (variable, rx_cmt.c)
- s_cmt_initialized (variable, rx_cmt.c)
- s_cmt_user_data (variable, rx_cmt.c)
- s_comm_created (variable, comm_task.c)
- s_comm_pulse_remaining (variable, led_status_task.c)
- s_comm_stack (variable, comm_task.c)
- s_comm_thread (variable, comm_task.c)
- s_config_desc (variable, rx_usb_cdc.c)
- s_control_line_state (variable, rx_usb_cdc.c)
- s_counts_per_rev (variable, rx_encoder_tpu.c)
- s_counts_per_rev (variable, rx_mtu_encoder.c)
- s_crc32_table (variable, rx_crc32_sw.c)
- s_crc_initialized (variable, rx_crc32_hw.c)
- s_default_pid_integral_max (variable, shared_data.c)
- s_default_pid_integral_min (variable, shared_data.c)
- s_default_pid_kd (variable, shared_data.c)
- s_default_pid_ki (variable, shared_data.c)
- s_default_pid_kp (variable, shared_data.c)
- s_default_pid_output_max (variable, shared_data.c)
- s_default_pid_output_min (variable, shared_data.c)
- s_default_temperature_celsius (variable, rx_hcsr04.c)
- s_delay_timer_initialized (variable, rx_bus_onewire.c)
- s_device_desc (variable, rx_usb_cdc.c)
- s_ds18b20 (variable, temp_sensor_task.c)
- s_dt_sec (variable, motor_control_task.c)
- s_encoder_initialized (variable, rx_mtu_encoder.c)
- s_encoder_state (variable, rx_mtu_encoder.c)
- s_encoder_state (variable, motor_control_task.c)
- s_error_counter (variable, led_status_task.c)
- s_estop_pending_from_isr (variable, shared_data.c)

- s_exception_names (variable, rx_exception.c)
- s_exception_stats (variable, rx_exception.c)
- s_flush_poll_interval_ms (variable, rx_usb.c)
- s_global_error_handler (variable, rx_infrastructure.c)
- s_global_error_interface (variable, rx_infrastructure.c)
- s_global_pin_interface (variable, rx_infrastructure.c)
- s_global_pin_validator (variable, rx_infrastructure.c)
- s_gpio_config (variable, main.c)
- s_gptw_channels (variable, rx_poeg.c)
- s_gptw_initialized (variable, rx_gptw.c)
- s_gptw_ns_per_second (variable, rx_gptw.c)
- s_gptw_period (variable, rx_gptw.c)
- s_gptw_period_max (variable, rx_gptw.c)
- s_gtintad_values (variable, rx_poeg.c)
- s_hcsr04_us_per_cm (variable, rx_hcsr04_isr.h)
- s_hcsr04_worker_thread (variable, rx_hcsr04.c)
- s_heartbeat_counter (variable, led_status_task.c)
- s_hex_chars (variable, rx_frame_ascii.c)
- s_hw_initialized (variable, rx_usb_hw.c)
- s_infrastructure_initialized (variable, rx_infrastructure.c)
- s_initialized (variable, rx_exception.c)
- s_initialized (variable, rx_encoder_tpu.c)
- s_initialized (variable, rx_host_irq.c)
- s_invert_direction (variable, rx_encoder_tpu.c)
- s_invert_direction (variable, rx_mtu_encoder.c)
- s_irq_max (variable, rx_irq_filter.h)
- s_irq_state (variable, rx_hcsr04_isr.c)
- s_iwdt_config (variable, main.c)
- s_iwdt_initialized (variable, rx_iwdt.c)
- s_iwdt_state (variable, rx_iwdt.c)
- s_iwdt_timeout_table_size (variable, rx_iwdt.c)
- s_last_comm_sequence (variable, led_status_task.c)
- s_last_count (variable, rx_encoder_tpu.c)
- s_last_count (variable, rx_mtu_encoder.c)
- s_led_created (variable, led_status_task.c)
- s_led_pins (variable, led_status_task.c)
- s_led_ports (variable, led_status_task.c)
- s_led_stack (variable, led_status_task.c)
- s_led_thread (variable, led_status_task.c)
- s_line_coding (variable, rx_usb_cdc.c)
- s_log_mutex (variable, rx_log_usb.c)
- s_log_tag (variable, rx_exception.c)
- s_max_temp_celsius (variable, rx_hcsr04.c)
- s_measurement_request (variable, rx_hcsr04.c)
- s_min_temp_celsius (variable, rx_hcsr04.c)
- s_motor_created (variable, motor_control_task.c)
- s_motor_ptrs (variable, motor_control_task.c)
- s_motor_stack (variable, motor_control_task.c)
- s_motor_stack_initialized (variable, motor_control_task.c)

- s_motor_thread (variable, motor_control_task.c)
- s_motors (variable, motor_control_task.c)
- s_mps_to_cm_per_us (variable, rx_hcsr04.c)
- s_mtu_initialized (variable, rx_mtu.c)
- s_mtu_period (variable, rx_mtu.c)
- s_mutex_initialized (variable, rx_log_usb.c)
- s_obstacle_created (variable, obstacle_detect_task.c)
- s_obstacle_handle (variable, obstacle_detect_task.c)
- s_obstacle_stack (variable, obstacle_detect_task.c)
- s_obstacle_thread (variable, obstacle_detect_task.c)
- s_owewire0_config (variable, main.c)
- s_owewire_bus_name (variable, temp_sensor_task.c)
- s_owewire_max_search_devices (variable, rx_bus_owewire.c)
- s_pending (variable, rx_hcsr04.c)
- s_pending_estop_reason (variable, shared_data.c)
- s_pending_mutex (variable, rx_hcsr04.c)
- s_pid_integral_max_percent (variable, motor_control_task.c)
- s_pid_integral_min_percent (variable, motor_control_task.c)
- s_pid_kd_default (variable, motor_control_task.c)
- s_pid_ki_default (variable, motor_control_task.c)
- s_pid_kp_default (variable, motor_control_task.c)
- s_pid_output_max_percent (variable, motor_control_task.c)
- s_pid_output_min_percent (variable, motor_control_task.c)
- s_pids (variable, motor_control_task.c)
- s_poeg_groups (variable, rx_poeg.c)
- s_poeg_vectors (variable, rx_poeg.c)
- s_port0_rx_buf (variable, rx_usb.c)
- s_port0_tx_buf (variable, rx_usb.c)
- s_port1_rx_buf (variable, rx_usb.c)
- s_port1_tx_buf (variable, rx_usb.c)
- s_port2_rx_buf (variable, rx_usb.c)
- s_port2_tx_buf (variable, rx_usb.c)
- s_port_configs (variable, rx_usb.c)
- s_riic_channel_initialized (variable, riic.c)
- s_roundtrip_divisor (variable, rx_hcsr04.c)
- s_rspi_channel_initialized (variable, rspi.c)
- s_rspi_controller_initialized (variable, rspi.c)
- s_rspi_cs_config (variable, rspi.c)
- s_rx_threadx_tick_rate_hz (variable, rx_threadx_config.h)
- s_sckcr3_reset_state (variable, hardware_init.c)
- s_sensor_map (variable, rx_hcsr04_isr.c)
- s_sensor_ptrs (variable, obstacle_detect_task.c)
- s_sensors (variable, obstacle_detect_task.c)
- s_sequence (variable, telemetry_task.c)
- s_session_mutex (variable, rx_session.c)
- s_session_state (variable, comm_task.c)
- s_speed_of_sound_base_mps (variable, rx_hcsr04.c)
- s_speed_of_sound_coeff (variable, rx_hcsr04.c)
- s_spi_comm_handle (variable, comm_task.c)

- s_state (variable, rx_register_guard.c)
- s_state (variable, rx_encoder_tpu.c)
- s_state_pool (variable, rx_bus_owewire.c)
- s_stats (variable, rx_log_usb.c)
- s_string_langid (variable, rx_usb_cdc.c)
- s_string_manufacturer (variable, rx_usb_cdc.c)
- s_string_product (variable, rx_usb_cdc.c)
- s_string_serial (variable, rx_usb_cdc.c)
- s_tag (variable, rx_bus_adc.c)
- s_tag (variable, rx_bus_config.c)
- s_tag (variable, rx_bus_gpio.c)
- s_tag (variable, rx_bus_i2c.c)
- s_tag (variable, rx_bus_manager.c)
- s_tag (variable, rx_bus_owewire.c)
- s_tag (variable, rx_comm_manager.c)
- s_tag (variable, rx_iwdt.c)
- s_tag (variable, rx_encoder_tpu.c)
- s_tag (variable, rx_mtu_encoder.c)
- s_tag (variable, adc.c)
- s_tag (variable, riic.c)
- s_tag (variable, rspi.c)
- s_tag (variable, rx_cmt.c)
- s_tag (variable, rx_gptw.c)
- s_tag (variable, rx_host_irq.c)
- s_tag (variable, rx_mpc.c)
- s_tag (variable, rx_mtu.c)
- s_tag (variable, rx_poeg.c)
- s_tag (variable, rx_tpu.c)
- s_tag (variable, rx_hcsr04_icu.c)
- s_tag (variable, rx_motor.c)
- s_tag (variable, rx_pid.c)
- s_tag (variable, rx_session.c)
- s_tag (variable, rx_spi_comm.c)
- s_tag (variable, rx_spi_link.c)
- s_tag (variable, rx_usb.c)
- s_tag (variable, rx_usb_cdc.c)
- s_tag (variable, rx_usb_hw.c)
- s_tag (variable, rx_usb_isr.c)
- s_tag (variable, rx_usb_comm.c)
- s_tag (variable, rx_clock_power_init.c)
- s_tag (variable, rx_stack_monitor.c)
- s_tag (variable, shared_data.c)
- s_tag (variable, comm_task.c)
- s_tag (variable, led_status_task.c)
- s_tag (variable, motor_control_task.c)
- s_tag (variable, obstacle_detect_task.c)
- s_tag (variable, telemetry_task.c)
- s_tag (variable, temp_sensor_task.c)
- s_tag (variable, watchdog_monitor_task.c)

- s_task_name (variable, motor_control_task.c)
- s_task_name (variable, watchdog_monitor_task.c)
- s_telem_buffer (variable, telemetry_task.c)
- s_telem_created (variable, telemetry_task.c)
- s_telem_stack (variable, telemetry_task.c)
- s_telem_thread (variable, telemetry_task.c)
- s_temp_created (variable, temp_sensor_task.c)
- s_temp_stack (variable, temp_sensor_task.c)
- s_temp_thread (variable, temp_sensor_task.c)
- s_time_mutex (variable, rx_hcsr04_hal_hw.c)
- s_time_mutex_initialized (variable, rx_hcsr04_hal_hw.c)
- s_timeout_estop_triggered (variable, motor_control_task.c)
- s_timeout_table (variable, rx_iwtdt.c)
- s_tpu_initialized (variable, rx_tpu.c)
- s_usb (variable, rx_usb.c)
- s_usb_comm_handle (variable, comm_task.c)
- s_usb_ready (variable, rx_log_usb.c)
- s_velocity_near_stopped_mps (variable, motor_control_task.c)
- s_watchdog_created (variable, watchdog_monitor_task.c)
- s_watchdog_stack (variable, watchdog_monitor_task.c)
- s_watchdog_thread (variable, watchdog_monitor_task.c)
- s_wdt_state (variable, rx_wdt.c)
- s_worker_initialized (variable, rx_hcsr04.c)
- s_worker_shutdown_sem (variable, rx_hcsr04.c)
- s_worker_stack (variable, rx_hcsr04.c)
- s_zero_handle (variable, rx_obstacle_detect.c)
- s_zero_handle (variable, rx_spi_comm.c)
- s_zero_handle (variable, rx_usb_comm.c)
- s_zero_link (variable, rx_spi_link.c)
- s_zero_mgr (variable, rx_comm_manager.c)
- s_zero_validator (variable, rx_pin_validator.c)
- sci_mstp_b_bits_t (enum, uart.c)
- sci_register_values_t (enum, uart.c)
- sckcr3_cksel_t (enum, rx72n_system_regs.h)
- sckcr_bits_t (enum, rx72n_system_regs.h)
- sckcr_divider_t (enum, rx72n_system_regs.h)
- shared_data_clear_estop (function, shared_data.c)
- shared_data_clear_estop (function, shared_data.h)
- shared_data_clear_pid_update_flag (function, shared_data.c)
- shared_data_clear_pid_update_flag (function, shared_data.h)
- shared_data_commit_isr_estop (function, shared_data.c)
- shared_data_commit_isr_estop (function, shared_data.h)
- shared_data_constants_t (enum, shared_data.h)
- shared_data_get_estop_reason (function, shared_data.c)
- shared_data_get_estop_reason (function, shared_data.h)
- shared_data_get_motor_command (function, shared_data.c)
- shared_data_get_motor_command (function, shared_data.h)
- shared_data_get_motor_state (function, shared_data.c)
- shared_data_get_motor_state (function, shared_data.h)

- `shared_data_get_obstacle` (function, `shared_data.c`)
- `shared_data_get_obstacle` (function, `shared_data.h`)
- `shared_data_get_pid_gains` (function, `shared_data.c`)
- `shared_data_get_pid_gains` (function, `shared_data.h`)
- `shared_data_get_temp` (function, `shared_data.c`)
- `shared_data_get_temp` (function, `shared_data.h`)
- `shared_data_init` (function, `shared_data.c`)
- `shared_data_init` (function, `shared_data.h`)
- `shared_data_internal_constants_t` (enum, `shared_data.c`)
- `shared_data_is_comm_timeout` (function, `shared_data.c`)
- `shared_data_is_comm_timeout` (function, `shared_data.h`)
- `shared_data_is_estop_active` (function, `shared_data.c`)
- `shared_data_is_estop_active` (function, `shared_data.h`)
- `shared_data_pid_update_pending` (function, `shared_data.c`)
- `shared_data_pid_update_pending` (function, `shared_data.h`)
- `shared_data_set_event` (function, `shared_data.c`)
- `shared_data_set_event` (function, `shared_data.h`)
- `shared_data_set_motor_command` (function, `shared_data.c`)
- `shared_data_set_motor_command` (function, `shared_data.h`)
- `shared_data_set_pid_gains` (function, `shared_data.c`)
- `shared_data_set_pid_gains` (function, `shared_data.h`)
- `shared_data_trigger_estop` (function, `shared_data.c`)
- `shared_data_trigger_estop` (function, `shared_data.h`)
- `shared_data_trigger_estop_isr_safe` (function, `shared_data.c`)
- `shared_data_trigger_estop_isr_safe` (function, `shared_data.h`)
- `shared_data_update_last_comm_tick` (function, `shared_data.c`)
- `shared_data_update_last_comm_tick` (function, `shared_data.h`)
- `shared_data_update_motor_state` (function, `shared_data.c`)
- `shared_data_update_motor_state` (function, `shared_data.h`)
- `shared_data_update_obstacle` (function, `shared_data.c`)
- `shared_data_update_obstacle` (function, `shared_data.h`)
- `shared_data_update_temp` (function, `shared_data.c`)
- `shared_data_update_temp` (function, `shared_data.h`)
- `shared_data_wait_event` (function, `shared_data.c`)
- `shared_data_wait_event` (function, `shared_data.h`)
- `shared_event_flags_t` (enum, `shared_data.h`)
- `simulator_clock_timing_t` (enum, `rx_simulator_config.h`)
- `spi_init` (function, `hardware_init.c`)
- `spoer_bits_t` (enum, `rx72n_poe3_regs.h`)
- `stack_fill_constants_t` (enum, `rx_stack_monitor.c`)
- `string` (variable, `rx_usb_cdc.c`)
- `swrr_reg` (function, `rx72n_system_regs.h`)
- `syscr0_bits_t` (enum, `rx72n_system_regs.h`)
- `syscr1_bits_t` (enum, `rx72n_system_regs.h`)
- `system_clock_config_t` (enum, `rx_clock_power_init.c`)
- `system_regs` (function, `rx72n_system_regs.h`)
- `system_reserved_sizes_t` (enum, `rx72n_system_regs.h`)
- `system_state_t` (enum, `rx_iwtdt.h`)

T

- task_constants_t (enum, rx_obstacle_detect.c)
- telem_fault_shift_t (enum, telemetry_task.c)
- telem_motor_idx_t (enum, telemetry_task.c)
- telem_sensor_idx_t (enum, telemetry_task.c)
- telemetry_task_constants_t (enum, telemetry_task.c)
- telemetry_task_create (function, telemetry_task.h)
- telemetry_task_create (function, telemetry_task.c)
- telemetry_time_constants_t (enum, telemetry_task.c)
- telemetry_transport_t (enum, telemetry_task.c)
- temp_sensor_constants_t (enum, temp_sensor_task.c)
- temp_sensor_task_create (function, temp_sensor_task.h)
- temp_sensor_task_create (function, temp_sensor_task.c)
- temp_task_constants_t (enum, temp_sensor_task.c)
- temps_addresses_t (enum, rx72n_temps_regs.h)
- temps_calibration_t (enum, rx72n_temps_regs.h)
- temps_mstpcr_t (enum, rx72n_temps_regs.h)
- temps_timing_t (enum, rx72n_temps_regs.h)
- temps_tscdr_reg (function, rx72n_temps_regs.h)
- temps_tscr_reg (function, rx72n_temps_regs.h)
- timer_get_count (function, hardware.h)
- timer_get_count (function, timer.c)
- timer_init (function, hardware.h)
- timer_init (function, timer.c)
- timer_stop (function, hardware.h)
- timer_stop (function, timer.c)
- tpu0 (function, rx72n_tpu_regs.h)
- tpu1 (function, rx72n_tpu_regs.h)
- tpu2 (function, rx72n_tpu_regs.h)
- tpu3 (function, rx72n_tpu_regs.h)
- tpu4 (function, rx72n_tpu_regs.h)
- tpu5 (function, rx72n_tpu_regs.h)
- tpu_channel_index_t (enum, rx_tpu.c)
- tpu_control (function, rx72n_tpu_regs.h)
- tpu_enc_channel_limits_t (enum, rx_encoder_tpu.c)
- tpu_enc_counter_t (enum, rx_encoder_tpu.c)
- tpu_enc_params_t (enum, rx_encoder_tpu.c)
- tpu_enc_velocity_t (enum, rx_encoder_tpu.c)
- tpu_phase_channel_count_t (enum, rx_tpu.c)
- tpu_tcmt_zero_t (enum, rx_tpu.c)
- tpu_tcr_phase_count_t (enum, rx_tpu.c)
- tpu_tier_disable_t (enum, rx_tpu.c)
- tpu_tmdr_reset_t (enum, rx_tpu.c)
- tscr_bits_t (enum, rx72n_temps_regs.h)
- tx_application_define (function, main.c)

U

- uart_buffer_constants_t (enum, uart.c)
- uart_channel_t (enum, hardware.h)

- `uart_config_constants_t` (enum, `uart.c`)
- `uart_debug_init` (function, `hardware.h`)
- `uart_debug_init` (function, `uart.c`)
- `uart_debug_pins_t` (enum, `uart.c`)
- `uart_debug_putc` (function, `rx_log.h`)
- `uart_debug_putc` (function, `hardware.h`)
- `uart_debug_putc` (function, `uart.c`)
- `uart_debug_puthex` (function, `rx_log.h`)
- `uart_debug_puthex` (function, `hardware.h`)
- `uart_debug_puthex` (function, `uart.c`)
- `uart_debug_putint` (function, `rx_log.h`)
- `uart_debug_putint` (function, `hardware.h`)
- `uart_debug_putint` (function, `uart.c`)
- `uart_debug_puts` (function, `rx_log.h`)
- `uart_debug_puts` (function, `hardware.h`)
- `uart_debug_puts` (function, `uart.c`)
- `uart_defaults_t` (enum, `hardware.h`)
- `uart_deinit_channel` (function, `hardware.h`)
- `uart_deinit_channel` (function, `uart.c`)
- `uart_flag_constants_t` (enum, `uart.c`)
- `uart_getc_channel` (function, `hardware.h`)
- `uart_getc_channel` (function, `uart.c`)
- `uart_gpio_constants_t` (enum, `uart.c`)
- `uart_hex_constants_t` (enum, `uart.c`)
- `uart_init_channel` (function, `hardware.h`)
- `uart_init_channel` (function, `uart.c`)
- `uart_int_constants_t` (enum, `uart.c`)
- `uart_internal_constants_t` (enum, `uart.c`)
- `uart_mstpcrb_constants_t` (enum, `uart.c`)
- `uart_putc_channel` (function, `hardware.h`)
- `uart_putc_channel` (function, `uart.c`)
- `uart_puts_channel` (function, `hardware.h`)
- `uart_puts_channel` (function, `uart.c`)
- `uart_read_channel` (function, `hardware.h`)
- `uart_read_channel` (function, `uart.c`)
- `uart_rx_available` (function, `hardware.h`)
- `uart_rx_available` (function, `uart.c`)
- `uart_timeout_t` (enum, `uart.c`)
- `uart_validation_limits_t` (enum, `uart.c`)
- `uart_write_channel` (function, `hardware.h`)
- `uart_write_channel` (function, `uart.c`)
- `uidr0_reg` (function, `rx72n_flash_regs.h`)
- `uidr1_reg` (function, `rx72n_flash_regs.h`)
- `uidr2_reg` (function, `rx72n_flash_regs.h`)
- `uidr3_reg` (function, `rx72n_flash_regs.h`)
- `undefined_interrupt_source_isr` (function, `default_interrupt_handlers.c`)
- `undefined_interrupt_source_isr` (function, `r_bsp.h`)
- `usb0` (function, `rx72n_usb_regs.h`)
- `usb0_d0fifo_isr` (function, `rx_usb_isr.c`)

- `usb0_dlfifo_isr` (function, `rx_usb_isr.c`)
- `usb0_usbi_isr` (function, `rx_usb_isr.c`)
- `usb0_usbr_isr` (function, `rx_usb_isr.c`)
- `usb_address_mask_t` (enum, `rx_usb_hw.c`)
- `usb_bit_constants_t` (enum, `rx_usb_cdc.c`)
- `usb_cdc_endpoints_t` (enum, `rx72n_usb_regs.h`)
- `usb_class_code_t` (enum, `rx_usb_cdc.c`)
- `usb_config_attributes_t` (enum, `rx_usb_cdc.c`)
- `usb_config_value_t` (enum, `rx_usb_cdc.c`)
- `usb_control_endpoint_t` (enum, `rx_usb_cdc.c`)
- `usb_control_line_constants_t` (enum, `rx_usb_cdc.c`)
- `usb_dcpcfg_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_dcpcctr_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_descriptor_defaults_t` (enum, `rx_usb_cdc.c`)
- `usb_descriptor_type_t` (enum, `rx_usb_cdc.c`)
- `usb_device_id_t` (enum, `rx_usb_cdc.c`)
- `usb_dvstctr0_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_endpoint_addresses_t` (enum, `rx_usb_endpoints.h`)
- `usb_endpoint_type_t` (enum, `rx_usb_cdc.c`)
- `usb_fifo_constants_t` (enum, `rx_usb_hw.c`)
- `usb_fifoctr_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_fifosel_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_hw_timing_t` (enum, `rx_usb_hw.c`)
- `usb_icu_config_t` (enum, `rx_usb_hw.c`)
- `usb_intenb0_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_interface_number_t` (enum, `rx_usb_internal.h`)
- `usb_intsts0_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_log_buffer_config_t` (enum, `rx_log_usb.c`)
- `USB_LOG_MIRROR` (macro, `rx_log.h`)
- `usb_max_power_t` (enum, `rx_usb_cdc.c`)
- `usb_packet_size_t` (enum, `rx_usb_cdc.c`)
- `usb_pipe_assignment_t` (enum, `rx_usb_isr.c`)
- `usb_pipe_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_pipe_number_t` (enum, `rx_usb_cdc.c`)
- `usb_pipe_numbers_t` (enum, `rx_usb_isr.c`)
- `usb_pipecfg_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_pipectr_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_polling_interval_t` (enum, `rx_usb_cdc.c`)
- `usb_register_offsets_t` (enum, `rx72n_usb_regs.h`)
- `usb_request_code_t` (enum, `rx_usb_cdc.c`)
- `usb_request_type_mask_t` (enum, `rx_usb_cdc.c`)
- `usb_reserved_sizes_t` (enum, `rx72n_usb_regs.h`)
- `usb_string_index_t` (enum, `rx_usb_cdc.c`)
- `usb_string_size_t` (enum, `rx_usb_cdc.c`)
- `usb_syscfg_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_syscfg_value_t` (enum, `rx_usb_hw.c`)
- `usb_syssts0_bits_t` (enum, `rx72n_usb_regs.h`)
- `usb_validation_limits_t` (enum, `rx_usb_hw.c`)
- `usb_version_t` (enum, `rx_usb_cdc.c`)

V

- `validate_peripherals` (function, `hardware_init.c`)
- `validation_constants_t` (enum, `rx_obstacle_detect.c`)

W

- `watchdog_monitor_task_create` (function, `watchdog_monitor_task.h`)
- `watchdog_monitor_task_create` (function, `watchdog_monitor_task.c`)
- `watchdog_task_constants_t` (enum, `watchdog_monitor_task.c`)
- `wdt` (function, `rx72n_wdt_regs.h`)
- `wdt_clock_division_t` (enum, `rx_wdt.h`)
- `wdt_refresh_sequence_t` (enum, `rx72n_wdt_regs.h`)
- `wdt_reserved_sizes_t` (enum, `rx72n_wdt_regs.h`)
- `wdt_timeout_cycles_t` (enum, `rx_wdt.h`)
- `wdt_wdtr_bits_t` (enum, `rx72n_wdt_regs.h`)
- `wdt_wdtr_pos_t` (enum, `rx72n_wdt_regs.h`)
- `wdt_wdtrcr_bits_t` (enum, `rx72n_wdt_regs.h`)
- `wdt_wdtsr_bits_t` (enum, `rx72n_wdt_regs.h`)
- `wdt_wdtsr_pos_t` (enum, `rx72n_wdt_regs.h`)

Part V

API Reference – Go Gateway

Go Gateway API Reference

This section contains the complete auto-generated API reference for the STAR Gateway service. Generated from Go source code documentation.

Source: star-gateway/

Language: Go 1.25

Generator: go doc + Typst

STAR Gateway API Reference

Go Package Documentation

February 2026

Contents

1	Package: cmd.virtual_rx72n	10
1.0.1	CONSTANTS	10
1.0.2	const (.....	10
2	Package: internal.app	11
2.0.1	FUNCTIONS	11
2.0.2	func Run(ctx context.Context, config Config) error	11
2.0.3	TYPES	11
2.0.4	type Config struct {	11
2.0.5	type Servers struct {	11
3	Package: internal.controller	12
3.0.1	FUNCTIONS	12
3.0.2	func ArcadeDrive(linearVel, angularVel float32) (float32, float32)	12
3.0.3	TYPES	12
3.0.4	type Handler struct {	12
3.0.5	func NewHandler() *Handler	12
3.0.6	func NewHandlerWithGateway(gatewaySvc *service.GatewayService) *Handler	12
3.0.7	func (h *Handler) GetLastState() *starv1.ControllerState	12
3.0.8	func (h *Handler) GetSafeState() *starv1.ControllerState	12
3.0.9	func (h *Handler) ServeHTTP(w http.ResponseWriter, r *http.Request)	12
4	Package: internal.dispatcher	13
4.0.1	CONSTANTS	13
4.0.2	const (.....	13
4.0.3	VARIABLES	13
4.0.4	var (.....	13
4.0.5	TYPES	13
4.0.6	type Config struct {	13
4.0.7	type DispatchedMessage struct {	13
4.0.8	type Dispatcher interface {	13
4.0.9	func NewDispatcher(h harq.HARQ, logger *slog.Logger, cfg *Config) (Dispatcher, error) .	14
4.0.10	type MessageType uint32	14
4.0.11	const (.....	14
4.0.12	type State uint8	14
4.0.13	const (.....	14
5	Package: internal.fec	15
5.0.1	CONSTANTS	15
5.0.2	const (.....	15
5.0.3	const (.....	16
5.0.4	const DefaultMaxCombines = 3	16

5.0.5	VARIABLES	16
5.0.6	var (	16
5.0.7	var (	16
5.0.8	TYPES	16
5.0.9	type ChaseCombiner struct {	16
5.0.10	func NewChaseCombiner(config *CombinerConfig) *ChaseCombiner	16
5.0.11	func (c *ChaseCombiner) Add(softBits []SoftBit) error	16
5.0.12	func (c *ChaseCombiner) CanAdd() bool	16
5.0.13	func (c *ChaseCombiner) Combined() []SoftBit	16
5.0.14	func (c *ChaseCombiner) Count() int	16
5.0.15	func (c *ChaseCombiner) MaxCombines() int	16
5.0.16	func (c *ChaseCombiner) Reset()	17
5.0.17	type CombinerConfig struct {	17
5.0.18	type ConvolutionalEncoder struct {	17
5.0.19	func NewConvolutionalEncoder() *ConvolutionalEncoder	17
5.0.20	func (e *ConvolutionalEncoder) Encode(data []byte) ([]byte, error)	17
5.0.21	func (e *ConvolutionalEncoder) OutputLength(inputLen int) int	17
5.0.22	func (e *ConvolutionalEncoder) Rate() float64	17
5.0.23	func (e *ConvolutionalEncoder) Reset()	17
5.0.24	type Decoder interface {	17
5.0.25	type Encoder interface {	17
5.0.26	type SoftBit int8	17
5.0.27	const (	18
5.0.28	func HardToSoft(bit uint8) SoftBit	18
5.0.29	func (s SoftBit) Confidence() uint8	18
5.0.30	func (s SoftBit) ToBit() uint8	18
5.0.31	type SoftCombiner interface {	18
5.0.32	type ViterbiDecoder struct {	18
5.0.33	func NewViterbiDecoder() *ViterbiDecoder	18
5.0.34	func (d *ViterbiDecoder) DecodeHard(data []byte, expectedOutputLen int) ([]byte, error)	18
5.0.35	func (d *ViterbiDecoder) DecodeSoft(softBits []SoftBit, expectedOutputLen int) ([]byte, int, error)	18
5.0.36	func (d *ViterbiDecoder) InputLength(outputLen int) int	18
6	Package: internal.frame	19
6.0.1	CONSTANTS	19
6.0.2	const (	19
6.0.3	VARIABLES	20
6.0.4	var (	20
6.0.5	var EmptyPayload = []byte{}	20
6.0.6	var ErrInvalidLength = errors.New("frame length exceeds maximum")	20
6.0.7	var ErrSyncLoss = errors.New("frame sync lost after max search")	20
6.0.8	TYPES	20
6.0.9	type CRCError struct {	20
6.0.10	func (e *CRCError) Error() string	20
6.0.11	func (e *CRCError) Unwrap() error	20
6.0.12	type DecodeMetrics struct {	20
6.0.13	type Decoder interface {	20
6.0.14	type DefaultDecoder struct{}	20

6.0.15	func NewDecoder() *DefaultDecoder	20
6.0.16	func (d *DefaultDecoder) Decode(data []byte) (*Frame, error)	20
6.0.17	type DefaultEncoder struct {}	21
6.0.18	func NewEncoder() *DefaultEncoder	21
6.0.19	func (e *DefaultEncoder) Encode(frame *Frame) ([]byte, error)	21
6.0.20	type Encoder interface {}	21
6.0.21	type Flags uint8	21
6.0.22	const (	21
6.0.23	type Frame struct {}	21
6.0.24	func NewFrame(frameType Type, payload []byte) (*Frame, error)	21
6.0.25	type Header struct {}	21
6.0.26	type StreamDecoder struct {}	22
6.0.27	func NewStreamDecoder(r io.Reader) *StreamDecoder	22
6.0.28	func (d *StreamDecoder) Decode() (*Frame, error)	22
6.0.29	func (d *StreamDecoder) GetMetrics() DecodeMetrics	22
6.0.30	type Type uint8	22
6.0.31	const (	22
6.0.32	func (ft Type) String() string	22
7	Package: internal.harq	23
7.0.1	CONSTANTS	23
7.0.2	const (	23
7.0.3	const (	23
7.0.4	VARIABLES	23
7.0.5	var (	23
7.0.6	TYPES	24
7.0.7	type ChaseCombining struct {}	24
7.0.8	func NewChaseCombining()	24
7.0.9	func (h *ChaseCombining) Config() *Config	24
7.0.10	func (h *ChaseCombining) GetExpectedLenForTesting() int	24
7.0.11	func (h *ChaseCombining) GetRxSequence() uint16	24
7.0.12	func (h *ChaseCombining) GetState() State	24
7.0.13	func (h *ChaseCombining) GetTxSequence() uint16	24
7.0.14	func (h *ChaseCombining) Receive(ctx context.Context) (*ReceiveResult, error)	24
7.0.15	func (h *ChaseCombining) Reset()	24
7.0.16	func (h *ChaseCombining) Send(ctx context.Context, data []byte, p ...Priority) error	24
7.0.17	func (h *ChaseCombining) SetExpectedLenForTesting(expectedLen int)	24
7.0.18	func (h *ChaseCombining) SetStateForTesting(state State, txSeq, rxSeq uint16, retryCount int)	25
7.0.19	type Config struct {}	25
7.0.20	func DefaultConfig() *Config	25
7.0.21	type FrameMetadata struct {}	25
7.0.22	type HARQ interface {}	25
7.0.23	type Priority uint8	25
7.0.24	const (	25
7.0.25	type ReceiveResult struct {}	25
7.0.26	type State uint8	25
7.0.27	const (	25
7.0.28	func (s State) String() string	26
8	Package: internal.link	27

8.0.1	VARIABLES	27
8.0.2	var (	27
8.0.3	var (	27
8.0.4	TYPES	28
8.0.5	type CDCLink struct {	28
8.0.6	func NewCDCLink(t transport.Device, session *manager.SessionState) (*CDCLink, error)	28
8.0.7	func (c *CDCLink) GetRxSequence() uint16	28
8.0.8	func (c *CDCLink) GetState() harq.State	28
8.0.9	func (c *CDCLink) GetTxSequence() uint16	28
8.0.10	func (c *CDCLink) Receive(ctx context.Context) (*harq.ReceiveResult, error)	28
8.0.11	func (c *CDCLink) Reset()	28
8.0.12	func (c *CDCLink) Send(ctx context.Context, data []byte, p ...harq.Priority) error	28
8.0.13	func (c *CDCLink) SendWithSeq(ctx context.Context, data []byte, seq uint16, flags frame.Flags) error	28
8.0.14	func (c *CDCLink) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error	28
8.0.15	type SPILink struct {	29
8.0.16	func NewSPILink(t transport.Device, session *manager.SessionState, config *SPILinkConfig) (*SPILink, error)	29
8.0.17	func (s *SPILink) GetRxSequence() uint16	29
8.0.18	func (s *SPILink) GetState() harq.State	29
8.0.19	func (s *SPILink) GetTxSequence() uint16	29
8.0.20	func (s *SPILink) Receive(ctx context.Context) (*harq.ReceiveResult, error)	29
8.0.21	func (s *SPILink) Reset()	30
8.0.22	func (s *SPILink) Send(ctx context.Context, data []byte, p ...harq.Priority) error	30
8.0.23	func (s *SPILink) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error	30
8.0.24	type SPILinkConfig struct {	30
8.0.25	func DefaultSPILinkConfig() *SPILinkConfig	30
9	Package: internal.manager	31
9.0.1	CONSTANTS	31
9.0.2	const (	31
9.0.3	const (	31
9.0.4	const (	31
9.0.5	const (	32
9.0.6	const (	32
9.0.7	const (	32
9.0.8	const (	32
9.0.9	const MaxGapTolerance uint16 = 10	32
9.0.10	const SessionIDPayloadSize = 4	32
9.0.11	TYPES	32
9.0.12	type Config struct {	32
9.0.13	func DefaultConfig() *Config	33
9.0.14	func (c *Config) Validate() error	33
9.0.15	type ConfigError struct {	33
9.0.16	func (e *ConfigError) Error() string	33
9.0.17	type FailureType int	33
9.0.18	const (	33

9.0.19	func (ft FailureType) String() string	33
9.0.20	type HARQReader struct {	33
9.0.21	func NewHARQReader(h harq.HARQ) *HARQReader	34
9.0.22	func (r *HARQReader) Read(p []byte) (int, error)	34
9.0.23	type HealthMetrics struct {	34
9.0.24	func NewHealthMetrics() *HealthMetrics	34
9.0.25	type HealthMonitor struct {	34
9.0.26	func NewHealthMonitor(interval time.Duration) *HealthMonitor	34
9.0.27	func (hm *HealthMonitor) Run(ctx context.Context, tm *TransportManager)	34
9.0.28	type HeartbeatManager struct {	34
9.0.29	func NewHeartbeatManager(pingInterval, failureTimeout time.Duration, onFailure func()) (*HeartbeatManager, error)	35
9.0.30	func (hm *HeartbeatManager) OnFrameReceived()	35
9.0.31	func (hm *HeartbeatManager) OnPongReceived(payload []byte) bool	35
9.0.32	func (hm *HeartbeatManager) Run(ctx context.Context, tm *TransportManager)	36
9.0.33	type HotPlugDetector struct {	36
9.0.34	func NewHotPlugDetector(pollInterval time.Duration, vid, pid uint16) *HotPlugDetector	36
9.0.35	func (hpd *HotPlugDetector) Run(ctx context.Context, eventHandler func(HotPlugEvent))	36
9.0.36	type HotPlugEvent struct {	37
9.0.37	type SessionState struct {	37
9.0.38	func NewSessionState() *SessionState	37
9.0.39	func (s *SessionState) ActivateBarrier(sessionID uint32)	37
9.0.40	func (s *SessionState) DeactivateBarrier()	37
9.0.41	func (s *SessionState) GetRxSequence() uint16	37
9.0.42	func (s *SessionState) GetSessionID() uint32	37
9.0.43	func (s *SessionState) GetTxSequence() uint16	38
9.0.44	func (s *SessionState) IncrementSessionID() uint32	38
9.0.45	func (s *SessionState) IsBarrierActive() bool	38
9.0.46	func (s *SessionState) NextTxSequence() uint16	38
9.0.47	func (s *SessionState) Reset()	38
9.0.48	func (s *SessionState) ValidateResetAck(payload []byte) bool	38
9.0.49	func (s *SessionState) ValidateRxSequence(seq uint16) bool	38
9.0.50	type State uint8	39
9.0.51	const (	39
9.0.52	func (s State) String() string	39
9.0.53	type TransportManager struct {	39
9.0.54	func NewTransportManager(config *Config) *TransportManager	39
9.0.55	func (tm *TransportManager) ForceSwitch(transportName string) error	40
9.0.56	func (tm *TransportManager) GetActiveTransport() string	40
9.0.57	func (tm *TransportManager) GetAvailableTransports() []string	40
9.0.58	func (tm *TransportManager) GetHealthMetrics() map[string]*HealthMetrics	40
9.0.59	func (tm *TransportManager) GetInternalState() State	40
9.0.60	func (tm *TransportManager) GetRxSequence() uint16	40
9.0.61	func (tm *TransportManager) GetSessionState() *SessionState	40
9.0.62	func (tm *TransportManager) GetState() harq.State	40
9.0.63	func (tm *TransportManager) GetTxSequence() uint16	41

9.0.64	func (tm *TransportManager) Receive(ctx context.Context) (*harq.ReceiveResult, error)	41
9.0.65	func (tm *TransportManager) RegisterTransport(name string, transport harq.HARQ, priority int)	41
9.0.66	func (tm *TransportManager) Reset()	41
9.0.67	func (tm *TransportManager) Send(ctx context.Context, data []byte, p ...harq.Priority) error	41
9.0.68	func (tm *TransportManager) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error	42
9.0.69	func (tm *TransportManager) Start(ctx context.Context) error	42
9.0.70	func (tm *TransportManager) Stop() error	42
9.0.71	type TransportMode string	42
9.0.72	const (.....	42
9.0.73	func ParseTransportMode(s string) (TransportMode, error)	42
9.0.74	func (tm TransportMode) IsValid() bool	42
9.0.75	type TransportWrapper struct {	42
10	Package: internal.server	44
10.0.1	CONSTANTS	46
10.0.2	const (.....	46
10.0.3	const (.....	46
10.0.4	const DefaultGracefulStopTimeout = 5 * time.Second	46
10.0.5	FUNCTIONS	46
10.0.6	func NewGRPCServer(config *GRPCConfig, logger *slog.Logger) (*grpc.Server, error) ..	46
10.0.7	func NewHTTPServer(config *HTTPConfig, logger *slog.Logger) (*http.Server, error) ..	47
10.0.8	func RunGRPCServer(ctx context.Context, srv *grpc.Server, addr string, logger *slog.Logger) (<-chan error, error)	47
10.0.9	func RunHTTPServer(ctx context.Context, srv *http.Server, logger *slog.Logger) (<-chan error, error)	48
10.0.10	TYPES	48
10.0.11	type GRPCConfig struct {	48
10.0.12	func DefaultGRPCConfig() *GRPCConfig	49
10.0.13	func (c *GRPCConfig) Validate() error	49
10.0.14	type HTTPConfig struct {	49
10.0.15	func DefaultHTTPConfig() *HTTPConfig	49
10.0.16	func (c *HTTPConfig) Validate() error	49
11	Package: internal.service	50
11.0.1	CONSTANTS	50
11.0.2	const (.....	50
11.0.3	const (.....	50
11.0.4	TYPES	50
11.0.5	type ConfigurationService struct {	50
11.0.6	func NewConfigurationService(h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *ConfigurationService	50
11.0.7	func (s *ConfigurationService) GetConfiguration(ctx context.Context, req *starv1.GetConfigurationRequest) (*starv1.GetConfigurationResponse, error)	50
11.0.8	func (s *ConfigurationService) GetMotorPidConfig(ctx context.Context, req *starv1.GetMotorPidConfigRequest) (*starv1.GetMotorPidConfigResponse, error)	51
11.0.9	func (s *ConfigurationService) ResetToDefaults(ctx context.Context, req *starv1.ResetToDefaultsRequest) (*starv1.ResetToDefaultsResponse, error)	51

11.0.10	func (s *ConfigurationService) SaveConfiguration(ctx context.Context, req *starv1.SaveConfigurationRequest) (*starv1.SaveConfigurationResponse, error)	51
11.0.11	func (s *ConfigurationService) SetConfiguration(ctx context.Context, req *starv1.SetConfigurationRequest) (*starv1.SetConfigurationResponse, error)	51
11.0.12	func (s *ConfigurationService) SetMotorPidConfig(ctx context.Context, req *starv1.SetMotorPidConfigRequest) (*starv1.SetMotorPidConfigResponse, error)	51
11.0.13	func (s *ConfigurationService) ValidateConfiguration(_ context.Context, req *starv1.ValidateConfigurationRequest) (*starv1.ValidateConfigurationResponse, error) ..	51
11.0.14	type FirmwareService struct {	51
11.0.15	func NewFirmwareService() *FirmwareService	51
11.0.16	type GatewayService struct {	51
11.0.17	func NewGatewayService() *GatewayService	52
11.0.18	func (s *GatewayService) ForwardTelemetry(.....	52
11.0.19	func (s *GatewayService) GetActiveClientCount() int32	52
11.0.20	func (s *GatewayService) GetLatestTelemetry() (.....	52
11.0.21	func (s *GatewayService) GetTelemetryAge() time.Duration	52
11.0.22	func (s *GatewayService) GetTeleopCommand(.....	52
11.0.23	func (s *GatewayService) Hub() HubNotifier	52
11.0.24	func (s *GatewayService) RegisterClient(clientID string)	52
11.0.25	func (s *GatewayService) SetHub(h HubNotifier)	52
11.0.26	func (s *GatewayService) SetPIDGains(.....	53
11.0.27	func (s *GatewayService) UnregisterClient(clientID string)	53
11.0.28	func (s *GatewayService) UpdateTeleopCommand(cmd *starv1.VelocityCommand)	53
11.0.29	type HubNotifier interface {	53
11.0.30	type MotorControlService struct {	53
11.0.31	func NewMotorControlService(h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *MotorControlService	53
11.0.32	func (s *MotorControlService) ControlStream(stream starv1.MotorControlService_ControlStreamServer) error	53
11.0.33	func (s *MotorControlService) EmergencyStop(ctx context.Context, req *starv1.EmergencyStopRequest) (*starv1.EmergencyStopResponse, error)	54
11.0.34	func (s *MotorControlService) SetMotorPower(ctx context.Context, req *starv1.SetMotorPowerRequest) (*starv1.SetMotorPowerResponse, error)	54
11.0.35	func (s *MotorControlService) SetVelocity(ctx context.Context, req *starv1.SetVelocityRequest) (*starv1.SetVelocityResponse, error)	54
11.0.36	func (s *MotorControlService) StreamEncoders(req *starv1.StreamEncodersRequest, stream starv1.MotorControlService_StreamEncodersServer) error	54
11.0.37	type TelemetryService struct {	54
11.0.38	func NewTelemetryService(ctx context.Context, h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *TelemetryService	54
11.0.39	func (s *TelemetryService) GetSystemStatus(_ context.Context, req *starv1.GetSystemStatusRequest) (*starv1.GetSystemStatusResponse, error)	54
11.0.40	func (s *TelemetryService) GetTelemetry(_ context.Context, req *starv1.GetTelemetryRequest) (*starv1.GetTelemetryResponse, error)	54
11.0.41	func (s *TelemetryService) Shutdown()	55
11.0.42	func (s *TelemetryService) StreamTelemetry(req *starv1.StreamTelemetryRequest, stream starv1.TelemetryService_StreamTelemetryServer) error	55
12	Package: internal.testutil	56
12.0.1	CONSTANTS	56

12.0.2	const (	56
12.0.3	FUNCTIONS	56
12.0.4	func NewDiscardLogger() *slog.Logger	56
12.0.5	TYPES	56
12.0.6	type MockDispatcher struct {	56
12.0.7	func (m *MockDispatcher) GetState() dispatcher.State	56
12.0.8	func (m *MockDispatcher) Start(ctx context.Context) error	56
12.0.9	func (m *MockDispatcher) Stop() error	56
12.0.10	func (m *MockDispatcher) Subscribe(msgType dispatcher.MessageType) <-chan *dispatcher.DispatchedMessage	56
12.0.11	func (m *MockDispatcher) Unsubscribe(msgType dispatcher.MessageType, ch <-chan *dispatcher.DispatchedMessage)	56
12.0.12	type MockHARQ struct {	56
12.0.13	func (m *MockHARQ) GetLastSentPayload() []byte	56
12.0.14	func (m *MockHARQ) GetRxSequence() uint16	57
12.0.15	func (m *MockHARQ) GetState() harq.State	57
12.0.16	func (m *MockHARQ) GetTxSequence() uint16	57
12.0.17	func (m *MockHARQ) Receive(ctx context.Context) (*harq.ReceiveResult, error)	57
12.0.18	func (m *MockHARQ) Reset()	57
12.0.19	func (m *MockHARQ) Send(ctx context.Context, data []byte, p ...harq.Priority) error	57
12.0.20	func (m *MockHARQ) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error	57
13	Package: internal.transport	58
13.0.1	CONSTANTS	58
13.0.2	const (	58
13.0.3	const (	58
13.0.4	const (	58
13.0.5	VARIABLES	59
13.0.6	var (	59
13.0.7	var (	59
13.0.8	TYPES	59
13.0.9	type CDCConfig struct {	59
13.0.10	func DefaultCDCConfig() *CDCConfig	59
13.0.11	type CDCTransport struct {	59
13.0.12	func NewCDCTransport(config *CDCConfig) *CDCTransport	59
13.0.13	func (c *CDCTransport) Close() error	59
13.0.14	func (c *CDCTransport) Config() *CDCConfig	60
13.0.15	func (c *CDCTransport) IsOpen() bool	60
13.0.16	func (c *CDCTransport) Open() error	60
13.0.17	func (c *CDCTransport) Receive(maxLen int) ([]byte, error)	60
13.0.18	func (c *CDCTransport) Send(data []byte) (int, error)	60
13.0.19	func (c *CDCTransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)	60
13.0.20	type Device interface {	60
13.0.21	type SPICongig struct {	60
13.0.22	func DefaultConfig() *SPICongig	61
13.0.23	type SPITransport struct {	61
13.0.24	func NewSPITransport(config *SPICongig) *SPITransport	61
13.0.25	func (s *SPITransport) Close() error	61
13.0.26	func (s *SPITransport) Config() *SPICongig	61

13.0.27	func (s *SPITransport) IsOpen() bool	61
13.0.28	func (s *SPITransport) Open() error	61
13.0.29	func (s *SPITransport) Receive(maxLen int) ([]byte, error)	61
13.0.30	func (s *SPITransport) Send(data []byte) (int, error)	61
13.0.31	func (s *SPITransport) SetReadDeadline(deadline time.Time) error	61
13.0.32	func (s *SPITransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)	61
13.0.33	type SocketTransport struct {	61
13.0.34	func NewSocketTransport(socketPath string) *SocketTransport	62
13.0.35	func (s *SocketTransport) Close() error	62
13.0.36	func (s *SocketTransport) IsOpen() bool	62
13.0.37	func (s *SocketTransport) Open() error	62
13.0.38	func (s *SocketTransport) Path() string	62
13.0.39	func (s *SocketTransport) Receive(maxLen int) ([]byte, error)	62
13.0.40	func (s *SocketTransport) Send(data []byte) (int, error)	62
13.0.41	func (s *SocketTransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)	62
14	Package: internal.ws	63
14.0.1	CONSTANTS	63
14.0.2	const (	63
14.0.3	VARIABLES	63
14.0.4	var ErrBroadcastFull = errors.New("ws: hub broadcast channel full")	63
14.0.5	var ErrInvalidEnvelope = errors.New("ws: nil envelope passed to Broadcast")	63
14.0.6	FUNCTIONS	64
14.0.7	func NewHandler(hub *Hub, motorService MotorController, logger *slog.Logger, allowInsecureWebsocket bool) http.Handler	64
14.0.8	TYPES	64
14.0.9	type Client struct {	64
14.0.10	func NewClient(hub *Hub, conn *websocket.Conn, motorService MotorController, logger *slog.Logger) *Client	64
14.0.11	type Hub struct {	64
14.0.12	func NewHub(logger *slog.Logger) *Hub	64
14.0.13	func (h *Hub) Broadcast(ctx context.Context, env *starv1.STAREnvelope) error	65
14.0.14	func (h *Hub) ClientCount() int	65
14.0.15	func (h *Hub) Run(ctx context.Context)	65
14.0.16	func (h *Hub) SetInboundDispatcher(d InboundDispatcher)	65
14.0.17	type HubNotifier interface {	65
14.0.18	type InboundDispatcher interface {	65
14.0.19	type MotorController interface {	65

1 Package: cmd.virtual_rx72n

Virtual RX72N - Hardware-in-the-Loop Simulator

This program simulates the RX72N motor controller firmware for testing and development without physical hardware. It listens on a Unix domain socket and responds to Gateway commands.

Usage:

```
go run ./cmd/virtual_rx72n/
```

The simulator will:

1. Create a Unix socket at /tmp/star_rx72n.sock
2. Listen for connections from the Gateway
3. Echo back modified data to prove it's the simulator
4. Handle Ctrl+C gracefully

STAR Project - Texas A&M University January 2026

1.0.1 CONSTANTS

1.0.2 const (

SocketPath = "/tmp/star_rx72n.sock"

MaxFrameSize = frame.MaxFrameSize

SimulatorMarker = 0xFF

SignalBufferSize = 1

PingPayloadSize = 4

DefaultSimLatencyMs = 10

MinSimLatencyMs = 0

MaxSimLatencyMs = 1000

SeqMask = 0xFFFF

EnvSimLatencyMs = "SIM_LATENCY_MS")

2 Package: internal.app

Package app encapsulates the main application logic for the star-gateway.

STAR Project - Texas A&M University January 2026

2.0.1 FUNCTIONS

2.0.2 func Run(ctx context.Context, config Config) error

Run starts the STAR Gateway application with the given configuration.

This is the main entry point that orchestrates the complete application lifecycle:

1. Logger initialization
2. Transport layer `setup` (SPI/USB with automatic failover)
3. Message dispatcher initialization
4. Service layer `initialization` (gRPC and HTTP/WebSocket)
5. Server `startup` (background goroutines)
6. Signal `handling` (graceful shutdown on SIGINT/SIGTERM)

The function blocks until a shutdown signal is received or a fatal error occurs.

2.0.3 TYPES

2.0.4 type Config struct {

SimulationMode bool SocketPath string TransportMode manager.TransportMode }

Config holds the application configuration.

2.0.5 type Servers struct {

HTTPServer *http.Server GRPCServer *grpc.Server }

3 Package: internal.controller

3.0.1 FUNCTIONS

3.0.2 func ArcadeDrive(linearVel, angularVel float32) (float32, float32)

ArcadeDrive converts gamepad stick inputs to left/right motor velocities.

linearVel: [-1.0, 1.0] (forward/backward) angularVel: [-1.0, 1.0]

(right/left) Returns: (leftMotor, rightMotor) in range [-1.0, 1.0]

3.0.3 TYPES

3.0.4 type Handler struct { }

3.0.5 func NewHandler() *Handler

3.0.6 func NewHandlerWithGateway(gatewaySvc *service.GatewayService) *Handler

3.0.7 func (h *Handler) GetLastState() *starv1.ControllerState

3.0.8 func (h *Handler) GetSafeState() *starv1.ControllerState

GetSafeState returns the last state, or a zero state if the last message was received more than 200ms ago.

3.0.9 func (h *Handler) ServeHTTP(w http.ResponseWriter, r *http.Request)

4 Package: internal.dispatcher

Package dispatcher implements a centralized message router for the star-gateway.

The Dispatcher solves the concurrency race condition where multiple services (MotorControlService, TelemetryService) all call HARQ.Receive() in parallel, causing frame stealing and deserialization errors.

Architecture:

1. Dispatcher owns the single HARQ.Receive() loop
2. Services subscribe to specific message types
3. Dispatcher demultiplexes messages via WireMessage.oneof
4. Subscribed channels receive only their type

STAR Project - Texas A&M University January 2026

4.0.1 CONSTANTS

4.0.2 const (

DefaultShutdownTimeout = 5 * time.Second

DefaultSubscriberBufferSize = 100

DefaultReceiveInterval = 10 * time.Millisecond)

Configuration constants.

4.0.3 VARIABLES

4.0.4 var (

ErrDispatcherStopped = errors.New("dispatcher: dispatcher is stopped")

ErrHARQNil = errors.New("dispatcher: HARQ handler is nil")

ErrLoggerNil = errors.New("dispatcher: logger is nil"))

Predefined errors for dispatcher operations.

4.0.5 TYPES

4.0.6 type Config struct {

ShutdownTimeout time.Duration

ReceiveInterval time.Duration }

Config holds dispatcher configuration parameters.

4.0.7 type DispatchedMessage struct {

WireMsg *starv1.WireMessage Metadata *harq.FrameMetadata }

DispatchedMessage bundles a WireMessage with frame metadata for diagnostics.

Exported so services can access the metadata.

4.0.8 type Dispatcher interface {

Subscribe(msgType MessageType) <-chan *DispatchedMessage

Unsubscribe(msgType MessageType, ch <-chan *DispatchedMessage)

Start(ctx context.Context) error

Stop() error

```
GetState() State }
```

Dispatcher defines the **interface for** centralized message routing.

4.0.9 func NewDispatcher(h harq.HARQ, logger *slog.Logger, cfg *Config) (Dispatcher, error)

NewDispatcher creates a new message dispatcher. The harq handler and logger are required.

4.0.10 type MessageType uint32

MessageType identifies a message **type in the** WireMessage union.

4.0.11 const (

MessageTypeUnknown MessageType = iota

MessageTypeVelocityCommand

MessageTypeEmergencyStopCommand

MessageTypeMotorPowerCommand

MessageTypeTelemetryData

MessageTypeEncoderData

MessageTypePidConfig)

4.0.12 type State uint8

State represents the dispatcher operational state.

4.0.13 const (

StateIdle State = iota

StateRunning

StateStopping

StateStopped)

5 Package: internal.fec

Chase Combiner implementation for HARQ Type I.

Chase Combining stores soft bits from failed transmission attempts and combines them element-wise (addition) before decoding. This improves the effective SNR with each retransmission.

Example:

```
Transmission 1: soft bits S1 (decode fails)
Transmission 2: soft bits S2 (decode fails)
Combined: S_combined[i] = S1[i] + S2[i] (clamped to int16 range)
Decode: Viterbi(S_combined) -> success
```

STAR Project - Texas A&M University December 2025

Convolutional encoder implementation for rate-1/2, K=7 code.

Generator polynomials (NASA standard):

- G1 = 171 octal = 0b1111001 = 121 decimal
- G2 = 133 octal = 0b1011011 = 91 decimal

The encoder uses a 6-bit shift register (K-1 = 6 memory elements). For each input bit, two output bits are generated.

STAR Project - Texas A&M University December 2025

Package fec implements Forward Error Correction for the HARQ protocol.

This package provides a rate-1/2 convolutional encoder (K=7) with soft Viterbi decoding, designed for Chase Combining HARQ where soft bits from multiple transmissions are accumulated before decoding.

FEC Parameters:

- Rate: 1/2 (2 output bits per input bit)
- Constraint Length: K=7 (64 states)
- Generator Polynomials: G1=171o, G2=133o (NASA standard)

Reference: docs/sections/01_nanopb_protocol.tex (Layer 3: HARQ)

STAR Project - Texas A&M University December 2025

Viterbi decoder implementation for rate-1/2, K=7 convolutional code.

This decoder supports both soft and hard decision decoding. Soft decision decoding provides approximately 2dB coding gain over hard decision.

The decoder uses the add-compare-select (ACS) algorithm with traceback to find the maximum likelihood path through the trellis.

STAR Project - Texas A&M University December 2025

5.0.1 CONSTANTS

5.0.2 const (

ConstraintLength = 7

NumStates = 1 << (ConstraintLength - 1)

G1Octal = 0171

G2Octal = 0133

TailBits = ConstraintLength - 1)

Convolutional encoder constants.

5.0.3 const (

TracebackDepth = 5 * ConstraintLength

MaxPathMetric = 0x7FFFFFFF)

5.0.4 const DefaultMaxCombines = 3

DefaultMaxCombines is the **default** maximum number of combining attempts.

5.0.5 VARIABLES

5.0.6 var (

ErrCombinerFull = errors.New("fec: combiner full, max attempts reached")

ErrCombinerEmpty = errors.New("fec: combiner empty")

ErrSoftBitsLengthMismatch = errors.New("fec: soft bits length mismatch"))

Combiner error codes.

5.0.7 var (

ErrInvalidInput = errors.New("fec: invalid input data")

ErrInvalidSoftBits = errors.New("fec: invalid soft bits")

ErrDecodeFailed = errors.New("fec: decode failed")

ErrLengthMismatch = errors.New("fec: length mismatch"))

FEC error codes.

5.0.8 TYPES

5.0.9 type ChaseCombiner struct {

}

ChaseCombiner implements soft bit combining **for** HARQ Type I.

5.0.10 func NewChaseCombiner(config *CombinerConfig) *ChaseCombiner

NewChaseCombiner creates a new Chase Combiner with the given config.

5.0.11 func (c *ChaseCombiner) Add(softBits []SoftBit) error

Add accumulates soft bits from a new transmission attempt. The first call sets the expected length **for** subsequent calls.

5.0.12 func (c *ChaseCombiner) CanAdd() bool

CanAdd returns **true** if more transmissions can be combined.

5.0.13 func (c *ChaseCombiner) Combined() []SoftBit

Combined returns the accumulated soft bits ready **for** decoding. The accumulated values are scaled and clamped back to SoftBit **range**.

5.0.14 func (c *ChaseCombiner) Count() int

Count returns the number of transmissions combined so far.

5.0.15 func (c *ChaseCombiner) MaxCombines() int

MaxCombines returns the maximum number of combining attempts.

5.0.16 func (c *ChaseCombiner) Reset()

Reset clears the combining buffer for a new frame.

5.0.17 type CombinerConfig struct {
MaxCombines int }

CombinerConfig holds configuration for the Chase Combiner.

5.0.18 type ConvolutionalEncoder struct {
}

ConvolutionalEncoder implements a rate-1/2, K=7 convolutional encoder.

5.0.19 func NewConvolutionalEncoder() *ConvolutionalEncoder

NewConvolutionalEncoder creates a new convolutional encoder.

5.0.20 func (e *ConvolutionalEncoder) Encode(data []byte) ([]byte, error)

Encode applies convolutional encoding to the input data. For each input bit, two output bits are generated. Tail bits are appended to flush the encoder to the zero state.

Output length = (inputLen * 8 + TailBits) * 2 bits, packed into bytes.

5.0.21 func (e *ConvolutionalEncoder) OutputLength(inputLen int) int

OutputLength returns the encoded output length in bytes for a given input length.

5.0.22 func (e *ConvolutionalEncoder) Rate() float64

Rate returns the code rate (0.5 for rate-1/2).

5.0.23 func (e *ConvolutionalEncoder) Reset()

Reset resets the encoder state to zero.

5.0.24 type Decoder interface {

DecodeSoft(softBits []SoftBit, expectedOutputLen int) ([]byte, int, error)

DecodeHard(data []byte, expectedOutputLen int) ([]byte, error)

InputLength(outputLen int) int }

Decoder defines the interface for FEC decoding.

5.0.25 type Encoder interface {

Encode(data []byte) ([]byte, error)

Rate() float64

OutputLength(inputLen int) int }

Encoder defines the interface for FEC encoding.

5.0.26 type SoftBit int8

SoftBit represents a soft decision value for a received bit. Range: -127 to +127

- Negative values indicate a '0' bit (more negative = higher confidence)
- Positive values indicate a '1' bit (more positive = higher confidence)
- Zero indicates an erasure (no information)

The magnitude represents confidence `level` (log-likelihood ratio approximation).

5.0.27 const (

`SoftBitMax SoftBit = 127`

`SoftBitMin SoftBit = -127`

`SoftBitErasure SoftBit = 0`)

5.0.28 func `HardToSoft(bit uint8) SoftBit`

`HardToSoft` converts a hard `bit` (`0` or `1`) to maximum confidence soft bit.

5.0.29 func (s `SoftBit`) `Confidence() uint8`

`Confidence` returns the absolute confidence `level` (`0-127`).

5.0.30 func (s `SoftBit`) `ToBit() uint8`

`ToBit` converts a `SoftBit` to a hard `decision` (`0` or `1`).

5.0.31 type `SoftCombiner` interface {

`Add(softBits []SoftBit) error`

`Combined() []SoftBit`

`Count() int`

`Reset() }`

`SoftCombiner` accumulates soft bits from multiple transmission attempts. Used `for` Chase Combining in HARQ Type I.

5.0.32 type `ViterbiDecoder` struct {

}

`ViterbiDecoder` implements soft Viterbi decoding `for` rate-`1/2`, `K=7` code.

5.0.33 func `NewViterbiDecoder() *ViterbiDecoder`

`NewViterbiDecoder` creates a new Viterbi decoder.

5.0.34 func (d `*ViterbiDecoder`) `DecodeHard(data []byte, expectedOutputLen int) ([]byte, error)`

`DecodeHard` decodes hard decision bits. Converts hard bits to maximum confidence soft bits and decodes. `expectedOutputLen` specifies the expected decoded data length in `bytes` (`0` = auto-detect).

5.0.35 func (d `*ViterbiDecoder`) `DecodeSoft(softBits []SoftBit, expectedOutputLen int) ([]byte, int, error)`

`DecodeSoft` decodes soft bits using the Viterbi algorithm. `expectedOutputLen` specifies the expected decoded data length in bytes. If `0`, it's auto-calculated from the soft bits length. Returns the decoded data, final path metric, and any error.

5.0.36 func (d `*ViterbiDecoder`) `InputLength(outputLen int) int`

`InputLength` returns the expected soft bits length `for` a given output length.

6 Package: internal.frame

Package frame provides CRC-32 implementation.

Package frame provides frame decoding for the wire protocol.

STAR Project - Texas A&M University December 2025

Package frame defines the wire protocol frame structure and decoding logic.

The protocol uses a custom framing format: [SYNC(2)][SEQ(2)][LEN(2)][TYPE(1)][FLAGS(1)]
[PAYLOAD(0-1KB)][CRC-32(4)]

Key features: - 2-byte sync word for frame alignment (0x55AA) - 16-bit sequence number - CRC-32 (IEEE 802.3) data integrity - Max payload 1024 bytes - Type field for message classification

STAR Project - Texas A&M University

Package frame provides frame encoding for the wire protocol.

STAR Project - Texas A&M University December 2025

Package frame defines the wire protocol frame structure for RPi5 <-> RX72N communication.

Frame format:

```
[SYNC (2B, LE)][SEQ (2B, LE)][LEN (2B, LE)][TYPE (1B)][FLAGS (1B)][PAYLOAD (0-1KB)][CRC-32 (4B)]
```

All multi-byte header fields (SYNC, SEQ, LEN) are little-endian by project convention. The CRC-32 uses the IEEE 802.3 polynomial and bit-ordering; the resulting 32-bit CRC value is serialized in little-endian to match the header field convention.

STAR Project - Texas A&M University January 2026

Package frame provides a streaming decoder for the STAR protocol.

6.0.1 CONSTANTS

6.0.2 const (

SyncWord uint16 = 0x55AA

SyncSize = 2

SeqSize = 2 LenSize = 2 TypeSize = 1 FlagsSize = 1

HeaderSize = SeqSize + LenSize + TypeSize + FlagsSize

CRCSIZE = 4

MaxPayloadSize = 1024

MaxFrameSize = SyncSize + HeaderSize + MaxPayloadSize + CRCSIZE

MinFrameSize = SyncSize + HeaderSize + CRCSIZE

EmptyPayloadLength = 0)

Frame structure constants.

6.0.3 VARIABLES

6.0.4 var (

ErrNotImplemented = errors.New("not implemented") ErrInvalidSync = errors.New("invalid sync word")
 ErrInvalidCRC = errors.New("CRC validation failed") ErrPayloadTooLarge = errors.New("payload
 exceeds maximum size") ErrFrameTooShort = errors.New("frame data too short"))

Predefined errors.

6.0.5 var EmptyPayload = []byte{}

EmptyPayload is a pre-allocated empty byte slice for frames with no payload
 (ACK/NACK/PING). Using this constant avoids repeated allocations of empty
 slices.

6.0.6 var ErrInvalidLength = errors.New("frame length exceeds maximum")

ErrInvalidLength indicates frame length field exceeds limits.

6.0.7 var ErrSyncLoss = errors.New("frame sync lost after max search")

ErrSyncLoss indicates frame synchronization was lost.

6.0.8 TYPES

6.0.9 type CRCError struct {

Sequence uint16 }

CRCError is returned by the stream decoder when CRC validation fails.
 It carries the parsed sequence number so callers can send a NACK.

6.0.10 func (e *CRCError) Error() string

6.0.11 func (e *CRCError) Unwrap() error

Unwrap returns the underlying sentinel error (ErrInvalidCRC) to support Go
 1.13+ error unwrapping with errors.Is and errors.As.

6.0.12 type DecodeMetrics struct {

TotalFrames uint64 SyncErrors uint64 CRCErrors uint64 LengthErrors uint64 LastSuccess time.Time }

DecodeMetrics tracks decoder health statistics.

6.0.13 type Decoder interface {

Decode(data []byte) (*Frame, error) }

Decoder defines the interface for frame decoding.

6.0.14 type DefaultDecoder struct{}

DefaultDecoder implements the Decoder interface.

6.0.15 func NewDecoder() *DefaultDecoder

NewDecoder creates a new DefaultDecoder.

6.0.16 func (d *DefaultDecoder) Decode(data []byte) (*Frame, error)

Decode parses wire format bytes into a Frame.

Wire format (all fields in little-endian per IEEE 802.3):

[SYNC (2B)][SEQ (2B)][LEN (2B)][TYPE (1B)][FLAGS (1B)][PAYLOAD (0-1KB)][CRC-32 (4B)]

CRC-32 is validated over SYNC + Header + Payload.

6.0.17 type DefaultEncoder struct{}

DefaultEncoder implements the Encoder interface.

6.0.18 func NewEncoder() *DefaultEncoder

NewEncoder creates a new DefaultEncoder.

6.0.19 func (e *DefaultEncoder) Encode(frame *Frame) ([]byte, error)

Encode serializes a Frame into wire format bytes.

Wire format:

```
[SYNC (2B)][SEQ (2B)][LEN (2B)][TYPE (1B)][FLAGS (1B)][PAYLOAD (0-1KB)][CRC-32 (4B)]
```

Header fields (SYNC, SEQ, LEN, TYPE, FLAGS) are encoded in little-endian by project choice for consistency across platforms.

CRC-32 uses the IEEE 802.3 polynomial (0xEDB88320) with bit/byte ordering as defined by the standard. The CRC is calculated over SYNC + Header + Payload.

6.0.20 type Encoder interface {

Encode(frame *Frame) ([]byte, error) }

Encoder defines the interface for frame encoding.

6.0.21 type Flags uint8

Flags contains frame-level control flags.

6.0.22 const (

FlagNone Flags = 0x00 FlagRequiresAck Flags = 0x01 FlagRetransmit Flags = 0x02 FlagPriority Flags = 0x04 FlagFECEnabled Flags = 0x08 FlagSoftNACK Flags = 0x10)

6.0.23 type Frame struct {

Header Header

Type Type

Payload []byte

CRC uint32

Retransmits int

FECDecoded bool }

Frame represents a complete communication frame.

6.0.24 func NewFrame(frameType Type, payload []byte) (*Frame, error)

NewFrame creates a new Frame with the given type and payload.

6.0.25 type Header struct {

Sequence uint16

Length uint16

Flags Flags }

Header contains the frame header fields.

6.0.26 type StreamDecoder struct {
}

StreamDecoder reads and decodes frames from a continuous stream. It handles synchronization loss, variable length payloads, and CRC validation.

Thread Safety: StreamDecoder enforces single-reader semantics. Only one goroutine should call `Decode()` at a time. Concurrent calls will serialize via internal mutex. `GetMetrics()` may briefly block `if Decode()` is active, but this is typically negligible (<1ms) since metrics updates are fast.

The mutex protects both buffer state and metrics to ensure consistency. This design trades some concurrency `for` simplicity and correctness.

6.0.27 func NewStreamDecoder(r io.Reader) *StreamDecoder
NewStreamDecoder creates a new decoder reading from r.

6.0.28 func (d *StreamDecoder) Decode() (*Frame, error)
Decode blocks until a valid frame is read or an error occurs. It automatically handles resynchronization on stream errors.

6.0.29 func (d *StreamDecoder) GetMetrics() DecodeMetrics
GetMetrics returns the current health metrics.

6.0.30 type Type uint8
Type indicates the `type of frame` being transmitted. Not serialized in Header `struct`, but part of the wire header.

6.0.31 const (
FrameTypePing Type = 0x00 FrameTypePong Type = 0x01 FrameTypeResetAck Type = 0xFE
FrameTypeReset Type = 0xFF

FrameTypeCommand Type = 0x10 FrameTypeResponse Type = 0x11 FrameTypeAck Type = 0x12
FrameTypeNack Type = 0x13

FrameTypeUnknown Type = 0x14)

6.0.32 func (ft Type) String() string

7 Package: internal.harq

Package harq implements Hybrid Automatic Repeat reQuest (HARQ) protocol for reliable frame delivery over the SPI transport.

The HARQ mechanism uses Chase Combining (Type I) with the following properties:

- 16-bit sequence numbers with wraparound
- ACK/NACK response frames
- Configurable retry count and timeout
- Duplicate frame detection
- Forward Error Correction (FEC) with soft Viterbi decoding
- Soft bit combining across retransmissions

Chase Combining stores soft bits from failed transmissions and combines them element-wise before re-attempting Viterbi decoding. This provides approximately 2dB coding gain per additional transmission.

Reference: docs/sections/01_nanopb_protocol.tex (Layer 3: HARQ)

STAR Project - Texas A&M University December 2025

7.0.1 CONSTANTS

7.0.2 const (

DefaultMaxRetries = 3

DefaultTimeout = 10 * time.Millisecond

DefaultSequenceMax = 0xFFFF)

HARQ protocol constants from specification.

7.0.3 const (

PriorityValueEmergency uint8 = 0 PriorityValueHigh uint8 = 1 PriorityValueNormal uint8 = 2)

Protocol wire format values for priority levels

7.0.4 VARIABLES

7.0.5 var (

ErrNotImplemented = errors.New("harq: not implemented")

ErrMaxRetriesExceeded = errors.New("harq: max retries exceeded")

ErrTimeout = errors.New("harq: timeout waiting for acknowledgment")

ErrInvalidSequence = errors.New("harq: invalid sequence number")

ErrDuplicateFrame = errors.New("harq: duplicate frame detected")

ErrTransportNil = errors.New("harq: transport is nil")

ErrEncoderNil = errors.New("harq: encoder is nil")

ErrDecoderNil = errors.New("harq: decoder is nil")

ErrFECEncoderNil = errors.New("harq: FEC encoder is nil")

ErrFECDecoderNil = errors.New("harq: FEC decoder is nil")

ErrUnexpectedFrameType = errors.New("harq: unexpected frame type")

ErrInErrorState = errors.New("harq: in error state, call Reset() first")

ErrDecodeFailed = errors.New("harq: FEC decode failed after combining")

ErrNilContext = errors.New("harq: context is nil"))

Predefined errors for HARQ operations.

7.0.6 TYPES

7.0.7 type ChaseCombining struct {
}

ChaseCombining implements the HARQ interface using Chase Combining (Type I).

7.0.8 func NewChaseCombining(

config *Config, t transport.Device, encoder frame.Encoder, decoder frame.Decoder, fecEncoder
fec.Encoder, fecDecoder fec.Decoder,) *ChaseCombining

NewChaseCombining creates a new Chase Combining HARQ handler. If config is
nil, uses DefaultConfig(). The transport, frame encoder/decoder, and FEC
encoder/decoder are required dependencies.

7.0.9 func (h *ChaseCombining) Config() *Config

Config returns the current HARQ configuration.

7.0.10 func (h *ChaseCombining) GetExpectedLenForTesting() int

GetExpectedLenForTesting returns the expected decoded length for testing.

7.0.11 func (h *ChaseCombining) GetRxSequence() uint16

GetRxSequence returns the expected RX sequence number.

7.0.12 func (h *ChaseCombining) GetState() State

GetState returns the current HARQ state.

7.0.13 func (h *ChaseCombining) GetTxSequence() uint16

GetTxSequence returns the current TX sequence number.

7.0.14 func (h *ChaseCombining) Receive(ctx context.Context) (*ReceiveResult, error)

Receive waits for data with HARQ handling. On receive, attempts FEC decode.
On failure, stores soft bits and waits for retransmission. Combines soft
bits from multiple transmissions. Validates CRC and sequence number,
sends ACK for valid frames.

7.0.15 func (h *ChaseCombining) Reset()

Reset resets the HARQ state machine to initial state.

7.0.16 func (h *ChaseCombining) Send(ctx context.Context, data []byte, p ...Priority) error

Send transmits data with HARQ reliability. Applies FEC encoding, wraps in
a command frame, and waits for ACK. Retransmits the same encoded frame on
timeout or NACK up to MaxRetries. Priority defaults to PriorityNormal if not
specified.

7.0.17 func (h *ChaseCombining) SetExpectedLenForTesting(expectedLen int)

SetExpectedLenForTesting sets the expected decoded length for testing.

7.0.18 func (h *ChaseCombining) SetStateForTesting(state State, txSeq, rxSeq uint16, retryCount int)

SetStateForTesting sets internal state **for** testing purposes. This method should only be used in tests to simulate various states.

7.0.19 type Config struct {

MaxRetries int

Timeout time.Duration

FECEnabled bool }

Config holds HARQ configuration parameters.

7.0.20 func DefaultConfig() *Config

DefaultConfig returns the **default** HARQ configuration.

7.0.21 type FrameMetadata struct {

Sequence uint16 Type frame.Type ReceivedAt time.Time

Retransmits int FECDecoded bool

PathMetric int CombiningCount int }

FrameMetadata contains frame-level information **for** diagnostics.

7.0.22 type HARQ interface {

Send(ctx context.Context, data []byte, p ...Priority) error

Receive(ctx context.Context) (*ReceiveResult, error)

GetState() State

GetTxSequence() uint16

GetRxSequence() uint16

Reset() }

HARQ defines the **interface for** the HARQ protocol handler.

7.0.23 type Priority uint8

Priority represents the priority level of a HARQ transmission.

7.0.24 const (

PriorityEmergency Priority = 0 PriorityHigh Priority = 1 PriorityNormal Priority = 2)

7.0.25 type ReceiveResult struct {

Payload []byte Metadata FrameMetadata }

ReceiveResult bundles payload with frame metadata.

7.0.26 type State uint8

State represents the HARQ state machine state.

7.0.27 const (

StateIdle State = iota

StateWaitingAck

StateRetransmitting

StateCombining

StateError)

7.0.28 func (s State) String() string

String returns the string representation of a State.

8 Package: internal.link

Package link provides link layer implementations for the STAR Gateway.

CDCLink implements a lightweight link layer for USB CDC transport. Unlike SPI (which uses full HARQ with retransmissions and FEC), USB CDC relies on hardware-level reliability. CDCLink provides:

- CRC32 validation (end-to-end integrity)
- Sequence tracking via shared SessionState (prevents “Handoff Problem”)
- Gap tolerance (accepts sequence gaps <10 frames for packet loss recovery)
- Send serialization via sendMutex (prevents out-of-order transmission)
- NO retransmission (failures propagate to TransportManager for failover)
- NO FEC encoding/decoding (USB handles error correction)
- NO ACK/NACK frames (USB provides implicit ACK)

Critical Fixes Implemented:

- Fix #1: Shared SessionState prevents sequence resets during transport switching
- Fix #5: sendMutex ensures atomic sequence+write (no out-of-order frames)
- Fix #6: Gap tolerance allows recovery from rare USB packet loss
- Fix #7: Context timeout prevents blocked writes on USB disconnect

STAR Project - Texas A&M University January 2026

Package link provides link layer implementations for the STAR Gateway.

SPILink implements a full HARQ (Hybrid ARQ) link layer for SPI transport. Unlike CDCLink (lightweight USB), SPILink uses:

- ACK/NACK handshake for reliability
- Retransmission (up to 3 attempts)
- Optional FEC (Forward Error Correction) with Viterbi decoder
- Optional Chase Combining for soft-bit accumulation
- Sequence tracking via shared SessionState (prevents “Handoff Problem”)

Critical Fixes Implemented:

- Fix #1: Shared SessionState prevents sequence resets during transport switching
- Fix #2: Smart drain logic via FailureType (ACK timeout vs hard IO error)
- Fix #5: sendMutex ensures atomic sequence+write (no out-of-order frames)

STAR Project - Texas A&M University February 2026

8.0.1 VARIABLES

8.0.2 var (

```
ErrSPITransportNotInitialized = errors.New("SPI transport not initialized") ErrMaxRetriesExceeded = errors.New("maximum retries exceeded") ErrACKTimeout = errors.New("ACK timeout") )
```

```
    Predefined errors for SPI link operations.
```

8.0.3 var (

```
ErrCDCTransportNotInitialized = errors.New("CDC transport not initialized") )
```

```
    Predefined errors.
```

8.0.4 TYPES

8.0.5 type CDCLink struct {
}

CDCLink implements a lightweight link layer for USB CDC transport.
Unlike [SPILink](#) (full HARQ), CDCLink relies on USB hardware reliability.
Sequences are managed by shared [SessionState](#) (prevents Handoff Problem).

8.0.6 func NewCDCLink(t transport.Device, session *manager.SessionState) (*CDCLink, error)

NewCDCLink creates a new CDC link layer with shared session state. CRITICAL: session parameter MUST be shared across all [transports](#) (USB and SPI) to prevent sequence desynchronization during transport switching.

8.0.7 func (c *CDCLink) GetRxSequence() uint16

GetRxSequence implements harq.HARQ [interface](#). Delegates to shared SessionState.

8.0.8 func (c *CDCLink) GetState() harq.State

GetState implements harq.HARQ [interface](#). CDCLink has no retransmission state machine, always returns Idle or Error.

8.0.9 func (c *CDCLink) GetTxSequence() uint16

GetTxSequence implements harq.HARQ [interface](#). Delegates to shared SessionState.

8.0.10 func (c *CDCLink) Receive(ctx context.Context) (*harq.ReceiveResult, error)

Receive implements harq.HARQ [interface](#). Validates [CRC32](#) (decoder does this) and sequence using shared SessionState. CRITICAL FIX #6: Gap tolerance accepts sequence gaps <10 frames.

8.0.11 func (c *CDCLink) Reset()

Reset implements harq.HARQ [interface](#). Delegates to shared SessionState (resets both TX and RX sequences to 0). CRITICAL: This is called after receiving RESET_ACK from RX72N to synchronize sequences after Gateway restart (prevents restart deadlock - FIX #4).

8.0.12 func (c *CDCLink) Send(ctx context.Context, data []byte, p ...harq.Priority) error

Send implements harq.HARQ [interface](#). Uses shared SessionState for sequence management (CRITICAL FIX #1). Serializes Send operations with sendMutex (CRITICAL FIX #5).

8.0.13 func (c *CDCLink) SendWithSeq(ctx context.Context, data []byte, seq uint16, flags frame.Flags) error

SendWithSeq sends data with the specified sequence number and flags. CRITICAL FIX #5: sendMutex ensures atomic sequence+write (no out-of-order frames). CRITICAL FIX #7: Context timeout prevents blocked writes on USB disconnect.

8.0.14 func (c *CDCLink) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

SendWithType sends data with a specific frame type (PING, PONG, RESET, etc.). This allows explicit control over the frame type for protocol messages. Phase 3: Frame Type API implementation.

8.0.15 type SPILink struct {
}

SPILink implements a full HARQ link layer for SPI transport.

Pattern: Based on CDCLink structure but with HARQ additions:

- ACK/NACK handshake
- Retransmission logic
- FEC encoding/decoding (optional)
- Chase Combining for soft-bit accumulation (optional)

8.0.16 func NewSPILink(t transport.Device, session *manager.SessionState, config *SPILinkConfig) (*SPILink, error)

NewSPILink creates a new SPI link layer with shared session state.

CRITICAL: session parameter MUST be shared across all transports (USB and SPI) to prevent sequence desynchronization during transport switching.

Parameters:

- t: SPI transport device
- session: Shared SessionState instance (from TransportManager.GetSessionState())
- config: Configuration (nil = use defaults)

Returns:

- *SPILink: Initialized SPI link
- error: Initialization error (nil/invalid parameters)

8.0.17 func (s *SPILink) GetRxSequence() uint16

GetRxSequence implements harq.HARQ interface. Delegates to shared SessionState.

8.0.18 func (s *SPILink) GetState() harq.State

GetState implements harq.HARQ interface. Returns current HARQ state machine state.

8.0.19 func (s *SPILink) GetTxSequence() uint16

GetTxSequence implements harq.HARQ interface. Delegates to shared SessionState.

8.0.20 func (s *SPILink) Receive(ctx context.Context) (*harq.ReceiveResult, error)

Receive implements harq.HARQ interface with ACK/NACK response generation.

Protocol Flow:

1. Decode frame via StreamDecoder (validates CRC32 automatically)
2. CRITICAL FIX: Check if ACK/NACK frame - dispatch to Send() if so
3. Validate sequence using shared SessionState
4. CRITICAL FIX: On duplicate - send ACK (not NACK) to stop retransmit loop
5. Optional: FEC decode payload (if FlagFECEnabled)
6. On success: Send ACK if required, return payload
7. CRITICAL FIX: Don't fail if ACK send fails (just log)
8. On failure: Send NACK, return error

Thread Safety: receiveMutex prevents concurrent calls that would race on s.decoder. The decoder maintains internal buffer state and is NOT safe for concurrent access.

8.0.21 func (s *SPILink) Reset()

Reset implements harq.HARQ **interface**. Delegates to shared SessionState (resets both TX and RX sequences to **0**). Also resets FEC combiner **if** enabled.

8.0.22 func (s *SPILink) Send(ctx context.Context, data []byte, p ...harq.Priority) error

Send implements harq.HARQ **interface** with ACK/NACK handshake and retransmission. Uses frame.FrameTypeCommand **for** all data frames.

Thread Safety: sendMutex ensures **atomicity** (sequence assignment + Send)

8.0.23 func (s *SPILink) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

SendWithType sends data with explicit frame **type** (**PING**, **PONG**, **RESET**, etc.). This allows explicit control over the frame **type for** protocol messages. Phase **3**: Frame Type API implementation.

Thread Safety: sendMutex ensures **atomicity** (sequence assignment + Send)

8.0.24 type SPILinkConfig struct {

EnableFEC bool

MaxRetries int

ACKTimeout time.Duration }

SPILinkConfig holds configuration parameters **for** SPILink.

8.0.25 func DefaultSPILinkConfig() *SPILinkConfig

DefaultSPILinkConfig returns the **default** configuration **for** SPILink.

9 Package: internal.manager

Package manager ...

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

Package manager provides intelligent transport selection and failover for the STAR Gateway.

STAR Project - Texas A&M University January 2026

9.0.1 CONSTANTS

9.0.2 const (

SPIProbeTimeout = 50 * time.Millisecond

SPIProbePayloadSize = 4

SPIProbeTestMarker = 0xDEADBEEF)

9.0.3 const (

DefaultPingInterval = 1 * time.Second

DefaultFailureTimeout = 200 * time.Millisecond)

9.0.4 const (

DefaultConsecutiveMissThreshold = 1)

WDT Timing Coordination

The RX72N hardware Watchdog **Timer** (WDT) has an approximate timeout of **1** second. Our software heartbeat **MUST** detect failure significantly faster to prevent unintended hardware reboots.

Timing **budget** (dual detection strategy):

WDT timeout:	~1000ms	(hardware, configured in RX72N firmware)
Implicit timeout:	200ms	(any frame resets timer via OnFrameReceived)
Check interval:	50ms	(failureTimeout / 4, for timely detection)
PING interval:	1000ms	(1 Hz, idle-link probe -- rare when data is flowing)
Miss threshold:	1 PING	(single unanswered PING triggers failover)
Worst-case detect:	~250ms	(implicit timeout + one check interval)
Safety margin:	~4x	(250ms << 1000ms WDT)

How it works:

Active link:	Data frames (including ACK/NACK) keep lastSeen fresh. PING fires every pingInterval since last valid PONG (independent timer). Implicit timeout never fires because lastSeen stays current.
Zombie link:	Stale OS-buffered frames keep lastSeen fresh, but firmware is dead. PING fires at pingInterval since last PONG; no PONG arrives;

counter-based failover triggers. Implicit timeout cannot catch this because lastSeen is continuously refreshed by the buffered frames.

Dead link: No frames of any kind; implicit timeout fires at 200ms (faster than PING at 1s, so implicit path is the fast-path for hard failures).

WARNING: Do NOT increase DefaultFailureTimeout above 500ms without first verifying the RX72N WDT timeout. The implicit timeout is the critical path.

To verify RX72N WDT timeout:

1. Check star-rx72n-firmware WDT configuration registers (WDTCR)
2. WDT clock source and divider determine actual timeout
3. Typical: PCLKB/8192 with 16-bit counter ~ 1.09s at 60MHz PCLKB

9.0.5 const (
DefaultTargetDevice = “ttyACM0”)

9.0.6 const (
TransportNameUSB = “usb”
TransportNameSPI = “spi”)

9.0.7 const (
PriorityUSB = 10
PrioritySPI = 5)

9.0.8 const (
InvalidTransportModeError = “invalid transport mode: %q”)

9.0.9 const MaxGapTolerance uint16 = 10
MaxGapTolerance is the maximum number of skipped frames allowed before rejecting a received sequence as a large gap (transport failure). It controls how many frames may be lost (for example on lightweight USB) while still accepting and catching up the RX sequence.

9.0.10 const SessionIDPayloadSize = 4
SessionIDPayloadSize is the byte size of the session ID in RESET/RESET_ACK payloads. The session ID is encoded as a 4-byte little-endian uint32.

9.0.11 TYPES

9.0.12 type Config struct {
Mode TransportMode
HealthCheckInterval time.Duration
FailureThreshold int
SwitchTimeout time.Duration
EnableHotPlug bool
HotPlugPollInterval time.Duration
USBVID uint16
USBPID uint16
FailbackDamping time.Duration

HealthLatencyThreshold time.Duration

HealthLossThreshold float64 }

Config holds the configuration **for** the TransportManager.

9.0.13 func DefaultConfig() *Config

DefaultConfig returns a Config with production-ready **default** settings.

Default behavior:

- Auto mode: prefer USB, fallback to SPI
- Health checks every **5** seconds
- Failover after **3** consecutive failures
- **500ms** **switch** timeout
- Hot-plug detection enabled

Note: USBPID should be updated with the actual RX72N Product ID once known.

9.0.14 func (c *Config) Validate() error

Validate checks **if** the configuration is valid.

9.0.15 type ConfigError struct {

Field string Reason string }

ConfigError represents a configuration validation error.

9.0.16 func (e *ConfigError) Error() string

9.0.17 type FailureType int

FailureType indicates why a transport **switch** was triggered. This determines whether the TransportManager should drain in-flight operations before **switching** (CRITICAL FIX #2: Smart Drain Logic).

9.0.18 const (

FailureTypeGraceful FailureType = iota

FailureTypeTimeout

FailureTypeIOError

FailureTypeHealthCheck

FailureTypeHotPlugRemove)

9.0.19 func (ft FailureType) String() string

String returns a human-readable failure **type name**.

9.0.20 type HARQReader struct {

}

HARQReader adapts a harq.HARQ **interface** to io.Reader. It buffers payloads received from HARQ and provides them as a byte stream.

Architecture Note: This adapter exists because harq.HARQ.Receive() returns complete HARQ **packets** (which may contain multiple frames or partial frames), while StreamDecoder expects a continuous byte stream. The adapter bridges this impedance mismatch by:

1. Calling HARQ.Receive() to get packet **payloads** (blocking up to readTimeout)

2. Buffering the payload bytes
3. Exposing them as an `io.Reader` for `StreamDecoder` to parse frame boundaries

Thread Safety: `Read()` is safe for concurrent calls but only one will make progress at a time due to internal mutex. This matches `io.Reader` contract.

9.0.21 func NewHARQReader(h harq.HARQ) *HARQReader

`NewHARQReader` creates a new adapter.

9.0.22 func (r *HARQReader) Read(p []byte) (int, error)

`Read` implements `io.Reader`.

9.0.23 type HealthMetrics struct {

`LastSuccess` `time.Time`

`LastFailure` `time.Time`

`LastRecovery` `time.Time`

`ConsecutiveFailures` `int`

`TotalSent` `uint64`

`TotalReceived` `uint64`

`TotalErrors` `uint64`

`AvgLatency` `time.Duration`

`PacketLossRate` `float64`

`IsHealthy` `bool`

}

`HealthMetrics` tracks the operational health of a transport.

9.0.24 func NewHealthMetrics() *HealthMetrics

`NewHealthMetrics` creates a new `HealthMetrics` instance with `default` values.

9.0.25 type HealthMonitor struct {

}

`HealthMonitor` periodically checks the health of inactive transports.

It runs in a background goroutine and probes transports that are not currently active. This allows the `TransportManager` to detect when a previously failed transport has recovered.

9.0.26 func NewHealthMonitor(interval time.Duration) *HealthMonitor

`NewHealthMonitor` creates a new `HealthMonitor` with the given check interval.

9.0.27 func (hm *HealthMonitor) Run(ctx context.Context, tm *TransportManager)

`Run` starts the health monitoring loop. It exits when `ctx` is cancelled.

9.0.28 type HeartbeatManager struct {

}

`HeartbeatManager` implements hybrid implicit/explicit connectivity detection.

Hybrid Detection Strategy:

- Implicit: Updates LastSeen on ANY valid frame (telemetry, commands, ACKs)
- Explicit: Sends PING when idle >1s (rare idle-link probe)
- Failure: Declares link dead if no frames >200ms (implicit timeout fires first)

Benefits:

- Minimal bandwidth overhead (only PING when idle)
- Fast failure detection (200ms vs 5s polling)
- No false positives from buffered OS data (counter prevents replay)

Thread Safety: All methods use mutex protection for concurrent access.

9.0.29 func NewHeartbeatManager(pingInterval, failureTimeout time.Duration, onFailure func()) (*HeartbeatManager, error)

NewHeartbeatManager creates a new heartbeat manager with the specified timeouts.

Parameters:

- pingInterval: How long the link must be idle before sending explicit PING (recommended 1s)
- failureTimeout: How long without any frames before declaring failure (recommended 200ms)
- onFailure: Callback function to invoke when link is declared dead

Returns an error if:

- pingInterval or failureTimeout is <= 0
- onFailure is nil (required for triggering failover)

Note: pingInterval may be greater than failureTimeout. In the dual-detection model, the implicit timeout (failureTimeout) is the primary detection mechanism -- any valid frame resets the timer. PING is a rare idle-link probe that fires only when the link has been silent for pingInterval.

9.0.30 func (hm *HeartbeatManager) OnFrameReceived()

OnFrameReceived updates the LastSeen timestamp for implicit heartbeat detection.

This method should be called by TransportManager.Receive() for valid non-PONG frames, including:

- PING frames (from remote side)
- Command frames
- Response frames

PONG frames should use [OnPongReceived] for explicit counter validation.

By updating on any frame, we avoid sending unnecessary PINGs when the link is actively transferring data. Also clears the failureTriggered flag to allow future failover if the link fails again.

Thread Safety: Uses mutex to protect lastSeen timestamp and failureTriggered flag.

9.0.31 func (hm *HeartbeatManager) OnPongReceived(payload []byte) bool

OnPongReceived validates a PONG payload against the last sent PING counter.

This performs both implicit and explicit heartbeat detection:

- Implicit: Always updates lastSeen and clears `failureTriggered` (link is alive)
- Explicit: Validates the 4-byte counter matches lastPingSent

On counter match:

- Resets consecutiveMisses to 0
- Updates lastValidPong timestamp
- Clears pendingPing flag

On counter mismatch (stale/replayed PONG):

- Logs a warning with both counters
- Does NOT reset `consecutiveMisses` (link may be degraded)
- lastSeen is still `updated` (link is physically alive)

Returns `true` if the counter matched, `false` otherwise.

Thread Safety: Uses mutex to protect all state fields.

9.0.32 func (hm *HeartbeatManager) Run(ctx context.Context, tm *TransportManager)

Run starts the heartbeat monitoring loop as a background goroutine.

This method:

1. Periodically checks elapsed time since `lastSeen` (every `failureTimeout/4`)
2. Triggers failover if `idle > failureTimeout` (200ms)
3. Sends PING if `idle > pingInterval` (1s) and link not yet failed

The check interval is derived from `failureTimeout` to ensure timely detection. With 200ms timeout and check every 50ms, worst-case detection is ~250ms.

The loop runs until ctx is cancelled. It should be started as a goroutine:

```
go heartbeat.Run(ctx, transportManager)
```

Thread Safety: Safe to call concurrently with `OnFrameReceived()`.

9.0.33 type HotPlugDetector struct { }

HotPlugDetector monitors `for` USB device add/remove events.

On Linux, this uses inotify via fsnotify to watch `/sys/bus/usb/devices` `for` device creation/removal. This provides instant `detection` (<50ms) compared to polling-based approaches.

Phase 3: Full inotify implementation `for` instant hot-plug failover.

9.0.34 func NewHotPlugDetector(pollInterval time.Duration, vid, pid uint16) *HotPlugDetector

NewHotPlugDetector creates a new HotPlugDetector. The pollInterval parameter is ignored in the inotify implementation but kept `for` API compatibility.

9.0.35 func (hpd *HotPlugDetector) Run(ctx context.Context, eventHandler func(HotPlugEvent))

Run starts the hot-plug detection loop using inotify. It watches `/sys/bus/usb/devices` `for` Create/Remove events and calls eventHandler when

the target device is added or removed.

On non-Linux platforms or **if** `fsnotify` fails, it falls back to a no-op (graceful degradation).

9.0.36 type `HotPlugEvent` struct {

Action string

Device string

Timestamp `time.Time`

VendorID `uint16`

ProductID `uint16` }

`HotPlugEvent` represents a USB device add/remove event.

9.0.37 type `SessionState` struct {

}

`SessionState` holds sequence numbers shared across all transports. When Gateway fails over from `USB` (`seq=105`) to `SPI`, the `SPI` link MUST **continue** from `seq=106`, not reset to `0`. `SessionState` ensures all transports share the same sequence counter.

9.0.38 func `NewSessionState()` `*SessionState`

`NewSessionState` creates a new `SessionState` with initial sequence `0`.

9.0.39 func (s `*SessionState`) `ActivateBarrier(sessionID uint32)`

`ActivateBarrier` enables the reset barrier **for** the given session ID.

While the barrier is active, `ValidateRxSequence` rejects all **frames** (they are stale), and only a `RESET_ACK` with a matching session ID should be accepted.

Must be called BEFORE sending the `RESET` frame to ensure no stale frames from the USB FIFO buffer are processed during the handshake.

Thread Safety: Uses mutex to ensure atomicity.

9.0.40 func (s `*SessionState`) `DeactivateBarrier()`

`DeactivateBarrier` disables the reset barrier. Called after the reset handshake **completes** (success or failure) to resume normal frame processing. Should be called in a **defer** to guarantee cleanup.

Thread Safety: Uses mutex to ensure atomicity.

9.0.41 func (s `*SessionState`) `GetRxSequence()` `uint16`

`GetRxSequence` returns the current RX **sequence** (**for** diagnostics).

Thread Safety: Uses mutex **for** consistent reads.

9.0.42 func (s `*SessionState`) `GetSessionID()` `uint32`

`GetSessionID` returns the current session **ID** (**for** diagnostics).

Thread Safety: Uses mutex **for** consistent reads.

9.0.43 func (s *SessionState) GetTxSequence() uint16

GetTxSequence returns the current TX sequence (for diagnostics).

Thread Safety: Uses mutex for consistent reads.

9.0.44 func (s *SessionState) IncrementSessionID() uint32

IncrementSessionID atomically increments and returns the new session ID. Called at the start of each reset handshake to generate a unique identifier for matching RESET frames with their corresponding RESET_ACK responses.

Thread Safety: Uses mutex to ensure atomicity.

9.0.45 func (s *SessionState) IsBarrierActive() bool

IsBarrierActive returns whether the reset barrier is currently active.

Thread Safety: Uses mutex for consistent reads.

9.0.46 func (s *SessionState) NextTxSequence() uint16

NextTxSequence atomically increments and returns the next TX sequence.

This method is called by all transport link layers (CDCLink, SPILink) to generate the next sequence number for outgoing frames. The shared counter ensures continuity across transport switches.

Thread Safety: Uses mutex to ensure atomicity across concurrent goroutines.

9.0.47 func (s *SessionState) Reset()

Reset resets both sequences to 0 (used when RX72N resets).

CRITICAL FIX #4: This is called after receiving a RESET_ACK from RX72N to synchronize sequences after Gateway restart.

Thread Safety: Uses mutex to ensure atomicity.

9.0.48 func (s *SessionState) ValidateResetAck(payload []byte) bool

ValidateResetAck checks if a RESET_ACK payload matches the active barrier's session ID. Returns true only if:

- The barrier is currently active
- The payload is exactly SessionIDPayloadSize (4) bytes
- The little-endian decoded session ID matches the barrier's expected session ID

Thread Safety: Uses mutex to ensure atomicity.

9.0.49 func (s *SessionState) ValidateRxSequence(seq uint16) bool

ValidateRxSequence checks if the received sequence is valid. Returns true if valid (exact match or small gap), false if duplicate or large gap.

CRITICAL FIX #6: Allow small gaps for lightweight USB protocol (no retransmission). If USB drops a single frame due to rare bit flip, we accept the gap and continue rather than permanently stalling the link.

Gap Tolerance:

- diff == 0: Exact match (most common case) -> Accept
- 0 < diff < 10: Small gap (packet loss) -> Accept and catch up
- diff >= 10: Large gap (transport failure) -> Reject
- diff is large positive (near 65535): Likely duplicate from wraparound ->

Reject

Thread Safety: Uses mutex to ensure atomicity across concurrent goroutines.

9.0.50 type State uint8

State represents the internal state of the TransportManager.

9.0.51 const (

StateInitializing State = iota

StateActiveUSB

StateActiveSPI

StateSwitchingToUSB

StateSwitchingToSPI

StateDegraded

StateFailed)

9.0.52 func (s State) String() string

String returns a human-readable state name.

9.0.53 type TransportManager struct {

}

TransportManager manages multiple transports with automatic failover and health monitoring.

It implements the `harq.HARQ interface` and acts as a transparent proxy to the active transport. The Dispatcher interacts with TransportManager exactly as it would with a single HARQ instance, while TransportManager handles all transport selection, health monitoring, and failover logic.

Key features:

- Priority-based transport `selection` (USB=PriorityUSB, SPI=PrioritySPI)
- Health monitoring with configurable failure thresholds
- Automatic failover when active transport fails
- USB hot-plug `detection` (add/remove events)
- Zero-downtime transport switching with `pause-drain-resume`
- Thread-safe operations with fine-grained locking

Thread Safety:

- All public methods are thread-safe
- Uses `RWMutex` `for` high read concurrency
- Operations lock prevents race conditions during switching

9.0.54 func NewTransportManager(config *Config) *TransportManager

NewTransportManager creates a new TransportManager with the given configuration.

If config is `nil`, `DefaultConfig()` is used. The manager is created in `StateInitializing` and must be started with `Start()`.

Example:

```
tm := manager.NewTransportManager(manager.DefaultConfig())
```

```
tm.RegisterTransport("spi", spiLink, manager.PrioritySPI)
tm.RegisterTransport("usb", usbLink, manager.PriorityUSB)
tm.Start(ctx)
```

9.0.55 func (tm *TransportManager) ForceSwitch(transportName string) error

ForceSwitch manually switches to a specific transport, bypassing priority logic.

This method is useful for testing or manual override scenarios.

Returns an error if:

- The specified transport is not registered
- The specified transport is not available
- The switch operation fails

9.0.56 func (tm *TransportManager) GetActiveTransport() string

GetActiveTransport returns the name of the currently active transport. Returns empty string if no transport is active.

9.0.57 func (tm *TransportManager) GetAvailableTransports() []string

GetAvailableTransports returns a list of currently available transport names. Only includes transports marked as Available.

9.0.58 func (tm *TransportManager) GetHealthMetrics() map[string]*HealthMetrics

GetHealthMetrics returns a copy of health metrics for all registered transports.

9.0.59 func (tm *TransportManager) GetInternalState() State

GetInternalState returns the current internal State (for debugging/testing).

9.0.60 func (tm *TransportManager) GetRxSequence() uint16

GetRxSequence returns the RX sequence number from the active transport. Returns 0 if no transport is active.

9.0.61 func (tm *TransportManager) GetSessionState() *SessionState

GetSessionState returns the shared SessionState instance. CRITICAL FIX #1: All transports (USB CDC and SPI) MUST use the same SessionState to prevent sequence desynchronization during transport switching (Handoff Problem).

Usage in gateway.go:

```
tm := manager.NewTransportManager(config)
session := tm.GetSessionState()
cdcLink, _ := link.NewCDCLink(cdcTransport, session)
spiLink, _ := link.NewSPILink(spiTransport, session)
tm.RegisterTransport("usb", cdcLink, manager.PriorityUSB)
tm.RegisterTransport("spi", spiLink, manager.PrioritySPI)
```

9.0.62 func (tm *TransportManager) GetState() harq.State

GetState returns the current harq.State mapped from the internal State.

Mapping:

- StateActiveUSB, StateActiveSPI -> harq.StateIdle
- StateSwitching* -> harq.StateRetransmitting
- StateFailed -> harq.StateError

9.0.63 func (tm *TransportManager) GetTxSequence() uint16

GetTxSequence returns the TX sequence number from the active transport.
Returns 0 if no transport is active.

9.0.64 func (tm *TransportManager) Receive(ctx context.Context) (*harq.ReceiveResult, error)

Receive proxies the Receive operation to the active transport with health tracking.

This method implements the harq.HARQ interface. It blocks if a transport switch is in progress, then resumes once switching completes.

Control frames (PING, PONG, ACK, NACK, RESET_ACK) are consumed internally:

- PING: auto-replies with PONG (echoing payload), updates implicit heartbeat
- PONG: validates counter against last PING (explicit heartbeat)
- ACK/NACK/RESET_ACK: logged and consumed

Only data frames (COMMAND, RESPONSE) are returned to the caller.

9.0.65 func (tm *TransportManager) RegisterTransport(name string, transport harq.HARQ, priority int)

RegisterTransport adds a transport to the manager with the given priority.

Higher priority transports are preferred (e.g., USB=10, SPI=5).
If this transport has higher priority than the current active transport, an automatic switch will be attempted in the background.

This method is thread-safe and can be called while the manager is running.

Parameters:

- name: Transport identifier (e.g., "spi", "usb")
- transport: HARQ implementation
- priority: Selection priority (higher = preferred)

9.0.66 func (tm *TransportManager) Reset()

Reset resets the active transport to a clean state. This is a pass-through to the active transport's Reset method.

9.0.67 func (tm *TransportManager) Send(ctx context.Context, data []byte, p ... harq.Priority) error

Send proxies the Send operation to the active transport with health tracking.

This method implements the harq.HARQ interface. It blocks if a transport switch is in progress, then resumes once switching completes.

Returns an error if:

- No active transport is available (StateFailed)
- The active transport's Send operation fails

Failed operations increment the failure counter. When the failure threshold is reached, an automatic failover is triggered in the background.

9.0.68 func (tm *TransportManager) SendWithType(ctx context.Context, data []byte, frameType frame.Type) error

SendWithType sends data with an explicit frame **type** (PING, PONG, RESET, etc.). This is a Phase 3 API **for** protocol messages that require specific frame types.

If the active transport supports SendWithType(), it uses that method. Otherwise, it falls back to Send() (CDCLink and SPILink both support SendWithType).

This method blocks **if** a transport **switch** is in progress, then resumes once switching completes.

9.0.69 func (tm *TransportManager) Start(ctx context.Context) error
Start initializes and starts the TransportManager background routines.

This method:

1. Starts the health monitor **for** inactive transports
2. Starts the hot-plug **detector** (**if** enabled)
3. Selects the initial active transport

The provided context is used **for** graceful shutdown. Call Stop() to clean up.

Returns an error **if**:

- No transports are available and mode is ModeForceUSB or ModeForceSPI
- Config validation fails

9.0.70 func (tm *TransportManager) Stop() error

Stop gracefully shuts down the TransportManager.

This method:

1. Cancels all background routines
2. Waits **for** all goroutines to exit
3. Resets all registered transports

It is safe to call Stop multiple times.

9.0.71 type TransportMode string

TransportMode defines the transport selection strategy.

9.0.72 const (

ModeAuto TransportMode = "auto"

ModeForceUSB TransportMode = "force-usb"

ModeForceSPI TransportMode = "force-spi")

9.0.73 func ParseTransportMode(s string) (TransportMode, error)

ParseTransportMode converts a string to a TransportMode. Returns ModeAuto and an error **if** the input is invalid.

9.0.74 func (tm TransportMode) IsValid() bool

IsValid checks **if** the TransportMode is one of the defined constants.

9.0.75 type TransportWrapper struct {

Name string

Transport harq.HARQ

Decoder *frame.StreamDecoder

Health *HealthMetrics

Priority int

Available bool }

TransportWrapper wraps a transport with health metrics and metadata.

10 Package: internal.server

Package server provides HTTP and gRPC server construction and lifecycle management.

This package follows a two-phase construction pattern that separates server configuration from server startup:

1. Constructor phase: `NewHTTPServer()` / `NewGRPCServer()` create configured servers without side effects (no network I/O, no goroutines).
2. Run phase: `RunHTTPServer()` / `RunGRPCServer()` start servers in background goroutines with automatic graceful shutdown via context cancellation.

Two-Phase Construction Benefits

This pattern enables:

- Testable handler/service wiring without opening network ports
- Inspecting server configuration before startup
- Separating config errors (constructor) from runtime errors (Run)
- Graceful shutdown via context cancellation (no manual shutdown calls)

HTTP Server Example

```
// Phase 1: Construct server with handler
mux := http.NewServeMux()
mux.HandleFunc("/health", healthHandler)

const defaultHTTPPort = ":8080"

config := &server.HTTPConfig{
    ListenAddr: defaultHTTPPort,
    ReadTimeout: DefaultReadTimeout,
    WriteTimeout: DefaultWriteTimeout,
    Handler: mux,
}

httpSrv, err := server.NewHTTPServer(config, logger)
if err != nil {
    return fmt.Errorf("failed to create HTTP server: %w", err)
}

// Phase 2: Start server (non-blocking)
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

errChan, err := server.RunHTTPServer(ctx, httpSrv, logger)
if err != nil {
    return fmt.Errorf("failed to start HTTP server: %w", err)
}

// Monitor for runtime errors
go func() {
    if err := <-errChan; err != nil {
        logger.Error("HTTP server error", slog.String("error", err.Error()))
    }
}()

// Graceful shutdown: cancel context
```

```

    // (RunHTTPServer automatically shuts down when ctx is cancelled)
    cancel()

# gRPC Server Example

// Phase 1: Construct server with service registration
const bytesPerKiB = 1024
const bytesPerMiB = bytesPerKiB * 1024
const defaultMaxMessageMultiplier = 10
const defaultMaxMessageSize = defaultMaxMessageMultiplier * bytesPerMiB

config := &server.GRPCConfig{
    ListenAddr:    DefaultGRPCListenAddr,
    MaxMessageSize: defaultMaxMessageSize,
    ServiceRegistrar: func(srv *grpc.Server) error {
        starv1.RegisterMotorControlServiceServer(srv, motorService)
        starv1.RegisterTelemetryServiceServer(srv, telemetryService)
        return nil
    },
}

grpcSrv, err := server.NewGRPCServer(config, logger)
if err != nil {
    return fmt.Errorf("failed to create gRPC server: %w", err)
}

// Phase 2: Start server (non-blocking)
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

errChan, err := server.RunGRPCServer(ctx, grpcSrv, logger)
if err != nil {
    return fmt.Errorf("failed to start gRPC server: %w", err)
}

// Monitor for runtime errors
go func() {
    if err := <-errChan; err != nil {
        logger.Error("gRPC server error", slog.String("error", err.Error()))
    }
}()

// Graceful shutdown: cancel context
// (RunGRPCServer automatically shuts down with 5s timeout fallback)
cancel()

```

Testing Without Network I/O

The two-phase pattern enables testing handler wiring without opening ports:

```

// Test HTTP handler
config := &server.HTTPConfig{
    ListenAddr: ":8080",
    Handler:    myHandler,
}
httpSrv, _ := server.NewHTTPServer(config, testLogger)

req := httptest.NewRequest("GET", "/test", nil)
recorder := httptest.NewRecorder()

```

```
httpSrv.Handler.ServeHTTP(recorder, req)

// Assert on recorder.Code, recorder.Body

# Graceful Shutdown
```

Both RunHTTPServer and RunGRPCServer automatically handle graceful shutdown:

- HTTP: Calls Server.Shutdown() with DefaultShutdownTimeout
- gRPC: Calls GracefulStop() with DefaultGracefulStopTimeout, falls back to `Stop()`
- Shutdown is triggered automatically when the context passed to Run is cancelled
- No manual shutdown calls required

STAR Project - Texas A&M University February 2026

10.0.1 CONSTANTS

10.0.2 const (

DefaultListenAddr = ":8080" DefaultReadTimeout = 10 * time.Second DefaultWriteTimeout = 10 * time.Second DefaultGRPCListenAddr = ":50051" DefaultGRPCMaxMessageSize = 10 * 1024 * 1024)

10.0.3 const (

DefaultShutdownTimeout = 5 * time.Second)

10.0.4 const DefaultGracefulStopTimeout = 5 * time.Second

DefaultGracefulStopTimeout is the maximum time allowed for gRPC server graceful shutdown.

10.0.5 FUNCTIONS

10.0.6 func NewGRPCServer(config *GRPCConfig, logger *slog.Logger) (*grpc.Server, error)

NewGRPCServer creates a configured grpc.Server and registers services.

The server is configured with:

- MaxRecvMsgSize and MaxSendMsgSize from config.MaxMessageSize
- Services registered via config.ServiceRegistrar callback

Returns error if:

- Config validation fails
- ServiceRegistrar callback returns error

Example:

```
config := &GRPCConfig{
    ListenAddr:    ":50051",
    MaxMessageSize: 10 * 1024 * 1024,
    ServiceRegistrar: func(srv *grpc.Server) error {
        starv1.RegisterMotorControlServiceServer(srv, motorService)
        return nil
    },
}
```

```

    }
    grpcSrv, err := NewGRPCServer(config, logger)

```

10.0.7 func NewHTTPServer(config *HTTPConfig, logger *slog.Logger) (*http.Server, error)

NewHTTPServer creates a configured http.Server without starting it. This enables testing handler wiring without opening network ports.

The server is configured with:

- Address from config.ListenAddr
- Handler from config.Handler (typically http.ServeMux)
- Timeouts from config (ReadTimeout, WriteTimeout)
- Error logger that uses slog

Returns error if config validation fails.

10.0.8 func RunGRPCServer(ctx context.Context, srv *grpc.Server, addr string, logger *slog.Logger) (<-chan error, error)

RunGRPCServer starts the gRPC server in a background goroutine and returns immediately.

The function:

1. Creates a TCP listener on addr
2. Starts server in background goroutine via srv.Serve()
3. Spawns a shutdown watcher goroutine that triggers graceful shutdown on ctx.Done()

Returns:

- Error channel for async error reporting (buffered, size 1)
- Immediate error if listener creation fails

Graceful shutdown:

- Triggered automatically when ctx is cancelled
- Calls srv.GracefulStop() with grpcShutdownTimeout (5 seconds)
- Waits for in-flight RPCs to complete
- Falls back to srv.Stop() on timeout (forcefully terminates active RPCs)

The error channel receives:

- nil on clean shutdown
- error if srv.Serve() fails

Example:

```

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

errChan, err := RunGRPCServer(ctx, grpcSrv, ":50051", logger)
if err != nil {
    return fmt.Errorf("failed to start: %w", err)
}

// Monitor for runtime errors
go func() {
    if err := <-errChan; err != nil {
        logger.Error("server error", slog.String("error", err.Error()))
    }
}()

```

```
// Trigger graceful shutdown
cancel()
```

10.0.9 func RunHTTPServer(ctx context.Context, srv *http.Server, logger *slog.Logger) (<chan error, error)

RunHTTPServer starts the HTTP server in a background goroutine and returns immediately.

The function:

1. Creates a TCP listener on `srv.Addr`
2. Starts server in background goroutine via `srv.Serve()`
3. Spawns a shutdown watcher goroutine that triggers graceful shutdown on `ctx.Done()`

Returns:

- Error channel for async error reporting (buffered, size 1)
- Immediate error if listener creation fails

Graceful shutdown:

- Triggered automatically when `ctx` is cancelled
- Calls `srv.Shutdown()` with `httpShutdownTimeout` (5 seconds)
- Allows in-flight requests to complete
- After timeout, forcefully closes connections

The error channel receives:

- `nil` on clean shutdown
- error if `srv.Serve()` fails (excluding `http.ErrServerClosed`)

Example:

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

errChan, err := RunHTTPServer(ctx, httpSrv, logger)
if err != nil {
    return fmt.Errorf("failed to start: %w", err)
}

// Monitor for runtime errors
go func() {
    if err := <-errChan; err != nil {
        logger.Error("server error", slog.String("error", err.Error()))
    }
}()

// Trigger graceful shutdown
cancel()
```

10.0.10 TYPES

10.0.11 type GRPCConfig struct {

ListenAddr string

MaxMessageSize int

ServiceRegistrar func(*grpc.Server) error }

GRPCConfig holds the configuration **for** gRPC server construction.

10.0.12 func DefaultGRPCConfig() *GRPCConfig

DefaultGRPCConfig returns a GRPCConfig with production-ready defaults.
ServiceRegistrar must be set by the caller as it's application-specific.

10.0.13 func (c *GRPCConfig) Validate() error

Validate checks **if** the GRPCConfig is valid and returns an error **if** validation fails.

10.0.14 type HTTPConfig struct {

ListenAddr string

ReadTimeout time.Duration

WriteTimeout time.Duration

Handler http.Handler }

HTTPConfig holds the configuration **for** HTTP server construction.

10.0.15 func DefaultHTTPConfig() *HTTPConfig

DefaultHTTPConfig returns an HTTPConfig with production-ready defaults.
Handler must be set by the caller as it's application-specific.

10.0.16 func (c *HTTPConfig) Validate() error

Validate checks **if** the HTTPConfig is valid and returns an error **if** validation fails. This matches the existing config validation **pattern** (see `manager.Config.Validate`).

11 Package: internal.service

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University January 2026

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University December 2025

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University January 2026

Package service implements the gRPC service handlers for the star-gateway.

STAR Project - Texas A&M University January 2026

11.0.1 CONSTANTS

11.0.2 const (

MaxMotors = 4

MinMotorID = 0

MaxMotorID = 3)

11.0.3 const (

MinRateHz = 1 MaxRateHz = 100 DefaultRateHz = 10)

11.0.4 TYPES

11.0.5 type ConfigurationService struct {

starv1.UnimplementedConfigurationServiceServer

}

ConfigurationService implements the ConfigurationServiceServer **interface**.
This service handles runtime configuration management including NVS
persistence.

Architecture: - Uses WireMessage wrapper **for** protocol multiplexing - Uses
Dispatcher **for** centralized message routing - Validates all configuration
parameters before applying - TODO: Integrate HARQ **for** actual firmware
communication

11.0.6 func NewConfigurationService(h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *ConfigurationService

NewConfigurationService creates a new ConfigurationService. Requires
HARQ handler **for** sending requests, Dispatcher **for** receiving responses,
and logger.

11.0.7 func (s *ConfigurationService) GetConfiguration(ctx context.Context, req *starv1.GetConfigurationRequest) (*starv1.GetConfigurationResponse, error)

GetConfiguration retrieves the current system configuration from the RX72N
firmware. TODO: Implement proper request via HARQ once firmware integration
is complete. For now, returns mock configuration until firmware integration
is ready.

11.0.8 func (s *ConfigurationService) GetMotorPidConfig(ctx context.Context, req *starv1.GetMotorPidConfigRequest) (*starv1.GetMotorPidConfigResponse, error)
GetMotorPidConfig retrieves PID configuration for a specific motor.

11.0.9 func (s *ConfigurationService) ResetToDefaults(ctx context.Context, req *starv1.ResetToDefaultsRequest) (*starv1.ResetToDefaultsResponse, error)
ResetToDefaults resets configuration to factory defaults. If persist_to_nvs is true, saves defaults to NVS.

11.0.10 func (s *ConfigurationService) SaveConfiguration(ctx context.Context, req *starv1.SaveConfigurationRequest) (*starv1.SaveConfigurationResponse, error)
SaveConfiguration persists current in-memory configuration to NVS.

11.0.11 func (s *ConfigurationService) SetConfiguration(ctx context.Context, req *starv1.SetConfigurationRequest) (*starv1.SetConfigurationResponse, error)
SetConfiguration validates and applies new system configuration. Validates all parameters before applying. If persist_to_nvs is true, saves to NVS.

11.0.12 func (s *ConfigurationService) SetMotorPidConfig(ctx context.Context, req *starv1.SetMotorPidConfigRequest) (*starv1.SetMotorPidConfigResponse, error)
SetMotorPidConfig updates PID configuration for a specific motor. Validates PID parameters before applying. If persist_to_nvs is true, saves to NVS.

11.0.13 func (s *ConfigurationService) ValidateConfiguration(_ context.Context, req *starv1.ValidateConfigurationRequest) (*starv1.ValidateConfigurationResponse, error)
ValidateConfiguration validates configuration parameters without applying them. Returns detailed validation results including any errors found.

11.0.14 type FirmwareService struct {
starv1.UnimplementedFirmwareUpdateServiceServer }

FirmwareService implements the FirmwareUpdateServiceServer interface.
This service handles over-the-air (OTA) firmware updates with validation and rollback.

11.0.15 func NewFirmwareService() *FirmwareService
NewFirmwareService creates a new FirmwareService.

11.0.16 type GatewayService struct {
starv1.UnimplementedGatewayServiceServer
}

GatewayService implements the gRPC GatewayService for ROS2 <-> UI bridging.

Architecture:

ROS2 (C++) <-> gRPC <-> GatewayService (Go) <-> WebSocket <-> UI (TypeScript)

Data flows:

1. Telemetry (ROS2 -> UI): ROS2 calls ForwardTelemetry() -> cached -> WebSocket streams to UI
2. Teleop (UI -> ROS2): UI sends via WebSocket -> UpdateTeleopCommand() -> ROS2 polls GetTeleopCommand()

3. PID Gains (UI -> R0S2): UI sends via WebSocket -> SetPIDGains() -> R0S2
-> SPI -> RX72N

11.0.17 func NewGatewayService() *GatewayService

NewGatewayService creates a new GatewayService instance.

11.0.18 func (s *GatewayService) ForwardTelemetry(

ctx context.Context, req *starv1.ForwardTelemetryRequest,) (*starv1.ForwardTelemetryResponse, error)

ForwardTelemetry receives telemetry data from R0S2 and caches it for UI streaming.

Called by R0S2 bridge node at 10 Hz. This is a non-blocking operation that simply caches the telemetry data for WebSocket handlers to retrieve.

11.0.19 func (s *GatewayService) GetActiveClientCount() int32

GetActiveClientCount returns the number of connected WebSocket clients.

11.0.20 func (s *GatewayService) GetLatestTelemetry() (

*starv1.SystemStatus, *starv1.TelemetryData,)

GetLatestTelemetry returns the cached telemetry data for UI streaming.

Called by WebSocket handler to stream telemetry to UI clients. Returns nil if no telemetry has been received yet.

11.0.21 func (s *GatewayService) GetTelemetryAge() time.Duration

GetTelemetryAge returns how old the cached telemetry is.

Useful for UI to display data freshness indicator.

11.0.22 func (s *GatewayService) GetTeleopCommand(

ctx context.Context, req *starv1.GetTeleopCommandRequest,) (*starv1.GetTeleopCommandResponse, error)

GetTeleopCommand returns the latest teleop command for R0S2 to execute.

Called by R0S2 bridge node at 50 Hz. Returns cached command with age in ms. R0S2 is responsible for checking staleness (rejects commands > 500ms old).

11.0.23 func (s *GatewayService) Hub() HubNotifier

Hub returns the current HubNotifier under the read lock. Intended for use in tests that need to inspect the wired hub without accessing the unexported field directly, which would race with concurrent SetHub or ForwardTelemetry calls.

11.0.24 func (s *GatewayService) RegisterClient(clientID string)

RegisterClient increments the active WebSocket client count.

Called when a new WebSocket client connects.

11.0.25 func (s *GatewayService) SetHub(h HubNotifier)

SetHub wires the WebSocket hub into the service so ForwardTelemetry can push STAREnvelope messages directly to connected UI clients.

SetHub is safe to call concurrently with ForwardTelemetry and other hub

readers because it synchronises via `hubMu` (Lock/Rlock). It is intended to wire a `HubNotifier` and may also be used to replace `s.hub` at runtime.

11.0.26 func (s *GatewayService) SetPIDGains(
`ctx context.Context, req *starv1.SetPIDGainsRequest, ) (*starv1.SetPIDGainsResponse, error)`

`SetPIDGains` updates PID gains `for` motor velocity control.

Called by Gateway when UI requests PID tuning. This is a placeholder implementation that returns success but doesn't actually forward to R0S2 yet (will be implemented when R0S2 PID service is available).

11.0.27 func (s *GatewayService) UnregisterClient(clientID string)

`UnregisterClient` decrements the active WebSocket client count.

Called when a WebSocket client disconnects.

11.0.28 func (s *GatewayService) UpdateTeleopCommand(cmd *starv1.VelocityCommand)

`UpdateTeleopCommand` updates the cached teleop command from UI.

Called by WebSocket handler when UI sends a new controller input. This command will be polled by R0S2 via `GetTeleopCommand()`.

11.0.29 type HubNotifier interface {

`Broadcast(ctx context.Context, env *starv1.STAREnvelope) error }`

`HubNotifier` is the `interface` `GatewayService` uses to push `STAREnvelope` messages to the WebSocket hub. Defined [here](#) (not in `ws`) to avoid circular imports; `ws.Hub` satisfies it.

11.0.30 type MotorControlService struct {

`starv1.UnimplementedMotorControlServiceServer`

}

`MotorControlService` implements the `MotorControlServiceServer` `interface`. This service handles low-latency differential drive control.

Architecture: - Uses `WireMessage` wrapper `for` protocol multiplexing (solves ambiguity) - Uses `Dispatcher` `for` centralized message routing (solves concurrency race) - Uses structured `logging` (`slog`) `for` production observability

11.0.31 func NewMotorControlService(h harq.HARQ, d dispatcher.Dispatcher, logger

`*slog.Logger) *MotorControlService`

`NewMotorControlService` creates a new `MotorControlService`. Requires HARQ handler, `Dispatcher` `for` receiving telemetry, and logger.

11.0.32 func (s *MotorControlService) ControlStream(stream

`starv1.MotorControlService__ControlStreamServer) error`

`ControlStream` handles bidirectional streaming. Client streams velocity commands in, server streams encoder feedback out. Uses `Dispatcher` to receive telemetry and wraps commands in `WireMessage`.

11.0.33 func (s *MotorControlService) EmergencyStop(ctx context.Context, req *starv1.EmergencyStopRequest) (*starv1.EmergencyStopResponse, error)

EmergencyStop triggers an immediate stop using dedicated EmergencyStopCommand. This enables the RX72N to enter MOTOR_STATE_ESTOP and engage hardware safety features.

TODO (tracked in GitHub issue): Implement priority framing for E-Stop. The frame layer supports FlagPriority, but the HARQ interface doesn't expose it. EmergencyStop should use priority framing to ensure immediate processing by RX72N.

11.0.34 func (s *MotorControlService) SetMotorPower(ctx context.Context, req *starv1.SetMotorPowerRequest) (*starv1.SetMotorPowerResponse, error)

SetMotorPower sets raw motor power (bypass PID).

11.0.35 func (s *MotorControlService) SetVelocity(ctx context.Context, req *starv1.SetVelocityRequest) (*starv1.SetVelocityResponse, error)

SetVelocity sets wheel velocities for differential drive. Wraps the command in WireMessage for protocol multiplexing and sends via HARQ.

11.0.36 func (s *MotorControlService) StreamEncoders(req *starv1.StreamEncodersRequest, stream starv1.MotorControlService_StreamEncodersServer) error

StreamEncoders streams encoder readings from the motor controller. Uses Dispatcher to receive TelemetryData messages without contention.

11.0.37 type TelemetryService struct {
 starv1.UnimplementedTelemetryServiceServer
}

TelemetryService implements the TelemetryServiceServer interface. This service handles real-time sensor data aggregation and system health monitoring.

Architecture: - Uses WireMessage wrapper for protocol multiplexing - Uses Dispatcher for centralized message routing - Uses telemetryHolder for thread-safe caching of latest telemetry - Background goroutine updates cached telemetry from dispatcher

11.0.38 func NewTelemetryService(ctx context.Context, h harq.HARQ, d dispatcher.Dispatcher, logger *slog.Logger) *TelemetryService

NewTelemetryService creates a new TelemetryService. Requires a context for lifecycle management, HARQ handler for sending requests, Dispatcher for receiving telemetry, and logger.

11.0.39 func (s *TelemetryService) GetSystemStatus(_ context.Context, req *starv1.GetSystemStatusRequest) (*starv1.GetSystemStatusResponse, error)

GetSystemStatus requests system status from the RX72N firmware. **TODO**: Implement proper SystemStatusRequest message in wire.proto and send via HARQ. For now, returns mock status until firmware integration is complete.

11.0.40 func (s *TelemetryService) GetTelemetry(_ context.Context, req *starv1.GetTelemetryRequest) (*starv1.GetTelemetryResponse, error)

GetTelemetry returns a snapshot of the latest telemetry data. Uses cached telemetry updated by background goroutine for low latency.

11.0.41 func (s *TelemetryService) Shutdown()

Shutdown gracefully stops the TelemetryService background goroutines.

11.0.42 func (s *TelemetryService) StreamTelemetry(req *starv1.StreamTelemetryRequest, stream starv1.TelemetryService_StreamTelemetryServer) error

StreamTelemetry streams telemetry data at the requested [rate](#) (1-100 Hz).
Uses Dispatcher to receive telemetry and sends at controlled intervals.

12 Package: internal.testutil

Package testutil provides testing utilities and mocks for star-gateway.

STAR Project - Texas A&M University January 2026

12.0.1 CONSTANTS

12.0.2 const (

DefaultHARQTimeout = 50 * time.Millisecond)

12.0.3 FUNCTIONS

12.0.4 func NewDiscardLogger() *slog.Logger

NewDiscardLogger creates a logger that discards all `output` (for testing).

12.0.5 TYPES

12.0.6 type MockDispatcher struct {

SubscribeChan chan *dispatcher.DispatchedMessage SubscribeFunc func(msgType dispatcher.MessageType) <-chan *dispatcher.DispatchedMessage UnsubscribeFunc func(msgType dispatcher.MessageType, ch <-chan *dispatcher.DispatchedMessage) StartFunc func() error StopFunc func() error GetStateFunc func() dispatcher.State }

MockDispatcher is a mock implementation of the dispatcher.Dispatcher interface.

12.0.7 func (m *MockDispatcher) GetState() dispatcher.State

12.0.8 func (m *MockDispatcher) Start(ctx context.Context) error

12.0.9 func (m *MockDispatcher) Stop() error

12.0.10 func (m *MockDispatcher) Subscribe(msgType dispatcher.MessageType) <-chan *dispatcher.DispatchedMessage

12.0.11 func (m *MockDispatcher) Unsubscribe(msgType dispatcher.MessageType, ch <-chan *dispatcher.DispatchedMessage)

12.0.12 type MockHARQ struct {

SendFunc func(ctx context.Context, data []byte, p ...harq.Priority) error SendWithTypeFunc func(ctx context.Context, data []byte, frameType frame.Type) error ReceiveFunc func(ctx context.Context) (*harq.ReceiveResult, error) GetStateFunc func() harq.State GetTxSeqFunc func() uint16 GetRxSeqFunc func() uint16 ResetFunc func() LastSentPayload []byte

ReceiveChan chan *harq.ReceiveResult }

MockHARQ is a mock implementation of the harq.HARQ interface. Also implements the frameTypeSender interface (SendWithType) used by performResetHandshake.

12.0.13 func (m *MockHARQ) GetLastSentPayload() []byte

GetLastSentPayload returns a copy of the last sent payload in a thread-safe manner.

12.0.14 func (m *MockHARQ) GetRxSequence() uint16

12.0.15 func (m *MockHARQ) GetState() harq.State

12.0.16 func (m *MockHARQ) GetTxSequence() uint16

12.0.17 func (m *MockHARQ) Receive(ctx context.Context) (*harq.ReceiveResult, error)

12.0.18 func (m *MockHARQ) Reset()

12.0.19 func (m *MockHARQ) Send(ctx context.Context, data []byte, p ...harq.Priority)
error

12.0.20 func (m *MockHARQ) SendWithType(ctx context.Context, data []byte, frameType
frame.Type) error

SendWithType sends data with an explicit frame `type`. Used by
performResetHandshake to send `FrameTypeReset` frames.

13 Package: internal.transport

Package transport provides the USB CDC transport layer for RPi5 <-> RX72N communication.

The Raspberry Pi 5 communicates with the RX72N over USB CDC (Communications Device Class), which appears as a virtual serial port (e.g., /dev/ttyACM0). This provides a simpler interface than SPI and includes built-in flow control and reliability. This file helps us manage the raw serial communication over CDC. Doesnt check for any errors.

Reference: docs/sections/01_nanopb_protocol.tex

STAR Project - Texas A&M University January 2026

Package transport provides the low-level communication interfaces for the Gateway. This includes both real hardware (SPI) and simulation (Unix socket).

STAR Project - Texas A&M University January 2026

Package transport provides socket-based transport for HIL simulation.

Package transport provides the SPI transport layer for RPi5 <-> RX72N communication.

The Raspberry Pi 5 acts as the SPI controller, communicating with the RX72N peripheral at 10 MHz using SPI Mode 0.

Reference: docs/sections/01_nanopb_protocol.tex

STAR Project - Texas A&M University December 2025

13.0.1 CONSTANTS

13.0.2 const (

DefaultCDCDevice = “/dev/ttyACM0”

DefaultBaudRate = 115200

DefaultCDCTimeout = 100 * time.Millisecond

CDCDataBits = 8

RenesasVID = 0x045B

RX72NPID = 0x0235)

CDC configuration constants.

13.0.3 const (

DefaultSocketPath = “/tmp/star_rx72n.sock”

DefaultSocketTimeout = 100 * time.Millisecond

TransferSize = frame.MaxFrameSize)

Socket transport configuration constants.

13.0.4 const (

DefaultDevice = “/dev/spidev0.0”

DefaultSpeedHz = 10_000_000

DefaultMode = 0

DefaultBitsPerWord = 8

DefaultTimeout = 100 * time.Millisecond)

SPI configuration constants from hardware specification.

13.0.5 VARIABLES

13.0.6 var (

ErrDeviceNotFound = errors.New("cdc: device not found")

ErrReadTimeout = errors.New("cdc: read timeout"))

Predefined CDC-specific errors.

13.0.7 var (

ErrNotImplemented = errors.New("transport: not implemented")

ErrDeviceNotOpen = errors.New("transport: device not open")

ErrTimeout = errors.New("transport: operation timed out")

ErrTransferFailed = errors.New("transport: transfer failed"))

Predefined errors for transport operations.

13.0.8 TYPES

13.0.9 type CDCConfig struct {

Device string

BaudRate int

Timeout time.Duration

VID uint16

PID uint16 }

CDCConfig holds USB CDC configuration parameters.

13.0.10 func DefaultCDCConfig() *CDCConfig

DefaultCDCConfig returns the default CDC configuration.

13.0.11 type CDCTransport struct {

}

CDCTransport implements the Device interface for USB CDC communication. It provides context-aware operations for serial communication with the RX72N.

Unlike SPI (which is full-duplex), CDC is half-duplex:

- Send() writes data
- Receive() reads data
- Transfer() writes then reads (simulated full-duplex)

13.0.12 func NewCDCTransport(config *CDCConfig) *CDCTransport

NewCDCTransport creates a new CDCTransport with the given configuration.

If config is nil, uses DefaultCDCConfig().

13.0.13 func (c *CDCTransport) Close() error

Close releases the CDC device.

13.0.14 func (c *CDCTransport) Config() *CDCConfig

Config returns a copy of the current CDC configuration. Returns a copy to prevent external mutation of transport settings.

13.0.15 func (c *CDCTransport) IsOpen() bool

IsOpen returns whether the device is currently open.

13.0.16 func (c *CDCTransport) Open() error

Open initializes the CDC device. If Device is empty in config, attempts to auto-detect using VID/PID.

13.0.17 func (c *CDCTransport) Receive(maxLen int) ([]byte, error)

Receive reads data from CDC. Reads up to maxLen bytes from the serial port. Returns ErrReadTimeout if no data is available.

13.0.18 func (c *CDCTransport) Send(data []byte) (int, error)

Send transmits data over CDC. Ensures the full buffer is sent by looping until all bytes are written. Returns the number of bytes sent and any error.

13.0.19 func (c *CDCTransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)

Transfer performs a half-duplex transfer with context support. For CDC (half-duplex):

1. Sends txData
2. Waits for response
3. Reads response (same length as txData)

The context deadline is checked before and during the transfer. Uses a write lock because SetReadTimeout mutates port state.

13.0.20 type Device interface {

Transfer(ctx context.Context, txData []byte) ([]byte, error)

Open() error

Close() error

Receive(len int) ([]byte, error)

Send(data []byte) (int, error)

IsOpen() bool }

Device represents the low-level connection (SPI or Mock/Socket).

This interface enables dependency injection for testing and simulation, allowing the Gateway to communicate with either real hardware or a Virtual RX72N simulator without code changes.

Device is an alias for Transport with the addition of context-aware operations. Both SPITransport and SocketTransport implement this interface.

13.0.21 type SPIConfig struct {

Device string

SpeedHz uint32

Mode uint8

BitsPerWord uint8

Timeout time.Duration }

SPIConfig holds SPI configuration parameters.

13.0.22 func DefaultConfig() *SPIConfig

DefaultConfig returns the **default** SPI configuration.

13.0.23 type SPITransport struct { }

SPITransport implements the Device **interface** using **periph.io** for SPI communication. The Raspberry Pi 5 acts as the SPI controller, communicating with the RX72N at **10** MHz.

13.0.24 func NewSPITransport(config *SPIConfig) *SPITransport

NewSPITransport creates a new SPITransport with the given configuration. If config is **nil**, uses **DefaultConfig()**.

13.0.25 func (s *SPITransport) Close() error

Close releases the SPI device. Closes the **periph.io** SPI port and marks the transport as closed.

13.0.26 func (s *SPITransport) Config() *SPIConfig

Config returns the current SPI configuration.

13.0.27 func (s *SPITransport) IsOpen() bool

IsOpen returns whether the device is currently open.

13.0.28 func (s *SPITransport) Open() error

Open initializes the SPI device. Initializes **periph.io** host drivers, opens the SPI port, and configures it with the specified mode, speed, and bits per word.

13.0.29 func (s *SPITransport) Receive(maxLen int) ([]byte, error)

Receive reads data from SPI. Performs a read-only operation by sending dummy **bytes** (zeros). SPI is inherently full-duplex, so we must write while reading.

13.0.30 func (s *SPITransport) Send(data []byte) (int, error)

Send transmits data over SPI. Performs a write-only operation by discarding the received data. SPI is inherently full-duplex, so we must read while writing.

13.0.31 func (s *SPITransport) SetReadDeadline(deadline time.Time) error

SetReadDeadline sets a deadline **for** future Receive calls. Note: SPI transactions are synchronous and cannot be interrupted once started. The deadline is checked before each Receive.

13.0.32 func (s *SPITransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)

Transfer performs full-duplex SPI transfer with context support. Sends txData while simultaneously receiving data of the same length. This is the native SPI operation mode.

The context deadline is checked before the transfer. Note that SPI transactions are synchronous and cannot be interrupted once started.

13.0.33 type SocketTransport struct { }

SocketTransport implements the Device `interface` using Unix Domain Sockets. It uses the "Unlocking" pattern to ensure thread-safe, non-blocking shutdowns.

13.0.34 func NewSocketTransport(socketPath string) *SocketTransport

NewSocketTransport creates a new SocketTransport with the given socket path.

13.0.35 func (s *SocketTransport) Close() error

Close closes the socket connection.

Uses the "Unlocking" pattern: sets s.conn to `nil` under lock, then calls `conn.Close()` outside the lock. This ensures pending `Transfer()` calls are interrupted `immediately` (returning `net.ErrClosed`) without waiting `for` a mutex.

13.0.36 func (s *SocketTransport) IsOpen() bool

IsOpen returns whether the socket is currently connected.

13.0.37 func (s *SocketTransport) Open() error

Open establishes a connection to the Virtual RX72N process via Unix socket.

13.0.38 func (s *SocketTransport) Path() string

Path returns the Unix socket path.

13.0.39 func (s *SocketTransport) Receive(maxLen int) ([]byte, error)

Receive implements the legacy Transport `interface`.

13.0.40 func (s *SocketTransport) Send(data []byte) (int, error)

Send implements the legacy Transport `interface`.

13.0.41 func (s *SocketTransport) Transfer(ctx context.Context, txData []byte) ([]byte, error)

Transfer sends txData and receives the response.

Uses the "Unlocking" pattern: the mutex is held ONLY to retrieve the connection reference. The actual I/O is performed without the lock, allowing `Close()` to interrupt this operation immediately.

14 Package: internal.ws

Package ws provides WebSocket support for the STAR Gateway.

STAR Project - Texas A&M University February 2026

Package ws provides the WebSocket transport layer for the STAR Gateway.

STAR Project - Texas A&M University February 2026

Package ws provides WebSocket support for the STAR Gateway.

STAR Project - Texas A&M University February 2026

Package ws provides the WebSocket transport layer for the STAR Gateway's L5 (application/session) service layer.

Architecture role: ws sits between the HTTP server (net/http) and the GatewayService (internal/service).

It manages the lifecycle of individual WebSocket connections (Client), co-ordinates fan-out to all

connected UI clients (Hub), and serialises/deserialises binary Protobuf STAREnvelope frames.

GatewayService communicates with the hub exclusively through the HubNotifier interface so ws can be replaced or mocked in tests without touching service logic.

Thread-safety contract for implementations and consumers:

- Hub.Broadcast and Hub.Run are safe to call concurrently from different goroutines; Broadcast enqueues via a channel and never accesses the clients map directly.
- Hub.register and Hub.unregister channels are the ONLY external entry-points for modifying the clients map; all mutations happen inside Run's goroutine.
- Client.writePump is the ONLY goroutine that writes to the WebSocket connection; external code must not call conn.WriteMessage directly.
- MotorController.EmergencyStop may be called from Client.readPump while the connection is alive; implementations must be safe for concurrent calls.

STAR Project - Texas A&M University February 2026

14.0.1 CONSTANTS

14.0.2 const (

EstopTimeout = 5 * time.Second)

14.0.3 VARIABLES

14.0.4 var ErrBroadcastFull = errors.New("ws: hub broadcast channel full")

ErrBroadcastFull is returned by Broadcast when the hub's outbound channel is saturated and the envelope must be dropped to avoid blocking.

14.0.5 var ErrInvalidEnvelope = errors.New("ws: nil envelope passed to Broadcast")

ErrInvalidEnvelope is returned by Broadcast when the caller passes a nil envelope.

14.0.6 FUNCTIONS

14.0.7 func NewHandler(hub *Hub, motorService MotorController, logger *slog.Logger, allowInsecureWebsocket bool) http.Handler

NewHandler returns an http.Handler that upgrades each request to a WebSocket connection and wires it into hub.

allowInsecureWebsocket controls origin checking for incoming upgrade requests: when `true`, all origins are permitted -- useful for local-network or development deployments where requests arrive from a different port or host. When `false`, a same-origin policy is enforced -- the Origin header must match the request Host. Pass `false` for production deployments; pass `true` only for local/dev environments.

Panics at construction time if hub or motorService is `nil`:

- A `nil` hub has no client set and no event loop, so registration would block indefinitely and routing would be unreachable.
- A `nil` motorService would cause a `nil`-pointer dereference in Client.readPump on e-stop frames.

14.0.8 TYPES

14.0.9 type Client struct { }

Client represents a single connected WebSocket client managed by the Hub. Each Client owns two goroutines -- `readPump` (reads frames from the connection) and `writePump` (serialises frames onto the connection) -- and communicates with the Hub via the send channel and the `hub.register / hub.unregister` channels. `motorService` is called synchronously in `readPump` for e-stop frames. `logger` is used for structured diagnostic output; it is never `nil`.

14.0.10 func NewClient(hub *Hub, conn *websocket.Conn, motorService MotorController, logger *slog.Logger) *Client

NewClient creates a properly initialised Client and registers it with the provided hub. It is the canonical constructor for external callers; the hub's `handler` (`ws.NewHandler`) uses this to ensure all unexported fields are set.

Callers must start the read and write pump goroutines after construction:

```
go client.writePump()
client.readPump()
```

14.0.11 type Hub struct { }

Hub maintains the set of active clients and broadcasts messages to the clients.

14.0.12 func NewHub(logger *slog.Logger) *Hub

NewHub creates a new Hub with the provided logger.

14.0.13 func (h *Hub) Broadcast(ctx context.Context, env *starv1.STAREnvelope) error

Broadcast pushes env to all connected clients. Non-blocking: if the hub's internal channel is full, the envelope is dropped and ErrBroadcastFull is returned so callers can decide whether to log or meter the drop. If ctx is already cancelled when Broadcast is called, ctx.Err() is returned immediately.

Ownership transfer: the caller must not access or reuse env after calling Broadcast. The Hub's event loop (stampAndFanOut) mutates env.Seq and env.TimestampUs in-place before marshalling and fan-out.

14.0.14 func (h *Hub) ClientCount() int

ClientCount returns the current number of active registered WebSocket clients. The value is read atomically and is safe to call from any goroutine without locking. Intended for use by metrics collection and health checks.

14.0.15 func (h *Hub) Run(ctx context.Context)

Run starts the hub's main event loop. It blocks until ctx is cancelled and must therefore be invoked in its own goroutine so callers do not block.

Concurrency guarantees:

- Client registration and unregistration are safe from any goroutine via the h.register / h.unregister channels; internal client state is never accessed directly from outside Run.
- Outbound broadcasts are delivered from the h.broadcast channel; use Broadcast to enqueue messages without racing with the event loop.
- When ctx is cancelled, Run closes every active client's send channel before returning so writePump goroutines terminate cleanly.

14.0.16 func (h *Hub) SetInboundDispatcher(d InboundDispatcher)

SetInboundDispatcher registers an InboundDispatcher that receives all inbound WebSocket envelopes (non-e-stop payloads forwarded by client readPumps). Must be called before Hub.Run starts. Panics if called after Run has started to enforce the "set-before-start" invariant and prevent data races.

14.0.17 type HubNotifier interface {

Broadcast(ctx context.Context, env *starv1.STAREnvelope) error }

HubNotifier is the minimal interface GatewayService uses to push messages to the WebSocket hub.

14.0.18 type InboundDispatcher interface {

Dispatch(ctx context.Context, env *starv1.STAREnvelope) error }

InboundDispatcher receives envelopes forwarded inbound by WebSocket clients (all non-e-stop payloads read from the browser/UI). Implement this interface and register it with Hub.SetInboundDispatcher to route commands to the application layer instead of silently dropping them.

14.0.19 type MotorController interface {

EmergencyStop(ctx context.Context, reason string) error }

MotorController is the interface for motor safety operations. Implementations receive a context for cancellation and tracing; they may ignore cancellation but should propagate it to any downstream call that supports context.

Part VI

API Reference – ROS2 (C++)

ROS2 API Reference

This section contains the complete auto-generated API reference for the STAR ROS2 packages. Generated from C++ source code comments using Doxygen.

Source: `star-ros2/src/`

Language: C++17

Framework: ROS2 Humble

Generator: Doxygen 1.16.1

STAR ROS2 Packages

Generated by Doxygen 1.16.1

	i
1 Directory Hierarchy	1
1.1 Directories	1
2 Namespace Index	5
2.1 Namespace List	5
3 Hierarchical Index	7
3.1 Class Hierarchy	7
4 Class Index	9
4.1 Class List	9
5 File Index	11
5.1 File List	11
6 Directory Documentation	13
6.1 src/star_gateway_bridge/include Directory Reference	13
6.2 src/star_safety_monitor/include Directory Reference	14
6.3 src/star_spi_bridge/include Directory Reference	14
6.4 src Directory Reference	15
6.5 src/star_gateway_bridge/src Directory Reference	15
6.6 src/star_safety_monitor/src Directory Reference	16
6.7 src/star_spi_bridge/src Directory Reference	16
6.8 src/star_gateway_bridge Directory Reference	17
6.9 src/star_gateway_bridge/include/star_gateway_bridge Directory Reference	17
6.10 src/star_safety_monitor Directory Reference	18
6.11 src/star_safety_monitor/include/star_safety_monitor Directory Reference	18
6.12 src/star_spi_bridge Directory Reference	19
6.13 src/star_spi_bridge/include/star_spi_bridge Directory Reference	19
6.14 src/star_gateway_bridge/test Directory Reference	20
6.15 src/star_safety_monitor/test Directory Reference	20
6.16 src/star_spi_bridge/test Directory Reference	21
7 Namespace Documentation	23
7.1 star Namespace Reference	23
7.1.1 Function Documentation	24
7.1.1.1 TEST_F() [1/33]	24
7.1.1.2 TEST_F() [2/33]	24
7.1.1.3 TEST_F() [3/33]	24
7.1.1.4 TEST_F() [4/33]	24
7.1.1.5 TEST_F() [5/33]	24
7.1.1.6 TEST_F() [6/33]	25
7.1.1.7 TEST_F() [7/33]	25
7.1.1.8 TEST_F() [8/33]	25

7.1.1.9 TEST_F() [9/33]	25
7.1.1.10 TEST_F() [10/33]	25
7.1.1.11 TEST_F() [11/33]	25
7.1.1.12 TEST_F() [12/33]	26
7.1.1.13 TEST_F() [13/33]	26
7.1.1.14 TEST_F() [14/33]	26
7.1.1.15 TEST_F() [15/33]	26
7.1.1.16 TEST_F() [16/33]	26
7.1.1.17 TEST_F() [17/33]	26
7.1.1.18 TEST_F() [18/33]	27
7.1.1.19 TEST_F() [19/33]	27
7.1.1.20 TEST_F() [20/33]	27
7.1.1.21 TEST_F() [21/33]	27
7.1.1.22 TEST_F() [22/33]	27
7.1.1.23 TEST_F() [23/33]	27
7.1.1.24 TEST_F() [24/33]	28
7.1.1.25 TEST_F() [25/33]	28
7.1.1.26 TEST_F() [26/33]	28
7.1.1.27 TEST_F() [27/33]	28
7.1.1.28 TEST_F() [28/33]	28
7.1.1.29 TEST_F() [29/33]	28
7.1.1.30 TEST_F() [30/33]	29
7.1.1.31 TEST_F() [31/33]	29
7.1.1.32 TEST_F() [32/33]	29
7.1.1.33 TEST_F() [33/33]	29
7.2 star_safety_monitor Namespace Reference	29
7.2.1 Enumeration Type Documentation	29
7.2.1.1 SeverityLevel	29
7.3 star_spi_bridge Namespace Reference	30
7.3.1 Enumeration Type Documentation	30
7.3.1.1 FrameType	30
7.3.2 Variable Documentation	31
7.3.2.1 klmuAngularVelocityVar	31
7.3.2.2 klmuLinearAccelVar	31
7.3.2.3 klmuOrientationVar	31
7.3.2.4 kLogThrottleMs	31
8 Class Documentation	33
8.1 star::MessageConverter Class Reference	33
8.1.1 Detailed Description	33
8.1.2 Member Function Documentation	34
8.1.2.1 clamp()	34

8.1.2.2 is_valid_double()	34
8.1.2.3 pid_config_to_gains()	35
8.1.2.4 ros_time_to_us()	36
8.1.2.5 string_to_system_status()	36
8.1.2.6 twist_to_velocity_command()	37
8.1.2.7 velocity_command_to_twist()	38
8.2 star::MessageConverterTest Class Reference	39
8.2.1 Detailed Description	40
8.2.2 Member Data Documentation	40
8.2.2.1 converter_	40
8.2.2.2 TOLERANCE	40
8.2.2.3 WHEEL_BASE_M	41
8.3 SpiMessageConverter::Parameters Struct Reference	41
8.3.1 Detailed Description	41
8.3.2 Member Data Documentation	41
8.3.2.1 ticks_per_rev	41
8.3.2.2 wheel_base	41
8.3.2.3 wheel_radius	41
8.4 star_spi_bridge::SpiMessageConverter::Parameters Struct Reference	42
8.4.1 Detailed Description	42
8.4.2 Member Data Documentation	42
8.4.2.1 ticks_per_rev	42
8.4.2.2 wheel_base	42
8.4.2.3 wheel_radius	42
8.5 star_safety_monitor::SafetyMonitor Class Reference	43
8.5.1 Detailed Description	44
8.5.2 Constructor & Destructor Documentation	44
8.5.2.1 SafetyMonitor()	44
8.5.2.2 ~SafetyMonitor()	44
8.5.3 Member Function Documentation	44
8.5.3.1 on_activate()	44
8.5.3.2 on_cleanup()	44
8.5.3.3 on_configure()	44
8.5.3.4 on_deactivate()	45
8.5.3.5 on_shutdown()	45
8.6 SafetyMonitorTest Class Reference	45
8.6.1 Detailed Description	46
8.6.2 Member Function Documentation	46
8.6.2.1 SetUp()	46
8.6.2.2 TearDown()	46
8.7 spi_ioc_transfer Struct Reference	46
8.7.1 Detailed Description	46

8.7.2 Member Data Documentation	47
8.7.2.1 bits_per_word_	47
8.7.2.2 cs_change_	47
8.7.2.3 delay_usecs_	47
8.7.2.4 len_	47
8.7.2.5 pad_	47
8.7.2.6 rx_buf_	47
8.7.2.7 speed_hz_	47
8.7.2.8 tx_buf_	48
8.8 SpiDriver Class Reference	48
8.8.1 Detailed Description	49
8.8.2 Constructor & Destructor Documentation	49
8.8.2.1 SpiDriver()	49
8.8.2.2 ~SpiDriver()	50
8.8.3 Member Function Documentation	50
8.8.3.1 calculate_crc32()	50
8.8.3.2 close_device()	51
8.8.3.3 decode_frame()	51
8.8.3.4 encode_frame()	53
8.8.3.5 initialize()	55
8.8.3.6 transfer()	55
8.8.4 Member Data Documentation	56
8.8.4.1 k_crc_size	56
8.8.4.2 k_header_size	56
8.8.4.3 k_max_payload_size	56
8.8.4.4 k_sync_word	56
8.9 star_spi_bridge::SpiDriver Class Reference	56
8.9.1 Detailed Description	57
8.9.2 Constructor & Destructor Documentation	58
8.9.2.1 SpiDriver()	58
8.9.2.2 ~SpiDriver()	59
8.9.3 Member Function Documentation	59
8.9.3.1 calculate_crc32()	59
8.9.3.2 close_device()	60
8.9.3.3 decode_frame()	61
8.9.3.4 encode_frame()	62
8.9.3.5 initialize()	63
8.9.3.6 transfer()	63
8.9.4 Member Data Documentation	64
8.9.4.1 k_crc_size	64
8.9.4.2 k_header_size	64
8.9.4.3 k_max_payload_size	65

	v
8.9.4.4 k_sync_word	65
8.10 SpiDriverTest Class Reference	65
8.10.1 Detailed Description	66
8.10.2 Member Function Documentation	66
8.10.2.1 SetUp()	66
8.11 SpiMessageConverter Class Reference	66
8.11.1 Detailed Description	66
8.11.2 Constructor & Destructor Documentation	67
8.11.2.1 SpiMessageConverter()	67
8.11.3 Member Function Documentation	67
8.11.3.1 telemetry_to_joint_state()	67
8.11.3.2 telemetry_to_odometry()	67
8.11.3.3 twist_to_velocity_command()	67
8.12 star_spi_bridge::SpiMessageConverter Class Reference	67
8.12.1 Detailed Description	68
8.12.2 Constructor & Destructor Documentation	68
8.12.2.1 SpiMessageConverter()	68
8.12.3 Member Function Documentation	68
8.12.3.1 telemetry_to_joint_state()	68
8.12.3.2 telemetry_to_odometry()	68
8.12.3.3 twist_to_velocity_command()	69
8.13 SpiMessageConverterTest Class Reference	69
8.13.1 Detailed Description	70
8.13.2 Member Data Documentation	70
8.13.2.1 converter	70
8.13.2.2 params	70
8.14 star::StarGatewayBridgeNode Class Reference	70
8.14.1 Detailed Description	71
8.14.2 Constructor & Destructor Documentation	72
8.14.2.1 StarGatewayBridgeNode()	72
8.14.2.2 ~StarGatewayBridgeNode()	72
8.15 star_spi_bridge::StarSpiDriverNode Class Reference	73
8.15.1 Detailed Description	74
8.15.2 Constructor & Destructor Documentation	74
8.15.2.1 StarSpiDriverNode()	74
8.15.2.2 ~StarSpiDriverNode()	74
8.15.3 Member Function Documentation	74
8.15.3.1 on_activate()	74
8.15.3.2 on_cleanup()	74
8.15.3.3 on_configure()	75
8.15.3.4 on_deactivate()	75
8.15.3.5 on_shutdown()	75

9 File Documentation	77
9.1 src/star_gateway_bridge/include/star_gateway_bridge/message_converter.hpp File Reference	77
9.2 message_converter.hpp	78
9.3 src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp File Reference	79
9.4 star_gateway_bridge_node.hpp	80
9.5 src/star_gateway_bridge/src/main.cpp File Reference	81
9.5.1 Function Documentation	82
9.5.1.1 main()	82
9.6 main.cpp	82
9.7 src/star_spi_bridge/src/main.cpp File Reference	83
9.7.1 Function Documentation	83
9.7.1.1 main()	83
9.8 main.cpp	83
9.9 src/star_gateway_bridge/src/message_converter.cpp File Reference	84
9.10 message_converter.cpp	84
9.11 src/star_gateway_bridge/src/star_gateway_bridge_node.cpp File Reference	86
9.12 star_gateway_bridge_node.cpp	87
9.13 src/star_gateway_bridge/test/test_message_converter.cpp File Reference	94
9.13.1 Function Documentation	95
9.13.1.1 main()	95
9.14 test_message_converter.cpp	95
9.15 src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp File Reference	101
9.16 safety_monitor.hpp	102
9.17 src/star_safety_monitor/src/safety_monitor.cpp File Reference	104
9.18 safety_monitor.cpp	104
9.19 src/star_safety_monitor/src/safety_monitor_node.cpp File Reference	110
9.19.1 Function Documentation	110
9.19.1.1 main()	110
9.20 safety_monitor_node.cpp	110
9.21 src/star_safety_monitor/test/test_safety_monitor.cpp File Reference	111
9.21.1 Function Documentation	112
9.21.1.1 main()	112
9.21.1.2 TEST_F() [1/9]	112
9.21.1.3 TEST_F() [2/9]	112
9.21.1.4 TEST_F() [3/9]	112
9.21.1.5 TEST_F() [4/9]	112
9.21.1.6 TEST_F() [5/9]	113
9.21.1.7 TEST_F() [6/9]	113
9.21.1.8 TEST_F() [7/9]	113
9.21.1.9 TEST_F() [8/9]	113
9.21.1.10 TEST_F() [9/9]	113
9.22 test_safety_monitor.cpp	114

9.23 src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp File Reference	116
9.24 spi_driver.hpp	117
9.25 src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp File Reference	118
9.26 spi_message_converter.hpp	119
9.27 src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_node.hpp File Reference	120
9.28 star_spi_driver_node.hpp	121
9.29 src/star_spi_bridge/src/spi_driver.cpp File Reference	122
9.29.1 Macro Definition Documentation	123
9.29.1.1 SPI_IOC_MESSAGE	123
9.29.2 Variable Documentation	123
9.29.2.1 SPI_IOC_WR_BITS_PER_WORD	123
9.29.2.2 SPI_IOC_WR_MAX_SPEED_HZ	123
9.29.2.3 SPI_IOC_WR_MODE	124
9.29.2.4 SPI_MODE_0	124
9.30 spi_driver.cpp	124
9.31 src/star_spi_bridge/src/spi_message_converter.cpp File Reference	128
9.32 spi_message_converter.cpp	128
9.33 src/star_spi_bridge/src/star_spi_driver_node.cpp File Reference	131
9.34 star_spi_driver_node.cpp	131
9.35 src/star_spi_bridge/test/test_spi_driver.cpp File Reference	135
9.35.1 Enumeration Type Documentation	136
9.35.1.1 FrameType	136
9.35.2 Function Documentation	136
9.35.2.1 TEST_F() [1/6]	136
9.35.2.2 TEST_F() [2/6]	137
9.35.2.3 TEST_F() [3/6]	137
9.35.2.4 TEST_F() [4/6]	138
9.35.2.5 TEST_F() [5/6]	138
9.35.2.6 TEST_F() [6/6]	139
9.36 test_spi_driver.cpp	139
9.37 src/star_spi_bridge/test/test_spi_message_converter.cpp File Reference	140
9.37.1 Function Documentation	141
9.37.1.1 TEST_F() [1/10]	141
9.37.1.2 TEST_F() [2/10]	141
9.37.1.3 TEST_F() [3/10]	142
9.37.1.4 TEST_F() [4/10]	142
9.37.1.5 TEST_F() [5/10]	142
9.37.1.6 TEST_F() [6/10]	142
9.37.1.7 TEST_F() [7/10]	142
9.37.1.8 TEST_F() [8/10]	143
9.37.1.9 TEST_F() [9/10]	143
9.37.1.10 TEST_F() [10/10]	143

viii

9.38 test_spi_message_converter.cpp	143
Index	147

Chapter 1

Directory Hierarchy

1.1 Directories

include	13
star_gateway_bridge	17
message_converter.hpp	77
star_gateway_bridge_node.hpp	79
include	14
star_safety_monitor	18
safety_monitor.hpp	101
include	14
star_spi_bridge	19
spi_driver.hpp	116
spi_message_converter.hpp	118
star_spi_driver_node.hpp	120
src	15
star_gateway_bridge	17
include	13
star_gateway_bridge	17
message_converter.hpp	77
star_gateway_bridge_node.hpp	79
src	15
main.cpp	81
message_converter.cpp	84
star_gateway_bridge_node.cpp	86
test	20
test_message_converter.cpp	94
star_safety_monitor	18
include	14
star_safety_monitor	18
safety_monitor.hpp	101
src	16
safety_monitor.cpp	104
safety_monitor_node.cpp	110
test	20
test_safety_monitor.cpp	111
star_spi_bridge	19
include	14
star_spi_bridge	19

Generated by Doxygen

spi_driver.hpp	116
spi_message_converter.hpp	118
star_spi_driver_node.hpp	120
src	16
main.cpp	83
spi_driver.cpp	122
spi_message_converter.cpp	128
star_spi_driver_node.cpp	131
test	21
test_spi_driver.cpp	135
test_spi_message_converter.cpp	140
src	15
main.cpp	81
message_converter.cpp	84
star_gateway_bridge_node.cpp	86
src	16
safety_monitor.cpp	104
safety_monitor_node.cpp	110
src	16
main.cpp	83
spi_driver.cpp	122
spi_message_converter.cpp	128
star_spi_driver_node.cpp	131
star_gateway_bridge	17
include	13
star_gateway_bridge	17
message_converter.hpp	77
star_gateway_bridge_node.hpp	79
src	15
main.cpp	81
message_converter.cpp	84
star_gateway_bridge_node.cpp	86
test	20
test_message_converter.cpp	94
star_gateway_bridge	17
message_converter.hpp	77
star_gateway_bridge_node.hpp	79
star_safety_monitor	18
include	14
star_safety_monitor	18
safety_monitor.hpp	101
src	16
safety_monitor.cpp	104
safety_monitor_node.cpp	110
test	20
test_safety_monitor.cpp	111
star_safety_monitor	18
safety_monitor.hpp	101
star_spi_bridge	19
include	14
star_spi_bridge	19
spi_driver.hpp	116
spi_message_converter.hpp	118
star_spi_driver_node.hpp	120
src	16

1.1 Directories**3**

main.cpp	83
spi_driver.cpp	122
spi_message_converter.cpp	128
star_spi_driver_node.cpp	131
test	21
test_spi_driver.cpp	135
test_spi_message_converter.cpp	140
star_spi_bridge	19
spi_driver.hpp	116
spi_message_converter.hpp	118
star_spi_driver_node.hpp	120
test	20
test_message_converter.cpp	94
test	20
test_safety_monitor.cpp	111
test	21
test_spi_driver.cpp	135
test_spi_message_converter.cpp	140

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

star	23
star_safety_monitor	29
star_spi_bridge	30

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

rcldcpp_lifecycle::LifecycleNode	
star_safety_monitor::SafetyMonitor	43
star_spi_bridge::StarSpiDriverNode	73
star::MessageConverter	33
rcldcpp::Node	
star::StarGatewayBridgeNode	70
SpiMessageConverter::Parameters	41
star_spi_bridge::SpiMessageConverter::Parameters	42
spi_ioc_transfer	46
SpiDriver	48
star_spi_bridge::SpiDriver	56
SpiMessageConverter	66
star_spi_bridge::SpiMessageConverter	67
testing::Test	
SafetyMonitorTest	45
SpiDriverTest	65
SpiMessageConverterTest	69
star::MessageConverterTest	39

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

star::MessageConverter	
Message converter for ROS2 <-> Protobuf translation	33
star::MessageConverterTest	39
SpiMessageConverter::Parameters	41
star_spi_bridge::SpiMessageConverter::Parameters	42
star_safety_monitor::SafetyMonitor	43
SafetyMonitorTest	45
spi_ioc_transfer	46
SpiDriver	
SPI driver for framed communication with the RX72N peripheral MCU	48
star_spi_bridge::SpiDriver	
SPI driver for framed communication with the RX72N peripheral MCU	56
SpiDriverTest	65
SpiMessageConverter	66
star_spi_bridge::SpiMessageConverter	67
SpiMessageConverterTest	69
star::StarGatewayBridgeNode	
ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC	70
star_spi_bridge::StarSpiDriverNode	73

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

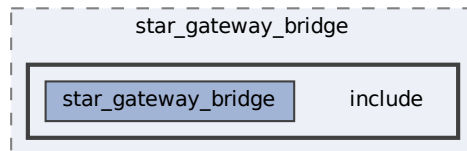
src/star_gateway_bridge/include/star_gateway_bridge/message_converter.hpp	77
src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp	79
src/star_gateway_bridge/src/main.cpp	81
src/star_gateway_bridge/src/message_converter.cpp	84
src/star_gateway_bridge/src/star_gateway_bridge_node.cpp	86
src/star_gateway_bridge/test/test_message_converter.cpp	94
src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp	101
src/star_safety_monitor/src/safety_monitor.cpp	104
src/star_safety_monitor/src/safety_monitor_node.cpp	110
src/star_safety_monitor/test/test_safety_monitor.cpp	111
src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp	116
src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp	118
src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_node.hpp	120
src/star_spi_bridge/src/main.cpp	83
src/star_spi_bridge/src/spi_driver.cpp	122
src/star_spi_bridge/src/spi_message_converter.cpp	128
src/star_spi_bridge/src/star_spi_driver_node.cpp	131
src/star_spi_bridge/test/test_spi_driver.cpp	135
src/star_spi_bridge/test/test_spi_message_converter.cpp	140

Chapter 6

Directory Documentation

6.1 src/star_gateway_bridge/include Directory Reference

Directory dependency graph for include:

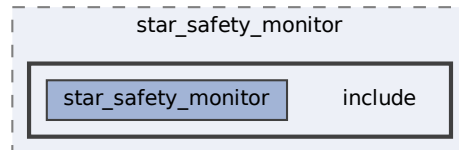


Directories

- directory [star_gateway_bridge](#)

6.2 src/star_safety_monitor/include Directory Reference

Directory dependency graph for include:

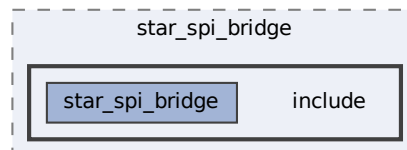


Directories

- directory [star_safety_monitor](#)

6.3 src/star_spi_bridge/include Directory Reference

Directory dependency graph for include:



Directories

- directory [star_spi_bridge](#)

6.4 src Directory Reference

Directory dependency graph for src:

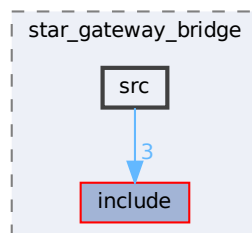


Directories

- directory [star_gateway_bridge](#)
- directory [star_safety_monitor](#)
- directory [star_spi_bridge](#)

6.5 src/star_gateway_bridge/src Directory Reference

Directory dependency graph for src:

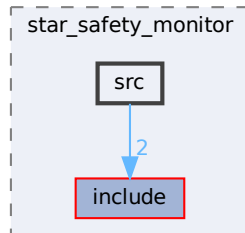


Files

- file [main.cpp](#)
- file [message_converter.cpp](#)
- file [star_gateway_bridge_node.cpp](#)

6.6 src/star_safety_monitor/src Directory Reference

Directory dependency graph for src:

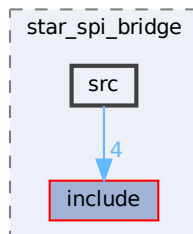


Files

- file [safety_monitor.cpp](#)
- file [safety_monitor_node.cpp](#)

6.7 src/star_spi_bridge/src Directory Reference

Directory dependency graph for src:



6.8 src/star_gateway_bridge Directory Reference

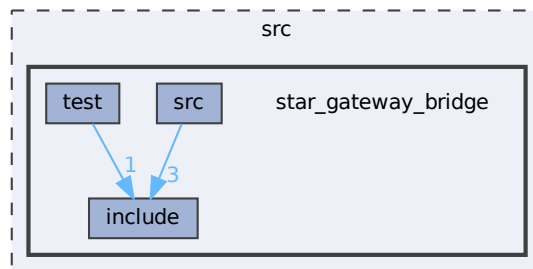
17

Files

- file [main.cpp](#)
- file [spi_driver.cpp](#)
- file [spi_message_converter.cpp](#)
- file [star_spi_driver_node.cpp](#)

6.8 src/star_gateway_bridge Directory Reference

Directory dependency graph for star_gateway_bridge:

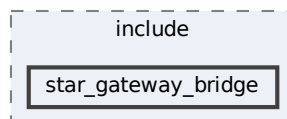


Directories

- directory [include](#)
- directory [src](#)
- directory [test](#)

6.9 src/star_gateway_bridge/include/star_gateway_bridge Directory Reference

Directory dependency graph for star_gateway_bridge:

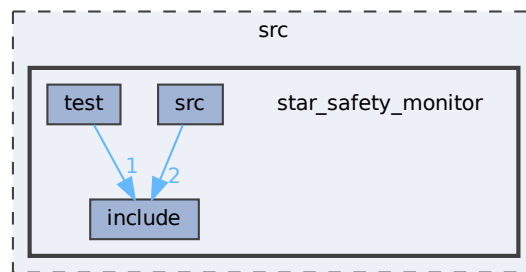


Files

- file [message_converter.hpp](#)
- file [star_gateway_bridge_node.hpp](#)

6.10 src/star_safety_monitor Directory Reference

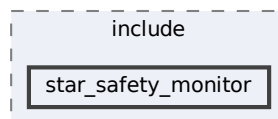
Directory dependency graph for star_safety_monitor:

**Directories**

- directory [include](#)
- directory [src](#)
- directory [test](#)

6.11 src/star_safety_monitor/include/star_safety_monitor Directory Reference

Directory dependency graph for star_safety_monitor:



6.12 src/star_spi_bridge Directory Reference

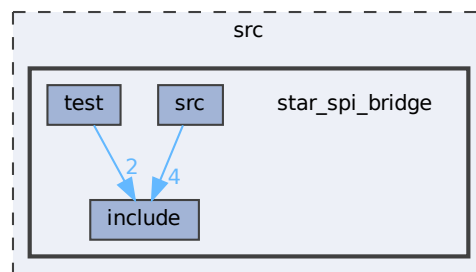
19

Files

- file [safety_monitor.hpp](#)

6.12 src/star_spi_bridge Directory Reference

Directory dependency graph for star_spi_bridge:

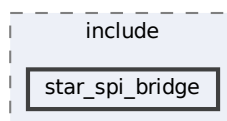


Directories

- directory [include](#)
- directory [src](#)
- directory [test](#)

6.13 src/star_spi_bridge/include/star_spi_bridge Directory Reference

Directory dependency graph for star_spi_bridge:

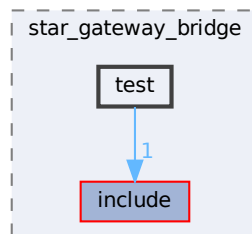


Files

- file [spi_driver.hpp](#)
- file [spi_message_converter.hpp](#)
- file [star_spi_driver_node.hpp](#)

6.14 src/star_gateway_bridge/test Directory Reference

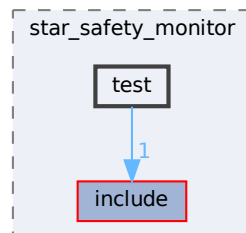
Directory dependency graph for test:

**Files**

- file [test_message_converter.cpp](#)

6.15 src/star_safety_monitor/test Directory Reference

Directory dependency graph for test:

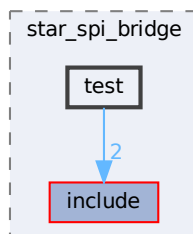


6.16 src/star_spi_bridge/test Directory Reference**21****Files**

- file [test_safety_monitor.cpp](#)

6.16 src/star_spi_bridge/test Directory Reference

Directory dependency graph for test:

**Files**

- file [test_spi_driver.cpp](#)
- file [test_spi_message_converter.cpp](#)

Chapter 7

Namespace Documentation

7.1 star Namespace Reference

Classes

- class [MessageConverter](#)
Message converter for ROS2 <-> Protobuf translation.
- class [StarGatewayBridgeNode](#)
ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.
- class [MessageConverterTest](#)

Functions

- [TEST_F \(MessageConverterTest, TwistToCommandZeroVelocity\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandPureTranslation\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandPureRotation\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandMixedMotion\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandNegativeLinear\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandNegativeAngular\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistZeroVelocity\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistPureTranslation\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistPureRotation\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistMixedMotion\)](#)
- [TEST_F \(MessageConverterTest, KinematicsRoundtripZero\)](#)
- [TEST_F \(MessageConverterTest, KinematicsRoundtripTranslation\)](#)
- [TEST_F \(MessageConverterTest, KinematicsRoundtripRotation\)](#)
- [TEST_F \(MessageConverterTest, KinematicsRoundtripMixed\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandNaNLinear\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandNaNAngular\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandInfinityLinear\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandInfinityAngular\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandNegativeInfinity\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandZeroWheelBase\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandNegativeWheelBase\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistNaNMotor0\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistNaNMotor1\)](#)
- [TEST_F \(MessageConverterTest, CommandToTwistInfinityMotor0\)](#)

Generated by Doxygen

- [TEST_F \(MessageConverterTest, PidConfigToGainsNominal\)](#)
- [TEST_F \(MessageConverterTest, PidConfigToGainsZeroGains\)](#)
- [TEST_F \(MessageConverterTest, PidConfigToGainsNaN\)](#)
- [TEST_F \(MessageConverterTest, PidConfigToGainsInfinity\)](#)
- [TEST_F \(MessageConverterTest, PidConfigToGainsNegativeGains\)](#)
- [TEST_F \(MessageConverterTest, PidConfigToGainsLargeValues\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandMaxVelocity\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandVerySmallWheelBase\)](#)
- [TEST_F \(MessageConverterTest, TwistToCommandVeryLargeWheelBase\)](#)

7.1.1 Function Documentation

7.1.1.1 TEST_F() [1/33]

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistInfinityMotor0 )
```

Definition at line 358 of file [test_message_converter.cpp](#).

7.1.1.2 TEST_F() [2/33]

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistMixedMotion )
```

Definition at line 176 of file [test_message_converter.cpp](#).

7.1.1.3 TEST_F() [3/33]

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistNaNMotor0 )
```

Definition at line 338 of file [test_message_converter.cpp](#).

7.1.1.4 TEST_F() [4/33]

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistNaNMotor1 )
```

Definition at line 348 of file [test_message_converter.cpp](#).

7.1.1.5 TEST_F() [5/33]

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistPureRotation )
```

Definition at line 155 of file [test_message_converter.cpp](#).

7.1 star Namespace Reference**25****7.1.1.6 TEST_F() [6/33]**

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistPureTranslation )
```

Definition at line 138 of file [test_message_converter.cpp](#).

7.1.1.7 TEST_F() [7/33]

```
star::TEST_F (
    MessageConverterTest ,
    CommandToTwistZeroVelocity )
```

Definition at line 125 of file [test_message_converter.cpp](#).

7.1.1.8 TEST_F() [8/33]

```
star::TEST_F (
    MessageConverterTest ,
    KinematicsRoundtripMixed )
```

Definition at line 248 of file [test_message_converter.cpp](#).

7.1.1.9 TEST_F() [9/33]

```
star::TEST_F (
    MessageConverterTest ,
    KinematicsRoundtripRotation )
```

Definition at line 232 of file [test_message_converter.cpp](#).

7.1.1.10 TEST_F() [10/33]

```
star::TEST_F (
    MessageConverterTest ,
    KinematicsRoundtripTranslation )
```

Definition at line 216 of file [test_message_converter.cpp](#).

7.1.1.11 TEST_F() [11/33]

```
star::TEST_F (
    MessageConverterTest ,
    KinematicsRoundtripZero )
```

Definition at line 200 of file [test_message_converter.cpp](#).

Generated by Doxygen

7.1.1.12 TEST_F() [12/33]

```
star::TEST_F (
    MessageConverterTest ,
    PidConfigToGainsInfinity )
```

Definition at line 413 of file [test_message_converter.cpp](#).

7.1.1.13 TEST_F() [13/33]

```
star::TEST_F (
    MessageConverterTest ,
    PidConfigToGainsLargeValues )
```

Definition at line 441 of file [test_message_converter.cpp](#).

7.1.1.14 TEST_F() [14/33]

```
star::TEST_F (
    MessageConverterTest ,
    PidConfigToGainsNaN )
```

Definition at line 402 of file [test_message_converter.cpp](#).

7.1.1.15 TEST_F() [15/33]

```
star::TEST_F (
    MessageConverterTest ,
    PidConfigToGainsNegativeGains )
```

Definition at line 424 of file [test_message_converter.cpp](#).

7.1.1.16 TEST_F() [16/33]

```
star::TEST_F (
    MessageConverterTest ,
    PidConfigToGainsNominal )
```

Definition at line 372 of file [test_message_converter.cpp](#).

7.1.1.17 TEST_F() [17/33]

```
star::TEST_F (
    MessageConverterTest ,
    PidConfigToGainsZeroGains )
```

Definition at line 387 of file [test_message_converter.cpp](#).

7.1 star Namespace Reference**27****7.1.1.18 TEST_F() [18/33]**

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandInfinityAngular )
```

Definition at line 298 of file [test_message_converter.cpp](#).

7.1.1.19 TEST_F() [19/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandInfinityLinear )
```

Definition at line 288 of file [test_message_converter.cpp](#).

7.1.1.20 TEST_F() [20/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandMaxVelocity )
```

Definition at line 460 of file [test_message_converter.cpp](#).

7.1.1.21 TEST_F() [21/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandMixedMotion )
```

Definition at line 72 of file [test_message_converter.cpp](#).

7.1.1.22 TEST_F() [22/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandNaNAngular )
```

Definition at line 278 of file [test_message_converter.cpp](#).

7.1.1.23 TEST_F() [23/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandNaNLinear )
```

Definition at line 268 of file [test_message_converter.cpp](#).

Generated by Doxygen

7.1.1.24 TEST_F() [24/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandNegativeAngular )
```

Definition at line 105 of file [test_message_converter.cpp](#).

7.1.1.25 TEST_F() [25/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandNegativeInfinity )
```

Definition at line 308 of file [test_message_converter.cpp](#).

7.1.1.26 TEST_F() [26/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandNegativeLinear )
```

Definition at line 91 of file [test_message_converter.cpp](#).

7.1.1.27 TEST_F() [27/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandNegativeWheelBase )
```

Definition at line 328 of file [test_message_converter.cpp](#).

7.1.1.28 TEST_F() [28/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandPureRotation )
```

Definition at line 56 of file [test_message_converter.cpp](#).

7.1.1.29 TEST_F() [29/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandPureTranslation )
```

Definition at line 42 of file [test_message_converter.cpp](#).

7.2 star_safety_monitor Namespace Reference**29****7.1.1.30 TEST_F()** [30/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandVeryLargeWheelBase )
```

Definition at line 492 of file [test_message_converter.cpp](#).

7.1.1.31 TEST_F() [31/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandVerySmallWheelBase )
```

Definition at line 477 of file [test_message_converter.cpp](#).

7.1.1.32 TEST_F() [32/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandZeroVelocity )
```

Definition at line 29 of file [test_message_converter.cpp](#).

7.1.1.33 TEST_F() [33/33]

```
star::TEST_F (
    MessageConverterTest ,
    TwistToCommandZeroWheelBase )
```

Definition at line 318 of file [test_message_converter.cpp](#).

7.2 star_safety_monitor Namespace Reference**Classes**

- class [SafetyMonitor](#)

Enumerations

- enum class [SeverityLevel](#) { [OK](#) = 0 , [WARN](#) = 1 , [ERROR](#) = 2 , [STALE](#) = 3 }

7.2.1 Enumeration Type Documentation**7.2.1.1 SeverityLevel**

```
enum class star_safety_monitor::SeverityLevel [strong]
```

Enumerator

OK	
----	--

WARN	
ERROR	
STALE	

Definition at line 41 of file [safety_monitor.hpp](#).

7.3 star_spi_bridge Namespace Reference

Classes

- class [SpiDriver](#)
SPI driver for framed communication with the RX72N peripheral MCU.
- class [SpiMessageConverter](#)
- class [StarSpiDriverNode](#)

Enumerations

- enum class [FrameType](#) : uint8_t { [VelocityCommand](#) = 0x01 , [TelemetryData](#) = 0x02 }
Frame type identifiers for SPI protocol messages.

Variables

- static constexpr int [kLogThrottleMs](#) = 1000
- static constexpr double [kImuOrientationVar](#) = 0.01
- static constexpr double [kImuAngularVelocityVar](#) = 0.001
- static constexpr double [kImuLinearAccelVar](#) = 0.01

7.3.1 Enumeration Type Documentation

7.3.1.1 FrameType

```
enum class star_spi_bridge::FrameType : uint8_t [strong]
```

Frame type identifiers for SPI protocol messages.

Each frame carries a one-byte type field at offset 6 in the header. Values match the RX72N firmware `rx_frame_type_t` enum so both sides agree on the semantics of every message.

See also

[SpiDriver::encode_frame](#) Frame construction
[SpiDriver::decode_frame](#) Frame parsing

Since

Version 1.0.0

Enumerator

VelocityCommand	Motor velocity setpoint (RPI5 -> RX72N)
-----------------	---

7.3 star_spi_bridge Namespace Reference**31**

TelemetryData	Sensor telemetry payload (RX72N -> RPi5)
---------------	--

Definition at line 27 of file [spi_driver.hpp](#).

7.3.2 Variable Documentation**7.3.2.1 kImuAngularVelocityVar**

```
double star_spi_bridge::kImuAngularVelocityVar = 0.001 [static], [constexpr]
```

Definition at line 18 of file [star_spi_driver_node.cpp](#).

7.3.2.2 kImuLinearAccelVar

```
double star_spi_bridge::kImuLinearAccelVar = 0.01 [static], [constexpr]
```

Definition at line 19 of file [star_spi_driver_node.cpp](#).

7.3.2.3 kImuOrientationVar

```
double star_spi_bridge::kImuOrientationVar = 0.01 [static], [constexpr]
```

Definition at line 17 of file [star_spi_driver_node.cpp](#).

7.3.2.4 kLogThrottleMs

```
int star_spi_bridge::kLogThrottleMs = 1000 [static], [constexpr]
```

Definition at line 13 of file [star_spi_driver_node.cpp](#).

Chapter 8

Class Documentation

8.1 star::MessageConverter Class Reference

Message converter for ROS2 <-> Protobuf translation.

```
#include <message_converter.hpp>
```

Static Public Member Functions

- static bool [twist_to_velocity_command](#) (const geometry_msgs::msg::Twist &twist, star::v1::VelocityCommand &command, double wheel_base=0.150, uint32_t sequence=0)
Convert ROS2 Twist message to Protobuf VelocityCommand.
- static bool [string_to_system_status](#) (const std_msgs::msg::String &status_msg, star::v1::SystemStatus &system_status)
Convert ROS2 String message to Protobuf SystemStatus.
- static bool [velocity_command_to_twist](#) (const star::v1::VelocityCommand &command, geometry_msgs::msg::Twist &twist, double wheel_base=0.150)
Convert Protobuf VelocityCommand to ROS2 Twist message.
- static bool [pid_config_to_gains](#) (const star::v1::PidConfig &pid_config, double &kp, double &ki, double &kd)
Convert Protobuf PidConfig to individual gains (for ROS2 service).
- static bool [is_valid_double](#) (double value)
Validate double value for NaN and infinity.
- static double [clamp](#) (double value, double min, double max)
Clamp value to range [min, max].
- static int64_t [ros_time_to_us](#) (const rclcpp::Time &time)
Convert ROS2 timestamp to microseconds since epoch.

8.1.1 Detailed Description

Message converter for ROS2 <-> Protobuf translation.

Provides stateless conversion functions with input validation and unit conversions. All conversions validate for NaN/infinity and clamp values to safe ranges.

Definition at line 30 of file [message_converter.hpp](#).

Generated by Doxygen

8.1.2 Member Function Documentation

8.1.2.1 clamp()

```
double star::MessageConverter::clamp (  
    double value,  
    double min,  
    double max) [inline], [static]
```

Clamp value to range [min, max].

Parameters

<i>value</i>	Value to clamp
<i>min</i>	Minimum allowed value
<i>max</i>	Maximum allowed value

Returns

Clamped value

Definition at line 143 of file [message_converter.hpp](#).

Referenced by [twist_to_velocity_command\(\)](#).

Here is the caller graph for this function:



8.1.2.2 is_valid_double()

```
bool star::MessageConverter::is_valid_double (  
    double value) [inline], [static]
```

Validate double value for NaN and infinity.

Parameters

<i>value</i>	Value to validate
--------------	-------------------

8.1 star::MessageConverter Class Reference

35

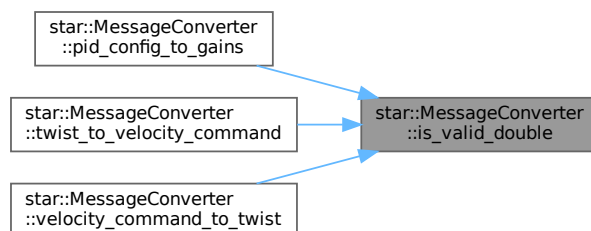
Returns

true if value is finite (not NaN, not infinity)

Definition at line 134 of file [message_converter.hpp](#).

Referenced by [pid_config_to_gains\(\)](#), [twist_to_velocity_command\(\)](#), and [velocity_command_to_twist\(\)](#).

Here is the caller graph for this function:



8.1.2.3 pid_config_to_gains()

```
bool star::MessageConverter::pid_config_to_gains (
    const star::v1::PidConfig & pid_config,
    double & kp,
    double & ki,
    double & kd) [static]
```

Convert Protobuf PidConfig to individual gains (for ROS2 service).

Extracts PID gains from STAR PidConfig protobuf. No unit conversion needed (all values are dimensionless gains).

Parameters

<i>pid_config</i>	Input PidConfig protobuf
<i>kp</i>	Output proportional gain
<i>ki</i>	Output integral gain
<i>kd</i>	Output derivative gain

Returns

true if conversion successful, false if input validation failed

Definition at line 157 of file [message_converter.cpp](#).

References [is_valid_double\(\)](#).

Here is the call graph for this function:

**8.1.2.4 ros_time_to_us()**

```
int64_t star::MessageConverter::ros_time_to_us (
    const rclcpp::Time & time) [static]
```

Convert ROS2 timestamp to microseconds since epoch.

Parameters

<i>time</i>	ROS2 time
-------------	-----------

Returns

Microseconds since epoch

Definition at line 179 of file [message_converter.cpp](#).

8.1.2.5 string_to_system_status()

```
bool star::MessageConverter::string_to_system_status (
    const std_msgs::msg::String & status_msg,
    star::vl::SystemStatus & system_status) [static]
```

Convert ROS2 String message to Protobuf SystemStatus.

Parses JSON-formatted robot status string into SystemStatus protobuf. Expected format: {"mode": "MANUAL", "esp32": true, "lidar": true, ...}

Parameters

<i>status_msg</i>	ROS2 String message (JSON format)
-------------------	-----------------------------------

8.1 star::MessageConverter Class Reference

37

<i>system_status</i>	Output SystemStatus protobuf
----------------------	------------------------------

Returns

true if conversion successful, false if parse failed

Definition at line 73 of file [message_converter.cpp](#).

8.1.2.6 twist_to_velocity_command()

```
bool star::MessageConverter::twist_to_velocity_command (
    const geometry_msgs::msg::Twist & twist,
    star::vl::VelocityCommand & command,
    double wheel_base = 0.150,
    uint32_t sequence = 0) [static]
```

Convert ROS2 Twist message to Protobuf VelocityCommand.

Maps differential drive velocities from ROS2 standard Twist to STAR VelocityCommand. Uses differential drive kinematics: (linear, angular) -> (motor_0, motor_1) wheel velocities. Note: motor_0 = left wheel, motor_1 = right wheel for differential drive.

Conversions:

- Linear velocity (m/s) -> motor_0/motor_1 wheel velocities (m/s)
- Angular velocity (rad/s) -> Wheel velocity differential (m/s)
- Timestamp -> Microseconds since epoch

Safety features:

- NaN/infinity validation (returns false on invalid input)
- Velocity clamping to +/-2.0 m/s (VelocityCommand valid range)

Parameters

<i>twist</i>	ROS2 Twist message (differential drive command)
<i>command</i>	Output VelocityCommand protobuf
<i>wheel_base</i>	Distance between wheels in meters (default: 0.150m)
<i>sequence</i>	Optional sequence number for command ordering

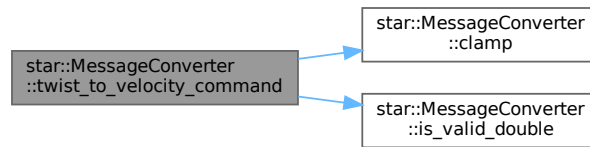
Returns

true if conversion successful, false if input validation failed

Definition at line 19 of file [message_converter.cpp](#).

References [clamp\(\)](#), and [is_valid_double\(\)](#).

Here is the call graph for this function:

**8.1.2.7 velocity_command_to_twist()**

```

bool star::MessageConverter::velocity_command_to_twist (
    const star::v1::VelocityCommand & command,
    geometry_msgs::msg::Twist & twist,
    double wheel_base = 0.150) [static]
  
```

Convert Protobuf VelocityCommand to ROS2 Twist message.

Maps STAR VelocityCommand (motor_0/motor_1 wheel velocities) to ROS2 Twist (linear/angular velocities) using differential drive kinematics. Note: motor_0 = left wheel, motor_1 = right wheel for differential drive.

Conversions:

- motor_0/motor_1 wheel velocities (m/s) -> Linear velocity (m/s)
- Wheel velocity differential (m/s) -> Angular velocity (rad/s)

Kinematics:

- $\text{linear.x} = (\text{motor_0} + \text{motor_1}) / 2$
- $\text{angular.z} = (\text{motor_1} - \text{motor_0}) / \text{wheel_base}$

Parameters

<i>command</i>	Input VelocityCommand protobuf
<i>twist</i>	Output ROS2 Twist message

8.2 star::MessageConverterTest Class Reference

39

<i>wheel_base</i>	Distance between wheels in meters (default: 0.150m)
-------------------	---

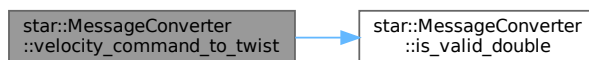
Returns

true if conversion successful, false if input validation failed

Definition at line 112 of file [message_converter.cpp](#).

References [is_valid_double\(\)](#).

Here is the call graph for this function:

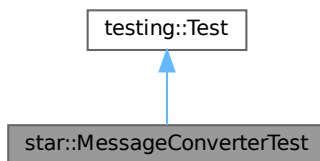


The documentation for this class was generated from the following files:

- [src/star_gateway_bridge/include/star_gateway_bridge/message_converter.hpp](#)
- [src/star_gateway_bridge/src/message_converter.cpp](#)

8.2 star::MessageConverterTest Class Reference

Inheritance diagram for star::MessageConverterTest:



8.3 SpiMessageConverter::Parameters Struct Reference

41

8.2.2.3 WHEEL_BASE_M

```
double star::MessageConverterTest::WHEEL_BASE_M = 0.150 [static], [constexpr], [protected]
```

Definition at line 22 of file [test_message_converter.cpp](#).

The documentation for this class was generated from the following file:

- [src/star_gateway_bridge/test/test_message_converter.cpp](#)

8.3 SpiMessageConverter::Parameters Struct Reference

```
#include <spi_message_converter.hpp>
```

Public Attributes

- double [wheel_base](#)
- double [wheel_radius](#)
- int32_t [ticks_per_rev](#)

8.3.1 Detailed Description

Definition at line 18 of file [spi_message_converter.hpp](#).

8.3.2 Member Data Documentation

8.3.2.1 ticks_per_rev

```
int32_t star_spi_bridge::SpiMessageConverter::Parameters::ticks_per_rev
```

Definition at line 22 of file [spi_message_converter.hpp](#).

8.3.2.2 wheel_base

```
double star_spi_bridge::SpiMessageConverter::Parameters::wheel_base
```

Definition at line 20 of file [spi_message_converter.hpp](#).

8.3.2.3 wheel_radius

```
double star_spi_bridge::SpiMessageConverter::Parameters::wheel_radius
```

Definition at line 21 of file [spi_message_converter.hpp](#).

The documentation for this struct was generated from the following file:

- [src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp](#)

Generated by Doxygen

8.4 `star_spi_bridge::SpiMessageConverter::Parameters` Struct Reference

```
#include <spi_message_converter.hpp>
```

Public Attributes

- double [wheel_base](#)
- double [wheel_radius](#)
- int32_t [ticks_per_rev](#)

8.4.1 Detailed Description

Definition at line 18 of file [spi_message_converter.hpp](#).

8.4.2 Member Data Documentation

8.4.2.1 `ticks_per_rev`

```
int32_t star_spi_bridge::SpiMessageConverter::Parameters::ticks_per_rev
```

Definition at line 22 of file [spi_message_converter.hpp](#).

Referenced by [star_spi_bridge::StarSpiDriverNode::on_configure\(\)](#).

8.4.2.2 `wheel_base`

```
double star_spi_bridge::SpiMessageConverter::Parameters::wheel_base
```

Definition at line 20 of file [spi_message_converter.hpp](#).

Referenced by [star_spi_bridge::StarSpiDriverNode::on_configure\(\)](#).

8.4.2.3 `wheel_radius`

```
double star_spi_bridge::SpiMessageConverter::Parameters::wheel_radius
```

Definition at line 21 of file [spi_message_converter.hpp](#).

Referenced by [star_spi_bridge::StarSpiDriverNode::on_configure\(\)](#).

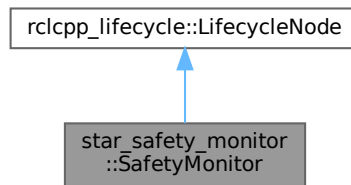
The documentation for this struct was generated from the following file:

- [src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp](#)

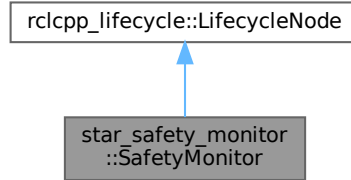
8.5 star_safety_monitor::SafetyMonitor Class Reference

```
#include <safety_monitor.hpp>
```

Inheritance diagram for star_safety_monitor::SafetyMonitor:



Collaboration diagram for star_safety_monitor::SafetyMonitor:



Public Member Functions

- [SafetyMonitor](#) (const rclcpp::NodeOptions &options=rclcpp::NodeOptions())
- [~SafetyMonitor](#) ()
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_configure](#) (const rclcpp_lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_activate](#) (const rclcpp_lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_deactivate](#) (const rclcpp_lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_cleanup](#) (const rclcpp_lifecycle::State &previous_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_shutdown](#) (const rclcpp_lifecycle::State &previous_state) override

8.5.1 Detailed Description

Definition at line 43 of file [safety_monitor.hpp](#).

8.5.2 Constructor & Destructor Documentation

8.5.2.1 SafetyMonitor()

```
star_safety_monitor::SafetyMonitor::SafetyMonitor (  
    const rclcpp::NodeOptions & options = rclcpp::NodeOptions()) [explicit]
```

Definition at line 30 of file [safety_monitor.cpp](#).

8.5.2.2 ~SafetyMonitor()

```
star_safety_monitor::SafetyMonitor::~~SafetyMonitor ()
```

Definition at line 36 of file [safety_monitor.cpp](#).

8.5.3 Member Function Documentation

8.5.3.1 on_activate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↔  
SafetyMonitor::on_activate (  
    const rclcpp_lifecycle::State & previous_state) [override]
```

Definition at line 114 of file [safety_monitor.cpp](#).

8.5.3.2 on_cleanup()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↔  
SafetyMonitor::on_cleanup (  
    const rclcpp_lifecycle::State & previous_state) [override]
```

Definition at line 165 of file [safety_monitor.cpp](#).

8.5.3.3 on_configure()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↔  
SafetyMonitor::on_configure (  
    const rclcpp_lifecycle::State & previous_state) [override]
```

Definition at line 42 of file [safety_monitor.cpp](#).

References [star_safety_monitor::OK](#).

8.6 SafetyMonitorTest Class Reference

45

8.5.3.4 on_deactivate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↔  
SafetyMonitor::on_deactivate (  
    const rclcpp_lifecycle::State & previous_state) [override]
```

Definition at line 140 of file [safety_monitor.cpp](#).

8.5.3.5 on_shutdown()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_safety_monitor::↔  
SafetyMonitor::on_shutdown (  
    const rclcpp_lifecycle::State & previous_state) [override]
```

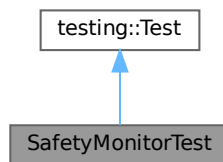
Definition at line 190 of file [safety_monitor.cpp](#).

The documentation for this class was generated from the following files:

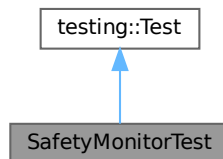
- [src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp](#)
- [src/star_safety_monitor/src/safety_monitor.cpp](#)

8.6 SafetyMonitorTest Class Reference

Inheritance diagram for SafetyMonitorTest:



Collaboration diagram for SafetyMonitorTest:



Protected Member Functions

- void [SetUp](#) () override
- void [TearDown](#) () override

8.6.1 Detailed Description

Definition at line 39 of file [test_safety_monitor.cpp](#).

8.6.2 Member Function Documentation**8.6.2.1 SetUp()**

```
void SafetyMonitorTest::SetUp () [inline], [override], [protected]
```

Definition at line 42 of file [test_safety_monitor.cpp](#).

8.6.2.2 TearDown()

```
void SafetyMonitorTest::TearDown () [inline], [override], [protected]
```

Definition at line 49 of file [test_safety_monitor.cpp](#).

The documentation for this class was generated from the following file:

- [src/star_safety_monitor/test/test_safety_monitor.cpp](#)

8.7 spi_ioc_transfer Struct Reference**Public Attributes**

- uint64_t [tx_buf_](#)
- uint64_t [rx_buf_](#)
- uint32_t [len_](#)
- uint32_t [speed_hz_](#)
- uint16_t [delay_usecs_](#)
- uint8_t [bits_per_word_](#)
- uint8_t [cs_change_](#)
- uint32_t [pad_](#)

8.7.1 Detailed Description

Definition at line 28 of file [spi_driver.cpp](#).

8.7 spi_ioc_transfer Struct Reference**47****8.7.2 Member Data Documentation****8.7.2.1 bits_per_word_**`uint8_t spi_ioc_transfer::bits_per_word_`Definition at line 35 of file [spi_driver.cpp](#).**8.7.2.2 cs_change_**`uint8_t spi_ioc_transfer::cs_change_`Definition at line 36 of file [spi_driver.cpp](#).**8.7.2.3 delay_usecs_**`uint16_t spi_ioc_transfer::delay_usecs_`Definition at line 34 of file [spi_driver.cpp](#).**8.7.2.4 len_**`uint32_t spi_ioc_transfer::len_`Definition at line 32 of file [spi_driver.cpp](#).**8.7.2.5 pad_**`uint32_t spi_ioc_transfer::pad_`Definition at line 37 of file [spi_driver.cpp](#).**8.7.2.6 rx_buf_**`uint64_t spi_ioc_transfer::rx_buf_`Definition at line 31 of file [spi_driver.cpp](#).**8.7.2.7 speed_hz_**`uint32_t spi_ioc_transfer::speed_hz_`Definition at line 33 of file [spi_driver.cpp](#).

Generated by Doxygen

8.7.2.8 tx_buf_

```
uint64_t spi_ioc_transfer::tx_buf_
```

Definition at line 30 of file [spi_driver.cpp](#).

The documentation for this struct was generated from the following file:

- [src/star_spi_bridge/src/spi_driver.cpp](#)

8.8 SpiDriver Class Reference

SPI driver for framed communication with the RX72N peripheral MCU.

```
#include <spi_driver.hpp>
```

Public Member Functions

- [SpiDriver](#) (const std::string &device_path, uint32_t speed_hz=10000000)
Construct a new [SpiDriver](#).
- virtual [~SpiDriver](#) ()
Destructor – calls [close_device\(\)](#) if the device is still open.
- bool [initialize](#) ()
Open and configure the SPI device.
- void [close_device](#) ()
Close the SPI device file descriptor.
- bool [transfer](#) (const std::vector< uint8_t > &tx_data, std::vector< uint8_t > &rx_data)
Perform a full-duplex SPI transfer.

Static Public Member Functions

- static void [encode_frame](#) (uint16_t seq, [FrameType](#) type, uint8_t flags, const std::vector< uint8_t > &payload, std::vector< uint8_t > &out_frame)
Encode an application payload into the STAR wire format.
- static bool [decode_frame](#) (const std::vector< uint8_t > &frame, uint16_t &seq, [FrameType](#) &type, uint8_t &flags, std::vector< uint8_t > &payload)
Decode a STAR wire frame into its constituent fields.
- static uint32_t [calculate_crc32](#) (const std::vector< uint8_t > &data)
Compute the IEEE 802.3 CRC-32 of the given byte vector.

Static Public Attributes

- static constexpr size_t [k_header_size](#) = 8
SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.
- static constexpr size_t [k_crc_size](#) = 4
CRC-32 trailer length in bytes.
- static constexpr size_t [k_max_payload_size](#) = 1024
Maximum application payload per frame (bytes).
- static constexpr uint16_t [k_sync_word](#) = 0x55AA
SYNC word transmitted in every frame header.

8.8 SpiDriver Class Reference**49****8.8.1 Detailed Description**

SPI driver for framed communication with the RX72N peripheral MCU.

Implements the STAR binary framing protocol over Linux spidev. The wire format is:

SYNC(2, LE) + SEQ(2, LE) + LEN(2, LE) + TYPE(1) + FLAGS(1)

- PAYLOAD(0-1024) + CRC-32 (IEEE 802.3, 4 B, LE)

The Raspberry Pi 5 acts as SPI controller at 10 MHz, Mode 0 (CPOL=0, CPHA=0).

Thread Safety

- CRC-32 table initialisation is thread-safe (`std::call_once`).
- `encode_frame` / `decode_frame` / `calculate_crc32` are stateless and safe to call from any thread.
- `initialize`, `close_device`, and `transfer` share `spi_fd_` and must not be called concurrently on the same instance.

Invariant

After a successful `initialize()`, `spi_fd_ >= 0`.

See also

`star_spi_bridge::FrameType` Supported frame types

Since

Version 1.0.0

Definition at line 58 of file `spi_driver.hpp`.

8.8.2 Constructor & Destructor Documentation**8.8.2.1 SpiDriver()**

```
star_spi_bridge::SpiDriver::SpiDriver (
    const std::string & device_path,
    uint32_t speed_hz = 10000000) [explicit]
```

Construct a new `SpiDriver`.

Parameters

in	<code>device_path</code>	Path to the Linux spidev node (e.g. <code>"/dev/spidev0.0"</code>).
----	--------------------------	--

in	<i>speed_hz</i>	SPI clock frequency in Hz (default 10 MHz).
----	-----------------	---

Precondition

device_path is a non-empty, null-terminated string.

Postcondition

CRC-32 lookup table is initialised (thread-safe, once).

Definition at line 102 of file [spi_driver.cpp](#).

8.8.2.2 ~SpiDriver()

```
star_spi_bridge::SpiDriver::~SpiDriver () [virtual]
```

Destructor – calls [close_device\(\)](#) if the device is still open.

Definition at line 108 of file [spi_driver.cpp](#).

8.8.3 Member Function Documentation**8.8.3.1 calculate_crc32()**

```
uint32_t star_spi_bridge::SpiDriver::calculate_crc32 (  
    const std::vector< uint8_t > & data) [static]
```

Compute the IEEE 802.3 CRC-32 of the given byte vector.

Parameters

in	<i>data</i>	Input bytes.
----	-------------	--------------

Returns

CRC-32 value.

Precondition

CRC-32 table has been initialised (guaranteed by the constructor).

Postcondition

Return value matches `crc32(0, data, len)` from `zlib`.

8.8 SpiDriver Class Reference

51

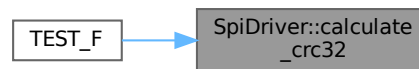
Note

Public for unit-test access.

Definition at line 311 of file [spi_driver.cpp](#).

Referenced by [TEST_F\(\)](#).

Here is the caller graph for this function:



8.8.3.2 close_device()

```
void star_spi_bridge::SpiDriver::close_device ()
```

Close the SPI device file descriptor.

Postcondition

```
spi_fd_ == -1.
```

Definition at line 153 of file [spi_driver.cpp](#).

8.8.3.3 decode_frame()

```
bool star_spi_bridge::SpiDriver::decode_frame (
    const std::vector< uint8_t > & frame,
    uint16_t & seq,
    FrameType & type,
    uint8_t & flags,
    std::vector< uint8_t > & payload) [static]
```

Decode a STAR wire frame into its constituent fields.

Validates SYNC word, frame length, and CRC-32. On success the sequence number, type, flags, and payload are written to the output parameters.

Parameters

in	<i>frame</i>	Raw wire bytes to parse.
out	<i>seq</i>	Decoded sequence number.

out	<i>type</i>	Decoded frame type.
out	<i>flags</i>	Decoded flags byte.
out	<i>payload</i>	Extracted payload bytes.

Returns

true Frame valid (SYNC, length, and CRC all match).
false Frame invalid or corrupted.

Precondition

`frame.size() >= k_header_size + k_crc_size` (12 bytes minimum).

Postcondition

On success, all output parameters are populated.

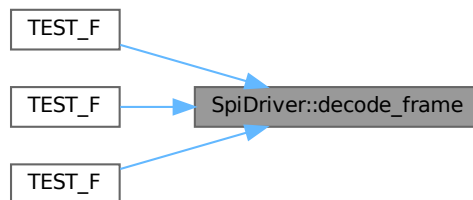
See also

[encode_frame](#) Inverse operation.

Definition at line 243 of file [spi_driver.cpp](#).

Referenced by [TEST_F\(\)](#), [TEST_F\(\)](#), and [TEST_F\(\)](#).

Here is the caller graph for this function:



8.8 SpiDriver Class Reference**53****8.8.3.4 encode_frame()**

```
void star_spi_bridge::SpiDriver::encode_frame (  
    uint16_t seq,  
    FrameType type,  
    uint8_t flags,  
    const std::vector< uint8_t > & payload,  
    std::vector< uint8_t > & out_frame) [static]
```

Encode an application payload into the STAR wire format.

Builds the header (little-endian SYNC, SEQ, LEN, TYPE, FLAGS), appends the payload, then computes and appends the CRC-32. The output buffer is cleared and filled in place for reuse.

Parameters

in	<i>seq</i>	Sequence number (0-65535).
----	------------	----------------------------

in	<i>type</i>	Frame type (see FrameType enum).
in	<i>flags</i>	Control flags byte.
in	<i>payload</i>	Application data (0- <code>k_max_payload_size</code> bytes).
out	<i>out_frame</i>	Encoded wire bytes (header + payload + CRC).

Precondition

```
payload.size() <= k_max_payload_size.
```

Postcondition

```
out_frame.size() == k_header_size + payload.size() + k_crc_size.
```

Exceptions

<code>std::invalid_argument</code>	If <code>payload.size() > k_max_payload_size</code> .
------------------------------------	--

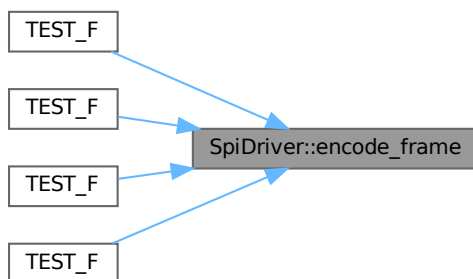
See also

[decode_frame](#) Inverse operation.

Definition at line 193 of file [spi_driver.cpp](#).

Referenced by [TEST_F\(\)](#), [TEST_F\(\)](#), [TEST_F\(\)](#), and [TEST_F\(\)](#).

Here is the caller graph for this function:



8.8 SpiDriver Class Reference

55

8.8.3.5 initialize()

```
bool star_spi_bridge::SpiDriver::initialize ()
```

Open and configure the SPI device.

Opens the spidev file descriptor, sets Mode 0, 8 bits per word, and the requested clock speed via ioctl calls.

Returns

true Device opened and configured successfully.
false Open or ioctl failed (error logged via RCLCPP_ERROR).

Precondition

device_path_ refers to a valid spidev node.

Postcondition

On success, spi_fd_ >= 0 and the device is ready for [transfer](#).

Note

Only supported on Linux; returns false on other platforms.

Definition at line 113 of file [spi_driver.cpp](#).

8.8.3.6 transfer()

```
bool star_spi_bridge::SpiDriver::transfer (
    const std::vector< uint8_t > & tx_data,
    std::vector< uint8_t > & rx_data)
```

Perform a full-duplex SPI transfer.

Parameters

in	<i>tx_data</i>	Data to transmit.
out	<i>rx_data</i>	Buffer filled with received data (resized to match tx_data).

Returns

true Transfer completed successfully.
false Device not open or ioctl failed.

Precondition

[initialize\(\)](#) returned true.

Postcondition

rx_data.size() == tx_data.size().

Note

Only supported on Linux; returns false on other platforms.

Definition at line 161 of file [spi_driver.cpp](#).

Generated by Doxygen

8.8.4 Member Data Documentation

8.8.4.1 k_crc_size

```
size_t star_spi_bridge::SpiDriver::k_crc_size = 4 [static], [constexpr]
```

CRC-32 trailer length in bytes.

Definition at line 65 of file [spi_driver.hpp](#).

8.8.4.2 k_header_size

```
size_t star_spi_bridge::SpiDriver::k_header_size = 8 [static], [constexpr]
```

SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.

Definition at line 62 of file [spi_driver.hpp](#).

8.8.4.3 k_max_payload_size

```
size_t star_spi_bridge::SpiDriver::k_max_payload_size = 1024 [static], [constexpr]
```

Maximum application payload per frame (bytes).

Definition at line 68 of file [spi_driver.hpp](#).

Referenced by [TEST_F\(\)](#).

8.8.4.4 k_sync_word

```
uint16_t star_spi_bridge::SpiDriver::k_sync_word = 0x55AA [static], [constexpr]
```

SYNC word transmitted in every frame header.

Definition at line 71 of file [spi_driver.hpp](#).

The documentation for this class was generated from the following files:

- [src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp](#)
- [src/star_spi_bridge/src/spi_driver.cpp](#)

8.9 star_spi_bridge::SpiDriver Class Reference

SPI driver for framed communication with the RX72N peripheral MCU.

```
#include <spi_driver.hpp>
```

8.9 star_spi_bridge::SpiDriver Class Reference

57

Public Member Functions

- [SpiDriver](#) (const std::string &device_path, uint32_t speed_hz=10000000)
Construct a new SpiDriver.
- virtual [~SpiDriver](#) ()
Destructor – calls [close_device\(\)](#) if the device is still open.
- bool [initialize](#) ()
Open and configure the SPI device.
- void [close_device](#) ()
Close the SPI device file descriptor.
- bool [transfer](#) (const std::vector< uint8_t > &tx_data, std::vector< uint8_t > &rx_data)
Perform a full-duplex SPI transfer.

Static Public Member Functions

- static void [encode_frame](#) (uint16_t seq, [FrameType](#) type, uint8_t flags, const std::vector< uint8_t > &payload, std::vector< uint8_t > &out_frame)
Encode an application payload into the STAR wire format.
- static bool [decode_frame](#) (const std::vector< uint8_t > &frame, uint16_t &seq, [FrameType](#) &type, uint8_t &flags, std::vector< uint8_t > &payload)
Decode a STAR wire frame into its constituent fields.
- static uint32_t [calculate_crc32](#) (const std::vector< uint8_t > &data)
Compute the IEEE 802.3 CRC-32 of the given byte vector.

Static Public Attributes

- static constexpr size_t [k_header_size](#) = 8
SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.
- static constexpr size_t [k_crc_size](#) = 4
CRC-32 trailer length in bytes.
- static constexpr size_t [k_max_payload_size](#) = 1024
Maximum application payload per frame (bytes).
- static constexpr uint16_t [k_sync_word](#) = 0x55AA
SYNC word transmitted in every frame header.

8.9.1 Detailed Description

SPI driver for framed communication with the RX72N peripheral MCU.

Implements the STAR binary framing protocol over Linux spidev. The wire format is:

SYNC(2, LE) + SEQ(2, LE) + LEN(2, LE) + TYPE(1) + FLAGS(1)

- PAYLOAD(0-1024) + CRC-32 (IEEE 802.3, 4 B, LE)

The Raspberry Pi 5 acts as SPI controller at 10 MHz, Mode 0 (CPOL=0, CPHA=0).

Thread Safety

- CRC-32 table initialisation is thread-safe (`std::call_once`).
- `encode_frame` / `decode_frame` / `calculate_crc32` are stateless and safe to call from any thread.
- `initialize`, `close_device`, and `transfer` share `spi_fd_` and must not be called concurrently on the same instance.

Invariant

After a successful `initialize()`, `spi_fd_ >= 0`.

See also

[star_spi_bridge::FrameType](#) Supported frame types

Since

Version 1.0.0

Definition at line 58 of file [spi_driver.hpp](#).

8.9.2 Constructor & Destructor Documentation

8.9.2.1 SpiDriver()

```
star_spi_bridge::SpiDriver::SpiDriver (
    const std::string & device_path,
    uint32_t speed_hz = 10000000) [explicit]
```

Construct a new [SpiDriver](#).

Parameters

in	<i>device_path</i>	Path to the Linux spidev node (e.g. <code>"/dev/spidev0.0"</code>).
in	<i>speed_hz</i>	SPI clock frequency in Hz (default 10 MHz).

Precondition

`device_path` is a non-empty, null-terminated string.

Postcondition

CRC-32 lookup table is initialised (thread-safe, once).

Definition at line 102 of file [spi_driver.cpp](#).

8.9 star_spi_bridge::SpiDriver Class Reference**59****8.9.2.2 ~SpiDriver()**

```
star_spi_bridge::SpiDriver::~SpiDriver () [virtual]
```

Destructor – calls [close_device\(\)](#) if the device is still open.

Definition at line 108 of file [spi_driver.cpp](#).

References [close_device\(\)](#).

Here is the call graph for this function:

**8.9.3 Member Function Documentation****8.9.3.1 calculate_crc32()**

```
uint32_t star_spi_bridge::SpiDriver::calculate_crc32 (  
    const std::vector< uint8_t > & data) [static]
```

Compute the IEEE 802.3 CRC-32 of the given byte vector.

Parameters

in	<i>data</i>	Input bytes.
----	-------------	--------------

Returns

CRC-32 value.

Precondition

CRC-32 table has been initialised (guaranteed by the constructor).

Postcondition

Return value matches `crc32(0, data, len)` from `zlib`.

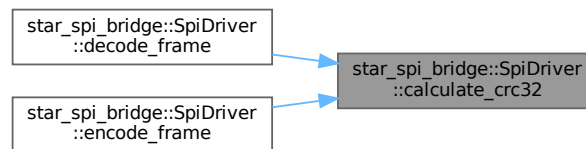
Note

Public for unit-test access.

Definition at line 311 of file [spi_driver.cpp](#).

Referenced by [decode_frame\(\)](#), and [encode_frame\(\)](#).

Here is the caller graph for this function:

**8.9.3.2 close_device()**

```
void star_spi_bridge::SpiDriver::close_device ()
```

Close the SPI device file descriptor.

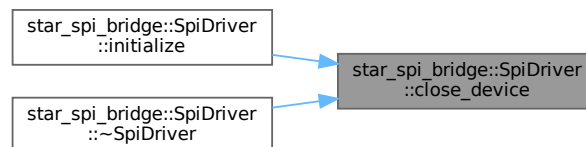
Postcondition

```
spi_fd_ == -1.
```

Definition at line 153 of file [spi_driver.cpp](#).

Referenced by [initialize\(\)](#), and [~SpiDriver\(\)](#).

Here is the caller graph for this function:



8.9 star_spi_bridge::SpiDriver Class Reference

61

8.9.3.3 decode_frame()

```
bool star_spi_bridge::SpiDriver::decode_frame (
    const std::vector< uint8_t > & frame,
    uint16_t & seq,
    FrameType & type,
    uint8_t & flags,
    std::vector< uint8_t > & payload) [static]
```

Decode a STAR wire frame into its constituent fields.

Validates SYNC word, frame length, and CRC-32. On success the sequence number, type, flags, and payload are written to the output parameters.

Parameters

in	<i>frame</i>	Raw wire bytes to parse.
out	<i>seq</i>	Decoded sequence number.
out	<i>type</i>	Decoded frame type.
out	<i>flags</i>	Decoded flags byte.
out	<i>payload</i>	Extracted payload bytes.

Returns

true Frame valid (SYNC, length, and CRC all match).

false Frame invalid or corrupted.

Precondition

`frame.size() >= k_header_size + k_crc_size` (12 bytes minimum).

Postcondition

On success, all output parameters are populated.

See also

[encode_frame](#) Inverse operation.

Definition at line 243 of file [spi_driver.cpp](#).

References [calculate_crc32\(\)](#), [k_crc_size](#), [k_header_size](#), and [k_sync_word](#).

Here is the call graph for this function:



8.9.3.4 encode_frame()

```
void star_spi_bridge::SpiDriver::encode_frame (
    uint16_t seq,
    FrameType type,
    uint8_t flags,
    const std::vector< uint8_t > & payload,
    std::vector< uint8_t > & out_frame) [static]
```

Encode an application payload into the STAR wire format.

Builds the header (little-endian SYNC, SEQ, LEN, TYPE, FLAGS), appends the payload, then computes and appends the CRC-32. The output buffer is cleared and filled in place for reuse.

Parameters

in	<i>seq</i>	Sequence number (0-65535).
in	<i>type</i>	Frame type (see FrameType enum).
in	<i>flags</i>	Control flags byte.
in	<i>payload</i>	Application data (0-k_max_payload_size bytes).
out	<i>out_frame</i>	Encoded wire bytes (header + payload + CRC).

Precondition

```
payload.size() <= k_max_payload_size.
```

Postcondition

```
out_frame.size() == k_header_size + payload.size() + k_crc_size.
```

Exceptions

<i>std::invalid_argument</i>	If <code>payload.size() > k_max_payload_size.</code>
------------------------------	---

See also

[decode_frame](#) Inverse operation.

Definition at line 193 of file [spi_driver.cpp](#).

References [calculate_crc32\(\)](#), [k_crc_size](#), [k_header_size](#), [k_max_payload_size](#), and [k_sync_word](#).

Here is the call graph for this function:



8.9 star_spi_bridge::SpiDriver Class Reference

63

8.9.3.5 initialize()

```
bool star_spi_bridge::SpiDriver::initialize ()
```

Open and configure the SPI device.

Opens the spidev file descriptor, sets Mode 0, 8 bits per word, and the requested clock speed via ioctl calls.

Returns

true Device opened and configured successfully.
false Open or ioctl failed (error logged via RCLCPP_ERROR).

Precondition

device_path_ refers to a valid spidev node.

Postcondition

On success, spi_fd_ >= 0 and the device is ready for [transfer](#).

Note

Only supported on Linux; returns false on other platforms.

Definition at line 113 of file [spi_driver.cpp](#).

References [close_device\(\)](#), [SPI_IOC_WR_BITS_PER_WORD](#), [SPI_IOC_WR_MAX_SPEED_HZ](#), [SPI_IOC_WR_MODE](#), and [SPI_MODE_0](#).

Here is the call graph for this function:



8.9.3.6 transfer()

```
bool star_spi_bridge::SpiDriver::transfer (  
    const std::vector< uint8_t > & tx_data,  
    std::vector< uint8_t > & rx_data)
```

Perform a full-duplex SPI transfer.

Parameters

in	tx_data	Data to transmit.
----	---------	-------------------

Generated by Doxygen

out	rx_data	Buffer filled with received data (resized to match tx_data).
-----	---------	--

Returns

true Transfer completed successfully.
false Device not open or ioctl failed.

Precondition

`initialize()` returned true.

Postcondition

`rx_data.size() == tx_data.size()`.

Note

Only supported on Linux; returns false on other platforms.

Definition at line 161 of file [spi_driver.cpp](#).

References [SPI_IOC_MESSAGE](#).

8.9.4 Member Data Documentation**8.9.4.1 k_crc_size**

```
size_t star_spi_bridge::SpiDriver::k_crc_size = 4 [static], [constexpr]
```

CRC-32 trailer length in bytes.

Definition at line 65 of file [spi_driver.hpp](#).

Referenced by [decode_frame\(\)](#), and [encode_frame\(\)](#).

8.9.4.2 k_header_size

```
size_t star_spi_bridge::SpiDriver::k_header_size = 8 [static], [constexpr]
```

SYNC + SEQ + LEN + TYPE + FLAGS = 8 bytes.

Definition at line 62 of file [spi_driver.hpp](#).

Referenced by [decode_frame\(\)](#), and [encode_frame\(\)](#).

8.10 SpiDriverTest Class Reference

65

8.9.4.3 k_max_payload_size

```
size_t star_spi_bridge::SpiDriver::k_max_payload_size = 1024 [static], [constexpr]
```

Maximum application payload per frame (bytes).

Definition at line 68 of file [spi_driver.hpp](#).

Referenced by [encode_frame\(\)](#).

8.9.4.4 k_sync_word

```
uint16_t star_spi_bridge::SpiDriver::k_sync_word = 0x55AA [static], [constexpr]
```

SYNC word transmitted in every frame header.

Definition at line 71 of file [spi_driver.hpp](#).

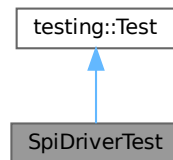
Referenced by [decode_frame\(\)](#), and [encode_frame\(\)](#).

The documentation for this class was generated from the following files:

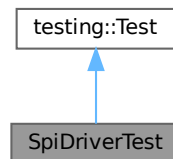
- [src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp](#)
- [src/star_spi_bridge/src/spi_driver.cpp](#)

8.10 SpiDriverTest Class Reference

Inheritance diagram for SpiDriverTest:



Collaboration diagram for SpiDriverTest:



Protected Member Functions

- void [SetUp](#) () override

8.10.1 Detailed Description

Definition at line 10 of file [test_spi_driver.cpp](#).

8.10.2 Member Function Documentation

8.10.2.1 SetUp()

```
void SpiDriverTest::SetUp () [inline], [override], [protected]
```

Definition at line 13 of file [test_spi_driver.cpp](#).

The documentation for this class was generated from the following file:

- [src/star_spi_bridge/test/test_spi_driver.cpp](#)

8.11 SpiMessageConverter Class Reference

```
#include <spi_message_converter.hpp>
```

Classes

- struct [Parameters](#)

Public Member Functions

- [SpiMessageConverter](#) (const [Parameters](#) ¶ms)
- bool [twist_to_velocity_command](#) (const geometry_msgs::msg::Twist &twist, star::v1::VelocityCommand &command)
- void [telemetry_to_odometry](#) (const star::v1::TelemetryData &telemetry, nav_msgs::msg::Odometry &odom)
- void [telemetry_to_joint_state](#) (const star::v1::TelemetryData &telemetry, sensor_msgs::msg::JointState &joint_state)

8.11.1 Detailed Description

Definition at line 16 of file [spi_message_converter.hpp](#).

8.12 star_spi_bridge::SpiMessageConverter Class Reference**67****8.11.2 Constructor & Destructor Documentation****8.11.2.1 SpiMessageConverter()**

```
star_spi_bridge::SpiMessageConverter::SpiMessageConverter (
    const Parameters & params) [explicit]
```

Definition at line 14 of file [spi_message_converter.cpp](#).

8.11.3 Member Function Documentation**8.11.3.1 telemetry_to_joint_state()**

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_joint_state (
    const star::v1::TelemetryData & telemetry,
    sensor_msgs::msg::JointState & joint_state)
```

Definition at line 158 of file [spi_message_converter.cpp](#).

8.11.3.2 telemetry_to_odometry()

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_odometry (
    const star::v1::TelemetryData & telemetry,
    nav_msgs::msg::Odometry & odom)
```

Definition at line 49 of file [spi_message_converter.cpp](#).

8.11.3.3 twist_to_velocity_command()

```
bool star_spi_bridge::SpiMessageConverter::twist_to_velocity_command (
    const geometry_msgs::msg::Twist & twist,
    star::v1::VelocityCommand & command)
```

Definition at line 17 of file [spi_message_converter.cpp](#).

The documentation for this class was generated from the following files:

- [src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp](#)
- [src/star_spi_bridge/src/spi_message_converter.cpp](#)

8.12 star_spi_bridge::SpiMessageConverter Class Reference

```
#include <spi_message_converter.hpp>
```

Classes

- struct [Parameters](#)

Generated by Doxygen

Public Member Functions

- [SpiMessageConverter](#) (const [Parameters](#) ¶ms)
- bool [twist_to_velocity_command](#) (const geometry_msgs::msg::Twist &twist, star::v1::VelocityCommand &command)
- void [telemetry_to_odometry](#) (const star::v1::TelemetryData &telemetry, nav_msgs::msg::Odometry &odom)
- void [telemetry_to_joint_state](#) (const star::v1::TelemetryData &telemetry, sensor_msgs::msg::JointState &joint_state)

8.12.1 Detailed Description

Definition at line 16 of file [spi_message_converter.hpp](#).

8.12.2 Constructor & Destructor Documentation**8.12.2.1 SpiMessageConverter()**

```
star_spi_bridge::SpiMessageConverter::SpiMessageConverter (
    const Parameters & params) [explicit]
```

Definition at line 14 of file [spi_message_converter.cpp](#).

8.12.3 Member Function Documentation**8.12.3.1 telemetry_to_joint_state()**

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_joint_state (
    const star::v1::TelemetryData & telemetry,
    sensor_msgs::msg::JointState & joint_state)
```

Definition at line 158 of file [spi_message_converter.cpp](#).

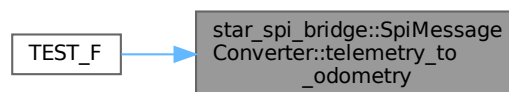
8.12.3.2 telemetry_to_odometry()

```
void star_spi_bridge::SpiMessageConverter::telemetry_to_odometry (
    const star::v1::TelemetryData & telemetry,
    nav_msgs::msg::Odometry & odom)
```

Definition at line 49 of file [spi_message_converter.cpp](#).

Referenced by [TEST_F\(\)](#).

Here is the caller graph for this function:



8.13 SpiMessageConverterTest Class Reference

69

8.12.3.3 twist_to_velocity_command()

```
bool star_spi_bridge::SpiMessageConverter::twist_to_velocity_command (
    const geometry_msgs::msg::Twist & twist,
    star::vl::VelocityCommand & command)
```

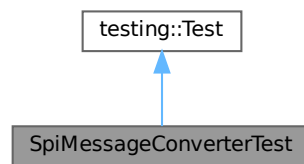
Definition at line 17 of file [spi_message_converter.cpp](#).

The documentation for this class was generated from the following files:

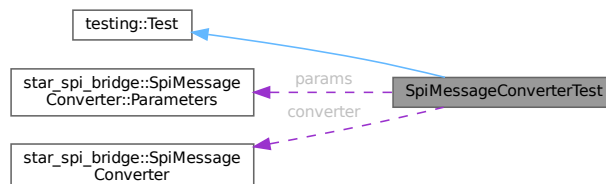
- [src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp](#)
- [src/star_spi_bridge/src/spi_message_converter.cpp](#)

8.13 SpiMessageConverterTest Class Reference

Inheritance diagram for SpiMessageConverterTest:



Collaboration diagram for SpiMessageConverterTest:



Protected Attributes

- [SpiMessageConverter::Parameters](#) `params` {0.150, 0.0325, 11599}
- [SpiMessageConverter](#) `converter` {`params`}

Generated by Doxygen

8.13.1 Detailed Description

Definition at line 11 of file [test_spi_message_converter.cpp](#).

8.13.2 Member Data Documentation

8.13.2.1 converter

[SpiMessageConverter](#) [SpiMessageConverterTest::converter](#) {[params](#)} [protected]

Definition at line 15 of file [test_spi_message_converter.cpp](#).

8.13.2.2 params

[SpiMessageConverter::Parameters](#) [SpiMessageConverterTest::params](#) {0.150, 0.0325, 11599} [protected]

Definition at line 14 of file [test_spi_message_converter.cpp](#).

The documentation for this class was generated from the following file:

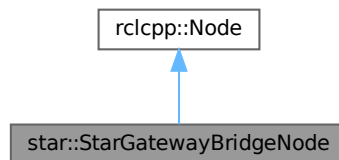
- [src/star_spi_bridge/test/test_spi_message_converter.cpp](#)

8.14 star::StarGatewayBridgeNode Class Reference

ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.

```
#include <star_gateway_bridge_node.hpp>
```

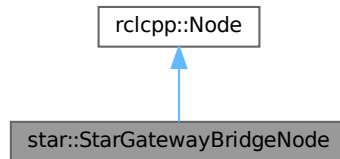
Inheritance diagram for star::StarGatewayBridgeNode:



8.14 star::StarGatewayBridgeNode Class Reference

71

Collaboration diagram for star::StarGatewayBridgeNode:



Public Member Functions

- [StarGatewayBridgeNode](#) (const rclcpp::NodeOptions &options=rclcpp::NodeOptions())
Construct [StarGatewayBridgeNode](#) with ROS2 and gRPC initialization.
- [~StarGatewayBridgeNode](#) ()
Destructor - sends stop command and shuts down gracefully.

8.14.1 Detailed Description

ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.

Architecture: ROS2 Topics/Services <-> [StarGatewayBridgeNode](#) <-> gRPC <-> Go Gateway <-> UI

Responsibilities:

1. Subscribe to /robot_status ROS2 topic
2. Forward critical telemetry to Gateway via gRPC for UI display
3. Poll Gateway for teleop commands from UI
4. Publish teleop commands to /teleop/cmd_vel ROS2 topic
5. Expose /set_pid_gains ROS2 service for PID tuning from UI

Safety Features:

- Teleop command staleness check (500ms timeout -> zero velocity)
- Connection watchdog (5s interval, automatic reconnection)
- Non-blocking gRPC calls (100ms deadline)
- Input validation (NaN, infinity checks via [MessageConverter](#))
- Graceful shutdown (stop command on node destruction)

gRPC Service Definition (to be implemented in Phase 4): service GatewayService { rpc ForwardTelemetry(← TelemetryRequest) returns (TelemetryResponse); rpc GetTeleopCommand(TeleopRequest) returns (Teleop← Response); rpc SetPIDGains(PIDGainsRequest) returns (PIDGainsResponse); }

Definition at line 63 of file [star_gateway_bridge_node.hpp](#).

Generated by Doxygen

8.14.2 Constructor & Destructor Documentation

8.14.2.1 StarGatewayBridgeNode()

```
star::StarGatewayBridgeNode::StarGatewayBridgeNode (
    const rclcpp::NodeOptions & options = rclcpp::NodeOptions()) [explicit]
```

Construct [StarGatewayBridgeNode](#) with ROS2 and gRPC initialization.

Declares parameters:

- gateway_address: Gateway gRPC server address (default: "localhost:50051")
- telemetry_rate_hz: Telemetry forwarding rate (default: 10.0 Hz)
- teleop_rate_hz: Teleop polling rate (default: 50.0 Hz)
- watchdog_timeout_sec: Connection health check interval (default: 5.0s)
- teleop_timeout_ms: Teleop command staleness timeout (default: 500ms)
- grpc_deadline_ms: gRPC call deadline (default: 100ms)
- wheel_base: Distance between wheels in meters (default: 0.150m)

Parameters

<i>options</i>	ROS2 node options for component configuration
----------------	---

Definition at line 18 of file [star_gateway_bridge_node.cpp](#).

8.14.2.2 ~StarGatewayBridgeNode()

```
star::StarGatewayBridgeNode::~~StarGatewayBridgeNode ()
```

Destructor - sends stop command and shuts down gracefully.

Definition at line 66 of file [star_gateway_bridge_node.cpp](#).

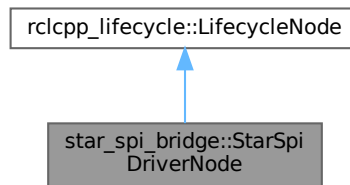
The documentation for this class was generated from the following files:

- [src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp](#)
- [src/star_gateway_bridge/src/star_gateway_bridge_node.cpp](#)

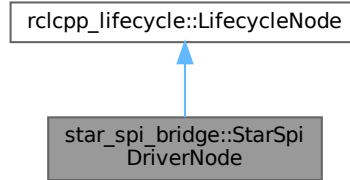
8.15 star_spi_bridge::StarSpiDriverNode Class Reference

```
#include <star_spi_driver_node.hpp>
```

Inheritance diagram for star_spi_bridge::StarSpiDriverNode:



Collaboration diagram for star_spi_bridge::StarSpiDriverNode:



Public Member Functions

- [StarSpiDriverNode](#) (const rclcpp::NodeOptions &options=rclcpp::NodeOptions())
- [~StarSpiDriverNode](#) () override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_configure](#) (const rclcpp_lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_activate](#) (const rclcpp_lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_deactivate](#) (const rclcpp_lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_cleanup](#) (const rclcpp_lifecycle::State &prev_state) override
- rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn [on_shutdown](#) (const rclcpp_lifecycle::State &prev_state) override

8.15.1 Detailed Description

Definition at line 22 of file [star_spi_driver_node.hpp](#).

8.15.2 Constructor & Destructor Documentation

8.15.2.1 StarSpiDriverNode()

```
star_spi_bridge::StarSpiDriverNode::StarSpiDriverNode (
    const rclcpp::NodeOptions & options = rclcpp::NodeOptions()) [explicit]
```

Definition at line 21 of file [star_spi_driver_node.cpp](#).

8.15.2.2 ~StarSpiDriverNode()

```
star_spi_bridge::StarSpiDriverNode::~~StarSpiDriverNode () [override]
```

Definition at line 33 of file [star_spi_driver_node.cpp](#).

8.15.3 Member Function Documentation

8.15.3.1 on_activate()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↔
StarSpiDriverNode::on_activate (
    const rclcpp_lifecycle::State & prev_state) [override]
```

Definition at line 86 of file [star_spi_driver_node.cpp](#).

8.15.3.2 on_cleanup()

```
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↔
StarSpiDriverNode::on_cleanup (
    const rclcpp_lifecycle::State & prev_state) [override]
```

Definition at line 129 of file [star_spi_driver_node.cpp](#).

Referenced by [on_shutdown\(\)](#).

Here is the caller graph for this function:



8.15 star_spi_bridge::StarSpiDriverNode Class Reference**75****8.15.3.3 on_configure()**

```

rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↔
StarSpiDriverNode::on_configure (
    const rclcpp_lifecycle::State & prev_state) [override]

```

Definition at line 42 of file [star_spi_driver_node.cpp](#).

References [star_spi_bridge::SpiMessageConverter::Parameters::ticks_per_rev](#), [star_spi_bridge::SpiMessageConverter::Parameters:](#) and [star_spi_bridge::SpiMessageConverter::Parameters::wheel_radius](#).

8.15.3.4 on_deactivate()

```

rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↔
StarSpiDriverNode::on_deactivate (
    const rclcpp_lifecycle::State & prev_state) [override]

```

Definition at line 103 of file [star_spi_driver_node.cpp](#).

8.15.3.5 on_shutdown()

```

rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn star_spi_bridge::↔
StarSpiDriverNode::on_shutdown (
    const rclcpp_lifecycle::State & prev_state) [override]

```

Definition at line 147 of file [star_spi_driver_node.cpp](#).

References [on_cleanup\(\)](#).

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_node.hpp](#)
- [src/star_spi_bridge/src/star_spi_driver_node.cpp](#)

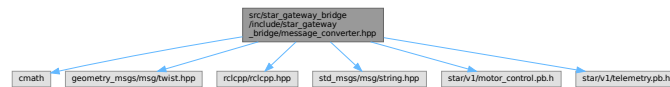
Chapter 9

File Documentation

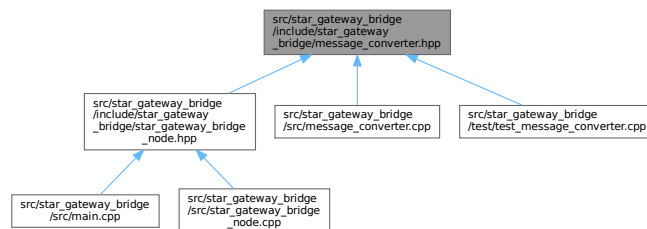
9.1 src/star_gateway_bridge/include/star_gateway_bridge/message_converter.hpp File Reference

```
#include <cmath>
#include <geometry_msgs/msg/twist.hpp>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/string.hpp>
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"
```

Include dependency graph for message_converter.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [star::MessageConverter](#)
Message converter for ROS2 <-> Protobuf translation.

Namespaces

- namespace `star`

9.2 `message_converter.hpp`

[Go to the documentation of this file.](#)

```

00001 // message_converter.hpp - ROS2 <-> Protobuf Message Converter
00002 // Bidirectional conversion between ROS2 standard messages and STAR Protocol
00003 // Buffers.
00004 //
00005 // STAR Project - Texas A&M University
00006 // January 2026
00007
00008 #ifndef STAR_GATEWAY_BRIDGE__MESSAGE_CONVERTER_HPP_
00009 #define STAR_GATEWAY_BRIDGE__MESSAGE_CONVERTER_HPP_
00010
00011 #include <cmath>
00012
00013 #include <geometry_msgs/msg/twist.hpp>
00014 #include <rclcpp/rclcpp.hpp>
00015 #include <std_msgs/msg/string.hpp>
00016
00017 #include "star/v1/motor_control.pb.h"
00018 #include "star/v1/telemetry.pb.h"
00019
00020 namespace star
00021 {
00022
00023     class MessageConverter {
00024     public:
00025         // =====
00026         // ROS2 -> Protobuf Conversions
00027         // =====
00028
00029         static bool twist_to_velocity_command(
00030             const geometry_msgs::msg::Twist & twist,
00031             star::v1::VelocityCommand & command,
00032             double wheel_base = 0.150,
00033             uint32_t sequence = 0);
00034
00035         static bool string_to_system_status(
00036             const std_msgs::msg::String & status_msg,
00037             star::v1::SystemStatus & system_status);
00038
00039         // =====
00040         // Protobuf -> ROS2 Conversions
00041         // =====
00042
00043         static bool
00044         velocity_command_to_twist(
00045             const star::v1::VelocityCommand & command,
00046             geometry_msgs::msg::Twist & twist,
00047             double wheel_base = 0.150);
00048
00049         static bool pid_config_to_gains(
00050             const star::v1::PidConfig & pid_config,
00051             double & kp, double & ki, double & kd);
00052
00053         // =====
00054         // Validation Helpers
00055         // =====
00056
00057         static bool is_valid_double(double value) {return std::isfinite(value);}
00058
00059         static double clamp(double value, double min, double max)
00060         {
00061             return std::max(min, std::min(max, value));
00062         }
00063
00064         static int64_t ros_time_to_us(const rclcpp::Time & time);
00065
00066     private:
00067         // Differential drive kinematics constants
00068         static constexpr double k_max_velocity_mps =
00069             2.0; // VelocityCommand valid range
00070         static constexpr double k_max_angular_vel =
00071             4.0; // Maximum angular velocity (rad/s)
00072     };
00073

```

9.3 src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp File Reference

```

00163
00164 } // namespace star
00165
00166 #endif // STAR_GATEWAY_BRIDGE_MESSAGE_CONVERTER_HPP_

```

9.3 src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge_node.hpp File Reference

```

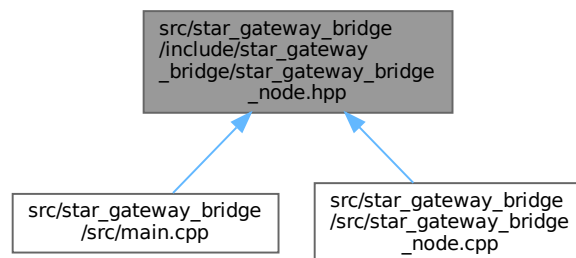
#include <memory>
#include <mutex>
#include <optional>
#include <string>
#include <diagnostic_msgs/msg/diagnostic_array.hpp>
#include <diagnostic_msgs/msg/diagnostic_status.hpp>
#include <diagnostic_msgs/msg/key_value.hpp>
#include <geometry_msgs/msg/twist.hpp>
#include <grpcpp/grpcpp.h>
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/string.hpp>
#include <std_srvs/srv/set_bool.hpp>
#include "star/v1/gateway_service.grpc.pb.h"
#include "star_gateway_bridge/message_converter.hpp"

```

Include dependency graph for star_gateway_bridge_node.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `star::StarGatewayBridgeNode`
ROS2 node that bridges the ROS2 ecosystem with the Go gateway service via gRPC.

Generated by Doxygen

Namespaces

- namespace `star`

9.4 `star_gateway_bridge_node.hpp`

[Go to the documentation of this file.](#)

```

00001 // star_gateway_bridge_node.hpp - ROS2 Gateway Bridge Node
00002 // Bridges ROS2 ecosystem with Go gateway service via gRPC.
00003 //
00004 // This node handles bidirectional communication:
00005 // - ROS2 -> Gateway: Forward telemetry (robot status) for UI display
00006 // - Gateway -> ROS2: Poll teleop commands and PID gain updates from UI
00007 //
00008 // STAR Project - Texas A&M University
00009 // Copyright 2026 STAR Project
00010 // January 2026
00011
00012 #ifndef STAR_GATEWAY_BRIDGE__STAR_GATEWAY_BRIDGE_NODE_HPP_
00013 #define STAR_GATEWAY_BRIDGE__STAR_GATEWAY_BRIDGE_NODE_HPP_
00014
00015 #include <memory>
00016 #include <mutex>
00017 #include <optional>
00018 #include <string>
00019
00020 #include <diagnostic_msgs/msg/diagnostic_array.hpp>
00021 #include <diagnostic_msgs/msg/diagnostic_status.hpp>
00022 #include <diagnostic_msgs/msg/key_value.hpp>
00023 #include <geometry_msgs/msg/twist.hpp>
00024 #include <grpcpp/grpcpp.h> // NOLINT(build/include_order)
00025 #include <rclcpp/rclcpp.hpp>
00026 #include <std_msgs/msg/string.hpp>
00027 #include <std_srvs/srv/set_bool.hpp>
00028
00029 #include "star/v1/gateway_service.grpc.pb.h"
00030 #include "star_gateway_bridge/message_converter.hpp"
00031
00032 namespace star
00033 {
00034
00063 class StarGatewayBridgeNode : public rclcpp::Node {
00064 public:
00079   explicit StarGatewayBridgeNode(
00080     const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
00081
00085   ~StarGatewayBridgeNode();
00086
00087 private:
00088   // =====
00089   // Initialization
00090   // =====
00091
00100   bool initialize_grpc_client();
00101
00106   void initialize_ros_interfaces();
00107
00108   // =====
00109   // ROS2 Callbacks
00110   // =====
00111
00120   void robot_status_callback(const std_msgs::msg::String::SharedPtr msg);
00121
00131   void set_pid_gains_callback(
00132     const std::shared_ptr<std_srvs::srv::SetBool::Request> request,
00133     std::shared_ptr<std_srvs::srv::SetBool::Response> response);
00134
00135   // =====
00136   // Timer Callbacks
00137   // =====
00138
00145   void telemetry_forward_timer_callback();
00146
00154   void teleop_poll_timer_callback();
00155
00161   void connection_watchdog_callback();
00162
00163   // =====
00164   // gRPC Helpers
00165   // =====

```

9.5 src/star_gateway_bridge/src/main.cpp File Reference

81

```

00166
00175     bool reconnect_grpc_client();
00176
00181     bool is_grpc_connected() const;
00182
00183     // =====
00184     // Member Variables
00185     // =====
00186
00187     // ROS2 Publishers
00188     rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr teleop_cmd_vel_pub_;
00189
00190     // ROS2 Subscribers
00191     rclcpp::Subscription<std_msgs::msg::String>::SharedPtr robot_status_sub_;
00192
00193     // ROS2 Services
00194     rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr set_pid_gains_service_;
00195
00196     // ROS2 Timers
00197     rclcpp::TimerBase::SharedPtr telemetry_timer_;
00198     rclcpp::TimerBase::SharedPtr teleop_timer_;
00199     rclcpp::TimerBase::SharedPtr watchdog_timer_;
00200
00201     // gRPC Client
00202     std::shared_ptr<grpc::Channel> grpc_channel_;
00203     std::unique_ptr<star::v1::GatewayService::Stub> grpc_stub_;
00204
00205     // Cached telemetry data (protected by mutexes)
00206     std::mutex robot_status_mutex_;
00207     std::optional<std_msgs::msg::String> cached_robot_status_;
00208
00209
00210     // Parameters (cached for performance)
00211     std::string gateway_address_;
00212     double telemetry_rate_hz_;
00213     double teleop_rate_hz_;
00214     double watchdog_timeout_sec_;
00215     int teleop_timeout_ms_;
00216     int grpc_deadline_ms_;
00217     double wheel_base_;
00218
00219     // Connection state
00220     bool grpc_connected_;
00221     int reconnect_attempts_;
00222     static constexpr int k_max_reconnect_attempts = 10;
00223     static constexpr int k_reconnect_backoff_ms_base = 100;
00224
00225     // Message converter (stateless utility)
00226     MessageConverter converter_;
00227
00228     // Frame drop detection for teleop commands and telemetry
00229     rclcpp::Publisher<diagnostic_msgs::msg::DiagnosticArray>::SharedPtr
00230         diagnostics_pub_;
00231     rclcpp::TimerBase::SharedPtr diagnostics_timer_;
00232
00233     uint32_t last_teleop_sequence_{0};
00234     uint64_t total_teleop_frames_{0};
00235     uint64_t teleop_frames_dropped_{0};
00236     bool first_teleop_frame_{true};
00237
00238     uint32_t last_telemetry_sequence_{0};
00239     uint64_t total_telemetry_frames_{0};
00240     uint64_t telemetry_frames_dropped_{0};
00241     bool first_telemetry_frame_{true};
00242
00243     // Methods
00244     void publish_diagnostics();
00245     void check_teleop_sequence_continuity(uint32_t current_sequence);
00246     void check_telemetry_sequence_continuity(uint32_t current_sequence);
00247 };
00248
00249 } // namespace star
00250
00251 #endif // STAR_GATEWAY_BRIDGE__STAR_GATEWAY_BRIDGE_NODE_HPP_

```

9.5 src/star_gateway_bridge/src/main.cpp File Reference

```

#include <memory>
#include "rclcpp/rclcpp.hpp"

```

- `int main (int argc, char **argv)`
Main entry point for star_gateway_bridge standalone executable.

9.5.1.1 main()

Main entry point for star_gateway_bridge standalone executable.

Usage: ros2 run star_gateway bridge star_gateway bridge main

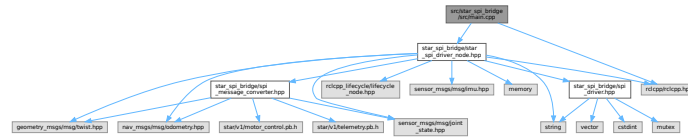
Parameters can be set via command line: `ros2 run star_gateway_bridge star_gateway_bridge_main \-ros-args -p gateway_address:=192.168.1.100:50051 \-p telemetry_rate_hz:=20.0`

Definition at line 23 of file main.cpp.

[Go to the documentation of this file.](#)

9.7 src/star_spi_bridge/src/main.cpp File Reference

```
#include "star_spi_bridge/star_spi_driver_node.hpp"
#include <rclcpp/rclcpp.hpp>
Include dependency graph for main.cpp:
```



Functions

- `int main (int argc, char **argv)`

9.7.1 Function Documentation

9.7.1.1 main()

```
int main (
    int argc,
    char ** argv)
```

Definition at line 7 of file [main.cpp](#).

9.8 main.cpp

[Go to the documentation of this file.](#)

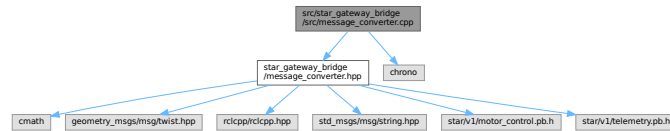
```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/star_spi_driver_node.hpp"
00004
00005 #include <rclcpp/rclcpp.hpp>
00006
00007 int main(int argc, char **argv)
00008 {
00009     rclcpp::init(argc, argv);
00010
00011     rclcpp::executors::SingleThreadedExecutor executor;
00012     auto node = std::make_shared<star_spi_bridge::StarSpiDriverNode>();
00013
00014     executor.add_node(node->get_node_base_interface());
00015     executor.spin();
00016
00017     rclcpp::shutdown();
00018     return 0;
00019 }
```

9.9 src/star_gateway_bridge/src/message_converter.cpp File Reference

```
#include "star_gateway_bridge/message_converter.hpp"
```

```
#include <chrono>
```

Include dependency graph for message_converter.cpp:



Namespaces

- namespace `star`

9.10 message_converter.cpp

[Go to the documentation of this file.](#)

```

00001 // message_converter.cpp - ROS2 <-> Protobuf Message Converter Implementation
00002 // Bidirectional conversion between ROS2 standard messages and STAR Protocol
00003 // Buffers.
00004 //
00005 // STAR Project - Texas A&M University
00006 // Copyright 2026 STAR Project
00007 // January 2026
00008
00009 #include "star_gateway_bridge/message_converter.hpp"
00010
00011 #include <chrono>
00012 namespace star
00013 {
00014
00015 // =====
00016 // ROS2 -> Protobuf Conversions
00017 // =====
00018
00019 bool MessageConverter::twist_to_velocity_command(
00020     const geometry_msgs::msg::Twist & twist, star::v1::VelocityCommand & command,
00021     double wheel_base, uint32_t sequence)
00022 {
00023     // Validate inputs for NaN/infinity
00024     if (!is_valid_double(twist.linear.x) || !is_valid_double(twist.angular.z)) {
00025         RCLCPP_WARN(rclcpp::get_logger("message_converter"),
00026             "Invalid Twist: NaN/infinity in linear.x or angular.z");
00027         return false;
00028     }
00029
00030     if (!is_valid_double(wheel_base) || wheel_base <= 0.0) {
00031         RCLCPP_ERROR(rclcpp::get_logger("message_converter"),
00032             "Invalid wheel_base: must be positive and finite");
00033         return false;
00034     }
00035
00036     // Clamp input velocities to safe ranges
00037     double linear =
00038         clamp(twist.linear.x, -k_max_velocity_mps, k_max_velocity_mps);
00039     double angular =
00040         clamp(twist.angular.z, -k_max_angular_vel, k_max_angular_vel);
00041
00042     // Differential drive kinematics: (linear, angular) -> (left, right)
00043     // left_vel = linear - (angular * wheel_base / 2)
00044     // right_vel = linear + (angular * wheel_base / 2)
00045     double half_base = wheel_base / 2.0;
00046     double left_velocity = linear - (angular * half_base);

```

9.10 message_converter.cpp

85

```

00047 double right_velocity = linear + (angular * half_base);
00048
00049 // Clamp wheel velocities to VelocityCommand valid range [-2.0, 2.0] m/s
00050 left_velocity = clamp(left_velocity, -k_max_velocity_mps, k_max_velocity_mps);
00051 right_velocity =
00052     clamp(right_velocity, -k_max_velocity_mps, k_max_velocity_mps);
00053
00054 // Populate protobuf message
00055 // Note: front_left/back_left = left side, front_right/back_right = right side
00056 // for differential drive
00057 command.set_front_left_velocity_mps(left_velocity);
00058 command.set_back_left_velocity_mps(left_velocity);
00059 command.set_front_right_velocity_mps(right_velocity);
00060 command.set_back_right_velocity_mps(right_velocity);
00061 command.set_sequence(sequence);
00062
00063 // Timestamp in microseconds since epoch
00064 auto now = std::chrono::system_clock::now();
00065 auto us = std::chrono::duration_cast<std::chrono::microseconds>(
00066     now.time_since_epoch())
00067     .count();
00068 command.set_timestamp_us(us);
00069
00070 return true;
00071 }
00072
00073 bool MessageConverter::string_to_system_status(
00074     const std_msgs::msg::String & status_msg,
00075     star::vl::SystemStatus & system_status)
00076 {
00077     // For now, implement basic string parsing
00078     // In production, use a JSON library (e.g., nlohmann/json) for robust parsing
00079     // This is a placeholder implementation
00080
00081     const std::string & data = status_msg.data;
00082
00083     // Simple keyword-based parsing (not robust, but sufficient for MVP)
00084     if (data.find("MANUAL") != std::string::npos) {
00085         system_status.set_mode(star::vl::ROBOT_MODE_MANUAL);
00086     } else if (data.find("AUTONOMOUS") != std::string::npos) {
00087         system_status.set_mode(star::vl::ROBOT_MODE_AUTONOMOUS);
00088     } else if (data.find("MAPPING") != std::string::npos) {
00089         system_status.set_mode(star::vl::ROBOT_MODE_MAPPING);
00090     } else if (data.find("EMERGENCY_STOP") != std::string::npos) {
00091         system_status.set_mode(star::vl::ROBOT_MODE_EMERGENCY_STOP);
00092     } else {
00093         system_status.set_mode(star::vl::ROBOT_MODE_IDLE);
00094     }
00095
00096 // Connection status (default to CONNECTED if we're receiving messages)
00097 system_status.set_connection_status(star::vl::CONNECTION_STATUS_CONNECTED);
00098
00099 // TODO(star): Parse additional fields from JSON when available
00100 // For now, assume all subsystems are connected
00101 system_status.set_rx72n_connected(true);
00102 system_status.set_lidar_connected(true);
00103 system_status.set_ros_connected(true);
00104
00105 return true;
00106 }
00107
00108 // =====
00109 // Protobuf -> ROS2 Conversions
00110 // =====
00111
00112 bool MessageConverter::velocity_command_to_twist(
00113     const star::vl::VelocityCommand & command, geometry_msgs::msg::Twist & twist,
00114     double wheel_base)
00115 {
00116     // Validate protobuf inputs
00117     // Note: front_left/back_left = left side, front_right/back_right = right side
00118     // for differential drive Average left/right side velocities for differential
00119     // drive
00120     double left_vel =
00121         (command.front_left_velocity_mps() + command.back_left_velocity_mps()) /
00122         2.0;
00123     double right_vel =
00124         (command.front_right_velocity_mps() + command.back_right_velocity_mps()) /
00125         2.0;
00126
00127     if (!is_valid_double(left_vel) || !is_valid_double(right_vel)) {
00128         RCLCPP_WARN(rclcpp::get_logger("message_converter"),
00129             "Invalid VelocityCommand: NaN/infinity in wheel velocities");
00130         return false;
00131     }
00132
00133     if (!is_valid_double(wheel_base) || wheel_base <= 0.0) {

```

9.11 src/star_gateway_bridge/src/star_gateway_bridge_node.cpp File Reference

Include dependency graph for `star_gateway_bridge_node.cpp`:



- Generated by Doxygen

9.12 star_gateway_bridge_node.cpp

[Go to the documentation of this file.](#)

```

00001 // star_gateway_bridge_node.cpp - ROS2 Gateway Bridge Node Implementation
00002 // Bridges ROS2 ecosystem with Go gateway service via gRPC.
00003 //
00004 // STAR Project - Texas A&M University
00005 // Copyright 2026 STAR Project
00006 // January 2026
00007
00008 #include "star_gateway_bridge/star_gateway_bridge_node.hpp"
00009
00010 #include <chrono>
00011 #include <thread>
00012
00013 using namespace std::chrono_literals;
00014
00015 namespace star
00016 {
00017
00018 StarGatewayBridgeNode::StarGatewayBridgeNode(const rclcpp::NodeOptions & options)
00019 : Node("star_gateway_bridge", options), grpc_connected_(false),
00020   reconnect_attempts_(0)
00021 {
00022   RCLCPP_INFO(this->get_logger(), "Initializing STAR Gateway Bridge Node");
00023
00024   // Declare parameters with defaults
00025   this->declare_parameter("gateway_address", "localhost:50051");
00026   this->declare_parameter("telemetry_rate_hz", 10.0);
00027   this->declare_parameter("teleop_rate_hz", 50.0);
00028   this->declare_parameter("watchdog_timeout_sec", 5.0);
00029   this->declare_parameter("teleop_timeout_ms", 500);
00030   this->declare_parameter("grpc_deadline_ms", 100);
00031   this->declare_parameter("wheel_base", 0.150); // 150mm track width
00032
00033   // Cache parameters
00034   gateway_address_ = this->get_parameter("gateway_address").as_string();
00035   telemetry_rate_hz_ = this->get_parameter("telemetry_rate_hz").as_double();
00036   teleop_rate_hz_ = this->get_parameter("teleop_rate_hz").as_double();
00037   watchdog_timeout_sec_ =
00038     this->get_parameter("watchdog_timeout_sec").as_double();
00039   teleop_timeout_ms_ = this->get_parameter("teleop_timeout_ms").as_int();
00040   grpc_deadline_ms_ = this->get_parameter("grpc_deadline_ms").as_int();
00041   wheel_base_ = this->get_parameter("wheel_base").as_double();
00042
00043   RCLCPP_INFO(
00044     this->get_logger(),
00045     "Configuration: gateway=%s, telemetry_rate=%.1fHz, teleop_rate=%.1fHz, "
00046     "watchdog=%.1fs, teleop_timeout=%dms, grpc_deadline=%dms, "
00047     "wheel_base=%.3fm",
00048     gateway_address_.c_str(), telemetry_rate_hz_, teleop_rate_hz_,
00049     watchdog_timeout_sec_, teleop_timeout_ms_, grpc_deadline_ms_,
00050     wheel_base_);
00051
00052   // Initialize gRPC client
00053   if (!initialize_grpc_client()) {
00054     RCLCPP_WARN(this->get_logger(),
00055       "Failed to connect to Gateway at %s - will retry in background",
00056       gateway_address_.c_str());
00057   }
00058
00059   // Initialize ROS2 interfaces
00060   initialize_ros_interfaces();
00061
00062   RCLCPP_INFO(this->get_logger(),
00063     "STAR Gateway Bridge Node initialized successfully");
00064 }
00065
00066 StarGatewayBridgeNode::~StarGatewayBridgeNode()
00067 {
00068   RCLCPP_INFO(this->get_logger(), "Shutting down STAR Gateway Bridge Node");
00069
00070   // Send stop command before shutdown
00071   auto zero_twist = geometry_msgs::msg::Twist();
00072   if (teleop_cmd_vel_pub_) {
00073     teleop_cmd_vel_pub_->publish(zero_twist);
00074     RCLCPP_INFO(this->get_logger(), "Sent stop command on shutdown");
00075   }
00076
00077   // Close gRPC channel gracefully
00078   if (grpc_channel_) {
00079     RCLCPP_INFO(this->get_logger(), "Closing gRPC channel");
00080   }
00081 }
00082

```

Generated by Doxygen

```

00083 // =====
00084 // Initialization
00085 // =====
00086
00087 bool StarGatewayBridgeNode::initialize_grpc_client()
00088 {
00089     RCLCPP_INFO(this->get_logger(), "Connecting to Gateway gRPC server at %s",
00090         gateway_address_.c_str());
00091
00092     // Create gRPC channel with keepalive settings
00093     grpc::ChannelArguments args;
00094     args.SetInt(GRPC_ARG_KEEPALIVE_TIME_MS, 10000); // 10s keepalive ping
00095     args.SetInt(GRPC_ARG_KEEPALIVE_TIMEOUT_MS, 5000); // 5s keepalive timeout
00096     args.SetInt(GRPC_ARG_KEEPALIVE_PERMIT_WITHOUT_CALLS,
00097         1); // Allow keepalive without calls
00098
00099     grpc_channel_ = grpc::CreateCustomChannel(
00100         gateway_address_, grpc::InsecureChannelCredentials(), args);
00101
00102     if (!grpc_channel_) {
00103         RCLCPP_ERROR(this->get_logger(), "Failed to create gRPC channel");
00104         return false;
00105     }
00106
00107     // Wait for channel to be ready (with timeout)
00108     auto deadline = std::chrono::system_clock::now() +
00109         std::chrono::milliseconds(grpc_deadline_ms_);
00110
00111     if (!grpc_channel_>WaitForConnected(deadline)) {
00112         RCLCPP_WARN(this->get_logger(), "gRPC channel not ready within %dms",
00113             grpc_deadline_ms_);
00114         grpc_connected_ = false;
00115         return false;
00116     }
00117
00118     // Create gRPC stub
00119     grpc_stub_ = star::v1::GatewayService::NewStub(grpc_channel_);
00120
00121     RCLCPP_INFO(this->get_logger(),
00122         "Successfully connected to Gateway gRPC server");
00123     grpc_connected_ = true;
00124     reconnect_attempts_ = 0;
00125     return true;
00126 }
00127
00128 void StarGatewayBridgeNode::initialize_ros_interfaces()
00129 {
00130     RCLCPP_INFO(this->get_logger(), "Initializing ROS2 interfaces");
00131
00132     // Publishers
00133     teleop_cmd_vel_pub_ =
00134         this->create_publisher<geometry_msgs::msg::Twist>("/teleop/cmd_vel", 10);
00135
00136     // Subscribers
00137     robot_status_sub_ = this->create_subscription<std_msgs::msg::String>(
00138         "/robot_status", 10,
00139         std::bind(&StarGatewayBridgeNode::robot_status_callback, this,
00140             std::placeholders::_1));
00141
00142     // Services
00143     set_pid_gains_service_ = this->create_service<std_srvs::srv::SetBool>(
00144         "/set_pid_gains",
00145         std::bind(&StarGatewayBridgeNode::set_pid_gains_callback, this,
00146             std::placeholders::_1, std::placeholders::_2));
00147
00148     // Timers
00149     auto telemetry_period_ms = static_cast<int>(1000.0 / telemetry_rate_hz_);
00150     telemetry_timer_ = this->create_wall_timer(
00151         std::chrono::milliseconds(telemetry_period_ms),
00152         std::bind(&StarGatewayBridgeNode::telemetry_forward_timer_callback,
00153             this));
00154
00155     auto teleop_period_ms = static_cast<int>(1000.0 / teleop_rate_hz_);
00156     teleop_timer_ = this->create_wall_timer(
00157         std::chrono::milliseconds(teleop_period_ms),
00158         std::bind(&StarGatewayBridgeNode::teleop_poll_timer_callback, this));
00159
00160     auto watchdog_period_ms = static_cast<int>(watchdog_timeout_sec_ * 1000.0);
00161     watchdog_timer_ = this->create_wall_timer(
00162         std::chrono::milliseconds(watchdog_period_ms),
00163         std::bind(&StarGatewayBridgeNode::connection_watchdog_callback, this));
00164
00165     // Create diagnostics publisher
00166     diagnostics_pub_ =
00167         this->create_publisher<diagnostic_msgs::msg::DiagnosticArray>(
00168         "/diagnostics", 10);
00169

```

9.12 star_gateway_bridge_node.cpp

89

```

00170
00171 // Create diagnostics timer (1 Hz for human readability)
00172 diagnostics_timer_ = this->create_wall_timer(
00173     std::chrono::seconds(1),
00174     std::bind(&StarGatewayBridgeNode::publish_diagnostics, this));
00175
00176 RCLCPP_INFO(
00177     this->get_logger(),
00178     "ROS2 interfaces initialized: telemetry=%dms, teleop=%dms, watchdog=%dms",
00179     telemetry_period_ms, teleop_period_ms, watchdog_period_ms);
00180 RCLCPP_INFO(this->get_logger(), "Diagnostics publisher initialized");
00181 }
00182
00183 // =====
00184 // ROS2 Callbacks
00185 // =====
00186
00187 void StarGatewayBridgeNode::robot_status_callback(
00188     const std_msgs::msg::String::SharedPtr msg)
00189 {
00190     // Use try_lock to avoid blocking callback (non-blocking pattern)
00191     if (robot_status_mutex_.try_lock()) {
00192         cached_robot_status_ = *msg;
00193         robot_status_mutex_.unlock();
00194     }
00195     // If try_lock fails, skip this update (cache will use previous value)
00196 }
00197
00198 void StarGatewayBridgeNode::set_pid_gains_callback(
00199     const std::shared_ptr<std_srvs::srv::SetBool::Request> request,
00200     std::shared_ptr<std_srvs::srv::SetBool::Response> response)
00201 {
00202     // TODO(star): Phase 4: Implement PID gains service after defining custom
00203     // service type. For now, placeholder implementation.
00204     (void)request; // Unused parameter (placeholder service)
00205
00206     RCLCPP_INFO(this->get_logger(), "set_pid_gains service called (placeholder)");
00207
00208     // TODO(star): Parse PID gains from request, forward to Gateway via gRPC
00209     // TODO(star): Gateway will forward to motor controller via SPI bridge
00210
00211     response->success = false;
00212     response->message = "PID gains service not yet implemented (Phase 4)";
00213 }
00214
00215 // =====
00216 // Timer Callbacks
00217 // =====
00218
00219 void StarGatewayBridgeNode::telemetry_forward_timer_callback()
00220 {
00221     if (!grpc_connected_) {
00222         RCLCPP_DEBUG_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
00223             "Skipping telemetry forward - gRPC not connected");
00224         return;
00225     }
00226
00227     // Get cached telemetry (non-blocking)
00228     std::optional<std_msgs::msg::String> robot_status;
00229
00230     if (robot_status_mutex_.try_lock()) {
00231         robot_status = cached_robot_status_;
00232         robot_status_mutex_.unlock();
00233     }
00234
00235     // Forward telemetry to Gateway via gRPC
00236     if (grpc_stub_ && robot_status.has_value()) {
00237         grpc::ClientContext context;
00238         context.set_deadline(std::chrono::system_clock::now() +
00239             std::chrono::milliseconds(grpc_deadline_ms_));
00240
00241         star::v1::ForwardTelemetryRequest request;
00242
00243         // Set request header
00244         auto *header = request.mutable_header();
00245         header->set_request_id(
00246             "telemetry_" +
00247             std::to_string(
00248                 std::chrono::system_clock::now().time_since_epoch().count()));
00249
00250         if (robot_status.has_value()) {
00251             converter_.string_to_system_status(*robot_status,
00252                 *request.mutable_system_status());
00253         }
00254
00255         star::v1::ForwardTelemetryResponse response;
00256         grpc::Status status =

```

Generated by Doxygen

```

00257     grpc_stub->ForwardTelemetry(&context, request, &response);
00258
00259     if (!status.ok()) {
00260         RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
00261             "ForwardTelemetry gRPC failed: %s",
00262             status.error_message().c_str());
00263         grpc_connected_ = false;
00264     } else {
00265         // Successful transmission - increment frame counter
00266         total_telemetry_frames++;
00267     }
00268 }
00269
00270 RCLCPP_DEBUG_THROTTLE(this->get_logger(), *this->get_clock(), 10000,
00271     "Telemetry forward: robot_status=%s",
00272     robot_status.has_value() ? "cached" : "none");
00273 }
00274
00275 void StarGatewayBridgeNode::teleop_poll_timer_callback()
00276 {
00277     if (!grpc_connected_ || !grpc_stub_) {
00278         // Publish zero velocity when not connected (safety feature)
00279         auto zero_twist = geometry_msgs::msg::Twist();
00280         teleop_cmd_vel_pub->publish(zero_twist);
00281         return;
00282     }
00283
00284     // Poll Gateway for latest teleop command via gRPC
00285     grpc::ClientContext context;
00286     context.set_deadline(std::chrono::system_clock::now() +
00287         std::chrono::milliseconds(grpc_deadline_ms_));
00288
00289     star::v1::GetTeleopCommandRequest request;
00290
00291     // Set request header
00292     auto *header = request.mutable_header();
00293     header->set_request_id(
00294         "teleop_" +
00295         std::to_string(
00296             std::chrono::system_clock::now().time_since_epoch().count()));
00297
00298     star::v1::GetTeleopCommandResponse response;
00299
00300     grpc::Status status =
00301         grpc_stub->GetTeleopCommand(&context, request, &response);
00302
00303     if (!status.ok()) {
00304         RCLCPP_WARN_THROTTLE(this->get_logger(), *this->get_clock(), 5000,
00305             "GetTeleopCommand gRPC failed: %s",
00306             status.error_message().c_str());
00307         grpc_connected_ = false;
00308         auto zero_twist = geometry_msgs::msg::Twist();
00309         teleop_cmd_vel_pub->publish(zero_twist);
00310         return;
00311     }
00312
00313     // Check if command is available
00314     if (!response.command_available()) {
00315         RCLCPP_DEBUG_THROTTLE(this->get_logger(), *this->get_clock(), 10000,
00316             "No teleop command available from Gateway");
00317         auto zero_twist = geometry_msgs::msg::Twist();
00318         teleop_cmd_vel_pub->publish(zero_twist);
00319         return;
00320     }
00321
00322     // Check command staleness (safety feature)
00323     auto now_us = std::chrono::duration_cast<std::chrono::microseconds>(
00324         std::chrono::system_clock::now().time_since_epoch())
00325         .count();
00326     auto cmd_age_us = now_us - response.command().timestamp_us();
00327     auto cmd_age_ms = cmd_age_us / 1000;
00328
00329     if (cmd_age_ms > teleop_timeout_ms_) {
00330         RCLCPP_WARN_THROTTLE(
00331             this->get_logger(), *this->get_clock(), 1000,
00332             "Teleop command stale (%ldms > %ldms) - sending zero velocity",
00333             cmd_age_ms, teleop_timeout_ms_);
00334         auto zero_twist = geometry_msgs::msg::Twist();
00335         teleop_cmd_vel_pub->publish(zero_twist);
00336         return;
00337     }
00338
00339     // Convert and publish fresh command
00340     geometry_msgs::msg::Twist twist;
00341     if (converter_.velocity_command_to_twist(response.command(), twist,
00342         wheel_base_))
00343     {

```


9.12 star_gateway_bridge_node.cpp

91

```

00344 // Check sequence continuity for frame drop detection
00345 uint32_t current_seq = response.command().sequence();
00346 check_teleop_sequence_continuity(current_seq);
00347
00348 teleop_cmd_vel_pub_->publish(twist);
00349 } else {
00350     RCLCPP_WARN(this->get_logger(),
00351                 "Failed to convert VelocityCommand to Twist");
00352     auto zero_twist = geometry_msgs::msg::Twist();
00353     teleop_cmd_vel_pub_->publish(zero_twist);
00354 }
00355 }
00356
00357 void StarGatewayBridgeNode::connection_watchdog_callback()
00358 {
00359     if (!is_grpc_connected()) {
00360         RCLCPP_WARN_THROTTLE(
00361             this->get_logger(), *this->get_clock(), 10000,
00362             "Gateway gRPC connection unhealthy - attempting reconnection");
00363         grpc_connected_ = false;
00364         reconnect_grpc_client();
00365     }
00366 }
00367
00368 // =====
00369 // gRPC Helpers
00370 // =====
00371
00372 bool StarGatewayBridgeNode::reconnect_grpc_client()
00373 {
00374     if (reconnect_attempts_ >= k_max_reconnect_attempts) {
00375         RCLCPP_ERROR_THROTTLE(this->get_logger(), *this->get_clock(), 30000,
00376                               "Max reconnection attempts (%d) reached - giving up",
00377                               k_max_reconnect_attempts);
00378         return false;
00379     }
00380     reconnect_attempts_++;
00381
00382     // Exponential backoff
00383     int backoff_ms =
00384         k_reconnect_backoff_ms_base * (1 « std::min(reconnect_attempts_, 5));
00385     RCLCPP_INFO(this->get_logger(), "Reconnection attempt %d/%d (backoff: %dms)",
00386                 reconnect_attempts_, k_max_reconnect_attempts, backoff_ms);
00387     std::this_thread::sleep_for(std::chrono::milliseconds(backoff_ms));
00388     return initialize_grpc_client();
00389 }
00390
00391 bool StarGatewayBridgeNode::is_grpc_connected() const
00392 {
00393     if (!grpc_channel_) {
00394         return false;
00395     }
00396     // false = don't try to connect
00397     auto state = grpc_channel_->GetState(false);
00398     return state == GRPC_CHANNEL_READY;
00399 }
00400
00401 void StarGatewayBridgeNode::check_teleop_sequence_continuity(
00402     uint32_t current_sequence)
00403 {
00404     if (first_teleop_frame_) {
00405         last_teleop_sequence_ = current_sequence;
00406         first_teleop_frame_ = false;
00407         RCLCPP_INFO(this->get_logger(), "First teleop frame received, sequence=%u",
00408                     current_sequence);
00409     } else {
00410         uint32_t expected_seq = last_teleop_sequence_ + 1;
00411
00412         // Detect sequence gap (accounting for uint32 wraparound)
00413         if (current_sequence != expected_seq) {
00414             uint32_t gap = current_sequence - expected_seq;
00415
00416             // Sanity check: ignore huge gaps (likely system reset or wraparound)
00417             if (gap < 10000) {
00418                 teleop_frames_dropped_ += gap;
00419                 RCLCPP_WARN(this->get_logger(),
00420                             "Teleop frame drop: expected seq %u, got %u (gap=%u, "
00421                             "total_dropped=%lu)",
00422                             expected_seq, current_sequence, gap,
00423                             teleop_frames_dropped_);
00424             } else {
00425                 RCLCPP_WARN(this->get_logger(),
00426                             "Teleop frame drop: expected seq %u, got %u (gap=%u, "
00427                             "total_dropped=%lu)",
00428                             expected_seq, current_sequence, gap,
00429                             teleop_frames_dropped_);
00430             }
00431         }
00432     }
00433 }

```

Generated by Doxygen

```

00431         "Teleop sequence reset detected: %u -> %u (ignoring as "
00432         "system restart)",
00433         last_teleop_sequence_, current_sequence);
00434     }
00435 }
00436
00437     last_teleop_sequence_ = current_sequence;
00438 }
00439
00440 total_teleop_frames++;
00441 }
00442
00443 void StarGatewayBridgeNode::check_teleometry_sequence_continuity(
00444     uint32_t current_sequence)
00445 {
00446     if (first_teleometry_frame_) {
00447         last_teleometry_sequence_ = current_sequence;
00448         first_teleometry_frame_ = false;
00449         RCLCPP_INFO(this->get_logger(),
00450             "First teleometry frame received, sequence=%u",
00451             current_sequence);
00452     } else {
00453         uint32_t expected_seq = last_teleometry_sequence_ + 1;
00454
00455         // Detect sequence gap (accounting for uint32 wraparound)
00456         if (current_sequence != expected_seq) {
00457             uint32_t gap = current_sequence - expected_seq;
00458
00459             // Sanity check: ignore huge gaps (likely system reset or wraparound)
00460             if (gap < 10000) {
00461                 telemetry_frames_dropped_ += gap;
00462                 RCLCPP_WARN(this->get_logger(),
00463                     "Telemetry frame drop: expected seq %u, got %u (gap=%u, "
00464                     "total_dropped=%lu)",
00465                     expected_seq, current_sequence, gap,
00466                     telemetry_frames_dropped_);
00467             } else {
00468                 RCLCPP_WARN(this->get_logger(),
00469                     "Telemetry sequence reset detected: %u -> %u (ignoring as "
00470                     "system restart)",
00471                     last_teleometry_sequence_, current_sequence);
00472             }
00473         }
00474         last_teleometry_sequence_ = current_sequence;
00475     }
00476 }
00477
00478 total_teleometry_frames++;
00479 }
00480
00481 void StarGatewayBridgeNode::publish_diagnostics()
00482 {
00483     auto diag_array = diagnostic_msgs::msg::DiagnosticArray();
00484     diag_array.header.stamp = this->now();
00485
00486     diagnostic_msgs::msg::DiagnosticStatus status;
00487     status.name = "star_gateway_bridge/teleop_command_drops";
00488     status.hardware_id = "gateway_bridge";
00489
00490     // Determine severity level
00491     if (total_teleop_frames_ == 0) {
00492         status.level = diagnostic_msgs::msg::DiagnosticStatus::STALE;
00493         status.message = "No teleop commands received yet";
00494     } else if (teleop_frames_dropped_ == 0) {
00495         status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
00496         status.message = "No teleop frame drops detected";
00497     } else {
00498         double drop_rate = (teleop_frames_dropped_ * 100.0) / total_teleop_frames_;
00499
00500         if (drop_rate < 1.0) {
00501             status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00502             status.message =
00503                 "Minor teleop drops (" + std::to_string(drop_rate) + "%)";
00504         } else {
00505             status.level = diagnostic_msgs::msg::DiagnosticStatus::ERROR;
00506             status.message =
00507                 "Critical teleop loss (" + std::to_string(drop_rate) + "%)";
00508         }
00509     }
00510
00511     // Add detailed metrics as key-value pairs
00512     diagnostic_msgs::msg::KeyValue kv;
00513
00514     kv.key = "total_frames";
00515     kv.value = std::to_string(total_teleop_frames_);
00516     status.values.push_back(kv);
00517 }

```

9.12 star_gateway_bridge_node.cpp

93

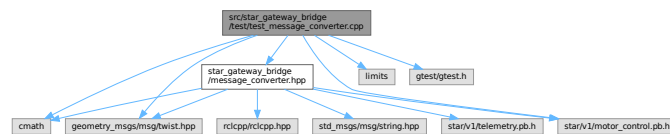
```

00518 kv.key = "dropped_frames";
00519 kv.value = std::to_string(teleop_frames_dropped_);
00520 status.values.push_back(kv);
00521
00522 double drop_rate =
00523     total_teleop_frames_ > 0 ?
00524     (teleop_frames_dropped_ * 100.0) / total_teleop_frames_ :
00525     0.0;
00526 kv.key = "drop_rate_percent";
00527 kv.value = std::to_string(drop_rate);
00528 status.values.push_back(kv);
00529
00530 kv.key = "last_sequence";
00531 kv.value = std::to_string(last_teleop_sequence_);
00532 status.values.push_back(kv);
00533
00534 diag_array.status.push_back(status);
00535
00536 // Add telemetry diagnostics
00537 diagnostic_msgs::msg::DiagnosticStatus telemetry_status;
00538 telemetry_status.name = "star_gateway_bridge/telemetry_frame_drops";
00539 telemetry_status.hardware_id = "gateway_bridge";
00540
00541 if (total_telemetry_frames_ == 0) {
00542     telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::STALE;
00543     telemetry_status.message = "No telemetry frames received";
00544 } else if (telemetry_frames_dropped_ == 0) {
00545     telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
00546     telemetry_status.message = "No telemetry drops";
00547 } else {
00548     double telemetry_drop_rate =
00549         (telemetry_frames_dropped_ * 100.0) / total_telemetry_frames_;
00550
00551     if (telemetry_drop_rate < 5.0) {
00552         telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00553         telemetry_status.message = "Minor telemetry drops (" +
00554             std::to_string(telemetry_drop_rate) + "%)";
00555     } else {
00556         telemetry_status.level = diagnostic_msgs::msg::DiagnosticStatus::ERROR;
00557         telemetry_status.message = "Critical telemetry loss (" +
00558             std::to_string(telemetry_drop_rate) + "%)";
00559     }
00560 }
00561
00562 // Add telemetry metrics
00563 diagnostic_msgs::msg::KeyValue telemetry_kv;
00564
00565 telemetry_kv.key = "total_frames";
00566 telemetry_kv.value = std::to_string(total_telemetry_frames_);
00567 telemetry_status.values.push_back(telemetry_kv);
00568
00569 telemetry_kv.key = "dropped_frames";
00570 telemetry_kv.value = std::to_string(telemetry_frames_dropped_);
00571 telemetry_status.values.push_back(telemetry_kv);
00572
00573 double telemetry_drop_rate =
00574     total_telemetry_frames_ > 0 ?
00575     (telemetry_frames_dropped_ * 100.0) / total_telemetry_frames_ :
00576     0.0;
00577 telemetry_kv.key = "drop_rate_percent";
00578 telemetry_kv.value = std::to_string(telemetry_drop_rate);
00579 telemetry_status.values.push_back(telemetry_kv);
00580
00581 telemetry_kv.key = "last_sequence";
00582 telemetry_kv.value = std::to_string(last_telemetry_sequence_);
00583 telemetry_status.values.push_back(telemetry_kv);
00584
00585 diag_array.status.push_back(telemetry_status);
00586
00587 // Publish combined diagnostics
00588 diagnostics_pub_>publish(diag_array);
00589 }
00590
00591 } // namespace star
00592
00593 // Component registration for composable node
00594 #include "rclcpp_components/register_node_macro.hpp"
00595 RCLCPP_COMPONENTS_REGISTER_NODE(star::StarGatewayBridgeNode)

```

9.13 src/star_gateway_bridge/test/test_message_converter.cpp File Reference

```
#include "star_gateway_bridge/message_converter.hpp"
#include <cmath>
#include <limits>
#include <geometry_msgs/msg/twist.hpp>
#include <gtest/gtest.h>
#include "star/v1/motor_control.pb.h"
Include dependency graph for test_message_converter.cpp:
```



Classes

- class [star::MessageConverterTest](#)

Namespaces

- namespace [star](#)

Functions

- [star::TEST_F \(MessageConverterTest, TwistToCommandZeroVelocity\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandPureTranslation\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandPureRotation\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandMixedMotion\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandNegativeLinear\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandNegativeAngular\)](#)
- [star::TEST_F \(MessageConverterTest, CommandToTwistZeroVelocity\)](#)
- [star::TEST_F \(MessageConverterTest, CommandToTwistPureTranslation\)](#)
- [star::TEST_F \(MessageConverterTest, CommandToTwistPureRotation\)](#)
- [star::TEST_F \(MessageConverterTest, CommandToTwistMixedMotion\)](#)
- [star::TEST_F \(MessageConverterTest, KinematicsRoundtripZero\)](#)
- [star::TEST_F \(MessageConverterTest, KinematicsRoundtripTranslation\)](#)
- [star::TEST_F \(MessageConverterTest, KinematicsRoundtripRotation\)](#)
- [star::TEST_F \(MessageConverterTest, KinematicsRoundtripMixed\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandNaNLinear\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandNaNAngular\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandInfinityLinear\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandInfinityAngular\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandNegativeInfinity\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandZeroWheelBase\)](#)
- [star::TEST_F \(MessageConverterTest, TwistToCommandNegativeWheelBase\)](#)

9.14 test_message_converter.cpp

95

- `star::TEST_F (MessageConverterTest, CommandToTwistNaNMotor0)`
- `star::TEST_F (MessageConverterTest, CommandToTwistNaNMotor1)`
- `star::TEST_F (MessageConverterTest, CommandToTwistInfinityMotor0)`
- `star::TEST_F (MessageConverterTest, PidConfigToGainsNominal)`
- `star::TEST_F (MessageConverterTest, PidConfigToGainsZeroGains)`
- `star::TEST_F (MessageConverterTest, PidConfigToGainsNaN)`
- `star::TEST_F (MessageConverterTest, PidConfigToGainsInfinity)`
- `star::TEST_F (MessageConverterTest, PidConfigToGainsNegativeGains)`
- `star::TEST_F (MessageConverterTest, PidConfigToGainsLargeValues)`
- `star::TEST_F (MessageConverterTest, TwistToCommandMaxVelocity)`
- `star::TEST_F (MessageConverterTest, TwistToCommandVerySmallWheelBase)`
- `star::TEST_F (MessageConverterTest, TwistToCommandVeryLargeWheelBase)`
- `int main (int argc, char **argv)`

9.13.1 Function Documentation

9.13.1.1 main()

```
int main (
    int argc,
    char ** argv)
```

Definition at line 508 of file `test_message_converter.cpp`.

9.14 test_message_converter.cpp

[Go to the documentation of this file.](#)

```
00001 // Copyright (c) 2026 STAR Project
00002 // Licensed under MIT
00003
00004 #include "star_gateway_bridge/message_converter.hpp"
00005
00006 #include <cmath> // NOLINT(build/include_order)
00007 #include <limits> // NOLINT(build/include_order)
00008
00009 #include <geometry_msgs/msg/twist.hpp> // NOLINT(build/include_order)
00010 #include <gtest/gtest.h> // NOLINT(build/include_order)
00011 #include "star/v1/motor_control.pb.h"
00012
00013 namespace star
00014 {
00015
00016 // Test fixture for MessageConverter tests
00017 class MessageConverterTest : public ::testing::Test
00018 {
00019 protected:
00020     MessageConverter converter_;
00021     static constexpr double TOLERANCE = 1e-6;
00022     static constexpr double WHEEL_BASE_M = 0.150; // 150mm wheel base
00023 };
00024
00025 // =====
00026 // Forward Kinematics Tests (Twist -> VelocityCommand)
00027 // =====
00028
00029 TEST_F(MessageConverterTest, TwistToCommandZeroVelocity)
00030 {
00031     geometry_msgs::msg::Twist twist;
00032     twist.linear.x = 0.0;
00033     twist.angular.z = 0.0;
00034
00035     star::v1::VelocityCommand command;
00036     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00037 }
```

Generated by Doxygen

```

00038 EXPECT_NEAR(command.front_left_velocity_mps(), 0.0, TOLERANCE);
00039 EXPECT_NEAR(command.front_right_velocity_mps(), 0.0, TOLERANCE);
00040 }
00041
00042 TEST_F(MessageConverterTest, TwistToCommandPureTranslation)
00043 {
00044     geometry_msgs::msg::Twist twist;
00045     twist.linear.x = 1.0; // 1 m/s forward
00046     twist.angular.z = 0.0;
00047
00048     star::v1::VelocityCommand command;
00049     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00050
00051     // Both wheels should move at same speed
00052     EXPECT_NEAR(command.front_left_velocity_mps(), 1.0, TOLERANCE);
00053     EXPECT_NEAR(command.front_right_velocity_mps(), 1.0, TOLERANCE);
00054 }
00055
00056 TEST_F(MessageConverterTest, TwistToCommandPureRotation)
00057 {
00058     geometry_msgs::msg::Twist twist;
00059     twist.linear.x = 0.0;
00060     twist.angular.z = 1.0; // 1 rad/s counter-clockwise
00061
00062     star::v1::VelocityCommand command;
00063     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00064
00065     // Differential rotation: v_left = -omega*L/2, v_right = +omega*L/2
00066     double expected_velocity = twist.angular.z * WHEEL_BASE_M / 2.0; // omega * L/2 = 0.075 m/s
00067
00068     EXPECT_NEAR(command.front_left_velocity_mps(), -expected_velocity, TOLERANCE);
00069     EXPECT_NEAR(command.front_right_velocity_mps(), expected_velocity, TOLERANCE);
00070 }
00071
00072 TEST_F(MessageConverterTest, TwistToCommandMixedMotion)
00073 {
00074     geometry_msgs::msg::Twist twist;
00075     twist.linear.x = 1.0; // 1 m/s forward
00076     twist.angular.z = 2.0; // 2 rad/s counter-clockwise
00077
00078     star::v1::VelocityCommand command;
00079     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00080
00081     // v_left = v - omega*L/2
00082     // v_right = v + omega*L/2
00083     double rotational_component = 2.0 * WHEEL_BASE_M / 2.0; // 0.150 m/s
00084     double expected_left = 1.0 - rotational_component; // 0.850 m/s
00085     double expected_right = 1.0 + rotational_component; // 1.150 m/s
00086
00087     EXPECT_NEAR(command.front_left_velocity_mps(), expected_left, TOLERANCE);
00088     EXPECT_NEAR(command.front_right_velocity_mps(), expected_right, TOLERANCE);
00089 }
00090
00091 TEST_F(MessageConverterTest, TwistToCommandNegativeLinear)
00092 {
00093     geometry_msgs::msg::Twist twist;
00094     twist.linear.x = -0.5; // 0.5 m/s backward
00095     twist.angular.z = 0.0;
00096
00097     star::v1::VelocityCommand command;
00098     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00099
00100     // Both wheels should be negative (backward)
00101     EXPECT_NEAR(command.front_left_velocity_mps(), -0.5, TOLERANCE);
00102     EXPECT_NEAR(command.front_right_velocity_mps(), -0.5, TOLERANCE);
00103 }
00104
00105 TEST_F(MessageConverterTest, TwistToCommandNegativeAngular)
00106 {
00107     geometry_msgs::msg::Twist twist;
00108     twist.linear.x = 0.0;
00109     twist.angular.z = -1.0; // 1 rad/s clockwise
00110
00111     star::v1::VelocityCommand command;
00112     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00113
00114     // Clockwise rotation: left positive, right negative
00115     double expected_velocity = std::abs(twist.angular.z) * WHEEL_BASE_M / 2.0;
00116
00117     EXPECT_NEAR(command.front_left_velocity_mps(), expected_velocity, TOLERANCE);
00118     EXPECT_NEAR(command.front_right_velocity_mps(), -expected_velocity, TOLERANCE);
00119 }
00120
00121 // =====
00122 // Inverse Kinematics Tests (VelocityCommand -> Twist)
00123 // =====
00124

```

9.14 test_message_converter.cpp

97

```

00125 TEST_F(MessageConverterTest, CommandToTwistZeroVelocity)
00126 {
00127     star::v1::VelocityCommand command;
00128     command.set_front_left_velocity_mps(0.0);
00129     command.set_front_right_velocity_mps(0.0);
00130
00131     geometry_msgs::msg::Twist twist;
00132     ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00133
00134     EXPECT_NEAR(twist.linear.x, 0.0, TOLERANCE);
00135     EXPECT_NEAR(twist.angular.z, 0.0, TOLERANCE);
00136 }
00137
00138 TEST_F(MessageConverterTest, CommandToTwistPureTranslation)
00139 {
00140     star::v1::VelocityCommand command;
00141     command.set_front_left_velocity_mps(1.0);
00142     command.set_front_right_velocity_mps(1.0);
00143     // Also set back wheels for completeness
00144     command.set_back_left_velocity_mps(1.0);
00145     command.set_back_right_velocity_mps(1.0);
00146
00147     geometry_msgs::msg::Twist twist;
00148     ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00149
00150     // v = (v_left + v_right) / 2
00151     EXPECT_NEAR(twist.linear.x, 1.0, TOLERANCE);
00152     EXPECT_NEAR(twist.angular.z, 0.0, TOLERANCE);
00153 }
00154
00155 TEST_F(MessageConverterTest, CommandToTwistPureRotation)
00156 {
00157     // For pure rotation: v_left = -v_right
00158     double wheel_velocity = 0.075; // Arbitrary value
00159
00160     star::v1::VelocityCommand command;
00161     command.set_front_left_velocity_mps(-wheel_velocity);
00162     command.set_front_right_velocity_mps(wheel_velocity);
00163     command.set_back_left_velocity_mps(-wheel_velocity);
00164     command.set_back_right_velocity_mps(wheel_velocity);
00165
00166     geometry_msgs::msg::Twist twist;
00167     ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00168
00169     // omega = (v_right - v_left) / L
00170     double expected_angular = (wheel_velocity - (-wheel_velocity)) / WHEEL_BASE_M;
00171
00172     EXPECT_NEAR(twist.linear.x, 0.0, TOLERANCE);
00173     EXPECT_NEAR(twist.angular.z, expected_angular, TOLERANCE);
00174 }
00175
00176 TEST_F(MessageConverterTest, CommandToTwistMixedMotion)
00177 {
00178     star::v1::VelocityCommand command;
00179     command.set_front_left_velocity_mps(0.5);
00180     command.set_front_right_velocity_mps(1.5);
00181     command.set_back_left_velocity_mps(0.5);
00182     command.set_back_right_velocity_mps(1.5);
00183
00184     geometry_msgs::msg::Twist twist;
00185     ASSERT_TRUE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00186
00187     // v = (v_left + v_right) / 2 = (0.5 + 1.5) / 2 = 1.0
00188     // omega = (v_right - v_left) / L = (1.5 - 0.5) / 0.150 = 6.667
00189     double expected_linear = (0.5 + 1.5) / 2.0;
00190     double expected_angular = (1.5 - 0.5) / WHEEL_BASE_M;
00191
00192     EXPECT_NEAR(twist.linear.x, expected_linear, TOLERANCE);
00193     EXPECT_NEAR(twist.angular.z, expected_angular, TOLERANCE);
00194 }
00195
00196 // =====
00197 // Kinematics Roundtrip Tests (Twist -> Command -> Twist)
00198 // =====
00199
00200 TEST_F(MessageConverterTest, KinematicsRoundtripZero)
00201 {
00202     geometry_msgs::msg::Twist original_twist;
00203     original_twist.linear.x = 0.0;
00204     original_twist.angular.z = 0.0;
00205
00206     star::v1::VelocityCommand command;
00207     ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00208
00209     geometry_msgs::msg::Twist reconstructed_twist;
00210     ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00211

```

Generated by Doxygen

```
00212 EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00213 EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00214 }
00215
00216 TEST_F(MessageConverterTest, KinematicsRoundtripTranslation)
00217 {
00218     geometry_msgs::msg::Twist original_twist;
00219     original_twist.linear.x = 1.23;
00220     original_twist.angular.z = 0.0;
00221
00222     star::v1::VelocityCommand command;
00223     ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00224
00225     geometry_msgs::msg::Twist reconstructed_twist;
00226     ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00227
00228     EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00229     EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00230 }
00231
00232 TEST_F(MessageConverterTest, KinematicsRoundtripRotation)
00233 {
00234     geometry_msgs::msg::Twist original_twist;
00235     original_twist.linear.x = 0.0;
00236     original_twist.angular.z = 2.5;
00237
00238     star::v1::VelocityCommand command;
00239     ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00240
00241     geometry_msgs::msg::Twist reconstructed_twist;
00242     ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00243
00244     EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00245     EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00246 }
00247
00248 TEST_F(MessageConverterTest, KinematicsRoundtripMixed)
00249 {
00250     geometry_msgs::msg::Twist original_twist;
00251     original_twist.linear.x = 0.75;
00252     original_twist.angular.z = -1.5;
00253
00254     star::v1::VelocityCommand command;
00255     ASSERT_TRUE(converter_.twist_to_velocity_command(original_twist, command, WHEEL_BASE_M));
00256
00257     geometry_msgs::msg::Twist reconstructed_twist;
00258     ASSERT_TRUE(converter_.velocity_command_to_twist(command, reconstructed_twist, WHEEL_BASE_M));
00259
00260     EXPECT_NEAR(reconstructed_twist.linear.x, original_twist.linear.x, TOLERANCE);
00261     EXPECT_NEAR(reconstructed_twist.angular.z, original_twist.angular.z, TOLERANCE);
00262 }
00263
00264 // =====
00265 // Validation Tests (NaN, Infinity, Invalid Values)
00266 // =====
00267
00268 TEST_F(MessageConverterTest, TwistToCommandNaNLinear)
00269 {
00270     geometry_msgs::msg::Twist twist;
00271     twist.linear.x = std::numeric_limits<double>::quiet_NaN();
00272     twist.angular.z = 0.0;
00273
00274     star::v1::VelocityCommand command;
00275     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00276 }
00277
00278 TEST_F(MessageConverterTest, TwistToCommandNaNAngular)
00279 {
00280     geometry_msgs::msg::Twist twist;
00281     twist.linear.x = 0.0;
00282     twist.angular.z = std::numeric_limits<double>::quiet_NaN();
00283
00284     star::v1::VelocityCommand command;
00285     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00286 }
00287
00288 TEST_F(MessageConverterTest, TwistToCommandInfinityLinear)
00289 {
00290     geometry_msgs::msg::Twist twist;
00291     twist.linear.x = std::numeric_limits<double>::infinity();
00292     twist.angular.z = 0.0;
00293
00294     star::v1::VelocityCommand command;
00295     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00296 }
00297
00298 TEST_F(MessageConverterTest, TwistToCommandInfinityAngular)
```


9.14 test_message_converter.cpp

99

```

00299 {
00300     geometry_msgs::msg::Twist twist;
00301     twist.linear.x = 0.0;
00302     twist.angular.z = std::numeric_limits<double>::infinity();
00303
00304     star::v1::VelocityCommand command;
00305     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00306 }
00307
00308 TEST_F(MessageConverterTest, TwistToCommandNegativeInfinity)
00309 {
00310     geometry_msgs::msg::Twist twist;
00311     twist.linear.x = -std::numeric_limits<double>::infinity();
00312     twist.angular.z = 0.0;
00313
00314     star::v1::VelocityCommand command;
00315     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00316 }
00317
00318 TEST_F(MessageConverterTest, TwistToCommandZeroWheelBase)
00319 {
00320     geometry_msgs::msg::Twist twist;
00321     twist.linear.x = 1.0;
00322     twist.angular.z = 0.0;
00323
00324     star::v1::VelocityCommand command;
00325     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, 0.0));
00326 }
00327
00328 TEST_F(MessageConverterTest, TwistToCommandNegativeWheelBase)
00329 {
00330     geometry_msgs::msg::Twist twist;
00331     twist.linear.x = 1.0;
00332     twist.angular.z = 0.0;
00333
00334     star::v1::VelocityCommand command;
00335     ASSERT_FALSE(converter_.twist_to_velocity_command(twist, command, -0.150));
00336 }
00337
00338 TEST_F(MessageConverterTest, CommandToTwistNaNMotor0)
00339 {
00340     star::v1::VelocityCommand command;
00341     command.set_front_left_velocity_mps(std::numeric_limits<double>::quiet_NaN());
00342     command.set_front_right_velocity_mps(0.0);
00343
00344     geometry_msgs::msg::Twist twist;
00345     ASSERT_FALSE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00346 }
00347
00348 TEST_F(MessageConverterTest, CommandToTwistNaNMotor1)
00349 {
00350     star::v1::VelocityCommand command;
00351     command.set_front_left_velocity_mps(0.0);
00352     command.set_front_right_velocity_mps(std::numeric_limits<double>::quiet_NaN());
00353
00354     geometry_msgs::msg::Twist twist;
00355     ASSERT_FALSE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00356 }
00357
00358 TEST_F(MessageConverterTest, CommandToTwistInfinityMotor0)
00359 {
00360     star::v1::VelocityCommand command;
00361     command.set_front_left_velocity_mps(std::numeric_limits<double>::infinity());
00362     command.set_front_right_velocity_mps(0.0);
00363
00364     geometry_msgs::msg::Twist twist;
00365     ASSERT_FALSE(converter_.velocity_command_to_twist(command, twist, WHEEL_BASE_M));
00366 }
00367
00368 // =====
00369 // PID Configuration Tests (Proto -> ROS2 direction)
00370 // =====
00371
00372 TEST_F(MessageConverterTest, PidConfigToGainsNominal)
00373 {
00374     star::v1::PidConfig proto_config;
00375     proto_config.set_kp(1.5);
00376     proto_config.set_ki(0.3);
00377     proto_config.set_kd(0.05);
00378
00379     double kp, ki, kd;
00380     ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00381
00382     EXPECT_NEAR(kp, 1.5, TOLERANCE);
00383     EXPECT_NEAR(ki, 0.3, TOLERANCE);
00384     EXPECT_NEAR(kd, 0.05, TOLERANCE);
00385 }

```

Generated by Doxygen

```

00386
00387 TEST_F(MessageConverterTest, PidConfigToGainsZeroGains)
00388 {
00389     star::v1::PidConfig proto_config;
00390     proto_config.set_kp(0.0);
00391     proto_config.set_ki(0.0);
00392     proto_config.set_kd(0.0);
00393
00394     double kp, ki, kd;
00395     ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00396
00397     EXPECT_NEAR(kp, 0.0, TOLERANCE);
00398     EXPECT_NEAR(ki, 0.0, TOLERANCE);
00399     EXPECT_NEAR(kd, 0.0, TOLERANCE);
00400 }
00401
00402 TEST_F(MessageConverterTest, PidConfigToGainsNaN)
00403 {
00404     star::v1::PidConfig proto_config;
00405     proto_config.set_kp(std::numeric_limits<double>::quiet_NaN());
00406     proto_config.set_ki(0.3);
00407     proto_config.set_kd(0.05);
00408
00409     double kp, ki, kd;
00410     ASSERT_FALSE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00411 }
00412
00413 TEST_F(MessageConverterTest, PidConfigToGainsInfinity)
00414 {
00415     star::v1::PidConfig proto_config;
00416     proto_config.set_kp(1.5);
00417     proto_config.set_ki(std::numeric_limits<double>::infinity());
00418     proto_config.set_kd(0.05);
00419
00420     double kp, ki, kd;
00421     ASSERT_FALSE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00422 }
00423
00424 TEST_F(MessageConverterTest, PidConfigToGainsNegativeGains)
00425 {
00426     // Negative PID gains are mathematically valid (though unusual)
00427     star::v1::PidConfig proto_config;
00428     proto_config.set_kp(-1.5);
00429     proto_config.set_ki(-0.3);
00430     proto_config.set_kd(-0.05);
00431
00432     double kp, ki, kd;
00433     // Implementation only checks for NaN/infinity, not negative values
00434     ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00435
00436     EXPECT_NEAR(kp, -1.5, TOLERANCE);
00437     EXPECT_NEAR(ki, -0.3, TOLERANCE);
00438     EXPECT_NEAR(kd, -0.05, TOLERANCE);
00439 }
00440
00441 TEST_F(MessageConverterTest, PidConfigToGainsLargeValues)
00442 {
00443     star::v1::PidConfig proto_config;
00444     proto_config.set_kp(1000.0);
00445     proto_config.set_ki(500.0);
00446     proto_config.set_kd(100.0);
00447
00448     double kp, ki, kd;
00449     ASSERT_TRUE(converter_.pid_config_to_gains(proto_config, kp, ki, kd));
00450
00451     EXPECT_NEAR(kp, 1000.0, TOLERANCE);
00452     EXPECT_NEAR(ki, 500.0, TOLERANCE);
00453     EXPECT_NEAR(kd, 100.0, TOLERANCE);
00454 }
00455 // =====
00456 // Edge Case Tests
00457 // =====
00458
00459
00460 TEST_F(MessageConverterTest, TwistToCommandMaxVelocity)
00461 {
00462     // Test with high velocities (near physical limits)
00463     geometry_msgs::msg::Twist twist;
00464     twist.linear.x = 10.0; // 10 m/s (unrealistic but valid)
00465     twist.angular.z = 20.0; // 20 rad/s
00466
00467     star::v1::VelocityCommand command;
00468     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, WHEEL_BASE_M));
00469
00470     // Should not crash or produce NaN
00471     EXPECT_FALSE(std::isnan(command.front_left_velocity_mps()));
00472     EXPECT_FALSE(std::isnan(command.front_right_velocity_mps()));

```

9.15 src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp File Reference

101

```

00473 EXPECT_FALSE(std::isinf(command.front_left_velocity_mps()));
00474 EXPECT_FALSE(std::isinf(command.front_right_velocity_mps()));
00475 }
00476
00477 TEST_F(MessageConverterTest, TwistToCommandVerySmallWheelBase)
00478 {
00479     geometry_msgs::msg::Twist twist;
00480     twist.linear.x = 1.0;
00481     twist.angular.z = 1.0;
00482
00483     star::v1::VelocityCommand command;
00484     // Very small but positive wheel base (1mm)
00485     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, 0.001));
00486
00487     // Should produce large angular velocities but not overflow
00488     EXPECT_FALSE(std::isinf(command.front_left_velocity_mps()));
00489     EXPECT_FALSE(std::isinf(command.front_right_velocity_mps()));
00490 }
00491
00492 TEST_F(MessageConverterTest, TwistToCommandVeryLargeWheelBase)
00493 {
00494     geometry_msgs::msg::Twist twist;
00495     twist.linear.x = 1.0;
00496     twist.angular.z = 1.0;
00497
00498     star::v1::VelocityCommand command;
00499     // Large wheel base (10m)
00500     ASSERT_TRUE(converter_.twist_to_velocity_command(twist, command, 10.0));
00501
00502     EXPECT_FALSE(std::isnan(command.front_left_velocity_mps()));
00503     EXPECT_FALSE(std::isnan(command.front_right_velocity_mps()));
00504 }
00505
00506 } // namespace star
00507
00508 int main(int argc, char **argv)
00509 {
00510     testing::InitGoogleTest(&argc, argv);
00511     return RUN_ALL_TESTS();
00512 }

```

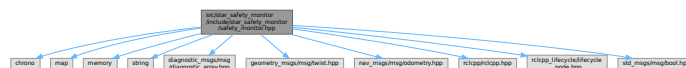
9.15 src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp File Reference

```

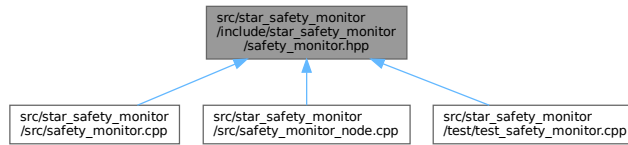
#include <chrono>
#include <map>
#include <memory>
#include <string>
#include <diagnostic_msgs/msg/diagnostic_array.hpp>
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_lifecycle/lifecycle_node.hpp>
#include <std_msgs/msg/bool.hpp>

```

Include dependency graph for safety_monitor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `star_safety_monitor::SafetyMonitor`

Namespaces

- namespace `star_safety_monitor`

Enumerations

- enum class `star_safety_monitor::SeverityLevel` { `star_safety_monitor::OK` = 0 , `star_safety_monitor::WARN` = 1 , `star_safety_monitor::ERROR` = 2 , `star_safety_monitor::STALE` = 3 }

9.16 safety_monitor.hpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
00010 //
00011 // The above copyright notice and this permission notice shall be included in
00012 // all copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #ifndef STAR_SAFETY_MONITOR__SAFETY_MONITOR_HPP_
00023 #define STAR_SAFETY_MONITOR__SAFETY_MONITOR_HPP_
00024
00025 #include <chrono>
00026 #include <map>
00027 #include <memory>
00028 #include <string>
00029
00030 #include <diagnostic_msgs/msg/diagnostic_array.hpp>
00031 #include <geometry_msgs/msg/twist.hpp>
00032 #include <nav_msgs/msg/odometry.hpp>
00033 #include <rclcpp/rclcpp.hpp>

```

Generated by Doxygen

9.16 safety_monitor.hpp

103

```

00034 #include <rcldcpp_lifecycle/lifecycle_node.hpp>
00035 #include <std_msgs/msg/bool.hpp>
00036
00037 namespace star_safety_monitor
00038 {
00039
00040 // Enum for safety severity levels
00041 enum class SeverityLevel { OK = 0, WARN = 1, ERROR = 2, STALE = 3 };
00042
00043 class SafetyMonitor : public rcldcpp_lifecycle::LifecycleNode {
00044 public:
00045     explicit SafetyMonitor(
00046         const rcldcpp::NodeOptions & options = rcldcpp::NodeOptions());
00047     ~SafetyMonitor();
00048
00049 // Lifecycle callbacks
00050 rcldcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00051 on_configure(const rcldcpp_lifecycle::State & previous_state) override;
00052
00053 rcldcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00054 on_activate(const rcldcpp_lifecycle::State & previous_state) override;
00055
00056 rcldcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00057 on_deactivate(const rcldcpp_lifecycle::State & previous_state) override;
00058
00059 rcldcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00060 on_cleanup(const rcldcpp_lifecycle::State & previous_state) override;
00061
00062 rcldcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00063 on_shutdown(const rcldcpp_lifecycle::State & previous_state) override;
00064
00065 private:
00066 // Subscription callbacks
00067 void odometry_callback(const nav_msgs::msg::Odometry::SharedPtr msg);
00068 void diagnostics_callback(
00069     const diagnostic_msgs::msg::DiagnosticArray::SharedPtr msg);
00070 void cmd_vel_callback(const geometry_msgs::msg::Twist::SharedPtr msg);
00071
00072 // Timer callback for monitoring and publishing diagnostics
00073 void monitoring_timer_callback();
00074
00075 // Monitoring and safety check methods
00076 void check_heartbeat_health();
00077 void check_velocity_limits();
00078 void check_motor_stall();
00079 void check_diagnostic_health();
00080 void update_overall_state();
00081 void publish_diagnostics();
00082 double compute_linear_magnitude(const geometry_msgs::msg::Twist & twist) const;
00083
00084 // Helper methods
00085 void add_diagnostic_status(
00086     diagnostic_msgs::msg::DiagnosticStatus & status,
00087     const std::string & name, SeverityLevel level,
00088     const std::string & message);
00089
00090 // Publishers and Subscribers
00091 rcldcpp::Subscription<nav_msgs::msg::Odometry>::SharedPtr odom_sub_;
00092 rcldcpp::Subscription<diagnostic_msgs::msg::DiagnosticArray>::SharedPtr
00093     diag_sub_;
00094 rcldcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_vel_sub_;
00095
00096 rcldcpp_lifecycle::LifecyclePublisher<
00097     diagnostic_msgs::msg::DiagnosticArray>::SharedPtr diagnostics_pub_;
00098 rcldcpp_lifecycle::LifecyclePublisher<std_msgs::msg::Bool>::SharedPtr
00099     emergency_stop_pub_;
00100
00101 // Timer
00102 rcldcpp::TimerBase::SharedPtr monitoring_timer_;
00103
00104 // Safety parameters (declared in on_configure)
00105 int heartbeat_timeout_ms_;
00106 double max_linear_velocity_;
00107 double max_angular_velocity_;
00108 double publish_rate_;
00109 bool enable_auto_estop_;
00110 double estop_recovery_delay_;
00111 double stall_detection_threshold_;
00112 int stall_samples_required_;
00113 int cmd_vel_timeout_ms_;
00114
00115 // State tracking
00116 std::chrono::system_clock::time_point last_diagnostics_time_;
00117 std::map<std::string, std::chrono::system_clock::time_point> heartbeat_times_;
00118 double current_linear_velocity_{0.0};
00119 double current_angular_velocity_{0.0};
00120 bool velocity_exceeded_{false};

```

Generated by Doxygen

```

00121 bool heartbeat_timeout_triggered_{false};
00122 bool emergency_stop_active_{false};
00123 std::chrono::system_clock::time_point estop_trigger_time_;
00124
00125 // Motor stall tracking
00126 geometry_msgs::msg::Twist last_cmd_vel_;
00127 std::chrono::system_clock::time_point last_cmd_vel_time_;
00128 int stall_detection_count_{0};
00129 bool motor_stall_detected_{false};
00130
00131 SeverityLevel overall_severity_{SeverityLevel::OK};
00132 };
00133
00134 } // namespace star_safety_monitor
00135
00136 #endif // STAR_SAFETY_MONITOR__SAFETY_MONITOR_HPP_

```

9.17 src/star_safety_monitor/src/safety_monitor.cpp File Reference

```

#include "star_safety_monitor/safety_monitor.hpp"
#include <cmath>
#include <iomanip>
#include <sstream>

```

Include dependency graph for safety_monitor.cpp:



Namespaces

- namespace [star_safety_monitor](#)

9.18 safety_monitor.cpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
00010 //
00011 // The above copyright notice and this permission notice shall be included in all
00012 // copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #include "star_safety_monitor/safety_monitor.hpp"
00023 #include <cmath>
00024 #include <iomanip>
00025 #include <sstream>

```

9.18 safety_monitor.cpp

105

```

00026
00027 namespace star_safety_monitor
00028 {
00029
00030 SafetyMonitor::SafetyMonitor(const rclcpp::NodeOptions & options)
00031 : rclcpp_lifecycle::LifecycleNode("safety_monitor", options)
00032 {
00033     RCLCPP_INFO(get_logger(), "SafetyMonitor constructor called");
00034 }
00035
00036 SafetyMonitor::~SafetyMonitor()
00037 {
00038     RCLCPP_INFO(get_logger(), "SafetyMonitor destructor called");
00039 }
00040
00041 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00042 SafetyMonitor::on_configure(const rclcpp_lifecycle::State & previous_state)
00043 {
00044     (void)previous_state;
00045     RCLCPP_INFO(get_logger(), "Configuring SafetyMonitor");
00046
00047     try {
00048         // Declare and get parameters
00049         declare_parameter("heartbeat_timeout_ms", 500);
00050         declare_parameter("max_linear_velocity", 1.0);
00051         declare_parameter("max_angular_velocity", 2.0);
00052         declare_parameter("publish_rate", 10.0);
00053         declare_parameter("enable_auto_estop", true);
00054         declare_parameter("estop_recovery_delay", 5.0);
00055         declare_parameter("stall_detection_threshold", 0.05);
00056         declare_parameter("stall_samples_required", 5);
00057         declare_parameter("cmd_vel_timeout_ms", 500);
00058
00059         heartbeat_timeout_ms_ = get_parameter("heartbeat_timeout_ms").as_int();
00060         max_linear_velocity_ = get_parameter("max_linear_velocity").as_double();
00061         max_angular_velocity_ = get_parameter("max_angular_velocity").as_double();
00062         publish_rate_ = get_parameter("publish_rate").as_double();
00063         enable_auto_estop_ = get_parameter("enable_auto_estop").as_bool();
00064         estop_recovery_delay_ = get_parameter("estop_recovery_delay").as_double();
00065         stall_detection_threshold_ = get_parameter("stall_detection_threshold").as_double();
00066         stall_samples_required_ = get_parameter("stall_samples_required").as_int();
00067         cmd_vel_timeout_ms_ = get_parameter("cmd_vel_timeout_ms").as_int();
00068
00069         RCLCPP_INFO(get_logger(), "Configuration loaded:");
00070         RCLCPP_INFO(get_logger(), "  Heartbeat timeout: %d ms", heartbeat_timeout_ms_);
00071         RCLCPP_INFO(get_logger(), "  Max linear velocity: %.2f m/s", max_linear_velocity_);
00072         RCLCPP_INFO(get_logger(), "  Max angular velocity: %.2f rad/s", max_angular_velocity_);
00073
00074         // Create lifecycle publishers
00075         diagnostics_pub_ =
00076             create_publisher<diagnostic_msgs::msg::DiagnosticArray>("/diagnostics", 10);
00077         emergency_stop_pub_ = create_publisher<std_msgs::msg::Bool>("/emergency_stop", 10);
00078
00079         // Create subscribers
00080         odom_sub_ = create_subscription<nav_msgs::msg::Odometry>(
00081             "/odom", 10,
00082             std::bind(&SafetyMonitor::odom_callback, this, std::placeholders::_1));
00083
00084         diag_sub_ = create_subscription<diagnostic_msgs::msg::DiagnosticArray>(
00085             "/diagnostics", 10,
00086             std::bind(&SafetyMonitor::diagnostics_callback, this, std::placeholders::_1));
00087
00088         cmd_vel_sub_ = create_subscription<geometry_msgs::msg::Twist>(
00089             "/cmd_vel", 10,
00090             std::bind(&SafetyMonitor::cmd_vel_callback, this, std::placeholders::_1));
00091
00092         // Initialize state
00093         last_diagnostics_time_ = std::chrono::system_clock::now();
00094         last_cmd_vel_time_ = std::chrono::system_clock::now();
00095         heartbeat_times_.clear();
00096         current_linear_velocity_ = 0.0;
00097         current_angular_velocity_ = 0.0;
00098         velocity_exceeded_ = false;
00099         heartbeat_timeout_triggered_ = false;
00100         emergency_stop_active_ = false;
00101         motor_stall_detected_ = false;
00102         stall_detection_count_ = 0;
00103         overall_severity_ = SeverityLevel::OK;
00104
00105         RCLCPP_INFO(get_logger(), "SafetyMonitor configured successfully");
00106         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00107     } catch (const std::exception & e) {
00108         RCLCPP_ERROR(get_logger(), "Configuration failed: %s", e.what());
00109         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00110     }
00111 }
00112

```

Generated by Doxygen

```

00113 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00114 SafetyMonitor::on_activate(const rclcpp_lifecycle::State & previous_state)
00115 {
00116     (void)previous_state;
00117     RCLCPP_INFO(get_logger(), "Activating SafetyMonitor");
00118
00119     try {
00120         // Activate publishers
00121         diagnostics_pub_->on_activate();
00122         emergency_stop_pub_->on_activate();
00123
00124         // Create monitoring timer (10 Hz by default)
00125         auto timer_period = std::chrono::milliseconds(
00126             static_cast<int>(1000.0 / publish_rate_));
00127         monitoring_timer_ =
00128             create_wall_timer(timer_period,
00129                 std::bind(&SafetyMonitor::monitoring_timer_callback, this));
00130
00131         RCLCPP_INFO(get_logger(), "SafetyMonitor activated successfully");
00132         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00133     } catch (const std::exception & e) {
00134         RCLCPP_ERROR(get_logger(), "Activation failed: %s", e.what());
00135         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00136     }
00137 }
00138
00139 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00140 SafetyMonitor::on_deactivate(const rclcpp_lifecycle::State & previous_state)
00141 {
00142     (void)previous_state;
00143     RCLCPP_INFO(get_logger(), "Deactivating SafetyMonitor");
00144
00145     try {
00146         // Cancel timer
00147         if (monitoring_timer_) {
00148             monitoring_timer_->cancel();
00149             monitoring_timer_.reset();
00150         }
00151
00152         // Deactivate publishers
00153         diagnostics_pub_->on_deactivate();
00154         emergency_stop_pub_->on_deactivate();
00155
00156         RCLCPP_INFO(get_logger(), "SafetyMonitor deactivated successfully");
00157         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00158     } catch (const std::exception & e) {
00159         RCLCPP_ERROR(get_logger(), "Deactivation failed: %s", e.what());
00160         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00161     }
00162 }
00163
00164 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00165 SafetyMonitor::on_cleanup(const rclcpp_lifecycle::State & previous_state)
00166 {
00167     (void)previous_state;
00168     RCLCPP_INFO(get_logger(), "Cleaning up SafetyMonitor");
00169
00170     try {
00171         // Reset publishers and subscribers
00172         diagnostics_pub_.reset();
00173         emergency_stop_pub_.reset();
00174         odom_sub_.reset();
00175         diag_sub_.reset();
00176         cmd_vel_sub_.reset();
00177
00178         // Clear state
00179         heartbeat_times_.clear();
00180
00181         RCLCPP_INFO(get_logger(), "SafetyMonitor cleaned up successfully");
00182         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00183     } catch (const std::exception & e) {
00184         RCLCPP_ERROR(get_logger(), "Cleanup failed: %s", e.what());
00185         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::FAILURE;
00186     }
00187 }
00188
00189 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00190 SafetyMonitor::on_shutdown(const rclcpp_lifecycle::State & previous_state)
00191 {
00192     (void)previous_state;
00193     RCLCPP_INFO(get_logger(), "Shutting down SafetyMonitor");
00194
00195     // Ensure emergency stop is triggered on shutdown if active
00196     if (emergency_stop_active_) {
00197         auto msg = std_msgs::msg::Bool();
00198         msg.data = true;
00199         emergency_stop_pub_->publish(msg);

```


9.18 safety_monitor.cpp

107

```

00200     }
00201
00202     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn::SUCCESS;
00203 }
00204
00205 void SafetyMonitor::odometry_callback(const nav_msgs::msg::Odometry::SharedPtr msg)
00206 {
00207     // Extract current velocities
00208     current_linear_velocity_ = std::sqrt(
00209         msg->twist.twist.linear.x * msg->twist.twist.linear.x +
00210         msg->twist.twist.linear.y * msg->twist.twist.linear.y +
00211         msg->twist.twist.linear.z * msg->twist.twist.linear.z);
00212
00213     current_angular_velocity_ = std::sqrt(
00214         msg->twist.twist.angular.x * msg->twist.twist.angular.x +
00215         msg->twist.twist.angular.y * msg->twist.twist.angular.y +
00216         msg->twist.twist.angular.z * msg->twist.twist.angular.z);
00217 }
00218
00219 void SafetyMonitor::diagnostics_callback(
00220     const diagnostic_msgs::msg::DiagnosticArray::SharedPtr msg)
00221 {
00222     last_diagnostics_time_ = std::chrono::system_clock::now();
00223
00224     // Update heartbeat times for known nodes
00225     for (const auto & status : msg->status) {
00226         heartbeat_times_[status.name] = std::chrono::system_clock::now();
00227     }
00228 }
00229
00230 void SafetyMonitor::cmd_vel_callback(
00231     const geometry_msgs::msg::Twist::SharedPtr msg)
00232 {
00233     last_cmd_vel_ = *msg;
00234     last_cmd_vel_time_ = std::chrono::system_clock::now();
00235 }
00236
00237 void SafetyMonitor::monitoring_timer_callback()
00238 {
00239     // Perform safety checks
00240     check_heartbeat_health();
00241     check_velocity_limits();
00242     check_motor_stall();
00243     check_diagnostic_health();
00244     update_overall_state();
00245     publish_diagnostics();
00246 }
00247
00248 void SafetyMonitor::check_heartbeat_health()
00249 {
00250     auto now = std::chrono::system_clock::now();
00251     auto timeout_duration = std::chrono::milliseconds(heartbeat_timeout_ms_);
00252
00253     // Check for stale diagnostics
00254     auto time_since_diag = now - last_diagnostics_time_;
00255     if (time_since_diag > timeout_duration) {
00256         if (!heartbeat_timeout_triggered_) {
00257             RCLCPP_WARN(get_logger(), "Heartbeat timeout detected!");
00258             heartbeat_timeout_triggered_ = true;
00259             if (enable_auto_estop_) {
00260                 emergency_stop_active_ = true;
00261                 estop_trigger_time_ = now;
00262             }
00263         }
00264     } else {
00265         // Reset timeout if heartbeat recovered
00266         if (heartbeat_timeout_triggered_) {
00267             auto estop_duration = now - estop_trigger_time_;
00268             if (estop_duration > std::chrono::duration<double>(estop_recovery_delay_)) {
00269                 RCLCPP_INFO(get_logger(), "Heartbeat recovered");
00270                 heartbeat_timeout_triggered_ = false;
00271                 emergency_stop_active_ = false;
00272             }
00273         }
00274     }
00275 }
00276
00277 void SafetyMonitor::check_velocity_limits()
00278 {
00279     velocity_exceeded_ = false;
00280
00281     if (current_linear_velocity_ > max_linear_velocity_) {
00282         RCLCPP_WARN(get_logger(),
00283             "Linear velocity limit exceeded: %.2f > %.2f m/s",
00284             current_linear_velocity_, max_linear_velocity_);
00285         velocity_exceeded_ = true;
00286     }

```

Generated by Doxygen

```

00287
00288     if (current_angular_velocity_ > max_angular_velocity_) {
00289         RCLCPP_WARN(get_logger(),
00290             "Angular velocity limit exceeded: %.2f > %.2f rad/s",
00291             current_angular_velocity_, max_angular_velocity_);
00292         velocity_exceeded_ = true;
00293     }
00294 }
00295
00296 void SafetyMonitor::check_motor_stall()
00297 {
00298     motor_stall_detected_ = false;
00299
00300     // Stall detection: command velocity present but actual velocity is near zero
00301     double cmd_linear = compute_linear_magnitude(last_cmd_vel_);
00302
00303     // Check if a command was issued recently
00304     auto now = std::chrono::system_clock::now();
00305     auto cmd_age = now - last_cmd_vel_time_;
00306
00307     if (cmd_age < std::chrono::milliseconds(cmd_vel_timeout_ms_) &&
00308         cmd_linear > stall_detection_threshold_)
00309     {
00310         // We have a recent command to move
00311         if (current_linear_velocity_ < stall_detection_threshold_) {
00312             // But we're not actually moving
00313             stall_detection_count_++;
00314
00315             if (stall_detection_count_ >= stall_samples_required_) {
00316                 RCLCPP_WARN(get_logger(),
00317                     "Motor stall detected: cmd=%.2f m/s, actual=%.2f m/s",
00318                     cmd_linear, current_linear_velocity_);
00319                 motor_stall_detected_ = true;
00320                 if (enable_auto_estop_) {
00321                     emergency_stop_active_ = true;
00322                     estop_trigger_time_ = now;
00323                 }
00324             }
00325         } else {
00326             // Motor is responding, reset counter
00327             stall_detection_count_ = 0;
00328         }
00329     } else {
00330         // No recent command, reset counter
00331         stall_detection_count_ = 0;
00332     }
00333 }
00334
00335 void SafetyMonitor::check_diagnostic_health()
00336 {
00337     // Additional diagnostic checks can be added here
00338     // For now, this is a placeholder for future expansion
00339 }
00340
00341 void SafetyMonitor::update_overall_state()
00342 {
00343     // Determine overall severity level
00344     overall_severity_ = SeverityLevel::OK;
00345
00346     if (heartbeat_timeout_triggered_) {
00347         overall_severity_ = SeverityLevel::ERROR;
00348     } else if (motor_stall_detected_) {
00349         overall_severity_ = SeverityLevel::WARN;
00350     } else if (velocity_exceeded_) {
00351         overall_severity_ = SeverityLevel::WARN;
00352     }
00353
00354     // Publish emergency stop signal if needed
00355     if (emergency_stop_active_) {
00356         auto msg = std_msgs::msg::Bool();
00357         msg.data = true;
00358         emergency_stop_pub_>publish(msg);
00359     }
00360 }
00361
00362 void SafetyMonitor::publish_diagnostics()
00363 {
00364     auto diag_array = diagnostic_msgs::msg::DiagnosticArray();
00365     diag_array.header.stamp = now();
00366
00367     // System health status
00368     diagnostic_msgs::msg::DiagnosticStatus system_status;
00369     system_status.name = "safety_monitor: System Health";
00370     system_status.hardware_id = "safety_monitor";
00371
00372     if (overall_severity_ == SeverityLevel::OK) {
00373         system_status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;

```

9.18 safety_monitor.cpp

109

```

00374     system_status.message = "All systems nominal";
00375 } else if (overall_severity_ == SeverityLevel::WARN) {
00376     system_status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00377     system_status.message = "Warning conditions detected";
00378 } else if (overall_severity_ == SeverityLevel::ERROR) {
00379     system_status.level = diagnostic_msgs::msg::DiagnosticStatus::ERROR;
00380     system_status.message = "Critical error - emergency stop triggered";
00381 }
00382
00383 // Add velocity data
00384 diagnostic_msgs::msg::KeyValue kv;
00385 kv.key = "Linear Velocity (m/s)";
00386 kv.value = std::to_string(current_linear_velocity_);
00387 system_status.values.push_back(kv);
00388
00389 kv.key = "Angular Velocity (rad/s)";
00390 kv.value = std::to_string(current_angular_velocity_);
00391 system_status.values.push_back(kv);
00392
00393 kv.key = "Velocity Limit Exceeded";
00394 kv.value = velocity_exceeded_ ? "true" : "false";
00395 system_status.values.push_back(kv);
00396
00397 kv.key = "Motor Stall Detected";
00398 kv.value = motor_stall_detected_ ? "true" : "false";
00399 system_status.values.push_back(kv);
00400
00401 kv.key = "Heartbeat Timeout";
00402 kv.value = heartbeat_timeout_triggered_ ? "true" : "false";
00403 system_status.values.push_back(kv);
00404
00405 kv.key = "Emergency Stop Active";
00406 kv.value = emergency_stop_active_ ? "true" : "false";
00407 system_status.values.push_back(kv);
00408
00409 // Add heartbeat information for each tracked node
00410 diagnostic_msgs::msg::DiagnosticStatus heartbeat_status;
00411 heartbeat_status.name = "safety_monitor: Heartbeat Status";
00412 heartbeat_status.hardware_id = "safety_monitor";
00413 heartbeat_status.level = heartbeat_timeout_triggered_ ?
00414     diagnostic_msgs::msg::DiagnosticStatus::ERROR :
00415     diagnostic_msgs::msg::DiagnosticStatus::OK;
00416
00417 auto now_time = std::chrono::system_clock::now();
00418 for (const auto & [node_name, last_time] : heartbeat_times_) {
00419     auto duration_ms = std::chrono::duration_cast<std::chrono::milliseconds>(
00420         now_time - last_time).count();
00421
00422     kv.key = node_name;
00423     kv.value = std::to_string(duration_ms) + " ms";
00424     heartbeat_status.values.push_back(kv);
00425 }
00426
00427 diag_array.status.push_back(system_status);
00428 diag_array.status.push_back(heartbeat_status);
00429
00430 // Motor status
00431 diagnostic_msgs::msg::DiagnosticStatus motor_status;
00432 motor_status.name = "safety_monitor: Motor Status";
00433 motor_status.hardware_id = "safety_monitor";
00434 if (motor_stall_detected_) {
00435     motor_status.level = diagnostic_msgs::msg::DiagnosticStatus::WARN;
00436     motor_status.message = "Motor stall detected";
00437 } else {
00438     motor_status.level = diagnostic_msgs::msg::DiagnosticStatus::OK;
00439     motor_status.message = "Motor nominal";
00440 }
00441
00442 kv.key = "Stall Detected";
00443 kv.value = motor_stall_detected_ ? "true" : "false";
00444 motor_status.values.push_back(kv);
00445
00446 kv.key = "Command Velocity (m/s)";
00447 double cmd_linear = compute_linear_magnitude(last_cmd_vel_);
00448 kv.value = std::to_string(cmd_linear);
00449 motor_status.values.push_back(kv);
00450
00451 kv.key = "Actual Velocity (m/s)";
00452 kv.value = std::to_string(current_linear_velocity_);
00453 motor_status.values.push_back(kv);
00454
00455 kv.key = "Stall Detection Count";
00456 kv.value = std::to_string(stall_detection_count_);
00457 motor_status.values.push_back(kv);
00458
00459 diag_array.status.push_back(motor_status);
00460

```

Generated by Doxygen

```

00461   diagnostics_pub_>publish(diag_array);
00462 }
00463
00464 double SafetyMonitor::compute_linear_magnitude(const geometry_msgs::msg::Twist & twist) const
00465 {
00466     return std::sqrt(
00467         twist.linear.x * twist.linear.x +
00468         twist.linear.y * twist.linear.y +
00469         twist.linear.z * twist.linear.z);
00470 }
00471
00472 } // namespace star_safety_monitor

```

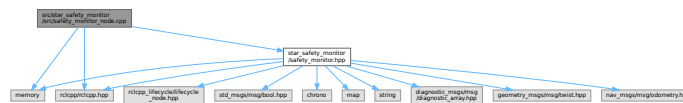
9.19 src/star_safety_monitor/src/safety_monitor_node.cpp File Reference

```

#include <memory>
#include <rclcpp/rclcpp.hpp>
#include "star_safety_monitor/safety_monitor.hpp"

```

Include dependency graph for safety_monitor_node.cpp:



Functions

- int [main](#) (int argc, char **argv)

9.19.1 Function Documentation

9.19.1.1 main()

```

int main (
    int argc,
    char ** argv)

```

Definition at line 26 of file [safety_monitor_node.cpp](#).

9.20 safety_monitor_node.cpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:

```

9.21 src/star_safety_monitor/test/test_safety_monitor.cpp File Reference

111

```

00010 //
00011 // The above copyright notice and this permission notice shall be included in all
00012 // copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #include <memory>
00023 #include <rclcpp/rclcpp.hpp>
00024 #include "star_safety_monitor/safety_monitor.hpp"
00025
00026 int main(int argc, char ** argv)
00027 {
00028     rclcpp::init(argc, argv);
00029
00030     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00031
00032     rclcpp::spin(node->get_node_base_interface());
00033
00034     rclcpp::shutdown();
00035     return 0;
00036 }

```

9.21 src/star_safety_monitor/test/test_safety_monitor.cpp File Reference

```

#include <gtest/gtest.h>
#include <chrono>
#include <thread>
#include <rclcpp/rclcpp.hpp>
#include "star_safety_monitor/safety_monitor.hpp"
#include <nav_msgs/msg/odometry.hpp>
#include <diagnostic_msgs/msg/diagnostic_array.hpp>
#include <diagnostic_msgs/msg/diagnostic_status.hpp>
#include <std_msgs/msg/bool.hpp>

```

Include dependency graph for test_safety_monitor.cpp:



Classes

- class [SafetyMonitorTest](#)

Functions

- [TEST_F](#) ([SafetyMonitorTest](#), NodeConstruction)
- [TEST_F](#) ([SafetyMonitorTest](#), LifecycleConfiguration)
- [TEST_F](#) ([SafetyMonitorTest](#), LifecycleActivation)
- [TEST_F](#) ([SafetyMonitorTest](#), LifecycleDeactivation)
- [TEST_F](#) ([SafetyMonitorTest](#), ParameterLoading)

Generated by Doxygen

- [TEST_F \(SafetyMonitorTest, DiagnosticsPublication\)](#)
- [TEST_F \(SafetyMonitorTest, OdometrySubscription\)](#)
- [TEST_F \(SafetyMonitorTest, EmergencyStopTrigger\)](#)
- [TEST_F \(SafetyMonitorTest, DiagnosticPublishing\)](#)
- [int main \(int argc, char **argv\)](#)

9.21.1 Function Documentation

9.21.1.1 main()

```
int main (  
    int argc,  
    char ** argv)
```

Definition at line 204 of file [test_safety_monitor.cpp](#).

9.21.1.2 TEST_F() [1/9]

```
TEST_F (  
    SafetyMonitorTest ,  
    DiagnosticPublishing )
```

Definition at line 198 of file [test_safety_monitor.cpp](#).

9.21.1.3 TEST_F() [2/9]

```
TEST_F (  
    SafetyMonitorTest ,  
    DiagnosticsPublication )
```

Definition at line 123 of file [test_safety_monitor.cpp](#).

9.21.1.4 TEST_F() [3/9]

```
TEST_F (  
    SafetyMonitorTest ,  
    EmergencyStopTrigger )
```

Definition at line 192 of file [test_safety_monitor.cpp](#).

9.21.1.5 TEST_F() [4/9]

```
TEST_F (  
    SafetyMonitorTest ,  
    LifecycleActivation )
```

Definition at line 74 of file [test_safety_monitor.cpp](#).

9.21 src/star_safety_monitor/test/test_safety_monitor.cpp File Reference**113****9.21.1.6 TEST_F() [5/9]**

```
TEST_F (
    SafetyMonitorTest ,
    LifecycleConfiguration )
```

Definition at line 64 of file [test_safety_monitor.cpp](#).

9.21.1.7 TEST_F() [6/9]

```
TEST_F (
    SafetyMonitorTest ,
    LifecycleDeactivation )
```

Definition at line 88 of file [test_safety_monitor.cpp](#).

9.21.1.8 TEST_F() [7/9]

```
TEST_F (
    SafetyMonitorTest ,
    NodeConstruction )
```

Definition at line 57 of file [test_safety_monitor.cpp](#).

9.21.1.9 TEST_F() [8/9]

```
TEST_F (
    SafetyMonitorTest ,
    OdometrySubscription )
```

Definition at line 154 of file [test_safety_monitor.cpp](#).

9.21.1.10 TEST_F() [9/9]

```
TEST_F (
    SafetyMonitorTest ,
    ParameterLoading )
```

Definition at line 104 of file [test_safety_monitor.cpp](#).

9.22 test_safety_monitor.cpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 STAR Team
00002 // SPDX-License-Identifier: MIT
00003 //
00004 // Permission is hereby granted, free of charge, to any person obtaining a copy
00005 // of this software and associated documentation files (the "Software"), to deal
00006 // in the Software without restriction, including without limitation the rights
00007 // to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00008 // copies of the Software, and to permit persons to whom the Software is
00009 // furnished to do so, subject to the following conditions:
00010 //
00011 // The above copyright notice and this permission notice shall be included in all
00012 // copies or substantial portions of the Software.
00013 //
00014 // THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00015 // IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00016 // FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
00017 // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
00018 // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
00019 // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
00020 // SOFTWARE.
00021
00022 #include <gtest/gtest.h>
00023 #include <chrono>
00024 #include <thread>
00025 #include <rclcpp/rclcpp.hpp>
00026 #include "star_safety_monitor/safety_monitor.hpp"
00027 #include <nav_msgs/msg/odometry.hpp>
00028 #include <diagnostic_msgs/msg/diagnostic_array.hpp>
00029 #include <diagnostic_msgs/msg/diagnostic_status.hpp>
00030 #include <std_msgs/msg/bool.hpp>
00031
00032 namespace
00033 {
00034     constexpr auto LIFECYCLE_TRANSITION_DELAY = std::chrono::milliseconds(100);
00035     constexpr auto SPIN_TIMEOUT = std::chrono::milliseconds(100);
00036     constexpr double MESSAGE_WAIT_TIMEOUT_S = 2.0;
00037 }
00038
00039 class SafetyMonitorTest : public ::testing::Test
00040 {
00041 protected:
00042     void SetUp() override
00043     {
00044         if (!rclcpp::ok()) {
00045             rclcpp::init(0, nullptr);
00046         }
00047     }
00048
00049     void TearDown() override
00050     {
00051         if (rclcpp::ok()) {
00052             rclcpp::shutdown();
00053         }
00054     }
00055 };
00056
00057 TEST_F(SafetyMonitorTest, NodeConstruction)
00058 {
00059     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00060     EXPECT_NE(node, nullptr);
00061     EXPECT_EQ(node->get_name(), std::string("safety_monitor"));
00062 }
00063
00064 TEST_F(SafetyMonitorTest, LifecycleConfiguration)
00065 {
00066     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00067
00068     // Test on_configure transition
00069     node->configure();
00070     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00071     EXPECT_EQ(node->get_current_state().label(), std::string("inactive"));
00072 }
00073
00074 TEST_F(SafetyMonitorTest, LifecycleActivation)
00075 {
00076     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00077
00078     // Configure
00079     node->configure();
00080     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00081
00082     // Activate

```


9.22 test_safety_monitor.cpp

115

```

00083     node->activate();
00084     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00085     EXPECT_EQ(node->get_current_state().label(), std::string("active"));
00086 }
00087
00088 TEST_F(SafetyMonitorTest, LifecycleDeactivation)
00089 {
00090     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00091
00092     // Configure and activate
00093     node->configure();
00094     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00095     node->activate();
00096     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00097
00098     // Deactivate
00099     node->deactivate();
00100     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00101     EXPECT_EQ(node->get_current_state().label(), std::string("inactive"));
00102 }
00103
00104 TEST_F(SafetyMonitorTest, ParameterLoading)
00105 {
00106     rclcpp::NodeOptions options;
00107     options.parameter_overrides({
00108         {"heartbeat_timeout_ms", 500},
00109         {"max_linear_velocity", 1.5},
00110         {"max_angular_velocity", 2.5},
00111         {"publish_rate", 20.0},
00112     });
00113
00114     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>(options);
00115     node->configure();
00116     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00117
00118     // Verify parameters were loaded
00119     auto hb_timeout = node->get_parameter("heartbeat_timeout_ms");
00120     EXPECT_EQ(hb_timeout.as_int(), 500);
00121 }
00122
00123 TEST_F(SafetyMonitorTest, DiagnosticsPublication)
00124 {
00125     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00126     node->configure();
00127     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00128     node->activate();
00129     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00130
00131     // Create a test subscriber to receive diagnostics
00132     std::atomic<int> diag_count{0};
00133     auto test_node = rclcpp::Node::make_shared("test_node");
00134     auto diag_sub = test_node->create_subscription<
00135         diagnostic_msgs::msg::DiagnosticArray>(
00136             "/diagnostics", 10,
00137             [&diag_count](const diagnostic_msgs::msg::DiagnosticArray::SharedPtr {
00138                 diag_count++;
00139             });
00140
00141     // Wait for a few diagnostic messages
00142     auto executor = rclcpp::executors::SingleThreadedExecutor();
00143     executor.add_node(test_node->get_node_base_interface());
00144     executor.add_node(node->get_node_base_interface());
00145
00146     rclcpp::Time start_time = node->now();
00147     while (diag_count < 2 && (node->now() - start_time).seconds() < MESSAGE_WAIT_TIMEOUT_S) {
00148         executor.spin_some(SPIN_TIMEOUT);
00149     }
00150
00151     EXPECT_GT(diag_count, 0);
00152 }
00153
00154 TEST_F(SafetyMonitorTest, OdometrySubscription)
00155 {
00156     auto node = std::make_shared<star_safety_monitor::SafetyMonitor>();
00157     node->configure();
00158     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00159     node->activate();
00160     std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00161
00162     // Create a test publisher for odometry
00163     auto test_node = rclcpp::Node::make_shared("test_odom_pub");
00164     auto odom_pub = test_node->create_publisher<nav_msgs::msg::Odometry>("/odom", 10);
00165
00166     // Create and publish odometry message
00167     auto odom_msg = nav_msgs::msg::Odometry();
00168     odom_msg.header.stamp = node->now();
00169     odom_msg.twist.twist.linear.x = 0.5;

```

Generated by Doxygen

```

00170 odom_msg.twist.twist.linear.y = 0.0;
00171 odom_msg.twist.twist.linear.z = 0.0;
00172 odom_msg.twist.twist.angular.x = 0.0;
00173 odom_msg.twist.twist.angular.y = 0.0;
00174 odom_msg.twist.twist.angular.z = 0.1;
00175
00176 auto executor = rclcpp::executors::SingleThreadedExecutor();
00177 executor.add_node(node->get_node_base_interface());
00178 executor.add_node(test_node->get_node_base_interface());
00179
00180 // Publish and spin
00181 odom_pub->publish(odom_msg);
00182 executor.spin_some(SPIN_TIMEOUT);
00183
00184 // Give the node time to process
00185 std::this_thread::sleep_for(LIFECYCLE_TRANSITION_DELAY);
00186 executor.spin_some(SPIN_TIMEOUT);
00187
00188 // Test passes if no exceptions are thrown
00189 EXPECT_TRUE(true);
00190 }
00191
00192 TEST_F(SafetyMonitorTest, EmergencyStopTrigger)
00193 {
00194     // TODO(locked-in): Test E-Stop triggering logic
00195     GTEST_SKIP() << "Emergency stop tests not yet implemented";
00196 }
00197
00198 TEST_F(SafetyMonitorTest, DiagnosticPublishing)
00199 {
00200     // TODO(locked-in): Test diagnostic message generation
00201     GTEST_SKIP() << "Diagnostic publishing tests not yet implemented";
00202 }
00203
00204 int main(int argc, char ** argv)
00205 {
00206     ::testing::InitGoogleTest(&argc, argv);
00207     return RUN_ALL_TESTS();
00208 }

```

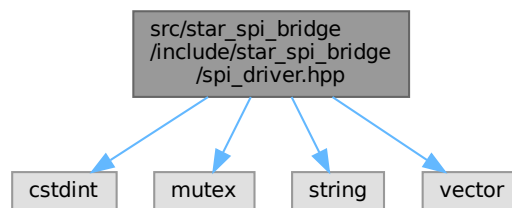
9.23 src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp File Reference

```

#include <stdint>
#include <mutex>
#include <string>
#include <vector>

```

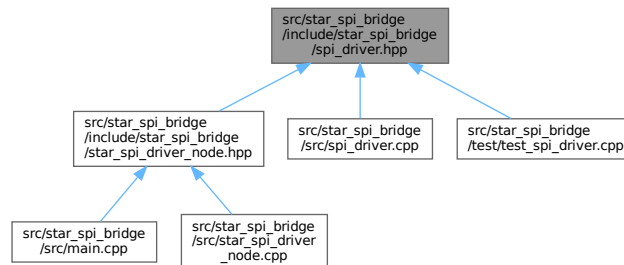
Include dependency graph for spi_driver.hpp:



9.24 spi_driver.hpp

117

This graph shows which files directly or indirectly include this file:



Classes

- class `star_spi_bridge::SpiDriver`
SPI driver for framed communication with the RX72N peripheral MCU.

Namespaces

- namespace `star_spi_bridge`

Enumerations

- enum class `star_spi_bridge::FrameType` : `uint8_t` { `star_spi_bridge::VelocityCommand` = 0x01 , `star_spi_bridge::TelemetryData` = 0x02 }
- Frame type identifiers for SPI protocol messages.*

9.24 spi_driver.hpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 Locked Inc.
00002
00003 #ifndef STAR_SPI_BRIDGE__SPI_DRIVER_HPP_
00004 #define STAR_SPI_BRIDGE__SPI_DRIVER_HPP_
00005
00006 #include <stdint>
00007 #include <mutex>
00008 #include <string>
00009 #include <vector>
00010
00011 namespace star_spi_bridge
00012 {
00013
00027 enum class FrameType : uint8_t
00028 {
00029     VelocityCommand = 0x01,
00030     TelemetryData = 0x02
00031 };
00032
00058 class SpiDriver {
00059 public:
00060     // -- Frame-structure constants -----

```

Generated by Doxygen

```

00062 static constexpr size_t k_header_size = 8;
00063
00064 static constexpr size_t k_crc_size = 4;
00065
00066 static constexpr size_t k_max_payload_size = 1024;
00067
00068 static constexpr uint16_t k_sync_word = 0x55AA;
00069
00070 explicit SpiDriver(
00071     const std::string & device_path,
00072     uint32_t speed_hz = 1000000);
00073
00074 virtual ~SpiDriver();
00075
00076 bool initialize();
00077
00078 void close_device();
00079
00080 bool transfer(
00081     const std::vector<uint8_t> & tx_data,
00082     std::vector<uint8_t> & rx_data);
00083
00084 static void encode_frame(
00085     uint16_t seq, FrameType type, uint8_t flags,
00086     const std::vector<uint8_t> & payload,
00087     std::vector<uint8_t> & out_frame);
00088
00089 static bool decode_frame(
00090     const std::vector<uint8_t> & frame, uint16_t & seq,
00091     FrameType & type, uint8_t & flags,
00092     std::vector<uint8_t> & payload);
00093
00094 static uint32_t calculate_crc32(const std::vector<uint8_t> & data);
00095
00096 private:
00097     std::string device_path_;
00098     uint32_t speed_hz_;
00099     int spi_fd_;
00100
00101     static const uint32_t k_crc32_polynomial = 0x04C11DB7;
00102     static constexpr uint8_t k_bits_per_word = 8;
00103     // [SYNC: 0x55AA (2B, LE) - wire: 0xAA, 0x55]
00104     // [SEQ: sequence number (2B, LE)]
00105     // [LEN: payload length (2B, LE)]
00106     // [TYPE: frame type (1B)]
00107     // [FLAGS: control flags (1B)]
00108     // Total header = 2+2+2+1+1 = 8 bytes.
00109
00110     static uint32_t crc32_table_[256];
00111     static std::once_flag crc32_table_init_flag_;
00112     static void init_crc32_table();
00113 };
00114
00115 } // namespace star_spi_bridge
00116
00117 #endif // STAR_SPI_BRIDGE__SPI_DRIVER_HPP_

```

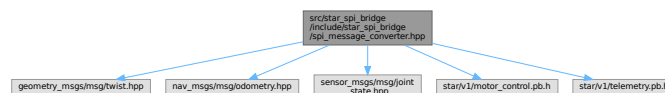
9.25 src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.hpp File Reference

```

#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <sensor_msgs/msg/joint_state.hpp>
#include "star/v1/motor_control.pb.h"
#include "star/v1/telemetry.pb.h"

```

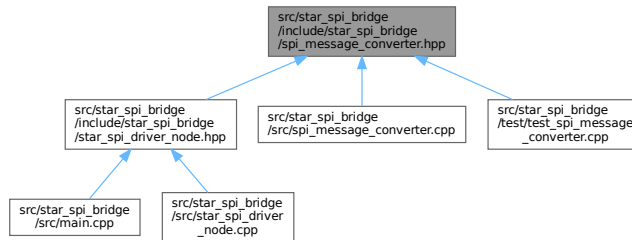
Include dependency graph for spi_message_converter.hpp:



9.26 spi_message_converter.hpp

119

This graph shows which files directly or indirectly include this file:



Classes

- class [star_spi_bridge::SpiMessageConverter](#)
- struct [star_spi_bridge::SpiMessageConverter::Parameters](#)

Namespaces

- namespace [star_spi_bridge](#)

9.26 spi_message_converter.hpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 Locked Inc.
00002
00003 #ifndef STAR_SPI_BRIDGE__SPI_MESSAGE_CONVERTER_HPP_
00004 #define STAR_SPI_BRIDGE__SPI_MESSAGE_CONVERTER_HPP_
00005
00006 #include <geometry_msgs/msg/twist.hpp>
00007 #include <nav_msgs/msg/odometry.hpp>
00008 #include <sensor_msgs/msg/joint_state.hpp>
00009
00010 #include "star/v1/motor_control.pb.h"
00011 #include "star/v1/telemetry.pb.h"
00012
00013 namespace star_spi_bridge
00014 {
00015
00016 class SpiMessageConverter {
00017 public:
00018     struct Parameters
00019     {
00020         double wheel_base;
00021         double wheel_radius;
00022         int32_t ticks_per_rev;
00023     };
00024
00025     explicit SpiMessageConverter(const Parameters & params);
00026
00027     // ROS2 -> Protobuf
00028     bool twist_to_velocity_command(
00029         const geometry_msgs::msg::Twist & twist,
00030         star::v1::VelocityCommand & command);
00031
00032     // Protobuf -> ROS2
00033     void telemetry_to_odometry(
00034         const star::v1::TelemetryData & telemetry,
00035         nav_msgs::msg::Odometry & odom);
00036     void telemetry_to_joint_state(

```

Generated by Doxygen

```

00037     const star::vl::TelemetryData & telemetry,
00038     sensor_msgs::msg::JointState & joint_state);
00039
00040 private:
00041     Parameters params_;
00042
00043     // Odometry state
00044     double x_ = 0.0;
00045     double y_ = 0.0;
00046     double theta_ = 0.0;
00047
00048     // Encoder state for delta calculation
00049     int64_t prev_ticks_fl_ = 0;
00050     int64_t prev_ticks_fr_ = 0;
00051     int64_t prev_ticks_bl_ = 0;
00052     int64_t prev_ticks_br_ = 0;
00053     bool first_odom_msg_ = true;
00054
00055     static double normalize_angle(double angle);
00056 };
00057
00058 } // namespace star_spi_bridge
00059
00060 #endif // STAR_SPI_BRIDGE__SPI_MESSAGE_CONVERTER_HPP_

```

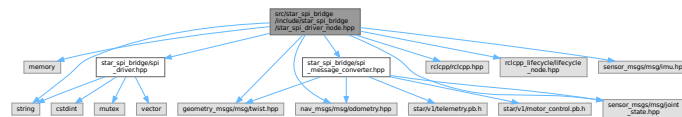
9.27 src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_node.hpp File Reference

```

#include <memory>
#include <string>
#include <geometry_msgs/msg/twist.hpp>
#include <nav_msgs/msg/odometry.hpp>
#include <rclcpp/rclcpp.hpp>
#include <rclcpp_lifecycle/lifecycle_node.hpp>
#include <sensor_msgs/msg/imu.hpp>
#include <sensor_msgs/msg/joint_state.hpp>
#include "star_spi_bridge/spi_driver.hpp"
#include "star_spi_bridge/spi_message_converter.hpp"

```

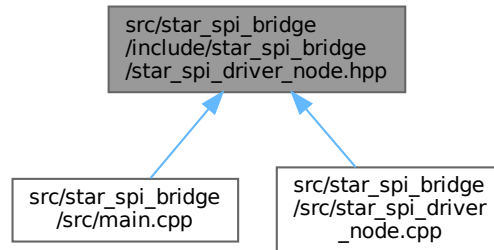
Include dependency graph for star_spi_driver_node.hpp:



9.28 star_spi_driver_node.hpp

121

This graph shows which files directly or indirectly include this file:



Classes

- class `star_spi_bridge::StarSpiDriverNode`

Namespaces

- namespace `star_spi_bridge`

9.28 star_spi_driver_node.hpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 Locked Inc.
00002
00003 #ifndef STAR_SPI_BRIDGE__STAR_SPI_DRIVER_NODE_HPP_
00004 #define STAR_SPI_BRIDGE__STAR_SPI_DRIVER_NODE_HPP_
00005
00006 #include <memory>
00007 #include <string>
00008
00009 #include <geometry_msgs/msg/twist.hpp>
00010 #include <nav_msgs/msg/odometry.hpp>
00011 #include <rclcpp/rclcpp.hpp>
00012 #include <rclcpp_lifecycle/lifecycle_node.hpp>
00013 #include <sensor_msgs/msg/imu.hpp>
00014 #include <sensor_msgs/msg/joint_state.hpp>
00015
00016 #include "star_spi_bridge/spi_driver.hpp"
00017 #include "star_spi_bridge/spi_message_converter.hpp"
00018
00019 namespace star_spi_bridge
00020 {
00021
00022 class StarSpiDriverNode : public rclcpp_lifecycle::LifecycleNode {
00023 public:
00024   explicit StarSpiDriverNode(
00025     const rclcpp::NodeOptions & options = rclcpp::NodeOptions());
00026   ~StarSpiDriverNode() override;
00027
00028   // Lifecycle transitions
00029   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00030   on_configure(const rclcpp_lifecycle::State & prev_state) override;
00031   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00032   on_activate(const rclcpp_lifecycle::State & prev_state) override;

```

Generated by Doxygen

```

00033   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00034   on_deactivate(const rclcpp_lifecycle::State & prev_state) override;
00035   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00036   on_cleanup(const rclcpp_lifecycle::State & prev_state) override;
00037   rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00038   on_shutdown(const rclcpp_lifecycle::State & prev_state) override;
00039
00040 private:
00041   // Callbacks
00042   void cmd_vel_callback(const geometry_msgs::msg::Twist::SharedPtr msg);
00043   void timer_callback(); // 100 Hz loop
00044
00045   // Components
00046   std::unique_ptr<SpiDriver> spi_driver_;
00047   std::unique_ptr<SpiMessageConverter> converter_;
00048
00049   // ROS handles
00050   rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_vel_sub_;
00051   rclcpp_lifecycle::LifecyclePublisher<nav_msgs::msg::Odometry>::SharedPtr
00052     odom_pub_;
00053   rclcpp_lifecycle::LifecyclePublisher<sensor_msgs::msg::JointState>::SharedPtr
00054     joint_state_pub_;
00055   rclcpp_lifecycle::LifecyclePublisher<sensor_msgs::msg::Imu>::SharedPtr
00056     imu_pub_;
00057   rclcpp::TimerBase::SharedPtr timer_;
00058
00059   // State
00060   geometry_msgs::msg::Twist current_cmd_vel_;
00061   rclcpp::Time last_cmd_vel_time_;
00062   uint16_t tx_seq_ = 0;
00063
00064   // Parameters
00065   std::string spi_device_path_;
00066   int spi_speed_hz_;
00067   int cmd_vel_timeout_ms_;
00068 };
00069
00070 } // namespace star_spi_bridge
00071
00072 #endif // STAR_SPI_BRIDGE__STAR_SPI_DRIVER_NODE_HPP_

```

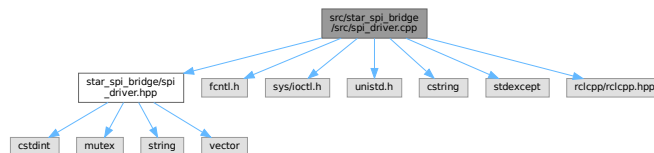
9.29 src/star_spi_bridge/src/spi_driver.cpp File Reference

```

#include "star_spi_bridge/spi_driver.hpp"
#include <fcntl.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <cstring>
#include <stdexcept>
#include <rclcpp/rclcpp.hpp>

```

Include dependency graph for spi_driver.cpp:



Classes

- struct [spi_ioc_transfer](#)

9.29 src/star_spi_bridge/src/spi_driver.cpp File Reference**123****Namespaces**

- namespace [star_spi_bridge](#)

Macros

- #define [SPI_IOC_MESSAGE](#)(N)

Variables

- constexpr uint8_t [SPI_MODE_0](#) = 0
- constexpr int [SPI_IOC_WR_MODE](#) = 0
- constexpr int [SPI_IOC_WR_BITS_PER_WORD](#) = 0
- constexpr int [SPI_IOC_WR_MAX_SPEED_HZ](#) = 0

9.29.1 Macro Definition Documentation**9.29.1.1 SPI_IOC_MESSAGE**

```
#define SPI_IOC_MESSAGE(  
    N)
```

Value:

0

Definition at line 22 of file [spi_driver.cpp](#).Referenced by [star_spi_bridge::SpiDriver::transfer\(\)](#).**9.29.2 Variable Documentation****9.29.2.1 SPI_IOC_WR_BITS_PER_WORD**

```
int SPI_IOC_WR_BITS_PER_WORD = 0 [constexpr]
```

Definition at line 25 of file [spi_driver.cpp](#).Referenced by [star_spi_bridge::SpiDriver::initialize\(\)](#).**9.29.2.2 SPI_IOC_WR_MAX_SPEED_HZ**

```
int SPI_IOC_WR_MAX_SPEED_HZ = 0 [constexpr]
```

Definition at line 26 of file [spi_driver.cpp](#).Referenced by [star_spi_bridge::SpiDriver::initialize\(\)](#).

Generated by Doxygen

9.29.2.3 SPI_IOC_WR_MODE

```
int SPI_IOC_WR_MODE = 0 [constexpr]
```

Definition at line 24 of file [spi_driver.cpp](#).

Referenced by [star_spi_bridge::SpiDriver::initialize\(\)](#).

9.29.2.4 SPI_MODE_0

```
uint8_t SPI_MODE_0 = 0 [constexpr]
```

Definition at line 23 of file [spi_driver.cpp](#).

Referenced by [star_spi_bridge::SpiDriver::initialize\(\)](#).

9.30 spi_driver.cpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/spi_driver.hpp"
00004
00005 #include <fcntl.h>
00006 #include <sys/ioctl.h>
00007 #include <unistd.h>
00008
00009 #include <cstring>
00010 #include <stdexcept>
00011
00012 #include <rcldcpp/rcldcpp.hpp>
00013
00014 #ifdef __linux__
00015 #include <linux/spi/spidev.h>
00016 #else
00017 // Mock spidev headers if on macOS/non-Linux to allow compilation (for
00018 // development/analysis only)
00019 // Ideally this code runs on Linux/RPi5. On macOS, these headers likely
00020 // don't exist or are different.
00021 #ifndef SPI_IOC_MESSAGE
00022 #define SPI_IOC_MESSAGE(N) 0
00023 constexpr uint8_t SPI_MODE_0 = 0;
00024 constexpr int SPI_IOC_WR_MODE = 0;
00025 constexpr int SPI_IOC_WR_BITS_PER_WORD = 0;
00026 constexpr int SPI_IOC_WR_MAX_SPEED_HZ = 0;
00027
00028 struct spi_ioc_transfer
00029 {
00030     uint64_t tx_buf_;
00031     uint64_t rx_buf_;
00032     uint32_t len_;
00033     uint32_t speed_hz_;
00034     uint16_t delay_usecs_;
00035     uint8_t bits_per_word_;
00036     uint8_t cs_change_;
00037     uint32_t pad_;
00038 };
00039 #endif
00040 #endif
00041
00042 namespace star_spi_bridge
00043 {
00044     namespace
00045     {
00046         auto logger()
00047         {
00048             return rcldcpp::get_logger("star_spi_bridge.spi_driver");
00049         }
00050     }
00051 }
00052

```

9.30 spi_driver.cpp

125

```

00053 inline void set_spi_xfer_tx_buf(spi_ioc_transfer & xfer, uint64_t value)
00054 {
00055     #ifdef __linux__
00056         xfer.tx_buf = value;
00057     #else
00058         xfer.tx_buf_ = value;
00059     #endif
00060 }
00061
00062 inline void set_spi_xfer_rx_buf(spi_ioc_transfer & xfer, uint64_t value)
00063 {
00064     #ifdef __linux__
00065         xfer.rx_buf = value;
00066     #else
00067         xfer.rx_buf_ = value;
00068     #endif
00069 }
00070
00071 inline void set_spi_xfer_len(spi_ioc_transfer & xfer, uint32_t value)
00072 {
00073     #ifdef __linux__
00074         xfer.len = value;
00075     #else
00076         xfer.len_ = value;
00077     #endif
00078 }
00079
00080 inline void set_spi_xfer_speed_hz(spi_ioc_transfer & xfer, uint32_t value)
00081 {
00082     #ifdef __linux__
00083         xfer.speed_hz = value;
00084     #else
00085         xfer.speed_hz_ = value;
00086     #endif
00087 }
00088
00089 inline void set_spi_xfer_bits_per_word(spi_ioc_transfer & xfer, uint8_t value)
00090 {
00091     #ifdef __linux__
00092         xfer.bits_per_word = value;
00093     #else
00094         xfer.bits_per_word_ = value;
00095     #endif
00096 }
00097 } // namespace
00098
00099 uint32_t SpiDriver::crc32_table[256];
00100 std::once_flag SpiDriver::crc32_table_init_flag_;
00101
00102 SpiDriver::SpiDriver(const std::string & device_path, uint32_t speed_hz)
00103 : device_path_(device_path), speed_hz_(speed_hz), spi_fd_(-1)
00104 {
00105     std::call_once(crc32_table_init_flag_, init_crc32_table);
00106 }
00107
00108 SpiDriver::~SpiDriver()
00109 {
00110     close_device();
00111 }
00112
00113 bool SpiDriver::initialize()
00114 {
00115     #ifndef __linux__
00116         RCLCPP_ERROR(logger(), "SPI driver is only supported on Linux platforms");
00117         return false;
00118     #endif
00119     // Open SPI device
00120     spi_fd_ = open(device_path_.c_str(), O_RDWR);
00121     if (spi_fd_ < 0) {
00122         RCLCPP_ERROR(logger(), "Failed to open SPI device: %s (%s)", device_path_.c_str(),
00123             strerror(errno));
00124         return false;
00125     }
00126     // Configure SPI mode (Mode 0: CPOL=0, CPHA=0)
00127     uint8_t mode = SPI_MODE_0;
00128     if (ioctl(spi_fd_, SPI_IOC_WR_MODE, &mode) < 0) {
00129         RCLCPP_ERROR(logger(), "Failed to set SPI mode");
00130         close_device();
00131         return false;
00132     }
00133     // Configure bits per word (8 bits)
00134     uint8_t bits = k_bits_per_word;
00135     if (ioctl(spi_fd_, SPI_IOC_WR_BITS_PER_WORD, &bits) < 0) {
00136         RCLCPP_ERROR(logger(), "Failed to set SPI bits per word");
00137         close_device();
00138     }
00139 }

```

Generated by Doxygen

```

00140     return false;
00141 }
00142
00143 // Configure max speed
00144 if (ioctl(spi_fd_, SPI_IOC_WR_MAX_SPEED_HZ, &speed_hz_) < 0) {
00145     RCLCPP_ERROR(logger(), "Failed to set SPI max speed");
00146     close_device();
00147     return false;
00148 }
00149
00150 return true;
00151 }
00152
00153 void SpiDriver::close_device()
00154 {
00155     if (spi_fd_ >= 0) {
00156         close(spi_fd_);
00157         spi_fd_ = -1;
00158     }
00159 }
00160
00161 bool SpiDriver::transfer(const std::vector<uint8_t> & tx_data, std::vector<uint8_t> & rx_data)
00162 {
00163     #ifndef __linux__
00164         RCLCPP_ERROR(logger(), "SPI driver is only supported on Linux platforms");
00165         return false;
00166     #endif
00167     if (spi_fd_ < 0) {
00168         return false;
00169     }
00170
00171     if (rx_data.size() != tx_data.size()) {
00172         rx_data.resize(tx_data.size());
00173     }
00174
00175     struct spi_ioc_transfer xfer;
00176     std::memset(&xfer, 0, sizeof(xfer));
00177
00178     set_spi_xfer_tx_buf(xfer, static_cast<uint64_t>(reinterpret_cast<uintptr_t>(tx_data.data())));
00179     set_spi_xfer_rx_buf(xfer, static_cast<uint64_t>(reinterpret_cast<uintptr_t>(rx_data.data())));
00180     set_spi_xfer_len(xfer, static_cast<uint32_t>(tx_data.size()));
00181     set_spi_xfer_speed_hz(xfer, speed_hz_);
00182     set_spi_xfer_bits_per_word(xfer, k_bits_per_word);
00183
00184     int ret = ioctl(spi_fd_, SPI_IOC_MESSAGE(1), &xfer);
00185     if (ret < 0) {
00186         RCLCPP_ERROR(logger(), "SPI transfer failed: %s", strerror(errno));
00187         return false;
00188     }
00189
00190     return true;
00191 }
00192
00193 void SpiDriver::encode_frame(
00194     uint16_t seq,
00195     FrameType type,
00196     uint8_t flags,
00197     const std::vector<uint8_t> & payload,
00198     std::vector<uint8_t> & out_frame)
00199 {
00200     if (payload.size() > k_max_payload_size) {
00201         throw std::invalid_argument("payload exceeds maximum size of 1024 bytes");
00202     }
00203
00204     // [SYNC(2)][SEQ(2)][LEN(2)][TYPE(1)][FLAGS(1)][PAYLOAD(N)][CRC(4)]
00205     size_t frame_size = k_header_size + payload.size() + k_crc_size;
00206     out_frame.clear();
00207     out_frame.reserve(frame_size);
00208
00209     // SYNC: 0x55AA (Little Endian)
00210     // Note: Header fields (SYNC, SEQ, LEN) use little-endian byte order
00211     // Wire format: [0xAA, 0x55] for 0x55AA
00212     out_frame.push_back(k_sync_word & 0xFF); // LSB first (0xAA)
00213     out_frame.push_back((k_sync_word >> 8) & 0xFF); // MSB second (0x55)
00214
00215     // SEQ (Little Endian)
00216     out_frame.push_back(seq & 0xFF); // LSB first
00217     out_frame.push_back((seq >> 8) & 0xFF); // MSB second
00218
00219     // LEN (Little Endian)
00220     uint16_t len = static_cast<uint16_t>(payload.size());
00221     out_frame.push_back(len & 0xFF); // LSB first
00222     out_frame.push_back((len >> 8) & 0xFF); // MSB second
00223
00224     // TYPE
00225     out_frame.push_back(static_cast<uint8_t>(type));
00226

```

9.30 spi_driver.cpp

127

```

00227 // FLAGS
00228 out_frame.push_back(flags);
00229
00230 // PAYLOAD
00231 out_frame.insert(out_frame.end(), payload.begin(), payload.end());
00232
00233 // CRC-32 (IEEE 802.3) - Calculated over Header + Payload
00234 uint32_t crc = calculate_crc32(out_frame);
00235
00236 // Append CRC (Little Endian as per spec "CRC-32: IEEE 802.3 (4B, LE)")
00237 out_frame.push_back(crc & 0xFF);
00238 out_frame.push_back((crc >> 8) & 0xFF);
00239 out_frame.push_back((crc >> 16) & 0xFF);
00240 out_frame.push_back((crc >> 24) & 0xFF);
00241 }
00242
00243 bool SpiDriver::decode_frame(
00244     const std::vector<uint8_t> & frame,
00245     uint16_t & seq,
00246     FrameType & type,
00247     uint8_t & flags,
00248     std::vector<uint8_t> & payload)
00249 {
00250     if (frame.size() < k_header_size + k_crc_size) {
00251         return false;
00252     }
00253
00254     // Check SYNC (Little Endian: [0xAA, 0x55] for 0x55AA)
00255     if (frame[0] != (k_sync_word & 0xFF) || frame[1] != ((k_sync_word >> 8) & 0xFF)) {
00256         return false;
00257     }
00258
00259     // Extract Length (Little Endian: LSB first)
00260     uint16_t len = frame[4] | (static_cast<uint16_t>(frame[5]) << 8);
00261
00262     // Validate Frame Size
00263     // Header(8) + Payload(len) + CRC(4)
00264     if (frame.size() != k_header_size + len + k_crc_size) {
00265         return false;
00266     }
00267
00268     // Verify CRC
00269     // Calculate CRC over Header + Payload (bytes 0 to end-4)
00270     std::vector<uint8_t> data_to_check(frame.begin(), frame.end() - k_crc_size);
00271     uint32_t calculated_crc = calculate_crc32(data_to_check);
00272
00273     uint32_t received_crc = static_cast<uint32_t>(frame[frame.size() - 4]) |
00274         (static_cast<uint32_t>(frame[frame.size() - 3]) << 8) |
00275         (static_cast<uint32_t>(frame[frame.size() - 2]) << 16) |
00276         (static_cast<uint32_t>(frame[frame.size() - 1]) << 24);
00277
00278     if (calculated_crc != received_crc) {
00279         return false;
00280     }
00281
00282     // Extract fields (Little Endian: LSB first)
00283     seq = frame[2] | (static_cast<uint16_t>(frame[3]) << 8);
00284     type = static_cast<FrameType>(frame[6]);
00285     flags = frame[7];
00286
00287     payload.assign(frame.begin() + k_header_size, frame.end() - k_crc_size);
00288
00289     return true;
00290 }
00291
00292 void SpiDriver::init_crc32_table()
00293 {
00294     // Generate CRC-32 lookup table using IEEE 802.3 polynomial (reflected form)
00295     // Polynomial 0xEDB88320 is the bit-reversed form of 0x04C11DB7
00296     // This produces standard CRC-32 values (e.g., "123456789" -> 0xBF43926)
00297     constexpr uint32_t polynomial = 0xEDB88320;
00298     for (uint32_t i = 0; i < 256; i++) {
00299         uint32_t crc = i;
00300         for (uint32_t j = 0; j < 8; j++) {
00301             if (crc & 1) {
00302                 crc = (crc >> 1) ^ polynomial;
00303             } else {
00304                 crc >>= 1;
00305             }
00306         }
00307         crc32_table_[i] = crc;
00308     }
00309 }
00310
00311 uint32_t SpiDriver::calculate_crc32(const std::vector<uint8_t> & data)
00312 {
00313     std::call_once(crc32_table_init_flag_, init_crc32_table);

```

Generated by Doxygen

```

00314 uint32_t crc = 0xFFFFFFFF;
00315 for (uint8_t byte : data) {
00316     uint8_t lookup_index = (crc ^ byte) & 0xFF;
00317     crc = (crc » 8) ^ crc32_table[lookup_index];
00318 }
00319 return crc ^ 0xFFFFFFFF;
00320 }
00321
00322 } // namespace star_spi_bridge

```

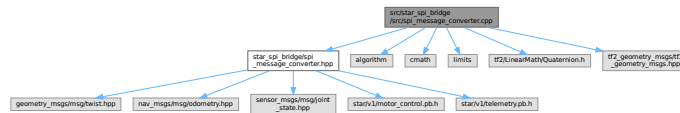
9.31 src/star_spi_bridge/src/spi_message_converter.cpp File Reference

```

#include "star_spi_bridge/spi_message_converter.hpp"
#include <algorithm>
#include <cmath>
#include <limits>
#include <tf2/LinearMath/Quaternion.h>
#include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>

```

Include dependency graph for spi_message_converter.cpp:



Namespaces

- namespace [star_spi_bridge](#)

9.32 spi_message_converter.cpp

[Go to the documentation of this file.](#)

```

00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/spi_message_converter.hpp"
00004
00005 #include <algorithm>
00006 #include <cmath>
00007 #include <limits>
00008 #include <tf2/LinearMath/Quaternion.h> // NOLINT(build/include_order)
00009 #include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>
00010
00011 namespace star_spi_bridge
00012 {
00013
00014 SpiMessageConverter::SpiMessageConverter(const Parameters & params)
00015 : params_(params) {}
00016
00017 bool SpiMessageConverter::twist_to_velocity_command(
00018     const geometry_msgs::msg::Twist & twist,
00019     star::v1::VelocityCommand & command)
00020 {
00021     if (!std::isfinite(twist.linear.x) || !std::isfinite(twist.angular.z)) {
00022         return false;
00023     }
00024
00025     double linear_x = twist.linear.x;
00026     double angular_z = twist.angular.z;
00027
00028     // Differential drive kinematics

```

9.32 spi_message_converter.cpp

129

```

00029 // v_r = v + (w * L / 2)
00030 // v_l = v - (w * L / 2)
00031 double right_vel = linear_x + (angular_z * params_.wheel_base / 2.0);
00032 double left_vel = linear_x - (angular_z * params_.wheel_base / 2.0);
00033
00034 // Clamp to hardware limits (e.g. +/- 2.0 m/s as per spec)
00035 // Although the firmware should also handle this, good to be safe.
00036 // The spec says: "Velocity Limits: +/-2.0 m/s per wheel"
00037 const double k_max_vel = 2.0;
00038 right_vel = std::clamp(right_vel, -k_max_vel, k_max_vel);
00039 left_vel = std::clamp(left_vel, -k_max_vel, k_max_vel);
00040
00041 command.set_front_left_velocity_mps(static_cast<float>(left_vel));
00042 command.set_back_left_velocity_mps(static_cast<float>(left_vel));
00043 command.set_front_right_velocity_mps(static_cast<float>(right_vel));
00044 command.set_back_right_velocity_mps(static_cast<float>(right_vel));
00045
00046 return true;
00047 }
00048
00049 void SpiMessageConverter::telemetry_to_odometry(
00050     const star::vl::TelemetryData & telemetry,
00051     nav_msgs::msg::Odometry & odom)
00052 {
00053     // Extract encoder data from telemetry
00054     const auto & enc_fl = telemetry.encoder_front_left();
00055     const auto & enc_fr = telemetry.encoder_front_right();
00056     const auto & enc_bl = telemetry.encoder_back_left();
00057     const auto & enc_br = telemetry.encoder_back_right();
00058
00059     // Encoder ticks are cumulative; we compute deltas between messages
00060     int64_t ticks_fl = enc_fl.ticks();
00061     int64_t ticks_fr = enc_fr.ticks();
00062     int64_t ticks_bl = enc_bl.ticks();
00063     int64_t ticks_br = enc_br.ticks();
00064
00065     if (first_odom_msg_) {
00066         prev_ticks_fl_ = ticks_fl;
00067         prev_ticks_fr_ = ticks_fr;
00068         prev_ticks_bl_ = ticks_bl;
00069         prev_ticks_br_ = ticks_br;
00070         first_odom_msg_ = false;
00071         return; // No movement to report on first message
00072     }
00073
00074     // Calculate deltas
00075     int64_t delta_fl = ticks_fl - prev_ticks_fl_;
00076     int64_t delta_fr = ticks_fr - prev_ticks_fr_;
00077     int64_t delta_bl = ticks_bl - prev_ticks_bl_;
00078     int64_t delta_br = ticks_br - prev_ticks_br_;
00079
00080     // Update previous
00081     prev_ticks_fl_ = ticks_fl;
00082     prev_ticks_fr_ = ticks_fr;
00083     prev_ticks_bl_ = ticks_bl;
00084     prev_ticks_br_ = ticks_br;
00085
00086     // Convert to distance
00087     double m_per_tick = (2.0 * M_PI * params_.wheel_radius) / params_.ticks_per_rev;
00088
00089     double dist_fl = delta_fl * m_per_tick;
00090     double dist_fr = delta_fr * m_per_tick;
00091     double dist_bl = delta_bl * m_per_tick;
00092     double dist_br = delta_br * m_per_tick;
00093
00094     double dist_left = (dist_fl + dist_bl) / 2.0;
00095     double dist_right = (dist_fr + dist_br) / 2.0;
00096
00097     double dist_center = (dist_right + dist_left) / 2.0;
00098     double delta_theta = (dist_right - dist_left) / params_.wheel_base;
00099
00100     // Update pose
00101     double avg_theta = theta_ + delta_theta / 2.0;
00102     x_ += dist_center * std::cos(avg_theta);
00103     y_ += dist_center * std::sin(avg_theta);
00104     theta_ = normalize_angle(theta_ + delta_theta);
00105
00106     // Populate Odometry message
00107     // Note: Header timestamp should be set by the caller (node)
00108     odom.header.frame_id = "odom";
00109     odom.child_frame_id = "base_link";
00110
00111     odom.pose.pose.position.x = x_;
00112     odom.pose.pose.position.y = y_;
00113     odom.pose.pose.position.z = 0.0;
00114
00115     tf2::Quaternion q;

```

Generated by Doxygen

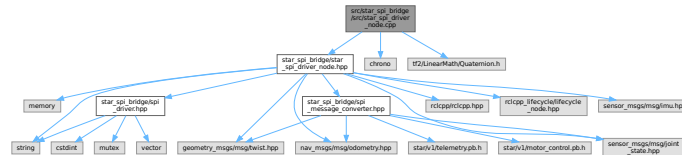
```

00116 q.setRPY(0, 0, theta_);
00117 odom.pose.pose.orientation.x = q.x();
00118 odom.pose.pose.orientation.y = q.y();
00119 odom.pose.pose.orientation.z = q.z();
00120 odom.pose.pose.orientation.w = q.w();
00121
00122 // Populate twist from encoder-reported velocities when available
00123 double v_fl = enc_fl.velocity_mps();
00124 double v_fr = enc_fr.velocity_mps();
00125 double v_bl = enc_bl.velocity_mps();
00126 double v_br = enc_br.velocity_mps();
00127
00128 bool has_velocity_data = (v_fl != 0.0 || v_fr != 0.0 || v_bl != 0.0 || v_br != 0.0);
00129 if (has_velocity_data) {
00130     double v_left = (v_fl + v_bl) / 2.0;
00131     double v_right = (v_fr + v_br) / 2.0;
00132
00133     double v_linear = (v_right + v_left) / 2.0;
00134     double v_angular = (v_right - v_left) / params_.wheel_base;
00135
00136     odom.twist.twist.linear.x = v_linear;
00137     odom.twist.twist.angular.z = v_angular;
00138 }
00139
00140 // Pose covariance -- row-major 6x6 (x, y, z, roll, pitch, yaw)
00141 // Zero covariance makes robot_localization unstable; use realistic dead-reckoning values.
00142 odom.pose.covariance[0] = 0.1; // x
00143 odom.pose.covariance[7] = 0.1; // y
00144 odom.pose.covariance[14] = 1e6; // z (not observed)
00145 odom.pose.covariance[21] = 1e6; // roll (not observed)
00146 odom.pose.covariance[28] = 1e6; // pitch (not observed)
00147 odom.pose.covariance[35] = 0.1; // yaw
00148
00149 // Twist covariance -- same layout
00150 odom.twist.covariance[0] = 0.05; // vx
00151 odom.twist.covariance[7] = 1e6; // vy (not observed -- diff drive constraint)
00152 odom.twist.covariance[14] = 1e6; // vz
00153 odom.twist.covariance[21] = 1e6; // roll rate
00154 odom.twist.covariance[28] = 1e6; // pitch rate
00155 odom.twist.covariance[35] = 0.05; // angular z (yaw rate)
00156 }
00157
00158 void SpiMessageConverter::telemetry_to_joint_state(
00159     const star::vl::TelemetryData & telemetry,
00160     sensor_msgs::msg::JointState & joint_state)
00161 {
00162     // Extract encoder data from telemetry
00163     const auto & enc_fl = telemetry.encoder_front_left();
00164     const auto & enc_fr = telemetry.encoder_front_right();
00165     const auto & enc_bl = telemetry.encoder_back_left();
00166     const auto & enc_br = telemetry.encoder_back_right();
00167
00168     joint_state.name = {
00169         "front_left_wheel", "front_right_wheel", "back_left_wheel", "back_right_wheel"};
00170
00171     // Position (radians)
00172     double rad_per_tick = (2.0 * M_PI) / params_.ticks_per_rev;
00173     joint_state.position = {static_cast<double>(enc_fl.ticks()) * rad_per_tick,
00174         static_cast<double>(enc_fr.ticks()) * rad_per_tick,
00175         static_cast<double>(enc_bl.ticks()) * rad_per_tick,
00176         static_cast<double>(enc_br.ticks()) * rad_per_tick};
00177
00178     // Velocity (rad/s)
00179     // velocity_mps = radius * angular_velocity_rad_s
00180     // angular_velocity = velocity_mps / radius
00181     double inv_radius = 1.0 / params_.wheel_radius;
00182     joint_state.velocity = {enc_fl.velocity_mps() * inv_radius,
00183         enc_fr.velocity_mps() * inv_radius,
00184         enc_bl.velocity_mps() * inv_radius,
00185         enc_br.velocity_mps() * inv_radius};
00186 }
00187
00188 double SpiMessageConverter::normalize_angle(double angle)
00189 {
00190     while (angle > M_PI) {
00191         angle -= 2.0 * M_PI;
00192     }
00193     while (angle < -M_PI) {
00194         angle += 2.0 * M_PI;
00195     }
00196     return angle;
00197 }
00198
00199 } // namespace star_spi_bridge

```


9.33 src/star_spi_bridge/src/star_spi_driver_node.cpp File Reference

```
#include "star_spi_bridge/star_spi_driver_node.hpp"
#include <chrono>
#include <tf2/LinearMath/Quaternion.h>
Include dependency graph for star_spi_driver_node.cpp:
```



Namespaces

- namespace [star_spi_bridge](#)

Variables

- static constexpr int [star_spi_bridge::kLogThrottleMs](#) = 1000
- static constexpr double [star_spi_bridge::kImuOrientationVar](#) = 0.01
- static constexpr double [star_spi_bridge::kImuAngularVelocityVar](#) = 0.001
- static constexpr double [star_spi_bridge::kImuLinearAccelVar](#) = 0.01

9.34 star_spi_driver_node.cpp

[Go to the documentation of this file.](#)

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/star_spi_driver_node.hpp"
00004
00005 #include <chrono>
00006
00007 #include <tf2/LinearMath/Quaternion.h> // NOLINT(build/include_order)
00008
00009 using namespace std::chrono_literals;
00010
00011 namespace star_spi_bridge
00012 {
00013   static constexpr int kLogThrottleMs = 1000;
00014
00015   // IMU sensor noise model (variance = sigma^2, all diagonal).
00016   // sigma_orientation ~5.7 deg, sigma_angular_vel ~0.032 rad/s, sigma_accel ~0.1 m/s^2
00017   static constexpr double kImuOrientationVar = 0.01; // rad^2
00018   static constexpr double kImuAngularVelocityVar = 0.001; // (rad/s)^2
00019   static constexpr double kImuLinearAccelVar = 0.01; // (m/s^2)^2
00020
00021   StarSpiDriverNode::StarSpiDriverNode(const rclcpp::NodeOptions & options)
00022   : rclcpp_lifecycle::LifecycleNode("star_spi_driver", options)
00023   {
00024     // Declare parameters
00025     declare_parameter("spi_device_path", "/dev/spidev0.0");
00026     declare_parameter("spi_speed_hz", 10000000); // 10 MHz
00027     declare_parameter("cmd_vel_timeout_ms", 500);
00028     declare_parameter("wheel_base", 0.150);
00029     declare_parameter("wheel_radius", 0.0325);
00030     declare_parameter("ticks_per_rev", 11599);
00031   }
```

Generated by Doxygen

```

00032
00033 StarSpiDriverNode::~StarSpiDriverNode()
00034 {
00035     // Ensure cleanup
00036     if (spi_driver_) {
00037         spi_driver_>close_device();
00038     }
00039 }
00040
00041 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00042 StarSpiDriverNode::on_configure(const rclcpp_lifecycle::State &)
00043 {
00044     RCLCPP_INFO(get_logger(), "Configuring StarSpiDriverNode...");
00045
00046     // Load parameters
00047     spi_device_path_ = get_parameter("spi_device_path").as_string();
00048     spi_speed_hz_ = get_parameter("spi_speed_hz").as_int();
00049     cmd_vel_timeout_ms_ = get_parameter("cmd_vel_timeout_ms").as_int();
00050
00051     SpiMessageConverter::Parameters converter_params;
00052     converter_params.wheel_base = get_parameter("wheel_base").as_double();
00053     converter_params.wheel_radius = get_parameter("wheel_radius").as_double();
00054     converter_params.ticks_per_rev = get_parameter("ticks_per_rev").as_int();
00055
00056     // Initialize components
00057     spi_driver_ = std::make_unique<SpiDriver>(spi_device_path_, spi_speed_hz_);
00058     converter_ = std::make_unique<SpiMessageConverter>(converter_params);
00059
00060     if (!spi_driver_>initialize()) {
00061         RCLCPP_ERROR(get_logger(), "Failed to initialize SPI driver on %s",
00062             spi_device_path_.c_str());
00063         return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00064             CallbackReturn::FAILURE;
00065     }
00066
00067     // Create publishers
00068     odom_pub_ = create_publisher<nav_msgs::msg::Odometry>("odom/unfiltered", 10);
00069     joint_state_pub_ =
00070         create_publisher<sensor_msgs::msg::JointState>("joint_states", 10);
00071     imu_pub_ = create_publisher<sensor_msgs::msg::Imu>("imu/data", 10);
00072     // Create subscription
00073     cmd_vel_sub_ = create_subscription<geometry_msgs::msg::Twist>(
00074         "cmd_vel", 10,
00075         std::bind(&StarSpiDriverNode::cmd_vel_callback, this,
00076             std::placeholders::_1));
00077
00078     // Initialize state
00079     last_cmd_vel_time_ = get_clock()->now();
00080
00081     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00082         CallbackReturn::SUCCESS;
00083 }
00084
00085 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00086 StarSpiDriverNode::on_activate(const rclcpp_lifecycle::State &)
00087 {
00088     RCLCPP_INFO(get_logger(), "Activating StarSpiDriverNode...");
00089
00090     odom_pub_>on_activate();
00091     joint_state_pub_>on_activate();
00092     imu_pub_>on_activate();
00093
00094     // Start 100 Hz timer (10ms)
00095     timer_ = create_wall_timer(
00096         10ms, std::bind(&StarSpiDriverNode::timer_callback, this));
00097
00098     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00099         CallbackReturn::SUCCESS;
00100 }
00101
00102 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00103 StarSpiDriverNode::on_deactivate(const rclcpp_lifecycle::State &)
00104 {
00105     RCLCPP_INFO(get_logger(), "Deactivating StarSpiDriverNode...");
00106
00107     // Stop timer
00108     timer_.reset();
00109
00110     // Send zero velocity for safety
00111     // Construct zero command
00112     star::v1::VelocityCommand cmd;
00113     cmd.set_front_left_velocity_mps(0.0f);
00114     cmd.set_front_right_velocity_mps(0.0f);
00115     cmd.set_back_left_velocity_mps(0.0f);
00116     cmd.set_back_right_velocity_mps(0.0f);
00117
00118     // TODO(safety): Send final zero-velocity frame before deactivation

```

9.34 star_spi_driver_node.cpp

133

```

00119
00120     odom_pub->on_deactivate();
00121     joint_state_pub->on_deactivate();
00122     imu_pub->on_deactivate();
00123
00124     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00125         CallbackReturn::SUCCESS;
00126 }
00127
00128 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00129 StarSpiDriverNode::on_cleanup(const rclcpp_lifecycle::State &)
00130 {
00131     RCLCPP_INFO(get_logger(), "Cleaning up StarSpiDriverNode...");
00132
00133     spi_driver->close_device();
00134     spi_driver_.reset();
00135     converter_.reset();
00136
00137     odom_pub_.reset();
00138     joint_state_pub_.reset();
00139     imu_pub_.reset();
00140     cmd_vel_sub_.reset();
00141
00142     return rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::
00143         CallbackReturn::SUCCESS;
00144 }
00145
00146 rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn
00147 StarSpiDriverNode::on_shutdown(const rclcpp_lifecycle::State & prev_state)
00148 {
00149     RCLCPP_INFO(get_logger(), "Shutting down StarSpiDriverNode...");
00150     return on_cleanup(prev_state);
00151 }
00152
00153 void StarSpiDriverNode::cmd_vel_callback(
00154     const geometry_msgs::msg::Twist::SharedPtr msg)
00155 {
00156     current_cmd_vel_ = *msg;
00157     last_cmd_vel_time_ = get_clock()->now();
00158 }
00159
00160 void StarSpiDriverNode::timer_callback()
00161 {
00162     // Check for command velocity timeout (watchdog safety)
00163     auto now = get_clock()->now();
00164     double time_since_cmd = (now - last_cmd_vel_time_).seconds();
00165
00166     geometry_msgs::msg::Twist cmd_vel_to_send;
00167     if (time_since_cmd > (cmd_vel_timeout_ms_ / 1000.0)) {
00168         // Timeout expired: send zero velocity for safety
00169     } else {
00170         cmd_vel_to_send = current_cmd_vel_;
00171     }
00172
00173     // Convert Twist to protobuf VelocityCommand
00174     star::v1::VelocityCommand velocity_cmd;
00175
00176     if (!converter->twist_to_velocity_command(cmd_vel_to_send, velocity_cmd)) {
00177         RCLCPP_WARN(get_logger(),
00178             "Failed to convert Twist to VelocityCommand (NaN/Inf?)");
00179         // Send zero if conversion fails
00180         velocity_cmd.set_front_left_velocity_mps(0.0f);
00181         velocity_cmd.set_front_right_velocity_mps(0.0f);
00182         velocity_cmd.set_back_left_velocity_mps(0.0f);
00183         velocity_cmd.set_back_right_velocity_mps(0.0f);
00184     }
00185
00186     // Encode protobuf payload into SPI frame
00187     std::vector<uint8_t> payload(velocity_cmd.ByteSizeLong());
00188     velocity_cmd.SerializeToArray(payload.data(), payload.size());
00189
00190     FrameType frame_type = FrameType::VelocityCommand;
00191     uint8_t flags = 0x00;
00192
00193     std::vector<uint8_t> tx_frame;
00194     SpiDriver::encode_frame(tx_seq_++, frame_type, flags, payload, tx_frame);
00195
00196     // Perform full-duplex SPI transfer
00197     std::vector<uint8_t> rx_frame;
00198     if (!spi_driver->transfer(tx_frame, rx_frame)) {
00199         RCLCPP_ERROR_THROTTLE(get_logger(), *get_clock(), kLogThrottleMs,
00200             "SPI Transfer Failed");
00201         return;
00202     }
00203
00204     // Decode received frame
00205

```

```

00206 uint16_t rx_seq_;
00207
00208 FrameType rx_type_;
00209
00210 uint8_t rx_flags_;
00211
00212 std::vector<uint8_t> rx_payload_;
00213
00214 if (!SpiDriver::decode_frame(rx_frame, rx_seq_, rx_type_, rx_flags_,
00215                             rx_payload_))
00216 {
00217     RCLCPP_WARN_THROTTLE(
00218         get_logger(), *get_clock(), kLogThrottleMs,
00219         "SPI Frame Decode Failed (CRC mismatch or garbage)");
00220
00221     return;
00222 }
00223
00224 // Parse telemetry and publish ROS2 messages
00225 if (rx_type_ == FrameType::TelemetryData) {
00226     star::v1::TelemetryData telemetry;
00227     if (telemetry.ParseFromArray(rx_payload_.data(), rx_payload_.size())) {
00228         // Check emergency stop flag from motor controller
00229         if (telemetry.emergency_stop()) {
00230             RCLCPP_ERROR_THROTTLE(get_logger(), *get_clock(), kLogThrottleMs,
00231                                   "RX72N EMERGENCY STOP ACTIVE");
00232         }
00233
00234         // Publish Odometry
00235         nav_msgs::msg::Odometry odom;
00236         odom.header.stamp = now;
00237         converter_>telemetry_to_odometry(telemetry, odom);
00238         odom_pub->publish(odom);
00239
00240         // Publish Joint States
00241         sensor_msgs::msg::JointState joint_state;
00242         joint_state.header.stamp = now;
00243         converter_>telemetry_to_joint_state(telemetry, joint_state);
00244         joint_state_pub->publish(joint_state);
00245
00246         // Publish IMU data
00247         const auto & imu_data = telemetry.imu();
00248         // Skip if firmware has not populated IMU fields (proto3 default = all zeros).
00249         // A functioning IMU at rest reads ~9.81 m/s^2 on Z; zero indicates no data.
00250         const bool imu_populated = (imu_data.accel_z_mps2() != 0.0 ||
00251                                     imu_data.gyro_z_rad_per_s() != 0.0);
00252         if (!imu_populated) {
00253             return; // wait for real IMU data
00254         }
00255
00256         sensor_msgs::msg::Imu imu_msg;
00257         imu_msg.header.stamp = now;
00258         imu_msg.header.frame_id = "imu_link";
00259
00260         tf2::Quaternion q;
00261         q.setRPY(imu_data.roll_rad(), imu_data.pitch_rad(), imu_data.yaw_rad());
00262         imu_msg.orientation.x = q.x();
00263         imu_msg.orientation.y = q.y();
00264         imu_msg.orientation.z = q.z();
00265         imu_msg.orientation.w = q.w();
00266         // Orientation covariance (3x3 row-major): sigma ~5.7 deg (rad^2)
00267         imu_msg.orientation_covariance = {
00268             kImuOrientationVar, 0.0, 0.0,
00269             0.0, kImuOrientationVar, 0.0,
00270             0.0, 0.0, kImuOrientationVar};
00271
00272         imu_msg.angular_velocity.x = imu_data.gyro_x_rad_per_s();
00273         imu_msg.angular_velocity.y = imu_data.gyro_y_rad_per_s();
00274         imu_msg.angular_velocity.z = imu_data.gyro_z_rad_per_s();
00275         // Angular velocity covariance (3x3 row-major): sigma ~0.032 rad/s
00276         imu_msg.angular_velocity_covariance = {
00277             kImuAngularVelocityVar, 0.0, 0.0,
00278             0.0, kImuAngularVelocityVar, 0.0,
00279             0.0, 0.0, kImuAngularVelocityVar};
00280
00281         imu_msg.linear_acceleration.x = imu_data.accel_x_mps2();
00282         imu_msg.linear_acceleration.y = imu_data.accel_y_mps2();
00283         imu_msg.linear_acceleration.z = imu_data.accel_z_mps2();
00284         // Linear acceleration covariance (3x3 row-major): sigma ~0.1 m/s^2
00285         imu_msg.linear_acceleration_covariance = {
00286             kImuLinearAccelVar, 0.0, 0.0,
00287             0.0, kImuLinearAccelVar, 0.0,
00288             0.0, 0.0, kImuLinearAccelVar};
00289
00290         imu_pub->publish(imu_msg);
00291
00292     } else {

```

9.35 src/star_spi_bridge/test/test_spi_driver.cpp File Reference

135

```

00293     RCLCPP_WARN(get_logger(), "Failed to parse TelemetryData protobuf");
00294   }
00295 }
00296 }
00297
00298 } // namespace star_spi_bridge

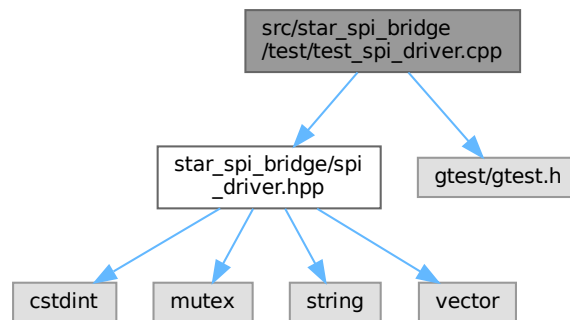
```

9.35 src/star_spi_bridge/test/test_spi_driver.cpp File Reference

```

#include "star_spi_bridge/spi_driver.hpp"
#include <gtest/gtest.h>
Include dependency graph for test_spi_driver.cpp:

```



Classes

- class [SpiDriverTest](#)
- class [SpiDriver](#)

SPI driver for framed communication with the RX72N peripheral MCU.

Enumerations

- enum class [FrameType](#)

Frame type identifiers for SPI protocol messages.

Functions

- [TEST_F](#) ([SpiDriverTest](#), [CRC32Calculation](#))
- [TEST_F](#) ([SpiDriverTest](#), [FrameEncoding](#))
- [TEST_F](#) ([SpiDriverTest](#), [FrameDecoding](#))
- [TEST_F](#) ([SpiDriverTest](#), [EncodeDecodeRoundTrip](#))
- [TEST_F](#) ([SpiDriverTest](#), [DecodeRejectsCRC Corruption](#))
- [TEST_F](#) ([SpiDriverTest](#), [EncodeRejectsOversizedPayload](#))

Generated by Doxygen

9.35.1 Enumeration Type Documentation

9.35.1.1 FrameType

```
enum class star_spi_bridge::FrameType : uint8_t [strong]
```

Frame type identifiers for SPI protocol messages.

Each frame carries a one-byte type field at offset 6 in the header. Values match the RX72N firmware `rx_frame_type_t` enum so both sides agree on the semantics of every message.

See also

[SpiDriver::encode_frame](#) Frame construction

[SpiDriver::decode_frame](#) Frame parsing

Since

Version 1.0.0

Definition at line 27 of file [spi_driver.hpp](#).

9.35.2 Function Documentation

9.35.2.1 TEST_F() [1/6]

```
TEST_F (
    SpiDriverTest ,
    CRC32Calculation )
```

Definition at line 19 of file [test_spi_driver.cpp](#).

References [SpiDriver::calculate_crc32\(\)](#).

Here is the call graph for this function:



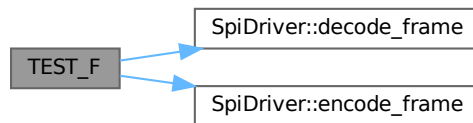
9.35 src/star_spi_bridge/test/test_spi_driver.cpp File Reference**137****9.35.2.2 TEST_F() [2/6]**

```
TEST_F (
    SpiDriverTest ,
    DecodeRejectsCRCCorruption )
```

Definition at line 89 of file [test_spi_driver.cpp](#).

References [SpiDriver::decode_frame\(\)](#), and [SpiDriver::encode_frame\(\)](#).

Here is the call graph for this function:

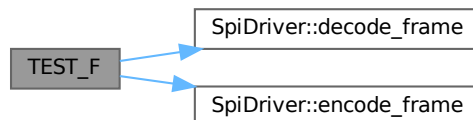
**9.35.2.3 TEST_F() [3/6]**

```
TEST_F (
    SpiDriverTest ,
    EncodeDecodeRoundTrip )
```

Definition at line 71 of file [test_spi_driver.cpp](#).

References [SpiDriver::decode_frame\(\)](#), and [SpiDriver::encode_frame\(\)](#).

Here is the call graph for this function:



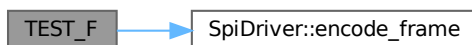
9.35.2.4 TEST_F() [4/6]

```
TEST_F (
    SpiDriverTest ,
    EncodeRejectsOversizedPayload )
```

Definition at line 106 of file [test_spi_driver.cpp](#).

References [SpiDriver::encode_frame\(\)](#), and [SpiDriver::k_max_payload_size](#).

Here is the call graph for this function:

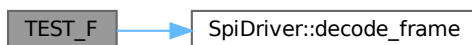
**9.35.2.5 TEST_F()** [5/6]

```
TEST_F (
    SpiDriverTest ,
    FrameDecoding )
```

Definition at line 54 of file [test_spi_driver.cpp](#).

References [SpiDriver::decode_frame\(\)](#).

Here is the call graph for this function:



9.36 test_spi_driver.cpp

139

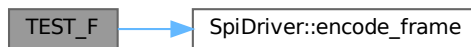
9.35.2.6 TEST_F() [6/6]

```
TEST_F (
    SpiDriverTest ,
    FrameEncoding )
```

Definition at line 28 of file test_spi_driver.cpp.

References [SpiDriver::encode_frame\(\)](#).

Here is the call graph for this function:



9.36 test_spi_driver.cpp

[Go to the documentation of this file.](#)

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include "star_spi_bridge/spi_driver.hpp"
00004
00005 #include <gtest/gtest.h> // NOLINT
00006
00007 using star_spi_bridge::FrameType;
00008 using star_spi_bridge::SpiDriver;
00009
00010 class SpiDriverTest : public ::testing::Test
00011 {
00012 protected:
00013     void SetUp() override
00014     {
00015         // Setup code
00016     }
00017 };
00018
00019 TEST_F(SpiDriverTest, CRC32Calculation)
00020 {
00021     // Test vector: "123456789" -> 0xCBF43926
00022     std::string test_str = "123456789";
00023     std::vector<uint8_t> data(test_str.begin(), test_str.end());
00024
00025     EXPECT_EQ(SpiDriver::calculate_crc32(data), 0xCBF43926);
00026 }
00027
00028 TEST_F(SpiDriverTest, FrameEncoding)
00029 {
00030     std::vector<uint8_t> payload = {0x01, 0x02, 0x03};
00031     std::vector<uint8_t> frame;
00032     SpiDriver::encode_frame(100, FrameType::VelocityCommand, 0x00, payload, frame);
00033
00034     // Header (8) + Payload (3) + CRC (4) = 15 bytes
00035     EXPECT_EQ(frame.size(), 15);
00036
00037     // Check SYNC word (Little Endian: [0xAA, 0x55] for 0x55AA)
00038     EXPECT_EQ(frame[0], 0xAA); // LSB first
00039     EXPECT_EQ(frame[1], 0x55); // MSB second
00040
00041     // Check SEQ field (Little Endian: seq=100=0x0064 -> [0x64, 0x00])
00042     EXPECT_EQ(frame[2], 0x64); // SEQ LSB
00043     EXPECT_EQ(frame[3], 0x00); // SEQ MSB
00044
00045     // Check LEN field (Little Endian: len=3=0x0003 -> [0x03, 0x00])
```

Generated by Doxygen

```

00046 EXPECT_EQ(frame[4], 0x03); // LEN LSB
00047 EXPECT_EQ(frame[5], 0x00); // LEN MSB
00048
00049 // Check TYPE and FLAGS
00050 EXPECT_EQ(frame[6], 0x01); // TYPE: VelocityCommand
00051 EXPECT_EQ(frame[7], 0x00); // FLAGS: none
00052 }
00053
00054 TEST_F(SpiDriverTest, FrameDecoding)
00055 {
00056     // Construct a valid frame manually
00057     // SYNC(55AA) SEQ(0001) LEN(0001) TYPE(01) FLAGS(00) PAYLOAD(A5) CRC(...)
00058     // CRC for header+payload needs to be correct for decode to succeed
00059
00060     // We can't easily test decode without a working encode or pre-calculated frame
00061     // For Red Phase, we just assert that decode fails on garbage
00062     std::vector<uint8_t> garbage = {0x00, 0x00, 0x00};
00063     uint16_t seq = 0;
00064     FrameType type = FrameType::TelemetryData;
00065     uint8_t flags = 0x00;
00066     std::vector<uint8_t> payload = {};
00067
00068     EXPECT_FALSE(SpiDriver::decode_frame(garbage, seq, type, flags, payload));
00069 }
00070
00071 TEST_F(SpiDriverTest, EncodeDecodeRoundTrip)
00072 {
00073     std::vector<uint8_t> payload_in = {0xDE, 0xAD, 0xBE, 0xEF};
00074     std::vector<uint8_t> frame;
00075     SpiDriver::encode_frame(42, FrameType::TelemetryData, 0x03, payload_in, frame);
00076
00077     uint16_t seq = 0;
00078     FrameType type = FrameType::VelocityCommand;
00079     uint8_t flags = 0;
00080     std::vector<uint8_t> payload_out;
00081
00082     ASSERT_TRUE(SpiDriver::decode_frame(frame, seq, type, flags, payload_out));
00083     EXPECT_EQ(seq, 42);
00084     EXPECT_EQ(type, FrameType::TelemetryData);
00085     EXPECT_EQ(flags, 0x03);
00086     EXPECT_EQ(payload_out, payload_in);
00087 }
00088
00089 TEST_F(SpiDriverTest, DecodeRejectsCRCCorruption)
00090 {
00091     std::vector<uint8_t> payload = {0x01};
00092     std::vector<uint8_t> frame;
00093     SpiDriver::encode_frame(1, FrameType::VelocityCommand, 0x00, payload, frame);
00094
00095     // Corrupt the last byte (CRC)
00096     frame.back() ^= 0xFF;
00097
00098     uint16_t seq = 0;
00099     FrameType type = FrameType::TelemetryData;
00100     uint8_t flags = 0;
00101     std::vector<uint8_t> decoded_payload;
00102
00103     EXPECT_FALSE(SpiDriver::decode_frame(frame, seq, type, flags, decoded_payload));
00104 }
00105
00106 TEST_F(SpiDriverTest, EncodeRejectsOversizedPayload)
00107 {
00108     std::vector<uint8_t> oversized(SpiDriver::k_max_payload_size + 1, 0xAA);
00109     std::vector<uint8_t> frame;
00110
00111     EXPECT_THROW(
00112         SpiDriver::encode_frame(0, FrameType::VelocityCommand, 0x00, oversized, frame),
00113         std::invalid_argument);
00114 }

```

9.37 src/star_spi_bridge/test/test_spi_message_converter.cpp File Reference

```

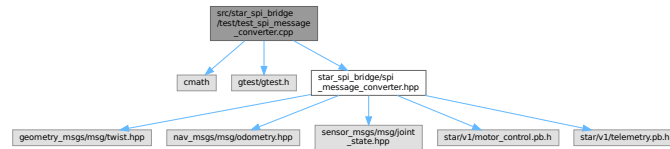
#include <cmath>
#include <gtest/gtest.h>
#include "star_spi_bridge/spi_message_converter.hpp"

```

9.37 src/star_spi_bridge/test/test_spi_message_converter.cpp File Reference

141

Include dependency graph for test_spi_message_converter.cpp:



Classes

- class [SpiMessageConverterTest](#)
- class [SpiMessageConverter](#)

Functions

- [TEST_F \(SpiMessageConverterTest, TwistToVelocity_Forward\)](#)
- [TEST_F \(SpiMessageConverterTest, TwistToVelocity_RotateLeft\)](#)
- [TEST_F \(SpiMessageConverterTest, TwistToVelocity_NaN\)](#)
- [TEST_F \(SpiMessageConverterTest, TelemetryToOdometry_FirstMessage\)](#)
- [TEST_F \(SpiMessageConverterTest, TelemetryToOdometry_ForwardMovement\)](#)
- [TEST_F \(SpiMessageConverterTest, TelemetryToOdometry_WithVelocity\)](#)
- [TEST_F \(SpiMessageConverterTest, TelemetryToJointState_Names\)](#)
- [TEST_F \(SpiMessageConverterTest, TelemetryToJointState_Position\)](#)
- [TEST_F \(SpiMessageConverterTest, TelemetryToJointState_Velocity\)](#)
- [TEST_F \(SpiMessageConverterTest, RoundTrip_TwistToVelocityToJointState\)](#)

9.37.1 Function Documentation

9.37.1.1 TEST_F() [1/10]

```

TEST_F (
    SpiMessageConverterTest ,
    RoundTrip_TwistToVelocityToJointState )

```

Definition at line 203 of file [test_spi_message_converter.cpp](#).

9.37.1.2 TEST_F() [2/10]

```

TEST_F (
    SpiMessageConverterTest ,
    TelemetryToJointState_Names )

```

Definition at line 140 of file [test_spi_message_converter.cpp](#).

9.37.1.3 TEST_F() [3/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TelemetryToJointState_Position )
```

Definition at line 158 of file [test_spi_message_converter.cpp](#).

9.37.1.4 TEST_F() [4/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TelemetryToJointState_Velocity )
```

Definition at line 178 of file [test_spi_message_converter.cpp](#).

9.37.1.5 TEST_F() [5/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TelemetryToOdometry_FirstMessage )
```

Definition at line 59 of file [test_spi_message_converter.cpp](#).

9.37.1.6 TEST_F() [6/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TelemetryToOdometry_ForwardMovement )
```

Definition at line 75 of file [test_spi_message_converter.cpp](#).

9.37.1.7 TEST_F() [7/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TelemetryToOdometry_WithVelocity )
```

Definition at line 104 of file [test_spi_message_converter.cpp](#).

References [star_spi_bridge::SpiMessageConverter::telemetry_to_odometry\(\)](#).

Here is the call graph for this function:



9.38 test_spi_message_converter.cpp**143****9.37.1.8 TEST_F() [8/10]**

```
TEST_F (
    SpiMessageConverterTest ,
    TwistToVelocity_Forward )
```

Definition at line 18 of file [test_spi_message_converter.cpp](#).

9.37.1.9 TEST_F() [9/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TwistToVelocity_NaN )
```

Definition at line 46 of file [test_spi_message_converter.cpp](#).

9.37.1.10 TEST_F() [10/10]

```
TEST_F (
    SpiMessageConverterTest ,
    TwistToVelocity_RotateLeft )
```

Definition at line 31 of file [test_spi_message_converter.cpp](#).

9.38 test_spi_message_converter.cpp

[Go to the documentation of this file.](#)

```
00001 // Copyright 2026 Locked Inc.
00002
00003 #include <cmath>
00004
00005 #include <gtest/gtest.h> // NOLINT
00006
00007 #include "star_spi_bridge/spi_message_converter.hpp"
00008
00009 using star_spi_bridge::SpiMessageConverter;
00010
00011 class SpiMessageConverterTest : public ::testing::Test
00012 {
00013 protected:
00014     SpiMessageConverter::Parameters params{0.150, 0.0325, 11599};
00015     SpiMessageConverter converter{params};
00016 };
00017
00018 TEST_F(SpiMessageConverterTest, TwistToVelocity_Forward)
00019 {
00020     geometry_msgs::msg::Twist twist;
00021     twist.linear.x = 1.0;
00022     twist.angular.z = 0.0;
00023
00024     star::v1::VelocityCommand cmd;
00025     EXPECT_TRUE(converter.twist_to_velocity_command(twist, cmd));
00026
00027     EXPECT_NEAR(cmd.front_left_velocity_mps(), 1.0, 0.001);
00028     EXPECT_NEAR(cmd.front_right_velocity_mps(), 1.0, 0.001);
00029 }
00030
00031 TEST_F(SpiMessageConverterTest, TwistToVelocity_RotateLeft)
00032 {
00033     geometry_msgs::msg::Twist twist;
00034     twist.linear.x = 0.0;
00035     twist.angular.z = 2.0; // 2 rad/s
00036
00037     star::v1::VelocityCommand cmd;
```

Generated by Doxygen

```

00038 EXPECT_TRUE(converter.twist_to_velocity_command(twist, cmd));
00039
00040 // v_right = 0 + 2 * (0.15/2) = 0.15
00041 // v_left = 0 - 2 * (0.15/2) = -0.15
00042 EXPECT_NEAR(cmd.front_right_velocity_mps(), 0.15, 0.001);
00043 EXPECT_NEAR(cmd.front_left_velocity_mps(), -0.15, 0.001);
00044 }
00045
00046 TEST_F(SpiMessageConverterTest, TwistToVelocity_NaN)
00047 {
00048     geometry_msgs::msg::Twist twist;
00049     twist.linear.x = std::numeric_limits<double>::quiet_NaN();
00050
00051     star::v1::VelocityCommand cmd;
00052     EXPECT_FALSE(converter.twist_to_velocity_command(twist, cmd));
00053 }
00054
00055 // =====
00056 // Tests for telemetry_to_odometry
00057 // =====
00058
00059 TEST_F(SpiMessageConverterTest, TelemetryToOdometry_FirstMessage)
00060 {
00061     star::v1::TelemetryData telemetry;
00062     telemetry.mutable_encoder_front_left()->set_ticks(100);
00063     telemetry.mutable_encoder_front_right()->set_ticks(100);
00064     telemetry.mutable_encoder_back_left()->set_ticks(100);
00065     telemetry.mutable_encoder_back_right()->set_ticks(100);
00066
00067     nav_msgs::msg::Odometry odom;
00068     converter.telemetry_to_odometry(telemetry, odom);
00069
00070     // First message should initialize state but not update pose
00071     EXPECT_NEAR(odom.pose.pose.position.x, 0.0, 0.001);
00072     EXPECT_NEAR(odom.pose.pose.position.y, 0.0, 0.001);
00073 }
00074
00075 TEST_F(SpiMessageConverterTest, TelemetryToOdometry_ForwardMovement)
00076 {
00077     // First message to initialize state
00078     star::v1::TelemetryData telemetry1;
00079     telemetry1.mutable_encoder_front_left()->set_ticks(0);
00080     telemetry1.mutable_encoder_front_right()->set_ticks(0);
00081     telemetry1.mutable_encoder_back_left()->set_ticks(0);
00082     telemetry1.mutable_encoder_back_right()->set_ticks(0);
00083
00084     nav_msgs::msg::Odometry odom1;
00085     converter.telemetry_to_odometry(telemetry1, odom1);
00086
00087     // Second message with forward movement (same ticks on both sides = straight)
00088     star::v1::TelemetryData telemetry2;
00089     int64_t ticks = 11599; // One full wheel revolution
00090     telemetry2.mutable_encoder_front_left()->set_ticks(ticks);
00091     telemetry2.mutable_encoder_front_right()->set_ticks(ticks);
00092     telemetry2.mutable_encoder_back_left()->set_ticks(ticks);
00093     telemetry2.mutable_encoder_back_right()->set_ticks(ticks);
00094
00095     nav_msgs::msg::Odometry odom2;
00096     converter.telemetry_to_odometry(telemetry2, odom2);
00097
00098     // One revolution = 2 * pi * radius = 2 * pi * 0.0325 ~ 0.204 m
00099     double expected_dist = 2.0 * M_PI * 0.0325;
00100     EXPECT_NEAR(odom2.pose.pose.position.x, expected_dist, 0.01);
00101     EXPECT_NEAR(odom2.pose.pose.position.y, 0.0, 0.001);
00102 }
00103
00104 TEST_F(SpiMessageConverterTest, TelemetryToOdometry_WithVelocity)
00105 {
00106     // Create fresh converter for this test
00107     SpiMessageConverter test_converter(params);
00108
00109     // First message
00110     star::v1::TelemetryData telemetry1;
00111     telemetry1.mutable_encoder_front_left()->set_ticks(0);
00112     telemetry1.mutable_encoder_front_right()->set_ticks(0);
00113     telemetry1.mutable_encoder_back_left()->set_ticks(0);
00114     telemetry1.mutable_encoder_back_right()->set_ticks(0);
00115     nav_msgs::msg::Odometry odom1;
00116     test_converter.telemetry_to_odometry(telemetry1, odom1);
00117
00118     // Second message with velocity data
00119     star::v1::TelemetryData telemetry2;
00120     telemetry2.mutable_encoder_front_left()->set_ticks(100);
00121     telemetry2.mutable_encoder_front_left()->set_velocity_mps(1.0);
00122     telemetry2.mutable_encoder_front_right()->set_ticks(100);
00123     telemetry2.mutable_encoder_front_right()->set_velocity_mps(1.0);
00124     telemetry2.mutable_encoder_back_left()->set_ticks(100);

```

9.38 test_spi_message_converter.cpp

145

```

00125 telemetry2.mutable_encoder_back_left()->set_velocity_mps(1.0);
00126 telemetry2.mutable_encoder_back_right()->set_ticks(100);
00127 telemetry2.mutable_encoder_back_right()->set_velocity_mps(1.0);
00128
00129 nav_msgs::msg::Odometry odom2;
00130 test_converter.telemetry_to_odometry(telemetry2, odom2);
00131
00132 EXPECT_NEAR(odom2.twist.twist.linear.x, 1.0, 0.001);
00133 EXPECT_NEAR(odom2.twist.twist.angular.z, 0.0, 0.001);
00134 }
00135
00136 // =====
00137 // Tests for telemetry_to_joint_state
00138 // =====
00139
00140 TEST_F(SpiMessageConverterTest, TelemetryToJointState_Names)
00141 {
00142     star::v1::TelemetryData telemetry;
00143     telemetry.mutable_encoder_front_left()->set_ticks(0);
00144     telemetry.mutable_encoder_front_right()->set_ticks(0);
00145     telemetry.mutable_encoder_back_left()->set_ticks(0);
00146     telemetry.mutable_encoder_back_right()->set_ticks(0);
00147
00148     sensor_msgs::msg::JointState joint_state;
00149     converter.telemetry_to_joint_state(telemetry, joint_state);
00150
00151     ASSERT_EQ(joint_state.name.size(), 4u);
00152     EXPECT_EQ(joint_state.name[0], "front_left_wheel");
00153     EXPECT_EQ(joint_state.name[1], "front_right_wheel");
00154     EXPECT_EQ(joint_state.name[2], "back_left_wheel");
00155     EXPECT_EQ(joint_state.name[3], "back_right_wheel");
00156 }
00157
00158 TEST_F(SpiMessageConverterTest, TelemetryToJointState_Position)
00159 {
00160     star::v1::TelemetryData telemetry;
00161     // One full revolution = 11599 ticks = 2*PI radians
00162     int64_t one_rev = 11599;
00163     telemetry.mutable_encoder_front_left()->set_ticks(one_rev);
00164     telemetry.mutable_encoder_front_right()->set_ticks(one_rev / 2);
00165     telemetry.mutable_encoder_back_left()->set_ticks(one_rev * 2);
00166     telemetry.mutable_encoder_back_right()->set_ticks(0);
00167
00168     sensor_msgs::msg::JointState joint_state;
00169     converter.telemetry_to_joint_state(telemetry, joint_state);
00170
00171     ASSERT_EQ(joint_state.position.size(), 4u);
00172     EXPECT_NEAR(joint_state.position[0], 2.0 * M_PI, 0.01); // FL: 1 rev
00173     EXPECT_NEAR(joint_state.position[1], M_PI, 0.01); // FR: 0.5 rev
00174     EXPECT_NEAR(joint_state.position[2], 4.0 * M_PI, 0.02); // BL: 2 rev
00175     EXPECT_NEAR(joint_state.position[3], 0.0, 0.001); // BR: 0 rev
00176 }
00177
00178 TEST_F(SpiMessageConverterTest, TelemetryToJointState_Velocity)
00179 {
00180     star::v1::TelemetryData telemetry;
00181     double velocity_mps = 1.0; // 1 m/s linear
00182     telemetry.mutable_encoder_front_left()->set_velocity_mps(velocity_mps);
00183     telemetry.mutable_encoder_front_right()->set_velocity_mps(velocity_mps);
00184     telemetry.mutable_encoder_back_left()->set_velocity_mps(velocity_mps);
00185     telemetry.mutable_encoder_back_right()->set_velocity_mps(velocity_mps);
00186
00187     sensor_msgs::msg::JointState joint_state;
00188     converter.telemetry_to_joint_state(telemetry, joint_state);
00189
00190     // angular_velocity = linear_velocity / radius = 1.0 / 0.0325 ~ 30.77 rad/s
00191     double expected angular = velocity_mps / 0.0325;
00192     ASSERT_EQ(joint_state.velocity.size(), 4u);
00193     EXPECT_NEAR(joint_state.velocity[0], expected angular, 0.1);
00194     EXPECT_NEAR(joint_state.velocity[1], expected angular, 0.1);
00195     EXPECT_NEAR(joint_state.velocity[2], expected angular, 0.1);
00196     EXPECT_NEAR(joint_state.velocity[3], expected angular, 0.1);
00197 }
00198
00199 // =====
00200 // Round-trip encode/decode test
00201 // =====
00202
00203 TEST_F(SpiMessageConverterTest, RoundTrip_TwistToVelocityToJointState)
00204 {
00205     // Create a twist command
00206     geometry_msgs::msg::Twist twist;
00207     twist.linear.x = 0.5;
00208     twist.angular.z = 0.2;
00209
00210     // Convert to velocity command
00211     star::v1::VelocityCommand cmd;

```

```
00212 ASSERT_TRUE(converter.twist_to_velocity_command(twist, cmd));
00213
00214 // Verify the velocity command was set correctly
00215 // v_right = 0.5 + 0.2 * (0.15/2) = 0.5 + 0.015 = 0.515
00216 // v_left = 0.5 - 0.2 * (0.15/2) = 0.5 - 0.015 = 0.485
00217 EXPECT_NEAR(cmd.front_right_velocity_mps(), 0.515, 0.001);
00218 EXPECT_NEAR(cmd.front_left_velocity_mps(), 0.485, 0.001);
00219 EXPECT_NEAR(cmd.back_right_velocity_mps(), 0.515, 0.001);
00220 EXPECT_NEAR(cmd.back_left_velocity_mps(), 0.485, 0.001);
00221
00222 // Create telemetry with matching velocities (simulating firmware response)
00223 star::v1::TelemetryData telemetry;
00224 telemetry.mutable_encoder_front_left()->set_velocity_mps(cmd.front_left_velocity_mps());
00225 telemetry.mutable_encoder_front_right()->set_velocity_mps(cmd.front_right_velocity_mps());
00226 telemetry.mutable_encoder_back_left()->set_velocity_mps(cmd.back_left_velocity_mps());
00227 telemetry.mutable_encoder_back_right()->set_velocity_mps(cmd.back_right_velocity_mps());
00228
00229 // Convert telemetry to joint state
00230 sensor_msgs::msg::JointState joint_state;
00231 converter.telemetry_to_joint_state(telemetry, joint_state);
00232
00233 // Verify velocities converted to rad/s
00234 double inv_radius = 1.0 / 0.0325;
00235 EXPECT_NEAR(joint_state.velocity[0], cmd.front_left_velocity_mps() * inv_radius, 0.1);
00236 EXPECT_NEAR(joint_state.velocity[1], cmd.front_right_velocity_mps() * inv_radius, 0.1);
00237 }
```


Index

- ~SafetyMonitor
 - star_safety_monitor::SafetyMonitor, [44](#)
- ~SpiDriver
 - SpiDriver, [50](#)
 - star_spi_bridge::SpiDriver, [58](#)
- ~StarGatewayBridgeNode
 - star::StarGatewayBridgeNode, [72](#)
- ~StarSpiDriverNode
 - star_spi_bridge::StarSpiDriverNode, [74](#)
- bits_per_word_
 - spi_ioc_transfer, [47](#)
- calculate_crc32
 - SpiDriver, [50](#)
 - star_spi_bridge::SpiDriver, [59](#)
- clamp
 - star::MessageConverter, [34](#)
- close_device
 - SpiDriver, [51](#)
 - star_spi_bridge::SpiDriver, [60](#)
- converter
 - SpiMessageConverterTest, [70](#)
- converter_
 - star::MessageConverterTest, [40](#)
- cs_change_
 - spi_ioc_transfer, [47](#)
- decode_frame
 - SpiDriver, [51](#)
 - star_spi_bridge::SpiDriver, [60](#)
- delay_usecs_
 - spi_ioc_transfer, [47](#)
- encode_frame
 - SpiDriver, [52](#)
 - star_spi_bridge::SpiDriver, [61](#)
- ERROR
 - star_safety_monitor, [30](#)
- FrameType
 - star_spi_bridge, [30](#)
 - test_spi_driver.cpp, [136](#)
- initialize
 - SpiDriver, [54](#)
 - star_spi_bridge::SpiDriver, [62](#)
- is_valid_double
 - star::MessageConverter, [34](#)
- k_crc_size
 - SpiDriver, [56](#)
 - star_spi_bridge::SpiDriver, [64](#)
- k_header_size
 - SpiDriver, [56](#)
 - star_spi_bridge::SpiDriver, [64](#)
- k_max_payload_size
 - SpiDriver, [56](#)
 - star_spi_bridge::SpiDriver, [64](#)
- k_sync_word
 - SpiDriver, [56](#)
 - star_spi_bridge::SpiDriver, [65](#)
- kImuAngularVelocityVar
 - star_spi_bridge, [31](#)
- kImuLinearAccelVar
 - star_spi_bridge, [31](#)
- kImuOrientationVar
 - star_spi_bridge, [31](#)
- kLogThrottleMs
 - star_spi_bridge, [31](#)
- len_
 - spi_ioc_transfer, [47](#)
- main
 - main.cpp, [82, 83](#)
 - safety_monitor_node.cpp, [110](#)
 - test_message_converter.cpp, [95](#)
 - test_safety_monitor.cpp, [112](#)
- main.cpp
 - main, [82, 83](#)
- OK
 - star_safety_monitor, [29](#)
- on_activate
 - star_safety_monitor::SafetyMonitor, [44](#)
 - star_spi_bridge::StarSpiDriverNode, [74](#)
- on_cleanup
 - star_safety_monitor::SafetyMonitor, [44](#)
 - star_spi_bridge::StarSpiDriverNode, [74](#)
- on_configure
 - star_safety_monitor::SafetyMonitor, [44](#)
 - star_spi_bridge::StarSpiDriverNode, [74](#)
- on_deactivate
 - star_safety_monitor::SafetyMonitor, [44](#)
 - star_spi_bridge::StarSpiDriverNode, [75](#)
- on_shutdown
 - star_safety_monitor::SafetyMonitor, [45](#)
 - star_spi_bridge::StarSpiDriverNode, [75](#)
- pad_

- spi_ioc_transfer, 47
- params
 - SpiMessageConverterTest, 70
- pid_config_to_gains
 - star::MessageConverter, 35
- ros_time_to_us
 - star::MessageConverter, 36
- rx_buf_
 - spi_ioc_transfer, 47
- safety_monitor_node.cpp
 - main, 110
- SafetyMonitor
 - star_safety_monitor::SafetyMonitor, 44
- SafetyMonitorTest, 45
 - SetUp, 46
 - TearDown, 46
- SetUp
 - SafetyMonitorTest, 46
 - SpiDriverTest, 66
- SeverityLevel
 - star_safety_monitor, 29
- speed_hz_
 - spi_ioc_transfer, 47
- spi_driver.cpp
 - SPI_IOC_MESSAGE, 123
 - SPI_IOC_WR_BITS_PER_WORD, 123
 - SPI_IOC_WR_MAX_SPEED_HZ, 123
 - SPI_IOC_WR_MODE, 123
 - SPI_MODE_0, 124
- SPI_IOC_MESSAGE
 - spi_driver.cpp, 123
- spi_ioc_transfer, 46
 - bits_per_word_, 47
 - cs_change_, 47
 - delay_usecs_, 47
 - len_, 47
 - pad_, 47
 - rx_buf_, 47
 - speed_hz_, 47
 - tx_buf_, 47
- SPI_IOC_WR_BITS_PER_WORD
 - spi_driver.cpp, 123
- SPI_IOC_WR_MAX_SPEED_HZ
 - spi_driver.cpp, 123
- SPI_IOC_WR_MODE
 - spi_driver.cpp, 123
- SPI_MODE_0
 - spi_driver.cpp, 124
- SpiDriver, 48
 - ~SpiDriver, 50
 - calculate_crc32, 50
 - close_device, 51
 - decode_frame, 51
 - encode_frame, 52
 - initialize, 54
 - k_crc_size, 56
 - k_header_size, 56
 - k_max_payload_size, 56
 - k_sync_word, 56
 - SpiDriver, 49
 - star_spi_bridge::SpiDriver, 58
 - transfer, 55
- SpiDriverTest, 65
 - SetUp, 66
- SpiMessageConverter, 66
 - SpiMessageConverter, 67
 - star_spi_bridge::SpiMessageConverter, 68
 - telemetry_to_joint_state, 67
 - telemetry_to_odometry, 67
 - twist_to_velocity_command, 67
- SpiMessageConverter::Parameters, 41
 - ticks_per_rev, 41
 - wheel_base, 41
 - wheel_radius, 41
- SpiMessageConverterTest, 69
 - converter, 70
 - params, 70
- src Directory Reference, 15
- src/star_gateway_bridge Directory Reference, 17
- src/star_gateway_bridge/include Directory Reference, 13
- src/star_gateway_bridge/include/star_gateway_bridge
 - Directory Reference, 17
- src/star_gateway_bridge/include/star_gateway_bridge/message_converter
 - 77, 78
- src/star_gateway_bridge/include/star_gateway_bridge/star_gateway_bridge
 - 79, 80
- src/star_gateway_bridge/src Directory Reference, 15
- src/star_gateway_bridge/src/main.cpp, 81, 82
- src/star_gateway_bridge/src/message_converter.cpp, 84
- src/star_gateway_bridge/src/star_gateway_bridge_node.cpp, 86, 87
- src/star_gateway_bridge/test Directory Reference, 20
- src/star_gateway_bridge/test/test_message_converter.cpp, 94, 95
- src/star_safety_monitor Directory Reference, 18
- src/star_safety_monitor/include Directory Reference, 14
- src/star_safety_monitor/include/star_safety_monitor Directory Reference, 18
- src/star_safety_monitor/include/star_safety_monitor/safety_monitor.hpp, 101, 102
- src/star_safety_monitor/src Directory Reference, 16
- src/star_safety_monitor/src/safety_monitor.cpp, 104
- src/star_safety_monitor/src/safety_monitor_node.cpp, 110
- src/star_safety_monitor/test Directory Reference, 20
- src/star_safety_monitor/test/test_safety_monitor.cpp, 111, 114
- src/star_spi_bridge Directory Reference, 19
- src/star_spi_bridge/include Directory Reference, 14
- src/star_spi_bridge/include/star_spi_bridge Directory Reference, 19
- src/star_spi_bridge/include/star_spi_bridge/spi_driver.hpp, 116, 117

INDEX

149

```

src/star_spi_bridge/include/star_spi_bridge/spi_message_converter.h, 118, 119
src/star_spi_bridge/include/star_spi_bridge/star_spi_driver_node.h, 120, 121
src/star_spi_bridge/src Directory Reference, 16
src/star_spi_bridge/src/main.cpp, 83
src/star_spi_bridge/src/spi_driver.cpp, 122, 124
src/star_spi_bridge/src/spi_message_converter.cpp, 128
src/star_spi_bridge/src/star_spi_driver_node.cpp, 131
src/star_spi_bridge/test Directory Reference, 21
src/star_spi_bridge/test/test_spi_driver.cpp, 135, 139
src/star_spi_bridge/test/test_spi_message_converter.cpp, 140, 143
STALE
    star_safety_monitor, 30
star, 23
    TEST_F, 24–29
star::MessageConverter, 33
    clamp, 34
    is_valid_double, 34
    pid_config_to_gains, 35
    ros_time_to_us, 36
    string_to_system_status, 36
    twist_to_velocity_command, 37
    velocity_command_to_twist, 38
star::MessageConverterTest, 39
    converter_, 40
    TOLERANCE, 40
    WHEEL_BASE_M, 40
star::StarGatewayBridgeNode, 70
    ~StarGatewayBridgeNode, 72
    StarGatewayBridgeNode, 72
star_safety_monitor, 29
    ERROR, 30
    OK, 29
    SeverityLevel, 29
    STALE, 30
    WARN, 30
star_safety_monitor::SafetyMonitor, 43
    ~SafetyMonitor, 44
    on_activate, 44
    on_cleanup, 44
    on_configure, 44
    on_deactivate, 44
    on_shutdown, 45
    SafetyMonitor, 44
star_spi_bridge, 30
    FrameType, 30
    kImuAngularVelocityVar, 31
    kImuLinearAccelVar, 31
    kImuOrientationVar, 31
    kLogThrottleMs, 31
    TelemetryData, 31
    VelocityCommand, 30
star_spi_bridge::SpiDriver, 56
    ~SpiDriver, 58
    calculate_crc32, 59
    activate, 60
    decode_frame, 60
    encode_frame, 61
    initialize, 62
    k_crc_size, 64
    k_header_size, 64
    k_max_payload_size, 64
    k_sync_word, 65
    SpiDriver, 58
    transfer, 63
star_spi_bridge::SpiMessageConverter, 67
    SpiMessageConverter, 68
    telemetry_to_joint_state, 68
    telemetry_to_odometry, 68
    twist_to_velocity_command, 68
star_spi_bridge::SpiMessageConverter::Parameters, 42
    ticks_per_rev, 42
    wheel_base, 42
    wheel_radius, 42
star_spi_bridge::StarSpiDriverNode, 73
    ~StarSpiDriverNode, 74
    on_activate, 74
    on_cleanup, 74
    on_configure, 74
    on_deactivate, 75
    on_shutdown, 75
    StarSpiDriverNode, 74
StarGatewayBridgeNode
    star::StarGatewayBridgeNode, 72
StarSpiDriverNode
    star_spi_bridge::StarSpiDriverNode, 74
string_to_system_status
    star::MessageConverter, 36
TearDown
    SafetyMonitorTest, 46
telemetry_to_joint_state
    SpiMessageConverter, 67
    star_spi_bridge::SpiMessageConverter, 68
telemetry_to_odometry
    SpiMessageConverter, 67
    star_spi_bridge::SpiMessageConverter, 68
TelemetryData
    star_spi_bridge, 31
TEST_F
    star, 24–29
    test_safety_monitor.cpp, 112, 113
    test_spi_driver.cpp, 136–138
    test_spi_message_converter.cpp, 141–143
test_message_converter.cpp
    main, 95
test_safety_monitor.cpp
    main, 112
    TEST_F, 112, 113
test_spi_driver.cpp
    FrameType, 136
    TEST_F, 136–138
test_spi_message_converter.cpp
    TEST_F, 141–143

```

Generated by Doxygen

ticks_per_rev
 SpiMessageConverter::Parameters, [41](#)
 star_spi_bridge::SpiMessageConverter::Parameters,
 [42](#)
TOLERANCE
 star::MessageConverterTest, [40](#)
transfer
 SpiDriver, [55](#)
 star_spi_bridge::SpiDriver, [63](#)
twist_to_velocity_command
 SpiMessageConverter, [67](#)
 star::MessageConverter, [37](#)
 star_spi_bridge::SpiMessageConverter, [68](#)
tx_buf_
 spi_ioc_transfer, [47](#)

velocity_command_to_twist
 star::MessageConverter, [38](#)
VelocityCommand
 star_spi_bridge, [30](#)

WARN
 star_safety_monitor, [30](#)
wheel_base
 SpiMessageConverter::Parameters, [41](#)
 star_spi_bridge::SpiMessageConverter::Parameters,
 [42](#)
WHEEL_BASE_M
 star::MessageConverterTest, [40](#)
wheel_radius
 SpiMessageConverter::Parameters, [41](#)
 star_spi_bridge::SpiMessageConverter::Parameters,
 [42](#)